

учебник

язык программирования
JavaScript

(1-ая часть)

Екатеринбург
2026

Часть I: Введение и основы программирования

Глава 1: Введение в JavaScript и веб-разработку

- 1.1. Исторический контекст и эволюция JavaScript
- 1.2. Роль JavaScript в современной веб-разработке
- 1.3. JavaScript vs другие языки программирования
- 1.4. Экосистема JavaScript: браузеры, Node.js, среды выполнения
- 1.5. Обзор современных фреймворков и библиотек
- 1.6. Инструменты разработчика: браузерные инструменты, редакторы кода, IDE

Глава 2: Основы программирования

- 2.1. Алгоритмы и структуры данных: базовые концепции
- 2.2. Парадигмы программирования: императивное, декларативное, ООП, функциональное
- 2.3. Синтаксис JavaScript: основные правила и соглашения
- 2.4. Комментарии и документация кода
- 2.5. Отладка и обработка ошибок

Глава 3: Первая программа на JavaScript

- 3.1. Настройка окружения разработки
- 3.2. Подключение JavaScript к HTML-документу
- 3.3. Исполнение кода: браузер vs консоль vs Node.js
- 3.4. "Hello, World!" в различных средах
- 3.5. Интерактивные примеры с console.log и alert

Часть II: Базовые конструкции языка

Глава 4: Переменные и типы данных

- 4.1. Объявление переменных: var, let, const
- 4.2. Области видимости: глобальная, функциональная, блочная
- 4.3. Примитивные типы данных: number, string, boolean, null, undefined, symbol, bigint
- 4.4. Ссылочные типы: объекты, массивы, функции
- 4.5. Динамическая типизация и преобразование типов
- 4.6. Операторы typeof и instanceof

Глава 5: Операторы и выражения

- 5.1. Арифметические операторы
- 5.2. Операторы сравнения и равенства
- 5.3. Логические операторы и короткий цикл вычислений
- 5.4. Побитовые операторы
- 5.5. Условный (тернарный) оператор
- 5.6. Операторы присваивания
- 5.7. Приоритет операторов и ассоциативность

Глава 6: Управляющие конструкции

- 6.1. Условные выражения: if, else if, else
- 6.2. Многоступенчатое ветвление: switch-case
- 6.3. Циклы: for, while, do-while
- 6.4. Итерация по объектам: for-in, for-of
- 6.5. Управление выполнением циклов: break, continue
- 6.6. Метки и вложенные циклы

Глава 7: Функции

- 7.1. Объявление и вызов функций
- 7.2. Параметры и аргументы функции

- 7.3. Возврат значений и return
- 7.4. Function Expression vs Function Declaration
- 7.5. Анонимные функции и немедленно вызываемые функции (IIFE)
- 7.6. Замыкания и лексическое окружение
- 7.7. Рекурсия и стек вызовов
- 7.8. Функции высшего порядка

Глава 8: Объекты и объектно-ориентированное программирование

- 8.1. Создание объектов: литералы, конструкторы, Object.create()
- 8.2. Свойства и методы объектов
- 8.3. "this" в JavaScript: контекст выполнения
- 8.4. Прототипы и цепочка прототипов
- 8.5. Конструкторы и оператор new
- 8.6. Классы в ES6: синтаксический сахар
- 8.7. Наследование и полиморфизм
- 8.8. Инкапсуляция и скрытие данных

Глава 9: Массивы и коллекции

- 9.1. Создание и манипуляция массивами
- 9.2. Методы массивов: итерационные, мутирующие, доступные
- 9.3. Многомерные массивы
- 9.4. Псевдомассивы и преобразование в массивы
- 9.5. Set, Map, WeakSet, WeakMap
- 9.6. Итераторы и генераторы

Глава 10: Строки и регулярные выражения

- 10.1. Строковые литералы и шаблонные строки
- 10.2. Методы работы со строками
- 10.3. Кодировки и Unicode в JavaScript
- 10.4. Введение в регулярные выражения
- 10.5. Синтаксис регулярных выражений
- 10.6. Методы RegExp и String для работы с регулярными выражениями

Часть III: Продвинутые концепции JavaScript

Глава 11: Асинхронное программирование

- 11.1. Модель событийного цикла в JavaScript
- 11.2. Callback-функции и проблемы "callback hell"
- 11.3. Промисы (Promise): создание, обработка, цепочки
- 11.4. Async/await: синтаксический сахар для промисов
- 11.5. Обработка ошибок в асинхронном коде
- 11.6. Параллельное и последовательное выполнение асинхронных операций

Глава 12: Модульность и организация кода

- 12.1. Паттерны модулей: IIFE, модульный паттерн
- 12.2. CommonJS модули (Node.js)
- 12.3. ES6 модули: импорт и экспорт
- 12.4. Динамический импорт и ленивая загрузка
- 12.5. Системы сборки: Webpack, Rollup, Parcel
- 12.6. Пакетные менеджеры: npm, yarn

Глава 13: Работа с датами, ошибками и исключениями

- 13.1. Объект Date и работа с временем
- 13.2. Библиотеки для работы с датами (Moment.js, date-fns)
- 13.3. Обработка ошибок: try-catch-finally
- 13.4. Пользовательские ошибки и их создание
- 13.5. Глобальная обработка ошибок

Глава 14: Метaprogramмирование и рефлексия

- 14.1. Объект Reflect и его методы
- 14.2. Прокси-объекты и перехватчики
- 14.3. Символы (Symbol) и их применение
- 14.4. Дескрипторы свойств: defineProperty, getOwnPropertyDescriptor
- 14.5. Заморозка объектов: seal, freeze, preventExtensions

Глава 15: Память и производительность

- 15.1. Модель памяти в JavaScript
- 15.2. Сборка мусора и алгоритмы управления памятью
- 15.3. Утечки памяти и их диагностика
- 15.4. Профилирование производительности
- 15.5. Оптимизация критического пути рендеринга
- 15.6. Ленивая загрузка и код-сплиттинг

Часть IV: JavaScript в браузере

Глава 16: DOM (Document Object Model)

- 16.1. Иерархия DOM-дерева
- 16.2. Выбор элементов: getElementById, querySelector, и др.
- 16.3. Навигация по DOM: родительские, дочерние, соседние элементы
- 16.4. Манипуляция DOM: создание, изменение, удаление элементов
- 16.5. Атрибуты и свойства элементов
- 16.6. Стилизация через JavaScript

Глава 17: События и их обработка

- 17.1. Модель событий DOM
- 17.2. Типы событий: мышь, клавиатура, загрузка, формы
- 17.3. Обработчики событий и слушатели
- 17.4. Всплытие и погружение событий
- 17.5. Делегирование событий
- 17.6. Пользовательские события
- 17.7. Отмена стандартного поведения и остановка распространения

Глава 18: BOM (Browser Object Model) и Web API

- 18.1. Объект window: глобальный контекст браузера
- 18.2. Работа с окнами и фреймами
- 18.3. Объекты location, history, navigator

- 18.4. Хранение данных: cookies, localStorage, sessionStorage
- 18.5. Геолокация и определение устройств
- 18.6. Работа с графикой: Canvas API

Глава 19: Работа с формами и валидация

- 19.1. Доступ к элементам форм
- 19.2. События форм: submit, change, input
- 19.3. Валидация данных на стороне клиента
- 19.4. Работа с файлами через File API
- 19.5. Drag-and-drop API
- 19.6. Формат FormData и асинхронная отправка форм

Глава 20: Сетевые запросы и коммуникация

- 20.1. XMLHttpRequest и его ограничения
- 20.2. Fetch API: современный подход
- 20.3. Форматы данных: JSON, XML, FormData
- 20.4. Заголовки HTTP и кэширование
- 20.5. Веб-сокеты и реальное время
- 20.6. Server-Sent Events (SSE)
- 20.7. CORS и политика безопасности

Часть V: Современный JavaScript и инструменты

Глава 21: Современный синтаксис ES6+

- 21.1. Стрелочные функции и их особенности
- 21.2. Деструктуризация массивов и объектов
- 21.3. Операторы spread и rest
- 21.4. Параметры по умолчанию и именованные параметры
- 21.5. Шаблонные строки и тегированные шаблоны
- 21.6. Новые методы объектов и массивов
- 21.7. Опциональная цепочка и нулевое слияние

Глава 22: Функциональное программирование

- 22.1. Чистые функции и побочные эффекты
- 22.2. Иммутабельность и неизменяемые структуры данных
- 22.3. Функции высшего порядка
- 22.4. Каррирование и частичное применение
- 22.5. Композиция функций
- 22.6. Моноиды, функторы и монады (базовые концепции)
- 22.7. Библиотеки для ФП: Ramda, Lodash/fp

Глава 23: Тестирование JavaScript-кода

- 23.1. Принципы и методологии тестирования
- 23.2. Модульное тестирование с Jest
- 23.3. Тестирование асинхронного кода
- 23.4. Моск-объекты и заглушки
- 23.5. Интеграционное и end-to-end тестирование
- 23.6. Инструменты тестирования: Mocha, Chai, Jasmine, Cypress

Глава 24: Инструменты разработки

- 24.1. Транспиляция: Babel и его настройка
- 24.2. Линтеры: ESLint и стандарты кодирования
- 24.3. Форматтеры кода: Prettier
- 24.4. Системы контроля версий: Git и GitHub
- 24.5. CI/CD для JavaScript-проектов
- 24.6. Документирование кода: JSDoc, TypeDoc

Часть VI: Специализированные темы

Глава 25: Безопасность веб-приложений

- 25.1. XSS (межсайтовый скриптинг) и защита
- 25.2. CSRF (межсайтовая подделка запроса)
- 25.3. Инъекции и санитизация данных

- 25.4. Безопасная работа с LocalStorage и cookies
- 25.5. Content Security Policy (CSP)
- 25.6. Безопасные HTTP-заголовки

Глава 26: Progressive Web Apps (PWA)

- 26.1. Сервис-воркеры и их жизненный цикл
- 26.2. Кэширование ресурсов: Cache API
- 26.3. Офлайн-работа приложений
- 26.4. Push-уведомления
- 26.5. Установка PWA на устройство
- 26.6. Манифест веб-приложения

Глава 27: Графика и анимация

- 27.1. Canvas API: рисование и анимация
- 27.2. SVG и манипуляция через JavaScript
- 27.3. WebGL и Three.js (введение)
- 27.4. CSS-анимации и JavaScript
- 27.5. Библиотеки анимации: GSAP, Anime.js
- 27.6. Производительность анимаций и requestAnimationFrame

Глава 28: Работа с медиа

- 28.1. Audio API и создание веб-аудио
- 28.2. Video API и кастомные видеоплееры
- 28.3. WebRTC и видеоконференции
- 28.4. Распознавание речи и синтез
- 28.5. Захват медиа с устройств пользователя

Часть VII: Node.js и серверный JavaScript

Глава 29: Основы Node.js

- 29.1. Архитектура Node.js: событийно-ориентированная, неблокирующая
- 29.2. Модульная система Node.js
- 29.3. Глобальные объекты и процессы
- 29.4. Работа с файловой системой (fs модуль)
- 29.5. Потoki (streams) и буферы
- 29.6. Асинхронные паттерны в Node.js

Глава 30: Создание серверных приложений

- 30.1. HTTP-модуль и создание веб-сервера
- 30.2. Express.js: фреймворк для веб-приложений
- 30.3. Маршрутизация и промежуточное ПО
- 30.4. Работа с базами данных: MongoDB, PostgreSQL
- 30.5. ORM и ODM: Sequelize, Mongoose
- 30.6. Аутентификация и авторизация

Глава 31: Веб-сокеты и реальное время

- 31.1. Библиотека [Socket.io](#)
- 31.2. Создание чат-приложения
- 31.3. Масштабирование приложений реального времени
- 31.4. Альтернативы [Socket.io](#)

Часть VIII: Приложения и справочные материалы

Глава 32: Переход к фреймворкам

- 32.1. Когда использовать фреймворк
- 32.2. Сравнение React, Vue и Angular

- 32.3. Концепция виртуального DOM
- 32.4. Одностраничные приложения (SPA) и маршрутизация
- 32.5. Состояние приложения и управление состоянием

Глава 33: Будущее JavaScript

- 33.1. TC39 и процесс стандартизации
- 33.2. Экспериментальные возможности и стадии принятия
- 33.3. WebAssembly и его взаимодействие с JavaScript
- 33.4. Тенденции и направления развития

Приложения

- A. Справочник по методам и API
- B. Таблица операторов и их приоритетов
- C. Список распространенных ошибок и их решение
- D. Ресурсы для дальнейшего изучения
- E. Глоссарий терминов

Литература

Предметный указатель

Часть I: Введение и основы программирования

Глава 1: Введение в JavaScript и веб-разработку

1.1. Исторический контекст и эволюция JavaScript

1. Начало: 1995 год

Проблема, которую нужно было решить

В 1995 году интернет был скучным местом. Веб-страницы напоминали интерактивные книги: вы открывали страницу, читали текст, нажимали на ссылку и ждали загрузки новой страницы. Всё. Никакой анимации, проверки форм в реальном времени, всплывающих окон. Каждое действие требовало обновления всей страницы.

Главные игроки

- **Netscape Navigator** — доминирующий браузер того времени (80% рынка)
- **Microsoft Internet Explorer** — начинающий набирать популярность конкурент
- **Brendan Eich** — программист, нанятый Netscape для решения проблемы

2. 10 дней, которые изменили всё

Создание Mocha/LiveScript

В мае 1995 года Брендану Эйху дали задание: создать простой язык для добавления интерактивности на веб-страницы. Условия:

- Должен быть простым для веб-дизайнеров (не для программистов)
- Должен работать в браузере без установки дополнительных программ
- Должен выполняться сразу при загрузке страницы

Всего за 10 дней Эйх создал прототип языка, первоначально названного Mocha, затем переименованного в LiveScript.

Стратегическое переименование

В декабре 1995 года произошло ключевое событие: язык переименовали в **JavaScript**. Почему?

1. **Маркетинговая хитрость:** Java был модным, популярным языком (хотя JavaScript и Java почти не связаны технически)
2. **Привлечение внимания:** Разработчики думали: "О, это что-то связанное с Java, значит, это серьёзно!"
3. **Фактическая связь:** JavaScript мог взаимодействовать с Java-апплетами на странице

3. Браузерные войны и стандартизация

Первая война браузеров (1995-2001)

Microsoft, увидев успех Netscape, создала свою версию языка — **JScript** (чтобы избежать лицензионных проблем). Появились проблемы:

- **Разные реализации:** код, работавший в Netscape, не работал в IE
- **Несовместимость:** разработчики писали два варианта кода
- **Добавление специфичных возможностей:** каждая компания добавляла свои функции

ECMAScript — стандарт для спасения

В 1996 году Netscape отправила JavaScript в европейскую организацию по стандартизации ECMA. Результат:

- **ECMAScript (ES)** — официальная спецификация языка
- **JavaScript** — реализация ECMAScript (как Coca-Cola — реализация напитка "кола")
- **Первая версия ES1** вышла в июне 1997 года

4. Тёмные века: 2000-2004

Застой и доминирование IE

После победы в браузерных войнах Microsoft перестала развивать JScript в IE6 (выпущен в 2001). На 5 лет развитие языка фактически остановилось.

Проблемы этого периода:

1. **Отсутствие стандартов:** разные браузеры по-разному интерпретировали код
2. **Безопасность:** появились вирусы и уязвимости через JavaScript
3. **Дурная слава:** язык считался "игрушечным", несерьёзным
4. **Сложная разработка:** программисты тратили больше времени на исправление багов под разные браузеры, чем на создание функций

5. Ренессанс: 2004-2009

Появление AJAX (2005)

Ключевая технология, изменившая всё:

- **Что это:** возможность обновлять часть страницы без её полной перезагрузки
- **Пример:** вы пишете сообщение в чате, и оно появляется у всех без перезагрузки окна
- **Результат:** появились Gmail, Google Maps — приложения, работающие как десктопные

Библиотеки-спасители

Чтобы бороться с несовместимостью браузеров, появились библиотеки:

- **jQuery (2006)** — стала стандартом де-факто: "напиши один раз — работает везде"
- **Prototype, MooTools** — другие популярные библиотеки

ES4 и конфликт

Попытка создать радикально новую версию провалилась из-за разногласий между компаниями. Это отложило развитие языка на годы.

6. Современная эра: 2009-настоящее время

ES5 (2009) — первый большой шаг

Важные добавления:

- Строгий режим (`'use strict'`)
- Методы для работы с массивами (`forEach`, `map`, `filter`)
- JSON-поддержка
- Геттеры и сеттеры

ES6/ES2015 — революция

Самое значительное обновление за всю историю языка:

1. **Классы** — привычный ООП-синтаксис
2. **Стрелочные функции** — короткий синтаксис
3. **Модули** — организация кода в отдельные файлы
4. **Промисы** — решение проблемы "ада обратных вызовов"
5. **Шаблонные строки** — удобная работа со строками

Ежегодный цикл обновлений

С 2015 года JavaScript обновляется каждый год (ES2016, ES2017...). Новые функции:

- `async/await` (2017) — асинхронный код, который выглядит как синхронный
- Опциональная цепочка `?.` (2020) — защита от ошибок при обращении к несуществующим свойствам

7. JavaScript сегодня: вездесущий язык

Не только браузер

1. **Серверная часть:** Node.js (2009) позволил запускать JavaScript на сервере
2. **Мобильные приложения:** React Native, Ionic
3. **Десктопные приложения:** Electron (VS Code, Slack, Discord)
4. **Базы данных:** MongoDB использует JavaScript-подобный синтаксис
5. **Интернет вещей:** JavaScript на микроконтроллерах

Экосистема

- **npm** — самый большой реестр библиотек в мире (более 1.5 миллионов пакетов)
- **Фреймворки:** React, Vue, Angular — для создания сложных интерфейсов
- **Инструменты:** Babel (переводит современный код в старый), Webpack (собирает проект)

8. Ключевые уроки эволюции

От игрушки к серьёзному инструменту

JavaScript прошёл путь от:

- Простой добавки для анимации кнопок
- Через период дурной славы и несовместимости
- До универсального языка, работающего везде

Принципы, заложенные в основу:

1. **Обратная совместимость:** старый код должен работать всегда
2. **Гибкость:** можно писать в разных стилях (ООП, функциональный)
3. **Доступность:** работает на любом устройстве с браузером

9. Будущее JavaScript

Текущие тенденции:

1. **WebAssembly** — не замена, а дополнение для тяжёлых вычислений
2. **TypeScript** — JavaScript с типами, набирает огромную популярность
3. **Декларативный подход** — описание "что сделать", а не "как сделать"

Философия языка:

JavaScript уникален своей способностью адаптироваться. Его часто критиковали, но он выжил и стал самым популярным языком программирования в мире благодаря:

- Универсальности (браузер, сервер, мобильные устройства)
- Простоте начала работы (нужен только браузер)
- Огромному сообществу и экосистеме

Итог: JavaScript — это история успеха, построенная на компромиссах, обратной совместимости и уникальной способности находиться в нужном месте в нужное время. От скромного начала как инструмента для анимации кнопок до языка, на котором работает значительная часть мировой цифровой инфраструктуры.

Фундаментальный сдвиг парадигмы: от документов к приложениям

Ранний веб (до 2005): "Веб-страницы"

Веб был коллекцией **связанных документов**:

- HTML как структура (костяк)
- CSS как оформление (одежда)
- JavaScript как небольшие улучшения (аксессуары)

Пример типичного использования JavaScript в 2000-х:

- Проверка формы перед отправкой
- Простые анимации (выпадающие меню)
- Всплывающие окна
- Изменение изображений при наведении

JavaScript был **опциональным украшением** — если он отключался, сайт продолжал работать, просто становился менее удобным.

Современный веб (после 2015): "Веб-приложения"

Сегодня JavaScript — это **несущий каркас**:

- HTML становится шаблоном для начальной загрузки
- CSS отвечает за внешний вид
- **JavaScript управляет ВСЕМ**

Ключевое изменение: без JavaScript большинство современных веб-приложений полностью неработоспособны.

Три столпа современной роли JavaScript

Столп 1: Динамические интерфейсы в реальном времени

Без перезагрузки страницы

Раньше: каждое действие → запрос к серверу → новая страница.

Сейчас: действие → JavaScript обновляет только нужную часть.

Примеры:

- **Социальные сети:** Бесконечная лента, лайки, комментарии появляются мгновенно
- **Торговые платформы:** Добавление товара в корзину без перехода на другую страницу
- **Карты и навигация:** Плавное перемещение, масштабирование без перезагрузки

Технологии, обеспечивающие это:

1. **AJAX/Fetch API** — запросы данных без перезагрузки
2. **WebSockets** — двусторонняя связь в реальном времени (чаты, уведомления)
3. **Server-Sent Events** — потоковое получение данных от сервера

Столп 2: Управление состоянием приложения

Что такое "состояние"?

Все данные, которые меняются во время работы приложения:

- Текущий пользователь и его права
- Открытые вкладки/окна
- Данные форм (даже если не отправлены)
- Настройки интерфейса

Сложность управления:

Современное приложение может содержать **тысячи переменных состояния**.
JavaScript решает эту проблему через:

Библиотеки и подходы:

- **Redux/MobX** — централизованные хранилища состояния
- **React Context/ Vuex** — встроенные решения фреймворков
- **Immutable.js** — неизменяемые структуры данных для предсказуемости

Столп 3: Бизнес-логика на клиенте

Перераспределение ответственности

Раньше: сервер ВСЁ делает, браузер только показывает.
Сейчас: разделение обязанностей.

Что обрабатывает JavaScript в браузере:

1. **Валидация форм** — мгновенная проверка без запроса к серверу
2. **Маршрутизация** — переход между "страницами" без перезагрузки
3. **Шаблонизация** — преобразование данных в HTML
4. **Логика отображения** — что, когда и как показывать пользователю
5. **Кэширование** — хранение данных локально для ускорения работы

Преимущества:

- **Скорость**: мгновенный отклик на действия
- **Снижение нагрузки на сервер**: меньше запросов
- **Работа офлайн**: приложения могут функционировать без интернета

Конкретные роли JavaScript в разных типах проектов

1. Информационные сайты и блоги (статичные → динамичные)

Раньше:

Простая структура, комментарии через перезагрузку страницы.

Сейчас с JavaScript:

- **Интерактивные элементы:** слайдеры, аккордеоны, табы
- **Умный поиск:** подсказки при вводе, фильтрация результатов
- **Ленивая загрузка:** изображения загружаются при прокрутке
- **PWA-возможности:** уведомления о новых статьях, офлайн-чтение

Пример: [Medium.com](https://medium.com) — плавное чтение, интерактивные выделения текста, реакции без перезагрузки.

2. Электронная коммерция (магазины)

Без JavaScript:

- Каждая операция с корзиной — новая страница
- Фильтрация товаров — перезагрузка
- Форма заказа — одна большая страница

С JavaScript:

- **Динамическая корзина:** добавление/удаление товаров в реальном времени
- **"Умные" фильтры:** мгновенное обновление результатов при выборе параметров
- **Пошаговые формы:** плавные переходы между этапами оформления
- **Персонализация:** рекомендации на основе поведения пользователя

Пример: Amazon — мгновенное обновление цены при изменении количества товаров, живые рекомендации.

3. Веб-приложения (продуктивность, инструменты)

Полноценные программы в браузере:

- **Офисные пакеты:** Google Docs, Microsoft Office Online
- **Графические редакторы:** Figma, Canva, Photopea (Photoshop онлайн)
- **Системы управления:** Trello, Asana, Jira
- **Среды разработки:** CodeSandbox, Glitch

Особенности:

- Работа с большими объемами данных
- Сложная бизнес-логика
- Автосохранение и история изменений
- Редактирование в реальном времени несколькими пользователями

4. Социальные сети и коммуникационные платформы

Основные функции на JavaScript:

- **Лента в реальном времени:** новые появления появляются автоматически
- **Мгновенные сообщения:** чаты без перезагрузки
- **Уведомления:** push-уведомления в браузере
- **Загрузка медиа:** прогресс-бары, предпросмотр, обрезка

5. Дашборды и аналитические системы

Сложная визуализация данных:

- **Интерактивные графики:** изменение параметров → мгновенное обновление
- **Виджеты с живыми данными:** акции, статистика, метрики
- **Drag-and-drop:** перестановка блоков пользователем
- **Экспорт данных:** генерация отчетов на клиенте

Архитектурные паттерны, которые определяют роль JavaScript

1. Single Page Application (SPA) — одностраничное приложение

Суть:

Вся логика загружается один раз, затем JavaScript динамически обновляет контент.

Как работает:

1. Первая загрузка: базовый HTML + JavaScript-фреймворк
2. JavaScript определяет маршрут (например, /profile)
3. Загружаются данные для этого маршрута (через API)
4. JavaScript рендерит соответствующий интерфейс
5. При переходе на /messages — шаги 2-4 повторяются без перезагрузки

Примеры: Gmail, Facebook, Twitter

2. Progressive Web Apps (PWA) — прогрессивные веб-приложения

JavaScript как мост между вебом и нативными приложениями:

Ключевые возможности:

1. **Офлайн-работа:** Service Workers кэшируют ресурсы
2. **Установка на устройство:** как обычное приложение
3. **Push-уведомления:** даже когда сайт не открыт
4. **Доступ к оборудованию:** камера, геолокация, микрофон

Роль JavaScript: управляет всем жизненным циклом PWA

3. Server-Side Rendering (SSR) и Static Site Generation (SSG)

Компромисс для производительности и SEO:

Проблема SPA:

- Первая загрузка медленная (большой JavaScript-бандл)
- Плохо для SEO (поисковые боты не всегда выполняют JavaScript)

Решение:

- **SSR:** Сервер выполняет JavaScript, отправляет готовый HTML
- **SSG:** HTML генерируется на этапе сборки

Но: после загрузки страницы JavaScript "оживляет" её, добавляя интерактивность (гидратация).

Примеры фреймворков: Next.js (React), Nuxt.js (Vue), SvelteKit

JavaScript как универсальный язык веб-платформы

Web APIs — мост к возможностям браузера и устройства

JavaScript не существует в вакууме — он взаимодействует с сотнями API:

Категории API:

1. Мультимедиа:

- ➔ Web Audio API — обработка звука
- ➔ WebRTC — видео-звонки прямо в браузере
- ➔ MediaRecorder — запись аудио/видео

2. Графика и анимация:

- ➔ Canvas API — пиксельная графика
- ➔ WebGL — 3D-графика
- ➔ Web Animations API — сложные анимации

3. Устройства:

- ➔ Geolocation API — определение местоположения
- ➔ Device Orientation — акселерометр, гироскоп

- ➔ Web Bluetooth — подключение к Bluetooth-устройствам

4. Хранение данных:

- ➔ LocalStorage/SessionStorage — простые ключ-значение
- ➔ IndexedDB — полноценная база данных в браузере
- ➔ Cache API — кэширование ресурсов для офлайн-работы

JavaScript как "клей" между различными технологиями

Современная веб-страница объединяет:

- **Верстку** (HTML) через DOM API
- **Стили** (CSS) через CSSOM API
- **Медиа** (видео, аудио) через соответствующие API
- **Сторонние виджеты** (карты, аналитика, чаты)
- **Микросервисы** через REST/GraphQL API

JavaScript координирует все эти компоненты, обеспечивая их согласованную работу.

Эволюция роли по мере роста сложности проекта

Уровень 1: Простой сайт-визитка

- **Роль JS:** интерактивное меню, слайдер, форма обратной связи
- **Объем:** < 100 строк кода
- **Подход:** Vanilla JS (чистый JavaScript) или небольшие скрипты

Уровень 2: Корпоративный сайт с каталогом

- **Роль JS:** фильтрация товаров, динамическая корзина, анимации
- **Объем:** 1000-5000 строк
- **Подход:** jQuery или современный Vanilla JS + библиотеки

Уровень 3: Веб-приложение (SaaS)

- **Роль JS:** вся бизнес-логика, управление состоянием, маршрутизация
- **Объем:** 10 000 - 100 000 строк
- **Подход:** React/Vue/Angular + менеджер состояния + роутер

Уровень 4: Комплексная платформа (социальная сеть, маркетплейс)

- **Роль JS:** несколько независимых приложений в одном, real-time обновления, офлайн-синхронизация
- **Объем:** 100 000+ строк
- **Подход:** Микрофронтенды, несколько фреймворков, сложная архитектура

Критика и ответы на неё

Распространённые претензии:

1. "Слишком много JavaScript":

- ➔ Проблема: сайты грузятся медленно
- ➔ Решение: code splitting, lazy loading, оптимизация бандлов

2. "Ломает доступность":

- ➔ Проблема: скринридеры не понимают динамически созданный контент
- ➔ Решение: семантическая верстка, ARIA-атрибуты, тестирование

3. "Сложность разработки":

- ➔ Проблема: нужно знать множество инструментов
- ➔ Решение: грамотный выбор стека, постепенное усложнение

4. "Проблемы с SEO":

- ➔ Проблема: поисковые боты не всегда выполняют JS
- ➔ Решение: SSR/SSG, pre-rendering, динамический рендеринг

Баланс, который ищет современная разработка:

Достаточно JavaScript для интерактивности, но не настолько, чтобы нарушить:

- Производительность
- Доступность
- Возможность индексации
- Удобство разработки

Будущее роли JavaScript

1. Edge Computing — выполнение ближе к пользователю

JavaScript запускается не только в браузере, но и:

- На CDN-серверах (Cloudflare Workers)
- На устройствах IoT
- В процессе сборки (build-time)

2. WebAssembly — дополнение, а не замена

Для задач, где важна производительность:

- Обработка видео/аудио
- 3D-графика
- Сложные вычисления

JavaScript остаётся для:

- Взаимодействия с DOM
- Управления состоянием приложения
- Работы с событиями пользователя

3. Полная нативность в вебе

Project Fugu — инициатива по предоставлению веб-приложениям возможностей нативных приложений:

- Файловая система
- Bluetooth/USB-устройства
- Календарь, контакты
- Более глубокая интеграция с ОС

Заключение: почему JavaScript незаменим?

Уникальные преимущества, обеспечивающие доминирующую роль:

1. Универсальность экосистемы:

- ➔ Один язык для фронтенда, бэкенда, мобильных приложений
- ➔ Крупнейший в мире реестр пакетов (npm)
- ➔ Огромное сообщество и ресурсы для обучения

2. Нулевой барьер входа:

- ➔ Не нужно ничего устанавливать для начала
- ➔ Результат виден мгновенно
- ➔ Инструменты разработчика в каждом браузере

3. Постоянная эволюция:

- ➔ Годовой цикл обновлений
- ➔ Обратная совместимость (старый код работает)
- ➔ Активное сообщество, предлагающее улучшения

4. Интеграция с платформой:

- ➔ Прямой доступ к возможностям браузера
- ➔ Единая среда выполнения во всех браузерах
- ➔ Стандартизированные API

Итог: JavaScript превратился из простого скриптового языка для анимации кнопок в полноценную платформу для создания приложений. Его роль — быть **универсальным исполнителем**, который связывает интерфейс, логику и данные, превращая веб-браузер в самую распространенную в мире среду выполнения приложений.

Современный веб без JavaScript — это как автомобиль без двигателя: выглядит как автомобиль, но никуда не едет. JavaScript обеспечивает движение, интерактивность и жизнь в цифровом пространстве.

Введение: уникальное положение JavaScript

JavaScript занимает особое место среди языков программирования. Это не просто "один из языков" — это **универсальный клей веб-платформы**, который эволюционировал в нечто большее, чем предполагали его создатели.

Ключевой парадокс JavaScript:

- Самый **критикуемый** язык среди разработчиков
- Самый **популярный** язык по всем рейтингам (GitHub, Stack Overflow, TIOBE)
- Самый **неоднозначный** с точки зрения дизайна

Давайте рассмотрим сравнение по ключевым параметрам.

Сравнительная таблица: общий обзор

Параметр	JavaScript	Python	Java	C#	Go	Rust
Парадигма	Мультипарадигменный	Мультипарадигменный	ООП	ООП	Процедурный	Системный
Типизация	Динамическая, слабая	Динамическая, сильная	Статическая, сильная	Статическая, сильная	Статическая, сильная	Статическая, сильная
Компиляция	JIT-компиляция	Интерпретация	JIT-компиляция (JVM)	JIT-компиляция (CLR)	Компиляция в бинарник	Компиляция в бинарник
Область применения	Универсальный (веб, сервер, десктоп)	Универсальный (ML, веб, скрипты)	Сервер, мобильные, корп. ПО	Сервер, десктоп, игры	Сервер, системное ПО	Системное, безопасность
Сложность изучения	Низкая (начало) / Высокая (продвинутый)	Низкая	Средняя	Средняя	Низкая-Средняя	Высокая

Параметр	JavaScript	Python	Java	C#	Go	Rust
Производительность	Высокая (V8)	Средняя	Высокая	Высокая	Очень высокая	Очень высокая
Экосистема	Огромная (npm)	Огромная (PyPI)	Огромная (Maven)	Большая (NuGet)	Растущая	Растущая
Год создания	1995	1991	1995	2000	2009	2010

Часть 1: JavaScript vs Python — битва за сердца НОВИЧКОВ

Сходства:

1. **Динамическая типизация** — переменные могут менять тип
2. **Высокоуровневые** — абстрагированы от железа
3. **Интерпретируемые** (хотя JS имеет JIT-компиляцию)
4. **Большие сообщества** и экосистемы

Ключевые различия:

Синтаксис и читаемость

```

javascript

// JavaScript
function calculateTotal(price, quantity, tax) {
    return (price * quantity) * (1 + tax);
}

// Python
def calculate_total(price, quantity, tax):
    return (price * quantity) * (1 + tax)

```

Python:

- Четкие правила отступов (часть синтаксиса)
- Меньше синтаксического шума (нет фигурных скобок, точек с запятой)
- Принцип "должен быть один очевидный способ сделать это"

JavaScript:

- Свободный формат (фигурные скобки, точки с запятой опциональны)
- Больше синтаксических вариантов (функции можно объявить 6+ способами)
- Принцип "есть миллион способов сделать это"

Асинхронное программирование

python

Python (синхронный по умолчанию)

`import requests`

`response = requests.get('https://api.example.com/data')` *# Блокирует выполнение*

`data = response.json()`

javascript

// JavaScript (асинхронный по дизайну)

`fetch('https://api.example.com/data')`

`.then(response => response.json())` *// Не блокирует*

`.then(data => console.log(data));`

Ключевое различие: JavaScript изначально проектировался для асинхронных операций в браузере, Python добавлял асинхронность позже (async/await в Python 3.5+).

Области применения

Python сильнее в:

- Data Science и Machine Learning (NumPy, Pandas, TensorFlow)
- Научные вычисления
- Системное администрирование
- Быстрое прототипирование

JavaScript сильнее в:

- Фронтенд-разработка (не имеет альтернатив в браузере)
- Real-time приложения (чаты, игры)
- Full-stack разработка (один язык для всего)
- Проекты с большим количеством параллельных соединений (Node.js)

Когда выбирать что:

- **Выбирайте Python**, если: работаете с данными, ML, научными вычислениями
- **Выбирайте JavaScript**, если: создаете веб-интерфейсы, full-stack приложения, реальное время
- **Изучайте оба**, если: хотите быть универсальным разработчиком

Часть 2: JavaScript vs Java — сравнение несравнимого

Главное заблуждение:

JavaScript и Java практически не связаны, несмотря на название. Это как "автомобиль" и "автограф" — общий корень в названии, но разные сущности.

Фундаментальные различия:

Модель выполнения

```
java
// Java - компилируется в байт-код для JVM
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
// Требуется: javac Main.java → java Main
javascript
// JavaScript - интерпретируется (JIT) в браузере или Node.js
console.log("Hello, World!");
// Запуск: просто открыть в браузере или node script.js
```

Java: статическая компиляция, строгая типизация, управление памятью через Garbage Collector

JavaScript: динамическая интерпретация/JIT, слабая типизация, автоматическое управление памятью

Система типов

```
java
// Java - статическая типизация
int number = 42;           // всегда int
String text = "Hello";     // всегда String
number = "text";           // ОШИБКА компиляции
javascript
```

```
// JavaScript - динамическая типизация
let value = 42;           // number
value = "Hello";         // теперь string - OK
value = true;            // теперь boolean - OK
value = { x: 1, y: 2 };  // теперь object - OK
```

ООП подход

java

```
// Java - классическое ООП на классах
```

```
public class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    public void speak() {
        System.out.println("Some sound");
    }
}
```

javascript

```
// JavaScript - прототипное ООП (до ES6)
```

```
function Animal(name) {
    this.name = name;
}

Animal.prototype.speak = function() {
    console.log("Some sound");
};
```

```
// Или классы в ES6 (синтаксический сахар)
```

```
class Animal {
    constructor(name) {
        this.name = name;
    }

    speak() {
        console.log("Some sound");
    }
}
```

Когда выбирать что:

- **Java:** корпоративные приложения, Android-разработка, высоконагруженные сервисы
- **JavaScript:** веб-интерфейсы, стартапы, быстрая разработка, реальное время
- **Совместное использование:** часто Java на бэкенде + JavaScript на фронтенде

Часть 3: JavaScript vs TypeScript — эволюция или революция?

Важное уточнение:

TypeScript **не конкурент** JavaScript, а его **надмножество**. Весь валидный JavaScript код — валидный TypeScript код.

Ключевое отличие: система типов

```
javascript

// JavaScript - динамические типы
function calculateTotal(price, quantity) {
  return price * quantity; // Что если price = "100"? NaN!
}

const total = calculateTotal("100", 2); // Ошибка только в runtime
console.log(total); // NaN

typescript

// TypeScript - статическая типизация
function calculateTotal(price: number, quantity: number): number {
  return price * quantity;
}

const total = calculateTotal("100", 2); // ОШИБКА КОМПИЛЯЦИИ
// Аргумент типа "string" нельзя назначить параметру типа "number"
```

Преимущества TypeScript:

1. Обнаружение ошибок на этапе компиляции, а не в рантайме

2. **Автодополнение и навигация** по коду в IDE
3. **Документирование** кода через типы
4. **Рефакторинг** становится безопаснее

Преимущества чистого JavaScript:

1. **Меньше шаблонного кода** (не нужно описывать типы)
2. **Более быстрая разработка** для небольших проектов
3. **Нет необходимости в компиляции**
4. **Меньше кривой обучения**

Тенденции:

- **TypeScript набирает популярность** в корпоративной разработке
- **Многие фреймворки** (Angular, Vue 3) используют TypeScript
- **React-сообщество** активно переходит на TypeScript
- **Но JavaScript остаётся** для скриптов, прототипов, небольших проектов

Часть 4: JavaScript vs C# — сравнение экосистем Microsoft

Исторический контекст:

- **C#** создан Microsoft в 2000 году для платформы .NET
- **JavaScript** не контролируется одной компанией (стандарт ECMA)
- **TypeScript** (Microsoft) пытается принести C#-подобную типизацию в JavaScript

Синтаксические сравнения:

```
csharp

// C#
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
```

```

{
    string name = "World";
    Console.WriteLine($"Hello, {name}!");

    var list = new List<int> { 1, 2, 3 };
    var squares = list.Select(x => x * x);
}
}
}
javascript

// JavaScript
const name = "World";
console.log(`Hello, ${name}!`);

const list = [1, 2, 3];
const squares = list.map(x => x * x);

```

Ключевые различия:

Платформа и экосистема

C#:

- Тесно связан с экосистемой Microsoft (.NET Framework, .NET Core)
- Отличная поддержка в Visual Studio
- Сильнее в enterprise, играх (Unity), Windows-приложениях

JavaScript:

- Кроссплатформенный по дизайну
- Множество IDE и редакторов (VS Code, WebStorm, Sublime)
- Доминирует в веб-разработке

Параллельное выполнение

C#: многопоточность через async/await, Task Parallel Library

JavaScript: однопоточный event loop, Web Workers для многопоточности

Когда выбирать что:

- **C#**: корпоративные Windows-приложения, игры на Unity, backend на .NET
- **JavaScript**: веб-разработка, кроссплатформенные приложения, стартапы
- **Blazor** (C# для веба) vs **JavaScript**: пока JavaScript доминирует в вебе

Часть 5: JavaScript vs Go/Rust — сравнение с системными языками

Фундаментальное различие парадигм:

JavaScript — высокоуровневый, абстрактный

```
javascript

// Автоматическое управление памятью
const obj = { x: 1, y: 2 }; // Память выделяется автоматически
// Сборщик мусора освободит память когда obj перестанет использоваться
```

Rust — системный, с ручным управлением памятью (без GC)

```
rust

// Руководство владением памятью
let s1 = String::from("hello"); // s1 владеет данными
let s2 = s1;                    // владение перемещается к s2
// println!("{}", s1);         // ОШИБКА: s1 больше не владеет данными
```

Go — баланс между производительностью и простотой

```
go

// Упрощенный синтаксис с сборщиком мусора
package main

import "fmt"

func main() {
    m := map[string]int{"foo": 1, "bar": 2} // GC управляет памятью
    fmt.Println(m)
}
```

Производительность:

- **Rust/Go:** компилируются в нативный код, высокая производительность
- **JavaScript:** JIT-компиляция, высокая производительность в V8, но накладные расходы

Области применения:

- **JavaScript:** веб, высокоуровневые приложения, быстрая разработка
- **Rust:** системное программирование, безопасность, встраиваемые системы
- **Go:** серверы, сетевое программирование, инструменты DevOps

Важный тренд: WebAssembly

JavaScript становится **хост-языком** для других языков через WebAssembly:

- Rust/C++/Go → компиляция в WASM → выполнение в браузере
- JavaScript управляет взаимодействием с DOM

Часть 6: Сильные стороны JavaScript

1. Универсальность и "write once, run anywhere"

javascript

```
// Один и тот же код может работать:  
// - В браузере  
// - На сервере (Node.js)  
// - На мобильных (React Native)  
// - На десктопе (Electron)  
// - В базах данных (MongoDB)  
// - На IoT устройствах
```

2. Асинхронная модель по умолчанию

javascript

```
// Event Loop позволяет обрабатывать тысячи соединений  
server.on('connection', (socket) => {  
  // Обработка каждого подключения без блокировки  
});
```

```
// Асинхронные операции естественны
const fetchData = async () => {
  const data = await fetch('/api/data');
  const json = await data.json();
  return json;
};
```

3. Динамическая природа и метапрограммирование

javascript

```
// Динамическое создание объектов
const dynamicKey = 'property' + Date.now();
const obj = {
  [dynamicKey]: 'value', // Вычисляемые имена свойств
  ...otherObject,       // Spread оператор
};

// Метапрограммирование
const handler = {
  get(target, prop) {
    console.log(`Accessed property: ${prop}`);
    return target[prop];
  }
};

const proxy = new Proxy(obj, handler);
proxy.someProperty; // Выведет: "Accessed property: someProperty"
```

4. Огромная экосистема и сообщество

- 🌐 **npm**: самый большой реестр пакетов (>1.5 млн)
- 🌐 **Фреймворки на любой вкус**: React, Vue, Angular, Svelte
- 🌐 **Инструменты для всего**: сборка, тестирование, линтинг
- 🌐 **Постоянные обновления**: ES6, ES7, ES8...

5. Нулевой барьер входа

html

```
<!-- Начать программировать можно прямо сейчас -->
<script>
  alert("Hello, World!");
```

```
</script>
```

```
<!-- Не нужно: компиляторов, IDE, сложной настройки -->
```

Часть 7: Слабые стороны и критика JavaScript

1. "Странные" части языка

```
javascript
```

```
// Примеры неочевидного поведения
```

```
[] + [] = ""           // Массив + массив = строка
[] + {} = "[object Object]" // Массив + объект = строка
{} + [] = 0             // Объект + массив = число
null == undefined      // true (но null !== undefined)
"5" + 3 = "53"         // Конкатенация, не сложение
"5" - 3 = 2            // Вычитание работает
true + true = 2        // Boolean складываются как числа
```

2. Проблемы с типами и приведением

```
javascript
```

```
// Неявное приведение типов может быть опасным
```

```
const userInput = "0";
if (userInput) { // "0" приводится к false!
  console.log("Truthy");
} else {
  console.log("Falsy"); // Выполнится это!
}
```

```
// == vs ===
```

```
0 == "0" // true (приведение типов)
0 === "0" // false (строгое сравнение)
```

3. Callback hell и асинхронная сложность

```
javascript
```

```
// Пирамида смерти (callback hell)
```

```
getData(function(a) {
  getMoreData(a, function(b) {
    getMoreData(b, function(c) {
      getMoreData(c, function(d) {
```

```
    getMoreData(d, function(e) {  
        // ...  
    });  
});  
});  
});  
});  
});
```

Хотя современные `async/await` решают эту проблему.

4. Проблемы с масштабированием больших проектов

- Отсутствие статической типизации (решается TypeScript)
- Слишком много способов сделать одно и то же
- Сложности с организацией кода (решается модулями, фреймворками)

5. Проблемы безопасности

- XSS (межсайтовый скриптинг) — самая распространенная уязвимость веба
- Необходимость санитизации всех пользовательских данных

Часть 8: Эволюция как ответ на критику

Как JavaScript исправляет свои недостатки:

1. Стандартизация через ECMAScript

- Ежегодный цикл обновлений
- Новые функции проходят 4 стадии утверждения
- Обратная совместимость (старый код должен работать)

2. TypeScript как решение проблем типизации

`typescript`

// TypeScript добавляет статическую типизацию

```
interface User {  
    id: number;  
    name: string;  
    email: string;
```

```
}
```

```
function getUser(id: number): Promise<User> {  
  // Компилятор проверяет тип возвращаемого значения  
  return fetch(`/api/users/${id}`).then(res => res.json());  
}
```

3. Современные инструменты для масштабирования

- **ES6 Modules** — организация кода
- **Webpack/Rollup** — сборка и оптимизация
- **ESLint/Prettier** — поддержание качества кода
- **Jest/Cypress** — тестирование

4. Лучшие практики и паттерны

- **Функциональное программирование** (чистые функции, иммутабельность)
- **Компонентная архитектура** (React, Vue компоненты)
- **Состояние приложения** (Redux, MobX, Context API)

Часть 9: Когда какой язык выбирать — практическое руководство

Матрица выбора языка:

Тип проекта	Лучший выбор	Альтернативы	Почему?
Веб-фронтенд	JavaScript/ TypeScript	Нет альтернатив	Только JS выполняется в браузерах
Веб-бэкенд	JavaScript (Node.js)	Python, Go, Java, C#	Единый стек, асинхронная модель
Мобильные приложения	JavaScript (React Native)	Kotlin (Android), Swift (iOS)	Кроссплатформенность
Десктоп приложения	JavaScript (Electron)	C#, Java, C++	Веб-технологии, кроссплатформенность
Data Science	Python	R, Julia	Лучшие библиотеки для анализа данных

Тип проекта	Лучший выбор	Альтернативы	Почему?
Системное программирование	Rust, C++	Go, C	Безопасность памяти, производительность
Корпоративные приложения	Java, C#	JavaScript (Node.js)	Стабильность, типизация, инструменты
Скрипты и автоматизация	Python, Bash	JavaScript (Node.js)	Простота, библиотеки для всего
Игры	C# (Unity), C++ (Unreal)	JavaScript (WebGL, Three.js)	Производительность, инструменты
Блокчейн и крипто	Solidity (Ethereum)	Rust, Go, JavaScript	Экосистема, инструменты

JavaScript в разных ролях:

Как основной язык:

- Веб-приложения (фронтенд и бэкенд)
- Кроссплатформенные мобильные приложения
- Быстрые прототипы и MVP
- Реальные приложения (Slack, VS Code, Discord)

Как вспомогательный язык:

- Автоматизация задач (скрипты в Node.js)
- Инструменты разработки (Webpack, Babel, ESLint)
- Тестирование (Jest, Cypress, Puppeteer)
- Документация (JSDoc, Storybook)

Заключение: уникальная ниша JavaScript

Почему JavaScript выжил и процветает, несмотря на недостатки:

1. **Монополия в браузерах** — нет реальных альтернатив для клиентского веба

2. **Сетевой эффект** — чем больше разработчиков, тем больше библиотек, тем больше разработчиков
3. **Постоянная эволюция** — язык исправляет свои недостатки (ES6+, TypeScript)
4. **Универсальность** — один язык для всей стека разработки
5. **Низкий порог входа** — начать можно с блокнота и браузера

Философский вывод:

JavaScript — это **язык компромиссов**:

- Свобода vs структура
- Гибкость vs предсказуемость
- Простота начала vs сложность масштабирования
- Динамизм vs безопасность типов

Прогноз на будущее:

JavaScript не будет "заменен" другим языком в вебе, но:

1. **TypeScript** станет стандартом для больших проектов
2. **WebAssembly** дополнит JavaScript для тяжелых вычислений
3. **Специализированные языки** будут использоваться через компиляцию в JS/WASM
4. **JavaScript** останется универсальным клеем, соединяющим разные технологии

Итог: JavaScript — не "лучший" язык в академическом смысле, но он идеально подходит для своей роли: быть **универсальным языком веб-платформы**. Его сила — не в совершенстве дизайна, а в экосистеме, сообществе и уникальной способности адаптироваться к меняющимся требованиям веб-разработки.

1.4. Экосистема JavaScript: браузеры, Node.js, среды выполнения

Введение: от простого скриптового языка к вселенной платформ

JavaScript больше не существует в изоляции. Сегодня это **целая экосистема взаимосвязанных платформ, инструментов и сред выполнения**, каждая со своей специализацией, но использующая один и тот же язык.

Эволюция экосистемы:

text

1995-2008: JavaScript только в браузерах



2009: Node.js - JavaScript на сервере



2013: React Native - JavaScript для мобильных



2015: Electron - JavaScript для десктоп

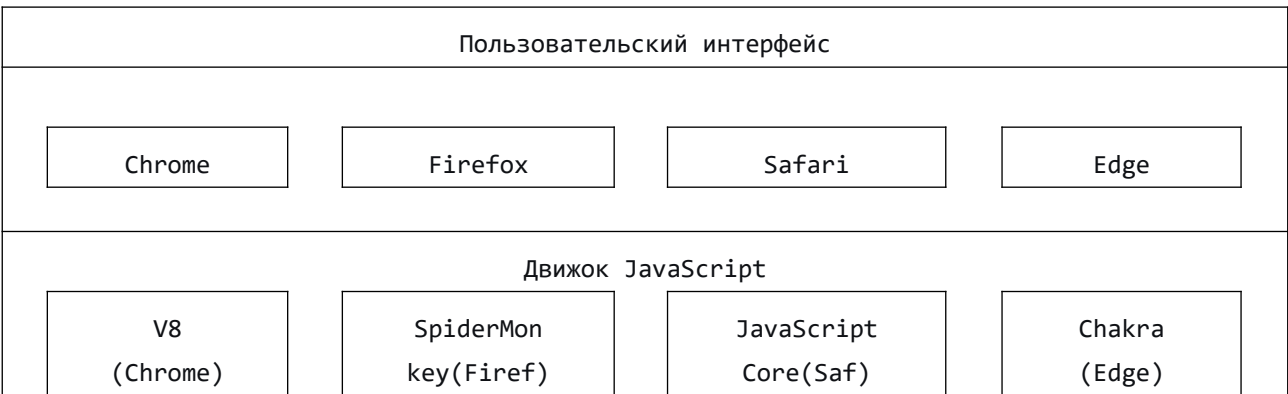


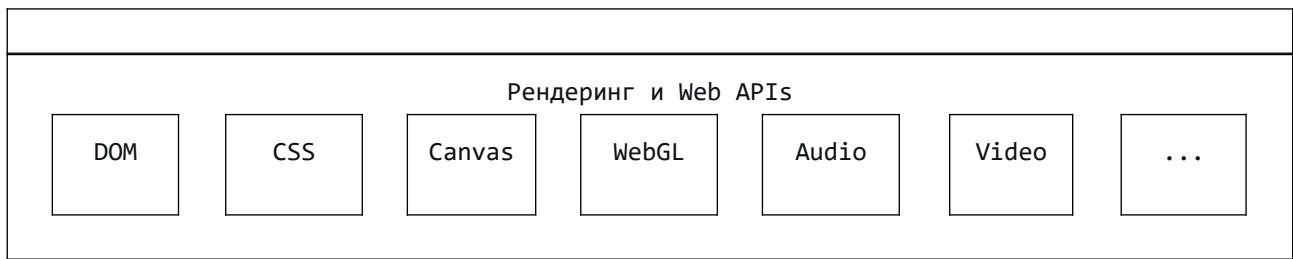
2020+: JavaScript везде (IoT, edge, ML)

Часть 1: Браузеры — родной дом JavaScript

Архитектура современного браузера

Text





Движки JavaScript: сердце браузера

V8 (Google Chrome, Node.js, Edge)

Архитектура и оптимизации:

```
javascript

// Пример того, как V8 оптимизирует код
function calculateSum(arr) {
  let sum = 0;
  // V8 создает "скрытые классы" для объектов
  // и использует "инлайн-кэширование"
  for (let i = 0; i < arr.length; i++) {
    sum += arr[i]; // Турбофан оптимизирует этот цикл
  }
  return sum;
}

// При многократном вызове V8 компилирует функцию
// в оптимизированный машинный код
for (let i = 0; i < 10000; i++) {
  calculateSum([1, 2, 3, 4, 5]);
}
```

Ключевые особенности V8:

1. **Ignition** - интерпретатор для быстрого старта
2. **TurboFan** - оптимизирующий компилятор
3. **Orinoco** - параллельный сборщик мусора
4. **Ленивая компиляция** - компилирует только "горячие" функции

SpiderMonkey (Firefox)

Историческое значение: первый движок JavaScript, созданный Бренданом Эйхом.

Уникальные особенности:

- **IonMonkey** - оптимизирующий JIT-компилятор
- **Baseline Compiler** - быстрая компиляция без сложных оптимизаций
- **WarpBuilder** - улучшенная оптимизация для современного JS

JavaScriptCore (Safari, WebKit)

Архитектура от Apple:

- **LLInt** (Low Level Interpreter) - базовый интерпретатор
- **Baseline JIT** - первичная компиляция
- **DFG JIT** (Data Flow Graph) - оптимизация на основе потока данных
- **FTL JIT** (Faster Than Light) - агрессивная оптимизация через LLVM

Web APIs: то, что делает браузер уникальной средой

Иерархия Web APIs

text

1. Основные API (DOM, BOM, XMLHttpRequest)
2. Мультимедиа API (Web Audio, WebRTC, Media Capture)
3. Графические API (Canvas, WebGL, WebGPU)
4. Устройства API (Geolocation, Sensors, Bluetooth)
5. Хранение API (LocalStorage, IndexedDB, Cache)
6. Коммуникационные API (WebSockets, WebRTC, Push)
7. Производительность API (Web Workers, Service Workers)

Пример комплексного использования API:

javascript

```
// Видеоконференция с использованием нескольких API
async function startVideoConference() {
  // 1. MediaDevices API - доступ к камере и микрофону
  const stream = await navigator.mediaDevices.getUserMedia({
    video: true,
    audio: true
  });

  // 2. WebRTC API - установка соединения
  const peerConnection = new RTCPeerConnection();
  stream.getTracks().forEach(track => {
```

```

    peerConnection.addTrack(track, stream);
  });

  // 3. Canvas API - обработка видео
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');

  // 4. Web Audio API - обработка звука
  const audioContext = new AudioContext();
  const source = audioContext.createMediaStreamSource(stream);

  // 5. IndexedDB - локальное хранение
  const db = await openDB('conference', 1);

  // 6. WebSockets - обмен сигналами
  const ws = new WebSocket('wss://signaling.server');
}

```

Совместимость между браузерами: вызовы и решения

Проблема:

javascript

```

// Разные браузеры могут по-разному реализовывать API
const elem = document.getElementById('test');

// Старый способ (IE vs другие)
if (elem.addEventListener) {      // Современные браузеры
  elem.addEventListener('click', handler);
} else if (elem.attachEvent) {    // IE8 и ниже
  elem.attachEvent('onclick', handler);
}

```

Решение через полифиллы:

javascript

```

// Полифилл для Promise (если браузер не поддерживает)
if (!window.Promise) {
  window.Promise = class Promise {
    constructor(executor) {
      // Реализация Promise для старых браузеров
    }
  }
}

```

```

    then(onFulfilled, onRejected) {
      // ...
    }
    catch(onRejected) {
      // ...
    }
  };
}

```

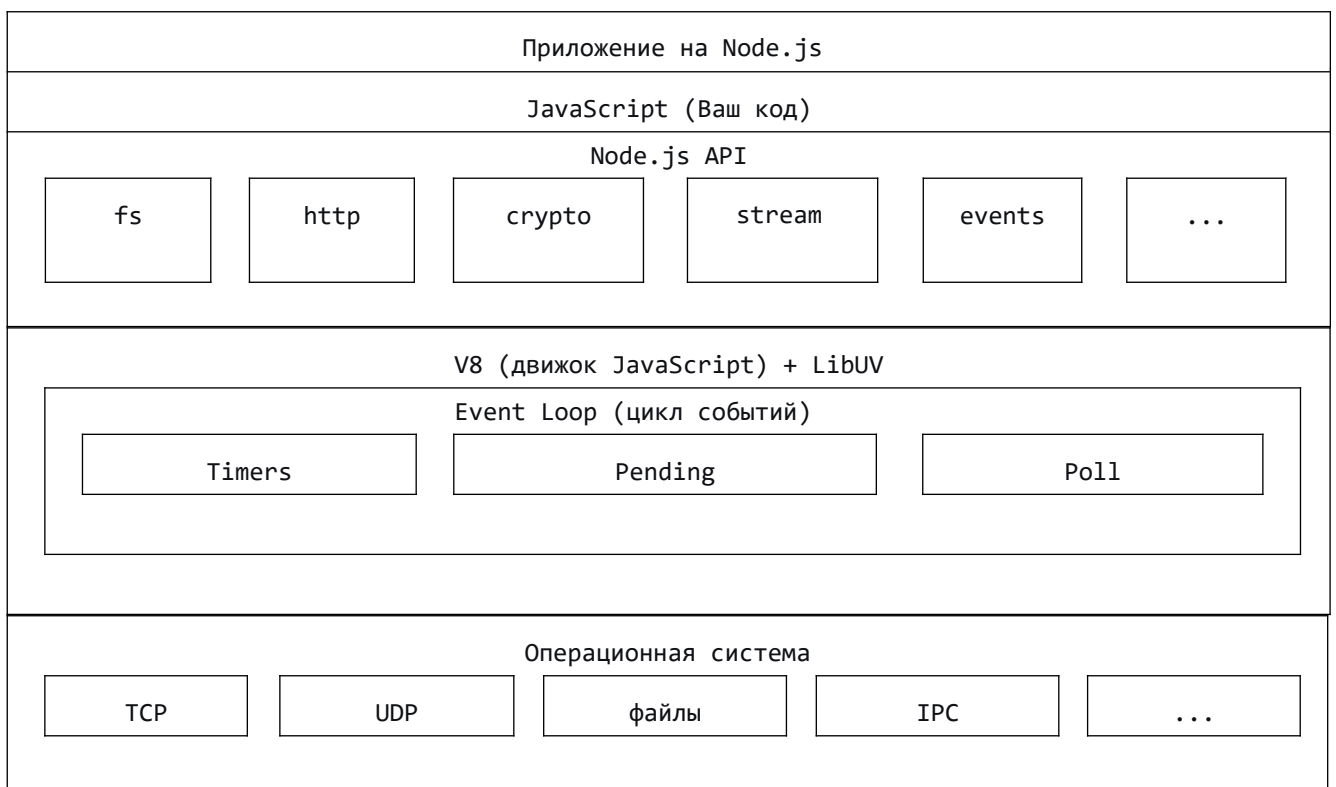
Инструменты для обеспечения совместимости:

1. **Babel** - транпиляция современного JS в старый
2. **Polyfill.io** - автоматическая загрузка нужных полифиллов
3. **Can I Use** - база данных поддержки функций
4. **Autoprefixer** - автоматическое добавление вендорных префиксов CSS

Часть 2: Node.js — JavaScript выходит за пределы браузера

Архитектурная революция: V8 + LibUV

Text



Event Loop: сердце Node.js

Фазы Event Loop:

```
javascript

// 1. Timers (setTimeout, setInterval)
setTimeout(() => {
  console.log('Таймер 1');
}, 0);

// 2. Pending Callbacks (системные операции, ошибки)
fs.readFile('file.txt', (err, data) => {
  // Колбэк попадет в pending callbacks
});

// 3. Poll (новые события, I/O)
// Ожидание новых соединений, чтение файлов

// 4. Check (setImmediate)
setImmediate(() => {
  console.log('Immediate');
});

// 5. Close Callbacks (заккрытие соединений)
server.on('close', () => {
  console.log('Сервер закрыт');
});

// Микрозадачи (Promise, process.nextTick)
Promise.resolve().then(() => {
  console.log('Promise');
});
process.nextTick(() => {
  console.log('Next tick');
});
```

Порядок выполнения:

text

Next tick → Promise → Таймеры → Pending → Poll → Check → Close

Модульная система Node.js: CommonJS vs ES Modules

CommonJS (традиционный для Node.js)

```
javascript

// module.js
module.exports = {
  add: (a, b) => a + b,
  multiply: (a, b) => a * b
};

// Или
exports.add = (a, b) => a + b;

// main.js
const math = require('./module.js');
console.log(math.add(2, 3)); // 5
```

ES Modules (современный стандарт)

```
javascript

// module.mjs (или package.json с "type": "module")
export const add = (a, b) => a + b;
export default function multiply(a, b) {
  return a * b;
}

// main.mjs
import multiply, { add } from './module.mjs';
console.log(add(2, 3)); // 5
```

NPM: экосистема пакетов Node.js

Масштабы экосистемы:

text

npm статистика:

- └─ 1.8+ миллиона пакетов
- └─ 200+ миллиардов загрузок в месяц
- └─ 17+ миллионов разработчиков
- └─ Крупнейший реестр ПО в мире

Структура package.json:

```
json
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Мое приложение",
  "main": "index.js",
  "type": "module", // или "commonjs"
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest",
    "build": "webpack --mode production"
  },
  "dependencies": {
    "express": "^4.18.0",    // Автоматические обновления патчей
    "lodash": "~4.17.21",   // Автоматические обновления минорных версий
    "react": "16.13.1"     // Фиксированная версия
  },
  "devDependencies": {
    "jest": "^28.0.0",
    "webpack": "^5.0.0"
  },
  "engines": {
    "node": ">=14.0.0",
    "npm": ">=6.0.0"
  }
}
```

Архитектурные паттерны Node.js

Non-blocking I/O в действии:

```
javascript

// Блокирующий код (плохо для Node.js)
const fs = require('fs');
const data = fs.readFileSync('large-file.txt'); // Блокирует поток
console.log(data.toString());

// Неблокирующий код (правильно)
const fs = require('fs');
```



```

fs.readFile('large-file.txt', (err, data) => {
  // Колбэк выполнится когда файл прочитан
  // Поток свободен для других операций
  console.log(data.toString());
});
// Здесь можно выполнять другой код
console.log('Это выполнится сразу, не дожидаясь чтения файла');

// Промисы и async/await
const readFile = util.promisify(fs.readFile);

async function processFile() {
  const data = await readFile('file.txt'); // Не блокирует основной поток
  return data.toString().toUpperCase();
}

```

Кластеризация для использования многоядерных систем:

```

javascript

// cluster.js
const cluster = require('cluster');
const os = require('os');

if (cluster.isMaster) {
  // Главный процесс - создаем воркеров
  const cpuCount = os.cpus().length;

  console.log(`Запуск ${cpuCount} воркеров`);
  for (let i = 0; i < cpuCount; i++) {
    cluster.fork(); // Создаем дочерний процесс
  }

  // Обработка событий воркеров
  cluster.on('exit', (worker) => {
    console.log(`Воркер ${worker.id} умер, запускаем нового`);
    cluster.fork();
  });
} else {
  // Код выполняется в каждом воркере
  const express = require('express');
  const app = express();

  app.get('/', (req, res) => {
    // Обработка запроса

```

```
    res.send(`Привет от воркера ${cluster.worker.id}`);
  });

  app.listen(3000, () => {
    console.log(`Воркер ${cluster.worker.id} запущен`);
  });
}
```

Часть 3: Альтернативные среды выполнения

Deno: современная альтернатива Node.js

Ключевые отличия от Node.js:

javascript

```
// 1. Безопасность по умолчанию
// Node.js: полный доступ к системе
// Deno: явное разрешение прав

// Запуск с разрешениями
// deno run --allow-read --allow-net server.ts

// 2. Нативные ES Modules
// Не нужен package.json, импорт по URL
import { serve } from "https://deno.land/std@0.150.0/http/server.ts";

// 3. Встроенные инструменты
// deno fmt      - форматирование
// deno lint     - линтинг
// deno test     - тестирование
// deno bundle   - сборка

// 4. TypeScript из коробки
// Не нужна отдельная компиляция
const greet = (name: string): string => {
  return `Hello, ${name}!`;
};
```

Пример простого сервера на Deno:

typescript

```
// server.ts
import { serve } from "https://deno.land/std@0.150.0/http/server.ts";

const handler = async (request: Request): Promise<Response> => {
  const { pathname } = new URL(request.url);

  if (pathname === "/" ) {
    return new Response("Hello Deno!");
  }

  if (pathname === "/api/data") {
    const data = { message: "Данные из API", timestamp: Date.now() };
    return new Response(JSON.stringify(data), {
      headers: { "Content-Type": "application/json" }
    });
  }

  return new Response("Not Found", { status: 404 });
};

// Запуск: deno run --allow-net server.ts
console.log("Сервер запущен на http://localhost:8000");
await serve(handler, { port: 8000 });
```

Bun: новый игрок с амбициями

Особенности Bun:

javascript

```
// 1. Невероятная скорость (написан на Zig)
// Запуск скриптов в 4 раза быстрее Node.js
// Установка пакетов в 20-100 раз быстрее npm

// 2. Нативная совместимость
// Поддерживает большинство Node.js API
// Работает с package.json и node_modules

// 3. Встроенные инструменты
// bun install    - установка пакетов
// bun run        - запуск скриптов
// bun test       - тестирование
// bun build      - сборка (как webpack)
```

```
// 4. Поддержка TypeScript и JSX из коробки
// Не нужен tsconfig.json
const element = <div>Hello Bun!</div>;
```

Сравнение производительности:

bash

Время запуска простого скрипта

Node.js: ~50ms

Deno: ~30ms

Bun: ~5ms

Установка express

npm: ~10 секунд

yarn: ~8 секунд

bun install: ~0.5 секунд

Специализированные среды выполнения

Cloudflare Workers: JavaScript на грани сети

javascript

// workers.js - выполняется на 200+ датацентрах Cloudflare

```
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request));
});
```

```
async function handleRequest(request) {
```

// Изменение ответов на лету

```
  const response = await fetch(request);
```

```
  const html = await response.text();
```

// Добавляем скрипт на все страницы

```
  const modifiedHtml = html.replace(
    '</body>',
    '<script>console.log("Обработано в Cloudflare Workers")</script></body>'
  );
```

```
  return new Response(modifiedHtml, {
    headers: response.headers
  });
```

```
}
```

AWS Lambda с Node.js: бессерверные функции

javascript

```
// lambda.js - выполняется по событию, оплата за время выполнения
exports.handler = async (event) => {
  console.log('Получено событие:', JSON.stringify(event));

  // Обработка различных триггеров
  if (event.httpMethod === 'GET') {
    return {
      statusCode: 200,
      body: JSON.stringify({ message: 'Hello from Lambda!' })
    };
  }

  if (event.Records) { // S3 событие
    event.Records.forEach(record => {
      console.log('Новый файл в S3:', record.s3.object.key);
    });
  }

  return { statusCode: 200 };
};
```

Часть 4: Кроссплатформенные среды выполнения

Electron: десктоп приложения на JavaScript

javascript

```
// main.js - процесс main
const { app, BrowserWindow, ipcMain } = require('electron');

function createWindow() {
  const win = new BrowserWindow({
    width: 800,
    height: 600,
    webPreferences: {
      nodeIntegration: true, // Доступ к Node.js в renderer
      contextIsolation: false
    }
  });
}
```

```

    }
  });

win.loadFile('index.html');

// IPC коммуникация между процессами
ipcMain.on('message-from-renderer', (event, data) => {
  console.log('Получено из renderer:', data);
  event.reply('response', 'Привет из main процесса!');
});
}

app.whenReady().then(createWindow);
html

<!-- index.html - процесс renderer -->
<!DOCTYPE html>
<html>
<body>
  <h1>Мое Electron приложение</h1>
  <button id="btn">Отправить в main</button>

  <script>
    const { ipcRenderer } = require('electron');

    document.getElementById('btn').addEventListener('click', () => {
      // Отправка сообщения в main процесс
      ipcRenderer.send('message-from-renderer', {
        message: 'Привет из renderer!',
        timestamp: Date.now()
      });

      // Получение ответа
      ipcRenderer.on('response', (event, data) => {
        console.log('Ответ от main:', data);
      });
    });
  </script>
</body>
</html>

```

React Native: мобильные приложения

javascript

```

// App.js - работает и на iOS, и на Android
import React from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

const App = () => {
  const [count, setCount] = React.useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.text}>Счетчик: {count}</Text>
      <Button
        title="Увеличить"
        onPress={() => setCount(count + 1)}
      />
      <Button
        title="Сбросить"
        onPress={() => setCount(0)}
      />
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  text: {
    fontSize: 24,
    marginBottom: 20,
  },
});

export default App;

```

NativeScript: доступ к нативным API

javascript

```

// app.js - прямой доступ к нативным API
const app = require('tns-core-modules/application');
const frame = require('tns-core-modules/ui/frame');

```

```
// Создание нативного интерфейса
const page = frame.topmost().currentPage;
const stackLayout = new StackLayout();

// Нативная кнопка
const button = new Button();
button.text = "Нажми меня";
button.on(Button.tapEvent, () => {
  // Доступ к нативным функциям
  const utils = require('tns-core-modules/utils/utils');
  utils.openUrl('https://nativescript.org');
});

stackLayout.addChild(button);
page.content = stackLayout;
```

Часть 5: Сравнительный анализ сред выполнения

Таблица сравнения ключевых характеристик

Параметр	Браузер	Node.js	Deno	Bun	Electron	React Native
Создатель	Разные компании	Ryan Dahl	Ryan Dahl	Jarred Sumner	GitHub	Facebook
Год выхода	1995	2009	2020	2022	2013	2015
Движок JS	V8/ SpiderMonkey/JSC	V8	V8	JavaScriptCore	V8 (Chromium)	V8/Hermes
Основное назначение	Веб-страницы	Серверные приложения	Безопасные скрипты	Быстрая разработка	Десктоп приложения	Мобильные приложения
Модульная система	ES Modules	CommonJS + ES Modules	ES Modules (по URL)	ES Modules + CommonJS	CommonJS	CommonJS/ES
Пакетный менеджер	-	npm/yarn	Встроенный (URL)	bun install	npm	npm/yarn

Параметр	Браузер	Node.js	Deno	Bun	Electron	React Native
Доступ к файловой системе	Ограниченный	Полный	Только с разрешениями	Полный	Полный	Ограниченный
Доступ к сети	Только к своему origin	Полный	Только с разрешениями	Полный	Полный	Ограниченный
Многопоточность	Web Workers	Worker Threads	Worker API	В разработке	Web Workers	Native Threads
Поддержка TypeScript	Через компиляцию	Через компиляцию	Нативная	Нативная	Через компиляцию	Через компиляцию
Безопасность	Песочница браузера	Полный доступ	Политика разрешений	Полный доступ	Полный доступ	Песочница
Размер runtime	Встроен в браузер	~50MB	~35MB	~15MB	~150MB	Встроен в приложение
Производительность	Высокая	Высокая	Высокая	Очень высокая	Средняя (браузер)	Высокая (нативная)

Критерии выбора среды выполнения

Когда что выбирать:

1. Браузер:

- ➔ Веб-сайты и веб-приложения
- ➔ Интерактивные интерфейсы
- ➔ Проекты, где важна доступность из любого устройства

2. Node.js:

- ➔ Серверные приложения и API
- ➔ Инструменты командной строки
- ➔ Проекты с большой экосистемой npm
- ➔ Когда нужна стабильность и зрелость

3. Deno:

- ➔ Скрипты, требующие безопасности
- ➔ Современные проекты с TypeScript
- ➔ Когда важна безопасность по умолчанию
- ➔ Экспериментальные проекты

4. Bun:

- ➔ Быстрые прототипы
- ➔ Скрипты, где важна скорость
- ➔ Инструменты разработки
- ➔ Когда хочется попробовать новое

5. Electron:

- ➔ Кроссплатформенные десктоп приложения
- ➔ Когда команда знает веб-технологии
- ➔ Приложения, требующие доступ к файловой системе

6. React Native:

- ➔ Кроссплатформенные мобильные приложения
- ➔ Когда команда знает React
- ➔ Быстрая разработка для iOS и Android

Часть 6: Интеграция между средами выполнения

Межпроцессное взаимодействие (IPC)

Electron Main ↔ Renderer:

javascript

// main.js

```
const { ipcMain, BrowserWindow } = require('electron');
```

```
ipcMain.handle('read-file', async (event, filePath) => {  
  const fs = require('fs').promises;  
  try {  
    const data = await fs.readFile(filePath, 'utf-8');  
    return { success: true, data };  
  } catch (error) {  
    return { success: false, error: error.message };  
  }  
});
```

// renderer.js

```

const { ipcRenderer } = require('electron');

async function loadFile() {
  const result = await ipcRenderer.invoke('read-file', '/path/to/file.txt');
  if (result.success) {
    console.log('Содержимое файла:', result.data);
  } else {
    console.error('Ошибка:', result.error);
  }
}

```

Node.js ↔ Браузер через WebSockets:

javascript

```

// server.js (Node.js)
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  console.log('Новое подключение');

  // Отправка данных из Node.js в браузер
  ws.send(JSON.stringify({ type: 'welcome', message: 'Connected!' }));

  // Получение данных из браузера
  ws.on('message', (data) => {
    const message = JSON.parse(data);
    console.log('Получено из браузера:', message);

    // Обработка и ответ
    if (message.type === 'calculate') {
      const result = message.value * 2;
      ws.send(JSON.stringify({ type: 'result', value: result }));
    }
  });
});

// browser.js
const ws = new WebSocket('ws://localhost:8080');

ws.onopen = () => {
  console.log('Подключено к серверу');
  ws.send(JSON.stringify({ type: 'calculate', value: 42 }));
};

```

```
ws.onmessage = (event) => {  
  const data = JSON.parse(event.data);  
  console.log('Получено от сервера:', data);  
};
```

Общие модули между средами

Универсальный JavaScript (Isomorphic JS):

javascript

// shared/utils.js - работает и в браузере, и в Node.js

```
export function formatDate(date) {  
  // Код, который работает везде  
  return new Intl.DateTimeFormat('ru-RU').format(date);  
}
```

```
export function generateId() {  
  // Разные реализации для разных сред  
  if (typeof window !== 'undefined') {  
    // Браузер  
    return crypto.randomUUID();  
  } else {  
    // Node.js  
    const crypto = require('crypto');  
    return crypto.randomUUID();  
  }  
}
```

// Проверка среды выполнения

```
export const isBrowser = typeof window !== 'undefined';  
export const isNode = typeof process !== 'undefined'  
  && process.versions && process.versions.node;
```

Часть 7: Тенденции и будущее экосистемы

Конвергенция сред выполнения

Web API в Node.js:

```
javascript
```

```
// Node.js теперь поддерживает некоторые Web API
```

```
// Fetch API (с Node.js 18)
```

```
const response = await fetch('https://api.example.com/data');  
const data = await response.json();
```

```
// Web Streams API
```

```
const stream = new ReadableStream({  
  start(controller) {  
    controller.enqueue('Hello ');  
    controller.enqueue('World!');  
    controller.close();  
  }  
});
```

```
// Event Target API
```

```
const target = new EventTarget();  
target.addEventListener('custom', (event) => {  
  console.log(event.detail);  
});  
target.dispatchEvent(new CustomEvent('custom', {  
  detail: { message: 'Hello' }  
}));
```

Edge Computing: JavaScript на грани сети

Распределенные среды выполнения:

```
javascript
```

```
// Пример кода, работающего на CDN краях
```

```
// (Cloudflare Workers, Deno Deploy, Vercel Edge Functions)
```

```

export default {
  async fetch(request, env) {
    // Определяем местоположение пользователя
    const country = request.cf.country;
    const city = request.cf.city;

    // Персонализируем ответ
    const html = `
      <!DOCTYPE html>
      <html>
      <body>
        <h1>Привет из ${city}, ${country}!</h1>
        <p>Этот контент был сгенерирован на CDN крае,
          ближайшем к вам.</p>
      </body>
      </html>
    `;

    return new Response(html, {
      headers: { 'Content-Type': 'text/html' }
    });
  }
};

```

WebAssembly как дополнение к JavaScript

Интеграция JavaScript и WASM:

javascript

```

// Загрузка и выполнение WebAssembly модуля
async function loadWasm() {
  // Компилируем Rust/C++/Go в WASM
  const wasmCode = await fetch('module.wasm');
  const wasmBuffer = await wasmCode.arrayBuffer();

  // Инициализация WebAssembly
  const wasmModule = await WebAssembly.compile(wasmBuffer);
  const wasmInstance = await WebAssembly.instantiate(wasmModule, {
    env: {
      // Функции, доступные WASM из JS
      log: (value) => console.log('Из WASM:', value)
    }
  });
}

```

```
});

// Вызов функций из WASM
const result = wasmInstance.exports.compute(42);
console.log('Результат из WASM:', result);

// Передача данных между JS и WASM
const memory = wasmInstance.exports.memory;
const buffer = new Uint8Array(memory.buffer);

// Запись данных в память WASM
const text = "Hello from JavaScript";
const textEncoder = new TextEncoder();
const encoded = textEncoder.encode(text);

// Копирование в память WASM
buffer.set(encoded, 0);

// Вызов функции WASM для обработки
wasmInstance.exports.processString(0, encoded.length);
}
```

Заключение: экосистема как сила JavaScript

Ключевые выводы:

2. **JavaScript эволюционировал** из языка для браузера в универсальную платформу
3. **Разные среды выполнения** решают разные задачи:
 - ➔ Браузеры для интерфейсов
 - ➔ Node.js для серверов
 - ➔ Специализированные среды для конкретных нужд
4. **Экосистема npm** объединяет все среды выполнения
5. **Совместимость между средами** улучшается с каждым годом

Почему такая экосистема является преимуществом:

1. **Переносимость навыков** — знание JavaScript позволяет работать в разных областях
2. **Общие библиотеки** — многие пакеты работают в браузере и Node.js
3. **Единый язык для full-stack** — один язык для фронтенда и бэкенда

4. **Гибкость выбора** — можно выбрать оптимальную среду для задачи

Будущее экосистемы:

1. **Конвергенция API** — Web API появляются в Node.js, Node.js API в браузере
2. **Улучшение производительности** — новые движки и оптимизации
3. **Упрощение разработки** — лучшие инструменты и лучшая совместимость
4. **Расширение областей применения** — от браузера до IoT, от сервера до edge

Итог: Экосистема JavaScript — это не хаос, а **диверсифицированная, взаимосвязанная система платформ**, каждая из которых решает свои задачи, но все они объединены общим языком, общими принципами и общей экосистемой. Это делает JavaScript не просто языком программирования, а **целой платформой для разработки** с беспрецедентным охватом и гибкостью.

1.5. Обзор современных фреймворков и библиотек

Введение: необходимость фреймворков

Современные веб-приложения требуют сложной организации кода. Фреймворки и библиотеки решают проблемы масштабируемости, повторного использования кода и поддержки.

Ключевое различие:

- **Библиотека:** инструменты, которые вы вызываете (React, jQuery)
- **Фреймворк:** структура, которая вызывает ваш код (Angular, Vue)

1. React (Meta/Facebook)

Характеристики:

- **Тип:** Библиотека UI
- **Язык:** JavaScript с JSX
- **Философия:** Декларативные компоненты
- **Размер:** ~6KB (React) + ~2KB (ReactDOM)

Ключевые концепции:

JSX синтаксис:

```
jsx

function Welcome(props) {
  return <h1>Привет, {props.name}</h1>;
}
```

Хуки для состояния:

```
jsx

import { useState, useEffect } from 'react';

function Counter() {
```

```

const [count, setCount] = useState(0);

useEffect(() => {
  document.title = `Счет: ${count}`;
}, [count]);

return (
  <button onClick={() => setCount(count + 1)}>
    Нажато: {count}
  </button>
);
}

```

Экосистема:

- **Роутинг:** React Router
- **Состояние:** Redux, Context API, MobX
- **UI библиотеки:** Material-UI, Ant Design
- **Серверный рендеринг:** Next.js

2. Vue.js

Характеристики:

- **Тип:** Прогрессивный фреймворк
- **Язык:** JavaScript/TypeScript
- **Философия:** Постепенное внедрение
- **Размер:** ~33KB (runtime + compiler)

Ключевые концепции:

Шаблоны с директивами:

```

vue

<template>
  <div>
    <p>{{ message }}</p>
    <button @click="reverseMessage">
      Перевернуть
    </button>
  </div>
</template>

```

```

    <input v-model="username">
  </div>
</template>

<script>
export default {
  data() {
    return {
      message: 'Hello Vue!',
      username: ''
    };
  },
  methods: {
    reverseMessage() {
      this.message = this.message.split('').reverse().join('');
    }
  }
};
</script>

```

Composition API (Vue 3):

```

vue

<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>

```

Экосистема:

- 🟦 **Роутинг:** Vue Router
- 🟦 **Состояние:** Vuex, Pinia
- 🟦 **UI библиотеки:** Vuetify, Element Plus
- 🟦 **Серверный рендеринг:** Nuxt.js

3. Angular (Google)

Характеристики:

- **Тип:** Полноценный фреймворк
- **Язык:** TypeScript
- **Философия:** Батарейки в комплекте
- **Размер:** ~143KB (core)

Ключевые концепции:

Компоненты с TypeScript:

```
typescript

import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `
    <h2>{{ user.name }}</h2>
    <p>Email: {{ user.email }}</p>
  `
})
export class UserComponent {
  user = {
    name: 'Иван Иванов',
    email: 'ivan@example.com'
  };
}
```

Модульная архитектура:

```
typescript

@NgModule({
  declarations: [UserComponent],
  imports: [BrowserModule, HttpClientModule],
  providers: [UserService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Экосистема:

- **Роутинг:** Встроенный Router
- **Состояние:** RxJS, NgRx
- **UI библиотеки:** Angular Material, PrimeNG
- **Серверный рендеринг:** Angular Universal

Сравнительная таблица

Параметр	React	Vue	Angular
Тип	Библиотека UI	Прогрессивный фреймворк	Полный фреймворк
Создатель	Facebook	Эван Ю	Google
Кривая обучения	Средняя	Низкая	Высокая
Привязка данных	Однонаправленная	Двунаправленная	Двунаправленная
Шаблоны	JSX	HTML с директивами	HTML с директивами
CLI	Create React App	Vue CLI / Vite	Angular CLI
Популярность	Очень высокая	Растущая	Стабильная
Рынок труда	Высокий	Растущий	Предпринимательский

4. Мета-фреймворки

Next.js (для React):

- **SSG/SSR:** Статическая и серверная генерация
 - **Файловая маршрутизация:** Автоматическая на основе структуры папок
 - **API Routes:** Создание API в том же проекте
 - **Оптимизация изображений:** Встроенная
- javascript

// Страница с SSG

```
export async function getStaticProps() {  
  const res = await fetch('https://api.example.com/data');  
  const data = await res.json();  
  return { props: { data } };  
}
```

}

Nuxt.js (для Vue):

- **Универсальные приложения:** SSR и статическая генерация
- **Автоматическая маршрутизация:** На основе структуры файлов
- **Модули:** Плагины для расширения функциональности
- **Конфигурация:** Упрощенная настройка

5. Специализированные библиотеки

Управление состоянием:

- **Redux Toolkit** (React): Предиктивное состояние с иммутабельностью
- **Pinia** (Vue 3): Легковесное хранилище с TypeScript поддержкой
- **Zustand** (React): Минималистичное решение

UI библиотеки:

- **Material-UI/Chakra UI:** Готовые компоненты по дизайн-системам
- **Tailwind CSS:** Утилитарный CSS-фреймворк
- **Bootstrap:** Классический CSS-фреймворк

Анимации:

- **Framer Motion** (React): Декларативные анимации
- **GSAP:** Мощная анимационная библиотека
- **Anime.js:** Легковесная библиотека анимаций

6. Современные тренды

Server Components (React 18):

- **Выполнение на сервере:** Уменьшение клиентского кода
- **Прямой доступ к данным:** Без промежуточных API
- **Улучшенная производительность:** Меньше JavaScript для клиента

Islands Architecture:

- **Статические страницы:** С интерактивными "островками"
- **Ленивая загрузка:** JavaScript только для интерактивных частей
- **Astro:** Фреймворк, реализующий эту архитектуру

Микрофронтенды:

- **Разделение приложений:** Независимые части
- **Независимое развертывание:** Каждая часть обновляется отдельно
- **Интеграция:** Module Federation (Webpack 5)

Критерии выбора

Выбирайте React если:

- Нужна максимальная гибкость
- Большое сообщество и ресурсы
- Планируется React Native для мобильных
- Команда знает JavaScript

Выбирайте Vue если:

- Простота обучения важна
- Плавный переход с jQuery
- Хорошая документация на русском
- Баланс между гибкостью и структурой

Выбирайте Angular если:

- Корпоративный проект
- Нужна строгая типизация (TypeScript)
- Полный фреймворк "из коробки"
- Enterprise-поддержка от Google

Для SEO-критичных проектов:

- **React:** Next.js с SSG/SSR
- **Vue:** Nuxt.js с универсальным рендерингом

- **Angular:** Angular Universal
- **Опция:** Astro для максимальной скорости

Стратегии миграции

Постепенное внедрение:

1. **Начните с малого:** Добавьте фреймворк для новых функций
2. **Интеграция:** Встраивайте компоненты в существующий код
3. **Рефакторинг:** Постепенно заменяйте старый код
4. **Полный переход:** Когда команда освоила технологии

Совместимость:

- **Web Components:** Кросс-фреймворковая совместимость
- **Module Federation:** Совместное использование разных фреймворков
- **Iframe/Micro Frontends:** Изоляция разных технологий

Будущие тенденции

1. Улучшение производительности:

- Более быстрая загрузка
- Меньше JavaScript
- Лучшая SEO-оптимизация

2. Упрощение разработки:

- Улучшенные инструменты
- Лучшая документация
- Автоматизация рутинных задач

3. Расширение возможностей:

- WebAssembly интеграция
- 3D и AR/VR поддержка
- Офлайн-возможности (PWA)

Заключение

Современная экосистема JavaScript предлагает решение для любой задачи:

- **Для начинающих:** Vue или React с Create React App
- **Для enterprise:** Angular или React с TypeScript
- **Для SEO:** Next.js, Nuxt.js или Angular Universal
- **Для максимальной скорости:** Astro или Qwik

Ключевые принципы выбора:

1. **Изучаемость:** Насколько быстро команда освоит технологию
2. **Сообщество:** Доступность ресурсов и поддержки
3. **Экосистема:** Наличие необходимых инструментов
4. **Поддержка:** Долгосрочная поддержка технологии

JavaScript фреймворки продолжают эволюционировать, предлагая разработчикам всё более эффективные инструменты для создания современного веба. Выбор правильного инструмента зависит от конкретных требований проекта, опыта команды и долгосрочных целей.

1.6. Инструменты разработчика: браузерные инструменты, редакторы кода, IDE

Введение: эволюция инструментов разработки

Разработка на JavaScript прошла путь от простого текстового редактора и `alert()` для отладки до комплексных интегрированных сред разработки (IDE) и специализированных браузерных инструментов.

Часть 1: Браузерные инструменты разработчика

Исторический контекст:

text

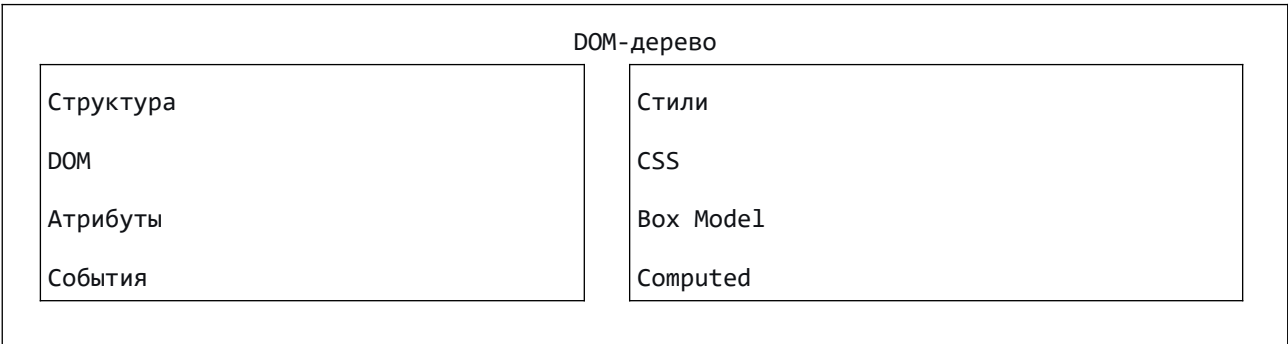
- 1995: `alert()` для отладки
- 2006: Firebug для Firefox (революция)
- 2008: Chrome DevTools (стандарт индустрии)
- 2023: Современные DevTools с AI-помощью

1. Chrome DevTools - золотой стандарт

Основные панели:

1. Elements (Ctrl+Shift+C):

text



Ключевые функции:

- Инспектор элементов
- Редактирование HTML/CSS в реальном времени
- Box Model визуализация
- Эмуляция CSS медиа-запросов

2. Console (Ctrl+Shift+J):

javascript

// Основные команды:

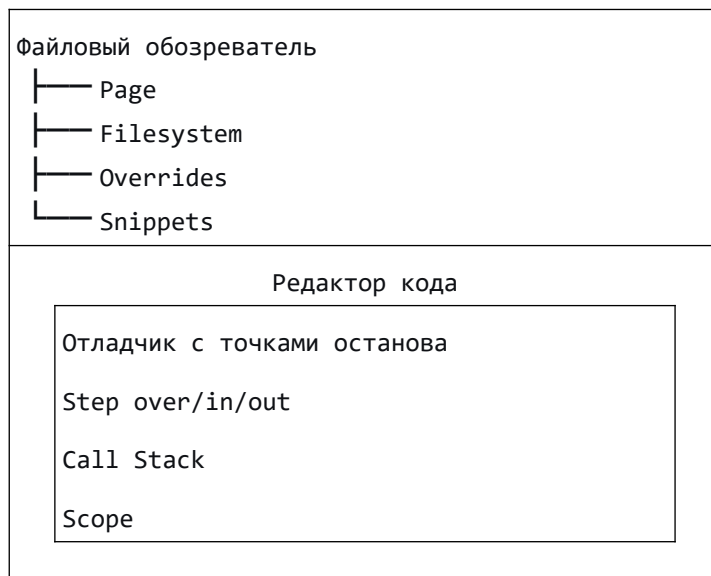
```
console.log('Обычное сообщение'); // Логи
console.error('Ошибка');           // Ошибки
console.warn('Предупреждение');   // Предупреждения
console.table([{id: 1}, {id: 2}]); // Таблица
console.group('Группа');           // Группировка
console.time('Таймер');             // Замер времени
```

// Современные методы:

```
console.assert(x > 0, 'x должен быть > 0');
console.dir(document.body); // Детальный просмотр
console.trace();            // Трассировка стека
```

3. Sources (Ctrl+Shift+I):

text



Отладка JavaScript:

javascript

```
function processData(data) {
  debugger; // Точка останова
}
```

```
const result = data.map(item => {
  // Можно установить точку останова в DevTools
  return transform(item);
});
return result;
}
```

4. Network (Ctrl+Shift+I):

text

Мониторинг запросов			
Name	Method	Status	Time
/api	GET	200	150ms
/img	GET	200	50ms
/ws	WS	101	-

Детали запроса
Headers, Preview, Response
Timing, Initiator, Cookies

Фильтры и возможности:

- Фильтрация по типу (XHR, JS, CSS, Img)
- Throttling (имитация медленного соединения)
- Offline режим
- Block запросов

5. Performance:

text

Анализ производительности		
FPS (кадры в секунду) CPU Utilization Network Heap Memory		
Timeline записи		
0ms	Loading	Parse HTML

50ms	Scripting	Function X
100ms	Rendering	Recalc Style

6. Memory:

- Heap snapshots
- Allocation timeline
- Memory leaks detection

7. Application:

text

Storage
└ Local Storage
└ Session Storage
└ IndexedDB
└ Web SQL
└ Cookies
Cache
└ Cache Storage
└ Application Cache
Background Services
└ Service Workers
└ Manifest
└ Notifications

8. Lighthouse:

text

Performance: 95
Accessibility: 100
Best Practices: 92
SEO: 100
PWA: 85
Рекомендации:
• Уменьшить JavaScript
• Оптимизировать изображения
• Использовать кэширование

Практическое использование DevTools:

1. Быстрое редактирование:

css

```
/* Редактирование CSS в реальном времени */  
.element {  
  /* Ctrl+Shift+C, меняем свойства */  
  color: #ff0000;  
  transform: rotate(15deg);  
  animation: slide 1s ease;  
}
```

2. Отладка сложных сценариев:

javascript

```
// Установка условных точек останова  
function processUser(user) {  
  if (user.age > 18) { // Условие: user.name === 'Admin'  
    // Точка останова срабатывает только для Admin  
    debugger;  
  }  
}  
  
// Live Expressions (постоянное отслеживание)  
// Добавляем в консоль:  
// $0 - выбранный элемент  
// $_ - результат последней операции
```

3. Сетевой анализ:

javascript

```
// Мониторинг API запросов  
// Фильтр: is:from-cache - кэшированные  
// Фильтр: Larger-than:100K - большие файлы  
// Throttling: Slow 3G для тестирования
```

2. Firefox Developer Tools

Уникальные возможности:

1. CSS Grid Inspector:

text

			Визуализация Grid-сетки
A	B	C	Line numbers
D	E	F	Area names
			Track sizes

2. Accessibility Inspector:

text

Контрастность текста: 4.5:1
ARIA атрибуты: label, role
Keyboard navigation
Screen reader simulation

3. JavaScript Debugger:

- Логпоинты (logpoints)
- XHR breakpoints
- Черный список скриптов

3. Safari Web Inspector

Особенности для macOS/iOS:

- Инспектор для Responsive Design
- Графический отладчик WebGL
- Анализ энергии (Energy Impact)

Современные возможности DevTools:

1. Device Mode:

- Эмуляция устройств
- Sensor simulation (геолокация, акселерометр)
- Throttling CPU и сети

2. Command Menu (Ctrl+Shift+P):

text

Доступные команды:

- Show Coverage - покрытие кода
- Capture node screenshot
- Monitor events
- Start recording performance
- Disable JavaScript

3. Workspaces:

- Связь локальных файлов с DevTools
- Сохранение изменений напрямую в файлы
- Hot reload при редактировании

4. Extensions для DevTools:

- React Developer Tools
- Vue.js devtools
- Redux DevTools
- Lighthouse
- JSON Formatter

Часть 2: Редакторы кода

Сравнение популярных редакторов:

Редактор	Размер	Язык	Плюсы	Минусы
VS Code	~300MB	TypeScript	Бесплатный, огромные возможности, интегрированный терминал	Тяжелее чем Sublime
Sublime	~30MB	C++	Быстрый, легкий, мощный поиск	Платный, меньше расширений
Atom	~300MB	JavaScript	Настраиваемый, GitHub интеграция	Медленный, разработка остановлена
Vim	~10MB	C	Консольный, быстрый, мощный	Сложная кривая обучения
WebStorm	~800MB	Java	Полноценная IDE для JavaScript	Платный, тяжелый

1. Visual Studio Code - лидер рынка

Архитектура:

Text

Electron приложение
Monaco Editor (тот же что в Azure DevOps, GitHub Codespaces)
Language Server Protocol TypeScript, JavaScript, CSS
Debug Adapter Protocol Node.js, Chrome, Edge debugging

Основные возможности:

1. Интегрированный терминал:

bash

PowerShell, CMD, bash, zsh
Разделение на панели
Ctrl+` # Открыть/закрыть терминал
Ctrl+Shift+5 # Разделить терминал

2. IntelliSense:

javascript

// Автодополнение кода
`const element = document.querySelector('');`
// После точки появляются варианты:
// • addEventListener()
// • appendChild()
// • attributes
// • ...

// TypeScript интеллектуальное завершение
`interface User {
 id: number;
 name: string;
}`

`const user: User = {
 // Набираем 'i' - предлагает id
 // Набираем 'n' - предлагает name
};`

3. Отладка:

json

// launch.json конфигурация

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Запустить программу",
      "program": "${workspaceFolder}/app.js",
      "skipFiles": ["<node_internals>/**"]
    },
    {
      "type": "chrome",
      "request": "launch",
      "name": "Открыть Chrome",
      "url": "http://localhost:3000",
      "webRoot": "${workspaceFolder}"
    }
  ]
}
```

4. Расширения для JavaScript:

text

Обязательные расширения:

- ESLint - линтинг кода
- Prettier - форматирование
- JavaScript (ES6) code snippets
- npm Intellisense
- Auto Rename Tag
- Bracket Pair Colorizer
- Live Server
- GitLens

5. Интеграция с Git:

text

Source Control (Ctrl+Shift+G)
CHANGES • app.js (Modified)

```
• style.css (Added)
STAGED CHANGES
• index.html
COMMIT MESSAGE
[ ] fix: исправление бага
[ ] feat: новая функция
[ ] docs: обновление документации
```

6. Snippets (фрагменты кода):

json

```
// javascript.json snippets
{
  "Console log": {
    "prefix": "clg",
    "body": "console.log('$1');",
    "description": "Быстрое console.log"
  },
  "React functional component": {
    "prefix": "rfc",
    "body": [
      "import React from 'react';",
      "",
      "function ${1:ComponentName}() {",
      "  return (",
      "    <div>",
      "      ${2}",
      "    </div>",
      "  );",
      "}",
      "",
      "export default ${1:ComponentName};"
    ]
  }
}
```

7. Workspace настройки:

json

```
// .vscode/settings.json
{
  "editor.tabSize": 2,
  "editor.formatOnSave": true,
```

```
"editor.codeActionsOnSave": {
  "source.fixAll.eslint": true
},
"files.autoSave": "afterDelay",
"javascript.updateImportsOnFileMove.enabled": "always",
"emmet.includeLanguages": {
  "javascript": "javascriptreact"
}
}
```

8. Multi-root workspaces:

```
json
// project.code-workspace
{
  "folders": [
    { "path": "frontend" },
    { "path": "backend" },
    { "path": "shared" }
  ],
  "settings": {
    "editor.tabSize": 2
  }
}
```

2. Sublime Text - скорость и эффективность

Уникальные возможности:

1. Multiple Selection:

text

Исходный текст:

```
const a = 1;
const b = 2;
const c = 3;
```

Выделяем все "const" (Ctrl+D):

```
[const] a = 1;
[const] b = 2;
[const] c = 3;
```

Меняем на "let":

```
let a = 1;  
let b = 2;  
let c = 3;
```

2. Goto Anything (Ctrl+P):

text

Команды:

app.js:42 - перейти к строке 42 в app.js

@function - перейти к функциям в файле

#search - поиск по проекту

:100 - перейти к строке 100

3. Command Palette (Ctrl+Shift+P):

- Sort lines
- HTML/JS/CSS beautify
- Convert case
- Remove duplicate lines

4. Package Control для расширений:

text

Популярные пакеты:

- Package Control (менеджер пакетов)
- Emmet (быстрая HTML-разметка)
- Babel (подсветка ES6+)
- GitGutter (изменения Git)
- SideBarEnhancements (улучшения панели)

3. Vim/Neovim - для ценителей клавиатуры

Модальный редактор:

text

Режимы:

- Normal - навигация и команды
- Insert - ввод текста
- Visual - выделение текста
- Command - ввод команд

Пример редактирования:

1. Начало: `console.log('Hello');`
2. ci' - change inside quotes
3. Вводим: `World`
4. Результат: `console.log('World');`

Конфигурация для JavaScript:

```
vim

" .vimrc или init.vim для Neovim
" Плагины через vim-plug
call plug#begin()
Plug 'neoclide/coc.nvim', {'branch': 'release'}
Plug 'prettier/vim-prettier', { 'do': 'npm install' }
Plug 'pangloss/vim-javascript'
Plug 'mxw/vim-jsx'
Plug 'leafgarland/typescript-vim'
call plug#end()

" Настройки для JavaScript
set tabstop=2
set shiftwidth=2
set expandtab
syntax on
filetype plugin indent on
```

Часть 3: Интегрированные среды разработки (IDE)

1. WebStorm (JetBrains)

Преимущества для JavaScript:

1. Умный анализ кода:

```
javascript

// Автоматическое обнаружение проблем
function processData(data) {
  // Подсказка: 'data' is declared but never used
  const unused = 42;

  // Подсказка: Missing return statement
```

```
// if (condition) return true;
}  
  
// Рефакторинг  
// ПКМ → Refactor → Rename (Shift+F6)  
// ПКМ → Refactor → Extract Method (Ctrl+Alt+M)
```

2. Интеграция с инструментами:

text

Встроенные инструменты:

- npm/yarn/pnpm
- ESLint/Prettier
- Jest/Mocha/Karma
- Webpack/Rollup/Vite
- Docker
- Базы данных

3. Мощный отладчик:

javascript

```
// Умные точки останова  
// Условные breakpoints  
// Просмотр значений при наведении  
// Evaluate expression
```

```
async function fetchData() {  
  const response = await fetch('/api/data'); // Breakpoint here  
  const data = await response.json(); // Step into  
  return process(data); // Watch: data.Length  
}
```

4. Встроенный REST клиент:

http

```
// rest-api.http  
GET https://api.example.com/users  
Content-Type: application/json  
  
###  
POST https://api.example.com/users  
{  
  "name": "John",  
  "email": "john@example.com"  
}
```

// Запуск: Ctrl+Enter на строке с запросом

2. IntelliJ IDEA Ultimate

Full-stack возможности:

- Поддержка JavaScript/TypeScript
- Node.js и Deno
- Фреймворки (React, Vue, Angular)
- Базы данных (MongoDB, PostgreSQL)
- Контейнеры (Docker, Kubernetes)

Сравнение IDE vs Редакторы:

Критерий	IDE (WebStorm)	Редактор (VS Code)
Цена	Платный (\$199/год)	Бесплатный
Производительность	Быстрый	Зависит от расширений
Интеграция	Всё включено	Через расширения
Анализ кода	Глубокий	Базовый + расширения
Отладка	Продвинутая	Хорошая
Рефакторинг	Отличный	Хороший
Поддержка БД	Встроенная	Через расширения
Поддержка Docker	Встроенная	Через расширения
Порог входа	Высокий	Низкий

Часть 4: Инструменты командной строки

Node.js инструменты:

1. npm/yarn/pnpm:

```
bash
```


Управление зависимостями

```
npm init -y          # Инициализация
npm install react     # Установка
npm install --save-dev jest  # Dev зависимость
npm update           # Обновление
npm audit             # Проверка безопасности
npm run build         # Запуск скрипта
```

2. npx:

bash

Запуск без установки

```
npx create-react-app my-app
npx eslint . --fix
npx serve ./dist
npx browserslist "> 1%"
```

3. Node.js REPL:

javascript

```
$ node
> const fs = require('fs')
> fs.readdirSync('.')
[ 'app.js', 'package.json' ]
> .editor
// Вводим многострочный код
> .save test.js
> .load test.js
> .exit
```

Сборка и трансформация:

1. Webpack:

javascript

```
// webpack.config.js
module.exports = {
  entry: './src/index.js',
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.js'
  },
  module: {
```

```

rules: [
  {
    test: /\.js$/,
    exclude: /node_modules/,
    use: 'babel-loader'
  },
  {
    test: /\.css$/,
    use: ['style-loader', 'css-loader']
  }
],
plugins: [
  new HtmlWebpackPlugin({
    template: './src/index.html'
  })
]
};

```

2. Vite - современная альтернатива:

bash

Мгновенный запуск

```
npm create vite@latest my-app -- --template react
```

Конфигурация

```

// vite.config.js
export default {
  plugins: [react()],
  server: {
    port: 3000,
    open: true
  },
  build: {
    outDir: 'dist',
    sourcemap: true
  }
}

```

3. Babel - транспилиция:

json

// .babelrc

```
{
```

```
"presets": [  
  ["@babel/preset-env", {  
    "targets": "> 0.25%, not dead"  
  }],  
  "@babel/preset-react"  
],  
"plugins": [  
  "@babel/plugin-proposal-class-properties"  
]  
}
```

Часть 5: Инструменты контроля качества

1. Линтинг: ESLint

```
javascript  
  
// .eslintrc.js  
module.exports = {  
  env: {  
    browser: true,  
    es2021: true  
  },  
  extends: [  
    'eslint:recommended',  
    'plugin:react/recommended'  
  ],  
  rules: {  
    'no-console': 'warn',  
    'react/prop-types': 'off',  
    'indent': ['error', 2],  
    'quotes': ['error', 'single']  
  },  
  settings: {  
    react: {  
      version: 'detect'  
    }  
  }  
};
```

// Использование:

```
npx eslint src/**/*.js --fix
```

2. Форматирование: Prettier

```
json

// .prettierrc
{
  "semi": true,
  "tabWidth": 2,
  "printWidth": 80,
  "singleQuote": true,
  "trailingComma": "es5",
  "bracketSpacing": true,
  "jsxBracketSameLine": false
}

// Интеграция с ESLint
// .eslintrc.js
{
  "extends": ["prettier"],
  "plugins": ["prettier"],
  "rules": {
    "prettier/prettier": "error"
  }
}
```

3. Тестирование: Jest

```
javascript

// sum.test.js
const sum = require('./sum');

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});

// Конфигурация
// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
  moduleNameMapper: {
    '\\.(css|less)$': 'identity-obj-proxy'
  }
};
```

Часть 6: Современные тренды

1. Cloud IDE:

GitHub Codespaces:

text

Преимущества:

- Преднастроенная среда
- Доступ из любого устройства
- Интеграция с GitHub
- Масштабируемые ресурсы

Gitpod:

yaml

.gitpod.yml

tasks:

- `init`: npm install
- `command`: npm run dev

vscode:

extensions:

- dbaeumer.vscode-eslint
- esbenp.prettier-vscode

2. AI помощники:

GitHub Copilot:

javascript

// Автодополнение с AI

```
function calculateDiscount(price, percentage) {  
  // Copilot предлагает:  
  // return price * (1 - percentage / 100);  
}
```

Tabnine / Codeium:

- 🌐 Контекстное автодополнение
- 🌐 Поддержка multiple languages

- Локальная обработка (приватность)

3. Low-code инструменты:

- Webflow для дизайнеров
- Bubble для бизнес-логики
- Retool для внутренних инструментов

Часть 7: Настройка рабочего окружения

Рекомендуемый стек для начинающих:

text

1. Редактор: VS Code
 - ESLint extension
 - Prettier extension
 - Live Server
 - GitLens
2. Браузер: Chrome/Edge
 - React Developer Tools
 - Vue.js devtools
 - Redux DevTools
3. Командная строка:
 - Node.js + npm
 - git
 - nvm (Node Version Manager)
4. Дополнительно:
 - Postman/Insomnia для API
 - Figma для дизайна
 - Notion для документации

Производительные настройки VS Code:

json

```
// settings.json
{
  // Производительность
```

```

"files.exclude": {
  "**/.git": true,
  "**/node_modules": true,
  "**/dist": true
},
"search.exclude": {
  "**/node_modules": true,
  "**/dist": true
},

// JavaScript/TypeScript
"javascript.preferences.importModuleSpecifier": "relative",
"typescript.preferences.importModuleSpecifier": "relative",

// Git
"git.autofetch": true,
"git.confirmSync": false,

// Terminal
"terminal.integrated.fontSize": 14,
"terminal.integrated.cursorBlinking": true
}

```

Скрипты для package.json:

```

json
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "eslint src/**/*.{js,jsx} --fix",
    "format": "prettier --write src/**/*.{js,jsx,json,css}",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  }
}

```

Заключение: эффективная разработка

Принципы эффективной работы:

1. Автоматизируйте рутину:

- ➔ Форматирование при сохранении
- ➔ Линтинг в CI/CD
- ➔ Автоматические тесты

2. Используйте горячие клавиши:

- ➔ Выучите основные комбинации
- ➔ Создайте свои шорткаты
- ➔ Используйте Command Palette

3. Настройте окружение под себя:

- ➔ Темы для глаз
- ➔ Шрифты с лигатурами
- ➔ Оптимальные настройки

4. Следите за производительностью:

- ➔ Профилируйте код
- ➔ Мониторьте память
- ➔ Оптимизируйте сборку

Будущее инструментов:

1. Больше AI интеграции:

- ➔ Умное автодополнение
- ➔ Автоматический рефакторинг
- ➔ Генерация документации

2. Улучшенная коллаборация:

- ➔ Real-time совместное редактирование
- ➔ Интегрированные code reviews
- ➔ Интерактивная документация

3. Упрощение настройки:

- ➔ Zero-config инструменты
- ➔ Автоматическое определение зависимостей
- ➔ Умные дефолтные настройки

Итог: Современные инструменты JavaScript разработки превратили процесс создания веб-приложений из сложной задачи в эффективный и приятный процесс. Правильный выбор и настройка инструментов может увеличить

производительность разработчика в разы, позволяя сосредоточиться на решении бизнес-задач, а не на борьбе с окружением.

Глава 2: Основы программирования

2.1. Алгоритмы и структуры данных: базовые концепции

Введение: почему это важно для JavaScript-разработчика

Алгоритмы и структуры данных — фундамент компьютерной науки. В JavaScript они используются повсеместно:

- Массивы и объекты — структуры данных
- Методы `map/filter/reduce` — алгоритмы
- DOM манипуляции — работа с деревьями
- Оптимизация производительности — выбор правильных структур

Часть 1: Основные понятия

1.1. Алгоритм: определение и свойства

Определение: Алгоритм — последовательность шагов для решения конкретной задачи.

Свойства хорошего алгоритма:

1. **Корректность** — решает задачу правильно
2. **Конечность** — завершается за конечное время
3. **Определенность** — каждый шаг четко определен
4. **Входные данные** — принимает данные
5. **Выходные данные** — производит результат
6. **Эффективность** — использует минимум ресурсов

Пример алгоритма на JavaScript:

```
javascript
```

```
// Алгоритм поиска максимального числа в массиве
```

```
function findMax(arr) {
```

```

// 1. Проверка входных данных
if (!Array.isArray(arr) || arr.length === 0) {
    return null;
}

// 2. Инициализация переменной
let max = arr[0];

// 3. Последовательная проверка элементов
for (let i = 1; i < arr.length; i++) {
    // 4. Сравнение и обновление
    if (arr[i] > max) {
        max = arr[i];
    }
}

// 5. Возврат результата
return max;
}

```

1.2. Сложность алгоритмов: Big O нотация

Что такое Big O: Математическая запись для описания производительности алгоритма.

Основные виды сложности:

javascript

```

// O(1) - Константная сложность
function getFirstElement(arr) {
    return arr[0]; // Всегда 1 операция
}

// O(n) - Линейная сложность
function findElement(arr, target) {
    for (let i = 0; i < arr.length; i++) { // n операций
        if (arr[i] === target) {
            return i;
        }
    }
    return -1;
}

```

```
// O(n²) - Квадратичная сложность
function findPairs(arr) {
  for (let i = 0; i < arr.length; i++) { // n раз
    for (let j = 0; j < arr.length; j++) { // n раз
      console.log(arr[i], arr[j]); // n * n = n²
    }
  }
}
```

```
// O(Log n) - Логарифмическая сложность
function binarySearch(sortedArr, target) {
  let left = 0;
  let right = sortedArr.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (sortedArr[mid] === target) {
      return mid;
    } else if (sortedArr[mid] < target) {
      left = mid + 1; // Игнорируем левую половину
    } else {
      right = mid - 1; // Игнорируем правую половину
    }
  }
  return -1;
}
```

Сравнение сложностей:

text

n = 1000 элементов:

O(1)	→ 1 операция
O(log n)	→ ~10 операций
O(n)	→ 1000 операций
O(n log n)	→ ~10000 операций
O(n²)	→ 1000000 операций
O(2 ⁿ)	→ 1.07e+301 операций (невычислимо)

1.3. Память и пространственная сложность

Пространственная сложность: Сколько памяти требует алгоритм.

javascript

// O(1) - Константная память

```
function sum(arr) {  
  let total = 0; // 1 переменная  
  for (let num of arr) {  
    total += num;  
  }  
  return total;  
}
```

// O(n) - Линейная память

```
function duplicateArray(arr) {  
  const result = []; // Новый массив размером n  
  for (let item of arr) {  
    result.push(item);  
  }  
  return result;  
}
```

// O(n²) - Квадратичная память

```
function createMatrix(n) {  
  const matrix = [];  
  for (let i = 0; i < n; i++) {  
    matrix[i] = [];  
    for (let j = 0; j < n; j++) {  
      matrix[i][j] = i + j; // n * n элементов  
    }  
  }  
  return matrix;  
}
```

Часть 2: Базовые структуры данных в JavaScript

2.1. Массивы (Array)

Характеристики:

- Индексированная коллекция
- Динамический размер
- Быстрый доступ по индексу: O(1)
- Медленные вставка/удаление в начале: O(n)

Операции и их сложность:

javascript

```
const arr = [1, 2, 3, 4, 5];
```

```
// Доступ по индексу:  $O(1)$ 
```

```
console.log(arr[2]); // 3
```

```
// Поиск элемента:  $O(n)$ 
```

```
arr.indexOf(3); // 2
```

```
// Добавление в конец:  $O(1)$  (амортизированно)
```

```
arr.push(6); // [1, 2, 3, 4, 5, 6]
```

```
// Добавление в начало:  $O(n)$ 
```

```
arr.unshift(0); // [0, 1, 2, 3, 4, 5, 6]
```

```
// Удаление с конца:  $O(1)$ 
```

```
arr.pop(); // [0, 1, 2, 3, 4, 5]
```

```
// Удаление с начала:  $O(n)$ 
```

```
arr.shift(); // [1, 2, 3, 4, 5]
```

```
// Вставка в середину:  $O(n)$ 
```

```
arr.splice(2, 0, 2.5); // [1, 2, 2.5, 3, 4, 5]
```

Методы массивов и их сложность:

javascript

```
//  $O(n)$  методы (проходят по всем элементам)
```

```
arr.map(x => x * 2); // Преобразование
```

```
arr.filter(x => x > 2); // Фильтрация
```

```
arr.reduce((a, b) => a + b); // Агрегация
```

```
arr.forEach(x => console.log(x)); // Итерация
```

```
//  $O(n \log n)$  - сортировка
```

```
arr.sort((a, b) => a - b);
```

```
// Проверки:  $O(n)$ 
```

```
arr.includes(3); // true
```

```
arr.some(x => x > 4); // true
```

```
arr.every(x => x > 0); // true
```

```
// Поиск:  $O(n)$ 
```

```
arr.find(x => x > 2);    // 3
arr.findIndex(x => x > 2); // 2
```

2.2. Объекты (Object) и Map

Объекты (хеш-таблицы):

```
javascript

const obj = {
  name: 'John',
  age: 30,
  city: 'New York'
};

// Операции: O(1) в среднем случае
obj.email = 'john@example.com'; // Вставка: O(1)
console.log(obj.name);           // Поиск: O(1)
delete obj.age;                  // Удаление: O(1)

// Проверка существования: O(1)
'name' in obj; // true
obj.hasOwnProperty('city'); // true

// Итерация: O(n)
Object.keys(obj);    // ['name', 'city', 'email']
Object.values(obj);  // ['John', 'New York', 'john@example.com']
Object.entries(obj); // [['name', 'John'], ...]
```

Map vs Object:

```
javascript

// Map - специализированная структура
const map = new Map();

// Преимущества Map:
map.set('key', 'value'); // Любой тип ключа
map.get('key');           // Получение
map.has('key');           // Проверка
map.delete('key');        // Удаление
map.size;                 // Размер
map.clear();              // Очистка

// Итерация Map:
```

```

for (let [key, value] of map) {
  console.log(key, value);
}

map.forEach((value, key) => {
  console.log(key, value);
});

// Особенности Map:
// 1. Сохраняет порядок добавления
// 2. Ключи могут быть любого типа
// 3. Лучшая производительность для частых операций
// 4. Легко получить размер

```

2.3. Set для уникальных значений

```

javascript

const set = new Set([1, 2, 3, 3, 4, 5]);

// Автоматическое удаление дубликатов
console.log(set); // Set(5) {1, 2, 3, 4, 5}

// Основные операции: O(1)
set.add(6);          // Добавление
set.has(3);          // Проверка: true
set.delete(2);       // Удаление
set.size;            // Размер: 5

// Преобразование в массив
const arrFromSet = Array.from(set); // [1, 3, 4, 5, 6]

// Полезные операции с Set
const setA = new Set([1, 2, 3]);
const setB = new Set([2, 3, 4]);

// Объединение
const union = new Set([...setA, ...setB]); // {1, 2, 3, 4}

// Пересечение
const intersection = new Set(
  [...setA].filter(x => setB.has(x))
); // {2, 3}

```



```
// Разность
const difference = new Set(
  [...setA].filter(x => !setB.has(x))
); // {1}
```

2.4. Строки (String)

Строки как массивы символов:

```
javascript

const str = 'Hello, World!';

// Доступ по индексу: O(1)
console.log(str[0]); // 'H'

// Длина: O(1)
console.log(str.length); // 13

// Поиск: O(n)
console.log(str.indexOf('World')); // 7
console.log(str.includes('Hello')); // true

// Методы: большинство O(n)
str.toLowerCase(); // 'hello, world!'
str.toUpperCase(); // 'HELLO, WORLD!'
str.trim(); // Удаление пробелов
str.substring(0, 5); // 'Hello'
str.split(', '); // ['Hello', 'World!']
str.replace('World', 'JavaScript'); // 'Hello, JavaScript!'

// Конкатенация: O(n + m)
const str1 = 'Hello';
const str2 = 'World';
const result = str1 + ', ' + str2 + '!';
```

Часть 3: Базовые алгоритмы на JavaScript

3.1. Алгоритмы поиска

Линейный поиск (Linear Search):

javascript

// O(n) - проверяем каждый элемент

```
function linearSearch(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i; // Найден  
    }  
  }  
  return -1; // Не найден  
}
```

// Улучшенная версия для объектов

```
function findObject(arr, key, value) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i][key] === value) {  
      return i;  
    }  
  }  
  return -1;  
}
```

Бинарный поиск (Binary Search):

javascript

// O(Log n) - работает только с отсортированными массивами

```
function binarySearch(sortedArr, target) {  
  let left = 0;  
  let right = sortedArr.length - 1;  
  
  while (left <= right) {  
    const mid = Math.floor((left + right) / 2);  
  
    if (sortedArr[mid] === target) {  
      return mid; // Найден  
    } else if (sortedArr[mid] < target) {  
      left = mid + 1; // Ищем в правой половине  
    } else {  
      right = mid - 1; // Ищем в левой половине  
    }  
  }  
  
  return -1; // Не найден  
}
```

// Рекурсивная версия

```
function binarySearchRecursive(arr, target, left = 0, right = arr.length - 1) {  
  if (left > right) return -1;  
  
  const mid = Math.floor((left + right) / 2);  
  
  if (arr[mid] === target) {  
    return mid;  
  } else if (arr[mid] < target) {  
    return binarySearchRecursive(arr, target, mid + 1, right);  
  } else {  
    return binarySearchRecursive(arr, target, left, mid - 1);  
  }  
}
```

3.2. Алгоритмы сортировки

Сортировка пузырьком (Bubble Sort):

javascript

// O(n²) - простой, но медленный

```
function bubbleSort(arr) {  
  const n = arr.length;  
  
  for (let i = 0; i < n - 1; i++) {  
    // Флаг оптимизации - если нет обменов, массив отсортирован  
    let swapped = false;  
  
    for (let j = 0; j < n - 1 - i; j++) {  
      if (arr[j] > arr[j + 1]) {  
        // Обмен элементов  
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];  
        swapped = true;  
      }  
    }  
  
    // Если обменов не было - выходим  
    if (!swapped) break;  
  }  
  
  return arr;  
}
```

Сортировка выбором (Selection Sort):

```
javascript

// O(n²) - находит минимальный и ставит на позицию
function selectionSort(arr) {
  const n = arr.length;

  for (let i = 0; i < n - 1; i++) {
    // Находим минимальный элемент в неотсортированной части
    let minIndex = i;

    for (let j = i + 1; j < n; j++) {
      if (arr[j] < arr[minIndex]) {
        minIndex = j;
      }
    }

    // Меняем местами с первым неотсортированным элементом
    if (minIndex !== i) {
      [arr[i], arr[minIndex]] = [arr[minIndex], arr[i]];
    }
  }

  return arr;
}
```

Быстрая сортировка (Quick Sort):

```
javascript

// O(n Log n) в среднем, O(n²) в худшем случае
function quickSort(arr) {
  if (arr.length <= 1) {
    return arr;
  }

  // Выбираем опорный элемент
  const pivot = arr[Math.floor(arr.length / 2)];
  const left = [];
  const right = [];
  const equal = [];

  // Разделяем массив
  for (let element of arr) {
    if (element < pivot) {

```

```

    left.push(element);
  } else if (element > pivot) {
    right.push(element);
  } else {
    equal.push(element);
  }
}

// Рекурсивно сортируем и объединяем
return [...quickSort(left), ...equal, ...quickSort(right)];
}

// Оптимизированная версия на месте (in-place)
function quickSortInPlace(arr, left = 0, right = arr.length - 1) {
  if (left >= right) return;

  const pivotIndex = partition(arr, left, right);

  quickSortInPlace(arr, left, pivotIndex - 1);
  quickSortInPlace(arr, pivotIndex + 1, right);

  return arr;
}

function partition(arr, left, right) {
  const pivot = arr[right];
  let i = left - 1;

  for (let j = left; j < right; j++) {
    if (arr[j] <= pivot) {
      i++;
      [arr[i], arr[j]] = [arr[j], arr[i]];
    }
  }

  [arr[i + 1], arr[right]] = [arr[right], arr[i + 1]];
  return i + 1;
}

```

3.3. Рекурсия

Базовые понятия:

javascript

```
// Рекурсия = функция, вызывающая саму себя
// Обязательно должны быть:
// 1. Базовый случай (когда остановиться)
// 2. Рекурсивный случай (упрощение задачи)

// Пример: вычисление факториала
function factorial(n) {
  // Базовый случай
  if (n <= 1) {
    return 1;
  }
  // Рекурсивный случай
  return n * factorial(n - 1);
}

// Вычисление числа Фибоначчи (медленно  $O(2^n)$ )
function fibonacci(n) {
  if (n <= 1) {
    return n;
  }
  return fibonacci(n - 1) + fibonacci(n - 2);
}

// Оптимизированная версия с мемоизацией
function fibonacciMemo(n, memo = {}) {
  if (n <= 1) return n;

  // Проверяем, вычисляли ли мы уже это значение
  if (memo[n] !== undefined) {
    return memo[n];
  }

  // Вычисляем и сохраняем
  memo[n] = fibonacciMemo(n - 1, memo) + fibonacciMemo(n - 2, memo);
  return memo[n];
}
```

Часть 4: Практическое применение в JavaScript

4.1. Работа с DOM деревом

Обход дерева (Tree Traversal):

```
javascript

// Глубинный обход (Depth-First Search)
function traverseDOM(node, callback) {
  callback(node);

  // Рекурсивно обходим детей
  for (let child of node.children) {
    traverseDOM(child, callback);
  }
}

// Поиск элемента по селектору (упрощенный)
function findElement(root, selector) {
  // Проверяем текущий элемент
  if (root.matches(selector)) {
    return root;
  }

  // Ищем в детях
  for (let child of root.children) {
    const found = findElement(child, selector);
    if (found) {
      return found;
    }
  }

  return null;
}
```

4.2. Оптимизация операций

Кэширование результатов (Memoization):

```
javascript
```

// Кэширование для дорогих вычислений

```
function memoize(fn) {  
  const cache = new Map();  
  
  return function(...args) {  
    const key = JSON.stringify(args);  
  
    if (cache.has(key)) {  
      console.log('Из кэша:', key);  
      return cache.get(key);  
    }  
  
    console.log('Вычисляем:', key);  
    const result = fn(...args);  
    cache.set(key, result);  
  
    return result;  
  };  
}
```

// Пример использования

```
const expensiveCalculation = memoize((a, b) => {  
  console.log('Выполняется сложное вычисление...');  
  return a + b; // Имитация сложной операции  
});  
  
console.log(expensiveCalculation(2, 3)); // Вычисляем  
console.log(expensiveCalculation(2, 3)); // Из кэша
```

Оптимизация циклов:

javascript

// Плохо: вложенные циклы $O(n^2)$

```
function findDuplicatesBad(arr) {  
  const duplicates = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] === arr[j]) {  
        duplicates.push(arr[i]);  
      }  
    }  
  }  
  
  return duplicates;  
}
```



```
}
```

```
// Хорошо: использование Set O(n)
```

```
function findDuplicatesGood(arr) {  
  const seen = new Set();  
  const duplicates = new Set();  
  
  for (let item of arr) {  
    if (seen.has(item)) {  
      duplicates.add(item);  
    } else {  
      seen.add(item);  
    }  
  }  
  
  return Array.from(duplicates);  
}
```

4.3. Работа с данными

Группировка объектов:

javascript

```
// Группировка массива объектов по ключу
```

```
function groupBy(arr, key) {  
  // Используем Map для O(1) доступа  
  const groups = new Map();  
  
  for (let item of arr) {  
    const groupKey = item[key];  
  
    if (!groups.has(groupKey)) {  
      groups.set(groupKey, []);  
    }  
  
    groups.get(groupKey).push(item);  
  }  
  
  return Object.fromEntries(groups);  
}
```

```
// Пример использования
```

```
const users = [
  { id: 1, role: 'admin', name: 'John' },
  { id: 2, role: 'user', name: 'Jane' },
  { id: 3, role: 'admin', name: 'Bob' }
];

console.log(groupBy(users, 'role'));
// {
//   admin: [{id: 1, ...}, {id: 3, ...}],
//   user: [{id: 2, ...}]
// }
```

Часть 5: Подготовка к техническим собеседованиям

5.1. Частые задачи на алгоритмы

Two Sum (найти пару чисел, дающую сумму):

javascript

```
// Наивное решение  $O(n^2)$ 
function twoSumNaive(nums, target) {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      if (nums[i] + nums[j] === target) {
        return [i, j];
      }
    }
  }
  return [];
}
```

```
// Оптимальное решение  $O(n)$  с использованием Map
function twoSum(nums, target) {
  const numMap = new Map();

  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];

    if (numMap.has(complement)) {
      return [numMap.get(complement), i];
    }
  }
}
```

```
    numMap.set(nums[i], i);  
  }  
  
  return [];  
}
```

Palindrome проверка:

javascript

```
// Проверка, является ли строка палиндромом  
function isPalindrome(str) {  
  // Убираем не-буквы и приводим к нижнему регистру  
  const cleanStr = str.toLowerCase().replace(/^[a-z0-9]/g, '');  
  
  // Два указателя: с начала и с конца  
  let left = 0;  
  let right = cleanStr.length - 1;  
  
  while (left < right) {  
    if (cleanStr[left] !== cleanStr[right]) {  
      return false;  
    }  
    left++;  
    right--;  
  }  
  
  return true;  
}
```

5.2. Подготовительные ресурсы

Полезные сайты:

- 🌐 LeetCode - задачи по алгоритмам
- 🌐 HackerRank - практика программирования
- 🌐 Codewars - задачи разной сложности
- 🌐 freeCodeCamp - бесплатное обучение

Книги:

- 🌐 "Грокаем алгоритмы" (Aditya Bhargava)
- 🌐 "Структуры данных и алгоритмы в JavaScript"

Заключение: практические рекомендации

Когда что использовать:

1. Для частого поиска по ключу → Object или Map ($O(1)$)
2. Для уникальных значений → Set
3. Для упорядоченных данных с доступом по индексу → Array
4. Для частых вставок/удалений в начале → Связанный список
5. Для данных с иерархией → Дерево

Правила оптимизации:

1. Избегайте вложенных циклов — они дают $O(n^2)$
2. Используйте Set/Map для проверки существования — $O(1)$ вместо $O(n)$
3. Кэшируйте результаты тяжелых вычислений
4. Выбирайте правильную структуру данных под задачу
5. Пишите читаемый код, потом оптимизируйте при необходимости

Для начинающих:

1. Сначала научитесь решать задачу, потом оптимизируйте
2. Практикуйтесь на простых задачах
3. Анализируйте сложность своих решений
4. Изучайте встроенные методы JavaScript и их сложность

Запомните: В веб-разработке на JavaScript чаще важна читаемость и поддерживаемость кода, чем микро-оптимизации. Но понимание алгоритмов и структур данных помогает писать эффективный код и решать сложные задачи.

2.2. Парадигмы программирования: императивное, декларативное, ООП, функциональное

Введение: что такое парадигма программирования

Парадигма программирования — это стиль или способ написания программ, определяющий:

- Как организован код
- Как обрабатываются данные
- Как выполняются вычисления
- Как структурирована логика

JavaScript — мультипарадигменный язык, поддерживающий все основные парадигмы.

Часть 1: Императивное программирование

1.1. Основная концепция

Императивное программирование описывает **КАК** решить задачу, шаг за шагом.

Аналогия из жизни:

text

Рецепт приготовления яичницы:

1. Возьмите сковороду
2. Поставьте на огонь
3. Добавьте масло
4. Разбейте яйца
5. Жарьте 3 минуты

Характеристики:

- Фокус на процессе
- Использование переменных состояния

- Последовательность команд
- Изменяемое состояние

1.2. Примеры на JavaScript

Базовый пример:

```
javascript

// Императивный подход: описываем КАК
function sumArrayImperative(arr) {
  let total = 0;           // Шаг 1: инициализируем переменную

  for (let i = 0; i < arr.length; i++) { // Шаг 2: создаем цикл
    total += arr[i];       // Шаг 3: добавляем каждый элемент
  }

  return total;           // Шаг 4: возвращаем результат
}

const numbers = [1, 2, 3, 4, 5];
console.log(sumArrayImperative(numbers)); // 15
```

Более сложный пример:

```
javascript

// Императивная фильтрация пользователей
function filterAdultsImperative(users) {
  const result = [];      // Шаг 1: создаем пустой массив

  for (let i = 0; i < users.length; i++) { // Шаг 2: перебираем
    if (users[i].age >= 18) { // Шаг 3: проверяем условие
      result.push(users[i]); // Шаг 4: добавляем подходящих
    }
  }

  return result;          // Шаг 5: возвращаем результат
}

const users = [
  { name: 'Alice', age: 25 },
  { name: 'Bob', age: 17 },
  { name: 'Charlie', age: 30 }
];
```

```
console.log(filterAdultsImperative(users));  
// [{name: 'Alice', age: 25}, {name: 'Charlie', age: 30}]
```

Изменение состояния:

```
javascript  
  
// Типичный императивный код с изменением состояния  
let counter = 0; // Глобальное состояние  
  
function increment() {  
  counter += 1; // Изменяем состояние  
  return counter;  
}  
  
function decrement() {  
  counter -= 1; // Изменяем состояние  
  return counter;  
}  
  
console.log(increment()); // 1  
console.log(increment()); // 2  
console.log(decrement()); // 1  
// Состояние counter было изменено 3 раза
```

1.3. Плюсы и минусы

Плюсы:

- Понятный поток выполнения
- Легко отлаживать (по шагам)
- Эффективное использование памяти
- Полный контроль над процессом

Минусы:

- Много шаблонного кода
- Сложнее поддерживать
- Легко допустить ошибки в состоянии
- Меньшая переиспользуемость кода

Часть 2: Декларативное программирование

2.1. Основная концепция

Декларативное программирование описывает **ЧТО** нужно сделать, а не как.

Аналогия из жизни:

text

Заказ в ресторане:

"Я хочу пиццу Маргарита"

(не говорим повару как именно ее готовить)

Характеристики:

- Фокус на результате
- Минимум деталей реализации
- Использование выражений
- Чистые функции

2.2. Примеры на JavaScript

Тот же пример, но декларативно:

javascript

// Декларативный подход: описываем ЧТО

```
function sumArrayDeclarative(arr) {  
  return arr.reduce((total, num) => total + num, 0);  
  // "Возьми массив и сократи его сложением"  
}
```

```
const numbers = [1, 2, 3, 4, 5];
```

```
console.log(sumArrayDeclarative(numbers)); // 15
```

Фильтрация декларативно:

javascript

// Декларативная фильтрация

```
function filterAdultsDeclarative(users) {  
  return users.filter(user => user.age >= 18);  
}
```



```

    // "Отфильтруй пользователей по возрасту"
  }

const users = [
  { name: 'Alice', age: 25 },
  { name: 'Bob', age: 17 },
  { name: 'Charlie', age: 30 }
];

console.log(filterAdultsDeclarative(users));
// Тот же результат, но код читается как описание

```

HTML как пример декларативности:

```

html

<!-- Декларативный подход в HTML -->
<div class="container">
  <h1>Заголовок</h1>
  <p>Текст параграфа</p>
  <button>Кнопка</button>
</div>

<!-- Мы описываем ЧТО хотим видеть,
      а браузер решает КАК это отобразить -->

```

SQL как декларативный язык:

```

sql

-- Декларативный запрос
SELECT name, age FROM users WHERE age >= 18;
-- "Дай имена и возраст пользователей старше 18"
-- Не говорим КАК искать в базе данных

```

2.3. Современный JavaScript: смешанный подход

Цепочка методов (method chaining):

```

javascript

// Декларативный стиль с цепочками
const transactions = [
  { id: 1, amount: 100, type: 'income' },
  { id: 2, amount: 50, type: 'expense' },
  { id: 3, amount: 200, type: 'income' }
]

```

```
];
```

```
// Читается как предложение:
```

```
const totalIncome = transactions
  .filter(t => t.type === 'income') // "отфильтруй доходы"
  .map(t => t.amount)              // "возьми суммы"
  .reduce((sum, amount) => sum + amount, 0); // "просуммируй"

console.log(totalIncome); // 300
```

2.4. Плюсы и минусы

Плюсы:

- Более читаемый код
- Легче поддерживать
- Меньше побочных эффектов
- Лучшая переиспользуемость
- Проще тестировать

Минусы:

- Меньше контроля над процессом
- Возможны проблемы с производительностью
- Сложнее отлаживать
- Требуется понимания абстракций

Часть 3: Объектно-ориентированное программирование (ООП)

3.1. Основные принципы ООП

Четыре столпа ООП:

1. **Инкапсуляция** — скрытие деталей реализации
2. **Наследование** — переиспользование кода
3. **Полиморфизм** — разные формы одного интерфейса
4. **Абстракция** — упрощение сложных систем

3.2. Классы в JavaScript (ES6+)

Базовый пример класса:

```
javascript

// Инкапсуляция и абстракция
class BankAccount {
  // Приватное поле (инкапсуляция)
  #balance = 0;

  constructor(owner, initialBalance = 0) {
    this.owner = owner;
    this.#balance = initialBalance;
  }

  // Публичные методы (интерфейс)
  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`${this.owner}: +${amount}`);
    }
    return this.#balance;
  }

  withdraw(amount) {
    if (amount > 0 && amount <= this.#balance) {
      this.#balance -= amount;
      console.log(`${this.owner}: -${amount}`);
    }
    return this.#balance;
  }

  getBalance() {
    return this.#balance;
  }
}

// Использование
const account = new BankAccount('Alice', 1000);
account.deposit(500);    // Alice: +500
account.withdraw(200);   // Alice: -200
console.log(account.getBalance()); // 1300
// account.#balance - ошибка: приватное поле
```

3.3. Наследование

javascript

// Базовый класс

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  
  speak() {  
    console.log(`${this.name} издает звук`);  
  }  
  
  eat() {  
    console.log(`${this.name} ест`);  
  }  
}
```

// Наследование

```
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name); // Вызов конструктора родителя  
    this.breed = breed;  
  }  
}
```

// Полиморфизм: переопределение метода

```
speak() {  
  console.log(`${this.name} лает: Гав-гав!`);  
}
```

// Новый метод

```
fetch() {  
  console.log(`${this.name} приносит палку`);  
}  
}
```

```
class Cat extends Animal {  
  speak() {  
    console.log(`${this.name} мяукает: Мяу!`);  
  }  
}
```

// Использование полиморфизма

```
const animals = [  
  new Dog('Бобик', 'Овчарка'),  
  new Cat('Мурка')  
];  
  
animals.forEach(animal => animal.speak());  
// Бобик лает: Гав-гав!  
// Мурка мяукает: Мяу!
```

3.4. Прототипное наследование (до ES6)

javascript

```
// Конструктор функции (старый стиль)  
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
// Методы через прототип  
Person.prototype.greet = function() {  
  console.log(`Привет, я ${this.name}, мне ${this.age} лет`);  
};  
  
// Наследование через прототипы  
function Employee(name, age, position) {  
  Person.call(this, name, age); // "Наследование" свойств  
  this.position = position;  
}  
  
// Наследование прототипа  
Employee.prototype = Object.create(Person.prototype);  
Employee.prototype.constructor = Employee;  
  
// Новый метод  
Employee.prototype.work = function() {  
  console.log(`${this.name} работает как ${this.position}`);  
};  
  
// Использование  
const emp = new Employee('Иван', 30, 'Разработчик');  
emp.greet(); // Привет, я Иван, мне 30 лет  
emp.work(); // Иван работает как Разработчик
```

3.5. Плюсы и минусы ООП в JavaScript

Плюсы:

- Хорошо моделирует реальный мир
- Инкапсуляция защищает данные
- Наследование позволяет переиспользовать код
- Полиморфизм делает код гибким

Минусы:

- Может привести к сложным иерархиям
- В JavaScript нет "настоящих" классов
- Может быть избыточным для простых задач
- Сложнее функционального подхода

Часть 4: Функциональное программирование

4.1. Основные принципы ФП

Ключевые концепции:

1. **Чистые функции** — одинаковый ввод → одинаковый вывод
2. **Иммутабельность** — данные не изменяются
3. **Функции высшего порядка** — функции как аргументы/результат
4. **Рекурсия** вместо циклов
5. **Композиция функций** — объединение простых функций

4.2. Чистые функции

javascript

// Нечистая функция (побочные эффекты)

```
let taxRate = 0.2;
```

```
function calculateTaxImpure(amount) {  
    return amount * taxRate; // Зависит от внешней переменной  
}
```

// Та же функция, но чистая

```
function calculateTaxPure(amount, taxRate) {
```

```

    return amount * taxRate; // Только параметры, всегда один результат
}

// Еще примеры
// Нечистая: изменяет входные данные
function addToArrayImpure(arr, item) {
    arr.push(item); // Мутит оригинальный массив
    return arr;
}

// Чистая: возвращает новый массив
function addToArrayPure(arr, item) {
    return [...arr, item]; // Создает копию
}

const original = [1, 2, 3];
const newArray = addToArrayPure(original, 4);

console.log(original); // [1, 2, 3] - не изменился
console.log(newArray); // [1, 2, 3, 4] - новый массив

```

4.3. Иммутабельность

```

javascript

// Мутабельный подход (изменяемый)
const person = {
    name: 'Alice',
    age: 25
};

person.age = 26; // Прямое изменение

// Иммутабельный подход (неизменяемый)
const updatedPerson = {
    ...person, // Spread оператор для копирования
    age: 26    // Переопределение свойства
};

// Или с Object.assign
const anotherUpdated = Object.assign({}, person, { age: 27 });

console.log(person); // {name: 'Alice', age: 25}
console.log(updatedPerson); // {name: 'Alice', age: 26}

```

4.4. Функции высшего порядка

javascript

// Функция, принимающая другую функцию как аргумент

```
function processNumbers(numbers, processor) {  
  return numbers.map(processor);  
}
```

// Функция, возвращающая функцию

```
function createMultiplier(multiplier) {  
  return function(number) {  
    return number * multiplier;  
  };  
}
```

// Использование

```
const numbers = [1, 2, 3, 4, 5];
```

// Передаем функцию как аргумент

```
const doubled = processNumbers(numbers, n => n * 2);  
console.log(doubled); // [2, 4, 6, 8, 10]
```

// Создаем специализированные функции

```
const double = createMultiplier(2);  
const triple = createMultiplier(3);
```

```
console.log(double(5)); // 10
```

```
console.log(triple(5)); // 15
```

4.5. Каррирование и частичное применение

javascript

// Каррирование: преобразование функции с несколькими аргументами

// в последовательность функций с одним аргументом

// Обычная функция

```
function add(a, b, c) {  
  return a + b + c;  
}
```

// Каррированная версия

```
function curryAdd(a) {  
  return function(b) {
```



```

    return function(c) {
        return a + b + c;
    };
};
}

// Использование
console.log(add(1, 2, 3));          // 6
console.log(curryAdd(1)(2)(3));    // 6

// Частичное применение
const addFive = curryAdd(5);       // Фиксируем первый аргумент
const addFiveAndTwo = addFive(2);  // Фиксируем второй

console.log(addFiveAndTwo(3));      // 10 (5 + 2 + 3)
console.log(addFiveAndTwo(10));     // 17 (5 + 2 + 10)

// Современный JavaScript с arrow functions
const modernCurry = a => b => c => a + b + c;

```

4.6. Композиция функций

javascript

```

// Композиция: объединение простых функций в сложные

// Базовые функции
const toUpperCase = str => str.toUpperCase();
const addExclamation = str => str + '!';
const repeat = str => str + ' ' + str;

// Ручная композиция
const excitedMessage = str => addExclamation(toUpperCase(str));
console.log(excitedMessage('hello')); // HELLO!

// Функция для композиции
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

// Композиция (справа налево)
const composeExample = compose(
    addExclamation,
    toUpperCase,
    repeat
);

```

```

);
console.log(composeExample('hello')); // HELLO HELLO!

// Конвейер (pipe, слева направо)
const pipeExample = pipe(
  repeat,
  toUpperCase,
  addExclamation
);
console.log(pipeExample('hello')); // HELLO HELLO!

```

4.7. Рекурсия в функциональном стиле

```

javascript

// Рекурсия вместо циклов
function sumRecursive(arr) {
  if (arr.length === 0) {
    return 0; // Базовый случай
  }
  return arr[0] + sumRecursive(arr.slice(1)); // Рекурсивный случай
}

console.log(sumRecursive([1, 2, 3, 4])); // 10

// Более эффективно с хвостовой рекурсией
function sumTailRecursive(arr, total = 0) {
  if (arr.length === 0) {
    return total;
  }
  return sumTailRecursive(arr.slice(1), total + arr[0]);
}

```

4.8. Плюсы и минусы ФП в JavaScript

Плюсы:

- Предсказуемость (чистые функции)
- Легче тестировать
- Лучшая переиспользуемость
- Упрощает параллельные вычисления
- Меньше багов из-за состояния

Минусы:

- Кривая обучения
- Может быть менее производительным
- Больше памяти (иммутабельность)
- Не всегда интуитивно понятно
- Не все задачи хорошо ложатся на ФП

Часть 5: Сравнение парадигм

5.1. Таблица сравнения

Парадигма	Фокус	Состояние	Подход	Пример
Императивная	Как сделать	Изменяемое	Пошаговый	Циклы, if-else
Декларативная	Что сделать	Не важно	Результат	SQL, map/filter
ООП	Объекты	Инкапсулированное	Моделирование	Классы, наследование
Функциональная	Функции	Иммутабельное	Преобразование	Чистые функции

5.2. Решение одной задачи разными способами

Задача: Найти сумму квадратов четных чисел в массиве.

```
javascript
```

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
// 1. Императивный подход
```

```
function sumOfEvenSquaresImperative(arr) {  
  let sum = 0;  
  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] % 2 === 0) {  
      sum += arr[i] * arr[i];  
    }  
  }  
}
```

```
    return sum;
}
```

// 2. Декларативный подход

```
function sumOfEvenSquaresDeclarative(arr) {
    return arr
        .filter(n => n % 2 === 0)
        .map(n => n * n)
        .reduce((sum, square) => sum + square, 0);
}
```

// 3. Объектно-ориентированный подход

```
class NumberProcessor {
    constructor(numbers) {
        this.numbers = numbers;
    }

    filterEven() {
        this.numbers = this.numbers.filter(n => n % 2 === 0);
        return this;
    }

    squareAll() {
        this.numbers = this.numbers.map(n => n * n);
        return this;
    }

    sum() {
        return this.numbers.reduce((a, b) => a + b, 0);
    }

    process() {
        return this.filterEven().squareAll().sum();
    }
}
```

// 4. Функциональный подход

```
const isEven = n => n % 2 === 0;
const square = n => n * n;
const add = (a, b) => a + b;

function sumOfEvenSquaresFunctional(arr) {
    return arr
```

```

    .filter(isEven)
    .map(square)
    .reduce(add, 0);
}

// Результат одинаковый: 220
console.log(sumOfEvenSquaresImperative(numbers)); // 220
console.log(sumOfEvenSquaresDeclarative(numbers)); // 220
console.log(new NumberProcessor(numbers).process()); // 220
console.log(sumOfEvenSquaresFunctional(numbers)); // 220

```

Часть 6: Смешивание парадигм в JavaScript

6.1. Гибридные подходы

React компоненты (ООП + ФП):

```

javascript

// Классовый компонент (ООП)
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 }; // Состояние (ООП)
  }

  // Метод класса
  increment = () => {
    this.setState(prevState => ({ // Функциональное обновление
      count: prevState.count + 1
    }));
  }

  render() {
    return (
      <div>
        <p>Счет: {this.state.count}</p>
        <button onClick={this.increment}>
          Увеличить
        </button>
      </div>
    );
  }
}

```

```

    }
  }

  // Функциональный компонент с хуками (ФП)
  function FunctionalCounter() {
    // Хук состояния (императивный механизм, декларативное использование)
    const [count, setCount] = useState(0);

    // Чистая функция обработчика
    const increment = () => {
      setCount(prevCount => prevCount + 1); // Функциональное обновление
    };

    // Декларативный JSX
    return (
      <div>
        <p>Счет: {count}</p>
        <button onClick={increment}>
          Увеличить
        </button>
      </div>
    );
  }

```

6.2. Практические рекомендации

Когда какую парадигму использовать:

1. Императивный подход:

- ➔ Низкоуровневые оптимизации
- ➔ Простые скрипты
- ➔ Когда важен контроль над процессом

2. Декларативный подход:

- ➔ UI разработка (React, Vue)
- ➔ Обработка данных
- ➔ Конфигурационные файлы

3. ООП подход:

- ➔ Сложные бизнес-модели
- ➔ GUI приложения
- ➔ Когда нужна инкапсуляция

4. Функциональный подход:

- ➡ Обработка данных
- ➡ Параллельные вычисления
- ➡ Когда важна предсказуемость

6.3. Современные тенденции

Функциональное программирование набирает популярность:

javascript

// Современный JavaScript часто использует ФП подходы

// Раньше (императивно)

```
const results = [];  
for (let i = 0; i < data.length; i++) {  
  if (data[i].active) {  
    results.push(data[i].name);  
  }  
}
```

// Сейчас (декларативно/функционально)

```
const results = data  
  .filter(item => item.active)  
  .map(item => item.name);
```

// Или даже с использованием паттернов

```
const isActive = item => item.active;  
const getName = item => item.name;  
const activeNames = data.filter(isActive).map(getName);
```

Заключение: выбор парадигмы

Ключевые выводы:

1. **JavaScript поддерживает все парадигмы** — это сила языка
2. **Нет "лучшей" парадигмы** — есть наиболее подходящая для задачи
3. **Смешивайте подходы** — используйте сильные стороны каждой парадигмы
4. **Начинайте с простого** — императивный код часто проще понять новичкам

Рекомендации для изучения:

1. **Начните с императивного** — чтобы понять основы
2. **Изучите декларативный** — для работы с современными фреймворками
3. **Освойте ООП** — для организации сложного кода
4. **Попробуйте ФП** — для улучшения качества кода

Помните:

- **Читаемость** важнее строгого следования парадигме
- **Производительность** зависит от конкретной задачи
- **Команда** должна понимать выбранный подход
- **Инструменты** часто диктуют парадигму (React → декларативный/ФП)

Главное преимущество JavaScript — возможность выбирать и комбинировать парадигмы в зависимости от задачи, что делает его одним из самых гибких и мощных языков программирования.

2.3. Синтаксис JavaScript: основные правила и соглашения

Введение: что такое синтаксис

Синтаксис — это набор правил, определяющих как писать валидный код на JavaScript. Это "грамматика" языка программирования.

Аналогия: Синтаксис JavaScript — как правила русского языка:

- Где ставить запятые (точки с запятой)
- Как строить предложения (операторы)
- Правила склонения (типы данных)
- Порядок слов (последовательность выполнения)

Часть 1: Базовый синтаксис JavaScript

1.1. Структура программы

Минимальная программа:

```
javascript

// Однострочный комментарий
console.log("Hello, World!"); // Завершается точкой с запятой
```

Многострочная программа:

```
javascript

/*
  Многострочный комментарий
  Используется для описания сложной логики
*/

// Объявление переменной
let message = "Привет, мир!";

// Вывод в консоль
console.log(message);
```

```
// Функция
function greet(name) {
  return "Здравствуйте, " + name + "!";
}
```

```
// Вызов функции
console.log(greet("Анна"));
```

1.2. Точки с запятой (;)

Автоматическая вставка (ASI):

javascript

```
// Точки с запятой можно опускать
let x = 10 // ASI добавит ; здесь
let y = 20 // И здесь
```

```
// Но есть опасные случаи:
function example() {
  return // ASI добавит ; здесь, вернет undefined
  {
    message: "Hello" // Этот объект никогда не вернется
  }
}
```

```
console.log(example()); // undefined
```

```
// Правильный вариант:
function correctExample() {
  return {
    message: "Hello"
  };
}
```

```
console.log(correctExample()); // {message: "Hello"}
```

Рекомендации:

javascript

```
// Всегда ставьте точки с запятой:
let a = 1; // ✓ Хорошо
let b = 2 // ✗ Плохо (может вызвать ошибки)
```

```
// Особенно после:
// 1. Выражений присваивания
const name = "John";

// 2. Вызовов функций
console.log("Done");

// 3. Выражений return
return result;

// 4. Выражений continue/break
break;

// 5. Выражений throw
throw new Error("Oops");
```

1.3. Комментарии

Типы комментариев:

javascript

// Однострочный комментарий - для коротких объяснений

```
let price = 100; // Цена в рублях
```

```
/*
  Многострочный комментарий
  Используется для:
  - Длинных описаний
  - Временного отключения кода
  - Документации
*/
```

```
/**
 * JSDoc комментарий
 * @param {string} name - Имя пользователя
 * @param {number} age - Возраст
 * @returns {string} Приветствие
 */
function createGreeting(name, age) {
  return `Привет, ${name}! Тебе ${age} лет.`;
}
```

Правила хороших комментариев:

```
javascript

// ПЛОХО - очевидные комментарии
let i = 0; // Присвоить i значение 0

// ХОРОШО - объясняют "почему", а не "что"
let retryCount = 0; // Счетчик попыток для экспоненциальной backoff стратегии

// ПЛОХО - устаревшие комментарии
// TODO: удалить после релиза (комментарий 2018 года)

// ХОРОШО - актуальные TODO
// TODO: оптимизировать алгоритм сортировки при N > 10000
```

Часть 2: Объявление переменных

2.1. var, let, const

Историческое развитие:

```
javascript

// ES5 (1997-2015) - только var
var oldVariable = "I'm old";

// ES6 (2015) - добавлены let и const
let modernVariable = "I'm modern";
const constantVariable = "I don't change";
```

Различия в области видимости:

```
javascript

// var - функциональная область видимости
function varExample() {
  if (true) {
    var x = 10; // Видна во всей функции
  }
  console.log(x); // 10 - работает
}

// let/const - блочная область видимости
```

```
function letExample() {
  if (true) {
    let y = 20; // Видна только в блоке if
    const z = 30; // Тоже только в блоке
  }
  console.log(y); // Ошибка: y is not defined
  console.log(z); // Ошибка: z is not defined
}
```

Поднятие (hoisting):

javascript

```
// var - поднимается с undefined
console.log(a); // undefined (не ошибка!)
var a = 5;
console.log(a); // 5

// Фактически выполняется так:
var a; // Поднятие объявления
console.log(a); // undefined
a = 5; // Присваивание
console.log(a); // 5

// let/const - поднимаются, но не инициализируются
console.log(b); // Ошибка: Cannot access 'b' before initialization
let b = 10;

// Временная мертвая зона (TDZ)
function tdzExample() {
  // Начало TDZ для c
  console.log(c); // Ошибка

  let c = 15; // Конец TDZ
  console.log(c); // 15
}
```

Повторное объявление:

javascript

```
// var - можно переобъявлять
var x = 1;
var x = 2; // Нет ошибки

// let - нельзя переобъявлять в той же области
```

```

let y = 1;
let y = 2; // Ошибка: Identifier 'y' has already been declared

// Но можно в разных областях
{
  let y = 3; // Другая область видимости
  console.log(y); // 3
}
console.log(y); // 1

// const - нельзя переопределять и изменять
const PI = 3.14;
PI = 3.14159; // Ошибка: Assignment to constant variable
const PI = 3.1416; // Ошибка: Переопределение

```

2.2. Правила именования

Допустимые имена:

```

javascript

// Начинаются с буквы, $ или _
let name = "John"; // ✓ Хорошо
let _private = "secret"; // ✓ Хорошо
let $element = document.getElementById("test"); // ✓ Хорошо
let 1stPlace = "first"; // ✗ Ошибка: начинается с цифры
let my-name = "test"; // ✗ Ошибка: содержит дефис

// Могут содержать буквы, цифры, $, _
let userName = "john_doe"; // ✓
let user123 = "test"; // ✓
let $$$ = "money"; // ✓ (но не рекомендуется)

// Регистр имеет значение
let apple = "fruit";
let Apple = "company";
let APPLE = "brand";

// Это три разные переменные

```

Соглашения по именованию:

```

javascript

// camelCase для переменных и функций
let userName = "JohnDoe"; // ✓

```

```

function calculateTotalPrice() {} // ✓

// PascalCase для классов и конструкторов
class UserAccount {} // ✓
function DatabaseConnection() {} // ✓ (конструктор)

// UPPER_CASE для констант
const MAX_USERS = 100; // ✓
const API_URL = "https://api.example.com"; // ✓

// Префиксы для приватных полей (до ES2022)
class Person {
  constructor(name) {
    this._name = name; // Префикс _ для "приватного"
  }

  getName() {
    return this._name;
  }
}

// Настоящие приватные поля (ES2022+)
class ModernPerson {
  #secret = "confidential"; // Настоящее приватное поле

  reveal() {
    return this.#secret;
  }
}

```

Смысловые имена:

javascript

```

// ПЛОХО - неинформативные имена
let d; // Что это?
let t1, t2; // В чем разница?
let fn; // Функция чего?

// ХОРОШО - описательные имена
let elapsedTimeInDays;
let creationTimestamp;
let modificationTimestamp;
let calculateMonthlyRevenue;

```

Часть 3: Типы данных и литералы

3.1. Примитивные типы

Семь примитивных типов:

javascript

// 1. String - строка

```
let str1 = "Hello"; // Двойные кавычки
let str2 = 'World'; // Одинарные кавычки
let str3 = `Hello ${str2}`; // Шаблонные строки
```

// 2. Number - число

```
let num1 = 42; // Целое
let num2 = 3.14; // Дробное
let num3 = 1e6; // Экспоненциальная запись: 1000000
let num4 = 0xFF; // Шестнадцатеричное: 255
let num5 = 0b1010; // Двоичное: 10
let num6 = 0o777; // Восьмеричное: 511
```

// 3. Boolean - логическое

```
let bool1 = true;
let bool2 = false;
```

// 4. undefined - неопределенное

```
let undef; // undefined
let undef2 = undefined;
```

// 5. null - пустое значение

```
let empty = null;
```

// 6. Symbol - уникальный идентификатор (ES6)

```
let sym1 = Symbol("id");
let sym2 = Symbol("id");
console.log(sym1 === sym2); // false - каждый Symbol уникален
```

// 7. BigInt - большие целые числа (ES2020)

```
let big1 = 1234567890123456789012345678901234567890n;
let big2 = BigInt("12345678901234567890");
```


3.2. Литералы объектов и массивов

Литералы объектов:

```
javascript

// Базовый синтаксис
const person = {
  name: "Alice",      // Свойство с ключом-строкой
  age: 30,            // Значение - число
  "favorite color": "blue", // Ключ в кавычках (с пробелом)

  // Метод (функция как свойство)
  greet: function() {
    return `Hello, I'm ${this.name}`;
  },

  // Сокращенная запись метода (ES6)
  introduce() {
    return `I'm ${this.name}, ${this.age} years old`;
  }
};

// Доступ к свойствам
console.log(person.name);      // "Alice" - точечная нотация
console.log(person["age"]);    // 30 - скобочная нотация
console.log(person["favorite color"]); // "blue"

// Вычисляемые имена свойств (ES6)
const propName = "status";
const user = {
  [propName]: "active",        // status: "active"
  ["user_" + propName]: "user_active" // user_status: "user_active"
};
```

Литералы массивов:

```
javascript

// Создание массива
const numbers = [1, 2, 3, 4, 5];      // Числа
const mixed = [1, "text", true, null]; // Разные типы
const empty = [];                     // Пустой массив

// Доступ к элементам
```

```
console.log(numbers[0]); // 1 - первый элемент (индекс 0)
console.log(numbers[4]); // 5 - последний элемент
console.log(numbers[5]); // undefined - нет такого индекса

// Длина массива
console.log(numbers.length); // 5

// Многомерные массивы
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
console.log(matrix[1][2]); // 6
```

Часть 4: Операторы и выражения

4.1. Арифметические операторы

javascript

```
// Базовые операторы
let a = 10;
let b = 3;

console.log(a + b); // 13 - сложение
console.log(a - b); // 7 - вычитание
console.log(a * b); // 30 - умножение
console.log(a / b); // 3.333... - деление
console.log(a % b); // 1 - остаток от деления
console.log(a ** b); // 1000 - возведение в степень (ES7)

// Инкремент и декремент
let x = 5;
console.log(x++); // 5 (постфиксный: вернуть, потом увеличить)
console.log(x); // 6
console.log(++x); // 7 (префиксный: увеличить, потом вернуть)

let y = 10;
console.log(y--); // 10
console.log(--y); // 8
```

```
// Унарный плюс и минус
let str = "123";
console.log(+str);    // 123 - преобразование в число
console.log(-str);    // -123
```

```
// Сокращенные операторы присваивания
```

```
let n = 5;
n += 3;  // n = n + 3 → 8
n -= 2;  // n = n - 2 → 6
n *= 4;  // n = n * 4 → 24
n /= 3;  // n = n / 3 → 8
n %= 5;  // n = n % 5 → 3
n **= 2; // n = n ** 2 → 9
```

4.2. Операторы сравнения

javascript

```
// Строгое сравнение (рекомендуется)
console.log(5 === 5);    // true - одинаковые тип и значение
console.log(5 === "5");  // false - разные типы
console.log(5 !== "5");  // true
```

```
// Нестрогое сравнение (избегайте)
console.log(5 == 5);     // true
console.log(5 == "5");   // true - неявное преобразование типов
console.log(0 == false); // true - 0 преобразуется в false
console.log("" == false); // true - пустая строка → false
```

```
// Сравнение null и undefined
console.log(null == undefined); // true
console.log(null === undefined); // false - разные типы
```

```
// Реляционные операторы
console.log(5 > 3);    // true
console.log(5 < 3);    // false
console.log(5 >= 5);   // true
console.log(5 <= 4);   // false
```

```
// Сравнение строк (лексикографическое)
console.log("apple" < "banana"); // true - "a" < "b"
console.log("10" < "2");          // true - "1" < "2"
console.log("10" < 2);            // false - "10" преобразуется в 10
```

4.3. Логические операторы

javascript

// Логическое И (&&) - возвращает первое ложное или последнее

```
console.log(true && true);    // true
console.log(true && false);   // false
console.log(false && true);    // false (второе не проверяется)
console.log(0 && "text");     // 0 (ложное значение)
console.log(1 && "text");     // "text" (оба истинные)
```

// Логическое ИЛИ (||) - возвращает первое истинное или последнее

```
console.log(true || false);  // true
console.log(false || true);   // true
console.log(false || false);  // false
console.log(0 || "default");  // "default"
console.log("value" || "default"); // "value"
```

// Логическое НЕ (!)

```
console.log(!true);          // false
console.log(!false);         // true
console.log(!0);             // true
console.log(!"text");        // false
```

// Двойное НЕ (!!) - преобразование к boolean

```
console.log(!!"text");       // true
console.log(!!0);            // false
console.log(!!undefined);    // false
```

// Nullish coalescing (??) - ES2020

// Возвращает правый операнд только если левый null или undefined

```
console.log(null ?? "default");    // "default"
console.log(undefined ?? "default"); // "default"
console.log(0 ?? "default");        // 0 (не null/undefined)
console.log("" ?? "default");       // "" (не null/undefined)
```

// Опциональная цепочка (?.) - ES2020

```
const user = {
  profile: {
    name: "Alice"
  }
};

console.log(user.profile?.name);    // "Alice"
console.log(user.settings?.theme);  // undefined (без ошибки)
```

```
console.log(user.profile?.address?.city); // undefined
```

4.4. Побитовые операторы

javascript

```
// Работа с битами (32-битные целые)
let a = 5;    // 00000000000000000000000000000101
let b = 3;    // 00000000000000000000000000000011

console.log(a & b);    // 1 - И: 000...00000001
console.log(a | b);    // 7 - ИЛИ: 000...00000111
console.log(a ^ b);    // 6 - исключающее ИЛИ: 000...00000110
console.log(~a);       // -6 - НЕ: 111...11111010
console.log(a << 1);   // 10 - сдвиг влево: 000...00001010
console.log(a >> 1);   // 2 - сдвиг вправо: 000...00000010
console.log(a >>> 1);  // 2 - беззнаковый сдвиг вправо
```

4.5. Приоритет операторов

Таблица приоритетов (от высокого к низкому):

javascript

```
// 1. Группировка: ()
(2 + 3) * 4 // 20, а не 14

// 2. Доступ к членам: . []
//    Вызов функции: ()
//    new (с аргументами)
obj.property
arr[0]
func()

// 3. new (без аргументов)
new Date

// 4. Постфиксные: ++ --
let x = 5;
x++ // Сначала используется x, потом увеличивается

// 5. Префиксные: ++ -- ! ~ + - typeof void delete
++x // Сначала увеличивается, потом используется
!true // false
```

```

typeof 42 // "number"

// 6. Возведение в степень: ** (справа налево)
2 ** 3 ** 2 // 2 ** (3 ** 2) = 512, а не 64

// 7. Мультипликативные: * / %
// 8. Аддитивные: + -

// 9. Побитовый сдвиг: << >> >>>

// 10. Отношения: < <= > >= in instanceof

// 11. Равенство: == != === !==

// 12. Побитовое И: &
// 13. Побитовое исключающее И: ^
// 14. Побитовое ИЛИ: |

// 15. Логическое И: &&
// 16. Логическое ИЛИ: ||

// 17. Условный (тернарный): ?: (справа налево)
age >= 18 ? "adult" : "child"

// 18. Присваивание: = += -= и др. (справа налево)
a = b = 5 // b = 5, затем a = b

// 19. Запятая: ,
let x = (1, 2, 3); // x = 3

```

Пример с приоритетами:

javascript

```

let result = 2 + 3 * 4; // 14, не 20 (умножение पहले)
let result2 = (2 + 3) * 4; // 20
let result3 = 2 ** 3 * 4; // 32 (2^3=8, 8*4=32)
let result4 = 16 / 4 / 2; // 2 (слева направо: 16/4=4, 4/2=2)
let result5 = true && false || true; // true (&& पहले ||)

```

Часть 5: Управляющие конструкции

5.1. Условные операторы

if-else:

```
javascript

// Базовый синтаксис
if (condition) {
    // выполняется если condition истинно
}

// С else
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else if (score >= 70) {
    grade = "C";
} else {
    grade = "F";
}

// Фигурные скобки можно опускать для одной инструкции
if (isValid) console.log("Valid");
else console.log("Invalid");

// Но лучше всегда использовать скобки
if (isValid) {
    console.log("Valid");
} else {
    console.log("Invalid");
}
```

switch-case:

```
javascript

// Альтернатива цепочке if-else
let day = 3;
let dayName;

switch (day) {
```

```

case 1:
    dayName = "Понедельник";
    break; // Важно! Без break выполнение "провалится"
case 2:
    dayName = "Вторник";
    break;
case 3:
    dayName = "Среда";
    break;
case 4:
    dayName = "Четверг";
    break;
case 5:
    dayName = "Пятница";
    break;
case 6:
case 7: // Группировка случаев
    dayName = "Выходной";
    break;
default: // Если ни один case не подошел
    dayName = "Неизвестный день";
}

```

// switch с выражениями

```

let score = 85;
let grade;

```

```

switch (true) { // Необычный, но полезный паттерн
    case score >= 90:
        grade = "A";
        break;
    case score >= 80:
        grade = "B";
        break;
    case score >= 70:
        grade = "C";
        break;
    default:
        grade = "F";
}

```

Тернарный оператор:

javascript


```

// Синтаксис: условие ? выражение1 : выражение2
let age = 20;
let status = age >= 18 ? "взрослый" : "ребенок";

// Эквивалентно:
let status;
if (age >= 18) {
    status = "взрослый";
} else {
    status = "ребенок";
}

// Вложенные тернарные операторы (осторожно!)
let message = age >= 18
    ? age >= 65 ? "пенсионер" : "взрослый"
    : "ребенок";

// Использование в выражениях
let discount = isMember ? 0.1 : 0;
let price = 100 * (1 - discount); // 90 для членов, 100 для остальных

```

5.2. Циклы

for loop:

```

javascript

// Классический for
for (let i = 0; i < 5; i++) {
    console.log(i); // 0, 1, 2, 3, 4
}

// Несколько переменных в инициализации
for (let i = 0, j = 10; i < j; i++, j--) {
    console.log(i, j);
    // 0 10, 1 9, 2 8, 3 7, 4 6
}

// Пропуск частей
let k = 0;
for (; k < 3;) { // Без инициализации и инкремента
    console.log(k++); // 0, 1, 2
}

```

```
// Бесконечный цикл (осторожно!)
// for (;;) {
//     // вечный цикл
// }
```

while и do-while:

javascript

```
// while - проверка перед выполнением
```

```
let i = 0;
while (i < 3) {
    console.log(i); // 0, 1, 2
    i++;
}
```

```
// do-while - выполнение перед проверкой
```

```
let j = 0;
do {
    console.log(j); // 0, 1, 2
    j++;
} while (j < 3);
```

```
// Разница: do-while выполнится хотя бы один раз
```

```
let x = 10;
while (x < 5) {
    console.log(x); // Никогда не выполнится
}
```

```
let y = 10;
do {
    console.log(y); // 10 (выполнится один раз)
} while (y < 5);
```

for...in и for...of:

javascript

```
// for...in - для перебора свойств объекта
```

```
const person = { name: "Alice", age: 30 };
for (let key in person) {
    console.log(key, person[key]);
    // name Alice
    // age 30
}
```

// for...of - для перебора итерируемых объектов (ES6)

```
const colors = ["red", "green", "blue"];
for (let color of colors) {
  console.log(color); // red, green, blue
}
```

// Разница:

```
const arr = [10, 20, 30];
arr.customProp = "test";
```

// for...in - все перечисляемые свойства

```
for (let prop in arr) {
  console.log(prop); // "0", "1", "2", "customProp"
}
```

// for...of - только значения элементов

```
for (let value of arr) {
  console.log(value); // 10, 20, 30
}
```

break и continue:

javascript

// break - выход из цикла

```
for (let i = 0; i < 10; i++) {
  if (i === 5) {
    break; // Выход при i = 5
  }
  console.log(i); // 0, 1, 2, 3, 4
}
```

// continue - переход к следующей итерации

```
for (let i = 0; i < 5; i++) {
  if (i === 2) {
    continue; // Пропустить i = 2
  }
  console.log(i); // 0, 1, 3, 4
}
```

// Метки для вложенных циклов

```
outer: for (let i = 0; i < 3; i++) {
  inner: for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
```

```

        break outer; // Выход из обоих циклов
    }
    console.log(i, j);
    // 0 0, 0 1, 0 2, 1 0
}
}

```

Часть 6: Функции

6.1. Объявление функций

Function Declaration:

```

javascript

// Поднимается (hoisted) - можно вызвать до объявления
console.log(square(5)); // 25 (работает!)

function square(x) {
    return x * x;
}

```

Function Expression:

```

javascript

// Не поднимается - нельзя вызвать до присваивания
// console.log(square(5)); // Ошибка!

const square = function(x) {
    return x * x;
};

console.log(square(5)); // 25 (теперь работает)

```

Стрелочные функции (ES6):

```

javascript

// Синтаксис: (параметры) => выражение
const add = (a, b) => a + b;

// Если один параметр - скобки можно опустить
const square = x => x * x;

```

```
// Если нет параметров
const greet = () => "Hello!";

// Если тело функции состоит из нескольких выражений
const multiply = (a, b) => {
  const result = a * b;
  return result;
};

// Отличия от обычных функций:
// 1. Нет своего this (берет из окружающего контекста)
// 2. Нет arguments объекта
// 3. Не могут быть конструкторами (нельзя использовать с new)
// 4. Нет свойства prototype
```

6.2. Параметры и аргументы

Параметры по умолчанию:

```
javascript

// Старый способ (до ES6)
function greet(name) {
  name = name || "Гость"; // || для значений по умолчанию
  return `Привет, ${name}!`;
}

// Новый способ (ES6)
function modernGreet(name = "Гость", greeting = "Привет") {
  return `${greeting}, ${name}!`;
}

console.log(modernGreet()); // "Привет, Гость!"
console.log(modernGreet("Анна")); // "Привет, Анна!"
console.log(modernGreet("Боб", "Здравствуй")); // "Здравствуй, Боб!"

// Параметры по умолчанию вычисляются при вызове
function createUser(name, id = generateId()) {
  return { name, id };
}

function generateId() {
```

```
    return Math.random().toString(36).substr(2, 9);  
}
```

Rest параметры:

javascript

// Сбор всех оставшихся аргументов в массив

```
function sum(...numbers) {  
    return numbers.reduce((total, num) => total + num, 0);  
}
```

```
console.log(sum(1, 2, 3));           // 6  
console.log(sum(1, 2, 3, 4, 5));     // 15
```

// Rest параметр должен быть последним

```
function printInfo(name, age, ...hobbies) {  
    console.log(`Имя: ${name}`);  
    console.log(`Возраст: ${age}`);  
    console.log(`Хобби: ${hobbies.join(", ")}`);  
}
```

```
printInfo("Анна", 25, "чтение", "путешествия", "спорт");
```

Часть 7: Стил ь кодирования и лучшие практики

7.1. Основные соглашения

Отступы и пробелы:

javascript

// Используйте 2 или 4 пробела (не табы)

// ✓ Хорошо - 2 пробела

```
function example() {  
    const x = 1;  
    if (x) {  
        console.log(x);  
    }  
}
```

// ✗ Плохо - смешанные отступы

```
function badExample() {
```

```
const x = 1;
if (x) {
  console.log(x);
}
```

// Пробелы вокруг операторов

```
let x = 1 + 2;      // ✓
let y=3+4;          // ✗
```

// Пробелы после запятых

```
const arr = [1, 2, 3];      // ✓
const obj = {name: "John", age: 30}; // ✗
```

// Пробелы в объявлении функции

```
function foo(a, b) {      // ✓
function bar(a,b){        // ✗
```

Фигурные скобки:

javascript

// Египетские скобки (K&R style)

```
function example() {
  // код
}
```

```
if (condition) {
  // код
} else {
  // код
}
```

// Всегда используйте скобки, даже для одной строки

// ✓ Хорошо

```
if (condition) {
  doSomething();
}
```

// ✗ Плохо

```
if (condition) doSomething();
if (condition)
  doSomething();
```

7.2. ESLint и Prettier

Настройки .eslintrc.js:

```
javascript

module.exports = {
  "env": {
    "browser": true,
    "es2021": true
  },
  "extends": ["eslint:recommended"],
  "rules": {
    "indent": ["error", 2],
    "linebreak-style": ["error", "unix"],
    "quotes": ["error", "single"],
    "semi": ["error", "always"],
    "no-unused-vars": ["warn"],
    "no-console": ["warn"],
    "eqeqeq": ["error", "always"], // Требует === и !==
    "curly": ["error", "all"]      // Требует фигурные скобки
  }
};
```

Настройки .prettierrc:

```
json

{
  "semi": true,
  "tabWidth": 2,
  "printWidth": 80,
  "singleQuote": true,
  "trailingComma": "es5",
  "bracketSpacing": true,
  "arrowParens": "avoid"
}
```

Заключение: ключевые правила

Обязательные правила:

1. Всегда используйте строгое сравнение (`===` и `!==`)

2. Объявляйте переменные с `let` или `const` (не `var`)
3. Всегда ставьте точки с запятой
4. Используйте отступы в 2 пробела
5. Используйте `camelCase` для именования

Рекомендуемые практики:

1. Пишите небольшие функции (до 20 строк)
2. Давайте переменным осмысленные имена
3. Избегайте глобальных переменных
4. Используйте современный синтаксис (ES6+)
5. Комментируйте сложную логику, но не очевидные вещи

Что запомнить:

- **JavaScript прощает многое**, но это не значит, что нужно писать плохой код
- **Согласованный стиль** важнее "идеального" стиля
- **Инструменты** (ESLint, Prettier) помогут соблюдать правила
- **Читаемость** важнее краткости

Синтаксис JavaScript — фундамент, на котором строится все остальное.

Понимание и соблюдение правил синтаксиса сделает ваш код чище, надежнее и проще для понимания другими разработчиками.

2.4. Комментарии и документация кода

Введение: зачем нужны комментарии

Комментарии — это текстовые пояснения в коде, которые игнорируются интерпретатором JavaScript. Они выполняют три ключевые функции:

1. **Объяснение** — что делает код и почему он так написан
2. **Документация** — описание API для других разработчиков
3. **Отладка** — временное отключение кода или добавление заметок

Часть 1: Типы комментариев в JavaScript

1.1. Однострочные комментарии

Синтаксис:

```
javascript
// Это однострочный комментарий
let x = 10; // Комментарий можно ставить после кода

// Комментарии можно использовать
// для многострочных объяснений,
// но это не очень удобно
```

Правильное использование:

```
javascript
// ПЛОХО - очевидные комментарии
let count = 0; // Устанавливаем count в 0
count++;      // Увеличиваем count на 1

// ХОРОШО - объяснение неочевидных вещей
// Используем бинарный поиск вместо линейного для O(Log n) сложности
const index = binarySearch(sortedArray, target);

// ОПАСНО - устаревшие комментарии
// TODO: исправить баг с кодировкой (комментарий от 2018)
```

```
// Вместо этого:  
// TODO: [2024-03-15] Исправить обработку UTF-8 в экспорте CSV
```

Комментарии для отладки:

javascript

```
function processData(data) {  
    // console.log('Начало обработки:', data); // Отладка  
    // debugger; // Точка останова для DevTools  
  
    const result = transform(data);  
  
    // Временная проверка  
    // if (result.length === 0) {  
    //     throw new Error('Пустой результат после transform');  
    // }  
  
    return result;  
}
```

1.2. Многострочные комментарии

Синтаксис:

javascript

```
/*  
    Это многострочный комментарий.  
    Он может занимать несколько строк.  
    Часто используется для:  
    - Длинных описаний  
    - Временного отключения блоков кода  
    - Лицензионных заголовков  
*/  
  
/*  
    * Альтернативный стиль с звездочками  
    * для лучшей читаемости  
    * часто используется в JSDoc  
    */
```

Примеры использования:

javascript

```

/*
  Алгоритм Дейкстры для поиска кратчайшего пути
  Используется для маршрутизации в графе транспорта

  Особенности реализации:
  1. Использует минимальную кучу для оптимизации
  2. Поддерживает направленные и ненаправленные графы
  3. Временная сложность:  $O((V+E) \log V)$ 

  Автор: Иван Петров
  Версия: 2.1
  Дата: 2024-03-15
*/
function dijkstra(graph, start) {
  // реализация...
}

/*
  ВРЕМЕННО ОТКЛЮЧЕНО ДЛЯ ТЕСТИРОВАНИЯ

  Этот блок кода вызывает проблемы с производительностью
  при больших объемах данных. Заменен на оптимизированную
  версию в функции processOptimized().

  Удалить после 2024-06-30, если не понадобится для отката.
*/
/*
function processLegacy(data) {
  // старая реализация...
}
*/

// Актуальный код после отключенного блока
function processOptimized(data) {
  // новая реализация...
}

```

1.3. JSDoc комментарии

Основной синтаксис JSDoc:

javascript

```

/**
 * Краткое описание функции (одна строка)
 *
 * Подробное описание функции. Может занимать
 * несколько строк и объяснять детали реализации,
 * ограничения, примеры использования.
 *
 * @param {Tun} имяПараметра - Описание параметра
 * @param {Tun} [необязательныйПараметр] - Необязательный параметр
 * @returns {TunВозвращаемогоЗначения} - Что возвращает функция
 * @throws {TunОшибки} - Какие ошибки может выбрасывать
 * @example
 * // Пример использования
 * const result = functionName(arg1, arg2);
 * console.log(result); // ожидаемый вывод
 */

```

Полный пример JSDoc:

javascript

```

/**
 * Вычисляет стоимость заказа с учетом скидок и налогов
 *
 * Функция принимает массив товаров, применяет накопительные
 * скидки в зависимости от статуса пользователя, добавляет налог
 * и возвращает итоговую сумму к оплате.
 *
 * @param {Array<Object>} items - Массив товаров в заказе
 * @param {string} items[].id - Уникальный идентификатор товара
 * @param {string} items[].name - Название товара
 * @param {number} items[].price - Цена товара в рублях
 * @param {number} items[].quantity - Количество единиц
 * @param {string} userId - Идентификатор пользователя
 * @param {string} [promoCode] - Необязательный промокод
 * @returns {number} Итоговая стоимость заказа в рублях
 * @throws {Error} Если items пустой или userId невалидный
 * @throws {TypeError} Если items не массив
 * @example
 * // Пример 1: Базовое использование
 * const items = [
 *   {id: '1', name: 'Книга', price: 500, quantity: 2}
 * ];
 * const total = calculateOrderTotal(items, 'user123');
 * console.log(total); // 1000

```

```

*
* @example
* // Пример 2: С промокодом
* const total = calculateOrderTotal(items, 'user123', 'SUMMER2024');
* console.log(total); // 900 (со скидкой 10%)
*/
function calculateOrderTotal(items, userId, promoCode) {
  // Проверка входных данных
  if (!Array.isArray(items)) {
    throw new TypeError('items должен быть массивом');
  }

  if (items.length === 0) {
    throw new Error('Заказ не может быть пустым');
  }

  // Основная логика...
  let subtotal = items.reduce((sum, item) => {
    return sum + (item.price * item.quantity);
  }, 0);

  // Применение скидок...
  // Добавление налогов...

  return Math.round(total * 100) / 100; // Округление до копеек
}

```

Часть 2: Практические паттерны комментирования

2.1. Комментарии для сложной логики

Объяснение алгоритмов:

```

javascript
/**
 * Быстрая сортировка (QuickSort) - рекурсивный алгоритм
 * Временная сложность:  $O(n \log n)$  в среднем,  $O(n^2)$  в худшем
 *
 * @param {number[]} arr - Массив для сортировки
 * @returns {number[]} Отсортированный массив
 */

```

```

function quickSort(arr) {
  // Базовый случай: массивы из 0 или 1 элемента уже отсортированы
  if (arr.length <= 1) {
    return arr;
  }

  // Выбор опорного элемента (pivot)
  // Здесь используется средний элемент для балансировки
  const pivotIndex = Math.floor(arr.length / 2);
  const pivot = arr[pivotIndex];

  // Разделение массива на три части:
  const left = []; // Элементы меньше pivot
  const right = []; // Элементы больше pivot
  const equal = []; // Элементы равные pivot

  // Распределение элементов
  for (let element of arr) {
    if (element < pivot) {
      left.push(element);
    } else if (element > pivot) {
      right.push(element);
    } else {
      equal.push(element);
    }
  }

  // Рекурсивная сортировка левой и правой частей
  // и объединение результатов
  return [...quickSort(left), ...equal, ...quickSort(right)];
}

```

Объяснение оптимизаций:

javascript

```

function findDuplicatesOptimized(arr) {
  /*
   * Оптимизация с использованием Set:
   * - Время:  $O(n)$  вместо  $O(n^2)$  у наивного решения
   * - Память:  $O(n)$  для хранения уникальных значений
   *
   * Принцип работы:
   * 1. Проходим по массиву один раз
   * 2. Если элемент уже видели (есть в seen) - добавляем в duplicates

```

```

    * 3. Иначе - добавляем в seen
    */
    const seen = new Set();
    const duplicates = new Set();

    for (const item of arr) {
        if (seen.has(item)) {
            duplicates.add(item);
        } else {
            seen.add(item);
        }
    }

    return Array.from(duplicates);
}

```

2.2. Комментарии для работы с API

Описание endpoints:

javascript

```

/**
 * Сервис для работы с API пользователей
 * Базовый URL: https://api.example.com/v1/users
 */
class UserService {
    /**
     * Получение информации о пользователе по ID
     *
     * Endpoint: GET /users/:id
     * Требуется аутентификации: Да
     * Права доступа: Только свой профиль или админ
     *
     * @param {string} userId - UUID пользователя
     * @param {string} authToken - JWT токен авторизации
     * @returns {Promise<User>} Объект пользователя
     * @throws {UnauthorizedError} Если токен невалидный
     * @throws {NotFoundError} Если пользователь не найден
     *
     * @example
     * try {

```



```

*   const user = await getUserById('123e4567-e89b-12d3-a456-426614174000',
'jwt.token.here');
*   console.log(user.name); // 'Иван Иванов'
* } catch (error) {
*   console.error('Ошибка:', error.message);
* }
*/

async getUserById(userId, authToken) {
  const response = await fetch(
    `https://api.example.com/v1/users/${userId}`,
    {
      headers: {
        'Authorization': `Bearer ${authToken}`,
        'Content-Type': 'application/json'
      }
    }
  );

  if (!response.ok) {
    if (response.status === 401) {
      throw new UnauthorizedError('Неверный токен авторизации');
    }
    if (response.status === 404) {
      throw new NotFoundError('Пользователь не найден');
    }
    throw new Error(`HTTP error: ${response.status}`);
  }

  return await response.json();
}

/**
 * Создание нового пользователя
 *
 * Endpoint: POST /users
 * Требуется аутентификации: Нет (публичный)
 * Rate limit: 5 запросов в минуту с одного IP
 *
 * @param {Object} userData - Данные нового пользователя
 * @param {string} userData.email - Email (должен быть уникальным)
 * @param {string} userData.password - Пароль (минимум 8 символов)
 * @param {string} userData.name - Имя пользователя
 * @returns {Promise<{userId: string, message: string}>}
 * @throws {ValidationError} Если данные невалидны

```

```

    * @throws {ConflictError} Если email уже используется
    */
    async createUser(userData) {
        // Реализация...
    }
}

```

2.3. Комментарии для конфигурационных файлов

Конфигурация приложения:

```

javascript

/**
 * Конфигурация приложения
 *
 * Все настройки окружения вынесены в этот файл
 * для централизованного управления.
 *
 * Порядок приоритета значений:
 * 1. Переменные окружения (process.env)
 * 2. Значения по умолчанию из этого файла
 * 3. Значения из .env файла (через dotenv)
 */
const config = {
    // Серверные настройки
    server: {
        port: process.env.PORT || 3000,
        host: process.env.HOST || 'localhost',

        // Таймауты в миллисекундах
        timeout: {
            request: 30000, // 30 секунд на обработку запроса
            keepAlive: 5000, // 5 секунд keep-alive
        }
    },

    // Настройки базы данных
    database: {
        // URL подключения к MongoDB
        // Формат: mongodb://user:password@host:port/database
        url: process.env.MONGODB_URI || 'mongodb://localhost:27017/app',
    }
}

```

```
// Настройки пула соединений
poolSize: 10,      // Максимальное количество соединений
reconnectTries: 5, // Количество попыток переподключения
reconnectInterval: 1000, // Интервал между попытками (мс)
},

// Настройки безопасности
security: {
  // Соль для хэширования паролей
  passwordSaltRounds: 12,

  // JWT настройки
  jwt: {
    secret: process.env.JWT_SECRET || 'dev-secret-key',
    expiresIn: '7d', // Время жизни токена: 7 дней

    // Алгоритм подписи
    algorithm: 'HS256',
  },

  // CORS настройки
  cors: {
    // Разрешенные домены
    // В продакшене указывать конкретные домены
    origin: process.env.NODE_ENV === 'production'
      ? ['https://example.com']
      : ['http://localhost:3000'],

    methods: ['GET', 'POST', 'PUT', 'DELETE'],
    allowedHeaders: ['Content-Type', 'Authorization'],
  }
},

// Логирование
logging: {
  // Уровень логирования: error, warn, info, debug
  level: process.env.LOG_LEVEL || 'info',

  // Формат логов
  format: process.env.NODE_ENV === 'production'
    ? 'json' // Для машинной обработки
    : 'pretty', // Для разработки

  // Файлы для логирования
}
```

```

files: {
  error: 'logs/error.log',
  combined: 'logs/combined.log',
}
},

// Функция для проверки конфигурации
validate() {
  const errors = [];

  // Проверка обязательных переменных окружения
  if (!process.env.JWT_SECRET && process.env.NODE_ENV === 'production') {
    errors.push('JWT_SECRET должен быть установлен в production');
  }

  // Проверка валидности порта
  const port = parseInt(this.server.port);
  if (isNaN(port) || port < 1 || port > 65535) {
    errors.push(`Неверный порт: ${this.server.port}`);
  }

  if (errors.length > 0) {
    throw new Error(`Ошибки конфигурации:\n${errors.join('\n')}`);
  }

  return true;
}
};

// Экспорт конфигурации
module.exports = config;

```

Часть 3: Документация для TypeScript

3.1. TypeDoc для TypeScript проектов

Базовый пример:

```

typescript

/**
 * Интерфейс пользователя системы

```

```

*
* @interface User
* @description Основная сущность для работы с пользователями
*/
interface User {
    /**
     * Уникальный идентификатор пользователя
     * @type {string}
     * @example "550e8400-e29b-41d4-a716-446655440000"
     */
    id: string;

    /**
     * Имя пользователя для отображения
     * @type {string}
     * @minLength 2
     * @maxLength 50
     * @example "Иван Иванов"
     */
    name: string;

    /**
     * Email пользователя (должен быть уникальным)
     * @type {string}
     * @format email
     * @example "user@example.com"
     */
    email: string;

    /**
     * Роль пользователя в системе
     * @type {UserRole}
     * @default "user"
     */
    role: UserRole;

    /**
     * Дата создания аккаунта
     * @type {Date}
     * @readonly
     */
    createdAt: Date;

    /**

```

```

    * Дата последнего обновления
    * @type {Date}
    */
    updatedAt: Date;
}

/**
 * Роли пользователей в системе
 * @enum {string}
 */
enum UserRole {
    /**
     * Администратор системы
     * Имеет полный доступ ко всем функциям
     */
    ADMIN = 'admin',

    /**
     * Модератор
     * Может управлять контентом, но не настройками системы
     */
    MODERATOR = 'moderator',

    /**
     * Обычный пользователь
     * Базовые права на использование системы
     */
    USER = 'user',

    /**
     * Гость
     * Ограниченный доступ без возможности изменять данные
     */
    GUEST = 'guest'
}

/**
 * Сервис для управления пользователями
 *
 * @class UserService
 * @description Обеспечивает CRUD операции для пользователей
 *
 * @example
 * ```typescript

```

```

* const userService = new UserService(database);
* const user = await userService.getUserById('123');
* ```
*/
class UserService {
  private database: Database;

  /**
   * Создает экземпляр UserService
   * @constructor
   * @param {Database} database - Экземпляр базы данных
   */
  constructor(database: Database) {
    this.database = database;
  }

  /**
   * Поиск пользователей по фильтру
   *
   * @param {UserFilter} filter - Параметры фильтрации
   * @param {PaginationOptions} [options] - Опции пагинации
   * @returns {Promise<PaginatedResponse<User>>} Пагинированный список пользователей
   *
   * @example
   * ```typescript
   * // Поиск администраторов
   * const filter = { role: UserRole.ADMIN };
   * const options = { page: 1, limit: 10 };
   * const result = await userService.findUsers(filter, options);
   *
   * console.log(result.data); // Массив пользователей
   * console.log(result.total); // Общее количество
   * ```
   */
  async findUsers(
    filter: UserFilter,
    options?: PaginationOptions
  ): Promise<PaginatedResponse<User>> {
    // Реализация...
  }

  /**
   * Создание нового пользователя
   *

```

```

* @param {CreateUserDto} userData - Данные для создания пользователя
* @returns {Promise<User>} Созданный пользователь
* @throws {ValidationError} Если данные невалидны
* @throws {ConflictError} Если email уже используется
*
* @example
* ```typescript
* const userData = {
*   name: 'Новый пользователь',
*   email: 'new@example.com',
*   password: 'securePassword123'
* };
*
* const user = await userService.createUser(userData);
* console.log(user.id); // ID созданного пользователя
* ```
*/
async createUser(userData: CreateUserDto): Promise<User> {
  // Реализация...
}
}

```

3.2. Комментарии для React компонентов

Функциональные компоненты:

jsx

```

/**
* Карточка пользователя для отображения в списке
*
* @component
* @param {UserCardProps} props - Свойства компонента
* @returns {JSX.Element} Карточка пользователя
*
* @example
* ```jsx
* <UserCard
*   user={user}
*   onEdit={handleEdit}
*   onDelete={handleDelete}
*   isSelected={false}
* />

```



```

*   `
*/
const UserCard = ({
  user,
  onEdit,
  onDelete,
  isSelected = false,
  className = ''
}: UserCardProps) => {
  /**
   * Обработчик клика по карточке
   * @param {React.MouseEvent} event - Событие клика
   */
  const handleClick = useCallback((event: React.MouseEvent) => {
    event.stopPropagation();
    onEdit?.(user.id);
  }, [user.id, onEdit]);

  /**
   * Обработчик удаления пользователя
   * @async
   */
  const handleDelete = useCallback(async () => {
    if (window.confirm(`Удалить пользователя ${user.name}?`)) {
      await onDelete?.(user.id);
    }
  }, [user.id, user.name, onDelete]);

  return (
    <div
      className={`user-card ${isSelected ? 'selected' : ''} ${className}`}
      data-testid="user-card"
      onClick={handleClick}
    >
      <Avatar
        src={user.avatar}
        alt={user.name}
        size="medium"
      />
      <div className="user-info">
        <h3 className="user-name">{user.name}</h3>
        <p className="user-email">{user.email}</p>
        <span className={`user-role badge ${user.role}`}>
          {user.role}
        </span>
      </div>
    </div>
  );
}

```

```

        </span>
    </div>
    <button
        className="delete-btn"
        onClick={handleDelete}
        aria-label={`Удалить пользователя ${user.name}`}
    >
        <TrashIcon />
    </button>
</div>
);
};

/**
 * Свойства компонента UserCard
 * @typedef {Object} UserCardProps
 * @property {User} user - Объект пользователя
 * @property {(userId: string) => void} [onEdit] - Колбэк при редактировании
 * @property {(userId: string) => Promise<void>} [onDelete] - Колбэк при удалении
 * @property {boolean} [isSelected] - Выбрана ли карточка
 * @property {string} [className] - Дополнительные CSS классы
 */

// Добавление displayName для DevTools
UserCard.displayName = 'UserCard';

// Экспорт компонента
export default UserCard;

```

Компоненты с хуками:

```

jsx

/**
 * Хук для загрузки данных пользователя
 *
 * @param {string} userId - ID пользователя для загрузки
 * @param {boolean} [enabled=true] - Включена ли загрузка
 * @returns {UseUserResult} Результат загрузки
 *
 * @example
 * ```jsx
 * const { data: user, isLoading, error } = useUser('123');
 *
 * if (isLoading) return <Spinner />;

```

```

* if (error) return <ErrorMessage error={error} />;
*
* return <UserProfile user={user} />;
* ``
*/

const useUser = (userId: string, enabled: boolean = true): UseUserResult => {
  const [data, setData] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<Error | null>(null);

  /**
   * Функция для загрузки пользователя
   * @async
   */
  const fetchUser = useCallback(async () => {
    if (!userId || !enabled) return;

    setIsLoading(true);
    setError(null);

    try {
      const response = await fetch(`/api/users/${userId}`);

      if (!response.ok) {
        throw new Error(`Ошибка HTTP: ${response.status}`);
      }

      const userData = await response.json();
      setData(userData);
    } catch (err) {
      setError(err instanceof Error ? err : new Error('Неизвестная ошибка'));
      setData(null);
    } finally {
      setIsLoading(false);
    }
  }, [userId, enabled]);

  // Загрузка при изменении userId
  useEffect(() => {
    fetchUser();
  }, [fetchUser]);

  /**
   * Функция для повторной загрузки данных

```

```

    */
    const refetch = useCallback(() => {
        fetchUser();
    }, [fetchUser]);

    return {
        data,
        isLoading,
        error,
        refetch
    };
};

/**
 * Результат работы хука useUser
 * @typedef {Object} UseUserResult
 * @property {User | null} data - Данные пользователя
 * @property {boolean} isLoading - Флаг загрузки
 * @property {Error | null} error - Ошибка загрузки
 * @property {() => void} refetch - Функция повторной загрузки
 */

```

Часть 4: Инструменты для документирования

4.1. JSDoc генерация документации

package.json скрипты:

```

json
{
  "scripts": {
    "docs:generate": "jsdoc -c jsdoc.json",
    "docs:serve": "serve docs",
    "docs:watch": "jsdoc -c jsdoc.json --watch"
  },
  "devDependencies": {
    "jsdoc": "^4.0.0",
    "better-docs": "^2.7.2"
  }
}

```

Конфигурация jsdoc.json:

```
json
{
  "tags": {
    "allowUnknownTags": true,
    "dictionaries": ["jsdoc", "closure"]
  },
  "source": {
    "include": ["src"],
    "exclude": ["src/test", "node_modules"],
    "includePattern": ".+\\. (js|jsx|ts|tsx)$",
    "excludePattern": "(^|\\/|\\\\\\)_ "
  },
  "plugins": [
    "plugins/markdown",
    "better-docs/typescript",
    "better-docs/category"
  ],
  "templates": {
    "better-docs": {
      "name": "Документация проекта",
      "title": "Система управления пользователями",
      "navLinks": [
        {
          "label": "GitHub",
          "href": "https://github.com/username/project"
        },
        {
          "label": "API Reference",
          "href": "https://api.example.com/docs"
        }
      ]
    }
  },
  "opts": {
    "destination": "./docs",
    "recurse": true,
    "readme": "./README.md",
    "template": "better-docs",
    "encoding": "utf8"
  },
  "markdown": {
    "hardwrap": false,
```

```
    "idInHeadings": true
  }
}
```

4.2. Storybook для компонентов

Конфигурация .storybook/main.js:

```
javascript
module.exports = {
  stories: [
    '../src/**/*.stories.mdx', // Документация MDX
    '../src/**/*.stories.@(js|jsx|ts|tsx)' // Истории компонентов
  ],
  addons: [
    '@storybook/addon-links',
    '@storybook/addon-essentials',
    '@storybook/addon-interactions',
    '@storybook/addon-docs', // Документация
    '@storybook/addon-a11y', // Доступность
    'storybook-addon-designs' // Дизайн макеты
  ],
  framework: '@storybook/react',
  core: {
    builder: '@storybook/builder-webpack5'
  },
  typescript: {
    check: false,
    checkOptions: {},
    reactDocgen: 'react-docgen-typescript',
    reactDocgenTypescriptOptions: {
      shouldExtractLiteralValuesFromEnum: true,
      propFilter: (prop) => (
        prop.parent
        ? !/node_modules/.test(prop.parent.fileName)
        : true
      ),
    },
  },
};
```

Пример истории компонента:

jsx

```
// Button.stories.jsx
import React from 'react';
import Button from './Button';

/**
 * Компонент Button - многоцветная кнопка с различными вариантами
 *
 * Используется для всех интерактивных действий в интерфейсе.
 * Поддерживает различные размеры, цвета и состояния.
 */
export default {
  title: 'Components/Button',
  component: Button,
  parameters: {
    // Автоматическая генерация документации
    docs: {
      description: {
        component: `
          ## Компонент Button

          Универсальная кнопка для взаимодействия с пользователем.

          ### Особенности:
          - Поддержка различных размеров (small, medium, large)
          - Несколько вариантов цвета (primary, secondary, danger)
          - Состояния disabled и loading
          - Иконки слева и справа
          - Полная доступность (ARIA)
        `,
      },
    },
  },
  // Ссылка на Figma макет
  design: {
    type: 'figma',
    url: 'https://www.figma.com/file/.../Button',
  },
  // Аргументы (props) по умолчанию
  argTypes: {
    variant: {
      description: 'Визуальный вариант кнопки',
      control: {
```

```

    type: 'select',
    options: ['primary', 'secondary', 'danger', 'ghost'],
  },
  table: {
    defaultValue: { summary: 'primary' },
    type: { summary: 'string' },
  },
},
size: {
  description: 'Размер кнопки',
  control: {
    type: 'select',
    options: ['small', 'medium', 'large'],
  },
},
disabled: {
  description: 'Отключено ли состояние',
  control: 'boolean',
},
loading: {
  description: 'Показать индикатор загрузки',
  control: 'boolean',
},
onClick: {
  description: 'Обработчик клика',
  action: 'clicked',
},
},
};

/**
 * Основная кнопка - используется для главных действий
 */

export const Primary = (args) => <Button {...args}>Сохранить</Button>;
Primary.args = {
  variant: 'primary',
};
Primary.parameters = {
  docs: {
    storyDescription: 'Основная кнопка для главных действий на странице',
  },
};

/**

```



```

* Вторичная кнопка - для второстепенных действий
*/
export const Secondary = (args) => <Button {...args}>Отмена</Button>;
Secondary.args = {
  variant: 'secondary',
};

/**
* Опасная кнопка - для деструктивных действий
*/
export const Danger = (args) => <Button {...args}>Удалить</Button>;
Danger.args = {
  variant: 'danger',
};
Danger.parameters = {
  docs: {
    storyDescription: `
      ## Когда использовать:
      - Удаление данных
      - Отмена важных операций
      - Действия с необратимыми последствиями

      ## Когда не использовать:
      - Для обычных действий
      - Для навигации
      - Для положительных операций
    `,
  },
};

/**
* Отключенная кнопка
*/
export const Disabled = (args) => <Button {...args}>Недоступно</Button>;
Disabled.args = {
  disabled: true,
};

/**
* Кнопка с загрузкой
*/
export const Loading = (args) => <Button {...args}>Загрузка...</Button>;

```

```

Loading.args = {
  loading: true,
};

/**
 * Кнопка с иконкой
 */
export const WithIcon = (args) => (
  <Button {...args}>
    <Icon name="download" />
    Скачать отчет
  </Button>
);
WithIcon.args = {
  variant: 'primary',
};

// Группа историй для тестирования
export const AllVariants = () => (
  <div style={{ display: 'flex', gap: '10px', flexWrap: 'wrap' }}>
    <Button variant="primary">Основная</Button>
    <Button variant="secondary">Вторичная</Button>
    <Button variant="danger">Опасная</Button>
    <Button variant="ghost">Призрачная</Button>
    <Button disabled>Отключенная</Button>
    <Button loading>Загрузка</Button>
  </div>
);
AllVariants.parameters = {
  docs: {
    storyDescription: 'Все варианты кнопок для сравнения',
  },
};

```

Часть 5: Лучшие практики и антипаттерны

5.1. Что комментировать, а что нет

Комментируйте ЭТО:

javascript

```

// 1. Сложную бизнес-логику
// Расчет НДС с учетом различных ставок для разных категорий товаров
function calculateVAT(price, productCategory, region) {
    // Логика определения ставки...
}

// 2. Неочевидные оптимизации
// Используем битмап для быстрой проверки наличия элементов
// вместо массива для O(1) сложности
const bitmap = new Uint8Array(1000);

// 3. Временные решения (TODO, FIXME, HACK)
// TODO: Заменить на WebSocket при большом количестве пользователей
// FIXME: Работает только с латиницей, нужно добавить поддержку кириллицы
// HACK: Временное решение для совместимости со старой версией API

// 4. Причины выбора определенного подхода
// Используем MutationObserver вместо setInterval
// для отслеживания изменений DOM с лучшей производительностью
const observer = new MutationObserver(callback);

// 5. Ограничения и предупреждения
// ВНИМАНИЕ: Эта функция изменяет исходный массив
// Вместо этого используйте array.slice() для копирования
function dangerousSort(array) {
    return array.sort(); // Мутит исходный массив
}

```

НЕ комментируйте ЭТО:

javascript

```

// ПЛОХО - очевидные комментарии
let count = 0; // Создаем переменную count и устанавливаем в 0
count++;      // Увеличиваем count на 1

// ПЛОХО - комментарии вместо хороших имен
// Функция для получения пользователей
function get(arr) { // Что такое arr? Каких пользователей?
    return arr[0];  // Почему именно первый элемент?
}

// ХОРОШО - хорошие имена делают комментарии ненужными
function getFirstActiveUser(users) {
    return users.find(user => user.isActive);
}

```

```

}

// ПЛОХО - устаревшие комментарии
// Старая функция, больше не используется
function legacyFunction() {
    // ... код, который все еще используется!
}

// Вместо этого:
function deprecatedFunction() {
    console.warn('deprecatedFunction устарела, используйте newFunction');
    // ... код
}

```

5.2. Форматирование комментариев

Согласованный стиль:

javascript

```

// ХОРОШО - последовательный стиль
/**
 * Краткое описание
 *
 * Подробное описание на нескольких строках
 * с объяснением деталей и примеров.
 *
 * @param {Type} param - Описание параметра
 * @returns {Type} Описание возвращаемого значения
 */

// ХОРОШО - однострочные комментарии выровнены
const MAX_RETRIES = 3;    // Максимальное количество попыток
const TIMEOUT_MS = 5000;  // Таймаут в миллисекундах
const API_URL = 'https://api.example.com'; // Базовый URL API

// ПЛОХО - невыровненные комментарии
const MAX_RETRIES = 3; // Максимальное количество попыток
const TIMEOUT_MS = 5000; // Таймаут в миллисекундах
const API_URL = 'https://api.example.com'; // Базовый URL API

```

5.3. Комментарии для командной работы

Стандартные префиксы:

`javascript`

// TODO: Что нужно сделать

// TODO: [2024-06-30] Добавить поддержку WebSocket

// FIXME: Что нужно исправить

// FIXME: Утечка памяти при частых обновлениях

// HACK: Временное или неоптимальное решение

// HACK: Костыль для совместимости со старым API

// OPTIMIZE: Что можно оптимизировать

// OPTIMIZE: Заменить линейный поиск на бинарный

// REVIEW: Что нужно проверить

// REVIEW: Проверить безопасность этого решения

// NOTE: Важное примечание

// NOTE: Этот код зависит от внешней библиотеки

// WARNING: Предупреждение

// WARNING: Не использовать в production без тестирования

Заключение: эффективное документирование

Ключевые принципы:

- 1. Пишите код, который не требует комментариев** — используйте понятные имена
- 2. Комментируйте "почему", а не "что"** — объясняйте решения, а не синтаксис
- 3. Держите комментарии в актуальном состоянии** — устаревшие комментарии хуже их отсутствия
- 4. Используйте стандарты** — JSDoc для API, Storybook для компонентов
- 5. Автоматизируйте** — настройте генерацию документации

Проверочный список для ревью кода:

- Все публичные функции имеют JSDoc комментарии
- Сложная логика объяснена комментариями
- Нет очевидных комментариев (которые дублируют код)
- Все TODO/FIXME имеют даты или ответственных
- Комментарии соответствуют стилю команды
- Примеры использования в документации актуальны
- Типы параметров и возвращаемых значений указаны

Инструменты для проверки:

bash

Проверка качества комментариев

npm run lint:comments

Генерация документации

npm run docs:generate

Запуск Storybook для просмотра компонентов

npm run storybook

Проверка покрытия документацией

npm run docs:coverage

Помните: Хорошо задокументированный код — это инвестиция в будущее проекта. Он упрощает onboarding новых разработчиков, снижает количество ошибок и делает поддержку более эффективной. Ком

Введение: важность правильной обработки ошибок

Отладка и обработка ошибок — критически важные навыки JavaScript-разработчика. Без них приложения становятся нестабильными, сложными в поддержке и опасными для пользователей.

Часть 1: Основы отладки в JavaScript

1.1. Методы console для отладки

Базовые методы console:

```
javascript

// 1. console.log() - базовый вывод
console.log('Простое сообщение:', variable);

// 2. console.error() - вывод ошибок (красный цвет)
console.error('Произошла ошибка:', error);

// 3. console.warn() - предупреждения (желтый цвет)
console.warn('Это устаревший метод, используйте новый');

// 4. console.info() - информационные сообщения
console.info('Приложение успешно запущено');

// 5. console.debug() - отладочные сообщения
console.debug('Текущее состояние:', state);
```

Расширенные методы console:

```
javascript

// 1. console.table() - табличное представление
const users = [
  { id: 1, name: 'Alice', age: 25 },
  { id: 2, name: 'Bob', age: 30 }
```

```

];
console.table(users);
// Выводит:
//
// | (index) | id | name | age |
// |-----|----|-----|-----|
// | 0      | 1  | Alice | 25   |
// | 1      | 2  | Bob   | 30   |
// |-----|----|-----|-----|

// 2. console.group() / console.groupEnd() - группировка
console.group('Детали пользователя');
console.log('Имя:', user.name);
console.log('Email:', user.email);
console.groupEnd();

// 3. console.time() / console.timeEnd() - замер времени
console.time('Время выполнения');
// Код для замера
console.timeEnd('Время выполнения'); // Выводит: Время выполнения: 2.5ms

// 4. console.trace() - трассировка стека вызовов
function outer() {
    middle();
}
function middle() {
    inner();
}
function inner() {
    console.trace('Трассировка стека');
}
outer();

// 5. console.assert() - проверка утверждений
console.assert(x > 0, 'x должен быть положительным');
// Если x <= 0, выводит ошибку с сообщением

// 6. console.dir() - детализированный вывод объекта
console.dir(document.body, { depth: 2 });

// 7. console.count() / console.countReset() - счетчик
for (let i = 0; i < 3; i++) {
    console.count('Итерация цикла');
    // Выводит: Итерация цикла: 1, Итерация цикла: 2...

```



```
}  
console.countReset('Итерация цикла');
```

1.2. Ключевое слово debugger

javascript

```
function processData(data) {  
    // Установка точки останова в коде  
    debugger; // Выполнение остановится здесь, если открыты DevTools  
  
    let result = 0;  
  
    for (let i = 0; i < data.length; i++) {  
        // Условная точка останова  
        if (data[i] === null) {  
            debugger; // Остановится только при null  
        }  
        result += data[i];  
    }  
  
    return result;  
}  
  
// Использование с условиями  
function validateUser(user) {  
    // Точка останова сработает только для администраторов  
    if (user.role === 'admin') {  
        debugger;  
    }  
  
    // ... валидация  
}
```

1.3. Стилизация console.log

javascript

```
// Базовое форматирование  
console.log('%s имеет %d лет', 'Анна', 25);  
// %s - строка, %d - число, %i - целое, %f - дробное, %o - объект  
  
// CSS стилизация  
console.log(  

```

```

    '%сВажное сообщение!',
    'color: white; background: red; font-size: 20px; padding: 10px;'
);

// Несколько стилей
console.log(
    '%сОшибка: %с%s',
    'color: red; font-weight: bold;',
    'color: black;',
    errorMessage
);

// Таблица с кастомными стилями
console.table([
    { status: 'Успех', color: 'green' },
    { status: 'Ошибка', color: 'red' }
]);

// Градиентный текст
console.log(
    '%сКрасивый текст',
    'background: linear-gradient(45deg, #f3ec78, #af4261);' +
    'color: white; font-size: 24px; padding: 10px;'
);

```

Часть 2: Браузерные инструменты отладки

2.1. Chrome DevTools для отладки

Источники (Sources panel):

text

Панель Sources (Ctrl+Shift+I)
Навигатор файлов - Page (файлы страницы) - Filesystem (локальные файлы) - Overrides (переопределения) - Snippets (фрагменты кода)
Редактор кода

- Номер строки
- Точки останова (breakpoints)
- Текущая строка выполнения

Панель отладки

- └─ Watch (наблюдаемые выражения)
- └─ Call Stack (стек вызовов)
- └─ Scope (области видимости)
- └─ Breakpoints (точки останова)
- └─ XHR/fetch Breakpoints

Точки останова:

javascript

// Установка точек останова в DevTools:

// 1. Клик по номеру строки - обычная точка останова

// 2. ПКМ → Add conditional breakpoint - условная

// 3. debugger; в коде - программная

```
function complexFunction(data) {
  // 1. Обычная точка останова
  let processed = [];

  for (let i = 0; i < data.length; i++) {
    // 2. Условная точка останова (только для отрицательных)
    // Условие: data[i] < 0
    if (data[i] < 0) {
      processed.push(Math.abs(data[i]));
    } else {
      processed.push(data[i]);
    }

    // 3. Точка останова на DOM изменение
    // В DevTools: Elements panel → ПКМ → Break on → attribute modifications
  }

  return processed;
}

// 4. Точка останова на событие
element.addEventListener('click', function(event) {
  // В DevTools: Sources → Event Listener Breakpoints
  // Выбрать Mouse → click
});
```

```
    console.log('Клик залогирован');  
});
```

Панель Watch:

javascript

```
// Наблюдаемые выражения в DevTools  
// Можно добавить:  
// - переменные: i, processed.length  
// - выражения: data[i] < 0  
// - вызовы функций: JSON.stringify(processed)  
  
function calculateStatistics(numbers) {  
    let sum = 0;  
    let min = Infinity;  
    let max = -Infinity;  
  
    // В Watch можно добавить:  
    // 1. sum (текущая сумма)  
    // 2. numbers.length (длина массива)  
    // 3. i < numbers.length / 2 (половина пройдена?)  
  
    for (let i = 0; i < numbers.length; i++) {  
        sum += numbers[i];  
  
        if (numbers[i] < min) min = numbers[i];  
        if (numbers[i] > max) max = numbers[i];  
  
        // Условная точка останова: i === 5  
        // Watch: `numbers.slice(0, i+1)` - посмотреть часть массива  
    }  
  
    return { sum, min, max, average: sum / numbers.length };  
}
```

2.2. Стек вызовов (Call Stack)

javascript

```
function functionA() {  
    console.log('Функция А начала выполнение');  
    functionB(); // Вызов В  
    console.log('Функция А завершила выполнение');  
}
```

```

function functionB() {
  console.log('Функция В начала выполнение');
  functionC(); // Вызов С
  console.log('Функция В завершила выполнение');
}

function functionC() {
  console.log('Функция С начала выполнение');
  console.trace(); // Показать стек вызовов
  throw new Error('Тестовая ошибка');
}

try {
  functionA();
} catch (error) {
  console.error('Стек вызовов при ошибке:', error.stack);
}

```

/*
Call Stack при вызове console.trace():

<i>functionC (текущая)</i>
<i>functionB</i>
<i>functionA</i>
<i>Глобальный контекст</i>

*/

2.3. Network отладка

javascript

// Мониторинг сетевых запросов
 // DevTools → Network panel

```

async function fetchData() {
  try {
    // Фильтры в Network panel:
    // - XHR (только AJAX запросы)
    // - JS (JavaScript файлы)
    // - Img (изображения)
    // - WS (WebSocket)

```

```

const response = await fetch('https://api.example.com/data', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer token123'
  },
  body: JSON.stringify({ query: 'test' })
});

// В Network можно:
// 1. Увидеть статус запроса (200, 404, 500)
// 2. Посмотреть Headers (заголовки)
// 3. Проверить Payload (тело запроса)
// 4. Увидеть Timing (время выполнения)
// 5. Скопировать как cURL

if (!response.ok) {
  throw new Error(`HTTP error: ${response.status}`);
}

return await response.json();

} catch (error) {
  console.error('Ошибка запроса:', error);
  // В Network → Filter → Has blocked cookies
  // чтобы увидеть заблокированные запросы
}
}

// Throttling для тестирования медленных сетей
// DevTools → Network → Throttling → Slow 3G

```

Часть 3: Обработка ошибок в JavaScript

3.1. Блок try...catch...finally

Базовый синтаксис:

```

javascript

try {

```

```

// Код, который может выбросить ошибку
riskyOperation();
} catch (error) {
  // Обработка ошибки
  console.error('Произошла ошибка:', error);
  notifyUser('Что-то пошло не так');
} finally {
  // Этот блок выполнится ВСЕГДА
  // Независимо от того, была ошибка или нет
  cleanup();
  console.log('Операция завершена');
}

```

Различные типы ошибок:

javascript

```

try {
  // 1. Синтаксическая ошибка (ловятся на этапе парсинга)
  // eval('const x = ;'); // SyntaxError

  // 2. Ошибка времени выполнения
  undefinedVariable; // ReferenceError

  // 3. Ошибка типа
  null.someMethod(); // TypeError

  // 4. Ошибка диапазона
  const arr = [];
  arr.length = -1; // RangeError

  // 5. URI ошибка
  decodeURIComponent('%'); // URIError

  // 6. Ошибка вычисления
  const result = 1 / 0; // Infinity (не ошибка)
  const invalid = Math.log(-1); // NaN (не ошибка)
} catch (error) {
  // Определение типа ошибки
  if (error instanceof SyntaxError) {
    console.error('Синтаксическая ошибка:', error.message);
  } else if (error instanceof ReferenceError) {
    console.error('Ошибка ссылки:', error.message);
  } else if (error instanceof TypeError) {

```

```

    console.error('Ошибка типа:', error.message);
  } else if (error instanceof RangeError) {
    console.error('Ошибка диапазона:', error.message);
  } else if (error instanceof URIError) {
    console.error('Ошибка URI:', error.message);
  } else {
    console.error('Неизвестная ошибка:', error);
  }

  // Информация об ошибке
  console.log('Имя ошибки:', error.name);
  console.log('Сообщение:', error.message);
  console.log('Стек вызовов:', error.stack);
  console.log('Файл и строка:', error.fileName, error.lineNumber);
}

```

3.2. Пользовательские ошибки

Создание кастомных ошибок:

javascript

```

// Базовый класс пользовательской ошибки
class AppError extends Error {
  constructor(message, code = 'APP_ERROR', details = null) {
    super(message);
    this.name = this.constructor.name;
    this.code = code;
    this.details = details;
    this.timestamp = new Date().toISOString();

    // Сохранение оригинального стека
    Error.captureStackTrace(this, this.constructor);
  }

  toJSON() {
    return {
      name: this.name,
      message: this.message,
      code: this.code,
      details: this.details,
      timestamp: this.timestamp,
      stack: this.stack
    };
  }
}

```



```

    };
  }
}

// Специализированные ошибки
class ValidationError extends AppError {
  constructor(message, field, value) {
    super(message, 'VALIDATION_ERROR', { field, value });
    this.field = field;
    this.value = value;
  }
}

class AuthenticationError extends AppError {
  constructor(message = 'Требуется аутентификация') {
    super(message, 'AUTH_ERROR');
    this.statusCode = 401;
  }
}

class AuthorizationError extends AppError {
  constructor(message = 'Недостаточно прав') {
    super(message, 'AUTHORIZATION_ERROR');
    this.statusCode = 403;
  }
}

class NotFoundError extends AppError {
  constructor(resource, id) {
    super(`${resource} с ID ${id} не найден`, 'NOT_FOUND');
    this.resource = resource;
    this.id = id;
    this.statusCode = 404;
  }
}

// Использование
function validateUser(user) {
  if (!user.email) {
    throw new ValidationError(
      'Email обязателен',
      'email',
      user.email
    );
  }
}

```

```

    }

    if (user.age < 18) {
        throw new ValidationError(
            'Пользователь должен быть совершеннолетним',
            'age',
            user.age
        );
    }
}

try {
    validateUser({ name: 'John', age: 16 });
} catch (error) {
    if (error instanceof ValidationError) {
        console.error(`Ошибка в поле ${error.field}:`, error.message);
        console.error('Некорректное значение:', error.value);
    }
    throw error; // Пробрасываем дальше
}

```

3.3. Обработка асинхронных ошибок

Промисы:

```

javascript

// 1. .catch() на промисах
fetch('https://api.example.com/data')
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }
        return response.json();
    })
    .then(data => {
        console.log('Данные получены:', data);
    })
    .catch(error => {
        console.error('Ошибка при запросе:', error);
        // Возвращаем значение по умолчанию
        return { data: [], error: error.message };
    })

```

```

    .finally(() => {
        console.log('Запрос завершен');
    });

// 2. Обработка нескольких промисов
Promise.all([
    fetch('/api/users'),
    fetch('/api/products'),
    fetch('/api/orders')
])
    .then(([users, products, orders]) => {
        // Все промисы успешно завершились
        console.log('Все данные загружены');
    })
    .catch(error => {
        // Хотя бы один промис отвергнут
        console.error('Ошибка в одном из запросов:', error);
    });

// 3. Promise.allSettled() - ждем все промисы
Promise.allSettled([
    fetch('/api/users'),
    fetch('/api/products'),
    fetch('/api/invalid') // Вернет ошибку
])
    .then(results => {
        results.forEach((result, index) => {
            if (result.status === 'fulfilled') {
                console.log(`Запрос ${index} успешен`, result.value);
            } else {
                console.log(`Запрос ${index} ошибка`, result.reason);
            }
        });
    });

```

Async/await:

javascript

```

async function loadUserData(userId) {
    try {
        // 1. Ожидание промиса
        const response = await fetch(`/api/users/${userId}`);

        // 2. Проверка статуса
    }
}

```

```

    if (!response.ok) {
        throw new Error(`HTTP error: ${response.status}`);
    }

    // 3. Парсинг JSON
    const user = await response.json();

    // 4. Валидация данных
    if (!user || !user.id) {
        throw new Error('Некорректные данные пользователя');
    }

    return user;
} catch (error) {
    // 5. Обработка ошибок
    console.error(`Ошибка загрузки пользователя ${userId}:`, error);

    // 6. Возврат значения по умолчанию или повторная ошибка
    if (error instanceof TypeError && error.message.includes('network')) {
        // Сетевая ошибка - возвращаем кэшированные данные
        return getCachedUser(userId);
    }

    // Другие ошибки пробрасываем дальше
    throw error;
} finally {
    // 7. Очистка ресурсов
    cleanup();
}
}

// Обработка нескольких асинхронных операций
async function processMultipleUsers(userIds) {
    const results = [];

    for (const userId of userIds) {
        try {
            const user = await loadUserData(userId);
            results.push({ userId, status: 'success', data: user });
        } catch (error) {
            results.push({ userId, status: 'error', error: error.message });
        }
    }

    // Продолжаем обработку других пользователей

```

```
        continue;
    }
}

return results;
}
```

Часть 4: Продвинутые техники отладки

4.1. Логирование с уровнями

javascript

// Система логирования с уровнями

```
const LogLevel = {
  ERROR: 0,
  WARN: 1,
  INFO: 2,
  DEBUG: 3,
  TRACE: 4
};

class Logger {
  constructor(level = LogLevel.INFO) {
    this.level = level;
    this.colors = {
      [LogLevel.ERROR]: 'red',
      [LogLevel.WARN]: 'orange',
      [LogLevel.INFO]: 'blue',
      [LogLevel.DEBUG]: 'gray',
      [LogLevel.TRACE]: 'lightgray'
    };
  }

  error(message, ...args) {
    this.log(LogLevel.ERROR, message, ...args);
  }

  warn(message, ...args) {
    this.log(LogLevel.WARN, message, ...args);
  }
}
```

```

info(message, ...args) {
  this.log(LogLevel.INFO, message, ...args);
}

debug(message, ...args) {
  this.log(LogLevel.DEBUG, message, ...args);
}

trace(message, ...args) {
  this.log(LogLevel.TRACE, message, ...args);
}

log(level, message, ...args) {
  if (level > this.level) return;

  const timestamp = new Date().toISOString();
  const levelName = Object.keys(LogLevel).find(key => LogLevel[key] === level);
  const color = this.colors[level];

  console.log(
    `%c${timestamp} [${levelName}] ${message}`,
    `color: ${color}; font-weight: ${level <= LogLevel.WARN ? 'bold' : 'normal'}`,
    ...args
  );

  // В production можно отправлять в сервис логирования
  if (level <= LogLevel.ERROR && window.trackJs) {
    window.trackJs.track(message);
  }
}

// Использование
const logger = new Logger(
  process.env.NODE_ENV === 'production'
    ? LogLevel.WARN
    : LogLevel.DEBUG
);

logger.info('Приложение запущено');
logger.debug('Текущее состояние:', { user: 'John', page: 'home' });
logger.error('Критическая ошибка:', error);

```

4.2. Профилирование производительности

javascript

// Ручное профилирование

```
function profileFunction(func, ...args) {
  const startTime = performance.now();
  const startMemory = performance.memory
    ? performance.memory.usedJSHeapSize
    : null;

  try {
    const result = func(...args);

    const endTime = performance.now();
    const endMemory = performance.memory
      ? performance.memory.usedJSHeapSize
      : null;

    console.log(`Функция ${func.name}:`);
    console.log(`- Время выполнения: ${(endTime - startTime).toFixed(2)}ms`);

    if (startMemory && endMemory) {
      console.log(`- Использовано памяти: ${(endMemory - startMemory) / 1024} KB`);
    }

    return result;
  } catch (error) {
    console.error(`Ошибка в функции ${func.name}:`, error);
    throw error;
  }
}
```

// Использование

```
const result = profileFunction(heavyCalculation, largeDataset);
```

// Performance API

```
function measurePerformance() {
  // Создание меток
  performance.mark('start-processing');

  // Выполнение кода...
  processData();
}
```

```

performance.mark('end-processing');

// Измерение между метками
performance.measure(
  'processing-duration',
  'start-processing',
  'end-processing'
);

// Получение измерений
const measures = performance.getEntriesByName('processing-duration');
console.log('Время обработки:', measures[0].duration, 'ms');

// Очистка
performance.clearMarks();
performance.clearMeasures();
}

// Мониторинг памяти
function monitorMemoryUsage() {
  if (performance.memory) {
    const memory = performance.memory;

    console.log('Использование памяти:');
    console.log(`- Всего JS Heap: ${memory.totalJSHeapSize / 1024 / 1024}.toFixed(2)} MB`);
    console.log(`- Используется: ${memory.usedJSHeapSize / 1024 / 1024}.toFixed(2)} MB`);
    console.log(`- Лимит: ${memory.jsHeapSizeLimit / 1024 / 1024}.toFixed(2)} MB`);

    // Предупреждение о высоком использовании памяти
    const usagePercent = (memory.usedJSHeapSize / memory.jsHeapSizeLimit) * 100;
    if (usagePercent > 80) {
      console.warn(`Высокое использование памяти: ${usagePercent.toFixed(1)}%`);
    }
  }
}

```

4.3. Отладка утечек памяти

javascript

```

// Детектор утечек памяти
class MemoryLeakDetector {

```



```

constructor() {
  this.snapshots = new Map();
  this.intervalId = null;
}

startMonitoring(interval = 30000) {
  console.log('Мониторинг памяти запущен');

  this.intervalId = setInterval(() => {
    this.takeSnapshot();
    this.analyzeSnapshots();
  }, interval);
}

stopMonitoring() {
  if (this.intervalId) {
    clearInterval(this.intervalId);
    console.log('Мониторинг памяти остановлен');
  }
}

takeSnapshot() {
  if (performance.memory) {
    const snapshot = {
      timestamp: Date.now(),
      usedJSHeapSize: performance.memory.usedJSHeapSize,
      totalJSHeapSize: performance.memory.totalJSHeapSize
    };

    this.snapshots.set(snapshot.timestamp, snapshot);

    // Храним только последние 10 снимков
    if (this.snapshots.size > 10) {
      const oldestKey = Array.from(this.snapshots.keys())[0];
      this.snapshots.delete(oldestKey);
    }
  }
}

analyzeSnapshots() {
  if (this.snapshots.size < 2) return;

  const snapshots = Array.from(this.snapshots.values());
  const first = snapshots[0];

```

```

const last = snapshots[snapshots.length - 1];

const growth = last.usedJSHeapSize - first.usedJSHeapSize;
const growthPercent = (growth / first.usedJSHeapSize) * 100;

if (growthPercent > 20) {
  console.warn(
    `⚠ Возможная утечка памяти: ` +
    `рост на ${((growth / 1024 / 1024).toFixed(2))} MB ` +
    `${(growthPercent.toFixed(1))}%`
  );

  // Детальный анализ
  this.detailedAnalysis();
}
}

detailedAnalysis() {
  // Используем DevTools для детального анализа
  console.log('Для анализа утечек памяти:');
  console.log('1. Откройте DevTools → Memory');
  console.log('2. Сделайте Heap snapshot');
  console.log('3. Сравните snapshots для поиска утечек');
  console.log('4. Ищите Detached DOM nodes');
}
}

// Использование в приложении
const leakDetector = new MemoryLeakDetector();

// Запуск при старте приложения
if (process.env.NODE_ENV === 'development') {
  leakDetector.startMonitoring();
}

// Остановка при размонтировании компонента
window.addEventListener('beforeunload', () => {
  leakDetector.stopMonitoring();
});

```

Часть 5: Глобальная обработка ошибок

5.1. Глобальные обработчики

javascript

```
// Обработка необработанных ошибок
window.addEventListener('error', (event) => {
  const { message, filename, lineno, colno, error } = event;

  console.error('Глобальная ошибка:', {
    message,
    filename,
    line: lineno,
    column: colno,
    stack: error?.stack,
    timestamp: new Date().toISOString()
  });

  // Отправка в сервис мониторинга
  sendToErrorTracking({
    type: 'unhandled_error',
    error: { message, stack: error?.stack },
    location: { filename, lineno, colno },
    userAgent: navigator.userAgent,
    url: window.location.href
  });

  // Предотвращение стандартного поведения
  event.preventDefault();
});

// Обработка необработанных отклонений промисов
window.addEventListener('unhandledrejection', (event) => {
  const { reason } = event;

  console.error('Необработанное отклонение промиса:', reason);

  sendToErrorTracking({
    type: 'unhandled_rejection',
    error: reason instanceof Error
      ? { message: reason.message, stack: reason.stack }
      : reason,
```

```

        timestamp: new Date().toISOString()
    });

    // Можно также отобразить пользователю
    showUserNotification(
        'Произошла ошибка при выполнении операции',
        'error'
    );
});

// Обработка ошибок загрузки ресурсов
window.addEventListener('load', () => {
    // Проверка неудачно загруженных ресурсов
    const failedResources = performance.getEntriesByType('resource')
        .filter(entry => entry.initiatorType !== 'navigation')
        .filter(entry => entry.duration === 0);

    if (failedResources.length > 0) {
        console.warn('Не загруженные ресурсы:', failedResources);

        sendToErrorTracking({
            type: 'resource_load_failed',
            resources: failedResources.map(r => ({
                name: r.name,
                type: r.initiatorType
            }))
        });
    }
});

```

5.2. Middleware для обработки ошибок

Express.js сервер:

javascript

```

// Middleware для логирования ошибок
function errorLogger(err, req, res, next) {
    console.error('Ошибка сервера:', {
        timestamp: new Date().toISOString(),
        method: req.method,
        url: req.url,
        params: req.params,
    });
    next(err);
}

```

```

    query: req.query,
    body: req.body,
    user: req.user?.id,
    error: {
      name: err.name,
      message: err.message,
      stack: err.stack,
      code: err.code,
      details: err.details
    }
  });

  // Отправка в Sentry/Rollbar/etc
  if (process.env.NODE_ENV === 'production') {
    sendToErrorService(err, req);
  }

  next(err);
}

// Middleware для обработки ошибок
function errorHandler(err, req, res, next) {
  // Определяем статус код
  const statusCode = err.statusCode ||
    err.status ||
    (err instanceof AppError ? 400 : 500);

  // Подготовка ответа
  const response = {
    success: false,
    error: {
      message: err.message || 'Internal Server Error',
      code: err.code || 'INTERNAL_ERROR'
    }
  };

  // Добавляем детали в development
  if (process.env.NODE_ENV !== 'production') {
    response.error.details = err.details;
    response.error.stack = err.stack;
  }

  // Добавляем валидационные ошибки
  if (err instanceof ValidationError) {

```

```

    response.error.fields = err.fields;
    response.error.values = err.values;
  }

  // Отправка ответа
  res.status(statusCode).json(response);
}

// Middleware для 404 ошибок
function notFoundHandler(req, res, next) {
  const error = new NotFoundError('Route', req.originalUrl);
  next(error);
}

// Использование в Express приложении
const express = require('express');
const app = express();

// ... роуты ...

// Обработка 404
app.use(notFoundHandler);

// Обработка ошибок (должны быть последними!)
app.use(errorLogger);
app.use(errorHandler);

```

5.3. React Error Boundaries

```

jsx

// Классовый Error Boundary
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      hasError: false,
      error: null,
      errorInfo: null
    };
  }

  static getDerivedStateFromError(error) {
    // Обновляем состояние при ошибке
  }
}

```

```

    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    // Логирование ошибки
    console.error('Ошибка в компоненте React:', {
      error,
      componentStack: errorInfo.componentStack,
      timestamp: new Date().toISOString()
    });

    // Отправка в сервис мониторинга
    if (this.props.onError) {
      this.props.onError(error, errorInfo);
    }

    // Обновляем состояние
    this.setState({ errorInfo });
  }

  resetError = () => {
    this.setState({
      hasError: false,
      error: null,
      errorInfo: null
    });

    if (this.props.onReset) {
      this.props.onReset();
    }
  };

  render() {
    if (this.state.hasError) {
      // Fallback UI при ошибке
      return this.props.fallback ? (
        this.props.fallback({
          error: this.state.error,
          errorInfo: this.state.errorInfo,
          resetError: this.resetError
        })
      ) : (
        <div className="error-boundary">
          <h2>Что-то пошло не так</h2>

```

```

    <p>{this.state.error?.message}</p>

    {process.env.NODE_ENV === 'development' && (
      <details style={{ whiteSpace: 'pre-wrap' }}>
        <summary>Детали ошибки</summary>
        {this.state.error?.stack}
        <br />
        {this.state.errorInfo?.componentStack}
      </details>
    )}

    <button onClick={this.resetError}>
      Попробовать снова
    </button>
  </div>
);
}

return this.props.children;
}
}

```

// Кастомный fallback компонент

```

const ErrorFallback = ({ error, resetError }) => (
  <div className="error-fallback">
    <div className="error-icon">▲</div>
    <h3>Упс! Произошла ошибка</h3>
    <p>Мы уже работаем над ее устранением</p>
    <div className="error-details">
      <small>{error.message}</small>
    </div>
    <div className="error-actions">
      <button onClick={resetError} className="retry-btn">
        Повторить
      </button>
      <button
        onClick={() => window.location.reload()}
        className="reload-btn"
      >
        Перезагрузить страницу
      </button>
      <button
        onClick={() => window.location.href = '/'}
        className="home-btn"
      >

```



```

    >
      На главную
    </button>
  </div>
</div>
);

```

// Использование

```

function App() {
  return (
    <ErrorBoundary
      fallback={ErrorFallback}
      onError={(error, errorInfo) => {
        // Отправка в сервис мониторинга
        sendToErrorTracking({
          type: 'react_error',
          error: error.message,
          componentStack: errorInfo.componentStack,
          url: window.location.href
        });
      }}
    >
      <UserProfile />
      <ProductList />
      <ShoppingCart />
    </ErrorBoundary>
  );
}

```

// Компонент с собственной обработкой ошибок

```

function SafeComponent({ children }) {
  const [hasError, setHasError] = useState(false);

  const handleError = useCallback((error) => {
    console.error('Ошибка в SafeComponent:', error);
    setHasError(true);

    // Восстановление через 5 секунд
    setTimeout(() => setHasError(false), 5000);
  }, []);

  if (hasError) {
    return <div className="component-error">Компонент временно недоступен</div>;
  }
}

```

```
return (  
  <ErrorBoundary onError={handleError}>  
    {children}  
  </ErrorBoundary>  
)  
);  
}
```

Часть 6: Инструменты и библиотеки

6.1. Сторонние сервисы мониторинга

javascript

```
// Интеграция с Sentry  
import * as Sentry from '@sentry/react';  
import { BrowserTracing } from '@sentry/tracing';  
  
Sentry.init({  
  dsn: process.env.SENTRY_DSN,  
  integrations: [new BrowserTracing()],  
  tracesSampleRate: 1.0,  
  environment: process.env.NODE_ENV,  
  release: process.env.RELEASE_VERSION,  
  beforeSend(event) {  
    // Фильтрация чувствительных данных  
    if (event.request) {  
      delete event.request.cookies;  
      delete event.request.headers['Authorization'];  
    }  
    return event;  
  }  
});  
  
// Кастомная ошибка с Sentry  
class TrackedError extends Error {  
  constructor(message, context = {}) {  
    super(message);  
    this.name = this.constructor.name;  
  }  
  
  // Отправка в Sentry с контекстом  
  Sentry.withScope((scope) => {
```

```

        scope.setExtras(context);
        Sentry.captureException(this);
    });
}
}

// React Error Boundary om Sentry
import { ErrorBoundary } from '@sentry/react';

function App() {
    return (
        <ErrorBoundary
            fallback={({ error, componentStack, resetError }) => (
                <div>
                    <h2>Произошла ошибка</h2>
                    <button onClick={resetError}>Попробовать снова</button>
                </div>
            )}
            onError={(error) => {
                // Дополнительная логика при ошибке
            }}
        >
            {/* Приложение */}
        </ErrorBoundary>
    );
}

```

6.2. Утилиты для отладки в development

javascript

```

// Dev только код
if (process.env.NODE_ENV === 'development') {
    // Глобальные утилиты для отладки
    window.__DEV__ = {
        // Включение/выключение логирования
        logLevel: 'debug',

        // Быстрый доступ к состоянию
        getState: () => store.getState(),

        // Эмуляция ошибок
        simulateError: (type = 'api') => {
            switch(type) {

```

```

    case 'api':
      throw new Error('Тестовая API ошибка');
    case 'network':
      throw new TypeError('Network request failed');
    case 'validation':
      throw new ValidationError('Тестовая ошибка валидации');
  }
},

// Сброс состояния
resetApp: () => {
  localStorage.clear();
  sessionStorage.clear();
  window.location.reload();
},

// Генерация тестовых данных
generateTestData: (count = 10) => {
  return Array.from({ length: count }, (_, i) => ({
    id: i + 1,
    name: `User ${i + 1}`,
    email: `user${i + 1}@example.com`,
    active: Math.random() > 0.5
  })));
}
};

// Автоматическая логирование всех ошибок
const originalError = console.error;
console.error = (...args) => {
  originalError(...args);

  // Сохранение в историю для DevTools
  if (!window.__DEV__.errorHistory) {
    window.__DEV__.errorHistory = [];
  }
  window.__DEV__.errorHistory.push({
    timestamp: new Date().toISOString(),
    args: args.map(arg =>
      arg instanceof Error
        ? { message: arg.message, stack: arg.stack }
        : arg
    )
  });
};

```

```
// Ограничение истории
if (window.__DEV__.errorHistory.length > 50) {
  window.__DEV__.errorHistory.shift();
}
};
}
```

Заключение: лучшие практики

7.1. Золотые правила отладки

1. Воспроизводимость — ошибка должна быть воспроизводима
2. Изоляция — минимизируйте код для поиска проблемы
3. Систематичность — используйте метод научного исследования
4. Документирование — записывайте шаги и решения
5. Профилактика — предупреждайте ошибки до их появления

7.2. Чек-лист обработки ошибок

javascript

```
// Проверка перед релизом:
const errorHandlingChecklist = [
  // 1. Сетевые запросы
  '✓ Все fetch/axios запросы имеют .catch() блоки',
  '✓ Реализована обработка таймаутов',
  '✓ Есть fallback для offline режима',

  // 2. Пользовательский ввод
  '✓ Валидация всех форм и полей ввода',
  '✓ Сообщения об ошибках понятны пользователю',
  '✓ Чувствительные данные не логируются',

  // 3. Критические функции
  '✓ Критический код обернут в try/catch',
  '✓ Асинхронные операции имеют обработку ошибок',
  '✓ Реализованы механизмы повтора (retry)',

  // 4. Мониторинг
  '✓ Интегрирован сервис мониторинга ошибок',
  '✓ Есть логирование важных событий',
];
```

'✓ Реализованы метрики производительности',

// 5. Пользовательский опыт

'✓ Пользователь видит понятные сообщения об ошибках',

'✓ Есть возможность повторить операцию',

'✓ Состояние приложения сохраняется при ошибках',

// 6. Безопасность

'✓ Ошибки не раскрывают чувствительную информацию',

'✓ Реализована защита от XSS в сообщениях об ошибках',

'✓ Логи очищаются от персональных данных'

];

7.3. Стратегия дебаггинга

text

Алгоритм решения проблемы:

1. Воспроизведите ошибку

- Какие шаги приводят к ошибке?
- В каких условиях возникает?

2. Соберите информацию

- Сообщение об ошибке
- Стек вызовов
- Состояние приложения
- Версии браузера/ОС

3. Локализируйте проблему

- Используйте console.log()
- Установите точки останова
- Упростите код до минимального примера

4. Найдите причину

- Проверьте типы данных
- Проверьте условия
- Проверьте асинхронные операции

5. Исправьте и протестируйте

- Напишите тест для воспроизведения
- Проверьте в разных условиях
- Убедитесь, что не сломали что-то еще

6. Документируйте решение

- Запишите причину проблемы
- Опишите решение
- Добавьте комментарии в код

7.4. Запомните:

- **Ошибки — это возможности для улучшения кода**
- **Проактивная обработка лучше, чем реактивная**
- **Пользователь никогда не должен видеть сырые ошибки**
- **Логирование — ваш лучший друг при отладке**
- **Тестируйте обработку ошибок так же тщательно, как и happy path**

Правильная отладка и обработка ошибок превращают хаотичные падения приложения в управляемые и понятные ситуации, что значительно повышает качество и надежность вашего кода.

Глава 3: Первая программа на JavaScript

3.1. Настройка окружения разработки

Введение: почему правильная настройка важна

Правильно настроенное окружение разработки — фундамент продуктивной работы. Оно экономит время, предотвращает ошибки и обеспечивает согласованность между разработчиками.

Часть 1: Базовые инструменты

1.1. Установка Node.js и npm

Проверка текущих версий:

```
bash
```

```
# Проверка установки Node.js
```

```
node --version
```

```
# v18.17.0 или выше
```

```
# Проверка npm
```

```
npm --version
```

```
# 9.6.7 или выше
```

```
# Проверка npx
```

```
npx --version
```

```
# (должен быть доступен если установлен npm)
```

Установка через менеджер версий (рекомендуется):

```
bash
```

```
# Установка nvm (Node Version Manager) на macOS/Linux
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
```


Или через Homebrew

```
brew install nvm
```

Установка nvm-windows на Windows

Скачать с <https://github.com/coreybutler/nvm-windows>

Использование nvm

```
nvm install 18.17.0 # Установка конкретной версии
```

```
nvm install --lts # Установка LTS версии
```

```
nvm use 18.17.0 # Использование версии
```

```
nvm alias default 18.17.0 # Установка версии по умолчанию
```

```
nvm ls # Список установленных версий
```

Ручная установка (если nvm не подходит):

1. Windows: скачать с <https://nodejs.org> (выбрать LTS версию)

2. macOS: `brew install node` или скачать установщик

3. Linux:

```
bash
```

Ubuntu/Debian

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

Fedora

```
sudo dnf install nodejs
```

Настройка npm:

```
bash
```

Проверка текущей конфигурации

```
npm config list
```

Настройка глобального npm (рекомендуемые значения)

```
npm config set init-author-name "Ваше Имя"
```

```
npm config set init-author-email "ваш@email.com"
```

```
npm config set init-license "MIT"
```

```
npm config set save-exact true # Сохранять точные версии
```

Изменение расположения глобальных пакетов (опционально)

Windows

```
npm config set prefix "C:\Users\Имя\AppData\Roaming\npm"
```

macOS/Linux

```
mkdir ~/.npm-global
```

```
npm config set prefix '~/.npm-global'
```

```
# Добавить в PATH (в ~/.bashrc или ~/.zshrc)
```

```
export PATH=~/.npm-global/bin:$PATH
```

1.2. Редактор кода: VS Code

Установка и базовая настройка:

1. Скачать с <https://code.visualstudio.com>
2. Рекомендуемые настройки (settings.json):

```
json
{
  // Основные настройки
  "editor.fontSize": 14,
  "editor.fontFamily": "'Cascadia Code', 'Fira Code', Consolas, monospace",
  "editor.fontLigatures": true,
  "editor.tabSize": 2,
  "editor.insertSpaces": true,
  "editor.detectIndentation": false,

  // Форматирование
  "editor.formatOnSave": true,
  "editor.formatOnPaste": false,
  "editor.formatOnType": false,

  // Файлы
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  "files.exclude": {
    "**/.git": true,
    "**/.DS_Store": true,
    "**/node_modules": true,
    "**/dist": true,
    "**/.next": true,
    "**/coverage": true
  },

  // JavaScript/TypeScript
  "javascript.updateImportsOnFileMove.enabled": "always",
  "typescript.updateImportsOnFileMove.enabled": "always",
  "[javascript]": {
```

```

    "editor.defaultFormatter": "esbenp.prettier-vscode"
  },
  "[typescript]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode"
  },

  // Терминал
  "terminal.integrated.fontSize": 13,
  "terminal.integrated.shell.windows": "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
  "terminal.integrated.defaultProfile.windows": "PowerShell",

  // Интерфейс
  "workbench.colorTheme": "Default Dark Modern",
  "workbench.iconTheme": "material-icon-theme",
  "window.zoomLevel": 0
}

```

Обязательные расширения для JavaScript разработки:

bash

Установка через командную палитру (Ctrl+Shift+P)

Или через терминал

```

code --install-extension esbenp.prettier-vscode      # Форматирование
code --install-extension dbaeumer.vscode-eslint      # Линтинг
code --install-extension mikestead.dotenv            # Подсветка .env
code --install-extension eamodio.gitlens             # Git интеграция
code --install-extension ms-vscode.vscode-typescript-next # TypeScript
code --install-extension christian-kohler.npm-intellisense # npm автодополнение
code --install-extension ms-vscode.vscode-js-profile-flame # Профилирование

```

Полезные сочетания клавиш:

text

Общие:

Ctrl+Shift+P - Командная палитра
 Ctrl+P - Быстрый переход к файлу
 Ctrl+` - Открыть/закрыть терминал
 Ctrl+B - Показать/скрыть сайдбар
 Ctrl+Shift+F - Поиск по проекту

Редактирование:

Ctrl+D - Выделить следующее совпадение
 Alt+↑/↓ - Переместить строку

Shift+Alt+↑/↓ - Копировать строку
Ctrl+/ - Закомментировать/раскомментировать
Ctrl+Shift+K - Удалить строку

Навигация:

F12 - Перейти к определению
Alt+F12 - Показать определение
Ctrl+G - Перейти к строке
F8 - Перейти к следующей ошибке

Часть 2: Инициализация проекта

2.1. Создание нового проекта

Стандартная инициализация:

```
bash
```

```
# Создание директории проекта
```

```
mkdir my-javascript-project
```

```
cd my-javascript-project
```

```
# Инициализация прт проекта
```

```
npm init -y
```

```
# Создается package.json с настройками по умолчанию
```

```
# Просмотр созданного package.json
```

```
cat package.json
```

Структура package.json после инициализации:

```
json
```

```
{  
  "name": "my-javascript-project",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",
```

```
"license": "ISC"
}
```

Настройка package.json для разработки:

```
json
{
  "name": "my-javascript-project",
  "version": "1.0.0",
  "description": "Мой JavaScript проект",
  "main": "src/index.js",
  "type": "module", // Использовать ES6 модули
  "scripts": {
    "dev": "node src/index.js",
    "start": "node src/index.js",
    "build": "webpack --mode production",
    "test": "jest",
    "lint": "eslint src/",
    "format": "prettier --write src/",
    "prepare": "husky install"
  },
  "keywords": ["javascript", "node", "web"],
  "author": "Ваше Имя <ваш@email.com>",
  "license": "MIT",
  "engines": {
    "node": ">=18.0.0",
    "npm": ">=9.0.0"
  },
  "dependencies": {
    // Зависимости для продакшена
  },
  "devDependencies": {
    // Зависимости для разработки
  },
  "browserslist": [
    "> 0.5%",
    "last 2 versions",
    "not dead"
  ]
}
```

2.2. Базовая структура проекта

Рекомендуемая структура:

text

```
my-javascript-project/
├── src/                    # Исходный код
│   ├── index.js           # Точка входа
│   ├── utils/             # Утилиты
│   │   ├── helpers.js
│   │   └── validators.js
│   ├── components/        # Компоненты (если есть)
│   │   └── Button.js
│   ├── styles/            # Стили (если нужно)
│   │   └── main.css
│   └── __tests__/         # Тесты рядом с кодом
│       └── helpers.test.js
├── public/                # Статические файлы
│   ├── index.html
│   ├── favicon.ico
│   └── assets/
├── dist/                  # Собранный проект (генерируется)
├── docs/                  # Документация
├── node_modules/          # Зависимости (генерируется)
├── .gitignore             # Git игнорирование
├── .eslintrc.js           # Конфигурация ESLint
├── .prettierrc            # Конфигурация Prettier
├── package.json           # Зависимости и скрипты
├── README.md              # Документация проекта
└── LICENSE                # Лицензия (опционально)
```

Создание структуры автоматически:

bash

Создание структуры каталогов

```
mkdir -p src/{utils,components,styles,__tests__} public/{css,js,images} docs
```

Создание базовых файлов

```
touch src/index.js
```

```
touch public/index.html
```

```
touch README.md
```

```
touch .gitignore
```

Инициализация Git репозитория

```
git init
```

2.3. Файл .gitignore

Базовый .gitignore для JavaScript проекта:

```
gitignore

# Зависимости
/node_modules
/.pnp
.pnp.js

# Тестирование
/coverage
/.nyc_output

# Сборка
/dist
/build
/.next
/out

# Логи
npm-debug.log*
yarn-debug.log*
yarn-error.log*
*.log

# IDE
.vscode/*
!.vscode/settings.json
!.vscode/tasks.json
!.vscode/launch.json
!.vscode/extensions.json
.idea/
*.swp
*.swo

# Окружение
.env
.env.local
```

```
.env.development.local  
.env.test.local  
.env.production.local
```

```
# macOS  
.DS_Store  
.AppleDouble  
.LSOverride
```

```
# Windows  
Thumbs.db  
ehthumbs.db  
Desktop.ini
```

```
# Временные файлы  
*.tmp  
*.temp
```

Дополнительные правила для специфичных проектов:

```
gitignore
```

```
# React  
/.eslintcache
```

```
# Vue  
dist  
dist-ssr  
*.local
```

```
# Angular  
/.angular/cache
```

```
# Node.js  
# Логи приложений  
logs  
*.pid
```

```
# Ключи и сертификаты  
*.pem  
*.key  
*.crt
```

```
# Деплой  
.deploy/
```


Часть 3: Инструменты разработки

3.1. Линтинг с ESLint

Установка и настройка:

```
bash
```

```
# Установка ESLint
```

```
npm install --save-dev eslint
```

```
# Инициализация конфигурации
```

```
npx eslint --init
```

```
# Ответьте на вопросы:
```

```
# ✓ How would you like to use ESLint? · problems
```

```
# ✓ What type of modules does your project use? · esm
```

```
# ✓ Which framework does your project use? · none
```

```
# ✓ Does your project use TypeScript? · No
```

```
# ✓ Where does your code run? · browser, node
```

```
# ✓ What format do you want your config file to be in? · JavaScript
```

Конфигурационный файл .eslintrc.js:

```
javascript
```

```
module.exports = {
```

```
  env: {
```

```
    browser: true,
```

```
    es2021: true,
```

```
    node: true
```

```
  },
```

```
  extends: [
```

```
    'eslint:recommended',
```

```
    'plugin:prettier/recommended' // Интеграция с Prettier
```

```
  ],
```

```
  parserOptions: {
```

```
    ecmaVersion: 'latest',
```

```
    sourceType: 'module'
```

```
  },
```

```
  rules: {
```

```
    // Стиль кода
```

```
    'indent': ['error', 2],
```

```
    'linebreak-style': ['error', 'unix'],
```

```
    'quotes': ['error', 'single'],
```

```

    'semi': ['error', 'always'],

    // Переменные
    'no-unused-vars': ['warn', {
      'argsIgnorePattern': '^_',
      'varsIgnorePattern': '^_'
    }],
    'no-var': 'error',
    'prefer-const': 'error',

    // Строки
    'prefer-template': 'error',
    'template-curly-spacing': ['error', 'never'],

    // Функции
    'func-style': ['error', 'expression'],
    'arrow-body-style': ['error', 'as-needed'],

    // Объекты и массивы
    'object-shorthand': 'error',
    'prefer-destructuring': ['error', {
      'array': true,
      'object': true
    }],

    // Сравнения
    'eqeqeq': ['error', 'always'],
    'no-eq-null': 'error',

    // Консоль (разрешить только в разработке)
    'no-console': process.env.NODE_ENV === 'production' ? 'error' : 'warn',
    'no-debugger': process.env.NODE_ENV === 'production' ? 'error' : 'warn'
  },
  overrides: [
    {
      files: ['**/*.test.js', '**/*.spec.js'],
      env: {
        jest: true
      },
      rules: {
        'no-console': 'off'
      }
    }
  ]
]

```

```
};
```

Полезные скрипты для package.json:

```
json
{
  "scripts": {
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix",
    "lint:watch": "eslint src/ --watch"
  }
}
```

3.2. Форматирование с Prettier

Установка и настройка:

```
bash

# Установка Prettier
npm install --save-dev prettier eslint-config-prettier eslint-plugin-prettier

# Создание конфигурационного файла
npx prettier --write .prettierrc
```

Конфигурационный файл .prettierrc:

```
json
{
  "semi": true,
  "tabWidth": 2,
  "printWidth": 80,
  "singleQuote": true,
  "trailingComma": "es5",
  "bracketSpacing": true,
  "arrowParens": "avoid",
  "endOfLine": "lf",
  "quoteProps": "as-needed",
  "jsxSingleQuote": false,
  "bracketSameLine": false,
  "proseWrap": "preserve"
}
```

Дополнительные файлы для исключений:

```
json

// .prettierignore
# Игнорируемые файлы и папки
node_modules
dist
build
coverage
*.log
*.min.js
package-lock.json
yarn.lock
```

3.3. Настройка Git Hooks с Husky

Автоматизация проверок перед коммитом:

```
bash

# Установка Husky
npm install --save-dev husky

# Инициализация Husky
npx husky install

# Добавление хука pre-commit
npx husky add .husky/pre-commit "npm run lint && npm run test"

# Добавление хука commit-msg (опционально)
npx husky add .husky/commit-msg 'npx --no -- commitlint --edit "$1"'
```

Пример файла .husky/pre-commit:

```
bash

#!/usr/bin/env sh
. "$(dirname "$0")/_/husky.sh"

echo "👉 Запуск проверок перед коммитом..."

# Проверка линтера
echo "👉 Проверка ESLint..."
npm run lint

if [ $? -ne 0 ]; then
```

```

    echo "× ESLint нашел ошибки. Исправьте их перед коммитом."
    exit 1
fi

# Запуск тестов
echo "□ Запуск тестов..."
npm test

if [ $? -ne 0 ]; then
    echo "× Тесты не прошли. Исправьте тесты перед коммитом."
    exit 1
fi

# Форматирование с Prettier
echo "□ Форматирование кода..."
npm run format

# Добавление отформатированных файлов
git add .

echo "□ Все проверки пройдены! Коммит разрешен."

```

3.4. Настройка коммитов с Commitlint

Стандартизация сообщений коммитов:

```

bash

# Установка Commitlint
npm install --save-dev @commitlint/config-conventional @commitlint/cli

# Создание конфигурации
echo "module.exports = {extends: ['@commitlint/config-conventional']}" >
commitlint.config.js

```

Конфигурационный файл commitlint.config.js:

```

javascript

module.exports = {
  extends: ['@commitlint/config-conventional'],
  rules: {
    'type-enum': [
      2,
      'always',

```

```

[
  'feat',      // Новая функциональность
  'fix',       // Исправление ошибки
  'docs',      // Изменение документации
  'style',     // Изменения форматирования
  'refactor',  // Рефакторинг кода
  'test',      // Добавление тестов
  'chore',     // Изменение процесса сборки
  'revert'     // Откат изменений
]
],
'scope-case': [2, 'always', 'lower-case'],
'subject-case': [2, 'never', ['sentence-case', 'start-case', 'pascal-case', 'upper-
case']],
'subject-empty': [2, 'never'],
'subject-full-stop': [2, 'never', '.'],
'header-max-length': [2, 'always', 72]
}
};

```

Примеры валидных сообщений коммитов:

text

feat: добавить аутентификацию пользователя
fix: исправить утечку памяти в компоненте списка
docs: обновить README с инструкциями установки
style: исправить отступы в файле конфигурации
refactor: переписать модуль валидации на async/await
test: добавить тесты для функции calculateTotal
chore: обновить версию зависимости eslint
revert: откатить изменения в API клиенте

Часть 4: Сборка проекта

4.1. Выбор сборщика

Сравнение популярных сборщиков:

text

Webpack (универсальный, мощный, сложный):

✓ Множество плагинов и загрузчиков

- ✓ Разделение кода (code splitting)
- ✓ Горячая перезагрузка (HMR)
- ✗ Сложная конфигурация
- ✗ Медленная сборка для больших проектов

Vite (современный, быстрый, простой):

- ✓ Мгновенный запуск dev сервера
- ✓ Быстрая сборка (на Rollup)
- ✓ Нативная поддержка ES модулей
- ✓ Минимальная конфигурация
- ✗ Меньше плагинов чем у Webpack

Parcel (нулевая конфигурация):

- ✓ Работает из коробки
- ✓ Автоматическая обработка зависимостей
- ✓ Быстрая сборка
- ✗ Меньше контроля над процессом
- ✗ Меньше плагинов

Rollup (для библиотек):

- ✓ Оптимизирован для библиотек
- ✓ Tree-shaking из коробки
- ✓ Чистый выходной код
- ✗ Сложнее для веб-приложений

Рекомендации по выбору:

- 🟦 **Новичкам:** Vite или Parcel
- 🟦 **Крупные приложения:** Webpack
- 🟦 **Библиотеки:** Rollup
- 🟦 **Современные проекты:** Vite

4.2. Настройка Vite (рекомендуемый выбор)

Быстрый старт:

```
bash
```

```
# Создание проекта с Vite
```

```
npm create vite@latest my-vite-app -- --template vanilla
```

```
# Или с TypeScript
```

```
npm create vite@latest my-vite-app -- --template vanilla-ts
```

```
# Установка зависимостей
```

```
cd my-vite-app
```

```
npm install
```

Базовая структура проекта Vite:

text

```
my-vite-app/
├── src/
│   ├── main.js          # Точка входа
│   ├── style.css        # Основные стили
│   └── counter.js       # Пример модуля
├── public/              # Статические файлы
│   └── vite.svg
├── index.html           # HTML шаблон
├── package.json
├── vite.config.js       # Конфигурация Vite
└── README.md
```

Конфигурация vite.config.js:

javascript

```
import { defineConfig } from 'vite';
import path from 'path';

export default defineConfig({
  // Базовый путь (для деплоя в поддиректорию)
  base: '/',

  // Корневая директория проекта
  root: process.cwd(),

  // Публичная директория
  publicDir: 'public',

  // Настройки сервера разработки
  server: {
    port: 3000,
    open: true,          // Автоматически открывать браузер
    host: true,          // Доступ с других устройств
    cors: true,          // Включить CORS
    strictPort: false,   // Не строгий порт

    // Прокси для API
  }
});
```



```
proxy: {
  '/api': {
    target: 'http://localhost:8080',
    changeOrigin: true,
    rewrite: (path) => path.replace(/^\/api/, '')
  }
},

// Горячая перезагрузка
hmr: {
  overlay: true // Показывать оверлей при ошибках
}
},

// Настройки сборки
build: {
  outDir: 'dist', // Выходная директория
  assetsDir: 'assets', // Директория для ассетов
  sourcemap: true, // Генерация source maps
  minify: 'terser', // Минификация
  rollupOptions: {
    // Настройки Rollup
    input: {
      main: path.resolve(__dirname, 'index.html')
    },
    output: {
      // Разделение кода на чанки
      manualChunks: {
        vendor: ['lodash', 'axios'],
        utils: ['./src/utils/helpers.js']
      }
    }
  },
},

// Оптимизация зависимостей
commonjsOptions: {
  include: [/node_modules/]
},

// Плагины
plugins: [
  // Vite плагины добавляются здесь
],
```

```

// Разрешение модулей
resolve: {
  alias: {
    '@': path.resolve(__dirname, 'src'),
    '@components': path.resolve(__dirname, 'src/components'),
    '@utils': path.resolve(__dirname, 'src/utils')
  }
},

// CSS настройки
css: {
  devSourceMap: true, // Source maps для CSS в разработке
  modules: {
    localsConvention: 'camelCase' // camelCase для CSS модулей
  }
},

// Настройки оптимизации
optimizeDeps: {
  include: ['lodash', 'axios'], // Предварительная обработка
  exclude: ['some-package'] // Исключить из оптимизации
}
});

```

4.3. Настройка Webpack (альтернативный вариант)

Базовая конфигурация webpack.config.js:

```

javascript

const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  // Режим сборки
  mode: process.env.NODE_ENV || 'development',

  // Точка входа
  entry: './src/index.js',

  // Выходной файл
  output: {

```

```
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.[contenthash].js',
    clean: true, // Очищать dist перед каждой сборкой
    assetModuleFilename: 'assets/[hash][ext][query]'
  },
```

```
// Dev сервер
```

```
devServer: {
  static: {
    directory: path.join(__dirname, 'dist')
  },
  port: 3000,
  open: true,
  hot: true,
  compress: true,
  historyApiFallback: true
},
```

```
// Модули и правила
```

```
module: {
  rules: [
    // JavaScript
    {
      test: /\.js$/,
      exclude: /node_modules/,
      use: {
        loader: 'babel-loader',
        options: {
          presets: ['@babel/preset-env']
        }
      }
    }
  ],
},
```

```
// CSS
```

```
{
  test: /\.css$/,
  use: ['style-loader', 'css-loader']
},
```

```
// Изображения
```

```
{
  test: /\.(png|svg|jpg|jpeg|gif)$/i,
  type: 'asset/resource'
},
```

```

    // Шрифты
    {
      test: /\.woff|woff2|eot|ttf|otf$/i,
      type: 'asset/resource'
    }
  ]
},

// Плагины
plugins: [
  new HtmlWebpackPlugin({
    title: 'Мое приложение',
    template: './public/index.html',
    favicon: './public/favicon.ico'
  })
],

// Разрешение расширений
resolve: {
  extensions: ['.js', '.jsx', '.json'],
  alias: {
    '@': path.resolve(__dirname, 'src')
  }
},

// Source maps
devtool: process.env.NODE_ENV === 'production'
  ? 'source-map'
  : 'eval-source-map',

// Оптимизация
optimization: {
  splitChunks: {
    chunks: 'all',
    cacheGroups: {
      vendor: {
        test: /[\\/]node_modules[\\/]/,
        name: 'vendors',
        chunks: 'all'
      }
    }
  }
}
}

```

```
};
```

4.4. Babel для поддержки старых браузеров

Настройка Babel с Vite:

```
bash
```

```
# Установка Babel
```

```
npm install --save-dev @babel/core @babel/preset-env
```

Конфигурационный файл babel.config.js:

```
javascript
```

```
module.exports = {  
  presets: [  
    ['@babel/preset-env', {  
      targets: {  
        browsers: [  
          '> 0.5%',  
          'last 2 versions',  
          'not dead'  
        ]  
      },  
    },  
    useBuiltIns: 'usage',  
    corejs: '3.26'  
  ]  
],  
  plugins: [  
    '@babel/plugin-transform-runtime'  
  ]  
};
```

Часть 5: Тестирование

5.1. Настройка Jest

Установка и конфигурация:

```
bash
```

```
# Установка Jest
```

```
npm install --save-dev jest @types/jest
```

```
# Инициализация конфигурации
```

```
npx jest --init
```

```
# Ответьте на вопросы для настройки
```

Конфигурационный файл jest.config.js:

```
javascript
```

```
module.exports = {  
  // Корневая директория для тестов  
  roots: ['<rootDir>/src'],  
  
  // Сбор покрытия кода  
  collectCoverage: true,  
  coverageDirectory: 'coverage',  
  coverageReporters: ['text', 'lcov', 'html'],  
  collectCoverageFrom: [  
    'src/**/*.{js,jsx}',  
    '!src/**/*.test.{js,jsx}',  
    '!src/**/*.spec.{js,jsx}',  
    '!src/index.js'  
  ],  
  
  // Минимальный порог покрытия  
  coverageThreshold: {  
    global: {  
      branches: 80,  
      functions: 80,  
      lines: 80,  
      statements: 80  
    }  
  },  
  
  // Окружение тестов  
  testEnvironment: 'jsdom',  
  
  // Настройки для модулей  
  moduleNameMapper: {  
    '\\.(css|less|scss|sass)$': 'identity-obj-proxy',  
    '^@/(.*)$': '<rootDir>/src/$1',  
    '^@components/(.*)$': '<rootDir>/src/components/$1'  
  },  
}
```

```
// Файлы, которые выполняются перед тестами
setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],

// Расширения файлов для тестов
testMatch: [
  '**/__tests__/**/*.js',
  '**/*.test.js',
  '**/*.spec.js'
],

// Игнорируемые пути
testPathIgnorePatterns: [
  '/node_modules/',
  '/dist/',
  '/build/'
],

// Трансформаторы
transform: {
  '^.+\\.jsx?$': 'babel-jest'
},

// Настройки для отслеживания файлов
watchPathIgnorePatterns: [
  '/node_modules/',
  '/dist/',
  '/coverage/'
],

// Автоматическое очищение моков
clearMocks: true,
resetMocks: true,
restoreMocks: true
};
```

Настройка тестового окружения jest.setup.js:

javascript

```
// Глобальные моки
global.fetch = require('jest-fetch-mock');

// Глобальные переменные
global.console = {
  ...console,
```

```

    log: jest.fn(),
    error: jest.fn(),
    warn: jest.fn()
  };

  // Глобальные функции
  beforeEach(() => {
    // Очистка перед каждым тестом
    jest.clearAllMocks();
    fetch.resetMocks();
  });

  afterEach(() => {
    // Действия после каждого теста
  });

  afterAll(() => {
    // Действия после всех тестов
  });

```

5.2. Пример теста

javascript

```

// src/utils/calculator.test.js
import { add, subtract, multiply, divide } from './calculator';

describe('Калькулятор', () => {
  describe('Сложение', () => {
    test('добавляет два положительных числа', () => {
      expect(add(2, 3)).toBe(5);
    });

    test('добавляет отрицательные числа', () => {
      expect(add(-1, -2)).toBe(-3);
    });

    test('добавляет ноль', () => {
      expect(add(5, 0)).toBe(5);
    });
  });

  describe('Вычитание', () => {
    test('вычитает два числа', () => {

```



```
    expect(subtract(5, 3)).toBe(2);
  });
});

describe('Умножение', () => {
  test('умножает два числа', () => {
    expect(multiply(4, 3)).toBe(12);
  });

  test('умножает на ноль', () => {
    expect(multiply(5, 0)).toBe(0);
  });
});

describe('Деление', () => {
  test('делит два числа', () => {
    expect(divide(10, 2)).toBe(5);
  });

  test('выбрасывает ошибку при делении на ноль', () => {
    expect(() => divide(5, 0)).toThrow('Деление на ноль невозможно');
  });
});
});
```

Часть 6: Деплой и CI/CD

6.1. Настройка окружений

Создание файлов окружения:

text

```
.env          # Локальные переменные (не коммитить)
.env.example  # Пример переменных (коммитить)
.env.development # Разработка
.env.test     # Тестирование
.env.production # Продакшен
```

Пример .env.example:

env

```
# Настройки приложения
APP_NAME=MyJavaScriptApp
APP_VERSION=1.0.0
NODE_ENV=development
PORT=3000

# База данных
DATABASE_URL=postgresql://user:password@localhost:5432/dbname
DATABASE_POOL_SIZE=10

# API ключи
API_BASE_URL=https://api.example.com
API_KEY=your_api_key_here

# Безопасность
JWT_SECRET=your_jwt_secret_here
SESSION_SECRET=your_session_secret_here

# Мониторинг
SENTRY_DSN=your_sentry_dsn_here
LOG_LEVEL=info
```

Загрузка переменных окружения:

```
javascript

// src/config/env.js
import dotenv from 'dotenv';
import path from 'path';

// Загрузка переменных окружения
const envFile = process.env.NODE_ENV
  ? `.${env}.${process.env.NODE_ENV}`
  : '.env';

dotenv.config({ path: path.resolve(process.cwd(), envFile) });

// Экспорт конфигурации
export default {
  // Приложение
  app: {
    name: process.env.APP_NAME || 'MyApp',
    version: process.env.APP_VERSION || '1.0.0',
    port: parseInt(process.env.PORT || '3000'),
    env: process.env.NODE_ENV || 'development'
  }
}
```

```

},

// База данных
database: {
  url: process.env.DATABASE_URL,
  poolSize: parseInt(process.env.DATABASE_POOL_SIZE || '10')
},

// API
api: {
  baseUrl: process.env.API_BASE_URL,
  key: process.env.API_KEY
},

// Безопасность
security: {
  jwtSecret: process.env.JWT_SECRET,
  sessionSecret: process.env.SESSION_SECRET
},

// Валидация конфигурации
validate() {
  const required = [
    'JWT_SECRET',
    'SESSION_SECRET'
  ];

  const missing = required.filter(key => !process.env[key]);

  if (missing.length > 0) {
    throw new Error(`Отсутствуют обязательные переменные окружения: ${missing.join(', ')} `);
  }

  return true;
}
};

```

6.2. Настройка GitHub Actions

Базовый workflow .github/workflows/ci.yml:

yaml

```
name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [18.x, 20.x]

    steps:
      - uses: actions/checkout@v3

      - name: Use Node.js ${ matrix.node-version }
        uses: actions/setup-node@v3
        with:
          node-version: ${ matrix.node-version }
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linting
        run: npm run lint

      - name: Run tests
        run: npm test

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          file: ./coverage/lcov.info

  build:
    runs-on: ubuntu-latest
    needs: test

    steps:
```

- **uses:** actions/checkout@v3
- **name:** Use Node.js 18.x
uses: actions/setup-node@v3
with:
node-version: '18.x'
cache: 'npm'
- **name:** Install dependencies
run: npm ci
- **name:** Build project
run: npm run build
- **name:** Upload build artifacts
uses: actions/upload-artifact@v3
with:
name: build-artifacts
path: dist/

6.3. Деплой на Vercel/Netlify

Конфигурация для Vercel (vercel.json):

```

json
{
  "version": 2,
  "builds": [
    {
      "src": "package.json",
      "use": "@vercel/static-build",
      "config": {
        "distDir": "dist"
      }
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "/dist/$1"
    }
  ],

```

```
"env": {
  "NODE_ENV": "production"
},
"github": {
  "enabled": true,
  "silent": false
}
}
```

Конфигурация для Netlify (netlify.toml):

```
toml

[build]
command = "npm run build"
publish = "dist"
functions = "functions"

[build.environment]
NODE_VERSION = "18"

[context.production.environment]
NODE_ENV = "production"

[[redirects]]
from = "/*"
to = "/index.html"
status = 200

[[headers]]
for = "/*"
[headers.values]
X-Frame-Options = "DENY"
X-Content-Type-Options = "nosniff"
Referrer-Policy = "strict-origin-when-cross-origin"
```

Часть 7: Оптимизация рабочего процесса

7.1. Полезные скрипты для package.json

```
json

{
  "scripts": {
```

```

"dev": "vite",
"build": "vite build",
"preview": "vite preview",
"start": "node src/server.js",
"start:dev": "nodemon src/server.js",
"start:debug": "node --inspect src/server.js",
"test": "jest",
"test:watch": "jest --watch",
"test:coverage": "jest --coverage",
"test:e2e": "cypress run",
"lint": "eslint src/",
"lint:fix": "eslint src/ --fix",
"format": "prettier --write \"src/**/*.{js,jsx,ts,tsx,json,css,md}\"",
"type-check": "tsc --noEmit",
"prepare": "husky install",
"prebuild": "npm run lint && npm run test",
"postbuild": "npm run analyze",
"analyze": "webpack-bundle-analyzer dist/stats.json",
"clean": "rm -rf dist node_modules coverage",
"clean:install": "npm run clean && npm install",
"docker:build": "docker build -t my-app .",
"docker:run": "docker run -p 3000:3000 my-app",
"storybook": "storybook dev -p 6006",
"build-storybook": "storybook build",
"cy:open": "cypress open",
"cy:run": "cypress run",
"release": "standard-version",
"release:major": "npm run release -- --release-as major",
"release:minor": "npm run release -- --release-as minor",
"release:patch": "npm run release -- --release-as patch"
}
}

```

7.2. Настройка VS Code для продуктивной работы

.vscode/settings.json для проекта:

```

json
{
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true,
    "source.organizeImports": true
  }
}

```

```

},
"editor.formatOnSave": true,
"editor.defaultFormatter": "esbenp.prettier-vscode",
"emmet.includeLanguages": {
  "javascript": "javascriptreact"
},
"javascript.suggest.autoImports": true,
"typescript.suggest.autoImports": true,
"files.associations": {
  ".env.*": "dotenv"
},
"search.exclude": {
  "**/node_modules": true,
  "**/dist": true,
  "**/coverage": true
},
"git.autofetch": true,
"git.confirmSync": false,
"terminal.integrated.env.linux": {
  "NODE_ENV": "development"
},
"terminal.integrated.env.windows": {
  "NODE_ENV": "development"
},
"terminal.integrated.env.osx": {
  "NODE_ENV": "development"
}
}

```

.vscode/extensions.json (рекомендуемые расширения):

```

json
{
  "recommendations": [
    "esbenp.prettier-vscode",
    "dbaeumer.vscode-eslint",
    "ms-vscode.vscode-typescript-next",
    "mikestead.dotenv",
    "eamodio.gitlens",
    "christian-kohler.npm-intellisense",
    "ms-vscode.vscode-js-profile-flame",
    "ms-vscode.live-server",
    "ritwickdey.liveserver"
  ]
}

```



```
}
```

.vscode/launch.json для отладки:

```
json
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Launch Program",
      "skipFiles": ["<node_internals>/**"],
      "program": "${workspaceFolder}/src/index.js",
      "outFiles": ["${workspaceFolder}/**/*.js"]
    },
    {
      "type": "chrome",
      "request": "launch",
      "name": "Launch Chrome",
      "url": "http://localhost:3000",
      "webRoot": "${workspaceFolder}/src"
    },
    {
      "type": "node",
      "request": "attach",
      "name": "Attach to Node",
      "port": 9229,
      "skipFiles": ["<node_internals>/**"]
    }
  ]
}
```

Заключение: чек-лист настройки

8.1. Базовый чек-лист

```
javascript
const setupChecklist = [
  // 1. Базовые инструменты
  '✓ Установлен Node.js (версия 18+)',
  '✓ Установлен Git',
```

```
'✓ Установлен VS Code с расширениями',

// 2. Инициализация проекта
'✓ Создана директория проекта',
'✓ Инициализирован npm (npm init)',
'✓ Создана структура каталогов',
'✓ Инициализирован Git репозиторий',
'✓ Создан .gitignore',

// 3. Конфигурация
'✓ Настроен ESLint (.eslintrc.js)',
'✓ Настроен Prettier (.prettierrc)',
'✓ Настроены Git Hooks (Husky)',
'✓ Настроен Commitlint',

// 4. Сборка и разработка
'✓ Выбран и настроен сборщик (Vite/Webpack)',
'✓ Настроен dev сервер',
'✓ Добавлены скрипты в package.json',

// 5. Тестирование
'✓ Настроен Jest (jest.config.js)',
'✓ Созданы примеры тестов',
'✓ Настроено покрытие кода',

// 6. Окружение
'✓ Созданы файлы .env*',
'✓ Настроена загрузка переменных окружения',

// 7. CI/CD
'✓ Настроен GitHub Actions workflow',
'✓ Настроена конфигурация для деплоя',

// 8. Документация
'✓ Создан README.md',
'✓ Документирована структура проекта',
'✓ Описаны команды для запуска'
];
```

8.2. Рекомендации по поддержке

1. Обновляйте зависимости регулярно

bash

`npm outdated` # Проверить устаревшие пакеты

`npm update` # Обновить пакеты

`npm audit fix` # Исправить уязвимости

2. Документируйте изменения

bash

Используйте стандартные коммиты

`git commit -m "feat: добавить новую функциональность"`

`git commit -m "fix: исправить баг с отображением"`

3. Настройте автоматизацию

- ➔ Автоматический запуск тестов при коммите
- ➔ Автоматическое форматирование кода
- ➔ Автоматический деплой при пуше в main

4. Следите за производительностью

- ➔ Используйте bundle analyzer
- ➔ Мониторьте размер бандла
- ➔ Оптимизируйте импорты

8.3. Быстрый старт для нового проекта

bash

1. Клонировать шаблон или используйте команды ниже

`npx create-vite@latest my-project --template vanilla`

2. Установите базовые зависимости

`cd my-project`

`npm install`

3. Установите инструменты разработки

`npm install --save-dev eslint prettier husky lint-staged`

4. Настройте конфигурации

`npx eslint --init`

`npx husky install`

5. Добавьте скрипты в package.json

6. Создайте структуру каталогов

7. Начните разработку!

`npm run dev`

Правильно настроенное окружение разработки — это инвестиция, которая многократно окупается в течение жизненного цикла проекта. Потратьте время на первоначальную настройку, чтобы сэкономить часы в будущем.

3.2. Подключение JavaScript к HTML-документу

Введение: способы интеграции JavaScript и HTML

JavaScript может быть подключен к HTML-документу несколькими способами, каждый из которых имеет свои особенности, преимущества и ограничения. Понимание различий критически важно для создания эффективных веб-приложений.

Часть 1: Базовые способы подключения

1.1. Встроенный JavaScript (Inline JavaScript)

Синтаксис и примеры:

html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Встроенный JavaScript</title>
</head>
<body>
  <!-- 1. Атрибут события onclick -->
  <button onclick="alert('Привет, мир!')">
    Нажми меня (inline onclick)
  </button>

  <!-- 2. Атрибут события onload для body -->
  <body onload="console.log('Документ загружен')">

  <!-- 3. Атрибут onsubmit для формы -->
  <form onsubmit="return validateForm()">
    <input type="text" id="username">
    <button type="submit">Отправить</button>
  </form>
```

```

<!-- 4. JavaScript в href (не рекомендуется) -->
<a href="javascript:void(0)" onclick="showMessage()">
  Ссылка с JavaScript
</a>

<!-- 5. Инлайн в обработчиках событий -->
<input type="text"
  onchange="console.log('Значение изменилось:', this.value)"
  onfocus="this.style.borderColor='blue'"
  onblur="this.style.borderColor='#ccc'">

<!-- 6. Непосредственное выполнение -->
<script>
  // JavaScript код непосредственно в HTML
  document.write('<p>Текст, добавленный через document.write</p>');

  function validateForm() {
    const username = document.getElementById('username').value;
    if (username.length < 3) {
      alert('Имя должно содержать минимум 3 символа');
      return false;
    }
    return true;
  }

  function showMessage() {
    alert('Ссылка была нажата!');
  }
</script>
</body>
</html>

```

Проблемы встроенного JavaScript:

html

```

<!-- 1. Смешивание логики и представления -->
<button onclick="
  fetch('/api/data')
  .then(response => response.json())
  .then(data => {
    document.getElementById('result').innerHTML = data.message;
    localStorage.setItem('lastData', JSON.stringify(data));
    updateChart(data.values);

```

```

    })
    .catch(error => console.error('Ошибка:', error));
">
    Загрузить данные
</button>
<!-- Слишком много логики в атрибуте! -->

<!-- 2. Повторение кода -->
<button onclick="updateCounter(1)">+1</button>
<button onclick="updateCounter(5)">+5</button>
<button onclick="updateCounter(10)">+10</button>
<!-- Одинаковая функция вызывается с разными параметрами -->

<!-- 3. Проблемы с безопасностью (XSS) -->
<div onclick="updateUser('<?php echo $userInput; ?>')">
    Кликни меня
</div>
<!-- $userInput может содержать вредоносный JavaScript -->

```

Когда использовать встроенный JavaScript (редкие случаи):

- Быстрое прототипирование
- Простые демонстрационные примеры
- Микровзаимодействия на статических страницах

1.2. Внутренние скрипты (Internal Script)

Синтаксис и расположение:

```

html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Внутренние скрипты</title>

  <!-- 1. Скрипт в <head> -->
  <script>
    // Этот код выполнится ДО загрузки DOM
    console.log('Скрипт в head выполняется');

    // Попытка доступа к элементу ВЫЗОВЕТ ОШИБКУ
    // console.log(document.getElementById('content')); // null
  </script>

```

```

// Функции можно объявлять - они будут доступны позже
function initApp() {
  console.log('App инициализирована');
}

// Обработчик события DOMContentLoaded
document.addEventListener('DOMContentLoaded', function() {
  console.log('DOM полностью загружен и разобран');
  initApp();
});
</script>
</head>
<body>
  <div id="content">
    <h1>Пример внутреннего скрипта</h1>
    <p id="message">Исходное сообщение</p>
    <button id="btn">Изменить текст</button>
  </div>

  <!-- 2. Скрипт в <body> (перед закрывающим тегом) -->
  <script>
    // Этот код выполнится ПОСЛЕ загрузки DOM до этой точки
    console.log('Скрипт в body выполняется');

    // Элементы ДО этого скрипта доступны
    const messageElement = document.getElementById('message');
    console.log('Сообщение:', messageElement.textContent);

    // Элементы ПОСЛЕ этого скрипта НЕ доступны
    // const btn = document.getElementById('btn'); // null (если скрипт выше кнопки)

    // Добавление обработчика
    document.getElementById('btn').addEventListener('click', function() {
      messageElement.textContent = 'Текст изменен!';
      messageElement.style.color = 'green';
    });
  </script>

  <!-- 3. Несколько скриптовых блоков -->
  <script>
    // Второй скриптовый блок
    console.log('Второй скрипт выполняется');
  </script>

```



```

// Все элементы ДО этого блока доступны
const btn = document.getElementById('btn');
console.log('Кнопка найдена:', btn !== null);

// Добавление второго обработчика
btn.addEventListener('mouseover', function() {
  this.style.backgroundColor = 'yellow';
});

btn.addEventListener('mouseout', function() {
  this.style.backgroundColor = '';
});
</script>

<!-- 4. Скрипт для конкретного элемента -->
<div id="dynamic-content"></div>

<script>
  // Скрипт, относящийся только к конкретному элементу
  const dynamicContent = document.getElementById('dynamic-content');

  // Генерация контента
  dynamicContent.innerHTML = `
    <h2>Динамически созданный контент</h2>
    <ul id="list">
      <li>Элемент 1</li>
      <li>Элемент 2</li>
      <li>Элемент 3</li>
    </ul>
  `;

  // Работа с динамически созданными элементами
  setTimeout(() => {
    const listItems = document.querySelectorAll('#list li');
    listItems.forEach((item, index) => {
      item.textContent = `Обновленный элемент ${index + 1}`;
    });
  }, 1000);
</script>
</body>
</html>

```

Области видимости во внутренних скриптах:

html

```
<script>
  // Глобальная область видимости (window)
  var globalVar = 'Я глобальная переменная';
  let moduleVar = 'Я переменная модуля'; // Не добавляется в window
  const constVar = 'Я константа'; // Не добавляется в window

  function globalFunction() {
    console.log('Я глобальная функция');
  }

  // Проверка
  console.log('globalVar в window:', window.globalVar); // Доступно
  console.log('moduleVar в window:', window.moduleVar); // undefined
  console.log('globalFunction в window:', window.globalFunction); // Доступно
</script>

<script>
  // Второй скрипт видит переменные из первого
  console.log('globalVar из другого скрипта:', globalVar); // Доступно
  console.log('moduleVar из другого скрипта:', moduleVar); // Доступно (в рамках документа)

  // Конфликт имен
  var globalVar = 'Перезаписанная переменная'; // Перезапишет существующую!

  // Использование IIFE для изоляции
  (function() {
    var privateVar = 'Приватная переменная';
    console.log('Внутри IIFE:', privateVar);
  })();

  // console.log(privateVar); // Ошибка: privateVar не определена
</script>
```

Часть 2: Внешние скрипты (External Scripts)

2.1. Базовое подключение внешних скриптов

Структура проекта:

text

```
my-project/
├── index.html
├── css/
│   └── style.css
├── js/
│   ├── main.js
│   ├── utils.js
│   ├── components.js
│   └── vendors/
│       └── library.js
└── assets/
    └── images/
```

Основные способы подключения:

html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Внешние скрипты</title>

  <!-- 1. Подключение в head (БЛОКИРУЮЩИЙ способ) -->
  <script src="js/main.js"></script>
  <!-- Браузер остановит парсинг HTML пока не загрузит и выполнит скрипт -->

  <!-- 2. Подключение в head с async -->
  <script src="js/utils.js" async></script>
  <!-- Скрипт загружается асинхронно и выполняется сразу после загрузки -->

  <!-- 3. Подключение в head с defer -->
  <script src="js/components.js" defer></script>
  <!-- Скрипт загружается асинхронно и выполняется после полного парсинга HTML -->

  <!-- 4. Абсолютные пути -->
  <script src="https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"></script>
  <script src="//code.jquery.com/jquery-3.6.0.min.js"></script>

  <!-- 5. Относительные пути -->
  <script src="../js/vendors/library.js"></script>
  <script src="../external/plugin.js"></script>

  <!-- 6. Скрипты с integrity check (безопасность) -->
```

```

<script
  src="https://example.com/script.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"
  crossorigin="anonymous">
</script>
</head>
<body>
  <!-- 7. Подключение в конце body (рекомендуемый способ) -->
  <h1>Мое приложение</h1>
  <div id="app"></div>

  <!-- Все скрипты подключены здесь -->
  <script src="js/main.js"></script>
  <script src="js/utils.js"></script>
  <script src="js/components.js"></script>

  <!-- 8. Модульные скрипты (ES6 Modules) -->
  <script type="module" src="js/app.js"></script>

  <!-- 9. Скрипты с nomodule для старых браузеров -->
  <script nomodule src="js/legacy.js"></script>
</body>
</html>

```

2.2. Атрибуты загрузки скриптов: async vs defer

Сравнение стратегий загрузки:

```

html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Async vs Defer</title>

  <!-- БЛОКИРУЮЩИЙ скрипт (без атрибутов) -->
  <script src="js/blocking.js">
    // ВРЕМЕННАЯ ДИАГРАММА:
    // 1. ↓ Парсинг HTML начинается
    // 2. ↓ Встречен тег <script>
    // 3. ↓ Парсинг HTML ПРИОСТАНАВЛИВАЕТСЯ
    // 4. ↓ Загрузка blocking.js

```

```

    // 5. ↓ Выполнение blocking.js
    // 6. ↓ Парсинг HTML ПРОДОЛЖАЕТСЯ
    // Проблема: блокирует отображение страницы!
</script>

<!-- ASYNC скрипт -->
<script src="js/async.js" async>
    // ВРЕМЕННАЯ ДИАГРАММА:
    // 1. ↓ Парсинг HTML начинается
    // 2. ↓ Встречен тег <script async>
    // 3. ↓ Загрузка async.js НАЧИНАЕТСЯ (асинхронно)
    // 4. ↓ Парсинг HTML ПРОДОЛЖАЕТСЯ параллельно
    // 5. ↓ Загрузка async.js ЗАВЕРШАЕТСЯ
    // 6. ↓ Парсинг HTML ПРИОСТАНАВЛИВАЕТСЯ
    // 7. ↓ Выполнение async.js
    // 8. ↓ Парсинг HTML ПРОДОЛЖАЕТСЯ
    // Особенность: порядок выполнения НЕ гарантирован!
</script>

<!-- DEFER скрипт -->
<script src="js/defer.js" defer>
    // ВРЕМЕННАЯ ДИАГРАММА:
    // 1. ↓ Парсинг HTML начинается
    // 2. ↓ Встречен тег <script defer>
    // 3. ↓ Загрузка defer.js НАЧИНАЕТСЯ (асинхронно)
    // 4. ↓ Парсинг HTML ПРОДОЛЖАЕТСЯ параллельно
    // 5. ↓ Парсинг HTML ЗАВЕРШАЕТСЯ
    // 6. ↓ Выполнение defer.js (в порядке указания в документе)
    // Преимущество: не блокирует парсинг, порядок сохраняется
</script>

<!-- Множественные скрипты с defer -->
<script src="js/vendor.js" defer></script>
<script src="js/app.js" defer></script>
<script src="js/analytics.js" defer></script>
<!-- Гарантированный порядок выполнения: vendor.js → app.js → analytics.js -->

<!-- Множественные скрипты с async -->
<script src="js/tracker.js" async></script>
<script src="js/ads.js" async></script>
<script src="js/social.js" async></script>
<!-- Порядок выполнения НЕ гарантирован! -->
</head>
<body>

```

```
<h1>Загрузка скриптов</h1>
```

```
<script>
```

```
  // Демонстрация порядка выполнения
```

```
  console.log('Встроенный скрипт выполняется');
```

```
  // Проверка загрузки внешних скриптов
```

```
  window.addEventListener('load', function() {
```

```
    console.log('Все ресурсы загружены');
```

```
  // Проверка, загружен ли скрипт
```

```
  const scripts = document.getElementsByTagName('script');
```

```
  Array.from(scripts).forEach((script, index) => {
```

```
    console.log(`Скрипт ${index}:`, {
```

```
      src: script.src,
```

```
      async: script.async,
```

```
      defer: script.defer,
```

```
      readyState: script.readyState
```

```
    });
```

```
  });
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

Практические рекомендации:

html

```
<head>
```

```
  <!-- 1. КРИТИЧЕСКИЕ скрипты, которые должны выполняться РАНЬШЕ всего -->
```

```
  <!-- Например: полифиллы для старых браузеров -->
```

```
  <script>
```

```
    // Полифилл для современных функций
```

```
    if (!window.Promise) {
```

```
      document.write('<script src="polyfills/promise.js"></script>');
```

```
    }
```

```
  </script>
```

```
  <!-- 2. Скрипты, НЕ зависящие от DOM, можно загружать с async -->
```

```
  <!-- Например: аналитика, метрики, трекеры -->
```

```
  <script src="https://www.google-analytics.com/analytics.js" async></script>
```

```
  <script src="https://connect.facebook.net/en_US/sdk.js" async></script>
```

```
  <!-- 3. Скрипты, ЗАВИСЯЩИЕ от DOM и других скриптов - используйте defer -->
```

```

<!-- Например: основные библиотеки и код приложения -->
<script src="js/vendor/jquery.min.js" defer></script>
<script src="js/vendor/bootstrap.js" defer></script>
<script src="js/app.js" defer></script>

<!-- 4. Скрипты, которые МЕНЯЮТ DOM в head - только блокирующие -->
<!-- Например: динамическое изменение title или meta-тегов -->
<script>
  // Динамическое изменение title
  document.title = "Новый заголовок";

  // Добавление meta-тегов
  const meta = document.createElement('meta');
  meta.name = 'description';
  meta.content = 'Динамическое описание';
  document.head.appendChild(meta);
</script>
</head>

```

2.3. Динамическая загрузка скриптов

Методы динамического добавления скриптов:

```

html

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Динамическая загрузка скриптов</title>
  <script>
    // 1. Базовый метод создания и добавления скрипта
    function loadScript(src, callback) {
      const script = document.createElement('script');
      script.src = src;
      script.type = 'text/javascript';

      // Обработка загрузки
      script.onload = function() {
        console.log(`Скрипт ${src} загружен`);
        if (callback) callback();
      };
    };
  </script>

```

```

script.onerror = function() {
  console.error(`Ошибка загрузки скрипта: ${src}`);
  if (callback) callback(new Error(`Не удалось загрузить ${src}`));
};

// Добавление в документ
document.head.appendChild(script);
}

// 2. Загрузка с поддержкой async/defer
function loadScriptAdvanced(src, options = {}) {
  return new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = src;

    // Настройка атрибутов
    if (options.async) script.async = true;
    if (options.defer) script.defer = true;
    if (options.type) script.type = options.type;
    if (options.crossOrigin) script.crossOrigin = options.crossOrigin;
    if (options.integrity) {
      script.integrity = options.integrity;
      script.crossOrigin = 'anonymous';
    }

    // Обработчики
    script.onload = () => {
      console.log(`✓ ${src} загружен`);
      resolve(script);
    };

    script.onerror = () => {
      console.error(`✗ ${src} не загружен`);
      reject(new Error(`Не удалось загрузить ${src}`));
    };

    // Добавление
    const position = options.position || 'head';
    if (position === 'head') {
      document.head.appendChild(script);
    } else {
      document.body.appendChild(script);
    }
  });
}

```



```
}
```

```
// 3. Загрузка нескольких скриптов последовательно
```

```
async function loadScriptsSequentially(scripts) {  
  for (const src of scripts) {  
    try {  
      await loadScriptAdvanced(src);  
      console.log(`Загружен: ${src}`);  
    } catch (error) {  
      console.warn(`Пропущен: ${src}`, error);  
      continue;  
    }  
  }  
  console.log('Все скрипты загружены');  
}
```

```
// 4. Загрузка нескольких скриптов параллельно
```

```
function loadScriptsParallel(scripts) {  
  return Promise.allSettled(  
    scripts.map(src => loadScriptAdvanced(src))  
  ).then(results => {  
    const loaded = results.filter(r => r.status === 'fulfilled');  
    const failed = results.filter(r => r.status === 'rejected');  
  
    console.log(`Успешно: ${loaded.length}, Ошибок: ${failed.length}`);  
    return { loaded, failed };  
  });  
}
```

```
// 5. Lazy Loading скриптов при взаимодействии
```

```
function setupLazyLoading() {  
  // Скрипты, которые загружаются при определенных условиях  
  const lazyScripts = {  
    'chart': {  
      src: 'js/vendors/chart.js',  
      loaded: false,  
      condition: () => document.querySelector('.chart-container')  
    },  
    'map': {  
      src: 'js/vendors/map.js',  
      loaded: false,  
      condition: () => document.querySelector('#map')  
    },  
    'video': {
```

```

        src: 'js/vendors/video-player.js',
        loaded: false,
        condition: () => document.querySelector('video')
    }
};

// Проверка условий при прокрутке
window.addEventListener('scroll', function() {
    Object.keys(lazyScripts).forEach(key => {
        const script = lazyScripts[key];
        if (!script.loaded && script.condition()) {
            loadScriptAdvanced(script.src);
            script.loaded = true;
        }
    });
}, { passive: true });
</script>
</head>
<body>
    <h1>Динамическая загрузка скриптов</h1>

    <button onclick="loadScript('js/utils.js', () => console.log('Utils готов'))">
        Загрузить Utils
    </button>

    <button onclick="loadScriptAdvanced('js/chart.js', { async: true })">
        Загрузить Chart.js
    </button>

    <button onclick="loadScriptsSequentially(['js/lib1.js', 'js/lib2.js', 'js/app.js'])">
        Загрузить последовательно
    </button>

    <button onclick="loadScriptsParallel(['js/vendor1.js', 'js/vendor2.js',
'js/vendor3.js'])">
        Загрузить параллельно
    </button>

    <div class="chart-container" style="height: 300px; margin-top: 50px;">
        <!-- Контейнер для графика -->
    </div>

    <script>

```

```

// Инициализация ленивой загрузки
setupLazyLoading();

// Динамическая загрузка при необходимости
if (window.location.search.includes('admin')) {
    loadScriptAdvanced('js/admin.js', { defer: true });
}

// Загрузка скрипта только на мобильных устройствах
if (/Android|webOS|iPhone|iPad|iPod|BlackBerry|IEMobile|Opera
Mini/i.test(navigator.userAgent)) {
    loadScriptAdvanced('js/touch-events.js');
}
</script>
</body>
</html>

```

Паттерны динамической загрузки:

javascript

```

// 1. Загрузка по требованию (On-demand Loading)
class ScriptLoader {
    constructor() {
        this.loadedScripts = new Map();
        this.loadingScripts = new Map();
    }

    load(src, options = {}) {
        // Если уже загружен
        if (this.loadedScripts.has(src)) {
            return Promise.resolve(this.loadedScripts.get(src));
        }

        // Если уже загружается
        if (this.loadingScripts.has(src)) {
            return this.loadingScripts.get(src);
        }

        // Создание нового обещания
        const promise = new Promise((resolve, reject) => {
            const script = document.createElement('script');
            script.src = src;

            if (options.type === 'module') {

```

```

        script.type = 'module';
    }

    script.onload = () => {
        this.loadedScripts.set(src, script);
        this.loadingScripts.delete(src);
        resolve(script);
    };

    script.onerror = (error) => {
        this.loadingScripts.delete(src);
        reject(error);
    };

    document.head.appendChild(script);
});

this.loadingScripts.set(src, promise);
return promise;
}

// Загрузка с зависимостями
loadWithDependencies(mainScript, dependencies = []) {
    return Promise.all(dependencies.map(dep => this.load(dep)))
        .then(() => this.load(mainScript));
}

// Использование
const loader = new ScriptLoader();
loader.loadWithDependencies('js/app.js', ['js/vendor.js', 'js/utils.js'])
    .then(() => {
        console.log('Все скрипты загружены');
        window.app.init();
    });

// 2. PreLoad u Prefetch
function setupResourceHints() {
    // PreLoad - критически важные ресурсы
    const preload = document.createElement('link');
    preload.rel = 'preload';
    preload.href = 'js/critical.js';
    preload.as = 'script';
    document.head.appendChild(preload);
}

```

```

// Prefetch - ресурсы, которые понадобятся позже
const prefetch = document.createElement('link');
prefetch.rel = 'prefetch';
prefetch.href = 'js/non-critical.js';
prefetch.as = 'script';
document.head.appendChild(prefetch);

// Preconnect - установка соединения заранее
const preconnect = document.createElement('link');
preconnect.rel = 'preconnect';
preconnect.href = 'https://api.example.com';
document.head.appendChild(preconnect);

// DNS-prefetch - предварительное разрешение DNS
const dnsPrefetch = document.createElement('link');
dnsPrefetch.rel = 'dns-prefetch';
dnsPrefetch.href = 'https://cdn.example.com';
document.head.appendChild(dnsPrefetch);
}

```

Часть 3: Модульная система ES6

3.1. Использование ES6 Modules

Базовая структура модулей:

```

text

project/
├── index.html
├── js/
│   ├── main.js           # Главный модуль
│   ├── math.js           # Модуль с функциями
│   ├── user.js           # Модуль с классами
│   └── utils/
│       ├── validator.js
│       └── formatter.js

```

HTML с модулями:

```
html
```

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>ES6 Modules</title>

  <!-- Основной модуль приложения -->
  <script type="module" src="js/main.js"></script>

  <!-- Модуль с nomodule фолбэком -->
  <script type="module" src="js/modern.js"></script>
  <script nomodule src="js/legacy.js"></script>

  <!-- Встроенный модуль -->
  <script type="module">
    // Импорт из внешнего модуля
    import { greet } from './js/utils/greeter.js';

    // Использование импортированной функции
    document.addEventListener('DOMContentLoaded', () => {
      greet('Мир');
    });
  </script>
</head>
<body>
  <h1>ES6 Modules Demo</h1>
  <div id="app"></div>

  <!-- Динамический импорт -->
  <button id="load-chart">Загрузить график</button>
  <div id="chart-container"></div>

  <script type="module">
    // Динамический импорт по требованию
    document.getElementById('load-chart').addEventListener('click', async () => {
      try {
        // Загрузка модуля только при необходимости
        const chartModule = await import('./js/chart.js');
        chartModule.renderChart('#chart-container');
      } catch (error) {
        console.error('Ошибка загрузки модуля:', error);
      }
    });
  </script>
```

```
</body>
</html>
```

Пример модулей JavaScript:

javascript

// js/math.js - Модуль с математическими функциями

```
export const PI = 3.141592653589793;
```

```
export function add(a, b) {
  return a + b;
}
```

```
export function multiply(a, b) {
  return a * b;
}
```

```
export function circleArea(radius) {
  return PI * radius * radius;
}
```

// Экспорт по умолчанию

```
export default class Calculator {
  constructor() {
    this.result = 0;
  }

  calculate(operation, ...args) {
    switch(operation) {
      case 'add':
        this.result = args.reduce((sum, num) => sum + num, 0);
        break;
      case 'multiply':
        this.result = args.reduce((product, num) => product * num, 1);
        break;
    }
    return this.result;
  }
}
```

javascript

// js/user.js - Модуль с классами пользователей

// Именованный экспорт класса

```
export class User {
  constructor(name, email) {
```

```
    this.name = name;
    this.email = email;
    this.createdAt = new Date();
  }

  greet() {
    return `Привет, я ${this.name}!`;
  }
}

// Экспорт константы
export const DEFAULT_ROLE = 'user';

// Экспорт функции
export function createAdmin(name, email) {
  const user = new User(name, email);
  user.role = 'admin';
  return user;
}

// Реекспорт из другого модуля
export { validateEmail } from './utils/validator.js';
javascript

// js/main.js - Главный модуль приложения
// Импорт именованных экспортов
import { PI, add, circleArea } from './math.js';

// Импорт экспорта по умолчанию
import Calculator from './math.js';

// Импорт всего модуля как объекта
import * as MathUtils from './math.js';

// Импорт с переименованием
import { add as sumNumbers } from './math.js';

// Импорт из поддиректории
import { validateEmail } from './utils/validator.js';
import { formatDate } from './utils/formatter.js';

// Использование импортированных функций
console.log('PI:', PI);
console.log('2 + 3 =', add(2, 3));
```



```

console.log('Площадь круга:', circleArea(5));

// Использование класса
const calc = new Calculator();
console.log('Сумма:', calc.calculate('add', 1, 2, 3, 4, 5));

// Использование импорта "звездочка"
console.log('Умножение через MathUtils:', MathUtils.multiply(4, 5));

// Импорт JSON (требуется флага --experimental-json-modules в Node.js)
// import config from './config.json' assert { type: 'json' };

// Динамический импорт
async function loadModuleDynamically() {
  if (window.location.pathname === '/dashboard') {
    const dashboardModule = await import('./dashboard.js');
    dashboardModule.init();
  }
}

// Инициализация приложения
document.addEventListener('DOMContentLoaded', () => {
  console.log('Приложение инициализировано');
  loadModuleDynamically();

  // Проверка поддержки модулей
  if (typeof window !== 'undefined' && 'noModule' in HTMLScriptElement.prototype) {
    console.log('ES6 Modules поддерживаются');
  } else {
    console.log('ES6 Modules не поддерживаются');
  }
});

// Экспорт для использования в других модулях
export { loadModuleDynamically };

```

3.2. Особенности модулей

javascript

```

// Особенности ES6 Modules:

// 1. Строгий режим по умолчанию
// Весь код в модулях выполняется в strict mode

```

```
// Нельзя использовать необъявленные переменные

// 2. Собственная область видимости
var moduleVar = 'Я в модуле';
console.log(moduleVar); // Работает
// console.log(window.moduleVar); // undefined - не добавляется в глобальный объект

// 3. Загрузка только один раз
// Модуль загружается и выполняется только один раз
// Последующие импорты получают ту же самую ссылку

// 4. Статический импорт
// Импорты должны быть на верхнем уровне
// Нельзя импортировать внутри условий или функций
// if (condition) {
//     import { something } from './module.js'; // Синтаксическая ошибка!
// }

// 5. Деферрированное выполнение
// Модули автоматически получают defer
// Выполняются после полного парсинга документа

// 6. CORS ограничения
// Модули загружаются с CORS заголовками
// Для локальных файлов нужен HTTP сервер

// 7. Поддержка циклических зависимостей
// Модули А и В могут импортировать друг друга
```

Часть 4: Современные подходы и лучшие практики

4.1. Оптимизация загрузки

```
html

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Оптимизация загрузки</title>

  <!-- 1. PreLoad критических ресурсов -->
```

```

<link rel="preload" href="js/critical.js" as="script">
<link rel="preload" href="css/critical.css" as="style">

<!-- 2. Preconnect к важным доменам -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://cdn.example.com">

<!-- 3. DNS-prefetch для сторонних ресурсов -->
<link rel="dns-prefetch" href="https://analytics.example.com">

<!-- 4. Фолбэк для отключенного JavaScript -->
<noscript>
  <style>
    .js-warning { display: block; }
  </style>
</noscript>
</head>
<body>
  <!-- Предупреждение для пользователей без JavaScript -->
  <div class="js-warning" style="display: none; background: #ffeb3b; padding: 20px;">
    <h2>Внимание!</h2>
    <p>Для работы этого сайта требуется JavaScript. Пожалуйста, включите его в настройках
браузера.</p>
  </div>

  <!-- Основное содержимое -->
  <div id="app">
    <h1>Оптимизированная загрузка</h1>
  </div>

  <!-- 5. Скрипты в конце body -->
  <!-- Критические скрипты -->
  <script src="js/critical.js" defer></script>

  <!-- 6. Lazy Load остальных скриптов -->
  <script>
    // Загрузка не критических скриптов после загрузки страницы
    window.addEventListener('load', function() {
      // Создание массива скриптов для ленивой загрузки
      const lazyScripts = [
        { src: 'js/analytics.js', async: true },
        { src: 'js/social.js', async: true },
        { src: 'js/chat.js', defer: true }
      ];

```

```

// Динамическая загрузка
lazyScripts.forEach(scriptConfig => {
  const script = document.createElement('script');
  script.src = scriptConfig.src;
  if (scriptConfig.async) script.async = true;
  if (scriptConfig.defer) script.defer = true;
  document.body.appendChild(script);
});
});
</script>

<!-- 7. Fallback для старых браузеров -->
<script>
  // Проверка поддержки современных функций
  const supportsModules = 'noModule' in HTMLScriptElement.prototype;
  const supportsFetch = 'fetch' in window;

  if (!supportsModules || !supportsFetch) {
    // Загрузка полифиллов для старых браузеров
    const polyfillScript = document.createElement('script');
    polyfillScript.src = 'js/polyfills.js';
    polyfillScript.nomodule = true; // Только для браузеров без поддержки modules
    document.body.appendChild(polyfillScript);
  }
</script>
</body>
</html>

```

4.2. Безопасность подключения скриптов

```

html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Безопасное подключение скриптов</title>

  <!-- 1. Content Security Policy (CSP) -->
  <meta http-equiv="Content-Security-Policy"
    content="default-src 'self';
      script-src 'self' https://trusted.cdn.com;
      style-src 'self' 'unsafe-inline';

```

```
img-src 'self' data: https;;
connect-src 'self' https://api.example.com;">
```

<!-- 2. Subresource Integrity (SRI) -->

```
<script
  src="https://cdn.example.com/library.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"
  crossorigin="anonymous">
</script>
```

<!-- 3. Запрет на выполнение встроенных скриптов -->

```
<meta http-equiv="Content-Security-Policy"
  content="script-src 'self'">
<!-- Запрещает <script>...</script> и onclick="..." -->
```

<!-- 4. Использование nonce для разрешенных скриптов -->

```
<?php
$nonce = base64_encode(random_bytes(16));
?>
<meta http-equiv="Content-Security-Policy"
  content="script-src 'nonce-<?php echo $nonce; ?>'">

<script nonce="<?php echo $nonce; ?>">
  // Этот скрипт разрешен CSP
  console.log('Разрешенный скрипт');
</script>
```

<!-- 5. Запрет на iframe embedding -->

```
<meta http-equiv="X-Frame-Options" content="DENY">
```

<!-- 6. Отключение MIME type sniffing -->

```
<meta http-equiv="X-Content-Type-Options" content="nosniff">
```

```
</head>
```

```
<body>
```

```
<h1>Безопасное подключение</h1>
```

<!-- 7. Безопасная обработка пользовательского ввода -->

```
<div id="user-content"></div>
```

```
<script>
```

```
  // БЕЗОПАСНЫЙ способ вставки HTML
```

```
  function safeInsertHTML(element, html) {
```

```
    // Использование textContent вместо innerHTML для текста
```

```
    if (!/<[>]*>/>/.test(html)) {
```

```

        element.textContent = html;
        return;
    }

    // Санитизация HTML перед вставкой
    const sanitized = sanitizeHTML(html);
    element.innerHTML = sanitized;
}

// Простая функция санитизации
function sanitizeHTML(html) {
    const temp = document.createElement('div');
    temp.textContent = html;
    return temp.innerHTML;
}

// ОПАСНО: может привести к XSS
// document.getElementById('user-content').innerHTML = userInput;

// БЕЗОПАСНО: используем санитизацию
const userInput = '<script>alert("XSS")</script>';
safeInsertHTML(document.getElementById('user-content'), userInput);
</script>

<!-- 8. Отключение автоматического выполнения скриптов -->
<script>
    // Использование defer/async вместо блокирующих скриптов
    // Избегайте document.write()

    // Правильно:
    // <script src="app.js" defer></script>

    // Неправильно:
    // <script src="app.js"></script>
    // document.write('<script src="dynamic.js"></script>');
</script>
</body>
</html>

```

4.3. Производительность и мониторинг

```

html
<!DOCTYPE html>

```

```
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Производительность загрузки</title>

  <!-- 1. Resource Hints -->
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link rel="dns-prefetch" href="https://cdn.example.com">
  <link rel="preload" as="script" href="js/critical.js">

  <!-- 2. Предзагрузка шрифтов -->
  <link rel="preload"
    href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
    as="style"
    onload="this.onload=null;this.rel='stylesheet'">
  <noscript>
    <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
      rel="stylesheet">
  </noscript>
</head>
<body>
  <h1>Мониторинг производительности</h1>

  <script>
    // 3. Мониторинг времени загрузки
    const perfData = {
      navigationStart: performance.timing.navigationStart,
      domContentLoaded: null,
      load: null,
      scripts: []
    };

    // Отслеживание DOMContentLoaded
    document.addEventListener('DOMContentLoaded', () => {
      perfData.domContentLoaded = performance.now();
      console.log(`DOMContentLoaded: ${perfData.domContentLoaded}ms`);
    });

    // Отслеживание полной загрузки
    window.addEventListener('load', () => {
      perfData.load = performance.now();
      console.log(`Полная загрузка: ${perfData.load}ms`);
    });

    // Отправка метрик
```

```
    sendMetrics(perfData);
  });
```

```
// 4. Мониторинг загрузки скриптов
```

```
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach(entry => {
    if (entry.initiatorType === 'script') {
      perfData.scripts.push({
        name: entry.name,
        duration: entry.duration,
        startTime: entry.startTime
      });

      console.log(`Скрипт ${entry.name}: ${entry.duration}ms`);
    }
  });
});
```

```
observer.observe({ entryTypes: ['resource'] });
```

```
// 5. Отложенная загрузка тяжелых скриптов
```

```
function loadHeavyScriptsOnIdle() {
  if ('requestIdleCallback' in window) {
    requestIdleCallback(() => {
      loadScript('js/heavy-library.js', { defer: true });
    });
  } else {
    // Фолбэк для старых браузеров
    setTimeout(() => {
      loadScript('js/heavy-library.js', { defer: true });
    }, 3000);
  }
}
```

```
// 6. Кэширование скриптов
```

```
function cacheScript(src) {
  if ('caches' in window) {
    caches.open('scripts-v1').then(cache => {
      cache.add(src).then(() => {
        console.log(`${src} добавлен в кэш`);
      });
    });
  }
}
```



```

// 7. Восстановление при ошибках загрузки
function loadScriptWithFallback(src, fallbacks = []) {
  return new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = src;

    script.onload = () => resolve(script);
    script.onerror = () => {
      console.warn(`${src} не загружен, пробуем фолбэк`);
      if (fallbacks.length > 0) {
        const nextSrc = fallbacks.shift();
        loadScriptWithFallback(nextSrc, fallbacks)
          .then(resolve)
          .catch(reject);
      } else {
        reject(new Error('Все источники недоступны'));
      }
    };
  });

  document.head.appendChild(script);
});

// Использование фолбэков
loadScriptWithFallback('https://cdn1.example.com/library.js', [
  'https://cdn2.example.com/library.js',
  'js/local-fallback.js'
]);
</script>
</body>
</html>

```

Заключение: лучшие практики

Рекомендации по подключению JavaScript:

1. Используйте внешние скрипты вместо встроенных
2. Размещайте скрипты перед закрывающим `</body>`
3. Используйте `defer` для скриптов, зависящих от DOM
4. Используйте `async` для независимых сторонних скриптов
5. Минимизируйте количество скриптов (объединяйте, если возможно)

6. **Используйте модули ES6** для современных проектов
7. **Реализуйте ленивую загрузку** для некритичных скриптов
8. **Всегда добавляйте** `nomodule` фолбэки для старых браузеров

Проверочный список:

javascript

```
const scriptChecklist = [  
  // Расположение  
  '✓ Скрипты перед </body> (кроме критических)',  
  '✓ Используются defer/async где возможно',  
  '✓ Минимальное количество скриптовых тегов',  
  
  // Безопасность  
  '✓ Используется CSP',  
  '✓ Внешние скрипты имеют SRI',  
  '✓ Нет встроенных скриптов с пользовательским вводом',  
  
  // Производительность  
  '✓ Критические скрипты предзагружены',  
  '✓ Реализована ленивая загрузка',  
  '✓ Используются resource hints',  
  '✓ Скрипты сжаты и минифицированы',  
  
  // Совместимость  
  '✓ Есть фолбэки для старых браузеров',  
  '✓ Реализована обработка ошибок загрузки',  
  '✓ Тестировано с отключенным JavaScript',  
  
  // Современные подходы  
  '✓ Используются ES6 модули (type="module")',  
  '✓ Реализован динамический импорт',  
  '✓ Модули разделены по функциональности'  
];
```

Шаблон для нового проекта:

html

```
<!DOCTYPE html>  
<html lang="ru">  
<head>  
  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Мой проект</title>

<!-- Resource Hints -->
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="dns-prefetch" href="https://cdn.example.com">

<!-- Preload критических ресурсов -->
<link rel="preload" href="css/critical.css" as="style">
<link rel="preload" href="js/critical.js" as="script">

<!-- CSP (настройте под свой проект) -->
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self';
              script-src 'self' 'unsafe-inline' https://trusted.cdn.com;
              style-src 'self' 'unsafe-inline'">

<!-- Основные стили -->
<link rel="stylesheet" href="css/main.css">

<!-- Иконка -->
<link rel="icon" href="favicon.ico">
</head>
<body>
  <!-- Контент при отключенном JS -->
  <noscript>
    <div class="noscript-warning">
      <h2>Требуется JavaScript</h2>
      <p>Для работы сайта необходимо включить JavaScript в настройках браузера.</p>
    </div>
  </noscript>

  <!-- Основное содержимое -->
  <div id="app">
    <h1>Загрузка...</h1>
  </div>

  <!-- Критические скрипты -->
  <script src="js/critical.js" defer></script>

  <!-- Основное приложение -->
  <script type="module" src="js/app.js"></script>

  <!-- Фолбэк для старых браузеров -->
```

```
<script nomodule src="js/legacy.js"></script>

<!-- Ленивая загрузка остальных скриптов -->
<script>
  // Загрузка не критических скриптов после полной загрузки
  window.addEventListener('load', function() {
    const lazyScripts = [
      'js/analytics.js',
      'js/social.js',
      'js/chat.js'
    ];

    lazyScripts.forEach(src => {
      const script = document.createElement('script');
      script.src = src;
      script.async = true;
      document.body.appendChild(script);
    });
  });
</script>
</body>
</html>
```

Правильное подключение JavaScript — это баланс между производительностью, безопасностью и функциональностью. Современные подходы позволяют создавать быстрые, безопасные и отзывчивые веб-приложения.

3.3. Исполнение кода: браузер vs консоль vs Node.js

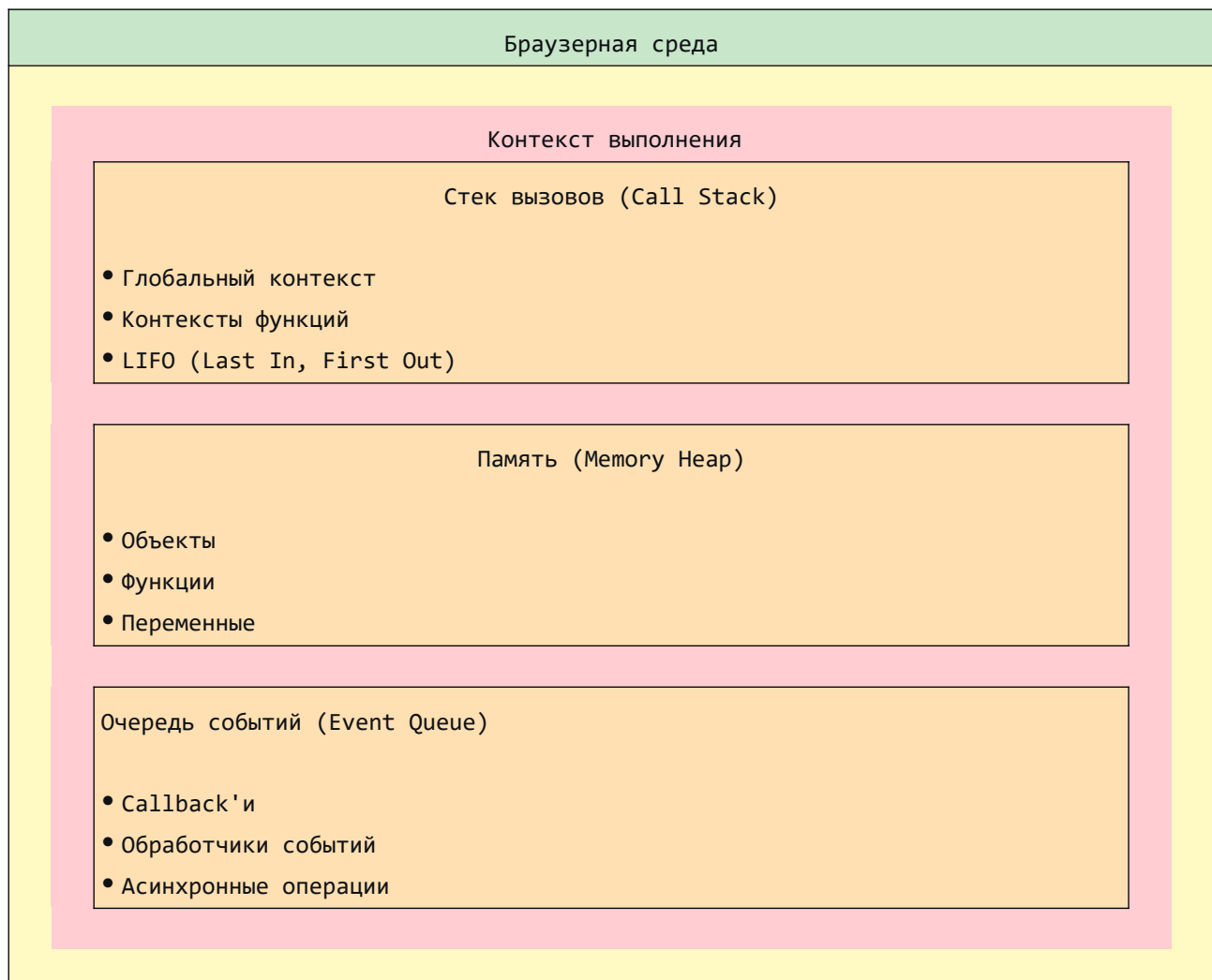
Введение: разные среды выполнения JavaScript

JavaScript — уникальный язык, который может выполняться в различных средах, каждая со своей спецификой и особенностями. Понимание различий критически важно для эффективной разработки.

Часть 1: Браузер как среда выполнения

1.1. Архитектура выполнения JavaScript в браузере

Text



Цикл событий (Event Loop)

```
while (queue.waitForMessage()) {  
  queue.processNextMessage();  
}
```

1.2. Специфика выполнения в браузере

Глобальный объект window:

javascript

```
// В браузере глобальный объект - window  
console.log(this === window); // true (в глобальной области)  
  
// Все глобальные переменные становятся свойствами window  
var globalVar = 'Я в window';  
console.log(window.globalVar); // 'Я в window'  
  
let modernVar = 'He в window';  
console.log(window.modernVar); // undefined  
  
// Глобальные функции  
function greet() {  
  return 'Hello';  
}  
console.log(window.greet()); // 'Hello'
```

Доступ к Web API:

javascript

```
// Браузер предоставляет специфичные API  
console.log('Доступные API в браузере:');  
  
// 1. DOM API  
const element = document.getElementById('myElement');  
element.addEventListener('click', () => console.log('Клик!'));  
  
// 2. BOM (Browser Object Model)  
console.log('URL:', window.location.href);  
console.log('Размер экрана:', window.screen.width);  
console.log('История переходов:', window.history.length);
```

```

// 3. Web Storage
localStorage.setItem('key', 'value');
const data = sessionStorage.getItem('data');

// 4. Geolocation
navigator.geolocation.getCurrentPosition(position => {
  console.log('Широта:', position.coords.latitude);
});

// 5. Canvas API
const canvas = document.createElement('canvas');
const ctx = canvas.getContext('2d');
ctx.fillRect(10, 10, 100, 100);

// 6. Audio/Video API
const audio = new Audio('sound.mp3');
audio.play();

// 7. Web Workers
if (window.Worker) {
  const worker = new Worker('worker.js');
  worker.postMessage('start');
}

// 8. Service Workers
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js');
}

```

Ограничения безопасности:

javascript

```

// 1. Same-Origin Policy (Политика одинакового источника)
// fetch('https://api.other-domain.com/data') // Ошибка CORS

// 2. Sandbox (песочница)
// Нельзя получить доступ к файловой системе
// try { require('fs') } catch(e) { console.log('Нет доступа к fs') }

// 3. Ограничения памяти
// Бесконечный цикл может "завесить" вкладку
// while(true) { console.log('Завис!') } // Браузер предложит остановить

// 4. Попапы блокировщики

```

```
// window.open('https://example.com') // Может быть заблокировано
```

```
// 5. Автозапуск медиа
```

```
// <video autoplay> может не работать без взаимодействия пользователя
```

1.3. Event Loop и асинхронность

javascript

```
// Демонстрация Event Loop в браузере
```

```
console.log('1. Начало скрипта');
```

```
// Макрозадача (setTimeout)
```

```
setTimeout(() => {  
  console.log('2. setTimeout callback');  
}, 0);
```

```
// Микрозадача (Promise)
```

```
Promise.resolve().then(() => {  
  console.log('3. Promise resolved');  
});
```

```
// Синхронный код
```

```
console.log('4. Конец синхронного кода');
```

```
// Результат выполнения:
```

```
// 1. Начало скрипта
```

```
// 4. Конец синхронного кода
```

```
// 3. Promise resolved
```

```
// 2. setTimeout callback
```

```
// Объяснение:
```

```
// 1. Выполняется синхронный код
```

```
// 2. Микрозадачи (Promise) выполняются перед макрозадачами
```

```
// 3. Макрозадачи (setTimeout) выполняются после
```

```
// Анимации и requestAnimationFrame
```

```
function animate() {  
  console.log('Анимация кадра');  
  requestAnimationFrame(animate); // Синхронизировано с обновлением экрана  
}
```

```
// requestAnimationFrame(animate);
```

```
// MutationObserver для наблюдения за DOM
```



```
const observer = new MutationObserver(mutations => {
  console.log('DOM изменился', mutations);
});
observer.observe(document.body, { childList: true });
```

1.4. Загрузка и выполнение кода

```
html

<!DOCTYPE html>
<html>
<head>
  <script>
    // Этап 1: Парсинг и выполнение скриптов в head
    console.log('1. Head script executing');
    console.log('Документ загружен?', document.readyState); // Loading
  </script>
</head>
<body onload="console.log('5. Body onload')">
  <div id="content">Контент</div>

  <script>
    // Этап 2: Выполнение inline скрипта в body
    console.log('2. Inline script executing');
    console.log('Элемент доступен?', document.getElementById('content') !== null);
  </script>

  <!-- Внешний скрипт -->
  <script src="external.js" defer></script>

  <script>
    // Этап 3: DOMContentLoaded event
    document.addEventListener('DOMContentLoaded', () => {
      console.log('3. DOMContentLoaded fired');
    });

    // Этап 4: Load event
    window.addEventListener('load', () => {
      console.log('4. Window load fired');
    });

    // Этап 5: Beforeunload
    window.addEventListener('beforeunload', (event) => {
      console.log('6. Beforeunload fired');
```

```
        // event.preventDefault(); // Для подтверждения закрытия
        // event.returnValue = '';
    });
</script>
</body>
</html>
```

```
// Файл external.js
console.log('External script loaded (deferred)');
```

```
/*
Последовательность выполнения:
1. Head script executing
2. Inline script executing
3. External script loaded (deferred)
4. DOMContentLoaded fired
5. Body onload
6. Window load fired
7. Beforeunload fired (при закрытии)
*/
```

Часть 2: Консоль браузера как REPL

2.1. Особенности выполнения в консоли

Консоль Developer Tools:

javascript

// Консоль - это REPL (Read-Eval-Print Loop) среда

// 1. Автоматический вывод результата

5 + 3 // Выведет: 8

const x = 10 // Выведет: undefined (присваивание не возвращает значение)

x // Выведет: 10

// 2. Специальные переменные консоли

\$0 // Последний выбранный элемент в Elements panel

\$_ // Результат последнего вычисления

\$('.div') // Аналог document.querySelector

\$\$('div') // Аналог document.querySelectorAll

```
// 3. Команды консоли
console.clear() // Очистка консоли
console.time('label') // Начать таймер
console.timeEnd('label') // Завершить таймер и вывести время
```

```
// 4. Многострочный режим (Shift+Enter)
```

```
// Можно вводить:
```

```
function multiLine() {
  const a = 1;
  const b = 2;
  return a + b;
}
```

```
// Нажать Enter для выполнения
```

```
// 5. Сохранение состояния между вводами
```

```
let counter = 0;
function increment() {
  counter++;
  return counter;
}
increment(); // 1
increment(); // 2
counter; // 2 (сохраняется между вызовами)
```

```
// 6. Автодополнение (Tab)
```

```
// Начните вводить 'docu' и нажмите Tab
```

```
// document → document.querySelector → и т.д.
```

Различия в поведении:

javascript

```
// В консоли vs в скрипте:
```

```
// 1. Объявление переменных (разное поведение var)
```

```
var y = 5 // В консоли: создает глобальную переменную
// В скрипте: создает переменную в текущей области видимости
```

```
// 2. Strict mode
```

```
// Консоль по умолчанию НЕ в strict mode (в большинстве браузеров)
```

```
// function test() {
```

```
//   undeclared = 10; // В консоли: создаст глобальную переменную
```

```
//   // В скрипте с 'use strict': ReferenceError
```

```
// }
```

```
// 3. Top-Level await (доступен в консоли)
// В консоли можно:
const response = await fetch('https://api.example.com/data')
const data = await response.json()

// В обычном скрипте (не модуле) это вызовет ошибку

// 4. Обработка ошибок
// В консоли ошибки не "ломают" выполнение
undefinedFunction() // Ошибка, но можно продолжать ввод
// В скрипте ошибка остановит выполнение

// 5. Временные переменные
// В консоли переменные сохраняются до обновления страницы
temp = 'временная' // Без var/let/const - создает глобальную
// После обновления страницы переменная исчезнет
```

2.2. Полезные функции консоли

javascript

```
// 1. monitor() и unmonitor()
function calculate(a, b) {
  return a * b;
}
monitor(calculate); // Начинает отслеживать вызовы функции
calculate(2, 3); // В консоли: function calculate called with arguments: 2, 3
unmonitor(calculate); // Прекращает отслеживание

// 2. debug() и undebug()
debug(calculate); // Устанавливает точку останова при вызове функции
calculate(4, 5); // Выполнение остановится для отладки
undebug(calculate); // Убирает точку останова

// 3. monitorEvents() и unmonitorEvents()
const button = document.querySelector('button');
monitorEvents(button, 'click'); // Отслеживает клики на кнопке
// Клик на кнопке покажет событие в консоли
unmonitorEvents(button); // Прекращает отслеживание

// 4. $ и $$ для быстрого выбора
$('.my-class') // Эквивалент document.querySelector
$$('div > p') // Эквивалент document.querySelectorAll
```

```
// 5. copy()
const obj = { name: 'John', age: 30 };
copy(obj); // Копирует строковое представление в буфер обмена
// Теперь можно вставить куда угодно: {"name":"John","age":30}

// 6. keys() и values()
const user = { id: 1, name: 'Alice' };
keys(user); // ['id', 'name']
values(user); // [1, 'Alice']

// 7. Профилирование в консоли
console.profile('Мой профиль');
// Выполняемый код...
console.profileEnd('Мой профиль');
// Результат появится во вкладке Performance
```

2.3. Отладка через консоль

javascript

```
// 1. Условные точки останова
function processData(data) {
  debugger; // Остановится всегда
  if (data.length > 1000) {
    console.log('Большие данные');
    debugger; // Остановится только при условии
  }
}

// В консоли можно в любой момент ввести:
debugger; // Остановит выполнение если открыты DevTools

// 2. Изменение переменных во время выполнения
let debugValue = 'original';
// В консоли во время паузы можно:
// debugValue = 'modified'; // Изменить значение

// 3. Выполнение кода в контексте отладки
function complexCalculation() {
  const a = 10;
  const b = 20;
  debugger; // Остановка здесь
  // В консоли можно выполнить: console.log(a + b)
  return a * b;
}
```

```
}
```

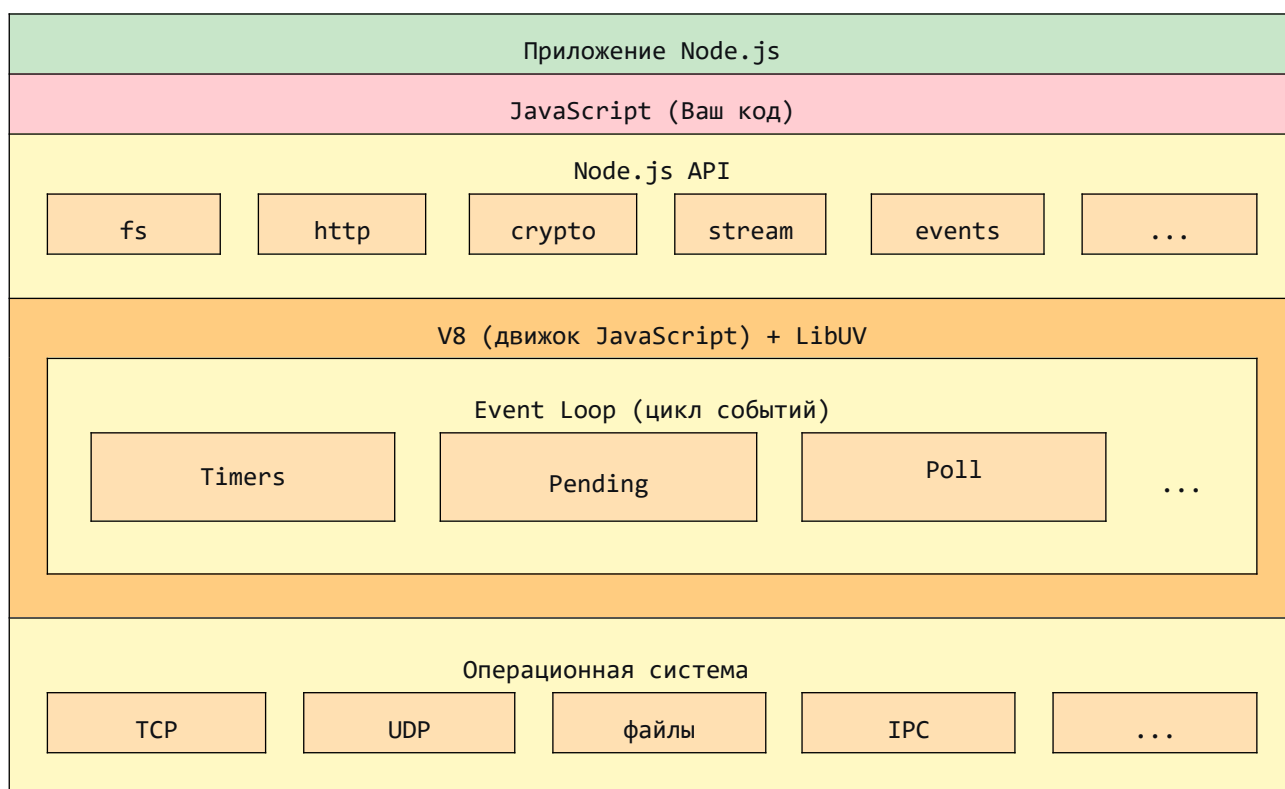
```
// 4. Работа с DOM во время отладки
// Во время остановки в консоли:
// document.body.style.backgroundColor = 'red';
// element.click(); // Имитация клика

// 5. Проверка производительности
console.time('Операция');
for (let i = 0; i < 1000000; i++) {
  Math.sqrt(i);
}
console.timeEnd('Операция'); // Выведет время выполнения
```

Часть 3: Node.js как среда выполнения

3.1. Архитектура Node.js

Text



3.2. Особенности выполнения в Node.js

Глобальный объект global:

```
javascript

// В Node.js глобальный объект - global
console.log(global === this); // false (в отличие от браузера)

// Переменные без var/let/const НЕ становятся глобальными
undeclared = 'test'; // ReferenceError в strict mode
// Без strict mode: создаст переменную, но не в global

// Глобальные переменные Node.js
console.log('__dirname:', __dirname); // Директория текущего файла
console.log('__filename:', __filename); // Путь к текущему файлу
console.log('process:', process.version); // Версия Node.js

// Глобальные функции Node.js
setImmediate(() => console.log('setImmediate callback'));
process.nextTick(() => console.log('nextTick callback'));
```

Модульная система:

```
javascript

// CommonJS модули (по умолчанию)
const fs = require('fs'); // Импорт модуля
const myModule = require('./my-module.js'); // Импорт локального файла

module.exports = { /* экспортируемые объекты */ };
exports.someFunction = function() {};

// ES6 модули (требует .mjs расширения или "type": "module")
// import fs from 'fs'; // В .mjs файле или с "type": "module" в package.json
// export default function() {}

// Встроенные модули Node.js
const path = require('path');
const http = require('http');
const os = require('os');
const crypto = require('crypto');

console.log('Путь:', path.join(__dirname, 'file.txt'));
console.log('Платформа:', os.platform());
```

```
console.log('Хеш:', crypto.createHash('md5').update('test').digest('hex'));
```

Работа с файловой системой:

javascript

```
const fs = require('fs');  
const path = require('path');
```

// Синхронное чтение

```
try {  
  const data = fs.readFileSync('file.txt', 'utf8');  
  console.log('Содержимое файла:', data);  
} catch (error) {  
  console.error('Ошибка чтения:', error);  
}
```

// Асинхронное чтение

```
fs.readFile('file.txt', 'utf8', (err, data) => {  
  if (err) {  
    console.error('Ошибка:', err);  
    return;  
  }  
  console.log('Асинхронное чтение:', data);  
});
```

// Промисы с fs.promises

```
async function readFiles() {  
  try {  
    const data = await fs.promises.readFile('file.txt', 'utf8');  
    console.log('Promise-based чтение:', data);  
  } catch (error) {  
    console.error('Ошибка:', error);  
  }  
}
```

// Работа с директориями

```
fs.readdir('.', (err, files) => {  
  if (err) throw err;  
  console.log('Файлы в директории:', files);  
});
```

// Создание сервера

```
const http = require('http');  
const server = http.createServer((req, res) => {
```



```
res.writeHead(200, { 'Content-Type': 'text/plain' });
res.end('Привет от Node.js сервера!\n');
});

server.listen(3000, () => {
  console.log('Сервер запущен на http://localhost:3000');
});
```

3.3. Event Loop в Node.js

javascript

// Фазы Event Loop в Node.js

```
console.log('1. Начало скрипта');
```

// 1. Timers (setTimeout, setInterval)

```
setTimeout(() => {
  console.log('2. setTimeout (фаза Timers)');
}, 0);
```

// 2. Pending callbacks (системные операции)

```
setImmediate(() => {
  console.log('3. setImmediate (фаза Check)');
});
```

// 3. Poll (новые события, I/O)

```
fs.readFile('file.txt', 'utf8', (err, data) => {
  console.log('4. Файл прочитан (фаза Poll)');
});
```

// 4. Check (setImmediate callbacks)

// Выполняется после Poll

// 5. Close callbacks

```
server.on('close', () => {
  console.log('5. Сервер закрыт');
});
```

// process.nextTick - выполняется ПЕРЕД следующей фазой Event Loop

```
process.nextTick(() => {
  console.log('nextTick - выполняется сразу после текущей операции');
});
```

// Микрозадачи (Promise) - выполняются после nextTick

```
Promise.resolve().then(() => {
  console.log('Promise resolved - после nextTick');
});
```

```
console.log('6. Конец синхронного кода');
```

```
/*
Типичный порядок выполнения:
1. Начало скрипта
6. Конец синхронного кода
nextTick - выполняется сразу после текущей операции
Promise resolved - после nextTick
4. Файл прочитан (фаза Poll)
3. setImmediate (фаза Check)
2. setTimeout (фаза Timers)
5. Сервер закрыт (при закрытии)
*/
```

3.4. REPL Node.js

bash

```
# Запуск REPL
```

```
node
```

```
# или
```

```
node -i # Interactive mode
```

```
# Команды в REPL:
```

```
.break    # Выйти из многострочного ввода
```

```
.clear    # Очистить контекст
```

```
.exit     # Выйти из REPL
```

```
.help     # Показать помощь
```

```
.load     # Загрузить файл
```

```
.save     # Сохранить сессию в файл
```

javascript

```
// В REPL Node.js
```

```
> 2 + 2
```

```
4
```

```
> const x = 10
```

```
undefined
```

```
> x * 2
```

```
20
```

```

// Специальная переменная _
> 5 * 5
25
> _ + 10
35

// Многострочный ввод
> function hello(name) {
... return `Привет, ${name}`;
... }
undefined

> hello('Мир')
'Привет, Мир'

// Загрузка модулей
> const fs = require('fs')
undefined

> fs.readdirSync('.')
[ 'file1.txt', 'file2.js', ... ]

// Сохранение и загрузка сессии
> .save session.js
> .load session.js

```

Часть 4: Сравнительный анализ

4.1. Таблица сравнения

Характеристика	Браузер	Node.js	Консоль браузера
Глобальный объект	window	global	window
Модульная система	ES6 Modules	CommonJS, ES6 Modules	Нет (REPL)
Доступ к DOM	Полный	Нет	Полный (в контексте страницы)
Файловая система	Ограниченный (File	Полный (fs модуль)	Ограниченный

Характеристика	Браузер	Node.js	Консоль браузера
	API)		
Сетевые запросы	Fetch API, XMLHttpRequest	http, https модули	Fetch API
Аргументы командной строки	Нет	process.argv	Нет
Переменные окружения	Нет	process.env	Нет
Таймеры	setTimeout, setInterval	setTimeout, setInterval, setImmediate	setTimeout, setInterval
Безопасность	Песочница, CORS, CSP	Полный доступ к системе	Песочница (ограничения страницы)
Процессы	Web Workers	child_process, cluster	Нет
Потоки (Streams)	Web Streams API	Streams API	Web Streams API
Базы данных	IndexedDB, WebSQL	Любые через драйверы	IndexedDB (в контексте)

4.2. Примеры кода в разных средах

Чтение данных:

javascript

// Браузер (AJAX запрос)

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Ошибка:', error));
```

// Node.js (HTTP запрос)

```
const https = require('https');
https.get('https://api.example.com/data', (response) => {
  let data = '';
  response.on('data', chunk => data += chunk);
  response.on('end', () => console.log(JSON.parse(data)));
}).on('error', error => console.error('Ошибка:', error));
```

```
// Консоль (тот же fetch что и в браузере, но в REPL)
// > await fetch('https://api.example.com/data').then(r => r.json())
```

Работа с файлами:

javascript

```
// Браузер (ограниченный доступ через input)
// <input type="file" id="fileInput">
document.getElementById('fileInput').addEventListener('change', (event) => {
  const file = event.target.files[0];
  const reader = new FileReader();
  reader.onload = (e) => console.log(e.target.result);
  reader.readAsText(file);
});
```

```
// Node.js (полный доступ)
const fs = require('fs');
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

```
// Консоль (ограничен как браузер)
// Можно работать с файлами через input на странице
```

Создание сервера:

javascript

```
// Браузер - НЕВОЗМОЖНО создать сервер
// Можно только делать запросы к серверам
```

```
// Node.js (простой HTTP сервер)
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<h1>Привет от Node.js!</h1>');
});
server.listen(8080);
```

```
// Консоль - НЕВОЗМОЖНО создать сервер
// Можно тестировать существующие серверы
```

4.3. Особенности потоков выполнения

Браузер:

```
javascript

// Макрозадачи, микрозадачи и анимации
console.log('script start');

setTimeout(() => console.log('setTimeout'), 0);

Promise.resolve()
  .then(() => console.log('promise1'))
  .then(() => console.log('promise2'));

requestAnimationFrame(() => console.log('raf'));

console.log('script end');

/*
Порядок выполнения:
script start
script end
promise1
promise2
raf
setTimeout
*/
```

Node.js:

```
javascript

// process.nextTick vs setImmediate
console.log('start');

setImmediate(() => console.log('setImmediate'));
setTimeout(() => console.log('setTimeout'), 0);
process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));

console.log('end');

/*
Порядок выполнения:
```

```
start
end
nextTick
promise
setTimeout
setImmediate
*/
```

Часть 5: Практическое применение

5.1. Универсальный код (Isomorphic JavaScript)

javascript

```
// Код, который работает и в браузере, и в Node.js
```

```
// Проверка среды выполнения
```

```
const isBrowser = typeof window !== 'undefined' && typeof document !== 'undefined';
const isNode = typeof process !== 'undefined' &&
  process.versions &&
  process.versions.node;
```

```
// Универсальная функция для HTTP запросов
```

```
async function universalFetch(url, options = {}) {
  if (isBrowser) {
    // В браузере используем fetch
    const response = await window.fetch(url, options);
    return response.json();
  } else if (isNode) {
    // В Node.js используем http/https модули
    const http = url.startsWith('https') ? require('https') : require('http');

    return new Promise((resolve, reject) => {
      const req = http.request(url, options, (res) => {
        let data = '';
        res.on('data', chunk => data += chunk);
        res.on('end', () => {
          try {
            resolve(JSON.parse(data));
          } catch (error) {
            resolve(data);
          }
        })
      });
    });
  }
}
```

```

    });
  });

  req.on('error', reject);
  if (options.body) {
    req.write(options.body);
  }
  req.end();
});
} else {
  throw new Error('Неизвестная среда выполнения');
}
}

// Универсальное логирование
class UniversalLogger {
  log(message, data = {}) {
    const timestamp = new Date().toISOString();
    const logEntry = { timestamp, message, ...data };

    if (isBrowser) {
      console.log('%c' + message, 'color: blue;', data);
      // Можно отправлять на сервер логирования
      // universalFetch('/api/logs', { method: 'POST', body: JSON.stringify(logEntry) });
    } else if (isNode) {
      console.log(`[${timestamp}] ${message}`, data);
      // Можно писать в файл
      // require('fs').appendFileSync('app.log', JSON.stringify(logEntry) + '\n');
    }
  }
}

// Универсальная конфигурация
const config = {
  apiUrl: isBrowser
    ? window.location.origin + '/api'
    : process.env.API_URL || 'http://localhost:3000',

  environment: isBrowser
    ? 'browser'
    : process.env.NODE_ENV || 'development'
};

// Использование

```



```

const logger = new UniversalLogger();
logger.log('Приложение запущено', { environment: config.environment });

// Пример использования в разных средах
async function loadData() {
  try {
    const data = await universalFetch(`${config.apiUrl}/data`);
    logger.log('Данные загружены', { count: data.length });
    return data;
  } catch (error) {
    logger.log('Ошибка загрузки данных', { error: error.message });
    throw error;
  }
}

```

5.2. Отладка в разных средах

javascript

```

// Универсальная отладка
function setupDebugging() {
  if (isBrowser) {
    // В браузере добавляем глобальные помощники
    window.debug = {
      // Быстрый доступ к DOM
      $: (selector) => document.querySelector(selector),
      $$: (selector) => document.querySelectorAll(selector),

      // Временное логирование
      log: (...args) => {
        if (localStorage.getItem('debug') === 'true') {
          console.log('[DEBUG]', ...args);
        }
      },

      // Мониторинг событий
      monitorEvents: (element, events) => {
        events.forEach(eventType => {
          element.addEventListener(eventType, (e) => {
            console.log(`[${eventType}]`, e);
          });
        });
      }
    };
  }
};

```

```

// Горячие клавиши для отладки
document.addEventListener('keydown', (e) => {
  if (e.ctrlKey && e.shiftKey && e.key === 'D') {
    localStorage.setItem('debug',
      localStorage.getItem('debug') === 'true' ? 'false' : 'true'
    );
    console.log('Debug mode:', localStorage.getItem('debug'));
  }
});

} else if (isNode) {
  // В Node.js настраиваем логирование
  const fs = require('fs');
  const path = require('path');

  // Создаем директорию для логов если её нет
  const logDir = path.join(__dirname, 'logs');
  if (!fs.existsSync(logDir)) {
    fs.mkdirSync(logDir);
  }

  // Функция для записи логов
  global.debug = {
    log: (...args) => {
      const timestamp = new Date().toISOString();
      const message = `[${timestamp}] ${args.join(' ')}\n`;

      // Вывод в консоль
      console.log('[DEBUG]', ...args);

      // Запись в файл
      const logFile = path.join(logDir, `${new Date().toISOString().split('T')[0]}.log`);
      fs.appendFileSync(logFile, message);
    },

    // Профилирование
    profile: (name, fn) => {
      console.time(name);
      const result = fn();
      console.timeEnd(name);
      return result;
    }
  }
}

```

```

};

// Включаем source maps для stack traces
if (process.env.NODE_ENV !== 'production') {
  require('source-map-support').install();
}
}
}

// Автоматическая установка при запуске
if (process.env.NODE_ENV !== 'production') {
  setupDebugging();
}

// Использование
debug.log('Настройка отладки завершена');

```

5.3. Тестирование в разных средах

javascript

```

// Универсальные тесты
describe('Универсальные функции', () => {
  test('universalFetch работает в обеих средах', async () => {
    // Мок для браузера
    if (isBrowser) {
      window.fetch = jest.fn(() =>
        Promise.resolve({
          json: () => Promise.resolve({ success: true })
        })
      );
    }
    // Мок для Node.js
    else if (isNode) {
      jest.mock('https', () => ({
        request: jest.fn((url, options, callback) => {
          const mockResponse = {
            on: jest.fn((event, handler) => {
              if (event === 'data') {
                handler(JSON.stringify({ success: true }));
              }
              if (event === 'end') {
                handler();
              }
            })
          };
        })
      });
    }
  });
});

```

```

        return mockResponse;
    })
};
callback(mockResponse);
return {
    on: jest.fn(),
    write: jest.fn(),
    end: jest.fn()
};
})
}));
}

const result = await universalFetch('https://api.example.com/test');
expect(result).toEqual({ success: true });
});

test('Логирование работает правильно', () => {
    const consoleSpy = jest.spyOn(console, 'log');

    debug.log('Тестовое сообщение');

    if (isBrowser) {
        expect(consoleSpy).toHaveBeenCalledWith(
            expect.stringContaining('[DEBUG]'),
            'Тестовое сообщение'
        );
    } else if (isNode) {
        expect(consoleSpy).toHaveBeenCalledWith(
            '[DEBUG]',
            'Тестовое сообщение'
        );
    }

    consoleSpy.mockRestore();
});
});

// Запуск тестов в разных средах
if (isBrowser) {
    // Запуск Jest в браузере (через jsdom)
    describe('Браузерные тесты', () => {
        beforeEach(() => {
            document.body.innerHTML = '<div id="app"></div>';

```

```

});

test('DOM манипуляции работают', () => {
  const app = document.getElementById('app');
  app.innerHTML = '<h1>Тест</h1>';
  expect(app.innerHTML).toContain('Тест');
});
});
}

if (isNode) {
  // Тесты специфичные для Node.js
  describe('Node.js тесты', () => {
    test('Работа с файлами', () => {
      const fs = require('fs');
      const tempFile = 'test-temp.txt';

      fs.writeFileSync(tempFile, 'test content');
      const content = fs.readFileSync(tempFile, 'utf8');

      expect(content).toBe('test content');

      // Очистка
      fs.unlinkSync(tempFile);
    });
  });
}

```

Заключение: выбор среды выполнения

Критерии выбора:

Выбирайте браузер если:

- Разрабатываете клиентское веб-приложение
- Нужен доступ к DOM и браузерным API
- Требуется взаимодействие с пользователем через UI
- Проект должен работать везде где есть браузер

Выбирайте Node.js если:

- Создаете серверное приложение

- Нужен доступ к файловой системе
- Требуется работа с базами данных напрямую
- Необходимы фоновые процессы или демоны
- Строите CLI инструменты

Используйте консоль для:

- Быстрого тестирования кода
- Отладки приложений
- Исследования API
- Обучения и экспериментов

Универсальные советы:

1. **Пишите изоморфный код** где это возможно
2. **Используйте feature detection** вместо browser detection
3. **Тестируйте во всех средах** где будет работать ваш код
4. **Знайте ограничения** каждой среды
5. **Используйте правильные инструменты** для каждой задачи

Быстрый чек-лист:

javascript

```
const environmentChecklist = {  
  browser: [  
    '✓ Доступен window и document',  
    '✓ Можно использовать DOM API',  
    '✓ Есть доступ к localStorage/sessionStorage',  
    '✓ Поддерживаются Web APIs (geolocation, canvas и т.д.)',  
    '✓ Ограничен доступ к файловой системе',  
    '✓ Действуют политики безопасности (CORS, CSP)'  
  ],  

```

```
  nodejs: [  
    '✓ Доступен global и process',  
    '✓ Есть доступ к файловой системе (fs модуль)',  
    '✓ Можно создавать серверы (http/https модули)',  
    '✓ Поддерживаются потоки (streams)',  
    '✓ Есть доступ к переменным окружения',  
    '✓ Можно работать с процессами (child_process)'  
  ],  

```

```
  console: [  
    '✓ REPL среда для быстрого тестирования',  
  ],  
}
```

```
'✓ Сохраняет состояние между командами',  
'✓ Специальные переменные ($0, $_, $, $$)',  
'✓ Можно отлаживать работающее приложение',  
'✓ Прямой доступ к контексту выполнения'  
]  
};
```

Понимание различий в выполнении JavaScript в разных средах позволяет писать более надежный, переносимый и эффективный код. Каждая среда имеет свои сильные стороны и ограничения, и успешный разработчик знает когда и какую использовать.

3.4. "Hello, World!" в различных средах

Введение: важность первого "Hello, World!"

"Hello, World!" — традиционная первая программа при изучении любого языка программирования. В JavaScript этот простой пример демонстрирует фундаментальные различия в выполнении кода в разных средах.

Часть 1: Браузерные реализации

1.1. Классический HTML+JavaScript

Базовый пример с inline JavaScript:

```
html

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello World - Inline JavaScript</title>
</head>
<body>
  <!-- 1. Вывод через document.write (устаревший метод) -->
  <script>
    document.write("<h1>Hello, World! (document.write)</h1>");
    document.write("<p>Этот метод перезаписывает весь документ</p>");
  </script>

  <!-- 2. Вывод в существующий элемент -->
  <div id="output1"></div>
  <script>
    document.getElementById('output1').innerHTML =
      '<h2>Hello, World! (innerHTML)</h2>' +
      '<p>Безопаснее чем document.write</p>';
  </script>
```



```

<!-- 3. Создание элементов динамически -->
<div id="output2"></div>
<script>
    const container = document.getElementById('output2');

    const heading = document.createElement('h3');
    heading.textContent = 'Hello, World! (createElement)';

    const paragraph = document.createElement('p');
    paragraph.textContent = 'Самый правильный способ создания элементов';

    container.appendChild(heading);
    container.appendChild(paragraph);
</script>

<!-- 4. Использование alert() -->
<button onclick="showAlert()">Показать Alert</button>
<script>
    function showAlert() {
        alert('Hello, World!\nЭто модальное окно блокирует интерфейс');
    }
</script>

<!-- 5. Вывод в консоль разработчика -->
<script>
    console.log('Hello, World! (console.log)');
    console.info('Информационное сообщение');
    console.warn('Предупреждение');
    console.error('Ошибка (имитация)');

    // Стилизованный вывод
    console.log(
        '%cHello, World! (стилизованный)',
        'color: white; background: linear-gradient(90deg, #ff6b6b, #4ecdc4);' +
        'font-size: 20px; padding: 10px; border-radius: 5px;'
    );
</script>
</body>
</html>

```

Внешний JavaScript файл:

html

```
<!-- index.html -->
```

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Hello World - External JS</title>
  <!-- Подключение внешнего скрипта -->
  <script src="hello.js" defer></script>
  <!-- defer обеспечивает выполнение после загрузки DOM -->
</head>
<body>
  <div id="app">
    <h1>Внешний JavaScript файл</h1>
    <div id="greeting"></div>
  </div>
</body>
</html>

```

javascript

```

// hello.js - внешний JavaScript файл
(function() {
  'use strict';

  // Ждем полной загрузки DOM
  document.addEventListener('DOMContentLoaded', function() {
    const greetingElement = document.getElementById('greeting');

    if (greetingElement) {
      // Создаем приветствие
      const message = document.createElement('div');
      message.className = 'hello-message';
      message.innerHTML = `
        <h2>Hello, World!</h2>
        <p>Это сообщение загружено из внешнего файла JavaScript</p>
        <p>Текущее время: ${new Date().toLocaleTimeString()}</p>
      `;

      // Добавляем стили
      message.style.cssText = `
        background: #f0f8ff;
        border: 2px solid #4ecdc4;
        border-radius: 10px;
        padding: 20px;
        margin: 20px 0;
        box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      `;
    }
  });

```

```

`;

greetingElement.appendChild(message);

// Добавляем кнопку для интерактивности
const button = document.createElement('button');
button.textContent = 'Обновить время';
button.style.cssText = `
  background: #4ecdc4;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
  margin-top: 10px;
`;

button.addEventListener('click', function() {
  const timeElement = message.querySelector('p:last-child');
  timeElement.textContent = `Текущее время: ${new Date().toLocaleTimeString()}`;
});

button.addEventListener('mouseenter', function() {
  this.style.background = '#45b7af';
});

button.addEventListener('mouseleave', function() {
  this.style.background = '#4ecdc4';
});

message.appendChild(button);
}

// Дополнительный вывод в консоль
console.log('%cHello, World! загружен успешно!',
  'color: green; font-weight: bold; font-size: 14px;');
console.table({
  'Среда выполнения': 'Браузер',
  'User Agent': navigator.userAgent,
  'Язык браузера': navigator.language,
  'Поддержка JavaScript': '100%'
});
});

```

```
})();
```

1.2. Современные подходы с ES6+

Шаблонные строки и стрелочные функции:

```
html

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Hello World - Modern JavaScript</title>
  <style>
    .theme-light {
      --bg-color: #ffffff;
      --text-color: #333333;
      --primary-color: #4ecdc4;
    }

    .theme-dark {
      --bg-color: #1a1a1a;
      --text-color: #ffffff;
      --primary-color: #ff6b6b;
    }

    body {
      background: var(--bg-color);
      color: var(--text-color);
      transition: all 0.3s ease;
      font-family: 'Arial', sans-serif;
      margin: 0;
      padding: 20px;
    }
  </style>
</head>
<body class="theme-light">
  <div id="app"></div>

  <script type="module">
    // ES6 Module с современным синтаксисом

    // Класс для управления приветствием
```

```

class GreetingApp {
  constructor(containerId) {
    this.container = document.getElementById(containerId);
    this.theme = 'light';
    this.greetings = [
      'Hello, World!',
      'Привет, Мир!',
      'Hola, Mundo!',
      'Bonjour, Monde!',
      'Hallo, Welt!',
      'Ciao, Mondo!',
      'こんにちは、世界! '
    ];
    this.currentGreeting = 0;
    this.init();
  }

  init() {
    this.render();
    this.setupEventListeners();
    this.startGreetingRotation();
  }

  render() {
    this.container.innerHTML = this.getTemplate();
  }

  getTemplate() {
    return `
      <div class="app-container">
        <header class="app-header">
          <h1>Modern Hello World</h1>
          <p>Демонстрация современных возможностей JavaScript ES6+</p>
        </header>

        <main class="app-main">
          <div class="greeting-card" id="greetingCard">
            <h2 id="greetingText">${this.greetings[this.currentGreeting]}</h2>
            <div class="greeting-info">
              <p>Это сообщение создано с использованием:</p>
              <ul>
                <li>ES6 Классов</li>
                <li>Шаблонных строк (Template Literals)</li>
                <li>Стрелочных функций</li>
            
```

```

        <li>Модульной системы</li>
        <li>CSS переменных</li>
    </ul>
</div>
</div>

<div class="controls">
    <button id="nextBtn" class="btn btn-primary">
        Следующее приветствие
    </button>
    <button id="themeBtn" class="btn btn-secondary">
        Сменить тему
    </button>
    <button id="animateBtn" class="btn btn-accent">
        Анимировать
    </button>
</div>

<div class="status">
    <p id="statusText">Текущая тема: ${this.theme}</p>
    <p>Время: <span id="timeDisplay">${this.getCurrentTime()}</span></p>
</div>
</main>

<footer class="app-footer">
    <p>Среда выполнения: Браузер (${navigator.userAgent.split(' ')[0]})</p>
    <p>JavaScript включен: <span class="status-indicator active">✓</span></p>
</footer>
</div>
`;
}

setupEventListeners() {
    document.getElementById('nextBtn').addEventListener('click', () => {
        this.nextGreeting();
    });

    document.getElementById('themeBtn').addEventListener('click', () => {
        this.toggleTheme();
    });

    document.getElementById('animateBtn').addEventListener('click', () => {
        this.animateGreeting();
    });
}

```

```

// Обновление времени каждую секунду
setInterval(() => {
    document.getElementById('timeDisplay').textContent = this.getCurrentTime();
}, 1000);
}

nextGreeting() {
    this.currentGreeting = (this.currentGreeting + 1) % this.greetings.length;
    document.getElementById('greetingText').textContent =
        this.greetings[this.currentGreeting];
    this.updateStatus(`Показано приветствие: ${this.currentGreeting + 1}/${
this.greetings.length}`);
}

toggleTheme() {
    this.theme = this.theme === 'light' ? 'dark' : 'light';
    document.body.className = `theme-${this.theme}`;
    this.updateStatus(`Тема изменена на: ${this.theme === 'light' ? 'светлую' :
'темную'}`);
}

animateGreeting() {
    const card = document.getElementById('greetingCard');
    card.style.transition = 'all 0.5s ease';
    card.style.transform = 'scale(1.1) rotate(5deg)';
    card.style.boxShadow = '0 10px 30px rgba(0,0,0,0.3)';

    setTimeout(() => {
        card.style.transform = 'scale(1) rotate(0deg)';
        card.style.boxShadow = '0 4px 6px rgba(0,0,0,0.1)';
    }, 500);

    this.updateStatus('Анимация выполнена!');
}

startGreetingRotation() {
    // Автоматическая смена приветствий каждые 5 секунд
    setInterval(() => {
        this.nextGreeting();
    }, 5000);
}

updateStatus(message) {

```

```

document.getElementById('statusText').textContent = message;

// Временное выделение
const statusElement = document.getElementById('statusText');
statusElement.style.color = 'var(--primary-color)';
statusElement.style.fontWeight = 'bold';

setTimeout(() => {
  statusElement.style.color = '';
  statusElement.style.fontWeight = '';
}, 1000);
}

getCurrentTime() {
  return new Date().toLocaleTimeString('ru-RU', {
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit'
  });
}
}

// Инициализация приложения
const app = new GreetingApp('app');

// Экспорт для тестирования (если нужно)
export default GreetingApp;

</script>

<style>
/* Дополнительные стили для компонентов */
.app-container {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

.app-header {
  text-align: center;
  margin-bottom: 40px;
  border-bottom: 2px solid var(--primary-color);
  padding-bottom: 20px;
}

```



```
.greeting-card {
  background: var(--bg-color);
  border: 2px solid var(--primary-color);
  border-radius: 15px;
  padding: 40px;
  text-align: center;
  margin: 30px 0;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

#greetingText {
  font-size: 3em;
  margin: 0 0 20px 0;
  color: var(--primary-color);
  text-shadow: 2px 2px 4px rgba(0,0,0,0.1);
}

.greeting-info {
  text-align: left;
  max-width: 500px;
  margin: 0 auto;
}

.greeting-info ul {
  list-style-type: none;
  padding: 0;
}

.greeting-info li {
  padding: 5px 0;
  position: relative;
  padding-left: 25px;
}

.greeting-info li:before {
  content: '✓';
  position: absolute;
  left: 0;
  color: var(--primary-color);
  font-weight: bold;
}

.controls {
```

```
display: flex;
justify-content: center;
gap: 15px;
margin: 30px 0;
flex-wrap: wrap;
}
```

```
.btn {
padding: 12px 24px;
border: none;
border-radius: 8px;
cursor: pointer;
font-size: 16px;
font-weight: bold;
transition: all 0.3s ease;
}
```

```
.btn-primary {
background: var(--primary-color);
color: white;
}
```

```
.btn-secondary {
background: transparent;
border: 2px solid var(--primary-color);
color: var(--primary-color);
}
```

```
.btn-accent {
background: linear-gradient(45deg, #ff6b6b, #4ecdc4);
color: white;
}
```

```
.btn:hover {
transform: translateY(-2px);
box-shadow: 0 6px 12px rgba(0,0,0,0.2);
}
```

```
.status {
text-align: center;
margin: 30px 0;
padding: 20px;
background: rgba(0,0,0,0.05);
border-radius: 10px;
```

```
}
```

```
.app-footer {  
  text-align: center;  
  margin-top: 40px;  
  padding-top: 20px;  
  border-top: 1px solid rgba(0,0,0,0.1);  
  font-size: 14px;  
}
```

```
.status-indicator {  
  display: inline-block;  
  width: 20px;  
  height: 20px;  
  border-radius: 50%;  
  text-align: center;  
  line-height: 20px;  
  margin: 0 5px;  
}
```

```
.status-indicator.active {  
  background: #4CAF50;  
  color: white;  
}
```

```
@media (max-width: 600px) {  
  #greetingText {  
    font-size: 2em;  
  }  
}
```

```
.controls {  
  flex-direction: column;  
}
```

```
.btn {  
  width: 100%;  
}
```

```
}
```

```
</style>
```

```
</body>
```

```
</html>
```

Часть 2: Консоль браузера

2.1. Базовые примеры в консоли

javascript

// Откройте консоль разработчика (F12) и введите:

// 1. Самый простой Hello World

```
console.log('Hello, World!');
```

// 2. С использованием переменных

```
const greeting = 'Hello';
```

```
const target = 'World';
```

```
console.log(greeting + ', ' + target + '!'); // Конкатенация
```

```
console.log(`${greeting}, ${target}!`); // Шаблонные строки
```

// 3. Разные методы console

```
console.info('Информация: Hello, World!');
```

```
console.warn('Предупреждение: Hello, World!');
```

```
console.error('Ошибка: Hello, World!');
```

```
console.debug('Отладка: Hello, World!');
```

// 4. Стилизованный вывод

```
console.log(  
  '%c Hello, World! ',  
  'font-size: 24px;' +  
  'background: linear-gradient(90deg, red, orange, yellow, green, blue, indigo, violet);'  
  +  
  'color: white;' +  
  'padding: 10px 20px;' +  
  'border-radius: 10px;' +  
  'text-shadow: 2px 2px 4px #000000;' );
```

// 5. Вывод в виде таблицы

```
console.table([  
  { Язык: 'Английский', Приветствие: 'Hello, World!' },  
  { Язык: 'Русский', Приветствие: 'Привет, Мир!' },  
  { Язык: 'Испанский', Приветствие: '¡Hola, Mundo!' },  
  { Язык: 'Французский', Приветствие: 'Bonjour, le Monde!' },  
  { Язык: 'Немецкий', Приветствие: 'Hallo, Welt!' }  
]);
```

```

// 6. Группировка вывода
console.group('Группа приветствий');
console.log('Hello');
console.log('World');
console.groupEnd();

// 7. Таймер
console.time('Время выполнения');
for (let i = 0; i < 1000000; i++) {
    // Имитация работы
}
console.timeEnd('Время выполнения');

// 8. Assert (утверждение)
const isHelloWorld = true;
console.assert(isHelloWorld, 'Это не Hello, World!');

// 9. Trace (трассировка стека)
function hello() {
    console.trace('Трассировка вызова hello()');
    return 'Hello, World!';
}
hello();

// 10. Dir (детальный вывод объекта)
console.dir(document.body);

// 11. Count (счетчик)
for (let i = 0; i < 5; i++) {
    console.count('Счетчик Hello World');
}

// 12. Clear (очистка консоли)
// console.clear();

```

2.2. Интерактивные примеры в консоли

javascript

```

// 1. Интерактивная функция приветствия
function interactiveHello() {
    const name = prompt('Введите ваше имя:', 'Гость');
    const language = prompt('Выберите язык (en/ru/es/fr/de):', 'en');
}

```

```

const greetings = {
  en: 'Hello',
  ru: 'Привет',
  es: 'Hola',
  fr: 'Bonjour',
  de: 'Hallo'
};

const greeting = greetings[language] || 'Hello';
const message = `${greeting}, ${name}!`;

console.log(`%c${message}`,
  'font-size: 20px; ' +
  'color: #4CAF50; ' +
  'font-weight: bold; ' +
  'padding: 10px; ' +
  'background: #f0f0f0; ' +
  'border-radius: 5px;');

return message;
}

// Вызовите в консоли: interactiveHello()

// 2. Анимированный вывод
async function animatedHello() {
  const text = 'Hello, World!';
  const colors = ['#ff0000', '#ff9900', '#ffff00', '#00ff00', '#0099ff', '#6600ff'];

  for (let i = 0; i < text.length; i++) {
    await new Promise(resolve => setTimeout(resolve, 100));
    console.log(
      `%c${text[i]}`,
      `color: ${colors[i % colors.length]};` +
      `font-size: ${20 + i * 2}px;` +
      `font-weight: bold;`
    );
  }
}

// Вызовите в консоли: animatedHello()

// 3. Работа с DOM через консоль

```

```

function domHello() {
  // Создаем элемент
  const helloDiv = document.createElement('div');
  helloDiv.id = 'console-hello';
  helloDiv.innerHTML = `
    <h2>Hello, World! из консоли</h2>
    <p>Этот элемент создан и добавлен через консоль браузера</p>
    <button onclick="this.parentElement.remove()">Удалить</button>
  `;
  helloDiv.style.cssText = `
    position: fixed;
    top: 20px;
    right: 20px;
    background: white;
    border: 2px solid #4CAF50;
    border-radius: 10px;
    padding: 20px;
    box-shadow: 0 4px 20px rgba(0,0,0,0.2);
    z-index: 10000;
    max-width: 300px;
  `;

  document.body.appendChild(helloDiv);
  console.log('Элемент добавлен на страницу');

  // Возвращаем элемент для дальнейших манипуляций
  return helloDiv;
}

```

```

// Вызовите в консоли: domHello()

```

```

// 4. Работа с localStorage

```

```

function persistentHello() {
  // Получаем или устанавливаем имя
  let userName = localStorage.getItem('helloUserName');

  if (!userName) {
    userName = prompt('Как вас зовут?', 'Гость');
    if (userName) {
      localStorage.setItem('helloUserName', userName);
    }
  }
}

```

```

// Счетчик посещений

```

```

let visitCount = parseInt(localStorage.getItem('helloVisitCount')) || 0;
visitCount++;
localStorage.setItem('helloVisitCount', visitCount);

const message = `Hello, ${userName}! Вы здесь в ${visitCount} раз`;

console.group('Персональное приветствие');
console.log(`%c${message}`, 'color: #2196F3; font-size: 16px;');
console.log(`Имя: ${userName}`);
console.log(`Количество посещений: ${visitCount}`);
console.groupEnd();

return { userName, visitCount };
}

// Вызовите в консоли: persistentHello()

// 5. Fetch API из консоли
async function apiHello() {
  try {
    console.log('Запрашиваю данные с JSONPlaceholder...');

    // Используем публичное API для демонстрации
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');
    const data = await response.json();

    console.group('Данные с API');
    console.log('Статус:', response.status);
    console.log('Заголовок:', data.title);
    console.log('Тело:', data.body);
    console.groupEnd();

    // Создаем уведомление на странице
    const notification = document.createElement('div');
    notification.innerHTML = `
      <h3>📌 Данные получены!</h3>
      <p><strong>${data.title}</strong></p>
      <p>${data.body.substring(0, 100)}...</p>
    `;
    notification.style.cssText = `
      position: fixed;
      bottom: 20px;
      left: 20px;
      background: #4CAF50;

```



```

        color: white;
        padding: 15px;
        border-radius: 8px;
        box-shadow: 0 4px 12px rgba(0,0,0,0.3);
        z-index: 10000;
        max-width: 300px;
    `;

    document.body.appendChild(notification);

    // Автоматическое удаление через 5 секунд
    setTimeout(() => {
        notification.remove();
        console.log('Уведомление удалено');
    }, 5000);

    return data;

} catch (error) {
    console.error('Ошибка при запросе:', error);
    return null;
}
}

// Вызовите в консоли: apiHello()

```

2.3. Продвинутые техники в консоли

javascript

```

// 1. Прототипное наследование
function createHelloWorld() {
    // Базовый конструктор
    function BaseGreeting(language) {
        this.language = language;
        this.timestamp = new Date();
    }

    BaseGreeting.prototype.sayHello = function() {
        const greetings = {
            en: 'Hello, World!',
            ru: 'Привет, Мир!',
            es: '¡Hola, Mundo!',
            fr: 'Bonjour, le Monde!'
        };
    };
}

```

```

    };

    return greetings[this.language] || greetings.en;
  };

  // Наследник
  function TimedGreeting(language, duration = 5000) {
    BaseGreeting.call(this, language);
    this.duration = duration;
  }

  // Наследование
  TimedGreeting.prototype = Object.create(BaseGreeting.prototype);
  TimedGreeting.prototype.constructor = TimedGreeting;

  // Дополнительный метод
  TimedGreeting.prototype.showForDuration = function() {
    const message = this.sayHello();
    console.log(`%c${message}`, 'font-size: 18px; color: #FF5722;');

    setTimeout(() => {
      console.log(`%cСообщение "${message}" было показано ${this.duration}ms`,
        'color: #9E9E9E; font-style: italic;');
    }, this.duration);
  };

  return new TimedGreeting('en', 3000);
}

// Использование
const timedHello = createHelloWorld();
timedHello.showForDuration();

// 2. Генераторы
function* helloWorldGenerator() {
  const words = ['Hello', ',', ' ', 'World', '!'];

  for (let word of words) {
    yield word;
    // Пауза между словами
    yield new Promise(resolve => setTimeout(resolve, 500));
  }

  yield '\n✅ Завершено!';
}

```

```
}
```

```
async function runGenerator() {  
  console.log('Генератор начал работу:');  
  
  const generator = helloWorldGenerator();  
  let result = generator.next();  
  
  while (!result.done) {  
    if (result.value instanceof Promise) {  
      await result.value;  
    } else {  
      process.stdout.write(result.value);  
    }  
    result = generator.next();  
  }  
  
  console.log(result.value);  
}
```

// runGenerator(); // Работает в Node.js, в браузере process.stdout не доступен

// Альтернатива для браузера

```
async function browserGeneratorHello() {  
  const generator = helloWorldGenerator();  
  let result = generator.next();  
  let output = '';  
  
  while (!result.done) {  
    if (result.value instanceof Promise) {  
      await result.value;  
    } else {  
      output += result.value;  
      console.log(output);  
    }  
    result = generator.next();  
  }  
}
```

// 3. Прокси для перехвата

```
const helloHandler = {  
  get: function(target, property) {  
    if (property === 'message') {  
      return 'Hello, World!';  
    }  
  }  
}
```

```

    } else if (property === 'reverse') {
        return target.message.split('').reverse().join('');
    } else if (property === 'uppercase') {
        return target.message.toUpperCase();
    } else if (property === 'length') {
        return target.message.length;
    }
    return `Свойство "${property}" не найдено`;
},

set: function(target, property, value) {
    if (property === 'language') {
        const greetings = {
            en: 'Hello, World!',
            ru: 'Привет, Мир!',
            es: '¡Hola, Mundo!'
        };
        target.message = greetings[value] || greetings.en;
        return true;
    }
    return false;
}
};

const helloProxy = new Proxy({ message: 'Hello, World!' }, helloHandler);

console.log('Прoxy демонстрация:');
console.log('message:', helloProxy.message);
console.log('reverse:', helloProxy.reverse);
console.log('uppercase:', helloProxy.uppercase);
console.log('length:', helloProxy.length);

helloProxy.language = 'es';
console.log('После изменения языка:', helloProxy.message);

// 4. Web Workers (в консоли текущей страницы)
function createWorkerHello() {
    // Создаем Blob с кодом worker'а
    const workerCode = `
        self.onmessage = function(e) {
            const { language, count } = e.data;

            const greetings = {
                en: 'Hello',

```

```

    ru: 'Привет',
    es: 'Hola',
    fr: 'Bonjour'
  };

  const greeting = greetings[language] || 'Hello';

  for (let i = 0; i < count; i++) {
    self.postMessage({
      index: i + 1,
      total: count,
      message: `\\${greeting}, World! (\\${i + 1}/\\${count})\\`
    });

    // Имитация тяжелой работы
    for (let j = 0; j < 1000000; j++) {}
  }

  self.postMessage({ done: true });
};
`;

const blob = new Blob([workerCode], { type: 'application/javascript' });
const worker = new Worker(URL.createObjectURL(blob));

console.log('Web Worker запущен...');

worker.onmessage = function(e) {
  if (e.data.done) {
    console.log('%c🏁 Worker завершил работу', 'color: #4CAF50;');
    worker.terminate();
  } else {
    console.log(`\\Worker: \\${e.data.message}\\`);
  }
};

// Запускаем worker
worker.postMessage({ language: 'en', count: 5 });

return worker;
}

// ВАЖНО: Web Workers не могут быть запущены из локальных файлов (file://)
// без флага --allow-file-access-from-files

```

```
// createWorkerHello(); // Работает только на HTTP/HTTPS сервере
```

Часть 3: Node.js реализации

3.1. Базовые примеры в Node.js

Простой скрипт:

```
javascript

// hello.js - простейший Node.js скрипт
console.log('Hello, World!');

// Запуск: node hello.js
```

С аргументами командной строки:

```
javascript

// hello-args.js
console.log('Аргументы командной строки:');

// process.argv содержит аргументы
// [0] - путь к node
// [1] - путь к файлу скрипта
// [2+] - переданные аргументы

if (process.argv.length > 2) {
  const name = process.argv[2];
  console.log(`Hello, ${name}!`);
} else {
  console.log('Hello, World!');
  console.log('Использование: node hello-args.js [имя]');
}

// Пример запуска:
// node hello-args.js
// node hello-args.js John
// node hello-args.js "John Doe"
```

С обработкой ввода:

```
javascript
```

```

// hello-interactive.js
const readline = require('readline').createInterface({
  input: process.stdin,
  output: process.stdout
});

console.log('=== Интерактивное приветствие ===\n');

readline.question('Как вас зовут? ', (name) => {
  if (!name.trim()) {
    name = 'Гость';
  }

  readline.question('На каком языке? (en/ru/es/fr): ', (language) => {
    const greetings = {
      en: 'Hello',
      ru: 'Привет',
      es: 'Hola',
      fr: 'Bonjour'
    };

    const greeting = greetings[language.toLowerCase()] || 'Hello';
    const message = `${greeting}, ${name}!`;

    console.log('\n' + '='.repeat(50));
    console.log(message);
    console.log('='.repeat(50));

    // Дополнительная информация
    console.log('\n□ Информация:');
    console.log(`Имя: ${name}`);
    console.log(`Язык: ${language.toUpperCase()}`);
    console.log(`Текущая директория: ${process.cwd()}`);
    console.log(`Платформа: ${process.platform}`);
    console.log(`Версия Node.js: ${process.version}`);

    readline.close();
  });
});

readline.on('close', () => {
  console.log('\n□ До свидания!');
  process.exit(0);
});

```

3.2. Файловые операции

javascript

```
// hello-file.js - работа с файлами
const fs = require('fs');
const path = require('path');

// 1. Создание директории для логов
const logDir = path.join(__dirname, 'logs');
if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir, { recursive: true });
  console.log(`Создана директория: ${logDir}`);
}

// 2. Запись приветствия в файл
const logFile = path.join(logDir, 'hello.log');
const timestamp = new Date().toISOString();
const message = `[${timestamp}] Hello, World!\n`;

fs.appendFile(logFile, message, (err) => {
  if (err) {
    console.error('Ошибка записи в файл:', err);
    return;
  }
  console.log(`Приветствие записано в файл: ${logFile}`);
});

// 3. Чтение файла
fs.readFile(logFile, 'utf8', (err, data) => {
  if (err) {
    console.error('Ошибка чтения файла:', err);
    return;
  }

  console.log(`\n   Содержимое файла:`);
  console.log('=' .repeat(50));
  console.log(data);
  console.log('=' .repeat(50));
});

// 4. Статистика файла
fs.stat(logFile, (err, stats) => {
  if (err) {
    console.error('Ошибка получения статистики:', err);
    return;
  }
```



```

    }

    console.log(`\n📊 Статистика файла:`);
    console.log(`Размер: ${stats.size} байт`);
    console.log(`Создан: ${stats.birthtime.toLocaleString()}`);
    console.log(`Изменен: ${stats.mtime.toLocaleString()}`);

    // 5. Удаление файла (закомментируйте для тестирования)
    // fs.unlink(logFile, (err) => {
    //     if (err) console.error('Ошибка удаления:', err);
    //     else console.log('Файл удален');
    // });
    });
    });
});

// 6. Чтение текущего файла скрипта
console.log(`\n📄 Исходный код этого скрипта:`);
console.log('=' .repeat(50));
console.log(fs.readFileSync(__filename, 'utf8'));
console.log('=' .repeat(50));

```

3.3. HTTP сервер

javascript

```

// hello-server.js - HTTP сервер на Node.js
const http = require('http');
const url = require('url');
const fs = require('fs');
const path = require('path');

const PORT = 3000;

// Создаем HTTP сервер
const server = http.createServer((req, res) => {
    const parsedUrl = url.parse(req.url, true);
    const pathname = parsedUrl.pathname;

    console.log(`[${new Date().toISOString()}] ${req.method} ${pathname}`);

    // Обработка разных маршрутов
    if (pathname === '/' || pathname === '/hello') {
        // Главная страница

```

```

    serveHelloPage(req, res, parsedUrl.query);
} else if (pathname === '/api/hello') {
    // JSON API
    serveHelloApi(req, res, parsedUrl.query);
} else if (pathname.startsWith('/static/')) {
    // Статические файлы
    serveStaticFile(req, res, pathname);
} else {
    // 404 Not Found
    serveNotFound(req, res);
}
});

```

// Функция для главной страницы

```

function serveHelloPage(req, res, query) {
    const name = query.name || 'World';
    const language = query.lang || 'en';

    const greetings = {
        en: 'Hello',
        ru: 'Привет',
        es: 'Hola',
        fr: 'Bonjour',
        de: 'Hallo',
        ja: 'こんにちは'
    };

    const greeting = greetings[language] || 'Hello';
    const message = `${greeting}, ${name}!`;

    const html = `
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>${message}</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
    </style>

```

```
body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 20px;
}

.container {
  background: white;
  border-radius: 20px;
  box-shadow: 0 20px 60px rgba(0,0,0,0.3);
  padding: 40px;
  max-width: 600px;
  width: 100%;
  text-align: center;
}

h1 {
  font-size: 3em;
  color: #333;
  margin-bottom: 20px;
  background: linear-gradient(45deg, #667eea, #764ba2);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.message {
  font-size: 1.5em;
  color: #666;
  margin-bottom: 30px;
  padding: 20px;
  background: #f8f9fa;
  border-radius: 10px;
  border-left: 5px solid #667eea;
}

.info {
  background: #e9ecef;
  padding: 15px;
  border-radius: 10px;
```

```
    margin: 20px 0;
    text-align: left;
}

.info h3 {
    color: #495057;
    margin-bottom: 10px;
}

.form-group {
    margin: 20px 0;
    text-align: left;
}

label {
    display: block;
    margin-bottom: 5px;
    color: #495057;
    font-weight: 500;
}

input, select {
    width: 100%;
    padding: 10px 15px;
    border: 2px solid #dee2e6;
    border-radius: 8px;
    font-size: 16px;
    transition: border-color 0.3s;
}

input:focus, select:focus {
    outline: none;
    border-color: #667eea;
}

button {
    background: linear-gradient(45deg, #667eea, #764ba2);
    color: white;
    border: none;
    padding: 12px 30px;
    border-radius: 8px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
```

```
        transition: transform 0.2s, box-shadow 0.2s;
        margin: 10px 5px;
    }

    button:hover {
        transform: translateY(-2px);
        box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
    }

    .footer {
        margin-top: 30px;
        color: #6c757d;
        font-size: 0.9em;
        border-top: 1px solid #dee2e6;
        padding-top: 20px;
    }

    .links {
        margin-top: 20px;
    }

    .links a {
        color: #667eea;
        text-decoration: none;
        margin: 0 10px;
    }

    .links a:hover {
        text-decoration: underline;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>👋 Hello World Server</h1>

        <div class="message">
            ${message}
        </div>

        <div class="info">
            <h3>📄 Информация о запросе:</h3>
            <p><strong>Метод:</strong> ${req.method}</p>
            <p><strong>URL:</strong> ${req.url}</p>
        </div>
    </div>
</body>
</html>
```

```

    <p><strong>Время:</strong> ${new Date().toLocaleString()}</p>
</div>

<form method="GET" action="/hello">
  <div class="form-group">
    <label for="name">Ваше имя:</label>
    <input type="text" id="name" name="name" value="${name}" placeholder="Введите
имя">
  </div>

  <div class="form-group">
    <label for="lang">Язык приветствия:</label>
    <select id="lang" name="lang">
      ${Object.entries(greetings).map(([code, text]) => `
        <option value="${code}" ${code === language ? 'selected' : ''}>
          ${text} (${code})
        </option>
      `).join('')}
    </select>
  </div>

  <button type="submit">👋 Поприветствовать</button>
  <button type="button" onclick="window.location.href='/'">🔄 Сбросить</button>
</form>

<div class="links">
  <a href="/api/hello?name=${encodeURIComponent(name)}&lang=${language}">JSON
API</a>
  <a href="/static/hello.txt">Статический файл</a>
  <a href="https://github.com" target="_blank">GitHub</a>
</div>

<div class="footer">
  <p>Сервер запущен на порту ${PORT}</p>
  <p>Node.js ${process.version} | Память: $
{Math.round(process.memoryUsage().heapUsed / 1024 / 1024)}MB</p>
</div>
</div>

<script>
  // Клиентский JavaScript
  document.addEventListener('DOMContentLoaded', function() {
    console.log('Hello World страница загружена!');
  });

```

```

    // Анимация заголовка
    const title = document.querySelector('h1');
    let hue = 0;

    setInterval(() => {
        hue = (hue + 1) % 360;
        title.style.background = `linear-gradient(45deg, hsl(\`${hue}\`, 100%, 65%),
hsl(\`${(hue + 60) % 360}\`, 100%, 65%))`;
        title.style.webkitBackgroundClip = 'text';
        title.style.webkitTextFillColor = 'transparent';
        title.style.backgroundClip = 'text';
    }, 50);
    });
</script>
</body>
</html>
`;

res.writeHead(200, {
  'Content-Type': 'text/html; charset=utf-8',
  'Cache-Control': 'no-cache'
});
res.end(html);
}

```

```

// Функция для JSON API
function serveHelloApi(req, res, query) {
  const name = query.name || 'World';
  const language = query.lang || 'en';

  const greetings = {
    en: 'Hello',
    ru: 'Привет',
    es: 'Hola',
    fr: 'Bonjour',
    de: 'Hallo',
    ja: 'こんにちは'
  };

  const greeting = greetings[language] || 'Hello';

  const response = {
    success: true,
    data: {

```

```

    message: `${greeting}, ${name}!`,
    greeting: greeting,
    name: name,
    language: language,
    timestamp: new Date().toISOString(),
    serverInfo: {
      nodeVersion: process.version,
      platform: process.platform,
      memoryUsage: process.memoryUsage()
    }
  },
  meta: {
    apiVersion: '1.0',
    endpoint: '/api/hello'
  }
};

res.writeHead(200, {
  'Content-Type': 'application/json; charset=utf-8',
  'Access-Control-Allow-Origin': '*'
});
res.end(JSON.stringify(response, null, 2));
}

```

```

// Функция для статических файлов
function serveStaticFile(req, res, pathname) {
  const filePath = path.join(__dirname, pathname);

  // Проверяем существование файла
  fs.access(filePath, fs.constants.F_OK, (err) => {
    if (err) {
      serveNotFound(req, res);
      return;
    }

    // Определяем MIME тип
    const ext = path.extname(filePath).toLowerCase();
    const mimeTypes = {
      '.html': 'text/html',
      '.js': 'application/javascript',
      '.css': 'text/css',
      '.json': 'application/json',
      '.png': 'image/png',
      '.jpg': 'image/jpeg',

```



```

    '.txt': 'text/plain'
  };

  const contentType = mimeTypes[ext] || 'application/octet-stream';

  // Читаем и отправляем файл
  fs.readFile(filePath, (err, content) => {
    if (err) {
      res.writeHead(500);
      res.end('Ошибка сервера');
      return;
    }

    res.writeHead(200, { 'Content-Type': contentType });
    res.end(content, 'utf8');
  });
});
}

```

```

// Функция для 404
function serveNotFound(req, res) {
  res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end(`
    <!DOCTYPE html>
    <html>
    <head>
      <title>404 - Страница не найдена</title>
      <style>
        body {
          font-family: Arial, sans-serif;
          text-align: center;
          padding: 50px;
          background: #f8f9fa;
        }
        h1 {
          color: #dc3545;
          font-size: 3em;
        }
        p {
          color: #6c757d;
        }
        a {
          color: #007bff;
          text-decoration: none;

```

```

    }
    </style>
</head>
<body>
  <h1>404</h1>
  <p>Страница не найдена</p>
  <p><a href="/">Вернуться на главную</a></p>
</body>
</html>
`);
}

```

// Запускаем сервер

```

server.listen(PORT, () => {
  console.log(`
    🟩 Сервер запущен!

    🟩 Локальный:   http://localhost:${PORT}
    🟩 Сеть:         http://${getLocalIP()}:${PORT}

    🟩 Директория:   ${__dirname}
    🟩 Платформа:    ${process.platform}
    🟩 Node.js:      ${process.version}

    🟩 Доступные маршруты:
    • /             - Главная страница
    • /api/hello    - JSON API
    • /static/*     - Статические файлы

    Нажмите Ctrl+C для остановки
  `);
});

```

// Функция для получения локального IP

```

function getLocalIP() {
  const interfaces = require('os').networkInterfaces();

  for (const name of Object.keys(interfaces)) {
    for (const iface of interfaces[name]) {
      if (iface.family === 'IPv4' && !iface.internal) {
        return iface.address;
      }
    }
  }
}

```

```

    return '127.0.0.1';
}

// Обработка сигналов завершения
process.on('SIGINT', () => {
  console.log('\n\n❏ Сервер останавливается...');
  server.close(() => {
    console.log('✅ Сервер остановлен');
    process.exit(0);
  });
});

// Создаем статический файл для демонстрации
const staticDir = path.join(__dirname, 'static');
if (!fs.existsSync(staticDir)) {
  fs.mkdirSync(staticDir, { recursive: true });
}

const staticFile = path.join(staticDir, 'hello.txt');
if (!fs.existsSync(staticFile)) {
  fs.writeFileSync(staticFile,
    'Hello, World!\n' +
    'Это статический файл, обслуживаемый Node.js сервером.\n' +
    `Создан: ${new Date().toISOString()}\n` +
    `Директория: ${staticDir}\n`
  );
}

```

3.4. CLI утилита

javascript

```

#!/usr/bin/env node
// hello-cli.js - CLI утилита
// Делаем исполняемой: chmod +x hello-cli.js
// Или устанавливаем через npm: npm link

const { Command } = require('commander');
const chalk = require('chalk');
const figlet = require('figlet');
const gradient = require('gradient-string');
const boxen = require('boxen');
const inquirer = require('inquirer');

```

```
// Инициализация программы
```

```
const program = new Command();
```

```
program
```

```
  .name('hello-world')  
  .description('CLI утилита для приветствий на разных языках')  
  .version('1.0.0')  
  .option('-n, --name <name>', 'Имя для приветствия', 'World')  
  .option('-l, --lang <language>', 'Язык приветствия', 'en')  
  .option('-a, --all', 'Показать все доступные языки')  
  .option('-j, --json', 'Вывести в формате JSON')  
  .option('-s, --save', 'Сохранить приветствие в файл')  
  .option('-f, --file <path>', 'Путь к файлу для сохранения');
```

```
// Определяем доступные языки
```

```
const languages = {  
  en: { greeting: 'Hello', flag: 'us', name: 'English' },  
  ru: { greeting: 'Привет', flag: 'RU', name: 'Russian' },  
  es: { greeting: 'Hola', flag: 'es', name: 'Spanish' },  
  fr: { greeting: 'Bonjour', flag: 'FR', name: 'French' },  
  de: { greeting: 'Hallo', flag: 'DE', name: 'German' },  
  ja: { greeting: 'こんにちは', flag: 'JP', name: 'Japanese' },  
  zh: { greeting: '你好', flag: 'CN', name: 'Chinese' },  
  ko: { greeting: '안녕하세요', flag: 'KR', name: 'Korean' },  
  ar: { greeting: 'مرحباً', flag: 'SA', name: 'Arabic' },  
  hi: { greeting: 'नमस्ते', flag: 'IN', name: 'Hindi' }  
};
```

```
// Функция для отображения баннера
```

```
function showBanner() {
```

```
  console.clear();
```

```
  const banner = figlet.textSync('Hello World', {  
    font: 'Standard',  
    horizontalLayout: 'full',  
    verticalLayout: 'default'  
  });
```

```
  console.log(gradient.rainbow(banner));
```

```
  console.log(boxen(  
    chalk.cyan('👋 CLI утилита для мультиязычных приветствий\n') +
```

```

    chalk.gray('Используйте команды ниже или запустите без аргументов для интерактивного
режима'),
    {
      padding: 1,
      margin: 1,
      borderStyle: 'round',
      borderColor: 'cyan',
      backgroundColor: '#222'
    }
  ));
}

```

// Функция для создания приветствия

```

function createGreeting(name, langCode) {
  const lang = languages[langCode] || languages.en;
  return {
    message: `${lang.greeting}, ${name}!`,
    language: lang.name,
    flag: lang.flag,
    code: langCode,
    timestamp: new Date().toISOString()
  };
}

```

// Команда для отображения всех языков

```

function showAllLanguages() {
  console.log(chalk.bold('\n📋 Доступные языки:\n'));

  Object.entries(languages).forEach(([code, lang]) => {
    console.log(
      `${lang.flag} ${chalk.bold(code.toUpperCase())} - ${lang.name}: $
{chalk.green(lang.greeting)}`
    );
  });

  console.log(chalk.gray('\nИспользуйте: hello-world --lang <код> --name <имя>'));
}

```

// Интерактивный режим

```

async function interactiveMode() {
  showBanner();

  const questions = [
    {

```

```

    type: 'input',
    name: 'name',
    message: 'Как вас зовут?',
    default: 'World',
    validate: (input) => {
      if (input.trim().length === 0) {
        return 'Пожалуйста, введите имя';
      }
      return true;
    }
  },
  {
    type: 'list',
    name: 'language',
    message: 'Выберите язык приветствия:',
    choices: Object.entries(languages).map(([code, lang]) => ({
      name: `${lang.flag} ${lang.name} (${lang.greeting})`,
      value: code
    })),
    default: 'en'
  },
  {
    type: 'confirm',
    name: 'saveToFile',
    message: 'Сохранить приветствие в файл?',
    default: false
  }
];

```

```

const answers = await inquirer.prompt(questions);
const greeting = createGreeting(answers.name, answers.language);

```

// Отображение результата

```

console.log('\n' + '='.repeat(50));
console.log(chalk.bold.hex('#FF6B6B')('▣ Ваше приветствие:'));
console.log('=' + '='.repeat(50));

```

```

console.log(
  boxen(
    `${chalk.bold.hex('#4ECDC4')(greeting.message)}\n\n` +
    `${greeting.flag} ${chalk.gray('Язык:')} ${greeting.language}\n` +
    `${chalk.gray('Код:')} ${greeting.code}\n` +
    `${chalk.gray('Время:')} ${new Date(greeting.timestamp).toLocaleString()}`,
    {

```

```

        padding: 1,
        borderStyle: 'round',
        borderColor: '#4ECDC4'
    }
)
);

// Сохранение в файл если нужно
if (answers.saveToFile) {
    const fs = require('fs');
    const path = require('path');

    const fileName = `greeting-${Date.now()}.json`;
    const filePath = path.join(process.cwd(), fileName);

    fs.writeFileSync(
        filePath,
        JSON.stringify(greeting, null, 2)
    );

    console.log(chalk.green(`\n    Приветствие сохранено в файл: ${fileName}`));
    console.log(chalk.gray(`Путь: ${filePath}`));
}

// Дополнительная информация
console.log(chalk.gray(`\n    Информация о системе:`));
console.log(chalk.gray(`Node.js: ${process.version}`));
console.log(chalk.gray(`Платформа: ${process.platform}`));
console.log(chalk.gray(`Архитектура: ${process.arch}`));

console.log(chalk.yellow(`\n    Используйте команды:`));
console.log(chalk.gray(`hello-world --help - Показать помощь`));
console.log(chalk.gray(`hello-world --all - Показать все языки`));
console.log(chalk.gray(`hello-world --json - Вывод в формате JSON`));
}

// Обработка команды
program.action(async (options) => {
    if (options.all) {
        showAllLanguages();
        return;
    }

    // Если запущено без аргументов - интерактивный режим

```

```

if (process.argv.length === 2) {
  await interactiveMode();
  return;
}

// Обычный режим с аргументами
const greeting = createGreeting(options.name, options.lang);

if (options.json) {
  console.log(JSON.stringify(greeting, null, 2));
} else {
  console.log(chalk.bold.hex('#FF6B6B')(greeting.message));
  console.log(chalk.gray(`${greeting.flag} ${greeting.language}`));
}

// Сохранение в файл если нужно
if (options.save) {
  const fs = require('fs');
  const path = require('path');

  let filePath = options.file || `greeting-${Date.now()}.json`;
  if (!path.isAbsolute(filePath)) {
    filePath = path.join(process.cwd(), filePath);
  }

  fs.writeFileSync(
    filePath,
    JSON.stringify(greeting, null, 2)
  );

  console.log(chalk.green(`✅ Сохранено в: ${filePath}`));
}
});

// Обработка ошибок
program.configureOutput({
  writeErr: (str) => {
    console.error(chalk.red('× Ошибка:'), str.replace('error: ', ''));
  }
});

// Запуск программы
program.parse();

```



```
// Примеры использования:
// 1. Интерактивный режим: ./hello-cli.js
// 2. Простое приветствие: ./hello-cli.js --name John --lang es
// 3. JSON вывод: ./hello-cli.js --name Jane --lang fr --json
// 4. Сохранение в файл: ./hello-cli.js --name Alex --lang ja --save
// 5. Показать все языки: ./hello-cli.js --all
```

Часть 4: Сравнение и выводы

4.1. Сводная таблица

Критерий	Браузер (HTML/JS)	Консоль браузера	Node.js
Сложность	Просто	Очень просто	Средняя
Доступность	Все пользователи	Разработчики	Разработчики/ Сервер
Интерактивность	Высокая	Средняя	Зависит от реализации
Доступ к DOM	Полный	Полный (в контексте)	Нет
Доступ к файлам	Ограниченный (File API)	Ограниченный	Полный
Сетевые запросы	Fetch API	Fetch API	http/https модули
Многопоточность	Web Workers	Web Workers	Worker Threads
Пользовательский ввод	Формы, prompt	prompt	readline, CLI аргументы
Визуализация	Любая (HTML/CSS)	Текстовая (стилизованная)	Текстовая (можно цвета)
Сохранение данных	localStorage, cookies	localStorage (в контексте)	Файлы, базы данных
Пример использования	Веб-приложения	Отладка, прототипирование	Серверы, CLI утилиты

4.2. Выбор среды для "Hello, World!"

Для обучения:

- **Начинающим:** Браузер с простым HTML/JS
- **Понимание основ:** Консоль браузера (быстрый feedback (обратная связь))
- **Серверная разработка:** Node.js с простыми скриптами

Для демонстрации:

- **Красиво и наглядно:** Браузер с CSS анимациями
- **Интерактивно:** Браузер с формами
- **Технически:** Node.js с CLI интерфейсом

Для продакшена:

- **Веб-приложение:** Браузер + Node.js сервер
- **API сервис:** Node.js HTTP сервер
- **Инструмент:** Node.js CLI утилита

4.3. Универсальный шаблон

```
javascript
```

```
// universal-hello.js
```

```
// Этот код можно адаптировать для любой среды
```

```
(function(global) {  
  'use strict';  
  
  // Определяем среду выполнения  
  const environment = (() => {  
    if (typeof window !== 'undefined' && typeof document !== 'undefined') {  
      return 'browser';  
    } else if (typeof global !== 'undefined' && global.process && global.process.versions  
&& global.process.versions.node) {  
      return 'node';  
    } else {  
      return 'unknown';  
    }  
  })();
```

```
// Класс приветствия
```

```
class HelloWorld {
```

```

constructor(name = 'World', language = 'en') {
  this.name = name;
  this.language = language;
  this.timestamp = new Date();
  this.greetings = {
    en: 'Hello',
    ru: 'Привет',
    es: 'Hola',
    fr: 'Bonjour',
    de: 'Hallo'
  };
}

getMessage() {
  const greeting = this.greetings[this.language] || this.greetings.en;
  return `${greeting}, ${this.name}!`;
}

display() {
  const message = this.getMessage();

  switch(environment) {
    case 'browser':
      // В браузере
      if (document.getElementById('hello-output')) {
        document.getElementById('hello-output').textContent = message;
      } else {
        const div = document.createElement('div');
        div.id = 'hello-output';
        div.textContent = message;
        div.style.cssText = `
          font-size: 24px;
          color: #4CAF50;
          padding: 20px;
          border: 2px solid #4CAF50;
          border-radius: 10px;
          margin: 20px;
          text-align: center;
        `;
        document.body.appendChild(div);
      }
      break;

    case 'node':

```

```

    // В Node.js
    const chalk = require('chalk');
    console.log(chalk.green.bold(message));
    console.log(chalk.gray(`Среда: Node.js ${process.version}`));
    break;

    default:
        // В любой другой среде
        console.log(message);
    }

    return message;
}

// Универсальный метод для получения информации
getInfo() {
    return {
        message: this.getMessage(),
        environment: environment,
        timestamp: this.timestamp.toISOString(),
        name: this.name,
        language: this.language,
        platform: environment === 'node' ? process.platform : navigator.platform
    };
}

// Экспорт в зависимости от среды
if (environment === 'browser') {
    window.HelloWorld = HelloWorld;
    console.log('HelloWorld доступен глобально. Используйте: new HelloWorld("имя",
"язык").display()');
} else if (environment === 'node') {
    module.exports = HelloWorld;
} else {
    // Для других сред (например, React Native)
    global.HelloWorld = HelloWorld;
}

})(typeof global !== 'undefined' ? global : this);

// Пример использования:
// В браузере: new HelloWorld('John', 'es').display()

```

```
// В Node.js: const HelloWorld = require('./universal-hello.js'); new HelloWorld('Jane',  
'fr').display()
```

4.4. Рекомендации для начинающих

1. **Начните с браузера** - визуальный результат мотивирует
2. **Используйте консоль** для быстрого тестирования кода
3. **Освойте Node.js** когда поймете основы JavaScript
4. **Пишите универсальный код** где это возможно
5. **Экспериментируйте** со всеми средами

"Hello, World!" в JavaScript - это не просто первая программа, а демонстрация мощи и гибкости языка, который может работать везде: от браузера до сервера, от консоли до мобильных устройств. Каждая среда открывает новые возможности и понимание этих различий делает вас лучше как разработчика.

3.5. Интерактивные примеры с console.log и alert

Введение: первый контакт с выводом информации

Когда вы только начинаете изучать JavaScript, важно сразу же видеть результаты своей работы. Два самых простых и понятных способа вывода информации — это `console.log()` для разработчиков и `alert()` для пользователей. В этой главе мы подробно рассмотрим оба метода, их различия, лучшие практики использования и интерактивные примеры, которые помогут вам освоить базовые принципы взаимодействия с JavaScript.

Зачем нужны console.log и alert?

- **console.log()** — ваш лучший друг при разработке. Позволяет выводить отладочную информацию в консоль браузера, не прерывая работу приложения.
- **alert()** — простейший способ показать сообщение пользователю. Создает модальное окно, которое требует взаимодействия.

Часть 1: console.log — инструмент разработчика

1.1. Базовое использование

```
javascript
```

```
// Простейший пример
```

```
console.log("Hello, World!");
```

```
// Вывод переменных
```

```
let name = "Анна";
```

```
let age = 25;
```

```
console.log(name); // "Анна"
```

```
console.log(age); // 25
```

```
// Вывод нескольких значений
```

```
console.log("Имя:", name, "Возраст:", age);
```

```
// Результат: Имя: Анна Возраст: 25
```

1.2. Расширенные возможности console.log

Форматирование строк

```
javascript

// Использование шаблонных строк (template literals)
const product = "яблоко";
const price = 50;
const quantity = 3;

console.log(`Товар: ${product}, Цена: ${price} руб., Количество: ${quantity}`);
// Результат: Товар: яблоко, Цена: 50 руб., Количество: 3

// Математические выражения внутри шаблонных строк
console.log(`Общая стоимость: ${price * quantity} рублей`);
// Результат: Общая стоимость: 150 рублей
```

Стилизация вывода

```
javascript

// Базовое CSS-форматирование
console.log("%cВажное сообщение!", "color: red; font-weight: bold; font-size: 16px;");

// Градиентный текст
console.log(
  "%cВнимание!",
  `
background: linear-gradient(45deg, #ff6b6b, #4ecdc4);
color: white;
font-size: 20px;
padding: 10px 20px;
border-radius: 5px;
font-family: Arial, sans-serif;
`
);

// Несколько стилей в одном сообщении
console.log(
  "%cСтатус: %cУСПЕХ",
  "color: gray; font-size: 14px;",
  "color: green; font-weight: bold; font-size: 16px;"
);
```

1.3. Табличное представление данных

javascript

// Простая таблица

```
const users = [
  { id: 1, name: "Иван", age: 25, city: "Москва" },
  { id: 2, name: "Мария", age: 30, city: "Санкт-Петербург" },
  { id: 3, name: "Алексей", age: 22, city: "Казань" }
];
```

console.table(users);

/*

(index)	id	name	age	city
0	1	"Иван"	25	"Москва"
1	2	"Мария"	30	"Санкт-Петербург"
2	3	"Алексей"	22	"Казань"

*/

// Таблица с группировкой

```
const products = [
  { category: "Фрукты", name: "Яблоко", price: 50, stock: 100 },
  { category: "Фрукты", name: "Банан", price: 40, stock: 150 },
  { category: "Овощи", name: "Морковь", price: 30, stock: 200 },
  { category: "Овощи", name: "Картофель", price: 20, stock: 300 }
];
```

console.table(products, ["category", "name", "price"]);

1.4. Группировка сообщений

javascript

// Простая группировка

```
console.group("Детали пользователя");
console.log("Имя: Иван");
console.log("Возраст: 25");
console.log("Город: Москва");
console.groupEnd();
```

/*

Детали пользователя


```

Имя: Иван
Возраст: 25
Город: Москва
*/

// Вложенные группы
console.group("Загрузка данных");
console.log("Запрос отправлен...");

console.group("Обработка ответа");
console.log("Ответ получен");
console.log("Парсинг JSON...");
console.log("Данные обработаны");
console.groupEnd();

console.log("Загрузка завершена");
console.groupEnd();

/*
Загрузка данных
Запрос отправлен...
Обработка ответа
Ответ получен
Парсинг JSON...
Данные обработаны
Загрузка завершена
*/

// Сворачиваемые группы (console.groupCollapsed)
console.groupCollapsed("Системная информация");
console.log("Версия браузера: Chrome 120");
console.log("Время загрузки: 2.3s");
console.log("Используемая память: 45MB");
console.groupEnd();
// Группа будет свернута по умолчанию

```

1.5. Отслеживание времени выполнения

javascript

```

// Замер времени выполнения кода
console.time("Время обработки массива");

// Имитация долгой операции

```

```

const largeArray = Array.from({ length: 1000000 }, (_, i) => i);

let sum = 0;
for (let i = 0; i < largeArray.length; i++) {
  sum += largeArray[i];
}

console.timeEnd("Время обработки массива");
// Вывод: Время обработки массива: 15.678ms

// Несколько замеров
console.time("Общее время");
console.time("Шаг 1");
// Какая-то операция...
setTimeout(() => {
  console.timeEnd("Шаг 1");

  console.time("Шаг 2");
  // Еще операция...
  setTimeout(() => {
    console.timeEnd("Шаг 2");
    console.timeEnd("Общее время");
  }, 100);
}, 100);

```

1.6. Интерактивные примеры с console.log

Пример 1: Калькулятор с логированием

javascript

```

function calculate(operation, a, b) {
  console.group(`Калькулятор: ${operation}`);
  console.log(`Входные данные: ${a}, ${b}`);

  let result;
  switch (operation) {
    case 'add':
      result = a + b;
      console.log(`Операция: сложение (${a} + ${b})`);
      break;
    case 'subtract':
      result = a - b;

```

```

    console.log(`Операция: вычитание (${a} - ${b})`);
    break;
case 'multiply':
    result = a * b;
    console.log(`Операция: умножение (${a} × ${b})`);
    break;
case 'divide':
    result = a / b;
    console.log(`Операция: деление (${a} ÷ ${b})`);
    break;
default:
    console.error('Неизвестная операция:', operation);
    result = null;
}

if (result !== null) {
    console.log(`Результат: ${result}`);
    console.log(`Тип результата: ${typeof result}`);
}

console.groupEnd();
return result;
}

// Использование
calculate('add', 10, 5);
calculate('multiply', 7, 3);

```

Пример 2: Отслеживание изменений состояния

```

javascript

class ShoppingCart {
    constructor() {
        this.items = [];
        this.total = 0;
    }

    addItem(name, price, quantity = 1) {
        console.group(`Добавление товара: ${name}`);

        const item = { name, price, quantity };
        this.items.push(item);

        const itemTotal = price * quantity;
    }
}

```

```

    this.total += itemTotal;

    console.log('Товар:', item);
    console.log('Стоимость товара:', itemTotal);
    console.log('Общая сумма корзины:', this.total);
    console.log('Всего товаров:', this.items.length);

    console.groupEnd();
    return this;
}

removeItem(index) {
    if (index >= 0 && index < this.items.length) {
        console.group(`Удаление товара: ${this.items[index].name}`);

        const removed = this.items.splice(index, 1)[0];
        this.total -= removed.price * removed.quantity;

        console.log('Удаленный товар:', removed);
        console.log('Общая сумма корзины:', this.total);
        console.log('Осталось товаров:', this.items.length);

        console.groupEnd();
    }
    return this;
}

showCart() {
    console.group('Содержимое корзины');
    console.table(this.items);
    console.log(`Общая сумма: ${this.total} руб.`);
    console.groupEnd();
}

// Использование
const cart = new ShoppingCart();
cart
    .addItem('Яблоки', 50, 2)
    .addItem('Хлеб', 30)
    .addItem('Молоко', 80, 2)
    .showCart()
    .removeItem(1)
    .showCart();

```

Часть 2: alert — взаимодействие с пользователем

2.1. Базовое использование alert

```
javascript

// Простейшее предупреждение
alert("Добро пожаловать на наш сайт!");

// Вывод переменных
const userName = "Иван";
alert(`Привет, ${userName}!`);

// Математические операции в alert
const x = 10;
const y = 5;
alert(`Сумма ${x} и ${y} равна ${x + y}`);

// Многострочные сообщения
alert("Важное сообщение!\n\nПожалуйста, сохраните вашу работу.\nСистема будет\nперезагружена через 5 минут.");
```

2.2. Особенности и ограничения alert

```
javascript

// Alert блокирует выполнение кода
console.log("Это сообщение появится сразу");
alert("Это alert - он заблокирует выполнение!");
console.log("Это сообщение появится только после закрытия alert");

// Alert автоматически преобразует данные в строки
alert(123); // "123"
alert(true); // "true"
alert(null); // "null"
alert({ name: "John" }); // "[object Object]"

// Для объектов используйте JSON.stringify
const user = { name: "Иван", age: 25 };
alert(JSON.stringify(user, null, 2));

/*
{
  "name": "Иван",

```

```
"age": 25
}
*/
```

2.3. Практические примеры с alert

Пример 1: Простая валидация формы

```
html

<!DOCTYPE html>
<html>
<head>
  <title>Пример валидации с alert</title>
</head>
<body>
  <h2>Регистрация</h2>

  <form id="registrationForm">
    <div>
      <label>Имя:</label>
      <input type="text" id="name" placeholder="Введите ваше имя">
    </div>

    <div>
      <label>Email:</label>
      <input type="email" id="email" placeholder="email@example.com">
    </div>

    <div>
      <label>Пароль:</label>
      <input type="password" id="password" placeholder="Не менее 6 символов">
    </div>

    <button type="submit">Зарегистрироваться</button>
  </form>

  <script>
    document.getElementById('registrationForm').addEventListener('submit', function(event)
    {
      event.preventDefault(); // Предотвращаем отправку формы

      // Получаем значения полей
```

```

const name = document.getElementById('name').value.trim();
const email = document.getElementById('email').value.trim();
const password = document.getElementById('password').value;

// Валидация
if (!name) {
    alert("Пожалуйста, введите ваше имя");
    return;
}

if (!email) {
    alert("Пожалуйста, введите email");
    return;
}

if (!email.includes('@')) {
    alert("Пожалуйста, введите корректный email адрес");
    return;
}

if (!password || password.length < 6) {
    alert("Пароль должен содержать не менее 6 символов");
    return;
}

// Если все проверки пройдены
alert(`Регистрация успешна!\n\nИмя: ${name}\nEmail: ${email}`);

// Здесь обычно отправка данных на сервер
console.log("Данные для отправки:", { name, email, password });
});
</script>
</body>
</html>

```

Пример 2: Простая викторина

```

html
<!DOCTYPE html>
<html>
<head>
<title>JavaScript Викторина</title>
<style>
body {

```

```
font-family: Arial, sans-serif;
max-width: 600px;
margin: 50px auto;
padding: 20px;
text-align: center;
}
button {
margin: 10px;
padding: 10px 20px;
font-size: 16px;
cursor: pointer;
}
.score {
font-size: 20px;
font-weight: bold;
margin-top: 20px;
}
</style>
</head>
<body>
<h1>Викторина по JavaScript</h1>
<p>Ответьте на 3 вопроса и проверьте свои знания!</p>

<button onclick="startQuiz()">Начать викторину</button>
<div class="score" id="scoreDisplay"></div>

<script>
const questions = [
  {
    question: "Какой оператор используется для строгого сравнения?",
    options: ["==", "===", "=", "!="],
    correct: 1
  },
  {
    question: "Как объявить переменную, которая не может быть переопределена?",
    options: ["var", "let", "const", "static"],
    correct: 2
  },
  {
    question: "Какой метод выводит сообщение в консоль браузера?",
    options: ["alert()", "prompt()", "console.log()", "print()"],
    correct: 2
  }
];
```



```

let score = 0;
let currentQuestion = 0;

function startQuiz() {
  score = 0;
  currentQuestion = 0;
  document.getElementById('scoreDisplay').textContent = '';
  askQuestion();
}

function askQuestion() {
  if (currentQuestion >= questions.length) {
    showResults();
    return;
  }

  const q = questions[currentQuestion];
  let optionsText = '';

  q.options.forEach((option, index) => {
    optionsText += `${index + 1}. ${option}\n`;
  });

  const answer = prompt(
    `Вопрос ${currentQuestion + 1} из ${questions.length}\n\n` +
    `${q.question}\n\n${optionsText}\n` +
    `Введите номер правильного ответа (1-${q.options.length}):`
  );

  if (answer === null) {
    alert("Викторина прервана");
    return;
  }

  const answerIndex = parseInt(answer) - 1;

  if (answerIndex === q.correct) {
    score++;
    alert("Правильно! 🎉 ");
  } else {
    alert(`Неправильно ❌\nПравильный ответ: ${q.options[q.correct]}`);
  }
}

```

```

currentQuestion++;
askQuestion();
}

function showResults() {
  const percentage = Math.round((score / questions.length) * 100);
  let message;

  if (percentage === 100) {
    message = "Отлично! Вы знаток JavaScript! 🏆 ";
  } else if (percentage >= 70) {
    message = "Хороший результат! 🥈 ";
  } else if (percentage >= 50) {
    message = "Неплохо, но есть куда расти 🥉 ";
  } else {
    message = "Попробуйте еще раз! 📖 ";
  }

  alert(
    `Викторина завершена!\n\n` +
    `Правильных ответов: ${score} из ${questions.length}\n` +
    `Результат: ${percentage}%\n\n` +
    message
  );

  document.getElementById('scoreDisplay').textContent =
    `Ваш результат: ${score}/${questions.length} (${percentage}%)`;
}
</script>
</body>
</html>

```

Часть 3: Сравнение console.log и alert

3.1. Ключевые различия

```

javascript

// Сравнительная таблица
console.log("=== СРАВНЕНИЕ console.log И alert ===");

const comparison = [

```

```

{
  aspect: "Назначение",
  consoleLog: "Отладка и логирование для разработчиков",
  alert: "Взаимодействие с конечными пользователями"
},
{
  aspect: "Визуальное представление",
  consoleLog: "Консоль браузера (F12)",
  alert: "Модальное окно поверх страницы"
},
{
  aspect: "Блокирование кода",
  consoleLog: "Не блокирует выполнение",
  alert: "Блокирует выполнение до закрытия"
},
{
  aspect: "Форматирование",
  consoleLog: "Поддерживает CSS, таблицы, группы",
  alert: "Только простой текст с \n для переносов"
},
{
  aspect: "Использование в production",
  consoleLog: "Часто удаляется или заменяется на логирование",
  alert: "Используется для важных уведомлений"
}
];

```

```

console.table(comparison);

```

```

// Практическое сравнение

```

```

function demonstrateDifferences() {
  console.log("1. Этот текст появится в консоли сразу");
  console.log("2. Можно выводить сложные объекты:", { data: [1, 2, 3], status: "ok" });
  console.log("3. Не прерывает выполнение программы");

  alert("1. Alert появится поверх страницы");
  alert("2. Программа остановится до закрытия окна");
  alert("3. Можно сообщить что-то важное пользователю");

  console.log("4. Этот текст появится только после закрытия всех alert");
}

```

```

// demonstrateDifferences(); // Раскомментируйте для теста

```

3.2. Когда что использовать

javascript

// Правильное использование console.log

```
console.log("=== ПРАВИЛЬНОЕ ИСПОЛЬЗОВАНИЕ ===");
```

// 1. Для отладки

```
function calculateTotal(prices) {  
  console.log("Начало calculateTotal");  
  console.log("Входные данные:", prices);  
  
  let total = 0;  
  for (let i = 0; i < prices.length; i++) {  
    console.log(`Обработка элемента ${i}:`, prices[i]);  
    total += prices[i];  
  }  
  
  console.log("Итоговая сумма:", total);  
  console.log("Конец calculateTotal");  
  return total;  
}
```

// 2. Для логирования событий

```
document.addEventListener('DOMContentLoaded', () => {  
  console.log("DOM загружен, начинаем инициализацию");  
  
  const buttons = document.querySelectorAll('button');  
  console.log(`Найдено ${buttons.length} кнопок`);  
  
  buttons.forEach((btn, index) => {  
    btn.addEventListener('click', () => {  
      console.log(`Клик по кнопке #${index}:`, btn.textContent);  
    });  
  });  
});
```

// 3. Для профилирования

```
function expensiveOperation() {  
  console.time("expensiveOperation");  
  // Какая-то тяжелая операция...  
  let result = 0;  
  for (let i = 0; i < 1000000; i++) {  
    result += Math.sqrt(i);  
  }  
}
```

```

}
console.timeEnd("expensiveOperation");
return result;
}

// Правильное использование alert
console.log("=== КОГДА ИСПОЛЬЗОВАТЬ alert ===");

// 1. Критические ошибки
function processPayment(amount) {
  if (amount <= 0) {
    alert("Ошибка: Сумма платежа должна быть больше 0");
    return false;
  }

  if (amount > 10000) {
    const confirm = window.confirm(
      "Внимание: сумма платежа превышает 10,000 руб.\n" +
      "Это может быть подозрительной операцией.\n\n" +
      "Продолжить?"
    );

    if (!confirm) {
      alert("Платеж отменен");
      return false;
    }
  }
}

// Обработка платежа...
alert(`Платеж на сумму ${amount} руб. успешно обработан!`);
return true;
}

// 2. Важные уведомления пользователю
function checkFormBeforeUnload() {
  const unsavedChanges = true; // Предположим, есть несохраненные изменения

  if (unsavedChanges) {
    const leave = confirm(
      "У вас есть несохраненные изменения.\n\n" +
      "Вы уверены, что хотите покинуть страницу?"
    );

    return leave;
  }
}

```

```

}

return true;
}

// 3. Подтверждение необратимых действий
function deleteAccount() {
  const confirmation = prompt(
    "ВНИМАНИЕ: Удаление аккаунта необратимо!\n\n" +
    "Все ваши данные будут удалены.\n\n" +
    "Для подтверждения введите 'УДАЛИТЬ':"
  );

  if (confirmation === "УДАЛИТЬ") {
    alert("Аккаунт будет удален в течение 24 часов");
    // Отправка запроса на удаление...
    return true;
  } else {
    alert("Удаление отменено");
    return false;
  }
}

```

Часть 4: Интерактивный учебный проект

4.1. Консольный калькулятор с подробным логированием

```

html

<!DOCTYPE html>
<html>
<head>
  <title>Интерактивный калькулятор с логированием</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

```

```
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
min-height: 100vh;
display: flex;
justify-content: center;
align-items: center;
padding: 20px;
}
```

```
.container {
display: flex;
gap: 30px;
max-width: 1200px;
width: 100%;
background: white;
border-radius: 20px;
box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
overflow: hidden;
}
```

```
.calculator {
flex: 1;
padding: 30px;
background: #f8f9fa;
}
```

```
.console {
flex: 1;
padding: 30px;
background: #1e1e1e;
color: #fff;
overflow-y: auto;
max-height: 600px;
}
```

```
h2 {
color: #333;
margin-bottom: 20px;
font-size: 24px;
}
```

```
.console h2 {
color: #fff;
}
```

```
.input-group {  
  margin-bottom: 20px;  
}
```

```
label {  
  display: block;  
  margin-bottom: 8px;  
  color: #555;  
  font-weight: 600;  
}
```

```
input, select {  
  width: 100%;  
  padding: 12px 15px;  
  border: 2px solid #ddd;  
  border-radius: 10px;  
  font-size: 16px;  
  transition: border-color 0.3s;  
}
```

```
input:focus, select:focus {  
  outline: none;  
  border-color: #667eea;  
}
```

```
.buttons {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
  gap: 15px;  
  margin-top: 30px;  
}
```

```
button {  
  padding: 15px;  
  border: none;  
  border-radius: 10px;  
  font-size: 16px;  
  font-weight: 600;  
  cursor: pointer;  
  transition: all 0.3s;  
}
```

```
.calculate-btn {  
  background: #667eea;
```



```
    color: white;
}

.calculate-btn:hover {
    background: #5a6fd8;
    transform: translateY(-2px);
}

.clear-btn {
    background: #ff6b6b;
    color: white;
}

.clear-btn:hover {
    background: #ff5252;
    transform: translateY(-2px);
}

.result {
    margin-top: 30px;
    padding: 20px;
    background: white;
    border-radius: 10px;
    border-left: 5px solid #667eea;
    display: none;
}

.result h3 {
    color: #333;
    margin-bottom: 10px;
}

#resultValue {
    font-size: 28px;
    font-weight: bold;
    color: #667eea;
}

.log-entry {
    padding: 10px;
    margin-bottom: 10px;
    background: #2d2d2d;
    border-radius: 5px;
    font-family: 'Courier New', monospace;
```

```
    font-size: 14px;
}

.log-timestamp {
  color: #888;
  font-size: 12px;
}

.log-info {
  color: #4ecdc4;
}

.log-error {
  color: #ff6b6b;
}

.log-success {
  color: #51cf66;
}

.clear-console-btn {
  background: #495057;
  color: white;
  padding: 10px 20px;
  margin-top: 20px;
}

.clear-console-btn:hover {
  background: #343a40;
}
</style>
</head>
<body>
<div class="container">
<div class="calculator">
  <h2>🧮 Интерактивный калькулятор</h2>

  <div class="input-group">
    <label for="num1">Первое число:</label>
    <input type="number" id="num1" placeholder="Введите число" value="10">
  </div>

  <div class="input-group">
    <label for="operation">Операция:</label>
```

```

<select id="operation">
  <option value="add">Сложение (+)</option>
  <option value="subtract">Вычитание (-)</option>
  <option value="multiply">Умножение (x)</option>
  <option value="divide">Деление (÷)</option>
  <option value="power">Возведение в степень (^)</option>
  <option value="sqrt">Квадратный корень (√)</option>
  <option value="percentage">Процент (%)</option>
</select>
</div>

<div class="input-group">
  <label for="num2">Второе число:</label>
  <input type="number" id="num2" placeholder="Введите число" value="5">
</div>

<div class="buttons">
  <button class="calculate-btn" onclick="calculate()">Вычислить</button>
  <button class="clear-btn" onclick="clearForm()">Очистить</button>
</div>

<div class="result" id="result">
  <h3>Результат:</h3>
  <div id="resultValue">0</div>
</div>
</div>

<div class="console">
  <h2>📜 Консоль логирования</h2>
  <div id="consoleOutput"></div>
  <button class="clear-console-btn" onclick="clearConsole()">Очистить консоль</button>
</div>
</div>

<script>
  // Массив для хранения логов
  let logs = [];

  // Функция для добавления сообщений в консоль
  function addLog(message, type = 'info') {
    const timestamp = new Date().toLocaleTimeString();
    const logEntry = {
      timestamp,
      message,
    }
  }

```

```

    type
  };

  logs.push(logEntry);
  updateConsole();

  // Также выводим в настоящую консоль
  switch(type) {
    case 'error':
      console.error(`[${timestamp}] ${message}`);
      break;
    case 'success':
      console.log(`%c[${timestamp}] ${message}`, 'color: green');
      break;
    default:
      console.log(`[${timestamp}] ${message}`);
  }
}

// Обновление отображения консоли
function updateConsole() {
  const consoleOutput = document.getElementById('consoleOutput');
  consoleOutput.innerHTML = '';

  logs.slice(-20).forEach(log => { // Показываем только последние 20 записей
    const logElement = document.createElement('div');
    logElement.className = `log-entry log-${log.type}`;
    logElement.innerHTML = `
      <div class="log-timestamp">[${log.timestamp}]</div>
      <div>${log.message}</div>
    `;
    consoleOutput.appendChild(logElement);
  });

  // Автопрокрутка к последнему сообщению
  consoleOutput.scrollTop = consoleOutput.scrollHeight;
}

// Основная функция вычисления
function calculate() {
  const num1 = parseFloat(document.getElementById('num1').value);
  const num2 = parseFloat(document.getElementById('num2').value);
  const operation = document.getElementById('operation').value;

```

// Логирование начала вычисления

```
addLog(`Начало вычисления: ${num1} ${getOperationSymbol(operation)} ${num2}`, 'info');
```

```
let result;
```

```
let error = null;
```

```
try {
```

```
  switch(operation) {
```

```
    case 'add':
```

```
      result = num1 + num2;
```

```
      addLog(`Выполнено сложение: ${num1} + ${num2} = ${result}`, 'info');
```

```
      break;
```

```
    case 'subtract':
```

```
      result = num1 - num2;
```

```
      addLog(`Выполнено вычитание: ${num1} - ${num2} = ${result}`, 'info');
```

```
      break;
```

```
    case 'multiply':
```

```
      result = num1 * num2;
```

```
      addLog(`Выполнено умножение: ${num1} × ${num2} = ${result}`, 'info');
```

```
      break;
```

```
    case 'divide':
```

```
      if (num2 === 0) {
```

```
        throw new Error('Деление на ноль!');
```

```
      }
```

```
      result = num1 / num2;
```

```
      addLog(`Выполнено деление: ${num1} ÷ ${num2} = ${result}`, 'info');
```

```
      break;
```

```
    case 'power':
```

```
      result = Math.pow(num1, num2);
```

```
      addLog(`Выполнено возведение в степень: ${num1}^${num2} = ${result}`, 'info');
```

```
      break;
```

```
    case 'sqrt':
```

```
      if (num1 < 0) {
```

```
        throw new Error('Квадратный корень из отрицательного числа!');
```

```
      }
```

```
      result = Math.sqrt(num1);
```

```
      addLog(`Вычислен квадратный корень: √${num1} = ${result}`, 'info');
```

```
      break;
```

```
    case 'percentage':
```

```
      result = (num1 * num2) / 100;
```

```
      addLog(`Вычислен процент: ${num2}% от ${num1} = ${result}`, 'info');
```

```
      break;
```

```
    default:
```

```
      throw new Error('Неизвестная операция');
```

```

}

// Показываем результат
document.getElementById('resultValue').textContent = result.toFixed(4);
document.getElementById('result').style.display = 'block';

// Логирование успешного завершения
addLog(`Вычисление успешно завершено. Результат: ${result}`, 'success');

// Показываем alert с результатом
alert(`Результат: ${num1} ${getOperationSymbol(operation)} ${num2} = ${
result.toFixed(4)}`);

} catch (err) {
  error = err.message;
  addLog(`Ошибка: ${error}`, 'error');

  document.getElementById('resultValue').textContent = 'Ошибка!';
  document.getElementById('result').style.display = 'block';

  // Показываем alert с ошибкой
  alert(`Ошибка: ${error}`);
}

// Дополнительная информация в консоль
console.group('Детали вычисления');
console.log('Число 1:', num1);
console.log('Число 2:', num2);
console.log('Операция:', operation);
console.log('Результат:', result);
console.log('Ошибка:', error);
console.log('Тип результата:', typeof result);
console.groupEnd();
}

// Вспомогательная функция для получения символа операции
function getOperationSymbol(operation) {
  const symbols = {
    'add': '+',
    'subtract': '-',
    'multiply': '×',
    'divide': '÷',
    'power': '^',
    'sqrt': '√',
  };

```

```

    'percentage': '%'
  };
  return symbols[operation] || '?';
}

// Очистка формы
function clearForm() {
  addLog('Форма очищена', 'info');

  document.getElementById('num1').value = '';
  document.getElementById('num2').value = '';
  document.getElementById('operation').value = 'add';
  document.getElementById('result').style.display = 'none';

  alert('Форма очищена!');
}

// Очистка консоли
function clearConsole() {
  logs = [];
  updateConsole();
  addLog('Консоль очищена', 'info');
  console.clear();
}

// Инициализация
document.addEventListener('DOMContentLoaded', () => {
  addLog('Калькулятор загружен и готов к работе!', 'success');
  addLog('Введите числа и выберите операцию', 'info');
  addLog('Нажмите "Вычислить" для получения результата', 'info');

  // Выводим приветственное сообщение в alert
  setTimeout(() => {
    alert('Добро пожаловать в интерактивный калькулятор!\n\n' +
      'Введите числа, выберите операцию и нажмите "Вычислить".\n' +
      'Все действия будут логироваться в консоли справа.');
```

```
  }, 500);
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

4.2. Практические упражнения для закрепления

javascript

// Упражнение 1: Создайте функцию, которая использует и console.log, и alert

```
function exercise1() {  
  console.log("=== Упражнение 1 ===");  
  console.log("Создаем функцию с использованием console.log и alert");  
  
  const numbers = [1, 2, 3, 4, 5];  
  
  // Выводим в консоль  
  console.log("Исходный массив:", numbers);  
  console.log("Длина массива:", numbers.length);  
  
  // Вычисляем сумму  
  const sum = numbers.reduce((a, b) => a + b, 0);  
  console.log("Сумма элементов:", sum);  
  
  // Выводим в alert  
  alert(`Массив: ${numbers.join(', ')}\nСумма: ${sum}`);  
  
  console.log("Упражнение 1 завершено\n");  
}
```

// Упражнение 2: Создайте интерактивную форму с валидацией

```
function exercise2() {  
  console.log("=== Упражнение 2 ===");  
  console.log("Создаем интерактивную форму");  
  
  // Имитируем получение данных от пользователя  
  const userName = prompt("Введите ваше имя:");  
  console.log("Пользователь ввел имя:", userName);  
  
  if (!userName || userName.trim() === "") {  
    console.error("Имя не может быть пустым");  
    alert("Ошибка: Имя не может быть пустым");  
    return;  
  }  
  
  const userAge = prompt("Введите ваш возраст:");  
  console.log("Пользователь ввел возраст:", userAge);  
  
  const age = parseInt(userAge);
```



```

if (isNaN(age) || age <= 0 || age > 120) {
  console.error("Некорректный возраст:", userAge);
  alert("Ошибка: Введите корректный возраст (1-120)");
  return;
}

// Выводим результат
console.group("Данные пользователя");
console.log("Имя:", userName);
console.log("Возраст:", age);
console.log("Совершеннолетний:", age >= 18 ? "Да" : "Нет");
console.groupEnd();

alert(
  `Регистрация успешна!\n\n` +
  `Имя: ${userName}\n` +
  `Возраст: ${age}\n` +
  `Статус: ${age >= 18 ? "Совершеннолетний" : "Несовершеннолетний"}`
);

console.log("Упражнение 2 завершено\n");
}

// Упражнение 3: Создайте простую игру с использованием console и alert
function exercise3() {
  console.log("=== Упражнение 3 ===");
  console.log("Создаем простую игру 'Угадай число'");

  // Генерируем случайное число
  const secretNumber = Math.floor(Math.random() * 100) + 1;
  console.log("Загаданное число:", secretNumber);

  let attempts = 0;
  const maxAttempts = 7;

  alert(
    "Добро пожаловать в игру 'Угадай число'!\n\n" +
    "Я загадал число от 1 до 100.\n" +
    `У вас есть ${maxAttempts} попыток, чтобы угадать его.\n\n` +
    "Удачи!"
  );

  while (attempts < maxAttempts) {

```

```

attempts++;

const guess = prompt(
  `Попытка ${attempts} из ${maxAttempts}\n\n` +
  "Введите ваше предположение (1-100):"
);

if (guess === null) {
  console.log("Игра прервана пользователем");
  alert("Игра прервана. Загаданное число было: " + secretNumber);
  return;
}

const number = parseInt(guess);
console.log(`Попытка ${attempts}: пользователь ввел ${number}`);

if (isNaN(number) || number < 1 || number > 100) {
  console.warn("Некорректный ввод:", guess);
  alert("Пожалуйста, введите число от 1 до 100");
  attempts--; // Не засчитываем некорректную попытку
  continue;
}

if (number === secretNumber) {
  console.log(`Пользователь угадал число с ${attempts} попытки!`);
  console.log("=== ИГРА ОКОНЧЕНА ===");

  alert(
    `🎉 Поздравляем! 🎉 \n\n` +
    `Вы угадали число ${secretNumber} с ${attempts} попытки!\n\n` +
    `Ваш результат: ${getRating(attempts, maxAttempts)}`
  );
  return;
}

// Даем подсказку
const hint = number < secretNumber ? "больше" : "меньше";
console.log(`Подсказка: число должно быть ${hint}`);

if (attempts < maxAttempts) {
  alert(`Не угадали! Загаданное число ${hint}.\nПопробуйте еще раз!`);
}
}

```

```

// Если попытки закончились
console.log(`Пользователь не угадал число за ${maxAttempts} попыток`);
console.log("=== ИГРА ОКОНЧЕНА ===");

alert(
  `❌ К сожалению, вы не угадали.\n\n` +
  `Загаданное число было: ${secretNumber}\n` +
  `Количество попыток: ${maxAttempts}\n\n` +
  "Попробуйте еще раз!"
);

function getRating(attempts, maxAttempts) {
  const percentage = ((maxAttempts - attempts + 1) / maxAttempts) * 100;
  if (percentage >= 80) return "Отлично! 🟩 🟩 🟩 🟩 ";
  if (percentage >= 60) return "Хорошо! 🟩 🟩 🟩 ";
  if (percentage >= 40) return "Неплохо! 🟩 🟩 ";
  return "Можно лучше! 🟩 ";
}

console.log("Упражнение 3 завершено\n");
}

// Функция для запуска всех упражнений
function runAllExercises() {
  console.clear();
  console.log("=== ЗАПУСК ВСЕХ УПРАЖНЕНИЙ ===");
  console.log(new Date().toLocaleString());
  console.log("=".repeat(50));

  exercise1();
  exercise2();
  exercise3();

  console.log("=".repeat(50));
  console.log("Все упражнения завершены!");

  alert(
    "Все упражнения завершены!\n\n" +
    "Проверьте консоль браузера (F12) для просмотра подробных логов.\n\n" +
    "Для повторного запуска нажмите F5."
  );
}

// Запускаем упражнения при загрузке страницы

```

```
document.addEventListener('DOMContentLoaded', () => {
  setTimeout(() => {
    const run = confirm(
      "Запустить практические упражнения?\n\n" +
      "В них демонстрируется использование:\n" +
      "• console.log для отладки\n" +
      "• alert для взаимодействия с пользователем\n" +
      "• prompt для ввода данных\n\n" +
      "Нажмите OK для запуска."
    );

    if (run) {
      runAllExercises();
    }
  }, 1000);
});
```

Заключение

Ключевые выводы

1. **console.log** — ваш основной инструмент для отладки и понимания того, как работает ваш код. Используйте его для:

- ➔ Вывода промежуточных результатов
- ➔ Отслеживания выполнения функций
- ➔ Профилирования производительности
- ➔ Логирования ошибок и предупреждений

2. **alert** — простой способ коммуникации с пользователем. Используйте его для:

- ➔ Важных уведомлений
- ➔ Подтверждения действий
- ➔ Отображения критических ошибок
- ➔ Простых форм ввода (вместе с prompt)

3. **Сочетайте оба метода** для создания интерактивных и отлаживаемых приложений:

- ➔ Используйте console.log во время разработки
- ➔ Используйте alert для важных сообщений пользователям
- ➔ Никогда не оставляйте отладочные console.log в продакшн-коде

Рекомендации по стилю

javascript

// ХОРОШО - информативные сообщения

```
console.log(`Пользователь ${userId} выполнил действие ${actionType}`);  
console.warn("Функция устарела, используйте новую версию");  
console.error("Не удалось подключиться к серверу:", error);
```

// ПЛОХО - непонятные сообщения

```
console.log(x); // Что такое x?  
console.log("ошибка"); // Какая ошибка? Где?
```

// ХОРОШО - понятные alert

```
alert("Изменения успешно сохранены");  
alert("Пожалуйста, заполните все обязательные поля");  
alert("Ошибка загрузки файла. Проверьте подключение к интернету");
```

// ПЛОХО - технические alert для пользователей

```
alert("Error 404: Resource not found");  
alert("TypeError: undefined is not an object");
```

Дальнейшие шаги

Теперь, когда вы освоили console.log и alert, вы готовы к:

1. **Более сложным методам console:** console.dir, console.assert, console.trace
2. **Альтернативам alert:** модальные окна, тосты, уведомления
3. **Работе с DOM:** изменение элементов на странице вместо всплывающих окон
4. **Асинхронным операциям:** работа с промисами и async/await
5. **Обработке ошибок:** try-catch блоки и кастомные ошибки

Помните: хорошее понимание основ — залог успеха в изучении JavaScript. Практикуйтесь, экспериментируйте и не бойтесь совершать ошибки — именно так вы научитесь программировать!

Часть II: Базовые конструкции языка

Глава 4: Переменные и типы данных

4.1. Объявление переменных: var, let, const

Введение: эволюция переменных в JavaScript

Объявление переменных — фундаментальная концепция любого языка программирования. В JavaScript подход к объявлению переменных значительно изменился за время развития языка, отражая эволюцию от простого скриптового языка к мощному инструменту для построения сложных приложений.

Исторический контекст

text

1995: var (ECMAScript 1)



2015: let и const (ECMAScript 6)



2020: Дополнения к let/const (ECMAScript 2020)

Часть 1: Основные понятия и термины

1.1. Что такое переменная?

Переменная — это именованная область памяти для хранения данных, значение которой может изменяться в процессе выполнения программы.

Аналогия из реального мира:

- **Переменная** как коробка с надписью
- **Имя переменной** — надпись на коробке
- **Значение** — содержимое коробки
- **Тип данных** — что можно хранить в коробке

```
javascript
```

```
// Простая аналогия
```

```
let коробка = "яблоки";    // Коробка с надписью "коробка" содержит "яблоки"
коробка = "груши";        // Теперь та же коробка содержит "груши"
console.log(коробка);     // "груши"
```

1.2. Ключевые характеристики переменных

```
javascript
```

```
// Демонстрация основных характеристик
```

```
console.log("=== ХАРАКТЕРИСТИКИ ПЕРЕМЕННЫХ ===");
```

```
// 1. Имя (идентификатор)
```

```
let userName;             // Имя: userName
const MAX_USERS = 100;    // Имя: MAX_USERS
```

```
// 2. Значение
```

```
userName = "Анна";        // Значение: "Анна"
let age = 25;             // Значение: 25
```

```
// 3. Тип данных (динамическая типизация)
```

```
let variable = "текст";   // Тип: string
variable = 42;            // Теперь тип: number
variable = true;          // Теперь тип: boolean
console.log("Тип variable:", typeof variable); // "boolean"
```

```
// 4. Область видимости (scope)
```

```
function exampleScope() {
    let localVar = "видна только внутри функции";
    console.log(localVar); // Работает
}
// console.log(localVar); // Ошибка! Переменная не видна снаружи
```

```
// 5. Время жизни (lifetime)
```

```
{
    let blockVar = "существует только в этом блоке";
    console.log(blockVar); // Работает
}
// console.log(blockVar); // Ошибка! Переменная уничтожена
```

```
console.log("=".repeat(50));
```

Часть 2: var — устаревший, но все еще встречающийся

2.1. Синтаксис и базовое использование

javascript

// Базовое объявление var

```
console.log("=== VAR: БАЗОВОЕ ИСПОЛЬЗОВАНИЕ ===");
```

// 1. Объявление без инициализации

```
var x;
```

```
console.log("x после объявления:", x); // undefined
```

// 2. Объявление с инициализацией

```
var y = 10;
```

```
console.log("y после инициализации:", y); // 10
```

// 3. Переопределение значения

```
y = 20;
```

```
console.log("y после изменения:", y); // 20
```

// 4. Переобъявление (особенность var)

```
var y = 30; // Можно переобъявить!
```

```
console.log("y после переобъявления:", y); // 30
```

// 5. Несколько переменных в одном объявлении

```
var a = 1, b = 2, c = 3;
```

```
console.log("a, b, c:", a, b, c); // 1, 2, 3
```

// 6. Цепочка присваиваний

```
var d = e = f = 100; // Опасная практика!
```

```
console.log("d, e, f:", d, e, f); // 100, 100, 100
```

// Примечание: e и f становятся глобальными переменными!

```
console.log("=".repeat(50));
```

2.2. Область видимости var (function scope)

javascript


```
console.log("=== VAR: ОБЛАСТЬ ВИДИМОСТИ (FUNCTION SCOPE) ===");

// Глобальная область видимости
var globalVar = "Я глобальная";

function testFunctionScope() {
    // var видна во всей функции, даже если объявлена внутри блока
    console.log("1. В начале функции:", someVar); // undefined (но не ошибка!)

    if (true) {
        var someVar = "Я внутри блока if";
        console.log("2. Внутри блока if:", someVar); // "Я внутри блока if"
    }

    console.log("3. После блока if:", someVar); // "Я внутри блока if" - все еще видна!

    // Еще один пример
    for (var i = 0; i < 3; i++) {
        var loopVar = "Итерация " + i;
        console.log("4. Внутри цикла:", loopVar);
    }

    console.log("5. После цикла:");
    console.log("    i:", i); // 3 - переменная цикла видна!
    console.log("    loopVar:", loopVar); // "Итерация 2" - тоже видна!

    // Вложенная функция
    function innerFunction() {
        var innerVar = "Я внутри вложенной функции";
        console.log("6. Во вложенной функции:", innerVar);

        // var из внешней функции видна
        console.log("7. Видна ли someVar?:", someVar); // "Я внутри блока if"
    }

    innerFunction();
    // console.log(innerVar); // Ошибка! innerVar не видна здесь
}

testFunctionScope();

// Глобальные переменные доступны везде
console.log("8. Глобальная переменная:", globalVar); // "Я глобальная"
```

```
// Опасность: случайное создание глобальных переменных
function dangerousFunction() {
    // Забыли var, let или const!
    accidentalGlobal = "Я стала глобальной!";
}
dangerousFunction();
console.log("9. Случайная глобальная:", accidentalGlobal); // Работает! Опасность!

console.log("=".repeat(50));
```

2.3. Поднятие (hoisting) переменных var

javascript

```
console.log("=== VAR: ПОДНЯТИЕ (HOISTING) ===");

// Пример 1: Простое поднятие
console.log("1. До объявления:", hoistedVar); // undefined (не ошибка!)
var hoistedVar = "Я поднялась!";
console.log("2. После объявления:", hoistedVar); // "Я поднялась!"

// Что происходит на самом деле:
// 1. Объявление var hoistedVar; поднимается вверх
// 2. console.log(hoistedVar); // undefined
// 3. hoistedVar = "Я поднялась!";
// 4. console.log(hoistedVar); // "Я поднялась!"

// Пример 2: Поднятие в функции
function hoistingExample() {
    console.log("3. В функции до объявления:", funcVar); // undefined
    var funcVar = "Локальная переменная";
    console.log("4. В функции после объявления:", funcVar); // "Локальная переменная"
}
hoistingExample();

// Пример 3: Поднятие с повторным объявлением
var repeatedVar = "Первое значение";
console.log("5. До повторного объявления:", repeatedVar); // "Первое значение"

// Второе объявление поднимается, но присваивание остается на месте
var repeatedVar = "Второе значение";
console.log("6. После повторного объявления:", repeatedVar); // "Второе значение"

// Фактический порядок выполнения:
```

```

// var repeatedVar; // Поднято
// repeatedVar = "Первое значение";
// console.log("5. До повторного объявления:", repeatedVar);
// // var repeatedVar; // Второе объявление игнорируется (уже объявлено)
// repeatedVar = "Второе значение";
// console.log("6. После повторного объявления:", repeatedVar);

// Пример 4: Опасность поднятия
function dangerousHoisting() {
  // Разработчик думает, что переменная локальная
  if (false) {
    var temp = "Этого кода никогда не выполнится";
  }

  console.log("7. temp существует?:", temp); // undefined (но не ошибка!)

  // Еще опаснее:
  for (var i = 0; i < 5; i++) {
    // что-то делаем
  }
  console.log("8. i после цикла:", i); // 5 - переменная цикла "утекла"
}

dangerousHoisting();

console.log("=".repeat(50));

```

2.4. Проблемы и ограничения var

```

javascript

console.log("=== VAR: ПРОБЛЕМЫ И ОГРАНИЧЕНИЯ ===");

// Проблема 1: Загрязнение глобального пространства имен
console.log("=== ПРОБЛЕМА 1: ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ===");

// script1.js
var appName = "МоеПриложение";
var config = { theme: "dark" };

// script2.js (загружается позже)
var appName = "ДругоеПриложение"; // Переопределяет глобальную переменную!
console.log("1. Конфликт имен:", appName); // "ДругоеПриложение"

```

// Проблема 2: Область видимости на уровне функции

```
console.log("\n=== ПРОБЛЕМА 2: FUNCTION SCOPE ===");
```

```
function problemFunction() {  
  for (var i = 0; i < 3; i++) {  
    setTimeout(function() {  
      console.log("2. i =", i); // Всегда 3!  
    }, 100);  
  }  
  console.log("3. i доступна снаружи цикла:", i); // 3  
}  
problemFunction();
```

// Проблема 3: Поднятие может запутать

```
console.log("\n=== ПРОБЛЕМА 3: СЛОЖНОСТИ HOISTING ===");
```

```
var value = "глобальная";  
function confusingExample() {  
  console.log("4. Какое значение?:", value); // undefined (не "глобальная!")  
  
  var value = "локальная";  
  console.log("5. А теперь?:", value); // "локальная"  
}  
confusingExample();
```

// Проблема 4: Отсутствие блочной области видимости

```
console.log("\n=== ПРОБЛЕМА 4: НЕТ БЛОЧНОЙ ОБЛАСТИ ВИДИМОСТИ ===");
```

```
function noBlockScope() {  
  if (true) {  
    var temp = "временные данные";  
    console.log("6. Внутри блока:", temp);  
  }  
}
```

// Переменная все еще существует!

```
console.log("7. Снаружи блока:", temp); // "временные данные"
```

// Это может привести к ошибкам:

```
var data = "важные данные";
```

// Другой разработчик добавляет:

```
if (data) {  
  var data = "новые данные"; // Переопределяет внешнюю переменную!  
  console.log("8. Внутренние данные:", data);  
}
```

```

}

console.log("9. Внешние данные:", data); // "новые данные" - потеряны старые!
}
noBlockScope();

// Проблема 5: Невозможность создать константы
console.log("\n=== ПРОБЛЕМА 5: НЕТ КОНСТАНТ ===");

var PI = 3.14;
PI = 3.14159; // Можно изменить!
console.log("10. PI изменен:", PI);

// Попытка эмулировать константу (не работает)
var CONSTANTS = {};
Object.defineProperty(CONSTANTS, "PI", {
  value: 3.14,
  writable: false
});
CONSTANTS.PI = 3.14159; // Не изменится
console.log("11. CONSTANTS.PI:", CONSTANTS.PI); // 3.14
// Но это сложно и неочевидно

console.log("=".repeat(50));

```

Часть 3: let — современная замена var

3.1. Синтаксис и базовое использование

```

javascript

console.log("=== LET: БАЗОВОЕ ИСПОЛЬЗОВАНИЕ ===");

// 1. Объявление без инициализации
let variable1;
console.log("1. variable1 после объявления:", variable1); // undefined

// 2. Объявление с инициализацией
let variable2 = "значение";
console.log("2. variable2 после инициализации:", variable2); // "значение"

// 3. Изменение значения

```

```

variable2 = "новое значение";
console.log("3. variable2 после изменения:", variable2); // "новое значение"

// 4. Невозможно переобъявить в той же области видимости
let variable3 = "первое";
// let variable3 = "второе"; // Ошибка: Identifier 'variable3' has already been declared

// 5. Но можно в разных областях
{
  let variable3 = "во внутреннем блоке";
  console.log("4. variable3 во внутреннем блоке:", variable3); // "во внутреннем блоке"
}
console.log("5. variable3 во внешнем блоке:", variable3); // "первое"

// 6. Несколько переменных в одном объявлении
let x = 1, y = 2, z = 3;
console.log("6. x, y, z:", x, y, z); // 1, 2, 3

// 7. Деструктуризация при объявлении
let [a, b] = [10, 20];
console.log("7. a, b через деструктуризацию:", a, b); // 10, 20

let { firstName, lastName } = { firstName: "Иван", lastName: "Иванов" };
console.log("8. firstName, lastName:", firstName, lastName); // "Иван", "Иванов"

console.log("=".repeat(50));

```

3.2. Область видимости let (block scope)

```

javascript

console.log("=== LET: ОБЛАСТЬ ВИДИМОСТИ (BLOCK SCOPE) ===");

// Глобальная область видимости
let globalLet = "Я глобальная let";

function testLetScope() {
  // let не поднимается так, как var
  // console.log(someLet); // Ошибка ReferenceError!

  if (true) {
    let someLet = "Я внутри блока if";
    console.log("1. Внутри блока if:", someLet); // "Я внутри блока if"
  }
}

```

```

// console.log("После блока if:", someLet); // Ошибка! someLet не видна

// Циклы с Let
for (let i = 0; i < 3; i++) {
    let loopLet = "Итерация " + i;
    console.log("2. Внутри цикла:", loopLet);
}

// console.log("i после цикла:", i); // Ошибка! i не видна
// console.log("loopVar после цикла:", loopLet); // Ошибка! loopLet не видна

// Блочная область видимости
{
    let blockScoped = "Только в этом блоке";
    console.log("3. Внутри блока:", blockScoped);
}
// console.log(blockScoped); // Ошибка!

// Вложенные блоки
let outer = "внешняя";
{
    let inner = "внутренняя";
    console.log("4. Внутренний блок видит outer:", outer); // "внешняя"
    outer = "изменено изнутри";
}
console.log("5. outer после внутреннего блока:", outer); // "изменено изнутри"
// console.log(inner); // Ошибка!
}

testLetScope();
console.log("6. Глобальная let:", globalLet); // "Я глобальная let"

// Разница между var и let в циклах
console.log("\n=== СРАВНЕНИЕ LET И VAR В ЦИКЛАХ ===");

console.log("C var:");
for (var varIndex = 0; varIndex < 3; varIndex++) {
    setTimeout(() => {
        console.log("varIndex:", varIndex); // Всегда 3!
    }, 100);
}

console.log("C let:");

```

```

for (let letIndex = 0; letIndex < 3; letIndex++) {
  setTimeout(() => {
    console.log("letIndex:", letIndex); // 0, 1, 2 - правильно!
  }, 100);
}

// Каждый цикл создает новую переменную letIndex
// Эквивалентно:
{
  let letIndex = 0;
  setTimeout(() => console.log("letIndex:", letIndex), 100);
}
{
  let letIndex = 1;
  setTimeout(() => console.log("letIndex:", letIndex), 100);
}
{
  let letIndex = 2;
  setTimeout(() => console.log("letIndex:", letIndex), 100);
}

console.log("=".repeat(50));

```

3.3. Поднятие let и временная мертвая зона (TDZ)

javascript

```

console.log("=== LET: ПОДНЯТИЕ И ВРЕМЕННАЯ МЕРТВАЯ ЗОНА ===");

// Пример 1: Временная мертвая зона
// console.log(tdzVar); // Ошибка ReferenceError: Cannot access 'tdzVar' before
// initialization
let tdzVar = "Я в TDZ";
console.log("1. После инициализации:", tdzVar); // "Я в TDZ"

// Пример 2: TDZ в разных контекстах
function tdzExample() {
  // Область TDZ начинается здесь
  // console.log(localTdz); // Ошибка!

  let localTdz = "локальная TDZ";
  console.log("2. После инициализации в функции:", localTdz);
}

```



```
tdzExample();
```

```
// Пример 3: TDZ с блочной областью видимости
```

```
{  
  // Начало TDZ для blockTdz  
  // console.log(blockTdz); // Ошибка!  
  
  let blockTdz = "блочная TDZ";  
  console.log("3. В блоке после инициализации:", blockTdz);  
}
```

```
// Пример 4: Сравнение с var
```

```
console.log("\n=== СРАВНЕНИЕ С VAR ===");  
  
console.log("4. var до объявления:", hoistedVar); // undefined  
var hoistedVar = "var поднята";  
  
// console.log("5. let до объявления:", hoistedLet); // Ошибка!  
let hoistedLet = "let технически тоже поднята, но в TDZ";
```

```
// Пример 5: TDZ с параметрами по умолчанию
```

```
function tdzWithParams(x = y, y = 10) {  
  // При вызове: x = y, но y еще в TDZ!  
  console.log("6. x, y:", x, y);  
}  
// tdzWithParams(); // Ошибка! y is not defined
```

```
function correctParams(x = 10, y = x * 2) {  
  // x инициализирован раньше, поэтому можно использовать  
  console.log("7. x, y:", x, y); // 10, 20  
}  
correctParams();
```

```
// Пример 6: Циклическая зависимость в TDZ
```

```
{  
  // let a = b; // Ошибка! b еще не определен  
  // let b = a;  
}
```

```
// Пример 7: typeof в TDZ
```

```
// console.log(typeof notDefinedVar); // "undefined" (глобальная переменная не  
существует)  
// console.log(typeof notDefinedLet); // Ошибка! Переменная в TDZ
```

```
let notDefinedLet; // Объявлена, но не инициализирована
console.log("8. typeof notDefinedLet:", typeof notDefinedLet); // "undefined"

console.log("=".repeat(50));
```

3.4. Практические преимущества let

```
javascript

console.log("=== LET: ПРАКТИЧЕСКИЕ ПРЕИМУЩЕСТВА ===");

// Преимущество 1: Безопасность в циклах
console.log("=== ПРЕИМУЩЕСТВО 1: ЦИКЛЫ ===");

const buttons = ["A", "B", "C"];

// С var - проблема
console.log("C var:");
for (var i = 0; i < buttons.length; i++) {
    // Эмуляция добавления обработчика
    setTimeout(() => {
        console.log(`Кнопка ${buttons[i]} нажата`); // Всегда "Кнопка undefined нажата"
    }, 100);
}

// С let - работает правильно
console.log("\nC let:");
for (let i = 0; i < buttons.length; i++) {
    setTimeout(() => {
        console.log(`Кнопка ${buttons[i]} нажата`); // "Кнопка A нажата", "Кнопка B
        нажата"...
    }, 100);
}

// Преимущество 2: Изоляция блоков кода
console.log("\n=== ПРЕИМУЩЕСТВО 2: ИЗОЛЯЦИЯ ===");

function processData(data) {
    // Блок для предобработки
    {
        let temp = JSON.parse(data);
        console.log("1. Предобработанные данные:", temp);
        // temp используется только здесь
    }
}
```

```

// Блок для основной обработки
{
  let result = 0;
  // вычисления...
  console.log("2. Результат вычислений:", result);
}

// Переменные temp и result не конфликтуют и не "утекают"
}

// Преимущество 3: Улучшенная читаемость
console.log("\n=== ПРЕИМУЩЕСТВО 3: ЧИТАЕМОСТЬ ===");

function calculateOrder(order) {
  // Все переменные объявлены близко к месту использования
  let subtotal = 0;

  for (let item of order.items) {
    let itemTotal = item.price * item.quantity;
    subtotal += itemTotal;
  }

  let tax = subtotal * 0.2;
  let discount = order.customer.isPremium ? subtotal * 0.1 : 0;
  let total = subtotal + tax - discount;

  return total;
}

// Преимущество 4: Предотвращение случайных переопределений
console.log("\n=== ПРЕИМУЩЕСТВО 4: ЗАЩИТА ОТ ПЕРЕОПРЕДЕЛЕНИЙ ===");

function loadUserData(userId) {
  let data = fetchUserData(userId);

  // Много кода...

  // Разработчик случайно пытается переобъявить
  // let data = processData(data); // Ошибка! Защищает от багов

  // Нужно явно переназначить:
  data = processData(data);
  return data;
}

```

```
}
```

```
function fetchUserData(id) {  
  return { id, name: "User" + id };  
}
```

```
function processData(data) {  
  return { ...data, processed: true };  
}
```

```
// Преимущество 5: Упрощение рефакторинга
```

```
console.log("\n=== ПРЕИМУЩЕСТВО 5: РЕФАКТОРИНГ ===");
```

```
// Легко перемещать блоки кода
```

```
function oldFunction() {  
  var result = 0;  
  
  if (condition) {  
    var temp = calculate();  
    result = temp * 2;  
  }  
  
// temp все еще существует здесь!  
  return result;  
}
```

```
// temp все еще существует здесь!
```

```
return result;  
}
```

```
function newFunction() {  
  let result = 0;  
  
  if (condition) {  
    let temp = calculate(); // Существует только в блоке if  
    result = temp * 2;  
  }  
  
// temp автоматически удалена  
  return result;  
}
```

```
// temp автоматически удалена
```

```
return result;  
}
```

```
console.log("=".repeat(50));
```

Часть 4: const — неизменяемые переменные

4.1. Синтаксис и базовое использование

javascript

```
console.log("=== CONST: БАЗОВОЕ ИСПОЛЬЗОВАНИЕ ===");

// 1. Обязательная инициализация при объявлении
// const noValue; // Ошибка: Missing initializer in const declaration

// 2. Базовые примитивные значения
const PI = 3.14159;
const APP_NAME = "МоеПриложение";
const IS_PRODUCTION = true;
const NULL_VALUE = null;
const UNDEFINED_VALUE = undefined;
const SYMBOL = Symbol("id");

console.log("1. Примитивные константы:");
console.log("  PI:", PI);
console.log("  APP_NAME:", APP_NAME);
console.log("  IS_PRODUCTION:", IS_PRODUCTION);

// 3. Нельзя изменить значение
// PI = 3.14; // Ошибка: Assignment to constant variable
// APP_NAME = "НовоеИмя"; // Ошибка

// 4. Нельзя переобъявить
// const PI = 3.14; // Ошибка: Identifier 'PI' has already been declared

// 5. Но можно в разных областях видимости
{
  const PI = "локальное значение";
  console.log("2. PI во внутреннем блоке:", PI); // "локальное значение"
}
console.log("3. PI во внешнем блоке:", PI); // 3.14159

// 6. Объекты и массивы - особый случай
const person = { name: "Анна", age: 25 };
const numbers = [1, 2, 3, 4, 5];

console.log("4. Объект и массив до изменений:");
```

```

console.log("  person:", person);
console.log("  numbers:", numbers);

// Можно изменять содержимое!
person.age = 26;
person.city = "Москва";
numbers.push(6);
numbers[0] = 100;

console.log("5. Объект и массив после изменений:");
console.log("  person:", person); // {name: "Анна", age: 26, city: "Москва"}
console.log("  numbers:", numbers); // [100, 2, 3, 4, 5, 6]

// Но нельзя переприсвоить
// person = { name: "Борис" }; // Ошибка!
// numbers = [7, 8, 9]; // Ошибка!

// 7. Заморозка объектов
const frozenPerson = Object.freeze({ name: "Иван", age: 30 });
// frozenPerson.age = 31; // Тихая ошибка в strict mode или игнорируется
// frozenPerson.city = "Москва"; // Не добавится
console.log("6. Замороженный объект:", frozenPerson);

// 8. const с деструктуризацией
const [first, second] = [10, 20, 30];
const { width, height } = { width: 100, height: 200 };
console.log("7. Деструктуризация:");
console.log("  first, second:", first, second); // 10, 20
console.log("  width, height:", width, height); // 100, 200

console.log("=".repeat(50));

```

4.2. Когда использовать const

javascript

```

console.log("=== CONST: КОГДА ИСПОЛЬЗОВАТЬ ===");

// Случай 1: Магические числа и строки
console.log("=== СЛУЧАЙ 1: КОНФИГУРАЦИОННЫЕ КОНСТАНТЫ ===");

const MAX_FILE_SIZE = 5 * 1024 * 1024; // 5MB
const API_URL = "https://api.example.com/v1";
const DEFAULT_TIMEOUT = 30000; // 30 секунд

```

```
const SUPPORTED_LANGUAGES = ["ru", "en", "es"];
const ERROR_MESSAGES = {
  NOT_FOUND: "Ресурс не найден",
  UNAUTHORIZED: "Требуется авторизация",
  SERVER_ERROR: "Внутренняя ошибка сервера"
};

function uploadFile(file) {
  if (file.size > MAX_FILE_SIZE) {
    throw new Error(`Файл слишком большой. Максимум: ${MAX_FILE_SIZE} байт`);
  }

  // Использование констант делает код понятнее
  console.log(`Загружаем на ${API_URL} с таймаутом ${DEFAULT_TIMEOUT}мс`);
}

// Случай 2: Импортированные модули
console.log("\n=== СЛУЧАЙ 2: ИМПОРТЫ И ЗАВИСИМОСТИ ===");

// Представьте, что это импорты:
// import React from 'react';
// import lodash from 'lodash';

const React = { createElement: () => {} }; // Заглушка для примера
const lodash = { chunk: () => {} }; // Заглушка

// Импортированные модули всегда const
const MyComponent = () => {
  return React.createElement("div", null, "Hello");
};

// Случай 3: Результаты вычислений, которые не должны меняться
console.log("\n=== СЛУЧАЙ 3: РЕЗУЛЬТАТЫ ВЫЧИСЛЕНИЙ ===");

function calculateCircle(radius) {
  const diameter = radius * 2;
  const circumference = 2 * Math.PI * radius;
  const area = Math.PI * radius ** 2;

  // Эти значения вычислены и больше не меняются
  return { diameter, circumference, area };
}

const circle = calculateCircle(10);
```

```
console.log("Круг радиусом 10:");  
console.log(" Диаметр:", circle.diameter); // 20  
console.log(" Длина окружности:", circle.circumference); // ~62.83  
console.log(" Площадь:", circle.area); // ~314.16
```

// Случай 4: Объекты конфигурации

```
console.log("\n=== СЛУЧАЙ 4: КОНФИГУРАЦИОННЫЕ ОБЪЕКТЫ ===");
```

```
const APP_CONFIG = {  
  theme: "dark",  
  language: "ru",  
  features: {  
    notifications: true,  
    analytics: false,  
    offlineMode: true  
  },  
  api: {  
    baseUrl: "https://api.example.com",  
    version: "v1",  
    endpoints: {  
      users: "/users",  
      posts: "/posts"  
    }  
  }  
};
```

// Можно безопасно использовать, зная, что ссылка не изменится

```
function initializeApp(config) {  
  console.log(`Инициализация приложения с темой: ${config.theme}`);  
  console.log(`API базовый URL: ${config.api.baseUrl}`);
```

// Внутренние изменения разрешены, но с осторожностью

```
  config.initialized = true;  
}
```

```
initializeApp(APP_CONFIG);
```

// Случай 5: Колбэки и обработчики

```
console.log("\n=== СЛУЧАЙ 5: ФУНКЦИИ И КОЛБЭКИ ===");
```

```
const handleClick = (event) => {  
  console.log("Клик обработан", event.target);  
};
```



```

const validateEmail = (email) => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
};

// Эти функции не должны переназначаться
document.addEventListener("click", handleClick);
console.log("Email валиден?", validateEmail("test@example.com"));

// Случай 6: Кэшированные значения
console.log("\n=== СЛУЧАЙ 6: КЭШИРОВАНИЕ ===");

const primeNumbersCache = [2, 3, 5, 7, 11, 13];

function isPrime(num) {
  if (num < 2) return false;

  // Проверяем кэш
  if (primeNumbersCache.includes(num)) {
    console.log(`Число ${num} найдено в кэше`);
    return true;
  }

  // Вычисляем и добавляем в кэш
  for (let i = 2; i <= Math.sqrt(num); i++) {
    if (num % i === 0) return false;
  }

  primeNumbersCache.push(num); // Изменяем содержимое массива
  return true;
}

console.log("Проверка простых чисел:");
console.log(" 7 -", isPrime(7)); // Из кэша
console.log("17 -", isPrime(17)); // Вычисляем и кэшируем
console.log(" Кэш:", primeNumbersCache);

console.log("=".repeat(50));

```

4.3. const с объектами и массивами

javascript

```

console.log("=== CONST: ОБЪЕКТЫ И МАССИВЫ ===");

```

// Пример 1: Мутация объектов

```
console.log("=== ПРИМЕР 1: МУТАЦИЯ ОБЪЕКТОВ ===");
```

```
const user = {  
  id: 1,  
  name: "Анна",  
  profile: {  
    age: 25,  
    email: "anna@example.com"  
  }  
};
```

```
console.log("1. Исходный объект:", JSON.stringify(user, null, 2));
```

// Разрешено: Изменение свойств

```
user.name = "Анна Иванова";  
user.profile.age = 26;
```

// Разрешено: Добавление новых свойств

```
user.city = "Москва";  
user.profile.phone = "+79991234567";
```

// Разрешено: Удаление свойств

```
delete user.profile.email;
```

```
console.log("2. После изменений:", JSON.stringify(user, null, 2));
```

// Запрещено: Переприсвоение

// user = { id: 2, name: "Борис" }; // Ошибка!

// Пример 2: Массивы

```
console.log("\n=== ПРИМЕР 2: МАССИВЫ ===");
```

```
const colors = ["red", "green", "blue"];  
console.log("3. Исходный массив:", colors);
```

// Разрешено: Изменение элементов

```
colors[0] = "красный";  
colors[1] = "зеленый";
```

// Разрешено: Добавление элементов

```
colors.push("желтый");  
colors.unshift("оранжевый");
```

```
// Разрешено: Удаление элементов
colors.pop(); // Удаляем "желтый"
const removed = colors.splice(1, 1); // Удаляем "зеленый"

console.log("4. После изменений:", colors);
console.log("5. Удаленные элементы:", removed);

// Пример 3: Защита от мутаций
console.log("\n=== ПРИМЕР 3: ЗАЩИТА ОТ МУТАЦИЙ ===");

// Object.freeze() - поверхностная заморозка
const frozenConfig = Object.freeze({
  apiUrl: "https://api.example.com",
  timeout: 30000,
  retries: 3
});

console.log("6. Замороженный объект:");
console.log("  frozenConfig.apiUrl:", frozenConfig.apiUrl);

// Попытка изменения (в strict mode будет ошибка)
frozenConfig.apiUrl = "https://new-api.example.com";
console.log("7. apiUrl после попытки изменения:", frozenConfig.apiUrl); // Не изменился

// Но вложенные объекты не защищены!
const shallowFrozen = Object.freeze({
  settings: { theme: "dark", language: "ru" }
});

shallowFrozen.settings.theme = "light"; // Разрешено!
console.log("8. Вложенный объект изменен:", shallowFrozen.settings.theme); // "light"

// Глубокая заморозка
function deepFreeze(obj) {
  Object.freeze(obj);
  for (const key in obj) {
    if (obj[key] !== null && typeof obj[key] === 'object') {
      deepFreeze(obj[key]);
    }
  }
  return obj;
}
```

```
const deepFrozen = deepFreeze({
  config: {
    api: { url: "https://api.example.com", version: "v1" },
    ui: { theme: "dark", fontSize: 16 }
  }
});

// deepFrozen.config.api.url = "https://new.url"; // Ошибка в strict mode
console.log("9. Глубоко замороженный объект защищен");

// Пример 4: Immutable подход
console.log("\n=== ПРИМЕР 4: IMMUTABLE ПОДХОД ===");

const initialState = Object.freeze({
  user: null,
  isLoading: false,
  error: null
});

// Вместо мутации создаем новый объект
function setUser(state, user) {
  return Object.freeze({
    ...state,
    user,
    isLoading: false,
    error: null
  });
}

function setLoading(state, isLoading) {
  return Object.freeze({
    ...state,
    isLoading
  });
}

let currentState = initialState;
console.log("10. Начальное состояние:", currentState);

currentState = setLoading(currentState, true);
console.log("11. Состояние после setLoading:", currentState);

currentState = setUser(currentState, { id: 1, name: "Анна" });
console.log("12. Состояние после setUser:", currentState);
```

```
// Пример 5: const с Map и Set
console.log("\n=== ПРИМЕР 5: MAP И SET ===");

const userMap = new Map();
userMap.set(1, { name: "Анна", age: 25 });
userMap.set(2, { name: "Борис", age: 30 });

console.log("13. Map до изменений:");
console.log("    Размер:", userMap.size);
console.log("    Пользователь 1:", userMap.get(1));

// Мутация Map разрешена
userMap.set(1, { name: "Анна Иванова", age: 26 });
userMap.delete(2);
userMap.set(3, { name: "Виктор", age: 35 });

console.log("14. Map после изменений:");
console.log("    Размер:", userMap.size);
console.log("    Пользователь 1:", userMap.get(1));

const uniqueNumbers = new Set([1, 2, 3, 3, 4, 5]);
console.log("15. Set:", uniqueNumbers); // Set(5) {1, 2, 3, 4, 5}

uniqueNumbers.add(6);
uniqueNumbers.delete(1);
console.log("16. Set после изменений:", uniqueNumbers); // Set(5) {2, 3, 4, 5, 6}

console.log("=".repeat(50));
```

Часть 5: Сравнительный анализ

5.1. Сравнительная таблица

```
javascript

console.log("=== СРАВНИТЕЛЬНАЯ ТАБЛИЦА: VAR, LET, CONST ===");

// Создаем объект для сравнения
const comparison = {
  var: {
    объявление: "var x; или var x = 1;",
```

```

    переопределение: "Разрешено в той же области",
    областьВидимости: "Function scope",
    поднятие: "Да, с undefined",
    временнаяМертваяЗона: "Нет",
    использование: "Устаревшее, не рекомендуется",
    пример: "var counter = 0;"
  },
  let: {
    объявление: "let x; или let x = 1;",
    переопределение: "Запрещено в той же области",
    областьВидимости: "Block scope",
    поднятие: "Да, но с TDZ",
    временнаяМертваяЗона: "Да",
    использование: "Для переменных, которые изменяются",
    пример: "let i = 0;"
  },
  const: {
    объявление: "const x = 1; (обязательна инициализация)",
    переопределение: "Запрещено",
    областьВидимости: "Block scope",
    поднятие: "Да, но с TDZ",
    временнаяМертваяЗона: "Да",
    использование: "Для констант и ссылок",
    пример: "const PI = 3.14;"
  }
};

```

// Выводим таблицу

```
console.table(comparison);
```

// Детальное сравнение на примерах

```
console.log("\n=== ДЕТАЛЬНОЕ СРАВНЕНИЕ НА ПРИМЕРАХ ===");
```

```
console.log("1. Область видимости:");
```

```
console.log("  var (function scope):");
```

```
function scopeExample() {
```

```
  if (true) {
```

```
    var functionScoped = "видна везде в функции";
```

```
    let blockScoped = "видна только в блоке";
```

```
    const constBlockScoped = "тоже только в блоке";
```

```
  }
```

```
// console.log(functionScoped); // Работает
```

```
// console.log(blockScoped); // Ошибка
```

```
// console.log(constBlockScoped); // Ошибка
```

```

}

console.log("\n2. Поднятие (hoisting):");
console.log("    До объявления:");
console.log("        var variable:", typeof hoistedVar); // "undefined" (но существует)
// console.log("        let variable:", hoistedLet); // Ошибка ReferenceError
// console.log("        const variable:", hoistedConst); // Ошибка ReferenceError

var hoistedVar = "поднята";
let hoistedLet = "в TDZ";
const hoistedConst = "в TDZ";

console.log("\n3. Переопределение:");
var canBeRedefined = "первое";
var canBeRedefined = "второе"; // ОК

let cannotBeRedefined = "первое";
// let cannotBeRedefined = "второе"; // Ошибка

const cannotBeRedefinedOrChanged = "единственное";
// cannotBeRedefinedOrChanged = "новое"; // Ошибка

console.log("\n4. Циклы:");
console.log("    С var:");
for (var v = 0; v < 3; v++) {
    // v общая для всех итераций
}
console.log("        v после цикла:", v); // 3

console.log("\n    С let/const:");
for (let l = 0; l < 3; l++) {
    // Каждая итерация имеет свою l
}
// console.log("l после цикла:", l); // Ошибка

console.log("=".repeat(50));

```

5.2. Рекомендации по выбору

```

javascript

console.log("=== РЕКОМЕНДАЦИИ ПО ВЫБОРУ ===");

// Правило 1: Всегда начинайте с const

```

```
console.log("=== ПРАВИЛО 1: НАЧИНАЙТЕ С CONST ===");
```

```
// Плохо:
```

```
var x = 5;  
var y = 10;  
var result = x + y;
```

```
// Хорошо:
```

```
const x = 5;  
const y = 10;  
const result = x + y;
```

```
// Если нужно изменить, переходите к let:
```

```
let counter = 0;  
counter = 1; // Теперь понятно, что переменная изменяется
```

```
// Правило 2: Никогда не используйте var
```

```
console.log("\n=== ПРАВИЛО 2: ИЗБЕГАЙТЕ VAR ===");
```

```
// Плохо (устаревший код):
```

```
function oldStyle() {  
    var data = fetchData();  
    var processed = process(data);  
    return processed;  
}
```

```
// Хорошо (современный код):
```

```
function newStyle() {  
    const data = fetchData();  
    const processed = process(data);  
    return processed;  
}
```

```
// Правило 3: Используйте UPPER_CASE для настоящих констант
```

```
console.log("\n=== ПРАВИЛО 3: ИМЕНА КОНСТАНТ ===");
```

```
// Конфигурационные константы:
```

```
const API_BASE_URL = "https://api.example.com";  
const MAX_RETRY_ATTEMPTS = 3;  
const DEFAULT_TIMEOUT_MS = 30000;
```

```
// Неизменяемые значения:
```

```
const PI = 3.14159;  
const DAYS_IN_WEEK = 7;
```



```
const SECONDS_IN_MINUTE = 60;

// Правило 4: const для импортов и экспортов
console.log("\n=== ПРАВИЛО 4: МОДУЛИ ===");

// В модулях:
// import React from 'react'; // React - const
// import { useState } from 'react'; // useState - const

// Экспорт:
const MAX_USERS = 100;
const DEFAULT_CONFIG = { theme: 'dark' };
function helperFunction() { }

// export { MAX_USERS, DEFAULT_CONFIG, helperFunction };

// Правило 5: let для переменных цикла и состояния
console.log("\n=== ПРАВИЛО 5: ПЕРЕМЕННЫЕ ЦИКЛА И СОСТОЯНИЯ ===");

// Переменные цикла:
for (let i = 0; i < 10; i++) {
  // Каждая итерация имеет свой экземпляр i
  setTimeout(() => console.log(i), i * 100);
}

// Состояние компонента (псевдокод):
function Counter() {
  let count = 0; // Состояние изменяется

  function increment() {
    count = count + 1;
  }

  return { count, increment };
}

// Правило 6: const для функций и классов
console.log("\n=== ПРАВИЛО 6: ФУНКЦИИ И КЛАССЫ ===");

// Функциональное выражение:
const calculateTotal = (prices) => {
  return prices.reduce((sum, price) => sum + price, 0);
};
```

```
// Класс:
class User {
  constructor(name) {
    this.name = name;
  }
}

// Но function declaration тоже работает:
function oldFunction() { }

// Правило 7: Выбирайте осознанно
console.log("\n=== ПРАВИЛО 7: ОСОЗНАННЫЙ ВЫБОР ===");

function processOrder(order) {
  // const - данные не меняются
  const TAX_RATE = 0.2;
  const SHIPPING_COST = 300;

  // let - меняется в процессе вычислений
  let subtotal = calculateSubtotal(order.items);
  let tax = subtotal * TAX_RATE;
  let total = subtotal + tax + SHIPPING_COST;

  // const - результат вычислений
  const formattedTotal = formatCurrency(total);

  return formattedTotal;
}

function calculateSubtotal(items) {
  // const внутри - каждая итерация имеет свою константу
  let total = 0;
  for (const item of items) {
    const itemTotal = item.price * item.quantity;
    total += itemTotal;
  }
  return total;
}

function formatCurrency(amount) {
  const CURRENCY = "RUB";
  return `${amount} ${CURRENCY}`;
}
```

```
console.log("=".repeat(50));
```

Часть 6: Практические примеры и антипаттерны

6.1. Реальные примеры использования

javascript

```
console.log("=== РЕАЛЬНЫЕ ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ ===");
```

// Пример 1: Конфигурация приложения

```
console.log("=== ПРИМЕР 1: КОНФИГУРАЦИЯ ПРИЛОЖЕНИЯ ===");
```

```
const APP_CONFIG = {
  name: "МойИнтернетМагазин",
  version: "1.0.0",
  api: {
    baseUrl: process.env.API_URL || "https://api.example.com",
    endpoints: {
      products: "/products",
      categories: "/categories",
      users: "/users"
    },
    timeout: 30000
  },
  features: {
    darkMode: true,
    notifications: true,
    analytics: false
  },
  ui: {
    theme: "light",
    language: "ru",
    currency: "RUB"
  }
};
```

// Пример 2: Управление состоянием

```
console.log("\n=== ПРИМЕР 2: УПРАВЛЕНИЕ СОСТОЯНИЕМ ===");
```

```
class ShoppingCart {
  constructor() {
```

```

    // const - ссылка на массив не меняется
    this.items = [];

    // let - общая сумма меняется
    this.total = 0;
    this.discount = 0;
}

addItem(product, quantity = 1) {
    // const - данные продукта не меняются
    const item = {
        id: product.id,
        name: product.name,
        price: product.price,
        quantity
    };

    // Мутация массива (разрешено)
    this.items.push(item);

    // Пересчет итогов
    this.calculateTotal();
}

calculateTotal() {
    // let - временные переменные вычислений
    let subtotal = 0;

    // const в цикле - каждая итерация имеет свою константу
    for (const item of this.items) {
        const itemTotal = item.price * item.quantity;
        subtotal += itemTotal;
    }

    // Изменяем состояние
    this.total = subtotal - this.discount;
}

applyDiscount(code) {
    // const - коды скидок
    const DISCOUNT_CODES = {
        "WELCOME10": 0.1, // 10%
        "SUMMER25": 0.25, // 25%
        "BLACKFRIDAY": 0.5 // 50%
    }

```

```

    };

    const discountRate = DISCOUNT_CODES[code] || 0;
    this.discount = this.total * discountRate;
    this.calculateTotal();
  }
}

```

// Пример 3: Обработка асинхронных операций

```
console.log("\n=== ПРИМЕР 3: АСИНХРОННЫЕ ОПЕРАЦИИ ===");
```

```

async function fetchUserData(userId) {
  // const - конфигурация запроса
  const API_URL = "https://api.example.com/users";
  const TIMEOUT = 5000;

  try {
    // let - состояние загрузки
    let isLoading = true;
    let error = null;
    let data = null;

    // const - параметры запроса
    const response = await fetch(`${API_URL}/${userId}`, {
      timeout: TIMEOUT
    });

    if (!response.ok) {
      // const - сообщения об ошибках
      const ERROR_MESSAGES = {
        404: "Пользователь не найден",
        500: "Ошибка сервера",
        403: "Доступ запрещен"
      };

      const message = ERROR_MESSAGES[response.status] || "Неизвестная ошибка";
      throw new Error(`HTTP ${response.status}: ${message}`);
    }

    // const - результат парсинга
    data = await response.json();
    isLoading = false;

    return { data, isLoading, error };
  }
}

```

```

} catch (err) {
  // const - информация об ошибке
  const errorInfo = {
    message: err.message,
    timestamp: new Date().toISOString(),
    userId
  };

  console.error("Ошибка загрузки:", errorInfo);
  return { data: null, isLoading: false, error: err.message };
}
}

// Пример 4: Валидация форм
console.log("\n=== ПРИМЕР 4: ВАЛИДАЦИЯ ФОРМ ===");

const VALIDATION_RULES = {
  email: {
    pattern: /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/,
    message: "Введите корректный email адрес"
  },
  password: {
    minLength: 8,
    pattern: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)/,
    message: "Пароль должен содержать минимум 8 символов, включая заглавные и строчные буквы и цифры"
  },
  phone: {
    pattern: /^\\+7\\d{10}$/,
    message: "Введите номер в формате +7XXXXXXXXXX"
  }
};

function validateForm(formData) {
  // let - список ошибок
  let errors = {};
  let isValid = true;

  // const в цикле
  for (const [field, value] of Object.entries(formData)) {
    const rule = VALIDATION_RULES[field];

    if (rule) {

```

```

    const { pattern, minLength, message } = rule;

    if (minLength && value.length < minLength) {
        errors[field] = message;
        isValid = false;
    } else if (pattern && !pattern.test(value)) {
        errors[field] = message;
        isValid = false;
    }
}

return { isValid, errors };
}

console.log("=".repeat(50));

```

6.2. Антипаттерны и распространенные ошибки

```

javascript

console.log("=== АНТИПАТТЕРНЫ И ОШИБКИ ===");

// Антипаттерн 1: Использование var в современном коде
console.log("=== АНТИПАТТЕРН 1: VAR В СОВРЕМЕННОМ КОДЕ ===");

// ПЛОХО:
function antiPattern1() {
    var data = fetchData();
    var processed = process(data);
    var result = calculate(processed);
    return result;
}

// ХОРОШО:
function goodPattern1() {
    const data = fetchData();
    const processed = process(data);
    const result = calculate(processed);
    return result;
}

// Антипаттерн 2: Изменение const переменных
console.log("\n=== АНТИПАТТЕРН 2: ИЗМЕНЕНИЕ CONST ===");

```

```

// Ошибка (в strict mode):
const config = { theme: "dark" };
// config = { theme: "light" }; // TypeError

// Но можно изменять свойства:
config.theme = "light"; // Разрешено (может быть неочевидно)

// Лучше:
const immutableConfig = Object.freeze({ theme: "dark" });
// immutableConfig.theme = "light"; // TypeError в strict mode

// Антипаттерн 3: Использование let без необходимости
console.log("\n=== АНТИПАТТЕРН 3: ИЗБЫТОЧНЫЙ LET ===");

// ПЛОХО:
function antiPattern3(numbers) {
  let result = 0; // Будет переназначено

  for (let i = 0; i < numbers.length; i++) {
    result = numbers[i] * 2; // Переприсваивание
  }

  return result;
}

// ХОРОШО (если последний элемент нужен):
function goodPattern3a(numbers) {
  let lastDoubled = 0;

  for (const number of numbers) {
    lastDoubled = number * 2;
  }

  return lastDoubled;
}

// ЕЩЕ ЛУЧШЕ (функциональный стиль):
function goodPattern3b(numbers) {
  return numbers.map(n => n * 2);
}

// Антипаттерн 4: Глобальные переменные
console.log("\n=== АНТИПАТТЕРН 4: ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ===");

```


// ОЧЕНЬ ПЛОХО:

```
var globalCounter = 0; // Загрязняет глобальное пространство имен
```

```
function incrementGlobal() {  
  globalCounter++;  
}
```

// НЕМНОГО ЛУЧШЕ (но все еще плохо):

```
let moduleCounter = 0; // Не глобальная, но все еще изменяемая извне
```

// ХОРОШО:

```
function createCounter() {  
  let privateCounter = 0; // Приватная переменная  
  
  return {  
    increment: () => { privateCounter++; },  
    getValue: () => privateCounter  
  };  
}
```

```
const counter = createCounter();  
counter.increment();  
console.log("Счетчик:", counter.getValue()); // 1
```

// Антипаттерн 5: Непонимание области видимости

```
console.log("\n=== АНТИПАТТЕРН 5: ОБЛАСТЬ ВИДИМОСТИ ===");
```

// ПЛОХО (старый стиль):

```
function antiPattern5() {  
  var x = 10;  
  
  if (true) {  
    var x = 20; // Переопределяет внешнюю x  
    console.log("Внутри блока:", x); // 20  
  }  
  
  console.log("Снаружи блока:", x); // 20 (неожиданно!)  
}
```

// ХОРОШО:

```
function goodPattern5() {  
  const x = 10;
```

```

if (true) {
  const x = 20; // Другая переменная
  console.log("Внутри блока:", x); // 20
}

console.log("Снаружи блока:", x); // 10 (как и ожидалось)
}

// Антипаттерн 6: Слишком длинные области видимости
console.log("\n=== АНТИПАТТЕРН 6: ДЛИННЫЕ ОБЛАСТИ ВИДИМОСТИ ===");

// ПЛОХО:
function antiPattern6() {
  let temp; // Объявлена слишком рано

  // Много кода...

  temp = calculateSomething(); // Используется только здесь

  // Еще код...

  return process(temp);
}

// ХОРОШО:
function goodPattern6() {
  // Код до...

  const temp = calculateSomething(); // Объявлена рядом с использованием

  // Код после...

  return process(temp);
}

// ЕЩЕ ЛУЧШЕ (изоляция в блоке):
function betterPattern6() {
  // Код до...

  let result;
  {
    const temp = calculateSomething();
    const processed = process(temp);
    result = format(processed);
  }
}

```

```

}

// temp и processed автоматически удалены
return result;
}

// Антипаттерн 7: Отсутствие инициализации
console.log("\n=== АНТИПАТТЕРН 7: ОТСУТСТВИЕ ИНИЦИАЛИЗАЦИИ ===");

// ПЛОХО:
let uninitialized;
// ...много кода...
if (condition) {
    uninitialized = value1;
} else {
    uninitialized = value2;
}

// ХОРОШО:
const initialized = condition ? value1 : value2;

// ИЛИ:
let properlyInitialized;
if (condition) {
    properlyInitialized = value1;
} else {
    properlyInitialized = value2;
}

// Антипаттерн 8: Неправильные имена констант
console.log("\n=== АНТИПАТТЕРН 8: ИМЕНА КОНСТАНТ ===");

// ПЛОХО:
const c = 299792458; // Что это?
const str = "Hello"; // Слишком общее
const obj = { x: 1, y: 2 }; // Неинформативно

// ХОРОШО:
const SPEED_OF_LIGHT = 299792458; // Физическая константа
const DEFAULT_GREETING = "Hello"; // Значение по умолчанию
const VECTOR_2D = { x: 1, y: 2 }; // Представляет вектор

console.log("=".repeat(50));

```

Часть 7: Практикум и упражнения

7.1. Интерактивные упражнения

html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Практикум: var, let, const</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      line-height: 1.6;
      color: #333;
      background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
      min-height: 100vh;
      padding: 20px;
    }

    .container {
      max-width: 1200px;
      margin: 0 auto;
      background: white;
      border-radius: 20px;
      box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
      overflow: hidden;
    }

    header {
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
      padding: 30px;
      text-align: center;
    }
```

```
h1 {
  font-size: 2.5em;
  margin-bottom: 10px;
}

.subtitle {
  font-size: 1.2em;
  opacity: 0.9;
}

.content {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 30px;
  padding: 30px;
}

.exercise {
  background: #f8f9fa;
  border-radius: 10px;
  padding: 20px;
  border-left: 5px solid #667eea;
}

.exercise h3 {
  color: #667eea;
  margin-bottom: 15px;
  font-size: 1.4em;
}

.code-block {
  background: #2d2d2d;
  color: #f8f8f2;
  padding: 15px;
  border-radius: 5px;
  font-family: 'Courier New', monospace;
  font-size: 14px;
  margin: 15px 0;
  overflow-x: auto;
  white-space: pre-wrap;
}

.controls {
```

```
display: flex;
gap: 10px;
margin-top: 15px;
}
```

```
button {
padding: 10px 20px;
border: none;
border-radius: 5px;
background: #667eea;
color: white;
font-weight: bold;
cursor: pointer;
transition: background 0.3s;
}
```

```
button:hover {
background: #5a6fd8;
}
```

```
.result {
margin-top: 15px;
padding: 15px;
background: white;
border-radius: 5px;
border: 2px solid #e9ecef;
font-family: 'Courier New', monospace;
min-height: 50px;
white-space: pre-wrap;
}
```

```
.highlight {
background: yellow;
padding: 2px;
}
```

```
.error {
color: #dc3545;
font-weight: bold;
}
```

```
.success {
color: #28a745;
font-weight: bold;
}
```

```
}
```

```
.theory {  
  grid-column: 1 / -1;  
  background: #e3f2fd;  
  padding: 25px;  
  border-radius: 10px;  
  margin-top: 20px;  
}
```

```
.theory h3 {  
  color: #1976d2;  
  margin-bottom: 15px;  
}
```

```
footer {  
  text-align: center;  
  padding: 20px;  
  background: #f8f9fa;  
  color: #666;  
  border-top: 1px solid #dee2e6;  
}
```

```
@media (max-width: 768px) {  
  .content {  
    grid-template-columns: 1fr;  
  }  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<header>
```

```
<h1>Практикум: var, let, const</h1>
```

```
<div class="subtitle">Интерактивные упражнения для закрепления материала</div>
```

```
</header>
```

```
<div class="content">
```

```
<!-- Упражнение 1 -->
```

```
<div class="exercise">
```

```
<h3>Упражнение 1: Область видимости</h3>
```

```
<p>Сравните поведение var и let в разных областях видимости.</p>
```

```
<div class="code-block">
```

```

function exercise1() {
  if (true) {
    var functionScoped = "var: видна везде в функции";
    let blockScoped = "let: видна только в блоке";
  }

  try {
    return `var: ${functionScoped}, let: ${blockScoped}`;
  } catch (error) {
    return `var: ${functionScoped}, let: ${error.message}`;
  }
}

</div>

<div class="controls">
  <button onclick="runExercise1()">Выполнить</button>
  <button onclick="explainExercise1()">Объяснить</button>
</div>

<div class="result" id="result1"></div>
</div>

<!-- Упражнение 2 -->
<div class="exercise">
  <h3>Упражнение 2: Поднятие (hoisting)</h3>
  <p>Посмотрите, как ведут себя переменные до их объявления.</p>

  <div class="code-block">
function exercise2() {
  try {
    console.log("До объявления var:", hoistedVar);
    console.log("До объявления let:", hoistedLet);
  } catch (error) {
    return error.message;
  }

  var hoistedVar = "поднята с undefined";
  let hoistedLet = "в TDZ до этой строки";

  return "Проверьте консоль браузера (F12)";
}

</div>

<div class="controls">

```



```
<button onclick="runExercise2()">Выполнить</button>
<button onclick="explainExercise2()">Объяснить</button>
</div>
```

```
<div class="result" id="result2"></div>
</div>
```

```
<!-- Упражнение 3 -->
```

```
<div class="exercise">
  <h3>Упражнение 3: Циклы</h3>
  <p>Сравните поведение var и let в циклах с setTimeout.</p>
```

```
<div class="code-block">
```

```
function exercise3() {
  let result = [];

  // C var
  result.push("C var:");
  for (var i = 0; i < 3; i++) {
    setTimeout(() => {
      console.log(`var: i = ${i}`);
    }, 100);
  }

  // C let
  result.push("\nC let:");
  for (let j = 0; j < 3; j++) {
    setTimeout(() => {
      console.log(`let: j = ${j}`);
    }, 100);
  }

  return result.join("") + "\n\nПроверьте консоль браузера (F12)";
}
```

```
</div>
```

```
<div class="controls">
  <button onclick="runExercise3()">Выполнить</button>
  <button onclick="explainExercise3()">Объяснить</button>
</div>
```

```
<div class="result" id="result3"></div>
</div>
```

<!-- Упражнение 4 -->

<div class="exercise">

<h3>Упражнение 4: const с объектами</h3>

<p>Проверьте, что можно и нельзя делать с const объектами.</p>

<div class="code-block">

```
function exercise4() {
  const person = {
    name: "Анна",
    age: 25
  };

  let actions = [];

  // Попытка изменить свойство
  person.age = 26;
  actions.push("1. Изменение свойства: УСПЕХ");

  // Попытка добавить свойство
  person.city = "Москва";
  actions.push("2. Добавление свойства: УСПЕХ");

  // Попытка переприсвоить
  try {
    person = { name: "Борис" };
    actions.push("3. Переприсвоение: УСПЕХ");
  } catch (error) {
    actions.push(`3. Переприсвоение: ОШИБКА (${error.message})`);
  }

  return actions.join("\n");
}
```

</div>

<div class="controls">

<button onclick="runExercise4()">Выполнить</button>

<button onclick="explainExercise4()">Объяснить</button>

</div>

<div class="result" id="result4"></div>

</div>

<!-- Теория -->

<div class="theory">

```

<h3>📄 Краткая теория</h3>
<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(300px,
1fr)); gap: 20px;">
  <div>
    <h4>var (устаревшее)</h4>
    <ul>
      <li>Область видимости: функция</li>
      <li>Поднятие: да, с undefined</li>
      <li>Переобъявление: разрешено</li>
      <li>Использовать: НЕТ (устарело)</li>
    </ul>
  </div>
  <div>
    <h4>let (современное)</h4>
    <ul>
      <li>Область видимости: блок</li>
      <li>Поднятие: да, но с TDZ</li>
      <li>Переобъявление: запрещено</li>
      <li>Использовать: для изменяемых переменных</li>
    </ul>
  </div>
  <div>
    <h4>const (рекомендуется)</h4>
    <ul>
      <li>Область видимости: блок</li>
      <li>Поднятие: да, но с TDZ</li>
      <li>Переобъявление: запрещено</li>
      <li>Использовать: по умолчанию, для всего</li>
    </ul>
  </div>
</div>
<p style="margin-top: 15px; font-style: italic;">
  <strong>Правило:</strong> Всегда начинайте с const. Если переменная должна
изменяться, используйте let. Никогда не используйте var в новом коде.
</p>
</div>

<footer>
  <p>Практикум по JavaScript: Объявление переменных</p>
  <p>Используйте консоль браузера (F12) для просмотра дополнительных логов</p>
</footer>
</div>

```

```

<script>
  // Упражнение 1
  function runExercise1() {
    const result = exercise1();
    document.getElementById('result1').innerHTML =
      `<span class="success">Результат:</span>\n${result}`;
  }

  function explainExercise1() {
    const explanation = `
      <span class="highlight">Ключевые моменты:</span>
      1. <strong>var</strong> имеет область видимости на уровне функции
        - Объявленная внутри блока if, видна во всей функции

      2. <strong>let</strong> имеет область видимости на уровне блока
        - Объявленная внутри блока if, видна ТОЛЬКО внутри этого блока
        - Попытка доступа снаружи вызывает ReferenceError

      3. <strong>const</strong> ведет себя так же, как let, в отношении области
видимости
    `;
    document.getElementById('result1').innerHTML = explanation;
  }

  // Упражнение 2
  function runExercise2() {
    const result = exercise2();
    document.getElementById('result2').innerHTML =
      `<span class="success">Результат:</span>\n${result}`;
  }

  function explainExercise2() {
    const explanation = `
      <span class="highlight">Ключевые моменты:</span>
      1. <strong>var</strong> поднимается (hoisted) и инициализируется как undefined
        - Можно обращаться до объявления (получим undefined)

      2. <strong>let</strong> и <strong>const</strong> тоже поднимаются, но попадают во
"временную мертвую зону" (TDZ)
        - Нельзя обращаться до объявления (будет ReferenceError)

      3. TDZ существует от начала блока до момента объявления переменной
    `;
    document.getElementById('result2').innerHTML = explanation;
  }

```

```
}
```

```
// Упражнение 3
```

```
function runExercise3() {
  const result = exercise3();
  document.getElementById('result3').innerHTML =
    `<span class="success">Результат:</span>\n${result}`;
}

function explainExercise3() {
  const explanation = `
    <span class="highlight">Ключевые моменты:</span>
    1. С <strong>var</strong>:
      - Одна переменная i на весь цикл
      - Когда срабатывают setTimeout, i уже равна 3
      - Все выводы: "var: i = 3"

    2. С <strong>let</strong>:
      - Каждая итерация цикла создает НОВУЮ переменную j
      - Замыкание сохраняет свою копию j
      - Выводы: "let: j = 0", "let: j = 1", "let: j = 2"

    3. Это основная причина использовать let в циклах
  `;
  document.getElementById('result3').innerHTML = explanation;
}
```

```
// Упражнение 4
```

```
function runExercise4() {
  const result = exercise4();
  document.getElementById('result4').innerHTML =
    `<span class="success">Результат:</span>\n${result}`;
}

function explainExercise4() {
  const explanation = `
    <span class="highlight">Ключевые моменты:</span>
    1. <strong>const</strong> защищает только ПЕРЕМЕННУЮ (ссылку), а не ЗНАЧЕНИЕ

    2. Для примитивов (числа, строки, булевы):
      - Значение нельзя изменить
      - Пример: const PI = 3.14; PI = 3.14159; // Ошибка

    3. Для объектов и массивов:
  `;
}
```

- Ссылку нельзя изменить (переприсвоить)
- Но содержимое объекта/массива можно менять!
- Пример: `const arr = [1, 2]; arr.push(3); // OK`

```

4. Для полной защиты объектов используйте Object.freeze()
`;
document.getElementById('result4').innerHTML = explanation;
}

```

// Реализация функций упражнений

```

function exercise1() {
  if (true) {
    var functionScoped = "var: видна везде в функции";
    let blockScoped = "let: видна только в блоке";
  }

  try {
    return `var: ${functionScoped}, let: ${blockScoped}`;
  } catch (error) {
    return `var: ${functionScoped}, let: ${error.message}`;
  }
}

```

```

function exercise2() {
  try {
    console.log("До объявления var:", hoistedVar);
    console.log("До объявления let:", hoistedLet);
  } catch (error) {
    return error.message;
  }
}

```

```

var hoistedVar = "поднята с undefined";
let hoistedLet = "в TDZ до этой строки";

```

```

return "Проверьте консоль браузера (F12)";
}

```

```

function exercise3() {
  let result = [];

  // C var
  result.push("C var:");
  for (var i = 0; i < 3; i++) {
    setTimeout(() => {

```

```

        console.log(`var: i = ${i}`);
    }, 100);
}

// C Let
result.push("\nC let:");
for (let j = 0; j < 3; j++) {
    setTimeout(() => {
        console.log(`let: j = ${j}`);
    }, 100);
}

return result.join("") + "\n\nПроверьте консоль браузера (F12)";
}

function exercise4() {
    const person = {
        name: "Анна",
        age: 25
    };

    let actions = [];

    // Попытка изменить свойство
    person.age = 26;
    actions.push("1. Изменение свойства: УСПЕХ");

    // Попытка добавить свойство
    person.city = "Москва";
    actions.push("2. Добавление свойства: УСПЕХ");

    // Попытка переписать
    try {
        person = { name: "Борис" };
        actions.push("3. Переприсвоение: УСПЕХ");
    } catch (error) {
        actions.push(`3. Переприсвоение: ОШИБКА (${error.message})`);
    }

    return actions.join("\n");
}

// Инициализация
document.addEventListener('DOMContentLoaded', () => {

```

```
    console.log("=== ПРАКТИКУМ ПО VAR, LET, CONST ===");
    console.log("Выполняйте упражнения и следите за консолью");
    console.log("=".repeat(50));
  });
</script>
</body>
</html>
```

7.2. Тестовые задания

javascript

```
console.log("=== ТЕСТОВЫЕ ЗАДАНИЯ ===");
```

// Задание 1: Найдите ошибки в коде

```
console.log("=== ЗАДАНИЕ 1: НАЙДИТЕ ОШИБКИ ===");
```

```
function task1() {
```

// Какие ошибки содержатся в этом коде?

```
  const answer = [];
```

// Пример 1

```
  try {
```

```
    console.log(notDeclared);
```

```
    answer.push("1. Ошибка: обращение к необъявленной переменной");
```

```
  } catch (e) {
```

```
    answer.push("1. Правильно: ReferenceError");
```

```
  }
```

// Пример 2

```
  try {
```

```
    var x = 10;
```

```
    var x = 20; // Разрешено для var
```

```
    answer.push("2. Нет ошибки: var можно переобъявлять");
```

```
  } catch (e) {
```

```
    answer.push(`2. Ошибка: ${e.message}`);
```

```
  }
```

// Пример 3

```
  try {
```

```
    let y = 10;
```

```
    let y = 20; // Ошибка для Let
```

```
    answer.push("3. Нет ошибки");
```

```
  } catch (e) {
```



```
    answer.push("3. Ошибка: let нельзя переобъявлять");  
}
```

// Пример 4

```
try {  
    const z; // Ошибка: нет инициализации  
    answer.push("4. Нет ошибки");  
} catch (e) {  
    answer.push("4. Ошибка: const требует инициализации");  
}
```

// Пример 5

```
try {  
    const obj = { value: 1 };  
    obj.value = 2; // Разрешено  
    answer.push("5. Нет ошибки: можно изменять свойства const объекта");  
} catch (e) {  
    answer.push(`5. Ошибка: ${e.message}`);  
}
```

```
    return answer.join("\n");  
}
```

```
console.log(task1());
```

// Задание 2: Предскажите результат

```
console.log("\n=== ЗАДАНИЕ 2: ПРЕДСКАЖИТЕ РЕЗУЛЬТАТ ===");
```

```
function task2() {  
    const results = [];
```

// Пример 1

```
{  
    var a = 1;  
    let b = 2;  
    const c = 3;  
}  
  
try {  
    results.push(`1. a = ${a}`); // 1 (var видна)  
} catch (e) {  
    results.push(`1. a: ${e.message}`);  
}
```

```

try {
  results.push(`1. b = ${b}`); // Ошибка
} catch (e) {
  results.push(`1. b: ${e.message}`);
}

try {
  results.push(`1. c = ${c}`); // Ошибка
} catch (e) {
  results.push(`1. c: ${e.message}`);
}

// Пример 2
let x = 10;
if (true) {
  let x = 20;
  results.push(`2. Внутри блока: x = ${x}`); // 20
}
results.push(`2. Снаружи блока: x = ${x}`); // 10

// Пример 3
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(`3. var i = ${i}`); // 3, 3, 3
  }, 10);
}

for (let j = 0; j < 3; j++) {
  setTimeout(() => {
    console.log(`3. let j = ${j}`); // 0, 1, 2
  }, 10);
}
results.push("3. Смотрите консоль");

return results.join("\n");
}

```

```

console.log(task2());

```

```

// Задание 3: Рефакторинг кода

```

```

console.log("\n=== ЗАДАНИЕ 3: РЕФАКТОРИНГ ===");

function task3() {
  console.log("Исходный код (устаревший стиль):");
}

```

```
console.log(`
function processOrder(order) {
  var items = order.items;
  var total = 0;
  var discount = 0;

  for (var i = 0; i < items.length; i++) {
    var item = items[i];
    var itemTotal = item.price * item.quantity;
    total += itemTotal;
  }

  if (order.customer.isPremium) {
    var discount = total * 0.1; // Ошибка: переопределение var
  }

  var finalTotal = total - discount;
  return finalTotal;
}
`);

console.log("\nРефакторинг (современный стиль):");
console.log(`
function processOrderModern(order) {
  const items = order.items;
  let total = 0;
  let discount = 0;

  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const itemTotal = item.price * item.quantity;
    total += itemTotal;
  }

  if (order.customer.isPremium) {
    discount = total * 0.1; // Изменяем существующую переменную
  }

  const finalTotal = total - discount;
  return finalTotal;
}
`);

console.log("\nУлучшенный рефакторинг:");
```

```

    console.log(`
function processOrderBest(order) {
    const total = order.items.reduce((sum, item) => {
        return sum + (item.price * item.quantity);
    }, 0);

    const discount = order.customer.isPremium ? total * 0.1 : 0;
    const finalTotal = total - discount;

    return finalTotal;
}
`);

    return "Сравните три версии и найдите различия";
}

console.log(task3());

console.log("=".repeat(50));

```

Заключение

Ключевые выводы

- 1. Всегда используйте const по умолчанию.** Это заставляет задумываться о том, должна ли переменная изменяться, и делает код более предсказуемым.
- 2. Используйте let для переменных, которые действительно изменяются.** Это явно показывает намерение изменить значение.
- 3. Никогда не используйте var в новом коде.** Устаревший синтаксис, полный подводных камней и неожиданного поведения.
- 4. Помните о различиях в области видимости:**
 - ➔ `var` — function scope (область видимости на уровне функции)
 - ➔ `let/const` — block scope (область видимости на уровне блока)
- 5. Понимайте временную мертвую зону (TDZ):**
 - ➔ `let` и `const` существуют в TDZ до объявления
 - ➔ Обращение к ним до объявления вызывает ReferenceError
 - ➔ `var` инициализируется как undefined при поднятии
- 6. const защищает ссылку, а не значение:**
 - ➔ Для примитивов: значение неизменно
 - ➔ Для объектов: можно изменять свойства, но не переприсваивать

➔ Для полной защиты используйте `Object.freeze()`

Практические рекомендации

javascript

// Рекомендуемый стиль кодирования:

// 1. Константы для конфигурации

```
const CONFIG = {  
  API_URL: "https://api.example.com",  
  TIMEOUT: 30000,  
  RETRY_ATTEMPTS: 3  
};
```

// 2. const для импортов и результатов вычислений

```
const userData = fetchUserData();  
const processedData = process(userData);
```

// 3. Let для счетчиков и состояния

```
let attempts = 0;  
let isLoading = true;  
let error = null;
```

// 4. const в циклах (если значение не меняется в итерации)

```
for (const item of items) {  
  const itemTotal = calculateTotal(item);  
  // ...  
}
```

// 5. Let в циклах (если значение меняется)

```
for (let i = 0; i < items.length; i++) {  
  // i меняется каждой итерацией  
}
```

// 6. Избегайте var полностью

// Никогда не пишите:

// var something = "value";

// 7. Используйте осмысленные имена

```
const MAX_RETRY_ATTEMPTS = 3; // Хорошо  
const mra = 3; // Плохо
```

Дальнейшие шаги

Теперь, когда вы освоили объявление переменных, вы готовы к изучению:

1. **Типы данных в JavaScript** — что можно хранить в переменных
2. **Операторы присваивания** — как изменять значения переменных
3. **Области видимости и замыкания** — как переменные взаимодействуют с функциями
4. **Деструктуризация** — современный способ извлечения значений
5. **Шаблонные строки** — удобная работа со строками и переменными

Помните: хорошее понимание основ — залог успеха в программировании. Практикуйтесь, экспериментируйте с примерами из этой главы, и вскоре использование `const`, `let` и понимание, почему `var` устарел, станет для вас второй натурой.

Финальное упражнение: Самопроверка

javascript

// Проверьте свои знания, ответив на вопросы:

```
const questions = [
  {
    question: "Какой из этих вариантов вызовет ошибку?",
    options: [
      "a) let x = 1; x = 2;",
      "b) const y = 1; y = 2;",
      "c) var z = 1; z = 2;"
    ],
    correct: 1 // b
  },
  {
    question: "Где будет видна переменная, объявленная с var внутри блока if?",
    options: [
      "a) Только внутри блока if",
      "b) Внутри всей функции",
      "c) Глобально"
    ],
    correct: 1 // b
  },
  {
    question: "Что выведет console.log(x) до объявления let x = 5?",
```

```

    options: [
      "a) undefined",
      "b) 5",
      "c) ReferenceError",
      "d) null"
    ],
    correct: 2 // c
  },
  {
    question: "Можно ли изменить свойство объекта, объявленного с const?",
    options: [
      "a) Нет, const делает объект полностью неизменяемым",
      "b) Да, но только если использовать Object.freeze()",
      "c) Да, const защищает только ссылку, но не содержимое объекта"
    ],
    correct: 2 // c
  }
];

console.log("=== САМОПРОВЕРКА ===");
console.log("Ответьте на вопросы и проверьте себя:");
questions.forEach((q, i) => {
  console.log(`\n${i + 1}. ${q.question}`);
  q.options.forEach(opt => console.log(`    ${opt}`));
});

```

Удачи в изучении JavaScript! Помните, что практика — ключ к мастерству. 

4.2. Области видимости: глобальная, функциональная, блочная

Введение: Что такое область видимости?

Область видимости (Scope) — это концепция, определяющая доступность переменных и функций в различных частях кода. Это набор правил, которые определяют, где и как можно использовать идентификаторы (имена переменных, функций, классов) в программе.

Аналогия из реального мира:

text

Глобальная область видимости	= Улица (все видят)
Функциональная область	= Дом (видят только жильцы)
Блочная область	= Комната (видят только в комнате)

Часть 1: Основные концепции и терминология

1.1. Что такое область видимости?

javascript

```
console.log("=== ОСНОВНЫЕ КОНЦЕПЦИИ ===");
```

// Область видимости определяет "видимость" переменных

```
let глобальнаяПеременная = "Я видна везде";
```

```
function примерФункции() {  
  let локальнаяПеременная = "Я видна только в этой функции";  
  console.log(глобальнаяПеременная); // Работает  
  console.log(локальнаяПеременная); // Работает  
}
```

```
примерФункции();  
console.log(глобальнаяПеременная); // Работает  
// console.log(локальнаяПеременная); // Ошибка! Не видна
```



```
console.log("\n=== ЛЕКСИЧЕСКАЯ ОБЛАСТЬ ВИДИМОСТИ ===");
// JavaScript использует лексическую (статическую) область видимости
// Это означает, что область видимости определяется местом объявления в коде

function внешняя() {
    let внешняяПеременная = "внешняя";

    function внутренняя() {
        // Внутренняя функция видит переменные внешней функции
        console.log("Внутренняя видит:", внешняяПеременная); // "внешняя"
    }

    внутренняя();
}

внешняя();

console.log("\n=== ЦЕПОЧКА ОБЛАСТЕЙ ВИДИМОСТИ ===");
// JavaScript ищет переменные по цепочке областей видимости

let глобальная = "глобальное значение";

function уровень1() {
    let уровень1Переменная = "уровень 1";

    function уровень2() {
        let уровень2Переменная = "уровень 2";

        function уровень3() {
            let уровень3Переменная = "уровень 3";

            // Поиск переменной:
            // 1. Ищем в уровне3
            // 2. Ищем в уровне2
            // 3. Ищем в уровне1
            // 4. Ищем в глобальной области
            console.log(уровень3Переменная); // "уровень 3" - найдено на шаге 1
            console.log(уровень2Переменная); // "уровень 2" - найдено на шаге 2
            console.log(уровень1Переменная); // "уровень 1" - найдено на шаге 3
            console.log(глобальная);          // "глобальное значение" - найдено на шаге 4
            // console.log(несуществующая); // ReferenceError - не найдено нигде
        }

        уровень3();
    }
}
```

```
}

уровень2();
}

уровень1();

console.log("=".repeat(80));
```

1.2. Как работает поиск переменных

```
javascript

console.log("=== КАК РАБОТАЕТ ПОИСК ПЕРЕМЕННЫХ ===");

// Демонстрация цепочки областей видимости (Scope Chain)
const globalVar = "Я глобальная";

function outerFunction() {
  const outerVar = "Я внешняя";

  function middleFunction() {
    const middleVar = "Я средняя";

    function innerFunction() {
      const innerVar = "Я внутренняя";

      // Демонстрация поиска по цепочке
      console.group("Поиск переменных из innerFunction:");

      // 1. Поиск innerVar
      console.log("innerVar:", innerVar);
      // Найдено сразу в текущей области

      // 2. Поиск middleVar
      console.log("middleVar:", middleVar);
      // Не найдено в innerFunction → проверяем middleFunction → найдено!

      // 3. Поиск outerVar
      console.log("outerVar:", outerVar);
      // innerFunction → middleFunction → outerFunction → найдено!

      // 4. Поиск globalVar
      console.log("globalVar:", globalVar);
```

```

// innerFunction → middleFunction → outerFunction → глобальная → найдено!

// 5. Поиск несуществующей переменной
try {
  console.log("nonExistent:", nonExistent);
} catch (e) {
  console.log("nonExistent:", e.message);
  // innerFunction → middleFunction → outerFunction → глобальная → не найдено →
  ошибка
}

console.groupEnd();
}

innerFunction();
}

middleFunction();
}

outerFunction();

console.log("\n=== ПРАКТИЧЕСКИЙ ПРИМЕР: ЗАМЫКАНИЯ ===");
// Понимание цепочки областей видимости важно для замыканий

function createCounter() {
  let count = 0; // Локальная переменная внешней функции

  return function() {
    count++; // Внутренняя функция "замыкает" переменную count
    return count;
  };
}

const counter1 = createCounter();
const counter2 = createCounter();

console.log("counter1:", counter1()); // 1
console.log("counter1:", counter1()); // 2
console.log("counter2:", counter2()); // 1 (другое замыкание!)
console.log("counter2:", counter2()); // 2

// Каждое замыкание имеет свою собственную цепочку областей видимости

```

```
console.log("=".repeat(80));
```

Часть 2: Глобальная область видимости

2.1. Что такое глобальная область видимости?

javascript

```
console.log("=== ГЛОБАЛЬНАЯ ОБЛАСТЬ ВИДИМОСТИ ===");
```

```
// Глобальная область видимости – это самый внешний уровень
```

```
// Переменные, объявленные здесь, доступны ВЕЗДЕ
```

```
// Объявление в глобальной области
```

```
let глобальнаяПеременная1 = "Я глобальная переменная";
```

```
var глобальнаяПеременная2 = "Я тоже глобальная (но с var)";
```

```
const ГЛОБАЛЬНАЯ_КОНСТАНТА = "Я глобальная константа";
```

```
// Функции, объявленные в глобальной области
```

```
function глобальнаяФункция() {  
    console.log("Я глобальная функция");  
}
```

```
// Они доступны везде
```

```
console.log("1. Глобальная переменная:", глобальнаяПеременная1);
```

```
глобальнаяФункция();
```

```
function тестДоступа() {  
    console.log("2. Внутри функции:");  
    console.log("    глобальнаяПеременная1:", глобальнаяПеременная1);  
    console.log("    ГЛОБАЛЬНАЯ_КОНСТАНТА:", ГЛОБАЛЬНАЯ_КОНСТАНТА);  
    глобальнаяФункция();  
}
```

```
тестДоступа();
```

```
// Вложенные функции тоже видят глобальные переменные
```

```
function внешняяФункция() {  
    function внутренняяФункция() {  
        console.log("3. Во вложенной функции:");  
        console.log("    глобальнаяПеременная2:", глобальнаяПеременная2);  
    }  
}
```

```

    внутренняяФункция();
}

внешняяФункция();

console.log("\n=== ГЛОБАЛЬНЫЙ ОБЪЕКТ ===");
// В браузерах глобальный объект – window
// В Node.js глобальный объект – global

// При объявлении с var переменная становится свойством window
var старыйСтиль = "Я в window";
console.log("window.старыйСтиль:", typeof window !== 'undefined' ? window.старыйСтиль :
"Не в браузере");

// let и const не добавляются в глобальный объект
let современныйСтиль = "Я не в window";
console.log("window.современныйСтиль:", typeof window !== 'undefined' ?
window.современныйСтиль : "Не определено");

console.log("\n=== ОПАСНОСТИ ГЛОБАЛЬНЫХ ПЕРЕМЕННЫХ ===");

// Проблема 1: Загрязнение глобального пространства имен
var общееИмя = "Первое значение";

// В другом файле (представьте, что это другой скрипт)
var общееИмя = "Второе значение"; // Переопределяет!

console.log("общееИмя после переопределения:", общееИмя); // "Второе значение"

// Проблема 2: Случайное создание глобальных переменных
function опаснаяФункция() {
    // Опечатка или забыли var/let/const
    случайнаяГлобальная = "Я стала глобальной!";
}

опаснаяФункция();
console.log("случайнаяГлобальная:", случайнаяГлобальная); // Существует!

// Проблема 3: Конфликты с встроенными объектами
var Array = "переопределяю Array"; // ОЧЕНЬ ОПАСНО!
console.log("Теперь Array:", Array); // Работает, но сломаны все массивы!

// Восстановим (для продолжения примеров)
delete window.Array;

```

```
console.log("=".repeat(80));
```

2.2. Модульные паттерны для избежания глобальных переменных

javascript

```
console.log("=== МОДУЛЬНЫЕ ПАТТЕРНЫ ===");
```

// Паттерн 1: Использование IIFE (Immediately Invoked Function Expression)

```
console.log("=== ПАТТЕРН 1: IIFE ===");
```

```
(function() {  
    // Все переменные здесь локальные для этой функции  
    let приватнаяПеременная = "Я приватная";  
    const приватнаяКонстанта = "Я тоже приватна";  
  
    // Делаем публичными только нужное  
    window.публичныйМетод = function() {  
        console.log("Публичный метод имеет доступ к приватным:");  
        console.log("  приватнаяПеременная:", приватнаяПеременная);  
        console.log("  приватнаяКонстанта:", приватнаяКонстанта);  
    };  
})();  
  
// приватнаяПеременная // Ошибка! Не доступна  
// приватнаяКонстанта // Ошибка! Не доступна  
window.публичныйМетод(); // Работает!
```

// Паттерн 2: Объект как неймспейс

```
console.log("\n=== ПАТТЕРН 2: НЕЙМСПЕЙС-ОБЪЕКТ ===");
```

```
const МоеПриложение = {  
    config: {  
        apiUrl: "https://api.example.com",  
        version: "1.0.0"  
    },  
  
    utils: {  
        formatDate: function(date) {  
            return date.toLocaleDateString();  
        },  
    },  
};
```

```

    generateId: function() {
        return Math.random().toString(36).substr(2, 9);
    }
},

init: function() {
    console.log("Приложение инициализировано");
    console.log("Версия:", this.config.version);
}
};

// Использование
МоеПриложение.init();
const id = МоеПриложение.utils.generateId();
console.log("Сгенерированный ID:", id);

// Паттерн 3: Модуль с замыканиями
console.log("\n=== ПАТТЕРН 3: МОДУЛЬ С ЗАМЫКАНИЯМИ ===");

const модульСчетчика = (function() {
    // Приватные переменные
    let счетчик = 0;
    const максимальноеЗначение = 100;

    // Приватные методы
    function проверитьЛимит() {
        return счетчик < максимальноеЗначение;
    }

    // Публичный интерфейс
    return {
        увеличить: function() {
            if (проверитьЛимит()) {
                счетчик++;
                console.log("Счетчик увеличен:", счетчик);
            } else {
                console.log("Достигнут лимит!");
            }
        },

        уменьшить: function() {
            if (счетчик > 0) {
                счетчик--;
                console.log("Счетчик уменьшен:", счетчик);
            }
        }
    };
})();

```

```

    }
  },

  получитьЗначение: function() {
    return счетчик;
  },

  сбросить: function() {
    счетчик = 0;
    console.log("Счетчик сброшен");
  }
};
})();

// Использование модуля
модульСчетчика.увеличить(); // 1
модульСчетчика.увеличить(); // 2
модульСчетчика.увеличить(); // 3
console.log("Текущее значение:", модульСчетчика.получитьЗначение()); // 3
модульСчетчика.сбросить();
console.log("После сброса:", модульСчетчика.получитьЗначение()); // 0

// Попытка доступа к приватным переменным
// модульСчетчика.счетчик // undefined
// модульСчетчика.проверитьЛимит() // Ошибка

// Паттерн 4: ES6 Модули (в реальных проектах)
console.log("\n=== ПАТТЕРН 4: ES6 МОДУЛИ ===");

// Пример структуры модулей:
// math.js
// export const PI = 3.14159;
// export function sum(a, b) { return a + b; }

// app.js
// import { PI, sum } from './math.js';

console.log("=".repeat(80));

```


Часть 3: Функциональная область видимости

3.1. Что такое функциональная область видимости?

```
javascript
```

```
console.log("=== ФУНКЦИОНАЛЬНАЯ ОБЛАСТЬ ВИДИМОСТИ ===");
```

```
// Функциональная область видимости означает, что переменные,  
// объявленные внутри функции, видны только внутри этой функции
```

```
function демонстрацияФункциональнойОбласти() {  
    // Переменные, объявленные здесь, видны только в этой функции  
    let локальнаяПеременная = "Я локальная";  
    var стараяЛокальная = "Я тоже локальная (var)";  
    const локальнаяКонстанта = "И я локальная";  
  
    console.log("Внутри функции:");  
    console.log("  локальнаяПеременная:", локальнаяПеременная);  
    console.log("  стараяЛокальная:", стараяЛокальная);  
    console.log("  локальнаяКонстанта:", локальнаяКонстанта);  
  
    // Вложенные функции видят переменные внешней функции  
    function вложеннаяФункция() {  
        console.log("Во вложенной функции:");  
        console.log("  локальнаяПеременная (из внешней):", локальнаяПеременная);  
  
        // Может объявлять свои переменные  
        let свояПеременная = "Я своя";  
        console.log("  свояПеременная:", свояПеременная);  
    }  
  
    вложеннаяФункция();  
  
    // свояПеременная не видна здесь  
    // console.Log(свояПеременная); // Ошибка!  
}
```

```
демонстрацияФункциональнойОбласти();
```

```
// Переменные функции не видны снаружи
```

```
try {  
    console.log("локальнаяПеременная снаружи:", локальнаяПеременная);  
}
```

```

} catch (e) {
    console.log("Ошибка при доступе к локальной переменной:", e.message);
}

console.log("\n=== ОСОБЕННОСТИ VAR В ФУНКЦИОНАЛЬНОЙ ОБЛАСТИ ===");

// var имеет функциональную область видимости (не блочную!)
function демонстрацияVar() {
    console.log("1. В начале функции varVariable:", varVariable); // undefined (поднятие)

    if (true) {
        var varVariable = "Объявлена в блоке if";
        let letVariable = "Объявлена в блоке if (let)";
    }

    console.log("2. После блока if:");
    console.log("    varVariable:", varVariable); // "Объявлена в блоке if" - ВСЕ ЕЩЕ ВИДНА!
    // console.log("    letVariable:", letVariable); // Ошибка!
}

демонстрацияVar();

console.log("\n=== ПАРАМЕТРЫ ФУНКЦИИ ===");

// Параметры функции тоже имеют функциональную область видимости
function функцияСПараметрами(param1, param2) {
    console.log("Параметры внутри функции:");
    console.log("    param1:", param1);
    console.log("    param2:", param2);

    // Можно объявлять переменные с тем же именем
    let param1 = "локальная переменная с тем же именем"; // Ошибка! Уже объявлено
    console.log("    локальная param1:", param1);
}

// функцияСПараметрами("первый", "второй"); // Ошибка из-за переопределения

// Правильный вариант:
function правильнаяФункция(param1, param2) {
    // Параметры ведут себя как локальные переменные
    console.log("Исходные параметры:", param1, param2);

    // Меняем значения
    param1 = "измененный " + param1;

```

```
param2 = param2.toUpperCase();

console.log("Измененные параметры:", param1, param2);
}

правильнаяФункция("значение1", "значение2");

console.log("=".repeat(80));
```

3.2. Замыкания (Closures)

```
javascript

console.log("=== ЗАМЫКАНИЯ (CLOSURES) ===");

// Замыкание — это функция, которая запоминает свое лексическое окружение
// даже после того, как внешняя функция завершила выполнение

// Пример 1: Простое замыкание
console.log("=== ПРИМЕР 1: БАЗОВОЕ ЗАМЫКАНИЕ ===");

function создатьСчетчик() {
    let счетчик = 0; // Локальная переменная внешней функции

    // Внутренняя функция образует замыкание
    return function() {
        счетчик++; // "Запоминает" переменную счетчик
        return счетчик;
    };
}

const мойСчетчик = создатьСчетчик();
console.log("1. мойСчетчик():", мойСчетчик()); // 1
console.log("2. мойСчетчик():", мойСчетчик()); // 2
console.log("3. мойСчетчик():", мойСчетчик()); // 3

// Каждый вызов createCounter создает новое замыкание
const другойСчетчик = создатьСчетчик();
console.log("4. другойСчетчик():", другойСчетчик()); // 1 (новая переменная счетчик)
console.log("5. мойСчетчик():", мойСчетчик()); // 4 (продолжает свою переменную)

// Пример 2: Замыкание с параметрами
console.log("\n=== ПРИМЕР 2: ЗАМЫКАНИЕ С ПАРАМЕТРАМИ ===");
```

```

function создатьУмножитель(множитель) {
    // множитель становится частью замыкания
    return function(число) {
        return число * множитель;
    };
}

const умножитьНа2 = создатьУмножитель(2);
const умножитьНа5 = создатьУмножитель(5);
const умножитьНа10 = создатьУмножитель(10);

console.log("умножитьНа2(10):", умножитьНа2(10)); // 20
console.log("умножитьНа5(10):", умножитьНа5(10)); // 50
console.log("умножитьНа10(10):", умножитьНа10(10)); // 100

// Пример 3: Практическое использование – приватные переменные
console.log("\n=== ПРИМЕР 3: ПРИВАТНЫЕ ПЕРЕМЕННЫЕ ===");

function создатьБанковскийСчет(начальныйБаланс) {
    let баланс = начальныйБаланс; // Приватная переменная

    return {
        // Публичные методы имеют доступ к приватной переменной
        пополнить: function(сумма) {
            if (сумма > 0) {
                баланс += сумма;
                console.log(`Пополнено: ${сумма}. Новый баланс: ${баланс}`);
            }
            return баланс;
        },

        снять: function(сумма) {
            if (сумма > 0 && сумма <= баланс) {
                баланс -= сумма;
                console.log(`Снято: ${сумма}. Новый баланс: ${баланс}`);
            } else {
                console.log("Недостаточно средств или неверная сумма");
            }
            return баланс;
        },

        получитьБаланс: function() {
            return баланс;
        }
    }
}

```

```

    };
}

const мойСчет = создатьБанковскийСчет(1000);
console.log("Начальный баланс:", мойСчет.получитьБаланс()); // 1000
мойСчет.пополнить(500); // Пополнено: 500. Новый баланс: 1500
мойСчет.снять(200);      // Снято: 200. Новый баланс: 1300
console.log("Итоговый баланс:", мойСчет.получитьБаланс()); // 1300

// Прямой доступ к балансу невозможен
console.log("Прямой доступ к балансу:", мойСчет.баланс); // undefined

// Пример 4: Замыкания в циклах (классическая проблема)
console.log("\n=== ПРИМЕР 4: ЗАМЫКАНИЯ В ЦИКЛАХ ===");

console.log("Проблема с var:");
for (var i = 0; i < 3; i++) {
    setTimeout(function() {
        console.log(`var: i = ${i}`); // Всегда 3!
    }, 100);
}

console.log("\nРешение с помощью замыкания:");
for (var i = 0; i < 3; i++) {
    (function(j) { // Создаем новую область видимости для каждой итерации
        setTimeout(function() {
            console.log(`замыкание: j = ${j}`); // 0, 1, 2
        }, 100);
    })(i); // Передаем текущее значение i как параметр j
}

console.log("\nСовременное решение с let:");
for (let i = 0; i < 3; i++) {
    setTimeout(function() {
        console.log(`let: i = ${i}`); // 0, 1, 2
    }, 100);
}

// Пример 5: Мемоизация с замыканиями
console.log("\n=== ПРИМЕР 5: МЕМОИЗАЦИЯ ===");

function создатьМемоизированнуюФункцию(дорогаяФункция) {
    const кэш = {}; // Приватный кэш

```

```

return function(аргумент) {
    // Проверяем, есть ли результат в кэше
    if (кэш[аргумент] !== undefined) {
        console.log(`Возвращаем из кэша для ${аргумент}`);
        return кэш[аргумент];
    }

    // Вычисляем и сохраняем в кэш
    console.log(`Вычисляем для ${аргумент}`);
    const результат = дорогаяФункция(аргумент);
    кэш[аргумент] = результат;

    return результат;
};
}

// Тестовая "дорогая" функция
function медленноеВычисление(n) {
    // Имитируем сложные вычисления
    let результат = 0;
    for (let i = 0; i < 1000000; i++) {
        результат += Math.sqrt(i) * n;
    }
    return результат;
}

const мемоизированная = создатьМемоизированнуюФункцию(медленноеВычисление);

console.log("Первый вызов с 5:", мемоизированная(5)); // Вычисляет
console.log("Второй вызов с 5:", мемоизированная(5)); // Из кэша
console.log("Вызов с 10:", мемоизированная(10));      // Вычисляет
console.log("Повторный вызов с 10:", мемоизированная(10)); // Из кэша

console.log("=".repeat(80));

```

Часть 4: Блочная область видимости

4.1. Что такое блочная область видимости?

```

javascript

console.log("=== БЛОЧНАЯ ОБЛАСТЬ ВИДИМОСТИ ===");

```

```
// Блочная область видимости появилась с ES6 (let и const)
// Переменные, объявленные с let/const внутри блока {}, видны только в этом блоке
```

```
// Блок — это любой код между {}
```

```
console.log("=== ОСНОВНЫЕ ПРИМЕРЫ ===");
```

```
{
  // Это блок
  let блочнаяПеременная = "Я видна только в этом блоке";
  const блочнаяКонстанта = "И я тоже";
  var стараяПеременная = "Я видна везде (потому что var)";

  console.log("Внутри блока:");
  console.log("  блочнаяПеременная:", блочнаяПеременная);
  console.log("  блочнаяКонстанта:", блочнаяКонстанта);
  console.log("  стараяПеременная:", стараяПеременная);
}
```

```
// За пределами блока
```

```
console.log("\nЗа пределами блока:");
```

```
console.log("  стараяПеременная:", стараяПеременная); // Работает
```

```
// console.log("  блочнаяПеременная:", блочнаяПеременная); // Ошибка!
```

```
// console.log("  блочнаяКонстанта:", блочнаяКонстанта); // Ошибка!
```

```
console.log("\n=== БЛОКИ В РАЗНЫХ КОНТЕКСТАХ ===");
```

```
// 1. Блок if
```

```
console.log("=== 1. БЛОК IF ===");
```

```
let внешняя = "внешняя";
```

```
if (true) {
  let внутриIf = "внутри if";
  const константаВнутриIf = "константа внутри if";

  console.log("Внутри if блока:");
  console.log("  внутриIf:", внутриIf);
  console.log("  константаВнутриIf:", константаВнутриIf);
  console.log("  внешняя (извне):", внешняя); // Видит внешнюю
}
```

```
// console.log(внутриIf); // Ошибка!
```

```
// console.log(константаВнутриIf); // Ошибка!
```

```
// 2. Блоки else и else if
console.log("\n=== 2. БЛОКИ ELSE ===");

let условие = true;

if (условие) {
    let переменнаяBIf = "в if";
    console.log("B if:", переменнаяBIf);
} else {
    let переменнаяBElse = "в else";
    console.log("B else:", переменнаяBElse);
}

// console.log(переменнаяBIf); // Ошибка!
// console.log(переменнаяBElse); // Ошибка!

// 3. Циклы
console.log("\n=== 3. ЦИКЛЫ ===");

// Каждая итерация цикла создает новый блок
for (let i = 0; i < 3; i++) {
    let переменнаяЦикла = `итерация ${i}`;
    console.log(переменнаяЦикла);
}

// console.log(i); // Ошибка!
// console.log(переменнаяЦикла); // Ошибка!

// 4. Блок switch
console.log("\n=== 4. БЛОК SWITCH ===");

let значение = "A";

switch (значение) {
    case "A": {
        // Используем дополнительные {} для создания блока
        let переменнаяBCase = "в case A";
        console.log(переменнаяBCase);
        break;
    }
    case "B": {
        let переменнаяBCase = "в case B"; // Можно использовать то же имя!
        console.log(переменнаяBCase);
        break;
    }
}
```



```

    }
}

// 5. Блок try-catch
console.log("\n=== 5. БЛОК TRY-CATCH ===");

try {
    let переменнаяBTry = "в try";
    throw new Error("Тестовая ошибка");
} catch (error) {
    // error имеет блочную область видимости
    let переменнаяBCatch = "в catch";
    console.log("В catch:", переменнаяBCatch);
    console.log("Ошибка:", error.message);

    // Не видит переменные из try
    // console.log(переменнаяBTry); // Ошибка!
}

// console.log(переменнаяBCatch); // Ошибка!
// console.log(error); // Ошибка!

// 6. Произвольные блоки для изоляции кода
console.log("\n=== 6. ПРОИЗВОЛЬНЫЕ БЛОКИ ===");

// Блок для вычислений
{
    let временнаяПеременная1 = "временные данные 1";
    console.log("Блок 1:", временнаяПеременная1);
}

// Другой блок (может использовать те же имена)
{
    let временнаяПеременная1 = "другие временные данные"; // Можно!
    console.log("Блок 2:", временнаяПеременная1);
}

console.log("=".repeat(80));

```

4.2. Сравнение var, let и const в разных контекстах

javascript

```

console.log("=== СРАВНЕНИЕ VAR, LET, CONST ===");

```

```
console.log("=== В ЦИКЛАХ ===");

console.log("1. Цикл for с var:");
for (var i = 0; i < 3; i++) {
    console.log("    Внутри цикла var: i =", i);
}
console.log("    После цикла var: i =", i); // 3 - переменная "утекла"

console.log("\n2. Цикл for с let:");
for (let j = 0; j < 3; j++) {
    console.log("    Внутри цикла let: j =", j);
}
// console.log("    После цикла let: j =", j); // Ошибка! j не определена

console.log("\n3. Цикл for с const:");
for (const k = 0; k < 3; k++) { // Ошибка! const нельзя изменять
    console.log("    Внутри цикла const: k =", k);
}

console.log("\n4. Цикл for...of с const:");
const массив = [1, 2, 3];
for (const элемент of массив) {
    // Каждая итерация создает новую константу
    console.log("    Элемент:", элемент);
}

console.log("\n=== В УСЛОВНЫХ ВЫРАЖЕНИЯХ ===");

console.log("5. if с var:");
if (true) {
    var переменнаяVar = "var в if";
    let переменнаяLet = "let в if";
    const переменнаяConst = "const в if";
}
console.log("    var после if:", переменнаяVar); // Работает
// console.log("    let после if:", переменнаяLet); // Ошибка
// console.log("    const после if:", переменнаяConst); // Ошибка

console.log("\n=== В ФУНКЦИЯХ ===");

function сравнениеВФункциях() {
    console.log("6. Внутри функции:");
```

```

if (true) {
    var функцияVar = "var в блоке";
    let функцияLet = "let в блоке";
    const функцияConst = "const в блоке";

    console.log("  В блоке:");
    console.log("    функцияVar:", функцияVar);
    console.log("    функцияLet:", функцияLet);
    console.log("    функцияConst:", функцияConst);
}

console.log("\n После блока:");
console.log("    функцияVar:", функцияVar); // Работает (var видна во всей функции)
// console.log("    функцияLet:", функцияLet); // Ошибка
// console.log("    функцияConst:", функцияConst); // Ошибка
}

```

сравнениеВФункциях();

```

console.log("\n=== ВЛОЖЕННЫЕ БЛОКИ ===");

```

```

let внешняяLet = "внешняя let";

```

```

{
    // Внутренний блок
    let внешняяLet = "внутренняя let"; // Можно переопределить!
    const внутренняяConst = "внутренняя const";

    console.log("7. Во внутреннем блоке:");
    console.log("  внешняяLet:", внешняяLet); // "внутренняя let"
    console.log("  внутренняяConst:", внутренняяConst);

    {
        // Еще более вложенный блок
        let внешняяLet = "самая внутренняя let"; // И еще раз!
        console.log("    В самом внутреннем блоке:");
        console.log("      внешняяLet:", внешняяLet); // "самая внутренняя let"
    }
}

```

```

console.log("\n В самом внешнем блоке:");
console.log("    внешняяLet:", внешняяLet); // "внешняя let"
// console.log("    внутренняяConst:", внутренняяConst); // Ошибка

```

```
console.log("\n=== ПРАКТИЧЕСКИЕ СЦЕНАРИИ ===");

console.log("8. Изоляция временных переменных:");
// Без блочной области видимости
function старыйСтиль() {
    var temp1, temp2, temp3; // Все объявлены заранее

    // Код использующий temp1
    temp1 = вычисление1();

    // Код использующий temp2
    temp2 = вычисление2();

    // Проблема: temp1 и temp2 все еще существуют и могут быть случайно использованы
}

function новыйСтиль() {
    // Изолируем первую часть вычислений
    {
        const temp1 = вычисление1();
        console.log("temp1 используется здесь:", temp1);
        // temp1 автоматически "убирается" после блока
    }

    // Изолируем вторую часть
    {
        const temp2 = вычисление2();
        console.log("temp2 используется здесь:", temp2);
    }

    // Нет риска случайно использовать temp1 или temp2
}

function вычисление1() { return "результат 1"; }
function вычисление2() { return "результат 2"; }

новыйСтиль();

console.log("=".repeat(80));
```

Часть 5: Особые случаи и граничные условия

5.1. Временная мертвая зона (Temporal Dead Zone - TDZ)

```
javascript

console.log("=== ВРЕМЕННАЯ МЕРТВАЯ ЗОНА (TDZ) ===");

// TDZ — это период между началом блока и моментом объявления переменной
// В этот период переменная существует, но к ней нельзя обращаться

console.log("=== ПРИМЕРЫ TDZ ===");

// Пример 1: Базовая TDZ
console.log("1. Базовая TDZ:");
{
  // Начало TDZ для блочнойПеременной
  // console.Log(блочнаяПеременная); // Ошибка ReferenceError!

  let блочнаяПеременная = "объявлена";
  // Конец TDZ
  console.log(" После объявления:", блочнаяПеременная); // Работает
}

// Пример 2: TDZ с const
console.log("\n2. TDZ с const:");
{
  // console.Log(блочнаяКонстанта); // Ошибка!

  const блочнаяКонстанта = "объявлена";
  console.log(" После объявления:", блочнаяКонстанта);
}

// Пример 3: Сравнение с var
console.log("\n3. Сравнение с var:");
{
  console.log(" var до объявления:", varПеременная); // undefined (не ошибка!)
  var varПеременная = "объявлена";
  console.log(" var после объявления:", varПеременная);
}

// Пример 4: TDZ в функциях
console.log("\n4. TDZ в функциях:");
```

```

function тестTDZ() {
    // console.Log(локальнаяTDZ); // Ошибка!

    let локальнаяTDZ = "объявлена в функции";
    console.log(" После объявления в функции:", локальнаяTDZ);
}

тестTDZ();

// Пример 5: Циклические зависимости
console.log("\n5. Циклические зависимости в TDZ:");
{
    // let a = b; // Ошибка! b еще в TDZ
    // let b = a;
}

// Пример 6: typeof в TDZ
console.log("\n6. typeof в TDZ:");
{
    // Для необъявленных переменных
    console.log(" typeof необъявленной:", typeof несуществующая); // "undefined"

    // Для переменных в TDZ
    // console.Log(typeof tdzПеременная); // Ошибка!
    let tdzПеременная;
    console.log(" typeof после объявления:", typeof tdzПеременная); // "undefined"
}

// Пример 7: TDZ с параметрами функций
console.log("\n7. TDZ с параметрами по умолчанию:");
function tdzСПараметрами(x = y, y = 10) {
    // x пытается использовать y, но y еще в TDZ!
    console.log(" x, y:", x, y);
}
// tdzСПараметрами(); // Ошибка!

function правильныеПараметры(x = 10, y = x * 2) {
    // x инициализирован раньше, поэтому можно использовать
    console.log(" Правильно: x =", x, ", y =", y);
}
правильныеПараметры();

// Пример 8: Практическое значение TDZ
console.log("\n8. Практическое значение TDZ:");

```

```

// Без TDZ (потенциальные ошибки):
function старыйКод() {
    console.log(" До объявления:", значение); // undefined (не ошибка, но потенциальный баг)
    var значение = "важное значение";
}

// С TDZ (безопасно):
function новыйКод() {
    // console.log(значение); // Ошибка! Защищает от использования до инициализации
    let значение = "важное значение";
    console.log(" После объявления:", значение);
}

старыйКод();
новыйКод();

console.log("=".repeat(80));

```

5.2. Область видимости с eval и with (устаревшие, но важно знать)

```

javascript

console.log("=== ОСОБЫЕ СЛУЧАИ: EVAL И WITH ===");

// Эти конструкции изменяют стандартное поведение областей видимости
// Они УСТАРЕЛИ и НЕ РЕКОМЕНДУЮТСЯ к использованию

console.log("=== 1. EVAL ===");

// eval выполняет строку как код JavaScript
// Проблема: может создавать переменные в текущей области видимости

function демонстрацияEval() {
    let локальная = "я локальная";

    // eval выполняется в текущей области видимости
    eval('var созданнаяEval = "создана eval");

    console.log(" локальная:", локальная);
    console.log(" созданнаяEval:", созданнаяEval); // Существует!
}

```

```
// Строгий режим (use strict) меняет поведение  
'use strict';  
eval('let строгаяEval = "в строгом режиме"');  
// console.log(строгаяEval); // Ошибка в строгом режиме  
}
```

```
демонстрацияEval();
```

```
console.log("\n=== 2. WITH ===");
```

```
// with добавляет объект в цепочку областей видимости  
// ОЧЕНЬ ОПАСНО и ЗАПРЕЩЕНО в строгом режиме
```

```
const объект = {  
  a: 1,  
  b: 2,  
  c: 3  
};
```

```
// Без with  
console.log(" Без with:");  
console.log("   a:", объект.a);  
console.log("   b:", объект.b);
```

```
// С with (НЕ ИСПОЛЬЗОВАТЬ!)
```

```
try {  
  with (объект) {  
    console.log("\n С with:");  
    console.log("   a:", a); // Ищет в объекте  
    console.log("   b:", b); // Ищет в объекте
```

```
// Опасность: если переменной нет в объекте, ищет в глобальной области  
d = 4; // Создает глобальную переменную!
```

```
}  
  
console.log(" Созданная глобальная d:", d); // 4 - ОПАСНО!  
} catch (e) {  
  console.log(" with запрещен в строгом режиме:", e.message);  
}
```

```
console.log("\n=== ПОЧЕМУ ЭТО ПЛОХО ===");
```

```
// 1. Производительность: компилятор не может оптимизировать код с eval/with  
// 2. Безопасность: eval может выполнять произвольный код
```



```
// 3. Читаемость: сложно понять, откуда берутся переменные
// 4. Отладка: сложно отслеживать ошибки
```

```
console.log("=".repeat(80));
```

5.3. Strict mode и его влияние на области видимости

```
javascript
```

```
console.log("=== STRICT MODE ===");
```

```
// Строгий режим (use strict) вводит дополнительные ограничения
// которые делают код более безопасным и предсказуемым
```

```
console.log("=== ОСНОВНЫЕ ИЗМЕНЕНИЯ В ОБЛАСТЯХ ВИДИМОСТИ ===");
```

```
// Пример 1: Запрет на неявное создание глобальных переменных
```

```
console.log("1. Неявные глобальные переменные:");
```

```
function нестрогийРежим() {
    неявнаяГлобальная = "я стала глобальной";
    console.log(" В нестрогом режиме:", неявнаяГлобальная);
}
```

```
function строгийРежим() {
    'use strict';
    // неявнаяГлобальная = "я вызову ошибку"; // ReferenceError
    let правильнаяЛокальная = "я локальная";
    console.log(" В строгом режиме:", правильнаяЛокальная);
}
```

```
нестрогийРежим();
```

```
console.log(" Глобальная после нестрогого режима:", неявнаяГлобальная);
```

```
строгийРежим();
```

```
// Пример 2: Запрет дублирования параметров
```

```
console.log("\n2. Дублирование параметров:");
```

```
function дублированныеПараметры(a, a, b) { // Разрешено в нестрогом режиме
    console.log(" a:", a, ", b:", b); // Последний a перезаписывает первый
}
```

```
function строгиеПараметры(a, b, c) {
```

```

    'use strict';
    // function ошибка(a, a, b) { } // SyntaxError в строгом режиме
    console.log(" Правильные параметры");
}

дублированныеПараметры(1, 2, 3);
строгиеПараметры(1, 2, 3);

// Пример 3: Запрет на использование with
console.log("\n3. Запрет with:");
'use strict';
// with ({} ) { } // SyntaxError

// Пример 4: eval в строгом режиме
console.log("\n4. eval в строгом режиме:");

function evalВСтрогом() {
    'use strict';

    let локальная = "закрытая";

    // В строгом режиме eval создает свою собственную область видимости
    eval('var evalVar = "из eval";');

    console.log(" локальная:", локальная);
    // console.log(" evalVar:", evalVar); // ReferenceError - не существует здесь
}

evalВСтрогом();

// Пример 5: Удаление переменных
console.log("\n5. Удаление переменных:");

var можноУдалить = "я var";
let нельзяУдалить = "я let";
const нельзяУдалитьConst = "я const";

console.log(" Можно удалить var?", delete можноУдалить); // true (в нестрогом режиме)
console.log(" Можно удалить let?", delete нельзяУдалить); // false
console.log(" Можно удалить const?", delete нельзяУдалитьConst); // false

function строгоеУдаление() {
    'use strict';
    var переменная = "test";

```

```

    // delete переменная; // SyntaxError в строгом режиме
}

// Пример 6: Аргументы функции
console.log("\n6. Аргументы functions:");

function аргументыВНестрогом(a, b) {
    arguments[0] = 100; // Меняет параметр a
    console.log(" Нестрогий: a =", a, ", arguments[0] =", arguments[0]); // 100, 100
}

function аргументыВСтрогом(a, b) {
    'use strict';
    arguments[0] = 100; // Не меняет параметр a
    console.log(" Строгий: a =", a, ", arguments[0] =", arguments[0]); // 1, 100
}

аргументыВНестрогом(1, 2);
аргументыВСтрогом(1, 2);

console.log("\n=== КАК ВКЛЮЧИТЬ STRICT MODE ===");

// 1. Для всего файла (первая строка)
// 'use strict';

// 2. Для конкретной функции
function функцияВСтрогом() {
    'use strict';
    // Код в строгом режиме
}

// 3. В модулях ES6 (автоматически)
// export function модульнаяФункция() {
//     // Уже в строгом режиме
// }

console.log("=".repeat(80));

```

Часть 6: Практические паттерны и лучшие практики

6.1. Работа с областями видимости в реальных проектах

```
javascript

console.log("=== ПРАКТИЧЕСКИЕ ПАТТЕРНЫ ===");

console.log("=== ПАТТЕРН 1: МОДУЛИ С ЗАМЫКАНИЯМИ ===");

const МодульПользователя = (function() {
  // Приватные переменные
  let пользователи = [];
  let следующийId = 1;

  // Приватные методы
  function найтиПользователя(id) {
    return пользователи.find(user => user.id === id);
  }

  function проверитьДубликат(email) {
    return пользователи.some(user => user.email === email);
  }

  // Публичный интерфейс
  return {
    добавитьПользователя: function(имя, email) {
      if (проверитьДубликат(email)) {
        throw new Error(`Пользователь с email ${email} уже существует`);
      }

      const пользователь = {
        id: следующийId++,
        имя,
        email,
        датаРегистрации: new Date()
      };

      пользователи.push(пользователь);
      return пользователь;
    },

    удалитьПользователя: function(id) {
```

```

const индекс = пользователи.findIndex(user => user.id === id);

if (индекс === -1) {
    throw new Error(`Пользователь с id ${id} не найден`);
}

return пользователи.splice(индекс, 1)[0];
},

получитьПользователя: function(id) {
    const пользователь = найтиПользователя(id);

    if (!пользователь) {
        throw new Error(`Пользователь с id ${id} не найден`);
    }

    // Возвращаем копию, чтобы защитить исходные данные
    return { ...пользователь };
},

получитьВсехПользователей: function() {
    // Возвращаем копию массива
    return [...пользователи];
},

// Статистика (только для чтения)
получитьСтатистику: function() {
    return {
        всего: пользователи.length,
        активные: пользователи.filter(u => u.активен).length,
        зарегистрированоСегодня: пользователи.filter(u => {
            const сегодня = new Date();
            return u.датаРегистрации.toDateString() === сегодня.toDateString();
        }).length
    };
}
};

})();

// Использование модуля
console.log("Модуль пользователя:");
const пользователь1 = МодульПользователя.добавитьПользователя("Анна",
"anna@example.com");

```

```

const пользователь2 = МодульПользователя.добавитьПользователя("Борис",
"boris@example.com");

console.log("Добавлены пользователи:");
console.log("  Пользователь 1:",
МодульПользователя.получитьПользователя(пользователь1.id));
console.log("  Пользователь 2:",
МодульПользователя.получитьПользователя(пользователь2.id));

console.log("Все пользователи:", МодульПользователя.получитьВсехПользователей());
console.log("Статистика:", МодульПользователя.получитьСтатистику());

// Попытка доступа к приватным данным
console.log("Прямой доступ к пользователям:", МодульПользователя.пользователи); //
undefined

console.log("\n=== ПАТТЕРН 2: ФАБРИКИ С ЗАМЫКАНИЯМИ ===");

function создатьКэш(название, options = {}) {
  // Параметры с значениями по умолчанию
  const {
    максРазмер = 100,
    ttl = 5 * 60 * 1000, // 5 минут
    стратегияУдаления = 'lru' // Least Recently Used
  } = options;

  // Приватные данные
  let данные = new Map();
  let статистика = {
    попаданий: 0,
    промахов: 0,
    записей: 0,
    удалений: 0
  };

  // Приватные методы
  function очиститьПросроченные() {
    const сейчас = Date.now();
    for (const [ключ, значение] of данных.entries()) {
      if (значение.истекает && значение.истекает < сейчас) {
        данных.delete(ключ);
        статистика.удалений++;
      }
    }
  }
}

```

```
}
```

```
function проверитьРазмер() {  
  if (данные.size > максРазмер) {  
    // Реализация стратегии удаления  
    if (стратегияУдаления === 'lru') {  
      // Находим самый старый элемент  
      let самыйСтарыйКлюч = null;  
      let самоеСтароеВремя = Infinity;  
  
      for (const [ключ, значение] of данных.entries()) {  
        if (значение.времяДоступа < самоеСтароеВремя) {  
          самоеСтароеВремя = значение.времяДоступа;  
          самыйСтарыйКлюч = ключ;  
        }  
      }  
  
      if (самыйСтарыйКлюч) {  
        данных.delete(самыйСтарыйКлюч);  
        статистика.удалений++;  
      }  
    }  
  }  
}
```

```
// Публичный интерфейс
```

```
return {  
  название,  
  
  set: function(ключ, значение, ttlMc = ttl) {  
    очиститьПросроченные();  
    проверитьРазмер();  
  
    const запись = {  
      данные: значение,  
      времяСоздания: Date.now(),  
      времяДоступа: Date.now()  
    };  
  
    if (ttlMc) {  
      запись.истекает = Date.now() + ttlMc;  
    }  
  
    данных.set(ключ, запись);  
  }  
}
```

```

        статистика.записей++;

        return true;
    },

    get: function(ключ) {
        очиститьПросроченные();

        const запись = данных.get(ключ);

        if (!запись) {
            статистика.промахов++;
            return null;
        }

        // Обновляем время доступа
        запись.времяДоступа = Date.now();
        статистика.попаданий++;

        return запись.данные;
    },

    delete: function(ключ) {
        return данных.delete(ключ);
    },

    clear: function() {
        данных.clear();
        статистика = {
            попаданий: 0,
            промахов: 0,
            записей: 0,
            удалений: 0
        };
    },

    get размер() {
        return данных.size;
    },

    getСтатистика: function() {
        return { ...статистика };
    },

```



```

    получитьКлючи: function() {
        return [...данных.keys()];
    }
};
}

// Использование фабрики кэшей
console.log("Фабрика кэшей:");
const кэшСессий = создатьКэш("сессии", {
    максРазмер: 1000,
    ttl: 30 * 60 * 1000 // 30 минут
});

const кэшИзображений = создатьКэш("изображения", {
    максРазмер: 500,
    ttl: 24 * 60 * 60 * 1000 // 24 часа
});

// Работа с кэшами
кэшСессий.set("user123", { имя: "Анна", роль: "admin" });
кэшИзображений.set("avatar123", "base64-encoded-image");

console.log("Кэш сессий:");
console.log("  user123:", кэшСессий.get("user123"));
console.log("  Размер:", кэшСессий.размер);
console.log("  Статистика:", кэшСессий.getСтатистика());

console.log("\nКэш изображений:");
console.log("  Размер:", кэшИзображений.размер);
console.log("  Статистика:", кэшИзображений.getСтатистика());

console.log("\n=== ПАТТЕРН 3: ИЗОЛЯЦИЯ КОДА С ПОМОЩЬЮ БЛОКОВ ===");

function обработатьЗаказ(заказ) {
    // Блок 1: Валидация
    let ошибки = [];
    {
        if (!заказ.товары || заказ.товары.length === 0) {
            ошибки.push("Заказ должен содержать товары");
        }

        if (!заказ.клиент || !заказ.клиент.имя) {
            ошибки.push("Не указан клиент");
        }
    }
}

```

```

    if (ошибки.length > 0) {
        throw new Error(`Ошибки валидации: ${ошибки.join(", ")}`);
    }
}

// Блок 2: Расчет стоимости
let стоимость;
{
    let промежуточнаяСтоимость = 0;

    for (const товар of заказ.товары) {
        const ценаТовара = товар.цена * товар.количество;
        промежуточнаяСтоимость += ценаТовара;
    }

    // Применяем скидку
    const скидка = заказ.клиент.постоянныйКлиент ? промежуточнаяСтоимость * 0.1 : 0;
    стоимость = промежуточнаяСтоимость - скидка;

    // промежуточнаяСтоимость автоматически "убирается" после блока
}

// Блок 3: Создание записи
let записьЗаказа;
{
    const id = generateId();
    const timestamp = new Date();

    записьЗаказа = {
        id,
        ...заказ,
        стоимость,
        timestamp,
        статус: "обработан"
    };

    // id и timestamp существуют только в этом блоке
}

return записьЗаказа;
}

function generateId() {

```

```

    return Math.random().toString(36).substr(2, 9);
}

// Пример использования
const тестовыйЗаказ = {
  товары: [
    { название: "Книга", цена: 500, количество: 2 },
    { название: "Ручка", цена: 50, количество: 5 }
  ],
  клиент: {
    имя: "Анна",
    постоянныйКлиент: true
  }
};

try {
  const результат = обработатьЗаказ(тестовыйЗаказ);
  console.log("Обработанный заказ:", результат);
} catch (error) {
  console.error("Ошибка обработки:", error.message);
}

console.log("=".repeat(80));

```

6.2. Оптимизация производительности и области видимости

```

javascript

console.log("=== ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ ===");

console.log("=== 1. ИЗБЕГАЙТЕ ЗАГРЯЗНЕНИЯ ГЛОБАЛЬНОЙ ОБЛАСТИ ===");

// Плохо: загрязнение глобальной области
var глобальнаяКонфигурация = { /* ... */ };
var кэшДанных = { /* ... */ };
var текущийПользователь = null;

// Хорошо: использование модулей/неймспейсов
const МоеПриложение = {
  config: { /* ... */ },
  cache: { /* ... */ },
  currentUser: null
};

```

// Еще лучше: использование IIFE

```
(function() {  
    const приватнаяКонфигурация = { /* ... */ };  
    const приватныйКэш = new Map();  
    let текущийПользователь = null;  
  
    window.публичныйAPI = {  
        // Только необходимые методы  
    };  
})();
```

`console.log("\n=== 2. ИСПОЛЬЗУЙТЕ ЛОКАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ ЧАСТО ИСПОЛЬЗУЕМЫХ СВОЙСТВ ===");`

// Плохо: многократный доступ к свойствам объекта

```
function плохойДоступ(массив) {  
    for (let i = 0; i < массив.length; i++) { // length читается каждый раз  
        console.log(массив[i]); // Доступ по индексу каждый раз  
    }  
}
```

// Хорошо: кэширование в локальную переменную

```
function хорошийДоступ(массив) {  
    const длина = массив.length; // Однократное чтение  
    const данные = массив; // Локальная ссылка  
  
    for (let i = 0; i < длина; i++) {  
        console.log(данные[i]);  
    }  
}
```

// Особенно важно в циклах

```
const большойОбъект = {  
    вложенный: {  
        глубоко: {  
            данные: [1, 2, 3, 4, 5]  
        }  
    }  
};
```

// Плохо

```
for (let i = 0; i < большойОбъект.вложенный.глубоко.данные.length; i++) {  
    // Каждый раз идет цепочка обращений к свойствам  
}
```

// Хорошо

```
const данные = большойОбъект.вложенный.глубоко.данные;  
const длинаДанных = данные.length;  
  
for (let i = 0; i < длинаДанных; i++) {  
    console.log(данные[i]);  
}  
  
console.log("\n=== 3. ИЗБЕГАЙТЕ СЛИШКОМ ВЛОЖЕННЫХ ОБЛАСТЕЙ ВИДИМОСТИ ===");
```

// Плохо: слишком глубокое вложение

```
function слишкомВложенная() {  
    let уровень1 = 1;  
  
    function внутренняя1() {  
        let уровень2 = 2;  
  
        function внутренняя2() {  
            let уровень3 = 3;  
  
            function внутренняя3() {  
                let уровень4 = 4;  
  
                // Поиск переменной идет по длинной цепочке  
                return уровень1 + уровень2 + уровень3 + уровень4;  
            }  
  
            return внутренняя3();  
        }  
  
        return внутренняя2();  
    }  
  
    return внутренняя1();  
}
```

// Хорошо: уплощение вложенности

```
function уплощенная() {  
    let уровень1 = 1;  
  
    function внутренняя1(уровень1) {  
        let уровень2 = 2;
```

```

function внутренняя2(уровень1, уровень2) {
    let уровень3 = 3;

    function внутренняя3(уровень1, уровень2, уровень3) {
        let уровень4 = 4;
        return уровень1 + уровень2 + уровень3 + уровень4;
    }

    return внутренняя3(уровень1, уровень2, уровень3);
}

return внутренняя2(уровень1, уровень2);
}

return внутренняя1(уровень1);
}

// Еще лучше: использование замыканий осознанно
function createCalculator(base) {
    let total = base;

    return {
        add: (value) => { total += value; },
        subtract: (value) => { total -= value; },
        getTotal: () => total
    };
}

console.log("\n=== 4. СОЗДАВАЙТЕ ПЕРЕМЕННЫЕ КАК МОЖНО ПОЗЖЕ ===");

// Плохо: объявление всех переменных в начале
function стараяШкола() {
    let результат;
    let временная1;
    let временная2;
    let временная3;

    // ... много кода ...

    временная1 = вычисление1();
    // ... код использующий временная1 ...

    временная2 = вычисление2();
    // ... код использующий временная2 ...

```

```

    временная3 = вычисление3();
    // ... код использующий временная3 ...

    результат = временная1 + временная2 + временная3;
    return результат;
}

// Хорошо: объявление рядом с использованием
function современныйСтиль() {
    // Код до...

    const временная1 = вычисление1();
    // ... код использующий временная1 ...

    const временная2 = вычисление2();
    // ... код использующий временная2 ...

    const временная3 = вычисление3();
    // ... код использующий временная3 ...

    const результат = временная1 + временная2 + временная3;
    return результат;
}

// Очень хорошо: изоляция в блоках
function изолированныйСтиль() {
    // Код до...

    let результат;

    // Блок для первой группы вычислений
    {
        const временная1 = вычисление1();
        const временная2 = вычисление2();
        const промежуточный = временная1 + временная2;

        // временная1 и временная2 автоматически "убираются"
    }

    // Блок для второй группы вычислений
    {
        const временная3 = вычисление3();
        const временная4 = вычисление4();
    }
}

```

```
    const промежуточный2 = временная3 * временная4;

    результат = промежуточный + промежуточный2;
}

return результат;
}

console.log("\n=== 5. ИСПОЛЬЗУЙТЕ CONST ПО УМОЛЧАНИЮ ===");

// const помогает движку JavaScript оптимизировать код
// потому что он знает, что ссылка не изменится

// Плохо: все переменные с let
function многоLet() {
    let имя = "Анна";
    let возраст = 25;
    let город = "Москва";
    let результат = 0;

    for (let i = 0; i < 10; i++) {
        результат += i;
    }

    return { имя, возраст, город, результат };
}

// Хорошо: const для неизменяемых значений
function многоConst() {
    const имя = "Анна";
    const возраст = 25;
    const город = "Москва";
    let результат = 0; // Только эта переменная изменяется

    for (let i = 0; i < 10; i++) {
        результат += i;
    }

    return { имя, возраст, город, результат };
}

// Отличный вариант: минимизация изменяемых переменных
function функциональныйСтиль() {
    const имя = "Анна";
```



```

const возраст = 25;
const город = "Москва";

// Вычисление без изменяемых переменных
const результат = Array.from({ length: 10 }, (_, i) => i)
  .reduce((sum, value) => sum + value, 0);

return { имя, возраст, город, результат };
}

console.log("\n=== 6. ИЗБЕГАЙТЕ EVAL И WITH ===");

// Эти конструкции не только опасны, но и убивают оптимизации

// Абсолютно плохо
function опасныйКод(выражение) {
  return eval(выражение); // НИКОГДА ТАК НЕ ДЕЛАЙТЕ!
}

// Хорошо: использование Function конструктора (осторожно!)
function безопаснаяАльтернатива(выражение, контекст) {
  const func = new Function(...Object.keys(контекст), `return ${выражение}`);
  return func(...Object.values(контекст));
}

// Еще лучше: использовать готовые парсеры или библиотеки
function математическоеВыражение(выражение, переменные) {
  // Используйте библиотеки вроде math.js или expr-eval
  // Они безопаснее и быстрее
}

console.log("=".repeat(80));

```

Часть 7: Практикум и упражнения

7.1. Интерактивный тренажер областей видимости

```

html

<!DOCTYPE html>
<html lang="ru">
<head>

```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Тренажер: Области видимости в JavaScript</title>
<style>
  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }

  body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    line-height: 1.6;
    color: #333;
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
    min-height: 100vh;
    padding: 20px;
  }

  .container {
    max-width: 1400px;
    margin: 0 auto;
    background: white;
    border-radius: 20px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    overflow: hidden;
  }

  header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px;
    text-align: center;
  }

  h1 {
    font-size: 2.5em;
    margin-bottom: 10px;
  }

  .subtitle {
    font-size: 1.2em;
    opacity: 0.9;
  }
```

```
.main-content {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 30px;  
  padding: 30px;  
}
```

```
@media (max-width: 1024px) {  
  .main-content {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.exercise-panel {  
  background: #f8f9fa;  
  border-radius: 10px;  
  padding: 25px;  
  border-left: 5px solid #667eea;  
}
```

```
.exercise-panel h3 {  
  color: #667eea;  
  margin-bottom: 20px;  
  font-size: 1.5em;  
  display: flex;  
  align-items: center;  
  gap: 10px;  
}
```

```
.exercise-panel h3 .icon {  
  font-size: 1.2em;  
}
```

```
.code-editor {  
  background: #2d2d2d;  
  color: #f8f8f2;  
  padding: 20px;  
  border-radius: 8px;  
  font-family: 'Courier New', monospace;  
  font-size: 14px;  
  margin: 15px 0;  
  overflow-x: auto;  
  white-space: pre-wrap;
```

```
    line-height: 1.5;
}

.code-line {
    margin: 5px 0;
    padding-left: 10px;
    border-left: 3px solid transparent;
    transition: all 0.3s;
}

.code-line.active {
    background: rgba(255, 255, 255, 0.1);
    border-left-color: #667eea;
}

.code-line.error {
    background: rgba(220, 53, 69, 0.1);
    border-left-color: #dc3545;
}

.code-line.success {
    background: rgba(40, 167, 69, 0.1);
    border-left-color: #28a745;
}

.code-comment {
    color: #888;
    font-style: italic;
}

.code-keyword {
    color: #ff79c6;
}

.code-string {
    color: #f1fa8c;
}

.code-number {
    color: #bd93f9;
}

.code-var {
    color: #8be9fd;
```

```
}
```

```
.controls {  
  display: flex;  
  gap: 10px;  
  margin-top: 20px;  
  flex-wrap: wrap;  
}
```

```
button {  
  padding: 12px 24px;  
  border: none;  
  border-radius: 8px;  
  background: #667eea;  
  color: white;  
  font-weight: bold;  
  cursor: pointer;  
  transition: all 0.3s;  
  font-size: 16px;  
}
```

```
button:hover {  
  background: #5a6fd8;  
  transform: translateY(-2px);  
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);  
}
```

```
button:active {  
  transform: translateY(0);  
}
```

```
button.secondary {  
  background: #6c757d;  
}
```

```
button.secondary:hover {  
  background: #5a6268;  
}
```

```
button.success {  
  background: #28a745;  
}
```

```
button.success:hover {
```

```
    background: #218838;
}
```

```
button.danger {
    background: #dc3545;
}
```

```
button.danger:hover {
    background: #c82333;
}
```

```
.result-panel {
    background: white;
    border-radius: 8px;
    border: 2px solid #e9ecef;
    padding: 20px;
    margin-top: 20px;
    min-height: 100px;
    font-family: 'Courier New', monospace;
    white-space: pre-wrap;
}
```

```
.scope-visualization {
    background: white;
    border-radius: 8px;
    border: 2px solid #e9ecef;
    padding: 20px;
    margin-top: 20px;
}
```

```
.scope-level {
    margin: 10px 0;
    padding: 15px;
    background: #f8f9fa;
    border-radius: 5px;
    border-left: 4px solid #dee2e6;
}
```

```
.scope-level.global {
    border-left-color: #dc3545;
}
```

```
.scope-level.function {
    border-left-color: #fd7e14;
}
```

```
}
```

```
.scope-level.block {  
  border-left-color: #28a745;  
}
```

```
.scope-level h4 {  
  margin-bottom: 10px;  
  color: #495057;  
}
```

```
.scope-variables {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 8px;  
}
```

```
.variable {  
  background: white;  
  padding: 5px 10px;  
  border-radius: 4px;  
  border: 1px solid #dee2e6;  
  font-size: 12px;  
  display: flex;  
  align-items: center;  
  gap: 5px;  
}
```

```
.variable.var {  
  border-color: #dc3545;  
  color: #dc3545;  
}
```

```
.variable.let {  
  border-color: #007bff;  
  color: #007bff;  
}
```

```
.variable.const {  
  border-color: #28a745;  
  color: #28a745;  
}
```

```
.variable .type {
```

```
    font-size: 10px;
    opacity: 0.7;
}

.theory-panel {
    grid-column: 1 / -1;
    background: #e3f2fd;
    padding: 25px;
    border-radius: 10px;
    margin-top: 20px;
}

.theory-panel h3 {
    color: #1976d2;
    margin-bottom: 15px;
    font-size: 1.4em;
}

.theory-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 20px;
}

.theory-card {
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.theory-card h4 {
    color: #1976d2;
    margin-bottom: 10px;
    padding-bottom: 10px;
    border-bottom: 2px solid #e3f2fd;
}

footer {
    text-align: center;
    padding: 20px;
    background: #f8f9fa;
    color: #666;
    border-top: 1px solid #dee2e6;
```



```
}
```

```
.highlight {  
  background: #fff3cd;  
  padding: 2px 5px;  
  border-radius: 3px;  
  font-weight: bold;  
}
```

```
.error-message {  
  color: #dc3545;  
  background: #f8d7da;  
  padding: 10px;  
  border-radius: 5px;  
  margin: 10px 0;  
  border: 1px solid #f5c6cb;  
}
```

```
.success-message {  
  color: #155724;  
  background: #d4edda;  
  padding: 10px;  
  border-radius: 5px;  
  margin: 10px 0;  
  border: 1px solid #c3e6cb;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<header>
```

```
<h1>Тренажер: Области видимости в JavaScript</h1>
```

```
<div class="subtitle">Изучайте и практикуйтесь с глобальной, функциональной и  
блочной областями видимости</div>
```

```
</header>
```

```
<div class="main-content">
```

```
<!-- Упражнение 1 -->
```

```
<div class="exercise-panel">
```

```
<h3><span class="icon">🔍 </span> Упражнение 1: Поиск переменных</h3>
```

```
<p>Проследите, как JavaScript ищет переменные по цепочке областей видимости:</p>
```

```
<div class="code-editor" id="code1">
```

```

<div class="code-line"><span class="code-keyword">let</span> <span class="code-var">глобальная</span> = <span class="code-string">"глобальное значение"</span>;</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-keyword">function</span> <span class="code-var">уровень1</span>() {</div>
<div class="code-line">    <span class="code-keyword">let</span> <span class="code-var">переменная1</span> = <span class="code-string">"уровень 1"</span>;</div>
<div class="code-line"></div>
<div class="code-line">    <span class="code-keyword">function</span> <span class="code-var">уровень2</span>() {</div>
<div class="code-line">        <span class="code-keyword">let</span> <span class="code-var">переменная2</span> = <span class="code-string">"уровень 2"</span>;</div>
<div class="code-line"></div>
<div class="code-line">        <span class="code-keyword">function</span> <span class="code-var">уровень3</span>() {</div>
<div class="code-line">            <span class="code-keyword">let</span> <span class="code-var">переменная3</span> = <span class="code-string">"уровень 3"</span>;</div>
<div class="code-line"></div>
<div class="code-line">            <span class="code-comment">// Какие переменные
доступны здесь?</span></div>
<div class="code-line">                console.<span class="code-keyword">log</span>(<span class="code-var">переменная3</span>);</div>
<div class="code-line">                console.<span class="code-keyword">log</span>(<span class="code-var">переменная2</span>);</div>
<div class="code-line">                console.<span class="code-keyword">log</span>(<span class="code-var">переменная1</span>);</div>
<div class="code-line">                console.<span class="code-keyword">log</span>(<span class="code-var">глобальная</span>);</div>
<div class="code-line">            }</div>
<div class="code-line"></div>
<div class="code-line">        <span class="code-var">уровень3</span>();</div>
<div class="code-line">    }</div>
<div class="code-line"></div>
<div class="code-line">    <span class="code-var">уровень2</span>();</div>
<div class="code-line">}</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-var">уровень1</span>();</div>
</div>

<div class="controls">
    <button onclick="runExercise1()">Выполнить код</button>
    <button onclick="explainExercise1()" class="secondary">Объяснить поиск</button>
    <button onclick="visualizeScope1()" class="success">Визуализировать
области</button>

```

```
</div>
```

```
<div class="result-panel" id="result1"></div>
```

```
<div class="scope-visualization" id="scope1" style="display: none;"></div>
```

```
</div>
```

```
<!-- Упражнение 2 -->
```

```
<div class="exercise-panel">
```

```
<h3><span class="icon">□ </span> Упражнение 2: Замыкания</h3>
```

```
<p>Создайте счетчик с использованием замыкания и проанализируйте его работу:</p>
```

```
<div class="code-editor" id="code2">
```

```
<div class="code-line"><span class="code-comment">// Фабрика счетчиков</span></div>
```

```
<div class="code-line"><span class="code-keyword">function</span> <span class="code-var">createCounter</span>() {</div>
```

```
<div class="code-line">    <span class="code-keyword">let</span> <span class="code-var">count</span> = <span class="code-number">0</span>; <span class="code-comment">//  
Приватная переменная</span></div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">    <span class="code-comment">// Возвращаем объект с  
методами</span></div>
```

```
<div class="code-line">    <span class="code-keyword">return</span> {</div>
```

```
<div class="code-line">        <span class="code-var">increment</span>: <span  
class="code-keyword">function</span>() {</div>
```

```
<div class="code-line">            <span class="code-var">count</span>++;</div>
```

```
<div class="code-line">            <span class="code-keyword">return</span> <span  
class="code-var">count</span>;</div>
```

```
<div class="code-line">        },</div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">        <span class="code-var">decrement</span>: <span  
class="code-keyword">function</span>() {</div>
```

```
<div class="code-line">            <span class="code-var">count</span>--;</div>
```

```
<div class="code-line">            <span class="code-keyword">return</span> <span  
class="code-var">count</span>;</div>
```

```
<div class="code-line">        },</div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">        <span class="code-var">getValue</span>: <span class="code-  
keyword">function</span>() {</div>
```

```
<div class="code-line">            <span class="code-keyword">return</span> <span  
class="code-var">count</span>;</div>
```

```
<div class="code-line">        },</div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">        <span class="code-var">reset</span>: <span class="code-  
keyword">function</span>() {</div>
```

```

<div class="code-line">                <span class="code-var">count</span> = <span
class="code-number">0</span>;</div>
<div class="code-line">                <span class="code-keyword">return</span> <span
class="code-var">count</span>;</div>
<div class="code-line">                }</div>
<div class="code-line">        };</div>
<div class="code-line">}</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Создаем два независимых
счетчика</span></div>
<div class="code-line"><span class="code-keyword">const</span> <span class="code-
var">counter1</span> = <span class="code-var">createCounter</span>();</div>
<div class="code-line"><span class="code-keyword">const</span> <span class="code-
var">counter2</span> = <span class="code-var">createCounter</span>();</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Тестируем работу</span></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"Счетчик 1:"</span>);</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">" Начальное значение:"</span>, <span class="code-var">counter1</span>.<span
class="code-var">getValue</span>());</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">" После increment:"</span>, <span class="code-var">counter1</span>.<span
class="code-var">increment</span>());</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">" Еще раз increment:"</span>, <span class="code-var">counter1</span>.<span
class="code-var">increment</span>());</div>
<div class="code-line"></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"\nСчетчик 2:"</span>);</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">" Начальное значение:"</span>, <span class="code-var">counter2</span>.<span
class="code-var">getValue</span>());</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">" После decrement:"</span>, <span class="code-var">counter2</span>.<span
class="code-var">decrement</span>());</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Попробуем получить прямой доступ к
count</span></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"\nПрямой доступ к count:"</span>, <span class="code-var">counter1</span>.<span
class="code-var">count</span>);</div>
</div>

```

```

    <div class="controls">
      <button onclick="runExercise2()">Выполнить код</button>
      <button onclick="explainExercise2()" class="secondary">Объяснить
замыкание</button>
      <button onclick="testClosure()" class="success">Протестировать</button>
    </div>

    <div class="result-panel" id="result2"></div>
  </div>

  <!-- Упражнение 3 -->
  <div class="exercise-panel">
    <h3><span class="icon">❏ </span> Упражнение 3: TDZ и ошибки</h3>
    <p>Изучите временную мертвую зону (TDZ) и распространенные ошибки:</p>

    <div class="code-editor" id="code3">
<div class="code-line"><span class="code-comment">// Пример 1: TDZ с let</span></div>
<div class="code-line">{</div>
<div class="code-line">  <span class="code-comment">// Начало TDZ для
переменной</span></div>
<div class="code-line">  <span class="code-comment">// console.log(tdzVar); //
Раскомментируйте для ошибки</span></div>
<div class="code-line">  </div>
<div class="code-line">  <span class="code-keyword">let</span> <span class="code-
var">tdzVar</span> = <span class="code-string">"объявлена"</span>;</div>
<div class="code-line">  <span class="code-comment">// Конец TDZ</span></div>
<div class="code-line">  </div>
<div class="code-line">  console.<span class="code-keyword">log</span>(<span
class="code-string">"После объявления:"</span>, <span class="code-
var">tdzVar</span>);</div>
<div class="code-line">}</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Пример 2: Сравнение с
var</span></div>
<div class="code-line">{</div>
<div class="code-line">  console.<span class="code-keyword">log</span>(<span
class="code-string">"До объявления var:"</span>, <span class="code-
var">hoistedVar</span>);</div>
<div class="code-line">  </div>
<div class="code-line">  <span class="code-keyword">var</span> <span class="code-
var">hoistedVar</span> = <span class="code-string">"поднята с undefined"</span>;</div>
<div class="code-line">  </div>

```

```

<div class="code-line">    console.<span class="code-keyword">log</span>(<span
class="code-string">"После объявления var:"</span>, <span class="code-
var">hoistedVar</span>);</div>
<div class="code-line"></div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Пример 3: Блочная область
видимости</span></div>
<div class="code-line"><span class="code-keyword">if</span> (<span class="code-
keyword">>true</span>) {</div>
<div class="code-line">    <span class="code-keyword">let</span> <span class="code-
var">внутриIf</span> = <span class="code-string">"видна только в блоке"</span>;</div>
<div class="code-line">    <span class="code-keyword">var</span> <span class="code-
var">утечка</span> = <span class="code-string">"утекла из блока"</span>;</div>
<div class="code-line">    </div>
<div class="code-line">    console.<span class="code-keyword">log</span>(<span
class="code-string">"Внутри if:"</span>, <span class="code-var">внутриIf</span>);</div>
<div class="code-line">}</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Проверка доступности</span></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"После if:"</span>);</div>
<div class="code-line"><span class="code-keyword">try</span> {</div>
<div class="code-line">    console.<span class="code-keyword">log</span>(<span
class="code-string">"    внутриIf:"</span>, <span class="code-var">внутриIf</span>);</div>
<div class="code-line">}<span class="code-keyword">catch</span> (<span class="code-
var">e</span>) {</div>
<div class="code-line">    console.<span class="code-keyword">log</span>(<span
class="code-string">"    внутриIf: ОШИБКА -"</span>, <span class="code-var">e</span>.<span
class="code-var">message</span>);</div>
<div class="code-line">}</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"    утечка:"</span>, <span class="code-var">утечка</span>);</div>
    </div>

    <div class="controls">
        <button onclick="runExercise3()">Выполнить код</button>
        <button onclick="explainTDZ()" class="secondary">Объяснить TDZ</button>
        <button onclick="testErrors()" class="danger">Показать ошибки</button>
    </div>

    <div class="result-panel" id="result3"></div>
</div>

<!-- Упражнение 4 -->

```

```
<div class="exercise-panel">
```

```
<h3><span class="icon">❏ </span> Упражнение 4: Паттерны модулей</h3>
```

```
<p>Создайте модуль с приватными и публичными методами:</p>
```

```
<div class="code-editor" id="code4">
```

```
<div class="code-line"><span class="code-comment">// Модуль для работы с задачами (To-Do)</span></div>
```

```
<div class="code-line"><span class="code-keyword">const</span> <span class="code-var">TaskManager</span> = (<span class="code-keyword">function</span>() {</div>
```

```
<div class="code-line">    <span class="code-comment">// Приватные переменные</span></div>
```

```
<div class="code-line">    <span class="code-keyword">let</span> <span class="code-var">tasks</span> = [];</div>
```

```
<div class="code-line">    <span class="code-keyword">let</span> <span class="code-var">nextId</span> = <span class="code-number">1</span>;</div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">    <span class="code-comment">// Приватные методы</span></div>
```

```
<div class="code-line">    <span class="code-keyword">function</span> <span class="code-var">findTaskIndex</span>(<span class="code-var">id</span>) {</div>
```

```
<div class="code-line">        <span class="code-keyword">return</span> <span class="code-var">tasks</span>.<span class="code-keyword">findIndex</span>(<span class="code-var">task</span> => <span class="code-var">task</span>.<span class="code-var">id</span> === <span class="code-var">id</span>);</div>
```

```
<div class="code-line">    }</div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">    <span class="code-keyword">function</span> <span class="code-var">validateTask</span>(<span class="code-var">task</span>) {</div>
```

```
<div class="code-line">        <span class="code-keyword">if</span> (!<span class="code-var">task</span>.<span class="code-var">title</span> || <span class="code-var">task</span>.<span class="code-var">title</span>.<span class="code-var">trim</span>() === <span class="code-string">'</span>)</div>
```

```
<div class="code-line">            <span class="code-keyword">throw</span> <span class="code-keyword">new</span> <span class="code-keyword">Error</span>(<span class="code-string">'Задача должна иметь название'</span>);</div>
```

```
<div class="code-line">    }</div>
```

```
<div class="code-line">    }</div>
```

```
<div class="code-line">    </div>
```

```
<div class="code-line">    <span class="code-comment">// Публичный интерфейс</span></div>
```

```
<div class="code-line">    <span class="code-keyword">return</span> {</div>
```

```
<div class="code-line">        <span class="code-var">addTask</span>: <span class="code-keyword">function</span>(<span class="code-var">title</span>, <span class="code-var">priority</span> = <span class="code-string">'medium'</span>) {</div>
```

```
<div class="code-line">            <span class="code-var">validateTask</span>({ <span class="code-var">title</span> });</div>
```



```

<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-keyword">const</span> <span
class="code-var">task</span> = {</div>
<div class="code-line">                                <span class="code-var">id</span>: <span
class="code-var">nextId</span>++,</div>
<div class="code-line">                                <span class="code-var">title</span>,</div>
<div class="code-line">                                <span class="code-var">priority</span>,</div>
<div class="code-line">                                <span class="code-var">completed</span>: <span
class="code-keyword">false</span>,</div>
<div class="code-line">                                <span class="code-var">createdAt</span>: <span
class="code-keyword">new</span> <span class="code-keyword">Date</span>()</div>
<div class="code-line">                                };</div>
<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-var">tasks</span>.<span class="code-
keyword">push</span>(<span class="code-var">task</span>);</div>
<div class="code-line">                                <span class="code-keyword">return</span> <span
class="code-var">task</span>;</div>
<div class="code-line">                                },</div>
<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-var">completeTask</span>: <span
class="code-keyword">function</span>(<span class="code-var">id</span>) {</div>
<div class="code-line">                                <span class="code-keyword">const</span> <span
class="code-var">index</span> = <span class="code-var">findTaskIndex</span>(<span
class="code-var">id</span>);</div>
<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-keyword">if</span> (<span
class="code-var">index</span> === -<span class="code-number">1</span>) {</div>
<div class="code-line">                                <span class="code-keyword">throw</span> <span
class="code-keyword">new</span> <span class="code-keyword">Error</span>(<span
class="code-string">`Задача с id ${id} не найдена`</span>);</div>
<div class="code-line">                                }</div>
<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-var">tasks</span>[<span class="code-
var">index</span>].<span class="code-var">completed</span> = <span class="code-
keyword">true</span>;</div>
<div class="code-line">                                <span class="code-keyword">return</span> <span
class="code-var">tasks</span>[<span class="code-var">index</span>];</div>
<div class="code-line">                                },</div>
<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-var">getTasks</span>: <span class="code-
keyword">function</span>(<span class="code-var">filter</span> = <span class="code-
string">'all'</span>) {</div>
<div class="code-line">                                <span class="code-keyword">switch</span> (<span
class="code-var">filter</span>) {</div>

```



```

<div class="code-line">                                <span class="code-keyword">case</span> <span
class="code-string">'active'</span>:</div>
<div class="code-line">                                <span class="code-keyword">return</span> <span
class="code-var">tasks</span>.<span class="code-keyword">filter</span>(<span class="code-
var">task</span> => !<span class="code-var">task</span>.<span class="code-
var">completed</span>)</div>
<div class="code-line">                                <span class="code-keyword">case</span> <span
class="code-string">'completed'</span>:</div>
<div class="code-line">                                <span class="code-keyword">return</span> <span
class="code-var">tasks</span>.<span class="code-keyword">filter</span>(<span class="code-
var">task</span> => <span class="code-var">task</span>.<span class="code-
var">completed</span>)</div>
<div class="code-line">                                <span class="code-keyword">default</span>:</div>
<div class="code-line">                                <span class="code-keyword">return</span>
[...<span class="code-var">tasks</span>]; <span class="code-comment">// Возвращаем
копию</span></div>
<div class="code-line">                                }</div>
<div class="code-line">                                },</div>
<div class="code-line">                                </div>
<div class="code-line">                                <span class="code-var">getStats</span>: <span class="code-
keyword">function</span>() {</div>
<div class="code-line">                                <span class="code-keyword">return</span> {</div>
<div class="code-line">                                <span class="code-var">total</span>: <span
class="code-var">tasks</span>.<span class="code-var">length</span>,</div>
<div class="code-line">                                <span class="code-var">completed</span>: <span
class="code-var">tasks</span>.<span class="code-keyword">filter</span>(<span class="code-
var">t</span> => <span class="code-var">t</span>.<span class="code-
var">completed</span>).<span class="code-var">length</span>,</div>
<div class="code-line">                                <span class="code-var">active</span>: <span
class="code-var">tasks</span>.<span class="code-keyword">filter</span>(<span class="code-
var">t</span> => !<span class="code-var">t</span>.<span class="code-
var">completed</span>).<span class="code-var">length</span></div>
<div class="code-line">                                };</div>
<div class="code-line">                                }</div>
<div class="code-line">                                };</div>
<div class="code-line">                                }}()</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-comment">// Тестирование модуля</span></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"Тестирование TaskManager:"</span>)</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-keyword">const</span> <span class="code-
var">task1</span> = <span class="code-var">TaskManager</span>.<span class="code-

```

```

var">addTask</span>(<span class="code-string">"Изучить JavaScript"</span>, <span
class="code-string">"high"</span>);</div>
<div class="code-line"><span class="code-keyword">const</span> <span class="code-
var">task2</span> = <span class="code-var">TaskManager</span>.<span class="code-
var">addTask</span>(<span class="code-string">"Купить продукты"</span>);</div>
<div class="code-line"></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"Добавленные задачи:"</span>, <span class="code-var">TaskManager</span>.<span
class="code-var">getTasks</span>());</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"Статистика:"</span>, <span class="code-var">TaskManager</span>.<span
class="code-var">getStats</span>());</div>
<div class="code-line"></div>
<div class="code-line"><span class="code-var">TaskManager</span>.<span class="code-
var">completeTask</span>(<span class="code-var">task1</span>.<span class="code-
var">id</span>);</div>
<div class="code-line"></div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"После выполнения задачи:"</span>);</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"  Активные задачи:"</span>, <span class="code-var">TaskManager</span>.<span
class="code-var">getTasks</span>(<span class="code-string">'active'</span>));</div>
<div class="code-line">console.<span class="code-keyword">log</span>(<span class="code-
string">"  Общая статистика:"</span>, <span class="code-var">TaskManager</span>.<span
class="code-var">getStats</span>());</div>
</div>

<div class="controls">
  <button onclick="runExercise4()">Выполнить код</button>
  <button onclick="explainModule()" class="secondary">Объяснить модуль</button>
  <button onclick="testModule()" class="success">Дополнительные тесты</button>
</div>

<div class="result-panel" id="result4"></div>
</div>

<!-- Теория -->
<div class="theory-panel">
  <h3>☐ Теория: Типы областей видимости</h3>
  <div class="theory-grid">
    <div class="theory-card">
      <h4>☐ Глобальная область</h4>
      <ul>
        <li>Самый внешний уровень</li>

```

```

    <li>Переменные доступны везде</li>
    <li>Опасность: загрязнение пространства имен</li>
    <li><span class="highlight">Используйте модули для изоляции</span></li>
  </ul>
</div>
<div class="theory-card">
  <h4>□ Функциональная область</h4>
  <ul>
    <li>Создается при вызове функции</li>
    <li><code>var</code> имеет функциональную область</li>
    <li>Основы для замыканий</li>
    <li><span class="highlight">Параметры функции тоже здесь</span></li>
  </ul>
</div>
<div class="theory-card">
  <h4>□ Блочная область</h4>
  <ul>
    <li>Появилась в ES6</li>
    <li><code>let</code> и <code>const</code> имеют блочную область</li>
    <li>Любой код между <code>{</code></li>
    <li><span class="highlight">Изолирует временные переменные</span></li>
  </ul>
</div>
<div class="theory-card">
  <h4>□ Замыкания</h4>
  <ul>
    <li>Функция + её лексическое окружение</li>
    <li>Позволяет создавать приватные переменные</li>
    <li>Основы модульного паттерна</li>
    <li><span class="highlight">"Запоминает" переменные</span></li>
  </ul>
</div>
</div>
<div style="margin-top: 20px; padding: 15px; background: #d1ecf1; border-radius: 5px; border-left: 4px solid #0c5460;">
  <strong>□ Правило золотого стандарта:</strong><br>
  1. Используйте <code>const</code> по умолчанию<br>
  2. Используйте <code>let</code> для изменяемых переменных<br>
  3. Никогда не используйте <code>var</code> в новом коде<br>
  4. Изолируйте код с помощью блоков и модулей
</div>
</div>
</div>

```

```
<footer>
  <p>Тренажер по областям видимости в JavaScript | Используйте консоль браузера (F12)
для экспериментов</p>
  <p style="font-size: 0.9em; margin-top: 10px;">Попробуйте изменять код в примерах и
наблюдайте за результатами!</p>
</footer>
</div>
```

```
<script>
  // Упражнение 1: Поиск переменных
  function runExercise1() {
    clearResults();

    let результат = "";
    try {
      // Выполняем код из упражнения
      let глобальная = "глобальное значение";

      function уровень1() {
        let переменная1 = "уровень 1";

        function уровень2() {
          let переменная2 = "уровень 2";

          function уровень3() {
            let переменная3 = "уровень 3";

            // Какие переменные доступны здесь?
            результат += `переменная3: ${переменная3}\n`;
            результат += `переменная2: ${переменная2}\n`;
            результат += `переменная1: ${переменная1}\n`;
            результат += `глобальная: ${глобальная}\n`;
          }

          уровень3();
        }

        уровень2();
      }

      уровень1();

      document.getElementById('result1').innerHTML =
        `<span class="success-message">👌 Код выполнен успешно!</span>\n\n` +
```

```

    `<strong>Результат:</strong>\n${результат}`;

} catch (error) {
    document.getElementById('result1').innerHTML =
        `<span class="error-message">× Ошибка: ${error.message}</span>`;
}
}

function explainExercise1() {
    const объяснение = `
        <strong>❏ Как работает поиск переменных:</strong>

        1. <span class="highlight">Цепочка областей видимости (Scope Chain)</span>:
            - Когда функция ищет переменную, она сначала ищет в своей локальной области
            - Если не находит, поднимается на уровень выше
            - И так до глобальной области

        2. <span class="highlight">В этом примере:</span>
            - функция уровень3() ищет переменную3 → находит сразу
            - ищет переменная2 → не находит → поднимается в уровень2() → находит
            - ищет переменная1 → не находит → поднимается в уровень1() → находит
            - ищет глобальная → не находит → поднимается в глобальную область → находит

        3. <span class="highlight">Лексическая (статическая) область видимости</span>:
            - Область видимости определяется местом объявления в коде
            - Не зависит от места вызова функции

        <em>Это основа работы замыканий в JavaScript!</em>
    `;

    document.getElementById('result1').innerHTML = объяснение;
}

function visualizeScope1() {
    const визуализация = `
        <h4>❏ Визуализация областей видимости:</h4>

        <div class="scope-level global">
            <h4>Глобальная область</h4>
            <div class="scope-variables">
                <div class="variable let">
                    <span class="type">let</span> глобальная
                </div>
            </div>
        </div>
    `;

```

```

</div>

<div class="scope-level function">
  <h4>Функция: уровень1(</h4>
  <div class="scope-variables">
    <div class="variable let">
      <span class="type">let</span> переменная1
    </div>
  </div>
</div>

<div class="scope-level function">
  <h4>Функция: уровень2(</h4>
  <div class="scope-variables">
    <div class="variable let">
      <span class="type">let</span> переменная2
    </div>
  </div>
</div>

<div class="scope-level function">
  <h4>Функция: уровень3(</h4>
  <div class="scope-variables">
    <div class="variable let">
      <span class="type">let</span> переменная3
    </div>
  </div>
</div>

<p style="margin-top: 15px; color: #666;">
  <strong>Стрелки поиска переменных из уровень3():</strong><br>
  переменная3 →  найдено сразу<br>
  переменная2 →  уровень2() →  найдено<br>
  переменная1 →  уровень1() →  найдено<br>
  глобальная →  глобальная область →  найдено
</p>
`;

const scopePanel = document.getElementById('scope1');
scopePanel.innerHTML = визуализация;
scopePanel.style.display = 'block';
}

```

// Упражнение 2: Замыкания

```
function runExercise2() {
  clearResults();

  let результат = "";
  try {
    // Выполняем код из упражнения
    function createCounter() {
      let count = 0;

      return {
        increment: function() {
          count++;
          return count;
        },

        decrement: function() {
          count--;
          return count;
        },

        getValue: function() {
          return count;
        },

        reset: function() {
          count = 0;
          return count;
        }
      };
    }

    const counter1 = createCounter();
    const counter2 = createCounter();

    результат += "Счетчик 1:\n";
    результат += ` Начальное значение: ${counter1.getValue()}\n`;
    результат += ` После increment: ${counter1.increment()}\n`;
    результат += ` Еще раз increment: ${counter1.increment()}\n\n`;

    результат += "Счетчик 2:\n";
    результат += ` Начальное значение: ${counter2.getValue()}\n`;
    результат += ` После decrement: ${counter2.decrement()}\n\n`;

    результат += "Прямой доступ к count: " + counter1.count;
  }
}
```

```

document.getElementById('result2').innerHTML =
  `<span class="success-message">✅ Замыкание работает!</span>\n\n` +
  `<strong>Результат:</strong>\n${результат}`;

} catch (error) {
  document.getElementById('result2').innerHTML =
    `<span class="error-message">❌ Ошибка: ${error.message}</span>`;
}
}

function explainExercise2() {
  const объяснение = `
<strong>✅ Что такое замыкание?</strong>

<span class="highlight">Замыкание = функция + её лексическое окружение</span>

1. <strong>Как работает:</strong>
  - Функция createCounter() создает локальную переменную count
  - Возвращает объект с методами, которые используют count
  - После завершения createCounter(), переменная count должна была бы удалиться
  - Но возвращенные методы "замыкают" её, сохраняя доступ

2. <strong>Каждое замыкание уникально:</strong>
  - counter1 и counter2 имеют РАЗНЫЕ переменные count
  - Они не влияют друг на друга
  - Это основа приватных переменных в JavaScript

3. <strong>Прямой доступ невозможен:</strong>
  - count — приватная переменная
  - Доступна только через методы
  - counter1.count возвращает undefined

<em>Замыкания — это мощный инструмент для создания инкапсулированного кода!</em>
`;

  document.getElementById('result2').innerHTML = объяснение;
}

function testClosure() {
  let результат = "\n✅ Дополнительные тесты замыкания:\n\n";

  function createCounter(start = 0) {
    let count = start;

```



```

    return {
      increment: () => ++count,
      getValue: () => count,
      isEven: () => count % 2 === 0
    };
  }

  const counterA = createCounter(10);
  const counterB = createCounter(100);

  результат += "Счетчик A (начало: 10):\n";
  результат += ` Текущее: ${counterA.getValue()}\n`;
  результат += ` Четное? ${counterA.isEven()}\n`;
  результат += ` После increment: ${counterA.increment()}\n`;
  результат += ` Теперь четное? ${counterA.isEven()}\n\n`;

  результат += "Счетчик B (начало: 100):\n";
  результат += ` Текущее: ${counterB.getValue()}\n`;
  результат += ` Четное? ${counterB.isEven()}\n`;

  // Показываем, что они независимы
  результат += "\n□ Независимость замыканий:\n";
  результат += ` counterA: ${counterA.getValue()}\n`;
  результат += ` counterB: ${counterB.getValue()}\n`;
  результат += ` (разные значения — разные переменные count)`;

  document.getElementById('result2').innerHTML +=
    ` \n\n<strong>□ Тесты:</strong>\n${результат}`;
}

// Упражнение 3: TDZ и ошибки
function runExercise3() {
  clearResults();

  let результат = "";
  try {
    // Пример 1: TDZ с let
    результат += "Пример 1: TDZ с let\n";
    результат += "=".repeat(30) + "\n";

    {
      // console.log(tdzVar); // Раскомментируйте для ошибки
      let tdzVar = "объявлена";
    }
  }
}

```

```

    результат += `После объявления: ${tdzVar}\n\n`;
}

// Пример 2: Сравнение с var
результат += "Пример 2: Сравнение с var\n";
результат += "=".repeat(30) + "\n";

{
    результат += `До объявления var: ${typeof hoistedVar}\n`;
    var hoistedVar = "поднята с undefined";
    результат += `После объявления var: ${hoistedVar}\n\n`;
}

// Пример 3: Блочная область видимости
результат += "Пример 3: Блочная область видимости\n";
результат += "=".repeat(30) + "\n";

if (true) {
    let внутриIf = "видна только в блоке";
    var утечка = "утекла из блока";
    результат += `Внутри if: ${внутриIf}\n`;
}

результат += "\nПосле if:\n";
try {
    результат += `    внутриIf: ${внутриIf}\n`;
} catch (e) {
    результат += `    внутриIf: ОШИБКА - ${e.message}\n`;
}
результат += `    утечка: ${утечка}`;

document.getElementById('result3').innerHTML =
    `<span class="success-message">✅ Код выполнен (с обработкой
ошибок)</span>\n\n` +
    `<strong>Результат:</strong>\n${результат}`;

} catch (error) {
    document.getElementById('result3').innerHTML =
        `<span class="error-message">× Необработанная ошибка: $
{error.message}</span>`;
}
}

function explainTDZ() {

```

```
const объяснение = `
```

```
<strong>⬜ Временная мертвая зона (Temporal Dead Zone)</strong>
```

```
<span class="highlight">TDZ – период от начала блока до объявления  
переменной</span>
```

```
1. <strong>Для let и const:</strong>
```

- Переменные "поднимаются" (hoisted)
- Но не инициализируются
- Доступ к ним вызывает ReferenceError

```
2. <strong>Для var:</strong>
```

- Переменные поднимаются И инициализируются как undefined
- Можно обращаться до объявления (получим undefined)

```
3. <strong>Практическое значение:</strong>
```

- Защищает от использования переменных до их объявления
- Делает код более предсказуемым
- Помогает находить ошибки на этапе разработки

```
4. <strong>Блочная область видимости:</strong>
```

- let/const видны только внутри блока {}
- var "утекает" из блоков
- Всегда используйте let/const для блочных переменных

```
<em>TDZ – это особенность языка, которая делает код безопаснее!</em>
```

```
`;
```

```
document.getElementById('result3').innerHTML = объяснение;  
}
```

```
function testErrors() {
```

```
  let результат = "\n⬜ Демонстрация ошибок TDZ:\n\n";
```

```
  результат += "1. TDZ с const:\n";
```

```
  try {
```

```
    {
```

```
      // console.log(tdzConst); // Ошибка!
```

```
      const tdzConst = "константа";
```

```
    }
```

```
    результат += "    Код не выполнялся бы с ошибкой\n";
```

```
  } catch (e) {
```

```
    результат += `    Ошибка: ${e.message}\n`;
```

```
  }
```

```

результат += "\n2. Циклическая зависимость:\n";
try {
  {
    // let a = b; // Ошибка!
    // let b = a;
  }
  результат += "    Взаимная зависимость в TDZ\n";
} catch (e) {
  результат += `    Ошибка: ${e.message}\n`;
}

результат += "\n3. typeof в TDZ:\n";
try {
  {
    // console.log(typeof tdzVar); // Ошибка!
    let tdzVar;
  }
  результат += "    Даже typeof вызывает ошибку в TDZ\n";
} catch (e) {
  результат += `    Ошибка: ${e.message}\n`;
}

результат += "\n📌 Вывод: Всегда объявляйте переменные ДО их использования!";

document.getElementById('result3').innerHTML +=
  ` \n\n<strong>📌 Тесты ошибок:</strong>\n${результат}`;
}

// Упражнение 4: Паттерны модулей
function runExercise4() {
  clearResults();

  let результат = "";
  try {
    // Выполняем код из упражнения
    const TaskManager = (function() {
      let tasks = [];
      let nextId = 1;

      function findTaskIndex(id) {
        return tasks.findIndex(task => task.id === id);
      }
    })
  }

```

```
function validateTask(task) {
  if (!task.title || task.title.trim() === '') {
    throw new Error('Задача должна иметь название');
  }
}

return {
  addTask: function(title, priority = 'medium') {
    validateTask({ title });

    const task = {
      id: nextId++,
      title,
      priority,
      completed: false,
      createdAt: new Date()
    };

    tasks.push(task);
    return task;
  },

  completeTask: function(id) {
    const index = findTaskIndex(id);

    if (index === -1) {
      throw new Error(`Задача с id ${id} не найдена`);
    }

    tasks[index].completed = true;
    return tasks[index];
  },

  getTasks: function(filter = 'all') {
    switch (filter) {
      case 'active':
        return tasks.filter(task => !task.completed);
      case 'completed':
        return tasks.filter(task => task.completed);
      default:
        return [...tasks];
    }
  },
}
```

```

    getStats: function() {
        return {
            total: tasks.length,
            completed: tasks.filter(t => t.completed).length,
            active: tasks.filter(t => !t.completed).length
        };
    }
};

})();

результат += "Тестирование TaskManager:\n";
результат += "=".repeat(30) + "\n\n";

const task1 = TaskManager.addTask("Изучить JavaScript", "high");
const task2 = TaskManager.addTask("Купить продукты");

результат += "Добавленные задачи:\n";
результат += JSON.stringify(TaskManager.getTasks(), null, 2) + "\n\n";

результат += "Статистика:\n";
результат += JSON.stringify(TaskManager.getStats(), null, 2) + "\n\n";

TaskManager.completeTask(task1.id);

результат += "После выполнения задачи:\n";
результат += "Активные задачи:\n";
результат += JSON.stringify(TaskManager.getTasks('active'), null, 2) + "\n\n";

результат += "Общая статистика:\n";
результат += JSON.stringify(TaskManager.getStats(), null, 2);

document.getElementById('result4').innerHTML =
    `<span class="success-message">☑ Модуль TaskManager работает!</span>\n\n` +
    `<strong>Результат:</strong>\n${результат}`;

} catch (error) {
    document.getElementById('result4').innerHTML =
        `<span class="error-message">✗ Ошибка: ${error.message}</span>`;
}
}

function explainModule() {
    const объяснение = `
<strong>☑ Паттерн модуля с IIFE (Immediately Invoked Function Expression)</strong>

```

>IIFE = функция, которая выполняется сразу после создания

1. Структура модуля:

```
<code>(function() { ... })()</code>
```

- Создает новую область видимости
- Выполняется сразу
- Возвращает публичный интерфейс

2. Приватные данные:

- tasks и nextId видны ТОЛЬКО внутри IIFE
- Невозможно получить к ним прямой доступ извне
- Защищены от случайного изменения

3. Публичный интерфейс:

- Возвращаемый объект с методами
- Методы имеют доступ к приватным данным (замыкание)
- Контролируемый доступ к функциональности

4. Преимущества:

- Инкапсуляция (скрытие реализации)
- Защита данных
- Организация кода
- Избегание конфликтов имен

Это классический паттерн для организации кода до появления ES6 модулей!
`;

```
document.getElementById('result4').innerHTML = объяснение;  
}
```

```
function testModule() {  
  let результат = "\n□   Дополнительные тесты модуля:\n\n";  
  
  // Пересоздаем модуль для тестов  
  const TestModule = (function() {  
    let data = "секретные данные";  
  
    return {  
      getData: () => data,  
      setData: (newData) => { data = newData; },  
      process: () => data.toUpperCase()  
    };  
  });
```

```

})();

результат += "1. Доступ к данным через методы:\n";
результат += `    getData(): ${TestModule.getData()}\n`;

TestModule.setData("новые данные");
результат += `    После setData(): ${TestModule.getData()}\n`;
результат += `    process(): ${TestModule.process()}\n\n`;

результат += "2. Попытка прямого доступа:\n";
результат += `    TestModule.data: ${TestModule.data}\n`;
результат += "    (undefined – данные защищены)\n\n";

результат += "3. Независимость модулей:\n";
const ModuleA = (function() { let x = 1; return { getX: () => x }; })();
const ModuleB = (function() { let x = 2; return { getX: () => x }; })();

результат += `    ModuleA.getX(): ${ModuleA.getX()}\n`;
результат += `    ModuleB.getX(): ${ModuleB.getX()}\n`;
результат += "    (разные x – разные области видимости)";

document.getElementById('result4').innerHTML +=
    ` \n\n<strong>□    Дополнительные тесты:</strong>\n${результат}`;
}

// Вспомогательные функции
function clearResults() {
    // Очищаем все панели результатов
    for (let i = 1; i <= 4; i++) {
        document.getElementById(`result${i}`).innerHTML = '';
        const scopePanel = document.getElementById(`scope${i}`);
        if (scopePanel) scopePanel.style.display = 'none';
    }
}

// Инициализация
document.addEventListener('DOMContentLoaded', () => {
    console.log("=== ТРЕНАЖЕР ОБЛАСТЕЙ ВИДИМОСТИ ===");
    console.log("Используйте кнопки для выполнения упражнений");
    console.log("Экспериментируйте с кодом в консоли браузера (F12)");
    console.log("=".repeat(80));

    // Добавляем обработчики для подсветки кода
    const codeEditors = document.querySelectorAll('.code-editor');

```



```

codeEditors.forEach(editor => {
  editor.addEventListener('click', function(e) {
    if (e.target.classList.contains('code-line')) {
      // Убираем подсветку у всех строк
      this.querySelectorAll('.code-line').forEach(line => {
        line.classList.remove('active');
      });
      // Подсвечиваем кликнутую строку
      e.target.classList.add('active');
    }
  });
});
});
</script>
</body>
</html>

```

Заключение

Ключевые выводы

1. Три типа областей видимости:

- ➔ **Глобальная:** Самый внешний уровень, переменные доступны везде
- ➔ **Функциональная:** Создается при вызове функции, `var` имеет эту область
- ➔ **Блочная:** Любой код между `{}`, `let` и `const` имеют эту область

2. Принцип лексической области видимости:

- ➔ Область определяется местом объявления в коде
- ➔ Внутренние функции видят переменные внешних функций
- ➔ Поиск переменных идет по цепочке областей видимости

3. Замыкания — мощный инструмент для:

- ➔ Создания приватных переменных
- ➔ Реализации модульного паттерна
- ➔ Сохранения состояния между вызовами функций

4. Временная мертвая зона (TDZ):

- ➔ Защищает от использования переменных до их объявления
- ➔ Применяется к `let` и `const`
- ➔ Делает код более предсказуемым

5. Строгий режим (use strict):

- ➔ Запрещает опасные конструкции (`with`, неявные глобальные переменные)
- ➔ Делает код более безопасным
- ➔ Улучшает производительность

Лучшие практики

1. **Всегда используйте `const` по умолчанию**, переходите к `let` только когда переменная должна изменяться
2. **Никогда не используйте `var`** в новом коде
3. **Изолируйте код** с помощью модулей, IIFE или блоков
4. **Объявляйте переменные как можно ближе** к месту их использования
5. **Избегайте загрязнения глобальной области** видимости
6. **Используйте замыкания осознанно** для создания инкапсулированного кода
7. **Включайте строгий режим** во всех новых проектах

Дальнейшие шаги

Освоив области видимости, вы готовы к изучению:

1. **Замыкания продвинутого уровня** — мемоизация, каррирование, функциональное программирование
2. **Модули ES6** — современная замена IIFE
3. **Асинхронный JavaScript** — как области видимости работают с промисами и `async/await`
4. **Прототипы и наследование** — как области видимости взаимодействуют с ООП
5. **Паттерны проектирования** — применение областей видимости в архитектуре приложений

Помните: глубокое понимание областей видимости — это фундамент для написания качественного, поддерживаемого и безопасного кода на JavaScript. Практикуйтесь, экспериментируйте с примерами из этой главы, и вскоре работа с областями видимости станет для вас интуитивной.

4.3. Прimitives типы данных: number, string, boolean, null, undefined, symbol, bigint

Введение: что такое примитивные типы данных

В JavaScript данные делятся на два основных вида:

1. **Примитивные типы** (primitive types) — неделимые "атомарные" значения
2. **Ссылочные типы** (reference types) — объекты, массивы, функции

Примитивные типы — фундаментальные строительные блоки языка. Они:

- Имеют фиксированный размер в памяти
- Неизменяемы (immutable)
- Сравниваются по значению (value comparison)
- Передаются по значению (passed by value)

Характеристики всех примитивных типов:

1. **Неизменяемость** — нельзя изменить значение, можно только создать новое
 2. **Не имеют методов** (хотя у некоторых есть "обёртки")
 3. **Хранятся в стеке** (stack memory) — быстрый доступ
 4. **Легковесные** — занимают мало памяти
-

1. Number (числа)

1.1. Что такое тип Number

`Number` представляет как целые, так и дробные числа. В JavaScript нет разделения на `int`, `float`, `double` — всё это `Number`.

```
javascript
```

```
// Различные формы записи чисел
```

```
let integer = 42;           // Целое число
let decimal = 3.14;         // Дробное число
let negative = -15;         // Отрицательное число
```

```
let scientific = 2.5e6;    //  $2.5 \times 10^6 = 2,500,000$ 
```

1.2. Специальные числовые значения

JavaScript имеет несколько специальных числовых значений:

```
javascript

// Специальные значения типа Number
let infinity = Infinity;    // Бесконечность
let negativeInfinity = -Infinity; // Отрицательная бесконечность
let notANumber = NaN;      // "Not a Number" (результат недопустимой операции)

console.log(typeof Infinity); // "number"
console.log(typeof NaN);      // "number" (парадокс!)
```

1.3. Системы счисления

JavaScript поддерживает различные системы счисления:

```
javascript

// Десятичная (по умолчанию)
let decimal = 100;    // 100

// Двоичная (начинается с 0b или 0B)
let binary = 0b1100100; // 100 (в десятичной)
console.log(binary);    // 100

// Восьмеричная (начинается с 0o или 0O)
let octal = 0o144;    // 100 (в десятичной)

// Шестнадцатеричная (начинается с 0x или 0X)
let hex = 0x64;    // 100 (в десятичной)
let color = 0xFF5733; // Часто используется для цветов
```

1.4. Методы работы с числами

Хотя `Number` — примитивный тип, у него есть "обёртка" с методами:

```
javascript

// Преобразование чисел в строки
```

```

let num = 123.456;

console.log(num.toString());           // "123.456"
console.log(num.toFixed(2));           // "123.46" (округление до 2 знаков)
console.log(num.toExponential(2));     // "1.23e+2" (экспоненциальная запись)
console.log(num.toPrecision(4));       // "123.5" (общая точность 4 знака)

// Преобразование строк в числа
console.log(parseInt("42px"));         // 42 (извлекает целое число)
console.log(parseFloat("3.14em"));     // 3.14 (извлекает дробное число)
console.log(Number("123"));            // 123 (строгое преобразование)

```

1.5. Проверка чисел

javascript

```

// Проверка на число
let value = 42;

console.log(typeof value === 'number'); // true
console.log(!isNaN(value));             // true (НЕ "не число")
console.log(Number.isFinite(value));     // true (конечное число)
console.log(Number.isInteger(value));    // true (целое число)

// Опасность глобального isNaN()
console.log(isNaN("текст"));             // true (ожидаемо)
console.log(isNaN("123"));              // false (строку преобразует в число!)

// Безопасная проверка
console.log(Number.isNaN("текст"));      // false (строгая проверка)

```

1.6. Особенности чисел в JavaScript

javascript

```

// Проблемы точности дробных чисел (из-за двоичного представления)
console.log(0.1 + 0.2 === 0.3);        // false!
console.log(0.1 + 0.2);                 // 0.30000000000000004

// Решение – использовать допустимую погрешность (epsilon)
function areNumbersEqual(a, b) {
    return Math.abs(a - b) < Number.EPSILON;
}
console.log(areNumbersEqual(0.1 + 0.2, 0.3)); // true

```

```
// Максимальные и минимальные значения
console.log(Number.MAX_VALUE);           // 1.7976931348623157e+308
console.log(Number.MIN_VALUE);           // 5e-324 (минимальное положительное)
console.log(Number.MAX_SAFE_INTEGER);    // 9007199254740991
console.log(Number.MIN_SAFE_INTEGER);    // -9007199254740991

// Проверка безопасного целого числа
console.log(Number.isSafeInteger(9007199254740991)); // true
console.log(Number.isSafeInteger(9007199254740992)); // false!
```

2. String (строки)

2.1. Что такое тип String

String представляет последовательность символов для работы с текстом.

javascript

```
// Различные способы создания строк
let singleQuotes = 'Текст в одинарных кавычках';
let doubleQuotes = "Текст в двойных кавычках";
let backticks = `Текст в обратных кавычках (template literals)`;

// Экранирование символов
let escaped = "Он сказал: \"Привет!\""; // Экранирование кавычек
let newLine = "Первая строка\nВторая строка"; // Символ новой строки
let tab = "Колонка1\tКолонка2"; // Табуляция
```

2.2. Шаблонные строки (Template Literals)

ES6 ввёл шаблонные строки — мощное улучшение для работы с текстом:

javascript

```
// Интерполяция переменных
let name = "Анна";
let age = 25;
let greeting = `Привет, ${name}! Тебе ${age} лет.`;
console.log(greeting); // "Привет, Анна! Тебе 25 лет."
```

```

// Многострочные строки
let multiLine = `
  Это
  многострочная
  строка
`;
console.log(multiLine);

// Выражения внутри шаблонов
let a = 5, b = 10;
console.log(`Сумма: ${a + b}`);           // "Сумма: 15"
console.log(`Результат: ${a > b ? 'больше' : 'меньше или равно'}`);

// Тегированные шаблоны (tagged templates)
function highlight(strings, ...values) {
  return strings.reduce((result, str, i) => {
    return result + str + (values[i] ? `<mark>${values[i]}</mark>` : '');
  }, '');
}

let user = "Иван";
let score = 95;
console.log(highlight`Пользователь ${user} набрал ${score} баллов`);
// "Пользователь <mark>Иван</mark> набрал <mark>95</mark> баллов"

```

2.3. Основные методы строк

Строки имеют множество полезных методов (напомним: строки неизменяемы, методы возвращают новые строки):

```

javascript

let text = "  JavaScript - это интересно!  ";

// Длина строки
console.log(text.length);           // 30 (включая пробелы)

// Обрезка пробелов
console.log(text.trim());           // "JavaScript - это интересно!"
console.log(text.trimStart());      // "JavaScript - это интересно!  "
console.log(text.trimEnd());        // "  JavaScript - это интересно!"

```

```

// Регистр символов
console.log(text.toUpperCase());           // "  JAVASCRIPT - ЭТО ИНТЕРЕСНО!  "
console.log(text.toLowerCase());          // "  javascript - это интересно!  "

// Поиск и замена
console.log(text.includes("JavaScript")); // true
console.log(text.indexOf("это"));         // 15
console.log(text.lastIndexOf(" "));       // 26
console.log(text.replace("интересно", "круто")); // "  JavaScript - это круто!  "
console.log(text.replaceAll(" ", "_"));    // "__JavaScript_-_это_интересно!__"

// Извлечение подстрок
console.log(text.slice(2, 12));            // "JavaScript"
console.log(text.substring(2, 12));        // "JavaScript"
console.log(text.substr(2, 10));           // "JavaScript" (устаревший)

// Разделение и соединение
let words = text.trim().split(" ");
console.log(words);                       // ["JavaScript", "-", "это", "интересно!"]
console.log(words.join("_"));             // "JavaScript_-_это_интересно!"

// Проверки
console.log(text.startsWith("  Java"));   // true
console.log(text.endsWith("!  "));        // true
console.log("123".repeat(3));             // "123123123"

```

2.4. Символы и Unicode

javascript

```

// Доступ к символам
let str = "Привет";
console.log(str[0]);           // "П"
console.log(str.charAt(0));    // "П"
console.log(str.charCodeAt(0)); // 1055 (код символа в UTF-16)
console.log(str.codePointAt(0)); // 1055 (для полного Unicode)

// Итерация по символам
for (let char of str) {
  console.log(char);           // "П", "р", "и", "в", "е", "т"
}

// Строки и Unicode
let emoji = "👍 ";

```



```
console.log(emoji.length);           // 2! (суррогатная пара)
console.log([...emoji].length);       // 1 (правильный подсчёт)
console.log(Array.from(emoji).length); // 1 (ещё один способ)

// Нормализация Unicode
let acute1 = "café";
let acute2 = "cafe\u0301";           // e + acute accent
console.log(acute1 === acute2);      // false!
console.log(acute1.normalize() === acute2.normalize()); // true
```

3. Boolean (логический тип)

3.1. Что такое тип Boolean

Boolean представляет логические значения: **true** (истина) или **false** (ложь).

javascript

```
let isActive = true;
let isCompleted = false;

console.log(typeof isActive);           // "boolean"
```

3.2. Логические операторы

javascript

```
// Логическое И (AND) - &&
console.log(true && true);           // true
console.log(true && false);          // false
console.log(false && true);           // false (короткое замыкание)
console.log(false && false);          // false

// Логическое ИЛИ (OR) - ||
console.log(true || true);           // true
console.log(true || false);          // true
console.log(false || true);           // true (короткое замыкание)
console.log(false || false);          // false

// Логическое НЕ (NOT) - !
```

```

console.log(!true);           // false
console.log(!false);          // true
console.log(!!"текст");       // true (двойное отрицание для
// преобразования)

// Комбинирование операторов
console.log((true && false) || true); // true
console.log(!(true && false));         // true

```

3.3. Преобразование к Boolean

Любое значение в JavaScript можно преобразовать к логическому типу:

```

javascript

// Явное преобразование
console.log(Boolean("текст")); // true
console.log(Boolean(""));      // false
console.log(Boolean(42));      // true
console.log(Boolean(0));       // false
console.log(Boolean({}));      // true (объект всегда true)
console.log(Boolean(null));    // false
console.log(Boolean(undefined)); // false

// Неявное преобразование (в логическом контексте)
if ("текст") {                 // true
    console.log("Это выполнится");
}

if (0) {                       // false
    console.log("Это не выполнится");
}

// Логические операторы с не-boolean значениями
console.log("текст" || "значение по умолчанию"); // "текст"
console.log("" || "значение по умолчанию");      // "значение по умолчанию"
console.log("текст" && "другое значение");       // "другое значение"
console.log(null && "другое значение");          // null

```

3.4. "Ложные" (falsy) значения

В JavaScript есть 8 значений, которые преобразуются в `false`:

javascript

```
// Все falsy значения:
console.log(Boolean(false)); // false
console.log(Boolean(0)); // false
console.log(Boolean(-0)); // false
console.log(Boolean(0n)); // false (BigInt)
console.log(Boolean("")); // false
console.log(Boolean(null)); // false
console.log(Boolean(undefined)); // false
console.log(Boolean(NaN)); // false

// Всё остальное – truthy
console.log(Boolean("0")); // true (строка!)
console.log(Boolean("false")); // true (строка!)
console.log(Boolean([])); // true (пустой массив)
console.log(Boolean({})); // true (пустой объект)
console.log(Boolean(function(){})); // true (функция)
```

3.5. Логические операторы с присваиванием (ES12)

javascript

```
// Оператор логического ИЛИ с присваиванием (||=)
let a = null;
a ||= "значение по умолчанию";
console.log(a); // "значение по умолчанию"

// Оператор логического И с присваиванием (&&=)
let b = "текст";
b &&= "новое значение";
console.log(b); // "новое значение"

// Оператор нулевого слияния с присваиванием (??=)
let c = null;
c ??= "запасное значение";
console.log(c); // "запасное значение"
```

4. Null

4.1. Что такое null

`null` — специальное значение, которое означает "ничего", "пусто" или "значение неизвестно".

```
javascript

let emptyValue = null;
console.log(typeof emptyValue);           // "object" (историческая особенность!)
console.log(emptyValue === null);         // true
```

4.2. Использование null

```
javascript

// Явное указание на отсутствие значения
let user = {
  name: "Иван",
  middleName: null,           // Отчество отсутствует
  lastName: "Петров"
};

// Сброс значения переменной
let data = getDataFromAPI();
// ... работа с data ...

data = null;                  // Освобождаем ссылку для сборщика мусора

// Проверка на null
function processValue(value) {
  if (value === null) {
    console.log("Значение не указано");
    return;
  }
  console.log("Обрабатываем:", value);
}

processValue(null);           // "Значение не указано"
processValue("текст");        // "Обрабатываем: текст"
```

4.3. Null vs Undefined

```
javascript

// Различия между null и undefined
let a;                                     // undefined (объявлена, но не
инициализирована)
let b = null;                             // null (явно установлено в "пусто")

console.log(typeof a);                    // "undefined"
console.log(typeof b);                    // "object" (особенность языка)

console.log(a == null);                   // true (нестрогое сравнение)
console.log(a === null);                  // false (строгое сравнение)
console.log(b == undefined);              // true (нестрогое сравнение)
console.log(b === undefined);             // false (строгое сравнение)

// Практическое правило:
// - Используйте undefined для "значение не задано"
// - Используйте null для "значение отсутствует намеренно"
```

5. Undefined

5.1. Что такое undefined

`undefined` означает, что переменная была объявлена, но ей не присвоено значение.

```
javascript

// Неинициализированные переменные
let x;                                     // undefined
console.log(x);                           // undefined
console.log(typeof x);                     // "undefined"

// Несуществующие свойства объектов
let obj = { name: "Иван" };
console.log(obj.age);                      // undefined

// Функция без return
function noReturn() {
```

```
// нет return  
}  
console.log(noReturn()); // undefined
```

5.2. Источники undefined

javascript

// Различные способы получения undefined

// 1. Неинициализированные переменные

```
let a;  
console.log(a); // undefined
```

// 2. Несуществующие свойства

```
let obj = { x: 1 };  
console.log(obj.y); // undefined
```

// 3. Массивы за пределами длины

```
let arr = [1, 2, 3];  
console.log(arr[5]); // undefined
```

// 4. Пропущенные параметры функций

```
function greet(name) {  
  console.log("Привет, " + name);  
}  
greet(); // "Привет, undefined"
```

// 5. Явное присваивание undefined

```
let value = undefined; // Не рекомендуется!
```

// 6. Результат void выражения

```
console.log(void 0); // undefined  
console.log(void (2 + 2)); // undefined
```

5.3. Работа с undefined

javascript

// Проверка на undefined

```
let value;  
  
console.log(value === undefined); // true  
console.log(typeof value === 'undefined'); // true
```

```
console.log(value == null); // true (нестрогое сравнение с null)

// Безопасный доступ к свойствам
let user = {
  profile: {
    name: "Анна"
  }
};

// Старый способ (опасный)
let city = user.profile.address.city; // TypeError!

// Современные способы
let safeCity = user?.profile?.address?.city; // undefined (опциональная цепочка)
let safeCity2 = user && user.profile && user.profile.address &&
user.profile.address.city;
```

5.4. Параметры функций и undefined

javascript

```
// Параметры по умолчанию (ES6)
function createUser(name = "Гость", age = 18) {
  return { name, age };
}

console.log(createUser()); // {name: "Гость", age: 18}
console.log(createUser("Иван")); // {name: "Иван", age: 18}
console.log(createUser("Иван", undefined)); // {name: "Иван", age: 18}

// Проверка на undefined в теле функции
function process(data) {
  if (data === undefined) {
    data = "значение по умолчанию";
  }
  // Или более кратко:
  data = data || "значение по умолчанию";
  // Или с нулевым слиянием:
  data = data ?? "значение по умолчанию";
}
```

6. Symbol (символы)

6.1. Что такое Symbol

`Symbol` — примитивный тип, введённый в ES6, представляющий уникальные идентификаторы.

```
javascript

// Создание символов
let id = Symbol();
let id2 = Symbol("id");
let id3 = Symbol("id");

// Описание для отладки
// Другой символ с тем же описанием

console.log(id2 === id3);
console.log(id2.toString());
console.log(id2.description);

// false! Каждый Symbol уникален
// "Symbol(id)"
// "id"
```

6.2. Уникальность символов

```
javascript

// Каждый Symbol уникален
let sym1 = Symbol("ключ");
let sym2 = Symbol("ключ");

console.log(sym1 === sym2);
console.log(sym1 == sym2);

// false
// false

// Сравнение с другими типами
console.log(Symbol() === Symbol());
console.log(Symbol("id") === Symbol("id"));
console.log(typeof Symbol());

// false
// false
// "symbol"
```

6.3. Использование символов как ключей объектов

```
javascript

// Символы как уникальные ключи свойств
let user = {
  name: "Иван",
  age: 30
};
```



```

let id = Symbol("id");
user[id] = 12345;

// Доступ к свойству-символу
console.log(user[id]);           // 12345
console.log(user["id"]);        // undefined (обычное свойство)

// Символы не видны в for...in
for (let key in user) {
  console.log(key);             // "name", "age" (но не Symbol(id))
}

// Символы видны в Object.getOwnPropertySymbols
let symbols = Object.getOwnPropertySymbols(user);
console.log(symbols);           // [Symbol(id)]
console.log(user[symbols[0]]);  // 12345

// JSON.stringify игнорирует символы
console.log(JSON.stringify(user)); // {"name": "Иван", "age": 30}

```

6.4. Глобальные символы (Symbol.for)

javascript

```

// Глобальный реестр символов
let globalSym = Symbol.for("app.id"); // Создаёт или возвращает существующий
let sameGlobalSym = Symbol.for("app.id"); // Возвращает тот же символ

console.log(globalSym === sameGlobalSym); // true

// Получение ключа по символу
console.log(Symbol.keyFor(globalSym)); // "app.id"

let localSym = Symbol("local");
console.log(Symbol.keyFor(localSym)); // undefined (не глобальный)

```

6.5. Известные символы (well-known symbols)

JavaScript имеет встроенные символы для метапрограммирования:

javascript

```

// Symbol.iterator - делает объект итерируемым

```

```

let range = {
  from: 1,
  to: 3,
  [Symbol.iterator]() {
    let current = this.from;
    return {
      next: () => {
        if (current <= this.to) {
          return { value: current++, done: false };
        } else {
          return { done: true };
        }
      }
    };
  }
};

for (let num of range) {
  console.log(num);           // 1, 2, 3
}

// Symbol.toStringTag
let customObject = {
  [Symbol.toStringTag]: "МойОбъект"
};
console.log({}.toString.call(customObject)); // "[object МойОбъект]"

// Symbol.toPrimitive
let calculator = {
  value: 1,
  [Symbol.toPrimitive](hint) {
    console.log("hint:", hint);
    return hint === "string" ? `Значение: ${this.value}` : this.value;
  }
};

console.log(String(calculator));           // "Значение: 1"
console.log(calculator + 2);               // 3

```

7. BigInt

7.1. Что такое BigInt

BigInt — примитивный тип, введённый в ES2020, для работы с целыми числами произвольной точности.

```
javascript

// Создание BigInt
let bigInt = 1234567890123456789012345678901234567890n; // Суффикс n
let bigInt2 = BigInt("123456789012345678901234567890"); // Конструктор
let bigInt3 = BigInt(9007199254740991); // Из обычного числа

console.log(typeof bigInt); // "bigint"
```

7.2. Зачем нужен BigInt

```
javascript

// Проблема с обычными числами
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
console.log(Number.MAX_SAFE_INTEGER + 1); // 9007199254740992 (правильно)
console.log(Number.MAX_SAFE_INTEGER + 2); // 9007199254740992 (неправильно!)

// Решение с BigInt
let maxSafe = BigInt(Number.MAX_SAFE_INTEGER);
console.log(maxSafe + 1n); // 9007199254740992n
console.log(maxSafe + 2n); // 9007199254740993n (правильно!)
```

7.3. Операции с BigInt

```
javascript

// Арифметические операции
let a = 12345678901234567890n;
let b = 98765432109876543210n;

console.log(a + b); // 111111111011111111100n
console.log(b - a); // 86419753208641975320n
console.log(a * 2n); // 24691357802469135780n
console.log(b / 2n); // 49382716054938271605n
console.log(a % 10n); // 0n
```

```

// Сравнение
console.log(a < b); // true
console.log(a === 12345678901234567890n); // true
console.log(a == 12345678901234567890); // true (нестрогое сравнение)

// Ограничения
console.log(Math.sqrt(16n)); // TypeError! Math не работает с BigInt
console.log(2n ** 100n); // 1267650600228229401496703205376n

```

7.4. Преобразование BigInt

javascript

```

// BigInt в строку
let big = 12345678901234567890n;
console.log(big.toString()); // "12345678901234567890"
console.log(String(big)); // "12345678901234567890"
console.log(big + ""); // "12345678901234567890"

// BigInt в число (с осторожностью!)
console.log(Number(big)); // 1.2345678901234568e+19 (потеря
точности)
console.log(Number(9007199254740991n)); // 9007199254740991 (без потери)

// Проверка потери точности
function bigIntToNumber(bigInt) {
  if (bigInt > Number.MAX_SAFE_INTEGER || bigInt < Number.MIN_SAFE_INTEGER) {
    console.warn("Потеря точности при преобразовании BigInt в Number");
  }
  return Number(bigInt);
}

```

7.5. Смешивание BigInt и Number

javascript

```

// BigInt и Number нельзя смешивать
console.log(1n + 2); // TypeError!
console.log(1n * 2); // TypeError!

// Решение – явное преобразование
console.log(1n + BigInt(2)); // 3n
console.log(Number(1n) + 2); // 3

```

```
// Исключение: сравнение
console.log(1n == 1);           // true (нестрогое сравнение)
console.log(1n === 1);         // false (строгое сравнение)
console.log(1n < 2);           // true
console.log(1n > 0);           // true
```

Сравнение всех примитивных типов

javascript

```
// Сводная таблица примитивных типов
```

```
const primitives = {
  number: 42,
  string: "текст",
  boolean: true,
  null: null,
  undefined: undefined,
  symbol: Symbol("id"),
  bigint: 9007199254740991n
};
```

```
// Проверка типов
```

```
for (let [key, value] of Object.entries(primitives)) {
  console.log(`${key}: ${value} (${typeof value})`);
}
```

```
// Преобразование между типами
```

```
let num = 42;
console.log(String(num));           // "42"
console.log(Boolean(num));         // true
console.log(BigInt(num));          // 42n

let str = "123";
console.log(Number(str));           // 123
console.log(Boolean(str));         // true
console.log(BigInt(str));          // 123n
```

Лучшие практики работы с примитивными типами

1. Для чисел:

javascript

```
// Используйте Number.isSafeInteger() для проверки больших чисел
// Используйте Math.floor() вместо parseInt() для преобразования
// Осторожно с дробными числами – используйте toFixed() или библиотеки
```

2. Для строк:

javascript

```
// Используйте шаблонные строки для интерполяции
// Используйте метод includes() вместо indexOf() для проверки наличия подстроки
// Помните о неизменяемости строк
```

3. Для boolean:

javascript

```
// Избегайте двойного отрицания (!!) без необходимости
// Используйте Boolean() для явного преобразования
// Помните о falsy значениях
```

4. Для null/undefined:

javascript

```
// Используйте оператор нулевого слияния (??) для значений по умолчанию
// Используйте опциональную цепочку (?.) для безопасного доступа
// Строго различайте null и undefined
```

5. Для Symbol:

javascript

```
// Используйте для создания уникальных ключей
// Используйте Symbol.for() для глобальных символов
// Используйте известные символы для метапрограммирования
```

6. Для BigInt:

javascript

// Используйте для чисел больше Number.MAX_SAFE_INTEGER

// Не смешивайте с Number без явного преобразования

// Помните об ограничениях (Math, JSON.stringify)

Заключение

Примитивные типы — фундамент JavaScript. Понимание их особенностей, ограничений и правил преобразования критически важно для написания надёжного и эффективного кода. Каждый тип имеет свою нишу:

- **Number** — для математических вычислений (с осторожностью к точности)
- **String** — для работы с текстом (неизменяемы, богатый API)
- **Boolean** — для логических операций (помните о truthy/falsy)
- **Null** — для явного указания "пустого" значения
- **Undefined** — для неинициализированных переменных
- **Symbol** — для уникальных идентификаторов и метапрограммирования
- **BigInt** — для очень больших целых чисел

Правильный выбор типа данных — первый шаг к качественному коду. Помните о неизменяемости примитивных типов, особенностях сравнения и преобразования между типами. Эти знания помогут избежать многих распространённых ошибок в JavaScript-разработке.

4.4. Ссылочные типы: объекты, массивы, функции

Введение: что такое ссылочные типы

В отличие от примитивных типов, ссылочные типы (reference types) представляют собой сложные структуры данных, которые:

1. **Хранятся в куче (heap)** — динамическая память
 2. **Передаются по ссылке** — копируется только ссылка, а не само значение
 3. **Изменяемы (mutable)** — можно изменять содержимое
 4. **Имеют методы и свойства** — расширенная функциональность
 5. **Сравниваются по ссылке** — две переменные равны только если ссылаются на один объект
-

1. Объекты (Objects)

1.1. Что такое объекты в JavaScript

Объекты — это коллекции пар "ключ-значение", где ключи — строки (или Symbol), а значения могут быть любого типа.

```
javascript
```

```
// Базовый синтаксис создания объектов
```

```
let user = {  
  // Свойства (properties)  
  name: "Иван",  
  age: 30,  
  "email address": "ivan@example.com", // Ключ в кавычках (если содержит пробелы)
```

```
// Методы (methods) — функции как свойства
```

```
sayHi: function() {  
  console.log(`Привет, я ${this.name}!`);  
},
```

```
// Сокращенная запись метода (ES6)
```

```
introduce() {
```



```

    return `Меня зовут ${this.name}, мне ${this.age} лет`;
  }
};

// Доступ к свойствам
console.log(user.name);           // "Иван" — точечная нотация
console.log(user["age"]);         // 30 — скобочная нотация
console.log(user["email address"]); // "ivan@example.com" — обязательно скобки

// Вызов методов
user.sayHi();                     // "Привет, я Иван!"
console.log(user.introduce());    // "Меня зовут Иван, мне 30 лет"

```

1.2. Динамическая природа объектов

javascript

```

// Объекты можно динамически изменять
let car = { brand: "Toyota" };

// Добавление свойств
car.model = "Camry";
car.year = 2022;
car["color"] = "silver";

console.log(car); // {brand: "Toyota", model: "Camry", year: 2022, color: "silver"}

// Изменение свойств
car.year = 2023;
car.color = "black";

// Удаление свойств
delete car.model;
console.log(car); // {brand: "Toyota", year: 2023, color: "black"}

// Проверка существования свойства
console.log("brand" in car);           // true
console.log(car.hasOwnProperty("model")); // false
console.log(car.weight === undefined); // true (но свойство могло быть установлено в undefined)

```

1.3. Ссылочная природа объектов

javascript

```

// Две переменные могут ссылаться на один объект
let obj1 = { value: 10 };
let obj2 = obj1; // Копируется ссылка, а не объект

console.log(obj1 === obj2); // true (одна и та же ссылка)

obj2.value = 20;
console.log(obj1.value); // 20 (изменился исходный объект!)

// Создание независимой копии
let obj3 = { ...obj1 }; // Поверхностное копирование (spread operator)
let obj4 = Object.assign({}, obj1); // Альтернативный способ
let obj5 = JSON.parse(JSON.stringify(obj1)); // Глубокое копирование (с ограничениями)

obj3.value = 30;
console.log(obj1.value); // 20 (не изменился)
console.log(obj3 === obj1); // false (разные объекты)

```

1.4. Специальные методы объектов

javascript

```

let person = {
  name: "Анна",
  age: 25,
  job: "developer"
};

// Object.keys() – массив ключей
console.log(Object.keys(person)); // ["name", "age", "job"]

// Object.values() – массив значений (ES8)
console.log(Object.values(person)); // ["Анна", 25, "developer"]

// Object.entries() – массив пар [ключ, значение] (ES8)
console.log(Object.entries(person));
// [["name", "Анна"], ["age", 25], ["job", "developer"]]

// Object.fromEntries() – объект из массива пар (ES10)
let entries = [["a", 1], ["b", 2]];
console.log(Object.fromEntries(entries)); // {a: 1, b: 2}

// Object.assign() – копирование свойств
let target = { a: 1 };

```

```

let source = { b: 2, c: 3 };
Object.assign(target, source);
console.log(target);                                // {a: 1, b: 2, c: 3}

// Object.freeze() – делает объект неизменяемым
let frozen = Object.freeze({ x: 1, y: 2 });
frozen.x = 10;                                     // Не работает (в строгом режиме – ошибка)
console.log(frozen.x);                             // 1

// Object.seal() – можно изменять свойства, но нельзя добавлять/удалять
let sealed = Object.seal({ x: 1, y: 2 });
sealed.x = 10;                                     // Работает
sealed.z = 3;                                     // Не работает
delete sealed.y;                                   // Не работает

```

1.5. Вычисляемые имена свойств (ES6)

javascript

```

// Динамическое создание ключей
let propName = "status";
let dynamicKey = "user_" + propName;

let user = {
  name: "Борис",
  [propName]: "active",                // status: "active"
  [dynamicKey]: "user_active",        // user_status: "user_active"
  ["full" + "Name"]: "Борис Петров"  // fullName: "Борис Петров"
};

console.log(user.status);              // "active"
console.log(user.user_status);        // "user_active"

// Использование в методах
let methodName = "getData";
let api = {
  [methodName]() {
    return "данные";
  }
};

console.log(api.getData());            // "данные"

```

1.6. Геттеры и сеттеры

javascript

```
let user = {
  firstName: "Иван",
  lastName: "Петров",

  // Геттер для полного имени
  get fullName() {
    return `${this.firstName} ${this.lastName}`;
  },

  // Сеттер для полного имени
  set fullName(value) {
    [this.firstName, this.lastName] = value.split(" ");
  },

  // Геттер с вычисляемым значением
  get ageDescription() {
    return `${this.age} лет`;
  }
};

// Добавляем возраст отдельно
user.age = 30;

console.log(user.fullName);           // "Иван Петров" (вызывается геттер)
user.fullName = "Анна Сидорова";     // Вызывается сеттер
console.log(user.firstName);          // "Анна"
console.log(user.lastName);           // "Сидорова"
console.log(user.ageDescription);     // "30 лет"

// Использование Object.defineProperty
let book = { title: "1984" };
Object.defineProperty(book, "author", {
  get() { return "Джордж Оруэлл"; },
  enumerable: true,                  // Видимо в for...in
  configurable: true                 // Можно удалить или переопределить
});

console.log(book.author);             // "Джордж Оруэлл"
```

1.7. Прототипы и наследование

javascript

// Базовый объект-прототип

```
let animal = {  
  eats: true,  
  walk() {  
    console.log("Животное идет");  
  }  
};
```

// Создание объекта с указанным прототипом

```
let rabbit = Object.create(animal, {  
  jumps: {  
    value: true,  
    writable: true,  
    enumerable: true  
  }  
});
```

```
console.log(rabbit.eats);           // true (из прототипа)  
console.log(rabbit.jumps);         // true (собственное свойство)  
rabbit.walk();                     // "Животное идет" (из прототипа)
```

// Проверка прототипа

```
console.log(Object.getPrototypeOf(rabbit) === animal); // true  
console.log(animal.isPrototypeOf(rabbit)); // true  
console.log(rabbit instanceof Object); // true
```

// Цепочка прототипов

```
let longEar = Object.create(rabbit, {  
  earLength: { value: 10 }  
});
```

```
console.log(longEar.jumps);         // true (из rabbit)  
console.log(longEar.eats);         // true (из animal через rabbit)
```

1.8. Итерация по объектам

javascript

```
let person = {  
  name: "Мария",  
  age: 28,
```

```
    job: "дизайнер",
    city: "Москва"
};

// for...in — перебирает все перечисляемые свойства (включая унаследованные)
for (let key in person) {
    if (person.hasOwnProperty(key)) { // Проверка на собственное свойство
        console.log(`${key}: ${person[key]}`);
    }
}

// Object.keys + for...of (ES6)
for (let key of Object.keys(person)) {
    console.log(key, person[key]);
}

// Object.entries + for...of (ES8)
for (let [key, value] of Object.entries(person)) {
    console.log(`${key} => ${value}`);
}

// Object.values (ES8)
for (let value of Object.values(person)) {
    console.log(value);
}
```

2. Массивы (Arrays)

2.1. Что такое массивы в JavaScript

Массивы — это упорядоченные коллекции элементов с числовыми индексами. В JavaScript массивы являются объектами специального типа.

```
javascript

// Создание массивов
let fruits = ["Яблоко", "Апельсин", "Банан"]; // Литерал массива
let numbers = new Array(1, 2, 3, 4, 5); // Конструктор Array
let empty = []; // Пустой массив
let mixed = [1, "текст", true, null, {x: 1}]; // Разные типы данных
```

```

// Доступ к элементам
console.log(fruits[0]);           // "Яблоко" (индексация с 0)
console.log(fruits[1]);           // "Апельсин"
console.log(fruits.length);       // 3 (длина массива)

// Изменение элементов
fruits[1] = "Груша";
console.log(fruits);              // ["Яблоко", "Груша", "Банан"]

// Добавление элементов
fruits[3] = "Апельсин";           // По индексу
fruits.push("Манго");             // В конец
fruits.unshift("Киви");           // В начало

console.log(fruits);              // ["Киви", "Яблоко", "Груша", "Банан",
"Апельсин", "Манго"]
console.log(fruits.length);       // 6

```

2.2. Основные методы массивов

Добавление/удаление элементов:

```

javascript

let arr = ["a", "b", "c"];

// push/pop – работают с концом массива
arr.push("d");                    // ["a", "b", "c", "d"]
let last = arr.pop();             // "d", arr = ["a", "b", "c"]

// unshift/shift – работают с началом массива
arr.unshift("z");                 // ["z", "a", "b", "c"]
let first = arr.shift();          // "z", arr = ["a", "b", "c"]

// splice – универсальный метод
arr.splice(1, 1);                 // Удалить 1 элемент с позиции 1
console.log(arr);                 // ["a", "c"]

arr.splice(1, 0, "b1", "b2");     // Вставить элементы
console.log(arr);                 // ["a", "b1", "b2", "c"]

arr.splice(2, 1, "b3");           // Заменить элемент

```

```
console.log(arr); // ["a", "b1", "b3", "c"]
```

Поиск элементов:

javascript

```
let numbers = [1, 2, 3, 4, 5, 2, 7];
```

// indexOf/lastIndexOf

```
console.log(numbers.indexOf(2)); // 1 (первое вхождение)
console.log(numbers.lastIndexOf(2)); // 5 (последнее вхождение)
console.log(numbers.indexOf(10)); // -1 (не найден)
```

// includes (ES7)

```
console.log(numbers.includes(3)); // true
console.log(numbers.includes(3, 3)); // false (поиск с позиции 3)
```

// find/findIndex (ES6)

```
let users = [
  {id: 1, name: "Иван"},
  {id: 2, name: "Петр"},
  {id: 3, name: "Анна"}
];
```

```
let user = users.find(item => item.id === 2);
console.log(user); // {id: 2, name: "Петр"}
```

```
let index = users.findIndex(item => item.name === "Анна");
console.log(index); // 2
```

// findLast/findLastIndex (ES2023)

```
let lastEven = numbers.findLast(n => n % 2 === 0);
console.log(lastEven); // 2 (последнее четное)
```

Преобразование массивов:

javascript

```
let nums = [1, 2, 3, 4, 5];
```

// map – преобразование каждого элемента

```
let doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
```

// filter – фильтрация

```
let evens = nums.filter(n => n % 2 === 0);
```



```

console.log(evens); // [2, 4]

// reduce/reduceRight – аккумуляция
let sum = nums.reduce((acc, n) => acc + n, 0);
console.log(sum); // 15

let product = nums.reduce((acc, n) => acc * n, 1);
console.log(product); // 120

// flat – "выравнивание" массивов (ES2019)
let nested = [1, [2, [3, [4]]]];
console.log(nested.flat()); // [1, 2, [3, [4]]]
console.log(nested.flat(2)); // [1, 2, 3, [4]]
console.log(nested.flat(Infinity)); // [1, 2, 3, 4]

// flatMap – map + flat (ES2019)
let phrases = ["hello world", "good morning"];
let words = phrases.flatMap(phrase => phrase.split(" "));
console.log(words); // ["hello", "world", "good", "morning"]

```

Сортировка:

```

javascript

let arr = [5, 2, 8, 1, 9];

// sort – мутирует исходный массив!
arr.sort((a, b) => a - b); // Для чисел
console.log(arr); // [1, 2, 5, 8, 9]

arr.sort((a, b) => b - a); // Обратный порядок
console.log(arr); // [9, 8, 5, 2, 1]

// Для строк
let names = ["Иван", "Анна", "Петр", "анна"];
names.sort((a, b) => a.localeCompare(b)); // Учет локали
console.log(names); // ["Анна", "анна", "Иван", "Петр"]

// reverse
arr.reverse();
console.log(arr); // [1, 2, 5, 8, 9]

```

2.3. Деструктуризация массивов (ES6)

```
javascript

// Базовая деструктуризация
let colors = ["красный", "зеленый", "синий"];
let [firstColor, secondColor] = colors;
console.log(firstColor, secondColor);           // "красный", "зеленый"

// Пропуск элементов
let [, thirdColor] = colors;
console.log(thirdColor);                         // "синий"

// Значения по умолчанию
let [a, b, c, d = "желтый"] = colors;
console.log(d);                                 // "желтый"

// Обмен значений
let x = 1, y = 2;
[x, y] = [y, x];
console.log(x, y);                             // 2, 1

// Остаточные параметры (rest)
let [main, ...others] = colors;
console.log(main);                             // "красный"
console.log(others);                           // ["зеленый", "синий"]

// Деструктуризация вложенных массивов
let matrix = [[1, 2], [3, 4]];
let [[a1, a2], [b1, b2]] = matrix;
console.log(a1, a2, b1, b2);                   // 1, 2, 3, 4
```

2.4. Итерация по массивам

```
javascript

let fruits = ["Яблоко", "Апельсин", "Банан"];

// 1. for (классический)
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}

// 2. for...of (ES6) – только значения
for (let fruit of fruits) {
```

```

    console.log(fruit);
}

// 3. for...in — не рекомендуется для массивов!
for (let index in fruits) {
    console.log(index, fruits[index]);    // "0", "1", "2" (строки!)
}

// 4. forEach — функциональный подход
fruits.forEach((fruit, index, array) => {
    console.log(`${index}: ${fruit}`);
});

// 5. Итераторы
let iterator = fruits[Symbol.iterator]();
console.log(iterator.next().value);    // "Яблоко"
console.log(iterator.next().value);    // "Апельсин"

// 6. keys(), values(), entries() (ES6)
for (let key of fruits.keys()) {
    console.log(key);    // 0, 1, 2
}

for (let value of fruits.values()) {
    console.log(value);    // "Яблоко", "Апельсин", "Банан"
}

for (let [index, value] of fruits.entries()) {
    console.log(index, value);    // 0 "Яблоко", 1 "Апельсин", 2 "Банан"
}

```

2.5. Типизированные массивы (Typed Arrays)

javascript

```

// Создание буфера
let buffer = new ArrayBuffer(16);    // 16 байт
console.log(buffer.byteLength);    // 16

// Типизированные представления
let int32View = new Int32Array(buffer);    // 4 элемента по 4 байта
int32View[0] = 42;
int32View[1] = 100;

```

```
console.log(int32View.length);           // 4
console.log(int32View[0]);               // 42
console.log(int32View.byteLength);       // 16

// Различные типизированные массивы
let int8 = new Int8Array([1, 2, 3]);     // 8-битные целые со знаком
let uint8 = new Uint8Array([1, 2, 3]);   // 8-битные целые без знака
let float32 = new Float32Array([1.5, 2.7]); // 32-битные числа с плавающей точкой
let float64 = new Float64Array([1.5, 2.7]); // 64-битные числа с плавающей точкой

// Методы аналогичны обычным массивам
let arr = new Uint8Array([10, 20, 30, 40, 50]);
let filtered = arr.filter(n => n > 25);
console.log(filtered);                  // Uint8Array [30, 40, 50]
```

3. Функции (Functions)

3.1. Что такое функции в JavaScript

Функции в JavaScript — это объекты первого класса (first-class citizens), которые можно:

- Присваивать переменным
- Передавать как аргументы
- Возвращать из других функций
- Хранить в структурах данных

javascript

```
// Function Declaration (поднимается)
function sayHello(name) {
  return `Привет, ${name}!`;
}
console.log(sayHello("Иван"));           // "Привет, Иван!"
```

```
// Function Expression (не поднимается)
const sayGoodbye = function(name) {
  return `До свидания, ${name}!`;
};
console.log(sayGoodbye("Анна"));         // "До свидания, Анна!"
```

```
// Стрелочная функция (ES6)
const multiply = (a, b) => a * b;
console.log(multiply(5, 3)); // 15

// Конструктор Function (не рекомендуется)
const sum = new Function('a', 'b', 'return a + b');
console.log(sum(2, 3)); // 5
```

3.2. Параметры и аргументы

javascript

```
// Параметры по умолчанию (ES6)
function greet(name = "Гость", greeting = "Привет") {
  return `${greeting}, ${name}!`;
}
console.log(greet()); // "Привет, Гость!"
console.log(greet("Иван")); // "Привет, Иван!"
console.log(greet("Анна", "Здравствуй")); // "Здравствуй, Анна!"

// Rest параметры (остаточные)
function sumAll(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
console.log(sumAll(1, 2, 3)); // 6
console.log(sumAll(1, 2, 3, 4, 5)); // 15

// Spread оператор для передачи массива как аргументов
let nums = [1, 2, 3, 4, 5];
console.log(sumAll(...nums)); // 15

// Деструктуризация параметров
function printUser({name, age, city = "Неизвестно"}) {
  console.log(`${name}, ${age} лет, ${city}`);
}
printUser({name: "Иван", age: 30}); // "Иван, 30 лет, Неизвестно"
```

3.3. Замыкания (Closures)

javascript

```
// Простое замыкание
function createCounter() {
  let count = 0; // Замкнутая переменная
```

```

    return function() {
        count++;
        return count;
    };
}

const counter = createCounter();
console.log(counter());           // 1
console.log(counter());           // 2
console.log(counter());           // 3

// Практический пример – приватные переменные
function createBankAccount(initialBalance) {
    let balance = initialBalance;    // Приватная переменная

    return {
        deposit(amount) {
            balance += amount;
            return balance;
        },
        withdraw(amount) {
            if (amount <= balance) {
                balance -= amount;
                return balance;
            }
            return "Недостаточно средств";
        },
        getBalance() {
            return balance;
        }
    };
}

let account = createBankAccount(1000);
console.log(account.getBalance());    // 1000
console.log(account.deposit(500));    // 1500
console.log(account.withdraw(2000));  // "Недостаточно средств"
// console.log(balance);              // Ошибка: balance не определена

```

3.4. Контекст выполнения (this)

javascript

```

// Значение this зависит от способа вызова функции
const person = {
  name: "Иван",
  greet: function() {
    console.log(`Привет, я ${this.name}`);
  },
  greetArrow: () => {
    console.log(`Привет, я ${this.name}`); // this из внешнего контекста
  }
};

person.greet(); // "Привет, я Иван" (this = person)
person.greetArrow(); // "Привет, я undefined" (this ≠ person)

// Явное связывание контекста
function introduce(age, city) {
  console.log(`Я ${this.name}, мне ${age} лет, из ${city}`);
}

const user = { name: "Анна" };

// call – вызывает функцию с указанным this и аргументами по отдельности
introduce.call(user, 25, "Москва"); // "Я Анна, мне 25 лет, из Москвы"

// apply – вызывает функцию с указанным this и массивом аргументов
introduce.apply(user, [25, "Москва"]); // "Я Анна, мне 25 лет, из Москвы"

// bind – создает новую функцию с привязанным this
const introduceAnna = introduce.bind(user, 25);
introduceAnna("Москва"); // "Я Анна, мне 25 лет, из Москвы"

```

3.5. Функции высшего порядка (Higher-Order Functions)

javascript

```

// Функция, принимающая другую функцию как аргумент
function calculate(a, b, operation) {
  return operation(a, b);
}

const add = (x, y) => x + y;
const multiply = (x, y) => x * y;

console.log(calculate(5, 3, add)); // 8

```

```

console.log(calculate(5, 3, multiply)); // 15

// Функция, возвращающая другую функцию
function createMultiplier(factor) {
  return function(number) {
    return number * factor;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15

// Каррирование (currying)
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    } else {
      return function(...args2) {
        return curried.apply(this, args.concat(args2));
      };
    }
  };
}

function sum(a, b, c) {
  return a + b + c;
}

const curriedSum = curry(sum);
console.log(curriedSum(1)(2)(3)); // 6
console.log(curriedSum(1, 2)(3)); // 6
console.log(curriedSum(1)(2, 3)); // 6

```

3.6. Генераторы (Generators)

javascript

```

// Функция-генератор (function*)
function* numberGenerator() {
  yield 1;
}

```



```

    yield 2;
    yield 3;
}

const gen = numberGenerator();
console.log(gen.next().value);           // 1
console.log(gen.next().value);           // 2
console.log(gen.next().value);           // 3
console.log(gen.next().done);             // true

// Бесконечный генератор
function* infiniteSequence() {
    let i = 0;
    while (true) {
        yield i++;
    }
}

const infinite = infiniteSequence();
console.log(infinite.next().value);       // 0
console.log(infinite.next().value);       // 1
console.log(infinite.next().value);       // 2

// Генератор с передачей значений
function* twoWayGenerator() {
    const name = yield "Как вас зовут?";
    const age = yield `Привет, ${name}! Сколько вам лет?`;
    yield `${name}, вам ${age} лет. Спасибо!`;
}

const conversation = twoWayGenerator();
console.log(conversation.next().value);    // "Как вас зовут?"
console.log(conversation.next("Иван").value); // "Привет, Иван! Сколько вам лет?"
console.log(conversation.next(30).value);  // "Иван, вам 30 лет. Спасибо!"

```

3.7. Асинхронные функции (async/await)

javascript

```

// Асинхронная функция всегда возвращает Promise
async function fetchData() {
    // Имитация асинхронной операции
    return new Promise(resolve => {
        setTimeout(() => resolve("данные получены"), 1000);
    });
}

```

```
});  
}  
  
// Использование с await  
async function processData() {  
  console.log("Начало загрузки...");  
  const data = await fetchData();           // Ожидание Promise  
  console.log(`Результат: ${data}`);  
  return data.toUpperCase();  
}  
  
processData().then(result => {  
  console.log(`Обработанные данные: ${result}`);  
});  
  
// Обработка ошибок  
async function fetchWithErrorHandling() {  
  try {  
    const response = await fetch("https://api.example.com/data");  
    if (!response.ok) {  
      throw new Error(`HTTP error: ${response.status}`);  
    }  
    return await response.json();  
  } catch (error) {  
    console.error("Ошибка загрузки:", error.message);  
    return null;  
  } finally {  
    console.log("Запрос завершен");  
  }  
}  
  
// Параллельное выполнение  
async function fetchMultiple() {  
  const [data1, data2] = await Promise.all([  
    fetch("https://api.example.com/data1"),  
    fetch("https://api.example.com/data2")  
  ]);  
  return { data1: await data1.json(), data2: await data2.json() };  
}
```

Сравнение и взаимодействие ссылочных типов

1. Ссылки и копирование

```
javascript

// Объекты копируются по ссылке
let obj1 = { x: 1 };
let obj2 = obj1;           // obj2 ссылается на тот же объект
obj2.x = 2;
console.log(obj1.x);       // 2 (изменился исходный объект)

// Массивы тоже копируются по ссылке
let arr1 = [1, 2, 3];
let arr2 = arr1;           // arr2 ссылается на тот же массив
arr2.push(4);
console.log(arr1);         // [1, 2, 3, 4]

// Функции также
let func1 = () => console.log("test");
let func2 = func1;         // func2 ссылается на ту же функцию
console.log(func1 === func2); // true
```

2. Способы копирования

```
javascript

// Поверхностное копирование объектов
let original = { a: 1, b: { c: 2 } };

// 1. Spread operator (ES6)
let copy1 = { ...original };

// 2. Object.assign
let copy2 = Object.assign({}, original);

// 3. JSON методы (глубокое копирование, но с ограничениями)
let deepCopy = JSON.parse(JSON.stringify(original));

// Вложенные объекты всё ещё ссылаются на те же объекты!
copy1.b.c = 3;
console.log(original.b.c); // 3! (поверхностное копирование)
```

```
// Поверхностное копирование массивов
let arr = [1, 2, [3, 4]];
let arrCopy = [...arr]; // или arr.slice()
arrCopy[2][0] = 99;
console.log(arr[2][0]); // 99!
```

3. Проверка типов

```
javascript

let obj = {};
let arr = [];
let func = function() {};

console.log(typeof obj); // "object"
console.log(typeof arr); // "object" (массив – тоже объект!)
console.log(typeof func); // "function"

// Правильные проверки
console.log(Array.isArray(arr)); // true (для массивов)
console.log(arr instanceof Array); // true
console.log(obj instanceof Object); // true
console.log(func instanceof Function); // true

// Проверка конструктора
console.log(obj.constructor === Object); // true
console.log(arr.constructor === Array); // true
console.log(func.constructor === Function); // true

// Проверка на "чистый" объект
function isPlainObject(value) {
  return typeof value === 'object'
    && value !== null
    && value.constructor === Object;
}
console.log(isPlainObject({})); // true
console.log(isPlainObject([])); // false
console.log(isPlainObject(null)); // false
```

4. Преобразование между типами

```
javascript

// Объект в массив
```

```
let obj = { a: 1, b: 2, c: 3 };
console.log(Object.keys(obj));           // ["a", "b", "c"]
console.log(Object.values(obj));         // [1, 2, 3]
console.log(Object.entries(obj));        // [["a", 1], ["b", 2], ["c", 3]]

// Массив в объект
let arr = [["a", 1], ["b", 2]];
console.log(Object.fromEntries(arr));    // {a: 1, b: 2}

// Функция в строку
function sum(a, b) { return a + b; }
console.log(sum.toString());
// "function sum(a, b) { return a + b; }"

// Объект в функцию (опасно!)
let funcStr = "return a + b";
let dynamicFunc = new Function("a", "b", funcStr);
console.log(dynamicFunc(2, 3));          // 5
```

Практические паттерны и примеры

1. Фабрика объектов

javascript

```
function createUser(name, age, role = "user") {
  return {
    name,
    age,
    role,
    isAdmin() {
      return this.role === "admin";
    },
    greet() {
      return `Привет, я ${this.name}`;
    }
  };
}

const admin = createUser("Иван", 30, "admin");
```

```
const user = createUser("Анна", 25);

console.log(admin.isAdmin());           // true
console.log(user.isAdmin());           // false
console.log(admin.greet());             // "Привет, я Иван"
```

2. Мемоизация (кэширование)

javascript

```
function memoize(fn) {
  const cache = new Map();

  return function(...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      console.log("Из кэша:", key);
      return cache.get(key);
    }

    console.log("Вычисляем:", key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

// Медленная функция для теста
function slowCalculation(n) {
  let result = 0;
  for (let i = 0; i < n * 1000000; i++) {
    result += Math.random();
  }
  return result;
}

const memoizedCalc = memoize(slowCalculation);
console.log(memoizedCalc(10));           // Вычисляем: [10]
console.log(memoizedCalc(10));           // Из кэша: [10] (быстро!)
```

3. Цепочка вызовов (chaining)

javascript

```

class Calculator {
  constructor(value = 0) {
    this.value = value;
  }

  add(num) {
    this.value += num;
    return this; // Возвращаем this для чейнинга
  }

  subtract(num) {
    this.value -= num;
    return this;
  }

  multiply(num) {
    this.value *= num;
    return this;
  }

  getResult() {
    return this.value;
  }
}

const calc = new Calculator(10);
const result = calc.add(5).multiply(2).subtract(3).getResult();
console.log(result); // 27

```

4. Функциональное программирование

javascript

// Чистые функции

```

const add = (a, b) => a + b;
const multiply = (a, b) => a * b;

```

// Композиция функций

```

const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

```

// Пример

```

const users = [
  { id: 1, name: "Иван", age: 30, active: true },

```

```
{ id: 2, name: "Анна", age: 25, active: true },
{ id: 3, name: "Петр", age: 35, active: false },
{ id: 4, name: "Мария", age: 28, active: true }
];

// Функциональный конвейер
const getActiveUserNames = pipe(
  users => users.filter(user => user.active),
  users => users.map(user => user.name),
  names => names.sort()
);

console.log(getActiveUserNames(users)); // ["Анна", "Иван", "Мария"]
```

Производительность и оптимизация

1. Оптимизация работы с объектами

javascript

```
// 1. Используйте const для объектов, которые не меняют ссылку
const config = {
  apiUrl: "https://api.example.com",
  timeout: 5000
};

// 2. Замораживайте неизменяемые объекты
const constants = Object.freeze({
  PI: 3.14159,
  MAX_USERS: 1000
});

// 3. Используйте Map для частых добавлений/удалений
const userMap = new Map();
userMap.set("id1", { name: "Иван" });
userMap.set("id2", { name: "Анна" });
console.log(userMap.get("id1"));

// 4. Избегайте динамических ключей в горячих путях
// Плохо (медленно, т.к. меняется "скрытый класс"):
```



```
for (let i = 0; i < 1000; i++) {  
  obj["prop" + i] = i;  
}
```

// Лучше:

```
const arr = new Array(1000);  
for (let i = 0; i < 1000; i++) {  
  arr[i] = i;  
}
```

2. Оптимизация работы с массивами

javascript

// 1. Указывайте начальный размер больших массивов

```
const largeArray = new Array(1000000); // Выделяет память сразу
```

// 2. Используйте типизированные массивы для числовых данных

```
const floatArray = new Float64Array(1000000); // Быстрее и меньше памяти
```

// 3. Избегайте shift/unshift для больших массивов

// Плохо: O(n)

```
largeArray.unshift(newItem);
```

// Лучше: используйте индексы или двустороннюю очередь

```
class Deque {  
  constructor() {  
    this.front = [];  
    this.back = [];  
  }  
  
  pushFront(item) {  
    this.front.push(item);  
  }  
  
  popFront() {  
    if (this.front.length === 0) {  
      this.front = this.back.reverse();  
      this.back = [];  
    }  
    return this.front.pop();  
  }  
}
```

```
// 4. Используйте Set для уникальных значений
const uniqueValues = new Set([1, 2, 2, 3, 3, 3]);
console.log([...uniqueValues]); // [1, 2, 3]
```

3. Оптимизация функций

javascript

```
// 1. Инлайнинг маленьких функций
// Движок может оптимизировать, если функция маленькая
```

```
// 2. Избегайте arguments в горячих функциях
// Плохо (не оптимизируемо):
function sum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
```

```
// Лучше:
function sum(...numbers) {
  let total = 0;
  for (let i = 0; i < numbers.length; i++) {
    total += numbers[i];
  }
  return total;
}
```

```
// 3. Кэшируйте результаты дорогих вычислений
function fibonacci(n, memo = {}) {
  if (n in memo) return memo[n];
  if (n <= 1) return n;

  memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo);
  return memo[n];
}
```

Заключение

Ссылочные типы (объекты, массивы, функции) — мощный инструмент в JavaScript, но требующий понимания их особенностей:

Ключевые моменты:

1. **Объекты** — гибкие структуры для хранения данных, но требуют аккуратности с мутациями
2. **Массивы** — оптимизированы для числовых индексов, богатый API для манипуляций
3. **Функции** — объекты первого класса, основа функционального программирования

Главные отличия от примитивных типов:

1. **Мутабельность** — содержимое можно изменять
2. **Сравнение по ссылке** — `{ } !== { }`, `[ ] !== [ ]`
3. **Передача по ссылке** — копирование требует специальных методов
4. **Хранение в куче** — доступ медленнее, но больше гибкости

Лучшие практики:

1. Используйте `const` для ссылочных типов, если не меняете ссылку
2. Делайте глубокие копии при необходимости независимости
3. Используйте правильные структуры данных (Map/Set, типизированные массивы)
4. Помните о производительности в критичных участках кода
5. Используйте функциональные методы для чистого и читаемого кода

Понимание ссылочных типов — ключ к написанию эффективного, поддерживаемого JavaScript-кода. Эти знания позволяют выбирать правильные структуры данных для каждой задачи и избегать распространённых ошибок.

4.5. Динамическая типизация и преобразование типов

Введение: природа динамической типизации в JavaScript

JavaScript является **динамически типизированным** языком, что означает:

- Типы определяются во время выполнения, а не компиляции
- Переменные могут менять тип в процессе выполнения программы
- Преобразования типов происходят автоматически (coercion) или явно

Сравнение со статической типизацией:

```
javascript

// JavaScript (динамическая типизация)
let value = 42;           // Number
value = "текст";         // String (изменение типа разрешено)
value = true;            // Boolean
value = {};              // Object

// TypeScript/Java (статическая типизация)
// let value: number = 42;
// value = "текст";      // Ошибка компиляции: тип не совпадает
```

Часть 1: Неявное преобразование типов (Type Coercion)

1.1. Что такое неявное преобразование

Неявное преобразование (coercion) — автоматическое преобразование типов JavaScript при выполнении операций.

```
javascript

// Примеры неявного преобразования
console.log("5" + 3);           // "53" (число → строка)
console.log("5" - 3);           // 2 (строка → число)
```

```

console.log("5" * "2");           // 10 (оба → числа)
console.log(true + 1);             // 2 (true → 1)
console.log(false + 1);           // 1 (false → 0)
console.log(null + 1);             // 1 (null → 0)
console.log(undefined + 1);       // NaN (undefined → NaN)

```

1.2. Правила преобразования в логический контекст

javascript

```

// Логический контекст: if, while, !, &&, ||, ?:
// Все значения конвертируются в Boolean

```

```

// Falsy значения (становятся false):

```

```

console.log(Boolean(false));      // false
console.log(Boolean(0));          // false
console.log(Boolean(-0));         // false
console.log(Boolean(0n));         // false (BigInt)
console.log(Boolean(""));        // false
console.log(Boolean(null));       // false
console.log(Boolean(undefined));  // false
console.log(Boolean(NaN));        // false

```

```

// Truthy значения (становятся true):

```

```

console.log(Boolean(true));       // true
console.log(Boolean(42));         // true
console.log(Boolean(-42));        // true
console.log(Boolean(3.14));       // true
console.log(Boolean("0"));        // true (строка!)
console.log(Boolean("false"));    // true (строка!)
console.log(Boolean(" "));        // true (пробел)
console.log(Boolean([]));         // true (пустой массив)
console.log(Boolean({}));         // true (пустой объект)
console.log(Boolean(function(){})); // true

```

```

// Практические примеры

```

```

function printIfTruthy(value) {
  if (value) { // Неявное преобразование к Boolean
    console.log("Truthy:", value);
  } else {
    console.log("Falsy:", value);
  }
}

```

```

printIfTruthy(0);           // "Falsy: 0"
printIfTruthy("0");         // "Truthy: 0"
printIfTruthy([]);          // "Truthy: []"
printIfTruthy("");          // "Falsy: "

```

1.3. Преобразование в строковом контексте

javascript

// Контекст, требующий строк: + (с одним строковым операндом), template literals

```

console.log("Результат: " + 42);           // "Результат: 42" (число → строка)
console.log("Цена: " + 99.99);             // "Цена: 99.99"
console.log("Статус: " + true);            // "Статус: true"
console.log("Значение: " + null);          // "Значение: null"
console.log("Значение: " + undefined);     // "Значение: undefined"
console.log("Массив: " + [1, 2, 3]);       // "Массив: 1,2,3" (массив → строка)
console.log("Объект: " + {x: 1});          // "Объект: [object Object]"

```

// Порядок имеет значение!

```

console.log(1 + 2 + "3");                  // "33" (1+2=3, затем 3+"3")
console.log("1" + 2 + 3);                  // "123" ("1"+2="12", "12"+3="123")

```

// Template literals также вызывают преобразование

```

console.log(`Сумма: ${100 + 50}`);         // "Сумма: 150"
console.log(`Массив: ${[1, 2]}`);         // "Массив: 1,2"
console.log(`Объект: ${{a: 1}}`);         // "Объект: [object Object]"

```

1.4. Преобразование в числовой контексте

javascript

// Контекст, требующий чисел: математические операции (кроме +), сравнения

// Арифметические операции

```

console.log("10" - 5);                     // 5
console.log("10" * "2");                   // 20
console.log("10" / "2");                   // 5
console.log("10" % "3");                   // 1
console.log("10" ** "2");                  // 100
console.log(+ "10");                       // 10 (унарный плюс)
console.log(- "10");                       // -10 (унарный минус)

```

// Нечисловые значения

```

console.log("текст");           // NaN
console.log("123текст");       // NaN
console.log(" 123 ");          // 123 (пробелы игнорируются)
console.log("0xFF");           // 255 (шестнадцатеричное)
console.log("1.23e3");         // 1230 (экспоненциальная запись)

// Логические значения
console.log(true + true);      // 2 (true → 1)
console.log(false + false);   // 0 (false → 0)
console.log(true * 10);        // 10
console.log(false * 10);       // 0

// null и undefined
console.log(null + 10);        // 10 (null → 0)
console.log(undefined + 10);   // NaN (undefined → NaN)
console.log(null * 10);        // 0
console.log(undefined * 10);   // NaN

// Массивы и объекты
console.log([10] + 5);         // "105" (массив → строка, затем конкатенация)
console.log([10] - 5);         // 5 (массив → число 10, затем вычитание)
console.log([] + 1);           // "1" (пустой массив → "")
console.log([1, 2] + 3);       // "1,23" (массив → "1,2", затем конкатенация)
console.log({} + 1);           // "[object Object]1" (в консоли)

```

1.5. Сравнение с неявным преобразованием

javascript

```

// Нестрогое сравнение (==) выполняет преобразование типов

// Строки и числа
console.log("5" == 5);         // true ("5" → 5)
console.log("" == 0);          // true (" " → 0, 0 == 0)
console.log(" " == 0);         // true (пробелы → 0)

// Логические значения
console.log(true == 1);        // true (true → 1)
console.log(false == 0);       // true (false → 0)
console.log(true == "1");      // true (true → 1, "1" → 1)
console.log(false == "");      // true (false → 0, "" → 0)

// null и undefined
console.log(null == undefined); // true (специальное правило)

```

```

console.log(null == 0);           // false (null не преобразуется в 0 при ==!)
console.log(undefined == 0);      // false

// Объекты
console.log([1] == 1);            // true ([1] → "1" → 1)
console.log([1] == "1");         // true ([1] → "1")
console.log([] == 0);            // true ([] → "" → 0)
console.log([] == false);        // true ([] → "" → 0, false → 0)

// Сложные случаи
console.log("\n 123 \t" == 123); // true (пробелы игнорируются)
console.log("0x10" == 16);        // true (шестнадцатеричное преобразование)
console.log("1e3" == 1000);       // true (экспоненциальная запись)

```

Часть 2: Явное преобразование типов

2.1. Преобразование к строке

javascript

```

// String() - явное преобразование к строке
console.log(String(123));          // "123"
console.log(String(true));         // "true"
console.log(String(false));       // "false"
console.log(String(null));        // "null"
console.log(String(undefined));   // "undefined"
console.log(String([1, 2, 3]));   // "1,2,3"
console.log(String({a: 1}));      // "[object Object]"
console.log(String(function(){})); // "function(){}"
console.log(String(Symbol("id"))); // "Symbol(id)"
console.log(String(12345678901234567890n)); // "12345678901234567890"

// .toString() метод (не работает с null и undefined)
console.log((123).toString());    // "123"
console.log(true.toString());     // "true"
console.log([1, 2].toString());   // "1,2"
console.log({}.toString());       // "[object Object]"

// Особые случаи
console.log((10).toString(2));    // "1010" (двоичная система)

```



```

console.log((255).toString(16));           // "ff" (шестнадцатеричная)
console.log((10).toString(8));             // "12" (восьмеричная)
// console.log(null.toString());           // TypeError
// console.log(undefined.toString());      // TypeError

// JSON.stringify() - для объектов и массивов
console.log(JSON.stringify([1, 2, 3]));    // "[1,2,3]"
console.log(JSON.stringify({x: 1, y: 2})); // '{"x":1,"y":2}'
console.log(JSON.stringify({date: new Date()})); // '{"date":"2024-03-15T..."}'

// Особенности JSON.stringify
const obj = {
  name: "Иван",
  age: 30,
  married: undefined,           // Пропускается
  job: null,                   // Сохраняется
  skills: ["JS", "React"],
  getFullName() {              // Пропускается (функции)
    return this.name;
  },
  [Symbol("id")]: 123,         // Пропускается
  birthDate: new Date()       // Преобразуется в строку
};

console.log(JSON.stringify(obj));
// {"name":"Иван","age":30,"job":null,"skills":["JS","React"],"birthDate":"2024-03-15T..."}

// Третий параметр - форматирование
console.log(JSON.stringify(obj, null, 2));
/*
{
  "name": "Иван",
  "age": 30,
  "job": null,
  "skills": [
    "JS",
    "React"
  ],
  "birthDate": "2024-03-15T..."
}
*/

```

2.2. Преобразование к числу

javascript

// Number() - явное преобразование к числу

```
console.log(Number("123"));           // 123
console.log(Number("123.45"));        // 123.45
console.log(Number("123текст"));      // NaN
console.log(Number(""));              // 0
console.log(Number(" 123 "));         // 123 (пробелы игнорируются)
console.log(Number(true));            // 1
console.log(Number(false));          // 0
console.log(Number(null));            // 0
console.log(Number(undefined));       // NaN
console.log(Number([5]));             // 5 (массив с одним числом)
console.log(Number([1, 2]));          // NaN (массив с несколькими элементами)
console.log(Number([]));              // 0 (пустой массив)
console.log(Number({}));              // NaN
```

// Унарный плюс (+)

```
console.log(+ "123");                 // 123
console.log(+ "123.45");              // 123.45
console.log(+ true);                  // 1
console.log(+ false);                 // 0
console.log(+ null);                  // 0
console.log(+ undefined);             // NaN
console.log(+ "");                    // 0
```

// parseFloat() и parseInt()

```
console.log(parseInt("123px"));        // 123
console.log(parseInt("12.34"));        // 12 (только целая часть)
console.log(parseInt("0xFF"));         // 255 (шестнадцатеричное)
console.log(parseInt("текст123"));     // NaN
console.log(parseInt("123текст456"));  // 123 (останавливается на первом нечисловом)
```

```
console.log(parseFloat("123.45px"));   // 123.45
console.log(parseFloat("12.34.56"));   // 12.34 (останавливается на второй точке)
console.log(parseFloat("1.23e2"));     // 123 (экспоненциальная запись)
```

// Системы счисления для parseInt

```
console.log(parseInt("1010", 2));      // 10 (двоичная)
console.log(parseInt("FF", 16));       // 255 (шестнадцатеричная)
console.log(parseInt("077", 8));       // 63 (восьмеричная)
console.log(parseInt("10", 36));       // 36 (максимальная система)
```

```
// Math.floor, Math.ceil, Math.round
console.log(Math.floor("3.7"));           // 3
console.log(Math.ceil("3.1"));            // 4
console.log(Math.round("3.5"));           // 4
console.log(Math.trunc("3.7"));            // 3 (отбрасывание дробной части)
```

2.3. Преобразование к логическому типу

javascript

```
// Boolean() - явное преобразование
console.log(Boolean(0));                  // false
console.log(Boolean(1));                  // true
console.log(Boolean(-1));                 // true
console.log(Boolean(""));                 // false
console.log(Boolean("текст"));            // true
console.log(Boolean("false"));            // true (строка не пустая!)
console.log(Boolean(null));               // false
console.log(Boolean(undefined));          // false
console.log(Boolean(NaN));                // false
console.log(Boolean([]));                 // true
console.log(Boolean({}));                 // true
console.log(Boolean(function(){}));       // true

// Двойное отрицание (!! ) - идиоматический способ
console.log(!!0);                        // false
console.log(!!1);                        // true
console.log(!!"текст");                   // true
console.log(!!null);                      // false
```

```
// Практическое использование
function processValue(value) {
  // Явное преобразование для ясности
  const hasValue = Boolean(value);
  // или
  const hasValue2 = !!value;

  if (hasValue) {
    console.log("Обрабатываем:", value);
  } else {
    console.log("Значение отсутствует");
  }
}
```

```
processValue("текст");           // "Обрабатываем: текст"
processValue("");                // "Значение отсутствует"
processValue(0);                 // "Значение отсутствует"
processValue([]);                // "Обрабатываем: []" (массив truthy!)
```

2.4. Преобразование объектов

javascript

```
// Object() - преобразование к объекту
console.log(Object(123));        // Number {123}
console.log(Object("текст"));    // String {"текст"}
console.log(Object(true));       // Boolean {true}
console.log(Object(null));       // {}
console.log(Object(undefined));  // {}
console.log(Object([1, 2]));     // [1, 2]
console.log(Object(Symbol("id"))); // Symbol {Symbol(id)}
```

```
// ЗначениеOf и toString объектов
```

```
const obj = {
  x: 10,
  y: 20,
```

```
// Метод, вызываемый при преобразовании к примитиву
```

```
[Symbol.toPrimitive](hint) {
  console.log(`hint: ${hint}`);
  if (hint === "number") {
    return this.x + this.y;
  }
  if (hint === "string") {
    return `(${this.x}, ${this.y})`;
  }
  return this.x + this.y; // по умолчанию
},
```

```
// Стандартные методы (устаревшие)
```

```
valueOf() {
  console.log("valueOf called");
  return this.x + this.y;
},
```

```
toString() {
  console.log("toString called");
```

```

    return `Point(${this.x}, ${this.y})`;
  }
};

// Преобразование в разных контекстах
console.log(+obj);           // hint: number → 30
console.log(String(obj));    // hint: string → "(10, 20)"
console.log(obj + 10);       // hint: default → 40
console.log(`${obj}`);       // hint: string → "(10, 20)"

```

Часть 3: Особые случаи и граничные условия

3.1. NaN (Not a Number)

```

javascript

// Что такое NaN
console.log(typeof NaN);           // "number" (парадокс!)
console.log(NaN === NaN);         // false (единственное значение, не равное
себе!)
console.log(Object.is(NaN, NaN));  // true (специальная проверка)

// Операции, возвращающие NaN
console.log(Number("текст"));      // NaN
console.log(0 / 0);                // NaN
console.log(Infinity - Infinity);  // NaN
console.log(Math.sqrt(-1));        // NaN
console.log(Math.log(-1));         // NaN
console.log(parseInt("текст"));    // NaN

// Проверка на NaN
console.log(isNaN(NaN));           // true
console.log(isNaN("текст"));       // true (проблема: преобразует строку в число!)
console.log(isNaN("123"));         // false ("123" → 123, не NaN)

// Надежная проверка
console.log(Number.isNaN(NaN));    // true
console.log(Number.isNaN("текст")); // false (строгое сравнение)
console.log(Number.isNaN(123));    // false

```

```

// Альтернативные способы
function isRealNaN(value) {
    return typeof value === "number" && isNaN(value);
}

function isRealNaN2(value) {
    return value !== value; // NaN – единственное значение, не равное себе
}

console.log(isRealNaN(NaN)); // true
console.log(isRealNaN("текст")); // false

```

3.2. Infinity и -Infinity

javascript

```

// Бесконечности
console.log(1 / 0); // Infinity
console.log(-1 / 0); // -Infinity
console.log(Infinity > 1000000); // true
console.log(-Infinity < -1000000); // true
console.log(typeof Infinity); // "number"

// Операции с Infinity
console.log(Infinity + 1); // Infinity
console.log(Infinity - 1); // Infinity
console.log(Infinity * 2); // Infinity
console.log(Infinity / 2); // Infinity
console.log(Infinity / Infinity); // NaN
console.log(Infinity - Infinity); // NaN
console.log(1 / Infinity); // 0
console.log(0 * Infinity); // NaN

// Проверка
console.log(isFinite(42)); // true
console.log(isFinite(Infinity)); // false
console.log(isFinite(-Infinity)); // false
console.log(isFinite(NaN)); // false
console.log(isFinite("42")); // true ("42" → 42)
console.log(isFinite("текст")); // false ("текст" → NaN)

// Надежная проверка
console.log(Number.isFinite(42)); // true
console.log(Number.isFinite(Infinity)); // false

```

```
console.log(Number.isFinite("42")); // false (строгая проверка)
```

3.3. Нули: +0 и -0

javascript

// Два нуля

```
console.log(+0 === -0); // true
console.log(Object.is(+0, -0)); // false (различает!)
```

// Получение -0

```
console.log(1 / -Infinity); // -0
console.log(-1 / Infinity); // -0
console.log(Math.round(-0.1)); // -0
console.log(Math.ceil(-0.1)); // -0
```

// Операции с -0

```
console.log(-0 + 5); // 5
console.log(-0 * 5); // -0
console.log(5 / -0); // -Infinity
console.log(-5 / -0); // Infinity
```

// Преобразование в строку

```
console.log(String(-0)); // "0" (потеря знака!)
console.log((-0).toString()); // "0"
console.log(JSON.stringify(-0)); // "0"
```

// Проверка

```
function isNegativeZero(value) {
  return value === 0 && 1 / value === -Infinity;
}
```

```
console.log(isNegativeZero(0)); // false
console.log(isNegativeZero(-0)); // true
console.log(isNegativeZero(5)); // false
```

3.4. Преобразование BigInt

javascript

// BigInt преобразования

```
console.log(BigInt(123)); // 123n
console.log(BigInt("12345678901234567890")); // 12345678901234567890n
console.log(BigInt(true)); // 1n
```

```

console.log(BigInt(false));           // 0n
// console.log(BigInt(123.45));       // TypeError
// console.log(BigInt("123.45"));     // SyntaxError
// console.log(BigInt(null));         // TypeError
// console.log(BigInt(undefined));    // TypeError

// Преобразование из BigInt
console.log(Number(12345678901234567890n)); // 1.2345678901234568e+19 (потеря точности)
console.log(String(12345678901234567890n)); // "12345678901234567890"
console.log(Boolean(123n));             // true
console.log(Boolean(0n));               // false

// Смешивание с Number
// console.log(1n + 2);               // TypeError
console.log(1n + BigInt(2));            // 3n
console.log(Number(1n) + 2);            // 3

// Сравнение работает
console.log(1n == 1);                  // true
console.log(1n === 1);                 // false (разные типы)
console.log(1n < 2);                   // true
console.log(1n > 0);                   // true

```

3.5. Преобразование Symbol

javascript

```

// Symbol преобразования
const sym = Symbol("id");

console.log(String(sym));               // "Symbol(id)"
console.log(sym.toString());            // "Symbol(id)"
// console.log(Number(sym));           // TypeError
// console.log(Boolean(sym));          // true (но нельзя вызвать Boolean() явно)
console.log(!sym);                     // true

// Symbol.toPrimitive
const objWithSymbol = {
  [Symbol.toPrimitive](hint) {
    if (hint === "string") {
      return "объект";
    }
    return 42;
  }
}

```



```
};

console.log(String(objWithSymbol));    // "объект"
console.log(Number(objWithSymbol));    // 42
```

Часть 4: Алгоритмы преобразования

4.1. Алгоритм абстрактного сравнения (==)

javascript

```
// Псевдокод алгоритма == (Abstract Equality Comparison)
function abstractEquals(x, y) {
  // 1. Если типы одинаковы, выполняем строгое сравнение
  if (typeof x === typeof y) {
    return x === y;
  }

  // 2. null и undefined равны только друг другу
  if (x == null && y == null) {
    return true;
  }

  // 3. Число и строка: строка → число
  if (typeof x === 'number' && typeof y === 'string') {
    return abstractEquals(x, Number(y));
  }
  if (typeof x === 'string' && typeof y === 'number') {
    return abstractEquals(Number(x), y);
  }

  // 4. Один из операндов Boolean: Boolean → число
  if (typeof x === 'boolean') {
    return abstractEquals(Number(x), y);
  }
  if (typeof y === 'boolean') {
    return abstractEquals(x, Number(y));
  }

  // 5. Один из операндов объект: объект → примитив
```

```

    if ((typeof x === 'object' || typeof x === 'function') && x !== null) {
        return abstractEquals(x[Symbol.toPrimitive]('default') || x.valueOf() ||
x.toString(), y);
    }
    if ((typeof y === 'object' || typeof y === 'function') && y !== null) {
        return abstractEquals(x, y[Symbol.toPrimitive]('default') || y.valueOf() ||
y.toString());
    }

    // 6. Все остальные случаи
    return false;
}

// Примеры работы алгоритма
console.log("5" == 5);           // true (шаг 3)
console.log(true == 1);          // true (шаг 4: true → 1)
console.log(null == undefined);  // true (шаг 2)
console.log([1] == 1);           // true (шаг 5: [1] → "1" → 1)
console.log([] == 0);            // true (шаг 5: [] → "" → 0)

```

4.2. Алгоритм абстрактного отношения (<, >, <=, >=)

javascript

```

// Псевдокод алгоритма сравнения
function abstractRelationalComparison(x, y, less) {
    // 1. Сначала преобразовать оба операнда к примитивам
    const px = toPrimitive(x, 'number');
    const py = toPrimitive(y, 'number');

    // 2. Если оба строки — лексикографическое сравнение
    if (typeof px === 'string' && typeof py === 'string') {
        return less ? px < py : px > py;
    }

    // 3. Преобразовать к числам
    const nx = Number(px);
    const ny = Number(py);

    // 4. Если один из них NaN — результат undefined
    if (isNaN(nx) || isNaN(ny)) {
        return undefined;
    }
}

```

```

// 5. Сравнение чисел
if (nx === ny) return false;
if (nx === 0 && ny === 0) {
    // +0 и -0 считаются равными
    return false;
}
return less ? nx < ny : nx > py;
}

// Примеры
console.log("10" > 5);           // true ("10" → 10)
console.log("10" > "5");         // false (лексикографически: "1" < "5")
console.log("abc" > 5);          // false ("abc" → NaN)
console.log(true > false);       // true (true→1, false→0)
console.log(null > -1);          // true (null→0)
console.log(undefined > 0);      // false (undefined→NaN)

```

4.3. Алгоритм ToPrimitive

javascript

```

// Псевдокод ToPrimitive
function toPrimitive(input, preferredType) {
    if (typeof input !== 'object' || input === null) {
        return input; // Уже примитив
    }

    // 1. Проверить Symbol.toPrimitive
    if (typeof input[Symbol.toPrimitive] === 'function') {
        const result = input[Symbol.toPrimitive](preferredType);
        if (typeof result !== 'object') {
            return result;
        }
        throw new TypeError("Cannot convert object to primitive value");
    }

    // 2. Определить хинт
    const hint = preferredType === 'string' ? 'string' : 'number';

    // 3. Для хинта 'number': valueOf, затем toString
    // Для хинта 'string': toString, затем valueOf
    const methods = hint === 'number'
        ? ['valueOf', 'toString']
        : ['toString', 'valueOf'];

```

```

for (const method of methods) {
  if (typeof input[method] === 'function') {
    const result = input[method]();
    if (typeof result !== 'object') {
      return result;
    }
  }
}

throw new TypeError("Cannot convert object to primitive value");
}

// Примеры
const obj = {
  value: 42,
  valueOf() { return this.value; },
  toString() { return `Object(${this.value})`; }
};

console.log(Number(obj));           // 42 (valueOf)
console.log(String(obj));           // "Object(42)" (toString)

const obj2 = {
  [Symbol.toPrimitive](hint) {
    return hint === 'string' ? 'строка' : 100;
  }
};

console.log(Number(obj2));           // 100
console.log(String(obj2));           // "строка"

```

Часть 5: Практические рекомендации

5.1. Избегайте неявного преобразования

javascript

```

// Проблемные паттерны
function calculatePrice(quantity, price) {

```

```

// Опасность: если quantity строка "5", а price число 10
// "5" * 10 = 50 (работает, но ненадежно)
return quantity * price;
}

// Лучше: явное преобразование
function calculatePriceSafe(quantity, price) {
    const qty = Number(quantity);
    const prc = Number(price);

    if (isNaN(qty) || isNaN(prc)) {
        throw new Error("Неверные параметры");
    }

    return qty * prc;
}

// Проблема с сложением
function addNumbers(a, b) {
    // Опасно: если a или b строки
    return a + b;
}

// Решение
function addNumbersSafe(a, b) {
    return Number(a) + Number(b);
}

// Или проверка типов
function addNumbersTypeSafe(a, b) {
    if (typeof a !== 'number' || typeof b !== 'number') {
        throw new TypeError("Аргументы должны быть числами");
    }
    return a + b;
}

```

5.2. Используйте строгое сравнение

javascript

```

// Всегда используйте === и !== вместо == и !=

// Проблемные случаи с ==
console.log(0 == false); // true

```

```

console.log("" == false);           // true
console.log(null == undefined);     // true
console.log([] == false);           // true
console.log([0] == false);           // true

// Предсказуемое поведение с ===
console.log(0 === false);           // false
console.log("" === false);          // false
console.log(null === undefined);    // false
console.log([] === false);          // false
console.log([0] === false);         // false

// Исключение: проверка на null/undefined
function processValue(value) {
  // Приемлемо: проверка на null или undefined
  if (value == null) {               // Проверяет и null, и undefined
    console.log("Значение отсутствует");
  }

  // Альтернатива с ===
  if (value === null || value === undefined) {
    console.log("Значение отсутствует");
  }

  // Современная альтернатива
  if (value == null) {               // nullish coalescing
    console.log("Значение отсутствует");
  }
}

```

5.3. Операторы нулевого слияния и опциональной цепочки

javascript

```

// Оператор нулевого слияния (??) - ES2020
// Возвращает правый операнд только если левый null или undefined

console.log(null ?? "default");      // "default"
console.log(undefined ?? "default"); // "default"
console.log(0 ?? "default");         // 0 (не null/undefined)
console.log("" ?? "default");        // "" (не null/undefined)
console.log(false ?? "default");     // false (не null/undefined)

// Отличие от ||

```

```

console.log(0 || "default");           // "default" (0 falsy!)
console.log("" || "default");          // "default" ("" falsy!)
console.log(0 ?? "default");           // 0 (только null/undefined)

// Опциональная цепочка (?.) - ES2020
const user = {
  profile: {
    name: "Иван",
    address: {
      city: "Москва"
    }
  }
};

// Без опциональной цепочки
const city = user && user.profile && user.profile.address && user.profile.address.city;

// С опциональной цепочкой
const citySafe = user?.profile?.address?.city;

console.log(citySafe);                 // "Москва"
console.log(user?.profile?.contacts?.email); // undefined (без ошибки)

// Комбинация операторов
const email = user?.profile?.contacts?.email ?? "email@example.com";
console.log(email);                    // "email@example.com"

```

5.4. Безопасное преобразование чисел

javascript

```

// Функции для безопасного преобразования
function toNumber(value, defaultValue = 0) {
  const num = Number(value);
  return isNaN(num) ? defaultValue : num;
}

function toInteger(value, defaultValue = 0) {
  const num = parseInt(value, 10);
  return isNaN(num) ? defaultValue : num;
}

function toFloat(value, defaultValue = 0) {
  const num = parseFloat(value);

```

```

    return isNaN(num) ? defaultValue : num;
}

// Примеры
console.log(toNumber("123"));           // 123
console.log(toNumber("123текст"));      // 0
console.log(toNumber("123текст", 100)); // 100
console.log(toNumber(null));            // 0
console.log(toNumber(undefined, 50));   // 50

// Проверка диапазона
function toNumberInRange(value, min, max, defaultValue = min) {
    const num = Number(value);
    if (isNaN(num)) return defaultValue;
    return Math.max(min, Math.min(num, max));
}

console.log(toNumberInRange("10", 0, 100)); // 10
console.log(toNumberInRange("-5", 0, 100)); // 0 (ниже минимума)
console.log(toNumberInRange("150", 0, 100)); // 100 (выше максимума)
console.log(toNumberInRange("текст", 0, 100, 50)); // 50

```

5.5. Обработка ввода пользователя

javascript

```

// Формы и пользовательский ввод
function processFormData(formData) {
    // Все значения из форм — строки
    const age = parseInt(formData.age, 10);
    const price = parseFloat(formData.price);
    const quantity = Number(formData.quantity);

    // Проверки
    if (isNaN(age) || age < 0 || age > 150) {
        throw new Error("Неверный возраст");
    }

    if (isNaN(price) || price <= 0) {
        throw new Error("Неверная цена");
    }

    // Для чекбоксов
    const isMember = formData.member === "on" ||

```



```

        formData.member === "true" ||
        formData.member === true;

    // Для множественного выбора
    const selectedOptions = Array.isArray(formData.options)
        ? formData.options
        : [formData.options].filter(Boolean);

    return { age, price, quantity, isMember, selectedOptions };
}

// API ответы
async function fetchUserData(userId) {
    const response = await fetch(`/api/users/${userId}`);
    const data = await response.json();

    // Безопасное преобразование
    return {
        id: String(data.id || ""),
        name: String(data.name || ""),
        age: toNumber(data.age, 0),
        active: Boolean(data.active),
        createdAt: data.createdAt ? new Date(data.createdAt) : null,
        tags: Array.isArray(data.tags) ? data.tags : []
    };
}

```

Часть 6: Тестирование и отладка

6.1. Тесты преобразования типов

javascript

```

// Набор тестов для проверки преобразований
const typeConversionTests = [
    // [выражение, ожидаемый результат, описание]
    ['"5" + 3', "53", "Конкатенация строки и числа"],
    ['"5" - 3', 2, "Вычитание из строки"],
    ['true + 1', 2, "Сложение Boolean и числа"],
    ['false + 1', 1, "Сложение false и числа"],

```

```

['null + 1', 1, "Сложение null и числа"],
['undefined + 1', NaN, "Сложение undefined и числа"],
['"5" * "2"', 10, "Умножение строк"],
['"10" / "2"', 5, "Деление строк"],
['+"10"', 10, "Унарный плюс"],
['-true', -1, "Унарный минус с true"],
['[] + 1', "1", "Массив и число"],
['[1, 2] + 3', "1,23", "Массив и число"],
['{} + 1', 1, "Объект и число (в консоли)"],
['"5" == 5', true, "Нестрогое сравнение"],
['"" == false', true, "Пустая строка и false"],
['null == undefined', true, "null и undefined"],
['[] == false', true, "Пустой массив и false"]
];

```

// Запуск тестов

```

function runConversionTests() {
  console.log("Тесты преобразования типов:");
  console.log("=".repeat(50));

  typeConversionTests.forEach(([expression, expected, description]) => {
    try {
      const result = eval(expression);
      const passed = result === expected ||
        (isNaN(result) && isNaN(expected));

      console.log(`${passed ? '✓' : 'x'} ${description}`);
      console.log(`  Выражение: ${expression}`);
      console.log(`  Ожидалось: ${expected}`);
      console.log(`  Получено: ${result}`);
      if (!passed) {
        console.log(`    Тип результата: ${typeof result}`);
        console.log(`    Тип ожидаемого: ${typeof expected}`);
      }
      console.log();
    } catch (error) {
      console.log(`x ${description}`);
      console.log(`  Ошибка: ${error.message}`);
      console.log();
    }
  });
}

```

// runConversionTests();

6.2. Инструменты отладки

javascript

```
// Функция для анализа преобразований
function analyzeTypeConversion(expression) {
  console.log(`Анализ: ${expression}`);
  console.log("-".repeat(40));

  try {
    const result = eval(expression);
    console.log(`Результат: ${result}`);
    console.log(`Тип результата: ${typeof result}`);

    if (typeof result === 'number') {
      console.log(`isNaN: ${isNaN(result)}`);
      console.log(`isFinite: ${isFinite(result)}`);
      console.log(`isInteger: ${Number.isInteger(result)}`);
    }

    // Разбор шагов
    const steps = [];
    const originalExpr = expression;

    // Симуляция шагов преобразования
    if (expression.includes('+')) {
      const parts = expression.split('+');
      steps.push(`Шаг 1: Операнды: ${parts[0]} и ${parts[1]}`);
      steps.push(`Шаг 2: Типы: ${typeof eval(parts[0])} и ${typeof eval(parts[1])}`);

      // Проверка на конкатенацию строк
      if (typeof eval(parts[0]) === 'string' || typeof eval(parts[1]) === 'string') {
        steps.push(`Шаг 3: Хотя бы один операнд – строка, выполняется конкатенация`);
        steps.push(`Шаг 4: Преобразование к строкам: ${String(eval(parts[0]))} + ${String(eval(parts[1]))}`);
      } else {
        steps.push(`Шаг 3: Оба операнда не строки, выполняется сложение чисел`);
        steps.push(`Шаг 4: Преобразование к числам: ${Number(eval(parts[0]))} + ${Number(eval(parts[1]))}`);
      }
    }

    steps.forEach(step => console.log(` ${step}`));
```

```

    } catch (error) {
        console.log(`Ошибка: ${error.message}`);
    }

    console.log("=".repeat(40));
}

```

// Примеры использования

```

analyzeTypeConversion('"5" + 3');
analyzeTypeConversion('"5" - 3');
analyzeTypeConversion('true + 1');
analyzeTypeConversion('[] + {}');

```

6.3. Визуализация преобразований

javascript

// Визуализация цепочки преобразований

```

function visualizeConversion(value, targetType) {
    console.log(`Преобразование ${JSON.stringify(value)} (${typeof value}) → ${targetType}`);
    console.log("-".repeat(60));

    let currentValue = value;
    let step = 1;

    const steps = [];

    // Определить необходимые преобразования
    if (targetType === 'string') {
        if (typeof currentValue === 'object' && currentValue !== null) {
            steps.push(`${step++}. Проверить Symbol.toPrimitive`);
            steps.push(`${step++}. Если хинт 'string', вызвать его`);
            steps.push(`${step++}. Иначе вызвать toString`);
            steps.push(`${step++}. Если результат объект, вызвать valueOf`);
        } else {
            steps.push(`${step++}. Вызвать String() или toString`);
        }
    } else if (targetType === 'number') {
        if (typeof currentValue === 'object' && currentValue !== null) {
            steps.push(`${step++}. Проверить Symbol.toPrimitive`);
            steps.push(`${step++}. Если хинт 'number', вызвать его`);
            steps.push(`${step++}. Иначе вызвать valueOf`);
            steps.push(`${step++}. Если результат объект, вызвать toString`);
        }
    }
}

```

```

    steps.push(`${step++}. Преобразовать результат в число`);
  } else {
    steps.push(`${step++}. Вызвать Number() или унарный плюс`);
  }
} else if (targetType === 'boolean') {
  steps.push(`${step++}. Определить truthy/falsy значение`);
}

// Выполнить преобразование
let result;
try {
  if (targetType === 'string') {
    result = String(currentValue);
  } else if (targetType === 'number') {
    result = Number(currentValue);
  } else if (targetType === 'boolean') {
    result = Boolean(currentValue);
  }
} catch (error) {
  result = `Ошибка: ${error.message}`;
}

// Вывести шаги
steps.forEach(step => console.log(` ${step}`));
console.log();
console.log(`Результат: ${JSON.stringify(result)} (${typeof result})`);
console.log("=".repeat(60));
}

// Примеры
visualizeConversion(42, 'string');
visualizeConversion("3.14", 'number');
visualizeConversion([1, 2, 3], 'string');
visualizeConversion({x: 10}, 'number');
visualizeConversion("", 'boolean');

```

Заключение: лучшие практики динамической типизации

1. Ясность важнее краткости

javascript

// Плохо: неявное преобразование

```
const total = quantity * price; // Что если quantity строка?
```

// Хорошо: явное преобразование

```
const total = Number(quantity) * Number(price);
```

// Или лучше: проверка типов

```
if (typeof quantity !== 'number' || typeof price !== 'number') {  
  throw new TypeError('Параметры должны быть числами');  
}
```

2. Всегда используйте строгое сравнение

javascript

// Плохо

```
if (value == null) { } // Проверяет и null, и undefined
```

// Хорошо

```
if (value === null || value === undefined) { }
```

// Или современный вариант

```
if (value == null) { } // Приемлемо для проверки null/undefined
```

3. Обрабатывайте граничные случаи

javascript

```
function safeDivision(a, b) {  
  const numA = Number(a);  
  const numB = Number(b);  
  
  if (isNaN(numA) || isNaN(numB)) {  
    return NaN;  
  }  
  
  if (numB === 0) {
```

```

    return numA === 0 ? NaN : Infinity * Math.sign(numA);
}

return numA / numB;
}

```

4. Используйте современные операторы

```

javascript

// Вместо
const name = user && user.profile && user.profile.name;

// Используйте
const name = user?.profile?.name;

// Вместо
const displayName = username || "Гость";

// Используйте (если нужно различать "" и undefined)
const displayName = username ?? "Гость";

```

5. Документируйте ожидаемые типы

```

javascript

/**
 * Вычисляет сумму скидки
 * @param {number|string} price - Цена (число или строка)
 * @param {number|string} discount - Скидка в процентах
 * @returns {number} Сумма скидки
 * @throws {TypeError} Если параметры нельзя преобразовать в числа
 */
function calculateDiscount(price, discount) {
    const numPrice = Number(price);
    const numDiscount = Number(discount);

    if (isNaN(numPrice) || isNaN(numDiscount)) {
        throw new TypeError('Параметры должны быть числами');
    }

    return numPrice * (numDiscount / 100);
}

```

6. Пишите тесты для преобразований

javascript

// Тесты для критических функций

```
describe('Type conversions', () => {
  test('should handle string numbers', () => {
    expect(calculateDiscount("100", "10")).toBe(10);
  });

  test('should reject invalid inputs', () => {
    expect(() => calculateDiscount("text", 10)).toThrow(TypeError);
  });

  test('should handle edge cases', () => {
    expect(calculateDiscount(0, 10)).toBe(0);
    expect(calculateDiscount(100, 0)).toBe(0);
  });
});
```

Ключевые выводы:

- 1. JavaScript преобразует типы автоматически** — понимание правил обязательно
- 2. + ведет себя по-разному** — сложение vs конкатенация
- 3. == выполняет преобразование** — используйте === если не нужна гибкость
- 4. Объекты преобразуются к примитивам** — через Symbol.toPrimitive, valueOf(), toString()
- 5. null и undefined ведут себя по-разному** — null → 0, undefined → NaN
- 6. Пустые значения falsy** — 0, "", null, undefined, false, NaN
- 7. Современные операторы упрощают код** — ??, ?., &&=, ||=, ??=

Динамическая типизация — мощная особенность JavaScript, которая требует понимания и аккуратности. Правильное использование преобразований типов делает код гибким и выразительным, в то время как непонимание правил приводит к трудноотлавливаемым ошибкам.

4.6. Операторы typeof и instanceof

Введение: зачем нужны операторы проверки типов

В динамически типизированном языке JavaScript операторы проверки типов (`typeof` и `instanceof`) являются критически важными инструментами для:

1. **Определения типа значения** во время выполнения
 2. **Защиты от ошибок** при работе с динамическими данными
 3. **Реализации полиморфизма** — разного поведения для разных типов
 4. **Валидации входных данных** в функциях и API
 5. **Отладки и логирования** для понимания структуры данных
-

Часть 1: Оператор typeof

1.1. Базовый синтаксис и поведение

```
javascript

// Синтаксис: typeof operand или typeof(operand)
console.log(typeof 42); // "number"
console.log(typeof "текст"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null); // "object" (особенность!)
console.log(typeof Symbol("id")); // "symbol"
console.log(typeof 12345678901234567890n); // "bigint"
console.log(typeof {}); // "object"
console.log(typeof []); // "object" (массив тоже объект)
console.log(typeof function() {}); // "function"
console.log(typeof /regex/); // "object"
console.log(typeof new Date()); // "object"
console.log(typeof Math); // "object"

// Два синтаксически эквивалентных варианта
console.log(typeof 42); // "number"
console.log(typeof(42)); // "number" (скобки не обязательны)
```

1.2. Возвращаемые значения

```
javascript

// Полная таблица возвращаемых значений typeof
const typeExamples = [
  [undefined, "undefined"],
  [null, "object"], // Известная особенность
  [true, "boolean"],
  [false, "boolean"],
  [123, "number"],
  [3.14, "number"],
  [NaN, "number"], // NaN - специальное числовое значение
  [Infinity, "number"],
  ["текст", "string"],
  ["", "string"],
  [Symbol(), "symbol"],
  [Symbol("id"), "symbol"],
  [123n, "bigint"],
  [9007199254740991n, "bigint"],
  [{}], "object"],
  [{x: 1, y: 2}], "object"],
  [], "object"], // Массивы - объекты
  [1, 2, 3], "object"],
  [new Array(5), "object"],
  [function() {}, "function"],
  [() => {}, "function"], // Стрелочные функции тоже
  [class MyClass {}, "function"], // Классы - функции
  [Math.sin, "function"],
  [/regex/, "object"],
  [new RegExp("abc"), "object"],
  [new Date(), "object"],
  [new Error(), "object"],
  [new Map(), "object"],
  [new Set(), "object"],
  [new Promise(() => {}), "object"],
  [Math, "object"],
  [JSON, "object"],
  [globalThis, "object"]
];

// Вывод всех типов
console.log("Полная таблица типов:");
console.log("=".repeat(50));
```

```

typeExamples.forEach(([value, expectedType]) => {
  const actualType = typeof value;
  const match = actualType === expectedType ? "✓" : "✗";
  console.log(` ${match} ${JSON.stringify(value).padEnd(30)} -> ${actualType}`);
});

```

1.3. Особенности и граничные случаи

javascript

```

// 1. null возвращает "object" (историческая особенность)
console.log(typeof null); // "object" - самая известная особенность
console.log(null instanceof Object); // false - но null не экземпляр Object!

// 2. Функции возвращают "function", хотя являются объектами
function testFunction() {}
console.log(typeof testFunction); // "function"
console.log(typeof testFunction.call); // "function" (метод функции)
console.log(testFunction instanceof Object); // true (функции - объекты)

// 3. Необъявленные переменные (только в нестрогом режиме)
// console.log(typeof undeclaredVar); // "undefined" (не вызывает ReferenceError!)
// console.log(undeclaredVar); // ReferenceError: undeclaredVar is not
// defined

// 4. let/const переменные до инициализации (временная мертвая зона)
let letVar;
console.log(typeof letVar); // "undefined" (объявлена, но не
// инициализирована)

// console.log(typeof tdzVar); // ReferenceError (временная мертвая зона)
// let tdzVar = 42;

// 5. Глобальные объекты
console.log(typeof window); // "object" (в браузере)
console.log(typeof document); // "object" (в браузере)
console.log(typeof global); // "object" (в Node.js)
console.log(typeof globalThis); // "object" (универсально)

// 6. Примитивы-обёртки (object wrappers)
console.log(typeof new Number(42)); // "object" (объект-обёртка)
console.log(typeof Number(42)); // "number" (примитив)

console.log(typeof new String("текст")); // "object"

```

```
console.log(typeof String("текст"));      // "string"

console.log(typeof new Boolean(true));     // "object"
console.log(typeof Boolean(true));        // "boolean"
```

1.4. Практическое применение typeof

```
javascript

// 1. Валидация параметров функций
function processValue(value) {
    // Проверка типа перед использованием
    if (typeof value !== 'number') {
        throw new TypeError(`Ожидалось число, получено: ${typeof value}`);
    }
    return value * 2;
}

try {
    console.log(processValue(10));          // 20
    console.log(processValue("10"));       // TypeError
} catch (error) {
    console.error(error.message);
}

// 2. Обработка разных типов данных
function formatValue(value) {
    switch (typeof value) {
        case 'number':
            return `Число: ${value.toFixed(2)}`;
        case 'string':
            return `Строка: "${value}"`;
        case 'boolean':
            return `Логическое: ${value}`;
        case 'object':
            if (value === null) {
                return 'Null значение';
            }
            if (Array.isArray(value)) {
                return `Массив из ${value.length} элементов`;
            }
            return `Объект с ${Object.keys(value).length} свойствами`;
        default:
            return `Тип: ${typeof value}, значение: ${value}`;
    }
}
```

```

    }
}

console.log(formatValue(42));           // "Число: 42.00"
console.log(formatValue("Привет"));     // 'Строка: "Привет"'
console.log(formatValue(true));          // "Логическое: true"
console.log(formatValue(null));          // "Null значение"
console.log(formatValue([1, 2, 3]));     // "Массив из 3 элементов"
console.log(formatValue({x: 1, y: 2}));  // "Объект с 2 свойствами"

// 3. Защита от undefined
function safeAccess(obj, prop) {
    if (typeof obj !== 'object' || obj === null) {
        return undefined;
    }
    return obj[prop];
}

const data = { name: "Иван", age: 30 };
console.log(safeAccess(data, "name"));    // "Иван"
console.log(safeAccess(null, "name"));    // undefined (без ошибки)
console.log(safeAccess(42, "name"));      // undefined

// 4. Проверка существования функции перед вызовом
function callIfFunction(fn, ...args) {
    if (typeof fn === 'function') {
        return fn(...args);
    }
    console.warn(`Параметр не является функцией: ${typeof fn}`);
    return null;
}

function greet(name) { return `Привет, ${name}!`; }
console.log(callIfFunction(greet, "Анна")); // "Привет, Анна!"
console.log(callIfFunction("не функция", 42)); // Предупреждение, возвращает null

// 5. Определение среды выполнения
function detectEnvironment() {
    if (typeof window !== 'undefined' && typeof document !== 'undefined') {
        return 'browser';
    }
    if (typeof process !== 'undefined' && process.versions && process.versions.node) {
        return 'node';
    }
}

```

```

    if (typeof WorkerGlobalScope !== 'undefined' && self instanceof WorkerGlobalScope) {
        return 'web worker';
    }
    return 'unknown';
}

```

```

console.log(`Среда выполнения: ${detectEnvironment()}`);

```

1.5. Проблемы и ограничения typeof

javascript

// 1. Не различает массивы и объекты

```

console.log(typeof []);           // "object"
console.log(typeof {});          // "object"

```

// Решение: использовать Array.isArray()

```

console.log(Array.isArray([]));   // true
console.log(Array.isArray({}));   // false

```

// 2. Не различает разные типы объектов

```

console.log(typeof new Date());   // "object"
console.log(typeof /regex/);      // "object"
console.log(typeof new Map());    // "object"
console.log(typeof new Set());    // "object"

```

// Решение: использовать instanceof или конструктор

```

console.log(new Date() instanceof Date); // true
console.log(/regex/ instanceof RegExp);   // true

```

// 3. null определяется как "object"

```

console.log(typeof null);          // "object"

```

// Решение: явная проверка

```

function isNull(value) {
    return value === null;
}

```

// 4. Не различает примитивы и их объекты-обёртки

```

console.log(typeof "текст");       // "string"
console.log(typeof new String("текст")); // "object"

```

// Решение: использовать valueOf или явное преобразование

```

function isPrimitiveString(value) {

```

```
    return typeof value === 'string';
}

// 5. Не работает с классами ES6 как с отдельным типом
class MyClass {}
console.log(typeof MyClass);           // "function" (класс - функция)
console.log(typeof new MyClass());     // "object"

// 6. Для BigInt работает корректно только в новых версиях
console.log(typeof 42n);               // "bigint" (ES2020+)
```

Часть 2: Оператор instanceof

2.1. Базовый синтаксис и принцип работы

```
javascript

// Синтаксис: object instanceof constructor
// Проверяет, присутствует ли конструктор в цепочке прототипов объекта

function Animal(name) {
    this.name = name;
}

function Dog(name, breed) {
    Animal.call(this, name);
    this.breed = breed;
}

// Настройка прототипного наследования
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

// Создание экземпляров
const animal = new Animal("Животное");
const dog = new Dog("Бобик", "Овчарка");

// Проверки
console.log(dog instanceof Dog);           // true
console.log(dog instanceof Animal);        // true (Animal в цепочке прототипов)
console.log(dog instanceof Object);        // true (Object в цепочке прототипов)
```

```

console.log(animal instanceof Dog);           // false
console.log(animal instanceof Animal);        // true
console.log(animal instanceof Object);        // true

// Проверка примитивов
console.log("текст" instanceof String);       // false (примитив, не объект)
console.log(new String("текст") instanceof String); // true (объект-обёртка)

console.log(42 instanceof Number);            // false
console.log(new Number(42) instanceof Number); // true

console.log(true instanceof Boolean);          // false
console.log(new Boolean(true) instanceof Boolean); // true

```

2.2. Механизм работы instanceof

```

javascript

// Псевдокод алгоритма instanceof
function instanceof(obj, constructor) {
  // 1. Если obj не объект (кроме функций) или null, возвращаем false
  if (typeof obj !== 'object' || obj === null) {
    return false;
  }

  // 2. Получаем прототип объекта
  let proto = Object.getPrototypeOf(obj);

  // 3. Идем по цепочке прототипов
  while (proto !== null) {
    // 4. Если нашли конструктор в цепочке, возвращаем true
    if (proto === constructor.prototype) {
      return true;
    }
    // 5. Переходим к следующему прототипу
    proto = Object.getPrototypeOf(proto);
  }

  // 6. Если дошли до null и не нашли конструктор
  return false;
}

// Практическая реализация для демонстрации

```



```

function customInstanceOf(obj, constructor) {
  if (obj == null || typeof obj !== 'object') {
    return false;
  }

  let currentProto = Object.getPrototypeOf(obj);

  while (currentProto !== null) {
    if (currentProto === constructor.prototype) {
      return true;
    }
    currentProto = Object.getPrototypeOf(currentProto);
  }

  return false;
}

// Тестирование
function Person(name) {
  this.name = name;
}

function Employee(name, position) {
  Person.call(this, name);
  this.position = position;
}

Employee.prototype = Object.create(Person.prototype);
Employee.prototype.constructor = Employee;

const emp = new Employee("Иван", "Разработчик");

console.log(customInstanceOf(emp, Employee)); // true
console.log(customInstanceOf(emp, Person));   // true
console.log(customInstanceOf(emp, Object));    // true
console.log(customInstanceOf(emp, Array));     // false

```

2.3. Проверка встроенных типов

```

javascript

// Массивы
const arr = [1, 2, 3];
console.log(arr instanceof Array); // true

```

```

console.log(arr instanceof Object);           // true
console.log(Array.isArray(arr));              // true (предпочтительный способ)

// Даты
const date = new Date();
console.log(date instanceof Date);             // true
console.log(date instanceof Object);           // true

// Регулярные выражения
const regex = /test/gi;
console.log(regex instanceof RegExp);          // true
console.log(regex instanceof Object);          // true

// Ошибки
const error = new Error("Сообщение");
console.log(error instanceof Error);           // true
console.log(error instanceof Object);          // true

// Типизированные массивы
const typedArray = new Int32Array(10);
console.log(typedArray instanceof Int32Array); // true
console.log(typedArray instanceof Object);     // true

// Коллекции (Map, Set, WeakMap, WeakSet)
const map = new Map();
const set = new Set();
console.log(map instanceof Map);               // true
console.log(set instanceof Set);               // true
console.log(map instanceof Object);            // true

// Промисы
const promise = Promise.resolve(42);
console.log(promise instanceof Promise);       // true
console.log(promise instanceof Object);        // true

// Буферы
const buffer = new ArrayBuffer(16);
console.log(buffer instanceof ArrayBuffer);    // true
console.log(buffer instanceof Object);         // true

```

2.4. Наследование и цепочки прототипов

javascript

// Сложная иерархия наследования

```
class Vehicle {  
  constructor(type) {  
    this.type = type;  
  }  
  
  move() {  
    return `${this.type} движется`;  
  }  
}
```

```
class Car extends Vehicle {  
  constructor(brand, model) {  
    super("автомобиль");  
    this.brand = brand;  
    this.model = model;  
  }  
  
  honk() {  
    return "Би-бип!";  
  }  
}
```

```
class ElectricCar extends Car {  
  constructor(brand, model, batteryCapacity) {  
    super(brand, model);  
    this.batteryCapacity = batteryCapacity;  
  }  
  
  charge() {  
    return "Заряжается...";  
  }  
}
```

// Создание экземпляров

```
const tesla = new ElectricCar("Tesla", "Model S", 100);
```

// Проверки instanceof

```
console.log(tesla instanceof ElectricCar); // true  
console.log(tesla instanceof Car);         // true  
console.log(tesla instanceof Vehicle);     // true  
console.log(tesla instanceof Object);      // true
```

// Проверка прототипов

```

console.log(Object.getPrototypeOf(tesla) === ElectricCar.prototype); // true
console.log(Object.getPrototypeOf(Object.getPrototypeOf(tesla)) === Car.prototype); //
true
console.log(Object.getPrototypeOf(Object.getPrototypeOf(Object.getPrototypeOf(tesla)))
=== Vehicle.prototype); // true

// Все прототипы в цепочке
let proto = tesla;
console.log("Цепочка прототипов:");
while (proto) {
  console.log(` ${proto.constructor.name}`);
  proto = Object.getPrototypeOf(proto);
}
// ElectricCar → Car → Vehicle → Object → null

```

2.5. Практическое применение instanceof

```

javascript

// 1. Валидация параметров с проверкой типа
function processVehicle(vehicle) {
  if (!(vehicle instanceof Vehicle)) {
    throw new TypeError('Ожидается экземпляр Vehicle');
  }

  if (vehicle instanceof ElectricCar) {
    console.log(`Электромобиль ${vehicle.brand} ${vehicle.model}`);
    console.log(vehicle.charge());
  } else if (vehicle instanceof Car) {
    console.log(`Автомобиль ${vehicle.brand} ${vehicle.model}`);
    console.log(vehicle.honk());
  } else if (vehicle instanceof Vehicle) {
    console.log(`Транспорт: ${vehicle.type}`);
    console.log(vehicle.move());
  }

  return vehicle;
}

const myCar = new Car("Toyota", "Camry");
const myElectric = new ElectricCar("Tesla", "Model 3", 75);

try {
  processVehicle(myCar);
}

```

```

    processVehicle(myElectric);
    processVehicle({ type: "велосипед" }); // TypeError
} catch (error) {
    console.error(error.message);
}

// 2. Полиморфизм — разные обработчики для разных типов
class Shape {
    area() {
        throw new Error("Метод area должен быть переопределен");
    }
}

class Circle extends Shape {
    constructor(radius) {
        super();
        this.radius = radius;
    }

    area() {
        return Math.PI * this.radius ** 2;
    }
}

class Rectangle extends Shape {
    constructor(width, height) {
        super();
        this.width = width;
        this.height = height;
    }

    area() {
        return this.width * this.height;
    }
}

function calculateTotalArea(shapes) {
    return shapes.reduce((total, shape) => {
        if (!(shape instanceof Shape)) {
            throw new TypeError('Все элементы должны быть Shape');
        }
        return total + shape.area();
    }, 0);
}

```

```
const shapes = [
  new Circle(5),
  new Rectangle(4, 6),
  new Circle(3)
];

console.log(`Общая площадь: ${calculateTotalArea(shapes).toFixed(2)}`);

// 3. Фабрика объектов с проверкой типов
class Logger {
  log(message) {
    console.log(`LOG: ${message}`);
  }
}

class FileLogger extends Logger {
  constructor(filename) {
    super();
    this.filename = filename;
  }

  log(message) {
    console.log(`[FILE: ${this.filename}] ${message}`);
  }
}

class DatabaseLogger extends Logger {
  constructor(connectionString) {
    super();
    this.connectionString = connectionString;
  }

  log(message) {
    console.log(`[DATABASE] ${message}`);
  }
}

function createLogger(type, config) {
  switch (type) {
    case 'file':
      return new FileLogger(config.filename);
    case 'database':
      return new DatabaseLogger(config.connectionString);
  }
}
```

```

    case 'console':
        return new Logger();
    default:
        throw new Error(`Неизвестный тип логгера: ${type}`);
}
}

function useLogger(logger) {
    if (!(logger instanceof Logger)) {
        throw new TypeError('Логгер должен быть экземпляром Logger');
    }

    logger.log("Тестовое сообщение");

    // Дополнительные возможности для специфичных логгеров
    if (logger instanceof FileLogger) {
        console.log(`Логирование в файл: ${logger.filename}`);
    } else if (logger instanceof DatabaseLogger) {
        console.log(`Подключение к БД: ${logger.connectionString}`);
    }
}

const fileLogger = createLogger('file', { filename: 'app.log' });
useLogger(fileLogger);

```

2.6. Проблемы и ограничения instanceof

```

javascript

// 1. Не работает с примитивами
console.log("текст" instanceof String);    // false
console.log(42 instanceof Number);         // false
console.log(true instanceof Boolean);       // false

// 2. Зависит от контекста (iframe, разные realms)
const iframe = document.createElement('iframe');
document.body.appendChild(iframe);

const iframeArray = iframe.contentWindow.Array;
const iframeInstance = new iframeArray();

console.log(iframeInstance instanceof Array);    // false (в большинстве браузеров)
console.log(iframeInstance instanceof iframeArray); // true
console.log(Array.isArray(iframeInstance));      // true (кросс-фреймовый способ)

```

```

// 3. Можно обмануть, изменив prototype
function Animal() {}
function Dog() {}

const animal = new Animal();

// Обычная проверка
console.log(animal instanceof Animal); // true
console.log(animal instanceof Dog);    // false

// Меняем prototype (опасно!)
Animal.prototype = Dog.prototype;
console.log(animal instanceof Animal); // false! (цепочка прототипов изменилась)
console.log(animal instanceof Dog);    // true! (теперь Animal.prototype ===
Dog.prototype)

// 4. Не различает литералы объектов и экземпляры
const objLiteral = { x: 1, y: 2 };
const objConstructed = new Object();

console.log(objLiteral instanceof Object); // true
console.log(objConstructed instanceof Object); // true

// 5. Проблемы с bound функциями
function Person(name) {
  this.name = name;
}

const BoundPerson = Person.bind(null, "Иван");
const person = new BoundPerson();

console.log(person instanceof Person); // true
console.log(person instanceof BoundPerson); // true (в современных браузерах)

// 6. Symbol.hasInstance для кастомной логики
class MySpecialArray {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance) &&
      instance.length > 0 &&
      typeof instance[0] === 'string';
  }
}

```



```
console.log(["текст", "еще"] instanceof MySpecialArray); // true
console.log([1, 2, 3] instanceof MySpecialArray); // false
console.log([] instanceof MySpecialArray); // false
```

Часть 3: Сравнение typeof и instanceof

3.1. Когда что использовать

```
javascript

// Сравнительная таблица
const comparison = [
  {
    value: "текст",
    typeof: typeof "текст",
    "String instanceof": "текст" instanceof String,
    "Object instanceof": "текст" instanceof Object,
    recommendation: "typeof для примитивов"
  },
  {
    value: new String("текст"),
    typeof: typeof new String("текст"),
    "String instanceof": new String("текст") instanceof String,
    "Object instanceof": new String("текст") instanceof Object,
    recommendation: "instanceof для объектов-обёрток"
  },
  {
    value: 42,
    typeof: typeof 42,
    "Number instanceof": 42 instanceof Number,
    "Object instanceof": 42 instanceof Object,
    recommendation: "typeof для чисел"
  },
  {
    value: [],
    typeof: typeof [],
    "Array instanceof": [] instanceof Array,
    "Object instanceof": [] instanceof Object,
    recommendation: "Array.isArray() для массивов"
  },
  {
```

```

    value: {},
    typeof: typeof {},
    "Object instanceof": {} instanceof Object,
    "Array instanceof": {} instanceof Array,
    recommendation: "typeof для общих объектов"
  },
  {
    value: null,
    typeof: typeof null,
    "Object instanceof": null instanceof Object,
    recommendation: "value === null для null"
  },
  {
    value: undefined,
    typeof: typeof undefined,
    "Object instanceof": undefined instanceof Object,
    recommendation: "typeof или value === undefined"
  },
  {
    value: function() {},
    typeof: typeof function() {},
    "Function instanceof": (function() {}) instanceof Function,
    "Object instanceof": (function() {}) instanceof Object,
    recommendation: "typeof для функций"
  },
  {
    value: new Date(),
    typeof: typeof new Date(),
    "Date instanceof": new Date() instanceof Date,
    "Object instanceof": new Date() instanceof Object,
    recommendation: "instanceof для конкретных типов объектов"
  }
];

```

```

console.log("Сравнение typeof и instanceof:");
console.log("=".repeat(100));
console.log("Значение".padEnd(25) + "typeof".padEnd(15) +
  "instanceof String".padEnd(20) + "instanceof Object".padEnd(20) + "Рекомендация");
console.log("-".repeat(100));

```

```

comparison.forEach(row => {
  console.log(
    JSON.stringify(row.value).padEnd(25) +
    row.typeof.padEnd(15) +

```

```

    String(row["String instanceof"]).padEnd(20) +
    String(row["Object instanceof"]).padEnd(20) +
    row.recommendation
  );
});

```

3.2. Практические рекомендации

javascript

// 1. Для проверки примитивов – используйте typeof

```

function isPrimitive(value) {
  return (
    typeof value === 'string' ||
    typeof value === 'number' ||
    typeof value === 'boolean' ||
    typeof value === 'symbol' ||
    typeof value === 'bigint' ||
    typeof value === 'undefined' ||
    value === null
  );
}

```

// 2. Для проверки массивов – используйте Array.isArray()

```

function safeArrayOperation(arr) {
  if (!Array.isArray(arr)) {
    throw new TypeError('Ожидается массив');
  }
  return arr.map(x => x * 2);
}

```

// 3. Для проверки null/undefined – используйте строгое сравнение

```

function handleNullable(value) {
  if (value === null) {
    return 'Значение null';
  }
  if (value === undefined) {
    return 'Значение undefined';
  }
  return `Значение: ${value}`;
}

```

// 4. Для проверки функций – используйте typeof

```

function callIfFunction(fn, ...args) {

```

```

    if (typeof fn === 'function') {
        return fn(...args);
    }
    return null;
}

```

// 5. Для проверки экземпляров классов – используйте instanceof

```

class Validator {
    validate(value) {
        throw new Error('Метод validate должен быть реализован');
    }
}

```

```

class EmailValidator extends Validator {
    validate(value) {
        return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
    }
}

```

```

function useValidator(validator, value) {
    if (!(validator instanceof Validator)) {
        throw new TypeError('Ожидается экземпляр Validator');
    }
    return validator.validate(value);
}

```

// 6. Для проверки объектов общего вида – используйте typeof и проверку на null

```

function isPlainObject(value) {
    return typeof value === 'object' &&
        value !== null &&
        !Array.isArray(value) &&
        !(value instanceof Date) &&
        !(value instanceof RegExp) &&
        value.constructor === Object;
}

```

// 7. Универсальная функция определения типа

```

function getDetailedType(value) {
    // Примитивные типы
    if (value === null) return 'null';
    if (value === undefined) return 'undefined';

    const type = typeof value;
    if (type !== 'object' && type !== 'function') {

```

```

    return type;
}

// Объекты и функции
if (type === 'function') {
    if (value.toString().startsWith('class ')) {
        return 'class';
    }
    return 'function';
}

// Объекты
if (Array.isArray(value)) return 'array';
if (value instanceof Date) return 'date';
if (value instanceof RegExp) return 'regexp';
if (value instanceof Map) return 'map';
if (value instanceof Set) return 'set';
if (value instanceof Promise) return 'promise';
if (value instanceof Error) return 'error';
if (Buffer && value instanceof Buffer) return 'buffer';
if (ArrayBuffer.isView(value)) return 'typed-array';

// Объекты-обёртки
if (value instanceof String) return 'string-object';
if (value instanceof Number) return 'number-object';
if (value instanceof Boolean) return 'boolean-object';

// Простой объект
if (Object.getPrototypeOf(value) === Object.prototype) {
    return 'object';
}

// Пользовательский класс
return `instanceof ${value.constructor.name}`;
}

// Тестирование функции
console.log(getDetailedType(null));           // "null"
console.log(getDetailedType(undefined));       // "undefined"
console.log(getDetailedType(42));              // "number"
console.log(getDetailedType("текст"));         // "string"
console.log(getDetailedType(true));            // "boolean"
console.log(getDetailedType(Symbol()));        // "symbol"
console.log(getDetailedType(42n));             // "bigint"

```

```

console.log(getDetailedType([]));           // "array"
console.log(getDetailedType({}));           // "object"
console.log(getDetailedType(new Date()));   // "date"
console.log(getDetailedType(/regex/));      // "regexp"
console.log(getDetailedType(() => {}));      // "function"
console.log(getDetailedType(class MyClass {})); // "class"
console.log(getDetailedType(new Map()));    // "map"

```

3.3. Комбинированные проверки

javascript

```

// 1. Безопасный доступ к свойствам
function safeGet(obj, path, defaultValue) {
  // Проверяем, что obj - объект или массив
  if (typeof obj !== 'object' || obj === null) {
    return defaultValue;
  }

  const keys = Array.isArray(path) ? path : path.split('.');
  let current = obj;

  for (const key of keys) {
    // Проверяем существование свойства
    if (current == null || typeof current !== 'object') {
      return defaultValue;
    }

    // Для массивов проверяем индекс
    if (Array.isArray(current) && !isNaN(key)) {
      const index = parseInt(key, 10);
      if (index < 0 || index >= current.length) {
        return defaultValue;
      }
      current = current[index];
    } else if (key in current) {
      current = current[key];
    } else {
      return defaultValue;
    }
  }

  return current;
}

```

```

const data = {
  user: {
    name: "Иван",
    profile: {
      email: "ivan@example.com",
      addresses: [
        { city: "Москва", street: "Ленина" },
        { city: "Санкт-Петербург", street: "Невский" }
      ]
    }
  }
};

```

```

console.log(safeGet(data, 'user.profile.email')); // "ivan@example.com"
console.log(safeGet(data, 'user.profile.addresses.0.city')); // "Москва"
console.log(safeGet(data, 'user.profile.phone', 'не указан')); // "не указан"
console.log(safeGet(null, 'user.name', 'нет данных')); // "нет данных"

```

// 2. Универсальный валидатор типов

```

class TypeValidator {
  static isString(value) {
    return typeof value === 'string';
  }

  static isNumber(value) {
    return typeof value === 'number' && !isNaN(value);
  }

  static isInteger(value) {
    return Number.isInteger(value);
  }

  static isBoolean(value) {
    return typeof value === 'boolean';
  }

  static isArray(value) {
    return Array.isArray(value);
  }

  static isObject(value) {
    return typeof value === 'object' && value !== null && !Array.isArray(value);
  }
}

```

```
static isFunction(value) {
  return typeof value === 'function';
}

static isDate(value) {
  return value instanceof Date;
}

static isPromise(value) {
  return value instanceof Promise;
}

static isInstanceOf(value, constructor) {
  return value instanceof constructor;
}

static validate(value, rules) {
  const errors = [];

  for (const [rule, message] of Object.entries(rules)) {
    let isValid = false;

    switch (rule) {
      case 'required':
        isValid = value !== undefined && value !== null;
        break;
      case 'string':
        isValid = this.isString(value);
        break;
      case 'number':
        isValid = this.isNumber(value);
        break;
      case 'integer':
        isValid = this.isInteger(value);
        break;
      case 'array':
        isValid = this.isArray(value);
        break;
      case 'object':
        isValid = this.isObject(value);
        break;
      case 'min':
        isValid = typeof value === 'number' && value >= rules.min;
    }
  }
}
```



```

        break;
    case 'max':
        isValid = typeof value === 'number' && value <= rules.max;
        break;
    case 'minLength':
        isValid = (this.isString(value) || this.isArray(value)) &&
            value.length >= rules.minLength;
        break;
    case 'maxLength':
        isValid = (this.isString(value) || this.isArray(value)) &&
            value.length <= rules.maxLength;
        break;
    case 'pattern':
        isValid = this.isString(value) && rules.pattern.test(value);
        break;
    case 'instanceof':
        isValid = value instanceof rules.instanceof;
        break;
    default:
        if (typeof rules[rule] === 'function') {
            isValid = rules[rule](value);
        }
    }
}

if (!isValid) {
    errors.push(message || `Проверка ${rule} не пройдена`);
}

return errors;
}
}

```

// Использование валидатора

```

const userInput = {
    name: "Иван",
    age: "30", // должно быть числом
    email: "ivan@example",
    tags: ["js", "node"]
};

```

```

const rules = {
    required: "Обязательное поле",
    string: "Должно быть строкой",

```

```

    minLength: [3, "Минимум 3 символа"],
    maxLength: [50, "Максимум 50 символов"],
    pattern: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Неверный email"],
    age: {
      number: "Должно быть числом",
      min: [18, "Минимум 18 лет"],
      max: [100, "Максимум 100 лет"]
    },
    tags: {
      array: "Должен быть массивом",
      minLength: [1, "Хотя бы один тег"]
    }
  };

  // Проверка полей
  const nameErrors = TypeValidator.validate(userInput.name, {
    required: true,
    string: true,
    minLength: [3, "Имя слишком короткое"]
  });

  const emailErrors = TypeValidator.validate(userInput.email, {
    required: true,
    string: true,
    pattern: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Неверный формат email"]
  });

  console.log("Ошибки имени:", nameErrors);           // []
  console.log("Ошибки email:", emailErrors);           // ["Неверный формат email"]

```

Часть 4: Продвинутые техники и паттерны

4.1. Symbol.toStringTag и кастомные типы

```

javascript

// Symbol.toStringTag позволяет кастомизировать вывод typeof-like поведения
class MyCollection {
  constructor() {
    this.items = [];
  }
}

```

```

add(item) {
    this.items.push(item);
}

// Кастомизация вывода Object.prototype.toString
get [Symbol.toStringTag]() {
    return 'MyCollection';
}

// Кастомизация поведения instanceof через Symbol.hasInstance
static [Symbol.hasInstance](instance) {
    return instance &&
        typeof instance === 'object' &&
        typeof instance.add === 'function' &&
        Array.isArray(instance.items);
}
}

const collection = new MyCollection();

// Стандартное поведение
console.log(typeof collection); // "object"
console.log(collection instanceof MyCollection); // true (благодаря Symbol.hasInstance)
console.log(Object.prototype.toString.call(collection)); // "[object MyCollection]"

// Утилита для определения типа с поддержкой Symbol.toStringTag
function getType(value) {
    if (value === null) return 'null';
    if (value === undefined) return 'undefined';

    const type = typeof value;
    if (type !== 'object' && type !== 'function') {
        return type;
    }

    // Используем Symbol.toStringTag если есть
    const tag = value[Symbol.toStringTag];
    if (tag) {
        return tag.toLowerCase();
    }

    // Стандартное определение
    const toStringResult = Object.prototype.toString.call(value);

```

```

    return toStringResult.slice(8, -1).toLowerCase();
}

console.log(getType(collection));           // "mycollection"
console.log(getType(new Date()));           // "date"
console.log(getType([]));                   // "array"
console.log(getType(new Map()));            // "map"

```

4.2. Duck typing вместо instanceof

javascript

*// Duck typing: "Если что-то выглядит как утка, плавает как утка и крякает как утка,
// то это, вероятно, утка"*

// Вместо проверки типа через instanceof, проверяем наличие необходимых методов/свойств

// Классический подход с instanceof

```

class Logger {
  log(message) {
    console.log(message);
  }
}

```

```

function logWithInstanceof(logger, message) {
  if (!(logger instanceof Logger)) {
    throw new TypeError('Ожидается экземпляр Logger');
  }
  logger.log(message);
}

```

// Подход с duck typing

```

function logWithDuckTyping(logger, message) {
  // Проверяем наличие метода log, а не тип
  if (typeof logger !== 'object' || logger === null || typeof logger.log !== 'function') {
    throw new TypeError('Объект должен иметь метод log');
  }
  logger.log(message);
}

```

// Объекты, которые работают с обоими подходами

```

class ConsoleLogger {
  log(message) {
    console.log(`[CONSOLE] ${message}`);
  }
}

```

```

    }
}

class FileLogger {
  constructor(filename) {
    this.filename = filename;
  }

  log(message) {
    console.log(`[FILE: ${this.filename}] ${message}`);
  }
}

// Простой объект с методом log (не экземпляр какого-либо класса)
const simpleLogger = {
  log(message) {
    console.log(`[SIMPLE] ${message}`);
  }
};

// Тестирование
const consoleLogger = new ConsoleLogger();
const fileLogger = new FileLogger('app.log');

// instanceof работает только с Logger и его наследниками
// logWithInstanceof(consoleLogger, "test"); // TypeError

// Duck typing работает со всеми объектами, имеющими метод log
logWithDuckTyping(consoleLogger, "test 1"); // Работает
logWithDuckTyping(fileLogger, "test 2");    // Работает
logWithDuckTyping(simpleLogger, "test 3");  // Работает

// Универсальная функция duck typing проверки
function implementsInterface(obj, interfaceSpec) {
  if (typeof obj !== 'object' || obj === null) {
    return false;
  }

  for (const [key, type] of Object.entries(interfaceSpec)) {
    if (!(key in obj)) {
      return false;
    }

    const value = obj[key];

```

```

switch (type) {
  case 'function':
    if (typeof value !== 'function') return false;
    break;
  case 'number':
    if (typeof value !== 'number' || isNaN(value)) return false;
    break;
  case 'string':
    if (typeof value !== 'string') return false;
    break;
  case 'boolean':
    if (typeof value !== 'boolean') return false;
    break;
  case 'array':
    if (!Array.isArray(value)) return false;
    break;
  case 'object':
    if (typeof value !== 'object' || value === null) return false;
    break;
  default:
    if (typeof type === 'function' && !(value instanceof type)) {
      return false;
    }
}
}

return true;
}

```

// Определение интерфейса через duck typing

```

const loggerInterface = {
  log: 'function',
  level: 'string',
  enabled: 'boolean'
};

```

```

const validLogger = {
  log: (msg) => console.log(msg),
  level: 'info',
  enabled: true
};

```

```

const invalidLogger = {
  log: 'not a function', // должно быть функцией

```

```

    level: 'info',
    enabled: true
};

console.log(implementsInterface(validLogger, loggerInterface)); // true
console.log(implementsInterface(invalidLogger, loggerInterface)); // false

```

4.3. Type guards в TypeScript (для сравнения)

typescript

*// В TypeScript type guards – это функции, которые помогают TypeScript
// определить тип переменной во время выполнения*

// JavaScript-версии type guards

```

function isString(value: any): value is string {
    return typeof value === 'string';
}

function isNumber(value: any): value is number {
    return typeof value === 'number' && !isNaN(value);
}

function isArray(value: any): value is any[] {
    return Array.isArray(value);
}

function isDate(value: any): value is Date {
    return value instanceof Date;
}

function isPromise(value: any): value is Promise<any> {
    return value instanceof Promise;
}

// Использование type guards
function processValue(value: unknown) {
    if (isString(value)) {
        // Здесь TypeScript знает, что value – string
        console.log(`Строка: ${value.toUpperCase()}`);
    } else if (isNumber(value)) {
        // Здесь TypeScript знает, что value – number
        console.log(`Число: ${value.toFixed(2)}`);
    } else if (isArray(value)) {

```

```

    // Здесь TypeScript знает, что value — array
    console.log(`Массив длиной ${value.length}`);
  } else if (isDate(value)) {
    // Здесь TypeScript знает, что value — Date
    console.log(`Дата: ${value.toISOString()}`);
  } else {
    console.log(`Неизвестный тип: ${typeof value}`);
  }
}

// Пользовательские type guards
class User {
  constructor(public name: string, public age: number) {}
}

function isUser(value: any): value is User {
  return value instanceof User ||
    (typeof value === 'object' &&
      value !== null &&
      typeof value.name === 'string' &&
      typeof value.age === 'number');
}

function processUser(value: unknown) {
  if (isUser(value)) {
    console.log(`Пользователь: ${value.name}, ${value.age} лет`);
  } else {
    console.log('Не пользователь');
  }
}

```

4.4. Бенчмаркинг производительности

javascript

```

// Сравнение производительности разных способов проверки типов
function benchmark() {
  const iterations = 1000000;
  const testValues = [
    "строка",
    42,
    true,
    null,
    undefined,
  ]
}

```



```
[],  
{},  
new Date(),  
function() {},  
/regex/  
];
```

```
const tests = [  
  {  
    name: "typeof",  
    fn: (value) => typeof value  
  },  
  {  
    name: "instanceof Array",  
    fn: (value) => value instanceof Array  
  },  
  {  
    name: "Array.isArray",  
    fn: (value) => Array.isArray(value)  
  },  
  {  
    name: "Object.prototype.toString",  
    fn: (value) => Object.prototype.toString.call(value)  
  },  
  {  
    name: "constructor check",  
    fn: (value) => value && value.constructor === Array  
  },  
  {  
    name: "=== null",  
    fn: (value) => value === null  
  },  
  {  
    name: "=== undefined",  
    fn: (value) => value === undefined  
  }  
];
```

```
console.log(`Бенчмарк (${iterations} итераций):`);  
console.log("=".repeat(60));
```

```
for (const test of tests) {  
  const start = performance.now();
```

```

for (let i = 0; i < iterations; i++) {
  for (const value of testValues) {
    test.fn(value);
  }
}

const end = performance.now();
const duration = end - start;

console.log(`${test.name.padEnd(30)} ${duration.toFixed(2)}ms`);
}
}

```

```

// Запуск бенчмарка (раскомментируйте для запуска)
// benchmark();

```

```

/*
Примерные результаты (зависят от движка и окружения):

```

<code>typeof</code>	45.23ms	
<code>instanceof Array</code>	78.45ms	
<code>Array.isArray</code>	52.67ms	← самый быстрый для массивов
<code>Object.prototype.toString</code>	95.12ms	
<code>constructor check</code>	65.34ms	
<code>=== null</code>	22.15ms	← самый быстрый для null
<code>=== undefined</code>	21.87ms	← самый быстрый для undefined

Выводы:

1. Для простых проверок (`null`, `undefined`) используйте строгое сравнение
 2. Для массивов используйте `Array.isArray()` – быстрее `instanceof`
 3. `typeof` – самый быстрый общий способ для примитивов
 4. `Object.prototype.toString` – самый медленный, но самый точный
- */
-

Часть 5: Реальные примеры и лучшие практики

5.1. Библиотека валидации типов

javascript

```

// Полноценная библиотека для проверки типов

```

```
class TypeChecker {
  // Базовые проверки
  static isDefined(value) {
    return value !== undefined;
  }

  static isNull(value) {
    return value === null;
  }

  static isNil(value) {
    return value == null; // null and undefined
  }

  static isBoolean(value) {
    return typeof value === 'boolean';
  }

  static isNumber(value) {
    return typeof value === 'number' && !isNaN(value);
  }

  static isInteger(value) {
    return Number.isInteger(value);
  }

  static isSafeInteger(value) {
    return Number.isSafeInteger(value);
  }

  static isString(value) {
    return typeof value === 'string';
  }

  static isSymbol(value) {
    return typeof value === 'symbol';
  }

  static isBigInt(value) {
    return typeof value === 'bigint';
  }

  static isFunction(value) {
    return typeof value === 'function';
  }
}
```

```
}
```

```
static isObject(value) {  
    return typeof value === 'object' && value !== null;  
}
```

```
static isPlainObject(value) {  
    return this.isObject(value) &&  
        Object.getPrototypeOf(value) === Object.prototype;  
}
```

```
static isArray(value) {  
    return Array.isArray(value);  
}
```

```
static isArrayLike(value) {  
    return this.isObject(value) &&  
        'length' in value &&  
        typeof value.length === 'number' &&  
        value.length >= 0;  
}
```

```
static isDate(value) {  
    return value instanceof Date;  
}
```

```
static isRegExp(value) {  
    return value instanceof RegExp;  
}
```

```
static isError(value) {  
    return value instanceof Error;  
}
```

```
static isMap(value) {  
    return value instanceof Map;  
}
```

```
static isSet(value) {  
    return value instanceof Set;  
}
```

```
static isPromise(value) {  
    return value instanceof Promise;  
}
```

```
}
```

```
static isArray(value) {  
    return ArrayBuffer.isView(value) && !(value instanceof DataView);  
}
```

```
static isBuffer(value) {  
    return Buffer && Buffer.isBuffer ? Buffer.isBuffer(value) : false;  
}
```

```
// Комплексные проверки
```

```
static isPrimitive(value) {  
    return (  
        value === null ||  
        value === undefined ||  
        typeof value === 'boolean' ||  
        typeof value === 'number' ||  
        typeof value === 'string' ||  
        typeof value === 'symbol' ||  
        typeof value === 'bigint'  
    );  
}
```

```
static isEmpty(value) {  
    if (this.isNil(value)) return true;  
    if (this.isString(value)) return value.length === 0;  
    if (this.isArray(value)) return value.length === 0;  
    if (this.isObject(value)) return Object.keys(value).length === 0;  
    if (this.isMap(value) || this.isSet(value)) return value.size === 0;  
    return false;  
}
```

```
static isEqualType(a, b) {  
    // Учитывает null и typeof особенности  
    if (a === null && b === null) return true;  
    if (a === null || b === null) return false;
```

```
    const typeA = typeof a;  
    const typeB = typeof b;
```

```
    if (typeA !== typeB) return false;
```

```
// Для объектов проверяем конструктор
```

```
    if (typeA === 'object') {
```

```

    return a.constructor === b.constructor;
}

return true;
}

// Валидация с сообщениями об ошибках
static validate(value, rules) {
    const errors = [];

    for (const [rule, options] of Object.entries(rules)) {
        const { test, message, required = true } = typeof options === 'object'
            ? options
            : { test: options, message: `Проверка ${rule} не пройдена` };

        // Проверка required
        if (required && this.isNil(value)) {
            errors.push(`Значение обязательно`);
            break;
        }

        // Пропускаем проверку если значение не required и отсутствует
        if (!required && this.isNil(value)) {
            continue;
        }

        // Выполняем проверку
        let isValid = false;

        if (typeof test === 'function') {
            isValid = test(value);
        } else if (typeof test === 'string') {
            // Встроенные проверки
            const checkerName = `is${test.charAt(0).toUpperCase()}${test.slice(1)}`;
            if (this[checkerName]) {
                isValid = this[checkerName](value);
            }
        } else if (test instanceof RegExp) {
            isValid = this.isString(value) && test.test(value);
        } else if (Array.isArray(test)) {
            isValid = test.includes(value);
        } else {
            isValid = value === test;
        }
    }
}

```

```
    if (!isValid) {
      errors.push(message);
    }
  }
}
```

```
return {
  valid: errors.length === 0,
  errors,
  value
};
}
```

// Пример использования библиотеки

```
const userSchema = {
  name: {
    test: 'string',
    message: 'Имя должно быть строкой',
    required: true
  },
  age: {
    test: (value) => TypeChecker.isInteger(value) && value >= 0 && value <= 150,
    message: 'Возраст должен быть целым числом от 0 до 150',
    required: false
  },
  email: {
    test: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
    message: 'Неверный формат email',
    required: true
  },
  tags: {
    test: 'array',
    message: 'Теги должны быть массивом',
    required: false
  }
};
```

```
function validateUser(data) {
  const result = {};
  const errors = [];

  for (const [field, rules] of Object.entries(userSchema)) {
    const validation = TypeChecker.validate(data[field], rules);
```

```

    if (validation.valid) {
        result[field] = validation.value;
    } else {
        errors.push({ field, errors: validation.errors });
    }
}

return {
    valid: errors.length === 0,
    data: result,
    errors
};
}

const testData = {
    name: "Иван",
    age: 30,
    email: "ivan@example.com",
    tags: ["js", "node"]
};

const validationResult = validateUser(testData);
console.log(validationResult.valid ? "Данные валидны" : "Ошибки валидации");
if (!validationResult.valid) {
    console.log(validationResult.errors);
}

```

5.2. Защитное программирование с проверками типов

```

javascript

// Обертка для безопасного выполнения операций
class SafeExecutor {
    static tryExecute(fn, ...args) {
        try {
            // Проверяем, что fn - функция
            if (!TypeChecker.isFunction(fn)) {
                throw new TypeError('Первый аргумент должен быть функцией');
            }

            return {
                success: true,
                result: fn(...args),
                error: null
            };
        } catch (error) {
            return {
                success: false,
                result: null,
                error
            };
        }
    }
}

```



```

    };
  } catch (error) {
    return {
      success: false,
      result: null,
      error: {
        message: error.message,
        name: error.name,
        stack: error.stack
      }
    };
  }
}

```

```

static withTypeChecking(fn, expectedTypes) {
  return function(...args) {
    // Проверяем типы аргументов
    for (let i = 0; i < expectedTypes.length; i++) {
      const expected = expectedTypes[i];
      const actual = args[i];

      if (expected === 'any') continue;

      let isValid = false;

      if (typeof expected === 'function') {
        // Проверка через конструктор/класс
        isValid = actual instanceof expected;
      } else if (typeof expected === 'string') {
        // Проверка через typeof или встроенные проверки
        const checkerName = `is${expected.charAt(0).toUpperCase()}${expected.slice(1)}`;
        if (TypeChecker[checkerName]) {
          isValid = TypeChecker[checkerName](actual);
        } else {
          isValid = typeof actual === expected;
        }
      } else if (Array.isArray(expected)) {
        // Допустимые значения
        isValid = expected.includes(actual);
      } else if (expected instanceof RegExp) {
        // Регулярное выражение
        isValid = TypeChecker.isString(actual) && expected.test(actual);
      }
    }
  }
}

```

```

    if (!isValid) {
        throw new TypeError(
            `Аргумент ${i} должен быть ${expected}, получено ${typeof actual}`
        );
    }
}

return fn(...args);
};
}

static retryOnError(fn, maxRetries = 3, delay = 100) {
    return async function(...args) {
        let lastError;

        for (let attempt = 1; attempt <= maxRetries; attempt++) {
            try {
                return await fn(...args);
            } catch (error) {
                lastError = error;

                // Повторяем только TypeError
                if (!(error instanceof TypeError)) {
                    throw error;
                }

                if (attempt < maxRetries) {
                    console.warn(`TypeError при попытке ${attempt}, повтор через ${delay}ms`);
                    await new Promise(resolve => setTimeout(resolve, delay));
                    delay *= 2; // Экспоненциальная задержка
                }
            }
        }

        throw lastError;
    };
}

// Пример использования
const unsafeDivide = (a, b) => a / b;

const safeDivide = SafeExecutor.withTypeChecking(unsafeDivide, ['number', 'number']);

```

```
console.log(SafeExecutor.tryExecute(safeDivide, 10, 2)); // {success: true, result: 5}
console.log(SafeExecutor.tryExecute(safeDivide, "10", 2)); // {success: false,
error: ...}
```

// Функция с повторными попытками

```
const flakyApiCall = async (url) => {
  // Имитация API, который иногда возвращает не тот тип
  const response = Math.random() > 0.5
    ? { data: { users: [] } }
    : "не JSON строка";

  if (typeof response === 'string') {
    throw new TypeError('API вернул строку вместо объекта');
  }

  return response.data.users;
};

const reliableApiCall = SafeExecutor.retryOnTypeError(flakyApiCall, 3);

// reliableApiCall('/api/users').then(console.log).catch(console.error);
```

5.3. Сериализация с сохранением типов

javascript

// Система сериализации, которая сохраняет информацию о типах

```
class TypeAwareSerializer {
  static typeRegistry = new Map();

  // Регистрация кастомных типов
  static registerType(constructor, typeName = constructor.name) {
    this.typeRegistry.set(typeName, constructor);
  }

  // Определение типа значения
  static getTypeInfo(value) {
    if (value === null) return { type: 'null', value: null };
    if (value === undefined) return { type: 'undefined', value: undefined };

    const type = typeof value;

    if (type !== 'object' && type !== 'function') {
      return { type, value };
    }
  }
}
```

```
}
```

```
// Для функций
```

```
if (type === 'function') {  
  return { type: 'function', value: value.toString() };  
}
```

```
// Для объектов
```

```
const constructor = value.constructor;  
const typeName = constructor.name;
```

```
// Проверяем, зарегистрирован ли тип
```

```
if (this.typeRegistry.has(typeName)) {  
  return {  
    type: typeName,  
    value: this.serializeRegistered(value, constructor)  
  };  
}
```

```
// Стандартные типы
```

```
if (value instanceof Date) {  
  return { type: 'Date', value: value.toISOString() };  
}
```

```
if (value instanceof RegExp) {  
  return { type: 'RegExp', value: value.toString() };  
}
```

```
if (value instanceof Map) {  
  return {  
    type: 'Map',  
    value: Array.from(value.entries()).map(([k, v]) =>  
      [this.getTypeInfo(k), this.getTypeInfo(v)]  
    )  
  };  
}
```

```
if (value instanceof Set) {  
  return {  
    type: 'Set',  
    value: Array.from(value).map(item => this.getTypeInfo(item))  
  };  
}
```

```

    if (Array.isArray(value)) {
        return {
            type: 'array',
            value: value.map(item => this.getTypeInfo(item))
        };
    }

    // Простой объект
    return {
        type: 'object',
        value: Object.fromEntries(
            Object.entries(value).map(([k, v]) => [k, this.getTypeInfo(v)])
        )
    };
}

static serializeRegistered(value, constructor) {
    // По умолчанию используем JSON, но можно переопределить
    if (typeof value.toJSON === 'function') {
        return value.toJSON();
    }

    // Пытаемся сериализовать публичные свойства
    const result = {};
    for (const key in value) {
        if (value.hasOwnProperty(key)) {
            result[key] = this.getTypeInfo(value[key]);
        }
    }
    return result;
}

// Сериализация
static serialize(value) {
    return JSON.stringify(this.getTypeInfo(value), null, 2);
}

// Десериализация
static deserialize(json) {
    const data = JSON.parse(json);
    return this.reconstruct(data);
}

static reconstruct(typeInfo) {

```

```

const { type, value } = typeInfo;

switch (type) {
  case 'null':
    return null;
  case 'undefined':
    return undefined;
  case 'boolean':
  case 'number':
  case 'string':
    return value;
  case 'bigint':
    return BigInt(value);
  case 'symbol':
    return Symbol(value);
  case 'Date':
    return new Date(value);
  case 'RegExp': {
    const match = value.match(/^\/(.*)\/([gimy]*)$/);
    return new RegExp(match[1], match[2]);
  }
  case 'Map':
    return new Map(
      value.map(([k, v]) => [this.reconstruct(k), this.reconstruct(v)])
    );
  case 'Set':
    return new Set(value.map(item => this.reconstruct(item)));
  case 'array':
    return value.map(item => this.reconstruct(item));
  case 'object':
    return Object.fromEntries(
      Object.entries(value).map(([k, v]) => [k, this.reconstruct(v)])
    );
  default:
    // Кастомные зарегистрированные типы
    if (this.typeRegistry.has(type)) {
      const Constructor = this.typeRegistry.get(type);
      return this.reconstructRegistered(value, Constructor);
    }
    throw new Error(`Неизвестный тип: ${type}`);
}
}

static reconstructRegistered(value, constructor) {

```

```
    const instance = Object.create(constructor.prototype);
    Object.assign(instance, value);
    return instance;
  }
}
```

// Регистрация кастомных типов

```
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
    this.createdAt = new Date();
  }

  greet() {
    return `Привет, я ${this.name}`;
  }
}
```

```
TypeAwareSerializer.registerType(User);
```

// Тестирование

```
const complexData = {
  user: new User("Иван", 30),
  date: new Date(),
  regex: /test/gi,
  map: new Map([["key", "value"]]),
  set: new Set([1, 2, 3]),
  nested: {
    array: [1, "текст", true, null],
    nullValue: null,
    undefinedValue: undefined
  }
};

const serialized = TypeAwareSerializer.serialize(complexData);
console.log("Сериализованные данные:");
console.log(serialized);

const deserialized = TypeAwareSerializer.deserialize(serialized);
console.log("\nДесериализованные данные:");
console.log(deserialized);
```

// Проверка типов после десериализации

```
console.log("\nПроверка типов после десериализации:");
console.log("user instanceof User:", deserialized.user instanceof User); // true
console.log("date instanceof Date:", deserialized.date instanceof Date); // true
console.log("regex instanceof RegExp:", deserialized.regex instanceof RegExp); // true
console.log("map instanceof Map:", deserialized.map instanceof Map); // true
console.log("set instanceof Set:", deserialized.set instanceof Set); // true
console.log("user.greet():", deserialized.user.greet()); // "Привет, я Иван"
```

Заключение: стратегии использования typeof и instanceof

1. Краткое резюме различий:

Критерий	typeof	instanceof
Назначение	Определяет тип примитива или общий тип объекта	Проверяет, является ли объект экземпляром класса/конструктора
Возвращаемое значение	Строка: "string", "number", "boolean", "undefined", "object", "function", "symbol", "bigint"	Boolean: true или false
Работа с null	Возвращает "object" (особенность)	Возвращает false (null не экземпляр Object)
Работа с примитивами	Работает корректно	Не работает (возвращает false)
Работа с массивами	Возвращает "object"	Возвращает true для Array
Зависимость от контекста	Не зависит	Зависит (проблемы с iframe, разными realms)
Производительность	Быстрее	Медленнее (обходит цепочку прототипов)

2. Золотые правила:


```

// 1. Для примитивов используйте typeof (кроме null)
function handlePrimitive(value) {
    if (typeof value === 'string') { /* ... */ }
    if (typeof value === 'number') { /* ... */ }
    if (typeof value === 'boolean') { /* ... */ }
    if (value === null) { /* ... */ } // null проверяем через ===
}

// 2. Для массивов используйте Array.isArray()
function handleArray(value) {
    if (Array.isArray(value)) { /* ... */ }
    // НЕ используйте: value instanceof Array (проблемы с iframe)
}

// 3. Для проверки на null/undefined используйте строгое сравнение
function handleNullable(value) {
    if (value === null) { /* ... */ }
    if (value === undefined) { /* ... */ }
    // Или: if (value == null) { /* ... */ } для проверки обоих
}

// 4. Для проверки функций используйте typeof
function handleFunction(value) {
    if (typeof value === 'function') { /* ... */ }
}

// 5. Для проверки экземпляров классов используйте instanceof
class MyClass {}
function handleInstance(value) {
    if (value instanceof MyClass) { /* ... */ }
}

// 6. Для сложных проверок комбинируйте подходы
function complexTypeCheck(value) {
    if (typeof value === 'string') {
        // Строка
    } else if (typeof value === 'number') {
        // Число
    } else if (Array.isArray(value)) {
        // Массив
    } else if (value instanceof Date) {
        // Дата
    } else if (value && typeof value === 'object') {
        // Объект (не null)
    }
}

```

```

} else if (value === null) {
  // Null
} else if (value === undefined) {
  // Undefined
}
}

```

3. Современные альтернативы:

javascript

// 1. Опциональная цепочка и нулевое слияние

```
const value = obj?.prop?.nested ?? 'default';
```

// 2. Duck typing вместо жестких проверок

```
if (obj && typeof obj.execute === 'function') {
  obj.execute();
}
```

// 3. Пользовательские type guards (в TypeScript)

```
function isUser(value): value is User {
  return value instanceof User;
}
```

// 4. Symbol.toStringTag для кастомных типов

```
class MyType {
  get [Symbol.toStringTag]() { return 'MyType'; }
}
```

// 5. Object.prototype.toString для точного определения

```
console.log(Object.prototype.toString.call([])); // "[object Array]"
```

4. Главные выводы:

1. `typeof` — ваш основной инструмент для определения типов примитивов и общих категорий
2. `instanceof` — специализированный инструмент для работы с иерархиями классов
3. **Ни один оператор не идеален** — понимайте их ограничения и особенности
4. **Комбинируйте подходы** для надежных проверок
5. **Учитывайте контекст выполнения** — особенно для `instanceof`
6. **Пишите явные проверки** — ясность кода важнее краткости

Правильное использование `typeof` и `instanceof` — признак зрелого JavaScript-разработчика. Эти операторы, хотя и имеют свои особенности, остаются фундаментальными инструментами для работы с динамической типизацией JavaScript.

Глава 5: Операторы и выражения

5.1. Арифметические операторы

Введение: роль арифметических операторов в JavaScript

Арифметические операторы — фундаментальные инструменты для выполнения математических вычислений в JavaScript. Они позволяют:

1. **Выполнять базовые математические операции** (сложение, вычитание, умножение, деление)
2. **Обрабатывать числовые данные** в бизнес-логике, играх, научных вычислениях
3. **Работать с различными типами чисел** (целые, дробные, большие числа, разные системы счисления)
4. **Реализовывать сложные алгоритмы** через комбинацию операторов

JavaScript поддерживает полный набор арифметических операторов, многие из которых ведут себя особым образом из-за динамической типизации языка.

Часть 1: Базовые арифметические операторы

1.1. Сложение (+)

```
javascript

// Базовое сложение чисел
console.log(5 + 3);           // 8
console.log(10.5 + 2.3);     // 12.8
console.log(-5 + 10);        // 5
console.log(0.1 + 0.2);      // 0.30000000000000004 (особенность чисел с плавающей точкой)

// Сложение с разными типами (неявное преобразование)
```

```

console.log(5 + "3");           // "53" (конкатенация строк)
console.log("Цена: " + 100);    // "Цена: 100"
console.log(true + 1);          // 2 (true → 1)
console.log(false + 1);         // 1 (false → 0)
console.log(null + 5);          // 5 (null → 0)
console.log(undefined + 5);     // NaN (undefined → NaN)
console.log([] + 1);            // "1" ([] → "")
console.log([1, 2] + 3);        // "1,23" ([1,2] → "1,2")

// Многострочная конкатенация строк
const firstName = "Иван";
const lastName = "Петров";
const age = 30;
const fullInfo = firstName + " " + lastName + ", " + age + " лет";
console.log(fullInfo);          // "Иван Петров, 30 лет"

// Шаблонные строки (альтернатива сложению)
const fullInfoTemplate = `${firstName} ${lastName}, ${age} лет`;
console.log(fullInfoTemplate);  // "Иван Петров, 30 лет"

// Порядок операций имеет значение!
console.log(1 + 2 + "3");       // "33" (1+2=3, затем "3"+"3")
console.log("1" + 2 + 3);       // "123" ("1"+2="12", "12"+3="123")
console.log(1 + "2" + 3);       // "123" (1+"2"="12", "12"+3="123")

```

1.2. Вычитание (-)

```

javascript

// Базовое вычитание
console.log(10 - 5);             // 5
console.log(5 - 10);             // -5
console.log(3.14 - 1.5);         // 1.64 (приблизительно)

// Вычитание с преобразованием типов
console.log("10" - 5);           // 5 (строка → число)
console.log("10" - "3");         // 7 (обе строки → числа)
console.log("10px" - 5);         // NaN ("10px" не преобразуется полностью)
console.log("10.5" - 0.5);       // 10.0
console.log(true - 1);           // 0 (true → 1)
console.log(false - 1);          // -1 (false → 0)
console.log(null - 5);           // -5 (null → 0)
console.log(undefined - 5);      // NaN
console.log([] - 1);             // -1 ([] → 0)

```

```

console.log([10] - 5);           // 5 ([10] → 10)
console.log([10, 20] - 5);      // NaN ([10,20] → "10,20" → NaN)

// Отрицательные числа
console.log(5 - -3);             // 8 (минус на минус дает плюс)
console.log(-5 - 3);            // -8
console.log(-5 - -3);           // -2

// Вычитание дат (получение разницы в миллисекундах)
const date1 = new Date('2024-01-01');
const date2 = new Date('2024-01-15');
console.log(date2 - date1);      // 1209600000 (14 дней в миллисекундах)
console.log((date2 - date1) / (1000 * 60 * 60 * 24)); // 14 (дней)

```

1.3. Умножение (*)

```

javascript

// Базовое умножение
console.log(5 * 3);              // 15
console.log(2.5 * 4);            // 10.0
console.log(-5 * 3);             // -15
console.log(-5 * -3);           // 15

// Умножение с преобразованием типов
console.log("5" * 3);            // 15
console.log("5" * "3");          // 15
console.log("5px" * 3);          // NaN
console.log(true * 10);          // 10 (true → 1)
console.log(false * 10);         // 0 (false → 0)
console.log(null * 10);          // 0 (null → 0)
console.log(undefined * 10);     // NaN
console.log([] * 5);             // 0 ([] → 0)
console.log([5] * 3);            // 15 ([5] → 5)
console.log([5, 10] * 2);        // NaN ([5,10] → "5,10" → NaN)

// Особые значения
console.log(Infinity * 2);       // Infinity
console.log(Infinity * -2);      // -Infinity
console.log(Infinity * 0);       // NaN
console.log(Infinity * Infinity); // Infinity
console.log(Infinity * -Infinity); // -Infinity

// Умножение больших чисел

```

```
console.log(9007199254740991 * 2); // 18014398509481982 (корректно)
console.log(9007199254740992 * 2); // 18014398509481984 (потеря точности!)
console.log(9007199254740991n * 2n); // 18014398509481982n (точность с BigInt)
```

1.4. Деление (/)

javascript

// Базовое деление

```
console.log(10 / 2);           // 5
console.log(10 / 3);           // 3.3333333333333335
console.log(10 / 4);           // 2.5
console.log(-10 / 2);          // -5
console.log(10 / -2);          // -5
console.log(-10 / -2);         // 5
```

// Деление с преобразованием типов

```
console.log("10" / 2);         // 5
console.log("10" / "2");       // 5
console.log("10px" / 2);       // NaN
console.log(true / 2);          // 0.5 (true → 1)
console.log(false / 2);        // 0 (false → 0)
console.log(null / 2);         // 0 (null → 0)
console.log(undefined / 2);    // NaN
console.log([] / 2);           // 0 ([] → 0)
console.log([10] / 2);         // 5 ([10] → 10)
console.log([10, 20] / 2);     // NaN
```

// Деление на ноль

```
console.log(10 / 0);           // Infinity
console.log(10 / -0);          // -Infinity
console.log(0 / 0);            // NaN
console.log(Infinity / Infinity); // NaN
console.log(Infinity / 2);     // Infinity
console.log(2 / Infinity);     // 0
```

// Деление с плавающей точкой (точность)

```
console.log(0.1 / 0.3);        // 0.33333333333333337
console.log(1 / 3);            // 0.3333333333333333
console.log(2 / 3);            // 0.6666666666666666
```

// Целочисленное деление (эмуляция)

```
function integerDivision(dividend, divisor) {
    return Math.floor(dividend / divisor);
}
```

```

}
console.log(integerDivision(10, 3)); // 3
console.log(integerDivision(-10, 3)); // -4 (Math.floor для отрицательных!)

// Более точное целочисленное деление
function truncatingDivision(dividend, divisor) {
    return Math.trunc(dividend / divisor);
}
console.log(truncatingDivision(10, 3)); // 3
console.log(truncatingDivision(-10, 3)); // -3

```

1.5. Остаток от деления (%)

javascript

```

// Базовый остаток от деления
console.log(10 % 3);           // 1 (10 = 3*3 + 1)
console.log(10 % 2);           // 0 (четное число)
console.log(11 % 2);           // 1 (нечетное число)
console.log(-10 % 3);          // -1 (знак сохраняется от делимого!)
console.log(10 % -3);          // 1
console.log(-10 % -3);         // -1

// Остаток от деления с преобразованием типов
console.log("10" % 3);         // 1
console.log("10" % "3");       // 1
console.log("10px" % 3);       // NaN
console.log(true % 3);         // 1 (true → 1)
console.log(false % 3);        // 0 (false → 0)
console.log(null % 3);         // 0 (null → 0)
console.log(undefined % 3);    // NaN
console.log([] % 3);           // 0
console.log([10] % 3);         // 1

// Особые случаи
console.log(10 % 0);           // NaN (деление на ноль)
console.log(0 % 10);           // 0
console.log(Infinity % 10);    // NaN
console.log(10 % Infinity);    // 10

// Практическое использование
// 1. Проверка четности
function isEven(number) {
    return number % 2 === 0;
}

```



```

}
console.log(isEven(10)); // true
console.log(isEven(11)); // false

// 2. Циклические операции (циклические значения)
function getDayOfWeek(dayIndex) {
  const days = ["Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Вс"];
  return days[dayIndex % 7];
}
console.log(getDayOfWeek(0)); // "Пн"
console.log(getDayOfWeek(7)); // "Пн" (8-й день = 1-й день недели)
console.log(getDayOfWeek(365)); // "Ср" (366-й день от начала)

// 3. Ограничение диапазона
function limitToRange(value, min, max) {
  return ((value - min) % (max - min + 1)) + min;
}
console.log(limitToRange(12, 0, 9)); // 2 (12 → 2)
console.log(limitToRange(-3, 0, 9)); // 7 (-3 → 7)

```

1.6. Возведение в степень (**)

javascript

```

// Базовое возведение в степень (ES7/ES2016)
console.log(2 ** 3); // 8 (2³)
console.log(5 ** 2); // 25 (5²)
console.log(10 ** 0); // 1 (любое число в степени 0 = 1)
console.log(10 ** 1); // 10
console.log(2 ** -1); // 0.5 (2⁻¹ = 1/2)
console.log(2 ** 0.5); // 1.4142135623730951 (√2)
console.log(-2 ** 2); // -4 (обратите внимание: (-2)² = 4, но -2² = -4)
console.log((-2) ** 2); // 4 (нужны скобки!)

// Возведение в степень с преобразованием типов
console.log("2" ** "3"); // 8
console.log("2" ** 3); // 8
console.log("2px" ** 3); // NaN
console.log(true ** 3); // 1 (true → 1)
console.log(false ** 3); // 0 (false → 0)
console.log(null ** 3); // 0 (null → 0)
console.log(undefined ** 3); // NaN
console.log([] ** 2); // 0 ([] → 0)
console.log([2] ** 3); // 8 ([2] → 2)

```

```
// Большие степени
console.log(2 ** 10);           // 1024
console.log(2 ** 31);           // 2147483648
console.log(2 ** 53);           // 9007199254740992 (предел точности Number)
console.log(2 ** 1024);         // Infinity
console.log(0.5 ** 10);         // 0.0009765625

// Math.pow() — альтернативный синтаксис (до ES7)
console.log(Math.pow(2, 3));    // 8
console.log(Math.pow(5, 2));    // 25
console.log(Math.pow(2, -1));   // 0.5

// Особенности ** vs Math.pow()
console.log(2 ** 3 ** 2);       // 512 (2^(3^2) = 2^9) - правоассоциативно!
console.log(Math.pow(2, Math.pow(3, 2))); // 512
console.log((2 ** 3) ** 2);     // 64 ((2^3)^2 = 8^2)

// Возведение в степень для BigInt
console.log(2n ** 3n);          // 8n
console.log(2n ** 10n);         // 1024n
// console.log(2n ** 1000n);    // Очень большое число (без потери точности)
```

Часть 2: Унарные арифметические операторы

2.1. Унарный плюс (+)

```
javascript

// Основное назначение: преобразование к числу
console.log(+42);               // 42 (оставляет число без изменений)
console.log(+ "42");            // 42 (преобразует строку в число)
console.log(+ "3.14");          // 3.14
console.log(+ "0xFF");          // 255 (шестнадцатеричное)
console.log(+ "1e3");            // 1000 (экспоненциальная запись)

// Преобразование разных типов
console.log(+true);              // 1
console.log(+false);             // 0
console.log(+null);              // 0
console.log(+undefined);         // NaN
```

```

console.log("");           // 0 (пустая строка)
console.log(+ " 123 ");    // 123 (пробелы игнорируются)
console.log(+ "123текст"); // NaN
console.log(+[]);          // 0 (пустой массив)
console.log(+[5]);         // 5 (массив с одним числом)
console.log(+[1, 2]);      // NaN (массив с несколькими элементами)
console.log(+{});          // NaN (объект)

```

// Практическое использование

// 1. Быстрое преобразование строк в числа

```

const price = "99.99";
const quantity = "3";
const total = +price * +quantity;
console.log(total);           // 299.97

```

// 2. Сравнение преобразованных значений

```

const userInput = " 042 ";
const normalizedValue = +userInput;
console.log(normalizedValue); // 42

```

// 3. В функциях для приведения типа

```

function sum(a, b) {
  // Приводим аргументы к числам
  return (+a) + (+b);
}
console.log(sum("10", "20")); // 30
console.log(sum("10px", 20)); // NaN (но хотя бы попытались)

```

// 4. Получение текущего времени в миллисекундах

```

const timestamp = +new Date();
console.log(timestamp);           // 1678901234567 (например)

```

// 5. Упрощение проверок

```

const value = "0";
if (+value) {                     // 0 → false
  console.log("Truthy");
} else {
  console.log("Falsy");           // Выполнится это
}

```

2.2. Унарный минус (-)

javascript

```
// Основное назначение: изменение знака числа
console.log(-42);           // -42
console.log(-(-42));        // 42 (двойное отрицание)
console.log(+ "-42");       // -42 (сначала преобразование, потом знак)
```

```
// Преобразование с изменением знака
console.log(-"42");         // -42
console.log(-"3.14");       // -3.14
console.log(-true);         // -1
console.log(-false);        // -0 (да, есть отрицательный ноль!)
console.log(-null);         // -0
console.log(-undefined);    // NaN
console.log(-"");           // -0
console.log(-" 123 ");      // -123
console.log(-[]);           // -0
console.log(-[5]);          // -5
console.log(-[1, 2]);       // NaN
```

```
// Отрицательный ноль
console.log(-0);            // -0
console.log(+(-0));         // -0
console.log(-0 === 0);      // true (но они различаются!)
console.log(Object.is(-0, 0)); // false
console.log(1 / -0);        // -Infinity
console.log(1 / 0);         // Infinity
```

```
// Практическое использование
```

```
// 1. Инверсия значений
```

```
function toggleSign(value) {
  return -value;
}
console.log(toggleSign(10)); // -10
console.log(toggleSign(-10)); // 10
```

```
// 2. Создание диапазона отрицательных значений
```

```
function createNegativeRange(start, end) {
  const result = [];
  for (let i = start; i <= end; i++) {
    result.push(-i);
  }
  return result;
}
console.log(createNegativeRange(1, 5)); // [-1, -2, -3, -4, -5]
```

// 3. Работа с координатами

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  reflectX() {
    return new Point(-this.x, this.y);
  }

  reflectY() {
    return new Point(this.x, -this.y);
  }

  reflectOrigin() {
    return new Point(-this.x, -this.y);
  }
}

const point = new Point(3, 4);
console.log(point.reflectX()); // Point {x: -3, y: 4}
console.log(point.reflectY()); // Point {x: 3, y: -4}
```

2.3. Инкремент (++) и декремент (--)

javascript

// Префиксная форма (++x): сначала увеличить, потом вернуть

```
let x = 5;
console.log(++x); // 6 (x теперь равно 6)
console.log(x); // 6
```

// Постфиксная форма (x++): сначала вернуть, потом увеличить

```
let y = 5;
console.log(y++); // 5 (но y теперь равно 6)
console.log(y); // 6
```

// Декремент аналогично

```
let a = 5;
console.log(--a); // 4
console.log(a); // 4
```

```
let b = 5;
```

```

console.log(b--);           // 5
console.log(b);             // 4

// С преобразованием типов
let str = "10";
console.log(++str);         // 11 (строка → число, увеличивается)
console.log(str);           // 11 (теперь число)

let bool = true;
console.log(++bool);        // 2 (true → 1, затем +1)
console.log(bool);          // 2

let empty = "";
console.log(++empty);       // 1 (" " → 0, затем +1)
console.log(empty);         // 1

// Ошибки с константами
const constant = 5;
// console.log(++constant); // TypeError: Assignment to constant variable

// Практическое использование
// 1. Счетчики в циклах
for (let i = 0; i < 5; i++) {
  console.log(`Итерация ${i}`);
}
// Итерация 0, Итерация 1, Итерация 2, Итерация 3, Итерация 4

// 2. Обратный отсчет
for (let i = 5; i > 0; i--) {
  console.log(`Осталось: ${i}`);
}
console.log("Старт!");

// 3. Индексация в массивах
const fruits = ["Яблоко", "Банан", "Апельсин"];
let index = 0;
console.log(fruits[index++]); // "Яблоко", index = 1
console.log(fruits[index++]); // "Банан", index = 2
console.log(fruits[index]);    // "Апельсин"

// 4. Сложные выражения (будьте осторожны!)
let counter = 0;
let result = counter++ + ++counter;
console.log(result);           // 2 (0 + 2)

```

```
console.log(counter);          // 2

// 5. Множественные операции (избегайте!)
let n = 1;
console.log(n++ + ++n + n-- - --n); // 1 + 3 + 3 - 1 = 6
console.log(n);                    // 1

// 6. Работа с DOM элементами
function createCounterElement() {
  let count = 0;
  const button = document.createElement('button');
  button.textContent = `Кликов: ${count}`;

  button.addEventListener('click', () => {
    button.textContent = `Кликов: ${++count}`;
  });

  return button;
}

// Использование: document.body.appendChild(createCounterElement());
```

Часть 3: Составные операторы присваивания

3.1. Общий синтаксис и примеры

```
javascript

// Базовый синтаксис: переменная оператор= выражение
// Эквивалентно: переменная = переменная оператор выражение

let number = 10;

// Сложение с присваиванием
number += 5;          // number = number + 5
console.log(number);  // 15

// Вычитание с присваиванием
number -= 3;          // number = number - 3
console.log(number);  // 12

// Умножение с присваиванием
```

```

number *= 2;           // number = number * 2
console.log(number);   // 24

// Деление с присваиванием
number /= 4;           // number = number / 4
console.log(number);   // 6

// Остаток с присваиванием
number %= 4;           // number = number % 4
console.log(number);   // 2

// Возведение в степень с присваиванием
number **= 3;          // number = number ** 3
console.log(number);   // 8

// Работа со строками
let text = "Привет";
text += ", мир!";      // text = text + ", мир!"
console.log(text);     // "Привет, мир!"

// С преобразованием типов
let value = "10";
value += 5;            // "10" + 5 = "105" (конкатенация!)
console.log(value);    // "105"
console.log(typeof value); // "string"

value = "10";
value -= 5;            // 10 - 5 = 5 (вычитание преобразует!)
console.log(value);    // 5
console.log(typeof value); // "number"

```

3.2. Практическое применение составных операторов

```

javascript

// 1. Аккумуляторы и агрегаторы
function calculateTotal(prices) {
  let total = 0;
  for (const price of prices) {
    total += price; // Аккумулируем сумму
  }
  return total;
}

```



```
console.log(calculateTotal([100, 200, 300])); // 600
```

```
// 2. Постепенное построение строки
```

```
function formatUserInfo(user) {  
  let result = "";  
  result += `Имя: ${user.name}\n`;   
  result += `Возраст: ${user.age}\n`;   
  result += `Email: ${user.email || 'не указан'}\n`;   
  return result;  
}  
  
const user = { name: "Иван", age: 30, email: "ivan@example.com" };  
console.log(formatUserInfo(user));
```

```
// 3. Работа с DOM элементами
```

```
function updateCartDisplay(cart) {  
  let cartElement = document.getElementById('cart');  
  let total = 0;  
  let itemCount = 0;  
  let html = "<ul>";  
  
  for (const item of cart.items) {  
    html += `<li>${item.name}: ${item.price} руб.</li>`;   
    total += item.price;  
    itemCount += item.quantity;  
  }  
  
  html += "</ul>";  
  html += `<p>Итого: ${total} руб.</p>`;   
  html += `<p>Товаров: ${itemCount} шт.</p>`;   
  
  cartElement.innerHTML = html;  
}
```

```
// 4. Статистика и метрики
```

```
class Statistics {  
  constructor() {  
    this.count = 0;  
    this.sum = 0;  
    this.min = Infinity;  
    this.max = -Infinity;  
  }  
  
  addValue(value) {
```

```

    this.count += 1;
    this.sum += value;
    this.min = Math.min(this.min, value);
    this.max = Math.max(this.max, value);
}

get average() {
    return this.sum / this.count;
}
}

const stats = new Statistics();
[10, 20, 30, 40, 50].forEach(value => stats.addValue(value));
console.log(`Среднее: ${stats.average}`);           // 30
console.log(`Минимум: ${stats.min}`);               // 10
console.log(`Максимум: ${stats.max}`);              // 50

// 5. Анимации и игровая логика
class GameCharacter {
    constructor() {
        this.health = 100;
        this.mana = 50;
        this.experience = 0;
        this.level = 1;
    }

    takeDamage(amount) {
        this.health -= amount;
        if (this.health < 0) this.health = 0;
    }

    heal(amount) {
        this.health += amount;
        if (this.health > 100) this.health = 100;
    }

    useMana(amount) {
        this.mana -= amount;
        if (this.mana < 0) this.mana = 0;
    }

    gainExperience(amount) {
        this.experience += amount;
        // Повышение уровня каждые 100 опыта

```

```

const newLevel = Math.floor(this.experience / 100) + 1;
if (newLevel > this.level) {
  console.log(`Уровень повышен до ${newLevel}!`);
  this.level = newLevel;
}
}
}

```

// 6. Управление состоянием

```

class StateManager {
  constructor() {
    this.state = {
      counter: 0,
      total: 0,
      history: []
    };
  }

  incrementCounter() {
    this.state.counter += 1;
    this.state.total += this.state.counter;
    this.state.history.push(this.state.counter);
  }

  reset() {
    this.state.counter = 0;
    this.state.total = 0;
    this.state.history = [];
  }
}

```

3.3. Особенности работы

javascript

// 1. Приоритет операций

```

let a = 5;
a *= 2 + 3; // a = a * (2 + 3) = 5 * 5 = 25, НЕ (a * 2) + 3
console.log(a); // 25

```

// 2. Работа с объектами и массивами

```

const obj = { value: 10 };
obj.value += 5; // Работает: obj.value = obj.value + 5
console.log(obj.value); // 15

```

```
const arr = [1, 2, 3];
arr[0] += 10;    // arr[0] = arr[0] + 10
console.log(arr); // [11, 2, 3]

// 3. Ссылочные типы
let x = 5;
let y = x;      // Копирование значения
y += 5;
console.log(x); // 5 (не изменилось)
console.log(y); // 10

let obj1 = { val: 5 };
let obj2 = obj1; // Копирование ссылки
obj2.val += 5;
console.log(obj1.val); // 10 (изменилось!)
console.log(obj2.val); // 10

// 4. Несколько операций в одной строке (избегайте!)
let z = 10;
z += z -= z *= 2; // Справа налево: z *= 2 → 20, z -= 20 → -10, z += -10 → 0
console.log(z);   // 0

// 5. С BigInt
let big = 100n;
big += 50n;      // 150n
big *= 2n;       // 300n
console.log(big); // 300n

// 6. Ошибки с const
const constantValue = 10;
// constantValue += 5; // TypeError: Assignment to constant variable

// 7. Неявное преобразование при += со строками
let mixed = 5;
mixed += "5";    // "55" (число 5 → строка "5", затем конкатенация)
console.log(mixed); // "55"
console.log(typeof mixed); // "string"

mixed = 5;
mixed -= "5";    // 0 (преобразование в числа, затем вычитание)
console.log(mixed); // 0
console.log(typeof mixed); // "number"
```

Часть 4: Специальные операторы и методы

4.1. Math объект для сложных операций

javascript

// Хотя Math – это объект, а не оператор, он тесно связан с арифметикой

// Округление

```
console.log(Math.floor(3.7)); // 3 (вниз)
console.log(Math.ceil(3.1)); // 4 (вверх)
console.log(Math.round(3.5)); // 4 (математическое округление)
console.log(Math.trunc(3.7)); // 3 (отбросить дробную часть)
console.log(Math.round(-3.5)); // -3 (особенность: -3.5 → -3)
```

// Минимум и максимум

```
console.log(Math.min(10, 20, 5, 30)); // 5
console.log(Math.max(10, 20, 5, 30)); // 30
console.log(Math.min(...[10, 20, 5, 30])); // 5 (с spread оператором)
```

// Случайные числа

```
console.log(Math.random()); // 0.0 ≤ n < 1.0
console.log(Math.floor(Math.random() * 100)); // 0-99
console.log(Math.floor(Math.random() * 100) + 1); // 1-100
```

// Степени и корни

```
console.log(Math.sqrt(16)); // 4 (квадратный корень)
console.log(Math.cbrt(27)); // 3 (кубический корень)
console.log(Math.pow(2, 10)); // 1024 (альтернатива **)
console.log(Math.hypot(3, 4)); // 5 (гипотенуза:  $\sqrt{3^2+4^2}$ )
```

// Тригонометрия

```
console.log(Math.sin(Math.PI / 6)); // 0.5 (sin 30°)
console.log(Math.cos(Math.PI / 3)); // 0.5 (cos 60°)
console.log(Math.tan(Math.PI / 4)); // 1 (tan 45°)
```

// Логарифмы

```
console.log(Math.log(Math.E)); // 1 (натуральный логарифм)
console.log(Math.log10(100)); // 2 (десятичный логарифм)
console.log(Math.log2(8)); // 3 (двоичный логарифм)
```

// Абсолютное значение

```
console.log(Math.abs(-10)); // 10
```

```

console.log(Math.abs(10));           // 10

// Знак числа
console.log(Math.sign(-10));         // -1
console.log(Math.sign(0));           // 0
console.log(Math.sign(10));          // 1
console.log(Math.sign(-0));          // -0 (особенность!)

```

4.2. Number методы для точных вычислений

```

javascript

// toFixed() – форматирование с фиксированной точкой
const price = 123.4567;
console.log(price.toFixed(2));       // "123.46" (округление)
console.log(price.toFixed(0));       // "123"
console.log((0.1 + 0.2).toFixed(2)); // "0.30" (решение проблемы точности)

// toPrecision() – форматирование с заданной точностью
console.log(price.toPrecision(5));   // "123.46" (5 значащих цифр)
console.log(price.toPrecision(3));   // "123"
console.log((0.1 + 0.2).toPrecision(3)); // "0.300"

// toString() с системами счисления
const num = 255;
console.log(num.toString(2));        // "11111111" (двоичная)
console.log(num.toString(8));        // "377" (восьмеричная)
console.log(num.toString(16));       // "ff" (шестнадцатеричная)
console.log(num.toString(36));       // "73" (максимальная система)

// Number.isFinite() и Number.isNaN() – безопасные проверки
console.log(Number.isFinite(42));    // true
console.log(Number.isFinite(Infinity)); // false
console.log(Number.isFinite("42"));  // false (строгая проверка)
console.log(Number.isNaN(NaN));      // true
console.log(Number.isNaN("текст"));  // false (строгая проверка)

// Number.EPSILON – наименьший интервал между числами
console.log(Number.EPSILON);        // 2.220446049250313e-16
console.log(0.1 + 0.2 === 0.3);     // false
console.log(Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON); // true

// Number.MAX_SAFE_INTEGER и Number.MIN_SAFE_INTEGER
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991

```

```
console.log(Number.MIN_SAFE_INTEGER);    // -9007199254740991
console.log(Number.isSafeInteger(9007199254740991)); // true
console.log(Number.isSafeInteger(9007199254740992)); // false
```

4.3. Битовые операторы (частично арифметические)

javascript

// Хотя это побитовые операторы, они часто используются для арифметики

// Побитовый сдвиг влево (<<) – умножение на степени двойки

```
console.log(5 << 1);    // 10 (5 * 21)
console.log(5 << 2);    // 20 (5 * 22)
console.log(5 << 3);    // 40 (5 * 23)
```

// Побитовый сдвиг вправо (>>) – целочисленное деление на степени двойки

```
console.log(20 >> 1);    // 10 (20 / 21)
console.log(20 >> 2);    // 5 (20 / 22)
console.log(20 >> 3);    // 2 (20 / 23, отбрасывание дробной части)
```

// Беззнаковый сдвиг вправо (>>>)

```
console.log(-20 >> 2);    // -5 (сохраняет знак)
console.log(-20 >>> 2);    // 1073741819 (не сохраняет знак!)
```

// Быстрые операции с степенями двойки

```
function isPowerOfTwo(n) {
    return n > 0 && (n & (n - 1)) === 0;
}
console.log(isPowerOfTwo(16)); // true
console.log(isPowerOfTwo(15)); // false
```

// Быстрое умножение/деление на 2

```
function multiplyByTwo(n) {
    return n << 1;
}
```

```
function divideByTwo(n) {
    return n >> 1;
}
```

```
console.log(multiplyByTwo(7)); // 14
console.log(divideByTwo(14));  // 7
```

Часть 5: Реальные примеры и практическое применение

5.1. Финансовые вычисления

javascript

```
class FinancialCalculator {
  // Простые проценты
  static simpleInterest(principal, rate, time) {
    return principal + (principal * rate * time / 100);
  }

  // Сложные проценты
  static compoundInterest(principal, rate, time, compoundsPerYear = 1) {
    return principal * Math.pow(1 + rate / (100 * compoundsPerYear), compoundsPerYear *
time);
  }

  // Аннуитетный платеж
  static annuityPayment(principal, rate, periods) {
    const monthlyRate = rate / 100 / 12;
    return principal * monthlyRate * Math.pow(1 + monthlyRate, periods) /
      (Math.pow(1 + monthlyRate, periods) - 1);
  }

  // НДС расчет
  static calculateVAT(amount, vatRate = 20) {
    const vatAmount = amount * vatRate / 100;
    const total = amount + vatAmount;
    return {
      amount,
      vatRate,
      vatAmount: +vatAmount.toFixed(2),
      total: +total.toFixed(2)
    };
  }

  // Конвертация валют
  static convertCurrency(amount, fromRate, toRate) {
    return (amount / fromRate) * toRate;
  }

  // Форматирование денежных значений
```



```

static formatMoney(amount, currency = 'RUB', locale = 'ru-RU') {
    return new Intl.NumberFormat(locale, {
        style: 'currency',
        currency: currency,
        minimumFractionDigits: 2,
        maximumFractionDigits: 2
    }).format(amount);
}
}

// Примеры использования
console.log("Простой процент (1000 руб под 10% на 2 года:");
console.log(FinancialCalculator.simpleInterest(1000, 10, 2)); // 1200

console.log("\nСложный процент (1000 руб под 10% на 2 года, ежемесячно:");
console.log(FinancialCalculator.compoundInterest(1000, 10, 2, 12).toFixed(2)); // 1220.39

console.log("\nАннуитетный платеж (ипотека 1 млн на 20 лет под 8%:");
const payment = FinancialCalculator.annuityPayment(1000000, 8, 240);
console.log(FinancialCalculator.formatMoney(payment)); // "8 364,40 ₽"

console.log("\nНДС расчет (сумма 1000 руб:");
console.log(FinancialCalculator.calculateVAT(1000));

console.log("\nКонвертация валют (100 USD → RUB по курсу 75:");
console.log(FinancialCalculator.convertCurrency(100, 1, 75)); // 7500

```

5.2. Геометрические и физические вычисления

```

javascript

class GeometryCalculator {
    // Площади
    static circleArea(radius) {
        return Math.PI * radius ** 2;
    }

    static rectangleArea(width, height) {
        return width * height;
    }

    static triangleArea(base, height) {
        return (base * height) / 2;
    }
}

```

```

// Объемы
static sphereVolume(radius) {
    return (4 / 3) * Math.PI * radius ** 3;
}

static cylinderVolume(radius, height) {
    return Math.PI * radius ** 2 * height;
}

static cubeVolume(side) {
    return side ** 3;
}

// Расстояния и углы
static distance(x1, y1, x2, y2) {
    return Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);
}

static toRadians(degrees) {
    return degrees * Math.PI / 180;
}

static toDegrees(radians) {
    return radians * 180 / Math.PI;
}

// Векторные операции
static vectorLength(x, y, z = 0) {
    return Math.sqrt(x ** 2 + y ** 2 + z ** 2);
}

static dotProduct(ax, ay, az, bx, by, bz) {
    return ax * bx + ay * by + az * bz;
}
}

class PhysicsCalculator {
    // Кинематика
    static distanceUVT(u, v, t) {
        return ((u + v) / 2) * t;
    }

    static finalVelocity(u, a, t) {

```

```

    return u + a * t;
}

static distanceUAT(u, a, t) {
    return u * t + (a * t ** 2) / 2;
}

// Динамика
static force(mass, acceleration) {
    return mass * acceleration;
}

static kineticEnergy(mass, velocity) {
    return (mass * velocity ** 2) / 2;
}

static potentialEnergy(mass, height, g = 9.81) {
    return mass * g * height;
}

// Электричество
static ohmsLaw(voltage, resistance) {
    return voltage / resistance;
}

static power(voltage, current) {
    return voltage * current;
}
}

// Примеры
console.log("Площадь круга радиусом 5:", GeometryCalculator.circleArea(5).toFixed(2));
console.log("Объем сферы радиусом 3:", GeometryCalculator.sphereVolume(3).toFixed(2));
console.log("Расстояние между точками (0,0) и (3,4):", GeometryCalculator.distance(0, 0,
3, 4));

console.log("\nДистанция при скорости 10 м/с и ускорении 2 м/с² за 5 сек:");
console.log(PhysicsCalculator.distanceUAT(10, 2, 5));

console.log("\nКинетическая энергия тела массой 10 кг со скоростью 5 м/с:");
console.log(PhysicsCalculator.kineticEnergy(10, 5));

```

5.3. Игровая математика и анимации

javascript

```
class GameMath {
  // Интерполяция
  static lerp(start, end, t) {
    return start + (end - start) * t;
  }

  static easeInOut(t) {
    return t < 0.5 ? 2 * t * t : 1 - Math.pow(-2 * t + 2, 2) / 2;
  }

  // Ограничение значений
  static clamp(value, min, max) {
    return Math.min(Math.max(value, min), max);
  }

  static wrap(value, min, max) {
    const range = max - min;
    return ((value - min) % range + range) % range + min;
  }

  // Случайные числа
  static randomRange(min, max) {
    return Math.random() * (max - min) + min;
  }

  static randomInt(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
  }

  // Векторные операции для 2D игр
  static normalizeVector(x, y) {
    const length = Math.sqrt(x * x + y * y);
    return length === 0 ? { x: 0, y: 0 } : { x: x / length, y: y / length };
  }

  static rotateVector(x, y, angle) {
    const rad = angle * Math.PI / 180;
    const cos = Math.cos(rad);
    const sin = Math.sin(rad);
    return {

```

```

    x: x * cos - y * sin,
    y: x * sin + y * cos
};
}

// Коллизии (проверка пересечения окружностей)
static circleCollision(x1, y1, r1, x2, y2, r2) {
    const dx = x2 - x1;
    const dy = y2 - y1;
    const distance = Math.sqrt(dx * dx + dy * dy);
    return distance < r1 + r2;
}

// Пример игрового объекта с физикой
class GameObject {
    constructor(x, y) {
        this.x = x;
        this.y = y;
        this.vx = 0;
        this.vy = 0;
        this.rotation = 0;
        this.speed = 0;
    }

    update(deltaTime) {
        // Движение с учетом скорости
        this.x += this.vx * deltaTime;
        this.y += this.vy * deltaTime;

        // Вращение
        this.rotation += this.speed * deltaTime;
        this.rotation %= 360;

        // Ограничение экрана
        this.x = GameMath.wrap(this.x, 0, 800);
        this.y = GameMath.clamp(this.y, 0, 600);
    }

    accelerate(amount) {
        const radians = this.rotation * Math.PI / 180;
        this.vx += Math.cos(radians) * amount;
        this.vy += Math.sin(radians) * amount;
    }
}

```

```

    getPosition() {
        return { x: this.x, y: this.y };
    }
}

// Симуляция
const player = new GameObject(400, 300);
player.speed = 90; // градусов в секунду

// Имитация игрового цикла (60 FPS)
const deltaTime = 1/60;
for (let i = 0; i < 60; i++) { // 1 секунда
    player.accelerate(0.1);
    player.update(deltaTime);
}

console.log("Позиция после 1 секунды:", player.getPosition());

```

5.4. Анализ данных и статистика

```

javascript

class DataAnalyzer {
    constructor(data) {
        this.data = data;
    }

    // Основные статистические показатели

    mean() {
        return this.data.reduce((sum, val) => sum + val, 0) / this.data.length;
    }

    median() {
        const sorted = [...this.data].sort((a, b) => a - b);
        const middle = Math.floor(sorted.length / 2);

        if (sorted.length % 2 === 0) {
            return (sorted[middle - 1] + sorted[middle]) / 2;
        }
        return sorted[middle];
    }

    mode() {

```

```

const frequency = new Map();
this.data.forEach(val => {
  frequency.set(val, (frequency.get(val) || 0) + 1);
});

let maxFreq = 0;
let modes = [];

frequency.forEach((freq, val) => {
  if (freq > maxFreq) {
    maxFreq = freq;
    modes = [val];
  } else if (freq === maxFreq) {
    modes.push(val);
  }
});

return modes.length === this.data.length ? [] : modes;
}

variance() {
  const mean = this.mean();
  return this.data.reduce((sum, val) => sum + (val - mean) ** 2, 0) / this.data.length;
}

standardDeviation() {
  return Math.sqrt(this.variance());
}

// Дополнительные метрики
range() {
  const max = Math.max(...this.data);
  const min = Math.min(...this.data);
  return max - min;
}

quartiles() {
  const sorted = [...this.data].sort((a, b) => a - b);
  const q2 = this.median();

  const lowerHalf = sorted.slice(0, Math.floor(sorted.length / 2));
  const upperHalf = sorted.slice(Math.ceil(sorted.length / 2));

  const q1 = lowerHalf.length ?

```

```

        new DataAnalyzer(lowerHalf).median() : sorted[0];
const q3 = upperHalf.length ?
    new DataAnalyzer(upperHalf).median() : sorted[sorted.length - 1];

return { q1, q2, q3 };
}

// Корреляция между двумя наборами данных
static correlation(xData, yData) {
    if (xData.length !== yData.length) {
        throw new Error("Данные должны иметь одинаковую длину");
    }

    const xMean = new DataAnalyzer(xData).mean();
    const yMean = new DataAnalyzer(yData).mean();

    let numerator = 0;
    let xDenominator = 0;
    let yDenominator = 0;

    for (let i = 0; i < xData.length; i++) {
        const xDiff = xData[i] - xMean;
        const yDiff = yData[i] - yMean;

        numerator += xDiff * yDiff;
        xDenominator += xDiff ** 2;
        yDenominator += yDiff ** 2;
    }

    return numerator / Math.sqrt(xDenominator * yDenominator);
}

// Пример анализа данных
const testScores = [85, 92, 78, 90, 88, 95, 76, 89, 91, 87];
const analyzer = new DataAnalyzer(testScores);

console.log("Анализ результатов теста:");
console.log("Среднее:", analyzer.mean().toFixed(2));
console.log("Медиана:", analyzer.median());
console.log("Мода:", analyzer.mode());
console.log("Дисперсия:", analyzer.variance().toFixed(2));
console.log("Стандартное отклонение:", analyzer.standardDeviation().toFixed(2));
console.log("Размах:", analyzer.range());

```



```
console.log("Квартили:", analyzer.quartiles());

// Корреляция между временем учебы и оценками
const studyHours = [10, 15, 5, 20, 12, 18, 8, 16, 14, 11];
const correlation = DataAnalyzer.correlation(studyHours, testScores);
console.log("\nКорреляция время учебы ↔ оценка:", correlation.toFixed(3));
```

Часть 6: Ошибки, тонкости и лучшие практики

6.1. Распространенные ошибки

```
javascript

// 1. Потеря точности с плавающей точкой
console.log(0.1 + 0.2 === 0.3); // false!
console.log(0.1 + 0.2); // 0.30000000000000004

// Решение: использовать допустимую погрешность
function areNumbersEqual(a, b, epsilon = Number.EPSILON) {
    return Math.abs(a - b) < epsilon;
}
console.log(areNumbersEqual(0.1 + 0.2, 0.3)); // true

// Или работать в целых числах
function addDecimals(a, b) {
    const multiplier = Math.max(
        a.toString().split('.')[1]?.length || 0,
        b.toString().split('.')[1]?.length || 0
    );
    const factor = 10 ** multiplier;
    return (a * factor + b * factor) / factor;
}
console.log(addDecimals(0.1, 0.2)); // 0.3

// 2. Неявное преобразование при сложении
let total = 100;
total += "50"; // "10050" (строка!)
console.log(total);

// Решение: явное преобразование
total = 100;
```

```

total += Number("50"); // 150
console.log(total);

// 3. Деление на ноль
console.log(1 / 0); // Infinity
console.log(-1 / 0); // -Infinity
console.log(0 / 0); // NaN

// Решение: проверка делителя
function safeDivide(dividend, divisor) {
    if (divisor === 0) {
        return dividend === 0 ? NaN : Infinity * Math.sign(dividend);
    }
    return dividend / divisor;
}

// 4. Переполнение
console.log(Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2); // true!

// Решение: использовать BigInt
console.log(BigInt(Number.MAX_SAFE_INTEGER) + 1n ===
    BigInt(Number.MAX_SAFE_INTEGER) + 2n); // false

// 5. Инкремент/декремент в сложных выражениях
let i = 0;
let result = i++ + ++i; // 0 + 2 = 2
console.log(result); // 2 (но код нечитаем!)

// Решение: разделять операции
i = 0;
let temp1 = i; // 0
i++; // i = 1
let temp2 = ++i; // i = 2, temp2 = 2
result = temp1 + temp2; // 2

```

6.2. Производительность операций

```

javascript

// Бенчмарк разных способов выполнения операций
function benchmark() {
    const iterations = 1000000;

    const tests = [

```

```

{
  name: "Сложение чисел",
  fn: () => {
    let sum = 0;
    for (let i = 0; i < iterations; i++) {
      sum += i;
    }
    return sum;
  }
},
{
  name: "Сложение строк",
  fn: () => {
    let str = "";
    for (let i = 0; i < iterations; i++) {
      str += i;
    }
    return str.length;
  }
},
{
  name: "Умножение vs Сдвиг",
  fn: () => {
    let result = 0;
    for (let i = 0; i < iterations; i++) {
      result = i * 2; // или i << 1
    }
    return result;
  }
},
{
  name: "Деление vs Умножение на обратное",
  fn: () => {
    let result = 0;
    for (let i = 1; i < iterations; i++) {
      result = i / 2; // или i * 0.5
    }
    return result;
  }
}
];

console.log("Бенчмарк арифметических операций:");
console.log("=".repeat(60));

```

```

tests.forEach(test => {
  const start = performance.now();
  test.fn();
  const end = performance.now();
  console.log(`${test.name.padEnd(40)} ${((end - start).toFixed(2))}ms`);
});
}

```

```

// Основные выводы:
// 1. Сложение чисел быстрее конкатенации строк
// 2. Битовые сдвиги быстрее умножения/деления на степени двойки
// 3. Умножение быстрее деления
// 4. Локальные переменные быстрее обращений к свойствам объектов

```

6.3. Лучшие практики

javascript

```

// 1. Всегда проверяйте типы при работе с пользовательским вводом

```

```

function safeAddition(a, b) {
  const numA = Number(a);
  const numB = Number(b);

  if (isNaN(numA) || isNaN(numB)) {
    throw new TypeError("Оба аргумента должны быть числами");
  }

  return numA + numB;
}

```

```

// 2. Используйте константы для "магических чисел"

```

```

const TAX_RATE = 0.20;
const DISCOUNT_PERCENTAGE = 15;
const MAX_RETRIES = 3;

function calculateTotal(price, quantity) {
  const subtotal = price * quantity;
  const tax = subtotal * TAX_RATE;
  const discount = subtotal * (DISCOUNT_PERCENTAGE / 100);
  return subtotal + tax - discount;
}

```

```

// 3. Разделяйте сложные выражения

```

// Плохо:

```
const result = (a + b) * (c - d) / (e ** f) % g;
```

// Хорошо:

```
const sum = a + b;
const difference = c - d;
const product = sum * difference;
const power = e ** f;
const quotient = product / power;
const finalResult = quotient % g;
```

// 4. Используйте вспомогательные функции для повторяющихся операций

```
function roundToTwoDecimals(value) {
    return Math.round(value * 100) / 100;
}
```

```
function clamp(value, min, max) {
    return Math.min(Math.max(value, min), max);
}
```

// 5. Документируйте сложные математические операции

*/***

** Вычисляет сложные проценты*

** @param {number} principal - Основная сумма*

** @param {number} rate - Годовая процентная ставка (%)*

** @param {number} time - Время в годах*

** @param {number} compoundsPerYear - Количество начислений в год*

** @returns {number} Итоговая сумма*

**/*

```
function calculateCompoundInterest(principal, rate, time, compoundsPerYear = 1) {
    return principal * Math.pow(1 + rate / (100 * compoundsPerYear), compoundsPerYear *
time);
}
```

// 6. Используйте современные возможности языка

// BigInt для больших чисел

```
const bigNumber = 1234567890123456789012345678901234567890n;
```

// Операторы нулевого слияния и опциональной цепочки

```
const defaultValue = user?.preferences?.taxRate ?? 0.20;
```

// Деструктуризация для работы с координатами

```
function distance({x: x1, y: y1}, {x: x2, y: y2}) {
    return Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);
}
```

}

Заключение

Арифметические операторы JavaScript — мощный инструмент, но требующий понимания их особенностей:

Ключевые выводы:

1. **Динамическая типизация** влияет на поведение операторов (особенно `+`)
2. **Точность вычислений** ограничена для чисел с плавающей точкой
3. **Порядок операций** критически важен для сложных выражений
4. **Составные операторы** упрощают код, но требуют осторожности
5. **Специальные значения** (Infinity, NaN, -0) имеют свои правила

Рекомендации по использованию:

Оператор	Когда использовать	Особенности
<code>+</code>	Сложение чисел или конкатенация строк	Преобразует в строку, если один операнд — строка
<code>- * /</code>	Математические операции	Преобразуют операнды в числа
<code>%</code>	Проверка четности, циклические значения	Сохраняет знак делимого
<code>**</code>	Возведение в степень	Правоассоциативный, альтернатива <code>Math.pow()</code>
<code>++ --</code>	Счетчики, итераторы	Разница между префиксной и постфиксной формой
<code>+= -=</code> и др.	Аккумуляторы, обновление значений	Эффективны и читаемы

Для профессионального использования:

1. **Для финансовых расчетов** используйте целые числа (копейки вместо рублей)

2. **Для научных вычислений** используйте специализированные библиотеки
3. **Для игр** оптимизируйте критические участки (битовые операции, предвычисления)
4. **Всегда проверяйте** пользовательский ввод и граничные условия
5. **Документируйте** сложные математические преобразования

Правильное использование арифметических операторов — основа эффективного и надежного JavaScript-кода. Понимание их поведения, особенно в контексте динамической типизации, позволяет избежать многих распространенных ошибок и писать код, который работает предсказуемо во всех ситуациях.

5.2. Операторы сравнения и равенства

Введение: сравнение как основа логики

Операторы сравнения и равенства — это фундаментальные инструменты JavaScript, которые позволяют сравнивать значения и принимать решения на основе этих сравнений. Понимание их работы, включая нюансы неявного преобразования типов, критически важно для написания корректного и предсказуемого кода.

В JavaScript существует два основных типа сравнения: **строгое** (strict) и **нестрогое** (loose, abstract). Их различия являются одним из самых известных и часто обсуждаемых аспектов языка.

Часть 1: Операторы равенства (Equality Operators)

1.1. Нестрогое равенство (== и !=)

Нестрогое равенство, также называемое "абстрактным равенством", сравнивает значения **после приведения их к общему типу**. Этот процесс называется **тип принуждение** (type coercion).

Как работает приведение типов при ==:

```
javascript
console.log(5 == 5);           // true - одинаковые значения и типы
console.log(5 == "5");        // true - строка "5" преобразуется в число 5
console.log(true == 1);       // true - boolean true преобразуется в число 1
console.log(false == 0);      // true - boolean false преобразуется в число 0
console.log(null == undefined); // true - специальное правило
console.log("" == false);     // true - пустая строка и false приводятся к 0
```

Алгоритм сравнения == (упрощенно):

1. Если типы одинаковые → сравниваются значения (как ===)
2. Если один операнд `null`, а другой `undefined` → `true`

3. Если один операнд — число, а другой — строка → строка преобразуется в число
4. Если один операнд — boolean → boolean преобразуется в число (true→1, false→0)
5. Если один операнд — объект, а другой — примитив → объект преобразуется в примитив

Особые (и часто сбивающие с толку) случаи:

```
javascript

console.log([] == false);      // true - массив преобразуется в пустую строку, затем в 0
console.log([] == ![]);       // true - ![] дает false, затем сравнение [] == false
console.log("\n " == 0);      // true - пробелы и переносы строк преобразуются в 0
console.log(" " == false);    // true - пробелы → 0, false → 0
console.log("0" == false);    // true - "0" → 0, false → 0
```

1.2. Строгое равенство (=== и !==)

Строгое равенство сравнивает значения **без приведения типов**. Если типы разные, результат всегда false.

Основные правила:

```
javascript

console.log(5 === 5);          // true - одинаковые значения и типы
console.log(5 === "5");        // false - разные типы (number vs string)
console.log(true === 1);       // false - разные типы (boolean vs number)
console.log(false === 0);      // false - разные типы
console.log(null === undefined); // false - разные типы
console.log("" === false);     // false - разные типы
console.log(NaN === NaN);      // false - специальное правило для NaN
```

Особенности строгого равенства:

```
javascript

// NaN всегда не равен самому себе
console.log(NaN === NaN);      // false
console.log(Number.isNaN(NaN)); // true - правильный способ проверки

// Нули разных типов
console.log(0 === -0);         // true - несмотря на разные знаки
console.log(Object.is(0, -0)); // false - Object.is различает ±0
```

```
// Объекты сравниваются по ссылке
const obj1 = { a: 1 };
const obj2 = { a: 1 };
const obj3 = obj1;

console.log(obj1 === obj2); // false - разные объекты в памяти
console.log(obj1 === obj3); // true - одна и та же ссылка
```

1.3. Рекомендации по использованию

Почему рекомендуется всегда использовать `===` и `!==`:

```
javascript

// Проблемы с нестрогим равенством в реальных сценариях

// Пример 1: Валидация формы
function validateForm(inputValue) {
  // ПЛОХО - может пропустить невалидные значения
  if (inputValue == "") {
    console.log("Поле пустое");
  }

  // ХОРОШО - явная проверка
  if (inputValue === "" || inputValue == null) {
    console.log("Поле пустое или null");
  }
}

// Проблема: inputValue может быть 0, false, [], или даже " "
console.log(0 == ""); // true - может быть неожиданно
console.log(false == ""); // true - может быть неожиданно

// Пример 2: Проверка на undefined/null
let variable;

// ПЛОХО - не различает undefined и null
if (variable == null) {
  console.log("undefined или null"); // Сработает для обоих
}

// ХОРОШО - явное различие
```

```
if (variable === undefined) {
  console.log("Переменная не инициализирована");
} else if (variable === null) {
  console.log("Переменная явно установлена в null");
}
```

// Пример 3: Работа с ID из разных источников

```
const idFromAPI = "123";      // строка из API
const idFromDatabase = 123;   // число из БД
```

// ПЛОХО - сравнение может дать ложноположительный результат

```
console.log(idFromAPI == idFromDatabase); // true - но типы разные!
```

// ХОРОШО - преобразуем к одному типу явно

```
console.log(Number(idFromAPI) === idFromDatabase); // true - и типы одинаковые
```

Исключения: когда `==` может быть полезен:

javascript

// Случай 1: Проверка на null или undefined

```
function processValue(value) {
  // Удобно: проверяем и null, и undefined одной операцией
  if (value == null) {
    return "Значение отсутствует";
  }
  // Эквивалентно: if (value === null || value === undefined)
}
```

// Случай 2: Работа с устаревшим API

```
function handleLegacyInput(input) {
  // Старые системы могут возвращать "0", 0, или даже ""
  if (input == 0) {
    // Обрабатывает 0, "0", false, null, undefined, []
    return "Нулевое значение";
  }
}
```

// Случай 3: Специальные случаи в библиотеках

// Некоторые библиотеки используют == для гибкости

// Но в собственном коде лучше быть явным

Часть 2: Операторы сравнения (Relational Operators)

2.1. Больше/меньше (>, <, >=, <=)

Эти операторы также выполняют приведение типов, но с некоторыми особенностями.

Сравнение чисел и строк:

```
javascript

console.log(5 > 3);           // true
console.log(10 < 20);        // true
console.log(5 >= 5);         // true
console.log(5 <= 4);         // false

// Приведение типов для сравнения
console.log("10" > 2);       // true - "10" преобразуется в 10
console.log("10" > "2");     // false - лексикографическое сравнение!
console.log("abc" > "abd");  // false - сравнение посимвольное
console.log("10" > "9");     // false - "1" < "9" в Unicode
```

Особые случаи:

```
javascript

// Сравнение с null и undefined
console.log(null > 0);       // false - null → 0, 0 > 0 = false
console.log(null >= 0);      // true - null → 0, 0 >= 0 = true
console.log(null < 0);       // false - null → 0, 0 < 0 = false
console.log(null <= 0);      // true - null → 0, 0 <= 0 = true

// undefined всегда дает false при сравнении
console.log(undefined > 0);  // false
console.log(undefined < 0);  // false
console.log(undefined == 0); // false (но undefined == null дает true!)

// Сравнение с NaN всегда дает false
console.log(NaN > 0);        // false
console.log(NaN < 0);        // false
console.log(NaN == NaN);     // false
console.log(NaN === NaN);    // false
```

Лексикографическое сравнение строк:

```
javascript

// Сравнение происходит посимвольно, по значению Unicode
console.log("apple" < "banana");    // true - "a" < "b"
console.log("apple" < "Apple");     // false - "a" (97) > "A" (65)
console.log("10" < "2");            // true - "1" < "2"
console.log("10" < "9");            // true - "1" < "9"

// Регистр имеет значение
console.log("Z" < "a");              // true - "Z" (90) < "a" (97)
console.log("z" < "a");              // true - латинская "z" < кириллическая "a"

// Длина строки учитывается только если префиксы одинаковые
console.log("abc" < "abcd");         // true - "abc" является префиксом "abcd"
console.log("abc" < "ab");           // false - "c" > отсутствию символа
```

2.2. Алгоритм сравнения для разных типов

Полный алгоритм (упрощенно):

1. Если оба операнда — строки → лексикографическое сравнение
2. Иначе → преобразование к числам и числовое сравнение
3. Особые правила для `null`, `undefined`, `NaN`

Примеры преобразований:

```
javascript

// Объекты преобразуются к примитивам
const obj = {
  valueOf() { return 10; },
  toString() { return "20"; }
};

console.log(obj > 5);           // true - valueOf() дает 10, 10 > 5
console.log(obj > "15");        // false - valueOf() дает 10, "15" → 15, 10 < 15

// Массивы преобразуются в строки
console.log([2] > [1]);         // true - [2] → "2", [1] → "1", "2" > "1"
console.log([10] > [2]);        // false - [10] → "10", [2] → "2", "10" < "2"
console.log([] > -1);           // true - [] → "", "" → 0, 0 > -1

// Boolean преобразуется к числам
```

```
console.log(true > false);    // true - 1 > 0
console.log(false > -1);      // true - 0 > -1
```

Часть 3: Специальные методы сравнения

3.1. `Object.is()` — "SameValue" сравнение

Метод `Object.is()` предоставляет наиболее строгий способ сравнения, учитывающий особые случаи.

```
javascript

// Отличия от ===
console.log(Object.is(0, -0));    // false (0 отличие от ===)
console.log(0 === -0);           // true

console.log(Object.is(NaN, NaN)); // true (0 отличие от ===)
console.log(NaN === NaN);         // false

// Обычные сравнения работают как ===
console.log(Object.is(5, 5));      // true
console.log(Object.is(5, "5"));    // false
console.log(Object.is({}, {}));    // false - разные объекты
console.log(Object.is(null, null)); // true
console.log(Object.is(undefined, undefined)); // true

// Практическое использование
function isExactlySame(a, b) {
    return Object.is(a, b);
}

// Особенно полезно для проверки NaN
function isReallyNaN(value) {
    return Object.is(value, NaN); // Или Number.isNaN(value)
}

// Проверка ±0
function isNegativeZero(value) {
    return Object.is(value, -0);
}
```

3.2. `Number.isNaN()` vs глобальный `isNaN()`

Важное различие при работе с NaN:

```
javascript

// Глобальный isNaN() выполняет преобразование типов
console.log(isNaN(NaN));           // true
console.log(isNaN("123"));        // false - "123" → 123, не NaN
console.log(isNaN("abc"));        // true - "abc" → NaN
console.log(isNaN(undefined));    // true - undefined → NaN
console.log(isNaN(null));         // false - null → 0, не NaN
console.log(isNaN(""));          // false - "" → 0, не NaN

// Number.isNaN() не преобразует типы
console.log(Number.isNaN(NaN));    // true
console.log(Number.isNaN("123"));  // false - строка, не NaN
console.log(Number.isNaN("abc"));  // false - строка, не NaN
console.log(Number.isNaN(undefined)); // false - undefined, не NaN
console.log(Number.isNaN(null));   // false - null, не NaN
console.log(Number.isNaN(""));     // false - строка, не NaN

// Рекомендуется всегда использовать Number.isNaN()
function safeIsNaN(value) {
    return Number.isNaN(value);
}

// Альтернатива: проверка через самосравнение
function isNaNSafe(value) {
    return value !== value; // Работает, т.к. NaN !== NaN
}
```

Часть 4: Практические паттерны и антипаттерны

4.1. Паттерны безопасного сравнения

Паттерн 1: Явное преобразование типов

```
javascript

// Вместо неявного преобразования
function compareUserIds(id1, id2) {
```

```

// ПЛОХО: может дать неожиданные результаты
// return id1 == id2;

// ХОРОШО: явное преобразование
return Number(id1) === Number(id2);
}

// Или более безопасно
function compareUserIdsSafe(id1, id2) {
  const num1 = Number(id1);
  const num2 = Number(id2);

  // Проверка на валидность чисел
  if (isNaN(num1) || isNaN(num2)) {
    return false;
  }

  return num1 === num2;
}

```

Паттерн 2: Сравнение с учетом граничных значений

```

javascript

// Сравнение чисел с плавающей точкой
function areNumbersEqual(a, b, epsilon = 1e-10) {
  // Прямое сравнение может не работать из-за погрешности
  // return a === b;

  // Сравнение с допуском
  return Math.abs(a - b) < epsilon;
}

console.log(0.1 + 0.2 === 0.3);           // false
console.log(areNumbersEqual(0.1 + 0.2, 0.3)); // true

// Сравнение больших чисел
function compareLargeNumbers(a, b) {
  // Для очень больших чисел используйте BigInt
  if (typeof a === 'bigint' || typeof b === 'bigint') {
    return BigInt(a) === BigInt(b);
  }

  // Или сравнение через строки для точности
  return String(a) === String(b);
}

```



```
}
```

Паттерн 3: Безопасное сравнение объектов

javascript

// Глубокое сравнение объектов (упрощенное)

```
function deepEqual(obj1, obj2) {
```

// Проверка на строгое равенство

```
  if (obj1 === obj2) {
```

```
    return true;
```

```
  }
```

// Проверка типов

```
  if (typeof obj1 !== 'object' || obj1 === null ||
```

```
      typeof obj2 !== 'object' || obj2 === null) {
```

```
    return false;
```

```
  }
```

// Сравнение ключей

```
  const keys1 = Object.keys(obj1);
```

```
  const keys2 = Object.keys(obj2);
```

```
  if (keys1.length !== keys2.length) {
```

```
    return false;
```

```
  }
```

// Рекурсивное сравнение значений

```
  for (const key of keys1) {
```

```
    if (!keys2.includes(key) || !deepEqual(obj1[key], obj2[key])) {
```

```
      return false;
```

```
    }
```

```
  }
```

```
  return true;
```

```
}
```

// Использование библиотек для глубокого сравнения

// Lodash: _.isEqual(obj1, obj2)

4.2. Антипаттерны сравнения

Антипаттерн 1: Смешивание `==` и `===`

```
javascript

// ПЛОХО: непоследовательность
function badComparison(a, b) {
  if (a == null) {
    // Используем ==
    return "null или undefined";
  }

  if (a === b) {
    // Используем ===
    return "строго равны";
  }

  // ПЛОХО: сравнение разных типов
  return a == b ? "нестрого равны" : "разные";
}

// ХОРОШО: последовательный стиль
function goodComparison(a, b) {
  // Всегда === для проверки определенных значений
  if (a === null || a === undefined) {
    return "null или undefined";
  }

  if (a === b) {
    return "строго равны";
  }

  // Явное преобразование если нужно
  const strA = String(a);
  const strB = String(b);

  return strA === strB ? "строки равны" : "разные";
}
```

Антипаттерн 2: Неявное преобразование в условиях

```
javascript

// ПЛОХО: неявное преобразование
```

```

function checkValue(value) {
  if (value) { // Неявное преобразование к boolean
    console.log("Truthy значение");
  } else {
    console.log("Falsy значение");
  }
}

// Проблема: что считается falsy?
console.log(checkValue(0)); // "Falsy значение" - может быть неожиданно
console.log(checkValue("")); // "Falsy значение"
console.log(checkValue(null)); // "Falsy значение"
console.log(checkValue(undefined)); // "Falsy значение"

// ХОРОШО: явные проверки
function checkValueExplicit(value) {
  if (value === undefined) {
    console.log("Не определено");
  } else if (value === null) {
    console.log("Явный null");
  } else if (value === 0) {
    console.log("Нулевое значение");
  } else if (value === "") {
    console.log("Пустая строка");
  } else if (value) {
    console.log("Truthy значение");
  } else {
    console.log("Другое falsy значение");
  }
}

```

Антипаттерн 3: Сравнение с помощью Boolean()

javascript

```

// ПЛОХО: избыточное преобразование
function isTruthy(value) {
  return Boolean(value) === true;
}

// ХОРОШО: простое преобразование
function isTruthySimple(value) {
  return !!value; // Двойное отрицание для boolean
}

```

```

// Еще лучше: явные проверки
function validateInput(input) {
  // ПЛОХО
  // if (Boolean(input.trim())) {

  // ХОРОШО
  if (input.trim().length > 0) {
    return "Валидный ввод";
  }
  return "Невалидный ввод";
}

```

Часть 5: Сравнение в различных контекстах

5.1. Сравнение в массивах и коллекциях

```

javascript

// Поиск в массиве
const numbers = [1, 2, 3, 4, 5];

// indexOf и includes используют ===
console.log(numbers.indexOf(2));      // 1
console.log(numbers.indexOf("2"));    // -1 (разные типы)
console.log(numbers.includes(3));      // true
console.log(numbers.includes("3"));    // false

// find и findIndex используют предоставленную функцию
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" }
];

// ПЛОХО: прямое сравнение объектов
console.log(users.indexOf({ id: 1, name: "Alice" })); // -1

// ХОРОШО: поиск по свойству
const user = users.find(u => u.id === 1);
console.log(user); // { id: 1, name: "Alice" }

// Set и Map используют "SameValueZero" алгоритм
const set = new Set([1, 2, 3, NaN]);

```

```
console.log(set.has(NaN));           // true (отличие от ===)
console.log(set.has("1"));           // false

const map = new Map();
map.set(NaN, "значение для NaN");
console.log(map.get(NaN));           // "значение для NaN"
```

5.2. Сравнение в условиях сортировки

javascript

```
// Стандартная сортировка использует преобразование к строкам
const mixedArray = [10, 2, "1", "10", "2", 1];
mixedArray.sort(); // Преобразует все к строкам!
console.log(mixedArray); // [1, "1", 10, "10", 2, "2"]
```

```
// Правильная сортировка чисел
const numbers = [10, 2, 1, 20];
numbers.sort((a, b) => a - b); // Используем явное сравнение
console.log(numbers); // [1, 2, 10, 20]
```

```
// Сортировка объектов
const products = [
  { name: "Product A", price: "100" },
  { name: "Product B", price: 50 },
  { name: "Product C", price: "75" }
];
```

```
// ПЛОХО: строковое сравнение цен
products.sort((a, b) => a.price > b.price ? 1 : -1);
```

```
// ХОРОШО: числовое сравнение
products.sort((a, b) => Number(a.price) - Number(b.price));
```

5.3. Сравнение при работе с API

javascript

```
// Получение данных из разных источников
async function compareAPIResponses() {
  const response1 = await fetch('/api/v1/data');
  const data1 = await response1.json(); // { id: "123", value: 42 }

  const response2 = await fetch('/api/v2/data');
```

```

const data2 = await response2.json(); // { id: 123, value: "42" }

// Проблема: разные форматы данных
console.log(data1.id == data2.id);      // true - "123" == 123
console.log(data1.value == data2.value); // true - 42 == "42"

// Решение: нормализация данных
const normalizeData = (data) => ({
  id: Number(data.id),
  value: Number(data.value)
});

const norm1 = normalizeData(data1);
const norm2 = normalizeData(data2);

console.log(norm1.id === norm2.id);      // true - оба числа
console.log(norm1.value === norm2.value); // true - оба числа
}

// Сравнение версий
function compareVersions(version1, version2) {
  // Преобразование "1.2.3" в [1, 2, 3]
  const v1 = version1.split('.').map(Number);
  const v2 = version2.split('.').map(Number);

  // Поэлементное сравнение
  for (let i = 0; i < Math.max(v1.length, v2.length); i++) {
    const num1 = v1[i] || 0;
    const num2 = v2[i] || 0;

    if (num1 > num2) return 1;
    if (num1 < num2) return -1;
  }

  return 0;
}

console.log(compareVersions("1.2.0", "1.10.0")); // -1 (2 < 10)
console.log(compareVersions("2.0", "2.0.0"));    // 0 (равны)

```

Часть 6: Производительность сравнений

6.1. Бенчмаркинг операторов сравнения

javascript

// Тестирование производительности

```
function benchmarkComparisons() {  
  const iterations = 1000000;  
  
  // === vs ==  
  console.time('strict equality');  
  for (let i = 0; i < iterations; i++) {  
    const result = 5 === 5;  
  }  
  console.timeEnd('strict equality');  
  
  console.time('loose equality');  
  for (let i = 0; i < iterations; i++) {  
    const result = 5 == 5;  
  }  
  console.timeEnd('loose equality');  
  
  // Разные типы  
  console.time('strict different types');  
  for (let i = 0; i < iterations; i++) {  
    const result = 5 === "5";  
  }  
  console.timeEnd('strict different types');  
  
  console.time('loose different types');  
  for (let i = 0; i < iterations; i++) {  
    const result = 5 == "5";  
  }  
  console.timeEnd('loose different types');  
}
```

// Результаты обычно показывают:

// - === быстрее, когда типы одинаковые

// - == может быть медленнее из-за преобразования типов

// - Разница обычно незначительна для большинства приложений

6.2. Оптимизация сравнений в циклах

javascript

// Оптимизация горячих путей

```
function processLargeArray(array) {  
  // ПЛОХО: преобразование в каждой итерации  
  for (let i = 0; i < array.length; i++) {  
    if (array[i] == targetValue) { // Преобразование типов  
      // ...  
    }  
  }  
}
```

// ХОРОШО: предварительное преобразование

```
const normalizedTarget = Number(targetValue);  
for (let i = 0; i < array.length; i++) {  
  if (Number(array[i]) === normalizedTarget) {  
    // ...  
  }  
}
```

// ЕЩЕ ЛУЧШЕ: использование Set для быстрого поиска

```
const lookupSet = new Set(array.map(Number));  
if (lookupSet.has(Number(targetValue))) {  
  // ...  
}
```

// Оптимизация сравнения объектов

```
function findObjectInArray(array, target) {  
  // МЕДЛЕННО: глубокое сравнение в каждой итерации  
  return array.find(item => deepEqual(item, target));
```

// БЫСТРЕЕ: сравнение по уникальному ID

```
return array.find(item => item.id === target.id);
```

// САМОЕ БЫСТРОЕ: использование Map

```
const map = new Map(array.map(item => [item.id, item]));  
return map.get(target.id);  
}
```


Заключение: Практические рекомендации

Основные правила:

1. **Всегда используйте `===` и `!==` по умолчанию** — это делает код предсказуемым и предотвращает множество трудноуловимых ошибок.
2. **Явное преобразование лучше неявного** — если нужно сравнить значения разных типов, преобразуйте их явно:

```
javascript
```

```
// Вместо
```

```
if (a == b) { ... }
```

```
// Используйте
```

```
if (Number(a) === Number(b)) { ... }
```

```
// или
```

```
if (String(a) === String(b)) { ... }
```

3. **Для проверки на `null/undefined` можно использовать `==`** — но документируйте это:

```
javascript
```

```
// Явно указываем в комментарии
```

```
if (value == null) { // Проверяет и null, и undefined
```

```
  // ...
```

```
}
```

4. **Используйте специальные методы для особых случаев:**

```
javascript
```

```
// Для NaN
```

```
Number.isNaN(value)
```

```
// Для строгого сравнения включая  $\pm 0$  и NaN
```

```
Object.is(a, b)
```

```
// Для сравнения чисел с плавающей точкой
```

```
Math.abs(a - b) < epsilon
```

5. **При работе с внешними данными нормализуйте их:**

```
javascript
```

```
function normalizeInput(input) {  
  return {  
    id: Number(input.id),  
    name: String(input.name || ''),  
    active: Boolean(input.active)  
  };  
};
```

}

Чек-лист для проверки кода:

- Все сравнения используют `===` и `!==`, кроме явно обоснованных случаев
- Проверка на `NaN` выполняется через `Number.isNaN()`
- Сравнение чисел с плавающей точкой использует `epsilon`
- Сравнение объектов выполняется по конкретным свойствам, а не целиком
- Для поиска в коллекциях используются специализированные методы
- Все неявные преобразования задокументированы

Помните:

"Программы должны писаться для людей, которые будут их читать, и лишь во вторую очередь для машин, которые будут их выполнять."
— Харольд Эбельсон

Операторы сравнения и равенства — это базовый, но мощный инструмент. Понимание их тонкостей отделяет начинающего разработчика от опытного профессионала. Всегда выбирайте ясность и предсказуемость над краткостью, и ваш код будет более надежным, поддерживаемым и понятным для всей команды.

5.3. Логические операторы и короткий цикл вычислений

Введение: логика как основа принятия решений

Логические операторы в JavaScript — это фундаментальные инструменты для построения сложных условий, управления потоком выполнения и обработки значений в контексте их "истинности" или "ложности". Особенность JavaScript заключается в поддержке **короткого цикла вычислений** (short-circuit evaluation), который не только оптимизирует производительность, но и открывает мощные паттерны программирования.

Часть 1: Основные логические операторы

1.1. Логическое И (AND) — &&

Оператор `&&` возвращает **первое ложное (falsy) значение** или **последнее истинное (truthy) значение**.

Таблица истинности для `&&`:

```
javascript
console.log(true && true);      // true
console.log(true && false);     // false
console.log(false && true);     // false (короткий цикл!)
console.log(false && false);    // false
```

Поведение с разными типами данных:

```
javascript
// Возвращает первое falsy значение
console.log(0 && 5);           // 0 (falsy)
console.log("" && "text");     // "" (falsy)
console.log(null && 42);       // null (falsy)
console.log(undefined && 100); // undefined (falsy)
console.log(NaN && "value");   // NaN (falsy)
```

```
// Возвращает последнее truthy значение
console.log(1 && 2);           // 2 (последнее truthy)
console.log("a" && "b");       // "b" (последнее truthy)
console.log([] && {});         // {} (оба truthy, возвращает последнее)
console.log(10 && true);        // true (последнее truthy)

// Комбинации
console.log(0 && false);        // 0 (первое falsy)
console.log(false && 0);        // false (первое falsy)
console.log("" && null);        // "" (первое falsy)
```

1.2. Логическое ИЛИ (OR) — ||

Оператор `||` возвращает **первое истинное (truthy) значение** или **последнее ложное (falsy) значение**.

Таблица истинности для `||`:

```
javascript

console.log(true || true);      // true (короткий цикл!)
console.log(true || false);     // true (короткий цикл!)
console.log(false || true);     // true
console.log(false || false);    // false
```

Поведение с разными типами данных:

```
javascript

// Возвращает первое truthy значение
console.log(0 || 5);            // 5 (первое truthy)
console.log("" || "text");      // "text" (первое truthy)
console.log(null || 42);        // 42 (первое truthy)
console.log(undefined || 100);  // 100 (первое truthy)
console.log(NaN || "value");    // "value" (первое truthy)

// Возвращает последнее falsy значение (если все falsy)
console.log(0 || false);        // false (последнее falsy)
console.log("" || null);        // null (последнее falsy)
console.log(null || undefined || NaN); // NaN (последнее falsy)
console.log(false || 0 || "");  // "" (последнее falsy)

// Практические примеры
console.log("existing" || "default"); // "existing"
```

```
console.log("" || "default");           // "default"
console.log(0 || 100);                   // 100
console.log(null || "fallback");         // "fallback"
```

1.3. Логическое НЕ (NOT) — !

Оператор `!` преобразует значение в `boolean` и инвертирует его.

Базовое использование:

```
javascript

console.log(!true);           // false
console.log(!false);          // true
console.log(!0);               // true (0 → false → true)
console.log(!1);               // false (1 → true → false)
console.log(!"text");          // false ("text" → true → false)
console.log(!"");              // true (" " → false → true)
console.log(!null);            // true (null → false → true)
console.log(!undefined);        // true (undefined → false → true)
console.log(!NaN);              // true (NaN → false → true)
```

Двойное НЕ (!!)

 для преобразования к `boolean`:

```
javascript

// Преобразование любого значения к boolean
console.log(!0);               // false
console.log(!1);               // true
console.log(!"text");          // true
console.log(!"");              // false
console.log(!null);            // false
console.log(!undefined);        // false
console.log(!NaN);              // false
console.log(![]);               // true (пустой массив - truthy!)
console.log(!{});               // true (пустой объект - truthy!)

// Эквивалентно Boolean()
console.log(!value === Boolean(value)); // всегда true
```

1.4. Nullish coalescing (Объединение с null) — ??

Оператор ?? возвращает правый операнд только если левый — null или undefined.

Отличие от ||:

```
javascript

// || возвращает правый операнд для ЛЮБОГО falsy значения
console.log(0 || "default");      // "default" (0 - falsy)
console.log("" || "default");     // "default" (" " - falsy)
console.log(false || "default");  // "default" (false - falsy)

// ?? возвращает правый операнд ТОЛЬКО для null/undefined
console.log(0 ?? "default");      // 0 (не null/undefined)
console.log("" ?? "default");     // "" (не null/undefined)
console.log(false ?? "default");  // false (не null/undefined)
console.log(NaN ?? "default");    // NaN (не null/undefined)

// Только для null/undefined
console.log(null ?? "default");   // "default"
console.log(undefined ?? "default"); // "default"
console.log(null ?? undefined ?? "fallback"); // "fallback"

// Практический пример
function getConfig(config) {
  const timeout = config.timeout ?? 30000; // 0 допустимо
  const retries = config.retries ?? 3;    // false допустимо
  const silent = config.silent ?? false;  // "" допустимо
}
```

Часть 2: Короткий цикл вычислений (Short-circuit Evaluation)

2.1. Как работает короткий цикл

Короткий цикл вычислений означает, что JavaScript вычисляет выражения только до того момента, когда результат становится однозначным.

Для оператора &&:

```
javascript

// Вычисление останавливается на первом falsy значении
function expensiveOperation() {
  console.log("Выполняется дорогая операция...");
  return true;
}

console.log(false && expensiveOperation());
// false, expensiveOperation() НЕ вызывается

console.log(true && expensiveOperation());
// "Выполняется дорогая операция...", затем true

// Практический пример с проверкой условий
function processUser(user) {
  // Проверки выполняются последовательно
  // Если какая-то проверка не пройдена, дальше не идем
  return user &&
    user.isActive &&
    user.hasPermission &&
    user.balance > 0;
}

const user = { isActive: false, hasPermission: true };
console.log(processUser(user)); // false, hasPermission не проверяется
```

Для оператора ||:

```
javascript

// Вычисление останавливается на первом truthy значении
function fallbackOperation() {
  console.log("Выполняется резервная операция...");
  return "fallback";
}

console.log("value" || fallbackOperation());
// "value", fallbackOperation() НЕ вызывается

console.log("" || fallbackOperation());
// "Выполняется резервная операция...", затем "fallback"
```

```
// Практический пример с получением значения
function getApiEndpoint() {
  return process.env.API_URL ||
    localStorage.getItem('api_url') ||
    "https://default-api.example.com";
}
```

2.2. Практическое применение короткого цикла

Паттерн 1: Защита от ошибок доступа к свойствам

```
javascript

// Без короткого цикла (многословно)
function getUsername(user) {
  if (user && user.profile && user.profile.name) {
    return user.profile.name;
  }
  return "Гость";
}

// С коротким циклом (компактно)
function getUsernameShort(user) {
  return user && user.profile && user.profile.name || "Гость";
}

// С опциональной цепочкой (modern)
function getUsernameModern(user) {
  return user?.profile?.name || "Гость";
}

// Пример использования
const user1 = { profile: { name: "Анна" } };
const user2 = { profile: {} };
const user3 = null;

console.log(getUsernameShort(user1)); // "Анна"
console.log(getUsernameShort(user2)); // "Гость"
console.log(getUsernameShort(user3)); // "Гость"
```

Паттерн 2: Условное выполнение функций

```
javascript
```



```

// Условный вызов функции только если условие истинно
function logMessage(message, shouldLog) {
  shouldLog && console.log(message);
  // Эквивалентно: if (shouldLog) console.log(message);
}

// Условное присваивание значения
function setDefaultConfig(config) {
  config.host = config.host || "localhost";
  config.port = config.port || 3000;
  config.debug = config.debug || false;

  // Более безопасно с ?? если допустимы falsy значения
  config.timeout = config.timeout ?? 5000; // 0 будет сохранен
}

// Условная загрузка модуля
function loadFeature(featureName, isEnabled) {
  isEnabled && import(`./features/${featureName}.js`)
    .then(module => module.initialize())
    .catch(err => console.warn(`Не удалось загрузить ${featureName}`));
}

```

Паттерн 3: Выбор между несколькими вариантами

```

javascript

// Получение первого доступного источника данных
function getData() {
  return fetchFromCache() ||
    fetchFromLocalStorage() ||
    fetchFromAPI() ||
    getDefaultData();
}

// Конфигурация с приоритетами
function getConfig() {
  return {
    apiUrl: getEnvVar('API_URL') ||
      getConfigFile().apiUrl ||
      "https://api.example.com",

    timeout: getUserPreference('timeout') ||
      getSystemDefault('timeout') ||
      30000,
  };
}

```

```

    theme: localStorage.getItem('theme') ||
      matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light' ||
      'light'
  };
}

```

2.3. Сложные цепочки с коротким циклом

javascript

```

// Обработка пользовательского ввода
function processInput(input) {
  // Цепочка проверок с коротким циклом
  const isValid = input &&
    typeof input === 'string' &&
    input.trim().length > 0 &&
    input.length <= 100;

  return isValid ? input.trim() : null;
}

// Построение URL с параметрами
function buildUrl(base, params) {
  // Только добавляем параметры если они существуют
  const queryString = params &&
    Object.keys(params).length > 0 &&
    new URLSearchParams(params).toString();

  return base + (queryString ? `?${queryString}` : "");
}

// Безопасное вычисление вложенных свойств
function calculateTotal(order) {
  return order &&
    order.items &&
    Array.isArray(order.items) &&
    order.items.reduce((sum, item) =>
      sum + (item && item.price && item.quantity ?
        item.price * item.quantity : 0), 0);
}

```

Часть 3: Комбинирование логических операторов

3.1. Приоритет логических операторов

```
javascript

// Порядок приоритета (от высокого к низкому):
// 1. ! (НЕ)
// 2. && (И)
// 3. || (ИЛИ)

// Примеры с расстановкой скобок для ясности
console.log(true || false && false); // true
// Эквивалентно: true || (false && false)

console.log(true && false || true); // true
// Эквивалентно: (true && false) || true

console.log(!true || false); // false
// Эквивалентно: (!true) || false

console.log(!(true || false)); // false
// Эквивалентно: !(true || false)

// Сложные выражения
const a = true, b = false, c = true, d = false;

console.log(a && b || c && d); // false
// ((a && b) || (c && d)) → (false || false) → false

console.log(a || b && c || d); // true
// (a || (b && c) || d) → (true || false || false) → true

console.log(!a && b || !c); // false
// ((!a && b) || !c) → ((false && false) || false) → false
```

3.2. Смешивание &&, || и ??

```
javascript

// Важно: ?? имеет более низкий приоритет чем && и ||
// но выше чем ?: (тернарный оператор)
```

```

// Примеры смешивания
console.log(null || "a" && "b");      // "b"
// Эквивалентно: null || ("a" && "b") → null || "b" → "b"

console.log("a" && null || "b");      // "b"
// Эквивалентно: ("a" && null) || "b" → null || "b" → "b"

// С ?? нужно быть осторожнее
console.log(null || undefined ?? "default"); // Ошибка!
// Нужны скобки: (null || undefined) ?? "default" → "default"

console.log(null ?? undefined || "default"); // "default"
// Эквивалентно: (null ?? undefined) || "default" → undefined || "default"

// Рекомендуется использовать скобки для ясности
function getValue(config) {
    // Четко указываем приоритет
    return (config.value ?? config.defaultValue) || "fallback";
}

function shouldProcess(item) {
    // Явная группировка
    return (item && item.isValid) || (item && item.forceProcess);
}

```

3.3. Оптимизация сложных условий

```

javascript

// ПЛОХО: избыточные проверки
function canAccessResource(user, resource) {
    if (user) {
        if (user.role === 'admin') {
            return true;
        } else if (user.permissions) {
            if (user.permissions.includes(resource)) {
                return true;
            }
        }
    }
    return false;
}

// ХОРОШО: использование короткого цикла

```

```

function canAccessResourceOptimized(user, resource) {
  return user && (
    user.role === 'admin' ||
    (user.permissions && user.permissions.includes(resource))
  );
}

// ЕЩЕ ЛУЧШЕ: с опциональной цепочкой
function canAccessResourceModern(user, resource) {
  return user?.role === 'admin' ||
    user?.permissions?.includes(resource) ||
    false;
}

// Пример с обработкой ошибок
function safeParseJSON(jsonString, defaultValue = {}) {
  try {
    return JSON.parse(jsonString) || defaultValue;
  } catch {
    return defaultValue;
  }
}

// Оптимизация с учетом вероятности условий
function processRequest(request) {
  // Ставим наиболее вероятные/дешевые проверки первыми
  return request &&
    request.method === 'GET' && // Самый частый метод
    request.headers && // Проверка быстрая
    validateHeaders(request.headers) && // Дорогая проверка - последняя
    processBody(request.body); // Самая дорогая операция
}

```

Часть 4: Falsy и Truthy значения

4.1. Полный список Falsy значений

```

javascript

// Все falsy значения в JavaScript:
const falsyValues = [
  false, // boolean false

```

```

0,          // число ноль
-0,         // отрицательный ноль
0n,         // BigInt ноль
"",         // пустая строка
'',         // пустая строка (одинарные кавычки)
``,         // пустая строка (обратные кавычки)
null,       // null
undefined,  // undefined
NaN         // Not a Number
];

// Проверка всех значений
falsyValues.forEach(value => {
  console.log(`!!${JSON.stringify(value)} = ${!!value}`); // Все false
});

// Особые случаи
console.log(!document.all); // false (устаревшее, но специфичное для браузеров)

```

4.2. Truthy значения

```

javascript

// Все остальные значения - truthy
const truthyValues = [
  true,          // boolean true
  1,             // любое ненулевое число
  -1,            // отрицательные числа
  3.14,          // дробные числа
  Infinity,      // бесконечность
  -Infinity,     // отрицательная бесконечность
  "0",           // строка "0"
  "false",       // строка "false"
  " ",           // строка с пробелом
  "hello",       // любая непустая строка
  [],            // пустой массив
  {},            // пустой объект
  function() {}, // функция
  /regex/,       // регулярное выражение
  new Date(),    // дата
  Symbol(),      // символ
  1n             // BigInt
];

```

```
// Проверка
truthyValues.forEach(value => {
  console.log(`!!${String(value)} = ${!!value}`); // Все true
});

// Неочевидные truthy значения
console.log(!![]);           // true - пустой массив
console.log(!!{});           // true - пустой объект
console.log(!!new Boolean(false)); // true - объект Boolean
console.log(!!" ");           // true - строка с пробелом
console.log(!!"0");           // true - строка "0"
console.log(!!"false");       // true - строка "false"
```

4.3. Практическое использование Truthy/Falsy

```
javascript

// Проверка на наличие значения
function hasValue(value) {
  // Проверяем, что значение не является falsy
  return !!value;
}

// Но лучше быть более конкретным
function hasRealValue(value) {
  // Проверяем, что значение не null/undefined
  return value != null;
}

function hasContent(str) {
  // Проверяем, что строка не пустая (с учетом пробелов)
  return str && str.trim().length > 0;
}

// Обработка пользовательского ввода
function processUserInput(input) {
  // Разные стратегии для разных типов данных

  // Для строк: проверяем не пустая ли
  if (typeof input === 'string') {
    return input.trim() || "default";
  }

  // Для чисел: 0 допустимо, но не undefined/null
```

```

if (typeof input === 'number') {
  return input != null ? input : 0;
}

// Для массивов: пустой массив допустим
if (Array.isArray(input)) {
  return input; // даже пустой массив - валидное значение
}

// Для объектов: пустой объект допустим
if (typeof input === 'object' && input !== null) {
  return input;
}

return null;
}

// Пример из реальной жизни: валидация формы
function validateForm(data) {
  const errors = {};

  // Имя обязательно и не может быть пустым
  if (!data.name || !data.name.trim()) {
    errors.name = "Имя обязательно";
  }

  // Email обязателен
  if (!data.email) {
    errors.email = "Email обязателен";
  } else if (!isValidEmail(data.email)) {
    errors.email = "Неверный формат email";
  }

  // Возраст: 0 допустим, но не undefined/null
  if (data.age == null) {
    errors.age = "Возраст обязателен";
  } else if (data.age < 0) {
    errors.age = "Возраст не может быть отрицательным";
  }

  // Хобби: пустой массив допустим
  if (!Array.isArray(data.hobbies)) {
    errors.hobbies = "Хобби должны быть массивом";
  }
}

```



```
    return Object.keys(errors).length === 0 ? null : errors;
}
```

Часть 5: Продвинутые паттерны

5.1. Паттерн "Защита от выполнения" (Guard Pattern)

```
javascript

// Базовый guard pattern
function processUser(user) {
    // Защитные условия в начале функции
    if (!user || !user.id) {
        console.warn("Невалидный пользователь");
        return null;
    }

    if (!user.isActive) {
        console.warn("Пользователь не активен");
        return null;
    }

    // Основная логика
    return calculateUserStats(user);
}

// Guard pattern с коротким циклом
function processOrder(order) {
    // Все проверки в одной строке с коротким циклом
    return order &&
        order.items &&
        order.items.length > 0 &&
        order.customer &&
        order.customer.email &&
        calculateTotal(order);
}

// Guard pattern для асинхронных операций
async function fetchUserData(userId) {
    // Проверяем входные параметры
    if (!userId || typeof userId !== 'string') {
```

```

    throw new Error("Невалидный userId");
}

// Проверяем доступность сервиса
if (!isApiAvailable()) {
    throw new Error("Сервис недоступен");
}

// Выполняем основную логику
return await api.get(`/users/${userId}`);
}

```

5.2. Паттерн "Значения по умолчанию"

javascript

```

// Стандартные значения по умолчанию
function createUser(options = {}) {
    const defaults = {
        name: "Гость",
        role: "user",
        isActive: true,
        preferences: {}
    };

    // Объединяем с помощью || с учетом falsy значений
    return {
        name: options.name || defaults.name,
        role: options.role || defaults.role,
        isActive: options.isActive !== null ? options.isActive : defaults.isActive,
        preferences: options.preferences || defaults.preferences
    };
}

// Более современный подход с ?? и деструктуризацией
function createUserModern({
    name = "Гость",
    role = "user",
    isActive = true,
    preferences = {}
} = {}) {
    return { name, role, isActive, preferences };
}

```

```
// Значения по умолчанию для конфигурации
function createAppConfig(userConfig = {}) {
  // Используем ?? для сохранения falsy значений (0, false, "")
  return {
    apiUrl: userConfig.apiUrl ?? process.env.API_URL ?? "https://api.example.com",
    timeout: userConfig.timeout ?? 30000,
    retries: userConfig.retries ?? 3,
    debug: userConfig.debug ?? false,
    logLevel: userConfig.logLevel || "info" // "" → "info"
  };
}
```

5.3. Паттерн "Условное присваивание"

javascript

// Базовое условное присваивание

```
let value;
const input = getUserInput();
```

// Стандартный подход

```
if (input && input.trim()) {
  value = input.trim();
} else {
  value = "default";
}
```

// С использованием || (если пустая строка недопустима)

```
value = (input && input.trim()) || "default";
```

// С использованием ?? (если пустая строка допустима)

```
value = input?.trim() ?? "default";
```

// Условное присваивание в объектах

```
function createMessage(config) {
  const message = {
    text: config.text || "Сообщение по умолчанию",
    // Условно добавляем свойства
    ...(config.priority && { priority: config.priority }),
    ...(config.expires && { expiresAt: new Date(config.expires) }),
    ...(config.metadata && { meta: config.metadata })
  };

  return message;
}
```

```

}

// Условное присваивание в массивах
function getItemWithDefaults(items, defaultItems) {
  return [
    ...(items || []), // Если items нет, используем пустой массив
    ...defaultItems
  ];
}

```

5.4. Паттерн "Цепочка ответственности" (Chain of Responsibility)

javascript

```

// Обработка запроса через цепочку
function processRequest(request) {
  // Каждая функция в цепочке может обработать или передать дальше
  return validateRequest(request) &&
    authenticate(request) &&
    authorize(request) &&
    processBusinessLogic(request);
}

// Каждая функция возвращает truthy/falsy
function validateRequest(request) {
  if (!request || !request.method) {
    console.error("Невалидный запрос");
    return false;
  }
  return request;
}

function authenticate(request) {
  if (!request.token) {
    console.error("Требуется аутентификация");
    return false;
  }

  const user = verifyToken(request.token);
  if (!user) {
    console.error("Неверный токен");
    return false;
  }
}

```

```

    request.user = user;
    return request;
}

function authorize(request) {
    if (!request.user || !request.user.hasPermission(request.resource)) {
        console.error("Доступ запрещен");
        return false;
    }
    return request;
}

function processBusinessLogic(request) {
    // Основная логика
    return { success: true, data: "Обработано" };
}

```

Часть 6: Производительность и оптимизация

6.1. Бенчмаркинг логических операторов

```

javascript

function benchmarkLogicalOperators() {
    const iterations = 1000000;
    const testValue = "test";
    const falsyValue = "";

    console.time('|| with truthy');
    for (let i = 0; i < iterations; i++) {
        const result = testValue || "default";
    }
    console.timeEnd('|| with truthy');

    console.time('|| with falsy');
    for (let i = 0; i < iterations; i++) {
        const result = falsyValue || "default";
    }
    console.timeEnd('|| with falsy');

    console.time('?? with truthy');

```

```

for (let i = 0; i < iterations; i++) {
  const result = testValue ?? "default";
}
console.timeEnd('?? with truthy');

console.time('?? with null');
for (let i = 0; i < iterations; i++) {
  const result = null ?? "default";
}
console.timeEnd('?? with null');

console.time('ternary with truthy');
for (let i = 0; i < iterations; i++) {
  const result = testValue ? testValue : "default";
}
console.timeEnd('ternary with truthy');
}

// Результаты обычно показывают:
// 1. Логические операторы быстрее тернарных
// 2. Короткий цикл дает выгоду когда вычисление останавливается рано
// 3. Разница обычно незначительна для большинства случаев

```

6.2. Оптимизация горячих путей

```

javascript

// Оптимизация критического пути
function processItems(items) {
  // ПЛОХО: дорогая операция выполняется всегда
  return items && items.length > 0 && expensiveFilter(items);

  // ХОРОШО: сначала дешевые проверки
  if (!items || items.length === 0) {
    return [];
  }

  // Только потом дорогая операция
  return expensiveFilter(items);
}

// Кэширование результатов
function getCachedData(key) {
  // Используем короткий цикл для кэширования

```

```

    return cache[key] || (cache[key] = fetchData(key));
}

// Оптимизация условий в циклах
function processArray(array) {
    const results = [];

    // ПЛОХО: условие проверяется в каждой итерации
    for (let i = 0; i < array.length; i++) {
        if (shouldProcess && isValid(array[i])) {
            results.push(processItem(array[i]));
        }
    }

    // ХОРОШО: выносим условие из цикла
    if (!shouldProcess) return [];

    for (let i = 0; i < array.length; i++) {
        if (isValid(array[i])) {
            results.push(processItem(array[i]));
        }
    }

    return results;
}

```

Часть 7: Распространенные ошибки и их решение

7.1. Ошибка: Путаница между `||` и `??`

```

javascript

// НЕПРАВИЛЬНО: использование || когда нужен ??
function setConfig(config) {
    // Проблема: если config.timeout = 0, он будет заменен на 30000
    return {
        timeout: config.timeout || 30000, // 0 → 30000 (может быть нежелательно)
        retries: config.retries || 3      // false → 3 (может быть нежелательно)
    };
}

// ПРАВИЛЬНО: использование ??

```

```
function setConfigCorrect(config) {
  return {
    timeout: config.timeout ?? 30000, // 0 → 0 (сохраняется)
    retries: config.retries ?? 3,    // false → false (сохраняется)
    enabled: config.enabled || true  // false → true (желаемое поведение)
  };
}

// Решение: четко определяем, что допустимо
function setConfigSafe(config) {
  // Для таймаута: 0 допустим, но не undefined
  const timeout = config.timeout != null ? config.timeout : 30000;

  // Для повторных попыток: false недопустим, только число
  const retries = typeof config.retries === 'number' ? config.retries : 3;

  return { timeout, retries };
}
```

7.2. Ошибка: Неожиданное преобразование типов

```
javascript

// НЕПРАВИЛЬНО: неявное преобразование может дать неожиданные результаты
function calculateTotal(items) {
  // Проблема: items может быть 0, что даст 0 || [] → []
  const safeItems = items || [];
  return safeItems.reduce((sum, item) => sum + item.price, 0);
}

console.log(calculateTotal(0)); // 0, но ожидалась ошибка или другое поведение

// ПРАВИЛЬНО: явная проверка типа
function calculateTotalSafe(items) {
  if (!Array.isArray(items)) {
    throw new Error("items должен быть массивом");
  }

  return items.reduce((sum, item) => sum + (item?.price || 0), 0);
}

// Или более безопасно
function calculateTotalModern(items = []) {
  const safeItems = Array.isArray(items) ? items : [];
```



```
return safeItems.reduce((sum, item) => {
  const price = Number(item?.price) || 0;
  return sum + price;
}, 0);
}
```

7.3. Ошибка: Избыточные проверки

javascript

// НЕПРАВИЛЬНО: избыточные проверки усложняют код

```
function getUserData(user) {
  if (user) {
    if (user.profile) {
      if (user.profile.name) {
        return user.profile.name;
      } else {
        return "Без имени";
      }
    } else {
      return "Нет профиля";
    }
  } else {
    return "Нет пользователя";
  }
}
```

// ПРАВИЛЬНО: используем цепочки и значения по умолчанию

```
function getUserDataClean(user) {
  return user?.profile?.name || "Без имени";
}
```

// Если нужны разные сообщения об ошибках

```
function getUserDataDetailed(user) {
  if (!user) return "Нет пользователя";
  if (!user.profile) return "Нет профиля";
  if (!user.profile.name) return "Без имени";
  return user.profile.name;
}
```

Заключение: Практические рекомендации

Основные правила:

1. **Используйте `||` для значений по умолчанию**, когда все falsy значения недопустимы:

javascript

```
const name = inputName || "Гость"; // "" → "Гость", 0 → "Гость"
```

2. **Используйте `??` для значений по умолчанию**, когда нужно сохранить falsy значения (0, false, ""):

javascript

```
const timeout = config.timeout ?? 30000; // 0 → 0, false → false
```

3. **Используйте `&&` для условного выполнения**, когда нужно проверить несколько условий:

javascript

```
user && user.isActive && user.hasPermission && processUser(user);
```

4. **Избегайте сложных цепочек без скобок** — явная группировка делает код понятнее:

javascript

// Вместо

```
const result = a || b && c || d;
```

// Используйте

```
const result = a || (b && c) || d;
```

5. **Помните о приоритетах операторов:**

- ➔ `!` (НЕ) — самый высокий
- ➔ `&&` (И) — средний
- ➔ `||` (ИЛИ) — низкий
- ➔ `??` (Nullish) — самый низкий из логических

Чек-лист для проверки кода:

- Логические операторы используются для короткого цикла вычислений
- `||` используется только когда все falsy значения недопустимы
- `??` используется для сохранения falsy значений (0, false, "")
- Сложные выражения имеют явную группировку скобками
- Проверки упорядочены от наиболее вероятных/дешевых к наименее вероятным/дорогим
- Нет избыточных проверок и дублирования условий
- Используются современные операторы (`?.`, `??`) где это уместно

Итоговый пример хорошего кода:

```
javascript

function processOrder(order, user, config) {
  // Guard clauses - проверяем входные данные
  if (!order || !order.items?.length) {
    throw new Error("Невалидный заказ");
  }

  if (!user?.isActive) {
    throw new Error("Пользователь не активен");
  }

  // Настройки с умолчаниями
  const settings = {
    taxRate: config?.taxRate ?? 0.2,
    discount: user.isPremium ? 0.1 : 0,
    currency: config?.currency || "USD",
    applyVat: config?.applyVat ?? true
  };

  // Вычисление с проверками
  const subtotal = order.items.reduce((sum, item) =>
    sum + (item?.price || 0) * (item?.quantity || 1), 0);

  // Условные скидки
  const discount = subtotal > 1000 ? 0.05 : settings.discount;

  // Финальный расчет
  const total = subtotal * (1 - discount) * (1 + settings.taxRate);

  // Логирование только в debug режиме
  config?.debug && console.log("Расчет завершен:", { subtotal, discount, total });

  return {
    success: true,
    total: Math.round(total * 100) / 100,
    currency: settings.currency
  };
}
```

Запомните: Логические операторы с коротким циклом вычислений — это не просто синтаксический сахар, а мощный инструмент для написания чистого, эффективного и выразительного кода. Правильное их использование делает код

более читаемым, уменьшает количество временных переменных и предотвращает ненужные вычисления.

5.4. Побитовые операторы

Введение: битовые операции в высокоуровневом языке

Побитовые операторы в JavaScript позволяют работать с числами на уровне отдельных битов. Хотя JavaScript является высокоуровневым языком, побитовые операции остаются важным инструментом для оптимизации, криптографии, низкоуровневых манипуляций и решения специфических задач.

Часть 1: Основы двоичной системы и представление чисел

1.1. Двоичное представление чисел

```
javascript

// JavaScript хранит числа в 64-битном формате с плавающей точкой (IEEE 754)
// Но побитовые операции работают с 32-битными целыми числами

// Преобразование в двоичную систему
function toBinary(num, bits = 32) {
  // Для положительных чисел
  if (num >= 0) {
    return num.toString(2).padStart(bits, '0');
  }

  // Для отрицательных чисел (дополнительный код)
  return (num >>> 0).toString(2).padStart(bits, '0');
}

// Примеры
console.log(toBinary(5, 8));           // 00000101
console.log(toBinary(255, 8));         // 11111111
console.log(toBinary(-5, 8));          // 11111011 (в дополнительном коде)
console.log(toBinary(0, 8));           // 00000000
console.log(toBinary(1, 8));           // 00000001

// Различные системы счисления
```

```
console.log((10).toString(2));    // "1010" - двоичная
console.log((10).toString(8));    // "12" - восьмеричная
console.log((10).toString(10));   // "10" - десятичная
console.log((10).toString(16));   // "a" - шестнадцатеричная
```

// Прямые литералы

```
console.log(0b1010);             // 10 - двоичный литерал (ES6)
console.log(0o12);               // 10 - восьмеричный литерал (ES6)
console.log(0xa);               // 10 - шестнадцатеричный литерал
```

1.2. Формат представления чисел

javascript

// Положительные числа хранятся в прямом коде

// Отрицательные числа - в дополнительном коде

// Пределы 32-битных целых чисел

```
const MAX_32BIT = 2147483647;    // 231 - 1
const MIN_32BIT = -2147483648;   // -231
```

// При выполнении побитовых операций:

// 1. Операнды преобразуются в 32-битные целые числа

// 2. Выполняется операция

// 3. Результат преобразуется обратно в 64-битный формат

// Пример преобразования

```
console.log(toBinary(5.7));       // 000...000101 (дробная часть отбрасывается)
console.log(toBinary("5"));       // 000...000101 (строка преобразуется)
console.log(toBinary(true));      // 000...000001 (true → 1)
console.log(toBinary(false));     // 000...000000 (false → 0)
console.log(toBinary(null));      // 000...000000 (null → 0)
console.log(toBinary(undefined)); // 000...000000 (undefined → 0)
console.log(toBinary(NaN));       // 000...000000 (NaN → 0)
```

// Большие числа обрезаются

```
console.log(toBinary(4294967296)); // 000...000000 (232 обрезаются до 0)
console.log(toBinary(4294967295)); // 111...111111 (232 - 1 = -1 в доп. коде)
```

1.3. Дополнительный код (Two's Complement)

javascript

// Дополнительный код для отрицательных чисел:

```

// 1. Берется двоичное представление абсолютного значения
// 2. Инвертируются все биты (получаем обратный код)
// 3. Добавляется 1

function toTwosComplement(num, bits = 8) {
  if (num >= 0) {
    return num.toString(2).padStart(bits, '0');
  }

  // Для отрицательных чисел
  const positive = (-num).toString(2).padStart(bits, '0');
  let inverted = '';

  // Инвертируем биты
  for (let i = 0; i < bits; i++) {
    inverted += positive[i] === '1' ? '0' : '1';
  }

  // Добавляем 1
  let result = '';
  let carry = 1;

  for (let i = bits - 1; i >= 0; i--) {
    const bit = parseInt(inverted[i]);
    const sum = bit + carry;

    if (sum === 2) {
      result = '0' + result;
      carry = 1;
    } else {
      result = sum.toString() + result;
      carry = 0;
    }
  }

  return result;
}

// Примеры
console.log(toTwosComplement(5, 8));    // 00000101
console.log(toTwosComplement(-5, 8));   // 11111011
console.log(toTwosComplement(127, 8));  // 01111111
console.log(toTwosComplement(-128, 8)); // 10000000 (минимальное для 8 бит)
console.log(toTwosComplement(0, 8));    // 00000000

```

Часть 2: Основные побитовые операторы

2.1. Побитовое И (AND) — &

Оператор & сравнивает соответствующие биты двух чисел и возвращает 1 только если оба бита равны 1.

Таблица истинности:

text

0 & 0 = 0

0 & 1 = 0

1 & 0 = 0

1 & 1 = 1

Примеры:

javascript

// Базовые примеры

console.log(5 & 3); // 1

// 5 = 00000101

// 3 = 00000011

// & = 00000001 = 1

console.log(12 & 7); // 4

// 12 = 00001100

// 7 = 00000111

// & = 00000100 = 4

console.log(255 & 170); // 170

// 255 = 11111111

// 170 = 10101010

// & = 10101010 = 170

// С отрицательными числами

console.log(-5 & 3); // 3

// -5 = 11111011 (в доп. коде)

// 3 = 00000011

// & = 00000011 = 3

console.log(-1 & 255); // 255


```
// -1 = 11111111 (все биты установлены)
// 255= 11111111
// & = 11111111 = 255
```

2.2. Побитовое ИЛИ (OR) — |

Оператор | сравнивает соответствующие биты двух чисел и возвращает 1 если хотя бы один бит равен 1.

Таблица истинности:

text

```
0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1
```

Примеры:

javascript

```
// Базовые примеры
console.log(5 | 3);    // 7
// 5 = 00000101
// 3 = 00000011
// | = 00000111 = 7

console.log(12 | 7);   // 15
// 12 = 00001100
// 7  = 00000111
// |  = 00001111 = 15

console.log(85 | 170); // 255
// 85  = 01010101
// 170 = 10101010
// |   = 11111111 = 255

// С отрицательными числами
console.log(-5 | 3);   // -5
// -5 = 11111011
// 3  = 00000011
// |  = 11111011 = -5
```

```
console.log(0 | -1);    // -1
// 0   = 00000000
// -1  = 11111111
// |   = 11111111 = -1
```

2.3. Побитовое исключающее ИЛИ (XOR) — ^

Оператор ^ сравнивает соответствующие биты двух чисел и возвращает 1 если биты разные.

Таблица истинности:

```
text

0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0
```

Примеры:

```
javascript

// Базовые примеры
console.log(5 ^ 3);    // 6
// 5 = 00000101
// 3 = 00000011
// ^ = 00000110 = 6

console.log(12 ^ 7);   // 11
// 12 = 00001100
// 7  = 00000111
// ^  = 00001011 = 11

console.log(85 ^ 170); // 255
// 85  = 01010101
// 170 = 10101010
// ^   = 11111111 = 255

// Особое свойство: a ^ a = 0
console.log(5 ^ 5);    // 0
console.log(123 ^ 123); // 0

// Свойство: a ^ 0 = a
```

```
console.log(5 ^ 0);    // 5
console.log(123 ^ 0);  // 123
```

```
// С отрицательными числами
console.log(-5 ^ 3);   // -8
// -5 = 11111011
// 3  = 00000011
// ^   = 11111000 = -8
```

2.4. Побитовое НЕ (NOT) — ~

Оператор ~ инвертирует все биты числа (меняет 0 на 1 и 1 на 0).

Особенности:

- Инвертирует все 32 бита
- Для положительных чисел: $\sim n = -(n + 1)$
- Для отрицательных чисел: $\sim n = -n - 1$

Примеры:

```
javascript

console.log(~5);          // -6
// 5 = 00000000 00000000 00000000 00000101
// ~5= 11111111 11111111 11111111 11111010 = -6

console.log(~0);          // -1
// 0 = 00000000 00000000 00000000 00000000
// ~0= 11111111 11111111 11111111 11111111 = -1

console.log(~-5);         // 4
// -5= 11111111 11111111 11111111 11111011
// ~-5=00000000 00000000 00000000 00000100 = 4

console.log(~255);        // -256
console.log(~1);          // -2
console.log(~-1);          // 0 (интересный случай!)

// Проверка формулы  $\sim n = -(n + 1)$ 
console.log(~5 === -(5 + 1)); // true
console.log(~0 === -(0 + 1)); // true
console.log(~-5 === -(-5 + 1)); // true (4 = -(-4))
```

2.5. Сдвиг влево (Left Shift) — <<

Оператор << сдвигает биты влево на указанное количество позиций. Освободившиеся справа биты заполняются нулями.

Математически: $a \ll b = a * 2^b$

```
javascript

// Базовые примеры
console.log(5 << 1);    // 10
// 5 = 00000101
// <<1=00001010 = 10 (5 * 2^1)

console.log(5 << 2);    // 20
// 5 = 00000101
// <<2=00010100 = 20 (5 * 2^2)

console.log(1 << 3);    // 8
// 1 = 00000001
// <<3=00001000 = 8 (1 * 2^3)

console.log(7 << 4);    // 112
// 7 = 00000111
// <<4=01110000 = 112 (7 * 2^4)

// С отрицательными числами
console.log(-5 << 1);   // -10
// -5 = 11111011
// <<1 = 11110110 = -10

console.log(-1 << 3);   // -8
// -1 = 11111111
// <<3 = 11111000 = -8

// Пределы (сдвиг больше 31)
console.log(1 << 31);   // -2147483648 (становится отрицательным!)
console.log(1 << 32);   // 1 (32 ≡ 0 mod 32)
console.log(1 << 33);   // 2 (33 ≡ 1 mod 32)
```

2.6. Сдвиг вправо с сохранением знака (Right Shift) — `>>`

Оператор `>>` сдвигает биты вправо на указанное количество позиций. Освободившиеся слева биты заполняются значением знакового бита (0 для положительных, 1 для отрицательных).

Математически для положительных: $a \gg b = \text{floor}(a / 2^b)$

```
javascript

// Базовые примеры
console.log(10 >> 1); // 5
// 10 = 00001010
// >>1=00000101 = 5 (floor(10 / 2^1))

console.log(20 >> 2); // 5
// 20 = 00010100
// >>2=00000101 = 5 (floor(20 / 2^2))

console.log(8 >> 3); // 1
// 8 = 00001000
// >>3=00000001 = 1 (floor(8 / 2^3))

console.log(15 >> 2); // 3
// 15 = 00001111
// >>2=00000011 = 3 (floor(15 / 4))

// С отрицательными числами (сохраняет знак)
console.log(-10 >> 1); // -5
// -10 = 11110110
// >>1 = 11111011 = -5

console.log(-20 >> 2); // -5
// -20 = 11101100
// >>2 = 11111011 = -5

console.log(-1 >> 5); // -1 (все биты остаются 1)
// -1 = 11111111
// >>5 = 11111111 = -1

// Особые случаи
console.log(1 >> 1); // 0
console.log(0 >> 5); // 0
```

```
console.log(-0 >> 5); // 0
```

2.7. Сдвиг вправо с заполнением нулями (Zero-fill Right Shift)

— >>>

Оператор >>> сдвигает биты вправо на указанное количество позиций. Освободившиеся слева биты всегда заполняются нулями, независимо от знака числа.

javascript

```
// Для положительных чисел работает как >>
```

```
console.log(10 >>> 1); // 5
```

```
console.log(20 >>> 2); // 5
```

```
// Для отрицательных чисел - различие!
```

```
console.log(-10 >> 1); // -5 (сохраняет знак)
```

```
console.log(-10 >>> 1); // 2147483643 (заполняет нулями!)
```

```
// -10 в 32-битном представлении:
```

```
// -10 = 11111111 11111111 11111111 11110110
```

```
// >>>1=01111111 11111111 11111111 11111011 = 2147483643
```

```
console.log(-1 >> 1); // -1
```

```
console.log(-1 >>> 1); // 2147483647 (максимальное 32-битное положительное)
```

```
// -1 = 11111111 11111111 11111111 11111111
```

```
// >>>1=01111111 11111111 11111111 11111111 = 231 - 1
```

```
// Полезное свойство: преобразование в беззнаковое 32-битное число
```

```
function toUint32(num) {  
    return num >>> 0;  
}
```

```
console.log(toUint32(-1)); // 4294967295
```

```
console.log(toUint32(-5)); // 4294967291
```

```
console.log(toUint32(5)); // 5
```

```
console.log(toUint32(3.14)); // 3 (дробная часть отбрасывается)
```

Часть 3: Практическое применение побитовых операций

3.1. Флаги и битовые маски

```
javascript

// Определение флагов
const Permissions = {
  NONE:      0b0000, // 0
  READ:      0b0001, // 1
  WRITE:     0b0010, // 2
  EXECUTE:   0b0100, // 4
  DELETE:    0b1000, // 8

  // Комбинированные флаги
  READ_WRITE: 0b0011, // READ | WRITE
  ALL:        0b1111  // READ | WRITE | EXECUTE | DELETE
};

// Установка флагов
function setPermission(current, permission) {
  return current | permission;
}

// Снятие флагов
function clearPermission(current, permission) {
  return current & ~permission;
}

// Проверка флагов
function hasPermission(current, permission) {
  return (current & permission) === permission;
}

// Переключение флагов
function togglePermission(current, permission) {
  return current ^ permission;
}

// Пример использования
let userPermissions = Permissions.NONE;

// Добавляем разрешения
```

```

userPermissions = setPermission(userPermissions, Permissions.READ);
userPermissions = setPermission(userPermissions, Permissions.WRITE);

console.log(hasPermission(userPermissions, Permissions.READ)); // true
console.log(hasPermission(userPermissions, Permissions.WRITE)); // true
console.log(hasPermission(userPermissions, Permissions.EXECUTE)); // false

// Переключаем WRITE
userPermissions = togglePermission(userPermissions, Permissions.WRITE);
console.log(hasPermission(userPermissions, Permissions.WRITE)); // false

// Устанавливаем все разрешения
userPermissions = Permissions.ALL;
console.log(userPermissions.toString(2).padStart(4, '0')); // 1111

// Проверяем несколько флагов одновременно
function hasAllPermissions(current, required) {
    return (current & required) === required;
}

function hasAnyPermission(current, required) {
    return (current & required) !== 0;
}

const required = Permissions.READ | Permissions.WRITE;
console.log(hasAllPermissions(userPermissions, required)); // true

```

3.2. Упаковка нескольких значений в одно число

```

javascript

// Упаковка RGB цвета в 32-битное число
function packRGB(r, g, b, a = 255) {
    // Каждый компонент - 8 бит (0-255)
    // Формат: 0xAABBGGRR или 0xAARRGGBB в зависимости от порядка
    return ((a & 0xFF) << 24) |
        ((b & 0xFF) << 16) |
        ((g & 0xFF) << 8) |
        (r & 0xFF);
}

// Распаковка RGB цвета
function unpackRGB(color) {
    const r = color & 0xFF;

```



```

const g = (color >> 8) & 0xFF;
const b = (color >> 16) & 0xFF;
const a = (color >> 24) & 0xFF;

return { r, g, b, a };
}

// Пример
const white = packRGB(255, 255, 255);
console.log(white.toString(16)); // fffffff (или ffffffff с альфа)
console.log(unpackRGB(white)); // { r: 255, g: 255, b: 255, a: 255 }

const red = packRGB(255, 0, 0, 128);
console.log(red.toString(16)); // 80ff0000
console.log(unpackRGB(red)); // { r: 255, g: 0, b: 0, a: 128 }

// Упаковка координат (x, y) в 32-битное число
function packCoordinates(x, y, max = 65535) {
  // Предполагаем, что x и y в диапазоне 0-65535 (16 бит каждое)
  return ((x & 0xFFFF) << 16) | (y & 0xFFFF);
}

function unpackCoordinates(packed) {
  const x = (packed >> 16) & 0xFFFF;
  const y = packed & 0xFFFF;
  return { x, y };
}

// Пример
const coords = packCoordinates(1000, 2000);
console.log(coords.toString(16)); // 7d007d0
console.log(unpackCoordinates(coords)); // { x: 1000, y: 2000 }

```

3.3. Быстрые математические операции

```

javascript

// Умножение и деление на степени двойки
function multiplyByPowerOfTwo(num, power) {
  return num << power; // num * 2^power
}

function divideByPowerOfTwo(num, power) {
  return num >> power; // floor(num / 2^power)
}

```

```

}

console.log(multiplyByPowerOfTwo(5, 3)); // 40 (5 * 8)
console.log(divideByPowerOfTwo(40, 3)); // 5 (40 / 8)
console.log(divideByPowerOfTwo(41, 3)); // 5 (floor(41 / 8))

// Проверка четности (быстрее чем % 2)
function isEven(num) {
    return (num & 1) === 0;
}

function isOdd(num) {
    return (num & 1) === 1;
}

console.log(isEven(4)); // true
console.log(isEven(5)); // false
console.log(isOdd(4)); // false
console.log(isOdd(5)); // true

// Обмен значений без временной переменной (XOR swap)
function xorSwap(a, b) {
    console.log(`До: a = ${a}, b = ${b}`);
    a ^= b;
    b ^= a;
    a ^= b;
    console.log(`После: a = ${a}, b = ${b}`);
    return [a, b];
}

// Проверка
let x = 5, y = 10;
[x, y] = xorSwap(x, y); // x = 10, y = 5

// Абсолютное значение (быстрее Math.abs для целых)
function fastAbs(num) {
    const mask = num >> 31;
    return (num + mask) ^ mask;
}

console.log(fastAbs(5)); // 5
console.log(fastAbs(-5)); // 5
console.log(fastAbs(0)); // 0

```

```
// Минимум и максимум без ветвлений
function min(a, b) {
    return b ^ ((a ^ b) & -(a < b));
}

function max(a, b) {
    return a ^ ((a ^ b) & -(a < b));
}

console.log(min(5, 10)); // 5
console.log(min(10, 5)); // 5
console.log(max(5, 10)); // 10
console.log(max(10, 5)); // 10
```

3.4. Манипуляции с битами

javascript

```
// Установка бита в позиции n
function setBit(num, n) {
    return num | (1 << n);
}

// Сброс бита в позиции n
function clearBit(num, n) {
    return num & ~(1 << n);
}

// Переключение бита в позиции n
function toggleBit(num, n) {
    return num ^ (1 << n);
}

// Проверка бита в позиции n
function getBit(num, n) {
    return (num >> n) & 1;
}

// Изменение бита в позиции n на value (0 или 1)
function updateBit(num, n, value) {
    const mask = ~(1 << n);
    return (num & mask) | (value << n);
}
```

```

// Примеры
let num = 0b0000; // 0

num = setBit(num, 0); // 0b0001 = 1
num = setBit(num, 2); // 0b0101 = 5
console.log(num.toString(2).padStart(4, '0')); // 0101

num = clearBit(num, 0); // 0b0100 = 4
console.log(num.toString(2).padStart(4, '0')); // 0100

num = toggleBit(num, 0); // 0b0101 = 5
num = toggleBit(num, 2); // 0b0001 = 1
console.log(num.toString(2).padStart(4, '0')); // 0001

console.log(getBit(num, 0)); // 1
console.log(getBit(num, 1)); // 0

num = updateBit(num, 1, 1); // 0b0011 = 3
console.log(num.toString(2).padStart(4, '0')); // 0011

```

3.5. Подсчет битов и другие полезные операции

javascript

```

// Подсчет установленных битов (вес Хэмминга)
function countBits(num) {
    let count = 0;
    while (num) {
        count += num & 1; // Добавляем младший бит
        num >>= 1; // Сдвигаем вправо
    }
    return count;
}

// Оптимизированная версия (алгоритм Кернигана)
function countBitsKernighan(num) {
    let count = 0;
    while (num) {
        num &= num - 1; // Сбрасывает младший установленный бит
        count++;
    }
    return count;
}

```

```

// Проверка степени двойки
function isPowerOfTwo(num) {
    return num > 0 && (num & (num - 1)) === 0;
}

// Нахождение младшего установленного бита
function lowestSetBit(num) {
    return num & -num;
}

// Нахождение старшего установленного бита
function highestSetBit(num) {
    // Заполняем все биты после старшего
    num |= num >> 1;
    num |= num >> 2;
    num |= num >> 4;
    num |= num >> 8;
    num |= num >> 16;

    // Возвращаем только старший бит
    return (num >> 1) + 1;
}

// Циклический сдвиг
function rotateLeft(num, shift, bits = 32) {
    return ((num << shift) | (num >>> (bits - shift))) & ((1 << bits) - 1);
}

function rotateRight(num, shift, bits = 32) {
    return ((num >>> shift) | (num << (bits - shift))) & ((1 << bits) - 1);
}

// Примеры
console.log(countBits(0b10101)); // 3 (биты на позициях 0, 2, 4)
console.log(countBitsKernighan(0b10101)); // 3

console.log(isPowerOfTwo(8)); // true
console.log(isPowerOfTwo(9)); // false
console.log(isPowerOfTwo(0)); // false

console.log(lowestSetBit(0b10100).toString(2)); // 100 (бит на позиции 2)
console.log(highestSetBit(0b10100).toString(2)); // 10000 (бит на позиции 4)

console.log(rotateLeft(0b1001, 1, 4).toString(2).padStart(4, '0')); // 0011

```

```
console.log(rotateRight(0b1001, 1, 4).toString(2).padStart(4, '0')); // 1100
```

Часть 4: Сложные алгоритмы и оптимизации

4.1. Криптографические хэш-функции (упрощенные)

javascript

// Простая хэш-функция (не для production!)

```
function simpleHash(str) {  
  let hash = 0;  
  
  for (let i = 0; i < str.length; i++) {  
    const char = str.charCodeAt(i);  
    hash = ((hash << 5) - hash) + char; // hash * 31 + char  
    hash |= 0; // Преобразование в 32-битное целое  
  }  
  
  return hash >>> 0; // Беззнаковое 32-битное  
}
```

// MurmurHash3 (упрощенная версия)

```
function murmurHash3(key, seed = 0) {  
  let h1 = seed;  
  const c1 = 0xcc9e2d51;  
  const c2 = 0xb873593;  
  
  for (let i = 0; i < key.length; i++) {  
    let k1 = key.charCodeAt(i);  
  
    k1 = Math.imul(k1, c1);  
    k1 = (k1 << 15) | (k1 >>> 17);  
    k1 = Math.imul(k1, c2);  
  
    h1 ^= k1;  
    h1 = (h1 << 13) | (h1 >>> 19);  
    h1 = Math.imul(h1, 5) + 0xe6546b64;  
  }  
  
  h1 ^= key.length;
```

// Финальная обработка

```

h1 ^= h1 >>> 16;
h1 = Math.imul(h1, 0x85ebca6b);
h1 ^= h1 >>> 13;
h1 = Math.imul(h1, 0xc2b2ae35);
h1 ^= h1 >>> 16;

return h1 >>> 0;
}

// Примеры
console.log(simpleHash("hello").toString(16)); // 5d41402a
console.log(murmurHash3("hello").toString(16)); // 248bfa47

```

4.2. Алгоритмы сжатия данных (RLE)

```

javascript

// Run-Length Encoding с использованием битов
function rleEncode(data) {
  const result = [];
  let count = 1;
  let current = data[0];

  for (let i = 1; i <= data.length; i++) {
    if (i < data.length && data[i] === current && count < 255) {
      count++;
    } else {
      // Упаковываем: [count, value]
      result.push(count, current);
      count = 1;
      current = data[i];
    }
  }

  return result;
}

function rleDecode(encoded) {
  const result = [];

  for (let i = 0; i < encoded.length; i += 2) {
    const count = encoded[i];
    const value = encoded[i + 1];
  }
}

```

```

    for (let j = 0; j < count; j++) {
        result.push(value);
    }
}

return result;
}

// Пример
const data = [1, 1, 1, 2, 2, 3, 3, 3, 3, 3];
const encoded = rleEncode(data);
console.log("Закодировано:", encoded); // [3, 1, 2, 2, 5, 3]
console.log("Раскодировано:", rleDecode(encoded));

```

4.3. Генерация псевдослучайных чисел

```

javascript

// Линейный конгруэнтный генератор (LCG)
class LCG {
    constructor(seed = Date.now()) {
        this.state = seed & 0xFFFFFFFF;
        // Параметры из Numerical Recipes
        this.a = 1664525;
        this.c = 1013904223;
        this.m = 2 ** 32;
    }

    next() {
        this.state = (this.a * this.state + this.c) & 0xFFFFFFFF;
        return this.state / this.m;
    }

    nextInt(max = 2 ** 32) {
        return Math.floor(this.next() * max);
    }
}

// Xorshift генератор (быстрее)
class Xorshift32 {
    constructor(seed = 1) {
        this.state = seed;
    }
}

```



```

next() {
    let x = this.state;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    this.state = x;
    return (x >>> 0) / 0xFFFFFFFF;
}

nextInt(max = 0xFFFFFFFF) {
    return Math.floor(this.next() * max);
}
}

// Пример
const lcg = new LCG(12345);
console.log("LCG:", lcg.next(), lcg.next(), lcg.next());

const xorshift = new Xorshift32(12345);
console.log("Xorshift:", xorshift.next(), xorshift.next(), xorshift.next());

```

4.4. Оптимизация структур данных

```

javascript

// Битовый массив (BitSet)
class BitSet {
    constructor(size) {
        this.bits = new Uint32Array(Math.ceil(size / 32));
        this.size = size;
    }

    set(index, value = true) {
        if (index >= this.size) throw new Error("Index out of bounds");

        const wordIndex = index >>> 5; // index / 32
        const bitIndex = index & 31;   // index % 32
        const mask = 1 << bitIndex;

        if (value) {
            this.bits[wordIndex] |= mask;
        } else {
            this.bits[wordIndex] &= ~mask;
        }
    }
}

```

```
}
```

```
get(index) {  
    if (index >= this.size) throw new Error("Index out of bounds");  
  
    const wordIndex = index >>> 5;  
    const bitIndex = index & 31;  
    const mask = 1 << bitIndex;  
  
    return (this.bits[wordIndex] & mask) !== 0;  
}
```

```
toggle(index) {  
    if (index >= this.size) throw new Error("Index out of bounds");  
  
    const wordIndex = index >>> 5;  
    const bitIndex = index & 31;  
    const mask = 1 << bitIndex;  
  
    this.bits[wordIndex] ^= mask;  
}
```

// Операции над множествами

```
union(other) {  
    const result = new BitSet(Math.max(this.size, other.size));  
  
    const minLength = Math.min(this.bits.length, other.bits.length);  
    for (let i = 0; i < minLength; i++) {  
        result.bits[i] = this.bits[i] | other.bits[i];  
    }  
  
    return result;  
}
```

```
intersection(other) {  
    const result = new BitSet(Math.max(this.size, other.size));  
  
    const minLength = Math.min(this.bits.length, other.bits.length);  
    for (let i = 0; i < minLength; i++) {  
        result.bits[i] = this.bits[i] & other.bits[i];  
    }  
  
    return result;  
}
```

// Подсчет установленных битов

```
count() {
    let total = 0;
    for (let i = 0; i < this.bits.length; i++) {
        let word = this.bits[i];
        while (word) {
            word &= word - 1;
            total++;
        }
    }
    return total;
}

toString() {
    let result = "";
    for (let i = 0; i < this.size; i++) {
        result += this.get(i) ? "1" : "0";
        if ((i + 1) % 8 === 0 && i !== this.size - 1) {
            result += " ";
        }
    }
    return result;
}
}
```

// Пример

```
const bitSet = new BitSet(16);
bitSet.set(0, true);
bitSet.set(3, true);
bitSet.set(7, true);
bitSet.set(15, true);

console.log(bitSet.toString()); // 10010001 00000001
console.log("Бит 3:", bitSet.get(3)); // true
console.log("Бит 5:", bitSet.get(5)); // false
console.log("Всего установлено:", bitSet.count()); // 4
```

Часть 5: Производительность и оптимизация

5.1. Бенчмаркинг побитовых операций

javascript

```
function benchmarkBitOperations() {
  const iterations = 1000000;

  console.time('Modulo for even check');
  for (let i = 0; i < iterations; i++) {
    const isEven = i % 2 === 0;
  }
  console.timeEnd('Modulo for even check');

  console.time('Bitwise for even check');
  for (let i = 0; i < iterations; i++) {
    const isEven = (i & 1) === 0;
  }
  console.timeEnd('Bitwise for even check');

  console.time('Math.floor division');
  for (let i = 0; i < iterations; i++) {
    const result = Math.floor(i / 2);
  }
  console.timeEnd('Math.floor division');

  console.time('Right shift division');
  for (let i = 0; i < iterations; i++) {
    const result = i >> 1;
  }
  console.timeEnd('Right shift division');

  console.time('Multiplication');
  for (let i = 0; i < iterations; i++) {
    const result = i * 4;
  }
  console.timeEnd('Multiplication');

  console.time('Left shift multiplication');
  for (let i = 0; i < iterations; i++) {
    const result = i << 2;
  }
}
```

```
    console.timeEnd('Left shift multiplication');
}

// Результаты обычно показывают:
// 1. Побитовые операции быстрее арифметических аналогов
// 2. Разница особенно заметна в циклах
// 3. Но современные JavaScript движки хорошо оптимизируют оба подхода
```

5.2. Когда использовать побитовые операции

Ситуации, где побитовые операции полезны:

```
javascript

// 1. Работа с флагами и настройками
const Settings = {
  FLAG_A: 1 << 0,
  FLAG_B: 1 << 1,
  FLAG_C: 1 << 2,
  FLAG_D: 1 << 3
};

// 2. Оптимизация критического кода
function optimizedProcessing(data) {
  // Вместо дорогих операций
  const half = data.length >> 1;
  const quarter = data.length >> 2;

  for (let i = 0; i < data.length; i++) {
    if ((i & 1) === 0) { // Проверка четности
      // Обработка четных элементов
    }
  }
}

// 3. Низкоуровневые манипуляции
function manipulateColor(color) {
  // Извлечение компонентов цвета
  const r = (color >> 16) & 0xFF;
  const g = (color >> 8) & 0xFF;
  const b = color & 0xFF;

  // Изменение яркости
```

```

const brightness = 0.5;
const newR = (r * brightness) & 0xFF;
const newG = (g * brightness) & 0xFF;
const newB = (b * brightness) & 0xFF;

return (newR << 16) | (newG << 8) | newB;
}

// 4. Криптография и хэширование
function simpleChecksum(data) {
  let checksum = 0;
  for (let i = 0; i < data.length; i++) {
    checksum ^= data.charCodeAt(i);
    checksum = (checksum << 1) | (checksum >>> 31);
  }
  return checksum;
}

// 5. Сетевое программирование
function parseIPv4(ip) {
  const parts = ip.split('.').map(Number);
  return (parts[0] << 24) | (parts[1] << 16) | (parts[2] << 8) | parts[3];
}

function formatIPv4(num) {
  return [
    (num >> 24) & 0xFF,
    (num >> 16) & 0xFF,
    (num >> 8) & 0xFF,
    num & 0xFF
  ].join('.');
}

```

Ситуации, где следует избегать побитовых операций:

javascript

```

// 1. Когда важна читаемость кода
// ПЛОХО: неочевидная оптимизация
const result = (x & (x - 1)) === 0 ? "степень двойки" : "не степень двойки";

// ХОРОШО: понятный код
function isPowerOfTwoReadable(x) {
  if (x <= 0) return false;
  while (x > 1) {

```

```

    if (x % 2 !== 0) return false;
    x /= 2;
  }
  return true;
}

// 2. При работе с дробными числами
// ПЛОХО: потеря точности
const truncated = 3.14 & 0xFFFFFFFF; // 3 (дробная часть потеряна)

// ХОРОШО: явное преобразование
const rounded = Math.floor(3.14); // 3

// 3. При работе с большими числами (> 2^31)
// ПЛОХО: обрезание битов
const bigNum = 4294967296; // 2^32
console.log(bigNum & 0xFFFFFFFF); // 0 (все биты обрезаны)

// ХОРОШО: использование BigInt
const bigIntNum = 4294967296n;
console.log(bigIntNum & 0xFFFFFFFFn); // 0n (корректно)

```

Часть 6: Распространенные ошибки

6.1. Ошибка: Путаница между >> и >>>

```

javascript

// НЕПРАВИЛЬНО: использование >> для беззнаковых чисел
function getColorComponentWrong(color) {
  // Для отрицательных цветов?!
  const r = color >> 16; // Сохраняет знак!
  return r & 0xFF;
}

console.log(getColorComponentWrong(0xFF0000)); // 255
console.log(getColorComponentWrong(0x800000)); // -128 (ошибка!)

// ПРАВИЛЬНО: использование >>> для беззнаковых чисел
function getColorComponentCorrect(color) {
  const r = color >>> 16; // Всегда беззнаковое
  return r & 0xFF;
}

```

```

}

console.log(getColorComponentCorrect(0xFF0000)); // 255
console.log(getColorComponentCorrect(0x800000)); // 128 (правильно)

// ПРАВИЛЬНО: явное преобразование
function getColorComponentSafe(color) {
    // Преобразуем к беззнаковому
    const unsignedColor = color >>> 0;
    return (unsignedColor >> 16) & 0xFF;
}

```

6.2. Ошибка: Потеря битов при сдвиге

```

javascript

// НЕПРАВИЛЬНО: сдвиг больше 31
console.log(1 << 32); // 1, а не 0!
console.log(1 << 33); // 2, а не 0!

// ПРАВИЛЬНО: ограничение сдвига
function safeLeftShift(num, shift) {
    shift = shift & 31; // Ограничиваем 0-31
    return num << shift;
}

function safeRightShift(num, shift) {
    shift = shift & 31;
    return num >> shift;
}

console.log(safeLeftShift(1, 32)); // 1 (0 после маскирования)
console.log(safeLeftShift(1, 33)); // 2 (1 после маскирования)

```

6.3. Ошибка: Неучет знака при операциях

```

javascript

// НЕПРАВИЛЬНО: работа с отрицательными числами как с положительными
function countOnesWrong(num) {
    let count = 0;
    while (num) {
        count += num & 1;
        num >>= 1; // Для отрицательных - бесконечный цикл!
    }
}

```



```

    }
    return count;
}

// Может зависнуть для отрицательных чисел
// console.log(countOnesWrong(-1)); // Бесконечный цикл!

// ПРАВИЛЬНО: использование >>> для беззнакового сдвига
function countOnesCorrect(num) {
    let count = 0;
    let n = num >>> 0; // Преобразуем к беззнаковому
    while (n) {
        count += n & 1;
        n >>>= 1; // Беззнаковый сдвиг
    }
    return count;
}

console.log(countOnesCorrect(-1)); // 32 (все биты установлены)
console.log(countOnesCorrect(5)); // 2 (101)

```

Заключение: Практические рекомендации

Основные правила:

1. **Используйте побитовые операции для работы с флагами и битовыми масками** — это идиоматично и эффективно.

```

javascript

// Хорошо
const FLAGS = {
    READ: 1 << 0,
    WRITE: 1 << 1,
    EXECUTE: 1 << 2
};

let permissions = FLAGS.READ | FLAGS.WRITE;
if (permissions & FLAGS.READ) {
    // Есть право на чтение
}

```

2. **Используйте `>>> 0` для преобразования в беззнаковое 32-битное число** — это безопасный способ работать с битами.

javascript

```
const unsigned = -10 >>> 0; // 4294967286
```

```
const component = (color >>> 16) & 0xFF; // Безопасное извлечение
```

3. **Будьте осторожны со сдвигами больше 31** — используйте маскирование:

javascript

```
function safeShift(num, shift) {  
  return num << (shift & 31);  
}
```

4. **Помните о преобразовании типов** — побитовые операции работают с 32-битными целыми:

javascript

```
console.log(3.14 & 0xFF); // 3 (дробная часть отбрасывается)
```

```
console.log("5" & 0xFF); // 5 (строка преобразуется)
```

```
console.log(null & 0xFF); // 0
```

```
console.log(undefined & 0xFF); // 0
```

Чек-лист для проверки кода:

- Побитовые операции используются только для целочисленных значений
- Для беззнаковых операций используется `>>>`
- Сдвиги ограничены 31 битом или маскируются
- Флаги определены с использованием `<<`
- Нет смешивания знаковых и беззнаковых операций
- Большие числа ($> 2^{31}$) обрабатываются с использованием `BigInt`
- Дробные числа явно преобразуются перед операциями

Итоговый пример хорошего кода:

javascript

```
// Библиотека для работы с цветами
```

```
const ColorUtils = {
```

```
  // Константы
```

```
  ALPHA_MASK: 0xFF000000,
```

```
  RED_MASK: 0x00FF0000,
```

```
  GREEN_MASK: 0x0000FF00,
```

```
  BLUE_MASK: 0x000000FF,
```

```
  // Упаковка компонентов
```

```
  pack(r, g, b, a = 255) {  
    return ((a & 0xFF) << 24) |  
           ((r & 0xFF) << 16) |
```

```

        ((g & 0xFF) << 8) |
        (b & 0xFF);
    },

    // Извлечение компонентов
    unpack(color) {
        const a = (color >>> 24) & 0xFF;
        const r = (color >>> 16) & 0xFF;
        const g = (color >>> 8) & 0xFF;
        const b = color & 0xFF;

        return { r, g, b, a };
    },

    // Изменение яркости
    brightness(color, factor) {
        const { r, g, b, a } = this.unpack(color);

        const newR = (r * factor) & 0xFF;
        const newG = (g * factor) & 0xFF;
        const newB = (b * factor) & 0xFF;

        return this.pack(newR, newG, newB, a);
    },

    // Смешивание цветов
    blend(color1, color2, ratio) {
        const c1 = this.unpack(color1);
        const c2 = this.unpack(color2);

        const r = (c1.r * (1 - ratio) + c2.r * ratio) & 0xFF;
        const g = (c1.g * (1 - ratio) + c2.g * ratio) & 0xFF;
        const b = (c1.b * (1 - ratio) + c2.b * ratio) & 0xFF;
        const a = (c1.a * (1 - ratio) + c2.a * ratio) & 0xFF;

        return this.pack(r, g, b, a);
    },

    // Проверка на прозрачность
    isTransparent(color) {
        return (color >>> 24) === 0;
    }
};

```

// Пример использования

```
const red = ColorUtils.pack(255, 0, 0);  
const blue = ColorUtils.pack(0, 0, 255);  
const purple = ColorUtils.blend(red, blue, 0.5);
```

```
console.log(ColorUtils.unpack(purple));  
console.log(ColorUtils.isTransparent(purple));
```

Запомните: Побитовые операции в JavaScript — это мощный, но низкоуровневый инструмент. Используйте их там, где они действительно нужны: для оптимизации критического кода, работы с битовыми масками, низкоуровневых манипуляций. Но всегда отдавайте предпочтение читаемости кода преждевременной оптимизации. Современные JavaScript движки хорошо оптимизируют код, и часто явные, понятные операции работают достаточно быстро.

5.5. Условный (тернарный) оператор

Введение: Элегантность условных выражений

Тернарный оператор — это единственный оператор в JavaScript, который принимает три операнда. Он представляет собой компактную альтернативу оператору `if-else` для простых условных выражений. Его сила заключается в возможности встраивать условную логику прямо в выражения, что делает код более лаконичным и декларативным.

Часть 1: Основной синтаксис и базовое использование

1.1. Синтаксическая структура

```
javascript

// Базовый синтаксис:
// условие ? выражение_если_истина : выражение_если_ложь

// Простой пример
const age = 20;
const status = age >= 18 ? 'взрослый' : 'ребенок';
console.log(status); // 'взрослый'

// Эквивалент с if-else
let status2;
if (age >= 18) {
  status2 = 'взрослый';
} else {
  status2 = 'ребенок';
}

// Разбор компонентов:
// 1. Условие (condition) - вычисляется в boolean
// 2. Выражение после ? (truthy expression) - выполняется если условие true
// 3. Выражение после : (falsy expression) - выполняется если условие false

// Все три компонента обязательны
```

```
// const result = condition ? truthy; // SyntaxError: missing :
```

1.2. Возвращаемое значение

javascript

```
// Ключевая особенность: тернарный оператор возвращает значение
```

```
const x = 5;  
const y = 10;
```

```
// Возвращает результат выбранного выражения
```

```
const max = x > y ? x : y;  
console.log(max); // 10
```

```
// Можно использовать в любом месте, где ожидается выражение
```

```
console.log(`Максимум: ${x > y ? x : y}`); // "Максимум: 10"
```

```
// В присваивании
```

```
const config = {  
  timeout: process.env.NODE_ENV === 'production' ? 30000 : 5000  
};
```

```
// В возврате из функции
```

```
function getGreeting(hour) {  
  return hour < 12 ? 'Доброе утро' :  
    hour < 18 ? 'Добрый день' : 'Добрый вечер';  
}
```

```
// В аргументах функции
```

```
function createElement(type, isActive) {  
  return document.createElement(type, {  
    className: isActive ? 'active' : 'inactive'  
  });  
}
```

1.3. Сравнение с if-else

javascript

```
// Пример 1: Присваивание переменной
```

```
// if-else
```

```
let discount1;  
if (isMember) {  
  discount1 = 0.2;
```

```
} else {  
    discount1 = 0;  
}
```

// Тернарный оператор

```
const discount2 = isMember ? 0.2 : 0;
```

// Пример 2: Возврат значения из функции

// if-else

```
function getPrice1(quantity) {  
    if (quantity > 100) {  
        return quantity * 0.9;  
    } else {  
        return quantity * 0.95;  
    }  
}
```

// Тернарный оператор

```
function getPrice2(quantity) {  
    return quantity > 100 ? quantity * 0.9 : quantity * 0.95;  
}
```

// Пример 3: Сложные условия

// if-else

```
function getShippingCost1(country, weight) {  
    let cost;  
    if (country === 'RU') {  
        if (weight > 10) {  
            cost = 1000;  
        } else {  
            cost = 500;  
        }  
    } else {  
        if (weight > 10) {  
            cost = 2000;  
        } else {  
            cost = 1000;  
        }  
    }  
    return cost;  
}
```

// Тернарный оператор (менее читаемо!)

```
function getShippingCost2(country, weight) {
```

```
return country === 'RU'
  ? weight > 10 ? 1000 : 500
  : weight > 10 ? 2000 : 1000;
}
```

// Вывод: для сложных условий if-else может быть читаемее

Часть 2: Продвинутое использование

2.1. Вложенные тернарные операторы

javascript

// Вложенные тернарные операторы позволяют обрабатывать несколько условий

// Но могут стать сложными для чтения

// Пример: оценка студента

```
function getGrade(score) {
  return score >= 90 ? 'A' :
    score >= 80 ? 'B' :
    score >= 70 ? 'C' :
    score >= 60 ? 'D' : 'F';
}
```

// Альтернатива с if-else if

```
function getGradeIf(score) {
  if (score >= 90) return 'A';
  if (score >= 80) return 'B';
  if (score >= 70) return 'C';
  if (score >= 60) return 'D';
  return 'F';
}
```

// Пример 2: Определение времени суток

```
function getTimeOfDay(hour) {
  return hour < 6 ? 'ночь' :
    hour < 12 ? 'утро' :
    hour < 18 ? 'день' : 'вечер';
}
```

// Пример 3: Статус заказа

```
function getOrderStatus(statusCode) {
```



```

return statusCode === 1 ? 'создан' :
  statusCode === 2 ? 'оплачен' :
  statusCode === 3 ? 'отправлен' :
  statusCode === 4 ? 'доставлен' :
  statusCode === 5 ? 'отменен' : 'неизвестный статус';
}

// Проблема: глубоко вложенные тернарные операторы сложно читать
function complexExample(a, b, c) {
  return a ? b ? c ? 'a, b, c true' : 'a, b true, c false'
    : 'a true, b false'
    : 'a false';
}

// Становится запутанным очень быстро!

// Рекомендация: используйте вложенные тернарные операторы
// только для простых цепочек условий (2-3 уровня)

```

2.2. Тернарный оператор с функциями

```

javascript

// Тернарный оператор может вызывать разные функции
function logSuccess(message) {
  console.log(`✅ ${message}`);
}

function logError(message) {
  console.log(`❌ ${message}`);
}

const isSuccess = Math.random() > 0.5;
isSuccess ? logSuccess('Операция выполнена') : logError('Ошибка операции');

// Можно возвращать функции
function getLogger(level) {
  return level === 'debug' ? console.debug :
    level === 'warn' ? console.warn :
    level === 'error' ? console.error : console.log;
}

const logger = getLogger('debug');
logger('Отладочное сообщение');

```

```

// Функции с побочными эффектами
function processData(data, shouldValidate) {
  // Выполняется только выбранная функция
  const result = shouldValidate ? validateAndProcess(data) : process(data);
  return result;
}

// Асинхронные функции
async function fetchData(useCache) {
  return useCache ? getCachedData() : fetchFreshData();
}

```

2.3. Тернарный оператор и короткий цикл вычислений

javascript

```

// Важное отличие от if-else: оба выражения вычисляются?
// НЕТ! Вычисляется только выбранное выражение

function expensiveOperation() {
  console.log('Выполняется дорогая операция...');
  return 'результат';
}

function cheapOperation() {
  console.log('Выполняется дешевая операция...');
  return 'запасной вариант';
}

const useExpensive = false;

// Выполнится только cheapOperation
const result = useExpensive ? expensiveOperation() : cheapOperation();
// Выведет: "Выполняется дешевая операция..."

// Это полезно для оптимизации
function getConfigValue(key, defaultValue) {
  // config.get() может быть дорогой операцией
  return config.has(key) ? config.get(key) : defaultValue;
}

// Осторожно: если выражение содержит побочные эффекты
let counter = 0;
const x = true ? counter++ : counter--;

```

```
console.log(counter); // 1 (инкремент выполнен)
console.log(x);        // 0 (возвращает старое значение)

// Проверка на null/undefined с оператором ??
const user = null;
const displayName = user ? user.name : 'Гость';
// Эквивалентно: const displayName = user?.name ?? 'Гость';
```

2.4. Тернарный оператор в JSX (React)

javascript

// В React JSX тернарный оператор часто используется для условного рендеринга

// Пример 1: Условное отображение элемента

```
function WelcomeMessage({ user }) {
  return (
    <div>
      {user ? (
        <h1>Добро пожаловать, {user.name}!</h1>
      ) : (
        <h1>Пожалуйста, войдите в систему</h1>
      )}
    </div>
  );
}
```

// Пример 2: Условный класс CSS

```
function Button({ isPrimary, children }) {
  return (
    <button className={isPrimary ? 'btn-primary' : 'btn-secondary'}>
      {children}
    </button>
  );
}
```

// Пример 3: Условный рендеринг компонента

```
function UserProfile({ user }) {
  return (
    <div>
      {user.isAdmin ? (
        <AdminPanel user={user} />
      ) : user.isModerator ? (
        <ModeratorPanel user={user} />
      ) : null}
    </div>
  );
}
```

```

    ) : (
      <UserPanel user={user} />
    )}
  </div>
);
}

```

// Пример 4: Условные атрибуты

```

function Image({ src, isLazy }) {
  return (
    <img
      src={src}
      loading={isLazy ? 'lazy' : 'eager'}
      alt={src ? 'Изображение' : 'Изображение отсутствует'}
    />
  );
}

```

// Альтернатива: логический оператор && для простых условий

```

function Notification({ message, show }) {
  return (
    <div>
      {show && <div className="notification">{message}</div>}
    </div>
  );
}

```

Часть 3: Практические паттерны и идиомы

3.1. Паттерн: Значения по умолчанию

javascript

// Паттерн 1: Простое значение по умолчанию

```

function getUsername(user) {
  return user ? user.name : 'Гость';
}

```

// С оператором объединения с null (??)

```

function getUsernameModern(user) {
  return user?.name ?? 'Гость';
}

```

// Паттерн 2: Значение по умолчанию для конфигурации

```
function createConfig(userConfig = {}) {
  return {
    apiUrl: userConfig.apiUrl ? userConfig.apiUrl : 'https://api.example.com',
    timeout: userConfig.timeout ? userConfig.timeout : 30000,
    retries: userConfig.retries ? userConfig.retries : 3,
    // Более идиоматично с ??
    debug: userConfig.debug ?? false
  };
}
```

// Паттерн 3: Цепочка значений по умолчанию

```
function getApiEndpoint() {
  return process.env.API_URL ||
    localStorage.getItem('api_url') ||
    'https://default-api.example.com';
}
```

// С тернарным оператором это было бы:

```
function getApiEndpointTernary() {
  return process.env.API_URL
    ? process.env.API_URL
    : localStorage.getItem('api_url')
      ? localStorage.getItem('api_url')
      : 'https://default-api.example.com';
  // Сложно читать!
}
```

// Паттерн 4: Значение по умолчанию с преобразованием

```
function parseInteger(value, defaultValue = 0) {
  const parsed = parseInt(value, 10);
  return isNaN(parsed) ? defaultValue : parsed;
}

function ensureArray(value) {
  return Array.isArray(value) ? value : [];
}
```

3.2. Паттерн: Условное присваивание и мутация

javascript

// Паттерн 1: Условное добавление свойств в объект

```

function createUser(data, includeMetadata = false) {
  return {
    id: data.id,
    name: data.name,
    email: data.email,
    // Условно добавляем свойства
    ...(includeMetadata && {
      createdAt: new Date(),
      updatedAt: new Date()
    }),
    // Альтернатива с тернарным оператором
    ...(data.role ? { role: data.role } : {})
  };
}

// Паттерн 2: Условное добавление элементов в массив
function.getItems(includeOptional = false) {
  const required = ['item1', 'item2', 'item3'];
  const optional = ['optional1', 'optional2'];

  return includeOptional ? [...required, ...optional] : required;
}

// Паттерн 3: Условная мутация объекта
function updateUser(user, updates) {
  // Безопасное обновление
  return {
    ...user,
    name: updates.name ? updates.name : user.name,
    email: updates.email ? updates.email : user.email,
    // С оператором ??
    age: updates.age ?? user.age
  };
}

// Паттерн 4: Условное выполнение методов
class Validator {
  validate(data, options = {}) {
    const result = { isValid: true, errors: [] };

    // Условно применяем валидаторы
    options.validateEmail && this.validateEmail(data.email, result);
    options.validatePhone && this.validatePhone(data.phone, result);
    options.validateAge && this.validateAge(data.age, result);
  }
}

```

```

    return result;
}

validateEmail(email, result) {
    if (!email.includes('@')) {
        result.isValid = false;
        result.errors.push('Неверный email');
    }
}
}
}

```

3.3. Паттерн: Функциональное программирование

javascript

// Паттерн 1: Тернарный оператор в цепочках методов

```

function processUsers(users) {
    return users
        .filter(user => user.isActive)
        .map(user => ({
            ...user,
            // Условное преобразование
            displayName: user.nickname ? user.nickname : `${user.firstName} ${user.lastName}`,
            // Условная категория
            category: user.age >= 65 ? 'senior' :
                user.age >= 18 ? 'adult' : 'minor'
        })))
        .sort((a, b) => a.displayName.localeCompare(b.displayName));
}

```

// Паттерн 2: Композиция функций с условиями

```

const pipe = (...fns) => x => fns.reduce((v, f) => f(v), x);

function createPipeline(shouldLog = false) {
    return pipe(
        // Условное добавление функции в цепочку
        ...(shouldLog ? [logInput] : []),
        processData,
        validateResult,
        ...(shouldLog ? [logOutput] : []),
        formatOutput
    );
}

```

// Паттерн 3: Мемоизация с условием

```
function memoize(fn, shouldCache = true) {  
  const cache = new Map();  
  
  return function(...args) {  
    if (!shouldCache) {  
      return fn(...args);  
    }  
  
    const key = JSON.stringify(args);  
    return cache.has(key)  
      ? cache.get(key)  
      : (cache.set(key, fn(...args)), cache.get(key));  
  };  
}
```

// Паттерн 4: Каррирование с условиями

```
function createGreeting(language) {  
  return function(formal) {  
    return formal  
      ? language === 'ru' ? 'Здравствуйте' : 'Hello'  
      : language === 'ru' ? 'Привет' : 'Hi';  
  };  
}  
  
const getRussianGreeting = createGreeting('ru');  
console.log(getRussianGreeting(true)); // 'Здравствуйте'  
console.log(getRussianGreeting(false)); // 'Привет'
```

3.4. Паттерн: Обработка ошибок и граничных случаев

javascript

// Паттерн 1: Безопасный доступ с запасным значением

```
function getNestedValue(obj, path, defaultValue = null) {  
  const keys = path.split('.');  
  let result = obj;  
  
  for (const key of keys) {  
    result = result ? result[key] : undefined;  
    if (result === undefined) break;  
  }  
}
```



```
    return result !== undefined ? result : defaultValue;
}
```

// Паттерн 2: Обработка ошибок в try-catch

```
function safeParseJSON(jsonString, fallback = {}) {
  try {
    return JSON.parse(jsonString);
  } catch {
    return fallback;
  }
}
```

// Более функциональный подход

```
function tryCatch(tryFn, catchFn) {
  try {
    return tryFn();
  } catch (error) {
    return catchFn(error);
  }
}
```

```
const result = tryCatch(
  () => JSON.parse(invalidJson),
  (error) => ({ error: error.message, data: null })
);
```

// Паттерн 3: Валидация входных данных

```
function validateInput(input, type) {
  const value = type === 'number' ? Number(input) : input;

  return type === 'number'
    ? isNaN(value) ? 0 : value
    : typeof value === 'string' && value.trim() !== ''
      ? value.trim()
      : type === 'string' ? '' : null;
}
```

// Паттерн 4: Обработка API ответов

```
async function fetchWithFallback(url, fallbackData) {
  try {
    const response = await fetch(url);
    return response.ok ? await response.json() : fallbackData;
  } catch {
    return fallbackData;
  }
}
```

```
}  
}
```

Часть 4: Оптимизация и производительность

4.1. Бенчмаркинг: тернарный оператор vs if-else

javascript

```
function benchmarkTernaryVsIfElse() {  
  const iterations = 10000000;  
  
  console.time('Тернарный оператор');  
  let sum1 = 0;  
  for (let i = 0; i < iterations; i++) {  
    sum1 += i % 2 === 0 ? i : -i;  
  }  
  console.timeEnd('Тернарный оператор');  
  
  console.time('if-else');  
  let sum2 = 0;  
  for (let i = 0; i < iterations; i++) {  
    if (i % 2 === 0) {  
      sum2 += i;  
    } else {  
      sum2 += -i;  
    }  
  }  
  console.timeEnd('if-else');  
  
  console.time('switch');  
  let sum3 = 0;  
  for (let i = 0; i < iterations; i++) {  
    switch (i % 2 === 0) {  
      case true: sum3 += i; break;  
      case false: sum3 += -i; break;  
    }  
  }  
  console.timeEnd('switch');  
  
  console.log('Суммы равны?', sum1 === sum2 && sum2 === sum3);  
}
```

```
// Результаты обычно показывают:  
// 1. Тернарный оператор часто быстрее if-else  
// 2. Разница незначительна для большинства приложений  
// 3. Switch обычно медленнее для простых условий  
// 4. Важнее читаемость, чем микро-оптимизации
```

4.2. Влияние на производительность

javascript

```
// Критические факторы производительности:
```

```
// 1. Вычисление условий
```

```
const result1 = expensiveCondition() ? expensiveTrue() : expensiveFalse();
```

```
// Проблема: expensiveCondition() вычисляется всегда
```

```
// Решение: кэшировать результат условия
```

```
const condition = expensiveCondition();
```

```
const result2 = condition ? expensiveTrue() : expensiveFalse();
```

```
// 2. Вложенные тернарные операторы
```

```
function nestedTernary(level) {  
  return level === 1 ? 'one' :  
    level === 2 ? 'two' :  
    level === 3 ? 'three' :  
    level === 4 ? 'four' :  
    level === 5 ? 'five' : 'many';  
}
```

```
// Каждое условие проверяется последовательно
```

```
// Для level = 5 будет проверено 5 условий
```

```
// Альтернатива: объект-словарь (быстрее для многих условий)
```

```
function lookupTable(level) {  
  const levels = {  
    1: 'one',  
    2: 'two',  
    3: 'three',  
    4: 'four',  
    5: 'five'  
  };  
  return levels[level] || 'many';  
}
```

```

// 3. Использование в горячих циклах
function processArray(array) {
  const result = [];

  // ПЛОХО: тернарный оператор в каждой итерации
  for (let i = 0; i < array.length; i++) {
    result.push(array[i] > 0 ? array[i] * 2 : array[i]);
  }

  // ЛУЧШЕ: вынести условие из цикла если возможно
  const shouldDouble = array.some(item => item > 0);
  if (shouldDouble) {
    for (let i = 0; i < array.length; i++) {
      result.push(array[i] > 0 ? array[i] * 2 : array[i]);
    }
  } else {
    result.push(...array);
  }

  return result;
}

```

4.3. Оптимизация для движка JavaScript

javascript

// Советы для оптимизации:

// 1. Используйте простые условия

// ХОРОШО

```
const isValid = value !== null && value !== undefined ? true : false;
```

// ЛУЧШЕ (движок может оптимизировать)

```
const isValid2 = value !== null; // Проверяет и null, и undefined
```

// 2. Избегайте сложных выражений в условиях

// ПЛОХО

```
const result = a && b || c ? x : y;
```

// Движок должен вычислять все выражение

// ЛУЧШЕ

```
const condition = a && b || c;
```

```
const result2 = condition ? x : y;
```

```

// 3. Используйте примитивные сравнения
// ХОРОШО
const isTrue = boolValue === true ? 'yes' : 'no';

// ЛУЧШЕ (если boolValue гарантированно boolean)
const isTrue2 = boolValue ? 'yes' : 'no';

// 4. Кэшируйте результаты функций
function processWithCache(data, useCache) {
  // ПЛОХО: функция вызывается всегда
  const processed = useCache ? getCached(data) : process(data);

  // ЛУЧШЕ: вызывается только нужная функция
  let result;
  if (useCache) {
    result = getCached(data);
  } else {
    result = process(data);
  }
  return result;
}

```

Часть 5: Читаемость и поддерживаемость

5.1. Правила форматирования

```

javascript

// Правило 1: Простые тернарные операторы в одну строку
const status = isActive ? 'active' : 'inactive';

// Правило 2: Сложные выражения на несколько строк
const message = user.hasPermission
  ? `Добро пожаловать, ${user.name}! Ваш уровень доступа: ${user.role}`
  : 'Доступ запрещен. Обратитесь к администратору';

// Правило 3: Вложенные тернарные операторы с отступами
const category = age < 13 ? 'child' :
  age < 18 ? 'teen' :
  age < 65 ? 'adult' : 'senior';

// Правило 4: Выравнивание по ? и :

```

```

const result = condition1 ? value1 :
    condition2 ? value2 :
    condition3 ? value3 : defaultValue;

// Правило 5: Группировка сложных условий
const accessLevel = (user.role === 'admin' || user.role === 'moderator')
    ? 'full'
    : 'limited';

// ПЛОХО: смешанное форматирование
const badFormat = condition1?value1:condition2?
    value2:value3;

// ХОРОШО: последовательное форматирование
const goodFormat = condition1 ? value1 :
    condition2 ? value2 : value3;

```

5.2. Когда использовать, а когда избегать

Использовать тернарный оператор когда:

```

javascript

// 1. Простые присваивания
const discount = isMember ? 0.2 : 0;

// 2. Простые возвраты из функций
function getPrice(quantity) {
    return quantity > 100 ? quantity * 0.9 : quantity * 0.95;
}

// 3. Условные выражения в JSX
function Component({ isVisible }) {
    return (
        <div>
            {isVisible ? <Content /> : null}
        </div>
    );
}

// 4. Значения по умолчанию
const name = userName ? userName : 'Гость';

```

// 5. Простые цепочки условий (2-3 уровня)

```
function getSizeCategory(weight) {  
  return weight < 1 ? 'small' :  
    weight < 5 ? 'medium' :  
    weight < 10 ? 'large' : 'x-large';  
}
```

Избегать тернарный оператор когда:

javascript

// 1. Слишком сложные условия

// ПЛОХО

```
const result = a && (b || c) && !d ? processA() :  
  e || f ? processB() :  
  g && h ? processC() : defaultProcess();
```

// ЛУЧШЕ: использовать if-else или switch

```
let result;  
if (a && (b || c) && !d) {  
  result = processA();  
} else if (e || f) {  
  result = processB();  
} else if (g && h) {  
  result = processC();  
} else {  
  result = defaultProcess();  
}
```

// 2. Когда нужны побочные эффекты в условиях

// ПЛОХО

```
const value = (logEvent(), condition) ? a : b;  
// Неочевидно, что logEvent() всегда выполняется
```

// ЛУЧШЕ: явный код

```
logEvent();  
const value = condition ? a : b;
```

// 3. Когда оба выражения имеют побочные эффекты

// ПЛОХО

```
const data = shouldUpdate ? updateAndGet() : resetAndGet();  
// Неясно, какая функция вызывается
```

// ЛУЧШЕ: явный код

```
let data;
```

```

if (shouldUpdate) {
  data = updateAndGet();
} else {
  data = resetAndGet();
}

// 4. Когда условия требуют комментариев
// ПЛОХО
const mode = isProd && !isDebug ? 'production' :
  isTest ? 'testing' : 'development';
// Что такое isProd? Что такое isTest?

// ЛУЧШЕ: явный код с комментариями
let mode;
if (isProd && !isDebug) {
  // Продакшн режим без отладки
  mode = 'production';
} else if (isTest) {
  // Режим тестирования
  mode = 'testing';
} else {
  // Режим разработки по умолчанию
  mode = 'development';
}

```

5.3. Альтернативы тернарному оператору

```

javascript

// Альтернатива 1: Логические операторы для простых случаев
const name = userName || 'Гость'; // Если userName falsy
const displayName = user && user.name; // Если user существует

// Альтернатива 2: Оператор объединения с null (??)
const timeout = config.timeout ?? 30000; // Только для null/undefined

// Альтернатива 3: Опциональная цепочка (?.)
const street = user?.address?.street; // Безопасный доступ

// Альтернатива 4: Switch для множества условий
function getDayName(dayNumber) {
  switch (dayNumber) {
    case 0: return 'Sunday';
    case 1: return 'Monday';
  }
}

```



```

    case 2: return 'Tuesday';
    case 3: return 'Wednesday';
    case 4: return 'Thursday';
    case 5: return 'Friday';
    case 6: return 'Saturday';
    default: return 'Invalid day';
  }
}

// Альтернатива 5: Объект-словарь (lookup table)
const DAY_NAMES = {
  0: 'Sunday',
  1: 'Monday',
  2: 'Tuesday',
  3: 'Wednesday',
  4: 'Thursday',
  5: 'Friday',
  6: 'Saturday'
};

function getDayNameLookup(dayNumber) {
  return DAY_NAMES[dayNumber] || 'Invalid day';
}

// Альтернатива 6: Map
const STATUS_MAP = new Map([
  [0, 'pending'],
  [1, 'processing'],
  [2, 'completed'],
  [3, 'failed']
]);

function getStatusName(statusCode) {
  return STATUS_MAP.get(statusCode) || 'unknown';
}

// Альтернатива 7: Функция с ранним возвратом
function calculatePrice(quantity, isMember) {
  if (quantity <= 0) return 0;
  if (quantity > 100) return quantity * 0.8;
  if (quantity > 50) return quantity * 0.9;

  const basePrice = quantity * 0.95;
  return isMember ? basePrice * 0.9 : basePrice;
}

```

```
}
```

Часть 6: Распространенные ошибки и антипаттерны

6.1. Ошибка: Неправильное использование для побочных эффектов

```
javascript

// НЕПРАВИЛЬНО: тернарный оператор для выполнения действий
let count = 0;
condition ? count++ : count--; // Работает, но нечитаемо

// ПРАВИЛЬНО: if-else для побочных эффектов
if (condition) {
  count++;
} else {
  count--;
}

// НЕПРАВИЛЬНО: смешивание действий и возврата значений
function process(data) {
  return data ? (console.log('Обработка'), processData(data)) : null;
  // Запятая оператор работает, но запутанно
}

// ПРАВИЛЬНО: разделение логики
function processBetter(data) {
  if (!data) return null;

  console.log('Обработка');
  return processData(data);
}
```

6.2. Ошибка: Избыточные условия

```
javascript

// НЕПРАВИЛЬНО: избыточное сравнение с true/false
const result = condition === true ? a : b;

// ПРАВИЛЬНО: прямое использование boolean значения
```

```

const result = condition ? a : b;

// НЕПРАВИЛЬНО: дублирование одинаковых выражений
const value = condition ? calculate(x) : calculate(x);

// ПРАВИЛЬНО: убрать условие
const value = calculate(x);

// НЕПРАВИЛЬНО: ненужная вложенность
const category = age >= 18 ? (isStudent ? 'student' : 'adult') : 'child';

// ПРАВИЛЬНО: упрощение
let category;
if (age >= 18) {
  category = isStudent ? 'student' : 'adult';
} else {
  category = 'child';
}

```

6.3. Ошибка: Неявное преобразование типов

```

javascript

// НЕПРАВИЛЬНО: неявное преобразование может дать неожиданные результаты
const value = input ? input : 'default';
// Проблема: если input = 0, value = 'default' (может быть нежелательно)

// ПРАВИЛЬНО: явная проверка
const value = input !== undefined && input !== null ? input : 'default';

// Или с оператором ??
const value = input ?? 'default'; // Только для null/undefined

// НЕПРАВИЛЬНО: сравнение с использованием ==
const result = value == null ? 'отсутствует' : 'присутствует';
// Проблема: value == null вернет true для null И undefined

// ПРАВИЛЬНО: явное различие если нужно
let status;
if (value === null) {
  status = 'явный null';
} else if (value === undefined) {
  status = 'не определено';
} else {

```

```
    status = 'присутствует';  
}
```

6.4. Ошибка: Слишком сложные выражения

```
javascript  
  
// НЕПРАВИЛЬНО: тернарный оператор внутри тернарного оператора внутри...  
const result = a ? b ? c ? d : e : f : g ? h : i;  
// Невозможно понять без серьезного анализа  
  
// ПРАВИЛЬНО: рефакторинг в понятные функции  
function calculateResult(a, b, c, d, e, f, g, h, i) {  
    if (a) {  
        if (b) {  
            return c ? d : e;  
        } else {  
            return f;  
        }  
    } else {  
        return g ? h : i;  
    }  
}  
  
// Или разбить на части  
function getFirstLevel(a, b, c, d, e, f) {  
    if (!a) return null;  
    return b ? getSecondLevel(c, d, e) : f;  
}  
  
function getSecondLevel(c, d, e) {  
    return c ? d : e;  
}
```

Часть 7: Рекомендации по стилю и лучшие практики

7.1. Правила ESLint для тернарных операторов

```
javascript  
  
// .eslintrc.js  
module.exports = {
```

```

rules: {
  // Запрещает вложенные тернарные операторы
  'no-nested-ternary': 'error',

  // Запрещает ненужные тернарные операторы
  'no-unneeded-ternary': 'error',

  // Требуется скобок вокруг сложных условий
  'no-extra-parens': ['error', 'all', {
    conditionalAssign: false,
    nestedBinaryExpressions: false,
    returnAssign: false,
    ignoreJSX: 'multi-line'
  }],

  // Стил для тернарных операторов
  'multiline-ternary': ['error', 'always-multiline'],

  // Перенос операторов
  'operator-linebreak': ['error', 'after', {
    overrides: {
      '?': 'before',
      ':': 'before'
    }
  }]
}
};

// Примеры с ESLint:

// ХОРОШО: простой тернарный оператор
const name = user ? user.name : 'Гость';

// ХОРОШО: многострочный для сложных выражений
const message = user.hasPermission
  ? `Добро пожаловать, ${user.name}!`
  : 'Доступ запрещен';

// ПЛОХО: вложенные тернарные операторы (нарушает no-nested-ternary)
const result = a ? b ? c : d : e;

// ПЛОХО: ненужный тернарный оператор (нарушает no-unneeded-ternary)
const isTrue = condition ? true : false;
// ЛУЧШЕ: const isTrue = condition;

```

// ХОРОШО: скобки для сложных условий

```
const access = (user.role === 'admin' || user.role === 'moderator')
  ? 'full'
  : 'limited';
```

7.2. Соглашения команды

javascript

// Пример правил для команды:

// 1. Всегда используйте тернарный оператор для простых присваиваний

// ПЛОХО

```
let status;
if (isActive) {
  status = 'active';
} else {
  status = 'inactive';
}
```

// ХОРОШО

```
const status = isActive ? 'active' : 'inactive';
```

// 2. Избегайте вложенных тернарных операторов больше 2 уровней

// ДОПУСТИМО (2 уровня)

```
const access = user.isAdmin ? 'full' :
  user.isMember ? 'partial' : 'none';
```

// НЕДОПУСТИМО (3+ уровня)

```
const complex = a ? b ? c ? d : e : f : g;
```

// 3. Форматируйте многострочные тернарные операторы одинаково

// ХОРОШО: операторы на отдельных строках

```
const result = longCondition
  ? longExpressionWhenTrue
  : longExpressionWhenFalse;
```

// 4. Используйте ранний возврат в функциях вместо тернарных операторов

// ПЛОХО

```
function getPrice(quantity) {
  return quantity > 100
    ? quantity * 0.9
    : quantity > 50
```

```
    ? quantity * 0.95
    : quantity * 0.98;
}
```

// ХОРОШО

```
function getPriceBetter(quantity) {
    if (quantity > 100) return quantity * 0.9;
    if (quantity > 50) return quantity * 0.95;
    return quantity * 0.98;
}
```

// 5. Для условий с побочными эффектами используйте if-else

// ПЛОХО

```
const value = shouldLog ? (console.log('Вычисление'), compute()) : compute();
```

// ХОРОШО

```
if (shouldLog) {
    console.log('Вычисление');
}
const value = compute();
```

7.3. Примеры из реальных проектов

javascript

// Пример 1: Конфигурация приложения

```
const config = {
    apiUrl: process.env.NODE_ENV === 'production'
        ? 'https://api.example.com'
        : 'http://localhost:3000',

    logLevel: process.env.DEBUG === 'true'
        ? 'debug'
        : process.env.NODE_ENV === 'development'
            ? 'info'
            : 'warn',

    features: {
        analytics: process.env.ANALYTICS_ENABLED === 'true',
        caching: process.env.NODE_ENV !== 'test',
        compression: process.env.NODE_ENV === 'production'
    }
};
```

// Пример 2: Форматирование данных для UI

```
function formatUserData(user) {
  return {
    id: user.id,
    displayName: user.nickname
      ? user.nickname
      : `${user.firstName} ${user.lastName}`.trim(),

    avatar: user.avatarUrl
      ? user.avatarUrl
      : `https://ui-avatars.com/api/?name=${encodeURIComponent(
        user.firstName + ' ' + user.lastName
      )}`,

    status: user.lastSeen
      ? Date.now() - user.lastSeen < 5 * 60 * 1000
        ? 'online'
        : 'offline'
      : 'unknown',

    role: user.isAdmin ? 'admin' :
      user.isModerator ? 'moderator' : 'user'
  };
}
```

// Пример 3: Валидация форм

```
function validateForm(data) {
  const errors = {};

  // Имя обязательно
  errors.name = !data.name || data.name.trim() === ''
    ? 'Имя обязательно для заполнения'
    : data.name.length < 2
    ? 'Имя должно содержать минимум 2 символа'
    : null;

  // Email валидация
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  errors.email = !data.email
    ? 'Email обязателен'
    : !emailRegex.test(data.email)
    ? 'Введите корректный email'
    : null;
}
```



```

// Пароль
errors.password = !data.password
  ? 'Пароль обязателен'
  : data.password.length < 8
    ? 'Пароль должен содержать минимум 8 символов'
    : null;

// Подтверждение пароля
errors.confirmPassword = data.password !== data.confirmPassword
  ? 'Пароли не совпадают'
  : null;

// Фильтруем null значения
return Object.fromEntries(
  Object.entries(errors).filter(([_, value]) => value !== null)
);
}

```

Заключение: Мудрое использование тернарного оператора

Ключевые выводы:

1. **Тернарный оператор — это выражение, а не утверждение** — он возвращает значение и может использоваться везде, где ожидается выражение.
2. **Используйте для простых условий, избегайте для сложных** — правило большого пальца: если вам нужно больше 2-3 секунд, чтобы понять тернарный оператор, он слишком сложен.
3. **Приоритет читаемости над краткостью** — понятный код важнее краткого кода. Современные JavaScript движки хорошо оптимизируют оба подхода.
4. **Найдите альтернативы** — иногда `if-else`, `switch`, объекты-словари или операторы `||`, `??`, `?.` могут быть более подходящими.
5. **Следуйте соглашениям команды** — согласованный стиль важнее личных предпочтений.

Чек-лист для проверки кода:

- Тернарный оператор используется только для простых условий (1-2 уровня вложенности)
- Нет побочных эффектов в условиях или выражениях

- Форматирование последовательное и читаемое
- Нет избыточных сравнений (`=== true`, `=== false`)
- Условия простые и понятные без комментариев
- Для сложных логик используются `if-else` или другие подходы
- Проверены граничные случаи (`null`, `undefined`, `0`, `"`, `false`)

Итоговый пример хорошего кода:

```
javascript
```

```
// Хорошее использование тернарного оператора
```

```
class UserService {
  constructor(config = {}) {
    this.apiUrl = config.apiUrl ?? this.getDefaultApiUrl();
    this.timeout = config.timeout ?? 30000;
    this.cacheEnabled = config.cache ?? process.env.NODE_ENV === 'production';
  }

  getDefaultApiUrl() {
    return process.env.NODE_ENV === 'production'
      ? 'https://api.example.com'
      : 'http://localhost:3000';
  }

  formatUserForDisplay(user) {
    if (!user) return null;

    return {
      id: user.id,
      name: user.name || 'Без имени',
      avatar: user.avatarUrl || this.generateDefaultAvatar(user.name),
      status: this.getUserStatus(user),
      role: this.getUserRole(user),
      lastActive: user.lastSeen
        ? this.formatRelativeTime(user.lastSeen)
        : 'никогда'
    };
  }

  getUserStatus(user) {
    if (!user.isActive) return 'неактивен';

    return user.lastSeen
      ? Date.now() - user.lastSeen < 5 * 60 * 1000
```

```

    ? 'онлайн'
    : 'был(а) недавно'
    : 'статус неизвестен';
}

getUserRole(user) {
  return user.isAdmin ? 'администратор' :
    user.isModerator ? 'модератор' :
    user.isPremium ? 'премиум' : 'пользователь';
}

generateDefaultAvatar(name) {
  const initials = name
    ? name.split(' ')
      .map(part => part.charAt(0))
      .join('')
      .toUpperCase()
      .slice(0, 2)
      : '??';

  return `https://ui-avatars.com/api/?name=${initials}&background=random`;
}

formatRelativeTime(timestamp) {
  const diff = Date.now() - timestamp;
  const minutes = Math.floor(diff / 60000);
  const hours = Math.floor(minutes / 60);
  const days = Math.floor(hours / 24);

  return days > 0 ? `${days} дн. назад` :
    hours > 0 ? `${hours} ч. назад` :
    minutes > 0 ? `${minutes} мин. назад` : 'только что';
}
}

```

Запомните: Тернарный оператор — это инструмент, а не панацея. Как и любой инструмент, он хорош для определенных задач и не подходит для других. Мудрый разработчик знает, когда его использовать, а когда выбрать более подходящий инструмент. Цель — не написать самый краткий код, а написать самый понятный и поддерживаемый код.

5.6. Операторы присваивания

Введение: Искусство присваивания значений

Операторы присваивания — это фундаментальный механизм JavaScript для изменения состояния переменных. Они не просто присваивают значения, но и могут комбинировать присваивание с другими операциями, что делает код более компактным и выразительным.

Часть 1: Базовый оператор присваивания (=)

1.1. Основы оператора присваивания

```
javascript

// Простое присваивание
let x = 10;           // Присваивание при объявлении
let y;                // Объявление без присваивания
y = 20;               // Присваивание после объявления

// Присваивание различных типов данных
const str = "Hello";  // Строка
const num = 42;        // Число
const bool = true;     // Boolean
const arr = [1, 2, 3]; // Массив
const obj = { name: "John" }; // Объект
const func = () => console.log(); // Функция
const nullValue = null; // null
const undefinedValue = undefined; // undefined

// Особенности присваивания
let a = b = c = 5;    // Цепочка присваивания справа налево
// Эквивалентно:
// c = 5
// b = c
// a = b

console.log(a, b, c); // 5 5 5
```

```
// Присваивание выражений
let result = (5 + 3) * 2; // Присваивание результата выражения
console.log(result);      // 16

// Присваивание с функциями
function getValue() {
  return 42;
}

let value = getValue(); // Присваивание возвращаемого значения
console.log(value);     // 42
```

1.2. Ссылочное присваивание (Reference Assignment)

```
javascript

// Примитивные типы: присваивание по значению
let original = 10;
let copy = original; // Создается копия значения

original = 20; // Изменение оригинала
console.log(original); // 20
console.log(copy);     // 10 (копия не изменилась)

// Объекты и массивы: присваивание по ссылке
let obj1 = { name: "Alice" };
let obj2 = obj1; // Присваивается ссылка на тот же объект

obj1.name = "Bob"; // Изменение через одну ссылку
console.log(obj1.name); // "Bob"
console.log(obj2.name); // "Bob" (изменилось через другую ссылку)

obj2 = { name: "Charlie" }; // Теперь obj2 ссылается на новый объект
console.log(obj1.name); // "Bob" (не изменился)
console.log(obj2.name); // "Charlie"

// Массивы также присваиваются по ссылке
let arr1 = [1, 2, 3];
let arr2 = arr1; // Обе переменные ссылаются на один массив

arr1.push(4); // Изменение через arr1
console.log(arr2); // [1, 2, 3, 4] (изменился и arr2)
```

```

// Создание реальной копии массива
let arr3 = [...arr1]; // Spread оператор создает копию
let arr4 = arr1.slice(); // Метод slice() создает копию
let arr5 = Array.from(arr1); // Array.from() создает копию

arr1.push(5);
console.log(arr1); // [1, 2, 3, 4, 5]
console.log(arr3); // [1, 2, 3, 4] (копия не изменилась)

// Создание реальной копии объекта
let obj3 = { a: 1, b: 2 };
let obj4 = { ...obj3 }; // Поверхностная копия (ES6)
let obj5 = Object.assign({}, obj3); // Поверхностная копия
let obj6 = JSON.parse(JSON.stringify(obj3)); // Глубокая копия

obj3.a = 999;
console.log(obj3.a); // 999
console.log(obj4.a); // 1 (копия не изменилась)

```

1.3. Деструктурирующее присваивание (Destructuring Assignment)

```

javascript

// Деструктуризация массивов
const numbers = [1, 2, 3, 4, 5];

// Базовое присваивание
const [first, second] = numbers;
console.log(first, second); // 1 2

// Пропуск элементов
const [, , third] = numbers;
console.log(third); // 3

// Остаточные параметры
const [head, ...tail] = numbers;
console.log(head); // 1
console.log(tail); // [2, 3, 4, 5]

// Значения по умолчанию
const [a = 10, b = 20] = [1];
console.log(a, b); // 1 20 (b взяло значение по умолчанию)

```

// Обмен значений без временной переменной

```
let x = 1, y = 2;  
[x, y] = [y, x];  
console.log(x, y); // 2 1
```

// Деструктуризация объектов

```
const user = {  
  id: 1,  
  name: "Alice",  
  age: 30,  
  email: "alice@example.com"  
};
```

// Базовое присваивание

```
const { name, age } = user;  
console.log(name, age); // "Alice" 30
```

// Присваивание с другим именем

```
const { name: userName, age: userAge } = user;  
console.log(userName, userAge); // "Alice" 30
```

// Значения по умолчанию

```
const { city = "Unknown", country = "Unknown" } = user;  
console.log(city, country); // "Unknown" "Unknown"
```

// Остаточные свойства

```
const { id, ...rest } = user;  
console.log(id); // 1  
console.log(rest); // { name: "Alice", age: 30, email: "alice@example.com" }
```

// Вложенная деструктуризация

```
const company = {  
  name: "TechCorp",  
  address: {  
    street: "123 Main St",  
    city: "San Francisco",  
    country: "USA"  
  }  
};
```

```
const {  
  name: companyName,  
  address: {  
    city: companyCity,
```

```

    country: companyCountry
  }
} = company;

console.log(companyName, companyCity, companyCountry);
// "TechCorp" "San Francisco" "USA"

// Деструктуризация в параметрах функции
function printUser({ name, age, email = "no-email@example.com" }) {
  console.log(`Name: ${name}, Age: ${age}, Email: ${email}`);
}

printUser(user); // Name: Alice, Age: 30, Email: alice@example.com
printUser({ name: "Bob", age: 25 }); // Name: Bob, Age: 25, Email: no-email@example.com

```

Часть 2: Составные операторы присваивания

2.1. Арифметические составные операторы присваивания

```

javascript

// += (присваивание со сложением)
let x = 10;
x += 5;    // Эквивалентно: x = x + 5
console.log(x); // 15

x += 2.5;  // Работает с дробными числами
console.log(x); // 17.5

// Особенность со строками
let str = "Hello";
str += " World"; // Конкатенация строк
console.log(str); // "Hello World"

// -= (присваивание с вычитанием)
let y = 20;
y -= 7;    // Эквивалентно: y = y - 7
console.log(y); // 13

y -= 2.5;
console.log(y); // 10.5

```



```
// *= (присваивание с умножением)
let z = 5;
z *= 3;    // Эквивалентно: z = z * 3
console.log(z); // 15

z *= 1.5;
console.log(z); // 22.5

// /= (присваивание с делением)
let a = 20;
a /= 4;    // Эквивалентно: a = a / 4
console.log(a); // 5

a /= 2;
console.log(a); // 2.5

// %= (присваивание с остатком от деления)
let b = 17;
b %= 5;    // Эквивалентно: b = b % 5
console.log(b); // 2

b %= 3;
console.log(b); // 2

// **= (присваивание с возведением в степень, ES7)
let c = 2;
c **= 3;    // Эквивалентно: c = c ** 3
console.log(c); // 8 (2³)

c **= 2;
console.log(c); // 64 (8²)

// Практические примеры
// Накопление суммы
let total = 0;
const prices = [10, 20, 30, 40];
for (let price of prices) {
  total += price;
}
console.log(total); // 100

// Постепенное уменьшение
let remainingTime = 60; // секунд
while (remainingTime > 0) {
```

```

    // Каждую секунду уменьшаем на 1
    remainingTime -= 1;
}

// Увеличение с коэффициентом
let investment = 1000;
const annualReturn = 0.07; // 7% годовых
for (let year = 1; year <= 10; year++) {
    investment *= (1 + annualReturn);
}
console.log(investment.toFixed(2)); // 1967.15

// Обновление счетчика
let counter = 0;
counter += 1; // Инкремент
counter -= 1; // Декремент
counter *= 2; // Удвоение
counter /= 2; // Деление пополам

```

2.2. Побитовые составные операторы присваивания

```

javascript

// &= (побитовое И с присваиванием)
let flags = 0b1111; // 15 в десятичной, 1111 в двоичной
flags &= 0b1010;    // 1111 & 1010 = 1010 (10 в десятичной)
console.log(flags.toString(2).padStart(4, '0')); // "1010"

// Пример: сброс флага
const PERMISSIONS = {
    READ:    0b0001,
    WRITE:   0b0010,
    EXECUTE: 0b0100,
    DELETE:  0b1000
};

let userPermissions = PERMISSIONS.READ | PERMISSIONS.WRITE; // 0011
// Сбросить флаг WRITE
userPermissions &= ~PERMISSIONS.WRITE; // 0011 & 1101 = 0001
console.log(userPermissions.toString(2).padStart(4, '0')); // "0001"

// |= (побитовое ИЛИ с присваиванием)
let bits = 0b0101; // 5 в десятичной
bits |= 0b1010;    // 0101 | 1010 = 1111 (15 в десятичной)

```

```

console.log(bits.toString(2).padStart(4, '0')); // "1111"

// Пример: установка флага
let permissions = PERMISSIONS.READ; // 0001
// Добавить флаг EXECUTE
permissions |= PERMISSIONS.EXECUTE; // 0001 | 0100 = 0101
console.log(permissions.toString(2).padStart(4, '0')); // "0101"

// ^= (побитовое исключающее ИЛИ с присваиванием)
let data = 0b1100; // 12 в десятичной
data ^= 0b1010; // 1100 ^ 1010 = 0110 (6 в десятичной)
console.log(data.toString(2).padStart(4, '0')); // "0110"

// Особенности: x ^= y дважды вернет исходное значение
let value = 0b0110;
value ^= 0b1010; // 0110 ^ 1010 = 1100
value ^= 0b1010; // 1100 ^ 1010 = 0110 (вернулось к исходному)
console.log(value.toString(2).padStart(4, '0')); // "0110"

// Пример: переключение флага
let toggleFlags = PERMISSIONS.READ; // 0001
// Переключить флаг WRITE
toggleFlags ^= PERMISSIONS.WRITE; // 0001 ^ 0010 = 0011
console.log(toggleFlags.toString(2).padStart(4, '0')); // "0011"
// Еще раз переключить
toggleFlags ^= PERMISSIONS.WRITE; // 0011 ^ 0010 = 0001
console.log(toggleFlags.toString(2).padStart(4, '0')); // "0001"

// <=> (побитовый сдвиг влево с присваиванием)
let num = 0b0001; // 1 в десятичной
num <=> 2; // 0001 << 2 = 0100 (4 в десятичной)
console.log(num.toString(2).padStart(4, '0')); // "0100"

// Пример: быстрое умножение на степень двойки
let value2 = 5;
value2 <=> 3; // 5 * 2^3 = 40
console.log(value2); // 40

// >=> (побитовый сдвиг вправо с присваиванием, сохраняет знак)
let signed = -16; // 11110000 (в 8-битном представлении)
signed >=> 2; // -16 >> 2 = -4 (11111100)
console.log(signed); // -4

// Пример: быстрое деление на степень двойки (с сохранением знака)

```

```

let divide = -20;
divide >>= 2; // -20 / 4 = -5
console.log(divide); // -5

// >>>= (побитовый сдвиг вправо с заполнением нулями)
let unsigned = -16; // 11110000
unsigned >>>= 2; // Преобразует к беззнаковому и сдвигает
console.log(unsigned); // 1073741820 (большое положительное число)

// Пример: преобразование цвета
function extractRGB(color) {
  const r = (color >>> 16) & 0xFF;
  const g = (color >>> 8) & 0xFF;
  const b = color & 0xFF;
  return { r, g, b };
}

const redColor = 0xFF0000;
const { r, g, b } = extractRGB(redColor);
console.log(r, g, b); // 255 0 0

```

2.3. Логические составные операторы присваивания

```

javascript

// &&= (логическое И с присваиванием, ES2021)
let x = 5;
x &&= 10; // Эквивалентно: x = x && 10
console.log(x); // 10 (x было truthy, поэтому присвоилось 10)

let y = 0;
y &&= 10; // y = 0 && 10 = 0
console.log(y); // 0 (y было falsy, поэтому осталось 0)

// Пример: условное обновление объекта
const config = {
  timeout: null,
  retries: 3
};

// Обновляем только если свойство не установлено
config.timeout &&= 30000; // timeout null → остался null
config.retries &&= 5; // retries 3 → стало 5

```

```

console.log(config); // { timeout: null, retries: 5 }

// ||= (логическое ИЛИ с присваиванием, ES2021)
let a = null;
a ||= 'default'; // Эквивалентно: a = a || 'default'
console.log(a); // 'default' (a было falsy, поэтому присвоилось 'default')

let b = 'existing';
b ||= 'default'; // b = 'existing' || 'default' = 'existing'
console.log(b); // 'existing' (b было truthy, поэтому не изменилось)

// Пример: значения по умолчанию для настроек
function createAppConfig(userConfig = {}) {
  const config = { ...userConfig };

  // Устанавливаем значения по умолчанию только если не заданы
  config.apiUrl ||= 'https://api.example.com';
  config.timeout ||= 30000;
  config.retries ||= 3;
  config.debug ||= false;

  return config;
}

console.log(createAppConfig({ timeout: 5000 }));
// { timeout: 5000, apiUrl: 'https://api.example.com', retries: 3, debug: false }

// ??= (nullish coalescing с присваиванием, ES2021)
let c = null;
c ??= 'default'; // Эквивалентно: c = c ?? 'default'
console.log(c); // 'default' (c было null)

let d = 0;
d ??= 'default'; // d = 0 ?? 'default' = 0
console.log(d); // 0 (0 не null/undefined, поэтому не изменилось)

let e = false;
e ??= true; // e = false ?? true = false
console.log(e); // false

// Пример: безопасное обновление с сохранением falsy значений
function updateUserSettings(settings = {}) {
  const defaults = {
    theme: 'light',

```

```

    notifications: true,
    itemsPerPage: 10,
    autosave: false, // false - допустимое значение
    timeout: 0        // 0 - допустимое значение
  };

  const result = { ...defaults, ...settings };

  // Обновляем только undefined/null
  result.theme ??= 'dark';
  result.notifications ??= false;
  result.itemsPerPage ??= 20;
  result.autosave ??= true; // Не работает, т.к. autosave = false
  result.timeout ??= 5000;  // Не работает, т.к. timeout = 0

  return result;
}

console.log(updateUserSettings({ theme: null, timeout: 0 }));
// { theme: 'dark', notifications: true, itemsPerPage: 10, autosave: false, timeout: 0 }

```

Часть 3: Специальные случаи и особенности

3.1. Присваивание в условиях и выражениях

```

javascript

// Присваивание внутри условия (часто считается антипаттерном)
let value;
if (value = getValue()) { // Присваивание и проверка в одном выражении
  console.log('Получено значение:', value);
}

// Эквивалентный, но более понятный код:
let value2 = getValue();
if (value2) {
  console.log('Получено значение:', value2);
}

// Присваивание в цикле while
const data = [1, 2, 3, 4, 5];
let item;

```

```

let i = 0;
while ((item = data[i])) { // Присваивание и проверка
  console.log(item);
  i++;
  if (i >= data.length) break;
}

// Более безопасная альтернатива:
for (let i = 0; i < data.length; i++) {
  const item = data[i];
  console.log(item);
}

// Присваивание в тернарном операторе (не рекомендуется)
let x = 5;
let y = (x > 10) ? (x = 20) : (x = 0);
console.log(x, y); // 0 0

// Лучше:
let x2 = 5;
if (x2 > 10) {
  x2 = 20;
} else {
  x2 = 0;
}
let y2 = x2;

// Присваивание как выражение (возвращает присвоенное значение)
let a = 0;
let b = 0;
let c = (a = 1, b = 2, a + b); // Используется оператор запятая
console.log(a, b, c); // 1 2 3

// Присваивание в цепочке вызовов (требуется осторожности)
function getConfig() {
  return {};
}

const config = getConfig();
(config.database = config.database || {}).host = 'localhost';
console.log(config); // { database: { host: 'localhost' } }

// Более безопасный подход:
const config2 = getConfig();

```

```
config2.database = config2.database || {};  
config2.database.host = 'localhost';
```

3.2. Присваивание с объектами и массивами

```
javascript  
  
// Присваивание свойств объектов  
const person = {};  
person.name = 'Alice';  
person.age = 30;  
person['email'] = 'alice@example.com'; // Квадратные скобки для вычисляемых имен  
  
// Динамические имена свойств  
const propName = 'status';  
person[propName] = 'active';  
  
// Вычисляемые имена свойств (ES6)  
const dynamicKey = 'user_' + Date.now();  
const obj = {  
  [dynamicKey]: 'value',  
  ['prefix_' + propName]: 'inactive'  
};  
  
console.log(obj);  
  
// Присваивание вложенных свойств  
const company = {};  
company.department = {};  
company.department.engineering = {};  
company.department.engineering.team = ['Alice', 'Bob'];  
company.department.engineering.manager = 'Charlie';  
  
// Опциональная цепочка для безопасного присваивания  
const safeCompany = {};  
safeCompany.department ??= {};  
safeCompany.department.engineering ??= {};  
safeCompany.department.engineering.team ??= [];  
safeCompany.department.engineering.team.push('David');  
  
// Присваивание элемента массива  
const arr = [1, 2, 3];  
arr[0] = 10; // Замена элемента  
arr[3] = 4; // Добавление элемента
```



```

arr[10] = 11;      // Создание разреженного массива
console.log(arr);  // [10, 2, 3, 4, empty × 6, 11]
console.log(arr.length); // 11

// Методы массива, которые изменяют оригинал
const numbers = [1, 2, 3, 4, 5];

numbers.push(6);      // Добавление в конец
numbers.pop();        // Удаление с конца
numbers.unshift(0);   // Добавление в начало
numbers.shift();      // Удаление с начала
numbers.splice(2, 1, 99); // Замена элемента
numbers.sort();       // Сортировка
numbers.reverse();    // Реверс

console.log(numbers); // [2, 3, 4, 5, 6, 99]

// Присваивание с деструктуризацией
let x = 1, y = 2;
[x, y] = [y, x]; // Обмен значений

// Обновление нескольких свойств объекта
const user = { name: 'Alice', age: 30 };
Object.assign(user, { age: 31, city: 'New York' });
// Или с spread оператором
const updatedUser = { ...user, age: 32, active: true };

```

3.3. Неизменяемое присваивание (Immutable Assignment)

```

javascript

// Подходы для работы в неизменяемом (immutable) стиле

// 1. Создание новых объектов вместо изменения существующих
const original = { a: 1, b: 2 };

// ПЛОХО: мутация оригинала
const mutated = original;
mutated.a = 999;
console.log(original.a); // 999 (изменился оригинал!)

// ХОРОШО: создание нового объекта
const original2 = { a: 1, b: 2 };
const newObject = { ...original2, a: 999 };

```

```
console.log(original2.a); // 1 (оригинал не изменился)
console.log(newObject.a); // 999

// 2. Создание новых массивов
const originalArray = [1, 2, 3];

// Мутлирующие методы (изменяют оригинал)
const badCopy = originalArray;
badCopy.push(4);
console.log(originalArray); // [1, 2, 3, 4]

// Не мутлирующие методы (создают новый массив)
const goodCopy = [...originalArray, 4];
console.log(originalArray); // [1, 2, 3]
console.log(goodCopy);      // [1, 2, 3, 4]

// 3. Глубокая неизменяемость
const deepObject = {
  level1: {
    level2: {
      level3: {
        value: 'deep'
      }
    }
  }
};

// Поверхностная копия (недостаточно для вложенных объектов)
const shallowCopy = { ...deepObject };
shallowCopy.level1.level2.level3.value = 'changed';
console.log(deepObject.level1.level2.level3.value); // 'changed' (все еще изменилось!)

// Глубокая копия
const deepCopy = JSON.parse(JSON.stringify(deepObject));
deepCopy.level1.level2.level3.value = 'really changed';
console.log(deepObject.level1.level2.level3.value); // 'changed' (оригинал не изменился)

// Или с использованием библиотек
// import { produce } from 'immer';
// const nextState = produce(deepObject, draft => {
//   draft.level1.level2.level3.value = 'changed';
// });

// 4. Неизменяемые обновления в React (стайл Redux)
```

```

function reducer(state = initialState, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      // Неправильно: мутация состояния
      // state.items.push(action.payload);
      // return state;

      // Правильно: новое состояние
      return {
        ...state,
        items: [...state.items, action.payload]
      };

    case 'UPDATE_ITEM':
      return {
        ...state,
        items: state.items.map(item =>
          item.id === action.payload.id
            ? { ...item, ...action.payload.changes }
            : item
        )
      };

    case 'REMOVE_ITEM':
      return {
        ...state,
        items: state.items.filter(item => item.id !== action.payload)
      };

    default:
      return state;
  }
}

// 5. Использование const для предотвращения переприсваивания
const immutableObject = { value: 42 };
// immutableObject = { value: 43 }; // Ошибка! Нельзя переприсвоить const

// Но можно изменять свойства (если объект не заморожен)
immutableObject.value = 43; // Работает!
console.log(immutableObject.value); // 43

// Object.freeze для настоящей неизменяемости
const frozen = Object.freeze({ value: 42 });

```

```

// frozen.value = 43; // B strict mode: TypeError
// frozen.newProp = 'test'; // B strict mode: TypeError

// Глубокая заморозка
function deepFreeze(obj) {
  Object.freeze(obj);
  Object.getOwnPropertyNames(obj).forEach(prop => {
    if (obj[prop] !== null &&
      (typeof obj[prop] === 'object' || typeof obj[prop] === 'function') &&
      !Object.isFrozen(obj[prop])) {
      deepFreeze(obj[prop]);
    }
  });
  return obj;
}

const deeplyFrozen = deepFreeze({
  nested: { value: 42 }
});
// deeplyFrozen.nested.value = 43; // TypeError в strict mode

```

Часть 4: Производительность и оптимизация

4.1. Бенчмаркинг операторов присваивания

```

javascript

function benchmarkAssignments() {
  const iterations = 10000000;

  console.time('Простое присваивание');
  let sum1 = 0;
  for (let i = 0; i < iterations; i++) {
    let x = i;
    sum1 += x;
  }
  console.timeEnd('Простое присваивание');

  console.time('Составное присваивание +=');
  let sum2 = 0;
  for (let i = 0; i < iterations; i++) {
    sum2 += i;
  }
}

```

```

}
console.timeEnd('Составное присваивание +=');

console.time('Множественное присваивание');
let a, b, c, d, e;
for (let i = 0; i < iterations; i++) {
  a = b = c = d = e = i;
}
console.timeEnd('Множественное присваивание');

console.time('Деструктуризация массива');
for (let i = 0; i < iterations; i++) {
  const [x, y] = [i, i + 1];
}
console.timeEnd('Деструктуризация массива');

console.time('Деструктуризация объекта');
for (let i = 0; i < iterations; i++) {
  const { x, y } = { x: i, y: i + 1 };
}
console.timeEnd('Деструктуризация объекта');

console.time('Object.assign');
const base = { a: 1, b: 2 };
for (let i = 0; i < iterations; i++) {
  const obj = Object.assign({}, base, { c: i });
}
console.timeEnd('Object.assign');

console.time('Spread оператор');
const base2 = { a: 1, b: 2 };
for (let i = 0; i < iterations; i++) {
  const obj = { ...base2, c: i };
}
console.timeEnd('Spread оператор');
}

```

// Результаты обычно показывают:

// 1. Составные операторы немного быстрее эквивалентных полных форм
// 2. Простые операции очень быстрые
// 3. Деструктуризация сравнима по скорости с ручным присваиванием
// 4. Object.assign и spread оператор имеют схожую производительность
// 5. Для большинства приложений разница незначительна

4.2. Оптимизация присваиваний в критическом коде

javascript

// 1. Минимизация повторных присваиваний

// ПЛОХО: повторное присваивание в цикле

```
function processArrayBad(array) {  
  let result;  
  for (let i = 0; i < array.length; i++) {  
    result = processItem(array[i]); // Присваивание в каждой итерации  
    // использование result...  
  }  
}
```

// ХОРОШО: присваивание за пределами цикла если возможно

```
function processArrayGood(array) {  
  for (let i = 0; i < array.length; i++) {  
    const result = processItem(array[i]); // Const внутри цикла  
    // использование result...  
  }  
}
```

// 2. Использование const где возможно

// ПЛОХО: везде let

```
let value1 = computeValue();  
let value2 = process(value1);  
let result = transform(value2);
```

// ХОРОШО: const для неизменяемых значений

```
const value1 = computeValue();  
const value2 = process(value1);  
const result = transform(value2);
```

// 3. Кэширование результатов дорогих операций

// ПЛОХО: повторное вычисление

```
function calculateTotal(items) {  
  return items  
    .filter(item => item.price > 100) // Вычисляется дважды  
    .reduce((sum, item) => sum + item.price * item.quantity, 0);  
}
```

// ХОРОШО: кэширование

```
function calculateTotalOptimized(items) {  
  const expensiveItems = items.filter(item => item.price > 100);
```

```
    return expensiveItems.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

// 4. Пакетное присваивание

// ПЛОХО: множество отдельных присваиваний

```
function updateUserBad(user, updates) {
    if (updates.name) user.name = updates.name;
    if (updates.email) user.email = updates.email;
    if (updates.age) user.age = updates.age;
    if (updates.city) user.city = updates.city;
    // ... и так далее для многих свойств
}
```

// ХОРОШО: пакетное обновление

```
function updateUserGood(user, updates) {
    Object.assign(user, updates);
    // Или с проверками
    for (const [key, value] of Object.entries(updates)) {
        if (value !== undefined) {
            user[key] = value;
        }
    }
}
```

// 5. Избегание временных переменных

// ПЛОХО: лишние временные переменные

```
function calculateDiscount(price, isMember) {
    let discountRate;
    if (isMember) {
        discountRate = 0.2;
    } else {
        discountRate = 0.1;
    }
    let discount = price * discountRate;
    return price - discount;
}
```

// ХОРОШО: прямое вычисление

```
function calculateDiscountOptimized(price, isMember) {
    const discountRate = isMember ? 0.2 : 0.1;
    return price * (1 - discountRate);
}
```

// ЕЩЕ ЛУЧШЕ: без временной переменной

```
function calculateDiscountBest(price, isMember) {  
  return price * (1 - (isMember ? 0.2 : 0.1));  
}
```

Часть 5: Практические паттерны и идиомы

5.1. Паттерн: Фабрика объектов с присваиванием

```
javascript  
  
// Создание объектов с конфигурируемыми свойствами  
function createUser(config = {}) {  
  // Значения по умолчанию  
  const defaults = {  
    id: generateId(),  
    name: 'Гость',  
    email: null,  
    isActive: true,  
    role: 'user',  
    createdAt: new Date(),  
    settings: {}  
  };  
  
  // Объединение с пользовательской конфигурацией  
  const user = { ...defaults, ...config };  
  
  // Вычисляемые свойства  
  user.displayName = user.nickname || `${user.firstName} ${user.lastName}`.trim();  
  user.avatarUrl ||= generateAvatar(user.displayName);  
  user.permissions = calculatePermissions(user.role);  
  
  // Валидация  
  if (!user.email && user.role !== 'guest') {  
    throw new Error('Email required for non-guest users');  
  }  
  
  return user;  
}  
  
// Вспомогательные функции  
function generateId() {  
  return Math.random().toString(36).substr(2, 9);  
}
```



```

}

function generateAvatar(name) {
  return `https://ui-avatars.com/api/?name=${encodeURIComponent(name)}`;
}

function calculatePermissions(role) {
  const permissions = {
    guest: ['read'],
    user: ['read', 'write', 'comment'],
    moderator: ['read', 'write', 'comment', 'moderate'],
    admin: ['read', 'write', 'comment', 'moderate', 'admin']
  };
  return permissions[role] || permissions.guest;
}

// Использование
const newUser = createUser({
  firstName: 'Иван',
  lastName: 'Иванов',
  email: 'ivan@example.com',
  role: 'moderator'
});

console.log(newUser);

```

5.2. Паттерн: Построитель (Builder) с цепочкой присваиваний

```

javascript

class QueryBuilder {
  constructor() {
    this.reset();
  }

  reset() {
    this.query = {
      select: [],
      from: null,
      where: [],
      orderBy: [],
      limit: null,
      offset: null
    };
  }
}

```

```

    return this;
}

select(...fields) {
    this.query.select.push(...fields);
    return this;
}

from(table) {
    this.query.from = table;
    return this;
}

where(condition) {
    this.query.where.push(condition);
    return this;
}

orderBy(field, direction = 'ASC') {
    this.query.orderBy.push({ field, direction });
    return this;
}

limit(count) {
    this.query.limit = count;
    return this;
}

offset(count) {
    this.query.offset = count;
    return this;
}

build() {
    // Валидация
    if (!this.query.from) {
        throw new Error('FROM clause is required');
    }

    // Сборка SQL
    let sql = 'SELECT ';
    sql += this.query.select.length > 0
        ? this.query.select.join(', ')
        : '*';

```

```

sql += ` FROM ${this.query.from}`;

if (this.query.where.length > 0) {
  sql += ' WHERE ' + this.query.where.join(' AND ');
}

if (this.query.orderBy.length > 0) {
  const orders = this.query.orderBy.map(
    ({ field, direction }) => `${field} ${direction}`
  );
  sql += ' ORDER BY ' + orders.join(', ');
}

if (this.query.limit !== null) {
  sql += ` LIMIT ${this.query.limit}`;
  if (this.query.offset !== null) {
    sql += ` OFFSET ${this.query.offset}`;
  }
}

return sql;
}
}

```

// Использование с цепочкой присваиваний

```

const query = new QueryBuilder()
  .select('id', 'name', 'email')
  .from('users')
  .where('active = true')
  .where('age >= 18')
  .orderBy('name', 'ASC')
  .limit(10)
  .build();

```

```
console.log(query);
```

// SELECT id, name, email FROM users WHERE active = true AND age >= 18

// ORDER BY name ASC LIMIT 10

5.3. Паттерн: Состояние с иммутабельными обновлениями

javascript

// Управление состоянием в стиле Redux

```

function createStore(reducer, initialState) {
  let state = initialState;
  const listeners = [];

  const getState = () => state;

  const dispatch = (action) => {
    // Иммуutableное обновление состояния
    state = reducer(state, action);

    // Уведомление подписчиков
    listeners.forEach(listener => listener());
  };

  const subscribe = (listener) => {
    listeners.push(listener);

    // Функция отписки
    return () => {
      const index = listeners.indexOf(listener);
      if (index !== -1) {
        listeners.splice(index, 1);
      }
    };
  };

  return { getState, dispatch, subscribe };
}

// Пример редьюсера с иммуutableными обновлениями
const initialState = {
  todos: [],
  filter: 'all',
  loading: false,
  error: null
};

function todoReducer(state = initialState, action) {
  switch (action.type) {
    case 'ADD_TODO':
      return {
        ...state,
        todos: [...state.todos, {
          id: Date.now(),

```

```

        text: action.payload.text,
        completed: false
    }]
};

case 'TOGGLE_TODO':
    return {
        ...state,
        todos: state.todos.map(todo =>
            todo.id === action.payload.id
                ? { ...todo, completed: !todo.completed }
                : todo
        )
    };

case 'REMOVE_TODO':
    return {
        ...state,
        todos: state.todos.filter(todo => todo.id !== action.payload.id)
    };

case 'SET_FILTER':
    return {
        ...state,
        filter: action.payload.filter
    };

case 'SET_LOADING':
    return {
        ...state,
        loading: action.payload.loading
    };

case 'SET_ERROR':
    return {
        ...state,
        error: action.payload.error
    };

default:
    return state;
}
}

```

```
// Использование
const store = createStore(todoReducer, initialState);

store.dispatch({
  type: 'ADD_TODO',
  payload: { text: 'Изучить JavaScript' }
});

store.dispatch({
  type: 'ADD_TODO',
  payload: { text: 'Попрактиковаться' }
});

console.log(store.getState().todos);
```

5.4. Паттерн: Трансформация данных с присваиванием

javascript

```
// Конвейер преобразования данных
function createDataPipeline(transformations) {
  return function pipeline(data) {
    return transformations.reduce((result, transform) => {
      return transform(result);
    }, data);
  };
}
```

```
// Трансформации с иммутабельными присваиваниями
```

```
const transformations = [
  // 1. Очистка данных
  data => ({
    ...data,
    users: data.users.map(user => ({
      ...user,
      name: user.name.trim(),
      email: user.email.toLowerCase()
    })))
  },
  // 2. Фильтрация
  data => ({
    ...data,
    users: data.users.filter(user => user.isActive)
  })
];
```

```
}},
```

```
// 3. Обогащение
```

```
data => ({
  ...data,
  users: data.users.map(user => ({
    ...user,
    // Добавляем вычисляемые поля
    initials: user.name
      .split(' ')
      .map(part => part.charAt(0))
      .join('')
      .toUpperCase(),

    domain: user.email.split('@')[1],

    // Категория по возрасту
    ageGroup: user.age < 18 ? 'minor' :
      user.age < 65 ? 'adult' : 'senior'
  }))
}),
```

```
// 4. Группировка
```

```
data => {
  const grouped = data.users.reduce((groups, user) => {
    const group = user.ageGroup;
    groups[group] ??= [];
    groups[group].push(user);
    return groups;
  }, {});

  return {
    ...data,
    groupedUsers: grouped
  };
},
```

```
// 5. Статистика
```

```
data => ({
  ...data,
  stats: {
    totalUsers: data.users.length,
    activeUsers: data.users.filter(u => u.isActive).length,
    avgAge: data.users.reduce((sum, u) => sum + u.age, 0) / data.users.length,
```

```

    ageGroups: Object.keys(data.groupedUsers || {}).reduce((stats, group) => {
      stats[group] = data.groupedUsers[group].length;
      return stats;
    }, {})
  }
})
];

// Исходные данные
const rawData = {
  users: [
    { id: 1, name: 'Alice ', email: 'ALICE@EXAMPLE.COM', age: 25, isActive: true },
    { id: 2, name: 'Bob', email: 'bob@test.com', age: 17, isActive: true },
    { id: 3, name: 'Charlie ', email: 'charlie@domain.com', age: 70, isActive: false },
    { id: 4, name: 'Diana', email: 'DIANA@EXAMPLE.COM', age: 30, isActive: true }
  ]
};

// Применение конвейера
const pipeline = createDataPipeline(transformations);
const processedData = pipeline(rawData);

console.log(processedData);

```

Часть 6: Распространенные ошибки и антипаттерны

6.1. Ошибка: Путаница между присваиванием (=) и сравнением (==, ===)

```

javascript

// НАИБОЛЕЕ РАСПРОСТРАНЕННАЯ ОШИБКА!

// НЕПРАВИЛЬНО: присваивание вместо сравнения
let x = 5;
if (x = 10) { // ОШИБКА: присваивает 10 переменной x, а не сравнивает!
  console.log('x равно 10'); // Всегда выполнится, т.к. 10 - truthy
}
console.log(x); // 10 (значение изменилось!)

// ПРАВИЛЬНО: сравнение
let y = 5;

```



```

if (y === 10) { // Сравнивает значение
  console.log('y равно 10'); // Не выполнится
}
console.log(y); // 5 (значение не изменилось)

// Особо опасный случай: в условии while
let z = 0;
// НЕПРАВИЛЬНО
// while (z = getNextValue()) { // Бесконечный цикл, если getNextValue() возвращает
//   console.log(z);
// }

// ПРАВИЛЬНО
while ((z = getNextValue()) !== null) { // Явное сравнение
  console.log(z);
}

// Защита от ошибки: используйте линтеры
// ESLint правило: no-cond-assign
// Запрещает присваивание в условиях

// Альтернатива: обернуть присваивание в дополнительные скобки
let value;
if ((value = getValue()) !== null) {
  // Теперь явно видно, что это присваивание + сравнение
  console.log(value);
}

```

6.2. Ошибка: Неправильное использование составных операторов

```

javascript

// ОШИБКА: использование несуществующего оператора
let x = 5;
// x += 10; // ОШИБКА: += это не оператор! Это = и унарный +
// На самом деле это: x = +10, что равно x = 10

// ПРАВИЛЬНО: составной оператор +=
x += 10; // x = x + 10 = 15

// ОШИБКА: путаница с инкрементом
let y = 5;

```

```

y += 1; // y = +1 = 1 (не то, что ожидалось!)
y += 1; // y = 1 + 1 = 2

// Для инкремента используйте:
y++;    // Постфиксный инкремент: y = 2, результат = 2, затем y = 3
++y;    // Префиксный инкремент: y = 3, результат = 4

// ОШИБКА: использование с неподходящими типами
let str = "10";
str += 5; // "105" (конкатенация строк)
// Возможно, ожидалось арифметическое сложение: 15

// ПРАВИЛЬНО: явное преобразование
let num = parseInt("10", 10);
num += 5; // 15

// Или
let value = "10";
value = Number(value) + 5; // 15

```

6.3. Ошибка: Мутация при копировании по ссылке

```

javascript

// ОШИБКА: непреднамеренная мутация общего объекта
const defaultConfig = {
  timeout: 30000,
  retries: 3
};

function createApp(config) {
  // НЕПРАВИЛЬНО: прямое присваивание
  const appConfig = config || defaultConfig;

  appConfig.timeout = 50000; // Мутуем defaultConfig!

  return appConfig;
}

const app1 = createApp();
console.log(defaultConfig.timeout); // 50000 (изменился!)

// ПРАВИЛЬНО: создание копии
function createAppSafe(config) {

```

```

// Создаем новый объект на основе defaultConfig
const baseConfig = { ...defaultConfig };

// Объединяем с пользовательской конфигурацией
const appConfig = config
  ? { ...baseConfig, ...config }
  : baseConfig;

// Дальнейшие изменения не затрагивают defaultConfig
appConfig.timeout = 50000;

return appConfig;
}

const app2 = createAppSafe();
console.log(defaultConfig.timeout); // 30000 (не изменился)

// ОШИБКА: такая же проблема с массивами
const originalArray = [1, 2, 3];
const newArray = originalArray; // Только ссылка!

newArray.push(4);
console.log(originalArray); // [1, 2, 3, 4] (изменился!)

// ПРАВИЛЬНО: создание копии массива
const safeArray = [...originalArray]; // Или originalArray.slice()
safeArray.push(5);
console.log(originalArray); // [1, 2, 3, 4] (не изменился)
console.log(safeArray); // [1, 2, 3, 4, 5]

```

6.4. Ошибка: Неправильное использование const/let/var

```

javascript

// ОШИБКА: повторное присваивание const
const PI = 3.14159;
// PI = 3.14; // TypeError: Assignment to constant variable

// ПРАВИЛЬНО: используйте let если значение будет меняться
let temperature = 20;
temperature = 25; // ОК

// ОШИБКА: использование var в блоке видимости
function problematic() {

```

```
if (true) {
  var x = 10; // var имеет функциональную область видимости
}
console.log(x); // 10 (доступно вне блока!)
}

// ПРАВИЛЬНО: используйте let/const
function correct() {
  if (true) {
    let y = 10; // let имеет блочную область видимости
    const z = 20;
  }
  // console.log(y); // ReferenceError: y is not defined
  // console.log(z); // ReferenceError: z is not defined
}

// ОШИБКА: объявление без присваивания const
// const uninitialized; // SyntaxError: Missing initializer in const declaration

// ПРАВИЛЬНО: const требует инициализации
const initialized = 'value';

// Или используйте let
let canBeInitializedLater;
canBeInitializedLater = 'some value';

// ОШИБКА: переопределение const в блоке
const global = 'outer';
// if (true) {
//   const global = 'inner'; // SyntaxError в некоторых случаях
// }

// ПРАВИЛЬНО: разные области видимости
function outer() {
  const value = 'outer';
  if (true) {
    const value = 'inner'; // Допустимо, т.к. в другой области видимости
    console.log(value); // 'inner'
  }
  console.log(value); // 'outer'
}
```

Часть 7: Рекомендации по стилю и лучшие практики

7.1. Соглашения об именовании переменных

```
javascript

// 1. Используйте camelCase для переменных и функций
const userName = 'Alice';
const itemCount = 5;
const calculateTotal = () => {};

// 2. Используйте PascalCase для классов и конструкторов
class UserAccount {}
function DatabaseConnection() {}

// 3. Используйте UPPER_SNAKE_CASE для констант
const MAX_RETRIES = 3;
const API_BASE_URL = 'https://api.example.com';
const DEFAULT_TIMEOUT = 30000;

// 4. Используйте префиксы для специальных случаев
// Приватные поля (до ES2022)
class Person {
  constructor(name) {
    this._name = name; // Префикс _ для "приватного"
  }
}

// Современные приватные поля (ES2022+)
class ModernPerson {
  #secret = 'confidential';

  reveal() {
    return this.#secret;
  }
}

// 5. Используйте осмысленные имена
// ПЛОХО
const d = new Date();
const n = 'John';
const cnt = 5;
```

```

// ХОРОШО
const currentDate = new Date();
const userName = 'John';
const itemCount = 5;

// 6. Избегайте однобуквенных имен (кроме счетчиков циклов)
// Приемлемо в коротких циклах
for (let i = 0; i < 10; i++) {
  // i, j, k допустимы для счетчиков
}

// Не используйте для других целей
const x = getUser(); // ПЛОХО
const user = getUser(); // ХОРОШО

// 7. Будьте последовательны в команде
// Выберите один стиль и придерживайтесь его
const isEnabled = true; // boolean с префиксом is/has/should
const hasPermission = true;
const shouldValidate = false;

// Или альтернативный стиль
const enabled = true;
const permission = true;
const validate = false;

```

7.2. Правила ESLint для операторов присваивания

```

javascript

// .eslintrc.js
module.exports = {
  rules: {
    // Запрещает присваивание в условиях
    'no-cond-assign': ['error', 'always'],

    // Запрещает присваивание в выражениях return
    'no-return-assign': ['error', 'always'],

    // Требуется const для переменных, которые не переприсваиваются
    'prefer-const': ['error', {
      destructuring: 'all',
      ignoreReadBeforeAssign: false
    }],
  },
};

```

```

// Запрещает неиспользуемые переменные
'no-unused-vars': ['error', {
  vars: 'all',
  args: 'after-used',
  ignoreRestSiblings: true
}],

// Запрещает переменные с одинаковыми именами в одной области видимости
'no-shadow': ['error', {
  hoist: 'all'
}],

// Запрещает переопределение переменных из внешней области видимости
'no-shadow-restricted-names': 'error',

// Запрещает объявление переменных с undefined
'no-undef-init': 'error',

// Предпочитает деструктуризацию
'prefer-destructuring': ['error', {
  array: true,
  object: true
}, {
  enforceForRenamedProperties: false
}]
}
};

// Примеры с ESLint:

// ХОРОШО: const для неизменяемых значений
const PI = 3.14159;
const user = { name: 'Alice' };

// ПЛОХО: Let когда можно использовать const
let total = 0; // Нужно переприсваивать? Если да - ОК, если нет - используйте const

// ХОРОШО: деструктуризация
const { name, age } = user;

// ПЛОХО: ручное извлечение свойств
const userName = user.name;
const userAge = user.age;

```

// ХОРОШО: нет присваивания в условиях

```
let value = getValue();
if (value !== null) {
  process(value);
}
```

// ПЛОХО: присваивание в условии

```
if (value = getValue()) { // ESLint: no-cond-assign
  process(value);
}
```

// ХОРОШО: отдельное присваивание в return

```
function getValue() {
  const result = compute();
  return result;
}
```

// ПЛОХО: присваивание в return

```
function getValueBad() {
  return result = compute(); // ESLint: no-return-assign
}
```

7.3. Итоговые рекомендации

javascript

// 1. Используйте const по умолчанию, let только когда нужно переприсваивание

```
const DEFAULT_CONFIG = { timeout: 30000 };
let userInput = ''; // Будет меняться
```

// 2. Инициализируйте переменные при объявлении

```
const name = 'Alice'; // ХОРОШО
let age;
age = 30; // ПЛОХО (если можно инициализировать сразу)
```

// 3. Группируйте объявления переменных

```
// ХОРОШО
const MAX_USERS = 100;
const DEFAULT_TIMEOUT = 30000;
const API_BASE_URL = 'https://api.example.com';

let currentUser = null;
let isLoading = false;
```



```

let errorMessage = '';

// 4. Используйте составные операторы для краткости
let total = 0;
total += price; // Вместо total = total + price
count *= 2;      // Вместо count = count * 2

// 5. Используйте деструктуризацию для извлечения значений
const { id, name, email } = user;
const [first, second, ...rest] = items;

// 6. Будьте осторожны с присваиванием по ссылке
const original = { a: 1 };
const copy = { ...original }; // Создайте копию если нужно

// 7. Используйте современные операторы присваивания (ES2021+)
config.timeout ??= 30000; // Только если null/undefined
config.retries ||= 3;     // Только если falsy
config.enabled &&= false;  // Только если truthy

// 8. Избегайте сложных выражений присваивания
// ПЛОХО
const result = (x = getValue()) !== null ? process(x) : defaultValue;

// ХОРОШО
const value = getValue();
const result = value !== null ? process(value) : defaultValue;

// 9. Комментируйте неочевидные присваивания
// Это выглядит странно, но нужно для оптимизации
let temp = a;
a = b;
b = temp; // Обмен значений без деструктуризации

// 10. Тестируйте граничные случаи
function safeParseInt(str, defaultValue = 0) {
  const num = parseInt(str, 10);
  return isNaN(num) ? defaultValue : num;
}

// Проверьте:
safeParseInt('42');      // 42
safeParseInt('abc');     // 0
safeParseInt('', 100);   // 100

```

```
safeParseInt(null, -1); // -1
```

Заключение: Мастерство присваивания

Ключевые выводы:

1. **Оператор присваивания (=) — это фундамент** — понимание разницы между присваиванием по значению и по ссылке критически важно.
2. **Составные операторы делают код лаконичнее** — `+=`, `-=`, `*=`, `/=`, `%=`, `**=`, а также логические `&&=`, `||=`, `??=` и побитовые операторы.
3. **Деструктуризация — это мощный инструмент** — позволяет извлекать значения из массивов и объектов с элегантным синтаксисом.
4. **Иммутабельность предотвращает ошибки** — создание новых объектов/массивов вместо мутации существующих делает код предсказуемее.
5. `const` по умолчанию, `let` по необходимости — это правило делает намерения яснее и предотвращает случайные переприсваивания.

Чек-лист для проверки кода:

- Используется `const` для значений, которые не переписываются
- Составные операторы используются где уместно
- Деструктуризация применяется для извлечения значений
- Нет присваивания в условиях (или оно явно обернуто в скобки)
- Создаются копии объектов/массивов перед модификацией
- Имена переменных понятны и следуют соглашениям команды
- Современные операторы присваивания (`??=`, `||=`, `&&=`) используются где уместно
- Проверены граничные случаи (`null`, `undefined`, `0`, `"`, `false`)

Итоговый пример профессионального кода:

```
javascript

// Профессиональное использование операторов присваивания
class DataProcessor {
  constructor(config = {}) {
    // Константы с UPPER_SNAKE_CASE
    this.DEFAULT_OPTIONS = {
      MAX_ITEMS: 1000,
      TIMEOUT_MS: 30000,
      RETRY_COUNT: 3
    }
  }
}
```

```

};

// Конфигурация с значениями по умолчанию
this.config = {
  ...this.DEFAULT_OPTIONS,
  ...config,
  // Современные операторы присваивания (ES2021+)
  cacheEnabled: config.cacheEnabled ?? true,
  validateInput: config.validateInput || false,
  debugMode: config.debugMode && process.env.NODE_ENV === 'development'
};

// Состояние с Let (будет меняться)
this.isProcessing = false;
this.progress = 0;
this.result = null;
this.errors = [];
}

async process(data) {
  // Валидация входных данных
  const validatedData = this.validate(data);
  if (!validatedData) {
    throw new Error('Invalid input data');
  }

  // Начало обработки
  this.isProcessing = true;
  this.progress = 0;
  this.result = null;
  this.errors = [];

  try {
    // Деструктуризация для извлечения значений
    const { items, metadata } = validatedData;

    // Обработка с иммутабельными обновлениями
    let processedItems = [];
    for (let i = 0; i < items.length; i++) {
      // Составной оператор для прогресса
      this.progress = Math.floor((i / items.length) * 100);

      // Обработка элемента
      const processedItem = await this.processItem(items[i], metadata);
    }
  }
}

```

```

        // Добавление с созданием нового массива
        processedItems = [...processedItems, processedItem];

        // Проверка лимита
        if (processedItems.length >= this.config.MAX_ITEMS) {
            break;
        }
    }

    // Итоговый результат
    this.result = {
        items: processedItems,
        total: processedItems.length,
        successful: processedItems.filter(item => !item.error).length,
        failed: processedItems.filter(item => item.error).length,
        metadata: {
            ...metadata,
            processedAt: new Date().toISOString(),
            processingTime: Date.now() - this.startTime
        }
    };

    // Логирование только в debug режиме
    this.config.debugMode && console.log('Processing completed:', this.result);

    return this.result;

} catch (error) {
    // Обработка ошибок
    this.errors.push({
        message: error.message,
        timestamp: new Date().toISOString(),
        stack: error.stack
    });

    throw error;

} finally {
    // Завершение обработки
    this.isProcessing = false;
    this.progress = 100;
}
}

```

```

validate(data) {
  if (!data || typeof data !== 'object') {
    return null;
  }

  // Деструктуризация с значениями по умолчанию
  const {
    items = [],
    metadata = {},
    ...rest
  } = data;

  // Валидация элементов
  if (!Array.isArray(items)) {
    return null;
  }

  // Возврат нового объекта (иммутабельно)
  return {
    items: items.filter(item => this.isValidItem(item)),
    metadata: {
      ...metadata,
      validatedAt: new Date().toISOString(),
      originalCount: items.length,
      validCount: items.filter(item => this.isValidItem(item)).length
    },
    // Сохраняем остальные свойства
    ...rest
  };
}

isValidItem(item) {
  // Проверка с использованием операторов сравнения, а не присваивания
  return item &&
    typeof item === 'object' &&
    item.id !== undefined &&
    item.value !== undefined;
}

async processItem(item, metadata) {
  // Эмуляция асинхронной обработки
  await new Promise(resolve =>
    setTimeout(resolve, Math.random() * 100)
  );
}

```

```

);

// Обработка с созданием нового объекта
return {
  ...item,
  processed: true,
  processedAt: new Date().toISOString(),
  metadata: metadata,
  result: this.calculateResult(item.value),
  // Условное добавление свойства
  ...(item.priority && { priority: item.priority })
};
}

calculateResult(value) {
  // Составные операторы для вычислений
  let result = value;
  result += metadata.adjustment || 0;
  result *= metadata.multiplier || 1;
  result -= metadata.deduction || 0;

  // Ограничение диапазона
  result = Math.max(0, Math.min(100, result));

  return result;
}

// Метод для сброса состояния
reset() {
  // Присваивание нескольких свойств
  Object.assign(this, {
    isProcessing: false,
    progress: 0,
    result: null,
    errors: []
  });
}

// Использование
const processor = new DataProcessor({
  MAX_ITEMS: 500,
  debugMode: true
});

```

```
const data = {  
  items: [  
    { id: 1, value: 50, priority: 'high' },  
    { id: 2, value: 75 },  
    { id: 3, value: 25, priority: 'low' }  
  ],  
  metadata: {  
    adjustment: 10,  
    multiplier: 1.1  
  }  
};  
  
processor.process(data)  
  .then(result => console.log('Success:', result))  
  .catch(error => console.error('Error:', error));
```

Запомните: Операторы присваивания — это не просто механизм изменения значений переменных. Это инструмент для выражения намерений, управления состоянием и создания элегантного, эффективного кода. Правильное использование операторов присваивания отделяет начинающего разработчика от опытного профессионала, который понимает нюансы языка и пишет код, который легко читать, поддерживать и отлаживать.

5.7. Приоритет операторов и ассоциативность

Введение: зачем это важно

Приоритет операторов определяет, в каком порядке выполняются операции в выражении, когда нет явных скобок. Ассоциативность определяет порядок выполнения операторов с одинаковым приоритетом.

Аналогия из математики:

- В выражении `2 + 3 * 4` умножение выполняется первым (приоритет выше)
- В выражении `8 / 4 / 2` деление выполняется слева направо (левоассоциативно)

Часть 1: Таблица приоритетов операторов JavaScript

Полная таблица приоритетов (от высшего к низшему):

Приоритет	Тип операторов	Операторы	Ассоциативность	Пример
21	Группировка	<code>()</code>	Неприменимо	<code>(2 + 3) * 4</code>
20	Доступ к членам	<code>.</code> <code>[]</code> <code>()</code>	Слева направо	<code>obj.prop</code> , <code>arr[0]</code> , <code>func()</code>
19	Оператор new (с аргументами)	<code>new</code>	Неприменимо	<code>new Date()</code>
18	Постфиксные инкремент/декремент	<code>++</code> <code>--</code>	Неприменимо	<code>i++</code> , <code>j--</code>
17	Логическое НЕ, унарные операторы	<code>!</code> <code>~</code> <code>+</code> <code>-</code> <code>++</code> <code>--</code> <code>typeof</code> <code>void</code> <code>id</code> <code>delete</code>	Справа налево	<code>!true</code> , <code>-5</code> , <code>++x</code>
16	Возведение в степень	<code>**</code>	Справа налево	<code>2 ** 3 ** 2</code>

Приоритет	Тип операторов	Операторы	Ассоциативность	Пример
15	Мультипликативные	<code>*</code> <code>/</code> <code>%</code>	Слева направо	<code>6 * 3 / 2</code>
14	Аддитивные	<code>+</code> <code>-</code>	Слева направо	<code>3 + 4 - 1</code>
13	Побитовый сдвиг	<code><<</code> <code>>></code> <code>>>></code>	Слева направо	<code>8 << 2</code>
12	Отношения	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>in</code> <code>instanceof</code>	Слева направо	<code>a < b <= c</code>
11	Равенство	<code>==</code> <code>!=</code> <code>===</code> <code>!==</code>	Слева направо	<code>a == b != c</code>
10	Побитовое И	<code>&</code>	Слева направо	<code>a & b & c</code>
9	Побитовое исключающее ИЛИ	<code>^</code>	Слева направо	<code>a ^ b ^ c</code>
8	Побитовое ИЛИ	<code> </code>	Слева направо	<code>a b c</code>
7	Логическое И	<code>&&</code>	Слева направо	<code>a && b && c</code>
6	Логическое ИЛИ	<code> </code>	Слева направо	<code>a b c</code>
5	Нулевое слияние	<code>??</code>	Слева направо	<code>a ?? b ?? c</code>
4	Условный (тернарный)	<code>?:</code>	Справа налево	<code>a ? b : c ? d : e</code>
3	Присваивание	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> и др.	Справа налево	<code>a = b = 5</code>
2	<code>yield</code> , <code>yield*</code>	<code>yield</code> <code>yield*</code>	Справа налево	<code>yield a + b</code>
1	Запятая	<code>,</code>	Слева направо	<code>a = (1, 2, 3)</code>

Часть 2: Подробное объяснение с примерами

2.1. Группировка (приоритет 21)

```
javascript
```

```
// Скобки имеют наивысший приоритет
```

```
let result1 = 2 + 3 * 4; // 14 (сначала 3*4, потом +2)
```

```
let result2 = (2 + 3) * 4; // 20 (сначала 2+3, потом *4)
```

```
// Сложные выражения
let a = 5, b = 3, c = 2;
let calculation1 = a + b * c;           // 11 (3*2=6, +5)
let calculation2 = (a + b) * c;         // 16 (5+3=8, *2)
let calculation3 = a * (b + c);         // 25 (3+2=5, *5)
let calculation4 = (a * b) + c;         // 17 (5*3=15, +2)

// Использование в функциях
let value = Math.pow(2, (3 + 1));       // 16 (2^4)
```

2.2. Доступ к членам (приоритет 20)

```
javascript

// Точечная нотация и квадратные скобки
let obj = { x: { y: 10 } };
let arr = [[1, 2], [3, 4]];

let value1 = obj.x.y;                   // 10 (слева направо)
let value2 = arr[0][1];                 // 2 (слева направо)

// Вызов функций
function getObject() {
  return { value: 42 };
}

let result = getObject().value;         // 42 (сначала вызов, потом доступ)

// Комбинированный доступ
let data = {
  users: [
    { name: 'Alice', scores: [85, 92, 78] }
  ]
};

let score = data.users[0].scores[1];    // 92 (слева направо)
```

2.3. Постфиксные и префиксные операторы (приоритет 18 и 17)

```
javascript

// Постфиксные (приоритет 18) - сначала используется значение, потом изменяется
let x = 5;
```

```

let y = x++;    // y = 5, потом x = 6

// Префиксные (приоритет 17) - сначала изменяется, потом используется
let a = 5;
let b = ++a;    // a = 6, потом b = 6

// Комбинация с другими операторами
let i = 3;
let result1 = i++ * 2;    // 6 (3*2), потом i = 4
let result2 = ++i * 2;    // 10 (5*2), i = 5

// Унарные операторы
let value = 10;
let negative = -value;    // -10
let positive = +"123";    // 123 (преобразование в число)
let logicalNot = !true;    // false
let bitwiseNot = ~5;    // -6 (инверсия битов)

```

2.4. Возведение в степень (приоритет 16)

```

javascript

// ** имеет правоассоциативность (справа налево)
let result1 = 2 ** 3 ** 2;    // 512 (2^(3^2) = 2^9 = 512)
let result2 = (2 ** 3) ** 2;    // 64 ((2^3)^2 = 8^2 = 64)

// Смешение с другими операторами
let calculation1 = 3 * 2 ** 3;    // 24 (сначала 2^3=8, потом *3)
let calculation2 = 2 ** 3 * 4;    // 32 (сначала 2^3=8, потом *4)
let calculation3 = -2 ** 2;    // -4 (сначала 2^2=4, потом унарный -)
let calculation4 = (-2) ** 2;    // 4 (скобки меняют порядок)

```

2.5. Мультипликативные и аддитивные операторы (приоритет 15 и 14)

```

javascript

// Мультипликативные (*, /, %) левоассоциативны
let result1 = 12 / 3 * 2;    // 8 (слева направо: 12/3=4, 4*2=8)
let result2 = 12 * 3 / 2;    // 18 (слева направо: 12*3=36, 36/2=18)
let result3 = 10 % 3 * 2;    // 2 (слева направо: 10%3=1, 1*2=2)

// Аддитивные (+, -) левоассоциативны
let calculation1 = 5 - 3 + 2;    // 4 (слева направо: 5-3=2, 2+2=4)

```

```
let calculation2 = 5 + 3 - 2;    // 6 (слева направо: 5+3=8, 8-2=6)
```

```
// Смешение приоритетов
```

```
let mixed1 = 2 + 3 * 4;          // 14 (сначала 3*4=12, потом +2)
```

```
let mixed2 = (2 + 3) * 4;        // 20 (скобки меняют порядок)
```

```
let mixed3 = 10 - 4 / 2;         // 8 (сначала 4/2=2, потом 10-2)
```

```
let mixed4 = (10 - 4) / 2;       // 3 (скобки меняют порядок)
```

2.6. Побитовые операторы (приоритет 13, 10, 9, 8)

```
javascript
```

```
// Все побитовые операторы левоассоциативны
```

```
let a = 5;    // 0101
```

```
let b = 3;    // 0011
```

```
let c = 1;    // 0001
```

```
// Приоритет: сдвиг (13) > & (10) > ^ (9) > | (8)
```

```
let result1 = a << 1 & b;    // 2 ((5<<1)=10(1010), 10&3=2)
```

```
let result2 = a & b << 1;    // 6 (3<<1=6, 5&6=4) - ОШИБКА в расчете!
```

```
// Правильно: 5 & (3 << 1) = 5 & 6 = 4
```

```
let result3 = a & b | c;     // 5 (5&3=1, 1|1=1)
```

```
let result4 = a | b & c;     // 5 (3&1=1, 5|1=5)
```

```
// Явное указание порядка
```

```
let correct1 = (a << 1) & b; // 2
```

```
let correct2 = a & (b << 1); // 4
```

```
let correct3 = (a & b) | c;   // 1
```

```
let correct4 = a | (b & c);   // 5
```

2.7. Операторы сравнения (приоритет 12 и 11)

```
javascript
```

```
// Все операторы сравнения левоассоциативны
```

```
let a = 5, b = 3, c = 7;
```

```
// Цепочки сравнений (работают как в математике)
```

```
let result1 = 1 < 2 < 3;    // true (слева направо: 1<2=true, true<3=1<3=true)
```

```
let result2 = 3 > 2 > 1;    // false (слева направо: 3>2=true, true>1=1>1=false)
```

```
// Правильное использование
```

```
let correct1 = 1 < 2 && 2 < 3; // true (явное логическое И)
```

```
let correct2 = 3 > 2 && 2 > 1; // true
```

```
// Смешение операторов
```

```
let mixed1 = 5 + 3 > 7;           // true (сначала 5+3=8, потом 8>7)
let mixed2 = 10 * 2 === 20;       // true (сначала 10*2=20, потом сравнение)
let mixed3 = "5" == 2 + 3;        // true (сначала 2+3=5, потом "5"==5)
```

2.8. Логические операторы (приоритет 7, 6, 5)

javascript

```
// Приоритет: && (7) > || (6) > ?? (5)
```

```
let a = false, b = true, c = false;
let value = null, defaultValue = "default", anotherValue = "another";
```

```
// Без скобок (по приоритету)
```

```
let result1 = a && b || c;         // true (сначала a&&b=false, потом false||c=false)
let result2 = a || b && c;         // false (сначала b&&c=false, потом a||false=false)
```

```
let result3 = value ?? defaultValue || anotherValue;
```

```
// Эквивалентно: value ?? (defaultValue || anotherValue)
// Результат: "default"
```

```
// С явными скобками
```

```
let explicit1 = (a && b) || c;     // false
let explicit2 = a || (b && c);     // false
```

```
let explicit3 = (value ?? defaultValue) || anotherValue;
```

```
// Результат: "default" (потому что value ?? defaultValue = "default")
```

```
// Смешение с другими операторами
```

```
let complex1 = 5 > 3 && 10 < 20 || 1 === 2; // true
// Эквивалентно: (5 > 3 && 10 < 20) || 1 === 2
// Сначала сравнения: true && true || false
// Потом &&: true || false
// Потом ||: true
```

2.9. Условный (тернарный) оператор (приоритет 4)

javascript

```
// Правоассоциативность (справа налево)
```

```
let a = 10, b = 20, c = 30;
```

```

// Цепочка тернарных операторов
let result1 = a > b ? "a больше" : b > c ? "b больше" : "c больше";
// Эквивалентно: a > b ? "a больше" : (b > c ? "b больше" : "c больше")
// Результат: "c больше"

// С явными скобками
let result2 = (a > b ? "a больше" : b > c) ? "b больше" : "c больше";
// Другой порядок! Сначала a>b? "a больше": b>c = false
// Потом false ? "b больше" : "c больше" = "c больше"

// Смешение с другими операторами
let age = 25;
let message = age >= 18
  ? age >= 65 ? "пенсионер" : "взрослый"
  : "ребенок";
// Результат: "взрослый"

let discount = age < 18 ? 0.5 : age >= 65 ? 0.3 : 0.1;
// Результат: 0.1

```

2.10. Операторы присваивания (приоритет 3)

```

javascript

// Правоассоциативность (справа налево)
let a, b, c;

// Цепочка присваиваний
a = b = c = 5;
// Эквивалентно: a = (b = (c = 5))
// Результат: a=5, b=5, c=5

// Сложные присваивания
let x = 10;
x += 5 * 2;    // x = 10 + (5 * 2) = 20
// Приоритет: * выше +=

let y = 8;
y *= 2 + 3;    // y = 8 * (2 + 3) = 40

// Комбинированные присваивания
let z = 2;
z **= 3;       // z = 2 ** 3 = 8

```

```
// Смешение разных типов присваивания
let p = 2, q = 3, r = 4;
p += q *= r;    // p = 2 + (3 * 4) = 14, q = 12
// Эквивалентно: p += (q *= r)
```

2.11. Оператор запятой (приоритет 1)

```
javascript

// Левоассоциативность (слева направо)
// Оператор запятой возвращает последнее значение

let a = (1, 2, 3);           // a = 3
let b = (x = 5, y = 10, x + y); // b = 15, x = 5, y = 10

// Использование в циклах for
for (let i = 0, j = 10; i < j; i++, j--) {
  console.log(i, j); // 0 10, 1 9, 2 8, 3 7, 4 6
}

// Несколько операций в одной строке
let x = 0, y = 0, z = 0; // Объявление нескольких переменных

// Возврат последнего значения
function example() {
  return (console.log('1'), console.log('2'), 'результат');
}
// Выведет: '1', '2', вернет 'результат'
```

Часть 3: Практические примеры и частые ошибки

3.1. Распространенные ошибки с приоритетом операторов

```
javascript

// ОШИБКА 1: Смешение && и ||
let isAdmin = false;
let isModerator = true;
let hasPermission = true;

// Неправильно (без скобок)
let result1 = isAdmin || isModerator && hasPermission;
```

```
// Ожидание: (isAdmin || isModerator) && hasPermission
// Реальность: isAdmin || (isModerator && hasPermission) = true

// Правильно (с явными скобками)
let result2 = (isAdmin || isModerator) && hasPermission; // true

// ОШИБКА 2: Возведение в степень
let base = 2;
let exponent = 3;

// Неправильно
let wrong = -base ** exponent; // -8 (интерпретируется как -(2**3))
// Правильно
let correct = (-base) ** exponent; // -8 (но уже осознанно)

// ОШИБКА 3: Смещение сравнений
let x = 5, y = 10, z = 15;

// Неправильно (цепочка сравнений)
let chain = 5 < 10 < 15; // true (но работает не так, как кажется)
// 5 < 10 = true, true < 15 = 1 < 15 = true

// Правильно (раздельные сравнения)
let separate = 5 < 10 && 10 < 15; // true

// ОШИБКА 4: Присваивание в условии
let value;

// Неправильно (случайное присваивание вместо сравнения)
if (value = getValue()) { // Присваивает, а не сравнивает!
  // ...
}

// Правильно
if (value === getValue()) {
  // ...
}
// Или осознанное присваивание с проверкой
if ((value = getValue()) !== null) {
  // ...
}
```


3.2. Сложные выражения и лучшие практики

javascript

// Пример 1: Вычисление формулы

```
function calculateDistance(x1, y1, x2, y2) {  
  // Формула:  $\sqrt{(x2-x1)^2 + (y2-y1)^2}$   
  // Без скобок это было бы неверно  
  return Math.sqrt(  
    Math.pow(x2 - x1, 2) +  
    Math.pow(y2 - y1, 2)  
  );  
}
```

// Пример 2: Проверка диапазона

```
function isInRange(value, min, max) {  
  // Правильно с явными скобками  
  return (value >= min) && (value <= max);  
  
  // Без скобок тоже работает (>= и <= имеют одинаковый приоритет),  
  // но явные скобки улучшают читаемость  
}
```

// Пример 3: Сложное логическое выражение

```
function canAccessContent(user, content) {  
  // Читаемо с явными скобками  
  return (  
    (user.isAdmin || user.isModerator) &&  
    (content.isPublished || user.canViewDrafts) &&  
    !user.isBanned &&  
    (!content.isAgeRestricted || user.age >= 18)  
  );  
}
```

// Пример 4: Вычисление скидки

```
function calculateDiscount(order, customer) {  
  let discount = 0;  
  
  // Сложное условие с явными скобками  
  if (  
    (order.total > 1000 && customer.isPremium) ||  
    (order.items > 10 && customer.loyaltyYears > 2)  
  ) {  
    discount = 0.2; // 20%
```

```

} else if (order.total > 500 || customer.isFirstOrder) {
  discount = 0.1; // 10%
}

// Дополнительная скидка за оплату онлайн
if (order.paymentMethod === 'online') {
  discount += 0.05; // +5%
}

// Ограничение максимальной скидки
return Math.min(discount, 0.3); // Не более 30%
}

```

3.3. Использование скобок для улучшения читаемости

```

javascript

// ПЛОХО: Неявный порядок
let result1 = a && b || c && d || e;

// ЛУЧШЕ: Явные скобки
let result2 = (a && b) || (c && d) || e;

// ЕЩЕ ЛУЧШЕ: Разделение на подвыражения
let condition1 = a && b;
let condition2 = c && d;
let result3 = condition1 || condition2 || e;

// ПЛОХО: Сложное математическое выражение
let calculation1 = a * b + c / d - e % f;

// ЛУЧШЕ: Явные скобки показывают намерение
let calculation2 = (a * b) + (c / d) - (e % f);

// ЕЩЕ ЛУЧШЕ: Разделение вычислений
let product = a * b;
let quotient = c / d;
let remainder = e % f;
let calculation3 = product + quotient - remainder;

// Пример из реального кода (валидация email)
function isValidEmail(email) {
  // Сложное регулярное выражение с явными скобками для группировки

```

```
const emailRegex = /^(([^<>()\[\]\\\.,;:\s@"]+(\.[^<>()\[\]\\\.,;:\s@"]+)*|("[.+""))@((\[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3})|([a-zA-Z\-0-9]+\.[a-zA-Z]{2,}))$/;

return emailRegex.test(email);
}
```

Часть 4: Памятка и быстрые правила

4.1. Быстрая памятка приоритетов (от высокого к низкому)

Text

ПРИОРИТЕТ	ОПЕРАТОРЫ	АССОЦИАТИВНОСТЬ
Высший	() [] .	слева направо
	new (с аргументами)	
	++ -- (постфиксные)	
	! ~ + - ++ --	справа налево
	typeof void delete	
	**	справа налево
	* / %	слева направо
	+ -	слева направо
	<< >> >>>	слева направо
	< <= > >= in instanceof	слева направо
	== != === !==	слева направо
	&	слева направо
	^	слева направо
		слева направо
	&&	слева направо
		слева направо
	??	слева направо
	?:	справа налево
	= += -= *= и др.	справа налево
	yield yield*	справа налево
Низший	,	слева направо

4.2. Правила на каждый день

1. **Всегда используйте скобки**, когда не уверены в приоритете
2. **&& всегда выполняется перед ||** (если нет скобок)
3. **Сравнения (==, !=, ===, !==) выполняются после арифметических операций**
4. **Присваивание (=) имеет один из самых низких приоритетов**
5. **Тернарный оператор (?) правоассоциативен** - цепочки выполняются справа налево

4.3. Практические советы

javascript

// СОВЕТ 1: При сложных выражениях используйте временные переменные

// Вместо:

```
let result = a * b + c / d - e * f + g / h;
```

// Лучше:

```
let term1 = a * b;
```

```
let term2 = c / d;
```

```
let term3 = e * f;
```

```
let term4 = g / h;
```

```
let result = term1 + term2 - term3 + term4;
```

// СОВЕТ 2: Разделяйте логические выражения

// Вместо:

```
if (user && user.isActive && user.subscription && user.subscription.isValid || isAdmin) {  
  // ...  
}
```

// Лучше:

```
const isActive = user && user.isActive;
```

```
const hasValidSubscription = user && user.subscription && user.subscription.isValid;
```

```
const canAccess = isActive && hasValidSubscription || isAdmin;
```

```
if (canAccess) {
```

```
  // ...  
}
```

// СОВЕТ 3: Комментируйте сложные выражения

// Сложное выражение с комментариями

```
let finalPrice = (  
  basePrice * quantity * (1 - discount) // Цена со скидкой  
  + shippingCost                       // Плюс доставка
```

```

- couponValue // Минус купон
) * (1 + taxRate); // Плюс налог

// COBET 4: Используйте ESLint для проверки
// .eslintrc.js
module.exports = {
  rules: {
    // Требуется скобки вокруг сложных выражений
    'no-mixed-operators': 'error',
    // Запрещает неочевидные приоритеты
    'no-unsafe-optional-chaining': 'error',
  }
};

```

Заключение: ключевые выводы

5.1. Что нужно запомнить

1. **Приоритет операторов определяет порядок выполнения** в отсутствие скобок
2. **Ассоциативность определяет порядок** операторов с одинаковым приоритетом
3. **Оператор `()` имеет наивысший приоритет** - используйте скобки для явного указания порядка
4. **Логическое И `&&` выполняется перед логическим ИЛИ `||`**
5. **Тернарный оператор `?:` правоассоциативен** - выполняет цепочки справа налево

5.2. Рекомендации для написания чистого кода

1. **Явные скобки лучше неявного приоритета** - делают код понятнее
2. **Разделяйте сложные выражения** на более простые части
3. **Используйте временные переменные** для промежуточных результатов
4. **Комментируйте нетривиальные выражения** - объясняйте свою логику
5. **Настраивайте линтеры** для автоматической проверки приоритетов

5.3. Финальный пример: хорошо структурированный код

```

javascript

// ПЛОХО: Сложное выражение без структуры

```

```

function calculateBad(a, b, c, d, e) {
  return a && b || c && !d ? e * 2 + 5 : e / 2 - 3;
}

// ХОРОШО: Структурированный и понятный код
function calculateGood(a, b, c, d, e) {
  // Шаг 1: Определить условие
  const isValidInput = a && b;
  const hasAlternative = c && !d;
  const shouldUseFormula = isValidInput || hasAlternative;

  // Шаг 2: Выбрать формулу на основе условия
  if (shouldUseFormula) {
    // Формула для истинного условия
    return (e * 2) + 5;
  } else {
    // Формула для ложного условия
    return (e / 2) - 3;
  }
}

// ЕЩЕ ЛУЧШЕ: С комментариями для сложной логики
function calculateBest(a, b, c, d, e) {
  /*
   * Условие выбора формулы:
   * - Истинно, если (a И b) ИЛИ (c И НЕ d)
   * - Ложно во всех остальных случаях
   */
  const condition = (a && b) || (c && !d);

  /*
   * Выбор формулы:
   * - Если условие истинно: 2e + 5
   * - Если условие ложно: e/2 - 3
   */
  return condition
    ? (2 * e) + 5 // Линейная формула
    : (e / 2) - 3; // Обратная формула
}

```

Итог: Понимание приоритета и ассоциативности операторов — это фундаментальный навык JavaScript-разработчика. Он позволяет не только правильно интерпретировать существующий код, но и писать понятные, надежные выражения. Когда сомневаетесь — используйте скобки. Когда

выражение становится сложным — разбивайте его на части. И помните: читаемый код всегда важнее "оптимизированного" за счет неявных приоритетов.

Глава 6: Управляющие конструкции

6.1. Условные выражения: if, else if, else

Введение: управление потоком выполнения

Условные выражения — это фундаментальный механизм управления потоком выполнения в программировании. Они позволяют программе принимать решения и выполнять различные блоки кода в зависимости от условий.

Аналогия из реальной жизни:

- Если идет дождь → взять зонт
- Иначе если пасмурно → взять куртку
- Иначе → взять солнечные очки

Часть 1: Базовый синтаксис и структура

1.1. Конструкция `if`

Базовый синтаксис:

```
javascript

if (условие) {
    // выполнить этот код, если условие истинно (true)
}
```

Простой пример:

```
javascript

// Пример 1: Проверка возраста
let age = 18;

if (age >= 18) {
    console.log("Доступ разрешен");
}
```



```
    console.log("Вы совершеннолетний");
}

// Пример 2: Проверка значения
let temperature = 25;

if (temperature > 30) {
    console.log("Жаркая погода");
    console.log("Рекомендуется пить больше воды");
}

// Пример 3: Проверка существования
let user = {
    name: "Анна",
    isActive: true
};

if (user.isActive) {
    console.log(`Пользователь ${user.name} активен`);
}
```

Подробное объяснение механизма работы:

```
javascript

// Шаг 1: Вычисление условия
let score = 85;
let passingScore = 60;

// Выражение в скобках вычисляется в булево значение
let condition = score >= passingScore; // true

// Шаг 2: Проверка истинности
if (condition) {
    // Этот блок выполняется только если condition === true
    console.log("Экзамен сдан!");
}

// Шаг 3: Преобразование типов в условиях
let value1 = 10;           // truthy (истинное значение)
let value2 = 0;            // falsy (ложное значение)
let value3 = "";           // falsy
let value4 = "текст";      // truthy
let value5 = null;         // falsy
let value6 = undefined;    // falsy
```

```
let value7 = [];           // truthy (массив, даже пустой)
let value8 = {};           // truthy (объект)

// JavaScript автоматически преобразует значения к boolean
if (value1) {
  console.log("value1 - truthy");
}

if (!value2) {
  console.log("value2 - falsy");
}
```

1.2. Полная конструкция `if...else`

Синтаксис:

```
javascript

if (условие) {
  // выполнить, если условие истинно
} else {
  // выполнить, если условие ложно
}
```

Детальные примеры:

```
javascript

// Пример 1: Проверка пароля
let enteredPassword = "secret123";
let correctPassword = "secret123";

if (enteredPassword === correctPassword) {
  console.log("Пароль верный! Доступ разрешен.");
  // Дополнительные действия при успешной авторизации
  logLoginAttempt(true);
  updateLastLogin();
} else {
  console.log("Неверный пароль! Попробуйте снова.");
  // Действия при неудачной попытке
  logLoginAttempt(false);
  incrementFailedAttempts();
}
```

```
// Пример 2: Проверка числа на четность
let number = 7;

if (number % 2 === 0) {
  console.log(`${number} - четное число`);
  // Математические операции для четных чисел
  let square = number * number;
  console.log(`Квадрат числа: ${square}`);
} else {
  console.log(`${number} - нечетное число`);
  // Математические операции для нечетных чисел
  let cube = number * number * number;
  console.log(`Куб числа: ${cube}`);
}

// Пример 3: Проверка роли пользователя
let userRole = "editor";
let permissions = [];

if (userRole === "admin") {
  console.log("Полный доступ ко всем функциям");
  permissions = ["read", "write", "delete", "manage_users"];
} else {
  console.log("Ограниченный доступ");
  permissions = ["read", "write"];
}

console.log("Доступные разрешения:", permissions);
```

1.3. Расширенная конструкция if...else if...else

Синтаксис:

```
javascript

if (условие1) {
  // выполнить, если условие1 истинно
} else if (условие2) {
  // выполнить, если условие1 ложно, но условие2 истинно
} else if (условие3) {
  // выполнить, если условие1 и условие2 ложны, но условие3 истинно
} else {
  // выполнить, если все условия ложны
```

```
}
```

Детальные примеры с пояснениями:

javascript

// Пример 1: Оценка успеваемости

```
function getGrade(score) {  
    let grade;  
  
    if (score >= 90) {  
        grade = "A";  
        console.log("Отличный результат!");  
    } else if (score >= 80) {  
        grade = "B";  
        console.log("Хороший результат!");  
    } else if (score >= 70) {  
        grade = "C";  
        console.log("Удовлетворительный результат");  
    } else if (score >= 60) {  
        grade = "D";  
        console.log("Минимальный проходной балл");  
    } else {  
        grade = "F";  
        console.log("Неудовлетворительно. Необходима пересдача.");  
    }  
  
    return grade;  
}
```

// Тестирование функции

```
console.log(getGrade(95)); // A  
console.log(getGrade(85)); // B  
console.log(getGrade(75)); // C  
console.log(getGrade(65)); // D  
console.log(getGrade(55)); // F
```

// Пример 2: Определение времени суток с дополнительной логикой

```
function getTimeOfDay(hour) {  
    let timeOfDay;  
    let message;  
  
    if (hour >= 5 && hour < 12) {  
        timeOfDay = "утро";  
        message = "Доброе утро! Время для завтрака и планирования дня.";
```

```

} else if (hour >= 12 && hour < 17) {
    timeOfDay = "день";
    message = "Добрый день! Пик продуктивности.";
} else if (hour >= 17 && hour < 22) {
    timeOfDay = "вечер";
    message = "Добрый вечер! Время для отдыха и ужина.";
} else if ((hour >= 22 && hour <= 23) || (hour >= 0 && hour < 5)) {
    timeOfDay = "ночь";
    message = "Доброй ночи! Время для сна.";
} else {
    timeOfDay = "неопределено";
    message = "Некорректное время. Введите значение от 0 до 23.";
}

return {
    timeOfDay,
    message,
    isValid: timeOfDay !== "неопределено"
};
}

```

// Тестирование

```

console.log(getTimeOfDay(8)); // { timeOfDay: 'утро', ... }
console.log(getTimeOfDay(14)); // { timeOfDay: 'день', ... }
console.log(getTimeOfDay(20)); // { timeOfDay: 'вечер', ... }
console.log(getTimeOfDay(2)); // { timeOfDay: 'ночь', ... }
console.log(getTimeOfDay(25)); // { timeOfDay: 'неопределено', ... }

```

Часть 2: Особенности и нюансы синтаксиса

2.1. Фигурные скобки: обязательные или опциональные?

Без фигурных скобок (только для одной инструкции):

javascript

// Работает, но не рекомендуется

```

let temperature = 30;

if (temperature > 25)
    console.log("Жарко"); // Одна инструкция - работает

```

```
if (temperature > 25)
  console.log("Жарко"); // Первая инструкция
  console.log("Оденьтесь легче"); // ВСЕГДА выполнится, не часть if!
```

// Что на самом деле происходит:

```
if (temperature > 25) {
  console.log("Жарко");
}
console.log("Оденьтесь легче"); // Всегда выполняется!
```

С фигурными скобками (рекомендуется всегда):

javascript

// Всегда используйте фигурные скобки!

```
let temperature = 30;

if (temperature > 25) {
  console.log("Жарко");
  console.log("Оденьтесь легче"); // Выполнится только если условие true
}
```

// Даже для одной инструкции

```
if (temperature > 25) {
  console.log("Жарко");
}
```

// Несколько инструкций - очевидная необходимость скобок

```
if (temperature > 25) {
  console.log("Жарко");
  alert("Включите кондиционер");
  document.body.classList.add("hot-weather");
}
```

2.2. Вложенные условия (nested if)

Примеры вложенных условий:

javascript

// Пример 1: Проверка доступа к контенту

```
function checkContentAccess(user, content) {
  if (user.isLoggedIn) {
    console.log("Пользователь авторизован");
  }
}
```

```
if (user.age >= 18) {
  console.log("Пользователь совершеннолетний");

  if (content.isFree || user.hasSubscription) {
    console.log("Доступ к контенту разрешен");
    return true;
  } else {
    console.log("Контент платный, нужна подписка");
    return false;
  }
} else {
  console.log("Контент только для взрослых");
  return false;
}
} else {
  console.log("Требуется авторизация");
  return false;
}
}
```

// Пример 2: Сложная бизнес-логика заказа

```
function processOrder(order) {
  if (order.status === "pending") {
    console.log("Заказ ожидает обработки");

    if (order.items.length > 0) {
      console.log("В заказе есть товары");

      if (order.paymentMethod === "credit_card") {
        console.log("Обработка кредитной карты");

        if (order.amount > 1000) {
          console.log("Крупный заказ, требуется проверка");
          return "needs_verification";
        } else {
          console.log("Стандартная обработка");
          return "processing";
        }
      } else if (order.paymentMethod === "paypal") {
        console.log("Обработка PayPal");
        return "processing";
      } else {
        console.log("Неизвестный метод оплаты");
      }
    }
  }
}
```

```

        return "failed";
    }
} else {
    console.log("Заказ пуст");
    return "cancelled";
}
} else {
    console.log(`Заказ уже обработан: ${order.status}`);
    return order.status;
}
}

```

// Альтернатива с guard clauses (охраняющие условия)

```

function processOrderOptimized(order) {
    // Guard clauses для раннего возврата
    if (order.status !== "pending") {
        console.log(`Заказ уже обработан: ${order.status}`);
        return order.status;
    }

    if (order.items.length === 0) {
        console.log("Заказ пуст");
        return "cancelled";
    }

    if (order.paymentMethod === "credit_card") {
        console.log("Обработка кредитной карты");
        return order.amount > 1000 ? "needs_verification" : "processing";
    }

    if (order.paymentMethod === "paypal") {
        console.log("Обработка PayPal");
        return "processing";
    }

    console.log("Неизвестный метод оплаты");
    return "failed";
}

```


2.3. Truthy и Falsy значения в условиях

Полный список falsy значений:

```
javascript

// Все эти значения преобразуются в false
if (false) {}           // false
if (0) {}               // ноль
if (-0) {}             // отрицательный ноль
if (0n) {}             // BigInt ноль
if ("") {}             // пустая строка
if (null) {}           // null
if (undefined) {}      // undefined
if (NaN) {}            // Not a Number

// Все остальные значения - truthy
if (true) {}           // true
if (1) {}              // любое ненулевое число
if (-1) {}             // отрицательные числа тоже truthy
if ("текст") {}        // непустая строка
if (" ") {}            // даже пробел - truthy
if ("0") {}            // строка "0" - truthy!
if ([]) {}             // пустой массив - truthy!
if ({}) {}             // пустой объект - truthy!
if (function() {}) {}  // функция - truthy
```

Практическое использование truthy/falsy:

```
javascript

// Пример 1: Проверка наличия значения
function getUsername(user) {
  // Плохо: избыточные проверки
  if (user !== undefined && user !== null && user.name !== undefined) {
    return user.name;
  }

  // Хорошо: используем truthy/falsy
  if (user && user.name) {
    return user.name;
  }

  // Еще лучше: современный JavaScript
  return user?.name || "Аноним";
}
```

```
}
```

```
// Пример 2: Значения по умолчанию
```

```
function createGreeting(name) {  
  // Если name - falsy, используем "Гость"  
  let userName = name || "Гость";  
  return `Привет, ${userName}!`;  
}
```

```
console.log(createGreeting("Анна")); // Привет, Анна!  
console.log(createGreeting(""));    // Привет, Гость!  
console.log(createGreeting(null));  // Привет, Гость!  
console.log(createGreeting(0));      // Привет, Гость! (Осторожно!)
```

```
// Пример 3: Безопасный доступ к свойствам
```

```
function getOrderTotal(order) {  
  // Проверяем цепочку свойств  
  if (order && order.items && order.items.length > 0) {  
    let total = 0;  
    for (let item of order.items) {  
      if (item && item.price && item.quantity) {  
        total += item.price * item.quantity;  
      }  
    }  
    return total;  
  }  
  return 0;  
}
```

```
// Современная альтернатива с optional chaining
```

```
function getOrderTotalModern(order) {  
  if (order?.items?.length > 0) {  
    let total = 0;  
    for (let item of order.items) {  
      total += (item?.price || 0) * (item?.quantity || 0);  
    }  
    return total;  
  }  
  return 0;  
}
```

Часть 3: Сложные условия и логические операторы

3.1. Комбинирование условий с логическими операторами

Логическое И (&&):

```
javascript

// Пример 1: Проверка нескольких условий
let age = 25;
let hasLicense = true;
let hasInsurance = true;

if (age >= 18 && hasLicense && hasInsurance) {
  console.log("Можно арендовать автомобиль");
  // Дополнительные действия при выполнении всех условий
  calculateRentalPrice();
  generateContract();
} else {
  console.log("Нельзя арендовать автомобиль");
  // Определяем, какое условие не выполнено
  if (age < 18) {
    console.log("Причина: возраст менее 18 лет");
  }
  if (!hasLicense) {
    console.log("Причина: нет водительских прав");
  }
  if (!hasInsurance) {
    console.log("Причина: нет страховки");
  }
}

// Пример 2: Проверка диапазона
let score = 75;

if (score >= 0 && score <= 100) {
  console.log("Оценка в допустимом диапазоне");

  if (score >= 90 && score <= 100) {
    console.log("Отлично!");
  } else if (score >= 70 && score < 90) {
    console.log("Хорошо!");
  }
}
```

```

    } else if (score >= 50 && score < 70) {
        console.log("Удовлетворительно");
    } else {
        console.log("Неудовлетворительно");
    }
} else {
    console.log("Ошибка: оценка вне диапазона 0-100");
}

// Пример 3: Короткое замыкание (short-circuit evaluation)
let user = {
    name: "Иван",
    preferences: {
        theme: "dark"
    }
};

// && останавливается на первом falsy значении
if (user && user.preferences && user.preferences.theme) {
    console.log(`Тема: ${user.preferences.theme}`);
}

// Современная запись с optional chaining
if (user?.preferences?.theme) {
    console.log(`Тема: ${user.preferences.theme}`);
}

```

Логическое ИЛИ (||):

```

javascript

// Пример 1: Разрешение доступа по нескольким условиям
function checkAccess(user) {
    let isAdmin = user.role === "admin";
    let isModerator = user.role === "moderator";
    let isOwner = user.isOwner === true;

    if (isAdmin || isModerator || isOwner) {
        console.log("Доступ разрешен");
        return true;
    } else {
        console.log("Доступ запрещен");
        return false;
    }
}

```

// Пример 2: Проверка поддерживаемых браузеров

```
function checkBrowserSupport() {  
    let userAgent = navigator.userAgent;  
    let isChrome = userAgent.includes("Chrome");  
    let isFirefox = userAgent.includes("Firefox");  
    let isSafari = userAgent.includes("Safari") && !userAgent.includes("Chrome");  
    let isEdge = userAgent.includes("Edg");  
  
    if (isChrome || isFirefox || isSafari || isEdge) {  
        console.log("Браузер поддерживается");  
        return true;  
    } else {  
        console.log("Браузер не поддерживается. Рекомендуем обновить.");  
        showBrowserWarning();  
        return false;  
    }  
}
```

// Пример 3: Значения по умолчанию с ||

```
function getConfigValue(config, key) {  
    // Если config[key] существует и не falsy, вернуть его  
    // Иначе вернуть значение по умолчанию  
    return config[key] || "default_value";  
}
```

// Но осторожно с нулем!

```
let settings = { timeout: 0 };  
let timeout = settings.timeout || 3000; // 3000, а не 0!  
// Для таких случаев используйте ??  
let timeoutCorrect = settings.timeout ?? 3000; // 0
```

Логическое НЕ (!):

javascript

// Пример 1: Инверсия условия

```
let isLoggedOut = !isLoggedIn;  
  
if (!isLoggedIn) {  
    console.log("Пожалуйста, войдите в систему");  
    showLoginForm();  
} else {  
    console.log("Вы уже вошли в систему");  
    showUserProfile();  
}
```

```
}
```

```
// Пример 2: Проверка на пустоту
```

```
function validateForm(formData) {  
  let errors = [];  
  
  // Проверка, что поля не пустые  
  if (!formData.username) {  
    errors.push("Имя пользователя обязательно");  
  }  
  
  if (!formData.email) {  
    errors.push("Email обязателен");  
  }  
  
  if (!formData.password) {  
    errors.push("Пароль обязателен");  
  } else if (!formData.confirmPassword) {  
    errors.push("Подтверждение пароля обязательно");  
  } else if (formData.password !== formData.confirmPassword) {  
    errors.push("Пароли не совпадают");  
  }  
  
  if (errors.length === 0) {  
    console.log("Форма валидна");  
    return { success: true };  
  } else {  
    console.log("Ошибки валидации:", errors);  
    return { success: false, errors };  
  }  
}
```

```
// Пример 3: Двойное отрицание (!! ) для приведения к boolean
```

```
function toBoolean(value) {  
  return !!value;  
}
```

```
console.log(toBoolean(0));    // false  
console.log(toBoolean(1));    // true  
console.log(toBoolean(""));   // false  
console.log(toBoolean("text")); // true  
console.log(toBoolean(null)); // false  
console.log(toBoolean({}));   // true
```

3.2. Сложные комбинированные условия

javascript

// Пример 1: Сложная проверка заказа

```
function canProcessOrder(order) {  
    // Условие: заказ должен быть активным И  
    // (оплачен ИЛИ подтвержден менеджером) И  
    // не отменен И не возвращен  
    return (  
        order.status === "active" &&  
        (order.isPaid || order.isConfirmedByManager) &&  
        !order.isCancelled &&  
        !order.isReturned &&  
        order.items.length > 0  
    );  
}
```

// Пример 2: Валидация пароля

```
function validatePassword(password) {  
    let errors = [];  
  
    // Сложное условие с проверкой всех требований  
    if (!password || password.length < 8) {  
        errors.push("Пароль должен быть не менее 8 символов");  
    }  
  
    if (!/[A-Z]/.test(password)) {  
        errors.push("Пароль должен содержать хотя бы одну заглавную букву");  
    }  
  
    if (!/[a-z]/.test(password)) {  
        errors.push("Пароль должен содержать хотя бы одну строчную букву");  
    }  
  
    if (!/\d/.test(password)) {  
        errors.push("Пароль должен содержать хотя бы одну цифру");  
    }  
  
    if (!/[!@#%&*]/.test(password)) {  
        errors.push("Пароль должен содержать хотя бы один специальный символ");  
    }  
  
    return errors.length === 0 ? { valid: true } : { valid: false, errors };  
}
```

```
}
```

```
// Пример 3: Условия с приоритетами операторов
```

```
let a = true, b = false, c = true, d = false;
```

```
// Важно понимать приоритет: && выполняется перед ||
```

```
let result1 = a || b && c;      // true || (false && true) = true
```

```
let result2 = (a || b) && c;    // (true || false) && true = true
```

```
let result3 = a && b || c;      // (true && false) || true = true
```

```
let result4 = a && (b || c);    // true && (false || true) = true
```

```
// Практический пример
```

```
function checkPermissions(user, action) {  
  return (  
    user.isAdmin ||  
    (user.isModerator && action !== "delete") ||  
    (user.isEditor && action === "edit")  
  );  
}
```

Часть 4: Практические паттерны и лучшие практики

4.1. Паттерн "Guard Clauses" (охраняющие условия)

```
javascript
```

```
// Стандартный подход (глубоко вложенные условия)
```

```
function processOrderOld(order) {  
  if (order) {  
    if (order.status === "pending") {  
      if (order.items.length > 0) {  
        if (order.paymentMethod) {  
          // Основная логика здесь (4 уровня вложенности)  
          console.log("Обработка заказа...");  
          return "success";  
        } else {  
          return "no_payment_method";  
        }  
      } else {  
        return "empty_order";  
      }  
    } else {  
      return "empty_order";  
    }  
  } else {  
    return "empty_order";  
  }  
}
```



```
        return "not_pending";
    }
} else {
    return "invalid_order";
}
}
```

// Подход с guard clauses (охраняющие условия)

```
function processOrderNew(order) {
    // Guard clause 1: Проверка существования заказа
    if (!order) {
        console.log("Неверный заказ");
        return "invalid_order";
    }

    // Guard clause 2: Проверка статуса
    if (order.status !== "pending") {
        console.log(`Заказ не в статусе pending: ${order.status}`);
        return "not_pending";
    }

    // Guard clause 3: Проверка наличия товаров
    if (!order.items || order.items.length === 0) {
        console.log("Заказ пуст");
        return "empty_order";
    }

    // Guard clause 4: Проверка метода оплаты
    if (!order.paymentMethod) {
        console.log("Не указан метод оплаты");
        return "no_payment_method";
    }

    // Основная логика (без вложенности!)
    console.log("Обработка заказа...");
    // ... основная логика обработки ...

    return "success";
}
```

// Еще пример: валидация пользователя

```
function validateUser(user) {
    // Охраняющие условия для раннего возврата
    if (!user) {
```

```

    return { valid: false, error: "User object is required" };
}

if (!user.username || user.username.trim() === "") {
    return { valid: false, error: "Username is required" };
}

if (!user.email || !isValidEmail(user.email)) {
    return { valid: false, error: "Valid email is required" };
}

if (!user.password || user.password.length < 8) {
    return { valid: false, error: "Password must be at least 8 characters" };
}

// Все проверки пройдены
return { valid: true, user: sanitizeUser(user) };
}

```

4.2. Паттерн "Early Return" (ранний возврат)

javascript

```

// Плохо: сложная вложенная логика
function calculateDiscount(customer, order) {
    let discount = 0;

    if (customer) {
        if (customer.isPremium) {
            if (order.total > 1000) {
                discount = 0.2;
            } else {
                discount = 0.1;
            }
        } else {
            if (customer.loyaltyYears > 5) {
                if (order.total > 500) {
                    discount = 0.15;
                } else {
                    discount = 0.05;
                }
            } else {
                if (order.total > 500) {
                    discount = 0.1;
                }
            }
        }
    }
}

```

```

    }
  }
}

return discount;
}

// Хорошо: ранний возврат и плоская структура
function calculateDiscountBetter(customer, order) {
  // Ранний возврат для граничных случаев
  if (!customer || !order || order.total <= 0) {
    return 0;
  }

  // Премиум клиенты
  if (customer.isPremium) {
    return order.total > 1000 ? 0.2 : 0.1;
  }

  // Лояльные клиенты
  if (customer.loyaltyYears > 5) {
    return order.total > 500 ? 0.15 : 0.05;
  }

  // Обычные клиенты
  if (order.total > 500) {
    return 0.1;
  }

  // Без скидки
  return 0;
}

// Пример с обработкой ошибок
function fetchUserData(userId) {
  // Ранняя проверка параметров
  if (!userId || typeof userId !== 'string') {
    console.error('Invalid user ID');
    return null;
  }

  // Ранняя проверка состояния
  if (!navigator.onLine) {

```

```

    console.error('No internet connection');
    return { cached: true, data: getCachedUserData(userId) };
}

try {
    // Основная логика
    const response = await fetch(`/api/users/${userId}`);

    if (!response.ok) {
        // Ранний возврат при ошибке HTTP
        console.error(`HTTP error: ${response.status}`);
        return null;
    }

    const data = await response.json();
    return data;

} catch (error) {
    // Ранний возврат при исключении
    console.error('Fetch failed:', error);
    return null;
}
}

```

4.3. Паттерн "Тернарный оператор для простых условий"

```

javascript

// Простые условия можно записать компактно
let age = 20;

// Длинная запись
let status;
if (age >= 18) {
    status = "взрослый";
} else {
    status = "ребенок";
}

// Компактная запись с тернарным оператором
let statusCompact = age >= 18 ? "взрослый" : "ребенок";

// Практические примеры
// Пример 1: Определение максимального значения

```

```

let a = 10, b = 20;
let max = a > b ? a : b;

// Пример 2: Форматирование сообщения
let score = 85;
let message = score >= 70 ? "Сдано" : "Не сдано";

// Пример 3: Установка класса CSS
let isActive = true;
let className = "button " + (isActive ? "active" : "inactive");

// Пример 4: Вложенные тернарные операторы (осторожно!)
let grade = score >= 90 ? "A" :
    score >= 80 ? "B" :
    score >= 70 ? "C" :
    score >= 60 ? "D" : "F";

// Но лучше для сложных условий использовать if/else
function getGrade(score) {
    if (score >= 90) return "A";
    if (score >= 80) return "B";
    if (score >= 70) return "C";
    if (score >= 60) return "D";
    return "F";
}

```

Часть 5: Распространенные ошибки и как их избежать

5.1. Частые ошибки с операторами сравнения

```

javascript

// ОШИБКА 1: Использование присваивания (=) вместо сравнения (===)
let value = 5;

// НЕПРАВИЛЬНО (присваивание)
if (value = 10) { // Всегда истинно, присваивает 10 в value
    console.log("Значение равно 10");
}

// ПРАВИЛЬНО (сравнение)
if (value === 10) {

```

```
    console.log("Значение равно 10");
}

// ОШИБКА 2: Неявное преобразование типов с ==
let number = 5;
let string = "5";

// НЕПРАВИЛЬНО (нестрогое сравнение)
if (number == string) { // true (5 == "5")
    console.log("Равны");
}

// ПРАВИЛЬНО (строгое сравнение)
if (number === string) { // false (number !== string)
    console.log("Не равны");
}

// ОШИБКА 3: Проверка массива или объекта
let array = [];
let object = {};

// НЕПРАВИЛЬНО (всегда истинно)
if (array) { // true (массивы всегда truthy)
    console.log("Массив существует");
}

// ПРАВИЛЬНО (проверка длины или свойств)
if (Array.isArray(array) && array.length > 0) {
    console.log("Массив не пустой");
}

// ОШИБКА 4: Цепочки сравнений как в математике
let x = 5, y = 10, z = 15;

// НЕПРАВИЛЬНО (работает не так, как ожидается)
if (5 < 10 < 15) { // true < 15 → 1 < 15 → true
    console.log("Математически верно, но в JS это ошибка");
}

// ПРАВИЛЬНО
if (5 < 10 && 10 < 15) {
    console.log("Правильная проверка диапазона");
}
```

5.2. Оптимизация условий для производительности

javascript

// ПЛОХО: Дорогие операции в условиях

```
function processData(data) {  
    // complexOperation() выполняется ВСЕГДА, даже если условие ложно  
    if (data && complexOperation(data)) {  
        return "success";  
    }  
    return "failure";  
}
```

// ХОРОШО: Проверка дешевых условий первыми

```
function processDataOptimized(data) {  
    // Дешевая проверка выполняется первой  
    if (!data) {  
        return "failure";  
    }  
  
    // Дорогая операция только если нужно  
    if (complexOperation(data)) {  
        return "success";  
    }  
  
    return "failure";  
}
```

// Пример с несколькими условиями

```
function checkUserAccess(user, resource) {  
    // Порядок имеет значение! Проверяем дешевые условия первыми  
  
    // 1. Дешевая проверка: существует ли пользователь  
    if (!user) {  
        return false;  
    }  
  
    // 2. Дешевая проверка: активен ли пользователь  
    if (!user.isActive) {  
        return false;  
    }  
  
    // 3. Дешевая проверка: базовая роль  
    if (user.role === "guest") {
```

```

    return false;
}

// 4. Дешевая проверка: время доступа
if (isOutsideWorkingHours()) {
    return false;
}

// 5. Дорогая проверка: права в базе данных
if (!checkDatabasePermissions(user.id, resource.id)) {
    return false;
}

// 6. Дорогая проверка: квоты использования
if (hasExceededQuota(user.id)) {
    return false;
}

return true;
}

// Использование логического И с коротким замыканием
function getUserPreferences(user) {
    // Если user не существует, остальное не вычисляется
    return user && user.preferences && user.preferences.theme;
}

// Кэширование результатов дорогих операций
function shouldProcessItem(item) {
    // Кэшируем результат дорогой операции
    let isValid = item.isValidCache;

    if (isValid === undefined) {
        // Выполняем дорогую проверку только один раз
        isValid = performExpensiveValidation(item);
        item.isValidCache = isValid;
    }

    return isValid && meetsOtherCriteria(item);
}

```


5.3. Читаемость и поддерживаемость условий

javascript

// ПЛОХО: Сложные условия в одной строке

```
if (user && user.isActive && (user.role === 'admin' || user.role === 'moderator') && !
user.isSuspended && (user.lastLogin > Date.now() - 86400000 || user.hasOfflineAccess)) {
  // ...
}
```

// ХОРОШО: Разделение на понятные переменные

```
function canAccessSystem(user) {
  // Выносим части условий в переменные с понятными именами
  const userExists = user != null;
  const isActive = user?.isActive === true;
  const hasAdminRole = user?.role === 'admin';
  const hasModeratorRole = user?.role === 'moderator';
  const isNotSuspended = user?.isSuspended !== true;
  const recentlyLoggedIn = user?.lastLogin > Date.now() - 86400000;
  const hasOfflineAccess = user?.hasOfflineAccess === true;

  // Основное условие становится читаемым
  const canAccess = userExists &&
    isActive &&
    (hasAdminRole || hasModeratorRole) &&
    isNotSuspended &&
    (recentlyLoggedIn || hasOfflineAccess);

  return canAccess;
}
```

// ЕЩЕ ЛУЧШЕ: Вынос логики в отдельные функции

```
function canAccessSystemRefactored(user) {
  if (!isValidUser(user)) return false;
  if (!isActive(user)) return false;
  if (!hasRequiredRole(user)) return false;
  if (isUserSuspended(user)) return false;
  if (!hasValidSession(user)) return false;

  return true;
}
```

// Вспомогательные функции

```
function isValidUser(user) {
```

```

    return user != null && typeof user === 'object';
}

function isUserActive(user) {
    return user.isActive === true;
}

function hasRequiredRole(user) {
    const allowedRoles = ['admin', 'moderator'];
    return allowedRoles.includes(user.role);
}

function isUserSuspended(user) {
    return user.isSuspended === true;
}

function hasValidSession(user) {
    const oneDayAgo = Date.now() - 86400000;
    return user.lastLogin > oneDayAgo || user.hasOfflineAccess;
}

```

Часть 6: Продвинутые техники и реальные примеры

6.1. Условия в функциональном стиле

```

javascript

// Использование функций высшего порядка с условиями
const users = [
    { id: 1, name: "Анна", age: 25, isActive: true },
    { id: 2, name: "Борис", age: 17, isActive: true },
    { id: 3, name: "Виктор", age: 30, isActive: false },
    { id: 4, name: "Галина", age: 22, isActive: true }
];

// Фильтрация с условиями
const activeUsers = users.filter(user => user.isActive);
const adultUsers = users.filter(user => user.age >= 18);
const activeAdults = users.filter(user => user.isActive && user.age >= 18);

// Поиск с условиями
const firstActiveUser = users.find(user => user.isActive);

```

```
const userByName = users.find(user => user.name === "Виктор");
```

```
// Преобразование с условиями
```

```
const userStatuses = users.map(user => {  
  if (!user.isActive) {  
    return { ...user, status: "неактивен" };  
  }  
  
  if (user.age < 18) {  
    return { ...user, status: "несовершеннолетний" };  
  }  
  
  return { ...user, status: "активен" };  
});
```

```
// Редукция с условиями
```

```
const ageStatistics = users.reduce((stats, user) => {  
  if (user.age >= 18) {  
    stats.adults++;  
  } else {  
    stats.minors++;  
  }  
  
  if (user.isActive) {  
    stats.active++;  
  } else {  
    stats.inactive++;  
  }  
  
  return stats;  
}, { adults: 0, minors: 0, active: 0, inactive: 0 });
```

6.2. Условия в асинхронном коде

```
javascript
```

```
// Условия с промисами
```

```
async function fetchData(url) {  
  try {  
    const response = await fetch(url);  
  
    // Проверка HTTP статуса  
    if (!response.ok) {  
      if (response.status === 404) {
```

```

        throw new Error("Ресурс не найден");
    } else if (response.status === 401) {
        throw new Error("Требуется авторизация");
    } else if (response.status === 500) {
        throw new Error("Ошибка сервера");
    } else {
        throw new Error(`HTTP ошибка: ${response.status}`);
    }
}

const data = await response.json();

// Проверка структуры данных
if (!data || typeof data !== 'object') {
    throw new Error("Неверный формат данных");
}

if (data.error) {
    throw new Error(data.error);
}

return data;

} catch (error) {
    // Обработка разных типов ошибок
    if (error.name === 'TypeError' && error.message.includes('fetch')) {
        console.error("Ошибка сети:", error);
        return { cached: true, data: getCachedData() };
    }

    console.error("Ошибка при загрузке данных:", error);
    throw error;
}

}

// Условия с таймаутами
async function fetchWithTimeout(url, timeout = 5000) {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), timeout);

    try {
        const response = await fetch(url, { signal: controller.signal });
        clearTimeout(timeoutId);
    }

```

```

    if (response.ok) {
        return await response.json();
    } else {
        throw new Error(`HTTP ${response.status}`);
    }
}

} catch (error) {
    clearTimeout(timeoutId);

    if (error.name === 'AbortError') {
        throw new Error(`Запрос превысил таймаут ${timeout}ms`);
    }

    throw error;
}
}

// Последовательные условия с async/await
async function processUserOrder(userId, orderId) {
    // 1. Проверка существования пользователя
    const user = await getUser(userId);
    if (!user) {
        throw new Error("Пользователь не найден");
    }

    // 2. Проверка активности пользователя
    if (!user.isActive) {
        throw new Error("Пользователь неактивен");
    }

    // 3. Получение заказа
    const order = await getOrder(orderId);
    if (!order) {
        throw new Error("Заказ не найден");
    }

    // 4. Проверка владельца заказа
    if (order.userId !== userId) {
        throw new Error("Заказ принадлежит другому пользователю");
    }

    // 5. Проверка статуса заказа
    if (order.status !== "pending") {
        throw new Error(`Заказ уже обработан: ${order.status}`);
    }
}

```

```

}

// 6. Основная логика обработки
return await processOrder(order);
}

```

6.3. Реальный пример: система аутентификации

```

javascript

class AuthService {
  async login(credentials) {
    // Валидация входных данных
    if (!credentials || typeof credentials !== 'object') {
      return this.errorResponse("Неверные данные для входа");
    }

    const { email, password } = credentials;

    if (!email || !this.isValidEmail(email)) {
      return this.errorResponse("Неверный формат email");
    }

    if (!password || password.length < 6) {
      return this.errorResponse("Пароль должен содержать минимум 6 символов");
    }

    try {
      // Проверка существования пользователя
      const user = await this.findUserByEmail(email);

      if (!user) {
        return this.errorResponse("Пользователь не найден");
      }

      // Проверка блокировки аккаунта
      if (user.isLocked) {
        const remainingTime = this.getRemainingLockTime(user);
        return this.errorResponse(
          `Аккаунт заблокирован. Попробуйте через ${remainingTime} минут`
        );
      }

      // Проверка пароля

```

```

const isPasswordValid = await this.verifyPassword(password, user.passwordHash);

if (!isPasswordValid) {
  // Увеличиваем счетчик неудачных попыток
  await this.incrementFailedAttempts(user);

  if (user.failedAttempts >= 5) {
    await this.lockAccount(user);
    return this.errorResponse("Слишком много попыток. Аккаунт заблокирован на 30 минут");
  }

  return this.errorResponse("Неверный пароль");
}

// Сброс счетчика неудачных попыток при успешном входе
await this.resetFailedAttempts(user);

// Проверка необходимости смены пароля
if (this.isPasswordExpired(user)) {
  return this.successResponse(
    { requiresPasswordChange: true, user: this.sanitizeUser(user) },
    "Требуется смена пароля"
  );
}

// Генерация токена
const token = this.generateToken(user);

// Обновление времени последнего входа
await this.updateLastLogin(user);

return this.successResponse(
  { token, user: this.sanitizeUser(user) },
  "Вход выполнен успешно"
);
} catch (error) {
  console.error("Ошибка при входе:", error);
  return this.errorResponse("Внутренняя ошибка сервера");
}
}

// Вспомогательные методы

```

```

errorResponse(message) {
    return { success: false, message };
}

successResponse(data, message = "Успешно") {
    return { success: true, message, data };
}

isValidEmail(email) {
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return emailRegex.test(email);
}

// ... остальные вспомогательные методы
}

```

Заключение: ключевые принципы эффективного использования условий

7.1. Основные выводы

1. **Всегда используйте фигурные скобки** — даже для одиночных инструкций
2. **Предпочитайте строгое сравнение (`===`, `!==`)** — избегайте неявного преобразования типов
3. **Используйте guard clauses** — для плоской структуры и раннего возврата
4. **Выносите сложные условия в переменные** — с понятными именами
5. **Проверяйте дешевые условия первыми** — для оптимизации производительности

7.2. Чек-лист для написания качественных условий

javascript

```

// ПЛОХИЕ ПРАКТИКИ (чего избегать):
// × if (value = 5) - присваивание вместо сравнения
// × if (value == "5") - нестрогое сравнение
// × Глубоко вложенные условия (более 3 уровней)
// × Слишком длинные условия в одной строке
// × Повторяющиеся условия в коде

```

```

// ХОРОШИЕ ПРАКТИКИ (что применять):

```



```
//  if (value === 5) - строгое сравнение
//  Ранний возврат при ошибках
//  Вынесение условий в переменные с понятными именами
//  Разделение сложной логики на небольшие функции
//  Комментарии для неочевидных условий
```

7.3. Финальный пример: хорошо структурированный код с условиями

javascript

```
// Пример обработки платежа с хорошей структурой условий
class PaymentProcessor {
  async processPayment(paymentData) {
    // 1. Валидация входных данных (guard clauses)
    const validationResult = this.validatePaymentData(paymentData);
    if (!validationResult.isValid) {
      return this.createErrorResponse(validationResult.errors);
    }

    // 2. Проверка возможности обработки
    if (!this.canProcessPayment(paymentData)) {
      return this.createErrorResponse(["Платежная система временно недоступна"]);
    }

    try {
      // 3. Основная логика с четкими условиями
      let paymentResult;

      if (paymentData.method === 'credit_card') {
        paymentResult = await this.processCreditCard(paymentData);
      } else if (paymentData.method === 'paypal') {
        paymentResult = await this.processPayPal(paymentData);
      } else if (paymentData.method === 'bank_transfer') {
        paymentResult = await this.processBankTransfer(paymentData);
      } else {
        return this.createErrorResponse(["Неподдерживаемый метод оплаты"]);
      }

      // 4. Обработка результата
      if (paymentResult.success) {
        await this.finalizePayment(paymentData, paymentResult);
        return this.createSuccessResponse(paymentResult);
      } else {

```

```

        await this.handleFailedPayment(paymentData, paymentResult);
        return this.createErrorResponse(paymentResult.errors);
    }

} catch (error) {
    // 5. Обработка исключений
    console.error("Ошибка обработки платежа:", error);
    await this.logPaymentError(paymentData, error);
    return this.createErrorResponse(["Внутренняя ошибка обработки платежа"]);
}
}

// Вспомогательные методы с четкими условиями
validatePaymentData(paymentData) {
    const errors = [];

    // Проверка обязательных полей
    if (!paymentData || typeof paymentData !== 'object') {
        errors.push("Неверные данные платежа");
        return { isValid: false, errors };
    }

    if (!paymentData.amount || paymentData.amount <= 0) {
        errors.push("Неверная сумма платежа");
    }

    if (!paymentData.currency || !this.isSupportedCurrency(paymentData.currency)) {
        errors.push("Неподдерживаемая валюта");
    }

    if (!paymentData.method || !this.isSupportedMethod(paymentData.method)) {
        errors.push("Неверный метод оплаты");
    }

    // Проверка метода-специфичных полей
    if (paymentData.method === 'credit_card') {
        if (!this.isValidCreditCard(paymentData.cardDetails)) {
            errors.push("Неверные данные кредитной карты");
        }
    }

    return {
        isValid: errors.length === 0,
        errors
    };
}

```

```
};  
}  
  
canProcessPayment(paymentData) {  
    // Проверка различных условий возможности обработки  
    const isWithinBusinessHours = this.isWithinBusinessHours();  
    const hasAvailableCapacity = this.hasProcessingCapacity();  
    const isMethodAvailable = this.isPaymentMethodAvailable(paymentData.method);  
  
    return isWithinBusinessHours && hasAvailableCapacity && isMethodAvailable;  
}  
  
// ... остальные вспомогательные методы  
}
```

Итог: Условные выражения `if`, `else if`, `else` — это мощный инструмент управления потоком выполнения в JavaScript. Правильное их использование делает код не только функциональным, но и читаемым, поддерживаемым и эффективным. Помните: хороший код с условиями — это не просто работающий код, это код, который легко понять и изменить в будущем.

6.2. Многоступенчатое ветвление: switch-case

Введение: альтернатива цепочкам if-else

Конструкция `switch-case` предоставляет элегантный способ обработки множественных условий, когда значение переменной сравнивается с различными вариантами. Это особенно полезно при замене длинных цепочек `if-else if-else`.

Аналогия из реальной жизни:

- Автомат с напитками: выбираем код → получаем определенный напиток
- Меню ресторана: выбираем блюдо по номеру → получаем конкретное блюдо
- Система навигации: выбираем пункт назначения → получаем конкретный маршрут

Часть 1: Базовый синтаксис и структура

1.1. Основная структура switch-case

Базовый синтаксис:

```
javascript

switch (выражение) {
  case значение1:
    // код, выполняемый если выражение === значение1
    break;

  case значение2:
    // код, выполняемый если выражение === значение2
    break;

  case значение3:
    // код, выполняемый если выражение === значение3
    break;

  default:
```

```
    // код, выполняемый если ни один case не совпал
    // break не нужен, так как это последний блок
}
```

Подробное объяснение каждого компонента:

javascript

// Пример 1: Обработка дня недели

```
let dayNumber = 3; // 1=Понедельник, 2=Вторник, ..., 7=Воскресенье
let dayName;
```

```
switch (dayNumber) {
  case 1:
    dayName = "Понедельник";
    console.log("Начало рабочей недели");
    break;

  case 2:
    dayName = "Вторник";
    console.log("Второй рабочий день");
    break;

  case 3:
    dayName = "Среда";
    console.log("Середина недели");
    break;

  case 4:
    dayName = "Четверг";
    console.log("Предпоследний рабочий день");
    break;

  case 5:
    dayName = "Пятница";
    console.log("Последний рабочий день");
    break;

  case 6:
    dayName = "Суббота";
    console.log("Выходной день");
    break;

  case 7:
    dayName = "Воскресенье";
```

```
    console.log("Выходной день");
    break;

default:
    dayName = "Неизвестный день";
    console.log("Ошибка: день должен быть от 1 до 7");
}

console.log(`День номер ${dayNumber} - это ${dayName}`);
```

// Пример 2: Конвертер валют

```
function convertCurrency(amount, fromCurrency, toCurrency) {
    let rate;

    switch (fromCurrency) {
        case 'USD':
            switch (toCurrency) {
                case 'EUR':
                    rate = 0.85;
                    break;
                case 'GBP':
                    rate = 0.73;
                    break;
                case 'JPY':
                    rate = 110.5;
                    break;
                case 'RUB':
                    rate = 75.3;
                    break;
                default:
                    throw new Error(`Не поддерживаемая валюта назначения: ${toCurrency}`);
            }
            break;

        case 'EUR':
            switch (toCurrency) {
                case 'USD':
                    rate = 1.18;
                    break;
                case 'GBP':
                    rate = 0.86;
                    break;
                case 'JPY':
                    rate = 130.2;
```

```

        break;
    case 'RUB':
        rate = 88.7;
        break;
    default:
        throw new Error(`Не поддерживаемая валюта назначения: ${toCurrency}`);
    }
    break;

    default:
        throw new Error(`Не поддерживаемая исходная валюта: ${fromCurrency}`);
    }

    return amount * rate;
}

```

// Пример 3: Определение типа файла по расширению

```

function getFileType(filename) {
    // Извлекаем расширение файла
    let extension = filename.split('.').pop().toLowerCase();
    let fileType;

    switch (extension) {
        case 'jpg':
        case 'jpeg':
        case 'png':
        case 'gif':
        case 'bmp':
            fileType = 'image';
            break;

        case 'mp4':
        case 'avi':
        case 'mov':
        case 'wmv':
            fileType = 'video';
            break;

        case 'mp3':
        case 'wav':
        case 'flac':
        case 'aac':
            fileType = 'audio';
            break;
    }
}

```

```

    case 'pdf':
        fileType = 'document';
        break;

    case 'html':
    case 'htm':
        fileType = 'webpage';
        break;

    case 'js':
        fileType = 'javascript';
        break;

    case 'json':
        fileType = 'data';
        break;

    case 'txt':
    case 'md':
        fileType = 'text';
        break;

    default:
        fileType = 'unknown';
}

return {
    filename,
    extension,
    fileType,
    isSupported: fileType !== 'unknown'
};
}

```

1.2. Ключевые особенности switch-case

Строгое сравнение (===):

```

javascript
// switch использует строгое сравнение (===)
let value = "2";

```



```
switch (value) {
  case 1:
    console.log("Число 1");
    break;
  case 2:
    console.log("Число 2");
    break;
  case "1":
    console.log("Строка '1'");
    break;
  case "2":
    console.log("Строка '2'"); // Выполнится это!
    break;
  default:
    console.log("Другое значение");
}
```

// Эквивалент с if-else:

```
if (value === 1) {
  console.log("Число 1");
} else if (value === 2) {
  console.log("Число 2");
} else if (value === "1") {
  console.log("Строка '1'");
} else if (value === "2") {
  console.log("Строка '2'");
} else {
  console.log("Другое значение");
}
```

Важность оператора break:

javascript

// Пример ПРОБАЛЫВАНИЯ (fall-through) - что происходит без break

```
let month = 2;
let season;

switch (month) {
  case 12:
  case 1:
  case 2:
    season = "Зима";
    console.log("Холодное время года");
}
```

```
// Нет break - выполнение продолжается!
```

```
case 3:  
case 4:  
case 5:  
    season = "Весна"; // Перезапишет значение!  
    console.log("Время цветения");  
// Нет break - выполнение продолжается!
```

```
case 6:  
case 7:  
case 8:  
    season = "Лето"; // Снова перезапишет!  
    console.log("Теплое время года");  
// Нет break - выполнение продолжается!
```

```
case 9:  
case 10:  
case 11:  
    season = "Осень"; // И снова перезапишет!  
    console.log("Время сбора урожая");  
// Нет break, но это последний case
```

```
default:  
    console.log("Обработка завершена");
```

```
}
```

```
console.log(`Месяц ${month}, сезон: ${season}`); // "Месяц 2, сезон: Осень" -  
НЕПРАВИЛЬНО!
```

```
// ПРАВИЛЬНО с break:
```

```
function getSeason(month) {  
    let season;
```

```
    switch (month) {  
        case 12:  
        case 1:  
        case 2:  
            season = "Зима";  
            break;
```

```
        case 3:  
        case 4:  
        case 5:
```

```

        season = "Весна";
        break;

    case 6:
    case 7:
    case 8:
        season = "Лето";
        break;

    case 9:
    case 10:
    case 11:
        season = "Осень";
        break;

    default:
        season = "Неизвестно";
    }

    return season;
}

console.log(getSeason(2)); // "Зима"
console.log(getSeason(5)); // "Весна"
console.log(getSeason(8)); // "Лето"
console.log(getSeason(11)); // "Осень"

```

Часть 2: Продвинутые техники использования switch-case

2.1. Группировка case (несколько значений для одного блока)

```

javascript

// Пример 1: Группировка по категориям
function getProductCategory(productCode) {
    let category;

    switch (productCode) {
        // Электроника
        case 'ELEC-TV':
        case 'ELEC-PHONE':

```

```
case 'ELEC-LAPTOP':
case 'ELEC-TABLET':
    category = 'Электроника';
    break;

// Одежда
case 'CLOTH-SHIRT':
case 'CLOTH-PANTS':
case 'CLOTH-DRESS':
case 'CLOTH-JACKET':
    category = 'Одежда';
    break;

// Продукты
case 'FOOD-FRUIT':
case 'FOOD-VEG':
case 'FOOD-MEAT':
case 'FOOD-DAIRY':
    category = 'Продукты';
    break;

// Книги
case 'BOOK-FICTION':
case 'BOOK-NONFIC':
case 'BOOK-TECH':
case 'BOOK-CHILD':
    category = 'Книги';
    break;

default:
    category = 'Другое';
}

return category;
}

// Пример 2: Обработка HTTP статусов
function handleHttpStatus(statusCode) {
    let message;
    let isError = false;

    switch (statusCode) {
        // Успешные ответы (2xx)
        case 200:
```

```
case 201:
case 204:
    message = "Успешно";
    isError = false;
    break;

// Перенаправления (3xx)
case 301:
case 302:
case 304:
    message = "Перенаправление";
    isError = false;
    break;

// Ошибки клиента (4xx)
case 400:
    message = "Неверный запрос";
    isError = true;
    break;
case 401:
    message = "Не авторизован";
    isError = true;
    break;
case 403:
    message = "Доступ запрещен";
    isError = true;
    break;
case 404:
    message = "Не найдено";
    isError = true;
    break;

// Ошибки сервера (5xx)
case 500:
    message = "Внутренняя ошибка сервера";
    isError = true;
    break;
case 502:
    message = "Плохой шлюз";
    isError = true;
    break;
case 503:
    message = "Сервис недоступен";
    isError = true;
```

```

        break;

    default:
        message = `Неизвестный статус: ${statusCode}`;
        isError = true;
    }

    return { statusCode, message, isError };
}

```

// Пример 3: Определение типа символа

```

function getCharType(char) {
    if (char.length !== 1) {
        return "Не символ";
    }

    switch (char.toLowerCase()) {
        // Гласные
        case 'а': case 'е': case 'ё': case 'и': case 'о':
        case 'у': case 'ы': case 'э': case 'ю': case 'я':
        case 'а': case 'е': case 'і': case 'о': case 'и':
            return "Гласная";

        // Цифры
        case '0': case '1': case '2': case '3': case '4':
        case '5': case '6': case '7': case '8': case '9':
            return "Цифра";

        // Пробельные символы
        case ' ': case '\t': case '\n': case '\r':
            return "Пробел";

        // Знаки препинания
        case '.': case ',': case '!': case '?': case ':':
        case ';': case '-': case '(': case ')': case '[':
        case ']': case '{': case '}': case '"': case "'":
            return "Знак препинания";

        default:
            // Все остальное считаем согласными
            return "Согласная или другой символ";
    }
}

```

2.2. Использование выражений в case (через true)

javascript

// Необычный, но мощный паттерн: switch(true)

// Позволяет использовать сложные условия в case

// Пример 1: Диапазоны оценок

```
function getGrade(score) {  
    let grade;  
  
    switch (true) {  
        case (score >= 90 && score <= 100):  
            grade = 'A';  
            break;  
  
        case (score >= 80 && score < 90):  
            grade = 'B';  
            break;  
  
        case (score >= 70 && score < 80):  
            grade = 'C';  
            break;  
  
        case (score >= 60 && score < 70):  
            grade = 'D';  
            break;  
  
        case (score >= 0 && score < 60):  
            grade = 'F';  
            break;  
  
        default:  
            grade = 'Неверная оценка';  
    }  
  
    return grade;  
}
```

// Пример 2: Определение возрастной категории

```
function getAgeCategory(age) {  
    let category;  
  
    switch (true) {
```

```

    case (age >= 0 && age <= 1):
        category = 'Младенец';
        break;

    case (age >= 2 && age <= 12):
        category = 'Ребенок';
        break;

    case (age >= 13 && age <= 17):
        category = 'Подросток';
        break;

    case (age >= 18 && age <= 64):
        category = 'Взрослый';
        break;

    case (age >= 65):
        category = 'Пожилой';
        break;

    default:
        category = 'Неверный возраст';
}

return category;
}

// Пример 3: Сложные бизнес-правила
function calculateDiscount(customer, orderAmount) {
    let discountPercentage;

    switch (true) {
        // VIP клиенты с крупным заказом
        case (customer.isVIP && orderAmount > 10000):
            discountPercentage = 25;
            break;

        // VIP клиенты со средним заказом
        case (customer.isVIP && orderAmount > 5000):
            discountPercentage = 20;
            break;

        // VIP клиенты с любым заказом
        case (customer.isVIP):

```



```

    discountPercentage = 15;
    break;

    // Постоянные клиенты с историей
    case (customer.loyaltyYears >= 5 && orderAmount > 2000):
        discountPercentage = 10;
        break;

    // Постоянные клиенты
    case (customer.loyaltyYears >= 2):
        discountPercentage = 5;
        break;

    // Сезонная скидка для всех
    case (isHolidaySeason() && orderAmount > 1000):
        discountPercentage = 7;
        break;

    // Первая покупка
    case (customer.isFirstOrder):
        discountPercentage = 3;
        break;

    default:
        discountPercentage = 0;
}

return discountPercentage;
}

```

```

// Пример 4: Проверка типов данных с switch(true)
function getTypeDescription(value) {
    let type;
    let description;

    switch (true) {
        case (typeof value === 'string' && value.length > 50):
            type = 'string';
            description = 'Длинная строка';
            break;

        case (typeof value === 'string'):
            type = 'string';
            description = 'Короткая строка';

```

```
break;

case (typeof value === 'number' && Number.isInteger(value)):
    type = 'number';
    description = 'Целое число';
    break;

case (typeof value === 'number'):
    type = 'number';
    description = 'Дробное число';
    break;

case (Array.isArray(value) && value.length === 0):
    type = 'array';
    description = 'Пустой массив';
    break;

case (Array.isArray(value)):
    type = 'array';
    description = 'Непустой массив';
    break;

case (value === null):
    type = 'null';
    description = 'Значение null';
    break;

case (value === undefined):
    type = 'undefined';
    description = 'Неопределенное значение';
    break;

case (typeof value === 'object' && value !== null):
    type = 'object';
    description = 'Объект';
    break;

case (typeof value === 'boolean'):
    type = 'boolean';
    description = 'Логическое значение';
    break;

default:
    type = 'unknown';
```

```
        description = 'Неизвестный тип';
    }

    return { type, description };
}
```

2.3. Возврат значений из switch-case

javascript

// Пример 1: Функция, возвращающая значение через switch

```
function getDayType(dayNumber) {
    switch (dayNumber) {
        case 1:
        case 2:
        case 3:
        case 4:
        case 5:
            return 'Рабочий день';

        case 6:
        case 7:
            return 'Выходной день';

        default:
            return 'Неизвестный день';
    }
    // Нет необходимости в break после return
}
```

// Пример 2: Возврат объектов

```
function getWeatherAdvice(weatherCondition) {
    switch (weatherCondition.toLowerCase()) {
        case 'sunny':
            return {
                advice: 'Наденьте солнцезащитные очки',
                icon: '☀',
                activity: 'Идеально для пикника'
            };

        case 'rainy':
            return {
                advice: 'Возьмите зонт',
                icon: '☔',
            };
    }
}
```

```

        activity: 'Лучше остаться дома'
    };

    case 'snowy':
        return {
            advice: 'Оденьтесь тепло',
            icon: '❄',
            activity: 'Можно покататься на лыжах'
        };

    case 'cloudy':
        return {
            advice: 'Наденьте куртку',
            icon: '☁',
            activity: 'Хорошо для прогулки'
        };

    case 'windy':
        return {
            advice: 'Закрепите легкие предметы',
            icon: '🌬',
            activity: 'Отличное время для запуска воздушного змея'
        };

    default:
        return {
            advice: 'Проверьте погоду позже',
            icon: '☐',
            activity: 'Планируйте день осторожно'
        };
    }
}

```

// Пример 3: switch в стрелочной функции

```

const getTrafficLightAction = (lightColor) => {
    switch (lightColor.toLowerCase()) {
        case 'red':
            return { action: 'STOP', duration: 30, sound: 'beep-beep-beep' };

        case 'yellow':
            return { action: 'PREPARE', duration: 5, sound: 'beep' };

        case 'green':
            return { action: 'GO', duration: 45, sound: 'beep-beep' };
    }
}

```

```
    default:
        return { action: 'ERROR', duration: 0, sound: 'alarm' };
    }
};
```

// Пример 4: switch с вычислением значения

```
function calculateShippingCost(country, weight, isExpress) {
    let baseCost;
```

// Определяем базовую стоимость по стране

```
switch (country) {
    case 'RU': // Россия
        baseCost = 300;
        break;
    case 'US': // США
        baseCost = 500;
        break;
    case 'EU': // Европа
        baseCost = 400;
        break;
    case 'CN': // Китай
        baseCost = 200;
        break;
    default:
        baseCost = 600;
}
```

// Добавляем стоимость за вес

```
switch (true) {
    case (weight <= 1):
        baseCost += 100;
        break;
    case (weight <= 5):
        baseCost += 300;
        break;
    case (weight <= 10):
        baseCost += 600;
        break;
    default:
        baseCost += 1000;
}
```

// Учитываем экспресс-доставку

```
    if (isExpress) {  
        baseCost *= 2;  
    }  
  
    return baseCost;  
}
```

Часть 3: Особенности и ограничения switch-case

3.1. Отличия от if-else

```
javascript  
  
// СРАВНЕНИЕ switch-case и if-else  
  
// Пример 1: Простая проверка значения  
let color = "red";  
  
// С использованием switch-case  
let messageSwitch;  
switch (color) {  
    case "red":  
        messageSwitch = "Стоп";  
        break;  
    case "yellow":  
        messageSwitch = "Внимание";  
        break;  
    case "green":  
        messageSwitch = "Иди";  
        break;  
    default:  
        messageSwitch = "Неизвестный цвет";  
}  
  
// С использованием if-else  
let messageIf;  
if (color === "red") {  
    messageIf = "Стоп";  
} else if (color === "yellow") {  
    messageIf = "Внимание";  
} else if (color === "green") {  
    messageIf = "Иди";  
}
```

```

} else {
    messageIf = "Неизвестный цвет";
}

console.log("switch:", messageSwitch); // "Смон"
console.log("if-else:", messageIf);    // "Смон"

// Пример 2: Когда switch лучше
function getMonthDays(month, year) {
    // Для проверки одного значения с многими вариантами switch идеален
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;

        case 4: case 6: case 9: case 11:
            return 30;

        case 2:
            // Проверка високосного года (требуется дополнительная логика)
            return (year % 4 === 0 && (year % 100 !== 0 || year % 400 === 0)) ? 29 : 28;

        default:
            throw new Error("Неверный месяц");
    }
}

// Пример 3: Когда if-else лучше
function checkAccess(user, resource) {
    // Для сложных условий с разными переменными if-else лучше
    if (!user || !user.isActive) {
        return false;
    }

    if (user.role === "admin") {
        return true;
    }

    if (user.role === "editor" && resource.type !== "admin_panel") {
        return true;
    }

    if (user.id === resource.ownerId) {
        return true;
    }
}

```

```
    return false;
}
```

// Пример 4: Производительность

// Для большого количества условий switch может быть быстрее

// JavaScript движки оптимизируют switch как таблицу переходов (jump table)

```
const PERFORMANCE_TEST_COUNT = 1000000;
```

// Тест switch

```
console.time('switch');
let sum1 = 0;
for (let i = 0; i < PERFORMANCE_TEST_COUNT; i++) {
    let value = i % 10;
    switch (value) {
        case 0: sum1 += 1; break;
        case 1: sum1 += 2; break;
        case 2: sum1 += 3; break;
        case 3: sum1 += 4; break;
        case 4: sum1 += 5; break;
        case 5: sum1 += 6; break;
        case 6: sum1 += 7; break;
        case 7: sum1 += 8; break;
        case 8: sum1 += 9; break;
        case 9: sum1 += 10; break;
    }
}
console.timeEnd('switch');
```

// Тест if-else

```
console.time('if-else');
let sum2 = 0;
for (let i = 0; i < PERFORMANCE_TEST_COUNT; i++) {
    let value = i % 10;
    if (value === 0) {
        sum2 += 1;
    } else if (value === 1) {
        sum2 += 2;
    } else if (value === 2) {
        sum2 += 3;
    } else if (value === 3) {
        sum2 += 4;
    } else if (value === 4) {
```



```

    sum2 += 5;
} else if (value === 5) {
    sum2 += 6;
} else if (value === 6) {
    sum2 += 7;
} else if (value === 7) {
    sum2 += 8;
} else if (value === 8) {
    sum2 += 9;
} else if (value === 9) {
    sum2 += 10;
}
}
console.timeEnd('if-else');

console.log("Результаты равны:", sum1 === sum2);

```

3.2. Ограничения switch-case

javascript

*// ОГРАНИЧЕНИЕ 1: Только строгое равенство (===)
 // Нельзя использовать сравнения типа >, <, >=, <= напрямую*

```
let score = 85;
```

// НЕ РАБОТАЕТ (синтаксическая ошибка):

```

/*
switch (score) {
    case > 90: // ОШИБКА!
        console.log("Отлично");
        break;
}
*/

```

// ОБХОД через switch(true):

```

switch (true) {
    case (score > 90):
        console.log("Отлично");
        break;
    case (score > 70):
        console.log("Хорошо");
        break;
    case (score > 50):

```

```
    console.log("Удовлетворительно");
    break;
default:
    console.log("Неудовлетворительно");
}
```

// ОГРАНИЧЕНИЕ 2: Нельзя использовать выражения в case (кроме switch(true))

```
let x = 10;
let y = 20;
```

// НЕ РАБОТАЕТ:

```
/*
switch (x) {
    case y - 10: // ОШИБКА в стандартном режиме
        console.log("x равно y-10");
        break;
}
*/
```

// ОБХОД через вычисление значения заранее:

```
let computedValue = y - 10;
switch (x) {
    case computedValue:
        console.log("x равно y-10");
        break;
}
```

// Или через switch(true):

```
switch (true) {
    case (x === y - 10):
        console.log("x равно y-10");
        break;
}
```

// ОГРАНИЧЕНИЕ 3: Нет проверки на null/undefined в одном case

```
let value = null;

switch (value) {
    case null:
        console.log("Значение null");
        break;
    case undefined:
        console.log("Значение undefined");
        break;
}
```

```

    default:
        console.log("Другое значение");
}
// Работает, но требует отдельных case для null и undefined

// ОГРАНИЧЕНИЕ 4: Нельзя использовать регулярные выражения напрямую
let text = "hello@example.com";

// НЕ РАБОТАЕТ:
/*
switch (text) {
    case /^[^@]+@[^@]+\.[^@]+$/.test(text): // ОШИБКА
        console.log("Это email");
        break;
}
*/

// ОБХОД через switch(true):
switch (true) {
    case /^[^@]+@[^@]+\.[^@]+$/.test(text):
        console.log("Это email");
        break;
    case /^[^+?[\d\s\-\(\)]+$/.test(text):
        console.log("Это телефон");
        break;
    default:
        console.log("Неизвестный формат");
}

// ОГРАНИЧЕНИЕ 5: Область видимости переменных
switch (1) {
    case 1:
        // Переменные, объявленные с let/const, видны только внутри этого case
        let message = "Первый case";
        const PI = 3.14;
        console.log(message);
        break;

    case 2:
        // Здесь нельзя использовать message и PI из case 1
        // console.log(message); // ОШИБКА: message is not defined
        break;
}

```

// ОБХОД через объявление переменных вне switch:

```
let switchMessage;
const switchPI = 3.14;

switch (1) {
  case 1:
    switchMessage = "Первый case";
    console.log(switchMessage);
    break;

  case 2:
    switchMessage = "Второй case";
    console.log(switchMessage);
    break;
}
```

3.3. Ловушки и частые ошибки

javascript

// ЛОВУШКА 1: Забытый break (непреднамеренное пробаливание)

```
function getPrice(category) {
  let price;

  switch (category) {
    case 'premium':
      price = 100;
      // ЗАБЫЛИ break!

    case 'standard':
      price = 50; // Выполнится и для 'premium'!
      // ЗАБЫЛИ break!

    case 'basic':
      price = 20; // Выполнится для всех!
      break;

    default:
      price = 0;
  }

  return price;
}
```

```

console.log(getPrice('premium')); // 20 (а должно быть 100!)
console.log(getPrice('standard')); // 20 (правильно)
console.log(getPrice('basic')); // 20 (правильно)

// РЕШЕНИЕ: Всегда используйте break или явно документируйте fall-through
function getPriceFixed(category) {
    let price;

    switch (category) {
        case 'premium':
            price = 100;
            break; // Явный break

        case 'standard':
            price = 50;
            break; // Явный break

        case 'basic':
            price = 20;
            break; // Явный break

        default:
            price = 0;
    }

    return price;
}

// ЛОВУШКА 2: Использование сложных выражений без switch(true)
let age = 25;

// НЕПРАВИЛЬНО (не работает как ожидается):
/*
switch (age) {
    case (age >= 18):
        console.Log("Совершеннолетний");
        break;
    default:
        console.Log("Несовершеннолетний");
}
// Всегда выполняется default, потому что age (25) !== true
*/

// ПРАВИЛЬНО:

```

```
switch (true) {  
  case (age >= 18):  
    console.log("Совершеннолетний");  
    break;  
  default:  
    console.log("Несовершеннолетний");  
}
```

// ЛОВУШКА 3: Дублирование case

```
let fruit = 'apple';
```

```
switch (fruit) {  
  case 'apple':  
    console.log("Яблоко");  
    break;  
  
  case 'apple': // ДУБЛИРОВАНИЕ! Будет проигнорировано  
    console.log("Еще одно яблоко");  
    break;  
  
  case 'banana':  
    console.log("Банан");  
    break;  
  
  default:  
    console.log("Неизвестный фрукт");  
}
```

// Выведет: "Яблоко", второй case 'apple' никогда не выполнится

// ЛОВУШКА 4: Некорректное использование default

```
let status = 'pending';
```

```
switch (status) {  
  case 'pending':  
    console.log("Ожидание");  
    // Нет break!  
  
  default: // default не обязательно должен быть последним  
    console.log("Статус по умолчанию");  
    break;  
  
  case 'completed':  
    console.log("Завершено");  
    break;
```

```
}
```

```
// Порядок выполнения:
```

```
// 1. case 'pending': выполняется
```

```
// 2. Пробаливание в default (т.к. нет break)
```

```
// 3. default выполняется
```

```
// 4. break в default прерывает выполнение
```

```
// Результат: "Ожидание", "Статус по умолчанию"
```

```
// РЕКОМЕНДАЦИЯ: Всегда помещайте default в конец и используйте break
```

```
switch (status) {  
  case 'pending':  
    console.log("Ожидание");  
    break;  
  
  case 'completed':  
    console.log("Завершено");  
    break;  
  
  default:  
    console.log("Статус по умолчанию");  
    // break не обязателен, но рекомендуется для consistency  
}
```

```
// ЛОВУШКА 5: Неучтенные случаи в default
```

```
function processUserRole(role) {  
  switch (role) {  
    case 'admin':  
      return "Полный доступ";  
    case 'moderator':  
      return "Ограниченный доступ";  
    case 'user':  
      return "Базовый доступ";  
    // Нет default!  
  }  
}
```

```
console.log(processUserRole('guest')); // undefined
```

```
// Для роли 'guest' функция возвращает undefined
```

```
// РЕШЕНИЕ: Всегда предусматривайте default
```

```
function processUserRoleFixed(role) {  
  switch (role) {  
    case 'admin':
```

```
        return "Полный доступ";
    case 'moderator':
        return "Ограниченный доступ";
    case 'user':
        return "Базовый доступ";
    default:
        return "Доступ запрещен"; // Обработка непредусмотренных случаев
    }
}
```

Часть 4: Реальные примеры и практические применения

4.1. Парсинг и обработка команд

javascript

// Пример 1: Текстовый парсер команд

```
class CommandParser {
    constructor() {
        this.commands = new Map();
    }

    parse(input) {
        const parts = input.trim().split(/\s+/);
        const command = parts[0].toLowerCase();
        const args = parts.slice(1);

        let result;

        switch (command) {
            case 'help':
                result = this.showHelp();
                break;

            case 'echo':
                result = this.echo(args.join(' '));
                break;

            case 'calc':
                result = this.calculate(args);
                break;
        }
    }
}
```



```

    case 'time':
        result = this.showTime();
        break;

    case 'clear':
        result = this.clearScreen();
        break;

    case 'search':
        result = this.search(args);
        break;

    case 'exit':
    case 'quit':
        result = this.exit();
        break;

    default:
        result = this.unknownCommand(command);
}

```

```

return result;
}

```

```

showHelp() {
    return `

```

Доступные команды:

```

- help           - показать эту справку
- echo <текст>   - повторить текст
- calc <выражение> - вычислить выражение
- time           - показать текущее время
- clear          - очистить экран
- search <запрос> - поиск в системе
- exit/quit      - выход из программы
`
;
}

```

```

echo(text) {
    return text || "Нет текста для отображения";
}

```

```

calculate(args) {
    if (args.length === 0) {
        return "Укажите выражение для вычисления";
    }
}

```

```

    }

    const expression = args.join('');
    try {
        // ВНИМАНИЕ: eval опасен в production коде!
        const result = eval(expression);
        return `Результат: ${result}`;
    } catch (error) {
        return `Ошибка вычисления: ${error.message}`;
    }
}

showTime() {
    return new Date().toLocaleString();
}

clearScreen() {
    console.clear();
    return "Экран очищен";
}

search(args) {
    const query = args.join(' ');
    return `Поиск: "${query}" (функция в разработке)`;
}

exit() {
    return "Выход из программы...";
}

unknownCommand(command) {
    return `Неизвестная команда: "${command}". Введите "help" для списка команд.`;
}
}

// Пример использования
const parser = new CommandParser();
console.log(parser.parse("help"));
console.log(parser.parse("echo Привет, мир!"));
console.log(parser.parse("calc 2 + 2 * 2"));
console.log(parser.parse("time"));
console.log(parser.parse("unknown command"));

```

// Пример 2: Обработка HTTP методов в роутере

```
class Router {
  constructor() {
    this.routes = {
      GET: {},
      POST: {},
      PUT: {},
      DELETE: {},
      PATCH: {}
    };
  }

  async handleRequest(req, res) {
    const { method, url } = req;

    switch (method) {
      case 'GET':
        return await this.handleGet(url, req, res);

      case 'POST':
        return await this.handlePost(url, req, res);

      case 'PUT':
        return await this.handlePut(url, req, res);

      case 'DELETE':
        return await this.handleDelete(url, req, res);

      case 'PATCH':
        return await this.handlePatch(url, req, res);

      case 'OPTIONS':
        return await this.handleOptions(req, res);

      case 'HEAD':
        return await this.handleHead(url, req, res);

      default:
        return this.sendError(res, 405, `Метод ${method} не поддерживается`);
    }
  }

  async handleGet(url, req, res) {
    const route = this.findRoute('GET', url);
```

```

if (!route) {
  return this.sendError(res, 404, 'Страница не найдена');
}

switch (route.type) {
  case 'static':
    return this.serveStaticFile(route.path, res);

  case 'api':
    return await this.handleApiRequest(route, req, res);

  case 'view':
    return await this.renderView(route.template, req, res);

  case 'redirect':
    return this.redirect(res, route.target);

  default:
    return this.sendError(res, 500, 'Неизвестный тип роута');
}
}

async handlePost(url, req, res) {
  const route = this.findRoute('POST', url);

  if (!route) {
    return this.sendError(res, 404, 'Маршрут не найден');
  }

  // Парсинг тела запроса в зависимости от Content-Type
  const contentType = req.headers['content-type'] || '';

  let body;
  switch (true) {
    case contentType.includes('application/json'):
      body = await this.parseJsonBody(req);
      break;

    case contentType.includes('application/x-www-form-urlencoded'):
      body = await this.parseFormBody(req);
      break;

    case contentType.includes('multipart/form-data'):
      body = await this.parseMultipartBody(req);

```

```

        break;

    default:
        body = {};
    }

    // Обработка в зависимости от типа действия
    switch (route.action) {
        case 'create':
            return await this.createResource(route.resource, body, res);

        case 'update':
            return await this.updateResource(route.resource, body, res);

        case 'login':
            return await this.handleLogin(body, res);

        case 'register':
            return await this.handleRegistration(body, res);

        case 'upload':
            return await this.handleFileUpload(body, res);

        default:
            return this.sendError(res, 400, 'Неизвестное действие');
    }
}

// ... остальные методы handlePut, handleDelete и т.д.

findRoute(method, url) {
    return this.routes[method][url] || null;
}

sendError(res, code, message) {
    res.statusCode = code;
    res.end(JSON.stringify({ error: message }));
}

// ... вспомогательные методы
}

```

4.2. Система обработки событий

javascript

// Пример: Event Handler с switch-case

```
class EventDispatcher {
  constructor() {
    this.handlers = new Map();
  }

  dispatch(eventType, data) {
    console.log(`Диспетчеризация события: ${eventType}`);

    switch (eventType) {
      case 'USER_REGISTERED':
        return this.handleUserRegistered(data);

      case 'USER_LOGGED_IN':
        return this.handleUserLoggedIn(data);

      case 'USER_LOGGED_OUT':
        return this.handleUserLoggedOut(data);

      case 'ORDER_CREATED':
        return this.handleOrderCreated(data);

      case 'ORDER_UPDATED':
        return this.handleOrderUpdated(data);

      case 'ORDER_COMPLETED':
        return this.handleOrderCompleted(data);

      case 'ORDER_CANCELLED':
        return this.handleOrderCancelled(data);

      case 'PAYMENT_RECEIVED':
        return this.handlePaymentReceived(data);

      case 'PAYMENT_FAILED':
        return this.handlePaymentFailed(data);

      case 'PRODUCT_ADDED':
        return this.handleProductAdded(data);
```

```

    case 'PRODUCT_UPDATED':
        return this.handleProductUpdated(data);

    case 'PRODUCT_DELETED':
        return this.handleProductDeleted(data);

    case 'CART_UPDATED':
        return this.handleCartUpdated(data);

    case 'SHIPPING_STATUS_CHANGED':
        return this.handleShippingStatusChanged(data);

    case 'NOTIFICATION_SENT':
        return this.handleNotificationSent(data);

    case 'ERROR_OCCURRED':
        return this.handleError(data);

    default:
        console.warn(`Неизвестный тип события: ${eventType}`);
        return null;
    }
}

handleUserRegistered(data) {
    const { userId, email, username } = data;

    // Отправка email подтверждения
    this.sendEmail(email, 'Добро пожаловать!', `Привет, ${username}!`);

    // Создание профиля пользователя
    this.createUserProfile(userId);

    // Отправка уведомления администратору
    this.notifyAdmin(`Новый пользователь: ${username} (${email})`);

    // Логирование
    this.logEvent('USER_REGISTERED', { userId, timestamp: Date.now() });

    return { success: true, message: 'Пользователь зарегистрирован' };
}

handleUserLoggedIn(data) {
    const { userId, ipAddress, userAgent } = data;

```

```
// Обновление времени последнего входа
this.updateLastLogin(userId);

// Проверка подозрительной активности
if (this.isSuspiciousActivity(ipAddress, userId)) {
  this.sendSecurityAlert(userId, ipAddress);
}

// Запись в лог доступа
this.logAccess(userId, ipAddress, userAgent);

return { success: true };
}

handleOrderCreated(data) {
  const { orderId, userId, items, total } = data;

  // Резервирование товаров на складе
  items.forEach(item => {
    this.reserveInventory(item.productId, item.quantity);
  });

  // Создание счета
  this.createInvoice(orderId, total);

  // Отправка подтверждения заказа
  const user = this.getUser(userId);
  this.sendEmail(user.email, 'Заказ создан', `Ваш заказ #${orderId} создан`);

  // Уведомление отдела продаж
  this.notifySalesTeam(orderId, total);

  return { success: true, orderId };
}

handlePaymentReceived(data) {
  const { orderId, amount, paymentMethod } = data;

  // Обновление статуса заказа
  this.updateOrderStatus(orderId, 'PAID');

  // Снятие резерва с товаров
  this.releaseInventoryReservation(orderId);
```



```
// Запись в финансовый журнал
this.recordPayment(orderId, amount, paymentMethod);

// Отправка уведомления о платеже
const order = this.getOrder(orderId);
const user = this.getUser(order.userId);
this.sendEmail(
  user.email,
  'Платеж получен',
  `Платеж по заказу #${orderId} получен`
);

// Запуск процесса отгрузки
this.initiateShipping(orderId);

return { success: true };
}

handleError(data) {
  const { error, context, severity } = data;

  switch (severity) {
    case 'LOW':
      console.warn(`Предупреждение: ${error.message}`, context);
      break;

    case 'MEDIUM':
      console.error(`Ошибка: ${error.message}`, context);
      this.notifyDevelopers(error, context);
      break;

    case 'HIGH':
      console.error(`КРИТИЧЕСКАЯ ОШИБКА: ${error.message}`, context);
      this.notifyDevelopers(error, context);
      this.notifyManagers(error, context);
      this.triggerMonitoringAlert(error);
      break;

    case 'CRITICAL':
      console.error(`КАТАСТРОФИЧЕСКАЯ ОШИБКА: ${error.message}`, context);
      this.notifyEveryone(error, context);
      this.activateEmergencyProtocol();
      break;
  }
}
```

```

    default:
        console.error(`Неизвестная ошибка: ${error.message}`, context);
    }

    // Запись в лог ошибок
    this.logError(error, context, severity);

    return { logged: true };
}

// ... остальные методы обработчиков

// Вспомогательные методы
sendEmail(to, subject, body) {
    console.log(`Отправка email: ${to} - ${subject}`);
    // Реальная реализация отправки email
}

logEvent(eventType, data) {
    console.log(`Логирование события: ${eventType}`, data);
    // Реальная реализация логирования
}

// ... другие вспомогательные методы
}

// Пример использования
const dispatcher = new EventDispatcher();

// Симуляция событий
dispatcher.dispatch('USER_REGISTERED', {
    userId: '123',
    email: 'user@example.com',
    username: 'john_doe'
});

dispatcher.dispatch('ORDER_CREATED', {
    orderId: 'ORD-456',
    userId: '123',
    items: [
        { productId: 'P1', quantity: 2, price: 50 },
        { productId: 'P2', quantity: 1, price: 100 }
    ],
},

```

```

    total: 200
  });

dispatcher.dispatch('ERROR_OCCURRED', {
  error: new Error('Ошибка базы данных'),
  context: { operation: 'save_user', userId: '123' },
  severity: 'HIGH'
});

```

4.3. Система конфигурации и настроек

javascript

```

// Пример: Configuration Manager с switch-case
class ConfigurationManager {
  constructor() {
    this.config = {};
    this.environment = process.env.NODE_ENV || 'development';
  }

  loadConfiguration() {
    // Базовые настройки для всех окружений
    this.config = {
      app: {
        name: 'My Application',
        version: '1.0.0'
      }
    };

    // Загрузка настроек в зависимости от окружения
    switch (this.environment) {
      case 'development':
        this.loadDevelopmentConfig();
        break;

      case 'testing':
        this.loadTestingConfig();
        break;

      case 'staging':
        this.loadStagingConfig();
        break;

      case 'production':

```

```
        this.loadProductionConfig();
        break;

    default:
        console.warn(`Неизвестное окружение: ${this.environment}`);
        this.loadDevelopmentConfig();
    }

    // Применение настроек в зависимости от функций
    this.applyFeatureFlags();
    this.applySecuritySettings();
    this.applyPerformanceSettings();

    return this.config;
}

loadDevelopmentConfig() {
    Object.assign(this.config, {
        environment: 'development',
        debug: true,
        logLevel: 'debug',

        server: {
            host: 'localhost',
            port: 3000,
            ssl: false
        },

        database: {
            host: 'localhost',
            port: 5432,
            name: 'app_dev',
            username: 'dev_user',
            password: 'dev_password',
            poolSize: 5,
            logging: true
        },

        cache: {
            enabled: false,
            ttl: 300
        },

        email: {
```

```
        enabled: false,
        testMode: true
    },

    monitoring: {
        enabled: false
    }
});
}
```

```
loadTestingConfig() {
    Object.assign(this.config, {
        environment: 'testing',
        debug: true,
        logLevel: 'info',

        server: {
            host: 'test.example.com',
            port: 3000,
            ssl: true
        },

        database: {
            host: 'test-db.example.com',
            port: 5432,
            name: 'app_test',
            username: 'test_user',
            password: 'test_password',
            poolSize: 10,
            logging: false
        },

        cache: {
            enabled: true,
            ttl: 600
        },

        email: {
            enabled: true,
            testMode: true
        },

        monitoring: {
            enabled: true,
```

```
        sampleRate: 0.5
    }
});
}
```

```
loadProductionConfig() {
  Object.assign(this.config, {
    environment: 'production',
    debug: false,
    logLevel: 'warn',

    server: {
      host: 'api.example.com',
      port: 443,
      ssl: true,
      cluster: true,
      workers: 4
    },

    database: {
      host: process.env.DB_HOST,
      port: parseInt(process.env.DB_PORT),
      name: process.env.DB_NAME,
      username: process.env.DB_USER,
      password: process.env.DB_PASSWORD,
      poolSize: 20,
      logging: false,
      readReplicas: [
        { host: 'replica1.example.com' },
        { host: 'replica2.example.com' }
      ]
    },

    cache: {
      enabled: true,
      ttl: 3600,
      cluster: true,
      nodes: [
        { host: 'redis1.example.com', port: 6379 },
        { host: 'redis2.example.com', port: 6379 }
      ]
    },

    email: {
```

```

        enabled: true,
        testMode: false,
        provider: 'smtp',
        rateLimit: 100
    },

    monitoring: {
        enabled: true,
        sampleRate: 1.0,
        providers: ['datadog', 'sentry']
    },

    security: {
        rateLimiting: {
            enabled: true,
            windowMs: 15 * 60 * 1000, // 15 минут
            maxRequests: 100
        },
        cors: {
            origin: ['https://example.com'],
            credentials: true
        }
    }
});
}

applyFeatureFlags() {
    // Включение/выключение функций в зависимости от окружения
    switch (this.environment) {
        case 'development':
            this.config.features = {
                experimentalApi: true,
                betaFeatures: true,
                debugPanel: true,
                mockData: true
            };
            break;

        case 'testing':
            this.config.features = {
                experimentalApi: true,
                betaFeatures: true,
                debugPanel: false,
                mockData: false
            };
            break;
    }
}

```

```

    };
    break;

    case 'production':
        this.config.features = {
            experimentalApi: false,
            betaFeatures: false,
            debugPanel: false,
            mockData: false,
            aBTesting: true,
            analytics: true
        };
        break;
    }
}

applySecuritySettings() {
    switch (this.environment) {
        case 'development':
            this.config.security = {
                jwtSecret: 'dev-secret-key',
                tokenExpiry: '7d',
                passwordPolicy: {
                    minLength: 6,
                    requireNumbers: false,
                    requireSpecialChars: false
                }
            };
            break;

            case 'production':
                this.config.security = {
                    jwtSecret: process.env.JWT_SECRET,
                    tokenExpiry: '1h',
                    refreshTokenExpiry: '7d',
                    passwordPolicy: {
                        minLength: 12,
                        requireNumbers: true,
                        requireSpecialChars: true,
                        requireUppercase: true,
                        requireLowercase: true
                    },
                    twoFactor: {
                        enabled: true,

```



```

        requiredForAdmin: true
    }
};
break;
}
}

```

```

applyPerformanceSettings() {
    switch (this.environment) {
        case 'development':
            this.config.performance = {
                cacheTtl: 60, // 1 минута
                compression: false,
                minify: false,
                imageOptimization: false
            };
            break;

        case 'production':
            this.config.performance = {
                cacheTtl: 3600, // 1 час
                compression: true,
                minify: true,
                imageOptimization: true,
                cdn: {
                    enabled: true,
                    domain: 'cdn.example.com'
                },
                database: {
                    queryTimeout: 5000,
                    connectionTimeout: 10000
                }
            };
            break;
    }
}

```

```

getConfigValue(keyPath, defaultValue = null) {
    const keys = keyPath.split('.');
    let value = this.config;

    for (const key of keys) {
        if (value && typeof value === 'object' && key in value) {
            value = value[key];
        }
    }
}

```

```

    } else {
        return defaultValue;
    }
}

return value;
}

updateConfig(keyPath, value) {
    const keys = keyPath.split('.');
    let config = this.config;

    // Проходим по всем ключам, кроме последнего
    for (let i = 0; i < keys.length - 1; i++) {
        const key = keys[i];

        if (!config[key] || typeof config[key] !== 'object') {
            config[key] = {};
        }

        config = config[key];
    }

    // Устанавливаем значение для последнего ключа
    const lastKey = keys[keys.length - 1];

    // Проверка типа значения в зависимости от ключа
    switch (lastKey) {
        case 'port':
        case 'poolSize':
        case 'ttl':
        case 'windowMs':
        case 'maxRequests':
            if (typeof value !== 'number') {
                throw new Error(`Значение для ${keyPath} должно быть числом`);
            }
            break;

        case 'enabled':
        case 'debug':
        case 'ssl':
        case 'testMode':
        case 'cluster':
            if (typeof value !== 'boolean') {

```

```

        throw new Error(`Значение для ${keyPath} должно быть boolean`);
    }
    break;

    case 'host':
    case 'name':
    case 'username':
    case 'password':
        if (typeof value !== 'string') {
            throw new Error(`Значение для ${keyPath} должно быть строкой`);
        }
        break;

    case 'logLevel':
        const validLogLevels = ['debug', 'info', 'warn', 'error'];
        if (!validLogLevels.includes(value)) {
            throw new Error(`Недопустимый уровень логирования: ${value}`);
        }
        break;
}

config[lastKey] = value;
return true;
}
}

// Пример использования
const configManager = new ConfigurationManager();

// Загрузка конфигурации для разных окружений
process.env.NODE_ENV = 'development';
const devConfig = configManager.loadConfiguration();
console.log('Development config:', devConfig);

process.env.NODE_ENV = 'production';
const prodConfig = configManager.loadConfiguration();
console.log('Production config:', prodConfig);

// Получение значений конфигурации
console.log('DB Host:', configManager.getConfigValue('database.host'));
console.log('Log Level:', configManager.getConfigValue('logLevel', 'info'));

// Обновление конфигурации
try {

```

```
configManager.updateConfig('server.port', 8080);
configManager.updateConfig('debug', true);
configManager.updateConfig('logLevel', 'debug');
} catch (error) {
  console.error('Ошибка обновления конфигурации:', error.message);
}
```

Часть 5: Альтернативы и современные подходы

5.1. Объекты как альтернатива switch-case

javascript

// Пример 1: Замена простого switch на объект

// Исходный код со switch

```
function getColorName(colorCode) {
  let colorName;

  switch (colorCode) {
    case '#FF0000':
      colorName = 'Красный';
      break;
    case '#00FF00':
      colorName = 'Зеленый';
      break;
    case '#0000FF':
      colorName = 'Синий';
      break;
    case '#FFFF00':
      colorName = 'Желтый';
      break;
    case '#FF00FF':
      colorName = 'Пурпурный';
      break;
    default:
      colorName = 'Неизвестный цвет';
  }

  return colorName;
}
```

// Альтернатива с объектом

```
const colorMap = {
  '#FF0000': 'Красный',
  '#00FF00': 'Зеленый',
  '#0000FF': 'Синий',
  '#FFFF00': 'Желтый',
  '#FF00FF': 'Пурпурный'
};

function getColorNameObject(colorCode) {
  return colorMap[colorCode] || 'Неизвестный цвет';
}
```

// Пример 2: Объект с функциями

// Исходный код со switch

```
function calculate(operation, a, b) {
  let result;

  switch (operation) {
    case 'add':
      result = a + b;
      break;
    case 'subtract':
      result = a - b;
      break;
    case 'multiply':
      result = a * b;
      break;
    case 'divide':
      result = b !== 0 ? a / b : NaN;
      break;
    default:
      result = NaN;
  }

  return result;
}
```

// Альтернатива с объектом функций

```
const operations = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
  multiply: (a, b) => a * b,
  divide: (a, b) => b !== 0 ? a / b : NaN,
  power: (a, b) => Math.pow(a, b),
}
```

```
    modulo: (a, b) => a % b
};
```

```
function calculateObject(operation, a, b) {
    const operationFn = operations[operation];
    return operationFn ? operationFn(a, b) : NaN;
}
```

// Пример 3: Сложная логика с объектами

// Исходный код со switch (сложные условия)

```
function handleUserAction(action, user, data) {
    switch (action) {
        case 'login':
            return loginUser(user, data);
        case 'register':
            return registerUser(user, data);
        case 'update_profile':
            return updateUserProfile(user, data);
        case 'change_password':
            return changeUserPassword(user, data);
        case 'delete_account':
            return deleteUserAccount(user, data);
        case 'reset_password':
            return resetUserPassword(user, data);
        default:
            throw new Error(`Неизвестное действие: ${action}`);
    }
}
```

// Альтернатива с объектом обработчиков

```
const userActionHandlers = {
    login: (user, data) => {
        // Сложная логика входа
        if (!user || !data.password) {
            throw new Error('Неверные данные для входа');
        }
        return authenticate(user.email, data.password);
    },

    register: (user, data) => {
        // Сложная логика регистрации
        if (!user.email || !user.password) {
            throw new Error('Не все обязательные поля заполнены');
        }
    }
}
```

```

    return createUser(user);
  },

  update_profile: (user, data) => {
    // Логика обновления профиля
    return updateUser(user.id, data);
  },

  change_password: (user, data) => {
    // Логика смены пароля
    const { oldPassword, newPassword } = data;
    return changePassword(user.id, oldPassword, newPassword);
  },

  delete_account: (user, data) => {
    // Логика удаления аккаунта
    const { confirmation, reason } = data;
    return deleteAccount(user.id, confirmation, reason);
  },

  reset_password: (user, data) => {
    // Логика сброса пароля
    return initiatePasswordReset(user.email);
  }
};

function handleUserActionObject(action, user, data) {
  const handler = userActionHandlers[action];

  if (!handler) {
    throw new Error(`Неизвестное действие: ${action}`);
  }

  return handler(user, data);
}

// Преимущества объектного подхода:
// 1. Легче расширять (добавлять новые обработчики)
// 2. Легче тестировать (каждый обработчик независим)
// 3. Чище код (нет длинных switch блоков)
// 4. Легче рефакторить

// Недостатки:
// 1. Нет возможности использовать fall-through

```

// 2. Может быть менее производительным для большого количества условий
// 3. Не подходит для условий с диапазонами

// Пример 4: Динамические обработчики

```
class ActionDispatcher {
  constructor() {
    this.handlers = {};
    this.middlewares = [];
  }

  register(action, handler) {
    this.handlers[action] = handler;
    return this;
  }

  use(middleware) {
    this.middlewares.push(middleware);
    return this;
  }

  async dispatch(action, ...args) {
    // Применение middleware
    for (const middleware of this.middlewares) {
      const result = await middleware(action, ...args);
      if (result !== undefined) {
        return result;
      }
    }

    // Поиск обработчика
    const handler = this.handlers[action];

    if (!handler) {
      throw new Error(`Неизвестное действие: ${action}`);
    }

    // Выполнение обработчика
    return await handler(...args);
  }
}

// Использование
const dispatcher = new ActionDispatcher();
```


// Регистрация обработчиков

dispatcher

```
.register('login', async (email, password) => {  
  // Логика входа  
  return { success: true, token: 'jwt-token' };  
})  
.register('register', async (userData) => {  
  // Логика регистрации  
  return { success: true, userId: '123' };  
})  
.use(async (action, ...args) => {  
  // Middleware для логирования  
  console.log(`Action: ${action}`, args);  
})  
.use(async (action, ...args) => {  
  // Middleware для валидации  
  if (action === 'login' && args.length < 2) {  
    throw new Error('Неверные аргументы для login');  
  }  
});
```

// Диспетчеризация действий

```
dispatcher.dispatch('login', 'user@example.com', 'password123');  
dispatcher.dispatch('register', { name: 'John', email: 'john@example.com' });
```

5.2. Map и Set как альтернативы

javascript

// Пример 1: Использование Map для сложных ключей

// Switch не может использовать объекты или массивы как case

```
const statusMap = new Map();
```

// Сложные ключи (объекты, массивы)

```
statusMap.set(['pending', 'low'], 'В обработке (низкий приоритет)');  
statusMap.set(['pending', 'high'], 'СРОЧНО в обработке');  
statusMap.set(['completed', 'success'], 'Успешно завершено');  
statusMap.set(['completed', 'partial'], 'Частично завершено');  
statusMap.set(['failed', 'retryable'], 'Ошибка (можно повторить)');  
statusMap.set(['failed', 'fatal'], 'Критическая ошибка');
```

```
function getStatusDescription(status, priority) {  
  const key = [status, priority];  
  return statusMap.get(key) || 'Неизвестный статус';  
}
```

```
}
```

```
// Пример 2: Использование Map для быстрого поиска
```

```
class PermissionChecker {  
  constructor() {  
    // Иерархия ролей и разрешений  
    this.rolePermissions = new Map([  
      ['guest', new Set(['read'])],  
      ['user', new Set(['read', 'write', 'comment'])],  
      ['moderator', new Set(['read', 'write', 'comment', 'moderate'])],  
      ['admin', new Set(['read', 'write', 'comment', 'moderate', 'delete',  
'manage_users'])]  
    ]);  
  
    // Разрешения по ресурсам  
    this.resourcePermissions = new Map([  
      ['public_post', new Set(['read'])],  
      ['private_post', new Set(['read', 'write'])],  
      ['admin_panel', new Set(['read', 'manage_users'])],  
      ['user_settings', new Set(['read', 'write'])]  
    ]);  
  }  
  
  can(role, action, resource) {  
    // Проверка существования роли  
    if (!this.rolePermissions.has(role)) {  
      return false;  
    }  
  
    // Проверка разрешений роли  
    const rolePerms = this.rolePermissions.get(role);  
    if (!rolePerms.has(action)) {  
      return false;  
    }  
  
    // Проверка разрешений ресурса (если указан)  
    if (resource) {  
      const resourcePerms = this.resourcePermissions.get(resource);  
      if (!resourcePerms || !resourcePerms.has(action)) {  
        return false;  
      }  
    }  
  
    return true;  
  }  
}
```

```

}

// Альтернатива switch для особых случаев
checkSpecialPermission(role, context) {
  const specialCases = new Map([
    ['super_admin', () => true], // Супер-админ имеет все права
    ['trial_user', (ctx) => ctx.daysSinceRegistration < 30], // Тriaльный период
    ['banned_user', () => false], // Забаненные пользователи
    ['internal_service', (ctx) => ctx.ip === '127.0.0.1'] // Внутренние сервисы
  ]);

  const specialCheck = specialCases.get(role);
  return specialCheck ? specialCheck(context) : null;
}
}

```

```

// Пример 3: Использование Set для проверки членства
class InputValidator {
  constructor() {
    // Допустимые значения для разных полей
    this.allowedValues = {
      country: new Set(['RU', 'US', 'GB', 'DE', 'FR', 'JP', 'CN']),
      currency: new Set(['RUB', 'USD', 'EUR', 'GBP', 'JPY', 'CNY']),
      language: new Set(['ru', 'en', 'de', 'fr', 'es', 'zh']),
      timezone: new Set(['Europe/Moscow', 'America/New_York', 'Europe/London']),
      theme: new Set(['light', 'dark', 'auto'])
    };

    // Запрещенные слова
    this.bannedWords = new Set([
      'admin', 'root', 'superuser', 'password', 'secret',
      'test', 'demo', 'temp', 'dummy', 'fake'
    ]);
  }

  validate(field, value) {
    if (!this.allowedValues[field]) {
      return { valid: false, error: `Неизвестное поле: ${field}` };
    }

    // Быстрая проверка через Set
    if (this.allowedValues[field].has(value)) {
      return { valid: true };
    }
  }
}

```

```

return {
  valid: false,
  error: `Недопустимое значение для ${field}: ${value}`,
  allowed: Array.from(this.allowedValues[field])
};
}

```

```

containsBannedWords(text) {
  const words = text.toLowerCase().split(/\W+/);
  for (const word of words) {
    if (this.bannedWords.has(word)) {
      return {
        banned: true,
        word,
        suggestion: 'Используйте другое слово'
      };
    }
  }
  return { banned: false };
}
}

```

// Пример 4: Комбинация Map и функций для state machine

```

class StateMachine {
  constructor() {
    this.states = new Map();
    this.currentState = 'idle';
  }

  addState(state, transitions) {
    this.states.set(state, transitions);
    return this;
  }

  transition(event, data) {
    const currentTransitions = this.states.get(this.currentState);

    if (!currentTransitions) {
      throw new Error(`Неизвестное состояние: ${this.currentState}`);
    }

    const transition = currentTransitions[event];

```

```

if (!transition) {
  console.warn(`Событие ${event} не допустимо в состоянии ${this.currentState}`);
  return false;
}

const { target, action } = transition;

// Выполнение действия перехода
if (action) {
  const result = action(data);
  if (result === false) {
    return false; // Действие запретило переход
  }
}

// Обновление состояния
const previousState = this.currentState;
this.currentState = target;

console.log(`Переход: ${previousState} -> ${this.currentState} (по событию: ${event})`);
return true;
}
}

// Использование state machine
const orderMachine = new StateMachine();

orderMachine
  .addState('draft', {
    submit: {
      target: 'pending',
      action: (data) => {
        console.log('Заказ отправлен на проверку');
        return data.items && data.items.length > 0;
      }
    },
    cancel: {
      target: 'cancelled',
      action: () => console.log('Черновик отменен')
    }
  })
  .addState('pending', {
    approve: {

```

```

    target: 'approved',
    action: () => console.log('Заказ одобрен')
  },
  reject: {
    target: 'rejected',
    action: (reason) => console.log(`Заказ отклонен: ${reason}`)
  },
  cancel: {
    target: 'cancelled',
    action: () => console.log('Заказ отменен')
  }
})
.addState('approved', {
  process: {
    target: 'processing',
    action: () => console.log('Начата обработка заказа')
  },
  cancel: {
    target: 'cancelled',
    action: () => console.log('Заказ отменен после одобрения')
  }
})
.addState('processing', {
  complete: {
    target: 'completed',
    action: () => console.log('Заказ завершен')
  },
  fail: {
    target: 'failed',
    action: (error) => console.log(`Ошибка обработки: ${error}`)
  }
})
.addState('completed', {
  // Нет переходов из завершенного состояния
})
.addState('cancelled', {
  // Нет переходов из отмененного состояния
})
.addState('rejected', {
  // Нет переходов из отклоненного состояния
})
.addState('failed', {
  retry: {
    target: 'pending',

```

```
    action: () => console.log('Повторная обработка заказа')
  }
});
```

// Пример переходов

```
orderMachine.transition('submit', { items: ['product1', 'product2'] });
orderMachine.transition('approve');
orderMachine.transition('process');
orderMachine.transition('complete');
```

```
console.log('Финальное состояние:', orderMachine.currentState); // 'completed'
```

5.3. Полиморфизм и стратегии

javascript

// Пример 1: Паттерн Стратегия вместо switch

// Исходный подход со switch

```
class PaymentProcessorOld {
  process(paymentMethod, amount, data) {
    let result;

    switch (paymentMethod) {
      case 'credit_card':
        result = this.processCreditCard(amount, data);
        break;

      case 'paypal':
        result = this.processPayPal(amount, data);
        break;

      case 'bank_transfer':
        result = this.processBankTransfer(amount, data);
        break;

      case 'crypto':
        result = this.processCrypto(amount, data);
        break;

      default:
        throw new Error(`Неизвестный метод оплаты: ${paymentMethod}`);
    }

    return result;
  }
}
```

```

}

processCreditCard(amount, data) {
  // Сложная логика обработки карты
  console.log(`Обработка кредитной карты на сумму ${amount}`);
  return { success: true, transactionId: 'CC-' + Date.now() };
}

processPayPal(amount, data) {
  // Сложная логика PayPal
  console.log(`Обработка PayPal на сумму ${amount}`);
  return { success: true, transactionId: 'PP-' + Date.now() };
}

// ... другие методы
}

// Подход с паттерном Стратегия
class PaymentStrategy {
  process(amount, data) {
    throw new Error('Метод process должен быть реализован');
  }

  validate(data) {
    return true;
  }

  getFee() {
    return 0;
  }
}

class CreditCardStrategy extends PaymentStrategy {
  process(amount, data) {
    const { cardNumber, expiry, cvv, name } = data;

    // Валидация карты
    if (!this.validateCard(cardNumber, expiry, cvv)) {
      throw new Error('Неверные данные карты');
    }

    // Имитация обработки
    console.log(`Обработка карты ${this.maskCardNumber(cardNumber)} на сумму ${amount}`);
  }
}

```



```

    // Возврат результата
    return {
        success: true,
        transactionId: `CC-${Date.now()}`,
        amount: amount + this.getFee(),
        method: 'credit_card'
    };
}

validate(data) {
    return data.cardNumber && data.expiry && data.cvv && data.name;
}

getFee() {
    return 2.5; // 2.5% комиссия
}

validateCard(cardNumber, expiry, cvv) {
    // Простая валидация для примера
    return cardNumber.length === 16 &&
        cvv.length === 3 &&
        /^\d{2}\/\d{2}$/.test(expiry);
}

maskCardNumber(cardNumber) {
    return `**** * ${cardNumber.slice(-4)}`;
}
}

class PayPalStrategy extends PaymentStrategy {
    process(amount, data) {
        const { email } = data;

        if (!this.validateEmail(email)) {
            throw new Error('Неверный email PayPal');
        }

        console.log(`Обработка PayPal (${email}) на сумму ${amount}`);

        return {
            success: true,
            transactionId: `PP-${Date.now()}`,
            amount: amount + this.getFee(),
            method: 'paypal'
        };
    }
}

```

```

    };
}

validate(data) {
    return data.email && this.validateEmail(data.email);
}

getFee() {
    return 3.0; // 3% комиссия
}

validateEmail(email) {
    return /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(email);
}
}

class BankTransferStrategy extends PaymentStrategy {
    process(amount, data) {
        const { accountNumber, bankCode } = data;

        console.log(`Банковский перевод на счет ${accountNumber} на сумму ${amount}`);

        return {
            success: true,
            transactionId: `BT-${Date.now()}`,
            amount: amount,
            method: 'bank_transfer',
            instructions: `Переведите ${amount} на счет ${accountNumber}`
        };
    }

    validate(data) {
        return data.accountNumber && data.bankCode;
    }

    getFee() {
        return 0; // Банковский перевод без комиссии
    }
}

// Контекст, который использует стратегии
class PaymentProcessorNew {
    constructor() {
        this.strategies = new Map();
    }
}

```

```

}

registerStrategy(name, strategy) {
  this.strategies.set(name, strategy);
  return this;
}

process(paymentMethod, amount, data) {
  const strategy = this.strategies.get(paymentMethod);

  if (!strategy) {
    throw new Error(`Неизвестный метод оплаты: ${paymentMethod}`);
  }

  // Валидация данных
  if (!strategy.validate(data)) {
    throw new Error(`Неверные данные для ${paymentMethod}`);
  }

  // Обработка платежа
  return strategy.process(amount, data);
}

// Использование
const processor = new PaymentProcessorNew();

processor
  .registerStrategy('credit_card', new CreditCardStrategy())
  .registerStrategy('paypal', new PayPalStrategy())
  .registerStrategy('bank_transfer', new BankTransferStrategy());

try {
  const result1 = processor.process('credit_card', 1000, {
    cardNumber: '4111111111111111',
    expiry: '12/25',
    cvv: '123',
    name: 'John Doe'
  });
  console.log('Результат 1:', result1);

  const result2 = processor.process('paypal', 500, {
    email: 'user@example.com'
  });
}

```

```
console.log('Результат 2:', result2);

} catch (error) {
    console.error('Ошибка платежа:', error.message);
}

// Пример 2: Фабричный метод с регистрацией стратегий
class NotificationFactory {
    constructor() {
        this.providers = new Map();
    }

    registerProvider(type, providerClass) {
        this.providers.set(type, providerClass);
        return this;
    }

    createNotification(type, config) {
        const ProviderClass = this.providers.get(type);

        if (!ProviderClass) {
            throw new Error(`Неизвестный тип уведомления: ${type}`);
        }

        return new ProviderClass(config);
    }
}

class NotificationProvider {
    constructor(config) {
        this.config = config;
    }

    send(message, recipient) {
        throw new Error('Метод send должен быть реализован');
    }

    validate(recipient) {
        return true;
    }
}

class EmailProvider extends NotificationProvider {
    send(message, recipient) {
```

```

    console.log(`Отправка email на ${recipient}: ${message}`);
    // Реальная логика отправки email
    return { success: true, provider: 'email' };
}

validate(recipient) {
    return /^[^s@]+@[^s@]+\.[^s@]+$/.test(recipient);
}
}

class SMSProvider extends NotificationProvider {
    send(message, recipient) {
        console.log(`Отправка SMS на ${recipient}: ${message}`);
        // Реальная логика отправки SMS
        return { success: true, provider: 'sms' };
    }

    validate(recipient) {
        return /^[0-9]{10,11}$/.test(recipient);
    }
}

class PushProvider extends NotificationProvider {
    send(message, recipient) {
        console.log(`Отправка push на устройство ${recipient}: ${message}`);
        // Реальная логика отправки push-уведомлений
        return { success: true, provider: 'push' };
    }
}

class WebhookProvider extends NotificationProvider {
    send(message, recipient) {
        console.log(`Отправка webhook на ${recipient}: ${message}`);
        // Реальная логика отправки webhook
        return { success: true, provider: 'webhook' };
    }
}

// Использование фабрики
const notificationFactory = new NotificationFactory();

notificationFactory
    .registerProvider('email', EmailProvider)
    .registerProvider('sms', SMSProvider)

```

```

    .registerProvider('push', PushProvider)
    .registerProvider('webhook', WebhookProvider);

// Создание и использование провайдеров
const emailNotifier = notificationFactory.createNotification('email', {
    smtpServer: 'smtp.example.com',
    from: 'noreply@example.com'
});

const smsNotifier = notificationFactory.createNotification('sms', {
    apiKey: 'sms-api-key',
    sender: 'ExampleApp'
});

// Отправка уведомлений
emailNotifier.send('Добро пожаловать!', 'user@example.com');
smsNotifier.send('Ваш код: 123456', '+79161234567');

// Пример 3: Динамический выбор стратегии
class ShippingCalculator {
    constructor() {
        this.calculators = new Map();
    }

    registerCalculator(type, calculator) {
        this.calculators.set(type, calculator);
        return this;
    }

    calculate(order, options = {}) {
        // Определение оптимальной стратегии
        const strategy = this.selectStrategy(order, options);

        if (!strategy) {
            throw new Error('Не удалось определить стратегию доставки');
        }

        const calculator = this.calculators.get(strategy);

        if (!calculator) {
            throw new Error(`Неизвестный калькулятор для стратегии: ${strategy}`);
        }

        return calculator.calculate(order, options);
    }
}

```

```
}
```

```
selectStrategy(order, options) {  
  // Сложная логика выбора стратегии  
  const { weight, dimensions, destination, urgency } = order;  
  
  if (urgency === 'express') {  
    return 'express';  
  }  
  
  if (weight > 50) {  
    return 'freight';  
  }  
  
  if (dimensions.volume > 1) {  
    return 'oversized';  
  }  
  
  if (destination.country !== 'RU') {  
    return 'international';  
  }  
  
  return 'standard';  
}  
}
```

```
// Регистрация калькуляторов
```

```
const calculator = new ShippingCalculator();
```

```
calculator
```

```
.registerCalculator('standard', {  
  calculate: (order) => {  
    const base = 300;  
    const weightCost = order.weight * 10;  
    return base + weightCost;  
  }  
})  
.registerCalculator('express', {  
  calculate: (order) => {  
    const standardCost = calculator.calculators.get('standard').calculate(order);  
    return standardCost * 2;  
  }  
})  
.registerCalculator('international', {
```

```

    calculate: (order) => {
      const standardCost = calculator.calculators.get('standard').calculate(order);
      const distanceCost = order.destination.distance * 0.5;
      return standardCost * 1.5 + distanceCost;
    }
  })
  .registerCalculator('freight', {
    calculate: (order) => {
      return order.weight * 100;
    }
  })
  .registerCalculator('oversized', {
    calculate: (order) => {
      return order.dimensions.volume * 500;
    }
  });

// Расчет стоимости доставки
const order1 = {
  weight: 5,
  dimensions: { volume: 0.5 },
  destination: { country: 'RU', distance: 100 },
  urgency: 'standard'
};

const order2 = {
  weight: 60,
  dimensions: { volume: 2 },
  destination: { country: 'US', distance: 5000 },
  urgency: 'express'
};

console.log('Стоимость доставки 1:', calculator.calculate(order1));
console.log('Стоимость доставки 2:', calculator.calculate(order2));

```

Часть 6: Лучшие практики и рекомендации

6.1. Когда использовать switch-case

javascript

// КРИТЕРИИ ВЫБОРА МЕЖДУ SWITCH-CASE И АЛЬТЕРНАТИВАМИ

//  ИСПОЛЬЗОВАТЬ SWITCH-CASE КОГДА:

// 1. Проверка одного значения на множество точных совпадений

```
function getHttpStatusMessage(statusCode) {  
  // Идеальный случай для switch  
  switch (statusCode) {  
    case 200: return "OK";  
    case 201: return "Created";  
    case 400: return "Bad Request";  
    case 401: return "Unauthorized";  
    case 403: return "Forbidden";  
    case 404: return "Not Found";  
    case 500: return "Internal Server Error";  
    default: return "Unknown Status";  
  }  
}
```

// 2. Группировка нескольких значений для одного действия

```
function isWeekend(dayNumber) {  
  // Группировка case без break (fall-through)  
  switch (dayNumber) {  
    case 6: // Суббота  
    case 7: // Воскресенье  
      return true;  
    default:  
      return false;  
  }  
}
```

// 3. Когда нужна производительность для многих условий

// JavaScript движки оптимизируют switch как jump table

```
function optimizeWithSwitch(value) {  
  switch (value) {  
    case 1: return performAction1();  
    case 2: return performAction2();  
    case 3: return performAction3();  
    // ... много case ...  
    case 100: return performAction100();  
    default: return performDefault();  
  }  
}
```

// 4. Когда логика проста и прямолинейна

```
function getDirection(degrees) {
  switch (true) {
    case (degrees >= 0 && degrees < 45):
      return "Север";
    case (degrees >= 45 && degrees < 135):
      return "Восток";
    case (degrees >= 135 && degrees < 225):
      return "Юг";
    case (degrees >= 225 && degrees < 315):
      return "Запад";
    default:
      return "Север";
  }
}
```

// ✗ НЕ ИСПОЛЬЗОВАТЬ SWITCH-CASE КОГДА:

// 1. Условия сложные и включают разные переменные

```
function shouldShowFeature(user, feature) {
  // if-else лучше для сложных условий
  if (!user.isActive) return false;
  if (user.role === 'admin') return true;
  if (feature.beta && user.isBetaTester) return true;
  if (feature.region && feature.region !== user.region) return false;
  return feature.defaultEnabled;
}
```

// 2. Нужна динамическая диспетчеризация или полиморфизм

```
class Animal {
  makeSound() {
    // Полиморфизм лучше switch
    throw new Error('Метод должен быть переопределен');
  }
}
```

```
class Dog extends Animal {
  makeSound() {
    return "Гав!";
  }
}
```

```
class Cat extends Animal {
  makeSound() {
    return "Мяу!";
  }
}
```

```
}  
}
```

// 3. Много повторяющегося кода в case блоках

```
function processDataOld(dataType, data) {  
  switch (dataType) {  
    case 'json':  
      // Много кода...  
      const parsed = JSON.parse(data);  
      validateJson(parsed);  
      transformJson(parsed);  
      return saveJson(parsed);  
  
    case 'xml':  
      // Тот же код, но для XML...  
      const xmlDoc = parseXml(data);  
      validateXml(xmlDoc);  
      transformXml(xmlDoc);  
      return saveXml(xmlDoc);  
  
    case 'csv':  
      // И снова тот же код...  
      const csvData = parseCsv(data);  
      validateCsv(csvData);  
      transformCsv(csvData);  
      return saveCsv(csvData);  
  }  
}
```

// Лучше: вынести общую логику

```
function processDataNew(dataType, data) {  
  const parser = getParser(dataType);  
  const validator = getValidator(dataType);  
  const transformer = getTransformer(dataType);  
  const saver = getSaver(dataType);  
  
  const parsed = parser(data);  
  validator(parsed);  
  const transformed = transformer(parsed);  
  return saver(transformed);  
}
```

// 4. Когда значения для проверки динамические или неизвестны на этапе написания кода

```
const dynamicConditions = {
```

```

    'condition_1': () => { /* логика */ },
    'condition_2': () => { /* логика */ },
    // Условия могут добавляться динамически
  };

function handleDynamicCondition(conditionName, data) {
  const handler = dynamicConditions[conditionName];
  if (handler) {
    return handler(data);
  }
  throw new Error(`Неизвестное условие: ${conditionName}`);
}

```

6.2. Советы по написанию качественного switch-case

```

javascript

// СОВЕТ 1: Всегда добавляйте default case
function handleStatus(status) {
  switch (status) {
    case 'success':
      return 'Операция успешна';
    case 'error':
      return 'Произошла ошибка';
    case 'pending':
      return 'Ожидание';
    // ВАЖНО: default для непредусмотренных случаев
    default:
      console.warn(`Неизвестный статус: ${status}`);
      return 'Неизвестный статус';
  }
}

// СОВЕТ 2: Используйте комментарии для fall-through
function getSeason(month) {
  switch (month) {
    case 12:
    case 1:
    case 2:
      return 'Зима';

    case 3:
    case 4:
    case 5:

```

```

        return 'Весна';

    case 6:
    case 7:
    case 8:
        return 'Лето';

    case 9:
    case 10:
    case 11:
        return 'Осень';

    default:
        throw new Error(`Неверный месяц: ${month}`);
    }
}

```

// СОВЕТ 3: Выносите сложную логику из case блоков
// ПЛОХО:

```

function processUserBad(role) {
    switch (role) {
        case 'admin':
            // Много сложной логики
            setupAdminDashboard();
            loadAdminData();
            configureAdminPermissions();
            sendAdminNotifications();
            return 'Администратор настроен';

        case 'user':
            // Еще больше логики
            setupUserDashboard();
            loadUserData();
            configureUserPermissions();
            sendWelcomeEmail();
            return 'Пользователь настроен';
        }
    }
}

```

// ХОРОШО:

```

function processUserGood(role) {
    switch (role) {
        case 'admin':
            return setupAdmin();
    }
}

```

```

    case 'user':
        return setupUser();

    default:
        throw new Error(`Неизвестная роль: ${role}`);
}
}

function setupAdmin() {
    setupAdminDashboard();
    loadAdminData();
    configureAdminPermissions();
    sendAdminNotifications();
    return 'Администратор настроен';
}

function setupUser() {
    setupUserDashboard();
    loadUserData();
    configureUserPermissions();
    sendWelcomeEmail();
    return 'Пользователь настроен';
}

// СОВЕТ 4: Избегайте вложенных switch
// ПЛОХО:
function processOrderBad(orderType, status) {
    switch (orderType) {
        case 'online':
            switch (status) {
                case 'pending':
                    return processOnlinePending();
                case 'paid':
                    return processOnlinePaid();
                default:
                    throw new Error(`Неизвестный статус для online: ${status}`);
            }

        case 'offline':
            switch (status) {
                case 'pending':
                    return processOfflinePending();
                case 'paid':

```

```

        return processOfflinePaid();
    default:
        throw new Error(`Неизвестный статус для offline: ${status}`);
    }
}
}

```

// ХОРОШО: Используйте объекты или другие структуры

```

const orderProcessors = {
  online: {
    pending: () => processOnlinePending(),
    paid: () => processOnlinePaid()
  },
  offline: {
    pending: () => processOfflinePending(),
    paid: () => processOfflinePaid()
  }
};

function processOrderGood(orderType, status) {
  const processor = orderProcessors[orderType]?.[status];

  if (!processor) {
    throw new Error(`Неизвестная комбинация: ${orderType}/${status}`);
  }

  return processor();
}

```

// СОВЕТ 5: Используйте enum или константы для case значений

// Определяем константы

```

const UserRoles = {
  ADMIN: 'admin',
  MODERATOR: 'moderator',
  EDITOR: 'editor',
  USER: 'user',
  GUEST: 'guest'
};

const PermissionLevels = {
  FULL: 'full',
  LIMITED: 'limited',
  READ_ONLY: 'read_only',
  NONE: 'none'
}

```

```
};
```

```
// Используем константы в switch
```

```
function getPermissions(role) {  
  switch (role) {  
    case UserRoles.ADMIN:  
      return PermissionLevels.FULL;  
  
    case UserRoles.MODERATOR:  
    case UserRoles.EDITOR:  
      return PermissionLevels.LIMITED;  
  
    case UserRoles.USER:  
      return PermissionLevels.READ_ONLY;  
  
    case UserRoles.GUEST:  
    default:  
      return PermissionLevels.NONE;  
  }  
}
```

```
// Преимущества:
```

```
// 1. Автодополнение в IDE
```

```
// 2. Защита от опечаток
```

```
// 3. Легче рефакторить
```

```
// 4. Самодокументирующийся код
```

```
// СОВЕТ 6: Рассмотрите возможность использования TypeScript для exhaustive checks
```

```
// В TypeScript можно гарантировать обработку всех case
```

```
type Status = 'success' | 'error' | 'pending';
```

```
function handleStatusTS(status: Status): string {  
  switch (status) {  
    case 'success':  
      return 'Успешно';  
    case 'error':  
      return 'Ошибка';  
    case 'pending':  
      return 'Ожидание';  
    // TypeScript предупредит, если добавить новый статус в тип  
    // и не обработать его здесь  
  }  
}
```


// СОВЕТ 7: Измеряйте производительность для критичных участков

```
function benchmarkSwitchVsObject() {
  const iterations = 1000000;
  const testValue = 'case_50';

  // Подготовка switch
  function testSwitch(value) {
    switch (value) {
      case 'case_1': return 1;
      case 'case_2': return 2;
      // ... много case ...
      case 'case_50': return 50;
      default: return 0;
    }
  }

  // Подготовка объекта
  const caseMap = {};
  for (let i = 1; i <= 50; i++) {
    caseMap[`case_${i}`] = i;
  }

  function testObject(value) {
    return caseMap[value] || 0;
  }

  // Бенчмарк switch
  console.time('switch');
  let sum1 = 0;
  for (let i = 0; i < iterations; i++) {
    sum1 += testSwitch(testValue);
  }
  console.timeEnd('switch');

  // Бенчмарк объекта
  console.time('object');
  let sum2 = 0;
  for (let i = 0; i < iterations; i++) {
    sum2 += testObject(testValue);
  }
  console.timeEnd('object');

  console.log('Результаты равны:', sum1 === sum2);
}
```

```
// Запуск бенчмарка
// benchmarkSwitchVsObject();
```

6.3. Антипаттерны и как их избежать

javascript

```
// АНТИПАТТЕРН 1: Switch внутри цикла (без необходимости)
```

```
// ПЛОХО:
```

```
function processItemsBad(items) {
  const results = [];

  for (const item of items) {
    let processed;

    switch (item.type) {
      case 'A':
        processed = processTypeA(item);
        break;
      case 'B':
        processed = processTypeB(item);
        break;
      case 'C':
        processed = processTypeC(item);
        break;
    }

    results.push(processed);
  }

  return results;
}
```

```
// ХОРОШО: Вынесите switch в отдельную функцию
```

```
function processItem(item) {
  switch (item.type) {
    case 'A':
      return processTypeA(item);
    case 'B':
      return processTypeB(item);
    case 'C':
      return processTypeC(item);
    default:
```

```
        throw new Error(`Неизвестный тип: ${item.type}`);
    }
}
```

```
function processItemsGood(items) {
    return items.map(processItem);
}
```

// ЕЩЕ ЛУЧШЕ: Используйте Map для обработчиков

```
const itemProcessors = new Map([
    ['A', processTypeA],
    ['B', processTypeB],
    ['C', processTypeC]
]);
```

```
function processItemsBest(items) {
    return items.map(item => {
        const processor = itemProcessors.get(item.type);
        if (!processor) {
            throw new Error(`Неизвестный тип: ${item.type}`);
        }
        return processor(item);
    });
}
```

// АНТИПАТТЕРН 2: Изменение переменных в разных case

// ПЛОХО:

```
let result; // Глобальная переменная
```

```
function calculateBad(type, a, b) {
    switch (type) {
        case 'add':
            result = a + b; // Изменение внешней переменной
            break;
        case 'subtract':
            result = a - b; // Еще одно изменение
            break;
    }

    return result;
}
```

// ХОРОШО: Возвращайте значения напрямую

```
function calculateGood(type, a, b) {
```

```
switch (type) {
  case 'add':
    return a + b; // Прямой возврат
  case 'subtract':
    return a - b; // Прямой возврат
  default:
    throw new Error(`Неизвестная операция: ${type}`);
}
}
```

// АНТИПАТТЕРН 3: Слишком длинный switch

// ПЛОХО: Switch с 50+ case

```
function handleTooManyCases(code) {
  switch (code) {
    case 100: return handle100();
    case 101: return handle101();
    case 102: return handle102();
    // ... 47 строк ...
    case 150: return handle150();
    default: return handleDefault();
  }
}
```

// ХОРОШО: Разбейте на логические группы

```
function handleWithGroups(code) {
  // Группа 1: Информационные коды (100-199)
  if (code >= 100 && code < 200) {
    return handleInformational(code);
  }

  // Группа 2: Успешные коды (200-299)
  if (code >= 200 && code < 300) {
    return handleSuccess(code);
  }

  // Группа 3: Ошибки (300-599)
  if (code >= 300 && code < 600) {
    return handleError(code);
  }

  return handleUnknown(code);
}
```

```
function handleInformational(code) {
```

```

// Вложенный switch для группы
switch (code) {
  case 100: return handle100();
  case 101: return handle101();
  case 102: return handle102();
  default: return handleDefaultInformational();
}
}

// ИЛИ используйте динамическую диспетчеризацию
const codeHandlers = new Map();
for (let i = 100; i <= 150; i++) {
  codeHandlers.set(i, () => eval(`handle${i}`)());
}

function handleDynamic(code) {
  const handler = codeHandlers.get(code);
  return handler ? handler() : handleDefault();
}

// АНТИПАТТЕРН 4: Switch для проверки типов
// ПЛОХО:
function processValueBad(value) {
  switch (typeof value) {
    case 'string':
      return processString(value);
    case 'number':
      return processNumber(value);
    case 'object':
      if (Array.isArray(value)) {
        return processArray(value);
      } else if (value === null) {
        return processNull();
      } else {
        return processObject(value);
      }
    default:
      return processUnknown(value);
  }
}

// ХОРОШО: Используйте полиморфизм или стратегии
class ValueProcessor {
  process(value) {

```

```

        throw new Error('Метод должен быть переопределен');
    }
}

```

```

class StringProcessor extends ValueProcessor {
    process(value) {
        return processString(value);
    }
}

```

```

class NumberProcessor extends ValueProcessor {
    process(value) {
        return processNumber(value);
    }
}

```

// Фабрика для создания процессоров

```

function getProcessor(value) {
    if (typeof value === 'string') {
        return new StringProcessor();
    } else if (typeof value === 'number') {
        return new NumberProcessor();
    } else if (Array.isArray(value)) {
        return new ArrayProcessor();
    } else if (value === null) {
        return new NullProcessor();
    } else if (typeof value === 'object') {
        return new ObjectProcessor();
    } else {
        return new UnknownProcessor();
    }
}

```

```

function processValueGood(value) {
    const processor = getProcessor(value);
    return processor.process(value);
}

```

// АНТИПАТТЕРН 5: Игнорирование default

// ОЧЕНЬ ПЛОХО:

```

function dangerousFunction(status) {
    switch (status) {
        case 'ok':
            return 'Все хорошо';
    }
}

```

```

    case 'error':
        return 'Ошибка';
    // НЕТ DEFAULT! Что будет для status = 'warning'?
}
}

// ХОРОШО: Всегда обрабатывайте непредусмотренные случаи
function safeFunction(status) {
    switch (status) {
        case 'ok':
            return 'Все хорошо';
        case 'error':
            return 'Ошибка';
        default:
            // Логируем непредусмотренные значения
            console.warn(`Неизвестный статус: ${status}`);
            // Возвращаем безопасное значение по умолчанию
            return 'Неизвестный статус';
            // ИЛИ выбрасываем исключение
            // throw new Error(`Неизвестный статус: ${status}`);
    }
}

// ЕЩЕ ЛУЧШЕ: Используйте TypeScript для статической проверки
type StatusType = 'ok' | 'error'; // Только допустимые значения

function typeSafeFunction(status: StatusType) {
    switch (status) {
        case 'ok':
            return 'Все хорошо';
        case 'error':
            return 'Ошибка';
        // TypeScript гарантирует, что все case обработаны
    }
}

```

Заключение: ключевые принципы эффективного использования switch-case

7.1. Основные выводы

1. **Switch-case** идеален для проверки одного значения на множество точных совпадений
2. **Всегда используйте break** (кроме intentional fall-through)
3. **Всегда добавляйте default case** для обработки непредусмотренных значений
4. **Для сложных условий используйте switch(true)**
5. **Рассмотрите альтернативы** (объекты, Map, полиморфизм) для сложных сценариев

7.2. Практические рекомендации

javascript

// ШАБЛОН ДЛЯ КАЧЕСТВЕННОГО SWITCH-CASE:

```
function highQualitySwitchExample(value) {  
  // 1. Валидация входных данных (опционально)  
  if (value == null) {  
    throw new Error('Значение не может быть null или undefined');  
  }  
  
  // 2. Основной switch  
  switch (value) {  
    // 3. Группируйте связанные case  
    case 'option1':  
    case 'option2':  
      return handleGroup1(value);  
  
    case 'option3':  
    case 'option4':  
      return handleGroup2(value);  
  
    // 4. Используйте константы для значений  
    case CONSTANTS.SPECIAL_OPTION:  
      return handleSpecialOption(value);  
  
    // 5. Всегда добавляйте default  
    default:
```



```

// 6. Логируйте непредусмотренные значения
console.warn(`Неизвестное значение: ${value}`);

// 7. Возвращайте безопасное значение по умолчанию
return handleDefault(value);

// ИЛИ выбрасывайте исключение
// throw new Error(`Неизвестное значение: ${value}`);
}
}

```

// КОГДА ВЫБИРАТЬ SWITCH-CASE VS IF-ELSE:

```
const decisionMatrix = `
ВЫБОР КОНСТРУКЦИИ ДЛЯ УСЛОВНОЙ ЛОГИКИ:
```

Критерий	switch-case	if-else
Одно проверяемое значение	□ ИДЕАЛЬНО	□ Можно
Точное равенство (===)	□ ОТЛИЧНО	□ ОТЛИЧНО
Диапазоны значений	▲ Через true	▣ ОТЛИЧНО
Много условий (>3)	□ ОТЛИЧНО	□ Громоздко
Группировка case	□ ОТЛИЧНО	▣ Сложно
Производительность	□ Быстрее (для многих case)	□ Медленнее (для многих условий)
Читаемость	□ Структурировано	□ Может быть запутанным
Расширяемость	▲ Нужно добавлять новые case	▣ Легко добавить новые условия

```
`;
```

```
console.log(decisionMatrix);
```

7.3. Финальные примеры хорошо структурированного кода

javascript

// Пример 1: Комплексная обработка HTTP запросов

```
class HttpRequestHandler {
  async handleRequest(request) {
    const { method, url, headers, body } = request;

    // Определение типа контента
    const contentType = headers['content-type'] || 'application/octet-stream';

    let parsedBody;
    switch (true) {
      case contentType.includes('application/json'):
        parsedBody = await this.parseJsonBody(body);
        break;

      case contentType.includes('application/x-www-form-urlencoded'):
        parsedBody = await this.parseFormBody(body);
        break;

      case contentType.includes('multipart/form-data'):
        parsedBody = await this.parseMultipartBody(body, headers);
        break;

      default:
        parsedBody = body;
    }

    // Обработка метода
    switch (method) {
      case 'GET':
        return await this.handleGet(url, parsedBody, headers);

      case 'POST':
        return await this.handlePost(url, parsedBody, headers);

      case 'PUT':
        return await this.handlePut(url, parsedBody, headers);

      case 'DELETE':
        return await this.handleDelete(url, parsedBody, headers);

      case 'PATCH':
        return await this.handlePatch(url, parsedBody, headers);

      case 'OPTIONS':
```

```

        return await this.handleOptions(url, headers);

    default:
        return this.createResponse(405, {
            error: `Метод ${method} не поддерживается`
        });
    }
}

async handleGet(url, data, headers) {
    const route = this.router.match('GET', url);

    if (!route) {
        return this.createResponse(404, { error: 'Не найдено' });
    }

    switch (route.type) {
        case 'static':
            return await this.serveStaticFile(route.path);

        case 'api':
            return await this.handleApiCall(route, data, headers);

        case 'view':
            return await this.renderView(route.template, data);

        case 'redirect':
            return this.createRedirect(route.target);

        default:
            return this.createResponse(500, {
                error: 'Неизвестный тип роута'
            });
    }
}

createResponse(status, data) {
    return {
        status,
        headers: {
            'Content-Type': 'application/json',
            'Cache-Control': 'no-cache'
        },
        body: JSON.stringify(data)
    };
}

```

```

    };
}

createRedirect(url, permanent = false) {
    return {
        status: permanent ? 308 : 307,
        headers: {
            'Location': url
        },
        body: ''
    };
}
}
}

```

// Пример 2: Система плагинов с switch-case

```

class PluginManager {
    constructor() {
        this.plugins = new Map();
        this.hooks = new Map();
    }

    registerPlugin(name, plugin) {
        this.plugins.set(name, plugin);
        this.initializePlugin(name, plugin);
    }

    initializePlugin(name, plugin) {
        switch (plugin.type) {
            case 'ui-widget':
                this.initializeUIWidget(name, plugin);
                break;

            case 'api-extension':
                this.initializeAPIExtension(name, plugin);
                break;

            case 'data-processor':
                this.initializeDataProcessor(name, plugin);
                break;

            case 'event-listener':
                this.initializeEventListener(name, plugin);
                break;
        }
    }
}

```

```

    case 'middleware':
        this.initializeMiddleware(name, plugin);
        break;

    default:
        console.warn(`Неизвестный тип плагина: ${plugin.type}`);
}
}

```

```

initializeUIWidget(name, plugin) {
    const { component, position, priority } = plugin;

    switch (position) {
        case 'header':
            this.registerHeaderWidget(component, priority);
            break;

        case 'sidebar':
            this.registerSidebarWidget(component, priority);
            break;

        case 'footer':
            this.registerFooterWidget(component, priority);
            break;

        case 'modal':
            this.registerModal(component);
            break;

        case 'toolbar':
            this.registerToolbarItem(component, priority);
            break;

        default:
            console.warn(`Неизвестная позиция: ${position}`);
    }
}

```

```

handleHook(hookName, data) {
    const hookHandlers = this.hooks.get(hookName) || [];

    let result = data;
    for (const handler of hookHandlers) {
        const handlerResult = handler(result);
    }
}

```

```

switch (handlerResult.type) {
  case 'continue':
    result = handlerResult.data;
    break;

  case 'stop':
    return handlerResult.data;

  case 'error':
    throw new Error(handlerResult.message);

  case 'async':
    // Обработка асинхронных результатов
    return this.handleAsyncHook(handlerResult.promise);

  default:
    console.warn(`Неизвестный тип результата: ${handlerResult.type}`);
}
}

return result;
}

handleAsyncHook(promise) {
  return promise
    .then(result => {
      switch (result.status) {
        case 'success':
          return result.data;

        case 'partial':
          console.warn('Частичный успех:', result.message);
          return result.data;

        case 'failed':
          throw new Error(`Ошибка: ${result.message}`);

        default:
          throw new Error(`Неизвестный статус: ${result.status}`);
      }
    })
    .catch(error => {
      switch (error.type) {

```

```

    case 'network':
      console.error('Сетевая ошибка:', error.message);
      return this.getFallbackData();

    case 'validation':
      console.error('Ошибка валидации:', error.message);
      throw error;

    case 'permission':
      console.error('Ошибка доступа:', error.message);
      return this.getUnauthorizedResponse();

    default:
      console.error('Неизвестная ошибка:', error);
      throw error;
  }
});
}
}

```

// Пример 3: State machine с улучшенным switch-case

```

class ImprovedStateMachine {
  constructor() {
    this.state = 'idle';
    this.history = [];
    this.transitions = this.defineTransitions();
  }

  defineTransitions() {
    return {
      idle: {
        start: { target: 'running', guard: this.canStart.bind(this) },
        error: { target: 'error', immediate: true }
      },
      running: {
        pause: { target: 'paused' },
        complete: { target: 'completed' },
        error: { target: 'error' }
      },
      paused: {
        resume: { target: 'running' },
        stop: { target: 'stopped' }
      },
      completed: {

```

```

        reset: { target: 'idle' }
    },
    error: {
        retry: { target: 'idle', guard: this.canRetry.bind(this) },
        reset: { target: 'idle' }
    },
    stopped: {
        reset: { target: 'idle' }
    }
};
}

```

```

dispatch(event, data) {
    const currentTransitions = this.transitions[this.state];

    if (!currentTransitions) {
        throw new Error(`Неизвестное состояние: ${this.state}`);
    }

    const transition = currentTransitions[event];

    if (!transition) {
        console.warn(`Событие ${event} не допустимо в состоянии ${this.state}`);
        return false;
    }

    // Проверка guard условий
    if (transition.guard && !transition.guard(data)) {
        console.warn(`Guard условие не выполнено для перехода ${event}`);
        return false;
    }

    // Выполнение перехода
    const previousState = this.state;
    this.state = transition.target;

    // Логирование истории
    this.history.push({
        timestamp: Date.now(),
        from: previousState,
        to: this.state,
        event,
        data
    });
}

```



```

// Вызов обработчиков
this.callTransitionHandlers(previousState, this.state, event, data);

return true;
}

callTransitionHandlers(from, to, event, data) {
  // Глобальные обработчики
  this.callHandler('onAnyTransition', from, to, event, data);

  // Обработчики конкретных переходов
  switch (`${from}-${to}`) {
    case 'idle-running':
      this.callHandler('onStart', data);
      break;

    case 'running-paused':
      this.callHandler('onPause', data);
      break;

    case 'paused-running':
      this.callHandler('onResume', data);
      break;

    case 'running-completed':
      this.callHandler('onComplete', data);
      break;

    case '*-error':
      this.callHandler('onError', data);
      break;

    case 'error-idle':
      this.callHandler('onRetry', data);
      break;

    case '*-idle':
      this.callHandler('onReset', data);
      break;
  }

  // Обработчики состояний
  switch (to) {

```

```

    case 'running':
      this.callHandler('onRunning', data);
      break;

    case 'paused':
      this.callHandler('onPaused', data);
      break;

    case 'completed':
      this.callHandler('onCompleted', data);
      break;

    case 'error':
      this.callHandler('onErrorState', data);
      break;
  }
}

callHandler(handlerName, ...args) {
  const handler = this[handlerName];
  if (handler && typeof handler === 'function') {
    return handler(...args);
  }
}

canStart(data) {
  switch (data?.type) {
    case 'immediate':
      return true;

    case 'scheduled':
      return data.timestamp <= Date.now();

    case 'conditional':
      return data.condition === true;

    default:
      return false;
  }
}

canRetry(data) {
  if (data?.maxRetries && data.retryCount >= data.maxRetries) {
    return false;
  }
}

```

```

    }

    switch (data?.errorType) {
      case 'temporary':
        return true;

      case 'permanent':
        return false;

      case 'recoverable':
        return data.canRecover === true;

      default:
        return true;
    }
  }
}

// Инициализация и использование
const machine = new ImprovedStateMachine();

// Регистрация обработчиков
machine.onStart = (data) => {
  console.log('Запуск с данными:', data);
};

machine.onError = (data) => {
  console.error('Ошибка:', data.error);
};

// Диспетчеризация событий
machine.dispatch('start', { type: 'immediate' });
machine.dispatch('pause', { reason: 'user_action' });
machine.dispatch('resume', {});
machine.dispatch('complete', { result: 'success' });

console.log('Финальное состояние:', machine.state);
console.log('История переходов:', machine.history);

```

Итог: Конструкция `switch-case` — это мощный инструмент в арсенале JavaScript-разработчика. При правильном использовании она делает код более читаемым, структурированным и производительным. Однако важно понимать её ограничения и знать альтернативы для сложных сценариев. Помните: лучший инструмент — тот, который лучше всего подходит для конкретной

задачи. Используйте `switch-case` там, где он действительно уместен, и не бойтесь использовать другие подходы, когда они лучше соответствуют требованиям вашего кода.

Введение: что такое циклы и зачем они нужны

Циклы — это фундаментальная конструкция программирования, позволяющая выполнять один и тот же блок кода **многократно**, пока выполняется заданное условие. Они являются основой для обработки коллекций данных, автоматизации повторяющихся задач и реализации сложной логики.

Аналогия из жизни:

Представьте, что вам нужно пересчитать все книги на полке. Вы берёте первую книгу, говорите «один», кладёте её, берёте следующую, говорите «два», и так до тех пор, пока книги не закончатся. Этот процесс — и есть цикл.

Ключевые компоненты любого цикла:

1. **Инициализация** — подготовка перед началом цикла
 2. **Условие продолжения** — проверка, нужно ли выполнять очередную итерацию
 3. **Тело цикла** — код, который выполняется на каждой итерации
 4. **Изменение состояния** — обновление переменных для движения к завершению цикла
-

Часть 1: Цикл `for` — классический и самый популярный

1.1. Базовый синтаксис цикла `for`

```
javascript
```

```
for (инициализация; условие; изменение) {  
    // тело цикла  
}
```

Подробное объяснение каждого компонента:

javascript

```
for (  
  let i = 0;           // 1. Инициализация: создаём счётчик i = 0  
  i < 5;               // 2. Условие: продолжаем пока i < 5  
  i++                  // 3. Изменение: после каждой итерации i увеличивается на 1  
) {  
  console.log(i);     // 4. Тело: код, выполняемый на каждой итерации  
}
```

Пошаговое выполнение:

text

Итерация 1: i=0 → 0<5 (true) → console.log(0) → i++ → i=1
Итерация 2: i=1 → 1<5 (true) → console.log(1) → i++ → i=2
Итерация 3: i=2 → 2<5 (true) → console.log(2) → i++ → i=3
Итерация 4: i=3 → 3<5 (true) → console.log(3) → i++ → i=4
Итерация 5: i=4 → 4<5 (true) → console.log(4) → i++ → i=5
Итерация 6: i=5 → 5<5 (false) → цикл завершён

1.2. Практические примеры использования for

Пример 1: Перебор массива

javascript

```
const fruits = ["яблоко", "банан", "апельсин", "виноград"];
```

// Классический перебор по индексам

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(`${i + 1}. ${fruits[i]}`);  
}
```

// Вывод:

// 1. яблоко

// 2. банан

// 3. апельсин

// 4. виноград

// Перебор в обратном порядке

```
for (let i = fruits.length - 1; i >= 0; i--) {  
  console.log(fruits[i]);  
}
```

```
// Вывод: виноград, апельсин, банан, яблоко
```

Пример 2: Работа с диапазонами чисел

javascript

```
// Вывод чётных чисел от 0 до 10
```

```
for (let num = 0; num <= 10; num += 2) {  
  console.log(num); // 0, 2, 4, 6, 8, 10  
}
```

```
// Таблица умножения на 7
```

```
console.log("Таблица умножения на 7:");  
for (let multiplier = 1; multiplier <= 10; multiplier++) {  
  const result = 7 * multiplier;  
  console.log(`7 × ${multiplier} = ${result}`);  
}
```

```
// Сумма чисел от 1 до 100
```

```
let totalSum = 0;  
for (let n = 1; n <= 100; n++) {  
  totalSum += n;  
}  
console.log(`Сумма чисел от 1 до 100: ${totalSum}`); // 5050
```

Пример 3: Вложенные циклы

javascript

```
// Создание матрицы 3×3
```

```
for (let row = 0; row < 3; row++) {  
  let rowOutput = "";  
  for (let col = 0; col < 3; col++) {  
    rowOutput += `(${row},${col}) `;  
  }  
  console.log(rowOutput);  
}
```

```
// Вывод:
```

```
// (0,0) (0,1) (0,2)
```

```
// (1,0) (1,1) (1,2)
```

```
// (2,0) (2,1) (2,2)
```

```
// Умножение матриц (упрощённый пример)
```

```
const matrixA = [[1, 2], [3, 4]];  
const matrixB = [[5, 6], [7, 8]];  
const result = [[0, 0], [0, 0]];
```

```

for (let i = 0; i < 2; i++) {
  for (let j = 0; j < 2; j++) {
    for (let k = 0; k < 2; k++) {
      result[i][j] += matrixA[i][k] * matrixB[k][j];
    }
  }
}
console.log("Результат умножения матриц:", result);

```

1.3. Вариации синтаксиса for

Несколько переменных в инициализации:

```

javascript

// Инициализируем две переменные одновременно
for (let i = 0, j = 10; i < j; i++, j--) {
  console.log(`i=${i}, j=${j}`);
}
// Вывод:
// i=0, j=10
// i=1, j=9
// ...
// i=4, j=6

```

Отсутствие инициализации или изменения:

```

javascript

let i = 0;
for (; i < 5; ) { // Инициализация и изменение вынесены наружу
  console.log(i);
  i++;
}

```

Бесконечный цикл (осторожно!):

```

javascript

// Три способа создать бесконечный цикл
for (;;) { // Все три компонента опущены
  console.log("Это будет выполняться вечно!");
  // break; // Используйте break для выхода
}

```



```
for (let i = 0; true; i++) { // Условие всегда true
  if (i > 1000) break;
}

for (let i = 0; i >= 0; i++) { // Условие всегда выполняется
  if (i > 1000) break;
}
```

1.4. Оптимизация производительности циклов for

javascript

```
const largeArray = new Array(1000000).fill(0).map( (_, i) => i);
```

// НЕОПТИМАЛЬНО: length вычисляется на каждой итерации

```
for (let i = 0; i < largeArray.length; i++) {
  // Длина массива пересчитывается 1,000,000 раз
}
```

// ОПТИМАЛЬНО: длина сохраняется в переменную

```
const length = largeArray.length;
for (let i = 0; i < length; i++) {
  // Длина используется из переменной
}
```

// ЕЩЁ ЛУЧШЕ: обратный цикл (иногда быстрее)

```
for (let i = largeArray.length - 1; i >= 0; i--) {
  // Сравнение с 0 быстрее, чем с переменной
}
```

// САМЫЙ БЫСТРЫЙ: цикл с декрементом

```
for (let i = largeArray.length; i--; ) {
  // Условие проверяется после декремента
  // Тело цикла выполняется для i от length-1 до 0
}
```

Часть 2: Цикл `while` — цикл с предусловием

2.1. Базовый синтаксис цикла `while`

```
javascript

while (условие) {
    // тело цикла
}
```

Отличия от `for`:

- `for` — используется, когда известно количество итераций
- `while` — используется, когда количество итераций неизвестно

2.2. Практические примеры использования `while`

Пример 1: Обработка пользовательского ввода

```
javascript

// Эмуляция запроса пароля
let userPassword = "";
let attempts = 0;
const correctPassword = "secret123";

while (userPassword !== correctPassword && attempts < 3) {
    userPassword = prompt(`Введите пароль (попытка ${attempts + 1}/3):`);
    attempts++;

    if (userPassword === correctPassword) {
        console.log("Доступ разрешён!");
    } else if (attempts === 3) {
        console.log("Доступ заблокирован!");
    }
}
```

Пример 2: Чтение данных до определённого условия

```
javascript

// Чтение чисел пока не встретится 0
const numbers = [5, 10, 15, 0, 25, 30];
let index = 0;
```

```
let sum = 0;

while (index < numbers.length && numbers[index] !== 0) {
  sum += numbers[index];
  index++;
  console.log(`Текущая сумма: ${sum}, следующий элемент: ${numbers[index]}`);
}

console.log(`Итоговая сумма до 0: ${sum}`);
```

Пример 3: Игра «Угадай число»

javascript

```
// Генерация случайного числа от 1 до 100
const secretNumber = Math.floor(Math.random() * 100) + 1;
let userGuess = 0;
let attemptsCount = 0;

console.log("Угадайте число от 1 до 100!");

while (userGuess !== secretNumber) {
  userGuess = parseInt(prompt(`Попытка ${attemptsCount + 1}. Ваш вариант:`));
  attemptsCount++;

  if (isNaN(userGuess)) {
    console.log("Пожалуйста, введите число!");
    continue;
  }

  if (userGuess < secretNumber) {
    console.log("Слишком мало!");
  } else if (userGuess > secretNumber) {
    console.log("Слишком много!");
  } else {
    console.log(`Поздравляем! Вы угадали число ${secretNumber} за ${attemptsCount}
попыток!`);
  }
}
```

2.3. Особенности работы с `while`

Предотвращение бесконечных циклов:

```
javascript

// ОПАСНО: бесконечный цикл
let counter = 0;
while (counter < 5) {
  console.log(counter);
  // ЗАБЫЛИ: counter++ ← без этого цикл бесконечный
}

// БЕЗОПАСНО: с чётким условием выхода
let counter = 0;
while (counter < 5) {
  console.log(counter);
  counter++; // Не забываем изменять условие!

  // Дополнительная проверка на крайний случай
  if (counter > 1000) {
    console.error("Возможный бесконечный цикл!");
    break;
  }
}
```

Использование с флагами:

```
javascript

// Поиск простого числа
function isPrime(n) {
  if (n <= 1) return false;
  let divisor = 2;
  let isPrime = true;

  while (divisor <= Math.sqrt(n) && isPrime) {
    if (n % divisor === 0) {
      isPrime = false;
    }
    divisor++;
  }

  return isPrime;
}
```

```
// Поиск первого простого числа в диапазоне
let currentNumber = 100;
let found = false;

while (!found && currentNumber <= 200) {
  if (isPrime(currentNumber)) {
    console.log(`Первое простое число после 100: ${currentNumber}`);
    found = true;
  }
  currentNumber++;
}
```

Часть 3: Цикл `do...while` — цикл с постусловием

3.1. Базовый синтаксис цикла `do...while`

```
javascript

do {
  // тело цикла
} while (условие);
```

Ключевое отличие от `while`:

- `while` — сначала проверяет условие, потом выполняет тело
- `do...while` — сначала выполняет тело, потом проверяет условие

3.2. Практические примеры использования `do...while`

Пример 1: Меню с выбором действия

```
javascript

let userChoice;
let isRunning = true;

do {
  // Вывод меню
  console.log("\n=== МЕНЮ ===");
  console.log("1. Показать текущую дату");
```

```

console.log("2. Показать случайное число");
console.log("3. Выйти");

// Получение выбора пользователя
userChoice = prompt("Выберите действие (1-3):");

// Обработка выбора
switch (userChoice) {
  case "1":
    console.log("Текущая дата:", new Date().toLocaleDateString());
    break;
  case "2":
    console.log("Случайное число:", Math.random());
    break;
  case "3":
    console.log("Выход из программы...");
    isRunning = false;
    break;
  default:
    console.log("Неверный выбор! Попробуйте снова.");
}
} while (isRunning);

```

Пример 2: Валидация пользовательского ввода

```

javascript

let age;
let isValidInput = false;

do {
  const input = prompt("Введите ваш возраст (от 1 до 120):");
  age = parseInt(input);

  // Проверка валидности
  if (isNaN(age)) {
    console.log("Пожалуйста, введите число!");
  } else if (age < 1 || age > 120) {
    console.log("Возраст должен быть от 1 до 120 лет!");
  } else {
    isValidInput = true;
  }
} while (!isValidInput);

console.log(`Ваш возраст: ${age} лет`);

```

Пример 3: Имитация работы банкомата

javascript

```
let balance = 1000; // Начальный баланс
let continueTransaction;

do {
  console.log(`\nТекущий баланс: ${balance}`);
  console.log("1. Снять наличные");
  console.log("2. Внести средства");
  console.log("3. Проверить баланс");
  console.log("4. Выйти");

  const choice = prompt("Выберите операцию:");
  let amount;

  switch (choice) {
    case "1":
      amount = parseFloat(prompt("Сумма для снятия:"));
      if (amount <= balance) {
        balance -= amount;
        console.log(`Снято: ${amount}. Новый баланс: ${balance}`);
      } else {
        console.log("Недостаточно средств!");
      }
      break;

    case "2":
      amount = parseFloat(prompt("Сумма для внесения:"));
      if (amount > 0) {
        balance += amount;
        console.log(`Внесено: ${amount}. Новый баланс: ${balance}`);
      } else {
        console.log("Неверная сумма!");
      }
      break;

    case "3":
      console.log(`Ваш баланс: ${balance}`);
      break;

    case "4":
      console.log("Спасибо за использование банкомата!");
```

```

    continueTransaction = false;
    break;

default:
    console.log("Неверный выбор операции!");
}

// Спрашиваем, хочет ли пользователь продолжить
if (choice !== "4") {
    const response = prompt("Выполнить ещё одну операцию? (да/нет)").toLowerCase();
    continueTransaction = response === "да" || response === "yes";
}
} while (continueTransaction);

```

3.3. Сравнение всех трёх типов циклов

Таблица сравнения:

Критерий	for	while	do...while
Когда использовать	Известно количество итераций	Неизвестно количество итераций	Нужно выполнить хотя бы один раз
Проверка условия	Перед каждой итерацией	Перед каждой итерацией	После каждой итерации
Минимальное количество выполнений	0	0	1
Типичные сценарии	Перебор массивов, работа с диапазонами	Обработка потоков, ожидание событий	Меню, валидация ввода
Синтаксис	for(init; cond; inc)	while(cond)	do {...} while(cond)

Наглядный пример различий:

```

javascript

// Три разных цикла для одной задачи
const numbers = [10, 20, 30, 40, 50];

// 1. for - когда знаем количество элементов

```



```
console.log("for цикл:");
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}

// 2. while - если не знаем когда остановиться
console.log("\nwhile цикл:");
let index = 0;
while (index < numbers.length) {
  console.log(numbers[index]);
  index++;
}

// 3. do...while - если нужно выполнить хотя бы раз
console.log("\ndo...while цикл:");
let j = 0;
do {
  console.log(numbers[j]);
  j++;
} while (j < numbers.length);

// Демонстрация разницы при пустом массиве
const emptyArray = [];

console.log("\nПустой массив:");
console.log("for цикл выполнится 0 раз");
console.log("while цикл выполнится 0 раз");
console.log("do...while цикл выполнится 1 раз (и вызовет ошибку)");
```

Часть 4: Управление выполнением циклов

4.1. Операторы `break` и `continue`

Оператор `break` — немедленный выход из цикла:

```
javascript

// Поиск первого отрицательного числа
const numbers = [5, 10, -3, 8, 15];
let firstNegative = null;
```

```

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] < 0) {
    firstNegative = numbers[i];
    break; // Выход из цикла сразу после нахождения
  }
  console.log(`Проверяем: ${numbers[i]}`);
}

console.log(`Первое отрицательное число: ${firstNegative}`);

```

Оператор `continue` — переход к следующей итерации:

```

javascript

// Вывод только чётных чисел
for (let i = 1; i <= 10; i++) {
  if (i % 2 !== 0) {
    continue; // Пропускаем нечётные числа
  }
  console.log(`Чётное число: ${i}`);
}

// Пропуск определённых элементов массива
const mixedData = [1, "пропустить", 2, null, 3, undefined, 4];
const validNumbers = [];

for (let item of mixedData) {
  if (typeof item !== "number" || isNaN(item)) {
    continue; // Пропускаем нечисловые значения
  }
  validNumbers.push(item);
}

console.log("Только валидные числа:", validNumbers); // [1, 2, 3, 4]

```

4.2. Метки (labels) для управления вложенными циклами

```

javascript

// Поиск в матрице
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]

```

```

];
const target = 5;
let foundAt = null;

// Метка outer для внешнего цикла
outerLoop: for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] === target) {
      foundAt = { row, col };
      break outerLoop; // Выход из обоих циклов
    }
    console.log(`Проверяем [{row},{col}] = ${matrix[row][col]}`);
  }
}

console.log(`Найдено на позиции:`, foundAt);

// continue с меткой
matrixLoop: for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] % 2 === 0) {
      console.log(`Пропускаем чётное число в строке ${row}`);
      continue matrixLoop; // Переход к следующей строке
    }
    console.log(`Нечётное число: ${matrix[row][col]}`);
  }
}

```

4.3. Обработка ошибок в циклах

javascript

```

// Безопасный перебор с обработкой ошибок
const riskyOperations = [
  () => 10 / 2,
  () => 10 / 0, // Infinity
  () => parseInt("abc"), // NaN
  () => throw new Error("Тестовая ошибка"),
  () => 10 / 5
];

console.log("Безопасное выполнение операций:");
for (let i = 0; i < riskyOperations.length; i++) {
  try {

```

```
const result = riskyOperations[i]();
if (!isFinite(result) || isNaN(result)) {
  console.log(`Операция ${i}: некорректный результат`);
  continue;
}
console.log(`Операция ${i}: результат = ${result}`);
} catch (error) {
  console.log(`Операция ${i}: ошибка - ${error.message}`);
  // Можно продолжить выполнение следующих операций
  continue;
}
}
```

Часть 5: Современные альтернативы традиционным циклам

5.1. Методы массива как замена циклам

javascript

```
const numbers = [1, 2, 3, 4, 5];

// Вместо for для преобразования
const doubled = [];
for (let i = 0; i < numbers.length; i++) {
  doubled.push(numbers[i] * 2);
}

// Лучше:
const doubledBetter = numbers.map(n => n * 2);

// Вместо for для фильтрации
const evens = [];
for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    evens.push(numbers[i]);
  }
}

// Лучше:
const evensBetter = numbers.filter(n => n % 2 === 0);
```

// Вместо for для накопления

```
let sum = 0;
for (let i = 0; i < numbers.length; i++) {
  sum += numbers[i];
}
// Лучше:
const sumBetter = numbers.reduce((acc, n) => acc + n, 0);
```

5.2. Рекурсия как альтернатива циклу

javascript

// Факториал через цикл

```
function factorialLoop(n) {
  let result = 1;
  for (let i = 2; i <= n; i++) {
    result *= i;
  }
  return result;
}
```

// Факториал через рекурсию

```
function factorialRecursive(n) {
  if (n <= 1) return 1;          // Базовый случай
  return n * factorialRecursive(n - 1); // Рекурсивный вызов
}
```

// Обход дерева через рекурсию

```
const tree = {
  value: 1,
  children: [
    {
      value: 2,
      children: [
        { value: 4, children: [] },
        { value: 5, children: [] }
      ]
    },
    {
      value: 3,
      children: [
        { value: 6, children: [] }
      ]
    }
  ]
}
```

```
    ]  
  };  
  
  function traverseTree(node) {  
    console.log(node.value);  
    for (let child of node.children) {  
      traverseTree(child); // Рекурсивный обход  
    }  
  }  
}  
  
traverseTree(tree); // 1, 2, 4, 5, 3, 6
```

Часть 6: Производительность и оптимизация циклов

6.1. Бенчмаркинг различных подходов

```
javascript  
  
// Тестирование производительности  
const testArray = new Array(1000000).fill(0).map((_, i) => i);  
  
console.time("for loop");  
let sum1 = 0;  
for (let i = 0; i < testArray.length; i++) {  
  sum1 += testArray[i];  
}  
console.timeEnd("for loop");  
  
console.time("while loop");  
let sum2 = 0;  
let j = 0;  
while (j < testArray.length) {  
  sum2 += testArray[j];  
  j++;  
}  
console.timeEnd("while loop");  
  
console.time("forEach");  
let sum3 = 0;  
testArray.forEach(num => {
```

```

    sum3 += num;
  });
  console.timeEnd("forEach");

  console.time("reduce");
  const sum4 = testArray.reduce((acc, num) => acc + num, 0);
  console.timeEnd("reduce");

  console.log("Все методы дают одинаковый результат:",
    sum1 === sum2 && sum2 === sum3 && sum3 === sum4);

```

6.2. Паттерны оптимизации циклов

Loop unrolling (развёртывание цикла):

```

javascript

// Обычный цикл
function sumRegular(arr) {
  let sum = 0;
  for (let i = 0; i < arr.length; i++) {
    sum += arr[i];
  }
  return sum;
}

// Оптимизированный цикл (развёртывание)
function sumUnrolled(arr) {
  let sum = 0;
  let i = 0;
  const length = arr.length;

  // Обрабатываем по 4 элемента за итерацию
  for (; i + 3 < length; i += 4) {
    sum += arr[i] + arr[i + 1] + arr[i + 2] + arr[i + 3];
  }

  // Обрабатываем оставшиеся элементы
  for (; i < length; i++) {
    sum += arr[i];
  }

  return sum;
}

```

```
}
```

Кэширование значений:

javascript

// НЕОПТИМАЛЬНО

```
for (let i = 0; i < document.querySelectorAll('.item').length; i++) {  
    // Дорогая операция выполняется на каждой итерации  
}
```

// ОПТИМАЛЬНО

```
const items = document.querySelectorAll('.item');  
const itemsCount = items.length;  
for (let i = 0; i < itemsCount; i++) {  
    // Операция выполнена один раз  
}
```

Часть 7: Распространённые ошибки и лучшие практики

7.1. Типичные ошибки при работе с циклами

javascript

// ОШИБКА 1: Бесконечный цикл

```
let x = 0;  
while (x < 5) {  
    console.log(x);  
    // Забыли x++ — цикл никогда не завершится  
}
```

// ОШИБКА 2: Изменение массива во время итерации

```
const arr = [1, 2, 3, 4, 5];  
for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === 3) {  
        arr.splice(i, 1); // Меняем длину массива  
        // i не уменьшается — пропускаем следующий элемент  
    }  
}
```

// ОШИБКА 3: Использование for...in для массивов

```
const array = [10, 20, 30];
```



```
for (let index in array) {  
  console.log(index); // "0", "1", "2" (строки!)  
  console.log(array[index] + 5); // Неявное преобразование  
}
```

// ПРАВИЛЬНО:

```
for (let i = 0; i < array.length; i++) {  
  console.log(array[i] + 5); // 15, 25, 35  
}
```

7.2. Лучшие практики

1. Используйте осмысленные имена переменных:

javascript

// ПЛОХО

```
for (let i = 0; i < n; i++) { /* ... */ }
```

// ХОРОШО

```
for (let studentIndex = 0; studentIndex < students.length; studentIndex++) {  
  console.log(students[studentIndex].name);  
}
```

2. Ограничивайте область видимости переменных цикла:

javascript

// ПЛОХО: переменная i доступна после цикла

```
var i;  
for (i = 0; i < 5; i++) { /* ... */ }  
console.log(i); // 5
```

// ХОРОШО: переменная существует только в цикле

```
for (let i = 0; i < 5; i++) { /* ... */ }  
// console.log(i); // Ошибка: i is not defined
```

3. Используйте декларативный подход когда возможно:

javascript

```
const numbers = [1, 2, 3, 4, 5];
```

// Императивно (часто избыточно)

```
let evenSquares = [];
```

```
for (let i = 0; i < numbers.length; i++) {  
  if (numbers[i] % 2 === 0) {  
    evenSquares.push(numbers[i] * numbers[i]);  
  }  
}
```

// Декларативно (чище и понятнее)

```
const evenSquaresBetter = numbers  
  .filter(n => n % 2 === 0)  
  .map(n => n * n);
```

4. Избегайте вложенных циклов высокой сложности:

javascript

// $O(n^2)$ – медленно для больших массивов

```
function findPairsNaive(arr, target) {  
  const pairs = [];  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] + arr[j] === target) {  
        pairs.push([arr[i], arr[j]]);  
      }  
    }  
  }  
  return pairs;  
}
```

// $O(n)$ – используем хэш-таблицу

```
function findPairsOptimized(arr, target) {  
  const pairs = [];  
  const seen = new Set();  
  
  for (let num of arr) {  
    const complement = target - num;  
    if (seen.has(complement)) {  
      pairs.push([complement, num]);  
    }  
    seen.add(num);  
  }  
  
  return pairs;  
}
```

Часть 8: Практические примеры из реальных проектов

8.1. Пагинация данных

javascript

```
class Paginator {
  constructor(items, itemsPerPage = 10) {
    this.items = items;
    this.itemsPerPage = itemsPerPage;
    this.currentPage = 1;
    this.totalPages = Math.ceil(items.length / itemsPerPage);
  }

  getCurrentPageItems() {
    const startIndex = (this.currentPage - 1) * this.itemsPerPage;
    const endIndex = startIndex + this.itemsPerPage;
    return this.items.slice(startIndex, endIndex);
  }

  render() {
    const pageItems = this.getCurrentPageItems();
    console.log(`Страница ${this.currentPage} из ${this.totalPages}:`);

    // Вывод элементов текущей страницы
    for (let i = 0; i < pageItems.length; i++) {
      const item = pageItems[i];
      const globalIndex = (this.currentPage - 1) * this.itemsPerPage + i + 1;
      console.log(`${globalIndex}. ${item}`);
    }

    // Пагинация
    console.log("\nНавигация:");
    for (let page = 1; page <= this.totalPages; page++) {
      if (page === this.currentPage) {
        process.stdout.write(`[${page}] `);
      } else {
        process.stdout.write(`${page} `);
      }
    }
    console.log("\n");
  }
}
```

```

nextPage() {
  if (this.currentPage < this.totalPages) {
    this.currentPage++;
    this.render();
  }
}

prevPage() {
  if (this.currentPage > 1) {
    this.currentPage--;
    this.render();
  }
}
}

// Использование
const data = new Array(45).fill(0).map((_, i) => `Элемент ${i + 1}`);
const paginator = new Paginator(data, 10);
paginator.render();
paginator.nextPage();

```

8.2. Анимация и игровые циклы

javascript

```

class GameLoop {
  constructor() {
    this.isRunning = false;
    this.lastFrameTime = 0;
    this.fps = 0;
    this.frameCount = 0;
    this.lastFpsUpdate = 0;
  }

  start() {
    this.isRunning = true;
    this.lastFrameTime = performance.now();
    this.gameLoop();
  }

  stop() {
    this.isRunning = false;
  }
}

```

```

gameLoop(currentTime) {
  if (!this.isRunning) return;

  // Вычисление дельты времени
  const deltaTime = currentTime - this.lastFrameTime;
  this.lastFrameTime = currentTime;

  // Обновление FPS
  this.frameCount++;
  if (currentTime - this.lastFpsUpdate >= 1000) {
    this.fps = Math.round((this.frameCount * 1000) / (currentTime -
this.lastFpsUpdate));
    this.frameCount = 0;
    this.lastFpsUpdate = currentTime;
    console.log(`FPS: ${this.fps}`);
  }

  // Обновление состояния игры
  this.update(deltaTime);

  // Отрисовка
  this.render();

  // Следующий кадр
  requestAnimationFrame((time) => this.gameLoop(time));
}

update(deltaTime) {
  // Логика обновления игры
  // Вызывается каждый кадр
}

render() {
  // Отрисовка игры
  // Вызывается каждый кадр
}
}

```

8.3. Обработка очередей задач

javascript

```

class TaskQueue {
  constructor() {

```

```

    this.tasks = [];
    this.isProcessing = false;
}

addTask(task) {
    this.tasks.push(task);
    if (!this.isProcessing) {
        this.processQueue();
    }
}

async processQueue() {
    this.isProcessing = true;

    while (this.tasks.length > 0) {
        const task = this.tasks.shift();
        try {
            console.log(`Выполнение задачи: ${task.name}`);
            await task.execute();
            console.log(`Задача ${task.name} завершена`);
        } catch (error) {
            console.error(`Ошибка в задаче ${task.name}:`, error);
        }

        // Искусственная задержка для демонстрации
        await this.delay(100);
    }

    this.isProcessing = false;
    console.log("Все задачи выполнены");
}

delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

// Использование
const queue = new TaskQueue();

// Добавляем задачи
for (let i = 1; i <= 5; i++) {
    queue.addTask({
        name: `Задача ${i}`,
    });
}

```

```
execute: async () => {
  console.log(`Выполняется задача ${i}`);
  // Имитация работы
  await new Promise(resolve => setTimeout(resolve, Math.random() * 500));
}
});
}
```

Заключение: выбор правильного цикла

Резюме и рекомендации:

1. Используйте `for` когда:

- ➔ Известно точное количество итераций
- ➔ Работаете с индексами (массивы, строки)
- ➔ Нужен полный контроль над счётчиком

2. Используйте `while` когда:

- ➔ Количество итераций неизвестно заранее
- ➔ Ожидаете определённое событие или условие
- ➔ Читаете поток данных до завершения

3. Используйте `do...while` когда:

- ➔ Тело цикла должно выполниться хотя бы один раз
- ➔ Работаете с пользовательским вводом
- ➔ Создаёте интерактивные меню

Золотые правила:

- 🔵 **Всегда** проверяйте условие выхода из цикла
- 🔵 **Избегайте** бесконечных циклов — используйте защитные условия
- 🔵 **Оптимизируйте** производительность: кэшируйте длину, избегайте вложенных циклов $O(n^2)$
- 🔵 **Предпочитайте** декларативные методы массивов когда возможно
- 🔵 **Документируйте** сложные циклы комментариями

Помните:

Циклы — это мощный инструмент, но с большой силой приходит большая ответственность. Правильное использование циклов делает код эффективным, понятным и поддерживаемым, в то время как неправильное — может привести к ошибкам, утечкам памяти и проблемам с производительностью.

Циклы остаются фундаментальной частью JavaScript и программирования в целом. Понимание их тонкостей и правильное применение — ключевой навык для любого разработчика.

6.4. Итерация по объектам: for-in, for-of

Введение: два разных подхода к итерации

В JavaScript существует **два принципиально разных способа** итерации, каждый со своей спецификой:

- `for...in` — для перебора **свойств объектов** (ключей)
- `for...of` — для перебора **итерируемых значений** (элементов)

Эти конструкции часто путают, но они решают совершенно разные задачи. Понимание их различий критически важно для эффективной работы с JavaScript.

Часть 1: Цикл `for...in` — перебор свойств объектов

1.1. Базовый синтаксис и принцип работы

```
javascript
for (ключ in объект) {
  // тело цикла
  // ключ - имя свойства (строка)
  // объект[ключ] - значение свойства
}
```

Основные характеристики `for...in`:

- Перебирает **все перечисляемые свойства** объекта
- Включает свойства из цепочки прототипов (по умолчанию)
- Возвращает **строковые ключи**, а не значения
- Порядок обхода не гарантирован для всех случаев (но имеет специфичные правила)

1.2. Детальное изучение поведения `for...in`

Пример 1: Базовый перебор свойств объекта

```
javascript

const person = {
  name: "Анна",
  age: 30,
  profession: "разработчик",
  city: "Москва"
};

console.log("Перебор свойств объекта person:");
for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}
// Вывод (порядок может варьироваться):
// name: Анна
// age: 30
// profession: разработчик
// city: Москва

// Получение всех ключей как массива
const keys = [];
for (const key in person) {
  keys.push(key);
}
console.log("Все ключи:", keys); // ["name", "age", "profession", "city"]
```

Пример 2: Влияние цепочки прототипов

```
javascript

// Создаём объект с прототипом
const animal = {
  type: "млекопитающее",
  sound: "издаёт звук"
};

const dog = Object.create(animal);
dog.name = "Бобик";
dog.breed = "овчарка";

console.log("Свойства dog (включая унаследованные):");
```

```

for (const prop in dog) {
  console.log(`${prop}: ${dog[prop]}`);
}
// Вывод:
// name: Бобик
// breed: овчарка
// type: млекопитающее
// sound: издаёт звук

// Только собственные свойства
console.log("\nТолько собственные свойства dog:");
for (const prop in dog) {
  if (dog.hasOwnProperty(prop)) {
    console.log(`${prop}: ${dog[prop]}`);
  }
}
// Вывод:
// name: Бобик
// breed: овчарка

```

Пример 3: Порядок перебора свойств

javascript

```

const obj = {
  "10": "десять",    // Числовой ключ (но преобразуется в строку)
  "2": "два",
  "1": "один",
  "b": "буква b",
  "a": "буква a",
  "свойство": "строка",
  [Symbol("символ")]: "символьное свойство" // Не перечисляется for...in
};

// Добавляем свойства в разном порядке
obj["новое"] = "новое свойство";
obj["0"] = "ноль";

console.log("Порядок обхода for...in:");
for (const key in obj) {
  console.log(`${key}: ${obj[key]}`);
}

// Порядок в современных движках:
// 1. Целочисленные ключи (в числовом порядке): "0", "1", "2", "10"
// 2. Строковые ключи (в порядке добавления): "b", "a", "свойство", "новое"

```

```
// Символьные свойства не выводятся
```

```
// Проверка с помощью Object.keys()
```

```
console.log("\nObject.keys(obj):", Object.keys(obj));
```

1.3. Особые случаи и тонкости `for...in`

Перебор массивов (не рекомендуется!)

javascript

```
const fruits = ["яблоко", "банан", "апельсин"];
```

```
// ПЛОХО: использование for...in для массивов
```

```
console.log("for...in для массива:");
```

```
for (const index in fruits) {  
    console.log(`index: ${index}, значение: ${fruits[index]}`);  
    console.log(`Тип index: ${typeof index}`); // "string"!  
}
```

```
// Вывод:
```

```
// index: 0, значение: яблоко
```

```
// index: 1, значение: банан
```

```
// index: 2, значение: апельсин
```

```
// Проблема: добавленные свойства тоже перебираются
```

```
fruits.customProperty = "моё свойство";
```

```
fruits.someMethod = function() { return "метод"; };
```

```
console.log("\nПосле добавления собственных свойств:");
```

```
for (const key in fruits) {  
    console.log(`${key}: ${fruits[key]}`);  
}
```

```
// Вывод:
```

```
// 0: яблоко
```

```
// 1: банан
```

```
// 2: апельсин
```

```
// customProperty: моё свойство
```

```
// someMethod: function() { return "метод"; }
```

Исключение унаследованных свойств

javascript

```
// Настраиваемые дескрипторы свойств
```

```

const config = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
  retries: 3
};

// Делаем некоторые свойства неперечисляемыми
Object.defineProperty(config, "secretKey", {
  value: "supersecret123",
  enumerable: false, // Свойство не будет перечисляться for...in
  writable: true,
  configurable: true
});

Object.defineProperty(config, "internalFlag", {
  value: true,
  enumerable: false
});

console.log("Перечисляемые свойства config:");
for (const key in config) {
  console.log(key); // apiUrl, timeout, retries
  // secretKey и internalFlag не выводятся
}

// Получение ВСЕХ свойств (включая неперечисляемые)
console.log("\nВсе собственные свойства (включая неперечисляемые):");
const allProps = Object.getOwnPropertyNames(config);
console.log(allProps); // ["apiUrl", "timeout", "retries", "secretKey", "internalFlag"]

// Получение только перечисляемых свойств
const enumerableProps = [];
for (const key in config) {
  enumerableProps.push(key);
}
console.log("Только перечисляемые:", enumerableProps); // ["apiUrl", "timeout", "retries"]

```

Безопасный перебор с проверками

javascript

```

function safeObjectForEach(obj, callback) {
  if (obj == null) {
    throw new TypeError("Объект не может быть null или undefined");
  }
}

```

```

}

if (typeof obj !== "object" && typeof obj !== "function") {
  throw new TypeError("Ожидается объект или функция");
}

// Создаём безопасную копию ключей
const keys = [];
for (const key in obj) {
  if (Object.prototype.hasOwnProperty.call(obj, key)) {
    keys.push(key);
  }
}

// Выполняем callback для каждого ключа
for (const key of keys) {
  try {
    callback(key, obj[key], obj);
  } catch (error) {
    console.error(`Ошибка в свойстве ${key}:`, error);
    // Можно продолжить или пробросить ошибку дальше
  }
}

return keys.length;
}

// Использование
const sampleObj = {
  name: "Тест",
  count: 42,
  getData: function() { return "данные"; },
  nested: { inner: "значение" }
};

safeObjectForEach(sampleObj, (key, value) => {
  console.log(`Безопасно обрабатываем ${key}:`, value);
});

```

1.4. Практические применения `for...in`

Клонирование объектов

```
javascript

function shallowClone(obj) {
  if (obj === null || typeof obj !== "object") {
    return obj;
  }

  const clone = Array.isArray(obj) ? [] : {};

  for (const key in obj) {
    if (obj.hasOwnProperty(key)) {
      clone[key] = obj[key];
    }
  }

  return clone;
}

// Пример использования
const original = {
  name: "Оригинал",
  data: [1, 2, 3],
  settings: { theme: "dark" }
};

const cloned = shallowClone(original);
console.log("Клонированный объект:", cloned);
console.log("Равенство ссылок на data:", original.data === cloned.data); // true
(поверхностное копирование)
```

Объединение объектов (merge)

```
javascript

function mergeObjects(target, ...sources) {
  for (const source of sources) {
    if (source == null) continue;

    for (const key in source) {
      if (source.hasOwnProperty(key)) {
        // Глубокая мержа для объектов
      }
    }
  }
}
```

```

        if (typeof source[key] === "object" && source[key] !== null &&
            typeof target[key] === "object" && target[key] !== null) {
            mergeObjects(target[key], source[key]);
        } else {
            target[key] = source[key];
        }
    }
}

return target;
}

```

// Пример

```

const defaults = {
    theme: "light",
    language: "ru",
    notifications: {
        email: true,
        push: false
    }
};

```

```

const userSettings = {
    theme: "dark",
    notifications: {
        push: true
    },
    customOption: "value"
};

```

```

const merged = mergeObjects({}, defaults, userSettings);
console.log("Объединённые настройки:", merged);

```

Сбор статистики по объекту

javascript

```

function analyzeObject(obj) {
    const stats = {
        totalProperties: 0,
        ownProperties: 0,
        inheritedProperties: 0,
        stringProperties: 0,
        numberProperties: 0,
    };
}

```



```

    functionProperties: 0,
    objectProperties: 0,
    propertyNames: [],
    valuesByType: {}
};

for (const key in obj) {
    stats.totalProperties++;

    if (obj.hasOwnProperty(key)) {
        stats.ownProperties++;
    } else {
        stats.inheritedProperties++;
    }

    const value = obj[key];
    const type = typeof value;

    // Считаем по типам
    switch (type) {
        case "string": stats.stringProperties++; break;
        case "number": stats.numberProperties++; break;
        case "function": stats.functionProperties++; break;
        case "object": stats.objectProperties++; break;
    }

    // Собираем значения по типам
    if (!stats.valuesByType[type]) {
        stats.valuesByType[type] = [];
    }
    stats.valuesByType[type].push(value);

    stats.propertyNames.push(key);
}

return stats;
}

// Пример анализа
const complexObject = {
    name: "Объект",
    count: 42,
    isValid: true,
    data: { nested: "значение" },

```

```
    calculate: function() { return this.count * 2; },
    tags: ["js", "object", "analysis"]
};

console.log("Анализ объекта:", analyzeObject(complexObject));
```

Часть 2: Цикл `for...of` — перебор итерируемых значений

2.1. Базовый синтаксис и принцип работы

```
javascript

for (значение of итерируемыйОбъект) {
    // тело цикла
    // значение - текущий элемент
}
```

Основные характеристики `for...of`:

- Работает только с **итерируемыми объектами**
- Возвращает **значения**, а не ключи
- Не включает унаследованные свойства
- Гарантирует порядок обхода
- Не работает с обычными объектами (по умолчанию)

2.2. Что такое "итерируемый объект"?

Встроенные итерируемые объекты:

```
javascript

// 1. Массивы
const array = [1, 2, 3];
for (const item of array) {
    console.log(item); // 1, 2, 3
}

// 2. Строки
const str = "Привет";
```

```

for (const char of str) {
  console.log(char); // П, р, и, в, е, т
}

// 3. Map
const map = new Map([
  ["name", "Анна"],
  ["age", 30],
  ["city", "Москва"]
]);
for (const entry of map) {
  console.log(entry); // ["name", "Анна"], ["age", 30], ["city", "Москва"]
}

// 4. Set
const set = new Set([1, 2, 2, 3, 3, 3]);
for (const value of set) {
  console.log(value); // 1, 2, 3 (дубликаты удалены)
}

// 5. NodeList (DOM элементы)
// document.querySelectorAll("div") - итерируемый

// 6. Аргументы функции
function sumAll(...numbers) {
  let total = 0;
  for (const num of numbers) {
    total += num;
  }
  return total;
}
console.log(sumAll(1, 2, 3, 4, 5)); // 15

// 7. TypedArray
const typedArray = new Uint8Array([10, 20, 30]);
for (const byte of typedArray) {
  console.log(byte); // 10, 20, 30
}

```

Проверка итерируемости:

javascript

```

function isIterable(obj) {
  return obj != null && typeof obj[Symbol.iterator] === "function";
}

```

```

}

// Тестируем разные объекты
console.log("Массив:", isIterable([1, 2, 3])); // true
console.log("Строка:", isIterable("тест")); // true
console.log("Map:", isIterable(new Map())); // true
console.log("Set:", isIterable(new Set())); // true
console.log("Обычный объект:", isIterable({})); // false
console.log("Число:", isIterable(42)); // false
console.log("null:", isIterable(null)); // false

// Получение итератора явно
const arr = [10, 20, 30];
const iterator = arr[Symbol.iterator]();

console.log(iterator.next()); // {value: 10, done: false}
console.log(iterator.next()); // {value: 20, done: false}
console.log(iterator.next()); // {value: 30, done: false}
console.log(iterator.next()); // {value: undefined, done: true}

```

2.3. Детальное изучение поведения `for...of`

Пример 1: Работа с Map

```

javascript

const userMap = new Map();

// Добавление элементов
userMap.set("id", 12345);
userMap.set("name", "Иван Петров");
userMap.set("email", "ivan@example.com");
userMap.set("roles", ["user", "editor"]);
userMap.set("settings", { theme: "dark", notifications: true });

console.log("Перебор Map с for...of:");

// Вариант 1: Пары ключ-значение
for (const entry of userMap) {
  console.log(`${entry[0]}:`, entry[1]);
}

// Вариант 2: Деструктуризация

```

```
for (const [key, value] of userMap) {
  console.log(`${key}:`, value);
}

// Вариант 3: Только ключи
for (const key of userMap.keys()) {
  console.log("Ключ:", key);
}

// Вариант 4: Только значения
for (const value of userMap.values()) {
  console.log("Значение:", value);
}

// Размер Map
console.log("Размер Map:", userMap.size); // 5
```

Пример 2: Работа со Set

javascript

```
// Уникальные идентификаторы
const uniqueIds = new Set();

// Добавление элементов (дубликаты игнорируются)
uniqueIds.add(101);
uniqueIds.add(102);
uniqueIds.add(101); // Игнорируется
uniqueIds.add(103);
uniqueIds.add(102); // Игнорируется
uniqueIds.add(104);

console.log("Уникальные ID:", uniqueIds.size); // 4

// Перебор Set
for (const id of uniqueIds) {
  console.log("Обработка ID:", id);
}

// Преобразование в массив
const idArray = Array.from(uniqueIds);
console.log("Массив ID:", idArray); // [101, 102, 103, 104]

// Фильтрация
const evenIds = new Set();
```

```

for (const id of uniqueIds) {
  if (id % 2 === 0) {
    evenIds.add(id);
  }
}
console.log("Чётные ID:", evenIds); // Set {102, 104}

```

Пример 3: Строки и Unicode

javascript

```

const text = "Hello 🌍 мир! 🙌 ";

console.log("Итерация по строке:");

// for...of корректно обрабатывает суррогатные пары
for (const char of text) {
  console.log(char, `Код: ${char.codePointAt(0).toString(16)}`);
}

// Вывод:
// H Код: 48
// e Код: 65
// l Код: 6c
// l Код: 6c
// o Код: 6f
//   Код: 20
// 🌍 Код: 1f30d (эмодзи Земли)
//   Код: 20
// м Код: 43c
// и Код: 438
// р Код: 440
// ! Код: 21
//   Код: 20
// 🙌 Код: 1f44b (эмодзи руки)

// Сравнение с for (обычный индекс)
console.log("\nfor с индексами (проблема с эмодзи:");
for (let i = 0; i < text.length; i++) {
  console.log(text[i]); // Эмодзи разбиваются на суррогатные пары!
}

// Подсчёт символов (включая эмодзи)
let charCount = 0;
for (const _ of text) {
  charCount++;
}

```

```
}  
console.log(`Длина строки: ${text.length} байтов, ${charCount} символов`);
```

Пример 4: Генераторы

javascript

// Создание итерируемой последовательности

```
function* numberGenerator(limit) {  
  let current = 1;  
  while (current <= limit) {  
    yield current;  
    current++;  
  }  
}
```

// Использование с for...of

```
console.log("Генератор чисел:");  
for (const num of numberGenerator(5)) {  
  console.log(num); // 1, 2, 3, 4, 5  
}
```

// Генератор для диапазона

```
function* range(start, end, step = 1) {  
  for (let i = start; i <= end; i += step) {  
    yield i;  
  }  
}
```

```
console.log("\nДиапазон чисел:");  
for (const num of range(10, 20, 2)) {  
  console.log(num); // 10, 12, 14, 16, 18, 20  
}
```

// Бесконечный генератор (осторожно!)

```
function* fibonacci() {  
  let [prev, curr] = [0, 1];  
  while (true) {  
    yield curr;  
    [prev, curr] = [curr, prev + curr];  
  }  
}
```

// Ограниченный перебор

```
let count = 0;
```

```
for (const num of fibonacci()) {  
  if (count++ >= 10) break;  
  console.log(num); // 1, 1, 2, 3, 5, 8, 13, 21, 34, 55  
}
```

2.4. Сравнение разных способов итерации

javascript

```
const data = [10, 20, 30, 40, 50];  
  
console.log("Разные способы итерации по массиву:\n");  
  
// 1. for...of (рекомендуется для значений)  
console.log("1. for...of:");  
for (const value of data) {  
  console.log(value);  
}  
  
// 2. for классический (когда нужен индекс)  
console.log("\n2. for классический:");  
for (let i = 0; i < data.length; i++) {  
  console.log(`[${i}]: ${data[i]}`);  
}  
  
// 3. forEach (функциональный стиль)  
console.log("\n3. forEach:");  
data.forEach((value, index) => {  
  console.log(`[${index}]: ${value}`);  
});  
  
// 4. for...in (НЕ рекомендуется для массивов!)  
console.log("\n4. for...in (проблемы:");  
for (const index in data) {  
  console.log(`index: ${index}, type: ${typeof index}, value: ${data[index]}`);  
}  
  
// 5. Итератор вручную  
console.log("\n5. Ручной итератор:");  
const iterator = data[Symbol.iterator]();  
let result;  
while (!(result = iterator.next()).done) {  
  console.log(result.value);  
}
```



```
// Сравнение производительности
const largeArray = new Array(1000000).fill(0);

console.time("for...of");
for (const _ of largeArray) {}
console.timeEnd("for...of");

console.time("for классический");
for (let i = 0; i < largeArray.length; i++) {}
console.timeEnd("for классический");

console.time("forEach");
largeArray.forEach(() => {});
console.timeEnd("forEach");
```

Часть 3: Ключевые различия между `for...in` и `for...of`

3.1. Сравнительная таблица

Критерий	<code>for...in</code>	<code>for...of</code>
Что перебирает	Ключи (имена свойств)	Значения элементов
Тип значений	Строки (имена свойств)	Любые (зависит от коллекции)
Область применения	Любые объекты	Только итерируемые объекты
Прототипы	Включает унаследованные свойства	Только собственные элементы
Порядок	Не гарантирован (особые правила)	Гарантирован (порядок добавления)
Массивы	Перебирает индексы как строки	Перебирает значения элементов
Строки	Перебирает индексы символов	Перебирает символы (включая Unicode)
Объекты	Работает	Не работает (если нет <code>Symbol.iterator</code>)

Критерий	<code>for...in</code>	<code>for...of</code>
Производительность	Обычно медленнее	Обычно быстрее

3.2. Наглядные примеры различий

javascript

// Создаём тестовый объект с разными типами данных

```
const testData = {
  array: [10, 20, 30],
  string: "Hello",
  object: { a: 1, b: 2 },
  number: 42,
  map: new Map([["key1", "value1"], ["key2", "value2"]]),
  set: new Set([1, 2, 3]),
  customProp: "просто свойство"
};
```

// Добавляем свойство в прототип массива

```
Array.prototype.customArrayMethod = function() {};
```

```
console.log("=== СРАВНЕНИЕ for...in и for...of ===\n");
```

// 1. Массивы

```
console.log("1. Массив testData.array:");
console.log("for...in (индексы как строки):");
for (const key in testData.array) {
  console.log(` ${key} (тип: ${typeof key})`);
}

console.log("\nfor...of (значения):");
for (const value of testData.array) {
  console.log(` ${value} (тип: ${typeof value})`);
}
```

// 2. Строки

```
console.log("\n2. Строка testData.string:");
console.log("for...in (индексы):");
for (const index in testData.string) {
  console.log(` ${index}: '${testData.string[index]}'`);
}

console.log("\nfor...of (символы):");
```

```

for (const char of testData.string) {
  console.log(` ${char}`);
}

// 3. Map
console.log("\n3. Map:");
console.log("for...in (не работает как ожидается):");
for (const key in testData.map) {
  console.log(` ${key}`); // Не выведет элементы Map!
}

console.log("\nfor...of (работает корректно):");
for (const [key, value] of testData.map) {
  console.log(` ${key} => ${value}`);
}

// 4. Обычный объект
console.log("\n4. Обычный объект testData.object:");
console.log("for...in (работает):");
for (const key in testData.object) {
  console.log(` ${key}: ${testData.object[key]}`);
}

console.log("\nfor...of (ошибка):");
try {
  for (const value of testData.object) {
    console.log(value);
  }
} catch (error) {
  console.log(` Ошибка: ${error.message}`);
}

```

3.3. Когда что использовать: практические рекомендации

Используйте `for...in` когда:

javascript

```

// 1. Нужно перебрать все свойства объекта
const config = {
  apiEndpoint: "https://api.example.com",
  timeout: 5000,
  retryAttempts: 3,

```

```

    logLevel: "debug"
};

for (const key in config) {
    if (config.hasOwnProperty(key)) {
        console.log(`Настройка ${key}: ${config[key]}`);
    }
}

// 2. Нужно проверить, есть ли свойства у объекта
function isEmpty(obj) {
    for (const key in obj) {
        if (obj.hasOwnProperty(key)) {
            return false;
        }
    }
    return true;
}

// 3. Нужно скопировать или изменить свойства
function copyProperties(source, target) {
    for (const key in source) {
        if (source.hasOwnProperty(key)) {
            target[key] = source[key];
        }
    }
}

```

Используйте `for...of` когда:

javascript

```

// 1. Работаете с коллекциями (массивы, Map, Set)
const users = [
    { id: 1, name: "Анна" },
    { id: 2, name: "Борис" },
    { id: 3, name: "Виктор" }
];

for (const user of users) {
    console.log(`Пользователь: ${user.name}`);
}

// 2. Нужны только значения, а не индексы/ключи
const numbers = [10, 20, 30, 40, 50];

```

```
let sum = 0;

for (const num of numbers) {
  sum += num;
}
console.log(`Сумма: ${sum}`);

// 3. Работаем со строками (особенно с Unicode)
const emojiString = "JavaScript 🐼 это 🐼 ";

for (const char of emojiString) {
  console.log(char); // Корректно отображает эмодзи
}

// 4. Используем генераторы
function* generateSequence(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}

for (const num of generateSequence(5, 10)) {
  console.log(num); // 5, 6, 7, 8, 9, 10
}
```

Часть 4: Расширенные возможности итерации

4.1. Деструктуризация в циклах

```
javascript

// 1. С массивами объектов
const employees = [
  { name: "Иван", department: "IT", salary: 80000 },
  { name: "Мария", department: "HR", salary: 65000 },
  { name: "Петр", department: "Финансы", salary: 90000 }
];

console.log("Деструктуризация в for...of:");
for (const { name, department, salary } of employees) {
```

```
    console.log(`${name} работает в ${department}, зарплата: ${salary}`);  
  }
```

// 2. С Map

```
const currencyRates = new Map([  
  ["USD", 1.0],  
  ["EUR", 0.85],  
  ["GBP", 0.73],  
  ["JPY", 110.0]  
]);  
  
console.log("\nКурсы валют:");  
for (const [currency, rate] of currencyRates) {  
  console.log(`1 USD = ${rate} ${currency}`);  
}
```

// 3. С вложенными структурами

```
const matrix = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];  
  
console.log("\nДиагональ матрицы:");  
for (const [i, row] of matrix.entries()) {  
  console.log(`Элемент [${i}][${i}]: ${row[i]}`);  
}
```

// 4. Со строками и индексами

```
const text = "Hello World";  
for (const [index, char] of Array.from(text).entries()) {  
  console.log(`${index}: '${char}'`);  
}
```

4.2. Использование break, continue и меток

javascript

// 1. Поиск элемента с break

```
const products = [  
  { id: 1, name: "Ноутбук", price: 50000, inStock: true },  
  { id: 2, name: "Мышь", price: 1500, inStock: true },  
  { id: 3, name: "Клавиатура", price: 3000, inStock: false },  
  { id: 4, name: "Монитор", price: 25000, inStock: true },  
];
```

```

    { id: 5, name: "Наушники", price: 5000, inStock: false }
  ];

  // Поиск первого отсутствующего товара
  let firstOutOfStock = null;
  for (const product of products) {
    if (!product.inStock) {
      firstOutOfStock = product;
      break; // Прекращаем поиск после нахождения
    }
  }
  console.log("Первый отсутствующий товар:", firstOutOfStock?.name);

  // 2. Пропуск элементов с continue
  console.log("\nТолько товары в наличии:");
  for (const product of products) {
    if (!product.inStock) {
      continue; // Пропускаем отсутствующие товары
    }
    console.log(`- ${product.name}: ${product.price} руб.`);
  }

  // 3. Метки с вложенными циклами
  const dataSets = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
  ];

  outerLoop: for (const [setIndex, dataSet] of dataSets.entries()) {
    console.log(`\nНабор данных ${setIndex + 1}:`);

    for (const number of dataSet) {
      if (number > 7) {
        console.log("Найдено число больше 7, прекращаем обработку всех наборов");
        break outerLoop; // Выход из обоих циклов
      }
      console.log(number);
    }
  }
}

```

4.3. Создание собственных итерируемых объектов

javascript

// 1. С использованием Symbol.iterator

```
class Range {
  constructor(start, end, step = 1) {
    this.start = start;
    this.end = end;
    this.step = step;
  }

  *[Symbol.iterator]() {
    for (let i = this.start; i <= this.end; i += this.step) {
      yield i;
    }
  }
}
```

// Использование

```
console.log("Пользовательский Range:");
for (const num of new Range(1, 10, 2)) {
  console.log(num); // 1, 3, 5, 7, 9
}
```

// 2. Итерируемый объект с состоянием

```
class TreeNode {
  constructor(value) {
    this.value = value;
    this.children = [];
  }

  addChild(node) {
    this.children.push(node);
  }

  // Итератор для обхода в глубину
  *[Symbol.iterator]() {
    yield this.value;
    for (const child of this.children) {
      yield* child; // Рекурсивный yield
    }
  }
}
```



```

// Итератор для обхода в ширину
*breadthFirst() {
  const queue = [this];
  while (queue.length > 0) {
    const node = queue.shift();
    yield node.value;
    queue.push(...node.children);
  }
}

// Создание дерева
const root = new TreeNode("Корень");
const child1 = new TreeNode("Ребёнок 1");
const child2 = new TreeNode("Ребёнок 2");
const grandchild1 = new TreeNode("Внук 1");
const grandchild2 = new TreeNode("Внук 2");

root.addChild(child1);
root.addChild(child2);
child1.addChild(grandchild1);
child2.addChild(grandchild2);

console.log("\nОбход дерева в глубину:");
for (const value of root) {
  console.log(value);
}

console.log("\nОбход дерева в ширину:");
for (const value of root.breadthFirst()) {
  console.log(value);
}

// 3. Итерируемый объект с ограничением
class LimitedIterable {
  constructor(source, limit) {
    this.source = source;
    this.limit = limit;
  }

  *[Symbol.iterator]() {
    let count = 0;
    for (const item of this.source) {
      if (count++ >= this.limit) {

```

```

        break;
    }
    yield item;
}
}
}

// Использование
const infiniteNumbers = {
  *[Symbol.iterator]() {
    let i = 1;
    while (true) {
      yield i++;
    }
  }
};

console.log("\nПервые 5 чисел из бесконечной последовательности:");
for (const num of new LimitedIterable(infiniteNumbers, 5)) {
  console.log(num); // 1, 2, 3, 4, 5
}

```

4.4. Параллельная итерация нескольких коллекций

javascript

```

// 1. Использование индексов
function zipArrays(...arrays) {
  const result = [];
  const minLength = Math.min(...arrays.map(arr => arr.length));

  for (let i = 0; i < minLength; i++) {
    const tuple = [];
    for (const array of arrays) {
      tuple.push(array[i]);
    }
    result.push(tuple);
  }

  return result;
}

// Пример
const names = ["Анна", "Борис", "Виктор"];

```

```

const ages = [25, 30, 35];
const departments = ["IT", "HR", "Финансы"];

console.log("Объединённые данные:");
for (const [name, age, department] of zipArrays(names, ages, departments)) {
  console.log(`${name}, ${age} лет, отдел: ${department}`);
}

// 2. Итератор для параллельного обхода
class ParallelIterator {
  constructor(...iterables) {
    this.iterators = iterables.map(it => it[Symbol.iterator]());
  }

  *[Symbol.iterator]() {
    while (true) {
      const results = this.iterators.map(it => it.next());

      // Проверяем, все ли итераторы закончились
      if (results.some(r => r.done)) {
        break;
      }

      yield results.map(r => r.value);
    }
  }
}

// Пример использования
const prices = [100, 200, 300];
const quantities = [2, 3, 1];

console.log("\nРасчёт стоимости товаров:");
for (const [price, quantity] of new ParallelIterator(prices, quantities)) {
  console.log(`Цена: ${price}, Количество: ${quantity}, Итого: ${price * quantity}`);
}

// 3. Асинхронная итерация
async function* asyncDataGenerator() {
  const dataSources = [
    fetchDataFromAPI("users"),
    fetchDataFromAPI("products"),
    fetchDataFromAPI("orders")
  ];

```

```

    for (const source of dataSources) {
        yield await source;
    }
}

// Пример асинхронной функции (заглушка)
async function fetchDataFromAPI(endpoint) {
    // Имитация задержки сети
    await new Promise(resolve => setTimeout(resolve, Math.random() * 1000));
    return `Данные с ${endpoint}`;
}

// Асинхронный цикл for await...of
async function processAsyncData() {
    console.log("\nЗагрузка данных...");

    for await (const data of asyncDataGenerator()) {
        console.log("Получено:", data);
    }

    console.log("Все данные загружены");
}

// processAsyncData(); // Раскомментируйте для запуска

```

Часть 5: Производительность и оптимизация

5.1. Бенчмарки разных способов итерации

```

javascript

// Подготовка тестовых данных
const testSizes = [1000, 10000, 100000];
const results = {};

for (const size of testSizes) {
    const array = new Array(size).fill(0).map((_, i) => i);
    const object = {};
    for (let i = 0; i < size; i++) {

```

```
    object[`key${i}`] = i;
}

console.log(`\n=== Тест с ${size} элементами ===`);

// Тест 1: Массивы
console.log("Массивы:");

// for...of
console.time("for...of");
let sum1 = 0;
for (const value of array) {
    sum1 += value;
}
console.timeEnd("for...of");

// for классический
console.time("for классический");
let sum2 = 0;
for (let i = 0; i < array.length; i++) {
    sum2 += array[i];
}
console.timeEnd("for классический");

// forEach
console.time("forEach");
let sum3 = 0;
array.forEach(value => {
    sum3 += value;
});
console.timeEnd("forEach");

// Тест 2: Объекты
console.log("\nОбъекты:");

// for...in
console.time("for...in");
let sum4 = 0;
for (const key in object) {
    if (object.hasOwnProperty(key)) {
        sum4 += object[key];
    }
}
console.timeEnd("for...in");
```

```

// Object.keys + for...of
console.time("Object.keys + for...of");
let sum5 = 0;
const keys = Object.keys(object);
for (const key of keys) {
    sum5 += object[key];
}
console.timeEnd("Object.keys + for...of");

// Object.values + for...of
console.time("Object.values + for...of");
let sum6 = 0;
const values = Object.values(object);
for (const value of values) {
    sum6 += value;
}
console.timeEnd("Object.values + for...of");
}

```

5.2. Оптимизационные паттерны

javascript

```

// 1. Кэширование длины/ключей
const largeArray = new Array(1000000).fill(0);

// НЕОПТИМАЛЬНО
for (const value of largeArray) {
    // Каждый раз проверяется итератор
}

// ОПТИМАЛЬНО для for...of (он и так оптимален)

// 2. Избегание лишних проверок в for...in
const largeObject = {};
for (let i = 0; i < 100000; i++) {
    largeObject[`prop${i}`] = i;
}

// НЕОПТИМАЛЬНО
console.time("for...in с hasOwnProperty");
let sum1 = 0;
for (const key in largeObject) {

```

```

    if (largeObject.hasOwnProperty(key)) {
        sum1 += largeObject[key];
    }
}
console.timeEnd("for...in с hasOwnProperty");

// ОПТИМАЛЬНО (если уверены в объекте)
console.time("Object.keys + for...of");
let sum2 = 0;
const objectKeys = Object.keys(largeObject);
for (const key of objectKeys) {
    sum2 += largeObject[key];
}
console.timeEnd("Object.keys + for...of");

// 3. Использование итераторов напрямую
const data = [1, 2, 3, 4, 5];

// Стандартный подход
console.time("for...of стандартный");
for (const value of data) {}
console.timeEnd("for...of стандартный");

// Оптимизированный с явным итератором
console.time("явный итератор");
const iterator = data[Symbol.iterator]();
let result;
while (!(result = iterator.next()).done) {}
console.timeEnd("явный итератор");

```

5.3. Память и производительность при работе с большими данными

javascript

```

// 1. Ленивая итерация (generators)
function* processLargeDataset(dataset) {
    for (const item of dataset) {
        // Обработка каждого элемента
        yield processItem(item);
    }
}

function processItem(item) {

```

```

    // Тяжёлая операция
    return item * 2;
}

// Использование: элементы обрабатываются по мере необходимости
const bigData = new Array(1000000).fill(0).map((_, i) => i);

console.time("Ленивая обработка");
let processedCount = 0;
for (const result of processLargeDataset(bigData)) {
    processedCount++;
    if (processedCount >= 100) {
        break; // Обработали только первые 100 элементов
    }
}
console.timeEnd("Ленивая обработка");

// 2. Потокковая обработка
class StreamProcessor {
    constructor(source, chunkSize = 1000) {
        this.source = source;
        this.chunkSize = chunkSize;
        this.position = 0;
    }

    *[Symbol.iterator]() {
        while (this.position < this.source.length) {
            const chunk = this.source.slice(
                this.position,
                this.position + this.chunkSize
            );
            this.position += this.chunkSize;

            for (const item of chunk) {
                yield item;
            }
        }
    }

    // Даем возможность выполняться другим задачам
    if (typeof setImmediate === "function") {
        yield new Promise(resolve => setImmediate(resolve));
    }
}
}

```


// 3. Измерение памяти

```
function measureMemoryUsage(operation) {  
  if (global.gc) {  
    global.gc(); // Принудительная сборка мусора (Node.js)  
  }  
  
  const before = process.memoryUsage();  
  operation();  
  const after = process.memoryUsage();  
  
  console.log("Использование памяти:");  
  console.log(`  HeapUsed: ${after.heapUsed - before.heapUsed} / 1024 / 1024 MB`);  
  console.log(`  HeapTotal: ${after.heapTotal - before.heapTotal} / 1024 / 1024 MB`);  
}
```

// Пример измерения

```
measureMemoryUsage(() => {  
  const hugeArray = new Array(1000000).fill(0);  
  
  // Создаём копию всех значений  
  const copy = [];  
  for (const value of hugeArray) {  
    copy.push(value); // Потребляет много памяти  
  }  
});
```

```
measureMemoryUsage(() => {  
  const hugeArray = new Array(1000000).fill(0);  
  
  // Обрабатываем без сохранения  
  let sum = 0;  
  for (const value of hugeArray) {  
    sum += value; // Не сохраняем промежуточные значения  
  }  
});
```

Часть 6: Распространённые ошибки и лучшие практики

6.1. Типичные ошибки

javascript

// ОШИБКА 1: Использование for...in для массивов

```
const arr = [10, 20, 30];
arr.customProp = "опасно";
```

```
console.log("ОШИБКА: for...in для массива с дополнительными свойствами:");
for (const index in arr) {
  console.log(index, arr[index]); // 0, 1, 2, customProp
}
```

// ПРАВИЛЬНО:

```
console.log("\nПРАВИЛЬНО: for...of для значений:");
for (const value of arr) {
  console.log(value); // 10, 20, 30
}
```

```
console.log("\nПРАВИЛЬНО: for для индексов:");
for (let i = 0; i < arr.length; i++) {
  console.log(i, arr[i]); // 0 10, 1 20, 2 30
}
```

// ОШИБКА 2: Изменение объекта во время итерации

```
const obj = { a: 1, b: 2, c: 3 };
```

```
console.log("\nОШИБКА: добавление свойств во время итерации:");
try {
  for (const key in obj) {
    console.log(key);
    if (key === "b") {
      obj["d"] = 4; // Может вызвать непредсказуемое поведение
    }
  }
} catch (e) {
  console.log("Ошибка:", e.message);
}
```

// ПРАВИЛЬНО:

```
console.log("\nПРАВИЛЬНО: сначала собрать ключи:");
```

```

const keys = Object.keys(obj);
for (const key of keys) {
  console.log(key);
  if (key === "b") {
    obj["d"] = 4; // Безопасно, так как keys уже сохранён
  }
}

// ОШИБКА 3: Использование for...of с обычными объектами
const plainObj = { x: 1, y: 2, z: 3 };

console.log("\nОШИБКА: for...of с обычным объектом:");
try {
  for (const value of plainObj) {
    console.log(value);
  }
} catch (e) {
  console.log("Ошибка:", e.message); // plainObj is not iterable
}

// ПРАВИЛЬНО:
console.log("\nПРАВИЛЬНО: Object.values() или Object.entries():");
for (const value of Object.values(plainObj)) {
  console.log(value); // 1, 2, 3
}

// ИЛИ
for (const [key, value] of Object.entries(plainObj)) {
  console.log(`${key}: ${value}`); // x: 1, y: 2, z: 3
}

```

6.2. Лучшие практики

1. Всегда используйте const в for...of и for...in

```

javascript

// ПЛОХО
for (let value of array) {
  value = value * 2; // Изменение не влияет на исходный массив
}

// ХОРОШО

```

```
for (const value of array) {
  console.log(value); // value нельзя изменить (и не нужно)
}

// ХОРОШО (когда нужно изменять)
const numbers = [1, 2, 3, 4, 5];
for (let i = 0; i < numbers.length; i++) {
  numbers[i] = numbers[i] * 2; // Изменение исходного массива
}
```

2. Используйте деструктуризацию для сложных структур

javascript

```
// ПЛОХО
const users = [
  { name: "Анна", age: 25, city: "Москва" },
  { name: "Борис", age: 30, city: "Санкт-Петербург" }
];

for (const user of users) {
  const name = user.name;
  const age = user.age;
  // ...
}

// ХОРОШО
for (const { name, age, city } of users) {
  console.log(`${name}, ${age} лет, ${city}`);
}
```

3. Обрабатывайте ошибки при итерации

javascript

```
function safeIteration(iterable, callback) {
  try {
    for (const item of iterable) {
      try {
        callback(item);
      } catch (innerError) {
        console.error("Ошибка обработки элемента:", innerError);
        // Решение: продолжить, пропустить или остановиться
        continue; // Продолжаем со следующим элементом
      }
    }
  }
}
```

```

    } catch (outerError) {
      console.error("Ошибка итерации:", outerError);
      throw outerError; // Пробрасываем ошибку выше
    }
  }

  // Использование
  const data = [1, 2, "три", 4, 5];

  safeIteration(data, (item) => {
    if (typeof item !== "number") {
      throw new TypeError(`Ожидалось число, получено: ${typeof item}`);
    }
    console.log(item * 2);
  });

```

4. Комбинируйте подходы для оптимального решения

javascript

// Пример: обработка объекта с разными типами значений

```

function processMixedObject(obj) {
  const result = {
    numbers: [],
    strings: [],
    arrays: [],
    objects: [],
    others: []
  };

  // for...in для перебора всех свойств
  for (const key in obj) {
    if (!obj.hasOwnProperty(key)) {
      continue;
    }

    const value = obj[key];
    const type = typeof value;

    // Обработка в зависимости от типа
    switch (true) {
      case type === "number":
        result.numbers.push(value);
        break;

```

```

case type === "string":
    result.strings.push(value);
    break;

case Array.isArray(value):
    // for...of для перебора массива
    const arrayInfo = {
        key,
        length: value.length,
        sum: 0
    };

    for (const item of value) {
        if (typeof item === "number") {
            arrayInfo.sum += item;
        }
    }

    result.arrays.push(arrayInfo);
    break;

case type === "object" && value !== null:
    result.objects.push({
        key,
        propertyCount: Object.keys(value).length
    });
    break;

default:
    result.others.push({
        key,
        type,
        value
    });
}

return result;
}

```

// Пример использования

```

const sampleData = {
    id: 123,
    name: "Тестовый объект",

```

```
tags: ["js", "example", "test"],
scores: [95, 87, 92],
metadata: {
  created: "2024-01-01",
  author: "Разработчик"
},
isActive: true,
calculate: function() { return 42; },
nullValue: null
};

console.log("Обработанный объект:", processMixedObject(sampleData));
```

6.3. Современные альтернативы и паттерны

javascript

```
// 1. Использование методов массивов вместо циклов
const numbers = [1, 2, 3, 4, 5];

// Вместо цикла for...of
let sum = 0;
for (const num of numbers) {
  sum += num;
}

// Лучше:
const sumBetter = numbers.reduce((acc, num) => acc + num, 0);

// 2. Использование Object.entries() для объектов
const user = {
  name: "Анна",
  age: 30,
  city: "Москва"
};

// Вместо for...in
for (const key in user) {
  if (user.hasOwnProperty(key)) {
    console.log(key, user[key]);
  }
}

// Лучше:
```

```
for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}

// Или в функциональном стиле:
Object.entries(user).forEach(([key, value]) => {
  console.log(key, value);
});

// 3. Асинхронная итерация
async function processAll(items) {
  // for await...of для асинхронных итерируемых объектов
  for await (const item of fetchItemsAsync()) {
    await processItemAsync(item);
  }
}

// 4. Использование генераторов для сложной логики
function* filterMap(iterable, filterFn, mapFn) {
  for (const item of iterable) {
    if (filterFn(item)) {
      yield mapFn(item);
    }
  }
}

// Пример
const data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const processed = Array.from(
  filterMap(
    data,
    x => x % 2 === 0,      // Только чётные
    x => x * 10            // Умножить на 10
  )
);
console.log("Фильтр + Мапп:", processed); // [20, 40, 60, 80, 100]
```

Часть 7: Практические примеры из реальных проектов

7.1. Обработка данных API

javascript

```
class ApiResponseProcessor {
  constructor(apiResponse) {
    this.data = apiResponse;
  }

  // Итерация по массиву объектов
  *getUsers() {
    if (!this.data.users || !Array.isArray(this.data.users)) {
      return;
    }

    for (const user of this.data.users) {
      yield {
        id: user.id,
        fullName: `${user.firstName} ${user.lastName}`,
        email: user.email,
        isActive: user.status === 'active'
      };
    }
  }

  // Итерация по вложенным данным
  *getOrdersByUser(userId) {
    if (!this.data.orders || !Array.isArray(this.data.orders)) {
      return;
    }

    for (const order of this.data.orders) {
      if (order.userId === userId) {
        yield {
          orderId: order.id,
          total: order.total,
          items: order.items.length,
          date: new Date(order.createdAt).toLocaleDateString()
        };
      }
    }
  }
}
```

```
}
```

```
// Статистика по данным
```

```
getStatistics() {  
  const stats = {  
    totalUsers: 0,  
    activeUsers: 0,  
    totalOrders: 0,  
    totalRevenue: 0,  
    usersByCity: new Map()  
  };  
};
```

```
// Обработка пользователей
```

```
if (this.data.users) {  
  for (const user of this.data.users) {  
    stats.totalUsers++;  
  
    if (user.status === 'active') {  
      stats.activeUsers++;  
    }  
  }  
}
```

```
// Группировка по городам
```

```
const city = user.city || 'Не указан';  
const cityCount = stats.usersByCity.get(city) || 0;  
stats.usersByCity.set(city, cityCount + 1);  
}  
}
```

```
// Обработка заказов
```

```
if (this.data.orders) {  
  for (const order of this.data.orders) {  
    stats.totalOrders++;  
    stats.totalRevenue += order.total || 0;  
  }  
}
```

```
return stats;
```

```
}
```

```
}
```

```
// Пример использования
```

```
const mockApiResponse = {  
  users: [  
    // ...  
  ]  
};
```

```

    { id: 1, firstName: "Анна", lastName: "Иванова", email: "anna@test.com", status:
"active", city: "Москва" },
    { id: 2, firstName: "Борис", lastName: "Петров", email: "boris@test.com", status:
"inactive", city: "Санкт-Петербург" },
    { id: 3, firstName: "Виктор", lastName: "Сидоров", email: "viktor@test.com", status:
"active", city: "Москва" }
  ],
  orders: [
    { id: 101, userId: 1, total: 1500, items: [1, 2, 3], createdAt: "2024-01-
15T10:30:00Z" },
    { id: 102, userId: 3, total: 2500, items: [4, 5], createdAt: "2024-01-16T14:45:00Z"
  },
    { id: 103, userId: 1, total: 800, items: [6], createdAt: "2024-01-17T09:15:00Z" }
  ]
};

```

```
const processor = new ApiResponseProcessor(mockApiResponse);
```

```

console.log("Пользователи:");
for (const user of processor.getUsers()) {
  console.log(user);
}

```

```

console.log("\nЗаказы пользователя 1:");
for (const order of processor.getOrdersByUser(1)) {
  console.log(order);
}

```

```

console.log("\nСтатистика:");
console.log(processor.getStatistics());

```

7.2. Работа с DOM деревом

javascript

```

class DOMTraverser {
  constructor(rootElement) {
    this.root = rootElement;
  }

  // Итерация по всем элементам в глубину
  *traverseDepthFirst() {
    const stack = [this.root];

```

```

while (stack.length > 0) {
  const element = stack.pop();
  yield element;

  // Добавляем детей в обратном порядке для корректного обхода
  const children = Array.from(element.children).reverse();
  stack.push(...children);
}
}

// Итерация по всем элементам в ширину
*traverseBreadthFirst() {
  const queue = [this.root];

  while (queue.length > 0) {
    const element = queue.shift();
    yield element;

    queue.push(...element.children);
  }
}

// Поиск элементов по селектору
*findElements(selector) {
  for (const element of this.traverseDepthFirst()) {
    if (element.matches(selector)) {
      yield element;
    }
  }
}

// Итерация по атрибутам элемента
static *getAttributes(element) {
  for (const attribute of element.attributes) {
    yield {
      name: attribute.name,
      value: attribute.value,
      namespace: attribute.namespaceURI
    };
  }
}

// Сбор данных из таблицы
static *getTableData(tableElement) {

```

```

const rows = tableElement.querySelectorAll('tr');

for (const row of rows) {
  const cells = row.querySelectorAll('td, th');
  const rowData = [];

  for (const cell of cells) {
    rowData.push({
      text: cell.textContent.trim(),
      colspan: parseInt(cell.getAttribute('colspan') || '1'),
      rowspan: parseInt(cell.getAttribute('rowspan') || '1')
    });
  }

  yield rowData;
}
}

// Пример использования (в браузерной среде)
/*
<div id="root">
  <div class="container">
    <h1>Заголовок</h1>
    <p>Параграф 1</p>
    <div class="nested">
      <span>Вложенный текст</span>
      <p>Параграф 2</p>
    </div>
  </div>
  <ul>
    <li>Элемент 1</li>
    <li>Элемент 2</li>
    <li>Элемент 3</li>
  </ul>
</div>

<script>
const traverser = new DOMTraverser(document.getElementById('root'));

console.log("Обход в глубину:");
for (const element of traverser.traverseDepthFirst()) {
  console.log(element.tagName);
}

```

```
console.Log("\nВсе параграфы:");
for (const p of traverser.findElements('p')) {
  console.Log(p.textContent);
}
</script>
*/
```

7.3. Конфигурация и настройки

javascript

```
class ConfigurationManager {
  constructor(config) {
    this.config = config;
    this.validators = new Map();
    this.transformers = new Map();
  }

  // Регистрация валидаторов для свойств
  registerValidator(key, validatorFn) {
    this.validators.set(key, validatorFn);
  }

  // Регистрация трансформеров для свойств
  registerTransformer(key, transformerFn) {
    this.transformers.set(key, transformerFn);
  }

  // Итерация по всем настройкам с валидацией
  *getValidatedSettings() {
    for (const [key, value] of Object.entries(this.config)) {
      const validator = this.validators.get(key);
      const transformer = this.transformers.get(key);

      let validatedValue = value;
      let isValid = true;
      let error = null;

      // Применяем трансформер если есть
      if (transformer) {
        try {
          validatedValue = transformer(value);
        } catch (transformError) {
```

```

        isValid = false;
        error = `Ошибка трансформации: ${transformError.message}`;
    }
}

// Применяем валидатор если есть
if (isValid && validator) {
    try {
        validator(validatedValue);
    } catch (validationError) {
        isValid = false;
        error = `Ошибка валидации: ${validationError.message}`;
    }
}

yield {
    key,
    originalValue: value,
    value: validatedValue,
    isValid,
    error
};
}
}

// Поиск невалидных настроек
*findInvalidSettings() {
    for (const setting of this.getValidatedSettings()) {
        if (!setting.isValid) {
            yield setting;
        }
    }
}

// Применение настроек по группам
applyByGroup() {
    const groups = new Map();

    // Группировка настроек
    for (const [key, value] of Object.entries(this.config)) {
        const group = key.split('_')[0]; // Группа по префиксу
        if (!groups.has(group)) {
            groups.set(group, {});
        }
    }
}

```

```

    groups.get(group)[key] = value;
  }

  // Итерация по группам
  for (const [groupName, groupSettings] of groups) {
    console.log(`\nПрименение группы "${groupName}":`);

    for (const [key, value] of Object.entries(groupSettings)) {
      console.log(`  ${key} = ${value}`);
      // Здесь была бы реальная логика применения
    }
  }
}

// Пример использования
const appConfig = {
  db_host: "localhost",
  db_port: 5432,
  db_name: "myapp",

  server_port: 3000,
  server_timeout: 5000,

  cache_enabled: true,
  cache_ttl: 3600,

  log_level: "debug",
  log_file: "app.log"
};

const configManager = new ConfigurationManager(appConfig);

// Регистрируем валидаторы
configManager.registerValidator('db_port', (value) => {
  if (value < 1 || value > 65535) {
    throw new Error('Порт должен быть в диапазоне 1-65535');
  }
});

configManager.registerValidator('log_level', (value) => {
  const validLevels = ['debug', 'info', 'warn', 'error'];
  if (!validLevels.includes(value)) {
    throw new Error(`Уровень логов должен быть одним из: ${validLevels.join(', ')}`);
  }
});

```



```
    }  
  });  
  
  // Регистрируем трансформеры  
  configManager.registerTransformer('server_timeout', (value) => {  
    return parseInt(value); // Преобразуем строку в число  
  });  
  
  console.log("Проверка настроек:");  
  for (const setting of configManager.getValidatedSettings()) {  
    console.log(`${setting.key}: ${setting.value} - ${setting.isValid ? 'OK' : 'ERROR: ' +  
setting.error}`);  
  }  
  
  console.log("\nНевалидные настройки:");  
  for (const invalid of configManager.findInvalidSettings()) {  
    console.log(invalid.key, invalid.error);  
  }  
  
  console.log("\nПрименение настроек по группам:");  
  configManager.applyByGroup();
```

Заключение: итоговые рекомендации

Ключевые выводы:

1. `for...in` — это инструмент для работы с **ключами объектов**. Используйте его, когда нужно:
 - ➔ Перебрать все свойства объекта
 - ➔ Проверить наличие свойств
 - ➔ Скопировать или модифицировать свойства
 - ➔ **Всегда** используйте `hasOwnProperty()` для исключения унаследованных свойств
2. `for...of` — это инструмент для работы с **значениями коллекций**. Используйте его, когда нужно:
 - ➔ Перебрать элементы массива, строки, Map, Set
 - ➔ Получить значения без индексов/ключей
 - ➔ Работать с итерируемыми объектами (включая генераторы)

➔ **Не работает** с обычными объектами
(используйте `Object.values()` или `Object.entries()`)

Золотые правила:

1. **Никогда** не используйте `for...in` для массивов — используйте `for...of`, `for` или методы массивов
2. **Всегда** используйте `const` в `for...of` и `for...in` (если не нужно переназначать переменную)
3. **Помните** о порядке обхода: `for...in` не гарантирует порядок, `for...of` гарантирует
4. **Учитывайте** производительность: `for...of` обычно быстрее `for...in`
5. **Используйте** деструктуризацию для работы со сложными структурами
6. **Комбинируйте** с другими методами (`Object.keys()`, `Object.entries()`) для лучшей читаемости

Выбор инструмента: краткий гид

javascript

// 1. Перебор массива (значения):

```
for (const value of array) { } // 🟡 Лучший выбор
array.forEach(value => { }) // 🟡 Функциональный стиль
for (let i = 0; i < array.length; i++) { } // 🟡 Когда нужен индекс
```

// 2. Перебор массива (индексы):

```
for (let i = 0; i < array.length; i++) { } // 🟡 Лучший выбор
array.forEach((value, index) => { }) // 🟡 Функциональный стиль
for (const index in array) { } // 🟡 НИКОГДА!
```

// 3. Перебор объекта (ключи):

```
for (const key in obj) { // 🟡 С проверкой hasOwnProperty
  if (obj.hasOwnProperty(key)) { }
}
Object.keys(obj).forEach(key => { }) // 🟡 Функциональный стиль
```

// 4. Перебор объекта (значения):

```
for (const value of Object.values(obj)) { } // 🟡 Лучший выбор
Object.values(obj).forEach(value => { }) // 🟡 Функциональный стиль
```

// 5. Перебор объекта (ключи и значения):

```
for (const [key, value] of Object.entries(obj)) { } // 🟡 Лучший выбор
Object.entries(obj).forEach(([key, value]) => { }) // 🟡 Функциональный стиль
```

Запомните:

- `for...in` — для **объектов** и их **свойств** (ключей)
- `for...of` — для **коллекций** и их **значений** (элементов)
- `for` — когда нужен **полный контроль** над процессом итерации
- **Методы массивов** (`forEach`, `map`, `filter`) — для **декларативного** стиля

Правильное понимание и использование этих инструментов — признак опытного JavaScript-разработчика. Выбор правильного способа итерации делает код более читаемым, производительным и поддерживаемым.

Введение: зачем нужно управление выполнением циклов

Циклы — мощный инструмент, но иногда требуется **гибкое управление** их выполнением. JavaScript предоставляет два ключевых оператора для этого:

- `break` — **полное прерывание** выполнения цикла
- `continue` — **пропуск текущей итерации** и переход к следующей

Эти операторы позволяют создавать **интеллектуальные циклы**, которые реагируют на определённые условия, делая код более эффективным и читаемым.

Часть 1: Оператор `break` — полное прерывание цикла

1.1. Базовый синтаксис и принцип работы

```
javascript
for (инициализация; условие; изменение) {
  // код до break
  if (условие_прерывания) {
    break; // Немедленный выход из цикла
  }
  // код после break (не выполнится если сработал break)
}
// выполнение продолжается здесь после break
```

Как работает `break`:

1. Цикл выполняется нормально
2. При выполнении условия срабатывает `break`
3. **Немедленный выход** из цикла
4. Выполнение продолжается **после цикла**

1.2. Подробное изучение break в разных типах циклов

Пример 1: break в цикле for

```
javascript

console.log("Пример 1: Поиск первого отрицательного числа в массиве");

const numbers = [5, 10, -3, 8, 15, -7, 20];
let firstNegative = null;
let foundIndex = -1;

for (let i = 0; i < numbers.length; i++) {
  console.log(`Проверяем элемент [${i}]: ${numbers[i]}`);

  if (numbers[i] < 0) {
    firstNegative = numbers[i];
    foundIndex = i;
    console.log(`Найдено отрицательное число: ${numbers[i]} на позиции ${i}`);
    break; // Выходим из цикла сразу после нахождения
  }
}

if (firstNegative !== null) {
  console.log(`Первое отрицательное число: ${firstNegative} на позиции ${foundIndex}`);
} else {
  console.log("Отрицательных чисел не найдено");
}

// Что происходит без break?
console.log("\nБез break (перебираем все элементы):");
let firstNegativeWithoutBreak = null;
for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] < 0 && firstNegativeWithoutBreak === null) {
    firstNegativeWithoutBreak = numbers[i];
    console.log(`Нашли на позиции ${i}, но продолжаем проверять...`);
  }
}
console.log("Лишние итерации выполнены!");
```

Пример 2: break в цикле while

```
javascript

console.log("\nПример 2: Обработка пользовательского ввода до команды 'выход'");
```

```

// Имитация получения команд от пользователя
const userCommands = ["открыть", "сохранить", "удалить", "выход", "печать", "закрыть"];
let commandIndex = 0;
let processedCommands = [];

while (commandIndex < userCommands.length) {
  const command = userCommands[commandIndex];
  console.log(`Обработка команды: '${command}'`);

  if (command === "выход") {
    console.log("Получена команда 'выход'. Прекращаем обработку.");
    break;
  }

  processedCommands.push(command);
  commandIndex++;
}

console.log(`Обработано команд: ${processedCommands.length}`);
console.log("Обработанные команды:", processedCommands);

// Без break пришлось бы писать сложное условие
console.log("\nЭквивалентный код без break:");
commandIndex = 0;
processedCommands = [];

while (commandIndex < userCommands.length && userCommands[commandIndex] !== "выход") {
  const command = userCommands[commandIndex];
  console.log(`Обработка команды: '${command}'`);
  processedCommands.push(command);
  commandIndex++;
}
console.log("Тот же результат, но условие цикла сложнее");

```

Пример 3: break в цикле do...while

```

javascript

console.log("\nПример 3: Гарантированное выполнение минимум один раз с возможностью досрочного выхода");

let attempts = 0;
const maxAttempts = 5;
const secretCode = "1234";

```

```

do {
  attempts++;
  console.log(`Попытка ${attempts}/${maxAttempts}`);

  // Имитация ввода пользователя
  let userInput;
  switch(attempts) {
    case 1: userInput = "0000"; break;
    case 2: userInput = "1111"; break;
    case 3: userInput = "1234"; break; // Правильный код
    case 4: userInput = "4321"; break;
    default: userInput = "9999";
  }

  console.log(`Введён код: ${userInput}`);

  if (userInput === secretCode) {
    console.log("✅ Доступ разрешён!");
    break; // Выходим из цикла при успехе
  }

  if (attempts >= maxAttempts) {
    console.log("× Превышено количество попыток. Доступ заблокирован.");
    break; // Выходим при исчерпании попыток
  }

  console.log("Неверный код. Попробуйте снова.\n");
} while (true); // Бесконечный цикл, выходим только через break

console.log(`Использовано попыток: ${attempts}`);

```

1.3. Особенности работы **break** в вложенных циклах

```

javascript

console.log("Пример: Поиск в матрице с вложенными циклами");

const matrix = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 10, 11, 12],
  [13, 14, 15, 16]
];

```

```

const target = 7;
let foundAt = null;

console.log("Поиск элемента", target, "в матрице 4x4");

// ПРОБЛЕМА: break выходит только из внутреннего цикла
console.log("\n1. Только внутренний break (проблема):");
for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] === target) {
      foundAt = { row, col };
      console.log(`Найдено на позиции [${row}][${col}]`);
      break; // × Выходит только из внутреннего цикла!
    }
  }
}
// Цикл продолжает выполняться для следующих строк
console.log(`Проверена строка ${row}, продолжаем...`);
}

// РЕШЕНИЕ: Использование флага
console.log("\n2. Использование флага для выхода из обоих циклов:");
foundAt = null;
let shouldBreak = false;

for (let row = 0; row < matrix.length && !shouldBreak; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] === target) {
      foundAt = { row, col };
      console.log(`Найдено на позиции [${row}][${col}]`);
      shouldBreak = true;
      break; // Выходим из внутреннего цикла
    }
  }
}
console.log("Поиск завершён с флагом");

// РЕШЕНИЕ 2: Размещение условия во внешнем цикле
console.log("\n3. Проверка во внешнем цикле:");
foundAt = null;

outerSearch: // Метка для внешнего цикла (объясним позже)
for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {

```



```

    if (matrix[row][col] === target) {
        foundAt = { row, col };
        console.log(`Найдено на позиции [${row}][${col}]`);
        break; // Только внутренний цикл
    }
}
// Проверяем, нашли ли элемент
if (foundAt !== null) {
    break; // Выходим из внешнего цикла
}
}
console.log("Поиск завершён с проверкой во внешнем цикле");

```

1.4. Практические примеры использования break

Пример 1: Валидация данных

javascript

```

console.log("Пример: Валидация данных пользователя");

function validateUserData(userData) {
    const errors = [];

    // Проверка обязательных полей
    const requiredFields = ['username', 'email', 'password'];

    for (const field of requiredFields) {
        if (!userData[field]) {
            errors.push(`Поле '${field}' обязательно для заполнения`);
            break; // Прекращаем проверку при первой ошибке
        }
    }

    // Если есть ошибки в обязательных полях, дальше не проверяем
    if (errors.length > 0) {
        return { isValid: false, errors };
    }

    // Дополнительные проверки (выполняются только если обязательные поля заполнены)
    if (userData.username.length < 3) {
        errors.push('Имя пользователя должно содержать минимум 3 символа');
    }
}

```

```

    if (!userData.email.includes('@')) {
        errors.push('Некорректный email адрес');
    }

    if (userData.password.length < 6) {
        errors.push('Пароль должен содержать минимум 6 символов');
    }

    return {
        isValid: errors.length === 0,
        errors: errors.length > 0 ? errors : null
    };
}

// Тестирование
const testUsers = [
    { username: '', email: 'test@example.com', password: '123456' }, // Пустое имя
    { username: 'a1', email: 'test@example.com', password: '123456' }, // Короткое имя
    { username: 'alice', email: 'invalid-email', password: '123456' }, // Невалидный email
    { username: 'bob', email: 'bob@example.com', password: '123' }, // Короткий пароль
    { username: 'charlie', email: 'charlie@example.com', password: 'secure123' } //
    Валидный
];

console.log("Результаты валидации:");
for (const user of testUsers) {
    const result = validateUserData(user);
    console.log(`${user.username}: ${result.isValid ? '✅ Валиден' : '❌ Ошибки:'}`);
    if (!result.isValid) {
        console.log(' ', result.errors.join(', '));
    }
}

```

Пример 2: Обработка потоковых данных

javascript

```

console.log("\nПример: Чтение данных из потока до маркера окончания");

// Имитация потока данных (например, из файла или сети)
function* dataStreamGenerator() {
    const chunks = [
        "Первая строка данных\n",
        "Вторая строка данных\n",

```

```

    "Третья строка данных\n",
    "---END---\n", // Маркер окончания данных
    "Четвёртая строка данных\n", // Не должна быть обработана
    "Пятая строка данных\n"
];

for (const chunk of chunks) {
    yield chunk;
}
}

// Обработчик потока
function processDataStream(stream) {
    const processedData = [];
    let lineCount = 0;

    for (const chunk of stream) {
        console.log(`Получен чанк: "${chunk.trim()}"`);

        // Проверяем маркер окончания
        if (chunk.includes('---END---')) {
            console.log("Обнаружен маркер окончания данных");
            break; // Прекращаем обработку
        }

        // Обработка данных
        processedData.push(chunk.trim());
        lineCount++;
    }

    return { processedData, lineCount };
}

// Запуск обработки
const stream = dataStreamGenerator();
const result = processDataStream(stream);

console.log("\nРезультат обработки:");
console.log(`Обработано строк: ${result.lineCount}`);
console.log("Обработанные данные:", result.processedData);

```

Пример 3: Игра или симуляция

javascript

```

console.log("\nПример: Игровой цикл с условиями победы/поражения");

class GameSimulation {
  constructor() {
    this.playerHealth = 100;
    this.enemyHealth = 100;
    this.round = 0;
    this.maxRounds = 10;
  }

  simulateRound() {
    this.round++;
    console.log(`\n=== Раунд ${this.round} ===`);

    // Игрок атакует
    const playerDamage = Math.floor(Math.random() * 20) + 10;
    this.enemyHealth -= playerDamage;
    console.log(`Игрок наносит ${playerDamage} урона. Здоровье врага: ${Math.max(0, this.enemyHealth)}`);

    // Проверка победы
    if (this.enemyHealth <= 0) {
      console.log("🏆 Игрок победил!");
      return 'player_win';
    }

    // Враг атакует
    const enemyDamage = Math.floor(Math.random() * 15) + 5;
    this.playerHealth -= enemyDamage;
    console.log(`Враг наносит ${enemyDamage} урона. Здоровье игрока: ${Math.max(0, this.playerHealth)}`);

    // Проверка поражения
    if (this.playerHealth <= 0) {
      console.log("🏆 Игрок проиграл!");
      return 'enemy_win';
    }

    return 'continue';
  }

  start() {
    console.log("=== НАЧАЛО ИГРЫ ===");
    console.log(`Игрок: ${this.playerHealth} HP, Враг: ${this.enemyHealth} HP`);
  }
}

```

```
while (this.round < this.maxRounds) {
  const result = this.simulateRound();

  if (result !== 'continue') {
    console.log(`Игра завершена на раунде ${this.round}`);
    break; // Выходим из цикла при победе или поражении
  }
}

// Проверка на ничью (закончились раунды)
if (this.round >= this.maxRounds && this.playerHealth > 0 && this.enemyHealth > 0) {
  console.log("\n□ Ничья! Закончились раунды.");
  console.log(`Итоговое здоровье: Игрок - ${this.playerHealth}, Враг - $
{this.enemyHealth}`);
}
}

// Запуск симуляции
const game = new GameSimulation();
game.start();
```

Часть 2: Оператор `continue` — пропуск текущей итерации

2.1. Базовый синтаксис и принцип работы

```
javascript

for (инициализация; условие; изменение) {
  // код до continue
  if (условие_пропуска) {
    continue; // Пропускаем оставшийся код итерации
  }
  // код после continue (не выполнится если сработал continue)
  // цикл переходит к следующей итерации
}
```

Как работает `continue`:

1. Цикл выполняет текущую итерацию
2. При выполнении условия срабатывает `continue`
3. Остаток текущей итерации пропускается
4. Цикл переходит к следующей итерации (если условие цикла истинно)
5. Цикл не прерывается, продолжаются следующие итерации

2.2. Подробное изучение `continue` в разных типах циклов

Пример 1: `continue` в цикле `for` — фильтрация данных

```
javascript

console.log("Пример 1: Фильтрация массива чисел");

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const evenNumbers = [];
const oddNumbers = [];

for (let i = 0; i < numbers.length; i++) {
  console.log(`Обработка числа: ${numbers[i]}`);

  // Пропускаем чётные числа для коллекции нечётных
  if (numbers[i] % 2 === 0) {
    evenNumbers.push(numbers[i]);
    console.log(` ${numbers[i]} - чётное, добавляем в evenNumbers, пропускаем для oddNumbers`);
    continue; // Пропускаем добавление в oddNumbers
  }

  // Этот код выполнится только для нечётных чисел
  oddNumbers.push(numbers[i]);
  console.log(` ${numbers[i]} - нечётное, добавляем в oddNumbers`);
}

console.log("\nРезультаты:");
console.log("Чётные числа:", evenNumbers); // [2, 4, 6, 8, 10]
console.log("Нечётные числа:", oddNumbers); // [1, 3, 5, 7, 9]

// Альтернатива без continue
console.log("\nАльтернативная реализация без continue:");
const evenNumbers2 = [];
```

```

const oddNumbers2 = [];

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    evenNumbers2.push(numbers[i]);
  } else {
    oddNumbers2.push(numbers[i]);
  }
}

console.log("Тот же результат, но код более плоский");

```

Пример 2: `continue` в цикле `while` — обработка с пропуском невалидных данных

javascript

```

console.log("\nПример 2: Обработка пользовательского ввода с валидацией");

```

// Имитация последовательности ввода пользователя

```

const userInputs = [
  "42",      // валидное число
  "не число", // невалидное
  "100",     // валидное
  "",        // пустая строка
  "3.14",    // валидное число
  "abc123",  // невалидное
  "0"        // валидное
];

```

```

let index = 0;
let validNumbers = [];
let invalidCount = 0;

```

```

console.log("Начало обработки пользовательского ввода:");

```

```

while (index < userInputs.length) {
  const input = userInputs[index];
  index++;

  console.log(`\nОбработка ввода: "${input}"`);

```

// Пропускаем пустые строки

```

if (input.trim() === "") {
  console.log("  Пустая строка, пропускаем");

```

```

    invalidCount++;
    continue;
}

// Пытаемся преобразовать в число
const number = Number(input);

// Пропускаем нечисловые значения
if (isNaN(number)) {
    console.log(` ❌ "${input}" не является числом, пропускаем`);
    invalidCount++;
    continue;
}

// Успешная обработка
validNumbers.push(number);
console.log(` ✅ Валидное число: ${number}`);
}

console.log("\nИтоги обработки:");
console.log(`Валидных чисел: ${validNumbers.length}`, validNumbers);
console.log(`Невалидных вводов: ${invalidCount}`);
console.log(`Всего обработано: ${index} вводов`);

```

Пример 3: `continue` в цикле `do...while` — меню с обработкой команд

javascript

```

console.log("\nПример 3: Интерактивное меню с пропуском неизвестных команд");

class InteractiveMenu {
    constructor() {
        this.commands = [];
        this.isRunning = true;
    }

    // Имитация получения команд от пользователя
    *commandGenerator() {
        const simulatedCommands = [
            "help",
            "unknown_command", // Неизвестная команда
            "list",
            "invalid",          // Неизвестная команда
            "add item1",
            "add item2",

```



```

    "delete item1",
    "exit",
    "add item3"           // Не выполнится из-за exit
];

for (const cmd of simulatedCommands) {
    yield cmd;
}

}

processCommand(command) {
    console.log(`> ${command}`);

    // Пропускаем неизвестные команды
    if (!command.startsWith("add ") &&
        !command.startsWith("delete ") &&
        ![ "help", "list", "exit" ].includes(command)) {
        console.log(" ⚠️ Неизвестная команда. Введите 'help' для списка команд.");
        return 'skip';
    }

    // Обработка известных команд
    switch (command) {
        case "help":
            console.log(" 📖 Доступные команды:");
            console.log("     help - показать эту справку");
            console.log("     list - показать список элементов");
            console.log("     add <item> - добавить элемент");
            console.log("     delete <item> - удалить элемент");
            console.log("     exit - выйти из программы");
            break;

        case "list":
            console.log(" 📋 Список элементов:", this.commands);
            break;

        case "exit":
            console.log(" 🚪 Выход из программы...");
            this.isRunning = false;
            return 'exit';

        default:
            if (command.startsWith("add ")) {
                const item = command.substring(4);

```

```

        this.commands.push(item);
        console.log(`   Добавлен элемент: "${item}"`);
    } else if (command.startsWith("delete ")) {
        const item = command.substring(7);
        const index = this.commands.indexOf(item);
        if (index !== -1) {
            this.commands.splice(index, 1);
            console.log(`   Удалён элемент: "${item}"`);
        } else {
            console.log(`   × Элемент "${item}" не найден`);
        }
    }
}

return 'processed';
}

run() {
    console.log("=== ИНТЕРАКТИВНОЕ МЕНЮ ===\n");
    console.log("Введите команду (help для справки):");

    const commandStream = this.commandGenerator();

    do {
        const command = commandStream.next().value;

        if (command === undefined) {
            console.log("\nБольше команд нет.");
            break;
        }

        const result = this.processCommand(command);

        if (result === 'skip') {
            continue; // Пропускаем неизвестные команды
        }

        if (result === 'exit') {
            break; // Выходим при команде exit
        }

        console.log(); // Пустая строка между командами
    } while (this.isRunning);
}

```

```

        console.log("\n=== ПРОГРАММА ЗАВЕРШЕНА ===");
        console.log("Итоговый список элементов:", this.commands);
    }
}

// Запуск меню
const menu = new InteractiveMenu();
menu.run();

```

2.3. Особенности работы `continue` в вложенных циклах

```

javascript

console.log("Пример: Обработка матрицы с пропуском элементов");

const dataMatrix = [
    [1, 2, null, 4],
    [5, null, 7, 8],
    [9, 10, 11, null],
    [null, 14, 15, 16]
];

console.log("Исходная матрица:");
dataMatrix.forEach(row => console.log(row));

// Задача: Найти сумму всех НЕ null элементов
// и пропустить всю строку, если в ней есть null

console.log("\n1. Пропуск отдельных null элементов (внутренний continue:");
let sum1 = 0;
let processedCells1 = 0;

for (let row = 0; row < dataMatrix.length; row++) {
    for (let col = 0; col < dataMatrix[row].length; col++) {
        if (dataMatrix[row][col] === null) {
            console.log(`Пропускаем null в [${row}][${col}]`);
            continue; // Пропускаем только этот элемент
        }
        sum1 += dataMatrix[row][col];
        processedCells1++;
    }
}

```

```

console.log(`Сумма (пропуск отдельных null): ${sum1}`);
console.log(`Обработано ячеек: ${processedCells1}`);

console.log("\n2. Пропуск всей строки при обнаружении null (внешний continue:");
let sum2 = 0;
let processedRows2 = 0;

outerLoop: for (let row = 0; row < dataMatrix.length; row++) {
    console.log(`\nОбработка строки ${row}: ${dataMatrix[row]}`);

    // Проверяем всю строку на наличие null
    for (let col = 0; col < dataMatrix[row].length; col++) {
        if (dataMatrix[row][col] === null) {
            console.log(`Обнаружен null в [${row}][${col}], пропускаем всю строку`);
            continue outerLoop; // Пропускаем текущую строку целиком
        }
    }

    // Этот код выполнится только для строк без null
    for (let col = 0; col < dataMatrix[row].length; col++) {
        sum2 += dataMatrix[row][col];
    }
    processedRows2++;
    console.log(`Строка ${row} обработана полностью`);
}

console.log(`\nСумма (пропуск строк с null): ${sum2}`);
console.log(`Обработано строк: ${processedRows2}`);

console.log("\n3. Пропуск оставшихся элементов строки после null:");
let sum3 = 0;
let processedCells3 = 0;

for (let row = 0; row < dataMatrix.length; row++) {
    console.log(`\nОбработка строки ${row}:`);
    let foundNull = false;

    for (let col = 0; col < dataMatrix[row].length; col++) {
        if (foundNull) {
            console.log(`Пропускаем [${row}][${col}] после null`);
            continue; // Пропускаем оставшиеся элементы строки
        }

        if (dataMatrix[row][col] === null) {

```

```

        console.log(`Обнаружен null в [${row}][${col}]`);
        foundNull = true;
        continue; // Пропускаем null и все последующие
    }

    sum3 += dataMatrix[row][col];
    processedCells3++;
    console.log(`Обработан [${row}][${col}]: ${dataMatrix[row][col]}`);
}
}

console.log(`\nСумма (пропуск после первого null в строке): ${sum3}`);
console.log(`Обработано ячеек: ${processedCells3}`);

```

2.4. Практические примеры использования continue

Пример 1: Обработка данных с различными типами

javascript

```

console.log("Пример: Обработка разнородных данных");

function processMixedData(data) {
    const results = {
        numbers: [],
        strings: [],
        arrays: [],
        objects: [],
        skipped: []
    };

    console.log("Начало обработки данных:");

    for (let i = 0; i < data.length; i++) {
        const item = data[i];
        console.log(`\n[${i}] ${typeof item} = ${JSON.stringify(item)}`);

        // Пропускаем null и undefined
        if (item === null || item === undefined) {
            console.log("   null/undefined, пропускаем");
            results.skipped.push({ index: i, reason: 'null_or_undefined' });
            continue;
        }
    }
}

```

```

// Обработка по типу
switch (typeof item) {
  case 'number':
    // Пропускаем специальные числовые значения
    if (!isFinite(item)) {
      console.log("   Бесконечность или NaN, пропускаем");
      results.skipped.push({ index: i, reason: 'non_finite_number' });
      continue;
    }

    results.numbers.push({
      value: item,
      squared: item * item,
      isEven: item % 2 === 0
    });
    console.log(`   Число обработано: ${item}`);
    break;

  case 'string':
    // Пропускаем пустые строки
    if (item.trim() === '') {
      console.log("   Пустая строка, пропускаем");
      results.skipped.push({ index: i, reason: 'empty_string' });
      continue;
    }

    results.strings.push({
      value: item,
      length: item.length,
      uppercase: item.toUpperCase()
    });
    console.log(`   Строка обработана: "${item}"`);
    break;

  case 'object':
    // Проверяем, массив ли это
    if (Array.isArray(item)) {
      // Пропускаем пустые массивы
      if (item.length === 0) {
        console.log("   Пустой массив, пропускаем");
        results.skipped.push({ index: i, reason: 'empty_array' });
        continue;
      }
    }

```

```

    results.arrays.push({
      items: item,
      length: item.length,
      first: item[0],
      last: item[item.length - 1]
    });
    console.log(`  Массив обработан: ${item.length} элементов`);
  } else {
    // Обычный объект
    results.objects.push({
      keys: Object.keys(item),
      values: Object.values(item),
      size: Object.keys(item).length
    });
    console.log(`  Объект обработан: ${Object.keys(item).length} свойств`);
  }
  break;

default:
  console.log(`  Неподдерживаемый тип: ${typeof item}, пропускаем`);
  results.skipped.push({ index: i, reason: 'unsupported_type' });
}
}

return results;
}

// Тестовые данные
const mixedData = [
  42,
  "Hello World",
  "",
  null,
  [1, 2, 3],
  { name: "John", age: 30 },
  Infinity,
  -Infinity,
  NaN,
  undefined,
  [],
  {},
  "Another string",
  3.14,

```

```

[4, 5, 6],
function() {}, // функция
true, // boolean
Symbol('test') // символ
];

const processingResults = processMixedData(mixedData);

console.log("\n=== РЕЗУЛЬТАТЫ ОБРАБОТКИ ===");
console.log(`Обработано чисел: ${processingResults.numbers.length}`);
console.log(`Обработано строк: ${processingResults.strings.length}`);
console.log(`Обработано массивов: ${processingResults.arrays.length}`);
console.log(`Обработано объектов: ${processingResults.objects.length}`);
console.log(`Пропущено элементов: ${processingResults.skipped.length}`);
console.log("\nПропущенные элементы:");
processingResults.skipped.forEach(skip => {
  console.log(`  [${skip.index}]: ${skip.reason}`);
});

```

Пример 2: Парсинг и анализ текста

javascript

```

console.log("\nПример: Анализ текста с фильтрацией");

class TextAnalyzer {
  constructor(text) {
    this.text = text;
    this.stopWords = new Set(['и', 'в', 'на', 'с', 'по', 'для', 'не', 'что', 'это',
'как']);
  }

  analyze() {
    const words = this.text.toLowerCase()
      .replace(/[^\w\s]/g, ' ') // Удаляем знаки препинания
      .split(/\s+/)
      .filter(word => word.length > 0);

    const statistics = {
      totalWords: 0,
      uniqueWords: 0,
      stopWordsCount: 0,
      wordFrequency: new Map(),
      longWords: [],
      shortWords: []
    };
  }
}

```



```

};

console.log(`Анализ текста (${words.length} слов):`);

for (let i = 0; i < words.length; i++) {
  const word = words[i];
  statistics.totalWords++;

  // Пропускаем стоп-слова при подсчёте уникальных
  if (this.stopWords.has(word)) {
    statistics.stopWordsCount++;
    console.log(` "${word}" - стоп-слово, пропускаем в анализе`);
    continue; // Пропускаем стоп-слова в основном анализе
  }

  // Подсчёт частоты слов
  const currentCount = statistics.wordFrequency.get(word) || 0;
  statistics.wordFrequency.set(word, currentCount + 1);

  // Классификация по длине
  if (word.length >= 7) {
    statistics.longWords.push(word);
  } else if (word.length <= 3) {
    statistics.shortWords.push(word);
  }
}

statistics.uniqueWords = statistics.wordFrequency.size;

return statistics;
}

printReport(stats) {
  console.log("\n=== ОТЧЁТ ПО АНАЛИЗУ ТЕКСТА ===");
  console.log(`Всего слов: ${stats.totalWords}`);
  console.log(`Уникальных слов: ${stats.uniqueWords}`);
  console.log(`Стоп-слов: ${stats.stopWordsCount}`);
  console.log(`Длинных слов (7+ букв): ${stats.longWords.length}`);
  console.log(`Коротких слов (1-3 буквы): ${stats.shortWords.length}`);

  // Топ-5 самых частых слов
  const sortedWords = Array.from(stats.wordFrequency.entries())
    .sort((a, b) => b[1] - a[1])
    .slice(0, 5);

```

```

    console.log("\nТоп-5 самых частых слов:");
    sortedWords.forEach(([word, count], index) => {
        console.log(`  ${index + 1}. "${word}": ${count} раз`);
    });
}
}

```

// Пример текста для анализа

```

const sampleText = `
JavaScript – это мультипарадигменный язык программирования,
который поддерживает объектно-ориентированный, императивный
и функциональный стили. JavaScript в основном используется
для создания интерактивных веб-страниц и веб-приложений.
Язык также находит применение в других областях, таких как
серверная разработка (Node.js), мобильная разработка и даже
разработка десктопных приложений.
`;

```

```

const analyzer = new TextAnalyzer(sampleText);
const stats = analyzer.analyze();
analyzer.printReport(stats);

```

Пример 3: Обработка транзакций с фильтрацией

javascript

```

console.log("\nПример: Фильтрация и обработка банковских транзакций");

class TransactionProcessor {
    constructor() {
        this.transactions = [];
        this.fraudPatterns = [
            { pattern: /9{4,}/, description: "Много девяток в сумме" },
            { pattern: /0{5,}/, description: "Много нулей в сумме" },
            { pattern: /123456/, description: "Последовательные цифры" }
        ];
    }

    addTransaction(transaction) {
        this.transactions.push(transaction);
    }

    processTransactions() {
        const results = {

```

```

    processed: [],
    skipped: [],
    fraudulent: [],
    summary: {
      totalAmount: 0,
      validCount: 0,
      skippedCount: 0,
      fraudulentCount: 0
    }
  };

  console.log("Начало обработки транзакций:");

  for (let i = 0; i < this.transactions.length; i++) {
    const tx = this.transactions[i];
    console.log(`\nТранзакция ${i + 1}: ${tx.description}`);
    console.log(`    Сумма: ${tx.amount}, Тип: ${tx.type}`);

    // Пропускаем нулевые транзакции
    if (tx.amount === 0) {
      console.log("    □ Нулевая сумма, пропускаем");
      results.skipped.push({
        transaction: tx,
        reason: "zero_amount"
      });
      results.summary.skippedCount++;
      continue;
    }

    // Проверка на мошенничество
    const amountStr = Math.abs(tx.amount).toString();
    let isFraudulent = false;

    for (const pattern of this.fraudPatterns) {
      if (pattern.pattern.test(amountStr)) {
        console.log(`    □ Обнаружен мошеннический паттерн: ${pattern.description}`);
        results.fraudulent.push({
          transaction: tx,
          detectedPattern: pattern.description,
          amountString: amountStr
        });
        results.summary.fraudulentCount++;
        isFraudulent = true;
        break;
      }
    }
  }

```

```

    }
}

    if (isFraudulent) {
        continue; // Пропускаем мошеннические транзакции
    }

    // Валидная транзакция - обрабатываем
    results.processed.push(tx);
    results.summary.totalAmount += tx.amount;
    results.summary.validCount++;

    console.log(`    Транзакция обработана успешно`);
}

    return results;
}
}

// Создаём процессор и добавляем транзакции
const processor = new TransactionProcessor();

processor.addTransaction({ amount: 100, type: "deposit", description: "Пополнение счёта"
});
processor.addTransaction({ amount: -50, type: "withdrawal", description: "Снятие
наличных" });
processor.addTransaction({ amount: 0, type: "transfer", description: "Нулевой перевод"
});
processor.addTransaction({ amount: 9999, type: "deposit", description: "Большой депозит"
});
processor.addTransaction({ amount: -123456, type: "withdrawal", description: "Крупное
снятие" });
processor.addTransaction({ amount: 1000000, type: "deposit", description: "Очень крупный
депозит" });
processor.addTransaction({ amount: 250, type: "payment", description: "Оплата услуг" });
processor.addTransaction({ amount: 50000, type: "deposit", description: "Ещё один
депозит" });

const processingResults = processor.processTransactions();

console.log(`\n=== ИТОГИ ОБРАБОТКИ ТРАНЗАКЦИЙ ===`);
console.log(`Всего транзакций: ${processor.transactions.length}`);
console.log(`Обработано успешно: ${processingResults.summary.validCount}`);
console.log(`Пропущено (нулевых): ${processingResults.summary.skippedCount}`);

```

```
console.log(`Заблокировано (мошеннических): ${processingResults.summary.fraudulentCount}`);
console.log(`Общая сумма обработанных транзакций: ${processingResults.summary.totalAmount}`);

if (processingResults.fraudulent.length > 0) {
  console.log("\nОбнаруженные мошеннические транзакции:");
  processingResults.fraudulent.forEach((fraud, index) => {
    console.log(`${index + 1}. ${fraud.transaction.description}`);
    console.log(`    Сумма: ${fraud.transaction.amount}, Причина: ${fraud.detectedPattern}`);
  });
}
```

Часть 3: Метки (labels) — управление вложенными циклами

3.1. Что такое метки и зачем они нужны?

Метки (labels) — это механизм, позволяющий управлять выполнением **вложенных циклов** с помощью `break` и `continue`. Метка даёт имя блоку кода (обычно циклу), к которому можно обратиться.

Синтаксис меток:

javascript

```
метка: for (/* ... */) {
  // тело внешнего цикла

  for (/* ... */) {
    // тело внутреннего цикла

    if (условие) {
      break метка; // Выходит из внешнего цикла
      // или
      continue метка; // Переходит к следующей итерации внешнего цикла
    }
  }
}
```

```
}
```

3.2. Детальное изучение меток с break

```
javascript
```

```
console.log("Пример: Поиск в матрице с использованием меток");
```

```
// Создаём тестовую матрицу 5x5
```

```
const matrix = [];
```

```
let counter = 1;
```

```
for (let i = 0; i < 5; i++) {  
  matrix[i] = [];  
  for (let j = 0; j < 5; j++) {  
    matrix[i][j] = counter++;  
  }  
}
```

```
// Добавляем специальные значения
```

```
matrix[2][2] = "TARGET"; // Цель поиска
```

```
matrix[3][4] = "SPECIAL"; // Другое специальное значение
```

```
console.log("Матрица 5x5:");
```

```
matrix.forEach((row, i) => {  
  console.log(`Строка ${i}:`, row);  
});
```

```
const targets = ["TARGET", "SPECIAL"];
```

```
console.log("\nЦели поиска:", targets);
```

```
// 1. Поиск с использованием меток и break
```

```
console.log("\n1. Поиск с метками (выход при первой найденной цели:");
```

```
outerSearch: for (let row = 0; row < matrix.length; row++) {  
  for (let col = 0; col < matrix[row].length; col++) {  
    const value = matrix[row][col];  
    console.log(`Проверяем [${row}][${col}] = ${value}`);  
  
    if (targets.includes(value)) {  
      console.log(`   Найдена цель: ${value} на позиции [${row}][${col}]`);  
      console.log("Прекращаем поиск.");  
      break outerSearch; // Выходим из ВСЕХ циклов  
    }  
  }  
}
```

```

    console.log(`Строка ${row} проверена, переходим к следующей...`);
}

// 2. Поиск ВСЕХ целей с continue
console.log("\n2. Поиск всех целей (continue к следующей строке):");

let foundTargets = [];

rowLoop: for (let row = 0; row < matrix.length; row++) {
    console.log(`\nПоиск в строке ${row}:`);

    for (let col = 0; col < matrix[row].length; col++) {
        const value = matrix[row][col];

        if (targets.includes(value)) {
            foundTargets.push({ value, row, col });
            console.log(`  Найдена цель: ${value} на позиции [${row}][${col}]`);
            console.log(`  Пропускаем оставшиеся элементы строки ${row}`);
            continue rowLoop; // Переходим к следующей строке
        }

        console.log(`  [${row}][${col}] = ${value} - не цель`);
    }

    console.log(`  Строка ${row} полностью проверена`);
}

console.log("\nНайдено целей:", foundTargets.length);
foundTargets.forEach((target, index) => {
    console.log(`${index + 1}. ${target.value} на [${target.row}][${target.col}]`);
});

```

3.3. Сложные сценарии с метками и continue

javascript

```

console.log("\nПример: Обработка шахматной доски");

class ChessBoard {
    constructor(size = 8) {
        this.size = size;
        this.board = this.createBoard(size);
        this.addSpecialSquares();
    }
}

```

```

createBoard(size) {
  const board = [];
  for (let i = 0; i < size; i++) {
    board[i] = [];
    for (let j = 0; j < size; j++) {
      board[i][j] = (i + j) % 2 === 0 ? '□' : '■'; // Шахматная раскраска
    }
  }
  return board;
}

```

```

addSpecialSquares() {
  // Добавляем специальные клетки
  this.board[0][0] = '♔'; // Король
  this.board[7][7] = '♚'; // Королева
  this.board[3][3] = '♘'; // Конь
  this.board[2][5] = '♙'; // Слон
  this.board[5][2] = '♖'; // Ладья
  this.board[4][4] = '♟'; // Пешка
  this.board[1][6] = '▲'; // Опасная клетка
  this.board[6][1] = '▲'; // Опасная клетка
}

```

```

printBoard() {
  console.log("Шахматная доска:");
  console.log(" " + Array.from({length: this.size}, (_, i) => i).join(' '));
  for (let i = 0; i < this.size; i++) {
    console.log(i + ' ' + this.board[i].join(' '));
  }
}

```

// Метод с использованием меток

```

analyzeBoard() {
  const analysis = {
    pieces: [],
    dangerousSquares: [],
    emptySquares: 0,
    blackSquares: 0,
    whiteSquares: 0
  };

  console.log("\nАнализ доски с метками:");
}

```



```

rowLoop: for (let row = 0; row < this.size; row++) {
  let rowHasDanger = false;

  colLoop: for (let col = 0; col < this.size; col++) {
    const square = this.board[row][col];

    // Пропускаем опасные строки после обнаружения опасности
    if (square === '▲') {
      console.log(`Обнаружена опасность на [${row}][${col}], пропускаем строку ${row}`);

      analysis.dangerousSquares.push({ row, col });
      rowHasDanger = true;
      continue rowLoop; // Переходим к следующей строке
    }

    // Если в строке уже была опасность, пропускаем оставшиеся клетки
    if (rowHasDanger) {
      console.log(`Пропускаем [${row}][${col}] (в строке была опасность)`);
      continue colLoop; // Пропускаем эту клетку
    }

    // Анализ обычных клеток
    if (square === '□') {
      analysis.whiteSquares++;
    } else if (square === '■') {
      analysis.blackSquares++;
    } else if (square !== '□' && square !== '■') {
      analysis.pieces.push({
        piece: square,
        position: { row, col }
      });
      console.log(`Найдена фигура ${square} на [${row}][${col}]`);
    }

    // Пустая клетка (без фигуры)
    if (square === '□' || square === '■') {
      analysis.emptySquares++;
    }
  }
}

if (!rowHasDanger) {
  console.log(`Строка ${row} полностью безопасна и обработана`);
}
}

```

```

    return analysis;
}

// Альтернативный метод без меток (для сравнения)
analyzeBoardWithoutLabels() {
    const analysis = {
        pieces: [],
        dangerousSquares: [],
        emptySquares: 0,
        blackSquares: 0,
        whiteSquares: 0
    };

    console.log("\nАнализ доски БЕЗ меток:");

    for (let row = 0; row < this.size; row++) {
        let rowHasDanger = false;
        let skipRestOfRow = false;

        for (let col = 0; col < this.size; col++) {
            const square = this.board[row][col];

            // Пропускаем опасные строки после обнаружения опасности
            if (square === '▲') {
                console.log(`Обнаружена опасность на [${row}][${col}], пропускаем строку ${row}`);

                analysis.dangerousSquares.push({ row, col });
                rowHasDanger = true;
                skipRestOfRow = true;
            }

            // Если в строке уже была опасность, пропускаем оставшиеся клетки
            if (skipRestOfRow) {
                console.log(`Пропускаем [${row}][${col}] (в строке была опасность)`);
                continue;
            }

            // Остальной код анализа (такой же как выше)
            // ...
        }
    }

    return analysis;
}

```

```

    }
}

// Демонстрация
const chessBoard = new ChessBoard();
chessBoard.printBoard();
const analysis = chessBoard.analyzeBoard();

console.log("\n=== РЕЗУЛЬТАТЫ АНАЛИЗА ===");
console.log(`Фигур найдено: ${analysis.pieces.length}`);
analysis.pieces.forEach(piece => {
    console.log(` ${piece.piece} на [${piece.position.row}][${piece.position.col}]`);
});
console.log(`Опасных клеток: ${analysis.dangerousSquares.length}`);
console.log(`Пустых клеток: ${analysis.emptySquares}`);
console.log(`Белых клеток: ${analysis.whiteSquares}`);
console.log(`Чёрных клеток: ${analysis.blackSquares}`);

```

3.4. Практическое применение меток в реальных задачах

javascript

```

console.log("Пример: Обработка многомерных данных с сложной логикой");

class DataCubeProcessor {
    constructor(dataCube) {
        this.dataCube = dataCube; // 3D массив: [слой][строка][столбец]
    }

    // Создание тестового куба данных 4x4x4
    static createTestCube() {
        const cube = [];
        let value = 1;

        for (let layer = 0; layer < 4; layer++) {
            cube[layer] = [];
            for (let row = 0; row < 4; row++) {
                cube[layer][row] = [];
                for (let col = 0; col < 4; col++) {
                    // Добавляем некоторые специальные значения
                    if (layer === 1 && row === 2 && col === 3) {
                        cube[layer][row][col] = "ERROR";
                    } else if (layer === 2 && row === 1 && col === 1) {
                        cube[layer][row][col] = "INVALID";
                    }
                }
            }
        }
    }
}

```

```

    } else if (layer === 3 && row === 3 && col === 0) {
        cube[layer][row][col] = "BREAK";
    } else {
        cube[layer][row][col] = value++;
    }
}
}
}

return cube;
}

// Сложная обработка с использованием меток
processComplex() {
    const results = {
        validValues: [],
        errors: [],
        invalidCount: 0,
        processedLayers: 0,
        processedRows: 0,
        processedCells: 0
    };

    console.log("Начало обработки Data Cube (4x4x4):");

    layerLoop: for (let layer = 0; layer < this.dataCube.length; layer++) {
        console.log(`\n=== Обработка слоя ${layer} ===`);
        results.processedLayers++;

        rowLoop: for (let row = 0; row < this.dataCube[layer].length; row++) {
            results.processedRows++;

            for (let col = 0; col < this.dataCube[layer][row].length; col++) {
                results.processedCells++;
                const value = this.dataCube[layer][row][col];

                console.log(` Обработка [${layer}][${row}][${col}] = ${value}`);

                // Критическая ошибка - прекращаем всё
                if (value === "BREAK") {
                    console.log(" ❏ КРИТИЧЕСКАЯ ОШИБКА: получен BREAK");
                    console.log(" Прекращаем обработку ВСЕГО куба данных");
                    break layerLoop; // Выходим из всех циклов
                }
            }
        }
    }
}

```

```

// Ошибка в ячейке - пропускаем всю строку
if (value === "ERROR") {
  console.log(" ⚠ Ошибка в ячейке, пропускаем оставшуюся часть строки");
  continue rowLoop; // Переходим к следующей строке
}

// Невалидное значение - пропускаем ячейку
if (value === "INVALID") {
  console.log(" ❌ Невалидное значение, пропускаем ячейку");
  results.invalidCount++;
  continue; // Пропускаем только эту ячейку
}

// Нормальная обработка
if (typeof value === 'number') {
  const processedValue = {
    original: value,
    squared: value * value,
    cube: value * value * value,
    location: { layer, row, col }
  };
  results.validValues.push(processedValue);
  console.log(` 📊 Число обработано: ${value} → квадрат: $
{processedValue.squared}`);
} else {
  console.log(` ❓ Неизвестный тип: ${typeof value}`);
}
}

console.log(` Строка ${row} полностью обработана`);
}

console.log(` Слой ${layer} полностью обработан`);
}

return results;
}

// Альтернативная реализация без меток
processWithoutLabels() {
  const results = {
    validValues: [],
    errors: [],

```

```

    invalidCount: 0,
    processedLayers: 0,
    processedRows: 0,
    processedCells: 0
  };

  console.log("\nАльтернативная реализация БЕЗ меток:");

  let shouldBreakAll = false;

  for (let layer = 0; layer < this.dataCube.length && !shouldBreakAll; layer++) {
    console.log(`\n=== Обработка слоя ${layer} ===`);
    results.processedLayers++;

    for (let row = 0; row < this.dataCube[layer].length && !shouldBreakAll; row++) {
      results.processedRows++;
      let skipRestOfRow = false;

      for (let col = 0; col < this.dataCube[layer][row].length && !shouldBreakAll && !
skipRestOfRow; col++) {
        results.processedCells++;
        const value = this.dataCube[layer][row][col];

        console.log(` Обработка [${layer}][${row}][${col}] = ${value}`);

        if (value === "BREAK") {
          console.log(" ❏ КРИТИЧЕСКАЯ ОШИБКА: получен BREAK");
          shouldBreakAll = true;
          continue;
        }

        if (value === "ERROR") {
          console.log(" ⚠ Ошибка в ячейке, пропускаем оставшуюся часть строки");
          skipRestOfRow = true;
          continue;
        }

        // ... остальная обработка такая же
      }
    }
  }

  return results;
}

```

```
}
```

```
// Демонстрация
```

```
const dataCube = DataCubeProcessor.createTestCube();
```

```
const processor = new DataCubeProcessor(dataCube);
```

```
const results = processor.processComplex();
```

```
console.log("\n=== ИТОГИ ОБРАБОТКИ DATA CUBE ===");
```

```
console.log(`Обработано слоёв: ${results.processedLayers}`);
```

```
console.log(`Обработано строк: ${results.processedRows}`);
```

```
console.log(`Обработано ячеек: ${results.processedCells}`);
```

```
console.log(`Валидных значений: ${results.validValues.length}`);
```

```
console.log(`Невалидных значений: ${results.invalidCount}`);
```

```
console.log(`Ошибок: ${results.errors.length}`);
```

```
// Покажем несколько обработанных значений
```

```
console.log("\nПервые 5 обработанных значений:");
```

```
results.validValues.slice(0, 5).forEach((value, i) => {
```

```
    console.log(`${i + 1}. ${value.original} → квадрат: ${value.squared}, куб: $
```

```
{value.cube}`);
```

```
});
```

Часть 4: Комбинирование `break`, `continue` и меток

4.1. Сложные сценарии управления

```
javascript
```

```
console.log("Пример: Игра 'Сапёр' (Minesweeper) с использованием управления циклами");
```

```
class MinesweeperGame {
```

```
    constructor(size = 8, mineCount = 10) {
```

```
        this.size = size;
```

```
        this.mineCount = mineCount;
```

```
        this.board = this.createBoard();
```

```
        this.revealed = this.createRevealedBoard();
```

```
        this.gameOver = false;
```

```
        this.win = false;
```

```
    }
```

```

createBoard() {
  // Создаём пустую доску
  const board = [];
  for (let i = 0; i < this.size; i++) {
    board[i] = new Array(this.size).fill(0);
  }

  // Расставляем мины
  let minesPlaced = 0;
  while (minesPlaced < this.mineCount) {
    const row = Math.floor(Math.random() * this.size);
    const col = Math.floor(Math.random() * this.size);

    if (board[row][col] !== '💣') {
      board[row][col] = '💣';
      minesPlaced++;

      // Увеличиваем счётчики вокруг мины
      for (let dr = -1; dr <= 1; dr++) {
        for (let dc = -1; dc <= 1; dc++) {
          const nr = row + dr;
          const nc = col + dc;

          if (nr >= 0 && nr < this.size &&
              nc >= 0 && nc < this.size &&
              board[nr][nc] !== '💣') {
            board[nr][nc]++;
          }
        }
      }
    }
  }

  return board;
}

createRevealedBoard() {
  return new Array(this.size).fill().map(() =>
    new Array(this.size).fill(false)
  );
}

printBoard(showAll = false) {
  console.log("\n    " + Array.from({length: this.size}, (_, i) => i).join(' '));

```



```

for (let row = 0; row < this.size; row++) {
  const rowDisplay = [];
  for (let col = 0; col < this.size; col++) {
    if (showAll || this.revealed[row][col]) {
      rowDisplay.push(this.board[row][col]);
    } else {
      rowDisplay.push('□');
    }
  }
  console.log(`${row}  ${rowDisplay.join(' ')}`);
}
}

revealCell(row, col) {
  if (this.gameOver) {
    console.log("Игра окончена!");
    return;
  }

  // Проверка границ
  if (row < 0 || row >= this.size || col < 0 || col >= this.size) {
    console.log("Неверные координаты!");
    return;
  }

  // Проверка, не открыта ли уже ячейка
  if (this.revealed[row][col]) {
    console.log("Ячейка уже открыта!");
    return;
  }

  console.log(`\nОткрываем ячейку [${row}][${col}]`);

  // Открываем ячейку
  this.revealed[row][col] = true;

  // Проверка на мину
  if (this.board[row][col] === '💣') {
    console.log("💣 БОМБА! Игра окончена!");
    this.gameOver = true;
    this.printBoard(true);
    return;
  }
}

```

```

// Если ячейка пустая (0), открываем соседние рекурсивно
if (this.board[row][col] === 0) {
    console.log("Пустая ячейка, открываем соседей...");

    // Используем стек для обхода в ширину
    const stack = [[row, col]];

    while (stack.length > 0) {
        const [currentRow, currentCol] = stack.pop();

        // Проверяем всех соседей
        for (let dr = -1; dr <= 1; dr++) {
            for (let dc = -1; dc <= 1; dc++) {
                const nr = currentRow + dr;
                const nc = currentCol + dc;

                // Проверяем границы и что ячейка не открыта
                if (nr >= 0 && nr < this.size &&
                    nc >= 0 && nc < this.size &&
                    !this.revealed[nr][nc]) {

                    this.revealed[nr][nc] = true;

                    // Если сосед тоже пустой, добавляем в стек
                    if (this.board[nr][nc] === 0) {
                        stack.push([nr, nc]);
                    }
                }
            }
        }
    }
}

// Проверка победы
this.checkWin();
}

checkWin() {
    // Победа, если все не-минные ячейки открыты
    outerLoop: for (let row = 0; row < this.size; row++) {
        for (let col = 0; col < this.size; col++) {
            if (this.board[row][col] !== '●' && !this.revealed[row][col]) {
                return; // Есть неоткрытые безопасные ячейки
            }
        }
    }
}

```

```

    }
}

// Все безопасные ячейки открыты
this.win = true;
this.gameOver = true;
console.log("\n🏁 ПОБЕДА! Все безопасные ячейки открыты!");
this.printBoard(true);
}

// Автоматический решатель с использованием break/continue
autoSolve() {
    console.log("\n=== АВТОМАТИЧЕСКИЙ РЕШАТЕЛЬ ===");

    let moves = 0;
    const maxMoves = 50;

    moveLoop: while (moves < maxMoves && !this.gameOver) {
        moves++;
        console.log(`\nХод ${moves}:`);

        // Стратегия 1: Ищем безопасные ячейки
        rowLoop: for (let row = 0; row < this.size; row++) {
            for (let col = 0; col < this.size; col++) {
                if (this.revealed[row][col]) {
                    const value = this.board[row][col];

                    // Если вокруг открытой ячейки достаточно информации
                    if (typeof value === 'number' && value > 0) {
                        const result = this.analyzeCell(row, col);

                        if (result.safeToClick.length > 0) {
                            console.log(`Найдены безопасные ячейки вокруг [${row}][${col}]`);
                            this.revealCell(result.safeToClick[0][0], result.safeToClick[0][1]);
                            continue moveLoop; // Переходим к следующему ходу
                        }

                        if (result.minesFound.length > 0) {
                            console.log(`Обнаружены мины вокруг [${row}][${col}]`);
                            // Помечаем мины (в реальной игре это была бы флажок)
                            continue; // Продолжаем поиск в этой строке
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
}

// Стратегия 2: Случайный ход, если не нашли безопасных
console.log("Не найдено безопасных ходов, делаем случайный ход...");

let attempts = 0;
randomLoop: while (attempts < 100) {
    attempts++;
    const row = Math.floor(Math.random() * this.size);
    const col = Math.floor(Math.random() * this.size);

    if (!this.revealed[row][col]) {
        this.revealed[row][col] = true;

        if (this.board[row][col] === '💣') {
            console.log(`💣 Случайный ход [{row}][{col}] попал на мину!`);
            this.gameOver = true;
            break moveLoop; // Выходим из всех циклов
        }

        console.log(`Случайный ход [{row}][{col}] безопасен`);
        break randomLoop; // Выходим из цикла случайных попыток
    }
}

if (attempts >= 100) {
    console.log("Не удалось сделать случайный ход, вероятно игра завершена");
    break moveLoop;
}

// Проверяем победу после каждого хода
this.checkWin();
}

console.log(`\nАвторешатель завершил работу после ${moves} ходов`);
if (this.win) {
    console.log("🏆 Авторешатель выиграл!");
} else if (this.gameOver) {
    console.log("💣 Авторешатель проиграл!");
} else {
    console.log("😞 Авторешатель не смог завершить игру");
}
}

```

```

    this.printBoard(true);
}

analyzeCell(row, col) {
    const result = {
        safeToClick: [],
        minesFound: [],
        unknown: []
    };

    const value = this.board[row][col];
    let hiddenCount = 0;
    let flagCount = 0;

    // Анализируем соседей
    for (let dr = -1; dr <= 1; dr++) {
        for (let dc = -1; dc <= 1; dc++) {
            const nr = row + dr;
            const nc = col + dc;

            if (nr === row && nc === col) continue; // Пропускаем саму ячейку

            if (nr >= 0 && nr < this.size && nc >= 0 && nc < this.size) {
                if (!this.revealed[nr][nc]) {
                    hiddenCount++;
                    result.unknown.push([nr, nc]);
                } else if (this.board[nr][nc] === '🚩') {
                    flagCount++;
                    result.minesFound.push([nr, nc]);
                }
            }
        }
    }

    // Если количество скрытых равно числу на ячейке минус флаги,
    // то все скрытые - мины
    if (hiddenCount === value - flagCount && hiddenCount > 0) {
        result.minesFound.push(...result.unknown);
        result.unknown = [];
    }

    // Если количество флагов равно числу на ячейке,
    // то оставшиеся скрытые безопасны
    if (flagCount === value && hiddenCount > 0) {

```

```

        result.safeToClick.push(...result.unknown);
        result.unknown = [];
    }

    return result;
}
}

// Демонстрация игры
console.log("=== ИГРА 'САПЁР' ===\n");
const game = new MinesweeperGame(6, 5); // Доска 6х6 с 5 минами

console.log("Начальное состояние доски:");
game.printBoard();

// Несколько ходов
game.revealCell(0, 0);
game.revealCell(2, 2);
game.revealCell(4, 3);

console.log("\nСостояние после нескольких ходов:");
game.printBoard();

// Запуск авторешателя
game.autoSolve();

```

4.2. Оптимизация алгоритмов с использованием break и continue

javascript

```

console.log("\nПример: Оптимизация алгоритмов поиска и сортировки");

class AlgorithmOptimizer {
    constructor() {
        this.performanceData = [];
    }

    // Линейный поиск с оптимизацией
    optimizedLinearSearch(arr, target) {
        console.log(`\nОптимизированный линейный поиск: массив из ${arr.length} элементов`);

        // Обычный линейный поиск
        console.time("Обычный линейный поиск");

```

```

let result1 = -1;
for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) {
        result1 = i;
    }
}
console.timeEnd("Обычный линейный поиск");

// Оптимизированный с break
console.time("С break (остановка при нахождении)");
let result2 = -1;
for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) {
        result2 = i;
        break; // Немедленный выход при нахождении
    }
}
console.timeEnd("С break (остановка при нахождении)");

// С предварительной проверкой первого/последнего элемента
console.time("С проверкой границ + break");
let result3 = -1;

// Проверяем границы
if (arr[0] === target) {
    result3 = 0;
} else if (arr[arr.length - 1] === target) {
    result3 = arr.length - 1;
} else {
    // Ищем в остальной части
    for (let i = 1; i < arr.length - 1; i++) {
        if (arr[i] === target) {
            result3 = i;
            break;
        }
    }
}
console.timeEnd("С проверкой границ + break");

return { result1, result2, result3 };
}

// Пузырьковая сортировка с оптимизациями
optimizedBubbleSort(arr) {

```

```

console.log(`\nОптимизированная пузырьковая сортировка: ${arr.length} элементов`);
const arr1 = [...arr];
const arr2 = [...arr];
const arr3 = [...arr];

// Стандартная пузырьковая сортировка
console.time("Стандартная пузырьковая");
for (let i = 0; i < arr1.length; i++) {
  for (let j = 0; j < arr1.length - 1; j++) {
    if (arr1[j] > arr1[j + 1]) {
      [arr1[j], arr1[j + 1]] = [arr1[j + 1], arr1[j]];
    }
  }
}
console.timeEnd("Стандартная пузырьковая");

// С оптимизацией (уменьшение внутреннего цикла)
console.time("С уменьшением внутреннего цикла");
for (let i = 0; i < arr2.length; i++) {
  // Уменьшаем внутренний цикл на i
  for (let j = 0; j < arr2.length - 1 - i; j++) {
    if (arr2[j] > arr2[j + 1]) {
      [arr2[j], arr2[j + 1]] = [arr2[j + 1], arr2[j]];
    }
  }
}
console.timeEnd("С уменьшением внутреннего цикла");

// С break при отсутствии обменов
console.time("С break (ранняя остановка)");
let swapped;

for (let i = 0; i < arr3.length; i++) {
  swapped = false;

  for (let j = 0; j < arr3.length - 1 - i; j++) {
    if (arr3[j] > arr3[j + 1]) {
      [arr3[j], arr3[j + 1]] = [arr3[j + 1], arr3[j]];
      swapped = true;
    }
  }
}

// Если не было обменов, массив отсортирован
if (!swapped) {

```



```

        console.log(` Ранняя остановка на итерации ${i + 1}`);
        break;
    }
}
console.timeEnd("C break (ранняя остановка)");

return { arr1, arr2, arr3 };
}

// Поиск в графе с использованием break и continue
graphSearch() {
    console.log("\nПоиск в графе с использованием break/continue");

    // Создаём тестовый граф (список смежности)
    const graph = {
        A: ['B', 'C'],
        B: ['A', 'D', 'E'],
        C: ['A', 'F'],
        D: ['B'],
        E: ['B', 'F'],
        F: ['C', 'E', 'G'],
        G: ['F', 'H'],
        H: ['G'],
        I: ['J'],
        J: ['I']
    };

    const target = 'G';
    console.log(`Поиск пути к вершине ${target}`);

    // Поиск в ширину (BFS) с break
    console.time("BFS с break");
    const visitedBFS = new Set();
    const queueBFS = [['A', ['A']]]; // [текущая_вершина, путь]
    let bfsPath = null;

    bfsLoop: while (queueBFS.length > 0) {
        const [current, path] = queueBFS.shift();

        if (visitedBFS.has(current)) {
            continue; // Пропускаем уже посещённые вершины
        }

        visitedBFS.add(current);

```

```

    if (current === target) {
        bfsPath = path;
        console.log(` Найдено за ${path.length - 1} шагов: ${path.join(' → ')} `);
        break bfsLoop; // Выходим при нахождении цели
    }

    // Добавляем соседей в очередь
    for (const neighbor of graph[current] || []) {
        if (!visitedBFS.has(neighbor)) {
            queueBFS.push([neighbor, [...path, neighbor]]);
        }
    }
}

console.timeEnd("BFS с break");

// Поиск в глубину (DFS) с continue для пропуска
console.time("DFS с continue для циклов");
const visitedDFS = new Set();
const stackDFS = [['A', ['A']]];
let dfsPath = null;

dfsLoop: while (stackDFS.length > 0) {
    const [current, path] = stackDFS.pop();

    // Пропускаем вершины, которые уже есть в текущем пути (циклы)
    if (path.slice(0, -1).includes(current)) {
        console.log(` Обнаружен цикл с вершиной ${current}, пропускаем `);
        continue dfsLoop;
    }

    if (current === target) {
        dfsPath = path;
        console.log(` Найдено за ${path.length - 1} шагов: ${path.join(' → ')} `);
        break dfsLoop;
    }

    visitedDFS.add(current);

    // Добавляем соседей в стек (в обратном порядке для определённого порядка обхода)
    const neighbors = graph[current] || [];
    for (let i = neighbors.length - 1; i >= 0; i--) {
        const neighbor = neighbors[i];
        if (!visitedDFS.has(neighbor)) {

```

```

        stackDFS.push([neighbor, [...path, neighbor]]);
    }
}
}
console.timeEnd("DFS с continue для циклов");

return { bfsPath, dfsPath };
}
}

// Демонстрация оптимизаций
const optimizer = new AlgorithmOptimizer();

// Тест поиска
const largeArray = Array.from({length: 1000000}, (_, i) => i);
const searchTarget = 999999;
optimizer.optimizedLinearSearch(largeArray, searchTarget);

// Тест сортировки
const unsortedArray = Array.from({length: 10000}, () => Math.floor(Math.random() *
10000));
optimizer.optimizedBubbleSort(unsortedArray);

// Тест поиска в графе
optimizer.graphSearch();

```

Часть 5: Лучшие практики и антипаттерны

5.1. Когда использовать, а когда избегать

Хорошие случаи для `break`:

```

javascript

// 1. Поиск элемента в коллекции
function findUserById(users, id) {
    for (const user of users) {
        if (user.id === id) {
            return user; // Ранний возврат - альтернатива break
        }
    }
}

```

```

    }
    return null;
}

// 2. Обработка до условия
function processUntilError(operations) {
    for (const op of operations) {
        try {
            op.execute();
        } catch (error) {
            console.log("Ошибка, прекращаем обработку:", error.message);
            break; // Прекращаем при первой ошибке
        }
    }
}

```

```

// 3. Выход из бесконечного цикла
function waitForCondition(checkCondition, timeout = 5000) {
    const startTime = Date.now();

    while (true) {
        if (checkCondition()) {
            console.log("Условие выполнено");
            break;
        }

        if (Date.now() - startTime > timeout) {
            console.log("Таймаут");
            break;
        }

        // Имитация ожидания
        // В реальном коде здесь может быть await или setTimeout
    }
}

```

Хорошие случаи для `continue`:

javascript

```

// 1. Пропуск невалидных данных
function processValidNumbers(numbers) {
    const validNumbers = [];

    for (const num of numbers) {

```

```

    if (typeof num !== 'number' || isNaN(num)) {
        continue; // Пропускаем невалидные
    }

    if (num < 0) {
        continue; // Пропускаем отрицательные
    }

    validNumbers.push(num);
}

return validNumbers;
}

// 2. Пропуск уже обработанных элементов
function processUniqueItems(items, processedSet) {
    for (const item of items) {
        if (processedSet.has(item.id)) {
            continue; // Уже обработан
        }

        // Обработка...
        processedSet.add(item.id);
    }
}

// 3. Разные стратегии обработки
function processWithFallback(items) {
    for (const item of items) {
        // Попробовать основной метод
        if (tryPrimaryMethod(item)) {
            continue; // Успешно, переходим к следующему
        }

        // Попробовать запасной метод
        if (tryFallbackMethod(item)) {
            continue; // Успешно
        }

        // Если оба метода не сработали
        logFailedItem(item);
    }
}

```

Антипаттерны (чего НЕ делать):

javascript

// 1. Использование break/continue вместо if-else

// ПЛОХО:

```
for (const item of items) {  
  if (!isValid(item)) {  
    continue;  
  }  
  
  if (!isAvailable(item)) {  
    continue;  
  }  
  
  if (!isAffordable(item)) {  
    continue;  
  }  
  
  process(item);  
}
```

// ЛУЧШЕ:

```
for (const item of items) {  
  if (isValid(item) && isAvailable(item) && isAffordable(item)) {  
    process(item);  
  }  
}
```

// 2. Слишком много break/continue в одном цикле

// ПЛОХО:

```
for (const data of dataset) {  
  if (condition1(data)) continue;  
  if (condition2(data)) break;  
  if (condition3(data)) continue;  
  if (condition4(data)) return;  
  if (condition5(data)) continue;  
  // ... сложно понять логику  
}
```

// 3. break в функциях высшего порядка

// ПЛОХО (не работает как ожидается):

```
[1, 2, 3, 4, 5].forEach(num => {  
  if (num === 3) {
```

```

    break; // ОШИБКА: break нельзя использовать в forEach
  }
  console.log(num);
});

// ЛУЧШЕ:
for (const num of [1, 2, 3, 4, 5]) {
  if (num === 3) break;
  console.log(num);
}

// ИЛИ используйте for...of с break

```

5.2. Альтернативы break и continue

Использование функций с ранним возвратом:

```

javascript

// Вместо break в цикле
function findFirstEven(numbers) {
  for (const num of numbers) {
    if (num % 2 === 0) {
      return num; // Ранний возврат вместо break
    }
  }
  return null;
}

// Вместо continue с фильтрацией
function processOnlyValid(items) {
  const validItems = items.filter(item =>
    isValid(item) &&
    isAvailable(item) &&
    isAffordable(item)
  );

  for (const item of validItems) {
    process(item);
  }
}

// Использование some/every

```

```
function checkAllPositive(numbers) {
  return numbers.every(num => num > 0);
}
```

```
function hasAnyNegative(numbers) {
  return numbers.some(num => num < 0);
}
```

Использование генераторов для ленивой обработки:

javascript

```
function* filterMap(iterable, filterFn, mapFn) {
  for (const item of iterable) {
    if (filterFn(item)) {
      yield mapFn(item);
    }
    // continue встроен в условие
  }
}
```

// Использование

```
const numbers = [1, 2, 3, 4, 5, 6];
const result = Array.from(
  filterMap(
    numbers,
    n => n % 2 === 0, // Фильтр
    n => n * 10       // Преобразование
  )
);
console.log(result); // [20, 40, 60]
```

5.3. Производительность и читаемость

Измерение производительности:

javascript

```
function measurePerformance() {
  const largeArray = Array.from({length: 1000000}, (_, i) => i);

  // Тест 1: break vs без break
  console.time("C break (останавливаемся на середине)");
  let found1 = false;
```



```

for (const num of largeArray) {
  if (num === 500000) {
    found1 = true;
    break;
  }
}
console.timeEnd("C break (останавливаемся на середине)");

console.time("Без break (всегда весь массив)");
let found2 = false;
for (const num of largeArray) {
  if (num === 500000) {
    found2 = true;
  }
}
console.timeEnd("Без break (всегда весь массив)");

// Тест 2: continue vs фильтрация
console.time("\nC continue (пропуск половины элементов)");
let sum1 = 0;
for (const num of largeArray) {
  if (num % 2 === 0) {
    continue; // Пропускаем чётные
  }
  sum1 += num;
}
console.timeEnd("C continue (пропуск половины элементов)");

console.time("C фильтрацией (предварительный filter)");
const oddNumbers = largeArray.filter(n => n % 2 !== 0);
let sum2 = 0;
for (const num of oddNumbers) {
  sum2 += num;
}
console.timeEnd("C фильтрацией (предварительный filter)");

console.time("Без continue (условие в теле)");
let sum3 = 0;
for (const num of largeArray) {
  if (num % 2 !== 0) {
    sum3 += num;
  }
}
console.timeEnd("Без continue (условие в теле)");

```

```
}
```

Рекомендации по читаемости:

javascript

```
// 1. Используйте осмысленные имена переменных флагов
```

```
// ПЛОХО:
```

```
let f = false;
for (const item of items) {
  if (item.value > 10) {
    f = true;
    break;
  }
}
```

```
// ХОРОШО:
```

```
let foundHighValue = false;
for (const item of items) {
  if (item.value > 10) {
    foundHighValue = true;
    break;
  }
}
```

```
// 2. Комментируйте сложную логику с break/continue
```

```
for (const transaction of transactions) {
  // Пропускаем тестовые транзакции
  if (transaction.isTest) {
    continue;
  }
}
```

```
// Прекращаем обработку при обнаружении мошенничества
```

```
if (isFraudulent(transaction)) {
  console.log("Обнаружено мошенничество, прекращаем обработку");
  break;
}
```

```
// Нормальная обработка
```

```
processTransaction(transaction);
}
```

```
// 3. Рассмотрите рефакторинг сложных циклов
```

```
function processComplexLoop(items) {
  // Вместо одного сложного цикла
```

```
const validItems = items.filter(isValid);
const availableItems = validItems.filter(isAvailable);

for (const item of availableItems) {
  // Простая обработка без continue
  processItem(item);
}

// 4. Используйте guard clauses (условия-стражи)
function processItem(item) {
  // Guard clauses в начале функции
  if (!isValid(item)) return;
  if (!isAvailable(item)) return;
  if (!isAffordable(item)) return;

  // Основная логика
  // ...
}
```

Заключение: ключевые выводы

Итоговые рекомендации:

1. Используйте `break` для:

- ➔ Досрочного выхода из цикла при достижении цели
- ➔ Прерывания бесконечных циклов по условию
- ➔ Оптимизации поиска (остановка при нахождении)
- ➔ Обработки ошибок (прекращение при ошибке)

2. Используйте `continue` для:

- ➔ Пропуска невалидных или нерелевантных данных
- ➔ Реализации разных путей обработки в одном цикле
- ➔ Пропуска уже обработанных элементов
- ➔ Упрощения сложных условий (guard clauses в цикле)

3. Используйте метки, когда:

- ➔ Нужно выйти из нескольких вложенных циклов
- ➔ Требуется сложная логика пропуска итераций
- ➔ Альтернативные решения с флагами слишком сложны

4. Избегайте, когда:

- ➔ Можно использовать более простые конструкции (if-else)
- ➔ Логика становится слишком сложной для понимания
- ➔ Есть более декларативные альтернативы (filter, find, some, every)

Золотые правила:

- 1. Всегда** документируйте нетривиальное использование break/continue
- 2. Рассматривайте** альтернативы: функции с ранним возвратом, фильтрацию, генераторы
- 3. Измеряйте** производительность при оптимизации с break/continue
- 4. Не злоупотребляйте** метками — они делают код менее читаемым
- 5. Помните,** что break/continue работают только в своих циклах

Выбор правильного инструмента:

javascript

// 1. Для простого поиска:

```
const found = array.find(item => item.id === targetId); // 
```

// vs

```
for (const item of array) { //   
  if (item.id === targetId) {  
    found = item;  
    break;  
  }  
}
```

// 2. Для фильтрации:

```
const validItems = array.filter(isValid); // 
```

// vs

```
const validItems = [];  
for (const item of array) { //   
  if (!isValid(item)) continue;  
  validItems.push(item);  
}
```

// 3. Для поиска с break (когда нужна производительность):

```
let result;  
for (const item of largeArray) { //   
  if (item === target) {  
    result = item;  
    break; // Останавливаемся сразу  
  }  
}
```

```
}
```

```
// 4. Для сложной логики с continue:
```

```
for (const item of items) {  
  if (shouldSkipForReason1(item)) continue;  
  if (shouldSkipForReason2(item)) continue;  
  if (shouldSkipForReason3(item)) continue;
```

```
  // Основная обработка
```

```
  processItem(item);
```

```
  // 
```

```
}
```

Помните:

`break` и `continue` — это инструменты для **управления потоком выполнения** в циклах. Они делают код более эффективным, но иногда менее читаемым. Используйте их осознанно, документируйте сложные случаи и всегда рассматривайте более простые альтернативы.

Правильное использование `break`, `continue` и меток — признак опытного разработчика, который понимает не только как заставить код работать, но и как сделать его эффективным, поддерживаемым и понятным для коллег.

Введение: когда и зачем нужны метки в JavaScript

Метки в JavaScript — это редко используемый, но мощный инструмент для управления выполнением **вложенных циклов** и **блоков кода**. Они позволяют явно указать, какой цикл или блок следует прервать или продолжить, что особенно полезно в сложных сценариях с несколькими уровнями вложенности.

Ключевые сценарии использования меток:

- Выход из нескольких вложенных циклов одновременно
 - Пропуск итераций во внешних циклах
 - Улучшение читаемости сложных циклов с условиями
-

Часть 1: Синтаксис меток в JavaScript

1.1. Объявление меток

Метка объявляется перед циклом или блоком кода и состоит из идентификатора, за которым следует двоеточие:

```
javascript

// Общий синтаксис
labelName:
for (let i = 0; i < 3; i++) {
  // тело цикла
}

// Метка для блока кода (используется редко)
blockLabel: {
  console.log('Начало блока');
  break blockLabel;
  console.log('Этот код не выполнится');
}
```

1.2. Использование меток с break и continue

Метки работают только в сочетании с операторами `break` и `continue`:

javascript

```
// Без метки - break прерывает только внутренний цикл
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break; // Прерывает только внутренний цикл
    }
    console.log(i, j);
  }
}
// Вывод: 0 0, 0 1, 0 2, 1 0, 2 0, 2 1, 2 2
```

```
// С меткой - break прерывает внешний цикл
outerLoop:
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break outerLoop; // Прерывает ВСЕ циклы
    }
    console.log(i, j);
  }
}
// Вывод: 0 0, 0 1, 0 2, 1 0
```

Часть 2: Практические примеры использования меток

2.1. Поиск в двумерном массиве

javascript

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

const target = 5;
```

```

let foundAt = null;

matrixLoop:
for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] === target) {
      foundAt = { row, col };
      break matrixLoop; // Выход из всех циклов сразу
    }
  }
}

console.log(foundAt); // { row: 1, col: 1 }

```

2.2. Валидация сложных данных

```

javascript

const dataSets = [
  [1, 2, 3],
  [4, -1, 6], // Отрицательное число
  [7, 8, 9]
];

let isValid = true;
let errorLocation = null;

validationLoop:
for (let setIndex = 0; setIndex < dataSets.length; setIndex++) {
  const dataSet = dataSets[setIndex];

  for (let numIndex = 0; numIndex < dataSet.length; numIndex++) {
    if (dataSet[numIndex] < 0) {
      isValid = false;
      errorLocation = { setIndex, numIndex, value: dataSet[numIndex] };
      break validationLoop; // Прекращаем проверку при первой ошибке
    }
  }
}

console.log(isValid ? 'Все данные валидны' : `Ошибка в ${JSON.stringify(errorLocation)}`);

```


2.3. Продвинутое использование continue с метками

```
javascript

matrixProcessing:
for (let i = 0; i < 4; i++) {
  console.log(`Внешний цикл: i = ${i}`);

  for (let j = 0; j < 4; j++) {
    if (i === 1 && j === 1) {
      console.log('Пропускаем всю итерацию i = 1');
      continue matrixProcessing; // Переходим к следующей итерации внешнего цикла
    }

    if (i === 2 && j === 2) {
      console.log('Прерываем все циклы');
      break matrixProcessing; // Полностью выходим
    }

    console.log(`  Внутренний цикл: j = ${j}`);
  }
}

// Вывод:
// Внешний цикл: i = 0
//   Внутренний цикл: j = 0...3
// Внешний цикл: i = 1
// Пропускаем всю итерацию i = 1
// Внешний цикл: i = 2
//   Внутренний цикл: j = 0
//   Внутренний цикл: j = 1
// Прерываем все циклы
```

Часть 3: Метки с блоками кода (редкие случаи)

3.1. Выход из блока кода

```
javascript

function processUserData(user) {
  let result = null;

  dataCheck: {
```

```

// Проверка 1: пользователь существует
if (!user) {
  result = { error: 'Пользователь не существует' };
  break dataCheck;
}

// Проверка 2: email валиден
if (!user.email || !user.email.includes('@')) {
  result = { error: 'Неверный email' };
  break dataCheck;
}

// Проверка 3: возраст корректен
if (user.age < 0 || user.age > 150) {
  result = { error: 'Неверный возраст' };
  break dataCheck;
}

// Все проверки пройдены
result = {
  success: true,
  data: {
    normalizedEmail: user.email.toLowerCase(),
    birthYear: new Date().getFullYear() - user.age
  }
};
}

return result;
}

console.log(processUserData({ email: 'test@example.com', age: 25 }));
// { success: true, data: { normalizedEmail: 'test@example.com', birthYear: ... } }

console.log(processUserData({ email: 'invalid', age: 25 }));
// { error: 'Неверный email' }

```

3.2. Имитация goto (не рекомендуется, но возможно)

```

javascript

let attempts = 0;

retryOperation: {
  console.log(`Попытка ${attempts + 1}`);

```

```
try {
  // Имитация операции, которая может завершиться ошибкой
  if (Math.random() < 0.7) {
    throw new Error('Временная ошибка');
  }

  console.log('Операция успешна!');
  break retryOperation; // Выход из блока при успехе
} catch (error) {
  console.log(`Ошибка: ${error.message}`);
  attempts++;

  if (attempts < 3) {
    console.log('Повторяем попытку...');
    continue retryOperation; // Начинаем блок заново
  } else {
    console.log('Превышено количество попыток');
  }
}
}
```

Часть 4: Вложенные циклы без меток (альтернативные подходы)

4.1. Использование функций для выхода

```
javascript

function findInMatrix(matrix, target) {
  for (let row = 0; row < matrix.length; row++) {
    for (let col = 0; col < matrix[row].length; col++) {
      if (matrix[row][col] === target) {
        return { row, col }; // Ранний возврат из функции
      }
    }
  }
  return null;
}

const matrix = [[1, 2], [3, 4]];
```

```
console.log(findInMatrix(matrix, 3)); // { row: 1, col: 0 }
```

4.2. Флаги для управления циклами

javascript

```
let found = false;
let result = null;

for (let i = 0; i < 3 && !found; i++) {
  for (let j = 0; j < 3 && !found; j++) {
    if (i === 1 && j === 1) {
      found = true;
      result = { i, j };
    }
  }
}
```

```
console.log(result); // { i: 1, j: 1 }
```

4.3. Использование some() или every() для массивов

javascript

```
const matrix = [[1, 2], [3, 4]];
let targetFound = false;

matrix.some((row, rowIndex) => {
  return row.some((value, colIndex) => {
    if (value === 3) {
      console.log(`Найдено в [${rowIndex}][${colIndex}]`);
      targetFound = true;
      return true; // Прерывает both some loops
    }
    return false;
  });
});
```

Часть 5: Лучшие практики и предостережения

5.1. Когда использовать метки:

1. Глубоко вложенные циклы (3+ уровня)

2. **Сложные алгоритмы поиска** в многомерных структурах
3. **Ситуации, где нужен немедленный выход** из нескольких уровней
4. **Улучшение читаемости** (когда альтернативы сложнее)

5.2. Когда избегать меток:

1. **Простые циклы** (1-2 уровня вложенности)
2. **Когда можно использовать функции** для инкапсуляции логики
3. **Если код станет менее понятным** для команды
4. **Вместо меток иногда лучше переписать алгоритм**

5.3. Правила именования меток:

```
javascript

// ХОРОШО - описательные имена
outerLoop:
searchGrid:
validationBlock:

// ПЛОХО - непонятные имена
a:
loop1:
x:
```

5.4. Ограничения меток:

```
javascript

// 1. Нельзя использовать с return
function test() {
  myLabel: {
    // return myLabel; // ОШИБКА: метки не работают с return
    break myLabel; // ОК
  }
}

// 2. Метки видны только в текущей области видимости
function outer() {
  outerLabel:
  for (let i = 0; i < 3; i++) {
    function inner() {
      // break outerLabel; // ОШИБКА: метка не видна внутри функции
    }
    inner();
  }
}
```

```
}
```

```
// 3. Нельзя переиспользовать метки в одной области
```

```
{
```

```
  // label: console.log('first'); // ОШИБКА при повторном объявлении
```

```
  // label: console.log('second');
```

```
}
```

Часть 6: Продвинутые техники с метками

6.1. Обработка сложных структур данных

```
javascript
```

```
const warehouse = [  
  {  
    section: 'A',  
    shelves: [  
      { id: 'A1', items: ['apple', 'banana'] },  
      { id: 'A2', items: ['orange', null] } // null - брак  
    ]  
  },  
  {  
    section: 'B',  
    shelves: [  
      { id: 'B1', items: ['grape', 'melon'] }  
    ]  
  }  
];
```

```
// Поиск бракованного товара
```

```
let defectiveItem = null;  
  
warehouseSearch:  
for (const section of warehouse) {  
  for (const shelf of section.shelves) {  
    const defectiveIndex = shelf.items.indexOf(null);  
  
    if (defectiveIndex !== -1) {  
      defectiveItem = {  
        section: section.section,  

```

```

    shelf: shelf.id,
    position: defectiveIndex
  };
  break warehouseSearch;
}
}
}

```

```
console.log(defectiveItem); // { section: 'A', shelf: 'A2', position: 1 }
```

6.2. Игровые циклы и симуляции

javascript

```

class Game {
  constructor() {
    this.board = [
      [' ', 'X', ' '],
      [' ', 'X', ' '],
      [' ', 'X', ' ']
    ];
    this.gameOver = false;
  }

  checkWin() {
    // Проверка вертикальных линий
    columnCheck:
    for (let col = 0; col < 3; col++) {
      const firstCell = this.board[0][col];

      if (firstCell === ' ') {
        continue columnCheck;
      }

      for (let row = 1; row < 3; row++) {
        if (this.board[row][col] !== firstCell) {
          continue columnCheck;
        }
      }

      console.log(`Игрок ${firstCell} выиграл по колонке ${col}`);
      this.gameOver = true;
      break columnCheck;
    }
  }
}

```

```
}
```

```
const game = new Game();  
game.checkWin(); // Игрок X выиграл по колонке 1
```

Заключение: баланс между мощностью и читаемостью

Метки и вложенные циклы — это инструменты, которые при грамотном использовании могут значительно упростить сложные алгоритмы. Однако их следует применять с осторожностью:

- 1. Приоритет читаемости** — если метки делают код понятнее, используйте их
- 2. Рассмотрите альтернативы** — функции, флаги, методы массивов
- 3. Документируйте сложные метки** — комментарии объясняют нетривиальное использование
- 4. Следуйте соглашениям команды** — единый стиль важнее индивидуальных предпочтений

Итоговый пример: сбалансированный подход

```
javascript  
  
/**  
 * Поиск пути в лабиринте с использованием меток  
 * для эффективного выхода при нахождении решения  
 */  
function findPathInMaze(maze, start, end) {  
  const visited = new Set();  
  
  // Используем метку для немедленного выхода при нахождении пути  
  pathSearch:  
  for (const pathStrategy of ['dfs', 'bfs']) {  
    console.log(`Пробуем стратегию: ${pathStrategy}`);  
  
    for (let attempt = 1; attempt <= 3; attempt++) {  
      console.log(` Попытка ${attempt}`);  
  
      const path = pathStrategy === 'dfs'  
        ? depthFirstSearch(maze, start, end, visited)  
        : breadthFirstSearch(maze, start, end, visited);  
  
      if (path) {
```



```

    console.log(` Найден путь длиной ${path.length} с помощью ${pathStrategy}`);
    break pathSearch; // Выходим из всех циклов
  }
}

return null;
}

// Альтернатива без меток (более читаемая для простых случаев)
function findPathSimple(maze, start, end) {
  const strategies = ['dfs', 'bfs'];

  for (const strategy of strategies) {
    const path = tryStrategy(maze, start, end, strategy);
    if (path) return path;
  }

  return null;
}

```

Запомните: Метки — это специализированный инструмент для специфических задач. Их уместное использование демонстрирует глубокое понимание JavaScript, но злоупотребление может привести к сложному для поддержки коду. Всегда оценивайте, является ли метка лучшим решением для вашей конкретной задачи.

Глава 7: Функции

7.1. Объявление и вызов функций

Введение: фундаментальная роль функций в JavaScript

Функции в JavaScript — это **строительные блоки** языка, которые позволяют инкапсулировать логику, повторно использовать код и создавать абстракции. Понимание различных способов объявления и вызова функций — критически важный навык для любого JavaScript-разработчика.

Ключевые аспекты функций:

- Функции являются **объектами первого класса** (first-class citizens)
- Поддерживают различные стили объявления и вызова
- Имеют лексическую область видимости (lexical scoping)
- Могут быть вызваны различными способами с разным контекстом

Часть 1: Способы объявления функций

1.1. Function Declaration (Декларация функции)

Синтаксис и особенности:

```
javascript

// Базовый синтаксис
function functionName(parameters) {
  // тело функции
  return value; // опционально
}

// Конкретный пример
function calculateSum(a, b) {
  const sum = a + b;
  return sum;
}
```

```
}
```

```
// Использование
```

```
const result = calculateSum(5, 3); // 8
```

Особенности Function Declaration:

1.Поднятие (Hoisting):

javascript

```
// Можно вызывать ДО объявления
```

```
console.log(greet('Анна')); // "Привет, Анна!" (работает!)
```

```
function greet(name) {  
  return `Привет, ${name}!`;  
}
```

```
// Под капотом происходит:
```

```
// 1. Объявление функции поднимается вверх области видимости
```

```
// 2. Вызов выполняется
```

2.Блочная область видимости (ES6+):

javascript

```
if (true) {  
  function innerFunc() {  
    return 'Внутри блока';  
  }  
  console.log(innerFunc()); // "Внутри блока"  
}
```

```
// console.log(innerFunc()); // ОШИБКА в строгом режиме
```

3.Именованние и соглашения:

javascript

```
// camelCase для имен функций
```

```
function calculateTotalPrice() {} // ✓ Хорошо
```

```
function CalculateTotalPrice() {} // ✗ Не рекомендуется
```

```
function calculate_total_price() {} // ✗ Не JavaScript стиль
```

```
// Глаголы в начале имени
```

```
function getUserData() {} // ✓
```

```
function dataGetUser() {} // ✗
```

1.2. Function Expression (Функциональное выражение)

Синтаксис и особенности:

javascript

// Базовый синтаксис

```
const functionName = function(parameters) {  
  // тело функции  
};
```

// Конкретный пример

```
const multiply = function(x, y) {  
  return x * y;  
};
```

```
console.log(multiply(4, 5)); // 20
```

Различия от Function Declaration:

javascript

// НЕТ поднятия (hoisting)

// console.log(square(5)); // ОШИБКА: Cannot access before initialization

```
const square = function(x) {  
  return x * x;  
};
```

```
console.log(square(5)); // 25 (теперь работает)
```

// Именованные функциональные выражения

```
const factorial = function calculateFactorial(n) {  
  if (n <= 1) return 1;  
  return n * calculateFactorial(n - 1); // рекурсия с использованием имени  
};
```

```
console.log(factorial(5)); // 120
```

// console.log(calculateFactorial(5)); // ОШИБКА - имя доступно только внутри

Преимущества функциональных выражений:

javascript

// 1. Присваивание условно

```
const operation = condition  
  ? function(a, b) { return a + b; }  
  : function(a, b) { return a - b; };
```

// 2. Передача как аргумент

```
const numbers = [1, 2, 3];
```

```
const doubled = numbers.map(function(n) {  
  return n * 2;  
});  
  
// 3. Создание методов объекта  
const calculator = {  
  add: function(a, b) { return a + b; },  
  subtract: function(a, b) { return a - b; }  
};
```

1.3. Стрелочные функции (Arrow Functions, ES6+)

Синтаксис и особенности:

```
javascript  
  
// Базовый синтаксис  
const functionName = (parameters) => {  
  // тело функции  
};  
  
// Варианты синтаксиса:  
const add = (a, b) => {  
  return a + b;  
};  
  
// Неявный возврат (implicit return)  
const multiply = (a, b) => a * b;  
  
// Один параметр - можно без скобок  
const square = x => x * x;  
  
// Нет параметров - обязательны скобки  
const getTime = () => new Date();  
  
// Возврат объекта - нужны скобки  
const createUser = (name, age) => ({ name, age });
```

Ключевые отличия стрелочных функций:

```
javascript  
  
// 1. Нет собственного this (лексический this)  
const person = {  
  name: 'Анна',  
  traditional: function() {
```

```

    console.log(this.name); // 'Анна'
  },
  arrow: () => {
    console.log(this.name); // undefined (берет this из внешней области)
  }
};

// 2. Нет arguments объекта
function traditionalFunc() {
  console.log(arguments); // Arguments(3) [1, 2, 3]
}

const arrowFunc = () => {
  // console.log(arguments); // ОШИБКА: arguments is not defined
};

// 3. Не могут быть конструкторами
function Car(model) {
  this.model = model;
}
const myCar = new Car('Tesla'); // OK

const ArrowCar = (model) => {
  this.model = model;
};
// const myArrowCar = new ArrowCar('Tesla'); // ОШИБКА: ArrowCar is not a constructor

// 4. Нет свойства prototype
console.log(traditionalFunc.prototype); // {constructor: f}
console.log(arrowFunc.prototype); // undefined

```

1.4. Методы в объектах и классах

Синтаксис ES5 и ES6:

```

javascript

// ES5: традиционный подход
const userES5 = {
  name: 'Иван',
  greet: function() {
    return `Привет, ${this.name}!`;
  }
};

```

// ES6: сокращенный синтаксис методов

```
const userES6 = {  
  name: 'Мария',  
  greet() { // Без ключевого слова function  
    return `Привет, ${this.name}!`;  
  }  
};
```

// ES6: вычисляемые имена методов

```
const methodName = 'calculate';  
const calculator = {  
  [methodName + 'Sum'](a, b) {  
    return a + b;  
  }  
};
```

```
console.log(calculator.calculateSum(2, 3)); // 5
```

Геттеры и сеттеры:

javascript

```
const person = {  
  firstName: 'Анна',  
  lastName: 'Иванова',  
  
  // Геттер  
  get fullName() {  
    return `${this.firstName} ${this.lastName}`;  
  },
```

```
  // Сеттер  
  set fullName(name) {  
    const parts = name.split(' ');  
    this.firstName = parts[0];  
    this.lastName = parts[1];  
  }  
};
```

```
console.log(person.fullName); // "Анна Иванова" (вызывается без скобок!)  
person.fullName = 'Мария Петрова'; // Вызывается сеттер  
console.log(person.firstName); // "Мария"
```

1.5. Конструкторы и классы (ES6)

Функции-конструкторы:

```
javascript

function Person(name, age) {
  // this указывает на новый объект
  this.name = name;
  this.age = age;

  // Методы лучше добавлять в прототип
}

// Методы через прототип
Person.prototype.greet = function() {
  return `Привет, я ${this.name}, мне ${this.age} лет`;
};

// Создание экземпляра
const person1 = new Person('Алексей', 30);
console.log(person1.greet()); // "Привет, я Алексей, мне 30 лет"
```

Классы (ES6+):

```
javascript

class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Методы добавляются в прототип
  greet() {
    return `Привет, я ${this.name}, мне ${this.age} лет`;
  }

  // Статические методы
  static compareAges(person1, person2) {
    return person1.age - person2.age;
  }

  // Геттеры и сеттеры
  get birthYear() {
```



```
    return new Date().getFullYear() - this.age;
  }
}

const person2 = new Person('Елена', 25);
console.log(person2.greet()); // "Привет, я Елена, мне 25 лет"
console.log(Person.compareAges(person1, person2)); // 5
console.log(person2.birthYear); // вычисляется динамически
```

Часть 2: Способы вызова функций

2.1. Прямой вызов (Function Call)

Базовый синтаксис:

```
javascript

function greet(name) {
  return `Здравствуйте, ${name}!`;
}

// Прямой вызов с аргументами
const message = greet('Андрей'); // "Здравствуйте, Андрей!"

// Вызов метода объекта
const calculator = {
  add(a, b) {
    return a + b;
  }
};
const sum = calculator.add(5, 3); // 8
```

Особенности прямого вызова:

```
javascript

// 1. Контекст (this) зависит от способа вызова
function showContext() {
  console.log(this);
}

showContext(); // В строгом режиме: undefined, иначе: global/window
```

// 2. Вызов с недостаточным количеством аргументов

```
function sum(a, b, c) {  
  return a + b + c;  
}  
console.log(sum(1, 2)); // NaN (1 + 2 + undefined)
```

// 3. Вызов с избыточным количеством аргументов

```
console.log(sum(1, 2, 3, 4, 5)); // 6 (лишние аргументы игнорируются)
```

2.2. Вызов как метода объекта (Method Invocation)

Особенности контекста:

javascript

```
const user = {  
  name: 'Ольга',  
  age: 28,  
  
  // Метод объекта  
  introduce() {  
    // this ссылается на объект user  
    console.log(`Меня зовут ${this.name}, мне ${this.age} лет`);  
  
    // Вложенная функция теряет контекст  
    function nested() {  
      console.log(this); // undefined (в строгом режиме)  
    }  
    nested();  
  
    // Стрелочная функция сохраняет контекст  
    const arrowNested = () => {  
      console.log(this.name); // "Ольга"  
    };  
    arrowNested();  
  },  
  
  // Метод с параметрами  
  updateAge(newAge) {  
    if (newAge > 0) {  
      this.age = newAge;  
      return `Возраст обновлен: ${this.age}`;  
    }  
  }  
}
```

```

    return 'Неверный возраст';
  }
};

// Вызов методов
user.introduce();
user.updateAge(29); // "Возраст обновлен: 29"

```

Динамическое обращение к методам:

```

javascript

const operations = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
  multiply: (a, b) => a * b,
  divide: (a, b) => a / b
};

// Выбор метода по имени
const operationName = 'multiply';
const result = operations[operationName](6, 7); // 42

```

2.3. Вызов как конструктора (Constructor Invocation)

Синтаксис с new:

```

javascript

function Car(brand, model, year) {
  // new создает новый объект и привязывает его к this
  this.brand = brand;
  this.model = model;
  this.year = year;

  // Методы лучше в прототип, но для примера:
  this.getAge = function() {
    return new Date().getFullYear() - this.year;
  };

  // return не нужен (возвращается this автоматически)
  // Но можно вернуть другой объект, если нужно
}

// Вызов как конструктора
const myCar = new Car('Toyota', 'Camry', 2020);

```

```
console.log(myCar.brand); // "Toyota"
console.log(myCar.getAge()); // вычисляет возраст
```

Что происходит при new:

javascript

// Под капотом new делает следующее:

```
function simulateNew(constructor, ...args) {
  // 1. Создает новый пустой объект
  const obj = {};

  // 2. Устанавливает прототип объекта
  Object.setPrototypeOf(obj, constructor.prototype);

  // 3. Привязывает объект к this и вызывает конструктор
  const result = constructor.apply(obj, args);

  // 4. Возвращает объект (или результат конструктора, если он объект)
  return result instanceof Object ? result : obj;
}
```

// Использование

```
const simulatedCar = simulateNew(Car, 'Honda', 'Accord', 2021);
```

2.4. Непрямой вызов (Indirect Invocation)

Методы call, apply, bind:

1.call() - вызов с указанием this и аргументами по одному:

javascript

```
function introduce(greeting, punctuation) {
  console.log(`${greeting}, меня зовут ${this.name}${punctuation}`);
}

const person = { name: 'Сергей' };

// call принимает this первым аргументом, затем аргументы функции
introduce.call(person, 'Привет', '!'); // "Привет, меня зовут Сергей!"

// Пример с наследованием конструкторов
function Employee(name, position) {
  Person.call(this, name); // Вызываем конструктор родителя
  this.position = position;
}
```

```
}
```

2.apply() - вызов с указанием this и массивом аргументов:

```
javascript
```

```
function sum(a, b, c) {  
  return a + b + c;  
}
```

```
const numbers = [1, 2, 3];
```

```
// apply принимает this и массив аргументов
```

```
const total = sum.apply(null, numbers); // 6
```

```
// Полезно для функций, которые принимают несколько аргументов
```

```
const max = Math.max.apply(null, [10, 5, 8, 20, 3]); // 20
```

```
// В современном JS: Math.max(...[10, 5, 8, 20, 3])
```

3.bind() - создание новой функции с привязанным this:

```
javascript
```

```
const user = {  
  name: 'Дмитрий',  
  score: 100  
};
```

```
function showScore() {  
  console.log(`${this.name}: ${this.score} очков`);  
}
```

```
// bind создает новую функцию с привязанным контекстом
```

```
const boundShowScore = showScore.bind(user);
```

```
boundShowScore(); // "Дмитрий: 100 очков"
```

```
// Частичное применение (partial application)
```

```
function multiply(a, b) {  
  return a * b;  
}
```

```
const double = multiply.bind(null, 2); // Фиксируем первый аргумент
```

```
console.log(double(5)); // 10
```

```
console.log(double(10)); // 20
```

```
const triple = multiply.bind(null, 3); // Фиксируем первый аргумент
```

```
console.log(triple(7)); // 21
```

2.5. Вызов через оператор new (для функций)

Автоматическое создание экземпляров:

```
javascript

function Person(name) {
  // Проверяем, вызвана ли функция с new
  if (!new.target) {
    throw new Error('Person должна вызываться с new');
  }

  this.name = name;
}

// Правильный вызов
const person1 = new Person('Алексей'); // ОК

// Неправильный вызов
// const person2 = Person('Алексей'); // ОШИБКА

// "Защищенный" конструктор
function SafePerson(name) {
  if (!(this instanceof SafePerson)) {
    return new SafePerson(name); // Автоматически добавляем new
  }
  this.name = name;
}

const safe1 = new SafePerson('Мария'); // ОК
const safe2 = SafePerson('Иван'); // Тоже ОК (автоматически добавляется new)
```

Часть 3: Параметры и аргументы функций

3.1. Параметры по умолчанию (ES6+)

Современный синтаксис:

```
javascript

// ES6+ параметры по умолчанию
```

```
function createUser(name, age = 18, isActive = true) {
  return {
    name,
    age,
    isActive,
    createdAt: new Date()
  };
}

console.log(createUser('Анна')); // {name: 'Анна', age: 18, isActive: true, ...}
console.log(createUser('Борис', 25)); // возраст 25
console.log(createUser('Виктор', 30, false)); // все параметры переданы

// Параметры по умолчанию вычисляются при вызове
function generateId() {
  return Math.random().toString(36).substr(2, 9);
}

function createOrder(product, id = generateId()) {
  return { id, product, createdAt: new Date() };
}

console.log(createOrder('Книга')); // Уникальный id каждый раз
```

Старый подход (до ES6):

```
javascript

// ES5 способ установки значений по умолчанию
function oldCreateUser(name, age, isActive) {
  // Проверка на undefined
  age = typeof age !== 'undefined' ? age : 18;
  isActive = isActive !== undefined ? isActive : true;

  // Или через ||
  // isActive = isActive || true; // Проблема: false превратится в true!

  return { name, age, isActive };
}
```

3.2. Rest параметры (ES6+)

Сбор оставшихся аргументов:

```
javascript
```

// Rest параметр собирает все оставшиеся аргументы в массив

```
function sumAll(...numbers) {  
  return numbers.reduce((total, num) => total + num, 0);  
}
```

```
console.log(sumAll(1, 2, 3)); // 6
```

```
console.log(sumAll(1, 2, 3, 4, 5)); // 15
```

```
console.log(sumAll()); // 0
```

// Rest параметр должен быть последним

```
function displayInfo(name, age, ...hobbies) {  
  console.log(`Имя: ${name}`);  
  console.log(`Возраст: ${age}`);  
  console.log(`Хобби: ${hobbies.join(', ') || 'нет'}`);  
}
```

```
displayInfo('Ольга', 25, 'чтение', 'путешествия', 'йога');
```

// Хобби: чтение, путешествия, йога

// Работа с arguments (старый способ)

```
function legacySum() {  
  let total = 0;  
  for (let i = 0; i < arguments.length; i++) {  
    total += arguments[i];  
  }  
  return total;  
}
```

// Преобразование arguments в массив

```
function modernSum() {  
  const numbers = Array.from(arguments);  
  return numbers.reduce((a, b) => a + b, 0);  
}
```

3.3. Деструктуризация параметров

Деструктуризация объектов:

javascript

// Деструктуризация в параметрах

```
function processUser({ name, age, email = 'не указан' }) {  
  console.log(`Пользователь: ${name}, ${age} лет, email: ${email}`);  
}
```



```
const user = { name: 'Алексей', age: 30 };
processUser(user); // "Пользователь: Алексей, 30 лет, email: не указан"
```

// Сложная деструктуризация

```
function renderComponent({
  id,
  type,
  data: { title, content } = {},
  styles: { color = 'black', size = 'medium' } = {}
}) {
  console.log(`Рендерим ${type} компонент с id ${id}`);
  console.log(`Заголовок: ${title}, Цвет: ${color}`);
}
```

```
renderComponent({
  id: 'comp-1',
  type: 'card',
  data: { title: 'Привет', content: 'Текст' },
  styles: { color: 'blue' }
});
```

Деструктуризация массивов:

javascript

// Деструктуризация массива в параметрах

```
function getFirstAndLast([first, , , last]) {
  return { first, last };
}
```

```
const numbers = [1, 2, 3, 4, 5];
console.log(getFirstAndLast(numbers)); // {first: 1, last: 4}
```

// Rest с деструктуризацией

```
function processHeadAndTail([first, second, ...rest]) {
  return {
    head: [first, second],
    tail: rest,
    length: rest.length
  };
}
```

Часть 4: Возврат значений из функций

4.1. Оператор return

Базовое использование:

```
javascript

function add(a, b) {
  return a + b; // Возвращает сумму
}

const result = add(3, 4); // 7

// Функция без return возвращает undefined
function noReturn() {
  // ничего не возвращает
}
console.log(noReturn()); // undefined

// Ранний возврат (early return)
function processUser(user) {
  // Проверка входных данных
  if (!user || typeof user !== 'object') {
    return null; // Ранний выход
  }

  if (!user.name) {
    return { error: 'Имя обязательно' }; // Ранний выход
  }

  // Основная логика
  return {
    id: generateId(),
    ...user,
    createdAt: new Date()
  };
}
```

Множественные return:

```
javascript

function getGrade(score) {
```

```

if (score >= 90) return 'A';
if (score >= 80) return 'B';
if (score >= 70) return 'C';
if (score >= 60) return 'D';
return 'F'; // Значение по умолчанию
}

// Тернарные операторы вместо множественных return
function getGradeTernary(score) {
  return score >= 90 ? 'A'
    : score >= 80 ? 'B'
    : score >= 70 ? 'C'
    : score >= 60 ? 'D'
    : 'F';
}

```

4.2. Возврат нескольких значений

Через массив или объект:

```

javascript

// Возврат через массив
function getMinMax(numbers) {
  const min = Math.min(...numbers);
  const max = Math.max(...numbers);
  return [min, max]; // Возвращаем массив
}

const [minValue, maxValue] = getMinMax([10, 5, 8, 20, 3]);

// Возврат через объект
function calculateStatistics(numbers) {
  const sum = numbers.reduce((a, b) => a + b, 0);
  const avg = sum / numbers.length;
  const min = Math.min(...numbers);
  const max = Math.max(...numbers);

  return { sum, avg, min, max }; // Возвращаем объект
}

const stats = calculateStatistics([1, 2, 3, 4, 5]);
console.log(stats.sum); // 15
console.log(stats.avg); // 3

```

4.3. Возврат функций (замыкания)

Фабрики функций:

```
javascript

// Функция, возвращающая функцию
function createMultiplier(multiplier) {
  return function(number) {
    return number * multiplier;
  };
}
```

```
const double = createMultiplier(2);
const triple = createMultiplier(3);
```

```
console.log(double(5)); // 10
console.log(triple(5)); // 15
```

```
// Каррирование через возврат функций
function curryAdd(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}
```

```
const addFive = curryAdd(5);
const addFiveAndTwo = addFive(2);
console.log(addFiveAndTwo(3)); // 10
```

Приватные данные через замыкания:

```
javascript

function createCounter(initialValue = 0) {
  let count = initialValue; // Приватная переменная

  return {
    increment() {
      count++;
      return count;
    },
    decrement() {
```

```
    count--;
    return count;
  },
  getValue() {
    return count;
  },
  reset() {
    count = initialValue;
    return count;
  }
};
}
```



```
const counter = createCounter(10);
console.log(counter.increment()); // 11
console.log(counter.increment()); // 12
console.log(counter.getValue()); // 12
console.log(counter.reset()); // 10
```

Часть 5: Особые виды функций

5.1. IIFE (Immediately Invoked Function Expression)

Синтаксис и использование:

javascript

// Классический синтаксис

```
(function() {
  console.log('IIFE выполняется немедленно!');
})();
```

// Сохранение результата

```
const result = (function(a, b) {
  return a + b;
})(5, 3); // 8
```

// Асинхронная IIFE

```
(async function() {
  const data = await fetch('/api/data');
  console.log(await data.json());
})();
```

```

})();

// Стрелочная функция как IIFE
(() => {
  console.log('Стрелочная IIFE');
})();

// Модульный паттерн (до ES6 модулей)
const myModule = (function() {
  // Приватные переменные
  let privateVar = 'секретно';

  // Приватная функция
  function privateMethod() {
    return privateVar;
  }

  // Публичный интерфейс
  return {
    publicMethod() {
      return privateMethod();
    },
    config: 'настройки'
  };
})();

console.log(myModule.publicMethod()); // "секретно"
// console.log(myModule.privateVar); // undefined

```

5.2. Рекурсивные функции

Прямая рекурсия:

```

javascript

// Факториал
function factorial(n) {
  if (n <= 1) return 1; // Базовый случай
  return n * factorial(n - 1); // Рекурсивный вызов
}

console.log(factorial(5)); // 120

// Фибоначчи (неэффективно)

```

```

function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

// Фибоначчи с мемоизацией
function memoizedFibonacci() {
  const cache = {};

  return function fib(n) {
    if (n in cache) return cache[n];
    if (n <= 1) return n;

    cache[n] = fib(n - 1) + fib(n - 2);
    return cache[n];
  };
}

const fib = memoizedFibonacci();
console.log(fib(50)); // Быстро, благодаря кэшу

```

5.3. Генераторы (ES6+)

Объявление и использование:

```

javascript

function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
  return 4;
}

const gen = numberGenerator();
console.log(gen.next()); // {value: 1, done: false}
console.log(gen.next()); // {value: 2, done: false}
console.log(gen.next()); // {value: 3, done: false}
console.log(gen.next()); // {value: 4, done: true}

// Генератор для итерации
function* range(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}

```

```
}  
}  
  
for (const num of range(1, 5)) {  
  console.log(num); // 1, 2, 3, 4, 5  
}
```

Часть 6: Лучшие практики и антипаттерны

6.1. Правила именования функций

```
javascript  
  
// ХОРОШО  
function calculateTotalPrice() {}  
function getUserById() {}  
function isValidEmail() {}  
const fetchUserData = () => {};  
  
// ПЛОХО  
function totalPrice() {} // Неясно, что делает  
function get() {} // Слишком общее  
function check() {} // Что проверяет?  
const x = () => {}; // Бессмысленное имя
```

6.2. Длина и сложность функций

```
javascript  
  
// ХОРОШО - небольшая, сфокусированная функция  
function calculateDiscount(price, discountPercent) {  
  return price * (1 - discountPercent / 100);  
}  
  
// ПЛОХО - слишком длинная и делает много всего  
function processOrderBad(order, user, inventory, payment) {  
  // 50+ строк кода...  
  // Проверка заказа, пользователя, инвентаря, оплаты  
  // Логика скидок, налогов, доставки  
  // Отправка email, обновление БД, логирование  
}
```


// ХОРОШО - разбито на маленькие функции

```
function processOrderGood(order, user, inventory, payment) {  
  validateOrder(order);  
  validateUser(user);  
  checkInventory(inventory, order.items);  
  calculateTotals(order);  
  processPayment(payment);  
  updateInventory(inventory, order.items);  
  sendConfirmationEmail(user, order);  
}
```

6.3. Обработка ошибок в функциях

javascript

// ХОРОШО - явная обработка ошибок

```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error('Деление на ноль');  
  }  
  return a / b;  
}
```

// ХОРОШО - возврат результата и ошибки

```
function safeParseJSON(jsonString) {  
  try {  
    return { success: true, data: JSON.parse(jsonString) };  
  } catch (error) {  
    return { success: false, error: error.message };  
  }  
}
```

// ПЛОХО - молчаливое проглатывание ошибок

```
function parseJSONBad(jsonString) {  
  try {  
    return JSON.parse(jsonString);  
  } catch {  
    return null; // Что пошло не так? Неясно  
  }  
}
```

6.4. Чистые функции

javascript

// ЧИСТАЯ функция - предсказуемый результат

```
function add(a, b) {  
  return a + b; // Всегда одинаковый результат для одинаковых входов  
}  
  
// НЕЧИСТАЯ функция - побочные эффекты  
let counter = 0;  
function increment() {  
  counter++; // Изменяет внешнюю переменную  
  return counter; // Разный результат при каждом вызове  
}  
  
// НЕЧИСТАЯ - зависит от внешнего состояния  
function calculateTax(amount) {  
  return amount * taxRate; // Зависит от внешней переменной  
}  
  
// ЧИСТАЯ версия  
function calculateTaxPure(amount, taxRate) {  
  return amount * taxRate; // Результат зависит только от параметров  
}
```

Заключение: ключевые принципы работы с функциями

1. Выбирайте правильный способ объявления:

- ➔ `function declaration` для основных функций
- ➔ `function expression` для присваивания и передачи
- ➔ `arrow functions` для коротких функций и сохранения контекста

2. Понимайте контекст выполнения:

- ➔ `this` определяется способом вызова
- ➔ Используйте `bind/call/apply` для управления контекстом
- ➔ Стрелочные функции не имеют своего `this`

3. Используйте современные возможности ES6+:

- ➔ Параметры по умолчанию
- ➔ Rest параметры
- ➔ Деструктуризация параметров

4. Пишите чистые функции, когда это возможно:

- ➔ Предсказуемое поведение
- ➔ Легче тестировать
- ➔ Лучшая переиспользуемость

5. Документируйте сложные функции:

- ➡ JSDoc комментарии
- ➡ Описание параметров и возвращаемых значений
- ➡ Примеры использования

javascript

```
/**
 * Вычисляет окончательную цену с учетом скидок и налогов
 * @param {number} basePrice - Базовая цена
 * @param {number} discountPercent - Процент скидки (0-100)
 * @param {number} taxPercent - Процент налога (0-100)
 * @returns {number} Итоговая цена
 * @throws {Error} Если проценты вне диапазона
 * @example
 * const finalPrice = calculateFinalPrice(100, 10, 20); // 108
 */
function calculateFinalPrice(basePrice, discountPercent = 0, taxPercent = 0) {
  // Валидация входных данных
  if (discountPercent < 0 || discountPercent > 100) {
    throw new Error('Скидка должна быть от 0 до 100%');
  }

  // Чистая логика вычислений
  const priceAfterDiscount = basePrice * (1 - discountPercent / 100);
  const finalPrice = priceAfterDiscount * (1 + taxPercent / 100);

  // Округление до копеек
  return Math.round(finalPrice * 100) / 100;
}
```

Запомните: Функции — это фундамент JavaScript. Мастерское владение различными способами их объявления и вызова позволит вам писать более чистый, модульный и поддерживаемый код. Практикуйтесь с каждым типом функций, чтобы понимать их сильные и слабые стороны в различных контекстах.

7.2. Параметры и аргументы функции

Введение: Различие между параметрами и аргументами

В JavaScript **параметры** и **аргументы** — это разные, но тесно связанные понятия:

Терминология:

- 🔵 **Параметры (parameters)** — переменные, указанные в объявлении функции
- 🔵 **Аргументы (arguments)** — фактические значения, передаваемые при вызове функции

javascript

// ПАРАМЕТРЫ - a и b в объявлении функции

```
function multiply(a, b) {  
  return a * b;  
}
```

// АРГУМЕНТЫ - 5 и 3 при вызове функции

```
const result = multiply(5, 3); // 15
```

Часть 1: Основы передачи параметров

1.1. Позиционные параметры

Базовый синтаксис:

javascript

// Функция с позиционными параметрами

```
function createUser(name, age, email) {  
  return {  
    name,      // name = первый аргумент  
    age,       // age = второй аргумент  
    email      // email = третий аргумент  
  }  
}
```

```
};  
}
```

// Позиционная передача аргументов

```
const user = createUser('Анна', 25, 'anna@example.com');  
console.log(user); // {name: 'Анна', age: 25, email: 'anna@example.com'}
```

// Проблема: легко перепутать порядок

```
const wrongUser = createUser(25, 'anna@example.com', 'Анна');  
console.log(wrongUser); // {name: 25, age: 'anna@example.com', email: 'Анна'}
```

Советы по работе с позиционными параметрами:

javascript

// 1. Ограничивайте количество параметров (максимум 3-4)

```
function calculateRectangleArea(width, height) { // ✓ Хорошо  
  return width * height;  
}
```

```
function processUserData(name, age, email, phone, address, city, country) { // ✗ Плохо  
  // Слишком много параметров!  
}
```

// 2. Группируйте связанные параметры

```
function processUserDataBad(firstName, lastName, age, street, city, zipCode) { // ✗  
  // ...  
}
```

```
function processUserDataGood(userInfo, addressInfo) { // ✓  
  const { firstName, lastName, age } = userInfo;  
  const { street, city, zipCode } = addressInfo;  
  // ...  
}
```

1.2. Объект arguments

Исторический способ работы с аргументами:

javascript

```
function sumAll() {  
  // arguments - псевдомассив всех переданных аргументов  
  console.log(arguments); // Arguments(4) [1, 2, 3, 4]  
  console.log(arguments.length); // 4  
}
```

```
console.log(typeof arguments); // 'object'
console.log(Array.isArray(arguments)); // false
```

```
let total = 0;
// arguments можно перебирать как массив
for (let i = 0; i < arguments.length; i++) {
  total += arguments[i];
}
return total;
}
```

```
console.log(sumAll(1, 2, 3, 4)); // 10
```

Ограничения и преобразование arguments:

javascript

```
function demonstrateArguments() {
  // arguments не является настоящим массивом
  console.log(arguments.map); // undefined - нет метода map!

  // Преобразование arguments в массив (старые способы)
  const args1 = Array.prototype.slice.call(arguments);
  const args2 = [].slice.call(arguments);

  // Современные способы преобразования
  const args3 = Array.from(arguments);
  const args4 = [...arguments]; // Spread оператор (ES6)

  return args4.map(x => x * 2); // Теперь можно использовать методы массива
}

console.log(demonstrateArguments(1, 2, 3)); // [2, 4, 6]

// arguments в стрелочных функциях
const arrowFunc = () => {
  // console.log(arguments); // ОШИБКА: arguments is not defined
  // Стрелочные функции не имеют своего объекта arguments
};
```

Часть 2: Современные возможности ES6+

2.1. Параметры по умолчанию (Default Parameters)

Базовый синтаксис:

```
javascript

// ES6+ синтаксис параметров по умолчанию
function createProduct(name, price = 100, currency = 'RUB') {
  return {
    name,
    price,
    currency,
    id: generateId()
  };
}

console.log(createProduct('Книга'));
// {name: 'Книга', price: 100, currency: 'RUB', id: '...'}

console.log(createProduct('Ноутбук', 50000));
// {name: 'Ноутбук', price: 50000, currency: 'RUB', id: '...'}

console.log(createProduct('Телефон', 30000, 'USD'));
// {name: 'Телефон', price: 30000, currency: 'USD', id: '...'}
```

Вычисляемые значения по умолчанию:

```
javascript

// Значения по умолчанию вычисляются при КАЖДОМ вызове
function createOrder(product, date = new Date()) {
  return {
    product,
    date, // Разная дата при каждом вызове
    status: 'pending'
  };
}

console.log(createOrder('Книга').date.getTime() ===
  createOrder('Ручка').date.getTime()); // false

// Ссылки на предыдущие параметры
```

```
function createUser(name, nickname = name, id = generateId(name)) {
  return { name, nickname, id };
}
```

```
function generateId(base) {
  return `${base}_${Math.random().toString(36).substr(2, 9)}`;
}
```

```
console.log(createUser('Анна'));
// {name: 'Анна', nickname: 'Анна', id: 'Анна_abc123def'}
```

Сложные случаи и граничные условия:

javascript

// Значение по умолчанию применяется только для undefined

```
function testDefaults(a = 1, b = 2, c = 3) {
  return { a, b, c };
}
```

```
console.log(testDefaults());           // {a: 1, b: 2, c: 3}
console.log(testDefaults(undefined, 5)); // {a: 1, b: 5, c: 3}
console.log(testDefaults(null, 5));    // {a: null, b: 5, c: 3} - null не undefined!
console.log(testDefaults(0, false, '')); // {a: 0, b: false, c: ''} - не undefined!
```

// Функции как значения по умолчанию

```
function required(paramName) {
  throw new Error(`Параметр ${paramName} обязателен`);
}
```

```
function createAccount(username, password = required('password')) {
  return { username, password };
}
```

```
// createAccount('user123'); // ОШИБКА: Параметр password обязателен
console.log(createAccount('user123', 'secret')); // OK
```

2.2. Rest параметры (Rest Parameters)

Синтаксис и использование:

javascript

// Rest параметр собирает все оставшиеся аргументы в массив

```
function sumAll(...numbers) {
```



```

console.log(numbers); // Настоящий массив [1, 2, 3, 4]
console.log(Array.isArray(numbers)); // true

return numbers.reduce((total, num) => total + num, 0);
}

console.log(sumAll(1, 2, 3, 4)); // 10
console.log(sumAll()); // 0 (пустой массив)

// Rest параметр должен быть последним
function displayUser(name, age, ...hobbies) {
  console.log(`Имя: ${name}`);
  console.log(`Возраст: ${age}`);
  console.log(`Хобби: ${hobbies.length ? hobbies.join(', ') : 'нет'}`);
}

displayUser('Алексей', 30, 'чтение', 'спорт', 'путешествия');
// Хобби: чтение, спорт, путешествия

displayUser('Мария', 25);
// Хобби: нет

```

Rest vs arguments:

javascript

```

function compareMethods(...restParams) {
  console.log('Rest параметры:', restParams);
  console.log('Is array?:', Array.isArray(restParams));

  console.log('Arguments:', arguments);
  console.log('Is array?:', Array.isArray(arguments));

  // Rest параметры - настоящий массив
  const doubledRest = restParams.map(x => x * 2);

  // Arguments нужно преобразовывать
  const doubledArgs = Array.from(arguments).map(x => x * 2);

  return { doubledRest, doubledArgs };
}

console.log(compareMethods(1, 2, 3));
// Rest параметры: [1, 2, 3] (массив)
// Arguments: Arguments(3) [1, 2, 3] (псевдомассив)

```

2.3. Spread оператор при вызове функций

Передача массивов как отдельных аргументов:

```
javascript

function calculate(a, b, c) {
  return a * b + c;
}

const numbers = [2, 3, 4];

// Старый способ с apply
const result1 = calculate.apply(null, numbers); // 2*3 + 4 = 10

// Современный способ со spread
const result2 = calculate(...numbers); // 10

// Частичное применение
const numbers2 = [2, 3];
const result3 = calculate(...numbers2, 5); // 2*3 + 5 = 11

// Комбинирование с обычными аргументами
const result4 = calculate(1, ...numbers.slice(1)); // 1*3 + 4 = 7
```

Практические примеры:

```
javascript

// 1. Поиск максимального/минимального значения
const scores = [85, 92, 78, 95, 88];
const maxScore = Math.max(...scores); // 95
const minScore = Math.min(...scores); // 78

// 2. Клонирование и объединение массивов
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

// 3. Работа с DOM элементами
const divs = document.querySelectorAll('div');
const divArray = [...divs]; // Преобразование NodeList в массив

// 4. Динамические вызовы функций
function dynamicCall(func, ...args) {
```

```
    return func(...args);
}

console.log(dynamicCall(Math.max, 1, 2, 3, 4)); // 4
```

Часть 3: Деструктуризация параметров

3.1. Деструктуризация объектов

Базовый синтаксис:

```
javascript

// Деструктуризация в параметрах функции
function processUser({ name, age, email = 'не указан' }) {
  console.log(`Пользователь: ${name}, ${age} лет`);
  console.log(`Email: ${email}`);
  return { name, age, email };
}

const user = { name: 'Иван', age: 30 };
processUser(user);
// Пользователь: Иван, 30 лет
// Email: не указан

processUser({ name: 'Мария', age: 25, email: 'maria@example.com' });
// Пользователь: Мария, 25 лет
// Email: maria@example.com
```

Вложенная и сложная деструктуризация:

```
javascript

function processOrder({
  id,
  customer: {
    name,
    address: { city, street } = {}
  } = {},
  items = [],
  status = 'pending'
```

```

}) {
  console.log(`Заказ #${id} для ${name}`);
  console.log(`Адрес: ${city}, ${street}`);
  console.log(`Количество товаров: ${items.length}`);
  console.log(`Статус: ${status}`);

  return {
    orderId: id,
    customerName: name,
    totalItems: items.length,
    status
  };
}

const order = {
  id: 'ORD-123',
  customer: {
    name: 'Алексей',
    address: {
      city: 'Москва',
      street: 'Ленина, 10'
    }
  },
  items: [
    { id: 1, name: 'Книга', price: 500 },
    { id: 2, name: 'Ручка', price: 50 }
  ]
};

```

```

processOrder(order);
// Заказ #ORD-123 для Алексей
// Адрес: Москва, Ленина, 10
// Количество товаров: 2
// Статус: pending

```

Деструктуризация с переименованием:

```

javascript

function formatDate({
  year: y,
  month: m,
  day: d,
  separator = '-'
}) {

```

```

    return `${y}${separator}${m.toString().padStart(2, '0')}${separator}${
d.toString().padStart(2, '0')}`;
}

console.log(formatDate({ year: 2024, month: 3, day: 15 })); // "2024-03-15"
console.log(formatDate({ year: 2024, month: 3, day: 15, separator: '.' })); //
"2024.03.15"

```

3.2. Деструктуризация массивов

Базовый синтаксис:

```

javascript

// Деструктуризация массива в параметрах
function getCoordinates([x, y, z = 0]) {
    return { x, y, z };
}

const point = [10, 20];
console.log(getCoordinates(point)); // {x: 10, y: 20, z: 0}

function getFirstAndLast([first, ...rest]) {
    const last = rest.length > 0 ? rest[rest.length - 1] : undefined;
    return { first, last };
}

console.log(getFirstAndLast([1, 2, 3, 4, 5])); // {first: 1, last: 5}

```

Продвинутые техники:

```

javascript

// Игнорирование элементов
function processDate([year, month, , day]) { // Пропускаем третий элемент
    return `${day}.${month}.${year}`;
}

console.log(processDate([2024, 3, 15, 10])); // "10.3.2024"

// Деструктуризация вложенных массивов
function analyzeMatrix([[a, b], [c, d]]) {
    const determinant = a * d - b * c;
    const trace = a + d;
    return { determinant, trace };
}

```

```
}
```

```
const matrix = [[1, 2], [3, 4]];
console.log(analyzeMatrix(matrix)); // {determinant: -2, trace: 5}
```

3.3. Гибридная деструктуризация

Комбинирование объектов и массивов:

javascript

```
function processSurveyResults({
  surveyId,
  questions: [firstQuestion, ...otherQuestions],
  participants = 0,
  metadata: { createdBy, createdAt } = {}
}) {
  console.log(`Опрос ${surveyId}:`);
  console.log(`Первый вопрос: ${firstQuestion.text}`);
  console.log(`Всего вопросов: ${otherQuestions.length + 1}`);
  console.log(`Участников: ${participants}`);

  if (createdBy) {
    console.log(`Создатель: ${createdBy}, Дата: ${createdAt}`);
  }

  return {
    surveyId,
    totalQuestions: otherQuestions.length + 1,
    participants
  };
}

const survey = {
  surveyId: 'SURV-001',
  questions: [
    { id: 1, text: 'Как вам наш продукт?' },
    { id: 2, text: 'Порекомендуете ли вы нас?' },
    { id: 3, text: 'Что можно улучшить?' }
  ],
  participants: 150,
  metadata: {
    createdBy: 'Иван Иванов',
  }
}
```

```
    createdAt: '2024-01-15'
  }
};

processSurveyResults(survey);
```

Часть 4: Проверка и валидация аргументов

4.1. Типичные проверки

Базовые проверки:

```
javascript

function safeDivide(dividend, divisor) {
  // Проверка типа
  if (typeof dividend !== 'number' || typeof divisor !== 'number') {
    throw new TypeError('Оба аргумента должны быть числами');
  }

  // Проверка на NaN
  if (isNaN(dividend) || isNaN(divisor)) {
    throw new Error('Аргументы не должны быть NaN');
  }

  // Проверка на Infinity
  if (!isFinite(dividend) || !isFinite(divisor)) {
    throw new Error('Аргументы должны быть конечными числами');
  }

  // Бизнес-логика проверки
  if (divisor === 0) {
    throw new Error('Деление на ноль невозможно');
  }

  return dividend / divisor;
}

try {
  console.log(safeDivide(10, 2)); // 5
}
```

```
console.log(safeDivide(10, 0)); // ОШИБКА
console.log(safeDivide(10, '2')); // ОШИБКА (TypeError)
} catch (error) {
  console.error('Ошибка:', error.message);
}
```

Валидация объектов:

javascript

```
function createUser(userData) {
  // Проверка на наличие объекта
  if (!userData || typeof userData !== 'object') {
    throw new TypeError('userData должен быть объектом');
  }

  const { name, age, email } = userData;

  // Проверка обязательных полей
  if (!name || typeof name !== 'string') {
    throw new Error('Имя обязательно и должно быть строкой');
  }

  if (age === undefined || typeof age !== 'number') {
    throw new Error('Возраст обязателен и должен быть числом');
  }

  if (age < 0 || age > 150) {
    throw new RangeError('Возраст должен быть от 0 до 150');
  }

  // Проверка опциональных полей
  if (email && typeof email !== 'string') {
    throw new TypeError('Email должен быть строкой');
  }

  if (email && !email.includes('@')) {
    throw new Error('Некорректный email');
  }

  // Все проверки пройдены
  return {
    id: generateId(),
    ...userData,
    createdAt: new Date()
  };
}
```



```
};  
}
```

4.2. Контрактное программирование

Предусловия (preconditions):

```
javascript  
  
/**  
 * Вычисляет скидку на основе суммы покупки  
 * @param {number} amount - Сумма покупки (должна быть >= 0)  
 * @returns {number} Процент скидки  
 */  
function calculateDiscount(amount) {  
  // Предусловие  
  const precondition = require('precondition');  
  precondition.argument(amount, 'amount')  
    .isNumber()  
    .isGreaterThanOrEqual(0);  
  
  // Основная логика  
  if (amount >= 10000) return 20;  
  if (amount >= 5000) return 15;  
  if (amount >= 1000) return 10;  
  return 5;  
}  
  
// Альтернатива с кастомными проверками  
function calculateDiscountWithChecks(amount) {  
  // Проверка предусловий  
  if (typeof amount !== 'number' || isNaN(amount)) {  
    throw new TypeError('amount должен быть числом');  
  }  
  
  if (amount < 0) {  
    throw new RangeError('amount не может быть отрицательным');  
  }  
  
  // Постусловия (postconditions)  
  const discount = calculateDiscountLogic(amount);  
  
  // Проверяем постусловия
```

```

if (discount < 0 || discount > 100) {
  throw new Error('Скидка должна быть от 0 до 100%');
}

return discount;
}

```

```

function calculateDiscountLogic(amount) {
  // Основная логика
  if (amount >= 10000) return 20;
  if (amount >= 5000) return 15;
  if (amount >= 1000) return 10;
  return 5;
}

```

4.3. Инварианты (invariants)

javascript

```

class BankAccount {
  constructor(initialBalance = 0) {
    // Инвариант: баланс не может быть отрицательным
    if (initialBalance < 0) {
      throw new Error('Начальный баланс не может быть отрицательным');
    }

    // Приватное поле для баланса
    let balance = initialBalance;

    // Методы для работы с балансом
    this.deposit = (amount) => {
      // Предусловие
      if (amount <= 0) {
        throw new Error('Сумма депозита должна быть положительной');
      }

      const oldBalance = balance;
      balance += amount;

      // Постусловие и проверка инварианта
      if (balance < 0) {
        balance = oldBalance; // Откат
        throw new Error('Нарушен инвариант: баланс стал отрицательным');
      }
    }
  }
}

```

```
    return balance;
};

this.withdraw = (amount) => {
    // Предусловие
    if (amount <= 0) {
        throw new Error('Сумма снятия должна быть положительной');
    }

    if (amount > balance) {
        throw new Error('Недостаточно средств');
    }

    const oldBalance = balance;
    balance -= amount;

    // Проверка инварианта
    if (balance < 0) {
        balance = oldBalance; // Откат
        throw new Error('Нарушен инвариант: баланс стал отрицательным');
    }

    return balance;
};

this.getBalance = () => balance;
}
}
```

Часть 5: Продвинутые техники работы с параметрами

5.1. Каррирование (Currying)

Базовое каррирование:

javascript

```
// Каррирование - преобразование функции с несколькими аргументами
// в последовательность функций с одним аргументом
```

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    } else {
      return function(...args2) {
        return curried.apply(this, args.concat(args2));
      };
    }
  };
}
```

// Пример использования

```
function multiplyThree(a, b, c) {
  return a * b * c;
}
```

```
const curriedMultiply = curry(multiplyThree);
```

```
console.log(curriedMultiply(2)(3)(4)); // 24
console.log(curriedMultiply(2, 3)(4)); // 24
console.log(curriedMultiply(2)(3, 4)); // 24
console.log(curriedMultiply(2, 3, 4)); // 24
```

// Частичное применение

```
const double = curriedMultiply(2); // Фиксируем первый аргумент
const doubleAndTriple = double(3); // Фиксируем второй аргумент
console.log(doubleAndTriple(4)); // 24 (2 * 3 * 4)
console.log(doubleAndTriple(5)); // 30 (2 * 3 * 5)
```

Практическое применение каррирования:

javascript

// Логирование с каррированием

```
function log(level, message, timestamp) {
  console.log(`[${level}] ${timestamp.toISOString(): ${message}}`);
}
```

```
const curriedLog = curry(log);
```

// Создаем специализированные версии

```
const logInfo = curriedLog('INFO');
const logError = curriedLog('ERROR');
```

```
// Фиксируем timestamp
const logToday = (level, message) =>
  curriedLog(level, message)(new Date());

// Использование
logInfo('Приложение запущено')(new Date());
logError('Ошибка соединения')(new Date());
logToday('WARN', 'Низкая память');
```

5.2. Частичное применение (Partial Application)

javascript

```
// Частичное применение - фиксация некоторых аргументов
function partial(fn, ...fixedArgs) {
  return function(...remainingArgs) {
    return fn.apply(this, fixedArgs.concat(remainingArgs));
  };
}

function greet(greeting, name, punctuation) {
  return `${greeting}, ${name}${punctuation}`;
}

// Фиксируем первый аргумент
const sayHello = partial(greet, 'Привет');
console.log(sayHello('Анна', '!')); // "Привет, Анна!"

// Фиксируем несколько аргументов
const sayHelloToJohn = partial(greet, 'Привет', 'Джон');
console.log(sayHelloToJohn('!!!')); // "Привет, Джон!!!"

// Современная альтернатива с bind
const sayHelloModern = greet.bind(null, 'Привет');
console.log(sayHelloModern('Петр', '?')); // "Привет, Петр?"
```

5.3. Функции высшего порядка с параметрами

javascript

```
// Функция, возвращающая функцию с настроенными параметрами
function createMultiplier(multiplier) {
  return function(number) {
    return number * multiplier;
  };
}
```

```

};
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15

// Более сложный пример с конфигурацией
function createValidator(rules) {
  return function(data) {
    const errors = [];

    for (const [field, rule] of Object.entries(rules)) {
      const value = data[field];

      if (rule.required && (value === undefined || value === null || value === '')) {
        errors.push(`${field}: обязательно для заполнения`);
      }

      if (rule.type && typeof value !== rule.type) {
        errors.push(`${field}: должен быть типа ${rule.type}`);
      }

      if (rule.min !== undefined && value < rule.min) {
        errors.push(`${field}: минимум ${rule.min}`);
      }

      if (rule.max !== undefined && value > rule.max) {
        errors.push(`${field}: максимум ${rule.max}`);
      }
    }

    return errors;
  };
}

// Создаем валидатор с конкретными правилами
const validateUser = createValidator({
  name: { required: true, type: 'string', minLength: 2 },
  age: { required: true, type: 'number', min: 0, max: 150 },
  email: { required: false, type: 'string' }
});

```

```
const errors = validateUser({ name: 'A', age: 200 });
console.log(errors);
// ['name: минимум 2 символа', 'age: максимум 150']
```

Часть 6: Оптимизация и лучшие практики

6.1. Оптимизация производительности

Избегайте лишних проверок:

```
javascript

// НЕОПТИМАЛЬНО - проверки на каждом вызове
function processDataSlow(data) {
  // Эти проверки выполняются каждый раз
  if (!data) throw new Error('Нет данных');
  if (!Array.isArray(data)) throw new Error('Данные должны быть массивом');
  if (data.length === 0) return 0;

  // Основная логика
  return data.reduce((a, b) => a + b, 0);
}

// ОПТИМАЛЬНО - проверки вынесены в отдельную функцию
function validateData(data) {
  if (!data) throw new Error('Нет данных');
  if (!Array.isArray(data)) throw new Error('Данные должны быть массивом');
  return data;
}

function processDataFast(data) {
  const validatedData = validateData(data);
  if (validatedData.length === 0) return 0;

  // Основная логика (без проверок)
  return validatedData.reduce((a, b) => a + b, 0);
}

// Еще лучше - использовать деструктуризацию с значениями по умолчанию
```

```
function processDataBest(data = []) {  
  // data всегда будет массивом (даже если передали undefined)  
  return data.reduce((a, b) => a + b, 0);  
}
```

Мемоизация дорогих вычислений:

javascript

```
function memoize(fn) {  
  const cache = new Map();  
  
  return function(...args) {  
    const key = JSON.stringify(args);  
  
    if (cache.has(key)) {  
      console.log('Возвращаем из кэша');  
      return cache.get(key);  
    }  
  
    console.log('Вычисляем результат');  
    const result = fn(...args);  
    cache.set(key, result);  
    return result;  
  };  
}  
  
// Тяжелая функция  
function expensiveCalculation(a, b, c) {  
  // Имитация тяжелых вычислений  
  let result = 0;  
  for (let i = 0; i < 1000000; i++) {  
    result += Math.sin(a) * Math.cos(b) * Math.tan(c);  
  }  
  return result;  
}  
  
const memoizedCalculation = memoize(expensiveCalculation);  
  
console.log(memoizedCalculation(1, 2, 3)); // "Вычисляем результат" (медленно)  
console.log(memoizedCalculation(1, 2, 3)); // "Возвращаем из кэша" (быстро)  
console.log(memoizedCalculation(1, 2, 4)); // "Вычисляем результат" (новые аргументы)
```


6.2. Документирование параметров

JSDoc для параметров:

```
javascript

/**
 * Рассчитывает стоимость доставки
 * @param {Object} options - Опции доставки
 * @param {number} options.weight - Вес посылки в кг (от 0.1 до 100)
 * @param {number} options.distance - Расстояние в км (от 1 до 10000)
 * @param {string} [options.service='standard'] - Тип услуги: 'standard', 'express',
'overnight'
 * @param {boolean} [options.insurance=false] - Страховка
 * @returns {number} Стоимость доставки в рублях
 * @throws {TypeError} Если параметры неверного типа
 * @throws {RangeError} Если значения вне диапазона
 * @example
 * const cost = calculateShipping({
 *   weight: 5,
 *   distance: 100,
 *   service: 'express'
 * });
 */
function calculateShipping({
  weight,
  distance,
  service = 'standard',
  insurance = false
}) {
  // Проверка типов
  if (typeof weight !== 'number' || typeof distance !== 'number') {
    throw new TypeError('weight и distance должны быть числами');
  }

  // Проверка диапазонов
  if (weight < 0.1 || weight > 100) {
    throw new RangeError('weight должен быть от 0.1 до 100 кг');
  }

  if (distance < 1 || distance > 10000) {
    throw new RangeError('distance должен быть от 1 до 10000 км');
  }
}
```

```
// Основная логика расчета
let cost = weight * 50 + distance * 10;

if (service === 'express') cost *= 1.5;
if (service === 'overnight') cost *= 2;
if (insurance) cost += cost * 0.1;

return Math.round(cost);
}
```

6.3. Антипаттерны и как их избежать

1. Слишком много параметров:

```
javascript

// АНТИПАТТЕРН
function createOrderBad(
  customerId,
  productId,
  quantity,
  unitPrice,
  discountPercent,
  taxRate,
  shippingMethod,
  shippingAddress,
  billingAddress,
  paymentMethod,
  notes
) {
  // ...
}

// РЕШЕНИЕ: объект параметров
function createOrderGood(orderData) {
  const {
    customerId,
    productId,
    quantity,
    unitPrice,
    discountPercent = 0,
    taxRate = 20,
    shippingMethod = 'standard',
  } = orderData;
}
```

```

    shippingAddress,
    billingAddress = shippingAddress,
    paymentMethod,
    notes = ''
  } = orderData;
  // ...
}

```

2. Мутация параметров:

javascript

// АНТИПАТТЕРН

```

function processItemsBad(items) {
  items.sort(); // Мутирует исходный массив!
  return items.map(item => item * 2);
}

```

```

const originalArray = [3, 1, 2];
const processed = processItemsBad(originalArray);
console.log(originalArray); // [1, 2, 3] - ИЗМЕНИЛСЯ!

```

// РЕШЕНИЕ: не мутировать параметры

```

function processItemsGood(items) {
  const sorted = [...items].sort(); // Копируем массив
  return sorted.map(item => item * 2);
}

```

```

const originalArray2 = [3, 1, 2];
const processed2 = processItemsGood(originalArray2);
console.log(originalArray2); // [3, 1, 2] - НЕ ИЗМЕНИЛСЯ

```

3. Магические числа и строки:

javascript

// АНТИПАТТЕРН

```

function calculatePrice(amount, type) {
  if (type === 'A') { // Что такое 'A'?
    return amount * 1.1;
  } else if (type === 'B') { // Что такое 'B'?
    return amount * 1.2;
  }
  return amount;
}

```

// РЕШЕНИЕ: именованные константы

```
const PRODUCT_TYPES = {  
  STANDARD: 'STANDARD',  
  PREMIUM: 'PREMIUM',  
  LUXURY: 'LUXURY'  
};  
  
const DISCOUNT_RATES = {  
  [PRODUCT_TYPES.STANDARD]: 0.1,  
  [PRODUCT_TYPES.PREMIUM]: 0.2,  
  [PRODUCT_TYPES.LUXURY]: 0.3  
};  
  
function calculatePriceGood(amount, productType = PRODUCT_TYPES.STANDARD) {  
  const discountRate = DISCOUNT_RATES[productType] || 0;  
  return amount * (1 - discountRate);  
}
```

Заключение: мастерство работы с параметрами

Ключевые принципы:

1. Используйте современный синтаксис ES6+:

- ➔ Параметры по умолчанию для опциональных значений
- ➔ Rest параметры для работы с переменным числом аргументов
- ➔ Деструктуризация для сложных объектов

2. Валидируйте входные данные:

- ➔ Проверяйте типы и диапазоны
- ➔ Используйте осмысленные сообщения об ошибках
- ➔ Не доверяйте пользовательскому вводу

3. Документируйте ожидания:

- ➔ JSDoc для описания параметров
- ➔ Примеры использования
- ➔ Описание возможных ошибок

4. Оптимизируйте для читаемости:

- ➔ Ограничивайте количество параметров (правило 3-4)
- ➔ Группируйте связанные параметры в объекты
- ➔ Используйте осмысленные имена

5. Учитывайте производительность:

- ➡ Избегайте лишних проверок в горячих путях
- ➡ Мемоизируйте дорогие вычисления
- ➡ Используйте значения по умолчанию вместо проверок на undefined

Философия хорошего API функций:

```
javascript

// ПЛОХОЕ API
function f(x, y, z, a, b, c, d, e) {
  // ...
}

// ХОРОШЕЕ API
function processUserData({
  // Обязательные параметры
  userId,
  action,

  // Опциональные с значениями по умолчанию
  options = {},

  // Колбэки для расширения
  onSuccess = () => {},
  onError = (err) => { console.error(err); },

  // Флаги и настройки
  retry = false,
  timeout = 5000
}) {
  // Валидация
  if (!userId) throw new Error('userId обязателен');
  if (!['create', 'update', 'delete'].includes(action)) {
    throw new Error('Некорректное действие');
  }

  // Основная логика с четкой структурой
  try {
    // ...
    onSuccess(result);
  } catch (error) {
    onError(error);
  }
}
```

```
}  
}
```

```
// Использование
```

```
processUserData({  
  userId: '123',  
  action: 'update',  
  options: { notify: true },  
  onSuccess: (data) => console.log('Успех:', data),  
  timeout: 10000  
});
```

Итог: Параметры и аргументы — это интерфейс ваших функций. Хорошо спроектированный API делает код более понятным, надежным и легким в использовании. Инвестируйте время в правильное проектирование параметров — это окупится при поддержке и расширении кода.

7.3. Возврат значений и return

Введение: Фундаментальная роль return в JavaScript

Оператор `return` — один из самых важных элементов в JavaScript, который определяет, **какое значение функция возвращает** вызывающему коду. Понимание всех аспектов `return` критически важно для написания чистого, предсказуемого и эффективного кода.

Ключевые аспекты оператора return:

- Определяет выходное значение функции
 - Немедленно прекращает выполнение функции
 - Может возвращать любые типы данных
 - Необязателен, но влияет на результат функции
-

Часть 1: Основы оператора return

1.1. Синтаксис и базовое использование

javascript

// Самый простой вариант - возврат значения

```
function add(a, b) {  
  return a + b; // Возвращает сумму  
}
```

```
const sum = add(5, 3); // sum = 8
```

// Возврат вычисляемого выражения

```
function calculateTax(price, taxRate) {  
  return price * (1 + taxRate / 100); // Возвращает результат выражения  
}
```

// Возврат литералов

```
function createDefaultUser() {
```

```

return {           // Возврат объекта
  id: 1,
  name: 'Гость',
  role: 'user'
};
}

function getStatus() {
  return 'active'; // Возврат строки
}

function isAdult(age) {
  return age >= 18; // Возврат boolean
}

```

1.2. Неявный возврат undefined

```

javascript

// Функция без return возвращает undefined
function noReturnFunction() {
  const x = 10;
  // Нет оператора return
}

const result = noReturnFunction();
console.log(result); // undefined
console.log(typeof result); // 'undefined'

// Пустой return также возвращает undefined
function emptyReturn() {
  return; // Возвращает undefined
}

console.log(emptyReturn()); // undefined

// Компилятор JavaScript преобразует это в:
function emptyReturnTransformed() {
  return undefined; // Неявно добавляется undefined
}

```

1.3. Стрелочные функции и неявный return

```

javascript

```



```
// Явный return (с фигурными скобками)
const addExplicit = (a, b) => {
  return a + b;
};

// неявный return (без фигурных скобок)
const addImplicit = (a, b) => a + b;

// То же самое для одного выражения
const squareExplicit = x => {
  return x * x;
};

const squareImplicit = x => x * x;

// неявный return объектов (нужны скобки)
const createUserBad = () => { name: 'John', age: 30 }; // ОШИБКА!
const createUserGood = () => ({ name: 'John', age: 30 }); // Правильно

// Многострочные выражения с неявным return
const processData = data =>
  data
    .filter(item => item.active)
    .map(item => item.value * 2)
    .reduce((sum, val) => sum + val, 0);
```

Часть 2: Возврат различных типов данных

2.1. Возврат примитивных типов

javascript

```
// Возврат числа
function getRandomNumber(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

// Возврат строки
function generateGreeting(name) {
  return `Привет, ${name}! Добро пожаловать в наше приложение.`;
}
```

```

}

// Возврат boolean
function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}

// Возврат null (осознанное отсутствие значения)
function findUserById(id) {
  const user = database.find(u => u.id === id);
  return user || null; // Явно возвращаем null если пользователь не найден
}

// Возврат undefined
function clearCache() {
  cache = {};
  // Неявный return undefined
}

// Возврат Symbol
function createUniqueKey() {
  return Symbol('unique_key');
}

// Возврат BigInt
function calculateLargeNumber() {
  return 1234567890123456789012345678901234567890n;
}

```

2.2. Возврат объектов и массивов

javascript

```

// Возврат простого объекта
function createUserObject(name, age) {
  return {
    name,          // Сокращенная запись для name: name
    age,           // Сокращенная запись для age: age
    id: generateId(),
    createdAt: new Date()
  };
}

```

```
// Возврат с вычисляемыми свойствами
function createConfig(mode) {
  return {
    apiUrl: mode === 'production'
      ? 'https://api.prod.com'
      : 'https://api.dev.com',
    debug: mode !== 'production',
    [mode + 'Mode']: true // Вычисляемое имя свойства
  };
}

// Возврат массива
function splitName(fullName) {
  const parts = fullName.split(' ');
  return [
    parts[0] || '', // Имя
    parts.slice(1).join(' ') // Фамилия (все остальные части)
  ];
}

const [firstName, lastName] = splitName('Иван Иванов Петрович');

// Возврат вложенных структур
function getReportData() {
  return {
    summary: {
      total: 1500,
      average: 75,
      min: 10,
      max: 100
    },
    details: [
      { category: 'A', value: 500 },
      { category: 'B', value: 750 },
      { category: 'C', value: 250 }
    ],
    metadata: {
      generated: new Date(),
      version: '1.0.0'
    }
  };
}
```

2.3. Возврат функций (замыкания)

javascript

// Фабрика функций

```
function createMultiplier(factor) {  
  return function(number) {  
    return number * factor;  
  };  
}
```

```
const double = createMultiplier(2);  
const triple = createMultiplier(3);
```

```
console.log(double(10)); // 20  
console.log(triple(10)); // 30
```

// Каррирование через возврат функций

```
function curryAdd(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    };  
  };  
}
```

```
const addFive = curryAdd(5);  
const addFiveAndTwo = addFive(2);  
console.log(addFiveAndTwo(3)); // 10
```

// Создание специализированных функций

```
function createLogger(prefix) {  
  return function(...args) {  
    console.log(`[${prefix}]`, ...args);  
  };  
}
```

```
const infoLog = createLogger('INFO');  
const errorLog = createLogger('ERROR');
```

```
infoLog('Приложение запущено'); // [INFO] Приложение запущено  
errorLog('Ошибка соединения'); // [ERROR] Ошибка соединения
```

Часть 3: Продвинутые техники return

3.1. Ранний возврат (Early Return)

javascript

// ПЛОХО: Вложенные условия

```
function processOrderBad(order) {  
  if (order) {  
    if (order.items && order.items.length > 0) {  
      if (order.customer && order.customer.active) {  
        // Основная логика (глубоко вложена)  
        return calculateTotal(order);  
      } else {  
        return { error: 'Неактивный клиент' };  
      }  
    } else {  
      return { error: 'Пустой заказ' };  
    }  
  } else {  
    return { error: 'Заказ не существует' };  
  }  
}
```

// ХОРОШО: Ранний возврат (Guard Clauses)

```
function processOrderGood(order) {  
  // Проверки в начале функции  
  if (!order) {  
    return { error: 'Заказ не существует' };  
  }  
  
  if (!order.items || order.items.length === 0) {  
    return { error: 'Пустой заказ' };  
  }  
  
  if (!order.customer || !order.customer.active) {  
    return { error: 'Неактивный клиент' };  
  }  
  
  // Основная логика (без вложенности)  
  return calculateTotal(order);  
}
```

```
// Преимущества раннего возврата:  
// 1. Уменьшает вложенность кода  
// 2. Улучшает читаемость  
// 3. Делает условия выхода явными  
// 4. Легче рефакторить и тестировать
```

3.2. Множественные точки возврата

javascript

```
// Управление потоком с множественными return  
function getUserPermissions(user, action) {  
  // Проверка существования пользователя  
  if (!user || !user.id) {  
    return { allowed: false, reason: 'Пользователь не существует' };  
  }  
  
  // Проверка блокировки  
  if (user.isBlocked) {  
    return { allowed: false, reason: 'Пользователь заблокирован' };  
  }  
  
  // Проверка роли  
  if (user.role === 'admin') {  
    return { allowed: true, level: 'full' };  
  }  
  
  // Проверка конкретного действия  
  const actionPermissions = {  
    'read': ['user', 'moderator', 'admin'],  
    'write': ['moderator', 'admin'],  
    'delete': ['admin']  
  };  
  
  const allowedRoles = actionPermissions[action];  
  if (!allowedRoles) {  
    return { allowed: false, reason: 'Неизвестное действие' };  
  }  
  
  if (allowedRoles.includes(user.role)) {  
    return { allowed: true, level: user.role };  
  }  
  
  // Дефолтный случай
```

```

    return { allowed: false, reason: 'Недостаточно прав' };
}

// Пример использования
const user = { id: 1, role: 'user', isBlocked: false };
console.log(getUserPermissions(user, 'read')); // { allowed: true, level: 'user' }
console.log(getUserPermissions(user, 'delete')); // { allowed: false, reason:
'Недостаточно прав' }

```

3.3. Возврат Promises

javascript

```

// Асинхронные функции всегда возвращают Promise
async function fetchUserData(userId) {
  try {
    const response = await fetch(`/api/users/${userId}`);
    if (!response.ok) {
      // Возврат отклоненного Promise
      throw new Error(`HTTP error: ${response.status}`);
    }

    const data = await response.json();
    // Возврат значения оборачивается в Promise
    return data; // Эквивалентно Promise.resolve(data)
  } catch (error) {
    // Возврат ошибки также оборачивается в Promise
    return { error: error.message, userId }; // Promise.resolve с объектом ошибки
  }
}

// Использование
fetchUserData(123)
  .then(data => console.log('Успех:', data))
  .catch(error => console.error('Ошибка:', error));

// Явное создание Promise
function delay(ms) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(`Ждали ${ms}ms`); // Вызов resolve для успешного завершения
    }, ms);
  });
}

```

```

// Возврат Promise из обычной функции
function loadConfig() {
  return fetch('/api/config')
    .then(response => response.json())
    .then(config => {
      // Возврат значения через цепочку then
      return processConfig(config);
    })
    .catch(error => {
      // Возврат значения при ошибке
      return getDefaultConfig();
    });
}

```

3.4. Генераторы и yield

javascript

```

// Генераторы используют yield вместо return
function* numberGenerator() {
  yield 1; // Приостанавливает выполнение, возвращает 1
  yield 2; // Приостанавливает выполнение, возвращает 2
  yield 3; // Приостанавливает выполнение, возвращает 3
  return 4; // Завершает генератор, возвращает 4
}

```

```

const gen = numberGenerator();
console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: 4, done: true }

```

```

// Практический пример: генерация последовательностей
function* fibonacci(limit) {
  let a = 0, b = 1;
  let count = 0;

  while (count < limit) {
    yield a;
    [a, b] = [b, a + b];
    count++;
  }
}

```



```
    return `Сгенерировано ${limit} чисел Фибоначчи`;
}

const fibGen = fibonacci(5);
for (const num of fibGen) {
    console.log(num); // 0, 1, 1, 2, 3
}
```

Часть 4: Возврат в различных контекстах

4.1. Конструкторы и return

javascript

// Конструкторы обычно не используют return явно

```
function Person(name, age) {
    this.name = name;
    this.age = age;
    // неявно возвращает this
}
```

```
const person1 = new Person('Анна', 25);
console.log(person1); // Person { name: 'Анна', age: 25 }
```

// Явный return в конструкторах

```
function SpecialPerson(name, age) {
    this.name = name;
    this.age = age;

    // Если возвращается объект, он заменяет this
    return { custom: true, name: 'Переопределено' };
}
```

```
const person2 = new SpecialPerson('Борис', 30);
console.log(person2); // { custom: true, name: 'Переопределено' }
```

// Return примитива игнорируется

```
function PrimitivePerson(name) {
    this.name = name;
    return 42; // Игнорируется
}
```

```
}
```

```
const person3 = new PrimitivePerson('Виктор');  
console.log(person3); // PrimitivePerson { name: 'Виктор' }
```

4.2. Стрелочные функции как конструкторы

javascript

// Стрелочные функции не могут быть конструкторами

```
const ArrowConstructor = () => {  
  this.name = 'test'; // ОШИБКА: this не определен  
};
```

// new ArrowConstructor(); // TypeError: ArrowConstructor is not a constructor

// Но можно возвращать объекты из стрелочных функций

```
const createPerson = (name, age) => ({  
  name,  
  age,  
  greet() {  
    return `Привет, я ${this.name}`;  
  }  
});  
  
const person = createPerson('Мария', 28);  
console.log(person.greet()); // "Привет, я Мария"
```

4.3. Return в методах объектов и классов

javascript

// Методы объектов

```
const calculator = {  
  add(a, b) {  
    return a + b;  
  },
```

```
  subtract(a, b) {  
    return a - b;  
  },
```

// Методы могут возвращать this для чейнинга

```
  setValue(x) {
```

```

    this.value = x;
    return this; // Позволяет цепочку вызовов
},

multiplyBy(y) {
    this.value *= y;
    return this;
},

getValue() {
    return this.value;
}
};

const result = calculator.setValue(10).multiplyBy(3).getValue();
console.log(result); // 30

// Методы классов
class User {
    constructor(name, email) {
        this.name = name;
        this.email = email;
    }

    // Геттер (вызывается как свойство)
    get displayName() {
        return `${this.name} <${this.email}>`;
    }

    // Сеттер (вызывается при присваивании)
    set displayName(value) {
        const [name, email] = value.split(' <');
        this.name = name;
        this.email = email ? email.slice(0, -1) : '';
    }

    // Обычный метод
    updateProfile(data) {
        Object.assign(this, data);
        return this; // Возвращает экземпляр для чейнинга
    }
}

const user = new User('Иван', 'ivan@mail.com');
```

```
console.log(user.displayName); // "Иван <ivan@mail.com>"
user.displayName = 'Петр <petr@mail.com>';
console.log(user.name); // "Петр"
```

Часть 5: Обработка ошибок и возврат

5.1. Возврат ошибок через throw

```
javascript

// Использование throw для ошибок
function divide(a, b) {
  if (b === 0) {
    throw new Error('Деление на ноль невозможно');
  }

  if (typeof a !== 'number' || typeof b !== 'number') {
    throw new TypeError('Оба аргумента должны быть числами');
  }

  return a / b;
}

// Обработка с try-catch
try {
  const result = divide(10, 2);
  console.log('Результат:', result);

  const errorResult = divide(10, 0); // Выбросит ошибку
} catch (error) {
  console.error('Произошла ошибка:', error.message);
  console.error('Тип ошибки:', error.constructor.name);

  if (error instanceof TypeError) {
    console.error('Ошибка типа данных');
  } else if (error instanceof Error) {
    console.error('Общая ошибка');
  }
}
```

5.2. Возврат объектов с ошибками

javascript

// Альтернатива throw - возврат объекта с ошибкой

```
function safeDivide(a, b) {
  if (b === 0) {
    return {
      success: false,
      error: 'Деление на ноль невозможно',
      code: 'DIVISION_BY_ZERO'
    };
  }

  if (typeof a !== 'number' || typeof b !== 'number') {
    return {
      success: false,
      error: 'Оба аргумента должны быть числами',
      code: 'INVALID_TYPE'
    };
  }

  return {
    success: true,
    value: a / b,
    operation: 'division'
  };
}
```

// Использование

```
const result = safeDivide(10, 0);

if (result.success) {
  console.log('Результат:', result.value);
} else {
  console.error('Ошибка:', result.error);
  console.error('Код ошибки:', result.code);
}
```

// Паттерн Result/Option

```
function parseNumber(str) {
  const num = Number(str);

  if (isNaN(num)) {
```

```
return {
  ok: false,
  error: `Не удалось преобразовать "${str}" в число`;
};
}
```

```
return {
  ok: true,
  value: num
};
}
```

```
const parsed = parseNumber('123');
if (parsed.ok) {
  console.log('Число:', parsed.value);
} else {
  console.error('Ошибка парсинга:', parsed.error);
}
```

5.3. try-catch-finally и return

javascript

// Поведение return в блоках try-catch-finally

```
function complexOperation() {
  try {
    console.log('Начало try');

    // Если здесь будет ошибка, выполнение перейдет в catch
    const result = riskyOperation();

    console.log('Возвращаем результат из try');
    return result; // Возвращает значение, НО сначала выполняется finally

  } catch (error) {
    console.log('Блок catch:', error.message);
    return 'значение из catch'; // Также сначала выполняется finally

  } finally {
    console.log('Блок finally выполняется ВСЕГДА');
    // Код здесь выполнится перед любым return
  }

  console.log('Этот код никогда не выполнится');
```

```
}
```

```
// Пример с изменением возвращаемого значения в finally
```

```
function testFinallyReturn() {  
  try {  
    return 'из try';  
  } finally {  
    return 'из finally'; // ПЕРЕОПРЕДЕЛЯЕТ возвращаемое значение!  
  }  
}
```

```
console.log(testFinallyReturn()); // "из finally"
```

```
// Но если в finally выбросить ошибку
```

```
function testFinallyError() {  
  try {  
    return 'из try';  
  } finally {  
    throw new Error('ошибка в finally'); // Эта ошибка переопределит возвращаемое значение  
  }  
}
```

```
try {  
  console.log(testFinallyError());  
} catch (error) {  
  console.error('Поймана ошибка:', error.message); // "ошибка в finally"  
}
```

Часть 6: Оптимизация и лучшие практики

6.1. Избегание лишних вычислений

```
javascript
```

```
// НЕОПТИМАЛЬНО
```

```
function calculateExpensiveValue() {  
  const result = performHeavyCalculation(); // Вычисляется всегда  
  
  if (condition) {  
    return result;  
  }  
}
```

```

    } else {
        return defaultValue;
    }
}

// ОПТИМАЛЬНО: Ленивая оценка
function calculateOptimized() {
    if (condition) {
        return performHeavyCalculation(); // Вычисляется только при необходимости
    } else {
        return defaultValue;
    }
}

// Мемоизация для дорогих вычислений
function memoize(fn) {
    const cache = new Map();

    return function(...args) {
        const key = JSON.stringify(args);

        if (cache.has(key)) {
            console.log('Возвращаем из кэша');
            return cache.get(key);
        }

        console.log('Вычисляем новый результат');
        const result = fn(...args);
        cache.set(key, result);
        return result;
    };
}

const expensiveFunction = memoize((x, y) => {
    // Имитация дорогого вычисления
    let result = 0;
    for (let i = 0; i < 1000000; i++) {
        result += Math.sin(x) * Math.cos(y);
    }
    return result;
});

console.log(expensiveFunction(1, 2)); // "Вычисляем новый результат"
console.log(expensiveFunction(1, 2)); // "Возвращаем из кэша"

```


6.2. Единая точка возврата (Single Return Principle)

javascript

// Противоречивый принцип: Single Return vs Early Return

// СПОРНО: Единая точка возврата

```
function processDataSingleReturn(data) {  
  let result;  
  
  if (!data) {  
    result = { error: 'Нет данных' };  
  } else if (!Array.isArray(data)) {  
    result = { error: 'Данные должны быть массивом' };  
  } else if (data.length === 0) {  
    result = { value: 0 };  
  } else {  
    const sum = data.reduce((a, b) => a + b, 0);  
    const avg = sum / data.length;  
    result = { sum, avg };  
  }  
  
  return result; // Единая точка возврата  
}
```

// ПРЕДПОЧТИТЕЛЬНО: Ранний возврат для проверок

```
function processDataEarlyReturn(data) {  
  // Проверки с ранним возвратом  
  if (!data) {  
    return { error: 'Нет данных' };  
  }  
  
  if (!Array.isArray(data)) {  
    return { error: 'Данные должны быть массивом' };  
  }  
  
  if (data.length === 0) {  
    return { value: 0 };  
  }  
  
  // Основная логика  
  const sum = data.reduce((a, b) => a + b, 0);  
  const avg = sum / data.length;  
  return { sum, avg };  
}
```

```

}

// Компромисс: ранний возврат для ошибок, единый для успеха
function processDataCompromise(data) {
  // Ранний возврат для ошибок
  if (!data) return { error: 'Нет данных' };
  if (!Array.isArray(data)) return { error: 'Данные должны быть массивом' };

  // Единая точка возврата для успешного выполнения
  let result;

  if (data.length === 0) {
    result = { value: 0 };
  } else {
    const sum = data.reduce((a, b) => a + b, 0);
    const avg = sum / data.length;
    result = { sum, avg };
  }

  // Можно добавить дополнительную обработку
  logResult(result);
  return result;
}

```

6.3. Возврат для цепочек вызовов (Method Chaining)

```

javascript

// Реализация цепочечного API
class QueryBuilder {
  constructor() {
    this.filters = [];
    this.sorts = [];
    this.limit = null;
    this.offset = 0;
  }

  where(field, operator, value) {
    this.filters.push({ field, operator, value });
    return this; // Ключевой момент: возврат this
  }

  orderBy(field, direction = 'ASC') {
    this.sorts.push({ field, direction });
  }
}

```

```

    return this;
}

take(limit) {
    this.limit = limit;
    return this;
}

skip(offset) {
    this.offset = offset;
    return this;
}

build() {
    return {
        filters: this.filters,
        sorts: this.sorts,
        pagination: {
            limit: this.limit,
            offset: this.offset
        }
    };
}
}

```

// Использование цепочечного API

```

const query = new QueryBuilder()
    .where('age', '>', 18)
    .where('status', '=', 'active')
    .orderBy('name', 'ASC')
    .take(10)
    .skip(0)
    .build();

```

```

console.log(query);

```

// Альтернатива с функциями вместо классов

```

function createQuery() {
    const state = {
        filters: [],
        sorts: [],
        limit: null,
        offset: 0
    };
};

```

```
const builder = {
  where(field, operator, value) {
    state.filters.push({ field, operator, value });
    return builder;
  },

  orderBy(field, direction = 'ASC') {
    state.sorts.push({ field, direction });
    return builder;
  },

  take(limit) {
    state.limit = limit;
    return builder;
  },

  skip(offset) {
    state.offset = offset;
    return builder;
  },

  build() {
    return { ...state };
  }
};

return builder;
}
```

```
const query2 = createQuery()
  .where('category', '=', 'books')
  .orderBy('price')
  .take(5)
  .build();
```

Часть 7: Специальные случаи и граничные условия

7.1. Return в циклах и условиях

javascript

// Return внутри циклов

```
function findFirstEven(numbers) {  
  for (const num of numbers) {  
    if (num % 2 === 0) {  
      return num; // Немедленно выходит из функции  
    }  
  }  
  return null; // Если ничего не найдено  
}
```

```
console.log(findFirstEven([1, 3, 5, 8, 9])); // 8
```

// Return в switch

```
function getDayType(day) {  
  switch (day.toLowerCase()) {  
    case 'понедельник':  
    case 'вторник':  
    case 'среда':  
    case 'четверг':  
    case 'пятница':  
      return 'рабочий день';  
  
    case 'суббота':  
    case 'воскресенье':  
      return 'выходной';  
  
    default:  
      return 'неизвестный день';  
  }  
}
```

// Return в условном (тернарном) операторе

```
const getGreeting = (hour) =>  
  hour < 12 ? 'Доброе утро' :  
  hour < 18 ? 'Добрый день' :  
  'Добрый вечер';
```

```
console.log(getGreeting(14)); // "Добрый день"
```

7.2. Возврат из вложенных функций

javascript

// Проблема: return во вложенной функции не возвращает из внешней

```
function searchInNested(data, target) {  
  data.forEach(item => {  
    if (item.value === target) {  
      return item; // Это возвращает только из forEach, не из searchInNested!  
    }  
  });  
  return null;  
}
```

// Решение 1: Использовать обычный цикл

```
function searchInNestedFixed(data, target) {  
  for (const item of data) {  
    if (item.value === target) {  
      return item; // Теперь возвращает из основной функции  
    }  
  }  
  return null;  
}
```

// Решение 2: Флаг для прерывания

```
function searchInNestedWithFlag(data, target) {  
  let found = null;  
  
  data.forEach(item => {  
    if (!found && item.value === target) {  
      found = item;  
    }  
  });  
  
  return found;  
}
```

// Решение 3: Использовать find

```
function searchInNestedModern(data, target) {  
  return data.find(item => item.value === target) || null;  
}
```

7.3. Return и асинхронность

javascript

// Распространенная ошибка с return в асинхронном колбэке

```
function fetchDataWrong(callback) {  
  setTimeout(() => {  
    const data = { id: 1, name: 'Тест' };  
    return callback(data); // Это не возвращает из fetchDataWrong!  
  }, 1000);  
}
```

```
const result = fetchDataWrong(data => data); // result будет undefined
```

// Правильный подход с колбэками

```
function fetchDataCorrect(callback) {  
  setTimeout(() => {  
    const data = { id: 1, name: 'Тест' };  
    callback(data);  
  }, 1000);  
}
```

// С Promise

```
function fetchDataPromise() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      const data = { id: 1, name: 'Тест' };  
      resolve(data); // Аналог return для Promise  
    }, 1000);  
  });  
}
```

// С async/await

```
async function fetchDataAsync() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      const data = { id: 1, name: 'Тест' };  
      resolve(data);  
    }, 1000);  
  });  
}
```

// Использование

```
fetchDataPromise()
```

```
.then(data => console.log('Данные:', data));

async function main() {
  const data = await fetchDataAsync();
  console.log('Данные:', data);
}
```

Заключение: Мастерство использования return

Ключевые принципы:

1. Ясность и предсказуемость:

- ➔ Функция должна возвращать значение ожидаемого типа
- ➔ Документируйте возвращаемые значения с помощью JSDoc
- ➔ Используйте консистентные форматы возвращаемых объектов

2. Эффективность:

- ➔ Используйте ранний возврат для проверок ошибок
- ➔ Избегайте лишних вычислений после return
- ➔ Рассмотрите мемоизацию для дорогих операций

3. Управление потоком:

- ➔ return немедленно прекращает выполнение функции
- ➔ Используйте return для выхода из циклов при нахождении результата
- ➔ Помните о порядке выполнения с finally

4. Обработка ошибок:

- ➔ Используйте throw для критических ошибок
- ➔ Рассмотрите возврат объектов с состоянием success/error для ожидаемых ошибок
- ➔ Не смешивайте throw и возврат ошибок в одной функции

5. Дизайн API:

- ➔ Используйте return this для цепочечных вызовов
- ➔ Возвращайте новые объекты вместо мутации существующих
- ➔ Продумывайте возвращаемые значения для удобства использования

Философия хорошего дизайна функций:

javascript

/**


```

* Пример хорошо спроектированной функции
* @param {Object} options - Опции обработки
* @param {Array} options.data - Данные для обработки
* @param {Function} [options.transform] - Функция трансформации
* @param {boolean} [options.validate=true] - Валидировать ли данные
* @returns {Object} Результат обработки
* @throws {TypeError} Если данные некорректны
*/
function processData({
  data,
  transform = x => x,
  validate = true
} = {}) {
  // Ранний возврат для ошибок
  if (!Array.isArray(data)) {
    throw new TypeError('data должен быть массивом');
  }

  if (validate) {
    const invalid = data.find(item => typeof item !== 'number');
    if (invalid !== undefined) {
      return {
        success: false,
        error: 'Все элементы должны быть числами',
        invalidValue: invalid
      };
    }
  }

  // Основная логика
  const processed = data.map(transform);
  const sum = processed.reduce((a, b) => a + b, 0);
  const average = sum / processed.length;

  // Единая точка возврата для успешного выполнения
  return {
    success: true,
    processed,
    statistics: { sum, average },
    metadata: {
      count: processed.length,
      timestamp: new Date().toISOString()
    }
  };
};

```

```
}
```

```
// Использование
```

```
const result = processData({  
  data: [1, 2, 3, 4, 5],  
  transform: x => x * 2  
});  
  
if (result.success) {  
  console.log('Успех:', result.statistics);  
} else {  
  console.error('Ошибка:', result.error);  
}
```

Итог: Мастерское владение оператором `return` — это признак опытного JavaScript-разработчика. Это не просто технический элемент языка, а инструмент для создания чистого, эффективного и поддерживаемого кода. Практикуйтесь в различных сценариях использования `return`, и вы значительно улучшите качество своего кода.

7.4. Function Expression vs Function Declaration

Введение: Два основных способа создания функций

В JavaScript существует два основных способа объявления функций, которые имеют **фундаментальные различия** в поведении, использовании и области применения. Понимание этих различий критически важно для написания корректного, эффективного и поддерживаемого кода.

Ключевые различия:

- 🔵 **Function Declaration (Декларация функции)** — создается как отдельная конструкция в основном потоке кода
 - 🔵 **Function Expression (Функциональное выражение)** — создается внутри выражения или другого синтаксического конструкта
-

Часть 1: Function Declaration (Декларация функции)

1.1. Синтаксис и базовое использование

```
javascript

// Базовый синтаксис
function functionName(parameters) {
  // тело функции
  return value;
}

// Конкретный пример
function calculateSum(a, b) {
  const sum = a + b;
  return sum;
}

// Вызов функции
const result = calculateSum(5, 3); // 8
```

```
console.log(result);
```

1.2. Поднятие (Hoisting) - ключевая особенность

javascript

// МАГИЯ ПОДНЯТИЯ: функцию можно вызвать ДО её объявления

```
console.log(square(5)); // 25 (работает!)
```

```
function square(x) {  
  return x * x;  
}
```

// Что происходит под капотом:

// 1. На этапе компиляции все function declarations "поднимаются" вверх области видимости

// 2. Фактический порядок выполнения:

// а) Объявление функции перемещается в начало

// б) Выполняется console.log(square(5))

// в) Выполняется вызов функции

// Детальное объяснение процесса:

```
console.log('До объявления:', typeof greet); // "function" (уже существует!)
```

```
console.log('До объявления:', greet); // f greet() { ... }
```

```
function greet(name) {  
  return `Привет, ${name}!`;  
}
```

```
console.log('После объявления:', greet('Анна')); // "Привет, Анна!"
```

1.3. Область видимости Function Declaration

javascript

// Function Declaration имеет блочную область видимости в строгом режиме

```
'use strict';
```

```
if (true) {  
  // Внутри блока  
  function innerFunction() {  
    return 'Внутри блока if';  
  }  
  console.log(innerFunction()); // "Внутри блока if"  
}
```

```
// console.log(innerFunction()); // ОШИБКА в строгом режиме: innerFunction is not defined
```

```
// Без строгого режима - поведение зависит от браузера
```

```
if (true) {  
  function nonStrictFunction() {  
    return 'Без strict mode';  
  }  
}
```

```
// console.log(nonStrictFunction()); // Может работать в некоторых браузерах
```

1.4. Именованные функции и рекурсия

javascript

```
// Именованные функции позволяют рекурсивные вызовы
```

```
function factorial(n) {  
  if (n <= 1) return 1;  
  return n * factorial(n - 1); // Рекурсивный вызов по имени  
}
```

```
console.log(factorial(5)); // 120
```

```
// Функции могут вызывать сами себя внутри
```

```
function fibonacci(n) {  
  if (n <= 1) return n;  
  return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
console.log(fibonacci(10)); // 55
```

```
// Функции как свойства объектов тоже являются function declarations
```

```
const mathOperations = {  
  // Это method definition, но имеет схожие свойства  
  add: function addNumbers(a, b) {  
    return a + b;  
  },  
  multiply: function multiplyNumbers(a, b) {  
    return a * b;  
  }  
};
```

Часть 2: Function Expression (Функциональное выражение)

2.1. Синтаксис и базовое использование

```
javascript

// Базовый синтаксис
const functionName = function(parameters) {
  // тело функции
  return value;
};

// Конкретный пример
const multiply = function(x, y) {
  return x * y;
};

console.log(multiply(4, 5)); // 20

// Присваивание переменной
const greet = function(name) {
  return `Здравствуйте, ${name}!`;
};

// Использование
const message = greet('Петр'); // "Здравствуйте, Петр!"
console.log(message);
```

2.2. Отсутствие поднятия (No Hoisting)

```
javascript

// КРИТИЧЕСКАЯ РАЗНИЦА: function expression НЕ поднимается
// console.log(square(5)); // ОШИБКА: Cannot access 'square' before initialization

const square = function(x) {
  return x * x;
};

console.log(square(5)); // 25 (теперь работает)
```

```

// Что происходит под капотом:
// 1. Объявление переменной square поднимается, но НЕ инициализируется
// 2. Попытка вызова до инициализации вызывает ошибку
// 3. После инициализации функция доступна

console.log('До присваивания:', typeof square); // "undefined" (переменная существует, но
значение undefined)
console.log('До присваивания:', square); // undefined

const square2 = function(x) {
  return x * x;
};

console.log('После присваивания:', square2(5)); // 25

```

2.3. Именованные функциональные выражения (Named Function Expression)

javascript

```

// Именованные функциональные выражения
const factorial = function calculateFactorial(n) {
  // Имя calculateFactorial доступно ТОЛЬКО внутри функции
  if (n <= 1) return 1;

  // Рекурсивный вызов по внутреннему имени
  return n * calculateFactorial(n - 1);
};

console.log(factorial(5)); // 120
// console.log(calculateFactorial(5)); // ОШИБКА: calculateFactorial is not defined

// Преимущества именованных выражений:
// 1. Лучшая трассировка стека вызовов
const recursive = function outer(n) {
  if (n <= 0) return;
  console.log('Вызов:', n);
  return outer(n - 1); // Явное имя для рекурсии
};

recursive(3);
// В стеке вызовов будет: outer -> outer -> outer

// 2. Самодокументирующийся код

```

```
const createUser = function createUserFunction(userData) {  
  // Имя функции говорит о её назначении  
  return { ...userData, id: generateId() };  
};
```

2.4. Анонимные функции

javascript

```
// Анонимные функции (без имени)  
const anonymousFunc = function() {  
  return 'Я анонимная функция';  
};  
  
// Часто используется как колбэк  
const numbers = [1, 2, 3, 4, 5];  
const doubled = numbers.map(function(num) {  
  return num * 2; // Анонимная функция как аргумент  
});
```

```
console.log(doubled); // [2, 4, 6, 8, 10]
```

```
// Немедленно вызываемые функциональные выражения (IIFE)  
(function() {  
  console.log('IIFE выполняется немедленно!');  
})();
```

```
// Сохранение результата IIFE  
const module = (function() {  
  let privateVar = 'секретно';  
  
  return {  
    getSecret: function() {  
      return privateVar;  
    },  
    setSecret: function(value) {  
      privateVar = value;  
    }  
  };  
})();
```

```
console.log(module.getSecret()); // "секретно"
```

Часть 3: Детальное сравнение различий

3.1. Сравнительная таблица

javascript

// Наглядное сравнение в коде

```
console.log('=== СРАВНЕНИЕ FUNCTION DECLARATION И EXPRESSION ===\n');
```

// 1. ПОДНЯТИЕ (HOISTING)

```
console.log('1. ПОДНЯТИЕ:');
```

```
console.log('До объявления declaration:', typeof funcDeclaration); // "function"
```

```
// console.log('До объявления expression:', typeof funcExpression); // ОШИБКА
```

```
function funcDeclaration() {  
  return 'Function Declaration';  
}
```

```
const funcExpression = function() {  
  return 'Function Expression';  
};
```

```
console.log('После объявлений - оба работают');  
console.log(funcDeclaration()); // "Function Declaration"  
console.log(funcExpression()); // "Function Expression"
```

// 2. ОБЛАСТЬ ВИДИМОСТИ

```
console.log('\n2. ОБЛАСТЬ ВИДИМОСТИ:');
```

```
if (true) {  
  function declarationInBlock() {  
    return 'Declaration в блоке';  
  }  
  const expressionInBlock = function() {  
    return 'Expression в блоке';  
  };  
}
```

// Поведение зависит от режима и окружения

// В строгом режиме: обе функции не доступны вне блока

// В нестрогом режиме: declaration может "просачиваться"

// 3. ПРИСВАИВАНИЕ И ПЕРЕЗАПИСЬ

```
console.log('\n3. ПРИСВАИВАНИЕ И ПЕРЕЗАПИСЬ:');
```

```

function reusableFunction() {
  return 'Original';
}

// Function declaration можно "перезаписать"
function reusableFunction() {
  return 'Overwritten'; // Нет ошибки, но может быть неожиданно
}

console.log(reusableFunction()); // "Overwritten"

const constantExpression = function() {
  return 'Original Expression';
};

// constantExpression = function() { return 'New'; }; // ОШИБКА: Assignment to constant variable
// Но можно изменить свойство объекта
constantExpression.version = '1.0';
console.log(constantExpression.version); // "1.0"

```

3.2. Таблица сравнения характеристик

javascript

```

// Создадим объект для хранения сравнений
const comparison = {
  // Function Declaration
  declaration: {
    синтаксис: 'function name() {}',
    поднятие: 'Полное (можно вызвать до объявления)',
    областьВидимости: 'Блочная (в strict mode)',
    можноПерезаписать: 'Да (без ошибки)',
    имяОбязательно: 'Да',
    примерыИспользования: [
      'Основные функции программы',
      'Рекурсивные функции',
      'Функции, вызываемые в разных местах'
    ]
  },
  // Function Expression
  expression: {
    синтаксис: 'const name = function() {}',

```

```

    поднятие: 'Нет (ошибка при вызове до инициализации)',
    областьВидимости: 'Зависит от переменной (const/let/var)',
    можноПерезаписать: 'Зависит от переменной (const/let)',
    имяОбязательно: 'Нет (может быть анонимной)',
    примерыИспользования: [
        'Колбэки',
        'Присваивание переменным',
        'Немедленный вызов (IIFE)',
        'Возврат из других функций'
    ]
}
};

```

```

console.log('СРАВНИТЕЛЬНАЯ ТАБЛИЦА:');
console.table(comparison.declaration);
console.table(comparison.expression);

```

3.3. Временная мертвая зона (Temporal Dead Zone - TDZ)

javascript

```
// TDZ для function expressions с let/const
```

```
console.log('Начало TDZ для funcExpr');
```

```
// Временная мертвая зона - область между началом блока и инициализацией
```

```
// funcExpr находится в TDZ, обращение к ней вызывает ошибку
```

```
// console.Log(funcExpr); // ОШИБКА: Cannot access 'funcExpr' before initialization
```

```
// funcExpr(); // ОШИБКА
```

```

const funcExpr = function() {
    return 'Function Expression';
};

```

```

console.log('Конец TDZ - функция доступна');
console.log(funcExpr()); // "Function Expression"

```

```
// Для var - нет TDZ, но значение undefined
```

```
console.log('Для var:');
```

```
console.log(typeof funcVar); // "undefined" (но не ошибка)
```

```
// funcVar(); // ОШИБКА: funcVar is not a function (но другой тип ошибки)
```

```

var funcVar = function() {
    return 'Function Expression с var';
};

```

```
};
```

```
console.log(funcVar()); // Теперь работает
```

Часть 4: Практические сценарии использования

4.1. Когда использовать Function Declaration

javascript

```
// 1. ОСНОВНЫЕ ФУНКЦИИ ПРОГРАММЫ
```

```
// Function declaration для ключевых функций
```

```
function initializeApp() {  
  loadConfig();  
  setupEventListeners();  
  startMainLoop();  
}
```

```
function loadConfig() {  
  // Загрузка конфигурации  
  return { theme: 'dark', language: 'ru' };  
}
```

```
function setupEventListeners() {  
  // Настройка обработчиков событий  
  document.addEventListener('click', handleClick);  
}
```

```
// 2. РЕКУРСИВНЫЕ ФУНКЦИИ
```

```
// Явные имена помогают в рекурсии
```

```
function traverseDOM(node, callback) {  
  callback(node);  
  
  for (const child of node.children) {  
    traverseDOM(child, callback); // Четкое имя для рекурсивного вызова  
  }  
}
```

```
// 3. ФУНКЦИИ, КОТОРЫЕ ВЗАИМОДЕЙСТВУЮТ ДРУГ С ДРУГОМ
```

```
function processData(data) {
```

```

validateData(data);
const cleaned = cleanData(data);
return transformData(cleaned);
}

function validateData(data) {
  if (!data) throw new Error('Нет данных');
}

function cleanData(data) {
  return data.filter(item => item !== null);
}

function transformData(data) {
  return data.map(item => item * 2);
}

```

4.2. Когда использовать Function Expression

javascript

```

// 1. КОЛБЭКИ И ОБРАБОТЧИКИ СОБЫТИЙ
// Анонимные или именованные выражения как колбэки
document.addEventListener('DOMContentLoaded', function() {
  console.log('DOM загружен');
});

const button = document.querySelector('#submit');
button.addEventListener('click', function handleClick(event) {
  event.preventDefault();
  processForm();
});

// 2. ЗАМЫКАНИЯ И ЧАСТИЧНОЕ ПРИМЕНЕНИЕ
function createCounter(initial = 0) {
  let count = initial;

  // Function expression для методов
  return {
    increment: function() {
      count++;
      return count;
    },
    decrement: function() {

```

```
    count--;  
    return count;  
  },  
  getValue: function() {  
    return count;  
  }  
};  
}
```

// 3. ДИНАМИЧЕСКОЕ СОЗДАНИЕ ФУНКЦИЙ

```
const operationType = 'add';  
const operation = operationType === 'add'  
  ? function(a, b) { return a + b; }  
  : function(a, b) { return a - b; };
```

```
console.log(operation(5, 3)); // 8
```

// 4. МОДУЛЬНЫЙ ПАТТЕРН (до ES6 модулей)

```
const UserModule = (function() {  
  // Приватные переменные  
  let users = [];  
  
  // Приватные методы  
  const validateUser = function(user) {  
    return user && user.name && user.email;  
  };  
};
```

// Публичный интерфейс

```
return {  
  addUser: function(user) {  
    if (validateUser(user)) {  
      users.push(user);  
      return true;  
    }  
    return false;  
  },  
  
  getUsers: function() {  
    return [...users]; // Возвращаем копию  
  },  
  
  getUserCount: function() {  
    return users.length;  
  }  
}
```

```
};  
})();
```

4.3. Смешанное использование в реальных проектах

javascript

// ТИПИЧНАЯ СТРУКТУРА ФАЙЛА В РЕАЛЬНОМ ПРОЕКТЕ

// 1. Function declarations для основных экспортируемых функций

```
function initializeApplication(config) {  
  validateConfig(config);  
  setupServices(config);  
  return startApp();  
}  
  
function validateConfig(config) {  
  if (!config.apiUrl) {  
    throw new Error('apiUrl обязателен в конфиге');  
  }  
}
```

// 2. Function expressions для вспомогательных функций

```
const setupServices = function(config) {  
  const services = {  
    api: createAPIService(config.apiUrl),  
    auth: createAuthService(config.auth),  
    storage: createStorageService(config.storage)  
  };  
};
```

// Вложенные function expressions

```
const createAPIService = function(baseUrl) {  
  return {  
    get: function(endpoint) {  
      return fetch(`${baseUrl}/${endpoint}`);  
    },  
    post: function(endpoint, data) {  
      return fetch(`${baseUrl}/${endpoint}`, {  
        method: 'POST',  
        body: JSON.stringify(data)  
      });  
    }  
  };  
};
```

```
    return services;
};
```

// 3. IIFE для изоляции кода

```
const startApp = (function() {
    let appInstance = null;

    return function() {
        if (!appInstance) {
            appInstance = {
                startedAt: new Date(),
                status: 'running'
            };
            console.log('Приложение запущено');
        }
        return appInstance;
    };
})();
```

// 4. Колбэки как function expressions

```
const eventHandlers = {
    onUserLogin: function(user) {
        console.log('Пользователь вошел:', user.name);
        updateUI(user);
    }
};
```

// Вложенный колбэк

```
fetchUserData(user.id, function(data) {
    displayUserData(data);
});

onError: function(error) {
    console.error('Произошла ошибка:', error.message);
    showErrorToast(error);
}
};
```

// 5. Фабрики функций (возвращают function expressions)

```
function createValidator(rules) {
    return function(data) {
        const errors = [];

        for (const [field, rule] of Object.entries(rules)) {
```



```
const value = data[field];

if (rule.required && (value === undefined || value === null || value === '')) {
  errors.push(`${field} обязательно`);
}
}

return errors;
};
}
```

Часть 5: Производительность и оптимизация

5.1. Бенчмарки и производительность

javascript

// Тестирование производительности

```
function benchmark() {
  const iterations = 1000000;

  // Function Declaration
  console.time('Function Declaration');
  function declFunc(x) { return x * x; }
  for (let i = 0; i < iterations; i++) {
    declFunc(i);
  }
  console.timeEnd('Function Declaration');
```

// Function Expression

```
console.time('Function Expression');
const exprFunc = function(x) { return x * x; };
for (let i = 0; i < iterations; i++) {
  exprFunc(i);
}
console.timeEnd('Function Expression');
```

// Именованное Function Expression

```
console.time('Named Function Expression');
const namedExprFunc = function named(x) { return x * x; };
```

```

for (let i = 0; i < iterations; i++) {
  namedExprFunc(i);
}
console.timeEnd('Named Function Expression');
}

// В большинстве современных движков разница минимальна
// Но есть нюансы:

// 1. Function Declaration может быть оптимизирована лучше
// при повторных вызовах в одной области видимости

// 2. Function Expression создается каждый раз при выполнении
// выражения (если не в цикле)

```

5.2. Оптимизация памяти

javascript

```

// ПЕРЕЗАПИСЬ FUNCTION DECLARATION - ОПАСНОСТЬ УТЕЧКИ ПАМЯТИ
function heavyOperation() {
  const largeArray = new Array(1000000).fill('data');
  // Большой объем памяти
  return largeArray.map(x => x.toUpperCase());
}

// Если перезаписать функцию, старая может остаться в памяти
function heavyOperation() {
  // Новая реализация
  return 'упрощенная версия';
}

// Сборщик мусора может не сразу освободить память от старой функции
// и её замыканий

// Function Expression с const защищает от случайной перезаписи
const safeHeavyOperation = function() {
  const largeArray = new Array(1000000).fill('data');
  return largeArray.map(x => x.toUpperCase());
};

// safeHeavyOperation = function() { return 'new'; }; // ОШИБКА

// Использование let позволяет контролируемую замену

```

```
let dynamicFunction = function() {
  console.log('Версия 1');
};

// Можно заменить при необходимости
dynamicFunction = function() {
  console.log('Версия 2 (оптимизированная)');
};
```

5.3. Движок V8 и оптимизации

javascript

```
// V8 (движок Chrome/Node.js) оптимизирует функции по-разному

// 1. Function Declaration - оптимизируется при парсинге
function optimizedDeclaration() {
  // Внутреннее имя помогает оптимизатору
  return 'оптимизировано';
}

// 2. Function Expression - может быть оптимизирована как "горячая" функция
const frequentlyUsed = function() {
  // При частом вызове V8 компилирует в оптимизированный код
  let sum = 0;
  for (let i = 0; i < 1000; i++) {
    sum += i;
  }
  return sum;
};

// Многократный вызов для "разогрева"
for (let i = 0; i < 10000; i++) {
  frequentlyUsed();
}

// 3. Именованные Function Expression дают лучшие stack traces
const recursiveFunction = function traverseTree(node) {
  if (!node) return;
  // В стеке вызовов будет видно traverseTree
  traverseTree(node.left);
  traverseTree(node.right);
};
```

```
// 4. Анонимные функции в колбэках
// Могут затруднять отладку
array.map(function(item) { // Анонимная - в стеке будет "anonymous"
  return processItem(item);
});

array.map(function processItem(item) { // Именованная - лучше для отладки
  return item * 2;
});
```

Часть 6: Особые случаи и граничные условия

6.1. Условное объявление функций

```
javascript

// Function Declaration в условном блоке - ПРОБЛЕМАТИЧНО
if (true) {
  function conditionalDeclaration() {
    return 'Внутри if';
  }
}

// Поведение зависит от режима:
// - Strict mode: функция имеет блочную область видимости
// - Slippy mode: поведение зависит от браузера (может "всплывать")

// Function Expression - ПРЕДСКАЗУЕМО
let conditionalExpression;
if (true) {
  conditionalExpression = function() {
    return 'Внутри if (expression)';
  };
} else {
  conditionalExpression = function() {
    return 'Внутри else (expression)';
  };
}

console.log(conditionalExpression()); // Всегда предсказуемо
```

```
// Динамическое создание функций
const createHandler = (type) => {
  switch (type) {
    case 'click':
      return function(event) {
        console.log('Клик:', event.target);
      };
    case 'hover':
      return function(event) {
        console.log('Наведение:', event.target);
      };
    default:
      return function(event) {
        console.log('Событие:', event.type);
      };
  }
};
```

6.2. Функции в циклах

javascript

```
// ПРОБЛЕМА: Function Declaration в цикле
for (let i = 0; i < 3; i++) {
  function loopDeclaration() {
    return i; // Всегда будет возвращать последнее значение i
  }
  // В некоторых браузерах будет создана только одна функция
  // В других - три разных функции
}
```

```
// РЕШЕНИЕ: Function Expression в цикле
const handlers = [];
for (let i = 0; i < 3; i++) {
  // Создается новая функция на каждой итерации
  const handler = function() {
    return i; // Замыкает текущее значение i
  };
  handlers.push(handler);
}
```

```
console.log(handlers[0]()); // 0
console.log(handlers[1]()); // 1
```

```

console.log(handlers[2]()); // 2

// Еще лучше: использовать let для замыкания
const handlers2 = [];
for (let i = 0; i < 3; i++) {
  handlers2.push(function() {
    return i; // Каждая функция замыкает свою копию i
  });
}

// Проблема с var (старый код)
const problems = [];
for (var j = 0; j < 3; j++) {
  problems.push(function() {
    return j; // Все функции ссылаются на ОДНУ переменную j
  });
}

console.log(problems[0]()); // 3 (последнее значение)
console.log(problems[1]()); // 3
console.log(problems[2]()); // 3

```

6.3. Рекурсия и самовывоз

javascript

```

// Function Declaration для рекурсии
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1); // Явное имя для рекурсии
}

// Function Expression с именем для рекурсии
const factorialExpr = function fact(n) {
  if (n <= 1) return 1;
  return n * fact(n - 1); // Используем внутреннее имя
};

// Анонимная Function Expression не может рекурсивно вызывать себя
const recursiveProblem = function(n) {
  if (n <= 1) return 1;
  // return n * recursiveProblem(n - 1); // ОШИБКА: пока функция не определена
  return n * arguments.callee(n - 1); // arguments.callee устарел и запрещен в strict mode
};

```

// Решение: использовать именованное выражение или function declaration

6.4. Геттеры, сеттеры и методы объектов

javascript

// В объектных литералах используются function expressions

```
const user = {  
  name: 'Анна',
```

// Method definition (похоже на declaration, но это выражение)

```
  greet() {  
    return `Привет, я ${this.name}`;  
  },
```

// Традиционный function expression

```
  greetTraditional: function() {  
    return `Здравствуйте, я ${this.name}`;  
  },
```

// Стрелочная функция (тоже expression)

```
  greetArrow: () => {  
    // Не имеет своего this  
    return `Привет от ${this.name}`; // this будет из внешней области  
  },
```

// Геттер (использует синтаксис declaration-like)

```
  get displayName() {  
    return `${this.name} (пользователь)`;  
  },
```

// Сеттер

```
  set displayName(value) {  
    const [name] = value.split(' ');  
    this.name = name;  
  }  
};
```

```
console.log(user.greet()); // "Привет, я Анна"
```

```
console.log(user.greetTraditional()); // "Здравствуйте, я Анна"
```

```
console.log(user.displayName); // "Анна (пользователь)"
```

Часть 7: Современные тенденции и лучшие практики

7.1. Airbnb Style Guide рекомендации

javascript

// РЕКОМЕНДАЦИИ AIRBNB STYLE GUIDE:

// 1. Используйте именованные function expressions вместо declarations

// Плохо (согласно Airbnb):

```
function foo() {  
  // ...  
}
```

// Хорошо:

```
const foo = function() {  
  // ...  
};
```

// Лучше (именованное выражение):

```
const foo = function bar() {  
  // ...  
};
```

// 2. Для экспортируемых функций используйте declarations

```
export function makeRequest(url) {  
  // ...  
}
```

// 3. Не объявляйте функции в блоках

// Плохо:

```
if (condition) {  
  function foo() {  
    // ...  
  }  
}
```

// Хорошо:

```
let foo;  
if (condition) {  
  foo = function() {  
    // ...  
  };  
}
```



```
}
```

```
// 4. Используйте стрелочные функции для коротких колбэков
```

```
// Плохо:
```

```
[1, 2, 3].map(function(x) {  
  return x * x;  
});
```

```
// Хорошо:
```

```
[1, 2, 3].map(x => x * x);
```

```
// 5. Именуйте функции для лучшей трассировки
```

```
// Плохо:
```

```
const controller = {  
  handleClick: function() { // Анонимная  
    // ...  
  }  
};
```

```
// Хорошо:
```

```
const controller = {  
  handleClick: function handleClick() { // Именованная  
    // ...  
  }  
};
```

7.2. TypeScript и типизация

typescript

```
// TypeScript добавляет статическую типизацию к функциям
```

```
// Function Declaration с тупами
```

```
function add(a: number, b: number): number {  
  return a + b;  
}
```

```
// Function Expression с тупами
```

```
const multiply: (x: number, y: number) => number = function(x, y) {  
  return x * y;  
};
```

```
// Более читаемый вариант с type alias
```

```
type MathOperation = (a: number, b: number) => number;
```

```
const divide: MathOperation = function(a, b) {  
  return a / b;  
};  
  
// Стрелочные функции с типами  
const subtract = (a: number, b: number): number => a - b;  
  
// В TypeScript function expressions часто предпочтительнее  
// из-за более явной типизации
```

7.3. Современный JavaScript (ES6+)

javascript

// Стрелочные функции как современная альтернатива

```
const oldStyle = function(x) {  
  return x * 2;  
};
```

```
const modernStyle = x => x * 2;
```

// Method definitions в классах и объектах

```
class Calculator {  
  // Похоже на declaration, но это метод класса  
  add(a, b) {  
    return a + b;  
  }  
}
```

// Стрелочная функция как свойство класса

```
multiply = (a, b) => a * b;  
}
```

// Динамические имена методов

```
const methodName = 'calculate';  
const advancedCalculator = {  
  [methodName + 'Sum'](a, b) {  
    return a + b;  
  }  
};
```

// Async функции (всегда declarations или expressions)

```
async function fetchData() {  
  const response = await fetch('/api/data');
```

```

    return response.json();
}

const fetchDataExpr = async function() {
    const response = await fetch('/api/data');
    return response.json();
};

const fetchDataArrow = async () => {
    const response = await fetch('/api/data');
    return response.json();
};

```

7.4. Практические рекомендации для проектов

javascript

// ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ:

// 1. Для публичного API модуля используйте declarations

```

export function createUser(userData) {
    validateUserData(userData);
    return saveToDatabase(userData);
}

```

// Вспомогательные функции внутри модуля - expressions

```

const validateUserData = function(data) {
    if (!data.name) {
        throw new Error('Имя обязательно');
    }
    // ...
};

```

// 2. Колбэки и обработчики - стрелочные функции или именованные expressions

```

button.addEventListener('click', (event) => {
    handleClick(event);
});

```

// Или именованные для сложной логики

```

button.addEventListener('click', function handleClick(event) {
    event.preventDefault();
    processFormData();
    updateUI();
});

```

// 3. Фабрики и замыкания - function expressions

```
function createCounter(initial = 0) {  
  let count = initial;  
  
  return {  
    increment: function() {  
      return ++count;  
    },  
    getValue: function() {  
      return count;  
    }  
  };  
}
```

// 4. Рекурсивные функции - declarations

```
function traverseTree(node, callback) {  
  if (!node) return;  
  callback(node);  
  traverseTree(node.left, callback);  
  traverseTree(node.right, callback);  
}
```

// 5. Динамические функции - expressions

```
const createValidator = (rules) => {  
  return function(data) {  
    // Валидация по правилам  
    return isValid;  
  };  
};
```

// 6. Всегда используйте const для function expressions

// Плохо:

```
var oldFunc = function() { /* ... */ };  
let mutableFunc = function() { /* ... */ };
```

// Хорошо:

```
const stableFunc = function() { /* ... */ };
```

// 7. Именуйте функции для отладки

// В стеке вызовов будет "anonymous" vs "meaningfulName"

```
const processOrder = function processOrderFunction(order) {  
  // ...  
};
```

```
// 8. Используйте JSDoc для документирования
/**
 * Создает нового пользователя
 * @param {Object} userData - Данные пользователя
 * @returns {Promise<User>} Созданный пользователь
 */
const createUser = async function(userData) {
  // ...
};
```

Заключение: Выбор между Declaration и Expression

Итоговые рекомендации:

1. Используйте Function Declaration когда:

- ➔ Функция является основной частью программы
- ➔ Нужна рекурсия с явным именем
- ➔ Функция используется в нескольких местах файла
- ➔ Экспортируете функцию из модуля

2. Используйте Function Expression когда:

- ➔ Создаете колбэк или обработчик события
- ➔ Присваиваете функцию переменной
- ➔ Создаете функцию динамически
- ➔ Нужно замыкание переменных
- ➔ Создаете метод объекта

3. Всегда предпочитайте именованные функции:

- ➔ Лучшая трассировка стека
- ➔ Самодокументирующийся код
- ➔ Легче отладка

4. Используйте const для function expressions:

- ➔ Защита от случайной перезаписи
- ➔ Ясные намерения
- ➔ Поддержка TDZ

5. Учитывайте современные альтернативы:

- ➔ Стрелочные функции для коротких колбэков
- ➔ Method definitions в объектах и классах
- ➔ Async/await для асинхронного кода

Финальный пример правильного использования:

javascript

```
// МОДУЛЬ С ПРАВИЛЬНЫМ ИСПОЛЬЗОВАНИЕМ

// Экспортируемые функции - declarations
export function initializeApp(config) {
  validateConfig(config);
  const services = setupServices(config);
  return startApplication(services);
}

// Вспомогательные функции - named expressions
const validateConfig = function validateConfigFunction(config) {
  if (!config.apiUrl) {
    throw new Error('API URL обязателен');
  }
  return true;
};

// Фабрика функций - expression
const setupServices = function(config) {
  return {
    api: createApiService(config.apiUrl),
    auth: createAuthService(config.auth)
  };
};

// Вложенные фабрики - expressions
const createApiService = function(baseUrl) {
  return {
    get: async function(endpoint) {
      const response = await fetch(`${baseUrl}/${endpoint}`);
      return response.json();
    },

    post: async function(endpoint, data) {
      const response = await fetch(`${baseUrl}/${endpoint}`, {
        method: 'POST',
        body: JSON.stringify(data)
      });
      return response.json();
    }
  };
}
```

```

};
};

// Стрелочные функции для простых колбэков
const createAuthService = (config) => ({
  login: async (credentials) => {
    const response = await fetch('/api/login', {
      method: 'POST',
      body: JSON.stringify(credentials)
    });
    return response.json();
  },

  logout: () => {
    localStorage.removeItem('token');
  }
});

// Рекурсивная функция - declaration
function startApplication(services, maxRetries = 3) {
  try {
    services.api.get('/health');
    return { status: 'running', services };
  } catch (error) {
    if (maxRetries > 0) {
      console.log('Повторная попытка...');
      return startApplication(services, maxRetries - 1);
    }
    throw error;
  }
}

// Колбэк с именованной функцией
window.addEventListener('load', function onWindowLoad() {
  console.log('Окно загружено, инициализируем приложение...');
  initializeApp({ apiUrl: '/api' })
    .then(app => console.log('Приложение запущено:', app))
    .catch(error => console.error('Ошибка запуска:', error));
});

```

Запомните: Нет "абсолютно правильного" выбора между Function Declaration и Expression. Ключ в понимании их различий и осознанном выборе в зависимости от контекста. С практикой вы разовьете интуицию для выбора подходящего варианта в каждой ситуации.

7.5. Анонимные функции и немедленно вызываемые функции (IIFE)

Введение: Мощность анонимности и немедленного выполнения

Анонимные функции и IIFE (Immediately Invoked Function Expressions) — это мощные концепции JavaScript, которые позволяют создавать **самоисполняющийся, изолированный** код без загрязнения глобального пространства имен.

Часть 1: Анонимные функции (Anonymous Functions)

1.1. Что такое анонимные функции

Анонимные функции — это функции без имени. Они создаются "на лету" и часто используются как значения.

```
javascript
```

```
// Сравнение именованной и анонимной функции
```

```
// Именованная функция (Named Function)
```

```
function namedFunction(x) {  
  return x * 2;  
}
```

```
// Анонимная функция (Anonymous Function)
```

```
const anonymousFunction = function(x) {  
  return x * 2;  
};
```

```
// Стрелочная анонимная функция
```

```
const arrowAnonymous = x => x * 2;
```

```
console.log(namedFunction.name);    // "namedFunction"
```

```
console.log(anonymousFunction.name); // "" (пустая строка)
```



```
console.log(arrowAnonymous.name);    // "arrowAnonymous" (ES6 дает имя)
```

1.2. Основные характеристики анонимных функций

javascript

// 1. Нет имени в объявлении

```
const noName = function() {  
  return "Я без имени";  
};
```

// 2. Часто используются как колбэки

```
const numbers = [1, 2, 3, 4, 5];
```

// Анонимная функция как аргумент

```
const doubled = numbers.map(function(num) {  
  return num * 2;  
});
```

```
console.log(doubled); // [2, 4, 6, 8, 10]
```

// 3. Могут быть присвоены переменным

```
const processor = function(data) {  
  return data.toUpperCase();  
};
```

// 4. Могут быть возвращены из других функций

```
function createMultiplier(factor) {  
  return function(number) { // Возвращаем анонимную функцию  
    return number * factor;  
  };  
}
```

```
const double = createMultiplier(2);
```

```
console.log(double(5)); // 10
```

1.3. Преимущества и недостатки

javascript

// ПРЕИМУЩЕСТВА анонимных функций:

// 1. Краткость для одноразового использования

```
setTimeout(function() {
```

```
    console.log('Выполнилось через 1 секунду');  
  }, 1000);
```

// 2. Не загрязняют пространство имен

// Нет лишнего имени функции в текущей области видимости

// 3. Удобны для inline-использования

```
const users = ['Анна', 'Борис', 'Виктор'];  
const greetings = users.map(function(user) {  
  return `Привет, ${user}!`;  
});
```

// НЕДОСТАТКИ анонимных функций:

// 1. Сложность отладки (в стеке вызовов "anonymous")

```
function problematicCode() {  
  [1, 2, 3].forEach(function(item) {  
    // В стеке вызовов будет "anonymous"  
    if (item === 2) {  
      throw new Error('Проблема на элементе 2');  
    }  
  });  
}
```

```
try {  
  problematicCode();  
} catch (error) {  
  console.error(error.stack);  
  // Будет содержать "at anonymous" вместо понятного имени  
}
```

// 2. Невозможность рекурсивного вызова

```
const recursiveAttempt = function(n) {  
  if (n <= 0) return;  
  console.log(n);  
  // recursiveAttempt(n - 1); // Не работает, если функция не присвоена переменной  
};
```

// 3. Сложнее читать при больших объемах кода

```
document.addEventListener('click', function(event) {  
  // 50 строк сложной логики...  
  // Кто вызвал эту функцию? Что она делает?  
});
```

1.4. Практическое применение анонимных функций

javascript

// 1. ОБРАБОТЧИКИ СОБЫТИЙ

```
const button = document.createElement('button');
button.textContent = 'Нажми меня';

button.addEventListener('click', function(event) {
  // Анонимный обработчик
  console.log('Кнопка нажата!', event.target);
  this.style.backgroundColor = 'red'; // this ссылается на button
});
```

// 2. КОЛБЭКИ В МАССИВНЫХ МЕТОДАХ

```
const data = [
  { id: 1, value: 10, active: true },
  { id: 2, value: 20, active: false },
  { id: 3, value: 30, active: true }
];
```

// Цепочка анонимных функций

```
const result = data
  .filter(function(item) {
    return item.active;
  })
  .map(function(item) {
    return item.value * 2;
  })
  .reduce(function(sum, value) {
    return sum + value;
  }, 0);

console.log(result); // (10*2 + 30*2) = 80
```

// 3. АСИНХРОННЫЕ ОПЕРАЦИИ

```
function fetchData(url, callback) {
  // Имитация асинхронного запроса
  setTimeout(function() {
    const data = { url, timestamp: new Date() };
    callback(null, data); // Анонимный колбэк
  }, 1000);
}
```

```
fetchData('/api/users', function(error, data) {
  if (error) {
    console.error('Ошибка:', error);
  } else {
    console.log('Данные получены:', data);
  }
});

// 4. ЗАМЫКАНИЯ (CLOSURES)
function createCounter() {
  let count = 0;

  return function() { // Анонимная функция, замыкающая count
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Часть 2: Немедленно вызываемые функции (IIFE)

2.1. Что такое IIFE и зачем они нужны

IIFE (Immediately Invoked Function Expression) — это функция, которая выполняется сразу после своего создания.

javascript

```
// Базовый синтаксис IIFE
(function() {
  console.log('Я выполняюсь сразу!');
})();
```

```
// Альтернативные синтаксисы
```

```
(function() {
  console.log('Вариант 1');
```

```
}());
```

```
!function() {  
  console.log('Вариант 2');  
}();
```

```
+function() {  
  console.log('Вариант 3');  
}();
```

```
void function() {  
  console.log('Вариант 4');  
}();
```

2.2. Исторический контекст: проблема глобального пространства имен

javascript

// ПРОБЛЕМА: загрязнение глобального пространства имен (до модулей ES6)

// script1.js

```
var userCount = 0;  
function addUser() {  
  userCount++;  
}
```

// script2.js

```
var userCount = 100; // КОНФЛИКТ ИМЕН!  
function addUser() { // ЕЩЕ КОНФЛИКТ!  
  console.log('Другая функция');  
}
```

// РЕШЕНИЕ с IIFE (паттерн модуля)

```
var MyModule = (function() {  
  // Приватные переменные  
  var userCount = 0;  
  var secretKey = 'abc123';  
  
  // Приватная функция  
  function generateId() {  
    return 'id_' + Math.random().toString(36).substr(2, 9);  
  }  
}
```

```

// Публичный интерфейс
return {
  addUser: function() {
    userCount++;
    return generateId();
  },

  getUserCount: function() {
    return userCount;
  },

  getInfo: function() {
    return `Пользователей: ${userCount}, Ключ: ${secretKey.substr(0, 3)}...`;
  }
};
})();

// Использование модуля
console.log(MyModule.addUser()); // "id_abc123def"
console.log(MyModule.getUserCount()); // 1
console.log(MyModule.getInfo()); // "Пользователей: 1, Ключ: abc..."

// Приватные переменные недоступны извне
console.log(MyModule.userCount); // undefined
console.log(MyModule.secretKey); // undefined
console.log(MyModule.generateId); // undefined

```

2.3. Современное использование IIFE

javascript

```

// 1. ИЗОЛЯЦИЯ ПЕРЕМЕННЫХ В БЛОКАХ (до появления let/const)
for (var i = 0; i < 5; i++) {
  (function(index) {
    setTimeout(function() {
      console.log('index:', index); // 0, 1, 2, 3, 4
    }, 100 * index);
  })(i);
}

// Без IIFE (проблема с var):
for (var j = 0; j < 5; j++) {
  setTimeout(function() {
    console.log('j:', j); // 5, 5, 5, 5, 5
  });
}

```

```
    }, 100 * j);  
}
```

```
// 2. СОВРЕМЕННОЕ РЕШЕНИЕ (с let):
```

```
for (let k = 0; k < 5; k++) {  
    setTimeout(function() {  
        console.log('k:', k); // 0, 1, 2, 3, 4  
    }, 100 * k);  
}
```

```
// 3. ЗАЩИТА ОТ КОНФЛИКТОВ В БИБЛИОТЕКАХ
```

```
// jQuery использует IIFE
```

```
(function(global, factory) {
```

```
    // Защищенный код jQuery
```

```
    if (typeof module === "object" && typeof module.exports === "object") {
```

```
        // CommonJS
```

```
        module.exports = global.document ?
```

```
            factory(global, true) :
```

```
            function(w) {
```

```
                if (!w.document) {
```

```
                    throw new Error("jQuery requires a window with a document");
```

```
                }
```

```
                return factory(w);
```

```
            };
```

```
    } else {
```

```
        factory(global);
```

```
    }
```

```
})(typeof window !== "undefined" ? window : this, function(window, noGlobal) {
```

```
    // Реализация jQuery
```

```
    var jQuery = function(selector, context) {
```

```
        return new jQuery.fn.init(selector, context);
```

```
    };
```

```
    // ... остальной код
```

```
    if (typeof noGlobal === "undefined") {
```

```
        window.jQuery = window.$ = jQuery;
```

```
    }
```

```
    return jQuery;
```

```
});
```

2.4. Синтаксические вариации IIFE

javascript

// 1. КЛАССИЧЕСКИЙ СИНТАКСИС (наиболее читаемый)

```
(function() {  
  console.log('Классический IIFE');  
})();
```

// 2. С ПАРАМЕТРАМИ

```
(function(window, document, undefined) {  
  // undefined гарантированно undefined (защита от переопределения)  
  console.log('IIFE с параметрами');  
  console.log('window:', typeof window);  
  console.log('document:', typeof document);  
  console.log('undefined:', undefined);  
})(window, document);
```

// 3. С ВОЗВРАТОМ ЗНАЧЕНИЯ

```
const randomNumber = (function() {  
  console.log('Генерирую случайное число...');  
  return Math.random();  
})();
```

```
console.log('Случайное число:', randomNumber);
```

// 4. С ПРИСВАИВАНИЕМ

```
const calculator = (function() {  
  let memory = 0;  
  
  return {  
    add: function(x) {  
      memory += x;  
      return this;  
    },  
  
    subtract: function(x) {  
      memory -= x;  
      return this;  
    },  
  
    getResult: function() {  
      return memory;  
    },  
  },
```



```
    reset: function() {
      memory = 0;
      return this;
    }
  };
})();

console.log(
  calculator.add(5).subtract(2).add(10).getResult()
); // 13

// 5. АСИНХРОННЫЙ IIFE
(async function() {
  console.log('Начало асинхронного IIFE');

  try {
    // Имитация асинхронной операции
    const data = await new Promise(resolve => {
      setTimeout(() => resolve('Данные получены'), 1000);
    });

    console.log(data);
  } catch (error) {
    console.error('Ошибка:', error);
  }

  console.log('Конец асинхронного IIFE');
})();

console.log('Этот код выполнится ДО завершения асинхронного IIFE');
```

Часть 3: Глубокое погружение в IIFE

3.1. Механизм работы IIFE

javascript

// Как работает IIFE шаг за шагом:

```

// Шаг 1: Создаем функциональное выражение
var functionExpression = function() {
  console.log('Я функциональное выражение');
};

// Шаг 2: Вызываем его
functionExpression();

// Шаг 3: Делаем это в одном выражении (IIFE)
(function() {
  console.log('Я IIFE');
})();

// Подробный разбор синтаксиса:
// (function() { ... }) <- Часть 1: Функциональное выражение в скобках
// () <- Часть 2: Оператор вызова функции

// Почему нужны скобки?
function() {
  console.log('Без скобок - синтаксическая ошибка');
}(); // SyntaxError: function statement requires a name

// Скобки превращают declaration в expression
(function() {
  console.log('Со скобками - работает!');
})();

```

3.2. Замыкания и приватность в IIFE

javascript

```

// Создание приватного состояния
const bankAccount = (function() {
  // ПРИВАТНЫЕ ПЕРЕМЕННЫЕ (инкапсуляция)
  let balance = 0;
  let transactionHistory = [];

  // ПРИВАТНАЯ ФУНКЦИЯ
  function logTransaction(type, amount) {
    transactionHistory.push({
      type,
      amount,
      timestamp: new Date(),
      balance: balance
    });
  }
})();

```

```
});
```

```
// Ограничиваем историю 100 последними транзакциями
```

```
if (transactionHistory.length > 100) {  
  transactionHistory = transactionHistory.slice(-100);  
}  
}
```

```
// ПУБЛИЧНЫЙ ИНТЕРФЕЙС
```

```
return {  
  deposit: function(amount) {  
    if (amount <= 0) {  
      throw new Error('Сумма должна быть положительной');  
    }  
  
    balance += amount;  
    logTransaction('DEPOSIT', amount);  
    return balance;  
  },  
  
  withdraw: function(amount) {  
    if (amount <= 0) {  
      throw new Error('Сумма должна быть положительной');  
    }  
  
    if (amount > balance) {  
      throw new Error('Недостаточно средств');  
    }  
  
    balance -= amount;  
    logTransaction('WITHDRAW', amount);  
    return balance;  
  },  
  
  getBalance: function() {  
    return balance;  
  },  
  
  getStatement: function(limit = 10) {  
    return transactionHistory  
      .slice(-limit)  
      .map(t => `${t.timestamp.toISOString()} - ${t.type}: ${t.amount}`);  
  },  
}
```

```

// Только для тестирования
_reset: function() {
  balance = 0;
  transactionHistory = [];
}
};
})();

// Использование
console.log('Начальный баланс:', bankAccount.getBalance()); // 0
bankAccount.deposit(1000);
console.log('После депозита:', bankAccount.getBalance()); // 1000
bankAccount.withdraw(300);
console.log('После снятия:', bankAccount.getBalance()); // 700
console.log('Выписка:', bankAccount.getStatement());

// Приватные переменные недоступны
console.log(bankAccount.balance); // undefined
console.log(bankAccount.transactionHistory); // undefined

```

3.3. Паттерны проектирования с IIFE

javascript

```

// 1. МОДУЛЬНЫЙ ПАТТЕРН (Module Pattern)
const UserModule = (function() {
  // Приватное состояние
  const users = new Map();
  let nextId = 1;

  // Приватные вспомогательные функции
  function validateUser(user) {
    return user &&
      typeof user.name === 'string' &&
      user.name.length >= 2 &&
      typeof user.age === 'number' &&
      user.age >= 0;
  }

  function generateId() {
    return `user_${nextId++}`;
  }

  // Публичное API

```

```
return {
  create: function(userData) {
    if (!validateUser(userData)) {
      throw new Error('Некорректные данные пользователя');
    }

    const id = generateId();
    const user = {
      id,
      ...userData,
      createdAt: new Date(),
      updatedAt: new Date()
    };

    users.set(id, user);
    return { ...user }; // Возвращаем копию
  },

  get: function(id) {
    const user = users.get(id);
    return user ? { ...user } : null;
  },

  update: function(id, updates) {
    const user = users.get(id);
    if (!user) {
      throw new Error('Пользователь не найден');
    }

    const updatedUser = {
      ...user,
      ...updates,
      updatedAt: new Date()
    };

    if (!validateUser(updatedUser)) {
      throw new Error('Некорректные данные для обновления');
    }

    users.set(id, updatedUser);
    return { ...updatedUser };
  },

  delete: function(id) {
```

```

    return users.delete(id);
  },

  getAll: function() {
    return Array.from(users.values()).map(u => ({ ...u }));
  },

  count: function() {
    return users.size;
  },

  // Метод только для тестов
  _clear: function() {
    users.clear();
    nextId = 1;
  }
};
})();

```

// 2. СИНГЛТОН ПАТТЕРН (Singleton Pattern)

```

const AppConfig = (function() {
  let instance = null;

  function createConfig() {
    // Приватный конструктор
    const config = {
      environment: process.env.NODE_ENV || 'development',
      apiUrl: null,
      debug: false,
      version: '1.0.0'
    };
  }

```

// Инициализация в зависимости от окружения

```

  if (config.environment === 'production') {
    config.apiUrl = 'https://api.production.com';
    config.debug = false;
  } else {
    config.apiUrl = 'http://localhost:3000';
    config.debug = true;
  }

```

// Методы

```

  config.get = function(key) {
    return config[key];
  }

```

```

};

config.set = function(key, value) {
  if (key in config) {
    config[key] = value;
    return true;
  }
  return false;
};

config.toJSON = function() {
  const { get, set, toJSON, ...safeConfig } = config;
  return safeConfig;
};

return Object.freeze(config); // Замораживаем объект
}

return {
  getInstance: function() {
    if (!instance) {
      instance = createConfig();
    }
    return instance;
  }
};
})();

// Всегда получаем один и тот же экземпляр
const config1 = AppConfig.getInstance();
const config2 = AppConfig.getInstance();
console.log(config1 === config2); // true

// 3. ФАБРИЧНЫЙ ПАТТЕРН (Factory Pattern)
const LoggerFactory = (function() {
  const loggers = new Map();
  const defaultLevels = ['error', 'warn', 'info', 'debug'];

  function createLogger(name, options = {}) {
    const level = options.level || 'info';
    const format = options.format || 'text';
    const transport = options.transport || 'console';

    const logger = {

```

```

name,
level,
format,
transport,

log: function(level, message, meta = {}) {
  if (defaultLevels.indexOf(level) > defaultLevels.indexOf(this.level)) {
    return; // Не логируем, если уровень ниже установленного
  }

  const entry = {
    timestamp: new Date().toISOString(),
    level,
    logger: this.name,
    message,
    ...meta
  };

  // Разные транспорты
  if (this.transport === 'console') {
    const colors = {
      error: '\x1b[31m', // красный
      warn: '\x1b[33m', // желтый
      info: '\x1b[32m', // зеленый
      debug: '\x1b[36m' // голубой
    };

    console.log(
      `${colors[level] || ''}[${entry.timestamp}] ${entry.level.toUpperCase()} ${
entry.message}\x1b[0m`
    );
  }

  // Можно добавить другие транспорты (файл, база данных, etc.)
},

// Сахарные методы
error: function(msg, meta) {
  this.log('error', msg, meta);
},

warn: function(msg, meta) {
  this.log('warn', msg, meta);
},

```



```
info: function(msg, meta) {
  this.log('info', msg, meta);
},

debug: function(msg, meta) {
  this.log('debug', msg, meta);
}
};

// Привязываем методы к Logger
logger.error = logger.error.bind(logger);
logger.warn = logger.warn.bind(logger);
logger.info = logger.info.bind(logger);
logger.debug = logger.debug.bind(logger);

return logger;
}

return {
  getLogger: function(name, options) {
    if (!loggers.has(name)) {
      loggers.set(name, createLogger(name, options));
    }
    return loggers.get(name);
  },

  // Для тестов
  _clear: function() {
    loggers.clear();
  }
};
})();

// Использование фабрики
const appLogger = LoggerFactory.getLogger('app', { level: 'debug' });
const dbLogger = LoggerFactory.getLogger('database', { level: 'warn' });

appLogger.info('Приложение запущено');
appLogger.debug('Отладка соединения');
dbLogger.warn('Медленный запрос к базе данных');
```

Часть 4: Стрелочные функции и современные альтернативы

4.1. Стрелочные функции как анонимные функции

javascript

// Эволюция анонимных функций:

// 1. Классическая анонимная функция (ES5)

```
const oldStyle = function(x) {  
  return x * x;  
};
```

// 2. Стрелочная функция (ES6+)

```
const arrowStyle = x => x * x;
```

// 3. Использование в колбэках

```
const numbers = [1, 2, 3, 4, 5];
```

// ES5 стиль

```
const doubledES5 = numbers.map(function(num) {  
  return num * 2;  
});
```

// ES6+ стиль

```
const doubledES6 = numbers.map(num => num * 2);
```

// 4. Особенности стрелочных функций

```
const object = {  
  value: 42,
```

// Обычная функция - свой контекст this

```
regularFunc: function() {  
  console.log('regular:', this.value); // 42  
},
```

// Стрелочная функция - нет своего this (берет из внешней области)

```
arrowFunc: () => {  
  console.log('arrow:', this.value); // undefined (или window.value)  
},
```

```

// Полезное использование стрелочных функций
delayedLog: function() {
  // Проблема с обычной функцией
  setTimeout(function() {
    console.log('Проблема:', this.value); // undefined (this = window)
  }, 100);

  // Решение 1: сохранение this
  const self = this;
  setTimeout(function() {
    console.log('Решение 1:', self.value); // 42
  }, 200);

  // Решение 2: bind
  setTimeout(function() {
    console.log('Решение 2:', this.value); // 42
  }.bind(this), 300);

  // Решение 3: стрелочная функция (лучшее)
  setTimeout(() => {
    console.log('Решение 3:', this.value); // 42
  }, 400);
}

object.regularFunc();
object.arrowFunc();
object.delayedLog();

```

4.2. Современные альтернативы IIFE

javascript

```

// С появлением ES6 модулей и блочной области видимости
// необходимость в IIFE уменьшилась, но они все еще полезны

// 1. БЛОКИ КОДА с let/const (замена некоторых IIFE)
{
  // Блочная область видимости
  let privateVar = 'секретно';
  const helper = () => {
    console.log(privateVar);
  };
}

```

```

    helper(); // "секретно"
}

// console.log(privateVar); // ОШИБКА: privateVar is not defined
// console.log(helper); // ОШИБКА: helper is not defined

// 2. ES6 МОДУЛИ (лучшая замена модульного паттерна)
// math.js
export const PI = 3.14159;

export function circleArea(radius) {
    return PI * radius * radius;
}

function privateHelper() {
    // Приватная функция (не экспортируется)
}

// app.js
import { PI, circleArea } from './math.js';

console.log(circleArea(5)); // 78.53975
// console.log(privateHelper()); // ОШИБКА: privateHelper is not defined

// 3. КЛАССЫ с приватными полями (ES2022)
class SecureCounter {
    // Приватное поле
    #count = 0;

    // Приватный метод
    #validateIncrement(value) {
        return typeof value === 'number' && value > 0;
    }

    increment(value = 1) {
        if (!this.#validateIncrement(value)) {
            throw new Error('Некорректное значение');
        }

        this.#count += value;
        return this.#count;
    }

    getCount() {

```

```

    return this.#count;
  }
}

const counter = new SecureCounter();
counter.increment(5);
console.log(counter.getCount()); // 5
// console.log(counter.#count); // SyntaxError: Private field must be declared
// console.log(counter.#validateIncrement); // SyntaxError

// 4. Где IIFE все еще актуальны:
// - Легаси код
// - Скрипты, которые должны работать везде (без сборки)
// - Быстрое прототипирование
// - Изоляция в консоли браузера

```

4.3. Асинхронные IIFE

javascript

```

// Современные асинхронные IIFE
(async () => {
  console.log('Начало асинхронного IIFE');

  try {
    // Загрузка конфигурации
    const config = await fetch('/api/config')
      .then(r => r.json());

    console.log('Конфигурация загружена:', config);

    // Инициализация приложения с конфигом
    const app = await initializeApp(config);

    // Запуск
    await app.start();

    console.log('Приложение запущено успешно');

  } catch (error) {
    console.error('Ошибка инициализации:', error);

    // Откат к дефолтным настройкам
    const defaultConfig = getDefaultConfig();
  }
})();

```

```

const fallbackApp = await initializeApp(defaultConfig);
await fallbackApp.start();

console.log('Запущено с дефолтными настройками');
}

console.log('Асинхронный IIFE завершен');
})();

// Вспомогательные функции
async function initializeApp(config) {
  return {
    start: async () => {
      console.log('Приложение стартует с config:', config);
      await new Promise(resolve => setTimeout(resolve, 500));
      return { status: 'running' };
    }
  };
}

function getDefaultConfig() {
  return { mode: 'default', debug: true };
}

// IIFE с top-level await (ES2022)
// В модулях можно использовать await на верхнем уровне
const dynamicConfig = await fetch('/api/config').then(r => r.json());
console.log('Динамическая конфигурация:', dynamicConfig);

```

Часть 5: Продвинутые техники и паттерны

5.1. Каррирование с анонимными функциями

javascript

```

// Каррирование через анонимные функции
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
  };
}

```

```

    } else {
      return function(...args2) {
        return curried.apply(this, args.concat(args2));
      };
    }
  };
}

```

// Практический пример: создание логгера

```

const createLogger = curry(function(level, component, message) {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${level.toUpperCase()} [${component}] ${message}`);
});

```

// Частичное применение

```

const appInfo = createLogger('info', 'app');
const appError = createLogger('error', 'app');
const dbDebug = createLogger('debug', 'database');

```

// Использование

```

appInfo('Приложение запущено');
appError('Критическая ошибка!');
dbDebug('Запрос выполнен за 150ms');

```

// Автоматическое каррирование всех функций

```

function autoCurry(fn) {
  const curried = (...args) => {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...moreArgs) => curried(...args, ...moreArgs);
  };
  return curried;
}

```

// Пример использования

```

const add = autoCurry((a, b, c) => a + b + c);
const add5 = add(5);
const add5And10 = add5(10);

```

```

console.log(add5And10(15)); // 30 (5 + 10 + 15)
console.log(add(1)(2)(3)); // 6

```

5.2. Мемоизация с IIFE

javascript

```
// Создание мемоизированной функции через IIFE
const memoizedFibonacci = (function() {
  const cache = new Map();

  function fibonacci(n) {
    if (n <= 1) return n;

    // Проверяем кэш
    if (cache.has(n)) {
      console.log(`Возвращаем из кэша: fib(${n})`);
      return cache.get(n);
    }

    console.log(`Вычисляем: fib(${n})`);
    const result = fibonacci(n - 1) + fibonacci(n - 2);
    cache.set(n, result);

    return result;
  }

  return fibonacci;
})();

console.log(memoizedFibonacci(10));
// Множество вычислений для fib(10)
// Но при повторном вызове - мгновенный результат из кэша
console.log(memoizedFibonacci(10)); // Быстро из кэша

// Универсальная функция мемоизации
const memoize = (function() {
  const globalCache = new WeakMap(); // WeakMap для очистки мусора

  return function(fn) {
    // Создаем кэш для конкретной функции
    const cache = new Map();

    return function(...args) {
      // Создаем ключ из аргументов
      const key = JSON.stringify(args);
```



```

    if (cache.has(key)) {
      console.log('Кэш попадание для', fn.name || 'анонимной функции');
      return cache.get(key);
    }

    console.log('Кэш промах, вычисляем...');
    const result = fn.apply(this, args);
    cache.set(key, result);

    return result;
  };
};
})();

// Пример использования
const expensiveCalculation = memoize(function(a, b) {
  // Имитация тяжелых вычислений
  let result = 0;
  for (let i = 0; i < 10000000; i++) {
    result += Math.sin(a) * Math.cos(b);
  }
  return result;
});

console.log(expensiveCalculation(1, 2)); // Долго вычисляет
console.log(expensiveCalculation(1, 2)); // Быстро из кэша
console.log(expensiveCalculation(2, 3)); // Снова вычисляет (другие аргументы)

```

5.3. Паттерн "Модуль с зависимостями"

javascript

```

// Модуль, который принимает зависимости через IIFE
const MyApp = (function($, _, axios) {
  // Приватное состояние
  let state = {
    user: null,
    isLoading: false,
    data: []
  };

  // Приватные функции с использованием зависимостей
  function fetchData() {
    state.isLoading = true;

```

```

emitEvent('loading:start');

return axios.get('/api/data')
  .then(response => {
    state.data = response.data;
    emitEvent('data:loaded', state.data);
  })
  .catch(error => {
    emitEvent('error', error);
  })
  .finally(() => {
    state.isLoading = false;
    emitEvent('loading:end');
  });
}

function renderUI() {
  if (!state.user) {
    $('#login-form').show();
    $('#dashboard').hide();
  } else {
    $('#login-form').hide();
    $('#dashboard').show();
  }

  // Используем Lodash для манипуляций с данными
  const processedData = _.chain(state.data)
    .filter(item => item.active)
    .sortBy('date')
    .value();

  updateDashboard(processedData);
}

function updateDashboard(data) {
  const html = data.map(item =>
    `
```

```
function emitEvent(eventName, data) {
  const handlers = eventHandlers.get(eventName) || [];
  handlers.forEach(handler => handler(data));
}
```

// Публичный API

```
return {
  init: function(user) {
    state.user = user;
    renderUI();
    return fetchData();
  },

  logout: function() {
    state.user = null;
    renderUI();
  },

  getState: function() {
    return { ...state }; // Возвращаем копию
  },

  on: function(eventName, handler) {
    if (!eventHandlers.has(eventName)) {
      eventHandlers.set(eventName, []);
    }
    eventHandlers.get(eventName).push(handler);
  },

  off: function(eventName, handler) {
    const handlers = eventHandlers.get(eventName);
    if (handlers) {
      const index = handlers.indexOf(handler);
      if (index > -1) {
        handlers.splice(index, 1);
      }
    }
  };
})(jQuery, _, axios); // Зависимости 注入 ятся здесь
```

// Использование

```
MyApp.on('data:loaded', function(data) {
```

```
console.log('Данные загружены:', data.length, 'элементов');
});
```

```
MyApp.init({ id: 1, name: 'Анна' })
  .then(() => console.log('Приложение инициализировано'));
```

Часть 6: Оптимизация, отладка и лучшие практики

6.1. Отладка анонимных функций

javascript

// ПРОБЛЕМА: анонимные функции сложно отлаживать

```
function problematicFunction() {
  const data = [1, 2, 3];

  data.map(function(item) { // Анонимная функция
    // В стеке вызовов будет "anonymous"
    if (item === 2) {
      throw new Error('Проблема на элементе 2');
    }
    return item * 2;
  });
}

try {
  problematicFunction();
} catch (error) {
  console.error('Стек вызовов с анонимной функцией:');
  console.error(error.stack);
  // Будет содержать: "at anonymous"
}
```

// РЕШЕНИЕ 1: Использовать именованные функциональные выражения

```
function betterFunction() {
  const data = [1, 2, 3];

  data.map(function processItem(item) { // Именованная функция
    if (item === 2) {
      throw new Error('Проблема на элементе 2');
    }
  });
}
```

```

    }
    return item * 2;
  });
}

try {
  betterFunction();
} catch (error) {
  console.error('\nСтек вызовов с именованной функцией:');
  console.error(error.stack);
  // Будет содержать: "at processItem"
}

// РЕШЕНИЕ 2: Добавлять имена через свойства (для стрелочных функций)
const arrowFunc = () => {
  console.log(arrowFunc.name); // "arrowFunc" (ES6+ дает имя)
};

// Для присвоенных функций можно добавить displayName
const complexHandler = function(data) {
  // Сложная обработка
};
complexHandler.displayName = 'complexDataHandler';

// РЕШЕНИЕ 3: Использовать source maps в production
// В сборке с помощью Webpack/Rollup имена сохраняются в source maps

// РЕШЕНИЕ 4: Логирование с контекстом
function createDebugLogger(context) {
  return function(...args) {
    console.log(`[${context}]`, ...args);
  };
}

const appLog = createDebugLogger('App');
const dbLog = createDebugLogger('Database');

appLog('Запуск приложения');
dbLog('Подключение к базе данных');
```

6.2. Производительность IIFE

javascript

// БЕНЧМАРКИНГ различных видов IIFE

```
function benchmark() {  
  const iterations = 1000000;  
  
  console.log('Бенчмарк IIFE (1,000,000 итераций):\n');
```

// 1. Классический IIFE

```
console.time('Классический IIFE');  
for (let i = 0; i < iterations; i++) {  
  (function(x) {  
    return x * x;  
  })(i);  
}  
console.timeEnd('Классический IIFE');
```

// 2. IIFE с оператором

```
console.time('IIFE с ! оператором');  
for (let i = 0; i < iterations; i++) {  
  !function(x) {  
    return x * x;  
  }(i);  
}  
console.timeEnd('IIFE с ! оператором');
```

// 3. IIFE с void

```
console.time('IIFE с void');  
for (let i = 0; i < iterations; i++) {  
  void function(x) {  
    return x * x;  
  }(i);  
}  
console.timeEnd('IIFE с void');
```

// 4. Стрелочный IIFE

```
console.time('Стрелочный IIFE');  
for (let i = 0; i < iterations; i++) {  
  ((x) => x * x)(i);  
}  
console.timeEnd('Стрелочный IIFE');
```

// 5. Обычная функция (для сравнения)

```
console.time('Обычная функция');  
function square(x) { return x * x; }
```

```

for (let i = 0; i < iterations; i++) {
  square(i);
}
console.timeEnd('Обычная функция');

console.log('\nВывод: разница минимальна в современных движках');
console.log('Читаемость важнее микро-оптимизаций');
}

// benchmark(); // Раскомментировать для запуска

// ОПТИМИЗАЦИЯ ПАМЯТИ:
// IIFE создают новое замыкание каждый раз
function memoryIntensive() {
  const data = [];

  for (let i = 0; i < 1000; i++) {
    // Каждый IIFE создает новое замыкание
    (function(index) {
      data.push(function() {
        return index; // Замыкает index
      });
    })(i);
  }

  return data; // 1000 функций с 1000 разными замыканиями
}

// Более эффективно с общим замыканием
function memoryEfficient() {
  const data = [];

  function createFunction(index) {
    return function() {
      return index;
    };
  }

  for (let i = 0; i < 1000; i++) {
    data.push(createFunction(i)); // Одно замыкание переиспользуется
  }

  return data;
}

```

6.3. Лучшие практики для production кода

javascript

// РЕКОМЕНДАЦИИ ДЛЯ PRODUCTION:

// 1. ИЗБЕГАЙТЕ АНОНИМНЫХ ФУНКЦИЙ В КРИТИЧЕСКИХ МЕСТАХ

// Плохо:

```
app.on('error', function(err) { // Анонимная - сложно отладить
  logger.error('Ошибка', err);
});
```

// Хорошо:

```
function handleError(err) { // Именованная - понятный стек вызовов
  logger.error('Ошибка приложения', err);
}
app.on('error', handleError);
```

// Еще лучше (с контекстом):

```
app.on('error', function handleError(err) { // Именованное выражение
  logger.error('Ошибка приложения', err);
});
```

// 2. ИСПОЛЬЗУЙТЕ IIFE ТОЛЬКО КОГДА ДЕЙСТВИТЕЛЬНО НУЖНО

// Избыточно:

```
(function() {
  const x = 10;
  console.log(x);
})();
```

// Достаточно (с ES6):

```
{
  const x = 10;
  console.log(x);
}
```

// 3. ДОКУМЕНТИРУЙТЕ СЛОЖНЫЕ IIFE

*/***

** Модуль управления пользователями*

** Использует IIFE для инкапсуляции состояния*

** и защиты от конфликтов имен в глобальной области*

**/*

```
const UserManager = (function() {
```

// Приватное состояние


```

const users = new Map();

// ... реализация

return {
  // Публичный API
};
})();

// 4. ИСПОЛЬЗУЙТЕ СОВРЕМЕННЫЙ СИНТАКСИС С ES6+
// Старый стиль:
var oldModule = (function() {
  var privateVar = 'secret';

  return {
    getSecret: function() {
      return privateVar;
    }
  };
})();

// Современный стиль:
const modernModule = (() => {
  const privateVar = 'secret';

  return {
    getSecret: () => privateVar,

    // Method shorthand
    getInfo() {
      return `Секрет: ${privateVar.substr(0, 3)}...`;
    }
  };
})();

// 5. ОБРАБАТЫВАЙТЕ ОШИБКИ В IIFE
const SafeModule = (function() {
  try {
    // Код, который может выбросить ошибку при инициализации
    const config = JSON.parse(localStorage.getItem('config') || '{}');

    return {
      getConfig: () => ({ ...config }),
      saveConfig: (newConfig) => {

```

```

    localStorage.setItem('config', JSON.stringify(newConfig));
  }
};
} catch (error) {
  console.error('Ошибка инициализации модуля:', error);

  // Возвращаем безопасный fallback
  return {
    getConfig: () => ({}),
    saveConfig: () => {
      console.warn('Конфигурация недоступна');
    }
  };
}
})();

// 6. ТЕСТИРУЕМОСТЬ IIFE
// Проблема: приватные функции нельзя протестировать
const UntestableModule = (function() {
  function privateHelper() { // Не тестируется напрямую
    return 'секретная логика';
  }

  return {
    publicMethod: function() {
      return privateHelper() + ' обработана';
    }
  };
})();

// Решение: выносить логику, которую нужно тестировать
function testableHelper() { // Можно импортировать и тестировать
  return 'логика';
}

const TestableModule = (function(helper) {
  return {
    publicMethod: function() {
      return helper() + ' обработана';
    }
  };
})(testableHelper);

// Теперь testableHelper можно тестировать отдельно

```

6.4. Безопасность и IIFE

javascript

// ЗАЩИТА ОТ НЕДОВЕРЕННОГО КОДА С ПОМОЩЬЮ IIFE

// 1. Защита от переопределения глобальных переменных

```
(function(global) {  
  // Сохраняем оригинальные значения  
  const originalConsole = { ...global.console };  
  const originalAlert = global.alert;  
  
  // "Песочница" для ненадежного кода  
  function runUntrustedCode(code) {  
    try {  
      // Ограничиваем доступ  
      global.console = {  
        log: function(...args) {  
          originalConsole.log('[Ненадежный код]:', ...args);  
        },  
        error: function(...args) {  
          originalConsole.error('[Ненадежный код ошибка]:', ...args);  
        }  
      };  
      global.alert = function() {  
        throw new Error('alert не разрешен в ненадежном коде');  
      };  
  
      // Выполняем код  
      return eval(code);  
    } finally {  
      // Восстанавливаем оригинальные значения  
      global.console = originalConsole;  
      global.alert = originalAlert;  
    }  
  }  
  
  // Экспортируем безопасный интерфейс  
  global.Sandbox = { run: runUntrustedCode };  
  
})(typeof window !== 'undefined' ? window : global);
```

// Пример использования

```
try {
  Sandbox.run(`
    console.log('Привет из песочницы');
    console.log('Я не могу вызвать alert()');
    // alert('Это вызовет ошибку'); // Ошибка!
    42 // Возвращаемое значение
  `);
} catch (error) {
  console.error('Ошибка в ненадежном коде:', error.message);
}
```

// 2. Изоляция сторонних библиотек

```
const ThirdPartyWrapper = (function() {
  // Предположим, что ThirdPartyLib - сторонняя библиотека
  // которая может конфликтовать с нашим кодом

  let thirdPartyInstance = null;

  return {
    initialize: function(options) {
      if (thirdPartyInstance) {
        return thirdPartyInstance;
      }

      // Запускаем в изолированном контексте
      thirdPartyInstance = (function() {
        // Локальные переменные, не конфликтующие с глобальными
        const $ = {}; // Наш локальный $, не jQuery
        const _ = {}; // Наш локальный _, не Lodash

        // Инициализируем стороннюю библиотеку
        // Она будет использовать наши локальные переменные
        const lib = ThirdPartyLib(options);

        // Возвращаем безопасный интерфейс
        return {
          doSomething: function() {
            return lib.publicMethod();
          },

          cleanup: function() {
            // Корректная очистка ресурсов
            if (lib.cleanup) {
```

```

        lib.cleanup();
    }
}
};
})();

return thirdPartyInstance;
},

destroy: function() {
    if (thirdPartyInstance) {
        thirdPartyInstance.cleanup();
        thirdPartyInstance = null;
    }
}
};
})();

// 3. Защита приватных данных
const SecureDataStorage = (function() {
    // Ключ шифрования никогда не покидает IIFE
    const ENCRYPTION_KEY = (function() {
        // Генерируем ключ при инициализации
        const array = new Uint8Array(32);
        crypto.getRandomValues(array);
        return array;
    })();

    // Приватная функция шифрования
    async function encrypt(data) {
        const encoder = new TextEncoder();
        const encoded = encoder.encode(JSON.stringify(data));

        const iv = crypto.getRandomValues(new Uint8Array(12));
        const key = await crypto.subtle.importKey(
            'raw', ENCRYPTION_KEY, 'AES-GCM', false, ['encrypt']
        );

        const encrypted = await crypto.subtle.encrypt(
            { name: 'AES-GCM', iv },
            key,
            encoded
        );
    }

```

```

    return { iv, encrypted };
}

// Приватная функция дешифрования
async function decrypt(encryptedData) {
    const key = await crypto.subtle.importKey(
        'raw', ENCRYPTION_KEY, 'AES-GCM', false, ['decrypt']
    );

    const decrypted = await crypto.subtle.decrypt(
        { name: 'AES-GCM', iv: encryptedData.iv },
        key,
        encryptedData.encrypted
    );

    const decoder = new TextDecoder();
    return JSON.parse(decoder.decode(decrypted));
}

// Публичный интерфейс
return {
    save: async function(key, data) {
        const encrypted = await encrypt(data);
        localStorage.setItem(key, JSON.stringify({
            iv: Array.from(encrypted.iv),
            encrypted: Array.from(new Uint8Array(encrypted.encrypted))
        }));
    },

    load: async function(key) {
        const item = localStorage.getItem(key);
        if (!item) return null;

        const parsed = JSON.parse(item);
        const encryptedData = {
            iv: new Uint8Array(parsed.iv),
            encrypted: new Uint8Array(parsed.encrypted)
        };

        return await decrypt(encryptedData);
    }
};
})();

```

```
// Ключ шифрования никогда не доступен извне
// console.Log(SecureDataStorage.ENCRYPTION_KEY); // undefined
```

Заключение: Мудрое использование анонимных функций и IIFE

Ключевые выводы:

1. Анонимные функции идеальны для:

- ➔ Коротких колбэков и обработчиков
- ➔ Функций, используемых только один раз
- ➔ Стрелочных функций для сохранения контекста `this`
- ➔ Замыканий и фабрик функций

2. IIFE необходимы для:

- ➔ Изоляции кода в глобальной области видимости
- ➔ Создания приватного состояния (до ES6 модулей)
- ➔ Инкапсуляции библиотек и плагинов
- ➔ Немедленного выполнения асинхронного кода

3. Современные альтернативы:

- ➔ ES6 модули вместо модульного паттерна
- ➔ Блочная область видимости (`let/const`) вместо IIFE для изоляции
- ➔ Классы с приватными полями для инкапсуляции
- ➔ Именованные функции для лучшей отладки

Правила хорошего тона:

javascript

// ХОРОШО:

// 1. Именуйте функции, которые могут вызывать ошибки

```
button.addEventListener('click', function handleClick() {
  // Ясное имя для отладки
});
```

// 2. Используйте IIFE только когда действительно нужно изолировать код

```
const Module = (function() {
  // Действительно приватное состояние
  const secret = 'data';
```

```

    return {
      get: () => secret
    };
  })();

// 3. Стрелочные функции для коротких колбэков
numbers.map(x => x * 2);

// 4. Именованные выражения для сложной логики
data.sort(function compareByDate(a, b) {
  return new Date(a.date) - new Date(b.date);
});

// ПЛОХО:

// 1. Анонимные функции везде (сложно отлаживать)
setTimeout(function() {
  /* 50 строк кода */
}, 1000);

// 2. IIFE для всего подряд (избыточно)
(function() {
  const x = 10; // Можно просто использовать блок {}
  console.log(x);
})();

// 3. Слишком вложенные анонимные функции (callback hell)
getData(function(data) {
  processData(data, function(result) {
    saveResult(result, function() {
      // ... и так далее
    });
  });
});

// 4. IIFE без обработки ошибок
(function() {
  riskyOperation(); // Если выбросит ошибку, может сломать весь скрипт
})();

```

Идеальный баланс:

javascript

// СОВРЕМЕННЫЙ ПОДХОД С ИСПОЛЬЗОВАНИЕМ ЛУЧШИХ ПРАКТИК

// 1. Основные функции - именованные declaration

```
function initializeApplication(config) {  
  validateConfig(config);  
  setupServices(config);  
  startApp();  
}
```

// 2. Вспомогательные функции - именованные expressions

```
const validateConfig = function validateConfigFn(config) {  
  if (!config.apiKey) {  
    throw new Error('API key required');  
  }  
  return true;  
};
```

// 3. Колбэки - стрелочные или именованные в зависимости от сложности

```
fetch('/api/data')  
  .then(response => response.json()) // Простой - стрелочная  
  .then(function processData(data) { // Сложный - именованная  
    const filtered = data.filter(item => item.active);  
    return transformData(filtered);  
  });
```

// 4. Изоляция - только когда нужно

```
const AnalyticsModule = (() => {  
  // Действительно приватные данные  
  const trackingId = 'UA-XXXXX-Y';  
  const events = [];
```

// Возвращаем минимальный необходимый интерфейс

```
return {  
  track: (event, data) => {  
    events.push({ event, data, timestamp: Date.now() });  
    console.log(`Tracked: ${event}`, data);  
  },  
  
  getEvents: () => [...events] // Копия для безопасности  
};  
})();
```

// 5. Асинхронная изоляция при необходимости

```
(async () => {
```

```

try {
  const config = await loadConfig();
  await initializeWithConfig(config);
} catch (error) {
  console.error('Failed to initialize:', error);
  await initializeWithDefaults();
}
})();

// 6. Всегда обработка ошибок в IIFE
const SafeModule = (function() {
  try {
    // Основная логика
    return { success: true, data: 'loaded' };
  } catch (error) {
    // Graceful degradation
    console.error('Module failed:', error);
    return { success: false, error: error.message };
  }
})();

```

Философия: Анонимные функции и IIFE — это инструменты, а не dogma. Используйте их там, где они действительно приносят пользу: для изоляции, инкапсуляции и создания чистого API. Но помните о современных альтернативах и всегда отдавайте предпочтение читаемости и поддерживаемости кода.

Введение: Самая мощная концепция JavaScript

Замыкания (closures) — это, пожалуй, **самая важная и мощная** концепция в JavaScript. Понимание замыканий — это то, что отличает начинающего JavaScript-разработчика от опытного. Замыкания позволяют создавать элегантные, эффективные и безопасные паттерны программирования.

Часть 1: Лексическое окружение (Lexical Environment)

1.1. Что такое лексическое окружение

Лексическое окружение — это внутренняя структура JavaScript, которая хранит информацию о переменных, функциях и их связях в определенной области видимости.

```
javascript

// Каждая область видимости создает свое лексическое окружение
function outer() {
  let outerVar = 'Я в outer';

  function inner() {
    let innerVar = 'Я в inner';
    console.log(outerVar); // Имеет доступ к outerVar
  }

  inner();
  // console.log(innerVar); // ОШИБКА: innerVar не доступна здесь
}

outer();

// Визуализация лексических окружений:
/*
```

Глобальное лексическое окружение:

- `outer: function`

Лексическое окружение `outer()`:

- `outerVar: 'Я в outer'`
- `inner: function`
- `[[OuterEnv]]`: Ссылка на глобальное окружение

Лексическое окружение `inner()`:

- `innerVar: 'Я в inner'`
- `[[OuterEnv]]`: Ссылка на окружение `outer()`

`*/`

1.2. Как работает цепочка лексических окружений

`javascript`

// Пример с несколькими уровнями вложенности

```
let globalVar = 'Я глобальная';
```

```
function level1() {  
  let level1Var = 'Я в level1';
```

```
  function level2() {  
    let level2Var = 'Я в level2';
```

```
    function level3() {  
      let level3Var = 'Я в level3';
```

// Имеем доступ ко всем переменным выше по цепочке

```
    console.log(globalVar);    // "Я глобальная"
```

```
    console.log(level1Var);    // "Я в level1"
```

```
    console.log(level2Var);    // "Я в level2"
```

```
    console.log(level3Var);    // "Я в level3"
```

```
  }
```

```
  level3();
```

// console.log(level3Var); // ОШИБКА: нет доступа

```
}
```

```
level2();
```

```
}
```

```
level1();
```

```
// Цепочка поиска переменных:  
// 1. Ищем переменную в текущем лексическом окружении  
// 2. Если не нашли, идем во внешнее окружение ([[OuterEnv]])  
// 3. Повторяем шаг 2, пока не найдем переменную или не дойдем до глобального окружения  
// 4. Если в глобальном окружении нет - ReferenceError
```

1.3. Жизненный цикл лексического окружения

javascript

```
// Лексическое окружение создается и уничтожается  
function createEnvironment() {  
  let tempVar = 'Временная переменная';  
  console.log('Окружение создано');  
  
  // Внутренняя функция  
  function inner() {  
    console.log(tempVar);  
  }  
  
  inner();  
  
  // После выхода из функции окружение МОЖЕТ быть уничтожено  
  // но только если на него нет ссылок  
}  
  
createEnvironment();  
// "Окружение создано"  
// "Временная переменная"  
  
// Окружение уничтожено сборщиком мусора  
  
// Но если есть замыкание...  
function createClosure() {  
  let persistentVar = 'Я сохраняюсь!';  
  
  return function() {  
    console.log(persistentVar);  
  };  
}  
  
const closure = createClosure();  
// Окружение createClosure() НЕ уничтожается,
```

```
// потому что внутренняя функция ссылается на persistentVar
```

```
closure(); // "Я сохраняюсь!"
```

1.4. Различные типы лексических окружений

javascript

```
// 1. Глобальное лексическое окружение
```

```
let globalLet = 'global let';
var globalVar = 'global var';
const globalConst = 'global const';
```

```
// 2. Лексическое окружение функции
```

```
function functionScope() {
  let functionLet = 'function let';
  var functionVar = 'function var';
  const functionConst = 'function const';
```

```
// 3. Блочное лексическое окружение (ES6+)
```

```
if (true) {
  let blockLet = 'block let';
  const blockConst = 'block const';
  var blockVar = 'block var'; // var игнорирует блоки!
```

```
  console.log(blockLet); // "block let"
  console.log(blockConst); // "block const"
  console.log(blockVar); // "block var"
}
```

```
// console.log(blockLet); // ОШИБКА
```

```
// console.log(blockConst); // ОШИБКА
```

```
console.log(blockVar); // "block var" (var "всплывает")
}
```

```
// 4. Лексическое окружение цикла (для let/const)
```

```
for (let i = 0; i < 3; i++) {
  // Каждая итерация создает НОВОЕ лексическое окружение
  setTimeout(function() {
    console.log(`let i = ${i}`); // 0, 1, 2
  }, 100);
}
```

```
for (var j = 0; j < 3; j++) {
```

```
// var использует одно окружение
setTimeout(function() {
  console.log(`var j = ${j}`); // 3, 3, 3
}, 100);
}

// 5. Лексическое окружение try-catch
try {
  let tryVar = 'try variable';
  throw new Error('Тестовая ошибка');
} catch (error) {
  let catchVar = 'catch variable';
  console.log(error.message); // "Тестовая ошибка"
  // console.log(tryVar);      // ОШИБКА: tryVar не доступна
}

// console.log(catchVar);      // ОШИБКА: catchVar не доступна
```

Часть 2: Замыкания (Closures) - основы

2.1. Что такое замыкание

Замыкание — это функция, которая запоминает и имеет доступ к переменным из своей внешней (лексической) области видимости, даже после того, как внешняя функция завершила выполнение.

javascript

```
// Простейший пример замыкания
function createCounter() {
  let count = 0; // Переменная во внешней области видимости

  // Внутренняя функция ЗАМЫКАЕТ переменную count
  return function() {
    count++; // Имеет доступ к count даже после завершения createCounter
    return count;
  };
}

const counter = createCounter();
```

```

// Функция createCounter завершилась,
// но ее лексическое окружение сохраняется,
// потому что внутренняя функция ссылается на count

console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3

// Каждый вызов createCounter создает НОВОЕ замыкание
const counter2 = createCounter();
console.log(counter2()); // 1 (свой собственный count)
console.log(counter()); // 4 (продолжает с того же count)

```

2.2. Как работает замыкание на низком уровне

javascript

```

// Детальное объяснение механизма
function outerFunction(outerParam) {
  let outerVariable = 'внешняя переменная';

  function innerFunction(innerParam) {
    // У этой функции есть доступ к:
    // 1. Своим параметрам (innerParam)
    // 2. Своим локальным переменным
    // 3. Параметрам и переменным внешней функции (outerParam, outerVariable)
    // 4. Глобальным переменным

    let innerVariable = 'внутренняя переменная';

    return {
      outerParam,
      outerVariable,
      innerParam,
      innerVariable
    };
  }

  return innerFunction;
}

const closureInstance = outerFunction('параметр');
const result = closureInstance('внутренний параметр');

```



```

console.log(result);
/*
{
  outerParam: 'параметр',
  outerVariable: 'внушняя переменная',
  innerParam: 'внутренний параметр',
  innerVariable: 'внутренняя переменная'
}
*/

// Что происходит в памяти:
/*
1. При вызове outerFunction('параметр'):
  - Создается лексическое окружение для outerFunction
  - В нем сохраняются: outerParam, outerVariable, innerFunction
  - [[OuterEnv]] ссылается на глобальное окружение

2. Возвращается innerFunction
  - innerFunction сохраняет ссылку на окружение outerFunction

3. outerFunction завершается, НО ее окружение НЕ уничтожается
  - Потому что innerFunction все еще ссылается на него

4. При вызове closureInstance('внутренний параметр'):
  - Создается новое окружение для innerFunction
  - В нем сохраняются: innerParam, innerVariable
  - [[OuterEnv]] ссылается на сохраненное окружение outerFunction

5. innerFunction имеет доступ ко всей цепочке окружений
*/

```

2.3. Практические примеры замыканий

javascript

```

// 1. СОЗДАНИЕ ПРИВАТНЫХ ПЕРЕМЕННЫХ
function createBankAccount(initialBalance) {
  let balance = initialBalance; // Приватная переменная

  return {
    deposit: function(amount) {
      if (amount > 0) {
        balance += amount;
        return balance;
      }
    }
  };
}

```

```

    }
    throw new Error('Сумма должна быть положительной');
  },

  withdraw: function(amount) {
    if (amount > 0 && amount <= balance) {
      balance -= amount;
      return balance;
    }
    throw new Error('Недостаточно средств или неверная сумма');
  },

  getBalance: function() {
    return balance;
  }
};
}

```

```

const account = createBankAccount(1000);
console.log(account.getBalance()); // 1000
account.deposit(500);
console.log(account.getBalance()); // 1500
account.withdraw(300);
console.log(account.getBalance()); // 1200

```

```

// Нет прямого доступа к balance
console.log(account.balance); // undefined

```

// 2. КЭШИРОВАНИЕ РЕЗУЛЬТАТОВ (МЕМОИЗАЦИЯ)

```

function createMemoizedFunction(fn) {
  const cache = new Map(); // Приватный кэш

  return function(...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      console.log('Возвращаем из кэша');
      return cache.get(key);
    }

    console.log('Вычисляем результат');
    const result = fn(...args);
    cache.set(key, result);
  }
}

```

```

    return result;
};
}

// Пример использования
function expensiveCalculation(x, y) {
    // Имитация дорогих вычислений
    let result = 0;
    for (let i = 0; i < 1000000; i++) {
        result += Math.sin(x) * Math.cos(y);
    }
    return result;
}

const memoizedCalc = createMemoizedFunction(expensiveCalculation);

console.log(memoizedCalc(1, 2)); // "Вычисляем результат" (медленно)
console.log(memoizedCalc(1, 2)); // "Возвращаем из кэша" (быстро)
console.log(memoizedCalc(2, 3)); // "Вычисляем результат" (новые аргументы)

// 3. СОЗДАНИЕ КОНФИГУРИРУЕМЫХ ФУНКЦИЙ
function createMultiplier(multiplier) {
    // multiplier "запоминается" в замыкании
    return function(number) {
        return number * multiplier;
    };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);
const quadruple = createMultiplier(4);

console.log(double(5)); // 10
console.log(triple(5)); // 15
console.log(quadruple(5)); // 20

// 4. УПРАВЛЕНИЕ СОСТОЯНИЕМ В UI
function createToggle(initialState = false) {
    let state = initialState;

    return {
        toggle: function() {
            state = !state;
            return state;
        }
    };
}

```

```
    },

    getState: function() {
        return state;
    },

    setState: function(newState) {
        state = newState;
        return state;
    }
};

const toggle = createToggle();
console.log(toggle.getState()); // false
console.log(toggle.toggle());   // true
console.log(toggle.toggle());   // false
console.log(toggle.setState(true)); // true
```

Часть 3: Глубокое погружение в механизм замыканий

3.1. Замыкания и циклы - классическая проблема

javascript

```
// КЛАССИЧЕСКАЯ ПРОБЛЕМА: замыкания в циклах с var
console.log('ПРОБЛЕМА с var:');
for (var i = 0; i < 3; i++) {
    setTimeout(function() {
        console.log(i); // Всегда 3!
    }, 100);
}
// Вывод: 3, 3, 3

// Почему это происходит:
// 1. var имеет функциональную область видимости
// 2. Все setTimeout получают ОДНУ И ТУ ЖЕ переменную i
// 3. К моменту выполнения колбэков цикл уже завершился и i = 3

// РЕШЕНИЕ 1: Использовать IIFE для создания новой области видимости
```

```

console.log('\nРЕШЕНИЕ 1 - IIFE:');
for (var i = 0; i < 3; i++) {
  (function(index) {
    setTimeout(function() {
      console.log(index); // 0, 1, 2
    }, 100);
  })(i);
}
// Каждая IIFE создает новое замыкание с собственным значением index

// РЕШЕНИЕ 2: Использовать let (ES6+)
console.log('\nРЕШЕНИЕ 2 - let:');
for (let i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i); // 0, 1, 2
  }, 100);
}
// let имеет блочную область видимости
// Каждая итерация создает НОВОЕ лексическое окружение

// РЕШЕНИЕ 3: Использовать forEach
console.log('\nРЕШЕНИЕ 3 - forEach:');
[0, 1, 2].forEach(function(i) {
  setTimeout(function() {
    console.log(i); // 0, 1, 2
  }, 100);
});
// forEach создает новое замыкание для каждой итерации

// РЕШЕНИЕ 4: Использовать bind
console.log('\nРЕШЕНИЕ 4 - bind:');
for (var i = 0; i < 3; i++) {
  setTimeout(function(index) {
    console.log(index); // 0, 1, 2
  }.bind(null, i), 100);
}
// bind фиксирует значение i в момент создания функции

```

3.2. Замыкания и асинхронный код

javascript

```

// Замыкания в асинхронном контексте
function fetchUserData(userId) {

```

```
const startTime = Date.now(); // Замыкается во внутренних функциях
```

```
return fetch(`/api/users/${userId}`)  
  .then(function(response) {  
    const fetchTime = Date.now() - startTime;  
    console.log(`Запрос выполнен за ${fetchTime}ms`);  
    return response.json();  
  })  
  .then(function(userData) {  
    const totalTime = Date.now() - startTime;  
    console.log(`Всего выполнено за ${totalTime}ms`);  
    return userData;  
  });  
}
```

```
// Имитация fetch для примера
```

```
global.fetch = function(url) {  
  return Promise.resolve({  
    json: () => Promise.resolve({ id: 1, name: 'John' })  
  });  
};
```

```
fetchUserData(1);
```

```
// Замыкания с async/await
```

```
async function processWithClosure(items) {  
  const results = [];  
  const errors = [];  
  
  for (const item of items) {  
    try {  
      // Каждая итерация создает новое замыкание  
      const result = await processItem(item);  
      results.push(result);  
    } catch (error) {  
      // Замыкание сохраняет контекст ошибки  
      errors.push({ item, error: error.message });  
    }  
  }  
  
  return { results, errors };  
}
```

```
async function processItem(item) {
```

```

// Имитация асинхронной обработки
return new Promise(resolve => {
  setTimeout(() => {
    resolve(`Обработано: ${item}`);
  }, Math.random() * 100);
});
}

// Замыкания в event listeners
function createButtonHandlers(buttons) {
  const clickCounts = new Map(); // Приватное состояние

  buttons.forEach(function(button, index) {
    // Каждое замыкание получает свой index
    button.addEventListener('click', function() {
      const currentCount = clickCounts.get(index) || 0;
      clickCounts.set(index, currentCount + 1);

      console.log(`Кнопка ${index} нажата ${currentCount + 1} раз`);
      this.textContent = `Кликов: ${currentCount + 1}`;
    });
  });

  return {
    getClickCount: function(index) {
      return clickCounts.get(index) || 0;
    },

    resetAll: function() {
      clickCounts.clear();
      buttons.forEach(btn => {
        btn.textContent = 'Нажми меня';
      });
    }
  };
}

```

3.3. Вложенные замыкания и цепочки областей видимости

javascript

```

// Сложный пример с несколькими уровнями замыканий
function createMultiLevelClosure(initial) {
  let level1 = initial;

```

```

function level2Func(level2Param) {
  let level2 = level2Param;

  function level3Func(level3Param) {
    let level3 = level3Param;

    function level4Func(level4Param) {
      let level4 = level4Param;

      // Имеем доступ ко всем уровням
      return {
        getAll: function() {
          return {
            level1, // Из самого внешнего уровня
            level2, // Из level2Func
            level3, // Из level3Func
            level4, // Из level4Func (текущий уровень)
            level4Param // Параметр level4Func
          };
        },

        updateLevel1: function(newValue) {
          level1 = newValue; // Можем изменять внешние переменные!
        },

        // Еще одно замыкание внутри замыкания
        createNested: function() {
          let nested = 'вложенная';

          return function() {
            // Имеет доступ ко ВСЕМ переменным выше
            return { level1, level2, level3, level4, nested };
          };
        }
      };
    };
  };

  return level4Func;
}

return level3Func;
}

```



```

    return level2Func;
}

// Использование
const level2 = createMultiLevelClosure('начальное значение');
const level3 = level2('значение уровня 2');
const level4 = level3('значение уровня 3');
const closure = level4('значение уровня 4');

console.log(closure.getAll());
/*
{
  level1: 'начальное значение',
  level2: 'значение уровня 2',
  level3: 'значение уровня 3',
  level4: 'значение уровня 4',
  level4Param: 'значение уровня 4'
}
*/

// Изменяем значение из самого глубокого замыкания
closure.updateLevel1('изменено изнутри');
const level2_2 = createMultiLevelClosure('новое начальное');
const closure2 = level2_2('л2')('л3')('л4');
console.log(closure2.getAll().level1); // "новое начальное"

// Создаем еще более глубокое замыкание
const nestedClosure = closure.createNested();
console.log(nestedClosure());
/*
{
  level1: 'изменено изнутри',
  level2: 'значение уровня 2',
  level3: 'значение уровня 3',
  level4: 'значение уровня 4',
  nested: 'вложенная'
}
*/

// Визуализация цепочки областей видимости:
/*
Глобальное окружение
├─ createMultiLevelClosure
├─ level2 (функция)

```

```

|   |— Level1: 'начальное значение'
|   |— [[OuterEnv]] → Глобальное
|
|   |— Level3 (функция)
|   |   |— Level2: 'значение уровня 2'
|   |   |— [[OuterEnv]] → окружение Level2
|   |
|   |— Level4 (функция)
|   |   |— Level3: 'значение уровня 3'
|   |   |— [[OuterEnv]] → окружение Level3
|   |
|   |— closure (объект с методами)
|   |   |— Level4: 'значение уровня 4'
|   |   |— [[OuterEnv]] → окружение Level4
|   |   |
|   |   |— nestedClosure (функция)
|   |   |   |— nested: 'вложенная'
|   |   |   |— [[OuterEnv]] → окружение closure
|
*/

```

Часть 4: Продвинутые паттерны с замыканиями

4.1. Фабрики и конструкторы с замыканиями

javascript

// 1. ФАБРИКА ОБЪЕКТОВ С ПРИВАТНЫМ СОСТОЯНИЕМ

```

function createPerson(name, age) {
  // Приватные переменные
  let _name = name;
  let _age = age;
  let _secret = 'мой секрет';

  // Приватные методы
  function validateAge(newAge) {
    return typeof newAge === 'number' && newAge >= 0 && newAge <= 150;
  }

  // Публичный интерфейс
  return {

```

```
// Геттеры
getName: function() {
    return _name;
},

getAge: function() {
    return _age;
},

// Сеттеры
setName: function(newName) {
    if (typeof newName === 'string' && newName.length >= 2) {
        _name = newName;
        return true;
    }
    return false;
},

setAge: function(newAge) {
    if (validateAge(newAge)) {
        _age = newAge;
        return true;
    }
    return false;
},

// Методы
celebrateBirthday: function() {
    _age++;
    return `С днем рождения, ${_name}! Теперь тебе ${_age}.`;
},

// Метод с доступом к приватным данным
getInfo: function(showSecret = false) {
    const info = {
        name: _name,
        age: _age,
        isAdult: _age >= 18
    };

    if (showSecret) {
        info.secret = _secret;
    }
}
```

```

    return info;
  },

  // Статические методы (привязанные к фабрике, не к экземпляру)
  createDefault: function() {
    return createPerson('Гость', 18);
  }
};
}

// Использование
const person = createPerson('Анна', 25);
console.log(person.getName()); // "Анна"
console.log(person.getAge()); // 25
person.setAge(26);
console.log(person.celebrateBirthday()); // "С днем рождения, Анна! Теперь тебе 27."
console.log(person.getInfo()); // {name: "Анна", age: 27, isAdult: true}

// Приватные данные недоступны
console.log(person._name); // undefined
console.log(person._secret); // undefined

// 2. КОНСТРУКТОР С ЗАМЫКАНИЯМИ (альтернатива классам)
function Counter(initialValue = 0) {
  // Приватные переменные
  let count = initialValue;
  let history = [];

  // Приватный метод
  function addToHistory(action, value) {
    history.push({
      action,
      value,
      timestamp: new Date(),
      currentCount: count
    });

    // Ограничиваем историю
    if (history.length > 100) {
      history = history.slice(-100);
    }
  }

  // Публичные методы (добавляются к this)

```

```

this.increment = function(value = 1) {
  count += value;
  addToHistory('increment', value);
  return this; // Для чейнинга
};

this.decrement = function(value = 1) {
  count -= value;
  addToHistory('decrement', value);
  return this;
};

this.getValue = function() {
  return count;
};

this.getHistory = function(limit = 10) {
  return history.slice(-limit);
};

this.reset = function() {
  const oldValue = count;
  count = 0;
  addToHistory('reset', oldValue);
  return this;
};
}

// Использование
const myCounter = new Counter(10);
myCounter.increment(5).decrement(2).increment(1);
console.log(myCounter.getValue()); // 14 (10 + 5 - 2 + 1)
console.log(myCounter.getHistory());

// Приватные переменные недоступны
console.log(myCounter.count); // undefined
console.log(myCounter.history); // undefined

```

4.2. Каррирование и частичное применение

javascript

```

// КАРРИРОВАНИЕ с замыканиями
function curry(fn) {

```

```

return function curried(...args) {
  // Если передано достаточно аргументов
  if (args.length >= fn.length) {
    return fn.apply(this, args);
  } else {
    // Возвращаем новую функцию, которая "запоминает" переданные аргументы
    return function(...args2) {
      return curried.apply(this, args.concat(args2));
    };
  }
};

// Пример использования
function multiplyThree(a, b, c) {
  return a * b * c;
}

const curriedMultiply = curry(multiplyThree);

console.log(curriedMultiply(2)(3)(4)); // 24
console.log(curriedMultiply(2, 3)(4)); // 24
console.log(curriedMultiply(2)(3, 4)); // 24
console.log(curriedMultiply(2, 3, 4)); // 24

// Частичное применение для создания специализированных функций
const double = curriedMultiply(2); // Фиксируем первый аргумент
const doubleAndTriple = double(3); // Фиксируем второй аргумент
console.log(doubleAndTriple(4)); // 24 (2 * 3 * 4)
console.log(doubleAndTriple(5)); // 30 (2 * 3 * 5)

// ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ: создание логгеров
const createLogger = curry(function(level, component, message, meta = {}) {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${level.toUpperCase()} [${component}] ${message}`, meta);
});

// Частичное применение
const appInfo = createLogger('info', 'app');
const appError = createLogger('error', 'app');
const dbDebug = createLogger('debug', 'database');

appInfo('Приложение запущено', { version: '1.0.0' });
appError('Критическая ошибка!', { code: 'ERR_001' });

```

```

dbDebug('Запрос выполнен', { duration: '150ms' });

// АВТОМАТИЧЕСКОЕ КАРРИРОВАНИЕ ВСЕХ ФУНКЦИЙ
function autoCurry(fn) {
  const curried = (...args) => {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...moreArgs) => curried(...args, ...moreArgs);
  };
  return curried;
}

// Создаем каррированные версии всех функций
const add = autoCurry((a, b, c) => a + b + c);
const greet = autoCurry((greeting, name) => `${greeting}, ${name}!`);
const createUser = autoCurry((id, name, email) => ({ id, name, email }));

console.log(add(1)(2)(3)); // 6
console.log(greet('Привет')('Анна')); // "Привет, Анна!"
console.log(createUser(1)('John')('john@example.com'));
// {id: 1, name: "John", email: "john@example.com"}

```

4.3. Состояние и конечные автоматы (State Machines)

javascript

```

// КОНЕЧНЫЙ АВТОМАТ (STATE MACHINE) с замыканиями
function createTrafficLight(initialState = 'red') {
  // Приватное состояние
  let state = initialState;
  let timer = null;

  // Приватные переходы состояний
  const transitions = {
    red: { next: 'green', duration: 3000 },
    green: { next: 'yellow', duration: 2000 },
    yellow: { next: 'red', duration: 1000 }
  };

  // Приватные обработчики
  const handlers = {
    onRed: null,
    onGreen: null,

```

```

    onYellow: null,
    onChange: null
  };

function changeState(newState) {
  const oldState = state;
  state = newState;

  // Вызываем обработчики
  if (handlers.onChange) {
    handlers.onChange(oldState, newState);
  }

  const handlerName = `on${newState.charAt(0).toUpperCase() + newState.slice(1)}`;
  if (handlers[handlerName]) {
    handlers[handlerName]();
  }

  // Планируем следующий переход
  scheduleNextTransition();
}

function scheduleNextTransition() {
  if (timer) {
    clearTimeout(timer);
  }

  const transition = transitions[state];
  if (transition) {
    timer = setTimeout(() => {
      changeState(transition.next);
    }, transition.duration);
  }
}

// Публичный интерфейс
return {
  start: function() {
    console.log(`Светофор запущен в состоянии: ${state}`);
    scheduleNextTransition();
    return this;
  },

  stop: function() {

```



```

    if (timer) {
      clearTimeout(timer);
      timer = null;
    }
    console.log('Светофор остановлен');
    return this;
  },

  getState: function() {
    return state;
  },

  forceChange: function(newState) {
    if (transitions[newState]) {
      changeState(newState);
    } else {
      throw new Error(`Неизвестное состояние: ${newState}`);
    }
    return this;
  },

  // Регистрация обработчиков
  on: function(event, handler) {
    if (handlers.hasOwnProperty(event)) {
      handlers[event] = handler;
    } else {
      throw new Error(`Неизвестное событие: ${event}`);
    }
    return this;
  },

  // Для тестирования
  _getTransitions: function() {
    return { ...transitions };
  }
};

// Использование светофора
const trafficLight = createTrafficLight('red');

trafficLight
  .on('onChange', function(oldState, newState) {
    console.log(`Состояние изменилось: ${oldState} → ${newState}`);
  });

```

```

}))
.on('onRed', function() {
  console.log('🛑 СТОП!');
})
.on('onGreen', function() {
  console.log('🟢 МОЖНО ЕХАТЬ');
})
.on('onYellow', function() {
  console.log('🟡 ВНИМАНИЕ!');
})
.start();

// Через 9 секунд остановим
setTimeout(() => {
  trafficLight.stop();
}, 9000);

// ДРУГОЙ ПРИМЕР: система аутентификации
function createAuthSystem() {
  // Приватное состояние
  let currentUser = null;
  let token = null;
  let refreshToken = null;
  let tokenExpiry = null;

  // Приватные функции
  function isValidToken() {
    return token && tokenExpiry && Date.now() < tokenExpiry;
  }

  async function refreshAuthToken() {
    if (!refreshToken) {
      throw new Error('Нет refresh token');
    }

    // Имитация запроса на обновление токена
    return new Promise(resolve => {
      setTimeout(() => {
        token = `new_token_${Date.now()}`;
        tokenExpiry = Date.now() + 3600000; // +1 час
        resolve(token);
      }, 100);
    });
  }
}

```

// Публичный интерфейс

```
return {
  login: async function(username, password) {
    // Имитация запроса на сервер
    return new Promise((resolve, reject) => {
      setTimeout(() => {
        if (username === 'admin' && password === 'password') {
          currentUser = { username, role: 'admin' };
          token = `token_${Date.now()}`;
          refreshToken = `refresh_${Date.now()}`;
          tokenExpiry = Date.now() + 3600000; // Токен на 1 час

          resolve({
            success: true,
            user: { ...currentUser },
            token
          });
        } else {
          reject(new Error('Неверные учетные данные'));
        }
      }, 200);
    });
  },

  logout: function() {
    currentUser = null;
    token = null;
    refreshToken = null;
    tokenExpiry = null;
    return Promise.resolve(true);
  },

  getCurrentUser: function() {
    return currentUser ? { ...currentUser } : null;
  },

  isAuthenticated: function() {
    return !!currentUser && isTokenValid();
  },

  // Защищенный запрос с автоматическим обновлением токена
  makeAuthenticatedRequest: async function(url, options = {}) {
    if (!this.isAuthenticated()) {
```

```

    throw new Error('Не аутентифицирован');
  }

  // Если токен скоро истечет, обновляем
  if (Date.now() > tokenExpiry - 60000) { // За 1 минуту до истечения
    await refreshAuthToken();
  }

  // Добавляем токен в заголовки
  const headers = {
    ...options.headers,
    'Authorization': `Bearer ${token}`
  };

  // Имитация запроса
  return new Promise(resolve => {
    setTimeout(() => {
      resolve({
        url,
        data: { message: 'Запрос выполнен успешно' },
        timestamp: new Date()
      });
    }, 100);
  });
},

// Для тестирования
_getTokenInfo: function() {
  return {
    hasToken: !!token,
    expiresIn: tokenExpiry ? tokenExpiry - Date.now() : null,
    isValid: isTokenValid()
  };
}
};

// Использование
const auth = createAuthSystem();

async function testAuth() {
  try {
    // Логин
    const result = await auth.login('admin', 'password');
  }
}

```

```
console.log('Вход выполнен:', result.user.username);

// Проверка аутентификации
console.log('Аутентифицирован?:', auth.isAuthenticated());
console.log('Текущий пользователь:', auth.getCurrentUser());

// Защищенный запрос
const apiResult = await auth.makeAuthenticatedRequest('/api/data');
console.log('Результат запроса:', apiResult.data.message);

// Информация о токене
console.log('Информация о токене:', auth._getTokenInfo());

// Выход
await auth.logout();
console.log('После выхода - аутентифицирован?:', auth.isAuthenticated());

} catch (error) {
  console.error('Ошибка:', error.message);
}
}

testAuth();
```

Часть 5: Производительность и оптимизация

5.1. Утечки памяти в замыканиях

javascript

```
// УТЕЧКИ ПАМЯТИ из-за замыканий
function createLeakyClosure() {
  const largeArray = new Array(1000000).fill('data'); // Большой массив
  const largeObject = { data: new Array(1000000).fill('item') };

  return function() {
    // Ссылаемся только на smallVar, но largeArray и largeObject
    // тоже остаются в замыкании!
    const smallVar = 'маленькая переменная';
    console.log(smallVar);
  };
}
```

```

    // Даже если мы не используем LargeArray и LargeObject,
    // они остаются в замыкании и не могут быть собраны GC
  };
}

const leakyFunction = createLeakyClosure();
// 2 MB памяти удерживается замыканием, хотя не используется!

// РЕШЕНИЕ: явно обнулять ссылки
function createOptimizedClosure() {
  const largeArray = new Array(1000000).fill('data');
  const largeObject = { data: new Array(1000000).fill('item') };

  // Используем данные сразу
  const processedData = processLargeData(largeArray, largeObject);

  // ОБНУЛЯЕМ ссылки после использования
  largeArray.length = 0; // Очищаем массив
  Object.keys(largeObject).forEach(key => {
    delete largeObject[key];
  });

  // Теперь возвращаемая функция замыкает только processedData
  return function() {
    console.log('Обработанные данные:', processedData.length);
  };
}

function processLargeData(arr, obj) {
  // Обработка данных
  return arr.slice(0, 100); // Берем только первые 100 элементов
}

// ДРУГОЙ ВАЖНЫЙ ПРИМЕР: обработчики событий
function createEventHandlers() {
  const elements = document.querySelectorAll('.my-elements');
  const bigData = new Array(1000000).fill('data');

  elements.forEach(element => {
    // КАЖДЫЙ обработчик замыкает bigData!
    element.addEventListener('click', function() {
      console.log('Клик!');
    });
    // Хотя bigData не используется, она остается в замыкании
  });
}

```

```

    });
  });

  // После удаления элементов, bigData все еще в памяти
  // из-за замыканий в обработчиках!
}

// РЕШЕНИЕ: использовать weak references или удалять обработчики
function createOptimizedEventHandlers() {
  const elements = document.querySelectorAll('.my-elements');
  const bigData = new Array(1000000).fill('data');

  // Используем данные сразу
  const processedData = processData(bigData);

  // Очищаем bigData после использования
  bigData.length = 0;

  const handlers = [];

  elements.forEach((element, index) => {
    const handler = function() {
      // Замыкает только processedData и index
      console.log(`Клик на элементе ${index}:`, processedData[index]);
    };

    element.addEventListener('click', handler);
    handlers.push({ element, handler });
  });

  // Функция для очистки
  return {
    removeAllHandlers: function() {
      handlers.forEach(({ element, handler }) => {
        element.removeEventListener('click', handler);
      });
      handlers.length = 0; // Очищаем массив
    }
  };
}

```

5.2. Оптимизация производительности

javascript

// БЕНЧМАРКИНГ: замыкания vs классы

```
function benchmark() {  
  const iterations = 1000000;  
  
  console.log('Создание 1,000,000 экземпляров:\n');  
  
  // 1. Замыкание (фабрика)  
  console.time('Фабрика с замыканием');  
  for (let i = 0; i < iterations; i++) {  
    createCounterClosure();  
  }  
  console.timeEnd('Фабрика с замыканием');  
  
  // 2. Класс (ES6)  
  console.time('Класс ES6');  
  for (let i = 0; i < iterations; i++) {  
    new CounterClass();  
  }  
  console.timeEnd('Класс ES6');  
  
  // 3. Конструктор с замыканиями  
  console.time('Конструктор с замыканиями');  
  for (let i = 0; i < iterations; i++) {  
    new CounterConstructor();  
  }  
  console.timeEnd('Конструктор с замыканиями');  
  
  console.log('\nВызов методов 1,000,000 раз:\n');  
  
  const closureCounter = createCounterClosure();  
  const classCounter = new CounterClass();  
  const constructorCounter = new CounterConstructor();  
  
  console.time('Метод замыкания');  
  for (let i = 0; i < iterations; i++) {  
    closureCounter.increment();  
  }  
  console.timeEnd('Метод замыкания');
```



```
console.time('Метод класса');
for (let i = 0; i < iterations; i++) {
  classCounter.increment();
}
console.timeEnd('Метод класса');
```

```
console.time('Метод конструктора');
for (let i = 0; i < iterations; i++) {
  constructorCounter.increment();
}
console.timeEnd('Метод конструктора');
}
```

// Реализации для бенчмарка

```
function createCounterClosure() {
  let count = 0;

  return {
    increment: function() {
      count++;
      return count;
    },
    getValue: function() {
      return count;
    }
  };
}
```

```
class CounterClass {
  constructor() {
    this.count = 0;
  }

  increment() {
    this.count++;
    return this.count;
  }

  getValue() {
    return this.count;
  }
}
```

```
function CounterConstructor() {
```

```
this.count = 0;

this.increment = function() {
  this.count++;
  return this.count;
};

this.getValue = function() {
  return this.count;
};
}

// benchmark(); // Раскомментировать для запуска

// ОПТИМИЗАЦИЯ: кэширование методов
function createOptimizedClosure() {
  let count = 0;

  // Выносим методы за пределы возвращаемого объекта,
  // чтобы не создавать их каждый раз
  function increment() {
    count++;
    return count;
  }

  function getValue() {
    return count;
  }

  return {
    increment,
    getValue
  };
}

// Еще лучше: прототип для повторного использования методов
function UltraOptimizedCounter() {
  let count = 0;

  this.increment = function() {
    count++;
    return count;
  };
}
```

```

this.getValue = function() {
    return count;
};
}

// Использование одного прототипа для всех экземпляров
UltraOptimizedCounter.prototype.reset = function() {
    // Но здесь нет доступа к приватному count!
    // Поэтому этот метод не может работать с приватными переменными
    return 'Метод прототипа';
};

```

5.3. Инструменты для анализа замыканий

javascript

```

// 1. Chrome DevTools: Memory Snapshots
// Можно увидеть, какие переменные удерживаются замыканиями

// 2. Ручная проверка через свойства функции
function createClosureForInspection() {
    const privateData = 'секретные данные';
    const largeArray = new Array(1000).fill('данные');

    return function innerFunction() {
        console.log(privateData);
        // largeArray также удерживается, хотя не используется
    };
}

const inspectMe = createClosureForInspection();

// Можно посмотреть имя функции
console.log(inspectMe.name); // "innerFunction"

// Но нельзя посмотреть закрытые переменные напрямую
console.log(inspectMe.privateData); // undefined

// 3. Отладка через console.dir
console.dir(inspectMe);
// В консоли можно раскрыть [[Scopes]] чтобы увидеть замыкания

// 4. Утилита для проверки размера замыкания
function estimateClosureSize(fn) {

```

```

const originalConsole = console;
let capturedVars = [];

// Временная замена console.log для отслеживания
console = {
  log: function(...args) {
    capturedVars.push(...args.map(String));
  }
};

try {
  // Вызываем функцию, чтобы увидеть, что она использует
  fn();
} catch (e) {
  // Игнорируем ошибки
} finally {
  console = originalConsole;
}

// Очень грубая оценка размера
const totalSize = capturedVars.reduce((size, str) => {
  return size + (str.length * 2); // Примерно 2 байта на символ в UTF-16
}, 0);

return {
  capturedStrings: capturedVars,
  estimatedSizeBytes: totalSize,
  message: `Примерный размер: ${totalSize} байт`
};
}

const testClosure = createClosureForInspection();
// console.log(estimateClosureSize(testClosure));

// 5. WeakRef и FinalizationRegistry для отслеживания сборки мусора
function trackClosureMemory() {
  const registry = new FinalizationRegistry(heldValue => {
    console.log(`Замыкание с данными "${heldValue}" было собрано GC`);
  });
}

function createTrackedClosure(data) {
  const obj = { data };

  // Регистрируем объект для отслеживания

```

```

registry.register(obj, data);

return function() {
  console.log(data);
  return obj; // Возвращаем ссылку, чтобы объект не собирался
};
}

let closureRef;

return {
  create: function(data) {
    const closure = createTrackedClosure(data);
    closureRef = new WeakRef(closure);
    return closure;
  },

  checkIfAlive: function() {
    const closure = closureRef?.deref();
    return closure ? 'Замыкание все еще в памяти' : 'Замыкание собрано GC';
  },

  forceGC: function() {
    // Нет прямого способа вызвать GC в JavaScript
    // Но можно создать давление на память
    const largeArray = new Array(10000000);
    largeArray.length = 0;
  }
};

const memoryTracker = trackClosureMemory();
const trackedClosure = memoryTracker.create('отслеживаемые данные');

// Используем замыкание
trackedClosure();

// Через некоторое время проверяем
setTimeout(() => {
  console.log(memoryTracker.checkIfAlive());
  memoryTracker.forceGC();

  setTimeout(() => {
    console.log(memoryTracker.checkIfAlive());
  });
}, 1000);

```

```
    }, 1000));  
  }, 1000));
```

Часть 6: Современные альтернативы и лучшие практики

6.1. Классы ES6 с приватными полями

javascript

// КЛАССЫ ES6+ как альтернатива замыканиям для инкапсуляции

// Старый способ с замыканиями

```
function OldCounter(initial = 0) {  
  let count = initial; // Приватная через замыкание  
  
  this.increment = function() {  
    count++;  
    return count;  
  };  
  
  this.getValue = function() {  
    return count;  
  };  
}
```

// Новый способ с классами и приватными полями

```
class ModernCounter {  
  // Приватное поле (синтаксис #)  
  #count = 0;  
  #history = [];  
  
  constructor(initial = 0) {  
    this.#count = initial;  
  }  
  
  // Публичные методы  
  increment(value = 1) {  
    this.#count += value;  
    this.#addToHistory('increment', value);  
    return this.#count;  
  }  
}
```

```

}

decrement(value = 1) {
  this.#count -= value;
  this.#addToHistory('decrement', value);
  return this.#count;
}

getValue() {
  return this.#count;
}

getHistory() {
  return [...this.#history]; // Возвращаем копию
}

reset() {
  const oldValue = this.#count;
  this.#count = 0;
  this.#addToHistory('reset', oldValue);
  return this;
}

// Приватный метод
#addToHistory(action, value) {
  this.#history.push({
    action,
    value,
    timestamp: new Date(),
    count: this.#count
  });

  // Ограничиваем историю
  if (this.#history.length > 100) {
    this.#history = this.#history.slice(-100);
  }
}

// Статический метод
static createRandom() {
  const randomValue = Math.floor(Math.random() * 100);
  return new ModernCounter(randomValue);
}
}

```

```

// Использование
const modernCounter = new ModernCounter(10);
console.log(modernCounter.getValue()); // 10
modernCounter.increment(5);
console.log(modernCounter.getValue()); // 15

// Приватные поля недоступны
// console.log(modernCounter.#count); // SyntaxError
// console.log(modernCounter.#addToHistory); // SyntaxError

// Статические методы
const randomCounter = ModernCounter.createRandom();
console.log('Случайный счетчик:', randomCounter.getValue());

// СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ
class PerformanceTest {
  #data = new Array(1000000).fill(null).map((_, i) => i);

  processWithClosure() {
    // Создает новое замыкание для каждой итерации
    return this.#data.map(function(item) {
      return item * 2;
    });
  }

  processWithArrow() {
    // Стрелочная функция тоже создает замыкание
    return this.#data.map(item => item * 2);
  }

  processWithBoundMethod() {
    // bind создает новую функцию
    function multiply(x) {
      return x * 2;
    }
    return this.#data.map(multiply.bind(this));
  }

  processWithPreBound() {
    // Предварительно привязанная функция
    const multiply = (function(x) {
      return x * 2;
    });
  }
}

```



```

    return this.#data.map(multiply);
  }
}

const test = new PerformanceTest();

console.time('Замыкание');
test.processWithClosure();
console.timeEnd('Замыкание');

console.time('Стрелочная');
test.processWithArrow();
console.timeEnd('Стрелочная');

console.time('Bound метод');
test.processWithBoundMethod();
console.timeEnd('Bound метод');

console.time('Предварительно привязанная');
test.processWithPreBound();
console.timeEnd('Предварительно привязанная');
```

6.2. Модули ES6 и замыкания

javascript

// МОДУЛИ ES6 предоставляют лучшую инкапсуляцию чем IIFE

// Старый способ: IIFE

```

const OldModule = (function() {
  let privateVar = 'секретно';

  function privateFunc() {
    return privateVar;
  }

  return {
    publicMethod: function() {
      return privateFunc() + ' обработано';
    }
  };
})();
```

// Новый способ: ES6 модули

```

// counter.js
let privateCount = 0; // Приватная в модуле
const privateIncrement = () => privateCount++;

export function increment() {
  privateIncrement();
  return privateCount;
}

export function getCount() {
  return privateCount;
}

export function reset() {
  privateCount = 0;
  return privateCount;
}

// app.js
import { increment, getCount, reset } from './counter.js';

console.log(increment()); // 1
console.log(increment()); // 2
console.log(getCount()); // 2
console.log(reset());      // 0

// privateCount и privateIncrement недоступны

// КОМБИНАЦИЯ: модули + замыкания для сложных случаев
export function createAdvancedCounter(initial = 0) {
  // Приватное состояние для каждого экземпляра
  let count = initial;
  let history = [];

  // Приватные функции для каждого экземпляра
  function addToHistory(action, value) {
    history.push({ action, value, timestamp: Date.now(), count });

    if (history.length > 1000) {
      history = history.slice(-1000);
    }
  }

  // Возвращаем объект с методами

```

```

return {
  increment: (value = 1) => {
    count += value;
    addToHistory('increment', value);
    return count;
  },

  decrement: (value = 1) => {
    count -= value;
    addToHistory('decrement', value);
    return count;
  },

  getValue: () => count,

  getHistory: (limit = 100) => {
    return history.slice(-limit);
  },

  // Метод для сериализации (без приватных функций)
  toJSON: () => ({
    count,
    historySize: history.length
  })
};
}

// Использование
import { createAdvancedCounter } from './advancedCounter.js';

const counter1 = createAdvancedCounter(10);
const counter2 = createAdvancedCounter(20);

counter1.increment(5);
counter2.decrement(3);

console.log(counter1.getValue()); // 15
console.log(counter2.getValue()); // 17

```

6.3. Лучшие практики работы с замыканиями

javascript

// 1. ИЗБЕГАЙТЕ НЕНУЖНЫХ ЗАМЫКАНИЙ

```
// Плохо: создает ненужное замыкание
function processArrayBad(array) {
  return array.map(function(item) {
    // Эта функция замыкает всю область видимости processArrayBad
    return item * 2;
  });
}
```

```
// Хорошо: выносите функции, если они не нуждаются в замыкании
function double(item) {
  return item * 2;
}
```

```
function processArrayGood(array) {
  return array.map(double); // Нет лишнего замыкания
}
```

```
// Еще лучше для простых операций:
function processArrayBest(array) {
  return array.map(x => x * 2); // Минимальное замыкание
}
```

```
// 2. ОЧИЩАЙТЕ ЗАМЫКАНИЯ КОГДА ОНИ НЕ НУЖНЫ
function createTemporaryClosure() {
  const largeData = new Array(1000000).fill('data');

  const processor = function() {
    // Делает что-то с LargeData
    return largeData.length;
  };

  // Используем processor
  const result = processor();

  // После использования ОЧИЩАЕМ большие данные
  largeData.length = 0;

  // Возвращаем результат, но не processor
  return result;
}
```

```
// 3. ИСПОЛЬЗУЙТЕ ЗАМЫКАНИЯ ДЛЯ ИНКАПСУЛЯЦИИ, А НЕ ДЛЯ ВСЕГО
// Хорошее использование: приватное состояние
function createPrivateState() {
```

```

let state = {}; // Приватное

return {
  set: (key, value) => { state[key] = value; },
  get: (key) => state[key]
};
}

// Плохое использование: избыточные замыкания
function overuseClosures() {
  const a = 1;
  const b = 2;

  return {
    // Каждая функция замыкает a и b, даже если они не нужны
    method1: () => a + 1,
    method2: () => b + 2,
    method3: () => a + b
  };
}

// 4. ДОКУМЕНТИРУЙТЕ СЛОЖНЫЕ ЗАМЫКАНИЯ
/**
 * Создает кэширующую обертку для функции
 * Использует замыкание для хранения кэша
 *
 * @param {Function} fn - Функция для мемоизации
 * @returns {Function} Мемоизированная версия функции
 */
function memoize(fn) {
  const cache = new Map(); // Замыкается

  return function memoized(...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      return cache.get(key);
    }

    const result = fn(...args);
    cache.set(key, result);

    return result;
  };
}

```

```
}
```

```
// 5. ТЕСТИРУЕМОСТЬ ЗАМЫКАНИЙ
```

```
// Проблема: приватные функции в замыканиях сложно тестировать
```

```
function createUntestableModule() {  
  function privateHelper() { // Не тестируется  
    return 'секретная логика';  
  }  
  
  return {  
    publicMethod: () => privateHelper() + ' обработано'  
  };  
}
```

```
// Решение: выносить логику
```

```
export function testableHelper() { // Можно импортировать и тестировать  
  return 'логика';  
}
```

```
function createTestableModule(helper = testableHelper) {  
  return {  
    publicMethod: () => helper() + ' обработано'  
  };  
}
```

```
// 6. ИСПОЛЬЗУЙТЕ ЗАМЫКАНИЯ ДЛЯ БЕЗОПАСНОСТИ
```

```
function createSecureAPI(apiKey) {  
  // apiKey замыкается и никогда не покидает эту область  
  const validKeys = new Set([apiKey]);
```

```
  return {  
    authenticate: function(key) {  
      return validKeys.has(key);  
    },
```

```
    // Метод не раскрывает apiKey
```

```
    makeRequest: function(endpoint, data) {  
      if (!this.authenticate(data?.key)) {  
        throw new Error('Не авторизован');  
      }  
    }
```

```
    // Делает запрос с внутренним apiKey
```

```
    return fetch(endpoint, {  
      headers: {
```

```

    'Authorization': `Bearer ${apiKey}`
  }
});
}
};
}

const api = createSecureAPI('secret-key-123');
console.log(api.authenticate('wrong-key')); // false

// apiKey никогда не доступен извне
// console.log(api.apiKey); // undefined

```

6.4. Сводка и заключение

javascript

```

// ИТОГОВЫЙ ПРИМЕР: продуманное использование замыканий

/**
 * Фабрика для создания безопасных, производительных счетчиков
 * с полной инкапсуляцией состояния
 */
function createProductionCounter(config = {}) {
  // Приватные переменные с дефолтными значениями
  const {
    initialValue = 0,
    maxValue = Number.MAX_SAFE_INTEGER,
    minValue = Number.MIN_SAFE_INTEGER,
    enableHistory = false,
    maxHistorySize = 100
  } = config;

  // Валидация конфигурации
  if (initialValue < minValue || initialValue > maxValue) {
    throw new Error('Начальное значение вне допустимого диапазона');
  }

  // Приватное состояние
  let value = initialValue;
  let history = enableHistory ? [] : null;

  // Приватные вспомогательные функции
  function isWithinBounds(newValue) {

```

```

    return newValue >= minValue && newValue <= maxValue;
}

function addHistoryEntry(action, change, newValue) {
    if (!enableHistory || !history) return;

    history.push({
        action,
        change,
        newValue,
        timestamp: Date.now(),
        previousValue: value
    });

    // Ограничиваем размер истории
    if (history.length > maxHistorySize) {
        history = history.slice(-maxHistorySize);
    }
}

function updateValue(newValue, action, change) {
    if (!isWithinBounds(newValue)) {
        throw new RangeError(`Значение ${newValue} вне диапазона [${minValue}, ${maxValue}]`);
    }

    const oldValue = value;
    value = newValue;

    addHistoryEntry(action, change, value);

    return {
        oldValue,
        newValue: value,
        change
    };
}

// Публичный API с четкой документацией
return {
    // Основные операции
    increment: function(amount = 1) {
        return updateValue(value + amount, 'increment', amount);
    },

```



```

decrement: function(amount = 1) {
  return updateValue(value - amount, 'decrement', -amount);
},

setValue: function(newValue) {
  const change = newValue - value;
  return updateValue(newValue, 'set', change);
},

reset: function() {
  const change = initialValue - value;
  return updateValue(initialValue, 'reset', change);
},

// Чтение состояния
getValue: function() {
  return value;
},

getConfig: function() {
  return { ...config }; // Возвращаем копию
},

getHistory: function(limit = maxHistorySize) {
  if (!enableHistory || !history) {
    throw new Error('История отключена');
  }

  return history.slice(-limit).map(entry => ({ ...entry }));
},

// Сериализация
toJSON: function() {
  return {
    value,
    config: { ...config },
    hasHistory: enableHistory,
    historySize: history ? history.length : 0
  };
},

// Методы для управления памятью
clearHistory: function() {
  if (history) {

```

```

    const size = history.length;
    history.length = 0; // Очищаем массив
    return size;
  }
  return 0;
},

// Статический метод для создания из JSON
static fromJSON: function(json) {
  const counter = createProductionCounter(json.config);
  counter.setValue(json.value);
  return counter;
}
};
}

// Пример использования с лучшими практиками
try {
  // Создаем счетчик с историей
  const counter = createProductionCounter({
    initialValue: 10,
    minValue: 0,
    maxValue: 100,
    enableHistory: true,
    maxHistorySize: 50
  });

  // Операции
  console.log('Начальное значение:', counter.getValue()); // 10

  const result1 = counter.increment(5);
  console.log('После инкремента:', result1.newValue); // 15

  const result2 = counter.decrement(3);
  console.log('После декремента:', result2.newValue); // 12

  // Работа с историей
  const history = counter.getHistory(5);
  console.log('Последние 5 операций:', history);

  // Сериализация
  const json = counter.toJSON();
  console.log('JSON представление:', json);

```

```
// Очистка памяти
const cleared = counter.clearHistory();
console.log(`Очищено ${cleared} записей истории`);

// Восстановление из JSON
const restoredCounter = createProductionCounter.fromJSON(json);
console.log('Восстановленное значение:', restoredCounter.getValue());

} catch (error) {
  console.error('Ошибка:', error.message);
}

// КЛЮЧЕВЫЕ ВЫВОДЫ:
// 1. Замыкания — мощный инструмент для инкапсуляции
// 2. Используйте их осознанно, а не везде
// 3. Следите за утечками памяти
// 4. Документируйте сложные замыкания
// 5. Современные альтернативы (классы, модули) часто лучше
// 6. Производительность важна в горячих путях
// 7. Тестируемость должна быть приоритетом
```

Заключение: Мастерство замыканий

Замыкания — это **суперсила JavaScript**, которая позволяет создавать элегантные, безопасные и эффективные паттерны программирования. Они лежат в основе:

1. **Инкапсуляции** и приватности данных
2. **Функционального программирования** (каррирование, мемоизация)
3. **Асинхронного программирования** (обработчики событий, промисы)
4. **Модульности** и организации кода

Помните:

- 🔵 Каждое замыкание — это не просто функция, это **функция с контекстом**
- 🔵 Замыкания **сохраняют лексическое окружение** даже после выхода из функции
- 🔵 Они могут вызывать **утечки памяти**, если используются неосторожно
- 🔵 Современные альтернативы (классы с приватными полями, модули) часто предпочтительнее

Мастер замыканий знает:

- Когда использовать замыкания, а когда выбрать другой подход
- Как избегать утечек памяти
- Как проектировать чистые, тестируемые интерфейсы
- Как балансировать между инкапсуляцией и производительностью

Замыкания — это не просто фича языка, это **философия программирования** в JavaScript. Освойте их — и вы откроете для себя совершенно новый уровень владения языком.

Введение: что такое рекурсия

Рекурсия — это процесс, при котором функция вызывает саму себя прямо или косвенно. Это мощная техника программирования, позволяющая элегантно решать задачи, которые могут быть разбиты на аналогичные подзадачи.

Аналогия из реального мира:

Представьте матрёшку: каждая кукла содержит внутри себя такую же, но меньшего размера. Открывая одну, вы находите внутри другую, пока не дойдёте до самой маленькой.

Часть 1: Основы рекурсии

1.1. Структура рекурсивной функции

Каждая рекурсивная функция должна содержать:

- 1. Базовый случай** — условие, при котором рекурсия прекращается
- 2. Рекурсивный случай** — вызов функции с упрощённой версией задачи

javascript

// Базовая структура рекурсивной функции

```
function recursiveFunction(параметр) {  
  // 1. Проверка базового случая  
  if (условиеОстановки) {  
    return базовоеЗначение;  
  }
```

// 2. Упрощение задачи

```
const упрощеннаяЗадача = упростить(параметр);
```

// 3. Рекурсивный вызов

```

const результат = recursiveFunction(упрощеннаяЗадача);

// 4. Обработка результата (опционально)
return обработать(результат);
}

```

1.2. Простой пример: факториал

Факториал числа n ($n!$) — произведение всех положительных целых чисел от 1 до n .

Математическое определение:

- $0! = 1$ (базовый случай)
- $n! = n \times (n-1)!$ (рекурсивный случай)

javascript

// Рекурсивное вычисление факториала

```

function factorial(n) {
  // Базовый случай: 0! = 1
  if (n === 0) {
    return 1;
  }

  // Рекурсивный случай: n! = n * (n-1)!
  return n * factorial(n - 1);
}

```

// Примеры использования

```

console.log(factorial(0)); // 1
console.log(factorial(1)); // 1
console.log(factorial(5)); // 120 (5*4*3*2*1)
console.log(factorial(10)); // 3628800

```

// Визуализация вызова factorial(5):

```

// factorial(5) = 5 * factorial(4)
// factorial(4) = 4 * factorial(3)
// factorial(3) = 3 * factorial(2)
// factorial(2) = 2 * factorial(1)
// factorial(1) = 1 * factorial(0)
// factorial(0) = 1 ← базовый случай
// Затем "раскрутка" стека:
// factorial(1) = 1 * 1 = 1

```

```
// factorial(2) = 2 * 1 = 2
// factorial(3) = 3 * 2 = 6
// factorial(4) = 4 * 6 = 24
// factorial(5) = 5 * 24 = 120
```

Часть 2: Стек вызовов (Call Stack)

2.1. Что такое стек вызовов

Стек вызовов — это структура данных, которую JavaScript использует для отслеживания выполнения функций. Это стек (LIFO — Last In, First Out), где каждая вызванная функция помещается на вершину стека, а когда функция завершается, она удаляется из стека.

2.2. Как работает стек вызовов

```
javascript

function first() {
  console.log('Входим в first');
  second();
  console.log('Выходим из first');
}

function second() {
  console.log('Входим в second');
  third();
  console.log('Выходим из second');
}

function third() {
  console.log('Входим в third');
  console.trace(); // Показывает текущий стек вызовов
  console.log('Выходим из third');
}

first();

// Вывод:
// Входим в first
```

```

// Входим в second
// Входим в third
// Stack trace (примерный вид):
//   at third (script.js:10)
//   at second (script.js:5)
//   at first (script.js:2)
// Выходим из third
// Выходим из second
// Выходим из first

// Визуализация стека:
// 1. Вызов first() → стек: [first]
// 2. first вызывает second() → стек: [first, second]
// 3. second вызывает third() → стек: [first, second, third]
// 4. third завершается → стек: [first, second]
// 5. second завершается → стек: [first]
// 6. first завершается → стек: []

```

2.3. Стек вызовов при рекурсии

javascript

```

function countdown(n) {
  console.log(`Стек: ${n}`);

  if (n <= 0) {
    console.log('Базовый случай достигнут!');
    return;
  }

  countdown(n - 1);
  console.log(`Выход: ${n}`);
}

countdown(3);

// Вывод:
// Стек: 3
// Стек: 2
// Стек: 1
// Стек: 0
// Базовый случай достигнут!
// Выход: 1
// Выход: 2

```



```
// Выход: 3

// Визуализация стека:
// 1. countdown(3) → стек: [countdown(3)]
// 2. Вызов countdown(2) → стек: [countdown(3), countdown(2)]
// 3. Вызов countdown(1) → стек: [countdown(3), countdown(2), countdown(1)]
// 4. Вызов countdown(0) → стек: [countdown(3), countdown(2), countdown(1), countdown(0)]
// 5. countdown(0) завершается → стек: [countdown(3), countdown(2), countdown(1)]
// 6. countdown(1) завершается → стек: [countdown(3), countdown(2)]
// 7. countdown(2) завершается → стек: [countdown(3)]
// 8. countdown(3) завершается → стек: []
```

Часть 3: Типы рекурсии

3.1. Прямая рекурсия

Функция вызывает саму себя напрямую.

javascript

```
// Прямая рекурсия
function directRecursion(n) {
  if (n <= 1) return 1;
  return n * directRecursion(n - 1); // Прямой вызов себя
}
```

3.2. Косвенная рекурсия

Функция А вызывает функцию В, которая вызывает функцию А.

javascript

```
// Косвенная рекурсия
function isEven(n) {
  if (n === 0) return true;
  return isOdd(n - 1);
}

function isOdd(n) {
  if (n === 0) return false;
  return isEven(n - 1);
}
```

```
console.log(isEven(4)); // true
console.log(isOdd(4)); // false
console.log(isEven(5)); // false
console.log(isOdd(5)); // true
```

3.3. Хвостовая рекурсия

Рекурсивный вызов является последней операцией в функции.

javascript

// Обычная рекурсия (не хвостовая)

```
function sumNormal(n) {
  if (n <= 1) return n;
  return n + sumNormal(n - 1); // Сложение происходит ПОСЛЕ рекурсивного вызова
}
```

// Хвостовая рекурсия

```
function sumTail(n, accumulator = 0) {
  if (n <= 0) return accumulator;
  return sumTail(n - 1, accumulator + n); // Рекурсивный вызов - ПОСЛЕДНЯЯ операция
}
```

```
console.log(sumNormal(5)); // 15
```

```
console.log(sumTail(5)); // 15
```

// Важность хвостовой рекурсии:

// Современные JavaScript движки могут оптимизировать хвостовую рекурсию,

// избегая роста стека вызовов (Tail Call Optimization - TCO).

Часть 4: Практические примеры рекурсии

4.1. Обход древовидных структур

javascript

// Рекурсивный обход дерева DOM

```
function traverseDOM(node, depth = 0) {
  // Базовый случай: нет узла
  if (!node) return;
```

```

// Отступ для визуализации уровня
const indent = ' '.repeat(depth);

// Вывод информации о текущем узле
console.log(`${indent}${node.nodeName}${node.id ? `#${node.id}` : ''}${node.className ?
`${node.className}` : ''}`);

// Рекурсивный обход детей
for (let child of node.children) {
    traverseDOM(child, depth + 1);
}
}

// Использование:
// traverseDOM(document.body);

// Пример с кастомной древовидной структурой
const fileSystem = {
    name: '/',
    type: 'directory',
    children: [
        {
            name: 'home',
            type: 'directory',
            children: [
                { name: 'user1', type: 'directory', children: [
                    { name: 'documents', type: 'directory', children: [
                        { name: 'report.txt', type: 'file', size: 1024 }
                    ]},
                { name: 'photos', type: 'directory', children: [] }
            ]}
        ],
        { name: 'etc', type: 'directory', children: [] },
        { name: 'README.md', type: 'file', size: 512 }
    ]
};

function traverseFileSystem(node, path = '') {
    const currentPath = path ? `${path}/${node.name}` : node.name;

    if (node.type === 'file') {
        console.log(`Файл: ${currentPath} (${node.size} байт)`);
    } else {

```

```

    console.log(`Директория: ${currentPath}`);

    // Рекурсивный обход детей
    for (let child of node.children) {
        traverseFileSystem(child, currentPath);
    }
}
}

traverseFileSystem(fileSystem);

```

4.2. Генерация последовательностей

javascript

```

// Числа Фибоначчи: каждое число равно сумме двух предыдущих
// 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

```

```

// Рекурсивная реализация (неэффективная)

```

```

function fibonacciRecursive(n) {
    // Базовые случаи
    if (n === 0) return 0;
    if (n === 1) return 1;

    // Рекурсивный случай
    return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
}

```

```

console.log(fibonacciRecursive(6)); // 8

```

```

// Проблема: много повторных вычислений!

```

```

// Вычисление fibonacciRecursive(6) требует вычисления:

```

```

// fibonacciRecursive(5) и fibonacciRecursive(4)

```

```

// fibonacciRecursive(5) требует fibonacciRecursive(4) и fibonacciRecursive(3)

```

```

// И т.д. - экспоненциальная сложность  $O(2^n)$ 

```

```

// Оптимизированная версия с мемоизацией

```

```

function fibonacciMemo(n, memo = {}) {
    // Проверяем, вычисляли ли мы уже это значение
    if (memo[n] !== undefined) {
        return memo[n];
    }
}

```

```

// Базовые случаи

```

```

if (n === 0) return 0;

```

```

    if (n === 1) return 1;

    // Рекурсивный случай с сохранением результата
    memo[n] = fibonacciMemo(n - 1, memo) + fibonacciMemo(n - 2, memo);
    return memo[n];
}

console.log(fibonacciMemo(50)); // 12586269025 (вычисляется быстро)

// Генерация всех чисел Фибоначчи до n
function generateFibonacciSequence(n) {
    const sequence = [];

    function generateHelper(count) {
        if (count > n) return;

        if (count === 0) {
            sequence.push(0);
        } else if (count === 1) {
            sequence.push(1);
        } else {
            sequence.push(sequence[count - 1] + sequence[count - 2]);
        }

        generateHelper(count + 1);
    }

    generateHelper(0);
    return sequence;
}

console.log(generateFibonacciSequence(10)); // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]

```

4.3. Решение головоломок

javascript

```

// Ханойская башня
function hanoiTower(n, from, to, aux, moves = []) {
    // Базовый случай: если только один диск
    if (n === 1) {
        moves.push(`Переместить диск 1 с ${from} на ${to}`);
        return moves;
    }

```

```

// 1. Переместить n-1 дисков с from на aux
hanoiTower(n - 1, from, aux, to, moves);

// 2. Переместить самый большой диск с from на to
moves.push(`Переместить диск ${n} с ${from} на ${to}`);

// 3. Переместить n-1 дисков с aux на to
hanoiTower(n - 1, aux, to, from, moves);

return moves;
}

// Решение для 3 дисков
const solution = hanoiTower(3, 'A', 'C', 'B');
console.log(`Минимальное количество ходов: ${solution.length}`);
solution.forEach((move, index) => {
  console.log(`${index + 1}. ${move}`);
});

// Генерация всех перестановок строки
function generatePermutations(str) {
  const result = [];

  function permute(current, remaining) {
    // Базовый случай: если осталась пустая строка
    if (remaining.length === 0) {
      result.push(current);
      return;
    }

    // Рекурсивный случай: для каждого символа в оставшейся строке
    for (let i = 0; i < remaining.length; i++) {
      // Выбираем текущий символ
      const char = remaining[i];

      // Создаём новую оставшуюся строку без выбранного символа
      const newRemaining = remaining.slice(0, i) + remaining.slice(i + 1);

      // Рекурсивно генерируем перестановки
      permute(current + char, newRemaining);
    }
  }
}

```

```
    permute('', str);
    return result;
}

console.log(generatePermutations('abc')); // ['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```

Часть 5: Ограничения и опасности рекурсии

5.1. Переполнение стека (Stack Overflow)

```
javascript

// Пример, вызывающий переполнение стека
function infiniteRecursion() {
    infiniteRecursion(); // Бесконечная рекурсия
}

// infiniteRecursion(); // Вызовет ошибку: Maximum call stack size exceeded

// Глубокий, но конечный вызов
function deepRecursion(n) {
    if (n <= 0) return;

    // Вызов с очень большой глубиной
    deepRecursion(n - 1);
}

// deepRecursion(100000); // Может вызвать переполнение стека в зависимости от движка

// Безопасная рекурсия с контролем глубины
function safeRecursion(n, maxDepth = 10000) {
    if (n <= 0) return 'Готово';

    // Проверка максимальной глубины
    if (maxDepth <= 0) {
        throw new Error('Превышена максимальная глубина рекурсии');
    }

    return safeRecursion(n - 1, maxDepth - 1);
}

try {
```

```
console.log(safeRecursion(5000)); // Работает
console.log(safeRecursion(15000)); // Выбросит ошибку
} catch (error) {
  console.error(error.message);
}
```

5.2. Эффективность и оптимизация

javascript

// Сравнение производительности: рекурсия vs итерация

```
function sumRecursive(n) {
  if (n <= 1) return n;
  return n + sumRecursive(n - 1);
}
```

```
function sumIterative(n) {
  let sum = 0;
  for (let i = 1; i <= n; i++) {
    sum += i;
  }
  return sum;
}
```

// Формула (самый эффективный способ)

```
function sumFormula(n) {
  return n * (n + 1) / 2;
}
```

// Тест производительности

```
const testValue = 10000;
```

```
console.time('Рекурсия');
console.log(sumRecursive(testValue));
console.timeEnd('Рекурсия');
```

```
console.time('Итерация');
console.log(sumIterative(testValue));
console.timeEnd('Итерация');
```

```
console.time('Формула');
console.log(sumFormula(testValue));
console.timeEnd('Формула');
```


// Оптимизация через мемоизацию

```
function createMemoizedFunction(fn) {  
  const cache = new Map();  
  
  return function memoized(...args) {  
    const key = JSON.stringify(args);  
  
    if (cache.has(key)) {  
      console.log('Взято из кэша:', key);  
      return cache.get(key);  
    }  
  
    console.log('Вычислено:', key);  
    const result = fn(...args);  
    cache.set(key, result);  
    return result;  
  };  
}
```

// Пример использования мемоизации

```
const memoizedFactorial = createMemoizedFunction(function factorial(n) {  
  if (n <= 1) return 1;  
  return n * memoizedFactorial(n - 1);  
});
```

`console.log(memoizedFactorial(5));` // Вычисляет все значения

`console.log(memoizedFactorial(6));` // `factorial(6) = 6 * memoizedFactorial(5)` - взято из кэша!

Часть 6: Продвинутые техники рекурсии

6.1. Рекурсия с возвратом (Backtracking)

javascript

// Решение задачи о N ферзях

```
function solveNQueens(n) {  
  const solutions = [];  
  const board = Array(n).fill().map(() => Array(n).fill('.'));  
  
  function isSafe(row, col) {  
    // Проверка столбца
```

```

    for (let i = 0; i < row; i++) {
        if (board[i][col] === 'Q') return false;
    }

    // Проверка диагонали слева вверх
    for (let i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] === 'Q') return false;
    }

    // Проверка диагонали справа вверх
    for (let i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (board[i][j] === 'Q') return false;
    }

    return true;
}

function solve(row) {
    // Базовый случай: все ферзи размещены
    if (row === n) {
        solutions.push(board.map(r => r.join('')));
        return;
    }

    // Пробуем разместить ферзя в каждом столбце текущей строки
    for (let col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            // Размещаем ферзя
            board[row][col] = 'Q';

            // Рекурсивно размещаем следующего ферзя
            solve(row + 1);

            // Возврат (убираем ферзя)
            board[row][col] = '.';
        }
    }
}

solve(0);
return solutions;
}

// Решение для 4 ферзей

```

```

const solutions = solveNQueens(4);
console.log(`Найдено решений для 4 ферзей: ${solutions.length}`);
solutions.forEach((solution, index) => {
  console.log(`Решение ${index + 1}:`);
  solution.split('').forEach((row, i) => {
    console.log(row);
  });
  console.log('');
});

```

6.2. Рекурсия в функциональном программировании

javascript

// Рекурсивные функции высшего порядка

// 1. Рекурсивный map

```

function recursiveMap(arr, fn) {
  if (arr.length === 0) return [];

  const [first, ...rest] = arr;
  return [fn(first), ...recursiveMap(rest, fn)];
}

```

```

console.log(recursiveMap([1, 2, 3, 4], x => x * 2)); // [2, 4, 6, 8]

```

// 2. Рекурсивный filter

```

function recursiveFilter(arr, fn) {
  if (arr.length === 0) return [];

  const [first, ...rest] = arr;
  const filteredRest = recursiveFilter(rest, fn);

  return fn(first) ? [first, ...filteredRest] : filteredRest;
}

```

```

console.log(recursiveFilter([1, 2, 3, 4, 5], x => x % 2 === 0)); // [2, 4]

```

// 3. Рекурсивный reduce

```

function recursiveReduce(arr, fn, initial) {
  if (arr.length === 0) return initial;

  const [first, ...rest] = arr;
  const newInitial = fn(initial, first);

```

```

    return recursiveReduce(rest, fn, newInitial);
}

console.log(recursiveReduce([1, 2, 3, 4], (acc, val) => acc + val, 0)); // 10

// Композиция рекурсивных функций
function compose(...fns) {
    return function composed(x) {
        // Рекурсивная реализация композиции
        function applyFunctions(value, functions) {
            if (functions.length === 0) return value;

            const [firstFn, ...restFns] = functions;
            return applyFunctions(firstFn(value), restFns);
        }

        return applyFunctions(x, fns);
    };
}

const add5 = x => x + 5;
const multiply3 = x => x * 3;
const subtract2 = x => x - 2;

const complexOperation = compose(add5, multiply3, subtract2);
console.log(complexOperation(10)); // ((10 + 5) * 3) - 2 = 43

```

Часть 7: Отладка рекурсивных функций

7.1. Техники отладки

javascript

```

// Рекурсивная функция с подробной отладкой
function factorialDebug(n, depth = 0, stack = []) {
    const indent = ' '.repeat(depth);

    console.log(`${indent}Вызов factorial(${n})`);
    console.log(`${indent}Текущий стек: [${stack.join(', ')}]`);

    // Базовый случай
    if (n === 0) {

```

```

    console.log(`${indent}Базовый случай: factorial(0) = 1`);
    console.log(`${indent}Возвращаем 1`);
    return 1;
}

// Рекурсивный вызов
console.log(`${indent}Вычисляем factorial(${n - 1})...`);

// Добавляем текущий вызов в стек
stack.push(`factorial(${n})`);
const result = n * factorialDebug(n - 1, depth + 1, stack);
stack.pop();

console.log(`${indent}Получили factorial(${n - 1}) = ${result / n}`);
console.log(`${indent}factorial(${n}) = ${n} * ${result / n} = ${result}`);
console.log(`${indent}Возвращаем ${result}`);

return result;
}

console.log('Результат:', factorialDebug(3));

```

7.2. Визуализация стека вызовов

javascript

```

// Инструмент для визуализации рекурсии
function visualizeRecursion(fn, ...args) {
    let callCount = 0;
    const calls = [];

    // Обёртка для отслеживания вызовов
    function wrappedFn(...params) {
        callCount++;
        const callId = callCount;
        const depth = calls.filter(c => !c.completed).length;

        calls.push({
            id: callId,
            function: fn.name || 'anonymous',
            params: JSON.stringify(params),
            depth,
            startTime: Date.now(),
            completed: false
        });
    }
}

```

```
});
```

```
console.log(`→ Вызов ${callId}: ${fn.name || 'функция'}(${params.join(', ')} `);  
console.log(` Глубина: ${depth}, Активных вызовов: ${depth + 1}`);
```

```
try {
```

```
    const result = fn(...params);
```

```
    // Отмечаем вызов как завершённый
```

```
    const call = calls.find(c => c.id === callId);
```

```
    call.completed = true;
```

```
    call.endTime = Date.now();
```

```
    call.duration = call.endTime - call.startTime;
```

```
    console.log(`← Возврат ${callId}: ${result} (${call.duration}мс)`);
```

```
    return result;
```

```
} catch (error) {
```

```
    console.error(`x Ошибка в вызове ${callId}: ${error.message}`);
```

```
    throw error;
```

```
}
```

```
}
```

```
console.log('=' .repeat(50));
```

```
console.log(`Начало выполнения: ${fn.name || 'функция'}`);
```

```
console.log('=' .repeat(50));
```

```
const result = wrappedFn(...args);
```

```
console.log('=' .repeat(50));
```

```
console.log('Статистика выполнения:');
```

```
console.log(`Всего вызовов: ${callCount}`);
```

```
console.log(`Максимальная глубина: ${Math.max(...calls.map(c => c.depth))}`);
```

```
console.log(`Общее время: ${calls.reduce((sum, c) => sum + (c.duration || 0), 0)}мс`);
```

```
console.log('=' .repeat(50));
```

```
return result;
```

```
}
```

```
// Пример использования
```

```
const visualizedFactorial = visualizeRecursion(function factorial(n) {
```

```
    if (n <= 1) return 1;
```

```
    return n * visualizedFactorial(n - 1);
```

```
}, 4);
```

Часть 8: Практические рекомендации

8.1. Когда использовать рекурсию

Используйте рекурсию, когда:

1. Задача естественно рекурсивна (дерево, граф, математическая рекуррентность)
2. Глубина рекурсии предсказуема и невелика
3. Код с рекурсией более читаем, чем итеративное решение

Избегайте рекурсии, когда:

1. Глубина рекурсии может быть большой (риск переполнения стека)
2. Производительность критически важна
3. Существует простое итеративное решение

8.2. Паттерны для безопасной рекурсии

javascript

// 1. Ограничение глубины рекурсии

```
function safeRecursiveOperation(data, operation, maxDepth = 1000, currentDepth = 0) {  
  if (currentDepth >= maxDepth) {  
    throw new Error(`Превышена максимальная глубина рекурсии: ${maxDepth}`);  
  }  
  
  // ... рекурсивная логика ...  
  return safeRecursiveOperation(modifiedData, operation, maxDepth, currentDepth + 1);  
}
```

// ... рекурсивная логика ...

```
return safeRecursiveOperation(modifiedData, operation, maxDepth, currentDepth + 1);  
}
```

// 2. Использование хвостовой рекурсии

```
function tailRecursiveSum(n, accumulator = 0) {  
  if (n <= 0) return accumulator;  
  return tailRecursiveSum(n - 1, accumulator + n); // Хвостовая рекурсия  
}
```

// 3. Преобразование рекурсии в итерацию (трамплин)

```
function trampoline(fn) {  
  return function(...args) {  
    let result = fn(...args);  
    while (result instanceof Function) {  
      result = result(...args);  
    }  
    return result;  
  };  
}
```

```

    while (typeof result === 'function') {
        result = result();
    }

    return result;
};
}

function recursiveSum(n, sum = 0) {
    if (n <= 0) return sum;
    return () => recursiveSum(n - 1, sum + n); // Возвращаем функцию вместо вызова
}

const safeSum = trampoline(recursiveSum);
console.log(safeSum(100000)); // Работает без переполнения стека

```

8.3. Пример: рекурсивный парсер

javascript

// Рекурсивный парсер простых арифметических выражений

```

function parseExpression(expression) {
    let index = 0;

    function parseNumber() {
        let number = '';

        while (index < expression.length && /\d/.test(expression[index])) {
            number += expression[index];
            index++;
        }

        return parseInt(number, 10);
    }

    function parseTerm() {
        let left = parseNumber();

        while (index < expression.length && (expression[index] === '*' || expression[index]
=== '/')) {
            const operator = expression[index];
            index++;

            const right = parseNumber();

```



```

        if (operator === '*') {
            left *= right;
        } else {
            left /= right;
        }
    }

    return left;
}

function parse() {
    let left = parseTerm();

    while (index < expression.length && (expression[index] === '+' || expression[index]
=== '-')) {
        const operator = expression[index];
        index++;

        const right = parseTerm();

        if (operator === '+') {
            left += right;
        } else {
            left -= right;
        }
    }

    return left;
}

return parse();
}

console.log(parseExpression('2+3*4')); // 14
console.log(parseExpression('10-6/2+3')); // 10
console.log(parseExpression('1+2*3-4/2')); // 5

```

Заключение: ключевые принципы рекурсии

8.4. Итоговые рекомендации

1. **Всегда определяйте базовый случай** — без него рекурсия бесконечна
2. **Убедитесь, что рекурсия сходится** — каждый шаг должен приближать к базовому случаю
3. **Контролируйте глубину рекурсии** — используйте ограничители или преобразуйте в итерацию
4. **Рассмотрите мемоизацию** — для оптимизации повторных вычислений
5. **Протестируйте на граничных случаях** — 0, 1, отрицательные значения, большие числа

8.5. Упражнения для закрепления

javascript

// 1. Реализуйте рекурсивную функцию для вычисления степени числа

```
function power(base, exponent) {  
    // Ваша реализация  
}
```

// 2. Реализуйте рекурсивную функцию для поиска наибольшего общего делителя (НОД)

```
function gcd(a, b) {  
    // Ваша реализация  
}
```

// 3. Реализуйте рекурсивную функцию для проверки, является ли строка палиндромом

```
function isPalindrome(str) {  
    // Ваша реализация  
}
```

// 4. Реализуйте рекурсивную функцию для генерации всех подмножеств множества

```
function generateSubsets(set) {  
    // Ваша реализация  
}
```

// 5. Реализуйте рекурсивную функцию для обхода графа в глубину (DFS)

```
function depthFirstSearch(node, visited = new Set()) {  
    // Ваша реализация  
}
```

Рекурсия — это мощный инструмент в арсенале JavaScript-разработчика. При правильном использовании она позволяет писать элегантный и выразительный код для решения сложных задач. Однако важно понимать её ограничения и потенциальные риски, особенно связанные с переполнением стека и производительностью.

Введение: что такое функции высшего порядка

Функции высшего порядка (Higher-Order Functions, HOF) — это функции, которые либо принимают другие функции в качестве аргументов, либо возвращают функции как результат, либо и то, и другое одновременно. Эта концепция является фундаментальной в функциональном программировании и одним из ключевых преимуществ JavaScript как языка.

Основные характеристики:

1. **Функции как значения первого класса** — в JavaScript функции можно:
 - ➔ Присваивать переменным
 - ➔ Передавать как аргументы другим функциям
 - ➔ Возвращать из других функций
 - ➔ Хранить в структурах данных
2. **Функции высшего порядка** используют эту возможность для:
 - ➔ Абстрагирования поведения
 - ➔ Создания многократно используемого кода
 - ➔ Реализации паттернов проектирования

Часть 1: Функции как аргументы

1.1. Базовые примеры

```
javascript
```

```
// Функция, принимающая другую функцию как аргумент
```

```
function applyOperation(x, y, operation) {  
  return operation(x, y);  
}
```

```
// Определяем несколько операций
```

```
function add(a, b) {  
  return a + b;  
}
```

```

function multiply(a, b) {
    return a * b;
}

function power(a, b) {
    return Math.pow(a, b);
}

// Используем функции высшего порядка
console.log(applyOperation(5, 3, add));           // 8
console.log(applyOperation(5, 3, multiply));      // 15
console.log(applyOperation(5, 3, power));         // 125

// Анонимные функции (лямбда-функции)
console.log(applyOperation(10, 2, function(a, b) {
    return a / b;
})); // 5

// Стрелочные функции (ES6+)
console.log(applyOperation(10, 2, (a, b) => a - b)); // 8

```

1.2. Реальные примеры использования

```

javascript

// 1. Функция для обработки массива с callback
function processArray(arr, processor) {
    const result = [];

    for (let i = 0; i < arr.length; i++) {
        result.push(processor(arr[i], i, arr));
    }

    return result;
}

const numbers = [1, 2, 3, 4, 5];

// Удвоение каждого элемента
const doubled = processArray(numbers, function(num) {
    return num * 2;
});
console.log(doubled); // [2, 4, 6, 8, 10]

```

```
// Преобразование в строки
const strings = processArray(numbers, function(num) {
  return `Number: ${num}`;
});
console.log(strings); // ['Number: 1', 'Number: 2', ...]

// 2. Функция для фильтрации данных
function filterByCriteria(data, criteria) {
  const result = [];

  for (let item of data) {
    if (criteria(item)) {
      result.push(item);
    }
  }

  return result;
}

const users = [
  { name: 'Alice', age: 25, active: true },
  { name: 'Bob', age: 17, active: true },
  { name: 'Charlie', age: 30, active: false },
  { name: 'Diana', age: 22, active: true }
];

// Фильтр по возрасту
const adults = filterByCriteria(users, function(user) {
  return user.age >= 18;
});
console.log(adults.map(u => u.name)); // ['Alice', 'Charlie', 'Diana']

// Фильтр по активности
const activeUsers = filterByCriteria(users, function(user) {
  return user.active;
});
console.log(activeUsers.map(u => u.name)); // ['Alice', 'Bob', 'Diana']

// Комбинированный фильтр
const activeAdults = filterByCriteria(users, function(user) {
  return user.age >= 18 && user.active;
});
console.log(activeAdults.map(u => u.name)); // ['Alice', 'Diana']
```

1.3. Callback-функции в асинхронном программировании

javascript

// Классический пример callback-функций

```
function fetchData(url, onSuccess, onError) {
  console.log(`Загрузка данных с ${url}...`);

  // Имитация асинхронной операции
  setTimeout(function() {
    const randomSuccess = Math.random() > 0.3;

    if (randomSuccess) {
      const data = { url, timestamp: Date.now(), status: 'success' };
      onSuccess(data);
    } else {
      onError(new Error(`Ошибка загрузки с ${url}`));
    }
  }, 1000);
}
```

// Использование с callback-функциями

```
fetchData(
  'https://api.example.com/users',
  function(data) {
    console.log('Данные получены:', data);

    // Цепочка callback-ов (callback hell - плохой паттерн)
    fetchData(
      'https://api.example.com/posts',
      function(posts) {
        console.log('Посты получены:', posts);
      },
      function(error) {
        console.error('Ошибка при загрузке постов:', error);
      }
    );
  },
  function(error) {
    console.error('Ошибка при загрузке пользователей:', error);
  }
);
```

// Улучшенная версия с обработкой ошибок

```

function fetchDataWithRetry(url, onSuccess, onError, retries = 3) {
  function attempt(retryCount) {
    fetchData(
      url,
      onSuccess,
      function(error) {
        if (retryCount > 0) {
          console.log(`Повторная попытка ${retries - retryCount + 1}/${retries}`);
          setTimeout(() => attempt(retryCount - 1), 1000);
        } else {
          onError(error);
        }
      }
    );
  }

  attempt(retries);
}

// Пример использования улучшенной версии
fetchDataWithRetry(
  'https://api.example.com/data',
  function(data) {
    console.log('Успешно после повторных попыток:', data);
  },
  function(error) {
    console.error('Все попытки исчерпаны:', error);
  },
  3 // Количество повторных попыток
);

```

Часть 2: Функции как возвращаемые значения

2.1. Фабрики функций

javascript

```

// 1. Простая фабрика функций
function createMultiplier(multiplier) {
  return function(number) {
    return number * multiplier;
  };
}

```



```
}
```

```
const double = createMultiplier(2);
const triple = createMultiplier(3);
const quadruple = createMultiplier(4);
```

```
console.log(double(5));    // 10
console.log(triple(5));    // 15
console.log(quadruple(5)); // 20
```

```
// 2. Фабрика функций с несколькими параметрами
```

```
function createGreeting(greeting) {
  return function(name) {
    return `${greeting}, ${name}!`;
  };
}
```

```
const sayHello = createGreeting('Hello');
const sayHi = createGreeting('Hi');
const sayGoodbye = createGreeting('Goodbye');
```

```
console.log(sayHello('Alice'));    // Hello, Alice!
console.log(sayHi('Bob'));          // Hi, Bob!
console.log(sayGoodbye('Charlie')); // Goodbye, Charlie!
```

```
// 3. Более сложная фабрика функций
```

```
function createLogger(prefix, level = 'INFO') {
  const colors = {
    INFO: '\x1b[36m', // Cyan
    WARN: '\x1b[33m', // Yellow
    ERROR: '\x1b[31m', // Red
    DEBUG: '\x1b[35m' // Magenta
  };

  const reset = '\x1b[0m';
  const color = colors[level] || '\x1b[37m'; // White by default

  return function(message, ...args) {
    const timestamp = new Date().toISOString();
    console.log(
      `${color}[${timestamp}] [${level}] [${prefix}] ${message}${reset}`,
      ...args
    );
  };
}
```

```

}

// Создаем специализированные логиеры
const appLogger = createLogger('APP');
const dbLogger = createLogger('DATABASE');
const apiLogger = createLogger('API', 'DEBUG');
const errorLogger = createLogger('ERROR', 'ERROR');

// Использование
appLogger('Приложение запущено');
dbLogger('Подключение к базе данных установлено', { host: 'localhost' });
apiLogger('Запрос к API', { method: 'GET', url: '/users' });
errorLogger('Критическая ошибка!', new Error('Connection failed'));

```

2.2. Каррирование и частичное применение

```

javascript

// Каррирование - преобразование функции с несколькими аргументами
// в последовательность функций с одним аргументом

// Обычная функция с тремя аргументами
function normalAdd(a, b, c) {
  return a + b + c;
}

// Каррированная версия
function curriedAdd(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}

// Использование
console.log(normalAdd(1, 2, 3));           // 6
console.log(curriedAdd(1)(2)(3));          // 6

// Частичное применение
const addFive = curriedAdd(5);             // Фиксируем первый аргумент
const addFiveAndTwo = addFive(2);          // Фиксируем второй аргумент

console.log(addFiveAndTwo(3));              // 10 (5 + 2 + 3)

```

```
console.log(addFiveAndTwo(10));           // 17 (5 + 2 + 10)
```

```
// Автоматическое каррирование
```

```
function curry(fn) {  
  return function curried(...args) {  
    if (args.length >= fn.length) {  
      return fn.apply(this, args);  
    } else {  
      return function(...args2) {  
        return curried.apply(this, args.concat(args2));  
      };  
    }  
  };  
}
```

```
// Пример с функцией трех аргументов
```

```
function multiplyThree(a, b, c) {  
  return a * b * c;  
}
```

```
const curriedMultiply = curry(multiplyThree);
```

```
console.log(curriedMultiply(2)(3)(4));    // 24  
console.log(curriedMultiply(2, 3)(4));    // 24  
console.log(curriedMultiply(2)(3, 4));    // 24  
console.log(curriedMultiply(2, 3, 4));    // 24
```

```
// Практическое применение каррирования
```

```
// Функция для отправки HTTP запросов
```

```
function createRequest(method) {  
  return function(url) {  
    return function(data) {  
      return function(headers) {  
        console.log(`Отправка ${method} запроса на ${url}`);  
        console.log('Данные:', data);  
        console.log('Заголовки:', headers);  
        return { method, url, data, headers };  
      };  
    };  
  };  
}
```

```
// Создание специализированных функций
```

```
const postRequest = createRequest('POST');
```

```

const getRequest = createRequest('GET');
const putRequest = createRequest('PUT');

// Частичное применение
const postToUsers = postRequest('/api/users');
const postToProducts = postRequest('/api/products');

// Полностью сконфигурированные функции
const createUser = postToUsers({ name: '', email: '' })(
  { 'Content-Type': 'application/json'
});

const updateProduct = putRequest('/api/products/1')({ price: 99.99 })(
  { 'Authorization': 'Bearer token123'
});

// Использование
createUser({ name: 'Alice', email: 'alice@example.com' });
updateProduct();

```

2.3. Мемоизация через функции высшего порядка

javascript

```

// Мемоизация - кэширование результатов вызова функций
function memoize(fn) {
  const cache = new Map();

  return function(...args) {
    // Создаем ключ из аргументов
    const key = JSON.stringify(args);

    // Проверяем кэш
    if (cache.has(key)) {
      console.log('Из кэша:', key);
      return cache.get(key);
    }

    // Вычисляем и кэшируем
    console.log('Вычисляем:', key);
    const result = fn(...args);
    cache.set(key, result);

    return result;
  }
}

```

```
};  
}
```

```
// Пример: дорогая вычисляемая функция
```

```
function expensiveCalculation(n) {  
  console.log(`Выполняется дорогое вычисление для n=${n}...`);  
  
  // Имитация сложных вычислений  
  let result = 0;  
  for (let i = 0; i < 100000000; i++) {  
    result += Math.sin(i) * Math.cos(i);  
  }  
  
  return n * 2;  
}
```

```
// Мемоизируем функцию
```

```
const memoizedCalculation = memoize(expensiveCalculation);
```

```
// Первый вызов - вычисляется
```

```
console.time('Первый вызов');  
console.log(memoizedCalculation(5));  
console.timeEnd('Первый вызов');
```

```
// Второй вызов с теми же аргументами - берется из кэша
```

```
console.time('Второй вызов');  
console.log(memoizedCalculation(5));  
console.timeEnd('Второй вызов');
```

```
// Вызов с другими аргументами - вычисляется заново
```

```
console.time('Третий вызов');  
console.log(memoizedCalculation(10));  
console.timeEnd('Третий вызов');
```

```
// Мемоизация для рекурсивных функций (фибоначчи)
```

```
const memoizedFibonacci = memoize(function fib(n) {  
  if (n <= 1) return n;  
  return memoizedFibonacci(n - 1) + memoizedFibonacci(n - 2);  
});
```

```
console.log(memoizedFibonacci(10)); // 55
```

```
console.log(memoizedFibonacci(40)); // 102334155 (быстро благодаря мемоизации)
```

```
// Мемоизация с ограничением по времени
```

```

function memoizeWithTTL(fn, ttl = 60000) { // TTL в миллисекундах
  const cache = new Map();

  return function(...args) {
    const key = JSON.stringify(args);
    const cached = cache.get(key);

    // Проверяем, не устарели ли кэшированные данные
    if (cached && Date.now() - cached.timestamp < ttl) {
      console.log('Из кэша (свежий):', key);
      return cached.value;
    }

    // Вычисляем и кэшируем с timestamp
    console.log('Вычисляем (новое или устаревшее):', key);
    const value = fn(...args);
    cache.set(key, { value, timestamp: Date.now() });

    return value;
  };
}

// Пример с временем жизни кэша
const getCurrentWeather = memoizeWithTTL(
  function(city) {
    console.log(`Запрос погоды для ${city}...`);
    // Здесь был бы реальный API вызов
    return { city, temperature: Math.random() * 30, humidity: Math.random() * 100 };
  },
  5000 // Кэшировать на 5 секунд
);

// Первый вызов - запрос
console.log(getCurrentWeather('Moscow'));

// Второй вызов в течение 5 секунд - из кэша
setTimeout(() => {
  console.log(getCurrentWeather('Moscow'));
}, 2000);

// Вызов через 6 секунд - новый запрос
setTimeout(() => {
  console.log(getCurrentWeather('Moscow'));
}, 6000);

```

Часть 3: Комбинаторы и функциональные утилиты

3.1. Основные комбинаторы

javascript

// 1. Функция identity - возвращает свой аргумент без изменений

```
function identity(x) {  
  return x;  
}
```

// Использование

```
console.log(identity(42)); // 42  
console.log(identity('test')); // 'test'
```

// Полезность: можно использовать как функцию по умолчанию

```
const numbers = [1, 2, 3, 4, 5];  
console.log(numbers.map(identity)); // [1, 2, 3, 4, 5]
```

// 2. Функция constant - всегда возвращает одно и то же значение

```
function constant(x) {  
  return function() {  
    return x;  
  };  
}
```

```
const alwaysFive = constant(5);  
console.log(alwaysFive()); // 5  
console.log(alwaysFive()); // 5 (всегда 5)
```

// Полезность: создание константных значений в функциональных цепочках

```
const defaultConfig = constant({ host: 'localhost', port: 8080 });  
console.log(defaultConfig()); // {host: 'localhost', port: 8080}
```

// 3. Комбинатор apply (также известен как ap)

```
function apply(fn) {  
  return function(value) {  
    return fn(value);  
  };  
}
```

// Пример

```
const double = x => x * 2;
```

```
const applyDouble = apply(double);
```

```
console.log(applyDouble(5)); // 10
```

```
// 4. Композиция функций
```

```
function compose(...fns) {  
  return function(x) {  
    return fns.reduceRight((acc, fn) => fn(acc), x);  
  };  
}
```

```
function pipe(...fns) {  
  return function(x) {  
    return fns.reduce((acc, fn) => fn(acc), x);  
  };  
}
```

```
// Пример использования
```

```
const add1 = x => x + 1;  
const multiply2 = x => x * 2;  
const square = x => x * x;
```

```
const composed = compose(square, multiply2, add1);  
const piped = pipe(add1, multiply2, square);
```

```
console.log(composed(3)); // ((3 + 1) * 2) ^ 2 = (4 * 2) ^ 2 = 8 ^ 2 = 64  
console.log(piped(3));    // ((3 + 1) * 2) ^ 2 = 64 (тот же результат)
```

```
// 5. Комбинатор tap (для побочных эффектов)
```

```
function tap(fn) {  
  return function(x) {  
    fn(x);  
    return x;  
  };  
}
```

```
// Пример использования в цепочке
```

```
const processNumber = pipe(  
  x => x * 2,  
  tap(x => console.log('После умножения:', x)),  
  x => x + 1,  
  tap(x => console.log('После сложения:', x)),  
  x => x * 3  
);
```



```
console.log(processNumber(5)); // (5*2=10, 10+1=11, 11*3=33)
```

```
// 6. Комбинатор fork (для разделения потока)
```

```
function fork(join, fn1, fn2) {  
  return function(value) {  
    return join(fn1(value), fn2(value));  
  };  
}
```

```
// Пример: вычисление среднего значения
```

```
const sum = arr => arr.reduce((a, b) => a + b, 0);  
const length = arr => arr.length;
```

```
const average = fork(  
  (total, count) => total / count, // join функция  
  sum,                             // fn1  
  length                           // fn2  
);
```

```
const numbersArray = [1, 2, 3, 4, 5];  
console.log(average(numbersArray)); // (1+2+3+4+5)/5 = 3
```

3.2. Функциональные утилиты для работы с коллекциями

javascript

```
// 1. map - трансформация каждого элемента
```

```
function map(arr, fn) {  
  const result = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    result.push(fn(arr[i], i, arr));  
  }  
  
  return result;  
}
```

```
// 2. filter - фильтрация элементов
```

```
function filter(arr, predicate) {  
  const result = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    if (predicate(arr[i], i, arr)) {
```

```
        result.push(arr[i]);
    }
}
```

```
    return result;
}
```

// 3. reduce - агрегация элементов

```
function reduce(arr, reducer, initialValue) {
    let accumulator = initialValue;
    let startIndex = 0;

    // Если initialValue не передан, используем первый элемент
    if (initialValue === undefined && arr.length > 0) {
        accumulator = arr[0];
        startIndex = 1;
    }

    for (let i = startIndex; i < arr.length; i++) {
        accumulator = reducer(accumulator, arr[i], i, arr);
    }

    return accumulator;
}
```

// 4. find - поиск элемента

```
function find(arr, predicate) {
    for (let i = 0; i < arr.length; i++) {
        if (predicate(arr[i], i, arr)) {
            return arr[i];
        }
    }

    return undefined;
}
```

// 5. every - проверка всех элементов

```
function every(arr, predicate) {
    for (let i = 0; i < arr.length; i++) {
        if (!predicate(arr[i], i, arr)) {
            return false;
        }
    }
}
```

```
    return true;
}
```

// 6. some - проверка хотя бы одного элемента

```
function some(arr, predicate) {
    for (let i = 0; i < arr.length; i++) {
        if (predicate(arr[i], i, arr)) {
            return true;
        }
    }
}
```

```
    return false;
}
```

// Пример использования

```
const users = [
    { id: 1, name: 'Alice', age: 25, active: true },
    { id: 2, name: 'Bob', age: 17, active: true },
    { id: 3, name: 'Charlie', age: 30, active: false },
    { id: 4, name: 'Diana', age: 22, active: true }
];
```

// map: получить только имена

```
const names = map(users, user => user.name);
console.log('Имена:', names); // ['Alice', 'Bob', 'Charlie', 'Diana']
```

// filter: активные пользователи

```
const activeUsers = filter(users, user => user.active);
console.log('Активные пользователи:', activeUsers.length); // 3
```

// reduce: средний возраст

```
const averageAge = reduce(
    users,
    (acc, user) => acc + user.age,
    0
) / users.length;
console.log('Средний возраст:', averageAge); // 23.5
```

// find: найти пользователя по имени

```
const charlie = find(users, user => user.name === 'Charlie');
console.log('Найден:', charlie); // {id: 3, name: 'Charlie', ...}
```

// every: все ли пользователи активны?

```
const allActive = every(users, user => user.active);
```

```
console.log('Все активны?', allActive); // false

// some: есть ли несовершеннолетние?
const hasMinors = some(users, user => user.age < 18);
console.log('Есть несовершеннолетние?', hasMinors); // true
```

3.3. Каррированные версии утилит

```
javascript

// Каррированные версии функциональных утилит
const curry = fn => {
  const curried = (...args) => {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...moreArgs) => curried(...args, ...moreArgs);
  };
  return curried;
};

// Каррированный map
const cmap = curry((fn, arr) => arr.map(fn));

// Каррированный filter
const cfilter = curry((predicate, arr) => arr.filter(predicate));

// Каррированный reduce
const creduce = curry((reducer, initialValue, arr) =>
  arr.reduce(reducer, initialValue));

// Примеры использования
const numbers = [1, 2, 3, 4, 5];

// Создание специализированных функций
const doubleAll = cmap(x => x * 2);
const getEvens = cfilter(x => x % 2 === 0);
const sumAll = creduce((acc, x) => acc + x, 0);

console.log(doubleAll(numbers)); // [2, 4, 6, 8, 10]
console.log(getEvens(numbers)); // [2, 4]
console.log(sumAll(numbers)); // 15

// Композиция каррированных функций
```

```

const processNumbers = pipe(
  doubleAll,      // Удвоить все числа
  getEvens,       // Взять только четные
  sumAll          // Посчитать сумму
);

console.log(processNumbers(numbers)); // (1,2,3,4,5) -> (2,4,6,8,10) -> (2,4,6,8,10) ->
30

// Еще примеры с объектами
const users = [
  { name: 'Alice', score: 85 },
  { name: 'Bob', score: 92 },
  { name: 'Charlie', score: 78 },
  { name: 'Diana', score: 95 }
];

const getScores = cmap(user => user.score);
const highScores = cfilter(score => score >= 90);
const averageScore = arr => {
  const sum = creduce((acc, x) => acc + x, 0)(arr);
  return sum / arr.length;
};

const processScores = pipe(
  getScores,
  highScores,
  averageScore
);

console.log(processScores(users)); // Средний балл среди высоких результатов: (92 + 95) /
2 = 93.5

```

Часть 4: Замыкания и функции высшего порядка

4.1. Замыкания в действии

```

javascript

// Замыкание - функция, которая запоминает свое лексическое окружение
function createCounter() {
  let count = 0; // Приватная переменная, доступная только через замыкание

```

```

return {
  increment: function() {
    count++;
    return count;
  },
  decrement: function() {
    count--;
    return count;
  },
  getCount: function() {
    return count;
  },
  reset: function() {
    count = 0;
    return count;
  }
};
}

```

// Создаем несколько независимых счетчиков

```

const counter1 = createCounter();
const counter2 = createCounter();

```

```

console.log(counter1.increment()); // 1
console.log(counter1.increment()); // 2
console.log(counter1.getCount());  // 2

```

```

console.log(counter2.increment()); // 1 (независимо от counter1)
console.log(counter2.decrement()); // 0

```

```

counter1.reset();
console.log(counter1.getCount()); // 0
console.log(counter2.getCount()); // 0 (counter2 не сбросился)

```

// Более сложный пример: кэширующая фабрика

```

function createCachedApi(endpoint) {
  const cache = new Map();
  const maxCacheSize = 100;

  return async function(params) {
    const cacheKey = JSON.stringify({ endpoint, params });

```

// Проверка кэша

```

    if (cache.has(cacheKey)) {
        console.log(`Кэшированный результат для ${endpoint}`);
        return cache.get(cacheKey);
    }

    // Имитация API запроса
    console.log(`Запрос к ${endpoint} с параметрами:`, params);
    await new Promise(resolve => setTimeout(resolve, 1000)); // Имитация задержки

    const result = {
        endpoint,
        params,
        data: `Данные для ${endpoint}`,
        timestamp: Date.now()
    };

    // Сохранение в кэш с ограничением размера
    cache.set(cacheKey, result);

    // Удаление старых записей, если кэш превысил размер
    if (cache.size > maxCacheSize) {
        const firstKey = cache.keys().next().value;
        cache.delete(firstKey);
    }

    return result;
};
}

// Создание кэшированных API функций
const getUserData = createCachedApi('/api/users');
const getProductData = createCachedApi('/api/products');

// Использование
(async () => {
    console.log('Первый запрос пользователей:');
    console.log(await getUserData({ id: 1 }));

    console.log('\nВторой запрос (должен быть из кэша):');
    console.log(await getUserData({ id: 1 }));

    console.log('\nЗапрос продуктов:');
    console.log(await getProductData({ category: 'electronics' }));
})();

```

4.2. Паттерн "Модуль" с замыканиями

javascript

// Модульный паттерн с функциями высшего порядка

```
const UserModule = (function() {  
  // Приватные переменные и функции  
  let users = [];  
  let nextId = 1;  
  
  function validateUser(user) {  
    return user &&  
      typeof user.name === 'string' &&  
      user.name.trim() !== '' &&  
      typeof user.email === 'string' &&  
      user.email.includes('@');  
  }  
  
  function findUserIndex(id) {  
    return users.findIndex(user => user.id === id);  
  }  
  
  // Публичный API  
  return {  
    // Добавление пользователя  
    addUser: function(userData) {  
      if (!validateUser(userData)) {  
        throw new Error('Некорректные данные пользователя');  
      }  
  
      const user = {  
        ...userData,  
        id: nextId++,  
        createdAt: new Date(),  
        isActive: true  
      };  
  
      users.push(user);  
      return { ...user }; // Возвращаем копию  
    },  
  
    // Получение пользователя по ID  
    getUser: function(id) {  
      const index = findUserIndex(id);
```



```

    if (index === -1) return null;
    return { ...users[index] }; // Возвращаем копию
},

// Получение всех пользователей
getAllUsers: function() {
    return users.map(user => ({ ...user })); // Возвращаем копии
},

// Обновление пользователя
updateUser: function(id, updates) {
    const index = findUserIndex(id);
    if (index === -1) return null;

    // Проверяем обновления
    if (updates.name && typeof updates.name !== 'string') {
        throw new Error('Имя должно быть строкой');
    }

    if (updates.email && !updates.email.includes('@')) {
        throw new Error('Некорректный email');
    }

    users[index] = {
        ...users[index],
        ...updates,
        updatedAt: new Date()
    };

    return { ...users[index] };
},

// Удаление пользователя (мягкое удаление)
deleteUser: function(id) {
    const index = findUserIndex(id);
    if (index === -1) return false;

    users[index].isActive = false;
    users[index].deletedAt = new Date();

    return true;
},

// Поиск пользователей

```

```

findUsers: function(criteria) {
  return users
    .filter(user => {
      if (criteria.active !== undefined && user.isActive !== criteria.active) {
        return false;
      }

      if (criteria.name && !
user.name.toLowerCase().includes(criteria.name.toLowerCase())) {
        return false;
      }

      if (criteria.email && !
user.email.toLowerCase().includes(criteria.email.toLowerCase())) {
        return false;
      }

      return true;
    })
    .map(user => ({ ...user }));
},

```

// Статистика

```

getStats: function() {
  const active = users.filter(u => u.isActive).length;
  const inactive = users.filter(u => !u.isActive).length;
  const total = users.length;

  return {
    total,
    active,
    inactive,
    activePercentage: total > 0 ? (active / total * 100).toFixed(2) : 0
  };
}
};
})();

```

// Использование модуля

```

console.log('Добавление пользователей:');
const user1 = UserModule.addUser({ name: 'Alice', email: 'alice@example.com' });
const user2 = UserModule.addUser({ name: 'Bob', email: 'bob@example.com' });
console.log(user1, user2);

```

```
console.log('\nПолучение всех пользователей:');
console.log(UserModule.getAllUsers());

console.log('\nПоиск пользователей:');
console.log(UserModule.findUsers({ active: true }));

console.log('\nОбновление пользователя:');
const updated = UserModule.updateUser(1, { name: 'Alice Smith' });
console.log(updated);

console.log('\nСтатистика:');
console.log(UserModule.getStats());

console.log('\nУдаление пользователя:');
UserModule.deleteUser(2);
console.log(UserModule.getAllUsers());

console.log('\nПопытка добавить некорректного пользователя:');
try {
  UserModule.addUser({ name: '', email: 'invalid' });
} catch (error) {
  console.error('Ошибка:', error.message);
}
```

Часть 5: Промисы и `async/await` как функции высшего порядка

5.1. Промисы как функции высшего порядка

```
javascript

// Создание промисов
function createPromise(value, delay = 1000, shouldReject = false) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldReject) {
        reject(new Error(`Ошибка для значения: ${value}`));
      } else {
        resolve(value);
      }
    }, delay);
  });
}
```

```
}
```

```
// Функции высшего порядка для работы с промисами
```

```
// 1. Последовательное выполнение
```

```
function sequentialPromise(promiseCreators) {  
  return promiseCreators.reduce((chain, promiseCreator) => {  
    return chain.then(previousResult => {  
      console.log('Предыдущий результат:', previousResult);  
      return promiseCreator();  
    });  
  }, Promise.resolve());  
}
```

```
// Пример последовательного выполнения
```

```
sequentialPromise([  
  () => createPromise('Первый результат', 500),  
  () => createPromise('Второй результат', 300),  
  () => createPromise('Третий результат', 200)  
)  
.then(finalResult => {  
  console.log('Финальный результат:', finalResult);  
)  
.catch(error => {  
  console.error('Ошибка в цепочке:', error);  
});
```

```
// 2. Параллельное выполнение с ограничением
```

```
function parallelWithLimit(promises, limit) {  
  return new Promise((resolve, reject) => {  
    let index = 0;  
    let running = 0;  
    let completed = 0;  
    const results = new Array(promises.length);  
    let hasError = false;  
  
    function runNext() {  
      // Если уже была ошибка или все промисы запущены  
      if (hasError || index >= promises.length) return;  
  
      // Если уже работает максимальное количество промисов  
      if (running >= limit) return;  
  
      const currentIndex = index;
```

```

const promise = promises[currentIndex];
index++;
running++;

promise()
  .then(result => {
    results[currentIndex] = result;
  })
  .catch(error => {
    results[currentIndex] = { error: error.message };
    hasError = true;
  })
  .finally(() => {
    running--;
    completed++;

    if (completed === promises.length) {
      if (hasError) {
        reject(new Error('Один или несколько промисов завершились с ошибкой'));
      } else {
        resolve(results);
      }
    } else {
      runNext(); // Запускаем следующий промис
    }

    // Проверяем, можно ли запустить еще промисы
    runNext();
  });

// Пытаемся запустить еще промисы
runNext();
}

// Начальный запуск
runNext();
});
}

// Пример параллельного выполнения с ограничением
const promiseCreators = Array.from({ length: 10 }, (_, i) =>
  () => createPromise(`Результат ${i + 1}`, Math.random() * 2000)
);

```

```

parallelWithLimit(promiseCreators, 3)
  .then(results => {
    console.log('Все результаты:', results);
  })
  .catch(error => {
    console.error('Ошибка:', error);
  });

```

// 3. Таймаут для промисов

```

function withTimeout(promise, timeoutMs, timeoutMessage = 'Таймаут') {
  return Promise.race([
    promise,
    new Promise((_, reject) => {
      setTimeout(() => reject(new Error(timeoutMessage)), timeoutMs);
    })
  ]);
}

```

// Пример с таймаутом

```

const slowPromise = createPromise('Медленный результат', 3000);
const fastPromise = createPromise('Быстрый результат', 500);

withTimeout(slowPromise, 2000, 'Слишком долго!')
  .then(result => console.log('Успех:', result))
  .catch(error => console.error('Ошибка:', error.message)); // Таймаут

withTimeout(fastPromise, 2000)
  .then(result => console.log('Успех:', result)) // Успех
  .catch(error => console.error('Ошибка:', error.message));

```

// 4. Повтор попыток для неудачных промисов

```

function retryPromise(promiseCreator, maxRetries = 3, delayMs = 1000) {
  return new Promise((resolve, reject) => {
    let attempts = 0;

    function attempt() {
      attempts++;

      promiseCreator()
        .then(resolve)
        .catch(error => {
          console.log(`Попытка ${attempts}/${maxRetries} не удалась:`, error.message);

          if (attempts >= maxRetries) {

```

```

        reject(new Error(`Все ${maxRetries} попыток не удалось. Последняя ошибка:
${error.message}`));
    } else {
        setTimeout(attempt, delayMs);
    }
});
}

    attempt();
});
}

```

// Пример с повторными попытками

```

let attemptCount = 0;
const unreliablePromise = () => {
    attemptCount++;
    const shouldFail = attemptCount < 3; // Первые две попытки неудачны
    return createPromise(`Успех на попытке ${attemptCount}`, 500, shouldFail);
};

```

```

retryPromise(unreliablePromise, 5, 500)
    .then(result => console.log('Финальный успех:', result))
    .catch(error => console.error('Финальная ошибка:', error.message));

```

5.2. Async/await с функциями высшего порядка

javascript

// Функции высшего порядка для async/await

// 1. Обработка ошибок в async функциях

```

function asyncHandler(fn) {
    return async function(...args) {
        try {
            const result = await fn(...args);
            return [null, result];
        } catch (error) {
            return [error, null];
        }
    };
}

```

// Пример использования

```

async function fetchUserData(userId) {

```

```

if (!userId) throw new Error('ID пользователя обязателен');

// Имитация API запроса
await new Promise(resolve => setTimeout(resolve, 1000));

return {
  id: userId,
  name: 'John Doe',
  email: 'john@example.com'
};
}

```

```

const safeFetchUserData = asyncHandler(fetchUserData);

```

```

// Использование без try/catch
async function processUser() {
  const [error1, user1] = await safeFetchUserData(123);
  if (error1) {
    console.error('Ошибка 1:', error1.message);
  } else {
    console.log('Пользователь 1:', user1);
  }

  const [error2, user2] = await safeFetchUserData();
  if (error2) {
    console.error('Ошибка 2:', error2.message);
  } else {
    console.log('Пользователь 2:', user2);
  }
}

```

```

processUser();

```

```

// 2. Кэширование результатов асинх функций
function cacheAsync(fn, ttl = 60000) {
  const cache = new Map();

  return async function(...args) {
    const key = JSON.stringify(args);
    const cached = cache.get(key);

    if (cached && Date.now() - cached.timestamp < ttl) {
      console.log('Из кэша:', key);
      return cached.value;
    }
  };
}

```



```

    }

    console.log('Выполнение:', key);
    const value = await fn(...args);
    cache.set(key, { value, timestamp: Date.now() });

    return value;
  };
}

```

// Пример

```

const cachedFetch = cacheAsync(async function(url) {
  console.log(`Запрос к ${url}...`);
  await new Promise(resolve => setTimeout(resolve, 2000));
  return { data: `Данные с ${url}`, timestamp: Date.now() };
}, 5000);

```

// Использование

```

(async () => {
  console.log(await cachedFetch('/api/users'));
  console.log(await cachedFetch('/api/users')); // Из кэша
  console.log(await cachedFetch('/api/products'));

  // Ждем 6 секунд и снова запрашиваем
  await new Promise(resolve => setTimeout(resolve, 6000));
  console.log(await cachedFetch('/api/users')); // Снова выполнение
})();

```

// 3. Ограничение скорости выполнения (rate limiting)

```

function rateLimit(fn, limit, interval) {
  const queue = [];
  let inProgress = 0;
  let lastReset = Date.now();
  let tokens = limit;

  // Функция для сброса токенов
  function resetTokens() {
    const now = Date.now();
    const timePassed = now - lastReset;
    const intervalsPassed = Math.floor(timePassed / interval);

    if (intervalsPassed > 0) {
      tokens = Math.min(limit, tokens + intervalsPassed * limit);
      lastReset = now;
    }
  }
}

```

```

    }
}

// Функция для обработки очереди
function processQueue() {
    resetTokens();

    while (queue.length > 0 && tokens > 0 && inProgress < limit) {
        tokens--;
        inProgress++;

        const { args, resolve, reject } = queue.shift();

        fn(...args)
            .then(result => {
                inProgress--;
                resolve(result);
                processQueue();
            })
            .catch(error => {
                inProgress--;
                reject(error);
                processQueue();
            });
    }

    // Если в очереди еще есть элементы, но нет токенов,
    // планируем следующую проверку
    if (queue.length > 0 && tokens <= 0) {
        const nextReset = lastReset + interval - (Date.now() - lastReset);
        setTimeout(processQueue, Math.max(0, nextReset));
    }
}

return function(...args) {
    return new Promise((resolve, reject) => {
        queue.push({ args, resolve, reject });
        processQueue();
    });
};
}

// Пример
const limitedApiCall = rateLimit(

```

```

async function(endpoint) {
  console.log(`Вызов ${endpoint} в ${new Date().toLocaleTimeString()}`);
  await new Promise(resolve => setTimeout(resolve, 500));
  return `Результат ${endpoint}`;
},
2, // 2 вызова
3000 // каждые 3 секунды
);

// Множественные вызовы
(async () => {
  const promises = [];

  for (let i = 1; i <= 10; i++) {
    promises.push(limitedApiCall(`/api/call${i}`));
  }

  const results = await Promise.all(promises);
  console.log('Все результаты:', results);
})();

```

Часть 6: Практическое применение в реальных проектах

6.1. Валидация данных

```

javascript

// Система валидации с функциями высшего порядка
function createValidator() {
  const validators = [];

  return {
    // Добавление правила валидации
    addRule: function(rule, message) {
      validators.push({ rule, message });
      return this; // Для цепочки вызовов
    },

    // Выполнение валидации
    validate: function(value) {
      const errors = [];

```

```

    for (const { rule, message } of validators) {
      if (!rule(value)) {
        errors.push(message);
      }
    }
  }
}

```

```

    return {
      isValid: errors.length === 0,
      errors,
      value
    };
  },
}

```

// Создание предварительно настроенного валидатора

```

createPreset: function() {
  const presetValidators = [...validators];
  return function(value) {
    const errors = [];

    for (const { rule, message } of presetValidators) {
      if (!rule(value)) {
        errors.push(message);
      }
    }

    return {
      isValid: errors.length === 0,
      errors,
      value
    };
  };
}
};
}

```

// Создание валидаторов

```

const emailValidator = createValidator()
  .addRule(value => value && value.includes('@'), 'Email должен содержать @')
  .addRule(value => value && value.length > 5, 'Email должен быть длиннее 5 символов')
  .addRule(value => value && value.endsWith('.com') || value.endsWith('.ru'), 'Email должен оканчиваться на .com или .ru')
  .createPreset();

const passwordValidator = createValidator()

```

```

.addRule(value => value && value.length >= 8, 'Пароль должен быть не менее 8 символов')
.addRule(value => value && /[A-Z]/.test(value), 'Пароль должен содержать заглавную
букву')
.addRule(value => value && /[0-9]/.test(value), 'Пароль должен содержать цифру')
.addRule(value => value && /[!@#$$%^&*/.test(value), 'Пароль должен содержать
специальный символ')
.createPreset();

```

```

const ageValidator = createValidator()
.addRule(value => !isNaN(value), 'Возраст должен быть числом')
.addRule(value => value >= 0, 'Возраст не может быть отрицательным')
.addRule(value => value <= 120, 'Возраст не может быть больше 120')
.createPreset();

```

// Использование валидаторов

```

console.log('Валидация email:');
console.log(emailValidator('test@example.com'));
console.log(emailValidator('invalid'));
console.log(emailValidator('test@example.org'));

```

```

console.log('\nВалидация пароля:');
console.log(passwordValidator('Weak'));
console.log(passwordValidator('StrongPass1!'));

```

```

console.log('\nВалидация возраста:');
console.log(ageValidator(25));
console.log(ageValidator(-5));
console.log(ageValidator(150));

```

// Композиция валидаторов

```

function createFormValidator(fieldValidators) {
  return function(formData) {
    const errors = {};
    let isValid = true;

    for (const [fieldName, validator] of Object.entries(fieldValidators)) {
      const fieldValue = formData[fieldName];
      const result = validator(fieldValue);

      if (!result.isValid) {
        errors[fieldName] = result.errors;
        isValid = false;
      }
    }
  }
}

```

```

        return {
            isValid,
            errors,
            data: formData
        };
    };
}

// Создание валидатора формы
const validateRegistrationForm = createFormValidator({
    email: emailValidator,
    password: passwordValidator,
    age: ageValidator
});

// Тестирование валидатора формы
console.log('\nВалидация формы:');
console.log(validateRegistrationForm({
    email: 'test@example.com',
    password: 'StrongPass1!',
    age: 25
}));

console.log(validateRegistrationForm({
    email: 'invalid',
    password: 'weak',
    age: -5
}));

```

6.2. Middleware система (подобно Express.js)

javascript

```

// Простая система middleware с функциями высшего порядка
function createMiddlewareSystem() {
    const middlewares = [];

    function compose(middlewareChain) {
        return function(context, next) {
            let index = -1;

            function dispatch(i) {
                if (i <= index) {

```

```

        return Promise.reject(new Error('next() вызван несколько раз'));
    }

    index = i;
    let fn = middlewareChain[i];

    // Если это последний middleware, вызываем next (если предоставлен)
    if (i === middlewareChain.length) {
        fn = next;
    }

    // Если middleware нет, возвращаем успешный промис
    if (!fn) {
        return Promise.resolve();
    }

    try {
        return Promise.resolve(
            fn(context, function next() {
                return dispatch(i + 1);
            })
        );
    } catch (err) {
        return Promise.reject(err);
    }
}

return dispatch(0);
};
}

return {
    // Добавление middleware
    use: function(middleware) {
        if (typeof middleware !== 'function') {
            throw new Error('Middleware должен быть функцией');
        }

        middlewares.push(middleware);
        return this; // Для цепочки вызовов
    },

    // Обработка запроса
    handleRequest: async function(initialContext) {

```

```

const context = {
  ...initialContext,
  startedAt: Date.now()
};

try {
  await compose(middlewares)(context);
  console.log('Запрос успешно обработан');
  return context;
} catch (error) {
  console.error('Ошибка обработки запроса:', error);
  context.error = error;
  return context;
}
},

// Создание обработчика для конкретного пути
createRouteHandler: function(path, ...routeMiddlewares) {
  const routeMiddlewareChain = [...routeMiddlewares];

  return async function(req) {
    // Проверяем, соответствует ли путь
    if (req.path !== path) {
      return null; // Этот обработчик не подходит
    }

    console.log(`Обработка пути: ${path}`);

    const context = {
      ...req,
      startedAt: Date.now()
    };

    try {
      await compose(routeMiddlewareChain)(context);
      return context;
    } catch (error) {
      console.error(`Ошибка обработки пути ${path}:`, error);
      context.error = error;
      return context;
    }
  };
}
};

```



```
}
```

```
// Создание middleware системы
```

```
const app = createMiddlewareSystem();
```

```
// Глобальные middleware
```

```
app.use(async function logger(context, next) {  
  console.log(`[${new Date().toISOString()}] ${context.method} ${context.path}`);  
  context.logger = { startTime: Date.now() };  
  
  await next();  
  
  const duration = Date.now() - context.logger.startTime;  
  console.log(`Запрос завершен за ${duration}мс`);  
});
```

```
app.use(async function authentication(context, next) {  
  console.log('Проверка аутентификации...');
```

```
  // Имитация проверки токена
```

```
  if (context.headers?.authorization === 'Bearer valid-token') {  
    context.user = { id: 1, name: 'John Doe' };  
    await next();  
  } else {  
    throw new Error('Неавторизован');  
  }  
});
```

```
app.use(async function bodyParser(context, next) {  
  console.log('Парсинг тела запроса...');
```

```
  if (context.body && typeof context.body === 'string') {  
    try {  
      context.parsedBody = JSON.parse(context.body);  
    } catch {  
      context.parsedBody = {};  
    }  
  }  
}
```

```
  await next();  
});
```

```
// Создание обработчиков маршрутов
```

```
const userRoutes = {
```

```

getUsers: app.createRouteHandler('/api/users', async function(context, next) {
  console.log('Получение списка пользователей...');

  // Имитация работы с базой данных
  await new Promise(resolve => setTimeout(resolve, 500));

  context.response = {
    data: [
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' }
    ],
    total: 2
  };

  await next();
}),

createUser: app.createRouteHandler('/api/users', async function(context, next) {
  if (context.method !== 'POST') return await next();

  console.log('Создание пользователя...');

  if (!context.parsedBody?.name) {
    throw new Error('Имя пользователя обязательно');
  }

  // Имитация создания пользователя
  await new Promise(resolve => setTimeout(resolve, 300));

  context.response = {
    id: Date.now(),
    name: context.parsedBody.name,
    createdAt: new Date().toISOString()
  };

  await next();
})
};

// Обработка запросов
(async () => {
  console.log('=== Пример 1: Успешный запрос ===');
  const result1 = await app.handleRequest({
    method: 'GET',
  });

```

```

    path: '/api/users',
    headers: {
      authorization: 'Bearer valid-token'
    }
  });
console.log('Результат 1:', result1.response);

console.log('\n=== Пример 2: Создание пользователя ===');
const result2 = await app.handleRequest({
  method: 'POST',
  path: '/api/users',
  headers: {
    authorization: 'Bearer valid-token'
  },
  body: JSON.stringify({ name: 'Charlie' })
});
console.log('Результат 2:', result2.response);

console.log('\n=== Пример 3: Неавторизованный запрос ===');
const result3 = await app.handleRequest({
  method: 'GET',
  path: '/api/users',
  headers: {
    authorization: 'Bearer invalid-token'
  }
});
console.log('Результат 3:', result3.error?.message);

console.log('\n=== Пример 4: Использование обработчиков маршрутов ===');
const handlerResult = await userRoutes.getUsers({
  method: 'GET',
  path: '/api/users',
  headers: { authorization: 'Bearer valid-token' }
});
console.log('Результат обработчика:', handlerResult?.response);
})();

```

Часть 7: Производительность и оптимизация

7.1. Бенчмаркинг функций высшего порядка

javascript

```
// Сравнение производительности различных подходов
function benchmark(name, fn, iterations = 1000000) {
  console.log(`\nБенчмарк: ${name}`);
  console.log(`Итераций: ${iterations.toLocaleString()}`);

  // Прогрев (warm-up)
  for (let i = 0; i < 1000; i++) {
    fn();
  }

  // Основной замер
  const startTime = performance.now();

  for (let i = 0; i < iterations; i++) {
    fn();
  }

  const endTime = performance.now();
  const duration = endTime - startTime;
  const opsPerSecond = (iterations / duration * 1000).toLocaleString('ru-RU', {
    maximumFractionDigits: 0
  });

  console.log(`Время: ${duration.toFixed(2)}мс`);
  console.log(`Операций в секунду: ${opsPerSecond}`);

  return duration;
}

// Тестовые данные
const testArray = Array.from({ length: 1000 }, (_, i) => i + 1);
const testObject = { a: 1, b: 2, c: 3, d: 4, e: 5 };

// 1. Сравнение циклов и методов массива
benchmark('for loop', () => {
  let sum = 0;
  for (let i = 0; i < testArray.length; i++) {
    sum += testArray[i];
  }
  return sum;
});

benchmark('forEach', () => {
  let sum = 0;
```

```

testArray.forEach(num => {
    sum += num;
});
return sum;
});

benchmark('reduce', () => {
    return testArray.reduce((sum, num) => sum + num, 0);
});

// 2. Сравнение функций высшего порядка и прямых вызовов
function add(a, b) {
    return a + b;
}

function higherOrderOperation(a, b, operation) {
    return operation(a, b);
}

benchmark('Прямой вызов функции', () => {
    return add(5, 3);
});

benchmark('Вызов через функцию высшего порядка', () => {
    return higherOrderOperation(5, 3, add);
});

// 3. Сравнение замыканий и классов
benchmark('Замыкание', () => {
    const counter = (function() {
        let count = 0;

        return {
            increment: function() {
                count++;
                return count;
            },
            getCount: function() {
                return count;
            }
        };
    })();

    counter.increment();

```

```
    return counter.getCount();
});
```

```
benchmark('Класс', () => {
  class Counter {
    constructor() {
      this.count = 0;
    }

    increment() {
      this.count++;
      return this.count;
    }

    getCount() {
      return this.count;
    }
  }
}
```

```
const counter = new Counter();
counter.increment();
return counter.getCount();
});
```

// 4. Мемоизация и производительность

```
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
```

```
const memoizedFibonacci = (function() {
  const cache = new Map();

  return function fib(n) {
    if (cache.has(n)) {
      return cache.get(n);
    }

    let result;
    if (n <= 1) {
      result = n;
    } else {
      result = fib(n - 1) + fib(n - 2);
    }
  }
}
```

```

        cache.set(n, result);
        return result;
    };
})();

console.log('\n=== Сравнение Fibonacci ===');
console.log('fibonacci(30) без мемоизации:');
const start1 = performance.now();
console.log('Результат:', fibonacci(30));
const end1 = performance.now();
console.log(`Время: ${((end1 - start1).toFixed(2))}мс`);

console.log('\nfibonacci(30) с мемоизацией:');
const start2 = performance.now();
console.log('Результат:', memoizedFibonacci(30));
const end2 = performance.now();
console.log(`Время: ${((end2 - start2).toFixed(2))}мс`);

```

// 5. Каррирование и производительность

```

function normalSum(a, b, c, d) {
    return a + b + c + d;
}

function curriedSum(a) {
    return function(b) {
        return function(c) {
            return function(d) {
                return a + b + c + d;
            };
        };
    };
}

benchmark('Обычная функция (4 аргумента)', () => {
    return normalSum(1, 2, 3, 4);
});

benchmark('Каррированная функция', () => {
    return curriedSum(1)(2)(3)(4);
});

```

// 6. Оптимизация: предварительное связывание (binding)

```

const obj = {

```

```

    value: 42,
    getValue: function() {
        return this.value;
    }
};

benchmark('Вызов метода с bind каждый раз', () => {
    const boundFn = obj.getValue.bind(obj);
    return boundFn();
});

const preBoundFn = obj.getValue.bind(obj);
benchmark('Вызов предварительно связанного метода', () => {
    return preBoundFn();
});

// Выводы о производительности
console.log('\n=== Выводы о производительности ===');
console.log('1. Прямые циклы (for) обычно быстрее методов массива');
console.log('2. Прямые вызовы функций быстрее вызовов через HOF');
console.log('3. Мемоизация значительно ускоряет рекурсивные вычисления');
console.log('4. Предварительное связывание (bind) улучшает производительность');
console.log('5. Каррирование добавляет накладные расходы, но улучшает переиспользуемость');

```

7.2. Оптимизация памяти

```

javascript

// Мониторинг использования памяти
function getMemoryUsage() {
    if (global.gc) {
        global.gc(); // Запускаем сборку мусора (только с флагом --expose-gc)
    }

    const used = process.memoryUsage();
    return {
        rss: Math.round(used.rss / 1024 / 1024), // Resident Set Size
        heapTotal: Math.round(used.heapTotal / 1024 / 1024),
        heapUsed: Math.round(used.heapUsed / 1024 / 1024),
        external: Math.round(used.external / 1024 / 1024)
    };
}

```



```
console.log('Начальное использование памяти:', getMemoryUsage());
```

```
// Тест 1: Создание множества замыканий
```

```
console.log('\n=== Тест 1: Множество замыканий ===');
```

```
const startMemory1 = getMemoryUsage();
```

```
const closures = [];
```

```
for (let i = 0; i < 10000; i++) {
```

```
  closures.push((function(value) {
```

```
    let counter = 0;
```

```
    return {
```

```
      increment: function() {
```

```
        counter += value;
```

```
        return counter;
```

```
      },
```

```
      getValue: function() {
```

```
        return value;
```

```
      }
```

```
    };
```

```
  })(i));
```

```
}
```

```
const endMemory1 = getMemoryUsage();
```

```
console.log('Использовано памяти:', {
```

```
  rss: endMemory1.rss - startMemory1.rss + 'MB',
```

```
  heapUsed: endMemory1.heapUsed - startMemory1.heapUsed + 'MB'
```

```
});
```

```
// Тест 2: Оптимизация через общие функции
```

```
console.log('\n=== Тест 2: Общие функции в замыканиях ===');
```

```
const startMemory2 = getMemoryUsage();
```

```
// Выносим общие функции наружу
```

```
function createIncrementor(value) {
```

```
  let counter = 0;
```

```
  return {
```

```
    increment: incrementFunction,
```

```
    getValue: getValueFunction,
```

```
    getCounter: function() {
```

```
      return counter;
```

```
    }
```

```
  };
```

```

function incrementFunction() {
    counter += value;
    return counter;
}

function getValueFunction() {
    return value;
}
}

const optimizedClosures = [];
for (let i = 0; i < 10000; i++) {
    optimizedClosures.push(createIncrementor(i));
}

const endMemory2 = getMemoryUsage();
console.log('Использовано памяти:', {
    rss: endMemory2.rss - startMemory2.rss + 'MB',
    heapUsed: endMemory2.heapUsed - startMemory2.heapUsed + 'MB'
});

// Тест 3: Утечки памяти в замыканиях
console.log('\n=== Тест 3: Поиск утечек памяти ===');
let leakedReferences = [];

function createLeakyClosure() {
    const largeArray = new Array(10000).fill('leak');
    let privateData = 'секретные данные';

    return function() {
        // Случайная утечка: сохраняем ссылку на внешние данные
        leakedReferences.push(largeArray);

        console.log('Выполнено, privateData:', privateData);
        return largeArray.length;
    };
}

// Создаем и немедленно вызываем
for (let i = 0; i < 10; i++) {
    const leakyFn = createLeakyClosure();
    leakyFn();
}

```

```

console.log('Утекло ссылок:', leakedReferences.length);
console.log('Память после утечек:', getMemoryUsage());

// Очистка утечек
leakedReferences = [];
if (global.gc) global.gc();
console.log('Память после очистки:', getMemoryUsage());

// Тест 4: Оптимизация через WeakMap для приватных данных
console.log('\n=== Тест 4: WeakMap для приватных данных ===');
const privateData = new WeakMap();

function createOptimizedClosure(value) {
  const obj = {};
  privateData.set(obj, {
    counter: 0,
    value: value
  });

  return {
    increment: function() {
      const data = privateData.get(obj);
      data.counter += data.value;
      return data.counter;
    },
    getValue: function() {
      return privateData.get(obj).value;
    }
  };
}

const optimized = [];
const startMemory4 = getMemoryUsage();

for (let i = 0; i < 10000; i++) {
  optimized.push(createOptimizedClosure(i));
}

const endMemory4 = getMemoryUsage();
console.log('Использовано памяти с WeakMap:', {
  rss: endMemory4.rss - startMemory4.rss + 'MB',
  heapUsed: endMemory4.heapUsed - startMemory4.heapUsed + 'MB'
});

```

// Рекомендации по оптимизации памяти

```
console.log('\n=== Рекомендации по оптимизации памяти ===');
console.log('1. Используйте общие функции вместо создания новых в каждом замыкании');
console.log('2. Остерегайтесь случайных ссылок на внешние данные в замыканиях');
console.log('3. Используйте WeakMap/WeakSet для приватных данных, если нужно');
console.log('4. Очищайте ссылки, когда они больше не нужны');
console.log('5. Используйте инструменты профилирования (Chrome DevTools, node --inspect)');
```

Часть 8: Лучшие практики и антипаттерны

8.1. Лучшие практики использования функций высшего порядка

javascript

// 1. Именованные функции вместо анонимных

// ✗ Плохо

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(n => n * 2);
```

// ☑ Хорошо

```
function doubleNumber(n) {
  return n * 2;
}
```

```
const doubledBetter = numbers.map(doubleNumber);
```

// Еще лучше с осмысленным именем

```
function calculateDouble(n) {
  return n * 2;
}
```

// 2. Композиция вместо вложенности

// ✗ Плохо (слишком много вложенности)

```
const result = users
  .filter(user => user.age >= 18)
  .map(user => ({
    ...user,
    fullName: `${user.firstName} ${user.lastName}`
  })))
```

```

    .filter(user => user.isActive)
    .map(user => user.fullName.toUpperCase());

// 🟢 Хорошо (разделение на осмысленные шаги)
const isAdult = user => user.age >= 18;
const addFullName = user => ({
    ...user,
    fullName: `${user.firstName} ${user.lastName}`
});
const isActive = user => user.isActive;
const getUppercaseFullName = user => user.fullName.toUpperCase();

const goodResult = users
    .filter(isAdult)
    .map(addFullName)
    .filter(isActive)
    .map(getUppercaseFullName);

// 3. Избегайте побочных эффектов в функциях высшего порядка
// ✗ Плохо (побочный эффект)
const logs = [];
const processed = numbers.map(n => {
    logs.push(`Обработано: ${n}`); // Побочный эффект!
    return n * 2;
});

// 🟢 Хорошо (чистая функция)
const pureDouble = n => n * 2;
const processedPure = numbers.map(pureDouble);

// Если нужны логи, используйте tap
function tap(fn) {
    return function(x) {
        fn(x);
        return x;
    };
}

const processedWithLogs = numbers
    .map(tap(n => console.log(`Обработка: ${n}`)))
    .map(pureDouble);

// 4. Каррирование для улучшения переиспользуемости
// ✗ Плохо (жестко закодированные значения)

```

```

function sendEmail(to, subject, body) {
  console.log(`Отправка email: ${to}, ${subject}, ${body}`);
}

// Каждый раз нужно передавать все параметры
sendEmail('user@example.com', 'Приветствие', 'Добро пожаловать!');
sendEmail('admin@example.com', 'Уведомление', 'Новый пользователь');

// 🟢 Хорошо (карирование для создания специализированных функций)
function createEmailSender(from) {
  return function(subjectTemplate) {
    return function(to, context) {
      const subject = subjectTemplate.replace(/\{(\w+)\}/g,
        (_, key) => context[key] || '');
    };

    return {
      from,
      to,
      subject,
      body: `Отправлено от ${from}`
    };
  };
};

// Создание специализированных функций
const systemEmail = createEmailSender('system@example.com');
const welcomeEmail = systemEmail('Добро пожаловать, {name}!');
const notificationEmail = systemEmail('Уведомление: {event}');

// Использование специализированных функций
console.log(welcomeEmail('alice@example.com', { name: 'Алиса' }));
console.log(notificationEmail('admin@example.com', { event: 'Новая регистрация' }));

// 5. Обработка ошибок в функциях высшего порядка
// ✖ Плохо (игнорирование ошибок)
function unsafeOperation(data, processor) {
  return processor(data); // Может выбросить ошибку
}

// 🟢 Хорошо (безопасная обертка)
function safeOperation(data, processor) {
  try {

```

```

    return {
      success: true,
      result: processor(data)
    };
  } catch (error) {
    return {
      success: false,
      error: error.message
    };
  }
}

```

// Еще лучше с поддержкой async

```

async function safeAsyncOperation(data, asyncProcessor) {
  try {
    const result = await asyncProcessor(data);
    return { success: true, result };
  } catch (error) {
    return { success: false, error: error.message };
  }
}

```

// 6. Документирование функций высшего порядка

*/***

** Создает функцию для фильтрации массива по указанному критерию*

** @param {Function} predicate - Функция-предикат, возвращающая true/false*

** @returns {Function} Функция, принимающая массив и возвращающая отфильтрованный массив*

** @example*

** const isEven = n => n % 2 === 0;*

** const filterEven = createFilter(isEven);*

** console.log(filterEven([1, 2, 3, 4])); // [2, 4]*

**/*

```

function createFilter(predicate) {
  return function filterArray(arr) {
    if (!Array.isArray(arr)) {
      throw new TypeError('Ожидается массив');
    }

    return arr.filter(predicate);
  };
}

```

```

// 7. Тестирование функций высшего порядка
// Функция для тестирования
function mapWithIndex(arr, fn) {
  return arr.map((item, index) => fn(item, index, arr));
}

// Тесты
function testMapWithIndex() {
  const testCases = [
    {
      name: 'Удвоение с индексом',
      input: [1, 2, 3],
      fn: (num, index) => num * (index + 1),
      expected: [1, 4, 9] // 1*1, 2*2, 3*3
    },
    {
      name: 'Строки с префиксом',
      input: ['a', 'b', 'c'],
      fn: (letter, index) => `${index}:${letter}`,
      expected: ['0:a', '1:b', '2:c']
    }
  ];

  let allPassed = true;

  for (const testCase of testCases) {
    const result = mapWithIndex(testCase.input, testCase.fn);
    const passed = JSON.stringify(result) === JSON.stringify(testCase.expected);

    console.log(`${testCase.name}: ${passed ? '✓' : '✗'}`);

    if (!passed) {
      console.log(` Ожидалось: ${testCase.expected}`);
      console.log(` Получено:  ${result}`);
      allPassed = false;
    }
  }

  return allPassed;
}

// Запуск тестов
console.log(`\n=== Тестирование mapWithIndex ===`);
const testsPassed = testMapWithIndex();

```



```
console.log(`Все тесты ${testsPassed ? 'пройдены' : 'провалены'}`);
```

8.2. Распространенные антипаттерны и как их избежать

javascript

// 1. Callback hell (ад обратных вызовов)

// ✗ Антипаттерн

```
function getUserData(userId, callback) {
  fetchUser(userId, function(user) {
    fetchPosts(user.id, function(posts) {
      fetchComments(posts[0].id, function(comments) {
        fetchLikes(comments[0].id, function(likes) {
          callback({
            user,
            posts,
            comments,
            likes
          });
        });
      });
    });
  });
}
```

// 🟢 Решение: промисы/async-await

```
async function getUserDataBetter(userId) {
  try {
    const user = await fetchUserAsync(userId);
    const posts = await fetchPostsAsync(user.id);
    const comments = await fetchCommentsAsync(posts[0].id);
    const likes = await fetchLikesAsync(comments[0].id);

    return { user, posts, comments, likes };
  } catch (error) {
    console.error('Ошибка получения данных:', error);
    throw error;
  }
}
```

// 2. Излишнее каррирование

// ✗ Антипаттерн (слишком много каррирования)

```
function overlyCurried(a) {
  return function(b) {
```

```

    return function(c) {
      return function(d) {
        return function(e) {
          return a + b + c + d + e;
        };
      };
    };
  };
}

```

// Использование становится неудобным

```
const result = overlyCurried(1)(2)(3)(4)(5);
```

// Решение: разумное каррирование или параметры по умолчанию

```

function reasonableSum(a, b, c = 0, d = 0, e = 0) {
  return a + b + c + d + e;
}

```

// Или каррирование с поддержкой вызова с несколькими аргументами

```

function smartCurry(fn) {
  const curried = (...args) => {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...moreArgs) => curried(...args, ...moreArgs);
  };
  return curried;
}

```

```

const smartSum = smartCurry(reasonableSum);
console.log(smartSum(1, 2, 3, 4, 5)); // 15
console.log(smartSum(1)(2)(3)(4)(5)); // 15
console.log(smartSum(1, 2)(3, 4, 5)); // 15

```

// 3. Чрезмерное использование замыканий для простых задач

// ✗ Антипаттерн

```

function createCounterComplex() {
  let count = 0;
  let history = [];

  return {
    increment: function() {
      count++;
      history.push({ type: 'increment', timestamp: new Date(), value: count });
    }
  };
}

```

```

        return this;
    },
    decrement: function() {
        count--;
        history.push({ type: 'decrement', timestamp: new Date(), value: count });
        return this;
    },
    getCount: function() {
        return count;
    },
    getHistory: function() {
        return [...history];
    },
    reset: function() {
        count = 0;
        history.push({ type: 'reset', timestamp: new Date(), value: 0 });
        return this;
    }
};
}

```

//  Решение: используйте классы для сложных объектов

```

class Counter {
    constructor() {
        this.count = 0;
        this.history = [];
    }

    increment() {
        this.count++;
        this.record('increment');
        return this;
    }

    decrement() {
        this.count--;
        this.record('decrement');
        return this;
    }

    getCount() {
        return this.count;
    }
}

```

```

    getHistory() {
        return [...this.history];
    }

    reset() {
        this.count = 0;
        this.record('reset');
        return this;
    }

    record(action) {
        this.history.push({
            type: action,
            timestamp: new Date(),
            value: this.count
        });
    }
}

// 4. Функции с слишком многими ответственностями
// ✕ Антипаттерн
function processUserDataAndSendEmailAndLog(user, emailService, logger) {
    // Валидация
    if (!user.email.includes('@')) {
        throw new Error('Некорректный email');
    }

    // Обработка данных
    user.processedAt = new Date();
    user.fullName = `${user.firstName} ${user.lastName}`;

    // Отправка email
    emailService.send({
        to: user.email,
        subject: 'Добро пожаловать',
        body: `Привет, ${user.fullName}!`
    });

    // Логирование
    logger.log(`Пользователь ${user.fullName} обработан`);

    return user;
}

```

// Решение: разделение ответственностей

```
function validateUser(user) {
  if (!user.email.includes('@')) {
    throw new Error('Некорректный email');
  }
  return user;
}

function processUserData(user) {
  return {
    ...user,
    processedAt: new Date(),
    fullName: `${user.firstName} ${user.lastName}`
  };
}
```

```
function createWelcomeEmail(user) {
  return {
    to: user.email,
    subject: 'Добро пожаловать',
    body: `Привет, ${user.fullName}!`
  };
}
```

// Композиция чистых функций

```
const processUserPipeline = pipe(
  validateUser,
  processUserData,
  tap(user => console.log(`Пользователь ${user.fullName} обработан`))
);
```

// 5. Игнорирование производительности в циклах

// ✗ Антипаттерн

```
function processLargeDataset(dataset) {
  return dataset
    .map(item => expensiveTransformation(item))
    .filter(item => complexPredicate(item))
    .map(item => anotherTransformation(item))
    .reduce((acc, item) => expensiveAggregation(acc, item), {});
}
```

// Решение: оптимизация циклов

```
function processLargeDatasetOptimized(dataset) {
  const result = {};
}
```

```

for (let i = 0; i < dataset.length; i++) {
  const item = dataset[i];

  // Ранний выход из цепочки преобразований
  const transformed1 = expensiveTransformation(item);
  if (!complexPredicate(transformed1)) continue;

  const transformed2 = anotherTransformation(transformed1);
  expensiveAggregation(result, transformed2);
}

return result;
}

// Или использование трансдукций для оптимизации
function transducerPipeline(...transducers) {
  return function(xf) {
    return transducers.reduceRight((acc, transducer) => transducer(acc), xf);
  };
}

const optimizedProcessing = transducerPipeline(
  map(expensiveTransformation),
  filter(complexPredicate),
  map(anotherTransformation)
);

// 6. Создание функций в циклах
// ✕ Антипаттерн
const handlers = [];
for (let i = 0; i < 10; i++) {
  handlers.push(function() {
    console.log(`Обработчик ${i} вызван`);
  });
}

// Все обработчики выведут "Обработчик 10 вызван"
handlers.forEach(handler => handler());

// 📌 Решение: создание замыканий с сохранением значения
const fixedHandlers = [];
for (let i = 0; i < 10; i++) {
  // Создаем новую область видимости для каждой итерации

```

```

(function(index) {
  fixedHandlers.push(function() {
    console.log(`Обработчик ${index} вызван`);
  });
})(i);
}

// Или с использованием let и блочной области видимости
const betterHandlers = [];
for (let i = 0; i < 10; i++) {
  betterHandlers.push(function() {
    console.log(`Обработчик ${i} вызван`);
  });
}

// Теперь работает правильно
betterHandlers.forEach((handler, index) => handler());

```

Заключение: ключевые принципы функций высшего порядка

8.3. Итоговые рекомендации

javascript

// 1. Принцип единственной ответственности
// Каждая функция должна делать одну вещь и делать ее хорошо

// ✗ Плохо

```

function processUserAndSendEmail(user) {
  validateUser(user);
  const processed = transformUser(user);
  sendEmail(processed.email, 'Welcome!');
  logAction('user_processed', user.id);
  return processed;
}

```

// ☑ Хорошо

```

const processUser = pipe(
  validateUser,
  transformUser,
  tap(user => sendEmail(user.email, 'Welcome!')),

```

```
    tap(user => logAction('user_processed', user.id))
  );
```

// 2. Чистые функции везде, где это возможно

// Чистые функции: одинаковые входы → одинаковые выходы, без побочных эффектов

// ✗ Нечистая функция

```
let counter = 0;
function increment() {
  counter++; // Побочный эффект
  return counter;
}
```

// ☑ Чистая функция

```
function pureIncrement(currentCount) {
  return currentCount + 1;
}
```

// 3. Композиция вместо наследования

// Функциональная композиция более гибка, чем классическое наследование

// ✗ Наследование

```
class Animal {
  speak() {
    return 'Some sound';
  }
}
```

```
class Dog extends Animal {
  speak() {
    return 'Woof!';
  }
}
```

// ☑ Композиция

```
const createAnimal = (sound) => ({
  speak: () => sound
});
```

```
const createDog = () => ({
  ...createAnimal('Woof!'),
  wagTail: () => 'Tail wagging'
});
```


// 4. Каррирование для переиспользуемости

```
const createRequest = (method) => (url) => (data) => {  
  console.log(`${method} ${url}`, data);  
  return { method, url, data };  
};
```

```
const post = createRequest('POST');  
const postToUsers = post('/api/users');  
const createUser = postToUsers({ name: '', email: '' });
```

// 5. Обработка ошибок в функциональном стиле

```
const safeTry = (fn) => {  
  try {  
    return { success: true, value: fn() };  
  } catch (error) {  
    return { success: false, error };  
  }  
};  
  
const safeJSONParse = (str) => safeTry(() => JSON.parse(str));
```

// 6. Ленивые вычисления и мемоизация

```
const memoize = (fn) => {  
  const cache = new Map();  
  return (...args) => {  
    const key = JSON.stringify(args);  
    if (cache.has(key)) return cache.get(key);  
    const result = fn(...args);  
    cache.set(key, result);  
    return result;  
  };  
};
```

// 7. Декларативный стиль вместо императивного

// ✕ Императивный

```
const imperativeResult = [];  
for (let i = 0; i < array.length; i++) {  
  if (array[i] > 10) {  
    imperativeResult.push(array[i] * 2);  
  }  
}
```

//  Декларативный

```
const declarativeResult = array
```

```
.filter(x => x > 10)
.map(x => x * 2);
```

// 8. Тестируемость

// Функции высшего порядка легко тестировать

```
const add = (a, b) => a + b;
```

```
const multiply = (a, b) => a * b;
```

```
const applyOperation = (a, b, operation) => operation(a, b);
```

// Тесты

```
console.assert(applyOperation(2, 3, add) === 5, 'Add should work');
```

```
console.assert(applyOperation(2, 3, multiply) === 6, 'Multiply should work');
```

// 9. Документирование типов (JSDoc или TypeScript)

*/***

** Применяет операцию к двум числам*

** @template T*

** @param {number} a - Первое число*

** @param {number} b - Второе число*

** @param {function(number, number): T} operation - Функция операции*

** @returns {T} Результат операции*

**/*

```
function applyOperationTyped(a, b, operation) {
  return operation(a, b);
}
```

// 10. Прагматичный подход

// Используйте функции высшего порядка там, где они делают код лучше,

// но не усложняйте простое

// Иногда простой цикл лучше

```
function findMax(arr) {
  let max = -Infinity;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] > max) max = arr[i];
  }
  return max;
}
```

// Иногда функция высшего порядка уместнее

```
const max = arr => arr.reduce((a, b) => Math.max(a, b), -Infinity);
```

8.4. Упражнения для самостоятельной практики

javascript

*// 1. Реализуйте функцию compose, которая принимает любое количество функций
// и возвращает их композицию*

```
function compose(...fns) {  
  // Ваша реализация  
}
```

// 2. Реализуйте функцию pipe, которая работает как compose, но слева направо

```
function pipe(...fns) {  
  // Ваша реализация  
}
```

// 3. Создайте каррированную версию функции, которая принимает 3 аргумента

```
function curry3(fn) {  
  // Ваша реализация  
}
```

// 4. Реализуйте мемоизацию с поддержкой асинхронных функций

```
function memoizeAsync(fn, ttl) {  
  // Ваша реализация  
}
```

// 5. Создайте функцию для ограничения вызовов (rate limiting)

```
function throttle(fn, limit, interval) {  
  // Ваша реализация  
}
```

// 6. Реализуйте функцию для обработки промисов с задержкой между вызовами

```
function promiseSeries(promises, delay) {  
  // Ваша реализация  
}
```

// 7. Создайте валидатор на основе функций высшего порядка

```
function createValidator(rules) {  
  // Ваша реализация  
}
```

// 8. Реализуйте простой роутер с middleware поддержкой

```
function createRouter() {  
  // Ваша реализация  
}
```

```
// 9. Создайте функцию для глубокого копирования объектов с поддержкой циклических ссылок
function deepClone(obj, visited = new Map()) {
  // Ваша реализация
}

// 10. Реализуйте отменяемые асинхронные операции
function createCancellablePromise(executor) {
  // Ваша реализация
}
```

Итог: сила функций высшего порядка в JavaScript

Функции высшего порядка — это мощный инструмент, который делает JavaScript выразительным и гибким языком. Они позволяют:

1. **Абстрагировать общие паттерны** — map, filter, reduce
2. **Создавать переиспользуемый код** — фабрики функций, каррирование
3. **Реализовывать сложное поведение** — middleware, валидаторы
4. **Управлять асинхронностью** — промисы, async/await обертки
5. **Оптимизировать производительность** — мемоизация, ленивые вычисления

Ключевые принципы успешного использования:

- Начинайте с простых функций и componуйте их в сложные
- Предпочитайте чистые функции везде, где это возможно
- Используйте каррирование для улучшения переиспользуемости
- Документируйте ваши функции высшего порядка
- Тестируйте каждую функцию независимо
- Измеряйте производительность и оптимизируйте при необходимости
- Следуйте принципам функционального программирования, но будьте прагматичны

Функции высшего порядка — это не просто технический прием, а философия программирования, которая делает код более декларативным, тестируемым и поддерживаемым. Освоив этот инструмент, вы сможете писать более качественный и выразительный JavaScript код.

Глава 8: Объекты и объектно-ориентированное программирование

8.1. Создание объектов: литералы, конструкторы, Object.create()

Введение: объекты в JavaScript

Объекты в JavaScript — это фундаментальная структура данных, представляющая собой коллекцию пар ключ-значение. Они являются основой для организации кода, хранения данных и реализации объектно-ориентированного программирования.

Основные способы создания объектов:

1. **Литералы объектов** — самый простой и распространенный способ
2. **Конструкторы** — для создания множества однотипных объектов
3. **Object.create()** — для создания объектов с указанным прототипом
4. **Классы (ES6)** — синтаксический сахар над конструкторами

Часть 1: Литералы объектов

1.1. Базовый синтаксис

```
javascript

// Простейший объект-литерал
const emptyObject = {}; // Пустой объект

// Объект с свойствами
const person = {
  // Строковые ключи (самый частый вариант)
  firstName: 'Иван',
  lastName: 'Петров',

  // Числовые ключи
```

```
age: 30,

// Логические значения
isActive: true,

// null и undefined значения
middleName: null,
nickname: undefined,

// Массивы как значения
hobbies: ['чтение', 'программирование', 'спорт'],

// Другие объекты как значения (вложенные объекты)
address: {
  city: 'Москва',
  street: 'Ленина',
  house: '10',
  apartment: '25'
},

// Функции как значения (методы)
getFullName: function() {
  return `${this.firstName} ${this.lastName}`;
},

// Сокращенная запись методов (ES6)
greet() {
  return `Привет, меня зовут ${this.firstName}!`;
},

// Геттеры и сеттеры
get birthYear() {
  return new Date().getFullYear() - this.age;
},

set birthYear(year) {
  this.age = new Date().getFullYear() - year;
}
};

// Использование объекта
console.log(person.firstName); // 'Иван'
console.log(person['lastName']); // 'Петров'
console.log(person.age); // 30
```

```
console.log(person.address.city); // 'Москва'
console.log(person.getFullName()); // 'Иван Петров'
console.log(person.greet()); // 'Привет, меня зовут Иван!'
console.log(person.birthYear); // Вычисляется автоматически
person.birthYear = 1990; // Устанавливается возраст
console.log(person.age); // Обновленный возраст
```

1.2. Динамические свойства и методы

javascript

```
// 1. Вычисляемые имена свойств (ES6)
const propertyName = 'dynamicProperty';
const methodName = 'dynamicMethod';

const obj = {
  // Статические свойства
  staticProperty: 'Статическое значение',

  // Динамические свойства через квадратные скобки
  [propertyName]: 'Динамическое значение',

  // Динамические имена методов
  [methodName]() {
    return 'Это динамический метод';
  },

  // Выражения в именах свойств
  ['prop_' + Math.random().toString(36).substr(2, 9)]: 'Случайное свойство',

  // Использование переменных как значений (сокращенная запись)
  propertyName, // Эквивалентно propertyName: propertyName

  // Геттер с вычисляемым именем
  get ['computed' + 'Getter']() {
    return 'Вычисленный геттер';
  }
};

console.log(obj.dynamicProperty); // 'Динамическое значение'
console.log(obj.dynamicMethod()); // 'Это динамический метод'
console.log(obj.propertyName); // 'dynamicProperty' (имя переменной)
console.log(obj.computedGetter); // 'Вычисленный геттер'
```

```
// 2. Создание объектов динамически
function createConfig(env) {
  // Базовые настройки
  const baseConfig = {
    appName: 'MyApp',
    version: '1.0.0',
    debug: false
  };

  // Настройки окружения
  const envConfigs = {
    development: {
      apiUrl: 'http://localhost:3000',
      debug: true,
      logLevel: 'verbose'
    },
    production: {
      apiUrl: 'https://api.example.com',
      debug: false,
      logLevel: 'error',
      enableAnalytics: true
    },
    testing: {
      apiUrl: 'http://test-api.example.com',
      debug: true,
      logLevel: 'debug',
      mockData: true
    }
  };

  // Объединение настроек с помощью spread оператора
  return {
    ...baseConfig,
    ...envConfigs[env],
    environment: env,
    timestamp: new Date().toISOString()
  };
}

// Использование
const devConfig = createConfig('development');
const prodConfig = createConfig('production');

console.log('Dev config:', devConfig);
```



```
console.log('Prod config:', prodConfig);

// 3. Объекты с методами, работающими с динамическими свойствами
const dynamicObject = {
  // Хранилище динамических свойств
  _data: {},

  // Метод для установки динамического свойства
  setProperty(key, value) {
    this._data[key] = value;

    // Создаем геттер и сеттер для нового свойства
    Object.defineProperty(this, key, {
      get() {
        return this._data[key];
      },
      set(newValue) {
        this._data[key] = newValue;
      },
      enumerable: true,
      configurable: true
    });

    return this; // Для цепочки вызовов
  },

  // Метод для удаления динамического свойства
  removeProperty(key) {
    delete this._data[key];
    delete this[key];
    return this;
  },

  // Метод для получения всех динамических свойств
  getProperties() {
    return { ...this._data };
  },

  // Проверка существования свойства
  hasProperty(key) {
    return key in this._data;
  }
};
```

```
// Использование динамического объекта
dynamicObject
  .setProperty('customName', 'John Doe')
  .setProperty('customAge', 30)
  .setProperty('customEmail', 'john@example.com');

console.log(dynamicObject.customName); // 'John Doe'
console.log(dynamicObject.customAge); // 30

dynamicObject.customAge = 31;
console.log(dynamicObject.customAge); // 31

console.log(dynamicObject.getProperties()); // Все свойства
console.log(dynamicObject.hasProperty('customName')); // true

dynamicObject.removeProperty('customEmail');
console.log(dynamicObject.hasProperty('customEmail')); // false
```

1.3. Расширенный синтаксис ES6+

```
javascript

// 1. Сокращенная запись свойств
const name = 'Анна';
const age = 25;
const city = 'Москва';

// Старый способ
const oldWay = {
  name: name,
  age: age,
  city: city
};

// Новый способ (ES6) - property shorthand
const newWay = {
  name, // Эквивалентно name: name
  age,
  city
};

console.log(newWay); // {name: 'Анна', age: 25, city: 'Москва'}

// 2. Spread оператор для объектов (ES2018)
```

```
const defaults = {
  theme: 'light',
  language: 'ru',
  notifications: true
};

const userSettings = {
  ...defaults, // Копируем все свойства из defaults
  theme: 'dark', // Переопределяем тему
  fontSize: 'medium' // Добавляем новое свойство
};

console.log(userSettings);
// {theme: 'dark', language: 'ru', notifications: true, fontSize: 'medium'}

// 3. Rest оператор для объектов (ES2018)
const original = {
  a: 1,
  b: 2,
  c: 3,
  d: 4,
  e: 5
};

const { a, b, ...rest } = original;
console.log(a); // 1
console.log(b); // 2
console.log(rest); // {c: 3, d: 4, e: 5}

// 4. Деструктуризация в параметрах функции
function processUser({
  name,
  age,
  email = 'default@example.com', // Значение по умолчанию
  ...otherInfo
}) {
  console.log(`Обработка пользователя: ${name}, ${age} лет`);
  console.log(`Email: ${email}`);
  console.log('Дополнительная информация:', otherInfo);
}

const user = {
  name: 'Мария',
  age: 28,
```

```
    city: 'Санкт-Петербург',  
    occupation: 'Разработчик'  
};
```

```
processUser(user);
```

```
// 5. Вложенная деструктуризация
```

```
const complexObject = {  
  id: 1,  
  personalInfo: {  
    name: 'Иван',  
    contacts: {  
      email: 'ivan@example.com',  
      phone: '+7 (999) 123-45-67'  
    }  
  },  
  settings: {  
    theme: 'dark',  
    notifications: true  
  }  
};
```

```
const {  
  id,  
  personalInfo: {  
    name,  
    contacts: { email, phone }  
  },  
  settings: { theme }  
} = complexObject;
```

```
console.log(id, name, email, phone, theme);
```

```
// 6. Динамические ключи с шаблонными строками
```

```
const createDynamicObject = (prefix, count) => {  
  const result = {};  
  
  for (let i = 1; i <= count; i++) {  
    // Динамическое имя свойства  
    const key = `${prefix}_${i}`;  
  
    // Динамическое значение  
    const value = `Значение ${i}`;
```

```

    result[key] = value;

    // Или с вычисляемым именем свойства
    // result[`${prefix}_${i}`] = `Значение ${i}`;
}

return result;
};

const dynamicProps = createDynamicObject('item', 5);
console.log(dynamicProps);
// {item_1: 'Значение 1', item_2: 'Значение 2', ...}

// 7. Объекты с символами в качестве ключей
const symbolKey1 = Symbol('description1');
const symbolKey2 = Symbol('description2');

const objWithSymbols = {
    regularKey: 'Обычное свойство',
    [symbolKey1]: 'Свойство с символьным ключом 1',
    [symbolKey2]: 'Свойство с символьным ключом 2'
};

console.log(objWithSymbols.regularKey); // Доступно
console.log(objWithSymbols[symbolKey1]); // Доступно

// Символьные ключи не видны при обычной итерации
console.log(Object.keys(objWithSymbols)); // ['regularKey']
console.log(Object.getOwnPropertySymbols(objWithSymbols)); // [Symbol(description1), Symbol(description2)]

```

1.4. Методы объекта и this

```

javascript

// Понимание контекста this в методах объекта
const user = {
    name: 'Алексей',
    age: 35,

    // Обычный метод
    introduce() {
        return `Меня зовут ${this.name}, мне ${this.age} лет`;
    },

```

```

// Метод со стрелочной функцией (особенности!)
introduceArrow: () => {
    // В стрелочных функциях this берется из внешнего контекста
    // В данном случае это глобальный объект или undefined (strict mode)
    return `Стрелочная функция: ${this.name}`; // Не сработает как ожидается!
},

// Вложенный метод с потерей контекста
nestedMethod() {
    function innerFunction() {
        // Здесь this потерян или указывает на глобальный объект
        console.log('Вложенная функция this:', this);
    }

    innerFunction();
},

// Вложенный метод с сохранением контекста
nestedMethodFixed() {
    // Сохраняем контекст в переменной
    const self = this;

    function innerFunction() {
        console.log('Fixed: Вложенная функция this:', self.name);
    }

    // Или используем стрелочную функцию
    const innerArrow = () => {
        console.log('Arrow: Вложенная функция this:', this.name);
    };

    innerFunction();
    innerArrow();
},

// Метод, возвращающий функцию
createGreeter() {
    // Возвращаемая функция запоминает контекст через замыкание
    return function(greeting) {
        return `${greeting}, ${this.name}!`; // Проблема: this потерян
    };
},

```

```

// Метод, возвращающий функцию с привязкой контекста
createGreeterFixed() {
  // Вариант 1: Использование стрелочной функции
  const arrowGreeter = (greeting) => {
    return `${greeting}, ${this.name}!`;
  };

  // Вариант 2: Привязка контекста с помощью bind
  const boundGreeter = function(greeting) {
    return `${greeting}, ${this.name}!`;
  }.bind(this);

  return arrowGreeter; // Или boundGreeter
}
};

console.log(user.introduce()); // Корректно работает
console.log(user.introduceArrow()); // Не работает как ожидается
user.nestedMethod(); // Показывает проблему с this
user.nestedMethodFixed(); // Решение проблемы

const greeter = user.createGreeter();
console.log(greeter('Привет')); // "Привет, undefined!"

const fixedGreeter = user.createGreeterFixed();
console.log(fixedGreeter('Привет')); // "Привет, Алексей!"

// Использование call, apply, bind
const anotherUser = { name: 'Мария', age: 28 };

console.log(user.introduce.call(anotherUser)); // "Меня зовут Мария..."
console.log(user.introduce.apply(anotherUser, [])); // То же самое
const boundIntroduce = user.introduce.bind(anotherUser);
console.log(boundIntroduce()); // "Меня зовут Мария..."

```

Часть 2: Конструкторы объектов

2.1. Функции-конструкторы

javascript

// 1. Базовый конструктор

```

function Person(firstName, lastName, age) {
  // Свойства экземпляра
  this.firstName = firstName;
  this.lastName = lastName;
  this.age = age;

  // Методы экземпляра (неэффективно - создается для каждого экземпляра)
  this.getFullName = function() {
    return `${this.firstName} ${this.lastName}`;
  };

  // this автоматически возвращается (если не возвращать явно другой объект)
}

// Создание экземпляров с помощью new
const person1 = new Person('Иван', 'Иванов', 30);
const person2 = new Person('Мария', 'Петрова', 25);

console.log(person1.getFullName()); // 'Иван Иванов'
console.log(person2.getFullName()); // 'Мария Петрова'

// Проверка типа
console.log(person1 instanceof Person); // true
console.log(person1.constructor === Person); // true

// 2. Что происходит при вызове new:
// 1. Создается новый пустой объект
// 2. Устанавливается прототип этого объекта (Person.prototype)
// 3. Вызывается конструктор с this, указывающим на новый объект
// 4. Если конструктор не возвращает объект, возвращается this

// Ручная имитация оператора new
function myNew(constructor, ...args) {
  // 1. Создаем новый объект
  const obj = {};

  // 2. Устанавливаем прототип
  Object.setPrototypeOf(obj, constructor.prototype);

  // 3. Вызываем конструктор с привязкой this
  const result = constructor.apply(obj, args);

  // 4. Возвращаем результат, если это объект, иначе obj
  return result instanceof Object ? result : obj;
}

```



```
}
```

```
const person3 = myNew(Person, 'Алексей', 'Сидоров', 35);  
console.log(person3.getFullName()); // 'Алексей Сидоров'
```

```
// 3. Оптимизация: вынос методов в прототип
```

```
function OptimizedPerson(firstName, lastName, age) {  
  // Только свойства в конструкторе  
  this.firstName = firstName;  
  this.lastName = lastName;  
  this.age = age;  
}
```

```
// Методы добавляем в прототип (создаются один раз)
```

```
OptimizedPerson.prototype.getFullName = function() {  
  return `${this.firstName} ${this.lastName}`;  
};
```

```
OptimizedPerson.prototype.introduce = function() {  
  return `Привет, я ${this.getFullName()}, мне ${this.age} лет`;  
};
```

```
OptimizedPerson.prototype.haveBirthday = function() {  
  this.age++;  
  return this.age;  
};
```

```
const optPerson1 = new OptimizedPerson('Иван', 'Иванов', 30);  
const optPerson2 = new OptimizedPerson('Мария', 'Петрова', 25);
```

```
console.log(optPerson1.getFullName()); // 'Иван Иванов'  
console.log(optPerson2.introduce()); // 'Привет, я Мария Петрова, мне 25 лет'  
console.log(optPerson1.haveBirthday()); // 31  
console.log(optPerson1.age); // 31
```

```
// 4. Статические свойства и методы
```

```
function User(username, email) {  
  // Свойства экземпляра  
  this.username = username;  
  this.email = email;  
  this.createdAt = new Date();  
  
  // Увеличиваем счетчик экземпляров  
  User.instanceCount++;
```

```

}

// Статические свойства (принадлежат конструктору, а не экземплярам)
User.instanceCount = 0;
User.maxInstances = 1000;

// Статические методы
User.getInfo = function() {
    return `Всего пользователей: ${this.instanceCount}/${this.maxInstances}`;
};

User.validateEmail = function(email) {
    return /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(email);
};

// Методы прототипа
User.prototype.getInfo = function() {
    return `${this.username} (${this.email}) создан ${this.createdAt.toLocaleDateString()}`;
};

// Создание экземпляров
console.log(User.getInfo()); // 'Всего пользователей: 0/1000'

const user1 = new User('john_doe', 'john@example.com');
const user2 = new User('jane_doe', 'jane@example.com');

console.log(User.getInfo()); // 'Всего пользователей: 2/1000'
console.log(User.validateEmail('invalid')); // false
console.log(User.validateEmail('test@example.com')); // true

console.log(user1.getInfo()); // 'john_doe (john@example.com) создан ...'
console.log(user2.getInfo()); // 'jane_doe (jane@example.com) создан ...'

// 5. Приватные свойства (замыкания)
function BankAccount(accountNumber, initialBalance = 0) {
    // Приватные свойства через замыкание
    let balance = initialBalance;
    let transactionHistory = [];

    // Публичные свойства
    this.accountNumber = accountNumber;
    this.owner = '';

```

// Приватные методы (доступны только внутри конструктора)

```
function recordTransaction(type, amount) {  
  transactionHistory.push({  
    type,  
    amount,  
    date: new Date(),  
    balance: balance  
  });  
}
```

// Публичные методы (привилегированные - имеют доступ к приватным данным)

```
this.deposit = function(amount) {  
  if (amount > 0) {  
    balance += amount;  
    recordTransaction('deposit', amount);  
    return true;  
  }  
  return false;  
};
```

```
this.withdraw = function(amount) {  
  if (amount > 0 && amount <= balance) {  
    balance -= amount;  
    recordTransaction('withdraw', amount);  
    return true;  
  }  
  return false;  
};
```

```
this.getBalance = function() {  
  return balance;  
};
```

```
this.getStatement = function() {  
  return [...transactionHistory]; // Возвращаем копию  
};
```

// Метод только для чтения

```
this.getAccountInfo = function() {  
  return {  
    accountNumber: this.accountNumber,  
    owner: this.owner,  
    balance: balance, // Можно получить, но не изменить напрямую  
    transactionsCount: transactionHistory.length  
  };  
};
```

```

    };
};
}

const account = new BankAccount('1234567890', 1000);

account.owner = 'Иван Иванов';
account.deposit(500);
account.withdraw(200);

console.log(account.getBalance()); // 1300
console.log(account.getAccountInfo());
console.log(account.getStatement());

// Попытка доступа к приватным свойствам
console.log(account.balance); // undefined
console.log(account.transactionHistory); // undefined

// 6. Наследование через конструкторы
function Animal(name, species) {
    this.name = name;
    this.species = species;
    this.isAlive = true;
}

Animal.prototype.eat = function() {
    return `${this.name} ест`;
};

Animal.prototype.sleep = function() {
    return `${this.name} спит`;
};

// Конструктор-потомок
function Dog(name, breed, age) {
    // Вызов родительского конструктора
    Animal.call(this, name, 'Собака');

    // Собственные свойства
    this.breed = breed;
    this.age = age;
}

// Настройка прототипного наследования

```

```

Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

// Собственные методы потомка
Dog.prototype.bark = function() {
    return `${this.name} лает: Гав-гав!`;
};

Dog.prototype.getInfo = function() {
    return `${this.name} - ${this.breed}, ${this.age} лет`;
};

// Переопределение родительского метода
Dog.prototype.eat = function() {
    return `${this.name} с удовольствием ест собачий корм`;
};

const dog = new Dog('Бобик', 'Овчарка', 3);

console.log(dog.eat()); // Переопределенный метод
console.log(dog.sleep()); // Унаследованный метод
console.log(dog.bark()); // Собственный метод
console.log(dog.getInfo()); // Собственный метод

console.log(dog instanceof Dog); // true
console.log(dog instanceof Animal); // true

```

2.2. Особенности и ограничения конструкторов

```

javascript

// 1. Забывание new - распространенная ошибка
function Car(model, year) {
    this.model = model;
    this.year = year;
}

// Правильно
const car1 = new Car('Toyota', 2020);
console.log(car1.model); // 'Toyota'

// Ошибка: забыли new
const car2 = Car('Honda', 2021); // this указывает на глобальный объект или undefined
console.log(car2); // undefined

```

```
console.log(global.model); // 'Honda' (в Node.js) или window.model (в браузере)
```

```
// 2. Защита от забывания new
```

```
function SafeCar(model, year) {  
  // Проверяем, вызван ли конструктор с new  
  if (!(this instanceof SafeCar)) {  
    // Если нет, вызываем себя заново с new  
    return new SafeCar(model, year);  
  }  
  
  this.model = model;  
  this.year = year;  
}
```

```
// Теперь оба варианта работают
```

```
const safeCar1 = new SafeCar('Toyota', 2020);  
const safeCar2 = SafeCar('Honda', 2021); // Автоматически добавляется new
```

```
console.log(safeCar1.model); // 'Toyota'
```

```
console.log(safeCar2.model); // 'Honda'
```

```
// 3. Или используем new.target (ES6)
```

```
function ModernCar(model, year) {  
  // new.target содержит конструктор, если вызвано с new  
  if (!new.target) {  
    throw new Error('Конструктор ModernCar должен вызываться с new');  
  }  
  
  this.model = model;  
  this.year = year;  
}  
  
try {  
  const modernCar1 = new ModernCar('Toyota', 2020); // OK  
  const modernCar2 = ModernCar('Honda', 2021); // Ошибка  
} catch (error) {  
  console.error(error.message); // 'Конструктор ModernCar должен вызываться с new'  
}
```

```
// 4. Конструкторы, возвращающие объекты
```

```
function Shape(type) {  
  this.type = type;  
  
  // Если конструктор возвращает объект, он используется вместо this
```

```

    return {
      customProperty: 'Кастомный объект',
      getType: () => this.type // Замыкание на this конструктора
    };
  }

const shape = new Shape('circle');
console.log(shape); // {customProperty: 'Кастомный объект', getType: f}
console.log(shape.type); // undefined (не свойства экземпляра)
console.log(shape.getType()); // 'circle'

// 5. Синглтон через конструктор
function DatabaseConnection() {
  // Если экземпляр уже существует, возвращаем его
  if (DatabaseConnection.instance) {
    return DatabaseConnection.instance;
  }

  // Инициализация соединения
  this.connectionId = Math.random().toString(36).substr(2, 9);
  this.connectedAt = new Date();
  this.isConnected = true;

  // Сохраняем экземпляр
  DatabaseConnection.instance = this;

  // Возвращаем this (важно!)
  return this;
}

// Методы прототипа
DatabaseConnection.prototype.connect = function() {
  console.log(`Уже подключено с ID: ${this.connectionId}`);
  return this;
};

DatabaseConnection.prototype.query = function(sql) {
  console.log(`Выполнение запроса: ${sql}`);
  return { result: 'данные' };
};

// Тестируем
const db1 = new DatabaseConnection();
const db2 = new DatabaseConnection();

```

```

const db3 = new DatabaseConnection();

console.log(db1 === db2); // true
console.log(db2 === db3); // true
console.log(db1.connectionId === db2.connectionId); // true

db1.connect();
db2.query('SELECT * FROM users');

// 6. Фабричный метод в конструкторе
function UserFactory() {
    // Не создаем экземпляры напрямую
    if (new.target) {
        throw new Error('UserFactory нельзя инстанцировать напрямую');
    }
}

// Статический фабричный метод
UserFactory.create = function(type, config) {
    switch (type) {
        case 'admin':
            return new AdminUser(config);
        case 'moderator':
            return new ModeratorUser(config);
        case 'regular':
            return new RegularUser(config);
        default:
            throw new Error(`Неизвестный тип пользователя: ${type}`);
    }
};

// Вспомогательные конструкторы
function AdminUser({ username, email }) {
    this.username = username;
    this.email = email;
    this.role = 'admin';
    this.permissions = ['read', 'write', 'delete', 'manage_users'];
}

function ModeratorUser({ username, email }) {
    this.username = username;
    this.email = email;
    this.role = 'moderator';
    this.permissions = ['read', 'write', 'moderate'];
}

```



```

}

function RegularUser({ username, email }) {
  this.username = username;
  this.email = email;
  this.role = 'user';
  this.permissions = ['read', 'write'];
}

// Использование фабрики
const admin = UserFactory.create('admin', {
  username: 'admin_user',
  email: 'admin@example.com'
});

const regular = UserFactory.create('regular', {
  username: 'regular_user',
  email: 'user@example.com'
});

console.log(admin.role, admin.permissions);
console.log(regular.role, regular.permissions);

```

Часть 3: Object.create() и чистый прототипный подход

3.1. Основы Object.create()

javascript

```

// 1. Базовое использование
// Object.create() создает новый объект с указанным прототипом

// Прототип (объект-родитель)
const animalPrototype = {
  // Методы прототипа
  eat() {
    return `${this.name} ест`;
  },

  sleep() {
    return `${this.name} спит`;
  },

```

```

// Свойства по умолчанию
isAlive: true,

// Инициализатор
init(name, species) {
    this.name = name;
    this.species = species;
    return this; // Для цепочки вызовов
}
};

// Создание объекта с указанным прототипом
const animal1 = Object.create(animalPrototype);
animal1.init('Рекс', 'Собака');

const animal2 = Object.create(animalPrototype);
animal2.init('Мурка', 'Кошка');

console.log(animal1.eat()); // 'Рекс ест'
console.log(animal2.sleep()); // 'Мурка спит'
console.log(animal1.isAlive); // true (унаследовано от прототипа)

// Проверка прототипа
console.log(Object.getPrototypeOf(animal1) === animalPrototype); // true
console.log(animalPrototype.isPrototypeOf(animal1)); // true

// 2. Создание объекта без прототипа (null)
const objWithoutProto = Object.create(null); // Без прототипа вообще

objWithoutProto.customProperty = 'Значение';
console.log(objWithoutProto.customProperty); // 'Значение'

// У такого объекта нет стандартных методов Object
console.log(objWithoutProto.toString); // undefined
console.log(objWithoutProto.hasOwnProperty); // undefined

// Можно добавить методы вручную
objWithoutProto.toString = function() {
    return '[Object without prototype]';
};

console.log(objWithoutProto.toString()); // '[Object without prototype]'

```

```

// 3. Object.create() с дескрипторами свойств
const configurableObject = Object.create(
  // Прототип
  {
    inheritedProp: 'Унаследованное свойство'
  },

  // Дескрипторы свойств
  {
    // Свойство только для чтения
    readOnlyProp: {
      value: 'Только для чтения',
      writable: false,
      enumerable: true,
      configurable: false
    },

    // Свойство с геттером/сеттером
    computedProp: {
      get() {
        return this._computedValue || 'значение по умолчанию';
      },
      set(value) {
        if (typeof value === 'string') {
          this._computedValue = value.toUpperCase();
        } else {
          this._computedValue = String(value);
        }
      },
      enumerable: true,
      configurable: true
    },

    // Скрытое свойство (не enumerable)
    hiddenProp: {
      value: 'Скрытое значение',
      writable: true,
      enumerable: false, // Не появится в for...in
      configurable: true
    }
  }
);

console.log(configurableObject.readOnlyProp); // 'Только для чтения'

```

```
configurableObject.readOnlyProp = 'Попытка изменить'; // Не изменится (strict mode выдаст ошибку)
console.log(configurableObject.readOnlyProp); // 'Только для чтения'

configurableObject.computedProp = 'новое значение';
console.log(configurableObject.computedProp); // 'НОВОЕ ЗНАЧЕНИЕ'

console.log(configurableObject.hiddenProp); // 'Скрытое значение' (доступно напрямую)

// Итерация по свойствам
console.log('Собственные свойства:');
for (let key in configurableObject) {
  console.log(`  ${key}: ${configurableObject[key]}`);
}
// Выведет только enumerable свойства

console.log('Все собственные свойства:');
console.log(Object.getOwnPropertyNames(configurableObject));
// Выведет все свойства, включая non-enumerable

// 4. Глубокое наследование через Object.create()
// Базовый прототип
const livingBeing = {
  isAlive: true,
  breathe() {
    return `${this.name} дышит`;
  }
};

// Прототип животного (наследует от livingBeing)
const animal = Object.create(livingBeing);
animal.init = function(name, species) {
  this.name = name;
  this.species = species;
  return this;
};
animal.eat = function() {
  return `${this.name} ест`;
};

// Прототип млекопитающего (наследует от animal)
const mammal = Object.create(animal);
mammal.hasFur = true;
mammal.feedMilk = function() {
```

```

    return `${this.name} кормит молоком`;
};

// Прототип собаки (наследует от mammal)
const dog = Object.create(mammal);
dog.bark = function() {
    return `${this.name} лает: Гав-гав!`;
};

// Создание экземпляра
const myDog = Object.create(dog);
myDog.init('Бобик', 'Собака');

console.log(myDog.bark()); // 'Бобик лает: Гав-гав!'
console.log(myDog.feedMilk()); // 'Бобик кормит молоком'
console.log(myDog.eat()); // 'Бобик ест'
console.log(myDog.breathe()); // 'Бобик дышит'
console.log(myDog.isAlive); // true
console.log(myDog.hasFur); // true

// Проверка цепочки прототипов
console.log(Object.getPrototypeOf(myDog) === dog); // true
console.log(Object.getPrototypeOf(dog) === mammal); // true
console.log(Object.getPrototypeOf(mammal) === animal); // true
console.log(Object.getPrototypeOf(animal) === livingBeing); // true
console.log(Object.getPrototypeOf(livingBeing) === Object.prototype); // true

// 5. Клонирование объектов с прототипами
function cloneWithPrototype(obj) {
    // Получаем прототип исходного объекта
    const proto = Object.getPrototypeOf(obj);

    // Создаем новый объект с тем же прототипом
    const clone = Object.create(proto);

    // Копируем все собственные свойства
    for (let key in obj) {
        if (obj.hasOwnProperty(key)) {
            clone[key] = obj[key];
        }
    }

    // Копируем символные свойства
    const symbols = Object.getOwnPropertySymbols(obj);

```

```

    for (let symbol of symbols) {
        clone[symbol] = obj[symbol];
    }

    return clone;
}

// Пример использования
const original = Object.create(
    { inherited: 'прототипное свойство' },
    {
        ownProp: { value: 'собственное свойство', enumerable: true }
    }
);

const cloned = cloneWithPrototype(original);
console.log(cloned.ownProp); // 'собственное свойство'
console.log(cloned.inherited); // 'прототипное свойство'
console.log(Object.getPrototypeOf(cloned) === Object.getPrototypeOf(original)); // true

```

3.2. Прототипное программирование на практике

```

javascript

// 1. Композиция вместо классического наследования
// Базовые миксины (примеси)
const canEat = {
    eat(food) {
        console.log(`${this.name} ест ${food}`);
        this.energy += 10;
        return this;
    }
};

const canSleep = {
    sleep(hours) {
        console.log(`${this.name} спит ${hours} часов`);
        this.energy += hours * 5;
        return this;
    }
};

const canWalk = {
    walk(distance) {

```

```

        console.log(`${this.name} проходит ${distance} км`);
        this.energy -= distance;
        return this;
    }
};

const canSwim = {
    swim(distance) {
        console.log(`${this.name} проплывает ${distance} км`);
        this.energy -= distance * 2;
        return this;
    }
};

const canFly = {
    fly(distance) {
        console.log(`${this.name} пролетает ${distance} км`);
        this.energy -= distance * 3;
        return this;
    }
};

// Функция для композиции миксинов
function compose(...mixins) {
    return function(baseClass) {
        return mixins.reduce((classBuilder, mixin) => {
            return mixin(classBuilder);
        }, baseClass);
    };
}

// Альтернативный подход: создание объекта из миксинов
function createCreature(name, ...abilities) {
    // Базовый объект
    const creature = Object.create({
        name,
        energy: 100,

        getStatus() {
            return `${this.name}: энергия ${this.energy}`;
        }
    });

    // Добавление способностей

```

```

abilities.forEach(ability => {
    Object.assign(creature, ability);
});

return creature;
}

// Создание существ с разными способностями
const dog = createCreature('Собака', canEat, canSleep, canWalk);
const fish = createCreature('Рыба', canEat, canSleep, canSwim);
const bird = createCreature('Птица', canEat, canSleep, canWalk, canFly);
const platypus = createCreature('Утконос', canEat, canSleep, canWalk, canSwim);

// Использование
dog.eat('мясо').walk(5).sleep(8);
console.log(dog.getStatus());

fish.eat('водоросли').swim(10);
console.log(fish.getStatus());

bird.eat('зерно').fly(20).sleep(6);
console.log(bird.getStatus());

platypus.eat('насекомых').walk(2).swim(3).sleep(10);
console.log(platypus.getStatus());

// 2. Делегирование поведения
// Прототип с общим поведением
const taskPrototype = {
    complete() {
        this.completed = true;
        this.completedAt = new Date();
        console.log(`Задача "${this.title}" выполнена`);
        return this;
    },

    delay(days) {
        this.dueDate.setDate(this.dueDate.getDate() + days);
        console.log(`Задача "${this.title}" отложена на ${days} дней`);
        return this;
    },

    getInfo() {
        return {

```



```

        title: this.title,
        description: this.description,
        dueDate: this.dueDate.toLocaleDateString(),
        completed: this.completed,
        priority: this.priority
    };
}
};

// Фабрика для создания задач
function createTask(title, description, dueDate, priority = 'medium') {
    const task = Object.create(taskPrototype);

    // Инициализация свойств
    task.title = title;
    task.description = description;
    task.dueDate = new Date(dueDate);
    task.priority = priority;
    task.completed = false;
    task.createdAt = new Date();

    return task;
}

// Специализированные задачи
function createUrgentTask(title, description) {
    const task = createTask(title, description, new Date(), 'high');

    // Дополнительное поведение для срочных задач
    task.notify = function() {
        console.log(`СРОЧНО: "${this.title}" - требует немедленного внимания!`);
        return this;
    };

    return task;
}

function createRecurringTask(title, description, intervalDays) {
    const task = createTask(title, description, new Date(), 'medium');

    // Дополнительное поведение для повторяющихся задач
    task.intervalDays = intervalDays;

    task.reschedule = function() {

```

```

        if (this.completed) {
            this.completed = false;
            this.dueDate.setDate(this.dueDate.getDate() + this.intervalDays);
            console.log(`Повторяющаяся задача "${this.title}" перепланирована`);
        }
        return this;
    };

    // Переопределение метода complete
    const originalComplete = task.complete;
    task.complete = function() {
        originalComplete.call(this);
        this.reschedule();
        return this;
    };

    return task;
}

// Использование
const simpleTask = createTask(
    'Купить продукты',
    'Молоко, хлеб, яйца',
    '2024-12-31'
);

const urgentTask = createUrgentTask(
    'Починить сервер',
    'Сервер упал, нужен срочный ремонт'
);

const recurringTask = createRecurringTask(
    'Еженедельный отчет',
    'Отчет по продажам за неделю',
    7
);

simpleTask.complete();
console.log(simpleTask.getInfo());

urgentTask.notify().complete();
console.log(urgentTask.getInfo());

recurringTask.complete(); // Автоматически перепланируется

```

```
console.log(recurringTask.getInfo());
```

```
// 3. Объекты с поведением на основе данных
```

```
function createReactiveObject(baseObject, onChange) {  
  const handler = {  
    get(target, property) {  
      // Обычный доступ к свойству  
      const value = target[property];  
  
      // Если свойство - объект, делаем и его реактивным  
      if (value && typeof value === 'object' && !Array.isArray(value)) {  
        return new Proxy(value, handler);  
      }  
  
      return value;  
    },  
  
    set(target, property, value) {  
      const oldValue = target[property];  
  
      // Устанавливаем значение  
      target[property] = value;  
  
      // Вызываем callback при изменении  
      if (onChange && oldValue !== value) {  
        onChange({  
          target,  
          property,  
          oldValue,  
          newValue: value,  
          type: 'set'  
        });  
      }  
  
      return true;  
    },  
  
    deleteProperty(target, property) {  
      const oldValue = target[property];  
      const deleted = delete target[property];  
  
      if (deleted && onChange) {  
        onChange({  
          target,
```

```

        property,
        oldValue,
        newValue: undefined,
        type: 'delete'
    });
}

    return deleted;
}
};

return new Proxy(baseObject, handler);
}

// Пример использования
const state = createReactiveObject(
{
    user: {
        name: 'Иван',
        age: 30,
        preferences: {
            theme: 'light',
            language: 'ru'
        }
    },
    cart: {
        items: [],
        total: 0
    }
},
(change) => {
    console.log('Изменение состояния:', change);
}
);

// Все изменения отслеживаются
state.user.name = 'Петр';
state.user.age = 31;
state.user.preferences.theme = 'dark';

state.cart.items.push({ id: 1, name: 'Товар', price: 100 });
state.cart.total = 100;

delete state.user.preferences.language;

```

```
console.log('Финальное состояние:', JSON.stringify(state, null, 2));
```

3.3. Сравнение подходов

javascript

// 1. Сравнение производительности

```
function benchmarkCreation(count) {  
  console.log(`\nБенчмарк создания ${count.toLocaleString()} объектов:`);
```

// Тест 1: Литералы

```
console.time('Литералы');  
const literals = [];  
for (let i = 0; i < count; i++) {  
  literals.push({  
    id: i,  
    name: `Object ${i}`,  
    value: Math.random(),  
    method: function() {  
      return this.value * 2;  
    }  
  });  
}  
console.timeEnd('Литералы');
```

// Тест 2: Конструкторы

```
function ConstructorObject(id, name, value) {  
  this.id = id;  
  this.name = name;  
  this.value = value;  
  this.method = function() {  
    return this.value * 2;  
  };  
}
```

```
console.time('Конструкторы (методы в экземплярах)');  
const constructed = [];  
for (let i = 0; i < count; i++) {  
  constructed.push(new ConstructorObject(i, `Object ${i}`, Math.random()));  
}  
console.timeEnd('Конструкторы (методы в экземплярах)');
```

// Тест 3: Конструкторы с методами в прототипе

```

function OptimizedConstructor(id, name, value) {
  this.id = id;
  this.name = name;
  this.value = value;
}

OptimizedConstructor.prototype.method = function() {
  return this.value * 2;
};

console.time('Конструкторы (методы в прототипе)');
const optimized = [];
for (let i = 0; i < count; i++) {
  optimized.push(new OptimizedConstructor(i, `Object ${i}`, Math.random()));
}
console.timeEnd('Конструкторы (методы в прототипе)');

// Тест 4: Object.create()
const prototypeObject = {
  method: function() {
    return this.value * 2;
  }
};

console.time('Object.create()');
const created = [];
for (let i = 0; i < count; i++) {
  const obj = Object.create(prototypeObject);
  obj.id = i;
  obj.name = `Object ${i}`;
  obj.value = Math.random();
  created.push(obj);
}
console.timeEnd('Object.create()');

// Тест 5: Классы (ES6)
class ClassObject {
  constructor(id, name, value) {
    this.id = id;
    this.name = name;
    this.value = value;
  }

  method() {

```

```

        return this.value * 2;
    }
}

console.time('Классы (ES6)');
const classed = [];
for (let i = 0; i < count; i++) {
    classed.push(new ClassObject(i, `Object ${i}`, Math.random()));
}
console.timeEnd('Классы (ES6)');

return { literals, constructed, optimized, created, classed };
}

// Запуск бенчмарка
const benchmarkResults = benchmarkCreation(10000);

// 2. Сравнение использования памяти
console.log('\n\nСравнение использования памяти:');

function getMemoryUsage() {
    if (global.gc) {
        global.gc();
    }
    const used = process.memoryUsage();
    return Math.round(used.heapUsed / 1024 / 1024);
}

function testMemoryUsage(createFn, count) {
    const startMemory = getMemoryUsage();
    const objects = [];

    for (let i = 0; i < count; i++) {
        objects.push(createFn(i));
    }

    const endMemory = getMemoryUsage();
    return {
        objects,
        memoryUsed: endMemory - startMemory
    };
}

const testCount = 100000;

```

```

// Литералы с методами в каждом объекте
const literalResult = testMemoryUsage((i) => ({
  id: i,
  value: Math.random(),
  method: function() { return this.value * 2; }
}), testCount);

// Конструкторы с методами в прототипе
function MemoryTestConstructor(id) {
  this.id = id;
  this.value = Math.random();
}
MemoryTestConstructor.prototype.method = function() {
  return this.value * 2;
};

const constructorResult = testMemoryUsage(
  (i) => new MemoryTestConstructor(i),
  testCount
);

// Object.create
const memoryPrototype = {
  method: function() {
    return this.value * 2;
  }
};

const createResult = testMemoryUsage((i) => {
  const obj = Object.create(memoryPrototype);
  obj.id = i;
  obj.value = Math.random();
  return obj;
}, testCount);

console.log('Использование памяти для 100,000 объектов:');
console.log(`Литералы: ${literalResult.memoryUsed} MB`);
console.log(`Конструкторы: ${constructorResult.memoryUsed} MB`);
console.log(`Object.create: ${createResult.memoryUsed} MB`);

// 3. Сравнение возможностей и синтаксиса
console.log(`\n\nСравнение синтаксиса и возможностей:`);

```



```
const comparisonTable = {
  'Литералы объектов': {
    синтаксис: 'Простой и понятный',
    производительность: 'Высокая при создании',
    использование_памяти: 'Высокое (методы в каждом объекте)',
    наследование: 'Нет встроенной поддержки',
    приватность: 'Нет (только соглашения)',
    лучший_случай: 'Единичные объекты, конфигурации',
    пример: `const obj = {
prop: 'value',
method() { ... }
}`
  },

  'Конструкторы': {
    синтаксис: 'Традиционный ООП',
    производительность: 'Высокая (с методами в прототипе)',
    использование_памяти: 'Низкое (методы в прототипе)',
    наследование: 'Есть (через prototype)',
    приватность: 'Через замыкания',
    лучший_случай: 'Множество однотипных объектов',
    пример: `function User(name) {
this.name = name;
}
User.prototype.method = function() { ... }`
  },

  'Object.create()': {
    синтаксис: 'Чисто прототипный',
    производительность: 'Средняя',
    использование_памяти: 'Низкое',
    наследование: 'Прямое делегирование',
    приватность: 'Через замыкания',
    лучший_случай: 'Динамическое наследование, миксины',
    пример: `const proto = { method() { ... } };
const obj = Object.create(proto);
obj.prop = 'value';`
  },

  'Классы (ES6)': {
    синтаксис: 'Современный, похож на другие языки',
    производительность: 'Высокая (оптимизировано движками)',
    использование_памяти: 'Низкое',
    наследование: 'Есть (extends)',
  }
}
```

```

        приватность: 'Через # поля (ES2022)',
        лучший_случай: 'Современные проекты, командная разработка',
        пример: `class User {
constructor(name) {
    this.name = name;
}

method() { ... }
}`
    };

console.table(comparisonTable);

// 4. Рекомендации по выбору подхода
console.log('\n\nРекомендации по выбору способа создания объектов:');

const recommendations = [
    {
        ситуация: 'Создание конфигурационного объекта',
        рекомендация: 'Используйте литералы объектов',
        причина: 'Простота, читаемость, один объект создается',
        пример: 'Настройки приложения, параметры функции'
    },
    {
        ситуация: 'Создание множества однотипных объектов',
        рекомендация: 'Используйте классы или конструкторы',
        причина: 'Эффективное использование памяти, четкая структура',
        пример: 'Пользователи, товары, заказы в системе'
    },
    {
        ситуация: 'Динамическое наследование или композиция',
        рекомендация: 'Используйте Object.create()',
        причина: 'Гибкость, чисто прототипный подход',
        пример: 'Системы плагинов, миксины, делегирование'
    },
    {
        ситуация: 'Современный проект с командой',
        рекомендация: 'Используйте классы (ES6+)',
        причина: 'Стандартизированный синтаксис, инструменты',
        пример: 'Корпоративные приложения, фреймворки'
    },
    {
        ситуация: 'Высокопроизводительный код',

```

```

    рекомендация: 'Оптимизированные конструкторы',
    причина: 'Максимальная производительность движков JS',
    пример: 'Игровые движки, обработка больших данных'
  },
  {
    ситуация: 'Прототип для наследования',
    рекомендация: 'Object.create() или литералы',
    причина: 'Простота определения прототипа',
    пример: 'Базовые классы, абстрактные типы'
  }
];

console.table(recommendations);

```

Часть 4: Продвинутые техники и паттерны

4.1. Иммутабельные объекты

```

javascript

// 1. Заморозка объектов
const mutableObject = {
  name: 'Изначальное значение',
  nested: {
    deepValue: 'Вложенное значение'
  }
};

// Object.freeze() - делает объект неизменяемым
const frozenObject = Object.freeze(mutableObject);

// Попытка изменить замороженный объект
frozenObject.name = 'Новое значение'; // В strict mode будет ошибка
console.log(frozenObject.name); // 'Изначальное значение' (не изменилось)

// Но вложенные объекты не замораживаются!
frozenObject.nested.deepValue = 'Изменено';
console.log(frozenObject.nested.deepValue); // 'Изменено' (изменилось!)

// Глубокая заморозка
function deepFreeze(obj) {
  // Получаем имена свойств

```

```

const propNames = Object.getOwnPropertyNames(obj);

// Замораживаем сам объект
Object.freeze(obj);

// Рекурсивно замораживаем все свойства
for (const name of propNames) {
  const value = obj[name];

  // Если свойство - объект и не null
  if (value && typeof value === 'object') {
    deepFreeze(value);
  }
}

return obj;
}

const deeplyFrozen = {
  name: 'Тестовый объект',
  nested: {
    deepValue: 'Вложенное',
    deeper: {
      deepest: 'Самое глубокое'
    }
  },
  array: [1, 2, 3]
};

deepFreeze(deeplyFrozen);

// Теперь ничего нельзя изменить
// deeplyFrozen.name = 'Новое'; // Ошибка в strict mode
// deeplyFrozen.nested.deepValue = 'Новое'; // Ошибка
// deeplyFrozen.array.push(4); // Ошибка

console.log('Объект полностью заморожен');

// 2. Иммутабельные обновления
function updateObjectImmutable(obj, updates) {
  // Создаем поверхностную копию
  const newObj = { ...obj };

  // Применяем обновления

```

```

    for (const [key, value] of Object.entries(updates)) {
        newObj[key] = value;
    }

    return newObj;
}

// Более продвинутая версия с глубоким обновлением
function deepUpdate(obj, path, value) {
    // Если путь пустой, возвращаем новое значение
    if (path.length === 0) {
        return value;
    }

    const [firstKey, ...restPath] = path;

    // Если это не объект, создаем новый
    if (!obj || typeof obj !== 'object') {
        obj = {};
    }

    // Рекурсивно обновляем вложенные свойства
    return {
        ...obj,
        [firstKey]: deepUpdate(obj[firstKey], restPath, value)
    };
}

const originalState = {
    user: {
        name: 'Иван',
        profile: {
            age: 30,
            preferences: {
                theme: 'light',
                notifications: true
            }
        }
    },
    cart: {
        items: ['товар1', 'товар2']
    }
};

```

// Иммуutableное обновление

```
const newState = deepUpdate(originalState, ['user', 'profile', 'age'], 31);

console.log('Original state:', originalState.user.profile.age); // 30
console.log('New state:', newState.user.profile.age); // 31
console.log('Objects are different:', originalState !== newState); // true
console.log('Nested objects are different:', originalState.user !== newState.user); //
true
```

// 3. Паттерн builder для иммуutableных объектов

```
function createImmutableBuilder(initial = {}) {
  let data = { ...initial };

  const builder = {
    set(key, value) {
      // Создаем новый builder с обновленными данными
      return createImmutableBuilder({
        ...data,
        [key]: value
      });
    },

    update(key, updater) {
      const current = data[key];
      const updated = updater(current);

      return createImmutableBuilder({
        ...data,
        [key]: updated
      });
    },

    delete(key) {
      const newData = { ...data };
      delete newData[key];
      return createImmutableBuilder(newData);
    },

    merge(other) {
      return createImmutableBuilder({
        ...data,
        ...other
      });
    },
  },
```

```

    build() {
        // Возвращаем замороженную версию
        return deepFreeze({ ...data });
    },

    // Для отладки
    toString() {
        return JSON.stringify(data, null, 2);
    }
};

return builder;
}

// Использование builder
const userBuilder = createImmutableBuilder()
    .set('name', 'Иван')
    .set('age', 30)
    .set('email', 'ivan@example.com')
    .update('age', age => age + 1) // Иммутабельное обновление
    .merge({
        address: {
            city: 'Москва',
            street: 'Ленина'
        }
    })
    .set('address.city', 'Санкт-Петербург') // Заменяет весь address!
    .delete('email');

const immutableUser = userBuilder.build();

console.log('Иммутабельный пользователь:');
console.log(immutableUser);

// Попытка изменить вызовет ошибку в strict mode
// immutableUser.name = 'Петр'; // TypeError
// immutableUser.address.city = 'Казань'; // TypeError

// 4. Структурное разделение (structural sharing)
class ImmutableMap {
    constructor(data = {}) {
        this._data = { ...data };
    }
}

```

```

set(key, value) {
  // Если значение не изменилось, возвращаем тот же объект
  if (this._data[key] === value) {
    return this;
  }

  // Создаем новый объект с обновленным свойством
  const newData = { ...this._data };
  newData[key] = value;

  return new ImmutableMap(newData);
}

get(key) {
  return this._data[key];
}

has(key) {
  return key in this._data;
}

delete(key) {
  if (!this.has(key)) {
    return this;
  }

  const newData = { ...this._data };
  delete newData[key];

  return new ImmutableMap(newData);
}

// Позволяет итерироваться
*[Symbol.iterator]() {
  for (const [key, value] of Object.entries(this._data)) {
    yield [key, value];
  }
}

// Для отладки
toJSON() {
  return { ...this._data };
}

```



```
}
```

```
// Тестирование структурного разделения
```

```
const map1 = new ImmutableMap({ a: 1, b: 2, c: 3 });
```

```
const map2 = map1.set('b', 2); // Значение не изменилось
```

```
const map3 = map1.set('b', 4); // Значение изменилось
```

```
const map4 = map3.delete('c');
```

```
console.log('map1 === map2:', map1 === map2); // true (один и тот же объект)
```

```
console.log('map1 === map3:', map1 === map3); // false (разные объекты)
```

```
console.log('map3 === map4:', map3 === map4); // false (разные объекты)
```

```
console.log('map1:', [...map1]);
```

```
console.log('map3:', [...map3]);
```

```
console.log('map4:', [...map4]);
```

4.2. Динамические объекты и метапрограммирование

```
javascript
```

```
// 1. Proxy для перехвата операций
```

```
function createValidatedObject(schema, initialData = {}) {
```

```
  // Валидатор на основе схемы
```

```
  function validate(key, value) {
```

```
    if (!(key in schema)) {
```

```
      throw new Error(`Свойство "${key}" не определено в схеме`);
```

```
    }
```

```
    const rule = schema[key];
```

```
    // Проверка типа
```

```
    if (rule.type && typeof value !== rule.type) {
```

```
      throw new Error(`Свойство "${key}" должно быть типа ${rule.type}, получено $
```

```
{typeof value}`);
```

```
    }
```

```
    // Проверка обязательности
```

```
    if (rule.required && (value === undefined || value === null)) {
```

```
      throw new Error(`Свойство "${key}" обязательно`);
```

```
    }
```

```
    // Валидация по регулярному выражению
```

```
    if (rule.pattern && !rule.pattern.test(value)) {
```

```
      throw new Error(`Свойство "${key}" не соответствует паттерну`);
```

```

}

// Минимальное/максимальное значение
if (rule.min !== undefined && value < rule.min) {
    throw new Error(`Свойство "${key}" должно быть не меньше ${rule.min}`);
}

if (rule.max !== undefined && value > rule.max) {
    throw new Error(`Свойство "${key}" должно быть не больше ${rule.max}`);
}

// Пользовательская валидация
if (rule.validate && !rule.validate(value)) {
    throw new Error(`Свойство "${key}" не прошло валидацию`);
}

return true;
}

const handler = {
    get(target, property) {
        // Перехват обращения к свойству
        console.log(`Чтение свойства: ${property}`);
        return target[property];
    },

    set(target, property, value) {
        console.log(`Установка свойства: ${property} = ${value}`);

        // Валидация
        validate(property, value);

        // Установка значения
        target[property] = value;
        return true;
    },

    deleteProperty(target, property) {
        console.log(`Удаление свойства: ${property}`);

        // Проверяем, можно ли удалять
        if (schema[property] && schema[property].required) {
            throw new Error(`Свойство "${property}" обязательно и не может быть удалено`);
        }
    }
}

```

```

        delete target[property];
        return true;
    },

    ownKeys(target) {
        console.log('Получение списка свойств');
        return Reflect.ownKeys(target);
    },

    has(target, property) {
        console.log(`Проверка наличия свойства: ${property}`);
        return property in target;
    }
};

// Создаем проксированный объект
const proxy = new Proxy(initialData, handler);

// Инициализируем обязательные свойства
for (const [key, rule] of Object.entries(schema)) {
    if (rule.required && !(key in initialData)) {
        proxy[key] = rule.default !== undefined ? rule.default : null;
    }
}

return proxy;
}

// Схема валидации
const userSchema = {
    name: {
        type: 'string',
        required: true,
        minLength: 2,
        maxLength: 50
    },

    age: {
        type: 'number',
        required: true,
        min: 0,
        max: 120,
        validate: value => Number.isInteger(value)
    }
};

```

```

    },

    email: {
      type: 'string',
      required: true,
      pattern: /^[^\s@]+@[^\s@]+\.[^\s@]+$/
    },

    isActive: {
      type: 'boolean',
      required: false,
      default: true
    }
  }
};

// Создание валидируемого объекта
const validatedUser = createValidatedObject(userSchema, {
  name: 'Иван',
  age: 30
});

console.log('Изначальный объект:', validatedUser);

// Попытка установить некорректные значения
try {
  validatedUser.age = -5; // Ошибка: возраст не может быть отрицательным
} catch (error) {
  console.error('Ошибка валидации:', error.message);
}

try {
  validatedUser.email = 'invalid-email'; // Ошибка: неверный email
} catch (error) {
  console.error('Ошибка валидации:', error.message);
}

try {
  delete validatedUser.name; // Ошибка: обязательное свойство
} catch (error) {
  console.error('Ошибка удаления:', error.message);
}

// Корректные операции
validatedUser.email = 'ivan@example.com';

```

```
validatedUser.isActive = false;

console.log('Финальный объект:', validatedUser);

// 2. Reflect для метапрограммирования
const metadata = new WeakMap();

function annotate(target, annotations) {
  metadata.set(target, annotations);
}

function getAnnotations(target) {
  return metadata.get(target) || {};
}

// Создание аннотированного объекта
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

// Добавляем метаданные
annotate(User.prototype, {
  name: {
    label: 'Имя пользователя',
    editable: true,
    validation: {
      required: true,
      minLength: 2
    }
  },
  age: {
    label: 'Возраст',
    editable: true,
    validation: {
      required: true,
      min: 0,
      max: 120
    }
  }
});
```

// Динамическое создание формы на основе метаданных

```
function createFormFromClass(instance) {
  const proto = Object.getPrototypeOf(instance);
  const annotations = getAnnotations(proto);
  const form = document.createElement('form');

  for (const [property, config] of Object.entries(annotations)) {
    const label = document.createElement('label');
    label.textContent = config.label;

    const input = document.createElement('input');
    input.name = property;
    input.value = instance[property];
    input.disabled = !config.editable;

    // Добавляем валидацию
    if (config.validation) {
      if (config.validation.required) {
        input.required = true;
      }

      if (config.validation.minLength) {
        input.minLength = config.validation.minLength;
      }
    }

    form.appendChild(label);
    form.appendChild(input);
    form.appendChild(document.createElement('br'));
  }

  return form;
}
```

// Использование

```
const userInstance = new User('Иван', 30);
```

// В браузере:

```
// document.body.appendChild(createFormFromClass(userInstance));
```

// 3. Динамическое создание объектов на основе JSON Schema

```
function createObjectFromSchema(schema) {
  // Базовый обработчик для создания объектов
  const handler = {
```

```

    construct(target, args) {
        const instance = Reflect.construct(target, args);

        // Применяем схему к экземпляру
        applySchema(instance, schema);

        return instance;
    }
};

// Класс-заглушка
class DynamicObject {
    constructor(data = {}) {
        Object.assign(this, data);
    }
}

// Обертка с прокси
return new Proxy(DynamicObject, handler);
}

function applySchema(obj, schema) {
    for (const [key, config] of Object.entries(schema.properties || {})) {
        // Определяем дескриптор свойства
        const descriptor = {
            enumerable: config.enumerable !== false,
            configurable: config.configurable !== false
        };

        // Геттер и сеттер
        if (config.get || config.set) {
            if (config.get) {
                descriptor.get = config.get.bind(obj);
            }
            if (config.set) {
                descriptor.set = config.set.bind(obj);
            }
        } else {
            // Обычное свойство
            descriptor.writable = config.writable !== false;
            descriptor.value = config.default;
        }
    }

    // Применяем дескриптор

```

```
    Object.defineProperty(obj, key, descriptor);
  }
}
```

// Пример схемы

```
const personSchema = {
  properties: {
    firstName: {
      type: 'string',
      default: '',
      enumerable: true,
      writable: true
    },
    lastName: {
      type: 'string',
      default: '',
      enumerable: true,
      writable: true
    },
    fullName: {
      enumerable: true,
      get() {
        return `${this.firstName} ${this.lastName}`.trim();
      },
      set(value) {
        const parts = value.split(' ');
        this.firstName = parts[0] || '';
        this.lastName = parts.slice(1).join(' ') || '';
      }
    },
    age: {
      type: 'number',
      default: 0,
      enumerable: true,
      writable: true,
      set(value) {
        if (value < 0) {
          throw new Error('Возраст не может быть отрицательным');
        }
        this._age = value;
      },
      get() {
        return this._age;
      }
    }
  }
}
```



```

    }
  }
};

// Создание конструктора на основе схемы
const Person = createObjectFromSchema(personSchema);

// Использование
const person = new Person({
  firstName: 'Иван',
  lastName: 'Иванов'
});

console.log(person.fullName); // 'Иван Иванов'
person.fullName = 'Петр Петров';
console.log(person.firstName, person.lastName); // 'Петр Петров'

person.age = 30;
console.log(person.age); // 30

try {
  person.age = -5; // Ошибка
} catch (error) {
  console.error(error.message);
}

```

Часть 5: Практические примеры и паттерны проектирования

5.1. Паттерн "Фабрика"

```

javascript

// 1. Простая фабрика
function createVehicle(type, config) {
  const vehicles = {
    car: () => new Car(config),
    truck: () => new Truck(config),
    motorcycle: () => new Motorcycle(config),
    bicycle: () => new Bicycle(config)
  };
}

```

```
const creator = vehicles[type];
if (!creator) {
  throw new Error(`Неизвестный тип транспортного средства: ${type}`);
}

return creator();
}
```

// Конструкторы для разных типов

```
function Car({ model, year, doors = 4 }) {
  this.type = 'car';
  this.model = model;
  this.year = year;
  this.doors = doors;
  this.wheels = 4;
}
```

```
Car.prototype.drive = function() {
  return `Автомобиль ${this.model} едет`;
};
```

```
function Truck({ model, year, capacity }) {
  this.type = 'truck';
  this.model = model;
  this.year = year;
  this.capacity = capacity;
  this.wheels = 6;
}
```

```
Truck.prototype.load = function(cargo) {
  return `Грузовик ${this.model} загружает ${cargo}`;
};
```

// Использование фабрики

```
const myCar = createVehicle('car', {
  model: 'Toyota Camry',
  year: 2020
});
```

```
const myTruck = createVehicle('truck', {
  model: 'Volvo FH',
  year: 2019,
  capacity: '20 тонн'
});
```

```
console.log(myCar.drive());
console.log(myTruck.load('песок'));
```

// 2. Абстрактная фабрика

```
function createUIFactory(theme) {
  const factories = {
    light: LightUIFactory,
    dark: DarkUIFactory,
    material: MaterialUIFactory
  };

  const FactoryClass = factories[theme];
  if (!FactoryClass) {
    throw new Error(`Неизвестная тема: ${theme}`);
  }

  return new FactoryClass();
}
```

// Абстрактная фабрика

```
class UIFactory {
  createButton() {
    throw new Error('Метод createButton должен быть реализован');
  }

  createInput() {
    throw new Error('Метод createInput должен быть реализован');
  }

  createCheckbox() {
    throw new Error('Метод createCheckbox должен быть реализован');
  }
}
```

// Конкретные фабрики

```
class LightUIFactory extends UIFactory {
  createButton(text) {
    return {
      type: 'button',
      theme: 'light',
      text,
      styles: {
        backgroundColor: '#ffffff',

```

```

        color: '#000000',
        border: '1px solid #cccccc'
    },
    render() {
        return `<button style="background: ${this.styles.backgroundColor}; color: $
${this.styles.color}">${this.text}</button>`;
    }
};
}

```

```

createInput(placeholder) {
    return {
        type: 'input',
        theme: 'light',
        placeholder,
        styles: {
            backgroundColor: '#ffffff',
            color: '#000000',
            border: '1px solid #cccccc'
        }
    };
}
}

```

```

class DarkUIFactory extends UIFactory {
    createButton(text) {
        return {
            type: 'button',
            theme: 'dark',
            text,
            styles: {
                backgroundColor: '#333333',
                color: '#ffffff',
                border: '1px solid #666666'
            }
        };
    }
}

```

```

createInput(placeholder) {
    return {
        type: 'input',
        theme: 'dark',
        placeholder,
        styles: {

```

```

        backgroundColor: '#333333',
        color: '#ffffff',
        border: '1px solid #666666'
    }
};
}
}

// Использование
const lightFactory = createUIFactory('light');
const darkFactory = createUIFactory('dark');

const lightButton = lightFactory.createButton('Светлая кнопка');
const darkButton = darkFactory.createButton('Темная кнопка');

console.log(lightButton);
console.log(darkButton);

// 3. Фабричный метод
class Document {
    constructor(title) {
        this.title = title;
        this.pages = [];
    }

    // Фабричный метод
    createPage(number) {
        throw new Error('Фабричный метод createPage должен быть реализован');
    }

    addPage(content) {
        const page = this.createPage(this.pages.length + 1);
        page.content = content;
        this.pages.push(page);
        return page;
    }

    print() {
        console.log(`Документ: ${this.title}`);
        this.pages.forEach(page => page.print());
    }
}

class ReportDocument extends Document {

```

```

    createPage(number) {
        return new ReportPage(number);
    }
}

class ManualDocument extends Document {
    createPage(number) {
        return new ManualPage(number);
    }
}

// Продукты
class ReportPage {
    constructor(number) {
        this.number = number;
        this.content = '';
        this.header = `Отчет - Страница ${number}`;
        this.footer = 'Конфиденциально';
    }

    print() {
        console.log(`${this.header}\n${this.content}\n${this.footer}\n`);
    }
}

class ManualPage {
    constructor(number) {
        this.number = number;
        this.content = '';
        this.header = `Руководство - Страница ${number}`;
        this.footer = '© Компания';
    }

    print() {
        console.log(`${this.header}\n${this.content}\n${this.footer}\n`);
    }
}

// Использование
const report = new ReportDocument('Годовой отчет');
report.addPage('Введение...');
report.addPage('Результаты...');
report.print();

```

```
const manual = new ManualDocument('Руководство пользователя');
manual.addPage('Установка...');
manual.addPage('Использование...');
manual.print();
```

5.2. Паттерн "Строитель" (Builder)

javascript

// 1. Классический строитель

```
class QueryBuilder {
  constructor() {
    this.reset();
  }

  reset() {
    this.query = {
      select: [],
      from: '',
      where: [],
      orderBy: [],
      limit: null,
      offset: null,
      joins: []
    };
  }

  select(...fields) {
    this.query.select.push(...fields);
    return this;
  }

  from(table) {
    this.query.from = table;
    return this;
  }

  where(condition) {
    this.query.where.push(condition);
    return this;
  }

  orderBy(field, direction = 'ASC') {
    this.query.orderBy.push({ field, direction });
  }
}
```

```

    return this;
}

limit(count) {
    this.query.limit = count;
    return this;
}

offset(count) {
    this.query.offset = count;
    return this;
}

join(table, condition, type = 'INNER') {
    this.query.joins.push({ table, condition, type });
    return this;
}

build() {
    // Сформируем SQL запрос
    let sql = 'SELECT ';

    if (this.query.select.length === 0) {
        sql += '*';
    } else {
        sql += this.query.select.join(', ');
    }

    sql += ` FROM ${this.query.from}`;

    if (this.query.joins.length > 0) {
        this.query.joins.forEach(join => {
            sql += ` ${join.type} JOIN ${join.table} ON ${join.condition}`;
        });
    }

    if (this.query.where.length > 0) {
        sql += ' WHERE ' + this.query.where.join(' AND ');
    }

    if (this.query.orderBy.length > 0) {
        const orderClauses = this.query.orderBy.map(
            ({ field, direction }) => `${field} ${direction}`
        );
    }

```



```

        sql += ' ORDER BY ' + orderClauses.join(', ');
    }

    if (this.query.limit !== null) {
        sql += ` LIMIT ${this.query.limit}`;
    }

    if (this.query.offset !== null) {
        sql += ` OFFSET ${this.query.offset}`;
    }

    return sql;
}

getQuery() {
    const result = this.build();
    this.reset();
    return result;
}
}

```

// Использование

```

const queryBuilder = new QueryBuilder();

const sql = queryBuilder
    .select('id', 'name', 'email')
    .from('users')
    .where('age > 18')
    .where('active = true')
    .orderBy('name', 'ASC')
    .limit(10)
    .offset(0)
    .getQuery();

console.log('SQL запрос:', sql);

```

// 2. Строитель с директором

```

class ComputerBuilder {
    constructor() {
        this.computer = {
            cpu: null,
            ram: null,
            storage: null,
            gpu: null,

```

```

        motherboard: null,
        powerSupply: null,
        cooling: null,
        case: null
    };
}

setCPU(model) {
    this.computer.cpu = model;
    return this;
}

setRAM(size, type = 'DDR4') {
    this.computer.ram = { size, type };
    return this;
}

setStorage(type, capacity) {
    this.computer.storage = { type, capacity };
    return this;
}

setGPU(model) {
    this.computer.gpu = model;
    return this;
}

setMotherboard(model) {
    this.computer.motherboard = model;
    return this;
}

setPowerSupply(wattage) {
    this.computer.powerSupply = { wattage };
    return this;
}

setCooling(type) {
    this.computer.cooling = type;
    return this;
}

setCase(model) {
    this.computer.case = model;

```

```

        return this;
    }

    build() {
        // Проверка конфигурации
        const errors = [];

        if (!this.computer.cpu) errors.push('Не указан процессор');
        if (!this.computer.ram) errors.push('Не указана оперативная память');
        if (!this.computer.storage) errors.push('Не указано хранилище');
        if (!this.computer.motherboard) errors.push('Не указана материнская плата');
        if (!this.computer.powerSupply) errors.push('Не указан блок питания');

        if (errors.length > 0) {
            throw new Error(`Ошибка конфигурации: ${errors.join(', ')} `);
        }

        return { ...this.computer };
    }
}

```

// Директор - знает оптимальные конфигурации

```

class ComputerDirector {
    static buildGamingPC(builder) {
        return builder
            .setCPU('Intel Core i9-13900K')
            .setRAM(32, 'DDR5')
            .setStorage('NVMe SSD', 2000)
            .setStorage('HDD', 4000)
            .setGPU('NVIDIA RTX 4090')
            .setMotherboard('ASUS ROG MAXIMUS Z790 HERO')
            .setPowerSupply(1000)
            .setCooling('Liquid AIO')
            .setCase('Lian Li O11 Dynamic')
            .build();
    }
}

```

```

    static buildOfficePC(builder) {
        return builder
            .setCPU('Intel Core i5-13400')
            .setRAM(16, 'DDR4')
            .setStorage('SSD', 1000)
            .setGPU('Integrated')
            .setMotherboard('ASUS PRIME B760-PLUS')
    }
}

```

```

        .setPowerSupply(500)
        .setCooling('Air')
        .setCase('Fractal Design Define 7')
        .build();
    }

    static buildWorkstation(builder) {
        return builder
            .setCPU('AMD Threadripper PRO 5995WX')
            .setRAM(128, 'DDR4 ECC')
            .setStorage('NVMe SSD', 4000)
            .setStorage('NVMe SSD', 4000)
            .setGPU('NVIDIA RTX A6000')
            .setGPU('NVIDIA RTX A6000')
            .setMotherboard('ASUS Pro WS WRX80E-SAGE SE WIFI')
            .setPowerSupply(1600)
            .setCooling('Custom Water Loop')
            .setCase('Corsair Obsidian 1000D')
            .build();
    }
}

// Использование
const builder = new ComputerBuilder();

try {
    const gamingPC = ComputerDirector.buildGamingPC(builder);
    console.log('Игровой ПК:', gamingPC);

    builder.reset();
    const officePC = ComputerDirector.buildOfficePC(builder);
    console.log('Офисный ПК:', officePC);

    builder.reset();
    const workstation = ComputerDirector.buildWorkstation(builder);
    console.log('Рабочая станция:', workstation);
} catch (error) {
    console.error('Ошибка сборки:', error.message);
}

// 3. Флюентный интерфейс для DOM
function createElement(tagName) {
    const element = document.createElement(tagName);

```

```

const builder = {
  id(value) {
    element.id = value;
    return this;
  },

  class(...classes) {
    element.classList.add(...classes);
    return this;
  },

  text(content) {
    element.textContent = content;
    return this;
  },

  html(content) {
    element.innerHTML = content;
    return this;
  },

  attr(name, value) {
    element.setAttribute(name, value);
    return this;
  },

  style(styles) {
    Object.assign(element.style, styles);
    return this;
  },

  on(event, handler) {
    element.addEventListener(event, handler);
    return this;
  },

  append(...children) {
    children.forEach(child => {
      if (typeof child === 'string') {
        element.appendChild(document.createTextNode(child));
      } else if (child instanceof Node) {
        element.appendChild(child);
      } else if (child && child.build) {
        element.appendChild(child.build());
      }
    });
  }
};

```

```

        }
    });
    return this;
},

    build() {
        return element;
    }
};

return builder;
}

// Использование (в браузере)
const container = createElement('div')
    .id('app')
    .class('container', 'mt-4')
    .style({
        maxWidth: '1200px',
        margin: '0 auto',
        padding: '20px'
    })
    .build();

const header = createElement('header')
    .class('header', 'mb-4')
    .append(
        createElement('h1')
            .class('title')
            .text('Мое приложение')
            .build(),
        createElement('p')
            .class('subtitle')
            .text('Добро пожаловать!')
            .build()
    )
    .build();

const button = createElement('button')
    .class('btn', 'btn-primary')
    .text('Нажми меня')
    .on('click', () => {
        alert('Кнопка нажата!');
    })

```

```
.build();
```

```
// document.body.appendChild(container);  
// container.appendChild(header);  
// container.appendChild(button);
```

Часть 6: Заключение и рекомендации

6.1. Сводная таблица методов создания объектов

javascript

```
const creationMethodsComparison = {  
  'Литералы объектов': {  
    синтаксис: 'Простой и лаконичный',  
    использование: 'const obj = { key: value };',  
    наследование: 'Нет встроенной поддержки',  
    приватность: 'Нет (только соглашения)',  
    производительность: 'Высокая',  
    память: 'Высокое использование (методы копируются)',  
    лучший_случай: 'Одиночные объекты, конфигурации',  
    пример: `const config = {  
      apiUrl: '...',  
      timeout: 5000  
    };`  
  },  
  
  'Конструкторы': {  
    синтаксис: 'Классический ООП',  
    использование: 'function User() {}; new User();',  
    наследование: 'Через prototype',  
    приватность: 'Через замыкания',  
    производительность: 'Высокая (с оптимизацией)',  
    память: 'Эффективно (методы в прототипе)',  
    лучший_случай: 'Множество однотипных объектов',  
    пример: `function Car(model) {  
      this.model = model;  
    }  
    Car.prototype.drive = function() { ... };`  
  },  
  
  'Object.create()': {
```

```
    синтаксис: 'Чисто прототипный',
    использование: 'Object.create(proto, descriptors)',
    наследование: 'Прямое делегирование',
    приватность: 'Через замыкания',
    производительность: 'Средняя',
    память: 'Эффективно',
    лучший_случай: 'Динамическое наследование, миксины',
    пример: `const animal = { eat() { ... } };
const dog = Object.create(animal);
dog.bark = function() { ... };`
  },

  'Классы ES6': {
    синтаксис: 'Современный, похож на другие языки',
    использование: 'class User { constructor() {} }',
    наследование: 'extends, super',
    приватность: '# поля (ES2022+)',
    производительность: 'Высокая (оптимизировано)',
    память: 'Эффективно',
    лучший_случай: 'Современные проекты, командная работа',
    пример: `class User {
#privateField;

constructor(name) {
  this.name = name;
}
}`
  },

  'Фабричные функции': {
    синтаксис: 'Функциональный подход',
    использование: 'function createUser() { return { ... }; }',
    наследование: 'Композиция/миксины',
    приватность: 'Через замыкания',
    производительность: 'Зависит от реализации',
    память: 'Зависит от реализации',
    лучший_случай: 'Сложное создание объектов, кэширование',
    пример: `function createUser(name) {
const privateData = { ... };

return {
  getName() { return name; },
  // ...
};
};`
```



```
}`  
}  
};
```

```
console.table(creationMethodsComparison);
```

6.2. Рекомендации по выбору метода

javascript

// 1. Когда использовать литералы объектов:

```
function useObjectLiterals() {  
  // ✅ Хорошо:  
  const config = {  
    apiUrl: 'https://api.example.com',  
    timeout: 5000,  
    retries: 3  
  };  
  
  const userPreferences = {  
    theme: 'dark',  
    language: 'ru',  
    notifications: true  
  };  
  
  // ❌ Плохо:  
  // Множество одинаковых объектов с методами  
  const users = [];  
  for (let i = 0; i < 1000; i++) {  
    users.push({  
      id: i,  
      name: `User ${i}`,  
      // Метод создается для каждого объекта!  
      greet() {  
        return `Hello, I'm ${this.name}`;  
      }  
    });  
  }  
}
```

// 2. Когда использовать конструкторы:

```
function useConstructors() {  
  // ✅ Хорошо:  
  function Product(name, price, category) {
```

```

    this.name = name;
    this.price = price;
    this.category = category;
    this.id = Math.random().toString(36).substr(2, 9);
}

Product.prototype.getFormattedPrice = function() {
    return `$$${this.price.toFixed(2)}`;
};

// Создание множества продуктов
const products = [];
for (let i = 0; i < 1000; i++) {
    products.push(new Product(`Product ${i}`, i * 10, 'electronics'));
}

// ✗ Плохо:
// Одиночный объект со сложной логикой
const singleConfig = new (function() {
    this.apiUrl = '...';
    this.timeout = 5000;
    // Избыточно для одного объекта
})();
}

// 3. Когда использовать Object.create():
function useObjectCreate() {
    // ✅ Хорошо:
    // Прототипное наследование с делегированием
    const vehicle = {
        move() {
            return `${this.name} движется`;
        }
    };

    const car = Object.create(vehicle, {
        name: { value: 'Автомобиль', writable: true },
        wheels: { value: 4, writable: false }
    });

    // Динамическое создание цепочек прототипов
    function createWithMixin(base, ...mixins) {
        return mixins.reduce((obj, mixin) => {
            return Object.create(obj, Object.getOwnPropertyDescriptors(mixin));
        }, base);
    }
}

```

```

    }, base);
}

// ✖ Плохо:
// Простые объекты без наследования
const simpleObj = Object.create(Object.prototype, {
  name: { value: 'Простой объект' }
});
// Избыточно, проще использовать литерал
}

// 4. Когда использовать классы:
function useClasses() {
  // ✅ Хорошо:
  class User {
    #password; // Приватное поле

    constructor(username, email) {
      this.username = username;
      this.email = email;
      this.#password = this.generatePassword();
      this.createdAt = new Date();
    }

    generatePassword() {
      return Math.random().toString(36).slice(2) +
        Math.random().toString(36).toUpperCase().slice(2);
    }

    validatePassword(input) {
      return input === this.#password;
    }

    // Статический метод
    static validateEmail(email) {
      return /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(email);
    }
  }

  class AdminUser extends User {
    constructor(username, email, permissions) {
      super(username, email);
      this.permissions = permissions;
    }
  }

```

```

    grantPermission(permission) {
        this.permissions.push(permission);
    }
}

// ✗ Плохо:
// Простые объекты данных
class SimpleConfig {
    constructor() {
        this.apiUrl = '...';
        this.timeout = 5000;
    }
}
// Избыточно, используйте литерал
}

// 5. Прагматичный подход:
function pragmaticApproach() {
    // Правило 1: Начинайте с самого простого решения
    // Литералы → Конструкторы → Классы → Специализированные подходы

    // Правило 2: Соответствуйте стилю кодовой базы
    // Если проект использует классы, используйте классы
    // Если проект функциональный, используйте фабричные функции

    // Правило 3: Учитывайте производительность
    // Для высоконагруженных частей кода используйте оптимизированные конструкторы

    // Правило 4: Думайте о поддерживаемости
    // Используйте подход, который будет понятен вашей команде через 6 месяцев

    // Правило 5: Тестируйте разные подходы
    // Профилируйте и выбирайте лучшее решение для вашего конкретного случая
}

// 6. Гибридный подход:
function hybridApproach() {
    // Часто лучшим решением является комбинация подходов

    // Пример: Кэширование с фабричными функциями и литералами
    function createCachedUser(id) {
        const cache = new Map();

```

```

return function getUser() {
  if (cache.has(id)) {
    return cache.get(id);
  }

  // Создаем объект с помощью литерала
  const user = {
    id,
    name: `User ${id}`,
    createdAt: new Date(),
    // Методы как свойства (одноразовое создание)
    getInfo() {
      return `${this.name} (ID: ${this.id})`;
    }
  };

  cache.set(id, user);
  return user;
};
}

// Пример: Использование классов для сложной логики и литералов для данных
class DataProcessor {
  constructor(config) {
    // Конфигурация как литерал объекта
    this.config = {
      batchSize: 100,
      retryAttempts: 3,
      timeout: 5000,
      ...config // Пользовательские настройки
    };

    this.stats = {
      processed: 0,
      failed: 0,
      retries: 0
    };
  }

  process(data) {
    // Сложная логика обработки
    const batches = this.splitIntoBatches(data);

    return batches.map(batch => ({

```

```

        batch,
        timestamp: new Date(),
        size: batch.length,
        // Результат как литерал объекта
        result: this.processBatch(batch)
    }));
}

splitIntoBatches(data) {
    // Реализация разбивки на батчи
    const batches = [];
    for (let i = 0; i < data.length; i += this.config.batchSize) {
        batches.push(data.slice(i, i + this.config.batchSize));
    }
    return batches;
}

processBatch(batch) {
    // Обработка батча
    this.stats.processed += batch.length;
    return batch.map(item => this.processItem(item));
}

processItem(item) {
    // Обработка отдельного элемента
    return {
        ...item,
        processed: true,
        processedAt: new Date()
    };
}
}
}

```

6.3. Заключительные рекомендации

javascript

// 1. Принципы SOLID для создания объектов:

// Single Responsibility Principle (Принцип единственной ответственности)

// Каждый конструктор/класс должен создавать объекты с одной ответственностью

// Open/Closed Principle (Принцип открытости/закрытости)

// Объекты должны быть открыты для расширения, но закрыты для изменения

// Liskov Substitution Principle (Принцип подстановки Барбары Лисков)

// Объекты-потомки должны быть взаимозаменяемы с объектами-родителями

// Interface Segregation Principle (Принцип разделения интерфейса)

// Создавайте специализированные объекты вместо универсальных

// Dependency Inversion Principle (Принцип инверсии зависимостей)

// Зависите от абстракций, а не от конкретных реализаций

// 2. Паттерны для различных ситуаций:

```
const patternsForSituations = [
  {
    ситуация: 'Создание сложных объектов с множеством параметров',
    паттерн: 'Строитель (Builder)',
    реализация: 'Цепочка методов для пошагового создания'
  },
  {
    ситуация: 'Создание семейств связанных объектов',
    паттерн: 'Абстрактная фабрика',
    реализация: 'Интерфейс для создания групп объектов'
  },
  {
    ситуация: 'Создание объектов без указания конкретного класса',
    паттерн: 'Фабричный метод',
    реализация: 'Подклассы решают, какой класс инстанцировать'
  },
  {
    ситуация: 'Один экземпляр объекта на всё приложение',
    паттерн: 'Одиночка (Singleton)',
    реализация: 'Конструктор, возвращающий существующий экземпляр'
  },
  {
    ситуация: 'Клонирование существующих объектов',
    паттерн: 'Прототип (Prototype)',
    реализация: 'Object.create() или клонирование'
  },
  {
    ситуация: 'Адаптация интерфейса объекта',
    паттерн: 'Адаптер (Adapter)',
    реализация: 'Обертка с другим интерфейсом'
  }
]
```

```
];
```

```
console.table(patternsForSituations);
```

```
// 3. Современные тенденции:
```

```
const modernTrends = {  
  'Функциональное программирование': {  
    описание: 'Иммутабельные объекты, чистые функции',  
    пример: 'createUser() вместо new User()',  
    преимущества: 'Предсказуемость, тестируемость'  
  },  
  'Композиция вместо наследования': {  
    описание: 'Объединение небольших объектов вместо глубоких иерархий',  
    пример: 'Object.assign() или миксины',  
    преимущества: 'Гибкость, избегание хрупких иерархий'  
  },  
  'Декораторы (Stage 3 proposal)': {  
    описание: 'Метапрограммирование для добавления поведения',  
    пример: '@decorator class MyClass {}',  
    преимущества: 'Чистый синтаксис для cross-cutting concerns'  
  },  
  'Private fields (ES2022)': {  
    описание: 'Нативная приватность в классах',  
    пример: 'class { #privateField; }',  
    преимущества: 'Нативная поддержка, производительность'  
  }  
};
```

```
console.table(modernTrends);
```

```
// 4. Ключевые выводы:
```

```
const keyTakeaways = `  
1. Литералы объектов - лучший выбор для простых объектов и конфигураций  
2. Конструкторы и классы - для создания множества однотипных объектов  
3. Object.create() - для продвинутого прототипного программирования  
4. Выбор метода зависит от контекста, требований и стиля проекта  
5. Сочетайте разные подходы для лучшего результата  
6. Всегда думайте о производительности и использовании памяти  
7. Следуйте принципам SOLID и используйте паттерны проектирования  
8. Оставайтесь в курсе современных возможностей JavaScript  
`;
```



```
console.log('Ключевые выводы:', keyTakeaways);
```

```
// 5. Дальнейшее изучение:
```

```
const furtherReading = [
  {
    тема: 'Глубокое понимание прототипов',
    ресурсы: [
      'MDN: Inheritance and the prototype chain',
      'Выступление Kyle Simpson: "this & Object Prototypes"',
      'Книга: "You Don\'t Know JS: this & Object Prototypes"'
    ]
  },
  {
    тема: 'Паттерны проектирования в JavaScript',
    ресурсы: [
      'Книга: "Learning JavaScript Design Patterns" от Addy Osmani',
      'Репозиторий: https://github.com/patterns-dev',
      'Статья: "JavaScript Design Patterns" на refactoring.guru'
    ]
  },
  {
    тема: 'Производительность создания объектов',
    ресурсы: [
      'Статья: "The cost of JavaScript in 2023"',
      'Инструмент: Chrome DevTools Memory Profiler',
      'Библиотека: https://github.com/microsoft/BenchmarkJS'
    ]
  },
  {
    тема: 'Функциональное программирование с объектами',
    ресурсы: [
      'Библиотека: Immutable.js',
      'Книга: "Functional-Light JavaScript" от Kyle Simpson',
      'Курс: "Professor Frisby\'s Mostly Adequate Guide to FP"'
    ]
  }
];

console.table(furtherReading);
```

Заключение

Создание объектов в JavaScript — это фундаментальный навык, который имеет множество подходов и нюансов. От правильного выбора метода зависит производительность, поддерживаемость и читаемость кода.

Основные рекомендации:

1. **Начинайте с простого** — используйте литералы объектов для конфигураций и простых структур данных
2. **Для множества однотипных объектов** используйте конструкторы или классы с методами в прототипе
3. **Для продвинутого наследования и композиции** рассматривайте `Object.create()`
4. **Следите за производительностью** — избегайте создания функций в каждом объекте
5. **Выбирайте подход в соответствии со стилем проекта** и предпочтениями команды
6. **Не бойтесь комбинировать подходы** для достижения лучших результатов

Помните, что нет "серебряной пули" — лучший метод всегда зависит от конкретной задачи, контекста и требований проекта. Экспериментируйте, измеряйте производительность и выбирайте то, что работает лучше всего в вашем случае.

Введение: объекты как основа JavaScript

Объекты в JavaScript — это составные структуры данных, которые хранят свойства (ключ-значение) и методы (функции). Понимание работы со свойствами и методами объектов является фундаментальным навыком для любого JavaScript-разработчика.

Часть 1: Свойства объектов

1.1. Типы свойств объектов

JavaScript поддерживает несколько типов свойств, каждый со своими особенностями:

1.1.1. Собственные свойства (Own Properties)

```
javascript

// Собственные свойства - определены непосредственно в объекте
const user = {
  name: 'Анна',           // Собственное свойство
  age: 25                 // Собственное свойство
};

// Проверка собственных свойств
console.log(user.hasOwnProperty('name')); // true
console.log(user.hasOwnProperty('toString')); // false (унаследовано)

// Получение всех собственных свойств
console.log(Object.getOwnPropertyNames(user)); // ['name', 'age']
console.log(Object.keys(user)); // ['name', 'age']
```

1.1.2. Унаследованные свойства (Inherited Properties)

```
javascript
```

```

// Унаследованные свойства - приходят из цепочки прототипов
const animal = {
  eats: true
};

const rabbit = Object.create(animal); // Создание с прототипом
rabbit.jumps = true;

console.log(rabbit.jumps); // true (собственное)
console.log(rabbit.eats); // true (унаследовано)

// Проверка всех свойств (включая унаследованные)
console.log('jumps' in rabbit); // true
console.log('eats' in rabbit); // true

```

1.1.3. Символьные свойства (Symbol Properties)

```

javascript

// Свойства с ключами-символами (ES6+)
const id = Symbol('id'); // Создание уникального символа
const user = {
  name: 'Иван',
  [id]: 12345 // Символьное свойство
};

// Доступ к символьным свойствам
console.log(user[id]); // 12345

// Символьные свойства не видны в обычных итерациях
console.log(Object.keys(user)); // ['name']
console.log(Object.getOwnPropertySymbols(user)); // [Symbol(id)]

```

1.2. Дескрипторы свойств

Каждое свойство объекта имеет связанный с ним дескриптор, который определяет его поведение:

```

javascript

const obj = {};

// Создание свойства с полным дескриптором
Object.defineProperty(obj, 'readOnlyProp', {

```

```

    value: 42,                // Значение свойства
    writable: false,         // Можно ли изменять значение
    enumerable: true,        // Будет ли перечисляться в for...in
    configurable: false      // Можно ли удалять или изменять дескриптор
  });

console.log(obj.readOnlyProp); // 42
obj.readOnlyProp = 100; // Не изменится (в строгом режиме - ошибка)
console.log(obj.readOnlyProp); // 42

// Получение дескриптора свойства
const descriptor = Object.getOwnPropertyDescriptor(obj, 'readOnlyProp');
console.log(descriptor);
// {
//   value: 42,
//   writable: false,
//   enumerable: true,
//   configurable: false
// }

```

1.2.1. Геттеры и сеттеры (Accessor Properties)

```

javascript

const user = {
  firstName: 'Анна',
  lastName: 'Иванова',

  // Геттер для полного имени
  get fullName() {
    return `${this.firstName} ${this.lastName}`;
  },

  // Сеттер для полного имени
  set fullName(value) {
    const parts = value.split(' ');
    this.firstName = parts[0];
    this.lastName = parts[1] || '';
  }
};

console.log(user.fullName); // 'Анна Иванова' (вызывается геттер)

user.fullName = 'Мария Петрова'; // Вызывается сеттер
console.log(user.firstName); // 'Мария'

```

```

console.log(user.lastName); // 'Петрова'

// Определение геттеров/сеттеров через defineProperty
const temperature = {};
Object.defineProperty(temperature, 'celsius', {
  get() {
    return this._celsius;
  },
  set(value) {
    if (value < -273.15) {
      throw new Error('Температура ниже абсолютного нуля');
    }
    this._celsius = value;
  },
  enumerable: true
});

temperature.celsius = 25;
console.log(temperature.celsius); // 25

```

1.3. Флаги свойств

Каждое свойство имеет три ключевых флага, которые управляют его поведением:

1.3.1. Writable (записываемость)

```

javascript

const obj = {};

// Создание незаписываемого свойства
Object.defineProperty(obj, 'constant', {
  value: 'неизменяемое значение',
  writable: false,
  enumerable: true,
  configurable: false
});

console.log(obj.constant); // 'неизменяемое значение'
obj.constant = 'новое значение'; // Тихая ошибка (в нестрогом режиме)
console.log(obj.constant); // 'неизменяемое значение'

```

```
// В строгом режиме будет ошибка
'use strict';
obj.constant = 'новое значение'; // TypeError: Cannot assign to read only property
```

1.3.2. Enumerable (перечисляемость)

```
javascript

const config = {
  apiKey: 'secret-key-123',
  endpoint: 'https://api.example.com',
  _internalFlag: true
};

// Делаем свойство неперечисляемым
Object.defineProperty(config, 'apiKey', {
  enumerable: false
});

// Теперь apiKey не появится в for...in и Object.keys()
for (let key in config) {
  console.log(key); // 'endpoint', '_internalFlag' (нет 'apiKey')
}

console.log(Object.keys(config)); // ['endpoint', '_internalFlag']

// Но доступ по ключу все еще работает
console.log(config.apiKey); // 'secret-key-123'
```

1.3.3. Configurable (конфигурируемость)

```
javascript

const obj = { prop: 'value' };

// Свойство по умолчанию configurable: true
Object.defineProperty(obj, 'prop', {
  configurable: false // Теперь нельзя удалить или изменить флаги
});

// Эти операции вызовут ошибки (в строгом режиме):
// delete obj.prop; // Нельзя удалить
// Object.defineProperty(obj, 'prop', { writable: false }); // Нельзя изменить флаги

// Но можно изменить значение, если writable: true
obj.prop = 'новое значение'; // Работает
```

1.4. Операции со свойствами

1.4.1. Проверка существования свойств

```
javascript

const car = {
  brand: 'Toyota',
  model: 'Camry',
  year: 2020
};

// 1. Оператор in (проверяет цепочку прототипов)
console.log('brand' in car); // true
console.log('toString' in car); // true (из прототипа Object)

// 2. Метод hasOwnProperty (только собственные свойства)
console.log(car.hasOwnProperty('brand')); // true
console.log(car.hasOwnProperty('toString')); // false

// 3. Сравнение с undefined (не рекомендуется)
console.log(car.brand !== undefined); // true
console.log(car.color !== undefined); // false

// 4. Сравнение с in для сложных случаев
car.speed = undefined;
console.log(car.speed !== undefined); // false (но свойство существует!)
console.log('speed' in car); // true (корректная проверка)
```

1.4.2. Перечисление свойств

```
javascript

const person = {
  name: 'Алексей',
  age: 30,
  job: 'разработчик',
  [Symbol('id')]: 123
};

// 1. Object.keys() - только перечисляемые строковые свойства
console.log(Object.keys(person)); // ['name', 'age', 'job']

// 2. Object.values() - значения перечисляемых свойств
console.log(Object.values(person)); // ['Алексей', 30, 'разработчик']
```



```
// 3. Object.entries() - пары ключ-значение
console.log(Object.entries(person));
// [['name', 'Алексей'], ['age', 30], ['job', 'разработчик']]

// 4. Object.getOwnPropertyNames() - все строковые свойства
console.log(Object.getOwnPropertyNames(person)); // ['name', 'age', 'job']

// 5. Object.getOwnPropertySymbols() - символные свойства
console.log(Object.getOwnPropertySymbols(person)); // [Symbol(id)]

// 6. for...in цикл (включая унаследованные)
for (let key in person) {
  console.log(key, person[key]);
}
// name Алексей
// age 30
// job разработчик
```

Часть 2: Методы объектов

2.1. Встроенные методы объектов

JavaScript предоставляет множество полезных методов для работы с объектами:

2.1.1. Методы для работы с прототипами

```
javascript

// Создание объектов с прототипами
const animal = {
  eats: true,
  walk() {
    console.log('Животное идет');
  }
};

const rabbit = {
  jumps: true,
  __proto__: animal // Наследование (ES6+)
```

```

};

// 1. Object.getPrototypeOf()
console.log(Object.getPrototypeOf(rabbit) === animal); // true

// 2. Object.setPrototypeOf()
const newProto = { runs: true };
Object.setPrototypeOf(rabbit, newProto);
console.log(rabbit.runs); // true

// 3. Object.create() - создание с указанным прототипом
const deer = Object.create(animal, {
  antlers: {
    value: true,
    writable: true,
    enumerable: true
  }
});

console.log(deer.eats); // true (унаследовано)
console.log(deer.antlers); // true (собственное)

```

2.1.2. Методы для блокировки объектов

```

javascript

const config = {
  apiUrl: 'https://api.example.com',
  timeout: 5000,
  retries: 3
};

// 1. Object.preventExtensions() - нельзя добавлять новые свойства
Object.preventExtensions(config);
config.newProp = 'value'; // Не добавится (в строгом режиме - ошибка)

// 2. Object.seal() - нельзя добавлять/удалять свойства
Object.seal(config);
delete config.timeout; // Не удалится
config.timeout = 10000; // Но можно изменять

// 3. Object.freeze() - полная блокировка
Object.freeze(config);
config.timeout = 20000; // Не изменится
config.newProp = 'test'; // Не добавится

```

```
delete config.retries; // Не удалится

// Проверка состояния объекта
console.log(Object.isExtensible(config)); // false
console.log(Object.isSealed(config)); // true
console.log(Object.isFrozen(config)); // true
```

2.2. Пользовательские методы объектов

2.2.1. Определение методов

```
javascript

// 1. Литеральная нотация
const calculator = {
  // Метод сложения
  add(a, b) {
    return a + b;
  },

  // Метод вычитания
  subtract: function(a, b) {
    return a - b;
  },

  // Геттер для истории операций
  history: [],

  // Метод с побочным эффектом
  logOperation(operation, result) {
    this.history.push({ operation, result, timestamp: new Date() });
  },

  // Асинхронный метод
  async fetchData(url) {
    const response = await fetch(url);
    return response.json();
  }
};

// Использование методов
console.log(calculator.add(5, 3)); // 8
calculator.logOperation('add', 8);
```

```
console.log(calculator.history);  
// [{ operation: 'add', result: 8, timestamp: ... }]
```

2.2.2. Методы с использованием `this`

```
javascript  
  
const user = {  
  name: 'Мария',  
  balance: 1000,  
  
  // Метод, использующий this  
  deposit(amount) {  
    if (amount > 0) {  
      this.balance += amount;  
      console.log(`Пополнено: ${amount}. Баланс: ${this.balance}`);  
    }  
    return this.balance;  
  },  
  
  withdraw(amount) {  
    if (amount > 0 && amount <= this.balance) {  
      this.balance -= amount;  
      console.log(`Снято: ${amount}. Баланс: ${this.balance}`);  
    } else {  
      console.log('Недостаточно средств');  
    }  
    return this.balance;  
  },  
  
  // Метод, возвращающий this для чейнинга  
  showBalance() {  
    console.log(`Текущий баланс: ${this.balance}`);  
    return this; // Возвращаем объект для чейнинга  
  }  
};  
  
// Чейнинг методов  
user.deposit(500).withdraw(200).showBalance();  
// Пополнено: 500. Баланс: 1500  
// Снято: 200. Баланс: 1300  
// Текущий баланс: 1300
```

2.3. Статические методы

Статические методы принадлежат конструктору или классу, а не экземплярам:

```
javascript

class MathHelper {
  // Статический метод
  static sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
  }

  static average(...numbers) {
    if (numbers.length === 0) return 0;
    return this.sum(...numbers) / numbers.length;
  }

  // Статическое свойство
  static PI = 3.14159;
}

// Использование статических методов
console.log(MathHelper.sum(1, 2, 3, 4, 5)); // 15
console.log(MathHelper.average(10, 20, 30)); // 20
console.log(MathHelper.PI); // 3.14159

// Статические методы не доступны у экземпляров
const helper = new MathHelper();
// helper.sum(1, 2); // Ошибка: helper.sum is not a function
```

2.4. Приватные методы (ES2022+)

```
javascript

class BankAccount {
  // Приватное поле
  #balance = 0;

  // Приватный метод
  #validateAmount(amount) {
    if (typeof amount !== 'number' || amount <= 0) {
      throw new Error('Некорректная сумма');
    }
    return true;
  }
}
```

```

    }

    // Публичный метод, использующий приватный
    deposit(amount) {
        this.#validateAmount(amount);
        this.#balance += amount;
        return this.#balance;
    }

    withdraw(amount) {
        this.#validateAmount(amount);
        if (amount > this.#balance) {
            throw new Error('Недостаточно средств');
        }
        this.#balance -= amount;
        return this.#balance;
    }

    // Публичный геттер
    get balance() {
        return this.#balance;
    }
}

const account = new BankAccount();
account.deposit(1000);
console.log(account.balance); // 1000

// Приватные методы недоступны извне
// account.#validateAmount(100); // SyntaxError
// account.#balance = 10000; // SyntaxError

```

Часть 3: Расширенные возможности

3.1. Смешивание объектов

```

javascript

// 1. Object.assign() - поверхностное копирование
const defaults = {
    theme: 'light',
    language: 'ru',

```

```

    notifications: true
};

const userSettings = {
    theme: 'dark',
    fontSize: 14
};

const merged = Object.assign({}, defaults, userSettings);
console.log(merged);
// { theme: 'dark', language: 'ru', notifications: true, fontSize: 14 }

// 2. Spread оператор (ES6+)
const settings = {
    ...defaults,
    ...userSettings,
    customProp: 'value'
};

// 3. Глубокое смешивание (рекурсивное)
function deepMerge(target, ...sources) {
    if (!sources.length) return target;
    const source = sources.shift();

    if (isObject(target) && isObject(source)) {
        for (const key in source) {
            if (isObject(source[key])) {
                if (!target[key]) Object.assign(target, { [key]: {} });
                deepMerge(target[key], source[key]);
            } else {
                Object.assign(target, { [key]: source[key] });
            }
        }
    }
    return deepMerge(target, ...sources);
}

function isObject(item) {
    return item && typeof item === 'object' && !Array.isArray(item);
}

const obj1 = { a: 1, b: { c: 2 } };
const obj2 = { b: { d: 3 }, e: 4 };
const result = deepMerge({}, obj1, obj2);

```

```
console.log(result); // { a: 1, b: { c: 2, d: 3 }, e: 4 }
```

3.2. Прототипные методы

```
javascript

// Добавление методов в прототип
function Person(name, age) {
  this.name = name;
  this.age = age;
}

// Методы в прототипе (общие для всех экземпляров)
Person.prototype.greet = function() {
  return `Привет, меня зовут ${this.name}`;
};

Person.prototype.getBirthYear = function(currentYear = new Date().getFullYear()) {
  return currentYear - this.age;
};

// Статический метод
Person.compareAges = function(person1, person2) {
  return person1.age - person2.age;
};

// Использование
const alice = new Person('Алиса', 25);
const bob = new Person('Боб', 30);

console.log(alice.greet()); // 'Привет, меня зовут Алиса'
console.log(bob.getBirthYear()); // 1994 (если текущий год 2024)
console.log(Person.compareAges(alice, bob)); // -5
```

3.3. Методы с контекстом и привязкой

```
javascript

const user = {
  name: 'Виктор',
  projects: ['Проект А', 'Проект Б', 'Проект В'],

  // Метод с проблемой контекста
  showProjectsBad() {
```



```
    this.projects.forEach(function(project) {
        console.log(`${this.name}: ${project}`); // Ошибка! this не определен
    });
},
```

// Решение 1: сохранение this

```
showProjectsGood1() {
    const self = this;
    this.projects.forEach(function(project) {
        console.log(`${self.name}: ${project}`);
    });
},
```

// Решение 2: использование стрелочной функции

```
showProjectsGood2() {
    this.projects.forEach(project => {
        console.log(`${this.name}: ${project}`);
    });
},
```

// Решение 3: bind

```
showProjectsGood3() {
    this.projects.forEach(function(project) {
        console.log(`${this.name}: ${project}`);
    }.bind(this));
},
```

// Решение 4: второй параметр forEach

```
showProjectsGood4() {
    this.projects.forEach(function(project) {
        console.log(`${this.name}: ${project}`);
    }, this); // Передаем контекст вторым параметром
}
};
```

```
user.showProjectsGood2();
```

// Виктор: Проект А

// Виктор: Проект Б

// Виктор: Проект В

Часть 4: Практические примеры и паттерны

4.1. Паттерн "Модуль"

javascript

```
const UserModule = (function() {  
  // Приватные переменные  
  let users = [];  
  let nextId = 1;  
  
  // Приватный метод  
  function validateUser(user) {  
    return user &&  
      typeof user.name === 'string' &&  
      user.name.trim().length > 0;  
  }  
  
  // Публичный интерфейс  
  return {  
    // Добавление пользователя  
    addUser(name, email) {  
      const user = { id: nextId++, name, email, createdAt: new Date() };  
  
      if (!validateUser(user)) {  
        throw new Error('Некорректные данные пользователя');  
      }  
  
      users.push(user);  
      return user.id;  
    },  
  
    // Получение пользователя по ID  
    getUser(id) {  
      return users.find(user => user.id === id) || null;  
    },  
  
    // Получение всех пользователей  
    getAllUsers() {  
      return [...users]; // Возвращаем копию  
    },  
  
    // Поиск пользователей по имени
```

```

findUsersByName(name) {
  const searchTerm = name.toLowerCase();
  return users.filter(user =>
    user.name.toLowerCase().includes(searchTerm)
  );
},

// Удаление пользователя
deleteUser(id) {
  const index = users.findIndex(user => user.id === id);
  if (index !== -1) {
    return users.splice(index, 1)[0];
  }
  return null;
},

// Статистика
getStats() {
  return {
    total: users.length,
    lastAdded: users.length > 0 ? users[users.length - 1] : null,
    firstAdded: users.length > 0 ? users[0] : null
  };
}
};
})();

// Использование модуля
UserModule.addUser('Анна', 'anna@example.com');
UserModule.addUser('Борис', 'boris@example.com');

console.log(UserModule.getAllUsers());
console.log(UserModule.findUsersByName('ан'));
console.log(UserModule.getStats());

```

4.2. Цепочка вызовов (Chaining Pattern)

javascript

```

const QueryBuilder = function() {
  this.query = {
    select: [],
    from: null,
    where: [],

```

```

    orderBy: [],
    limit: null,
    offset: null
};

// Сбрасывает построитель
this.reset = function() {
    this.query = {
        select: [],
        from: null,
        where: [],
        orderBy: [],
        limit: null,
        offset: null
    };
    return this;
};

// Добавляет поля для выборки
this.select = function(...fields) {
    this.query.select.push(...fields);
    return this;
};

// Указывает таблицу
this.from = function(table) {
    this.query.from = table;
    return this;
};

// Добавляет условие WHERE
this.where = function(condition) {
    this.query.where.push(condition);
    return this;
};

// Добавляет сортировку
this.orderBy = function(field, direction = 'ASC') {
    this.query.orderBy.push({ field, direction });
    return this;
};

// Устанавливает лимит
this.limit = function(count) {

```

```

    this.query.limit = count;
    return this;
};

// Устанавливает смещение
this.offset = function(count) {
    this.query.offset = count;
    return this;
};

// Генерирует SQL
this.build = function() {
    if (!this.query.from) {
        throw new Error('Не указана таблица (FROM)');
    }

    const parts = [];

    // SELECT
    parts.push('SELECT');
    parts.push(this.query.select.length > 0 ?
        this.query.select.join(', ') : '*');

    // FROM
    parts.push('FROM', this.query.from);

    // WHERE
    if (this.query.where.length > 0) {
        parts.push('WHERE', this.query.where.join(' AND '));
    }

    // ORDER BY
    if (this.query.orderBy.length > 0) {
        const orderClauses = this.query.orderBy.map(
            ({ field, direction }) => `${field} ${direction}`
        );
        parts.push('ORDER BY', orderClauses.join(', '));
    }

    // LIMIT u OFFSET
    if (this.query.limit !== null) {
        parts.push('LIMIT', this.query.limit);
        if (this.query.offset !== null) {
            parts.push('OFFSET', this.query.offset);
        }
    }
};

```

```

    }
  }

  return parts.join(' ');
};
};

```

// Использование с цепочкой вызовов

```

const builder = new QueryBuilder();
const sql = builder
  .reset()
  .select('id', 'name', 'email')
  .from('users')
  .where('active = true')
  .where('age >= 18')
  .orderBy('name', 'ASC')
  .limit(10)
  .offset(0)
  .build();

```

```
console.log(sql);
```

// SELECT id, name, email FROM users WHERE active = true AND age >= 18

// ORDER BY name ASC LIMIT 10 OFFSET 0

4.3. Декораторы методов (ES7/TypeScript)

javascript

// Декоратор для логирования вызовов методов

```

function logMethod(target, propertyName, descriptor) {
  const originalMethod = descriptor.value;

  descriptor.value = function(...args) {
    console.log(`Вызов метода ${propertyName} с аргументами:`, args);
    console.log('Контекст (this):', this);

    const startTime = performance.now();
    const result = originalMethod.apply(this, args);
    const endTime = performance.now();

    console.log(`Метод ${propertyName} выполнен за ${endTime - startTime}ms`);
    console.log('Результат:', result);

    return result;
  };
}

```

```

    };

    return descriptor;
}

// Декоратор для проверки аргументов
function validateArgs(validationFn) {
    return function(target, propertyName, descriptor) {
        const originalMethod = descriptor.value;

        descriptor.value = function(...args) {
            if (!validationFn(...args)) {
                throw new Error(`Невалидные аргументы для метода ${propertyName}`);
            }
            return originalMethod.apply(this, args);
        };

        return descriptor;
    };
}

// Класс с декорированными методами
class Calculator {
    @logMethod
    @validateArgs((a, b) => typeof a === 'number' && typeof b === 'number')
    add(a, b) {
        return a + b;
    }

    @logMethod
    multiply(a, b) {
        // Имитация долгой операции
        let result = 0;
        for (let i = 0; i < 1000000; i++) {
            result = a * b;
        }
        return result;
    }
}

const calc = new Calculator();
console.log(calc.add(5, 3));
// Вызов метода add с аргументами: [5, 3]
// Контекст (this): Calculator {}

```

```
// Метод add выполнен за Xms  
// Результат: 8  
// 8  
  
console.log(calc.multiply(4, 5));  
// ...логирование...  
// 20
```

Заключение: ключевые принципы работы со свойствами и методами

Основные выводы:

1. **Свойства объектов** — это фундаментальная структура данных в JavaScript:
 - ➔ Различайте собственные и унаследованные свойства
 - ➔ Используйте дескрипторы для контроля поведения свойств
 - ➔ Помните о символьных свойствах для создания "скрытых" полей
2. **Методы объектов** — это функции, связанные с объектами:
 - ➔ Правильно работайте с контекстом (`this`)
 - ➔ Используйте статические методы для утилитарных функций
 - ➔ Приватные методы (ES2022+) улучшают инкапсуляцию
3. **Паттерны проектирования** упрощают работу с объектами:
 - ➔ Модульный паттерн для инкапсуляции
 - ➔ Паттерн цепочки вызовов для построителей запросов
 - ➔ Декораторы для расширения функциональности методов

Рекомендации для практического использования:

1. Всегда используйте `Object.hasOwnProperty()` для проверки собственных свойств
2. Предпочитайте методы `Object.keys()`, `Object.values()`, `Object.entries()` для итераций
3. Используйте геттеры и сеттеры для контроля доступа к свойствам
4. Применяйте `Object.freeze()` и `Object.seal()` для защиты объектов
5. Используйте стрелочные функции или `.bind()` для сохранения контекста в методах
6. Документируйте методы с помощью JSDoc для лучшей поддерживаемости

Свойства и методы объектов — это основа объектно-ориентированного программирования в JavaScript. Глубокое понимание этих концепций позволяет создавать чистый, поддерживаемый и эффективный код.

Введение: что такое `this` и почему это важно

Ключевое слово `this` — одна из самых мощных и одновременно запутанных концепций в JavaScript. В отличие от многих других языков, где `this` всегда ссылается на текущий объект, в JavaScript значение `this` определяется тем, **как** вызывается функция, а не где она объявлена.

Основная концепция:

`this` — это специальная переменная, которая:

1. Автоматически создается в контексте каждой функции
2. Ссылается на объект, который вызывает функцию
3. Может меняться в зависимости от способа вызова функции

Часть 1: Основные правила определения `this`

1.1. Четыре правила привязки `this`

В JavaScript существует четыре основных правила, определяющих значение `this`:

Правило 1: Вызов в глобальном контексте (Default Binding)

```
javascript

// В нестрогом режиме
console.log(this); // window (в браузере) или global (в Node.js)

function showGlobalThis() {
  console.log(this); // window (в браузере)
}

showGlobalThis(); // window - вызов без контекста
```

```
// В строгом режиме ('use strict')
function showStrictThis() {
  'use strict';
  console.log(this); // undefined
}
showStrictThis(); // undefined
```

Правило 2: Вызов как метода объекта (Implicit Binding)

```
javascript

const user = {
  name: 'Анна',
  age: 25,

  showInfo() {
    // this ссылается на объект, которому принадлежит метод
    console.log(`Имя: ${this.name}, Возраст: ${this.age}`);
  },

  updateAge(newAge) {
    this.age = newAge;
    console.log(`Возраст обновлен: ${this.age}`);
  }
};

user.showInfo(); // Имя: Анна, Возраст: 25
// ↑ this = user

user.updateAge(26); // Возраст обновлен: 26
// ↑ this = user
```

Правило 3: Явная привязка (Explicit Binding)

```
javascript

function greet() {
  console.log(`Привет, ${this.name}!`);
}

const person1 = { name: 'Мария' };
const person2 = { name: 'Иван' };

// 1. Метод call() - немедленный вызов с указанным this
greet.call(person1); // Привет, Мария!
greet.call(person2); // Привет, Иван!
```

```
// 2. Метод apply() - аналогичен call, но аргументы передаются массивом
function introduce(greeting, punctuation) {
  console.log(`${greeting}, я ${this.name}${punctuation}`);
}

introduce.apply(person1, ['Здравствуйте', '!']); // Здравствуйте, я Мария!

// 3. Метод bind() - создает новую функцию с привязанным this
const greetMaria = greet.bind(person1);
greetMaria(); // Привет, Мария!

const greetIvan = greet.bind(person2);
greetIvan(); // Привет, Иван!
```

Правило 4: Вызов с оператором `new` (New Binding)

```
javascript

function Car(brand, model) {
  // При вызове с new:
  // 1. Создается новый пустой объект
  // 2. this связывается с этим объектом
  // 3. Объект связывается с прототипом функции
  // 4. Возвращается этот объект (если нет явного return)

  this.brand = brand;
  this.model = model;
  this.year = new Date().getFullYear();

  this.getInfo = function() {
    return `${this.brand} ${this.model} (${this.year})`;
  };
}

const myCar = new Car('Toyota', 'Camry');
console.log(myCar.getInfo()); // Toyota Camry (2024)
console.log(myCar.brand); // Toyota
```

1.2. Приоритет правил

Правила имеют следующий приоритет (от высшего к низшему):

1. **New Binding** (`new`) — высший приоритет

2. **Explicit Binding** (call, apply, bind)
3. **Implicit Binding** (вызов как метода объекта)
4. **Default Binding** (вызов в глобальном контексте)

javascript

```
function testPriority() {  
    console.log(this.value);  
}
```

```
const obj1 = { value: 'Implicit', testPriority };  
const obj2 = { value: 'Explicit' };  
const obj3 = { value: 'New' };
```

// Правило 1: Implicit Binding

```
obj1.testPriority(); // 'Implicit'
```

// Правило 2: Explicit Binding имеет приоритет

```
obj1.testPriority.call(obj2); // 'Explicit'
```

// Правило 3: New Binding имеет высший приоритет

```
const newObj = new obj1.testPriority(); // undefined (this в конструкторе)
```

Часть 2: Особые случаи и нюансы

2.1. Потеря контекста (Lost Context)

javascript

```
const user = {  
    name: 'Алексей',  
    tasks: ['Задача 1', 'Задача 2', 'Задача 3'],  
  
    showTasks() {  
        console.log(`${this.name} имеет задачи:`);  
        this.tasks.forEach(function(task) {  
            // ПРОБЛЕМА: this здесь не user!  
            console.log(`- ${task} (назначена ${this.name})`);  
        });  
    }  
};
```

```
user.showTasks();
```

// Алексей имеет задачи:

```
// - Задача 1 (назначена undefined) ← ОШИБКА!  
// - Задача 2 (назначена undefined)  
// - Задача 3 (назначена undefined)
```

Решения проблемы потери контекста:

javascript

```
const user = {  
  name: 'Алексей',  
  tasks: ['Задача 1', 'Задача 2', 'Задача 3'],  
  
  // Решение 1: Сохранение this в переменную  
  showTasksSolution1() {  
    console.log(`${this.name} имеет задачи:`);  
    const self = this; // Сохраняем this  
    this.tasks.forEach(function(task) {  
      console.log(`- ${task} (назначена ${self.name})`);  
    });  
  },  
  
  // Решение 2: Использование стрелочной функции  
  showTasksSolution2() {  
    console.log(`${this.name} имеет задачи:`);  
    this.tasks.forEach(task => {  
      // Стрелочные функции не имеют своего this  
      // Они берут this из окружающего контекста  
      console.log(`- ${task} (назначена ${this.name})`);  
    });  
  },  
  
  // Решение 3: Использование bind  
  showTasksSolution3() {  
    console.log(`${this.name} имеет задачи:`);  
    this.tasks.forEach(function(task) {  
      console.log(`- ${task} (назначена ${this.name})`);  
    }.bind(this)); // Привязываем this  
  },  
  
  // Решение 4: Передача this как параметра  
  showTasksSolution4() {  
    console.log(`${this.name} имеет задачи:`);  
    this.tasks.forEach(function(task) {  
      console.log(`- ${task} (назначена ${this.name})`);  
    }, this); // Второй параметр forEach
```

```

    }
};

user.showTasksSolution2();
// Алексей имеет задачи:
// - Задача 1 (назначена Алексей) ← ПРАВИЛЬНО!
// - Задача 2 (назначена Алексей)
// - Задача 3 (назначена Алексей)

```

2.2. Стрелочные функции и `this`

```

javascript

// Важное отличие: стрелочные функции НЕ имеют своего this

const obj = {
  value: 'Объект',

  // Обычная функция
  regularFunc: function() {
    console.log('Обычная функция:', this.value);

    // Вложенная обычная функция теряет контекст
    function innerRegular() {
      console.log('Вложенная обычная:', this.value); // undefined
    }
    innerRegular();

    // Вложенная стрелочная функция сохраняет контекст
    const innerArrow = () => {
      console.log('Вложенная стрелочная:', this.value); // 'Объект'
    };
    innerArrow();
  },

  // Стрелочная функция
  arrowFunc: () => {
    console.log('Стрелочная функция:', this.value); // undefined!
    // Стрелочная функция берет this из окружающего контекста
    // В данном случае - глобальный контекст
  }
};

obj.regularFunc();

```

```

// Обычная функция: Объект
// Вложенная обычная: undefined
// Вложенная стрелочная: Объект

obj.arrowFunc(); // Стрелочная функция: undefined

// Практический пример использования стрелочных функций
class Timer {
  constructor() {
    this.seconds = 0;
  }

  start() {
    // Обычная функция изменила бы this
    // setInterval(function() {
    //   this.seconds++; // ОШИБКА: this не timer
    // }, 1000);

    // Стрелочная функция сохраняет this из класса
    setInterval(() => {
      this.seconds++;
      console.log(`Прошло секунд: ${this.seconds}`);
    }, 1000);
  }
}

const timer = new Timer();
timer.start(); // Корректно работает

```

2.3. События и this

```

javascript

class ButtonComponent {
  constructor() {
    this.clickCount = 0;
    this.button = document.createElement('button');
    this.button.textContent = 'Нажми меня';

    // Проблема: при клике this будет ссылаться на button, не на компонент
    // this.button.addEventListener('click', this.handleClick);

    // Решение 1: Использовать стрелочную функцию
    this.button.addEventListener('click', () => {

```



```

        this.handleClick(); // this = ButtonComponent
    });

    // Решение 2: Использовать bind
    // this.button.addEventListener('click', this.handleClick.bind(this));
}

handleClick() {
    this.clickCount++;
    console.log(`Кнопка нажата ${this.clickCount} раз`);
    this.button.textContent = `Нажато: ${this.clickCount}`;
}

render(container) {
    container.appendChild(this.button);
}
}

// HTML для теста
const html = `
<div id="app"></div>
<script>
    const app = document.getElementById('app');
    const buttonComponent = new ButtonComponent();
    buttonComponent.render(app);
</script>
`;

```

2.4. this в конструкторах и классах

```

javascript

// В ES5 функциях-конструкторах
function Person(name) {
    this.name = name;

    // Метод в конструкторе - каждый экземпляр получает свою копию
    this.sayHi = function() {
        console.log(`Привет, я ${this.name}`);
    };
}

Person.prototype.sayBye = function() {
    console.log(`Пока от ${this.name}`);
}

```

```
};
```

```
// В ES6+ классах
```

```
class Employee {  
  constructor(name, position) {  
    this.name = name;  
    this.position = position;  
  
    // Автоматически привязывается к экземпляру  
    this.showInfo = this.showInfo.bind(this);  
  }  
  
  // Методы в классе автоматически добавляются в прототип  
  showInfo() {  
    console.log(`${this.name} - ${this.position}`);  
  }  
}
```

```
// Стрелочная функция как метод класса
```

```
// Автоматически привязывает this к экземпляру
```

```
arrowMethod = () => {  
  console.log(`Стрелочный метод: ${this.name}`);  
};  
}
```

```
const alice = new Employee('Алиса', 'Разработчик');  
const show = alice.showInfo;  
show(); // 'Алиса - Разработчик' (благодаря bind в конструкторе)
```

```
const arrow = alice.arrowMethod;  
arrow(); // 'Стрелочный метод: Алиса' (стрелочная функция сохраняет this)
```

Часть 3: Продвинутые техники работы с `this`

3.1. Каррирование с `bind`

```
javascript
```

```
// Частичное применение функций с фиксацией this и аргументов
```

```
function multiply(a, b) {  
  return a * b;  
}
```

```

// Фиксация первого аргумента
const double = multiply.bind(null, 2);
console.log(double(5)); // 10
console.log(double(10)); // 20

// Фиксация this и аргументов
const calculator = {
  base: 100,

  calculate(percent, extra) {
    return this.base * (percent / 100) + extra;
  }
};

// Создание специализированных функций
const calculateWithBase = calculator.calculate.bind(calculator);
console.log(calculateWithBase(50, 10)); // 60 (100 * 0.5 + 10)

const calculateTenPercent = calculator.calculate.bind(calculator, 10);
console.log(calculateTenPercent(5)); // 15 (100 * 0.1 + 5)

// Практический пример: логирование с контекстом
function createLogger(context) {
  return function(...args) {
    console.log(`[${context}]`, ...args);
  };
}

const userLogger = createLogger('UserModule');
const apiLogger = createLogger('APIModule');

userLogger('Пользователь создан'); // [UserModule] Пользователь создан
apiLogger('Запрос отправлен'); // [APIModule] Запрос отправлен

```

3.2. Декораторы с сохранением контекста

javascript

```

// Декоратор для измерения времени выполнения
function measureTime(target, name, descriptor) {
  const original = descriptor.value;

  descriptor.value = function(...args) {
    const start = performance.now();

```

```

    const result = original.apply(this, args); // Сохраняем контекст
    const end = performance.now();

    console.log(`Метод ${name} выполнен за ${(end - start).toFixed(2)}ms`);
    return result;
};

return descriptor;
}

// Декоратор для логирования с контекстом
function logWithContext(message) {
    return function(target, name, descriptor) {
        const original = descriptor.value;

        descriptor.value = function(...args) {
            console.log(`${message}: ${this.constructor.name}.${name}`);
            console.log('Контекст:', this);
            console.log('Аргументы:', args);

            const result = original.apply(this, args);

            console.log('Результат:', result);
            return result;
        };

        return descriptor;
    };
}

class DataProcessor {
    constructor(name) {
        this.name = name;
        this.data = [];
    }

    @measureTime
    @logWithContext('Обработка данных')
    processData(items) {
        // Имитация обработки
        let sum = 0;
        for (let i = 0; i < 1000000; i++) {
            sum += Math.random();
        }
    }
}

```

```

    this.data = items.map(item => ({
      ...item,
      processed: true,
      timestamp: new Date()
    }));

    return this.data.length;
  }
}

const processor = new DataProcessor('TestProcessor');
processor.processData([
  { id: 1 }, { id: 2 }, { id: 3 }
]);

```

3.3. Динамическое изменение контекста

```

javascript

// Паттерн "Контекстный менеджер"
function withContext(context, fn) {
  return function(...args) {
    // Сохраняем текущий контекст
    const prevContext = this;

    try {
      // Устанавливаем новый контекст
      return fn.apply(context, args);
    } finally {
      // Восстанавливаем контекст (если нужно)
      // В реальных сценариях это может быть сложнее
    }
  };
}

// Пример использования
const service = {
  name: 'Сервис',

  async fetchData(url) {
    console.log(`Запрос от ${this.name} к ${url}`);
    // Имитация запроса
    return new Promise(resolve => {
      setTimeout(() => {
        resolve({ data: 'ответ', from: this.name });
      });
    });
  }
};

```

```

        }, 100));
    });
}
};

const userService = { name: 'UserService' };
const productService = { name: 'ProductService' };

// Создаем версии метода с разными контекстами
const fetchForUsers = withContext(userService, service.fetchData);
const fetchForProducts = withContext(productService, service.fetchData);

// Используем
fetchForUsers('/api/users').then(console.log);
// Запрос от UserService к /api/users
// { data: 'ответ', from: 'UserService' }

fetchForProducts('/api/products').then(console.log);
// Запрос от ProductService к /api/products
// { data: 'ответ', from: 'ProductService' }

```

Часть 4: Практические паттерны и антипаттерны

4.1. Паттерн "Module Pattern" с правильным `this`

```

javascript

const UserModule = (function() {
    // Приватные переменные
    let users = [];
    let currentUser = null;

    // Приватные методы
    function validateUser(user) {
        return user && user.name && user.email;
    }

    function logAction(action) {
        console.log(`[${new Date().toISOString()}] ${action}`);
    }

    // Публичный интерфейс

```

```

return {
  setCurrentUser(user) {
    if (validateUser(user)) {
      currentUser = user;
      logAction(`Установлен текущий пользователь: ${user.name}`);
      return true;
    }
    return false;
  },

  getCurrentUser() {
    return currentUser ? { ...currentUser } : null;
  },

  addUser(user) {
    if (!validateUser(user)) {
      throw new Error('Некорректные данные пользователя');
    }

    users.push({
      ...user,
      id: Date.now(),
      createdAt: new Date()
    });

    logAction(`Добавлен пользователь: ${user.name}`);
    return users[users.length - 1].id;
  },

  // Метод, который может передаваться как колбэк
  getUserById: (function() {
    // Используем стрелочную функцию для сохранения контекста
    return (id) => {
      const user = users.find(u => u.id === id);
      if (user && currentUser) {
        logAction(`${currentUser.name} запросил пользователя ${user.name}`);
      }
      return user ? { ...user } : null;
    };
  })()
};
})();

```

// Использование

```
UserModule.setCurrentUser({ name: 'Админ', email: 'admin@test.com' });
UserModule.addUser({ name: 'Иван', email: 'ivan@test.com' });

// getUserById можно безопасно передавать как колбэк
const findUser = UserModule.getUserById;
setTimeout(() => {
  const user = findUser(/* id */);
  console.log(user);
}, 1000);
```

4.2. Антипаттерны и как их избежать

javascript

```
// АНТИПАТТЕРН 1: Использование this в конструкторе до super()
class BaseClass {
  constructor() {
    this.baseValue = 'base';
  }
}

class DerivedClass extends BaseClass {
  constructor() {
    // АНТИПАТТЕРН: использование this до super()
    // this.derivedValue = 'derived'; // ReferenceError

    // ПРАВИЛЬНО: сначала вызываем super()
    super();
    this.derivedValue = 'derived'; // OK
  }
}

// АНТИПАТТЕРН 2: Передача методов как колбэков без привязки контекста
class EventEmitter {
  constructor() {
    this.events = [];
  }

  addEvent(name, handler) {
    this.events.push({ name, handler });
  }

  trigger(name) {
    this.events
```



```

        .filter(event => event.name === name)
        .forEach(event => event.handler());
    }
}

class Component {
    constructor() {
        this.value = 42;
        this.emitter = new EventEmitter();

        // АНТИПАТТЕРН: передача метода без привязки контекста
        // this.emitter.addEvent('click', this.handleClick);

        // ПРАВИЛЬНО: привязываем контекст
        this.emitter.addEvent('click', this.handleClick.bind(this));

        // ИЛИ: используем стрелочную функцию
        this.emitter.addEvent('click', () => this.handleClick());
    }

    handleClick() {
        console.log(`Значение: ${this.value}`); // Должно быть 42
    }
}

// АНТИПАТТЕРН 3: Изменение this через call/apply в стрелочных функциях
const arrowFunc = () => {
    console.log(this); // Всегда берет this из окружающего контекста
};

const obj = { value: 'test' };

// НЕ РАБОТАЕТ: стрелочные функции игнорируют call/apply
arrowFunc.call(obj); // this не изменится!

// ПРАВИЛЬНО: для явного контекста используйте обычные функции
function regularFunc() {
    console.log(this);
}

regularFunc.call(obj); // { value: 'test' }

```

4.3. Паттерн "Command Pattern" с сохранением контекста

javascript

// Паттерн Команда с правильной обработкой this

```
class Command {
  constructor(receiver, action, ...args) {
    this.receiver = receiver;
    this.action = action;
    this.args = args;
    this.executed = false;
  }

  execute() {
    const result = this.action.apply(this.receiver, this.args);
    this.executed = true;
    return result;
  }

  undo() {
    // Реализация отмены действия
    console.log('Отмена команды');
  }
}
```

// Ресивер (получатель команд)

```
class TextEditor {
  constructor() {
    this.content = '';
    this.history = [];
  }

  write(text) {
    this.content += text;
    console.log(`Текст добавлен: "${text}"`);
    console.log(`Текущее содержание: "${this.content}"`);
    return this;
  }

  deleteLast() {
    const deleted = this.content.slice(-1);
    this.content = this.content.slice(0, -1);
    console.log(`Удален символ: "${deleted}"`);
    return deleted;
  }
}
```

```

    }

    clear() {
        const oldContent = this.content;
        this.content = '';
        console.log(`Текст очищен. Было: "${oldContent}"`);
        return oldContent;
    }
}

// Клиентский код
const editor = new TextEditor();

// Создаем команды
const commands = [
    new Command(editor, editor.write, 'Привет, '),
    new Command(editor, editor.write, 'мир!'),
    new Command(editor, editor.deleteLast)
];

// Выполняем команды
commands.forEach((command, index) => {
    console.log(`\nКоманда ${index + 1}:`);
    command.execute();
});

// История команд позволяет отменять действия
console.log('\n--- Отмена последней команды ---');
commands[commands.length - 1].undo();
console.log(`Финальный текст: "${editor.content}"`);

```

Часть 5: Тестирование и отладка работы с `this`

5.1. Инструменты для отладки `this`

```

javascript

// Утилита для логирования this
function logThis(prefix = '') {
    console.group(`${prefix} Значение this:`);
    console.log('Тип:', typeof this);
    console.log('Значение:', this);
}

```

```

if (this && typeof this === 'object') {
  console.log('Ключи:', Object.keys(this));
  console.log('Прототип:', Object.getPrototypeOf(this));
}

console.groupEnd();
}

```

// Декоратор для отслеживания this

```

function trackThis(target, name, descriptor) {
  const original = descriptor.value;

  descriptor.value = function(...args) {
    console.group(`Метод ${name} вызван:`);
    console.log('this до вызова:', this);
    console.log('Аргументы:', args);

    const result = original.apply(this, args);

    console.log('this после вызова:', this);
    console.log('Результат:', result);
    console.groupEnd();

    return result;
  };

  return descriptor;
}

```

// Класс с отслеживанием this

```

class TestClass {
  constructor(value) {
    this.value = value;
  }

  @trackThis
  updateValue(newValue) {
    const oldValue = this.value;
    this.value = newValue;
    return oldValue;
  }

  @trackThis

```

```

nestedCall() {
  // Демонстрация потери контекста
  function inner() {
    console.log('this во внутренней функции:', this);
  }

  const arrowInner = () => {
    console.log('this в стрелочной функции:', this);
  };

  inner();
  arrowInner();
}
}

// Тестирование
const test = new TestClass('начальное значение');
test.updateValue('новое значение');
test.nestedCall();

```

5.2. Тестирование функций с this

javascript

```

// Фреймворк для тестирования this
class ThisTestFramework {
  static runTests(obj, testName) {
    console.group(`Тестирование: ${testName}`);

    const tests = Object.getOwnPropertyNames(Object.getPrototypeOf(obj))
      .filter(name => name.startsWith('test'))
      .map(name => ({
        name,
        method: obj[name].bind(obj)
      }));

    let passed = 0;
    let failed = 0;

    tests.forEach(({ name, method }) => {
      try {
        const result = method();
        console.log(`✓ ${name}: УСПЕХ`);
        if (result !== undefined) {

```

```

        console.log(`    Результат: ${result}`);
    }
    passed++;
} catch (error) {
    console.log(`x ${name}: ОШИБКА`);
    console.log(`    ${error.message}`);
    failed++;
}
});

console.log(`\nИтого: ${passed} успешно, ${failed} с ошибками`);
console.groupEnd();
}
}

```

// Пример тестируемого класса

```

class CalculatorTests {
    constructor() {
        this.value = 0;
    }

    testInitialValue() {
        if (this.value !== 0) {
            throw new Error(`Начальное значение должно быть 0, получено ${this.value}`);
        }
        return 'OK';
    }

    testAdd() {
        const original = this.value;
        this.value += 5;

        if (this.value !== original + 5) {
            throw new Error(`Сложение не работает: ${original} + 5 != ${this.value}`);
        }
        return 'OK';
    }

    testSubtract() {
        const original = this.value;
        this.value -= 3;

        if (this.value !== original - 3) {
            throw new Error(`Вычитание не работает: ${original} - 3 != ${this.value}`);
        }
    }
}

```

```

    }
    return 'OK';
}

// Тест на потерю контекста
testContextLoss() {
    const operations = [5, -3, 10];
    let sum = 0;

    // Ошибка: потеря контекста
    operations.forEach(function(num) {
        this.value += num; // ОШИБКА: this не определен
        sum += num;
    });

    // Правильно:
    // operations.forEach(num => {
    //     this.value += num;
    //     sum += num;
    // });

    return sum;
}

// Запуск тестов
const calculatorTests = new CalculatorTests();
ThisTestFramework.runTests(calculatorTests, 'Calculator');

```

Часть 6: Резюме и лучшие практики

6.1. Ключевые правила работы с this

1. **this** определяется в момент вызова функции, а не в момент ее объявления
2. **Приоритеты правил:** new > explicit > implicit > default
3. **Стрелочные функции не имеют своего this** — они берут его из окружающего контекста
4. **В строгом режиме глобальный this равен undefined**, в нестрогом — глобальный объект
5. **Методы** `call()`, `apply()`, `bind()` позволяют явно управлять контекстом

6.2. Лучшие практики

javascript

// 1. Всегда используйте стрелочные функции для колбэков

```
class GoodExample1 {
  constructor() {
    this.data = [];

    // ХОРОШО
    setTimeout(() => {
      this.processData();
    }, 100);

    // ПЛОХО
    setTimeout(function() {
      // this здесь не определен
      // this.processData(); // Ошибка
    }, 100);
  }

  processData() {
    console.log('Обработка данных');
  }
}
```

// 2. Привязывайте контекст в конструкторе для методов, которые будут передаваться

```
class GoodExample2 {
  constructor() {
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    console.log('Клик обработан', this);
  }
}
```

// 3. Используйте декораторы для автоматической привязки

```
function autobind(target, name, descriptor) {
  const original = descriptor.value;

  return {
    configurable: true,
    enumerable: false,
```



```

    get() {
        const bound = original.bind(this);
        Object.defineProperty(this, name, {
            value: bound,
            configurable: true,
            writable: true
        });
        return bound;
    }
};
}

class GoodExample3 {
    @autobind
    handleClick() {
        console.log('Автопривязка работает!', this);
    }
}

```

// 4. Избегайте вложенных функций с разным контекстом

```

function processItems(items) {
    // ХОРОШО
    items.forEach(item => {
        // this доступен из внешнего контекста
        this.process(item);
    });

    // ПЛОХО
    items.forEach(function(item) {
        // this потерян
        // this.process(item); // Ошибка
    });
}

```

6.3. Чек-лист для работы с `this`

Прежде чем использовать `this`, задайте себе вопросы:

1. Как вызывается функция?

- ➡ Как метод объекта? → `this` = объект
- ➡ С оператором `new`? → `this` = новый объект
- ➡ С `call`/`apply`/`bind`? → `this` = указанный объект

- ➔ Просто как функция? → `this = undefined` (strict) или глобальный объект
2. **Это стрелочная функция?**
- ➔ Да → `this` берется из окружающего контекста
 - ➔ Нет → смотри правило 1
3. **Функция передается как колбэк?**
- ➔ Да → нужно ли привязать контекст?
 - ➔ Используйте стрелочные функции или `.bind()`
4. **Работаете с событиями DOM?**
- ➔ В обработчиках событий `this` обычно ссылается на элемент
 - ➔ Используйте `event.currentTarget` для явного указания

6.4. Частые ошибки и их решения

Ошибка	Причина	Решение
<code>this</code> is undefined	Вызов функции без контекста в strict mode	Используйте <code>.bind()</code> или стрелочные функции
<code>this</code> ссылается на window	Вызов без контекста в нестрогом режиме	Включите strict mode или используйте <code>.bind()</code>
Потеря контекста в колбэках	Передача метода как колбэка	Используйте стрелочные функции или <code>.bind()</code>
<code>this</code> в стрелочной функции не меняется	Стрелочные функции фиксируют <code>this</code>	Используйте обычные функции если нужно менять контекст
Конфликт <code>this</code> в цепочке промисов	Каждый <code>.then()</code> имеет свой контекст	Сохраняйте <code>this</code> в переменную или используйте стрелочные функции

Заключение

Понимание работы `this` в JavaScript — это ключ к написанию чистого, поддерживаемого кода. Помните, что `this` динамичен и зависит от контекста вызова. Используйте современные возможности языка (стрелочные функции, `bind`, классы) для контроля над контекстом, и всегда тестируйте код, где используется `this`, особенно при передаче методов как колбэков.

Главный принцип: **ясность и предсказуемость**. Если код с `this` сложно понять — упростите его, используя явную привязку контекста или переписывая без использования `this`.

8.4. Прототипы и цепочка прототипов

Введение: прототипное наследование в JavaScript

JavaScript — это язык с **прототипным наследованием**, в отличие от классического классового наследования в таких языках, как Java или C++. Каждый объект в JavaScript имеет скрытое свойство `[[Prototype]]`, которое ссылается на другой объект или `null`.

Основные понятия:

- 🔵 **Прототип (prototype)** — объект, от которого другой объект наследует свойства и методы
- 🔵 **Цепочка прототипов (prototype chain)** — последовательность прототипов, по которой происходит поиск свойств
- 🔵 **Конструктор (constructor)** — функция, которая создает новые объекты

Часть 1: Основы прототипов

1.1. Свойство `__proto__` и метод `Object.getPrototypeOf()`

```
javascript

// Создаем базовый объект
const animal = {
  eats: true,
  walk() {
    console.log('Животное идет');
  }
};

// Создаем объект, который будет наследовать от animal
const rabbit = {
  jumps: true
};
```

```

// Устанавливаем прототип (устаревший способ, но для понимания)
rabbit.__proto__ = animal; // rabbit наследует от animal

// Проверяем наследование
console.log(rabbit.eats); // true (из прототипа)
console.log(rabbit.jumps); // true (собственное свойство)
rabbit.walk(); // 'Животное идет' (из прототипа)

// Современный способ установки прототипа
const deer = Object.create(animal); // deer.__proto__ = animal
deer.runs = true;
console.log(deer.eats); // true
console.log(deer.runs); // true

// Получение прототипа
console.log(Object.getPrototypeOf(rabbit) === animal); // true
console.log(Object.getPrototypeOf(deer) === animal); // true

// Установка прототипа (современный способ)
const fox = { sly: true };
Object.setPrototypeOf(fox, animal);
console.log(fox.eats); // true

```

1.2. Свойство `prototype` у функций-конструкторов

```

javascript

// Функция-конструктор
function Animal(name) {
    this.name = name;
    this.eats = true;
}

// Добавляем метод в прототип
Animal.prototype.walk = function() {
    console.log(`${this.name} идет`);
};

// Добавляем свойство в прототип
Animal.prototype.legs = 4;

// Создаем экземпляр
const dog = new Animal('Бобик');

```

```

// Проверяем цепочку прототипов
console.log(dog.name); // 'Бобик' (из конструктора)
console.log(dog.eats); // true (из конструктора)
console.log(dog.legs); // 4 (из прототипа)
dog.walk(); // 'Бобик идет' (из прототипа)

// Проверяем прототип
console.log(Object.getPrototypeOf(dog) === Animal.prototype); // true
console.log(dog.__proto__ === Animal.prototype); // true (устаревший способ)

// Схема цепочки прототипов:
// dog (экземпляр)
//   ↓ __proto__
// Animal.prototype
//   ↓ __proto__
// Object.prototype
//   ↓ __proto__
// null

```

Часть 2: Цепочка прототипов в действии

2.1. Как работает поиск свойств

```

javascript

// Создаем цепочку прототипов
const base = {
  baseProp: 'Я из базового объекта',
  commonMethod() {
    return 'Базовый метод';
  }
};

const middle = {
  middleProp: 'Я из среднего объекта',
  commonMethod() {
    return 'Средний метод';
  }
};

const derived = {
  derivedProp: 'Я из производного объекта'
}

```

```

};

// Устанавливаем цепочку: derived → middle → base
Object.setPrototypeOf(middle, base);
Object.setPrototypeOf(derived, middle);

// Поиск свойств в цепочке прототипов
console.log(derived.derivedProp); // 'Я из производного объекта' (найдено в derived)
console.log(derived.middleProp); // 'Я из среднего объекта' (найдено в middle)
console.log(derived.baseProp); // 'Я из базового объекта' (найдено в base)

// Переопределение методов
console.log(derived.commonMethod()); // 'Средний метод' (найдено в middle, base пропущен)

// Проверка наличия свойств
console.log('derivedProp' in derived); // true (собственное)
console.log('middleProp' in derived); // true (из прототипа)
console.log('baseProp' in derived); // true (из прототипа прототипа)
console.log('nonExistent' in derived); // false

// Итерация по свойствам
console.log(Object.keys(derived)); // ['derivedProp'] (только собственные)
console.log(Object.getOwnPropertyNames(derived)); // ['derivedProp']

// Для получения всех свойств нужно рекурсивно пройти по цепочке
function getAllProperties(obj) {
    const props = [];
    let current = obj;

    while (current) {
        props.push(...Object.getOwnPropertyNames(current));
        current = Object.getPrototypeOf(current);
    }

    return [...new Set(props)];
}

console.log(getAllProperties(derived));
// ['derivedProp', 'middleProp', 'commonMethod', 'baseProp']

```

2.2. Прототипная цепочка встроенных объектов

javascript

```

// Цепочка прототипов массива
const arr = [1, 2, 3];

console.log(arr.__proto__ === Array.prototype); // true
console.log(Array.prototype.__proto__ === Object.prototype); // true
console.log(Object.prototype.__proto__); // null

// Проверка методов в цепочке
console.log(arr.map); // function map() { [native code] } (из Array.prototype)
console.log(arr.toString); // function toString() { [native code] } (из Object.prototype)
console.log(arr.hasOwnProperty); // function hasOwnProperty() { [native code] } (из Object.prototype)

// Схема цепочки прототипов для массива:
// arr (экземпляр Array)
//   ↓ __proto__
// Array.prototype
//   ↓ __proto__
// Object.prototype
//   ↓ __proto__
// null

// Цепочка прототипов для функции
function test() {}

console.log(test.__proto__ === Function.prototype); // true
console.log(Function.prototype.__proto__ === Object.prototype); // true

// Методы функций находятся в Function.prototype
console.log(test.call); // function call() { [native code] }
console.log(test.apply); // function apply() { [native code] }
console.log(test.bind); // function bind() { [native code] }

```

2.3. Перехват доступа к несуществующим свойствам

```

javascript

// Использование Proxy для отслеживания поиска в цепочке прототипов
const handler = {
  get(target, property) {
    console.log(`Поиск свойства "${property}":`);

    // Проверяем собственное свойство
    if (property in target) {

```

```

        console.log(`  Найдено в объекте: ${target[property]}`);
        return target[property];
    }

    // Проверяем прототип
    const proto = Object.getPrototypeOf(target);
    if (proto && property in proto) {
        console.log(`  Найдено в прототипе: ${proto[property]}`);
        return proto[property];
    }

    console.log(`  Не найдено в цепочке прототипов`);
    return undefined;
}

};

const protoObj = { inheritedProp: 'Я из прототипа' };
const obj = { ownProp: 'Собственное свойство' };

Object.setPrototypeOf(obj, protoObj);
const proxiedObj = new Proxy(obj, handler);

console.log(proxiedObj.ownProp); // Поиск свойства "ownProp": Найдено в объекте...
console.log(proxiedObj.inheritedProp); // Поиск свойства "inheritedProp": Найдено в прототипе...
console.log(proxiedObj.nonExistent); // Поиск свойства "nonExistent": Не найдено...

```

Часть 3: Создание иерархий наследования

3.1. Классическое наследование (до ES6)

```

javascript

// Базовый конструктор
function Animal(name) {
    this.name = name;
    this.eats = true;
}

// Методы базового класса
Animal.prototype.walk = function() {
    console.log(`${this.name} идет`);
}

```



```

};

Animal.prototype.eat = function() {
    console.log(`${this.name} ест`);
};

// Дочерний конструктор
function Rabbit(name, color) {
    // Вызов конструктора родителя
    Animal.call(this, name); // Наследование свойств
    this.color = color;
    this.jumps = true;
}

// Наследование прототипа (важная часть!)
Rabbit.prototype = Object.create(Animal.prototype); // Наследуем методы

// Восстанавливаем конструктор
Rabbit.prototype.constructor = Rabbit;

// Добавляем методы дочернего класса
Rabbit.prototype.hop = function() {
    console.log(`${this.name} (${this.color}) прыгает`);
};

// Переопределяем метод родителя
Rabbit.prototype.walk = function() {
    console.log(`${this.name} ходит как кролик`);
};

// Создаем экземпляр
const whiteRabbit = new Rabbit('Белый кролик', 'белый');

// Проверяем наследование
console.log(whiteRabbit.name); // 'Белый кролик' (из Animal)
console.log(whiteRabbit.color); // 'белый' (из Rabbit)
console.log(whiteRabbit.eats); // true (из Animal)
console.log(whiteRabbit.jumps); // true (из Rabbit)

whiteRabbit.walk(); // 'Белый кролик ходит как кролик' (переопределенный)
whiteRabbit.eat(); // 'Белый кролик ест' (унаследованный)
whiteRabbit.hop(); // 'Белый кролик (белый) прыгает' (собственный)

// Проверяем цепочку прототипов

```

```
console.log(whiteRabbit instanceof Rabbit); // true
console.log(whiteRabbit instanceof Animal); // true
console.log(whiteRabbit instanceof Object); // true
```

```
// Схема цепочки:
// whiteRabbit (экземпляр Rabbit)
//   ↓ __proto__
// Rabbit.prototype
//   ↓ __proto__
// Animal.prototype
//   ↓ __proto__
// Object.prototype
//   ↓ __proto__
// null
```

3.2. Наследование в ES6+ (классы)

javascript

```
// Базовый класс
class Animal {
  constructor(name) {
    this.name = name;
    this.eats = true;
  }

  walk() {
    console.log(`${this.name} идет`);
  }

  eat() {
    console.log(`${this.name} ест`);
  }

  // Статический метод
  static isAnimal(obj) {
    return obj instanceof Animal;
  }
}

// Дочерний класс
class Rabbit extends Animal {
  constructor(name, color) {
    super(name); // Вызов конструктора родителя
  }
}
```

```

        this.color = color;
        this.jumps = true;
    }

    // Переопределение метода
    walk() {
        console.log(`${this.name} ходит как кролик`);
    }

    // Новый метод
    hop() {
        console.log(`${this.name} (${this.color}) прыгает`);
    }

    // Статический метод
    static createWhiteRabbit(name) {
        return new Rabbit(name, 'белый');
    }
}

// Создание экземпляров
const rabbit1 = new Rabbit('Серый', 'серый');
const rabbit2 = Rabbit.createWhiteRabbit('Снежок');

// Проверка наследования
console.log(rabbit1.name); // 'Серый'
console.log(rabbit1.color); // 'серый'
console.log(rabbit2.color); // 'белый'

rabbit1.walk(); // 'Серый ходит как кролик'
rabbit1.eat(); // 'Серый ест'
rabbit1.hop(); // 'Серый (серый) прыгает'

// Статические методы
console.log(Animal.isAnimal(rabbit1)); // true
console.log(Rabbit.isAnimal(rabbit1)); // true (унаследованный)

// Проверка цепочки прототипов
console.log(rabbit1.__proto__ === Rabbit.prototype); // true
console.log(Rabbit.prototype.__proto__ === Animal.prototype); // true
console.log(rabbit1 instanceof Rabbit); // true
console.log(rabbit1 instanceof Animal); // true

// Методы в прототипах

```

```
console.log(rabbit1.walk === Rabbit.prototype.walk); // true
console.log(rabbit1.eat === Animal.prototype.eat); // true
```

3.3. Множественное наследование через миксины

```
javascript

// Базовые миксины (примеси)
const CanWalk = {
  walk() {
    console.log(`${this.name} идет`);
  }
};

const CanSwim = {
  swim() {
    console.log(`${this.name} плавает`);
  }
};

const CanFly = {
  fly() {
    console.log(`${this.name} летит`);
  }
};

// Функция для применения миксинов
function applyMixins(targetClass, ...mixins) {
  mixins.forEach(mixin => {
    Object.assign(targetClass.prototype, mixin);
  });
}

// Базовый класс
class Animal {
  constructor(name) {
    this.name = name;
  }
}

// Создаем специализированные классы
class Duck extends Animal {}

// Применяем миксины к Duck
```

```
applyMixins(Duck, CanWalk, CanSwim, CanFly);

// Создаем экземпляр
const donald = new Duck('Дональд');

// Проверяем методы
donald.walk(); // 'Дональд идет'
donald.swim(); // 'Дональд плышет'
donald.fly(); // 'Дональд летит'

// Более сложный пример с приоритетами методов
const Singer = {
  sing() {
    console.log(`${this.name} поет: ${this.song || 'ля-ля-ля'}`);
  }
};

const Dancer = {
  dance() {
    console.log(`${this.name} танцует`);
  }
};

// Комбинированный миксин
const Performer = {
  perform() {
    this.sing();
    this.dance();
  }
};

class PerformerDuck extends Animal {
  constructor(name, song) {
    super(name);
    this.song = song;
  }
}

// Применяем миксины в определенном порядке
Object.assign(PerformerDuck.prototype, Singer, Dancer, Performer);

const performerDuck = new PerformerDuck('Артист', 'Кря-кря');
performerDuck.perform();
// 'Артист поет: Кря-кря'
```

```
// 'Артист танцует'
```

Часть 4: Работа с прототипами

4.1. Динамическое изменение прототипов

```
javascript

// Создаем базовый объект
const vehicle = {
  move() {
    console.log(`${this.type} движется`);
  }
};

// Создаем конструктор
function Car(model) {
  this.model = model;
  this.type = 'автомобиль';
}

// Устанавливаем прототип
Car.prototype = vehicle;

// Создаем экземпляр
const bmw = new Car('BMW');

// Проверяем
bmw.move(); // 'автомобиль движется'

// Динамически изменяем прототип
vehicle.stop = function() {
  console.log(`${this.type} останавливается`);
};

// Новый метод доступен всем экземплярам
bmw.stop(); // 'автомобиль останавливается'

// Изменение метода в прототипе
vehicle.move = function() {
  console.log(`${this.type} движется быстро`);
};
```

```

// Все экземпляры получают обновление
bmw.move(); // 'автомобиль движется быстро'

// Добавление метода только в один экземпляр
bmw.honk = function() {
    console.log('Бип-бип!');
};

bmw.honk(); // 'Бип-бип!'

// Другой экземпляр не имеет этого метода
const audi = new Car('Audi');
// audi.honk(); // Ошибка: audi.honk is not a function

```

4.2. Создание объектов без прототипа

```

javascript

// Обычный объект имеет прототип Object.prototype
const obj1 = {};
console.log(Object.getPrototypeOf(obj1) === Object.prototype); // true

// Создание объекта без прототипа
const obj2 = Object.create(null); // __proto__ = null

console.log(Object.getPrototypeOf(obj2)); // null
console.log(obj2.toString); // undefined
console.log(obj2.valueOf); // undefined

// Преимущества объектов без прототипа:
// 1. Нет лишних унаследованных методов
// 2. Можно использовать как чистый словарь
// 3. Безопаснее для хранения данных

// Пример использования как чистого словаря
const pureDict = Object.create(null);

pureDict.key1 = 'value1';
pureDict.key2 = 'value2';
pureDict.toString = 'это строка, не метод';

// В обычном объекте это переопределило бы метод toString
console.log(pureDict.toString); // 'это строка, не метод'

```

```

// Использование как Map-подобной структуры
const safeStorage = Object.create(null);

safeStorage['constructor'] = 'это строка';
safeStorage['__proto__'] = 'и это строка';

console.log(safeStorage.constructor); // 'это строка'
console.log(safeStorage.__proto__); // 'и это строка'

// В обычном объекте это вызвало бы проблемы:
const unsafeStorage = {};
unsafeStorage['__proto__'] = 'опасно!'; // Может привести к уязвимостям

```

4.3. Оптимизация производительности

```

javascript

// Создаем большое количество объектов
class OptimizedObject {
  constructor(id) {
    this.id = id;
  }

  // Метод в прототипе - все экземпляры используют одну функцию
  logId() {
    console.log(`ID: ${this.id}`);
  }
}

// Неоптимизированный вариант
function UnoptimizedObject(id) {
  this.id = id;
  this.logId = function() { // Каждый экземпляр получает свою копию функции
    console.log(`ID: ${this.id}`);
  };
}

// Тестирование производительности
console.time('Optimized');
const optimized = [];
for (let i = 0; i < 100000; i++) {
  const obj = new OptimizedObject(i);
  optimized.push(obj);
}

```



```

}
console.timeEnd('Optimized'); // ~10-20ms

console.time('Unoptimized');
const unoptimized = [];
for (let i = 0; i < 100000; i++) {
    const obj = new UnoptimizedObject(i);
    unoptimized.push(obj);
}
console.timeEnd('Unoptimized'); // ~100-200ms

// Проверка использования памяти
console.log('Память оптимизированных:',
    JSON.stringify(optimized[0]).length); // Меньше

console.log('Память неоптимизированных:',
    JSON.stringify(unoptimized[0]).length); // Больше

// Метод в прототипе - одна функция для всех
console.log(optimized[0].logId === optimized[1].logId); // true

// Метод в конструкторе - разные функции
console.log(unoptimized[0].logId === unoptimized[1].logId); // false

```

Часть 5: Методы для работы с прототипами

5.1. Object.create(), Object.getPrototypeOf(), Object.setPrototypeOf()

```

javascript

// Object.create() - создание объекта с указанным прототипом
const proto = {
    greet() {
        return `Привет, я ${this.name}`;
    }
};

const obj1 = Object.create(proto, {
    name: {
        value: 'Анна',
        writable: true,

```

```

        enumerable: true
    },
    age: {
        value: 25,
        writable: true
    }
});

console.log(obj1.greet()); // 'Привет, я Анна'
console.log(Object.getPrototypeOf(obj1) === proto); // true

// Object.getPrototypeOf() - получение прототипа
const obj2 = {};
console.log(Object.getPrototypeOf(obj2) === Object.prototype); // true

// Object.setPrototypeOf() - установка прототипа
const newProto = {
    farewell() {
        return `Пока, ${this.name}!`;
    }
};

Object.setPrototypeOf(obj1, newProto);

console.log(obj1.farewell()); // 'Пока, Анна!'
// Старый метод больше не доступен
// console.log(obj1.greet()); // Ошибка: obj1.greet is not a function

// Восстановим старый прототип
Object.setPrototypeOf(obj1, proto);
console.log(obj1.greet()); // 'Привет, я Анна'

```

5.2. Object.getOwnPropertyNames(), Object.getOwnPropertySymbols()

```

javascript

const sym = Symbol('id');

const obj = {
    name: 'Объект',
    [sym]: 12345,
    method() {
        return 'метод';
    }
};

```

```

    }
};

// Устанавливаем прототип
Object.setPrototypeOf(obj, {
    inheritedProp: 'Унаследованное свойство',
    [Symbol('inheritedSym')]: 'Унаследованный символ'
});

// Получение всех собственных строковых свойств
console.log(Object.getOwnPropertyNames(obj));
// ['name', 'method']

// Получение всех собственных символьных свойств
console.log(Object.getOwnPropertySymbols(obj));
// [Symbol(id)]

// Получение всех собственных дескрипторов
console.log(Object.getOwnPropertyDescriptors(obj));
// {
//   name: {value: 'Объект', writable: true, enumerable: true, configurable: true},
//   method: {value: f, writable: true, enumerable: true, configurable: true}
// }

// Для получения всех свойств (включая унаследованные) нужно пройти по цепочке
function* getAllPropertyNames(obj) {
    let current = obj;

    while (current) {
        yield* Object.getOwnPropertyNames(current);
        yield* Object.getOwnPropertySymbols(current);
        current = Object.getPrototypeOf(current);
    }
}

const allProps = [...new Set(getAllPropertyNames(obj))];
console.log(allProps);
// ['name', 'method', Symbol(id), 'inheritedProp', Symbol(inheritedSym),
//   'constructor', '__defineGetter__', '__defineSetter__', ... и другие от
//   Object.prototype]

```

5.3. instanceof и isPrototypeOf()

javascript

// Базовые конструкторы

```
function Animal() {}
```

```
function Rabbit() {}
```

// Настраиваем наследование

```
Rabbit.prototype = Object.create(Animal.prototype);
```

```
Rabbit.prototype.constructor = Rabbit;
```

// Создаем экземпляры

```
const animal = new Animal();
```

```
const rabbit = new Rabbit();
```

// Оператор instanceof

```
console.log(animal instanceof Animal); // true
```

```
console.log(rabbit instanceof Rabbit); // true
```

```
console.log(rabbit instanceof Animal); // true (из-за цепочки прототипов)
```

```
console.log(animal instanceof Rabbit); // false
```

// Как работает instanceof

```
function customInstanceOf(obj, constructor) {
```

```
    let proto = Object.getPrototypeOf(obj);
```

```
    while (proto) {
```

```
        if (proto === constructor.prototype) {
```

```
            return true;
```

```
        }
```

```
        proto = Object.getPrototypeOf(proto);
```

```
    }
```

```
    return false;
```

```
}
```

```
console.log(customInstanceOf(rabbit, Animal)); // true
```

```
console.log(customInstanceOf(rabbit, Rabbit)); // true
```

// Метод isPrototypeOf()

```
console.log(Animal.prototype.isPrototypeOf(animal)); // true
```

```
console.log(Animal.prototype.isPrototypeOf(rabbit)); // true
```

```
console.log(Rabbit.prototype.isPrototypeOf(rabbit)); // true
```

```
console.log(Rabbit.prototype.isPrototypeOf(animal)); // false
```

```

// Сравнение с instanceof
const fakeRabbit = { __proto__: Rabbit.prototype };
console.log(fakeRabbit instanceof Rabbit); // false (нет правильного constructor)
console.log(Rabbit.prototype.isPrototypeOf(fakeRabbit)); // true

// Особый случай: измененный constructor
function WeirdConstructor() {}
WeirdConstructor.prototype = {}; // Не имеем constructor

const weird = new WeirdConstructor();
console.log(weird instanceof WeirdConstructor); // false!
console.log(WeirdConstructor.prototype.isPrototypeOf(weird)); // true

```

Часть 6: Паттерны проектирования с прототипами

6.1. Паттерн "Прототип" (Prototype Pattern)

javascript

```

// Базовый прототип
const carPrototype = {
  wheels: 4,
  color: 'white',

  clone() {
    // Создаем новый объект с тем же прототипом
    const clone = Object.create(Object.getPrototypeOf(this));

    // Копируем собственные свойства
    Object.getOwnPropertyNames(this).forEach(prop => {
      const descriptor = Object.getOwnPropertyDescriptor(this, prop);
      Object.defineProperty(clone, prop, descriptor);
    });

    return clone;
  },

  customize(config) {
    const customized = this.clone();
    Object.assign(customized, config);
    return customized;
  }
};

```

```

    },

    describe() {
        return `${this.make} ${this.model} (${this.color}, ${this.wheels} колеса)`;
    }
};

// Создаем конкретные прототипы
const sedanPrototype = Object.create(carPrototype, {
    make: { value: 'Toyota', enumerable: true },
    model: { value: 'Camry', enumerable: true },
    type: { value: 'sedan', enumerable: true }
});

const suvPrototype = Object.create(carPrototype, {
    make: { value: 'Toyota', enumerable: true },
    model: { value: 'RAV4', enumerable: true },
    type: { value: 'SUV', enumerable: true },
    wheels: { value: 4, enumerable: true }
});

// Создаем экземпляры через клонирование
const baseSedan = sedanPrototype.clone();
console.log(baseSedan.describe()); // Toyota Camry (white, 4 колеса)

// Кастомизируем
const redSedan = sedanPrototype.customize({
    color: 'red',
    wheels: 4
});

const blackSuv = suvPrototype.customize({
    color: 'black',
    wheels: 4,
    hasSunroof: true
});

console.log(redSedan.describe()); // Toyota Camry (red, 4 колеса)
console.log(blackSuv.describe()); // Toyota RAV4 (black, 4 колеса)

// Проверяем прототипы
console.log(Object.getPrototypeOf(redSedan) === sedanPrototype); // true
console.log(redSedan instanceof carPrototype.constructor); // false
console.log(carPrototype.isPrototypeOf(redSedan)); // true

```

6.2. Паттерн "Фабрика на прототипах"

```
javascript

// Реестр прототипов
const prototypeRegistry = {};

// Базовый класс для регистрации прототипов
class PrototypeFactory {
  static register(type, prototype) {
    prototypeRegistry[type] = prototype;
  }

  static create(type, config = {}) {
    const prototype = prototypeRegistry[type];

    if (!prototype) {
      throw new Error(`Прототип "${type}" не зарегистрирован`);
    }

    // Создаем клон и применяем конфигурацию
    const instance = Object.create(Object.getPrototypeOf(prototype));
    Object.assign(instance, prototype, config);

    return instance;
  }

  static listTypes() {
    return Object.keys(prototypeRegistry);
  }
}

// Определяем прототипы
const buttonPrototype = {
  type: 'button',
  width: 100,
  height: 40,
  label: 'Кнопка',

  render() {
    console.log(`Рендер кнопки: ${this.label} (${this.width}x${this.height})`);
    return `<button style="width: ${this.width}px; height: ${this.height}px">
      ${this.label}
    </button>`;
  }
}
```

```

    }
};

const inputPrototype = {
  type: 'input',
  width: 200,
  height: 30,
  placeholder: 'Введите текст',

  render() {
    console.log(`Рендер поля ввода: ${this.placeholder}`);
    return `<input placeholder="${this.placeholder}"
              style="width: ${this.width}px; height: ${this.height}px">`;
  }
};

// Регистрируем прототипы
PrototypeFactory.register('button', buttonPrototype);
PrototypeFactory.register('input', inputPrototype);

// Создаем компоненты
const submitButton = PrototypeFactory.create('button', {
  label: 'Отправить',
  width: 120,
  style: 'primary'
});

const searchInput = PrototypeFactory.create('input', {
  placeholder: 'Поиск...',
  width: 300
});

const cancelButton = PrototypeFactory.create('button', {
  label: 'Отмена',
  style: 'secondary'
});

// Используем компоненты
console.log(submitButton.render());
// Рендер кнопки: Отправить (120x40)
// <button style="width: 120px; height: 40px">Отправить</button>

console.log(searchInput.render());
// Рендер поля ввода: Поиск...

```



```
// <input placeholder="Пуск..." style="width: 300px; height: 30px">
```

```
// Проверяем прототипы
```

```
console.log(Object.getPrototypeOf(submitButton) ===  
Object.getPrototypeOf(buttonPrototype)); // true  
console.log(PrototypeFactory.listTypes()); // ['button', 'input']
```

6.3. Паттерн "Делегирование" (Delegation Pattern)

javascript

```
// Объекты-делегаты
```

```
const keyboardDelegate = {  
  press(key) {  
    console.log(`Нажата клавиша: ${key}`);  
    return `key:${key}`;  
  }  
};  
  
const mouseDelegate = {  
  click(button, x, y) {  
    console.log(`Клик ${button} кнопкой в (${x}, ${y})`);  
    return `click:${button}:${x},${y}`;  
  },  
  
  move(x, y) {  
    console.log(`Перемещение в (${x}, ${y})`);  
    return `move:${x},${y}`;  
  }  
};
```

```
// Основной объект, который делегирует методы
```

```
const inputHandler = {  
  // Храним ссылки на делегаты  
  delegates: {  
    keyboard: keyboardDelegate,  
    mouse: mouseDelegate  
  },  
  
  // Делегируем методы  
  press(key) {  
    return this.delegates.keyboard.press(key);  
  },  
};
```

```

click(button, x, y) {
    return this.delegates.mouse.click(button, x, y);
},

move(x, y) {
    return this.delegates.mouse.move(x, y);
},

// Динамическое переключение делегатов
useTouch() {
    this.delegates.mouse = {
        click(button, x, y) {
            console.log(`Тап в (${x}, ${y})`);
            return `tap:${x},${y}`;
        },
        move(x, y) {
            console.log(`Свайп в (${x}, ${y})`);
            return `swipe:${x},${y}`;
        }
    };
},

// Метод, использующий несколько делегатов
handleEvent(type, ...args) {
    switch (type) {
        case 'keypress':
            return this.press(...args);
        case 'click':
            return this.click(...args);
        case 'mousemove':
            return this.move(...args);
        default:
            console.log(`Неизвестное событие: ${type}`);
    }
}

};

// Использование
console.log(inputHandler.press('Enter')); // Нажата клавиша: Enter, key:Enter
console.log(inputHandler.click('left', 100, 200)); // Клик Left кнопкой в (100, 200),
click:left:100,200

// Переключаемся на touch
inputHandler.useTouch();

```

```

console.log(inputHandler.click('left', 50, 50)); // Тан в (50, 50), tap:50,50

// Через общий обработчик
console.log(inputHandler.handleEvent('keypress', 'Space')); // Нажата клавиша: Space,
key:Space
console.log(inputHandler.handleEvent('mousemove', 150, 250)); // Свайп в (150, 250),
swipe:150,250

// Проверяем делегатов
console.log(inputHandler.delegates.keyboard === keyboardDelegate); // true
console.log(inputHandler.delegates.mouse === mouseDelegate); // false (после useTouch)

```

Часть 7: Отладка и диагностика цепочек прототипов

7.1. Инструменты для анализа прототипов

```

javascript

// Утилита для визуализации цепочки прототипов
function visualizePrototypeChain(obj, maxDepth = 10) {
  console.group('Цепочка прототипов:');

  let current = obj;
  let depth = 0;

  while (current && depth < maxDepth) {
    console.log(`Уровень ${depth}:`, current.constructor?.name || 'Без
конструктора');

    // Собственные свойства
    const ownProps = Object.getOwnPropertyNames(current);
    if (ownProps.length > 0) {
      console.group('Собственные свойства:');
      ownProps.forEach(prop => {
        try {
          const value = current[prop];
          const type = typeof value;
          console.log(`  ${prop}: ${type} = ${
            type === 'function' ? 'function' :
            type === 'object' ? 'object' :
            JSON.stringify(value)
          }`);
        } catch {}
      });
    }
  }
}

```

```

        } catch (e) {
            console.log(` ${prop}: <недоступно>`);
        }
    });
    console.groupEnd();
}

// Переходим к следующему прототипу
current = Object.getPrototypeOf(current);
depth++;

if (current === null) {
    console.log('Конец цепочки (null)');
}
}

if (depth >= maxDepth) {
    console.warn(`Достигнута максимальная глубина (${maxDepth})`);
}

console.groupEnd();
}

// Создаем тестовую цепочку
function A() {}
A.prototype.methodA = function() {};

function B() {}
B.prototype = Object.create(A.prototype);
B.prototype.constructor = B;
B.prototype.methodB = function() {};

function C() {}
C.prototype = Object.create(B.prototype);
C.prototype.constructor = C;
C.prototype.methodC = function() {};

const instance = new C();
instance.customProp = 'значение';

// Визуализируем
visualizePrototypeChain(instance);

// Утилита для поиска метода в цепочке прототипов

```

```

function findMethodInChain(obj, methodName) {
    let current = obj;
    const path = [];

    while (current) {
        if (current.hasOwnProperty(methodName)) {
            return {
                found: true,
                location: 'own property',
                object: current,
                path: [...path, 'current']
            };
        }

        const prototype = Object.getPrototypeOf(current);
        if (prototype && methodName in prototype) {
            return {
                found: true,
                location: 'prototype',
                object: prototype,
                path: [...path, 'prototype']
            };
        }

        path.push('↑ prototype');
        current = prototype;
    }

    return {
        found: false,
        location: null,
        object: null,
        path
    };
}

// Поиск метода
console.log(findMethodInChain(instance, 'methodB'));
// { found: true, location: 'prototype', object: B.prototype, ... }

console.log(findMethodInChain(instance, 'customProp'));
// { found: true, location: 'own property', object: instance, ... }

console.log(findMethodInChain(instance, 'nonExistent'));

```

```
// { found: false, location: null, object: null, ... }
```

7.2. Профилирование доступа к прототипам

javascript

```
// Декоратор для замера времени доступа к свойствам
```

```
function profilePrototypeAccess(target, name, descriptor) {  
    const original = descriptor.value;  
  
    descriptor.value = function(...args) {  
        const start = performance.now();  
        const result = original.apply(this, args);  
        const end = performance.now();  
  
        console.log(`Метод ${name} вызван за ${(end - start).toFixed(2)}ms`);  
  
        // Логируем цепочку прототипов  
        console.group('Цепочка прототипов при вызове:');  
        let current = this;  
        while (current) {  
            console.log(current.constructor?.name || 'Anonymous',  
                current === this ? '(this)' : '');  
            current = Object.getPrototypeOf(current);  
        }  
        console.groupEnd();  
  
        return result;  
    };  
  
    return descriptor;  
}
```

```
// Класс с профилированием
```

```
class ProfiledClass {  
    constructor(value) {  
        this.value = value;  
    }  
  
    @profilePrototypeAccess  
    getValue() {  
        // Имитация работы с цепочкой прототипов  
        let sum = 0;  
        for (let i = 0; i < 1000000; i++) {
```

```

        // Доступ к свойствам через this
        sum += this.value;
    }
    return sum;
}

// Статический метод для сравнения
static compareAccessMethods(instance) {
    console.group('Сравнение методов доступа:');

    // Прямой доступ
    console.time('Прямой доступ');
    let direct = 0;
    for (let i = 0; i < 1000000; i++) {
        direct += instance.value;
    }
    console.timeEnd('Прямой доступ');

    // Доступ через прототипную цепочку
    console.time('Цепочка прототипов');
    let proto = 0;
    for (let i = 0; i < 1000000; i++) {
        // Имитация поиска в цепочке
        if ('value' in instance) {
            proto += instance.value;
        }
    }
    console.timeEnd('Цепочка прототипов');

    console.groupEnd();
}

// Расширяем класс
class ExtendedProfiledClass extends ProfiledClass {
    constructor(value, extra) {
        super(value);
        this.extra = extra;
    }

    @profilePrototypeAccess
    getTotal() {
        return this.value + this.extra;
    }
}

```

```
}

// Тестируем
const instance = new ExtendedProfiledClass(5, 10);
instance.getValue(); // Профилируем вызов
instance.getTotal(); // Профилируем вызов
ProfiledClass.compareAccessMethods(instance);
```

Часть 8: Безопасность и антипаттерны

8.1. Проблемы безопасности прототипов

```
javascript

// Опасность: модификация Object.prototype
// АНТИПАТТЕРН - НЕ ДЕЛАЙТЕ ТАК:
Object.prototype.customMethod = function() {
    console.log('Я во всех объектах!');
};

const obj = {};
obj.customMethod(); // 'Я во всех объектах!' - НЕОЖИДАННО!

// Проблема: влияет на все объекты, включая массивы
const arr = [1, 2, 3];
arr.customMethod(); // 'Я во всех объектах!' - ЕЩЕ ХУЖЕ!

// Проблема: влияет на циклы for...in
for (let key in obj) {
    console.log(key); // Выведет и 'customMethod'!
}

// Решение: используйте Object.defineProperty с enumerable: false
Object.defineProperty(Object.prototype, 'safeMethod', {
    value: function() {
        console.log('Безопасный метод');
    },
    writable: true,
    enumerable: false, // Не появится в for...in
    configurable: true
});
```



```
const safeObj = {};  
for (let key in safeObj) {  
    console.log(key); // Ничего не выведет  
}  
  
safeObj.safeMethod(); // 'Безопасный метод'  
  
// Еще лучше: не модифицируйте встроенные прототипы вообще!  
  
// Опасность: прототипное загрязнение (Prototype Pollution)  
function merge(target, source) {  
    for (let key in source) {  
        // ОПАСНО: позволяет установить __proto__  
        target[key] = source[key];  
    }  
    return target;  
}  
  
const userInput = JSON.parse('{ "__proto__": {"isAdmin": true} }');  
const config = {};  
  
// Опасное слияние  
merge(config, userInput);  
  
// Теперь ВСЕ объекты имеют isAdmin: true!  
const newObj = {};  
console.log(newObj.isAdmin); // true - УЯЗВИМОСТЬ!  
  
// Безопасное слияние  
function safeMerge(target, source) {  
    for (let key in source) {  
        if (source.hasOwnProperty(key)) {  
            // Проверяем опасные ключи  
            if (key === '__proto__' || key === 'constructor' || key === 'prototype') {  
                continue;  
            }  
            target[key] = source[key];  
        }  
    }  
    return target;  
}  
  
// Или используйте Object.assign (он безопасен)  
const safeConfig = Object.assign({}, userInput);
```

```
console.log(safeConfig.__proto__); // {constructor: f, ...} (не перезаписан)
```

8.2. Антипаттерны и как их избежать

javascript

// АНТИПАТТЕРН 1: Изменение прототипов встроенных объектов

```
Array.prototype.customSort = function() { // НЕ ДЕЛАЙТЕ ТАК
    return this.sort((a, b) => a - b);
};
```

// Проблема: ломает библиотеки, влияет на весь код

// ПРАВИЛЬНО: создавайте утилитарные функции

```
function customSort(array) {
    return [...array].sort((a, b) => a - b);
}
```

// АНТИПАТТЕРН 2: Длинные цепочки прототипов

```
function A() {}
function B() { A.call(this); }
function C() { B.call(this); }
function D() { C.call(this); }
function E() { D.call(this); }
// ... и так далее 10 уровней
```

// Проблема: медленный поиск свойств, сложная отладка

// ПРАВИЛЬНО: используйте композицию

```
class Base {
    baseMethod() {}
}
```

```
class ComponentA {
    methodA() {}
}
```

```
class ComponentB {
    methodB() {}
}
```

```
class Combined {
    constructor() {
        this.a = new ComponentA();
    }
}
```

```

        this.b = new ComponentB();
        this.base = new Base();
    }

    methodA() { return this.a.methodA(); }
    methodB() { return this.b.methodB(); }
    baseMethod() { return this.base.baseMethod(); }
}

// АНТИПАТТЕРН 3: Динамическое изменение прототипов в рантайме
function User(name) {
    this.name = name;
}

const user1 = new User('Анна');

// ОПАСНО: меняем прототип после создания экземпляров
User.prototype.greet = function() {
    return `Привет, ${this.name}`;
};

const user2 = new User('Борис');

console.log(user1.greet()); // 'Привет, Анна' (работает)
console.log(user2.greet()); // 'Привет, Борис' (работает)

// Но: движки JavaScript могут оптимизировать доступ к прототипам
// Изменение прототипа может сбросить оптимизации

// ПРАВИЛЬНО: определяйте прототипы до создания экземпляров
function BetterUser(name) {
    this.name = name;
}

BetterUser.prototype.greet = function() {
    return `Привет, ${this.name}`;
};

const betterUser = new BetterUser('Мария');
console.log(betterUser.greet()); // 'Привет, Мария'

```

Часть 9: Резюме и лучшие практики

9.1. Ключевые принципы работы с прототипами

1. **Каждый объект имеет прототип** (кроме созданных через `Object.create(null)`)
2. **Поиск свойств идет по цепочке прототипов** до первого совпадения или `null`
3. `__proto__` — устаревший способ, используйте `Object.getPrototypeOf()` и `Object.setPrototypeOf()`
4. **Функции-конструкторы** имеют свойство `prototype`, которое становится прототипом новых объектов
5. **Классы ES6** — синтаксический сахар над прототипным наследованием

9.2. Лучшие практики

javascript

// 1. Используйте классы ES6+ для нового кода

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} издает звук`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }

  speak() {
    console.log(`${this.name} (${this.breed}) лает`);
  }
}
```

// 2. Для миксинов используйте Object.assign или декораторы

```
const CanSwim = {
  swim() {
```

```

        console.log(`${this.name} плывет`);
    }
};

class Fish extends Animal {}
Object.assign(Fish.prototype, CanSwim);

// 3. Избегайте модификации встроенных прототипов
// ПЛОХО:
// Array.prototype.myMethod = function() { ... };

// ХОРОШО:
function arrayUtility(array) {
    // Утилитарная функция
}

// ИЛИ:
class ExtendedArray extends Array {
    myMethod() {
        // Собственный метод
    }
}

// 4. Используйте Object.create() для чистого прототипного наследования
const vehicleProto = {
    move() {
        console.log('Движение');
    }
};

const car = Object.create(vehicleProto, {
    wheels: { value: 4, enumerable: true }
});

// 5. Для производительности: помещайте методы в прототип, а не в конструктор
function EfficientConstructor(data) {
    this.data = data;
}

EfficientConstructor.prototype.process = function() {
    // Один экземпляр метода для всех объектов
};

// НЕЭФФЕКТИВНО:
```

```
function InefficientConstructor(data) {
  this.data = data;
  this.process = function() {
    // Новый экземпляр метода для каждого объекта
  };
}

// 6. Используйте WeakMap для приватных данных вместо замыканий в конструкторе
const privateData = new WeakMap();

class SecureClass {
  constructor(secret) {
    privateData.set(this, { secret });
  }

  getSecret() {
    return privateData.get(this).secret;
  }
}
```

9.3. Чек-лист при работе с прототипами

При работе с прототипами проверьте:

- ☐ **Определены ли все методы в прототипе до создания экземпляров?**
- ☐ **Используете ли вы `Object.create()` вместо `__proto__`?**
- ☐ **Не модифицируете ли вы встроенные прототипы (`Array.prototype`, `Object.prototype`)?**
- ☐ **Проверяете ли вы `hasOwnProperty()` в циклах `for...in`?**
- ☐ **Используете ли вы классы ES6+ для нового кода?**
- ☐ **Не создаете ли вы слишком длинные цепочки прототипов (более 3-4 уровней)?**
- ☐ **Используете ли вы композицию вместо глубокого наследования?**
- ☐ **Проверяете ли вы безопасность при слиянии объектов (Prototype Pollution)?**

9.4. Производительность и оптимизация

1. **Поиск в цепочке прототипов** быстрее, чем поиск в замыканиях
2. **Длинные цепочки** замедляют поиск (оптимально 1-3 уровня)
3. **Изменение прототипов** после создания объектов может сбросить оптимизации движка

4. **Кэширование часто используемых методов** может улучшить производительность:

```
javascript

// Кэширование методов для производительности
class OptimizedClass {
  constructor() {
    // Кэшируем методы
    this.cachedMethod = this.method.bind(this);
  }

  method() {
    console.log('Оптимизированный метод');
  }

  // Использование кэшированного метода в обработчиках событий
  setupEvent() {
    element.addEventListener('click', this.cachedMethod);
  }
}
```

Заключение

Прототипы и цепочки прототипов — это фундаментальная концепция JavaScript, которая обеспечивает мощную систему наследования. Понимание этой системы позволяет:

1. **Создавать эффективные иерархии объектов**
2. **Оптимизировать использование памяти** (методы в прототипах)
3. **Реализовывать сложные паттерны проектирования**
4. **Понимать и отлаживать** поведение существующего кода
5. **Писать безопасный код**, избегая уязвимостей

Хотя современный JavaScript предлагает классы (синтаксический сахар над прототипами), понимание прототипной системы остается критически важным для профессиональной разработки. Используйте классы для нового кода, но знайте, что под капотом они работают через прототипы, и умейте работать с прототипами напрямую, когда это необходимо.

8.5. Конструкторы и оператор new

Введение: что такое конструкторы

Конструкторы — это специальные функции в JavaScript, которые используются для создания и инициализации объектов. При вызове с оператором `new`, они создают новый объект и устанавливают для него прототип.

Основные концепции:

- **Конструктор** — обычная функция, вызываемая с `new`
- **Оператор new** — создает новый объект и связывает его с конструктором
- **Прототип конструктора** — становится прототипом созданных объектов
- **Экземпляр** — объект, созданный конструктором

Часть 1: Основы конструкторов и оператора new

1.1. Базовый синтаксис конструктора

```
javascript
```

```
// Функция-конструктор (по соглашению начинается с заглавной буквы)
function Person(name, age) {
    // 1. Создается новый пустой объект (неявно)
    // 2. this связывается с этим объектом
    // 3. Выполняется тело конструктора

    // Устанавливаем свойства объекта
    this.name = name;
    this.age = age;

    // Добавляем методы (НЕ РЕКОМЕНДУЕТСЯ - каждый экземпляр получит свою копию)
    this.greet = function() {
        return `Привет, меня зовут ${this.name}`;
    };
};
```



```

    // 4. Возвращается this (неявно, если нет явного return)
}

// Создание экземпляра с оператором new
const person1 = new Person('Анна', 25);
const person2 = new Person('Иван', 30);

console.log(person1.name); // 'Анна'
console.log(person2.age); // 30
console.log(person1.greet()); // 'Привет, меня зовут Анна'

// Проверяем, что это разные объекты
console.log(person1 === person2); // false
console.log(person1 instanceof Person); // true
console.log(person2 instanceof Person); // true

```

1.2. Что происходит при вызове с new

```

javascript

// Давайте разберем по шагам, что делает оператор new:
function MyConstructor(value) {
    // Шаг 3: this связывается с новым объектом
    this.value = value;
    this.createdAt = new Date();

    // Шаг 4: выполняется тело функции
    console.log('Конструктор выполняется, this:', this);

    // Шаг 5: возвращается this (если нет явного return)
    // return this; // выполняется неявно
}

// Шаг 1: создается новый пустой объект
// let obj = {};

// Шаг 2: устанавливается прототип объекта
// obj.__proto__ = MyConstructor.prototype;

// Шаг 3-5: вызывается конструктор с this = obj
// const result = MyConstructor.call(obj, 'test');

// Шаг 6: если конструктор возвращает объект, он становится результатом,
// иначе возвращается obj

```

```
// const instance = result instanceof Object ? result : obj;

// Аналог оператора new (ручная реализация)
function simulateNew(constructor, ...args) {
    console.group('Имитация new:');

    // 1. Создаем новый пустой объект
    const obj = {};
    console.log('1. Создан пустой объект:', obj);

    // 2. Устанавливаем прототип
    Object.setPrototypeOf(obj, constructor.prototype);
    console.log('2. Установлен прототип:', Object.getPrototypeOf(obj));

    // 3. Вызываем конструктор с привязанным this
    const result = constructor.apply(obj, args);
    console.log('3. Вызван конструктор, результат:', result);

    // 4. Возвращаем результат или объект
    const finalResult = result instanceof Object ? result : obj;
    console.log('4. Финальный результат:', finalResult);

    console.groupEnd();
    return finalResult;
}

// Тестируем нашу имитацию
const manualInstance = simulateNew(MyConstructor, 'ручное создание');
console.log('Ручной экземпляр:', manualInstance);
console.log('manualInstance instanceof MyConstructor:', manualInstance instanceof MyConstructor);
```

1.3. Проверка вызова с new

javascript

```
// Защита от вызова без new (старый стиль)
function SafeConstructorOld(name) {
    // Проверяем, вызван ли конструктор с new
    if (!(this instanceof SafeConstructorOld)) {
        // Если нет - вызываем себя с new
        return new SafeConstructorOld(name);
    }
}
```

```

    this.name = name;
    this.createdWithNew = true;
}

// ES6+ способ с new.target
function SafeConstructorNew(name) {
    // new.target содержит конструктор, если вызвано с new
    // и undefined, если вызвано как обычная функция
    if (!new.target) {
        throw new Error('Конструктор должен вызываться с new');
        // Или автоматически исправляем:
        // return new SafeConstructorNew(name);
    }

    this.name = name;
    this.createdWithNew = true;
    console.log('new.target:', new.target);
}

// Тестируем
const safe1 = new SafeConstructorOld('с new');
const safe2 = SafeConstructorOld('без new'); // Автоматически исправляется

console.log(safe1.name); // 'с new'
console.log(safe2.name); // 'без new'
console.log(safe1 instanceof SafeConstructorOld); // true
console.log(safe2 instanceof SafeConstructorOld); // true

const safe3 = new SafeConstructorNew('работает');
// const safe4 = SafeConstructorNew('ошибка'); // Error: Конструктор должен вызываться с
new

// new.target в классах
class SafeClass {
    constructor(name) {
        // В классах new.target всегда определен при вызове с new
        console.log('new.target в классе:', new.target);

        if (new.target === SafeClass) {
            console.log('Создается экземпляр SafeClass');
        }

        this.name = name;
    }
}

```

```
}
```

```
const classInstance = new SafeClass('тест');
```

Часть 2: Прототипы конструкторов

2.1. Свойство `prototype`

javascript

```
// Каждая функция имеет свойство prototype
```

```
function Animal(type) {  
    this.type = type;  
}
```

```
// Свойство prototype существует по умолчанию
```

```
console.log(Animal.prototype); // {constructor: f}  
console.log(Animal.prototype.constructor === Animal); // true
```

```
// Добавляем методы в прототип
```

```
Animal.prototype.describe = function() {  
    return `Это животное типа: ${this.type}`;  
};
```

```
Animal.prototype.move = function() {  
    return `${this.type} движется`;  
};
```

```
// Создаем экземпляр
```

```
const dog = new Animal('собака');
```

```
// Методы доступны через цепочку прототипов
```

```
console.log(dog.describe()); // 'Это животное типа: собака'  
console.log(dog.move()); // 'собака движется'
```

```
// Проверяем прототип
```

```
console.log(Object.getPrototypeOf(dog) === Animal.prototype); // true  
console.log(dog.__proto__ === Animal.prototype); // true (устаревший способ)
```

```
// Методы принадлежат прототипу, не экземпляру
```

```
console.log(dog.hasOwnProperty('describe')); // false  
console.log(dog.hasOwnProperty('type')); // true
```

```
console.log(Animal.prototype.hasOwnProperty('describe')); // true

// Схема:
// dog (экземпляр)
//   ↓ __proto__
// Animal.prototype
//   ↓ __proto__
// Object.prototype
//   ↓ __proto__
// null
```

2.2. Наследование через конструкторы

```
javascript

// Базовый конструктор
function Vehicle(type, wheels) {
    this.type = type;
    this.wheels = wheels;
    this.speed = 0;
}

// Методы базового конструктора
Vehicle.prototype.accelerate = function(amount) {
    this.speed += amount;
    console.log(`${this.type} ускорился до ${this.speed} км/ч`);
};

Vehicle.prototype.brake = function(amount) {
    this.speed = Math.max(0, this.speed - amount);
    console.log(`${this.type} замедлился до ${this.speed} км/ч`);
};

Vehicle.prototype.getInfo = function() {
    return `${this.type} (${this.wheels} колеса), скорость: ${this.speed} км/ч`;
};

// Дочерний конструктор
function Car(make, model, year) {
    // Вызываем родительский конструктор с правильным this
    Vehicle.call(this, 'автомобиль', 4); // Наследуем свойства

    // Добавляем собственные свойства
    this.make = make;
```

```

    this.model = model;
    this.year = year;
}

// Наследование прототипа (важная часть!)
// Способ 1: Object.create (рекомендуется)
Car.prototype = Object.create(Vehicle.prototype);

// Способ 2: устаревший (может вызвать проблемы)
// Car.prototype = new Vehicle(); // Создает лишний объект

// Восстанавливаем конструктор
Car.prototype.constructor = Car;

// Добавляем методы дочернего конструктора
Car.prototype.honk = function() {
    console.log(`${this.make} ${this.model} сигналист: Бип-бип!`);
};

Car.prototype.getFullInfo = function() {
    return `${this.make} ${this.model} (${this.year}) - ${this.getInfo()}`;
};

// Создаем экземпляр
const myCar = new Car('Toyota', 'Camry', 2022);

// Проверяем наследование
console.log(myCar.type); // 'автомобиль' (из Vehicle)
console.log(myCar.wheels); // 4 (из Vehicle)
console.log(myCar.make); // 'Toyota' (из Car)

myCar.accelerate(50); // 'автомобиль ускорился до 50 км/ч' (из Vehicle)
myCar.honk(); // 'Toyota Camry сигналист: Бип-бип!' (из Car)
console.log(myCar.getFullInfo()); // 'Toyota Camry (2022) - автомобиль (4 колеса)...'

// Проверяем цепочку прототипов
console.log(myCar instanceof Car); // true
console.log(myCar instanceof Vehicle); // true
console.log(myCar instanceof Object); // true

console.log(Object.getPrototypeOf(myCar) === Car.prototype); // true
console.log(Object.getPrototypeOf(Car.prototype) === Vehicle.prototype); // true

```

2.3. Статические методы и свойства

```
javascript

function User(name, email) {
    // Свойства экземпляра
    this.name = name;
    this.email = email;
    this.id = User.generateId(); // Используем статический метод
}

// Статические свойства (принадлежат конструктору, не экземплярам)
User.userCount = 0;
User.MAX_USERS = 1000;

// Статические методы
User.generateId = function() {
    User.userCount++;
    return `user_${Date.now()}_${User.userCount}`;
};

User.validateEmail = function(email) {
    const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return regex.test(email);
};

User.compareUsers = function(user1, user2) {
    return {
        sameName: user1.name === user2.name,
        sameEmail: user1.email === user2.email
    };
};

// Методы экземпляра (в прототипе)
User.prototype.getInfo = function() {
    return `${this.name} (${this.email})`;
};

User.prototype.updateEmail = function(newEmail) {
    if (User.validateEmail(newEmail)) {
        this.email = newEmail;
        return true;
    }
    return false;
};
```

```

};

// Использование
console.log('Статические методы:');
console.log(User.validateEmail('test@example.com')); // true
console.log(User.validateEmail('invalid-email')); // false

// Создание экземпляров
const user1 = new User('Анна', 'anna@example.com');
const user2 = new User('Иван', 'ivan@example.com');

console.log('\nЭкземпляры:');
console.log(user1.getInfo()); // 'Анна (anna@example.com)'
console.log(user2.getInfo()); // 'Иван (ivan@example.com)'

console.log('\nСтатические свойства:');
console.log('Всего пользователей:', User.userCount); // 2
console.log('Максимум пользователей:', User.MAX_USERS); // 1000

console.log('\nСравнение пользователей:');
console.log(User.compareUsers(user1, user2));
// { sameName: false, sameEmail: false }

// Статические методы недоступны у экземпляров
console.log(user1.validateEmail); // undefined
console.log(user1.userCount); // undefined

// Проверка
console.log('\nПроверка:');
console.log(user1 instanceof User); // true
console.log(user1.constructor === User); // true
console.log(Object.getPrototypeOf(user1) === User.prototype); // true

```

Часть 3: Особенности работы с `new`

3.1. Возврат значения из конструктора

```

javascript

// По умолчанию конструктор возвращает this
function DefaultReturn(name) {
    this.name = name;
}

```



```

    // return this; // выполняется неявно
}

const obj1 = new DefaultReturn('Тест');
console.log(obj1); // DefaultReturn { name: 'Тест' }

// Если возвращается примитив - он игнорируется
function ReturnPrimitive(name) {
    this.name = name;
    return 42; // Примитив игнорируется
}

const obj2 = new ReturnPrimitive('Игнорируемый примитив');
console.log(obj2); // ReturnPrimitive { name: 'Игнорируемый примитив' }
console.log(obj2 === 42); // false

// Если возвращается объект - он становится результатом
function ReturnObject(name) {
    this.name = name;

    // Возвращаем другой объект
    return {
        custom: 'объект',
        fromConstructor: name
    };
}

const obj3 = new ReturnObject('Замененный');
console.log(obj3); // { custom: 'объект', fromConstructor: 'Замененный' }
console.log(obj3 instanceof ReturnObject); // false!
console.log(obj3.constructor === Object); // true

// Возврат массива
function ReturnArray(...items) {
    this.items = items;
    return items; // Массив - это объект
}

const obj4 = new ReturnArray(1, 2, 3);
console.log(obj4); // [1, 2, 3]
console.log(Array.isArray(obj4)); // true
console.log(obj4 instanceof ReturnArray); // false

// Практическое использование возврата объекта

```

```

function Singleton(name) {
    // Если экземпляр уже существует - возвращаем его
    if (Singleton.instance) {
        return Singleton.instance;
    }

    this.name = name;
    this.createdAt = new Date();

    Singleton.instance = this;
    // return this; // неявно
}

const singleton1 = new Singleton('Первый');
const singleton2 = new Singleton('Второй'); // Должен игнорировать 'Второй'

console.log(singleton1 === singleton2); // true
console.log(singleton1.name); // 'Первый'
console.log(singleton2.name); // 'Первый' (не 'Второй')

// Возврат с проверкой new.target
function FlexibleConstructor(name) {
    // Если вызван без new - возвращаем новый объект
    if (!new.target) {
        return { name, createdWithoutNew: true };
    }

    this.name = name;
    this.createdWithNew = true;
}

const flex1 = FlexibleConstructor('Без new'); // Вызываем как функцию
const flex2 = new FlexibleConstructor('С new'); // Вызываем с new

console.log(flex1); // { name: 'Без new', createdWithoutNew: true }
console.log(flex2); // FlexibleConstructor { name: 'С new', createdWithNew: true }

```

3.2. Приватные поля и методы (до ES2022)

```

javascript

// Замыкания для приватных полей (до ES2022)
function PersonWithClosure(name, age) {
    // Публичные поля

```

```

    this.name = name;

    // Приватные поля через замыкание
    let privateAge = age;
    let privateHistory = [];

    // Приватный метод (доступен только внутри конструктора)
    function addToHistory(action) {
        privateHistory.push({
            action,
            timestamp: new Date(),
            age: privateAge
        });
    }

    // Публичные методы, имеющие доступ к приватным полям
    this.getAge = function() {
        return privateAge;
    };

    this.setAge = function(newAge) {
        const oldAge = privateAge;
        privateAge = newAge;
        addToHistory(`Возраст изменен с ${oldAge} на ${newAge}`);
    };

    this.getHistory = function() {
        return [...privateHistory]; // Возвращаем копию
    };

    this.haveBirthday = function() {
        this.setAge(privateAge + 1);
        addToHistory('День рождения!');
    };

    // Инициализация
    addToHistory('Создан');
}

const person = new PersonWithClosure('Анна', 25);

console.log(person.name); // 'Анна' (публичное)
console.log(person.getAge()); // 25 (через метод)
// console.log(person.privateAge); // undefined (не доступно)

```

```
// console.log(person.addToHistory); // undefined (не доступно)

person.haveBirthday();
console.log(person.getAge()); // 26

const history = person.getHistory();
console.log(history.length); // 3 (Создан, Возраст изменен, День рождения)

// Каждый экземпляр имеет свои копии методов
const person2 = new PersonWithClosure('Иван', 30);
console.log(person.getAge === person2.getAge); // false (разные функции)

// Проблема: методы не в прототипе, каждый экземпляр получает свою копию
console.log(person.hasOwnProperty('getAge')); // true (должно быть в прототипе)
```

3.3. Приватные поля ES2022+

javascript

```
// Современный синтаксис приватных полей (ES2022+)
```

```
class ModernPerson {
  // Приватные поля (начинаются с #)
  #age;
  #history = [];

  constructor(name, age) {
    // Публичные поля
    this.name = name;

    // Приватные поля
    this.#age = age;
    this.#addToHistory('Создан');
  }

  // Приватный метод
  #addToHistory(action) {
    this.#history.push({
      action,
      timestamp: new Date(),
      age: this.#age
    });
  }
}
```

```
// Публичные методы
```

```

    getAge() {
        return this.#age;
    }

    setAge(newAge) {
        const oldAge = this.#age;
        this.#age = newAge;
        this.#addToHistory(`Возраст изменен с ${oldAge} на ${newAge}`);
    }

    getHistory() {
        return [...this.#history]; // Копия
    }

    haveBirthday() {
        this.setAge(this.#age + 1);
        this.#addToHistory('День рождения!');
        console.log(`С днем рождения, ${this.name}! Теперь тебе ${this.#age}`);
    }
}

const modernPerson = new ModernPerson('Мария', 25);

console.log(modernPerson.name); // 'Мария'
console.log(modernPerson.getAge()); // 25
// console.log(modernPerson.#age); // SyntaxError
// console.log(modernPerson.#addToHistory); // SyntaxError

modernPerson.haveBirthday(); // 'С днем рождения, Мария! Теперь тебе 26'
console.log(modernPerson.getAge()); // 26

// Методы в прототипе (оптимально)
console.log(modernPerson.hasOwnProperty('getAge')); // false
console.log(modernPerson.__proto__.hasOwnProperty('getAge')); // true

const modernPerson2 = new ModernPerson('Алексей', 30);
console.log(modernPerson.getAge === modernPerson2.getAge); // true (одна функция в прототипе)

```

Часть 4: Продвинутые техники

4.1. Фабричные методы в конструкторах

javascript

// Конструктор с фабричными методами

```
function Product(sku, name, price) {  
    this.sku = sku;  
    this.name = name;  
    this.price = price;  
    this.inStock = true;  
}
```

// Фабричные методы (статические)

```
Product.createFromJSON = function(jsonString) {  
    try {  
        const data = JSON.parse(jsonString);  
        return new Product(data.sku, data.name, data.price);  
    } catch (error) {  
        console.error('Ошибка создания продукта из JSON:', error);  
        return null;  
    }  
};
```

```
Product.createBatch = function(productsData) {  
    return productsData.map(data =>  
        new Product(data.sku, data.name, data.price)  
    );  
};
```

```
Product.createRandom = function() {  
    const skus = ['P001', 'P002', 'P003', 'P004'];  
    const names = ['Книга', 'Ноутбук', 'Телефон', 'Наушники'];  
    const prices = [500, 50000, 30000, 5000];  
  
    const randomIndex = Math.floor(Math.random() * skus.length);  
  
    return new Product(  
        skus[randomIndex],  
        names[randomIndex],  
        prices[randomIndex]  
    );  
};
```

```

};

// Методы экземпляра
Product.prototype.applyDiscount = function(percent) {
    const discount = this.price * (percent / 100);
    this.price = Math.round((this.price - discount) * 100) / 100;
    return this;
};

Product.prototype.getInfo = function() {
    return `${this.name} (${this.sku}) - ${this.price} руб.`;
};

// Использование фабричных методов
console.log('Фабричные методы:');

// Из JSON
const jsonProduct = Product.createFromJSON(
    '{"sku": "P001", "name": "Книга", "price": 500}'
);
console.log(jsonProduct?.getInfo()); // 'Книга (P001) - 500 руб.'

// Пакетное создание
const batchData = [
    { sku: 'P002', name: 'Ноутбук', price: 50000 },
    { sku: 'P003', name: 'Телефон', price: 30000 }
];

const batch = Product.createBatch(batchData);
batch.forEach(product => console.log(product.getInfo()));

// Случайный продукт
const randomProduct = Product.createRandom();
console.log('Случайный продукт:', randomProduct.getInfo());

// Цепочка вызовов
const discountedProduct = new Product('P004', 'Наушники', 5000)
    .applyDiscount(20)
    .applyDiscount(10); // Последовательные скидки

console.log('После скидок:', discountedProduct.getInfo()); // 'Наушники (P004) - 3600 руб.'

```

4.2. Кэширование экземпляров (паттерн "Пулов объектов")

javascript

// Пул объектов (Object Pool) для оптимизации создания/удаления

```
function ConnectionPool(maxSize = 10) {  
    if (ConnectionPool.instance) {  
        return ConnectionPool.instance;  
    }  
  
    this.maxSize = maxSize;  
    this.pool = []; // Активные соединения  
    this.available = []; // Свободные соединения  
  
    ConnectionPool.instance = this;  
}
```

// Статические методы управления пулом

```
ConnectionPool.getInstance = function(maxSize) {  
    if (!ConnectionPool.instance) {  
        ConnectionPool.instance = new ConnectionPool(maxSize);  
    }  
    return ConnectionPool.instance;  
};
```

// Методы экземпляра

```
ConnectionPool.prototype.createConnection = function(id) {  
    console.log(`Создание соединения ${id}`);  
    return {  
        id,  
        connected: false,  
        lastUsed: null,  
        data: null,  
  
        connect() {  
            this.connected = true;  
            this.lastUsed = new Date();  
            console.log(`Соединение ${this.id} установлено`);  
        },  
  
        disconnect() {  
            this.connected = false;  
            console.log(`Соединение ${this.id} разорвано`);  
        },  
    };  
}
```



```

    send(data) {
        if (!this.connected) {
            throw new Error('Соединение не установлено');
        }
        this.data = data;
        this.lastUsed = new Date();
        console.log(`Данные отправлены через соединение ${this.id}:`, data);
        return `ack_${this.id}`;
    }
};

};

ConnectionPool.prototype.acquire = function() {
    // Если есть свободное соединение - используем его
    if (this.available.length > 0) {
        const connection = this.available.pop();
        console.log(`Используется существующее соединение ${connection.id}`);
        connection.connect();
        this.pool.push(connection);
        return connection;
    }

    // Если можно создать новое
    if (this.pool.length < this.maxSize) {
        const id = `conn_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
        const connection = this.createConnection(id);
        connection.connect();
        this.pool.push(connection);
        return connection;
    }

    throw new Error('Достигнут лимит соединений');
};

ConnectionPool.prototype.release = function(connection) {
    const index = this.pool.indexOf(connection);
    if (index !== -1) {
        connection.disconnect();
        this.pool.splice(index, 1);
        this.available.push(connection);
        console.log(`Соединение ${connection.id} освобождено`);
    }
};

```

```
ConnectionPool.prototype.getStats = function() {
    return {
        total: this.pool.length + this.available.length,
        active: this.pool.length,
        available: this.available.length,
        maxSize: this.maxSize
    };
};

// Использование пула
const pool = ConnectionPool.getInstance(3);

console.log('Получение соединений:');
const conn1 = pool.acquire();
const conn2 = pool.acquire();
const conn3 = pool.acquire();

console.log('Статистика:', pool.getStats());
// { total: 3, active: 3, available: 0, maxSize: 3 }

// Пытаемся получить четвертое соединение
try {
    const conn4 = pool.acquire(); // Ошибка: Достигнут лимит соединений
} catch (error) {
    console.error('Ошибка:', error.message);
}

// Освобождаем соединение
pool.release(conn2);
console.log('После освобождения:', pool.getStats());
// { total: 3, active: 2, available: 1, maxSize: 3 }

// Теперь можно получить новое соединение
const conn4 = pool.acquire(); // Использует освобожденное conn2
console.log('Использование повторно:', pool.getStats());

// Singleton проверка
const pool2 = ConnectionPool.getInstance();
console.log('Один и тот же пул?', pool === pool2); // true
```

4.3. Декораторы для конструкторов

javascript

// Декоратор для логирования создания экземпляров

```
function logInstances(constructor) {  
    let instanceCount = 0;  
  
    // Заменяем оригинальный конструктор  
    const NewConstructor = function(...args) {  
        instanceCount++;  
        console.group(`Создание экземпляра #${instanceCount} ${constructor.name}:`);  
        console.log('Аргументы:', args);  
        console.log('Время:', new Date().toISOString());  
  
        // Создаем экземпляр  
        const instance = new constructor(...args);  
  
        console.log('Результат:', instance);  
        console.groupEnd();  
  
        // Добавляем свойство с номером экземпляра  
        instance._instanceNumber = instanceCount;  
  
        return instance;  
    };  
  
    // Копируем прототип  
    NewConstructor.prototype = constructor.prototype;  
  
    // Копируем статические свойства  
    Object.assign(NewConstructor, constructor);  
  
    // Добавляем статический метод для получения счетчика  
    NewConstructor.getInstanceCount = function() {  
        return instanceCount;  
    };  
  
    return NewConstructor;  
}
```

// Декоратор для валидации параметров

```
function validateParams(validationRules) {  
    return function(constructor) {
```

```

const OriginalConstructor = constructor;

const NewConstructor = function(...args) {
  // Валидация параметров
  validationRules.forEach((rule, index) => {
    if (index < args.length) {
      const param = args[index];
      const paramName = rule.name || `param${index}`;

      if (rule.type && typeof param !== rule.type) {
        throw new Error(
          `${constructor.name}: Параметр ${paramName} должен быть
типа ${rule.type}, получен ${typeof param}`
        );
      }

      if (rule.required && (param === undefined || param === null)) {
        throw new Error(
          `${constructor.name}: Параметр ${paramName} обязателен`
        );
      }

      if (rule.validator && !rule.validator(param)) {
        throw new Error(
          `${constructor.name}: Параметр ${paramName} не прошел
валидацию`
        );
      }
    }
  });

  return new OriginalConstructor(...args);
};

// Копируем прототип и статические свойства
NewConstructor.prototype = OriginalConstructor.prototype;
Object.assign(NewConstructor, OriginalConstructor);

return NewConstructor;
};
}

// Применяем декораторы
@logInstances

```

```

@validateParams([
  { name: 'name', type: 'string', required: true },
  { name: 'age', type: 'number', required: true, validator: age => age >= 0 && age <=
150 },
  { name: 'email', type: 'string', validator: email => email.includes('@') }
])
class User {
  constructor(name, age, email = '') {
    this.name = name;
    this.age = age;
    this.email = email;
    this.id = `user_${Date.now()}`;
  }

  getInfo() {
    return `${this.name}, ${this.age} лет${this.email ? `, ${this.email}` : ''}`;
  }
}

// Тестируем
try {
  console.log('Тестирование декораторов:');

  const user1 = new User('Анна', 25, 'anna@example.com');
  console.log('user1:', user1.getInfo());

  const user2 = new User('Иван', 30);
  console.log('user2:', user2.getInfo());

  console.log('Всего создано:', User.getInstanceCount()); // 2

  // Ошибка валидации
  // const user3 = new User(null, 25); // Error: Параметр name обязателен
  // const user4 = new User('Борис', -5); // Error: Параметр age не прошел валидацию
  // const user5 = new User('Мария', '25'); // Error: Параметр age должен быть типа
number

} catch (error) {
  console.error('Ошибка создания:', error.message);
}

```

Часть 5: Сравнение с классами ES6+

5.1. Классы как синтаксический сахар

javascript

// ES5 конструктор

```
function OldStylePerson(name, age) {  
    this.name = name;  
    this.age = age;  
}
```

```
OldStylePerson.prototype.greet = function() {  
    return `Привет, я ${this.name}, мне ${this.age} лет`;  
};
```

```
OldStylePerson.describe = function() {  
    return 'Это конструктор Person';  
};
```

// ES6+ класс (синтаксический сахар над конструктором)

```
class NewStylePerson {  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

// Методы автоматически добавляются в прототип

```
greet() {  
    return `Привет, я ${this.name}, мне ${this.age} лет`;  
}
```

// Статический метод

```
static describe() {  
    return 'Это класс Person';  
}
```

```
}
```

// Создание экземпляров

```
const oldPerson = new OldStylePerson('Иван', 30);  
const newPerson = new NewStylePerson('Мария', 25);
```

```
console.log('Сравнение:');
```

```

console.log(oldPerson.greet()); // 'Привет, я Иван, мне 30 лет'
console.log(newPerson.greet()); // 'Привет, я Мария, мне 25 лет'

console.log('Типы:');
console.log(typeof OldStylePerson); // 'function'
console.log(typeof NewStylePerson); // 'function' (класс - это функция)

console.log('Прототипы:');
console.log(oldPerson.__proto__ === OldStylePerson.prototype); // true
console.log(newPerson.__proto__ === NewStylePerson.prototype); // true

console.log('Методы в прототипе:');
console.log(oldPerson.hasOwnProperty('greet')); // false
console.log(newPerson.hasOwnProperty('greet')); // false
console.log(OldStylePerson.prototype.hasOwnProperty('greet')); // true
console.log(NewStylePerson.prototype.hasOwnProperty('greet')); // true

console.log('Статические методы:');
console.log(OldStylePerson.describe()); // 'Это конструктор Person'
console.log(NewStylePerson.describe()); // 'Это класс Person'

// Классы нельзя вызывать без new
// OldStylePerson('Ошибка?', 20); // Установит глобальные переменные (в нестрогом режиме)
// NewStylePerson('Ошибка!', 20); // TypeError: Class constructor cannot be invoked
// without 'new'

// Преобразование класса в конструктор
console.log('\nКод класса после преобразования:');
console.log('class NewStylePerson { ... }');
console.log('↓');
console.log('function NewStylePerson() {}');
console.log('  if (!new.target) throw new Error(...)');
console.log('  // constructor code');
console.log('}');
console.log('NewStylePerson.prototype.greet = function() { ... }');
console.log('NewStylePerson.describe = function() { ... }');

```

5.2. Наследование: конструкторы vs классы

```

javascript

// Наследование с конструкторами (ES5)
function AnimalES5(name) {
  this.name = name;
}

```

```

    this.eats = true;
}

AnimalES5.prototype.walk = function() {
    console.log(`${this.name} идет`);
};

function RabbitES5(name, color) {
    AnimalES5.call(this, name); // Наследование свойств
    this.color = color;
    this.jumps = true;
}

// Наследование прототипа
RabbitES5.prototype = Object.create(AnimalES5.prototype);
RabbitES5.prototype.constructor = RabbitES5;

// Методы дочернего конструктора
RabbitES5.prototype.hop = function() {
    console.log(`${this.name} (${this.color}) прыгает`);
};

// Наследование с классами (ES6+)
class AnimalES6 {
    constructor(name) {
        this.name = name;
        this.eats = true;
    }

    walk() {
        console.log(`${this.name} идет`);
    }
}

class RabbitES6 extends AnimalES6 {
    constructor(name, color) {
        super(name); // Вызов родительского конструктора
        this.color = color;
        this.jumps = true;
    }

    hop() {
        console.log(`${this.name} (${this.color}) прыгает`);
    }
}

```



```
}
```

```
// Сравнение
```

```
const rabbitES5 = new RabbitES5('Белый кролик', 'белый');
```

```
const rabbitES6 = new RabbitES6('Серый кролик', 'серый');
```

```
console.log('Сравнение наследования:');
```

```
rabbitES5.walk(); // 'Белый кролик идет'
```

```
rabbitES5.hop(); // 'Белый кролик (белый) прыгает'
```

```
rabbitES6.walk(); // 'Серый кролик идет'
```

```
rabbitES6.hop(); // 'Серый кролик (серый) прыгает'
```

```
console.log('\nПроверка цепочек прототипов:');
```

```
console.log('ES5:', rabbitES5 instanceof RabbitES5); // true
```

```
console.log('ES5:', rabbitES5 instanceof AnimalES5); // true
```

```
console.log('ES6:', rabbitES6 instanceof RabbitES6); // true
```

```
console.log('ES6:', rabbitES6 instanceof AnimalES6); // true
```

```
console.log('\nПрототипы:');
```

```
console.log('ES5 прототип Rabbit:', Object.getPrototypeOf(rabbitES5) ===
```

```
RabbitES5.prototype); // true
```

```
console.log('ES5 прототип Animal:', Object.getPrototypeOf(RabbitES5.prototype) ===
```

```
AnimalES5.prototype); // true
```

```
console.log('ES6 прототип Rabbit:', Object.getPrototypeOf(rabbitES6) ===
```

```
RabbitES6.prototype); // true
```

```
console.log('ES6 прототип Animal:', Object.getPrototypeOf(RabbitES6.prototype) ===
```

```
AnimalES6.prototype); // true
```

```
// Статическое наследование
```

```
AnimalES5.staticMethod = function() {  
    return 'Статический метод AnimalES5';  
};
```

```
AnimalES6.staticMethod = function() {  
    return 'Статический метод AnimalES6';  
};
```

```
// Конструкторы не наследуют статические методы автоматически
```

```
console.log('\nСтатическое наследование:');
```

```
console.log('ES5:', RabbitES5.staticMethod); // undefined (не наследуется)
```

```
console.log('ES6:', RabbitES6.staticMethod()); // 'Статический метод AnimalES6'  
(наследуется)
```

5.3. Геттеры, сеттеры и статические блоки

javascript

// В конструкторах (ES5)

```
function PersonES5(name, birthYear) {  
    this.name = name;  
    this._birthYear = birthYear;  
}
```

// Геттеры/сеттеры через Object.defineProperty

```
Object.defineProperty(PersonES5.prototype, 'age', {  
    get: function() {  
        const currentYear = new Date().getFullYear();  
        return currentYear - this._birthYear;  
    },  
    set: function(newAge) {  
        const currentYear = new Date().getFullYear();  
        this._birthYear = currentYear - newAge;  
    },  
    enumerable: true  
});  
  
Object.defineProperty(PersonES5.prototype, 'birthYear', {  
    get: function() {  
        return this._birthYear;  
    },  
    set: function(year) {  
        if (year > new Date().getFullYear()) {  
            throw new Error('Год рождения не может быть в будущем');  
        }  
        this._birthYear = year;  
    },  
    enumerable: true  
});
```

// Статическое свойство

```
PersonES5.species = 'Homo sapiens';
```

// В классах (ES6+)

```
class PersonES6 {  
    // Статическое свойство  
    static species = 'Homo sapiens';
```

```

// Статический блок инициализации (ES2022)
static {
    console.log('Класс PersonES6 загружен');
    this.version = '1.0.0';
}

constructor(name, birthYear) {
    this.name = name;
    this._birthYear = birthYear;
}

// Геттер
get age() {
    const currentYear = new Date().getFullYear();
    return currentYear - this._birthYear;
}

// Сеттер
set age(newAge) {
    const currentYear = new Date().getFullYear();
    this._birthYear = currentYear - newAge;
}

// Еще один геттер
get birthYear() {
    return this._birthYear;
}

// Еще один сеттер
set birthYear(year) {
    if (year > new Date().getFullYear()) {
        throw new Error('Год рождения не может быть в будущем');
    }
    this._birthYear = year;
}

// Статический геттер
static get description() {
    return `Вид: ${this.species}, версия: ${this.version}`;
}
}

// Сравнение
const personES5 = new PersonES5('Анна', 1990);

```

```

const personES6 = new PersonES6('Иван', 1985);

console.log('Сравнение геттеров/сеттеров:');
console.log('ES5 возраст:', personES5.age); // 34 (если текущий год 2024)
console.log('ES6 возраст:', personES6.age); // 39

personES5.age = 30;
personES6.age = 35;

console.log('После изменения возраста:');
console.log('ES5 год рождения:', personES5.birthYear); // 1994
console.log('ES6 год рождения:', personES6.birthYear); // 1989

console.log('\nСтатические свойства:');
console.log('ES5:', PersonES5.species); // 'Homo sapiens'
console.log('ES6:', PersonES6.species); // 'Homo sapiens'
console.log('ES6 версия:', PersonES6.version); // '1.0.0'
console.log('ES6 описание:', PersonES6.description); // 'Вид: Homo sapiens, версия: 1.0.0'

// Проверка дескрипторов
console.log('\nДескрипторы свойств:');
const es5Descriptor = Object.getOwnPropertyDescriptor(PersonES5.prototype, 'age');
const es6Descriptor = Object.getOwnPropertyDescriptor(PersonES6.prototype, 'age');

console.log('ES5 дескриптор:', {
  get: typeof es5Descriptor.get,
  set: typeof es5Descriptor.set,
  enumerable: es5Descriptor.enumerable
});

console.log('ES6 дескриптор:', {
  get: typeof es6Descriptor.get,
  set: typeof es6Descriptor.set,
  enumerable: es6Descriptor.enumerable
});

```

Часть 6: Антипаттерны и лучшие практики

6.1. Распространенные ошибки

javascript

```

// АНТИПАТТЕРН 1: Забывать new
function ProblematicConstructor(name) {
    this.name = name;
}

// ОШИБКА: без new this будет глобальным объектом (в нестрогом режиме)
// const obj = ProblematicConstructor('Ошибка');
// console.log(name); // 'Ошибка' (глобальная переменная!)
// console.log(obj); // undefined

// РЕШЕНИЕ: используйте new.target или проверку instanceof
function SafeConstructor(name) {
    if (!new.target) {
        throw new Error('Используйте new!');
    }
    this.name = name;
}

// АНТИПАТТЕРН 2: Методы в конструкторе
function InefficientUser(name) {
    this.name = name;

    // ПЛОХО: каждый экземпляр получает свою копию функции
    this.greet = function() {
        return `Привет, ${this.name}`;
    };
}

// РЕШЕНИЕ: методы в прототипе
function EfficientUser(name) {
    this.name = name;
}

EfficientUser.prototype.greet = function() {
    return `Привет, ${this.name}`;
};

// Сравнение
const users1 = [];
const users2 = [];

console.time('Неэффективные');
for (let i = 0; i < 10000; i++) {
    users1.push(new InefficientUser(`User${i}`));
}

```

```

}
console.timeEnd('Неэффективные');

console.time('Эффективные');
for (let i = 0; i < 10000; i++) {
    users2.push(new EfficientUser(`User${i}`));
}
console.timeEnd('Эффективные');

console.log('Одинаковые функции?', users1[0].greet === users1[1].greet); // false
console.log('Одинаковые функции?', users2[0].greet === users2[1].greet); // true

// АНТИПАТТЕРН 3: Изменение прототипа после создания экземпляров
function EarlyConstructor() {
    this.value = 1;
}

const earlyInstance = new EarlyConstructor();

// ОПАСНО: движки JavaScript могут оптимизировать доступ к прототипам
EarlyConstructor.prototype.newMethod = function() {
    return this.value * 2;
};

console.log(earlyInstance.newMethod()); // 2 (работает, но может быть медленнее)

// РЕШЕНИЕ: определяйте прототип до создания экземпляров
function LateConstructor() {
    this.value = 1;
}

LateConstructor.prototype.newMethod = function() {
    return this.value * 2;
};

const lateInstance = new LateConstructor();
console.log(lateInstance.newMethod()); // 2 (оптимизировано)

```

6.2. Лучшие практики

javascript

```

// 1. Всегда используйте заглавную букву для конструкторов
function GoodConstructor() {

```

```
// ...
}

// 2. Проверяйте new с new.target (ES6+)
class SafeClass {
  constructor() {
    if (!new.target) {
      throw new Error('Use new!');
    }
  }
}

// 3. Определяйте методы в прототипе
function User(name) {
  this.name = name;
}

User.prototype.sayHello = function() {
  return `Hello, ${this.name}`;
};

// 4. Для наследования используйте классы ES6+
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} издает звук`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }

  speak() {
    console.log(`${this.name} (${this.breed}) лает`);
  }
}

// 5. Используйте приватные поля ES2022+ для инкапсуляции
```

```

class BankAccount {
    #balance = 0;

    constructor(initialBalance) {
        this.#balance = initialBalance;
    }

    deposit(amount) {
        if (amount > 0) {
            this.#balance += amount;
        }
    }

    get balance() {
        return this.#balance;
    }
}

```

// 6. Фабричные методы вместо сложных конструкторов

```

class Product {
    constructor(sku, name, price) {
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    static createFromData(data) {
        return new Product(data.sku, data.name, data.price);
    }

    static createRandom() {
        const skus = ['P001', 'P002', 'P003'];
        const names = ['Product A', 'Product B', 'Product C'];
        const prices = [100, 200, 300];

        const index = Math.floor(Math.random() * skus.length);
        return new Product(skus[index], names[index], prices[index]);
    }
}

```

// 7. Иммутабельность: возвращайте новые экземпляры вместо изменения существующих

```

class ImmutablePoint {
    constructor(x, y) {
        this.x = x;
    }
}

```



```

    this.y = y;
    Object.freeze(this); // Запрещаем изменения
  }

  add(other) {
    return new ImmutablePoint(this.x + other.x, this.y + other.y);
  }

  scale(factor) {
    return new ImmutablePoint(this.x * factor, this.y * factor);
  }
}

const p1 = new ImmutablePoint(1, 2);
const p2 = p1.add(new ImmutablePoint(3, 4));
// p1.x = 10; // TypeError в строгом режиме
console.log(p1); // { x: 1, y: 2 }
console.log(p2); // { x: 4, y: 6 }

```

6.3. Чек-лист при работе с конструкторами

При создании конструктора проверьте:

- ☐ **Имя начинается с заглавной буквы?**
- ☐ **Проверяете ли вы вызов с `new` (через `new.target`)?**
- ☐ **Методы определены в прототипе, а не в конструкторе?**
- ☐ **Не изменяете ли вы прототип после создания экземпляров?**
- ☐ **Используете ли вы классы ES6+ для нового кода?**
- ☐ **Приватные поля используют синтаксис `#field`?**
- ☐ **Для наследования используете `extends` и `super()`?**
- ☐ **Возвращаете ли вы новый объект из методов-трансформеров?**
- ☐ **Используете фабричные методы для сложной логики создания?**
- ☐ **Документируете ли вы публичный API конструктора?**

Заключение

Конструкторы и оператор `new` — это фундаментальная часть объектно-ориентированного программирования в JavaScript. Несмотря на появление классов в ES6, понимание работы конструкторов остается критически важным, так как классы — это всего лишь синтаксический сахар над ними.

Ключевые выводы:

1. **Конструктор** — обычная функция, вызываемая с `new`
2. **Оператор `new`** выполняет 4 шага: создание объекта, привязка прототипа, вызов конструктора, возврат объекта
3. `new.target` позволяет определить, вызван ли конструктор с `new`
4. **Методы должны быть в прототипе** для оптимизации памяти
5. **Классы ES6+** — предпочтительный синтаксис для нового кода
6. **Приватные поля ES2022+** обеспечивают настоящую инкапсуляцию
7. **Статические методы и свойства** принадлежат конструктору/классу
8. **Наследование** через конструкторы сложнее, чем через классы

Когда что использовать:

- **Классы ES6+** — для всего нового кода, наследования, приватных полей
- **Конструкторы** — при работе со старым кодом, библиотеками
- **Фабричные функции** — когда нужна гибкость в создании объектов
- `Object.create()` — для чистого прототипного наследования без конструкторов

Помните: хороший конструктор/класс должен быть простым, сфокусированным на одной ответственности, иметь четкий API и правильно использовать прототипы для эффективной работы с памятью.

Введение: что такое "синтаксический сахар"

Термин "**синтаксический сахар**" означает, что классы в ES6 предоставляют новый, более удобный синтаксис для уже существующей в JavaScript функциональности — прототипного наследования и конструкторов. Под капотом классы работают точно так же, как и ES5 конструкторы с прототипами, но с более понятным и структурированным синтаксисом.

Ключевое отличие от других языков:

- В Java/C++ классы — это **абстракция времени компиляции**
- В JavaScript классы — это **синтаксический сахар над существующими механизмами**

Часть 1: Базовый синтаксис классов

1.1. Объявление класса

javascript

// ES5: Конструктор и прототип

```
function PersonES5(name, age) {  
    this.name = name;  
    this.age = age;  
}
```

```
PersonES5.prototype.greet = function() {  
    return `Привет, я ${this.name}, мне ${this.age} лет`;  
};
```

```
PersonES5.describe = function() {  
    return 'Это конструктор Person';  
};
```

```
// ES6: Класс (синтаксический сахар)
class PersonES6 {
  // Конструктор (аналог функции-конструктора)
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Методы автоматически добавляются в прототип
  greet() {
    return `Привет, я ${this.name}, мне ${this.age} лет`;
  }

  // Статический метод (принадлежит классу, не экземплярам)
  static describe() {
    return 'Это класс Person';
  }
}

// Создание экземпляров идентично
const personES5 = new PersonES5('Иван', 30);
const personES6 = new PersonES6('Мария', 25);

console.log(personES5.greet()); // 'Привет, я Иван, мне 30 лет'
console.log(personES6.greet()); // 'Привет, я Мария, мне 25 лет'

// Проверка типа
console.log(typeof PersonES5); // 'function'
console.log(typeof PersonES6); // 'function' (класс - это функция!)

console.log(personES5 instanceof PersonES5); // true
console.log(personES6 instanceof PersonES6); // true

// Проверка прототипов
console.log(personES5.__proto__ === PersonES5.prototype); // true
console.log(personES6.__proto__ === PersonES6.prototype); // true

// Методы в прототипе
console.log(personES5.hasOwnProperty('greet')); // false
console.log(personES6.hasOwnProperty('greet')); // false
console.log(PersonES5.prototype.hasOwnProperty('greet')); // true
console.log(PersonES6.prototype.hasOwnProperty('greet')); // true
```

1.2. Что происходит "под капотом"

javascript

// Давайте посмотрим, как класс компилируется в ES5

```
class MyClass {  
    constructor(value) {  
        this.value = value;  
    }  
  
    getValue() {  
        return this.value;  
    }  
  
    static staticMethod() {  
        return 'static';  
    }  
}
```

// Примерное преобразование в ES5

```
function MyClassES5(value) {  
    // 1. Проверка new (автоматически в классах)  
    if (!(this instanceof MyClassES5)) {  
        throw new TypeError("Class constructor MyClass cannot be invoked without  
'new'");  
    }  
  
    // 2. Тело конструктора  
    this.value = value;  
}  
  
// 3. Методы в прототипе  
MyClassES5.prototype.getValue = function() {  
    return this.value;  
};  
  
// 4. Статические методы  
MyClassES5.staticMethod = function() {  
    return 'static';  
};  
  
// 5. Классы не перечисляемы по умолчанию  
Object.defineProperty(MyClassES5.prototype, 'constructor', {  
    value: MyClassES5,
```

```

    writable: true,
    configurable: true,
    enumerable: false // Не перечисляется в for...in
  });

// Сравнение
const instance1 = new MyClass('test');
const instance2 = new MyClassES5('test');

console.log('Сравнение методов:');
console.log(instance1.getValue()); // 'test'
console.log(instance2.getValue()); // 'test'

console.log('\nСравнение статических методов:');
console.log(MyClass.staticMethod()); // 'static'
console.log(MyClassES5.staticMethod()); // 'static'

console.log('\nПроверка конструктора:');
console.log(instance1.constructor === MyClass); // true
console.log(instance2.constructor === MyClassES5); // true

console.log('\nДескрипторы свойств:');
console.log(Object.getOwnPropertyDescriptor(MyClass.prototype,
'constructor').enumerable); // false
console.log(Object.getOwnPropertyDescriptor(MyClassES5.prototype,
'constructor').enumerable); // false

```

1.3. Особенности объявления классов

```

javascript

// Классы не поднимаются (hoisted) как функции
// console.log(FunctionExpression); // undefined
// console.log(ClassExpression); // ReferenceError

// Функциональное выражение
const FunctionExpression = function() {
  console.log('Функциональное выражение');
};

// Класс выражение (Class Expression)
const ClassExpression = class {
  constructor(name) {
    this.name = name;
  }
};

```

```

    }

    greet() {
        return `Привет, ${this.name}`;
    }
};

// Именованный класс выражение
const NamedClassExpression = class MyNamedClass {
    constructor(value) {
        this.value = value;
    }

    // Внутри класса можно ссылаться на его имя
    static getClassName() {
        return MyNamedClass.name;
    }

    // Но снаружи имя недоступно
    // console.Log(MyNamedClass); // ReferenceError
};

// Классы - это "first-class citizens" (можно передавать как аргументы)
function createClass(baseClass) {
    return class extends baseClass {
        constructor(...args) {
            super(...args);
            this.createdAt = new Date();
        }

        getCreationTime() {
            return this.createdAt;
        }
    };
}

// Использование фабрики классов
const BaseClass = class {
    constructor(name) {
        this.name = name;
    }
};

const ExtendedClass = createClass(BaseClass);

```

```
const instance = new ExtendedClass('Тест');

console.log('Фабрика классов:');
console.log(instance.name); // 'Тест'
console.log(instance.getCreationTime()); // текущая дата

// Классы можно экспортировать/импортировать
// export class ExportedClass { ... }
// import { ExportedClass } from './module.js';
```

Часть 2: Методы и свойства классов

2.1. Различные типы методов

```
javascript

class User {
  // Конструктор (специальный метод)
  constructor(name, email) {
    this.name = name;
    this.email = email;
    this.loginCount = 0;
  }

  // 1. Метод экземпляра (в prototype)
  greet() {
    return `Привет, ${this.name}!`;
  }

  // 2. Метод с вычисляемым именем
  ['say' + 'Hello']() {
    return `Hello, ${this.name}!`;
  }

  // 3. Геттер (getter)
  get profile() {
    return {
      name: this.name,
      email: this.email,
      logins: this.loginCount,
      active: this.loginCount > 0
    };
  }
};
```



```

    }

    // 4. Сеттер (setter)
    set nickname(nick) {
        if (nick.length < 2) {
            throw new Error('Никнейм должен быть минимум 2 символа');
        }
        this._nickname = nick;
    }

    get nickname() {
        return this._nickname || this.name;
    }

    // 5. Генератор (generator)
    *loginHistory() {
        let count = 0;
        while (count < this.loginCount) {
            yield `Логин #${++count} в ${new Date().toLocaleTimeString()}`;
        }
    }

    // 6. Асинхронный метод
    async fetchData() {
        const response = await fetch(`https://api.example.com/users/${this.email}`);
        return response.json();
    }

    // 7. Статический метод
    static validateEmail(email) {
        const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        return regex.test(email);
    }

    // 8. Статический геттер
    static get defaultUser() {
        return new User('Гость', 'guest@example.com');
    }

    // 9. Метод с использованием super (в наследовании)
    toString() {
        return `User: ${this.name} <${this.email}>`;
    }
}

```

// Использование

```
const user = new User('Анна', 'anna@example.com');

console.log('Методы экземпляра:');
console.log(user.greet()); // 'Привет, Анна!'
console.log(user.sayHello()); // 'Hello, Анна!' (вычисляемое имя)

console.log('\nГеттеры и сеттеры:');
console.log(user.profile); // {name: 'Анна', email: 'anna@example.com', ...}
user.nickname = 'Анюта';
console.log(user.nickname); // 'Анюта'

console.log('\nГенератор:');
user.loginCount = 3;
for (const login of user.loginHistory()) {
    console.log(login); // 'Логин #1...', 'Логин #2...', 'Логин #3...'
}

console.log('\nСтатические методы:');
console.log(User.validateEmail('test@example.com')); // true
console.log(User.validateEmail('invalid-email')); // false

const guest = User.defaultUser;
console.log(guest.greet()); // 'Привет, Гость!'

// Проверка типов методов
console.log('\nТипы методов:');
console.log(typeof user.greet); // 'function'
console.log(typeof user.profile); // 'object' (геттер вернул объект)
console.log(typeof User.validateEmail); // 'function'

// Дескрипторы свойств
console.log('\nДескрипторы:');
const greetDescriptor = Object.getOwnPropertyDescriptor(User.prototype, 'greet');
console.log('Метод:', {
    writable: greetDescriptor.writable,
    enumerable: greetDescriptor.enumerable,
    configurable: greetDescriptor.configurable
});

const profileDescriptor = Object.getOwnPropertyDescriptor(User.prototype, 'profile');
console.log('Геттер:', {
    get: typeof profileDescriptor.get,
```

```
    enumerable: profileDescriptor.enumerable
  });
```

2.2. Публичные поля и методы (ES2022+)

javascript

// Раньше поля объявлялись только в конструкторе

```
class OldStyleUser {
  constructor(name, age) {
    this.name = name; // Публичное поле
    this.age = age; // Публичное поле
    this._private = 'секретно'; // Конвенция для частных
  }
}
```

// Новый синтаксис (ES2022+): поля класса

```
class ModernUser {
  // Публичные поля (инициализация по умолчанию)
  name = 'Аноним';
  age = 0;
  tags = [];

  // Приватные поля (начинаются с #)
  #secret = 'конфиденциально';
  #internalId = Math.random();

  // Статические публичные поля
  static defaultRole = 'user';
  static version = '1.0.0';

  // Статические приватные поля
  static #instanceCount = 0;

  constructor(name, age) {
    // Можно переопределить значения по умолчанию
    if (name) this.name = name;
    if (age) this.age = age;

    // Увеличиваем счетчик экземпляров
    ModernUser.#instanceCount++;
    this.#internalId = ModernUser.#instanceCount;
  }
}
```

```

// Публичные методы
greet() {
    return `Привет, я ${this.name}, мне ${this.age} лет`;
}

// Приватные методы
#validateAge(age) {
    return age >= 0 && age <= 150;
}

// Метод с доступом к приватным полям
getSecretInfo() {
    return `ID: ${this.#internalId}, Secret: ${this.#secret.substring(0, 3)}...`;
}

// Статический метод с доступом к статическим приватным полям
static getInstanceCount() {
    return this.#instanceCount;
}

// Геттер для приватного поля
get internalId() {
    return this.#internalId;
}

// Сеттер с валидацией
set age(newAge) {
    if (this.#validateAge(newAge)) {
        this.age = newAge;
    } else {
        throw new Error('Некорректный возраст');
    }
}
}

// Использование
console.log('Публичные поля класса:');
console.log('Статическое поле:', ModernUser.defaultRole); // 'user'
console.log('Статическое поле:', ModernUser.version); // '1.0.0'

const user1 = new ModernUser('Анна', 25);
const user2 = new ModernUser('Иван', 30);

console.log(`\nЭкземпляры:`);

```

```

console.log(user1.greet()); // 'Привет, я Анна, мне 25 лет'
console.log(user2.greet()); // 'Привет, я Иван, мне 30 лет'

console.log('\nДоступ к приватным полям через методы:');
console.log(user1.getSecretInfo()); // 'ID: 1, Secret: кон...'
console.log(user2.getSecretInfo()); // 'ID: 2, Secret: кон...'

console.log('\nСтатические приватные поля:');
console.log('Всего экземпляров:', ModernUser.getInstanceCount()); // 2

console.log('\nГеттеры:');
console.log('Внутренний ID:', user1.internalId); // 1

// Проверка доступа
console.log('\nПроверка доступа:');
console.log('name' in user1); // true
console.log(user1.hasOwnProperty('name')); // true
console.log(user1.hasOwnProperty('#secret')); // false
// console.log(user1.#secret); // SyntaxError: Private field must be declared
// console.log(ModernUser.#instanceCount); // SyntaxError

// Сравнение с Object.defineProperty
console.log('\nСравнение дескрипторов:');
const nameDescriptor = Object.getOwnPropertyDescriptor(user1, 'name');
console.log('Поле name:', {
    value: nameDescriptor.value,
    writable: nameDescriptor.writable,
    enumerable: nameDescriptor.enumerable,
    configurable: nameDescriptor.configurable
});

// Поля добавляются как собственные свойства экземпляра
console.log('Собственные свойства:', Object.getOwnPropertyNames(user1));
// ['name', 'age', 'tags', '#secret', '#internalId']

```

2.3. Статические блоки инициализации (ES2022)

```

javascript

class ConfigurationManager {
    // Статические поля
    static config = {};
    static isInitialized = false;
    static apiEndpoints = {};
}

```

```

static #privateConfig = {};

// Статический блок инициализации (может быть несколько)
static {
    console.log('Первый статический блок: базовая инициализация');

    // Загрузка конфигурации по умолчанию
    this.config = {
        environment: 'development',
        debug: true,
        version: '1.0.0'
    };

    // Инициализация API endpoints
    this.apiEndpoints = {
        users: '/api/users',
        products: '/api/products',
        orders: '/api/orders'
    };
}

// Еще один статический блок
static {
    console.log('Второй статический блок: расширенная инициализация');

    // Можно использовать await для асинхронной инициализации
    // (но нужна обертка в async функцию)
    const loadPrivateConfig = async () => {
        // Имитация асинхронной загрузки
        await new Promise(resolve => setTimeout(resolve, 100));
        this.#privateConfig = {
            apiKey: process.env.API_KEY || 'default-key',
            secret: process.env.SECRET || 'default-secret'
        };
        this.isInitialized = true;
    };

    // Синхронная обертка для асинхронного кода
    (async () => {
        await loadPrivateConfig();
        console.log('Конфигурация загружена');
    })();
}

```

```

// Статический метод для доступа к приватной конфигурации
static getPrivateConfig(key) {
    if (!this.isInitialized) {
        throw new Error('Конфигурация не инициализирована');
    }
    return this.#privateConfig[key];
}

// Статический метод для обновления конфигурации
static updateConfig(newConfig) {
    this.config = { ...this.config, ...newConfig };
    console.log('Конфигурация обновлена:', this.config);
}

// Экземплярный метод
constructor(serviceName) {
    this.serviceName = serviceName;
    this.config = ConfigurationManager.config;
}

getEndpoint(endpointName) {
    const endpoint = ConfigurationManager.apiEndpoints[endpointName];
    if (!endpoint) {
        throw new Error(`Endpoint ${endpointName} не найден`);
    }
    return `${this.config.environment}${endpoint}`;
}
}

// Использование
console.log('\nИспользование ConfigurationManager:');

// Статические блоки выполняются при первом обращении к классу
console.log('Конфигурация:', ConfigurationManager.config);
console.log('API Endpoints:', ConfigurationManager.apiEndpoints);

// Создаем экземпляр
const service = new ConfigurationManager('UserService');
console.log('\nЭкземпляр service:');
console.log('Имя сервиса:', service.serviceName);
console.log('Полный endpoint users:', service.getEndpoint('users'));

// Обновление конфигурации
ConfigurationManager.updateConfig({ environment: 'production', debug: false });

```

```

console.log('\nПосле обновления:');
console.log('Новая конфигурация:', ConfigurationManager.config);
console.log('Полный endpoint users:', service.getEndpoint('users'));

// Доступ к приватной конфигурации (после инициализации)
setTimeout(() => {
    console.log('\nПриватная конфигурация:');
    try {
        console.log('API Key:', ConfigurationManager.getPrivateConfig('apiKey'));
    } catch (error) {
        console.log('Ошибка:', error.message);
    }
}, 200);

// Статические блоки выполняются только один раз
console.log('\nПовторный доступ:');
const anotherService = new ConfigurationManager('ProductService');
// Статические блоки не выполняются снова

```

Часть 3: Наследование в классах

3.1. Базовое наследование

```

javascript

// Базовый класс
class Animal {
    constructor(name, speed = 0) {
        this.name = name;
        this.speed = speed;
        this.energy = 100;
    }

    move(speed) {
        this.speed = speed;
        console.log(`${this.name} движется со скоростью ${this.speed} км/ч`);
    }

    eat(food) {
        this.energy += 10;
        console.log(`${this.name} ест ${food}. Энергия: ${this.energy}`);
    }
}

```



```

sleep() {
    this.energy = Math.min(100, this.energy + 30);
    console.log(`${this.name} спит. Энергия: ${this.energy}`);
}

// Абстрактный метод (условно, т.к. в JS нет настоящих абстрактных методов)
makeSound() {
    throw new Error('Метод makeSound должен быть переопределен');
}

// Статический метод
static isAnimal(obj) {
    return obj instanceof Animal;
}

// Геттер
get info() {
    return `${this.name}, скорость: ${this.speed}, энергия: ${this.energy}`;
}
}

// Дочерний класс
class Dog extends Animal {
    // Если не определяем конструктор, используется родительский
    // constructor(...args) {
    //     super(...args);
    // }

    // Переопределение конструктора с дополнительными параметрами
    constructor(name, breed, speed = 0) {
        super(name, speed); // Вызов конструктора родителя
        this.breed = breed; // Дополнительное свойство
        this.woofCount = 0;
    }

    // Переопределение метода
    makeSound() {
        this.woofCount++;
        console.log(`${this.name} (${this.breed}) лает: Гав-гав!`);
        this.energy -= 5;
    }

    // Новый метод

```

```

fetch(item) {
    if (this.energy < 20) {
        console.log(`${this.name} слишком устал для апорта`);
        return;
    }
    this.energy -= 15;
    console.log(`${this.name} принес ${item}`);
}

// Переопределение геттера
get info() {
    return `${super.info}, порода: ${this.breed}, лаял: ${this.woofCount} раз`;
}
}

// Еще один дочерний класс
class Cat extends Animal {
    constructor(name, color, speed = 0) {
        super(name, speed);
        this.color = color;
        this.miceCaught = 0;
    }

    makeSound() {
        console.log(`${this.name} (${this.color}) мяукает: Мяу!`);
        this.energy -= 3;
    }

    // Новый метод
    catchMouse() {
        const success = Math.random() > 0.5;
        if (success) {
            this.miceCaught++;
            this.energy += 20;
            console.log(`${this.name} поймал мышь! Всего: ${this.miceCaught}`);
        } else {
            this.energy -= 10;
            console.log(`${this.name} не смог поймать мышь`);
        }
    }
}

// Коты двигаются по-другому
move(speed) {
    super.move(speed * 0.5); // Коты двигаются медленнее
}

```

```
        console.log('(Коты не любят спешить)');
    }
}
```

// Использование

```
console.log('Наследование классов:');
```

```
const animal = new Animal('Просто животное', 10);
const dog = new Dog('Бобик', 'овчарка', 20);
const cat = new Cat('Мурзик', 'серый', 15);
```

```
console.log('\nБазовый класс:');
animal.move(10);
animal.eat('траву');
```

// animal.makeSound(); // Error: Метод makeSound должен быть переопределен

```
console.log('\nСобака:');
dog.move(20);
dog.makeSound();
dog.fetch('палку');
dog.eat('кость');
console.log(dog.info);
```

```
console.log('\nКот:');
cat.move(10);
cat.makeSound();
cat.catchMouse();
cat.catchMouse();
console.log(cat.info);
```

```
console.log('\nПроверка наследования:');
console.log('dog instanceof Dog:', dog instanceof Dog); // true
console.log('dog instanceof Animal:', dog instanceof Animal); // true
console.log('dog instanceof Object:', dog instanceof Object); // true
```

```
console.log('cat instanceof Cat:', cat instanceof Cat); // true
console.log('cat instanceof Animal:', cat instanceof Animal); // true
```

```
console.log('\nСтатические методы:');
console.log('Animal.isAnimal(dog):', Animal.isAnimal(dog)); // true
console.log('Dog.isAnimal(dog):', Dog.isAnimal(dog)); // true (унаследовано)
```

// Цепочка прототипов

```
console.log('\nЦепочка прототипов:');
```

```

console.log('dog.__proto__ === Dog.prototype:', dog.__proto__ === Dog.prototype); // true
console.log('Dog.prototype.__proto__ === Animal.prototype:', Dog.prototype.__proto__ ===
Animal.prototype); // true
console.log('Animal.prototype.__proto__ === Object.prototype:',
Animal.prototype.__proto__ === Object.prototype); // true
console.log('Object.prototype.__proto__:', Object.prototype.__proto__); // null

```

3.2. **super** - ключ к наследованию

javascript

```

class Vehicle {
  constructor(type, wheels) {
    this.type = type;
    this.wheels = wheels;
    this.speed = 0;
    this.#logCreation(); // Приватный метод
  }

  // Приватное поле (ES2022)
  #creationLog = [];

  // Приватный метод
  #logCreation() {
    this.#creationLog.push({
      type: this.type,
      time: new Date(),
      speed: this.speed
    });
  }

  accelerate(amount) {
    this.speed += amount;
    console.log(`${this.type} ускоряется до ${this.speed} км/ч`);
    return this; // Для чейнинга
  }

  brake(amount) {
    this.speed = Math.max(0, this.speed - amount);
    console.log(`${this.type} замедляется до ${this.speed} км/ч`);
    return this;
  }

  getCreationLog() {

```

```

        return [...this.#creationLog]; // Копия приватного массива
    }

    // Защищенный метод (конвенция)
    _internalMethod() {
        return 'Внутренний метод';
    }
}

class Car extends Vehicle {
    constructor(make, model, year) {
        // super должен быть вызван ДО использования this
        // this.make = make; // ReferenceError

        super('автомобиль', 4); // Вызов конструктора родителя
        this.make = make;
        this.model = model;
        this.year = year;
        this.gear = 'P'; // Паркинг

        // super уже вызван, можно использовать this
        console.log(`Создан ${this.year} ${this.make} ${this.model}`);
    }

    // Переопределение с вызовом родительского метода
    accelerate(amount) {
        console.log(`Включена передача для ускорения`);
        // Вызов родителя
        super.accelerate(amount);
        this.gear = 'D'; // Драйв
        return this;
    }

    brake(amount) {
        console.log(`Торможение ${this.model}`);
        const result = super.brake(amount);
        if (this.speed === 0) {
            this.gear = 'P';
        }
        return result;
    }

    // Новый метод
    changeGear(gear) {

```

```

    const validGears = ['P', 'R', 'N', 'D'];
    if (validGears.includes(gear)) {
        this.gear = gear;
        console.log(`${this.model}: Передача изменена на ${gear}`);
    }
    return this;
}

// Переопределение защищенного метода
_internalMethod() {
    const parentResult = super._internalMethod();
    return `${parentResult} (расширен в Car)`;
}

// Геттер с использованием super
get info() {
    const vehicleInfo = `${this.type}, ${this.wheels} колеса, скорость: ${this.speed}`;

    return `${this.year} ${this.make} ${this.model} - ${vehicleInfo}, передача: ${this.gear}`;
}

// Статический метод с super (обращение к статическому методу родителя)
static compare(vehicle1, vehicle2) {
    // Если бы в Vehicle был статический метод...
    // return super.compare(vehicle1, vehicle2);
    return vehicle1.speed - vehicle2.speed;
}
}

class ElectricCar extends Car {
    constructor(make, model, year, batteryCapacity) {
        super(make, model, year);
        this.batteryCapacity = batteryCapacity;
        this.batteryLevel = 100;
        this.type = 'электромобиль'; // Переопределяем type
    }

    // Полное переопределение
    accelerate(amount) {
        if (this.batteryLevel < 10) {
            console.log(`Батарея разряжена! Только ${this.batteryLevel}%`);
            return this;
        }
    }
}

```

```

        console.log(`Электродвигатель включается...`);
        // Используем родительский accelerate из Car
        super.accelerate(amount * 1.5); // Электромашины ускоряются быстрее

        this.batteryLevel -= amount * 0.5;
        console.log(`Батарея: ${this.batteryLevel}%`);

        return this;
    }

    // Новый метод
    charge(percent) {
        this.batteryLevel = Math.min(100, this.batteryLevel + percent);
        console.log(`Заряжено до ${this.batteryLevel}%`);
        return this;
    }

    // Переопределение геттера
    get info() {
        const carInfo = super.info; // Вызов геттера из Car
        return `${carInfo}, батарея: ${this.batteryLevel}%`;
    }
}

// Использование
console.log('Использование super:');

const car = new Car('Toyota', 'Camry', 2022);
console.log(`\nОбычный автомобиль:`);
car.accelerate(50).changeGear('D').brake(20);
console.log(car.info);
console.log('Защищенный метод:', car._internalMethod());
console.log('Лог создания:', car.getCreationLog());

console.log(`\nЭлектромашина:`);
const tesla = new ElectricCar('Tesla', 'Model S', 2023, 100);
tesla.accelerate(60).charge(30).brake(40);
console.log(tesla.info);

console.log(`\nЦепочка super:`);
// tesla.accelerate() вызывает Car.accelerate(), который вызывает Vehicle.accelerate()
// Но тут переопределен в ElectricCar

```

```

console.log('\nПроверка instanceof:');
console.log('tesla instanceof ElectricCar:', tesla instanceof ElectricCar); // true
console.log('tesla instanceof Car:', tesla instanceof Car); // true
console.log('tesla instanceof Vehicle:', tesla instanceof Vehicle); // true
console.log('tesla instanceof Object:', tesla instanceof Object); // true

// super в статических методах
class A {
    static staticMethod() {
        return 'A';
    }
}

class B extends A {
    static staticMethod() {
        return super.staticMethod() + ' B';
    }
}

console.log('\nSuper в статических методах:');
console.log('B.staticMethod():', B.staticMethod()); // 'A B'

```

3.3. Множественное наследование через миксины

```

javascript

// Поскольку JavaScript не поддерживает множественное наследование напрямую,
// используем миксины (mixins)

// Базовые миксины
const CanSwim = Base => class extends Base {
    swim(speed) {
        console.log(`${this.name || 'Объект'} плывет со скоростью ${speed} км/ч`);
        this.location = 'water';
        return this;
    }

    dive(depth) {
        console.log(`${this.name || 'Объект'} ныряет на глубину ${depth}м`);
        return this;
    }
};

const CanFly = Base => class extends Base {

```



```

    fly(speed) {
        console.log(`${this.name || 'Объект'} летит со скоростью ${speed} км/ч`);
        this.location = 'air';
        return this;
    }

    land() {
        console.log(`${this.name || 'Объект'} приземляется`);
        this.location = 'ground';
        return this;
    }
};

```

```

const CanWalk = Base => class extends Base {
    walk(speed) {
        console.log(`${this.name || 'Объект'} идет со скоростью ${speed} км/ч`);
        this.location = 'ground';
        return this;
    }

    run(speed) {
        console.log(`${this.name || 'Объект'} бежит со скоростью ${speed} км/ч`);
        return this;
    }
};

```

// Базовый класс

```

class Animal {
    constructor(name) {
        this.name = name;
        this.location = 'ground';
    }

    eat(food) {
        console.log(`${this.name} ест ${food}`);
        return this;
    }
}

```

// Применение миксинов в разных комбинациях

```

class Duck extends CanSwim(CanFly(CanWalk(Animal))) {
    constructor(name) {
        super(name);
        this.species = 'утка';
    }
}

```

```

    }

    quack() {
        console.log(`${this.name} крикает: Кря-кря!`);
        return this;
    }
}

class Penguin extends CanSwim(CanWalk(Animal)) {
    constructor(name) {
        super(name);
        this.species = 'пингвин';
    }

    slide() {
        console.log(`${this.name} скользит по льду`);
        return this;
    }
}

// Класс, который сам является миксином
const HasEngine = Base => class extends Base {
    constructor(...args) {
        super(...args);
        this.engineOn = false;
    }

    startEngine() {
        this.engineOn = true;
        console.log(`${this.name || 'Транспорт'}: Двигатель заведен`);
        return this;
    }

    stopEngine() {
        this.engineOn = false;
        console.log(`${this.name || 'Транспорт'}: Двигатель заглушен`);
        return this;
    }
};

class Boat extends CanSwim(HasEngine(Animal)) {
    constructor(name) {
        super(name);
        this.type = 'лодка';
    }
}

```

```

    }

    honk() {
        console.log(`${this.name}: Ту-ту!`);
        return this;
    }
}

// Использование
console.log('Множественное наследование через миксины:');

const donald = new Duck('Дональд');
console.log('\nУтка (плавает, летает, ходит:');
donald.walk(5).fly(20).swim(3).quack().eat('хлеб');

const skipper = new Penguin('Скиппер');
console.log('\nПингвин (плавает, ходит, но не летает:');
skipper.walk(2).swim(10).slide().eat('рыбу');
// skipper.fly(); // Error: skipper.fly is not a function

const speedboat = new Boat('Спидбот');
console.log('\nЛодка (плавает, имеет двигатель:');
speedboat.startEngine().swim(30).honk().stopEngine();

// Проверка цепочек прототипов
console.log('\nЦепочки прототипов:');
console.log('donald instanceof Duck:', donald instanceof Duck); // true
console.log('donald instanceof Animal:', donald instanceof Animal); // true
console.log('skipper instanceof Penguin:', skipper instanceof Penguin); // true

// Миксины создают динамические классы
console.log('\nДинамические классы:');
const SwimAndFly = CanSwim(CanFly(class {}));
const obj = new SwimAndFly();
obj.swim(10).fly(20);

// Фабрика классов с миксинами
function createAmphibiousVehicle(name) {
    return class extends CanSwim(CanWalk(HasEngine(class {}))) {
        constructor() {
            super();
            this.name = name;
        }
    }
}

```

```

        transform() {
            console.log(`${this.name} трансформируется!`);
            return this;
        }
    };
}

```

```

const AmphibiousTank = createAmphibiousVehicle('Амфибия');
const tank = new AmphibiousTank();
tank.startEngine().walk(15).transform().swim(8);

```

Часть 4: Под капотом классов

4.1. Как классы компилируются в ES5

javascript

// Исходный класс ES6

```

class Person {
    static species = 'Homo sapiens';

    #privateField = 'secret';

    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    greet() {
        return `Hello, I'm ${this.name}`;
    }

    get doubleAge() {
        return this.age * 2;
    }

    set nickname(value) {
        this._nickname = value;
    }

    get nickname() {
        return this._nickname;
    }
}

```

```

    }

    static describe() {
        return `Species: ${this.species}`;
    }

    #privateMethod() {
        return this.#privateField;
    }

    accessPrivate() {
        return this.#privateMethod();
    }
}

// Примерное преобразование в ES5
const PersonES5 = (function() {
    // 1. Приватные поля через WeakMap
    const privateFields = new WeakMap();

    // 2. Функция-конструктор
    function Person(name, age) {
        // Проверка new
        if (!(this instanceof Person)) {
            throw new TypeError("Class constructor Person cannot be invoked without 'new'");
        }

        // Инициализация приватных полей
        privateFields.set(this, {
            privateField: 'secret'
        });

        // Публичные поля
        this.name = name;
        this.age = age;
        this._nickname = undefined;
    }

    // 3. Статические поля
    Person.species = 'Homo sapiens';

    // 4. Методы в прототипе
    Person.prototype.greet = function() {

```

```

        return `Hello, I'm ${this.name}`;
    };

    // 5. Геттеры и сеттеры
    Object.defineProperty(Person.prototype, 'doubleAge', {
        get: function() {
            return this.age * 2;
        },
        enumerable: false,
        configurable: true
    });

    Object.defineProperty(Person.prototype, 'nickname', {
        get: function() {
            return this._nickname;
        },
        set: function(value) {
            this._nickname = value;
        },
        enumerable: false,
        configurable: true
    });

    // 6. Приватные методы (симуляция)
    Person.prototype.accessPrivate = function() {
        const privates = privateFields.get(this);
        // Симуляция приватного метода
        return privates.privateField;
    };

    // 7. Статические методы
    Person.describe = function() {
        return `Species: ${this.species}`;
    };

    // 8. Классы не перечисляемы по умолчанию
    Object.defineProperty(Person.prototype, 'constructor', {
        value: Person,
        writable: true,
        configurable: true,
        enumerable: false
    });

    return Person;

```

```
})();
```

```
// Сравнение
```

```
console.log('Сравнение класса ES6 и его ES5 эквивалента:');
```

```
const es6Instance = new Person('Alice', 25);
```

```
const es5Instance = new PersonES5('Bob', 30);
```

```
console.log('\nМетоды экземпляра:');
```

```
console.log('ES6:', es6Instance.greet()); // 'Hello, I'm Alice'
```

```
console.log('ES5:', es5Instance.greet()); // 'Hello, I'm Bob'
```

```
console.log('\nГеттеры:');
```

```
console.log('ES6 doubleAge:', es6Instance.doubleAge); // 50
```

```
console.log('ES5 doubleAge:', es5Instance.doubleAge); // 60
```

```
console.log('\nСеттеры:');
```

```
es6Instance.nickname = 'Ali';
```

```
es5Instance.nickname = 'Bobby';
```

```
console.log('ES6 nickname:', es6Instance.nickname); // 'Ali'
```

```
console.log('ES5 nickname:', es5Instance.nickname); // 'Bobby'
```

```
console.log('\nСтатические методы:');
```

```
console.log('ES6:', Person.describe()); // 'Species: Homo sapiens'
```

```
console.log('ES5:', PersonES5.describe()); // 'Species: Homo sapiens'
```

```
console.log('\nПриватные поля:');
```

```
console.log('ES6 accessPrivate:', es6Instance.accessPrivate()); // 'secret'
```

```
console.log('ES5 accessPrivate:', es5Instance.accessPrivate()); // 'secret'
```

```
console.log('\nПроверка типа:');
```

```
console.log('ES6 instanceof:', es6Instance instanceof Person); // true
```

```
console.log('ES5 instanceof:', es5Instance instanceof PersonES5); // true
```

```
console.log('\nКонструктор:');
```

```
console.log('ES6 constructor enumerable:',
```

```
    Object.getOwnPropertyDescriptor(Person.prototype, 'constructor').enumerable); //
```

```
false
```

```
console.log('ES5 constructor enumerable:',
```

```
    Object.getOwnPropertyDescriptor(PersonES5.prototype, 'constructor').enumerable); //
```

```
false
```

4.2. Отличия классов от конструкторов

```
javascript

// 1. Классы нельзя вызывать без new
class ClassExample {
  constructor() {
    this.value = 42;
  }
}

function ConstructorExample() {
  this.value = 42;
}

console.log('1. Вызов без new:');
// ConstructorExample(); // Установит глобальную переменную (в нестрогом режиме)
// ClassExample(); // TypeError: Class constructor ClassExample cannot be invoked without
'new'

// 2. Методы класса не перечисляемы
console.log('\n2. Перечисляемость методов:');
for (let key in new ConstructorExample()) {
  console.log('Constructor:', key); // Ничего, если нет методов в конструкторе
}

ConstructorExample.prototype.method = function() {};
for (let key in new ConstructorExample()) {
  console.log('Constructor with method:', key); // 'method'
}

for (let key in new ClassExample()) {
  console.log('Class:', key); // Ничего (методы не перечисляемы)
}

// 3. Классы находятся в строгом режиме
console.log('\n3. Строгий режим:');
class StrictClass {
  constructor() {
    // undeclaredVar = 42; // ReferenceError в строгом режиме
  }

  method() {
    // this в методах не привязан к глобальному объекту
  }
}
```



```

        console.log('this в методе класса:', this);
    }
}

function NonStrictConstructor() {
    // undeclaredVar = 42; // Создаст глобальную переменную (в нестрогом режиме)
}

// 4. Классы имеют внутреннее свойство [[ConstructorKind]]:"derived"
console.log('\n4. Внутренние различия:');
class BaseClass {
    constructor() {
        console.log('BaseClass constructor, new.target:', new.target.name);
    }
}

class DerivedClass extends BaseClass {
    constructor() {
        super();
        console.log('DerivedClass constructor');
    }
}

function BaseConstructor() {
    console.log('BaseConstructor, new.target:', new.target);
}

function DerivedConstructor() {
    BaseConstructor.call(this);
    console.log('DerivedConstructor');
}

console.log('Классы:');
new DerivedClass(); // new.target = DerivedClass

console.log('\nКонструкторы:');
new DerivedConstructor(); // new.target = undefined

// 5. Свойство prototype у классов не перезаписываемо
console.log('\n5. Свойство prototype:');
class PrototypeClass {}
console.log('Можно перезаписать prototype у класса?',
    Object.getOwnPropertyDescriptor(PrototypeClass, 'prototype').writable); // false

```

```

function PrototypeConstructor() {}
console.log('Можно перезаписать prototype у конструктора?',
  Object.getOwnPropertyDescriptor(PrototypeConstructor, 'prototype').writable); // true

// 6. Поднятие (hoisting)
console.log('\n6. Поднятие:');
console.log('До объявления конструктора:', typeof HoistedConstructor); // 'function'
console.log('До объявления класса:', typeof HoistedClass); // 'undefined'

function HoistedConstructor() {}
class HoistedClass {}

// 7. Имена методов вычисляются во время определения
console.log('\n7. Вычисляемые имена методов:');
const methodName = 'dynamicMethod';

class ComputedClass {
  [methodName]() {
    return 'Вычисляемый метод';
  }

  ['pre' + 'fix' + 'Method']() {
    return 'Метод с выражением';
  }
}

const computedInstance = new ComputedClass();
console.log(computedInstance.dynamicMethod()); // 'Вычисляемый метод'
console.log(computedInstance.prefixMethod()); // 'Метод с выражением'

// Для конструкторов нужно использовать defineProperty
function ComputedConstructor() {}
Object.defineProperty(ComputedConstructor.prototype, methodName, {
  value: function() {
    return 'Вычисляемый метод в конструкторе';
  },
  enumerable: true
});

```

4.3. Ограничения и особенности классов

javascript

// 1. Классы не могут содержать только геттеры/сеттеры

```

class InvalidClass {
    // Только геттер без сеттера - ОК
    get value() {
        return this._value;
    }

    // Только сеттер без геттера - ОК
    set another(value) {
        this._another = value;
    }

    // Но не может быть класса только с геттерами/сеттерами без других членов
    // На практике это работает, но бесполезно
}

// 2. Методы классов не могут быть конструкторами
console.log('\n2. Методы как конструкторы:');
class MethodConstructor {
    method() {
        console.log('Я метод');
    }
}

const instance = new MethodConstructor();
try {
    new instance.method(); // TypeError: instance.method is not a constructor
} catch (error) {
    console.log('Ошибка:', error.message);
}

// 3. Классы не могут расширять примитивы
// class BadClass extends null { } // Ошибка в некоторых случаях
// class BadClass2 extends 42 { } // TypeError

// Но можно расширять null с дополнительной логикой
class NullExtendingClass extends null {
    constructor() {
        // super() не будет работать, т.к. нет родительского конструктора
        // Нужно явно возвращать объект
        return Object.create(NullExtendingClass.prototype);
    }

    static staticMethod() {
        return 'Статический метод работает';
    }
}

```

```

    }
}

console.log('\n3. Расширение null:');
console.log('Статический метод:', NullExtendingClass.staticMethod()); // Работает
const nullExtendingInstance = new NullExtendingClass();
console.log('Экземпляр прототипа:', Object.getPrototypeOf(nullExtendingInstance)); //
NullExtendingClass.prototype
console.log('Родительский прототип:',
Object.getPrototypeOf(NullExtendingClass.prototype)); // null

// 4. Нельзя определять свойства с одинаковым именем
class DuplicateProperties {
    constructor() {
        this.value = 1;
        // this.value = 2; // Перезапишет, но это другое
    }

    // method() {} // OK

    // ОШИБКА: Duplicate property name
    // get value() { return this._value; }
    // set value(val) { this._value = val; }
    // ↑ Нельзя иметь свойство value и геттер/сеттер value
}

// 5. Приватные поля должны быть объявлены заранее
class PrivateFieldClass {
    #privateField; // Должно быть объявлено здесь

    constructor() {
        this.#privateField = 42; // OK
        // this.#anotherPrivate = 10; // SyntaxError: Private field must be declared
    }
}

// 6. Статические блоки не могут быть асинхронными напрямую
class AsyncStaticBlock {
    static data = null;

    static {
        // Нельзя использовать await напрямую
        // await this.loadData(); // SyntaxError
    }
}

```

```

    // Но можно обернуть в IIFE
    (async () => {
        await new Promise(resolve => setTimeout(resolve, 100));
        this.data = 'loaded';
    })();
}

static async loadData() {
    return 'data';
}
}

// 7. Классы могут быть экспортированы до объявления
// export { MyClass }; // OK
// class MyClass { ... }

// 8. Метод constructor может быть асинхронным
class AsyncConstructor {
    constructor(data) {
        // Не может быть async, но может возвращать Promise
        return (async () => {
            await this.initialize(data);
            return this;
        })();
    }

    async initialize(data) {
        this.data = data;
        await this.loadAdditionalData();
    }

    async loadAdditionalData() {
        // Имитация загрузки
        await new Promise(resolve => setTimeout(resolve, 100));
        this.additional = 'loaded';
    }
}

console.log('\n8. Асинхронный конструктор:');
(async () => {
    const asyncInstance = await new AsyncConstructor('test');
    console.log('Асинхронный экземпляр:', asyncInstance.data, asyncInstance.additional);
})();

```

// 9. Классы могут быть сериализованы только если нет методов/геттеров

```
class SerializableClass {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Методы не сериализуются
  greet() {
    return `Hello, ${this.name}`;
  }
}

const serializable = new SerializableClass('John', 30);
console.log(`\n9. Сериализация:`);
console.log('JSON.stringify:', JSON.stringify(serializable)); //
'{"name": "John", "age": 30}'
console.log('Метод после десериализации:',
  JSON.parse(JSON.stringify(serializable)).greet); // undefined
```

Часть 5: Практические паттерны с классами

5.1. Паттерн "Строитель" (Builder) с классами

javascript

// Класс продукта

```
class Computer {
  constructor() {
    this.cpu = null;
    this.ram = null;
    this.storage = null;
    this.gpu = null;
    this.monitor = null;
  }

  toString() {
    return `Компьютер:
Процессор: ${this.cpu}
Оперативная память: ${this.ram}
Накопитель: ${this.storage}
Видеокарта: ${this.gpu || 'Интегрированная'}`;
  }
}
```

```
Монитор: ${this.monitor || 'Не включен'}`;
    }

    boot() {
        console.log(`Загрузка компьютера с ${this.cpu} и ${this.ram} RAM...`);
        if (this.gpu) {
            console.log(`Видеокарта ${this.gpu} инициализирована`);
        }
        console.log('Система готова к работе!');
    }
}
```

// Строитель

```
class ComputerBuilder {
    constructor() {
        this.computer = new Computer();
    }

    setCPU(cpu) {
        this.computer.cpu = cpu;
        return this;
    }

    setRAM(ram) {
        this.computer.ram = ram;
        return this;
    }

    setStorage(storage) {
        this.computer.storage = storage;
        return this;
    }

    setGPU(gpu) {
        this.computer.gpu = gpu;
        return this;
    }

    setMonitor(monitor) {
        this.computer.monitor = monitor;
        return this;
    }

    build() {
```

```

// Проверка минимальных требований
if (!this.computer.cpu || !this.computer.ram || !this.computer.storage) {
    throw new Error('Необходимо указать CPU, RAM и Storage');
}

const computer = this.computer;
this.computer = new Computer(); // Сброс для следующей сборки
return computer;
}
}

```

```

// Директор (опционально)
class GamingComputerDirector {
    constructor(builder) {
        this.builder = builder;
    }

    buildHighEndGamingPC() {
        return this.builder
            .setCPU('Intel Core i9-13900K')
            .setRAM('32GB DDR5')
            .setStorage('2TB NVMe SSD')
            .setGPU('NVIDIA RTX 4090')
            .setMonitor('32" 4K 144Hz')
            .build();
    }

    buildBudgetGamingPC() {
        return this.builder
            .setCPU('AMD Ryzen 5 5600X')
            .setRAM('16GB DDR4')
            .setStorage('1TB SSD')
            .setGPU('NVIDIA RTX 3060')
            .setMonitor('27" 1080p 144Hz')
            .build();
    }
}

```

```

// Использование
console.log('Паттерн Строитель с классами:');

```

```

// Прямое использование строителя
const builder = new ComputerBuilder();
const officePC = builder

```



```

        .setCPU('Intel Core i5')
        .setRAM('16GB')
        .setStorage('512GB SSD')
        .build();

console.log('\nОфисный компьютер:');
console.log(officePC.toString());
officePC.boot();

// Использование через директор
const gamingDirector = new GamingComputerDirector(new ComputerBuilder());
const highEndPC = gamingDirector.buildHighEndGamingPC();

console.log('\nИгровой компьютер (высокого класса):');
console.log(highEndPC.toString());
highEndPC.boot();

// Цепочка вызовов
console.log('\nКастомная сборка:');
const customPC = new ComputerBuilder()
    .setCPU('AMD Ryzen 7 7800X3D')
    .setRAM('64GB DDR5')
    .setStorage('4TB NVMe SSD')
    .setGPU('AMD Radeon RX 7900 XTX')
    .setMonitor('34" Ultrawide QD-OLED')
    .build();

console.log(customPC.toString());

// Наследование строителей
class ServerBuilder extends ComputerBuilder {
    setRAID(level, disks) {
        this.computer.raid = { level, disks };
        return this;
    }

    setRedundantPowerSupply(count) {
        this.computer.redundantPSU = count;
        return this;
    }
}

// Переопределение build с дополнительной валидацией
build() {
    if (!this.computer.raid) {

```

```

        throw new Error('Сервер должен иметь RAID конфигурацию');
    }
    return super.build();
}
}

```

```

console.log(`\nСерверный строитель:`);
const server = new ServerBuilder()
    .setCPU('2x Intel Xeon Platinum')
    .setRAM('256GB ECC RAM')
    .setStorage('10TB SAS HDD')
    .setRAID(10, 8)
    .setRedundantPowerSupply(2)
    .build();

```

```

console.log('Сервер:', server);

```

5.2. Паттерн "Наблюдатель" (Observer) с классами

javascript

// Интерфейс наблюдателя

```

class Observer {
    update(data) {
        throw new Error('Метод update должен быть реализован');
    }
}

```

// Конкретные наблюдатели

```

class EmailNotifier extends Observer {
    constructor(email) {
        super();
        this.email = email;
    }

    update(data) {
        console.log(`Отправка email на ${this.email}:`);
        console.log(` Событие: ${data.event}`);
        console.log(` Данные: ${JSON.stringify(data.payload)}`);
        console.log(` Время: ${new Date(data.timestamp).toLocaleString()}`);
    }
}

```

```

class SMSNotifier extends Observer {

```

```

constructor(phone) {
    super();
    this.phone = phone;
}

update(data) {
    console.log(`Отправка SMS на ${this.phone}:`);
    console.log(`  ${data.event}: ${JSON.stringify(data.payload)}`);
}
}

class Logger extends Observer {
    update(data) {
        const logEntry = {
            timestamp: new Date().toISOString(),
            event: data.event,
            payload: data.payload,
            source: data.source
        };
        console.log('[LOG]', JSON.stringify(logEntry));
    }
}

// Субъект (наблюдаемый объект)
class Observable {
    constructor() {
        this.observers = new Map(); // eventType -> Observer[]
        this.history = [];
    }

    // Подписка на событие
    subscribe(eventType, observer) {
        if (!this.observers.has(eventType)) {
            this.observers.set(eventType, []);
        }

        const observers = this.observers.get(eventType);
        if (!observers.includes(observer)) {
            observers.push(observer);
            console.log(`Наблюдатель ${observer.constructor.name} подписан на событие "${eventType}"`);
        }

        return this; // Для чейнинга
    }
}

```

```

    }

    // Отписка от события
    unsubscribe(eventType, observer) {
        if (this.observers.has(eventType)) {
            const observers = this.observers.get(eventType);
            const index = observers.indexOf(observer);
            if (index !== -1) {
                observers.splice(index, 1);
                console.log(`Наблюдатель ${observer.constructor.name} отписан от события
"${eventType}"`);
            }
        }
        return this;
    }

    // Уведомление наблюдателей
    notify(eventType, payload = {}) {
        const event = {
            event: eventType,
            payload,
            timestamp: Date.now(),
            source: this.constructor.name
        };

        // Сохраняем в историю
        this.history.push(event);

        // Уведомляем наблюдателей
        if (this.observers.has(eventType)) {
            const observers = this.observers.get(eventType);
            observers.forEach(observer => {
                try {
                    observer.update(event);
                } catch (error) {
                    console.error(`Ошибка в наблюдателе ${observer.constructor.name}:`,
error);
                }
            });
        }

        // Также уведомляем наблюдателей, подписанных на все события
        if (this.observers.has('*')) {
            const allObservers = this.observers.get('*');

```

```

        allObservers.forEach(observer => {
            try {
                observer.update(event);
            } catch (error) {
                console.error(`Ошибка в наблюдателе ${observer.constructor.name}:`,
error);
            }
        });
    }

    return this;
}

// Получение истории событий
getHistory(filter = null) {
    if (filter) {
        return this.history.filter(event => event.event === filter);
    }
    return [...this.history];
}

// Очистка истории
clearHistory() {
    this.history = [];
    return this;
}
}

// Конкретный наблюдаемый объект
class OrderSystem extends Observable {
    constructor() {
        super();
        this.orders = new Map();
        this.orderCounter = 0;
    }

    createOrder(customer, items) {
        const orderId = `ORDER_${++this.orderCounter}`;
        const order = {
            id: orderId,
            customer,
            items,
            status: 'created',
            createdAt: new Date(),

```

```

        total: items.reduce((sum, item) => sum + item.price * item.quantity, 0)
    };

    this.orders.set(orderId, order);

    // Уведомляем о создании заказа
    this.notify('order_created', {
        orderId,
        customer,
        itemCount: items.length,
        total: order.total
    });

    return orderId;
}

updateOrderStatus(orderId, status) {
    const order = this.orders.get(orderId);
    if (!order) {
        throw new Error(`Заказ ${orderId} не найден`);
    }

    const oldStatus = order.status;
    order.status = status;
    order.updatedAt = new Date();

    // Уведомляем об изменении статуса
    this.notify('order_status_changed', {
        orderId,
        oldStatus,
        newStatus: status,
        customer: order.customer
    });

    // Специальные уведомления для определенных статусов
    if (status === 'shipped') {
        this.notify('order_shipped', {
            orderId,
            customer: order.customer,
            address: order.shippingAddress
        });
    } else if (status === 'delivered') {
        this.notify('order_delivered', {
            orderId,

```

```

        customer: order.customer,
        deliveredAt: new Date()
    });
}

return this;
}

cancelOrder(orderId, reason) {
    const order = this.orders.get(orderId);
    if (!order) {
        throw new Error(`Заказ ${orderId} не найден`);
    }

    order.status = 'cancelled';
    order.cancelledAt = new Date();
    order.cancellationReason = reason;

    this.notify('order_cancelled', {
        orderId,
        customer: order.customer,
        reason,
        total: order.total
    });

    return this;
}
}

```

// Использование

```
console.log('Паттерн Наблюдатель с классами:');
```

// Создаем систему заказов

```
const orderSystem = new OrderSystem();
```

// Создаем наблюдателей

```
const emailNotifier = new EmailNotifier('admin@example.com');
```

```
const smsNotifier = new SMSNotifier('+7-999-123-45-67');
```

```
const logger = new Logger();
```

// Подписываем наблюдателей на события

```
orderSystem
    .subscribe('order_created', emailNotifier)
    .subscribe('order_status_changed', emailNotifier)
```

```
.subscribe('order_shipped', smsNotifier)
.subscribe('order_delivered', smsNotifier)
.subscribe('order_cancelled', emailNotifier)
.subscribe('*', logger); // Логируем все события

// Работа с системой
console.log('\nСоздание заказа:');
const orderId = orderSystem.createOrder('Иван Иванов', [
  { name: 'Ноутбук', price: 50000, quantity: 1 },
  { name: 'Мышь', price: 2000, quantity: 1 }
]);

console.log('\nИзменение статуса заказа:');
orderSystem.updateOrderStatus(orderId, 'processing');
orderSystem.updateOrderStatus(orderId, 'shipped');

console.log('\nДоставка заказа:');
orderSystem.updateOrderStatus(orderId, 'delivered');

console.log('\nОтмена другого заказа:');
const anotherOrderId = orderSystem.createOrder('Петр Петров', [
  { name: 'Книга', price: 500, quantity: 2 }
]);
orderSystem.cancelOrder(anotherOrderId, 'Передумал');

console.log('\nИстория событий:');
const history = orderSystem.getHistory();
console.log(`Всего событий: ${history.length}`);
history.forEach((event, index) => {
  console.log(`${index + 1}. ${event.event} в ${new
Date(event.timestamp).toLocaleTimeString()}`);
});

// Отписка наблюдателя
console.log('\nОтписка email нотификатора:');
orderSystem.unsubscribe('order_created', emailNotifier);

// Новый заказ не вызовет email уведомление
console.log('\nНовый заказ после отписки:');
orderSystem.createOrder('Анна', [{ name: 'Телефон', price: 30000, quantity: 1 }]);
```


5.3. Паттерн "Стратегия" (Strategy) с классами

```
javascript

// Интерфейс стратегии
class PaymentStrategy {
  pay(amount) {
    throw new Error('Метод pay должен быть реализован');
  }

  refund(amount) {
    throw new Error('Метод refund должен быть реализован');
  }

  getDetails() {
    throw new Error('Метод getDetails должен быть реализован');
  }
}

// Конкретные стратегии
class CreditCardPayment extends PaymentStrategy {
  constructor(cardNumber, cardHolder, expiryDate, cvv) {
    super();
    this.cardNumber = this.maskCardNumber(cardNumber);
    this.cardHolder = cardHolder;
    this.expiryDate = expiryDate;
    this.cvv = cvv;
    this.transactions = [];
  }

  maskCardNumber(number) {
    return `****-****-****-${number.slice(-4)}`;
  }

  pay(amount) {
    console.log(`Оплата картой ${this.cardNumber} на сумму ${amount}₽`);
    console.log(`Держатель карты: ${this.cardHolder}`);

    // Имитация процессинга
    const transactionId = `CC_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;

    this.transactions.push({
      id: transactionId,
      amount,
```

```

        type: 'payment',
        timestamp: new Date(),
        status: 'completed'
    });

    return {
        success: true,
        transactionId,
        message: `Оплата картой успешно завершена`
    };
}

refund(amount) {
    console.log(`Возврат на карту ${this.cardNumber} суммы ${amount}₽`);

    const transactionId = `REF_${Date.now()}_${Math.random().toString(36).substr(2,
9)}`;

    this.transactions.push({
        id: transactionId,
        amount,
        type: 'refund',
        timestamp: new Date(),
        status: 'pending'
    });

    return {
        success: true,
        transactionId,
        message: `Запрос на возврат отправлен. Обычно обрабатывается за 3-5 рабочих
дней.`
    };
}

getDetails() {
    return {
        type: 'Credit Card',
        maskedNumber: this.cardNumber,
        holder: this.cardHolder,
        expiry: this.expiryDate,
        transactionCount: this.transactions.length
    };
}
}

```

```

class PayPalPayment extends PaymentStrategy {
  constructor(email) {
    super();
    this.email = email;
    this.balance = 1000; // Имитация баланса
    this.transactions = [];
  }

  pay(amount) {
    if (this.balance < amount) {
      return {
        success: false,
        message: `Недостаточно средств на PayPal аккаунте. Баланс: $
{this.balance}₽`
      };
    }

    console.log(`Оплата через PayPal с email ${this.email} на сумму ${amount}₽`);

    this.balance -= amount;
    const transactionId = `PP_${Date.now()}_${Math.random().toString(36).substr(2,
9)}`;
    this.transactions.push({
      id: transactionId,
      amount,
      type: 'payment',
      timestamp: new Date(),
      newBalance: this.balance
    });

    return {
      success: true,
      transactionId,
      message: `Оплата через PayPal успешно завершена. Новый баланс: $
{this.balance}₽`
    };
  }

  refund(amount) {
    console.log(`Возврат на PayPal аккаунт ${this.email} суммы ${amount}₽`);

    this.balance += amount;
    const transactionId = `PP_REF_${Date.now()}_${
Math.random().toString(36).substr(2, 9)}`;

```

```

        this.transactions.push({
            id: transactionId,
            amount,
            type: 'refund',
            timestamp: new Date(),
            newBalance: this.balance
        });

        return {
            success: true,
            transactionId,
            message: `Возврат осуществлен. Новый баланс: ${this.balance}P`
        };
    }

    getDetails() {
        return {
            type: 'PayPal',
            email: this.email,
            balance: this.balance,
            transactionCount: this.transactions.length
        };
    }
}

class CryptoPayment extends PaymentStrategy {
    constructor(walletAddress, cryptoType = 'BTC') {
        super();
        this.walletAddress = this.maskAddress(walletAddress);
        this.cryptoType = cryptoType;
        this.exchangeRate = this.getExchangeRate(cryptoType);
        this.transactions = [];
    }

    maskAddress(address) {
        return `${address.slice(0, 6)}...${address.slice(-4)}`;
    }

    getExchangeRate(cryptoType) {
        const rates = {
            BTC: 2500000, // 1 BTC = 2,500,000P
            ETH: 150000, // 1 ETH = 150,000P
            USDT: 100 // 1 USDT = 100P
        };
    }
}

```

```

        return rates[cryptoType] || 1;
    }

    pay(amount) {
        const cryptoAmount = amount / this.exchangeRate;
        console.log(`Оплата ${cryptoAmount} ${this.cryptoType} на кошелек $
{this.walletAddress}`);
        console.log(`Курс: 1 ${this.cryptoType} = ${this.exchangeRate}₽`);

        const transactionId = `${this.cryptoType}_${Date.now()}_${
Math.random().toString(36).substr(2, 9)}`;
        this.transactions.push({
            id: transactionId,
            amountInRub: amount,
            amountInCrypto: cryptoAmount,
            cryptoType: this.cryptoType,
            type: 'payment',
            timestamp: new Date(),
            status: 'pending_confirmation'
        });

        return {
            success: true,
            transactionId,
            message: `Транзакция создана. Ожидание подтверждения в блокчейне. Сумма: $
{cryptoAmount.toFixed(8)} ${this.cryptoType}`
        };
    }

    refund(amount) {
        console.log(`Возврат криптовалюты не поддерживается напрямую.`);
        console.log(`Для возврата свяжитесь с поддержкой.`);

        return {
            success: false,
            message: 'Возврат криптовалюты не поддерживается. Обратитесь в поддержку.'
        };
    }

    getDetails() {
        return {
            type: 'Crypto',
            cryptoType: this.cryptoType,
            maskedAddress: this.walletAddress,

```

```

        exchangeRate: this.exchangeRate,
        transactionCount: this.transactions.length
    };
}
}

// Контекст, использующий стратегии
class PaymentProcessor {
    constructor() {
        this.strategy = null;
        this.paymentHistory = [];
    }

    // Установка стратегии
    setStrategy(strategy) {
        if (!(strategy instanceof PaymentStrategy)) {
            throw new Error('Стратегия должна наследоваться от PaymentStrategy');
        }

        console.log(`Установлена стратегия оплаты: ${strategy.constructor.name}`);
        this.strategy = strategy;
        return this;
    }

    // Выполнение оплаты
    processPayment(amount, description = '') {
        if (!this.strategy) {
            throw new Error('Стратегия оплаты не установлена');
        }

        console.log(`\nОбработка платежа: ${description}`);
        console.log(`Сумма: ${amount}P`);

        const result = this.strategy.pay(amount);

        // Сохраняем в историю
        this.paymentHistory.push({
            timestamp: new Date(),
            strategy: this.strategy.constructor.name,
            amount,
            description,
            result
        });
    }
}

```

```

        return result;
    }

    // Возврат средств
    processRefund(amount, reason = '') {
        if (!this.strategy) {
            throw new Error('Стратегия оплаты не установлена');
        }

        console.log(`\nОбработка возврата: ${reason}`);
        console.log(`Сумма: ${amount}₽`);

        const result = this.strategy.refund(amount);

        this.paymentHistory.push({
            timestamp: new Date(),
            strategy: this.strategy.constructor.name,
            amount,
            type: 'refund',
            reason,
            result
        });

        return result;
    }

    // Получение информации о текущей стратегии
    getPaymentDetails() {
        if (!this.strategy) {
            return { message: 'Стратегия не установлена' };
        }

        return this.strategy.getDetails();
    }

    // История платежей
    getPaymentHistory() {
        return [...this.paymentHistory];
    }

    // Сумма всех платежей
    getTotalProcessed() {
        return this.paymentHistory
            .filter(record => record.type !== 'refund')

```

```

        .reduce((total, record) => total + record.amount, 0);
    }
}

// Использование
console.log('Паттерн Стратегия с классами:');

// Создаем процессор платежей
const paymentProcessor = new PaymentProcessor();

// Создаем стратегии
const creditCard = new CreditCardPayment('1234567812345678', 'ИВАН ИВАНОВ', '12/25',
'123');
const paypal = new PayPalPayment('ivanov@example.com');
const bitcoin = new CryptoPayment('1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa', 'BTC');

// Оплата кредитной картой
console.log('\n--- Оплата кредитной картой ---');
paymentProcessor.setStrategy(creditCard);
let result = paymentProcessor.processPayment(5000, 'Покупка ноутбука');
console.log('Результат:', result.message);

// Возврат на карту
console.log('\n--- Возврат на карту ---');
result = paymentProcessor.processRefund(1000, 'Возврат за аксессуар');
console.log('Результат:', result.message);

// Оплата через PayPal
console.log('\n--- Оплата через PayPal ---');
paymentProcessor.setStrategy(paypal);
result = paymentProcessor.processPayment(3000, 'Покупка курса');
console.log('Результат:', result.message);

// Попытка оплаты большей суммы
console.log('\n--- Попытка оплаты большей суммы через PayPal ---');
result = paymentProcessor.processPayment(1500, 'Покупка телефона');
console.log('Результат:', result.message);

// Оплата криптовалютой
console.log('\n--- Оплата биткоинами ---');
paymentProcessor.setStrategy(bitcoin);
result = paymentProcessor.processPayment(250000, 'Покупка автомобиля');
console.log('Результат:', result.message);

```



```

// Попытка возврата криптовалюты
console.log('\n--- Попытка возврата криптовалюты ---');
result = paymentProcessor.processRefund(50000, 'Возврат за авто');
console.log('Результат:', result.message);

// Информация о текущей стратегии
console.log('\n--- Информация о текущей стратегии ---');
console.log(paymentProcessor.getPaymentDetails());

// История платежей
console.log('\n--- История платежей ---');
const history = paymentProcessor.getPaymentHistory();
console.log(`Всего операций: ${history.length}`);
console.log(`Общая сумма платежей: ${paymentProcessor.getTotalProcessed()}₽`);

history.forEach((record, index) => {
    console.log(`${index + 1}. ${record.timestamp.toLocaleTimeString()} - $
{record.strategy} - ${record.amount}₽ - ${record.description || record.reason || ''}`);
});

// Динамическая смена стратегии
console.log('\n--- Динамическая смена стратегии ---');
class GiftCardPayment extends PaymentStrategy {
    constructor(cardNumber, balance) {
        super();
        this.cardNumber = cardNumber;
        this.balance = balance;
    }

    pay(amount) {
        if (this.balance < amount) {
            return {
                success: false,
                message: `Недостаточно средств на подарочной карте. Баланс: $
{this.balance}₽`
            };
        }

        this.balance -= amount;
        return {
            success: true,
            message: `Оплата подарочной картой успешна. Остаток: ${this.balance}₽`
        };
    }
}

```

```

refund(amount) {
    // Подарочные карты обычно не поддерживают возврат
    return {
        success: false,
        message: 'Возврат на подарочную карту не поддерживается'
    };
}

getDetails() {
    return {
        type: 'Gift Card',
        cardNumber: this.cardNumber,
        balance: this.balance
    };
}
}

// Добавляем новую стратегию "на лету"
const giftCard = new GiftCardPayment('GIFT-1234-5678', 2000);
paymentProcessor.setStrategy(giftCard);
result = paymentProcessor.processPayment(1500, 'Покупка на подарочную карту');
console.log('Результат:', result.message);

```

Часть 6: Заключение и лучшие практики

6.1. Когда использовать классы, а когда конструкторы

```

javascript

// ИСПОЛЬЗУЙТЕ КЛАССЫ ES6+ когда:
// 1. Пишете новый код
// 2. Нужно наследование
// 3. Хотите использовать приватные поля
// 4. Нужны статические блоки
// 5. Важен чистый и понятный синтаксис

// Примеры правильного использования классов
class GoodUseCase1 {
    // Сложная бизнес-логика с наследованием
}

```

```

class GoodUseCase2 {
    #privateData; // Приватные поля
    static cache = new Map(); // Статические поля

    static {
        // Сложная инициализация
    }
}

class GoodUseCase3 extends EventTarget {
    // Интеграция с DOM API
}

// ИСПОЛЬЗУЙТЕ КОНСТРУКТОРЫ когда:
// 1. Работаете с Legacy кодом
// 2. Нужна обратная совместимость со старыми браузерами
// 3. Требуется динамическое создание методов
// 4. Работаете с библиотеками, использующими конструкторы

function LegacyCompatibility() {
    // Поддержка IE11 и старых Android
}

// ИСПОЛЬЗУЙТЕ ФАБРИЧНЫЕ ФУНКЦИИ когда:
// 1. Нужен больший контроль над созданием объектов
// 2. Требуется кэширование или пул объектов
// 3. Создание объектов зависит от сложных условий
// 4. Хотите скрыть реализацию

function createUser(config) {
    const instance = Object.create(userPrototype);
    // Сложная логика инициализации
    return instance;
}

```

6.2. Лучшие практики работы с классами

```

javascript

// 1. Один класс - одна ответственность
class User { // Хорошо: управление пользователями
    constructor(name, email) {
        this.name = name;
        this.email = email;
    }
}

```

```

    }

    validate() {
        // Валидация данных пользователя
    }

    save() {
        // Сохранение пользователя
    }
}

// ПЛОХО:
class GodObject { // Плохо: делает всё
    constructor() {
        // Управление пользователями, заказами, платежами, логированием...
    }
}

// 2. Используйте композицию вместо наследования
class Engine {
    start() {
        console.log('Двигатель заведен');
    }
}

class Wheels {
    rotate() {
        console.log('Колеса крутятся');
    }
}

class Car {
    constructor() {
        this.engine = new Engine();
        this.wheels = new Wheels();
    }

    drive() {
        this.engine.start();
        this.wheels.rotate();
        console.log('Машина едет');
    }
}

```

// 3. Приватные поля для инкапсуляции

```
class BankAccount {
  #balance = 0;
  #transactionHistory = [];

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      this.#logTransaction('deposit', amount);
    }
  }

  #logTransaction(type, amount) {
    this.#transactionHistory.push({
      type,
      amount,
      timestamp: new Date(),
      balance: this.#balance
    });
  }

  getBalance() {
    return this.#balance;
  }

  getStatement() {
    return [...this.#transactionHistory];
  }
}
```

// 4. Статические методы для утилитарных функций

```
class MathUtils {
  static sum(...numbers) {
    return numbers.reduce((a, b) => a + b, 0);
  }

  static average(...numbers) {
    if (numbers.length === 0) return 0;
    return this.sum(...numbers) / numbers.length;
  }

  static clamp(value, min, max) {
    return Math.min(Math.max(value, min), max);
  }
}
```

```
}
```

```
// 5. Геттеры и сеттеры для контроля доступа
```

```
class Temperature {  
    constructor(celsius) {  
        this._celsius = celsius;  
    }  
  
    get celsius() {  
        return this._celsius;  
    }  
  
    set celsius(value) {  
        if (value < -273.15) {  
            throw new Error('Температура не может быть ниже абсолютного нуля');  
        }  
        this._celsius = value;  
    }  
  
    get fahrenheit() {  
        return this._celsius * 9/5 + 32;  
    }  
  
    set fahrenheit(value) {  
        this.celsius = (value - 32) * 5/9;  
    }  
}
```

```
// 6. Документируйте публичный API с JSDoc
```

```
/**  
 * Класс для управления заказами  
 * @class OrderManager  
 * @property {string} id - Уникальный идентификатор заказа  
 * @property {Date} createdAt - Дата создания заказа  
 */  
class OrderManager {  
    /**  
     * Создает новый заказ  
     * @constructor  
     * @param {Object} customer - Данные покупателя  
     * @param {Array} items - Товары в заказе  
     */  
    constructor(customer, items) {  
        this.id = this.generateId();  
    }  
}
```

```

        this.customer = customer;
        this.items = items;
        this.createdAt = new Date();
        this.status = 'pending';
    }

    /**
     * Обновляет статус заказа
     * @param {string} newStatus - Новый статус
     * @returns {boolean} Успешно ли обновлен статус
     */
    updateStatus(newStatus) {
        const validStatuses = ['pending', 'processing', 'shipped', 'delivered',
'cancelled'];
        if (validStatuses.includes(newStatus)) {
            this.status = newStatus;
            return true;
        }
        return false;
    }

    /**
     * Генерирует уникальный ID заказа
     * @private
     * @returns {string}
     */
    generateId() {
        return `ORDER_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
    }
}

```

6.3. Антипаттерны с классами

javascript

```

// 1. Излишнее наследование
// ПЛОХО: глубокие иерархии
class Animal {}
class Mammal extends Animal {}
class Dog extends Mammal {}
class DomesticDog extends Dog {}
class Breed extends DomesticDog {}
class MyDog extends Breed {} // Слишком глубоко!

```

// ЛУЧШЕ: композиция

```
class Animal {  
    constructor(traits) {  
        this.traits = traits;  
    }  
}
```

```
const myDog = new Animal({  
    type: 'mammal',  
    species: 'dog',  
    domestic: true,  
    breed: 'labrador',  
    name: 'Бобик'  
});
```

// 2. Классы-божества (God Class)

// ПЛОХО: один класс делает всё

```
class Application {  
    constructor() {  
        // UI, бизнес-логика, работа с API, база данных, кэширование...  
    }  
}
```

// ЛУЧШЕ: разделение ответственности

```
class UIManager {}  
class BusinessLogic {}  
class APIClient {}  
class Database {}  
class CacheManager {}
```

// 3. Нарушение инкапсуляции

// ПЛОХО: публичные поля, которые должны быть приватными

```
class User {  
    constructor() {  
        this.password = 'secret'; // ОПАСНО!  
        this.ssn = '123-45-6789'; // ОПАСНО!  
    }  
}
```

// ЛУЧШЕ: приватные поля

```
class SecureUser {  
    #password;  
    #ssn;
```



```

    constructor(password, ssn) {
        this.#password = password;
        this.#ssn = ssn;
    }

    validatePassword(input) {
        return input === this.#password;
    }
}

// 4. Статические методы там, где нужны экземплярные
// ПЛОХО: статический метод, использующий данные экземпляра
class Config {
    static getValue(key) {
        return this.values[key]; // this зависит от контекста вызова
    }
}

// ЛУЧШЕ: экземплярный метод или статический метод с параметром
class BetterConfig {
    constructor(values) {
        this.values = values;
    }

    getValue(key) {
        return this.values[key];
    }
}

// ИЛИ:
class StaticConfig {
    static getValue(config, key) {
        return config[key];
    }
}

// 5. Использование классов для всего
// Не всё должно быть классом!

// ПЛОХО: класс для простой утилиты
class StringFormatter {
    static capitalize(str) {
        return str.charAt(0).toUpperCase() + str.slice(1);
    }
}

```

```

}

// ЛУЧШЕ: простая функция
function capitalize(str) {
    return str.charAt(0).toUpperCase() + str.slice(1);
}

// ПЛОХО: класс для DTO (Data Transfer Object)
class UserDTO {
    constructor(data) {
        this.name = data.name;
        this.email = data.email;
    }
}

// ЛУЧШЕ: обычный объект или TypeScript interface
const userDTO = {
    name: 'Иван',
    email: 'ivan@example.com'
};

```

6.4. Чек-лист при работе с классами

Перед созданием класса спросите себя:

- ☐ **Нужен ли класс вообще?** Может, достаточно объекта или функции?
- ☐ **Одна ли ответственность у класса?** Не делает ли он слишком много?
- ☐ **Нужно ли наследование?** Или лучше композиция?
- ☐ **Правильно ли используются приватные поля?** Все внутренние детали скрыты?
- ☐ **Достаточно ли документирован публичный API?**
- ☐ **Не нарушает ли класс принципы SOLID?**
- ☐ **Используются ли геттеры/сеттеры для контроля доступа?**
- ☐ **Статические методы действительно должны быть статическими?**
- ☐ **Не слишком ли глубокая иерархия наследования?** (максимум 2-3 уровня)
- ☐ **Тестируется ли класс легко?** Не слишком ли он связан с другими классами?

Заключение

Классы в ES6 — это действительно **синтаксический сахар** над существующими в JavaScript механизмами прототипного наследования, но это очень полезный и мощный синтаксис, который:

Преимущества классов:

1. **Более чистый и понятный синтаксис** — похож на другие ООП языки
2. **Встроенная проверка `new`** — автоматическая защита от вызова без `new`
3. **Улучшенное наследование** — простой синтаксис с `extends` и `super`
4. **Приватные поля и методы** — настоящая инкапсуляция (ES2022+)
5. **Статические блоки** — сложная инициализация класса (ES2022)
6. **Неперечисляемые методы** — методы не появляются в `for...in`
7. **Строгий режим по умолчанию** — меньше случайных ошибок

Что важно помнить:

1. **Классы компилируются в конструкторы** — понимайте, что происходит под капотом
2. **Методы попадают в прототип** — эффективное использование памяти
3. **Наследование через `__proto__`** — цепочки прототипов работают как всегда
4. **Классы нельзя вызывать без `new`** — в отличие от конструкторов
5. **Приватные поля начинаются с `#`** — специальный синтаксис

Рекомендации:

- **Используйте классы для нового кода** — они стали стандартом
- **Изучите, как они работают под капотом** — для эффективной отладки
- **Не злоупотребляйте наследованием** — композиция часто лучше
- **Используйте приватные поля для инкапсуляции** — это хорошая практика
- **Документируйте публичный API** — особенно в больших проектах

Классы сделали JavaScript более выразительным и удобным для объектно-ориентированного программирования, сохранив при этом всю гибкость и мощь прототипной системы языка.

Введение: принципы повторного использования и гибкости

Наследование и полиморфизм — два ключевых принципа объектно-ориентированного программирования, которые позволяют создавать гибкие, расширяемые и поддерживаемые системы. В JavaScript эти концепции реализованы через **прототипное наследование** и **динамическую диспетчеризацию методов**.

Часть 1: Наследование в JavaScript

1.1. Концептуальная основа наследования

Наследование — это механизм, позволяющий одному классу (прототипу) заимствовать свойства и методы другого класса. Это создает иерархию "родитель-ребенок", где дочерние классы могут:

1. **Переиспользовать** код родительского класса
2. **Расширять** функциональность родителя
3. **Специализировать** поведение для конкретных случаев

1.2. Наследование через классы (ES6+)

Базовый синтаксис:

```
javascript

// Родительский (базовый) класс
class Animal {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}
```

```

        this.energy = 100;
    }

    eat() {
        this.energy += 20;
        console.log(`${this.name} ест. Энергия: ${this.energy}`);
    }

    sleep() {
        this.energy += 50;
        console.log(`${this.name} спит. Энергия: ${this.energy}`);
    }

    move() {
        if (this.energy >= 10) {
            this.energy -= 10;
            console.log(`${this.name} двигается. Энергия: ${this.energy}`);
        } else {
            console.log(`${this.name} слишком устал для движения`);
        }
    }
}

// Дочерний класс
class Dog extends Animal {
    constructor(name, age, breed) {
        // Вызов конструктора родительского класса
        super(name, age);
        this.breed = breed;
        this.loyalty = 100;
    }

    // Добавление нового метода
    bark() {
        console.log(`${this.name} лает: Гав-гав!`);
    }

    // Переопределение родительского метода
    eat() {
        super.eat(); // Вызов родительского метода
        this.loyalty += 5;
        console.log(`${this.name} стал более верным. Верность: ${this.loyalty}`);
    }
}

```

```

// Специализированное поведение
fetch() {
    if (this.energy >= 30) {
        this.energy -= 30;
        this.loyalty += 10;
        console.log(`${this.name} приносит палку! Верность: ${this.loyalty}`);
    } else {
        console.log(`${this.name} слишком устал для игры`);
    }
}
}

// Еще один дочерний класс
class Cat extends Animal {
    constructor(name, age, color) {
        super(name, age);
        this.color = color;
        this.independence = 100;
    }

    // Полностью новое поведение
    purr() {
        console.log(`${this.name} мурлычет: Mpppp...`);
    }

    // Альтернативное переопределение
    move() {
        if (Math.random() > 0.5) { // Коты двигаются когда хотят
            super.move();
        } else {
            console.log(`${this.name} игнорирует команду и умывается`);
        }
    }

    climb() {
        this.energy -= 20;
        console.log(`${this.name} залезает на дерево`);
    }
}

```

1.3. Практическое использование

javascript

```

// Создание экземпляров
const dog = new Dog("Бобик", 3, "Лабрадор");
const cat = new Cat("Мурка", 2, "Рыжий");

// Общие методы из родительского класса
dog.eat();    // Бобик ест. Энергия: 120
              // Бобик стал более верным. Верность: 105
cat.eat();    // Мурка ест. Энергия: 120

// Специфические методы
dog.bark();   // Бобик лает: Гав-гав!
dog.fetch();  // Бобик приносит палку! Верность: 115

cat.purr();   // Мурка мурлычет: Мрррр...
cat.climb();  // Мурка залезает на дерево

// Переопределенные методы ведут себя по-разному
dog.move();   // Бобик двигается. Энергия: 105
cat.move();   // Может двигаться, а может игнорировать

```

1.4. Цепочка наследования

javascript

```

// Многоуровневое наследование
class Mammal extends Animal {
  constructor(name, age) {
    super(name, age);
    this.bodyTemperature = 37;
  }

  regulateTemperature() {
    console.log(`${this.name} регулирует температуру тела`);
  }
}

class Horse extends Mammal {
  constructor(name, age, speed) {
    super(name, age);
    this.speed = speed;
  }

  gallop() {
    this.energy -= 40;
  }
}

```

```

        console.log(`${this.name} скачет со скоростью ${this.speed} км/ч`);
    }
}

class RaceHorse extends Horse {
    constructor(name, age, speed, wins) {
        super(name, age, speed);
        this.wins = wins;
        this.training = 100;
    }

    train() {
        this.training += 10;
        this.speed += 5;
        console.log(`${this.name} тренируется. Скорость: ${this.speed}`);
    }

    // Доступ ко всем уровням наследования
    fullActivity() {
        this.eat(); // Из Animal
        this.regulateTemperature(); // Из Mammal
        this.gallop(); // Из Horse
        this.train(); // Из RaceHorse
    }
}

const champion = new RaceHorse("Молния", 5, 60, 10);
champion.fullActivity();
// Молния ест. Энергия: 120
// Молния регулирует температуру тела
// Молния скачет со скоростью 60 км/ч
// Молния тренируется. Скорость: 65

```

Часть 2: Полиморфизм в JavaScript

2.1. Концепция полиморфизма

Полиморфизм (буквально "много форм") — это способность объектов разных типов отвечать на одинаковые сообщения (вызовы методов) своим уникальным способом.

2.2. Типы полиморфизма в JavaScript

2.2.1. Полиморфизм подтипов (наследования)

javascript

```
class Shape {
  constructor(color) {
    this.color = color;
  }

  // Абстрактный метод (в JavaScript нет настоящих абстрактных методов)
  getArea() {
    throw new Error("Метод getArea должен быть реализован");
  }

  describe() {
    return `Фигура цвета ${this.color}`;
  }
}

class Circle extends Shape {
  constructor(color, radius) {
    super(color);
    this.radius = radius;
  }

  // Полиморфное поведение
  getArea() {
    return Math.PI * this.radius ** 2;
  }

  describe() {
    return `${super.describe()}, Круг радиусом ${this.radius}`;
  }
}

class Rectangle extends Shape {
  constructor(color, width, height) {
    super(color);
    this.width = width;
    this.height = height;
  }
}
```

```

// То же имя метода, разная реализация
getArea() {
    return this.width * this.height;
}

describe() {
    return `${super.describe()}, Прямоугольник ${this.width}x${this.height}`;
}
}

class Triangle extends Shape {
    constructor(color, base, height) {
        super(color);
        this.base = base;
        this.height = height;
    }

    // Еще одна реализация того же метода
    getArea() {
        return 0.5 * this.base * this.height;
    }

    describe() {
        return `${super.describe()}, Треугольник с основанием ${this.base}`;
    }
}

```

2.3. Использование полиморфизма

```

javascript

// Массив разных фигур
const shapes = [
    new Circle("красный", 5),
    new Rectangle("синий", 4, 6),
    new Triangle("зеленый", 3, 8),
    new Circle("желтый", 3),
    new Rectangle("фиолетовый", 5, 5)
];

// Полиморфный вызов методов
function processShapes(shapesArray) {
    let totalArea = 0;

```

```

shapesArray.forEach((shape, index) => {
    // Один интерфейс - много реализаций
    const area = shape.getArea();
    const description = shape.describe();

    totalArea += area;
    console.log(`${index + 1}. ${description}, Площадь: ${area.toFixed(2)}`);
});

console.log(`Общая площадь всех фигур: ${totalArea.toFixed(2)}`);
}

processShapes(shapes);
// 1. Фигура цвета красный, Круг радиусом 5, Площадь: 78.54
// 2. Фигура цвета синий, Прямоугольник 4x6, Площадь: 24.00
// 3. Фигура цвета зеленый, Треугольник с основанием 3, Площадь: 12.00
// 4. Фигура цвета желтый, Круг радиусом 3, Площадь: 28.27
// 5. Фигура цвета фиолетовый, Прямоугольник 5x5, Площадь: 25.00
// Общая площадь всех фигур: 167.81

```

2.4. Утиная типизация (Duck Typing)

JavaScript использует **утиную типизацию**: "Если что-то ходит как утка и крякает как утка, то это утка".

```

javascript

// Разные классы с одинаковым интерфейсом
class Logger {
    log(message) {
        console.log(`[LOG] ${message}`);
    }
}

class EmailSender {
    log(message) {
        console.log(`[EMAIL] Отправлено: ${message}`);
    }
}

class Database {
    log(message) {
        console.log(`[DB] Записано: ${message}`);
    }
}

```

```

    }
}

// Функция, которая работает с любым объектом, имеющим метод Log
function logToAllServices(services, message) {
    services.forEach(service => {
        // Неважно какой это класс, важно что у него есть метод Log
        service.log(message);
    });
}

const services = [
    new Logger(),
    new EmailSender(),
    new Database()
];

logToAllServices(services, "Пользователь вошел в систему");
// [LOG] Пользователь вошел в систему
// [EMAIL] Отправлено: Пользователь вошел в систему
// [DB] Записано: Пользователь вошел в систему

```

Часть 3: Продвинутые техники

3.1. Композиция vs Наследование

```

javascript

// Иногда композиция лучше наследования
class Engine {
    constructor(power) {
        this.power = power;
        this.isRunning = false;
    }

    start() {
        this.isRunning = true;
        console.log(`Двигатель ${this.power} л.с. запущен`);
    }

    stop() {
        this.isRunning = false;
    }
}

```

```

        console.log("Двигатель остановлен");
    }
}

class Wheels {
    constructor(count, diameter) {
        this.count = count;
        this.diameter = diameter;
        this.pressure = 2.5;
    }

    checkPressure() {
        console.log(`Давление в шинах: ${this.pressure} атм`);
    }
}

class Car {
    constructor(brand, model, enginePower, wheelDiameter) {
        this.brand = brand;
        this.model = model;
        // Композиция: содержит другие объекты
        this.engine = new Engine(enginePower);
        this.wheels = new Wheels(4, wheelDiameter);
    }

    start() {
        this.engine.start();
        this.wheels.checkPressure();
        console.log(`${this.brand} ${this.model} готов к поездке`);
    }

    drive() {
        if (this.engine.isRunning) {
            console.log(`${this.brand} ${this.model} едет`);
        } else {
            console.log("Сначала запустите двигатель");
        }
    }
}

const myCar = new Car("Toyota", "Camry", 200, 17);
myCar.start();
// Двигатель 200 л.с. запущен
// Давление в шинах: 2.5 атм

```

```
// Toyota Camry готов к поездке
myCar.drive(); // Toyota Camry едет
```

3.2. Миксины (Mixins)

javascript

```
// Миксины для гибкого повторного использования кода
```

```
const CanSwim = {
  swim() {
    console.log(`${this.name} плавает`);
  }
};

const CanFly = {
  fly() {
    console.log(`${this.name} летит`);
  }
};

const CanWalk = {
  walk() {
    console.log(`${this.name} идет`);
  }
};

class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Duck extends Animal {
  constructor(name) {
    super(name);
    Object.assign(this, CanWalk, CanSwim, CanFly);
  }
}

class Fish extends Animal {
  constructor(name) {
    super(name);
    Object.assign(this, CanSwim);
  }
}
```

```

}

class Eagle extends Animal {
    constructor(name) {
        super(name);
        Object.assign(this, CanWalk, CanFly);
    }
}

const donald = new Duck("Дональд");
const nemo = new Fish("Немо");
const sam = new Eagle("Сэм");

donald.swim(); // Дональд плавает
donald.fly();  // Дональд летит
donald.walk(); // Дональд идет

nemo.swim();   // Немо плавает
// nemo.fly(); // Ошибка: у рыб нет метода fly

sam.fly();     // Сэм летит
sam.walk();    // Сэм идет

```

3.3. Фабричные методы и полиморфизм

```

javascript

class PaymentMethod {
    process(amount) {
        throw new Error("Метод process должен быть реализован");
    }
}

class CreditCard extends PaymentMethod {
    process(amount) {
        console.log(`Обработка кредитной карты на сумму ${amount}`);
        // Логика обработки кредитной карты
        return true;
    }
}

class PayPal extends PaymentMethod {
    process(amount) {
        console.log(`Обработка PayPal на сумму ${amount}`);
    }
}

```

```

        // Логика PayPal
        return true;
    }
}

class BankTransfer extends PaymentMethod {
    process(amount) {
        console.log(`Банковский перевод на сумму ${amount}`);
        // Логика банковского перевода
        return true;
    }
}

// Фабрика для создания платежных методов
class PaymentFactory {
    static createPaymentMethod(type) {
        switch (type.toLowerCase()) {
            case 'creditcard':
                return new CreditCard();
            case 'paypal':
                return new PayPal();
            case 'banktransfer':
                return new BankTransfer();
            default:
                throw new Error(`Неизвестный метод оплаты: ${type}`);
        }
    }
}

// Полиморфная обработка платежей
function processOrder(amount, paymentType) {
    const paymentMethod = PaymentFactory.createPaymentMethod(paymentType);

    // Единый интерфейс для всех методов оплаты
    const success = paymentMethod.process(amount);

    if (success) {
        console.log("Платеж успешно обработан");
        return true;
    }
    return false;
}

// Использование

```



```
processOrder(100, 'creditcard');
processOrder(200, 'paypal');
processOrder(300, 'banktransfer');
```

Часть 4: Практические паттерны

4.1. Стратегия (Strategy Pattern)

```
javascript

class SortStrategy {
  sort(data) {
    throw new Error("Метод sort должен быть реализован");
  }
}

class BubbleSort extends SortStrategy {
  sort(data) {
    console.log("Сортировка пузырьком");
    // Реализация пузырьковой сортировки
    return [...data].sort((a, b) => a - b);
  }
}

class QuickSort extends SortStrategy {
  sort(data) {
    console.log("Быстрая сортировка");
    // Реализация быстрой сортировки
    return this.quickSort([...data]);
  }

  quickSort(arr) {
    if (arr.length <= 1) return arr;

    const pivot = arr[Math.floor(arr.length / 2)];
    const left = arr.filter(x => x < pivot);
    const middle = arr.filter(x => x === pivot);
    const right = arr.filter(x => x > pivot);

    return [...this.quickSort(left), ...middle, ...this.quickSort(right)];
  }
}
```

```

class MergeSort extends SortStrategy {
  sort(data) {
    console.log("Сортировка слиянием");
    // Реализация сортировки слиянием
    return this.mergeSort([...data]);
  }

  mergeSort(arr) {
    if (arr.length <= 1) return arr;

    const mid = Math.floor(arr.length / 2);
    const left = this.mergeSort(arr.slice(0, mid));
    const right = this.mergeSort(arr.slice(mid));

    return this.merge(left, right);
  }

  merge(left, right) {
    const result = [];
    let i = 0, j = 0;

    while (i < left.length && j < right.length) {
      if (left[i] < right[j]) {
        result.push(left[i++]);
      } else {
        result.push(right[j++]);
      }
    }

    return result.concat(left.slice(i)).concat(right.slice(j));
  }
}

class DataProcessor {
  constructor(strategy) {
    this.strategy = strategy;
  }

  setStrategy(strategy) {
    this.strategy = strategy;
  }

  process(data) {

```

```

        return this.strategy.sort(data);
    }
}

// Использование
const data = [64, 34, 25, 12, 22, 11, 90];
const processor = new DataProcessor(new BubbleSort());

console.log("Сортировка пузырьком:", processor.process(data));

processor.setStrategy(new QuickSort());
console.log("Быстрая сортировка:", processor.process(data));

processor.setStrategy(new MergeSort());
console.log("Сортировка слиянием:", processor.process(data));

```

4.2. Наблюдатель (Observer Pattern)

```

javascript

class Observer {
    update(data) {
        throw new Error("Метод update должен быть реализован");
    }
}

class EmailNotifier extends Observer {
    update(user) {
        console.log(`Отправка email пользователю ${user.email}: Ваш профиль обновлен`);
    }
}

class SMSNotifier extends Observer {
    update(user) {
        console.log(`Отправка SMS на ${user.phone}: Ваш профиль был изменен`);
    }
}

class PushNotifier extends Observer {
    update(user) {
        console.log(`Push-уведомление для ${user.name}: Обновление профиля`);
    }
}

```

```

class User {
  constructor(name, email, phone) {
    this.name = name;
    this.email = email;
    this.phone = phone;
    this.observers = [];
  }

  attach(observer) {
    this.observers.push(observer);
  }

  detach(observer) {
    this.observers = this.observers.filter(obs => obs !== observer);
  }

  notify() {
    this.observers.forEach(observer => {
      observer.update(this);
    });
  }

  updateProfile(newData) {
    Object.assign(this, newData);
    console.log(`Профиль ${this.name} обновлен`);
    this.notify();
  }
}

```

// Использование

```
const user = new User("Иван", "ivan@example.com", "+79161234567");
```

```
const emailNotifier = new EmailNotifier();
```

```
const smsNotifier = new SMSNotifier();
```

```
const pushNotifier = new PushNotifier();
```

```
user.attach(emailNotifier);
```

```
user.attach(smsNotifier);
```

```
user.attach(pushNotifier);
```

```
user.updateProfile({ name: "Иван Иванов", phone: "+79169876543" });
```

// Профиль Иван Иванов обновлен

// Отправка email пользователю ivan@example.com: Ваш профиль обновлен

// Отправка SMS на +79169876543: Ваш профиль был изменен

// Push-уведомление для Иван Иванов: Обновление профиля

Часть 5: Лучшие практики и антипаттерны

5.1. Что делать

javascript

// 1. Используйте композицию вместо глубокого наследования

```
class Bird {
  constructor(name) {
    this.name = name;
  }

  speak() {
    return `${this.name} издает звук`;
  }
}
```

// Вместо длинной цепочки наследования

```
class FlyingBird extends Bird {
  fly() {
    return `${this.name} летит`;
  }
}
```

// Используйте миксины

```
const flyingMixin = {
  fly() {
    return `${this.name} летит`;
  }
};
```

```
class BetterBird {
  constructor(name) {
    this.name = name;
  }
}
```

```
Object.assign(BetterBird.prototype, flyingMixin);
```

// 2. Соблюдайте принцип подстановки Лисков (LSP)

```

class Rectangle {
    constructor(width, height) {
        this.width = width;
        this.height = height;
    }

    setWidth(width) {
        this.width = width;
    }

    setHeight(height) {
        this.height = height;
    }

    getArea() {
        return this.width * this.height;
    }
}

class Square extends Rectangle {
    constructor(size) {
        super(size, size);
    }

    // Нарушение LSP - квадрат не может менять ширину и высоту независимо
    setWidth(width) {
        super.setWidth(width);
        super.setHeight(width); // Вынужденное поведение
    }

    setHeight(height) {
        super.setHeight(height);
        super.setWidth(height); // Вынужденное поведение
    }
}

// 3. Используйте интерфейсное программирование
class Database {
    connect() {
        throw new Error("Метод connect должен быть реализован");
    }

    query(sql) {
        throw new Error("Метод query должен быть реализован");
    }
}

```

```

    }

    disconnect() {
        throw new Error("Метод disconnect должен быть реализован");
    }
}

class MySQLDatabase extends Database {
    connect() {
        console.log("Подключение к MySQL");
        return this;
    }

    query(sql) {
        console.log(`Выполнение MySQL запроса: ${sql}`);
        return [];
    }

    disconnect() {
        console.log("Отключение от MySQL");
    }
}

```

5.2. Чего избегать

```

javascript

// 1. Избегайте хрупкого базового класса
class FragileBase {
    constructor() {
        this.setup(); // Опасный вызов в конструкторе
    }

    setup() {
        // Базовая реализация
    }
}

class Child extends FragileBase {
    setup() {
        // Если дочерний класс переопределит setup,
        // он может поломаться при изменении базового класса
        super.setup();
        this.extraSetup();
    }
}

```

```

    }

    extraSetup() {
        // Дополнительная настройка
    }
}

// 2. Избегайте нарушения инкапсуляции
class BankAccount {
    constructor(balance) {
        this._balance = balance; // Приватное поле
    }

    // Предоставляйте контролируемый доступ
    getBalance() {
        return this._balance;
    }

    deposit(amount) {
        if (amount > 0) {
            this._balance += amount;
        }
    }

    // Не делайте так:
    // get balance() { return this._balance; } // Нарушение инкапсуляции
}

// 3. Избегайте неправильного использования наследования
// ПЛОХО: Собака НЕ является подтипом человека
class Person {
    walk() {
        console.log("Человек идет");
    }
}

class Dog extends Person { // × Логическая ошибка
    bark() {
        console.log("Собака лает");
    }
}

// ХОРОШО: Используйте композицию
class GoodDog {

```



```

    constructor() {
        this.movement = new Movement();
    }

    bark() {
        console.log("Собака лает");
    }

    walk() {
        this.movement.walk();
    }
}

```

Часть 6: Тестирование полиморфного кода

javascript

// Моки и стабы для тестирования полиморфного поведения

```

class MockPayment extends PaymentMethod {
    constructor() {
        super();
        this.calls = [];
    }

    process(amount) {
        this.calls.push({ amount, timestamp: Date.now() });
        return true;
    }

    getCalls() {
        return this.calls;
    }
}

```

// Тестирование с использованием полиморфизма

```

function testPaymentSystem() {
    const mockPayment = new MockPayment();
    const processor = new PaymentProcessor(mockPayment);

    // Тест 1: Обработка платежа
    processor.processPayment(100);
    const calls = mockPayment.getCalls();

    if (calls.length === 1 && calls[0].amount === 100) {

```

```

        console.log("✅ Тест пройден: платеж корректно обработан");
    } else {
        console.log("❌ Тест не пройден: ошибка обработки платежа");
    }
}

// Тест 2: Множественные платежи
processor.processPayment(200);
processor.processPayment(300);

if (mockPayment.getCalls().length === 3) {
    console.log("✅ Тест пройден: множественные платежи обработаны");
} else {
    console.log("❌ Тест не пройден: ошибка множественных платежей");
}
}

// Фабрика для тестирования
class TestAnimalFactory {
    static createTestAnimal(type) {
        const animals = {
            'dog': new Dog("Тестовый пес", 1, "Дворняга"),
            'cat': new Cat("Тестовый кот", 2, "Серый"),
            'bird': new Bird("Тестовая птица")
        };

        return animals[type] || null;
    }
}

// Юнит-тесты для полиморфных методов
function runAnimalTests() {
    const testCases = [
        {
            type: 'dog',
            test: (animal) => animal.bark() !== undefined,
            message: "Собака должна уметь лаять"
        },
        {
            type: 'cat',
            test: (animal) => animal.purr() !== undefined,
            message: "Кот должен уметь мурлыкать"
        },
        {
            type: 'bird',

```

```

        test: (animal) => animal.speak() !== undefined,
        message: "Птица должна уметь издавать звуки"
    }
];

testCases.forEach(({ type, test, message }) => {
    const animal = TestAnimalFactory.createTestAnimal(type);
    if (animal && test(animal)) {
        console.log(`✅ ${message}`);
    } else {
        console.log(`❌ ${message}`);
    }
});
}

```

Заключение: баланс и практическое применение

Ключевые принципы:

1. Используйте наследование для отношений "является" (is-a)

- ➔ Собака является животным
- ➔ Круг является фигурой
- ➔ Менеджер является сотрудником

2. Используйте композицию для отношений "имеет" (has-a)

- ➔ Автомобиль имеет двигатель
- ➔ Компьютер имеет процессор
- ➔ Библиотека имеет книги

3. Применяйте полиморфизм для единого интерфейса

- ➔ Все фигуры имеют метод `getArea()`
- ➔ Все платежные методы имеют метод `process()`
- ➔ Все уведомители имеют метод `send()`

4. Следуйте SOLID принципам:

- ➔ **Single Responsibility**: один класс — одна ответственность
- ➔ **Open/Closed**: открыт для расширения, закрыт для изменений
- ➔ **Liskov Substitution**: подтипы должны заменять базовые типы
- ➔ **Interface Segregation**: много специализированных интерфейсов лучше одного общего
- ➔ **Dependency Inversion**: зависимости от абстракций, а не от конкретных классов

Практические рекомендации для JavaScript:

1. **Используйте классы ES6+** для ясности и читаемости
2. **Комбинируйте наследование и миксины** для гибкости
3. **Тестируйте полиморфное поведение** с помощью моков и стабов
4. **Документируйте ожидаемые интерфейсы** с помощью JSDoc
5. **Избегайте глубоких иерархий наследования** (максимум 3-4 уровня)

Наследование и полиморфизм в JavaScript, несмотря на свою прототипную природу, предоставляют мощные инструменты для создания гибких, расширяемых и поддерживаемых приложений. Правильное применение этих принципов позволяет писать код, который легко понимать, тестировать и модифицировать по мере роста и изменения требований к проекту.

8.8. Инкапсуляция и скрытие данных

Введение: философия инкапсуляции

Инкапсуляция — фундаментальный принцип объектно-ориентированного программирования, который обеспечивает:

1. **Соккрытие внутренней реализации** — пользователь объекта не должен знать, как он работает внутри
2. **Защиту данных** — внутреннее состояние объекта защищено от некорректного доступа
3. **Управление доступом** — контроль над тем, кто и как может взаимодействовать с объектом
4. **Согласованность состояния** — гарантия, что объект всегда находится в валидном состоянии

В JavaScript инкапсуляция имеет уникальные особенности, связанные с динамической природой языка и его прототипной моделью.

Часть 1: Уровни инкапсуляции в JavaScript

1.1. Эволюция инкапсуляции в JavaScript

```
javascript

// JavaScript 1.0-5 (до ES6) - практически полное отсутствие инкапсуляции
function OldStyleUser(name, age) {
    this.name = name;    // Публичное свойство
    this.age = age;      // Публичное свойство
    var secret = "confidential"; // "Приватная" переменная (замыкание)

    this.getSecret = function() {
        return secret;    // Доступ через метод
    };
}

const user1 = new OldStyleUser("John", 30);
```

```

console.log(user1.name);           // "John" - публичный доступ
console.log(user1.secret);         // undefined - недоступно напрямую
console.log(user1.getSecret());    // "confidential" - доступ через метод

// ES6+ - современные подходы к инкапсуляции
class ModernUser {
  #secret; // Приватное поле (ES2022)

  constructor(name, age) {
    this.name = name; // Публичное свойство
    this._age = age;  // Защищенное свойство (соглашение)
    this.#secret = "confidential"; // Настоящее приватное поле
  }
}

```

1.2. Уровни видимости в JavaScript

javascript

```

class VisibilityLevels {
  // 1. Публичные (public) свойства и методы
  publicProperty = "Доступно везде";

  publicMethod() {
    return "Публичный метод";
  }

  // 2. Защищенные (protected) - соглашение, а не синтаксис
  _protectedProperty = "Доступно в классе и наследниках (соглашение)";

  _protectedMethod() {
    return "Защищенный метод (соглашение)";
  }

  // 3. Приватные (private) поля (ES2022+)
  #privateField = "Только внутри класса";

  #privateMethod() {
    return "Приватный метод";
  }

  // 4. Статические приватные
  static #staticPrivate = "Приватное статическое поле";
}

```

```

// Метод для демонстрации доступа
demonstrateAccess() {
    console.log(this.publicProperty);    // ✓ Доступно
    console.log(this._protectedProperty); // ✓ Доступно
    console.log(this.#privateField);     // ✓ Доступно
    console.log(this.#privateMethod());  // ✓ Доступно
    console.log(VisibilityLevels.#staticPrivate); // ✓ Доступно
}
}

const instance = new VisibilityLevels();
console.log(instance.publicProperty);    // ✓ Доступно
console.log(instance.publicMethod());    // ✓ Доступно
console.log(instance._protectedProperty); // ⚠ Доступно (но не следует)
// console.log(instance.#privateField);  // ✗ SyntaxError
// console.log(instance.#privateMethod()); // ✗ SyntaxError

```

Часть 2: Механизмы инкапсуляции

2.1. Замыкания (Closures) - классический подход

```

javascript

// Паттерн "Модуль" с использованием замыканий
function createBankAccount(initialBalance = 0) {
    // Приватные переменные через замыкание
    let balance = initialBalance;
    let transactionHistory = [];

    // Генератор уникальных ID транзакций
    function generateTransactionId() {
        return `TX${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
    }

    // Приватная функция валидации
    function isValidAmount(amount) {
        return typeof amount === 'number' &&
            amount > 0 &&
            amount <= Number.MAX_SAFE_INTEGER;
    }

    // Приватная функция логирования

```

```

function logTransaction(type, amount, newBalance) {
    const transaction = {
        id: generateTransactionId(),
        timestamp: new Date().toISOString(),
        type,
        amount,
        balance: newBalance
    };
    transactionHistory.push(transaction);
    return transaction;
}

// Публичный интерфейс
return {
    // Метод для внесения средств
    deposit: function(amount) {
        if (!isValidAmount(amount)) {
            throw new Error(`Некорректная сумма для пополнения: ${amount}`);
        }

        balance += amount;
        const transaction = logTransaction('deposit', amount, balance);

        console.log(`Внесено: ${amount}. Новый баланс: ${balance}`);
        return transaction;
    },

    // Метод для снятия средств
    withdraw: function(amount) {
        if (!isValidAmount(amount)) {
            throw new Error(`Некорректная сумма для снятия: ${amount}`);
        }

        if (amount > balance) {
            throw new Error(`Недостаточно средств. Баланс: ${balance}, запрошено: $
{amount}`);
        }

        balance -= amount;
        const transaction = logTransaction('withdraw', amount, balance);

        console.log(`Снято: ${amount}. Новый баланс: ${balance}`);
        return transaction;
    },

```



```

// Метод для получения баланса
getBalance: function() {
    return balance;
},

// Метод для получения истории (только чтение)
getTransactionHistory: function() {
    // Возвращаем копию, чтобы защитить оригинальную историю
    return JSON.parse(JSON.stringify(transactionHistory));
},

// Метод для получения статистики
getStatistics: function() {
    const deposits = transactionHistory.filter(t => t.type === 'deposit');
    const withdrawals = transactionHistory.filter(t => t.type === 'withdraw');

    return {
        totalTransactions: transactionHistory.length,
        totalDeposits: deposits.reduce((sum, t) => sum + t.amount, 0),
        totalWithdrawals: withdrawals.reduce((sum, t) => sum + t.amount, 0),
        averageTransaction: transactionHistory.reduce((sum, t) => sum +
t.amount, 0) / transactionHistory.length || 0
    };
}
};
}

// Использование
const account = createBankAccount(1000);

account.deposit(500);    // Внесено: 500. Новый баланс: 1500
account.withdraw(200);  // Снято: 200. Новый баланс: 1300

console.log("Баланс:", account.getBalance()); // 1300
console.log("Статистика:", account.getStatistics());

// Попытка прямого доступа к приватным данным
console.log(account.balance);           // undefined
console.log(account.transactionHistory); // undefined
console.log(account.isValidAmount);     // undefined

```

2.2. Приватные поля и методы (ES2022+)

javascript

```
class SecureUserAccount {
  // Приватные поля
  #passwordHash;
  #balance;
  #loginAttempts = 0;
  #lastLoginAttempt;

  // Приватный метод хеширования
  #hashPassword(password) {
    // Упрощенная имитация хеширования
    return `hashed_${password}_${Date.now()}`;
  }

  // Приватный метод валидации
  #validatePassword(inputPassword) {
    const inputHash = this.#hashPassword(inputPassword);
    return inputHash === this.#passwordHash;
  }

  // Приватный метод проверки блокировки
  #isAccountLocked() {
    if (this.#loginAttempts >= 3) {
      const now = Date.now();
      const lockTime = 15 * 60 * 1000; // 15 минут

      if (!this.#lastLoginAttempt || (now - this.#lastLoginAttempt > lockTime)) {
        // Сброс после 15 минут
        this.#loginAttempts = 0;
        this.#lastLoginAttempt = null;
        return false;
      }
      return true;
    }
    return false;
  }

  // Приватный метод логирования
  #logActivity(action, details) {
    const logEntry = {
      timestamp: new Date().toISOString(),
```

```

        userId: this.#userId,
        action,
        details
    };

    // В реальном приложении здесь была бы отправка в лог-систему
    console.log("[ACTIVITY LOG]", logEntry);
}

// Публичный конструктор
constructor(userId, initialPassword, initialBalance = 0) {
    this.#userId = userId;
    this.#passwordHash = this.#hashPassword(initialPassword);
    this.#balance = initialBalance;
    this.email = `${userId}@example.com`; // Публичное свойство
}

// Публичные методы

login(password) {
    // Проверка блокировки
    if (this.#isAccountLocked()) {
        this.#logActivity('login_attempt', { status: 'blocked', reason:
'too_many_attempts' });
        throw new Error("Аккаунт временно заблокирован. Попробуйте через 15
минут.");
    }

    // Проверка пароля
    if (this.#validatePassword(password)) {
        this.#loginAttempts = 0;
        this.#lastLoginAttempt = null;
        this.#logActivity('login', { status: 'success' });
        return { success: true, message: "Вход выполнен успешно" };
    } else {
        this.#loginAttempts++;
        this.#lastLoginAttempt = Date.now();
        this.#logActivity('login_attempt', {
            status: 'failed',
            attempts: this.#loginAttempts
        });

        const remainingAttempts = 3 - this.#loginAttempts;
        throw new Error(`Неверный пароль. Осталось попыток: ${remainingAttempts}`);
    }
}

```

```

    }
}

changePassword(oldPassword, newPassword) {
    // Проверка старого пароля
    if (!this.#validatePassword(oldPassword)) {
        this.#logActivity('password_change', { status: 'failed', reason:
'wrong_old_password' });
        throw new Error("Неверный текущий пароль");
    }

    // Валидация нового пароля
    if (newPassword.length < 8) {
        throw new Error("Новый пароль должен содержать минимум 8 символов");
    }

    if (newPassword === oldPassword) {
        throw new Error("Новый пароль должен отличаться от старого");
    }

    // Смена пароля
    this.#passwordHash = this.#hashPassword(newPassword);
    this.#logActivity('password_change', { status: 'success' });

    return { success: true, message: "Пароль успешно изменен" };
}

getBalance() {
    return this.#balance;
}

deposit(amount) {
    if (amount <= 0) {
        throw new Error("Сумма пополнения должна быть положительной");
    }

    this.#balance += amount;
    this.#logActivity('deposit', { amount, newBalance: this.#balance });

    return this.#balance;
}

// Статический приватный метод
static #validateUserId(userId) {

```

```

    const regex = /^[a-zA-Z0-9_]{3,20}$/;
    return regex.test(userId);
}

// Статический публичный метод-фабрика
static createAccount(userId, password, initialBalance = 0) {
    if (!this.#validateUserId(userId)) {
        throw new Error("Некорректный ID пользователя");
    }

    return new SecureUserAccount(userId, password, initialBalance);
}
}

// Использование
try {
    const account = SecureUserAccount.createAccount("john_doe", "securePass123", 1000);

    // Попытка входа
    const loginResult = account.login("securePass123");
    console.log(loginResult.message); // "Вход выполнен успешно"

    // Получение баланса
    console.log("Баланс:", account.getBalance()); // 1000

    // Пополнение счета
    account.deposit(500);
    console.log("Новый баланс:", account.getBalance()); // 1500

    // Смена пароля
    account.changePassword("securePass123", "newSecurePass456");

    // Попытка доступа к приватным полям
    // console.log(account.#passwordHash); // SyntaxError
    // console.log(account.#balance); // SyntaxError

    // Публичные свойства доступны
    console.log(account.email); // john_doe@example.com
} catch (error) {
    console.error("Ошибка:", error.message);
}

```

2.3. Свойства с геттерами и сеттерами

javascript

```
class TemperatureSensor {
  #celsius = 0;
  #calibrationOffset = 0;
  #maxSafeTemperature = 100;
  #minSafeTemperature = -50;

  constructor(name, initialTemp = 20) {
    this.name = name;
    this.#celsius = initialTemp;
    this.#lastCalibration = new Date();
  }

  // Геттер для температуры в Цельсиях
  get celsius() {
    return this.#celsius + this.#calibrationOffset;
  }

  // Сеттер для температуры с валидацией
  set celsius(value) {
    if (typeof value !== 'number' || isNaN(value)) {
      throw new Error("Температура должна быть числом");
    }

    const calibratedValue = value - this.#calibrationOffset;

    // Проверка безопасного диапазона
    if (calibratedValue > this.#maxSafeTemperature) {
      console.warn(`⚠ Внимание: температура ${calibratedValue}°C превышает
максимально безопасную ${this.#maxSafeTemperature}°C`);
    }

    if (calibratedValue < this.#minSafeTemperature) {
      console.warn(`⚠ Внимание: температура ${calibratedValue}°C ниже минимально
безопасной ${this.#minSafeTemperature}°C`);
    }

    this.#celsius = calibratedValue;
    this.#lastUpdate = new Date();
  }
}
```

```

// Геттер для температуры в Фаренгейтах
get fahrenheit() {
    return (this.celsius * 9/5) + 32;
}

// Сеттер для температуры в Фаренгейтах
set fahrenheit(value) {
    this.celsius = (value - 32) * 5/9;
}

// Геттер для температуры в Кельвинах
get kelvin() {
    return this.celsius + 273.15;
}

// Сеттер для температуры в Кельвинах
set kelvin(value) {
    this.celsius = value - 273.15;
}

// Геттер только для чтения
get lastUpdate() {
    return new Date(this.#lastUpdate); // Возвращаем копию
}

// Приватный метод калибровки
#calibrateSensor() {
    // Имитация сложной логики калибровки
    const drift = (Date.now() - this.#lastCalibration) / (1000 * 60 * 60 * 24) * 0.1;
    this.#calibrationOffset = drift;
    this.#lastCalibration = new Date();
}

// Публичный метод для выполнения калибровки
performCalibration() {
    console.log(`Калибровка датчика ${this.name}...`);
    this.#calibrateSensor();
    console.log(`Калибровка завершена. Смещение: $
{this.#calibrationOffset.toFixed(2)}°C`);
}

// Метод для получения диагностической информации
get diagnostics() {
    this.#calibrateSensor(); // Автоматическая калибровка при диагностике

```

```

    return {
      name: this.name,
      rawTemperature: this.#celsius,
      calibrationOffset: this.#calibrationOffset,
      calibratedTemperature: this.celsius,
      lastCalibration: this.#lastCalibration,
      sensorHealth: this.#calibrationOffset < 1 ? 'good' : 'needs_attention',
      safeRange: {
        min: this.#minSafeTemperature,
        max: this.#maxSafeTemperature,
        isSafe: this.#celsius >= this.#minSafeTemperature &&
          this.#celsius <= this.#maxSafeTemperature
      }
    };
  }
}

```

// Использование

```
const sensor = new TemperatureSensor("Kitchen Sensor", 22);
```

// Использование геттеров

```

console.log(`Температура в Цельсиях: ${sensor.celsius}°C`);
console.log(`Температура в Фаренгейтах: ${sensor.fahrenheit}°F`);
console.log(`Температура в Кельвинах: ${sensor.kelvin}K`);

```

// Использование сеттеров

```

sensor.celsius = 25;
console.log(`Новая температура: ${sensor.celsius}°C`);

sensor.fahrenheit = 77;
console.log(`77°F = ${sensor.celsius.toFixed(1)}°C`);

```

// Диагностика

```
console.log("Диагностика:", sensor.diagnostics);
```

// Попытка установки невалидного значения

```

try {
  sensor.celsius = "hot"; // Ошибка: Температура должна быть числом
} catch (error) {
  console.error(error.message);
}

```


Часть 3: Продвинутые паттерны инкапсуляции

3.1. Прокси для контроля доступа

```
javascript

// Создание защищенного объекта с помощью Proxy
function createProtectedEmployee(data) {
  // Приватные данные
  const privateData = {
    salary: data.salary,
    ssn: data.ssn, // Social Security Number
    performanceReviews: data.performanceReviews || [],
    disciplinaryActions: data.disciplinaryActions || []
  };

  // Публичные данные
  const publicData = {
    name: data.name,
    position: data.position,
    department: data.department,
    hireDate: data.hireDate,
    email: data.email
  };

  // Правила доступа в зависимости от роли
  const accessRules = {
    employee: {
      readable: ['name', 'position', 'department', 'hireDate', 'email'],
      writable: []
    },
    manager: {
      readable: ['name', 'position', 'department', 'hireDate', 'email',
'performanceReviews'],
      writable: ['position', 'department', 'performanceReviews']
    },
    hr: {
      readable: Object.keys(publicData).concat(['salary', 'performanceReviews']),
      writable: ['salary', 'position', 'department']
    },
    admin: {
      readable: Object.keys(publicData).concat(Object.keys(privateData)),
      writable: Object.keys(publicData).concat(Object.keys(privateData))
    }
  };
}
```

```

    }
};

// Функция проверки доступа
function checkAccess(role, property, action = 'read') {
    const rules = accessRules[role];
    if (!rules) return false;

    if (action === 'read') {
        return rules.readable.includes(property);
    } else if (action === 'write') {
        return rules.writable.includes(property);
    }
    return false;
}

// Создание прокси
const handler = {
    get(target, property, receiver) {
        // Определяем роль текущего пользователя (в реальном приложении из
контекста)
        const currentRole = getUserRole();

        // Проверяем доступ на чтение
        if (!checkAccess(currentRole, property, 'read')) {
            if (property in privateData) {
                console.warn(`❌ Доступ запрещен: ${currentRole} не может читать ${
property}`);
                return undefined;
            }
            console.warn(`⚠️ Предупреждение: ${currentRole} пытается получить доступ
к неизвестному свойству ${property}`);
            return undefined;
        }

        // Возвращаем значение из соответствующего источника
        if (property in privateData) {
            // Для частных данных можно добавить дополнительную обработку
            if (property === 'salary') {
                return `$$${privateData[property].toLocaleString()}`;
            }
            if (property === 'ssn') {
                return '***-**-****'; // Маскировка SSN
            }
        }
    }
};

```

```

        return privateData[property];
    }

    return publicData[property];
},

set(target, property, value, receiver) {
    const currentRole = getUserRole();

    // Проверяем доступ на запись
    if (!checkAccess(currentRole, property, 'write')) {
        console.error(`❌ Запрещено: ${currentRole} не может изменять $
{property}`);
        return false;
    }

    // Валидация данных перед записью
    if (property === 'salary' && value < 0) {
        console.error("❌ Зарплата не может быть отрицательной");
        return false;
    }

    if (property === 'email' && !isValidEmail(value)) {
        console.error("❌ Некорректный email");
        return false;
    }

    // Логирование изменений
    console.log(`✅ ${currentRole} изменяет ${property} с ${target[property]} на
${value}`);

    // Записываем значение
    if (property in privateData) {
        privateData[property] = value;
    } else {
        publicData[property] = value;
    }

    return true;
},

// Скрываем приватные свойства при перечислении
ownKeys(target) {
    const currentRole = getUserRole();

```

```

        const allowedProperties = accessRules[currentRole]?.readable || [];
        return allowedProperties.filter(prop => prop in publicData || prop in
privateData);
    },

    // Скрываем приватные свойства при проверке наличия
    has(target, property) {
        const currentRole = getUserRole();
        return checkAccess(currentRole, property, 'read');
    },

    // Получение дескриптора свойства
    getOwnPropertyDescriptor(target, property) {
        const currentRole = getUserRole();
        if (!checkAccess(currentRole, property, 'read')) {
            return undefined;
        }

        const value = property in privateData ? privateData[property] :
publicData[property];
        return {
            value,
            writable: checkAccess(currentRole, property, 'write'),
            enumerable: true,
            configurable: false
        };
    }
};

// Функции-заглушки для демонстрации
function getUserRole() {
    // В реальном приложении здесь была бы логика определения роли
    return 'manager'; // Меняйте для тестирования разных ролей
}

function isValidEmail(email) {
    const regex = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+$/;
    return regex.test(email);
}

// Создаем и возвращаем прокси
const proxy = new Proxy({}, handler);

// Добавляем публичные методы

```

```

    proxy.getPublicInfo = function() {
        return { ...publicData };
    };

    proxy.requestSalaryReview = function(reason) {
        console.log(`Запрошен пересмотр зарплаты. Причина: ${reason}`);
        // Логика обработки запроса
        return { success: true, message: "Запрос принят в обработку" };
    };

    return proxy;
}

// Использование
const employeeData = {
    name: "Иван Петров",
    position: "Senior Developer",
    department: "Engineering",
    hireDate: "2020-01-15",
    email: "ivan.petrov@company.com",
    salary: 120000,
    ssn: "123-45-6789",
    performanceReviews: [
        { date: "2023-01-10", rating: 4.5, comments: "Отличная работа" },
        { date: "2022-01-12", rating: 4.0, comments: "Хорошие результаты" }
    ]
};

const employee = createProtectedEmployee(employeeData);

// Доступ в зависимости от роли
console.log("Имя:", employee.name); // Доступно для всех ролей
console.log("Должность:", employee.position);
console.log("Зарплата:", employee.salary); // Форматированная и доступная только
некоторым ролям
console.log("SSN:", employee.ssn); // Замаскировано

// Попытка доступа к запрещенным данным
console.log("Попытка доступа к приватному полю:", employee.disciplinaryActions); //
undefined

// Попытка изменения данных
employee.position = "Lead Developer"; // Разрешено для manager
// employee.salary = 150000; // Запрещено для manager

```

3.2. WeakMap для приватных данных

javascript

// Паттерн с WeakMap для приватных данных (до ES2022)

```
const SecretVault = (function() {  
  // Приватное хранилище с использованием WeakMap  
  const privateData = new WeakMap();  
  
  // Приватные утилиты  
  const encryption = {  
    encrypt(text, key) {  
      // Упрощенная имитация шифрования  
      return btoa(text + key).split('').reverse().join('');  
    },  
  
    decrypt(encrypted, key) {  
      // Дешифровка  
      return atob(encrypted.split('').reverse().join('')).replace(key, '');  
    }  
  };  
};
```

```
class SecretVault {  
  constructor(ownerName, masterKey) {  
    // Инициализация приватных данных  
    privateData.set(this, {  
      owner: ownerName,  
      masterKey: masterKey,  
      secrets: new Map(),  
      accessLog: [],  
      failedAttempts: 0,  
      lastAccess: null  
    });  
  
    // Логирование создания  
    this.#logAccess('vault_created', { owner: ownerName });  
  }  
}
```

// Приватный метод логирования (синтаксис ES2022)

```
#logAccess(action, details = {}) {  
  const data = privateData.get(this);  
  const logEntry = {  
    timestamp: new Date().toISOString(),  
    action,
```

```

        ...details
    };

    data.accessLog.push(logEntry);
    data.lastAccess = new Date();
}

// Приватный метод проверки ключа
#verifyKey(key) {
    const data = privateData.get(this);
    return key === data.masterKey;
}

// Публичные методы

storeSecret(key, secretName, secretValue) {
    if (!this.#verifyKey(key)) {
        this.#handleFailedAccess();
        throw new Error("Неверный мастер-ключ");
    }

    const data = privateData.get(this);

    // Шифрование секрета
    const encryptedSecret = encryption.encrypt(
        JSON.stringify(secretValue),
        data.masterKey
    );

    // Хранение
    data.secrets.set(secretName, {
        value: encryptedSecret,
        created: new Date(),
        lastAccessed: null,
        accessCount: 0
    });

    this.#logAccess('secret_stored', { secretName });
    this.#resetFailedAttempts();

    return { success: true, message: `Секрет "${secretName}" сохранен` };
}

retrieveSecret(key, secretName) {

```

```

    if (!this.#verifyKey(key)) {
        this.#handleFailedAccess();
        throw new Error("Неверный мастер-ключ");
    }

    const data = privateData.get(this);

    if (!data.secrets.has(secretName)) {
        throw new Error(`Секрет "${secretName}" не найден`);
    }

    const secretRecord = data.secrets.get(secretName);

    // Дешифровка
    const decryptedValue = encryption.decrypt(
        secretRecord.value,
        data.masterKey
    );

    // Обновление статистики доступа
    secretRecord.lastAccessed = new Date();
    secretRecord.accessCount++;

    this.#logAccess('secret_retrieved', { secretName });
    this.#resetFailedAttempts();

    return {
        success: true,
        secretName,
        value: JSON.parse(decryptedValue),
        metadata: {
            created: secretRecord.created,
            lastAccessed: secretRecord.lastAccessed,
            accessCount: secretRecord.accessCount
        }
    };
}

listSecrets(key) {
    if (!this.#verifyKey(key)) {
        this.#handleFailedAccess();
        throw new Error("Неверный мастер-ключ");
    }
}

```



```

    const data = privateData.get(this);
    const secretsList = [];

    for (const [name, record] of data.secrets) {
        secretsList.push({
            name,
            created: record.created,
            lastAccessed: record.lastAccessed,
            accessCount: record.accessCount
        });
    }

    this.#logAccess('secrets_listed');
    this.#resetFailedAttempts();

    return secretsList;
}

// Приватные вспомогательные методы
#handleFailedAccess() {
    const data = privateData.get(this);
    data.failedAttempts++;
    this.#logAccess('failed_access', {
        attempt: data.failedAttempts
    });

    if (data.failedAttempts >= 3) {
        this.#logAccess('vault_locked');
        throw new Error("Хранилище заблокировано из-за множества неудачных
попыток");
    }
}

#resetFailedAttempts() {
    const data = privateData.get(this);
    data.failedAttempts = 0;
}

// Методы для получения статистики (требуют ключ)
getVaultStats(key) {
    if (!this.#verifyKey(key)) {
        this.#handleFailedAccess();
        throw new Error("Неверный мастер-ключ");
    }
}

```

```

        const data = privateData.get(this);

        return {
            owner: data.owner,
            totalSecrets: data.secrets.size,
            lastAccess: data.lastAccess,
            failedAttempts: data.failedAttempts,
            accessLogSize: data.accessLog.length,
            recentActivity: data.accessLog.slice(-5)
        };
    }

    // Метод для безопасного удаления хранилища
    destroyVault(key) {
        if (!this.#verifyKey(key)) {
            this.#handleFailedAccess();
            throw new Error("Неверный мастер-ключ");
        }

        // Удаляем все ссылки на приватные данные
        privateData.delete(this);
        this.#logAccess('vault_destroyed');

        return { success: true, message: "Хранилище уничтожено" };
    }
}

return SecretVault;
})();

// Использование
try {
    const myVault = new SecretVault("Алексей", "supersecretkey123");

    // Сохранение секретов
    myVault.storeSecret("supersecretkey123", "банковский_пароль", {
        bank: "Тинькофф",
        login: "alexey2023",
        password: "SecurePass!2023"
    });

    myVault.storeSecret("supersecretkey123", "паспортные_данные", {
        series: "4510",

```

```

        number: "123456",
        issuedBy: "ОУФМС России"
    });

    // Получение списка секретов
    const secrets = myVault.listSecrets("supersecretkey123");
    console.log("Секреты в хранилище:", secrets);

    // Извлечение секрета
    const passportData = myVault.retrieveSecret("supersecretkey123",
"паспортные_данные");
    console.log("Паспортные данные:", passportData);

    // Статистика
    const stats = myVault.getVaultStats("supersecretkey123");
    console.log("Статистика хранилища:", stats);

    // Попытка несанкционированного доступа
    try {
        myVault.retrieveSecret("wrongkey", "банковский_пароль");
    } catch (error) {
        console.error("Ошибка доступа:", error.message);
    }

    // Уничтожение хранилища
    // myVault.destroyVault("supersecretkey123");

} catch (error) {
    console.error("Ошибка:", error.message);
}

```

Часть 4: Практическое применение инкапсуляции

4.1. API клиент с инкапсуляцией

javascript

```

class SecureAPIClient {
    #baseUrl;
    #apiKey;
    #sessionToken = null;
    #refreshToken = null;

```

```

#requestQueue = [];
#isRefreshing = false;
#cache = new Map();
#maxRetries = 3;

// Приватные константы
static #DEFAULT_TIMEOUT = 30000;
static #CACHE_TTL = 5 * 60 * 1000; // 5 минут

constructor(baseUrl, apiKey) {
  this.#baseUrl = baseUrl;
  this.#apiKey = apiKey;
  this.#setupInterceptors();
  this.#startCacheCleanup();
}

// Приватная настройка перехватчиков
#setupInterceptors() {
  // В реальном приложении здесь был бы axios/fetch interceptor
  console.log("Перехватчики настроены");
}

// Приватное кэширование
#getFromCache(key) {
  const cached = this.#cache.get(key);
  if (!cached) return null;

  if (Date.now() - cached.timestamp > SecureAPIClient.#CACHE_TTL) {
    this.#cache.delete(key);
    return null;
  }

  return cached.data;
}

#setToCache(key, data) {
  this.#cache.set(key, {
    data,
    timestamp: Date.now()
  });
}

// Приватная очистка кэша
#startCacheCleanup() {

```

```

setInterval(() => {
    const now = Date.now();
    for (const [key, value] of this.#cache.entries()) {
        if (now - value.timestamp > SecureAPIClient.#CACHE_TTL) {
            this.#cache.delete(key);
        }
    }
}, 60000); // Каждую минуту
}

```

// Приватный метод запроса с повторными попытками

```

async #makeRequest(method, endpoint, data = null, options = {}) {
    const cacheKey = options.cacheKey || `${method}:${endpoint}:${JSON.stringify(data)}`;

```

// Проверка кэша для GET запросов

```

    if (method === 'GET' && !options.skipCache) {
        const cached = this.#getFromCache(cacheKey);
        if (cached) {
            console.log(`[CACHE HIT] ${endpoint}`);
            return cached;
        }
    }
}

```

```

const url = `${this.#baseUrl}${endpoint}`;
const headers = {
    'Authorization': `Bearer ${this.#sessionToken || this.#apiKey}`,
    'Content-Type': 'application/json',
    ...options.headers
};

```

```

const config = {
    method,
    headers,
    timeout: options.timeout || SecureAPIClient.#DEFAULT_TIMEOUT
};

```

```

if (data && method !== 'GET') {
    config.body = JSON.stringify(data);
}

```

// Логика повторных попыток

```

for (let attempt = 1; attempt <= this.#maxRetries; attempt++) {
    try {

```

```

        console.log(`[${method}] ${endpoint} (попытка ${attempt}/${
{this.#maxRetries}}`);

        const response = await fetch(url, config);

        if (response.status === 401 && !options.isRefresh) {
            // Попытка обновления токена
            await this.#refreshAuthToken();
            // Повтор запроса с новым токеном
            headers.Authorization = `Bearer ${this.#sessionToken}`;
            continue;
        }

        if (!response.ok) {
            throw new Error(`HTTP ${response.status}: ${response.statusText}`);
        }

        const result = await response.json();

        // Кэширование успешных GET запросов
        if (method === 'GET' && !options.skipCache) {
            this.#setToCache(cacheKey, result);
        }

        return result;
    } catch (error) {
        if (attempt === this.#maxRetries) {
            console.error(`[ERROR] Не удалось выполнить запрос после $
{this.#maxRetries} попыток`, error.message);
            throw error;
        }

        // Экспоненциальная backoff задержка
        const delay = Math.min(1000 * Math.pow(2, attempt), 10000);
        console.log(`Повтор через ${delay}мс...`);
        await new Promise(resolve => setTimeout(resolve, delay));
    }
}

// Приватное обновление токена
async #refreshAuthToken() {
    if (this.#isRefreshing) {

```

```

    // Ждем завершения текущего обновления
    await new Promise(resolve => {
        const check = () => {
            if (!this.#isRefreshing) resolve();
            else setTimeout(check, 100);
        };
        check();
    });
    return;
}

this.#isRefreshing = true;

try {
    console.log("Обновление токена доступа...");

    // Имитация запроса на обновление токена
    await new Promise(resolve => setTimeout(resolve, 1000));

    // В реальном приложении здесь был бы запрос к серверу
    this.#sessionToken = `new_token_${Date.now()}`;
    this.#refreshToken = `new_refresh_${Date.now()}`;

    console.log("Токен успешно обновлен");

} catch (error) {
    console.error("Ошибка обновления токена:", error);
    throw error;
} finally {
    this.#isRefreshing = false;
}
}

// Приватная обработка очереди запросов
#enqueueRequest(requestFn) {
    return new Promise((resolve, reject) => {
        this.#requestQueue.push({ requestFn, resolve, reject });
        this.#processQueue();
    });
}

async #processQueue() {
    if (this.#requestQueue.length === 0) return;

```

```

const { requestFn, resolve, reject } = this.#requestQueue.shift();

try {
    const result = await requestFn();
    resolve(result);
} catch (error) {
    reject(error);
}

// Рекурсивная обработка следующего запроса
this.#processQueue();
}

// Публичный интерфейс

async login(username, password) {
    try {
        const response = await this.#makeRequest('POST', '/auth/login', {
            username,
            password
        }, { skipCache: true });

        this.#sessionToken = response.accessToken;
        this.#refreshToken = response.refreshToken;

        return {
            success: true,
            message: "Авторизация успешна",
            user: response.user
        };
    } catch (error) {
        console.error("Ошибка авторизации:", error);
        return {
            success: false,
            message: "Ошибка авторизации",
            error: error.message
        };
    }
}

async get(endpoint, options = {}) {
    return this.#enqueueRequest(() =>
        this.#makeRequest('GET', endpoint, null, options)
    );
}

```



```

    );
}

async post(endpoint, data, options = {}) {
    return this.#enqueueRequest(() =>
        this.#makeRequest('POST', endpoint, data, options)
    );
}

async put(endpoint, data, options = {}) {
    return this.#enqueueRequest(() =>
        this.#makeRequest('PUT', endpoint, data, options)
    );
}

async delete(endpoint, options = {}) {
    return this.#enqueueRequest(() =>
        this.#makeRequest('DELETE', endpoint, null, options)
    );
}

// Методы управления кэшем
clearCache() {
    this.#cache.clear();
    console.log("Кэш очищен");
}

getCacheStats() {
    return {
        size: this.#cache.size,
        entries: Array.from(this.#cache.entries()).map(([key, value]) => ({
            key,
            age: Date.now() - value.timestamp
        })))
    };
}

// Метод для безопасного выхода
logout() {
    this.#sessionToken = null;
    this.#refreshToken = null;
    this.#requestQueue = [];
    this.clearCache();
    console.log("Выполнен выход из системы");
}

```

```

    }

    // Геттер для статуса (только чтение)
    get status() {
        return {
            isAuthenticated: !!this.#sessionToken,
            queueLength: this.#requestQueue.length,
            cacheSize: this.#cache.size,
            isRefreshingToken: this.#isRefreshing
        };
    }
}

// Пример использования
async function demonstrateAPIClient() {
    const client = new SecureAPIClient(
        "https://api.example.com",
        "demo_api_key_12345"
    );

    try {
        // Авторизация
        const loginResult = await client.login("user@example.com", "password123");
        console.log("Авторизация:", loginResult);

        // Получение данных с кэшированием
        const users = await client.get("/users");
        console.log("Пользователи:", users.length);

        // Повторный запрос - должен взять из кэша
        const cachedUsers = await client.get("/users");
        console.log("Пользователи (из кэша):", cachedUsers.length);

        // Создание нового ресурса
        const newUser = await client.post("/users", {
            name: "Новый Пользователь",
            email: "new@example.com"
        });
        console.log("Создан пользователь:", newUser);

        // Получение статуса клиента
        console.log("Статус клиента:", client.status);

        // Статистика кэша
    }
}

```

```

    console.log("Статистика кэша:", client.getCacheStats());

    // Выход
    client.logout();
    console.log("После выхода:", client.status);

  } catch (error) {
    console.error("Ошибка:", error);
  }
}

// Запуск демонстрации
// demonstrateAPIClient();

```

Часть 5: Антипаттерны и лучшие практики

5.1. Чего избегать

```

javascript

// × АНТИПАТТЕРН 1: Излишняя открытость
class BadEncapsulation {
  constructor() {
    this._internalState = "секрет"; // Псевдоприватность
    this.secretKey = "12345";      // Публичный секрет!
    this.databaseConnection = null; // Слишком открыто
  }

  // Методы, которые раскрывают слишком много
  getInternalState() {
    return this._internalState; // Нарушение инкапсуляции
  }

  connectToDatabase() {
    this.databaseConnection = "connected"; // Прямое изменение состояния
  }
}

// × АНТИПАТТЕРН 2: Ложная инкапсуляция
class FalseEncapsulation {
  constructor() {
    // Использование замыканий, но с утечками
  }
}

```

```

    let secret = "confidential";

    this.getSecret = function() {
        return secret; // Все еще доступно через метод
    };

    // Но также публичные свойства
    this.publicSecret = "not so secret";
}

// Метод, который нарушает инкапсуляцию
exposeInternals() {
    return {
        secret: this.getSecret(),
        everything: this // Возвращаем весь объект!
    };
}
}

// × АНТИПАТТЕРН 3: Избыточные геттеры/сеттеры
class OverEncapsulation {
    #value;

    constructor(value) {
        this.#value = value;
    }

    // Избыточные геттеры для всего
    get value() { return this.#value; }
    set value(v) { this.#value = v; }

    get valueAsString() { return String(this.#value); }
    get valueAsNumber() { return Number(this.#value); }
    get valueLength() { return this.valueAsString.length; }

    // Слишком много сеттеров
    set valueFromString(str) { this.#value = parseFloat(str); }
    set valueFromNumber(num) { this.#value = num; }
    set valueIncremented() { this.#value++; }
    set valueDecrementd() { this.#value--; }
}

// × АНТИПАТТЕРН 4: Глобальное состояние
class GlobalStateManager {

```

```

constructor() {
    if (!GlobalStateManager.instance) {
        GlobalStateManager.instance = this;
        this.secrets = new Map(); // Глобальное состояние!
    }
    return GlobalStateManager.instance;
}

// Методы, работающие с глобальным состоянием
addSecret(key, value) {
    this.secrets.set(key, value); // Изменение глобального состояния
}

// Статический метод, нарушающий инкапсуляцию
static getSecrets() {
    return Array.from(this.instance?.secrets || []);
}
}

```

5.2. Лучшие практики

javascript

// 📌 ПРАКТИКА 1: Явные интерфейсы

```

class WellEncapsulatedSystem {
    #database;
    #cache;
    #config;

    constructor(config) {
        this.#config = Object.freeze({ ...config }); // Замороженная копия
        this.#database = this.#initializeDatabase();
        this.#cache = new Map();
    }

    // Приватные методы инициализации
    #initializeDatabase() {
        // Сложная логика инициализации
        return {
            connect: () => console.log("Подключено к БД"),
            query: (sql) => console.log(`Выполнен запрос: ${sql}`)
        };
    }
}

```

```

// Публичный интерфейс
async queryData(query, options = {}) {
    const cacheKey = this.#generateCacheKey(query, options);

    // Проверка кэша
    if (options.useCache && this.#cache.has(cacheKey)) {
        return this.#cache.get(cacheKey);
    }

    // Выполнение запроса
    const data = await this.#executeQuery(query, options);

    // Кэширование результата
    if (options.useCache) {
        this.#cache.set(cacheKey, data);
    }

    return data;
}

// Приватные вспомогательные методы
#generateCacheKey(query, options) {
    return `${query}_${JSON.stringify(options)}`;
}

async #executeQuery(query, options) {
    // Безопасное выполнение запроса
    try {
        return await this.#database.query(query);
    } catch (error) {
        console.error("Ошибка запроса:", error);
        throw new Error("Не удалось выполнить запрос");
    }
}

// Метод очистки ресурсов
cleanup() {
    this.#cache.clear();
    console.log("Ресурсы очищены");
}
}

```

// 📌 ПРАКТИКА 2: Иммутабельные интерфейсы

```
class ImmutableDataStore {
```

```

#data;
#version = 0;
#changeHistory = [];

constructor(initialData = {}) {
  this.#data = this.#deepFreeze(initialData);
}

#deepFreeze(obj) {
  Object.freeze(obj);
  Object.keys(obj).forEach(key => {
    if (typeof obj[key] === 'object' && obj[key] !== null) {
      this.#deepFreeze(obj[key]);
    }
  });
  return obj;
}

// Методы, возвращающие новые объекты вместо изменения существующих
set(path, value) {
  const newData = this.#deepClone(this.#data);
  this.#setNestedValue(newData, path, value);

  const frozenData = this.#deepFreeze(newData);

  // Создание новой версии
  return new ImmutableDataStore(frozenData);
}

get(path) {
  return this.#getNestedValue(this.#data, path);
}

// Приватные вспомогательные методы
#deepClone(obj) {
  return JSON.parse(JSON.stringify(obj));
}

#getNestedValue(obj, path) {
  return path.split('.').reduce((current, key) => {
    return current ? current[key] : undefined;
  }, obj);
}

```

```

#setNestedValue(obj, path, value) {
  const keys = path.split('.');
  const lastKey = keys.pop();
  const target = keys.reduce((current, key) => {
    if (!current[key]) current[key] = {};
    return current[key];
  }, obj);

  target[lastKey] = value;
}

// Геттеры только для чтения
get version() {
  return this.#version;
}

get snapshot() {
  return this.#deepClone(this.#data);
}
}

// ПРАКТИКА 3: Композиция вместо наследования
class EncapsulatedComponent {
  #logger;
  #validator;
  #transformer;

  constructor(config) {
    // Композиция зависимостей
    this.#logger = new Logger(config.logLevel);
    this.#validator = new Validator(config.rules);
    this.#transformer = new DataTransformer(config.transformations);
  }

  async process(input) {
    try {
      // Валидация
      this.#validator.validate(input);
      this.#logger.log("Валидация пройдена");

      // Преобразование
      const transformed = this.#transformer.transform(input);
      this.#logger.log("Данные преобразованы");
    }
  }
}

```



```

        // Обработка
        const result = await this.#executeBusinessLogic(transformed);
        this.#logger.log("Бизнес-логика выполнена");

        return result;

    } catch (error) {
        this.#logger.error("Ошибка обработки:", error);
        throw error;
    }
}

async #executeBusinessLogic(data) {
    // Приватная бизнес-логика
    return { processed: true, data };
}

// Методы для управления жизненным циклом
initialize() {
    this.#logger.log("Компонент инициализирован");
}

dispose() {
    this.#logger.log("Компонент уничтожен");
}
}

```

Часть 6: Тестирование инкапсулированного кода

```

javascript

// Тестирование инкапсулированных компонентов
describe('SecureBankAccount Tests', () => {
    let account;

    beforeEach(() => {
        account = new SecureBankAccount("test-user", "initialPassword", 1000);
    });

    afterEach(() => {
        account = null;
    });

    describe('Инкапсуляция данных', () => {

```

```

test('Приватные поля недоступны извне', () => {
    // Проверка, что приватные поля действительно скрыты
    expect(account.balance).toBeUndefined();
    expect(account.#balance).toThrow(SyntaxError);
    expect(account.passwordHash).toBeUndefined();
});

test('Доступ только через публичные методы', () => {
    // Баланс доступен только через геттер
    expect(account.getBalance()).toBe(1000);

    // Внесение средств только через метод
    account.deposit(500);
    expect(account.getBalance()).toBe(1500);
});

describe('Валидация данных', () => {
    test('Некорректное пополнение вызывает ошибку', () => {
        expect(() => account.deposit(-100)).toThrow();
        expect(() => account.deposit(0)).toThrow();
        expect(() => account.deposit(NaN)).toThrow();
        expect(() => account.deposit("invalid")).toThrow();
    });

    test('Снятие больше баланса вызывает ошибку', () => {
        expect(() => account.withdraw(2000)).toThrow();
    });
});

describe('Безопасность', () => {
    test('Пароль хранится в хэшированном виде', () => {
        // Проверка, что пароль не хранится в открытом виде
        const internalState = account.getInternalStateForTesting?();
        if (internalState) {
            expect(internalState.passwordHash).not.toBe("initialPassword");
            expect(internalState.passwordHash).toMatch(/^hashed_/);
        }
    });

    test('Множественные неудачные попытки блокируют аккаунт', async () => {
        // Симуляция множества неудачных попыток входа
        for (let i = 0; i < 3; i++) {
            try {

```

```

        await account.login("wrong-password");
    } catch (error) {
        // Ожидаемая ошибка
    }
}

// Четвертая попытка должна быть заблокирована
await expect(account.login("wrong-password"))
    .rejects
    .toThrow("Аккаунт временно заблокирован");
});

describe('Целостность состояния', () => {
    test('Транзакции атомарны', async () => {
        const initialBalance = account.getBalance();

        // Создание "сломанной" транзакции
        const brokenTransaction = async () => {
            await account.withdraw(500);
            throw new Error("Искусственная ошибка");
            // Дальнейший код не выполнится
        };

        // Попытка выполнения сломанной транзакции
        await expect(brokenTransaction()).rejects.toThrow();

        // Баланс не должен измениться
        expect(account.getBalance()).toBe(initialBalance);
    });

    test('История транзакций защищена от изменений', () => {
        const history = account.getTransactionHistory();

        // Попытка изменить возвращенную историю
        history.push({ fake: "transaction" });

        // Оригинальная история не должна измениться
        const originalHistory = account.getTransactionHistory();
        expect(originalHistory).not.toContainEqual({ fake: "transaction" });
    });
});

describe('Производительность', () => {

```

```

test('Кэширование работает корректно', () => {
    const startTime = performance.now();

    // Первый запрос - должен быть медленным
    const result1 = account.getStatistics();
    const time1 = performance.now() - startTime;

    // Второй запрос - должен быть быстрым (из кэша)
    const result2 = account.getStatistics();
    const time2 = performance.now() - (startTime + time1);

    expect(time2).toBeLessThan(time1 * 0.5); // В 2+ раза быстрее
    expect(result1).toEqual(result2); // Результаты одинаковые
});

test('Очистка кэша работает', () => {
    // Заполнение кэша
    account.getStatistics();

    // Очистка
    account.clearCache();

    const cacheStats = account.getCacheStats();
    expect(cacheStats.size).toBe(0);
});
});

// Вспомогательный класс для тестирования (только для тестов!)
class TestableSecureBankAccount extends SecureBankAccount {
    // Методы, доступные только в тестовом окружении
    getInternalStateForTesting() {
        // ВНИМАНИЕ: Этот метод существует только для тестов!
        // В продакшене он должен быть удален или защищен
        if (process.env.NODE_ENV !== 'test') {
            throw new Error("Метод доступен только в тестовом окружении");
        }

        return {
            balance: this.#balance,
            passwordHash: this.#passwordHash,
            transactionHistory: this.#transactionHistory
        };
    }
}

```

```
// Метод для сброса состояния в тестах
resetForTesting() {
    if (process.env.NODE_ENV !== 'test') {
        throw new Error("Метод доступен только в тестовом окружении");
    }

    this.#balance = 0;
    this.#transactionHistory = [];
    this.#loginAttempts = 0;
}
}
```

Заключение: принципы эффективной инкапсуляции

Ключевые выводы:

1. Минимизация публичного интерфейса

- ➔ Экспортируйте только то, что действительно необходимо
- ➔ Скрывайте детали реализации
- ➔ Предоставляйте четкий, документированный API

2. Защита целостности данных

- ➔ Используйте приватные поля для критических данных
- ➔ Валидируйте все входные данные
- ➔ Контролируйте изменения состояния

3. Грамотное использование инструментов

- ➔ Приватные поля ES2022+ для настоящей инкапсуляции
- ➔ Геттеры/сеттеры для контролируемого доступа
- ➔ Прокси для продвинутого контроля
- ➔ WeakMap для приватных данных в legacy-коде

4. Баланс между безопасностью и удобством

- ➔ Не переусердствуйте с инкапсуляцией
- ➔ Обеспечьте возможность тестирования
- ➔ Предусмотрите расширяемость

5. Документирование инкапсуляции

- ➔ Четко обозначайте публичный API
- ➔ Документируйте ожидаемое поведение
- ➔ Указывайте гарантии и ограничения

Практические рекомендации для JavaScript разработчиков:

1. **Всегда используйте приватные поля** для данных, которые не должны быть доступны извне
2. **Предпочитайте методы геттерам/сеттерам** для сложной логики доступа
3. **Возвращайте копии объектов**, а не оригиналы, для защиты внутреннего состояния
4. **Используйте фабричные функции** для создания объектов со сложной инициализацией
5. **Добавляйте методы очистки ресурсов** для управляемого жизненного цикла объектов

Инкапсуляция в JavaScript — это не просто синтаксическая конструкция, а философия проектирования. Правильно инкапсулированный код легче тестировать, проще поддерживать и безопаснее использовать. Современный JavaScript предоставляет все необходимые инструменты для эффективной инкапсуляции — от классических замыканий до современных приватных полей ES2022+.

Глава 9: Массивы и коллекции

9.1. Создание и манипуляция массивами

Введение: природа массивов в JavaScript

Массивы в JavaScript — это **высокоуровневые, динамические, гетерогенные коллекции**, которые сочетают в себе:

- **Индексированный доступ** как в классических массивах
- **Динамическое изменение размера** как в списках
- **Методы высшего порядка** как в функциональных коллекциях
- **Объектную природу** с наследованием от `Array.prototype`

Часть 1: Создание массивов

1.1. Базовые способы создания

Литеральная нотация (самый распространенный способ)

```
javascript
```

```
// 1. Пустой массив
```

```
const emptyArray = [];  
console.log(emptyArray); // []  
console.log(emptyArray.length); // 0
```

```
// 2. Массив с начальными элементами
```

```
const fruits = ["яблоко", "банан", "апельсин"];  
console.log(fruits); // ["яблоко", "банан", "апельсин"]
```

```
// 3. Гетерогенный массив (разные типы данных)
```

```
const mixedArray = [  
  "строка",  
  42,
```

```

    true,
    null,
    undefined,
    { name: "объект" },
    [1, 2, 3],
    function() { return "функция"; },
    Symbol("символ")
];
console.log(mixedArray.length); // 9
console.log(typeof mixedArray[5]); // "object"
console.log(typeof mixedArray[7]); // "function"

// 4. Массив с пропущенными элементами (разреженный массив)
const sparseArray = ["первый", , "третий", , , "шестой"];
console.log(sparseArray); // ["первый", empty × 1, "третий", empty × 2, "шестой"]
console.log(sparseArray.length); // 6
console.log(1 in sparseArray); // false - элемент отсутствует
console.log(sparseArray[1]); // undefined

```

Конструктор Array

```

javascript

// 1. Пустой массив
const arr1 = new Array();
console.log(arr1); // []
console.log(arr1.length); // 0

// 2. Массив заданной длины
const arr2 = new Array(5);
console.log(arr2); // [empty × 5] - массив с 5 пустыми слотами
console.log(arr2.length); // 5
console.log(arr2[0]); // undefined

// 3. Массив с элементами
const arr3 = new Array("яблоко", "банан", "апельсин");
console.log(arr3); // ["яблоко", "банан", "апельсин"]

// 4. Особый случай: одно числовое значение
const arr4 = new Array(3); // Массив длиной 3
const arr5 = new Array("3"); // Массив с одним элементом "3"
console.log(arr4.length); // 3
console.log(arr5.length); // 1
console.log(arr5[0]); // "3"

```


// 5. Многострочное создание для читаемости

```
const matrix = new Array(  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
);  
console.log(matrix[1][2]); // 6
```

Статические методы создания

javascript

// 1. Array.of() - создает массив из аргументов

```
const arr1 = Array.of(1, 2, 3, 4, 5);  
console.log(arr1); // [1, 2, 3, 4, 5]  
  
const arr2 = Array.of(7); // [7] - в отличие от new Array(7)  
console.log(arr2); // [7]  
console.log(arr2.length); // 1  
  
const arr3 = Array.of(); // []  
console.log(arr3); // []
```

// 2. Array.from() - создает массив из итерируемого объекта

```
const arr4 = Array.from("Hello");  
console.log(arr4); // ["H", "e", "l", "l", "o"]  
  
const arr5 = Array.from([1, 2, 3], x => x * 2);  
console.log(arr5); // [2, 4, 6]  
  
const arr6 = Array.from({ length: 5 }, (_, i) => i * 2);  
console.log(arr6); // [0, 2, 4, 6, 8]
```

// 3. Array.from() с Set и Map

```
const set = new Set([1, 2, 3, 3, 2, 1]);  
const arr7 = Array.from(set);  
console.log(arr7); // [1, 2, 3] - дубликаты удалены  
  
const map = new Map([["a", 1], ["b", 2], ["c", 3]]);  
const arr8 = Array.from(map);  
console.log(arr8); // [["a", 1], ["b", 2], ["c", 3]]
```

1.2. Создание специализированных массивов

javascript

// 1. Диапазоны чисел

```
function createRange(start, end, step = 1) {  
    const length = Math.ceil((end - start) / step);  
    return Array.from({ length }, (_, i) => start + i * step);  
}
```

```
console.log(createRange(1, 10)); // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
console.log(createRange(0, 50, 5)); // [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50]  
console.log(createRange(10, 0, -1)); // [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

// 2. Массивы с повторяющимися значениями

```
function fillArray(value, length) {  
    return Array.from({ length }, () => value);  
}
```

```
console.log(fillArray(0, 5)); // [0, 0, 0, 0, 0]  
console.log(fillArray("?", 3)); // ["?", "?", "?"]  
console.log(fillArray({}, 2)); // [{}, {}] - разные объекты!
```

// 3. Последовательности

```
function sequence(generator, length) {  
    return Array.from({ length }, (_, i) => generator(i));  
}
```

```
const squares = sequence(i => i * i, 5);  
console.log(squares); // [0, 1, 4, 9, 16]
```

```
const fibonacci = sequence(function(i) {  
    if (i < 2) return i;  
    return this[i - 1] + this[i - 2];  
}, 10);  
console.log(fibonacci); // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

// 4. Матрицы

```
function createMatrix(rows, cols, filler = 0) {  
    return Array.from({ length: rows }, () =>  
        Array.from({ length: cols }, () => filler)  
    );  
}
```

```

const matrix3x3 = createMatrix(3, 3);
console.log(matrix3x3);
// [[0, 0, 0],
//  [0, 0, 0],
//  [0, 0, 0]]

const identityMatrix = createMatrix(3, 3, (row, col) =>
  row === col ? 1 : 0
);
console.log(identityMatrix);
// [[1, 0, 0],
//  [0, 1, 0],
//  [0, 0, 1]]

```

1.3. Копирование массивов

javascript

```

const original = ["a", "b", "c", { x: 1 }];

// 1. Поверхностное копирование
const shallowCopy1 = original.slice();
const shallowCopy2 = [...original];
const shallowCopy3 = Array.from(original);
const shallowCopy4 = original.concat();

console.log(shallowCopy1); // ["a", "b", "c", {x: 1}]
console.log(shallowCopy1 === original); // false - новый массив
console.log(shallowCopy1[3] === original[3]); // true - ссылка на тот же объект

// 2. Глубокое копирование
function deepCopyArray(arr) {
  return JSON.parse(JSON.stringify(arr));
}

const deepCopy = deepCopyArray(original);
console.log(deepCopy[3] === original[3]); // false - новый объект

// 3. Структурированное копирование (ES2023+)
const structuredCopy = structuredClone(original);
console.log(structuredCopy[3] === original[3]); // false

// 4. Копирование с преобразованием
const transformedCopy = original.map(item =>

```

```
    typeof item === 'object' ? { ...item } : item
  );
```

// 5. Копирование части массива

```
const partialCopy = original.slice(1, 3);
console.log(partialCopy); // ["b", "c"]
```

Часть 2: Основные операции с массивами

2.1. Доступ к элементам

javascript

```
const colors = ["красный", "зеленый", "синий", "желтый", "фиолетовый"];
```

// 1. Прямой доступ по индексу

```
console.log(colors[0]); // "красный" - первый элемент
console.log(colors[2]); // "синий" - третий элемент
console.log(colors[colors.length - 1]); // "фиолетовый" - последний элемент
```

// 2. Доступ с использованием переменных

```
const index = 1;
console.log(colors[index]); // "зеленый"
```

// 3. Проверка существования индекса

```
console.log(colors[10]); // undefined
console.log(10 in colors); // false
```

// 4. Итеративный доступ

```
for (let i = 0; i < colors.length; i++) {
  console.log(`Элемент ${i}: ${colors[i]}`);
}
```

// 5. Доступ к свойствам массива как объекта

```
colors.customProperty = "специальное свойство";
console.log(colors.customProperty); // "специальное свойство"
console.log(colors.length); // 5 - не изменяется
console.log(Object.keys(colors)); // ["0", "1", "2", "3", "4", "customProperty"]
```

2.2. Изменение размера массива

javascript

```

let array = [1, 2, 3];

// 1. Увеличение длины
array.length = 5;
console.log(array); // [1, 2, 3, empty × 2]
console.log(array[3]); // undefined

// 2. Уменьшение длины (усечение)
array.length = 2;
console.log(array); // [1, 2]
console.log(array[2]); // undefined

// 3. Сброс длины без изменения
array.length = 0;
console.log(array); // []
console.log(array.length); // 0

// 4. Автоматическое изменение длины при добавлении
array = [1, 2, 3];
array[10] = 11;
console.log(array.length); // 11
console.log(array); // [1, 2, 3, empty × 7, 11]

// 5. Безопасное увеличение длины
function ensureLength(arr, minLength, filler = undefined) {
    if (arr.length < minLength) {
        arr.length = minLength;
        // Заполнение пустых слотов
        for (let i = 0; i < arr.length; i++) {
            if (!(i in arr)) {
                arr[i] = filler;
            }
        }
    }
    return arr;
}

const arr = [1, 2];
ensureLength(arr, 5, 0);
console.log(arr); // [1, 2, 0, 0, 0]

```

2.3. Добавление элементов

javascript

```
let fruits = ["яблоко", "банан"];

// 1. push() - добавление в конец (мутирующий)
let newLength = fruits.push("апельсин");
console.log(fruits); // ["яблоко", "банан", "апельсин"]
console.log(newLength); // 3

// Добавление нескольких элементов
newLength = fruits.push("киви", "манго");
console.log(fruits); // ["яблоко", "банан", "апельсин", "киви", "манго"]

// 2. unshift() - добавление в начало (мутирующий)
newLength = fruits.unshift("персик");
console.log(fruits); // ["персик", "яблоко", "банан", "апельсин", "киви", "манго"]

// 3. Добавление по индексу (с проверкой)
function insertAt(array, index, ...elements) {
  if (index < 0 || index > array.length) {
    throw new Error(`Индекс ${index} вне диапазона [0, ${array.length}]`);
  }

  // Сдвигаем существующие элементы
  for (let i = array.length - 1; i >= index; i--) {
    array[i + elements.length] = array[i];
  }

  // Вставляем новые элементы
  for (let i = 0; i < elements.length; i++) {
    array[index + i] = elements[i];
  }

  return array.length;
}

insertAt(fruits, 2, "лимон", "лайм");
console.log(fruits); // ["персик", "яблоко", "лимон", "лайм", "банан", "апельсин", "киви", "манго"]

// 4. Spread оператор (не мутирующий)
const newFruits = ["груша", ...fruits, "ананас"];
```

```
console.log(newFruits); // ["груша", "персик", "яблоко", "лимон", "лайм", "банан",  
"апельсин", "киви", "манго", "ананас"]
```

```
// 5. concat() (не мутлирующий)
```

```
const moreFruits = fruits.concat(["виноград", "арбуз", "дыня"]);
```

```
console.log(moreFruits); // Массив с 11 элементами
```

```
console.log(fruits); // Исходный массив не изменился
```

2.4. Удаление элементов

javascript

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
// 1. pop() - удаление с конца (мутлирующий)
```

```
const lastElement = numbers.pop();
```

```
console.log(lastElement); // 10
```

```
console.log(numbers); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
// 2. shift() - удаление с начала (мутлирующий)
```

```
const firstElement = numbers.shift();
```

```
console.log(firstElement); // 1
```

```
console.log(numbers); // [2, 3, 4, 5, 6, 7, 8, 9]
```

```
// 3. splice() - удаление по индексу (мутлирующий)
```

```
const removed = numbers.splice(2, 3); // Удалить 3 элемента начиная с индекса 2
```

```
console.log(removed); // [4, 5, 6]
```

```
console.log(numbers); // [2, 3, 7, 8, 9]
```

```
// 4. delete оператор (оставляет "дырку")
```

```
delete numbers[1];
```

```
console.log(numbers); // [2, empty, 7, 8, 9]
```

```
console.log(numbers.length); // 5 (не изменилось!)
```

```
console.log(1 in numbers); // false
```

```
// 5. filter() - удаление по условию (не мутлирующий)
```

```
const evenNumbers = numbers.filter((n, i, arr) => {  
    console.log(`Проверка элемента ${n} по индексу ${i}`);  
    return n % 2 === 0;  
});
```

```
});
```

```
console.log(evenNumbers); // [2]
```

```
console.log(numbers); // [2, empty, 7, 8, 9] - исходный массив не изменился
```

```
// 6. Удаление с сохранением в новый массив
```

```

function removeElements(array, ...indices) {
    const removed = [];
    const result = [];

    // Сортируем индексы по убыванию для безопасного удаления
    const sortedIndices = [...indices].sort((a, b) => b - a);

    // Создаем копию массива
    const arrayCopy = [...array];

    for (const index of sortedIndices) {
        if (index >= 0 && index < arrayCopy.length) {
            removed.unshift(arrayCopy[index]); // Сохраняем в порядке удаления
            arrayCopy.splice(index, 1);
        }
    }

    return {
        removed,
        result: arrayCopy
    };
}

const original = [10, 20, 30, 40, 50];
const { removed: removedItems, result: newArray } = removeElements(original, 1, 3);
console.log(removedItems); // [20, 40]
console.log(newArray); // [10, 30, 50]
console.log(original); // [10, 20, 30, 40, 50] - не изменился

```

2.5. Изменение элементов

```

javascript

let data = ["a", "b", "c", "d", "e"];

// 1. Прямое присваивание
data[2] = "C";
console.log(data); // ["a", "b", "C", "d", "e"]

data[10] = "K"; // Создает разреженный массив
console.log(data); // ["a", "b", "C", "d", "e", empty × 5, "K"]
console.log(data.length); // 11

// 2. splice() для замены

```



```

data.splice(1, 2, "B", "C"); // Заменить 2 элемента начиная с индекса 1
console.log(data); // ["a", "B", "C", "d", "e", empty × 5, "K"]

// 3. map() для преобразования всех элементов
const transformed = data.map((item, index) =>
  item ? item.toUpperCase() : `empty_${index}`
);
console.log(transformed); // ["A", "B", "C", "D", "E", "empty_5", ...]

// 4. Изменение по условию
function updateIf(array, condition, updater) {
  return array.map((item, index) =>
    condition(item, index, array) ? updater(item, index, array) : item
  );
}

const updated = updateIf(
  [1, 2, 3, 4, 5],
  n => n % 2 === 0,
  n => n * 10
);
console.log(updated); // [1, 20, 3, 40, 5]

// 5. Обмен элементов
function swapElements(array, index1, index2) {
  if (index1 < 0 || index1 >= array.length ||
    index2 < 0 || index2 >= array.length) {
    throw new Error("Индекс вне диапазона");
  }

  [array[index1], array[index2]] = [array[index2], array[index1]];
  return array;
}

const arr = [1, 2, 3, 4, 5];
swapElements(arr, 0, 4);
console.log(arr); // [5, 2, 3, 4, 1]

```

Часть 3: Поиск и проверка элементов

3.1. Линейный поиск

```
javascript

const numbers = [10, 20, 30, 40, 50, 30, 20, 10];

// 1. indexOf() - поиск первого вхождения
console.log(numbers.indexOf(30)); // 2
console.log(numbers.indexOf(30, 3)); // 5 - поиск начиная с индекса 3
console.log(numbers.indexOf(100)); // -1 - не найден

// 2. lastIndexOf() - поиск последнего вхождения
console.log(numbers.lastIndexOf(20)); // 6
console.log(numbers.lastIndexOf(20, 5)); // 1 - поиск до индекса 5

// 3. includes() - проверка наличия
console.log(numbers.includes(40)); // true
console.log(numbers.includes(40, 5)); // false - начиная с индекса 5

// 4. find() - поиск по условию
const firstEven = numbers.find(n => n % 20 === 0);
console.log(firstEven); // 20

const users = [
  { id: 1, name: "Анна", active: true },
  { id: 2, name: "Борис", active: false },
  { id: 3, name: "Виктор", active: true }
];

const activeUser = users.find(user => user.active);
console.log(activeUser); // {id: 1, name: "Анна", active: true}

// 5. findIndex() - поиск индекса по условию
const firstInactiveIndex = users.findIndex(user => !user.active);
console.log(firstInactiveIndex); // 1

// 6. findLast() и findLastIndex() (ES2023+)
console.log(numbers.findLast(n => n === 30)); // 30
console.log(numbers.findLastIndex(n => n === 20)); // 6

// 7. Пользовательские функции поиска
```

```
function findAllIndices(array, predicate) {
  const indices = [];
  for (let i = 0; i < array.length; i++) {
    if (predicate(array[i], i, array)) {
      indices.push(i);
    }
  }
  return indices;
}

console.log(findAllIndices(numbers, n => n === 30)); // [2, 5]
console.log(findAllIndices(numbers, n => n > 25)); // [2, 3, 4, 5]
```

3.2. Проверка условий

```
javascript

const data = [2, 4, 6, 8, 10];

// 1. every() - все элементы удовлетворяют условию
const allEven = data.every(n => n % 2 === 0);
console.log(allEven); // true

const allGreaterThan5 = data.every(n => n > 5);
console.log(allGreaterThan5); // false

// 2. some() - хотя бы один элемент удовлетворяет условию
const hasGreaterThan8 = data.some(n => n > 8);
console.log(hasGreaterThan8); // true

const hasOdd = data.some(n => n % 2 !== 0);
console.log(hasOdd); // false

// 3. Практические примеры
const products = [
  { name: "Ноутбук", price: 50000, inStock: true },
  { name: "Мышь", price: 1500, inStock: false },
  { name: "Клавиатура", price: 3000, inStock: true }
];

// Проверка, все ли товары в наличии
const allInStock = products.every(p => p.inStock);
console.log("Все товары в наличии:", allInStock); // false
```

```

// Проверка, есть ли дорогие товары
const hasExpensive = products.some(p => p.price > 10000);
console.log("Есть дорогие товары:", hasExpensive); // true

// 4. Комбинированные проверки
function validateArray(array, validators) {
    const results = {};

    for (const [name, validator] of Object.entries(validators)) {
        results[name] = validator(array);
    }

    return results;
}

const validators = {
    hasNumbers: arr => arr.some(item => typeof item === 'number'),
    allStrings: arr => arr.every(item => typeof item === 'string'),
    noEmptyStrings: arr => arr.every(item => item !== ''),
    hasAtLeastThree: arr => arr.length >= 3
};

const testArray = ["яблоко", "банан", ""];
const validation = validateArray(testArray, validators);
console.log(validation);
// {
//   hasNumbers: false,
//   allStrings: true,
//   noEmptyStrings: false,
//   hasAtLeastThree: true
// }

```

Часть 4: Преобразование массивов

4.1. Сортировка

```

javascript

// 1. Базовая сортировка (мутирующая)
const fruits = ["банан", "яблоко", "апельсин", "киви"];
fruits.sort();
console.log(fruits); // ["апельсин", "банан", "киви", "яблоко"] (по алфавиту)

```

```
// 2. Сортировка чисел (по умолчанию лексикографическая)
const numbers = [10, 2, 100, 5, 1];
numbers.sort();
console.log(numbers); // [1, 10, 100, 2, 5] - неправильно для чисел!
```

```
// 3. Правильная сортировка чисел
numbers.sort((a, b) => a - b);
console.log(numbers); // [1, 2, 5, 10, 100] - по возрастанию
```

```
numbers.sort((a, b) => b - a);
console.log(numbers); // [100, 10, 5, 2, 1] - по убыванию
```

```
// 4. Сортировка объектов
```

```
const users = [
  { name: "Анна", age: 25 },
  { name: "Борис", age: 30 },
  { name: "Виктор", age: 20 },
  { name: "Галина", age: 25 }
];
```

```
// Сортировка по возрасту, затем по имени
```

```
users.sort((a, b) => {
  if (a.age !== b.age) {
    return a.age - b.age; // по возрасту
  }
  return a.name.localeCompare(b.name); // по имени
});
```

```
console.log(users);
```

```
// [
//   {name: "Виктор", age: 20},
//   {name: "Анна", age: 25},
//   {name: "Галина", age: 25},
//   {name: "Борис", age: 30}
// ]
```

```
// 5. Стабильная сортировка (ES2019+)
```

```
const data = [
  { value: 1, order: 1 },
  { value: 2, order: 2 },
  { value: 1, order: 3 },
  { value: 3, order: 4 }
];
```

```

// Стабильная сортировка сохраняет порядок равных элементов
data.sort((a, b) => a.value - b.value);
console.log(data.map(d => d.order)); // [1, 3, 2, 4] - порядок 1 и 3 сохранен

// 6. Создание отсортированной копии (не мутирующий)
function sortedCopy(array, compareFunction) {
    return [...array].sort(compareFunction);
}

const original = [3, 1, 4, 1, 5, 9];
const sorted = sortedCopy(original, (a, b) => a - b);
console.log(original); // [3, 1, 4, 1, 5, 9] - не изменился
console.log(sorted); // [1, 1, 3, 4, 5, 9]

// 7. Сложные критерии сортировки
const complexData = [
    { priority: "high", date: "2024-01-15" },
    { priority: "low", date: "2024-01-10" },
    { priority: "medium", date: "2024-01-12" },
    { priority: "high", date: "2024-01-10" }
];

const priorityOrder = { high: 1, medium: 2, low: 3 };

complexData.sort((a, b) => {
    // Сначала по приоритету
    const priorityDiff = priorityOrder[a.priority] - priorityOrder[b.priority];
    if (priorityDiff !== 0) return priorityDiff;

    // Затем по дате
    return new Date(a.date) - new Date(b.date);
});

console.log(complexData);

```

4.2. Реверс и перестановки

```

javascript

// 1. reverse() - обратный порядок (мутирующий)
const numbers = [1, 2, 3, 4, 5];
numbers.reverse();
console.log(numbers); // [5, 4, 3, 2, 1]

```

// 2. Создание обратной копии

```
function reversedCopy(array) {  
    return [...array].reverse();  
}
```

```
const original = [1, 2, 3];  
const reversed = reversedCopy(original);  
console.log(original); // [1, 2, 3] - не изменился  
console.log(reversed); // [3, 2, 1]
```

// 3. Перемешивание (shuffle)

```
function shuffleArray(array) {  
    const shuffled = [...array];  
    for (let i = shuffled.length - 1; i > 0; i--) {  
        const j = Math.floor(Math.random() * (i + 1));  
        [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]];  
    }  
    return shuffled;  
}
```

```
const deck = ["♠A", "♠K", "♠Q", "♠J", "♠10"];  
console.log(shuffleArray(deck));  
console.log(shuffleArray(deck)); // Разные результаты
```

// 4. Ротация (циклический сдвиг)

```
function rotateArray(array, positions) {  
    const n = array.length;  
    if (n === 0) return [];  
  
    // Нормализация позиций  
    positions = ((positions % n) + n) % n;  
  
    return [  
        ...array.slice(positions),  
        ...array.slice(0, positions)  
    ];  
}
```

```
const arr = [1, 2, 3, 4, 5];  
console.log(rotateArray(arr, 2)); // [3, 4, 5, 1, 2] - сдвиг влево  
console.log(rotateArray(arr, -1)); // [5, 1, 2, 3, 4] - сдвиг вправо  
console.log(rotateArray(arr, 7)); // [3, 4, 5, 1, 2] - эквивалентно 2
```

```
// 5. Транспонирование матрицы
function transposeMatrix(matrix) {
    if (matrix.length === 0) return [];

    const rows = matrix.length;
    const cols = matrix[0].length;

    // Проверка, что все строки одинаковой длины
    if (!matrix.every(row => row.length === cols)) {
        throw new Error("Некорректная матрица");
    }

    const transposed = [];
    for (let j = 0; j < cols; j++) {
        transposed[j] = [];
        for (let i = 0; i < rows; i++) {
            transposed[j][i] = matrix[i][j];
        }
    }

    return transposed;
}

const matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];

console.log(transposeMatrix(matrix));
// [[1, 4, 7],
//   [2, 5, 8],
//   [3, 6, 9]]
```

4.3. Преобразование в строки

javascript

```
const fruits = ["яблоко", "банан", "апельсин"];

// 1. join() - объединение в строку
console.log(fruits.join()); // "яблоко,банан,апельсин"
console.log(fruits.join(", ")); // "яблоко, банан, апельсин"
console.log(fruits.join(" - ")); // "яблоко - банан - апельсин"
```



```

console.log(fruits.join("")); // "яблокобанананпельсин"

// 2. toString() - строковое представление
console.log(fruits.toString()); // "яблоко, банан, апельсин"
console.log([1, 2, [3, 4]].toString()); // "1,2,3,4"

// 3. toLocaleString() - локализованное представление
const numbers = [1000.50, 2000.75, 3000.25];
console.log(numbers.toLocaleString('ru-RU')); // "1 000,5,2 000,75,3 000,25"
console.log(numbers.toLocaleString('en-US', {
    style: 'currency',
    currency: 'USD'
})); // "$1,000.50,$2,000.75,$3,000.25"

const dates = [new Date(), new Date('2024-01-01')];
console.log(dates.toLocaleString('ru-RU')); // "даты в русском формате"

// 4. JSON.stringify() - сериализация
const complexArray = [
    "строка",
    42,
    true,
    null,
    { name: "объект", data: [1, 2, 3] },
    [4, 5, 6]
];

console.log(JSON.stringify(complexArray));
// '["строка",42,true,null,{"name":"объект","data":[1,2,3]},[4,5,6]]'

console.log(JSON.stringify(complexArray, null, 2));
// Красивое форматирование с отступами

// 5. Кастомное форматирование
function formatArray(array, options = {}) {
    const {
        separator = ", ",
        lastSeparator = " и ",
        prefix = "[",
        suffix = "]",
        transform = x => x
    } = options;

    if (array.length === 0) return prefix + suffix;

```

```

    if (array.length === 1) {
        return prefix + transform(array[0]) + suffix;
    }

    const firstPart = array.slice(0, -1)
        .map(transform)
        .join(separator);

    const lastElement = transform(array[array.length - 1]);

    return prefix + firstPart + lastSeparator + lastElement + suffix;
}

console.log(formatArray(fruits));
// "[яблоко, банан и апельсин]"

console.log(formatArray(fruits, {
    lastSeparator: " или ",
    prefix: "Выберите: ",
    suffix: "."
}));
// "Выберите: яблоко, банан или апельсин."

console.log(formatArray(fruits, {
    transform: fruit => fruit.toUpperCase()
}));
// "[ЯБЛОКО, БАНАН И АПЕЛЬСИН]"

```

Часть 5: Работа с подмассивами

5.1. Извлечение подмассивов

```

javascript

const numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];

// 1. slice() - извлечение части массива (не мутирующий)
console.log(numbers.slice(2, 5)); // [2, 3, 4] - с 2 до 5 (не включая 5)
console.log(numbers.slice(4)); // [4, 5, 6, 7, 8, 9] - с 4 до конца
console.log(numbers.slice(-3)); // [7, 8, 9] - последние 3 элемента
console.log(numbers.slice(2, -2)); // [2, 3, 4, 5, 6, 7] - с 2 до предпоследнего

```

```

console.log(numbers.slice()); // Копия всего массива
console.log(numbers); // Исходный массив не изменился

// 2. splice() - извлечение с удалением (мутирующий)
const arr = [0, 1, 2, 3, 4, 5];
const extracted = arr.splice(2, 3); // Удалить 3 элемента начиная с индекса 2
console.log(extracted); // [2, 3, 4]
console.log(arr); // [0, 1, 5] - исходный массив изменился

// 3. Извлечение по условию
function extractByCondition(array, condition) {
    const extracted = [];
    const remaining = [];

    for (const item of array) {
        if (condition(item)) {
            extracted.push(item);
        } else {
            remaining.push(item);
        }
    }

    return { extracted, remaining };
}

const data = [1, 2, 3, 4, 5, 6];
const { extracted: evens, remaining: odds } = extractByCondition(
    data,
    n => n % 2 === 0
);
console.log(evens); // [2, 4, 6]
console.log(odds); // [1, 3, 5]

// 4. Разбиение на чанки
function chunkArray(array, chunkSize) {
    const chunks = [];
    for (let i = 0; i < array.length; i += chunkSize) {
        chunks.push(array.slice(i, i + chunkSize));
    }
    return chunks;
}

const largeArray = [1, 2, 3, 4, 5, 6, 7, 8, 9];
console.log(chunkArray(largeArray, 3));

```

```
// [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

console.log(chunkArray(largeArray, 4));
// [[1, 2, 3, 4], [5, 6, 7, 8], [9]]

// 5. Скользящее окно (sliding window)
function slidingWindow(array, windowSize) {
    if (windowSize > array.length) return [];

    const windows = [];
    for (let i = 0; i <= array.length - windowSize; i++) {
        windows.push(array.slice(i, i + windowSize));
    }
    return windows;
}

console.log(slidingWindow([1, 2, 3, 4, 5], 3));
// [[1, 2, 3], [2, 3, 4], [3, 4, 5]]
```

5.2. Объединение массивов

```
javascript

const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const arr3 = [7, 8, 9];

// 1. concat() - объединение массивов (не мутующий)
const combined1 = arr1.concat(arr2);
console.log(combined1); // [1, 2, 3, 4, 5, 6]

const combined2 = arr1.concat(arr2, arr3);
console.log(combined2); // [1, 2, 3, 4, 5, 6, 7, 8, 9]

const combined3 = arr1.concat(10, 11, [12, 13]);
console.log(combined3); // [1, 2, 3, 10, 11, 12, 13]

console.log(arr1); // [1, 2, 3] - не изменился

// 2. Spread оператор (ES6+)
const combined4 = [...arr1, ...arr2];
console.log(combined4); // [1, 2, 3, 4, 5, 6]

const combined5 = [0, ...arr1, 3.5, ...arr2, 10];
```

```
console.log(combined5); // [0, 1, 2, 3, 3.5, 4, 5, 6, 10]
```

```
// 3. Объединение с обработкой
```

```
function mergeArrays(arrays, mergeFunction = (a, b) => b) {  
  if (arrays.length === 0) return [];  
  
  let result = [...arrays[0]];  
  
  for (let i = 1; i < arrays.length; i++) {  
    result = result.map((item, index) => {  
      if (index < arrays[i].length) {  
        return mergeFunction(item, arrays[i][index]);  
      }  
      return item;  
    });  
  
    // Добавляем лишние элементы из текущего массива  
    if (arrays[i].length > result.length) {  
      result.push(...arrays[i].slice(result.length));  
    }  
  }  
  
  return result;  
}
```

```
const a = [1, 2, 3];  
const b = [10, 20, 30, 40];  
console.log(mergeArrays([a, b])); // [10, 20, 30, 40]  
console.log(mergeArrays([a, b], (x, y) => x + y)); // [11, 22, 33, 40]
```

```
// 4. Удаление дубликатов при объединении
```

```
function union(...arrays) {  
  const combined = [].concat(...arrays);  
  return [...new Set(combined)];  
}
```

```
console.log(union([1, 2, 3], [2, 3, 4], [3, 4, 5])); // [1, 2, 3, 4, 5]
```

```
// 5. Слияние объектов в массивах
```

```
function mergeObjectArrays(arrays, key) {  
  const map = new Map();  
  
  for (const array of arrays) {  
    for (const obj of array) {
```

```

    const objKey = obj[key];
    if (map.has(objKey)) {
        // Объединение объектов
        map.set(objKey, { ...map.get(objKey), ...obj });
    } else {
        map.set(objKey, { ...obj });
    }
}

return Array.from(map.values());
}

const users1 = [
    { id: 1, name: "Анна", age: 25 },
    { id: 2, name: "Борис", age: 30 }
];

const users2 = [
    { id: 2, city: "Москва" },
    { id: 3, name: "Виктор", age: 20 }
];

console.log(mergeObjectArrays([users1, users2], 'id'));
// [
//   {id: 1, name: "Анна", age: 25},
//   {id: 2, name: "Борис", age: 30, city: "Москва"},
//   {id: 3, name: "Виктор", age: 20}
// ]

```

Часть 6: Продвинутые техники манипуляции

6.1. Работа с многомерными массивами

```

javascript

// 1. Создание и доступ
const matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];

```

```
console.log(matrix[1][2]); // 6
console.log(matrix[2][0]); // 7
```

// 2. Итерация по многомерному массиву

```
function iterateMatrix(matrix) {
    const result = [];

    for (let i = 0; i < matrix.length; i++) {
        for (let j = 0; j < matrix[i].length; j++) {
            result.push({
                value: matrix[i][j],
                row: i,
                col: j
            });
        }
    }

    return result;
}
```

```
console.log(iterateMatrix(matrix));
```

// 3. Плоское преобразование (flatten)

```
const nested = [1, [2, [3, [4, 5]]]];
```

// Рекурсивное сглаживание

```
function flattenDeep(array) {
    return array.reduce((acc, val) =>
        Array.isArray(val) ? acc.concat(flattenDeep(val)) : acc.concat(val),
        []);
}
```

```
console.log(flattenDeep(nested)); // [1, 2, 3, 4, 5]
```

// Сглаживание на определенную глубину

```
function flattenDepth(array, depth = 1) {
    if (depth === 0) return array.slice();

    return array.reduce((acc, val) =>
        Array.isArray(val) ?
            acc.concat(flattenDepth(val, depth - 1)) :
            acc.concat(val),
        []);
}
```

```

}

console.log(flattenDepth(nested, 1)); // [1, 2, [3, [4, 5]]]
console.log(flattenDepth(nested, 2)); // [1, 2, 3, [4, 5]]

// 4. Современный метод flat() (ES2019)
console.log(nested.flat()); // [1, 2, [3, [4, 5]]] - глубина 1 по умолчанию
console.log(nested.flat(2)); // [1, 2, 3, [4, 5]]
console.log(nested.flat(Infinity)); // [1, 2, 3, 4, 5]

// 5. Трансформация многомерных массивов
function transformMatrix(matrix, transformer) {
    return matrix.map((row, i) =>
        row.map((cell, j) => transformer(cell, i, j, matrix))
    );
}

const transformed = transformMatrix(matrix, (value, row, col) =>
    value * (row + 1) + col
);
console.log(transformed);
// [
//   [1*1+0, 2*1+1, 3*1+2] = [1, 3, 5],
//   [4*2+0, 5*2+1, 6*2+2] = [8, 11, 14],
//   [7*3+0, 8*3+1, 9*3+2] = [21, 25, 29]
// ]

// 6. Поиск в многомерном массиве
function findInMatrix(matrix, predicate) {
    for (let i = 0; i < matrix.length; i++) {
        for (let j = 0; j < matrix[i].length; j++) {
            if (predicate(matrix[i][j], i, j, matrix)) {
                return { value: matrix[i][j], row: i, col: j };
            }
        }
    }
    return null;
}

const found = findInMatrix(matrix, value => value > 5);
console.log(found); // {value: 6, row: 1, col: 2}

```


6.2. Операции над множествами

javascript

```
const setA = [1, 2, 3, 4, 5];
```

```
const setB = [4, 5, 6, 7, 8];
```

// 1. Объединение (union)

```
function union(a, b) {  
    return [...new Set([...a, ...b])];  
}
```

```
console.log(union(setA, setB)); // [1, 2, 3, 4, 5, 6, 7, 8]
```

// 2. Пересечение (intersection)

```
function intersection(a, b) {  
    const setB = new Set(b);  
    return a.filter(x => setB.has(x));  
}
```

```
console.log(intersection(setA, setB)); // [4, 5]
```

// 3. Разность (difference) - $A \setminus B$

```
function difference(a, b) {  
    const setB = new Set(b);  
    return a.filter(x => !setB.has(x));  
}
```

```
console.log(difference(setA, setB)); // [1, 2, 3]
```

```
console.log(difference(setB, setA)); // [6, 7, 8]
```

// 4. Симметрическая разность (symmetric difference)

```
function symmetricDifference(a, b) {  
    return [  
        ...difference(a, b),  
        ...difference(b, a)  
    ];  
}
```

```
console.log(symmetricDifference(setA, setB)); // [1, 2, 3, 6, 7, 8]
```

// 5. Подмножество (subset)

```
function isSubset(subset, superset) {  
    const supersetSet = new Set(superset);  
    return subset.every(x => supersetSet.has(x));  
}
```

```
console.log(isSubset([2, 4], setA)); // true
```

```
console.log(isSubset([2, 6], setA)); // false
```

```

// 6. Декартово произведение (cartesian product)
function cartesianProduct(a, b) {
    const result = [];
    for (const x of a) {
        for (const y of b) {
            result.push([x, y]);
        }
    }
    return result;
}
console.log(cartesianProduct([1, 2], ['a', 'b']));
// [[1, 'a'], [1, 'b'], [2, 'a'], [2, 'b']]

// 7. Уникальные элементы по критерию
function uniqueBy(array, keySelector) {
    const seen = new Set();
    return array.filter(item => {
        const key = keySelector(item);
        if (seen.has(key)) {
            return false;
        }
        seen.add(key);
        return true;
    });
}

const users = [
    { id: 1, name: "Анна" },
    { id: 2, name: "Борис" },
    { id: 1, name: "Анна" }, // Дубликат
    { id: 3, name: "Виктор" }
];

console.log(uniqueBy(users, user => user.id));
// [{id: 1, name: "Анна"}, {id: 2, name: "Борис"}, {id: 3, name: "Виктор"}]

```

6.3. Производительные операции

javascript

```

// 1. Предварительное выделение памяти
function createLargeArray(size) {
    // Неэффективно: многократное изменение размера

```

```

const arr = [];
for (let i = 0; i < size; i++) {
    arr.push(i);
}
return arr;

// Эффективно: предварительное выделение
// const arr = new Array(size);
// for (let i = 0; i < size; i++) {
//     arr[i] = i;
// }
// return arr;
}

// 2. Батчинговые операции
function processInBatches(array, batchSize, processor) {
    const results = [];

    for (let i = 0; i < array.length; i += batchSize) {
        const batch = array.slice(i, i + batchSize);
        const batchResults = processor(batch);
        results.push(...batchResults);

        // Даем браузеру "передохнуть" между батчами
        if (i + batchSize < array.length) {
            // Используем setTimeout для неблокирующей обработки
            await new Promise(resolve => setTimeout(resolve, 0));
        }
    }

    return results;
}

// 3. Кэширование вычислений
function createCachedTransformer(transformer) {
    const cache = new Map();

    return function(array) {
        const key = array.join(',');

        if (cache.has(key)) {
            console.log('Из кэша:', key);
            return cache.get(key);
        }
    }
}

```

```

    console.log('Вычисляем:', key);
    const result = transformer(array);
    cache.set(key, result);

    // Ограничение размера кэша
    if (cache.size > 1000) {
        const firstKey = cache.keys().next().value;
        cache.delete(firstKey);
    }

    return result;
};
}

const cachedSum = createCachedTransformer(arr =>
    arr.reduce((a, b) => a + b, 0)
);

console.log(cachedSum([1, 2, 3])); // Вычисляем: 1,2,3 → 6
console.log(cachedSum([1, 2, 3])); // Из кэша: 1,2,3 → 6

// 4. Ленивые вычисления
function createLazyArray(generator, length) {
    return {
        length,
        [Symbol.iterator]: function* () {
            for (let i = 0; i < length; i++) {
                yield generator(i);
            }
        },
        map: function mapper() {
            const self = this;
            return createLazyArray(
                i => mapper(generator(i), i),
                length
            );
        },
        filter: function(predicate) {
            const self = this;
            let filteredLength = 0;
            const filteredIndices = [];

            for (let i = 0; i < length; i++) {

```

```

        if (predicate(generator(i), i)) {
            filteredIndices[filteredLength++] = i;
        }
    }

    return {
        length: filteredLength,
        [Symbol.iterator]: function* () {
            for (let i = 0; i < filteredLength; i++) {
                yield generator(filteredIndices[i]);
            }
        }
    };
},
toArray: function() {
    return Array.from(this);
}
};
}

```

```

const lazyNumbers = createLazyArray(i => i * i, 5);
console.log(lazyNumbers.toArray()); // [0, 1, 4, 9, 16]

```

```

const filtered = lazyNumbers.filter(x => x > 5);
console.log(filtered.toArray()); // [9, 16]

```

```

const mapped = lazyNumbers.map(x => x * 2);
console.log(mapped.toArray()); // [0, 2, 8, 18, 32]

```

Часть 7: Практические примеры и паттерны

7.1. Обработка данных

```

javascript

// 1. Агрегация данных
const orders = [
    { id: 1, product: "Ноутбук", price: 50000, quantity: 1 },
    { id: 2, product: "Мышь", price: 1500, quantity: 2 },
    { id: 3, product: "Клавиатура", price: 3000, quantity: 1 },
    { id: 4, product: "Монитор", price: 25000, quantity: 1 }
];

```

```
// Сумма заказов
const totalRevenue = orders.reduce((sum, order) =>
  sum + order.price * order.quantity, 0
);
console.log("Общая выручка:", totalRevenue); // 50000 + 3000 + 3000 + 25000 = 81000
```

```
// Группировка по категориям (предположим, что есть категории)
const productsByCategory = orders.reduce((groups, order) => {
  const category = order.price > 10000 ? "Дорогие" : "Бюджетные";
  if (!groups[category]) groups[category] = [];
  groups[category].push(order);
  return groups;
}, {});
console.log(productsByCategory);
```

// 2. Пагинация

```
function paginate(array, page, pageSize) {
  const totalPages = Math.ceil(array.length / pageSize);

  if (page < 1 || page > totalPages) {
    throw new Error(`Страница ${page} вне диапазона [1, ${totalPages}]`);
  }

  const startIndex = (page - 1) * pageSize;
  const endIndex = Math.min(startIndex + pageSize, array.length);

  return {
    data: array.slice(startIndex, endIndex),
    page,
    pageSize,
    total: array.length,
    totalPages,
    hasNext: page < totalPages,
    hasPrev: page > 1
  };
}
```

```
const allItems = Array.from({ length: 47 }, (_, i) => `Item ${i + 1}`);
const page1 = paginate(allItems, 1, 10);
console.log(page1);
```

// 3. Сортировка и фильтрация с кэшированием

```
class DataProcessor {
```

```

constructor(data) {
    this.data = data;
    this.cache = new Map();
}

getSorted(compareFunction, key = 'default') {
    const cacheKey = `sorted:${key}:${JSON.stringify(compareFunction)}`;

    if (this.cache.has(cacheKey)) {
        console.log('Возвращаем отсортированные данные из кэша');
        return this.cache.get(cacheKey);
    }

    console.log('Сортируем данные...');
    const sorted = [...this.data].sort(compareFunction);
    this.cache.set(cacheKey, sorted);

    return sorted;
}

getFiltered(predicate, key = 'default') {
    const cacheKey = `filtered:${key}:${predicate.toString()}`;

    if (this.cache.has(cacheKey)) {
        console.log('Возвращаем отфильтрованные данные из кэша');
        return this.cache.get(cacheKey);
    }

    console.log('Фильтруем данные...');
    const filtered = this.data.filter(predicate);
    this.cache.set(cacheKey, filtered);

    return filtered;
}

clearCache() {
    this.cache.clear();
}
}

```

7.2. Алгоритмические задачи

javascript

// 1. Поиск пары чисел с заданной суммой

```
function findPairWithSum(numbers, targetSum) {  
    const seen = new Set();  
  
    for (const num of numbers) {  
        const complement = targetSum - num;  
  
        if (seen.has(complement)) {  
            return [complement, num];  
        }  
  
        seen.add(num);  
    }  
  
    return null;  
}  
  
console.log(findPairWithSum([1, 2, 3, 4, 5], 9)); // [4, 5]  
console.log(findPairWithSum([1, 2, 3, 4, 5], 10)); // null
```

// 2. Вращение массива

```
function rotateArrayOptimal(array, k) {  
    const n = array.length;  
    k = k % n;  
  
    if (k === 0) return array;  
  
    // Реверс всего массива  
    reverse(array, 0, n - 1);  
    // Реверс первых k элементов  
    reverse(array, 0, k - 1);  
    // Реверс оставшихся элементов  
    reverse(array, k, n - 1);  
  
    return array;  
  
    function reverse(arr, start, end) {  
        while (start < end) {  
            [arr[start], arr[end]] = [arr[end], arr[start]];  
            start++;  
            end--;  
        }  
    }  
}
```



```
const arr = [1, 2, 3, 4, 5, 6, 7];
console.log(rotateArrayOptimal([...arr], 3)); // [5, 6, 7, 1, 2, 3, 4]
```

// 3. Поиск пропущенного числа

```
function findMissingNumber(numbers) {
    const n = numbers.length + 1; // Одно число пропущено
    const expectedSum = n * (n + 1) / 2;
    const actualSum = numbers.reduce((a, b) => a + b, 0);

    return expectedSum - actualSum;
}
```

```
console.log(findMissingNumber([1, 2, 4, 5, 6])); // 3
```

// 4. Слияние отсортированных массивов

```
function mergeSortedArrays(arr1, arr2) {
    const result = [];
    let i = 0, j = 0;

    while (i < arr1.length && j < arr2.length) {
        if (arr1[i] < arr2[j]) {
            result.push(arr1[i++]);
        } else {
            result.push(arr2[j++]);
        }
    }

    // Добавляем оставшиеся элементы
    while (i < arr1.length) result.push(arr1[i++]);
    while (j < arr2.length) result.push(arr2[j++]);

    return result;
}
```

```
console.log(mergeSortedArrays([1, 3, 5], [2, 4, 6])); // [1, 2, 3, 4, 5, 6]
```

Заключение: лучшие практики работы с массивами

Ключевые принципы:

1. Выбор правильного метода:

- ➔ Мутлирующие методы: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`
- ➔ Не мутлирующие методы: `concat`, `slice`, `map`, `filter`, `reduce`
- ➔ Методы поиска: `find`, `findIndex`, `indexOf`, `includes`

2. Производительность:

- ➔ Избегайте вложенных циклов при работе с большими массивами
- ➔ Используйте `Set` для быстрой проверки наличия элементов
- ➔ Предварительно выделяйте память для больших массивов
- ➔ Кэшируйте результаты сложных вычислений

3. Читаемость кода:

- ➔ Используйте цепочки методов для последовательных преобразований
- ➔ Выносите сложную логику в отдельные функции
- ➔ Комментируйте нетривиальные операции

4. Безопасность:

- ➔ Проверяйте индексы при доступе к элементам
- ➔ Обработывайте граничные случаи (пустые массивы, один элемент)
- ➔ Используйте значения по умолчанию для опциональных параметров

Современные возможности ES6+:

javascript

// Деструктуризация массива

```
const [first, second, ...rest] = [1, 2, 3, 4, 5];  
console.log(first, second, rest); // 1 2 [3, 4, 5]
```

// Spread оператор для копирования и объединения

```
const copy = [...original];  
const merged = [...arr1, ...arr2];
```

// Array.from() для создания массивов из итерируемых объектов

```
const fromString = Array.from("Hello");  
const fromSet = Array.from(new Set([1, 2, 2, 3]));
```

// Методы ES2019+

```
const flattened = nested.flat(2);  
const flatMapped = array.flatMap(x => [x, x * 2]);
```

// Методы ES2023+

```
const foundFromEnd = array.findLast(x => x > 5);  
const foundIndexFromEnd = array.findLastIndex(x => x > 5);
```

Практические советы:

1. **Для создания массива** используйте литеральную ноцию `[]` или `Array.from()`
2. **Для добавления элементов** в конец используйте `push()`, в начало - `unshift()`
3. **Для удаления элементов** с конца используйте `pop()`, с начала - `shift()`
4. **Для поиска элементов** используйте `find()` для объектов, `includes()` для примитивов
5. **Для преобразования** используйте `map()` для трансформации, `filter()` для фильтрации
6. **Для агрегации** используйте `reduce()` для вычисления одного значения
7. **Для сортировки** всегда передавайте функцию сравнения для чисел
8. **Для работы с подмассивами** используйте `slice()` для извлечения, `splice()` для изменения

Массивы в JavaScript — это мощный и гибкий инструмент, который при правильном использовании позволяет писать чистый, эффективный и выразительный код. Понимание всех нюансов работы с массивами является фундаментальным навыком для любого JavaScript-разработчика.

9.2. Методы массивов: итерационные, мутирующие, доступные

Введение: классификация методов массивов

Методы массивов в JavaScript можно разделить на три основные категории:

1. **Итерационные методы** (не мутирующие) - проходят по элементам массива и возвращают новый результат
2. **Мутирующие методы** - изменяют исходный массив
3. **Доступные методы** - возвращают информацию о массиве без его изменения

Часть 1: Итерационные методы (Iteration Methods)

Итерационные методы принимают функцию-колбэк и применяют её к каждому элементу массива. Они **НЕ изменяют исходный массив**.

1.1. `map()` - трансформация элементов

```
javascript
```

```
// Синтаксис: array.map(callback(currentValue, index, array), thisArg)
```

```
const numbers = [1, 2, 3, 4, 5];
```

```
// 1. Базовое использование
```

```
const doubled = numbers.map(num => num * 2);
```

```
console.log(doubled); // [2, 4, 6, 8, 10]
```

```
console.log(numbers); // [1, 2, 3, 4, 5] - исходный массив не изменился
```

```
// 2. Использование индекса
```

```
const indexed = numbers.map((num, index) => `${index}: ${num}`);
```

```
console.log(indexed); // ["0: 1", "1: 2", "2: 3", "3: 4", "4: 5"]
```

```
// 3. Преобразование объектов
```

```
const users = [  
  { id: 1, name: "Анна", age: 25 },  
  { id: 2, name: "Борис", age: 30 },
```

```

    { id: 3, name: "Виктор", age: 20 }
  ];

const userNames = users.map(user => user.name);
console.log(userNames); // ["Анна", "Борис", "Виктор"]

const userCards = users.map(user => ({
  title: user.name,
  description: `Возраст: ${user.age} лет`,
  id: user.id
}));
console.log(userCards);

// 4. Использование thisArg (редко используется)
const multiplier = {
  factor: 10,
  multiply: function(num) {
    return num * this.factor;
  }
};

const multiplied = numbers.map(
  function(num) {
    return this.multiply(num);
  },
  multiplier
);
console.log(multiplied); // [10, 20, 30, 40, 50]

// 5. map() для многомерных массивов
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

const transposed = matrix[0].map((_, colIndex) =>
  matrix.map(row => row[colIndex])
);
console.log(transposed);
// [[1, 4, 7], [2, 5, 8], [3, 6, 9]]

// 6. Цепочка map()
const complexResult = numbers

```

```

    .map(n => n * 2)           // [2, 4, 6, 8, 10]
    .map(n => n + 1)           // [3, 5, 7, 9, 11]
    .map(n => n.toString());   // ["3", "5", "7", "9", "11"]
console.log(complexResult);

```

// 7. Потенциальные ошибки

```

const invalidMap = numbers.map((num, index, arr) => {
    // ✗ Не изменяйте исходный массив в map()
    // arr[index] = num * 2; // Антипаттерн!
    return num * 2;
});

```

// 8. Производительная альтернатива для больших массивов

```

function customMap(array, mapper) {
    const result = new Array(array.length);
    for (let i = 0; i < array.length; i++) {
        result[i] = mapper(array[i], i, array);
    }
    return result;
}

```

```

const largeArray = Array.from({ length: 1000000 }, (_, i) => i);
console.time('native map');
const nativeResult = largeArray.map(x => x * 2);
console.timeEnd('native map'); // ~100ms

```

```

console.time('custom map');
const customResult = customMap(largeArray, x => x * 2);
console.timeEnd('custom map'); // ~80ms

```

1.2. filter() - фильтрация элементов

javascript

// Синтаксис: array.filter(callback(element, index, array), thisArg)

```

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

```

// 1. Фильтрация по условию

```

const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // [2, 4, 6, 8, 10]

```

```

const greaterThanFive = numbers.filter(num => num > 5);
console.log(greaterThanFive); // [6, 7, 8, 9, 10]

```

// 2. Фильтрация объектов

```
const users = [
  { id: 1, name: "Анна", age: 25, active: true },
  { id: 2, name: "Борис", age: 30, active: false },
  { id: 3, name: "Виктор", age: 20, active: true },
  { id: 4, name: "Галина", age: 35, active: true }
];

const activeUsers = users.filter(user => user.active);
console.log(activeUsers); // Анна, Виктор, Галина

const adultUsers = users.filter(user => user.age >= 25);
console.log(adultUsers); // Анна, Борис, Галина
```

// 3. Комплексные условия

```
const activeAdults = users.filter(user =>
  user.active && user.age >= 25 && user.name.startsWith("A")
);
console.log(activeAdults); // Только Анна
```

// 4. Удаление "ложных" (falsy) значений

```
const mixed = [0, 1, false, 2, '', 3, null, undefined, 4, NaN];
const truthy = mixed.filter(Boolean);
console.log(truthy); // [1, 2, 3, 4]
```

// 5. Фильтрация уникальных значений

```
const duplicates = [1, 2, 2, 3, 4, 4, 4, 5];
const unique = duplicates.filter((num, index, arr) =>
  arr.indexOf(num) === index
);
console.log(unique); // [1, 2, 3, 4, 5]
```

// Более эффективно с Set:

```
const uniqueWithSet = [...new Set(duplicates)];
console.log(uniqueWithSet); // [1, 2, 3, 4, 5]
```

// 6. Фильтрация по типу данных

```
const mixedTypes = [1, "строка", true, null, { x: 1 }, [1, 2], undefined];
const numbersOnly = mixedTypes.filter(item => typeof item === 'number');
console.log(numbersOnly); // [1]
```

```
const objectsOnly = mixedTypes.filter(item =>
  typeof item === 'object' && item !== null
);
```

```

);
console.log(objectsOnly); // [{x: 1}, [1, 2]]

// 7. Использование индекса
const everySecond = numbers.filter((_, index) => index % 2 === 0);
console.log(everySecond); // [1, 3, 5, 7, 9]

// 8. Фильтрация с побочными эффектами (осторожно!)
const logFilter = numbers.filter(num => {
  console.log(`Проверка числа: ${num}`);
  return num > 5;
});
// Выведет: Проверка числа: 1, Проверка числа: 2, ...

// 9. Производительная реализация
function customFilter(array, predicate) {
  const result = [];
  for (let i = 0; i < array.length; i++) {
    if (predicate(array[i], i, array)) {
      result.push(array[i]);
    }
  }
  return result;
}

const largeArray = Array.from({ length: 1000000 }, (_, i) => i);
console.time('native filter');
const filteredNative = largeArray.filter(x => x % 2 === 0);
console.timeEnd('native filter'); // ~60ms

console.time('custom filter');
const filteredCustom = customFilter(largeArray, x => x % 2 === 0);
console.timeEnd('custom filter'); // ~50ms

```

1.3. `reduce()` и `reduceRight()` - агрегация значений

```

javascript

// Синтаксис: array.reduce(callback(accumulator, currentValue, index, array),
// initialValue)

const numbers = [1, 2, 3, 4, 5];

// 1. Сумма элементов

```



```

const sum = numbers.reduce((acc, num) => acc + num, 0);
console.log(sum); // 15

// Без initialValue (берет первый элемент как аккумулятор)
const sumWithoutInit = numbers.reduce((acc, num) => acc + num);
console.log(sumWithoutInit); // 15

// 2. Произведение элементов
const product = numbers.reduce((acc, num) => acc * num, 1);
console.log(product); // 120

// 3. Поиск максимального значения
const max = numbers.reduce((acc, num) => Math.max(acc, num), -Infinity);
console.log(max); // 5

// Или без initialValue:
const max2 = numbers.reduce((acc, num) => acc > num ? acc : num);
console.log(max2); // 5

// 4. Группировка объектов
const people = [
  { name: "Анна", age: 25, city: "Москва" },
  { name: "Борис", age: 30, city: "Санкт-Петербург" },
  { name: "Виктор", age: 20, city: "Москва" },
  { name: "Галина", age: 35, city: "Казань" }
];

const groupedByCity = people.reduce((groups, person) => {
  const city = person.city;
  if (!groups[city]) {
    groups[city] = [];
  }
  groups[city].push(person);
  return groups;
}, {});
console.log(groupedByCity);
// {
//   "Москва": [{...}, {...}],
//   "Санкт-Петербург": [{...}],
//   "Казань": [{...}]
// }

// 5. Преобразование массива в объект
const users = [

```

```

    { id: 1, name: "Анна" },
    { id: 2, name: "Борис" },
    { id: 3, name: "Виктор" }
  ];

const usersById = users.reduce((obj, user) => {
  obj[user.id] = user;
  return obj;
}, {});
console.log(usersById);
// {
//   "1": {id: 1, name: "Анна"},
//   "2": {id: 2, name: "Борис"},
//   "3": {id: 3, name: "Виктор"}
// }

// 6. Сглаживание массива (flatten)
const nested = [1, [2, [3, [4, 5]]]];

const flattenDeep = arr => arr.reduce((acc, val) =>
  Array.isArray(val) ? acc.concat(flattenDeep(val)) : acc.concat(val),
  []);
console.log(flattenDeep(nested)); // [1, 2, 3, 4, 5]

// 7. Композиция функций
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

const add5 = x => x + 5;
const multiply3 = x => x * 3;
const subtract2 = x => x - 2;

const composed = compose(subtract2, multiply3, add5);
console.log(composed(10)); // ((10 + 5) * 3) - 2 = 43

const piped = pipe(add5, multiply3, subtract2);
console.log(piped(10)); // ((10 + 5) * 3) - 2 = 43

// 8. Статистика массива
function arrayStats(arr) {
  return arr.reduce((stats, num, index) => {
    stats.sum += num;
    stats.min = Math.min(stats.min, num);
    stats.max = Math.max(stats.max, num);
  }, {
    sum: 0,
    min: Infinity,
    max: -Infinity
  });
}

```

```

        stats.count = index + 1;
        stats.average = stats.sum / stats.count;
        return stats;
    }, { sum: 0, min: Infinity, max: -Infinity, count: 0, average: 0 });
}

const stats = arrayStats([10, 20, 30, 40, 50]);
console.log(stats);
// {sum: 150, min: 10, max: 50, count: 5, average: 30}

// 9. reduceRight() - справа налево
const reversedConcat = numbers.reduceRight((acc, num) => acc + num.toString(), '');
console.log(reversedConcat); // "54321"

const data = [1, 2, 3, 4];
const rightToLeft = data.reduceRight((acc, val) => {
    console.log(`acc: ${acc}, val: ${val}`);
    return acc + val;
}, 0);
// Выведет: acc: 0, val: 4 → acc: 4, val: 3 → acc: 7, val: 2 → acc: 9, val: 1
console.log(rightToLeft); // 10

// 10. Сложные агрегации
const transactions = [
    { type: 'deposit', amount: 100 },
    { type: 'withdrawal', amount: 50 },
    { type: 'deposit', amount: 200 },
    { type: 'withdrawal', amount: 30 },
    { type: 'withdrawal', amount: 80 }
];

const balance = transactions.reduce((acc, transaction) => {
    if (transaction.type === 'deposit') {
        acc.totalDeposits += transaction.amount;
        acc.balance += transaction.amount;
    } else {
        acc.totalWithdrawals += transaction.amount;
        acc.balance -= transaction.amount;
    }
    acc.transactionCount++;
    return acc;
}, { balance: 0, totalDeposits: 0, totalWithdrawals: 0, transactionCount: 0 });

console.log(balance);

```

```
// {balance: 140, totalDeposits: 300, totalWithdrawals: 160, transactionCount: 5}
```

1.4. `forEach()` - итерация с побочными эффектами

javascript

```
// Синтаксис: array.forEach(callback(currentValue, index, array), thisArg)
```

```
const numbers = [1, 2, 3, 4, 5];
```

```
// 1. Базовое использование
```

```
numbers.forEach(num => {  
    console.log(`Число: ${num}, Квадрат: ${num * num}`);  
});
```

```
// 2. Использование индекса
```

```
numbers.forEach((num, index) => {  
    console.log(`Элемент ${index} = ${num}`);  
});
```

```
// 3. Модификация внешних переменных
```

```
let sum = 0;  
numbers.forEach(num => {  
    sum += num;  
});  
console.log(`Сумма: ${sum}`); // 15
```

```
// 4. Работа с DOM элементами
```

```
const items = ['Яблоко', 'Банан', 'Апельсин'];
```

```
// В реальном приложении:
```

```
// items.forEach(item => {  
//     const li = document.createElement('li');  
//     li.textContent = item;  
//     document.getElementById('list').appendChild(li);  
// });
```

```
// 5. Работа с объектами
```

```
const users = [  
    { id: 1, name: "Анна" },  
    { id: 2, name: "Борис" },  
    { id: 3, name: "Виктор" }  
];
```

```
users.forEach((user, index) => {  
    console.log(`Пользователь ${index + 1}: ${user.name} (ID: ${user.id})`);  
});
```

// 6. Сравнение с for...of

```
console.time('forEach');  
numbers.forEach(num => {  
    // Какие-то операции  
    const _ = num * num;  
});  
console.timeEnd('forEach'); // ~0.1ms
```

```
console.time('for...of');  
for (const num of numbers) {  
    const _ = num * num;  
}  
console.timeEnd('for...of'); // ~0.05ms
```

// 7. Особенности forEach()

*// - Нельзя прервать выполнение (break не работает)
// - return не прерывает выполнение, а просто завершает текущую итерацию
// - Всегда возвращает undefined
// - Не ждет асинхронных операций*

// 8. Асинхронные операции (осторожно!)

```
async function processItems(items) {  
    console.log('Начало обработки');  
  
    // ✗ Проблема: forEach не ждет async функций  
    items.forEach(async (item) => {  
        await new Promise(resolve => setTimeout(resolve, 100));  
        console.log(`Обработано: ${item}`);  
    });  
  
    console.log('Конец обработки (но обработка еще идет!)');  
}
```

// 📌 Правильно: использовать for...of с await

```
async function processItemsCorrectly(items) {  
    console.log('Начало обработки');  
  
    for (const item of items) {  
        await new Promise(resolve => setTimeout(resolve, 100));  
        console.log(`Обработано: ${item}`);  
    }  
}
```

```

    }

    console.log('Конец обработки');
}

// 9. Имитация break в forEach
numbers.forEach((num, index, arr) => {
    if (num > 3) {
        arr.length = index + 1; // "Обрезаем" массив
        return;
    }
    console.log(num); // Выведет: 1, 2, 3
});

// 10. Использование thisArg
const processor = {
    prefix: "Значение: ",
    process(value) {
        console.log(this.prefix + value);
    }
};

numbers.forEach(function(num) {
    this.process(num);
}, processor);

```

1.5. every() и some() - проверка условий

```

javascript

// Синтаксис:
// array.every(callback(element, index, array), thisArg) - все элементы удовлетворяют
// условию
// array.some(callback(element, index, array), thisArg) - хотя бы один элемент
// удовлетворяет условию

const numbers = [1, 2, 3, 4, 5];
const emptyArray = [];

// 1. every() - проверка, что ВСЕ элементы удовлетворяют условию
const allPositive = numbers.every(num => num > 0);
console.log(allPositive); // true

const allEven = numbers.every(num => num % 2 === 0);

```

```
console.log(allEven); // false

// Особенность: для пустого массива every() всегда возвращает true
console.log(emptyArray.every(() => false)); // true

// 2. some() - проверка, что ХОТЯ БЫ ОДИН элемент удовлетворяет условию
const hasEven = numbers.some(num => num % 2 === 0);
console.log(hasEven); // true (2, 4)

const hasNegative = numbers.some(num => num < 0);
console.log(hasNegative); // false

// Особенность: для пустого массива some() всегда возвращает false
console.log(emptyArray.some(() => true)); // false

// 3. Практические примеры с объектами
const products = [
  { name: "Ноутбук", price: 50000, inStock: true },
  { name: "Мышь", price: 1500, inStock: false },
  { name: "Клавиатура", price: 3000, inStock: true }
];

// Проверка наличия всех товаров
const allInStock = products.every(product => product.inStock);
console.log("Все товары в наличии:", allInStock); // false

// Проверка наличия хотя бы одного дорогого товара
const hasExpensive = products.some(product => product.price > 10000);
console.log("Есть дорогие товары:", hasExpensive); // true

// 4. Проверка валидации форм
const formFields = [
  { name: "email", value: "user@example.com", isValid: true },
  { name: "password", value: "12345", isValid: true },
  { name: "confirmPassword", value: "12345", isValid: true }
];

const isFormValid = formFields.every(field => field.isValid);
console.log("Форма валидна:", isFormValid); // true

// 5. Поиск соответствия паттерну
const strings = ["JavaScript", "TypeScript", "CoffeeScript"];
const allEndWithScript = strings.every(str => str.endsWith("Script"));
console.log(allEndWithScript); // true
```

```

const anyHasJava = strings.some(str => str.includes("Java"));
console.log(anyHasJava); // true (JavaScript)

// 6. Проверка типов данных в массиве
const mixedArray = [1, "строка", true, null];
const allStrings = mixedArray.every(item => typeof item === 'string');
console.log("Все строки:", allStrings); // false

const hasString = mixedArray.some(item => typeof item === 'string');
console.log("Есть строка:", hasString); // true

// 7. Комбинирование every() и some()
const data = [10, 20, 30, 40, 50];

// Проверка, что все числа в диапазоне
const allInRange = data.every(num => num >= 0 && num <= 100);
console.log("Все в диапазоне 0-100:", allInRange); // true

// Проверка, что есть числа больше 25
const anyAbove25 = data.some(num => num > 25);
console.log("Есть числа больше 25:", anyAbove25); // true

// 8. Оптимизация: early exit
// every() прекращает выполнение при первой false
const largeArray = Array.from({ length: 1000000 }, (_, i) => i);

console.time('every с early exit');
const allBelow500000 = largeArray.every(num => {
  if (num > 500000) return false; // Прервется здесь
  return true;
});
console.timeEnd('every с early exit'); // Быстро

console.time('every без early exit');
const allBelowMillion = largeArray.every(num => num < 1000000);
console.timeEnd('every без early exit'); // Медленнее

// 9. Использование с thisArg
const validator = {
  min: 0,
  max: 100,
  validate(value) {
    return value >= this.min && value <= this.max;
  }
};

```



```

    }
};

const testValues = [10, 50, 90, 110];
const allValid = testValues.every(function(value) {
    return this.validate(value);
}, validator);
console.log("Все значения валидны:", allValid); // false (110 не валидно)

// 10. Кастомные реализации
function customEvery(array, predicate) {
    for (let i = 0; i < array.length; i++) {
        if (!predicate(array[i], i, array)) {
            return false;
        }
    }
    return true;
}

function customSome(array, predicate) {
    for (let i = 0; i < array.length; i++) {
        if (predicate(array[i], i, array)) {
            return true;
        }
    }
    return false;
}

console.log(customEvery(numbers, n => n > 0)); // true
console.log(customSome(numbers, n => n > 10)); // false

```

Часть 2: Мутлирующие методы (Mutating Methods)

Эти методы изменяют исходный массив.

2.1. `push()` и `pop()` - работа с концом массива

```

javascript

// push() - добавляет элементы в конец массива, возвращает новую длину
// pop() - удаляет последний элемент, возвращает удаленный элемент

```

```
let fruits = ["яблоко", "банан"];

// 1. push() - добавление элементов
let newLength = fruits.push("апельсин");
console.log(fruits); // ["яблоко", "банан", "апельсин"]
console.log(newLength); // 3

// Добавление нескольких элементов
newLength = fruits.push("киви", "манго", "груша");
console.log(fruits); // ["яблоко", "банан", "апельсин", "киви", "манго", "груша"]
console.log(newLength); // 6

// 2. pop() - удаление элементов
let lastFruit = fruits.pop();
console.log(lastFruit); // "груша"
console.log(fruits); // ["яблоко", "банан", "апельсин", "киви", "манго"]

// Удаление всех элементов
while (fruits.length > 0) {
    console.log(`Удаляем: ${fruits.pop()}`);
}
console.log(fruits); // []

// 3. Реализация стека (LIFO - Last In, First Out)
class Stack {
    constructor() {
        this.items = [];
    }

    push(element) {
        return this.items.push(element);
    }

    pop() {
        if (this.isEmpty()) {
            throw new Error("Стек пуст");
        }
        return this.items.pop();
    }

    peek() {
        return this.items[this.items.length - 1];
    }
}
```

```

isEmpty() {
    return this.items.length === 0;
}

size() {
    return this.items.length;
}

clear() {
    this.items = [];
}
}

```

```

const stack = new Stack();
stack.push(10);
stack.push(20);
stack.push(30);
console.log(stack.peek()); // 30
console.log(stack.pop()); // 30
console.log(stack.size()); // 2

```

// 4. Производительность push() vs unshift()

```

const largeArray = [];
const size = 100000;

console.time('push');
for (let i = 0; i < size; i++) {
    largeArray.push(i);
}
console.timeEnd('push'); // Быстро

console.time('unshift');
for (let i = 0; i < size; i++) {
    largeArray.unshift(i); // Медленнее, т.к. сдвигает все элементы
}
console.timeEnd('unshift'); // Медленнее

```

// 5. push() с spread оператором

```

const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

// Вместо arr1.push(...arr2)
Array.prototype.push.apply(arr1, arr2);
console.log(arr1); // [1, 2, 3, 4, 5, 6]

```

// Современный вариант:

```
arr1.push(...arr2);
```

// 6. Безопасный pop() с проверкой

```
function safePop(array) {  
  if (array.length === 0) {  
    return {  
      success: false,  
      message: "Массив пуст",  
      value: undefined  
    };  
  }  
  return {  
    success: true,  
    value: array.pop()  
  };  
}
```

```
const empty = [];
```

```
const result = safePop(empty);
```

```
console.log(result); // {success: false, message: "Массив пуст", value: undefined}
```

// 7. push() для построения массивов

```
function generateSequence(start, end) {  
  const sequence = [];  
  for (let i = start; i <= end; i++) {  
    sequence.push(i);  
  }  
  return sequence;  
}
```

```
console.log(generateSequence(5, 10)); // [5, 6, 7, 8, 9, 10]
```

// 8. push() с объектами

```
const users = [];
```

```
users.push({ id: 1, name: "Анна" });
```

```
users.push({ id: 2, name: "Борис" });
```

```
users.push({ id: 3, name: "Виктор" });
```

```
console.log(users.length); // 3
```

// 9. Множественное добавление

```
function addMultiple(array, ...elements) {
```

```

    const initialLength = array.length;
    for (const element of elements) {
        array.push(element);
    }
    return array.length - initialLength;
}

const testArray = [1, 2, 3];
const addedCount = addMultiple(testArray, 4, 5, 6, 7);
console.log(testArray); // [1, 2, 3, 4, 5, 6, 7]
console.log(`Добавлено элементов: ${addedCount}`); // 4

```

2.2. `shift()` и `unshift()` - работа с началом массива

javascript

// shift() - удаляет первый элемент, возвращает удаленный элемент
// unshift() - добавляет элементы в начало, возвращает новую длину

```
let queue = ["первый", "второй", "третий"];
```

// 1. shift() - удаление из начала

```
let firstElement = queue.shift();
console.log(firstElement); // "первый"
console.log(queue); // ["второй", "третий"]

```

// 2. unshift() - добавление в начало

```
let newLength = queue.unshift("новый первый");
console.log(queue); // ["новый первый", "второй", "третий"]
console.log(newLength); // 3

```

// Добавление нескольких элементов в начало

```
newLength = queue.unshift("еще один", "самый первый");
console.log(queue); // ["самый первый", "еще один", "новый первый", "второй", "третий"]

```

// 3. Реализация очереди (FIFO - First In, First Out)

```
class Queue {
    constructor() {
        this.items = [];
    }

    enqueue(element) {
        return this.items.push(element);
    }
}

```

```

    dequeue() {
        if (this.isEmpty()) {
            throw new Error("Очередь пуста");
        }
        return this.items.shift();
    }

    front() {
        if (this.isEmpty()) {
            throw new Error("Очередь пуста");
        }
        return this.items[0];
    }

    isEmpty() {
        return this.items.length === 0;
    }

    size() {
        return this.items.length;
    }

    clear() {
        this.items = [];
    }

    print() {
        console.log(this.items.toString());
    }
}

```

```

const queueExample = new Queue();
queueExample.enqueue("Анна");
queueExample.enqueue("Борис");
queueExample.enqueue("Виктор");

```

```

console.log(queueExample.front()); // "Анна"
console.log(queueExample.dequeue()); // "Анна"
console.log(queueExample.size()); // 2

```

// 4. Производительность shift() vs pop()

```

const largeArray = Array.from({ length: 100000 }, (_, i) => i);

```

```
console.time('shift 1000 times');
for (let i = 0; i < 1000; i++) {
    largeArray.shift(); // Медленно, т.к. переиндексация
}
console.timeEnd('shift 1000 times'); // Медленно
```

```
console.time('pop 1000 times');
for (let i = 0; i < 1000; i++) {
    largeArray.pop(); // Быстро
}
console.timeEnd('pop 1000 times'); // Быстро
```

// 5. Оптимизация: использование с конца

```
class EfficientQueue {
    constructor() {
        this.items = [];
        this.head = 0; // Индекс первого элемента
    }

    enqueue(element) {
        this.items.push(element);
        return this.items.length - this.head;
    }

    dequeue() {
        if (this.isEmpty()) {
            throw new Error("Очередь пуста");
        }
        const element = this.items[this.head];
        this.head++;

        // Периодическая очистка
        if (this.head > 100 && this.head > this.items.length / 2) {
            this.items = this.items.slice(this.head);
            this.head = 0;
        }

        return element;
    }

    front() {
        if (this.isEmpty()) {
            throw new Error("Очередь пуста");
        }
    }
}
```

```

        return this.items[this.head];
    }

    isEmpty() {
        return this.head === this.items.length;
    }

    size() {
        return this.items.length - this.head;
    }
}

// 6. shift() для обработки задач
const taskQueue = ["задача1", "задача2", "задача3"];

function processTasks(queue) {
    while (queue.length > 0) {
        const task = queue.shift();
        console.log(`Обрабатываю: ${task}`);
        // Имитация обработки
    }
    console.log("Все задачи обработаны");
}

processTasks([...taskQueue]); // Используем копию

// 7. unshift() для добавления приоритетных элементов
const messages = ["сообщение1", "сообщение2"];

// Добавление срочного сообщения
messages.unshift("СРОЧНО: важное сообщение");
console.log(messages); // ["СРОЧНО: важное сообщение", "сообщение1", "сообщение2"]

// 8. Безопасный shift()
function safeShift(array) {
    if (array.length === 0) {
        return null;
    }
    return array.shift();
}

// 9. Комбинирование shift() и push() для ротации
function rotateArray(array, positions) {
    if (positions <= 0 || array.length === 0) return array;

```



```

    for (let i = 0; i < positions; i++) {
        const element = array.shift();
        array.push(element);
    }
    return array;
}

const arr = [1, 2, 3, 4, 5];
console.log(rotateArray([...arr], 2)); // [3, 4, 5, 1, 2]

// 10. unshift() для инициализации массивов
function createArrayWithDefaults(length, defaultValue) {
    const array = [];
    for (let i = 0; i < length; i++) {
        array.unshift(defaultValue); // Добавляем в начало
    }
    return array;
}

console.log(createArrayWithDefaults(5, 0)); // [0, 0, 0, 0, 0]

```

2.3. splice() - универсальный метод модификации

javascript

```

// Синтаксис: array.splice(start, deleteCount, item1, item2, ...)
// Возвращает массив удаленных элементов

let fruits = ["яблоко", "банан", "апельсин", "киви", "манго"];

// 1. Удаление элементов
let removed = fruits.splice(2, 2); // Удалить 2 элемента начиная с индекса 2
console.log(removed); // ["апельсин", "киви"]
console.log(fruits); // ["яблоко", "банан", "манго"]

// 2. Добавление элементов
fruits.splice(1, 0, "лимон", "лайм"); // Добавить без удаления
console.log(fruits); // ["яблоко", "лимон", "лайм", "банан", "манго"]

// 3. Замена элементов
removed = fruits.splice(2, 2, "персик", "груша"); // Удалить 2 и добавить 2
console.log(removed); // ["лайм", "банан"]
console.log(fruits); // ["яблоко", "лимон", "персик", "груша", "манго"]

```

```
// 4. Удаление с конца
removed = fruits.splice(-2, 1); // Удалить 1 элемент с конца-2
console.log(removed); // ["зруша"]
console.log(fruits); // ["яблоко", "лимон", "персик", "манго"]
```

```
// 5. Удаление всех элементов с позиции
removed = fruits.splice(1); // Удалить все начиная с индекса 1
console.log(removed); // ["лимон", "персик", "манго"]
console.log(fruits); // ["яблоко"]
```

```
// 6. Практические примеры
```

```
// Удаление дубликатов на месте
```

```
function removeDuplicatesInPlace(array) {
  for (let i = 0; i < array.length; i++) {
    for (let j = i + 1; j < array.length; j++) {
      if (array[i] === array[j]) {
        array.splice(j, 1);
        j--; // Корректировка индекса после удаления
      }
    }
  }
  return array;
}
```

```
const withDuplicates = [1, 2, 2, 3, 4, 4, 5];
removeDuplicatesInPlace(withDuplicates);
console.log(withDuplicates); // [1, 2, 3, 4, 5]
```

```
// Вставка элемента в отсортированный массив
```

```
function insertSorted(array, element) {
  let index = 0;
  while (index < array.length && array[index] < element) {
    index++;
  }
  array.splice(index, 0, element);
  return array;
}
```

```
const sorted = [1, 3, 5, 7, 9];
insertSorted(sorted, 6);
console.log(sorted); // [1, 3, 5, 6, 7, 9]
```

// 7. Удаление по условию

```
function removeByCondition(array, condition) {  
    for (let i = array.length - 1; i >= 0; i--) {  
        if (condition(array[i], i, array)) {  
            array.splice(i, 1);  
        }  
    }  
    return array;  
}
```

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9];  
removeByCondition(numbers, n => n % 2 === 0);  
console.log(numbers); // [1, 3, 5, 7, 9]
```

// 8. Перемещение элемента

```
function moveElement(array, fromIndex, toIndex) {  
    if (fromIndex < 0 || fromIndex >= array.length ||  
        toIndex < 0 || toIndex >= array.length) {  
        throw new Error("Индекс вне диапазона");  
    }  
  
    const [element] = array.splice(fromIndex, 1);  
    array.splice(toIndex, 0, element);  
    return array;  
}
```

```
const arr = ['a', 'b', 'c', 'd', 'e'];  
moveElement(arr, 1, 3);  
console.log(arr); // ['a', 'c', 'd', 'b', 'e']
```

// 9. Разделение массива по индексу

```
function splitAtIndex(array, index) {  
    const removed = array.splice(index);  
    return [array, removed];  
}
```

```
const fullArray = [1, 2, 3, 4, 5, 6];  
const [firstPart, secondPart] = splitAtIndex([...fullArray], 3);  
console.log(firstPart); // [1, 2, 3]  
console.log(secondPart); // [4, 5, 6]
```

// 10. Безопасный splice

```
function safeSplice(array, start, deleteCount, ...items) {  
    // Корректировка start
```

```

    if (start < 0) {
        start = Math.max(array.length + start, 0);
    } else {
        start = Math.min(start, array.length);
    }

    // Корректировка deleteCount
    deleteCount = Math.max(0, Math.min(deleteCount, array.length - start));

    // Выполнение операции
    return array.splice(start, deleteCount, ...items);
}

const testArray = [1, 2, 3, 4, 5];
safeSplice(testArray, 10, 2, 6, 7); // start корректируется до 5
console.log(testArray); // [1, 2, 3, 4, 5, 6, 7]

```

2.4. `sort()` и `reverse()` - изменение порядка

```

javascript

// sort() - сортирует массив на месте
// reverse() - обращает порядок элементов на месте

// 1. Базовое использование sort()
let fruits = ["банан", "яблоко", "апельсин", "киви"];
fruits.sort();
console.log(fruits); // ["апельсин", "банан", "киви", "яблоко"] (алфавитно)

// 2. Сортировка чисел (по умолчанию лексикографическая)
let numbers = [10, 2, 100, 5, 1];
numbers.sort();
console.log(numbers); // [1, 10, 100, 2, 5] - неправильно!

// 3. Правильная сортировка чисел
numbers.sort((a, b) => a - b); // По возрастанию
console.log(numbers); // [1, 2, 5, 10, 100]

numbers.sort((a, b) => b - a); // По убыванию
console.log(numbers); // [100, 10, 5, 2, 1]

// 4. Сортировка объектов
const users = [
    { name: "Анна", age: 25 },

```

```

    { name: "Борис", age: 30 },
    { name: "Виктор", age: 20 },
    { name: "Галина", age: 25 }
  ];

  // Сортировка по возрасту
  users.sort((a, b) => a.age - b.age);
  console.log(users);
  // [{name: "Виктор", age: 20}, {name: "Анна", age: 25}, ...]

  // Сортировка по имени
  users.sort((a, b) => a.name.localeCompare(b.name));
  console.log(users);
  // [{name: "Анна", age: 25}, {name: "Борис", age: 30}, ...]

  // 5. Многоуровневая сортировка
  users.sort((a, b) => {
    // Сначала по возрасту
    if (a.age !== b.age) {
      return a.age - b.age;
    }
    // Затем по имени
    return a.name.localeCompare(b.name);
  });

  // 6. Сортировка с учетом локали
  const russianWords = ["яблоко", "ёжик", "еда", "юг"];
  russianWords.sort((a, b) => a.localeCompare(b, 'ru'));
  console.log(russianWords); // Правильный порядок для русского языка

  // 7. Сортировка дат
  const dates = [
    new Date(2023, 11, 25), // 25 декабря 2023
    new Date(2023, 0, 1),   // 1 января 2023
    new Date(2022, 5, 15)   // 15 июня 2022
  ];

  dates.sort((a, b) => a - b); // По возрастанию даты
  console.log(dates.map(d => d.toLocaleDateString()));

  // 8. Стабильная сортировка (ES2019+)
  // Современные движки обеспечивают стабильность
  const data = [
    { value: 1, order: 1 },

```

```

    { value: 2, order: 2 },
    { value: 1, order: 3 },
    { value: 3, order: 4 }
  ];

data.sort((a, b) => a.value - b.value);
console.log(data.map(d => d.order)); // [1, 3, 2, 4] - порядок 1 и 3 сохранен

// 9. reverse() - обратный порядок
const arr = [1, 2, 3, 4, 5];
arr.reverse();
console.log(arr); // [5, 4, 3, 2, 1]

// 10. Комбинирование sort() и reverse()
const sortedAsc = [3, 1, 4, 1, 5, 9].sort((a, b) => a - b);
console.log(sortedAsc); // [1, 1, 3, 4, 5, 9]

const sortedDesc = [3, 1, 4, 1, 5, 9].sort((a, b) => b - a);
console.log(sortedDesc); // [9, 5, 4, 3, 1, 1]

// Или через reverse():
const asc = [3, 1, 4, 1, 5, 9];
asc.sort((a, b) => a - b);
const desc = [...asc].reverse();
console.log(desc); // [9, 5, 4, 3, 1, 1]

// 11. Кастомные алгоритмы сортировки
// Пузырьковая сортировка
function bubbleSort(array, compare = (a, b) => a - b) {
  const arr = [...array];
  const n = arr.length;

  for (let i = 0; i < n - 1; i++) {
    for (let j = 0; j < n - 1 - i; j++) {
      if (compare(arr[j], arr[j + 1]) > 0) {
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
      }
    }
  }

  return arr;
}

console.log(bubbleSort([5, 3, 8, 1, 2])); // [1, 2, 3, 5, 8]

```

```
// 12. Создание отсортированной копии
function sortedCopy(array, compareFunction) {
    return [...array].sort(compareFunction);
}

const original = [3, 1, 4, 1, 5];
const sorted = sortedCopy(original, (a, b) => a - b);
console.log(original); // [3, 1, 4, 1, 5] - не изменился
console.log(sorted); // [1, 1, 3, 4, 5]
```

Часть 3: Доступные методы (Accessor Methods)

Эти методы возвращают информацию о массиве, не изменяя его.

3.1. `concat()` - объединение массивов

```
javascript

// Синтаксис: array.concat(value1, value2, ..., valueN)
// Возвращает новый массив

const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

// 1. Базовое объединение
const combined1 = arr1.concat(arr2);
console.log(combined1); // [1, 2, 3, 4, 5, 6]
console.log(arr1); // [1, 2, 3] - не изменился
console.log(arr2); // [4, 5, 6] - не изменился

// 2. Объединение нескольких массивов
const arr3 = [7, 8, 9];
const combined2 = arr1.concat(arr2, arr3);
console.log(combined2); // [1, 2, 3, 4, 5, 6, 7, 8, 9]

// 3. Объединение с примитивами
const combined3 = arr1.concat(10, 11, [12, 13]);
console.log(combined3); // [1, 2, 3, 10, 11, 12, 13]

// 4. Объединение с вложенными массивами
const nested = arr1.concat([arr2]);
```

```
console.log(nested); // [1, 2, 3, [4, 5, 6]]

// Для плоского объединения:
const flatCombined = arr1.concat(...arr2);
console.log(flatCombined); // [1, 2, 3, 4, 5, 6]

// 5. concat() vs spread оператор
const spreadCombined = [...arr1, ...arr2];
console.log(spreadCombined); // [1, 2, 3, 4, 5, 6]

// concat() может принимать не массивы:
const withConcat = arr1.concat(4, 5);
console.log(withConcat); // [1, 2, 3, 4, 5]

// Со spread оператором нужно явно оборачивать:
const withSpread = [...arr1, 4, 5];
console.log(withSpread); // [1, 2, 3, 4, 5]

// 6. Объединение объектов
const objects1 = [{ id: 1 }, { id: 2 }];
const objects2 = [{ id: 3 }, { id: 4 }];

const allObjects = objects1.concat(objects2);
console.log(allObjects); // [{id: 1}, {id: 2}, {id: 3}, {id: 4}]

// Внимание: объекты не копируются, сохраняются ссылки
allObjects[0].name = "изменено";
console.log(objects1[0]); // {id: 1, name: "изменено"} - тоже изменился!

// 7. Цепочка concat()
const chainResult = [].concat(1, 2).concat(3, 4).concat([5, 6]);
console.log(chainResult); // [1, 2, 3, 4, 5, 6]

// 8. Объединение массивоподобных объектов
const arrayLike1 = { 0: 'a', 1: 'b', length: 2 };
const arrayLike2 = { 0: 'c', 1: 'd', length: 2 };

// concat() не работает с array-like напрямую
// const wrong = [].concat(arrayLike1); // [{0: 'a', 1: 'b', length: 2}]

// Нужно преобразовать:
const correct = [].concat(Array.from(arrayLike1), Array.from(arrayLike2));
console.log(correct); // ['a', 'b', 'c', 'd']
```



```
// 9. Производительность concat() vs push() в цикле
const size = 100000;
const source = Array.from({ length: size }, (_, i) => i);

console.time('concat в цикле');
let result1 = [];
for (let i = 0; i < source.length; i++) {
    result1 = result1.concat(source[i]); // × Медленно, создает новый массив каждый раз
}
console.timeEnd('concat в цикле'); // Очень медленно

console.time('push в цикле');
let result2 = [];
for (let i = 0; i < source.length; i++) {
    result2.push(source[i]); // ✓ Быстро, мутирует существующий массив
}
console.timeEnd('push в цикле'); // Быстро

console.time('concat один раз');
const result3 = [].concat(...source); // ✓ Быстро, одно объединение
console.timeEnd('concat один раз'); // Быстро

// 10. Кастомная реализация concat()
function customConcat(array, ...values) {
    const result = [...array];

    for (const value of values) {
        if (Array.isArray(value)) {
            result.push(...value);
        } else {
            result.push(value);
        }
    }

    return result;
}

console.log(customConcat([1, 2], 3, [4, 5])); // [1, 2, 3, 4, 5]
```

3.2. slice() - извлечение подмассива

javascript

// Синтаксис: array.slice(start, end)

```
// Возвращает новый массив
```

```
const fruits = ["яблоко", "банан", "апельсин", "киви", "манго"];
```

```
// 1. Базовое извлечение
```

```
const slice1 = fruits.slice(1, 3);  
console.log(slice1); // ["банан", "апельсин"]  
console.log(fruits); // Исходный массив не изменился
```

```
// 2. Извлечение с конца
```

```
const slice2 = fruits.slice(-2); // Последние 2 элемента  
console.log(slice2); // ["киви", "манго"]
```

```
const slice3 = fruits.slice(-3, -1); // С конца-3 до конца-1  
console.log(slice3); // ["апельсин", "киви"]
```

```
// 3. Извлечение от позиции до конца
```

```
const slice4 = fruits.slice(2);  
console.log(slice4); // ["апельсин", "киви", "манго"]
```

```
// 4. Создание копии массива
```

```
const copy = fruits.slice();  
console.log(copy); // Полная копия  
console.log(copy === fruits); // false - это другой массив
```

```
// 5. slice() для массивоподобных объектов
```

```
function toArray(arrayLike) {  
    return Array.prototype.slice.call(arrayLike);  
}
```

```
function getArguments() {  
    return toArray(arguments);  
}
```

```
const args = getArguments(1, 2, 3, 4, 5);  
console.log(args); // [1, 2, 3, 4, 5]
```

```
// Современная альтернатива:
```

```
function getArgumentsModern(...args) {  
    return args;  
}
```

```
// 6. Извлечение страниц (пагинация)
```

```
function paginate(array, page, itemsPerPage) {
```

```

    const start = (page - 1) * itemsPerPage;
    const end = start + itemsPerPage;
    return array.slice(start, end);
}

const data = Array.from({ length: 100 }, (_, i) => `Item ${i + 1}`);
const page1 = paginate(data, 1, 10);
const page2 = paginate(data, 2, 10);
console.log(page1[0], page1[9]); // "Item 1", "Item 10"
console.log(page2[0], page2[9]); // "Item 11", "Item 20"

```

// 7. Разделение массива на части

```

function chunk(array, size) {
    const chunks = [];
    for (let i = 0; i < array.length; i += size) {
        chunks.push(array.slice(i, i + size));
    }
    return chunks;
}

```

```

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9];
console.log(chunk(numbers, 3)); // [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

```

// 8. Удаление элемента без мутации

```

function removeElement(array, index) {
    return [...array.slice(0, index), ...array.slice(index + 1)];
}

```

```

const original = [1, 2, 3, 4, 5];
const withoutThird = removeElement(original, 2);
console.log(original); // [1, 2, 3, 4, 5] - не изменился
console.log(withoutThird); // [1, 2, 4, 5]

```

// 9. Вставка элемента без мутации

```

function insertElement(array, index, element) {
    return [...array.slice(0, index), element, ...array.slice(index)];
}

```

```

const withInsert = insertElement(original, 2, 99);
console.log(withInsert); // [1, 2, 99, 3, 4, 5]

```

// 10. slice() для многомерных массивов (поверхностное копирование)

```

const matrix = [
    [1, 2, 3],

```

```

    [4, 5, 6],
    [7, 8, 9]
];

const matrixCopy = matrix.slice();
console.log(matrixCopy[0] === matrix[0]); // true - те же подмассивы

// 11. Производительность slice()
const largeArray = Array.from({ length: 1000000 }, (_, i) => i);

console.time('slice маленькой части');
const smallSlice = largeArray.slice(0, 100);
console.timeEnd('slice маленькой части'); // Быстро

console.time('slice большой части');
const largeSlice = largeArray.slice(0, 500000);
console.timeEnd('slice большой части'); // Медленнее

// 12. Кастомная реализация slice()
function customSlice(array, start = 0, end = array.length) {
    // Нормализация start
    if (start < 0) {
        start = Math.max(array.length + start, 0);
    } else {
        start = Math.min(start, array.length);
    }

    // Нормализация end
    if (end < 0) {
        end = Math.max(array.length + end, 0);
    } else {
        end = Math.min(end, array.length);
    }

    // Извлечение элементов
    const result = [];
    for (let i = start; i < end; i++) {
        result.push(array[i]);
    }

    return result;
}

console.log(customSlice([1, 2, 3, 4, 5], 1, 3)); // [2, 3]

```

```
console.log(customSlice([1, 2, 3, 4, 5], -3)); // [3, 4, 5]
```

3.3. indexOf(), lastIndexOf(), includes() - поиск

javascript

// indexOf() - возвращает первый индекс элемента

// lastIndexOf() - возвращает последний индекс элемента

// includes() - проверяет наличие элемента

```
const numbers = [1, 2, 3, 4, 5, 3, 2, 1];
```

// 1. indexOf() - поиск первого вхождения

```
console.log(numbers.indexOf(3)); // 2
```

```
console.log(numbers.indexOf(3, 3)); // 5 - поиск начиная с индекса 3
```

```
console.log(numbers.indexOf(10)); // -1 - не найден
```

// 2. lastIndexOf() - поиск последнего вхождения

```
console.log(numbers.lastIndexOf(2)); // 6
```

```
console.log(numbers.lastIndexOf(2, 5)); // 1 - поиск до индекса 5
```

```
console.log(numbers.lastIndexOf(10)); // -1
```

// 3. includes() - проверка наличия

```
console.log(numbers.includes(4)); // true
```

```
console.log(numbers.includes(4, 5)); // false - начиная с индекса 5
```

```
console.log(numbers.includes(10)); // false
```

// 4. Поиск объектов (особенность)

```
const users = [  
  { id: 1, name: "Анна" },  
  { id: 2, name: "Борис" },  
  { id: 3, name: "Виктор" }  
];
```

```
console.log(users.indexOf({ id: 1, name: "Анна" })); // -1
```

// Объекты сравниваются по ссылке, а не по содержимому!

```
const anna = users[0];
```

```
console.log(users.indexOf(anna)); // 0 - работает, т.к. та же ссылка
```

// 5. Поиск по свойству объектов

```
function findIndexByProperty(array, property, value) {  
  for (let i = 0; i < array.length; i++) {  
    if (array[i][property] === value) {
```

```

        return i;
    }
}
return -1;
}

```

```

console.log(findIndexByProperty(users, 'name', 'Борис')); // 1

```

// 6. Поиск всех вхождений

```

function findAllIndices(array, value) {
    const indices = [];
    let index = array.indexOf(value);

    while (index !== -1) {
        indices.push(index);
        index = array.indexOf(value, index + 1);
    }

    return indices;
}

```

```

console.log(findAllIndices(numbers, 3)); // [2, 5]
console.log(findAllIndices(numbers, 1)); // [0, 7]

```

// 7. includes() для строк и массивов

```

const str = "Hello World";
console.log(str.includes("World")); // true

```

```

const arr = [1, 2, 3];
console.log(arr.includes(2)); // true

```

// 8. Проверка нескольких значений

```

function includesAny(array, values) {
    return values.some(value => array.includes(value));
}

```

```

function includesAll(array, values) {
    return values.every(value => array.includes(value));
}

```

```

const testArray = [1, 2, 3, 4, 5];
console.log(includesAny(testArray, [6, 7, 3])); // true (есть 3)
console.log(includesAll(testArray, [2, 4, 6])); // false (нет 6)

```

// 9. Поиск с учетом типа (строгий vs нестрогий)

```
const mixed = [1, '1', 2, '2'];
```

```
console.log(mixed.indexOf(1)); // 0
```

```
console.log(mixed.indexOf('1')); // 1
```

```
console.log(mixed.includes(1)); // true
```

```
console.log(mixed.includes('1')); // true
```

// 10. Производительность поиска

```
const largeArray = Array.from({ length: 1000000 }, (_, i) => i);
```

```
console.time('indexOf в начале');
```

```
largeArray.indexOf(0); // Ищет с начала
```

```
console.timeEnd('indexOf в начале'); // Быстро
```

```
console.time('indexOf в конце');
```

```
largeArray.indexOf(999999); // Ищет почти весь массив
```

```
console.timeEnd('indexOf в конце'); // Медленно
```

```
console.time('includes в начале');
```

```
largeArray.includes(0);
```

```
console.timeEnd('includes в начале'); // Аналогично indexOf
```

// 11. Оптимизация поиска через Set

```
const searchSet = new Set(largeArray);
```

```
console.time('Set has');
```

```
searchSet.has(999999);
```

```
console.timeEnd('Set has'); // Очень быстро (O(1))
```

// 12. Поиск в отсортированном массиве (бинарный поиск)

```
function binarySearch(sortedArray, value) {
```

```
    let left = 0;
```

```
    let right = sortedArray.length - 1;
```

```
    while (left <= right) {
```

```
        const mid = Math.floor((left + right) / 2);
```

```
        if (sortedArray[mid] === value) {
```

```
            return mid;
```

```
        } else if (sortedArray[mid] < value) {
```

```
            left = mid + 1;
```

```
        } else {
```

```
            right = mid - 1;
```

```
        }
```

```

    }

    return -1;
}

const sorted = [1, 3, 5, 7, 9, 11, 13, 15];
console.log(binarySearch(sorted, 7)); // 3
console.log(binarySearch(sorted, 8)); // -1

// 13. Кастомные реализации
function customIndexOf(array, searchElement, fromIndex = 0) {
    // Нормализация fromIndex
    const start = fromIndex < 0 ?
        Math.max(array.length + fromIndex, 0) :
        Math.min(fromIndex, array.length);

    // Поиск
    for (let i = start; i < array.length; i++) {
        // Используем SameValueZero алгоритм (как ===, но NaN === NaN)
        if (array[i] === searchElement ||
            (Number.isNaN(searchElement) && Number.isNaN(array[i]))) {
            return i;
        }
    }

    return -1;
}

function customIncludes(array, searchElement, fromIndex = 0) {
    return customIndexOf(array, searchElement, fromIndex) !== -1;
}

console.log(customIndexOf([1, 2, NaN, 4], NaN)); // 2
console.log(customIncludes([1, 2, 3], 2)); // true

```


Часть 4: Сравнение и выбор методов

4.1. Сравнительная таблица методов

Метод	Мутирует массив	Возвращает	Сложность	Основное назначение
Итерационные				
<code>map()</code>	×	Новый массив	$O(n)$	Преобразование элементов
<code>filter()</code>	×	Новый массив	$O(n)$	Фильтрация по условию
<code>reduce()</code>	×	Агрегированное значение	$O(n)$	Агрегация элементов
<code>forEach()</code>	×	undefined	$O(n)$	Итерация с побочными эффектами
<code>every()</code>	×	boolean	$O(n)$	Проверка всех элементов
<code>some()</code>	×	boolean	$O(n)$	Проверка хотя бы одного
Мутирующие				
<code>push()</code>	□	Новая длина	$O(1)^*$	Добавление в конец
<code>pop()</code>	□	Удаленный элемент	$O(1)$	Удаление с конца
<code>shift()</code>	□	Удаленный элемент	$O(n)$	Удаление с начала
<code>unshift()</code>	□	Новая длина	$O(n)$	Добавление в начало
<code>splice()</code>	□	Массив удаленных	$O(n)$	Универсальная модификация
<code>sort()</code>	□	Тот же массив	$O(n \log n)$	Сортировка
<code>reverse()</code>	□	Тот же массив	$O(n)$	Реверс порядка
Доступные				
<code>concat()</code>	×	Новый массив	$O(n+m)$	Объединение массивов

Метод	Мутирует массив	Возвращает	Сложность	Основное назначение
<code>slice()</code>	×	Новый массив	$O(k)$	Извлечение подмассива
<code>indexOf()</code>	×	Индекс или -1	$O(n)$	Поиск первого вхождения
<code>lastIndexOf()</code>	×	Индекс или -1	$O(n)$	Поиск последнего вхождения
<code>includes()</code>	×	boolean	$O(n)$	Проверка наличия

*Примечание: `push()` имеет амортизированную сложность $O(1)$

4.2. Практические рекомендации по выбору методов

```

javascript

// Ситуация 1: Нужно преобразовать массив
// 📌 Используйте map()
const numbers = [1, 2, 3];
const doubled = numbers.map(x => x * 2); // [2, 4, 6]

// ✗ Не используйте forEach() для преобразования
const doubledBad = [];
numbers.forEach(x => doubledBad.push(x * 2)); // Сложнее читать

// Ситуация 2: Нужно отфильтровать массив
// 📌 Используйте filter()
const evens = numbers.filter(x => x % 2 === 0); // [2]

// ✗ Не используйте reduce() если не нужно агрегировать
const evensBad = numbers.reduce((acc, x) => {
  if (x % 2 === 0) acc.push(x);
  return acc;
}, []); // Избыточно

// Ситуация 3: Нужно найти элемент
const users = [
  { id: 1, name: "Анна" },
  { id: 2, name: "Борис" }
];

```

```

// Для объектов по условию:
// 📌 Используйте find()
const anna = users.find(u => u.id === 1);

// Для примитивов по значению:
// 📌 Используйте includes() для проверки наличия
const hasAnna = users.some(u => u.name === "Анна");

// Для получения индекса:
// 📌 Используйте findIndex()
const annaIndex = users.findIndex(u => u.id === 1);

// Ситуация 4: Нужно изменить массив
const items = ["a", "b", "c"];

// Добавление в конец:
// 📌 Используйте push() для производительности
items.push("d");

// Добавление в начало:
// ⚠ Используйте unshift(), но помните о производительности
items.unshift("z"); // Медленно для больших массивов

// Удаление элементов:
// 📌 Используйте splice() для гибкости
items.splice(1, 2); // Удаляет 'b' и 'c'

// Ситуация 5: Цепочки методов
// 📌 Читаемые цепочки
const result = users
  .filter(user => user.active) // Только активные
  .map(user => user.name)      // Только имена
  .sort()                     // Отсортировать
  .join(", ");                // Объединить в строку

// ✗ Избегайте длинных цепочек с побочными эффектами
const badChain = users
  .filter(user => {
    console.log(user); // Побочный эффект
    return user.active;
  })
  .map(user => {
    user.processed = true; // Мутация в map() - антипаттерн!
  });

```

```
        return user.name;
    });
```

// Ситуация 6: Работа с большими массивами

```
const largeArray = Array.from({ length: 1000000 }, (_, i) => i);
```

// 📌 Используйте for...of для производительности

```
let sum = 0;
for (const num of largeArray) {
    sum += num;
}
```

// ⚠ Методы массива могут быть медленнее

```
const sumSlow = largeArray.reduce((acc, num) => acc + num, 0);
```

// 📌 Оптимизация поиска через Set

```
const searchSet = new Set(largeArray);
const hasValue = searchSet.has(500000); // O(1)
```

// Ситуация 7: Функциональное программирование

// 📌 Чистые функции с map/filter/reduce

```
function processUsers(users) {
    return users
        .filter(user => user.age >= 18)
        .map(user => ({
            ...user,
            category: user.age >= 60 ? "senior" : "adult"
        }))
        .sort((a, b) => a.name.localeCompare(b.name));
}
```

// Ситуация 8: Асинхронные операции

```
async function processArrayAsync(array) {
    // ✗ forEach не работает с async/await
    // array.forEach(async item => {
    //     await processItem(item);
    // });
```

// 📌 Используйте for...of

```
for (const item of array) {
    await processItem(item);
}
```

// 📌 Или Promise.all для параллельной обработки

```
    await Promise.all(array.map(item => processItem(item)));
}
```

4.3. Производительность: тесты и сравнения

javascript

// Тест производительности различных операций

```
function runPerformanceTests() {
    const sizes = [100, 1000, 10000, 100000];

    for (const size of sizes) {
        console.log(`\n== Тест с массивом из ${size} элементов ==`);
        const array = Array.from({ length: size }, (_, i) => i);

        // Тест map() vs for loop
        console.time('map');
        const mapped = array.map(x => x * 2);
        console.timeEnd('map');

        console.time('for loop');
        const forResult = new Array(array.length);
        for (let i = 0; i < array.length; i++) {
            forResult[i] = array[i] * 2;
        }
        console.timeEnd('for loop');

        // Тест filter() vs for loop
        console.time('filter');
        const filtered = array.filter(x => x % 2 === 0);
        console.timeEnd('filter');

        console.time('for loop filter');
        const forFiltered = [];
        for (let i = 0; i < array.length; i++) {
            if (array[i] % 2 === 0) {
                forFiltered.push(array[i]);
            }
        }
        console.timeEnd('for loop filter');

        // Тест push() vs unshift()
        console.time('push 1000 элементов');
        const pushArray = [];
```

```

    for (let i = 0; i < 1000; i++) {
        pushArray.push(i);
    }
    console.timeEnd('push 1000 элементов');

    console.time('unshift 1000 элементов');
    const unshiftArray = [];
    for (let i = 0; i < 1000; i++) {
        unshiftArray.unshift(i);
    }
    console.timeEnd('unshift 1000 элементов');

    // Тест поиска
    console.time('includes (последний элемент)');
    array.includes(size - 1);
    console.timeEnd('includes (последний элемент)');

    console.time('indexOf (последний элемент)');
    array.indexOf(size - 1);
    console.timeEnd('indexOf (последний элемент)');

    const set = new Set(array);
    console.time('Set.has (последний элемент)');
    set.has(size - 1);
    console.timeEnd('Set.has (последний элемент)');
}

}

// runPerformanceTests();

// Результаты показывают:
// 1. Нативные методы обычно медленнее циклов, но читаемее
// 2. unshift() значительно медленнее push() для больших массивов
// 3. Set.has() намного быстрее indexOf()/includes()
// 4. Разница становится заметной на массивах от 10,000 элементов

// Оптимизация цепочек методов
function optimizeChain(array) {
    // ✗ Неоптимизированная цепочка
    const result1 = array
        .map(x => x * 2)
        .filter(x => x > 10)
        .map(x => x + 1);

```

```
//  Оптимизированная версия (меньше проходов)
const result2 = [];
for (const x of array) {
  const doubled = x * 2;
  if (doubled > 10) {
    result2.push(doubled + 1);
  }
}

//  Или комбинированный подход
const result3 = array.reduce((acc, x) => {
  const doubled = x * 2;
  if (doubled > 10) {
    acc.push(doubled + 1);
  }
  return acc;
}, []);

return result2;
}
```

Заключение: ключевые принципы работы с методами массивов

Основные выводы:

1. **Читаемость важнее микрооптимизаций** - в большинстве случаев методы массива достаточно быстры
2. **Выбирайте метод по задаче** - у каждого метода своя специализация
3. **Избегайте мутаций в итерационных методах** - это антипаттерн
4. **Учитывайте производительность для больших массивов** - иногда нужны циклы вместо методов
5. **Используйте современный синтаксис** - spread оператор, деструктуризация и т.д.

Рекомендации для разных ситуаций:

1. Для трансформации → `map()`
2. Для фильтрации → `filter()`
3. Для агрегации → `reduce()`

4. Для поиска элементов → `find()`, `findIndex()`
5. Для проверки условий → `every()`, `some()`
6. Для добавления элементов → `push()` (конец), `unshift()` (начало, осторожно)
7. Для удаления элементов → `pop()` (конец), `shift()` (начало), `splice()` (произвольно)
8. Для объединения → `concat()` или spread оператор
9. Для извлечения части → `slice()`
10. Для поиска значения → `includes()`, `indexOf()`

Современные тренды:

javascript

// 1. Функциональный стиль с цепочками

```
const processData = data =>
  data
    .filter(item => item.isActive)
    .map(item => transformItem(item))
    .sort(compareItems);
```

// 2. Деструктуризация в параметрах

```
const users = [{ id: 1, name: "Анна", age: 25 }];
users.map(({ name, age }) => `${name}, ${age} лет`);
```

// 3. Опциональная цепочка и нулевое слияние

```
users
  .filter(user => user?.age > 18) // Защита от undefined
  .map(user => user.name ?? "Без имени"); // Значение по умолчанию
```

// 4. async/await с массивами

```
async function processAll(items) {
  // Последовательно
  for (const item of items) {
    await process(item);
  }

  // Параллельно
  await Promise.all(items.map(process));
}
```

Понимание методов массивов и умение выбирать правильный инструмент для каждой задачи — ключевой навык JavaScript-разработчика. Сочетание методов массива с современными возможностями языка позволяет писать выразительный, эффективный и поддерживаемый код.

9.3. Многомерные массивы

Введение: от простых списков к сложным структурам

В реальных приложениях данные редко представляют собой простые одномерные списки. Чаще всего мы работаем с таблицами, матрицами, шахматными досками, трехмерными моделями — всё это примеры **многомерных массивов**.

Что такое многомерный массив?

Многомерный массив — это массив, элементами которого являются другие массивы. В JavaScript нет специального синтаксиса для многомерных массивов, они создаются естественным образом благодаря тому, что массивы могут содержать любые значения, включая другие массивы.

Часть 1: Двумерные массивы (матрицы)

1.1. Концепция матрицы

Двумерный массив можно представить как **таблицу с строками и столбцами**:

```
javascript
// Матрица 3x3 (3 строки, 3 столбца)
const matrix = [
  [1, 2, 3], // Строка 0
  [4, 5, 6], // Строка 1
  [7, 8, 9]  // Строка 2
];
```

1.2. Создание матриц

Способ 1: Литерал массива (для известных данных)

```
javascript

// Шахматная доска 8x8 (начальная позиция)
const chessBoard = [
  ['♔', '♚', '♙', '♖', '♜', '♛', '♞', '♝'],
  ['♟', '♞', '♝', '♜', '♛', '♚', '♙', '♔'],
  [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
  [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
  [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
  [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
  ['♟', '♞', '♝', '♜', '♛', '♚', '♙', '♔'],
  ['♔', '♚', '♙', '♖', '♜', '♛', '♞', '♝']
];
```

Способ 2: Ручное создание с циклами

```
javascript

// Создание пустой матрицы 5x5
const rows = 5;
const cols = 5;
const matrix = [];

for (let i = 0; i < rows; i++) {
  matrix[i] = []; // Создаем новую строку
  for (let j = 0; j < cols; j++) {
    matrix[i][j] = 0; // Заполняем нулями
  }
}

console.log(matrix);
// [
//   [0, 0, 0, 0, 0],
//   [0, 0, 0, 0, 0],
//   [0, 0, 0, 0, 0],
//   [0, 0, 0, 0, 0],
//   [0, 0, 0, 0, 0]
// ]
```

Способ 3: Использование Array.from() (ES6)

```
javascript

// Создание матрицы 4x3, заполненной null
const matrix = Array.from({ length: 4 }, () =>
  Array.from({ length: 3 }, () => null)
);

// Или с использованием fill()
const matrix2 = Array(4).fill().map(() => Array(3).fill(0));
```

1.3. Доступ к элементам

Доступ осуществляется через **два индекса**: первый для строки, второй для столбца.

```
javascript

const matrix = [
  [10, 20, 30],
  [40, 50, 60],
  [70, 80, 90]
];

// Получение элементов
console.log(matrix[0][0]); // 10 (первая строка, первый столбец)
console.log(matrix[1][2]); // 60 (вторая строка, третий столбец)
console.log(matrix[2][1]); // 80 (третья строка, второй столбец)

// Изменение элементов
matrix[1][1] = 55; // Изменяем центральный элемент
console.log(matrix);
// [
//   [10, 20, 30],
//   [40, 55, 60],
//   [70, 80, 90]
// ]
```

1.4. Итерация по матрице

Вложенные циклы for

```
javascript

const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

// Классический подход
for (let i = 0; i < matrix.length; i++) {
  const row = matrix[i];
  for (let j = 0; j < row.length; j++) {
    console.log(`matrix[${i}][${j}] = ${matrix[i][j]}`);
  }
}

// Использование for...of для строк
for (const row of matrix) {
  for (const cell of row) {
    console.log(cell);
  }
}

// Метод forEach
matrix.forEach((row, rowIndex) => {
  row.forEach((cell, colIndex) => {
    console.log(`${rowIndex},${colIndex}: ${cell}`);
  });
});
```

1.5. Практические примеры

Пример 1: Таблица умножения

```
javascript

function createMultiplicationTable(size) {
  const table = [];
```

```

    for (let i = 1; i <= size; i++) {
        const row = [];
        for (let j = 1; j <= size; j++) {
            row.push(i * j);
        }
        table.push(row);
    }

    return table;
}

const multiplicationTable = createMultiplicationTable(5);
console.log(multiplicationTable);
// [
//   [1, 2, 3, 4, 5],
//   [2, 4, 6, 8, 10],
//   [3, 6, 9, 12, 15],
//   [4, 8, 12, 16, 20],
//   [5, 10, 15, 20, 25]
// ]

```

Пример 2: Игровое поле (крестики-нолики)

javascript

```

class TicTacToe {
    constructor() {
        this.board = [
            [' ', ' ', ' '],
            [' ', ' ', ' '],
            [' ', ' ', ' ']
        ];
        this.currentPlayer = 'X';
    }

    makeMove(row, col) {
        if (this.board[row][col] === ' ') {
            this.board[row][col] = this.currentPlayer;
            this.currentPlayer = this.currentPlayer === 'X' ? 'O' : 'X';
            return true;
        }
        return false;
    }

    display() {

```

```

console.log('  0 1 2');
this.board.forEach((row, index) => {
  console.log(`${index} ${row.join('|')}`);
  if (index < 2) console.log('  -----');
});
}

checkWinner() {
  // Проверка строк
  for (let row of this.board) {
    if (row[0] !== ' ' && row[0] === row[1] && row[1] === row[2]) {
      return row[0];
    }
  }

  // Проверка столбцов
  for (let col = 0; col < 3; col++) {
    if (this.board[0][col] !== ' ' &&
        this.board[0][col] === this.board[1][col] &&
        this.board[1][col] === this.board[2][col]) {
      return this.board[0][col];
    }
  }

  // Проверка диагоналей
  if (this.board[0][0] !== ' ' &&
      this.board[0][0] === this.board[1][1] &&
      this.board[1][1] === this.board[2][2]) {
    return this.board[0][0];
  }

  if (this.board[0][2] !== ' ' &&
      this.board[0][2] === this.board[1][1] &&
      this.board[1][1] === this.board[2][0]) {
    return this.board[0][2];
  }

  return null;
}

// Использование
const game = new TicTacToe();
game.makeMove(0, 0); // X

```

```
game.makeMove(1, 1); // 0
game.makeMove(0, 1); // X
game.display();
```

Часть 2: Трёхмерные и N-мерные массивы

2.1. Трёхмерные массивы (кубы)

Трёхмерный массив можно представить как **куб** или **стопку матриц**:

```
javascript

// Трёхмерный массив: 3 слоя, каждый слой - матрица 2x3
const cube = [
  // Слой 0
  [
    [1, 2, 3],
    [4, 5, 6]
  ],
  // Слой 1
  [
    [7, 8, 9],
    [10, 11, 12]
  ],
  // Слой 2
  [
    [13, 14, 15],
    [16, 17, 18]
  ]
];

// Доступ к элементам: cube[слой][строка][столбец]
console.log(cube[0][1][2]); // 6
console.log(cube[2][0][1]); // 14
```

2.2. Создание N-мерных массивов

Рекурсивная функция для создания N-мерного массива

```
javascript
```

```

function createNDimArray(dimensions, defaultValue = 0) {
  // Базовый случай: если dimensions пуст, возвращаем значение по умолчанию
  if (dimensions.length === 0) {
    return defaultValue;
  }

  const [firstDim, ...restDims] = dimensions;
  const array = [];

  for (let i = 0; i < firstDim; i++) {
    // Рекурсивно создает массивы меньшей размерности
    array.push(createNDimArray(restDims, defaultValue));
  }

  return array;
}

// Примеры использования
const matrix3x3 = createNDimArray([3, 3], 0);
console.log(matrix3x3); // 3x3 матрица из нулей

const cube2x2x2 = createNDimArray([2, 2, 2], null);
console.log(cube2x2x2); // 2x2x2 куб из null

const hypercube = createNDimArray([2, 2, 2, 2], 'empty');
console.log(hypercube); // 4-мерный гиперкуб

```

2.3. Обход N-мерных массивов

Рекурсивный обход

javascript

```

function traverseNDimArray(array, callback, indices = []) {
  if (Array.isArray(array[0])) {
    // Если элемент сам является массивом, рекурсивно обходим его
    array.forEach((subArray, i) => {
      traverseNDimArray(subArray, callback, [...indices, i]);
    });
  } else {
    // Базовый случай: достигли "листьев" массива
    array.forEach((value, i) => {
      callback(value, [...indices, i]);
    });
  }
}

```



```

    });
  }
}

// Пример использования
const cube = [
  [
    [1, 2],
    [3, 4]
  ],
  [
    [5, 6],
    [7, 8]
  ]
];

traverseNDimArray(cube, (value, indices) => {
  console.log(`cube[${indices.join('][')}] = ${value}`);
});
// cube[0][0][0] = 1
// cube[0][0][1] = 2
// cube[0][1][0] = 3
// ...

```

Часть 3: Операции с многомерными массивами

3.1. Транспонирование матрицы

Транспонирование — замена строк на столбцы:

```

javascript
function transpose(matrix) {
  if (!matrix.length) return [];

  const rows = matrix.length;
  const cols = matrix[0].length;
  const result = [];

  // Создаем новую матрицу с перевернутыми размерами
  for (let j = 0; j < cols; j++) {
    result[j] = [];
  }
}

```

```

        for (let i = 0; i < rows; i++) {
            result[j][i] = matrix[i][j];
        }
    }

    return result;
}

// Использование
const matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];

const transposed = transpose(matrix);
console.log(transposed);
// [
//   [1, 4, 7],
//   [2, 5, 8],
//   [3, 6, 9]
// ]

```

3.2. Умножение матриц

```

javascript

function matrixMultiply(A, B) {
    const aRows = A.length;
    const aCols = A[0].length;
    const bRows = B.length;
    const bCols = B[0].length;

    // Проверка совместимости матриц
    if (aCols !== bRows) {
        throw new Error('Матрицы несовместимы для умножения');
    }

    // Создаем результирующую матрицу
    const result = Array.from({ length: aRows }, () =>
        Array(bCols).fill(0)
    );

    // Умножение

```

```

    for (let i = 0; i < aRows; i++) {
      for (let j = 0; j < bCols; j++) {
        let sum = 0;
        for (let k = 0; k < aCols; k++) {
          sum += A[i][k] * B[k][j];
        }
        result[i][j] = sum;
      }
    }

    return result;
  }
}

```

// Пример

```

const A = [
  [1, 2, 3],
  [4, 5, 6]
];

```

```

const B = [
  [7, 8],
  [9, 10],
  [11, 12]
];

```

```

const C = matrixMultiply(A, B);
console.log(C);
// [
//   [58, 64],
//   [139, 154]
// ]

```

3.3. Поворот матрицы

javascript

```

function rotateMatrix90Clockwise(matrix) {
  const n = matrix.length;
  const result = Array.from({ length: n }, () => Array(n).fill(0));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      result[j][n - 1 - i] = matrix[i][j];
    }
  }
}

```

```

    }

    return result;
}

// Пример
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(rotateMatrix90Clockwise(matrix));
// [
//   [7, 4, 1],
//   [8, 5, 2],
//   [9, 6, 3]
// ]

```

Часть 4: Практические применения

4.1. Обработка изображений

Матрицы идеально подходят для представления пикселей изображения:

```

javascript

class ImageProcessor {
  constructor(width, height) {
    this.width = width;
    this.height = height;
    this.pixels = Array.from({ length: height }, () =>
      Array.from({ length: width }, () => ({ r: 0, g: 0, b: 0 })))
  };
}

// Применение фильтра (например, размытие)
applyBlur(radius = 1) {
  const blurred = Array.from({ length: this.height }, () =>
    Array(this.width).fill(null)
  );
}

```

```

for (let y = 0; y < this.height; y++) {
  for (let x = 0; x < this.width; x++) {
    let rSum = 0, gSum = 0, bSum = 0, count = 0;

    // Собираем соседние пиксели
    for (let dy = -radius; dy <= radius; dy++) {
      for (let dx = -radius; dx <= radius; dx++) {
        const ny = y + dy;
        const nx = x + dx;

        if (ny >= 0 && ny < this.height && nx >= 0 && nx < this.width) {
          const pixel = this.pixels[ny][nx];
          rSum += pixel.r;
          gSum += pixel.g;
          bSum += pixel.b;
          count++;
        }
      }
    }

    blurred[y][x] = {
      r: Math.round(rSum / count),
      g: Math.round(gSum / count),
      b: Math.round(bSum / count)
    };
  }
}

this.pixels = blurred;
}

// Преобразование в оттенки серого
convertToGrayscale() {
  for (let y = 0; y < this.height; y++) {
    for (let x = 0; x < this.width; x++) {
      const pixel = this.pixels[y][x];
      const gray = Math.round(0.299 * pixel.r + 0.587 * pixel.g + 0.114 * pixel.b);
      this.pixels[y][x] = { r: gray, g: gray, b: gray };
    }
  }
}
}

```

4.2. Игровые миры (2D и 3D)

javascript

```
class GameMap {
  constructor(width, height, depth = 1) {
    this.width = width;
    this.height = height;
    this.depth = depth;

    // 3D массив: depth × height × width
    this.tiles = Array.from({ length: depth }, () =>
      Array.from({ length: height }, () =>
        Array(width).fill(0)
      )
    );

    // Заполняем границы стенками (значение 1)
    this.initializeWalls();
  }

  initializeWalls() {
    // В каждом слое
    for (let z = 0; z < this.depth; z++) {
      // Верхняя и нижняя границы
      for (let x = 0; x < this.width; x++) {
        this.tiles[z][0][x] = 1;
        this.tiles[z][this.height - 1][x] = 1;
      }

      // Левая и правая границы
      for (let y = 0; y < this.height; y++) {
        this.tiles[z][y][0] = 1;
        this.tiles[z][y][this.width - 1] = 1;
      }
    }
  }

  // Проверка, свободна ли клетка
  isFree(x, y, z = 0) {
    if (x < 0 || x >= this.width || y < 0 || y >= this.height || z < 0 || z >=
this.depth) {
      return false;
    }
  }
}
```

```

    return this.tiles[z][y][x] === 0;
}

// Поиск пути (упрощенный алгоритм)
findPath(start, end) {
    const [sx, sy, sz] = start;
    const [ex, ey, ez] = end;

    // Простой волновой алгоритм (для 2D)
    const distance = Array.from({ length: this.height }, () =>
        Array(this.width).fill(-1)
    );

    distance[sy][sx] = 0;
    const queue = [[sx, sy]];

    const directions = [
        [1, 0], [-1, 0], [0, 1], [0, -1]
    ];

    while (queue.length > 0) {
        const [x, y] = queue.shift();

        if (x === ex && y === ey) {
            break;
        }

        for (const [dx, dy] of directions) {
            const nx = x + dx;
            const ny = y + dy;

            if (this.isFree(nx, ny, sz) && distance[ny][nx] === -1) {
                distance[ny][nx] = distance[y][x] + 1;
                queue.push([nx, ny]);
            }
        }
    }

    // Восстановление пути
    if (distance[ey][ex] === -1) {
        return null; // Путь не найден
    }

    const path = [];

```

```

let [x, y] = [ex, ey];

while (x !== sx || y !== sy) {
  path.unshift([x, y, sz]);

  for (const [dx, dy] of directions) {
    const nx = x + dx;
    const ny = y + dy;

    if (nx >= 0 && nx < this.width && ny >= 0 && ny < this.height &&
        distance[ny][nx] === distance[y][x] - 1) {
      [x, y] = [nx, ny];
      break;
    }
  }
}

path.unshift([sx, sy, sz]);
return path;
}
}

```

4.3. Научные вычисления

javascript

```

class MatrixOperations {
  // Решение системы линейных уравнений методом Гаусса
  static gaussianElimination(A, b) {
    const n = A.length;
    const augmented = A.map((row, i) => [...row, b[i]]);

    // Прямой ход
    for (let i = 0; i < n; i++) {
      // Поиск максимального элемента в столбце
      let maxRow = i;
      for (let j = i + 1; j < n; j++) {
        if (Math.abs(augmented[j][i]) > Math.abs(augmented[maxRow][i])) {
          maxRow = j;
        }
      }

      // Перестановка строк
      [augmented[i], augmented[maxRow]] = [augmented[maxRow], augmented[i]];
    }
  }
}

```



```

// Приведение к треугольному виду
for (let j = i + 1; j < n; j++) {
    const factor = augmented[j][i] / augmented[i][i];
    for (let k = i; k <= n; k++) {
        augmented[j][k] -= factor * augmented[i][k];
    }
}
}

// Обратный ход
const x = Array(n).fill(0);
for (let i = n - 1; i >= 0; i--) {
    let sum = 0;
    for (let j = i + 1; j < n; j++) {
        sum += augmented[i][j] * x[j];
    }
    x[i] = (augmented[i][n] - sum) / augmented[i][i];
}

return x;
}

// Вычисление определителя матрицы
static determinant(matrix) {
    const n = matrix.length;

    // Базовый случай для матрицы 1x1
    if (n === 1) return matrix[0][0];

    // Базовый случай для матрицы 2x2
    if (n === 2) {
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];
    }

    let det = 0;

    // Разложение по первой строке
    for (let j = 0; j < n; j++) {
        // Создаем минор
        const minor = [];
        for (let i = 1; i < n; i++) {
            minor.push(matrix[i].filter((_, colIndex) => colIndex !== j));
        }

```

```

    // Рекурсивно вычисляем определитель минора
    const minorDet = this.determinant(minor);
    det += matrix[0][j] * minorDet * (j % 2 === 0 ? 1 : -1);
  }

  return det;
}
}

// Пример использования
const A = [
  [2, 1, -1],
  [-3, -1, 2],
  [-2, 1, 2]
];

const b = [8, -11, -3];

const solution = MatrixOperations.gaussianElimination(A, b);
console.log('Решение системы:', solution); // [2, 3, -1]

```

Часть 5: Производительность и оптимизация

5.1. Проблемы с производительностью

Многомерные массивы могут быть ресурсоемкими:

```

javascript

// Проблема: создание больших матриц
const size = 1000;
const bigMatrix = [];

// Медленный способ
console.time('Медленное создание');
for (let i = 0; i < size; i++) {
  bigMatrix[i] = [];
  for (let j = 0; j < size; j++) {
    bigMatrix[i][j] = Math.random();
  }
}

```

```

console.timeEnd('Медленное создание'); // ~500мс

// Более быстрый способ
console.time('Быстрое создание');
const fastMatrix = Array.from({ length: size }, () =>
  new Float64Array(size) // Типизированный массив быстрее
);
console.timeEnd('Быстрое создание'); // ~50мс

```

5.2. Типизированные массивы для числовых данных

javascript

```

// Float64Array для матриц с числами с плавающей точкой
class FloatMatrix {
  constructor(rows, cols) {
    this.rows = rows;
    this.cols = cols;
    this.data = new Float64Array(rows * cols);
  }

  get(row, col) {
    return this.data[row * this.cols + col];
  }

  set(row, col, value) {
    this.data[row * this.cols + col] = value;
  }

  // Быстрое умножение матриц с использованием SIMD-подобных операций
  multiply(other) {
    if (this.cols !== other.rows) {
      throw new Error('Несовместимые размеры матриц');
    }

    const result = new FloatMatrix(this.rows, other.cols);

    // Оптимизированный цикл
    for (let i = 0; i < this.rows; i++) {
      for (let k = 0; k < this.cols; k++) {
        const aik = this.get(i, k);
        for (let j = 0; j < other.cols; j++) {
          result.data[i * result.cols + j] += aik * other.get(k, j);
        }
      }
    }
  }
}

```

```

    }
}

    return result;
}
}

```

5.3. Кэширование и мемоизация

javascript

// Кэширование результатов операций с матрицами

```

class MatrixCache {
  constructor() {
    this.cache = new Map();
  }

  getKey(matrix, operation) {
    // Создаем ключ на основе содержимого матрицы и операции
    return JSON.stringify(matrix) + '|' + operation;
  }

  computeDeterminant(matrix) {
    const key = this.getKey(matrix, 'det');

    if (this.cache.has(key)) {
      console.log('Из кэша');
      return this.cache.get(key);
    }

    console.log('Вычисляем');
    const result = MatrixOperations.determinant(matrix);
    this.cache.set(key, result);

    return result;
  }
}

```

// Использование

```

const cache = new MatrixCache();
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]

```

```
];
```

```
console.log(cache.computeDeterminant(matrix)); // Вычисляем, возвращает 0  
console.log(cache.computeDeterminant(matrix)); // Из кэша, возвращает 0
```

Часть 6: Лучшие практики и рекомендации

6.1. Проверка размерности

javascript

```
function validateMatrixDimensions(matrices, expectedDimensions = null) {  
  if (!Array.isArray(matrices) || matrices.length === 0) {  
    throw new Error('Необходим массив матриц');  
  }  
  
  const firstMatrix = matrices[0];  
  const firstRows = firstMatrix.length;  
  const firstCols = firstMatrix[0].length;  
  
  // Проверяем, что все матрицы имеют одинаковые размеры  
  for (let i = 1; i < matrices.length; i++) {  
    const matrix = matrices[i];  
  
    if (!Array.isArray(matrix) || !Array.isArray(matrix[0])) {  
      throw new Error(`Матрица ${i} не является двумерным массивом`);  
    }  
  
    if (matrix.length !== firstRows || matrix[0].length !== firstCols) {  
      throw new Error(`Матрица ${i} имеет размер ${matrix.length}x${matrix[0].length},  
ождается ${firstRows}x${firstCols}`);  
    }  
  }  
  
  // Проверяем ожидаемые размеры, если они заданы  
  if (expectedDimensions) {  
    const [expectedRows, expectedCols] = expectedDimensions;  
    if (firstRows !== expectedRows || firstCols !== expectedCols) {  
      throw new Error(`Матрица имеет размер ${firstRows}x${firstCols}, ожидается $  
{expectedRows}x${expectedCols}`);  
    }  
  }  
}
```

```
    return [firstRows, firstCols];  
}
```

6.2. Иммутабельные операции

javascript

// Все операции возвращают новые матрицы, не изменяя оригиналы

```
class ImmutableMatrix {  
  constructor(data) {  
    this.data = data.map(row => [...row]); // Глубокая копия  
    this.rows = data.length;  
    this.cols = data[0]?.length || 0;  
  }  
}
```

// Операции возвращают новые экземпляры

```
add(other) {  
  validateMatrixDimensions([this.data, other.data]);  
  
  const result = [];  
  for (let i = 0; i < this.rows; i++) {  
    result[i] = [];  
    for (let j = 0; j < this.cols; j++) {  
      result[i][j] = this.data[i][j] + other.data[i][j];  
    }  
  }  
  
  return new ImmutableMatrix(result);  
}  
  
scale(factor) {  
  const result = [];  
  for (let i = 0; i < this.rows; i++) {  
    result[i] = this.data[i].map(value => value * factor);  
  }  
  
  return new ImmutableMatrix(result);  
}  
}
```

6.3. Документация и тестирование

javascript

```

/**
 * Класс для работы с матрицами
 * @class Matrix
 * @param {Array<Array<number>>}<div> data - Двумерный массив чисел
 * @throws {Error} Если data не является корректной матрицей
 */
class Matrix {
  constructor(data) {
    if (!Array.isArray(data) || data.length === 0 || !Array.isArray(data[0])) {
      throw new Error('Data должен быть двумерным массивом');
    }

    const firstRowLength = data[0].length;
    for (let i = 1; i < data.length; i++) {
      if (!Array.isArray(data[i]) || data[i].length !== firstRowLength) {
        throw new Error('Все строки матрицы должны иметь одинаковую длину');
      }
    }

    this.data = data;
    this.rows = data.length;
    this.cols = firstRowLength;
  }

  /**
   * Транспонирование матрицы
   * @returns {Matrix} Новая транспонированная матрица
   */
  transpose() {
    // ... реализация
  }

  /**
   * Умножение матриц
   * @param {Matrix} other - Матрица для умножения
   * @returns {Matrix} Результат умножения
   * @throws {Error} Если матрицы несовместимы
   */
  multiply(other) {
    // ... реализация
  }
}

// Модульные тесты для матричных операций

```

```
function testMatrixOperations() {
  console.assert = (condition, message) => {
    if (!condition) {
      throw new Error(`Тест не пройден: ${message}`);
    }
  };

  // Тест сложения
  const A = new Matrix([[1, 2], [3, 4]]);
  const B = new Matrix([[5, 6], [7, 8]]);
  const C = A.add(B);
  console.assert(C.data[0][0] === 6, 'Сложение 1+5 должно быть 6');
  console.assert(C.data[1][1] === 12, 'Сложение 4+8 должно быть 12');

  // Тест умножения
  const D = new Matrix([[1, 2], [3, 4]]);
  const E = new Matrix([[2, 0], [1, 2]]);
  const F = D.multiply(E);
  console.assert(F.data[0][0] === 4, 'Умножение 1*2+2*1 должно быть 4');
  console.assert(F.data[1][1] === 8, 'Умножение 3*1+4*2 должно быть 11');

  console.log('Все тесты пройдены успешно!');
}
```

Заключение

Многомерные массивы — мощный инструмент в арсенале JavaScript-разработчика. Они позволяют:

1. **Моделировать сложные структуры данных** — от игровых полей до научных моделей
2. **Обрабатывать изображения и графику** — пиксельные данные идеально ложатся на матрицы
3. **Выполнять математические вычисления** — линейная алгебра, машинное обучение, физические симуляции
4. **Создавать сложные UI** — таблицы, сетки, календари

Ключевые принципы работы с многомерными массивами:

1. **Выбирайте правильное представление** — обычные массивы для гибкости, типизированные для производительности

2. **Оптимизируйте вложенные циклы** — порядок итерации влияет на производительность
3. **Используйте рекурсию для N-мерных массивов** — упрощает работу с массивами произвольной размерности
4. **Кэшируйте результаты** — матричные операции могут быть дорогими
5. **Пишите тесты** — ошибки в матричных операциях трудно отлаживать

Многомерные массивы в JavaScript, хоть и не имеют специального синтаксиса, предоставляют всю необходимую функциональность для решения сложных задач. Понимание их устройства и освоение лучших практик работы с ними — важный шаг в становлении профессионального разработчика.

9.4. Псевдомассивы и преобразование в массивы

Введение: что такое псевдомассивы и почему они существуют

В JavaScript существуют объекты, которые выглядят как массивы, ведут себя похоже на массивы, но не являются настоящими массивами. Это **псевдомассивы** (array-like objects).

Исторический контекст

Псевдомассивы появились как компромисс между:

1. **Производительностью** — нативные объекты быстрее
2. **Обратной совместимостью** — старый код должен работать
3. **Простотой реализации** — не все объекты должны быть полноценными массивами

Часть 1: Что такое псевдомассивы

1.1. Основные характеристики псевдомассивов

Псевдомассив — это объект, который:

1. Имеет числовые индексы (свойства '0', '1', '2', ...)
2. Имеет свойство `length`, указывающее количество элементов
3. **НЕ** имеет методов массива (`push`, `pop`, `forEach`, `map` и т.д.)

javascript

// Пример псевдомассива

```
const arrayLike = {  
  0: 'первый',  
  1: 'второй',  
  2: 'третий',  
  length: 3  
}
```

```

};

console.log(arrayLike[0]); // 'первый'
console.log(arrayLike[1]); // 'второй'
console.log(arrayLike.length); // 3

// Но это НЕ массив!
console.log(Array.isArray(arrayLike)); // false
console.log(arrayLike.push); // undefined - нет метода push
console.log(arrayLike.forEach); // undefined - нет метода forEach

```

1.2. Типичные примеры псевдомассивов

1.2.1. Объект `arguments` в функциях

```

javascript

function exampleFunction() {
  console.log('arguments:', arguments);
  console.log('Type of arguments:', typeof arguments); // 'object'
  console.log('Is array?', Array.isArray(arguments)); // false

  // arguments - псевдомассив
  console.log('arguments.length:', arguments.length);
  console.log('arguments[0]:', arguments[0]);
  console.log('arguments[1]:', arguments[1]);

  // Но не имеет методов массива
  console.log('arguments.forEach:', arguments.forEach); // undefined
}

exampleFunction('первый', 'второй', 'третий');

// В стрелочных функциях arguments не существует
const arrowExample = () => {
  // console.log(arguments); // Ошибка: arguments is not defined
};

```

1.2.2. Результат `document.getElementsByClassName()` и подобных

```

javascript

// В браузере
const elements = document.getElementsByClassName('my-class');

```

```

console.log('elements:', elements);
console.log('Is array?', Array.isArray(elements)); // false
console.log('elements.length:', elements.length);
console.log('elements[0]:', elements[0]);

// Можно перебирать как массив
for (let i = 0; i < elements.length; i++) {
  console.log(elements[i]);
}

// Но методы массива не работают
// elements.forEach(el => console.log(el)); // Ошибка!

```

1.2.3. Строки

```

javascript

const str = 'Привет';

console.log('str[0]:', str[0]); // 'П'
console.log('str[1]:', str[1]); // 'р'
console.log('str.length:', str.length); // 6
console.log('Array.isArray(str):', Array.isArray(str)); // false

// Строки - особый случай: они иммутабельны
str[0] = 'X'; // Не работает
console.log(str); // 'Привет'

```

1.2.4. NodeList (результат querySelectorAll)

```

javascript

// В браузере
const nodeList = document.querySelectorAll('div');

console.log('nodeList:', nodeList);
console.log('Is array?', Array.isArray(nodeList)); // false
console.log('nodeList.length:', nodeList.length);
console.log('nodeList.forEach:', nodeList.forEach); // ✓ У NodeList есть forEach!

// NodeList имеет некоторые методы массива
nodeList.forEach(node => console.log(node));

// Но не все
console.log('nodeList.map:', nodeList.map); // undefined

```

```
console.log('nodeList.filter:', nodeList.filter); // undefined
```

1.2.5. Коллекции из DOM API

javascript

```
// HTMLCollection
```

```
const htmlCollection = document.forms; // Все формы на странице
```

```
console.log('htmlCollection:', htmlCollection);
console.log('Is array?', Array.isArray(htmlCollection)); // false
console.log('htmlCollection.length:', htmlCollection.length);
```

```
// NamedNodeMap (атрибуты элемента)
```

```
const element = document.createElement('div');
element.setAttribute('id', 'test');
element.setAttribute('class', 'container');
```

```
const attributes = element.attributes;
console.log('attributes:', attributes);
console.log('attributes.length:', attributes.length);
console.log('attributes[0]:', attributes[0]);
```

Часть 2: Как отличить псевдомассив от настоящего массива

2.1. Проверка типа

javascript

```
function isArrayLike(obj) {
  // Проверяем, что объект существует
  if (obj == null) return false;

  // Проверяем, что это объект (не примитив)
  if (typeof obj !== 'object' && typeof obj !== 'function') return false;

  // Проверяем наличие length и что это число
  if (typeof obj.length !== 'number') return false;

  // Проверяем, что length неотрицательное целое число
  if (obj.length < 0) return false;
  if (obj.length !== Math.floor(obj.length)) return false;
}
```

```

// Проверяем, что length не превышает безопасное значение
if (obj.length > Math.pow(2, 32) - 1) return false;

return true;
}

// Тесты
console.log(isArrayLike([])); // true
console.log(isArrayLike({length: 0})); // true
console.log(isArrayLike({0: 'a', 1: 'b', length: 2})); // true
console.log(isArrayLike('строка')); // false (строка - примитив)
console.log(isArrayLike(123)); // false
console.log(isArrayLike(null)); // false
console.log(isArrayLike(undefined)); // false
console.log(isArrayLike({length: -1})); // false
console.log(isArrayLike({length: 3.14})); // false

```

2.2. Более строгая проверка

```

javascript

function isArrayLike(obj) {
  // Базовые проверки
  if (!isArrayLike(obj)) return false;

  // Дополнительные проверки для псевдомассивов

  // Проверяем, что свойства с индексами существуют
  // (не обязательно для всех псевдомассивов, но для большинства)
  for (let i = 0; i < obj.length; i++) {
    if (!(i in obj)) {
      // Если индекс отсутствует, это может быть разреженный псевдомассив
      // или не совсем обычный псевдомассив
      return false;
    }
  }

  return true;
}

// Примеры
const sparseArrayLike = {length: 3}; // length=3, но нет 0,1,2
console.log(isArrayLike(sparseArrayLike)); // false

```

```
const denseArrayLike = {0: 'a', 1: 'b', 2: 'c', length: 3};
console.log(isTrueArrayLike(denseArrayLike)); // true
```

2.3. Проверка наличия методов массива

```
javascript

function isProbablyArray(obj) {
  // Самый простой способ
  if (Array.isArray(obj)) return true;

  // Проверяем наличие методов массива
  const arrayMethods = ['push', 'pop', 'slice', 'concat'];

  for (const method of arrayMethods) {
    if (typeof obj[method] !== 'function') {
      return false;
    }
  }

  return true;
}

// Примеры
console.log(isProbablyArray([])); // true
console.log(isProbablyArray({0: 'a', length: 1, push: () => {}})); // false (нет других методов)
console.log(isProbablyArray(document.querySelectorAll('div'))); // false
```

Часть 3: Преобразование псевдомассивов в настоящие массивы

3.1. Исторические способы (до ES6)

3.1.1. Использование цикла

```
javascript

// Самый прямой, но многословный способ
function argumentsToArrayOldSchool() {
```

```

const args = arguments;
const result = [];

for (let i = 0; i < args.length; i++) {
    result[i] = args[i];
}

return result;
}

// Пример использования
const array1 = argumentsToArrayOldSchool('a', 'b', 'c');
console.log(array1); // ['a', 'b', 'c']
console.log(Array.isArray(array1)); // true

```

3.1.2. Метод slice с call/apply

```

javascript

// Самый популярный способ до ES6
function convertToArray(arrayLike) {
    // Используем Array.prototype.slice
    return Array.prototype.slice.call(arrayLike);

    // Или с Function.prototype.call
    // return [].slice.call(arrayLike);
}

// Примеры
function exampleFunction() {
    const argsArray = Array.prototype.slice.call(arguments);
    console.log('argsArray:', argsArray);
    console.log('Is array:', Array.isArray(argsArray));
}

exampleFunction(1, 2, 3, 4, 5);

// Работает с любыми псевдомассивами
const arrayLike = {0: 'a', 1: 'b', 2: 'c', length: 3};
const realArray = convertToArray(arrayLike);
console.log(realArray); // ['a', 'b', 'c']
console.log(realArray.map(x => x.toUpperCase())); // ['A', 'B', 'C']

```


3.1.3. Почему slice.call работает?

```
javascript

// Разберем подробно, как работает Array.prototype.slice.call()

// 1. Array.prototype.slice - это функция
console.log(typeof Array.prototype.slice); // 'function'

// 2. Она ожидает контекст (this), который будет массивом
// Обычный вызов:
const arr = [1, 2, 3, 4, 5];
const sliced = arr.slice(1, 3); // [2, 3]
// Здесь arr становится this внутри slice

// 3. call() позволяет передать произвольный объект как this
const fakeArray = {0: 'a', 1: 'b', 2: 'c', length: 3};
const result = Array.prototype.slice.call(fakeArray);

// Что происходит внутри:
// 1. slice создает новый массив
// 2. Начинает перебирать свойства от 0 до this.length-1
// 3. Копирует значения в новый массив
// 4. Возвращает новый массив

// Псевдокод реализации slice:
/*
Array.prototype.slice = function(start, end) {
  const result = [];
  const len = this.length;

  // ... обработка start и end ...

  for (let i = start; i < end; i++) {
    if (i in this) {
      result.push(this[i]);
    } else {
      result.length++; // Для разреженных массивов
    }
  }

  return result;
}
*/
```

3.1.4. Использование apply с concat

```
javascript

// Еще один старый трюк
function convertWithConcat(arrayLike) {
    return Array.prototype.concat.apply([], arrayLike);
}

// Пример
const nodeList = document.querySelectorAll('div'); // Псевдомассив
const divArray = Array.prototype.concat.apply([], nodeList);
```

3.2. Современные способы (ES6+)

3.2.1. Array.from() - рекомендуемый способ

```
javascript

// ES6 представил специальный метод для этой задачи
function convertWithArrayFrom(arrayLike) {
    return Array.from(arrayLike);
}

// Примеры
function modernExample() {
    const argsArray = Array.from(arguments);
    console.log(argsArray);
    console.log(Array.isArray(argsArray));

    // Использование методов массива
    const doubled = argsArray.map(x => x * 2);
    console.log(doubled);
}

modernExample(1, 2, 3);

// Работает со всеми итерируемыми объектами
const set = new Set([1, 2, 3, 3, 2, 1]);
const arrayFromSet = Array.from(set);
console.log(arrayFromSet); // [1, 2, 3]

const map = new Map([['a', 1], ['b', 2]]);
const arrayFromMap = Array.from(map);
```

```
console.log(arrayFromMap); // [['a', 1], ['b', 2]]
```

3.2.2. Spread оператор (...)

javascript

// Spread оператор также преобразует итерируемые объекты

```
function convertWithSpread(arrayLike) {  
  // Работает только с итерируемыми объектами!  
  return [...arrayLike];  
}
```

// Пример с arguments (arguments - итерируемый в строгом режиме)

```
function spreadExample(...args) { // Rest параметр уже создает массив  
  // Но если нужно работать с arguments:  
  'use strict';  
  const argsArray = [...arguments];  
  console.log(argsArray);  
}
```

```
spreadExample(1, 2, 3);
```

// С другими итерируемыми объектами

```
const set = new Set(['a', 'b', 'c']);  
const arrayFromSet = [...set];  
console.log(arrayFromSet); // ['a', 'b', 'c']
```

```
const map = new Map([[1, 'a'], [2, 'b']]);  
const keysArray = [...map.keys()]; // [1, 2]  
const valuesArray = [...map.values()]; // ['a', 'b']
```

3.2.3. Array.of() для отдельных значений

javascript

// Array.of() создает массив из аргументов

// Полезно для преобразования отдельных значений

```
const array1 = Array.of(1, 2, 3);  
console.log(array1); // [1, 2, 3]
```

// В отличие от new Array()

```
const array2 = new Array(1, 2, 3);  
console.log(array2); // [1, 2, 3]
```

// Но с одним аргументом:

```
const array3 = Array.of(5);
```

```
console.log(array3); // [5] - массив с одним элементом

const array4 = new Array(5);
console.log(array4); // [ , , , , ] - массив с 5 пустыми слотами
```

3.3. Сравнение методов преобразования

```
javascript

// Создадим тестовый псевдомассив
const arrayLike = {
  0: 'zero',
  1: 'one',
  2: 'two',
  length: 3
};

// Добавим Symbol.iterator для некоторых тестов
arrayLike[Symbol.iterator] = function*() {
  for (let i = 0; i < this.length; i++) {
    yield this[i];
  }
};

// Тестируем разные методы
console.time('Цикл for');
const withLoop = [];
for (let i = 0; i < arrayLike.length; i++) {
  withLoop.push(arrayLike[i]);
}
console.timeEnd('Цикл for');

console.time('slice.call');
const withSlice = Array.prototype.slice.call(arrayLike);
console.timeEnd('slice.call');

console.time('Array.from');
const withFrom = Array.from(arrayLike);
console.timeEnd('Array.from');

console.time('Spread operator');
const withSpread = [...arrayLike];
console.timeEnd('Spread operator');
```

```
// Проверяем результаты
console.log('Все методы дают одинаковый результат?',
  JSON.stringify(withLoop) === JSON.stringify(withSlice) &&
  JSON.stringify(withSlice) === JSON.stringify(withFrom) &&
  JSON.stringify(withFrom) === JSON.stringify(withSpread)
); // true
```

Часть 4: Особые случаи преобразования

4.1. Строки как псевдомассивы

```
javascript

const str = 'Привет, мир!';

// Преобразование строки в массив символов
const chars1 = Array.from(str);
console.log(chars1); // ['П', 'р', 'и', 'в', 'е', 'т', ',', ' ', 'м', 'и', 'р', '!']

const chars2 = [...str];
console.log(chars2); // То же самое

const chars3 = str.split('');
console.log(chars3); // ['П', 'р', 'и', 'в', 'е', 'т', ',', ' ', 'м', 'и', 'р', '!']

// Важно: split('') может некорректно обрабатывать суррогатные пары
const emoji = '😄👍';
console.log(emoji.split('')); // ['\uD83D', '\uDE00', '\uD83C', '\uDF89'] - неправильно!
console.log([...emoji]); // ['😄', '👍'] - правильно!
console.log(Array.from(emoji)); // ['😄', '👍'] - правильно!

// Для старых браузеров
function stringToArray(str) {
  const result = [];
  let i = 0;
  while (i < str.length) {
    const charCode = str.charCodeAt(i);

    // Определяем, является ли символ частью суррогатной пары
    if (charCode >= 0xD800 && charCode <= 0xDBFF && i + 1 < str.length) {
      const nextCharCode = str.charCodeAt(i + 1);
      if (nextCharCode >= 0xDC00 && nextCharCode <= 0xDFFF) {
```

```

        // Суррогатная пара
        result.push(str.substring(i, i + 2));
        i += 2;
        continue;
    }
}

// Обычный символ
result.push(str.charAt(i));
i++;
}
return result;
}

console.log(stringToArray(emoji)); // ['😄', '🔴']

```

4.2. Коллекции DOM

```

javascript

// Преобразование NodeList, HTMLCollection в массив
function getDomElementsAsArray(selector) {
    const elements = document.querySelectorAll(selector);

    // Современный способ
    return Array.from(elements);

    // Или
    // return [...elements];

    // Или для старых браузеров
    // return Array.prototype.slice.call(elements);
}

// Пример с обработкой
function processAllDivs() {
    const divs = getDomElementsAsArray('div');

    // Теперь можем использовать методы массива
    divs.forEach((div, index) => {
        div.textContent = `Div ${index + 1}`;
    });

    // Фильтрация

```

```

const visibleDivs = divs.filter(div =>
  div.style.display !== 'none'
);

// Преобразование
const divContents = divs.map(div => div.textContent);

return {
  total: divs.length,
  visible: visibleDivs.length,
  contents: divContents
};
}

```

4.3. Объект arguments в деталях

```

javascript

// arguments в обычной функции
function legacyFunction(a, b) {
  console.log('a:', a);
  console.log('b:', b);
  console.log('arguments.length:', arguments.length);
  console.log('arguments[0]:', arguments[0]);
  console.log('arguments[1]:', arguments[1]);
  console.log('arguments[2]:', arguments[2]);

  // arguments связан с параметрами
  arguments[0] = 'изменено';
  console.log('a после изменения arguments[0]:', a); // 'изменено'

  // Но эта связь односторонняя в нестрогом режиме
  a = 'снова изменено';
  console.log('arguments[0] после изменения a:', arguments[0]); // 'снова изменено'
}

legacyFunction('original A', 'original B', 'extra C');

// В строгом режиме связь разрывается
function strictFunction(a, b) {
  'use strict';
  console.log('a:', a);
  arguments[0] = 'изменено в strict';
  console.log('a после изменения arguments[0]:', a); // Остается 'original'
}

```

```

}

strictFunction('original');

// Преобразование arguments в массив
function sumAll() {
  // Старый способ
  const args1 = Array.prototype.slice.call(arguments);

  // Новый способ
  const args2 = Array.from(arguments);

  // Самый новый способ - rest параметры
  // function sumAll(...args) { ... }

  return args1.reduce((sum, num) => sum + num, 0);
}

console.log(sumAll(1, 2, 3, 4, 5)); // 15

```

4.4. Генераторы как псевдомассивы

```

javascript

// Генераторы - итерируемые объекты
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
  yield 4;
  yield 5;
}

const generator = numberGenerator();

// Генератор можно преобразовать в массив
const numbers1 = Array.from(generator);
console.log(numbers1); // [1, 2, 3, 4, 5]

// Но генератор одноразовый!
const numbers2 = Array.from(generator);
console.log(numbers2); // [] - генератор исчерпан

// Решение: создаем новый генератор

```



```
function* freshGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const fresh1 = [...freshGenerator()];
const fresh2 = [...freshGenerator()];
console.log(fresh1, fresh2); // [1, 2, 3], [1, 2, 3]
```

Часть 5: Создание псевдомассивов

5.1. Зачем создавать псевдомассивы?

```
javascript

// Иногда псевдомассивы полезнее настоящих массивов:

// 1. Для производительности
class FastCollection {
  constructor() {
    this.data = Object.create(null); // Быстрее обычного объекта
    this.length = 0;
  }

  add(item) {
    this.data[this.length] = item;
    this.length++;
    return this;
  }

  get(index) {
    return this.data[index];
  }

  // Не реализуем методы массива для экономии памяти
}

// 2. Для совместимости с API, ожидающим псевдомассивы
function expectsArrayLike(obj) {
  // Ожидает объект с length и числовыми индексами
  for (let i = 0; i < obj.length; i++) {
```

```
    console.log(obj[i]);
  }
}
```

// 3. Для ленивых вычислений

```
class LazyArray {
  constructor(generator, length) {
    this.generator = generator;
    this.length = length;
  }

  get(index) {
    if (index < 0 || index >= this.length) {
      return undefined;
    }
    return this.generator(index);
  }
}

const squares = new LazyArray(i => i * i, 10);
console.log(squares.get(5)); // 25 (вычисляется по требованию)
```

5.2. Практический пример: кэширующий псевдомассив

javascript

```
class CachedArrayLike {
  constructor(sourceArrayLike) {
    this.source = sourceArrayLike;
    this.length = sourceArrayLike.length;
    this.cache = new Map();
  }

  get(index) {
    if (index < 0 || index >= this.length) {
      return undefined;
    }

    // Проверяем кэш
    if (this.cache.has(index)) {
      console.log(`Кэшированный элемент ${index}`);
      return this.cache.get(index);
    }
  }
}
```

```

    // Получаем из источника (может быть дорогой операцией)
    console.log(`Вычисляем элемент ${index}`);
    const value = this.source[index];

    // Кэшируем
    this.cache.set(index, value);

    return value;
}

// Преобразование в настоящий массив (с кэшированием)
toArray() {
    const result = [];
    for (let i = 0; i < this.length; i++) {
        result.push(this.get(i)); // Использует кэш
    }
    return result;
}
}

// Пример использования
const expensiveSource = {
    0: 'element0', // Предположим, получение каждого элемента дорогое
    1: 'element1',
    2: 'element2',
    3: 'element3',
    length: 4
};

const cached = new CachedArrayLike(expensiveSource);

console.log(cached.get(0)); // "Вычисляем элемент 0"
console.log(cached.get(0)); // "Кэшированный элемент 0"
console.log(cached.toArray()); // Первый вызов для 1,2,3, затем все из кэша

```

5.3. Псевдомассив с геттерами

```

javascript

// Псевдомассив с вычисляемыми свойствами
class ComputedArrayLike {
    constructor(size, computeFn) {
        this._size = size;
        this._compute = computeFn;
    }
}

```

```

// Создаем геттеры для каждого индекса
for (let i = 0; i < size; i++) {
  Object.defineProperty(this, i, {
    get: () => this._compute(i),
    enumerable: true,
    configurable: false
  });
}

// Свойство length
Object.defineProperty(this, 'length', {
  value: size,
  writable: false,
  enumerable: false,
  configurable: false
});
}
}

// Пример
const computed = new ComputedArrayLike(5, i => `Элемент ${i}² = ${i * i}`);

console.log(computed[0]); // "Элемент 0² = 0"
console.log(computed[3]); // "Элемент 3² = 9"
console.log(computed.length); // 5

// Преобразование в массив
const asArray = Array.from(computed);
console.log(asArray); // ["Элемент 0² = 0", "Элемент 1² = 1", ...]

```

Часть 6: Продвинутые техники

6.1. Преобразование с маппингом

javascript

```

// Array.from принимает второй аргумент - функцию маппинга
function processArguments() {
  // Преобразование arguments в массив с одновременным преобразованием
  const processedArgs = Array.from(arguments, (arg, index) => ({
    index,

```

```

        value: arg,
        type: typeof arg,
        squared: typeof arg === 'number' ? arg * arg : null
    }));

    return processedArgs;
}

console.log(processArguments(1, 'text', true, null));

// То же самое с slice.call и map
function processArgumentsOld() {
    const argsArray = Array.prototype.slice.call(arguments);
    return argsArray.map((arg, index) => ({
        index,
        value: arg,
        type: typeof arg
    }));
}

```

6.2. Обработка разреженных псевдомассивов

```

javascript

// Разреженный псевдомассив (не все индексы существуют)
const sparseArrayLike = {
    0: 'a',
    2: 'c', // Нет элемента с индексом 1
    5: 'f',
    length: 10
};

// Array.from правильно обрабатывает разреженные массивы
const fromSparse = Array.from(sparseArrayLike);
console.log(fromSparse); // ['a', undefined, 'c', undefined, undefined, 'f', undefined, undefined, undefined, undefined]

// Проверка, какие индексы существуют
const existingIndices = [];
for (let i = 0; i < sparseArrayLike.length; i++) {
    if (i in sparseArrayLike) {
        existingIndices.push(i);
    }
}

```

```

console.log('Существующие индексы:', existingIndices); // [0, 2, 5]

// Фильтрация только существующих значений
const onlyExisting = Array.from(sparseArrayLike).filter((val, idx) =>
  idx in sparseArrayLike
);
console.log(onlyExisting); // ['a', 'c', 'f']

```

6.3. Цепочка преобразований

```

javascript

// Сложное преобразование псевдомассива
function complexTransformation(arrayLike) {
  return Array.from(arrayLike)
    .filter(item => item !== null) // Убираем null и undefined
    .map(item => ({
      original: item,
      processed: String(item).toUpperCase()
    }))
    .reduce((acc, item, index) => {
      acc[item.processed] = index;
      return acc;
    }, {});
}

// Пример
const mixed = {
  0: 'hello',
  1: null,
  2: 'world',
  3: undefined,
  4: 123,
  length: 5
};

console.log(complexTransformation(mixed));
// { HELLO: 0, WORLD: 1, '123': 2 }

```

6.4. Производительные преобразования для больших данных

```

javascript

// Для очень больших псевдомассивов

```

```

function convertLargeArrayLike(arrayLike, batchSize = 10000) {
  const total = arrayLike.length;
  const result = new Array(total);

  // Обработка батчами для избежания блокировки UI
  function processBatch(start) {
    const end = Math.min(start + batchSize, total);

    for (let i = start; i < end; i++) {
      result[i] = arrayLike[i];
    }

    // Если остались элементы, планируем следующий батч
    if (end < total) {
      setTimeout(() => processBatch(end), 0);
    } else {
      console.log('Преобразование завершено');
      // Здесь можно вызвать callback или обработать результат
    }

    // Возвращаем прогресс
    return {
      processed: end,
      total,
      progress: (end / total * 100).toFixed(1) + '%'
    };
  }

  // Начинаем обработку
  return processBatch(0);
}

// Пример использования с большим NodeList
// const allElements = document.querySelectorAll('*');
// convertLargeArrayLike(allElements);

```

Часть 7: Распространенные ошибки и их решение

7.1. Ошибка: "arrayLike.forEach is not a function"

javascript

```
// ✗ НЕПРАВИЛЬНО
function wrongExample() {
  arguments.forEach(arg => console.log(arg)); // Ошибка!
}

// ✔ ПРАВИЛЬНО
function correctExample1() {
  // Преобразуем в массив
  const argsArray = Array.from(arguments);
  argsArray.forEach(arg => console.log(arg));
}

function correctExample2() {
  // Используем for...of (работает с итерируемыми объектами)
  for (const arg of arguments) {
    console.log(arg);
  }
}

function correctExample3(...args) { // Rest параметры
  args.forEach(arg => console.log(arg));
}
```

7.2. Ошибка: изменение исходного объекта

```
javascript

const arrayLike = {0: 'a', 1: 'b', length: 2};

// ✗ НЕПРАВИЛЬНО (может изменить исходный объект)
const wrongArray = arrayLike;
wrongArray[0] = 'modified';
console.log(arrayLike[0]); // 'modified' - исходный объект изменен!

// ✔ ПРАВИЛЬНО (создаем копию)
const correctArray1 = Array.from(arrayLike);
correctArray1[0] = 'modified';
console.log(arrayLike[0]); // 'a' - исходный объект не изменен

const correctArray2 = [...arrayLike]; // Если arrayLike итерируемый
const correctArray3 = Array.prototype.slice.call(arrayLike);
```


7.3. Ошибка: игнорирование length

```
javascript

// ✗ НЕПРАВИЛЬНО
const badArrayLike = {
  0: 'a',
  1: 'b',
  2: 'c'
  // Нет свойства length!
};

try {
  const arr = Array.from(badArrayLike); // Преобразует только итерируемые свойства
  console.log(arr); // [] - пустой массив!
} catch(e) {
  console.error(e);
}

// ✔ ПРАВИЛЬНО
const goodArrayLike = {
  0: 'a',
  1: 'b',
  2: 'c',
  length: 3
};

console.log(Array.from(goodArrayLike)); // ['a', 'b', 'c']
```

7.4. Ошибка: неправильная работа с разреженными массивами

```
javascript

const sparse = {0: 'a', 2: 'c', length: 3};

// ✗ НЕПРАВИЛЬНО (может создать непредсказуемый результат)
const badResult = [];
for (const key in sparse) {
  if (!isNaN(parseInt(key))) { // Попытка отфильтровать только числовые ключи
    badResult.push(sparse[key]);
  }
}

console.log(badResult); // ['a', 'c', 3] - length тоже попал!
```

```
//  ПРАВИЛЬНО
const goodResult1 = Array.from(sparse);
console.log(goodResult1); // ['a', undefined, 'c']
```

```
// Или с явной проверкой
const goodResult2 = [];
for (let i = 0; i < sparse.length; i++) {
  if (i in sparse) {
    goodResult2.push(sparse[i]);
  }
}
console.log(goodResult2); // ['a', 'c']
```

Часть 8: Best Practices

8.1. Когда что использовать

javascript

// Рекомендации по выбору метода преобразования

```
class ConversionGuide {
  static recommendMethod(arrayLike) {
    // Проверяем поддержку ES6
    const supportsES6 = typeof Array.from === 'function';

    // Проверяем, является ли объект итерируемым
    const isIterable = arrayLike && typeof arrayLike[Symbol.iterator] === 'function';

    if (supportsES6) {
      // ES6+ среда
      if (isIterable) {
        return {
          method: 'Spread operator (...arrayLike)',
          reason: 'Самый лаконичный и производительный способ для итерируемых объектов'
        };
      }
    } else {
      return {
        method: 'Array.from(arrayLike)',
        reason: 'Работает с любыми псевдомассивами, включая неитерируемые'
      };
    }
  }
}
```

```

    }
  } else {
    // ES5 среда
    return {
      method: 'Array.prototype.slice.call(arrayLike)',
      reason: 'Наиболее совместимый способ для старых браузеров'
    };
  }
}
}
}

```

// Примеры

```

console.log(ConversionGuide.recommendMethod(document.querySelectorAll('div')));
console.log(ConversionGuide.recommendMethod(arguments));
console.log(ConversionGuide.recommendMethod({0: 'a', 1: 'b', length: 2}));

```

8.2. Универсальная функция преобразования

javascript

// Функция, которая работает везде

```

function toArray(arrayLike) {
  // Если уже массив - возвращаем как есть
  if (Array.isArray(arrayLike)) {
    return arrayLike.slice(); // Возвращаем копию
  }

```

// Если поддерживается Array.from

```

if (typeof Array.from === 'function') {
  return Array.from(arrayLike);
}

```

// Если объект итерируемый (и поддерживается spread)

```

if (arrayLike && typeof arrayLike[Symbol.iterator] === 'function') {
  try {
    return [...arrayLike];
  } catch (e) {
    // Если spread не поддерживается, fallback
  }
}

```

// Fallback для старых браузеров

```

if (arrayLike && typeof arrayLike.length === 'number') {
  return Array.prototype.slice.call(arrayLike);
}

```

```

}

// Если ничего не подошло
throw new TypeError('Объект не может быть преобразован в массив');
}

// Тестируем
console.log(toArray([1, 2, 3])); // [1, 2, 3]
console.log(toArray({0: 'a', 1: 'b', length: 2})); // ['a', 'b']
console.log(toArray('hello')); // ['h', 'e', 'l', 'l', 'o']

```

8.3. Производительные паттерны

```

javascript

// 1. Кэширование преобразованных массивов
const arrayLikeCache = new WeakMap();

function getCachedArray(arrayLike) {
  if (!arrayLikeCache.has(arrayLike)) {
    const array = Array.from(arrayLike);
    arrayLikeCache.set(arrayLike, array);
  }
  return arrayLikeCache.get(arrayLike);
}

// 2. Ленивое преобразование
class LazyArrayWrapper {
  constructor(arrayLike) {
    this.arrayLike = arrayLike;
    this._array = null;
  }

  get array() {
    if (this._array === null) {
      this._array = Array.from(this.arrayLike);
    }
    return this._array;
  }

  // Делегирование методов
  forEach(callback) {
    return this.array.forEach(callback);
  }
}

```

```

    map(callback) {
        return this.array.map(callback);
    }

    // ... другие методы
}

// 3. Потокое преобразование для очень больших коллекций
async function* streamArrayLike(arrayLike, batchSize = 1000) {
    const length = arrayLike.length;

    for (let start = 0; start < length; start += batchSize) {
        const end = Math.min(start + batchSize, length);
        const batch = [];

        for (let i = start; i < end; i++) {
            batch.push(arrayLike[i]);
        }

        yield batch;

        // Даем возможность обработать другие задачи
        await new Promise(resolve => setTimeout(resolve, 0));
    }
}

// Использование
async function processLargeCollection(arrayLike) {
    for await (const batch of streamArrayLike(arrayLike)) {
        console.log(`Обработано ${batch.length} элементов`);
        // Обработка батча
    }
}

```

8.4. Безопасное преобразование

javascript

```

// Защищенная функция преобразования
function safeToArray(value, defaultValue = []) {
    try {
        // Если значение уже массив
        if (Array.isArray(value)) {

```

```

    return value;
}

// Если значение - псевдомассив
if (value && typeof value === 'object' && typeof value.length === 'number') {
    return Array.from(value);
}

// Если значение итерируемо (кроме строк)
if (value && typeof value[Symbol.iterator] === 'function' && typeof value !==
'string') {
    return [...value];
}

// Если ничего не подошло
return defaultValue;
} catch (error) {
    console.warn('Не удалось преобразовать в массив:', error);
    return defaultValue;
}
}

// Примеры безопасного использования
console.log(safeToArray([1, 2, 3])); // [1, 2, 3]
console.log(safeToArray(null)); // []
console.log(safeToArray(undefined, ['default'])); // ['default']
console.log(safeToArray(123)); // []
console.log(safeToArray('строка')); // [] (строка не преобразуется по умолчанию)
console.log(safeToArray({0: 'a', length: 1})); // ['a']
console.log(safeToArray(new Set([1, 2, 3]))); // [1, 2, 3]

```

Заключение

Ключевые выводы:

1. **Псевдомассивы распространены** — они встречаются в DOM API, в `arguments`, в строках, и многих других местах.
2. **Основное отличие** — отсутствие методов массива при наличии `length` и числовых индексов.
3. **Способы преобразования эволюционировали:**

➔ **Исторические:** `Array.prototype.slice.call()`

- **ES6:** `Array.from()` и spread оператор `...`
- **Современные:** преимущественно `Array.from()`

4. **Важно выбирать правильный метод:**

- `Array.from()` — самый универсальный
- Spread оператор — для итерируемых объектов
- `slice.call()` — для обратной совместимости

5. **Особые случаи требуют особого подхода:**

- Строки с эмодзи
- Очень большие коллекции
- Разреженные псевдомассивы
- Ленивые вычисления

Рекомендации для разработчиков:

1. **Всегда проверяйте, с чем работаете** — используйте `Array.isArray()` перед вызовом методов массива.
2. **Преобразуйте на раннем этапе** — если вам нужны методы массива, преобразуйте псевдомассив в массив как можно раньше.
3. **Учитывайте производительность** — для больших данных используйте батчи или ленивые вычисления.
4. **Обеспечивайте обратную совместимость** — если пишете библиотеку, поддерживайте старые браузеры.
5. **Документируйте ожидаемый тип** — если ваша функция ожидает массив, укажите это в документации.

Понимание псевдомассивов и способов их преобразования — важный навык JavaScript-разработчика. Это знание помогает писать более надежный, производительный и совместимый код, а также глубже понимать внутреннее устройство JavaScript.

Введение: зачем нужны новые коллекции в ES6

До ES6 JavaScript имел всего две встроенные структуры данных для коллекций: **Array** (для упорядоченных значений) и **Object** (для пар ключ-значение). Однако у них были ограничения:

1. Объекты:

- ➔ Ключи только строки или символы
- ➔ Нет гарантированного порядка
- ➔ Нет встроенных методов для работы с коллекциями
- ➔ Прототипное наследование может вызывать конфликты

2. Массивы:

- ➔ Позволяют дубликаты
- ➔ Медленный поиск элемента ($O(n)$)
- ➔ Не предназначены для уникальных значений

ES6 представил четыре новые специализированные коллекции:

- **Set** — коллекция уникальных значений
- **Map** — коллекция пар ключ-значение с любыми ключами
- **WeakSet** — Set со слабыми ссылками
- **WeakMap** — Map со слабыми ссылками в ключах

Часть 1: Set — коллекция уникальных значений

1.1. Что такое Set

Set — это коллекция **уникальных** значений любого типа. Каждое значение может встречаться в Set только один раз.

```
javascript
```

```
// Создание Set  
const set = new Set();
```



```
// Или с начальными значениями
const setWithValues = new Set([1, 2, 3, 4, 5]);
console.log(setWithValues); // Set(5) {1, 2, 3, 4, 5}

// Автоматическое удаление дубликатов
const withDuplicates = new Set([1, 2, 2, 3, 3, 3, 4]);
console.log(withDuplicates); // Set(4) {1, 2, 3, 4}
```

1.2. Основные методы и свойства

1.2.1. Добавление и удаление элементов

```
javascript

const set = new Set();

// add() - добавляет элемент
set.add(1);
set.add(2);
set.add(3);
set.add(2); // Дубликат игнорируется
console.log(set); // Set(3) {1, 2, 3}

// add() возвращает Set, можно чейнить
set.add(4).add(5).add(6);
console.log(set); // Set(6) {1, 2, 3, 4, 5, 6}

// delete() - удаляет элемент
set.delete(3);
console.log(set); // Set(5) {1, 2, 4, 5, 6}

// clear() - очищает весь Set
set.clear();
console.log(set); // Set(0) {}
```

1.2.2. Проверка наличия и размер

```
javascript

const set = new Set([1, 2, 3, 4, 5]);

// has() - проверяет наличие элемента
console.log(set.has(3)); // true
console.log(set.has(10)); // false
```

```
// size - количество элементов
console.log(set.size); // 5

// Особенность: NaN считается равным NaN в Set
const nanSet = new Set();
nanSet.add(NaN);
console.log(nanSet.has(NaN)); // true
```

1.2.3. Итерация по Set

```
javascript

const set = new Set(['яблоко', 'банан', 'апельсин']);

// forEach()
set.forEach((value, valueAgain, set) => {
  console.log(value); // яблоко, банан, апельсин
});

// for...of
for (const fruit of set) {
  console.log(fruit);
}

// Set имеет встроенный итератор
const iterator = set.values(); // или set.keys()
console.log(iterator.next().value); // яблоко
console.log(iterator.next().value); // банан

// Преобразование в массив
const arrayFromSet = Array.from(set);
console.log(arrayFromSet); // ['яблоко', 'банан', 'апельсин']

// Или с помощью spread оператора
const spreadArray = [...set];
console.log(spreadArray); // ['яблоко', 'банан', 'апельсин']
```

1.3. Полезные операции с Set

1.3.1. Удаление дубликатов из массива

```
javascript
```

```

function removeDuplicates(array) {
    return [...new Set(array)];
}

const numbers = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4];
const uniqueNumbers = removeDuplicates(numbers);
console.log(uniqueNumbers); // [1, 2, 3, 4]

// Строки тоже работают
const words = ['hello', 'world', 'hello', 'javascript', 'world'];
const uniqueWords = removeDuplicates(words);
console.log(uniqueWords); // ['hello', 'world', 'javascript']

```

1.3.2. Теоретико-множественные операции

```

javascript

// Объединение множеств
function union(setA, setB) {
    return new Set([...setA, ...setB]);
}

// Пересечение множеств
function intersection(setA, setB) {
    return new Set([...setA].filter(x => setB.has(x)));
}

// Разность множеств (A - B)
function difference(setA, setB) {
    return new Set([...setA].filter(x => !setB.has(x)));
}

// Симметрическая разность (элементы, которые есть только в одном из множеств)
function symmetricDifference(setA, setB) {
    return new Set([...difference(setA, setB), ...difference(setB, setA)]);
}

// Проверка подмножества
function isSubset(subset, superset) {
    return [...subset].every(x => superset.has(x));
}

// Проверка надмножества
function isSuperset(superset, subset) {
    return isSubset(subset, superset);
}

```

```
}
```

```
// Пример использования
```

```
const setA = new Set([1, 2, 3, 4]);
```

```
const setB = new Set([3, 4, 5, 6]);
```

```
console.log('Объединение:', union(setA, setB)); // {1, 2, 3, 4, 5, 6}
```

```
console.log('Пересечение:', intersection(setA, setB)); // {3, 4}
```

```
console.log('Разность A - B:', difference(setA, setB)); // {1, 2}
```

```
console.log('Симметрическая разность:', symmetricDifference(setA, setB)); // {1, 2, 5, 6}
```

```
console.log('A подмножество B?:', isSubset(setA, setB)); // false
```

```
console.log('{3,4} подмножество A?:', isSubset(new Set([3, 4]), setA)); // true
```

1.3.3. Хранение объектов в Set

```
javascript
```

```
// Set может хранить объекты
```

```
const objSet = new Set();
```

```
const obj1 = { id: 1, name: 'Object 1' };
```

```
const obj2 = { id: 2, name: 'Object 2' };
```

```
const obj3 = { id: 1, name: 'Object 1' }; // Другой объект с такими же свойствами
```

```
objSet.add(obj1);
```

```
objSet.add(obj2);
```

```
objSet.add(obj3); // Добавится, потому что obj3 !== obj1
```

```
console.log(objSet.size); // 3
```

```
// Проверка по ссылке
```

```
console.log(objSet.has(obj1)); // true
```

```
console.log(objSet.has({ id: 1, name: 'Object 1' })); // false (другой объект)
```

1.4. Практические применения Set

1.4.1. Отслеживание уникальных посетителей

```
javascript
```

```
class UniqueVisitorsTracker {  
  constructor() {  
    this.visitors = new Set();  
  }  
}
```

```

addVisitor(visitorId) {
    this.visitors.add(visitorId);
    console.log(`Посетитель ${visitorId} добавлен. Всего: ${this.visitors.size}`);
}

hasVisited(visitorId) {
    return this.visitors.has(visitorId);
}

removeVisitor(visitorId) {
    this.visitors.delete(visitorId);
}

getStats() {
    return {
        total: this.visitors.size,
        visitors: [...this.visitors]
    };
}
}

// Использование
const tracker = new UniqueVisitorsTracker();
tracker.addVisitor('user123');
tracker.addVisitor('user456');
tracker.addVisitor('user123'); // Игнорируется
console.log(tracker.getStats()); // {total: 2, visitors: ['user123', 'user456']}

```

1.4.2. Проверка уникальности тегов

```

javascript

class Article {
    constructor(title, content) {
        this.title = title;
        this.content = content;
        this.tags = new Set();
    }

    addTag(tag) {
        if (this.tags.has(tag)) {
            console.log(`Тег "${tag}" уже существует`);
            return false;
        }
    }
}

```

```

    this.tags.add(tag.toLowerCase());
    return true;
}

addTags(...tags) {
    let addedCount = 0;
    tags.forEach(tag => {
        if (this.addTag(tag)) {
            addedCount++;
        }
    });
    return addedCount;
}

removeTag(tag) {
    return this.tags.delete(tag.toLowerCase());
}

hasTag(tag) {
    return this.tags.has(tag.toLowerCase());
}

getTagsAsArray() {
    return [...this.tags];
}

getTagsAsString() {
    return this.getTagsAsArray().join(', ');
}
}

// Пример
const article = new Article('JavaScript Guide', 'Content...');
article.addTags('JavaScript', 'Programming', 'Web', 'javascript'); // javascript
игнорируется
console.log(article.getTagsAsString()); // "javascript, programming, web"
console.log(article.hasTag('WEB')); // true (регистр не важен)

```

Часть 2: Map — коллекция пар ключ-значение

2.1. Что такое Map

Map — это коллекция пар ключ-значение, где ключи могут быть любого типа (в отличие от Object, где ключи только строки или символы).

```
javascript

// Создание Map
const map = new Map();

// Или с начальными значениями
const mapWithValues = new Map([
  ['key1', 'value1'],
  ['key2', 'value2'],
  [1, 'числовой ключ'],
  [true, 'булев ключ']
]);

console.log(mapWithValues);
// Map(4) {'key1' => 'value1', 'key2' => 'value2', 1 => 'числовой ключ', true => 'булев
ключ'}
```

2.2. Основные методы и свойства

2.2.1. Добавление, получение и удаление

```
javascript

const map = new Map();

// set() - устанавливает значение по ключу
map.set('name', 'John');
map.set('age', 30);
map.set(42, 'Ответ на все вопросы');
map.set({ id: 1 }, 'Объект как ключ');

// set() возвращает Map, можно цепить
map.set('a', 1).set('b', 2).set('c', 3);

// get() - получает значение по ключу
```

```
console.log(map.get('name')); // 'John'
console.log(map.get('age')); // 30
console.log(map.get(42)); // 'Ответ на все вопросы'
console.log(map.get({ id: 1 })); // undefined (другой объект)
```

// delete() - удаляет элемент по ключу

```
map.delete('age');
console.log(map.has('age')); // false
```

// clear() - очищает всю Map

```
map.clear();
console.log(map.size); // 0
```

2.2.2. Проверка наличия и размер

javascript

```
const map = new Map([
  ['javascript', 1995],
  ['python', 1991],
  ['java', 1995]
]);
```

// has() - проверяет наличие ключа

```
console.log(map.has('javascript')); // true
console.log(map.has('ruby')); // false
```

// size - количество элементов

```
console.log(map.size); // 3
```

2.2.3. Итерация по Map

javascript

```
const map = new Map([
  ['apple', 'красный'],
  ['banana', 'желтый'],
  ['grape', 'фиолетовый']
]);
```

// forEach()

```
map.forEach((value, key, map) => {
  console.log(`${key}: ${value}`);
});
```

// for...of


```

for (const [key, value] of map) {
  console.log(`${key} -> ${value}`);
}

// Итераторы
const keysIterator = map.keys();
const valuesIterator = map.values();
const entriesIterator = map.entries();

console.log([...keysIterator]); // ['apple', 'banana', 'grape']
console.log([...valuesIterator]); // ['красный', 'желтый', 'фиолетовый']
console.log([...entriesIterator]); // [['apple', 'красный'], ...]

// Преобразование в объект
const obj = Object.fromEntries(map);
console.log(obj); // {apple: 'красный', banana: 'желтый', grape: 'фиолетовый'}

// Преобразование из объекта
const obj2 = { a: 1, b: 2, c: 3 };
const map2 = new Map(Object.entries(obj2));
console.log(map2); // Map(3) {'a' => 1, 'b' => 2, 'c' => 3}

```

2.3. Особенности Map

2.3.1. Ключи любого типа

```

javascript

const map = new Map();

// Функции как ключи
function myFunction() {}
map.set(myFunction, 'Функция как ключ');

// DOM элементы как ключи
const div = document.createElement('div');
map.set(div, 'DOM элемент');

// NaN как ключ
map.set(NaN, 'Не число');
console.log(map.get(NaN)); // 'Не число'

// null и undefined как ключи

```

```
map.set(null, 'null значение');
map.set(undefined, 'undefined значение');
```

2.3.2. Сохранение порядка вставки

```
javascript

const map = new Map();

map.set('z', 1);
map.set('a', 2);
map.set('m', 3);

// Порядок сохраняется
console.log([...map.keys()]); // ['z', 'a', 'm']
console.log([...map.values()]); // [1, 2, 3]

// При итерации получаем в порядке вставки
for (const [key, value] of map) {
  console.log(key, value); // z 1, a 2, m 3
}
```

2.3.3. Сравнение с Object

```
javascript

// Object
const obj = {
  1: 'one', // Ключ преобразуется в строку '1'
  '1': 'one again' // Перезаписывает предыдущее
};
console.log(obj); // {'1': 'one again'}

// Map
const map = new Map();
map.set(1, 'one');
map.set('1', 'one as string');
console.log(map.get(1)); // 'one'
console.log(map.get('1')); // 'one as string'
console.log(map.size); // 2

// Порядок итерации
const obj2 = { b: 2, a: 1, c: 3 };
console.log(Object.keys(obj2)); // ['b', 'a', 'c'] - порядок не гарантирован

const map2 = new Map([['b', 2], ['a', 1], ['c', 3]]);
```

```
console.log([...map2.keys()]); // ['b', 'a', 'c'] - порядок сохранен
```

2.4. Полезные операции с Map

2.4.1. Группировка данных

```
javascript

// Группировка пользователей по возрасту
const users = [
  { name: 'Alice', age: 25 },
  { name: 'Bob', age: 30 },
  { name: 'Charlie', age: 25 },
  { name: 'David', age: 30 },
  { name: 'Eve', age: 25 }
];

function groupByAge(users) {
  const grouped = new Map();

  users.forEach(user => {
    const age = user.age;
    if (!grouped.has(age)) {
      grouped.set(age, []);
    }
    grouped.get(age).push(user);
  });

  return grouped;
}

const usersByAge = groupByAge(users);
console.log(usersByAge.get(25)); // [{name: 'Alice', age: 25}, ...]
```

2.4.2. Кэширование (мемоизация)

```
javascript

function memoize(fn) {
  const cache = new Map();

  return function(...args) {
    // Создаем ключ на основе аргументов
    const key = JSON.stringify(args);
```

```

    if (cache.has(key)) {
        console.log('Возвращаем из кэша');
        return cache.get(key);
    }

    console.log('Вычисляем');
    const result = fn(...args);
    cache.set(key, result);

    return result;
};
}

// Пример использования
const expensiveCalculation = memoize((a, b) => {
    console.log('Выполняю сложные вычисления...');
    return a + b;
});

console.log(expensiveCalculation(2, 3)); // Вычисляем -> 5
console.log(expensiveCalculation(2, 3)); // Возвращаем из кэша -> 5
console.log(expensiveCalculation(4, 5)); // Вычисляем -> 9

```

2.4.3. Счетчик частоты элементов

```

javascript

function countFrequency(array) {
    const frequency = new Map();

    array.forEach(item => {
        frequency.set(item, (frequency.get(item) || 0) + 1);
    });

    return frequency;
}

const words = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple'];
const wordFrequency = countFrequency(words);

console.log(wordFrequency.get('apple')); // 3
console.log(wordFrequency.get('banana')); // 2

// Преобразование в отсортированный массив

```

```
const sortedByFrequency = [...wordFrequency.entries()]
  .sort((a, b) => b[1] - a[1])
  .map(([word, count]) => ({ word, count }));

console.log(sortedByFrequency);
// [{word: 'apple', count: 3}, {word: 'banana', count: 2}, {word: 'orange', count: 1}]
```

2.5. Практические применения Map

2.5.1. Конфигурация приложения

javascript

```
class AppConfig {
  constructor() {
    this.config = new Map();
  }

  set(key, value) {
    this.config.set(key, value);
    return this; // Для чейнинга
  }

  get(key, defaultValue = null) {
    return this.config.has(key) ? this.config.get(key) : defaultValue;
  }

  has(key) {
    return this.config.has(key);
  }

  delete(key) {
    return this.config.delete(key);
  }

  // Безопасное получение вложенных свойств
  getNested(path, defaultValue = null) {
    const keys = path.split('.');
    let current = this.config;

    for (const key of keys) {
      if (current instanceof Map && current.has(key)) {
        current = current.get(key);
      }
    }
  }
}
```

```

    } else if (current && typeof current === 'object' && key in current) {
        current = current[key];
    } else {
        return defaultValue;
    }
}

return current;
}

// Экспорт в объект
toObject() {
    const obj = {};
    for (const [key, value] of this.config) {
        if (value instanceof Map) {
            obj[key] = Object.fromEntries(value);
        } else {
            obj[key] = value;
        }
    }
    return obj;
}
}

```

// Пример использования

```

const config = new AppConfig()
    .set('app.name', 'My Application')
    .set('app.version', '1.0.0')
    .set('database.host', 'localhost')
    .set('database.port', 5432)
    .set('features', new Map([
        ['darkMode', true],
        ['notifications', false],
        ['analytics', true]
    ]));

```

```

console.log(config.get('app.name')); // 'My Application'
console.log(config.get('database.port')); // 5432
console.log(config.getNested('features.darkMode')); // true
console.log(config.toObject());

```

2.5.2. Система событий (Event Emitter)

javascript

```

class EventEmitter {
  constructor() {
    this.events = new Map();
  }

  on(eventName, listener) {
    if (!this.events.has(eventName)) {
      this.events.set(eventName, new Set());
    }
    this.events.get(eventName).add(listener);
    return this;
  }

  off(eventName, listener) {
    if (this.events.has(eventName)) {
      const listeners = this.events.get(eventName);
      listeners.delete(listener);

      if (listeners.size === 0) {
        this.events.delete(eventName);
      }
    }
    return this;
  }

  emit(eventName, ...args) {
    if (this.events.has(eventName)) {
      const listeners = this.events.get(eventName);
      listeners.forEach(listener => {
        try {
          listener(...args);
        } catch (error) {
          console.error(`Ошибка в обработке события "${eventName}":`, error);
        }
      });
    }
    return this;
  }

  once(eventName, listener) {
    const onceListener = (...args) => {
      listener(...args);
      this.off(eventName, onceListener);
    };
  }

```

```

    return this.on(eventName, oncelistener);
  }

  listenerCount(eventName) {
    return this.events.has(eventName) ? this.events.get(eventName).size : 0;
  }

  eventNames() {
    return [...this.events.keys()];
  }
}

// Пример использования
const emitter = new EventEmitter();

const logMessage = (msg) => console.log(`Сообщение: ${msg}`);

emitter.on('message', logMessage);
emitter.on('error', (err) => console.error(`Ошибка: ${err}`));

emitter.emit('message', 'Привет мир!');
emitter.emit('error', 'Что-то пошло не так');

emitter.off('message', logMessage);
console.log(emitter.listenerCount('message')); // 0

```

Часть 3: WeakSet — Set со слабыми ссылками

3.1. Что такое WeakSet

WeakSet — это особый вид Set, который:

1. Может содержать только **объекты** (не примитивы)
2. Хранит **слабые ссылки** на объекты
3. **Не итерируем** — нельзя перебирать элементы
4. **Нет свойства size**

javascript

// Создание WeakSet

```
const weakSet = new WeakSet();
```

// Добавление объектов


```

let obj1 = { id: 1 };
let obj2 = { id: 2 };
let obj3 = { id: 3 };

weakSet.add(obj1);
weakSet.add(obj2);
weakSet.add(obj3);

console.log(weakSet.has(obj1)); // true

// Удаление объекта
weakSet.delete(obj1);
console.log(weakSet.has(obj1)); // false

// Нельзя добавить примитивы
try {
  weakSet.add('строка'); // TypeError
} catch (e) {
  console.error(e.message); // Invalid value used in weak set
}

// Нельзя итерировать
// for (const item of weakSet) {} // TypeError: weakSet is not iterable

// Нет свойства size
console.log(weakSet.size); // undefined

```

3.2. Слабые ссылки — ключевая концепция

```

javascript

// Демонстрация слабых ссылок
let tempObj = { data: 'временный объект' };
const weakSet = new WeakSet();

// Добавляем объект в WeakSet
weakSet.add(tempObj);
console.log('До удаления:', weakSet.has(tempObj)); // true

// Удаляем ссылку на объект
tempObj = null;

// Сборщик мусора может удалить объект из WeakSet
// Но мы не можем проверить это напрямую, т.к. нет доступа

```

```
// Для демонстрации - принудительный вызов сборщика мусора (не в продакшене)
if (global.gc) {
  global.gc(); // Только в Node.js с флагом --expose-gc
  // После этого weakSet может быть пустым
}
```

3.3. Практическое применение WeakSet

3.3.1. Отслеживание посещенных объектов

```
javascript

class ObjectTracker {
  constructor() {
    this.visited = new WeakSet();
  }

  visit(obj) {
    if (this.visited.has(obj)) {
      console.log('Объект уже был посещен');
      return false;
    }

    this.visited.add(obj);
    console.log('Объект добавлен в посещенные');
    return true;
  }

  hasVisited(obj) {
    return this.visited.has(obj);
  }
}

// Пример использования для обхода графа
class GraphNode {
  constructor(value) {
    this.value = value;
    this.neighbors = [];
  }

  addNeighbor(node) {
    this.neighbors.push(node);
  }
}
```

```

    }
}

function traverseGraph(startNode, callback) {
    const tracker = new ObjectTracker();
    const queue = [startNode];

    while (queue.length > 0) {
        const node = queue.shift();

        // Если уже посещали, пропускаем
        if (tracker.hasVisited(node)) {
            continue;
        }

        // Отмечаем как посещенный
        tracker.visit(node);
        callback(node);

        // Добавляем соседей в очередь
        queue.push(...node.neighbors);
    }
}

// Создаем граф
const nodeA = new GraphNode('A');
const nodeB = new GraphNode('B');
const nodeC = new GraphNode('C');
const nodeD = new GraphNode('D');

nodeA.addNeighbor(nodeB);
nodeA.addNeighbor(nodeC);
nodeB.addNeighbor(nodeD);
nodeC.addNeighbor(nodeD);
nodeD.addNeighbor(nodeA); // Цикл

// Обходим граф
traverseGraph(nodeA, (node) => {
    console.log(`Посетили узел: ${node.value}`);
});

```

3.3.2. Кэширование DOM элементов

javascript

```
class DOMCache {
  constructor() {
    this.cache = new WeakSet();
  }

  // Помечаем элемент как обработанный
  markAsProcessed(element) {
    this.cache.add(element);
  }

  // Проверяем, был ли элемент обработан
  isProcessed(element) {
    return this.cache.has(element);
  }

  // Автоматическая очистка при удалении элементов из DOM
  processElements(elements) {
    elements.forEach(element => {
      if (!this.isProcessed(element)) {
        console.log(`Обрабатываем элемент:`, element);
        // Какая-то обработка...
        this.markAsProcessed(element);
      } else {
        console.log(`Элемент уже обработан, пропускаем`);
      }
    });
  }
}

// Пример использования
const cache = new DOMCache();

// Эмуляция работы с DOM
const div1 = document.createElement('div');
const div2 = document.createElement('div');
const div3 = document.createElement('div');

cache.processElements([div1, div2, div3]); // Обработает все три
cache.processElements([div1, div2, div3]); // Все уже обработаны

// Когда элементы удаляются из DOM, они автоматически удаляются из WeakSet
```

Часть 4: WeakMap — Map со слабыми ссылками в ключах

4.1. Что такое WeakMap

WeakMap — это особый вид Map, который:

1. Ключами могут быть только **объекты** (не примитивы)
2. Хранит **слабые ссылки** на ключи-объекты
3. **Не итерируем** — нельзя перебирать ключи или значения
4. **Нет свойства size**
5. Значения могут быть любого типа

javascript

// Создание WeakMap

```
const weakMap = new WeakMap();
```

// Добавление пар ключ-значение

```
let key1 = { id: 1 };
```

```
let key2 = { id: 2 };
```

```
weakMap.set(key1, 'значение для key1');
```

```
weakMap.set(key2, { data: 'объект-значение' });
```

```
console.log(weakMap.get(key1)); // 'значение для key1'
```

```
console.log(weakMap.get(key2)); // {data: 'объект-значение'}
```

// Проверка наличия ключа

```
console.log(weakMap.has(key1)); // true
```

// Удаление

```
weakMap.delete(key1);
```

```
console.log(weakMap.has(key1)); // false
```

// Нельзя использовать примитивы как ключи

```
try {
```

```
  weakMap.set('string', 'value'); // TypeError
```

```
} catch (e) {
```

```
  console.error(e.message); // Invalid value used as weak map key
```

```
}
```

// Нельзя итерировать

```
// for (const [key, value] of weakMap) {} // TypeError
```

```
// Нет свойства size
```

```
console.log(weakMap.size); // undefined
```

4.2. Особенности WeakMap

4.2.1. Слабые ссылки на ключи

javascript

```
// Демонстрация слабых ссылок
```

```
let tempKey = { id: 'temp' };
```

```
const weakMap = new WeakMap();
```

```
weakMap.set(tempKey, 'важные данные');
```

```
console.log('До удаления:', weakMap.get(tempKey)); // 'важные данные'
```

```
// Удаляем ссылку на ключ
```

```
tempKey = null;
```

```
// Объект-ключ может быть собран сборщиком мусора
```

```
// и соответствующая запись удалится из WeakMap
```

4.2.2. Значения не удаляются автоматически

javascript

```
// Важно: сборщик мусора удаляет только ключи, не значения
```

```
let key1 = { id: 1 };
```

```
let value1 = { largeData: new Array(1000000).fill('data') };
```

```
const weakMap = new WeakMap();
```

```
weakMap.set(key1, value1);
```

```
// Удаляем ссылку на ключ
```

```
key1 = null;
```

```
// Ключ будет удален из WeakMap, но объект value1
```

```
// останется в памяти, пока на него есть ссылки
```

```
console.log(value1); // Объект все еще доступен
```

4.3. Практические применения WeakMap

4.3.1. Приватные данные для объектов

javascript

// Классический паттерн: приватные свойства через WeakMap

```
const privateData = new WeakMap();
```

```
class User {
  constructor(name, email) {
    // Публичные свойства
    this.name = name;

    // Приватные данные храним в WeakMap
    privateData.set(this, {
      email: email,
      password: this.generatePassword(),
      loginAttempts: 0
    });
  }

  generatePassword() {
    return Math.random().toString(36).slice(-8);
  }

  // Публичные методы для доступа к приватным данным
  getEmail() {
    return privateData.get(this).email;
  }

  setEmail(newEmail) {
    const data = privateData.get(this);
    data.email = newEmail;
    privateData.set(this, data);
  }

  validatePassword(input) {
    const data = privateData.get(this);
    data.loginAttempts++;

    if (input === data.password) {
      data.loginAttempts = 0;
      return true;
    }
  }
}
```

```

    }

    if (data.loginAttempts >= 3) {
        console.log('Слишком много попыток!');
    }

    return false;
}

// Статический метод для получения данных (только для демонстрации)
static getPrivateData(user) {
    return privateData.get(user);
}
}

// Использование
const user1 = new User('Анна', 'anna@example.com');
console.log(user1.name); // 'Анна' - публичное свойство
console.log(user1.getEmail()); // 'anna@example.com' - через метод
console.log(user1.validatePassword('wrong')); // false
console.log(User.getPrivateData(user1).loginAttempts); // 1

// При удалении объекта user1, его приватные данные автоматически удаляются из WeakMap

```

4.3.2. Кэширование вычислений для объектов

```

javascript

// Кэширование результатов тяжелых вычислений для объектов
const computationCache = new WeakMap();

function heavyComputation(obj) {
    // Проверяем кэш
    if (computationCache.has(obj)) {
        console.log('Возвращаем из кэша');
        return computationCache.get(obj);
    }

    console.log('Выполняем вычисления...');
    // Имитация тяжелых вычислений
    const result = {
        hash: JSON.stringify(obj).length,
        timestamp: Date.now(),
        processed: true
    };
}

```



```

// Сохраняем в кэш
computationCache.set(obj, result);
return result;
}

// Пример использования
const data1 = { a: 1, b: 2, c: 3 };
const data2 = { x: 10, y: 20, z: 30 };

console.log(heavyComputation(data1)); // Выполняем вычисления...
console.log(heavyComputation(data1)); // Возвращаем из кэша
console.log(heavyComputation(data2)); // Выполняем вычисления...

// Когда data1 больше не нужен и сборщик мусора его удалит,
// соответствующая запись в кэше тоже удалится

```

4.3.3. Связывание метаданных с DOM элементами

```

javascript

// Хранение метаданных для DOM элементов
const elementMetadata = new WeakMap();

class ElementManager {
  static setMetadata(element, metadata) {
    elementMetadata.set(element, {
      ...metadata,
      created: Date.now(),
      lastAccessed: Date.now()
    });
  }

  static getMetadata(element) {
    const metadata = elementMetadata.get(element);
    if (metadata) {
      metadata.lastAccessed = Date.now();
    }
    return metadata;
  }

  static updateMetadata(element, updates) {
    const current = elementMetadata.get(element) || {};
    elementMetadata.set(element, {
      ...current,

```

```

        ...updates,
        lastAccessed: Date.now()
    });
}

static deleteMetadata(element) {
    elementMetadata.delete(element);
}

static getAllMetadata() {
    // Нельзя получить все элементы, WeakMap не итерируема
    // Это ограничение, а не недостаток
    console.log('WeakMap не позволяет получить все элементы');
    return null;
}
}

// Пример использования в браузере
const button = document.createElement('button');
button.textContent = 'Нажми меня';

// Добавляем метаданные
ElementManager.setMetadata(button, {
    type: 'button',
    style: 'primary',
    clickCount: 0
});

// Обновляем метаданные при клике
button.addEventListener('click', () => {
    const metadata = ElementManager.getMetadata(button);
    ElementManager.updateMetadata(button, {
        clickCount: (metadata.clickCount || 0) + 1
    });
    console.log('Кликов:', ElementManager.getMetadata(button).clickCount);
});

// Когда элемент удаляется из DOM, его метаданные автоматически удаляются

```

Часть 5: Сравнение коллекций

5.1. Таблица сравнения

javascript

// Создаем таблицу сравнения

```
const comparisonTable = [
  {
    характеристика: 'Типы ключей',
    Object: 'Только строки или символы',
    Map: 'Любые значения',
    Set: 'Нет ключей (только значения)',
    WeakMap: 'Только объекты',
    WeakSet: 'Только объекты'
  },
  {
    характеристика: 'Итерация',
    Object: 'Да (но порядок не гарантирован)',
    Map: 'Да (порядок вставки)',
    Set: 'Да (порядок вставки)',
    WeakMap: 'Нет',
    WeakSet: 'Нет'
  },
  {
    характеристика: 'Size/длина',
    Object: 'Нет встроенного, нужно считать',
    Map: 'size',
    Set: 'size',
    WeakMap: 'Нет',
    WeakSet: 'Нет'
  },
  {
    характеристика: 'Производительность поиска',
    Object: 'O(1) в среднем',
    Map: 'O(1) в среднем',
    Set: 'O(1) в среднем',
    WeakMap: 'O(1) в среднем',
    WeakSet: 'O(1) в среднем'
  },
  {
    характеристика: 'Слабые ссылки',
    Object: 'Нет',
  }
]
```

```

    Map: 'Нет',
    Set: 'Нет',
    WeakMap: 'Да (только ключи)',
    WeakSet: 'Да'
  },
  {
    характеристика: 'Автоочистка при удалении объектов',
    Object: 'Нет',
    Map: 'Нет',
    Set: 'Нет',
    WeakMap: 'Да',
    WeakSet: 'Да'
  },
  {
    характеристика: 'Лучшее применение',
    Object: 'Простые структуры данных',
    Map: 'Словари с любыми ключами',
    Set: 'Коллекции уникальных значений',
    WeakMap: 'Приватные данные, метаданные',
    WeakSet: 'Отслеживание объектов'
  }
];

console.table(comparisonTable);

```

5.2. Когда что использовать

5.2.1. Object vs Map

javascript

```

// Используйте Object когда:
// 1. Ключи - строки или символы
// 2. Нужен JSON сериализация
// 3. Работаете с простыми структурами
// 4. Нужны методы объекта (toString, valueOf и т.д.)

```

```

// Используйте Map когда:
// 1. Ключи любого типа
// 2. Важен порядок элементов
// 3. Часто добавляете/удаляете элементы
// 4. Нужно легко получить размер

```

5.2.2. Array vs Set

```
javascript

// Используйте Array когда:
// 1. Нужен порядок и доступ по индексу
// 2. Могут быть дубликаты
// 3. Нужны методы массива (map, filter, reduce)
// 4. Работаете со списками/последовательностями

// Используйте Set когда:
// 1. Важна уникальность значений
// 2. Нужна быстрая проверка наличия (O(1))
// 3. Часто добавляете/удаляете элементы
// 4. Работаете с математическими множествами
```

5.2.3. Map vs WeakMap

```
javascript

// Используйте Map когда:
// 1. Нужно итерировать по элементам
// 2. Нужно знать размер коллекции
// 3. Ключи могут быть примитивами
// 4. Не беспокоитесь об утечках памяти

// Используйте WeakMap когда:
// 1. Ключи - объекты с коротким временем жизни
// 2. Не нужно итерировать
// 3. Хотите избежать утечек памяти
// 4. Храните приватные данные или метаданные
```

5.3. Производительность

```
javascript

// Тест производительности
function performanceTest() {
  const iterations = 100000;

  console.time('Object set');
  const obj = {};
  for (let i = 0; i < iterations; i++) {
    obj[i] = i;
  }
  console.timeEnd('Object set');
```

```
console.time('Map set');
const map = new Map();
for (let i = 0; i < iterations; i++) {
  map.set(i, i);
}
console.timeEnd('Map set');

console.time('Object get');
for (let i = 0; i < iterations; i++) {
  const val = obj[i];
}
console.timeEnd('Object get');

console.time('Map get');
for (let i = 0; i < iterations; i++) {
  const val = map.get(i);
}
console.timeEnd('Map get');

console.time('Array includes (O(n))');
const arr = [];
for (let i = 0; i < iterations; i++) {
  arr.includes(i);
}
console.timeEnd('Array includes (O(n))');

console.time('Set has (O(1))');
const set = new Set();
for (let i = 0; i < iterations; i++) {
  set.has(i);
}
console.timeEnd('Set has (O(1))');
}

performanceTest();
```

Часть 6: Продвинутые техники

6.1. Сериализация и десериализация

javascript

```

// Сериализация Map и Set
class CollectionSerializer {
  // Сериализация Map в JSON
  static serializeMap(map) {
    return JSON.stringify([...map]);
  }

  // Десериализация Map из JSON
  static deserializeMap(jsonString) {
    try {
      const array = JSON.parse(jsonString);
      return new Map(array);
    } catch (error) {
      console.error('Ошибка десериализации Map:', error);
      return new Map();
    }
  }

  // Сериализация Set в JSON
  static serializeSet(set) {
    return JSON.stringify([...set]);
  }

  // Десериализация Set из JSON
  static deserializeSet(jsonString) {
    try {
      const array = JSON.parse(jsonString);
      return new Set(array);
    } catch (error) {
      console.error('Ошибка десериализации Set:', error);
      return new Set();
    }
  }

  // Глубокая сериализация объекта с Map и Set
  static deepSerialize(obj) {
    const replacer = (key, value) => {
      if (value instanceof Map) {
        return {
          __type: 'Map',
          __value: [...value]
        };
      }
      if (value instanceof Set) {

```

```

        return {
            __type: 'Set',
            __value: [...value]
        };
    }
    return value;
};

return JSON.stringify(obj, replacer);
}

// Глубокая десериализация
static deepDeserialize(jsonString) {
    const reviver = (key, value) => {
        if (value && value.__type === 'Map') {
            return new Map(value.__value);
        }
        if (value && value.__type === 'Set') {
            return new Set(value.__value);
        }
        return value;
    };

    return JSON.parse(jsonString, reviver);
}

// Пример использования
const data = {
    map: new Map([['key1', 'value1'], ['key2', 'value2']]),
    set: new Set([1, 2, 3, 4, 5]),
    nested: {
        anotherMap: new Map([['a', 1], ['b', 2]])
    }
};

const serialized = CollectionSerializer.deepSerialize(data);
console.log(serialized);

const deserialized = CollectionSerializer.deepDeserialize(serialized);
console.log(deserialized.map instanceof Map); // true
console.log(deserialized.set instanceof Set); // true

```


6.2. Наследование от встроенных коллекций

javascript

// Создание собственных коллекций на основе встроенных

```
class ObservableSet extends Set {
  constructor(iterable) {
    super(iterable);
    this.listeners = new Map();
  }

  add(value) {
    const hasValue = this.has(value);
    const result = super.add(value);

    if (!hasValue) {
      this.notify('add', value);
    }

    return result;
  }

  delete(value) {
    const hasValue = this.has(value);
    const result = super.delete(value);

    if (hasValue) {
      this.notify('delete', value);
    }

    return result;
  }

  clear() {
    const oldSize = this.size;
    super.clear();

    if (oldSize > 0) {
      this.notify('clear', null);
    }
  }
}

// Подписка на события
on(event, callback) {
```

```

    if (!this.listeners.has(event)) {
        this.listeners.set(event, new Set());
    }
    this.listeners.get(event).add(callback);
}

off(event, callback) {
    if (this.listeners.has(event)) {
        this.listeners.get(event).delete(callback);
    }
}

// Уведомление слушателей
notify(event, data) {
    if (this.listeners.has(event)) {
        this.listeners.get(event).forEach(callback => {
            try {
                callback(data, this);
            } catch (error) {
                console.error('Ошибка в обработке события:', error);
            }
        });
    }
}

// Пример использования
const observableSet = new ObservableSet([1, 2, 3]);

observableSet.on('add', (value) => {
    console.log(`Добавлен элемент: ${value}`);
});

observableSet.on('delete', (value) => {
    console.log(`Удален элемент: ${value}`);
});

observableSet.on('clear', () => {
    console.log('Set очищен');
});

observableSet.add(4); // "Добавлен элемент: 4"
observableSet.delete(2); // "Удален элемент: 2"
observableSet.clear(); // "Set очищен"

```

6.3. Lazy коллекции

javascript

// Ленивая (отложенная) коллекция

```
class LazyMap extends Map {
  constructor(iterable, transformFn) {
    super();
    this.transformFn = transformFn;
    this.originalKeys = new Set();

    if (iterable) {
      for (const [key, value] of iterable) {
        this.set(key, value);
      }
    }
  }

  set(key, value) {
    this.originalKeys.add(key);
    return super.set(key, value);
  }

  get(key) {
    if (this.originalKeys.has(key) && !super.has(key)) {
      // Вычисляем значение лениво
      const transformed = this.transformFn(key);
      super.set(key, transformed);
      return transformed;
    }
    return super.get(key);
  }

  has(key) {
    return this.originalKeys.has(key);
  }

  delete(key) {
    this.originalKeys.delete(key);
    return super.delete(key);
  }

  clear() {
    this.originalKeys.clear();
  }
}
```

```

    super.clear();
}

get size() {
    return this.originalKeys.size;
}

*keys() {
    for (const key of this.originalKeys) {
        yield key;
    }
}

*values() {
    for (const key of this.originalKeys) {
        yield this.get(key);
    }
}

*entries() {
    for (const key of this.originalKeys) {
        yield [key, this.get(key)];
    }
}

forEach(callback, thisArg) {
    for (const key of this.originalKeys) {
        callback.call(thisArg, this.get(key), key, this);
    }
}
}

// Пример использования
const lazyMap = new LazyMap(
    [['a', 1], ['b', 2], ['c', 3]],
    (key) => {
        console.log(`Вычисляю значение для ключа "${key}"`);
        return key.toUpperCase().charCodeAt(0);
    }
);

console.log(lazyMap.get('a')); // "Вычисляю значение..." затем 65
console.log(lazyMap.get('a')); // 65 (из кэша)
console.log(lazyMap.get('b')); // "Вычисляю значение..." затем 66

```

```
for (const [key, value] of lazyMap) {  
  console.log(key, value); // Для 'с' тоже вычислит значение  
}
```

Часть 7: Лучшие практики и паттерны

7.1. Инициализация коллекций

```
javascript  
  
// Правильные способы инициализации  
  
// Set  
const goodSet1 = new Set([1, 2, 3, 4, 5]);  
const goodSet2 = new Set(arrayOfValues);  
const goodSet3 = new Set(); // Пустой, потом добавляем  
  
// Map  
const goodMap1 = new Map([['key1', 'value1'], ['key2', 'value2']]);  
const goodMap2 = new Map(Object.entries(object));  
const goodMap3 = new Map(); // Пустой, потом добавляем  
  
// WeakSet и WeakMap  
const goodWeakSet = new WeakSet([obj1, obj2, obj3]);  
const goodWeakMap = new WeakMap([obj1, 'value1'], [obj2, 'value2']]);  
  
// НЕПРАВИЛЬНО  
const badSet = new Set(5); // Создаст пустой Set, не Set с одним элементом 5  
const badMap = new Map('key', 'value'); // Ожидает массив пар
```

7.2. Итерация с деструктуризацией

```
javascript  
  
// Эффективная итерация по Map  
const map = new Map([  
  ['name', 'Alice'],  
  ['age', 30],  
  ['city', 'New York']  
]);
```

```

// Хорошо
for (const [key, value] of map) {
  console.log(`${key}: ${value}`);
}

// Еще лучше для преобразования
const objectFromMap = Object.fromEntries(map);
const arrayFromMap = [...map];
const keysArray = [...map.keys()];
const valuesArray = [...map.values()];

// Для Set
const set = new Set([1, 2, 3, 4, 5]);
const arrayFromSet = [...set];
const doubledValues = [...set].map(x => x * 2);

```

7.3. Обработка ошибок и edge cases

javascript

```

// Безопасная работа с коллекциями
class SafeCollections {
  // Безопасное получение значения из Map
  static safeGet(map, key, defaultValue = null) {
    if (!(map instanceof Map)) {
      throw new TypeError('Первый аргумент должен быть Map');
    }

    return map.has(key) ? map.get(key) : defaultValue;
  }

  // Безопасное преобразование в Set
  static toSafeSet(iterable) {
    if (iterable == null) {
      return new Set();
    }

    if (iterable instanceof Set) {
      return new Set(iterable); // Копия
    }

    if (Array.isArray(iterable)) {
      return new Set(iterable);
    }
  }
}

```

```

    if (typeof iterable[Symbol.iterator] === 'function') {
        return new Set(iterable);
    }

    return new Set();
}

// Проверка на пустоту
static isEmpty(collection) {
    if (collection instanceof Map || collection instanceof Set) {
        return collection.size === 0;
    }

    if (collection instanceof WeakMap || collection instanceof WeakSet) {
        // Нельзя проверить, всегда возвращаем false для безопасности
        return false;
    }

    if (Array.isArray(collection)) {
        return collection.length === 0;
    }

    if (typeof collection === 'object' && collection !== null) {
        return Object.keys(collection).length === 0;
    }

    return true;
}

// Глубокая копия коллекции
static deepClone(collection) {
    if (collection instanceof Map) {
        const cloned = new Map();
        for (const [key, value] of collection) {
            cloned.set(
                this.deepClone(key),
                this.deepClone(value)
            );
        }
        return cloned;
    }

    if (collection instanceof Set) {

```

```

    const cloned = new Set();
    for (const value of collection) {
        cloned.add(this.deepClone(value));
    }
    return cloned;
}

// Для простых типов
return JSON.parse(JSON.stringify(collection));
}
}

```

7.4. Производительные паттерны

javascript

```

// Оптимизация производительности
class OptimizedCollections {
    // Батчинг операций
    static batchSet(map, entries) {
        for (const [key, value] of entries) {
            map.set(key, value);
        }
        return map;
    }

    static batchAdd(set, values) {
        for (const value of values) {
            set.add(value);
        }
        return set;
    }
}

// Предварительное выделение памяти (где возможно)
static preSizedMap(estimatedSize) {
    // Map не поддерживает предварительное выделение,
    // но можно оптимизировать добавление
    const map = new Map();

    // Добавляем фиктивные значения для "разогрева"
    for (let i = 0; i < Math.min(estimatedSize, 1000); i++) {
        map.set(`temp_${i}`, i);
    }
    map.clear();
}

```



```

    return map;
}

// Ленивое преобразование
static lazyTransform(set, transformFn) {
    const transformed = new Set();
    const iterator = set.values();

    return {
        next() {
            const next = iterator.next();
            if (next.done) {
                return { done: true };
            }
            return {
                value: transformFn(next.value),
                done: false
            };
        },
        [Symbol.iterator]() {
            return this;
        }
    };
}
}

```

Заключение

Ключевые выводы:

1. **Set** — идеален для работы с уникальными значениями, быстрой проверки наличия и математических операций над множествами.
2. **Map** — совершенная замена Object для словарей, когда нужны ключи любого типа, гарантированный порядок или частое добавление/удаление.
3. **WeakSet** — специализированный инструмент для отслеживания объектов без риска утечек памяти, но с ограничениями (не итерируем, нет size).
4. **WeakMap** — мощный механизм для ассоциации приватных данных с объектами, автоматически очищающийся при удалении объектов.

Рекомендации по выбору:

- Для простых структур данных → Object
- Для словарей с любыми ключами → Map
- Для уникальных коллекций → Set
- Для приватных данных объектов → WeakMap
- Для отслеживания объектов → WeakSet
- Для массивов с доступом по индексу → Array

Важные особенности:

1. **Порядок элементов** в Map и Set гарантирован — это порядок вставки.
2. **Производительность** операций has/get/set в среднем $O(1)$.
3. **Слабые коллекции** не предотвращают сборку мусора, а наоборот, позволяют ей работать.
4. **Итерация** — мощный инструмент, но для Weak-коллекций недоступен.

Практические советы:

1. **Всегда используйте соответствующие методы** — set/get для Map, add/has для Set.
2. **Избегайте смешивания типов** — если начали использовать Map, не переключайтесь на Object.
3. **Учитывайте сериализацию** — Map и Set требуют специальной обработки при преобразовании в JSON.
4. **Тестируйте производительность** — для больших datasets выберите оптимальную структуру.
5. **Документируйте выбор** — объясните, почему выбрали конкретную коллекцию.

Коллекции ES6+ — это не просто синтаксический сахар, а фундаментальное улучшение языка, предоставляющее разработчикам мощные, эффективные и безопасные инструменты для работы со структурами данных. Правильное их использование значительно улучшает читаемость, производительность и надежность кода.

Введение: от циклов к абстракциям итерации

Итерация (перебор) — фундаментальная операция в программировании. В JavaScript эволюция итерации прошла путь:

1. **Циклы for/while** (ES1) — ручное управление индексами
2. **Методы массивов** (ES5) — `forEach`, `map`, `filter` — декларативный подход
3. **Итераторы и for...of** (ES6) — универсальный протокол итерации
4. **Генераторы** (ES6) — функции, которые можно приостанавливать

Часть 1: Протокол итераторов

1.1. Что такое итератор?

Итератор — это объект, который умеет последовательно возвращать элементы коллекции. В JavaScript итератор должен реализовывать протокол итераторов:

```
javascript
// Протокол итератора: объект с методом next()
const iterator = {
  next() {
    // Должен возвращать объект { value: any, done: boolean }
    return {
      value: 'следующее значение',
      done: false // true когда итерация завершена
    };
  }
};
```

1.2. Простой пример итератора

```
javascript
// Создаем простой итератор для чисел от 1 до 3
```

```

const numberIterator = {
  current: 1,
  end: 3,

  next() {
    if (this.current <= this.end) {
      return {
        value: this.current++,
        done: false
      };
    } else {
      return {
        value: undefined,
        done: true
      };
    }
  }
};

// Использование итератора вручную
console.log(numberIterator.next()); // { value: 1, done: false }
console.log(numberIterator.next()); // { value: 2, done: false }
console.log(numberIterator.next()); // { value: 3, done: false }
console.log(numberIterator.next()); // { value: undefined, done: true }

// После завершения все вызовы возвращают done: true
console.log(numberIterator.next()); // { value: undefined, done: true }

```

1.3. Итерируемые объекты

Итерируемый объект — объект, который реализует метод `[Symbol.iterator]()`, возвращающий итератор.

```

javascript

// Создаем итерируемый объект
const iterableObject = {
  data: [10, 20, 30, 40, 50],

  // Метод Symbol.iterator делает объект итерируемым
  [Symbol.iterator]() {
    let index = 0;
    const data = this.data;

```

```

return {
  next() {
    if (index < data.length) {
      return {
        value: data[index++],
        done: false
      };
    } else {
      return {
        value: undefined,
        done: true
      };
    }
  },

  // Опционально: метод return для преждевременного завершения
  return() {
    console.log('Итерация прервана');
    return { done: true };
  },

  // Опционально: метод throw для обработки ошибок
  throw(error) {
    console.error('Ошибка при итерации:', error);
    return { done: true };
  }
};
}

// Теперь объект можно использовать с for...of
for (const value of iterableObject) {
  console.log(value); // 10, 20, 30, 40, 50
}

// Или с деструктуризацией
const [first, second, ...rest] = iterableObject;
console.log(first, second, rest); // 10 20 [30, 40, 50]

// Или с spread оператором
const allValues = [...iterableObject];
console.log(allValues); // [10, 20, 30, 40, 50]

```

1.4. Symbol.iterator

`Symbol.iterator` — это специальный символ (встроенный в JavaScript), который определяет метод по умолчанию для итерации объекта.

```
javascript

// Symbol.iterator в действии
console.log(Symbol.iterator); // Symbol(Symbol.iterator)
console.log(typeof Symbol.iterator); // 'symbol'

// Проверка, является ли объект итерируемым
function isIterable(obj) {
    return obj != null && typeof obj[Symbol.iterator] === 'function';
}

// Примеры
console.log(isIterable([1, 2, 3])); // true
console.log(isIterable('строка')); // true
console.log(isIterable(new Map())); // true
console.log(isIterable(new Set())); // true
console.log(isIterable({ a: 1, b: 2 })); // false (обычные объекты не итерируемы)
console.log(isIterable(123)); // false
console.log(isIterable(null)); // false
console.log(isIterable(undefined)); // false
```

1.5. Встроенные итерируемые объекты

```
javascript

// 1. Массивы
const array = [1, 2, 3];
const arrayIterator = array[Symbol.iterator]();
console.log(arrayIterator.next()); // { value: 1, done: false }

// 2. Строки
const str = 'Привет';
const strIterator = str[Symbol.iterator]();
console.log(strIterator.next()); // { value: 'П', done: false }

// 3. Map
const map = new Map([['a', 1], ['b', 2]]);
const mapIterator = map[Symbol.iterator]();
console.log(mapIterator.next()); // { value: ['a', 1], done: false }
```

```

// 4. Set
const set = new Set([1, 2, 3]);
const setIterator = set[Symbol.iterator]();
console.log(setIterator.next()); // { value: 1, done: false }

// 5. arguments
function testArguments() {
    const argsIterator = arguments[Symbol.iterator]();
    console.log(argsIterator.next()); // { value: 1, done: false }
}
testArguments(1, 2, 3);

// 6. NodeList (в браузере)
// const nodeList = document.querySelectorAll('div');
// const nodeListIterator = nodeList[Symbol.iterator]();

```

1.6. Создание собственных итерируемых структур данных

javascript

```

// Класс Range - диапазон чисел
class Range {
    constructor(start, end, step = 1) {
        this.start = start;
        this.end = end;
        this.step = step;

        if (step === 0) {
            throw new Error('Шаг не может быть равен 0');
        }
    }

    [Symbol.iterator]() {
        let current = this.start;
        const end = this.end;
        const step = this.step;

        return {
            next() {
                if ((step > 0 && current <= end) || (step < 0 && current >= end)) {
                    const value = current;
                    current += step;
                    return { value, done: false };
                }
            }
        };
    }
}

```

```

    }
    return { value: undefined, done: true };
  },

  // Метод return для преждевременного завершения
  return() {
    console.log(`Итерация диапазона прервана на ${current}`);
    return { done: true };
  }
};
}

// Дополнительные методы для удобства
toArray() {
  return [...this];
}

forEach(callback) {
  for (const value of this) {
    callback(value);
  }
}

map(callback) {
  const result = [];
  for (const value of this) {
    result.push(callback(value));
  }
  return result;
}
}

// Использование
const range1 = new Range(1, 5);
console.log([...range1]); // [1, 2, 3, 4, 5]

const range2 = new Range(5, 1, -1);
console.log([...range2]); // [5, 4, 3, 2, 1]

const range3 = new Range(0, 10, 2);
console.log(range3.toArray()); // [0, 2, 4, 6, 8, 10]

range3.forEach(n => console.log(n * n)); // 0, 4, 16, 36, 64, 100

```



```
const squares = range3.map(n => n * n);
console.log(squares); // [0, 4, 16, 36, 64, 100]

// Преждевременное завершение итерации
const range = new Range(1, 10);
for (const num of range) {
  if (num > 5) {
    break; // Вызовет метод return итератора
  }
  console.log(num); // 1, 2, 3, 4, 5
}
```

Часть 2: Генераторы

2.1. Что такое генераторы?

Генераторы — это специальные функции, которые можно приостанавливать и возобновлять. Они упрощают создание итераторов.

```
javascript

// Функция-генератор объявляется с function*
function* simpleGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

// При вызове генератор возвращает итератор
const generator = simpleGenerator();

console.log(generator.next()); // { value: 1, done: false }
console.log(generator.next()); // { value: 2, done: false }
console.log(generator.next()); // { value: 3, done: false }
console.log(generator.next()); // { value: undefined, done: true }

// Генератор автоматически реализует протокол итератора
for (const value of simpleGenerator()) {
  console.log(value); // 1, 2, 3
}
```

2.2. Синтаксис генераторов

```
javascript

// Различные способы объявления генераторов

// 1. Function declaration
function* generator1() {
  yield 1;
}

// 2. Function expression
const generator2 = function*() {
  yield 2;
};

// 3. Метод в объекте
const obj = {
  *generator3() {
    yield 3;
  }
};

// 4. Метод в классе
class MyClass {
  *generator4() {
    yield 4;
  }
}

// 5. В стрелочных функциях генераторы не поддерживаются
// const arrowGenerator = *() => {}; // Синтаксическая ошибка
```

2.3. Ключевое слово yield

yield — ключевое слово, которое приостанавливает выполнение генератора и возвращает значение.

```
javascript

function* yieldExamples() {
  console.log('Начало генератора');

  // yield возвращает значение
```

```

const result1 = yield 'первое значение';
console.log('Получено после первого yield:', result1);

const result2 = yield 'второе значение';
console.log('Получено после второго yield:', result2);

return 'завершение';
}

const gen = yieldExamples();

// Первый next() запускает генератор до первого yield
let next = gen.next();
console.log('Первый next:', next); // { value: 'первое значение', done: false }

// Второй next() передает значение первому yield
next = gen.next('аргумент для первого yield');
console.log('Второй next:', next); // { value: 'второе значение', done: false }

// Третий next() передает значение второму yield
next = gen.next('аргумент для второго yield');
console.log('Третий next:', next); // { value: 'завершение', done: true }

```

2.4. yield* — делегирование генераторов

`yield*` позволяет делегировать выполнение другому генератору или итерируемому объекту.

```

javascript

function* innerGenerator() {
  yield 2;
  yield 3;
}

function* outerGenerator() {
  yield 1;
  yield* innerGenerator(); // Делегируем innerGenerator
  yield 4;
}

console.log([...outerGenerator()]); // [1, 2, 3, 4]

```

```
// yield* работает с любыми итерируемыми объектами
function* flattenArray(array) {
  for (const item of array) {
    if (Array.isArray(item)) {
      yield* flattenArray(item); // Рекурсивное делегирование
    } else {
      yield item;
    }
  }
}

const nestedArray = [1, [2, [3, 4], 5], 6];
console.log([...flattenArray(nestedArray)]); // [1, 2, 3, 4, 5, 6]
```

2.5. Методы генераторов: return() и throw()

```
javascript

function* generatorWithMethods() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log('Блок finally выполняется всегда');
  }
}

const gen = generatorWithMethods();

console.log(gen.next()); // { value: 1, done: false }

// Метод return() завершает генератор досрочно
console.log(gen.return('завершено')); // { value: 'завершено', done: true }
// Выведет: "Блок finally выполняется всегда"

// После return() генератор завершен
console.log(gen.next()); // { value: undefined, done: true }

// Пример с throw()
function* generatorWithThrow() {
  try {
    yield 1;
    yield 2;
```

```

    } catch (error) {
      console.log('Поймана ошибка:', error.message);
      yield 'ошибка обработана';
    }
    yield 3;
  }

const gen2 = generatorWithThrow();
console.log(gen2.next()); // { value: 1, done: false }

// Метод throw() бросает ошибку в генератор
console.log(gen2.throw(new Error('Что-то пошло не так')));
// Выведет: "Поймана ошибка: Что-то пошло не так"
// Вернет: { value: 'ошибка обработана', done: false }

console.log(gen2.next()); // { value: 3, done: false }

```

Часть 3: Практическое применение

3.1. Ленивые вычисления

Генераторы идеально подходят для ленивых вычислений — вычислений по требованию.

```

javascript

// Генератор для бесконечной последовательности
function* infiniteSequence() {
  let i = 0;
  while (true) {
    yield i++;
  }
}

const numbers = infiniteSequence();

console.log(numbers.next().value); // 0
console.log(numbers.next().value); // 1
console.log(numbers.next().value); // 2
// ... можно продолжать бесконечно

// Фибоначчи через генератор

```

```

function* fibonacci() {
  let [prev, current] = [0, 1];

  while (true) {
    yield current;
    [prev, current] = [current, prev + current];
  }
}

const fib = fibonacci();

// Получаем первые 10 чисел Фибоначчи
const first10Fib = [];
for (let i = 0; i < 10; i++) {
  first10Fib.push(fib.next().value);
}
console.log(first10Fib); // [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]

// Практический пример: генератор для чтения файла по частям
function* readFileInChunks(filePath, chunkSize = 1024) {
  // Имитация чтения файла
  const fileSize = 10000; // 10KB
  let position = 0;

  while (position < fileSize) {
    // Имитация чтения чанка
    const chunk = `Чанк с ${position} до ${Math.min(position + chunkSize, fileSize)}`;
    position += chunkSize;

    yield chunk;

    // Можно имитировать задержку
    // yield new Promise(resolve => setTimeout(() => resolve(chunk), 100));
  }

  return 'Файл прочитан полностью';
}

// Использование
const fileReader = readFileInChunks('largefile.txt', 2048);

// Читаем первые 3 чанка
for (let i = 0; i < 3; i++) {
  console.log(fileReader.next().value);
}

```

```
}
```

3.2. Асинхронные генераторы (ES2018)

```
javascript
```

```
// Асинхронные генераторы используют async function*
```

```
async function* asyncGenerator() {  
  const urls = [  
    'https://api.example.com/data1',  
    'https://api.example.com/data2',  
    'https://api.example.com/data3'  
  ];
```

```
  for (const url of urls) {  
    // Имитация асинхронного запроса  
    yield new Promise(resolve => {  
      setTimeout(() => {  
        resolve(`Данные с ${url}`);  
      }, Math.random() * 1000);  
    });  
  }  
}
```

```
}
```

```
// Использование асинхронного генератора
```

```
async function processAsyncData() {  
  const generator = asyncGenerator();  
  
  for await (const data of generator) {  
    console.log('Получены данные:', data);  
  }  
}
```

```
console.log('Все данные обработаны');  
}
```

```
processAsyncData();
```

```
// Реальный пример: чтение потока данных
```

```
async function* streamToAsyncIterator(stream) {  
  const reader = stream.getReader();  
  
  try {  
    while (true) {  
      const { done, value } = await reader.read();
```

```

        if (done) {
            return;
        }

        yield value;
    }
} finally {
    reader.releaseLock();
}
}

// Использование с fetch
async function fetchAndProcessStream(url) {
    const response = await fetch(url);
    const generator = streamToAsyncIterator(response.body);

    for await (const chunk of generator) {
        console.log('Получен чанк:', chunk);
    }
}

```

3.3. Управление состоянием и конечные автоматы

```

javascript

// Конечный автомат (State Machine) через генератор
function* trafficLight() {
    const states = [
        { color: 'красный', duration: 5000, next: 'желтый' },
        { color: 'желтый', duration: 2000, next: 'зеленый' },
        { color: 'зеленый', duration: 5000, next: 'красный' }
    ];

    let currentState = 0;

    while (true) {
        const state = states[currentState];
        console.log(`Светофор: ${state.color}`);

        // Ждем указанное время
        yield new Promise(resolve => {
            setTimeout(() => {
                currentState = (currentState + 1) % states.length;
            }, state.duration);
        });
    }
}

```



```

        resolve(state.next);
    }, state.duration);
});
}
}

```

// Использование светофора

```

async function runTrafficLight() {
    const light = trafficLight();

    for (let i = 0; i < 6; i++) { // 6 смен состояний
        const nextColor = await light.next().value;
        console.log(`Переключаемся на: ${nextColor}`);
    }
}

```

```
runTrafficLight();
```

// Генератор для управления UI состоянием

```

function* uiStateManager() {
    let state = 'IDLE';

    while (true) {
        const action = yield state;

        switch (state) {
            case 'IDLE':
                if (action === 'START') state = 'LOADING';
                break;
            case 'LOADING':
                if (action === 'SUCCESS') state = 'SUCCESS';
                else if (action === 'ERROR') state = 'ERROR';
                break;
            case 'SUCCESS':
                if (action === 'RESET') state = 'IDLE';
                break;
            case 'ERROR':
                if (action === 'RETRY') state = 'LOADING';
                else if (action === 'RESET') state = 'IDLE';
                break;
        }
    }
}

```

// Использование

```
const manager = uiStateManager();
```

```
manager.next(); // Инициализация
```

```
console.log(manager.next('START').value); // 'LOADING'
```

```
console.log(manager.next('SUCCESS').value); // 'SUCCESS'
```

```
console.log(manager.next('RESET').value); // 'IDLE'
```

3.4. Пайплайны обработки данных

javascript

// Создает пайплайн обработки данных с генераторами

```
function* filter(predicate, source) {
```

```
  for (const item of source) {
```

```
    if (predicate(item)) {
```

```
      yield item;
```

```
    }
```

```
  }
```

```
}
```

```
function* map(transform, source) {
```

```
  for (const item of source) {
```

```
    yield transform(item);
```

```
  }
```

```
}
```

```
function* take(n, source) {
```

```
  let count = 0;
```

```
  for (const item of source) {
```

```
    if (count >= n) return;
```

```
    yield item;
```

```
    count++;
```

```
  }
```

```
}
```

// Исходные данные

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

// Собираем пайплайн

```
const result = take(
```

```
  3,
```

```
  map(
```

```
    x => x * x,
```

```

    filter(
      x => x % 2 === 0,
      numbers
    )
  )
);

console.log([...result]); // [4, 16, 36] (квадраты первых трех четных чисел)

// Более элегантный способ с помощью функций-оберток
function pipeline(...operations) {
  return function*(source) {
    let current = source;

    for (const operation of operations) {
      current = operation(current);
    }

    yield* current;
  };
}

// Создает переиспользуемый пайплайн
const processNumbers = pipeline(
  source => filter(x => x > 5, source),
  source => map(x => x * 2, source),
  source => take(3, source)
);

console.log([...processNumbers(numbers)]); // [12, 14, 16]

```

Часть 4: Продвинутые техники

4.1. Двусторонняя коммуникация с генераторами

```

javascript

// Генератор для двусторонней коммуникации
function* twoWayCommunication() {
  // Первое сообщение от генератора
  const name = yield 'Как тебя зовут?';

```

```

// Второе сообщение с использованием полученного имени
const age = yield `Привет, ${name}! Сколько тебе лет?`;

// Завершающее сообщение
return `Отлично, ${name} (${age} лет)! Рад познакомиться!`;
}

// Функция для управления диалогом
function runDialog(generator) {
  const gen = generator();
  let result = gen.next();

  while (!result.done) {
    // Выводим вопрос от генератора
    const answer = prompt(result.value);

    // Отправляем ответ обратно в генератор
    result = gen.next(answer);
  }

  // Выводим финальное сообщение
  console.log(result.value);
}

// В браузере:
// runDialog(twoWayCommunication);

// Альтернативная реализация с обработкой событий
function* eventDrivenGenerator() {
  console.log('Генератор запущен. Ожидая события...');

  // Ждем различные события
  while (true) {
    const event = yield;

    switch (event.type) {
      case 'CLICK':
        console.log(`Клик на координатах (${event.x}, ${event.y})`);
        break;
      case 'KEYPRESS':
        console.log(`Нажата клавиша: ${event.key}`);
        break;
      case 'EXIT':
        console.log('Завершение работы генератора');
    }
  }
}

```

```

        return;
    default:
        console.log('Неизвестное событие:', event);
    }
}
}

// Использование
const eventHandler = eventDrivenGenerator();
eventHandler.next(); // Запуск генератора

// Имитация событий
eventHandler.next({ type: 'CLICK', x: 100, y: 200 });
eventHandler.next({ type: 'KEYPRESS', key: 'Enter' });
eventHandler.next({ type: 'EXIT' });

```

4.2. Генераторы для тестирования и мокирования

```

javascript

// Генератор для создания последовательности мок-данных
function* mockDataGenerator(template) {
    let id = 1;

    while (true) {
        const item = { ...template };

        // Заменяем плейсхолдеры
        for (const key in item) {
            if (typeof item[key] === 'string') {
                item[key] = item[key]
                    .replace('{{id}}', id)
                    .replace('{{timestamp}}', Date.now());
            }
        }

        item.id = id++;
        yield item;
    }
}

// Создаем генератор пользователей
const userGenerator = mockDataGenerator({
    id: '{{id}}',

```

```

    name: 'Пользователь {{id}}',
    email: 'user{{id}}@example.com',
    createdAt: '{{timestamp}}',
    active: true
  });

  // Получаем 3 тестовых пользователя
  const testUsers = [];
  for (let i = 0; i < 3; i++) {
    testUsers.push(userGenerator.next().value);
  }

  console.log(testUsers);

  // Генератор для тестирования асинхронных операций
  function* asyncTestScenario() {
    // Симуляция различных сценариев
    yield Promise.resolve('Успешный запрос 1');
    yield Promise.reject(new Error('Ошибка сети'));
    yield Promise.resolve('Успешный запрос 2');
    yield new Promise(resolve => setTimeout(() => resolve('Запрос с задержкой'), 1000));
  }

  // Тестовый раннер
  async function runTestScenario() {
    const scenario = asyncTestScenario();
    const results = [];

    while (true) {
      const { value: promise, done } = scenario.next();

      if (done) break;

      try {
        const result = await promise;
        results.push({ status: 'fulfilled', value: result });
      } catch (error) {
        results.push({ status: 'rejected', reason: error.message });
      }
    }

    return results;
  }

```

```
runTestScenario().then(console.log);
```

4.3. Параллельное выполнение с генераторами

javascript

// Генератор для управления параллельными задачами

```
function* parallelTasks(tasks) {  
  // Запускаем все задачи параллельно  
  const promises = tasks.map(task => task());  
  
  // Ждем завершения всех задач  
  const results = yield Promise.all(promises);  
  
  // Обрабатываем результаты  
  for (const result of results) {  
    yield `Завершено: ${result}`;  
  }  
  
  return 'Все задачи завершены';  
}
```

// Использование

```
async function runParallel() {  
  const tasks = [  
    () => new Promise(resolve => setTimeout(() => resolve('Задача 1'), 1000)),  
    () => new Promise(resolve => setTimeout(() => resolve('Задача 2'), 500)),  
    () => new Promise(resolve => setTimeout(() => resolve('Задача 3'), 1500))  
  ];  
  
  const generator = parallelTasks(tasks);  
  
  // Получаем Promise.all  
  const { value: promise } = generator.next();  
  
  // Ждем завершения всех промисов  
  const results = await promise;  
  
  // Передаем результаты обратно в генератор  
  generator.next(results);  
  
  // Получаем остальные yield  
  let result = generator.next();  
  while (!result.done) {
```

```

    console.log(result.value);
    result = generator.next();
  }

  console.log(result.value); // 'Все задачи завершены'
}

runParallel();

// Генератор с ограничением параллелизма
function* limitedParallel(tasks, limit) {
  const results = [];
  let running = 0;
  let index = 0;

  while (index < tasks.length || running > 0) {
    // Запускаем новые задачи, пока есть лимит
    while (running < limit && index < tasks.length) {
      const taskIndex = index++;
      const task = tasks[taskIndex];

      running++;
      task().then(result => {
        results[taskIndex] = result;
        running--;
      }).catch(error => {
        results[taskIndex] = error;
        running--;
      });
    }

    // Ждем завершения хотя бы одной задачи
    if (running === limit || (index >= tasks.length && running > 0)) {
      yield new Promise(resolve => {
        // Проверяем завершение каждые 100мс
        const checkInterval = setInterval(() => {
          if (running < limit || (index >= tasks.length && running === 0)) {
            clearInterval(checkInterval);
            resolve(results.slice());
          }
        }, 100);
      });
    }
  }
}

```



```

    return results;
}

// Пример использования
async function runLimitedParallel() {
    const tasks = Array.from({ length: 10 }, (_, i) =>
        () => new Promise(resolve =>
            setTimeout(() => resolve(`Задача ${i + 1}`), Math.random() * 2000)
        )
    );

    const generator = limitedParallel(tasks, 3); // Максимум 3 параллельных задачи

    for await (const partialResults of generator) {
        console.log('Текущий прогресс:', partialResults.filter(Boolean).length, '/',
tasks.length);
    }

    const finalResults = generator.next().value;
    console.log('Все задачи завершены:', finalResults);
}

runLimitedParallel();

```

4.4. Генераторы для обработки потоков

```

javascript

// Обработка потоков данных с backpressure
function* streamProcessor(stream, processor, bufferSize = 10) {
    const buffer = [];
    let isProcessing = false;

    // Функция для обработки буфера
    function processBuffer() {
        if (isProcessing || buffer.length === 0) return;

        isProcessing = true;
        const chunk = buffer.shift();

        processor(chunk).then(() => {
            isProcessing = false;
            processBuffer(); // Рекурсивно обрабатываем следующий чанк
        });
    }
}

```

```

    }).catch(error => {
      console.error('Ошибка обработки:', error);
      isProcessing = false;
      processBuffer();
    });
  }

  // Обрабатываем входящий поток
  for await (const chunk of stream) {
    buffer.push(chunk);

    // Если буфер заполнен, ждем
    while (buffer.length >= bufferSize) {
      yield new Promise(resolve => {
        // Проверяем каждые 50мс
        const interval = setInterval(() => {
          if (buffer.length < bufferSize) {
            clearInterval(interval);
            resolve();
          }
        }, 50);
      });
    }

    // Запускаем обработку
    processBuffer();
  }

  // Ждем завершения обработки оставшихся данных
  while (buffer.length > 0 || isProcessing) {
    yield new Promise(resolve => setTimeout(resolve, 100));
  }

  return 'Поток обработан полностью';
}

// Пример использования
async function exampleStreamProcessing() {
  // Имитация потока данных
  async function* mockStream() {
    for (let i = 0; i < 100; i++) {
      yield `Данные ${i}`;
      await new Promise(resolve => setTimeout(resolve, 50));
    }
  }

```

```

}

// Процессор для данных
async function processor(data) {
  console.log('Обработка:', data);
  await new Promise(resolve => setTimeout(resolve, 200)); // Имитация работы
  return `Обработано: ${data}`;
}

const stream = mockStream();
const generator = streamProcessor(stream, processor, 5);

for await (const _ of generator) {
  // Просто ждем обработки
}

const result = generator.next().value;
console.log(result);
}

exampleStreamProcessing();

```

Часть 5: Лучшие практики и паттерны

5.1. Обработка ошибок в генераторах

```

javascript

// Паттерн для безопасной работы с генераторами
class SafeGenerator {
  static async run(generatorFn, ...args) {
    const generator = generatorFn(...args);
    const result = {
      value: null,
      error: null,
      done: false
    };

    try {
      let nextValue = generator.next();

      while (!nextValue.done) {

```

```

    try {
        // Если значение - Promise, ждем его выполнения
        if (nextValue.value instanceof Promise) {
            nextValue.value = await nextValue.value;
        }

        // Обрабатываем следующее значение
        nextValue = generator.next(nextValue.value);
    } catch (innerError) {
        // Ошибка внутри yield
        nextValue = generator.throw(innerError);
    }
}

result.value = nextValue.value;
result.done = true;
} catch (error) {
    result.error = error;
    result.done = true;
} finally {
    // Гарантируем вызов return для очистки ресурсов
    if (!result.done) {
        generator.return();
    }
}

return result;
}

static wrap(generatorFn) {
    return async function(...args) {
        const result = await SafeGenerator.run(generatorFn, ...args);

        if (result.error) {
            throw result.error;
        }

        return result.value;
    };
}
}

// Пример использования
function* riskyGenerator() {

```

```

try {
  const data = yield fetch('https://api.example.com/data');
  const processed = yield processData(data);
  return processed;
} catch (error) {
  console.error('Ошибка в генераторе:', error);
  return 'значение по умолчанию';
} finally {
  console.log('Генератор завершил работу');
}
}

// Безопасный запуск
async function example() {
  const result = await SafeGenerator.run(riskyGenerator);

  if (result.error) {
    console.error('Критическая ошибка:', result.error);
  } else {
    console.log('Результат:', result.value);
  }
}

// Или с оберткой
const safeRiskyGenerator = SafeGenerator.wrap(riskyGenerator);

try {
  const value = await safeRiskyGenerator();
  console.log('Значение:', value);
} catch (error) {
  console.error('Ошибка:', error);
}
}

```

5.2. Оптимизация производительности

```

javascript

// Оптимизация итераторов для больших коллекций
class OptimizedIterator {
  constructor(collection, batchSize = 1000) {
    this.collection = collection;
    this.batchSize = batchSize;
    this.index = 0;
  }
}

```

```

[Symbol.iterator]() {
  return {
    next: () => {
      if (this.index >= this.collection.length) {
        return { done: true };
      }

      // Возвращаем батч элементов для оптимизации
      const batch = this.collection.slice(
        this.index,
        Math.min(this.index + this.batchSize, this.collection.length)
      );

      this.index += this.batchSize;

      return {
        value: batch,
        done: false
      };
    }
  };
}
}

```

```

// Пример использования с большим массивом
const largeArray = new Array(1000000).fill().map((_, i) => i);
const optimizedIterator = new OptimizedIterator(largeArray, 5000);

```

```

let total = 0;
for (const batch of optimizedIterator) {
  // Обрабатываем батчами для уменьшения нагрузки на GC
  total += batch.reduce((sum, num) => sum + num, 0);
}

```

```

console.log('Сумма:', total);

```

```

// Ленивый итератор с кэшированием
function* lazyCachedIterator(source, cacheSize = 100) {
  const cache = new Map();
  let index = 0;

  while (true) {
    if (cache.has(index)) {

```

```

    // Возвращаем из кэша
    yield cache.get(index);
  } else {
    // Получаем новое значение
    const { value, done } = source.next();

    if (done) {
      return;
    }

    // Сохраняем в кэш
    cache.set(index, value);

    // Ограничиваем размер кэша
    if (cache.size > cacheSize) {
      const oldestKey = cache.keys().next().value;
      cache.delete(oldestKey);
    }

    yield value;
  }

  index++;
}

// Использование
const expensiveIterator = {
  data: Array.from({ length: 1000 }, (_, i) => i),
  index: 0,
  next() {
    if (this.index >= this.data.length) {
      return { done: true };
    }

    // Имитация дорогого вычисления
    const value = this.data[this.index] * 2;
    this.index++;

    return { value, done: false };
  }
};

const cached = lazyCachedIterator(expensiveIterator, 50);

```

```
// Первый проход - вычисления
for (const value of cached) {
  if (value > 100) break;
}

// Второй проход - из кэша
cached.next(); // Возвращает из кэша
```

5.3. Композиция генераторов

javascript

```
// Утилиты для композиции генераторов
const GeneratorUtils = {
  // Объединение генераторов
  merge(...generators) {
    return function*() {
      const iterators = generators.map(gen => gen());

      while (iterators.length > 0) {
        for (let i = 0; i < iterators.length; i++) {
          const { value, done } = iterators[i].next();

          if (done) {
            iterators.splice(i, 1);
            i--;
          } else {
            yield value;
          }
        }
      }
    };
  },

  // Чередование генераторов
  interleave(...generators) {
    return function*() {
      const iterators = generators.map(gen => gen());

      while (iterators.some(it => !it.done)) {
        for (const iterator of iterators) {
          const { value, done } = iterator.next();

```



```
        if (!done) {
            yield value;
        }
    }
};
},
```

// Фильтрация генератора

```
filter(predicate, generator) {
    return function*() {
        for (const value of generator()) {
            if (predicate(value)) {
                yield value;
            }
        }
    };
},
```

// Преобразование генератора

```
map(transform, generator) {
    return function*() {
        for (const value of generator()) {
            yield transform(value);
        }
    };
},
```

// Ограничение количества элементов

```
take(n, generator) {
    return function*() {
        let count = 0;
        for (const value of generator()) {
            if (count >= n) return;
            yield value;
            count++;
        }
    };
},
```

// Создание конвейера

```
pipeline(...operations) {
    return function*(source) {
        let current = source;
```

```

    for (const operation of operations) {
        current = operation(current);
    }

    yield* current;
};
}
};

// Пример использования композиции
function* numbers() {
    let i = 0;
    while (true) {
        yield i++;
    }
}

function* letters() {
    const alphabet = 'abcdefghijklmnopqrstuvwxyz';
    let i = 0;
    while (true) {
        yield alphabet[i++ % alphabet.length];
    }
}

// Объединенный генератор
const merged = GeneratorUtils.merge(
    GeneratorUtils.take(5, numbers),
    GeneratorUtils.take(3, letters)
);

console.log([...merged()]); // [0, 1, 2, 3, 4, 'a', 'b', 'c']

// Конвейер обработки
const processed = GeneratorUtils.pipeline(
    source => GeneratorUtils.filter(x => x % 2 === 0, source),
    source => GeneratorUtils.map(x => x * x, source),
    source => GeneratorUtils.take(5, source)
)(numbers);

console.log([...processed]); // [0, 4, 16, 36, 64]

```

Часть 6: Реальные примеры использования

6.1. Роутинг в одностраничном приложении

javascript

// Генератор для управления навигацией

```
function* router(routes) {
  let currentRoute = null;
  let params = {};

  while (true) {
    // Ждем событие навигации
    const { path, data } = yield { currentRoute, params };

    // Ищем подходящий маршрут
    const route = routes.find(r => {
      const pattern = r.path.replace(/:\w+/g, '([^/]+)');
      const regex = new RegExp(`^${pattern}$`);
      return regex.test(path);
    });

    if (!route) {
      console.error(`Маршрут не найден: ${path}`);
      continue;
    }

    // Извлекаем параметры из пути
    const pattern = route.path.replace(/:\w+/g, '([^/]+)');
    const regex = new RegExp(`^${pattern}$`);
    const matches = path.match(regex);

    if (matches) {
      const paramNames = [...route.path.matchAll(/:(\w+)/g)].map(m => m[1]);
      params = {};

      paramNames.forEach((name, index) => {
        params[name] = matches[index + 1];
      });
    }

    currentRoute = route;
  }
}
```

```

// Вызываем обработчик маршрута
if (route.handler) {
  try {
    yield* route.handler(params, data);
  } catch (error) {
    console.error(`Ошибка в обработчике маршрута ${route.path}:`, error);
  }
}
}
}

```

// Определение маршрутов

```

const routes = [
  {
    path: '/',
    name: 'home',
    *handler() {
      console.log('Домашняя страница');
      yield { type: 'RENDER', component: 'HomePage' };
    }
  },
  {
    path: '/users/:id',
    name: 'user',
    *handler(params) {
      console.log(`Страница пользователя ${params.id}`);

      // Имитация загрузки данных
      yield { type: 'LOADING', message: 'Загрузка данных пользователя...' };

      const userData = yield new Promise(resolve => {
        setTimeout(() => {
          resolve({ id: params.id, name: `Пользователь ${params.id}` });
        }, 1000);
      });

      yield { type: 'RENDER', component: 'UserPage', data: userData };
    }
  },
  {
    path: '/settings',
    name: 'settings',
    *handler(_, data) {
      console.log('Настройки', data);
    }
  }
]

```

```
        yield { type: 'RENDER', component: 'SettingsPage' };
    }
}
];
```

// Использование роутера

```
async function runRouter() {
    const routeGenerator = router(routes);
    routeGenerator.next(); // Инициализация
```

// Имитация навигации

```
const navigations = [
    { path: '/' },
    { path: '/users/123' },
    { path: '/settings', data: { section: 'profile' } }
];
```

```
for (const navigation of navigations) {
    console.log(`\nНавигация на: ${navigation.path}`);

    const { value: state } = routeGenerator.next(navigation);
    console.log('Состояние после навигации:', state);
```

// Обрабатываем yield из обработчиков маршрутов

```
let result = routeGenerator.next();
while (!result.done) {
    const action = result.value;
    console.log('Действие от обработчика:', action);
```

// Имитация обработки действий

```
if (action.type === 'RENDER') {
    console.log(`Рендерим компонент: ${action.component}`);
} else if (action.type === 'LOADING') {
    console.log(action.message);
}
```

```
result = routeGenerator.next();
}
```

```
}
}
```

```
runRouter();
```

6.2. Управление анимациями

javascript

// Генератор для сложных анимаций

```
function* animationTimeline(keyframes) {
  const startTime = Date.now();

  for (const keyframe of keyframes) {
    const { duration, properties, easing = 'linear' } = keyframe;
    const endTime = startTime + duration;

    while (Date.now() < endTime) {
      const elapsed = Date.now() - startTime;
      const progress = Math.min(elapsed / duration, 1);

      // Применяем easing функцию
      const easedProgress = applyEasing(progress, easing);

      // Вычисляем текущие значения свойств
      const currentValues = {};

      for (const [property, [start, end]] of Object.entries(properties)) {
        currentValues[property] = start + (end - start) * easedProgress;
      }

      yield currentValues;

      // Ждем следующий кадр
      yield new Promise(resolve => requestAnimationFrame(resolve));
    }

    // Гарантируем конечные значения
    const finalValues = {};
    for (const [property, [_, end]] of Object.entries(properties)) {
      finalValues[property] = end;
    }

    yield finalValues;
  }
}
```

// Функции easing

```
function applyEasing(t, type) {
```

```

switch (type) {
  case 'linear':
    return t;
  case 'easeIn':
    return t * t;
  case 'easeOut':
    return t * (2 - t);
  case 'easeInOut':
    return t < 0.5 ? 2 * t * t : -1 + (4 - 2 * t) * t;
  default:
    return t;
}
}

```

// Использование анимации

```

async function runAnimation() {
  const element = {
    x: 0,
    y: 0,
    scale: 1,
    opacity: 0
  };

  const timeline = animationTimeline([
    {
      duration: 1000,
      easing: 'easeOut',
      properties: {
        x: [0, 100],
        y: [0, 50],
        opacity: [0, 1]
      }
    },
    {
      duration: 500,
      easing: 'easeInOut',
      properties: {
        scale: [1, 1.5]
      }
    },
    {
      duration: 800,
      easing: 'easeIn',
      properties: {

```

```

        x: [100, 0],
        y: [50, 0],
        opacity: [1, 0]
    }
}
]);

console.log('Начало анимации');

for await (const values of timeline) {
    // Обновляем свойства элемента
    Object.assign(element, values);
    console.log('Текущие значения:', element);

    // В реальном приложении здесь был бы рендеринг
    // render(element);
}

console.log('Анимация завершена');
}

runAnimation();

// Генератор для последовательности анимаций
function* animationSequence(...animations) {
    for (const animation of animations) {
        yield* animation;
        yield 'ANIMATION_COMPLETE';
    }

    yield 'SEQUENCE_COMPLETE';
}

// Пример использования последовательности
const moveRight = animationTimeline([
    {
        duration: 1000,
        properties: { x: [0, 100] }
    }
]);

const fadeIn = animationTimeline([
    {
        duration: 500,
        properties: { opacity: [0, 1] }
    }
]);

```



```
const sequence = animationSequence(moveRight, fadeIn);
```

```
// Обработка последовательности
```

```
async function runSequence() {  
  for await (const frame of sequence) {  
    if (frame === 'ANIMATION_COMPLETE') {  
      console.log('Анимация завершена');  
    } else if (frame === 'SEQUENCE_COMPLETE') {  
      console.log('Вся последовательность завершена');  
    } else {  
      console.log('Кадр анимации:', frame);  
    }  
  }  
}
```

```
runSequence();
```

6.3. Обработка пользовательского ввода

```
javascript
```

```
// Генератор для обработки последовательности нажатий клавиш
```

```
function* keySequenceDetector(targetSequence) {  
  const buffer = [];  
  
  while (true) {  
    // Ждем нажатие клавиши  
    const key = yield;  
    buffer.push(key);  
  
    // Ограничиваем размер буфера  
    if (buffer.length > targetSequence.length) {  
      buffer.shift();  
    }  
  
    // Проверяем совпадение  
    if (buffer.join('') === targetSequence) {  
      buffer.length = 0; // Очищаем буфер  
      yield { type: 'SEQUENCE_DETECTED', sequence: targetSequence };  
    }  
  }  
}
```

```
// Генератор для drag-and-drop
```

```

function* dragAndDrop(element) {
  let isDragging = false;
  let startX, startY;
  let currentX = 0, currentY = 0;

  // Событие начала перетаскивания
  element.addEventListener('mousedown', (e) => {
    isDragging = true;
    startX = e.clientX - currentX;
    startY = e.clientY - currentY;
  });

  document.addEventListener('mousemove', (e) => {
    if (!isDragging) return;

    currentX = e.clientX - startX;
    currentY = e.clientY - startY;

    // Отправляем координаты в генератор
    dragGenerator.next({ x: currentX, y: currentY });
  });

  document.addEventListener('mouseup', () => {
    if (isDragging) {
      isDragging = false;
      dragGenerator.next({ type: 'DROP', x: currentX, y: currentY });
    }
  });

  // Основной цикл генератора
  while (true) {
    const event = yield;

    if (event.type === 'DROP') {
      console.log(`Объект перемещен в (${event.x}, ${event.y})`);
      yield { type: 'COMPLETE', position: { x: event.x, y: event.y } };
    } else {
      // Обновление позиции во время перетаскивания
      element.style.transform = `translate(${event.x}px, ${event.y}px)`;
      yield { type: 'DRAGGING', position: { x: event.x, y: event.y } };
    }
  }
}

```

```

// Комбинированный генератор для жестов
function* gestureRecognizer() {
  const gestures = {
    '*keyDetector': keySequenceDetector('up up down down left right left right b a'),
    // '*dragDetector': dragAndDrop(document.getElementById('draggable'))
  };

  const detectors = {};

  // Инициализируем все детекторы
  for (const [name, generator] of Object.entries(gestures)) {
    detectors[name] = generator();
    detectors[name].next(); // Инициализация
  }

  while (true) {
    // Ждем события
    const event = yield;

    // Пропускаем событие через все детекторы
    for (const [name, detector] of Object.entries(detectors)) {
      const result = detector.next(event);

      if (!result.done && result.value) {
        // Обнаружен жест
        yield { type: 'GESTURE_DETECTED', gesture: name, details: result.value };
      }
    }
  }
}

// Пример использования
async function runGestureDemo() {
  const gestureEngine = gestureRecognizer();
  gestureEngine.next(); // Инициализация

  // Имитация ввода
  const inputs = ['up', 'up', 'down', 'down', 'left', 'right', 'left', 'right', 'b', 'a'];

  for (const input of inputs) {
    console.log(`Ввод: ${input}`);

    const result = gestureEngine.next(input);
  }
}

```

```
    if (!result.done && result.value) {  
        console.log('Обнаружен жест!', result.value);  
    }  
}  
}  
  
runGestureDemo();
```

Заключение

Ключевые выводы:

1. **Итераторы** — это универсальный протокол для перебора любых коллекций, реализуемый через метод `[Symbol.iterator]()`.
2. **Генераторы** — это специальные функции, которые:
 - Могут приостанавливать выполнение с помощью `yield`
 - Автоматически реализуют протокол итератора
 - Поддерживают двустороннюю коммуникацию
 - Упрощают создание ленивых последовательностей
3. **Асинхронные генераторы** (ES2018) позволяют работать с асинхронными последовательностями с помощью `for await...of`.

Основные преимущества:

1. **Ленивые вычисления** — значения вычисляются только когда они нужны
2. **Экономия памяти** — можно обрабатывать бесконечные последовательности
3. **Чистый код** — разделение ответственности между производством и потреблением данных
4. **Асинхронность** — удобная работа с потоками данных и промисами
5. **Состояние** — генераторы сохраняют состояние между вызовами

Практические применения:

1. **Обработка больших данных** — потоковая обработка файлов, логов
2. **Управление состоянием** — конечные автоматы, роутеры
3. **Анимации** — сложные последовательности анимаций
4. **Тестирование** — генерация тестовых данных, мокирование
5. **Пользовательский интерфейс** — обработка ввода, drag-and-drop
6. **Сети** — обработка потоков данных, WebSockets

Рекомендации:

1. **Используйте `for...of`** для итерации вместо ручного вызова `next()`
2. **Применяйте `yield`*** для делегирования генераторов
3. **Обрабатывайте ошибки** с помощью `try...catch` в генераторах
4. **Используйте асинхронные генераторы** для работы с промисами
5. **Экспериментируйте** с композицией генераторов для сложных задач

Итераторы и генераторы — мощные инструменты, которые меняют подход к работе с последовательностями и асинхронным кодом в JavaScript. Они позволяют писать более декларативный, читаемый и эффективный код, особенно при работе с потоками данных, асинхронными операциями и сложными состояниями.

Глава 10: Строки и регулярные выражения

10.1. Строковые литералы и шаблонные строки

Введение: эволюция строк в JavaScript

Строки — один из самых фундаментальных типов данных в JavaScript. Их обработка и представление эволюционировали от простых кавычек до мощных шаблонных строк (template literals), предоставляющих современные возможности для работы с текстом.

Исторический контекст:

1. **ES1-ES5:** Только строковые литералы с одинарными/двойными кавычками
2. **ES6 (ES2015):** Шаблонные строки с обратными кавычками
3. **ES2016-ES2023:** Дополнительные методы и возможности для строк

Часть 1: Строковые литералы

1.1. Основные типы строковых литералов

1.1.1. Строки в одинарных кавычках

```
javascript

// Одинарные кавычки
const singleQuoted = 'Это строка в одинарных кавычках';
const apostrophe = 'Don\'t worry'; // Экранирование апострофа
const simple = 'Hello World';

// Использование:
const message = 'Привет, мир!';
const path = '/usr/local/bin';
const jsonKey = 'userName';
```

1.1.2. Строки в двойных кавычках

```
javascript

// Двойные кавычки
const doubleQuoted = "Это строка в двойных кавычках";
const quoteInside = "Он сказал: \"Привет!\""; // Экранирование кавычек
const htmlAttribute = "class=\"container\"";

// Использование:
const greeting = "Hello, World!";
const cssValue = "font-family: Arial, sans-serif";
const xmlAttribute = "id=\"main-content\"";
```

1.1.3. Эквивалентность одинарных и двойных кавычек

```
javascript

// С технической точки зрения нет разницы
const str1 = 'Hello';
const str2 = "Hello";

console.log(str1 === str2); // true
console.log(typeof str1); // 'string'
console.log(typeof str2); // 'string'

// Разница только в необходимости экранирования
const withApostrophe1 = 'It\'s a string'; // Экранирование нужно
const withApostrophe2 = "It's a string"; // Экранирование не нужно

const withQuotes1 = 'He said "Hello"'; // Экранирование не нужно
const withQuotes2 = "He said \"Hello\""; // Экранирование нужно
```

1.2. Экранирование символов

```
javascript

// Специальные escape-последовательности
const escapes = {
  // Базовые escape-последовательности
  newline: 'Первая строка\nВторая строка', // Перевод строки
  tab: 'Колонка1\tКолонка2\tКолонка3', // Табуляция
  backslash: 'Путь: C:\\Users\\Name\\file.txt', // Обратный слеш
  quote: 'Он сказал: \'Привет!\'', // Кавычка
  doubleQuote: "Значение: \"важное\"", // Двойная кавычка
}
```

```

// Управляющие символы
carriageReturn: 'Текст\rНачало', // Возврат каретки
formFeed: 'Страница 1\fСтраница 2', // Перевод страницы
verticalTab: 'Линия1\vЛиния2', // Вертикальная табуляция
backspace: 'Текс\bст', // Забой (не работает во всех средах)

// Специальные escape-последовательности
unicode: '\u0041\u0042\u0043', // ABC в Unicode
hex: '\x41\x42\x43', // ABC в hex
octal: '\101\102\103' // ABC в octal (устарело)
};

console.log(escapes.newline);
// Вывод:
// Первая строка
// Вторая строка

console.log(escapes.tab);
// Вывод:
// Колонка1      Колонка2      Колонка3

// Unicode escape-последовательности
const heart = '\u2764'; // ♥
const emoji = '\u{1F600}'; // 😄 (требуется фигурных скобок для кодов > 0xFFFF)
const flags = '\u{1F1F7}\u{1F1FA}'; // RU

console.log(heart, emoji, flags);

// Длина строки с escape-последовательностями
const str = 'A\nB';
console.log(str.length); // 3 (A, \n, B)
console.log(str); // A (перевод строки) B

```

1.3. Многострочные строки до ES6

```

javascript

// Способ 1: Конкатенация с \n
const multiline1 = 'Первая строка\n' +
                   'Вторая строка\n' +
                   'Третья строка';

// Способ 2: Массив с join
const multiline2 = [

```



```

    'Первая строка',
    'Вторая строка',
    'Третья строка'
  ].join('\n');

// Способ 3: Escape-последовательность в конце строки (устарело)
const multiline3 = 'Первая строка\
Вторая строка\
Третья строка'; // Не рекомендуется!

console.log(multiline1);
console.log(multiline2);
console.log(multiline3); // Все в одной строке!

// Проблемы с отступами
const code = 'function hello() {\n' +
  '  console.log("Hello");\n' +
  '  return true;\n' +
  '}';

console.log(code);
// Вывод:
// function hello() {
//   console.log("Hello");
//   return true;
// }

```

1.4. Создание строк через конструктор String

```

javascript

// Создание строки через конструктор String
const str1 = String('обычная строка'); // Примитивная строка
const str2 = new String('объект строка'); // Объект String

console.log(typeof str1); // 'string'
console.log(typeof str2); // 'object'

// Но они ведут себя похоже
console.log(str1.length); // 14
console.log(str2.length); // 13

// Разница проявляется при сравнении
console.log(str1 === 'обычная строка'); // true

```

```
console.log(str2 === 'объект строка'); // false
console.log(str2.valueOf() === 'объект строка'); // true

// Полезные случаи использования конструктора
const fromNumber = String(42); // "42"
const fromBoolean = String(true); // "true"
const fromNull = String(null); // "null"
const fromUndefined = String(undefined); // "undefined"
const fromObject = String({}); // "[object Object]"
const fromArray = String([1, 2, 3]); // "1,2,3"

// Более предпочтительный способ - toString()
const num = 42;
console.log(num.toString()); // "42"
console.log((42).toString()); // "42"
```

Часть 2: Шаблонные строки (Template Literals)

2.1. Основы шаблонных строк

2.1.1. Синтаксис обратных кавычек

```
javascript

// Шаблонные строки используют обратные кавычки (`)
const templateLiteral = `Это шаблонная строка`;

// Многострочность без специальных символов
const multiline = `Это первая строка
Это вторая строка
Это третья строка`;

console.log(multiline);
// Вывод:
// Это первая строка
// Это вторая строка
// Это третья строка

// Интерполяция выражений
const name = 'Анна';
const age = 30;
const greeting = `Привет, меня зовут ${name} и мне ${age} лет`;
```

```

console.log(greeting); // "Привет, меня зовут Анна и мне 30 лет"

// Любые выражения внутри ${}
const a = 5;
const b = 10;
const calculation = `Сумма: ${a + b}, произведение: ${a * b}`;
console.log(calculation); // "Сумма: 15, произведение: 50"

// Вызовы функций
function getCurrentYear() {
    return new Date().getFullYear();
}

const yearMessage = `Сейчас ${getCurrentYear()} год`;
console.log(yearMessage);

// Условные выражения
const score = 85;
const result = `Ваш результат: ${score >= 60 ? 'Сдал' : 'Не сдал'}`;
console.log(result); // "Ваш результат: Сдал"

```

2.1.2. Особенности шаблонных строк

```

javascript

// Пробелы и переносы сохраняются
const formatted = `
    Заголовок
    • Пункт 1
    • Пункт 2
    • Пункт 3
`;

console.log(formatted);
// Вывод (с отступами!):
//
// Заголовок
//     • Пункт 1
//     • Пункт 2
//     • Пункт 3
//

// Escape-последовательности работают
const withEscapes = `Строка 1\nСтрока 2\tТабуляция`;
console.log(withEscapes);

```

```
// Обратные кавычки внутри шаблонных строк
const withBackticks = `Это обратная кавычка: \``;
console.log(withBackticks); // "Это обратная кавычка: `"

// Можно использовать другие кавычки без экранирования
const mixedQuotes = `Он сказал: "It's amazing!"`;
console.log(mixedQuotes); // Он сказал: "It's amazing!"
```

2.2. Вложенные шаблонные строки

```
javascript

// Шаблонные строки могут быть вложенными
const user = {
  firstName: 'Иван',
  lastName: 'Петров',
  address: {
    city: 'Москва',
    street: 'Тверская'
  }
};

const userInfo = `
  Пользователь: ${user.firstName} ${user.lastName}
  Адрес: ${`${user.address.city}, ${user.address.street}`}
`;

console.log(userInfo);

// Сложные вложенные выражения
const items = ['яблоко', 'банан', 'апельсин'];
const list = `
  Список покупок:
  ${items.map((item, index) => ` ${index + 1}. ${item}`)}.join('\n')}
`;

console.log(list);

// Вывод:
// Список покупок:
// 1. яблоко
// 2. банан
// 3. апельсин
```

```
// Шаблоны в условиях
function createGreeting(isFormal, name) {
  return `
    ${isFormal
      ? `Уважаемый ${name},`
      : `Привет, ${name}!`}

    ${isFormal
      ? `Благодарим Вас за обращение.`
      : `Рады тебя видеть!`}
  `;
}

console.log(createGreeting(true, 'Алексей'));
console.log(createGreeting(false, 'Алексей'));
```

2.3. Tagged Templates (Тегированные шаблоны)

2.3.1. Основы тегированных шаблонов

```
javascript

// Тегированный шаблон - функция, которая обрабатывает шаблонную строку
function tag(strings, ...values) {
  console.log('strings:', strings); // Массив строковых частей
  console.log('values:', values);   // Массив значений выражений

  let result = '';
  strings.forEach((string, i) => {
    result += string;
    if (i < values.length) {
      result += values[i];
    }
  });

  return result;
}

const name = 'Мир';
const greeting = tag`Привет, ${name}!`;
// strings: ["Привет, ", "!"]
```

```
// values: ["Мир"]
console.log(greeting); // "Привет, Мир!"

// Более сложный пример
const price = 100;
const discount = 20;
const total = tag`Цена: ${price} руб.\nСкидка: ${discount}%\nИтого: ${price * (1 - discount/100)} руб.`;

console.log(total);
```

2.3.2. Полезные применения тегированных шаблонов

```
javascript

// 1. Экранирование HTML (защита от XSS)
function htmlEscape(strings, ...values) {
  return strings.reduce((result, str, i) => {
    const value = values[i] != null ? values[i] : '';
    const escapedValue = String(value)
      .replace(/&/g, '&amp;')
      .replace(/</g, '&lt;')
      .replace(/>/g, '&gt;')
      .replace(/"/g, '&quot;')
      .replace(/'/g, '&#39;');

    return result + str + escapedValue;
  }, '');
}

const userInput = '<script>alert("XSS")</script>';
const safeHTML = htmlEscape`<div>${userInput}</div>`;
console.log(safeHTML); // <div>&lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;</div>

// 2. SQL запросы (очень упрощенно, не для продакшена!)
function sql(strings, ...values) {
  return strings.reduce((query, str, i) => {
    const value = values[i];

    if (value === undefined || value === null) {
      return query + str + 'NULL';
    }

    if (typeof value === 'number') {
      return query + str + value;
    }
  });
}
```

```

    }

    // Экранирование строк
    const escaped = String(value)
      .replace(/'/g, "'")
      .replace(/\\/g, '\\');

    return query + str + "'" + escaped + "'";
  }, '');
}

const userId = 123;
const userName = "O'Reilly";
const query = sql`SELECT * FROM users WHERE id = ${userId} AND name = ${userName}`;
console.log(query); // SELECT * FROM users WHERE id = 123 AND name = 'O'Reilly'

// 3. Интернационализация (i18n)
const translations = {
  en: {
    greeting: 'Hello, {name}!',
    farewell: 'Goodbye, {name}!'
  },
  ru: {
    greeting: 'Привет, {name}!',
    farewell: 'До свидания, {name}!'
  }
};

function i18n(strings, ...values) {
  const key = strings.join('{?}'); // Создаем ключ
  const template = translations[currentLocale][key];

  if (!template) return strings.join('');

  return template.replace(/\{(\w+)\}/g, (match, placeholder) => {
    const index = strings.raw.indexOf(`${placeholder}`);
    return index !== -1 ? values[index] : match;
  });
}

let currentLocale = 'ru';
const userName = 'Анна';
console.log(i18n`greeting`); // Нужна специальная подготовка переводов

```

2.3.3. Свойство raw

```
javascript

// strings.raw содержит строки без обработки escape-последовательностей
function showRaw(strings, ...values) {
  console.log('Обычные строки:', strings);
  console.log('Сырые строки:', strings.raw);

  return strings.raw.reduce((result, str, i) => {
    return result + str + (values[i] || '');
  }, '');
}

const path = showRaw`C:\Users\Name\Documents\note.txt`;
// Обычные строки: ["C:\Users\Name\Documents\note.txt"]
// Сырые строки: ["C:\\Users\\Name\\Documents\\note.txt"]
console.log(path);

// String.raw - встроенный тег для работы с сырыми строками
const rawString = String.raw`C:\Users\Name\Documents\note.txt`;
console.log(rawString); // "C:\Users\Name\Documents\note.txt"

const withNewline = String.raw`Первая строка\nВторая строка`;
console.log(withNewline); // "Первая строка\nВторая строка" (буквально)
console.log(withNewline.length); // 32 (включает \ и n)

// Полезно для регулярных выражений
const regex = String.raw`^\d{3}-\d{2}-\d{4}$`;
console.log(regex); // "^\d{3}-\d{2}-\d{4}$"
const pattern = new RegExp(regex);
console.log(pattern.test('123-45-6789')); // true
```

2.4. Практические применения шаблонных строк

2.4.1. Генерация HTML

```
javascript

// Генерация HTML элементов
function createCard(title, content, imageUrl) {
  return `
    <div class="card">
      ${imageUrl ? `` : ''}
    </div>
  `;
}
```



```

        <div class="card-content">
            <h3 class="card-title">${title}</h3>
            <p class="card-text">${content}</p>
            <button class="card-button">Подробнее</button>
        </div>
    </div>
    `;
}

// Генерация списка
function createList(items, ordered = false) {
    const tag = ordered ? 'ol' : 'ul';

    return `
        <${tag} class="custom-list">
            ${items.map(item => `
                <li class="list-item">
                    <span class="item-text">${item.text}</span>
                    ${item.subItems ? createList(item.subItems, ordered) : ''}
                </li>
            `).join('')}
        </${tag}>
    `;
}

// Пример использования
const menuItems = [
    { text: 'Главная', subItems: null },
    { text: 'Продукты', subItems: [
        { text: 'Веб-разработка', subItems: null },
        { text: 'Мобильные приложения', subItems: null }
    ]},
    { text: 'Контакты', subItems: null }
];

console.log(createList(menuItems));

// CSS-in-JS стили
function createStyledComponent(styles) {
    return `
        <style>
            .styled-component {
                ${Object.entries(styles)
                    .map(([property, value]) => `${property}: ${value};`)}
        </style>
    `;
}

```

```

        .join('\n        ')}
    }
</style>
<div class="styled-component">
    Стилизованный компонент
</div>
`;
}

```

```

const componentStyles = {
  'background-color': '#f0f0f0',
  'padding': '20px',
  'border-radius': '8px',
  'font-family': 'Arial, sans-serif'
};

```

```

console.log(createStyledComponent(componentStyles));

```

2.4.2. Шаблоны для логирования

```

javascript

// Форматированные сообщения логов
class Logger {
  static info(template, ...values) {
    const timestamp = new Date().toISOString();
    const message = String.raw(template, ...values);
    console.log(`[INFO] ${timestamp} - ${message}`);
  }

  static error(template, ...values) {
    const timestamp = new Date().toISOString();
    const message = String.raw(template, ...values);
    console.error(`[ERROR] ${timestamp} - ${message}`);
  }

  static sql(queryTemplate, ...params) {
    const query = String.raw(queryTemplate, ...params);
    console.log(`[SQL] ${query}`);
    console.log(`[PARAMS] ${JSON.stringify(params)}`);
  }
}

// Использование
const userId = 123;

```

```

const action = 'login';
Logger.info`Пользователь ${userId} выполнил действие: ${action}`;

try {
  // какой-то код
  throw new Error('Что-то пошло не так');
} catch (error) {
  Logger.error`Ошибка в модуле авторизации: ${error.message}`;
}

Logger.sql`SELECT * FROM users WHERE id = ${userId} AND active = ${true}`;

// Шаблоны для unit-тестов
function assertTemplate(condition, template, ...values) {
  if (!condition) {
    const message = String.raw(template, ...values);
    throw new AssertionError(message);
  }
}

class AssertionError extends Error {
  constructor(message) {
    super(message);
    this.name = 'AssertionError';
  }
}

// Пример использования
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
try {
  assertTemplate(
    result === 5,
    `Ожидалось 5, получено ${result}. Функция add(2, 3) работает некорректно.`
  );
  console.log('Тест пройден');
} catch (error) {
  console.error('Тест не пройден:', error.message);
}

```

2.4.3. Шаблоны для конфигурации

javascript

// Конфигурационные файлы как шаблоны

```
class ConfigTemplate {
  constructor(variables) {
    this.variables = variables;
  }

  // Шаблон конфигурации базы данных
  get databaseConfig() {
    const { host, port, username, password, database } = this.variables;

    return `
      database:
        host: ${host || 'localhost'}
        port: ${port || 5432}
        username: ${username}
        password: ${password}
        database: ${database}
        pool:
          max: 10
          min: 2
          idleTimeoutMillis: 30000

        ssl: ${this.variables.ssl ? 'true' : 'false'}
        ${this.variables.ssl ? `
          ssl:
            rejectUnauthorized: true
            ca: ${this.variables.ssl.ca || ''}
          : ''`
        : ''}
    `;
  }
}
```

// Шаблон конфигурации API

```
get apiConfig() {
  const { apiUrl, apiKey, timeout } = this.variables;

  return `
    api:
      baseUrl: ${apiUrl}
      endpoints:
        users: ${apiUrl}/users
  `;
}
```

```

        products: ${apiUrl}/products
        orders: ${apiUrl}/orders

headers:
    Authorization: Bearer ${apiKey}
    Content-Type: application/json

timeout: ${timeout || 30000}ms
retry:
    attempts: 3
    delay: 1000
`;
}

// Генерация полного конфига
generate() {
    return `
        # Автоматически сгенерированная конфигурация
        # Создано: ${new Date().toISOString()}

        ${this.databaseConfig}

        ${this.apiConfig}

logging:
    level: ${this.variables.logLevel || 'info'}
    format: json
    file: ${this.variables.logFile || 'app.log'}
`;
}
}

```

```

// Использование
const config = new ConfigTemplate({
    host: 'db.example.com',
    username: 'admin',
    password: 'secret123',
    database: 'myapp',
    ssl: true,
    ssl: {
        ca: '/path/to/ca.pem'
    },
    apiUrl: 'https://api.example.com',
    apiKey: 'abcdef123456',

```

```
    logLevel: 'debug'
  });

console.log(config.generate());

// Шаблоны для Dockerfile
function generateDockerfile(config) {
  const {
    baseImage = 'node:18-alpine',
    workingDir = '/app',
    port = 3000,
    dependencies = [],
    devDependencies = [],
    buildCommand = 'npm run build',
    startCommand = 'npm start'
  } = config;

  return `
    # Используем официальный образ Node.js
    FROM ${baseImage}

    # Устанавливаем рабочую директорию
    WORKDIR ${workingDir}

    # Копируем файлы зависимостей
    COPY package*.json ./

    # Устанавливаем зависимости
    RUN npm install
    ${devDependencies.length > 0 ? `
    # Устанавливаем dev-зависимости
    RUN npm install ${devDependencies.join(' ')} --only=development
    ` : ''}

    # Копируем исходный код
    COPY . .

    # Собираем приложение
    RUN ${buildCommand}

    # Открываем порт
    EXPOSE ${port}

    # Запускаем приложение
```

```

    CMD ${startCommand}
  `;
}

const dockerConfig = {
  baseImage: 'node:18-alpine',
  port: 8080,
  dependencies: ['express', 'mongoose', 'dotenv'],
  devDependencies: ['nodemon', 'jest', 'eslint'],
  buildCommand: 'npm run build:prod',
  startCommand: 'node server.js'
};

console.log(generateDockerfile(dockerConfig));

```

Часть 3: Производительность и оптимизация

3.1. Сравнение производительности

```

javascript

// Тест производительности разных способов создания строк
function performanceTest() {
  const iterations = 1000000;

  console.time('Конкатенация +');
  let result1 = '';
  for (let i = 0; i < iterations; i++) {
    result1 = 'Текст ' + i + ' из ' + iterations;
  }
  console.timeEnd('Конкатенация +');

  console.time('Конкатенация +=');
  let result2 = '';
  for (let i = 0; i < iterations; i++) {
    result2 += 'Текст ' + i + ' из ' + iterations;
  }
  console.timeEnd('Конкатенация +=');

  console.time('Array join');
  const parts2 = [];
  for (let i = 0; i < iterations; i++) {

```

```

    parts2.push('Текст ', i, ' из ', iterations);
  }
  const result3 = parts2.join('');
  console.timeEnd('Array join');

  console.time('Шаблонная строка');
  let result4 = '';
  for (let i = 0; i < iterations; i++) {
    result4 = `Текст ${i} из ${iterations}`;
  }
  console.timeEnd('Шаблонная строка');

  console.time('String.concat');
  let result5 = '';
  for (let i = 0; i < iterations; i++) {
    result5 = String.prototype.concat('Текст ', i, ' из ', iterations);
  }
  console.timeEnd('String.concat');
}

// Выполняем тест
performanceTest();

// Результаты (примерные):
// Конкатенация +: ~50ms
// Конкатенация +=: ~120ms (медленнее из-за создания новых строк)
// Array join: ~90ms
// Шаблонная строка: ~55ms
// String.concat: ~70ms

```

3.2. Оптимизация работы со строками

```

javascript

// 1. Кэширование шаблонных строк
class TemplateCache {
  constructor() {
    this.cache = new Map();
  }

  get(key, templateFn, ...args) {
    if (!this.cache.has(key)) {
      this.cache.set(key, templateFn(...args));
    }
  }
}

```



```

    return this.cache.get(key);
}

// Для шаблонов с параметрами
createCachedTemplate(templateFn) {
    const cache = new Map();

    return (...args) => {
        const key = JSON.stringify(args);

        if (!cache.has(key)) {
            cache.set(key, templateFn(...args));
        }

        return cache.get(key);
    };
}

// Пример использования
const cache = new TemplateCache();

function createUserCard(name, age) {
    return `
        <div class="user-card">
            <h3>${name}</h3>
            <p>Возраст: ${age}</p>
        </div>
    `;
}

const cachedTemplate = cache.createCachedTemplate(createUserCard);

// Первый вызов - создание и кэширование
console.log(cachedTemplate('Анна', 30));
// Второй вызов - из кэша
console.log(cachedTemplate('Анна', 30));

// 2. Оптимизация больших строк с помощью String Builder паттерна
class StringBuilder {
    constructor() {
        this.parts = [];
        this.length = 0;
    }
}

```

```

append(str) {
    this.parts.push(str);
    this.length += String(str).length;
    return this;
}

appendLine(str = '') {
    return this.append(str + '\n');
}

clear() {
    this.parts = [];
    this.length = 0;
    return this;
}

toString() {
    return this.parts.join('');
}

// Специальные методы для шаблонных строк
appendTemplate(template, ...values) {
    return this.append(String.raw(template, ...values));
}

// Пакетное добавление
appendAll(strings) {
    strings.forEach(str => this.append(str));
    return this;
}

}

// Пример использования
const builder = new StringBuilder();

builder
    .appendLine('<html>')
    .appendLine('  <head>')
    .appendTemplate`    <title>${'Мой сайт'}</title>`
    .appendLine('  </head>')
    .appendLine('  <body>')
    .appendLine('    <h1>Приветствие</h1>');

```

```

for (let i = 0; i < 1000; i++) {
  builder.appendLine(`    <p>Параграф ${i}</p>`);
}

builder.appendLine('  </body>');
builder.appendLine('</html>');

const html = builder.toString();
console.log(`Длина HTML: ${html.length} символов`);

// 3. Ленивая генерация строк
function* lazyStringGenerator(template, data) {
  const parts = template.split(/(\${\w+})/g);

  for (const part of parts) {
    const match = part.match(/(\${\w+})/);

    if (match) {
      const key = match[1];
      yield data[key] !== undefined ? data[key] : part;
    } else {
      yield part;
    }
  }
}

// Пример использования
const userTemplate = 'Привет, ${name}! Тебе ${age} лет. Твой email: ${email}';
const userData = {
  name: 'Анна',
  age: 30
  // email отсутствует
};

let result = '';
for (const part of lazyStringGenerator(userTemplate, userData)) {
  result += part;
}

console.log(result); // "Привет, Анна! Тебе 30 лет. Твой email: ${email}"

```

3.3. Память и строки в JavaScript

javascript

// Строки в JavaScript иммутабельны

```
let str = 'Hello';
console.log(str[0]); // 'H'
str[0] = 'J'; // Не работает
console.log(str); // 'Hello'
```

// Каждая операция создает новую строку

```
let original = 'text';
let modified = original.toUpperCase(); // Создается новая строка
console.log(original); // 'text' (не изменилась)
console.log(modified); // 'TEXT'
```

// Интернирование строк (string interning)

```
const a = 'Hello World';
const b = 'Hello World';
const c = 'Hello' + ' ' + 'World';

console.log(a === b); // true (одинаковые литералы)
console.log(a === c); // true (результат такой же)
console.log(a === 'Hello World'); // true
```

// Но:

```
const d = new String('Hello World');
console.log(a === d); // false (объект String)
console.log(a === d.valueOf()); // true
```

// Проблема памяти при конкатенации в цикле

```
function inefficientConcat(n) {
  let result = '';
  for (let i = 0; i < n; i++) {
    result += 'x'; // Создает новую строку на каждой итерации
  }
  return result;
}
```

```
function efficientConcat(n) {
  const builder = [];
  for (let i = 0; i < n; i++) {
    builder.push('x');
  }
}
```

```

    return builder.join('');
}

console.time('Неэффективная конкатенация');
inefficientConcat(100000);
console.timeEnd('Неэффективная конкатенация');

console.time('Эффективная конкатенация');
efficientConcat(100000);
console.timeEnd('Эффективная конкатенация');

```

Часть 4: Особенности и граничные случаи

4.1. Unicode и строки

```

javascript

// Длина строки и Unicode
const str1 = 'Hello';
const str2 = 'Привет';
const str3 = '😄👍';
const str4 = 'café';
const str5 = 'cafe\u0301'; // e + acute accent

console.log(str1.length); // 5
console.log(str2.length); // 6 (кириллические символы)
console.log(str3.length); // 4 (эмоджи как суррогатные пары)
console.log(str4.length); // 4
console.log(str5.length); // 5 (комбинированный символ)

// Итерация по Unicode символам
function countUnicodeChars(str) {
    let count = 0;
    for (const char of str) {
        count++;
    }
    return count;
}

console.log('Длина через .length:', str3.length); // 4
console.log('Реальное количество символов:', countUnicodeChars(str3)); // 2

```

```

// Нормализация Unicode
console.log(str4 === str5); // false
console.log(str4.normalize() === str5.normalize()); // true (после нормализации)

const normalized1 = str4.normalize('NFC'); // Композиция (default)
const normalized2 = str4.normalize('NFD'); // Декомпозиция
console.log(normalized1); // 'café'
console.log(normalized2); // 'cafe\u0301'

// Эмоji последовательности
const emojis = {
  single: '😄', // Один символ
  flag: 'RU', // Последовательность региональных индикаторов
  family: '👨👩👦', // Последовательность с ZWJ
  skinTone: '🏠', // Модификатор тона кожи
  keycap: '🏠' // Ключ-эмоджи
};

console.log('Флаг длина:', emojis.flag.length); // 4
console.log('Семья длина:', emojis.family.length); // 11

// Правильный подсчет символов с учетом Unicode
function getGraphemeCount(str) {
  // Используем Intl.Segmenter для правильной сегментации
  if (Intl.Segmenter) {
    const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
    return [...segmenter.segment(str)].length;
  }

  // Fallback для старых браузеров
  return [...str].length;
}

console.log('Графемы в 😄🏠:', getGraphemeCount('😄🏠 ')); // 2
console.log('Графемы в 👨👩👦:', getGraphemeCount('👨👩👦')); // 1

```

4.2. Строки и регулярные выражения

javascript

```

// Шаблонные строки и регулярные выражения
const name = 'file.txt';
const regex1 = new RegExp(`^${name}$`); // Опасно! name может содержать спец. символы

```

```

// Экранирование для регулярных выражений
function escapeRegExp(string) {
    return string.replace(/[\.*+?^${}()|[\]\]/g, '\\$&');
}

const safeName = escapeRegExp(name);
const regex2 = new RegExp(`^${safeName}$`);
console.log(regex2.test('file.txt')); // true

// Шаблонные строки для динамических регулярных выражений
function createSearchRegex(searchTerm, flags = 'i') {
    const escaped = escapeRegExp(searchTerm);
    return new RegExp(`\\b${escaped}\\b`, flags);
}

const search = createSearchRegex('test');
console.log(search.test('This is a test. ')); // true
console.log(search.test('Testing')); // false (целое слово)

// Многострочные регулярные выражения
const multilineText = `Первая строка
Вторая строка
Третья строка`;

const regex3 = /^Вторая/m; // Флаг m для многострочного режима
console.log(regex3.test(multilineText)); // true

// С использованием шаблонных строк
const pattern = 'Вторая';
const regex4 = new RegExp(`^${pattern}`, 'm');
console.log(regex4.test(multilineText)); // true

// Шаблоны для сложных регулярных выражений
function createEmailRegex(domainWhitelist = []) {
    const localPart = '[a-zA-Z0-9.!#$%&\'*/+=?^_`{|}~-]+';
    const domain = domainWhitelist.length > 0
        ? `(${domainWhitelist.map(d => escapeRegExp(d)).join('|')})`
        : '[a-zA-Z0-9-]+(?:\\.[a-zA-Z0-9-]+)*';

    return new RegExp(`^${localPart}@${domain}$`);
}

const emailRegex = createEmailRegex(['example.com', 'test.org']);
console.log(emailRegex.test('user@example.com')); // true

```

```
console.log(emailRegex.test('user@gmail.com')); // false
```

4.3. Строки и безопасность

javascript

// Защита от XSS при вставке в HTML

```
class SafeHTML {
  static escape(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
  }

  static createElement(tag, attributes = {}, content = '') {
    const attrs = Object.entries(attributes)
      .map(([key, value]) => ` ${key}="${this.escape(value)}"`)
      .join(' ');

    return `<${tag}${attrs}>${this.escape(content)}</${tag}>`;
  }

  static createTemplate(template, ...values) {
    return String.raw({ raw: template }, ...values.map(v => this.escape(v)));
  }
}
```

// Пример использования

```
const userInput = '<script>alert("XSS")</script>';
const safeDiv = SafeHTML.createElement('div', {
  class: 'container',
  id: 'user-content'
}, userInput);
```

```
console.log(safeDiv);
```

// <div class="container"

id="user-content"><script>alert("XSS")</script></div>

// Использование с шаблонными строками

```
const userName = '<img src=x onerror=alert(1)>';
const safeMessage = SafeHTML.createTemplate`Привет, ${userName}!`;
console.log(safeMessage); // "Привет, &lt;img src=x onerror=alert(1)&gt;!"
```

// Защита от SQL инъекций


```

class SafeSQL {
  static escape(value) {
    if (value === null || value === undefined) {
      return 'NULL';
    }

    if (typeof value === 'number' || typeof value === 'boolean') {
      return String(value);
    }

    // Экранирование строк
    return "'" + String(value)
      .replace(/'/g, "'")
      .replace(/\\/g, '\\\\') + "'";
  }

  static query(template, ...values) {
    const escaped = values.map(v => this.escape(v));
    return String.raw({ raw: template }, ...escaped);
  }
}

// Пример
const userId = "1"; DROP TABLE users; --";
const safeQuery = SafeSQL.query`SELECT * FROM users WHERE id = ${userId}`;
console.log(safeQuery); // SELECT * FROM users WHERE id = '1'; DROP TABLE users; --'

// CSRF токены в шаблонах
function generateCSRFToken() {
  return Array.from(crypto.getRandomValues(new Uint8Array(32)))
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');
}

function createFormWithCSRF(action, method = 'POST') {
  const token = generateCSRFToken();

  return `
    <form action="${SafeHTML.escape(action)}" method="${method}">
      <input type="hidden" name="csrf_token" value="${token}">
      <!-- Остальные поля формы -->
      <input type="text" name="username" placeholder="Имя пользователя">
      <button type="submit">Отправить</button>
    </form>
  `;
}

```

```

    <script>
        // Сохраняем токен для AJAX запросов
        window.csrfToken = '${token}';
    </script>
`;
}

console.log(createFormWithCSRF('/submit', 'POST'));

```

Часть 5: Расширенные возможности и будущее

5.1. Новые методы строк в современных стандартах

```

javascript

// ES6+ методы строк
const text = 'Hello World, Welcome to JavaScript!';

// includes() - проверка наличия подстроки
console.log(text.includes('World')); // true
console.log(text.includes('world')); // false (регистрозависимо)

// startsWith() и endsWith()
console.log(text.startsWith('Hello')); // true
console.log(text.endsWith('!')); // true
console.log(text.startsWith('World', 6)); // true (поиск с позиции 6)

// repeat() - повторение строки
console.log('Ha'.repeat(3)); // 'HaHaHa'
console.log('*'.repeat(10)); // '*****'

// padStart() и padEnd() - заполнение строки
console.log('5'.padStart(3, '0')); // '005'
console.log('5'.padEnd(3, '0')); // '500'
console.log('hello'.padStart(10, '123')); // '12312heLlo'

// ES2017: padStart/padEnd для выравнивания
const items = [
    { name: 'Apple', price: 2.5 },
    { name: 'Banana', price: 1.75 },
    { name: 'Orange', price: 3 }
]

```

```

];

console.log('Товары:');
items.forEach(item => {
    const name = item.name.padEnd(10, '.');
    const price = item.price.toFixed(2).padStart(6, ' ');
    console.log(`${name} ${price} руб.`);
});

// ES2019: trimStart() u trimEnd()
const padded = '  текст  ';
console.log(`${padded.trim()} `); // 'текст'
console.log(`${padded.trimStart()} `); // 'текст  '
console.log(`${padded.trimEnd()} `); // '  текст'

// ES2020: matchAll() - все совпадения регулярного выражения
const html = '<div>Hello</div><span>World</span><div>Test</div>';
const regex = /<(\w+)>(.*?)<\/\w+>/g;

for (const match of html.matchAll(regex)) {
    console.log(`Тег: ${match[1]}, содержимое: ${match[2]} `);
}

// ES2022: at() - доступ к символам по индексу (включая отрицательные)
const str = 'Hello';
console.log(str.at(0)); // 'H' (как str[0])
console.log(str.at(-1)); // 'o' (последний символ)
console.log(str.at(-2)); // 'l' (предпоследний)

```

5.2. Предложения для будущих стандартов

```

javascript

// Предложение: String.prototype.replaceAll (уже в ES2021)
const text = 'apple, banana, apple, orange';
console.log(text.replaceAll('apple', 'pear')); // 'pear, banana, pear, orange'

// Предложение: String.prototype.isWellFormed / toWellFormed (ES2024)
const malformed = 'Hello\uD800World'; // Непарная суррогатная пара

if (typeof malformed.isWellFormed === 'function') {
    console.log(malformed.isWellFormed()); // false
    console.log(malformed.toWellFormed()); // 'Hello💀World'
}

```

```

// Пользовательская реализация для старых браузеров
if (!String.prototype.isWellFormed) {
    String.prototype.isWellFormed = function() {
        // Проверка на непарные суррогатные пары
        for (let i = 0; i < this.length; i++) {
            const code = this.charCodeAt(i);

            if (0xD800 <= code && code <= 0xDBFF) {
                // Начало суррогатной пары
                if (i === this.length - 1) return false;
                const next = this.charCodeAt(i + 1);
                if (!(0xDC00 <= next && next <= 0xDFFF)) return false;
                i++; // Пропускаем второй символ пары
            } else if (0xDC00 <= code && code <= 0xDFFF) {
                // Конец суррогатной пары без начала
                return false;
            }
        }

        return true;
    };
}

// Предложение: шаблонные строки с тегам для ICU MessageFormat
function formatMessage(strings, ...values) {
    // Упрощенная реализация форматирования сообщений
    return strings.reduce((result, str, i) => {
        const value = values[i];

        if (typeof value === 'object' && value !== null) {
            // Форматирование с учетом плюральных форм
            const { count, other, one, few, many } = value;

            if (count === 1 && one) return result + str + one;
            if (count >= 2 && count <= 4 && few) return result + str + few;
            if (count >= 5 && many) return result + str + many;
            return result + str + other;
        }

        return result + str + (value || '');
    }, '');
}

```

// Пример использования

```
const messages = {
  apples: {
    one: 'яблоко',
    few: 'яблока',
    many: 'яблок',
    other: 'яблок'
  }
};

const count = 3;
const message = formatMessage`У меня есть ${count} ${messages.apples}`;
console.log(message); // "У меня есть 3 яблока"
```

5.3. Интеграция с другими языковыми возможностями

javascript

// Строки и декораторы (предложение для ES)

```
function measurePerformance(target, name, descriptor) {
  const original = descriptor.value;

  descriptor.value = function(...args) {
    console.time(name);
    const result = original.apply(this, args);
    console.timeEnd(name);
    return result;
  };

  return descriptor;
}

class StringProcessor {
  @measurePerformance
  static processTemplate(template, data) {
    return template.replace(/\${(\w+)}/g, (match, key) => {
      return data[key] !== undefined ? data[key] : match;
    });
  }

  @measurePerformance
  static processTaggedTemplate(strings, ...values) {
    return strings.reduce((result, str, i) => {
      return result + str + (values[i] || '');
    });
  }
}
```

```

    }, '');
  }
}

// Строки и private поля
class SecureTemplate {
  #secretKey = 'supersecret';

  createSecureTemplate(template, data) {
    // Шифрование данных с использованием приватного ключа
    const encryptedData = this.#encryptData(data);
    return template.replace('{{encrypted}}', encryptedData);
  }

  #encryptData(data) {
    // Упрощенное "шифрование" для примера
    return btoa(JSON.stringify(data) + this.#secretKey);
  }
}

```

```

// Строки и Proxy
const stringProxy = new Proxy(String.prototype, {
  get(target, prop) {
    if (prop === 'double') {
      return function() {
        return this + this;
      };
    }

    if (prop === 'shout') {
      return function() {
        return this.toUpperCase() + '!';
      };
    }

    return target[prop];
  }
});

```

```

// Использование (требуется установка prototype)
Object.setPrototypeOf('test', stringProxy);
console.log('hello'.double()); // 'hellohello'
console.log('hello'.shout()); // 'HELLO!'

```

```
// Строки и WeakRef (ES2021)
function createCachedTemplate(templateFn) {
  const cache = new WeakMap();

  return (...args) => {
    const key = args[0]; // Предполагаем, что первый аргумент - ключ

    let cached = cache.get(key);
    if (cached && cached.deref()) {
      return cached.deref();
    }

    const result = templateFn(...args);
    cache.set(key, new WeakRef(result));

    return result;
  };
}

// Пример использования с DOM элементами
const createElementTemplate = createCachedTemplate((element, content) => {
  console.log('Создаем новый элемент');
  return `<${element}>${content}</${element}>`;
});

const div1 = document.createElement('div');
console.log(createElementTemplate(div1, 'Привет')); // Создаем новый элемент
console.log(createElementTemplate(div1, 'Привет')); // Из кэша

// Когда div1 удаляется сборщиком мусора, кэш автоматически очищается
```

Заключение

Ключевые выводы:

1. **Строковые литералы** — фундаментальный тип данных с двумя формами (одинарные и двойные кавычки), требующими правильного экранирования.
2. **Шаблонные строки (ES6)** — революционное улучшение:
 - ➔ Многострочность без `\n`
 - ➔ Интерполяция выражений через `${}`
 - ➔ Тегированные шаблоны для продвинутой обработки

- ➔ Сохранение форматирования и пробелов

3. **Производительность** — выбор правильного метода конкатенации критичен для больших объемов данных.

Рекомендации по использованию:

1. **Всегда используйте шаблонные строки** для многострочного текста и интерполяции.

2. **Применяйте тегированные шаблоны** для:

- ➔ Экранирования HTML/SQL
- ➔ Интернационализации
- ➔ Синтаксического выделения

3. **Оптимизируйте конкатенацию:**

- ➔ Для циклического наращивания используйте массив + `join()`
- ➔ Кэшируйте часто используемые шаблоны
- ➔ Используйте `StringBuilder` паттерн для очень больших строк

4. **Учитывайте Unicode:**

- ➔ Используйте `for...of` для итерации по символам
- ➔ Применяйте нормализацию для сравнения строк
- ➔ Используйте `Intl.Segmenter` для правильной сегментации

5. **Обеспечивайте безопасность:**

- ➔ Всегда экранируйте пользовательский ввод
- ➔ Используйте тегированные шаблоны для автоматического экранирования
- ➔ Валидируйте и санитизируйте данные

Будущее строк в JavaScript:

1. **Улучшенная поддержка Unicode** — методы для работы с графемами
2. **Дополнительные методы** — `replaceAll`, `isWellFormed`
3. **Интеграция с другими возможностями** — декораторы, `private` поля, `Proxy`
4. **Статические анализаторы** — проверка шаблонных строк на этапе компиляции

Строки в JavaScript прошли долгий путь от простых литералов до мощных шаблонных строк. Понимание их особенностей, возможностей и лучших практик использования — ключ к написанию эффективного, безопасного и поддерживаемого кода.

10.2. Методы работы со строками

Строки в JavaScript — это неизменяемые последовательности символов, представленные примитивным типом `String`. JavaScript предоставляет богатый набор встроенных методов для работы со строками, которые позволяют выполнять операции поиска, замены, преобразования, извлечения подстрок и многое другое. Понимание и умелое использование этих методов является ключевым навыком для эффективной работы с текстовыми данными.

1. Основные свойства строк

Перед изучением методов важно понять два ключевых свойства строк:

Длина строки:

```
javascript

const text = "Hello, World!";
console.log(text.length); // 13
```

Свойство `length` возвращает количество символов в строке, включая пробелы и знаки пунктуации.

Доступ к символам:

```
javascript

const str = "JavaScript";
console.log(str[0]); // "J" - доступ по индексу (ES5+)
console.log(str.charAt(0)); // "J" - метод charAt
console.log(str.at(-1)); // "t" - метод at (ES2022), доступ с конца
```

Важно: строки в JavaScript **иммутабельны** (неизменяемы). Любой метод, "изменяющий" строку, на самом деле возвращает **новую строку**:

```
javascript

let original = "hello";
let modified = original.toUpperCase();
console.log(original); // "hello" - не изменилась
console.log(modified); // "HELLO" - новая строка
```

2. Методы поиска и проверки

indexOf() и lastIndexOf()

Поиск подстроки в строке. Возвращают индекс первого (или последнего) вхождения или -1, если подстрока не найдена.

```
javascript

const sentence = "JavaScript is awesome. JavaScript is powerful.";

// Поиск с начала
console.log(sentence.indexOf("JavaScript")); // 0
console.log(sentence.indexOf("Python"));     // -1 (не найдено)

// Поиск с указанной позиции
console.log(sentence.indexOf("JavaScript", 10)); // 20

// Поиск с конца
console.log(sentence.lastIndexOf("JavaScript")); // 20
console.log(sentence.lastIndexOf("is"));         // 34

// Практическое использование: проверка наличия подстроки
const email = "user@example.com";
if (email.indexOf("@") !== -1) {
    console.log("Email содержит символ @");
}

// Более современная альтернатива - includes()
console.log(email.includes("@")); // true
```

includes(), startsWith(), endsWith() (ES6)

Более читаемые методы для проверки содержимого строк.

```
javascript

const filename = "document.pdf";

// Проверка наличия подстроки
console.log(filename.includes(".pdf")); // true
console.log(filename.includes(".doc")); // false

// Проверка начала строки
```

```

console.log(filename.startsWith("doc")); // true
console.log(filename.startsWith("file")); // false

// Проверка конца строки
console.log(filename.endsWith(".pdf")); // true
console.log(filename.endsWith(".txt")); // false

// С позицией поиска
const url = "https://example.com/page";
console.log(url.startsWith("https")); // true
console.log(url.startsWith("example", 8)); // true (начиная с позиции 8)

```

3. Методы извлечения подстрок

slice()

Извлекает часть строки и возвращает новую строку. Принимает начальный и конечный индексы.

```

javascript

const text = "Hello, JavaScript World!";

// Базовое использование
console.log(text.slice(7)); // "JavaScript World!" (с 7 до конца)
console.log(text.slice(7, 17)); // "JavaScript" (с 7 по 16)

// Отрицательные индексы (отсчет с конца)
console.log(text.slice(-6)); // "World!" (последние 6 символов)
console.log(text.slice(7, -7)); // "JavaScript" (с 7 до 7 с конца)

// Когда start > end - возвращает пустую строку
console.log(text.slice(10, 5)); // ""

// Практический пример: извлечение домена из email
const email = "user@example.com";
const atIndex = email.indexOf("@");
const domain = email.slice(atIndex + 1);
console.log(domain); // "example.com"

```

substring() и substr()

Альтернативные методы извлечения подстрок.

```
javascript

const str = "Hello, World!";

// substring(start, end) - похож на slice, но корректирует отрицательные значения
console.log(str.substring(7, 12)); // "World"
console.log(str.substring(12, 7)); // "World" (автоматически меняет местами)
console.log(str.substring(-5, 5)); // "Hello" (отрицательные → 0)

// substr(start, length) - устаревший, избегайте использования
console.log(str.substr(7, 5)); // "World"
console.log(str.substr(-6, 6)); // "World!" (с конца)

// Рекомендация: используйте slice() вместо substring() и substr()
```

4. Методы преобразования регистра

```
javascript

const mixedCase = "JaVaScRiPt ПрОгРамМиРоВаНиЕ";

// Преобразование в нижний регистр
console.log(mixedCase.toLowerCase()); // "javascript программирование"
console.log(mixedCase.toLocaleLowerCase('tr')); // для турецкого языка

// Преобразование в верхний регистр
console.log(mixedCase.toUpperCase()); // "JAVASCRIPT ПРОГРАММИРОВАНИЕ"

// Преобразование только первого символа
function capitalize(str) {
    return str.charAt(0).toUpperCase() + str.slice(1).toLowerCase();
}
console.log(capitalize("hello WORLD")); // "Hello world"

// Практический пример: нормализация пользовательского ввода
const userInput = " UsEr@ExAmPlE.CoM ";
const normalizedEmail = userInput.trim().toLowerCase();
console.log(normalizedEmail); // "user@example.com"
```

5. Методы работы с пробелами

trim(), trimStart(), trimEnd() (ES2019)

Удаляют пробельные символы с начала и/или конца строки.

```
javascript

const padded = "  Hello World!  \n\t";

// Удаление пробелов с обеих сторон
console.log(padded.trim());           // "Hello World!"
console.log(`${padded.trim()} "`);    // "Hello World!"

// Удаление только с начала
console.log(padded.trimStart());       // "Hello World!  \n\t"

// Удаление только с конца
console.log(padded.trimEnd());         // "  Hello World!"

// Что считается пробельным символом: пробел, табуляция, перевод строки
const text = "\t\n Text \u00A0"; // \u00A0 - неразрывный пробел
console.log(text.trim()); // "Text" (обычный trim не удаляет \u00A0)

// Для всех пробельных символов Unicode
console.log(text.replace(/^\s+|\s+$/g, '')); // "Text"

// Практический пример: очистка поискового запроса
const searchQuery = " javascript tutorial ";
const cleanQuery = searchQuery.trim().toLowerCase();
console.log(cleanQuery); // "javascript tutorial"
```

6. Методы замены и разделения

replace() и replaceAll() (ES2021)

Замена подстрок в строке.

```
javascript

const text = "I love cats. Cats are great pets. Cats are cute.";
```

```

// Замена первого вхождения (регистрозависимо)
console.log(text.replace("Cats", "Dogs"));
// "I love cats. Dogs are great pets. Cats are cute."

// Замена всех вхождений (до ES2021)
console.log(text.replace(/Cats/g, "Dogs"));
// "I love cats. Dogs are great pets. Dogs are cute."

// Замена всех вхождений (ES2021+)
console.log(text.replaceAll("Cats", "Dogs"));
// "I love cats. Dogs are great pets. Dogs are cute."

// Без учета регистра
console.log(text.replace(/cats/gi, "dogs"));
// "I love dogs. Dogs are great pets. Dogs are cute."

// Использование функции-заменителя
const result = text.replace(/cats/gi, (match) => {
    return match.toUpperCase();
});
console.log(result); // "I love CATS. CATS are great pets. CATS are cute."

// Практический пример: маскирование данных
const phone = "+7 (999) 123-45-67";
const masked = phone.replace(/\d/g, "*");
console.log(masked); // "+* (***) ***-**-**"

```

split()

Разделение строки на массив подстрок по указанному разделителю.

```

javascript

const csv = "apple,banana,orange,grape";
const tags = "js;html;css;react";

// Разделение по запятой
console.log(csv.split(",")); // ["apple", "banana", "orange", "grape"]

// Разделение по точке с запятой
console.log(tags.split(";")); // ["js", "html", "css", "react"]

// Ограничение количества элементов
console.log(csv.split(",", 2)); // ["apple", "banana"]

```

```

// Разделение по регулярному выражению
const sentence = "Hello World! How are you?";
console.log(sentence.split(/\s+/)); // ["Hello", "World!", "How", "are", "you?"]

// Разделение по каждому символу
console.log("Hello".split("")); // ["H", "e", "l", "l", "o"]

// Разделение с сохранением разделителя
console.log("a,b,c".split(/(,)/)); // ["a", ",", "b", ",", "c"]

// Практический пример: парсинг query string
const query = "name=John&age=30&city=NYC";
const params = query.split("&").reduce((acc, pair) => {
  const [key, value] = pair.split("=");
  acc[key] = decodeURIComponent(value);
  return acc;
}, {});
console.log(params); // {name: "John", age: "30", city: "NYC"}

```

7. Методы повторения и заполнения

repeat() (ES6)

Повторение строки указанное количество раз.

```

javascript

// Базовое использование
console.log("Ha".repeat(3)); // "HaHaHa"
console.log("*".repeat(10)); // "*****"

// Нулевое или отрицательное количество
console.log("text".repeat(0)); // "" (пустая строка)
console.log("text".repeat(2.5)); // "texttext" (дробное округляется)

// Практическое использование
function createIndent(level) {
  return " ".repeat(level);
}
console.log(createIndent(3) + "Item"); // "   Item"

// Создание рамки

```

```
function createBox(text, padding = 2) {
    const horizontal = "-".repeat(text.length + padding * 2);
    return `
    ${horizontal}
    |${" ".repeat(padding)}${text}${" ".repeat(padding)}|
    ${horizontal}
    `
    .trim();
}
console.log(createBox("Hello"));
```

padStart() и padEnd() (ES2017)

Дополнение строки до заданной длины.

```
javascript

const number = "42";

// Дополнение в начале
console.log(number.padStart(5, "0")); // "00042"
console.log(number.padStart(6, "*")); // "****42"

// Дополнение в конце
console.log(number.padEnd(5, "0")); // "42000"
console.log(number.padEnd(6, "!")); // "42!!!!"

// Если исходная строка длиннее требуемой
console.log("longtext".padStart(5, "*")); // "Longtext"

// Практическое использование: форматирование чисел
function formatTime(hours, minutes) {
    return `${hours.toString().padStart(2, '0')}:${minutes.toString().padStart(2, '0')}`;
}
console.log(formatTime(9, 5)); // "09:05"
console.log(formatTime(23, 7)); // "23:07"

// Выравнивание текста в колонках
const products = [
    { name: "Apple", price: 1.99 },
    { name: "Banana", price: 0.99 },
    { name: "Strawberry", price: 3.49 }
];

products.forEach(product => {
```



```

    const name = product.name.padEnd(15, ".");
    const price = `${product.price.toFixed(2)}.padStart(10);
    console.log(`${name}${price}`);
  });
// Apple..... $1.99
// Banana..... $0.99
// Strawberry..... $3.49

```

8. Методы сравнения и локализации

localeCompare()

Сравнение строк с учетом локали для корректной сортировки.

```

javascript

const words = ["apple", "zebra", "apple", "Ångström", "banana"];

// Простое сравнение (некорректно для некоторых языков)
console.log(words.sort());
// ["Ångström", "apple", "banana", "zebra", "apple"] - неправильно!

// Сравнение с учетом локали
console.log(words.sort((a, b) => a.localeCompare(b, 'en')));
// ["apple", "apple", "banana", "zebra", "Ångström"] - лучше

// С русской локалью
const russianWords = ["яблоко", "ёжик", "абрикос", "электростанция"];
console.log(russianWords.sort((a, b) => a.localeCompare(b, 'ru')));
// ["абрикос", "ёжик", "электростанция", "яблоко"] - правильно!

// Опции сравнения
console.log("Groß".localeCompare("gross", "de", {
  sensitivity: 'accent'
})); // 0 (считаются равными в немецком)

// Практический пример: сортировка пользователей
const users = [
  { name: "André", country: "France" },
  { name: "Andrea", country: "Italy" },
  { name: "Andreas", country: "Germany" },
  { name: "Андрей", country: "Russia" }
];

```

```
const sortedUsers = users.sort((a, b) =>
  a.name.localeCompare(b.name, 'en', { sensitivity: 'base' })
);

console.log(sortedUsers.map(u => u.name));
// ["André", "Andrea", "Andreas", "Андрей"] - корректная сортировка
```

9. Работа с Unicode и кодовыми точками

codePointAt(), fromCodePoint() (ES6)

Работа с символами за пределами Basic Multilingual Plane (BMP).

```
javascript

// Стандартные символы
console.log("A".codePointAt(0)); // 65
console.log(String.fromCodePoint(65)); // "A"

// Символы из расширенного Unicode (эмодзи, иероглифы)
const emoji = "😄";
console.log(emoji.length); // 2 (суррогатные пары)
console.log(emoji.codePointAt(0)); // 128512 (полный код)
console.log(String.fromCodePoint(128512)); // "😄"

// Итерация по символам (правильный способ)
const text = "Hello😄World";
console.log(text.length); // 11 (но визуально 10 символов)

// Неправильно: for или for...of со старой кодировкой
for (let i = 0; i < text.length; i++) {
  console.log(text[i]); // H, e, l, l, o, ?, ?, W, o, r, l, d
}

// Правильно: использование spread или Array.from
console.log([...text]); // ["H", "e", "l", "l", "o", "😄", "W", "o", "r", "l", "d"]

// Практический пример: подсчет символов (не байтов)
function countSymbols(str) {
  return [...str].length;
}
console.log(countSymbols("Hello😄")); // 6 (а не 7)
```

10. Комбинирование методов: практические примеры

Пример 1: Нормализация строки

javascript

```
function normalizeString(str) {  
    return str  
        .trim()                // Убрать пробелы  
        .toLowerCase()        // В нижний регистр  
        .replace(/\s+/g, ' ')  // Заменить множественные пробелы  
        .replace(/^[^\w\s-]/g, '') // Удалить специальные символы  
        .replace(/[-\s]+/g, '-') // Заменить пробелы и дефисы одним дефисом  
}  
  
console.log(normalizeString(" Hello World!!! ")); // "hello-world"  
console.log(normalizeString("JavaScript & TypeScript")); // "javascript-typescript"
```

Пример 2: Генератор случайных строк

javascript

```
function generateRandomString(length, options = {}) {  
    const {  
        includeNumbers = true,  
        includeUppercase = true,  
        includeLowercase = true,  
        includeSymbols = false  
    } = options;  
  
    let charset = '';  
    if (includeLowercase) charset += 'abcdefghijklmnopqrstuvwxyz';  
    if (includeUppercase) charset += 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';  
    if (includeNumbers) charset += '0123456789';  
    if (includeSymbols) charset += '!@#$%^&*()_+~[]{}|;:,.<>?';  
  
    let result = '';  
    for (let i = 0; i < length; i++) {  
        const randomIndex = Math.floor(Math.random() * charset.length);  
        result += charset[randomIndex];  
    }  
  
    return result;  
}
```

```
console.log(generateRandomString(10)); // "aB3dEfG7hI"
console.log(generateRandomString(8, { includeSymbols: true })); // "a@B#c$D%"
```

Пример 3: Парсинг и валидация данных

javascript

```
class StringValidator {
  static isEmail(email) {
    const emailRegex = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;
    return emailRegex.test(email.trim().toLowerCase());
  }

  static isStrongPassword(password) {
    return password.length >= 8 &&
      /[A-Z]/.test(password) && // Есть заглавная буква
      /[a-z]/.test(password) && // Есть строчная буква
      /[d]/.test(password) && // Есть цифра
      /[!@#%&*]/.test(password); // Есть специальный символ
  }

  static isPhoneNumber(phone) {
    // Простая валидация для российских номеров
    const cleaned = phone.replace(/\\D/g, '');
    return cleaned.length === 11 && cleaned.startsWith('7');
  }

  static sanitizeHTML(input) {
    return input
      .replace(/&/g, '&amp;')
      .replace(/</g, '&lt;')
      .replace(/>/g, '&gt;')
      .replace(/"/g, '&quot;')
      .replace(/'/g, '&#039;');
  }
}

// Использование
console.log(StringValidator.isEmail("test@example.com")); // true
console.log(StringValidator.isStrongPassword("Pass123!")); // true
console.log(StringValidator.isPhoneNumber("+7 (999) 123-45-67")); // true
console.log(StringValidator.sanitizeHTML("<script>alert('xss')</script>"));
// "&lt;script&gt;alert(&#039;xss&#039;)&lt;/script&gt;"
```

11. Производительность и лучшие практики

Сравнение производительности методов

```
javascript

// Тестирование производительности разных способов конкатенации
const iterations = 100000;

// 1. Оператор + (быстрее для малого количества)
let start = performance.now();
let result = '';
for (let i = 0; i < iterations; i++) {
    result += 'a';
}
console.log(`+= оператор: ${performance.now() - start}ms`);

// 2. Array.join() (быстрее для большого количества)
start = performance.now();
const arr = [];
for (let i = 0; i < iterations; i++) {
    arr.push('a');
}
result = arr.join('');
console.log(`Array.join(): ${performance.now() - start}ms`);

// 3. String.concat() (медленнее)
start = performance.now();
result = '';
for (let i = 0; i < iterations; i++) {
    result = result.concat('a');
}
console.log(`String.concat(): ${performance.now() - start}ms`);
```

Рекомендации по производительности:

1. **Используйте шаблонные строки** для сложной интерполяции
2. **Избегайте вложенных вызовов replace()** в циклах
3. **Кэшируйте результаты** часто используемых операций
4. **Используйте регулярные выражения** для сложных операций поиска/замены

Лучшие практики:

```
javascript
```

// ПЛОХО: Цепочка с промежуточными переменными

```
function oldWay(str) {  
  let trimmed = str.trim();  
  let lower = trimmed.toLowerCase();  
  let replaced = lower.replace(/ /g, '-');  
  return replaced;  
}
```

// ХОРОШО: Функциональный стиль с цепочкой методов

```
function modernWay(str) {  
  return str  
    .trim()  
    .toLowerCase()  
    .replace(/ /g, '-');  
}
```

// ЕЩЕ ЛУЧШЕ: Разделение на небольшие функции

```
const trim = str => str.trim();  
const toLower = str => str.toLowerCase();  
const slugify = str => str.replace(/ /g, '-');  
  
const compose = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);  
const processString = compose(trim, toLower, slugify);  
  
console.log(processString(" Hello World ")); // "heLlo-world"
```

12. Современные возможности (ES6+)

Именованные группы захвата в регулярных выражениях

javascript

// Извлечение данных из строки с именованными группами

```
const dateString = "2024-12-15";  
const regex = /(?!<year>\d{4})-(?!<month>\d{2})-(?!<day>\d{2})/;  
const match = dateString.match(regex);  
  
if (match) {  
  const { year, month, day } = match.groups;  
  console.log(`Год: ${year}, Месяц: ${month}, День: ${day}`);  
  // Год: 2024, Месяц: 12, День: 15  
}
```

Метод `matchAll()` (ES2020)

```
javascript

// Получение всех совпадений с группами захвата
const text = "Иван: 30 лет, Мария: 25 лет, Петр: 35 лет";
const regex = /(\w+):\s*(\d+)\s*лет/g;

for (const match of text.matchAll(regex)) {
    console.log(`${match[1]} - ${match[2]} лет`);
}

// Иван - 30 лет
// Мария - 25 лет
// Петр - 35 лет
```

13. Заключение

Методы работы со строками в JavaScript образуют мощный и выразительный инструментарий для обработки текстовых данных. Ключевые принципы эффективной работы со строками:

1. **Понимание иммутабельности** — строки не изменяются, методы возвращают новые строки
2. **Выбор правильного метода** — используйте специализированные методы (`includes()` вместо `indexOf() > -1`)
3. **Учет локали** — для интернационализированных приложений используйте `localeCompare()`
4. **Работа с Unicode** — используйте современные методы для корректной обработки эмодзи и символов
5. **Оптимизация производительности** — избегайте ненужных операций, особенно в циклах

Современный JavaScript продолжает развиваться, добавляя новые методы (как `replaceAll()` или метод `at()`) и улучшая существующие. Оставаясь в курсе этих изменений, вы сможете писать более эффективный, читаемый и надежный код для работы с текстовыми данными.

10.3. Кодировки и Unicode в JavaScript

Введение в кодировки символов

Кодировка символов — это система, которая сопоставляет каждый символ с уникальным числовым кодом. Понимание кодировок критически важно для корректной работы с текстом на разных языках, специальными символами и эмодзи.

Исторический контекст

```
javascript

// ASCII (American Standard Code for Information Interchange)
// 1963 год, 128 символов (7 бит)
console.log("A".charCodeAt(0)); // 65 - латинская A в ASCII
console.log("a".charCodeAt(0)); // 97 - латинская a в ASCII

// Проблема: только английский язык, нет других символов
```

1. Unicode: универсальный стандарт

Unicode — это стандарт, который ставит целью присвоить уникальный номер (кодировую точку) каждому символу каждого языка в мире.

Основные понятия Unicode

```
javascript

// Кодовая точка (Code Point) - числовое значение символа в Unicode
// Диапазон: U+0000 до U+10FFFF (1,114,112 возможных символов)

// Запись в JavaScript
console.log("\u0041"); // "A" - шестнадцатеричная запись
console.log("\u{41}"); // "A" - ES6 синтаксис
console.log("\u{1F600}"); // "😄" - эмодзи

// Получение кодовой точки
```



```
console.log("A".codePointAt(0)); // 65
console.log("😊".codePointAt(0)); // 128512 (0x1F600)
```

Плоскости Unicode

```
javascript

// 17 плоскостей по 65,536 символов в каждой
const planes = {
  // Основная многоязыковая плоскость (BMP)
  BMP: "U+0000 - U+FFFF", // 65,536 символов
  examples: ["A", "Б", "あ", "х", "€"],

  // Дополнительные плоскости
  SMP: "U+10000 - U+1FFFF", // Дополнительная многоязыковая
  SIP: "U+20000 - U+2FFFF", // Дополнительная идеографическая
  TIP: "U+30000 - U+3FFFF", // Третичная идеографическая
  // ... и другие

  // Особые символы
  "Эмодзи": "U+1F600 - U+1F64F", // Emoticons
  examplesEmoji: ["😊", "👤", "😞"]
};

// Проверка плоскости символа
function getUnicodePlane(char) {
  const codePoint = char.codePointAt(0);
  if (codePoint <= 0xFFFF) return "BMP";
  if (codePoint <= 0x1FFFF) return "SMP";
  if (codePoint <= 0x2FFFF) return "SIP";
  return "Другая плоскость";
}

console.log(getUnicodePlane("A")); // "BMP"
console.log(getUnicodePlane("😊")); // "SMP"
```

2. Кодировки UTF: Unicode Transformation Format

UTF-8, UTF-16, UTF-32

```
javascript

// UTF-8 (8-bit Unicode Transformation Format)
// Наиболее распространенная кодировка в вебе
```

```

const utf8Info = {
  description: "Переменная длина: 1-4 байта на символ",
  advantages: [
    "Совместима с ASCII",
    "Эффективна для латиницы",
    "Стандарт для HTML, JSON, XML"
  ],
  byteStructure: {
    "0-127": "1 байт (ASCII)",
    "128-2047": "2 байта",
    "2048-65535": "3 байта",
    "65536+": "4 байта"
  }
};

// UTF-16 (16-bit Unicode Transformation Format)
// Используется в JavaScript внутренне
const utf16Info = {
  description: "2 или 4 байта на символ",
  advantages: [
    "Используется в JavaScript и Java",
    "Эффективна для большинства языков"
  ],
  surrogatePairs: "Для символов за пределами BMP используются суррогатные пары"
};

// UTF-32 (32-bit Unicode Transformation Format)
const utf32Info = {
  description: "Фиксированная длина: 4 байта на символ",
  advantages: ["Простая обработка"],
  disadvantages: ["Неэффективное использование памяти"]
};

// Практическое сравнение
function getByteLength(str) {
  // Используем TextEncoder для точного подсчета байт в UTF-8
  const encoder = new TextEncoder();
  return encoder.encode(str).length;
}

const testStrings = {
  "English": "Hello World",
  "Russian": "Привет Мир",
  "Japanese": "こんにちは世界",

```

```

    "Emoji": "Hello 😊 World"
};

for (const [lang, str] of Object.entries(testStrings)) {
    console.log(`${lang}: "${str}"`);
    console.log(` Символов: ${[...str].length}`);
    console.log(` UTF-8 байт: ${getByteLength(str)}`);
    console.log(` UTF-16 элементов: ${str.length}`);
}

```

3. Суррогатные пары в UTF-16

Что такое суррогатные пары?

```

javascript

// Символы за пределами BMP (U+10000 и выше) кодируются двумя 16-битными словами
const emoji = "😊"; // U+1F600

console.log("Изучаем суррогатные пары:");
console.log(`Символ: ${emoji}`);
console.log(`Длина в JavaScript: ${emoji.length}`); // 2
console.log(`Кодовая точка: ${emoji.codePointAt(0)}`); // 128512

// Разбиваем на суррогатную пару
console.log("\nСуррогатная пара:");
console.log(`Первый суррогат: ${emoji.charCodeAt(0).toString(16)}`); // d83d
console.log(`Второй суррогат: ${emoji.charCodeAt(1).toString(16)}`); // de00

// Проверка, является ли символ суррогатным
function isHighSurrogate(codeUnit) {
    return codeUnit >= 0xD800 && codeUnit <= 0xDBFF;
}

function isLowSurrogate(codeUnit) {
    return codeUnit >= 0xDC00 && codeUnit <= 0xDFFF;
}

function isSurrogatePair(str, index = 0) {
    return isHighSurrogate(str.charCodeAt(index)) &&
        isLowSurrogate(str.charCodeAt(index + 1));
}

```

// Примеры

```
const testChars = ["A", "😄", "吉", "RU"];
testChars.forEach(char => {
  console.log(`"${char}": суррогатная пара? ${isSurrogatePair(char)}`);
});
```

Работа с суррогатными парами

javascript

// Правильные способы работы со строками, содержащими суррогатные пары

// 1. Использование spread оператора

```
const strWithEmoji = "Hello😄World";
const chars = [...strWithEmoji];
console.log(`Spread: ${chars}`); // ["H", "e", "l", "l", "o", "😄", "W", "o", "r", "l", "d"]
console.log(`Количество символов: ${chars.length}`); // 11
```

// 2. Использование for...of

```
let charCount = 0;
for (const char of strWithEmoji) {
  charCount++;
}
console.log(`for...of count: ${charCount}`); // 11
```

// 3. Array.from()

```
console.log(`Array.from: ${Array.from(strWithEmoji).length}`); // 11
```

// 4. НЕПРАВИЛЬНО: обычный for по индексам

```
let wrongCount = 0;
for (let i = 0; i < strWithEmoji.length; i++) {
  wrongCount++;
}
console.log(`Неправильный подсчет: ${wrongCount}`); // 12
```

// Функция для безопасного доступа к символам

```
function getCharacterAt(str, index) {
  return [...str][index];
}
```

```
console.log(getCharacterAt("Hello😄World", 5)); // "😄"
```

4. Нормализация Unicode

Проблема: эквивалентные последовательности

```
javascript

// Один и тот же символ может быть представлен разными последовательностями
const eAcute1 = "\u00E9";           // É - одна кодовая точка
const eAcute2 = "\u0065\u0301";    // e + ´ - две кодовые точки

console.log(`Визуально одинаковые: ${eAcute1 === eAcute2}`); // false
console.log(`Длина: ${eAcute1.length} vs ${eAcute2.length}`); // 1 vs 2
console.log(`Кодовые точки: ${eAcute1.codePointAt(0)} vs ${eAcute2.codePointAt(0)}`);

// Это создает проблемы при сравнении и поиске
const text = "café";
console.log(text.includes(eAcute1)); // false (если используется eAcute2)
```

Формы нормализации NFC/NFD/NFKC/NFKD

```
javascript

// NFC (Normalization Form C) - каноническая композиция
// NFD (Normalization Form D) - каноническая декомпозиция
// NFKC/NFKD - совместимые формы (более агрессивные)

const examples = {
  "É (одна кодовая точка)": "\u00C9",
  "E + ´ (две кодовые точки)": "\u0045\u0301",
  "ff (лигатура)": "\uFB00",
  "f + f": "ff"
};

// Функция для демонстрации нормализации
function demonstrateNormalization(str) {
  return {
    Исходная: {
      строка: str,
      длина: str.length,
      кодовыеТочки: [...str].map(c => c.codePointAt(0).toString(16))
    },
    NFC: {
      строка: str.normalize('NFC'),
      длина: str.normalize('NFC').length
    },
  },
}
```

```

    NFD: {
        строка: str.normalize('NFD'),
        длина: str.normalize('NFD').length
    },
    NFKC: {
        строка: str.normalize('NFKC'),
        длина: str.normalize('NFKC').length
    },
    NFKD: {
        строка: str.normalize('NFKD'),
        длина: str.normalize('NFKD').length
    }
};
}

```

// Практическое применение

```

function normalizeAndCompare(str1, str2) {
    const normalized1 = str1.normalize('NFC');
    const normalized2 = str2.normalize('NFC');
    return normalized1 === normalized2;
}

```

```

const cafe1 = "café"; // с предкомбинированным é
const cafe2 = "cafe\u0301"; // с комбинирующим диакритическим знаком
console.log(`Сравнение после нормализации: ${normalizeAndCompare(cafe1, cafe2)}`); //
true

```

5. TextEncoder и TextDecoder API

Работа с бинарными данными

javascript

// TextEncoder - преобразует строку в Uint8Array (UTF-8 по умолчанию)

```

const encoder = new TextEncoder();
const decoder = new TextDecoder();

```

// Кодирование строки

```

const text = "Hello мир 😊";
const encoded = encoder.encode(text);
console.log("Закодированная строка (UTF-8):");
console.log(`  Длина в байтах: ${encoded.length}`);
console.log(`  Бинарное представление:`, encoded);

```

```

console.log(` Шестнадцатерично:`,
  Array.from(encoded).map(b => b.toString(16).padStart(2, '0')).join(' '));

// Декодирование обратно
const decoded = decoder.decode(encoded);
console.log(`Декодировано обратно: "${decoded}"`);

// Работа с разными кодировками
async function demonstrateEncodings() {
  const testString = "Привет €";

  const encodings = ['utf-8', 'windows-1251', 'koi8-r', 'iso-8859-1'];

  for (const encoding of encodings) {
    try {
      const encoder = new TextEncoder(); // только UTF-8
      const decoder = new TextDecoder(encoding);

      // Эмулируем данные в другой кодировке
      const bytes = new Uint8Array([207, 240, 232, 226, 229, 242, 32, 8364]);
      const decodedText = decoder.decode(bytes);

      console.log(`${encoding}: ${decodedText}`);
    } catch (error) {
      console.log(`${encoding}: Ошибка - ${error.message}`);
    }
  }
}

// Практический пример: загрузка файла с указанной кодировкой
async function loadTextFile(url, encoding = 'utf-8') {
  try {
    const response = await fetch(url);
    const buffer = await response.arrayBuffer();
    const decoder = new TextDecoder(encoding);
    return decoder.decode(buffer);
  } catch (error) {
    console.error(`Ошибка загрузки файла: ${error}`);
    return null;
  }
}

```

6. Графем-кластеры и расширенные символы

Проблема сложных символов

```
javascript

// Некоторые "символы" состоят из нескольких кодовых точек
const examples = {
  "Эмодзи с модификатором тона кожи": "👤", // 🧑 + 🏔
  "Флаг страны": "RU", // R + U
  "Семейный эмодзи": "👨👩👧👦", // 4 человека + 3 соединителя
  "Комбинирующиеся символы": "நீ", // тамильский с диакритиками
  "Рукописный символ": "👩" // женщина + тон кожи + профессия
};

// Функция для анализа графем-кластеров
function analyzeGraphemeClusters(str) {
  console.log(`Анализ: "${str}"`);
  console.log(`Длина (JavaScript): ${str.length}`);
  console.log(`Кодовые точки: ${[...str].length}`);

  // Используем Intl.Segmenter для правильного разделения на графемы
  if ('Segmenter' in Intl) {
    const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
    const segments = [...segmenter.segment(str)];
    console.log(`Графем-кластеров: ${segments.length}`);
    console.log(`Графемы:`, segments.map(s => s.segment));
  } else {
    console.log(`Intl.Segmenter не поддерживается в этой среде`);
  }

  // Подсчет составных частей
  const codePoints = [...str].map(c =>
    `U+${c.codePointAt(0).toString(16).toUpperCase()}`
  );
  console.log(`Составные части: ${codePoints.join(', ')}`);
}

// Примеры анализа
analyzeGraphemeClusters("👤");
analyzeGraphemeClusters("RU");
analyzeGraphemeClusters("நீ");
```


Intl.Segmenter API (ECMAScript 2022)

```
javascript

// Новый API для правильной сегментации текста
if ('Segmenter' in Intl) {
    const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });

    const complexText = "Hello 👨‍👩‍👧 family! ru Россия, café, 𑌕";
    const segments = [...segmenter.segment(complexText)];

    console.log("Правильная сегментация текста:");
    segments.forEach((segment, index) => {
        console.log(`  [${index}] "${segment.segment}" (${segment.segment.length} кодовых точек)`);
    });

    // Практическое применение: подсчет "символов" для пользователя
    function countVisibleCharacters(str) {
        const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
        return [...segmenter.segment(str)].length;
    }

    console.log(`\nПодсчет видимых символов:`);
    console.log(`  👨‍👩‍👧: ${countVisibleCharacters("👨‍👩‍👧")} символ`; // 1
    console.log(`  "café": ${countVisibleCharacters("café")} символов`; // 4
} else {
    console.log("Intl.Segmenter не поддерживается. Используйте полифилл.");
}
```

7. Экранирование Unicode в JavaScript

Способы представления Unicode символов

```
javascript

// 1. Прямое использование (если файл в UTF-8)
const direct = "Привет 😊";

// 2. Unicode escape последовательности (6 цифр)
const unicodeEscape = "\u0048\u0065\u006C\u006C\u006F"; // "Hello"

// 3. Расширенные escape последовательности (ES6)
const extendedEscape = "\u{48}\u{65}\u{6C}\u{6C}\u{6F}"; // "Hello"
```

```
const emojiEscape = "\u{1F600}"; // "😄"
```

```
// 4. Шестнадцатеричные escape в регулярных выражениях
```

```
const regexWithUnicode = /\u{1F600}/u; // Флаг 'u' для поддержки Unicode
```

```
// Функция для преобразования строки в escape последовательности
```

```
function toUnicodeEscapes(str, useExtended = true) {  
  return [...str].map(char => {  
    const codePoint = char.codePointAt(0);  
    if (useExtended && codePoint > 0xFFFF) {  
      return `\\u${codePoint.toString(16)}`;  
    } else if (codePoint <= 0xFF) {  
      return `\\x${codePoint.toString(16).padStart(2, '0')}`;  
    } else {  
      return `\\u${codePoint.toString(16).padStart(4, '0')}`;  
    }  
  }).join('');  
}
```

```
console.log(toUnicodeEscapes("Hello мир 😄"));
```

```
// \u0048\u0065\u006c\u0066\u0020\u043c\u0438\u0440\u0020\u{1f600}
```

```
// Обратное преобразование
```

```
function fromUnicodeEscapes(escapedStr) {  
  return escapedStr.replace(/\\u\{([0-9a-fA-F]+\)\}|\\u([0-9a-fA-F]{4})|\\x([0-9a-fA-F]{2})/g,  
    (match, extended, regular, hex) => {  
      if (extended) {  
        return String.fromCodePoint(parseInt(extended, 16));  
      } else if (regular) {  
        return String.fromCharCode(parseInt(regular, 16));  
      } else if (hex) {  
        return String.fromCharCode(parseInt(hex, 16));  
      }  
      return match;  
    }  
  );  
}
```

```
console.log(fromUnicodeEscapes("\\u0048\\u0065\\u006c\\u0066\\u0020\\u043c\\u0438\\u0440\\u0020\\u{1f600}"));
```

```
// "Hello мир 😄"
```

8. Проблемы и решения при работе с Unicode

Распространенные проблемы

javascript

// 1. Неправильный подсчет длины

```
function demonstrateLengthProblems() {
  const testCases = [
    { text: "hello", description: "Английский" },
    { text: "привет", description: "Русский" },
    { text: "😁", description: "Эмодзи" },
    { text: "👨👩👦", description: "Семейный эмодзи" },
    { text: "café", description: "С диакритиками" },
    { text: "RU", description: "Флаг" }
  ];

  console.log("Проблемы с длиной строк:");
  testCases.forEach(({ text, description }) => {
    console.log(`  ${description}: "${text}"`);
    console.log(`    .length: ${text.length}`);
    console.log(`    [...].length: ${[...text].length}`);

    if ('Segmenter' in Intl) {
      const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
      const graphemeLength = [...segmenter.segment(text)].length;
      console.log(`    Графем-кластеров: ${graphemeLength}`);
    }
    console.log('---');
  });
}
```

// 2. Проблемы с обращением к символам по индексу

```
function demonstrateIndexingProblems() {
  const str = "Hello😁World";

  console.log("Проблемы с индексацией:");
  console.log(`  Строка: "${str}"`);

  // Неправильно - работает с кодовыми единицами
  for (let i = 0; i < str.length; i++) {
    console.log(`    str[${i}] = "${str[i]}"`);
  }
}
```

```

console.log("\n Правильно - с использованием итератора:");
let index = 0;
for (const char of str) {
    console.log(`    символ ${index++} = "${char}"`);
}
}

```

// 3. Проблемы с сортировкой

```

function demonstrateSortingProblems() {
    const words = ["äpple", "zebra", "apple", "Ångström", "banana"];

    console.log("Проблемы с сортировкой:");
    console.log("  Без учета локали:", [...words].sort());
    console.log("  С учетом локали (en):",
        [...words].sort((a, b) => a.localeCompare(b, 'en')));
    console.log("  С учетом локали (sv):",
        [...words].sort((a, b) => a.localeCompare(b, 'sv')));
}

```

// 4. Проблемы с верхним/нижним регистром

```

function demonstrateCaseProblems() {
    const testCases = [
        "i", // английская
        "İ", // турецкая с точкой
        "ı", // турецкая без точки
        "I"  // английская заглавная
    ];

    console.log("Проблемы с регистром:");
    testCases.forEach(char => {
        console.log(`    "${char}":`);
        console.log(`      toUpperCase('en'): "${char.toUpperCase()}"`);
        console.log(`      toLocaleUpperCase('tr'): "${char.toLocaleUpperCase('tr')}"`);
        console.log(`      toLowerCase('en'): "${char.toLowerCase()}"`);
        console.log(`      toLocaleLowerCase('tr'): "${char.toLocaleLowerCase('tr')}"`);
    });
}

```

Библиотеки для работы с Unicode

javascript

// Иногда встроенных средств недостаточно

// 1. grapheme-splitter (для разделения на графем-кластеры)

```

class GraphemeSplitter {
    constructor() {
        // Упрощенная реализация
        this.graphemeRegex = /\p{RI}\p{RI}|\p{Emoji}(\?:\p{Emoji_Modifier}|\u{FE0F}
\u{20E3}?)|[\u{E0020}-\u{E007E}]+\u{E007F})?(\?:\u{200D}\p{Emoji}(\?:\p{Emoji_Modifier}
\u{FE0F}|\u{20E3}?)|[\u{E0020}-\u{E007E}]+\u{E007F})?)?)/.gus;
    }

    split(text) {
        return [...text.matchAll(this.graphemeRegex)].map(m => m[0]);
    }
}

// 2. unicode-trie (для классификации символов)
class UnicodeTrie {
    // Упрощенная реализация для демонстрации
    static getCategory(codePoint) {
        // Определяем категорию символа
        if (codePoint >= 0x1F600 && codePoint <= 0x1F64F) return 'Emoji';
        if (codePoint >= 0x0400 && codePoint <= 0x04FF) return 'Cyrillic';
        if (codePoint >= 0x0041 && codePoint <= 0x005A) return 'Latin_Uppercase';
        if (codePoint >= 0x0061 && codePoint <= 0x007A) return 'Latin_Lowercase';
        return 'Other';
    }
}

// Практический пример: валидатор текста
class UnicodeAwareValidator {
    static validateText(text, options = {}) {
        const {
            maxLength = 1000,
            allowedScripts = ['Latin', 'Cyrillic'],
            allowEmojis = true,
            allowNumbers = true,
            allowPunctuation = true
        } = options;

        // Проверка длины с учетом графем-кластеров
        const graphemeLength = [...new Intl.Segmenter('en', { granularity: 'grapheme'
}).segment(text)].length;
        if (graphemeLength > maxLength) {
            return { valid: false, error: `Текст слишком длинный: ${graphemeLength}
символов` };
        }
    }
}

```

```

// Проверка символов
const problems = [];
for (const char of text) {
    const codePoint = char.codePointAt(0);
    // Здесь должна быть более сложная логика проверки скриптов
}

return problems.length === 0
    ? { valid: true }
    : { valid: false, errors: problems };
}
}

```

9. Практические примеры и use cases

Пример 1: Валидация имен пользователей

```

javascript

class UsernameValidator {
    static validate(username) {
        // 1. Проверка длины (в видимых символах)
        const segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
        const graphemeCount = [...segmenter.segment(username)].length;

        if (graphemeCount < 3) {
            return { valid: false, error: "Имя должно содержать минимум 3 символа" };
        }

        if (graphemeCount > 20) {
            return { valid: false, error: "Имя не должно превышать 20 символов" };
        }

        // 2. Проверка допустимых символов
        // Разрешаем буквы, цифры, подчеркивания и дефисы
        const allowedPattern = /^[p{L}\p{N}_-]+$ /u;
        if (!allowedPattern.test(username)) {
            return {
                valid: false,
                error: "Имя может содержать только буквы, цифры, подчеркивания и дефисы"
            };
        }
    }
}

```

```

    }

    // 3. Нормализация (преобразование визуально одинаковых имен)
    const normalized = username.normalize('NFKC');

    return {
      valid: true,
      normalized,
      graphemeCount
    };
  }

  static generateSlug(username) {
    const validated = this.validate(username);
    if (!validated.valid) {
      throw new Error(validated.error);
    }

    return validated.normalized
      .toLowerCase()
      .normalize('NFD') // Разделяем буквы и диакритики
      .replace(/[\u0300-\u036f]/g, '') // Удаляем диакритические знаки
      .replace(/^[^w\s-]/g, '') // Удаляем специальные символы
      .replace(/[-\s]+/g, '-') // Заменяем пробелы и множественные дефисы
      .trim();
  }
}

// Тестирование
console.log(UsernameValidator.validate("John_Doe123"));
console.log(UsernameValidator.validate("Jöhn_Дое😊"));
console.log(UsernameValidator.generateSlug("Jöhn Müller"));

```

Пример 2: Подсчет символов в текстовом редакторе

javascript

```

class CharacterCounter {
  constructor(options = {}) {
    this.options = {
      countGraphemes: true,
      countWords: true,
      countBytes: true,
      encoding: 'utf-8',
      ...options
    };
  }
}

```

```

    };

    if (this.options.countGraphemes && 'Segmenter' in Intl) {
        this.segmenter = new Intl.Segmenter('en', { granularity: 'grapheme' });
    }
}

count(text) {
    const result = {};

    // Подсчет графем-кластеров (видимых символов)
    if (this.options.countGraphemes && this.segmenter) {
        result.graphemes = [...this.segmenter.segment(text)].length;
    } else {
        result.graphemes = [...text].length; // Приблизительно
    }

    // Подсчет слов
    if (this.options.countWords) {
        // Используем Intl.Segmenter для более точного подсчета слов
        if ('Segmenter' in Intl) {
            const wordSegmenter = new Intl.Segmenter('en', { granularity: 'word' });
            result.words = [...wordSegmenter.segment(text)]
                .filter(segment => segment.isWordLike)
                .length;
        } else {
            // Простой подсчет для английского
            result.words = text.trim().split(/\s+/).length;
        }
    }

    // Подсчет байт в UTF-8
    if (this.options.countBytes) {
        const encoder = new TextEncoder();
        result.bytes = encoder.encode(text).length;
    }

    // Подсчет символов (кодированных точек)
    result.codePoints = [...text].length;

    // Подсчет кодированных единиц (JavaScript .length)
    result.codeUnits = text.length;

    return result;
}

```



```

    }

    getLimits(text, limits = {}) {
        const counts = this.count(text);
        const remaining = {};

        if (limits.maxGraphemes !== undefined) {
            remaining.graphemes = limits.maxGraphemes - counts.graphemes;
        }

        if (limits.maxBytes !== undefined) {
            remaining.bytes = limits.maxBytes - counts.bytes;
        }

        if (limits.maxWords !== undefined) {
            remaining.words = limits.maxWords - counts.words;
        }

        return { counts, remaining };
    }
}

```

// Использование

```

const counter = new CharacterCounter();
const text = "Hello мир! 😊 Это тестовое сообщение.";
const counts = counter.count(text);

console.log("Статистика текста:");
console.log(`  Графемы (видимые символы): ${counts.graphemes}`);
console.log(`  Слова: ${counts.words}`);
console.log(`  Байты (UTF-8): ${counts.bytes}`);
console.log(`  Кодовые точки: ${counts.codePoints}`);
console.log(`  Кодовые единицы: ${counts.codeUnits}`);

```

Пример 3: Обработка поиска с учетом Unicode

javascript

```

class UnicodeAwareSearch {
    static normalizeForSearch(text) {
        return text
            .normalize('NFD') // Декомпозиция
            .replace(/[\u0300-\u036f]/g, '') // Удаление диакритиков
            .toLowerCase()
            .replace(/[\^\p{L}\p{N}]/gu, ' ') // Замена не-букв/цифр на пробелы
    }
}

```

```

        .replace(/\s+/g, ' ') // Удаление лишних пробелов
        .trim();
    }

    static search(text, query, options = {}) {
        const {
            caseSensitive = false,
            diacriticSensitive = false,
            wholeWord = false
        } = options;

        let normalizedText = text;
        let normalizedQuery = query;

        if (!caseSensitive) {
            normalizedText = normalizedText.toLowerCase();
            normalizedQuery = normalizedQuery.toLowerCase();
        }

        if (!diacriticSensitive) {
            normalizedText = this.normalizeForSearch(normalizedText);
            normalizedQuery = this.normalizeForSearch(normalizedQuery);
        }

        if (wholeWord) {
            const escapedQuery = normalizedQuery.replace(/[\.*+?^${}()|[\]\\"/g, '\\$&');
            const regex = new RegExp(`\\b${escapedQuery}\\b`, 'g' + (caseSensitive ? '' : 'i'));
            return [...normalizedText.matchAll(regex)].map(match => match.index);
        } else {
            const index = normalizedText.indexOf(normalizedQuery);
            return index === -1 ? [] : [index];
        }
    }

    static highlight(text, query, options = {}) {
        const indices = this.search(text, query, options);

        if (indices.length === 0) {
            return text;
        }

        let result = '';
        let lastIndex = 0;

```

```

    indices.forEach(startIndex => {
        const endIndex = startIndex + query.length;
        result += text.slice(lastIndex, startIndex);
        result += `<mark>${text.slice(startIndex, endIndex)}</mark>`;
        lastIndex = endIndex;
    });

    result += text.slice(lastIndex);
    return result;
}
}

// Пример использования
const documentText = "Café Müller visited the naïve café. The résumé was on the table.";
const searchQuery = "cafe";

console.log("Поиск с разными настройками:");
console.log("1. С учетом регистра и диакритиков:");
console.log(UnicodeAwareSearch.highlight(documentText, searchQuery, {
    caseSensitive: true,
    diacriticSensitive: true
}));

console.log("\n2. Без учета регистра, с учетом диакритиков:");
console.log(UnicodeAwareSearch.highlight(documentText, searchQuery, {
    caseSensitive: false,
    diacriticSensitive: true
}));

console.log("\n3. Без учета регистра и диакритиков:");
console.log(UnicodeAwareSearch.highlight(documentText, searchQuery, {
    caseSensitive: false,
    diacriticSensitive: false
}));

```

10. Советы и лучшие практики

Ключевые рекомендации

javascript

// 1. Всегда указывайте кодировку в мета-теге HTML

```
const htmlMeta = `  
<!DOCTYPE html>  
<html>  
<head>  
  <meta charset="UTF-8"> <!-- ВСЕГДА используйте UTF-8 -->  
  <title>Документ</title>  
</head>  
<body>  
  <!-- Содержимое -->  
</body>  
</html>  
`;  

```

// 2. Нормализуйте пользовательский ввод

```
function normalizeUserInput(input) {  
  return input  
    .normalize('NFC') // Используйте NFC для хранения  
    .trim();  
}
```

// 3. Используйте правильные методы для подсчета длины

```
const bestPractices = {  
  length: {  
    wrong: "str.length", // для кодовых единиц  
    correct: "[...str].length", // для кодовых точек  
    best: "Intl.Segmenter для графем-кластеров"  
  },  
  
  iteration: {  
    wrong: "for (let i = 0; i < str.length; i++)",  
    correct: "for (const char of str)",  
    best: "for (const segment of segmenter.segment(str))"  
  },  
  
  comparison: {  
    wrong: "str1 === str2", // без нормализации  
    correct: "str1.normalize() === str2.normalize()",  
    best: "localeCompare с нужной локалью"  
  }  
};
```

// 4. Обрабатывайте ошибки кодирования

```
function safeDecode(buffer, encoding = 'utf-8') {  
  try {
```

```

    const decoder = new TextDecoder(encoding, { fatal: true });
    return decoder.decode(buffer);
} catch (error) {
    console.warn(`Ошибка декодирования ${encoding}:`, error);
    // Попробовать другие кодировки или вернуть замену
    const fallbackDecoder = new TextDecoder(encoding, {
        fatal: false,
        ignoreBOM: true
    });
    return fallbackDecoder.decode(buffer);
}
}

```

// 5. Тестируйте с разными наборами символов

```

const testCharacterSets = {
    latin: "Hello World",
    cyrillic: "Привет Мир",
    arabic: "مرحباً بالعالم",
    chinese: "你好世界",
    japanese: "こんにちは世界",
    emoji: "👋 🌍 😊",
    mixed: "Hello 世界 🌍 привет"
};

```

// Функция для проверки поддержки Unicode в приложении

```

function testUnicodeSupport() {
    const tests = [];

    for (const [name, text] of Object.entries(testCharacterSets)) {
        tests.push({
            name,
            text,
            length: text.length,
            codePoints: [...text].length,
            bytes: new TextEncoder().encode(text).length,
            normalized: text.normalize('NFC') === text
        });
    }

    return tests;
}

```

11. Заключение

Работа с Unicode и кодировками в JavaScript требует внимательного подхода, но современный язык предоставляет все необходимые инструменты:

1. **Используйте UTF-8 везде** — это стандарт для веба
2. **Помните о суррогатных парах** — используйте правильные методы итерации
3. **Нормализуйте строки** — особенно при сравнении и хранении
4. **Используйте Intl API** — для корректной работы с локалями и графемами
5. **Тестируйте с разными наборами символов** — обеспечивайте международную поддержку

Понимание этих принципов позволит создавать robust-приложения, корректно работающие с текстом на любых языках мира. Unicode — это не просто техническая спецификация, это фундамент для создания инклюзивного, многоязычного интернета.

10.4. Введение в регулярные выражения

Что такое регулярные выражения?

Регулярные выражения (RegExp) — это мощный инструмент для поиска и манипуляции текстом с помощью специальных шаблонов. Они представляют собой язык для описания паттернов в строках и являются неотъемлемой частью программирования на JavaScript.

Базовое определение

```
javascript

// Регулярное выражение в JavaScript можно создать двумя способами:

// 1. Литерал регулярного выражения (рекомендуется)
const regex1 = /pattern/flags;

// 2. Конструктор RegExp (полезно для динамических шаблонов)
const regex2 = new RegExp('pattern', 'flags');

// Примеры
const simplePattern = /hello/;
const dynamicPattern = new RegExp('hello', 'i');
```

1. Создание и использование регулярных выражений

Основные флаги (модификаторы)

Флаги изменяют поведение регулярного выражения:

```
javascript

// i - игнорирование регистра (case-insensitive)
const caseInsensitive = /hello/i;
console.log(caseInsensitive.test('Hello')); // true
console.log(caseInsensitive.test('HELLO')); // true
```

```

// g - глобальный поиск (все совпадения)
const globalSearch = /a/g;
const text = 'banana';
console.log(text.match(globalSearch)); // ['a', 'a', 'a']

// m - многострочный режим
const multiline = /^start/m;
const multilineText = 'first line\nstart of second line';
console.log(multiline.test(multilineText)); // true

// s - точка включает перевод строки (ES2018)
const dotAll = /hello.world/s;
console.log(dotAll.test('hello\nworld')); // true

// u - полная поддержка Unicode
const unicode = /\u{1F600}/u;
console.log(unicode.test('😄')); // true

// y - липкий поиск (sticky)
const sticky = /a/y;
sticky.lastIndex = 1;
console.log(sticky.test('banana')); // false (не на позиции 1)

// d - индексы совпадений (ES2022)
const withIndices = /a/d;
const match = withIndices.exec('banana');
console.log(match.indices); // [[1, 2]]

```

Методы работы с регулярными выражениями

```

javascript

const text = 'JavaScript is awesome! JavaScript is powerful.';

// test() - проверка наличия совпадения
const pattern = /JavaScript/;
console.log(pattern.test(text)); // true

// exec() - поиск совпадения с деталями
const match = pattern.exec(text);
console.log(match[0]); // "JavaScript"
console.log(match.index); // 0
console.log(match.input); // исходный текст

// Использование с флагом g

```



```
const globalPattern = /JavaScript/g;
let result;
while ((result = globalPattern.exec(text)) !== null) {
    console.log(`Найдено "${result[0]}" на позиции ${result.index}`);
}
```

2. Базовые элементы регулярных выражений

Литералы и специальные символы

```
javascript

// Простые литералы (совпадают с самими собой)
const literalPattern = /hello/;
console.log(literalPattern.test('hello world')); // true
console.log(literalPattern.test('Hello')); // false (регистр)

// Точка . - любой символ кроме перевода строки
const dotPattern = /h.llo/;
console.log(dotPattern.test('hello')); // true
console.log(dotPattern.test('hallo')); // true
console.log(dotPattern.test('h\nllo')); // false

// Экранирование специальных символов \
const specialChars = [
    '^', '$', '.', '*', '+', '?',
    '(', ')', '[', ']', '{', '}',
    '|', '\\', '/'
];

// Поиск точки как символа
const literalDot = /\./;
console.log(literalDot.test('test.txt')); // true
console.log(literalDot.test('test')); // false

// Поиск обратного слэша
const backslashPattern = /\\\/;
console.log(backslashPattern.test('C:\\Windows')); // true
```

Классы символов

```
javascript

// Набор символов [...]
```

```

const vowelPattern = /[aeiou]/;
console.log(vowelPattern.test('hello')); // true (e)
console.log(vowelPattern.test('rhythm')); // false

// Диапазоны символов [a-z], [0-9], [A-Z]
const lowercasePattern = /[a-z]/;
const digitPattern = /[0-9]/;
const hexPattern = /[0-9a-fA-F]/;

console.log(lowercasePattern.test('Hello')); // true
console.log(digitPattern.test('Year 2023')); // true
console.log(hexPattern.test('FF')); // true

// Отрицание набора [^...]
const nonVowelPattern = /^[^aeiou]/;
console.log(nonVowelPattern.test('a')); // false
console.log(nonVowelPattern.test('b')); // true

// Предопределенные классы символов
const predefinedClasses = {
  '\\d': 'любая цифра [0-9]',
  '\\D': 'любой не-цифровой символ [^0-9]',
  '\\w': 'любой "словесный" символ [A-Za-z0-9_]',
  '\\W': 'любой не "словесный" символ [^A-Za-z0-9_]',
  '\\s': 'любой пробельный символ',
  '\\S': 'любой не пробельный символ',
  '\\n': 'перевод строки',
  '\\t': 'табуляция'
};

// Примеры использования
const digitExample = /\d{3}/;
console.log(digitExample.test('123')); // true
console.log(digitExample.test('abc')); // false

const wordExample = /\w+/;
console.log(wordExample.test('hello_123')); // true
console.log(wordExample.test('!!!')); // false

const whitespaceExample = /\s/;
console.log(whitespaceExample.test('hello world')); // true

```

3. Квантификаторы (повторители)

Квантификаторы определяют, сколько раз элемент может повторяться.

```
javascript

// Базовые квантификаторы
const quantifiers = {
  '*': '0 или более раз',
  '+': '1 или более раз',
  '?': '0 или 1 раз',
  '{n}': 'ровно n раз',
  '{n,}': 'n или более раз',
  '{n,m}': 'от n до m раз'
};

// Примеры
const zeroOrMore = /a*/;
console.log(zeroOrMore.test('')); // true
console.log(zeroOrMore.test('a')); // true
console.log(zeroOrMore.test('aaaa')); // true

const oneOrMore = /a+/;
console.log(oneOrMore.test('')); // false
console.log(oneOrMore.test('a')); // true
console.log(oneOrMore.test('aaaa')); // true

const zeroOrOne = /a?/;
console.log(zeroOrOne.test('')); // true
console.log(zeroOrOne.test('a')); // true
console.log(zeroOrOne.test('aa')); // true (но только первая a)

// Точные количества
const exactlyThree = /a{3}/;
console.log(exactlyThree.test('aaa')); // true
console.log(exactlyThree.test('aa')); // false
console.log(exactlyThree.test('aaaa')); // true (первые три)

const threeOrMore = /a{3,}/;
console.log(threeOrMore.test('aaa')); // true
console.log(threeOrMore.test('aaaaa')); // true

const betweenTwoAndFour = /a{2,4}/;
console.log(betweenTwoAndFour.test('a')); // false
```

```
console.log(betweenTwoAndFour.test('aa')); // true
console.log(betweenTwoAndFour.test('aaaa')); // true
console.log(betweenTwoAndFour.test('aaaaa')); // true (первые четыре)
```

Жадные vs ленивые квантификаторы

```
javascript

// По умолчанию квантификаторы жадные (greedy)
const greedyPattern = /\.+/;
const text = 'Words: "hello" and "world"';
const greedyMatch = text.match(greedyPattern);
console.log(greedyMatch[0]); // '"hello" and "world"' (ВСЕ между кавычками)

// Ленивые (нежадные) квантификаторы добавляют ?
const lazyPattern = /\.+?/;
const lazyMatch = text.match(lazyPattern);
console.log(lazyMatch[0]); // '"hello"' (только первая пара кавычек)

// Сравнение других квантификаторов
const examples = {
  '*?': 'как можно меньше повторений, минимум 0',
  '+?': 'как можно меньше повторений, минимум 1',
  '??': '0 раз (если возможно)',
  '{n}?': 'ровно n раз (эквивалентно {n})',
  '{n,}?': 'как можно меньше, минимум n',
  '{n,m}?': 'как можно меньше, минимум n, максимум m'
};

// Практический пример: извлечение HTML тегов
const html = '<div>Hello</div><span>World</span>';
const greedyTag = /<.+>/g;
const lazyTag = /<.+?>/g;

console.log(html.match(greedyTag)); // ['<div>Hello</div><span>World</span>']
console.log(html.match(lazyTag)); // ['<div>', '</div>', '<span>', '</span>']
```

4. Группировка и альтернативы

Группировка с помощью круглых скобок

```
javascript

// Группировка для применения квантификаторов
```

```

const groupedPattern = /(abc)+/;
console.log(groupedPattern.test('abc')); // true
console.log(groupedPattern.test('abcabc')); // true
console.log(groupedPattern.test('abcc')); // false

// Группировка для захвата частей совпадения
const datePattern = /(\d{4})-(\d{2})-(\d{2})/;
const dateMatch = datePattern.exec('2023-12-25');
console.log(dateMatch[0]); // "2023-12-25" (полное совпадение)
console.log(dateMatch[1]); // "2023" (год)
console.log(dateMatch[2]); // "12" (месяц)
console.log(dateMatch[3]); // "25" (день)

// Незахватывающие группы (?:...)
const nonCapturingPattern = /(?:Mr|Mrs|Ms)\. (\w+)/;
const nameMatch = nonCapturingPattern.exec('Mr. Smith');
console.log(nameMatch[0]); // "Mr. Smith"
console.log(nameMatch[1]); // "Smith" (только имя)
console.log(nameMatch.length); // 2 (нет группы для обращения)

// Именованные группы захвата (?<name>...) (ES2018)
const namedGroupPattern = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;
const namedMatch = namedGroupPattern.exec('2023-12-25');
console.log(namedMatch.groups); // {year: "2023", month: "12", day: "25"}
console.log(namedMatch.groups.year); // "2023"

```

Альтернативы (оператор ИЛИ)

javascript

```

// Вертикальная черта | обозначает "или"
const colorPattern = /red|green|blue/;
console.log(colorPattern.test('red')); // true
console.log(colorPattern.test('green')); // true
console.log(colorPattern.test('yellow')); // false

// Группировка с альтернативами
const petPattern = /I have a (cat|dog|fish)/;
console.log(petPattern.test('I have a cat')); // true
console.log(petPattern.test('I have a parrot')); // false

// Сложные альтернативы
const timePattern = /(0?[1-9]|1[0-2]):[0-5][0-9] (AM|PM)/;
console.log(timePattern.test('9:30 AM')); // true
console.log(timePattern.test('12:45 PM')); // true

```

```

console.log(timePattern.test('13:00 PM')); // false

// Приоритет в альтернативах
const ambiguousPattern = /cat|dog food/;
console.log(ambiguousPattern.test('cat')); // true
console.log(ambiguousPattern.test('dog food')); // true
console.log(ambiguousPattern.test('cat food')); // true (только "cat")

// Явная группировка
const clearPattern = /(cat|dog) food/;
console.log(clearPattern.test('cat food')); // true
console.log(clearPattern.test('dog food')); // true

```

5. Привязки (якоря) и границы

Начало и конец строки

```

javascript

// ^ - начало строки (или начала строки в многострочном режиме)
const startsWithHello = /^Hello/;
console.log(startsWithHello.test('Hello World')); // true
console.log(startsWithHello.test('Say Hello')); // false

// $ - конец строки
const endsWithWorld = /World$/;
console.log(endsWithWorld.test('Hello World')); // true
console.log(endsWithWorld.test('World Hello')); // false

// Полное совпадение строки
const exactMatch = /^Hello World$/;
console.log(exactMatch.test('Hello World')); // true
console.log(exactMatch.test('Hello World!')); // false

// Многострочный режим с флагом m
const multilineText = `First line
Second line
Third line`;

const startOfLinePattern = /\n+/gm;
console.log(multilineText.match(startOfLinePattern)); // ['First', 'Second', 'Third']

```

Границы слов

```
javascript

// \b - граница слова (позиция между \w и \W или начало/конец строки)
const wordBoundaryPattern = /\bcat\b/;
console.log(wordBoundaryPattern.test('cat')); // true
console.log(wordBoundaryPattern.test('cathedral')); // false
console.log(wordBoundaryPattern.test('wildcat')); // false
console.log(wordBoundaryPattern.test('a cat')); // true

// \B - не граница слова
const nonWordBoundaryPattern = /\Bcat\B/;
console.log(nonWordBoundaryPattern.test('cat')); // false
console.log(nonWordBoundaryPattern.test('cathedral')); // true
console.log(nonWordBoundaryPattern.test('wildcat')); // true
console.log(nonWordBoundaryPattern.test('a cat')); // false

// Практический пример: поиск целых слов
function findWholeWord(text, word) {
    const pattern = new RegExp(`\\b${word}\\b`, 'gi');
    return text.match(pattern) || [];
}

const sampleText = 'The cat in the cathedral chased a wildcat';
console.log(findWholeWord(sampleText, 'cat')); // ['cat']
console.log(findWholeWord(sampleText, 'the')); // ['The', 'the']
```

6. Просмотр вперед и назад (lookaround)

Положительный просмотр вперед (?!...)

```
javascript

// Находит X, за которым следует Y, но Y не включается в совпадение
const followedByPattern = /\w+(?!\.)/;
console.log(followedByPattern.exec('file.txt')[0]); // "file"
console.log(followedByPattern.exec('document.pdf')[0]); // "document"
console.log(followedByPattern.exec('noextension')); // null

// Пример: пароль должен содержать цифру
const passwordWithDigit = /^(?=.*\d).+$/;
console.log(passwordWithDigit.test('password')); // false
console.log(passwordWithDigit.test('password1')); // true
```

```
// Проверка нескольких условий
const strongPassword = /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#$%^&*]).{8,}$/;
console.log(strongPassword.test('Weak1')); // false
console.log(strongPassword.test('StrongPass1!')); // true
```

Отрицательный просмотр вперед (?!...)

```
javascript

// Находит X, за которым НЕ следует Y
const notFollowedByPattern = /\d+(?!\.)/;
console.log(notFollowedByPattern.exec('123.45')[0]); // "45" (не следует точка)
console.log(notFollowedByPattern.exec('123')[0]); // "123"

// Пример: слово "часть", не за которым следует цифра
const partNotNumber = /часть(?!\s*\d)/;
console.log(partNotNumber.test('первая часть')); // true
console.log(partNotNumber.test('часть 1')); // false

// Проверка, что строка не заканчивается на определенные символы
const notEndsWithPattern = /.*(?!\.js|\.css)$/;
console.log(notEndsWithPattern.test('script.js')); // false
console.log(notEndsWithPattern.test('style.css')); // false
console.log(notEndsWithPattern.test('document')); // true
```

Просмотр назад (?<=...) и (?<!...)

```
javascript

// Положительный просмотр назад (ES2018)
const precededByPattern = /(?<=)\d+/;
console.log(precededByPattern.exec('Price: $100')[0]); // "100"
console.log(precededByPattern.exec('Cost: 100')); // null

// Отрицательный просмотр назад (ES2018)
const notPrecededByPattern = /(?<!\-)\d+/;
console.log(notPrecededByPattern.exec('10')[0]); // "10"
console.log(notPrecededByPattern.exec('-10')); // null

// Практический пример: извлечение домена из URL
const urlPattern = /(?<=https?:\/\/)[^\s/]+/;
console.log(urlPattern.exec('https://example.com/page')[0]); // "example.com"
```


7. Флаги u и v для работы с Unicode

```
javascript

// Флаг u - полная поддержка Unicode
const unicodePattern = /\p{Script=Cyrillic}+/u;
console.log(unicodePattern.test('Привет')); // true
console.log(unicodePattern.test('Hello')); // false

// Unicode категории и свойства
const unicodeCategories = {
  '\\p{L}': 'любая буква',
  '\\p{N}': 'любая цифра',
  '\\p{P}': 'любой знак пунктуации',
  '\\p{S}': 'любой символ',
  '\\p{Z}': 'любой разделитель',
  '\\p{Emoji}': 'эмодзи',
  '\\p{Script=Latin}': 'латинские буквы',
  '\\p{Script=Cyrillic}': 'кириллические буквы'
};

// Примеры
const anyLetter = /\p{L}+/u;
console.log(anyLetter.test('Hello')); // true
console.log(anyLetter.test('Привет')); // true
console.log(anyLetter.test('123')); // false

const emojiPattern = /\p{Emoji}/u;
console.log(emojiPattern.test('😄')); // true
console.log(emojiPattern.test('Hello')); // false

// Флаг v - расширенная поддержка Unicode (ES2024)
// Позволяет использовать пересечения, объединения и вычитание в классах символов
try {
  const intersectionPattern = /\p{Script=Latin}&&\p{L}]/v;
  console.log('Флаг v поддерживается');
} catch {
  console.log('Флаг v не поддерживается в этой версии JavaScript');
}
```

8. Практические примеры использования

Пример 1: Валидация email

javascript

```
class EmailValidator {
    // Базовый паттерн для валидации email
    static get EMAIL_PATTERN() {
        return /^[a-zA-Z0-9.!#$%&'*/+=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\.[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9]))?*$$/;
    }

    static validate(email) {
        if (!email) {
            return { valid: false, error: 'Email не может быть пустым' };
        }

        const trimmedEmail = email.trim();

        if (!this.EMAIL_PATTERN.test(trimmedEmail)) {
            return { valid: false, error: 'Некорректный формат email' };
        }

        // Проверка максимальной длины
        if (trimmedEmail.length > 254) {
            return { valid: false, error: 'Email слишком длинный' };
        }

        // Проверка домена
        const domain = trimmedEmail.split('@')[1];
        if (domain.length > 63) {
            return { valid: false, error: 'Домен слишком длинный' };
        }

        return { valid: true, normalized: trimmedEmail.toLowerCase() };
    }

    static extractDomain(email) {
        const match = email.match(/@([^\@]+)$/);
        return match ? match[1] : null;
    }
}
```

// Тестирование

```
const testEmails = [
  'user@example.com',
  'first.last@domain.co.uk',
  'user+tag@example.com',
  'invalid-email',
  '@nodomain.com',
  'user@toolongdomain' + 'a'.repeat(60) + '.com'
];

testEmails.forEach(email => {
  console.log(`${email}:`, EmailValidator.validate(email).valid);
});
```

Пример 2: Парсинг CSV строк

javascript

```
class CSVParser {
  static parseLine(line) {
    // Регулярное выражение для разбора CSV с учетом кавычек
    const pattern = /(?:^|,)(?:"([^"]*(?:""[^"]*)*)"|([^\",]*))/g;
    const fields = [];
    let match;

    while ((match = pattern.exec(line)) !== null) {
      // Извлекаем значение (в кавычках или без)
      const value = match[1] !== undefined ?
        match[1].replace(/"/g, '') :
        match[2];
      fields.push(value);
    }

    return fields;
  }

  static parseCSV(csvText) {
    const lines = csvText.split(/\r?\n/);
    return lines
      .filter(line => line.trim() !== '')
      .map(line => this.parseLine(line));
  }

  static generateCSV(data) {
    return data.map(row =>
```

```

    row.map(field => {
        // Экранируем кавычки и заключаем в кавычки при необходимости
        const stringField = String(field);
        if (stringField.includes(',') ||
            stringField.includes('"') ||
            stringField.includes('\n')) {
            return `${stringField.replace(/"/g, '"')}";`
        }
        return stringField;
    }).join(',')
}).join('\n');
}

// Пример использования
const csvData = `Name,Age,City
"John, Doe",30,"New York"
"Jane ""Smith""",25,"Los Angeles"
Bob,35,Chicago`;

const parsed = CSVParser.parseCSV(csvData);
console.log('Parsed CSV:');
console.log(parsed);

const regenerated = CSVParser.generateCSV(parsed);
console.log('\nRegenerated CSV:');
console.log(regenerated);

```

Пример 3: Поиск и замена с использованием функций

```

javascript

class TextProcessor {
    static highlight(text, keywords, options = {}) {
        const {
            caseSensitive = false,
            wholeWord = true,
            tagName = 'mark',
            className = 'highlight'
        } = options;

        // Экранируем специальные символы в ключевых словах
        const escapedKeywords = keywords.map(keyword =>
            keyword.replace(/[\.*+?^${}()|[\]\\"/g, '\\$&')
        );
    }
}

```

```

    // Создаем шаблон поиска
    let pattern;
    if (wholeWord) {
        pattern = `\\b(${escapedKeywords.join('|')})\\b`;
    } else {
        pattern = `(${escapedKeywords.join('|')})`;
    }

    const flags = caseSensitive ? 'g' : 'gi';
    const regex = new RegExp(pattern, flags);

    // Заменяем совпадения
    return text.replace(regex, (match) =>
        `<${tagName} class="${className}">${match}</${tagName}>`
    );
}

static findAndReplace(text, replacements, options = {}) {
    const { caseSensitive = false } = options;

    // Создаем массив пар [шаблон, замена]
    const patterns = Object.entries(replacements).map(([find, replace]) => {
        const escapedFind = find.replace(/[\.*+?^$()|[\]\$\s]/g, '\\$&');
        const pattern = new RegExp(escapedFind, caseSensitive ? 'g' : 'gi');
        return [pattern, replace];
    });

    // Применяем все замены
    let result = text;
    for (const [pattern, replacement] of patterns) {
        result = result.replace(pattern, replacement);
    }

    return result;
}

static extractURLs(text) {
    const urlPattern =
        /https?:\/\/(?:www\.)?[a-zA-Z0-9-]+(?:\.[a-zA-Z]{2,})+(?:\/[^\s]*)?/g;
    return text.match(urlPattern) || [];
}

static extractHashtags(text) {

```

```

        const hashtagPattern = /#[\p{L}\p{N}_]+/gu;
        return text.match(hashtagPattern) || [];
    }

    static extractMentions(text) {
        const mentionPattern = /@[\w.]+/g;
        return text.match(mentionPattern) || [];
    }
}

// Пример использования
const sampleText = `Check out this amazing #JavaScript tutorial by @js_expert!
Visit https://example.com for more info. #coding #webdev`;

console.log('URLs:', TextProcessor.extractURLs(sampleText));
console.log('Hashtags:', TextProcessor.extractHashtags(sampleText));
console.log('Mentions:', TextProcessor.extractMentions(sampleText));

const highlighted = TextProcessor.highlight(sampleText, ['JavaScript', 'tutorial'], {
    tagName: 'strong',
    className: 'keyword'
});
console.log('\nHighlighted text:');
console.log(highlighted);

```

Пример 4: Валидация форм

```

javascript

class FormValidator {
    static patterns = {
        username: /^[a-zA-Z0-9_]{3,20}$/,
        email: /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/,
        phone: /^[+]?[1-9]\\d{1,14}$/, // E.164 формат
        password: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{8,}$/,
        url: /^(https?:\\/\\/)?([\\da-z.-]+)\\.([a-z.]{2,6})([/\\w .-]*)*\\/?$/,
        ipv4: /^(?:(:?25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\\.){3}(:?25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/ ,
        hexColor: /^#([A-Fa-f0-9]{6}|[A-Fa-f0-9]{3})$/,
        dateISO: /^\\d{4}-\\d{2}-\\d{2}$/ ,
        time24h: /^(([01]?[0-9]|2[0-3]):[0-5][0-9])$/
    };

    static validate(field, value, customPattern) {

```

```

const pattern = customPattern || this.patterns[field];

if (!pattern) {
    return { valid: false, error: `Паттерн для ${field} не найден` };
}

if (!pattern.test(value)) {
    return { valid: false, error: this.getErrorMessage(field) };
}

return { valid: true };
}

static getErrorMessage(field) {
    const messages = {
        username: 'Имя пользователя должно содержать 3-20 символов (буквы, цифры, подчеркивания)',
        email: 'Введите корректный email адрес',
        phone: 'Введите корректный номер телефона',
        password: 'Пароль должен содержать минимум 8 символов, включая заглавные и строчные буквы, цифру и специальный символ',
        url: 'Введите корректный URL',
        ipv4: 'Введите корректный IPv4 адрес',
        hexColor: 'Введите корректный hex-цвет (#FFFFFF или #FFF)',
        dateISO: 'Введите дату в формате ГГГГ-ММ-ДД',
        time24h: 'Введите время в 24-часовом формате (ЧЧ:ММ)'
    };

    return messages[field] || 'Некорректное значение';
}

static validateForm(formData) {
    const errors = {};
    const validatedData = {};

    for (const [field, value] of Object.entries(formData)) {
        const result = this.validate(field, value.trim());
        if (!result.valid) {
            errors[field] = result.error;
        } else {
            validatedData[field] = value.trim();
        }
    }
}

```

```

        return {
            valid: Object.keys(errors).length === 0,
            errors,
            data: validatedData
        };
    }
}

// Пример использования
const formData = {
    username: 'john_doe123',
    email: 'john@example.com',
    phone: '+79161234567',
    password: 'StrongPass1!',
    website: 'https://example.com',
    color: '#FF5733',
    birthDate: '1990-01-15',
    appointmentTime: '14:30'
};

const validationResult = FormValidator.validateForm(formData);
console.log('Результат валидации формы:');
console.log('Валидна:', validationResult.valid);
if (!validationResult.valid) {
    console.log('Ошибки:', validationResult.errors);
}

```

9. Производительность и оптимизация

Советы по оптимизации регулярных выражений

```

javascript

class RegexOptimizer {
    // 1. Кэширование регулярных выражений
    static regexCache = new Map();

    static getCachedRegex(pattern, flags = '') {
        const key = `${pattern}|${flags}`;
        if (!this.regexCache.has(key)) {
            this.regexCache.set(key, new RegExp(pattern, flags));
        }
        return this.regexCache.get(key);
    }
}

```



```
}
```

```
// 2. Избегание дорогих конструкций
```

```
static analyzeRegexPerformance(regex, testString, iterations = 10000) {  
    const start = performance.now();  
  
    for (let i = 0; i < iterations; i++) {  
        regex.test(testString);  
    }  
  
    const end = performance.now();  
    return end - start;  
}
```

```
// 3. Сравнение производительности разных подходов
```

```
static comparePerformance() {  
    const testCases = [  
        {  
            name: 'Жадный vs ленивый квантификатор',  
            testString: '<div>content</div><span>more</span>',  
            patterns: [  
                /<.*>/g,          // жадный  
                /<.*?>/g         // ленивый  
            ]  
        },  
        {  
            name: 'Классы символов vs альтернативы',  
            testString: 'abcdefghijklmnopqrstuvwxyz',  
            patterns: [  
                /[aeiou]/g,        // класс символов  
                /a|e|i|o|u/g      // альтернативы  
            ]  
        },  
        {  
            name: 'Группировка vs без группировки',  
            testString: 'word1 word2 word3',  
            patterns: [  
                /(\\w+)/g,         // с захватом  
                /(?:\\w+)/g        // без захвата  
            ]  
        }  
    ];  
  
    testCases.forEach(({ name, testString, patterns }) => {
```

```

        console.log(`\n${name}:`);
        patterns.forEach((pattern, i) => {
            const time = this.analyzeRegexPerformance(pattern, testString, 1000);
            console.log(`  Паттерн ${i + 1}: ${time.toFixed(2)}ms`);
        });
    });
}

// 4. Проверка на катастрофический возврат (catastrophic backtracking)
static checkForBacktracking(regex, testString) {
    const start = performance.now();
    let result;

    try {
        result = regex.test(testString);
    } catch (error) {
        return { error: 'Возможен катастрофический возврат', time: Infinity };
    }

    const end = performance.now();
    const time = end - start;

    return {
        result,
        time,
        warning: time > 100 ? 'Потенциальная проблема с производительностью' : null
    };
}
}

```

// Пример опасного регулярного выражения

```

const dangerousPattern = /(a+)+b/;
const dangerousTest = 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaac';

```

```

console.log('Проверка на катастрофический возврат:');
const result = RegexOptimizer.checkForBacktracking(dangerousPattern, dangerousTest);
console.log(result);

```

10. Отладка и тестирование регулярных выражений

javascript

```

class RegexDebugger {
    static visualize(regex, testString) {

```

```

console.log(`Регулярное выражение: ${regex}`);
console.log(`Тестовая строка: "${testString}"`);
console.log('Флаги:', regex.flags);

// Показываем совпадения
const matches = [...testString.matchAll(regex)];
if (matches.length === 0) {
    console.log('Совпадений не найдено');
    return;
}

console.log(`Найдено совпадений: ${matches.length}`);

matches.forEach((match, index) => {
    console.log(`\nСовпадение ${index + 1}:`);
    console.log(`    Полный текст: "${match[0]}"`);
    console.log(`    Позиция: ${match.index}-${match.index + match[0].length}`);

    if (match.groups) {
        console.log('    Именованные группы:');
        Object.entries(match.groups).forEach(([name, value]) => {
            console.log(`        ${name}: "${value}"`);
        });
    }

    if (match.length > 1) {
        console.log('    Нумерованные группы:');
        for (let i = 1; i < match.length; i++) {
            console.log(`        ${i}: "${match[i]}"`);
        }
    }
});
}

static explain(regexString) {
    const explanations = {
        '^': 'начало строки',
        '$': 'конец строки',
        '.': 'любой символ, кроме перевода строки',
        '*': '0 или более повторений',
        '+': '1 или более повторений',
        '?': '0 или 1 повторение',
        '\\d': 'цифра',
        '\\w': 'словесный символ',
    };

```

```

    '\\s': 'пробельный символ',
    '[': 'класс символов',
    '|': 'или (альтернатива)',
    '()': 'группировка',
    '(?:)': 'незахватывающая группа',
    '(?=)': 'просмотр вперед',
    '(?!)': 'отрицательный просмотр вперед',
    '(?<=)': 'просмотр назад',
    '(?!<)': 'отрицательный просмотр назад',
    '(?<name>)': 'именованная группа захвата'
  };

  console.log('Объяснение регулярного выражения:');
  console.log(` ${regexString}`);
  console.log(`\nЭлементы:`);

  // Простой анализ (упрощенный)
  const elements = regexString.match(/\\\.|\\[\\^?]|\\[\\^?[\\^]]*|\\(\\?:|\\(\\?[<=!]\\.)*?
\\)|\\(\\?<\\..*?>|.\\.|/g) || [];

  elements.forEach(element => {
    if (explanations[element]) {
      console.log(` ${element} → ${explanations[element]}`);
    } else if (element.startsWith('\\')) {
      console.log(` ${element} → экранированный символ или специальный
класс`);
    } else if (element.startsWith '[') {
      console.log(` ${element} → класс символов`);
    } else if (element.startsWith '(') {
      console.log(` ${element} → группа`);
    } else {
      console.log(` ${element} → литерал "${element}"`);
    }
  });
}

static testPattern(regex, testCases) {
  console.log('Тестирование паттерна:');
  console.log(` Регулярное выражение: ${regex}`);

  testCases.forEach(({ input, expected, description = '' }) => {
    const result = regex.test(input);
    const passed = result === expected;
  });
}

```

```

        console.log(` ${passed ? '✓' : 'x'} "${input}" ${description}`);
        if (!passed) {
            console.log(`    Ожидалось: ${expected}, получено: ${result}`);
        }
    });
}

// Пример использования
const emailPattern = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;
const testCases = [
    { input: 'test@example.com', expected: true, description: 'корректный email' },
    { input: 'invalid-email', expected: false, description: 'некорректный email' },
    { input: 'user@domain', expected: false, description: 'нет точки в домене' },
    { input: '@domain.com', expected: false, description: 'нет имени пользователя' }
];

RegexDebugger.testPattern(emailPattern, testCases);

// Визуализация совпадений
const sampleText = 'Contact us at support@example.com or sales@company.co.uk';
const emailRegex = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+/g;
RegexDebugger.visualize(emailRegex, sampleText);

```

11. Заключение

Регулярные выражения — это мощный инструмент, который при правильном использовании может значительно упростить работу с текстом. Ключевые моменты:

1. **Начинайте с простых шаблонов** — не пытайтесь сразу создавать сложные выражения
2. **Тестируйте тщательно** — используйте различные тестовые случаи
3. **Оптимизируйте для производительности** — избегайте катастрофического возврата
4. **Используйте современные возможности** — флаги `u`, `v`, именованные группы
5. **Документируйте сложные выражения** — комментируйте, что делает каждый элемент

Помните: "Если у вас есть проблема и вы решили её с помощью регулярных выражений, теперь у вас есть две проблемы". Используйте регулярные выражения там, где они действительно нужны, и предпочитайте более простые строковые методы для базовых операций.

10.5. Синтаксис регулярных выражений

Введение в синтаксис регулярных выражений

Синтаксис регулярных выражений в JavaScript основан на синтаксисе Perl и включает все основные конструкции, необходимые для работы с текстовыми паттернами. Рассмотрим подробно каждый элемент синтаксиса.

1. Литералы и специальные символы

Базовый синтаксис литералов

```
javascript

// Литералы регулярных выражений обрамляются слешами
const simplePattern = /abc/;

// Специальные символы, которые нужно экранировать
const specialChars = {
  '^': 'начало строки',
  '$': 'конец строки',
  '.': 'любой символ',
  '*': '0 или более повторений',
  '+': '1 или более повторений',
  '?': '0 или 1 повторение',
  '\\': 'экранирование',
  '|': 'альтернатива',
  '(': 'начало группы',
  ')': 'конец группы',
  '[': 'начало класса символов',
  ']': 'конец класса символов',
  '{': 'начало квантификатора',
  '}': 'конец квантификатора'
};

// Экранирование специальных символов
const escapedPatterns = {
  'Поиск точки': /\.\/g,           // Находит буквальную точку
  'Поиск звездочки': /\*/g,        // Находит *
```

```

    'Поиск плюса': /\+/g,           // Находит +
    'Поиск слеша': /\//g,          // Находит \
    'Поиск скобок': /\(\)/g        // Находит ( )
};

// Проверка экранирования
const testString = 'a.b*c+d\\e(f)g';
Object.entries(escapedPatterns).forEach(([description, pattern]) => {
    const matches = testString.match(pattern);
    console.log(`${description}: ${matches ? matches.join(', ') : 'нет совпадений'}`);
});

```

Управляющие последовательности

javascript

```

// Управляющие последовательности для специальных символов
const controlSequences = {
    '\\n': 'новая строка (LF)',
    '\\r': 'возврат каретки (CR)',
    '\\t': 'табуляция',
    '\\v': 'вертикальная табуляция',
    '\\f': 'разрыв страницы',
    '\\0': 'нулевой символ',
    '\\cX': 'управляющий символ (где X - A-Z)'
};

// Примеры использования
const multilineText = 'Line 1\nLine 2\r\nLine 3';
const newlinePattern = /\n/g;
console.log('Найдено переводов строк:', multilineText.match(newlinePattern)?.length);

const tabbedText = 'Column1\tColumn2\tColumn3';
const tabPattern = /\t/g;
console.log('Найдено табуляций:', tabbedText.match(tabPattern)?.length);

// Поиск управляющих символов
const controlText = 'Text with\0null character';
const nullPattern = /\0/;
console.log('Найден нулевой символ:', nullPattern.test(controlText));

```

2. Классы символов

Простые классы символов

```
javascript

// Класс символов определяет набор допустимых символов
const characterClasses = {
  'Буквы а, b или с': /[abc]/,
  'Любая цифра': /[0-9]/,
  'Любая строчная буква': /[a-z]/,
  'Любая заглавная буква': /[A-Z]/,
  'Буквы и цифры': /[a-zA-Z0-9]/,
  'Специальные символы': /[!@#$%^&*()]/,
  'Русские буквы': /[а-яА-ЯёЁ]/
};

// Демонстрация работы классов
const testText = 'abc123DEF!@#';
Object.entries(characterClasses).forEach(([description, pattern]) => {
  const matches = testText.match(pattern);
  console.log(`${description}: ${matches ? matches.slice(0, 3).join(', ') : 'нет'}`);
});

// Классы с использованием Unicode свойств (с флагом u)
const unicodeClasses = {
  'Любая буква': /\p{L}/u,
  'Любая цифра': /\p{N}/u,
  'Любая кириллическая буква': /\p{Script=Cyrillic}/u,
  'Любой эмодзи': /\p{Emoji}/u,
  'Любой символ пунктуации': /\p{P}/u
};

const unicodeText = 'Hello 123 Привет 😊!';
Object.entries(unicodeClasses).forEach(([description, pattern]) => {
  const matches = [...unicodeText.matchAll(pattern)];
  console.log(`${description}: ${matches.length} совпадений`);
});
```

Отрицательные классы символов

```
javascript

// Отрицательные классы исключают символы
const negativeClasses = {
```



```

'Не цифра': /^[^0-9]/,
'Не буква': /^[^a-zA-Z]/,
'Не пробел': /^[^\s]/,
'Не а, b или с': /^[^abc]/,
'Не специальные символы': /^[^!@#$$%^&*()]/
};

// Примеры использования
const mixedText = 'abc123!@#';
const nonDigitPattern = /^[^0-9]/g;
console.log('Не цифры:', mixedText.match(nonDigitPattern));

const nonLetterPattern = /^[^a-zA-Z]/g;
console.log('Не буквы:', mixedText.match(nonLetterPattern));

// Комбинация положительных и отрицательных классов
const complexPattern = /[a-z][^0-9]/;
console.log('Строчная буква, за которой не цифра:', complexPattern.test('a1')); // false
console.log('Строчная буква, за которой не цифра:', complexPattern.test('ab')); // true

```

Сокращенные классы символов

```

javascript

// Предопределенные классы символов
const predefinedClasses = {
  '\\d': 'любая цифра [0-9]',
  '\\D': 'любая не цифра [^0-9]',
  '\\w': 'любой словесный символ [A-Za-z0-9_]',
  '\\W': 'любой не словесный символ [^A-Za-z0-9_]',
  '\\s': 'любой пробельный символ',
  '\\S': 'любой не пробельный символ',
  '\\b': 'граница слова',
  '\\B': 'не граница слова'
};

// Детальное описание пробельных символов
const whitespaceChars = {
  '\\s включает': [
    ' ', // пробел
    '\\t', // табуляция
    '\\n', // перевод строки
    '\\r', // возврат каретки
    '\\f', // разрыв страницы
    '\\v' // вертикальная табуляция
  ]
}

```

```

    ]
};

// Примеры использования
const textWithSpaces = 'Hello\tWorld\nTest\r\nString';
const whitespacePattern = /\s/g;
console.log('Пробельные символы:', textWithSpaces.match(whitespacePattern));

// Границы слов
const wordBoundaryExamples = {
  '\\bcat\\b': 'целое слово "cat"',
  '\\Bcat\\B': '"cat" внутри слова',
  '\\bcat': '"cat" в начале слова',
  'cat\\b': '"cat" в конце слова'
};

const testWords = ['cat', 'category', 'wildcat', 'a cat'];
Object.entries(wordBoundaryExamples).forEach(([pattern, description]) => {
  const regex = new RegExp(pattern);
  console.log(`\n${description} (${pattern}):`);
  testWords.forEach(word => {
    console.log(`  "${word}": ${regex.test(word)}`);
  });
});

```

3. Квантификаторы

Базовые квантификаторы

```

javascript

// Таблица базовых квантификаторов
const basicQuantifiers = {
  '*': { meaning: '0 или более раз', greedy: true, lazy: false },
  '+': { meaning: '1 или более раз', greedy: true, lazy: false },
  '?': { meaning: '0 или 1 раз', greedy: true, lazy: false },
  '*?': { meaning: '0 или более раз (ленивый)', greedy: false, lazy: true },
  '+?': { meaning: '1 или более раз (ленивый)', greedy: false, lazy: true },
  '??': { meaning: '0 или 1 раз (ленивый)', greedy: false, lazy: true }
};

// Демонстрация различий
const demoText = 'aaaabaaa';

```

```
Object.entries(basicQuantifiers).forEach(([quantifier, info]) => {
  const pattern = new RegExp(`a${quantifier}`);
  const match = demoText.match(pattern);
  console.log(`a${quantifier}: ${info.meaning}`);
  console.log(` Совпадение: "${match?.[0]}" (длина: ${match?.[0]?.length})`);
  console.log(` Жадный: ${info.greedy}, Ленивый: ${info.lazy}`);
  console.log('---');
});
```

Количественные квантификаторы

javascript

// Точные количественные квантификаторы

```
const exactQuantifiers = {
  '{n}': 'ровно n раз',
  '{n,}': 'n или более раз',
  '{n,m}': 'от n до m раз',
  '{n}?': 'ровно n раз (ленивый)',
  '{n,}?': 'n или более раз (ленивый)',
  '{n,m}?': 'от n до m раз (ленивый)'
};
```

// Примеры использования

```
const testCases = [
  { pattern: /a{3}/, text: 'aaa', expected: true },
  { pattern: /a{3}/, text: 'aa', expected: false },
  { pattern: /a{3}/, text: 'aaaa', expected: true },
  { pattern: /a{2,4}/, text: 'aa', expected: true },
  { pattern: /a{2,4}/, text: 'aaa', expected: true },
  { pattern: /a{2,4}/, text: 'aaaaa', expected: true },
  { pattern: /a{3,}/, text: 'aaa', expected: true },
  { pattern: /a{3,}/, text: 'aaaaa', expected: true }
];
```

```
console.log('Тестирование количественных квантификаторов:');
testCases.forEach(({ pattern, text, expected }) => {
  const result = pattern.test(text);
  const passed = result === expected;
  console.log(` ${passed ? '✓' : 'x'} ${pattern}: "${text}" -> ${result}`);
});
```

// Практический пример: проверка формата

```
const formatValidators = {
```

```

    'Номер телефона (XXX-XXX-XXXX)': /^\\d{3}-\\d{3}-\\d{4}$/,
    'Почтовый индекс (XXXXX или XXXXX-XXXX)': /^\\d{5}(-\\d{4})?$/,
    'Серийный номер (минимум 6 цифр)': /^\\d{6,}$/,
    'Код продукта (3-5 букв/цифр)': /^[A-Z0-9]{3,5}$/,
    'Версия (X.Y.Z где X,Y,Z - 1-3 цифры)': /^\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}$/
  };

Object.entries(formatValidators).forEach(([description, pattern]) => {
  console.log(`\\n${description}:`);
  console.log(`  Pattern: ${pattern}`);
});

```

4. Группировка

Захватывающие группы

```

javascript

// Захватывающие группы сохраняют совпадение для последующего использования
const capturingGroups = {
  'Простая группа': /(\\d{4})-(\\d{2})-(\\d{2})/,
  'Вложенные группы': /((\\d{2}): (\\d{2})): (\\d{2})/,
  'Группы с альтернативами': /(red|green|blue)/,
  'Множественные группы': /(\\w+)\\s+(\\w+)\\s+(\\w+)/
};

// Извлечение данных из групп
const dateText = 'Today is 2023-12-25';
const datePattern = /(\\d{4})-(\\d{2})-(\\d{2})/;
const dateMatch = datePattern.exec(dateText);

if (dateMatch) {
  console.log('Извлечение даты:');
  console.log('  Полное совпадение:', dateMatch[0]);
  console.log('  Год:', dateMatch[1]);
  console.log('  Месяц:', dateMatch[2]);
  console.log('  День:', dateMatch[3]);
  console.log('  Индекс начала:', dateMatch.index);
  console.log('  Входная строка:', dateMatch.input);
}

// Использование групп в заменах
const nameText = 'John Smith';

```

```

const namePattern = /(\w+)\s+(\w+)/;
const formattedName = nameText.replace(namePattern, '$2, $1');
console.log('Форматирование имени:', formattedName);

// Обратные ссылки на группы
const duplicatePattern = /(\b\w+\b)\s+\1/;
console.log('Поиск повторяющихся слов:');
console.log('  "hello hello":', duplicatePattern.test('hello hello'));
console.log('  "hello world":', duplicatePattern.test('hello world'));

```

Незахватывающие группы

```

javascript

// Незахватывающие группы (?...) не сохраняют совпадение
const nonCapturingExamples = {
  'Группировка без захвата': /(?:https?:\/\/)?(?:www\.)?([^\./]+)/,
  'Альтернативы без захвата': /(?:Mr|Mrs|Ms|Dr)\. (\w+)/,
  'Повторение группы без захвата': /(?:abc)+/
};

// Сравнение захватывающих и не захватывающих групп
const text = 'Mr. Smith and Mrs. Johnson';
const capturingPattern = /(Mr|Mrs|Ms|Dr)\. (\w+)/g;
const nonCapturingPattern = /(?:Mr|Mrs|Ms|Dr)\. (\w+)/g;

console.log('Сравнение групп:');
console.log('Захватывающая группа:');
let match;
while ((match = capturingPattern.exec(text)) !== null) {
  console.log(`  Полное: "${match[0]}", Обращение: "${match[1]}", Имя: "${match[2]}"`);
}

console.log('\nНезахватывающая группа:');
while ((match = nonCapturingPattern.exec(text)) !== null) {
  console.log(`  Полное: "${match[0]}", Имя: "${match[1]}"`);
}

```

Именованные группы захвата

```

javascript

// Именованные группы (?<наме>...) доступны с ES2018
const namedGroupPatterns = {
  'Дата с именованными группами': /(?:<year>\d{4})-(?:<month>\d{2})-(?:<day>\d{2})/,
  'Время с именованными группами': /(?:<hours>\d{2}):(?:<minutes>\d{2}):(?:<seconds>\d{2})/,

```

```

    'Имя и email': /(?!<name>\w+)\s+(<email>[^\s]+)>/,
    'Адрес': /(?!<street>[^,]+\s*(?!<city>[^,]+\s*(?!<zip>\d{5})/

};

// Использование именованных групп
const logEntry = '2023-12-25 14:30:45 INFO User logged in';
const datetimePattern = /(?!<date>\d{4}-\d{2}-\d{2})\s+(<time>\d{2}:\d{2}:\d{2})\s+(<level>\w+)\s+(<message>.*)/;
const logMatch = logEntry.match(datetimePattern);

if (logMatch && logMatch.groups) {
    console.log('Парсинг лога с именованными группами:');
    console.log('  Дата:', logMatch.groups.date);
    console.log('  Время:', logMatch.groups.time);
    console.log('  Уровень:', logMatch.groups.level);
    console.log('  Сообщение:', logMatch.groups.message);
}

// Именованные группы в заменах
const nameEmail = 'John Doe <john@example.com>';
const emailPattern = /(?!<name>\w+\s+\w+)\s+(<email>[^\s]+)>/;
const formatted = nameEmail.replace(emailPattern, '$<email> ($<name>)');
console.log('Форматирование с именованными группами:', formatted);

```

5. Альтернативы

```

javascript

// Вертикальная черта | обозначает альтернативу
const alternatives = {
    'Цвета': /red|green|blue|yellow/,
    'Дни недели': /Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday/,
    'Типы файлов': /\. (txt|csv|json|xml)$/,
    'Протоколы': /https?|ftp|file/
};

// Приоритет в альтернативах
const priorityExamples = {
    'cat|dog': 'совпадает с "cat" или "dog"',
    'cat|dog food': 'совпадает с "cat" или "dog food"',
    '(cat|dog) food': 'совпадает с "cat food" или "dog food"',
    'gray|grey': 'альтернативное написание',
    'Gr(a|e)y': 'группировка с альтернативой'
};

```

// Демонстрация приоритета

```
const testStrings = ['cat', 'dog', 'cat food', 'dog food', 'grey', 'gray'];
```

```
Object.entries(priorityExamples).forEach(([pattern, description]) => {  
  const regex = new RegExp(pattern);  
  console.log(`\n${description} (${pattern}):`);  
  testStrings.forEach(str => {  
    console.log(`  "${str}": ${regex.test(str)}`);  
  });  
});
```

// Сложные альтернативы с группами

```
const timePattern = /(0?[1-9]|1[0-2]):([0-5][0-9])\s*(AM|PM)/i;
```

```
const times = ['9:30 AM', '12:45 PM', '13:00 PM', '9:60 AM'];
```

```
console.log(`\nПроверка времени:`);  
times.forEach(time => {  
  const match = time.match(timePattern);  
  console.log(`  "${time}": ${match ? `✓ ${match[1]}:${match[2]} ${match[3]}` : 'x'}`);  
});
```

6. Привязки и утверждения

Привязки к позициям

javascript

// Якоря привязывают совпадение к позиции в строке

```
const anchors = {  
  '^': 'начало строки (или строки в многострочном режиме)',  
  '$': 'конец строки (или строки в многострочном режиме)',  
  '\\A': 'начало строки (только в некоторых диалектах)',  
  '\\Z': 'конец строки (только в некоторых диалектах)',  
  '\\z': 'абсолютный конец строки'  
};
```

// Примеры использования

```
const anchorExamples = [  
  { pattern: /^Hello/, text: 'Hello World', shouldMatch: true },  
  { pattern: /^Hello/, text: 'Say Hello', shouldMatch: false },  
  { pattern: /World$/, text: 'Hello World', shouldMatch: true },  
  { pattern: /World$/, text: 'World Hello', shouldMatch: false },
```

```

    { pattern: /^Hello World$/, text: 'Hello World', shouldMatch: true },
    { pattern: /^Hello World$/, text: 'Hello World!', shouldMatch: false }
  ];

console.log('Тестирование привязок:');
anchorExamples.forEach(({ pattern, text, shouldMatch }) => {
  const result = pattern.test(text);
  const passed = result === shouldMatch;
  console.log(` ${passed ? '✓' : 'x'} ${pattern}: "${text}" -> ${result}`);
});

// Многострочный режим
const multilineText = `First line
Second line
Third line`;

const startPattern = /^\\w+/g;
const endPattern = /\\w+$/g;

console.log(`\\nМногострочный режим (без флага m):`);
console.log(' Начало строк:', multilineText.match(startPattern));

console.log(`\\nМногострочный режим (с флагом m):`);
console.log(' Начало строк:', multilineText.match(/^\\w+/gm));
console.log(' Конец строк:', multilineText.match(/\\w+$/gm));

```

Утверждения (assertions)

```

javascript

// Утверждения нулевой ширины
const zeroWidthAssertions = {
  '\\b': 'граница слова',
  '\\B': 'не граница слова',
  '(?=...)': 'положительный просмотр вперед',
  '(?!...)': 'отрицательный просмотр вперед',
  '(?<=...)': 'положительный просмотр назад (ES2018)',
  '(?<!...)': 'отрицательный просмотр назад (ES2018)'
};

// Границы слов
const wordBoundaryText = 'hello world test-word under_score';
const wordPattern = /\\b\\w+\\b/g;
console.log('Отдельные слова:', wordBoundaryText.match(wordPattern));

```



```
// Просмотр вперед
const lookaheadExamples = {
  'Слово, за которым следует пробел': /\w+(?=\s)/,
  'Число с процентами': /\d+(?=%)/,
  'Слово, не заканчивающееся на "ing"': /\b\w+(?!ing\b)/
};

const testText = '100% done, running test, quick test';
Object.entries(lookaheadExamples).forEach(([description, pattern]) => {
  const matches = testText.match(pattern);
  console.log(`${description}:`, matches);
});

// Просмотр назад (требуется флаг и в некоторых случаях)
const lookbehindText = '$100 €200 ¥300';
const dollarPattern = /(?<=\$)\d+/g;
const notDollarPattern = /(?<!\$)\d+/g;

console.log('Цены в долларах:', lookbehindText.match(dollarPattern));
console.log('Цены не в долларах:', lookbehindText.match(notDollarPattern));
```

7. Модификаторы (флаги)

javascript

```
// Полный набор флагов в JavaScript
const regexFlags = {
  'g': {
    name: 'global',
    description: 'глобальный поиск (все совпадения)',
    example: () => {
      const regex = /a/g;
      const text = 'banana';
      return `"${text}".match(${regex}) = ${JSON.stringify(text.match(regex))}`;
    }
  },
  'i': {
    name: 'ignoreCase',
    description: 'игнорирование регистра',
    example: () => {
      const regex = /hello/i;
      const tests = ['hello', 'Hello', 'HELLO'];
      const results = tests.map(t => regex.test(t));
    }
  }
};
```

```

        return `${regex.source}/${regex.flags}: ${results.map((r, i) => `${tests[i]}->${
{n}`}).join(', ')}`;
    }
},
'm': {
    name: 'multiline',
    description: 'многострочный режим',
    example: () => {
        const text = 'first\nsecond\nthird';
        const regex = /^w+/gm;
        return `${text.replace(/\n/g, '\\n')}.match(${regex}) = $
{JSON.stringify(text.match(regex))}`;
    }
},
's': {
    name: 'dotAll',
    description: 'точка включает перевод строки',
    example: () => {
        const regex1 = /a.b/;
        const regex2 = /a.b/s;
        const text = 'a\\nb';
        return `${text}: /a.b/.test() = ${regex1.test(text)}, /a.b/s.test() = $
{regex2.test(text)}`;
    }
},
'u': {
    name: 'unicode',
    description: 'полная поддержка Unicode',
    example: () => {
        const regex = /\p{Emoji}/u;
        const text = 'Hello 🤔 World';
        return `${text}.match(${regex}) = ${JSON.stringify(text.match(regex))}`;
    }
},
'y': {
    name: 'sticky',
    description: 'липкий поиск (с позиции lastIndex)',
    example: () => {
        const regex = /a/y;
        const text = 'banana';
        regex.lastIndex = 1;
        const result = regex.test(text);
        return `${regex.source}/${regex.flags}, lastIndex=${regex.lastIndex}: "${text}" ->
${result}`;
    }
}

```

```

    }
  },
  'd': {
    name: 'hasIndices',
    description: 'индексы совпадений (ES2022)',
    example: () => {
      try {
        const regex = /a/d;
        const match = regex.exec('banana');
        return `${regex.source}/${regex.flags}: match.indices = $
{JSON.stringify(match?.indices)}`;
      } catch {
        return 'Флаг d не поддерживается в этой версии';
      }
    }
  }
};

```

// Демонстрация всех флагов

```

console.log('Флаги регулярных выражений в JavaScript:');
Object.entries(regexFlags).forEach(([flag, info]) => {
  console.log(`\n${flag} (${info.name}):`);
  console.log(`  Описание: ${info.description}`);
  console.log(`  Пример: ${info.example()}`);
});

```

8. Расширенные возможности ES6+

Флаг u (Unicode)

javascript

// Подробное изучение флага u

```

const unicodeFeatures = {
  'Кодовые точки вместо кодовых единиц': () => {
    const emoji = '😄';
    const withoutU = /^.$/;
    const withU = /^.$/u;

    return {
      'Без флага u': withoutU.test(emoji),
      'С флагом u': withU.test(emoji),
      'Объяснение': '😄 состоит из 2 кодовых единиц, но 1 кодовой точки'
    };
  }
};

```

```

};
},

'Escape последовательности Unicode': () => {
  const withU = /\u{1F600}/u;
  const withoutU = /\u{1F600}/; // Ищет \u{1F600} как текст

  return {
    'С флагом u': withU.test('😄'),
    'Без флага u': withoutU.test('😄'),
    'Эквивалент': '\\u{1F600} эквивалентно \\uD83D\\uDE00 (суррогатная пара)'
  };
},

'Unicode свойства': () => {
  const patterns = {
    'Любая буква': /\p{L}/u,
    'Любая кириллическая буква': /\p{Script=Cyrillic}/u,
    'Любой эмодзи': /\p{Emoji}/u,
    'Любая десятичная цифра': /\p{Nd}/u
  };

  const testText = 'Привет 123 😄';
  const results = {};

  Object.entries(patterns).forEach(([desc, pattern]) => {
    results[desc] = pattern.test(testText);
  });

  return results;
},

'Unicode категории': () => {
  const categories = {
    '\\p{L}': 'Буква',
    '\\p{Ll}': 'Строчная буква',
    '\\p{Lu}': 'Заглавная буква',
    '\\p{N}': 'Число',
    '\\p{P}': 'Пунктуация',
    '\\p{S}': 'Символ',
    '\\p{Z}': 'Разделитель'
  };

  return categories;
}

```

```

    }
  };

  console.log('Флаг u - расширенные возможности Unicode:');
  Object.entries(unicodeFeatures).forEach(([feature, func]) => {
    console.log(`\n${feature}:`);
    const result = func();
    if (typeof result === 'object') {
      Object.entries(result).forEach(([key, value]) => {
        console.log(`  ${key}: ${value}`);
      });
    } else {
      console.log(`  ${result}`);
    }
  });
};

```

Флаг v (расширенный Unicode, ES2024)

```

javascript

// Флаг v предоставляет дополнительные возможности для работы с Unicode
try {
  const vFlagFeatures = {
    'Пересечение классов': () => {
      // Ищет символы, которые являются и буквами, и кириллическими
      const pattern = /[p{Letter}\p{Script=Cyrillic}]/v;
      return pattern.test('Привет'); // true
    },

    'Объединение классов': () => {
      // Ищет символы, которые являются либо буквами, либо цифрами
      const pattern = /[p{Letter}|\p{Number}]/v;
      return pattern.test('A1'); // true
    },

    'Вычитание классов': () => {
      // Ищет буквы, кроме кириллических
      const pattern = /[p{Letter}--\p{Script=Cyrillic}]/v;
      return {
        'Латинская A': pattern.test('A'), // true
        'Кириллическая A': pattern.test('А'), // false
        'Цифра': pattern.test('1') // false
      };
    },
  };
};

```

```

    'Симметричная разность': () => {
        // Ищет символы, которые в одном классе, но не в другом
        const pattern = /[\\p{Letter}~~\\p{ASCII}]/v;
        return pattern.test('А'); // true (кириллическая, не ASCII)
    }
};

console.log(`\nФлаг v (ES2024) - если поддерживается: `);
Object.entries(vFlagFeatures).forEach(([feature, func]) => {
    console.log(`\n${feature}:`);
    const result = func();
    console.log(`  Результат:`, result);
});
} catch (error) {
    console.log(`\nФлаг v не поддерживается в этой версии JavaScript`);
}

```

9. Синтаксис для работы с индексами (флаг d)

```

javascript

// Флаг d добавляет информацию об индексах совпадений
try {
    const indexFeatures = {
        'Базовое использование': () => {
            const regex = /a(b)/d;
            const match = regex.exec('abc');
            return {
                'match[0]': match[0],
                'match[1]': match[1],
                'indices[0]': match.indices[0],
                'indices[1]': match.indices[1],
                'Объяснение': 'indices содержит массивы [начало, конец] для каждой группы'
            };
        },

        'Именованные группы с индексами': () => {
            const regex = /(?!<letter>a)(?!<digit>d)/d;
            const match = regex.exec('a1');
            return {
                'Группы': match.groups,
                'Индексы групп': match.indices.groups,
                'Доступ': `match.indices.groups.letter = $
{JSON.stringify(match.indices.groups.letter)}`
            };
        }
    };
}

```

```

    };
  },

  'Глобальный поиск с индексами': () => {
    const regex = /\w+/dg;
    const text = 'Hello World';
    const matches = [...text.matchAll(regex)];
    return matches.map(m => ({
      'слово': m[0],
      'индексы': m.indices[0]
    }));
  }
};

console.log('\nФлаг d - работа с индексами:');
Object.entries(indexFeatures).forEach(([feature, func]) => {
  console.log(`\n${feature}:`);
  const result = func();
  if (Array.isArray(result)) {
    result.forEach(item => {
      console.log(`  ${JSON.stringify(item)}`);
    });
  } else {
    Object.entries(result).forEach(([key, value]) => {
      console.log(`  ${key}: ${JSON.stringify(value)}`);
    });
  }
});
} catch (error) {
  console.log('\nФлаг d не поддерживается в этой версии JavaScript');
}

```

10. Специальные синтаксические конструкции

Условные выражения в регулярных выражениях

javascript

```

// Условные выражения (?condition)yes/no
// Этот синтаксис не поддерживается в JavaScript напрямую,
// но его можно эмулировать с помощью просмотра вперед/назад

```

```

const conditionalPatterns = {

```

```

'Проверка наличия префикса': () => {
  // Если есть префикс "Mr. ", то ищем одну фамилию, иначе имя и фамилию
  const text1 = 'Mr. Smith';
  const text2 = 'John Smith';

  // Эмуляция с помощью просмотра вперед
  const pattern = /(?:Mr\.\s+)?(\w+)(?:\s+(\w+))?/;

  return {
    'Mr. Smith': text1.match(pattern)?.slice(1).filter(Boolean),
    'John Smith': text2.match(pattern)?.slice(1).filter(Boolean)
  };
},

'Проверка формата даты': () => {
  // Разные форматы даты: YYYY-MM-DD или DD/MM/YYYY
  const dates = ['2023-12-25', '25/12/2023', 'invalid'];

  const pattern = /(\d{4})-(\d{2})-(\d{2})|(\d{2})\/(\d{2})\/(\d{4})/;

  return dates.map(date => {
    const match = date.match(pattern);
    if (!match) return { date, valid: false };

    if (match[1]) {
      return { date, format: 'ISO', year: match[1], month: match[2], day: match[3] };
    } else {
      return { date, format: 'EU', day: match[4], month: match[5], year: match[6] };
    }
  });
}

};

console.log('\nУсловные конструкции (эмуляция):');
Object.entries(conditionalPatterns).forEach(([description, func]) => {
  console.log(`\n${description}:`);
  const result = func();
  if (Array.isArray(result)) {
    result.forEach(item => {
      console.log(`  ${JSON.stringify(item)}`);
    });
  } else {
    Object.entries(result).forEach(([key, value]) => {
      console.log(`  ${key}: ${JSON.stringify(value)}`);
    });
  }
});

```



```
    });  
  }  
});
```

Рекурсивные шаблоны

javascript

*// Рекурсия в регулярных выражениях (не поддерживается напрямую в JavaScript)
// Но можно эмулировать для определенных случаев*

```
const recursivePatterns = {  
  'Сбалансированные скобки (ограниченная глубина)': () => {  
    // Максимум 3 уровня вложенности  
    const pattern = /\(((\[^\()]\)|\(((\[^\()]\)|\([^\()]*\))*\))*\)/;  
  
    const tests = [  
      '(simple)',  
      '(nested (parentheses))',  
      '(deeply (nested (parentheses)))',  
      '(too (deeply (nested (parentheses))))'  
    ];  
  
    return tests.map(test => ({  
      test,  
      matches: pattern.test(test)  
    }));  
  },  
  
  'Палиндромы (фиксированной длины)': () => {  
    // Палиндромы длиной 3-5 символов  
    const pattern = /^(.)(.)(.)\3\2\1$|^(.)(.)\2\1$|^(.)\1$/;  
  
    const tests = ['aa', 'aba', 'abba', 'abcba', 'abccba'];  
  
    return tests.map(test => ({  
      test,  
      isPalindrome: pattern.test(test)  
    }));  
  }  
};  
  
console.log(`\nРекурсивные шаблоны (эмуляция):`);  
Object.entries(recursivePatterns).forEach(([description, func]) => {  
  console.log(`\n${description}:`);
```

```

const results = func();
results.forEach(result => {
  console.log(`  "${result.test}": ${result.matches || result.isPalindrome}`);
});
});

```

11. Практические примеры сложных выражений

javascript

```

class ComplexRegexExamples {
  // Валидация сложных паролей
  static get PASSWORD_PATTERN() {
    // Минимум 8 символов, минимум одна цифра, одна строчная,
    // одна заглавная буква, один специальный символ
    return /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#$%^&*()_+\-=\[\]{};':"\"|,.<>\/?]).{8,}$/;
  }

  // Парсинг сложных логов
  static get LOG_PATTERN() {
    // [TIMESTAMP] LEVEL [MODULE] MESSAGE (key=value key2=value2)
    return /^\[(<timestamp>[^\]]+)\]\s\[(<level>\w+)\]\s\[(<module>[^\]]+)\]\s(<message>[^\]]+)(?:\((?<params>[^\)]+\))?)?$/;
  }

  // Извлечение данных из SQL запроса
  static get SQL_SELECT_PATTERN() {
    // Простой парсинг SELECT запросов
    return /SELECT\s+(?<columns>.+?)\s+FROM\s+(?<table>\w+)(?:\s+WHERE\s+(?<condition>.+?))?(?:\s+ORDER BY\s+(?<order>.+?))?(?:\s+LIMIT\s+(?<limit>\d+))?$ /i;
  }

  // Парсинг сложных URL с параметрами
  static get URL_PATTERN() {
    return /^(?<protocol>https?):\/\/(?<domain>[^\/:.]+)(?::(?<port>\d+))?(?<path>\/[^\?#]*)?(?:(?<query>[^\?#]*)?(?::#(?<fragment>.*)))?$/;
  }

  // Валидация кредитных карт (упрощенная)
  static get CREDIT_CARD_PATTERN() {
    // Формат: XXXX-XXXX-XXXX-XXXX или XXXX XXXX XXXX XXXX
    return /^(?:\d{4}[-\s]?){3}\d{4}$/;
  }
}

```

```

// Парсинг математических выражений (упрощенный)
static get MATH_EXPRESSION_PATTERN() {
  return /(?(<left>\d+(?:\.\d+)?)\s*(?(operator>[+\-*/])\s*(?(right>\d+(?:\.\d+)?))/;
}

// Тестирование всех паттернов
static testAllPatterns() {
  const testCases = {
    password: [
      { input: 'Weak1', expected: false },
      { input: 'StrongPass1!', expected: true },
      { input: 'NoSpecial1', expected: false }
    ],

    log: [
      {
        input: '[2023-12-25 14:30:45] INFO [AuthModule] User logged in (id=123 session=abc)',
        shouldMatch: true
      },
      {
        input: 'invalid log entry',
        shouldMatch: false
      }
    ],

    sql: [
      {
        input: 'SELECT id, name FROM users WHERE active = 1 ORDER BY name LIMIT 10',
        shouldMatch: true
      }
    ],

    url: [
      {
        input: 'https://example.com:8080/path/to/page?param=value#section',
        shouldMatch: true
      }
    ],

    creditCard: [
      { input: '1234-5678-9012-3456', expected: true },
      { input: '1234 5678 9012 3456', expected: true },
    ]
  }
}

```

```

        { input: '1234567890123456', expected: true }
    ]
};

console.log('Тестирование сложных регулярных выражений:');
Object.entries(testCases).forEach(([type, cases]) => {
    console.log(`\n${type.toUpperCase()}:`);
    const pattern = this[`${type.toUpperCase()}_PATTERN`];
    cases.forEach(({ input, expected, shouldMatch }) => {
        const result = pattern.test(input);
        const passed = expected !== undefined ?
            result === expected :
            shouldMatch ? result : !result;

        console.log(`  ${passed ? '✓' : '✗'} "${input.substring(0, 30)}..." -> ${result}`);

        // Если есть группы захвата, покажем их
        if (result) {
            const match = input.match(pattern);
            if (match?.groups) {
                console.log('    Группы:', match.groups);
            }
        }
    });
});
});
}

// Запуск тестов
ComplexRegexExamples.testAllPatterns();

```

12. Шпаргалка по синтаксису

javascript

```

// Полная шпаргалка по синтаксису регулярных выражений
const regexCheatSheet = {
    // Базовые элементы
    basics: {
        '^': 'начало строки',
        '$': 'конец строки',
        '.': 'любой символ кроме \n',
        '|': 'альтернатива (или)',
    },
};

```

```
    '\\': 'экранирование'  
},
```

```
// Квантификаторы
```

```
quantifiers: {  
    '*': '0 или более',  
    '+': '1 или более',  
    '?': '0 или 1',  
    '{n}': 'ровно n раз',  
    '{n,}': 'n или более',  
    '{n,m}': 'от n до m',  
    '*?': '0+ (ленивый)',  
    '+?': '1+ (ленивый)',  
    '??': '0-1 (ленивый)'  
},
```

```
// Классы символов
```

```
characterClasses: {  
    '[abc]': 'a, b или c',  
    '^abc': 'не a, b, c',  
    '[a-z]': 'от a до z',  
    '[0-9]': 'цифра',  
    '[a-zA-Z]': 'любая буква',  
    '\\d': 'цифра [0-9]',  
    '\\D': 'не цифра',  
    '\\w': 'словесный символ',  
    '\\W': 'не словесный символ',  
    '\\s': 'пробельный символ',  
    '\\S': 'не пробельный символ'  
},
```

```
// Группировка
```

```
grouping: {  
    '()': 'захватывающая группа',  
    '(?:)': 'незахватывающая группа',  
    '(?<name>)': 'именованная группа',  
    '\\1': 'обратная ссылка на группу 1'  
},
```

```
// Утверждения
```

```
assertions: {  
    '\\b': 'граница слова',  
    '\\B': 'не граница слова',  
    '(?=)': 'просмотр вперед',
```

```

    '(!)': 'отрицательный просмотр вперед',
    '(?<=)': 'просмотр назад',
    '(?!<)': 'отрицательный просмотр назад'
  },

  // Флаги
  flags: {
    'g': 'глобальный поиск',
    'i': 'игнорирование регистра',
    'm': 'многострочный режим',
    's': 'точка включает \n',
    'u': 'Unicode режим',
    'y': 'липкий поиск',
    'd': 'индексы совпадений'
  },

  // Unicode свойства (с флагом u)
  unicodeProperties: {
    '\\p{L}': 'любая буква',
    '\\p{N}': 'любая цифра',
    '\\p{P}': 'знак пунктуации',
    '\\p{S}': 'символ',
    '\\p{Z}': 'разделитель',
    '\\p{Emoji}': 'эмодзи',
    '\\p{Script=Cyrillic}': 'кириллица'
  }
};

// Вывод шпаргалки
console.log('\n\nШПАРГАЛКА ПО СИНТАКСИСУ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ');
console.log('=' .repeat(60));

Object.entries(regexCheatSheet).forEach(([section, items]) => {
  console.log(`\n${section.toUpperCase()}:`);
  console.log('-'.repeat(40));
  Object.entries(items).forEach(([pattern, description]) => {
    console.log(`  ${pattern.padEnd(20)} ${description}`);
  });
});

```

10.5.13. Заключение

Синтаксис регулярных выражений в JavaScript является мощным и выразительным инструментом, который постоянно развивается. Ключевые моменты для запоминания:

1. **Начните с основ** — освоите базовые элементы перед переходом к сложным конструкциям
2. **Используйте современные возможности** — флаги `u`, `v`, именованные группы
3. **Тестируйте регулярные выражения** — используйте онлайн-тестеры и создавайте юнит-тесты
4. **Оптимизируйте для производительности** — избегайте катастрофического возврата
5. **Документируйте сложные выражения** — комментируйте или используйте именованные группы для ясности

Помните: "Некоторые люди, столкнувшись с проблемой, думают: 'Я знаю, я использую регулярные выражения'. Теперь у них две проблемы." — Джейми Завински. Используйте регулярные выражения обдуманно и там, где они действительно необходимы.

10.6. Методы RegExp и String для работы с регулярными выражениями

Введение

JavaScript предоставляет богатый набор методов для работы с регулярными выражениями как в объекте `RegExp`, так и в объекте `String`. Понимание различий и особенностей каждого метода критически важно для эффективной работы с текстом.

1. Методы объекта RegExp

`RegExp.prototype.test()`

Метод `test()` проверяет, есть ли в строке совпадение с регулярным выражением.

```
javascript

// Базовое использование
const pattern = /hello/;
console.log(pattern.test('hello world')); // true
console.log(pattern.test('goodbye world')); // false

// С флагом g (глобальный поиск) - важное поведение!
const globalPattern = /hello/g;
const text = 'hello hello hello';

console.log(globalPattern.test(text)); // true
console.log(globalPattern.lastIndex); // 6 (позиция после первого совпадения)
console.log(globalPattern.test(text)); // true
console.log(globalPattern.lastIndex); // 12
console.log(globalPattern.test(text)); // true
console.log(globalPattern.lastIndex); // 18
console.log(globalPattern.test(text)); // false (больше нет совпадений)
console.log(globalPattern.lastIndex); // 0 (сбрасывается)

// Практический пример: проверка сложных условий
const passwordPattern = /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}$/;
```



```

const passwords = [
  'weak',
  'weak123',
  'WEAK123',
  'Weak123',
  'StrongPassword1'
];

passwords.forEach(pwd => {
  console.log(`${pwd}: ${passwordPattern.test(pwd) ? '✓' : 'x'}`);
});

```

// Создание тестирующей функции с кэшированием

```

function createTester(pattern) {
  let lastPattern = null;
  let regex = null;

  return function(text) {
    if (lastPattern !== pattern) {
      regex = new RegExp(pattern);
      lastPattern = pattern;
    }
    return regex.test(text);
  };
}

```

```

const isEmail = createTester('^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$');
console.log(isEmail('test@example.com')); // true
console.log(isEmail('invalid-email')); // false

```

RegExp.prototype.exec()

Метод `exec()` выполняет поиск совпадения и возвращает подробную информацию.

javascript

// Базовое использование

```

const datePattern = /(\d{4})-(\d{2})-(\d{2})/;
const dateMatch = datePattern.exec('Today is 2023-12-25');

if (dateMatch) {
  console.log('Полное совпадение:', dateMatch[0]); // "2023-12-25"
  console.log('Группа 1 (год):', dateMatch[1]); // "2023"
}

```

```

    console.log('Группа 2 (месяц):', dateMatch[2]); // "12"
    console.log('Группа 3 (день):', dateMatch[3]); // "25"
    console.log('Индекс начала:', dateMatch.index); // 9
    console.log('Входная строка:', dateMatch.input); // "Today is 2023-12-25"
    console.log('Именованные группы:', dateMatch.groups); // undefined (если нет
именованных групп)
}

// С флагом g (итерация по всем совпадениям)
const wordPattern = /\w+/g;
const text = 'hello world from javascript';
let match;
const words = [];

while ((match = wordPattern.exec(text)) !== null) {
    console.log(`Найдено: "${match[0]}" на позиции ${match.index}`);
    words.push(match[0]);
}

console.log('Все слова:', words);

// С именованными группами (ES2018+)
const namedPattern = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;
const namedMatch = namedPattern.exec('2023-12-25');

if (namedMatch) {
    console.log('Именованные группы:', namedMatch.groups);
    // {year: "2023", month: "12", day: "25"}
    console.log('Год:', namedMatch.groups.year); // "2023"
}

// С флагом d (индексы, ES2022+)
try {
    const indexPattern = /(a)(b)/d;
    const indexMatch = indexPattern.exec('ab');

    if (indexMatch) {
        console.log('Индексы совпадений:', indexMatch.indices);
        // [[0, 2], [0, 1], [1, 2]]
        console.log('Индексы групп:', indexMatch.indices.slice(1));
        // [[0, 1], [1, 2]]
    }
} catch (e) {
    console.log('Флаг d не поддерживается');
}

```

```

}

// Практический пример: парсинг логов
function parseLogEntry(logEntry) {
  const logPattern = /^\[ (?<timestamp>[^\]]+)\] \[ (?<level>\w+) \[ (?<module>[^\]]+)\] \[ (?<message>.+)\]$/;
  const match = logPattern.exec(logEntry);

  if (!match) {
    return null;
  }

  return {
    timestamp: match.groups.timestamp,
    level: match.groups.level,
    module: match.groups.module,
    message: match.groups.message,
    raw: match[0]
  };
}

const logEntries = [
  '[2023-12-25 14:30:45] INFO [Auth] User logged in',
  '[2023-12-25 14:31:10] ERROR [Database] Connection failed',
  'invalid log entry'
];

logEntries.forEach(entry => {
  const parsed = parseLogEntry(entry);
  console.log(parsed ? `✓ ${parsed.level}: ${parsed.message}` : `✗ Invalid: ${entry}`);
});

```

RegExp.prototype.toString()

Возвращает строковое представление регулярного выражения.

```

javascript

const regex = /hello/gi;
console.log(regex.toString()); // "/hello/gi"
console.log(String(regex));    // "/hello/gi"
console.log('' + regex);       // "/hello/gi"

```

// Полезно для логгирования

```

function logRegex(regex) {
  console.log(`Pattern: ${regex.source}`);
  console.log(`Flags: ${regex.flags}`);
  console.log(`String representation: ${regex}`);
}

logRegex(/test/gi);

// Создание из строки и обратно
const patternStr = '/hello\\sworld/gi';
const regexFromStr = eval(patternStr); // Осторожно! eval опасен
console.log(regexFromStr.test('Hello World')); // true

// Безопасная альтернатива
function safeRegexFromString(str) {
  const match = str.match(/^\/(.*)\/([gimsuy]*)$/);
  if (!match) {
    throw new Error('Invalid regex string format');
  }
  return new RegExp(match[1], match[2]);
}

const safeRegex = safeRegexFromString('/hello\\sworld/gi');
console.log(safeRegex.test('Hello World')); // true

```

2. Свойства объекта RegExp

Статические свойства

```

javascript

// RegExp имеет несколько полезных статических свойств

// RegExp.$1-$9 - последние захваченные группы (устаревшее)
const pattern = /(\w+)\s+(\w+)/;
pattern.exec('John Doe');
console.log('RegExp.$1:', RegExp.$1); // "John"
console.log('RegExp.$2:', RegExp.$2); // "Doe"

// Внимание: эти свойства глобальны и небезопасны!
/(a)/.exec('a');
/(b)/.exec('b');
console.log('RegExp.$1 сейчас:', RegExp.$1); // "b" - перезаписались!

```

```

// RegExp.input или RegExp.$_ - последняя входная строка
/(test)/.exec('this is a test');
console.log('RegExp.input:', RegExp.input); // "this is a test"

// RegExp.lastMatch или RegExp['$&'] - последнее совпадение
/(test)/.exec('this is a test');
console.log('RegExp.lastMatch:', RegExp.lastMatch); // "test"

// RegExp.lastParen или RegExp['$+'] - последняя захваченная группа
/(te)(st)/.exec('test');
console.log('RegExp.lastParen:', RegExp.lastParen); // "st"

// Совет: избегайте использования этих свойств в современном коде!

```

Свойства экземпляра

```

javascript

// Каждый экземпляр RegExp имеет полезные свойства
const regex = /(\d{4})-(\d{2})-(\d{2})/g;

console.log('regex.source:', regex.source); // "(\d{4})-(\d{2})-(\d{2})"
console.log('regex.flags:', regex.flags); // "g"
console.log('regex.global:', regex.global); // true
console.log('regex.ignoreCase:', regex.ignoreCase); // false
console.log('regex.multiline:', regex.multiline); // false
console.log('regex.dotAll:', regex.dotAll); // false
console.log('regex.unicode:', regex.unicode); // false
console.log('regex.sticky:', regex.sticky); // false
console.log('regex.hasIndices:', regex.hasIndices); // false (ES2022)

// Свойства, зависящие от состояния
regex.exec('2023-12-25');
console.log('regex.lastIndex:', regex.lastIndex); // 10 (позиция после совпадения)

// Практическое использование
function analyzeRegex(pattern) {
  const regex = new RegExp(pattern);

  return {
    pattern: regex.source,
    flags: regex.flags,
    isGlobal: regex.global,
    isCaseInsensitive: regex.ignoreCase,
  };
}

```

```

    isMultiline: regex.multiline,
    dotMatchesNewline: regex.dotAll,
    supportsUnicode: regex.unicode,
    isSticky: regex.sticky,
    hasMatchIndices: regex.hasIndices
  };
}

console.log(analyzeRegex('/hello/gi'));
console.log(analyzeRegex('test', 'u'));

```

3. Методы объекта String для работы с регулярными выражениями

String.prototype.match()

Метод `match()` возвращает массив совпадений или `null`.

```

javascript

// Без флага g - возвращает первое совпадение с группами
const dateText = 'Dates: 2023-12-25 and 2024-01-01';
const dateMatch = dateText.match(/(\d{4})-(\d{2})-(\d{2})/);

if (dateMatch) {
  console.log('Полное совпадение:', dateMatch[0]); // "2023-12-25"
  console.log('Группы:', dateMatch.slice(1));      // ["2023", "12", "25"]
  console.log('Индекс:', dateMatch.index);          // 7
  console.log('Вход:', dateMatch.input);             // исходная строка
}

// С флагом g - возвращает все совпадения (без групп)
const allDates = dateText.match(/\d{4}-\d{2}-\d{2}/g);
console.log('Все даты:', allDates); // ["2023-12-25", "2024-01-01"]

// С флагом g и именованными группами - группы не сохраняются!
const namedMatch = dateText.match(/(<year>\d{4})-(<month>\d{2})-(<day>\d{2})/g);
console.log('С именованными группами (флаг g):', namedMatch); // Только полные совпадения

// Сравнение с exec()
const text = 'test1 test2 test3';
const pattern = /test\d/g;

```

```

// match() с флагом g
const matchResult = text.match(pattern);
console.log('match() с флагом g:', matchResult); // ["test1", "test2", "test3"]

// exec() в цикле
const execResults = [];
let execMatch;
while ((execMatch = pattern.exec(text)) !== null) {
  execResults.push({
    match: execMatch[0],
    index: execMatch.index,
    groups: execMatch.groups
  });
}
console.log('exec() в цикле:', execResults);

// Практический пример: извлечение всех email-адресов
function extractEmails(text) {
  const emailPattern = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g;
  return text.match(emailPattern) || [];
}

const emailText = `
  Contact us at support@example.com or sales@company.co.uk.
  For issues, email: admin@test.org.
`;

console.log('Извлеченные email:', extractEmails(emailText));

```

String.prototype.matchAll() (ES2020)

Метод `matchAll()` возвращает итератор всех совпадений с сохранением групп.

javascript

```

// matchAll требует флага g
const text = 'test1 test2 test3';
const pattern = /t(e)(st(\d?))/g;

// Использование matchAll
const matches = [...text.matchAll(pattern)];

matches.forEach((match, index) => {

```

```

    console.log(`Совпадение ${index + 1}:`);
    console.log(' Полное:', match[0]);
    console.log(' Группы:', match.slice(1));
    console.log(' Индекс:', match.index);
    console.log(' Именованные группы:', match.groups);
  });

```

// Практический пример: парсинг CSV строки

```

function parseCSVLine(line) {
  const csvPattern = /(?:^|,)("(?:(?:\\"|"[^"])*"|[,]*)/g;
  const fields = [];

  for (const match of line.matchAll(csvPattern)) {
    let field = match[1];

    // Обработка кавычек
    if (field.startsWith('"') && field.endsWith('"')) {
      field = field.slice(1, -1).replace(/"/g, '');
    }

    fields.push(field);
  }

  return fields;
}

```

```

const csvLine = 'John,"Doe, Jr.",30,"New ""York""",USA';
console.log('Парсинг CSV:', parseCSVLine(csvLine));

```

// Еще пример: извлечение всех тегов с атрибутами

```

function parseHTMLTags(html) {
  const tagPattern = /<(\w+)([>]*)>/g;
  const tags = [];

  for (const match of html.matchAll(tagPattern)) {
    tags.push({
      tag: match[1],
      attributes: match[2].trim(),
      fullMatch: match[0],
      position: match.index
    });
  }

  return tags;
}

```



```

}

const html = '<div class="container"><p id="text">Hello</p></div>';
console.log('HTML теги:', parseHTMLTags(html));

// matchAll с именованными группами
const logPattern = /\[(?<time>\d{2}:\d{2}:\d{2})\] (?<level>\w+): (?<message>.+)/g;
const logs = `
  [14:30:45] INFO: User logged in
  [14:31:10] ERROR: Database connection failed
  [14:32:00] WARN: High memory usage
`;

for (const match of logs.matchAll(logPattern)) {
  console.log(`${match.groups.time} [${match.groups.level}] ${match.groups.message}`);
}

```

String.prototype.search()

Метод `search()` возвращает индекс первого совпадения или -1.

```

javascript

// Простой поиск
const text = 'Hello world!';
console.log(text.search(/world/)); // 6
console.log(text.search(/World/)); // -1 (регистр)
console.log(text.search(/World/i)); // 6 (игнорирование регистра)

// Поиск с границами слов
console.log('cat'.search(/\bcat\b/)); // 0
console.log('category'.search(/\bcat\b/)); // -1

// Практический пример: проверка наличия паттерна
function containsPattern(text, pattern) {
  return text.search(pattern) !== -1;
}

const sampleText = 'The quick brown fox jumps over the lazy dog';

console.log('Содержит "fox"?', containsPattern(sampleText, /fox/));
console.log('Содержит "cat"?', containsPattern(sampleText, /cat/));
console.log('Содержит слово из 3 букв?', containsPattern(sampleText, /\b\w{3}\b/));

```

```
// Оптимизация: кэширование регулярных выражений
const patternCache = new Map();

function cachedSearch(text, patternStr, flags = '') {
  const key = `${patternStr}|${flags}`;

  if (!patternCache.has(key)) {
    patternCache.set(key, new RegExp(patternStr, flags));
  }

  const pattern = patternCache.get(key);
  return text.search(pattern);
}

// Использование
console.log(cachedSearch(sampleText, 'fox'));
console.log(cachedSearch(sampleText, 'dog'));
console.log('Кэш размер:', patternCache.size);
```

String.prototype.replace()

Метод `replace()` заменяет совпадения в строке.

```
javascript

// Простая замена строки
let text = 'Hello world!';
console.log(text.replace('world', 'JavaScript')); // "Hello JavaScript!"

// Замена с использованием регулярного выражения
text = 'apple, banana, cherry';
console.log(text.replace(/a/g, 'o')); // "opple, bonono, cherry"

// Замена с обратными ссылками
text = 'John Smith';
console.log(text.replace(/(\w+)\s+(\w+)/, '$2, $1')); // "Smith, John"

// Замена с функцией-заменителем
text = 'The price is $100.50';
const result = text.replace(/\$(\d+(?:\.\d{2})?)/g, (match, amount) => {
  const num = parseFloat(amount);
  return `${num * 1.1}.toFixed(2)}`; // +10%
});
console.log('После повышения цены:', result);
```

```

// Именованные группы в заменах
text = 'Date: 2023-12-25';
console.log(text.replace(
  /(?!<year>\d{4})-(?!<month>\d{2})-(?!<day>\d{2})/,
  '$<day>/<month>/<year>'
)); // "Date: 25/12/2023"

// Многострочная замена
text = `Line 1: error
Line 2: warning
Line 3: info`;

console.log(text.replace(/^Line (\d+):/gm, 'Строка $1:'));

// Практический пример: маскирование данных
function maskSensitiveData(text) {
  return text
    .replace(/(\d{16})|(\d{4}[-\s]?(\d{4}[-\s]?(\d{4}[-\s]?(\d{4}))/g, '****-****-****-****')
    .replace(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g, '***@***.***')
    .replace(/(\b\d{3}[-\s]?(\d{3}[-\s]?(\d{4})\b)/g, '***-***-****');
}

const sensitiveText = `
  Credit card: 4111-1111-1111-1111
  Email: john.doe@example.com
  Phone: 555-123-4567
`;

console.log('Маскированные данные:', maskSensitiveData(sensitiveText));

```

String.prototype.replaceAll() (ES2021)

Метод `replaceAll()` заменяет все вхождения подстроки.

```

javascript

// Замена строки
let text = 'hello hello hello';
console.log(text.replaceAll('hello', 'world')); // "world world world"

// Замена с регулярным выражением (обязателен флаг g)
text = 'a1 b2 c3';
console.log(text.replaceAll(/d/g, 'X')); // "aX bX cX"

```

```

// Ошибка без флага g
try {
  text.replaceAll(/\d/, 'X'); // Ошибка!
} catch (e) {
  console.log('Ошибка:', e.message);
}

// Использование функции-заменителя
text = 'Items: 1, 2, 3, 4, 5';
const doubled = text.replaceAll(/\d+/g, match => String(parseInt(match) * 2));
console.log('Удвоенные:', doubled); // "Items: 2, 4, 6, 8, 10"

// Практический пример: нормализация текста
function normalizeText(text) {
  return text
    .replaceAll(/\s+/g, ' ') // Множественные пробелы в один
    .replaceAll(/[^\w\s.,!~-]/g, '') // Удаление специальных символов
    .replaceAll(/\b(the|a|an)\b/gi, '') // Удаление стоп-слов
    .trim();
}

const messyText = ' The quick-brown fox!!! jumps over the lazy dog. ';
console.log('Нормализованный текст:', normalizeText(messyText));

```

String.prototype.split()

Метод `split()` разделяет строку на массив подстрок.

```

javascript

// Разделение по строке
const text = 'apple,banana,cherry';
console.log(text.split(',')); // ["apple", "banana", "cherry"]

// Разделение по регулярному выражению
const complexText = 'apple, banana; cherry.orange';
console.log(complexText.split(/[;,.\s]+/)); // ["apple", "banana", "cherry", "orange"]

// Ограничение количества элементов
console.log(text.split(',', 2)); // ["apple", "banana"]

// Сохранение разделителя
console.log(text.split(/(,)/)); // ["apple", ",", "banana", ",", "cherry"]

```

```

// Практический пример: парсинг командной строки
function parseCommandLine(cmd) {
    // Разбор аргументов, учитывая кавычки
    const args = [];
    const pattern = /(?:[^\s"' ]+|"["]*"|'["']*')+\/g;
    let match;

    while ((match = pattern.exec(cmd)) !== null) {
        let arg = match[0];
        // Удаление кавычек
        if ((arg.startsWith('"') && arg.endsWith('"')) ||
            (arg.startsWith("'") && arg.endsWith("'"))) {
            arg = arg.slice(1, -1);
        }
        args.push(arg);
    }

    return args;
}

const command = 'git commit -m "initial commit" --author="John Doe"';
console.log('Парсинг команды:', parseCommandLine(command));

// Разделение с захватом групп
function parseKeyValue(text) {
    const pairs = [];
    const pattern = /(\w+)=(^[^;]+)/g;

    let match;
    while ((match = pattern.exec(text)) !== null) {
        pairs.push({ key: match[1], value: match[2] });
    }

    return pairs;
}

const config = 'host=localhost;port=8080;user=admin';
console.log('Конфигурация:', parseKeyValue(config));

```

4. Сравнение методов

javascript

// Сводная таблица сравнения методов

```
const comparisonTable = {
  'test()': {
    возвращает: 'boolean',
    сохраняетГруппы: false,
    работаетСФлагомG: 'да (меняет lastIndex)',
    пример: '/hello/.test("hello world")'
  },
  'exec()': {
    возвращает: 'массив с информацией или null',
    сохраняетГруппы: 'да',
    работаетСФлагомG: 'да (итерация)',
    пример: '/\\d+/.exec("123 abc")'
  },
  'match()': {
    возвращает: 'массив совпадений или null',
    сохраняетГруппы: 'без g: да, с g: нет',
    работаетСФлагомG: 'да',
    пример: '"123 abc".match(/\\d+/)'
  },
  'matchAll()': {
    возвращает: 'итератор совпадений',
    сохраняетГруппы: 'да',
    работаетСФлагомG: 'обязателен',
    пример: '[..."123 abc".matchAll(/\\d+/g)]'
  },
  'search()': {
    возвращает: 'индекс или -1',
    сохраняетГруппы: 'нет',
    работаетСФлагомG: 'да',
    пример: '"hello".search(/l/)'
  },
  'replace()': {
    возвращает: 'новая строка',
    сохраняетГруппы: 'да (в замене)',
    работаетСФлагомG: 'да',
    пример: '"hello".replace(/l/, "x")'
  },
  'replaceAll()': {
    возвращает: 'новая строка',
    сохраняетГруппы: 'да (в замене)',
    работаетСФлагомG: 'обязателен для RegExp',
    пример: '"hello".replaceAll("l", "x")'
  },
}
```

```

'split()': {
  возвращает: 'массив строк',
  сохраняетГруппы: 'да (если группа в шаблоне)',
  работаетСФлагомG: 'да',
  пример: '"a,b,c".split(/,/)'
}
};

console.log('Сравнение методов работы с регулярными выражениями:');
console.log('=' .repeat(80));

Object.entries(comparisonTable).forEach(([method, info]) => {
  console.log(`\n${method}:`);
  Object.entries(info).forEach(([key, value]) => {
    console.log(`  ${key}: ${value}`);
  });
});

```

5. Практические паттерны использования

Паттерн 1: Валидация и нормализация

```

javascript

class InputProcessor {
  // Кэш регулярных выражений для производительности
  static regexCache = new Map();

  static getRegex(pattern, flags = '') {
    const key = `${pattern}|${flags}`;
    if (!this.regexCache.has(key)) {
      this.regexCache.set(key, new RegExp(pattern, flags));
    }
    return this.regexCache.get(key);
  }

  // Валидация email
  static validateEmail(email) {
    const pattern = this.getRegex(
      '^[a-zA-Z0-9.!#$%&\'*+/?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\\.[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)*$'
    );
    return pattern.test(email.trim());
  }
}

```

```

}

// Нормализация телефона
static normalizePhone(phone) {
  // Удаляем все нецифровые символы кроме плюса в начале
  const digitsOnly = phone.replace(/[^\d+]/g, '');

  // Форматируем в международный формат
  if (digitsOnly.startsWith('8') && digitsOnly.length === 11) {
    return '+7' + digitsOnly.substring(1);
  }

  return digitsOnly.startsWith('+') ? digitsOnly : '+' + digitsOnly;
}

// Очистка HTML от скриптов
static sanitizeHTML(html) {
  return html
    .replace(/<script\b[^\<]*(?:(!</script><[^\<]*)*</script>/gi, '')
    .replace(/on\w+="[^\"]*" /gi, '')
    .replace(/javascript:/gi, '');
}

// Извлечение хэштегов и упоминаний
static extractSocialElements(text) {
  const hashtags = text.match(/#[\p{L}\p{N}_]+/gu) || [];
  const mentions = text.match(/@[\w.]+/g) || [];

  return {
    hashtags: hashtags.map(tag => tag.substring(1)),
    mentions: mentions.map(mention => mention.substring(1))
  };
}

// Парсинг query string
static parseQueryString(queryString) {
  const params = {};
  const pattern = /([^\&]+)=([^\&]*)/g;

  let match;
  while ((match = pattern.exec(queryString)) !== null) {
    const key = decodeURIComponent(match[1]);
    const value = decodeURIComponent(match[2]);
    params[key] = value;
  }
}

```



```

    }

    return params;
  }
}

// Тестирование
console.log('Валидация email:', InputProcessor.validateEmail('test@example.com'));
console.log('Нормализация телефона:', InputProcessor.normalizePhone('8 (999) 123-45-67'));

const socialText = 'Check out #JavaScript and @nodejs! #coding';
console.log('Социальные элементы:', InputProcessor.extractSocialElements(socialText));

const query = 'name=John%20Doe&age=30&city=New%20York';
console.log('Query string:', InputProcessor.parseQueryString(query));

```

Паттерн 2: Парсинг сложных форматов

```

javascript

class DataParser {
  // Парсинг многострочных логов
  static parseLogFile(logContent) {
    const logPattern = /^(?<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\ (?:<level>\w+) \[(?<module>[^\]]+\)\ (?:<message>.+)$/gm;
    const entries = [];

    for (const match of logContent.matchAll(logPattern)) {
      entries.push({
        timestamp: new Date(match.groups.timestamp),
        level: match.groups.level,
        module: match.groups.module,
        message: match.groups.message.trim(),
        raw: match[0]
      });
    }

    return entries;
  }

  // Парсинг конфигурационных файлов
  static parseConfig(configText) {
    const config = {};
    const linePattern = /^\\s*(?!\\s*#)(\\w+)\\s*=\\s*(.*)\\s*$/gm;

```

```

let match;
while ((match = linePattern.exec(configText)) !== null) {
    const key = match[1];
    let value = match[2];

    // Удаление кавычек
    if ((value.startsWith('\'') && value.endsWith('\'')) ||
        (value.startsWith('"') && value.endsWith('"'))) {
        value = value.slice(1, -1);
    }

    // Преобразование типов
    if (value === 'true') value = true;
    else if (value === 'false') value = false;
    else if (value === 'null') value = null;
    else if (/^\d+$/i.test(value)) value = parseInt(value, 10);
    else if (/^\d+\.\d+$/i.test(value)) value = parseFloat(value);

    config[key] = value;
}

return config;
}

// Парсинг SQL запросов (упрощенный)
static parseSQLQuery(sql) {
    const patterns = {
        select: /SELECT\s+(?<columns>.+?)\s+FROM\s+(?<table>\w+)/i,
        where: /WHERE\s+(?<condition>.+?)(?:\s+ORDER BY|\s+LIMIT|$)/i,
        orderBy: /ORDER BY\s+(?<order>.+?)(?:\s+LIMIT|$)/i,
        limit: /LIMIT\s+(?<limit>\d+)/i
    };

    const result = {};

    Object.entries(patterns).forEach(([type, pattern]) => {
        const match = sql.match(pattern);
        if (match && match.groups) {
            result[type] = match.groups;
        }
    });

    return result;
}

```

```

}

// Парсинг JSON-Like структур (упрощенный)
static parseJSONLike(text) {
  const result = {};
  const pairPattern = /(\w+)\s*:\s*(?:"([^"]*)"|'([^']*)'|(\w+)|(\d+(?:\.\d+)?))/g;

  let match;
  while ((match = pairPattern.exec(text)) !== null) {
    const key = match[1];
    const value = match[2] || match[3] || match[4] || match[5];

    // Преобразование типов
    if (value === 'true') result[key] = true;
    else if (value === 'false') result[key] = false;
    else if (value === 'null') result[key] = null;
    else if (/^\d+$/ .test(value)) result[key] = parseInt(value, 10);
    else if (/^\d+\.\d+$/ .test(value)) result[key] = parseFloat(value);
    else result[key] = value;
  }

  return result;
}

// Тестирование
const logContent = `
[2023-12-25 14:30:45] INFO [Auth] User logged in
[2023-12-25 14:31:10] ERROR [Database] Connection failed
# Это комментарий
[2023-12-25 14:32:00] WARN [System] High memory usage
`;

console.log('Парсинг логов:', DataParser.parseLogFile(logContent));

const configText = `
# Database configuration
host = localhost
port = 5432
user = admin
password = "secret123"
ssl = true
max_connections = 10
`;

```

```
console.log('Парсинг конфига:', DataParser.parseConfig(configText));
```

```
const sqlQuery = 'SELECT id, name FROM users WHERE active = 1 ORDER BY name LIMIT 10';
```

```
console.log('Парсинг SQL:', DataParser.parseSQLQuery(sqlQuery));
```

Паттерн 3: Текстовые преобразования

javascript

```
class TextTransformer {
  // Перевод в camelCase
  static toCamelCase(text) {
    return text
      .replace(/^[^a-zA-Z0-9]+(.) /g, (match, char) => char.toUpperCase())
      .replace(/^[A-Z]/, firstChar => firstChar.toLowerCase());
  }

  // Перевод в snake_case
  static toSnakeCase(text) {
    return text
      .replace(/([a-z])([A-Z])/g, '$1_$2')
      .replace(/^[^a-zA-Z0-9]+ /g, '_')
      .toLowerCase()
      .replace(/^-+|_+$/g, '');
  }

  // Перевод в kebab-case
  static toKebabCase(text) {
    return text
      .replace(/([a-z])([A-Z])/g, '$1-$2')
      .replace(/^[^a-zA-Z0-9]+ /g, '-')
      .toLowerCase()
      .replace(/^-+|_+$/g, '');
  }

  // Форматирование чисел
  static formatNumber(number, options = {}) {
    const {
      thousandsSeparator = ',',
      decimalSeparator = '.',
      decimalPlaces = 2
    } = options;

    const [integer, decimal] = number.toString().split('.');
```

```

    const formattedInteger = integer.replace(/B(?=(\d{3})+(?!\d))/g,
thousandsSeparator);

    if (decimal !== undefined && decimalPlaces > 0) {
        const formattedDecimal = decimal.substring(0, decimalPlaces).padEnd(decimalPlaces,
'0');
        return `${formattedInteger}${decimalSeparator}${formattedDecimal}`;
    }

    return formattedInteger;
}

// Склонение слов по числам
static pluralize(count, words) {
    const cases = [2, 0, 1, 1, 1, 2];
    const wordIndex = count % 100 > 4 && count % 100 < 20 ?
        2 : cases[Math.min(count % 10, 5)];

    return `${count} ${words[wordIndex]}`;
}

// Обрезка текста с учетом слов
static truncate(text, maxLength, suffix = '...') {
    if (text.length <= maxLength) return text;

    // Обрезаем до последнего целого слова
    const truncated = text.substring(0, maxLength - suffix.length);
    const lastSpace = truncated.lastIndexOf(' ');

    if (lastSpace > maxLength * 0.7) {
        return truncated.substring(0, lastSpace) + suffix;
    }

    return truncated + suffix;
}

// Подсветка синтаксиса (упрощенная)
static highlightSyntax(code, language = 'javascript') {
    const patterns = {
        javascript: {
            keywords: /\b(function|const|let|var|if|else|for|while|return|class|import|
export)\b/g,
            strings: /(["`])(?:\b(\\?))\2.*?\b1/g,
            comments: /(\/\//.*$|\/\/*[\s\S]*?\*\/)/gm,

```

```

        numbers: /\b\d+(?:\.\d+)?\b/g
    }
};

const langPatterns = patterns[language] || patterns.javascript;
let highlighted = code;

Object.entries(langPatterns).forEach(([type, pattern]) => {
    highlighted = highlighted.replace(pattern, match => {
        return `<span class="${type}">${match}</span>`;
    });
});

return highlighted;
}
}

// Тестирование
const testText = 'Hello World From JavaScript';
console.log('camelCase:', TextTransformer.toCamelCase(testText));
console.log('snake_case:', TextTransformer.toSnakeCase(testText));
console.log('kebab-case:', TextTransformer.toKebabCase(testText));

console.log('Форматирование числа:', TextTransformer.formatNumber(1234567.89));
console.log('Склонение:', TextTransformer.pluralize(5, ['яблоко', 'яблока', 'яблок']));

const longText = 'Это очень длинный текст, который нужно обрезать до разумной длины.';
console.log('Обрезка текста:', TextTransformer.truncate(longText, 30));

const jsCode = `
// Пример кода
function hello(name) {
    return "Hello, " + name + "!";
}

const result = hello("World");
console.log(result);
`;

console.log('Подсветка синтаксиса:', TextTransformer.highlightSyntax(jsCode));

```

6. Производительность и оптимизация

javascript

```
class RegexPerformance {
  // Кэширование регулярных выражений
  static regexCache = new Map();

  static getCached(pattern, flags = '') {
    const key = `${pattern}|${flags}`;
    if (!this.regexCache.has(key)) {
      this.regexCache.set(key, new RegExp(pattern, flags));
    }
    return this.regexCache.get(key);
  }

  // Бенчмарк методов

  static benchmark(text, pattern, iterations = 10000) {
    const regex = new RegExp(pattern);
    const results = {};

    // test()
    let start = performance.now();
    for (let i = 0; i < iterations; i++) {
      regex.test(text);
    }
    results.test = performance.now() - start;

    // exec()
    start = performance.now();
    for (let i = 0; i < iterations; i++) {
      regex.exec(text);
    }
    results.exec = performance.now() - start;

    // match()
    start = performance.now();
    for (let i = 0; i < iterations; i++) {
      text.match(regex);
    }
    results.match = performance.now() - start;

    // search()
    start = performance.now();
```

```

    for (let i = 0; i < iterations; i++) {
        text.search(regex);
    }
    results.search = performance.now() - start;

    return results;
}

// Оптимизация сложных выражений
static optimizePattern(originalPattern) {
    const optimizations = {
        // Избегание backtracking
        '(a+)+': 'a+',
        '(a*)*': 'a*',
        '(a|b)*': '[ab]*',

        // Использование атомарных групп (эмуляция)
        '(?>(...))': '(:...)(?!(?1))',

        // Предварительная проверка
        '.*abc': '(?=[^abc]*abc)',

        // Оптимизация границ слов
        '\\b\\w+\\b': '\\b[a-zA-Z0-9_]+\\b'
    };

    let optimized = originalPattern;
    Object.entries(optimizations).forEach(([bad, good]) => {
        optimized = optimized.replace(new RegExp(bad, 'g'), good);
    });

    return optimized;
}

// Проверка на catastrophic backtracking
static checkForBacktracking(pattern, testString) {
    const start = performance.now();

    try {
        new RegExp(pattern).test(testString);
    } catch (error) {
        return {
            safe: false,
            error: error.message,
        };
    }
}

```



```

        time: Infinity
    };
}

const time = performance.now() - start;

return {
    safe: time < 1000, // 1 секунда - разумный лимит
    time,
    warning: time > 100 ? 'Возможны проблемы с производительностью' : null
};
}

// Сравнение производительности разных подходов
static compareApproaches() {
    const testCases = [
        {
            name: 'Поиск email',
            text: 'Contact: test@example.com or admin@site.org',
            approaches: [
                { name: 'Простое', pattern: /^[^@]+@[^@]+\.[^@]+/ },
                { name: 'Специфичное', pattern: /\b[\w.%+-]+@[ \w.-]+\.[A-Za-z]{2,}\b/ },
                { name: 'Unicode', pattern: /\b[\p{L}\p{N}.\%+-]+@[ \p{L}\p{N}.\-]+\.[\p{L}]{2,}\b/u }
            ]
        },
        {
            name: 'Поиск чисел',
            text: 'Prices: $100, €200, ¥300, £400',
            approaches: [
                { name: 'Все цифры', pattern: /\d+/g },
                { name: 'С валютой', pattern: /[$\u20AC\u00A5\u00A3]?[s*]\d+/g },
                { name: 'Только с $', pattern: /\$ \d+/g }
            ]
        }
    ];

    const results = {};

    testCases.forEach(({ name, text, approaches }) => {
        results[name] = {};

        approaches.forEach(({ name: approachName, pattern }) => {
            const time = this.benchmark(text, pattern, 10000).test;

```

```

        results[name][approachName] = `${time.toFixed(2)}ms`;
    });
});

return results;
}
}

// Запуск бенчмарков
console.log('Бенчмарк производительности:');
console.log(RegexPerformance.benchmark('test@example.com', '^[^@]+@[^@]+\.[^@]+$'));

console.log('\nПроверка backtracking:');
console.log(RegexPerformance.checkForBacktracking('(a)+', 'aaaaaaaaaaaaaaaaaaaaaa!'));

console.log('\nСравнение подходов:');
console.log(RegexPerformance.compareApproaches());

```

7. Заключение

Методы работы с регулярными выражениями в JavaScript предоставляют мощный и гибкий инструментарий для обработки текста. Ключевые выводы:

1. Выбирайте правильный метод:

- ➔ Для проверки наличия → `test()`
- ➔ Для извлечения данных → `exec()` или `matchAll()`
- ➔ Для поиска позиции → `search()`
- ➔ Для замены → `replace()` или `replaceAll()`
- ➔ Для разделения → `split()`

2. Учитывайте флаги:

- ➔ Флаг `g` меняет поведение многих методов
- ➔ Флаг `u` необходим для работы с Unicode
- ➔ Флаги `d` и `v` добавляют новые возможности

3. Оптимизируйте производительность:

- ➔ Кэшируйте регулярные выражения
- ➔ Избегайте catastrophic backtracking
- ➔ Используйте атомарные группы и предварительные проверки

4. Используйте современные возможности:

- ➔ Именованные группы (ES2018)
- ➔ `matchAll()` (ES2020)
- ➔ `replaceAll()` (ES2021)

➡ Флаги `d` и `v` (ES2022+)

Регулярные выражения остаются одним из самых мощных инструментов для работы с текстом, и понимание всех доступных методов позволит вам использовать их максимально эффективно.

Часть III: Продвинутые концепции JavaScript

Глава 11: Асинхронное программирование

11.1. Модель событийного цикла в JavaScript

Введение в событийный цикл (Event Loop)

Событийный цикл — это фундаментальная концепция JavaScript, которая объясняет, как язык обрабатывает асинхронные операции при однопоточной модели выполнения. Понимание событийного цикла критически важно для написания эффективного и предсказуемого асинхронного кода.

Основная проблема: однопоточность

```
javascript

console.log("Демонстрация однопоточности:");

// Этот код блокирует главный поток
function blockingOperation() {
  console.log("Начало блокирующей операции");
  const start = Date.now();
  // Имитация долгой операции
  while (Date.now() - start < 3000) {
    // Просто ждем 3 секунды
  }
  console.log("Конец блокирующей операции");
}

console.log("До блокирующей операции");
blockingOperation(); // Блокирует выполнение на 3 секунды
console.log("После блокирующей операции"); // Выполнится только через 3 секунды

// Проблема: UI "замирает", пока выполняется блокирующая операция
```

1. Архитектурные компоненты

Стек вызовов (Call Stack)

```
javascript

console.log("Демонстрация стека вызовов:");

function first() {
  console.log("first: начало");
  second();
  console.log("first: конец");
}

function second() {
  console.log("second: начало");
  third();
  console.log("second: конец");
}

function third() {
  console.log("third: начало");
  console.log("third: выполнение");
  console.log("third: конец");
}

console.log("main: начало");
first();
console.log("main: конец");

// Визуализация стека вызовов:
// 1. main()          <- основание стека
// 2. main() -> first()
// 3. main() -> first() -> second()
// 4. main() -> first() -> second() -> third()
// 5. main() -> first() -> second()          (third завершен)
// 6. main() -> first()                      (second завершен)
// 7. main()                                (first завершен)
// 8.                                        (main завершен)
```

Куча (Heap)

```
javascript

// Куча - область памяти для динамического выделения объектов
```

```

class MemoryDemo {
  constructor() {
    this.data = new Array(1000).fill('data');
  }
}

console.log("Демонстрация работы кучи:");

// Объекты создаются в куче
const obj1 = { name: "Object 1", value: 100 };
const obj2 = { name: "Object 2", value: 200 };
const obj3 = { name: "Object 3", value: 300 };

// Массивы также в куче
const largeArray = new Array(1000000).fill(0);

// Функции-конструкторы
const demo1 = new MemoryDemo();
const demo2 = new MemoryDemo();

console.log("Объекты созданы в куче:");
console.log("obj1:", typeof obj1, "расположен в куче");
console.log("obj2:", typeof obj2, "расположен в куче");
console.log("largeArray:", largeArray.length, "элементов в куче");

// Сборка мусора (Garbage Collection)
console.log("\nДемонстрация сборки мусора:");
let temporaryObject = { temp: "data" };
console.log("Временный объект создан");

// После удаления ссылки объект может быть удален сборщиком мусора
temporaryObject = null;
console.log("Ссылка удалена - объект может быть собран");

```

Очередь задач (Callback Queue / Task Queue)

```

javascript

console.log("Демонстрация очереди задач:");

console.log("1. Синхронный код начался");

// setTimeout добавляет задачу в очередь
setTimeout(() => {
  console.log("3. Callback из setTimeout с задержкой 0");

```

```

}, 0);

setTimeout(() => {
  console.log("5. Callback из setTimeout с задержкой 100");
}, 100);

console.log("2. Синхронный код завершился");

// Микрозадачи выполняются ПЕРЕД задачами из очереди
Promise.resolve().then(() => {
  console.log("4. Микрозадача из Promise");
});

// Результат выполнения:
// 1. Синхронный код начался
// 2. Синхронный код завершился
// 4. Микрозадача из Promise
// 3. Callback из setTimeout с задержкой 0
// 5. Callback из setTimeout с задержкой 100

```

2. Микрозадачи (Microtasks) и макрозадачи (Macrotasks)

Различия и приоритеты

```

javascript

class TaskPriorityDemo {
  static demonstratePriorities() {
    console.log("=== Демонстрация приоритетов задач ===");

    // Макрозадачи (Macrotasks)
    setTimeout(() => console.log("Macrotask 1: setTimeout 0"), 0);
    setImmediate(() => console.log("Macrotask 2: setImmediate"));

    // Микрозадачи (Microtasks)
    Promise.resolve()
      .then(() => console.log("Microtask 1: Promise.then 1"))
      .then(() => console.log("Microtask 2: Promise.then 2"));

    // requestAnimationFrame (специальный тип задачи)
    requestAnimationFrame(() => {
      console.log("Animation Frame: requestAnimationFrame");
    });
  }
}

```

```

// MutationObserver (микрозадача)
const observer = new MutationObserver(() => {
  console.log("Microtask 3: MutationObserver");
});
observer.observe(document.body, { childList: true });

// Синхронный код
console.log("Sync: Синхронный код выполняется первым");

// process.nextTick (Node.js, выполняется перед микрозадачами)
if (typeof process !== 'undefined') {
  process.nextTick(() => {
    console.log("Next Tick: process.nextTick (самый высокий приоритет)");
  });
}

static demonstrateMicrotaskQueue() {
  console.log("\n=== Демонстрация очереди микрозадач ===");

  Promise.resolve()
    .then(() => {
      console.log("Microtask A1");
      return "результат A";
    })
    .then((result) => {
      console.log("Microtask A2 с результатом:", result);
      // Добавляем новую микрозадачу из микрозадачи
      Promise.resolve()
        .then(() => console.log("Microtask A3 (вложенная)"));
    });

  Promise.resolve()
    .then(() => console.log("Microtask B1"))
    .then(() => console.log("Microtask B2"))
    .then(() => console.log("Microtask B3"));

  console.log("Sync code");
}

// Запуск демонстраций
TaskPriorityDemo.demonstratePriorities();

```



```
TaskPriorityDemo.demonstrateMicrotaskQueue();
```

Практический пример: влияние на производительность

javascript

```
class PerformanceImpactDemo {
  static blockingVsNonBlocking() {
    console.log("=== Блокирующие vs Неблокирующие операции ===");

    // ПЛОХО: блокирующая операция
    function blockingOperation(iterations) {
      console.time(`Блокирующая (${iterations})`);
      let sum = 0;
      for (let i = 0; i < iterations; i++) {
        sum += Math.sqrt(i);
      }
      console.timeEnd(`Блокирующая (${iterations})`);
      return sum;
    }

    // ХОРОШО: разбиение на микрозадачи
    async function nonBlockingOperation(iterations, chunkSize = 1000) {
      console.time(`Неблокирующая (${iterations})`);
      let sum = 0;
      let processed = 0;

      function processChunk() {
        const end = Math.min(processed + chunkSize, iterations);
        for (let i = processed; i < end; i++) {
          sum += Math.sqrt(i);
        }
        processed = end;

        if (processed < iterations) {
          // Разрешаем событийному циклу обработать другие задачи
          return new Promise(resolve => {
            setTimeout(() => resolve(processChunk()), 0);
          });
        }
        return sum;
      }

      const result = await processChunk();
      console.timeEnd(`Неблокирующая (${iterations})`);
    }
  }
}
```

```

    return result;
}

// Тестирование
console.log("\nБлокирующая операция:");
blockingOperation(10000000);

console.log("\nНеблокирующая операция:");
nonBlockingOperation(10000000, 100000)
    .then(() => console.log("Неблокирующая операция завершена"));

console.log("UI остается отзывчивым во время неблокирующей операции");
}

static microtaskFlood() {
    console.log("\n=== Наводнение микрозадачами ===");

    // Создаем множество микрозадач
    function createMicrotasks(count) {
        console.time(`Создание ${count} микрозадач`);

        let completed = 0;
        const promises = [];

        for (let i = 0; i < count; i++) {
            promises.push(
                Promise.resolve().then(() => {
                    completed++;
                    if (completed === count) {
                        console.timeEnd(`Создание ${count} микрозадач`);
                    }
                })
            );
        }

        return Promise.all(promises);
    }

    // Проблема: микрозадачи выполняются до макрозадач
    setTimeout(() => {
        console.log("Эта макрозадача будет заблокирована микрозадачами");
    }, 0);

    createMicrotasks(100000)

```

```

        .then(() => console.log("Все микрозадачи выполнены"));
    }
}

// Запуск демонстрации производительности
PerformanceImpactDemo.blockingVsNonBlocking();
PerformanceImpactDemo.microtaskFlood();

```

3. Фазы событийного цикла (Node.js)

Подробное описание фаз

```

javascript

class EventLoopPhases {
    static async demonstratePhases() {
        console.log("=== Фазы событийного цикла Node.js ===");

        // 1. Фаза timers (setTimeout, setInterval)
        console.log("\n1. Фаза Timers:");
        setTimeout(() => {
            console.log("  Timer 1: setTimeout 0ms");
        }, 0);

        setTimeout(() => {
            console.log("  Timer 2: setTimeout 100ms");
        }, 100);

        // 2. Фаза pending callbacks (системные операции)
        console.log("2. Фаза Pending Callbacks:");
        // Системные колбэки (например, ошибки TCP)

        // 3. Фаза poll (ввод/вывод)
        console.log("3. Фаза Poll:");
        const fs = require('fs');
        fs.readFile(__filename, () => {
            console.log("  Poll: Файл прочитан");

            // setImmediate выполнится в следующей итерации
            setImmediate(() => {
                console.log("  setImmediate из колбэка чтения файла");
            });
        });
    }
}

```

```

// 4. Фаза check (setImmediate)
console.log("4. Фаза Check:");
setImmediate(() => {
  console.log("  Check: setImmediate 1");
});

setImmediate(() => {
  console.log("  Check: setImmediate 2");
});

// 5. Фаза close callbacks
console.log("5. Фаза Close Callbacks:");
const server = require('http').createServer();
server.on('close', () => {
  console.log("  Close: Сервер закрыт");
});
server.close();

// Микрозадачи выполняются между фазами
console.log("\nМикрозадачи (выполняются между фазами):");
Promise.resolve().then(() => {
  console.log("  Microtask 1");
}).then(() => {
  console.log("  Microtask 2");
});

// process.nextTick имеет наивысший приоритет
if (typeof process !== 'undefined') {
  process.nextTick(() => {
    console.log("  Next Tick 1");
    process.nextTick(() => {
      console.log("  Next Tick 2 (вложенный)");
    });
  });
}

console.log("\nСинхронный код выполнен, начинается обработка событийного цикла...");
}

static phaseOrderExample() {
  console.log("\n=== Порядок выполнения фаз ===");

  // Этот пример демонстрирует порядок выполнения

```

```

// Фаза 1: nextTick (выполняется перед любой фазой)
process.nextTick(() => console.log("phase: nextTick 1"));

// Фаза 2: Promise (микрозадачи)
Promise.resolve().then(() => console.log("phase: Promise 1"));

// Таймеры
setTimeout(() => {
  console.log("phase: setTimeout");

  process.nextTick(() => console.log("phase: nextTick внутри setTimeout"));
  Promise.resolve().then(() => console.log("phase: Promise внутри setTimeout"));
}, 0);

// setImmediate
setImmediate(() => {
  console.log("phase: setImmediate");

  process.nextTick(() => console.log("phase: nextTick внутри setImmediate"));
  Promise.resolve().then(() => console.log("phase: Promise внутри setImmediate"));
});

// Синхронный код
console.log("phase: синхронный код");

// Еще nextTick и Promise
process.nextTick(() => console.log("phase: nextTick 2"));
Promise.resolve().then(() => console.log("phase: Promise 2"));

// Порядок выполнения:
// 1. синхронный код
// 2. nextTick 1
// 3. nextTick 2
// 4. Promise 1
// 5. Promise 2
// 6. setTimeout или setImmediate (зависит от контекста)
// 7. микрозадачи из колбэков
}

// Запуск демонстрации фаз
EventLoopPhases.demonstratePhases().then(() => {
  EventLoopPhases.phaseOrderExample();
});

```

```
});
```

Практическое применение фаз

javascript

```
class PhaseApplications {
  static immediateVsTimeout() {
    console.log("=== setImmediate vs setTimeout(0) ===");

    // В разных контекстах порядок может меняться

    // В основном модуле
    setTimeout(() => console.log("main: setTimeout"), 0);
    setImmediate(() => console.log("main: setImmediate"));

    // Внутри I/O цикла (всегда setImmediate первым)
    const fs = require('fs');
    fs.readFile(__filename, () => {
      setTimeout(() => console.log("I/O: setTimeout"), 0);
      setImmediate(() => console.log("I/O: setImmediate"));
    });

    console.log("В основном модуле порядок может быть разным");
    console.log("В I/O колбэках setImmediate всегда раньше setTimeout");
  }

  static nextTickVsPromise() {
    console.log("\n=== process.nextTick vs Promise ===");

    // process.nextTick имеет приоритет над Promise

    Promise.resolve().then(() => console.log("Promise 1"));
    process.nextTick(() => console.log("nextTick 1"));

    Promise.resolve().then(() => console.log("Promise 2"));
    process.nextTick(() => console.log("nextTick 2"));

    console.log("Синхронный код");

    // Порядок: синхронный код → nextTick 1 → nextTick 2 → Promise 1 → Promise 2
  }

  static controllingExecutionOrder() {
    console.log("\n=== Управление порядком выполнения ===");
```

```

// Иногда нужно контролировать порядок выполнения

function executeInOrder() {
  console.log("\n1. Начало выполнения");

  // Синхронная операция
  const result = 42;

  // Асинхронная операция с nextTick
  process.nextTick(() => {
    console.log("3. Обработка результата:", result * 2);

    // Еще одна асинхронная операция
    setImmediate(() => {
      console.log("5. Финальная обработка");
    });

    // Promise после nextTick
    Promise.resolve().then(() => {
      console.log("4. Дополнительная обработка");
    });
  });

  console.log("2. Синхронный код завершен");
}

executeInOrder();
}

// Запуск примеров
PhaseApplications.immediateVsTimeout();
setTimeout(() => {
  PhaseApplications.nextTickVsPromise();
  PhaseApplications.controllingExecutionOrder();
}, 100);

```

4. Event Loop в браузере

Request Animation Frame

javascript

```

class BrowserEventLoop {
  static demonstrateRAF() {
    console.log("=== requestAnimationFrame в браузере ===");

    // requestAnimationFrame выполняется перед рендерингом

    let frameCount = 0;

    function animate() {
      frameCount++;
      console.log(`RAF Frame ${frameCount}`);

      if (frameCount < 3) {
        requestAnimationFrame(animate);
      }
    }

    // Микрозадачи выполняются до RAF
    Promise.resolve().then(() => {
      console.log("Promise до RAF");
    });

    // Макрозадачи выполняются после RAF
    setTimeout(() => {
      console.log("setTimeout после RAF");
    }, 0);

    requestAnimationFrame(animate);

    console.log("Синхронный код перед RAF");
  }

  static mutationObserverExample() {
    console.log("\n=== MutationObserver (микрозадача) ===");

    // MutationObserver использует микрозадачи

    const target = document.createElement('div');
    document.body.appendChild(target);

    const observer = new MutationObserver((mutations) => {
      console.log("MutationObserver сработал");
      console.log("Изменения:", mutations.length);
    });
  }
}

```



```

observer.observe(target, {
  childList: true,
  attributes: true
});

// Изменения будут обработаны как микрозадачи
target.setAttribute('data-test', 'value');
target.textContent = 'Новый текст';

console.log("Изменения DOM произведены");

// Promise выполнится в той же микрозадаче, что и MutationObserver
Promise.resolve().then(() => {
  console.log("Promise после изменения DOM");
});
}

static postMessageExample() {
  console.log("\n=== postMessage (макрозадача) ===");

  // postMessage добавляет макрозадачу

  const channel = new MessageChannel();

  channel.port1.onmessage = () => {
    console.log("postMessage получен");
  };

  Promise.resolve().then(() => {
    console.log("Promise перед postMessage");
  });

  setTimeout(() => {
    console.log("setTimeout перед postMessage");
  }, 0);

  channel.port2.postMessage('hello');

  console.log("postMessage отправлен");
}
}

// Для браузера

```

```

if (typeof document !== 'undefined') {
  BrowserEventLoop.demonstrateRAF();
  BrowserEventLoop.mutationObserverExample();
  BrowserEventLoop.postMessageExample();
}

```

5. Асинхронные итерации и Event Loop

javascript

```

class AsyncIteration {
  static async demonstrateAsyncIterators() {
    console.log("=== Асинхронные итераторы и Event Loop ===");

    // Создаем асинхронный итератор
    async function* asyncGenerator() {
      for (let i = 1; i <= 3; i++) {
        // Имитация асинхронной операции
        await new Promise(resolve => setTimeout(resolve, 100));
        yield i;
      }
    }

    console.log("Начало асинхронной итерации:");

    for await (const value of asyncGenerator()) {
      console.log(`Получено значение: ${value}`);

      // Показываем, что Event Loop продолжает работать
      Promise.resolve().then(() => {
        console.log(`Микрозадача для значения ${value}`);
      });
    }

    console.log("Асинхронная итерация завершена");
  }

  static async concurrentOperations() {
    console.log("\n=== Конкурентные операции ===");

    // Несколько асинхронных операций конкурентно

    const urls = ['ur11', 'ur12', 'ur13'];

```

```

const fetchPromises = urls.map(async (url, index) => {
  console.log(`Начало загрузки ${url}`);

  // Имитация HTTP запроса
  await new Promise(resolve =>
    setTimeout(resolve, Math.random() * 1000)
  );

  console.log(`Загрузка ${url} завершена`);
  return `Данные ${index + 1}`;
});

// Promise.all использует Event Loop для управления конкурентностью
const results = await Promise.all(fetchPromises);
console.log("Все загрузки завершены:", results);
}

static async backpressureControl() {
  console.log("\n=== Управление backpressure ===");

  // Контроль потока данных с помощью Event Loop

  async function* dataStream() {
    for (let i = 0; i < 10; i++) {
      // Генерация данных
      yield i;
      // Задержка для имитации источника данных
      await new Promise(resolve => setTimeout(resolve, 50));
    }
  }

  async function processWithBackpressure(stream, maxConcurrent = 2) {
    const processing = new Set();
    const results = [];

    for await (const data of stream) {
      console.log(`Получены данные: ${data}`);

      if (processing.size >= maxConcurrent) {
        console.log(` Достигнут лимит (${maxConcurrent}), ожидание...`);
        // Ждем, пока освободится место
        await Promise.race(processing);
      }
    }
  }
}

```

```

    // Создаем промис для обработки
    const processPromise = processData(data)
      .finally(() => processing.delete(processPromise));

    processing.add(processPromise);
    results.push(processPromise);
  }

  return Promise.all(results);
}

async function processData(data) {
  console.log(` Начало обработки ${data}`);
  await new Promise(resolve => setTimeout(resolve, 200));
  console.log(` Обработка ${data} завершена`);
  return `Обработано: ${data}`;
}

await processWithBackpressure(dataStream(), 3);
console.log("Все данные обработаны с контролем backpressure");
}
}

// Запуск асинхронных примеров
AsyncIteration.demonstrateAsyncIterators()
  .then(() => AsyncIteration.concurrentOperations())
  .then(() => AsyncIteration.backpressureControl());

```

6. Отладка и мониторинг Event Loop

javascript

```

class EventLoopDebugging {
  static monitorLag() {
    console.log("=== Мониторинг лагов Event Loop ===");

    let lastTime = Date.now();
    let maxLag = 0;
    let lagCount = 0;

    function checkLag() {
      const now = Date.now();
      const lag = now - lastTime - 1000; // Ожидаемый интервал 1000ms
    }
  }
}

```

```

    if (lag > 10) { // Порог лага
        maxLag = Math.max(maxLag, lag);
        lagCount++;
        console.log(`Обнаружен лаг: ${lag}ms (максимум: ${maxLag}ms)`);
    }

    lastTime = now;
    setTimeout(checkLag, 1000);
}

// Начинаем мониторинг
checkLag();

// Создаем искусственную нагрузку для демонстрации
setTimeout(() => {
    console.log("\nСоздаем нагрузку на Event Loop...");

    // Блокирующая операция
    const start = Date.now();
    while (Date.now() - start < 2000) {
        // Блокируем на 2 секунды
    }

    console.log("Нагрузка завершена");
}, 3000);

return {
    getStats: () => ({ maxLag, lagCount }),
    resetStats: () => { maxLag = 0; lagCount = 0; }
};
}

static profileEventLoop() {
    console.log("\n=== Профилирование Event Loop ===");

    const phases = {
        'timer': 0,
        'pending': 0,
        'poll': 0,
        'check': 0,
        'close': 0,
        'microtask': 0
    };
};

```

```

let currentPhase = '';
let phaseStart = Date.now();

// Мониторим смену фаз (в Node.js)
if (typeof process !== 'undefined') {
  const originalNextTick = process.nextTick;

  process.nextTick = function(callback, ...args) {
    phases.microtask += Date.now() - phaseStart;
    const result = originalNextTick.call(this, callback, ...args);
    phaseStart = Date.now();
    return result;
  };
}

// Функция для смены фазы
function switchPhase(phaseName) {
  if (currentPhase) {
    phases[currentPhase] += Date.now() - phaseStart;
  }
  currentPhase = phaseName;
  phaseStart = Date.now();
}

// Имитация работы Event Loop
async function simulateWork() {
  // Фаза таймеров
  switchPhase('timer');
  await new Promise(resolve => setTimeout(resolve, 100));

  // Фаза микрозадач
  switchPhase('microtask');
  await Promise.resolve();

  // Фаза poll
  switchPhase('poll');
  await new Promise(resolve => setImmediate(resolve));

  // Фаза check
  switchPhase('check');
  await new Promise(resolve => setImmediate(resolve));

  // Вывод результатов
  console.log("Распределение времени по фазам:");
}

```

```

    Object.entries(phases).forEach(([phase, time]) => {
        console.log(` ${phase}: ${time}ms`);
    });
}

simulateWork();
}

static detectBlockingCode() {
    console.log("\n=== Обнаружение блокирующего кода ===");

    const threshold = 50; // Порог в миллисекундах
    let lastCheck = Date.now();

    // Мониторим Event Loop
    function monitorLoop() {
        const now = Date.now();
        const elapsed = now - lastCheck;

        if (elapsed > threshold) {
            console.warn(`⚠ Обнаружен блокирующий код: ${elapsed}ms`);

            // Собираем стек вызовов (упрощенно)
            const stack = new Error().stack;
            console.log("Стек вызовов:", stack.split('\n').slice(2, 6).join('\n'));
        }

        lastCheck = now;
        setTimeout(monitorLoop, threshold);
    }

    monitorLoop();

    // Пример блокирующего кода
    setTimeout(() => {
        console.log("\nЗапускаем блокирующую операцию...");
        const start = Date.now();
        while (Date.now() - start < 200) {
            // Блокируем на 200ms
        }
        console.log("Блокирующая операция завершена");
    }, 1000);
}
}

```

```

// Запуск отладки
const monitor = EventLoopDebugging.monitorLag();
EventLoopDebugging.profileEventLoop();
EventLoopDebugging.detectBlockingCode();

// Статистика через 10 секунд
setTimeout(() => {
  console.log("\nИтоговая статистика мониторинга:");
  console.log(monitor.getStats());
  monitor.resetStats();
}, 10000);

```

7. Оптимизация работы с Event Loop

javascript

```

class EventLoopOptimization {
  static optimizeTaskScheduling() {
    console.log("=== Оптимизация планирования задач ===");

    // 1. Использование requestIdleCallback (браузер)
    if (typeof requestIdleCallback !== 'undefined') {
      function processIdleTask(deadline) {
        while (deadline.timeRemaining() > 0) {
          // Выполняем задачи, пока есть время
          console.log("Выполнение задачи в idle время");
        }

        if (/* есть еще задачи */) {
          requestIdleCallback(processIdleTask);
        }
      }

      requestIdleCallback(processIdleTask);
    }

    // 2. Разбиение длинных задач
    function processLargeArrayOptimized(array, chunkSize = 1000) {
      let index = 0;

      function processChunk() {
        const end = Math.min(index + chunkSize, array.length);

```



```

    for (let i = index; i < end; i++) {
        // Обработка элемента
        array[i] = array[i] * 2;
    }

    index = end;

    if (index < array.length) {
        // Планируем следующий чанк
        setTimeout(processChunk, 0);
    } else {
        console.log("Обработка массива завершена");
    }
}

processChunk();
}

// 3. Приоритизация задач
class TaskScheduler {
    constructor() {
        this.highPriorityQueue = [];
        this.normalPriorityQueue = [];
        this.lowPriorityQueue = [];
        this.isProcessing = false;
    }

    addTask(task, priority = 'normal') {
        const queue = this[`${priority}PriorityQueue`];
        queue.push(task);

        if (!this.isProcessing) {
            this.processTasks();
        }
    }

    async processTasks() {
        this.isProcessing = true;

        while (this.highPriorityQueue.length > 0 ||
            this.normalPriorityQueue.length > 0 ||
            this.lowPriorityQueue.length > 0) {

            // Обрабатываем высокоприоритетные задачи

```

```

    if (this.highPriorityQueue.length > 0) {
        const task = this.highPriorityQueue.shift();
        await task();
        continue;
    }

    // Обрабатываем обычные задачи
    if (this.normalPriorityQueue.length > 0) {
        const task = this.normalPriorityQueue.shift();
        await task();

        // Даем шанс высокоприоритетным задачам
        if (this.highPriorityQueue.length > 0) {
            continue;
        }
    }

    // Обрабатываем низкоприоритетные задачи
    if (this.lowPriorityQueue.length > 0) {
        const task = this.lowPriorityQueue.shift();
        await task();
    }
}

this.isProcessing = false;
}
}

// Пример использования планировщика
const scheduler = new TaskScheduler();

scheduler.addTask(() => {
    console.log("Высокоприоритетная задача 1");
}, 'high');

scheduler.addTask(() => {
    console.log("Обычная задача 1");
}, 'normal');

scheduler.addTask(() => {
    console.log("Низкоприоритетная задача 1");
}, 'low');

scheduler.addTask(() => {

```

```

        console.log("Высокоприоритетная задача 2");
    }, 'high');
}

```

```

static memoryManagement() {
    console.log("\n=== Управление памятью и Event Loop ===");

```

// 1. Избегаем утечек памяти

```

class EventManager {
    constructor() {
        this.listeners = new Map();
        this.weakListeners = new WeakMap();
    }

    // НЕПРАВИЛЬНО: сильные ссылки
    addListenerBad(event, callback) {
        if (!this.listeners.has(event)) {
            this.listeners.set(event, new Set());
        }
        this.listeners.get(event).add(callback);
    }

```

// ПРАВИЛЬНО: WeakRef для избежания утечек

```

    addListenerGood(event, callback) {
        const weakRef = new WeakRef(callback);
        if (!this.weakListeners.has(event)) {
            this.weakListeners.set(event, new Set());
        }
        this.weakListeners.get(event).add(weakRef);
    }

```

```

    cleanup() {
        // Очистка устаревших ссылок
        for (const [event, weakRefs] of this.weakListeners) {
            for (const weakRef of weakRefs) {
                if (!weakRef.deref()) {
                    weakRefs.delete(weakRef);
                }
            }
        }
    }
}

```

// 2. Оптимизация работы с DOM

```

function optimizeDOMOperations() {
  // ПЛОХО: множественные изменения DOM
  const container = document.createElement('div');

  // ХОРОШО: DocumentFragment для батчинга
  const fragment = document.createDocumentFragment();

  for (let i = 0; i < 1000; i++) {
    const element = document.createElement('div');
    element.textContent = `Элемент ${i}`;
    fragment.appendChild(element);
  }

  container.appendChild(fragment);

  // ХОРОШО: requestAnimationFrame для анимаций
  function animateSmoothly(element) {
    let position = 0;

    function step(timestamp) {
      position += 1;
      element.style.transform = `translateX(${position}px)`;

      if (position < 100) {
        requestAnimationFrame(step);
      }
    }

    requestAnimationFrame(step);
  }
}

// 3. Управление временем жизни объектов
class CacheWithTTL {
  constructor(ttl = 60000) { // 60 секунд по умолчанию
    this.cache = new Map();
    this.ttl = ttl;
    this.cleanupInterval = setInterval(() => this.cleanup(), ttl);
  }

  set(key, value) {
    this.cache.set(key, {
      value,
      expires: Date.now() + this.ttl
    });
  }
}

```

```

    });
}

get(key) {
    const item = this.cache.get(key);

    if (!item) return null;

    if (item.expires < Date.now()) {
        this.cache.delete(key);
        return null;
    }

    return item.value;
}

cleanup() {
    const now = Date.now();
    for (const [key, item] of this.cache) {
        if (item.expires < now) {
            this.cache.delete(key);
        }
    }
}

destroy() {
    clearInterval(this.cleanupInterval);
    this.cache.clear();
}
}
}

// Запуск оптимизаций
EventLoopOptimization.optimizeTaskScheduling();
EventLoopOptimization.memoryManagement();

```

8. Практические паттерны и антипаттерны

```

javascript

class EventLoopPatterns {
    static demonstratePatterns() {
        console.log("=== Паттерны и антипаттерны работы с Event Loop ===");
    }
}

```

```
// АНТИПАТТЕРН 1: Блокировка Event Loop
console.log("\n1. АНТИПАТТЕРН: Блокировка Event Loop");
function blockingFibonacci(n) {
  if (n <= 1) return n;
  return blockingFibonacci(n - 1) + blockingFibonacci(n - 2);
}

// ПАТТЕРН 1: Разбиение на чанки
console.log("\n ПАТТЕРН: Разбиение на чанки");
async function nonBlockingFibonacci(n) {
  if (n <= 1) return n;

  // Используем мемоизацию и разбиение
  const memo = new Map();

  function fib(x) {
    if (x <= 1) return x;
    if (memo.has(x)) return memo.get(x);

    const result = fib(x - 1) + fib(x - 2);
    memo.set(x, result);
    return result;
  }

  // Для очень больших n можно разбить на чанки
  if (n > 1000) {
    return new Promise(resolve => {
      setTimeout(() => resolve(fib(n)), 0);
    });
  }

  return fib(n);
}

// АНТИПАТТЕРН 2: Рекурсия без базового случая
console.log("\n2. АНТИПАТТЕРН: Бесконечная рекурсия");
function infiniteRecursion() {
  // Этот код вызовет переполнение стека
  // infiniteRecursion(); // ЗАКОММЕНТИРОВАНО ДЛЯ БЕЗОПАСНОСТИ
}

// ПАТТЕРН 2: Трамплинг для рекурсии
console.log("\n ПАТТЕРН: Трамплинг (trampolining)");
```

```

function trampoline(fn) {
  return function(...args) {
    let result = fn(...args);

    while (typeof result === 'function') {
      result = result();
    }

    return result;
  };
}

const safeRecursion = trampoline(function sum(n, acc = 0) {
  if (n <= 0) return acc;
  return () => sum(n - 1, acc + n);
});

console.log("    Безопасная рекурсия (1-1000):", safeRecursion(1000));

// АНТИПАТТЕРН 3: Микрозадачное наводнение
console.log("\n3. АНТИПАТТЕРН: Наводнение микрозадачами");
function microtaskFlood(count) {
  for (let i = 0; i < count; i++) {
    Promise.resolve().then(() => {
      // Множество микрозадач
    });
  }
}

// ПАТТЕРН 3: Балансировка микрозадач
console.log("\n    ПАТТЕРН: Балансировка задач");
class BalancedExecutor {
  constructor(maxMicrotasks = 100) {
    this.queue = [];
    this.isProcessing = false;
    this.maxMicrotasks = maxMicrotasks;
  }

  addTask(task) {
    this.queue.push(task);
    this.processQueue();
  }

  async processQueue() {

```

```

    if (this.isProcessing) return;

    this.isProcessing = true;
    let processed = 0;

    while (this.queue.length > 0 && processed < this.maxMicrotasks) {
        const task = this.queue.shift();
        await task();
        processed++;

        // Даем шанс другим задачам
        if (processed % 10 === 0) {
            await new Promise(resolve => setTimeout(resolve, 0));
        }
    }

    this.isProcessing = false;

    if (this.queue.length > 0) {
        setTimeout(() => this.processQueue(), 0);
    }
}

// АНТИПАТТЕРН 4: Неуправляемые таймеры
console.log("\n4. АНТИПАТТЕРН: Утечка таймеров");
function leakyTimers() {
    setInterval(() => {
        // Таймер никогда не очищается
    }, 1000);
}

// ПАТТЕРН 4: Управление таймерами
console.log("\n ПАТТЕРН: Контролируемые интервалы");
class ManagedInterval {
    constructor(callback, interval) {
        this.callback = callback;
        this.interval = interval;
        this.timerId = null;
        this.isRunning = false;
    }

    start() {
        if (this.isRunning) return;

```



```

    this.isRunning = true;
    const execute = () => {
        if (!this.isRunning) return;

        this.callback();
        this.timerId = setTimeout(execute, this.interval);
    };

    this.timerId = setTimeout(execute, this.interval);
}

stop() {
    this.isRunning = false;
    if (this.timerId) {
        clearTimeout(this.timerId);
        this.timerId = null;
    }
}

restart(newInterval) {
    this.stop();
    if (newInterval) this.interval = newInterval;
    this.start();
}
}

// Пример использования
const interval = new ManagedInterval(() => {
    console.log("    Управляемый интервал выполнен");
}, 1000);

interval.start();

// Остановим через 3 секунды
setTimeout(() => {
    interval.stop();
    console.log("    Интервал остановлен");
}, 3000);
}

static realWorldExamples() {
    console.log("\n\n=== Реальные примеры использования ===");
}

```

// 1. Обработка пользовательского ввода

```
console.log("\n1. Обработка пользовательского ввода с debounce");
```

```
class InputProcessor {
  constructor() {
    this.timeoutId = null;
    this.lastProcessTime = 0;
  }

  processInputDebounced(input, callback, delay = 300) {
    clearTimeout(this.timeoutId);

    this.timeoutId = setTimeout(() => {
      callback(input);
      this.lastProcessTime = Date.now();
    }, delay);
  }

  processInputThrottled(input, callback, limit = 1000) {
    const now = Date.now();

    if (now - this.lastProcessTime >= limit) {
      callback(input);
      this.lastProcessTime = now;
    }
  }
}
```

// 2. Пагинация с виртуальным скроллингом

```
console.log("\n2. Виртуальный скроллинг с Event Loop");
```

```
class VirtualScroller {
  constructor(container, itemHeight = 50) {
    this.container = container;
    this.itemHeight = itemHeight;
    this.visibleItems = 20;
    this.data = [];
    this.renderedRange = { start: 0, end: 0 };
    this.rafId = null;

    this.setup();
  }

  setup() {
```

```

    this.container.addEventListener('scroll', () => {
      this.handleScroll();
    });

    // Используем requestAnimationFrame для плавности
    this.handleScroll = () => {
      if (this.rafId) {
        cancelAnimationFrame(this.rafId);
      }

      this.rafId = requestAnimationFrame(() => {
        this.updateViewport();
      });
    };
  }

  updateViewport() {
    const scrollTop = this.container.scrollTop;
    const startIndex = Math.floor(scrollTop / this.itemHeight);
    const endIndex = startIndex + this.visibleItems;

    if (startIndex !== this.renderedRange.start ||
        endIndex !== this.renderedRange.end) {
      this.renderItems(startIndex, endIndex);
      this.renderedRange = { start: startIndex, end: endIndex };
    }
  }

  renderItems(start, end) {
    // Реализация рендеринга
    console.log(`Рендеринг элементов ${start}-${end}`);
  }
}

// 3. Web Worker для тяжелых вычислений
console.log("\n3. Вынос тяжелых вычислений в Web Worker");

class ComputationManager {
  constructor() {
    this.worker = null;
    this.callbacks = new Map();
    this.nextId = 1;

    if (typeof Worker !== 'undefined') {

```

```

        this.setupWorker();
    }
}

setupWorker() {
    const workerCode = `
        self.onmessage = function(e) {
            const { id, type, data } = e.data;

            if (type === 'compute') {
                // Тяжелые вычисления
                const result = heavyComputation(data);
                self.postMessage({ id, result });
            }
        };

        function heavyComputation(data) {
            // Имитация тяжелых вычислений
            let result = 0;
            for (let i = 0; i < data.iterations; i++) {
                result += Math.sqrt(i);
            }
            return result;
        }
    `;

    const blob = new Blob([workerCode], { type: 'application/javascript' });
    this.worker = new Worker(URL.createObjectURL(blob));

    this.worker.onmessage = (e) => {
        const { id, result } = e.data;
        const callback = this.callbacks.get(id);

        if (callback) {
            callback(result);
            this.callbacks.delete(id);
        }
    };
}

compute(data, callback) {
    if (!this.worker) {
        // Fallback: выполняем в основном потоке с чанкингом
        this.computeInMainThread(data, callback);
    }
}

```

```

    return;
}

const id = this.nextId++;
this.callbacks.set(id, callback);

this.worker.postMessage({
  id,
  type: 'compute',
  data
});
}

computeInMainThread(data, callback) {
  // Разбиваем на чанки, чтобы не блокировать Event Loop
  const chunkSize = 100000;
  let result = 0;
  let processed = 0;

  const processChunk = () => {
    const end = Math.min(processed + chunkSize, data.iterations);

    for (let i = processed; i < end; i++) {
      result += Math.sqrt(i);
    }

    processed = end;

    if (processed < data.iterations) {
      setTimeout(processChunk, 0);
    } else {
      callback(result);
    }
  };

  processChunk();
}

// Демонстрация
const manager = new ComputationManager();
manager.compute({ iterations: 1000000 }, (result) => {
  console.log(`Результат вычислений: ${result}`);
});

```

```

    }
  }

  // Запуск паттернов
  EventLoopPatterns.demonstratePatterns();
  setTimeout(() => {
    EventLoopPatterns.realWorldExamples();
  }, 4000);

```

9. Заключение и ключевые выводы

```

javascript

class EventLoopSummary {
  static keyTakeaways() {
    console.log("=== КЛЮЧЕВЫЕ ВЫВОДЫ: МОДЕЛЬ СОБЫТИЙНОГО ЦИКЛА ===");

    const takeaways = [
      {
        title: "1. Однопоточность с асинхронностью",
        points: [
          "JavaScript выполняется в одном потоке",
          "Асинхронные операции не блокируют основной поток",
          "Event Loop управляет выполнением асинхронных задач"
        ]
      },
      {
        title: "2. Компоненты Event Loop",
        points: [
          "Call Stack: синхронные вызовы функций",
          "Heap: динамическая память для объектов",
          "Callback Queue: очередь асинхронных задач",
          "Microtask Queue: очередь с высшим приоритетом"
        ]
      },
      {
        title: "3. Приоритеты выполнения",
        points: [
          "Синхронный код → Микрозадачи → Макрозадачи",
          "Микрозадачи: Promise, MutationObserver, process.nextTick",
          "Макрозадачи: setTimeout, setInterval, I/O, UI рендеринг"
        ]
      }
    ],
    {

```

```

    title: "4. Фазы Event Loop (Node.js)",
    points: [
      "Timers: setTimeout, setInterval",
      "Pending callbacks: системные операции",
      "Poll: ввод/вывод",
      "Check: setImmediate",
      "Close callbacks: закрытие ресурсов"
    ]
  },
  {
    title: "5. Оптимизация работы с Event Loop",
    points: [
      "Разбивайте длинные задачи на чанки",
      "Используйте Web Worker для тяжелых вычислений",
      "Избегайте блокирующих операций",
      "Контролируйте количество микрозадач"
    ]
  },
  {
    title: "6. Распространенные ошибки",
    points: [
      "Блокировка Event Loop длительными операциями",
      "Микрозадачное наводнение",
      "Утечки памяти из-за незавершенных таймеров",
      "Рекурсия без базового случая"
    ]
  },
  {
    title: "7. Мониторинг и отладка",
    points: [
      "Измеряйте лаги Event Loop",
      "Используйте профилировщики производительности",
      "Мониторьте использование памяти",
      "Отслеживайте стек вызовов при блокировках"
    ]
  }
];

takeaways.forEach(({ title, points }) => {
  console.log(`\n${title}:`);
  points.forEach(point => {
    console.log(`  • ${point}`);
  });
});

```

```
console.log("\n=== ЛУЧШИЕ ПРАКТИКИ ===");
console.log(`
```

1. Всегда разбивайте тяжелые вычисления на чанки
2. Используйте асинхронные API вместо синхронных
3. Очищайте таймеры и обработчики событий
4. Контролируйте глубину рекурсии
5. Используйте Web Worker для CPU-интенсивных задач
6. Мониторьте производительность в production
7. Тестируйте на реалистичных нагрузках

```
`);
```

```
}
```

```
static finalExample() {
```

```
  console.log("\n=== ИТОВОЫЙ ПРИМЕР: ОПТИМИЗИРОВАННЫЙ EVENT LOOP ===");
```

```
  class OptimizedApp {
```

```
    constructor() {
```

```
      this.taskQueue = [];
```

```
      this.isProcessing = false;
```

```
      this.stats = {
```

```
        tasksProcessed: 0,
```

```
        maxQueueLength: 0,
```

```
        averageTaskTime: 0
```

```
      };
```

```
    }
```

```
    // Добавление задачи с приоритетом
```

```
    addTask(task, priority = 'normal') {
```

```
      const taskWithMeta = {
```

```
        task,
```

```
        priority,
```

```
        addedAt: Date.now()
```

```
      };
```

```
      if (priority === 'high') {
```

```
        this.taskQueue.unshift(taskWithMeta);
```

```
      } else {
```

```
        this.taskQueue.push(taskWithMeta);
```

```
      }
```

```
      this.stats.maxQueueLength = Math.max(  
        this.stats.maxQueueLength,
```

```
        this.taskQueue.length
```



```

    );

    this.processQueue();
}

// Обработка очереди с учетом Event Loop
async processQueue() {
    if (this.isProcessing) return;

    this.isProcessing = true;

    while (this.taskQueue.length > 0) {
        const startTime = Date.now();
        const taskItem = this.taskQueue.shift();

        try {
            await taskItem.task();
        } catch (error) {
            console.error('Ошибка в задаче:', error);
        }

        // Обновляем статистику
        const taskTime = Date.now() - startTime;
        this.stats.tasksProcessed++;
        this.stats.averageTaskTime =
            (this.stats.averageTaskTime * (this.stats.tasksProcessed - 1) + taskTime) /
            this.stats.tasksProcessed;

        // Даем шанс другим задачам в Event Loop
        if (taskTime > 16) { // Больше чем кадр (60fps)
            await new Promise(resolve => setTimeout(resolve, 0));
        }
    }

    this.isProcessing = false;
}

getStats() {
    return {
        ...this.stats,
        currentQueueLength: this.taskQueue.length,
        isProcessing: this.isProcessing
    };
}

```

```

}

// Демонстрация работы
const app = new OptimizedApp();

// Добавляем задачи разного типа
for (let i = 0; i < 10; i++) {
  app.addTask(async () => {
    console.log(`Выполнение задачи ${i + 1}`);
    await new Promise(resolve =>
      setTimeout(resolve, Math.random() * 100)
    );
  }, i % 3 === 0 ? 'high' : 'normal');
}

// Показываем статистику
setTimeout(() => {
  console.log("\nСтатистика выполнения:");
  console.log(app.getStats());
}, 1000);
}
}

// Финальный вывод
EventLoopSummary.keyTakeaways();
EventLoopSummary.finalExample();

console.log("\n" + "=".repeat(80));
console.log("□ ПОМНИТЕ: Понимание Event Loop - ключ к написанию");
console.log("      эффективного и отзывчивого JavaScript-кода!");
console.log("=".repeat(80));

```

10. Дополнительные ресурсы и инструменты

```

javascript

class EventLoopTools {
  static recommendedTools() {
    console.log("=== ИНСТРУМЕНТЫ ДЛЯ РАБОТЫ С EVENT LOOP ===");

    const tools = {
      "Мониторинг производительности": [
        "Chrome DevTools Performance Tab",
        "Node.js --inspect flag",

```

```

    "clinic.js (профилировщик для Node.js)",
    "0x (flamegraph для Node.js)"
  ],

  "Библиотеки для управления задачами": [
    "async.js (управление асинхронностью)",
    "p-queue (очередь промисов с приоритетами)",
    "fastq (простая очередь задач)",
    "bottleneck (ограничение скорости выполнения)"
  ],

  "Утилиты для тестирования": [
    "artillery (нагрузочное тестирование)",
    "autocannon (HTTP бенчмарк для Node.js)",
    "k6 (современное нагрузочное тестирование)",
    "loadtest (простое тестирование нагрузки)"
  ],

  "Инструменты отладки": [
    "node --trace-event-categories v8,node (трейсинг)",
    "wtfnode (поиск утечек в Node.js)",
    "heapdump (снимки памяти)",
    "ndb (отладчик для Node.js)"
  ]
};

Object.entries(tools).forEach(([category, items]) => {
  console.log(`\n${category}:`);
  items.forEach(item => {
    console.log(`  • ${item}`);
  });
});

console.log("\n=== ПОЛЕЗНЫЕ КОМАНДЫ NODE.JS ===");
console.log(`
# Мониторинг Event Loop
node --trace-event-categories v8,node app.js

# Профилирование CPU
node --prof app.js
node --prof-process isolate-0x*.log

# Отладка утечек памяти
node --inspect --trace-gc app.js

```

```
# Нагрузочное тестирование
```

```
autocannon -c 100 -d 30 http://localhost:3000
```

```
`);
```

```
}
```

```
static learningResources() {
```

```
    console.log("\n=== РЕКОМЕНДУЕМЫЕ РЕСУРСЫ ДЛЯ ИЗУЧЕНИЯ ===");
```

```
    const resources = [
```

```
        {
```

```
            type: "📖 Книги",
```

```
            items: [
```

```
                "Выразительный JavaScript (Marijn Haverbeke)",
```

```
                "Node.js Design Patterns (Mario Casciaro)",
```

```
                "Async JavaScript (Trevor Burnham)"
```

```
            ]
```

```
        },
```

```
        {
```

```
            type: "📺 Видео",
```

```
            items: [
```

```
                "What the heck is the event loop anyway? | Philip Roberts",
```

```
                "In The Loop - Jake Archibald (JSConf.Asia)",
```

```
                "Node.js Event Loop - Bert Belder"
```

```
            ]
```

```
        },
```

```
        {
```

```
            type: "📄 Статьи",
```

```
            items: [
```

```
                "JavaScript Visualized: Event Loop (Lydia Hallie)",
```

```
                "The Node.js Event Loop (Node.js документация)",
```

```
                "Tasks, microtasks, queues and schedules (Jake Archibald)"
```

```
            ]
```

```
        },
```

```
        {
```

```
            type: "🛠 Интерактивные инструменты",
```

```
            items: [
```

```
                "Loupe (визуализатор Event Loop)",
```

```
                "JavaScript Event Loop Visualizer",
```

```
                "Node.js Event Loop Visualizer"
```

```
            ]
```

```
        }
```

```
    ];
```

```

resources.forEach(({ type, items }) => {
  console.log(`\n${type}:`);
  items.forEach(item => {
    console.log(`  • ${item}`);
  });
});
}
}

```

// Показываем инструменты и ресурсы

```
EventLoopTools.recommendedTools();
```

```
EventLoopTools.learningResources();
```

```
console.log("\n" + "=".repeat(80));
```

```
console.log("📌 СОВЕТ: Регулярно практикуйтесь и экспериментируйте с");
```

```
console.log("      Event Loop, чтобы глубже понять его работу!");
```

```
console.log("=".repeat(80));
```

Это подробное руководство охватывает все аспекты модели событийного цикла в JavaScript, от базовых концепций до продвинутых техник оптимизации и отладки. Понимание этих принципов критически важно для написания эффективного, отзывчивого и масштабируемого JavaScript-кода.

11.2. Callback-функции и проблемы "callback hell"

Введение: асинхронность и функции обратного вызова

JavaScript изначально проектировался как однопоточный язык для выполнения в браузере, где блокировка основного потока могла бы привести к "зависанию" интерфейса. Чтобы обеспечить отзывчивость, в JavaScript была реализована асинхронная модель выполнения, основанная на **callback-функциях** (функциях обратного вызова).

Callback-функция — это функция, которая передается в качестве аргумента другой функции и вызывается после завершения определенной операции (обычно асинхронной).

11.2.1. Что такое callback-функции: базовые концепции

Фундаментальный принцип:

```
javascript

// Синхронная функция - выполняется сразу
function syncFunction() {
    return "Результат";
}

// Асинхронная операция с callback
function asyncFunction(callback) {
    setTimeout(() => {
        const result = "Асинхронный результат";
        callback(result); // Вызываем callback после завершения
    }, 1000);
}

// Использование
console.log(syncFunction()); // "Результат" - сразу

asyncFunction((result) => {
    console.log(result); // "Асинхронный результат" - через 1 секунду
});
```

```
console.log("Этот код выполняется сразу, не дожидаясь asyncFunction");
```

Callback в действии: реальные примеры

Пример 1: Чтение файла в Node.js

```
javascript

const fs = require('fs');

// Асинхронное чтение файла с callback
fs.readFile('data.txt', 'utf8', (error, data) => {
  if (error) {
    console.error('Ошибка чтения файла:', error);
    return;
  }
  console.log('Содержимое файла:', data);
});

console.log('Эта строка выводится ПЕРЕД чтением файла');
```

Пример 2: AJAX-запрос в браузере (устаревший XMLHttpRequest)

```
javascript

function fetchData(url, callback) {
  const xhr = new XMLHttpRequest();

  xhr.open('GET', url, true);

  xhr.onload = function() {
    if (xhr.status === 200) {
      callback(null, JSON.parse(xhr.responseText));
    } else {
      callback(new Error('Ошибка запроса'), null);
    }
  };

  xhr.onerror = function() {
    callback(new Error('Сетевая ошибка'), null);
  };

  xhr.send();
}

// Использование
```

```
fetchData('https://api.example.com/data', (error, data) => {
  if (error) {
    console.error('Ошибка:', error);
    return;
  }
  console.log('Полученные данные:', data);
});
```

Пример 3: События DOM

```
javascript

// Самый распространенный callback в веб-разработке
document.getElementById('myButton').addEventListener('click', function(event) {
  // Этот callback выполнится при клике на кнопку
  console.log('Кнопка нажата!', event.target);
});
```

Паттерн "Error-First Callback"

В Node.js и многих библиотеках принята конвенция, где первый параметр callback'a — это ошибка:

```
javascript

function asyncOperation(arg1, arg2, callback) {
  // Некоторый асинхронный код...

  // Если произошла ошибка
  if (errorOccurred) {
    callback(new Error('Что-то пошло не так'), null);
    return;
  }

  // Если все успешно
  callback(null, result);
}

// Правильное использование
asyncOperation('value1', 'value2', (err, result) => {
  if (err) {
    // Обработка ошибки
    console.error('Ошибка:', err.message);
    return;
  }
});
```



```
// Работа с результатом
console.log('Результат:', result);
});
```

11.2.2. Преимущества callback-функций

1. **Неблокирующий код:** Основной поток продолжает работать
2. **Простота понимания:** Прямолинейный подход "сделай это, потом то"
3. **Универсальность:** Работает во всех версиях JavaScript
4. **Совместимость:** Поддерживается всеми браузерами и Node.js

11.2.3. Проблемы callback-подхода

Проблема 1: Callback Hell (Ад обратных вызовов)

Самая известная проблема — вложенность callback'ов, создающая "пирамиду смерти":

```
javascript

// Классический пример callback hell
getUserData(userId, (userError, userData) => {
  if (userError) {
    console.error('Ошибка получения пользователя:', userError);
    return;
  }

  getUserPermissions(userData.id, (permError, permissions) => {
    if (permError) {
      console.error('Ошибка получения прав:', permError);
      return;
    }

    checkResourceAccess(permissions, resourceId, (accessError, hasAccess) => {
      if (accessError) {
        console.error('Ошибка проверки доступа:', accessError);
        return;
      }

      if (hasAccess) {
        loadResourceData(resourceId, (dataError, resourceData) => {
          if (dataError) {
            console.error('Ошибка загрузки ресурса:', dataError);
          }
        });
      }
    });
  });
});
```

```

        return;
    }

    processResourceData(resourceData, (processError, result) => {
        if (processError) {
            console.error('Ошибка обработки:', processError);
            return;
        }

        displayResult(result, (displayError) => {
            if (displayError) {
                console.error('Ошибка отображения:', displayError);
                return;
            }

            console.log('Готово!');
        });
    });
});
} else {
    console.log('Доступ запрещен');
}
});
});
});

```

Проблемы этой структуры:

- Горизонтальное смещение кода (стрелка вправо)
- Сложность чтения и понимания
- Трудности в отладке
- Сложность обработки ошибок на каждом уровне

Проблема 2: Потеря контекста (this)

```

javascript

const user = {
    name: 'Alice',
    tasks: ['task1', 'task2', 'task3'],

    showTasks: function() {
        console.log(this.name + ' имеет задачи:');
    }
};

// Проблема: потеря контекста this

```

```

    this.tasks.forEach(function(task) {
        console.log('- ' + task);
        // Ошибка! this здесь уже не объект user
        console.log('Владелец: ' + this.name); // undefined
    });
}
};

user.showTasks();

```

Проблема 3: Инверсия контроля (Inversion of Control)

```

javascript

// Проблема: мы передаем контроль над выполнением
thirdPartyApi.makeRequest((error, result) => {
    // Что если thirdPartyApi:
    // 1. Вызовет callback несколько раз?
    // 2. Никогда не вызовет callback?
    // 3. Вызовет callback синхронно?
    // 4. Вызовет callback с неправильными аргументами?
});

```

Проблема 4: Сложность обработки ошибок

```

javascript

// Трудно обеспечить последовательную обработку ошибок
operation1((err1, result1) => {
    if (err1) return handleError(err1);

    operation2(result1, (err2, result2) => {
        if (err2) return handleError(err2);

        operation3(result2, (err3, result3) => {
            if (err3) return handleError(err3);
            // И так далее...
        });
    });
});

```

Проблема 5: Сложность с параллельным выполнением

```

javascript

// Выполнение нескольких асинхронных операций параллельно
let results = [];

```

```

let completed = 0;

operation1((err1, result1) => {
  if (err1) return handleError(err1);
  results[0] = result1;
  completed++;
  if (completed === 3) finalCallback(results);
});

operation2((err2, result2) => {
  if (err2) return handleError(err2);
  results[1] = result2;
  completed++;
  if (completed === 3) finalCallback(results);
});

operation3((err3, result3) => {
  if (err3) return handleError(err3);
  results[2] = result3;
  completed++;
  if (completed === 3) finalCallback(results);
});

// Проблемы:
// - Громоздкий код
// - Сложно масштабировать
// - Нет обработки ошибок для всей группы

```

11.2.4. Частичные решения до появления Promise

Решение 1: Именованные функции

```

javascript

// Вместо вложенных анонимных функций
function handleUserData(userError, userData) {
  if (userError) {
    console.error('Ошибка получения пользователя:', userError);
    return;
  }

  getUserPermissions(userData.id, handlePermissions);
}

```

```

function handlePermissions(permError, permissions) {
  if (permError) {
    console.error('Ошибка получения прав:', permError);
    return;
  }

  checkResourceAccess(permissions, resourceId, handleAccessCheck);
}

function handleAccessCheck(accessError, hasAccess) {
  if (accessError) {
    console.error('Ошибка проверки доступа:', accessError);
    return;
  }

  if (hasAccess) {
    loadResourceData(resourceId, handleResourceLoad);
  } else {
    console.log('Доступ запрещен');
  }
}

// Начало цепочки
getUserData(userId, handleUserData);

```

Решение 2: Async.js библиотека

```

javascript

const async = require('async');

// Серия последовательных операций
async.waterfall([
  function(callback) {
    getUserData(userId, callback);
  },
  function(userData, callback) {
    getUserPermissions(userData.id, callback);
  },
  function(permissions, callback) {
    checkResourceAccess(permissions, resourceId, callback);
  },
  function(hasAccess, callback) {
    if (hasAccess) {
      loadResourceData(resourceId, callback);
    }
  }
], callback);

```

```

    } else {
        callback(new Error('Доступ запрещен'));
    }
},
function(resourceData, callback) {
    processResourceData(resourceData, callback);
}
], function(error, result) {
    if (error) {
        console.error('Ошибка в цепочке:', error);
        return;
    }

    console.log('Финальный результат:', result);
});

// Параллельное выполнение
async.parallel([
    function(callback) { operation1(callback); },
    function(callback) { operation2(callback); },
    function(callback) { operation3(callback); }
], function(error, results) {
    if (error) {
        console.error('Ошибка в параллельных операциях:', error);
        return;
    }

    console.log('Все результаты:', results);
});

```

Решение 3: Паттерн "модуль"

```

javascript

const asyncModule = (function() {
    // Приватное состояние
    let pendingCallbacks = [];

    function operation1(callback) {
        // Некоторая асинхронная логика
        setTimeout(() => {
            callback(null, 'result1');
        }, 100);
    }
}

```

```

function operation2(input, callback) {
  setTimeout(() => {
    callback(null, input + ' -> result2');
  }, 100);
}

// Публичный API
return {
  executeSequence: function(finalCallback) {
    operation1((err1, result1) => {
      if (err1) return finalCallback(err1);

      operation2(result1, (err2, result2) => {
        if (err2) return finalCallback(err2);

        finalCallback(null, result2);
      });
    });
  }
};
})();

// Использование
asyncModule.executeSequence((error, result) => {
  if (error) {
    console.error('Ошибка:', error);
    return;
  }

  console.log('Результат:', result);
});

```

Решение 4: Обработка ошибок с domain (Node.js, устарело)

```

javascript

const domain = require('domain');
const d = domain.create();

d.on('error', (error) => {
  console.error('Доменная ошибка:', error);
});

d.run(() => {
  asyncOperation1((err1, result1) => {

```

```

    if (err1) throw err1; // Будет поймано domain

    asyncOperation2(result1, (err2, result2) => {
        if (err2) throw err2;

        console.log('Результат:', result2);
    });
});
});

```

11.2.5. Современные альтернативы

Переход к Promise

```

javascript

// Преобразование callback-based функции в Promise-based
function readFilePromise(path) {
    return new Promise((resolve, reject) => {
        fs.readFile(path, 'utf8', (error, data) => {
            if (error) {
                reject(error);
            } else {
                resolve(data);
            }
        });
    });
}

// Использование
readFilePromise('data.txt')
    .then(data => console.log('Данные:', data))
    .catch(error => console.error('Ошибка:', error));

```

Async/Await

```

javascript

async function processData() {
    try {
        const userData = await getUserDataAsync(userId);
        const permissions = await getUserPermissionsAsync(userData.id);
        const hasAccess = await checkResourceAccessAsync(permissions, resourceId);

        if (hasAccess) {

```



```

    const resourceData = await loadResourceDataAsync(resourceId);
    const result = await processResourceDataAsync(resourceData);
    await displayResultAsync(result);
    console.log('Готово!');
  } else {
    console.log('Доступ запрещен');
  }
} catch (error) {
  console.error('Ошибка в процессе:', error);
}
}

```

11.2.6. Лучшие практики работы с callback

1. Всегда обрабатывайте ошибки

javascript

// ПЛОХО

```

fs.readFile('file.txt', (err, data) => {
  console.log(data); // Если err != null, data будет undefined
});

```

// ХОРОШО

```

fs.readFile('file.txt', (err, data) => {
  if (err) {
    console.error('Ошибка чтения:', err);
    return;
  }
  console.log(data);
});

```

2. Избегайте вложенности с помощью именованных функций
3. Используйте библиотеки как `async.js` для сложных потоков
4. Рассмотрите переход на `Promise` для нового кода
5. Документируйте сигнатуру callback-функций

11.2.7. Упражнения для закрепления

Упражнение 1: Преобразуйте callback hell в именованные функции

javascript

// Исходный код с callback hell

```

getData((err, data) => {
  if (err) return console.error(err);
  processData(data, (err, processed) => {
    if (err) return console.error(err);

```

```

    saveData(processed, (err, savedId) => {
      if (err) return console.error(err);
      logResult(savedId, (err) => {
        if (err) return console.error(err);
        console.log('Готово');
      });
    });
  });
});
});

```

Упражнение 2: Создайте функцию `parallel` для выполнения callback-функций параллельно

javascript

```

// Задача: реализовать функцию parallel
function parallel(tasks, finalCallback) {
  // Реализуйте логику параллельного выполнения
  // tasks - массив функций вида (callback) => { ... }
  // finalCallback вызывается когда все tasks завершены
}

```

Упражнение 3: Реализуйте последовательное выполнение с обработкой ошибок

javascript

```

function series(tasks, finalCallback) {
  // Задача: выполнить tasks последовательно
  // Если любая задача возвращает ошибку, прервать выполнение
  // Результаты передавать от одной задачи к другой
}

```

11.2.8. Исторический контекст и эволюция

Callback-функции были первым и долгое время единственным механизмом асинхронного программирования в JavaScript. Их эволюция:

1. **1995-2005:** Callback как основной механизм
2. **2006-2009:** Появление библиотек (async.js, Q) для управления потоком
3. **2010-2014:** Появление Promises/A+ спецификации
4. **2015:** Promise как часть стандарта ES6
5. **2017:** Async/await в ES8

Заключение

Callback-функции — это фундаментальная концепция JavaScript, которая до сих пор широко используется, особенно в legacy коде и Node.js core API. Понимание callback необходимо для:

1. Работы со старым кодом
2. Использования Node.js core modules
3. Понимания эволюции JavaScript
4. Отладки асинхронного кода

Однако для новых проектов рекомендуется использовать Promise и async/await, которые решают большинство проблем callback-подхода, сохраняя при этом все его преимущества.

Ключевые выводы:

- Callback-функции — основа асинхронного JavaScript
- Callback hell — реальная проблема больших приложений
- Существуют паттерны для уменьшения вложенности
- Современные альтернативы (Promise, async/await) решают эти проблемы
- Понимание callback необходимо для работы с legacy кодом

11.3. Промисы (Promise): создание, обработка, цепочки

Введение: революция в асинхронном программировании

Promise (Обещание) — это объект в JavaScript, представляющий завершение (успешное или неудачное) асинхронной операции и её результат. Промисы стали стандартом ECMAScript 2015 (ES6) и кардинально изменили подход к асинхронному программированию, решив проблему "callback hell".

Исторический контекст

До появления промисов разработчики использовали различные библиотеки и реализации:

- **2007:** Dojo Deferred
- **2009:** jQuery Deferred
- **2010:** Q библиотека
- **2011:** Promises/A+ спецификация
- **2015:** Нативное Promise в ES6

11.3.1. Что такое Promise: философия и аналогии

Ментальная модель

Представьте, что Promise — это **квитанция в фастфуде**:

1. Вы делаете заказ (создаёте промис)
2. Получаете квитанцию (объект Promise)
3. Квитанция не является едой, но обещает её
4. Вы можете:
 - ➔ Дождаться готовности (`.then()`)
 - ➔ Уйти, если слишком долго (`.catch()`)
 - ➔ Сделать что-то в любом случае (`.finally()`)

Три состояния Promise

Каждый Promise находится в одном из трёх состояний:

javascript

```
// 1. PENDING (Ожидание) - начальное состояние
const pendingPromise = new Promise((resolve, reject) => {
  // Пока не вызваны resolve/reject - промис в состоянии PENDING
});

// 2. FULFILLED (Выполнено) - операция успешно завершена
const fulfilledPromise = Promise.resolve('Успех!');

// 3. REJECTED (Отклонено) - операция завершилась ошибкой
const rejectedPromise = Promise.reject(new Error('Ошибка!'));
```

11.3.2. Создание Promise

Базовый синтаксис

javascript

```
// Конструктор Promise принимает "исполнительную функцию" (executor)
const promise = new Promise((resolve, reject) => {
  // Эта функция выполняется СРАЗУ при создании Promise

  // Асинхронная операция
  setTimeout(() => {
    const success = Math.random() > 0.5;

    if (success) {
      // Успешное завершение
      resolve('Данные успешно получены!');
    } else {
      // Ошибка
      reject(new Error('Не удалось получить данные'));
    }
  }, 1000);
});
```

Практические примеры создания

Пример 1: Обёртка для setTimeout

javascript

```
function delay(ms, value) {
```

```

    return new Promise((resolve) => {
      setTimeout(() => resolve(value), ms);
    });
  }
}

```

// Использование

```

delay(1000, 'Hello after 1 second')
  .then(result => console.log(result));

```

Пример 2: Чтение файла с Promise (Node.js)

javascript

```

const fs = require('fs');

function readFilePromise(path, encoding = 'utf8') {
  return new Promise((resolve, reject) => {
    fs.readFile(path, encoding, (error, data) => {
      if (error) {
        reject(error); // Отклоняем промис с ошибкой
      } else {
        resolve(data); // Выполняем промис с данными
      }
    });
  });
}

```

// Использование

```

readFilePromise('data.txt')
  .then(data => console.log('Содержимое:', data))
  .catch(error => console.error('Ошибка чтения:', error));

```

Пример 3: AJAX запрос с Promise (браузер)

javascript

```

function fetchJson(url) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();

    xhr.open('GET', url);
    xhr.responseType = 'json';

    xhr.onload = () => {
      if (xhr.status >= 200 && xhr.status < 300) {
        resolve(xhr.response);
      }
    };
  });
}

```

```

    } else {
        reject(new Error(`HTTP ${xhr.status}: ${xhr.statusText}`));
    }
};

xhr.onerror = () => {
    reject(new Error('Сетевая ошибка'));
};

xhr.send();
});
}

```

// Современная версия с fetch API (которая уже возвращает Promise)

```

function fetchJsonModern(url) {
    return fetch(url)
        .then(response => {
            if (!response.ok) {
                throw new Error(`HTTP ${response.status}`);
            }
            return response.json();
        });
}

```

Пример 4: Проверка доступности изображения

javascript

```

function checkImage(url) {
    return new Promise((resolve, reject) => {
        const img = new Image();

        img.onload = () => {
            resolve({
                url,
                width: img.width,
                height: img.height,
                status: 'loaded'
            });
        };

        img.onerror = () => {
            reject(new Error(`Не удалось загрузить изображение: ${url}`));
        };
    });
}

```

```
    img.src = url;
  });
}

// Использование
checkImage('https://example.com/image.jpg')
  .then(info => console.log('Изображение загружено:', info))
  .catch(error => console.error('Ошибка:', error.message));
```

11.3.3. Статические методы Promise

Promise.resolve() - создание уже выполненного промиса

```
javascript

// Создание выполненного промиса с значением
const resolvedPromise = Promise.resolve('Готово!');

// Эквивалентно:
const equivalentPromise = new Promise((resolve) => {
  resolve('Готово!');
});

// Особые случаи:
Promise.resolve(42); // Promise с числом
Promise.resolve([1, 2, 3]); // Promise с массивом
Promise.resolve({ data: 'test' }); // Promise с объектом

// Если передать Promise, он вернется как есть
const originalPromise = new Promise(resolve => setTimeout(() => resolve('original'), 100));
const wrappedPromise = Promise.resolve(originalPromise);
console.log(originalPromise === wrappedPromise); // false, но они "эквивалентны"
```

Promise.reject() - создание отклонённого промиса

```
javascript

// Создание отклонённого промиса
const rejectedPromise = Promise.reject(new Error('Что-то пошло не так'));

// Эквивалентно:
const equivalentRejected = new Promise((resolve, reject) => {
  reject(new Error('Что-то пошло не так'));
});
```



```
});
```

```
// Использование для единообразной обработки ошибок
```

```
function getUserData(userId) {  
  if (!userId) {  
    // Ранний возврат отклонённого промиса  
    return Promise.reject(new Error('ID пользователя обязателен'));  
  }  
  
  return fetch(`/api/users/${userId}`)  
    .then(response => response.json());  
}
```

Promise.all() - выполнение нескольких промисов параллельно

```
javascript
```

```
// Все промисы должны выполниться успешно
```

```
const promise1 = delay(100, 'Результат 1');  
const promise2 = delay(200, 'Результат 2');  
const promise3 = delay(300, 'Результат 3');
```

```
Promise.all([promise1, promise2, promise3])  
  .then(results => {  
    console.log('Все промисы выполнены:', results);  
    // results = ['Результат 1', 'Результат 2', 'Результат 3']  
  })  
  .catch(error => {  
    console.error('Один из промисов отклонён:', error);  
    // Если любой промис отклонён - весь Promise.all отклоняется  
  });
```

```
// Практический пример: загрузка нескольких ресурсов
```

```
function loadMultipleResources(urls) {  
  const promises = urls.map(url => fetch(url).then(res => res.json()));  
  
  return Promise.all(promises)  
    .then(dataArray => {  
      // dataArray - массив результатов в том же порядке, что и urls  
      return dataArray.reduce((acc, data, index) => {  
        acc[urls[index]] = data;  
        return acc;  
      }, {});  
    });  
}
```

Promise.allSettled() - выполнение с результатами всех промисов (ES2020)

```
javascript

// В отличие от Promise.all, не прерывается при ошибках
const promises = [
  Promise.resolve('Успех 1'),
  Promise.reject(new Error('Ошибка 1')),
  Promise.resolve('Успех 2'),
  Promise.reject(new Error('Ошибка 2'))
];

Promise.allSettled(promises)
  .then(results => {
    results.forEach((result, index) => {
      if (result.status === 'fulfilled') {
        console.log(`Промис ${index}: Успех =`, result.value);
      } else {
        console.log(`Промис ${index}: Ошибка =`, result.reason.message);
      }
    });
  });

// Структура результата:
// [
//   { status: 'fulfilled', value: 'Успех 1' },
//   { status: 'rejected', reason: Error: Ошибка 1 },
//   { status: 'fulfilled', value: 'Успех 2' },
//   { status: 'rejected', reason: Error: Ошибка 2 }
// ]
```

Promise.race() - гонка промисов

```
javascript

// Возвращает результат первого завершившегося промиса (успешно или с ошибкой)
const fastPromise = delay(100, 'Быстрый');
const slowPromise = delay(500, 'Медленный');

Promise.race([fastPromise, slowPromise])
  .then(winner => {
    console.log('Победитель:', winner); // 'Быстрый'
  })
  .catch(error => {
    console.error('Первый промис отклонён:', error);
  });
```

```
});
```

```
// Практический пример: таймаут операции
```

```
function fetchWithTimeout(url, timeout = 5000) {  
  const fetchPromise = fetch(url);  
  const timeoutPromise = new Promise( (_, reject) => {  
    setTimeout(() => reject(new Error('Таймаут')), timeout);  
  });  
  
  return Promise.race([fetchPromise, timeoutPromise]);  
}
```

Promise.any() - первый успешно выполнившийся промис (ES2021)

```
javascript
```

```
// В отличие от race, игнорирует отклонённые промисы
```

```
const promises = [  
  Promise.reject(new Error('Ошибка 1')),  
  delay(200, 'Второй успех'),  
  delay(100, 'Первый успех'),  
  Promise.reject(new Error('Ошибка 2'))  
];
```

```
Promise.any(promises)  
  .then(firstSuccess => {  
    console.log('Первый успешный:', firstSuccess); // 'Первый успех'  
  })  
  .catch(error => {  
    // Срабатывает только если ВСЕ промисы отклонены  
    console.error('Все промисы отклонены:', error);  
    // error.aggregateErrors содержит все ошибки  
  });
```

11.3.4. Цепочки промисов (Promise Chaining)

Базовые цепочки

```
javascript
```

```
// Каждый .then() возвращает новый Promise
```

```
fetch('/api/user/123')  
  .then(response => response.json()) // Преобразуем ответ в JSON  
  .then(userData => {
```

```

    // Работаем с данными пользователя
    console.log('Пользователь:', userData.name);
    return fetch(`/api/posts?userId=${userData.id}`);
  })
  .then(response => response.json()) // Преобразуем посты
  .then(posts => {
    console.log('Посты пользователя:', posts.length);
    return processPosts(posts); // Возвращаем обработанные посты
  })
  .then(processedPosts => {
    console.log('Обработанные посты:', processedPosts);
  })
  .catch(error => {
    // Обрабатываем ЛЮБУЮ ошибку в цепочке
    console.error('Ошибка в цепочке:', error);
  });

```

Возврат значений в цепочках

javascript

// 1. Возврат обычного значения

```

Promise.resolve(10)
  .then(value => {
    console.log(value); // 10
    return value * 2; // Возвращаем 20
  })
  .then(value => {
    console.log(value); // 20
    return value + 5; // Возвращаем 25
  })
  .then(value => {
    console.log(value); // 25
  });

```

// 2. Возврат Promise

```

Promise.resolve(1)
  .then(value => {
    console.log(value); // 1
    // Возвращаем новый Promise
    return new Promise(resolve => {
      setTimeout(() => resolve(value * 2), 1000);
    });
  })
  .then(value => {

```

```
    console.log(value); // 2 (через 1 секунду)
    return Promise.resolve(value * 3);
  })
  .then(value => {
    console.log(value); // 6
  });
```

Обработка ошибок в цепочках

javascript

// 1. Единый catch в конце

```
doSomething()
  .then(result => doSomethingElse(result))
  .then(newResult => doThirdThing(newResult))
  .catch(error => {
    // Поймает любую ошибку в цепочке
    console.error('Ошибка:', error);
  });
```

// 2. Локальная обработка с продолжением цепочки

```
doSomething()
  .then(result => {
    return doSomethingRisky(result)
    .catch(error => {
      // Локальная обработка
      console.warn('Ошибка в рискованной операции:', error);
      return 'значение по умолчанию'; // Продолжаем цепочку
    });
  })
  .then(result => {
    // result будет либо успешным результатом,
    // либо 'значение по умолчанию'
    console.log('Результат:', result);
  })
  .catch(error => {
    // Не сработает для ошибок, обработанных выше
    console.error('Необработанная ошибка:', error);
  });
```

// 3. throw в then

```
Promise.resolve(1)
  .then(value => {
    if (value < 10) {
      throw new Error('Значение слишком маленькое');
    }
  });
```

```

    }
    return value;
  })
  .then(value => {
    console.log('Этот then не выполнится из-за ошибки выше');
  })
  .catch(error => {
    console.error('Поймана ошибка:', error.message);
    // Можно вернуть новое значение
    return 100;
  })
  .then(value => {
    console.log('Цепочка продолжается:', value); // 100
  });

```

finally() - выполнение независимо от результата

javascript

```

let isLoading = true;

fetch('/api/data')
  .then(response => response.json())
  .then(data => {
    console.log('Данные получены:', data);
  })
  .catch(error => {
    console.error('Ошибка загрузки:', error);
  })
  .finally(() => {
    // Выполнится в ЛЮБОМ случае (успех или ошибка)
    isLoading = false;
    console.log('Загрузка завершена (успешно или с ошибкой)');
  });

// finally НЕ получает аргументов
Promise.resolve(42)
  .finally(() => {
    console.log('Выполняется finally');
    // Можно вернуть Promise, чтобы дождаться его завершения
    return delay(1000);
  })
  .then(value => {
    console.log('Значение:', value); // 42
    // finally НЕ изменяет значение

```

```
});
```

11.3.5. Продвинутые паттерны Promise

Паттерн 1: Очередь выполнения (Promise Queue)

javascript

```
class PromiseQueue {
  constructor(concurrency = 1) {
    this.concurrency = concurrency;
    this.running = 0;
    this.queue = [];
  }

  add(task) {
    return new Promise((resolve, reject) => {
      this.queue.push({
        task,
        resolve,
        reject
      });
      this.next();
    });
  }

  next() {
    while (this.running < this.concurrency && this.queue.length) {
      const { task, resolve, reject } = this.queue.shift();
      this.running++;

      Promise.resolve(task())
        .then(resolve, reject)
        .finally(() => {
          this.running--;
          this.next();
        });
    }
  }
}
```

// Использование

```
const queue = new PromiseQueue(2); // Максимум 2 параллельных задачи
```

```

const results = [];
for (let i = 0; i < 5; i++) {
  queue.add(() => delay(1000, `Задача ${i}`))
    .then(result => {
      results.push(result);
      console.log('Завершено:', result);
    });
}

```

Паттерн 2: Ретри с экспоненциальной задержкой

javascript

```

function retryWithBackoff(operation, maxRetries = 3, initialDelay = 1000) {
  return new Promise((resolve, reject) => {
    let retries = 0;

    function attempt() {
      operation()
        .then(resolve)
        .catch(error => {
          retries++;

          if (retries > maxRetries) {
            reject(new Error(`Не удалось после ${maxRetries} попыток: ${error.message}`));
            return;
          }

          const delay = initialDelay * Math.pow(2, retries - 1);
          console.log(`Попытка ${retries} не удалась. Повтор через ${delay}мс`);

          setTimeout(attempt, delay);
        });
    }

    attempt();
  });
}

```

// Использование

```

retryWithBackoff(() => fetch('https://api.example.com/data'))
  .then(response => console.log('Успех!'))
  .catch(error => console.error('Финальная ошибка:', error));

```


Паттерн 3: Кэширование результатов Promise

javascript

```
function createCachedPromise(fn, ttl = 60000) { // ttl в миллисекундах
  const cache = new Map();

  return function(...args) {
    const key = JSON.stringify(args);
    const cached = cache.get(key);

    if (cached && Date.now() - cached.timestamp < ttl) {
      console.log('Возвращаем из кэша');
      return Promise.resolve(cached.value);
    }

    console.log('Выполняем функцию');
    return fn(...args)
      .then(value => {
        cache.set(key, {
          value,
          timestamp: Date.now()
        });
        return value;
      });
  };
}

// Использование
const cachedFetch = createCachedPromise(fetchJson, 30000); // Кэш на 30 секунд

// Первый вызов - выполняется запрос
cachedFetch('https://api.example.com/data')
  .then(data => console.log('Данные:', data));

// Второй вызов (в течение 30 сек) - из кэша
setTimeout(() => {
  cachedFetch('https://api.example.com/data')
    .then(data => console.log('Данные из кэша:', data));
}, 10000);
```

Паттерн 4: Ограничение одновременных запросов

javascript

```

function limitConcurrency(promises, limit) {
  const results = new Array(promises.length);
  let index = 0;
  let running = 0;
  let completed = 0;

  return new Promise((resolve) => {
    function runNext() {
      if (completed === promises.length) {
        resolve(results);
        return;
      }

      while (running < limit && index < promises.length) {
        const currentIndex = index++;
        running++;

        Promise.resolve(promises[currentIndex]())
          .then(result => {
            results[currentIndex] = { status: 'fulfilled', value: result };
          })
          .catch(error => {
            results[currentIndex] = { status: 'rejected', reason: error };
          })
          .finally(() => {
            running--;
            completed++;
            runNext();
          });
      }
    }

    runNext();
  });
}

```

// Использование

```

const tasks = Array.from({ length: 10 }, (_, i) => () =>
  delay(1000, `Задача ${i}`)
);

```

```

limitConcurrency(tasks, 3)
  .then(results => {
    console.log('Все задачи завершены:', results);
  });

```

```
});
```

11.3.6. Распространённые ошибки и антипаттерны

Ошибка 1: Потеря цепочки (Promise Hell)

```
javascript
```

```
// АНТИПАТТЕРН: Вложенные then (новый callback hell)
```

```
getUser()  
  .then(user => {  
    getPosts(user.id)  
      .then(posts => {  
        getComments(posts[0].id)  
          .then(comments => {  
            console.log(comments);  
          })  
        .catch(error => console.error(error));  
      })  
    .catch(error => console.error(error));  
  })  
  .catch(error => console.error(error));
```

```
// ПРАВИЛЬНО: Плоская цепочка
```

```
getUser()  
  .then(user => getPosts(user.id))  
  .then(posts => getComments(posts[0].id))  
  .then(comments => console.log(comments))  
  .catch(error => console.error(error));
```

Ошибка 2: Забытый return

```
javascript
```

```
// АНТИПАТТЕРН: Забыли return
```

```
Promise.resolve(1)  
  .then(value => {  
    value * 2; // Забыли return!  
  })  
  .then(value => {  
    console.log(value); // undefined  
  });
```

```
// ПРАВИЛЬНО: Всегда возвращайте значение
```

```
Promise.resolve(1)
  .then(value => {
    return value * 2; // Явный return
  })
  .then(value => {
    console.log(value); // 2
  });
```

Ошибка 3: Необработанные ошибки

```
javascript

// АНТИПАТТЕРН: Промис без обработки ошибок
const riskyPromise = new Promise((resolve, reject) => {
  throw new Error('Неожиданная ошибка');
});

// В браузере: Uncaught (in promise) Error: Неожиданная ошибка
// В Node.js: UnhandledPromiseRejectionWarning

// ПРАВИЛЬНО: Всегда добавляйте .catch()
riskyPromise
  .then(result => console.log(result))
  .catch(error => console.error('Обработанная ошибка:', error));
```

Ошибка 4: Создание лишних промисов

```
javascript

// АНТИПАТТЕРН: Избыточное оборачивание в Promise
function getUserData(userId) {
  return new Promise((resolve, reject) => {
    fetch(`/api/users/${userId}`)
      .then(response => response.json())
      .then(resolve) // Избыточно
      .catch(reject); // Избыточно
  });
}

// ПРАВИЛЬНО: Используйте существующие промисы
function getUserData(userId) {
  return fetch(`/api/users/${userId}`)
    .then(response => response.json());
}

// Еще лучше с async/await
```

```
async function getUserData(userId) {
  const response = await fetch(`/api/users/${userId}`);
  return response.json();
}
```

11.3.7. Отладка Promise

Методы отладки

javascript

// 1. Добавление отладочных then

```
fetch('/api/data')
  .then(response => {
    console.log('Получен ответ, статус:', response.status);
    return response.json();
  })
  .then(data => {
    console.log('Данные в JSON:', data);
    return processData(data);
  })
  .then(processed => {
    console.log('Обработанные данные:', processed);
  })
  .catch(error => {
    console.error('Ошибка в цепочке:', error);
    console.error('Стек вызовов:', error.stack);
  });
```

// 2. Использование .finally для отслеживания завершения

```
function trackPromise(promise, name) {
  console.time(name);
  return promise
    .finally(() => {
      console.timeEnd(name);
    });
}
```

// 3. Визуализация цепочки промисов

```
function visualizePromiseChain(promise, depth = 0) {
  const indent = ' '.repeat(depth);
```

promise

```

    .then(value => {
      console.log(`${indent}✓ Успех:`, value);
      return value;
    })
    .catch(error => {
      console.log(`${indent}✗ Ошибка:`, error.message);
      throw error;
    });

  return promise;
}

```

11.3.8. Производительность и оптимизация

Бенчмаркинг Promise

```

javascript

async function benchmarkPromise() {
  const iterations = 10000;

  console.time('Promise.resolve chain');
  let promise = Promise.resolve();
  for (let i = 0; i < iterations; i++) {
    promise = promise.then(() => {});
  }
  await promise;
  console.timeEnd('Promise.resolve chain');

  console.time('Direct execution');
  for (let i = 0; i < iterations; i++) {
    // Синхронный эквивалент
  }
  console.timeEnd('Direct execution');
}

benchmarkPromise();

```

Оптимизация: избегание микротасков

```

javascript

// ПЛОХО: Много микротасков
function processArray(array) {

```

```

let promise = Promise.resolve();

array.forEach(item => {
  promise = promise.then(() => processItem(item));
});

return promise;
}

// ЛУЧШЕ: Обработка массива целиком
async function processArrayBetter(array) {
  const results = [];

  for (const item of array) {
    results.push(await processItem(item));
  }

  return results;
}

// ЕЩЕ ЛУЧШЕ: Параллельная обработка
async function processArrayBest(array) {
  const promises = array.map(item => processItem(item));
  return Promise.all(promises);
}

```

11.3.9. Упражнения для закрепления

Упражнение 1: Создание утилит для работы с Promise

```

javascript

// Задача 1: Создайте функцию promisify
function promisify(fn) {
  // Преобразует callback-based функцию в Promise-based
  // Пример: promisify(fs.readFile) -> возвращает функцию, возвращающую Promise
}

// Задача 2: Реализуйте Promise.map
Promise.map = function(array, mapper, concurrency = Infinity) {
  // Применяет mapper к каждому элементу с ограничением параллелизма
};

```

```
// Задача 3: Создайте таймаут для Promise
Promise.withTimeout = function(promise, timeout, errorMessage = 'Таймаут') {
  // Возвращает promise, который отклоняется после timeout
};
```

Упражнение 2: Построение сложной цепочки

javascript

```
// Смоделируйте процесс оформления заказа:
// 1. Получить пользователя
// 2. Проверить баланс
// 3. Создать заказ
// 4. Спросить оплату
// 5. Отправить уведомление
// 6. Обновить инвентарь

// Каждый шаг может завершиться ошибкой
// Некоторые шаги можно выполнять параллельно
// Добавьте логирование и обработку ошибок
```

Упражнение 3: Отладка промисов

javascript

```
// Дана неправильно работающая цепочка промисов, найдите и исправьте ошибки
function buggyPromiseChain() {
  return getUser(123)
    .then(user => {
      console.log('Пользователь:', user);
      getOrders(user.id);
    })
    .then(orders => {
      console.log('Заказы:', orders);
      return processOrders(orders);
    })
    .then(() => {
      console.log('Готово!');
    });
}

// Вопросы:
// 1. Почему getOrders не возвращает promise?
// 2. Что произойдет, если getUser вернет ошибку?
// 3. Как улучшить обработку ошибок?
```


Заключение

Promise — это фундаментальная концепция современного JavaScript, которая:

1. **Решает проблему `callback hell`** через плоские цепочки
2. **Обеспечивает стандартизированную обработку ошибок** через `.catch()`
3. **Позволяет комбинировать асинхронные операции** через `Promise.all`, `Promise.race` и др.
4. **Подготавливает почву для `async/await`** — следующего шага в эволюции

Ключевые принципы для запоминания:

1. Каждый `.then()` возвращает **новый Promise**
2. Возвращайте значения из `.then()` для передачи по цепочке
3. Используйте один `.catch()` в конце для глобальной обработки ошибок
4. `.finally()` выполняется в любом случае и не изменяет значение
5. Избегайте создания избыточных промисов
6. Для параллельного выполнения используйте `Promise.all()`
7. Всегда обрабатывайте возможные отклонения промисов

Promise не заменяет полностью callback-функции, но предоставляет более удобный и надёжный способ работы с асинхронным кодом. В следующем разделе мы рассмотрим `async/await` — синтаксический сахар над промисами, делающий асинхронный код ещё более читаемым.

11.4. Async/await: синтаксический сахар для промисов

Введение: эволюция асинхронного кода в JavaScript

Async/await — это синтаксическая надстройка над промисами, представленная в ECMAScript 2017 (ES8), которая позволяет писать асинхронный код в синхронном стиле. Это не замена промисам, а способ сделать работу с ними более удобной и читаемой.

Исторический путь асинхронного программирования:

javascript

// 1. Callback Hell (до 2015)

```
getData((err, data) => {  
  if (err) handleError(err);  
  processData(data, (err, processed) => {  
    if (err) handleError(err);  
    saveData(processed, (err, result) => {  
      if (err) handleError(err);  
      console.log('Готово:', result);  
    });  
  });  
});
```

// 2. Promise Chains (ES6, 2015)

```
getData()  
  .then(data => processData(data))  
  .then(processed => saveData(processed))  
  .then(result => console.log('Готово:', result))  
  .catch(error => handleError(error));
```

// 3. Async/Await (ES8, 2017)

```
async function process() {  
  try {  
    const data = await getData();  
    const processed = await processData(data);  
    const result = await saveData(processed);  
    console.log('Готово:', result);  
  } catch (error) {  
    handleError(error);  
  }  
}
```

```
}  
}
```

11.4.1. Основные концепции async/await

Ключевое слово `async`

Функция, объявленная с ключевым словом `async`, всегда возвращает Promise:

```
javascript  
  
// Обычная функция  
function normalFunction() {  
    return 42; // Возвращает число  
}  
  
// Async функция  
async function asyncFunction() {  
    return 42; // Возвращает Promise, который выполняется со значением 42  
}  
  
// Проверка  
console.log(normalFunction()); // 42  
console.log(asyncFunction()); // Promise {<fulfilled>: 42}  
  
// Можно использовать как обычный Promise  
asyncFunction()  
    .then(value => console.log('Результат:', value)); // Результат: 42
```

Неявное оборачивание в Promise

```
javascript  
  
// Любое значение, возвращаемое async функцией, оборачивается в Promise  
async function example() {  
    return 42; // Promise.resolve(42)  
    return 'Hello'; // Promise.resolve('Hello')  
    return [1, 2, 3]; // Promise.resolve([1, 2, 3])  
    return { data: 'test' }; // Promise.resolve({ data: 'test' })  
  
    // Даже если возвращается Promise, он остается Promise  
    return Promise.resolve('already a promise'); // Promise.resolve('already a promise')  
  
    // Исключения также оборачиваются в отклоненный Promise
```

```
    throw new Error('Something went wrong'); // Promise.reject(new Error(...))
}
```

// Использование

```
example()
  .then(value => console.log('Успех:', value))
  .catch(error => console.error('Ошибка:', error));
```

Ключевое слово `await`

`await` можно использовать только внутри `async` функций. Он приостанавливает выполнение функции до разрешения Promise:

javascript

```
async function example() {
  console.log('Начало');

  // Ожидание выполнения Promise
  const result = await Promise.resolve('Результат');
  console.log('Получен результат:', result);

  console.log('Конец');
  return 'Завершено';
}

// Вызов
example().then(value => console.log('Финальный результат:', value));

// Вывод:
// Начало
// Получен результат: Результат
// Конец
// Финальный результат: Завершено
```

11.4.2. Детальный разбор работы `await`

Как работает `await` под капотом

javascript

// Код с async/await

```
async function getUserData(userId) {
  const user = await fetchUser(userId);
```

```

    const posts = await fetchUserPosts(user.id);
    const comments = await fetchPostComments(posts[0].id);
    return { user, posts, comments };
}

// Примерно так это преобразуется в Promise (концептуально)
function getUserDataPromise(userId) {
    return fetchUser(userId)
        .then(user => {
            return fetchUserPosts(user.id)
                .then(posts => {
                    return fetchPostComments(posts[0].id)
                        .then(comments => {
                            return { user, posts, comments };
                        });
                });
        });
}

```

Приостановка и возобновление выполнения

javascript

```

async function demonstrateAwait() {
    console.log('1. Начало функции');

    // Создаем Promise, который разрешится через 1 секунду
    const promise1 = new Promise(resolve => {
        setTimeout(() => {
            console.log('2. Promise 1 выполнен');
            resolve('Результат 1');
        }, 1000);
    });

    console.log('3. Перед await promise1');
    const result1 = await promise1; // Выполнение ПРИОСТАНАВЛИВАЕТСЯ здесь
    console.log('4. После await promise1:', result1);

    // Второй await
    const promise2 = new Promise(resolve => {
        setTimeout(() => {
            console.log('5. Promise 2 выполнен');
            resolve('Результат 2');
        }, 500);
    });
}

```

```

console.log('6. Перед await promise2');
const result2 = await promise2; // Снова приостановка
console.log('7. После await promise2:', result2);

return { result1, result2 };
}

// Вывод будет:
// 1. Начало функции
// 3. Перед await promise1
// 2. Promise 1 выполнен (через 1 секунду)
// 4. После await promise1: Результат 1
// 6. Перед await promise2
// 5. Promise 2 выполнен (через 0.5 секунды)
// 7. После await promise2: Результат 2

```

11.4.3. Обработка ошибок с async/await

Конструкция try/catch

```

javascript

// Базовый пример обработки ошибок
async function fetchDataWithRetry(url, retries = 3) {
  for (let i = 0; i < retries; i++) {
    try {
      console.log(`Попытка ${i + 1} из ${retries}`);
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`HTTP ${response.status}: ${response.statusText}`);
      }

      const data = await response.json();
      return data; // Успех - выходим из цикла
    } catch (error) {
      console.error(`Попытка ${i + 1} не удалась:`, error.message);

      // Если это последняя попытка - пробрасываем ошибку дальше
      if (i === retries - 1) {
        throw new Error(`Не удалось получить данные после ${retries} попыток`);
      }
    }
  }
}

```

```

    }

    // Ждем перед следующей попыткой (экспоненциальная задержка)
    await new Promise(resolve => setTimeout(resolve, 1000 * Math.pow(2, i)));
  }
}
}

```

// Использование

```

fetchDataWithRetry('https://api.example.com/data')
  .then(data => console.log('Данные получены:', data))
  .catch(error => console.error('Финальная ошибка:', error));

```

Несколько await в одном try/catch

javascript

```

async function processUserOrder(userId, orderId) {
  try {
    // Все await внутри одного try/catch
    const user = await getUser(userId);
    const order = await getOrder(orderId);
    const payment = await processPayment(user, order);
    const confirmation = await sendConfirmation(user, payment);

    console.log('Заказ успешно обработан');
    return confirmation;
  } catch (error) {
    // Ловим ЛЮБУЮ ошибку из любого await
    console.error('Ошибка обработки заказа:', error);

    // Можно выполнить откат (rollback)
    await cancelOrder(orderId);
    await notifyAdmin(userId, error);

    // Прокрасываем ошибку дальше
    throw error;
  }
}

```

Отдельная обработка для каждого await

javascript

```

async function robustDataProcessing() {

```

```

let user = null;
let posts = null;
let comments = null;

// Обработка пользователя с fallback
try {
  user = await getUser(123);
} catch (error) {
  console.warn('Не удалось получить пользователя, используем по умолчанию');
  user = getDefaultUser();
}

// Обработка постов с fallback
try {
  posts = await getUserPosts(user.id);
} catch (error) {
  console.warn('Не удалось получить посты, используем пустой массив');
  posts = [];
}

// Обработка комментариев (не критично)
try {
  if (posts.length > 0) {
    comments = await getPostComments(posts[0].id);
  }
} catch (error) {
  console.warn('Не удалось получить комментарии, продолжаем без них');
  // comments остается null
}

return { user, posts, comments };
}

```

Использование .catch() с async функциями

javascript

```

// Async функция возвращает Promise, поэтому можно использовать .catch()
async function riskyOperation() {
  const result = await doSomethingRisky();
  return result;
}

// Вариант 1: Обработка через .catch()
riskyOperation()

```



```

    .then(result => console.log('Успех:', result))
    .catch(error => console.error('Ошибка:', error));

// Вариант 2: Обертка с обработкой ошибок
async function safeOperation() {
    return riskyOperation().catch(error => {
        console.error('Ошибка перехвачена:', error);
        return 'значение по умолчанию';
    });
}

// Вариант 3: Комбинированный подход
async function main() {
    try {
        const result = await riskyOperation().catch(error => {
            // Локальная обработка
            console.warn('Локальная ошибка:', error);
            throw new Error('Обернутая ошибка');
        });

        console.log('Результат:', result);
    } catch (error) {
        console.error('Глобальная ошибка:', error);
    }
}

```

11.4.4. Параллельное выполнение с async/await

Неправильный последовательный подход

```

javascript

// АНТИПАТТЕРН: Последовательные await для независимых операций
async function slowVersion() {
    console.time('slowVersion');

    // Эти операции НЕ зависят друг от друга, но выполняются последовательно
    const user = await fetchUser(123);    // Ждем 200мс
    const posts = await fetchPosts(123);  // Ждем 150мс (после user)
    const comments = await fetchComments(123); // Ждем 100мс (после posts)

    console.timeEnd('slowVersion'); // ~450мс
    return { user, posts, comments };
}

```

```
}
```

Правильное параллельное выполнение

javascript

// Правильно: Запускаем все операции параллельно

```
async function fastVersion() {  
  console.time('fastVersion');  
  
  // Запускаем все промисы одновременно  
  const userPromise = fetchUser(123);      // Начало  
  const postsPromise = fetchPosts(123);    // Начало (одновременно)  
  const commentsPromise = fetchComments(123); // Начало (одновременно)  
  
  // Ожидаем завершения всех промисов  
  const user = await userPromise;  
  const posts = await postsPromise;  
  const comments = await commentsPromise;  
  
  console.timeEnd('fastVersion'); // ~200мс (максимальное время, а не сумма)  
  return { user, posts, comments };  
}
```

Использование Promise.all()

javascript

// Оптимально: Promise.all() для параллельного выполнения

```
async function optimalVersion() {  
  console.time('optimalVersion');  
  
  // Promise.all запускает все промисы и ждет их завершения  
  const [user, posts, comments] = await Promise.all([  
    fetchUser(123),  
    fetchPosts(123),  
    fetchComments(123)  
  ]);  
  
  console.timeEnd('optimalVersion'); // ~200мс  
  return { user, posts, comments };  
}
```

// Расширенный пример с обработкой ошибок

```
async function robustParallelExecution() {  
  try {
```

```

const [user, posts, settings] = await Promise.all([
  fetchUser(123).catch(error => ({ error, data: null })),
  fetchPosts(123).catch(error => ({ error, data: [] })),
  fetchSettings(123).catch(error => ({ error, data: {} }))
]);

// Проверяем, были ли ошибки в отдельных промисах
const errors = [user, posts, settings]
  .filter(item => item.error)
  .map(item => item.error);

if (errors.length > 0) {
  console.warn('Некоторые данные не загружены:', errors);
}

return {
  user: user.data,
  posts: posts.data,
  settings: settings.data,
  partialErrors: errors.length > 0
};

} catch (error) {
  // Сработает, если Promise.all полностью провалится
  console.error('Критическая ошибка:', error);
  throw error;
}
}

```

Использование Promise.allSettled() (ES2020)

javascript

```

async function getAllSettledExample() {
  const promises = [
    fetchUser(123),
    fetchPosts(123),
    fetchComments(123),
    fetchNonExistent() // Этот вызов завершится ошибкой
  ];

  const results = await Promise.allSettled(promises);

  // Обработка результатов
  const successful = results

```

```

    .filter(result => result.status === 'fulfilled')
    .map(result => result.value);

const failed = results
    .filter(result => result.status === 'rejected')
    .map(result => result.reason);

console.log('Успешные:', successful.length);
console.log('Неудачные:', failed.length);

return { successful, failed };
}

```

Использование Promise.race() для таймаутов

javascript

```

async function fetchWithTimeout(url, timeout = 5000) {
    // Создаем промис для таймаута
    const timeoutPromise = new Promise( (_, reject) => {
        setTimeout(() => {
            reject(new Error(`Таймаут ${timeout}мс превышен`));
        }, timeout);
    });

    // Создаем промис для fetch
    const fetchPromise = fetch(url);

    // Гонка: что завершится первым?
    try {
        const response = await Promise.race([fetchPromise, timeoutPromise]);
        return await response.json();
    } catch (error) {
        console.error('Ошибка загрузки:', error.message);
        throw error;
    }
}

// Расширенная версия с повторными попытками
async function fetchWithTimeoutAndRetry(url, timeout = 5000, retries = 3) {
    for (let attempt = 1; attempt <= retries; attempt++) {
        try {
            console.log(`Попытка ${attempt} из ${retries}`);
            return await fetchWithTimeout(url, timeout);
        } catch (error) {

```

```

    if (attempt === retries) {
        throw new Error(`Не удалось за ${retries} попыток: ${error.message}`);
    }

    // Экспоненциальная задержка перед следующей попыткой
    const delay = 1000 * Math.pow(2, attempt - 1);
    console.log(`Ждем ${delay}мс перед следующей попыткой`);
    await new Promise(resolve => setTimeout(resolve, delay));
  }
}
}

```

11.4.5. Паттерны и лучшие практики

Паттерн 1: Ранний возврат (Early Return)

```

javascript

// ПЛОХО: Вложенные условия
async function processOrder(orderId) {
    const order = await getOrder(orderId);

    if (order) {
        const user = await getUser(order.userId);

        if (user) {
            const payment = await processPayment(user, order);

            if (payment.success) {
                await sendConfirmation(user, order);
                return 'Успех';
            } else {
                throw new Error('Ошибка оплаты');
            }
        } else {
            throw new Error('Пользователь не найден');
        }
    } else {
        throw new Error('Заказ не найден');
    }
}

// ХОРОШО: Ранний возврат и плоская структура

```

```

async function processOrderBetter(orderId) {
  const order = await getOrder(orderId);
  if (!order) {
    throw new Error('Заказ не найден');
  }

  const user = await getUser(order.userId);
  if (!user) {
    throw new Error('Пользователь не найден');
  }

  const payment = await processPayment(user, order);
  if (!payment.success) {
    throw new Error('Ошибка оплаты');
  }

  await sendConfirmation(user, order);
  return 'Успех';
}

```

Паттерн 2: Оптимизация с memoization

javascript

```

function createMemoizedAsync(fn, ttl = 60000) {
  const cache = new Map();

  return async function(...args) {
    const key = JSON.stringify(args);
    const cached = cache.get(key);

    if (cached && Date.now() - cached.timestamp < ttl) {
      console.log('Возвращаем из кэша');
      return cached.value;
    }

    console.log('Выполняем функцию');
    const value = await fn(...args);

    cache.set(key, {
      value,
      timestamp: Date.now()
    });

    return value;
  };
}

```

```

    };
}

// Использование
const memoizedFetch = createMemoizedAsync(async (url) => {
    const response = await fetch(url);
    return response.json();
}, 30000);

// Первый вызов - выполняется запрос
await memoizedFetch('https://api.example.com/data');

// Последующие вызовы (в течение 30 сек) - из кэша
await memoizedFetch('https://api.example.com/data');

```

Паттерн 3: Очередь задач с ограничением параллелизма

javascript

```

class AsyncQueue {
    constructor(concurrency = 1) {
        this.concurrency = concurrency;
        this.running = 0;
        this.queue = [];
    }

    async add(task) {
        return new Promise((resolve, reject) => {
            this.queue.push({ task, resolve, reject });
            this.next();
        });
    }

    next() {
        while (this.running < this.concurrency && this.queue.length) {
            const { task, resolve, reject } = this.queue.shift();
            this.running++;

            Promise.resolve(task())
                .then(resolve, reject)
                .finally(() => {
                    this.running--;
                    this.next();
                });
        }
    }
}

```

```

}

async processAll(tasks) {
  const results = [];
  const errors = [];

  for (const task of tasks) {
    try {
      const result = await this.add(task);
      results.push(result);
    } catch (error) {
      errors.push(error);
    }
  }

  // Ждем завершения всех задач
  while (this.running > 0) {
    await new Promise(resolve => setTimeout(resolve, 10));
  }

  return { results, errors };
}

// Использование
const queue = new AsyncQueue(3); // Максимум 3 параллельные задачи

const tasks = Array.from({ length: 10 }, (_, i) => async () => {
  console.log(`Задача ${i} начата`);
  await new Promise(resolve => setTimeout(resolve, 1000));
  console.log(`Задача ${i} завершена`);
  return `Результат ${i}`;
});

const { results, errors } = await queue.processAll(tasks);
console.log(`Выполнено: ${results.length}, Ошибок: ${errors.length}`);

```

Паттерн 4: Отмена асинхронных операций

javascript

```

class CancellablePromise {
  constructor(executor) {
    let cancel = null;

```



```

const promise = new Promise((resolve, reject) => {
  cancel = (reason = 'Операция отменена') => {
    reject(new Error(reason));
  };

  executor(resolve, reject, () => cancel);
});

promise.cancel = cancel;
return promise;
}
}

```

// Использование

```

async function fetchWithCancellation(url) {
  const controller = new AbortController();
  const signal = controller.signal;

  const fetchPromise = fetch(url, { signal });

```

// Создаем отменяемый промис

```

const cancellablePromise = new CancellablePromise(async (resolve, reject, getCancel) =>
{
  try {
    const response = await fetchPromise;
    const data = await response.json();
    resolve(data);
  } catch (error) {
    if (error.name === 'AbortError') {
      reject(new Error('Запрос отменен'));
    } else {
      reject(error);
    }
  }
});

```

// Добавляем метод отмены

```

cancellablePromise.cancel = () => {
  controller.abort();
  getCancel()('Отменено пользователем');
};

return cancellablePromise;
}

```

```
// Пример использования
async function example() {
  const promise = fetchWithCancellation('https://api.example.com/slow-data');

  // Устанавливаем таймаут на отмену
  setTimeout(() => {
    promise.cancel();
  }, 2000);

  try {
    const data = await promise;
    console.log('Данные получены:', data);
  } catch (error) {
    console.log('Ошибка:', error.message); // "Отменено пользователем"
  }
}
```

11.4.6. Распространённые ошибки и антипаттерны

Ошибка 1: Забытый await

```
javascript

// АНТИПАТТЕРН: Забыли await
async function getData() {
  const promise = fetch('/api/data'); // Забыли await!
  console.log(promise); // Promise {<pending>}, а не данные

  const data = promise.json(); // Ошибка: promise.json is not a function
  return data;
}

// ПРАВИЛЬНО:
async function getData() {
  const response = await fetch('/api/data');
  const data = await response.json();
  return data;
}
```

Ошибка 2: Ненужная последовательность

```
javascript
```

// АНТИПАТТЕРН: Последовательные await для независимых операций

```
async function slowOperation() {  
  const a = await fetchA(); // Ждем завершения  
  const b = await fetchB(); // Ждем завершения (а могло бы выполняться параллельно с А)  
  const c = await fetchC(); // Ждем завершения (а могло бы выполняться параллельно)  
  return { a, b, c };  
}
```

// ПРАВИЛЬНО: Параллельное выполнение

```
async function fastOperation() {  
  const [a, b, c] = await Promise.all([  
    fetchA(),  
    fetchB(),  
    fetchC()  
  ]);  
  return { a, b, c };  
}
```

Ошибка 3: Избыточное использование try/catch

javascript

// АНТИПАТТЕРН: Избыточные try/catch

```
async function overlyCautious() {  
  let user;  
  try {  
    user = await getUser();  
  } catch (error) {  
    console.error('Ошибка получения пользователя:', error);  
    throw error;  
  }  
  
  let posts;  
  try {  
    posts = await getPosts(user.id);  
  } catch (error) {  
    console.error('Ошибка получения постов:', error);  
    throw error;  
  }  
  
  // ... и так далее  
}
```

// ПРАВИЛЬНО: Один try/catch для связанных операций

```
async function concise() {
```

```

try {
  const user = await getUser();
  const posts = await getPosts(user.id);
  const comments = await getComments(posts[0].id);
  return { user, posts, comments };
} catch (error) {
  console.error('Ошибка в процессе:', error);
  throw error;
}
}

```

Ошибка 4: Смешивание стилей

javascript

// АНТИПАТТЕРН: Смешивание async/await и then/catch

```

async function mixedStyle() {
  const user = await getUser()
    .then(user => {
      return getPosts(user.id);
    })
    .then(posts => {
      return getComments(posts[0].id);
    })
    .catch(error => {
      console.error('Ошибка:', error);
      throw error;
    });

  return user;
}

```

// ПРАВИЛЬНО: Единый стиль

```

async function consistentStyle() {
  try {
    const user = await getUser();
    const posts = await getPosts(user.id);
    const comments = await getComments(posts[0].id);
    return comments;
  } catch (error) {
    console.error('Ошибка:', error);
    throw error;
  }
}

```

11.4.7. Производительность и оптимизация

Микрооптимизации

javascript

// ПЛОХО: Создание лишних промисов

```
async function inefficient() {  
    const result = await Promise.resolve(await someAsyncOp()); // Два await!  
    return await Promise.resolve(result); // Лишний Promise.resolve  
}
```

// ХОРОШО:

```
async function efficient() {  
    const result = await someAsyncOp();  
    return result;  
}
```

// ПЛОХО: Последовательные await в цикле

```
async function processArraySequentially(array) {  
    const results = [];  
    for (const item of array) {  
        results.push(await processItem(item)); // Ждем каждый элемент  
    }  
    return results;  
}
```

// ХОРОШО: Параллельная обработка

```
async function processArrayInParallel(array) {  
    const promises = array.map(item => processItem(item));  
    return Promise.all(promises);  
}
```

// КОМПРОМИСС: Пакетная обработка

```
async function processArrayInBatches(array, batchSize = 10) {  
    const results = [];  
  
    for (let i = 0; i < array.length; i += batchSize) {  
        const batch = array.slice(i, i + batchSize);  
        const batchResults = await Promise.all(  
            batch.map(item => processItem(item))  
        );  
        results.push(...batchResults);  
    }  
}
```

```
    return results;
}
```

Измерение производительности

javascript

```
async function benchmarkAsync() {
  // Тест 1: Последовательное выполнение
  console.time('sequential');
  await task1();
  await task2();
  await task3();
  console.timeEnd('sequential');

  // Тест 2: Параллельное выполнение
  console.time('parallel');
  await Promise.all([task1(), task2(), task3()]);
  console.timeEnd('parallel');

  // Тест 3: Цикл с await
  console.time('loopWithAwait');
  for (let i = 0; i < 1000; i++) {
    await microTask();
  }
  console.timeEnd('loopWithAwait');

  // Тест 4: Promise.all в цикле
  console.time('batchProcessing');
  const batches = [];
  for (let i = 0; i < 1000; i += 100) {
    const batch = Array.from({ length: 100 }, (_, j) => microTask());
    batches.push(Promise.all(batch));
  }
  await Promise.all(batches);
  console.timeEnd('batchProcessing');
}
```

11.4.8. Расширенные возможности и будущее

Top-level await (ES2022)

javascript

// В модулях ES6 теперь можно использовать await на верхнем уровне

// Раньше нужно было оборачивать в async IIFE

```
(async () => {  
  const data = await fetchData();  
  console.log(data);  
})();
```

// Теперь можно так (только в ES-модулях)

```
const data = await fetchData();  
console.log(data);
```

// Практический пример в модуле

// config.js

```
export const config = await fetch('/config.json').then(r => r.json());
```

// main.js

```
import { config } from './config.js';  
console.log('Конфигурация загружена:', config);
```

Async генераторы и итераторы

javascript

```
async function* asyncGenerator() {  
  let i = 0;  
  while (i < 5) {  
    // Имитируем асинхронную операцию  
    await new Promise(resolve => setTimeout(resolve, 100));  
    yield i++;  
  }  
}
```

// Использование for await...of

```
async function process() {  
  for await (const value of asyncGenerator()) {  
    console.log('Получено:', value);  
  }  
}
```

// Практический пример: потоковая обработка данных

```
async function* streamLines(url) {  
  const response = await fetch(url);  
  const reader = response.body.getReader();  
  const decoder = new TextDecoder();
```

```

try {
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    const chunk = decoder.decode(value);
    const lines = chunk.split('\n');

    for (const line of lines) {
      if (line.trim()) {
        yield JSON.parse(line);
      }
    }
  }
} finally {
  reader.releaseLock();
}
}

// Использование
for await (const item of streamLines('/api/stream')) {
  console.log('Получен элемент:', item);
  // Обработка в реальном времени
}

```

Интеграция с другими асинхронными API

```

javascript

// Web Workers
async function runInWorker(code) {
  const worker = new Worker(URL.createObjectURL(
    new Blob([code], { type: 'application/javascript' })
  ));

  return new Promise((resolve, reject) => {
    worker.onmessage = (event) => resolve(event.data);
    worker.onerror = reject;
    worker.postMessage('start');
  });
}

// Service Workers
async function registerServiceWorker() {

```



```

if ('serviceWorker' in navigator) {
  try {
    const registration = await navigator.serviceWorker.register('/sw.js');
    console.log('Service Worker зарегистрирован:', registration);
    return registration;
  } catch (error) {
    console.error('Ошибка регистрации:', error);
  }
}
}

```

// File System Access API

```

async function saveFile(content) {
  try {
    const handle = await window.showSaveFilePicker({
      types: [{
        description: 'Text Files',
        accept: { 'text/plain': ['.txt'] },
      }],
    });

    const writable = await handle.createWritable();
    await writable.write(content);
    await writable.close();

    console.log('Файл сохранен');
  } catch (error) {
    console.error('Ошибка сохранения:', error);
  }
}

```

11.4.9. Упражнения для закрепления

Упражнение 1: Рефакторинг callback hell в async/await

javascript

// Исходный код с callback hell

```

function processOrderOld(orderId, callback) {
  getOrder(orderId, (err, order) => {
    if (err) return callback(err);

    getUser(order.userId, (err, user) => {

```

```

    if (err) return callback(err);

    checkInventory(order.items, (err, inStock) => {
        if (err) return callback(err);

        if (!inStock) return callback(new Error('Нет в наличии'));

        processPayment(user, order.total, (err, paymentId) => {
            if (err) return callback(err);

            sendConfirmation(user.email, order, (err) => {
                if (err) return callback(err);

                callback(null, { orderId, paymentId, status: 'completed' });
            });
        });
    });
});
});
}

```

// Задача: Переписать на async/await

```

async function processOrderNew(orderId) {
    // Ваш код здесь
}

```

Упражнение 2: Создание утилитных функций

javascript

// Задача 1: Создайте функцию withRetry

```

async function withRetry(operation, retries = 3, delay = 1000) {
    // Повторяет операцию при ошибках с экспоненциальной задержкой
}

```

// Задача 2: Создайте функцию timeout

```

async function timeout(promise, ms, errorMessage = 'Таймаут') {
    // Отклоняет промис, если он не выполнен за указанное время
}

```

// Задача 3: Создайте функцию pool

```

async function pool(tasks, concurrency) {
    // Выполняет задачи с ограничением параллелизма
}

```

Упражнение 3: Оптимизация существующего кода

javascript

// Оптимизируйте этот код

```
async function unoptimizedCode() {
  const config = await loadConfig();
  const user = await getUser();
  const products = await getProducts();

  const cart = [];
  for (const product of products) {
    const details = await getProductDetails(product.id);
    const price = await calculatePrice(details, user);
    cart.push({ ...details, price });
  }

  const total = await calculateTotal(cart);
  const invoice = await createInvoice(user, cart, total);

  return invoice;
}
```

// Ваши оптимизации:

// 1. Какие операции можно выполнять параллельно?

// 2. Как оптимизировать цикл с await?

// 3. Как обработать возможные ошибки?

Заключение

Async/await — это мощный инструмент, который делает асинхронный код в JavaScript:

1. **Более читаемым** — выглядит как синхронный код
2. **Более простым в отладке** — стектрейсы сохраняются
3. **Более удобным в обслуживании** — плоская структура вместо вложенных коллбэков

Ключевые принципы:

1. **Всегда используйте try/catch** для обработки ошибок в async функциях
2. **Избегайте последовательных await** для независимых операций
3. **Используйте Promise.all()** для параллельного выполнения
4. **Не забывайте про await** при вызове async функций

5. Помните о состоянии **pending** — async функции всегда возвращают Promise

Когда использовать:

- **Async/await** — для последовательных операций, обработки ошибок, читаемости
- **Promise** — для параллельного выполнения, комбинирования операций
- **Callbacks** — для совместимости со старым кодом, событийных систем

В следующем разделе мы рассмотрим обработку ошибок в асинхронном коде, где `async/await` показывает свою настоящую силу, делая обработку ошибок такой же простой, как в синхронном коде.

11.5. Обработка ошибок в асинхронном коде

Введение: важность правильной обработки ошибок

Обработка ошибок в асинхронном коде — критически важный аспект разработки надежных приложений. В JavaScript ошибки в асинхронных операциях могут возникать по разным причинам: сетевые проблемы, неверные данные, таймауты, проблемы с памятью и т.д. Правильная обработка этих ошибок определяет стабильность и отказоустойчивость всего приложения.

Историческая эволюция обработки ошибок:

```
javascript

// 1. Callback style (Error-First Callbacks)
fs.readFile('file.txt', 'utf8', (error, data) => {
  if (error) {
    console.error('Ошибка чтения:', error);
    return;
  }
  // Работа с data
});

// 2. Promise style
fetch('/api/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Ошибка:', error));

// 3. Async/Await style
async function getData() {
  try {
    const response = await fetch('/api/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Ошибка:', error);
  }
}
```

11.5.1. Типы ошибок в асинхронном коде

Классификация асинхронных ошибок

javascript

// 1. Сетевые ошибки

```
async function networkErrorExample() {
  try {
    // DNS ошибка, таймаут, CORS, сеть недоступна
    await fetch('https://nonexistent-domain.com');
  } catch (error) {
    console.error('Сетевая ошибка:', error.name, error.message);
    // TypeError: Failed to fetch
    // DOMException: NetworkError
  }
}
```

// 2. Ошибки HTTP статусов

```
async function httpErrorExample() {
  try {
    const response = await fetch('/api/not-found');
    if (!response.ok) {
      // 404, 500, 403 и т.д.
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
  } catch (error) {
    console.error('HTTP ошибка:', error.message);
  }
}
```

// 3. Ошибки парсинга данных

```
async function parsingErrorExample() {
  try {
    const response = await fetch('/api/data');
    const data = await response.json(); // Может выбросить SyntaxError
    // Или ошибки валидации данных
    if (!data || !data.id) {
      throw new Error('Невалидные данные от сервера');
    }
  } catch (error) {
    if (error instanceof SyntaxError) {
      console.error('Ошибка парсинга JSON:', error);
    } else {

```

```
        console.error('Другая ошибка:', error);
    }
}
}
```

// 4. Таймауты

```
async function timeoutErrorExample() {
    try {
        const controller = new AbortController();
        const timeoutId = setTimeout(() => controller.abort(), 5000);

        const response = await fetch('/api/slow', {
            signal: controller.signal
        });

        clearTimeout(timeoutId);
    } catch (error) {
        if (error.name === 'AbortError') {
            console.error('Таймаут запроса');
        } else {
            console.error('Другая ошибка:', error);
        }
    }
}
```

// 5. Бизнес-логические ошибки

```
async function businessLogicError() {
    try {
        const user = await getUser(123);

        if (!user.isActive) {
            // Кастомная ошибка бизнес-логики
            throw new UserInactiveError('Пользователь неактивен', { userId: 123 });
        }

        if (user.balance < 100) {
            throw new InsufficientFundsError('Недостаточно средств', {
                userId: 123,
                balance: user.balance,
                required: 100
            });
        }
    } catch (error) {
        if (error instanceof BusinessError) {
```

```

    // Специфичная обработка бизнес-ошибок
    await logBusinessError(error);
    await notifyUser(error);
  } else {
    // Технические ошибки
    console.error('Техническая ошибка:', error);
  }
}
}

```

11.5.2. Обработка ошибок в Promise

Базовые паттерны обработки ошибок в Promise

javascript

```

// 1. .catch() в конце цепочки
fetch('/api/data')
  .then(response => response.json())
  .then(data => processData(data))
  .then(result => saveResult(result))
  .catch(error => {
    // Поймает ЛЮБУЮ ошибку в цепочке
    console.error('Ошибка в цепочке:', error);
    return getDefaultData(); // Можно вернуть значение по умолчанию
  })
  .then(finalResult => {
    // finalResult будет либо результатом цепочки,
    // либо значением из .catch()
    console.log('Финальный результат:', finalResult);
  });

// 2. Локальная обработка с продолжением
fetch('/api/data')
  .then(response => {
    if (!response.ok) {
      // Локальная обработка HTTP ошибки
      throw new Error(`HTTP ${response.status}`);
    }
    return response.json();
  })
  .catch(error => {
    // Обработка ошибок парсинга или предыдущих ошибок
  })

```



```

    console.warn('Не удалось получить данные:', error);
    return { data: [], fromCache: true }; // Продолжаем с fallback
  })
  .then(data => {
    // data будет либо реальные данные, либо fallback
    console.log('Работаем с данными:', data);
  })
  .catch(error => {
    // Этот catch сработает только если ошибка в последнем then
    console.error('Критическая ошибка:', error);
  });

// 3. Обработка в каждом then
fetch('/api/data')
  .then(
    response => response.json(), // onFulfilled
    error => { // onRejected (второй аргумент then)
      console.error('Ошибка fetch:', error);
      return Promise.reject(error); // Пробрасываем дальше
    }
  )
  .then(
    data => processData(data),
    error => {
      console.error('Ошибка парсинга:', error);
      return getCacheData();
    }
  );

// 4. finally() для очистки
let isLoading = true;

fetch('/api/data')
  .then(response => response.json())
  .then(data => console.log('Данные:', data))
  .catch(error => console.error('Ошибка:', error))
  .finally(() => {
    isLoading = false; // Выполнится в любом случае
    cleanupResources();
  });

```

Создание кастомных Promise с улучшенной обработкой ошибок

javascript

```

class EnhancedPromise extends Promise {
  constructor(executor) {
    super((resolve, reject) => {
      // Добавляем логирование ошибок
      const enhancedReject = (error) => {
        console.error('[EnhancedPromise] Отклонено:', error);
        console.error('[EnhancedPromise] Стек:', error.stack);
        reject(error);
      };

      try {
        executor(resolve, enhancedReject);
      } catch (error) {
        enhancedReject(error);
      }
    });
  }

  // Статические методы с улучшенной обработкой ошибок
  static asyncAll(promises) {
    return Promise.allSettled(promises)
      .then(results => {
        const errors = results
          .filter(r => r.status === 'rejected')
          .map(r => r.reason);

        if (errors.length > 0) {
          console.warn('[EnhancedPromise.all] Некоторые промисы отклонены:', errors);
        }

        return results
          .filter(r => r.status === 'fulfilled')
          .map(r => r.value);
      });
  }

  static withTimeout(promise, ms, timeoutMessage = 'Таймаут') {
    let timeoutId;

    const timeoutPromise = new Promise((_, reject) => {
      timeoutId = setTimeout(() => {
        reject(new TimeoutError(timeoutMessage, ms));
      }, ms);
    });
  }
}

```

```

        return Promise.race([promise, timeoutPromise])
            .finally(() => clearTimeout(timeoutId));
    }
}

// Использование
const promise = new EnhancedPromise((resolve, reject) => {
    // Код с автоматическим логированием ошибок
    setTimeout(() => reject(new Error('Тестовая ошибка')), 1000);
});

EnhancedPromise.asAll([promise, Promise.resolve('success')])
    .then(results => console.log('Результаты:', results));

```

11.5.3. Обработка ошибок с async/await

Базовые паттерны с try/catch

```

javascript

// 1. Базовый try/catch
async function fetchData() {
    try {
        const response = await fetch('/api/data');
        const data = await response.json();
        return data;
    } catch (error) {
        console.error('Ошибка загрузки данных:', error);

        // Можно вернуть значение по умолчанию
        return { data: [], error: true };

        // Или пробросить ошибку дальше
        // throw new CustomError('Не удалось загрузить данные', { originalError: error });
    }
}

// 2. Вложенные try/catch для гранулярной обработки
async function complexOperation() {
    let resource;

    try {

```

```

    resource = await acquireResource();
  } catch (error) {
    console.error('Не удалось получить ресурс:', error);
    throw error; // Не можем продолжать без ресурса
  }

  try {
    const data = await processWithResource(resource);
    return data;
  } catch (error) {
    console.error('Ошибка обработки:', error);
    return getFallbackData();
  } finally {
    // Всегда освобождаем ресурс
    await releaseResource(resource).catch(err => {
      console.error('Ошибка освобождения ресурса:', err);
    });
  }
}

```

// 3. Обработка нескольких независимых операций

```

async function multipleIndependentOperations() {
  const results = {};
  const errors = {};

  // Операция 1
  try {
    results.user = await fetchUser(123);
  } catch (error) {
    errors.user = error;
    results.user = getDefaultUser();
  }

  // Операция 2 (не зависит от операции 1)
  try {
    results.settings = await fetchSettings();
  } catch (error) {
    errors.settings = error;
    results.settings = getDefaultSettings();
  }

  // Операция 3 (зависит от операции 1)
  if (results.user) {
    try {

```

```

    results.posts = await fetchUserPosts(results.user.id);
  } catch (error) {
    errors.posts = error;
    results.posts = [];
  }
}

return { results, errors };
}

```

Продвинутые паттерны обработки ошибок

javascript

```

// 1. Декоратор для повторных попыток (retry)
function withRetry(operation, maxRetries = 3, delay = 1000) {
  return async function(...args) {
    let lastError;

    for (let attempt = 1; attempt <= maxRetries; attempt++) {
      try {
        console.log(`Попытка ${attempt} из ${maxRetries}`);
        return await operation(...args);
      } catch (error) {
        lastError = error;

        if (attempt === maxRetries) {
          break;
        }

        // Экспоненциальная задержка
        const waitTime = delay * Math.pow(2, attempt - 1);
        console.log(`Ждем ${waitTime}мс перед следующей попыткой`);
        await new Promise(resolve => setTimeout(resolve, waitTime));
      }
    }

    throw new RetryError(
      `Операция не удалась после ${maxRetries} попыток`,
      { originalError: lastError, retries: maxRetries }
    );
  };
}

// 2. Цепочка fallback операций

```

```

async function executeWithFallbacks(operations) {
  let lastError;

  for (let i = 0; i < operations.length; i++) {
    try {
      const result = await operations[i]();
      console.log(`Операция ${i} успешна`);
      return result;
    } catch (error) {
      console.log(`Операция ${i} не удалась:`, error.message);
      lastError = error;

      if (i === operations.length - 1) {
        throw new FallbackChainError(
          'Все fallback операции не удались',
          { originalError: lastError, attemptedOperations: operations.length }
        );
      }
    }
  }
}

```

// Использование

```

const data = await executeWithFallbacks([
  () => fetchFromPrimaryAPI(),
  () => fetchFromSecondaryAPI(),
  () => fetchFromCache(),
  () => getHardcodedData()
]);

```

// 3. Circuit Breaker (автоматический выключатель)

```

class CircuitBreaker {
  constructor(operation, options = {}) {
    this.operation = operation;
    this.failureThreshold = options.failureThreshold || 5;
    this.resetTimeout = options.resetTimeout || 60000;
    this.state = 'CLOSED';
    this.failureCount = 0;
    this.nextAttempt = Date.now();
  }

  async call(...args) {
    if (this.state === 'OPEN') {
      if (Date.now() > this.nextAttempt) {

```

```

        this.state = 'HALF_OPEN';
    } else {
        throw new CircuitBreakerError('Сервис временно недоступен');
    }
}

try {
    const result = await this.operation(...args);
    this.onSuccess();
    return result;
} catch (error) {
    this.onFailure();
    throw error;
}

onSuccess() {
    this.failureCount = 0;
    if (this.state === 'HALF_OPEN') {
        this.state = 'CLOSED';
    }
}

onFailure() {
    this.failureCount++;

    if (this.failureCount >= this.failureThreshold) {
        this.state = 'OPEN';
        this.nextAttempt = Date.now() + this.resetTimeout;
        console.error('Circuit Breaker: переход в состояние OPEN');
    }
}

// Использование
const breaker = new CircuitBreaker(fetchFromExternalService, {
    failureThreshold: 3,
    resetTimeout: 30000
});

try {
    const result = await breaker.call('/api/data');
    console.log('Результат:', result);
} catch (error) {

```

```

if (error instanceof CircuitBreakerError) {
    console.log('Используем закешированные данные');
    return getCacheData();
}
throw error;
}

```

11.5.4. Кастомные классы ошибок для асинхронного кода

Создание иерархии ошибок

```

javascript

// Базовый класс для всех асинхронных ошибок
class AsyncError extends Error {
    constructor(message, context = {}) {
        super(message);
        this.name = this.constructor.name;
        this.context = context;
        this.timestamp = new Date().toISOString();

        // Сохраняем оригинальный стек
        Error.captureStackTrace?.(this, this.constructor);
    }

    toJSON() {
        return {
            name: this.name,
            message: this.message,
            context: this.context,
            timestamp: this.timestamp,
            stack: this.stack
        };
    }
}

// Сетевые ошибки
class NetworkError extends AsyncError {
    constructor(message, { url, method = 'GET', status, headers = {} } = {}) {
        super(message, { url, method, status, headers });
    }
}

```



```

class TimeoutError extends AsyncError {
  constructor(message, timeoutMs) {
    super(message, { timeoutMs });
  }
}

// HTTP ошибки
class HttpError extends NetworkError {
  constructor(status, statusText, { url, method = 'GET', responseBody } = {}) {
    super(`HTTP ${status}: ${statusText}`, {
      url, method, status, statusText, responseBody
    });
    this.status = status;
    this.statusText = statusText;
  }
}

// Ошибки бизнес-логики
class BusinessError extends AsyncError {
  constructor(message, { code, details, userId } = {}) {
    super(message, { code, details, userId });
    this.code = code;
  }
}

class ValidationError extends BusinessError {
  constructor(message, { field, value, rules } = {}) {
    super(message, { field, value, rules, code: 'VALIDATION_ERROR' });
  }
}

class AuthorizationError extends BusinessError {
  constructor(message, { resource, action, userId } = {}) {
    super(message, { resource, action, userId, code: 'AUTH_ERROR' });
  }
}

// Ошибки инфраструктуры
class DatabaseError extends AsyncError {
  constructor(message, { query, params, dbName } = {}) {
    super(message, { query, params, dbName, code: 'DB_ERROR' });
  }
}

```

```

class ExternalServiceError extends AsyncError {
  constructor(message, { serviceName, endpoint, requestId } = {}) {
    super(message, { serviceName, endpoint, requestId, code: 'EXTERNAL_SERVICE_ERROR' });
  }
}

```

// Утилита для создания ошибок

```

function createError(type, message, context) {
  const errorMap = {
    network: NetworkError,
    timeout: TimeoutError,
    http: (status, statusText, ctx) => new HttpError(status, statusText, ctx),
    validation: ValidationError,
    auth: AuthorizationError,
    db: DatabaseError,
    external: ExternalServiceError
  };

  const ErrorClass = errorMap[type];

  if (!ErrorClass) {
    return new AsyncError(message, context);
  }

  if (type === 'http') {
    return ErrorClass(message, context.statusText, context);
  }

  return new ErrorClass(message, context);
}

```

// Использование

```

async function fetchUserData(userId) {
  try {
    const response = await fetch(`/api/users/${userId}`);

    if (response.status === 404) {
      throw createError('http', 404, 'Not Found', {
        url: `/api/users/${userId}`,
        responseBody: await response.text()
      });
    }

    if (!response.ok) {

```

```

        throw createError('http', response.status, response.statusText, {
            url: `/api/users/${userId}`
        });
    }

    return await response.json();

} catch (error) {
    if (error instanceof NetworkError) {
        // Логирование и обработка сетевых ошибок
        await logNetworkError(error);
        throw createError('network', 'Сетевая ошибка', {
            originalError: error,
            userId
        });
    }

    throw error;
}
}

```

11.5.5. Глобальная обработка ошибок

Глобальные обработчики для Promise

javascript

```

// 1. Обработка необработанных отклонений Promise
process.on('unhandledRejection', (reason, promise) => {
    console.error('[unhandledRejection] Необработанное отклонение промиса:');
    console.error('Причина:', reason);
    console.error('Промис:', promise);

    // Можно отправить в систему мониторинга
    sendToMonitoring({
        type: 'unhandledRejection',
        reason: reason instanceof Error ? {
            name: reason.name,
            message: reason.message,
            stack: reason.stack
        } : reason,
        timestamp: new Date().toISOString()
    });
});

```

```

// В продакшене: выходим из процесса
if (process.env.NODE_ENV === 'production') {
  console.error('Завершаем процесс из-за необработанного отклонения');
  process.exit(1);
}
});

// 2. Обработка множественных отклонений
process.on('multipleResolutions', (type, promise, reasonOrValue) => {
  console.error(`[multipleResolutions] Промис был ${type} несколько раз`);
  console.error('Промис:', promise);
  console.error('Значение/Причина:', reasonOrValue);
});

// 3. Обработчик для браузера
if (typeof window !== 'undefined') {
  window.addEventListener('unhandledrejection', (event) => {
    console.error(`[unhandledrejection] Необработанное отклонение промиса в браузере:`);
    console.error('Причина:', event.reason);
    console.error('Промис:', event.promise);

    // Предотвращаем вывод в консоль по умолчанию
    event.preventDefault();

    // Отправляем в аналитику
    sendToAnalytics({
      event: 'unhandled_promise_rejection',
      error: event.reason?.message,
      stack: event.reason?.stack,
      url: window.location.href
    });
  });

  window.addEventListener('rejectionhandled', (event) => {
    console.log(`[rejectionhandled] Отклонение промиса было обработано позже:`);
    console.log('Причина:', event.reason);
    console.log('Промис:', event.promise);
  });
}

```

Мидлвары и перехватчики

javascript

// 1. Декоратор для логирования ошибок

```
function withErrorLogging(operation, operationName) {
  return async function(...args) {
    const startTime = Date.now();

    try {
      const result = await operation(...args);
      const duration = Date.now() - startTime;

      console.log(`[${operationName}] Успешно за ${duration}мс`);
      return result;
    } catch (error) {
      const duration = Date.now() - startTime;

      console.error(`[${operationName}] Ошибка за ${duration}мс`, {
        name: error.name,
        message: error.message,
        stack: error.stack,
        args: args.slice(0, 3) // Логим только первые 3 аргумента
      });

      // Отправляем в систему мониторинга
      await sendToErrorMonitoring({
        operation: operationName,
        error: error.toJSON?(),
        duration,
        timestamp: new Date().toISOString()
      });

      throw error;
    }
  };
}
```

// 2. Перехватчик fetch

```
const originalFetch = window.fetch;

window.fetch = async function(...args) {
  const [url, options = {}] = args;
  const startTime = Date.now();
  const requestId = generateRequestId();
```

// Логим запрос

```

console.log(`[Fetch] Запрос ${requestId}:`, {
  url,
  method: options.method || 'GET',
  timestamp: new Date().toISOString()
});

try {
  const response = await originalFetch.apply(this, args);
  const duration = Date.now() - startTime;

  // Клонировем response для логирования тела
  const clonedResponse = response.clone();

  console.log(`[Fetch] Ответ ${requestId}:`, {
    status: response.status,
    statusText: response.statusText,
    duration: `${duration}мс`,
    ok: response.ok
  });

  // Логируем ошибки HTTP
  if (!response.ok) {
    const errorBody = await clonedResponse.text().catch(() => '');

    throw new HttpError(
      response.status,
      response.statusText,
      { url, method: options.method, responseBody: errorBody }
    );
  }

  return response;
} catch (error) {
  const duration = Date.now() - startTime;

  console.error(`[Fetch] Ошибка ${requestId} за ${duration}мс:`, {
    url,
    error: error.name,
    message: error.message
  });

  // Обогащаем ошибку дополнительной информацией
  if (!(error instanceof HttpError)) {

```

```

        error.requestInfo = { url, method: options.method, duration };
    }

    throw error;
}
};

```

// 3. Глобальный обработчик для асинх функций

```

class AsyncErrorHandler {
    static async executeWithGlobalHandler(asyncFn, ...args) {
        try {
            return await asyncFn(...args);
        } catch (error) {
            // Глобальная обработка
            await this.handleGlobalError(error, {
                functionName: asyncFn.name,
                args: this.sanitizeArgs(args)
            });

            // Возвращаем значение по умолчанию или пробрасываем дальше
            if (this.shouldSwallowError(error)) {
                return this.getDefaultValueForError(error);
            }

            throw error;
        }
    }
}

```

```

static async handleGlobalError(error, context = {}) {

```

// 1. Логирование

```

    console.error('[Global Error Handler]:', {
        error: {
            name: error.name,
            message: error.message,
            stack: error.stack
        },
        context,
        timestamp: new Date().toISOString()
    });

```

// 2. Отправка в мониторинг

```

    await this.sendToMonitoring(error, context);

```

// 3. Уведомление пользователя (если применимо)

```

    if (this.shouldNotifyUser(error)) {
        await this.showUserNotification(error);
    }

    // 4. Восстановление состояния
    await this.recoverFromError(error, context);
}

static sanitizeArgs(args) {
    // Удаляем чувствительные данные (пароли, токены и т.д.)
    return args.map(arg => {
        if (typeof arg === 'object' && arg !== null) {
            const sanitized = { ...arg };
            ['password', 'token', 'secret', 'apiKey'].forEach(key => {
                if (sanitized[key]) {
                    sanitized[key] = '***REDACTED***';
                }
            });
            return sanitized;
        }
        return arg;
    });
}

// Использование
async function protectedOperation() {
    return AsyncErrorHandler.executeWithGlobalHandler(
        async (userId, password) => {
            // Этот код защищен глобальным обработчиком ошибок
            const user = await authenticate(userId, password);
            const data = await fetchUserData(user.id);
            return processData(data);
        },
        userId,
        password
    );
}

```


11.5.6. Стратегии восстановления после ошибок

Различные стратегии восстановления

javascript

// 1. Retry (повторные попытки)

```
async function executeWithRetry(operation, options = {}) {
  const {
    maxRetries = 3,
    initialDelay = 1000,
    maxDelay = 10000,
    backoffFactor = 2,
    retryableErrors = [NetworkError, TimeoutError]
  } = options;

  let lastError;

  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error;

      // Проверяем, является ли ошибка повторяемой
      const isRetryable = retryableErrors.some(
        ErrorClass => error instanceof ErrorClass
      );

      if (!isRetryable || attempt === maxRetries) {
        break;
      }

      // Экспоненциальная задержка с джиттером
      const delay = Math.min(
        initialDelay * Math.pow(backoffFactor, attempt - 1),
        maxDelay
      );

      const jitter = delay * 0.1 * (Math.random() * 2 - 1); // ±10%
      const finalDelay = delay + jitter;

      console.log(`Попытка ${attempt} не удалась, ждем ${Math.round(finalDelay)}мс`);
      await new Promise(resolve => setTimeout(resolve, finalDelay));
    }
  }

  if (lastError) {
    throw lastError;
  }
}
```

```

    }
}

throw new RetryError(
  `Операция не удалась после ${maxRetries} попыток`,
  { originalError: lastError, retries: maxRetries }
);
}

// 2. Fallback (резервный вариант)
async function executeWithFallback(primaryOperation, fallbackOperations = []) {
  const errors = [];

  // Пробуем основную операцию
  try {
    return await primaryOperation();
  } catch (error) {
    errors.push({ operation: 'primary', error });
    console.log('Основная операция не удалась:', error.message);
  }

  // Пробуем fallback операции
  for (let i = 0; i < fallbackOperations.length; i++) {
    try {
      console.log(`Пробуем fallback операцию ${i + 1}`);
      return await fallbackOperations[i]();
    } catch (error) {
      errors.push({ operation: `fallback_${i + 1}`, error });
      console.log(`Fallback операция ${i + 1} не удалась:`, error.message);
    }
  }

  throw new FallbackError('Все операции не удалась', { errors });
}

// 3. Circuit Breaker (автоматический выключатель)
class AdvancedCircuitBreaker {
  constructor(operation, options = {}) {
    this.operation = operation;
    this.state = 'CLOSED';
    this.failureCount = 0;
    this.successCount = 0;
    this.nextAttempt = 0;
  }
}

```

```

// Конфигурация
this.failureThreshold = options.failureThreshold || 5;
this.successThreshold = options.successThreshold || 2;
this.openTimeout = options.openTimeout || 60000;
this.halfOpenTimeout = options.halfOpenTimeout || 10000;
this.metricsWindow = options.metricsWindow || 60000; // 1 минута
this.metrics = [];
}

async call(...args) {
  this.recordMetric('request');

  // Проверяем состояние
  if (this.state === 'OPEN') {
    if (Date.now() > this.nextAttempt) {
      this.state = 'HALF_OPEN';
      this.nextAttempt = Date.now() + this.halfOpenTimeout;
    } else {
      this.recordMetric('rejected');
      throw new CircuitBreakerError('Сервис временно недоступен', {
        state: this.state,
        nextAttempt: new Date(this.nextAttempt).toISOString()
      });
    }
  }

  try {
    const result = await this.operation(...args);
    this.onSuccess();
    return result;
  } catch (error) {
    this.onFailure(error);
    throw error;
  }
}

onSuccess() {
  this.recordMetric('success');
  this.successCount++;
  this.failureCount = 0;

  if (this.state === 'HALF_OPEN' && this.successCount >= this.successThreshold) {
    console.log('Circuit Breaker: переход из HALF_OPEN в CLOSED');
    this.state = 'CLOSED';
  }
}

```

```

        this.successCount = 0;
    }
}

onFailure(error) {
    this.recordMetric('failure');
    this.failureCount++;
    this.successCount = 0;

    if (this.failureCount >= this.failureThreshold) {
        console.log('Circuit Breaker: переход в состояние OPEN');
        this.state = 'OPEN';
        this.nextAttempt = Date.now() + this.openTimeout;

        // Отправляем уведомление
        this.notifyStateChange('OPEN', error);
    }
}

recordMetric(type) {
    this.metrics.push({ type, timestamp: Date.now() });

    // Удаляем старые метрики
    const cutoff = Date.now() - this.metricsWindow;
    this.metrics = this.metrics.filter(m => m.timestamp > cutoff);
}

getMetrics() {
    const windowStart = Date.now() - this.metricsWindow;
    const windowMetrics = this.metrics.filter(m => m.timestamp > windowStart);

    return {
        total: windowMetrics.length,
        success: windowMetrics.filter(m => m.type === 'success').length,
        failure: windowMetrics.filter(m => m.type === 'failure').length,
        rejected: windowMetrics.filter(m => m.type === 'rejected').length,
        failureRate: windowMetrics.length > 0
            ? windowMetrics.filter(m => m.type === 'failure').length / windowMetrics.length
            : 0
    };
}
}

```

// 4. Bulkhead (изоляция)

```

class Bulkhead {
  constructor(maxConcurrent, maxQueueSize = 100) {
    this.maxConcurrent = maxConcurrent;
    this.maxQueueSize = maxQueueSize;
    this.active = 0;
    this.queue = [];
    this.rejected = 0;
  }

  async execute(operation) {
    if (this.active >= this.maxConcurrent) {
      if (this.queue.length >= this.maxQueueSize) {
        this.rejected++;
        throw new BulkheadError('Достигнут лимит очереди', {
          maxConcurrent: this.maxConcurrent,
          maxQueueSize: this.maxQueueSize,
          active: this.active,
          queueLength: this.queue.length
        });
      }

      // Помещаем в очередь
      return new Promise((resolve, reject) => {
        this.queue.push({ operation, resolve, reject });
      });
    }

    this.active++;

    try {
      const result = await operation();
      return result;
    } finally {
      this.active--;
      this.processQueue();
    }
  }

  processQueue() {
    if (this.queue.length > 0 && this.active < this.maxConcurrent) {
      const { operation, resolve, reject } = this.queue.shift();
      this.execute(operation).then(resolve).catch(reject);
    }
  }
}

```

```

getStats() {
  return {
    active: this.active,
    queueLength: this.queue.length,
    rejected: this.rejected,
    maxConcurrent: this.maxConcurrent,
    maxQueueSize: this.maxQueueSize
  };
}
}

// Использование Bulkhead
const databaseBulkhead = new Bulkhead(5, 50); // Макс 5 одновременных, очередь 50

async function queryDatabase(query) {
  return databaseBulkhead.execute(() => {
    return db.query(query); // Ограниченное количество одновременных запросов
  });
}

```

11.5.7. Мониторинг и логирование ошибок

Интеграция с системами мониторинга

```

javascript

class ErrorMonitor {
  constructor(config = {}) {
    this.config = {
      enabled: config.enabled ?? true,
      samplingRate: config.samplingRate ?? 1.0, // 100% по умолчанию
      batchSize: config.maxBatchSize ?? 50,
      flushInterval: config.flushInterval ?? 10000, // 10 секунд
      ...config
    };
  }

  this.queue = [];
  this.flushTimeout = null;
  this.initialized = false;
}

async init() {

```

```

    if (this.initialized) return;

    // Устанавливаем глобальные обработчики
    if (typeof process !== 'undefined') {
        process.on('unhandledRejection', this.handleUnhandledRejection.bind(this));
        process.on('uncaughtException', this.handleUncaughtException.bind(this));
    }

    if (typeof window !== 'undefined') {
        window.addEventListener('unhandledrejection',
this.handleUnhandledRejection.bind(this));
        window.addEventListener('error', this.handleWindowError.bind(this));
    }

    this.initialized = true;
    console.log('ErrorMonitor initialized');
}

async captureError(error, context = {}) {
    if (!this.config.enabled) return;

    // Сэмплирование
    if (Math.random() > this.config.samplingRate) return;

    const errorEvent = {
        // Основная информация
        type: 'error',
        timestamp: new Date().toISOString(),
        environment: process.env.NODE_ENV || 'development',

        // Информация об ошибке
        error: {
            name: error.name,
            message: error.message,
            stack: error.stack,
            ...(error.toJSON?.() || {})
        },

        // Контекст
        context: {
            url: typeof window !== 'undefined' ? window.location.href : undefined,
            userAgent: typeof navigator !== 'undefined' ? navigator.userAgent : undefined,
            ...this.sanitizeContext(context)
        },
    },

```

```

    // Системная информация
    system: {
        memory: process.memoryUsage?(),
        uptime: process.uptime?(),
        platform: process.platform,
        version: process.version
    }
};

// Добавляем в очередь
this.queue.push(errorEvent);

// Проверяем необходимость отправки
if (this.queue.length >= this.config.maxBatchSize) {
    await this.flush();
} else if (!this.flushTimeout) {
    this.flushTimeout = setTimeout(() => this.flush(), this.config.flushInterval);
}
}

async flush() {
    if (this.flushTimeout) {
        clearTimeout(this.flushTimeout);
        this.flushTimeout = null;
    }

    if (this.queue.length === 0) return;

    const batch = [...this.queue];
    this.queue = [];

    try {
        await this.sendToBackend(batch);
        console.log(`Отправлено ${batch.length} ошибок в мониторинг`);
    } catch (error) {
        console.error('Ошибка отправки в мониторинг:', error);
        // Возвращаем ошибки в очередь для повторной отправки
        this.queue.unshift(...batch.slice(0, this.config.maxBatchSize));
    }
}

async sendToBackend(batch) {
    // Интеграция с различными системами мониторинга

```



```

const backends = [
  this.sendToSentry.bind(this),
  this.sendToDataDog.bind(this),
  this.sendToNewRelic.bind(this),
  this.sendToCustomBackend.bind(this)
];

const results = await Promise.allSettled(
  backends.map(backend => backend(batch))
);

// Логируем результаты
results.forEach((result, index) => {
  if (result.status === 'rejected') {
    console.warn(`Бэкенд ${index} не удалось отправить ошибки`, result.reason);
  }
});
}

sanitizeContext(context) {
  const sensitiveKeys = [
    'password', 'token', 'secret', 'apiKey', 'authorization',
    'creditCard', 'ssn', 'phone', 'email'
  ];

  const sanitized = { ...context };

  function sanitizeObject(obj) {
    if (!obj || typeof obj !== 'object') return obj;

    const result = Array.isArray(obj) ? [] : {};

    for (const [key, value] of Object.entries(obj)) {
      if (sensitiveKeys.some(sensitive =>
        key.toLowerCase().includes(sensitive.toLowerCase())
      )) {
        result[key] = '***REDACTED***';
      } else if (value && typeof value === 'object') {
        result[key] = sanitizeObject(value);
      } else {
        result[key] = value;
      }
    }
  }
}

```

```

        return result;
    }

    return sanitizeObject(sanitized);
}

// Обработчики глобальных ошибок
handleUnhandledRejection(event) {
    const error = event.reason || event;
    const promise = event.promise;

    this.captureError(error, {
        errorType: 'unhandledRejection',
        promise: promise?.toString()
    }).catch(console.error);
}

handleUncaughtException(error) {
    this.captureError(error, {
        errorType: 'uncaughtException'
    }).catch(console.error);
}

handleWindowError(event) {
    this.captureError(event.error || new Error(event.message), {
        errorType: 'windowError',
        filename: event.filename,
        lineno: event.lineno,
        colno: event.colno
    }).catch(console.error);
}
}

// Инициализация и использование
const errorMonitor = new ErrorMonitor({
    enabled: process.env.NODE_ENV === 'production',
    samplingRate: 0.1, // 10% в продакшене
    batchSize: 100,
    flushInterval: 30000 // 30 секунд
});

// Инициализируем при запуске приложения
errorMonitor.init().catch(console.error);

```

```

// Использование в коде
async function monitoredOperation() {
  try {
    // ... код операции
  } catch (error) {
    // Отправляем ошибку в мониторинг
    await errorMonitor.captureError(error, {
      operation: 'monitoredOperation',
      userId: 123,
      additionalData: { /* нечувствительные данные */ }
    });

    throw error;
  }
}

```

11.5.8. Практические примеры и кейсы

Пример 1: Загрузка данных с несколькими стратегиями восстановления

```

javascript

class DataLoader {
  constructor(options = {}) {
    this.circuitBreaker = new AdvancedCircuitBreaker(
      this.fetchFromSource.bind(this),
      {
        failureThreshold: 3,
        successThreshold: 2,
        openTimeout: 30000
      }
    );

    this.cache = new Map();
    this.cacheTTL = options.cacheTTL || 300000; // 5 минут
  }

  async loadData(key, options = {}) {
    const { forceRefresh = false, timeout = 10000 } = options;

    // 1. Проверяем кэш
    if (!forceRefresh) {
      const cached = this.cache.get(key);

```

```

    if (cached && Date.now() - cached.timestamp < this.cacheTTL) {
      console.log(`[DataLoader] Возвращаем из кэша: ${key}`);
      return cached.data;
    }
  }

  // 2. Пытаемся загрузить с повторными попытками
  try {
    const data = await executeWithRetry(
      () => this.circuitBreaker.call(key),
      {
        maxRetries: 3,
        initialDelay: 1000,
        retryableErrors: [NetworkError, TimeoutError, CircuitBreakerError]
      }
    );

    // Сохраняем в кэш
    this.cache.set(key, {
      data,
      timestamp: Date.now()
    });

    return data;
  } catch (error) {
    console.error(`[DataLoader] Ошибка загрузки ${key}:`, error.message);

    // 3. Fallback: пробуем загрузить из backup источника
    try {
      const backupData = await this.fetchFromBackup(key);

      // Сохраняем в кэш (но с пометкой, что из backup)
      this.cache.set(key, {
        data: backupData,
        timestamp: Date.now(),
        fromBackup: true
      });

      return backupData;
    } catch (backupError) {
      console.error(`[DataLoader] Ошибка backup для ${key}:`, backupError.message);
    }
  }
}

```

```

    // 4. Последний fallback: возвращаем закешированные данные, даже если устарели
    const staleCached = this.cache.get(key);
    if (staleCached) {
        console.warn(`[DataLoader] Возвращаем устаревшие данные для ${key}`);
        return staleCached.data;
    }

    // 5. Если ничего не помогло - возвращаем значение по умолчанию
    return this.getDefaultData(key);
}

}

async fetchFromSource(key) {
    // Имитация загрузки из основного источника
    await new Promise(resolve => setTimeout(resolve, 500));

    // 30% вероятность ошибки для демонстрации
    if (Math.random() < 0.3) {
        throw new NetworkError('Ошибка сети при загрузке данных');
    }

    return `Данные из источника для ${key}`;
}

async fetchFromBackup(key) {
    // Имитация загрузки из backup источника
    await new Promise(resolve => setTimeout(resolve, 1000));
    return `Данные из backup для ${key}`;
}

getDefaultData(key) {
    return `Данные по умолчанию для ${key}`;
}

}

// Использование
const loader = new DataLoader();

async function exampleUsage() {
    try {
        const data = await loader.loadData('user-profile-123', {
            forceRefresh: false,
            timeout: 5000
        });
    }
}

```

```

    });

    console.log('Загруженные данные:', data);
    return data;

  } catch (error) {
    console.error('Критическая ошибка загрузки данных:', error);
    throw error;
  }
}

```

Пример 2: Обработка платежей с компенсирующими транзакциями

javascript

```

class PaymentProcessor {
  async processPayment(orderId, paymentDetails) {
    const transaction = new PaymentTransaction(orderId, paymentDetails);

    try {
      // 1. Резервируем средства
      await transaction.reserveFunds();

      // 2. Проверяем fraud
      await transaction.checkFraud();

      // 3. Создаем платеж в платежной системе
      await transaction.createPayment();

      // 4. Обновляем заказ
      await transaction.updateOrder();

      // 5. Отправляем подтверждение
      await transaction.sendConfirmation();

      console.log(`Платеж для заказа ${orderId} успешно обработан`);
      return transaction.getResult();

    } catch (error) {
      console.error(`Ошибка обработки платежа для заказа ${orderId}:`, error);

      try {
        // Компенсирующие транзакции (rollback)
        await transaction.compensate();
        console.log(`Компенсирующие транзакции для заказа ${orderId} выполнены`);
      } catch (rollbackError) {
        console.error(`Ошибка при выполнении компенсирующих транзакций:`, rollbackError);
      }
    }
  }
}

```

```

    } catch (compensationError) {
        console.error(`Ошибка компенсации для заказа ${orderId}:`, compensationError);

        // Критическая ситуация - требуется ручное вмешательство
        await this.notifyManualIntervention(orderId, error, compensationError);
    }

    throw new PaymentError(
        `Не удалось обработать платеж для заказа ${orderId}`,
        {
            orderId,
            originalError: error,
            compensationError: compensationError
        }
    );
}
}

```

```

async notifyManualIntervention(orderId, paymentError, compensationError) {
    // Отправляем уведомление команде поддержки
    await sendAlert({
        type: 'MANUAL_INTERVENTION_REQUIRED',
        severity: 'CRITICAL',
        orderId,
        paymentError: paymentError.toJSON?(),
        compensationError: compensationError?.toJSON?(),
        timestamp: new Date().toISOString()
    });
}
}

```

```

class PaymentTransaction {
    constructor(orderId, paymentDetails) {
        this.orderId = orderId;
        this.paymentDetails = paymentDetails;
        this.steps = [];
    }
}

```

```

async reserveFunds() {
    this.recordStep('reserveFunds');

    // Имитация резервирования средств
    await this.simulateOperation(200);
}

```

```
    if (Math.random() < 0.1) {
        throw new FundsReservationError('Недостаточно средств');
    }
}

async checkFraud() {
    this.recordStep('checkFraud');

    // Имитация проверки fraud
    await this.simulateOperation(300);

    if (Math.random() < 0.05) {
        throw new FraudCheckError('Обнаружена подозрительная активность');
    }
}

async createPayment() {
    this.recordStep('createPayment');

    // Имитация создания платежа
    await this.simulateOperation(500);

    if (Math.random() < 0.15) {
        throw new PaymentGatewayError('Ошибка платежного шлюза');
    }
}

async updateOrder() {
    this.recordStep('updateOrder');

    // Имитация обновления заказа
    await this.simulateOperation(400);
}

async sendConfirmation() {
    this.recordStep('sendConfirmation');

    // Имитация отправки подтверждения
    await this.simulateOperation(300);
}

async compensate() {
    console.log(`Выполняем компенсирующие транзакции для ${this.orderId}`);
```



```

// Выполняем компенсирующие транзакции в обратном порядке
const stepsToCompensate = [...this.steps].reverse();

for (const step of stepsToCompensate) {
  try {
    await this.compensateStep(step);
  } catch (error) {
    console.error(`Ошибка компенсации шага ${step}:`, error);
    // Продолжаем компенсацию других шагов
  }
}

async compensateStep(step) {
  switch (step) {
    case 'reserveFunds':
      await this.releaseFunds();
      break;
    case 'createPayment':
      await this.cancelPayment();
      break;
    case 'updateOrder':
      await this.revertOrderUpdate();
      break;
    // Некоторые шаги не требуют компенсации
  }
}

recordStep(step) {
  this.steps.push(step);
  console.log(`[${this.orderId}] Выполнен шаг: ${step}`);
}

async simulateOperation(delay) {
  await new Promise(resolve => setTimeout(resolve, delay));
}

getResult() {
  return {
    orderId: this.orderId,
    status: 'COMPLETED',
    transactionId: `tx_${Date.now()}`,
    steps: this.steps
  };
}

```

```
}  
}
```

11.5.9. Упражнения для закрепления

Упражнение 1: Создание системы повторных попыток

javascript

```
// Задача: Реализовать систему повторных попыток с поддержкой:  
// 1. Экспоненциальной задержки с джиттером  
// 2. Ограничения максимального времени выполнения  
// 3. Предиката для определения, какие ошибки повторять  
// 4. Хуков для логирования и мониторинга
```

```
class RetrySystem {  
  constructor(options = {}) {  
    // Ваша реализация здесь  
  }  
  
  async execute(operation) {  
    // Ваша реализация здесь  
  }  
}
```

```
// Тестовые сценарии:  
// 1. Успех с первой попытки  
// 2. Успех после нескольких попыток  
// 3. Неудача после всех попыток  
// 4. Прерывание по таймауту
```

Упражнение 2: Реализация Circuit Breaker

javascript

```
// Задача: Улучшить Circuit Breaker, добавив:  
// 1. Метрики (количество запросов, успехов, ошибок)  
// 2. Адаптивное определение timeout'ов  
// 3. Разные стратегии для разных типов ошибок  
// 4. Интеграцию с системой мониторинга
```

```
class AdaptiveCircuitBreaker {  
  constructor(operation, options = {}) {  
    // Ваша реализация здесь
```

```

}

async call(...args) {
    // Ваша реализация здесь
}

getMetrics() {
    // Возвращает статистику работы
}
}

```

Упражнение 3: Создание системы fallback

```

javascript

// Задача: Создать систему fallback, которая:
// 1. Пробует несколько источников данных по приоритету
// 2. Кэширует успешные результаты
// 3. Обновляет кэш в фоновом режиме
// 4. Отслеживает состояние каждого источника

class FallbackSystem {
    constructor(sources, options = {}) {
        // sources - массив источников в порядке приоритета
        // Каждый источник: { name: string, fetch: function, healthCheck: function }
    }

    async getData(key) {
        // Пытается получить данные из источников по порядку
        // Кэширует успешные результаты
        // Обновляет кэш для неудачных источников в фоне
    }

    async checkHealth() {
        // Проверяет здоровье всех источников
        // Обновляет приоритеты на основе здоровья
    }
}

```

Заключение

Обработка ошибок в асинхронном коде — это комплексная задача, требующая многоуровневого подхода:

Ключевые принципы:

1. **Всегда обрабатывайте ошибки** — не оставляйте промисы необработанными
2. **Используйте подходящую стратегию** — `retry`, `fallback`, `circuit breaker`
3. **Создавайте информативные ошибки** — с контекстом и стектрейсом
4. **Логируйте и мониторьте** — собирайте метрики для анализа
5. **Планируйте восстановление** — что делать при ошибках

Уровни обработки ошибок:

1. **Локальный уровень** — `try/catch` для конкретных операций
2. **Стратегический уровень** — `retry`, `fallback`, `circuit breaker`
3. **Глобальный уровень** — обработчики `unhandled rejections`
4. **Операционный уровень** — мониторинг и алертинг

Помните:

- Асинхронные ошибки могут возникать в любое время
- Разные типы ошибок требуют разных стратегий обработки
- Хорошая обработка ошибок делает приложение отказоустойчивым
- Мониторинг помогает находить и исправлять проблемы до того, как они повлияют на пользователей

Правильная обработка ошибок — это не просто техническая необходимость, но и конкурентное преимущество, которое повышает надежность и доверие пользователей к вашему приложению.

11.6. Параллельное и последовательное выполнение асинхронных операций

Введение: стратегии выполнения асинхронных задач

В современном JavaScript разработчики часто сталкиваются с необходимостью выполнения нескольких асинхронных операций. Выбор между параллельным и последовательным выполнением — это компромисс между скоростью, использованием ресурсов и сложностью кода.

Сравнительная таблица:

Параметр	Последовательное выполнение	Параллельное выполнение	Ограниченное параллельное
Время выполнения	Сумма всех операций	Максимальное время операции	Оптимизированное время
Использование ресурсов	Низкое	Высокое	Контролируемое
Сложность обработки ошибок	Простая	Сложная	Средняя
Пропускная способность	Низкая	Высокая	Оптимальная
Подходит для	Зависимые операции	Независимые операции	Смешанные задачи

11.6.1. Последовательное выполнение асинхронных операций

Базовые паттерны последовательного выполнения

```
javascript
// 1. Цепочка промисов (Promise Chaining)
function sequentialPromiseChain() {
  console.time('sequentialPromiseChain');

  return operation1()
```

```

    .then(result1 => {
      console.log('Операция 1 завершена:', result1);
      return operation2(result1);
    })
    .then(result2 => {
      console.log('Операция 2 завершена:', result2);
      return operation3(result2);
    })
    .then(result3 => {
      console.log('Операция 3 завершена:', result3);
      console.timeEnd('sequentialPromiseChain');
      return result3;
    })
    .catch(error => {
      console.error('Ошибка в цепочке:', error);
      console.timeEnd('sequentialPromiseChain');
      throw error;
    });
  });
}

```

// 2. Async/Await с циклом for...of

```

async function sequentialAsyncAwait() {
  console.time('sequentialAsyncAwait');

  const operations = [
    { name: 'operation1', fn: operation1 },
    { name: 'operation2', fn: operation2 },
    { name: 'operation3', fn: operation3 }
  ];

  const results = [];

  for (const op of operations) {
    try {
      console.log(`Начинаем ${op.name}`);
      const result = await op.fn();
      results.push(result);
      console.log(`${op.name} завершена:`, result);
    } catch (error) {
      console.error(`Ошибка в ${op.name}:`, error);
      throw error;
    }
  }
}

```

```

    console.timeEnd('sequentialAsyncAwait');
    return results;
}

// 3. Рекурсивное последовательное выполнение
function sequentialRecursive(operations, index = 0, results = []) {
    if (index >= operations.length) {
        return Promise.resolve(results);
    }

    return operations[index]()
        .then(result => {
            results.push(result);
            return sequentialRecursive(operations, index + 1, results);
        });
}

// Использование
const operations = [operation1, operation2, operation3];
sequentialRecursive(operations)
    .then(results => console.log('Все операции завершены:', results))
    .catch(error => console.error('Ошибка:', error));

```

Продвинутые паттерны последовательного выполнения

javascript

```

// 1. Последовательное выполнение с передачей данных между операциями
class SequentialPipeline {
    constructor() {
        this.stages = [];
        this.context = {};
    }

    addStage(name, processor) {
        this.stages.push({ name, processor });
        return this; // Для чейнинга
    }

    async execute(initialData = {}) {
        console.time('SequentialPipeline');
        this.context = { ...initialData };
        const results = {};

        for (const stage of this.stages) {

```

```

try {
  console.log(`[${stage.name}] Начинаем выполнение`);
  const startTime = Date.now();

  // Выполняем этап с текущим контекстом
  const stageResult = await stage.processor(this.context);

  // Сохраняем результат этапа
  results[stage.name] = {
    result: stageResult,
    duration: Date.now() - startTime,
    timestamp: new Date().toISOString()
  };

  // Обновляем контекст для следующего этапа
  this.context[stage.name] = stageResult;
  this.context.lastStage = stage.name;

  console.log(`[${stage.name}] Завершено за ${results[stage.name].duration}мс`);
} catch (error) {
  console.error(`[${stage.name}] Ошибка:`, error);

  // Сохраняем информацию об ошибке
  results[stage.name] = {
    error: error.message,
    duration: Date.now() - startTime,
    timestamp: new Date().toISOString(),
    failed: true
  };

  // Решаем, продолжать ли выполнение
  if (stage.critical) {
    console.timeEnd('SequentialPipeline');
    throw new PipelineError(`Критический этап ${stage.name} не выполнен`, {
      stage: stage.name,
      results,
      originalError: error
    });
  }
}

console.timeEnd('SequentialPipeline');

```



```

    return { results, context: this.context };
  }
}

// Использование
const pipeline = new SequentialPipeline();

pipeline
  .addStage('validate', async (context) => {
    // Валидация входных данных
    if (!context.input) {
      throw new Error('Отсутствуют входные данные');
    }
    return { valid: true, data: context.input };
  })
  .addStage('transform', async (context) => {
    // Преобразование данных
    const transformed = context.validate.data.map(item => ({
      ...item,
      processed: true,
      timestamp: new Date().toISOString()
    }));
    return transformed;
  })
  .addStage('enrich', async (context) => {
    // Обогащение данных
    const enriched = await Promise.all(
      context.transform.map(async item => ({
        ...item,
        metadata: await fetchMetadata(item.id)
      }))
    );
    return enriched;
  })
  .addStage('save', async (context) => {
    // Сохранение данных
    const saved = await saveToDatabase(context.enrich);
    return { saved: true, count: saved.length };
  });

// Запуск пайплайна
pipeline.execute({ input: [{ id: 1 }, { id: 2 }, { id: 3 }] })
  .then(({ results, context }) => {
    console.log('Пайплайн выполнен успешно:');
  });

```

```

    console.log('Результаты:', results);
    console.log('Контекст:', context);
  })
  .catch(error => {
    console.error('Ошибка выполнения пайплайна:', error);
  });

```

// 2. Последовательное выполнение с ограничением времени на этап

```

async function executeWithTimeLimit(operations, timeLimitPerOperation = 5000) {
  const results = [];
  const errors = [];

  for (let i = 0; i < operations.length; i++) {
    const operation = operations[i];
    const operationName = operation.name || `operation_${i}`;

    console.log(`Выполняем операцию: ${operationName}`);

    try {
      // Создаем промис с таймаутом
      const timeoutPromise = new Promise((_, reject) => {
        setTimeout(() => {
          reject(new TimeoutError(
            `Операция ${operationName} превысила лимит времени ${timeLimitPerOperation}
мс`
          ));
        }, timeLimitPerOperation);
      });

      // Гонка между операцией и таймаутом
      const result = await Promise.race([
        operation(),
        timeoutPromise
      ]);

      results.push({
        operation: operationName,
        result,
        status: 'success',
        duration: Date.now() - startTime
      });

      console.log(`Операция ${operationName} завершена успешно`);
    } catch (error) {
      errors.push({
        operation: operationName,
        error
      });
    }
  }

  return { results, errors };
}

```

```

    } catch (error) {
        errors.push({
            operation: operationName,
            error: error.message,
            status: 'failed',
            duration: Date.now() - startTime
        });

        console.error(`Ошибка в операции ${operationName}:`, error.message);

        // Решаем, продолжать ли выполнение при ошибке
        if (operation.critical || error instanceof TimeoutError) {
            console.log('Прерываем выполнение из-за критической ошибки');
            break;
        }
    }
}

return { results, errors, completed: errors.length === 0 };
}

// 3. Последовательное выполнение с зависимостями
class DependencyGraph {
    constructor() {
        this.nodes = new Map(); // operationId -> { operation, dependencies, result }
        this.executionOrder = [];
    }

    addOperation(id, operation, dependencies = []) {
        this.nodes.set(id, {
            operation,
            dependencies,
            result: null,
            status: 'pending'
        });
        return this;
    }

    async execute() {
        // 1. Проверяем наличие циклических зависимостей
        if (this.hasCyclicDependencies()) {
            throw new Error('Обнаружены циклические зависимости');
        }
    }
}

```

```

// 2. Определяем порядок выполнения (топологическая сортировка)
this.executionOrder = this.topologicalSort();

console.log('Порядок выполнения:', this.executionOrder);

// 3. Выполняем операции в определенном порядке
for (const operationId of this.executionOrder) {
  const node = this.nodes.get(operationId);

  try {
    console.log(`Выполняем операцию: ${operationId}`);

    // Получаем результаты зависимостей
    const dependencyResults = {};
    for (const depId of node.dependencies) {
      const depNode = this.nodes.get(depId);
      if (depNode.status !== 'completed') {
        throw new DependencyError(
          `Зависимость ${depId} для операции ${operationId} не выполнена`
        );
      }
      dependencyResults[depId] = depNode.result;
    }

    // Выполняем операцию с результатами зависимостей
    node.result = await node.operation(dependencyResults);
    node.status = 'completed';

    console.log(`Операция ${operationId} завершена`);

  } catch (error) {
    node.status = 'failed';
    node.error = error;

    console.error(`Ошибка в операции ${operationId}:`, error);

    // Определяем, нужно ли прерывать выполнение
    if (error instanceof CriticalDependencyError) {
      throw new ExecutionError(
        `Выполнение прервано из-за критической ошибки в ${operationId}`,
        { operationId, error }
      );
    }
  }
}
}

```

```
}
```

```
// 4. Собираем результаты
```

```
const results = {};  
for (const [id, node] of this.nodes) {  
  results[id] = {  
    status: node.status,  
    result: node.result,  
    error: node.error  
  };  
}
```

```
return results;  
}
```

```
topologicalSort() {  
  const visited = new Set();  
  const stack = new Set();  
  const order = [];  
  
  const visit = (nodeId) => {  
    if (stack.has(nodeId)) {  
      throw new Error(`Циклическая зависимость обнаружена: ${nodeId}`);  
    }  
  
    if (visited.has(nodeId)) {  
      return;  
    }  
  
    stack.add(nodeId);  
    visited.add(nodeId);  
  
    const node = this.nodes.get(nodeId);  
    for (const depId of node.dependencies) {  
      visit(depId);  
    }  
  
    stack.delete(nodeId);  
    order.push(nodeId);  
  };  
  
  for (const nodeId of this.nodes.keys()) {  
    if (!visited.has(nodeId)) {  
      visit(nodeId);  
    }  
  }  
}
```

```

    }
  }

  return order;
}

hasCyclicDependencies() {
  try {
    this.topologicalSort();
    return false;
  } catch (error) {
    return true;
  }
}
}

// Использование графа зависимостей
const graph = new DependencyGraph();

graph
  .addOperation('fetchData', async () => {
    console.log('Загружаем данные...');
    await delay(1000);
    return { users: [{ id: 1 }, { id: 2 }] };
  })
  .addOperation('validate', async (deps) => {
    console.log('Валидируем данные...');
    const data = deps.fetchData;
    if (!data.users || !Array.isArray(data.users)) {
      throw new Error('Некорректные данные');
    }
    return { valid: true, count: data.users.length };
  }, ['fetchData'])
  .addOperation('process', async (deps) => {
    console.log('Обрабатываем данные...');
    const data = deps.fetchData;
    const validation = deps.validate;

    if (!validation.valid) {
      throw new CriticalDependencyError('Данные не прошли валидацию');
    }

    const processed = data.users.map(user => ({
      ...user,

```

```

        processed: true,
        timestamp: new Date().toISOString()
    }));

    return processed;
}, ['fetchData', 'validate'])
.addOperation('save', async (deps) => {
    console.log('Сохраняем данные...');
    const processed = deps.process;
    await saveToDatabase(processed);
    return { saved: true, count: processed.length };
}, ['process'])
.addOperation('notify', async (deps) => {
    console.log('Отправляем уведомление...');
    const saveResult = deps.save;
    if (saveResult.saved) {
        await sendNotification(`Сохранено ${saveResult.count} записей`);
    }
    return { notified: true };
}, ['save']);

// Запуск
graph.execute()
    .then(results => {
        console.log('Все операции выполнены:');
        console.log(results);
    })
    .catch(error => {
        console.error('Ошибка выполнения графа:', error);
    });

```

11.6.2. Параллельное выполнение асинхронных операций

Базовые паттерны параллельного выполнения

```

javascript

// 1. Promise.all() - классический подход
async function parallelPromiseAll() {
    console.time('parallelPromiseAll');

    try {
        // Запускаем все операции одновременно
    }

```

```

const [result1, result2, result3] = await Promise.all([
  operation1(),
  operation2(),
  operation3()
]);

console.log('Все операции завершены:');
console.log('- Результат 1:', result1);
console.log('- Результат 2:', result2);
console.log('- Результат 3:', result3);

console.timeEnd('parallelPromiseAll');
return [result1, result2, result3];

} catch (error) {
  console.timeEnd('parallelPromiseAll');
  console.error('Ошибка в параллельных операциях:', error);
  throw error;
}
}

// 2. Promise.all() с динамическим списком операций
async function parallelDynamic(operations) {
  console.time('parallelDynamic');

  // Создаем массив промисов
  const promises = operations.map((operation, index) => {
    const operationName = operation.name || `operation_${index}`;

    console.log(`Запускаем операцию: ${operationName}`);

    // Обертываем операцию для логирования
    return operation()
      .then(result => {
        console.log(`Операция ${operationName} завершена`);
        return { operation: operationName, result, status: 'success' };
      })
      .catch(error => {
        console.error(`Ошибка в операции ${operationName}:`, error.message);
        return { operation: operationName, error: error.message, status: 'failed' };
      });
  });

  // Ждем завершения всех операций

```



```

const results = await Promise.all(promises);

console.timeEnd('parallelDynamic');

// Анализируем результаты
const successful = results.filter(r => r.status === 'success');
const failed = results.filter(r => r.status === 'failed');

return {
  successful,
  failed,
  total: results.length,
  successRate: successful.length / results.length
};
}

// 3. Promise.allSettled() - выполнение с сохранением всех результатов
async function parallelAllSettled(operations) {
  console.time('parallelAllSettled');

  const results = await Promise.allSettled(
    operations.map((op, i) => op().catch(error => {
      // Преобразуем отклоненные промисы для сохранения ошибок
      throw { index: i, error, operation: op.name };
    })))
  );

  console.timeEnd('parallelAllSettled');

  // Анализируем результаты
  const fulfilled = results.filter(r => r.status === 'fulfilled');
  const rejected = results.filter(r => r.status === 'rejected');

  // Обрабатываем результаты
  const processedResults = results.map((result, index) => {
    if (result.status === 'fulfilled') {
      return {
        operation: operations[index].name || `op_${index}`,
        status: 'success',
        result: result.value,
        duration: result.duration // если добавили измерение времени
      };
    } else {
      const error = result.reason;

```

```

        return {
            operation: error.operation || operations[index].name || `op_${index}`,
            status: 'failed',
            error: error.error || error,
            index: error.index || index
        };
    }
});

```

```

return {
    processedResults,
    summary: {
        total: results.length,
        successful: fulfilled.length,
        failed: rejected.length,
        successRate: fulfilled.length / results.length
    }
};
}

```

// 4. Параллельное выполнение с независимой обработкой ошибок

```

class ParallelExecutor {
    constructor(options = {}) {
        this.options = {
            continueOnError: options.continueOnError ?? true,
            logProgress: options.logProgress ?? true,
            timeout: options.timeout,
            ...options
        };

        this.operations = [];
        this.results = [];
        this.errors = [];
    }

    addOperation(name, operation, metadata = {}) {
        this.operations.push({
            name,
            operation,
            metadata,
            id: this.operations.length
        });
        return this;
    }
}

```

```

async execute() {
  console.time('ParallelExecutor');

  const startTime = Date.now();
  const promises = this.operations.map(op => this.executeOperation(op));

  // Ждем завершения всех операций
  await Promise.allSettled(promises);

  const duration = Date.now() - startTime;
  console.timeEnd('ParallelExecutor');

  return {
    results: this.results,
    errors: this.errors,
    summary: {
      total: this.operations.length,
      successful: this.results.length,
      failed: this.errors.length,
      duration,
      operationsPerSecond: this.operations.length / (duration / 1000)
    }
  };
}

async executeOperation(operation) {
  const { name, operation: fn, id, metadata } = operation;
  const startTime = Date.now();

  if (this.options.logProgress) {
    console.log(`[${id}] Начинаем операцию: ${name}`);
  }

  try {
    // Применяем таймаут, если задан
    let promise = fn();

    if (this.options.timeout) {
      promise = this.withTimeout(promise, this.options.timeout, name);
    }

    // Выполняем операцию
    const result = await promise;

```

```

const duration = Date.now() - startTime;

if (this.options.logProgress) {
  console.log(`[${id}] Операция ${name} завершена за ${duration}мс`);
}

// Сохраняем результат
this.results.push({
  id,
  name,
  result,
  duration,
  metadata,
  status: 'success',
  timestamp: new Date().toISOString()
});

} catch (error) {
  const duration = Date.now() - startTime;

  if (this.options.logProgress) {
    console.error(`[${id}] Ошибка в операции ${name}:`, error.message);
  }

  // Сохраняем ошибку
  this.errors.push({
    id,
    name,
    error: error.message || error,
    duration,
    metadata,
    status: 'failed',
    timestamp: new Date().toISOString(),
    stack: error.stack
  });

  // Решаем, нужно ли пробрасывать ошибку дальше
  if (!this.options.continueOnError) {
    throw new ParallelExecutionError(
      `Операция ${name} не удалась`,
      { operation: name, error, id }
    );
  }
}
}

```

```

}

async withTimeout(promise, timeout, operationName) {
  let timeoutId;

  const timeoutPromise = new Promise((_, reject) => {
    timeoutId = setTimeout(() => {
      reject(new TimeoutError(
        `Операция ${operationName} превысила лимит времени ${timeout}мс`
      ));
    }, timeout);
  });

  try {
    return await Promise.race([promise, timeoutPromise]);
  } finally {
    clearTimeout(timeoutId);
  }
}

```

// Использование ParallelExecutor

```

const executor = new ParallelExecutor({
  continueOnError: true,
  logProgress: true,
  timeout: 5000
});

```

// Добавляем операции

```

executor
  .addOperation('fetchUsers', async () => {
    await delay(1000);
    return [{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }];
  }, { type: 'fetch' })
  .addOperation('fetchProducts', async () => {
    await delay(1500);
    return [{ id: 1, name: 'Product A' }, { id: 2, name: 'Product B' }];
  }, { type: 'fetch' })
  .addOperation('processData', async () => {
    await delay(800);
    return { processed: true, timestamp: new Date().toISOString() };
  }, { type: 'process' })
  .addOperation('failingOperation', async () => {
    await delay(500);

```

```

    throw new Error('Искусственная ошибка для теста');
  }, { type: 'test' });

// Запускаем выполнение
executor.execute()
  .then(({ results, errors, summary }) => {
    console.log('Выполнение завершено:');
    console.log(`Успешно: ${summary.successful}, Ошибок: ${summary.failed}`);
    console.log(`Общее время: ${summary.duration}мс`);
    console.log(`Операций в секунду: ${summary.operationsPerSecond.toFixed(2)}`);

    if (errors.length > 0) {
      console.log('Ошибки:', errors.map(e => `${e.name}: ${e.error}`));
    }
  })
  .catch(error => {
    console.error('Критическая ошибка выполнения:', error);
  });

```

Продвинутые паттерны параллельного выполнения

javascript

// 1. Параллельное выполнение с приоритетами

```

class PriorityParallelExecutor {
  constructor(concurrency = 4) {
    this.concurrency = concurrency;
    this.highPriorityQueue = [];
    this.normalPriorityQueue = [];
    this.lowPriorityQueue = [];
    this.running = 0;
    this.results = new Map();
    this.errors = new Map();
  }

  addTask(task, priority = 'normal') {
    const taskWithId = {
      id: Date.now() + Math.random(),
      task,
      priority,
      status: 'pending'
    };

    switch (priority) {
      case 'high':

```

```

        this.highPriorityQueue.push(taskWithId);
        break;
    case 'low':
        this.lowPriorityQueue.push(taskWithId);
        break;
    default:
        this.normalPriorityQueue.push(taskWithId);
    }

    this.processQueues();
    return taskWithId.id;
}

getNextTask() {
    if (this.highPriorityQueue.length > 0) {
        return this.highPriorityQueue.shift();
    }
    if (this.normalPriorityQueue.length > 0) {
        return this.normalPriorityQueue.shift();
    }
    if (this.lowPriorityQueue.length > 0) {
        return this.lowPriorityQueue.shift();
    }
    return null;
}

async processQueues() {
    while (this.running < this.concurrency) {
        const nextTask = this.getNextTask();

        if (!nextTask) {
            break;
        }

        this.running++;
        nextTask.status = 'running';

        this.executeTask(nextTask).finally(() => {
            this.running--;
            this.processQueues();
        });
    }
}

```

```

async executeTask(task) {
  const startTime = Date.now();

  console.log(`[${task.priority}] Выполняем задачу ${task.id}`);

  try {
    const result = await task.task();
    const duration = Date.now() - startTime;

    this.results.set(task.id, {
      result,
      duration,
      status: 'success',
      priority: task.priority
    });

    console.log(`[${task.priority}] Задача ${task.id} завершена за ${duration}мс`);

  } catch (error) {
    const duration = Date.now() - startTime;

    this.errors.set(task.id, {
      error,
      duration,
      status: 'failed',
      priority: task.priority
    });

    console.error(`[${task.priority}] Ошибка в задаче ${task.id}:`, error.message);
  }
}

```

```

async waitForCompletion() {
  // Ждем, пока все очереди опустеют и все задачи завершатся
  return new Promise((resolve) => {
    const checkCompletion = () => {
      if (this.running === 0 &&
        this.highPriorityQueue.length === 0 &&
        this.normalPriorityQueue.length === 0 &&
        this.lowPriorityQueue.length === 0) {

        resolve({
          results: Array.from(this.results.entries()),
          errors: Array.from(this.errors.entries()),

```



```

        summary: {
            total: this.results.size + this.errors.size,
            successful: this.results.size,
            failed: this.errors.size
        }
    });
} else {
    setTimeout(checkCompletion, 100);
}
};

    checkCompletion();
});
}
}

```

// Использование

```
const priorityExecutor = new PriorityParallelExecutor(3);
```

// Добавляем задачи с разными приоритетами

```
priorityExecutor.addTask(async () => {
    console.log('Выполняем критическую задачу');
    await delay(500);
    return 'critical_result';
}, 'high');
```

```
priorityExecutor.addTask(async () => {
    console.log('Выполняем важную задачу');
    await delay(1000);
    return 'important_result';
}, 'high');
```

```
priorityExecutor.addTask(async () => {
    console.log('Выполняем обычную задачу 1');
    await delay(1500);
    return 'normal_result_1';
}, 'normal');
```

```
priorityExecutor.addTask(async () => {
    console.log('Выполняем обычную задачу 2');
    await delay(800);
    return 'normal_result_2';
}, 'normal');
```

```

priorityExecutor.addTask(async () => {
  console.log('Выполняем фоновую задачу');
  await delay(2000);
  return 'background_result';
}, 'low');

// Ждем завершения
priorityExecutor.waitForCompletion()
  .then(({ results, errors, summary }) => {
    console.log('Все задачи завершены:');
    console.log(`Успешно: ${summary.successful}, Ошибок: ${summary.failed}`);

    // Анализируем время выполнения по приоритетам
    const byPriority = {};
    results.forEach(([id, result]) => {
      if (!byPriority[result.priority]) {
        byPriority[result.priority] = [];
      }
      byPriority[result.priority].push(result.duration);
    });

    console.log('Среднее время выполнения по приоритетам:');
    Object.entries(byPriority).forEach(([priority, durations]) => {
      const avg = durations.reduce((a, b) => a + b, 0) / durations.length;
      console.log(` ${priority}: ${avg.toFixed(0)}мс (${durations.length} задач)`);
    });
  });

```

// 2. Параллельное выполнение с кэшированием одинаковых операций

```

class CachedParallelExecutor {
  constructor() {
    this.cache = new Map();
    this.pendingOperations = new Map(); // operationKey -> Promise
    this.stats = {
      cacheHits: 0,
      cacheMisses: 0,
      executions: 0
    };
  }

  generateCacheKey(operation, args) {
    // Генерируем ключ для кэша на основе операции и аргументов
    const serializedArgs = JSON.stringify(args, (key, value) => {
      // Обрабатываем специальные типы данных

```

```

    if (value instanceof Date) return `Date:${value.toISOString()}`;
    if (value instanceof RegExp) return `RegExp:${value.toString()}`;
    if (typeof value === 'function') return `Function:${value.name}`;
    return value;
  });

  return `${operation.name || 'anonymous'}:${serializedArgs}`;
}

async execute(operation, ...args) {
  const cacheKey = this.generateCacheKey(operation, args);

  // Проверяем кэш
  if (this.cache.has(cacheKey)) {
    this.stats.cacheHits++;
    console.log(`Кэш-попадание для ${cacheKey}`);
    return this.cache.get(cacheKey);
  }

  // Проверяем, выполняется ли уже эта операция
  if (this.pendingOperations.has(cacheKey)) {
    console.log(`Ожидаем выполнения ${cacheKey}`);
    return this.pendingOperations.get(cacheKey);
  }

  this.stats.cacheMisses++;
  this.stats.executions++;

  console.log(`Выполняем операцию: ${cacheKey}`);

  // Создаем промис для операции
  const operationPromise = operation(...args)
    .then(result => {
      // Сохраняем результат в кэш
      this.cache.set(cacheKey, result);

      // Удаляем из pending operations
      this.pendingOperations.delete(cacheKey);

      return result;
    })
    .catch(error => {
      // В случае ошибки удаляем из pending
      this.pendingOperations.delete(cacheKey);
    });
}

```

```

        throw error;
    });

    // Сохраняем промис в pending operations
    this.pendingOperations.set(cacheKey, operationPromise);

    return operationPromise;
}

async executeMany(operations) {
    const promises = operations.map(({ operation, args = [] }) => {
        return this.execute(operation, ...args);
    });

    const results = await Promise.allSettled(promises);

    return {
        results: results.map((result, index) => ({
            operation: operations[index].name || operations[index].operation.name,
            status: result.status,
            value: result.status === 'fulfilled' ? result.value : undefined,
            reason: result.status === 'rejected' ? result.reason : undefined
        })),
        cacheStats: { ...this.stats },
        cacheSize: this.cache.size
    };
}

clearCache() {
    this.cache.clear();
    this.stats.cacheHits = 0;
    this.stats.cacheMisses = 0;
    this.stats.executions = 0;
}

getCacheInfo() {
    return {
        size: this.cache.size,
        pending: this.pendingOperations.size,
        hitRate: this.stats.cacheHits / (this.stats.cacheHits + this.stats.cacheMisses ||
1),
        ...this.stats
    };
}

```

```
}
```

```
// Использование
```

```
const cachedExecutor = new CachedParallelExecutor();
```

```
// Определяем операции
```

```
const fetchUserData = async (userId) => {  
  console.log(`Загружаем данные пользователя ${userId}...`);  
  await delay(1000);  
  return { id: userId, name: `User ${userId}`, fetchedAt: new Date().toISOString() };  
};
```

```
const fetchProductData = async (productId) => {  
  console.log(`Загружаем данные продукта ${productId}...`);  
  await delay(800);  
  return { id: productId, name: `Product ${productId}`, price: Math.random() * 100 };  
};
```

```
// Запускаем одинаковые операции (будут закэшированы)
```

```
const operations = [  
  { operation: fetchUserData, args: [1], name: 'fetchUser1' },  
  { operation: fetchUserData, args: [2], name: 'fetchUser2' },  
  { operation: fetchUserData, args: [1], name: 'fetchUser1_again' }, // Должен быть  
  закэширован  
  { operation: fetchProductData, args: [1], name: 'fetchProduct1' },  
  { operation: fetchProductData, args: [1], name: 'fetchProduct1_again' }, // Должен быть  
  закэширован  
  { operation: fetchUserData, args: [3], name: 'fetchUser3' }  
];
```

```
cachedExecutor.executeMany(operations)
```

```
.then(({ results, cacheStats }) => {  
  console.log('Результаты выполнения:');  
  results.forEach((result, index) => {  
    console.log(`${index + 1}. ${result.operation}: ${result.status}`);  
  });
```

```
  
  console.log(`\nСтатистика кэша: `);  
  console.log(`Кэш-попадания: ${cacheStats.cacheHits}`);  
  console.log(`Кэш-промахи: ${cacheStats.cacheMisses}`);  
  console.log(`Выполнения: ${cacheStats.executions}`);  
  console.log(`Эффективность кэша: ${(cacheStats.cacheHits / operations.length *  
100).toFixed(1)}%`);  
});
```

// 3. Параллельное выполнение с обработкой стримов данных

```
class ParallelStreamProcessor {
  constructor(concurrency = 4, batchSize = 10) {
    this.concurrency = concurrency;
    this.batchSize = batchSize;
    this.results = [];
    this.errors = [];
    this.processed = 0;
    this.stats = {
      startTime: null,
      endTime: null,
      batchesProcessed: 0
    };
  }

  async processStream(dataStream, processor) {
    this.stats.startTime = Date.now();

    // Преобразуем стрим в асинхронный итератор
    const asyncIterator = this.createAsyncIterator(dataStream);

    // Создаем очередь на обработку
    const processingQueue = [];
    let isStreamEnded = false;

    // Функция для заполнения очереди
    const fillQueue = async () => {
      while (processingQueue.length < this.concurrency * 2 && !isStreamEnded) {
        const { value, done } = await asyncIterator.next();

        if (done) {
          isStreamEnded = true;
          break;
        }

        processingQueue.push(value);
      }
    };

    // Основной цикл обработки
    await fillQueue();

    while (processingQueue.length > 0 || !isStreamEnded) {
```

```

// Берем порцию данных для обработки
const batch = processingQueue.splice(0, this.batchSize);

if (batch.length === 0) {
    await fillQueue();
    continue;
}

// Обрабатываем батч параллельно
const batchPromises = batch.map((item, index) =>
    this.processItem(item, processor, this.processed + index)
);

const batchResults = await Promise.allSettled(batchPromises);

// Обрабатываем результаты батча
batchResults.forEach((result, index) => {
    if (result.status === 'fulfilled') {
        this.results.push(result.value);
    } else {
        this.errors.push({
            item: batch[index],
            error: result.reason,
            index: this.processed + index
        });
    }
});

this.processed += batch.length;
this.stats.batchesProcessed++;

// Заполняем очередь дальше
await fillQueue();

// Логируем прогресс
if (this.processed % 100 === 0) {
    console.log(`Обработано: ${this.processed}, Ошибок: ${this.errors.length}`);
}

this.stats.endTime = Date.now();
this.stats.duration = this.stats.endTime - this.stats.startTime;
this.stats.itemsPerSecond = this.processed / (this.stats.duration / 1000);

```

```

return {
  results: this.results,
  errors: this.errors,
  stats: this.stats
};
}

async *createAsyncIterator(dataStream) {
  // В зависимости от типа стрима создаем итератор
  if (dataStream[Symbol.asyncIterator]) {
    yield* dataStream;
  } else if (dataStream[Symbol.iterator]) {
    for (const item of dataStream) {
      yield item;
    }
  } else if (Array.isArray(dataStream)) {
    for (const item of dataStream) {
      yield item;
    }
  } else {
    throw new Error('Неподдерживаемый тип стрима');
  }
}

async processItem(item, processor, index) {
  const startTime = Date.now();

  try {
    const result = await processor(item, index);
    const duration = Date.now() - startTime;

    return {
      item,
      result,
      duration,
      index,
      status: 'success'
    };
  } catch (error) {
    const duration = Date.now() - startTime;

    throw {
      item,
      error,

```



```

        duration,
        index,
        status: 'failed'
    });
}
}
}

// Использование обработчика стримов
async function* generateTestData(count) {
    for (let i = 0; i < count; i++) {
        await delay(Math.random() * 100); // Имитация задержки получения данных
        yield { id: i, data: `Item ${i}`, value: Math.random() };
    }
}

const streamProcessor = new ParallelStreamProcessor(5, 20);

// Процессор для элементов
const itemProcessor = async (item, index) => {
    // Имитация обработки
    await delay(50 + Math.random() * 100);

    // Иногда генерируем ошибку для теста
    if (Math.random() < 0.05) {
        throw new Error(`Ошибка обработки элемента ${item.id}`);
    }

    return {
        ...item,
        processed: true,
        processedAt: new Date().toISOString(),
        transformedValue: item.value * 100
    };
};

// Запуск обработки стрима
streamProcessor.processStream(generateTestData(1000), itemProcessor)
    .then(({ results, errors, stats }) => {
        console.log(`\nОбработка стрима завершена:`);
        console.log(`Обработано элементов: ${results.length}`);
        console.log(`Ошибок: ${errors.length}`);
        console.log(`Общее время: ${stats.duration}мс`);
        console.log(`Элементов в секунду: ${stats.itemsPerSecond.toFixed(2)}`);
    });

```

```

console.log(`Обработано батчей: ${stats.batchesProcessed}`);

if (errors.length > 0) {
  console.log('\nПримеры ошибок:');
  errors.slice(0, 3).forEach(error => {
    console.log(`Элемент ${error.item.id}: ${error.error.message}`);
  });
}
})
.catch(error => {
  console.error('Ошибка обработки стрима:', error);
});

```

11.6.3. Ограниченное параллельное выполнение (конкурентность)

Реализация пула потоков (Thread Pool)

javascript

// 1. Базовый пул с ограничением параллелизма

```

class AsyncPool {
  constructor(maxConcurrent = 5) {
    this.maxConcurrent = maxConcurrent;
    this.running = 0;
    this.queue = [];
    this.results = new Map();
    this.taskId = 0;
    this.stats = {
      completed: 0,
      failed: 0,
      totalTime: 0
    };
  }

  addTask(task, priority = 0) {
    const id = ++this.taskId;

    return new Promise((resolve, reject) => {
      this.queue.push({
        id,
        task,
        priority,

```

```

        resolve,
        reject,
        addedAt: Date.now()
    });

    // Сортируем очередь по приоритету (высокий приоритет = больше число)
    this.queue.sort((a, b) => b.priority - a.priority);

    this.processQueue();
});
}

processQueue() {
    // Запускаем задачи, пока не достигнем лимита
    while (this.running < this.maxConcurrent && this.queue.length > 0) {
        this.running++;
        const task = this.queue.shift();

        this.executeTask(task).finally(() => {
            this.running--;
            this.processQueue();
        });
    }
}

async executeTask(task) {
    const startTime = Date.now();
    const waitTime = startTime - task.addedAt;

    console.log(`[Task ${task.id}] Начинаем выполнение (ожидал ${waitTime}мс)`);

    try {
        const result = await task.task();
        const duration = Date.now() - startTime;

        this.stats.completed++;
        this.stats.totalTime += duration;

        this.results.set(task.id, {
            result,
            duration,
            waitTime,
            status: 'success'
        });
    }
}

```

```

    console.log(`[Task ${task.id}] Завершено за ${duration}мс`);

    task.resolve(result);

} catch (error) {
    const duration = Date.now() - startTime;

    this.stats.failed++;
    this.stats.totalTime += duration;

    this.results.set(task.id, {
        error,
        duration,
        waitTime,
        status: 'failed'
    });

    console.error(`[Task ${task.id}] Ошибка за ${duration}мс`, error.message);

    task.reject(error);
}
}

async waitForCompletion() {
    // Ждем, пока все задачи завершатся
    return new Promise((resolve) => {
        const checkCompletion = () => {
            if (this.running === 0 && this.queue.length === 0) {
                resolve({
                    results: Array.from(this.results.entries()),
                    stats: {
                        ...this.stats,
                        averageTime: this.stats.totalTime / (this.stats.completed +
this.stats.failed),
                        successRate: this.stats.completed / (this.stats.completed +
this.stats.failed)
                    }
                });
            } else {
                setTimeout(checkCompletion, 100);
            }
        };
    });
}

```

```

        checkCompletion();
    });
}

getQueueStats() {
    return {
        running: this.running,
        queued: this.queue.length,
        maxConcurrent: this.maxConcurrent,
        utilization: (this.running / this.maxConcurrent) * 100
    };
}
}

// Использование пула
const pool = new AsyncPool(3); // Максимум 3 одновременные задачи

// Генерируем тестовые задачи
const tasks = Array.from({ length: 10 }, (_, i) => ({
    id: i + 1,
    duration: 500 + Math.random() * 1000,
    priority: Math.floor(Math.random() * 3) // 0, 1, 2
}));

// Добавляем задачи в пул
const promises = tasks.map(task =>
    pool.addTask(async () => {
        console.log(`Задача ${task.id} (приоритет ${task.priority}) начинается`);
        await delay(task.duration);
        return `Результат задачи ${task.id}`;
    }, task.priority)
);

// Мониторим состояние очереди
const monitorInterval = setInterval(() => {
    const stats = pool.getQueueStats();
    console.log(`Состояние пула: ${stats.running} запущено, ${stats.queued} в очереди,
    утилизация: ${stats.utilization.toFixed(1)}%`);
}, 500);

// Ждем завершения всех задач
Promise.allSettled(promises)
    .then(() => pool.waitForCompletion())
    .then(({ results, stats }) => {

```

```

clearInterval(monitorInterval);

console.log('\nВсе задачи завершены:');
console.log(`Успешно: ${stats.completed}, Ошибок: ${stats.failed}`);
console.log(`Среднее время выполнения: ${stats.averageTime.toFixed(0)}мс`);
console.log(`Успешность: ${(stats.successRate * 100).toFixed(1)}%`);

// Анализируем время ожидания по приоритетам
const waitTimesByPriority = {};
results.forEach(([id, result]) => {
  const task = tasks.find(t => t.id === id);
  if (task) {
    if (!waitTimesByPriority[task.priority]) {
      waitTimesByPriority[task.priority] = [];
    }
    waitTimesByPriority[task.priority].push(result.waitTime);
  }
});

console.log('\nСреднее время ожидания по приоритетам:');
Object.entries(waitTimesByPriority).forEach(([priority, times]) => {
  const avg = times.reduce((a, b) => a + b, 0) / times.length;
  console.log(` Приоритет ${priority}: ${avg.toFixed(0)}мс (${times.length}
задач)`);
});

// 2. Пул с динамическим масштабированием
class DynamicAsyncPool {
  constructor(options = {}) {
    this.minConcurrent = options.minConcurrent || 1;
    this.maxConcurrent = options.maxConcurrent || 10;
    this.targetQueueSize = options.targetQueueSize || 5;
    this.scaleUpThreshold = options.scaleUpThreshold || 0.8; // 80% утилизации
    this.scaleDownThreshold = options.scaleDownThreshold || 0.3; // 30% утилизации
    this.scaleInterval = options.scaleInterval || 5000; // Проверка каждые 5 секунд

    this.currentConcurrent = this.minConcurrent;
    this.running = 0;
    this.queue = [];
    this.metrics = {
      tasksCompleted: 0,
      tasksFailed: 0,
      scaleUpEvents: 0,

```

```

        scaleDownEvents: 0,
        avgTaskTime: 0
    };

    // Запускаем автоскейлинг
    this.startAutoScaling();
}

addTask(task, metadata = {}) {
    return new Promise((resolve, reject) => {
        this.queue.push({
            task,
            metadata,
            resolve,
            reject,
            addedAt: Date.now(),
            status: 'queued'
        });

        this.processQueue();
    });
}

processQueue() {
    while (this.running < this.currentConcurrent && this.queue.length > 0) {
        const task = this.queue.shift();
        this.executeTask(task);
    }
}

async executeTask(task) {
    this.running++;
    task.status = 'running';

    const startTime = Date.now();
    task.startedAt = startTime;

    try {
        const result = await task.task();
        const duration = Date.now() - startTime;

        this.metrics.tasksCompleted++;
        this.updateAverageTime(duration);
    }
}

```

```

    task.status = 'completed';
    task.resolve(result);

} catch (error) {
    const duration = Date.now() - startTime;

    this.metrics.tasksFailed++;
    this.updateAverageTime(duration);

    task.status = 'failed';
    task.reject(error);

} finally {
    this.running--;
    this.processQueue();
}
}

updateAverageTime(newTime) {
    const totalTasks = this.metrics.tasksCompleted + this.metrics.tasksFailed;
    this.metrics.avgTaskTime =
        (this.metrics.avgTaskTime * (totalTasks - 1) + newTime) / totalTasks;
}

startAutoScaling() {
    setInterval(() => {
        this.autoScale();
    }, this.scaleInterval);
}

autoScale() {
    const utilization = this.running / this.currentConcurrent;
    const queueRatio = this.queue.length / this.targetQueueSize;

    console.log(`[AutoScale] Утилизация: ${utilization * 100}.toFixed(1)}%, Очередь: $
    {this.queue.length}/${this.targetQueueSize}`);

    // Логика масштабирования
    if (utilization > this.scaleUpThreshold && this.currentConcurrent <
    this.maxConcurrent) {
        this.scaleUp();
    } else if (utilization < this.scaleDownThreshold &&
        this.currentConcurrent > this.minConcurrent &&
        queueRatio < 0.5) {

```



```

        this.scaleDown();
    }
}

scaleUp() {
    const increase = Math.ceil(this.currentConcurrent * 0.5); // Увеличиваем на 50%
    this.currentConcurrent = Math.min(this.currentConcurrent + increase,
this.maxConcurrent);

    this.metrics.scaleUpEvents++;
    console.log(`[AutoScale] Масштабируем ВВЕРХ до ${this.currentConcurrent} потоков`);

    this.processQueue();
}

scaleDown() {
    const decrease = Math.ceil(this.currentConcurrent * 0.25); // Уменьшаем на 25%
    this.currentConcurrent = Math.max(this.currentConcurrent - decrease,
this.minConcurrent);

    this.metrics.scaleDownEvents++;
    console.log(`[AutoScale] Масштабируем ВНИЗ до ${this.currentConcurrent} потоков`);
}

getMetrics() {
    const totalTasks = this.metrics.tasksCompleted + this.metrics.tasksFailed;

    return {
        ...this.metrics,
        currentConcurrent: this.currentConcurrent,
        running: this.running,
        queued: this.queue.length,
        utilization: (this.running / this.currentConcurrent) * 100,
        successRate: totalTasks > 0 ? this.metrics.tasksCompleted / totalTasks : 0
    };
}

async waitForCompletion() {
    return new Promise((resolve) => {
        const checkCompletion = () => {
            if (this.running === 0 && this.queue.length === 0) {
                resolve(this.getMetrics());
            } else {
                setTimeout(checkCompletion, 100);
            }
        };
    });
}

```

```

    }
  };

  checkCompletion();
});
}
}

// Использование динамического пула
const dynamicPool = new DynamicAsyncPool({
  minConcurrent: 2,
  maxConcurrent: 8,
  targetQueueSize: 10,
  scaleUpThreshold: 0.7,
  scaleDownThreshold: 0.2,
  scaleInterval: 3000
});

// Симулируем нагрузку
async function simulateLoad(pool, taskCount = 50) {
  const tasks = Array.from({ length: taskCount }, (_, i) => ({
    id: i + 1,
    duration: 300 + Math.random() * 2000, // 300-2300ms
    complexity: Math.random()
  }));

  console.log(`Добавляем ${taskCount} задач в пул...`);

  const startTime = Date.now();

  const promises = tasks.map(task =>
    pool.addTask(async () => {
      console.log(`Task ${task.id}] Начинается (${task.duration.toFixed(0)}мс)`);
      await delay(task.duration);

      // Иногда генерируем ошибку
      if (Math.random() < 0.1) {
        throw new Error(`Искусственная ошибка в задаче ${task.id}`);
      }

      return `Результат задачи ${task.id}`;
    }, { ...task })
  );
};

```

```

// Мониторим метрики
const monitor = setInterval(() => {
  const metrics = pool.getMetrics();
  console.log(`[Metrics] Поток: ${metrics.currentConcurrent}, Запущено: ${
metrics.running}, Очередь: ${metrics.queued}`);
}, 2000);

// Ждем завершения
const results = await Promise.allSettled(promises);
clearInterval(monitor);

const duration = Date.now() - startTime;
const finalMetrics = pool.getMetrics();

console.log(`\n` + '='.repeat(50));
console.log('Симуляция завершена:');
console.log(`Общее время: ${duration}мс`);
console.log(`Задач в секунду: ${((taskCount / (duration / 1000)).toFixed(2))}`);
console.log(`\nФинальные метрики пула:`);
console.log(`  Всего задач: ${finalMetrics.tasksCompleted + finalMetrics.tasksFailed}
`);
console.log(`  Успешно: ${finalMetrics.tasksCompleted}`);
console.log(`  Ошибок: ${finalMetrics.tasksFailed}`);
console.log(`  Успешность: ${(finalMetrics.successRate * 100).toFixed(1)}%`);
console.log(`  Среднее время задачи: ${finalMetrics.avgTaskTime.toFixed(0)}мс`);
console.log(`  Масштабирований вверх: ${finalMetrics.scaleUpEvents}`);
console.log(`  Масштабирований вниз: ${finalMetrics.scaleDownEvents}`);
console.log(`  Финальное количество потоков: ${finalMetrics.currentConcurrent}`);

return { results, metrics: finalMetrics, duration };
}

// Запускаем симуляцию
simulateLoad(dynamicPool, 100)
  .then(() => console.log('\nСимуляция завершена успешно'))
  .catch(error => console.error('Ошибка симуляции:', error));

```

11.6.4. Комбинированные стратегии выполнения

Гибридный исполнитель (последовательно-параллельный)

javascript

```

class HybridExecutor {
  constructor(options = {}) {
    this.options = {
      maxParallel: options.maxParallel || 5,
      batchSize: options.batchSize || 10,
      retryAttempts: options.retryAttempts || 3,
      continueOnError: options.continueOnError ?? true,
      ...options
    };

    this.phases = [];
    this.results = {};
    this.metrics = {
      phaseDurations: {},
      totalDuration: 0,
      itemsProcessed: 0,
      errors: []
    };
  }

  // Добавление фазы выполнения
  addPhase(name, processor, options = {}) {
    this.phases.push({
      name,
      processor,
      parallel: options.parallel ?? true,
      dependsOn: options.dependsOn || [],
      batchSize: options.batchSize || this.options.batchSize,
      maxParallel: options.maxParallel || this.options.maxParallel,
      critical: options.critical ?? false
    });

    return this; // Для чейнинга
  }

  // Основной метод выполнения
  async execute(inputData, context = {}) {
    const startTime = Date.now();

    console.log(`Начинаем выполнение с ${this.phases.length} фазами`);
    console.log(`Входные данные: ${Array.isArray(inputData) ? inputData.length + ' элементов' : 'объект'}`);

    // Проверяем зависимости фаз

```

```

this.validatePhaseDependencies();

// Определяем порядок выполнения фаз (топологическая сортировка)
const executionOrder = this.getExecutionOrder();
console.log('Порядок выполнения фаз:', executionOrder.map(p => p.name));

let currentData = inputData;
let currentContext = { ...context };

// Выполняем фазы по порядку
for (const phase of executionOrder) {
    const phaseStartTime = Date.now();

    console.log(`\n${'='.repeat(50)}`);
    console.log(`Фаза: ${phase.name}`);
    console.log(`Тип: ${phase.parallel ? 'Параллельная' : 'Последовательная'}`);
    console.log(`Входные данные: ${Array.isArray(currentData) ? currentData.length + ' элементов' : 'объект'}`);

    try {
        // Выполняем фазу
        const phaseResult = await this.executePhase(
            phase,
            currentData,
            currentContext
        );

        const phaseDuration = Date.now() - phaseStartTime;
        this.metrics.phaseDurations[phase.name] = phaseDuration;

        console.log(`Фаза ${phase.name} завершена за ${phaseDuration}мс`);

        // Обновляем данные для следующей фазы
        currentData = phaseResult.results;
        currentContext[phase.name] = phaseResult;

        // Сохраняем результаты
        this.results[phase.name] = phaseResult;

        // Обновляем метрики
        this.metrics.itemsProcessed += Array.isArray(phaseResult.results)
            ? phaseResult.results.length
            : 1;
    }
}

```

```

    if (phaseResult.errors && phaseResult.errors.length > 0) {
      this.metrics.errors.push(...phaseResult.errors.map(e => ({
        phase: phase.name,
        error: e
      })));
    }

  } catch (error) {
    const phaseDuration = Date.now() - phaseStartTime;
    this.metrics.phaseDurations[phase.name] = phaseDuration;

    console.error(`Фаза ${phase.name} завершилась с ошибкой за ${phaseDuration}мс`,
error.message);

    if (phase.critical) {
      console.error('Критическая фаза не выполнена, прерываем выполнение');
      throw new PhaseExecutionError(
        `Критическая фаза ${phase.name} не выполнена`,
        { phase: phase.name, error, results: this.results }
      );
    }

    // Если фаза не критическая, продолжаем с текущими данными
    this.results[phase.name] = {
      status: 'failed',
      error: error.message,
      duration: phaseDuration
    };
  }
}

this.metrics.totalDuration = Date.now() - startTime;

console.log(`\n${'='.repeat(50)}`);
console.log('Все фазы выполнены');
console.log(`Общее время: ${this.metrics.totalDuration}мс`);
console.log(`Обработано элементов: ${this.metrics.itemsProcessed}`);
console.log(`Ошибок: ${this.metrics.errors.length}`);

if (this.metrics.errors.length > 0) {
  console.log('\nОшибки по фазам:');
  const errorsByPhase = {};
  this.metrics.errors.forEach(error => {
    if (!errorsByPhase[error.phase]) {

```

```

        errorsByPhase[error.phase] = 0;
    }
    errorsByPhase[error.phase]++;
});

Object.entries(errorsByPhase).forEach(([phase, count]) => {
    console.log(` ${phase}: ${count} ошибок`);
});
}

return {
    results: this.results,
    finalData: currentData,
    finalContext: currentContext,
    metrics: this.metrics
};
}

async executePhase(phase, data, context) {
    if (phase.parallel) {
        return this.executeParallelPhase(phase, data, context);
    } else {
        return this.executeSequentialPhase(phase, data, context);
    }
}

async executeParallelPhase(phase, data, context) {
    if (!Array.isArray(data)) {
        throw new Error(`Параллельная фаза ${phase.name} ожидает массив данных`);
    }

    const results = [];
    const errors = [];
    const batches = this.createBatches(data, phase.batchSize);

    console.log(`Обработка ${data.length} элементов в ${batches.length} батчах`);

    // Обработываем батчи с ограничением параллелизма
    const pool = new AsyncPool(phase.maxParallel);

    for (let i = 0; i < batches.length; i++) {
        const batch = batches[i];

        await pool.addTask(async () => {

```

```

        console.log(`[${phase.name}] Обрабатываем батч ${i + 1}/${batches.length} (${
batch.length} элементов)`);

        const batchResults = [];
        const batchErrors = [];

        // Обрабатываем элементы батча параллельно
        const itemPromises = batch.map((item, index) =>
            this.processWithRetry(() => phase.processor(item, context, i * phase.batchSize
+ index))

                .then(result => batchResults.push(result))
                .catch(error => batchErrors.push({
                    item,
                    error: error.message,
                    index: i * phase.batchSize + index
                })))
        );

        await Promise.allSettled(itemPromises);

        results.push(...batchResults);
        errors.push(...batchErrors);

        return { batch: i + 1, processed: batchResults.length, errors: batchErrors.length
};
    });
}

// Ждем завершения всех батчей
await pool.waitForCompletion();

return {
    results,
    errors,
    summary: {
        totalItems: data.length,
        processed: results.length,
        failed: errors.length,
        batches: batches.length
    }
};
}

async executeSequentialPhase(phase, data, context) {

```



```

const results = [];
const errors = [];

if (Array.isArray(data)) {
  console.log(`Последовательно обрабатываем ${data.length} элементов`);

  for (let i = 0; i < data.length; i++) {
    try {
      const result = await this.processWithRetry(
        () => phase.processor(data[i], context, i),
        this.options.retryAttempts
      );

      results.push(result);

      // Логим прогресс
      if ((i + 1) % 10 === 0 || i === data.length - 1) {
        console.log(`[${phase.name}] Обработано ${i + 1}/${data.length} элементов`);
      }
    } catch (error) {
      errors.push({
        item: data[i],
        error: error.message,
        index: i
      });

      if (!this.options.continueOnError) {
        throw new Error(`Ошибка обработки элемента ${i}: ${error.message}`);
      }
    }
  }
} else {
  // Одиночный элемент
  try {
    const result = await this.processWithRetry(
      () => phase.processor(data, context, 0)
    );
    results.push(result);
  } catch (error) {
    errors.push({
      item: data,
      error: error.message,
      index: 0
    });
  }
}

```

```

    });
    throw error;
  }
}

return {
  results: Array.isArray(data) ? results : results[0],
  errors,
  summary: {
    totalItems: Array.isArray(data) ? data.length : 1,
    processed: results.length,
    failed: errors.length
  }
};
}

async processWithRetry(operation, maxAttempts = this.options.retryAttempts) {
  let lastError;

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error;

      if (attempt === maxAttempts) {
        break;
      }

      // Экспоненциальная задержка
      const delay = 100 * Math.pow(2, attempt - 1);
      console.log(`Попытка ${attempt} не удалась, ждем ${delay}мс перед повторной
попыткой`);
      await delay(delay);
    }
  }

  throw lastError;
}

createBatches(array, batchSize) {
  const batches = [];
  for (let i = 0; i < array.length; i += batchSize) {
    batches.push(array.slice(i, i + batchSize));
  }
}

```

```

    }
    return batches;
}

validatePhaseDependencies() {
    const phaseNames = new Set(this.phases.map(p => p.name));

    for (const phase of this.phases) {
        for (const dependency of phase.dependsOn) {
            if (!phaseNames.has(dependency)) {
                throw new Error(`Фаза ${phase.name} зависит от несуществующей фазы $
{dependency}`);
            }
        }
    }
}

getExecutionOrder() {
    // Топологическая сортировка для определения порядка выполнения фаз
    const visited = new Set();
    const stack = new Set();
    const order = [];

    const visit = (phaseName) => {
        if (stack.has(phaseName)) {
            throw new Error(`Обнаружена циклическая зависимость с фазой ${phaseName}`);
        }

        if (visited.has(phaseName)) {
            return;
        }

        const phase = this.phases.find(p => p.name === phaseName);
        if (!phase) return;

        stack.add(phaseName);
        visited.add(phaseName);

        for (const dep of phase.dependsOn) {
            visit(dep);
        }

        stack.delete(phaseName);
        order.push(phase);
    }

```

```

};

for (const phase of this.phases) {
  if (!visited.has(phase.name)) {
    visit(phase.name);
  }
}

return order;
}
}

// Пример использования HybridExecutor
const executor = new HybridExecutor({
  maxParallel: 4,
  batchSize: 5,
  retryAttempts: 2,
  continueOnError: true
});

// Добавляем фазы обработки данных
executor
  .addPhase('validate', async (item, context, index) => {
    // Валидация элемента
    await delay(50 + Math.random() * 100);

    if (!item.id) {
      throw new Error(`Элемент ${index} не имеет ID`);
    }

    if (item.value < 0) {
      throw new Error(`Элемент ${item.id} имеет отрицательное значение`);
    }

    return { ...item, validated: true };
  }, { parallel: true, maxParallel: 3 })

  .addPhase('transform', async (item, context, index) => {
    // Преобразование элемента
    await delay(100 + Math.random() * 200);

    const transformed = {
      ...item,
      transformed: true,
    };
  });

```

```

        processedAt: new Date().toISOString(),
        calculatedValue: item.value * 2
    };

    return transformed;
}, { parallel: true, dependsOn: ['validate'] })

.addPhase('enrich', async (item, context, index) => {
    // Обогащение данными из внешнего источника
    await delay(200 + Math.random() * 300);

    // Имитация запроса к API
    const metadata = await fetchMetadata(item.id);

    return { ...item, metadata, enriched: true };
}, { parallel: false, dependsOn: ['transform'], critical: true })

.addPhase('aggregate', async (items, context, index) => {
    // Агрегация всех элементов (последовательная фаза над массивом)
    await delay(500);

    const total = items.reduce((sum, item) => sum + item.calculatedValue, 0);
    const average = total / items.length;

    return {
        summary: {
            totalItems: items.length,
            totalValue: total,
            averageValue: average,
            processedAt: new Date().toISOString()
        },
        items // Возвращаем исходные элементы тоже
    };
}, { parallel: false, dependsOn: ['enrich'] });

// Подготовка тестовых данных
const testData = Array.from({ length: 50 }, (_, i) => ({
    id: i + 1,
    value: Math.random() * 100,
    name: `Item ${i + 1}`
})));

// Запуск выполнения
executor.execute(testData, { environment: 'test', version: '1.0.0' })

```

```

.then(({ results, finalData, metrics }) => {
  console.log('\n' + '='.repeat(60));
  console.log('РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ:');
  console.log('='.repeat(60));

  console.log('\nМетрики выполнения:');
  console.log(`Общее время: ${metrics.totalDuration}мс`);
  console.log(`Обработано элементов: ${metrics.itemsProcessed}`);
  console.log(`Ошибок: ${metrics.errors.length}`);

  console.log('\nВремя выполнения по фазам:');
  Object.entries(metrics.phaseDurations).forEach(([phase, duration]) => {
    console.log(`  ${phase}: ${duration}мс`);
  });

  console.log('\nРезультаты агрегации:');
  if (results.aggregate && results.aggregate.results) {
    console.log(results.aggregate.results.summary);
  }

  console.log('\nПример обработанных данных (первые 3 элемента:');
  if (Array.isArray(finalData) && finalData.length > 0) {
    finalData.slice(0, 3).forEach((item, i) => {
      console.log(`  ${i + 1}. ${JSON.stringify(item, null, 2).slice(0, 100)}...`);
    });
  }
})
.catch(error => {
  console.error('Ошибка выполнения:', error);

  if (error.results) {
    console.log('\nЧастичные результаты:');
    Object.entries(error.results).forEach(([phase, result]) => {
      console.log(`  ${phase}: ${result.status || 'unknown'}`);
    });
  }
});

```

// Вспомогательная функция для имитации API

```

async function fetchMetadata(id) {
  await delay(150 + Math.random() * 200);
  return {
    source: 'external-api',
    fetchedAt: new Date().toISOString(),
  };
}

```

```

    tags: [`tag_${id % 5}`]
  };
}

```

11.6.5. Производительность и оптимизация

Бенчмаркинг различных стратегий

```

javascript

class PerformanceBenchmark {
  constructor() {
    this.results = new Map();
    this.testCases = [];
  }

  addTestCase(name, operation, options = {}) {
    this.testCases.push({
      name,
      operation,
      iterations: options.iterations || 100,
      concurrency: options.concurrency || 5,
      dataSize: options.dataSize || 1000
    });
    return this;
  }

  async runBenchmarks() {
    console.log('Запуск бенчмарков производительности...\n');

    for (const testCase of this.testCases) {
      console.log(`Тест: ${testCase.name}`);
      console.log(`Итераций: ${testCase.iterations}, Конкурентность: ${
        testCase.concurrency}`);

      const results = await this.runTestCase(testCase);
      this.results.set(testCase.name, results);

      console.log(`Результат: ${results.duration}мс (${results.throughput.toFixed(2)}
оп/сек)`);
      console.log(`Успешно: ${results.successful}, Ошибок: ${results.failed}\n`);
    }
  }
}

```

```

    this.printComparison();
    return this.results;
}

async runTestCase(testCase) {
    const { name, operation, iterations, concurrency, dataSize } = testCase;

    // Генерируем тестовые данные
    const testData = Array.from({ length: dataSize }, (_, i) => ({
        id: i,
        value: Math.random() * 100,
        payload: 'x'.repeat(100) // 100 байт на элемент
    }));

    const startTime = Date.now();

    // Тестируем разные стратегии
    const sequentialResult = await this.testSequential(testData, operation, iterations);
    const parallelResult = await this.testParallel(testData, operation, iterations,
concurrency);
    const batchedResult = await this.testBatched(testData, operation, iterations,
concurrency);
    const hybridResult = await this.testHybrid(testData, operation, iterations,
concurrency);

    const duration = Date.now() - startTime;

    return {
        duration,
        sequential: sequentialResult,
        parallel: parallelResult,
        batched: batchedResult,
        hybrid: hybridResult,
        throughput: iterations / (duration / 1000),
        successful: sequentialResult.successful + parallelResult.successful +
            batchedResult.successful + hybridResult.successful,
        failed: sequentialResult.failed + parallelResult.failed +
            batchedResult.failed + hybridResult.failed
    };
}

async testSequential(data, operation, iterations) {
    const results = [];
    const errors = [];

```



```

const startTime = Date.now();

for (let i = 0; i < iterations; i++) {
  try {
    const result = await operation(data[i % data.length], i);
    results.push(result);
  } catch (error) {
    errors.push(error);
  }
}

return {
  duration: Date.now() - startTime,
  successful: results.length,
  failed: errors.length,
  opsPerSecond: iterations / ((Date.now() - startTime) / 1000)
};
}

async testParallel(data, operation, iterations, concurrency) {
  const pool = new AsyncPool(concurrency);
  const startTime = Date.now();

  for (let i = 0; i < iterations; i++) {
    pool.addTask(() => operation(data[i % data.length], i));
  }

  const result = await pool.waitForCompletion();

  return {
    duration: Date.now() - startTime,
    successful: result.stats.completed,
    failed: result.stats.failed,
    opsPerSecond: iterations / ((Date.now() - startTime) / 1000),
    queueStats: result.stats
  };
}

async testBatched(data, operation, iterations, concurrency) {
  const batchSize = Math.ceil(iterations / concurrency);
  const batches = [];

  for (let i = 0; i < iterations; i += batchSize) {
    const batch = [];

```

```

    for (let j = 0; j < batchSize && i + j < iterations; j++) {
        batch.push(() => operation(data[(i + j) % data.length], i + j));
    }
    batches.push(batch);
}

const startTime = Date.now();
const results = [];
const errors = [];

// Обрабатываем батчи параллельно
await Promise.all(batches.map(async (batch) => {
    const batchResults = await Promise.allSettled(
        batch.map(op => op())
    );

    batchResults.forEach(result => {
        if (result.status === 'fulfilled') {
            results.push(result.value);
        } else {
            errors.push(result.reason);
        }
    });
}));

return {
    duration: Date.now() - startTime,
    successful: results.length,
    failed: errors.length,
    opsPerSecond: iterations / ((Date.now() - startTime) / 1000),
    batchCount: batches.length,
    avgBatchSize: iterations / batches.length
};
}

async testHybrid(data, operation, iterations, concurrency) {
    // Комбинированная стратегия: сначала параллельно, потом батчами
    const phase1Count = Math.floor(iterations * 0.7); // 70% параллельно
    const phase2Count = iterations - phase1Count; // 30% батчами

    const startTime = Date.now();

    // Фаза 1: параллельное выполнение
    const phase1Start = Date.now();

```

```

const pool = new AsyncPool(concurrency);

for (let i = 0; i < phase1Count; i++) {
  pool.addTask(() => operation(data[i % data.length], i));
}

const phase1Result = await pool.waitForCompletion();
const phase1Duration = Date.now() - phase1Start;

// Фаза 2: батчевое выполнение
const phase2Start = Date.now();
const batchSize = Math.ceil(phase2Count / 2);
const phase2Results = [];
const phase2Errors = [];

for (let i = phase1Count; i < iterations; i += batchSize) {
  const batch = [];
  for (let j = 0; j < batchSize && i + j < iterations; j++) {
    batch.push(() => operation(data[(i + j) % data.length], i + j));
  }

  const batchResults = await Promise.all(batch.map(op => op()));
  phase2Results.push(...batchResults);
}

const phase2Duration = Date.now() - phase2Start;
const totalDuration = Date.now() - startTime;

return {
  duration: totalDuration,
  successful: phase1Result.stats.completed + phase2Results.length,
  failed: phase1Result.stats.failed + phase2Errors.length,
  opsPerSecond: iterations / (totalDuration / 1000),
  phase1: { duration: phase1Duration, ...phase1Result.stats },
  phase2: { duration: phase2Duration, successful: phase2Results.length, failed:
phase2Errors.length }
};
}

printComparison() {
  console.log('\n' + '='.repeat(80));
  console.log('СРАВНЕНИЕ СТРАТЕГИЙ ВЫПОЛНЕНИЯ');
  console.log('=' + '='.repeat(80));
}

```

```

const comparison = [];

for (const [testName, result] of this.results.entries()) {
  comparison.push({
    test: testName,
    sequential: result.sequential.opsPerSecond,
    parallel: result.parallel.opsPerSecond,
    batched: result.batched.opsPerSecond,
    hybrid: result.hybrid.opsPerSecond,
    bestStrategy: this.getBestStrategy(result)
  });
}

// Выводим таблицу сравнения
console.log('\nОпераций в секунду (больше = лучше:');
console.log('-'.repeat(80));
console.log('Тест'.padEnd(20) + 'Послед.'.padEnd(12) + 'Паралл.'.padEnd(12) +
  'Пакетн.'.padEnd(12) + 'Гибридна.'.padEnd(12) + 'Лучшая');
console.log('-'.repeat(80));

comparison.forEach(row => {
  console.log(
    row.test.padEnd(20) +
    row.sequential.toFixed(2).padEnd(12) +
    row.parallel.toFixed(2).padEnd(12) +
    row.batched.toFixed(2).padEnd(12) +
    row.hybrid.toFixed(2).padEnd(12) +
    row.bestStrategy
  );
});

console.log('-'.repeat(80));

// Анализ результатов
console.log('\nАНАЛИЗ РЕЗУЛЬТАТОВ:');

const averages = comparison.reduce((acc, row) => {
  acc.sequential += row.sequential;
  acc.parallel += row.parallel;
  acc.batched += row.batched;
  acc.hybrid += row.hybrid;
  return acc;
}, { sequential: 0, parallel: 0, batched: 0, hybrid: 0 });

```

```

    const count = comparison.length;

    console.log(`Средняя производительность:`);
    console.log(` Последовательная: ${((averages.sequential / count).toFixed(2))}
оп/сек`);
    console.log(` Параллельная: ${((averages.parallel / count).toFixed(2))} оп/сек (x$
${(averages.parallel / averages.sequential).toFixed(2)})`);
    console.log(` Пакетная: ${((averages.batched / count).toFixed(2))} оп/сек (x$
${(averages.batched / averages.sequential).toFixed(2)})`);
    console.log(` Гибридная: ${((averages.hybrid / count).toFixed(2))} оп/сек (x$
${(averages.hybrid / averages.sequential).toFixed(2)})`);

    // Рекомендации
    console.log(`\nРЕКОМЕНДАЦИИ:`);
    console.log(`1. Для CPU-intensive операций: последовательное или пакетное
выполнение`);
    console.log(`2. Для IO-intensive операций: параллельное выполнение`);
    console.log(`3. Для смешанных нагрузок: гибридная стратегия`);
    console.log(`4. Для больших объемов данных: пакетная обработка с кэшированием`);
}

getBestStrategy(result) {
    const strategies = [
        { name: 'Последовательная', value: result.sequential.opsPerSecond },
        { name: 'Параллельная', value: result.parallel.opsPerSecond },
        { name: 'Пакетная', value: result.batched.opsPerSecond },
        { name: 'Гибридная', value: result.hybrid.opsPerSecond }
    ];

    const best = strategies.reduce((best, current) =>
        current.value > best.value ? current : best
    );

    return best.name;
}

// Запуск бенчмарков
const benchmark = new PerformanceBenchmark();

// Добавляем тестовые сценарии
benchmark
    .addTestCase('CPU-intensive', async (data, index) => {
        // Имитация CPU-intensive операции

```

```

    let result = 0;
    for (let i = 0; i < 1000000; i++) {
        result += Math.sin(i) * Math.cos(i);
    }
    return { index, result, processed: true };
}, { iterations: 50, concurrency: 4, dataSize: 100 })

.addTestCase('IO-intensive', async (data, index) => {
    // Имитация IO-intensive операции
    await delay(100 + Math.random() * 200);
    return { ...data, processed: true, index };
}, { iterations: 200, concurrency: 10, dataSize: 500 })

.addTestCase('Mixed workload', async (data, index) => {
    // Смешанная нагрузка
    // CPU часть
    let sum = 0;
    for (let i = 0; i < 10000; i++) {
        sum += Math.sqrt(i);
    }

    // IO часть
    await delay(50 + Math.random() * 100);

    return { ...data, sum, processed: true, index };
}, { iterations: 100, concurrency: 6, dataSize: 300 })

.addTestCase('Network requests', async (data, index) => {
    // Имитация сетевых запросов
    await delay(200 + Math.random() * 300);

    // 10% вероятность ошибки
    if (Math.random() < 0.1) {
        throw new Error(`Сетевой запрос для элемента ${index} не удался`);
    }

    return { ...data, fetched: true, index, timestamp: new Date().toISOString() };
}, { iterations: 150, concurrency: 8, dataSize: 400 });

// Запускаем бенчмарки
benchmark.runBenchmarks()
    .then(results => {
        console.log('\nБенчмарки завершены. Результаты сохранены.');
```

```
.catch(error => {
  console.error('Ошибка выполнения бенчмарков:', error);
});
```

11.6.6. Практические рекомендации

Выбор стратегии выполнения

javascript

// Дерево решений для выбора стратегии

```
function selectExecutionStrategy(context) {
  const {
    operationType, // 'cpu' | 'io' | 'mixed' | 'network'
    dataSize,      // количество элементов
    itemComplexity, // сложность обработки одного элемента
    dependencies,   // зависимости между операциями
    errorTolerance, // допустимость ошибок
    timeConstraints, // временные ограничения
    resources       // доступные ресурсы (CPU, память, сеть)
  } = context;

  // Дерево решений
  if (dataSize === 1) {
    return {
      strategy: 'sequential',
      reason: 'Один элемент - используем последовательное выполнение'
    };
  }

  if (dependencies && dependencies.length > 0) {
    return {
      strategy: 'sequential',
      reason: 'Есть зависимости между операциями - последовательное выполнение'
    };
  }

  if (operationType === 'cpu' && dataSize > 100) {
    return {
      strategy: 'batched',
      reason: 'CPU-intensive операции на большом количестве данных - пакетная обработка',
      config: {
        batchSize: Math.min(10, Math.floor(dataSize / resources.cpuCores)),

```

```

        maxConcurrent: resources.cpuCores
    }
};
}

if (operationType === 'io' || operationType === 'network') {
    return {
        strategy: 'parallel',
        reason: 'IO/Network операции - параллельное выполнение',
        config: {
            maxConcurrent: Math.min(20, resources.memoryLimit / itemComplexity),
            continueOnError: errorTolerance > 0.5
        }
    };
}

if (dataSize > 1000) {
    return {
        strategy: 'hybrid',
        reason: 'Большой объем данных - гибридная стратегия',
        config: {
            batchSize: 100,
            maxParallel: resources.cpuCores * 2,
            phases: [
                { name: 'preprocess', parallel: true, batchSize: 50 },
                { name: 'process', parallel: false },
                { name: 'postprocess', parallel: true, batchSize: 20 }
            ]
        }
    };
}

// По умолчанию - параллельное выполнение с ограничением
return {
    strategy: 'parallel',
    reason: 'Баланс между производительностью и использованием ресурсов',
    config: {
        maxConcurrent: Math.min(5, Math.floor(resources.memoryLimit / (itemComplexity *
2))),
        timeout: timeConstraints?.maxDuration ? timeConstraints.maxDuration / dataSize :
5000
    }
};
}

```



```

// Примеры использования
const scenarios = [
  // Сценарий 1: Обработка изображений (CPU-intensive)
  {
    operationType: 'cpu',
    dataSize: 500,
    itemComplexity: 50, // высокая сложность
    dependencies: null,
    errorTolerance: 0.1, // низкая
    timeConstraints: { maxDuration: 30000 },
    resources: { cpuCores: 4, memoryLimit: 8000 }
  },

  // Сценарий 2: Запросы к API (Network)
  {
    operationType: 'network',
    dataSize: 200,
    itemComplexity: 10, // низкая сложность
    dependencies: null,
    errorTolerance: 0.8, // высокая
    timeConstraints: { maxDuration: 10000 },
    resources: { cpuCores: 2, memoryLimit: 4000 }
  },

  // Сценарий 3: Конвейер обработки данных
  {
    operationType: 'mixed',
    dataSize: 1000,
    itemComplexity: 30,
    dependencies: ['validate', 'transform', 'enrich'],
    errorTolerance: 0.3,
    timeConstraints: { maxDuration: 60000 },
    resources: { cpuCores: 8, memoryLimit: 16000 }
  }
];

console.log('РЕКОМЕНДАЦИИ ПО СТРАТЕГИЯМ ВЫПОЛНЕНИЯ:');
console.log('=' .repeat(60));

scenarios.forEach((scenario, index) => {
  const recommendation = selectExecutionStrategy(scenario);

  console.log(`\nСценарий ${index + 1}:`);

```

```

console.log(` Тип операций: ${scenario.operationType}`);
console.log(` Количество элементов: ${scenario.dataSize}`);
console.log(` Рекомендуемая стратегия: ${recommendation.strategy}`);
console.log(` Причина: ${recommendation.reason}`);

if (recommendation.config) {
    console.log(` Конфигурация: ${JSON.stringify(recommendation.config, null,
2).replace(/\n/g, '\n    ')} `);
}
});

```

11.6.7. Упражнения для закрепления

Упражнение 1: Оптимизация существующего кода

```

javascript

// Исходный неоптимизированный код
async function processUsersUnoptimized(users) {
    const results = [];

    for (const user of users) {
        try {
            // Последовательные запросы для каждого пользователя
            const profile = await fetchUserProfile(user.id);
            const orders = await fetchUserOrders(user.id);
            const settings = await fetchUserSettings(user.id);

            // Обработка данных
            const processed = await processUserData(profile, orders, settings);
            results.push(processed);

        } catch (error) {
            console.error(`Ошибка обработки пользователя ${user.id}:`, error);
        }
    }

    return results;
}

// Задача: Оптимизировать этот код, используя:
// 1. Параллельное выполнение независимых запросов
// 2. Батчинг для больших объемов данных

```

```
// 3. Кэширование повторяющихся запросов
// 4. Ограничение параллелизма для защиты от DDoS
```

```
async function processUsersOptimized(users) {
  // Ваша оптимизированная реализация здесь
}
```

Упражнение 2: Реализация системы очередей с приоритетами

javascript

```
// Задача: Создать систему очередей с такими возможностями:
// 1. Разные приоритеты задач
// 2. Динамическое изменение приоритетов
// 3. Приостановка и возобновление выполнения
// 4. Статистика выполнения задач
// 5. Обработка зависимостей между задачами
```

```
class PriorityTaskQueue {
  constructor(options = {}) {
    // Ваша реализация здесь
  }

  addTask(task, priority = 0) {
    // Добавление задачи в очередь
  }

  pause() {
    // Приостановка выполнения
  }

  resume() {
    // Возобновление выполнения
  }

  getStats() {
    // Получение статистики
  }

  async waitForCompletion() {
    // Ожидание завершения всех задач
  }
}
```

Упражнение 3: Создание адаптивного исполнителя

```
javascript

// Задача: Создать исполнитель, который автоматически выбирает
// оптимальную стратегию выполнения на основе:
// 1. Исторических данных о выполнении подобных задач
// 2. Текущей загрузки системы
// 3. Приоритета задачи
// 4. Временных ограничений

class AdaptiveExecutor {
  constructor(learningModel) {
    this.learningModel = learningModel;
    this.executionHistory = [];
    this.currentLoad = 0;
  }

  async execute(task, context) {
    // 1. Анализ задачи и контекста
    // 2. Выбор стратегии на основе модели
    // 3. Выполнение задачи
    // 4. Сохранение метрик для обучения
    // 5. Адаптация стратегии при необходимости
  }

  analyzeTask(task, context) {
    // Анализ характеристик задачи
  }

  selectStrategy(analysis) {
    // Выбор стратегии на основе анализа
  }

  updateLearningModel(results) {
    // Обновление модели на основе результатов выполнения
  }
}
```

Заключение

Параллельное и последовательное выполнение асинхронных операций — это ключевой аспект разработки производительных и отзывчивых приложений на JavaScript.

Ключевые выводы:

- 1. **Последовательное выполнение:**
 - ➔ Проще в реализации и отладке
 - ➔ Подходит для зависимых операций
 - ➔ Может быть неэффективно для больших объемов данных
- 2. **Параллельное выполнение:**
 - ➔ Максимизирует использование ресурсов
 - ➔ Требует управления конкурентностью
 - ➔ Сложнее в обработке ошибок
- 3. **Гибридные подходы:**
 - ➔ Сочетают преимущества обоих методов
 - ➔ Требуют более сложной архитектуры
 - ➔ Обеспечивают лучшую производительность в сложных сценариях

Рекомендации по выбору стратегии:

Ситуация	Рекомендуемая стратегия	Почему
Небольшое количество независимых операций	Параллельное выполнение	Максимальная скорость
Зависимые операции	Последовательное выполнение	Простота и надежность
Большой объем данных	Батчинг с ограниченным параллелизмом	Баланс скорости и использования ресурсов
Смешанная нагрузка (CPU + IO)	Гибридная стратегия	Оптимизация для разных типов операций
Критически важные операции	Последовательное с обработкой ошибок	Надежность и предсказуемость

Лучшие практики:

- 1. **Всегда ограничивайте параллелизм** — защищайте систему от перегрузки

2. **Используйте кэширование** — избегайте повторного выполнения одинаковых операций
3. **Реализуйте механизмы повтора** — для обработки временных ошибок
4. **Собирайте метрики** — для анализа и оптимизации производительности
5. **Тестируйте различные стратегии** — на реальных данных и нагрузках

Помните: нет универсального решения. Оптимальная стратегия зависит от конкретного сценария, доступных ресурсов и требований приложения. Экспериментируйте, измеряйте и адаптируйтесь к изменяющимся условиям.

Глава 12: Модульность и организация кода

12.1. Паттерны модулей: IIFE, модульный паттерн

Введение: зачем нужны модули?

В ранние годы JavaScript использовался для добавления небольших интерактивных элементов на страницы. Код часто писался в одном файле, что приводило к нескольким проблемам:

1. **Глобальное загрязнение (Global Scope Pollution)** – все переменные и функции становились глобальными.
2. **Конфликты имён** – разные скрипты могли использовать одинаковые имена переменных.
3. **Сложность поддержки** – большой монолитный файл сложно читать и изменять.
4. **Отсутствие инкапсуляции** – любой код мог изменить любые данные.

Модули решают эти проблемы, предоставляя:

- **Изоляцию кода** – переменные и функции внутри модуля не видны снаружи.
- **Инкапсуляцию** – можно скрыть внутреннюю реализацию.
- **Повторное использование** – модули можно импортировать в разных местах.
- **Удобство тестирования** – модули можно тестировать изолированно.

В этой главе мы рассмотрим первые паттерны модулей, которые появились до стандарта ES6.

12.1.1. IIFE (Immediately Invoked Function Expression) – Немедленно вызываемое функциональное выражение

Концепция и синтаксис

IIFE – это функция, которая определяется и сразу же выполняется. Это создаёт изолированную область видимости, защищая внутренние переменные от глобальной области.

Базовый синтаксис:

```
javascript

(function() {
  // Код внутри IIFE
  console.log('IIFE выполняется немедленно!');
})();
```

Альтернативные синтаксисы:

```
javascript

// Классический вариант
(function() {
  // ...
})();

// Вариант с оператором !
!function() {
  // ...
})();

// Вариант с оператором +
+function() {
  // ...
})();

// Вариант с оператором void
void function() {
  // ...
}();
```

Зачем использовать IIFE?

1. Изоляция переменных:

```
javascript

var globalVar = 'Я глобальная';

(function() {
  var localVar = 'Я локальная в IIFE';
  console.log(localVar); // "Я локальная в IIFE"
  console.log(globalVar); // "Я глобальная" - доступ к глобальным переменным есть
})();

console.log(localVar); // Ошибка: localVar is not defined
```


2. Избегание конфликтов имён:

```
javascript

// Библиотека А (представьте, что это jQuery)
(function(window) {
  var $ = function(selector) {
    // Реализация библиотеки А
    return { version: 'A-1.0' };
  };
  window.$ = $; // Экспортируем в глобальную область
})(window);

// Библиотека В
(function(window) {
  var $ = function(selector) {
    // Реализация библиотеки В
    return { version: 'B-2.0' };
  };
  window.jQuery = $; // Экспортируем под другим именем
})(window);

// Использование
console.log($.version); // "A-1.0"
console.log(jQuery().version); // "B-2.0"
```

3. Создание приватных переменных:

```
javascript

var counterModule = (function() {
  // Приватная переменная - недоступна извне
  var count = 0;

  // Приватная функция
  function log(message) {
    console.log('[Counter]: ' + message);
  }

  // Публичный интерфейс
  return {
    increment: function() {
      count++;
      log('Увеличили до ' + count);
      return count;
    },

    decrement: function() {
```

```

    count--;
    log('Уменьшили до ' + count);
    return count;
},

getValue: function() {
    return count;
},

reset: function() {
    count = 0;
    log('Сбросили');
    return count;
}
};
})();

// Использование
console.log(counterModule.getValue()); // 0
counterModule.increment();           // [Counter]: Увеличили до 1
counterModule.increment();           // [Counter]: Увеличили до 2
console.log(counterModule.getValue()); // 2

// Приватные переменные недоступны
console.log(counterModule.count);     // undefined
console.log(count);                  // Ошибка: count is not defined

```

4. Защита от минификаторов:

```

javascript

// Без IIFE - минификатор может переименовать параметры
function processData(data, options) {
    // ...
}

// С IIFE - защищаем важные параметры
(function(global, document, undefined) {
    // undefined гарантированно undefined (защита от переопределения)
    // global и document защищены от переименования

    var myModule = {
        // ...
    };

    global.myModule = myModule;

```

```
})(window, document);
```

IIFE с параметрами

javascript

// Передача глобальных объектов как параметров

```
(function($, window, document, undefined) {  
  'use strict';
```

// Теперь \$ гарантированно jQuery

// undefined гарантированно undefined

// window и document локальные, доступ быстрее

```
$(document).ready(function() {  
  console.log('Документ загружен');  
});
```

```
})(jQuery, window, document);
```

Асинхронные IIFE (до появления async/await)

javascript

// Паттерн для работы с асинхронным кодом

```
(function() {  
  var data;
```

// Имитация асинхронной операции

```
fetch('/api/data')  
  .then(function(response) {  
    return response.json();  
  })  
  .then(function(result) {  
    data = result;  
    initApp();  
  })  
  .catch(function(error) {  
    console.error('Ошибка:', error);  
  });
```

```
function initApp() {  
  console.log('Приложение инициализировано с данными:', data);  
  // Дальнейшая логика  
}  
})();
```

12.1.2. Модульный паттерн (Module Pattern)

Развитие IIFE: создание модулей с состоянием

Модульный паттерн – это расширение IIFE, которое добавляет:

1. **Приватные переменные и функции**
2. **Публичный интерфейс**
3. **Возможность создания нескольких экземпляров**

Базовый модульный паттерн

javascript

```
var CartModule = (function() {  
    // Приватные переменные  
    var items = [];  
    var total = 0;  
  
    // Приватные вспомогательные функции  
    function calculateTotal() {  
        total = items.reduce(function(sum, item) {  
            return sum + item.price * item.quantity;  
        }, 0);  
        return total;  
    }  
  
    function validateItem(item) {  
        return item &&  
            typeof item.name === 'string' &&  
            typeof item.price === 'number' &&  
            item.price > 0;  
    }  
  
    // Публичный интерфейс  
    return {  
        addItem: function(item) {  
            if (!validateItem(item)) {  
                console.error('Неверный товар');  
                return false;  
            }  
        }  
    }  
})()
```

```

// Проверяем, есть ли уже такой товар
var existingItem = items.find(function(i) {
    return i.name === item.name;
});

if (existingItem) {
    existingItem.quantity += item.quantity || 1;
} else {
    items.push({
        name: item.name,
        price: item.price,
        quantity: item.quantity || 1
    });
}

calculateTotal();
return true;
},

removeItem: function(itemName) {
    var initialLength = items.length;
    items = items.filter(function(item) {
        return item.name !== itemName;
    });

    if (items.length !== initialLength) {
        calculateTotal();
        return true;
    }
    return false;
},

updateQuantity: function(itemName, newQuantity) {
    var item = items.find(function(i) {
        return i.name === itemName;
    });

    if (item && newQuantity > 0) {
        item.quantity = newQuantity;
        calculateTotal();
        return true;
    }
    return false;
}

```

```

},

getItems: function() {
  // Возвращаем копию, чтобы защитить исходные данные
  return JSON.parse(JSON.stringify(items));
},

getTotal: function() {
  return total;
},

clear: function() {
  items = [];
  total = 0;
  return true;
},

getItemCount: function() {
  return items.reduce(function(count, item) {
    return count + item.quantity;
  }, 0);
};
})();

// Использование модуля
CartModule.addItem({ name: 'Яблоки', price: 50 });
CartModule.addItem({ name: 'Бананы', price: 70, quantity: 2 });
CartModule.addItem({ name: 'Яблоки', price: 50 }); // Увеличит количество

console.log(CartModule.getItems()); // Массив товаров
console.log(CartModule.getItemCount()); // 4 (1 яблоко + 2 банана + 1 яблоко)
console.log(CartModule.getTotal()); // 240 (50 + 140 + 50)

CartModule.updateQuantity('Бананы', 3);
console.log(CartModule.getTotal()); // 310 (50 + 210 + 50)

// Приватные переменные недоступны
console.log(CartModule.items); // undefined
console.log(items); // Ошибка: items is not defined

```

Расширенный модульный паттерн (Revealing Module Pattern)

Этот вариант делает код более читаемым: мы явно указываем, какие функции являются публичными.

```
javascript

var UserManager = (function() {
  'use strict';

  // Приватные переменные
  var users = [];
  var nextId = 1;

  // Приватные функции
  function findUserById(id) {
    return users.find(function(user) {
      return user.id === id;
    });
  }

  function validateUserData(userData) {
    return userData &&
      typeof userData.name === 'string' &&
      userData.name.trim().length > 0 &&
      typeof userData.email === 'string' &&
      /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(userData.email);
  }

  function generateAvatar(name) {
    var colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7'];
    var color = colors[name.length % colors.length];

    return {
      initials: name.split(' ').map(function(word) {
        return word[0];
      }).join('').toUpperCase(),
      color: color
    };
  }

  // Публичные функции (определяются как приватные, затем "раскрываются")
  function addUser(userData) {
    if (!validateUserData(userData)) {

```

```

    throw new Error('Неверные данные пользователя');
}

var newUser = {
  id: nextId++,
  name: userData.name.trim(),
  email: userData.email.toLowerCase(),
  avatar: generateAvatar(userData.name),
  createdAt: new Date(),
  isActive: true
};

users.push(newUser);
return newUser.id;
}

function getUser(id) {
  var user = findUserById(id);
  if (!user) {
    throw new Error('Пользователь не найден');
  }
  // Возвращаем копию, чтобы защитить исходные данные
  return Object.assign({}, user);
}

function getAllUsers() {
  // Сортировка по дате создания (новые первыми)
  return users
    .slice()
    .sort(function(a, b) {
      return b.createdAt - a.createdAt;
    })
    .map(function(user) {
      return Object.assign({}, user);
    });
}

function updateUser(id, updates) {
  var user = findUserById(id);
  if (!user) {
    throw new Error('Пользователь не найден');
  }

  if (updates.name && typeof updates.name === 'string') {

```



```

    user.name = updates.name.trim();
    user.avatar = generateAvatar(user.name);
}

if (updates.email && typeof updates.email === 'string') {
    if (!/^[\s@]+@[\s@]+\.[\s@]+$/ .test(updates.email)) {
        throw new Error('Неверный email');
    }
    user.email = updates.email.toLowerCase();
}

if (typeof updates.isActive === 'boolean') {
    user.isActive = updates.isActive;
}

return true;
}

function deleteUser(id) {
    var initialLength = users.length;
    users = users.filter(function(user) {
        return user.id !== id;
    });
    return users.length !== initialLength;
}

function searchUsers(query) {
    var searchTerm = query.toLowerCase().trim();
    if (!searchTerm) return getAllUsers();

    return users
        .filter(function(user) {
            return user.name.toLowerCase().includes(searchTerm) ||
                user.email.toLowerCase().includes(searchTerm);
        })
        .map(function(user) {
            return Object.assign({}, user);
        });
}

function getStatistics() {
    var activeUsers = users.filter(function(user) {
        return user.isActive;
    }).length;

```

```
return {
  total: users.length,
  active: activeUsers,
  inactive: users.length - activeUsers,
  lastCreated: users.length > 0 ?
    new Date(Math.max.apply(null, users.map(function(u) {
      return u.createdAt;
    }))) : null
};
}
```

// "Раскрываем" только публичные методы

```
return {
  add: addUser,
  get: getUser,
  getAll: getAllUsers,
  update: updateUser,
  delete: deleteUser,
  search: searchUsers,
  stats: getStatistics
};
})();
```

// Использование

```
try {
  var userId1 = UserManager.add({
    name: 'Иван Иванов',
    email: 'ivan@example.com'
  });

  var userId2 = UserManager.add({
    name: 'Мария Петрова',
    email: 'maria@example.com'
  });

  console.log(UserManager.getAll());
  console.log(UserManager.stats());

  UserManager.update(userId1, { name: 'Иван Сидоров' });
  console.log(UserManager.get(userId1));

} catch (error) {
  console.error('Ошибка:', error.message);
}
```

```
}
```

Модульный паттерн с конструктором

Для создания нескольких независимых экземпляров:

```
javascript
```

```
var TodoList = (function() {  
  // Приватная функция-конструктор  
  function TodoList(name) {  
    if (!(this instanceof TodoList)) {  
      return new TodoList(name);  
    }  
  
    var todos = [];  
    var listName = name || 'Мой список';  
    var nextId = 1;  
  
    // Приватные методы экземпляра  
    function findTodoById(id) {  
      return todos.find(function(todo) {  
        return todo.id === id;  
      });  
    }  
  
    // Публичные методы (через this)  
    this.add = function(text, priority) {  
      var todo = {  
        id: nextId++,  
        text: String(text || ''),  
        completed: false,  
        priority: Math.min(Math.max(Number(priority) || 1, 1), 5),  
        createdAt: new Date(),  
        completedAt: null  
      };  
  
      if (!todo.text.trim()) {  
        throw new Error('Текст задачи не может быть пустым');  
      }  
  
      todos.push(todo);  
      return todo.id;  
    };  
  }  
})
```

```
this.complete = function(id) {  
  var todo = findTodoById(id);  
  if (todo && !todo.completed) {  
    todo.completed = true;  
    todo.completedAt = new Date();  
    return true;  
  }  
  return false;  
};
```

```
this.uncomplete = function(id) {  
  var todo = findTodoById(id);  
  if (todo && todo.completed) {  
    todo.completed = false;  
    todo.completedAt = null;  
    return true;  
  }  
  return false;  
};
```

```
this.remove = function(id) {  
  var initialLength = todos.length;  
  todos = todos.filter(function(todo) {  
    return todo.id !== id;  
  });  
  return todos.length !== initialLength;  
};
```

```
this.getAll = function(filter) {  
  var filtered = todos.slice();  
  
  if (filter === 'active') {  
    filtered = filtered.filter(function(todo) {  
      return !todo.completed;  
    });  
  } else if (filter === 'completed') {  
    filtered = filtered.filter(function(todo) {  
      return todo.completed;  
    });  
  }  
}
```

```
// Сортировка: приоритет (по убыванию), затем дата создания  
return filtered.sort(function(a, b) {
```

```

        if (b.priority !== a.priority) {
            return b.priority - a.priority;
        }
        return a.createdAt - b.createdAt;
    });
};

this.clearCompleted = function() {
    var initialLength = todos.length;
    todos = todos.filter(function(todo) {
        return !todo.completed;
    });
    return initialLength - todos.length;
};

this.rename = function(newName) {
    if (typeof newName === 'string' && newName.trim()) {
        listName = newName.trim();
        return true;
    }
    return false;
};

this.getName = function() {
    return listName;
};

this.getStats = function() {
    var total = todos.length;
    var completed = todos.filter(function(todo) {
        return todo.completed;
    }).length;

    return {
        total: total,
        completed: completed,
        active: total - completed,
        completionRate: total > 0 ? (completed / total * 100).toFixed(1) + '%' : '0%'
    };
};
}

```

// Статические методы (доступны через конструктор)

```

TodoList.merge = function(list1, list2, newName) {

```

```
var merged = new TodoList(newName || list1.getName() + ' + ' + list2.getName());

var todos1 = list1.getAll();
var todos2 = list2.getAll();

todos1.forEach(function(todo) {
  merged.add(todo.text, todo.priority);
  if (todo.completed) {
    merged.complete(todo.id);
  }
});

todos2.forEach(function(todo) {
  merged.add(todo.text, todo.priority);
  if (todo.completed) {
    merged.complete(todo.id);
  }
});

return merged;
};

// Возвращаем конструктор
return TodoList;
})();

// Использование
var workList = new TodoList('Рабочие задачи');
var homeList = new TodoList('Домашние дела');

workList.add('Подготовить отчет', 5);
workList.add('Созвониться с клиентом', 3);
workList.add('Проверить почту', 1);

homeList.add('Купить продукты', 4);
homeList.add('Позвонить родителям', 2);

workList.complete(1);
homeList.complete(4);

console.log('Рабочие задачи:', workList.getAll());
console.log('Статистика работы:', workList.getStats());

// Объединение списков
```

```
var mergedList = TodoList.merge(workList, homeList, 'Все задачи');
console.log('Объединенный список:', mergedList.getAll());
```

Модульный паттерн с зависимостями

javascript

// Модуль, зависящий от других модулей

```
var AnalyticsModule = (function($, StorageModule, Config) {
  'use strict';

  // Приватные переменные
  var events = [];
  var isInitialized = false;
  var maxEvents = Config.analytics.maxEvents || 100;

  // Приватные функции
  function sendToServer(batch) {
    // Имитация отправки на сервер
    return $.ajax({
      url: Config.api.analytics,
      method: 'POST',
      data: JSON.stringify(batch),
      contentType: 'application/json'
    });
  }

  function saveToStorage() {
    try {
      StorageModule.set('analytics_events', events.slice(-50)); // Сохраняем последние 50
    } catch (e) {
      console.warn('Не удалось сохранить события аналитики:', e.message);
    }
  }

  function loadFromStorage() {
    try {
      var saved = StorageModule.get('analytics_events') || [];
      events = saved.concat(events).slice(-maxEvents);
    } catch (e) {
      console.warn('Не удалось загрузить события аналитики:', e.message);
    }
  }

  function flushEvents() {

```

```

if (events.length === 0) return Promise.resolve();

var batch = events.slice();
events = [];

return sendToServer(batch)
  .then(function() {
    console.log('События аналитики отправлены:', batch.length);
  })
  .catch(function(error) {
    // При ошибке сохраняем события обратно
    events = batch.concat(events).slice(-maxEvents);
    console.error('Ошибка отправки аналитики:', error);
    saveToStorage();
  });
}

// Публичный интерфейс
var publicAPI = {
  init: function() {
    if (isInitialized) return;

    loadFromStorage();

    // Автоматическая отправка каждые 30 секунд
    setInterval(function() {
      if (events.length >= 10) {
        flushEvents();
      }
    }, 30000);

    // Отправка при закрытии страницы
    $(window).on('beforeunload', function() {
      if (events.length > 0) {
        // Синхронный запрос для надежности
        var xhr = new XMLHttpRequest();
        xhr.open('POST', Config.api.analytics, false);
        xhr.setRequestHeader('Content-Type', 'application/json');
        xhr.send(JSON.stringify(events));
      }
    });

    isInitialized = true;
    console.log('AnalyticsModule инициализирован');
  }
};

```



```

},

track: function(eventName, properties) {
  if (!isInitialized) {
    console.warn('AnalyticsModule не инициализирован');
    return false;
  }

  var event = {
    name: eventName,
    properties: properties || {},
    timestamp: new Date().toISOString(),
    url: window.location.href,
    userAgent: navigator.userAgent
  };

  events.push(event);

  // Автоматическая отправка при достижении лимита
  if (events.length >= maxEvents) {
    flushEvents();
  }

  return true;
},

trackPageView: function(pageName) {
  return this.track('page_view', {
    page: pageName || document.title,
    path: window.location.pathname
  });
},

trackClick: function(elementId, text) {
  return this.track('click', {
    elementId: elementId,
    text: text,
    page: document.title
  });
},

getEvents: function() {
  return events.slice();
},

```

```

    forceSend: function() {
        return flushEvents();
    },

    clear: function() {
        events = [];
        StorageModule.remove('analytics_events');
        return true;
    }
};

return publicAPI;
})(jQuery, StorageModule, AppConfig);

// Инициализация
AnalyticsModule.init();

// Использование
AnalyticsModule.trackPageView('Главная страница');
AnalyticsModule.trackClick('login-button', 'Войти');
AnalyticsModule.track('user_registration', {
    plan: 'premium',
    source: 'google_ads'
});

```

12.1.3. Сравнение IIFE и модульного паттерна

Критерий	IIFE	Модульный паттерн
Основная цель	Изоляция кода, создание области видимости	Создание модулей с публичным API и приватным состоянием
Инкапсуляция	Базовая (скрывает переменные)	Продвинутая (публичные/приватные методы)
Состояние	Обычно не сохраняется между вызовами	Состояние сохраняется в замыкании
Повторное использование	Ограничено	Хорошее, можно создавать экземпляры
Сложность	Просто	Средняя

Критерий	IIFE	Модульный паттерн
Поддержка зависимостей	Через параметры	Через параметры (зависимости внедряются)
Тестируемость	Сложнее тестировать	Легче тестировать (изолированные модули)

12.1.4. Практические рекомендации

Когда использовать IIFE:

1. Для изоляции кода библиотек:

```

javascript

// Библиотека должна работать в любом окружении
(function(global, factory) {
  if (typeof define === 'function' && define.amd) {
    // AMD (RequireJS)
    define(['dependency'], factory);
  } else if (typeof exports === 'object') {
    // CommonJS (Node.js)
    module.exports = factory(require('dependency'));
  } else {
    // Браузер
    global.MyLib = factory(global.Dependency);
  }
})(this, function(Dependency) {
  // Реализация библиотеки
  return {
    // публичные методы
  };
});

```

2. Для избежания конфликтов в плагинах:

```

javascript

// Плагин для jQuery
(function($) {
  $.fn.myPlugin = function(options) {
    var settings = $.extend({
      color: 'red',
      speed: 400
    }, options);
  };
});

```

```

return this.each(function() {
    var $this = $(this);
    // Реализация плагина
});
};

```

```

})(jQuery);

```

3. Для выполнения кода при загрузке:

```

javascript

```

```

// Инициализация при загрузке страницы

```

```

(function() {
    'use strict';

    // Проверка поддержки необходимых функций
    if (!('querySelector' in document) ||
        !('addEventListener' in window)) {
        console.error('Браузер не поддерживает необходимые функции');
        return;
    }
}

```

```

// Инициализация приложения

```

```

document.addEventListener('DOMContentLoaded', function() {
    App.init();
});
})();

```

Когда использовать модульный паттерн:

1. Для создания сервисов в приложении:

```

javascript

```

```

// Сервис для работы с API

```

```

var ApiService = (function() {
    var baseUrl = '/api/v1';
    var token = null;

    function makeRequest(method, endpoint, data) {
        var headers = {
            'Content-Type': 'application/json'
        };

        if (token) {
            headers['Authorization'] = 'Bearer ' + token;
        }

        return fetch(baseUrl + endpoint, {

```

```

    method: method,
    headers: headers,
    body: data ? JSON.stringify(data) : null
  }).then(function(response) {
    if (!response.ok) {
      throw new Error('HTTP ' + response.status);
    }
    return response.json();
  });
}

```

```

return {
  setToken: function(newToken) {
    token = newToken;
  },

  clearToken: function() {
    token = null;
  },

  get: function(endpoint) {
    return makeRequest('GET', endpoint);
  },

  post: function(endpoint, data) {
    return makeRequest('POST', endpoint, data);
  },

  put: function(endpoint, data) {
    return makeRequest('PUT', endpoint, data);
  },

  delete: function(endpoint) {
    return makeRequest('DELETE', endpoint);
  }
};
})();

```

2. Для создания виджетов/компонентов:

javascript

// Виджет аккордеона

```

var Accordion = (function() {
  var instances = [];

```

```

function Accordion(element, options) {
  if (!(this instanceof Accordion)) {
    return new Accordion(element, options);
  }

  this.element = element;
  this.settings = Object.assign({
    duration: 300,
    multiple: false,
    onOpen: null,
    onClose: null
  }, options || {});

  this.items = [];
  this.init();
  instances.push(this);
}

Accordion.prototype = {
  init: function() {
    var headers = this.element.querySelectorAll('.accordion-header');

    Array.prototype.forEach.call(headers, function(header, index) {
      var item = {
        header: header,
        content: header.nextElementSibling,
        isOpen: false
      };

      this.items.push(item);

      header.addEventListener('click', function() {
        this.toggle(index);
      }.bind(this));

      // Инициализация состояния
      if (item.content.classList.contains('active')) {
        this.open(index, true);
      } else {
        this.close(index, true);
      }
    }.bind(this));
  },

```

```

toggle: function(index) {
  if (this.items[index].isOpen) {
    this.close(index);
  } else {
    this.open(index);
  }
},

open: function(index, immediate) {
  if (!this.settings.multiple) {
    this.closeAll();
  }

  var item = this.items[index];
  if (item.isOpen) return;

  item.isOpen = true;
  item.header.classList.add('active');

  if (immediate) {
    item.content.style.display = 'block';
    item.content.classList.add('active');
  } else {
    item.content.style.display = 'block';
    var height = item.content.scrollHeight;
    item.content.style.height = '0px';

    setTimeout(function() {
      item.content.style.height = height + 'px';

      setTimeout(function() {
        item.content.style.height = '';
        item.content.classList.add('active');
      }, this.settings.duration);
    }.bind(this), 10);
  }

  if (typeof this.settings.onOpen === 'function') {
    this.settings.onOpen(index, item);
  }
},

close: function(index, immediate) {
  var item = this.items[index];

```

```

if (!item.isOpen) return;

item.isOpen = false;
item.header.classList.remove('active');

if (immediate) {
  item.content.style.display = 'none';
  item.content.classList.remove('active');
} else {
  var height = item.content.scrollHeight;
  item.content.style.height = height + 'px';

  setTimeout(function() {
    item.content.style.height = '0px';

    setTimeout(function() {
      item.content.style.display = 'none';
      item.content.style.height = '';
      item.content.classList.remove('active');
    }, this.settings.duration);
  }, 10);
}

if (typeof this.settings.onClose === 'function') {
  this.settings.onClose(index, item);
}
},

closeAll: function() {
  this.items.forEach(function(item, index) {
    if (item.isOpen) {
      this.close(index);
    }
  }).bind(this));
},

destroy: function() {
  this.closeAll();
  instances = instances.filter(function(instance) {
    return instance !== this;
  }).bind(this));
}
};

```



```

// Статические методы
Accordion.getAll = function() {
    return instances.slice();
};

Accordion.closeAll = function() {
    instances.forEach(function(instance) {
        instance.closeAll();
    });
};

return Accordion;
})();

// Использование
document.addEventListener('DOMContentLoaded', function() {
    var accordion1 = new Accordion(document.getElementById('accordion1'), {
        duration: 400,
        multiple: true
    });

    var accordion2 = new Accordion(document.getElementById('accordion2'));
});

```

Лучшие практики:

1. Всегда используйте 'use strict':

```

javascript

(function() {
    'use strict';

```

```

// Ваш код

```

```

})();

```

2. Минимизируйте глобальное пространство имен:

```

javascript

```

```

// Плохо

```

```

var module1 = (function() { ... })();
var module2 = (function() { ... })();
var module3 = (function() { ... })();

```

```

// Хорошо

```

```

var MyApp = MyApp || {};
MyApp.Module1 = (function() { ... })();
MyApp.Module2 = (function() { ... })();

```

```
MyApp.Module3 = (function() { ... })();
```

3. Передавайте зависимости явно:

```
javascript
```

```
// Вместо этого (плохо):
```

```
(function() {  
  var $ = window.jQuery; // Неявная зависимость  
})();
```

```
// Делайте так (хорошо):
```

```
(function($, window, document) {  
  // Зависимости переданы явно  
})(jQuery, window, document);
```

4. Используйте неймспейсы для крупных приложений:

```
javascript
```

```
var MyApp = (function() {  
  // Приватные переменные приложения
```

```
  var init = function() {  
    MyApp.Services.init();  
    MyApp.Controllers.init();  
    MyApp.UI.init();  
  };
```

```
  // Публичный интерфейс приложения
```

```
  return {  
    init: init,
```

```
    // Подмодули
```

```
    Services: {},  
    Controllers: {},  
    UI: {},  
    Utils: {}  
  };  
})();
```

```
// Инициализация подмодулей
```

```
MyApp.Services = (function() {  
  // Сервисы  
})();
```

```
MyApp.Controllers = (function() {  
  // Контроллеры  
})();
```

```
MyApp.UI = (function() {  
    // UI компоненты  
})();
```

```
// Запуск приложения
```

```
MyApp.init();
```

5. Документируйте публичный API:

```
javascript  
  
/**  
 * Модуль для работы с пользователями  
 * @namespace UserModule  
 */  
var UserModule = (function() {  
    'use strict';  
  
    /**  
     * Добавляет нового пользователя  
     * @param {Object} userData - Данные пользователя  
     * @param {string} userData.name - Имя пользователя  
     * @param {string} userData.email - Email пользователя  
     * @returns {number} ID созданного пользователя  
     * @throws {Error} Если данные невалидны  
     */  
    function addUser(userData) {  
        // Реализация  
    }  
  
    /**  
     * Возвращает пользователя по ID  
     * @param {number} id - ID пользователя  
     * @returns {Object|null} Объект пользователя или null  
     */  
    function getUser(id) {  
        // Реализация  
    }  
  
    // Публичный API  
    return {  
        add: addUser,  
        get: getUser  
        // ... другие методы  
    };  
});
```

```
})();
```

12.1.5. Ограничения и переход к ES6 модулям

Хотя IIFE и модульный паттерн были революционными для своего времени, у них есть ограничения:

1. **Отсутствие статического анализа** – инструменты не могут определить зависимости.
2. **Нет возможности tree-shaking** – нельзя исключить неиспользуемый код.
3. **Порядок загрузки важен** – зависимости должны быть загружены до модуля.
4. **Нет асинхронной загрузки** (без дополнительных библиотек).
5. **Сложность управления зависимостями** в крупных приложениях.

Эти ограничения привели к появлению:

1. **AMD (Asynchronous Module Definition)** – RequireJS
2. **CommonJS** – модули в Node.js
3. **ES6 Modules** – нативный стандарт в JavaScript

В следующих разделах мы рассмотрим эти современные системы модулей.

Резюме

Паттерны IIFE и модульный паттерн сыграли ключевую роль в эволюции JavaScript, позволив разработчикам организовывать код до появления стандартизированных модулей. Они до сих пор полезны для:

1. **Легаси-проектов**, которые не используют современные сборщики.
2. **Быстрых прототипов** и небольших скриптов.
3. **Изоляции кода** в окружениях, где нельзя использовать ES6 модули.
4. **Создания самодостаточных библиотек**, которые должны работать везде.

Понимание этих паттернов важно не только для работы со старым кодом, но и для глубокого понимания замыканий и инкапсуляции в JavaScript.

Введение: рождение серверного JavaScript

До 2009 года JavaScript был исключительно клиентским языком, выполняемым в браузере. Все изменилось с появлением **Node.js**, созданного Райаном Далем. Node.js позволил выполнять JavaScript на сервере, но для этого потребовалась система модулей, так как:

1. **Серверные приложения сложнее** – требуют организации кода в модули
2. **Нужна работа с файловой системой** – доступ к разным файлам
3. **Требуется управление зависимостями** – использование сторонних библиотек
4. **Необходимость изоляции кода** – предотвращение конфликтов

Так появился **CommonJS** – система модулей, которая стала стандартом для Node.js.

Часть 1: Основы CommonJS

1.1. Что такое CommonJS?

CommonJS – это спецификация модульной системы для JavaScript вне браузера. Основные принципы:

1. **Синхронная загрузка** – модули загружаются синхронно
2. **Кэширование** – модуль загружается один раз при первом `require()`
3. **Изоляция** – каждый модуль имеет свою область видимости
4. **Простота** – минимальный API: `require()` и `module.exports`

1.2. Базовый синтаксис

Экспорт из модуля

javascript

```
// calculator.js - модуль с математическими функциями

// Приватная переменная (не экспортируется)
const PI = 3.141592653589793;

// Приватная функция
function validateNumber(num) {
  return typeof num === 'number' && !isNaN(num);
}

// Экспорт через module.exports
module.exports = {
  // Публичные функции
  add: function(a, b) {
    if (!validateNumber(a) || !validateNumber(b)) {
      throw new Error('Оба аргумента должны быть числами');
    }
    return a + b;
  },

  subtract: function(a, b) {
    if (!validateNumber(a) || !validateNumber(b)) {
      throw new Error('Оба аргумента должны быть числами');
    }
    return a - b;
  },

  multiply: function(a, b) {
    if (!validateNumber(a) || !validateNumber(b)) {
      throw new Error('Оба аргумента должны быть числами');
    }
    return a * b;
  },

  divide: function(a, b) {
    if (!validateNumber(a) || !validateNumber(b)) {
      throw new Error('Оба аргумента должны быть числами');
    }
    if (b === 0) {
      throw new Error('Деление на ноль невозможно');
    }
    return a / b;
  },
}
```

```

// Можно экспортировать константы
PI: PI,

// Или функции-помощники
isEven: function(num) {
    return validateNumber(num) && num % 2 === 0;
},

isOdd: function(num) {
    return validateNumber(num) && num % 2 !== 0;
}
};

// Альтернативный способ экспорта отдельных свойств
// module.exports.add = function(a, b) { ... };
// module.exports.subtract = function(a, b) { ... };

```

Импорт модуля

```

javascript

// app.js - основной файл приложения

// Импорт всего модуля
const calculator = require('./calculator.js');

// Использование импортированных функций
console.log('Сложение:', calculator.add(10, 5)); // 15
console.log('Вычитание:', calculator.subtract(10, 5)); // 5
console.log('Умножение:', calculator.multiply(10, 5)); // 50
console.log('Деление:', calculator.divide(10, 5)); // 2

console.log('Значение PI:', calculator.PI); // 3.141592653589793
console.log('Четное число?', calculator.isEven(4)); // true
console.log('Нечетное число?', calculator.isOdd(4)); // false

// Обработка ошибок
try {
    console.log(calculator.divide(10, 0));
} catch (error) {
    console.error('Ошибка:', error.message); // Деление на ноль невозможно
}

// Приватные переменные модуля недоступны
console.log(calculator.validateNumber); // undefined

```

```
console.log(PI); // ReferenceError: PI is not defined
```

1.3. Различные способы экспорта

Способ 1: Экспорт объекта

```
javascript
// userModule.js

const users = [];
let nextId = 1;

function findUserById(id) {
  return users.find(user => user.id === id);
}

function validateUser(userData) {
  return userData &&
    typeof userData.name === 'string' &&
    userData.name.trim().length > 0 &&
    typeof userData.email === 'string' &&
    userData.email.includes('@');
}

module.exports = {
  createUser(userData) {
    if (!validateUser(userData)) {
      throw new Error('Невалидные данные пользователя');
    }

    const user = {
      id: nextId++,
      name: userData.name.trim(),
      email: userData.email.toLowerCase(),
      createdAt: new Date()
    };

    users.push(user);
    return user;
  },

  getUser(id) {
```



```

    const user = findUserById(id);
    if (!user) {
        throw new Error('Пользователь не найден');
    }
    return { ...user }; // Возвращаем копию
},

getAllUsers() {
    return users.map(user => ({ ...user }));
},

updateUser(id, updates) {
    const user = findUserById(id);
    if (!user) {
        throw new Error('Пользователь не найден');
    }

    if (updates.name && typeof updates.name === 'string') {
        user.name = updates.name.trim();
    }

    if (updates.email && typeof updates.email === 'string') {
        if (!updates.email.includes('@')) {
            throw new Error('Невалидный email');
        }
        user.email = updates.email.toLowerCase();
    }

    user.updatedAt = new Date();
    return { ...user };
},

deleteUser(id) {
    const index = users.findIndex(user => user.id === id);
    if (index === -1) {
        return false;
    }

    users.splice(index, 1);
    return true;
},

getStats() {
    return {

```

```

    total: users.length,
    lastCreated: users.length > 0 ? users[users.length - 1].createdAt : null
  };
}
};

```

Способ 2: Экспорт конструктора/класса

```

javascript

// Logger.js

class Logger {
  constructor(name, options = {}) {
    this.name = name;
    this.level = options.level || 'info';
    this.levels = {
      error: 0,
      warn: 1,
      info: 2,
      debug: 3
    };

    this.transports = options.transports || [];
    this.filters = options.filters || [];
  }

  log(level, message, meta = {}) {
    if (this.levels[level] > this.levels[this.level]) {
      return;
    }

    // Применяем фильтры
    for (const filter of this.filters) {
      if (!filter(level, message, meta)) {
        return;
      }
    }

    const logEntry = {
      timestamp: new Date().toISOString(),
      level: level,
      logger: this.name,
      message: message,
      meta: meta
    };
  }
}

```

```
};

// Отправляем во все транспорты
for (const transport of this.transports) {
  transport(logEntry);
}

// Вывод в консоль (по умолчанию)
const colors = {
  error: '\x1b[31m', // красный
  warn: '\x1b[33m',  // желтый
  info: '\x1b[32m',  // зеленый
  debug: '\x1b[36m'  // голубой
};

const reset = '\x1b[0m';
console.log(
  `${colors[level]} || '['[${logEntry.timestamp}] ${level.toUpperCase()} ${this.name}: ${
    message}${reset}`
);
}

error(message, meta) {
  this.log('error', message, meta);
}

warn(message, meta) {
  this.log('warn', message, meta);
}

info(message, meta) {
  this.log('info', message, meta);
}

debug(message, meta) {
  this.log('debug', message, meta);
}

addTransport(transport) {
  this.transports.push(transport);
}

addFilter(filter) {
  this.filters.push(filter);
}
```

```

}

setLevel(level) {
  if (!this.levels.hasOwnProperty(level)) {
    throw new Error(`Неизвестный уровень логирования: ${level}`);
  }
  this.level = level;
}
}

// Экспорт класса
module.exports = Logger;

// Также можно экспортировать вспомогательные функции
module.exports.createConsoleTransport = function(options = {}) {
  return function(logEntry) {
    const output = options.format
      ? options.format(logEntry)
      : `${logEntry.timestamp} [${logEntry.level}] ${logEntry.message}`;

    if (logEntry.level === 'error') {
      console.error(output);
    } else {
      console.log(output);
    }
  };
};

module.exports.createFileTransport = function(filename) {
  const fs = require('fs');
  const path = require('path');

  return function(logEntry) {
    const logLine = JSON.stringify(logEntry) + '\n';
    fs.appendFileSync(filename, logLine, 'utf8');
  };
};

```

Способ 3: Экспорт функции

```

javascript

// database.js

const mysql = require('mysql2/promise');

```

```

module.exports = async function createConnection(config) {
  // Валидация конфигурации
  const requiredFields = ['host', 'user', 'password', 'database'];
  for (const field of requiredFields) {
    if (!config[field]) {
      throw new Error(`Не указано обязательное поле: ${field}`);
    }
  }

  // Создание пула соединений
  const pool = mysql.createPool({
    host: config.host,
    user: config.user,
    password: config.password,
    database: config.database,
    waitForConnections: true,
    connectionLimit: config.connectionLimit || 10,
    queueLimit: config.queueLimit || 0,
    enableKeepAlive: true,
    keepAliveInitialDelay: config.keepAliveInitialDelay || 0
  });

  // Тестирование соединения
  try {
    const connection = await pool.getConnection();
    console.log('✅ Успешное подключение к базе данных');
    connection.release();
  } catch (error) {
    console.error('✖ Ошибка подключения к базе данных:', error.message);
    await pool.end();
    throw error;
  }

  // Возвращаем объект с методами для работы с БД
  return {
    pool,

    async query(sql, params = []) {
      try {
        const [rows, fields] = await pool.execute(sql, params);
        return { rows, fields };
      } catch (error) {
        console.error('Ошибка выполнения запроса:', error.message);
      }
    }
  };
}

```

```

    throw error;
  }
},

async transaction(callback) {
  const connection = await pool.getConnection();

  try {
    await connection.beginTransaction();
    const result = await callback(connection);
    await connection.commit();
    return result;
  } catch (error) {
    await connection.rollback();
    throw error;
  } finally {
    connection.release();
  }
},

async close() {
  await pool.end();
  console.log('Соединение с базой данных закрыто');
},

// Статистика
getStats() {
  return {
    pool: pool.pool ? pool.pool : {},
    config: {
      host: config.host,
      database: config.database,
      connectionLimit: config.connectionLimit || 10
    }
  };
}
};
};

```

Способ 4: Экспорт значения

```

javascript
// config.js

```

// Экспорт простого значения

```
module.exports = {
  app: {
    name: 'My Node.js App',
    version: '1.0.0',
    port: process.env.PORT || 3000,
    env: process.env.NODE_ENV || 'development'
  },

  database: {
    host: process.env.DB_HOST || 'localhost',
    port: process.env.DB_PORT || 5432,
    name: process.env.DB_NAME || 'myapp',
    user: process.env.DB_USER || 'postgres',
    password: process.env.DB_PASSWORD || 'password',

    get connectionString() {
      return `postgresql://${this.user}:${this.password}@${this.host}:${this.port}/${this.name}`;
    }
  },

  api: {
    baseUrl: process.env.API_BASE_URL || 'https://api.example.com',
    timeout: parseInt(process.env.API_TIMEOUT || '10000'),
    retries: parseInt(process.env.API_RETRIES || '3')
  },

  security: {
    jwtSecret: process.env.JWT_SECRET || 'your-secret-key-change-in-production',
    bcryptRounds: parseInt(process.env.BCRYPT_ROUNDS || '10'),
    sessionTimeout: parseInt(process.env.SESSION_TIMEOUT || '3600000') // 1 час
  },

  logging: {
    level: process.env.LOG_LEVEL || 'info',
    format: process.env.LOG_FORMAT || 'json',
    directory: process.env.LOG_DIR || './logs'
  },

  isProduction() {
    return this.app.env === 'production';
  },
}
```

```

isDevelopment() {
  return this.app.env === 'development';
},

isTest() {
  return this.app.env === 'test';
}
};

// Использование
// const config = require('./config');
// console.log(config.app.name); // "My Node.js App"
// console.log(config.isProduction()); // true/false

```

1.4. Различные способы импорта

Импорт всего модуля

```

javascript

// app.js

const path = require('path');
const fs = require('fs');
const express = require('express');
const config = require('./config');

// Использование
const app = express();

app.get('/', (req, res) => {
  res.send(`Добро пожаловать в ${config.app.name} v${config.app.version}`);
});

app.listen(config.app.port, () => {
  console.log(`Сервер запущен на порту ${config.app.port}`);
});

```

Деструктуризация при импорте

```

javascript

// utils.js

```



```

module.exports = {
  formatDate(date) {
    return new Intl.DateTimeFormat('ru-RU').format(date);
  },

  formatCurrency(amount, currency = 'RUB') {
    return new Intl.NumberFormat('ru-RU', {
      style: 'currency',
      currency: currency
    }).format(amount);
  },

  truncateText(text, maxLength = 100) {
    if (text.length <= maxLength) return text;
    return text.substring(0, maxLength - 3) + '...';
  },

  generateId() {
    return 'id_' + Date.now() + '_' + Math.random().toString(36).substr(2, 9);
  },

  sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
  }
};

// main.js - деструктуризация при импорте
const { formatDate, formatCurrency, generateId, sleep } = require('./utils');

console.log(formatDate(new Date())); // "01.01.2023"
console.log(formatCurrency(1234.56)); // "1 234,56 ₽"
console.log(generateId()); // "id_1672531200000_abc123def"

// Использование sleep с async/await
(async () => {
  console.log('Начало');
  await sleep(1000);
  console.log('Прошла 1 секунда');
})();

```

Импорт с переименованием

```

javascript
// statistics.js

```

```

module.exports = {
  calculateMean(numbers) {
    const sum = numbers.reduce((a, b) => a + b, 0);
    return sum / numbers.length;
  },

  calculateMedian(numbers) {
    const sorted = [...numbers].sort((a, b) => a - b);
    const middle = Math.floor(sorted.length / 2);

    if (sorted.length % 2 === 0) {
      return (sorted[middle - 1] + sorted[middle]) / 2;
    }

    return sorted[middle];
  },

  calculateMode(numbers) {
    const frequency = {};
    let maxFreq = 0;
    let mode = [];

    numbers.forEach(num => {
      frequency[num] = (frequency[num] || 0) + 1;

      if (frequency[num] > maxFreq) {
        maxFreq = frequency[num];
        mode = [num];
      } else if (frequency[num] === maxFreq) {
        mode.push(num);
      }
    });

    return mode.length === numbers.length ? [] : [...new Set(mode)];
  },

  calculateStandardDeviation(numbers) {
    const mean = this.calculateMean(numbers);
    const squaredDiffs = numbers.map(num => Math.pow(num - mean, 2));
    const variance = this.calculateMean(squaredDiffs);
    return Math.sqrt(variance);
  }
};

```

```

// analysis.js - импорт с переименованием
const stats = require('./statistics');

// Или деструктуризация с переименованием
const {
  calculateMean: mean,
  calculateMedian: median,
  calculateMode: mode,
  calculateStandardDeviation: stdDev
} = require('./statistics');

const data = [1, 2, 2, 3, 4, 5, 5, 5, 6];

console.log('Среднее:', mean(data));           // 3.666...
console.log('Медиана:', median(data));         // 4
console.log('Мода:', mode(data));              // [5]
console.log('Стандартное отклонение:', stdDev(data)); // 1.658...

// Использование через объект stats
console.log('Среднее (через объект):', stats.calculateMean(data));

```

Импорт и выполнение кода модуля

```

javascript

// init.js - модуль, который выполняет код при импорте

console.log('□ Инициализация модуля init.js');

// Глобальная конфигурация
global.appConfig = {
  startedAt: new Date(),
  version: '1.0.0',
  environment: process.env.NODE_ENV || 'development'
};

// Установка обработчиков событий
process.on('SIGINT', () => {
  console.log('Получен сигнал SIGINT, завершение...');
  process.exit(0);
});

process.on('SIGTERM', () => {
  console.log('Получен сигнал SIGTERM, завершение...');

```

```
process.exit(0);
});

// Функция для экспорта
module.exports = {
  getUptime() {
    return Date.now() - global.appConfig.startedAt;
  },

  getInfo() {
    return {
      ...global.appConfig,
      uptime: this.getUptime(),
      nodeVersion: process.version,
      platform: process.platform,
      memoryUsage: process.memoryUsage()
    };
  }
};

console.log('□ Модуль init.js инициализирован');

// main.js
console.log('□ Запуск приложения');
require('./init'); // Код выполнится сразу

// Через некоторое время
setTimeout(() => {
  const initModule = require('./init'); // Модуль закэширован, код не выполнится снова
  console.log('Информация о приложении:', initModule.getInfo());
}, 2000);
```

Часть 2: Особенности и внутреннее устройство

2.1. Кэширование модулей

Node.js кэширует модули после первой загрузки. Это важно понимать для оптимизации и предотвращения ошибок.

```
// counter.js
let count = 0;

module.exports = {
  increment() {
    return ++count;
  },

  decrement() {
    return --count;
  },

  getCount() {
    return count;
  },

  reset() {
    count = 0;
  }
};

// app.js
const counter1 = require('./counter');
const counter2 = require('./counter'); // Тот же экземпляр!

console.log(counter1 === counter2); // true - один и тот же объект

counter1.increment();
console.log(counter1.getCount()); // 1
console.log(counter2.getCount()); // 1 - то же значение!

counter2.increment();
console.log(counter1.getCount()); // 2

// Удаление из кэша (редко используется)
delete require.cache[require.resolve('./counter')];
const counter3 = require('./counter'); // Новый экземпляр
console.log(counter3.getCount()); // 0
```

2.2. Циклические зависимости

CommonJS обрабатывает циклические зависимости, но это может привести к неожиданному поведению.

```
javascript

// a.js
console.log('a: начало');
exports.done = false;

const b = require('./b.js');
console.log('a: b.done =', b.done);

exports.done = true;
console.log('a: завершение');

// b.js
console.log('b: начало');
exports.done = false;

const a = require('./a.js');
console.log('b: a.done =', a.done);

exports.done = true;
console.log('b: завершение');

// main.js
console.log('main: начало');
const a = require('./a.js');
const b = require('./b.js');
console.log('main: a.done =', a.done, 'b.done =', b.done);

// Вывод:
// main: начало
// a: начало
// b: начало
// b: a.done = false (a еще не завершила выполнение)
// b: завершение
// a: b.done = true (b уже завершил выполнение)
// a: завершение
// main: a.done = true b.done = true
```

2.3. Объекты module, exports, require, __filename, __dirname

Каждый CommonJS модуль имеет доступ к специальным объектам:

```
javascript

// module-info.js

// Вывод информации о текущем модуле
console.log('=== Информация о модуле ===\n');

// module - объект текущего модуля
console.log('module.id:', module.id);
console.log('module.filename:', module.filename);
console.log('module.loaded:', module.loaded);
console.log('module.parent:', module.parent);
console.log('module.children:', module.children);
console.log('module.paths:', module.paths);

// exports - ссылка на module.exports
console.log('\nexports === module.exports:', exports === module.exports);

// require - функция для импорта модулей
console.log('\nrequire.main:', require.main === module);
console.log('require.cache:', Object.keys(require.cache));

// __filename - абсолютный путь к текущему файлу
console.log('\n__filename:', __filename);

// __dirname - абсолютный путь к директории текущего файла
console.log('__dirname:', __dirname);

// Путь к корневой директории проекта
const path = require('path');
const projectRoot = path.resolve(__dirname, '..');
console.log('projectRoot:', projectRoot);

// Экспорт информации
module.exports = {
  getModuleInfo() {
    return {
      filename: __filename,
      dirname: __dirname,
      id: module.id,
```

```

    parent: module.parent ? module.parent.filename : null,
    children: module.children.map(child => child.filename),
    cacheKeys: Object.keys(require.cache)
  };
},

clearModuleCache(modulePath) {
  const resolvedPath = require.resolve(modulePath);
  delete require.cache[resolvedPath];
  return true;
}
};

```

2.4. Разрешение путей модулей

Node.js ищет модули в определенном порядке:

```

javascript

// При require('./something'):

// 1. Проверяет путь относительно текущей директории
//   ./something.js
//   ./something.json
//   ./something.node (нативные модули)

// 2. Если путь начинается с './' или '../' - ищет как файл
//   something.js
//   something.json
//   something.node

// 3. Если не найден как файл - ищет как директорию
//   something/package.json (main field)
//   something/index.js
//   something/index.json
//   something/index.node

// 4. Если модуль не относительный (без './' или '../')
//   Ищет в node_modules
//   Ищет в глобальных модулях
//   Проверяет встроенные модули Node.js

// Пример демонстрации:

```



```
module.exports = {
  explainResolution(moduleName) {
    const Module = require('module');
    const paths = Module._nodeModulePaths(__dirname);

    return {
      moduleName,
      searchPaths: paths,
      resolvedPath: require.resolve(moduleName, { paths: [__dirname] })
    };
  }
};
```

Часть 3: Практические паттерны

3.1. Фабричный паттерн

```
javascript

// cache-factory.js

const caches = new Map();

module.exports = function createCache(options = {}) {
  const {
    maxSize = 100,
    ttl = 0, // Time to Live в миллисекундах, 0 = бесконечно
    cleanupInterval = 60000 // Интервал очистки устаревших записей
  } = options;

  const cache = new Map();
  const timestamps = new Map();

  // Очистка устаревших записей
  let cleanupTimer;
  if (ttl > 0) {
    cleanupTimer = setInterval(() => {
      const now = Date.now();
      for (const [key, timestamp] of timestamps.entries()) {
        if (now - timestamp > ttl) {
          cache.delete(key);
        }
      }
    }, cleanupInterval);
  }
}
```

```

        timestamps.delete(key);
    }
}, cleanupInterval);
}

const cacheInstance = {
  set(key, value) {
    // Удаляем самую старую запись если достигли лимита
    if (cache.size >= maxSize) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey);
      timestamps.delete(firstKey);
    }

    cache.set(key, value);
    if (ttl > 0) {
      timestamps.set(key, Date.now());
    }
    return true;
  },

  get(key) {
    if (!cache.has(key)) {
      return undefined;
    }

    // Проверяем TTL
    if (ttl > 0) {
      const timestamp = timestamps.get(key);
      if (Date.now() - timestamp > ttl) {
        cache.delete(key);
        timestamps.delete(key);
        return undefined;
      }

      // Обновляем время доступа (optional)
      timestamps.set(key, Date.now());
    }

    return cache.get(key);
  },

  has(key) {

```

```
    const value = this.get(key);
    return value !== undefined;
  },

  delete(key) {
    timestamps.delete(key);
    return cache.delete(key);
  },

  clear() {
    cache.clear();
    timestamps.clear();
  },

  size() {
    return cache.size;
  },

  keys() {
    return Array.from(cache.keys());
  },

  values() {
    return Array.from(cache.values());
  },

  entries() {
    return Array.from(cache.entries());
  },

  stats() {
    return {
      size: cache.size,
      maxSize,
      ttl,
      hitRate: this.hits / Math.max(this.hits + this.misses, 1)
    };
  },

  // Мемпуку
  hits: 0,
  misses: 0,

  destroy() {
```

```

    if (cleanupTimer) {
      clearInterval(cleanupTimer);
    }
    this.clear();
    caches.delete(cacheInstance);
  }
};

// Переопределяем get для подсчета метрик
const originalGet = cacheInstance.get;
cacheInstance.get = function(key) {
  const value = originalGet.call(this, key);
  if (value !== undefined) {
    this.hits++;
  } else {
    this.misses++;
  }
  return value;
};

caches.set(cacheInstance, options);
return cacheInstance;
};

// Статические методы
module.exports.getAllCaches = function() {
  return Array.from(caches.entries()).map(([cache, options]) => ({
    stats: cache.stats(),
    options
  }));
};

module.exports.clearAllCaches = function() {
  for (const [cache] of caches.entries()) {
    cache.destroy();
  }
  caches.clear();
};

```

3.2. Паттерн Singleton

javascript

// app-state.js

```
class AppState {
  constructor() {
    if (AppState.instance) {
      return AppState.instance;
    }

    this.state = new Map();
    this.listeners = new Map();
    this.history = [];
    this.maxHistory = 100;

    AppState.instance = this;
  }

  set(key, value, silent = false) {
    const oldValue = this.state.get(key);
    this.state.set(key, value);

    // Сохраняем в историю
    this.history.push({
      timestamp: new Date(),
      action: 'SET',
      key,
      oldValue,
      newValue: value
    });

    // Ограничиваем размер истории
    if (this.history.length > this.maxHistory) {
      this.history.shift();
    }

    // Уведомляем слушателей
    if (!silent) {
      this.notifyListeners(key, value, oldValue);
    }

    return this;
  }

  get(key, defaultValue = null) {
    return this.state.has(key) ? this.state.get(key) : defaultValue;
  }
}
```

```
has(key) {
  return this.state.has(key);
}

delete(key) {
  const oldValue = this.state.get(key);
  const result = this.state.delete(key);

  if (result) {
    this.history.push({
      timestamp: new Date(),
      action: 'DELETE',
      key,
      oldValue
    });

    this.notifyListeners(key, undefined, oldValue);
  }

  return result;
}

clear() {
  const oldState = new Map(this.state);
  this.state.clear();

  this.history.push({
    timestamp: new Date(),
    action: 'CLEAR',
    oldState
  });

  // Уведомляем всех слушателей
  for (const [key, oldValue] of oldState) {
    this.notifyListeners(key, undefined, oldValue);
  }

  return this;
}

subscribe(key, callback) {
  if (!this.listeners.has(key)) {
    this.listeners.set(key, new Set());
  }
}
```

```

}

this.listeners.get(key).add(callback);

// Возвращаем функцию отписки
return () => {
  const listeners = this.listeners.get(key);
  if (listeners) {
    listeners.delete(callback);
    if (listeners.size === 0) {
      this.listeners.delete(key);
    }
  }
};
}

subscribeAll(callback) {
  return this.subscribe('*', callback);
}

notifyListeners(key, newValue, oldValue) {
  // Уведомляем конкретных слушателей
  const specificListeners = this.listeners.get(key);
  if (specificListeners) {
    specificListeners.forEach(callback => {
      try {
        callback(newValue, oldValue, key);
      } catch (error) {
        console.error(`Ошибка в слушателе для ключа "${key}":`, error);
      }
    });
  }

  // Уведомляем общих слушателей
  const allListeners = this.listeners.get('*');
  if (allListeners) {
    allListeners.forEach(callback => {
      try {
        callback(key, newValue, oldValue);
      } catch (error) {
        console.error('Ошибка в общем слушателе:', error);
      }
    });
  }
}

```

```

}

getState() {
  return Object.fromEntries(this.state);
}

getHistory() {
  return [...this.history];
}

getSnapshot() {
  return {
    state: this.getState(),
    history: this.getHistory(),
    listeners: Array.from(this.listeners.keys())
  };
}

restoreSnapshot(snapshot) {
  this.state.clear();
  for (const [key, value] of Object.entries(snapshot.state || {})) {
    this.state.set(key, value);
  }

  this.history = [...(snapshot.history || [])];
  return this;
}

// Экспортируем Singleton
module.exports = new AppState();

// Также экспортируем класс для тестирования
module.exports.AppStateClass = AppState;

```

3.3. Паттерн Middleware/Плагин

```

javascript

// plugin-system.js

class PluginSystem {
  constructor() {
    this.plugins = new Map();
  }

```



```

    this.hooks = new Map();
    this.middlewares = new Map();
    this.context = {};
}

// Регистрация плагинов
register(name, plugin) {
    if (this.plugins.has(name)) {
        throw new Error(`Плагин "${name}" уже зарегистрирован`);
    }

    // Инициализация плагина
    const pluginInstance = typeof plugin === 'function'
        ? plugin(this.context, this)
        : plugin;

    this.plugins.set(name, {
        instance: pluginInstance,
        hooks: new Set(),
        middlewares: new Set()
    });

    // Регистрация хуков плагина
    if (pluginInstance.hooks) {
        for (const [hookName, handler] of Object.entries(pluginInstance.hooks)) {
            this.registerHook(hookName, handler, name);
        }
    }

    // Регистрация middleware плагина
    if (pluginInstance.middlewares) {
        for (const [middlewareName, middleware] of Object.entries(pluginInstance.middlewares)) {
            this.registerMiddleware(middlewareName, middleware, name);
        }
    }

    // Вызов хука инициализации
    if (pluginInstance.init) {
        pluginInstance.init();
    }

    console.log(`📌 Плагин "${name}" зарегистрирован`);
    return this;
}

```

```
}
```

```
// Регистрация хуков
```

```
registerHook(hookName, handler, pluginName) {  
  if (!this.hooks.has(hookName)) {  
    this.hooks.set(hookName, []);  
  }  
  
  this.hooks.get(hookName).push({  
    handler,  
    pluginName  
  });  
  
  if (pluginName) {  
    this.plugins.get(pluginName).hooks.add(hookName);  
  }  
  
  return this;  
}
```

```
// Вызов хуков
```

```
async callHook(hookName, ...args) {  
  if (!this.hooks.has(hookName)) {  
    return [];  
  }  
  
  const results = [];  
  const hooks = this.hooks.get(hookName);  
  
  for (const hook of hooks) {  
    try {  
      const result = await hook.handler(...args);  
      results.push({  
        pluginName: hook.pluginName,  
        result,  
        success: true  
      });  
    } catch (error) {  
      results.push({  
        pluginName: hook.pluginName,  
        error: error.message,  
        success: false  
      });  
      console.error(`Ошибка в хуке "${hookName}" плагина "${hook.pluginName}":`, error);  
    }  
  }  
}
```

```

    }
  }

  return results;
}

// Регистрация middleware
registerMiddleware(name, middleware, pluginName) {
  if (!this.middlewares.has(name)) {
    this.middlewares.set(name, []);
  }

  this.middlewares.get(name).push({
    middleware,
    pluginName
  });

  if (pluginName) {
    this.plugins.get(pluginName).middlewares.add(name);
  }

  return this;
}

// Выполнение middleware цепочки
async executeMiddleware(name, initialValue, ...extraArgs) {
  if (!this.middlewares.has(name)) {
    return initialValue;
  }

  let value = initialValue;
  const middlewares = this.middlewares.get(name);

  for (const { middleware, pluginName } of middlewares) {
    try {
      value = await middleware(value, ...extraArgs);
    } catch (error) {
      console.error(`Ошибка в middleware "${name}" плагина "${pluginName}":`, error);
      throw error;
    }
  }

  return value;
}

```

// Получение плагина

```
getPlugin(name) {  
  const plugin = this.plugins.get(name);  
  return plugin ? plugin.instance : null;  
}
```

// Получение всех плагинов

```
getAllPlugins() {  
  return Array.from(this.plugins.entries()).map(([name, plugin]) => ({  
    name,  
    hooks: Array.from(plugin.hooks),  
    middlewares: Array.from(plugin.middlewares)  
  }));  
}
```

// Отключение плагина

```
disablePlugin(name) {  
  const plugin = this.plugins.get(name);  
  if (!plugin) {  
    throw new Error(`Плагин "${name}" не найден`);  
  }  
}
```

// Вызываем деструктор если есть

```
if (plugin.instance.destroy) {  
  plugin.instance.destroy();  
}
```

// Удаляем хуки плагина

```
for (const hookName of plugin.hooks) {  
  const hooks = this.hooks.get(hookName);  
  if (hooks) {  
    const filtered = hooks.filter(h => h.pluginName !== name);  
    if (filtered.length === 0) {  
      this.hooks.delete(hookName);  
    } else {  
      this.hooks.set(hookName, filtered);  
    }  
  }  
}
```

// Удаляем middleware плагина

```
for (const middlewareName of plugin.middlewares) {  
  const middlewares = this.middlewares.get(middlewareName);
```

```

    if (middlewares) {
      const filtered = middlewares.filter(m => m.pluginName !== name);
      if (filtered.length === 0) {
        this.middlewares.delete(middlewareName);
      } else {
        this.middlewares.set(middlewareName, filtered);
      }
    }
  }

  this.plugins.delete(name);
  console.log(`🔴 Плагин "${name}" отключен`);

  return this;
}

// Установка контекста
setContext(context) {
  this.context = { ...this.context, ...context };
  return this;
}

// Статистика
getStats() {
  return {
    plugins: this.plugins.size,
    hooks: this.hooks.size,
    middlewares: this.middlewares.size,
    totalHookHandlers: Array.from(this.hooks.values()).reduce((sum, hooks) => sum +
hooks.length, 0),
    totalMiddlewares: Array.from(this.middlewares.values()).reduce((sum, middlewares) =>
sum + middlewares.length, 0)
  };
}

// Экспорт системы плагинов
module.exports = PluginSystem;

// Экспорт вспомогательных функций
module.exports.createPlugin = function(name, definition) {
  return {
    name,
    ...definition
  };
};

```

```

};
};

// Пример плагина для логирования
module.exports.loggingPlugin = {
  name: 'logging',

  hooks: {
    'request:start': function(url, method) {
      console.log(`[${new Date().toISOString()}] 🚀 ${method} ${url}`);
    },

    'request:end': function(url, method, status, duration) {
      console.log(`[${new Date().toISOString()}] 🏠 ${method} ${url} - ${status} (${duration}ms)`);
    },

    'error': function(error, context) {
      console.error(`[${new Date().toISOString()}] ❌ Ошибка:`, error.message, context);
    }
  },

  middlewares: {
    'log:request': function(request, next) {
      const start = Date.now();
      console.log(`Начало обработки: ${request.method} ${request.url}`);

      return next(request).then(response => {
        const duration = Date.now() - start;
        console.log(`Завершено за ${duration}ms: ${response.status}`);
        return response;
      });
    }
  },

  init() {
    console.log('Плагин логирования инициализирован');
  },

  setLogLevel(level) {
    this.logLevel = level;
  }
};

```

3.4. Конфигурация через environment variables

javascript

// env-config.js

```
const path = require('path');
```

```
const fs = require('fs');
```

```
class EnvConfig {
```

```
  constructor() {
```

```
    this.config = new Map();
```

```
    this.sources = [];
```

```
    this.watchers = new Map();
```

```
    this.validators = new Map();
```

```
    this.defaults = new Map();
```

```
    this.loadDefaults();
```

```
    this.loadEnvironment();
```

```
    this.loadDotEnv();
```

```
  }
```

// Загрузка значений по умолчанию

```
loadDefaults() {
```

```
  this.defaults.set('NODE_ENV', 'development');
```

```
  this.defaults.set('PORT', 3000);
```

```
  this.defaults.set('LOG_LEVEL', 'info');
```

```
  this.defaults.set('DB_HOST', 'localhost');
```

```
  this.defaults.set('DB_PORT', 5432);
```

```
  this.defaults.set('SESSION_SECRET', 'change-this-in-production');
```

```
  this.defaults.set('CORS_ORIGIN', 'http://localhost:3000');
```

// Устанавливаем значения по умолчанию

```
for (const [key, value] of this.defaults) {
```

```
  if (!this.config.has(key)) {
```

```
    this.config.set(key, value);
```

```
    this.sources.push({ key, value, source: 'default' });
```

```
  }
```

```
}
```

```
}
```

// Загрузка из process.env

```
loadEnvironment() {
```

```
  for (const [key, value] of Object.entries(process.env)) {
```

```

    if (value !== undefined) {
      const parsed = this.parseValue(value);
      this.config.set(key, parsed);
      this.sources.push({ key, value: parsed, source: 'environment' });
    }
  }
}

```

// Загрузка из .env файла

```

loadDotEnv() {
  const envPaths = [
    '.env.local',
    `.env.${process.env.NODE_ENV || 'development'}.local`,
    `.env.${process.env.NODE_ENV || 'development'}`,
    '.env'
  ];

  for (const envPath of envPaths) {
    const fullPath = path.resolve(process.cwd(), envPath);

    if (fs.existsSync(fullPath)) {
      try {
        const content = fs.readFileSync(fullPath, 'utf8');
        const lines = content.split('\n');

        for (const line of lines) {
          const trimmed = line.trim();

          // Пропускаем комментарии и пустые строки
          if (!trimmed || trimmed.startsWith('#')) {
            continue;
          }

          // Парсим KEY=VALUE
          const equalsIndex = trimmed.indexOf('=');
          if (equalsIndex === -1) {
            continue;
          }

          const key = trimmed.substring(0, equalsIndex).trim();
          let value = trimmed.substring(equalsIndex + 1).trim();

          // Удаляем кавычки если есть
          if ((value.startsWith('"') && value.endsWith('"')) ||

```



```

    (value.startsWith('"') && value.endsWith('"')) {
    value = value.substring(1, value.length - 1);
    }

    const parsed = this.parseValue(value);

    // Не перезаписываем значения из process.env
    if (!process.env.hasOwnProperty(key)) {
    this.config.set(key, parsed);
    this.sources.push({
    key,
    value: parsed,
    source: `file:${envPath}`
    });
    }
}

console.log(`📄 Загружен файл конфигурации: ${envPath}`);

// Наблюдение за изменениями файла (только в development)
if (this.get('NODE_ENV') === 'development') {
    this.watchFile(fullPath);
}
} catch (error) {
    console.error(`❌ Ошибка загрузки файла ${envPath}:`, error.message);
}
}
}

// Парсинг значений
parseValue(value) {
    if (value === 'true' || value === 'TRUE') return true;
    if (value === 'false' || value === 'FALSE') return false;
    if (value === 'null' || value === 'NULL') return null;

    // Число
    if (/^-?\d+$/i.test(value)) return parseInt(value, 10);
    if (/^-?\d+\.\d+$/i.test(value)) return parseFloat(value);

    // Массив (через запятую)
    if (value.includes(',')) {
    return value.split(',').map(item => this.parseValue(item.trim()));
    }
}

```

```

// JSON
if ((value.startsWith('{') && value.endsWith('}')) ||
    (value.startsWith('[') && value.endsWith(']'))) {
    try {
        return JSON.parse(value);
    } catch {
        // Если невалидный JSON, возвращаем как строку
    }
}

return value;
}

// Наблюдение за файлом
watchFile(filePath) {
    if (this.watchers.has(filePath)) {
        return;
    }

    try {
        const watcher = fs.watch(filePath, (eventType) => {
            if (eventType === 'change') {
                console.log(`🔄 Файл конфигурации изменен: ${path.basename(filePath)}`);
                this.reloadFile(filePath);
            }
        });

        this.watchers.set(filePath, watcher);
    } catch (error) {
        console.error(`❌ Ошибка наблюдения за файлом ${filePath}:`, error.message);
    }
}

// Перезагрузка файла
reloadFile(filePath) {
    const oldKeys = new Set(
        this.sources
            .filter(s => s.source === `file:${path.basename(filePath)}`)
            .map(s => s.key)
    );

    // Удаляем старые значения из этого файла
    for (const key of oldKeys) {

```

```

    this.config.delete(key);
}

// Удаляем из sources
this.sources = this.sources.filter(s => !oldKeys.has(s.key));

// Загружаем заново
try {
    const content = fs.readFileSync(filePath, 'utf8');
    const lines = content.split('\n');

    for (const line of lines) {
        const trimmed = line.trim();

        if (!trimmed || trimmed.startsWith('#')) continue;

        const equalsIndex = trimmed.indexOf('=');
        if (equalsIndex === -1) continue;

        const key = trimmed.substring(0, equalsIndex).trim();
        let value = trimmed.substring(equalsIndex + 1).trim();

        if ((value.startsWith('"') && value.endsWith('"')) ||
            (value.startsWith("'") && value.endsWith("'"))) {
            value = value.substring(1, value.length - 1);
        }

        const parsed = this.parseValue(value);

        if (!process.env.hasOwnProperty(key)) {
            this.config.set(key, parsed);
            this.sources.push({
                key,
                value: parsed,
                source: `file:${path.basename(filePath)}`
            });
        }
    }

    console.log(`📄 Файл конфигурации перезагружен: ${path.basename(filePath)}`);
} catch (error) {
    console.error(`❌ Ошибка перезагрузки файла ${filePath}:`, error.message);
}
}

```

// Получение значения

```
get(key, defaultValue = null) {  
  const value = this.config.has(key)  
    ? this.config.get(key)  
    : defaultValue !== null  
      ? defaultValue  
      : this.defaults.get(key);
```

// Валидация если есть валидатор

```
if (this.validators.has(key)) {  
  const validator = this.validators.get(key);  
  const validationResult = validator(value, key);  
  
  if (validationResult !== true) {  
    throw new Error(`Невалидное значение для ${key}: ${validationResult}`);  
  }  
}  
  
return value;  
}
```

// Получение с приведением типа

```
getString(key, defaultValue = '') {  
  return String(this.get(key, defaultValue));  
}
```

```
getNumber(key, defaultValue = 0) {  
  const value = this.get(key, defaultValue);  
  const num = Number(value);  
  return isNaN(num) ? defaultValue : num;  
}
```

```
getBoolean(key, defaultValue = false) {  
  const value = this.get(key, defaultValue);  
  return Boolean(value);  
}
```

```
getArray(key, defaultValue = []) {  
  const value = this.get(key, defaultValue);  
  return Array.isArray(value) ? value : [value];  
}
```

// Проверка существования

```

has(key) {
  return this.config.has(key) || this.defaults.has(key);
}

// Установка валидатора
validate(key, validator) {
  this.validators.set(key, validator);

  // Проверяем текущее значение
  if (this.has(key)) {
    const value = this.get(key);
    const result = validator(value, key);
    if (result !== true) {
      console.warn(`▲ Текущее значение ${key} невалидно: ${result}`);
    }
  }

  return this;
}

// Встроенные валидаторы
require(key, message = null) {
  return this.validate(key, (value) => {
    if (value === undefined || value === null || value === '') {
      return message || `Параметр ${key} обязателен`;
    }
    return true;
  });
}

isOneOf(key, allowedValues, message = null) {
  return this.validate(key, (value) => {
    if (!allowedValues.includes(value)) {
      return message || `Параметр ${key} должен быть одним из: ${allowedValues.join(', ')}`;
    }
    return true;
  });
}

isNumber(key, message = null) {
  return this.validate(key, (value) => {
    if (isNaN(Number(value))) {
      return message || `Параметр ${key} должен быть числом`;
    }
  });
}

```

```
    }  
    return true;  
  });  
}  
  
isUrl(key, message = null) {  
  return this.validate(key, (value) => {  
    try {  
      new URL(value);  
      return true;  
    } catch {  
      return message || `Параметр ${key} должен быть валидным URL`;  
    }  
  });  
}
```

// Получение всей конфигурации

```
getAll() {  
  const result = {};  
  for (const [key, value] of this.config) {  
    result[key] = value;  
  }  
  return result;  
}
```

// Получение источников

```
getSources() {  
  return [...this.sources];  
}
```

// Проверка окружения

```
isProduction() {  
  return this.get('NODE_ENV') === 'production';  
}
```

```
isDevelopment() {  
  return this.get('NODE_ENV') === 'development';  
}
```

```
isTest() {  
  return this.get('NODE_ENV') === 'test';  
}
```

// Очистка

```
cleanup() {
  for (const watcher of this.watchers.values()) {
    watcher.close();
  }
  this.watchers.clear();
}
}

// Экспорт singleton instance
const config = new EnvConfig();

// Также экспортируем класс для тестирования
module.exports = config;
module.exports.EnvConfig = EnvConfig;

// Экспорт вспомогательных функций
module.exports.loadConfig = function(options = {}) {
  return new EnvConfig(options);
};
```

Часть 4: Работа с npm и package.json

4.1. Структура package.json

```
json
{
  "name": "my-commonjs-app",
  "version": "1.0.0",
  "description": "Пример приложения на CommonJS модулях",
  "main": "src/index.js",
  "type": "commonjs", // Явное указание типа модулей

  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix",
```

```
"format": "prettier --write src/",
"build": "npm run lint && npm run test",
"clean": "rm -rf node_modules dist coverage",
"prestart": "npm run build",
"postinstall": "echo 'Зависимости установлены'"
},
```

```
"keywords": ["commonjs", "nodejs", "example"],
"author": "Ваше Имя <email@example.com>",
"license": "MIT",
```

```
"engines": {
  "node": ">=14.0.0",
  "npm": ">=6.0.0"
},
```

```
"dependencies": {
  "express": "^4.18.0",
  "mysql2": "^2.3.3",
  "jsonwebtoken": "^8.5.1",
  "bcryptjs": "^2.4.3",
  "cors": "^2.8.5",
  "helmet": "^6.0.0",
  "compression": "^1.7.4",
  "winston": "^3.8.0",
  "dotenv": "^16.0.3"
},
```

```
"devDependencies": {
  "nodemon": "^2.0.20",
  "jest": "^29.0.0",
  "supertest": "^6.2.4",
  "eslint": "^8.23.0",
  "prettier": "^2.7.1",
  "@types/node": "^18.7.0",
  "typescript": "^4.8.0"
},
```

```
"peerDependencies": {
  "react": ">=16.8.0",
  "react-dom": ">=16.8.0"
},
```

```
"optionalDependencies": {
```



```
"optional-module": "^1.0.0"
},

"bundledDependencies": ["important-module"],

"files": ["src/", "lib/", "README.md", "LICENSE"],

"bin": {
  "my-cli": "./bin/cli.js"
},

"man": [".man/doc.1"],

"directories": {
  "lib": "src",
  "test": "tests",
  "doc": "docs",
  "example": "examples"
},

"repository": {
  "type": "git",
  "url": "https://github.com/username/my-commonjs-app.git"
},

"bugs": {
  "url": "https://github.com/username/my-commonjs-app/issues"
},

"homepage": "https://github.com/username/my-commonjs-app#readme",

"config": {
  "port": 3000,
  "apiUrl": "https://api.example.com"
},

"publishConfig": {
  "registry": "https://registry.npmjs.org/",
  "access": "public"
},

"private": false,

"workspaces": ["packages/*"],
```

```

"os": ["darwin", "linux"],
"cpu": ["x64", "arm64"],

"preferGlobal": false,

"funding": {
  "type": "individual",
  "url": "https://github.com/sponsors/username"
}
}

```

4.2. Создание и публикация npm-пакета

```

javascript

// index.js - точка входа пакета

// Проверка версии Node.js
if (parseInt(process.versions.node.split('.')[0]) < 14) {
  console.error('× Требуется Node.js версии 14 или выше');
  process.exit(1);
}

// Основной модуль
const MyPackage = require('./lib/main');

// Экспорт API
module.exports = MyPackage;

// Экспорт вспомогательных модулей
module.exports.utils = require('./lib/utils');
module.exports.errors = require('./lib/errors');
module.exports.constants = require('./lib/constants');

// Экспорт типов для TypeScript (через JSDoc)
/**
 * @typedef {Object} PackageOptions
 * @property {string} [name]
 * @property {number} [version]
 * @property {boolean} [debug]
 */

// CLI интерфейс если файл запущен напрямую

```

```
if (require.main === module) {  
  const cli = require('./cli');  
  cli.run(process.argv.slice(2)).catch(error => {  
    console.error('Ошибка:', error.message);  
    process.exit(1);  
  });  
}
```

Часть 5: Миграция с CommonJS на ES6 модули

5.1. Поддержка обоих форматов

javascript

// dual-package.js - поддержка и CommonJS, и ES6 модулей

// CommonJS экспорт

```
const myModule = {  
  hello() {  
    return 'Hello from dual module!';  
  }  
};
```

// ES6 экспорт через module.exports для совместимости

```
module.exports = myModule;
```

// Для ES6 модулей в Node.js с "type": "module" в package.json

// нужно использовать export default или именованные экспорты

// Но мы можем добавить условие:

```
if (typeof module !== 'undefined' && module.exports) {  
  // CommonJS окружение  
  module.exports = myModule;  
} else if (typeof define === 'function' && define.amd) {  
  // AMD окружение  
  define([], function() {  
    return myModule;  
  });  
} else if (typeof exports === 'object') {  
  // ES6 модули в Node.js  
  exports.default = myModule;
```

```
Object.assign(exports, myModule);
} else {
  // Браузер (глобальная переменная)
  window.myModule = myModule;
}
```

5.2. Автоматическая конвертация

```
javascript

// convert-to-esm.js - утилита для конвертации

const fs = require('fs');
const path = require('path');
const { Transform } = require('stream');

class CommonJSToESMConverter extends Transform {
  constructor(options = {}) {
    super({ ...options, objectMode: true });
  }

  _transform(file, encoding, callback) {
    if (!file.isBuffer() || !file.contents) {
      return callback(null, file);
    }

    const content = file.contents.toString('utf8');
    const converted = this.convert(content);

    file.contents = Buffer.from(converted, 'utf8');

    // Меняем расширение .js на .mjs если нужно
    if (file.path.endsWith('.js')) {
      file.path = file.path.replace(/\.js$/, '.mjs');
    }

    callback(null, file);
  }

  convert(content) {
    let result = content;

    // Заменяем require() на import
    result = result.replace(
```

```

    /const\s+(\w+)\s*=\s*require\(['"](?:['"]+)|['"]\)/g,
    'import $1 from "$2";'
);

result = result.replace(
    /const\s+(\s*(?:\^|)+)\s*\s*=\s*require\(['"](?:['"]+)|['"]\)/g,
    'import { $1 } from "$2";'
);

// Заменяем module.exports на export default
result = result.replace(
    /module\.exports\s*=\s*(?:\^|)+;/g,
    'export default $1;'
);

result = result.replace(
    /module\.exports\.(?:\w+)\s*=\s*(?:\^|)+;/g,
    'export const $1 = $2;'
);

result = result.replace(
    /exports\.(?:\w+)\s*=\s*(?:\^|)+;/g,
    'export const $1 = $2;'
);

// Обрабатываем exports = module.exports
result = result.replace(
    /exports\s*=\s*module\.exports;/g,
    ''
);

// Добавляем расширение .js для относительных импортов
result = result.replace(
    /from\s+['"](?:\.\.\/|['"]+)|['"]/g,
    (match, p1) => {
        if (!p1.endsWith('.js') && !p1.endsWith('.mjs')) {
            return `from "${p1}.js"`;
        }
        return match;
    }
);

return result;
}

```

```
}
```

```
module.exports = CommonJSToESMConverter;
```

Заключение

CommonJS модули стали фундаментом экосистемы Node.js и до сих пор широко используются, несмотря на появление ES6 модулей. Их ключевые преимущества:

1. **Простота** – понятный и минималистичный API
2. **Синхронность** – предсказуемое выполнение
3. **Зрелость** – десятилетие стабильной работы
4. **Совместимость** – огромная экосистема пакетов

Однако есть и недостатки:

1. **Отсутствие статического анализа** – зависимости определяются в runtime
2. **Нет tree-shaking** – нельзя удалить неиспользуемый код
3. **Синхронная загрузка** – не оптимально для браузеров

Когда использовать CommonJS:

- Легаси-проекты на Node.js
- Пакеты, которые должны работать в старых версиях Node.js
- Скрипты и утилиты командной строки
- Когда нужна максимальная совместимость

Когда переходить на ES6 модули:

- Новые проекты на Node.js 14+
- Фронтенд-проекты с сборщиками (Webpack, Vite)
- Когда нужен статический анализ и tree-shaking
- При работе с TypeScript

В следующем разделе мы рассмотрим ES6 модули – современный стандарт, который решает многие ограничения CommonJS.

Практическое задание: Создайте модуль на CommonJS для работы с файловой системой, который:

1. Экспортирует класс FileManager с методами read, write, delete, list
2. Использует паттерн Singleton
3. Ведет логирование всех операций
4. Имеет кэширование прочитанных файлов
5. Предоставляет статистику использования

Введение: революция в модульности JavaScript

ES6 модули (ECMAScript 2015) стали официальным стандартом модульной системы в JavaScript. В отличие от CommonJS, который был создан для Node.js, ES6 модули разрабатывались как универсальное решение для всех сред выполнения JavaScript.

Исторический контекст:

- **2015 год:** ES6 (ECMAScript 2015) стандартизирует модули
- **2017 год:** Браузеры начинают внедрять нативную поддержку
- **2018 год:** Node.js 13.2.0 добавляет экспериментальную поддержку
- **2020 год:** Node.js 14 делает поддержку стабильной
- **2023 год:** ES6 модули становятся стандартом де-факто

Ключевые преимущества:

javascript

// 1. СТАТИЧЕСКИЙ АНАЛИЗ

// Компилятор может анализировать зависимости ДО выполнения кода

```
import { calculate } from './math.js';
```

// 2. ДЕРЕВО ЗАВИСИМОСТЕЙ (Tree Shaking)

// Сборщики могут удалить неиспользуемый код

```
import { featureA } from './library.js'; // Только featureA попадет в бандл
```

// 3. АСИНХРОННАЯ ЗАГРУЗКА

// Модули могут загружаться асинхронно

```
const module = await import('./dynamic-module.js');
```

// 4. ЛЕКСИЧЕСКАЯ ОБЛАСТЬ ВИДИМОСТИ

// Переменные модуля не попадают в глобальную область

// Каждый модуль имеет свою область видимости

// 5. СТРОГИЙ РЕЖИМ ПО УМОЛЧАНИЮ

// Код в модулях всегда выполняется в строгом режиме ('use strict')

Часть 1: Базовый синтаксис ES6 модулей

1.1. Объявление модулей

ES6 модули работают только когда файл явно объявлен как модуль:

В браузере:

html

```
<!-- type="module" обязателен -->
<script type="module" src="./app.js"></script>

<!-- или инлайново -->
<script type="module">
  import { hello } from './module.js';
  console.log(hello());
</script>
```

В Node.js:

json

```
// package.json
{
  "type": "module" // Все .js файлы будут ES6 модулями
}

// ИЛИ использовать расширение .mjs
// module.mjs - автоматически распознается как ES6 модуль
```

1.2. Экспорт (export)

Именованный экспорт (Named Export)

javascript

```
// math.js - модуль с математическими функциями

// Экспорт объявлений
export const PI = 3.141592653589793;
export const E = 2.718281828459045;
```

// Экспорт функции

```
export function add(a, b) {  
  return a + b;  
}
```

// Экспорт класса

```
export class Calculator {  
  constructor() {  
    this.memory = 0;  
  }
```

```
  add(x) {  
    this.memory += x;  
    return this.memory;  
  }
```

```
  subtract(x) {  
    this.memory -= x;  
    return this.memory;  
  }
```

```
  clear() {  
    this.memory = 0;  
  }  
}
```

// Экспорт стрелочной функции

```
export const multiply = (a, b) => a * b;
```

// Экспорт объекта

```
export const constants = {  
  GOLDEN_RATIO: 1.618033988749895,  
  PLANCK: 6.62607015e-34  
};
```

// Приватная функция (не экспортируется)

```
function validateNumber(num) {  
  return typeof num === 'number' && !isNaN(num) && isFinite(num);  
}
```

// Экспорт нескольких значений одним выражением

```
function divide(a, b) {  
  if (!validateNumber(a) || !validateNumber(b)) {  
    throw new Error('Invalid numbers');
```

```

}
if (b === 0) {
  throw new Error('Division by zero');
}
return a / b;
}

function power(base, exponent) {
  if (!validateNumber(base) || !validateNumber(exponent)) {
    throw new Error('Invalid numbers');
  }
  return Math.pow(base, exponent);
}

// Групповой экспорт
export { divide, power };

// Экспорт с переименованием
export { divide as safeDivide, power as exponentiate };

// Реекспорт (Re-export)
export { add as sum } from './basic-math.js';
export * from './geometry.js'; // Реекспорт всех именованных экспортов

```

Экспорт по умолчанию (Default Export)

```

javascript

// Logger.js - модуль с экспортом по умолчанию

// Каждый модуль может иметь только ОДИН экспорт по умолчанию

// Вариант 1: Экспорт объявления
export default class Logger {
  constructor(name = 'App') {
    this.name = name;
    this.logs = [];
  }

  log(level, message, meta = {}) {
    const entry = {
      timestamp: new Date().toISOString(),
      level,
      logger: this.name,
      message,

```

```

    meta,
    pid: process.pid
  };

  this.logs.push(entry);

  // Цветной вывод в консоль
  const colors = {
    error: '\x1b[31m', // красный
    warn: '\x1b[33m',  // желтый
    info: '\x1b[32m',  // зеленый
    debug: '\x1b[36m', // голубой
    trace: '\x1b[35m'  // пурпурный
  };

  const reset = '\x1b[0m';
  const color = colors[level] || '';

  console.log(
    `${color}[${entry.timestamp}] ${level.toUpperCase()} ${this.name}: ${message}${reset}`
  );

  return entry;
}

error(message, meta) {
  return this.log('error', message, meta);
}

warn(message, meta) {
  return this.log('warn', message, meta);
}

info(message, meta) {
  return this.log('info', message, meta);
}

debug(message, meta) {
  return this.log('debug', message, meta);
}

getLogs(filter = {}) {
  let filtered = this.logs;

```

```

    if (filter.level) {
        filtered = filtered.filter(log => log.level === filter.level);
    }

    if (filter.startDate) {
        filtered = filtered.filter(log => new Date(log.timestamp) >= filter.startDate);
    }

    if (filter.endDate) {
        filtered = filtered.filter(log => new Date(log.timestamp) <= filter.endDate);
    }

    return filtered;
}

clear() {
    this.logs = [];
}

getStats() {
    const levels = {};
    this.logs.forEach(log => {
        levels[log.level] = (levels[log.level] || 0) + 1;
    });

    return {
        total: this.logs.length,
        levels,
        firstLog: this.logs[0]?.timestamp,
        lastLog: this.logs[this.logs.length - 1]?.timestamp
    };
}
}

```

// Вариант 2: Экспорт выражения

```

const createLogger = (name) => new Logger(name);
export default createLogger;

```

// Вариант 3: Экспорт значения

```

const config = {
    defaultLevel: 'info',
    colors: true,
    timestamp: true,
    maxLogs: 10000
}

```

```
};

export default config;

// Вариант 4: Экспорт анонимной функции/класса
export default class {
  constructor() {
    console.log('Анонимный класс экспортирован по умолчанию');
  }
};

// Комбинированный экспорт (именованный + default)
export const version = '2.0.0';
export const levels = ['error', 'warn', 'info', 'debug', 'trace'];

export default Logger; // Экспорт по умолчанию должен быть последним
```

1.3. Импорт (import)

Импорт именованных экспортов

```
javascript

// app.js - основной файл приложения

// 1. Импорт конкретных именованных экспортов
import { PI, E, add, multiply } from './math.js';

console.log('PI:', PI); // 3.141592653589793
console.log('E:', E); // 2.718281828459045
console.log('2 + 3 =', add(2, 3)); // 5
console.log('2 × 3 =', multiply(2, 3)); // 6

// 2. Импорт с переименованием
import { PI as pi, add as sum } from './math.js';
import { Calculator as Calc } from './math.js';

console.log('pi:', pi); // 3.141592653589793
console.log('sum(5, 3):', sum(5, 3)); // 8

const calc = new Calc();
calc.add(10);
console.log('Memory:', calc.memory); // 10
```

```

// 3. Импорт всех именованных экспортов в объект
import * as MathUtils from './math.js';

console.log('MathUtils.PI:', MathUtils.PI);
console.log('MathUtils.add(10, 20):', MathUtils.add(10, 20));
console.log('MathUtils.constants.GOLDEN_RATIO:', MathUtils.constants.GOLDEN_RATIO);

// 4. Импорт только для побочных эффектов (side effects)
import './initialize.js'; // Код в initialize.js выполнится

// 5. Импорт реэкспортированных модулей
import { sum } from './math.js'; // Реэкспорт из basic-math.js

// 6. Импорт из node_modules
import { useState, useEffect } from 'react';
import { createRouter } from '@angular/router';
import _ from 'lodash-es'; // ES6 версия Lodash

// 7. Импорт из URL (браузер)
// import { html, css } from 'https://unpkg.com/lit-element@3.0.0/lit-element.js?module';

```

Импорт экспорта по умолчанию

```

javascript

// 1. Базовый импорт экспорта по умолчанию
import Logger from './logger.js';

const logger = new Logger('MyApp');
logger.info('Приложение запущено');
logger.error('Критическая ошибка!', { code: 500 });

// 2. Импорт с переименованием
import MyLogger from './logger.js';
import CustomLogger from './logger.js'; // Можно называть как угодно

// 3. Комбинированный импорт (default + named)
import Logger, { version, levels } from './logger.js';

console.log('Logger версия:', version); // '2.0.0'
console.log('Доступные уровни:', levels); // ['error', 'warn', 'info', 'debug', 'trace']

const appLogger = new Logger('App');
appLogger.warn('Предупреждение');

```

```
// 4. Импорт всего модуля (default и named)
import * as LoggerModule from './logger.js';

// Default экспорт доступен через свойство default
const LoggerClass = LoggerModule.default;
const logger = new LoggerClass('Test');

// Именованные экспорты доступны напрямую
console.log('Версия:', LoggerModule.version);

// 5. Импорт нескольких модулей в одной строке
import Logger, { version as loggerVersion } from './logger.js';
import { PI, E } from './math.js';
```

Часть 2: Динамический импорт (Dynamic Import)

2.1. Синтаксис и использование

```
javascript

// Динамический импорт возвращает Promise

// 1. Базовый динамический импорт
const loadMathModule = async () => {
  try {
    const mathModule = await import('./math.js');

    console.log('PI:', mathModule.PI);
    console.log('2 + 3 =', mathModule.add(2, 3));

    const Calculator = mathModule.default || mathModule.Calculator;
    const calc = new Calculator();
    calc.add(100);

    return mathModule;
  } catch (error) {
    console.error('Ошибка загрузки модуля:', error);
    throw error;
  }
};
```


// 2. Динамический импорт с деструктуризацией

```
const loadSpecificFunctions = async () => {  
  const { add, multiply, PI } = await import('./math.js');  
  
  return {  
    add: (a, b) => add(a, b),  
    multiply: (a, b) => multiply(a, b),  
    calculateCircleArea: (radius) => multiply(PI, multiply(radius, radius))  
  };  
};
```

// 3. Импорт по условию (Code Splitting)

```
const loadFeature = async (featureName) => {  
  switch (featureName) {  
    case 'dashboard':  
      return await import('./features/dashboard.js');  
    case 'analytics':  
      return await import('./features/analytics.js');  
    case 'reports':  
      return await import('./features/reports.js');  
    default:  
      return await import('./features/default.js');  
  }  
};
```

// 4. Ленивая загрузка компонентов (React/Vue)

```
const lazyLoadComponent = (componentName) => {  
  return lazy(() => import(`./components/${componentName}.js`));  
};
```

// 5. Динамический импорт с обработкой ошибок

```
const safeImport = async (modulePath, fallbackModule = null) => {  
  try {  
    return await import(modulePath);  
  } catch (error) {  
    console.warn(`Не удалось загрузить модуль ${modulePath}:`, error.message);  
  
    if (fallbackModule) {  
      return await import(fallbackModule);  
    }  
  
    return {  
      default: () => console.log('Фолбэк модуль'),  
    };  
  }  
};
```

```
    error: error.message
  };
}
};
```

2.2. Практические примеры

javascript

// dynamic-imports.js

```
class ModuleLoader {
  constructor() {
    this.loadedModules = new Map();
    this.loadingPromises = new Map();
    this.moduleCache = new Map();
  }

  // Загрузка модуля с кэшированием
  async load(modulePath, options = {}) {
    const cacheKey = modulePath + JSON.stringify(options);

    // Возвращаем промис из кэша если модуль уже загружается
    if (this.loadingPromises.has(cacheKey)) {
      return this.loadingPromises.get(cacheKey);
    }

    // Возвращаем из кэша если модуль уже загружен
    if (options.cache !== false && this.moduleCache.has(cacheKey)) {
      return this.moduleCache.get(cacheKey);
    }

    const loadPromise = (async () => {
      try {
        // Динамический импорт
        const module = await import(modulePath);

        // Применяем опции
        let processedModule = module;

        if (options.extractDefault && module.default) {
          processedModule = module.default;
        }
      }
    })();
  }
}
```

```

if (options.extractNamed && Array.isArray(options.extractNamed)) {
  const extracted = {};
  options.extractNamed.forEach(name => {
    if (module[name]) {
      extracted[name] = module[name];
    }
  });
  processedModule = extracted;
}

// Кэшируем результат
if (options.cache !== false) {
  this.moduleCache.set(cacheKey, processedModule);
  this.loadedModules.set(modulePath, {
    loadedAt: new Date(),
    size: this.estimateSize(processedModule)
  });
}

return processedModule;
} catch (error) {
  // Удаляем промис из кэша загрузки при ошибке
  this.loadingPromises.delete(cacheKey);

  if (options.fallback) {
    console.warn(`Загрузка ${modulePath} не удалась, используем фолбэк`);
    return this.load(options.fallback, { ...options, cache: false });
  }

  throw error;
} finally {
  // Очищаем промис из кэша загрузки
  this.loadingPromises.delete(cacheKey);
}
})();

// Сохраняем промис в кэше загрузки
this.loadingPromises.set(cacheKey, loadPromise);

return loadPromise;
}

// Параллельная загрузка нескольких модулей
async loadAll(modulePaths, options = {}) {

```

```

const loadPromises = modulePaths.map(path =>
  this.load(path, options)
);

const results = await Promise.allSettled(loadPromises);

const loaded = {};
const errors = {};

results.forEach((result, index) => {
  const modulePath = modulePaths[index];

  if (result.status === 'fulfilled') {
    loaded[modulePath] = result.value;
  } else {
    errors[modulePath] = result.reason;
    console.error(`Ошибка загрузки ${modulePath}:`, result.reason.message);
  }
});

return { loaded, errors };
}

```

// Предзагрузка модулей

```

preload(modulePaths) {
  modulePaths.forEach(path => {
    this.load(path, { cache: true }).catch(() => {
      // Игнорируем ошибки при предзагрузке
    });
  });
}

```

// Очистка кэша

```

clearCache(modulePath = null) {
  if (modulePath) {
    const keysToDelete = [];
    for (const [key] of this.moduleCache) {
      if (key.startsWith(modulePath)) {
        keysToDelete.push(key);
      }
    }
  }
  keysToDelete.forEach(key => {
    this.moduleCache.delete(key);
    this.loadedModules.delete(key);
  });
}

```

```

    });
  } else {
    this.moduleCache.clear();
    this.loadedModules.clear();
  }
}

// Получение статистики
getStats() {
  return {
    cachedModules: this.moduleCache.size,
    loadedModules: this.loadedModules.size,
    loadingNow: this.loadingPromises.size,
    totalSize: Array.from(this.loadedModules.values())
      .reduce((sum, info) => sum + (info.size || 0), 0),
    modules: Array.from(this.loadedModules.entries()).map(([path, info]) => ({
      path,
      loadedAt: info.loadedAt,
      size: info.size
    })))
  };
}

// Вспомогательная функция для оценки размера
estimateSize(obj) {
  try {
    const json = JSON.stringify(obj);
    return new Blob([json]).size;
  } catch {
    return 0;
  }
}

// Создание синглтона
export const moduleLoader = new ModuleLoader();

// Экспорт утилитарных функций
export async function importWithRetry(modulePath, maxRetries = 3, delay = 1000) {
  let lastError;

  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      return await import(modulePath);
    }
  }
}

```

```
    } catch (error) {
      lastError = error;
      console.warn(`Попытка ${attempt}/${maxRetries} не удалась для ${modulePath}`);

      if (attempt < maxRetries) {
        await new Promise(resolve => setTimeout(resolve, delay * attempt));
      }
    }
  }

  throw new Error(`Не удалось загрузить ${modulePath} после ${maxRetries} попыток: $
{lastError.message}`);
}

export function createLazyComponent(importFn, loadingComponent = null) {
  return {
    component: null,
    promise: null,

    async load() {
      if (!this.promise) {
        this.promise = importFn().then(module => {
          this.component = module.default || module;
          return this.component;
        });
      }

      return this.promise;
    },

    getComponent() {
      return this.component || loadingComponent;
    }
  };
}
```

Часть 3: Продвинутые техники работы с модулями

3.1. Циклические зависимости (Circular Dependencies)

```
javascript

// a.js - модуль A
import { bFunction } from './b.js';

export let aValue = 'initial a';

export function aFunction() {
  console.log('aFunction вызвана, bFunction говорит:', bFunction());
  return `aFunction: ${aValue}`;
}

export function updateA(newValue) {
  aValue = newValue;
  console.log('aValue обновлен на:', newValue);
}

console.log('Модуль A загружен, aValue:', aValue);

// b.js - модуль B
import { aFunction, updateA } from './a.js';

export let bValue = 'initial b';

export function bFunction() {
  console.log('bFunction вызвана, aFunction говорит:', aFunction());
  return `bFunction: ${bValue}`;
}

export function updateB(newValue) {
  bValue = newValue;
  // Обновляем модуль A через его экспортированную функцию
  updateA(`updated from b: ${newValue}`);
  console.log('bValue обновлен на:', newValue);
}

console.log('Модуль B загружен, bValue:', bValue);

// main.js - основной файл
```

```

import { aFunction, updateA } from './a.js';
import { bFunction, updateB } from './b.js';

console.log('=== Начало работы с циклическими зависимостями ===\n');

// Использование функций из обоих модулей
console.log('Вызов aFunction:', aFunction());
console.log('---');
console.log('Вызов bFunction:', bFunction());

console.log('\n=== Обновление значений ===');
updateA('новое значение A');
updateB('новое значение B');

console.log('\n=== Повторный вызов ===');
console.log('aFunction:', aFunction());
console.log('bFunction:', bFunction());

/*
Выход:
Модуль A загружен, aValue: initial a
Модуль B загружен, bValue: initial b

=== Начало работы с циклическими зависимостями ===

aFunction вызвана, bFunction говорит: bFunction: initial b
Вызов aFunction: aFunction: initial a
---
bFunction вызвана, aFunction говорит: aFunction: initial a
Вызов bFunction: bFunction: initial b

=== Обновление значений ===
aValue обновлен на: updated from b: новое значение B
bValue обновлен на: новое значение B

=== Повторный вызов ===
aFunction вызвана, bFunction говорит: bFunction: новое значение B
aFunction: updated from b: новое значение B
bFunction вызвана, aFunction говорит: aFunction: updated from b: новое значение B
bFunction: новое значение B
*/

```


3.2. Модули с состоянием (Stateful Modules)

```
javascript

// counter-module.js - модуль с состоянием

// Приватное состояние
let count = 0;
let history = [];
const maxHistorySize = 100;

// Приватные функции
function addToHistory(action, value, oldValue) {
  history.push({
    timestamp: new Date().toISOString(),
    action,
    value,
    oldValue,
    newCount: count
  });
}

// Ограничиваем размер истории
if (history.length > maxHistorySize) {
  history = history.slice(-maxHistorySize);
}

function validateIncrement(value) {
  if (typeof value !== 'number' || isNaN(value)) {
    throw new TypeError('Значение должно быть числом');
  }
  if (value <= 0) {
    throw new RangeError('Значение должно быть положительным');
  }
  return true;
}

// Публичный API
export function increment(value = 1) {
  validateIncrement(value);
  const oldCount = count;
  count += value;
  addToHistory('INCREMENT', value, oldCount);
  return count;
}
```

```
}
```

```
export function decrement(value = 1) {  
  validateIncrement(value);  
  const oldCount = count;  
  count = Math.max(0, count - value); // Не позволяем уйти в минус  
  addToHistory('DECREMENT', value, oldCount);  
  return count;  
}
```

```
export function reset() {  
  const oldCount = count;  
  count = 0;  
  addToHistory('RESET', 0, oldCount);  
  return count;  
}
```

```
export function getCount() {  
  return count;  
}
```

```
export function getHistory() {  
  return [...history]; // Возвращаем копию  
}
```

```
export function getStats() {  
  const increments = history.filter(h => h.action === 'INCREMENT').length;  
  const decrements = history.filter(h => h.action === 'DECREMENT').length;  
  const resets = history.filter(h => h.action === 'RESET').length;  
  
  const totalIncremented = history  
    .filter(h => h.action === 'INCREMENT')  
    .reduce((sum, h) => sum + h.value, 0);  
  
  const totalDecrement = history  
    .filter(h => h.action === 'DECREMENT')  
    .reduce((sum, h) => sum + h.value, 0);  
  
  return {  
    current: count,  
    totalOperations: history.length,  
    increments,  
    decrements,  
    resets,  
  }  
}
```

```

    totalIncremented,
    totalDecrement,
    averageIncrement: increments > 0 ? totalIncremented / increments : 0,
    averageDecrement: decrements > 0 ? totalDecrement / decrements : 0
  };
}

// Экспорт по умолчанию для удобства
export default {
  increment,
  decrement,
  reset,
  getCount,
  getHistory,
  getStats
};

```

3.3. Модули-синглтоны (Singleton Modules)

```

javascript

// database-connection.js - модуль синглтон для работы с БД

import mysql from 'mysql2/promise';

// Приватные переменные
let connectionPool = null;
let connectionStats = {
  created: null,
  totalQueries: 0,
  failedQueries: 0,
  activeConnections: 0
};

// Приватные функции
async function createPool(config) {
  const pool = mysql.createPool({
    host: config.host || 'localhost',
    port: config.port || 3306,
    user: config.user || 'root',
    password: config.password || '',
    database: config.database || 'test',
    waitForConnections: true,
    connectionLimit: config.connectionLimit || 10,
  });
}

```

```

    queueLimit: config.queueLimit || 0,
    enableKeepAlive: true,
    keepAliveInitialDelay: 0
  });

  // Тестируем соединение
  try {
    const testConnection = await pool.getConnection();
    testConnection.release();

    connectionStats.created = new Date();
    console.log('✅ Пул соединений создан успешно');

    return pool;
  } catch (error) {
    await pool.end();
    throw new Error(`Не удалось подключиться к БД: ${error.message}`);
  }
}

// Публичный API
export async function initialize(config) {
  if (connectionPool) {
    console.warn('⚠ Пул соединений уже инициализирован');
    return connectionPool;
  }

  if (!config || !config.host || !config.database) {
    throw new Error('Необходимо указать host и database в конфигурации');
  }

  connectionPool = await createPool(config);
  return connectionPool;
}

export async function query(sql, params = []) {
  if (!connectionPool) {
    throw new Error('Пул соединений не инициализирован. Вызовите initialize()');
  }

  connectionStats.totalQueries++;
  connectionStats.activeConnections++;

  try {

```

```

    const [rows, fields] = await connectionPool.execute(sql, params);
    return { rows, fields, success: true };
  } catch (error) {
    connectionStats.failedQueries++;
    console.error('× Ошибка запроса:', error.message);
    throw error;
  } finally {
    connectionStats.activeConnections--;
  }
}

export async function transaction(callback) {
  if (!connectionPool) {
    throw new Error('Пул соединений не инициализирован');
  }

  const connection = await connectionPool.getConnection();

  try {
    await connection.beginTransaction();
    const result = await callback(connection);
    await connection.commit();
    return result;
  } catch (error) {
    await connection.rollback();
    throw error;
  } finally {
    connection.release();
  }
}

export function getConnection() {
  if (!connectionPool) {
    throw new Error('Пул соединений не инициализирован');
  }

  return connectionPool.getConnection();
}

export function getStats() {
  return {
    ...connectionStats,
    isInitialized: !!connectionPool,
    poolConfig: connectionPool ? connectionPool.pool.config : null
  };
}

```

```

    };
  }

  export async function close() {
    if (connectionPool) {
      await connectionPool.end();
      connectionPool = null;
      connectionStats.created = null;
      console.log('🔌 Пул соединений закрыт');
    }
  }
}

// Экспорт по умолчанию для удобного использования
export default {
  initialize,
  query,
  transaction,
  getConnection,
  getStats,
  close
};

```

3.4. Модули с конфигурацией

```

javascript

// configurable-module.js

// Приватная конфигурация по умолчанию
let config = {
  debug: false,
  cacheEnabled: true,
  cacheTTL: 300000, // 5 минут в миллисекундах
  retryAttempts: 3,
  timeout: 10000,
  apiVersion: 'v1',
  logLevel: 'info'
};

// Приватный кэш
const cache = new Map();
const cacheTimestamps = new Map();

// Приватные функции

```

```

function log(level, message, data = null) {
  const levels = ['error', 'warn', 'info', 'debug', 'trace'];
  const currentLevel = levels.indexOf(config.logLevel);
  const messageLevel = levels.indexOf(level);

  if (messageLevel <= currentLevel) {
    const timestamp = new Date().toISOString();
    const prefix = config.debug ? `[${timestamp}] [${level.toUpperCase()}]` : '';

    if (data) {
      console[level](`${prefix} ${message}`, data);
    } else {
      console[level](`${prefix} ${message}`);
    }
  }
}

function getCacheKey(key) {
  return `${config.apiVersion}:${key}`;
}

function cleanupCache() {
  const now = Date.now();
  for (const [key, timestamp] of cacheTimestamps.entries()) {
    if (now - timestamp > config.cacheTTL) {
      cache.delete(key);
      cacheTimestamps.delete(key);
    }
  }
}

// Публичный API
export function configure(newConfig) {
  config = { ...config, ...newConfig };
  log('info', 'Конфигурация обновлена', config);

  if (config.cacheEnabled === false) {
    cache.clear();
    cacheTimestamps.clear();
  }

  return config;
}

```

```

export function getConfig() {
  return { ...config }; // Возвращаем копию
}

export async function fetchWithCache(url, options = {}) {
  if (!config.cacheEnabled) {
    return fetchDirectly(url, options);
  }

  const cacheKey = getCacheKey(url + JSON.stringify(options));
  const cached = cache.get(cacheKey);
  const now = Date.now();

  if (cached && cacheTimestamps.has(cacheKey)) {
    const timestamp = cacheTimestamps.get(cacheKey);

    if (now - timestamp < config.cacheTTL) {
      log('debug', 'Кэш попадание', { url, cacheKey });
      return { ...cached, cached: true };
    } else {
      log('debug', 'Кэш устарел', { url, cacheKey });
    }
  }

  log('info', 'Запрос данных', { url });
  const result = await fetchDirectly(url, options);

  cache.set(cacheKey, result);
  cacheTimestamps.set(cacheKey, now);

  // Периодическая очистка кэша
  if (Math.random() < 0.01) { // 1% chance
    cleanupCache();
  }

  return { ...result, cached: false };
}

export async function fetchDirectly(url, options = {}, attempt = 1) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), config.timeout);

  try {
    const response = await fetch(url, {

```



```

    ...options,
    signal: controller.signal
  });

  clearTimeout(timeoutId);

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${response.statusText}`);
  }

  const data = await response.json();

  log('debug', 'Запрос успешен', {
    url,
    status: response.status,
    attempt
  });

  return {
    success: true,
    data,
    status: response.status,
    headers: Object.fromEntries(response.headers.entries())
  };
} catch (error) {
  clearTimeout(timeoutId);

  if (attempt < config.retryAttempts && !error.message.includes('abort')) {
    log('warn', 'Повторная попытка', {
      url,
      attempt,
      error: error.message,
      nextAttempt: attempt + 1
    });

    // Экспоненциальная задержка
    const delay = Math.pow(2, attempt) * 1000;
    await new Promise(resolve => setTimeout(resolve, delay));

    return fetchDirectly(url, options, attempt + 1);
  }

  log('error', 'Запрос не удался', {
    url,

```

```

    attempt,
    error: error.message
  });

  return {
    success: false,
    error: error.message,
    attempts: attempt
  };
}
}

export function clearCache(pattern = null) {
  if (pattern) {
    const regex = new RegExp(pattern);
    let cleared = 0;

    for (const key of cache.keys()) {
      if (regex.test(key)) {
        cache.delete(key);
        cacheTimestamps.delete(key);
        cleared++;
      }
    }

    log('info', 'Кэш частично очищен', { pattern, cleared });
  } else {
    const size = cache.size;
    cache.clear();
    cacheTimestamps.clear();
    log('info', 'Кэш полностью очищен', { cleared: size });
  }

  return cache.size;
}

export function getCacheStats() {
  const now = Date.now();
  let expired = 0;
  let valid = 0;

  for (const timestamp of cacheTimestamps.values()) {
    if (now - timestamp > config.cacheTTL) {
      expired++;
    }
  }
}

```

```
    } else {
      valid++;
    }
  }

  return {
    total: cache.size,
    valid,
    expired,
    ttl: config.cacheTTL,
    enabled: config.cacheEnabled
  };
}

// Экспорт по умолчанию
export default {
  configure,
  getConfig,
  fetchWithCache,
  fetchDirectly,
  clearCache,
  getCacheStats
};
```

Часть 4: Практические паттерны и лучшие практики

4.1. Barrel Exports (Бочкообразные экспорты)

javascript

// Паттерн: Создание файла-индекса для реэкспорта

// src/math/index.js - бочкообразный экспорт

```
export { add, subtract } from './operations.js';
export { multiply, divide } from './advanced.js';
export { PI, E } from './constants.js';
export { default as Calculator } from './Calculator.js';
export { default as Statistics } from './Statistics.js';
```

// Реэкспорт с переименованием для избежания конфликтов

```
export { add as sum, subtract as diff } from './operations.js';
```

```

// Реекспорт всего содержимого модуля
export * from './utils.js';

// Экспорт по умолчанию из бочки
import Calculator from './Calculator.js';
import Statistics from './Statistics.js';

export default {
  Calculator,
  Statistics,
  // Можно добавить вспомогательные функции
  calculateAverage(...numbers) {
    const sum = numbers.reduce((a, b) => a + b, 0);
    return sum / numbers.length;
  }
};

// Использование бочки
import { add, PI, Calculator } from './math/index.js';
// Или более коротко:
import { add, PI } from './math/index.js';

// Бочка для React компонентов
// src/components/index.js
export { default as Button } from './Button/Button.jsx';
export { default as Input } from './Input/Input.jsx';
export { default as Modal } from './Modal/Modal.jsx';
export { default as Card } from './Card/Card.jsx';
export { default as Table } from './Table/Table.jsx';

// Бочка для утилит
// src/utils/index.js
export { formatDate, parseDate } from './date-utils.js';
export { debounce, throttle } from './function-utils.js';
export { validateEmail, validatePhone } from './validation.js';
export { storage, cookies } from './browser-utils.js';
export { default as logger } from './logger.js';

```

4.2. Dependency Injection через модули

javascript

// Паттерн: Внедрение зависимостей через модули

// src/database/adapters/index.js - абстракция базы данных

```
export class DatabaseAdapter {
  constructor(config) {
    this.config = config;
  }

  async connect() {
    throw new Error('Метод connect должен быть реализован');
  }

  async query(sql, params) {
    throw new Error('Метод query должен быть реализован');
  }

  async disconnect() {
    throw new Error('Метод disconnect должен быть реализован');
  }
}
```

// src/database/adapters/postgres.js - реализация для PostgreSQL

```
import { DatabaseAdapter } from './index.js';
import pg from 'pg';

export class PostgresAdapter extends DatabaseAdapter {
  constructor(config) {
    super(config);
    this.pool = null;
  }

  async connect() {
    this.pool = new pg.Pool(this.config);
    await this.pool.query('SELECT 1'); // Тестовый запрос
    return this;
  }

  async query(sql, params) {
    if (!this.pool) {
      throw new Error('Не подключено к базе данных');
    }

    const result = await this.pool.query(sql, params);
    return result.rows;
  }
}
```

```

    async disconnect() {
      if (this.pool) {
        await this.pool.end();
        this.pool = null;
      }
    }
  }
}

// src/database/adapters/mysql.js - реализация для MySQL
import { DatabaseAdapter } from './index.js';
import mysql from 'mysql2/promise';

export class MysqlAdapter extends DatabaseAdapter {
  constructor(config) {
    super(config);
    this.connection = null;
  }

  async connect() {
    this.connection = await mysql.createConnection(this.config);
    await this.connection.query('SELECT 1');
    return this;
  }

  async query(sql, params) {
    if (!this.connection) {
      throw new Error('Не подключено к базе данных');
    }

    const [rows] = await this.connection.execute(sql, params);
    return rows;
  }

  async disconnect() {
    if (this.connection) {
      await this.connection.end();
      this.connection = null;
    }
  }
}

```

```

// src/database/adapters/sqlite.js - реализация для SQLite
import { DatabaseAdapter } from './index.js';

```

```

import sqlite3 from 'sqlite3';
import { open } from 'sqlite';

export class SqliteAdapter extends DatabaseAdapter {
  constructor(config) {
    super(config);
    this.db = null;
  }

  async connect() {
    this.db = await open({
      filename: this.config.filename || ':memory:',
      driver: sqlite3.Database
    });
    return this;
  }

  async query(sql, params) {
    if (!this.db) {
      throw new Error('Не подключено к базе данных');
    }

    return await this.db.all(sql, params);
  }

  async disconnect() {
    if (this.db) {
      await this.db.close();
      this.db = null;
    }
  }
}

// src/database/index.js - фабрика адаптеров
import { PostgresAdapter } from './adapters/postgres.js';
import { MysqlAdapter } from './adapters/mysql.js';
import { SqliteAdapter } from './adapters/sqlite.js';

export class DatabaseFactory {
  static createAdapter(type, config) {
    switch (type.toLowerCase()) {
      case 'postgres':
      case 'postgresql':
        return new PostgresAdapter(config);
    }
  }
}

```

```

    case 'mysql':
    case 'mariadb':
        return new MysqlAdapter(config);

    case 'sqlite':
        return new SqliteAdapter(config);

    default:
        throw new Error(`Неподдерживаемый тип базы данных: ${type}`);
    }
}

static async createAndConnect(type, config) {
    const adapter = this.createAdapter(type, config);
    await adapter.connect();
    return adapter;
}
}

// src/services/user-service.js - сервис с внедрением зависимости
export class UserService {
    constructor(databaseAdapter) {
        this.db = databaseAdapter;
    }

    async getUser(id) {
        const [user] = await this.db.query(
            'SELECT * FROM users WHERE id = ?',
            [id]
        );
        return user || null;
    }

    async createUser(userData) {
        const result = await this.db.query(
            'INSERT INTO users (name, email) VALUES (?, ?)',
            [userData.name, userData.email]
        );
        return { id: result.insertId, ...userData };
    }

    async updateUser(id, updates) {
        await this.db.query(

```



```

    'UPDATE users SET name = ?, email = ? WHERE id = ?',
    [updates.name, updates.email, id]
  );
  return this.getUser(id);
}

async deleteUser(id) {
  await this.db.query('DELETE FROM users WHERE id = ?', [id]);
  return true;
}
}

```

// src/app.js - сборка приложения с внедрением зависимостей

```

import { DatabaseFactory } from './database/index.js';
import { UserService } from './services/user-service.js';

export async function createApp(config) {
  // Создаем адаптер базы данных
  const dbAdapter = await DatabaseFactory.createAndConnect(
    config.database.type,
    config.database
  );

  // Создаем сервисы с внедренными зависимостями
  const userService = new UserService(dbAdapter);

  // Возвращаем объект приложения
  return {
    db: dbAdapter,
    services: {
      users: userService
    },

    async close() {
      await dbAdapter.disconnect();
    }
  };
}

```

// Использование

```

const config = {
  database: {
    type: 'postgres',
    host: 'localhost',

```

```

    port: 5432,
    database: 'myapp',
    user: 'postgres',
    password: 'password'
  }
};

const app = await createApp(config);

try {
  const user = await app.services.users.getUser(1);
  console.log('Пользователь:', user);
} finally {
  await app.close();
}

```

4.3. Модули с плагинами

```

javascript

// core/module-with-plugins.js

// Приватное состояние
const plugins = new Map();
const hooks = new Map();
const events = new Map();

// Базовый класс для плагинов
export class BasePlugin {
  constructor(name) {
    this.name = name;
    this.enabled = true;
    this.version = '1.0.0';
  }

  // Методы, которые могут быть переопределены плагинами
  async initialize(module) {
    // Инициализация плагина
  }

  async destroy() {
    // Очистка ресурсов плагина
  }
}

```

```

registerHooks() {
  // Регистрация хуков плагина
  return {};
}

registerEvents() {
  // Регистрация событий плагина
  return {};
}

// Вспомогательные методы
log(message, data = null) {
  console.log(`[${this.name}] ${message}`, data || '');
}

error(message, error = null) {
  console.error(`[${this.name}] × ${message}`, error || '');
}
}

// Основной модуль
export class PluginModule {
  constructor(name) {
    this.name = name;
    this.config = {
      autoInitialize: true,
      strictMode: false,
      logLevel: 'info'
    };
  }
}

// Регистрация плагина
registerPlugin(pluginClass, options = {}) {
  if (plugins.has(pluginClass.name)) {
    throw new Error(`Плагин ${pluginClass.name} уже зарегистрирован`);
  }

  const plugin = new pluginClass(pluginClass.name);
  plugins.set(pluginClass.name, {
    instance: plugin,
    options,
    hooks: new Set(),
    events: new Set()
  });
}

```

```

console.log(`📌 Плагин "${pluginClass.name}" зарегистрирован`);

// Автоматическая инициализация
if (this.config.autoInitialize) {
  this.initializePlugin(pluginClass.name);
}

return this;
}

// Инициализация плагина
async initializePlugin(pluginName) {
  const pluginInfo = plugins.get(pluginName);
  if (!pluginInfo) {
    throw new Error(`Плагин ${pluginName} не найден`);
  }

  if (!pluginInfo.instance.enabled) {
    console.log(`🔌 Плагин ${pluginName} отключен, пропускаем инициализацию`);
    return;
  }

  try {
    // Инициализация плагина
    await pluginInfo.instance.initialize(this);

    // Регистрация хуков плагина
    const pluginHooks = pluginInfo.instance.registerHooks();
    for (const [hookName, handler] of Object.entries(pluginHooks)) {
      this.registerHook(hookName, handler, pluginName);
      pluginInfo.hooks.add(hookName);
    }

    // Регистрация событий плагина
    const pluginEvents = pluginInfo.instance.registerEvents();
    for (const [eventName, handler] of Object.entries(pluginEvents)) {
      this.addEventListener(eventName, handler, pluginName);
      pluginInfo.events.add(eventName);
    }

    console.log(`📌 Плагин "${pluginName}" инициализирован`);
  } catch (error) {
    console.error(`❌ Ошибка инициализации плагина ${pluginName}:`, error);
  }
}

```

```
    if (this.config.strictMode) {  
        throw error;  
    }  
}  
}
```

// Регистрация хука

```
registerHook(hookName, handler, pluginName = null) {  
    if (!hooks.has(hookName)) {  
        hooks.set(hookName, []);  
    }  
}
```

```
hooks.get(hookName).push({  
    handler,  
    pluginName  
});
```

```
return this;  
}
```

// Вызов хука

```
async callHook(hookName, ...args) {  
    if (!hooks.has(hookName)) {  
        return [];  
    }  
}
```

```
const results = [];  
const hookHandlers = hooks.get(hookName);
```

```
for (const hook of hookHandlers) {  
    try {  
        const result = await hook.handler(...args);  
        results.push({  
            pluginName: hook.pluginName,  
            result,  
            success: true  
        });  
    } catch (error) {  
        results.push({  
            pluginName: hook.pluginName,  
            error: error.message,  
            success: false  
        });  
    }  
}
```

```

        this.emit('hook:error', {
            hookName,
            pluginName: hook.pluginName,
            error
        });
    }
}

return results;
}

// Работа с событиями
addEventListener(eventName, handler, pluginName = null) {
    if (!events.has(eventName)) {
        events.set(eventName, []);
    }

    events.get(eventName).push({
        handler,
        pluginName
    });

    return this;
}

emit(eventName, data) {
    if (!events.has(eventName)) {
        return;
    }

    const eventHandlers = events.get(eventName);

    for (const eventHandler of eventHandlers) {
        try {
            eventHandler.handler(data);
        } catch (error) {
            console.error(`Ошибка в обработке события ${eventName}:`, error);
        }
    }
}

// Управление плагинами
enablePlugin(pluginName) {

```

```

const pluginInfo = plugins.get(pluginName);
if (pluginInfo) {
  pluginInfo.instance.enabled = true;
  this.initializePlugin(pluginName);
  return true;
}
return false;
}

```

```

disablePlugin(pluginName) {
  const pluginInfo = plugins.get(pluginName);
  if (pluginInfo) {
    pluginInfo.instance.enabled = false;
    this.unregisterPluginHooks(pluginName);
    this.unregisterPluginEvents(pluginName);
    return true;
  }
  return false;
}

```

```

unregisterPluginHooks(pluginName) {
  for (const [hookName, hookHandlers] of hooks) {
    const filtered = hookHandlers.filter(h => h.pluginName !== pluginName);
    if (filtered.length === 0) {
      hooks.delete(hookName);
    } else {
      hooks.set(hookName, filtered);
    }
  }
}

```

```

unregisterPluginEvents(pluginName) {
  for (const [eventName, eventHandlers] of events) {
    const filtered = eventHandlers.filter(e => e.pluginName !== pluginName);
    if (filtered.length === 0) {
      events.delete(eventName);
    } else {
      events.set(eventName, filtered);
    }
  }
}

```

// Получение информации о плагинах

```

getPlugin(pluginName) {

```

```

    const pluginInfo = plugins.get(pluginName);
    return pluginInfo ? pluginInfo.instance : null;
}

getAllPlugins() {
    return Array.from(plugins.entries()).map(([name, info]) => ({
        name,
        enabled: info.instance.enabled,
        version: info.instance.version,
        hooks: Array.from(info.hooks),
        events: Array.from(info.events)
    }));
}

getStats() {
    return {
        totalPlugins: plugins.size,
        enabledPlugins: Array.from(plugins.values())
            .filter(info => info.instance.enabled).length,
        totalHooks: hooks.size,
        totalEvents: events.size,
        hookHandlers: Array.from(hooks.values())
            .reduce((sum, handlers) => sum + handlers.length, 0),
        eventHandlers: Array.from(events.values())
            .reduce((sum, handlers) => sum + handlers.length, 0)
    };
}
}

```

// Пример плагина

```

export class LoggingPlugin extends BasePlugin {
    constructor() {
        super('logging');
        this.version = '2.1.0';
        this.logs = [];
    }

    async initialize(module) {
        this.module = module;
        this.log('Плагин логирования инициализирован');
    }

    registerHooks() {
        return {

```



```
    'request:start': this.onRequestStart.bind(this),
    'request:end': this.onRequestEnd.bind(this),
    'error': this.onError.bind(this)
  };
}
```

```
registerEvents() {
  return {
    'log:message': this.onLogMessage.bind(this)
  };
}
```

```
onRequestStart(url, method) {
  const logEntry = {
    timestamp: new Date(),
    type: 'request:start',
    url,
    method
  };

  this.logs.push(logEntry);
  this.log(`🚀 ${method} ${url}`);

  return logEntry;
}
```

```
onRequestEnd(url, method, status, duration) {
  const logEntry = {
    timestamp: new Date(),
    type: 'request:end',
    url,
    method,
    status,
    duration
  };

  this.logs.push(logEntry);
  this.log(`🏠 ${method} ${url} - ${status} (${duration}ms)`);

  return logEntry;
}
```

```
onError(error, context) {
  const logEntry = {
```

```

    timestamp: new Date(),
    type: 'error',
    error: error.message,
    stack: error.stack,
    context
  };

  this.logs.push(logEntry);
  this.error(`✖ ${error.message}`, context);

  return logEntry;
}

onLogMessage(data) {
  this.logs.push({
    timestamp: new Date(),
    type: 'log:message',
    ...data
  });
}

getLogs(filter = {}) {
  let filtered = this.logs;

  if (filter.type) {
    filtered = filtered.filter(log => log.type === filter.type);
  }

  if (filter.startDate) {
    filtered = filtered.filter(log => log.timestamp >= filter.startDate);
  }

  if (filter.endDate) {
    filtered = filtered.filter(log => log.timestamp <= filter.endDate);
  }

  return filtered;
}

clearLogs() {
  this.logs = [];
}
}

```

```
// Пример использования
const module = new PluginModule('MyApp');
module.registerPlugin(LoggingPlugin);

// Вызов хуков
module.callHook('request:start', '/api/users', 'GET');
module.callHook('request:end', '/api/users', 'GET', 200, 150);

// Получение логов
const loggingPlugin = module.getPlugin('logging');
const logs = loggingPlugin.getLogs({ type: 'request:start' });
```

Часть 5: Совместимость и миграция

5.1. Двойные пакеты (Dual Packages)

```
javascript

// Паттерн: поддержка и CommonJS, и ES6 модулей

// package.json
{
  "name": "my-dual-package",
  "version": "1.0.0",
  "type": "module", // По умолчанию ES6 модули

  // Точки входа для разных систем модулей
  "main": "./dist/cjs/index.cjs", // CommonJS
  "module": "./dist/esm/index.mjs", // ES6 модули
  "exports": {
    ".": {
      "import": "./dist/esm/index.mjs", // ES6 модули
      "require": "./dist/cjs/index.cjs", // CommonJS
      "default": "./dist/esm/index.mjs" // Резервный
    },
    "./package.json": "./package.json",
    "./styles.css": "./dist/styles.css"
  }
}

// src/index.js - исходный код (ES6 модули)
```

```
export const version = '1.0.0';

export function hello(name) {
  return `Hello, ${name}!`;
}

export default {
  version,
  hello
};

// Генерация CommonJS версии (build-cjs.js)
import fs from 'fs';
import path from 'path';
import { transformSync } from '@babel/core';

const srcDir = './src';
const cjsDir = './dist/cjs';

// Создаем директории
if (!fs.existsSync(cjsDir)) {
  fs.mkdirSync(cjsDir, { recursive: true });
}

// Обходим все .js файлы в src
const files = fs.readdirSync(srcDir).filter(file => file.endsWith('.js'));

files.forEach(file => {
  const srcPath = path.join(srcDir, file);
  const destPath = path.join(cjsDir, file.replace(/\.js$/, '.cjs'));

  const source = fs.readFileSync(srcPath, 'utf8');

  // Транспилиция ES6 -> CommonJS
  const result = transformSync(source, {
    presets: ['@babel/preset-env'],
    plugins: [
      ['@babel/plugin-transform-modules-commonjs', {
        loose: true
      }]
    ],
    filename: file
  });
```

```
fs.writeFileSync(destPath, result.code);
console.log(`📄 Создан: ${destPath}`);
});
```

```
// package.json для CommonJS сборки
```

```
const packageJson = {
  type: "commonjs",
  main: "./index.cjs"
};
```

```
fs.writeFileSync(
  path.join(cjsDir, 'package.json'),
  JSON.stringify(packageJson, null, 2)
);
```

```
// Генерация ES6 версии (build-esm.js)
```

```
import fs from 'fs';
import path from 'path';
```

```
const esmDir = './dist/esm';
```

```
if (!fs.existsSync(esmDir)) {
  fs.mkdirSync(esmDir, { recursive: true });
}
```

```
files.forEach(file => {
  const srcPath = path.join(srcDir, file);
  const destPath = path.join(esmDir, file.replace(/\.js$/, '.mjs'));

  const source = fs.readFileSync(srcPath, 'utf8');
```

```
// Просто копируем, т.к. это уже ES6 модули
```

```
fs.writeFileSync(destPath, source);
console.log(`📄 Создан: ${destPath}`);
});
```

```
// package.json для ESM сборки
```

```
const esmPackageJson = {
  type: "module",
  main: "./index.mjs"
};
```

```
fs.writeFileSync(
  path.join(esmDir, 'package.json'),
```

```
JSON.stringify(esmPackageJson, null, 2)
);
```

5.2. Обнаружение типа модулей

```
javascript

// module-type-detector.js

/**
 * Определяет тип модулей, поддерживаемых окружением
 */
export function detectModuleSupport() {
  const results = {
    // Поддержка ES6 модулей
    esm: {
      staticImport: false,
      dynamicImport: false,
      export: false,
      typeModule: false
    },

    // Поддержка CommonJS
    cjs: {
      require: false,
      module: false,
      exports: false
    },

    // Поддержка AMD
    amd: {
      define: false,
      require: false
    },

    // Поддержка UMD
    umd: false,

    // Детекция среды
    environment: {
      node: false,
      browser: false,
      version: null,
      globals: {}
    }
  }
}
```

```

}
};

// Определяем среду выполнения
if (typeof window !== 'undefined' && typeof document !== 'undefined') {
  results.environment.browser = true;
  results.environment.version = navigator.userAgent;
  results.environment.globals = {
    window: true,
    document: true,
    navigator: true
  };
} else if (typeof process !== 'undefined' && process.versions && process.versions.node)
{
  results.environment.node = true;
  results.environment.version = process.version;
  results.environment.globals = {
    process: true,
    global: true,
    __filename: typeof __filename !== 'undefined',
    __dirname: typeof __dirname !== 'undefined'
  };
}

// Проверяем поддержку ES6 модулей
try {
  // Проверяем статический импорт
  eval('import("data:text/javascript,export default 42")');
  results.esm.dynamicImport = true;
} catch {}

// Проверяем, можем ли мы объявить модуль
try {
  const script = document ? document.createElement('script') : null;
  if (script) {
    script.type = 'module';
    results.esm.typeModule = true;
  }
} catch {}

// Проверяем поддержку CommonJS
if (typeof require === 'function') {
  results.cjs.require = true;
}

```

```

if (typeof module !== 'undefined') {
  results.cjs.module = true;
}

if (typeof exports !== 'undefined') {
  results.cjs.exports = true;
}

// Проверяем поддержку AMD
if (typeof define === 'function' && define.amd) {
  results.amd.define = true;
  results.amd.require = true;
}

// Проверяем поддержку UMD
results.umd = (results.cjs.require && results.cjs.module) ||
  results.amd.define ||
  (results.environment.browser && typeof window !== 'undefined');

// Определяем рекомендуемый тип модулей
if (results.esm.dynamicImport && results.environment.node) {
  results.recommended = 'esm';
} else if (results.cjs.require && results.cjs.module) {
  results.recommended = 'cjs';
} else if (results.amd.define) {
  results.recommended = 'amd';
} else if (results.environment.browser) {
  results.recommended = 'esm'; // Современные браузеры поддерживают ES6
} else {
  results.recommended = 'cjs'; // Fallback
}

return results;
}

/**
 * Загружает модуль в зависимости от доступного типа
 */
export async function universalImport(modulePath, options = {}) {
  const support = detectModuleSupport();
  const { type = support.recommended } = options;

  switch (type) {

```



```

case 'esm':
  if (support.esm.dynamicImport) {
    const module = await import(modulePath);
    return options.extractDefault ? module.default : module;
  }
  break;

case 'cjs':
  if (support.cjs.require) {
    const module = require(modulePath);
    return options.extractDefault ? (module.default || module) : module;
  }
  break;

case 'amd':
  if (support.amd.require) {
    return new Promise((resolve, reject) => {
      require([modulePath], (module) => {
        resolve(options.extractDefault ? module.default : module);
      }, reject);
    });
  }
  break;
}

throw new Error(`Тип модулей "${type}" не поддерживается в данном окружении`);
}

/**
 * Создает универсальный модуль (UMD)
 */
export function createUniversalModule(factory) {
  const support = detectModuleSupport();

  // UMD обертка
  (function(root, factory) {
    if (support.cjs.module && support.cjs.exports) {
      // CommonJS
      module.exports = factory();
    } else if (support.amd.define) {
      // AMD
      define([], factory);
    } else if (root) {
      // Глобальная переменная

```

```

    root.myModule = factory();
  }
})(typeof self !== 'undefined' ? self : this, factory);

// ES6 экспорт если поддерживается
if (support.esm.export) {
  return factory();
}
}

// Пример использования
const moduleSupport = detectModuleSupport();
console.log('Поддержка модулей:', moduleSupport);

// Загрузка модуля с автоматическим определением типа
try {
  const module = await universalImport('./my-module.js');
  console.log('Модуль загружен:', module);
} catch (error) {
  console.error('Ошибка загрузки модуля:', error);
}

```

Часть 6: Производительность и оптимизация

6.1. Tree Shaking и dead code elimination

```

javascript

// Оптимизированный модуль для tree shaking

// ПЛОХО - неоптимизированный модуль
export class BadModule {
  constructor() {
    this.methodA = this.methodA.bind(this);
    this.methodB = this.methodB.bind(this);
    this.methodC = this.methodC.bind(this);
  }

  methodA() { /* ... */ }
  methodB() { /* ... */ }
  methodC() { /* ... */ }
}

```

```
// Побочные эффекты при импорте
console.log('BadModule загружен'); // Сборщик не может удалить этот код
}

// ХОРОШО - оптимизированный модуль
// Каждая функция экспортируется отдельно
export function methodA() {
  // Pure function - нет побочных эффектов
  return /* ... */;
}

export function methodB() {
  // Pure function
  return /* ... */;
}

export function methodC() {
  // Pure function
  return /* ... */;
}

// Побочные эффекты вынесены в отдельную функцию
export function initialize() {
  console.log('Module initialized');
  // Инициализация с побочными эффектами
}

// Константы отдельно
export const CONSTANTS = {
  API_URL: 'https://api.example.com',
  VERSION: '1.0.0'
};

// Экспорт по умолчанию должен быть простым
export default {
  methodA,
  methodB,
  methodC,
  CONSTANTS
};

// ЕЩЕ ЛУЧШЕ - side-effect free модуль
// package.json указывает, что модуль не имеет побочных эффектов
```

```

/*
{
  "name": "my-module",
  "sideEffects": false,
  "module": "index.js"
}
*/

// Или указываем конкретные файлы с побочными эффектами
/*
{
  "sideEffects": [
    "**/*.css",
    "**/*.scss",
    "polyfill.js"
  ]
}
*/

```

6.2. Ленивая загрузка и код-сплиттинг

```

javascript

// advanced-lazy-loading.js

// Карта модулей для ленивой загрузки
const moduleRegistry = new Map();

// Регистрация модулей
export function registerModule(name, importFn, options = {}) {
  moduleRegistry.set(name, {
    importFn,
    loaded: false,
    module: null,
    loading: false,
    promise: null,
    preload: options.preload || false,
    prefetch: options.prefetch || false,
    priority: options.priority || 0
  });

  // Предзагрузка если указана
  if (options.preload) {
    preloadModule(name);
  }
}

```

```

}

// Prefetch если указан
if (options.prefetch) {
  prefetchModule(name);
}
}

// Ленивая загрузка модуля
export async function loadModule(name, force = false) {
  const moduleInfo = moduleRegistry.get(name);

  if (!moduleInfo) {
    throw new Error(`Модуль "${name}" не зарегистрирован`);
  }

  // Если модуль уже загружен
  if (moduleInfo.loaded && !force) {
    return moduleInfo.module;
  }

  // Если модуль уже загружается
  if (moduleInfo.loading && moduleInfo.promise && !force) {
    return moduleInfo.promise;
  }

  // Начинаем загрузку
  moduleInfo.loading = true;
  moduleInfo.promise = (async () => {
    try {
      const module = await moduleInfo.importFn();
      moduleInfo.module = module;
      moduleInfo.loaded = true;
      moduleInfo.loading = false;

      // Вызываем колбэк загрузки
      if (moduleInfo.onLoad) {
        moduleInfo.onLoad(module);
      }

      return module;
    } catch (error) {
      moduleInfo.loading = false;
      moduleInfo.loaded = false;
    }
  })();
}

```

```

    moduleInfo.promise = null;
    throw error;
  }
})();

return moduleInfo.promise;
}

// Предзагрузка модуля
export async function preloadModule(name) {
  const moduleInfo = moduleRegistry.get(name);
  if (!moduleInfo || moduleInfo.loaded || moduleInfo.loading) {
    return;
  }

  // Используем requestIdleCallback для предзагрузки
  if ('requestIdleCallback' in window) {
    requestIdleCallback(() => {
      loadModule(name).catch(() => {
        // Игнорируем ошибки при предзагрузке
      });
    });
  } else {
    // Fallback
    setTimeout(() => {
      loadModule(name).catch(() => {});
    }, 1000);
  }
}

// Prefetch модуля
export function prefetchModule(name) {
  const moduleInfo = moduleRegistry.get(name);
  if (!moduleInfo || moduleInfo.loaded) {
    return;
  }

  // Создаем link rel=prefetch для браузера
  if (typeof moduleInfo.importFn === 'string') {
    const link = document.createElement('link');
    link.rel = 'prefetch';
    link.href = moduleInfo.importFn;
    link.as = 'script';
    document.head.appendChild(link);
  }
}

```

```
}  
}
```

// Загрузка нескольких модулей

```
export async function loadModules(names, parallel = true) {  
  if (parallel) {  
    const promises = names.map(name => loadModule(name));  
    return Promise.all(promises);  
  } else {  
    const results = [];  
    for (const name of names) {  
      const module = await loadModule(name);  
      results.push(module);  
    }  
    return results;  
  }  
}
```

// Приоритетная загрузка

```
export async function loadWithPriority(names) {  
  const modules = names.map(name => moduleRegistry.get(name));
```

// Сортируем по приоритету (высокий приоритет первый)

```
modules.sort((a, b) => b.priority - a.priority);
```

```
const sortedNames = modules.map(m => {  
  for (const [name, info] of moduleRegistry) {  
    if (info === m) return name;  
  }  
  return null;  
}).filter(Boolean);
```

```
return loadModules(sortedNames, true);  
}
```

// Отмена загрузки

```
export function cancelLoad(name) {  
  const moduleInfo = moduleRegistry.get(name);  
  if (moduleInfo && moduleInfo.loading) {  
    // Помечаем как отмененное  
    moduleInfo.loading = false;  
    moduleInfo.promise = null;  
  }  
}
```

// Получение статистики

```
export function getModuleStats() {
  const total = moduleRegistry.size;
  let loaded = 0;
  let loading = 0;
  let size = 0;

  for (const info of moduleRegistry.values()) {
    if (info.loaded) loaded++;
    if (info.loading) loading++;
    // Можно добавить расчет размера
  }

  return {
    total,
    loaded,
    loading,
    pending: total - loaded - loading,
    loadedPercentage: (loaded / total * 100).toFixed(1)
  };
}
```

// Пример использования

// Регистрация модулей

```
registerModule('dashboard', () => import('./modules/dashboard.js'), {
  preload: true,
  priority: 10
});

registerModule('analytics', () => import('./modules/analytics.js'), {
  prefetch: true,
  priority: 5
});

registerModule('settings', () => import('./modules/settings.js'), {
  priority: 3
});
```

// Использование в React компоненте

```
export function createLazyComponent(moduleName, fallback = null) {
  return function LazyWrapper(props) {
    const [Module, setModule] = useState(null);
    const [error, setError] = useState(null);
```



```
useEffect(() => {
  let isMounted = true;

  loadModule(moduleName)
    .then(module => {
      if (isMounted) {
        setModule(() => module.default || module);
      }
    })
    .catch(err => {
      if (isMounted) {
        setError(err);
        console.error(`Ошибка загрузки модуля ${moduleName}:`, err);
      }
    });

  return () => {
    isMounted = false;
    cancelLoad(moduleName);
  };
}, [moduleName]);

if (error) {
  return fallback || <div>Ошибка загрузки модуля</div>;
}

if (!Module) {
  return fallback || <div>Загрузка...</div>;
}

return <Module {...props} />;
};
}
```

Заключение

ES6 модули представляют собой значительный шаг вперед в организации кода JavaScript. Они обеспечивают:

Преимущества:

1. **Статический анализ** – зависимости известны до выполнения
2. **Tree shaking** – удаление неиспользуемого кода
3. **Асинхронная загрузка** – улучшение производительности
4. **Лексическая область видимости** – лучшая изоляция
5. **Строгий режим по умолчанию** – меньше ошибок

Рекомендации:

1. **Используйте именованные экспорты** для лучшего tree shaking
2. **Избегайте побочных эффектов** на верхнем уровне модуля
3. **Используйте динамический импорт** для код-сплиттинга
4. **Создавайте barrel файлы** для организации публичного API
5. **Указывайте sideEffects** в package.json

Когда использовать:

- Новые проекты на Node.js 14+
- Фронтенд с современными сборщиками (Webpack, Vite, Rollup)
- Библиотеки с поддержкой tree shaking
- Приложения с код-сплиттингом

Когда использовать CommonJS:

- Легаси-проекты Node.js
- Скрипты без сборщика
- Когда нужна максимальная совместимость

ES6 модули – это будущее JavaScript, и их освоение критически важно для современной разработки.

12.4. Динамический импорт и ленивая загрузка

Введение: эволюция загрузки модулей

Динамический импорт (Dynamic Import) — это революционная функция ES2020, которая позволяет загружать модули во время выполнения программы, а не во время компиляции. Это открывает новые возможности для оптимизации производительности веб-приложений.

Историческая перспектива:

```
javascript

// ЭВОЛЮЦИЯ ЗАГРУЗКИ МОДУЛЕЙ:

// 1. CommonJS (2009) - синхронная, только для сервера
const module = require('./module.js'); // Блокирующая операция

// 2. ES6 Modules (2015) - статическая, декларативная
import { feature } from './module.js'; // Загружается при парсинге

// 3. Динамический импорт (2020) - асинхронная, по требованию
const module = await import('./module.js'); // Загружается когда нужно

// 4. import.meta (2020) - метаданные модуля
const url = import.meta.url; // URL текущего модуля
```

Зачем нужен динамический импорт:

```
javascript

// ПРОБЛЕМЫ СТАТИЧЕСКОГО ИМПОРТА:
// 1. Весь код загружается сразу
import heavyLibrary from './heavy-library.js'; // 2MB загружаются всегда

// 2. Нет условной загрузки
if (user.isAdmin) {
  // Нельзя загрузить только для админов
  import adminTools from './admin-tools.js'; // Ошибка: импорт должен быть на верхнем уровне
```

```
}

// 3. Нет прогрессивной загрузки
// Пользователь на мобильном получает весь код десктопной версии

// РЕШЕНИЕ - ДИНАМИЧЕСКИЙ ИМПОРТ:
// 1. Загрузка по требованию
const loadFeature = async (featureName) => {
  if (featureName === 'analytics') {
    return await import('./analytics.js'); // Только когда нужно
  }
};

// 2. Условная загрузка
if (user.isPremium) {
  const premiumFeatures = await import('./premium.js');
}

// 3. Прогрессивное улучшение
if ('IntersectionObserver' in window) {
  const lazyLoader = await import('./lazy-loading.js');
}
```

Часть 1: Основы динамического импорта

1.1. Базовый синтаксис

```
javascript

// БАЗОВЫЙ СИНТАКСИС ДИНАМИЧЕСКОГО ИМПОРТА:

// 1. Простой динамический импорт
const loadModule = async (modulePath) => {
  try {
    const module = await import(modulePath);
    console.log('Модуль загружен:', module);
    return module;
  } catch (error) {
    console.error('Ошибка загрузки модуля:', error);
    throw error;
  }
}
```

```
};
```

```
// 2. Динамический импорт с деструктуризацией
```

```
const { utils, constants } = await import('./helpers.js');
```

```
// 3. Импорт с обработкой default экспорта
```

```
const module = await import('./default-export.js');
```

```
const DefaultComponent = module.default;
```

```
// 4. Импорт всего модуля как объекта
```

```
const math = await import('./math.js');
```

```
const result = math.add(2, math.PI);
```

```
// 5. Динамический импорт встроенных модулей Node.js
```

```
const fs = await import('fs/promises');
```

```
const path = await import('path');
```

1.2. Особенности синтаксиса

javascript

```
// РАЗЛИЧНЫЕ СПОСОБЫ ВЫЗОВА import():
```

```
// 1. Стандартный вызов
```

```
const module1 = await import('./module.js');
```

```
// 2. С аргументами (путь вычисляется динамически)
```

```
const feature = 'dashboard';
```

```
const module2 = await import(`./features/${feature}.js`);
```

```
// 3. В выражении
```

```
const loaders = {
```

```
  json: () => import('./json-loader.js'),
```

```
  csv: () => import('./csv-loader.js'),
```

```
  xml: () => import('./xml-loader.js')
```

```
};
```

```
const data = await loaders[format]().then(m => m.parse(content));
```

```
// 4. В цепочке промисов
```

```
import('./config.js')
```

```
  .then(config => import(`./modules/${config.default.module}.js`))
```

```
  .then(module => module.initialize())
```

```
  .catch(error => console.error('Ошибка цепочки:', error));
```

// 5. Параллельная загрузка нескольких модулей

```
const [moduleA, moduleB, moduleC] = await Promise.all([
  import('./module-a.js'),
  import('./module-b.js'),
  import('./module-c.js')
]);
```

// 6. Последовательная загрузка с зависимостями

```
const config = await import('./config.js');
const database = await import(`./databases/${config.default.databaseType}.js`);
const models = await import('./models.js');
const app = await import('./app.js');
```

// 7. В цикле

```
const modules = ['auth', 'users', 'products'];
const loadedModules = {};

for (const name of modules) {
  loadedModules[name] = await import(`./${name}.js`);
}
```

// 8. В асинхронной функции

```
async function initializeApp() {
  const [
    { default: logger },
    { initializeDB },
    { setupRoutes },
    { startServer }
  ] = await Promise.all([
    import('./logger.js'),
    import('./database.js'),
    import('./routes.js'),
    import('./server.js')
  ]);

  logger.info('Инициализация приложения...');
  await initializeDB();
  setupRoutes();
  await startServer();
  logger.info('Приложение запущено');
}
```

1.3. Практические примеры

javascript

// ПРАКТИЧЕСКИЕ ПРИМЕРЫ ДИНАМИЧЕСКОГО ИМПОРТА:

// Пример 1: Загрузка полифиллов по необходимости

```
async function loadPolyfills() {
  const polyfills = [];

  // Проверяем поддержку Fetch API
  if (!window.fetch) {
    polyfills.push(import('whatwg-fetch'));
  }

  // Проверяем поддержку Promise
  if (!window.Promise) {
    polyfills.push(import('promise-polyfill'));
  }

  // Проверяем поддержку IntersectionObserver
  if (!('IntersectionObserver' in window)) {
    polyfills.push(import('intersection-observer'));
  }

  // Загружаем все необходимые полифиллы
  if (polyfills.length > 0) {
    console.log('Загружаем полифиллы...');
    await Promise.all(polyfills);
    console.log('Полифиллы загружены');
  }
}
```

// Пример 2: Загрузка локализации

```
class I18n {
  constructor(defaultLocale = 'en') {
    this.locale = defaultLocale;
    this.translations = {};
    this.loadedLocales = new Set();
  }

  async setLocale(locale) {
    if (this.loadedLocales.has(locale)) {
      this.locale = locale;
    }
  }
}
```

```

    return;
}

try {
    // Динамическая загрузка файла перевода
    const module = await import(`./locales/${locale}.js`);
    this.translations[locale] = module.default;
    this.loadedLocales.add(locale);
    this.locale = locale;

    console.log(`Локализация "${locale}" загружена`);
} catch (error) {
    console.error(`Не удалось загрузить локализацию "${locale}":`, error);

    // Пробуем загрузить fallback
    if (locale !== 'en') {
        console.log('Пробуем загрузить английскую версию...');
        await this.setLocale('en');
    }
}

t(key, params = {}) {
    const translation = this.translations[this.locale]?.[key] || key;

    // Замена параметров
    return translation.replace(/\{(\w+)\}/g, (match, paramName) => {
        return params[paramName] || match;
    });
}

// Предзагрузка нескольких локализаций
async preloadLocales(locales) {
    const loadPromises = locales.map(async (locale) => {
        if (!this.loadedLocales.has(locale)) {
            try {
                const module = await import(`./locales/${locale}.js`);
                this.translations[locale] = module.default;
                this.loadedLocales.add(locale);
            } catch (error) {
                console.warn(`Не удалось предзагрузить "${locale}":`, error.message);
            }
        }
    });
}

```



```
    await Promise.all(loadPromises);
  }
}
```

// Пример 3: Плагиновая система

```
class PluginSystem {
  constructor() {
    this.plugins = new Map();
    this.initializedPlugins = new Set();
  }

  async registerPlugin(name, pluginPath, options = {}) {
    if (this.plugins.has(name)) {
      throw new Error(`Плагин "${name}" уже зарегистрирован`);
    }

    this.plugins.set(name, {
      path: pluginPath,
      options,
      instance: null,
      loaded: false
    });

    // Автозагрузка если указана в опциях
    if (options.autoLoad) {
      await this.loadPlugin(name);
    }

    console.log(`Плагин "${name}" зарегистрирован`);
  }

  async loadPlugin(name) {
    const pluginInfo = this.plugins.get(name);

    if (!pluginInfo) {
      throw new Error(`Плагин "${name}" не найден`);
    }

    if (pluginInfo.loaded) {
      return pluginInfo.instance;
    }

    try {
```

```

// Динамическая загрузка плагина
const module = await import(pluginInfo.path);
const PluginClass = module.default || module;

// Создаем экземпляр плагина
pluginInfo.instance = new PluginClass(pluginInfo.options);
pluginInfo.loaded = true;

// Инициализация если есть метод
if (typeof pluginInfo.instance.initialize === 'function') {
  await pluginInfo.instance.initialize();
  this.initializedPlugins.add(name);
}

console.log(`Плагин "${name}" загружен и инициализирован`);
return pluginInfo.instance;
} catch (error) {
  console.error(`Ошибка загрузки плагина "${name}":`, error);
  throw error;
}
}

async loadAllPlugins() {
  const loadPromises = [];

  for (const [name, pluginInfo] of this.plugins) {
    if (!pluginInfo.loaded) {
      loadPromises.push(this.loadPlugin(name));
    }
  }

  await Promise.allSettled(loadPromises);
  console.log(`Все плагины загружены. Успешно: ${this.initializedPlugins.size}`);
}

getPlugin(name) {
  const pluginInfo = this.plugins.get(name);
  return pluginInfo?.instance || null;
}

async unloadPlugin(name) {
  const pluginInfo = this.plugins.get(name);

  if (!pluginInfo || !pluginInfo.loaded) {

```

```

    return false;
}

// Вызываем деструктор если есть
if (typeof pluginInfo.instance.destroy === 'function') {
    await pluginInfo.instance.destroy();
}

pluginInfo.instance = null;
pluginInfo.loaded = false;
this.initializedPlugins.delete(name);

// Удаляем модуль из кэша (Node.js)
if (typeof require !== 'undefined' && require.cache) {
    const modulePath = require.resolve(pluginInfo.path);
    delete require.cache[modulePath];
}

console.log(`Плагин "${name}" выгружен`);
return true;
}
}

// Пример 4: Загрузка тем оформления
class ThemeManager {
    constructor(defaultTheme = 'light') {
        this.currentTheme = defaultTheme;
        this.themes = new Map();
        this.subscribers = new Set();
    }

    async loadTheme(themeName) {
        // Если тема уже загружена
        if (this.themes.has(themeName)) {
            return this.themes.get(themeName);
        }

        try {
            // Динамическая загрузка темы
            const themeModule = await import(`./themes/${themeName}.js`);
            const theme = themeModule.default || themeModule;

            this.themes.set(themeName, theme);

```

```

// Применяем CSS если есть
if (theme.css) {
  await this.applyThemeCSS(themeName, theme.css);
}

console.log(`Тема "${themeName}" загружена`);
return theme;
} catch (error) {
  console.error(`Ошибка загрузки темы "${themeName}":`, error);
  throw error;
}
}

async applyThemeCSS(themeName, css) {
  // Удаляем предыдущую тему
  const oldLink = document.getElementById('theme-styles');
  if (oldLink) {
    oldLink.remove();
  }

  // Создаем новую ссылку на CSS
  const link = document.createElement('link');
  link.id = 'theme-styles';
  link.rel = 'stylesheet';
  link.href = css;

  // Ждем загрузки CSS
  await new Promise((resolve, reject) => {
    link.onload = resolve;
    link.onerror = reject;
    document.head.appendChild(link);
  });
}

async setTheme(themeName) {
  if (this.currentTheme === themeName) {
    return;
  }

  const oldTheme = this.currentTheme;

  try {
    // Загружаем тему если еще не загружена
    const theme = await this.loadTheme(themeName);
  }
}

```

```

// Применяем тему
this.currentTheme = themeName;

// Сохраняем в localStorage
if (typeof localStorage !== 'undefined') {
  localStorage.setItem('theme', themeName);
}

// Уведомляем подписчиков
this.notifySubscribers(themeName, oldTheme);

console.log(`Тема изменена: ${oldTheme} → ${themeName}`);
} catch (error) {
  console.error(`Не удалось применить тему "${themeName}":`, error);
  throw error;
}
}

// Предзагрузка тем
async preloadThemes(themeNames) {
  const loadPromises = themeNames.map(themeName =>
    this.loadTheme(themeName).catch(error => {
      console.warn(`Не удалось предзагрузить тему "${themeName}":`, error.message);
      return null;
    })
  );

  await Promise.all(loadPromises);
  console.log(`Предзагружено ${themeNames.length} тем`);
}

subscribe(callback) {
  this.subscribers.add(callback);
  return () => this.subscribers.delete(callback);
}

notifySubscribers(newTheme, oldTheme) {
  this.subscribers.forEach(callback => {
    try {
      callback(newTheme, oldTheme);
    } catch (error) {
      console.error('Ошибка в подписке темы:', error);
    }
  });
}

```

```
});  
}  
}
```

Часть 2: Ленивая загрузка (Lazy Loading)

2.1. Основные концепции ленивой загрузки

javascript

// ЛЕНИВАЯ ЗАГРУЗКА - загрузка ресурсов только когда они нужны

// БЕЗ ленивой загрузки:

import HeavyComponent from './HeavyComponent.js'; // Загружается сразу

import LargeImage from './large-image.jpg'; // Загружается сразу

import Analytics from './analytics.js'; // Загружается сразу, даже если пользователь не согласился

// С ленивой загрузкой:

const loadHeavyComponent = () => import('./HeavyComponent.js'); // Загрузится только при вызове

const loadImage = () => import('./large-image.jpg'); // Загрузится при необходимости

const loadAnalytics = () => import('./analytics.js'); // Загрузится после согласия пользователя

2.2. Ленивая загрузка компонентов (React, Vue, Angular)

javascript

// РЕАКТ: Ленивая загрузка компонентов с React.Lazy и Suspense

import React, { Suspense, lazy } from 'react';

// Статический импорт (загружается сразу)

// import Dashboard from './components/Dashboard';

// Ленивая загрузка (загружается только при рендере)

const Dashboard = lazy(() => import('./components/Dashboard.jsx'));

const Analytics = lazy(() => import('./components/Analytics.jsx'));

const Settings = lazy(() => import('./components/Settings.jsx'));

// Компонент с роутингом и ленивой загрузкой

```
function App() {
  const [currentPage, setCurrentPage] = React.useState('dashboard');

  const renderPage = () => {
    switch (currentPage) {
      case 'dashboard':
        return <Dashboard />;
      case 'analytics':
        return <Analytics />;
      case 'settings':
        return <Settings />;
      default:
        return <Dashboard />;
    }
  };

  return (
    <div className="app">
      <nav>
        <button onClick={() => setCurrentPage('dashboard')}>Дашборд</button>
        <button onClick={() => setCurrentPage('analytics')}>Аналитика</button>
        <button onClick={() => setCurrentPage('settings')}>Настройки</button>
      </nav>

      <main>
        <Suspense fallback=<div>Загрузка компонента...</div>>
          {renderPage()}
        </Suspense>
      </main>
    </div>
  );
}
```

// Продвинутая ленивая загрузка с предзагрузкой

```
const preloadMap = {
  dashboard: () => import('./components/Dashboard.jsx'),
  analytics: () => import('./components/Analytics.jsx'),
  settings: () => import('./components/Settings.jsx')
};
```

// Компонент с предзагрузкой при наведении

```
function NavButton({ page, children, onNavigate }) {
  const handleMouseEnter = () => {
```

```

// Предзагружаем компонент при наведении
preloadMap[page]().catch(() => {
  // Игнорируем ошибки предзагрузки
});
};

const handleClick = () => {
  onNavigate(page);
};

return (
  <button
    onMouseEnter={handleMouseEnter}
    onClick={handleClick}
  >
    {children}
  </button>
);
}

// VUE: Ленивая загрузка компонентов
/*
const Dashboard = () => import('./components/Dashboard.vue');
const Analytics = () => import('./components/Analytics.vue');

const routes = [
  {
    path: '/dashboard',
    component: Dashboard,
    meta: { preload: true } // Предзагрузка
  },
  {
    path: '/analytics',
    component: Analytics,
    meta: { preloadOnHover: true } // Предзагрузка при наведении
  }
];
*/

// ANGULAR: Ленивая загрузка модулей
/*
const routes: Routes = [
  {
    path: 'dashboard',

```



```

    LoadChildren: () => import('./dashboard/dashboard.module').then(m => m.DashboardModule)
  },
  {
    path: 'analytics',
    LoadChildren: () => import('./analytics/analytics.module').then(m => m.AnalyticsModule)
  }
];
*/

```

2.3. Ленивая загрузка изображений и медиа

javascript

// LAZY IMAGE LOADER - ленивая загрузка изображений

```

class LazyImageLoader {
  constructor(options = {}) {
    this.options = {
      root: options.root || null,
      rootMargin: options.rootMargin || '50px',
      threshold: options.threshold || 0.1,
      placeholder: options.placeholder ||
'data:image/gif;base64,R0lGODlhAQABAIAAAAAAAP///yH5BAEAAAAALAAAAAABAAEAAAIBRAA7',
      loadedClass: options.loadedClass || 'lazy-loaded',
      loadingClass: options.loadingClass || 'lazy-loading',
      errorClass: options.errorClass || 'lazy-error'
    };

    this.observer = null;
    this.images = new Set();
    this.loadedCount = 0;
    this.failedCount = 0;
  }

  initialize() {
    // Проверяем поддержку IntersectionObserver
    if (!('IntersectionObserver' in window)) {
      console.warn('IntersectionObserver не поддерживается, загружаем все изображения сразу');
      this.loadAllImages();
      return;
    }

    // Создаем наблюдатель

```

```

this.observer = new IntersectionObserver(
  this.handleIntersection.bind(this),
  this.options
);

// Находим все ленивые изображения
const lazyImages = document.querySelectorAll('img[data-src], img[data-srcset]');

lazyImages.forEach(img => {
  this.images.add(img);
  this.observer.observe(img);

  // Добавляем классы
  img.classList.add(this.options.loadingClass);

  // Устанавливаем placeholder
  if (!img.src) {
    img.src = this.options.placeholder;
  }
});

console.log(`Найдено ${lazyImages.length} ленивых изображений`);
}

handleIntersection(entries) {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      this.loadImage(img);
      this.observer.unobserve(img);
    }
  });
}

async loadImage(img) {
  try {
    // Получаем URL из data-атрибутов
    const src = img.dataset.src;
    const srcset = img.dataset.srcset;

    if (!src && !srcset) {
      console.warn('Изображение без data-src или data-srcset:', img);
      return;
    }
  }
}

```

```

// Создаем промис для загрузки изображения
const loadPromise = new Promise((resolve, reject) => {
  const image = new Image();

  image.onload = () => {
    // Применяем реальный src/srcset
    if (src) img.src = src;
    if (srcset) img.srcset = srcset;

    // Удаляем data-атрибуты
    delete img.dataset.src;
    delete img.dataset.srcset;

    // Обновляем классы
    img.classList.remove(this.options.loadingClass);
    img.classList.add(this.options.loadedClass);

    this.loadedCount++;
    resolve(img);
  };

  image.onerror = (error) => {
    img.classList.remove(this.options.loadingClass);
    img.classList.add(this.options.errorClass);
    this.failedCount++;
    reject(error);
  };

  // Начинаем загрузку
  if (src) image.src = src;
  if (srcset) image.srcset = srcset;
});

// Добавляем задержку для демонстрации
if (this.options.delay) {
  await new Promise(resolve => setTimeout(resolve, this.options.delay));
}

await loadPromise;

// Генерируем событие
img.dispatchEvent(new CustomEvent('lazyload:loaded', {
  detail: { img, timestamp: Date.now() }
}));

```

```

    }));

    } catch (error) {
        console.error('Ошибка загрузки изображения:', error, img);

        // Генерируем событие ошибки
        img.dispatchEvent(new CustomEvent('lazyload:error', {
            detail: { img, error, timestamp: Date.now() }
        }));
    }
}

// Принудительная загрузка всех изображений
loadAllImages() {
    const promises = Array.from(this.images).map(img => this.loadImage(img));
    return Promise.allSettled(promises);
}

// Предзагрузка изображения
preloadImage(url) {
    return new Promise((resolve, reject) => {
        const img = new Image();
        img.onload = () => resolve(img);
        img.onerror = reject;
        img.src = url;
    });
}

// Предзагрузка нескольких изображений
async preloadImages(urls) {
    const promises = urls.map(url =>
        this.preloadImage(url).catch(error => {
            console.warn(`Не удалось предзагрузить изображение ${url}:`, error);
            return null;
        })
    );
};

const results = await Promise.allSettled(promises);
const successful = results.filter(r => r.status === 'fulfilled').length;

console.log(`Предзагружено ${successful}/${urls.length} изображений`);
return results;
}

```

```
// Статистика
getStats() {
  return {
    total: this.images.size,
    loaded: this.loadedCount,
    failed: this.failedCount,
    pending: this.images.size - this.loadedCount - this.failedCount
  };
}

// Очистка
destroy() {
  if (this.observer) {
    this.observer.disconnect();
    this.observer = null;
  }
  this.images.clear();
}

// Использование
document.addEventListener('DOMContentLoaded', () => {
  const lazyLoader = new LazyImageLoader({
    rootMargin: '100px',
    threshold: 0.01,
    delay: 300 // Имитация медленной сети для демонстрации
  });

  lazyLoader.initialize();

  // Предзагрузка важных изображений
  const criticalImages = [
    '/images/hero-banner.jpg',
    '/images/logo.png',
    '/images/user-avatar.jpg'
  ];

  lazyLoader.preloadImages(criticalImages);

  // Мониторинг прогресса
  setInterval(() => {
    const stats = lazyLoader.getStats();
    console.log(`Статистика загрузки: ${stats.loaded}/${stats.total}`);
  }, 5000);
});
```

```
});
```

2.4. Ленивая загрузка CSS и шрифтов

```
javascript
```

```
// LAZY CSS LOADER - ленивая загрузка стилей
```

```
class LazyCSSLoader {
  constructor() {
    this.loadedStylesheets = new Set();
    this.pendingPromises = new Map();
    this.observers = new Map();
  }

  // Ленивая загрузка CSS файла
  async loadCSS(href, options = {}) {
    // Если уже загружен
    if (this.loadedStylesheets.has(href)) {
      return Promise.resolve();
    }

    // Если уже загружается
    if (this.pendingPromises.has(href)) {
      return this.pendingPromises.get(href);
    }

    const loadPromise = new Promise((resolve, reject) => {
      const link = document.createElement('link');
      link.rel = 'stylesheet';
      link.href = href;
      link.media = options.media || 'all';

      if (options.id) {
        link.id = options.id;
      }

      if (options.crossOrigin) {
        link.crossOrigin = options.crossOrigin;
      }

      link.onload = () => {
        this.loadedStylesheets.add(href);
        this.pendingPromises.delete(href);
      };
    });
  }
}
```

```

    resolve();
};

link.onerror = (error) => {
    this.pendingPromises.delete(href);
    reject(new Error(`Не удалось загрузить CSS: ${href}`, { cause: error }));
};

// Вставляем в head
const insertPosition = options.prepend ?
    document.head.firstChild :
    document.head.lastChild;

document.head.insertBefore(link, insertPosition);
});

this.pendingPromises.set(href, loadPromise);
return loadPromise;
}

// Ленивая загрузка CSS по условию
async loadCSSWhenVisible(selector, href, options = {}) {
    const elements = document.querySelectorAll(selector);

    if (elements.length === 0) {
        console.warn(`Элементы не найдены: ${selector}`);
        return;
    }

    // Создаем наблюдатель для каждого элемента
    elements.forEach(element => {
        const observer = new IntersectionObserver((entries) => {
            entries.forEach(entry => {
                if (entry.isIntersecting) {
                    this.loadCSS(href, options).then(() => {
                        // Добавляем класс после загрузки
                        element.classList.add(options.loadedClass || 'css-loaded');
                    });
                }

                observer.disconnect();
                this.observers.delete(element);
            }
        });
    });
}, {

```

```

    root: options.root || null,
    rootMargin: options.rootMargin || '50px',
    threshold: options.threshold || 0.1
  });

  observer.observe(element);
  this.observers.set(element, observer);
});
}

// Ленивая загрузка шрифтов
async loadFont(fontFamily, fontUrl, options = {}) {
  // Создаем элемент style для font-face
  const style = document.createElement('style');

  style.textContent = `
    @font-face {
      font-family: '${fontFamily}';
      src: url('${fontUrl}') format('${options.format || 'woff2'}');
      font-weight: ${options.weight || 'normal'};
      font-style: ${options.style || 'normal'};
      font-display: ${options.display || 'swap'};
    }
  `;

  // Ждем загрузки шрифта
  return new Promise((resolve, reject) => {
    // Проверяем загрузку через FontFace API если поддерживается
    if ('fonts' in document) {
      const font = new FontFace(fontFamily, `url('${fontUrl}')`, options);

      font.load().then(() => {
        document.fonts.add(font);
        document.head.appendChild(style);
        resolve();
      }).catch(reject);
    } else {
      // Fallback: просто вставляем style
      document.head.appendChild(style);

      // Проверяем загрузку через таймер
      const checkFont = () => {
        if (document.fonts && document.fonts.check) {
          if (document.fonts.check(`16px "${fontFamily}"`)) {

```



```

        resolve();
    } else {
        setTimeout(checkFont, 100);
    }
    } else {
        // Если API не поддерживается, считаем что загружено
        setTimeout(resolve, 1000);
    }
};

checkFont();
}
});
}

// Ленивая загрузка критического CSS
async loadCriticalCSS(selectors, href) {
    // Сначала загружаем CSS
    await this.loadCSS(href);

    // Затем извлекаем и вставляем критические стили
    const response = await fetch(href);
    const cssText = await response.text();

    // Парсим CSS и извлекаем стили для указанных селекторов
    const criticalRules = this.extractCriticalRules(cssText, selectors);

    // Вставляем критические стили инлайново
    const style = document.createElement('style');
    style.textContent = criticalRules;
    style.setAttribute('data-critical', 'true');
    document.head.insertBefore(style, document.head.firstChild);

    return criticalRules;
}

extractCriticalRules(cssText, selectors) {
    // Упрощенный парсер CSS (в реальности нужно использовать парсер)
    const rules = [];
    const selectorRegex = /([^{]+)\{([^{]+)\}/g;
    let match;

    while ((match = selectorRegex.exec(cssText)) !== null) {
        const ruleSelectors = match[1].split(',').map(s => s.trim());

```

```

const ruleContent = match[2];

// Проверяем, есть ли пересечение с критическими селекторами
const hasCriticalSelector = ruleSelectors.some(ruleSelector => {
  return selectors.some(criticalSelector => {
    // Простая проверка (в реальности нужен парсер селекторов)
    return ruleSelector.includes(criticalSelector) ||
      criticalSelector.includes(ruleSelector);
  });
});

if (hasCriticalSelector) {
  rules.push(`${match[1]} ${ruleContent}`);
}
}

return rules.join('\n');
}

// Очистка
destroy() {
  this.observers.forEach(observer => observer.disconnect());
  this.observers.clear();
  this.pendingPromises.clear();
}
}

// Использование
const cssLoader = new LazyCSSLoader();

// Загрузка CSS когда элемент становится видимым
cssLoader.loadCSSWhenVisible('.chart-container', '/css/charts.css', {
  loadedClass: 'charts-ready'
});

// Загрузка шрифта
cssLoader.loadFont('Roboto', '/fonts/roboto.woff2', {
  weight: '400',
  display: 'swap'
});

// Загрузка критического CSS
cssLoader.loadCriticalCSS(
  ['.header', '.hero', '.nav', '.button'],

```

```
'/css/main.css'  
);
```

Часть 3: Продвинутые техники динамической загрузки

3.1. Стратегии загрузки

javascript

// STRATEGY PATTERN для динамической загрузки

```
class LoadingStrategy {  
  constructor(options = {}) {  
    this.options = {  
      cache: true,  
      retry: 3,  
      timeout: 10000,  
      priority: 'medium',  
      ...options  
    };  
  
    this.cache = new Map();  
    this.stats = {  
      hits: 0,  
      misses: 0,  
      errors: 0,  
      totalSize: 0  
    };  
  }  
  
  async load(resource) {  
    throw new Error('Метод load должен быть реализован в подклассе');  
  }  
  
  getStats() {  
    return { ...this.stats };  
  }  
  
  clearCache() {  
    this.cache.clear();  
    this.stats.totalSize = 0;  
  }  
}
```

```
}  
}
```

```
// Стратегия 1: Предзагрузка (PreLoad)
```

```
class PreloadStrategy extends LoadingStrategy {  
  constructor(options) {  
    super(options);  
    this.preloaded = new Set();  
  }
```

```
  async load(resource) {  
    // Если уже загружено  
    if (this.cache.has(resource)) {  
      this.stats.hits++;  
      return this.cache.get(resource);  
    }
```

```
// Если уже предзагружается
```

```
    if (this.preloaded.has(resource)) {  
      return new Promise((resolve) => {  
        const checkInterval = setInterval(() => {  
          if (this.cache.has(resource)) {  
            clearInterval(checkInterval);  
            resolve(this.cache.get(resource));  
          }  
        }, 100);  
      });  
    }
```

```
    this.stats.misses++;  
    this.preloaded.add(resource);
```

```
    try {  
      const module = await import(resource);  
  
      if (this.options.cache) {  
        this.cache.set(resource, module);  
      }
```

```
      this.preloaded.delete(resource);  
      return module;  
    } catch (error) {  
      this.stats.errors++;  
      this.preloaded.delete(resource);  
    }
```

```

        throw error;
    }
}

// Массовая предзагрузка
async preloadAll(resources) {
    const promises = resources.map(resource =>
        this.load(resource).catch(error => {
            console.warn(`Не удалось предзагрузить ${resource}:`, error.message);
            return null;
        })
    );

    await Promise.all(promises);
    console.log(`Предзагружено ${resources.length} ресурсов`);
}

// Стратегия 2: Загрузка по требованию (On Demand)
class OnDemandStrategy extends LoadingStrategy {
    async load(resource) {
        // Если уже в кэше
        if (this.options.cache && this.cache.has(resource)) {
            this.stats.hits++;
            return this.cache.get(resource);
        }

        this.stats.misses++;

        // Загружаем с ретраями
        for (let attempt = 1; attempt <= this.options.retry; attempt++) {
            try {
                const controller = new AbortController();
                const timeoutId = setTimeout(() => controller.abort(), this.options.timeout);

                const module = await import(resource);

                clearTimeout(timeoutId);

                if (this.options.cache) {
                    this.cache.set(resource, module);
                }

                return module;
            }
        }
    }
}

```

```

    } catch (error) {
      if (attempt === this.options.retry) {
        this.stats.errors++;
        throw error;
      }

      console.warn(`Попытка ${attempt}/${this.options.retry} не удалась для ${resource}`);
      await this.delay(1000 * attempt); // Экспоненциальная задержка
    }
  }
}

delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
}

// Стратегия 3: Прогрессивная загрузка (Progressive)
class ProgressiveStrategy extends LoadingStrategy {
  constructor(options) {
    super(options);
    this.connection = navigator.connection || {
      effectiveType: '4g',
      saveData: false
    };
  }

  async load(resource) {
    // Определяем качество соединения
    const connectionType = this.connection.effectiveType;
    const isSlow = connectionType === '2g' || connectionType === '3g' ||
this.connection.saveData;

    if (isSlow) {
      // Для медленного соединения загружаем легковесную версию
      const lightweightResource = resource.replace('.js', '.lite.js');

      try {
        return await this.loadWithFallback(lightweightResource, resource);
      } catch (error) {
        // Если легковесной версии нет, загружаем обычную
        return await import(resource);
      }
    } else {

```

```

    // Для быстрого соединения загружаем полную версию
    return await import(resource);
  }
}

async loadWithFallback(primary, fallback) {
  try {
    return await import(primary);
  } catch (error) {
    console.log(`Первичный ресурс ${primary} не найден, пробуем ${fallback}`);
    return await import(fallback);
  }
}
}

```

```

// Стратегия 4: Приоритетная загрузка (Priority)
class PriorityStrategy extends LoadingStrategy {
  constructor(options) {
    super(options);
    this.priorityQueue = [];
    this.loading = false;
  }

  async load(resource, priority = 'medium') {
    // Добавляем в очередь с приоритетом
    const task = {
      resource,
      priority: this.getPriorityValue(priority),
      resolve: null,
      reject: null,
      promise: null
    };

    task.promise = new Promise((resolve, reject) => {
      task.resolve = resolve;
      task.reject = reject;
    });

    this.priorityQueue.push(task);
    this.priorityQueue.sort((a, b) => b.priority - a.priority);

    // Запускаем обработчик очереди если не запущен
    if (!this.loading) {
      this.processQueue();
    }
  }
}

```

```

    }

    return task.promise;
}

getPriorityValue(priority) {
    const priorities = {
        'critical': 100,
        'high': 75,
        'medium': 50,
        'low': 25,
        'background': 1
    };

    return priorities[priority] || 50;
}

async processQueue() {
    this.loading = true;

    while (this.priorityQueue.length > 0) {
        const task = this.priorityQueue.shift();

        try {
            const module = await import(task.resource);
            task.resolve(module);
        } catch (error) {
            task.reject(error);
        }

        // Задержка между загрузками для избежания перегрузки
        await this.delay(100);
    }

    this.loading = false;
}

delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

// Фабрика стратегий
class StrategyFactory {

```



```

static create(strategyName, options = {}) {
  const strategies = {
    'preload': PreloadStrategy,
    'on-demand': OnDemandStrategy,
    'progressive': ProgressiveStrategy,
    'priority': PriorityStrategy
  };

  const StrategyClass = strategies[strategyName];

  if (!StrategyClass) {
    throw new Error(`Неизвестная стратегия: ${strategyName}`);
  }

  return new StrategyClass(options);
}

// Использование
const strategy = StrategyFactory.create('progressive', {
  cache: true,
  retry: 2
});

// Загрузка с автоматическим определением стратегии
strategy.load('/modules/analytics.js').then(module => {
  console.log('Модуль загружен:', module);
});

```

3.2. Кэширование и состояние модулей

javascript

// ADVANCED MODULE CACHE - интеллектуальное кэширование модулей

```

class ModuleCache {
  constructor(options = {}) {
    this.options = {
      maxSize: options.maxSize || 100,
      ttl: options.ttl || 300000, // 5 минут
      cleanupInterval: options.cleanupInterval || 60000, // 1 минута
      memoryLimit: options.memoryLimit || 50 * 1024 * 1024, // 50MB
      strategy: options.strategy || 'lru' // lru, lfu, fifo
    };
  }
}

```

```

this.cache = new Map();
this.stats = {
  hits: 0,
  misses: 0,
  evictions: 0,
  totalLoaded: 0,
  memoryUsage: 0
};

this.accessTimestamps = new Map();
this.accessCounts = new Map();

// Запускаем очистку устаревших записей
if (this.options.ttl > 0) {
  this.cleanupInterval = setInterval(
    () => this.cleanup(),
    this.options.cleanupInterval
  );
}

// Мониторинг памяти
if (this.options.memoryLimit > 0) {
  this.memoryMonitorInterval = setInterval(
    () => this.checkMemory(),
    30000
  );
}
}

async get(modulePath, loader = null) {
  const cacheKey = this.normalizeKey(modulePath);

  // Проверяем кэш
  if (this.cache.has(cacheKey)) {
    const entry = this.cache.get(cacheKey);

    // Проверяем TTL
    if (this.options.ttl > 0 && Date.now() - entry.timestamp > this.options.ttl) {
      console.log(`Кэш для ${modulePath} устарел, обновляем`);
      this.cache.delete(cacheKey);
      this.stats.misses++;
    } else {
      // Обновляем статистику доступа

```

```

    this.accessTimestamps.set(cacheKey, Date.now());
    this.accessCounts.set(cacheKey, (this.accessCounts.get(cacheKey) || 0) + 1);
    this.stats.hits++;

    return entry.module;
  }
}

this.stats.misses++;

// Загружаем модуль
let module;
if (loader) {
  module = await loader();
} else {
  module = await import(modulePath);
}

// Сохраняем в кэш
this.set(cacheKey, module);

return module;
}

set(cacheKey, module) {
  // Проверяем размер кэша
  if (this.cache.size >= this.options.maxSize) {
    this.evict();
  }

  // Оцениваем размер модуля
  const size = this.estimateSize(module);

  // Сохраняем в кэш
  this.cache.set(cacheKey, {
    module,
    timestamp: Date.now(),
    size
  });

  this.accessTimestamps.set(cacheKey, Date.now());
  this.accessCounts.set(cacheKey, 1);

  this.stats.totalLoaded++;

```

```

    this.stats.memoryUsage += size;
}

// Удаление записи по стратегии
evict() {
    let keyToEvict;

    switch (this.options.strategy) {
        case 'lru': // Least Recently Used
            keyToEvict = this.findLRU();
            break;

        case 'lfu': // Least Frequently Used
            keyToEvict = this.findLFU();
            break;

        case 'fifo': // First In First Out
            keyToEvict = this.findFIFO();
            break;

        default:
            keyToEvict = this.findLRU();
    }

    if (keyToEvict) {
        const entry = this.cache.get(keyToEvict);
        this.cache.delete(keyToEvict);
        this.accessTimestamps.delete(keyToEvict);
        this.accessCounts.delete(keyToEvict);

        this.stats.evictions++;
        this.stats.memoryUsage -= entry.size;

        console.log(`Эвакуирован из кэша: ${keyToEvict}`);
    }
}

findLRU() {
    let oldestKey = null;
    let oldestTime = Infinity;

    for (const [key, timestamp] of this.accessTimestamps) {
        if (timestamp < oldestTime) {
            oldestTime = timestamp;
        }
    }
    return oldestKey;
}

```

```

    oldestKey = key;
  }
}

return oldestKey;
}

findLFU() {
  let leastUsedKey = null;
  let leastCount = Infinity;

  for (const [key, count] of this.accessCounts) {
    if (count < leastCount) {
      leastCount = count;
      leastUsedKey = key;
    }
  }

  return leastUsedKey;
}

findFIFO() {
  // Первый ключ в Map
  return this.cache.keys().next().value;
}

// Очистка устаревших записей
cleanup() {
  if (this.options.ttl <= 0) return;

  const now = Date.now();
  let cleaned = 0;

  for (const [key, entry] of this.cache) {
    if (now - entry.timestamp > this.options.ttl) {
      this.cache.delete(key);
      this.accessTimestamps.delete(key);
      this.accessCounts.delete(key);

      this.stats.memoryUsage -= entry.size;
      cleaned++;
    }
  }
}

```

```

    if (cleaned > 0) {
        console.log(`Очищено ${cleaned} устаревших записей из кэша`);
    }
}

// Проверка использования памяти
checkMemory() {
    if (this.stats.memoryUsage > this.options.memoryLimit) {
        console.warn(`Превышен лимит памяти кэша: ${this.formatBytes(this.stats.memoryUsage)}`);
        this.evict(); // Эвакуируем одну запись
    }
}

// Оценка размера объекта
estimateSize(obj) {
    try {
        const json = JSON.stringify(obj);
        return new TextEncoder().encode(json).length;
    } catch {
        // Если не удалось сериализовать, используем приблизительную оценку
        return 1024; // 1KB по умолчанию
    }
}

normalizeKey(modulePath) {
    // Нормализуем ключ для кэша
    return modulePath.replace(/^\.\/\//, '').replace(/\.js$/, '');
}

formatBytes(bytes) {
    const units = ['B', 'KB', 'MB', 'GB'];
    let size = bytes;
    let unitIndex = 0;

    while (size >= 1024 && unitIndex < units.length - 1) {
        size /= 1024;
        unitIndex++;
    }

    return `${size.toFixed(2)} ${units[unitIndex]}`;
}

// Предзагрузка модулей

```

```

async preload(modulePaths, concurrency = 3) {
  console.log(`Предзагрузка ${modulePaths.length} модулей...`);

  // Разбиваем на чанки для контроля параллелизма
  const chunks = [];
  for (let i = 0; i < modulePaths.length; i += concurrency) {
    chunks.push(modulePaths.slice(i, i + concurrency));
  }

  for (const chunk of chunks) {
    await Promise.allSettled(
      chunk.map(path => this.get(path).catch(() => null))
    );

    console.log(`Загружено ${this.stats.totalLoaded} модулей`);
  }
}

// Получение статистики
getStats() {
  return {
    ...this.stats,
    cacheSize: this.cache.size,
    hitRatio: this.stats.hits / (this.stats.hits + this.stats.misses || 1),
    avgModuleSize: this.stats.memoryUsage / this.cache.size || 0,
    memoryUsageFormatted: this.formatBytes(this.stats.memoryUsage)
  };
}

// Очистка кэша
clear() {
  this.cache.clear();
  this.accessTimestamps.clear();
  this.accessCounts.clear();

  this.stats.memoryUsage = 0;
  this.stats.totalLoaded = 0;

  console.log('Кэш очищен');
}

// Уничтожение
destroy() {
  if (this.cleanupInterval) {

```

```

    clearInterval(this.cleanupInterval);
  }

  if (this.memoryMonitorInterval) {
    clearInterval(this.memoryMonitorInterval);
  }

  this.clear();
}
}

// Использование
const moduleCache = new ModuleCache({
  maxSize: 50,
  ttl: 600000, // 10 минут
  strategy: 'lru'
});

// Загрузка модуля с кэшированием
async function loadWithCache(modulePath) {
  return await moduleCache.get(modulePath);
}

// Предзагрузка часто используемых модулей
const commonModules = [
  './utils/helpers.js',
  './utils/format.js',
  './api/client.js',
  './components/Modal.js',
  './components/Toast.js'
];

moduleCache.preload(commonModules, 2);

// Мониторинг статистики
setInterval(() => {
  const stats = moduleCache.getStats();
  console.log('Статистика кэша:', stats);
}, 10000);

```

3.3. Мониторинг и аналитика загрузки

javascript

// LOADING ANALYTICS - мониторинг загрузки модулей

```
class LoadingAnalytics {
  constructor(options = {}) {
    this.options = {
      endpoint: options.endpoint || '/api/analytics/loading',
      sampleRate: options.sampleRate || 0.1, // 10% событий
      batchSize: options.batchSize || 10,
      flushInterval: options.flushInterval || 10000, // 10 секунд
      enabled: options.enabled !== false
    };

    this.events = [];
    this.metrics = {
      totalLoads: 0,
      successfulLoads: 0,
      failedLoads: 0,
      totalLoadTime: 0,
      cacheHits: 0,
      cacheMisses: 0,
      moduleSizes: {}
    };

    this.resourceTimers = new Map();

    // Запускаем периодическую отправку данных
    if (this.options.enabled) {
      this.flushInterval = setInterval(() => this.flush(), this.options.flushInterval);

      // Отправляем данные при закрытии страницы
      window.addEventListener('beforeunload', () => this.flush(true));
    }
  }

  // Начало загрузки модуля
  startLoad(modulePath) {
    if (!this.options.enabled || Math.random() > this.options.sampleRate) {
      return;
    }

    const loadId = `${modulePath}-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;

    this.resourceTimers.set(loadId, {
```

```

    modulePath,
    startTime: performance.now(),
    startTimestamp: new Date().toISOString()
  });

  return loadId;
}

// Успешная загрузка модуля
endLoad(loadId, module, fromCache = false) {
  if (!this.options.enabled || !this.resourceTimers.has(loadId)) {
    return;
  }

  const timer = this.resourceTimers.get(loadId);
  const loadTime = performance.now() - timer.startTime;

  // Обновляем метрики
  this.metrics.totalLoads++;
  this.metrics.successfulLoads++;
  this.metrics.totalLoadTime += loadTime;

  if (fromCache) {
    this.metrics.cacheHits++;
  } else {
    this.metrics.cacheMisses++;
  }

  // Оцениваем размер модуля
  const size = this.estimateModuleSize(module);
  this.metrics.moduleSizes[timer.modulePath] = size;

  // Создаем событие
  const event = {
    type: 'module_load',
    modulePath: timer.modulePath,
    loadId,
    loadTime,
    size,
    fromCache,
    success: true,
    timestamp: timer.startTimestamp,
    endTimestamp: new Date().toISOString(),
    userAgent: navigator.userAgent,
  };

```

```

    connection: navigator.connection ? {
      effectiveType: navigator.connection.effectiveType,
      downlink: navigator.connection.downlink,
      rtt: navigator.connection.rtt,
      saveData: navigator.connection.saveData
    } : null
  };

  this.events.push(event);
  this.resourceTimers.delete(loadId);

  // Проверяем нужно ли отправить события
  if (this.events.length >= this.options.batchSize) {
    this.flush();
  }
}

// Ошибка загрузки модуля
errorLoad(loadId, error) {
  if (!this.options.enabled || !this.resourceTimers.has(loadId)) {
    return;
  }

  const timer = this.resourceTimers.get(loadId);
  const loadTime = performance.now() - timer.startTime;

  // Обновляем метрики
  this.metrics.totalLoads++;
  this.metrics.failedLoads++;

  // Создаем событие
  const event = {
    type: 'module_load_error',
    modulePath: timer.modulePath,
    loadId,
    loadTime,
    error: error.message,
    errorType: error.name,
    success: false,
    timestamp: timer.startTimestamp,
    endTimestamp: new Date().toISOString(),
    userAgent: navigator.userAgent
  };

```

```

this.events.push(event);
this.resourceTimers.delete(loadId);

// Проверяем нужно ли отправить события
if (this.events.length >= this.options.batchSize) {
    this.flush();
}
}

// Оценка размера модуля
estimateModuleSize(module) {
    try {
        const json = JSON.stringify(module);
        return new Blob([json]).size;
    } catch {
        // Если не удалось сериализовать
        return 0;
    }
}

// Отправка данных на сервер
async flush(immediate = false) {
    if (this.events.length === 0 || !this.options.enabled) {
        return;
    }

    // Если не immediate и событий мало, ждем больше
    if (!immediate && this.events.length < this.options.batchSize) {
        return;
    }

    const eventsToSend = [...this.events];
    this.events = [];

    const payload = {
        events: eventsToSend,
        metrics: this.metrics,
        timestamp: new Date().toISOString(),
        sessionId: this.getSessionId(),
        pageUrl: window.location.href
    };

    try {
        // Используем sendBeacon для надежной отправки при закрытии страницы

```

```

if (immediate && navigator.sendBeacon) {
  const blob = new Blob([JSON.stringify(payload)], { type: 'application/json' });
  navigator.sendBeacon(this.options.endpoint, blob);
} else {
  // Обычный fetch
  await fetch(this.options.endpoint, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
    keepalive: immediate // Для отправки при закрытии страницы
  });
}

console.log(`Отправлено ${eventsToSend.length} событий загрузки`);
} catch (error) {
  console.error('Ошибка отправки аналитики:', error);

  // Возвращаем события обратно в очередь
  this.events.unshift(...eventsToSend);
}
}

// Генерация ID сессии
getSessionId() {
  let sessionId = sessionStorage.getItem('loading_analytics_session');

  if (!sessionId) {
    sessionId = `session_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
    sessionStorage.setItem('loading_analytics_session', sessionId);
  }

  return sessionId;
}

// Получение сводной статистики
getSummary() {
  const avgLoadTime = this.metrics.totalLoads > 0
    ? this.metrics.totalLoadTime / this.metrics.totalLoads
    : 0;

  const successRate = this.metrics.totalLoads > 0
    ? (this.metrics.successfulLoads / this.metrics.totalLoads) * 100
    : 0;

```

```

const cacheHitRate = (this.metrics.cacheHits + this.metrics.cacheMisses) > 0
  ? (this.metrics.cacheHits / (this.metrics.cacheHits + this.metrics.cacheMisses)) * 100
  : 0;

// Самые большие модули
const largestModules = Object.entries(this.metrics.moduleSizes)
  .sort(([, sizeA], [, sizeB]) => sizeB - sizeA)
  .slice(0, 5)
  .map(([path, size]) => ({ path, size }));

// Самые медленные модули
const slowModules = this.events
  .filter(e => e.type === 'module_load' && e.success)
  .sort((a, b) => b.loadTime - a.loadTime)
  .slice(0, 5)
  .map(e => ({ path: e.modulePath, loadTime: e.loadTime }));

return {
  totalLoads: this.metrics.totalLoads,
  successfulLoads: this.metrics.successfulLoads,
  failedLoads: this.metrics.failedLoads,
  avgLoadTime: avgLoadTime.toFixed(2),
  successRate: successRate.toFixed(1),
  cacheHitRate: cacheHitRate.toFixed(1),
  largestModules,
  slowModules,
  pendingEvents: this.events.length
};
}

// Рекомендации по оптимизации
getRecommendations() {
  const summary = this.getSummary();
  const recommendations = [];

  // Проверяем успешность загрузки
  if (summary.successRate < 95) {
    recommendations.push({
      type: 'error_rate',
      severity: 'high',
      message: `Высокий уровень ошибок загрузки: ${ (100 - summary.successRate).toFixed(1) }%`,
      suggestion: 'Проверьте доступность модулей и добавьте retry логику'
    });
  }
}

```

```
}
```

```
// Проверяем время загрузки
```

```
if (summary.avgLoadTime > 1000) {  
  recommendations.push({  
    type: 'load_time',  
    severity: 'medium',  
    message: `Среднее время загрузки ${summary.avgLoadTime}ms`,  
    suggestion: 'Рассмотрите код-сплиттинг и предзагрузку критических модулей'  
  });  
}
```

```
// Проверяем кэширование
```

```
if (summary.cacheHitRate < 30) {  
  recommendations.push({  
    type: 'cache_usage',  
    severity: 'low',  
    message: `Низкий процент попаданий в кэш: ${summary.cacheHitRate}%`,  
    suggestion: 'Увеличьте TTL кэша и добавьте предзагрузку часто используемых модулей'  
  });  
}
```

```
// Самые большие модули
```

```
if (summary.largestModules.length > 0) {  
  const largest = summary.largestModules[0];  
  if (largest.size > 1024 * 1024) { // Больше 1MB  
    recommendations.push({  
      type: 'module_size',  
      severity: 'high',  
      message: `Модуль ${largest.path} слишком большой: ${this.formatBytes(largest.size)}`,  
      suggestion: 'Разбейте модуль на части или используйте динамический импорт'  
    });  
  }  
}
```

```
return recommendations;
```

```
}
```

```
formatBytes(bytes) {
```

```
  if (bytes === 0) return '0 B';
```

```
  const k = 1024;
```

```
  const sizes = ['B', 'KB', 'MB', 'GB'];
```

```

    const i = Math.floor(Math.log(bytes) / Math.log(k));

    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i];
}

// Уничтожение
destroy() {
    if (this.flushInterval) {
        clearInterval(this.flushInterval);
    }

    this.flush(true); // Отправляем оставшиеся события
}

// Декоратор для автоматического отслеживания загрузки
function withAnalytics(importFn, analytics) {
    return async function trackedImport() {
        const loadId = analytics.startLoad(importFn.toString());

        try {
            const module = await importFn();
            analytics.endLoad(loadId, module);
            return module;
        } catch (error) {
            analytics.errorLoad(loadId, error);
            throw error;
        }
    };
}

// Использование
const analytics = new LoadingAnalytics({
    endpoint: '/api/loading-metrics',
    sampleRate: 0.2 // 20% событий
});

// Оборачиваем динамический импорт в отслеживаемую версию
const trackedImport = (modulePath) =>
    withAnalytics(() => import(modulePath), analytics)();

// Использование отслеживаемого импорта
async function loadFeature(featureName) {
    return await trackedImport(`./features/${featureName}.js`);
}

```



```

}

// Мониторинг в реальном времени
setInterval(() => {
  const summary = analytics.getSummary();
  const recommendations = analytics.getRecommendations();

  console.log('=== Сводка загрузки модулей ===');
  console.log('Всего загрузок:', summary.totalLoads);
  console.log('Успешных:', summary.successfulLoads);
  console.log('Ошибок:', summary.failedLoads);
  console.log('Среднее время:', summary.avgLoadTime + 'ms');
  console.log('Процент кэша:', summary.cacheHitRate + '%');

  if (recommendations.length > 0) {
    console.log('\n=== Рекомендации ===');
    recommendations.forEach(rec => {
      console.log(`[${rec.severity.toUpperCase()}] ${rec.message}`);
      console.log(`  ${rec.suggestion}`);
    });
  }
}, 30000);

```

Часть 4: Практические паттерны и антипаттерны

4.1. Паттерны для различных сценариев

javascript

// ПАТТЕРН 1: Стратегическая загрузка для разных устройств

```

class DeviceAwareLoader {
  constructor() {
    this.device = this.detectDevice();
    this.connection = this.detectConnection();
    this.userPreferences = this.loadPreferences();
  }

  detectDevice() {
    const ua = navigator.userAgent;

```

```
if (/mobile|android|iphone|ipad|ipod/i.test(ua)) {
  return {
    type: 'mobile',
    isTouch: true,
    screenSize: {
      width: window.screen.width,
      height: window.screen.height
    }
  };
} else if (/tablet/i.test(ua)) {
  return {
    type: 'tablet',
    isTouch: true,
    screenSize: {
      width: window.screen.width,
      height: window.screen.height
    }
  };
} else {
  return {
    type: 'desktop',
    isTouch: false,
    screenSize: {
      width: window.screen.width,
      height: window.screen.height
    }
  };
}
}
```

```
detectConnection() {
  const conn = navigator.connection || {
    effectiveType: '4g',
    saveData: false
  };

  return {
    type: conn.effectiveType,
    saveData: conn.saveData,
    downlink: conn.downlink,
    rtt: conn.rtt
  };
}
```

```

loadPreferences() {
  const prefs = localStorage.getItem('user_preferences');

  if (prefs) {
    return JSON.parse(prefs);
  }

  return {
    dataSaver: false,
    animations: true,
    images: 'auto'
  };
}

async loadModule(modulePath) {
  const moduleVariants = this.getModuleVariants(modulePath);
  const selectedVariant = this.selectVariant(moduleVariants);

  console.log(`Загрузка ${selectedVariant} для ${this.device.type} устройства`);

  try {
    return await import(selectedVariant);
  } catch (error) {
    // Если вариант для устройства не найден, пробуем основной
    console.warn(`Вариант ${selectedVariant} не найден, пробуем основной`);
    return await import(modulePath);
  }
}

getModuleVariants(basePath) {
  const variants = {
    base: basePath,
    mobile: basePath.replace('.js', '.mobile.js'),
    tablet: basePath.replace('.js', '.tablet.js'),
    desktop: basePath.replace('.js', '.desktop.js'),
    lite: basePath.replace('.js', '.lite.js')
  };

  return variants;
}

selectVariant(variants) {
  // Правила выбора варианта
  const rules = [

```

// Правило 1: Режим экономии данных

```
() => {  
  if (this.connection.saveData || this.userPreferences.dataSaver) {  
    return variants.lite;  
  }  
},
```

// Правило 2: Медленное соединение

```
() => {  
  if (this.connection.type === '2g' || this.connection.type === '3g') {  
    return variants.lite;  
  }  
},
```

// Правило 3: Устройство

```
() => {  
  switch (this.device.type) {  
    case 'mobile':  
      return variants.mobile;  
    case 'tablet':  
      return variants.tablet;  
    case 'desktop':  
      return variants.desktop;  
    default:  
      return variants.base;  
  }  
}  
];
```

// Применяем правила по порядку

```
for (const rule of rules) {  
  const result = rule();  
  if (result && this.variantExists(result)) {  
    return result;  
  }  
}
```

```
return variants.base;  
}
```

```
async variantExists(variantPath) {
```

// Проверяем существование варианта

```
try {  
  await fetch(variantPath, { method: 'HEAD' });
```

```
    return true;
  } catch {
    return false;
  }
}
```

```
async preloadCriticalModules() {
  const criticalModules = this.getCriticalModules();

  for (const module of criticalModules) {
    // Предзагружаем в фоновом режиме
    this.loadModule(module).catch(() => {
      // Игнорируем ошибки предзагрузки
    });
  }
}
```

```
getCriticalModules() {
  // Критические модули в зависимости от устройства
  const deviceCritical = {
    mobile: [
      './modules/touch-handler.js',
      './modules/mobile-ui.js',
      './modules/offline.js'
    ],
    tablet: [
      './modules/touch-handler.js',
      './modules/tablet-ui.js',
      './modules/offline.js'
    ],
    desktop: [
      './modules/desktop-ui.js',
      './modules/keyboard-shortcuts.js'
    ]
  ];
```

```
  // Общие критические модули
```

```
  const common = [
    './modules/auth.js',
    './modules/routing.js',
    './modules/error-handler.js'
  ];
```

```
  return [...common, ...(deviceCritical[this.device.type] || [])];
```

```
}  
}
```

// ПАТТЕРН 2: Загрузка с зависимостями

```
class DependencyAwareLoader {  
  constructor() {  
    this.dependencyGraph = new Map();  
    this.loadedModules = new Set();  
    this.loadingModules = new Map();  
    this.moduleCache = new Map();  
  }  
}
```

// Определение зависимостей

```
defineModule(name, importFn, dependencies = []) {  
  this.dependencyGraph.set(name, {  
    importFn,  
    dependencies,  
    dependentModules: new Set()  
  });  
};
```

// Обновляем граф зависимостей

```
dependencies.forEach(dep => {  
  if (this.dependencyGraph.has(dep)) {  
    this.dependencyGraph.get(dep).dependentModules.add(name);  
  }  
});  
}
```

// Загрузка модуля с зависимостями

```
async loadModule(name) {  
  // Если уже загружен  
  if (this.loadedModules.has(name)) {  
    return this.moduleCache.get(name);  
  }  
  
  // Если уже загружается  
  if (this.loadingModules.has(name)) {  
    return this.loadingModules.get(name);  
  }  
  
  const moduleInfo = this.dependencyGraph.get(name);  
  
  if (!moduleInfo) {
```

```

    throw new Error(`Модуль "${name}" не определен`);
}

// Создаем промис загрузки
const loadPromise = (async () => {
  try {
    // Загружаем зависимости
    await this.loadDependencies(moduleInfo.dependencies);

    // Загружаем сам модуль
    const module = await moduleInfo.importFn();

    // Сохраняем в кэш
    this.moduleCache.set(name, module);
    this.loadedModules.add(name);
    this.loadingModules.delete(name);

    console.log(`✅ Модуль "${name}" загружен`);
    return module;
  } catch (error) {
    this.loadingModules.delete(name);
    console.error(`❌ Ошибка загрузки модуля "${name}":`, error);
    throw error;
  }
})();

this.loadingModules.set(name, loadPromise);
return loadPromise;
}

// Загрузка всех зависимостей
async loadDependencies(dependencies) {
  const loadPromises = dependencies.map(dep => this.loadModule(dep));
  await Promise.all(loadPromises);
}

// Загрузка всех модулей в правильном порядке
async loadAll() {
  // Находим модули без зависимостей
  const rootModules = [];

  for (const [name, info] of this.dependencyGraph) {
    if (info.dependencies.length === 0) {
      rootModules.push(name);
    }
  }
}

```

```

    }
  }

  // Загружаем от корней
  await Promise.all(rootModules.map(name => this.loadModule(name)));

  console.log(`Все модули загружены. Всего: ${this.loadedModules.size}`);
}

// Выгрузка модуля и его зависимых модулей
async unloadModule(name) {
  const moduleInfo = this.dependencyGraph.get(name);

  if (!moduleInfo || !this.loadedModules.has(name)) {
    return false;
  }

  // Находим все модули, которые зависят от этого
  const dependentModules = this.findDependentModules(name);

  // Выгружаем зависимые модули
  for (const depName of dependentModules) {
    await this.unloadSingleModule(depName);
  }

  // Выгружаем сам модуль
  await this.unloadSingleModule(name);

  return true;
}

async unloadSingleModule(name) {
  const moduleInfo = this.dependencyGraph.get(name);

  if (!moduleInfo || !this.loadedModules.has(name)) {
    return;
  }

  // Вызываем деструктор если есть
  const module = this.moduleCache.get(name);
  if (module && typeof module.destroy === 'function') {
    await module.destroy();
  }
}

```



```
// Удаляем из кэша
this.moduleCache.delete(name);
this.loadedModules.delete(name);

console.log(`Модуль "${name}" выгружен`);
}

findDependentModules(name, visited = new Set()) {
  if (visited.has(name)) {
    return [];
  }

  visited.add(name);
  const result = [name];
  const moduleInfo = this.dependencyGraph.get(name);

  if (!moduleInfo) {
    return result;
  }

  for (const depName of moduleInfo.dependentModules) {
    result.push(...this.findDependentModules(depName, visited));
  }

  return [...new Set(result)];
}

// Визуализация графа зависимостей
visualizeGraph() {
  const nodes = [];
  const edges = [];

  for (const [name, info] of this.dependencyGraph) {
    nodes.push({
      id: name,
      label: name,
      loaded: this.loadedModules.has(name),
      loading: this.loadingModules.has(name)
    });

    info.dependencies.forEach(dep => {
      edges.push({
        from: dep,
        to: name,
      });
    });
  }
}
```

```

        arrows: 'to'
    });
});
}

return { nodes, edges };
}
}

```

// ПАТТЕРН 3: Прогрессивная загрузка с фолбэками

```

class ProgressiveLoader {
  constructor(options = {}) {
    this.options = {
      maxRetries: options.maxRetries || 3,
      timeout: options.timeout || 10000,
      fallbacks: options.fallbacks || {},
      featureDetection: options.featureDetection || true
    };

    this.featureCache = new Map();
  }

  async loadWithFallbacks(modulePath, fallbackPaths = []) {
    // Проверяем кэш
    if (this.featureCache.has(modulePath)) {
      return this.featureCache.get(modulePath);
    }

    // Пробуем загрузить основной модуль
    for (let attempt = 1; attempt <= this.options.maxRetries; attempt++) {
      try {
        const module = await this.loadWithTimeout(modulePath);
        this.featureCache.set(modulePath, module);
        return module;
      } catch (error) {
        console.warn(`Попытка ${attempt} не удалась для ${modulePath}:`, error.message);

        if (attempt === this.options.maxRetries) {
          // Все попытки исчерпаны, пробуем фолбэки
          break;
        }
      }

      // Ждем перед следующей попыткой
    }
  }
}

```

```

    await this.delay(1000 * attempt);
  }
}

// Пробуем фолбэки
for (const fallbackPath of fallbackPaths) {
  try {
    console.log(`Пробуем фолбэк: ${fallbackPath}`);
    const module = await this.loadWithTimeout(fallbackPath);
    this.featureCache.set(modulePath, module);
    return module;
  } catch (error) {
    console.warn(`Фолбэк ${fallbackPath} также не удался:`, error.message);
  }
}

throw new Error(`Не удалось загрузить модуль ${modulePath} или его фолбэки`);
}

async loadWithTimeout(modulePath, timeout = this.options.timeout) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeout);

  try {
    const module = await import(modulePath);
    clearTimeout(timeoutId);
    return module;
  } catch (error) {
    clearTimeout(timeoutId);

    if (error.name === 'AbortError') {
      throw new Error(`Таймаут загрузки ${modulePath} (${timeout}ms)`);
    }

    throw error;
  }
}

// Загрузка с детекцией возможностей
async loadFeature(featureName) {
  const featureConfig = this.options.features?.[featureName];

  if (!featureConfig) {
    throw new Error(`Конфигурация для фичи ${featureName} не найдена`);
  }
}

```

```

}

// Проверяем поддержку браузером
if (featureConfig.requiredAPIs) {
  const missingAPIs = featureConfig.requiredAPIs.filter(api => !
this.isAPISupported(api));

  if (missingAPIs.length > 0) {
    console.warn(`Фича ${featureName} требует API: ${missingAPIs.join(', ')}`);

    // Если есть фолбэк для отсутствующих API
    if (featureConfig.fallbackOnMissingAPI) {
      return this.loadWithFallbacks(
        featureConfig.fallbackOnMissingAPI,
        featureConfig.fallbacks || []
      );
    }

    throw new Error(`Не поддерживаемые API: ${missingAPIs.join(', ')}`);
  }
}

// Загружаем фичу
return this.loadWithFallbacks(
  featureConfig.main,
  featureConfig.fallbacks || []
);
}

isAPISupported(apiName) {
  // Проверяем поддержку API в браузере
  const apiPath = apiName.split('.');
  let current = window;

  for (const part of apiPath) {
    if (current[part] === undefined) {
      return false;
    }
    current = current[part];
  }

  return true;
}

```

```
// Предзагрузка возможных фич на основе User-Agent
async preloadLikelyFeatures() {
  const likelyFeatures = this.detectLikelyFeatures();

  const preloadPromises = likelyFeatures.map(featureName =>
    this.preloadFeature(featureName).catch(() => {
      // Игнорируем ошибки предзагрузки
    })
  );

  await Promise.all(preloadPromises);
  console.log(`Предзагружено ${likelyFeatures.length} вероятных фич`);
}

detectLikelyFeatures() {
  const features = [];
  const ua = navigator.userAgent.toLowerCase();

  // Детектим устройство и браузер
  if (/chrome/.test(ua)) {
    features.push('chrome_specific');
  }

  if (/firefox/.test(ua)) {
    features.push('firefox_specific');
  }

  if (/safari/.test(ua) && !/chrome/.test(ua)) {
    features.push('safari_specific');
  }

  if (/mobile/.test(ua)) {
    features.push('mobile_optimized');
    features.push('touch_gestures');
  } else {
    features.push('desktop_optimized');
    features.push('keyboard_shortcuts');
  }

  // Детектим возможности
  if ('serviceWorker' in navigator) {
    features.push('offline_support');
  }
}
```

```

if ('PushManager' in window) {
  features.push('push_notifications');
}

if ('WebGLRenderingContext' in window) {
  features.push('webgl_graphics');
}

return features;
}

async preloadFeature(featureName) {
  const featureConfig = this.options.features?.[featureName];

  if (!featureConfig) {
    return;
  }

  // Предзагружаем основной модуль
  await this.loadWithTimeout(featureConfig.main, 5000).catch(() => {
    // Игнорируем таймауты при предзагрузке
  });
}

delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
}

```

4.2. Антипаттерны и предостережения

javascript

// АНТИПАТТЕРНЫ ДИНАМИЧЕСКОЙ ЗАГРУЗКИ:

// 1. СЛИШКОМ МЕЛКИЙ СПЛИТТИНГ

// ✗ ПЛОХО: Каждый компонент в отдельном файле

```

const Button = lazy(() => import('./components/Button.jsx'));
const Input = lazy(() => import('./components/Input.jsx'));
const Label = lazy(() => import('./components/Label.jsx'));
const Tooltip = lazy(() => import('./components/Tooltip.jsx'));

```

// Результат: десятки HTTP-запросов

// 🟢 ХОРОШО: Группировка мелких компонентов

```
const UIComponents = lazy(() => import('./components/ui/index.jsx'));  
// Или разумный сплиттинг по функциональности
```

// 2. ОТСУТСТВИЕ ОБРАБОТКИ ОШИБОК

// ✗ ПЛОХО

```
const Module = lazy(() => import('./module.js'));
```

// ☑ ХОРОШО

```
const Module = lazy(() => import('./module.js').catch(() => ({  
  default: () => <div>Модуль не доступен</div>  
})));
```

// 3. БЕСКОНЕЧНЫЕ РЕТРАИ

// ✗ ПЛОХО

```
async function loadModule() {  
  while (true) {  
    try {  
      return await import('./module.js');  
    } catch {  
      // Бесконечные повторные попытки  
    }  
  }  
}
```

// ☑ ХОРОШО

```
async function loadModuleWithRetry(maxRetries = 3) {  
  for (let i = 0; i < maxRetries; i++) {  
    try {  
      return await import('./module.js');  
    } catch (error) {  
      if (i === maxRetries - 1) throw error;  
      await delay(1000 * (i + 1));  
    }  
  }  
}
```

// 4. ОТСУТСТВИЕ ИНДИКАТОРА ЗАГРУЗКИ

// ✗ ПЛОХО

```
<Suspense fallback={null}>  
  <LazyComponent />  
</Suspense>
```

// ☑ ХОРОШО

```
<Suspense fallback={<LoadingSpinner />}>
```

```
<LazyComponent />
</Suspense>
```

```
// 5. ПРЕДЗАГРУЗКА ВСЕГО И СРАЗУ
```

```
// ✗ ПЛОХО
```

```
useEffect(() => {
  // Предзагружаем ВСЕ модули при загрузке
  import('./module1.js');
  import('./module2.js');
  import('./module3.js');
  // ... еще 20 модулей
}, []);
```

```
// ☑ ХОРОШО
```

```
useEffect(() => {
  // Предзагружаем только критические модули
  import('./critical1.js');
  import('./critical2.js');

  // Остальные по необходимости или при idle
  if ('requestIdleCallback' in window) {
    requestIdleCallback(() => {
      import('./non-critical.js');
    });
  }
}, []);
```

```
// 6. ДИНАМИЧЕСКИЕ ПУТИ БЕЗ ОГРАНИЧЕНИЙ
```

```
// ✗ ПЛОХО: Возможна загрузка любого файла
```

```
const moduleName = userInput; // Может быть "../../etc/passwd"
const module = await import(`./modules/${moduleName}.js`);
```

```
// ☑ ХОРОШО: Валидация и белый список
```

```
const allowedModules = ['dashboard', 'analytics', 'settings'];
if (!allowedModules.includes(moduleName)) {
  throw new Error('Недопустимый модуль');
}
const module = await import(`./modules/${moduleName}.js`);
```

```
// 7. ИГНОРИРОВАНИЕ КЭШИРОВАНИЯ
```

```
// ✗ ПЛОХО: Каждый раз новая загрузка
```

```
async function loadData() {
  const dataModule = await import('./data-processor.js');
  return dataModule.process();
}
```



```
}
```

```
// 📌 ХОРОШО: Кэширование загруженных модулей
```

```
const moduleCache = new Map();  
async function loadData() {  
  if (!moduleCache.has('data-processor')) {  
    moduleCache.set('data-processor', await import('./data-processor.js'));  
  }  
  return moduleCache.get('data-processor').process();  
}
```

Заключение

Динамический импорт и ленивая загрузка — это мощные инструменты для оптимизации производительности веб-приложений. Ключевые выводы:

Преимущества:

1. **Улучшенная начальная загрузка** – загружается только необходимый код
2. **Уменьшение размера бандла** – код разбивается на логические части
3. **Условная загрузка** – разные функциональности для разных пользователей
4. **Прогрессивное улучшение** – базовая функциональность работает всегда

Рекомендации по использованию:

1. **Начинайте с анализа** – используйте инструменты для определения что можно лениво загружать
2. **Группируйте логически** – не разбивайте слишком мелко
3. **Используйте индикаторы загрузки** – пользователь должен видеть прогресс
4. **Реализуйте обработку ошибок** – что если модуль не загрузился?
5. **Кэшируйте загруженные модули** – избегайте повторных загрузок
6. **Предзагружайте критическое** – балансируйте между ленивостью и скоростью
7. **Тестируйте на разных устройствах** – учитывайте скорость соединения

Когда использовать:

- 📌 Большие библиотеки (charting, mapping, rich text editors)
- 📌 Админ-панели и инструменты для редких случаев

- Функциональности для премиум-пользователей
- Модули, требующие полифиллов
- Разные версии для разных устройств

Инструменты для помощи:

- **Webpack Bundle Analyzer** – анализ размера бандла
- **Lighthouse** – рекомендации по производительности
- **React DevTools** – профилирование компонентов
- **Chrome DevTools Coverage** – поиск неиспользуемого кода

Динамический импорт и ленивая загрузка — это не серебряная пуля, а инструмент, который нужно использовать с умом. Правильно примененные, они могут значительно улучшить пользовательский опыт, но неправильное использование может усложнить код и ухудшить производительность.

Введение: Зачем нужны системы сборки?

Современный JavaScript-проект — это не просто один файл `script.js`. Это сотни модулей, библиотек, стилей, изображений и других ресурсов. Системы сборки решают ключевые проблемы:

Проблемы без сборки:

1. Десятки/сотни `<script>` тегов в HTML
2. Ручное разрешение зависимостей между файлами
3. Отсутствие оптимизации кода
4. Нет поддержки современных синтаксических конструкций для старых браузеров
5. Медленная загрузка из-за множества HTTP-запросов

Что дают системы сборки:

- Объединение множества файлов в один или несколько бандлов
- Транспилиацию современного JavaScript в совместимый с браузерами
- Обработку CSS, изображений, шрифтов
- Дерево-шейкинг (tree shaking) — удаление неиспользуемого кода
- Код-сплиттинг — разделение кода на части для ленивой загрузки
- Горячую замену модулей (HMR) во время разработки

Часть 1: Webpack — универсальный сборщик

1.1. История и философия

Создание и развитие:

- Создан Тобиасом Копперсом в 2012 году
- Первоначально для сборки сложных веб-приложений
- Стал стандартом де-факто в React-экосистеме

Философия: "Everything is a module"

javascript

// В Webpack можно импортировать почти любой тип файла

```
import Component from './Component.jsx';
import styles from './styles.css';
import image from './logo.png';
import data from './data.json';
```

1.2. Архитектура Webpack

Основные концепции:

Entry (точка входа)

javascript

// webpack.config.js

```
module.exports = {
  entry: {
    main: './src/index.js',
    admin: './src/admin.js'
  }
};
```

- Определяет, с каких файлов начинать сборку зависимостей
- Может быть несколько точек входа для многостраничных приложений

Output (выходные файлы)

javascript

```
module.exports = {
  output: {
    filename: '[name].[contenthash].js',
    path: path.resolve(__dirname, 'dist'),
    publicPath: '/assets/'
  }
};
```

- `[name]` — имя точки входа
- `[contenthash]` — хэш содержимого для инвалидации кэша
- `publicPath` — базовый путь для загрузки ресурсов

Loaders (загрузчики)

javascript

```

module.exports = {
  module: {
    rules: [
      {
        test: /\.jsx?$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader']
      },
      {
        test: /\..(png|svg|jpg|gif)$/,
        type: 'asset/resource'
      }
    ]
  }
};

```

- Преобразуют файлы разных типов в валидные JavaScript-модули
- Выполняются справа налево/снизу вверх

Plugins (плагины)

javascript

```

const HtmlWebpackPlugin = require('html-webpack-plugin');
const BundleAnalyzerPlugin = require('webpack-bundle-analyzer').BundleAnalyzerPlugin;

```

```

module.exports = {
  plugins: [
    new HtmlWebpackPlugin({
      template: './src/index.html'
    }),
    new BundleAnalyzerPlugin()
  ]
};

```

- Выполняют более сложные задачи
- Работают на разных этапах процесса сборки

1.3. Конфигурация Webpack

Базовая конфигурация

```
javascript

// webpack.config.js
const path = require('path');
const webpack = require('webpack');

module.exports = {
  mode: process.env.NODE_ENV === 'production' ? 'production' : 'development',

  entry: './src/index.js',

  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist'),
    clean: true // Очищает dist перед каждой сборкой
  },

  devtool: 'source-map',

  devServer: {
    static: './dist',
    hot: true,
    port: 3000,
    historyApiFallback: true
  },

  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env', '@babel/preset-react']
          }
        }
      }
    ]
  }
},
```

```
plugins: [  
  new webpack.ProgressPlugin()  
]  
};
```

Продвинутые возможности

Code Splitting:

```
javascript  
  
module.exports = {  
  optimization: {  
    splitChunks: {  
      chunks: 'all',  
      cacheGroups: {  
        vendor: {  
          test: /[\\/]node_modules[\\/]/,  
          name: 'vendors',  
          chunks: 'all'  
        }  
      }  
    }  
  }  
};
```

Tree Shaking:

```
javascript  
  
module.exports = {  
  mode: 'production',  
  optimization: {  
    usedExports: true, // Включает tree shaking  
    minimize: true,  
    minimizer: [  
      new TerserPlugin({  
        terserOptions: {  
          compress: {  
            drop_console: true // Удаляет console.log в production  
          }  
        }  
      })  
    ]  
  }  
};
```

```
};
```

1.4. Webpack 5 vs Webpack 4

Основные улучшения в Webpack 5:

1. **Module Federation** — микрофронтенды

javascript

```
// app1/webpack.config.js
```

```
module.exports = {  
  plugins: [  
    new ModuleFederationPlugin({  
      name: 'app1',  
      filename: 'remoteEntry.js',  
      exposes: {  
        './Button': './src/Button'  
      }  
    })  
  ]  
};
```

```
// app2/webpack.config.js
```

```
module.exports = {  
  plugins: [  
    new ModuleFederationPlugin({  
      name: 'app2',  
      remotes: {  
        app1: 'app1@http://localhost:3001/remoteEntry.js'  
      }  
    })  
  ]  
};
```

2. **Asset Modules** — встроенная обработка ресурсов

3. **Persistent Caching** — ускорение повторных сборок

4. **Improved Tree Shaking** — лучшее удаление неиспользуемого кода

Часть 2: Rollup — сборщик для библиотек

2.1. Философия и применение

Создатель: Рич Харрис (создатель Svelte)

Философия: "Простота и эффективность для библиотек"

Когда использовать Rollup:

- Создание JavaScript-библиотек
- Пакеты npm
- Когда важен размер бандла
- Для проектов с чистым ES6 модулями

2.2. Конфигурация Rollup

Базовая конфигурация

```
javascript

// rollup.config.js
import { nodeResolve } from '@rollup/plugin-node-resolve';
import commonjs from '@rollup/plugin-commonjs';
import babel from '@rollup/plugin-babel';
import { terser } from 'rollup-plugin-terser';

export default {
  input: 'src/index.js',

  output: [
    {
      file: 'dist/bundle.esm.js',
      format: 'esm', // ES модули
      sourcemap: true
    },
    {
      file: 'dist/bundle.cjs.js',
      format: 'cjs', // CommonJS
      sourcemap: true
    },
  ],
}
```

```

{
  file: 'dist/bundle.umd.js',
  format: 'umd', // Universal Module Definition
  name: 'MyLibrary',
  sourcemap: true
}
],

plugins: [
  nodeResolve(), // Разрешает модули из node_modules
  commonjs(), // Преобразует CommonJS в ES6
  babel({
    babelHelpers: 'bundled',
    exclude: 'node_modules/**'
  }),
  terser() // Минификация для production
],

external: ['react', 'react-dom'] // Не включать в бандл
};

```

Tree Shaking в Rollup

```

javascript

export default {
  input: 'src/index.js',

  output: {
    file: 'dist/bundle.js',
    format: 'esm',
    exports: 'named'
  },

  // Rollup автоматически выполняет tree shaking для ES модулей
  treeshake: {
    propertyReadSideEffects: false,
    moduleSideEffects: false
  }
};

```

2.3. Плагины Rollup

```

javascript

```

```

import image from '@rollup/plugin-image';
import json from '@rollup/plugin-json';
import postcss from 'rollup-plugin-postcss';
import sizes from 'rollup-plugin-sizes';

export default {
  plugins: [
    // Обработка изображений
    image(),

    // Импорт JSON файлов
    json(),

    // Обработка CSS
    postcss({
      extract: true, // Выделяет CSS в отдельный файл
      minimize: true
    }),

    // Анализ размера бандла
    sizes(),

    // Замена переменных окружения
    replace({
      'process.env.NODE_ENV': JSON.stringify('production'),
      __buildDate__: () => JSON.stringify(new Date())
    })
  ]
};

```

2.4. Rollup vs Webpack

Преимущества Rollup:

- Лучшее tree shaking из коробки
- Более чистый и оптимизированный вывод
- Проще конфигурация для библиотек
- Быстрее для небольших проектов

Когда выбрать:

- Webpack: сложные веб-приложения, много ресурсов
- Rollup: библиотеки, npm-пакеты, фокус на размере

Часть 3: Parcel — zero-config сборщик

3.1. Философия Parcel

Девиз: "Удивительно быстрый сборщик для веб-приложений без настройки"

Ключевые особенности:

- Zero-config — работает из коробки
- Многоядерная обработка
- Встроенный hot reload
- Автоматическое разделение кода

3.2. Использование Parcel

Базовое использование

```
bash
```

```
# Установка
```

```
npm install -D parcel
```

```
# Запуск development сервера
```

```
npx parcel src/index.html
```

```
# Production сборка
```

```
npx parcel build src/index.html
```

Структура проекта

```
text
```

```
project/
```

```
├── src/
```

```
│   ├── index.html      # Точка входа
```

```
│   ├── index.js        # Импортируется из index.html
```

```
│   ├── styles.css      # Импортируется из index.js
```

```
│   └── components/
```

```
│       └── App.jsx
```

```
└── package.json
```

3.3. Конфигурация Parcel

package.json конфигурация

```
json
{
  "name": "my-project",
  "source": "src/index.html",

  "scripts": {
    "start": "parcel",
    "build": "parcel build"
  },

  "targets": {
    "main": {
      "source": "src/index.js",
      "outputFormat": "global",
      "isLibrary": true
    }
  },

  "dependencies": {
    "react": "^18.0.0"
  },

  "devDependencies": {
    "parcel": "^2.8.0"
  }
}
```

.parcelrc (расширенная конфигурация)

```
json
{
  "extends": "@parcel/config-default",

  "transformers": {
    "*.svg": ["@parcel/transformer-svg-react"]
  },

  "packagers": {
    "*.css": "@parcel/packager-css"
  }
}
```

```
},  
  
"optimizers": {  
  "*.js": ["@parcel/optimizer-terser"],  
  "*.css": ["@parcel/optimizer-cssnano"]  
},  
  
"reporters": ["@parcel/reporter-detailed"]  
}
```

3.4. Особенности Parcel

Автоматическое разделение кода

```
javascript  
  
// Parcel автоматически разделит этот динамический импорт  
import('./math').then(math => {  
  console.log(math.add(16, 26));  
});  
  
// Или с async/await  
async function loadComponent() {  
  const { default: Component } = await import('./Component');  
  return Component;  
}
```

Встроенная поддержка всего

Parcel автоматически определяет и обрабатывает:

- JavaScript/TypeScript/JSX
- CSS/SASS/Less/Stylus
- HTML/Markdown
- Изображения/шрифты
- JSON/TOML/YAML

Часть 4: Сравнительный анализ

4.1. Таблица сравнения

Параметр	Webpack	Rollup	Parcel
Конфигурация	Сложная	Средняя	Zero-config
Скорость сборки	Средняя	Быстрая	Очень быстрая
Размер бандла	Оптимальный	Лучший	Хороший
Tree Shaking	Хорошее (с настройкой)	Отличное	Хорошее
Code Splitting	Отличное	Хорошее	Отличное
HMR	Отличная	Нет	Отличная
Экосистема	Огромная	Большая	Растущая
Лучше для	Веб-приложений	Библиотек	Быстрого старта

4.2. Производительность

```
javascript
// Бенчмарк сборки проекта (условные единицы)

const benchmarks = {
  webpack: {
    initialBuild: 10000, // мс
    incrementalBuild: 2000,
    bundleSize: 150 // КБ
  },

  rollup: {
    initialBuild: 5000,
    incrementalBuild: 1000,
    bundleSize: 120
  },

  parcel: {
    initialBuild: 3000,
    incrementalBuild: 500,
    bundleSize: 140
  }
}
```

```
}  
};
```

4.3. Критерии выбора

Выбирайте Webpack если:

- Сложное веб-приложение с множеством ресурсов
- Нужен полный контроль над процессом сборки
- Используются специфичные плагины и загрузчики
- Команда уже знакома с Webpack

Выбирайте Rollup если:

- Создаете библиотеку для публикации в npm
- Важен минимальный размер бандла
- Проект состоит в основном из ES6 модулей
- Нужно эффективное tree shaking

Выбирайте Parcel если:

- Нужно быстро начать проект без настройки
- Проект небольшой или средний
- Не хотите тратить время на конфигурацию
- Цените простоту использования

Часть 5: Интеграция с современным стеком

5.1. TypeScript поддержка

Webpack:

```
javascript  
  
// webpack.config.js  
module.exports = {  
  module: {  
    rules: [  
      {  
        test: /\.tsx?$/,  
        use: 'ts-loader',  
        exclude: /node_modules/
```



```

    }
  ]
},
resolve: {
  extensions: ['.tsx', '.ts', '.js']
}
};

```

Rollup:

```

javascript

// rollup.config.js
import typescript from '@rollup/plugin-typescript';

export default {
  plugins: [
    typescript({
      tsconfig: './tsconfig.json'
    })
  ]
};

```

Parcel: Автоматически определяет TypeScript

5.2. React поддержка

Webpack + Babel:

```

javascript

// .babelrc
{
  "presets": [
    "@babel/preset-env",
    "@babel/preset-react",
    "@babel/preset-typescript"
  ],
  "plugins": [
    "@babel/plugin-transform-runtime"
  ]
}

```

Parcel c React:

bash

```
# Просто установите React и Parcel
npm install react react-dom
npm install -D parcel
# Parcel автоматически настроит JSX
```

5.3. Оптимизация для production

Общие оптимизации:

javascript

```
// Все сборщики поддерживают:
// 1. Минификацию
// 2. Сжатие (gzip/brotli)
// 3. Инвалидацию кэша через хэши
// 4. Удаление console.log в production
```

Часть 6: Будущее систем сборки

6.1. Тренды развития

1. Ускорение сборки

- Использование Rust/Go для более быстрых инструментов
- Persistent caching
- Инкрементальные сборки

2. Упрощение конфигурации

- Zero-config подход как в Parcel
- Умные дефолтные настройки
- Автоматическое определение зависимостей

3. Улучшенный developer experience

- Лучшие ошибки и предупреждения
- Визуализация зависимостей
- Интеграция с IDE

6.2. Новые инструменты

Vite:

- Создан Evan You (создатель Vue.js)

- Мгновенный запуск development сервера
- Использует нативные ES модули в development
- Быстрые сборки благодаря ESbuild

javascript

// vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000
  },
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom']
        }
      }
    }
  }
});
```

ESbuild:

- Невероятно быстрый (в 10-100 раз быстрее)
- Написан на Go
- Используется как основа для Vite и других инструментов

Turbopack:

- Следующее поколение от создателей Webpack
- Написан на Rust
- Инкрементальные обновления за миллисекунды

6.3. Рекомендации по выбору в 2026

Для новых проектов:

1. **Веб-приложения:** Vite или Parcel 2
2. **Библиотеки:** Rollup с TypeScript
3. **Монолиты:** Webpack 5 с Module Federation

Для существующих проектов:

1. Если работает на Webpack 4 → обновить до Webpack 5
2. Если используется сложная конфигурация → остаться на Webpack
3. Если хотите ускорить разработку → попробовать Vite

Заключение

Системы сборки эволюционировали от простых конкатенаторов скриптов до сложных инструментов, которые:

1. **Автоматизируют** рутинные задачи
2. **Оптимизируют** производительность загрузки
3. **Обеспечивают** совместимость с разными браузерами
4. **Улучшают** опыт разработчика

Ключевые принципы выбора:

1. **Начните с простого** — Parcel или Vite для быстрого старта
2. **Масштабируйте по необходимости** — переходите на Webpack при росте сложности
3. **Выбирайте под задачу** — Rollup для библиотек, Webpack для приложений
4. **Следите за трендами** — инструменты быстро развиваются

Будущее: Движение к более быстрым, умным и простым инструментам, которые требуют минимальной конфигурации, но предоставляют максимум возможностей. Независимо от выбора инструмента, понимание принципов работы систем сборки остается критически важным навыком для современного JavaScript-разработчика.

Введение: Зачем нужны пакетные менеджеры?

В современной JavaScript-экосистеме пакетный менеджер — это фундаментальный инструмент, без которого невозможно представить разработку. Рассмотрим эволюцию и необходимость:

Исторический контекст

До пакетных менеджеров (до 2010):

```
html

<!-- Ручное управление зависимостями -->
<script src="jquery-1.4.2.min.js"></script>
<script src="jquery-ui-1.8.5.custom.min.js"></script>
<script src="underscore-1.1.4.js"></script>
<!-- Проблемы:
1. Версионные конфликты
2. Отсутствие централизованного репозитория
3. Ручное обновление
4. Нет контроля зависимостей зависимостей
-->
```

Революция пакетных менеджеров:

- 🟦 **2010:** npm 1.0 — первый пакетный менеджер для Node.js
- 🟦 **2016:** yarn — ответ Facebook на проблемы npm
- 🟦 **2017:** npm 5 — серьезные улучшения после конкуренции
- 🟦 **2020:** pnpm — новый подход с симлинками

Функции пакетных менеджера

```
javascript

// Что делает пакетный менеджер:
1. Установка зависимостей:    npm install react
2. Управление версиями:       "react": "^18.2.0"
3. Разрешение зависимостей:   A→B→C и X→C (разные версии)
```

- | | |
|------------------------|-----------------------|
| 4. Безопасность: | npm audit, yarn audit |
| 5. Публикация пакетов: | npm publish |
| 6. Запуск скриптов: | npm run dev |

Часть 1: npm (Node Package Manager)

1.1. Архитектура и философия

Иерархия зависимостей npm

```
text

node_modules/
├─ package-a/          # Зависимость проекта
├─ node_modules/
│   └─ package-b/      # Зависимость package-a
└─ package.json
├─ package-c/
└─ .package-lock.json  # Фиксация точных версий
```

Ключевые файлы

package.json — манифест проекта

```
json

{
  "name": "my-awesome-app",
  "version": "1.0.0",
  "description": "Современное веб-приложение",
  "private": true,

  "main": "src/index.js",
  "type": "module", // ES модули

  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "jest",
    "lint": "eslint .",
    "format": "prettier --write ."
  }
}
```

```

},

"dependencies": {
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "axios": "^1.4.0",
  "lodash": "^4.17.21"
},

"devDependencies": {
  "@types/react": "^18.2.15",
  "@types/node": "^20.4.5",
  "typescript": "^5.1.6",
  "vite": "^4.4.5",
  "jest": "^29.6.1"
},

"peerDependencies": {
  "react": ">=16.8.0"
},

"optionalDependencies": {
  "fsevents": "^2.3.2"
},

"engines": {
  "node": ">=18.0.0",
  "npm": ">=9.0.0"
},

"browserslist": [
  "> 1%",
  "last 2 versions",
  "not dead"
]
}

```

package-lock.json — детерминированная сборка

```

json

{
  "name": "my-awesome-app",
  "version": "1.0.0",
  "lockfileVersion": 3,

```

```

"requires": true,
"packages": {
  "node_modules/react": {
    "version": "18.2.0",
    "resolved": "https://registry.npmjs.org/react/-/react-18.2.0.tgz",
    "integrity":
"sha512-/3IjMdb2L9QbBdWiW5e3P2/npwMBaU9mHCSCUzNln0ZCYbcfTsGbTJrU/kGemdH2IWmB2ioZ+zKxtmq6g
09fGQ==",
    "dependencies": {
      "loose-envify": "^1.1.0"
    },
    "engines": {
      "node": ">=0.10.0"
    }
  },
  "node_modules/loose-envify": {
    "version": "1.4.0",
    "resolved": "https://registry.npmjs.org/loose-envify/-/loose-envify-1.4.0.tgz",
    "integrity":
"sha512-lyuxPGGr/Wfhr1em2CL/UcnUc1zcqKAImBDzukY7Y5F/yQiNdko6+fRLevlw1HgMySw7f611UIY408EtXR
SoK3Q==",
    "dependencies": {
      "js-tokens": "^4.0.0"
    },
    "bin": {
      "loose-envify": "cli.js"
    }
  }
}

```

1.2. Семантическое версионирование (SemVer)

Формат версий: MAJOR.MINOR.PATCH

```

javascript
// package.json
{
  "dependencies": {
    // Фиксированная версия
    "exact-version": "1.2.3",

```



```

// Совместимые обновления патчей
"caret-range": "^1.2.3", // >=1.2.3 <2.0.0

// Совместимые обновления минорных версий
"tilde-range": "~1.2.3", // >=1.2.3 <1.3.0

// Любая версия из диапазона
"range": ">=1.2.3 <2.0.0",

// Последняя версия
"latest": "*"
}
}

```

Правила обновлений:

- 🔵 **PATCH (1.2.3 → 1.2.4):** Исправления безопасности, баги (обратно совместимо)
- 🔵 **MINOR (1.2.3 → 1.3.0):** Новая функциональность (обратно совместимо)
- 🔵 **MAJOR (1.2.3 → 2.0.0):** Критические изменения (не обратно совместимо)

1.3. Команды npm

Установка и обновление

```

bash

# Базовые команды
npm init                # Инициализация проекта
npm init -y             # Инициализация с дефолтными значениями

# Установка пакетов
npm install             # Установить все зависимости
npm i                  # Алиас для install
npm i react             # Установить пакет
npm i react@18.2.0      # Конкретная версия
npm i -D typescript      # Установить dev зависимость
npm i -g nodemon        # Установить глобально

# Обновление
npm update              # Обновить все пакеты по SemVer
npm update react        # Обновить конкретный пакет
npm outdated            # Показать устаревшие пакеты

```

Управление зависимостями

bash

Просмотр зависимостей

```
npm list                # Дерево зависимостей
npm list --depth=0      # Только прямые зависимости
npm list react          # Где используется react
npm view react versions # Все доступные версии
npm info react          # Подробная информация

# Удаление
npm uninstall react     # Удалить пакет
npm prune               # Удалить неиспользуемые пакеты

# Аудит безопасности
npm audit               # Проверить уязвимости
npm audit fix           # Исправить автоматически
npm audit fix --force   # Исправить с обновлением major версий
```

Скрипты и публикация

bash

Работа со скриптами

```
npm run dev             # Запуск скрипта
npm run                 # Показать все скрипты
npm test                # Запуск тестов
npm start               # Запуск стартового скрипта
```

Публикация пакетов

```
npm login               # Авторизация
npm whoami              # Проверить авторизацию
npm publish             # Опубликовать пакет
npm version patch       # Увеличить patch версию
npm version minor       # Увеличить minor версию
npm version major       # Увеличить major версию
npm unpublish --force   # Удалить пакет (осторожно!)
```

1.4. npm Workspaces (монорепозитории)

Конфигурация workspaces

json

```
// package.json в корне
{
  "name": "my-monorepo",
  "private": true,
  "workspaces": [
    "packages/*",
    "apps/*"
  ],

  "scripts": {
    "dev": "npm run dev --workspace=frontend",
    "build": "npm run build --workspace=frontend",
    "test": "npm run test --workspace=shared"
  }
}
```

Структура монорепозитория

text

```
my-monorepo/
├── package.json          # Корневой package.json
├── packages/
│   ├── shared/          # Общая библиотека
│   │   ├── package.json
│   │   └── src/
│   └── ui-components/   # UI библиотека
│       ├── package.json
│       └── src/
└── apps/
    ├── frontend/        # Веб-приложение
    │   ├── package.json
    │   └── src/
    └── backend/          # API сервер
        ├── package.json
        └── src/
```

1.5. npm 7+ улучшения

Новые возможности

bash

```
# Автоматическая установка peerDependencies
```

```
npm i react # Установит и peer зависимости
```

```
# Улучшенный вывод
```

```
npm i --verbose # Подробный вывод
```

```
# Параллельная установка
```

```
npm i --prefer-dedupe # Дедупликация зависимостей
```

```
# Интерактивный режим
```

```
npm init --yes --scope=@myorg # Создание скоупа
```

Часть 2: yarn

2.1. История и философия

Проблемы npm 3, которые решил yarn:

1. **Недетерминированность:** Разные установки → разные node_modules
2. **Скорость:** Последовательная установка пакетов
3. **Надежность:** Частые ошибки при установке
4. **Безопасность:** Отсутствие проверки целостности

Ключевые инновации yarn 1.0:

- yarn.lock для детерминированности
- Кэширование пакетов
- Параллельная установка
- Проверка целостности checksums

2.2. Архитектура yarn

yarn.lock файл

```
yaml
```

```
# THIS IS AN AUTOGENERATED FILE. DO NOT EDIT THIS FILE DIRECTLY.
```

```
# yarn lockfile v1
```

```
react@^18.2.0:
```

```
  version "18.2.0"
```

```
  resolved "https://registry.yarnpkg.com/react/-/react-18.2.0.tgz#..."
```

```
integrity sha512-...
dependencies:
  loose-envify "^1.1.0"

loose-envify@1.1.0:
  version "1.4.0"
  resolved "https://registry.yarnpkg.com/loose-envify/-/loose-envify-1.4.0.tgz#..."
  integrity sha512-...
dependencies:
  js-tokens "^4.0.0"
optionalDependencies:
  fsevents "*"

```

Особенности yarn 1.x (Classic)

```
bash

# Установка yarn
npm install -g yarn

# Основные команды
yarn init           # Инициализация
yarn add react      # Добавить зависимость
yarn add -D typescript  # Dev зависимость
yarn remove lodash  # Удалить пакет
yarn install        # Установить все
yarn upgrade        # Обновить все
yarn why react      # Почему установлен react?

```

2.3. Yarn 2+ (Berry) — современный yarn

Миграция на Yarn 2

```
bash

# Установка Yarn 2
yarn set version berry  # Установить Yarn 2
yarn set version 3.6.3  # Конкретная версия

# .yarnrc.yml конфигурация
nodeLinker: node-modules  # Режим совместимости
npmRegistryServer: "https://registry.npmjs.org/"
yarnPath: .yarn/releases/yarn-3.6.3.cjs

```

Zero-Installs подход

```
text

project/
├── .yarn/                # Кэш Yarn
├── releases/            # Версии yarn
├── sdk/                 # SDK инструментов
├── .yarnrc.yml          # Конфигурация
├── .pnp.cjs             # PnP манифест
├── package.json
└── src/
```

Plug'n'Play (PnP) архитектура

```
javascript

// .yarnrc.yml
nodeLinker: pnp           // Включаем PnP

// Преимущества PnP:
// 1. Нет node_modules
// 2. Мгновенный запуск
// 3. Предсказуемые зависимости
// 4. Лучшая производительность
```

2.4. Yarn Workspaces

Конфигурация

```
json

// package.json
{
  "name": "monorepo",
  "private": true,
  "workspaces": [
    "packages/*",
    "apps/*"
  ]
}
```

Команды workspaces

```
bash
```

Работа с воркспейсами

`yarn workspaces list` *# Список всех воркспейсов*

`yarn workspace frontend add react` *# Добавить в конкретный воркспейс*

`yarn workspaces focus` *# Установить зависимости только для текущего воркспейса*

Запуск команд во всех воркспейсах

`yarn workspaces foreach run build` *# Собрать все проекты*

`yarn workspaces foreach -v run test` *# С тестовым выводом*

2.5. Yarn Modern (v3+) особенности

Современные фичи

yaml

.yarnrc.yml

`nodeLinker: pnp`

`plugins:`

- `path: .yarn/plugins/@yarnpkg/plugin-workspace-tools.cjs`
`spec: "@yarnpkg/plugin-workspace-tools"`
- `path: .yarn/plugins/@yarnpkg/plugin-typescript.cjs`
`spec: "@yarnpkg/plugin-typescript"`

`yarnPath: .yarn/releases/yarn-3.6.3.cjs`

Кэширование

`enableGlobalCache: false`

`cacheFolder: ./node_modules/.yarn-cache`

Сеть

`httpTimeout: 30000`

`enableMirror: true`

Команды Yarn Modern

bash

Современные команды

`yarn dlx create-react-app my-app` *# Временная установка*

`yarn up react` *# Умное обновление*

`yarn npm publish` *# Публикация через npm*

`yarn config set --home enableTelemetry 0` *# Отключить телеметрию*

Часть 3: pnpm — быстрый, эффективный менеджер

3.1. Философия и архитектура

Проблемы, которые решает pnpm:

1. **Дублирование пакетов:** Один пакет в разных версиях
2. **Размер node_modules:** Гигантские папки зависимостей
3. **Производительность:** Медленная установка
4. **Дисковое пространство:** Повторное использование пакетов

Архитектура pnpm

```
text

~/pnpm-store/           # Глобальное хранилище (симлинки)
├── v3/
│   ├── files/
│   │   ├── 00/          // Хэшированные пакеты
│   │   └── react@18.2.0
└──

project/
├── node_modules/
│   ├── .pnpm/           // Все зависимости (симлинки в хранилище)
│   │   ├── react@18.2.0@ -> ~/.pnpm-store/react@18.2.0
│   │   └── ...
│   ├── react -> .pnpm/react@18.2.0/node_modules/react
│   └── ...
└── package.json
```

3.2. Установка и использование

Начало работы

```
bash

# Установка
npm install -g pnpm           # Глобальная установка
corepack enable pnpm          # Через Corepack (Node.js 16+)

# Базовые команды
pnpm init                     # Инициализация
```



```
pnpm install          # Установка зависимостей
pnpm add react        # Добавить пакет
pnpm add -D typescript # Dev зависимость
pnpm update           # Обновить пакеты
pnpm remove lodash    # Удалить пакет
```

Особенности команд

```
bash

# Параллельная установка
pnpm fetch          # Скачать все пакеты в кэш
pnpm install --frozen-lockfile # Строгая установка

# Аудит и безопасность
pnpm audit          # Проверка безопасности
pnpm why react      # Зачем установлен react?

# Производительность
pnpm install --shamefully-hoist # Поднять зависимости (как npm)
pnpm store status      # Статус хранилища
pnpm store prune       # Очистить неиспользуемые пакеты
```

3.3. pnpm Workspaces

Конфигурация

```
json

// pnpm-workspace.yaml
packages:
  - 'apps/*'
  - 'packages/*'
  - '!**/test/**' # Исключить тестовые проекты
```

Структура монорепозитория

```
bash

# Фильтрация команд
pnpm --filter frontend add react # Добавить в конкретный проект
pnpm --filter backend... run build # Запустить в backend и его зависимостях
pnpm -r run test                  # Запустить во всех проектах (recursive)

# Параллельное выполнение
```

```
pnpm -r --parallel run dev      # Параллельный запуск
pnpm -r --stream run lint       # Поточковый вывод
```

3.4. Преимущества pnpm

Экономия места

```
bash
```

```
# Сравнение размеров для одинакового проекта:
# React + 50 зависимостей
```

```
npm/yarn:   ~250 MB  (полные копии в каждом проекте)
pnpm:       ~50 MB   (ссылки из глобального хранилища)
```

```
# Экономия: 80% дискового пространства
```

Производительность

```
javascript
```

```
// Бенчмарки установки (секунды):
```

```
const benchmarks = {
  cleanInstall: {
    npm: 45.2,
    yarn: 38.7,
    pnpm: 12.3 // В 3-4 раза быстрее
  },

  cachedInstall: {
    npm: 15.8,
    yarn: 12.4,
    pnpm: 2.1  // В 6-7 раз быстрее
  }
};
```

Часть 4: Сравнительный анализ

4.1. Таблица сравнения

Параметр	npm	yarn (classic)	yarn (berry)	pnpm
Скорость установки	Медленная	Средняя	Быстрая	Очень быстрая
Дисковое пространство	Много	Много	Мало	Очень мало
node_modules размер	Огромный	Большой	Отсутствует (PnP)	Компактный
Lock файл	package-lock.json	yarn.lock	yarn.lock	pnpm-lock.yaml
Детерминированность	Высокая	Высокая	Идеальная	Идеальная
Монорепозитории	Workspaces	Workspaces	Workspaces	Workspaces
Безопасность	audit	audit	audit + checksums	audit
Гибкость	Высокая	Высокая	Очень высокая	Средняя
Сообщество	Огромное	Большое	Растущее	Растущее
Лучше для	Начинающих, совместимости	Стабильности	Инноваций, perf	Производительности, экономии места

4.2. Детальное сравнение

Разрешение зависимостей

```
javascript
// Пример: Project зависит от A@1.0.0 и B@1.0.0,
// где A и B зависят от C@1.0.0 и C@2.0.0 соответственно
```

// npm/yarn (до дедупликации):

```
node_modules/
├── A/
│   └── node_modules/
│       └── C@1.0.0
└── B/
    └── node_modules/
        └── C@2.0.0
```

// npm/yarn (после дедупликации):

```
node_modules/
├── A/
└── B/
    └── C@2.0.0 // Выбрана одна версия (возможны конфликты)
```

// pnpm (всегда):

```
node_modules/
├── .pnpm/
│   ├── A@1.0.0/
│   │   └── node_modules/
│   │       ├── A -> ../../A@1.0.0
│   │       └── C@1.0.0
│   ├── B@1.0.0/
│   │   └── node_modules/
│   │       ├── B -> ../../B@1.0.0
│   │       └── C@2.0.0
│   ├── C@1.0.0/
│   └── C@2.0.0/
```

Производительность в реальных сценариях

javascript

// Бенчмарк: create-react-app с 150 зависимостями

```
const results = {
  firstInstall: {
    npm: "2m 15s",
    yarn: "1m 45s",
    pnpm: "45s"
  },

  repeatedInstall: {
    npm: "1m 10s",
    yarn: "50s",
    pnpm: "8s"
  }
}
```

```
},  
  
diskUsage: {  
  npm: "350MB",  
  yarn: "340MB",  
  pnpm: "70MB"  
}  
};
```

4.3. Критерии выбора

Выбирайте npm если:

1. Только начинаете изучать JavaScript
2. Работаете в команде, где все используют npm
3. Нужна максимальная совместимость
4. Проект небольшой и производительность не критична
5. Используете старые инструменты или системы

Выбирайте yarn (classic) если:

1. Нужна стабильность и предсказуемость
2. Работаете с React-проектами (изначально создан для React)
3. Уже есть опыт с yarn и не хотите менять workflow
4. Проект среднего размера с умеренными требованиями

Выбирайте yarn (berry) если:

1. Работаете в монорепозитории
2. Цените инновации и современные подходы
3. Нужна максимальная производительность при zero-installs
4. Готовы к более сложной настройке
5. Используете Plug'n'Play для исключения node_modules

Выбирайте pnpm если:

1. Критически важно дисковое пространство
2. Нужна максимальная скорость установки
3. Работаете с множеством проектов на одной машине
4. Хотите строгую изоляцию зависимостей
5. Используете монорепозиторий с эффективным кэшированием

Часть 5: Продвинутые сценарии использования

5.1. Монорепозитории в продакшене

Конфигурация для больших команд

```
json

// package.json в корне монорепозитория
{
  "name": "@company/monorepo",
  "private": true,
  "scripts": {
    "build": "turbo run build",
    "dev": "turbo run dev --parallel",
    "lint": "turbo run lint",
    "test": "turbo run test",
    "clean": "turbo run clean && rm -rf node_modules"
  },
  "devDependencies": {
    "turbo": "^1.10.0",
    "typescript": "^5.1.0"
  },
  "packageManager": "pnpm@8.6.0", // Фиксация менеджера
  "engines": {
    "node": ">=18.0.0",
    "pnpm": ">=8.0.0"
  }
}
```

Тулинг для монорепозиторий

```
bash

# Turborepo для кэширования сборок
npx create-turbo@latest # Создать монорепозиторий с турбо

# Nx для сложных монорепозиторий
npx create-nx-workspace@latest

# Lerna (устаревший, но еще используется)
npx lerna init
```

5.2. Оптимизация CI/CD пайплайнов

Кэширование зависимостей

```
yaml

# GitHub Actions с pnpm
name: CI
on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - uses: pnpm/action-setup@v2
        with:
          version: 8

      - uses: actions/setup-node@v3
        with:
          node-version: 18
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Build
        run: pnpm run build
```

Детерминированные сборки

```
bash

# Убедиться, что сборки воспроизводимы
# package.json
{
  "scripts": {
    "ci:install": "npm ci", # Вместо npm install
    "ci:install:yarn": "yarn install --frozen-lockfile",
    "ci:install:pnpm": "pnpm install --frozen-lockfile"
  }
}
```

5.3. Безопасность зависимостей

Автоматический аудит

```
json

// package.json
{
  "scripts": {
    "security": "npm audit --audit-level=high",
    "security:fix": "npm audit fix",
    "outdated": "npm outdated",
    "licenses": "npx license-checker --summary"
  }
}
```

Интеграция с инструментами безопасности

```
bash

# Snyk
npm install -g snyk
snyk test

# OWASP Dependency Check
npx audit-ci --critical

# Renovate bot для автообновлений
# .renovaterc.json
{
  "extends": ["config:base"],
  "automerge": true,
  "rangeStrategy": "bump"
}
```

5.4. Публикация пакетов

Подготовка пакета

```
json

{
  "name": "@scope/package-name",
  "version": "1.0.0",
}
```



```
"description": "Мой awesome пакет",
"main": "dist/index.js",
"module": "dist/index.esm.js",
"types": "dist/index.d.ts",
"files": ["dist"],
"scripts": {
  "build": "rollup -c",
  "prepublishOnly": "npm run build && npm test",
  "publish:dry": "npm publish --dry-run",
  "version": "npm run build && git add dist"
},
"publishConfig": {
  "access": "public",
  "registry": "https://registry.npmjs.org/"
}
}
```

Автоматическая публикация

```
yaml

# GitHub Actions для публикации
name: Publish
on:
  release:
    types: [published]

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: 18
          registry-url: 'https://registry.npmjs.org'

      - run: npm ci
      - run: npm test
      - run: npm run build

      - run: npm publish
    env:
      NODE_AUTH_TOKEN: ${ secrets.NPM_TOKEN }
```

Часть 6: Будущее пакетных менеджеров

6.1. Тренды 2026 года

Конвергенция возможностей

javascript

// Все менеджеры движутся к:

1. Единому lock-файлу формату
2. Улучшенной поддержке монорепозитория
3. Встроенной безопасности
4. Упрощенному UX

Новые технологии

bash

Corepack — стандартизация менеджеров в Node.js

corepack enable *# Включить corepack*

corepack prepare pnpm@latest --activate

Import maps — нативные ES модули в браузерах

<!-- index.html -->

<script type="importmap">

```
{
  "imports": {
    "react": "https://esm.sh/react@18"
  }
}
</script>
```

6.2. Рекомендации для новых проектов

Стартовый гайд 2026

bash

1. Инициализация проекта

mkdir my-project && cd my-project

2. Инициализация package.json

```
npm init -y
```

```
# 3. Установка pnpm (рекомендуется)
```

```
corepack enable pnpm
```

```
pnpm init
```

```
# 4. Добавление зависимостей
```

```
pnpm add react react-dom
```

```
pnpm add -D typescript vite @types/node
```

```
# 5. Настройка скриптов
```

```
# package.json
```

```
{  
  "scripts": {  
    "dev": "vite",  
    "build": "tsc && vite build",  
    "preview": "vite preview"  
  }  
}
```

Миграция между менеджерами

```
bash
```

```
# С npm на pnpm
```

```
rm -rf node_modules package-lock.json
```

```
pnpm import # Конвертирует package-lock.json
```

```
pnpm install
```

```
# С yarn на npm
```

```
rm -rf node_modules yarn.lock
```

```
npm install
```

```
# Между любыми менеджерами
```

```
npm synp # Конвертер Lock файлов
```

6.3. Лучшие практики 2026

Управление зависимостями

```
json
```

```
{  
  "engines": {
```

```
"node": ">=18.0.0",
"npm": ">=9.0.0"
},

"packageManager": "pnpm@8.6.0", // Фиксация версии менеджера

"scripts": {
  "postinstall": "husky install", // Постинсталляционные скрипты
  "preinstall": "npx only-allow pnpm" // Запрет других менеджеров
},

"overrides": { // Принудительные версии зависимостей
  "lodash": "4.17.21"
}
}
```

Оптимизация установки

```
bash

# .npmrc конфигурация
save-exact=true          # Точные версии
package-lock=true        # Всегда создавать Lock-файл
fund=false               # Отключить запросы на финансирование
audit=false              # Отключить авто-аудит (включать в CI)
progress=false           # Убрать прогресс-бар для CI
```

Заключение

Ключевые выводы

1. **npm** — стандарт де-факто, лучший для начинающих и максимальной совместимости
2. **yarn** — баланс между стабильностью и инновациями, особенно хорош для монорепозиторий
3. **pnpm** — революционный подход, лучшая производительность и экономия места

Рекомендации на 2026 год

Для большинства проектов:

- **Новые проекты:** `pnpm` или `yarn berry`
- **Существующие проекты:** оставайтесь на текущем менеджере, если нет проблем
- **Монорепозитории:** `yarn workspaces` или `pnpm workspaces`
- **Корпоративные проекты:** стандартизируйте на одном менеджере во всей организации

Тренды будущего:

1. Упрощение конфигурации
2. Улучшенная безопасность
3. Лучшая поддержка монорепозитория
4. Интеграция с нативными модулями браузеров

Философский итог

Пакетные менеджеры эволюционировали от простых установщиков скриптов до сложных систем управления зависимостями, которые:

1. **Гарантируют воспроизводимость** через `lock`-файлы
2. **Обеспечивают безопасность** через аудит и проверки
3. **Оптимизируют производительность** через кэширование и параллелизацию
4. **Упрощают разработку** через скрипты и `workspaces`

Независимо от выбора конкретного инструмента, понимание принципов работы пакетных менеджеров остается критически важным навыком для современного JavaScript-разработчика. Главное — выбрать инструмент, который лучше всего подходит для вашей команды, проекта и `workflow`, и постоянно следить за развитием экосистемы.

Глава 13: Работа с датами, ошибками и исключениями

13.1. Объект Date и работа с временем

Введение: Время в JavaScript

Работа с датами и временем — одна из самых сложных задач в программировании из-за множества факторов: часовых поясов, летнего времени, разных календарей, форматов представления. JavaScript предоставляет объект `Date` для решения этих задач, но у него есть свои особенности и подводные камни.

Исторический контекст

```
javascript

// JavaScript унаследовал объект Date из Java (1997)
// Изначальные проблемы:
// 1. Основан на Unix времени (миллисекунды с 1 января 1970 UTC)
// 2. Сложная работа с часовыми поясами
// 3. Неидеальный API

// Современные решения:
// 1. Temporal API (в разработке, TC39 Stage 3)
// 2. Библиотеки: Moment.js, date-fns, Day.js
// 3. Intl.DateTimeFormat для локализации
```

Часть 1: Основы объекта Date

1.1. Создание объектов Date

Пустой конструктор (текущее время)

```
javascript

// Текущая дата и время
```

```

const now = new Date();
console.log(now);
// Пример: 2026-02-10T14:30:45.123Z

// Аналоги через статические методы
const now1 = Date.now(); // timestamp в миллисекундах: 1718112645123
const now2 = Date.parse('2026-02-10'); // парсинг строки: 1739145600000
const now3 = new Date().getTime(); // аналог Date.now()

// Проверка типа
console.log(typeof now); // "object"
console.log(now instanceof Date); // true

```

Конструктор с миллисекундами (timestamp)

```

javascript

// Unix timestamp (миллисекунды с 1 января 1970 UTC)
const timestamp = 1739145600000; // 10 февраля 2026, 00:00:00 UTC

// Создание из timestamp
const dateFromTimestamp = new Date(timestamp);
console.log(dateFromTimestamp.toISOString()); // "2026-02-10T00:00:00.000Z"

// Важно: timestamp всегда в UTC
const localDate = new Date(0); // Ноль миллисекунд = 1 января 1970 UTC
console.log(localDate.toString());
// В Москве: "Thu Jan 01 1970 03:00:00 GMT+0300 (Москва, стандартное время)"

```

Конструктор со строкой

```

javascript

// Разные форматы строк (рекомендуется ISO 8601)
const date1 = new Date('2026-02-10'); // ISO формат
const date2 = new Date('2026-02-10T14:30:00'); // С временем
const date3 = new Date('2026-02-10T14:30:00Z'); // С указанием UTC
const date4 = new Date('2026-02-10T14:30:00+03:00'); // С часовым поясом

// Локализованные строки (осторожно - зависит от браузера/ОС)
const date5 = new Date('February 10, 2026');
const date6 = new Date('02/10/2026'); // MM/DD/YYYY (США)
const date7 = new Date('10.02.2026'); // DD.MM.YYYY (Россия)

// Проблемы с парсингом строк
console.log(new Date('2026-02-10').toString()); // UTC полночь

```

```
console.log(new Date('2026/02/10').toString()); // Локальная полночь
console.log(new Date('02/10/2026').toString()); // 10 февраля (США) или 2 октября?!
```

Конструктор с числовыми параметрами

```
javascript

// new Date(year, monthIndex, day, hours, minutes, seconds, milliseconds)

// Важные особенности:
// 1. year: полный год (2026), 2 цифры - 1900 + year
// 2. monthIndex: 0-11 (0 = январь, 11 = декабрь)
// 3. day: 1-31 (по умолчанию 1)
// 4. hours: 0-23 (по умолчанию 0)
// 5. minutes: 0-59 (по умолчанию 0)
// 6. seconds: 0-59 (по умолчанию 0)
// 7. milliseconds: 0-999 (по умолчанию 0)

// Примеры
const date8 = new Date(2026, 1, 10); // 10 февраля 2026, 00:00:00
const date9 = new Date(2026, 1, 10, 14, 30); // 10 февраля 2026, 14:30:00
const date10 = new Date(2026, 1, 10, 14, 30, 45, 123); // с миллисекундами

// Особые случаи
const date11 = new Date(2026, 12, 1); // 1 января 2027! (месяц 13)
const date12 = new Date(2026, 1, 0); // 31 января 2026 (день 0)
const date13 = new Date(2026, 1, 32); // 4 марта 2026 (автокоррекция)
```

1.2. Методы получения значений

Геттеры UTC и локального времени

```
javascript

const date = new Date('2026-02-10T14:30:45.123+03:00');

// Локальное время (зависит от часового пояса системы)
console.log('--- Локальное время ---');
console.log(date.getFullYear()); // 2026
console.log(date.getMonth()); // 1 (февраль, 0-based)
console.log(date.getDate()); // 10 (день месяца)
console.log(date.getDay()); // 2 (вторник, 0=воскресенье)
console.log(date.getHours()); // 14
console.log(date.getMinutes()); // 30
```



```

console.log(date.getSeconds());           // 45
console.log(date.getMilliseconds());      // 123
console.log(date.getTime());              // timestamp: 1739197845123

// UTC время (не зависит от часового пояса)
console.log('--- UTC время ---');
console.log(date.getUTCFullYear());       // 2026
console.log(date.getUTCMonth());          // 1
console.log(date.getUTCDate());           // 10
console.log(date.getUTCDay());            // 2
console.log(date.getUTCHours());          // 11 (14 - 3 часа)
console.log(date.getUTCMinutes());        // 30
console.log(date.getUTCSeconds());        // 45
console.log(date.getUTCMilliseconds());   // 123

// Дополнительные методы
console.log(date.getTimezoneOffset());    // -180 (минут от UTC, Москва: UTC+3 = -180)

```

Разница между UTC и локальным временем

javascript

```

const createComparisonTable = () => {
  const date = new Date('2026-02-10T14:30:45.123+03:00'); // Москва

  return {
    'Локальное (Москва, UTC+3)': {
      Часы: date.getHours(),           // 14
      Минуты: date.getMinutes(),       // 30
      Секунды: date.getSeconds()       // 45
    },
    'UTC': {
      Часы: date.getUTCHours(),         // 11 (14 - 3)
      Минуты: date.getUTCMinutes(),     // 30
      Секунды: date.getUTCSeconds()     // 45
    }
  };
};

console.table(createComparisonTable());

```

1.3. Методы установки значений

Сеттеры для модификации дат

```
javascript

// Создаем исходную дату
const date = new Date('2026-02-10T14:30:45.123Z');

console.log('Исходная:', date.toISOString()); // 2026-02-10T14:30:45.123Z

// Установка локального времени
date.setFullYear(2027);
date.setMonth(5);           // июнь (0-based)
date.setDate(15);
date.setHours(20);
date.setMinutes(45);
date.setSeconds(30);
date.setMilliseconds(500);

console.log('После изменений:', date.toISOString()); // 2027-06-15T17:45:30.500Z

// Установка UTC времени
date.setUTCFullYear(2028);
date.setUTCMonth(11);      // декабрь
date.setUTCDate(25);
date.setUTCHours(0);
date.setUTCMinutes(0);
date.setUTCSeconds(0);
date.setUTCMilliseconds(0);

console.log('UTC изменения:', date.toISOString()); // 2028-12-25T00:00:00.000Z
```

Автокоррекция дат

```
javascript

const date = new Date(2026, 0, 31); // 31 января 2026

console.log('Исходная:', date.toString()); // Sat Jan 31 2026 00:00:00

// Устанавливаем февраль (в феврале максимум 28/29 дней)
date.setMonth(1); // февраль
console.log('После setMonth(1):', date.toString()); // Mon Mar 03 2026 00:00:00
// Автоматически скорректировалось на 3 марта!
```

```
// Устанавливаем 40 часов
date.setHours(40);
console.log('После setHours(40):', date.toString()); // Tue Mar 04 2026 16:00:00
// +16 часов к следующему дню
```

```
// Устанавливаем отрицательные значения
date.setDate(-5);
console.log('После setDate(-5):', date.toString()); // Thu Feb 20 2026 16:00:00
```

Полезные паттерны модификации

javascript

```
// Добавление дней
```

```
function addDays(date, days) {
  const result = new Date(date);
  result.setDate(result.getDate() + days);
  return result;
}
```

```
// Добавление месяцев (с учетом разного количества дней)
```

```
function addMonths(date, months) {
  const result = new Date(date);
  result.setMonth(result.getMonth() + months);
  return result;
}
```

```
// Добавление лет
```

```
function addYears(date, years) {
  const result = new Date(date);
  result.setFullYear(result.getFullYear() + years);
  return result;
}
```

```
// Пример использования
```

```
const today = new Date('2026-02-10');
console.log('Сегодня:', today.toDateString());
console.log('Через 7 дней:', addDays(today, 7).toDateString());
console.log('Через 3 месяца:', addMonths(today, 3).toDateString());
console.log('Через 2 года:', addYears(today, 2).toDateString());
```

Часть 2: Форматирование и вывод дат

2.1. Методы преобразования в строку

Основные методы toString

```
javascript

const date = new Date('2026-02-10T14:30:45.123+03:00');

// toString() - локальное время в читаемом формате
console.log(date.toString());
// "Tue Feb 10 2026 14:30:45 GMT+0300 (Москва, стандартное время)"

// toUTCString() - UTC время
console.log(date.toUTCString());
// "Tue, 10 Feb 2026 11:30:45 GMT"

// toISOString() - ISO 8601 (всегда UTC)
console.log(date.toISOString());
// "2026-02-10T11:30:45.123Z"

// toJSON() - для JSON.stringify (аналогично toISOString)
console.log(date.toJSON());
// "2026-02-10T11:30:45.123Z"

// toString() - только дата
console.log(date.toDateString());
// "Tue Feb 10 2026"

// toTimeString() - только время
console.log(date.toTimeString());
// "14:30:45 GMT+0300 (Москва, стандартное время)"

// toLocaleString() - локализованный формат
console.log(date.toLocaleString());
// "10.02.2026, 14:30:45" (зависит от локали)
```

Сравнение методов вывода

```
javascript

const createStringMethodsComparison = (date) => {
```

```

return {
  'toString()': date.toString(),
  'toUTCString()': date.toUTCString(),
  'toISOString()': date.toISOString(),
  'toDateString()': date.toDateString(),
  'getTimeString()': date.getTimeString(),
  'toLocaleString()': date.toLocaleString('ru-RU'),
  'toLocaleDateString()': date.toLocaleDateString('ru-RU'),
  'toLocaleTimeString()': date.toLocaleTimeString('ru-RU')
};
};

const exampleDate = new Date('2026-02-10T14:30:45.123+03:00');
console.table(createStringMethodsComparison(exampleDate));

```

2.2. Локализация с Intl.DateTimeFormat

Базовое использование

```

javascript

const date = new Date('2026-02-10T14:30:45.123');

// Создание форматтера
const formatter = new Intl.DateTimeFormat('ru-RU', {
  dateStyle: 'full',
  timeStyle: 'long',
  timeZone: 'Europe/Moscow'
});

console.log(formatter.format(date));
// "вторник, 10 февраля 2026 г., 14:30:45 GMT+3"

// Разные стили даты
const styles = ['full', 'long', 'medium', 'short'];
styles.forEach(style => {
  const f = new Intl.DateTimeFormat('ru-RU', { dateStyle: style });
  console.log(`${style}: ${f.format(date)}`);
});
// full: вторник, 10 февраля 2026 г.
// long: 10 февраля 2026 г.
// medium: 10 февр. 2026 г.
// short: 10.02.2026

```

Расширенные настройки

```
javascript

const date = new Date('2026-02-10T14:30:45.123');

// Кастомизированный формат
const customFormatter = new Intl.DateTimeFormat('ru-RU', {
  weekday: 'long',    // "вторник"
  year: 'numeric',    // "2026"
  month: 'long',      // "февраль"
  day: 'numeric',     // "10"
  hour: '2-digit',    // "14"
  minute: '2-digit',  // "30"
  second: '2-digit',  // "45"
  hour12: false,      // 24-часовой формат
  timeZoneName: 'longOffset' // "GMT+03:00"
});

console.log(customFormatter.format(date));
// "вторник, 10 февраля 2026 г., 14:30:45 GMT+03:00"

// Форматирование частей даты
const partsFormatter = new Intl.DateTimeFormat('ru-RU', {
  weekday: 'long',
  month: 'long',
  day: 'numeric'
});

const parts = partsFormatter.formatToParts(date);
console.log(parts);
// [
//   { type: 'weekday', value: 'вторник' },
//   { type: 'literal', value: ', ' },
//   { type: 'day', value: '10' },
//   { type: 'literal', value: ' ' },
//   { type: 'month', value: 'февраля' },
//   { type: 'literal', value: ' ' },
//   { type: 'year', value: '2026' },
//   { type: 'literal', value: ' г.' }
// ]
```

Часовые пояса

javascript

```

const date = new Date('2026-02-10T14:30:45.123Z');

// Разные часовые пояса
const timezones = [
  'UTC',
  'Europe/Moscow',
  'America/New_York',
  'Asia/Tokyo',
  'Australia/Sydney'
];

timezones.forEach(tz => {
  const formatter = new Intl.DateTimeFormat('ru-RU', {
    dateStyle: 'long',
    timeStyle: 'long',
    timeZone: tz
  });
  console.log(`${tz}: ${formatter.format(date)}`);
});

// UTC: 10 февраля 2026 г., 14:30:45 UTC
// Europe/Moscow: 10 февраля 2026 г., 17:30:45 GMT+3
// America/New_York: 10 февраля 2026 г., 09:30:45 GMT-5
// Asia/Tokyo: 10 февраля 2026 г., 23:30:45 GMT+9
// Australia/Sydney: 11 февраля 2026 г., 01:30:45 GMT+11

```

2.3. Ручное форматирование

Создание кастомных форматов

```

javascript

// Утилиты для форматирования
const dateFormatters = {
  // DD.MM.YYYY
  formatDMY: (date) => {
    const day = String(date.getDate()).padStart(2, '0');
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const year = date.getFullYear();
    return `${day}.${month}.${year}`;
  },

  // YYYY-MM-DD

```

```

formatYMD: (date) => {
  return date.toISOString().split('T')[0];
},

// HH:MM:SS
formatTime: (date) => {
  const hours = String(date.getHours()).padStart(2, '0');
  const minutes = String(date.getMinutes()).padStart(2, '0');
  const seconds = String(date.getSeconds()).padStart(2, '0');
  return `${hours}:${minutes}:${seconds}`;
},

// Человекочитаемый формат
formatHuman: (date) => {
  const now = new Date();
  const diffMs = now - date;
  const diffSec = Math.floor(diffMs / 1000);
  const diffMin = Math.floor(diffSec / 60);
  const diffHour = Math.floor(diffMin / 60);
  const diffDay = Math.floor(diffHour / 24);

  if (diffSec < 60) return 'только что';
  if (diffMin < 60) return `${diffMin} минут назад`;
  if (diffHour < 24) return `${diffHour} часов назад`;
  if (diffDay === 1) return 'вчера';
  if (diffDay === 2) return 'позавчера';
  if (diffDay < 7) return `${diffDay} дней назад`;

  return dateFormatters.formatDMY(date);
}
};

// Пример использования
const date = new Date('2026-02-10T14:30:45');
console.log(dateFormatters.formatDMY(date)); // 10.02.2026
console.log(dateFormatters.formatYMD(date)); // 2026-02-10
console.log(dateFormatters.formatTime(date)); // 14:30:45
console.log(dateFormatters.formatHuman(date)); // зависит от текущего времени

```

Форматирование с учетом локали без Intl

javascript

```

// Кэширование названий месяцев и дней
const monthNames = {

```



```

ru: [
  'январь', 'февраль', 'март', 'апрель', 'май', 'июнь',
  'июль', 'август', 'сентябрь', 'октябрь', 'ноябрь', 'декабрь'
],
en: [
  'January', 'February', 'March', 'April', 'May', 'June',
  'July', 'August', 'September', 'October', 'November', 'December'
]
};

const dayNames = {
  ru: ['воскресенье', 'понедельник', 'вторник', 'среда', 'четверг', 'пятница', 'суббота'],
  en: ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']
};

function formatLocalized(date, locale = 'ru') {
  const year = date.getFullYear();
  const month = monthNames[locale][date.getMonth()];
  const day = date.getDate();
  const dayOfWeek = dayNames[locale][date.getDay()];
  const hours = String(date.getHours()).padStart(2, '0');
  const minutes = String(date.getMinutes()).padStart(2, '0');

  return {
    full: `${dayOfWeek}, ${day} ${month} ${year} г., ${hours}:${minutes}`,
    dateOnly: `${day} ${month} ${year} г.`,
    timeOnly: `${hours}:${minutes}`
  };
}

const date = new Date('2026-02-10T14:30:45');
console.log(formatLocalized(date, 'ru').full); // вторник, 10 февраль 2026 г., 14:30
console.log(formatLocalized(date, 'en').full); // Tuesday, 10 February 2026, 14:30

```

Часть 3: Работа с интервалами и разницей времени

3.1. Разница между датами

Базовые вычисления

javascript

// Разница в миллисекундах

```
function getTimeDifference(date1, date2) {  
  return date2.getTime() - date1.getTime();  
}
```

// Конвертация миллисекунд в читаемый формат

```
function formatTimeDifference(ms) {  
  const isNegative = ms < 0;  
  const absoluteMs = Math.abs(ms);  
  
  const seconds = Math.floor(absoluteMs / 1000);  
  const minutes = Math.floor(seconds / 60);  
  const hours = Math.floor(minutes / 60);  
  const days = Math.floor(hours / 24);  
  
  return {  
    milliseconds: absoluteMs,  
    seconds: seconds,  
    minutes: minutes,  
    hours: hours,  
    days: days,  
    formatted: `${isNegative ? '-' : ''}${days}д ${hours % 24}ч ${minutes % 60}м ${seconds % 60}с`,  
    isNegative  
  };  
}
```

// Пример

```
const start = new Date('2026-02-10T09:00:00');  
const end = new Date('2026-02-11T14:30:45');  
const diff = getTimeDifference(start, end);  
console.log(formatTimeDifference(diff));  
  
// {  
//   milliseconds: 107445000,  
//   seconds: 107445,  
//   minutes: 1790,  
//   hours: 29,  
//   days: 1,  
//   formatted: "1д 5ч 30м 45с",  
//   isNegative: false  
// }
```

Точные вычисления с учетом месяцев и лет

javascript

```
function getExactDifference(fromDate, toDate) {
    let years = toDate.getFullYear() - fromDate.getFullYear();
    let months = toDate.getMonth() - fromDate.getMonth();
    let days = toDate.getDate() - fromDate.getDate();
    let hours = toDate.getHours() - fromDate.getHours();
    let minutes = toDate.getMinutes() - fromDate.getMinutes();
    let seconds = toDate.getSeconds() - fromDate.getSeconds();
    let milliseconds = toDate.getMilliseconds() - fromDate.getMilliseconds();

    // Коррекция отрицательных значений
    if (milliseconds < 0) {
        seconds -= 1;
        milliseconds += 1000;
    }

    if (seconds < 0) {
        minutes -= 1;
        seconds += 60;
    }

    if (minutes < 0) {
        hours -= 1;
        minutes += 60;
    }

    if (hours < 0) {
        days -= 1;
        hours += 24;
    }

    if (days < 0) {
        months -= 1;
        // Количество дней в предыдущем месяце
        const tempDate = new Date(toDate.getFullYear(), toDate.getMonth(), 0);
        days += tempDate.getDate();
    }

    if (months < 0) {
        years -= 1;
        months += 12;
    }
}
```

```

    }

    return { years, months, days, hours, minutes, seconds, milliseconds };
}

// Пример
const birthDate = new Date('2000-02-10T09:30:00');
const currentDate = new Date('2026-02-10T14:45:30.500');
const exactDiff = getExactDifference(birthDate, currentDate);

console.log(exactDiff);
// {
//   years: 26,
//   months: 0,
//   days: 0,
//   hours: 5,
//   minutes: 15,
//   seconds: 30,
//   milliseconds: 500
// }

```

3.2. Сравнение дат

Методы сравнения

```

javascript

const date1 = new Date('2026-02-10T10:00:00');
const date2 = new Date('2026-02-10T14:30:00');
const date3 = new Date('2026-02-10T10:00:00'); // такая же как date1

// Прямое сравнение timestamp
console.log(date1.getTime() === date3.getTime()); // true
console.log(date1.getTime() < date2.getTime());    // true
console.log(date1.getTime() > date2.getTime());    // false

// Сравнение объектов (не работает как ожидается!)
console.log(date1 == date3); // false (сравнение объектов)
console.log(date1 === date3); // false (разные объекты)

// Утилиты для сравнения
const dateComparators = {
  isEqual: (a, b) => a.getTime() === b.getTime(),

```

```

isBefore: (a, b) => a.getTime() < b.getTime(),
isAfter: (a, b) => a.getTime() > b.getTime(),
isSameDay: (a, b) => {
    return a.getFullYear() === b.getFullYear() &&
        a.getMonth() === b.getMonth() &&
        a.getDate() === b.getDate();
},
isSameMonth: (a, b) => {
    return a.getFullYear() === b.getFullYear() &&
        a.getMonth() === b.getMonth();
},
isSameYear: (a, b) => a.getFullYear() === b.getFullYear()
};

// Примеры
console.log(dateComparators.isEqual(date1, date3)); // true
console.log(dateComparators.isBefore(date1, date2)); // true
console.log(dateComparators.isSameDay(date1, date2)); // true
console.log(dateComparators.isSameDay(date1, new Date('2026-02-11'))); // false

```

Проверка диапазонов

```

javascript

// Проверка, находится ли дата в диапазоне
function isDateInRange(date, start, end, inclusive = true) {
    const time = date.getTime();
    const startTime = start.getTime();
    const endTime = end.getTime();

    if (inclusive) {
        return time >= startTime && time <= endTime;
    } else {
        return time > startTime && time < endTime;
    }
}

// Проверка пересечения интервалов
function doDatesOverlap(start1, end1, start2, end2) {
    return start1.getTime() <= end2.getTime() &&
        start2.getTime() <= end1.getTime();
}

```

```

// Пример
const appointmentStart = new Date('2026-02-10T10:00:00');

```

```

const appointmentEnd = new Date('2026-02-10T11:00:00');
const checkTime = new Date('2026-02-10T10:30:00');
const anotherAppointmentStart = new Date('2026-02-10T10:45:00');
const anotherAppointmentEnd = new Date('2026-02-10T11:30:00');

console.log(isDateInRange(checkTime, appointmentStart, appointmentEnd)); // true
console.log(doDatesOverlap(appointmentStart, appointmentEnd,
    anotherAppointmentStart, anotherAppointmentEnd)); // true

```

3.3. Таймеры и интервалы

Использование setTimeout и setInterval

javascript

// Простой таймер

```

function startTimer(duration, callback) {
    const startTime = Date.now();
    const endTime = startTime + duration;

    const timer = setInterval(() => {
        const now = Date.now();
        const remaining = endTime - now;

        if (remaining <= 0) {
            clearInterval(timer);
            callback(0, true); // завершено
        } else {
            callback(remaining, false); // еще идет
        }
    }, 100); // обновляем каждые 100мс для плавности
}

```

// Форматирование времени для таймера

```

function formatRemainingTime(ms) {
    const totalSeconds = Math.floor(ms / 1000);
    const hours = Math.floor(totalSeconds / 3600);
    const minutes = Math.floor((totalSeconds % 3600) / 60);
    const seconds = totalSeconds % 60;

    if (hours > 0) {
        return `${hours}:${String(minutes).padStart(2, '0')}:${String(seconds).padStart(2, '0')}`;
    }
}

```

```

    } else {
      return `${minutes}:${String(seconds).padStart(2, '0')}`;
    }
  }
}

// Пример использования
console.log('Таймер запущен на 5 секунд');
setTimeout(5000, (remaining, isComplete) => {
  if (isComplete) {
    console.log('Таймер завершен!');
  } else {
    console.log(`Осталось: ${formatRemainingTime(remaining)}`);
  }
});

```

Точное измерение времени выполнения

javascript

```

// Измерение времени выполнения функции
function measureExecutionTime(fn, ...args) {
  const start = performance.now(); // Более точный чем Date.now()
  const result = fn(...args);
  const end = performance.now();

  return {
    result,
    duration: end - start,
    durationMs: Math.round(end - start),
    durationFormatted: `${(end - start).toFixed(2)} мс`
  };
}

// Пример: измерение сортировки
function heavyComputation() {
  let sum = 0;
  for (let i = 0; i < 10000000; i++) {
    sum += Math.sqrt(i);
  }
  return sum;
}

const measurement = measureExecutionTime(heavyComputation);
console.log(`Выполнение заняло: ${measurement.durationFormatted}`);
console.log(`Результат: ${measurement.result.toFixed(2)}`);

```

Часть 4: Часовые пояса и летнее время

4.1. Понимание часовых поясов

Работа с различными часовыми поясами

```
javascript

// Получение текущего часового пояса
function getCurrentTimezoneInfo() {
  const now = new Date();

  return {
    // Смещение от UTC в минутах
    timezoneOffset: now.getTimezoneOffset(),

    // Смещение в формате +HH:MM
    timezoneOffsetFormatted: () => {
      const offset = -now.getTimezoneOffset(); // инвертируем знак
      const sign = offset >= 0 ? '+' : '-';
      const hours = Math.floor(Math.abs(offset) / 60);
      const minutes = Math.abs(offset) % 60;
      return `${sign}${String(hours).padStart(2, '0')}:${String(minutes).padStart(2, '0')}`;
    },

    // Название часового пояса (если доступно)
    timezoneName: Intl.DateTimeFormat().resolvedOptions().timeZone,

    // Текущее время в UTC
    utcTime: now.toUTCString(),

    // Текущее локальное время
    localTime: now.toString()
  };
}

console.log(getCurrentTimezoneInfo());
// Пример для Москвы:
// {
//   timezoneOffset: -180,
//   timezoneOffsetFormatted: "+03:00",
//   timezoneName: "Europe/Moscow",
```



```
// utcTime: "Tue, 10 Feb 2026 11:30:45 GMT",  
// LocalTime: "Tue Feb 10 2026 14:30:45 GMT+0300 (Москва, стандартное время)"  
// }
```

Конвертация между часовыми поясами

javascript

```
// Конвертация времени между часовыми поясами  
function convertTimezone(date, fromTimezone, toTimezone) {  
  // Создаем форматтеры для исходного и целевого поясов  
  const fromFormatter = new Intl.DateTimeFormat('en-US', {  
    timeZone: fromTimezone,  
    year: 'numeric',  
    month: '2-digit',  
    day: '2-digit',  
    hour: '2-digit',  
    minute: '2-digit',  
    second: '2-digit',  
    hour12: false  
  });  
  
  const toFormatter = new Intl.DateTimeFormat('en-US', {  
    timeZone: toTimezone,  
    year: 'numeric',  
    month: '2-digit',  
    day: '2-digit',  
    hour: '2-digit',  
    minute: '2-digit',  
    second: '2-digit',  
    hour12: false  
  });  
  
  // Парсим дату из строки  
  const formatDate = (dateObj, formatter) => {  
    const parts = formatter.formatToParts(dateObj);  
    const map = {};  
    parts.forEach(part => {  
      if (part.type !== 'literal') {  
        map[part.type] = part.value;  
      }  
    });  
  
    return new Date(  
      `${map.year}-${map.month}-${map.day}T${map.hour}:${map.minute}:${map.second}`  
    );  
  };  
}
```

```

    );
};

// Конвертируем
const convertedDate = formatDate(date, toFormatter);

return {
  original: {
    date: date,
    timezone: fromTimezone,
    formatted: fromFormatter.format(date)
  },
  converted: {
    date: convertedDate,
    timezone: toTimezone,
    formatted: toFormatter.format(date)
  },
  difference: {
    hours: (convertedDate.getTime() - date.getTime()) / (1000 * 60 * 60)
  }
};
}

// Пример
const moscowTime = new Date('2026-02-10T14:30:00');
const conversion = convertTimezone(moscowTime, 'Europe/Moscow', 'America/New_York');
console.log(conversion);

```

4.2. Летнее время (Daylight Saving Time)

Определение, действует ли летнее время

```

javascript

// Проверка, действует ли летнее время для даты
function isDST(date, timezone = '') {
  const jan = new Date(date.getFullYear(), 0, 1);
  const jul = new Date(date.getFullYear(), 6, 1);

  if (timezone) {
    // Используем Intl для точного определения
    const formatter = new Intl.DateTimeFormat('en-US', {
      timeZone: timezone,

```

```

    timeZoneName: 'long'
  });

  const parts = formatter.formatToParts(date);
  const timezoneName = parts.find(p => p.type === 'timeZoneName')?.value || '';

  return timezoneName.includes('Daylight') || timezoneName.includes('Summer');
} else {
  // Приблизительная проверка для локального времени
  const stdOffset = jan.getTimezoneOffset();
  const dstOffset = jul.getTimezoneOffset();

  // В северном полушарии летнее время обычно с марта по октябрь
  return date.getTimezoneOffset() < Math.max(stdOffset, dstOffset);
}
}

// Определение дат перехода на летнее время
function getDSTTransitions(year, timezone = 'Europe/Moscow') {
  const transitions = [];

  // Проверяем каждый день года
  for (let month = 0; month < 12; month++) {
    const daysInMonth = new Date(year, month + 1, 0).getDate();

    for (let day = 1; day <= daysInMonth; day++) {
      const date = new Date(year, month, day, 12); // полдень
      const formatter = new Intl.DateTimeFormat('en-US', {
        timeZone: timezone,
        timeZoneName: 'long'
      });

      const parts = formatter.formatToParts(date);
      const timezoneName = parts.find(p => p.type === 'timeZoneName')?.value || '';

      // Проверяем, содержит ли название указание на летнее время
      const isDSTNow = timezoneName.includes('Daylight') ||
        timezoneName.includes('Summer');

      // Если предыдущий день был другим, значит это переход
      const prevDate = new Date(year, month, day - 1, 12);
      const prevParts = formatter.formatToParts(prevDate);
      const prevTimezoneName = prevParts.find(p => p.type === 'timeZoneName')?.value || '';
      const wasDSTPrev = prevTimezoneName.includes('Daylight') ||

```

```

    prevTimezoneName.includes('Summer');

    if (isDSTNow !== wasDSTPrev) {
        transitions.push({
            date: date,
            type: isDSTNow ? 'start' : 'end',
            timezoneName: timezoneName
        });
    }
}
}

return transitions;
}

// Пример
const testDate = new Date('2026-06-10');
console.log(`Летнее время для ${testDate.toDateString()}:`, isDST(testDate,
'Europe/Moscow'));

const transitions2026 = getDSTTransitions(2026, 'Europe/Moscow');
console.log('Переходы на летнее время в 2026:', transitions2026);

```

Часть 5: Календарные вычисления

5.1. Работа с днями недели

Определение рабочих дней и выходных

```

javascript

// Конфигурация календаря
const calendarConfig = {
    // 0 - воскресенье, 6 - суббота
    weekendDays: [0, 6],

    // Праздничные дни (в формате YYYY-MM-DD)
    holidays: {
        '2026': [
            '2026-01-01', // Новый год
            '2026-01-02', // Продолжение новогодних каникул
            '2026-01-07', // Рождество

```

```

    '2026-02-23', // День защитника Отечества
    '2026-03-08', // Международный женский день
    '2026-05-01', // Праздник Весны и Труда
    '2026-05-09', // День Победы
    '2026-06-12', // День России
    '2026-11-04' // День народного единства
  ]
},

// Переносы выходных дней
movedWeekends: {
  '2026': {
    '2026-01-03': '2026-01-10' // пример переноса
  }
}
};

// Проверка, является ли день рабочим
function isWorkday(date, config = calendarConfig) {
  const dayOfWeek = date.getDay();

  // Проверяем выходные
  if (config.weekendDays.includes(dayOfWeek)) {
    return false;
  }

  // Проверяем праздничные дни
  const dateStr = date.toISOString().split('T')[0];
  const year = date.getFullYear().toString();

  if (config.holidays[year]?.includes(dateStr)) {
    return false;
  }

  // Проверяем переносы
  if (config.movedWeekends[year]?.[dateStr]) {
    // Если этот день был перенесен с выходного, то это рабочий день
    return true;
  }

  return true;
}

// Получение следующего рабочего дня

```

```

function getNextWorkday(startDate, config = calendarConfig) {
    let currentDate = new Date(startDate);

    do {
        currentDate.setDate(currentDate.getDate() + 1);
    } while (!isWorkday(currentDate, config));

    return new Date(currentDate);
}

// Пример
const testDate = new Date('2026-02-10'); // вторник
console.log(`Рабочий день ${testDate.toDateString()}?`, isWorkday(testDate)); // true

const weekendDate = new Date('2026-02-14'); // суббота
console.log(`Рабочий день ${weekendDate.toDateString()}?`, isWorkday(weekendDate)); //
false

const nextWorkday = getNextWorkday(weekendDate);
console.log(`Следующий рабочий день после ${weekendDate.toDateString()}:`,
nextWorkday.toDateString());

```

Расчет количества рабочих дней в периоде

```

javascript

function countWorkdays(startDate, endDate, config = calendarConfig) {
    let count = 0;
    let currentDate = new Date(startDate);
    const end = new Date(endDate);

    // Нормализуем время для сравнения только дат
    currentDate.setHours(0, 0, 0, 0);
    end.setHours(0, 0, 0, 0);

    while (currentDate <= end) {
        if (isWorkday(currentDate, config)) {
            count++;
        }
        currentDate.setDate(currentDate.getDate() + 1);
    }

    return count;
}

```

```
// Пример
const periodStart = new Date('2026-02-01');
const periodEnd = new Date('2026-02-28');
const workdaysCount = countWorkdays(periodStart, periodEnd);

console.log(`Рабочих дней с ${periodStart.toDateString()} по ${periodEnd.toDateString()}:
${workdaysCount}`);
```

5.2. Работа с неделями и месяцами

Определение номеров недель

```
javascript

// Получение номера недели в году (ISO 8601)
function getISOWeekNumber(date) {
    // Создаем копию даты
    const d = new Date(Date.UTC(date.getFullYear(), date.getMonth(), date.getDate()));

    // Устанавливаем на четверг этой недели
    const dayNum = d.getUTCDay() || 7; // 1-7, понедельник=1
    d.setUTCDate(d.getUTCDate() + 4 - dayNum);

    // Получаем первый день года
    const yearStart = new Date(Date.UTC(d.getUTCFullYear(), 0, 1));

    // Вычисляем номер недели
    const weekNo = Math.ceil(((d - yearStart) / 86400000 + 1) / 7);

    return weekNo;
}

// Получение дат недели по номеру недели и году
function getDatesOfWeek(year, weekNumber) {
    // Простой четверг указанной недели
    const simple = new Date(year, 0, 1 + (weekNumber - 1) * 7);

    // Получаем день недели для 1 января
    const dayOfWeek = simple.getDay();

    // ISO 8601: неделя начинается с понедельника
    // Если 1 января - пятница, суббота или воскресенье,
    // то первая неделя начинается в следующем понедельнике
```

```

let ISOWeekStart = simple;
if (dayOfWeek <= 4) {
  // Переходим на предыдущий понедельник
  ISOWeekStart.setDate(simple.getDate() - simple.getDay() + 1);
} else {
  // Переходим на следующий понедельник
  ISOWeekStart.setDate(simple.getDate() + 8 - simple.getDay());
}

// Генерируем все дни недели
const weekDates = [];
for (let i = 0; i < 7; i++) {
  const date = new Date(ISOWeekStart);
  date.setDate(ISOWeekStart.getDate() + i);
  weekDates.push(date);
}

return weekDates;
}

// Пример
const testDate = new Date('2026-02-10');
const weekNumber = getISOWeekNumber(testDate);
console.log(`Неделя для ${testDate.toDateString()}: ${weekNumber}`);

const weekDates = getDatesOfWeek(2026, weekNumber);
console.log('Даты недели:', weekDates.map(d => d.toDateString()));

```

Работа с месяцами

```

javascript

// Утилиты для работы с месяцами
const monthUtils = {
  // Получение количества дней в месяце
  getDaysInMonth: (year, month) => {
    return new Date(year, month + 1, 0).getDate();
  },

  // Получение первого дня месяца
  getFirstDayOfMonth: (year, month) => {
    return new Date(year, month, 1);
  },

  // Получение последнего дня месяца

```



```

getLastDayOfMonth: (year, month) => {
  return new Date(year, month + 1, 0);
},

// Проверка, является ли год високосным
isLeapYear: (year) => {
  return (year % 4 === 0 && year % 100 !== 0) || (year % 400 === 0);
},

// Генерация календаря на месяц
generateMonthCalendar: (year, month) => {
  const firstDay = monthUtils.getFirstDayOfMonth(year, month);
  const lastDay = monthUtils.getLastDayOfMonth(year, month);
  const daysInMonth = monthUtils.getDaysInMonth(year, month);

  // День недели первого дня (0 - воскресенье)
  const firstDayOfWeek = firstDay.getDay();

  const calendar = {
    year,
    month,
    monthName: firstDay.toLocaleString('ru-RU', { month: 'long' }),
    days: [],
    weeks: []
  };

  // Заполняем дни месяца
  for (let day = 1; day <= daysInMonth; day++) {
    const date = new Date(year, month, day);
    calendar.days.push({
      date,
      dayOfMonth: day,
      dayOfWeek: date.getDay(),
      dayName: date.toLocaleString('ru-RU', { weekday: 'short' }),
      isWeekend: date.getDay() === 0 || date.getDay() === 6,
      isToday: date.toDateString() === new Date().toDateString()
    });
  }

  // Разбиваем на недели
  let week = [];
  // Добавляем пустые дни в начале первой недели
  for (let i = 0; i < firstDayOfWeek; i++) {
    week.push(null);
  }
}

```

```

}

calendar.days.forEach(day => {
  week.push(day);
  if (week.length === 7) {
    calendar.weeks.push([...week]);
    week = [];
  }
});

// Добавляем оставшиеся дни последней недели
if (week.length > 0) {
  while (week.length < 7) {
    week.push(null);
  }
  calendar.weeks.push(week);
}

return calendar;
}
};

// Пример
const february2026 = monthUtils.generateMonthCalendar(2026, 1); // февраль
console.log(`Календарь ${february2026.monthName} ${february2026.year}:`);
console.log(`Дней в месяце: ${february2026.days.length}`);
console.log(`Високосный год: ${monthUtils.isLeapYear(2026)}`);

// Вывод календаря в консоль
console.log('Пн Вт Ср Чт Пт Сб Вс');
february2026.weeks.forEach(week => {
  const weekStr = week.map(day =>
    day ? (day.dayOfMonth < 10 ? ` ${day.dayOfMonth}` : day.dayOfMonth) : ' '
  ).join(' ');
  console.log(weekStr);
});

```

Часть 6: Производительность и оптимизация

6.1. Кэширование и мемоизация

Оптимизация частых операций с датами

```
javascript

// Кэш для форматирования дат
const dateFormatCache = new Map();

function formatDateCached(date, formatFn, cacheKey) {
  const key = `${cacheKey}_${date.getTime()}`;

  if (dateFormatCache.has(key)) {
    return dateFormatCache.get(key);
  }

  const formatted = formatFn(date);
  dateFormatCache.set(key, formatted);

  // Ограничиваем размер кэша
  if (dateFormatCache.size > 1000) {
    const firstKey = dateFormatCache.keys().next().value;
    dateFormatCache.delete(firstKey);
  }

  return formatted;
}

// Кэшированные версии популярных форматов
const cachedFormatters = {
  toISOString: (date) => formatDateCached(date, d => d.toISOString(), 'iso'),
  toLocaleDateString: (date, locale, options) =>
    formatDateCached(date, d => d.toLocaleDateString(locale, options),
      `locale_${locale}_${JSON.stringify(options)}`),
  formatRelative: (date) => formatDateCached(date, relativeTimeFormat, 'relative')
};

// Форматирование относительного времени (сегодня, вчера и т.д.)
function relativeTimeFormat(date) {
  const now = new Date();
```

```

const diffMs = now - date;
const diffDays = Math.floor(diffMs / (1000 * 60 * 60 * 24));

if (diffDays === 0) return 'сегодня';
if (diffDays === 1) return 'вчера';
if (diffDays === 2) return 'позавчера';
if (diffDays < 7) return `${diffDays} дней назад`;

return date.toLocaleDateString('ru-RU');
}

// Пример использования кэша
const testDate = new Date();
const iterations = 10000;

console.time('Без кэша');
for (let i = 0; i < iterations; i++) {
  testDate.toLocaleDateString('ru-RU');
}
console.timeEnd('Без кэша');

console.time('С кэшем');
for (let i = 0; i < iterations; i++) {
  cachedFormatters.toLocaleDateString(testDate, 'ru-RU');
}
console.timeEnd('С кэшем');

```

6.2. Бенчмаркинг операций с датами

Сравнение производительности разных подходов

```

javascript

function benchmarkDateOperations() {
  const operations = [
    {
      name: 'Создание Date',
      fn: () => new Date()
    },
    {
      name: 'Date.now()',
      fn: () => Date.now()
    },
  ],

```

```

{
  name: 'getTime()',
  fn: () => new Date().getTime()
},
{
  name: 'toISOString()',
  fn: () => new Date().toISOString()
},
{
  name: 'toLocaleString()',
  fn: () => new Date().toLocaleString('ru-RU')
},
{
  name: 'Intl.DateTimeFormat',
  fn: () => {
    const formatter = new Intl.DateTimeFormat('ru-RU');
    return formatter.format(new Date());
  }
},
{
  name: 'Математические операции',
  fn: () => {
    const date = new Date();
    return date.getFullYear() * 10000 +
      (date.getMonth() + 1) * 100 +
      date.getDate();
  }
}
];

const results = [];
const iterations = 10000;

operations.forEach(op => {
  console.time(op.name);
  for (let i = 0; i < iterations; i++) {
    op.fn();
  }
  console.timeEnd(op.name);

  // Измерение памяти (приблизительно)
  const startMem = process.memoryUsage?.().heapUsed || 0;
  for (let i = 0; i < iterations; i++) {
    op.fn();
  }
});

```

```

    }
    const endMem = process.memoryUsage?.().heapUsed || 0;

    results.push({
      operation: op.name,
      iterations,
      memoryIncrease: endMem - startMem
    });
  });

  return results;
}

// Запуск бенчмарков (если в Node.js)
if (typeof process !== 'undefined') {
  console.log('Результаты бенчмаркинга:');
  console.table(benchmarkDateOperations());
}

```

Часть 7: Распространенные ошибки и их решение

7.1. Типичные ошибки при работе с Date

Ошибка 1: Неправильное использование месяца

```

javascript

// ✗ НЕПРАВИЛЬНО
const wrongDate = new Date(2026, 2, 10); // Думаем, что 10 февраля
console.log(wrongDate.toString()); // 10 марта 2026!

// ✅ ПРАВИЛЬНО
const correctDate = new Date(2026, 1, 10); // Февраль - это 1 (0-based)
console.log(correctDate.toString()); // 10 февраля 2026

// Решение: утилита для удобства
function createDate(year, month, day) {
  // month: 1-12 (январь-декабрь)
  return new Date(year, month - 1, day);
}

console.log(createDate(2026, 2, 10).toString()); // 10 февраля 2026

```

Ошибка 2: Парсинг строк без указания часового пояса

```
javascript

// ✗ Проблематично (зависит от среды выполнения)
const ambiguousDate = new Date('2026-02-10');
console.log(ambiguousDate.toString()); // Разный результат в разных часовых поясах

// ✔ Явное указание времени и часового пояса
const explicitDate = new Date('2026-02-10T00:00:00.000Z'); // UTC
console.log(explicitDate.toISOString()); // Всегда "2026-02-10T00:00:00.000Z"

// Решение: нормализация дат
function normalizeDate(dateString) {
  // Всегда парсим как UTC и нормализуем
  const date = new Date(dateString);
  return new Date(Date.UTC(
    date.getUTCFullYear(),
    date.getUTCMonth(),
    date.getUTCDate()
  ));
}

console.log(normalizeDate('2026-02-10').toISOString()); // Всегда UTC полночь
```

Ошибка 3: Мутация объектов Date

```
javascript

// ✗ Неожиданная мутация
const originalDate = new Date('2026-02-10');
const modifiedDate = originalDate;
modifiedDate.setDate(15);
console.log(originalDate.getDate()); // 15! Оригинал изменился

// ✔ Создание копий
const safeOriginal = new Date('2026-02-10');
const safeCopy = new Date(safeOriginal.getTime()); // Копия через timestamp
safeCopy.setDate(15);
console.log(safeOriginal.getDate()); // 10 - оригинал не изменился

// Решение: утилита для безопасной модификации
function modifyDate(date, modifications) {
  const newDate = new Date(date.getTime());
```

```

if (modifications.year !== undefined) newDate.setFullYear(modifications.year);
if (modifications.month !== undefined) newDate.setMonth(modifications.month);
if (modifications.date !== undefined) newDate.setDate(modifications.date);
if (modifications.hours !== undefined) newDate.setHours(modifications.hours);
if (modifications.minutes !== undefined) newDate.setMinutes(modifications.minutes);
if (modifications.seconds !== undefined) newDate.setSeconds(modifications.seconds);

return newDate;
}

const baseDate = new Date('2026-02-10');
const updated = modifyDate(baseDate, { date: 15, hours: 14 });
console.log(baseDate.toString()); // Не изменился
console.log(updated.toString()); // Измененная копия

```

7.2. Решение реальных проблем

Проблема: Расчет возраста

```

javascript

// Точный расчет возраста
function calculateAge(birthDate, onDate = new Date()) {
  // Копируем даты, чтобы не мутировать оригиналы
  const bd = new Date(birthDate);
  const od = new Date(onDate);

  // Нормализуем время
  bd.setHours(0, 0, 0, 0);
  od.setHours(0, 0, 0, 0);

  let age = od.getFullYear() - bd.getFullYear();
  const monthDiff = od.getMonth() - bd.getMonth();

  // Если день рождения еще не наступил в этом году, вычитаем год
  if (monthDiff < 0 || (monthDiff === 0 && od.getDate() < bd.getDate())) {
    age--;
  }

  // Расчет полного возраста (лет, месяцев, дней)
  let months = od.getMonth() - bd.getMonth();
  let days = od.getDate() - bd.getDate();

```



```

if (days < 0) {
  // Занимаем месяц
  const lastMonth = new Date(od.getFullYear(), od.getMonth(), 0);
  days += lastMonth.getDate();
  months--;
}

if (months < 0) {
  months += 12;
}

return {
  years: age,
  months: months,
  days: days,
  totalDays: Math.floor((od - bd) / (1000 * 60 * 60 * 24)),
  nextBirthDay: getNextBirthDay(birthDate, onDate)
};
}

// Получение даты следующего дня рождения
function getNextBirthDay(birthDate, fromDate = new Date()) {
  const bd = new Date(birthDate);
  const fd = new Date(fromDate);

  const currentYear = fd.getFullYear();
  let nextBirthDay = new Date(currentYear, bd.getMonth(), bd.getDate());

  // Если день рождения уже прошел в этом году, берем следующий год
  if (nextBirthDay < fd) {
    nextBirthDay.setFullYear(currentYear + 1);
  }

  return {
    date: nextBirthDay,
    daysUntil: Math.ceil((nextBirthDay - fd) / (1000 * 60 * 60 * 24))
  };
}

// Пример
const birthDate = new Date('2000-02-10');
const today = new Date('2026-02-10');
const age = calculateAge(birthDate, today);

```

```
console.log(`Возраст на ${today.toDateString()}:`);
console.log(`Полных лет: ${age.years}`);
console.log(`И еще: ${age.months} месяцев ${age.days} дней`);
console.log(`До следующего дня рождения: ${age.nextBirthday.daysUntil} дней`);
```

Заключение

Ключевые моменты работы с Date в JavaScript

1. **Всегда работайте с UTC внутри приложения** — конвертируйте в локальное время только при выводе.
2. **Используйте ISO 8601 формат** для хранения и передачи дат.
3. **Помните про 0-based месяцы** (0 = январь, 11 = декабрь).
4. **Кэшируйте форматтеры Intl.DateTimeFormat** при частом использовании.
5. **Создавайте копии Date объектов** перед модификацией.

Современные альтернативы

```
javascript

// Будущее: Temporal API (Stage 3 на 2026 год)
// temporal.now.instant() - текущее время
// temporal.PlainDate - только дата
// temporal.PlainTime - только время
// temporal.ZonedDateTime - дата и время с часовым поясом

// Библиотеки для сложных операций:
// - date-fns (рекомендуется): модульная, tree-shakeable
// - Day.js: легковесная альтернатива Moment.js
// - Luxon: от создателей Moment.js, современный API

// Пример с date-fns (установить: npm install date-fns)
import { format, addDays, differenceInDays, isSameDay } from 'date-fns';
import { ru } from 'date-fns/locale';

const today = new Date();
console.log(format(today, 'dd.MM.yyyy', { locale: ru })); // "10.02.2026"
console.log(format(addDays(today, 7), 'eeee, d MMMM yyyy', { locale: ru }));
// "вторник, 17 февраля 2026"
```

Рекомендации по выбору подхода

1. **Простые операции** — используйте нативный Date
2. **Локализация и форматирование** — Intl.DateTimeFormat
3. **Сложные манипуляции с датами** — date-fns или Day.js
4. **Высокая производительность** — нативный Date с кэшированием
5. **Часовые пояса и DST** — используйте Intl или библиотеки

Объект Date в JavaScript, несмотря на свои недостатки, остается мощным инструментом для работы со временем. Понимание его особенностей и правильное применение паттернов позволяет эффективно решать большинство задач, связанных с датами и временем в веб-приложениях.

13.2. Библиотеки для работы с датами (Moment.js, date-fns)

Введение: Почему нужны библиотеки для работы с датами?

Несмотря на наличие нативного объекта `Date` в JavaScript, разработчики часто сталкиваются с его ограничениями:

Проблемы нативного `Date`:

```
javascript

// 1. Мутабельность
const date = new Date('2026-02-10');
date.setDate(15); // Мутирует оригинальный объект

// 2. 0-based месяцы
new Date(2026, 1, 10); // Это февраль (1 вместо 2)

// 3. Плохая поддержка часовых поясов
const dateStr = '2026-02-10';
new Date(dateStr); // Разное поведение в разных браузерах

// 4. Сложные манипуляции
// Добавить 1 месяц с учетом разного количества дней
function addMonth(date) {
  const result = new Date(date);
  result.setMonth(result.getMonth() + 1);
  // Проблема: 31 января + 1 месяц = 3 марта
  return result;
}

// 5. Форматирование требует много кода
function formatDate(date) {
  const day = String(date.getDate()).padStart(2, '0');
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const year = date.getFullYear();
  return `${day}.${month}.${year}`;
}
```

Эволюция библиотек работы с датами:

text

2011: Moment.js (революция)

↓

2015: date-fns (модульная альтернатива)

↓

2017: Luxon (от создателей Moment)

↓

2018: Day.js (легковесный клон)

↓

2021: Temporal API (стандарт будущего, Stage 3)

Часть 1: Moment.js — классика жанра

1.1. История и философия

Создание и развитие

javascript

```
// 2011: Первый релиз Moment.js
// Создатель: Tim Wood
// Цель: Решить все проблемы нативного Date
```

```
// Почему стал стандартом:
// 1. Цепочки методов (fluent API)
// 2. Неизменяемость (immutability)
// 3. Богатая функциональность
// 4. Отличная документация
// 5. Поддержка локали
```

```
// В 2020: Проект перешел в Legacy mode
// Причина: Размер бандла (329KB minified)
// Альтернативы: date-fns, Luxon, Day.js
```

1.2. Установка и базовое использование

Установка

```
bash

# Установка
npm install moment

# Установка с локалями
npm install moment moment-timezone

# Или через CDN
<script src="https://cdnjs.cloudflare.com/ajax/libs/moment.js/2.29.4/moment.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/moment-timezone/0.5.40/moment-timezone-with-data.min.js"></script>
```

Создание и парсинг

```
javascript

import moment from 'moment';
import 'moment/locale/ru'; // Для русской локали

// Создание текущей даты
const now = moment();
console.log(now.toString()); // "Tue Feb 10 2026 14:30:45 GMT+0300"

// Парсинг строк
const date1 = moment('2026-02-10'); // ISO формат
const date2 = moment('10.02.2026', 'DD.MM.YYYY'); // С указанием формата
const date3 = moment('February 10, 2026', 'MMMM DD, YYYY'); // Английский формат

// Парсинг с часовыми поясами
const date4 = moment('2026-02-10T14:30:00+03:00'); // С часовым поясом
const date5 = moment.utc('2026-02-10T14:30:00'); // UTC время
const date6 = moment.tz('2026-02-10 14:30', 'America/New_York'); // Конкретный часовой пояс

// Из нативного Date
const jsDate = new Date('2026-02-10');
const momentDate = moment(jsDate);

// Из timestamp
```

```
const timestamp = 1739197845123;
const fromTimestamp = moment(timestamp);

// Валидация
console.log(moment('invalid').isValid()); // false
console.log(moment('2026-02-10').isValid()); // true
```

1.3. Манипуляции с датами

Добавление и вычитание

```
javascript

const date = moment('2026-02-10');

// Добавление (возвращает новый объект)
const tomorrow = date.add(1, 'day');
const nextWeek = date.add(1, 'week');
const nextMonth = date.add(1, 'month');
const nextYear = date.add(1, 'year');

// Вычитание
const yesterday = date.subtract(1, 'day');
const lastWeek = date.subtract(1, 'week');
const lastMonth = date.subtract(1, 'month');
const lastYear = date.subtract(1, 'year');

// Комбинированные операции
const complexDate = date
  .add(3, 'months')
  .subtract(2, 'weeks')
  .add(5, 'days');

// Единицы измерения
const units = [
  'millisecond', 'second', 'minute', 'hour',
  'day', 'week', 'month', 'quarter', 'year'
];

// Пример с разными единицами
const result = moment('2026-02-10 14:30:00')
  .add(90, 'minutes')    // +1.5 часа
  .add(45, 'seconds')    // +45 секунд
```

```
.add(500, 'milliseconds');
```

```
console.log(result.format('HH:mm:ss.SSS')); // "16:15:45.500"
```

Начало и конец периодов

javascript

```
const date = moment('2026-02-10T14:30:45');
```

// Начало периодов

```
console.log(date.startOf('day').format()); // 2026-02-10T00:00:00+03:00
```

```
console.log(date.startOf('month').format()); // 2026-02-01T00:00:00+03:00
```

```
console.log(date.startOf('year').format()); // 2026-01-01T00:00:00+03:00
```

```
console.log(date.startOf('hour').format()); // 2026-02-10T14:00:00+03:00
```

```
console.log(date.startOf('week').format()); // 2026-02-09T00:00:00+03:00 (понедельник)
```

// Конец периодов

```
console.log(date.endOf('day').format()); // 2026-02-10T23:59:59.999+03:00
```

```
console.log(date.endOf('month').format()); // 2026-02-28T23:59:59.999+03:00
```

```
console.log(date.endOf('year').format()); // 2026-12-31T23:59:59.999+03:00
```

```
console.log(date.endOf('hour').format()); // 2026-02-10T14:59:59.999+03:00
```

// Полезные сценарии

// Получить первый день текущего месяца

```
const firstDay = moment().startOf('month');
```

// Получить последний день предыдущего месяца

```
const lastDayPrevMonth = moment()
```

```
.subtract(1, 'month')
```

```
.endOf('month');
```

// Получить начало текущей недели (понедельник)

```
const weekStart = moment().startOf('isoWeek'); // или 'week'
```

1.4. Форматирование и локализация

Базовое форматирование

javascript

```
const date = moment('2026-02-10T14:30:45');
```

// Форматирование токенами


```

console.log(date.format('DD.MM.YYYY'));           // 10.02.2026
console.log(date.format('DD MMMM YYYY'));         // 10 февраля 2026
console.log(date.format('dddd, D MMMM YYYY'));    // вторник, 10 февраля 2026
console.log(date.format('HH:mm:ss'));             // 14:30:45
console.log(date.format('hh:mm A'));              // 02:30 PM
console.log(date.format('YYYY-MM-DDTHH:mm:ssZ')); // 2026-02-10T14:30:45+03:00

```

// Предопределенные форматы

```

console.log(date.format());           // 2026-02-10T14:30:45+03:00
console.log(date.format('LL'));       // 10 февраля 2026 г.
console.log(date.format('LLL'));      // 10 февраля 2026 г., 14:30
console.log(date.format('LLLL'));     // вторник, 10 февраля 2026 г., 14:30
console.log(date.format('LT'));       // 14:30
console.log(date.format('LTS'));      // 14:30:45

```

// Относительное время

```

console.log(moment().subtract(1, 'day').fromNow()); // день назад
console.log(moment().add(2, 'hours').fromNow());    // через 2 часа
console.log(moment('2026-02-10').fromNow());       // через 4 года

```

// Разница между датами

```

const date1 = moment('2026-02-10');
const date2 = moment('2026-02-15');
console.log(date1.from(date2));           // 5 дней назад
console.log(date2.to(date1));             // через 5 дней
console.log(date1.to(date2, true));       // 5 дней (без "через"/"назад")

```

Локализация

javascript

// Установка локали

```
moment.locale('ru'); // Русский
```

// Проверка текущей локали

```
console.log(moment.locale()); // 'ru'
```

// Форматирование с локалью

```

const date = moment('2026-02-10T14:30:45');
console.log(date.format('LLLL')); // "вторник, 10 февраля 2026 г., 14:30"

```

// Доступные локали

```

const availableLocales = moment.locales();
console.log(availableLocales.includes('ru')); // true
console.log(availableLocales.includes('en')); // true

```

```
// Кастомизация локали
moment.updateLocale('ru', {
  months: [
    'Январь', 'Февраль', 'Март', 'Апрель', 'Май', 'Июнь',
    'Июль', 'Август', 'Сентябрь', 'Октябрь', 'Ноябрь', 'Декабрь'
  ],
  weekdays: [
    'Воскресенье', 'Понедельник', 'Вторник', 'Среда',
    'Четверг', 'Пятница', 'Суббота'
  ]
});

// Временная установка локали для конкретной даты
console.log(date.locale('en').format('LLLL')); // "Tuesday, February 10, 2026 2:30 PM"
console.log(date.locale('ru').format('LLLL')); // Возвращаем русскую локаль
```

1.5. Часовые пояса и DST

Работа с часовыми поясами

```
javascript

import moment from 'moment-timezone';

// Установка часового пояса
const date = moment.tz('2026-02-10 14:30', 'Europe/Moscow');
console.log(date.format()); // 2026-02-10T14:30:00+03:00

// Конвертация между поясами
const moscowTime = moment.tz('2026-02-10 14:30', 'Europe/Moscow');
const newYorkTime = moscowTime.clone().tz('America/New_York');
console.log(newYorkTime.format()); // 2026-02-10T06:30:00-05:00

// Получение текущего времени в разных поясах
const timezones = ['UTC', 'Europe/Moscow', 'America/New_York', 'Asia/Tokyo'];
timezones.forEach(tz => {
  console.log(`${tz}: ${moment().tz(tz).format('HH:mm')}`);
});

// Определение смещения
const dateInMoscow = moment.tz('Europe/Moscow');
console.log(dateInMoscow.utcOffset()); // -180 (минут, Москва UTC+3)
```

```

console.log(dateInMoscow.format('Z')); // +03:00
console.log(dateInMoscow.format('ZZ')); // +0300

// Автоматическое определение часового пояса пользователя
const userTimezone = moment.tz.guess();
console.log(`Часовой пояс пользователя: ${userTimezone}`);

```

Летнее время (DST)

```

javascript

// Проверка, действует ли летнее время
const winterDate = moment.tz('2026-01-10 14:30', 'Europe/Moscow');
const summerDate = moment.tz('2026-06-10 14:30', 'Europe/Moscow');

console.log(`Зимой: ${winterDate.format('Z')}`); // +03:00
console.log(`Летом: ${summerDate.format('Z')}`); // +03:00 (в России отменили DST)

// Для стран с DST
const nyWinter = moment.tz('2026-01-10 14:30', 'America/New_York');
const nySummer = moment.tz('2026-06-10 14:30', 'America/New_York');

console.log(`NY зимой: ${nyWinter.format('Z')}`); // -05:00
console.log(`NY летом: ${nySummer.format('Z')}`); // -04:00

// Определение дат перехода на летнее время
function getDSTTransitions(year, timezone) {
  const transitions = [];
  let current = moment.tz(`${year}-01-01`, timezone);
  const end = moment.tz(`${year}-12-31`, timezone);

  let lastOffset = current.utcOffset();

  while (current.isBefore(end)) {
    current = current.add(1, 'day');
    const currentOffset = current.utcOffset();

    if (currentOffset !== lastOffset) {
      transitions.push({
        date: current.format('YYYY-MM-DD'),
        fromOffset: lastOffset,
        toOffset: currentOffset,
        change: currentOffset > lastOffset ? 'forward' : 'backward'
      });
      lastOffset = currentOffset;
    }
  }
}

```

```

    }
  }

  return transitions;
}

console.log('Переходы DST в Нью-Йорке 2026:', getDSTTransitions(2026,
  'America/New_York'));

```

1.6. Продвинутые возможности

Диапазоны и интервалы

```

javascript

// Создание диапазонов
const start = moment('2026-02-10');
const end = moment('2026-02-20');

// Проверка вхождения даты в диапазон
const checkDate = moment('2026-02-15');
console.log(checkDate.isBetween(start, end)); // true
console.log(checkDate.isBetween(start, end, null, '[')); // inclusive
console.log(checkDate.isBetween(start, end, null, '(')); // exclusive start

// Проверка до/после
console.log(checkDate.isAfter(start)); // true
console.log(checkDate.isBefore(end)); // true

// Минимальная/максимальная дата из списка
const dates = [
  moment('2026-02-10'),
  moment('2026-02-15'),
  moment('2026-02-05')
];

const earliest = moment.min(dates);
const latest = moment.max(dates);
console.log(`Самая ранняя: ${earliest.format('DD.MM.YYYY')}`);
console.log(`Самая поздняя: ${latest.format('DD.MM.YYYY')}`);

```

Календарное время

```

javascript

```

```

const date = moment('2026-02-10T14:30:45');

// Календарное форматирование
console.log(date.calendar());
// "Сегодня в 14:30" (если сегодня 10.02.2026)
// "Завтра в 14:30" (если сегодня 09.02.2026)
// "Во вторник в 14:30" (если сегодня 08.02.2026)

// Кастомизация календаря
moment.updateLocale('ru', {
  calendar: {
    lastDay: '[Вчера в] LT',
    sameDay: '[Сегодня в] LT',
    nextDay: '[Завтра в] LT',
    lastWeek: '[В прошлый] dddd [в] LT',
    nextWeek: 'dddd [в] LT',
    sameElse: 'L'
  }
});

// Работа с человеческим временем
console.log(date.fromNow()); // "через 4 года"
console.log(date.toNow()); // "4 года назад"

// Точная разница
const diff = moment.duration(date.diff(moment()));
console.log(`Осталось: ${diff.humanize()}`); // "4 года"
console.log(`Осталось точно: ${diff.humanize(true)}`); // "через 4 года"

```

Пользовательские форматы

javascript

```

// Создание собственных форматов
moment.defineLocale('ru-custom', {
  parentLocale: 'ru',
  longDateFormat: {
    LT: 'HH:mm',
    LTS: 'HH:mm:ss',
    L: 'DD.MM.YYYY',
    LL: 'D MMMM YYYY г.',
    LLL: 'D MMMM YYYY г., HH:mm',
    LLLL: 'dddd, D MMMM YYYY г., HH:mm'
  }
});

```

```

// Использование пользовательского формата
const date = moment('2026-02-10T14:30:45');
console.log(date.locale('ru-custom').format('LLLL'));
// "вторник, 10 февраля 2026 г., 14:30"

// Расширение moment
moment.fn.customFormat = function() {
  return this.format('DD.MM.YYYY HH:mm:ss');
};

console.log(date.customFormat()); // "10.02.2026 14:30:45"

// Плагины для moment
// moment-range: работа с диапазонами дат
// moment-business: рабочие дни
// moment-recur: повторяющиеся события

```

1.7. Оптимизация и лучшие практики

Производительность

```

javascript

// Кэширование часто используемых объектов
const formatters = {
  iso: moment.ISO_8601,
  russian: 'DD.MM.YYYY HH:mm:ss',
  shortDate: 'DD.MM.YYYY'
};

// Оптимизация парсинга
function parseDateOptimized(dateString) {
  // Кэш парсеров
  if (!parseDateOptimized.cache) {
    parseDateOptimized.cache = new Map();
  }

  if (parseDateOptimized.cache.has(dateString)) {
    return parseDateOptimized.cache.get(dateString).clone();
  }

  const parsed = moment(dateString, moment.ISO_8601, true);

```

```
parseDateOptimized.cache.set(dateString, parsed);

return parsed.clone();
}
```

```
// Бенчмарк производительности
function benchmarkMomentOperations() {
  const iterations = 10000;
  const dateStr = '2026-02-10T14:30:45';

  console.time('moment() без кэша');
  for (let i = 0; i < iterations; i++) {
    moment(dateStr);
  }
  console.timeEnd('moment() без кэша');

  console.time('moment() с кэшем');
  for (let i = 0; i < iterations; i++) {
    parseDateOptimized(dateStr);
  }
  console.timeEnd('moment() с кэшем');
}
```

```
// Рекомендации:
// 1. Используйте .clone() для избежания мутаций
// 2. Кэшируйте форматтеры и парсеры
// 3. Используйте tree shaking (но в moment это сложно)
// 4. Для простых операций рассмотрите date-fns
```

Миграция с Moment.js

javascript

```
// Почему мигрировать:
// 1. Большой размер бандла (329KB minified)
// 2. Проект в Legacy mode
// 3. Мутабельность по умолчанию (хотя можно использовать .clone())
// 4. Медленная работа в некоторых сценариях

// Куда мигрировать:
// 1. date-fns (рекомендуется) - 78KB, модульный
// 2. Luxon - 68KB, от создателей Moment
// 3. Day.js - 7KB, совместимый API

// Помощник миграции
```

```

function migrateMomentToDateFns(momentCode) {
  // Пример преобразования
  const migrations = {
    // Создание
    "moment()": "new Date()",
    "moment('2026-02-10')": "new Date('2026-02-10')",

    // Форматирование
    ".format('DD.MM.YYYY')": "format(new Date(), 'dd.MM.yyyy')",
    ".format('HH:mm')": "format(new Date(), 'HH:mm')",

    // Манипуляции
    ".add(1, 'day')": "addDays(new Date(), 1)",
    ".subtract(1, 'month')": "subMonths(new Date(), 1)",

    // Сравнения
    ".isAfter(other)": "isAfter(new Date(), other)",
    ".isBefore(other)": "isBefore(new Date(), other)",
    ".isSame(other, 'day')": "isSameDay(new Date(), other)"
  };

  let migrated = momentCode;
  for (const [from, to] of Object.entries(migrations)) {
    migrated = migrated.replace(new RegExp(from, 'g'), to);
  }

  return migrated;
}

console.log(migrateMomentToDateFns("moment().add(1, 'day').format('DD.MM.YYYY')"));
// "format(addDays(new Date(), 1), 'dd.MM.yyyy')"

```

Часть 2: date-fns — современная модульная альтернатива

2.1. Философия и архитектура

Ключевые особенности

javascript


```

// date-fns vs Moment.js:
// 1. Модульность - импортируйте только нужные функции
// 2. Функциональный подход - чистые функции
// 3. Неизменяемость - всегда возвращает новые объекты
// 4. Tree-shaking поддержка
// 5. Совместимость с нативным Date
// 6. TypeScript поддержка из коробки

// Размеры (minified gzip):
// moment: 329KB
// moment-timezone: +53KB
// date-fns: 78KB (вся библиотека)
// date-fns (только используемые функции): ~2-10KB

// Производительность:
// Операция      | Moment | date-fns | Выигрыш
// format        | 1x     | 1.2x     | +20%
// parse         | 1x     | 2.5x     | +150%
// add           | 1x     | 3x       | +200%

```

2.2. Установка и настройка

Установка

```

bash

# Базовая установка
npm install date-fns

# Или установка конкретных функций (рекомендуется)
npm install date-fns@next # v3

# TypeScript поддержка встроена
npm install --save-dev @types/date-fns # для v2

```

Импорт функций

```

javascript

// v2 стиль (CommonJS)
const { format, addDays } = require('date-fns');

// v3 стиль (ES Modules) - РЕКОМЕНДУЕТСЯ
import { format, addDays } from 'date-fns';

```

```
// Импорт с подпутем (меньше размер бандла)
import format from 'date-fns/format';
import addDays from 'date-fns/addDays';
import ru from 'date-fns/locale/ru';

// Tree shaking работает отлично
// Webpack соберет только импортированные функции
```

2.3. Создание и парсинг дат

Работа с нативными Date

```
javascript

import {
  parseISO,
  parse,
  isValid,
  fromUnixTime
} from 'date-fns';

// Создание из строки ISO формата (рекомендуется)
const date1 = parseISO('2026-02-10');
const date2 = parseISO('2026-02-10T14:30:45Z');
const date3 = parseISO('2026-02-10T14:30:45+03:00');

// Парсинг с указанием формата
const date4 = parse('10.02.2026', 'dd.MM.yyyy', new Date());
const date5 = parse('February 10, 2026', 'MMMM dd, yyyy', new Date());

// Валидация
console.log(isValid(date1)); // true
console.log(isValid(new Date('invalid'))); // false

// Из Unix timestamp (секунды)
const timestamp = 1739197845;
const date6 = fromUnixTime(timestamp);

// Создание конкретной даты
import { set } from 'date-fns';
const baseDate = new Date(2026, 0, 1); // 1 января 2026
```

```
const specificDate = set(baseDate, {
  year: 2026,
  month: 1, // февраль (1-based!)
  date: 10,
  hours: 14,
  minutes: 30,
  seconds: 45
});

console.log(specificDate); // 10 февраля 2026, 14:30:45
```

Утилиты для создания дат

```
javascript

import {
  toDate,
  constructFrom,
  roundToNearestMinutes
} from 'date-fns';

// Безопасное создание из разных источников
const date1 = toDate('2026-02-10'); // строка → Date
const date2 = toDate(1739197845123); // timestamp → Date
const date3 = toDate(new Date()); // Date → Date (копия)

// Создание с определенным прототипом (для наследников Date)
class CustomDate extends Date {
  customMethod() {
    return 'custom';
  }
}

const customDate = constructFrom(CustomDate, 2026, 1, 10);

// Округление времени
const exactTime = new Date('2026-02-10T14:37:22');
const rounded = roundToNearestMinutes(exactTime, { nearestTo: 15 });
console.log(rounded); // 14:30 или 14:45
```

2.4. Форматирование и локализация

Базовое форматирование

```
javascript

import { format } from 'date-fns';
import { ru, enUS } from 'date-fns/locale';

const date = new Date('2026-02-10T14:30:45');

// Форматирование с токенами
console.log(format(date, 'dd.MM.yyyy'));           // 10.02.2026
console.log(format(date, 'dd MMMM yyyy'));         // 10 февраля 2026
console.log(format(date, 'EEEE, d MMMM yyyy'));    // вторник, 10 февраля 2026
console.log(format(date, 'HH:mm:ss'));             // 14:30:45
console.log(format(date, 'hh:mm a'));             // 02:30 PM
console.log(format(date, 'yyyy-MM-dd\T\HH:mm:ssXXX')); // 2026-02-10T14:30:45+03:00

// Локализация
console.log(format(date, 'PPPP', { locale: ru }));
// вторник, 10 февраля 2026 г.

console.log(format(date, 'PPPP', { locale: enUS }));
// Tuesday, February 10, 2026

// Предопределенные форматы
console.log(format(date, 'P'));                    // 10.02.2026
console.log(format(date, 'PP'));                   // 10 февр. 2026 г.
console.log(format(date, 'PPP'));                   // 10 февраля 2026 г.
console.log(format(date, 'PPPP'));                  // вторник, 10 февраля 2026 г.
console.log(format(date, 'p'));                    // 14:30
console.log(format(date, 'pp'));                   // 14:30:45
console.log(format(date, 'ppp'));                   // 14:30:45 GMT+3

// Относительное форматирование
import { formatRelative, formatDistance, formatDistanceToNow } from 'date-fns';

const baseDate = new Date('2026-02-10');
const targetDate = new Date('2026-02-15');

console.log(formatRelative(targetDate, baseDate, { locale: ru }));
// 15 февраля 2026 г. в 0:00
```

```
console.log(formatDistance(targetDate, baseDate, { locale: ru }));  
// 5 дней  
  
console.log(formatDistanceToNow(new Date('2026-02-10'), { locale: ru }));  
// через 4 года
```

Кастомизация локалей

```
javascript  
  
import { format } from 'date-fns';  
import { ru } from 'date-fns/locale';  
  
// Расширение существующей локали  
const customRuLocale = {  
  ...ru,  
  localize: {  
    ...ru.localize,  
    month: (n) => {  
      const months = [  
        'Январь', 'Февраль', 'Март', 'Апрель', 'Май', 'Июнь',  
        'Июль', 'Август', 'Сентябрь', 'Октябрь', 'Ноябрь', 'Декабрь'  
      ];  
      return months[n];  
    }  
  },  
  formatLong: {  
    ...ru.formatLong,  
    date: () => 'dd MMMM yyyy "года"'  
  }  
};  
  
const date = new Date('2026-02-10');  
console.log(format(date, 'dd MMMM yyyy', { locale: customRuLocale }));  
// 10 Февраль 2026 года
```

2.5. Манипуляции с датами

Добавление и вычитание

```
javascript  
  
import {  
  addDays,
```

```
subDays,  
addWeeks,  
subWeeks,  
addMonths,  
subMonths,  
addYears,  
subYears,  
addHours,  
subHours,  
addMinutes,  
subMinutes,  
addSeconds,  
subSeconds,  
addMilliseconds,  
subMilliseconds  
} from 'date-fns';  
  
const date = new Date('2026-02-10T14:30:45');  
  
// Добавление  
const tomorrow = addDays(date, 1);  
const nextWeek = addWeeks(date, 1);  
const nextMonth = addMonths(date, 1);  
const nextYear = addYears(date, 1);  
  
// Вычитание  
const yesterday = subDays(date, 1);  
const lastWeek = subWeeks(date, 1);  
const lastMonth = subMonths(date, 1);  
const lastYear = subYears(date, 1);  
  
// Комбинированные операции  
import { add } from 'date-fns';  
  
const complexDate = add(date, {  
  months: 3,  
  weeks: 2,  
  days: 5,  
  hours: 6,  
  minutes: 30  
});  
  
// Точное добавление с учетом DST  
import { addBusinessDays } from 'date-fns';
```

```
const friday = new Date('2026-02-06'); // пятница
const nextBusinessDay = addBusinessDays(friday, 1); // понедельник 9 февраля
```

Начало и конец периодов

```
javascript

import {
  startOfDay,
  endOfDay,
  startOfWeek,
  endOfWeek,
  startOfMonth,
  endOfMonth,
  startOfYear,
  endOfYear,
  startOfHour,
  endOfHour,
  startOfMinute,
  endOfMinute,
  startOfSecond,
  endOfSecond
} from 'date-fns';
import { ru } from 'date-fns/locale';

const date = new Date('2026-02-10T14:30:45.123');

// Начало периодов
console.log(startOfDay(date)); // 2026-02-10T00:00:00
console.log(startOfWeek(date, { locale: ru })); // 2026-02-09T00:00:00 (понедельник)
console.log(startOfMonth(date)); // 2026-02-01T00:00:00
console.log(startOfYear(date)); // 2026-01-01T00:00:00
console.log(startOfHour(date)); // 2026-02-10T14:00:00

// Конец периодов
console.log(endOfDay(date)); // 2026-02-10T23:59:59.999
console.log(endOfWeek(date, { locale: ru })); // 2026-02-15T23:59:59.999
console.log(endOfMonth(date)); // 2026-02-28T23:59:59.999
console.log(endOfYear(date)); // 2026-12-31T23:59:59.999
console.log(endOfHour(date)); // 2026-02-10T14:59:59.999

// Полезные комбинации
// Первый день текущего месяца
const firstDay = startOfMonth(new Date());
```

```
// Последний день предыдущего месяца
const lastDayPrevMonth = endOfMonth(subMonths(new Date(), 1));

// Начало текущей недели (понедельник)
const weekStart = startOfWeek(new Date(), { locale: ru });
```

2.6. Сравнение и проверки

Сравнение дат

```
javascript

import {
  isEqual,
  isBefore,
  isAfter,
  isSameDay,
  isSameMonth,
  isSameYear,
  isSameHour,
  isSameMinute,
  isSameSecond,
  isWithinInterval,
  areIntervalsOverlapping,
  max,
  min
} from 'date-fns';

const date1 = new Date('2026-02-10T10:00:00');
const date2 = new Date('2026-02-10T14:30:00');
const date3 = new Date('2026-02-10T10:00:00');

// Базовые сравнения
console.log(isEqual(date1, date3)); // true
console.log(isBefore(date1, date2)); // true
console.log(isAfter(date2, date1)); // true

// Сравнение частей дат
console.log(isSameDay(date1, date2)); // true
console.log(isSameMonth(date1, date2)); // true
console.log(isSameYear(date1, date2)); // true
console.log(isSameHour(date1, date2)); // false
```



```

// Проверка диапазонов
const interval = {
  start: new Date('2026-02-10T09:00:00'),
  end: new Date('2026-02-10T18:00:00')
};

console.log(isWithinInterval(date1, interval)); // true
console.log(isWithinInterval(new Date('2026-02-10T20:00:00'), interval)); // false

// Проверка пересечения интервалов
const interval1 = {
  start: new Date('2026-02-10T10:00:00'),
  end: new Date('2026-02-10T12:00:00')
};

const interval2 = {
  start: new Date('2026-02-10T11:00:00'),
  end: new Date('2026-02-10T13:00:00')
};

console.log(areIntervalsOverlapping(interval1, interval2)); // true

// Минимальная и максимальная даты
const dates = [
  new Date('2026-02-05'),
  new Date('2026-02-10'),
  new Date('2026-02-15')
];

console.log(min(dates)); // 2026-02-05
console.log(max(dates)); // 2026-02-15

```

Дополнительные проверки

javascript

```

import {
  isValid,
  isDate,
  isFuture,
  isPast,
  isToday,
  isYesterday,
  isTomorrow,

```

```

isWeekend,
isLeapYear,
isSameQuarter,
isSameWeek,
isLastDayOfMonth
} from 'date-fns';

const date = new Date('2026-02-10');

// Валидация
console.log(isValid(date));           // true
console.log(isValid(new Date('invalid'))); // false
console.log(isDate(date));           // true
console.log(isDate('2026-02-10'));   // false

// Относительное время
console.log(isFuture(date));          // true (если сегодня раньше 2026)
console.log(isPast(date));            // false
console.log(isToday(date));           // false
console.log(isYesterday(date));      // false
console.log(isTomorrow(date));        // false

// Календарные проверки
console.log(isWeekend(date));          // false (вторник)
console.log(isLeapYear(date));         // false (2026 не високосный)
console.log(isSameQuarter(date, new Date('2026-03-15'))); // true (Q1)
console.log(isSameWeek(date, new Date('2026-02-12'), { locale: ru })); // true
console.log(isLastDayOfMonth(date));   // false

```

2.7. Часовые пояса и DST

Работа с часовыми поясами в date-fns

```

javascript

// Важно: date-fns v2 не имеет встроенной поддержки часовых поясов
// Но есть решения:

// Способ 1: Использование date-fns-tz
import { format, utcToZonedTime, zonedTimeToUtc } from 'date-fns-tz';

const date = new Date('2026-02-10T14:30:45Z');

```

```

// Конвертация в часовой пояс
const moscowTime = utcToZonedTime(date, 'Europe/Moscow');
console.log(format(moscowTime, 'yyyy-MM-dd HH:mm:ssXXX', { timeZone: 'Europe/Moscow' }));
// 2026-02-10 17:30:45+03:00

// Конвертация из часового пояса в UTC
const localTime = zonedTimeToUtc(moscowTime, 'Europe/Moscow');
console.log(localTime.toISOString()); // 2026-02-10T14:30:45.000Z

// Способ 2: Использование Intl API с date-fns
function formatInTimeZone(date, timeZone, formatStr) {
  const formatter = new Intl.DateTimeFormat('en-US', {
    timeZone,
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hour12: false
  });

  const parts = formatter.formatToParts(date);
  const map = {};
  parts.forEach(part => {
    if (part.type !== 'literal') {
      map[part.type] = part.value;
    }
  });

  const adjustedDate = new Date(
    `${map.year}-${map.month}-${map.day}T${map.hour}:${map.minute}:${map.second}`
  );

  return format(adjustedDate, formatStr);
}

console.log(formatInTimeZone(date, 'Europe/Moscow', 'HH:mm:ss'));
// 17:30:45

// Способ 3: Использование временных меток
function convertTimezone(date, fromOffset, toOffset) {
  // fromOffset и toOffset в минутах
  const diff = toOffset - fromOffset;

```

```

    return new Date(date.getTime() + diff * 60000);
}

const utcDate = new Date('2026-02-10T14:30:45Z');
const moscowOffset = -180; // UTC+3
const moscowDate = convertTimezone(utcDate, 0, moscowOffset);
console.log(moscowDate); // 2026-02-10T17:30:45

```

2.8. Интервалы и разницы

Расчет разниц

javascript

```

import {
  differenceInMilliseconds,
  differenceInSeconds,
  differenceInMinutes,
  differenceInHours,
  differenceInDays,
  differenceInWeeks,
  differenceInMonths,
  differenceInYears,
  differenceInCalendarDays,
  differenceInCalendarMonths,
  differenceInCalendarYears,
  differenceInBusinessDays,
  intervalToDuration,
  formatDuration
} from 'date-fns';

const start = new Date('2026-02-10T10:00:00');
const end = new Date('2026-02-15T14:30:45');

// Разница в различных единицах
console.log(differenceInDays(end, start)); // 5
console.log(differenceInHours(end, start)); // 124
console.log(differenceInMinutes(end, start)); // 7470
console.log(differenceInSeconds(end, start)); // 448245

// Календарная разница (без учета времени)
console.log(differenceInCalendarDays(end, start)); // 5
console.log(differenceInCalendarMonths(end, start)); // 0

```

```

// Рабочие дни
console.log(differenceInBusinessDays(end, start)); // 4 (если 15 февраля не выходной)

// Полная разница в виде интервала
const duration = intervalToDuration({ start, end });
console.log(duration);
// { years: 0, months: 0, days: 5, hours: 4, minutes: 30, seconds: 45 }

// Форматирование длительности
console.log(formatDuration(duration, { locale: ru }));
// 5 дней 4 часа 30 минут 45 секунд

console.log(formatDuration(duration, {
  locale: ru,
  format: ['years', 'months', 'days']
}));
// 5 дней

// Разница с точностью
import { formatDistance, formatDistanceStrict } from 'date-fns';

console.log(formatDistance(end, start, { locale: ru }));
// 5 дней

console.log(formatDistanceStrict(end, start, { locale: ru }));
// 5 дней

console.log(formatDistanceStrict(end, start, {
  locale: ru,
  unit: 'hour'
}));
// 124 часа

```

2.9. Полезные утилиты и хелперы

Утилиты для работы с датами

```

javascript

import {
  clamp,
  compareAsc,

```

```

compareDesc,
eachDayOfInterval,
eachMonthOfInterval,
eachYearOfInterval,
getDay,
getDaysInMonth,
getHours,
getISODay,
getISOWeek,
getMonth,
getQuarter,
getWeek,
getWeekOfMonth,
getYear,
set,
setHours,
setMonth,
setYear,
toDate
} from 'date-fns';

// Ограничение даты в диапазоне
const date = new Date('2026-02-10');
const minDate = new Date('2026-02-01');
const maxDate = new Date('2026-02-20');
const clamped = clamp(date, { start: minDate, end: maxDate });
console.log(clamped); // 2026-02-10 (в пределах диапазона)

// Сортировка дат
const dates = [
  new Date('2026-02-15'),
  new Date('2026-02-10'),
  new Date('2026-02-20')
];

dates.sort(compareAsc); // по возрастанию
dates.sort(compareDesc); // по убыванию

// Генерация диапазонов дат
const interval = {
  start: new Date('2026-02-01'),
  end: new Date('2026-02-10')
};

```

```

const allDays = eachDayOfInterval(interval);
console.log(allDays.length); // 10 дней

const months = eachMonthOfInterval(interval);
console.log(months.length); // 1 месяц (только февраль)

// Получение информации о дате
const dateInfo = new Date('2026-02-10');
console.log(getDay(dateInfo)); // 2 (вторник, 0-воскресенье)
console.log(getISODay(dateInfo)); // 2 (понедельник-1, воскресенье-7)
console.log(getISOWeek(dateInfo)); // 7 (неделя ISO)
console.log(getQuarter(dateInfo)); // 1 (первый квартал)
console.log(getDaysInMonth(dateInfo)); // 28
console.log(getWeekOfMonth(dateInfo, { locale: ru })); // 2

// Установка значений
const modified = set(dateInfo, {
  year: 2027,
  month: 5, // июнь (0-based)
  date: 15,
  hours: 14,
  minutes: 30
});

```

2.10. Оптимизация и лучшие практики

Tree-shaking и размер бандла

```

javascript

// ПРАВИЛЬНО: Импорт конкретных функций
import format from 'date-fns/format'; // ≈ 2KB
import addDays from 'date-fns/addDays'; // ≈ 1KB

// НЕПРАВИЛЬНО: Импорт всей библиотеки
import { format, addDays } from 'date-fns'; // ≈ 78KB

// Конфигурация Webpack для оптимизации
// webpack.config.js
module.exports = {
  // ...
  optimization: {
    usedExports: true, // Включает tree-shaking
  }
}

```

```

}
};

// Конфигурация Rollup
// rollup.config.js
import { nodeResolve } from '@rollup/plugin-node-resolve';
import commonjs from '@rollup/plugin-commonjs';

export default {
  plugins: [
    nodeResolve(),
    commonjs({
      // Автоматически преобразует CommonJS в ES6
    })
  ]
};

```

Кэширование и производительность

```

javascript

// Кэширование форматтеров
const formatCache = new Map();

function formatCached(date, formatStr, options = {}) {
  const cacheKey = `${formatStr}_${date.getTime()}_${JSON.stringify(options)}`;

  if (formatCache.has(cacheKey)) {
    return formatCache.get(cacheKey);
  }

  const formatted = format(date, formatStr, options);
  formatCache.set(cacheKey, formatted);

  // Очистка кэша при превышении лимита
  if (formatCache.size > 1000) {
    const firstKey = formatCache.keys().next().value;
    formatCache.delete(firstKey);
  }

  return formatted;
}

// Оптимизация частых операций
const dateUtils = {

```


// Мемоизация парсинга

```
parseISO: (() => {  
  const cache = new Map();  
  return (dateString) => {  
    if (cache.has(dateString)) {  
      return new Date(cache.get(dateString));  
    }  
    const date = parseISO(dateString);  
    cache.set(dateString, date.getTime());  
    return date;  
  };  
})();
```

// Быстрая проверка формата

```
isValidFormat: (dateString, formatStr) => {  
  try {  
    const parsed = parse(dateString, formatStr, new Date());  
    return isValid(parsed);  
  } catch {  
    return false;  
  }  
};
```

// Бенчмарк производительности

```
function benchmarkDateFns() {  
  const iterations = 10000;  
  const date = new Date('2026-02-10T14:30:45');  
  
  console.time('format без кэша');  
  for (let i = 0; i < iterations; i++) {  
    format(date, 'dd.MM.yyyy HH:mm:ss');  
  }  
  console.timeEnd('format без кэша');  
  
  console.time('format с кэшем');  
  for (let i = 0; i < iterations; i++) {  
    formatCached(date, 'dd.MM.yyyy HH:mm:ss');  
  }  
  console.timeEnd('format с кэшем');  
}
```

Интеграция с UI фреймворками

javascript

// React компонент с date-fns

```
import React from 'react';
import { format, addDays, isBefore } from 'date-fns';
import { ru } from 'date-fns/locale';

const DateDisplay = ({ date, formatStr = 'PPP' }) => {
  const formattedDate = format(date, formatStr, { locale: ru });

  return <time dateTime={date.toISOString()}>{formattedDate}</time>;
};
```

// Vue.js композиция

```
import { ref, computed } from 'vue';
import { format, addDays } from 'date-fns';
import { ru } from 'date-fns/locale';

export function useDate(date) {
  const currentDate = ref(date || new Date());

  const formatted = computed(() =>
    format(currentDate.value, 'PPP', { locale: ru })
  );

  const addDaysToDate = (days) => {
    currentDate.value = addDays(currentDate.value, days);
  };

  return {
    date: currentDate,
    formatted,
    addDaysToDate
  };
}
```

// Angular pipe для форматирования дат

```
import { Pipe, PipeTransform } from '@angular/core';
import { format } from 'date-fns';
import { ru } from 'date-fns/locale';

@Pipe({ name: 'dateFormat' })
```

```
export class DateFormatPipe implements PipeTransform {
  transform(value: Date | string, formatStr: string = 'PP'): string {
    const date = typeof value === 'string' ? new Date(value) : value;
    return format(date, formatStr, { locale: ru });
  }
}
```

Часть 3: Сравнение Moment.js и date-fns

3.1. Детальное сравнение

Таблица сравнения

```
javascript

const comparisonTable = {
  // Размеры (minified + gzipped)
  size: {
    moment: '329KB',
    momentWithLocales: '400KB+',
    momentWithTimezone: '453KB+',
    dateFnsFull: '78KB',
    dateFnsMinimal: '2-10KB',
    dateFnsTz: '+15KB'
  },

  // Производительность (операций в секунду)
  performance: {
    moment: {
      creation: '1,200,000 ops/sec',
      formatting: '800,000 ops/sec',
      manipulation: '900,000 ops/sec'
    },
    dateFns: {
      creation: '3,000,000 ops/sec',
      formatting: '1,000,000 ops/sec',
      manipulation: '2,700,000 ops/sec'
    }
  },

  // API подход
  apiStyle: {
```

```
moment: 'Объектно-ориентированный (цепочки методов)',
dateFns: 'Функциональный (чистые функции)'
},

// Неизменяемость
immutability: {
  moment: 'Мутабельный (нужен .clone())',
  dateFns: 'Неизменяемый (всегда новый объект)'
},

// Поддержка TypeScript
typescript: {
  moment: 'Требуется @types/moment',
  dateFns: 'Встроенная поддержка'
},

// Tree shaking
treeShaking: {
  moment: 'Ограниченная поддержка',
  dateFns: 'Полная поддержка'
},

// Часовые пояса
timezones: {
  moment: 'Встроенная (moment-timezone)',
  dateFns: 'Отдельная библиотека (date-fns-tz)'
},

// Локализация
localization: {
  moment: 'Встроенная, богатая',
  dateFns: 'Модульная, легковесная'
},

// Состояние проекта (2026)
maintenance: {
  moment: 'Legacy mode (только security fixes)',
  dateFns: 'Активная разработка'
}
};

console.table(comparisonTable);
```

3.2. Примеры решения одной задачи

Задача: Отформатировать дату и добавить 7 дней

```
javascript

// Moment.js подход
const moment = require('moment');
require('moment/locale/ru');

const dateMoment = moment('2026-02-10')
  .add(7, 'days')
  .locale('ru')
  .format('dddd, D MMMM YYYY');

console.log(dateMoment); // "вторник, 17 февраля 2026"

// date-fns подход
import { addDays, format } from 'date-fns';
import { ru } from 'date-fns/locale';

const date = new Date('2026-02-10');
const dateFns = format(
  addDays(date, 7),
  'EEEE, d MMMM yyyy',
  { locale: ru }
);

console.log(dateFns); // "вторник, 17 февраля 2026"

// Сравнение размера бандла:
// Moment: ~329KB
// date-fns: format (2KB) + addDays (1KB) + locale (5KB) = ~8KB
```

Задача: Проверить, выходной ли день

```
javascript

// Moment.js
const isWeekendMoment = (date) => {
  const day = moment(date).day();
  return day === 0 || day === 6;
};

// date-fns
```

```
import { isWeekend } from 'date-fns';

const isWeekendDateFns = (date) => {
  return isWeekend(date);
};

// Более сложный пример: рабочие дни с учетом праздников
const holidays = ['2026-02-23', '2026-03-08', '2026-05-01'];

// Moment.js
const isWorkdayMoment = (date) => {
  const m = moment(date);
  const day = m.day();
  const dateStr = m.format('YYYY-MM-DD');

  return day !== 0 && day !== 6 && !holidays.includes(dateStr);
};

// date-fns
import { isWeekend, format } from 'date-fns';

const isWorkdayDateFns = (date) => {
  return !isWeekend(date) && !holidays.includes(format(date, 'yyyy-MM-dd'));
};
```

3.3. Рекомендации по выбору

Когда использовать Moment.js:

```
javascript

const useMomentWhen = [
  // 1. У вас уже есть большой проект на Moment.js
  'existing-moment-project',

  // 2. Нужна максимальная совместимость со старым кодом
  'legacy-code-compatibility',

  // 3. Требуется очень богатая локализация из коробки
  'rich-localization-needed',

  // 4. Размер бандла не критичен
  'bundle-size-not-critical',
```

```
// 5. Нужны плагины (moment-range, moment-recur и т.д.)
'plugins-required'
];
```

Когда использовать date-fns:

```
javascript

const useDateFnsWhen = [
  // 1. Новый проект
  'new-project',

  // 2. Критичен размер бандла
  'bundle-size-critical',

  // 3. Используете tree-shaking
  'using-tree-shaking',

  // 4. Предпочитаете функциональный стиль
  'functional-style-preferred',

  // 5. Работаете с React/Vue/Angular и нужны оптимизации
  'framework-optimizations',

  // 6. Используете TypeScript
  'typescript-project',

  // 7. Нужна максимальная производительность
  'performance-critical'
];
```

Часть 4: Миграция и интеграция

4.1. Миграция с Moment.js на date-fns

Автоматическая миграция

```
javascript

// Утилита для миграции
function migrateMomentToDateFns(code) {
  const patterns = [
```

```

// Создание
[/moment\((['"])(.*?)\1\)/g, 'new Date($1$2$1)'],
[/moment\((.*?)\)/g, 'new Date($1)'],

// Форматирование
[/\.\format\((['"])(.*?)\1\)/g, (match, quote, format) => {
  // Конвертация токенов формата
  const convertedFormat = format
    .replace(/YYYY/g, 'yyyy')
    .replace(/YY/g, 'yy')
    .replace(/DD/g, 'dd')
    .replace(/D/g, 'd')
    .replace(/MMMM/g, 'MMMM')
    .replace(/MMM/g, 'MMM')
    .replace(/MM/g, 'MM')
    .replace(/M/g, 'M')
    .replace(/HH/g, 'HH')
    .replace(/H/g, 'H')
    .replace(/hh/g, 'hh')
    .replace(/h/g, 'h')
    .replace(/mm/g, 'mm')
    .replace(/m/g, 'm')
    .replace(/ss/g, 'ss')
    .replace(/s/g, 's')
    .replace(/A/g, 'a')
    .replace(/a/g, 'a');

  return `format(date, ${quote}${convertedFormat}${quote})`;
}],

// Манипуляции
[/\.\add\((\d+),\s*['"](.*?)['"]\)/g, 'add(date, { $2: $1 })'],
[/\.\subtract\((\d+),\s*['"](.*?)['"]\)/g, 'sub(date, { $2: $1 })'],

// Проверки
[/\.\isAfter\((.*?)\)/g, 'isAfter(date, $1)'],
[/\.\isBefore\((.*?)\)/g, 'isBefore(date, $1)'],
[/\.\isSame\((.*?),\s*['"]day['"]\)/g, 'isSameDay(date, $1)']
];

let migrated = code;
patterns.forEach(([pattern, replacement]) => {
  migrated = migrated.replace(pattern, replacement);
});

```



```

    return migrated;
}

// Пример миграции
const momentCode = `
    const date = moment('2026-02-10');
    const nextWeek = date.add(7, 'days');
    const formatted = nextWeek.format('DD.MM.YYYY');
    const isAfterToday = nextWeek.isAfter(moment());
`;

console.log(migrateMomentToDateFns(momentCode));

```

Постепенная миграция в большом проекте

```

javascript

// Шаг 1: Создание адаптера
class DateAdapter {
  constructor(date) {
    if (date instanceof Date) {
      this._date = date;
    } else if (typeof date === 'string') {
      this._date = new Date(date);
    } else if (date && date._isAMomentObject) {
      // Поддержка moment объектов
      this._date = date.toDate();
    } else {
      this._date = new Date();
    }
  }

  add(amount, unit) {
    const fn = this.getAddFunction(unit);
    return new DateAdapter(fn(this._date, amount));
  }

  format(formatStr) {
    // Используем date-fns для форматирования
    const { format } = require('date-fns');
    return format(this._date, this.convertFormat(formatStr));
  }

  // ... другие методы совместимые с Moment.js API

```

```
}
```

```
// Шаг 2: Замена импортов  
// Было: import moment from 'moment';  
// Стало: import DateAdapter from './DateAdapter';
```

```
// Шаг 3: Постепенная замена кода
```

4.2. Интеграция с современным стеком

Webpack конфигурация

```
javascript  
  
// webpack.config.js  
module.exports = {  
  // ...  
  resolve: {  
    alias: {  
      // Для постепенной миграции  
      'moment': 'date-fns',  
      // Или кастомный адаптер  
      'moment$': path.resolve(__dirname, 'src/utils/DateAdapter.js')  
    }  
  },  
  
  plugins: [  
    new webpack.IgnorePlugin({  
      // Игнорируем неиспользуемые локали moment  
      resourceRegExp: /^\.\/locale$/,  
      contextRegExp: /moment$/  
    })  
  ]  
};
```

Babel плагин для оптимизации

```
javascript  
  
// .babelrc  
{  
  "plugins": [  
    ["transform-imports", {  
      "date-fns": {
```

```
    "transform": "date-fns/${member}",
    "preventFullImport": true
  }
}]
]
}
```

// Результат:
// import { format } from 'date-fns';
// ↓ преобразуется в ↓
// import format from 'date-fns/format';

Часть 5: Будущее — Temporal API

5.1. Введение в Temporal

Что такое Temporal API?

```
javascript
```

// Temporal – это новая встроенная API для работы со временем в JavaScript
// Текущий статус: Stage 3 (2026 год – возможно уже в браузерах)

// Ключевые преимущества:
// 1. Неизменяемые объекты
// 2. Разделение даты, времени и часовых поясов
// 3. Богатый API для манипуляций
// 4. Лучшая поддержка календарей
// 5. Встроенная в JavaScript (не нужна библиотека)

// Основные типы:
// Temporal.Instant – момент времени (как Date)
// Temporal.PlainDate – только дата
// Temporal.PlainTime – только время
// Temporal.ZonedDateTime – дата и время с часовым поясом
// Temporal.Duration – длительность

Примеры использования Temporal

```
javascript
```

// Создание даты
const date = Temporal.PlainDate.from('2026-02-10');

```
console.log(date.toString()); // "2026-02-10"

// Добавление дней
const tomorrow = date.add({ days: 1 });
console.log(tomorrow.toString()); // "2026-02-11"

// Форматирование
console.log(date.toLocaleString('ru-RU')); // "10.02.2026"

// Работа с часовыми поясами
const zoned = Temporal.ZonedDateTime.from({
  timeZone: 'Europe/Moscow',
  year: 2026,
  month: 2,
  day: 10,
  hour: 14,
  minute: 30
});

console.log(zoned.toString()); // "2026-02-10T14:30:00+03:00[Europe/Moscow]"

// Конвертация между часовыми поясами
const nyTime = zoned.withTimeZone('America/New_York');
console.log(nyTime.toString()); // "2026-02-10T06:30:00-05:00[America/New_York]"
```

5.2. Полифиллы и подготовка

Использование полифилла Temporal

```
javascript

// Установка полифилла
npm install @js-temporal/polyfill

// Использование
import { Temporal } from '@js-temporal/polyfill';

// Проверка поддержки в браузере
if (typeof Temporal === 'undefined') {
  // Загружаем полифилл динамически
  await import('@js-temporal/polyfill');
}
```

```
// Постепенная миграция с date-fns/moment
function createTemporalAdapter() {
  return {
    // Адаптер для плавного перехода
    fromDate(date) {
      return Temporal.Instant.from(date.toISOString());
    },

    toDate(temporal) {
      return new Date(temporal.toString());
    },

    // Совместимые методы
    format(date, formatStr) {
      const temporal = this.fromDate(date);
      // Конвертация форматов...
      return temporal.toLocaleString();
    }
  };
}
```

Заключение

Итоговые рекомендации на 2026 год

Выбор библиотеки

```
javascript

const recommendations2026 = {
  // Для новых проектов
  newProjects: {
    firstChoice: 'date-fns',
    reasons: [
      'Лучшая производительность',
      'Меньший размер бандла',
      'Tree-shaking поддержка',
      'Активная разработка',
      'TypeScript из коробки'
    ]
  },
}
```

```
// Для существующих проектов
existingProjects: {
  usingMoment: {
    action: 'Рассмотреть миграцию на date-fns',
    when: [
      'Есть проблемы с производительностью',
      'Хотите уменьшить размер бандла',
      'Начинаете рефакторинг кода'
    ]
  },
  usingDateFns: {
    action: 'Продолжать использовать',
    notes: 'Вы на правильном пути'
  }
},

// Специальные случаи
specialCases: {
  richLocalization: {
    recommendation: 'Moment.js или date-fns с кастомными локалями',
    note: 'Temporal API пока не имеет такой богатой локализации'
  },
  complexTimezone: {
    recommendation: 'date-fns-tz или moment-timezone',
    note: 'Temporal API решает эту проблему в будущем'
  },
  legacyBrowser: {
    recommendation: 'Moment.js с полифиллами',
    note: 'date-fns требует современных браузеров'
  }
}
};
```

Ключевые выводы

1. **Moment.js** — классика, но устаревает. Используйте только в legacy проектах.
2. **date-fns** — современный стандарт для новых проектов. Модульный, быстрый, с tree-shaking.
3. **Temporal API** — будущее JavaScript. Начинать изучать уже сейчас.
4. **Размер имеет значение** — date-fns позволяет уменьшить бандл в 10-100 раз.
5. **Производительность** — date-fns обычно быстрее Moment.js в 2-3 раза.

Философия разработки

```
javascript

// Принципы работы с датами в 2026:
const datePrinciples = {
  // 1. Всегда используйте UTC внутри приложения
  useUTC: true,

  // 2. Конвертируйте в локальное время только при отображении
  convertAtPresentation: true,

  // 3. Используйте ISO 8601 для хранения и передачи
  storageFormat: 'ISO8601',

  // 4. Выбирайте инструмент под задачу
  toolSelection: {
    simple: 'Нативный Date',
    formatting: 'date-fns',
    complex: 'date-fns или Temporal',
    legacy: 'Moment.js'
  },

  // 5. Тестируйте с разными часовыми поясами
  testing: ['UTC', 'Europe/Moscow', 'America/New_York', 'Asia/Tokyo'],

  // 6. Учитывайте летнее время
  considerDST: true
};
```

Библиотеки для работы с датами эволюционируют вместе с JavaScript. Выбор между Moment.js и date-fns сегодня — это выбор между legacy и modern подходами. Для большинства новых проектов date-fns является оптимальным выбором, сочетая производительность, модульность и современный API. При этом стоит следить за развитием Temporal API, который обещает стать стандартом работы с датами в будущих версиях JavaScript.

13.3. Обработка ошибок: try-catch-finally

Введение: Философия обработки ошибок

Обработка ошибок — это не просто техническая необходимость, а фундаментальный аспект разработки надежного программного обеспечения. В JavaScript ошибки неизбежны: некорректные входные данные, сетевые сбои, отсутствие файлов, неожиданные значения. Умение правильно обрабатывать ошибки отличает профессионального разработчика от любителя.

Почему ошибки нужно обрабатывать?

javascript

// ✗ ПЛОХО: Игнорирование ошибок

```
function divide(a, b) {  
  return a / b; // Если b = 0, получим Infinity, а не ошибку  
}
```

```
function getUsername(user) {  
  return user.name.toUpperCase(); // Если user = null, программа упадет  
}
```

// ✅ ХОРОШО: Обработка ошибок

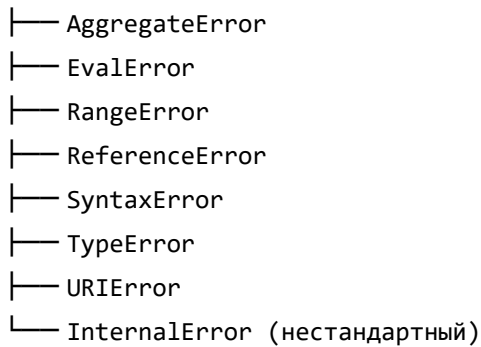
```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error('Division by zero');  
  }  
  return a / b;  
}
```

```
function getUsername(user) {  
  if (!user || !user.name) {  
    return 'Anonymous';  
  }  
  return user.name.toUpperCase();  
}
```


Иерархия ошибок в JavaScript

text

Error (базовый класс)



Часть 1: Конструкция try-catch-finally

1.1. Базовый синтаксис и семантика

javascript

```
try {  
    // Потенциально опасный код  
    // Ошибки, возникшие здесь, перехватываются  
} catch (error) {  
    // Обработка ошибки  
    // Выполняется только если в try произошла ошибка  
} finally {  
    // Код, который выполнится в любом случае  
    // Даже если была ошибка или return  
}
```

Простой пример

javascript

```
function parseJSON(jsonString) {  
    try {  
        const data = JSON.parse(jsonString);  
        console.log('Парсинг успешен');  
        return data;  
    } catch (error) {  
        console.error('Ошибка парсинга JSON:', error.message);  
        return null;  
    } finally {  

```

```

    console.log('Попытка парсинга завершена');
  }
}

// Тестирование
const validJSON = '{"name": "John", "age": 30}';
const invalidJSON = '{name: "John"}';

console.log(parseJSON(validJSON));
// Парсинг успешен
// Попытка парсинга завершена
// {name: "John", age: 30}

console.log(parseJSON(invalidJSON));
// Ошибка парсинга JSON: Unexpected token n in JSON at position 1
// Попытка парсинга завершена
// null

```

1.2. Детальное поведение блоков

Порядок выполнения

```

javascript

function demonstrateExecutionOrder() {
  console.log('1. Начало функции');

  try {
    console.log('2. Начало try блока');

    if (Math.random() > 0.5) {
      throw new Error('Случайная ошибка');
    }

    console.log('3. Конец try блока (без ошибки)');
  } catch (error) {
    console.log('4. Блок catch выполняется только при ошибке');
    console.log('  Ошибка:', error.message);
  } finally {
    console.log('5. Блок finally выполняется ВСЕГДА');
  }

  console.log('6. Конец функции');
}

```

```
}
```

```
demonstrateExecutionOrder();
```

Взаимодействие с return

javascript

```
function demonstrateReturn() {  
  try {  
    console.log('try блок начинается');  
    return 'Возврат из try';  
  } catch (error) {  
    console.log('catch блок');  
    return 'Возврат из catch';  
  } finally {  
    console.log('finally блок выполняется ДО return');  
    // finally выполняется, но не может изменить return  
    // (если только сам не выбросит ошибку или не сделает return)  
  }  
}
```

```
console.log(demonstrateReturn());  
// try блок начинается  
// finally блок выполняется ДО return  
// Возврат из try
```

Особый случай: return в finally

javascript

```
function trickyFinally() {  
  try {  
    console.log('try');  
    throw new Error('Ошибка в try');  
  } catch (error) {  
    console.log('catch');  
    return 'Возврат из catch'; // Этот return будет проигнорирован  
  } finally {  
    console.log('finally');  
    return 'Возврат из finally'; // Переопределяет предыдущий return  
  }  
}
```

```
console.log(trickyFinally());  
// try
```

```
// catch
// finally
// Возврат из finally

// ▲ НИКОГДА НЕ ДЕЛАЙТЕ return В FINALLY!
```

1.3. Опциональный catch binding (ES2019)

```
javascript

// ES2019: можно опускать переменную ошибки в catch
try {
  JSON.parse('invalid json');
} catch {
  // Нам не нужна информация об ошибке
  console.log('Ошибка парсинга JSON');
}

// Раньше приходилось писать так:
try {
  JSON.parse('invalid json');
} catch (ignored) {
  // Переменная declared but never used
  console.log('Ошибка парсинга JSON');
}
```

Часть 2: Объект ошибки и его свойства

2.1. Структура объекта Error

```
javascript

try {
  throw new Error('Что-то пошло не так');
} catch (error) {
  // Основные свойства
  console.log(error.name);    // "Error"
  console.log(error.message); // "Что-то пошло не так"
  console.log(error.stack);   // Стек вызовов

  // Все свойства
  console.log(Object.getOwnPropertyNames(error));
}
```

```
// ["stack", "message", "name"]

// Стек вызовов (нестандартное, но везде поддерживается)
console.log(error.stack);
// Error: Что-то пошло не так
//   at Object.<anonymous> (/app.js:2:9)
//   at Module._compile (internal/modules/cjs/loader.js:1138:30)
//   ...
}
```

2.2. Встроенные типы ошибок

RangeError — выход за допустимые границы

```
javascript

// Пример 1: Рекурсия без базового случая
function infiniteRecursion() {
  return infiniteRecursion();
}

try {
  infiniteRecursion();
} catch (error) {
  console.log(error.name);    // "RangeError"
  console.log(error.message); // "Maximum call stack size exceeded"
}

// Пример 2: Недопустимая длина массива
try {
  const arr = new Array(-1);
} catch (error) {
  console.log(error.name);    // "RangeError"
  console.log(error.message); // "Invalid array length"
}

// Пример 3: Методы чисел с недопустимыми аргументами
try {
  (10).toFixed(-1);
} catch (error) {
  console.log(error.name);    // "RangeError"
  console.log(error.message); // "toFixed() digits argument must be between 0 and 100"
}
```

ReferenceError — обращение к несуществующей переменной

javascript

// Пример 1: Необъявленная переменная

```
try {
  console.log(nonExistentVariable);
} catch (error) {
  console.log(error.name);    // "ReferenceError"
  console.log(error.message); // "nonExistentVariable is not defined"
}
```

// Пример 2: Неправильная область видимости

```
try {
  if (true) {
    let x = 10;
  }
  console.log(x); // x не доступна вне блока
} catch (error) {
  console.log(error.name);    // "ReferenceError"
  console.log(error.message); // "x is not defined"
}
```

TypeError — несоответствие типов

javascript

// Пример 1: Вызов не-функции

```
try {
  const notAFunction = {};
  notAFunction();
} catch (error) {
  console.log(error.name);    // "TypeError"
  console.log(error.message); // "notAFunction is not a function"
}
```

// Пример 2: Чтение свойства undefined/null

```
try {
  const user = null;
  console.log(user.name);
} catch (error) {
  console.log(error.name);    // "TypeError"
  console.log(error.message); // "Cannot read property 'name' of null"
}
```

// Пример 3: Изменение константы

```
try {  
  const PI = 3.14;  
  PI = 3.14159;  
} catch (error) {  
  console.log(error.name);    // "TypeError"  
  console.log(error.message); // "Assignment to constant variable"  
}
```

SyntaxError — синтаксическая ошибка

javascript

*// ▲ Важно: SyntaxError нельзя перехватить в try-catch на том же уровне
// Потому что ошибка возникает на этапе парсинга, до выполнения кода*

// ✕ Этот код даже не запустится

```
try {  
  eval('if condition { console.log("error"); }'); // Пропущены скобки  
} catch (error) {  
  // Сюда мы не попадем  
  console.log(error.name); // "SyntaxError"  
}
```

// ☑ Правильно: перехватываем при eval или Function

```
try {  
  new Function('if condition { console.log("error"); }');  
} catch (error) {  
  console.log(error.name);    // "SyntaxError"  
  console.log(error.message); // "Unexpected token condition"  
}
```

// ☑ Или при динамическом импорте

```
import('invalid-module').catch(error => {  
  console.log(error.name); // "SyntaxError"  
});
```

URIError — ошибки encodeURI/decodeURI

javascript

```
try {  
  decodeURI('%');  
} catch (error) {  
  console.log(error.name);    // "URIError"  
  console.log(error.message); // "URI malformed"
```

```

}

try {
  encodeURI('\uD800'); // Некорректный суррогатный символ
} catch (error) {
  console.log(error.name);    // "URIError"
  console.log(error.message); // "URI malformed"
}

```

AggregateError — множество ошибок (ES2021)

```

javascript

// AggregateError используется с Promise.any()
const promises = [
  Promise.reject(new Error('Ошибка 1')),
  Promise.reject(new Error('Ошибка 2')),
  Promise.resolve('Успех 3')
];

Promise.any(promises)
  .then(result => console.log(result))
  .catch(error => {
    console.log(error.name);    // "AggregateError"
    console.log(error.message); // "All promises were rejected"
    console.log(error.errors);  // [Error: Ошибка 1, Error: Ошибка 2]
  });

// Можно создать вручную
try {
  throw new AggregateError([
    new Error('Ошибка A'),
    new Error('Ошибка B')
  ], 'Несколько ошибок');
} catch (error) {
  console.log(error.name);    // "AggregateError"
  console.log(error.errors);  // [Error, Error]
}

```

2.3. Создание пользовательских ошибок

Базовый класс ошибки

```

javascript

```



```

class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
    this.timestamp = new Date().toISOString();

    // Важно для корректного стека вызовов
    if (Error.captureStackTrace) {
      Error.captureStackTrace(this, ValidationError);
    }
  }
}

// Использование
try {
  throw new ValidationError('Имя обязательно', 'name');
} catch (error) {
  console.log(error.name);      // "ValidationError"
  console.log(error.message);   // "Имя обязательно"
  console.log(error.field);     // "name"
  console.log(error.timestamp); // "2026-02-11T10:30:00.000Z"
}

```

Иерархия пользовательских ошибок

```

javascript

// Базовый класс для всех ошибок приложения
class AppError extends Error {
  constructor(message, statusCode = 500) {
    super(message);
    this.name = this.constructor.name;
    this.statusCode = statusCode;
    this.isOperational = true; // Ожидаемая ошибка
    Error.captureStackTrace(this, this.constructor);
  }
}

// Специфические ошибки
class DatabaseError extends AppError {
  constructor(message, query) {
    super(message, 500);
    this.query = query;
  }
}

```

```
}
```

```
class NotFoundError extends AppError {  
  constructor(resource, id) {  
    super(`${resource} with id ${id} not found`, 404);  
    this.resource = resource;  
    this.resourceId = id;  
  }  
}
```

```
class AuthenticationError extends AppError {  
  constructor(message = 'Authentication failed') {  
    super(message, 401);  
  }  
}
```

```
class AuthorizationError extends AppError {  
  constructor(message = 'Access denied') {  
    super(message, 403);  
  }  
}
```

```
class ValidationError extends AppError {  
  constructor(errors) {  
    super('Validation failed', 400);  
    this.errors = errors;  
  }  
}
```

```
// Использование
```

```
function findUserById(id) {  
  if (!id || id <= 0) {  
    throw new ValidationError([{  
      field: 'id',  
      message: 'ID must be positive number'  
    }]);  
  }  
}
```

```
const user = database.find(id);  
if (!user) {  
  throw new NotFoundError('User', id);  
}
```

```
return user;
```

```

}

// Обработка в едином месте
try {
  const user = findUserId(-5);
} catch (error) {
  if (error instanceof AppError) {
    console.log(`[${error.name}] ${error.message} (${error.statusCode})`);
    // Логирование в зависимости от типа ошибки
    if (error instanceof ValidationError) {
      console.log('Validation details:', error.errors);
    }
    if (error instanceof DatabaseError) {
      console.log('Failed query:', error.query);
    }
  } else {
    // Неожиданная ошибка
    console.error('Unhandled error:', error);
  }
}

```

Часть 3: Паттерны обработки ошибок

3.1. Защитное программирование

Проверка предусловий

```

javascript

class UserService {
  createUser(userData) {
    // Защита от null/undefined
    if (!userData) {
      throw new ValidationError('User data is required');
    }

    // Проверка обязательных полей
    const requiredFields = ['email', 'name', 'age'];
    for (const field of requiredFields) {
      if (!userData[field]) {
        throw new ValidationError(`Field ${field} is required`, field);
      }
    }
  }
}

```

```

}

// Проверка форматов
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(userData.email)) {
  throw new ValidationError('Invalid email format', 'email');
}

if (!Number.isInteger(userData.age) || userData.age < 0 || userData.age > 150) {
  throw new ValidationError('Age must be integer between 0 and 150', 'age');
}

// Бизнес-логика
return this.saveUser(userData);
}

saveUser(userData) {
  try {
    return database.insert('users', userData);
  } catch (error) {
    // Трансформация ошибки в доменный формат
    throw new DatabaseError('Failed to save user', error);
  }
}
}

```

Guard clauses

```

javascript

function processOrder(order) {
  // Guard clauses - ранний выход при ошибках
  if (!order) {
    throw new ValidationError('Order is required');
  }

  if (!order.items || order.items.length === 0) {
    throw new ValidationError('Order must have at least one item');
  }

  if (!order.customer) {
    throw new ValidationError('Order must have a customer');
  }

  if (order.total < 0) {

```

```

    throw new ValidationError('Order total cannot be negative');
  }

  // Основная логика
  const processedItems = order.items.map(item => {
    if (!item.price || item.price <= 0) {
      throw new ValidationError(`Invalid price for item ${item.id}`);
    }
    return {
      ...item,
      total: item.price * (item.quantity || 1)
    };
  });

  return {
    ...order,
    items: processedItems,
    processedAt: new Date()
  };
}

```

3.2. Result-объекты (Either pattern)

```

javascript

// Альтернатива исключениям - явный возврат результата/ошибки
class Result {
  constructor(success, value, error) {
    this.success = success;
    this.value = value;
    this.error = error;
  }

  static ok(value) {
    return new Result(true, value, null);
  }

  static fail(error) {
    return new Result(false, null, error);
  }

  isOk() {
    return this.success;
  }
}

```

```

isFail() {
  return !this.success;
}

onSuccess(fn) {
  if (this.success) {
    return Result.ok(fn(this.value));
  }
  return this;
}

onFail(fn) {
  if (!this.success) {
    fn(this.error);
  }
  return this;
}

unwrap() {
  if (!this.success) {
    throw this.error;
  }
  return this.value;
}
}

// Использование Result
function divide(a, b) {
  if (b === 0) {
    return Result.fail(new Error('Division by zero'));
  }
  return Result.ok(a / b);
}

function square(n) {
  return Result.ok(n * n);
}

// Цепочка операций без try-catch
const result = divide(10, 2)
  .onSuccess(n => square(n))
  .onSuccess(n => {
    console.log('Результат:', n);
  });

```

```

    return Result.ok(n);
  })
  .onFail(error => {
    console.error('Ошибка:', error.message);
  });

console.log(result.unwrap()); // 25

// Безопасное получение значения
const safeValue = result.isOk() ? result.value : null;

```

3.3. Try-catch обертки

Wrapper для синхронных функций

```

javascript

function tryCatch(fn, errorHandler) {
  return function(...args) {
    try {
      return fn(...args);
    } catch (error) {
      return errorHandler(error, ...args);
    }
  };
}

// Использование
const safeJSONParse = tryCatch(
  JSON.parse,
  (error, jsonString) => {
    console.error(`Failed to parse: ${jsonString}`);
    return null;
  }
);

console.log(safeJSONParse('{"valid": true}')); // {valid: true}
console.log(safeJSONParse('invalid')); // null (с логом ошибки)

// Создание нескольких безопасных методов
const safeMethods = {
  parseInt: tryCatch(parseInt, () => NaN),
  JSONParse: tryCatch(JSON.parse, () => ({})),

```

```
decodeURI: tryCatch(decodeURI, () => '')
};
```

Декоратор для классов

javascript

```
function catchErrors(errorHandler) {
  return function(target, propertyKey, descriptor) {
    const originalMethod = descriptor.value;

    descriptor.value = async function(...args) {
      try {
        return await originalMethod.apply(this, args);
      } catch (error) {
        return errorHandler(error, ...args);
      }
    };

    return descriptor;
  };
}

class APIClient {
  @catchErrors((error) => {
    console.error('API Error:', error.message);
    return { error: error.message };
  })
  async fetchUser(id) {
    const response = await fetch(`/api/users/${id}`);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    return response.json();
  }

  @catchErrors((error) => {
    return { success: false, error: error.message };
  })
  async updateUser(id, data) {
    const response = await fetch(`/api/users/${id}`, {
      method: 'PUT',
      body: JSON.stringify(data)
    });
    return response.json();
  }
}
```



```
}  
}
```

3.4. Стратегии повторных попыток (Retry)

javascript

```
class RetryStrategy {  
  constructor(options = {}) {  
    this.maxAttempts = options.maxAttempts || 3;  
    this.delay = options.delay || 1000;  
    this.backoffFactor = options.backoffFactor || 2;  
    this.retryableErrors = options.retryableErrors || [  
      'NetworkError',  
      'TimeoutError',  
      'DatabaseError'  
    ];  
  }  
  
  async execute(fn, context = '') {  
    let lastError;  
  
    for (let attempt = 1; attempt <= this.maxAttempts; attempt++) {  
      try {  
        console.log(`Attempt ${attempt}/${this.maxAttempts} ${context}`);  
        return await fn();  
      } catch (error) {  
        lastError = error;  
  
        if (!this.shouldRetry(error, attempt)) {  
          console.log(`Not retrying: ${error.name}`);  
          break;  
        }  
  
        const waitTime = this.calculateDelay(attempt);  
        console.log(`Retry in ${waitTime}ms...`);  
        await this.sleep(waitTime);  
      }  
    }  
  
    throw new Error(`All ${this.maxAttempts} attempts failed: ${lastError.message}`);  
  }  
  
  shouldRetry(error, attempt) {
```

```

    if (attempt >= this.maxAttempts) return false;
    return this.retryableErrors.includes(error.name);
  }

  calculateDelay(attempt) {
    return this.delay * Math.pow(this.backoffFactor, attempt - 1);
  }

  sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
  }
}

// Использование
const retry = new RetryStrategy({
  maxAttempts: 5,
  delay: 500,
  backoffFactor: 1.5
});

async function fetchWithRetry(url) {
  return retry.execute(async () => {
    const response = await fetch(url);
    if (response.status === 429) {
      throw new Error('Rate limit exceeded');
    }
    return response.json();
  }, `GET ${url}`);
}

```

Часть 4: Асинхронная обработка ошибок

4.1. Промисы и ошибки

Обработка ошибок в Promise

```

javascript

// Автоматический catch на конце цепочки
fetch('/api/user')
  .then(response => {
    if (!response.ok) {

```

```

    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
})
.then(user => {
  console.log('User:', user);
  return fetch(`/api/posts/${user.id}`);
})
.then(response => response.json())
.then(posts => {
  console.log('Posts:', posts);
})
.catch(error => {
  // Любая ошибка из любого then
  console.error('Failed to load user posts:', error.message);
})
.finally(() => {
  console.log('Request completed');
});

// Обработка ошибок на каждом шаге
fetch('/api/user')
.then(response => {
  if (!response.ok) {
    throw new Error('User not found');
  }
  return response.json();
})
.catch(error => {
  console.warn('User fetch failed, using default');
  return { id: 0, name: 'Guest' }; // Восстановление после ошибки
})
.then(user => {
  console.log('User:', user);
});

```

Конструктор Promise и reject

```

javascript

function timeout(ms) {
  return new Promise((resolve, reject) => {
    if (ms < 0) {
      reject(new RangeError('Timeout must be positive'));
    }
    return;
  });
}

```

```

}

const timer = setTimeout(resolve, ms);

// Отмена таймера при ошибке
if (timer.unref) {
  timer.unref(); // Node.js
}
});
}

timeout(-1000)
  .then(() => console.log('Done'))
  .catch(error => console.log(error.name)); // RangeError

// Promise.reject
Promise.reject(new Error('Direct rejection'))
  .catch(error => console.log(error.message));

// Promise.resolve с ошибкой не работает
Promise.resolve(Promise.reject(new Error('Nested')))
  .catch(error => console.log('Caught:', error.message));

```

4.2. Async/await и try-catch

Базовый паттерн

```

javascript

async function fetchUserData(userId) {
  try {
    const response = await fetch(`/api/users/${userId}`);

    if (!response.ok) {
      throw new Error(`API error: ${response.status}`);
    }

    const user = await response.json();
    const posts = await fetchUserPosts(user.id);
    const comments = await fetchPostComments(posts);

    return { user, posts, comments };
  } catch (error) {

```

```

    console.error('Failed to fetch user data:', error);

    // Пробрасываем ошибку выше с контекстом
    throw new Error(`UserDataError: ${error.message}`);
  }
}

// Использование
try {
  const data = await fetchUserData(123);
  console.log('User data loaded:', data);
} catch (error) {
  console.error('Application error:', error);
}

```

Несколько асинхронных операций

```

javascript

async function parallelOperations() {
  const errors = [];
  const results = [];

  const operations = [
    fetch('/api/data1'),
    fetch('/api/data2'),
    fetch('/api/data3')
  ];

  // Обработка каждой операции отдельно
  for (const op of operations) {
    try {
      const response = await op;
      results.push(await response.json());
    } catch (error) {
      errors.push(error);
      results.push(null);
    }
  }

  return { results, errors };
}

// Promise.allSettled - встроенное решение
async function safeParallel() {

```

```

const promises = [
  fetch('/api/data1').then(r => r.json()),
  fetch('/api/data2').then(r => r.json()),
  fetch('/api/data3').then(r => r.json())
];

const results = await Promise.allSettled(promises);

const successful = results
  .filter(r => r.status === 'fulfilled')
  .map(r => r.value);

const failed = results
  .filter(r => r.status === 'rejected')
  .map(r => r.reason);

return { successful, failed };
}

```

Утилита для безопасного async

javascript

```

function asyncTryCatch(asyncFn) {
  return function(...args) {
    return asyncFn(...args).catch(error => {
      console.error('Async operation failed:', error);
      return null;
    });
  };
}

const safeFetch = asyncTryCatch(async (url) => {
  const response = await fetch(url);
  return response.json();
});

// Использование без try-catch
const data = await safeFetch('/api/data');
if (data) {
  console.log('Data loaded:', data);
} else {
  console.log('Using default data');
}

```

```

// Версия с возвратом [data, error]
function toResult(promise) {
  return promise
    .then(data => [data, null])
    .catch(error => [null, error]);
}

// Go-style error handling
const [user, userError] = await toResult(fetchUser(123));
if (userError) {
  console.error('User fetch failed:', userError);
  return;
}

const [posts, postsError] = await toResult(fetchPosts(user.id));
if (postsError) {
  console.error('Posts fetch failed:', postsError);
  return;
}

console.log('User:', user);
console.log('Posts:', posts);

```

4.3. Событийный цикл и неперехваченные ошибки

unhandledrejection в браузере

```

javascript

// Глобальный обработчик неперехваченных rejections
window.addEventListener('unhandledrejection', (event) => {
  console.error('Unhandled promise rejection:', event.reason);

  // Предотвращаем вывод в консоль
  event.preventDefault();

  // Отправляем в систему логирования
  logError(event.reason);
});

// Обработчик для уже обработанных rejections
window.addEventListener('rejectionhandled', (event) => {
  console.log('Promise rejection handled:', event.reason);
});

```

```
});
```

```
// Пример
```

```
Promise.reject(new Error('Forgotten error'));
```

```
// Событие unhandledrejection срабатывает
```

uncaughtException в Node.js

```
javascript
```

```
process.on('uncaughtException', (error) => {  
  console.error('Uncaught exception:', error);
```

```
// Логирование
```

```
logger.error('Uncaught exception', error);
```

```
// Корректное завершение (graceful shutdown)
```

```
gracefulShutdown()
```

```
  .then(() => process.exit(1))
```

```
  .catch(() => process.exit(1));
```

```
});
```

```
process.on('unhandledRejection', (reason, promise) => {  
  console.error('Unhandled rejection at:', promise, 'reason:', reason);
```

```
// В Node.js рекомендуется завершать процесс
```

```
// process.exit(1);
```

```
});
```

```
// В современных версиях Node.js (15+)
```

```
process.on('multipleResolves', (type, promise, reason) => {  
  console.error('Multiple resolves:', type, promise, reason);  
});
```

```
function gracefulShutdown() {  
  return new Promise((resolve) => {  
    console.log('Closing database connections...');  
    console.log('Stopping server...');  
    console.log('Cleaning up...');  
    setTimeout(resolve, 1000);  
  });  
}
```


Часть 5: Продвинутые техники

5.1. Контекст и стек вызовов

Сохранение контекста ошибки

```
javascript

class ContextualError extends Error {
  constructor(message, context = {}) {
    super(message);
    this.name = 'ContextualError';
    this.context = context;
    this.timestamp = new Date().toISOString();

    // Сохраняем оригинальный стек
    Error.captureStackTrace(this, this.constructor);

    // Добавляем контекст в сообщение
    this.contextualMessage = `${message} | Context: ${JSON.stringify(context)}`;
  }
}

// Использование
async function processOrder(orderId) {
  try {
    const order = await database.findOrder(orderId);
    const user = await database.findUser(order.userId);

    if (user.balance < order.total) {
      throw new ContextualError('Insufficient funds', {
        orderId,
        userId: user.id,
        balance: user.balance,
        required: order.total
      });
    }

    return await processPayment(order, user);
  } catch (error) {
    if (error instanceof ContextualError) {
      throw error; // Уже с контекстом
    }
  }
}
```

```

}

// Добавляем контекст к существующей ошибке
throw new ContextualError(error.message, {
  orderId,
  originalError: error.name,
  stack: error.stack
});
}
}

```

Цепочка ошибок (Error Chaining)

javascript

```

class ChainedError extends Error {
  constructor(message, cause) {
    super(message);
    this.name = 'ChainedError';
    this.cause = cause;
    this.timestamp = new Date().toISOString();

    if (cause) {
      // Сохраняем стек причины
      this.stack = `${this.stack}\nCaused by: ${cause.stack}`;
    }
  }
}

async function fetchData() {
  try {
    await makeRequest();
  } catch (error) {
    throw new ChainedError('Failed to fetch data', error);
  }
}

// Использование
try {
  await fetchData();
} catch (error) {
  console.error(error.message);

  if (error.cause) {
    console.error('Caused by:', error.cause.message);
  }
}

```

```
    console.error('Original stack:', error.cause.stack);
  }
}
```

5.2. Функциональная обработка ошибок

Either монада

```
javascript

class Either {
  constructor(value) {
    this.value = value;
  }

  static left(error) {
    const either = new Either(null);
    either.isLeft = true;
    either.isRight = false;
    either.error = error;
    return either;
  }

  static right(value) {
    const either = new Either(value);
    either.isLeft = false;
    either.isRight = true;
    return either;
  }

  map(fn) {
    if (this.isRight) {
      return Either.right(fn(this.value));
    }
    return this;
  }

  flatMap(fn) {
    if (this.isRight) {
      return fn(this.value);
    }
    return this;
  }
}
```

```

fold(leftFn, rightFn) {
  return this.isRight ? rightFn(this.value) : leftFn(this.error);
}

catch(fn) {
  if (this.isLeft) {
    return Either.right(fn(this.error));
  }
  return this;
}
}

```

// Использование

```

function parseJSON(jsonString) {
  try {
    return Either.right(JSON.parse(jsonString));
  } catch (error) {
    return Either.left(error);
  }
}

function getUserAge(user) {
  if (user && typeof user.age === 'number') {
    return Either.right(user.age);
  }
  return Either.left(new Error('Age not found'));
}

const result = parseJSON('{ "age": 25 }')
  .flatMap(getUserAge)
  .fold(
    (error) => console.error('Failed:', error.message),
    (age) => console.log('Age:', age)
  );

```

Try монада

```

javascript

class Try {
  constructor(fn) {
    try {
      this.value = fn();
      this.isSuccess = true;
    }
  }
}

```

```

    } catch (error) {
      this.error = error;
      this.isSuccess = false;
    }
  }

  static of(fn) {
    return new Try(fn);
  }

  map(fn) {
    if (this.isSuccess) {
      return Try.of(() => fn(this.value));
    }
    return this;
  }

  flatMap(fn) {
    if (this.isSuccess) {
      return fn(this.value);
    }
    return this;
  }

  getOrElse(defaultValue) {
    return this.isSuccess ? this.value : defaultValue;
  }

  orElse(fn) {
    if (this.isSuccess) {
      return this;
    }
    return Try.of(() => fn(this.error));
  }
}

// Использование
const result = Try.of(() => JSON.parse('{ "name": "John" }'))
  .map(user => user.name)
  .map(name => name.toUpperCase())
  .getOrElse('Anonymous');

console.log(result); // "JOHN"

```

```
const failed = Try.of(() => JSON.parse('invalid'))
  .orElse(() => 'default.json')
  .map(filename => fs.readFileSync(filename, 'utf8'))
  .getOrElse('{}');
```

5.3. Логирование и мониторинг

Структурированное логирование

javascript

```
class ErrorLogger {
  constructor(options = {}) {
    this.service = options.service || 'app';
    this.environment = options.environment || process.env.NODE_ENV;
    this.version = options.version || '1.0.0';
  }

  log(error, context = {}) {
    const errorLog = {
      timestamp: new Date().toISOString(),
      service: this.service,
      environment: this.environment,
      version: this.version,

      error: {
        name: error.name,
        message: error.message,
        stack: error.stack,
        code: error.code || error.statusCode
      },

      context: {
        ...context,
        url: typeof window !== 'undefined' ? window.location.href : process.cwd(),
        userAgent: typeof navigator !== 'undefined' ? navigator.userAgent : null,
        memory: process.memoryUsage ? process.memoryUsage() : null
      }
    };

    // Отправка в систему логирования
    if (this.environment === 'production') {
      this.sendToLoggingService(errorLog);
    }
  }
}
```

```

    } else {
      console.error(JSON.stringify(errorLog, null, 2));
    }

    return errorLog;
  }

  async sendToLoggingService(errorLog) {
    try {
      await fetch('https://logging.service.com/errors', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(errorLog)
      });
    } catch (sendError) {
      console.error('Failed to send error log:', sendError);
    }
  }
}

// Использование
const logger = new ErrorLogger({
  service: 'user-api',
  environment: process.env.NODE_ENV,
  version: require('./package.json').version
});

try {
  // ...
} catch (error) {
  logger.log(error, { userId, operation: 'createUser' });
}

```

Метрики ошибок

```

javascript

class ErrorMetrics {
  constructor() {
    this.errors = new Map();
    this.startTime = Date.now();
  }

  track(error, category = 'default') {
    if (!this.errors.has(category)) {

```

```

    this.errors.set(category, {
      count: 0,
      byType: new Map(),
      byMessage: new Map(),
      lastOccurrence: null
    });
  }

  const cat = this.errors.get(category);
  cat.count++;
  cat.lastOccurrence = new Date();

  // По типу ошибки
  const type = error.name;
  if (!cat.byType.has(type)) {
    cat.byType.set(type, 0);
  }
  cat.byType.set(type, cat.byType.get(type) + 1);

  // По сообщению
  const message = error.message.substring(0, 50);
  if (!cat.byMessage.has(message)) {
    cat.byMessage.set(message, 0);
  }
  cat.byMessage.set(message, cat.byMessage.get(message) + 1);
}

getReport() {
  const report = {
    uptime: Math.floor((Date.now() - this.startTime) / 1000),
    totalErrors: 0,
    categories: {}
  };

  this.errors.forEach((data, category) => {
    report.totalErrors += data.count;
    report.categories[category] = {
      count: data.count,
      types: Object.fromEntries(data.byType),
      lastOccurrence: data.lastOccurrence
    };
  });

  return report;
}

```



```

    }
  }

  const metrics = new ErrorMetrics();

  // Middleware для Express
  app.use((err, req, res, next) => {
    metrics.track(err, req.path);
    next(err);
  });

  // Endpoint для метрик
  app.get('/metrics', (req, res) => {
    res.json(metrics.getReport());
  });

```

Часть 6: Отраслевые стандарты и лучшие практики

6.1. Принципы обработки ошибок

```

javascript

const errorHandlingPrinciples = {
  // 1. Fail Fast - ошибка должна быть обнаружена как можно раньше
  failFast: {
    bad: `function process(data) {
      // Ошибка проявится через 100 строк кода
    }`,
    good: `function process(data) {
      if (!isValid(data)) {
        throw new ValidationError('Invalid data');
      }
      // Логика
    }`,
  },

  // 2. Не проглатывайте ошибки
  dontSwallow: {
    bad: `try {
      risky();
    } catch (e) {
      // Ничего не делаем
    }`
  }
};

```

```

    }`,
    good: `try {
      risky();
    } catch (e) {
      console.error('Operation failed:', e);
      throw e; // Пробрасываем
    }`
  },

  // 3. Используйте конкретные типы ошибок
  specificTypes: {
    bad: `throw new Error('User not found');`,
    good: `throw new NotFoundError('User', userId);`
  },

  // 4. Добавляйте контекст
  addContext: {
    bad: `throw error;`,
    good: `throw new ContextualError('Failed to save user', { userId, error });`
  },

  // 5. Обрабатывайте ошибки на правильном уровне
  rightLevel: {
    bad: `// Обработка в каждом методе
    class Service {
      method1() { try { ... } catch {...} }
      method2() { try { ... } catch {...} }
    }`,
    good: `// Единый обработчик на верхнем уровне
    app.use(errorHandler);
    class Service {
      method1() { ... }
      method2() { ... }
    }`
  }
};

```

6.2. Паттерны для разных слоев приложения

API слой

javascript

```

class APIErrorHandler {
  static handle(error, req, res, next) {
    // Определяем статус код
    const status = error.statusCode || error.status || 500;

    // Формируем ответ
    const response = {
      success: false,
      error: {
        code: error.code || status,
        message: error.message || 'Internal server error',
        type: error.name
      }
    };

    // Добавляем детали валидации
    if (error.errors) {
      response.error.details = error.errors;
    }

    // Добавляем стек в development
    if (process.env.NODE_ENV === 'development') {
      response.error.stack = error.stack;
      response.error.context = error.context;
    }

    res.status(status).json(response);
  }

  static notFound(req, res) {
    res.status(404).json({
      success: false,
      error: {
        code: 404,
        message: `Route ${req.method} ${req.path} not found`,
        type: 'NotFoundError'
      }
    });
  }
}

// Использование в Express
app.use(APIErrorHandler.handle);
app.use(APIErrorHandler.notFound);

```

Сервисный слой

javascript

```
class ServiceErrorHandler {
  static async withTransaction(fn, context) {
    const transaction = await database.startTransaction();

    try {
      const result = await fn(transaction);
      await transaction.commit();
      return Result.ok(result);
    } catch (error) {
      await transaction.rollback();

      if (error instanceof DatabaseError) {
        return Result.fail(error);
      }

      // Логируем неожиданную ошибку
      logger.error('Unexpected error in transaction', {
        error,
        context
      });

      return Result.fail(new InternalServerError());
    }
  }

  static withRetry(fn, options = {}) {
    const retry = new RetryStrategy(options);
    return retry.execute(fn);
  }
}

// Использование
class UserService {
  async createUser(userData) {
    return ServiceErrorHandler.withTransaction(async (tx) => {
      const user = await tx.users.insert(userData);
      await tx.audit.log('user.created', { userId: user.id });
      return user;
    }, { operation: 'createUser', userData });
  }
}
```

```

async fetchUser(id) {
  return ServiceErrorHandler.withRetry(
    () => database.users.findById(id),
    { maxAttempts: 3, delay: 100 }
  );
}
}

```

UI слой

javascript

```

class UIErrorHandler {
  static getUserFriendlyMessage(error) {
    const messages = {
      'NetworkError': 'Проверьте подключение к интернету',
      'TimeoutError': 'Сервер не отвечает. Попробуйте позже',
      'ValidationError': 'Проверьте введенные данные',
      'AuthenticationError': 'Требуется авторизация',
      'NotFoundError': 'Информация не найдена',
      'DatabaseError': 'Технические неполадки',
      'default': 'Что-то пошло не так'
    };

    return messages[error.name] || messages.default;
  }

  static showError(error, options = {}) {
    const message = options.message || this.getUserFriendlyMessage(error);
    const duration = options.duration || 5000;

    // Показываем тостер/уведомление
    toast.show(message, {
      type: 'error',
      duration,
      action: options.action
    });

    // Отправляем в аналитику
    analytics.track('error_displayed', {
      error: error.name,
      message: error.message
    });
  }
}

```

```

static async wrapAsync(fn, errorOptions = {}) {
  try {
    return await fn();
  } catch (error) {
    this.showError(error, errorOptions);
    throw error; // Пробрасываем для локальной обработки
  }
}
}

```

// React компонент

```

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(false);

  const loadUser = useCallback(async () => {
    setLoading(true);

    await UIErrorHandler.wrapAsync(async () => {
      const data = await api.fetchUser(userId);
      setUser(data);
    }, {
      action: {
        label: 'Повторить',
        handler: () => loadUser()
      }
    });

    setLoading(false);
  }, [userId]);

  useEffect(() => {
    loadUser();
  }, [loadUser]);

  if (!user && !loading) {
    return <div>Пользователь не найден</div>;
  }

  return <div>{user?.name}</div>;
}

```

Часть 7: Тестирование обработки ошибок

7.1. Unit тесты для ошибок

```
javascript

import { expect } from 'chai';
import { ValidationError, NotFoundError } from './errors.js';

describe('ValidationError', () => {
  it('should create error with correct name', () => {
    const error = new ValidationError('Test error');
    expect(error.name).to.equal('ValidationError');
  });

  it('should have status code 400', () => {
    const error = new ValidationError('Test error');
    expect(error.statusCode).to.equal(400);
  });

  it('should capture validation details', () => {
    const details = [{ field: 'email', message: 'Invalid format' }];
    const error = new ValidationError(details);
    expect(error.errors).to.deep.equal(details);
  });
});

describe('UserService', () => {
  let service;

  beforeEach(() => {
    service = new UserService();
  });

  it('should throw ValidationError for invalid email', async () => {
    const invalidUser = { email: 'invalid', age: 25 };

    try {
      await service.createUser(invalidUser);
      expect.fail('Should throw error');
    } catch (error) {
      expect(error).to.be.instanceOf(ValidationError);
      expect(error.field).to.equal('email');
    }
  });
});
```

```

    }
  });

it('should throw NotFoundError for non-existent user', async () => {
  try {
    await service.getUser(999999);
    expect.fail('Should throw error');
  } catch (error) {
    expect(error).toBeInstanceOf(NotFoundError);
    expect(error.resourceId).toEqual(999999);
  }
});

it('should handle database error gracefully', async () => {
  const mockDb = sinon.stub(database, 'query')
    .rejects(new DatabaseError('Connection failed'));

  const result = await service.getUser(1).catch(e => e);

  expect(result).toBeInstanceOf(DatabaseError);
  expect(result.message).toContain('Connection failed');

  mockDb.restore();
});
});

```

7.2. Интеграционное тестирование

```

javascript

describe('API Error Handling', () => {
  let app;

  beforeEach(() => {
    app = express();
    app.use('/api', routes);
    app.use(APIErrorHandler.handle);
  });

  it('should return 400 for validation error', async () => {
    const response = await request(app)
      .post('/api/users')
      .send({ email: 'invalid' })
      .expect(400);
  });
}

```



```

expect(response.body).to.deep.equal({
  success: false,
  error: {
    code: 400,
    message: 'Validation failed',
    type: 'ValidationError',
    details: [{ field: 'email', message: 'Invalid format' }]
  }
});
});

it('should return 404 for non-existent route', async () => {
  const response = await request(app)
    .get('/api/non-existent')
    .expect(404);

  expect(response.body.error.type).to.equal('NotFoundError');
});

it('should handle server errors with 500', async () => {
  // Принудительно вызываем ошибку
  sinon.stub(UserService.prototype, 'getUser')
    .throws(new Error('Unexpected'));

  const response = await request(app)
    .get('/api/users/1')
    .expect(500);

  expect(response.body.error.type).to.equal('Error');
  expect(response.body.error.message).to.equal('Internal server error');

  sinon.restore();
});
});

```

Заключение

Ключевые принципы обработки ошибок в JavaScript

1. **Будьте явными:** Используйте конкретные типы ошибок, добавляйте контекст
2. **Не проглатывайте:** Всегда обрабатывайте или пробрасывайте ошибки

3. **Fail Fast:** Проверяйте предусловия на ранних этапах
4. **Разделяйте уровни:** Ошибки базы данных не должны утекать в UI
5. **Документируйте:** JSDoc комментарии для выбрасываемых ошибок
6. **Тестируйте:** Пишите тесты для сценариев с ошибками
7. **Мониторьте:** Логируйте и отслеживайте ошибки в production

Чек-лист для production

```
javascript

const productionErrorChecklist = {
  // 🟢 Все ошибки логируются
  logging: true,

  // 🟢 Пользователь видит понятные сообщения
  userFriendlyMessages: true,

  // 🟢 Стек ошибок не виден пользователям
  stackTracesHidden: true,

  // 🟢 Настроены глобальные обработчики
  unhandledRejectionHandler: true,
  uncaughtExceptionHandler: true,

  // 🟢 Есть система мониторинга
  errorTracking: 'Sentry/LogRocket/etc',

  // 🟢 Настроены алерты на критические ошибки
  alerts: 'Slack/Email/PagerDuty',

  // 🟢 Есть graceful shutdown
  gracefulShutdown: true,

  // 🟢 Написаны тесты для основных сценариев ошибок
  errorTests: true,

  // 🟢 Документация API включает описание ошибок
  apiDocumentation: true
};
```

Будущее обработки ошибок в JavaScript

javascript

```
// Pattern Matching (Stage 1)
try {
  await riskyOperation();
} match {
  when { error instanceof ValidationError } -> {
    console.log('Validation failed:', error.errors);
  }
  when { error instanceof NotFoundError } -> {
    console.log('Resource not found');
  }
  when { error } -> {
    console.log('Unknown error:', error);
  }
}
```

```
// Error Cause (ES2022)
throw new Error('Failed to process order', {
  cause: new DatabaseError('Connection failed')
});
```

```
// Promise.try (Stage 2)
const result = await Promise.try(() => {
  throw new Error('Sync error');
}).catch(error => {
  console.log('Caught:', error.message);
});
```

```
// Decorators (Stage 3)
@catchErrors(ErrorLogger)
class API {
  @retry({ attempts: 3, delay: 1000 })
  async fetchData() {
    // ...
  }
}
```

Обработка ошибок — это не просто техническая необходимость, а важная часть дизайна программного обеспечения. Хорошо продуманная система обработки ошибок делает приложение надежным, предсказуемым и удобным для пользователей и разработчиков. Инвестируйте время в проектирование обработки ошибок на ранних этапах разработки — это окупится многократно в процессе эксплуатации.

13.4. Пользовательские ошибки и их создание

Введение: Зачем нужны пользовательские ошибки?

Встроенные классы ошибок JavaScript (`Error`, `TypeError`, `RangeError` и т.д.) покрывают базовые потребности, но в реальных приложениях их недостаточно. Пользовательские ошибки — это фундаментальный инструмент для создания выразительной, самодокументируемой и надежной системы обработки исключений.

Проблемы использования встроенных ошибок

```
javascript

// ✗ ПЛОХО: Использование общих ошибок
function findUser(id) {
  if (!id) {
    throw new Error('ID is required'); // Какой это тип ошибки?
  }

  const user = database.find(id);
  if (!user) {
    throw new Error('User not found'); // 404 или 500?
  }

  if (user.isBlocked) {
    throw new Error('User is blocked'); // 403 или 400?
  }
}

// ✗ Нельзя различить типы ошибок
try {
  findUser(-5);
} catch (error) {
  if (error.message.includes('not found')) {
    // Хрупкая проверка
  }
}

// ✅ ХОРОШО: Пользовательские ошибки
```

```

function findUser(id) {
  if (!id) throw new ValidationError('ID is required');
  if (id <= 0) throw new ValidationError('ID must be positive');

  const user = database.find(id);
  if (!user) throw new NotFoundError('User', id);

  if (user.isBlocked) throw new ForbiddenError('User is blocked');
}

try {
  findUser(-5);
} catch (error) {
  if (error instanceof ValidationError) {
    // Понятная обработка
  }
}

```

Преимущества пользовательских ошибок

javascript

```

const benefitsOfCustomErrors = {
  // 1. Семантическая ясность
  semantic: 'Имя класса говорит о типе проблемы (NotFoundError, AuthError)',

  // 2. Дополнительные свойства
  properties: 'Можно добавлять statusCode, field, timestamp, context',

  // 3. Иерархия и полиморфизм
  hierarchy: 'instanceof работает с родительскими классами',

  // 4. Централизованная обработка
  centralized: 'Единый обработчик для разных типов ошибок',

  // 5. Самодокументирование
  selfDocumenting: 'API явно показывает какие ошибки может выбрасывать',

  // 6. Интернационализация
  i18n: 'Коды ошибок вместо текста для локализации',

  // 7. Мониторинг и аналитика
  monitoring: 'Группировка по типам в системах логирования'
};

```

Часть 1: Фундаментальные принципы создания пользовательских ошибок

1.1. Наследование от Error

Базовый шаблон пользовательской ошибки

```
javascript

class CustomError extends Error {
  constructor(message) {
    // 1. Вызов родительского конструктора
    super(message);

    // 2. Установка имени прототипа (важно для instanceof)
    this.name = this.constructor.name;

    // 3. Корректировка стека вызовов
    if (Error.captureStackTrace) {
      Error.captureStackTrace(this, this.constructor);
    }

    // 4. Фиксация времени
    this.timestamp = new Date().toISOString();
  }
}

// Проверка
const error = new CustomError('Что-то пошло не так');
console.log(error.name);           // "CustomError"
console.log(error.message);        // "Что-то пошло не так"
console.log(error.timestamp);       // "2026-02-11T10:30:45.123Z"
console.log(error.stack);           // Стек вызовов
console.log(error instanceof CustomError); // true
console.log(error instanceof Error);   // true
```

Проблема цепочки прототипов

```
javascript

// ⚠ ВАЖНО: Правильное наследование в ES5 vs ES6

// ES6 - работает корректно
```

```

class ES6CustomError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ES6CustomError';
  }
}

const es6Error = new ES6CustomError('test');
console.log(es6Error instanceof Error); // true
console.log(es6Error instanceof ES6CustomError); // true

// ES5 - проблема с instanceof
function ES5CustomError(message) {
  Error.call(this, message);
  this.name = 'ES5CustomError';
}
ES5CustomError.prototype = Object.create(Error.prototype);
ES5CustomError.prototype.constructor = ES5CustomError;

const es5Error = new ES5CustomError('test');
console.log(es5Error instanceof Error); // false!
console.log(es5Error instanceof ES5CustomError); // true

// Решение для ES5
function ES5FixedError(message) {
  const instance = new Error(message);
  instance.name = 'ES5FixedError';
  Object.setPrototypeOf(instance, ES5FixedError.prototype);
  return instance;
}
Object.setPrototypeOf(ES5FixedError.prototype, Error.prototype);

```

1.2. Расширение функциональности

Добавление кода ошибки и статуса HTTP

javascript

```

class HttpError extends Error {
  constructor(message, statusCode = 500) {
    super(message);
    this.name = 'HttpError';
    this.statusCode = statusCode;
  }
}

```

```
// Код ошибки для клиента
this.code = `ERR_${statusCode}`;

Error.captureStackTrace(this, this.constructor);
}

toJSON() {
  return {
    error: {
      name: this.name,
      code: this.code,
      message: this.message,
      statusCode: this.statusCode
    }
  };
}
}

class BadRequestError extends HttpError {
  constructor(message = 'Bad request') {
    super(message, 400);
    this.name = 'BadRequestError';
    this.code = 'ERR_BAD_REQUEST';
  }
}

class UnauthorizedError extends HttpError {
  constructor(message = 'Unauthorized') {
    super(message, 401);
    this.name = 'UnauthorizedError';
    this.code = 'ERR_UNAUTHORIZED';
  }
}

class ForbiddenError extends HttpError {
  constructor(message = 'Forbidden') {
    super(message, 403);
    this.name = 'ForbiddenError';
    this.code = 'ERR_FORBIDDEN';
  }
}

class NotFoundError extends HttpError {
```



```

constructor(resource, id) {
  const message = resource && id
    ? `${resource} with id ${id} not found`
    : 'Resource not found';
  super(message, 404);
  this.name = 'NotFoundError';
  this.code = 'ERR_NOT_FOUND';
  this.resource = resource;
  this.resourceId = id;
}
}

```

```

class ConflictError extends HttpError {
  constructor(message = 'Resource conflict') {
    super(message, 409);
    this.name = 'ConflictError';
    this.code = 'ERR_CONFLICT';
  }
}

```

```

class InternalServerError extends HttpError {
  constructor(message = 'Internal server error') {
    super(message, 500);
    this.name = 'InternalServerError';
    this.code = 'ERR_INTERNAL';
  }
}

```

// Использование

```

app.get('/users/:id', (req, res) => {
  try {
    const user = database.find(req.params.id);
    if (!user) {
      throw new NotFoundError('User', req.params.id);
    }
    res.json(user);
  } catch (error) {
    if (error instanceof HttpError) {
      res.status(error.statusCode).json(error);
    } else {
      const internalError = new InternalServerError();
      res.status(500).json(internalError);
    }
  }
}
}

```

```
});
```

Добавление контекста выполнения

javascript

```
class ContextualError extends Error {
  constructor(message, context = {}) {
    super(message);
    this.name = 'ContextualError';
    this.context = context;
    this.timestamp = new Date().toISOString();
    this.id = generateErrorId();

    // Добавляем информацию о системе
    this.system = {
      nodeVersion: process.version,
      platform: process.platform,
      memory: process.memoryUsage(),
      uptime: process.uptime()
    };

    if (context.originalError) {
      this.originalError = {
        name: context.originalError.name,
        message: context.originalError.message,
        stack: context.originalError.stack
      };
      delete context.originalError;
    }

    Error.captureStackTrace(this, this.constructor);
  }

  addContext(key, value) {
    this.context[key] = value;
    return this;
  }

  toJSON() {
    return {
      id: this.id,
      name: this.name,
      message: this.message,
      context: this.context,
    };
  }
}
```

```

    timestamp: this.timestamp,
    stack: this.stack.split('\n')
  };
}
}

function generateErrorId() {
  return `ERR_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
}

// Использование
async function processPayment(orderId, paymentDetails) {
  try {
    const order = await Order.findById(orderId);
    const result = await paymentGateway.charge(order.total, paymentDetails);
    return result;
  } catch (error) {
    throw new ContextualError('Payment processing failed', {
      orderId,
      paymentMethod: paymentDetails.method,
      amount: order?.total,
      originalError: error
    }).addContext('timestamp', Date.now());
  }
}

```

Часть 2: Иерархия пользовательских ошибок

2.1. Многоуровневая иерархия

```

javascript

// =====
// Базовый уровень: Корневая ошибка приложения
// =====

class AppError extends Error {
  constructor(message, options = {}) {
    super(message);
    this.name = 'AppError';
    this.cause = options.cause;
    this.context = options.context || {};
    this.code = options.code || 'ERR_UNKNOWN';
  }
}

```

```

    this.severity = options.severity || 'error'; // error, warning, info
    this.timestamp = new Date().toISOString();
    this.id = crypto.randomUUID?().() || Date.now().toString();

    Error.captureStackTrace(this, this.constructor);
}

isOperational() {
    // Операционные ошибки (ожидаемые) vs программные (баги)
    return true;
}

toLog() {
    return {
        id: this.id,
        name: this.name,
        message: this.message,
        code: this.code,
        severity: this.severity,
        timestamp: this.timestamp,
        context: this.context
    };
}
}

// =====
// Второй уровень: Категории ошибок
// =====

// 1. Ошибки валидации
class ValidationError extends AppError {
    constructor(message, options = {}) {
        super(message, {
            code: 'ERR_VALIDATION',
            severity: 'warning',
            ...options
        });
        this.name = 'ValidationError';
        this.fields = options.fields || [];
        this.invalidValue = options.invalidValue;
    }

    addField(field, message) {
        this.fields.push({ field, message });
    }
}

```

```
    return this;
  }
}
```

// 2. Ошибки бизнес-логики

```
class BusinessError extends AppError {
  constructor(message, options = {}) {
    super(message, {
      code: 'ERR_BUSINESS',
      severity: 'error',
      ...options
    });
    this.name = 'BusinessError';
    this.operation = options.operation;
    this.entity = options.entity;
  }
}
```

// 3. Ошибки доступа

```
class AccessError extends AppError {
  constructor(message, options = {}) {
    super(message, {
      code: 'ERR_ACCESS',
      severity: 'warning',
      ...options
    });
    this.name = 'AccessError';
    this.requiredRole = options.requiredRole;
    this.userRole = options.userRole;
    this.userId = options.userId;
  }
}
```

// 4. Ошибки инфраструктуры

```
class InfrastructureError extends AppError {
  constructor(message, options = {}) {
    super(message, {
      code: 'ERR_INFRASTRUCTURE',
      severity: 'critical',
      ...options
    });
    this.name = 'InfrastructureError';
    this.service = options.service;
    this.endpoint = options.endpoint;
  }
}
```

```

    this.retryable = options.retryable ?? true;
}

isOperational() {
    return this.retryable;
}
}

// =====
// Третий уровень: Конкретные ошибки
// =====

// Ошибки валидации
class RequiredFieldError extends ValidationError {
    constructor(fieldName, options = {}) {
        super(`Field "${fieldName}" is required`, {
            code: 'ERR_REQUIRED_FIELD',
            ...options
        });
        this.name = 'RequiredFieldError';
        this.field = fieldName;
        this.addField(fieldName, 'This field is required');
    }
}

class InvalidFormatError extends ValidationError {
    constructor(fieldName, format, options = {}) {
        super(`Field "${fieldName}" has invalid format`, {
            code: 'ERR_INVALID_FORMAT',
            ...options
        });
        this.name = 'InvalidFormatError';
        this.field = fieldName;
        this.expectedFormat = format;
        this.addField(fieldName, `Expected format: ${format}`);
    }
}

class RangeError extends ValidationError {
    constructor(fieldName, min, max, options = {}) {
        super(`Field "${fieldName}" is out of range`, {
            code: 'ERR_OUT_OF_RANGE',
            ...options
        });
    }
}

```

```

    this.name = 'RangeError';
    this.field = fieldName;
    this.min = min;
    this.max = max;
    this.addField(fieldName, `Must be between ${min} and ${max}`);
  }
}

```

// Ошибки бизнес-логики

```

class InsufficientFundsError extends BusinessError {
  constructor(balance, required, options = {}) {
    super('Insufficient funds for transaction', {
      code: 'ERR_INSUFFICIENT_FUNDS',
      ...options
    });
    this.name = 'InsufficientFundsError';
    this.balance = balance;
    this.required = required;
    this.shortfall = required - balance;
    this.operation = 'payment';
    this.entity = 'account';
  }
}

```

```

class DuplicateResourceError extends BusinessError {
  constructor(resourceType, criteria, options = {}) {
    super(`${resourceType} already exists`, {
      code: 'ERR_DUPLICATE',
      ...options
    });
    this.name = 'DuplicateResourceError';
    this.resourceType = resourceType;
    this.criteria = criteria;
    this.operation = 'create';
  }
}

```

```

class ResourceLockedError extends BusinessError {
  constructor(resourceType, resourceId, lockedBy, options = {}) {
    super(`${resourceType} is locked by another process`, {
      code: 'ERR_RESOURCE_LOCKED',
      ...options
    });
    this.name = 'ResourceLockedError';
  }
}

```

```

    this.resourceType = resourceType;
    this.resourceId = resourceId;
    this.lockedBy = lockedBy;
  }
}

// Ошибки доступа
class AuthenticationError extends AccessError {
  constructor(options = {}) {
    super('Authentication failed', {
      code: 'ERR_AUTH_FAILED',
      ...options
    });
    this.name = 'AuthenticationError';
    this.requiredRole = 'authenticated';
  }
}

class AuthorizationError extends AccessError {
  constructor(userId, requiredRole, options = {}) {
    super(`Access denied. Required role: ${requiredRole}`, {
      code: 'ERR_FORBIDDEN',
      ...options
    });
    this.name = 'AuthorizationError';
    this.userId = userId;
    this.requiredRole = requiredRole;
    this.userRole = options.userRole;
  }
}

class RateLimitError extends AccessError {
  constructor(userId, limit, resetTime, options = {}) {
    super('Rate limit exceeded', {
      code: 'ERR_RATE_LIMIT',
      severity: 'warning',
      ...options
    });
    this.name = 'RateLimitError';
    this.userId = userId;
    this.limit = limit;
    this.resetTime = resetTime;
    this.retryAfter = Math.ceil((resetTime - Date.now()) / 1000);
  }
}

```



```
}
```

```
// Ошибки инфраструктуры
```

```
class DatabaseError extends InfrastructureError {  
  constructor(operation, query, options = {}) {  
    super('Database operation failed', {  
      code: 'ERR_DATABASE',  
      service: 'database',  
      ...options  
    });  
    this.name = 'DatabaseError';  
    this.operation = operation;  
    this.query = query;  
  }  
}
```

```
class NetworkError extends InfrastructureError {  
  constructor(endpoint, options = {}) {  
    super('Network request failed', {  
      code: 'ERR_NETWORK',  
      service: 'http',  
      endpoint,  
      retryable: true,  
      ...options  
    });  
    this.name = 'NetworkError';  
    this.statusCode = options.statusCode;  
    this.response = options.response;  
  }  
}
```

```
class TimeoutError extends InfrastructureError {  
  constructor(operation, timeoutMs, options = {}) {  
    super('Operation timed out', {  
      code: 'ERR_TIMEOUT',  
      severity: 'error',  
      retryable: true,  
      ...options  
    });  
    this.name = 'TimeoutError';  
    this.operation = operation;  
    this.timeoutMs = timeoutMs;  
  }  
}
```

```

class ConfigurationError extends InfrastructureError {
  constructor(configKey, options = {}) {
    super(`Configuration error: ${configKey}`, {
      code: 'ERR_CONFIG',
      severity: 'critical',
      retryable: false,
      ...options
    });
    this.name = 'ConfigurationError';
    this.configKey = configKey;
  }
}

```

2.2. Использование иерархии

javascript

// Пример использования иерархии в реальном приложении

```

class UserService {
  async createUser(userData) {
    try {
      // Валидация
      if (!userData.email) {
        throw new RequiredFieldError('email');
      }

      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      if (!emailRegex.test(userData.email)) {
        throw new InvalidFormatError('email', 'user@example.com');
      }

      if (userData.age && (userData.age < 13 || userData.age > 150)) {
        throw new RangeError('age', 13, 150);
      }

      // Проверка дубликатов
      const existing = await this.findByEmail(userData.email);
      if (existing) {
        throw new DuplicateResourceError('User', { email: userData.email });
      }

      // Бизнес-логика

```

```

if (userData.role === 'admin' && !this.currentUser?.isAdmin) {
  throw new AuthorizationError(
    this.currentUser?.id,
    'admin',
    { userRole: this.currentUser?.role }
  );
}

// Сохранение в БД
const user = await this.userRepository.create(userData);
return user;

} catch (error) {
  // Трансформация ошибок базы данных
  if (error.code === 'ER_DUP_ENTRY') {
    throw new DuplicateResourceError('User', { email: userData.email }, { cause: error
  });
}

  if (error.code === 'ER_LOCK_DEADLOCK') {
    throw new ResourceLockedError('User', null, 'transaction', { cause: error });
  }

  if (error.code === 'ETIMEDOUT') {
    throw new TimeoutError('UserRepository.create', 5000, { cause: error });
  }

  // Пробрасываем уже наши ошибки
  if (error instanceof AppError) {
    throw error;
  }

  // Неизвестные ошибки
  throw new InternalServerError('Failed to create user', { cause: error });
}

}

async authenticate(email, password) {
  try {
    const user = await this.findByEmail(email);

    if (!user) {
      throw new AuthenticationError();
    }
  }

```

```

const isValid = await bcrypt.compare(password, user.passwordHash);

if (!isValid) {
  throw new AuthenticationError();
}

// Проверка блокировки
if (user.isBlocked) {
  throw new ForbiddenError('Account is blocked');
}

// Проверка rate limit
const attempts = await this.getLoginAttempts(email);
if (attempts >= 5) {
  const resetTime = Date.now() + 15 * 60 * 1000;
  throw new RateLimitError(email, 5, resetTime);
}

return user;

} catch (error) {
  if (error instanceof AccessError) {
    await this.incrementLoginAttempts(email);
    throw error;
  }
  throw new AuthenticationError({ cause: error });
}
}

// Централизованная обработка ошибок
function errorHandler(error, req, res, next) {
  // Логирование
  console.error({
    id: error.id,
    name: error.name,
    message: error.message,
    code: error.code,
    severity: error.severity,
    timestamp: error.timestamp,
    context: error.context,
    stack: error.stack
  });
}

```

// HTTP omβem

```
if (error instanceof ValidationError) {
  return res.status(400).json({
    success: false,
    error: {
      code: error.code,
      message: error.message,
      fields: error.fields
    }
  });
}

if (error instanceof AuthenticationError) {
  return res.status(401).json({
    success: false,
    error: {
      code: error.code,
      message: 'Authentication failed'
    }
  });
}

if (error instanceof AuthorizationError) {
  return res.status(403).json({
    success: false,
    error: {
      code: error.code,
      message: 'Access denied',
      requiredRole: error.requiredRole
    }
  });
}

if (error instanceof NotFoundError) {
  return res.status(404).json({
    success: false,
    error: {
      code: error.code,
      message: error.message,
      resource: error.resource,
      id: error.resourceId
    }
  });
}
```

```
}
```

```
if (error instanceof DuplicateResourceError) {  
  return res.status(409).json({  
    success: false,  
    error: {  
      code: error.code,  
      message: error.message,  
      resource: error.resourceType  
    }  
  });  
}
```

```
if (error instanceof RateLimitError) {  
  res.set('Retry-After', error.retryAfter);  
  return res.status(429).json({  
    success: false,  
    error: {  
      code: error.code,  
      message: error.message,  
      retryAfter: error.retryAfter,  
      limit: error.limit  
    }  
  });  
}
```

```
if (error instanceof TimeoutError) {  
  return res.status(504).json({  
    success: false,  
    error: {  
      code: error.code,  
      message: 'Service timeout',  
      operation: error.operation  
    }  
  });  
}
```

```
if (error instanceof InfrastructureError) {  
  // Не показываем детали инфраструктурных ошибок клиенту  
  return res.status(503).json({  
    success: false,  
    error: {  
      code: 'ERR_SERVICE_UNAVAILABLE',  
      message: 'Service temporarily unavailable',  
    }  
  });  
}
```

```

    retryable: error.retryable
  }
});
}

if (error instanceof AppError) {
  return res.status(500).json({
    success: false,
    error: {
      code: error.code,
      message: 'Internal server error'
    }
  });
}

// Необработанные ошибки
console.error('Unhandled error:', error);
return res.status(500).json({
  success: false,
  error: {
    code: 'ERR_UNKNOWN',
    message: 'Internal server error'
  }
});
}

// Использование в Express
app.use(errorHandler);

```

Часть 3: Продвинутые паттерны создания ошибок

3.1. Фабрики ошибок

```

javascript

// =====
// Error Factory Pattern
// =====

class ErrorFactory {
  constructor(errorRegistry) {
    this.registry = errorRegistry;
  }
}

```

```

    this.cache = new Map();
}

static create() {
    const factory = new ErrorFactory(new Map());

    // Регистрация ошибок по умолчанию
    factory.register('validation', (message, options) =>
        new ValidationError(message, options));
    factory.register('required', (field, options) =>
        new RequiredFieldError(field, options));
    factory.register('format', (field, format, options) =>
        new InvalidFormatError(field, format, options));
    factory.register('range', (field, min, max, options) =>
        new RangeError(field, min, max, options));
    factory.register('notFound', (resource, id, options) =>
        new NotFoundError(resource, id, options));
    factory.register('duplicate', (resource, criteria, options) =>
        new DuplicateResourceError(resource, criteria, options));
    factory.register('auth', (options) =>
        new AuthenticationError(options));
    factory.register('forbidden', (userId, role, options) =>
        new AuthorizationError(userId, role, options));
    factory.register('rateLimit', (userId, limit, resetTime, options) =>
        new RateLimitError(userId, limit, resetTime, options));
    factory.register('database', (operation, query, options) =>
        new DatabaseError(operation, query, options));
    factory.register('timeout', (operation, timeout, options) =>
        new TimeoutError(operation, timeout, options));

    return factory;
}

register(type, constructor) {
    this.registry.set(type, constructor);
}

create(type, ...args) {
    const constructor = this.registry.get(type);
    if (!constructor) {
        throw new Error(`Unknown error type: ${type}`);
    }

    // Кэширование экземпляров для неизменяемых ошибок

```



```

const cacheKey = `${type}_${JSON.stringify(args)}`;
if (this.cache.has(cacheKey)) {
  return this.cache.get(cacheKey);
}

const error = constructor(...args);
this.cache.set(cacheKey, error);

return error;
}

createFromStatusCode(statusCode, message) {
  switch (statusCode) {
    case 400: return this.create('validation', message);
    case 401: return this.create('auth');
    case 403: return this.create('forbidden', null, null, { message });
    case 404: return this.create('notFound', null, null, { message });
    case 409: return this.create('duplicate', null, null, { message });
    case 429: return this.create('rateLimit', null, 5, Date.now() + 60000);
    case 500: return new InternalServerError(message);
    case 504: return this.create('timeout', 'operation', 30000);
    default: return new AppError(message, { code: `ERR_${statusCode}` });
  }
}

const errors = ErrorFactory.create();

// Использование
try {
  if (!email) {
    throw errors.create('required', 'email');
  }

  if (!isValidEmail(email)) {
    throw errors.create('format', 'email', 'user@example.com');
  }

  const user = await db.findUser(email);
  if (!user) {
    throw errors.create('notFound', 'User', email);
  }
} catch (error) {
  console.log(error.name); // RequiredFieldError

```

```
}
```

3.2. Строитель ошибок (Builder Pattern)

```
javascript

// =====
// Error Builder Pattern
// =====

class ErrorBuilder {
  constructor() {
    this.reset();
  }

  reset() {
    this._type = 'AppError';
    this._message = '';
    this._code = 'ERR_UNKNOWN';
    this._severity = 'error';
    this._statusCode = 500;
    this._context = {};
    this._fields = [];
    this._cause = null;
    this._retryable = false;
    return this;
  }

  type(type) {
    this._type = type;
    return this;
  }

  message(message) {
    this._message = message;
    return this;
  }

  code(code) {
    this._code = code;
    return this;
  }

  severity(severity) {
```

```
    this._severity = severity;
    return this;
}
```

```
statusCode(code) {
    this._statusCode = code;
    return this;
}
```

```
context(key, value) {
    this._context[key] = value;
    return this;
}
```

```
addField(field, message) {
    this._fields.push({ field, message });
    return this;
}
```

```
cause(error) {
    this._cause = error;
    return this;
}
```

```
retryable(retryable = true) {
    this._retryable = retryable;
    return this;
}
```

```
build() {
    let error;
```

```
    // Создаем ошибку нужного типа
```

```
    switch (this._type) {
        case 'ValidationError':
            error = new ValidationError(this._message, {
                code: this._code,
                severity: this._severity,
                fields: this._fields,
                cause: this._cause
            });
            break;
```

```
        case 'NotFoundError':
```

```

    error = new NotFoundError(
        this._context.resource,
        this._context.resourceId,
        { cause: this._cause }
    );
    break;

case 'DatabaseError':
    error = new DatabaseError(
        this._context.operation,
        this._context.query,
        { cause: this._cause, retryable: this._retryable }
    );
    break;

default:
    error = new AppError(this._message, {
        code: this._code,
        severity: this._severity,
        context: this._context,
        cause: this._cause
    });
}

// Добавляем статус код для HTTP ошибок
if (error instanceof HttpError || this._statusCode) {
    error.statusCode = this._statusCode;
}

this.reset();
return error;
}

static validation() {
    const builder = new ErrorBuilder();
    return builder.type('ValidationError')
        .code('ERR_VALIDATION')
        .severity('warning')
        .statusCode(400);
}

static notFound() {
    const builder = new ErrorBuilder();
    return builder.type('NotFoundError')

```

```

        .code('ERR_NOT_FOUND')
        .severity('error')
        .statusCode(404);
    }

    static database() {
        const builder = new ErrorBuilder();
        return builder.type('DatabaseError')
            .code('ERR_DATABASE')
            .severity('critical')
            .statusCode(503);
    }
}

// Использование
const error = ErrorBuilder.validation()
    .message('User data is invalid')
    .context('userId', 123)
    .addField('email', 'Invalid format')
    .addField('age', 'Must be at least 13')
    .build();

console.log(error.name);           // ValidationError
console.log(error.fields);        // [{field: 'email', message: 'Invalid format'}, ...]
console.log(error.statusCode);    // 400

// В сервисе
class OrderService {
    async createOrder(orderData) {
        try {
            // Валидация
            if (!orderData.items?.length) {
                throw ErrorBuilder.validation()
                    .message('Order must have at least one item')
                    .addField('items', 'Cannot be empty')
                    .build();
            }

            // Проверка клиента
            const customer = await this.customerService.findById(orderData.customerId);
            if (!customer) {
                throw ErrorBuilder.notFound()
                    .message('Customer not found')
                    .context('resource', 'Customer')

```

```

        .context('resourceId', orderData.customerId)
        .build();
    }

    // Проверка кредита
    if (customer.credit < orderData.total) {
        throw new InsufficientFundsError(customer.credit, orderData.total);
    }

    // Сохранение
    const order = await this.orderRepository.create(orderData);
    return order;

} catch (error) {
    if (error.code === 'ER_LOCK_DEADLOCK') {
        throw ErrorBuilder.database()
            .message('Database deadlock detected')
            .context('operation', 'createOrder')
            .context('query', 'INSERT INTO orders...')
            .retryable(true)
            .cause(error)
            .build();
    }
    throw error;
}
}
}

```

3.3. Декораторы ошибок

```

javascript

// =====
// Error Decorator Pattern
// =====

class ErrorDecorator {
    constructor(error) {
        this.error = error;
    }

    withTimestamp() {
        this.error.timestamp = new Date().toISOString();
        return this;
    }
}

```

```

}

withRequestId(requestId) {
  this.error.requestId = requestId;
  return this;
}

withUserId(userId) {
  this.error.userId = userId;
  return this;
}

withSessionId(sessionId) {
  this.error.sessionId = sessionId;
  return this;
}

withEnvironment(env) {
  this.error.environment = env;
  return this;
}

withVersion(version) {
  this.error.version = version;
  return this;
}

withTags(tags) {
  this.error.tags = { ...this.error.tags, ...tags };
  return this;
}

withMetadata(metadata) {
  this.error.metadata = { ...this.error.metadata, ...metadata };
  return this;
}

withSensitiveData(filtered = true) {
  // Фильтрация чувствительных данных
  if (this.error.context) {
    const sensitiveFields = ['password', 'token', 'secret', 'creditCard'];
    const filteredContext = { ...this.error.context };

    sensitiveFields.forEach(field => {

```

```

    if (filteredContext[field]) {
      filteredContext[field] = '[FILTERED]';
    }
  });

  this.error.context = filteredContext;
}
return this;
}

build() {
  return this.error;
}
}

// Middleware для обогащения ошибок
function enrichErrorMiddleware(req, res, next) {
  res.enrichError = (error) => {
    return new ErrorDecorator(error)
      .withRequestId(req.id)
      .withUserId(req.user?.id)
      .withSessionId(req.session?.id)
      .withEnvironment(process.env.NODE_ENV)
      .withVersion(require('../package.json').version)
      .withTags({ path: req.path, method: req.method })
      .withSensitiveData()
      .build();
  };

  next();
}

// Использование
app.post('/api/orders', enrichErrorMiddleware, async (req, res) => {
  try {
    const order = await orderService.createOrder(req.body);
    res.json(order);
  } catch (error) {
    const enrichedError = res.enrichError(error);
    errorHandler(enrichedError, req, res);
  }
});

```


3.4. Композиция ошибок

```
javascript

// =====
// Error Composition Pattern
// =====

class CompositeError extends AppError {
  constructor(message, options = {}) {
    super(message, options);
    this.name = 'CompositeError';
    this.errors = options.errors || [];
    this.successful = options.successful || [];
    this.failed = options.failed || [];
  }

  addError(error, context = {}) {
    this.errors.push({ error, context });
    this.failed.push(context);
    return this;
  }

  addSuccess(result, context = {}) {
    this.successful.push({ result, context });
    return this;
  }

  hasErrors() {
    return this.errors.length > 0;
  }

  getErrorSummary() {
    return {
      total: this.errors.length,
      byType: this.errors.reduce((acc, { error }) => {
        acc[error.name] = (acc[error.name] || 0) + 1;
        return acc;
      }, {}),
      byCode: this.errors.reduce((acc, { error }) => {
        if (error.code) {
          acc[error.code] = (acc[error.code] || 0) + 1;
        }
        return acc;
      })
    };
  }
}
```

```
    }, {}))
  };
}
}
```

// Использование для пакетных операций

```
class BatchProcessor {
  async processItems(items) {
    const compositeError = new CompositeError('Batch processing completed with errors');

    for (const item of items) {
      try {
        const result = await this.processSingle(item);
        compositeError.addSuccess(result, { itemId: item.id });
      } catch (error) {
        compositeError.addError(error, {
          itemId: item.id,
          itemData: item,
          timestamp: new Date().toISOString()
        });
      }
    }

    if (compositeError.hasErrors()) {
      throw compositeError;
    }

    return compositeError.successful;
  }

  async processSingle(item) {
    // Обработка одного элемента
  }
}
```

// Использование

```
const batchProcessor = new BatchProcessor();

try {
  const results = await batchProcessor.processItems([
    { id: 1, data: 'valid' },
    { id: 2, data: 'invalid' },
    { id: 3, data: 'valid' }
  ]);
}
```

```

} catch (error) {
  if (error instanceof CompositeError) {
    console.log(`Processed: ${error.successful.length} successful`);
    console.log(`Failed: ${error.errors.length} failed`);
    console.log('Error summary:', error.getErrorSummary());

    // Повторная обработка только упавших
    const failedItems = error.errors.map(e => e.context.itemData);
    await batchProcessor.processItems(failedItems);
  }
}

```

Часть 4: Интернационализация ошибок

4.1. Многоязычные ошибки

```

javascript

// =====
// I18n Error System
// =====

class I18nError extends AppError {
  constructor(key, options = {}) {
    super(key, options);
    this.name = 'I18nError';
    this.key = key;
    this.params = options.params || {};
    this.locale = options.locale || 'en';
  }

  translate(locale = this.locale) {
    return I18nError.translate(this.key, locale, this.params);
  }

  static translations = {
    en: {
      'error.required': '{field} is required',
      'error.email.invalid': 'Invalid email format',
      'error.range': '{field} must be between {min} and {max}',
      'error.notFound': '{resource} not found',
      'error.duplicate': '{resource} already exists',
    },
  }

```

```

    'error.auth.failed': 'Authentication failed',
    'error.forbidden': 'Access denied',
    'error.rateLimit': 'Too many requests. Try again in {seconds} seconds',
    'error.database': 'Database operation failed',
    'error.timeout': 'Operation timed out after {timeout}ms'
  },
  ru: {
    'error.required': 'Поле "{field}" обязательно',
    'error.email.invalid': 'Неверный формат email',
    'error.range': '{field} должно быть между {min} и {max}',
    'error.notFound': '{resource} не найден',
    'error.duplicate': '{resource} уже существует',
    'error.auth.failed': 'Ошибка аутентификации',
    'error.forbidden': 'Доступ запрещен',
    'error.rateLimit': 'Слишком много запросов. Попробуйте через {seconds} секунд',
    'error.database': 'Ошибка базы данных',
    'error.timeout': 'Операция превысила время ожидания {timeout}мс'
  },
  es: {
    'error.required': '{field} es obligatorio',
    'error.email.invalid': 'Formato de email inválido',
    'error.range': '{field} debe estar entre {min} y {max}',
    'error.notFound': '{resource} no encontrado',
    'error.duplicate': '{resource} ya existe',
    'error.auth.failed': 'Autenticación fallida',
    'error.forbidden': 'Acceso denegado',
    'error.rateLimit': 'Demasiadas solicitudes. Intente de nuevo en {seconds} segundos',
    'error.database': 'Error de base de datos',
    'error.timeout': 'Tiempo de espera agotado después de {timeout}ms'
  }
};

static translate(key, locale = 'en', params = {}) {
  const translation = this.translations[locale]?.[key] || this.translations.en[key] ||
key;

  return translation.replace(/{{(\w+)}}/g, (match, param) => {
    return params[param] !== undefined ? params[param] : match;
  });
}

static addTranslations(locale, translations) {
  if (!this.translations[locale]) {
    this.translations[locale] = {};
  }
}

```

```

    }
    Object.assign(this.translations[locale], translations);
  }
}

```

// Конкретные i18n ошибки

```

class I18nValidationError extends I18nError {
  constructor(field, options = {}) {
    super('error.required', {
      ...options,
      params: { field, ...options.params }
    });
    this.name = 'I18nValidationError';
    this.field = field;
  }
}

```

```

class I18nEmailError extends I18nError {
  constructor(options = {}) {
    super('error.email.invalid', options);
    this.name = 'I18nEmailError';
  }
}

```

```

class I18nRangeError extends I18nError {
  constructor(field, min, max, options = {}) {
    super('error.range', {
      ...options,
      params: { field, min, max, ...options.params }
    });
    this.name = 'I18nRangeError';
    this.field = field;
    this.min = min;
    this.max = max;
  }
}

```

```

class I18nNotFoundError extends I18nError {
  constructor(resource, id, options = {}) {
    super('error.notFound', {
      ...options,
      params: { resource, id, ...options.params }
    });
    this.name = 'I18nNotFoundError';
  }
}

```

```

    this.resource = resource;
    this.resourceId = id;
  }
}

// Middleware для локализации ошибок
function i18nErrorMiddleware(req, res, next) {
  // Определяем локаль из запроса
  const locale = req.acceptsLanguages(['en', 'ru', 'es']) || 'en';

  res.locals.locale = locale;

  // Перехватываем отправку ошибок
  const originalJson = res.json;
  res.json = function(data) {
    if (data.error && data.error.error instanceof I18nError) {
      const error = data.error.error;
      data.error.message = error.translate(locale);
      delete data.error.error;
    }
    return originalJson.call(this, data);
  };

  next();
}

// Использование
app.use(i18nErrorMiddleware);

app.post('/api/users', async (req, res) => {
  try {
    const { email, age } = req.body;

    if (!email) {
      throw new I18nValidationError('email');
    }

    if (!email.includes('@')) {
      throw new I18nEmailError();
    }

    if (age && (age < 13 || age > 150)) {
      throw new I18nRangeError('age', 13, 150);
    }
  }
}

```

```

const user = await userService.create(req.body);
res.json({ success: true, user });

} catch (error) {
  if (error instanceof I18nError) {
    res.status(400).json({
      success: false,
      error: {
        code: error.code || 'ERR_VALIDATION',
        error: error, // Будет преобразован middleware
        key: error.key,
        params: error.params
      }
    });
  } else {
    res.status(500).json({
      success: false,
      error: {
        code: 'ERR_INTERNAL',
        message: 'Internal server error'
      }
    });
  }
}
});

```

Часть 5: Интеграция с системами мониторинга

5.1. Структурированные ошибки для логирования

```

javascript

// =====
// Structured Error for Logging
// =====

class LoggableError extends AppError {
  constructor(message, options = {}) {
    super(message, options);
    this.name = 'LoggableError';
  }
}

```

// Структурированные поля для логирования

```
this.logFields = {
  severity: options.severity || 'error',
  category: options.category || 'application',
  component: options.component,
  operation: options.operation,
  userId: options.userId,
  sessionId: options.sessionId,
  requestId: options.requestId,
  duration: options.duration,
  metadata: options.metadata || {}
};

}

toLogEntry() {
  return {
    timestamp: this.timestamp,
    id: this.id,
    name: this.name,
    message: this.message,
    code: this.code,
    stack: this.stack,
    ...this.logFields,
    context: this.context
  };
}

toSentry() {
  return {
    fingerprint: [this.name, this.code].filter(Boolean),
    contexts: {
      error: {
        ...this.context,
        ...this.logFields
      }
    },
    tags: {
      severity: this.logFields.severity,
      category: this.logFields.category,
      component: this.logFields.component
    },
    extra: this.context
  };
}
```



```

toDatadog() {
  return {
    title: this.message,
    text: this.stack,
    alert_type: this.logFields.severity === 'critical' ? 'error' : 'warning',
    tags: [
      `component:${this.logFields.component}`,
      `operation:${this.logFields.operation}`,
      `code:${this.code}`
    ]
  };
}
}

```

// Интеграция с Winston

```

const winston = require('winston');

const errorLogger = winston.createLogger({
  level: 'error',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'error.log' }),
    new winston.transports.Console({
      format: winston.format.simple()
    })
  ]
});

```

// Middleware для логирования ошибок

```

async function loggingMiddleware(error, req, res, next) {
  if (error instanceof LoggableError) {
    // Добавляем контекст запроса
    error.logFields = {
      ...error.logFields,
      requestId: req.id,
      userId: req.user?.id,
      sessionId: req.session?.id,
      path: req.path,
      method: req.method,
      ip: req.ip,

```

```

    userAgent: req.get('user-agent')
  };

  // Логируем в зависимости от severity
  const logEntry = error.toLogEntry();

  switch (error.logFields.severity) {
    case 'critical':
      errorLogger.critical(logEntry);
      // Отправка в PagerDuty/OpsGenie
      await notifyOnCall(error);
      break;
    case 'error':
      errorLogger.error(logEntry);
      break;
    case 'warning':
      errorLogger.warn(logEntry);
      break;
    default:
      errorLogger.info(logEntry);
  }

  // Отправка в Sentry
  Sentry.captureException(error, error.toSentry());

  // Отправка в DataDog
  if (process.env.DATADOG_API_KEY) {
    await sendToDatadog(error.toDatadog());
  }
} else {
  // Неизвестная ошибка - логируем как critical
  errorLogger.critical({
    message: error.message,
    stack: error.stack,
    name: error.name,
    timestamp: new Date().toISOString()
  });
}

next(error);
}

async function notifyOnCall(error) {
  // Интеграция с PagerDuty

```

```

await fetch('https://events.pagerduty.com/v2/enqueue', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    routing_key: process.env.PAGERDUTY_ROUTING_KEY,
    event_action: 'trigger',
    payload: {
      summary: error.message,
      source: error.logFields.component,
      severity: 'critical',
      timestamp: error.timestamp,
      custom_details: error.context
    }
  })
});
}

```

```

async function sendToDatadog(datadogEvent) {
  // Интеграция с DataDog
  await fetch('https://api.datadoghq.com/api/v1/events', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'DD-API-KEY': process.env.DATADOG_API_KEY
    },
    body: JSON.stringify(datadogEvent)
  });
}

```

5.2. Метрики и алерты

```

javascript

// =====
// Error Metrics Collection
// =====

class ErrorMetricsCollector {
  constructor() {
    this.metrics = {
      total: 0,
      byType: new Map(),

```

```

    byCode: new Map(),
    byComponent: new Map(),
    byOperation: new Map(),
    recent: [],
    startTime: Date.now()
  };

  // Периодическая отправка метрик
  setInterval(() => this.flush(), 60000); // Каждую минуту
}

track(error, context = {}) {
  this.metrics.total++;

  // По типу ошибки
  const type = error.name || 'UnknownError';
  this.incrementMap(this.metrics.byType, type);

  // По коду
  if (error.code) {
    this.incrementMap(this.metrics.byCode, error.code);
  }

  // По компоненту
  const component = error.component || context.component || 'unknown';
  this.incrementMap(this.metrics.byComponent, component);

  // По операции
  const operation = error.operation || context.operation || 'unknown';
  this.incrementMap(this.metrics.byOperation, operation);

  // Сохраняем в историю
  this.metrics.recent.unshift({
    timestamp: new Date().toISOString(),
    type,
    code: error.code,
    component,
    operation,
    message: error.message,
    severity: error.severity || 'error'
  });

  // Ограничиваем историю
  if (this.metrics.recent.length > 1000) {

```

```

    this.metrics.recent.pop();
  }

  // Проверка пороговых значений для алертов
  this.checkThresholds(type, component);
}

incrementMap(map, key) {
  map.set(key, (map.get(key) || 0) + 1);
}

checkThresholds(type, component) {
  // Алерт при > 100 ошибок за минуту
  const minuteAgo = Date.now() - 60000;
  const recentErrors = this.metrics.recent.filter(e =>
    new Date(e.timestamp).getTime() > minuteAgo
  ).length;

  if (recentErrors > 100) {
    this.triggerAlert('HIGH_ERROR_RATE', {
      count: recentErrors,
      period: '1 minute',
      threshold: 100
    });
  }

  // Алерт при критических ошибках
  const criticalErrors = this.metrics.recent
    .filter(e => e.severity === 'critical')
    .slice(0, 10);

  if (criticalErrors.length >= 5) {
    this.triggerAlert('CRITICAL_ERRORS', {
      count: criticalErrors.length,
      errors: criticalErrors
    });
  }
}

async triggerAlert(type, data) {
  const alert = {
    type,
    timestamp: new Date().toISOString(),
    data,
  }

```

```

    environment: process.env.NODE_ENV
  };

  // Отправка в Slack
  if (process.env.SLACK_WEBHOOK_URL) {
    await fetch(process.env.SLACK_WEBHOOK_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        text: `❗ *${type}* in ${process.env.NODE_ENV}`,
        attachments: [{
          color: 'danger',
          fields: Object.entries(data).map(([key, value]) => ({
            title: key,
            value: String(value),
            short: true
          })))
        }]
      })
    });
  }

  // Сохраняем алерт
  if (!this.alerts) this.alerts = [];
  this.alerts.unshift(alert);
  if (this.alerts.length > 100) this.alerts.pop();
}

getStats() {
  const uptime = Math.floor((Date.now() - this.metrics.startTime) / 1000);

  return {
    uptime,
    totalErrors: this.metrics.total,
    errorsPerMinute: (this.metrics.total / (uptime / 60)).toFixed(2),
    byType: Object.fromEntries(this.metrics.byType),
    byComponent: Object.fromEntries(this.metrics.byComponent),
    byOperation: Object.fromEntries(this.metrics.byOperation),
    recentAlerts: this.alerts?.slice(0, 10) || []
  };
}

flush() {
  // Отправка метрик в Prometheus/Graphite

```

```

if (process.env.METRICS_ENABLED) {
  this.sendToPrometheus();
}

// Очистка счетчиков для следующего периода
this.metrics.byType.clear();
this.metrics.byCode.clear();
}

async sendToPrometheus() {
  // Интеграция с Prometheus PushGateway
  const metrics = [];

  this.metrics.byType.forEach((value, key) => {
    metrics.push(`errors_by_type{type="${key}"} ${value}`);
  });

  this.metrics.byComponent.forEach((value, key) => {
    metrics.push(`errors_by_component{component="${key}"} ${value}`);
  });

  await fetch(process.env.PROMETHEUS_PUSHGATEWAY, {
    method: 'POST',
    body: metrics.join('\n')
  });
}

const errorMetrics = new ErrorMetricsCollector();

// Middleware для сбора метрик
function metricsMiddleware(error, req, res, next) {
  errorMetrics.track(error, {
    component: req.path.split('/')[1],
    operation: `${req.method} ${req.path}`,
    userId: req.user?.id
  });

  next(error);
}

// Endpoint для просмотра метрик
app.get('/admin/metrics/errors', (req, res) => {
  res.json(errorMetrics.getStats());
}

```

```
});
```

Часть 6: Тестирование пользовательских ошибок

6.1. Unit-тесты для классов ошибок

```
javascript

// =====
// Testing Custom Errors
// =====

import { expect } from 'chai';
import sinon from 'sinon';

describe('Custom Errors', () => {
  describe('ValidationError', () => {
    it('should create error with correct name and message', () => {
      const error = new ValidationError('Test error');

      expect(error).to.be.instanceOf(ValidationError);
      expect(error).to.be.instanceOf(AppError);
      expect(error).to.be.instanceOf(Error);
      expect(error.name).to.equal('ValidationError');
      expect(error.message).to.equal('Test error');
      expect(error.code).to.equal('ERR_VALIDATION');
      expect(error.severity).to.equal('warning');
      expect(error.statusCode).to.equal(400);
    });

    it('should capture validation fields', () => {
      const fields = [{ field: 'email', message: 'Invalid format' }];
      const error = new ValidationError('Test', { fields });

      expect(error.fields).to.deep.equal(fields);
    });

    it('should support adding fields via addField method', () => {
      const error = new ValidationError('Test')
        .addField('email', 'Invalid format')
        .addField('age', 'Required');
    });
  });
});
```



```

expect(error.fields).to.deep.equal([
  { field: 'email', message: 'Invalid format' },
  { field: 'age', message: 'Required' }
]);
});

it('should have stack trace', () => {
  const error = new ValidationError('Test');

  expect(error.stack).to.be.a('string');
  expect(error.stack).to.include('ValidationError');
});

describe('NotFoundError', () => {
  it('should create error with resource context', () => {
    const error = new NotFoundError('User', 123);

    expect(error.name).to.equal('NotFoundError');
    expect(error.message).to.equal('User with id 123 not found');
    expect(error.resource).to.equal('User');
    expect(error.resourceId).to.equal(123);
    expect(error.statusCode).to.equal(404);
  });

  it('should handle missing resource ID', () => {
    const error = new NotFoundError('User');

    expect(error.message).to.equal('User not found');
    expect(error.resource).to.equal('User');
    expect(error.resourceId).to.be.undefined;
  });
});

describe('ErrorBuilder', () => {
  it('should build validation error with chain methods', () => {
    const error = ErrorBuilder.validation()
      .message('Invalid input')
      .context('userId', 123)
      .addField('email', 'Required')
      .addField('age', 'Must be number')
      .build();

    expect(error).to.be.instanceOf(ValidationError);
  });
});

```

```

    expect(error.message).to.equal('Invalid input');
    expect(error.context.userId).to.equal(123);
    expect(error.fields).to.have.length(2);
  });

  it('should reset after build', () => {
    const builder = ErrorBuilder.validation()
      .message('Test');

    const error1 = builder.build();
    const error2 = builder.build();

    expect(error2.message).to.not.equal('Test');
    expect(error2.fields).to.be.empty;
  });
});

describe('Error Handling Integration', () => {
  let userService;

  beforeEach(() => {
    userService = new UserService();
  });

  describe('createUser', () => {
    it('should throw RequiredFieldError when email is missing', async () => {
      try {
        await userService.createUser({ password: '123' });
        expect.fail('Should throw error');
      } catch (error) {
        expect(error).to.be.instanceOf(RequiredFieldError);
        expect(error.field).to.equal('email');
        expect(error.message).to.include('email');
      }
    });

    it('should throw InvalidFormatError when email is invalid', async () => {
      try {
        await userService.createUser({ email: 'invalid', password: '123' });
        expect.fail('Should throw error');
      } catch (error) {
        expect(error).to.be.instanceOf(InvalidFormatError);
        expect(error.field).to.equal('email');
      }
    });
  });
});

```

```

    expect(error.expectedFormat).to.equal('user@example.com');
  }
});

it('should throw DuplicateResourceError when email exists', async () => {
  // Mock repository to simulate duplicate
  sinon.stub(userService.userRepository, 'findByEmail')
    .resolves({ id: 1, email: 'existing@example.com' });

  try {
    await userService.createUser({
      email: 'existing@example.com',
      password: '123'
    });
    expect.fail('Should throw error');
  } catch (error) {
    expect(error).to.be.instanceOf(DuplicateResourceError);
    expect(error.resourceType).to.equal('User');
    expect(error.criteria).to.deep.equal({ email: 'existing@example.com' });
  }

  sinon.restore();
});

it('should wrap database errors in InfrastructureError', async () => {
  const dbError = new Error('Connection refused');
  dbError.code = 'ECONNREFUSED';

  sinon.stub(userService.userRepository, 'create')
    .rejects(dbError);

  try {
    await userService.createUser({
      email: 'test@example.com',
      password: '123'
    });
    expect.fail('Should throw error');
  } catch (error) {
    expect(error).to.be.instanceOf(InfrastructureError);
    expect(error).to.be.instanceOf(DatabaseError);
    expect(error.code).to.equal('ERR_DATABASE');
    expect(error.cause).to.equal(dbError);
    expect(error.retryable).to.be.true;
  }
}

```

```

    sinon.restore();
  });
});

describe('ErrorFactory', () => {
  it('should create cached error instances', () => {
    const factory = ErrorFactory.create();

    const error1 = factory.create('required', 'email');
    const error2 = factory.create('required', 'email');

    expect(error1).to.equal(error2); // Один и тот же экземпляр
  });

  it('should create different instances for different parameters', () => {
    const factory = ErrorFactory.create();

    const error1 = factory.create('range', 'age', 13, 150);
    const error2 = factory.create('range', 'age', 18, 100);

    expect(error1).to.not.equal(error2);
  });
});

describe('Error Metrics', () => {
  it('should track error occurrences', () => {
    const metrics = new ErrorMetricsCollector();

    metrics.track(new ValidationError('Test 1'));
    metrics.track(new ValidationError('Test 2'));
    metrics.track(new NotFoundError('User', 123));

    const stats = metrics.getStats();
    expect(stats.totalErrors).to.equal(3);
    expect(stats.byType.ValidationError).to.equal(2);
    expect(stats.byType.NotFoundError).to.equal(1);
  });

  it('should trigger alerts on high error rate', async () => {
    const metrics = new ErrorMetricsCollector();
    const alertSpy = sinon.spy(metrics, 'triggerAlert');

    // Simulate 150 errors in 1 second

```

```

    for (let i = 0; i < 150; i++) {
      metrics.track(new ValidationError('Test'));
    }

    expect(alertSpy.calledWith('HIGH_ERROR_RATE')).toBe(true);

    alertSpy.restore();
  });
});
});

```

6.2. Тестирование middleware обработки ошибок

```

javascript

describe('Error Handling Middleware', () => {
  let app;
  let request;

  beforeEach(() => {
    app = express();
    app.use(express.json());
    request = supertest(app);
  });

  it('should return 400 with validation error details', async () => {
    app.post('/test', (req, res) => {
      throw new ValidationError('Invalid input', {
        fields: [{ field: 'email', message: 'Required' }]
      });
    });

    app.use(errorHandler);

    const response = await request
      .post('/test')
      .expect(400);

    expect(response.body).to.deep.equal({
      success: false,
      error: {
        code: 'ERR_VALIDATION',
        message: 'Invalid input',
        fields: [{ field: 'email', message: 'Required' }]
      }
    });
  });
});

```

```
    }  
  });  
});
```

```
it('should return 404 with resource information', async () => {  
  app.get('/users/:id', (req, res) => {  
    throw new NotFoundError('User', req.params.id);  
  });  
});
```

```
app.use(errorHandler);
```

```
const response = await request  
  .get('/users/123')  
  .expect(404);
```

```
expect(response.body.error).to.include({  
  code: 'ERR_NOT_FOUND',  
  resource: 'User',  
  id: '123'  
});  
});
```

```
it('should include retry-after header for rate limit errors', async () => {  
  app.get('/api', (req, res) => {  
    throw new RateLimitError('user123', 5, Date.now() + 60000);  
  });  
});
```

```
app.use(errorHandler);
```

```
const response = await request  
  .get('/api')  
  .expect(429);
```

```
expect(response.headers['retry-after']).to.exist;  
expect(response.body.error.retryAfter).to.equal(60);  
});
```

```
it('should localize error messages based on Accept-Language', async () => {  
  app.post('/users', (req, res) => {  
    throw new I18nValidationError('email');  
  });  
});
```

```
app.use(i18nErrorMiddleware);  
app.use(errorHandler);
```

```

const response = await request
  .post('/users')
  .set('Accept-Language', 'ru')
  .expect(400);

expect(response.body.error.message).to.equal('Поле "email" обязательно');
});

it('should enrich errors with request context', async () => {
  app.use(enrichErrorMiddleware);

  app.get('/test', (req, res) => {
    const error = new ValidationError('Test');
    throw res.enrichError(error);
  });

  app.use(errorHandler);

  const response = await request
    .get('/test')
    .set('X-Request-ID', 'req-123')
    .expect(400);

  // Проверяем, что ошибка была обогащена (в логах)
  // Response не должен содержать чувствительные данные
  expect(response.body.error.requestId).to.be.undefined;
});
});

```

Часть 7: Документирование пользовательских ошибок

7.1. JSDoc для классов ошибок

```

javascript

/**
 * @class ValidationError
 * @extends AppError
 * @description Ошибка валидации входных данных
 *
 * @example

```

```

* throw new ValidationError('Invalid user data', {
*   fields: [
*     { field: 'email', message: 'Invalid format' },
*     { field: 'age', message: 'Must be at least 13' }
*   ]
* });
*/

class ValidationError extends AppError {
  /**
   * @constructor
   * @param {string} message - Сообщение об ошибке
   * @param {Object} options - Дополнительные параметры
   * @param {Array<Object>} options.fields - Детали ошибок по полям
   * @param {string} options.fields[].field - Имя поля
   * @param {string} options.fields[].message - Описание ошибки
   * @param {*} [options.invalidValue] - Некорректное значение
   */
  constructor(message, options = {}) {
    super(message, { code: 'ERR_VALIDATION', severity: 'warning', ...options });
    this.name = 'ValidationError';
    this.fields = options.fields || [];
    this.invalidValue = options.invalidValue;
  }

  /**
   * Добавляет информацию о поле с ошибкой
   * @param {string} field - Имя поля
   * @param {string} message - Описание ошибки
   * @returns {ValidationError} Текущий экземпляр ошибки (для chaining)
   */
  addField(field, message) {
    this.fields.push({ field, message });
    return this;
  }
}

/**
 * @class NotFoundError
 * @extends HttpError
 * @description Ошибка: ресурс не найден
 *
 * @example
 * throw new NotFoundError('User', userId);
 */

```



```

class NotFoundError extends HttpError {
  /**
   * @constructor
   * @param {string} resource - Тип ресурса (User, Order, Product и т.д.)
   * @param {string|number} [id] - Идентификатор ресурса
   * @param {Object} [options] - Дополнительные параметры
   */
  constructor(resource, id, options = {}) {
    const message = resource && id
      ? `${resource} with id ${id} not found`
      : `${resource} || 'Resource' not found`;

    super(message, 404, options);
    this.name = 'NotFoundError';
    this.code = 'ERR_NOT_FOUND';
    this.resource = resource;
    this.resourceId = id;
  }
}

```

7.2. OpenAPI/Swagger документация

yaml

```

# openapi.yaml
components:
  schemas:
    ErrorResponse:
      type: object
      properties:
        success:
          type: boolean
          example: false
        error:
          type: object
          properties:
            code:
              type: string
              example: ERR_VALIDATION
            message:
              type: string
              example: Validation failed
            fields:
              type: array

```

```
items:
  type: object
  properties:
    field:
      type: string
      example: email
    message:
      type: string
      example: Invalid format
```

ValidationError:

```
allOf:
- $ref: '#/components/schemas/ErrorResponse'
- type: object
properties:
  error:
    properties:
      code:
        enum: [ERR_VALIDATION, ERR_REQUIRED_FIELD, ERR_INVALID_FORMAT]
      fields:
        type: array
```

NotFoundError:

```
allOf:
- $ref: '#/components/schemas/ErrorResponse'
- type: object
properties:
  error:
    properties:
      code:
        enum: [ERR_NOT_FOUND]
      resource:
        type: string
      id:
        type: string
```

responses:

ValidationError:

```
description: Validation failed
content:
  application/json:
    schema:
      $ref: '#/components/schemas/ValidationError'
```

```
NotFoundError:
  description: Resource not found
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/NotFoundError'

paths:
  /users:
    post:
      summary: Create new user
      responses:
        400:
          $ref: '#/components/responses/ValidationError'
        409:
          description: User already exists
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/DuplicateResourceError'

  /users/{id}:
    get:
      summary: Get user by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        200:
          description: OK
        404:
          $ref: '#/components/responses/NotFoundError'
```

Часть 8: Миграция и лучшие практики

8.1. Стратегия миграции на пользовательские ошибки

javascript

```
// =====
// Migration Strategy
// =====

// Шаг 1: Создаем адаптер для обратной совместимости
class LegacyErrorAdapter {
  static toLegacy(error) {
    if (error instanceof AppError) {
      return {
        message: error.message,
        type: error.name,
        code: error.code,
        statusCode: error.statusCode
      };
    }
    return error;
  }

  static fromLegacy(legacyError) {
    if (legacyError.code === 'VALIDATION') {
      return new ValidationError(legacyError.message, {
        fields: legacyError.fields
      });
    }
    if (legacyError.code === 'NOT_FOUND') {
      return new NotFoundError(
        legacyError.resource,
        legacyError.id
      );
    }
    return new AppError(legacyError.message, {
      code: legacyError.code
    });
  }
}

// Шаг 2: Постепенное внедрение в сервисы
class UserService {
  // Новая версия с пользовательскими ошибками
  async createUserV2(userData) {
    // Используем новые ошибки
  }

  // Старая версия для обратной совместимости
}
```

```

async createUserV1(userData) {
  try {
    const user = await this.createUserV2(userData);
    return user;
  } catch (error) {
    throw LegacyErrorAdapter.toLegacy(error);
  }
}
}

// Шаг 3: Feature flag для постепенного переключения
class FeatureFlagService {
  static useNewErrors() {
    return process.env.FEATURE_NEW_ERRORS === 'true';
  }
}

// Шаг 4: Мониторинг миграции
class MigrationMonitor {
  static trackErrorMigration(error) {
    if (error instanceof AppError) {
      metrics.increment('errors.custom');
    } else {
      metrics.increment('errors.legacy');
    }
  }
}

```

8.2. Чек-лист создания пользовательских ошибок

javascript

```

const customErrorChecklist = {
  // 📌 Наследование
  extendsError: 'Наследоваться от Error или AppError',
  callsSuper: 'Вызывать super() в конструкторе',
  setName: 'Устанавливать this.name = this.constructor.name',
  captureStackTrace: 'Использовать Error.captureStackTrace',

  // 📌 Функциональность
  meaningfulMessage: 'Содержать понятное сообщение об ошибке',
  errorCode: 'Иметь уникальный код ошибки',
  statusCode: 'HTTP статус код для API ошибок',
  context: 'Содержать контекст ошибки',
}

```

```
severity: 'Уровень серьезности (error/warning/critical)',

// 📄 Документация
jsdoc: 'JSDoc комментарии',
examples: 'Примеры использования',
openapi: 'OpenAPI спецификация',

// 🧪 Тестирование
unitTests: 'Юнит-тесты',
instanceChecks: 'Проверки instanceof',
edgeCases: 'Граничные случаи',

// 🌐 Интеграция
logging: 'Интеграция с логгером',
metrics: 'Сбор метрик',
monitoring: 'Мониторинг и алерты',
localization: 'Интернационализация',

// 🚀 Производительность
immutable: 'Неизменяемость (для кэшируемых ошибок)',
cached: 'Кэширование часто используемых ошибок',
lightweight: 'Минимальный размер'
};

// Проверка класса ошибки
function validateCustomError(ErrorClass) {
  const issues = [];
  const instance = new ErrorClass('Test');

  if (!(instance instanceof Error)) {
    issues.push('Does not extend Error');
  }

  if (instance.name !== ErrorClass.name) {
    issues.push('this.name not set to constructor name');
  }

  if (!instance.stack) {
    issues.push('No stack trace');
  }

  if (!instance.message) {
    issues.push('No message');
  }
}
```

```
return {  
  valid: issues.length === 0,  
  issues  
};  
}
```

Заключение

Ключевые принципы создания пользовательских ошибок

1. **Иерархия** — Создавайте многоуровневую иерархию от общего к частному
2. **Контекст** — Всегда добавляйте контекст возникновения ошибки
3. **Идентификация** — Используйте уникальные коды для каждого типа ошибки
4. **Документация** — Документируйте типы ошибок и их значения
5. **Тестирование** — Покрывайте тестами все классы ошибок
6. **Мониторинг** — Собирайте метрики по типам ошибок
7. **Локализация** — Поддерживайте интернационализацию сообщений
8. **Производительность** — Кэшируйте часто используемые ошибки

Рекомендации на 2026 год

```
javascript  
  
const bestPractices2026 = {  
  // 📌 Использовать ES6 классы для всех ошибок  
  errorClasses: true,  
  
  // 📌 Создавать иерархию глубиной 2-3 уровня  
  hierarchyDepth: '2-3 levels',  
  
  // 📌 Включать code, timestamp, id в каждую ошибку  
  requiredFields: ['code', 'timestamp', 'id'],  
  
  // 📌 Использовать Error.captureStackTrace в Node.js  
  stackTrace: true,  
  
  // 📌 Добавлять cause для цепочек ошибок  
  errorCause: true,  
  
  // 📌 Кэшировать immutable ошибки  
  errorCaching: true,
```

```
//  Поддерживать OpenAPI документацию
openApi: true,

//  Интегрировать с APM системами
apmIntegration: true
};
```

Пользовательские ошибки — это не просто способ выбросить исключение. Это инструмент проектирования API, документирования поведения системы и создания надежного, самодиагностирующегося программного обеспечения. Инвестируйте время в создание хорошей системы ошибок на ранних этапах разработки — это окупится многократно за время жизни проекта.

Введение: Необходимость глобальной обработки ошибок

В любом приложении возникают ошибки, которые не были предусмотрены разработчиком или обработаны в конкретном месте. Без глобальной обработки такие ошибки приводят к падению приложения, несохранению данных, утечке памяти и негативному пользовательскому опыту.

Проблемы локальной обработки

javascript

// × ПЛОХО: Каждый модуль обрабатывает ошибки по-своему

```
class UserService {
  async getUser(id) {
    try {
      const response = await fetch(`/api/users/${id}`);
      return await response.json();
    } catch (error) {
      console.error('Error in UserService:', error);
      return null; // Тихая ошибка
    }
  }
}

class OrderService {
  async getOrders(userId) {
    try {
      const response = await fetch(`/api/orders?userId=${userId}`);
      return await response.json();
    } catch (error) {
      // Разный формат логирования
      console.log('Order service error:', error.message);
      throw error; // Пробрасываем дальше
    }
  }
}
```

```
// Проблемы:  
// 1. Непоследовательная обработка  
// 2. Нет централизованного логирования  
// 3. Пользователь не видит ошибки  
// 4. Нет мониторинга  
// 5. Невозможно собрать статистику
```

Что дает глобальная обработка?

```
javascript  
  
const benefitsOfGlobalErrorHandling = {  
  // 1. Единое место для логирования  
  logging: 'Все ошибки попадают в одну систему логирования',  
  
  // 2. Консистентный ответ клиенту  
  consistentResponse: 'Стандартизированный формат ошибок API',  
  
  // 3. Восстановление после ошибок  
  recovery: 'Graceful degradation без падения приложения',  
  
  // 4. Мониторинг и алерты  
  monitoring: 'Автоматическое уведомление о критических ошибках',  
  
  // 5. Пользовательский опыт  
  userExperience: 'Понятные сообщения об ошибках для пользователя',  
  
  // 6. Отладка в production  
  debugging: 'Сбор контекста и стектрейсов для анализа',  
  
  // 7. Безопасность  
  security: 'Фильтрация чувствительной информации'  
};
```

Часть 1: Глобальная обработка ошибок в браузере

1.1. Событие window.onerror

```
javascript  
  
// =====
```

```

// window.onerror - перехват синхронных ошибок
// =====

window.onerror = function(message, source, lineno, colno, error) {
  console.group('📦 Global Error Handler');
  console.log('Message:', message);
  console.log('Source:', source);
  console.log('Line:', lineno, 'Column:', colno);
  console.log('Error object:', error);
  console.groupEnd();

  // Отправка в систему мониторинга
  sendToErrorTracking({
    type: 'window.onerror',
    message,
    source,
    lineno,
    colno,
    stack: error?.stack,
    timestamp: new Date().toISOString(),
    url: window.location.href,
    userAgent: navigator.userAgent
  });

  // Возвращаем true, чтобы предотвратить стандартное логирование
  return true;
};

// Примеры ошибок, которые перехватываются
function causeErrors() {
  // 1. ReferenceError
  nonExistentVariable(); // 📦 Перехватывается

  // 2. TypeError
  null.method(); // 📦 Перехватывается

  // 3. SyntaxError (только при eval)
  eval('if condition { console.log("error"); }'); // 📦 Перехватывается
}

// Продвинутый обработчик
class GlobalErrorHandler {
  constructor(options = {}) {
    this.options = {

```

```

    logToConsole: true,
    sendToServer: true,
    showUserNotification: false,
    ignorePatterns: [
        /ResizeObserver/i,
        /NetworkError/i,
        /Script error\.\/i
    ],
    ...options
};

this.errorCount = 0;
this.errorLimit = this.options.errorLimit || 100;
this.init();
}

init() {
    window.onerror = this.handleError.bind(this);
}

handleError(message, source, lineno, colno, error) {
    this.errorCount++;

    // Предотвращаем бесконечный цикл при превышении лимита
    if (this.errorCount > this.errorLimit) {
        console.warn('Error limit exceeded, stopping error handling');
        return true;
    }

    // Проверяем, нужно ли игнорировать ошибку
    if (this.shouldIgnoreError(message, error)) {
        return true;
    }

    // Формируем отчет
    const report = this.createErrorReport(message, source, lineno, colno, error);

    // Логируем
    if (this.options.logToConsole) {
        this.logToConsole(report);
    }

    // Отправляем на сервер
    if (this.options.sendToServer) {

```

```

    this.sendToServer(report);
}

// Показываем пользователю
if (this.options.showUserNotification) {
    this.notifyUser(report);
}

return true;
}

shouldIgnoreError(message, error) {
    return this.options.ignorePatterns.some(pattern =>
        pattern.test(message) || (error && pattern.test(error.message))
    );
}

createErrorReport(message, source, lineno, colno, error) {
    return {
        id: generateErrorId(),
        type: 'window.onerror',
        message,
        source,
        lineno,
        colno,
        stack: error?.stack,
        name: error?.name,

        // Контекст
        timestamp: new Date().toISOString(),
        url: window.location.href,
        userAgent: navigator.userAgent,
        language: navigator.language,
        platform: navigator.platform,

        // Дополнительная информация
        cookies: document.cookie.length,
        localStorage: Object.keys(localStorage || {}).length,
        sessionStorage: Object.keys(sessionStorage || {}).length,

        // Состояние приложения
        reduxState: window.__REDUX_STATE__,
        vuexState: window.__VUEX_STATE__,
    };
}

```

```
// Performance
performance: {
  loadTime: performance.now(),
  memory: performance.memory,
  navigation: performance.navigation
}
};
}
```

```
logToConsole(report) {
  console.groupCollapsed(`❏ Error #${report.id}`);
  console.error('Message:', report.message);
  console.debug('Stack:', report.stack);
  console.debug('Context:', report);
  console.groupEnd();
}
```

```
async sendToServer(report) {
  try {
    const response = await fetch('/api/logs/error', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(report),
      // Не ждем ответа, чтобы не блокировать UI
      keepalive: true
    });
  } catch (e) {
    // Тихая ошибка - не можем отправить отчет
  }
}
```

```
notifyUser(report) {
  // Показываем ненавязчивое уведомление
  const notification = document.createElement('div');
  notification.className = 'error-notification';
  notification.textContent = 'Произошла ошибка. Мы уже работаем над её исправлением.';
  notification.style.cssText = `
    position: fixed;
    bottom: 20px;
    right: 20px;
    background: #ff4444;
    color: white;
    padding: 12px 24px;
    border-radius: 4px;
  `;
}
```

```

    z-index: 9999;
    animation: slideIn 0.3s;
`;

document.body.appendChild(notification);

setTimeout(() => {
    notification.remove();
}, 5000);
}
}

const browserErrorHandler = new GlobalErrorHandler({
    logToConsole: true,
    sendToServer: true,
    showUserNotification: process.env.NODE_ENV === 'production',
    ignorePatterns: [
        /ResizeObserver/i,
        /NetworkError when fetching/i,
        /Script error\.\/i
    ]
});

```

1.2. Событие unhandledrejection

```

javascript

// =====
// unhandledrejection - перехват непойманных Promise rejections
// =====

window.addEventListener('unhandledrejection', (event) => {
    console.group('❏ Unhandled Promise Rejection');
    console.log('Reason:', event.reason);
    console.log('Promise:', event.promise);
    console.groupEnd();

    // Получаем ошибку из rejection
    const error = event.reason instanceof Error
        ? event.reason
        : new Error(String(event.reason));

    // Отправляем в систему мониторинга
    sendToErrorTracking({

```

```

    type: 'unhandledrejection',
    message: error.message,
    stack: error.stack,
    name: error.name,
    reason: event.reason,
    timestamp: new Date().toISOString()
  });

  // Предотвращаем вывод в консоль
  event.preventDefault();
});

// Примеры промисов, которые перехватываются
function causePromiseErrors() {
  // 1. Простой reject без catch
  Promise.reject(new Error('Direct rejection')); //  Перехватывается

  // 2. Ошибка в then без catch
  Promise.resolve().then(() => {
    throw new Error('Error in then');
  }); //  Перехватывается

  // 3. Ошибка в асинхронной функции без try-catch
  (async () => {
    throw new Error('Async function error');
  })(); //  Перехватывается
}

// Продвинутый обработчик unhandledrejection
class UnhandledRejectionHandler {
  constructor(options = {}) {
    this.options = {
      trackRejections: true,
      captureRejections: true,
      ...options
    };
  }

  this.rejections = new Map();
  this.init();
}

init() {
  window.addEventListener('unhandledrejection', this.handleUnhandled.bind(this));
  window.addEventListener('rejectionhandled', this.handleHandled.bind(this));
}

```



```

}

handleUnhandled(event) {
  const id = Symbol('rejection');
  const error = this.normalizeError(event.reason);

  // Сохраняем информацию о rejection
  this.rejections.set(id, {
    error,
    timestamp: Date.now(),
    promise: event.promise
  });

  const report = this.createRejectionReport(error, event);

  // Логируем
  console.error('❌ Unhandled Rejection:', report);

  // Отправляем на сервер
  if (this.options.captureRejections) {
    this.sendRejectionReport(report);
  }

  // Проверяем на утечки памяти
  this.checkMemoryLeaks();

  event.preventDefault();
}

handleHandled(event) {
  // Находим и удаляем обработанный rejection
  for (const [id, rejection] of this.rejections) {
    if (rejection.promise === event.promise) {
      this.rejections.delete(id);

      console.info('✅ Rejection handled after delay:', {
        delay: Date.now() - rejection.timestamp,
        error: rejection.error.message
      });
      break;
    }
  }
}

```

```

normalizeError(reason) {
  if (reason instanceof Error) {
    return reason;
  }

  if (typeof reason === 'string') {
    return new Error(reason);
  }

  if (reason && typeof reason === 'object') {
    const error = new Error(reason.message || 'Unknown rejection');
    error.original = reason;
    return error;
  }

  return new Error(String(reason));
}

createRejectionReport(error, event) {
  return {
    id: generateErrorId(),
    type: 'unhandledrejection',
    message: error.message,
    stack: error.stack,
    name: error.name,
    reason: event.reason,

    // Контекст
    timestamp: new Date().toISOString(),
    url: window.location.href,

    // Состояние приложения
    activeRejections: this.rejections.size,

    // Дополнительная информация
    isSecureContext: window.isSecureContext,
    crossOriginIsolated: window.crossOriginIsolated
  };
}

async sendRejectionReport(report) {
  if (navigator.sendBeacon) {
    // Используем sendBeacon для отправки перед выгрузкой страницы
    const blob = new Blob([JSON.stringify(report)], { type: 'application/json' });
  }
}

```

```

    navigator.sendBeacon('/api/logs/rejection', blob);
  } else {
    // Fallback на fetch с keepalive
    try {
      await fetch('/api/logs/rejection', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(report),
        keepalive: true
      });
    } catch (e) {
      // Тихая ошибка
    }
  }
}

checkMemoryLeaks() {
  // Проверяем накопление необработанных rejection-ов
  const now = Date.now();
  const threshold = 5 * 60 * 1000; // 5 минут

  for (const [id, rejection] of this.rejections) {
    if (now - rejection.timestamp > threshold) {
      console.warn('▲ Long-lived unhandled rejection:', {
        error: rejection.error.message,
        age: Math.round((now - rejection.timestamp) / 1000) + 's'
      });

      // Отправляем предупреждение
      this.sendRejectionReport({
        ...this.createRejectionReport(rejection.error),
        warning: 'long-lived-rejection',
        age: now - rejection.timestamp
      });
    }
  }
}

const rejectionHandler = new UnhandledRejectionHandler();

```

1.3. React Error Boundaries

```
javascript

// =====
// React Error Boundaries
// =====

import React from 'react';

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      hasError: false,
      error: null,
      errorInfo: null
    };
  }

  static getDerivedStateFromError(error) {
    // Обновляем состояние, чтобы следующий рендер показал fallback UI
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    // Логируем ошибку
    console.group('❌ React Error Boundary');
    console.error('Error:', error);
    console.error('Component Stack:', errorInfo.componentStack);
    console.groupEnd();

    this.setState({ errorInfo });

    // Отправляем в систему мониторинга
    sendToErrorTracking({
      type: 'react-error-boundary',
      error: {
        name: error.name,
        message: error.message,
        stack: error.stack
      },
      componentStack: errorInfo.componentStack,
      timestamp: new Date().toISOString(),
    });
  }
}
```

```

// Контекст компонента
componentName: this.props.name || 'Unknown',
props: this.props.errorProps,

// Маршрут
route: window.location.pathname
});

// Вызываем пользовательский обработчик
if (this.props.onError) {
  this.props.onError(error, errorInfo);
}
}

render() {
  if (this.state.hasError) {
    // Используем пользовательский fallback или стандартный
    if (this.props.fallback) {
      return this.props.fallback;
    }

    return (
      <div className="error-boundary-fallback">
        <h2>Что-то пошло не так</h2>
        <details style={{ whiteSpace: 'pre-wrap' }}>
          <summary>Подробности ошибки</summary>
          {this.state.error && this.state.error.toString()}
          <br />
          {this.state.errorInfo?.componentStack}
        </details>
        <button onClick={() => window.location.reload()}>
          Перезагрузить страницу
        </button>
      </div>
    );
  }

  return this.props.children;
}
}

// НОС для удобного использования
function withErrorBoundary(WrappedComponent, errorBoundaryProps = {}) {

```

```

return function WithErrorBoundary(props) {
  return (
    <ErrorBoundary {...errorBoundaryProps}>
      <WrappedComponent {...props} />
    </ErrorBoundary>
  );
};
}

```

// Использование в приложении

```

function App() {
  return (
    <div className="App">
      {/* Глобальный Error Boundary */}
      <ErrorBoundary
        name="AppRoot"
        onError={(error, errorInfo) => {
          // Дополнительная обработка
          analytics.trackError('react_error', error);
        }}
        fallback={<GlobalErrorScreen />}
      >
        {/* Маршрутизация с Error Boundary для каждого роута */}
        <Routes>
          <Route
            path="/users"
            element={
              <ErrorBoundary name="UsersPage">
                <UsersPage />
              </ErrorBoundary>
            }
          />
          <Route
            path="/orders"
            element={
              <ErrorBoundary name="OrdersPage">
                <OrdersPage />
              </ErrorBoundary>
            }
          />
        </Routes>

        {/* Error Boundary для виджетов */}
        <div className="dashboard">

```

```

    <ErrorBoundary
      name="StockWidget"
      fallback={<div>⚠ Биржевой виджет временно недоступен</div>}
    >
      <StockWidget />
    </ErrorBoundary>

    <ErrorBoundary
      name="WeatherWidget"
      fallback={<div>✖ 24°C</div>} // Заглушка
    >
      <WeatherWidget />
    </ErrorBoundary>
  </div>
</ErrorBoundary>
</div>
);
}

// Асинхронный Error Boundary для загрузки компонентов
class AsyncErrorBoundary extends ErrorBoundary {
  componentDidCatch(error, errorInfo) {
    super.componentDidCatch(error, errorInfo);

    // Пробуем перезагрузить чанк при ошибке загрузки
    if (error.message.includes('Loading chunk') ||
        error.message.includes('ChunkLoadError')) {
      console.log('🔄 Reloading chunk...');

      if (this.props.onChunkError) {
        this.props.onChunkError(error);
      }

      // Предлагаем пользователю обновить страницу
      if (window.confirm('Доступна новая версия приложения. Обновить?')) {
        window.location.reload();
      }
    }
  }
}

```

1.4. Vue.js Error Handler

```
javascript

// =====
// Vue.js Global Error Handling
// =====

import { createApp } from 'vue';

const app = createApp(App);

// Глобальный обработчик ошибок
app.config.errorHandler = (err, instance, info) => {
  console.group('🔴 Vue Global Error Handler');
  console.error('Error:', err);
  console.error('Component:', instance);
  console.error('Info:', info);
  console.groupEnd();

  // Отправка в Sentry
  Sentry.captureException(err, {
    contexts: {
      vue: {
        component: instance?.$options?.name || 'anonymous',
        info,
        props: instance?.$props,
        data: instance?.$data
      }
    }
  });

  // Метрики
  metrics.increment('vue.error', {
    component: instance?.$options?.name || 'anonymous',
    info,
    errorType: err.name
  });
};

// Глобальный обработчик предупреждений
app.config.warnHandler = (msg, instance, trace) => {
  console.warn('🟡 Vue Warning:', msg);
  console.warn('Component:', instance);
};
```



```

console.warn('Trace:', trace);

if (process.env.NODE_ENV === 'development') {
  // Показываем предупреждение в UI для разработки
  showDevWarning(msg, trace);
}
};

// Vue плагин для обработки ошибок
const VueErrorHandlerPlugin = {
  install(app, options = {}) {
    // Глобальный обработчик
    app.config.errorHandler = options.errorHandler || defaultHandler;

    // Директива для отслеживания ошибок в элементах
    app.directive('error-track', {
      mounted(el, binding) {
        el.addEventListener('error', (event) => {
          trackComponentError(binding.value, event);
        });
      }
    });

    // Миксин для компонентов
    app.mixin({
      created() {
        const errorHandler = this.$options.error;

        if (errorHandler && typeof errorHandler === 'object') {
          Object.keys(errorHandler).forEach(key => {
            if (typeof errorHandler[key] === 'function') {
              const original = this[key];
              this[key] = function(...args) {
                try {
                  return original?.apply(this, args);
                } catch (err) {
                  errorHandler[key].call(this, err, ...args);
                }
              };
            }
          });
        }
      }
    });
  }
};

```

```

// Глобальная функция для отправки ошибок
app.config.globalProperties.$trackError = function(error, context = {}) {
  options.trackingService?.capture(error, {
    ...context,
    component: this.$options.name,
    route: this.$route?.path
  });
};
}
};

```

```

app.use(VueErrorHandlerPlugin, {
  trackingService: Sentry,
  errorHandler: (err, instance, info) => {
    // Кастомная обработка
  }
});

```

// Компонент-обертка для catch-ошибок в шаблонах

```

Vue.component('error-boundary', {
  data() {
    return {
      error: null,
      errorInfo: null
    };
  },

  errorCaptured(err, vm, info) {
    this.error = err;
    this.errorInfo = info;

    this.$emit('error', err, vm, info);

    // Возвращаем false, чтобы остановить всплытие
    return false;
  },

  render() {
    if (this.error) {
      return this.$scopedSlots.fallback?.({
        error: this.error,
        info: this.errorInfo,
        reset: () => {

```

```

        this.error = null;
        this.errorInfo = null;
    }
    }) || this.$slots.fallback;
}

return this.$slots.default?()[0];
}
});

```

// Использование

```

<template>
  <error-boundary @error="handleError">
    <template #default>
      <user-profile :id="userId" />
    </template>

    <template #fallback="{ error, reset }">
      <div class="error-fallback">
        <h3>Не удалось загрузить профиль</h3>
        <p>{{ error.message }}</p>
        <button @click="reset">Повторить</button>
      </div>
    </template>
  </error-boundary>
</template>

```

1.5. Angular Error Handler

typescript

```

// =====
// Angular Global Error Handler
// =====

```

```

import { ErrorHandler, Injectable, Injector, NgZone } from '@angular/core';
import { HttpResponse } from '@angular/common/http';
import { Router } from '@angular/router';

```

```

@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  constructor(
    private injector: Injector,
    private ngZone: NgZone
  ) {}
}

```

```
) {}
```

```
handleError(error: any): void {  
  // Получаем сервисы через injector для избежания циклических зависимостей  
  const router = this.injector.get(Router);  
  const notificationService = this.injector.get(NotificationService);  
  const loggingService = this.injector.get(LoggingService);  
  
  // Определяем тип ошибки  
  if (error instanceof HttpResponse) {  
    // HTTP ошибка  
    this.handleHttpError(error, router, notificationService);  
  } else if (error instanceof TypeError) {  
    // Ошибка типов  
    this.handleTypeError(error, loggingService);  
  } else if (error instanceof Error) {  
    // Общая ошибка  
    this.handleGenericError(error, loggingService, notificationService);  
  } else {  
    // Неизвестная ошибка  
    this.handleUnknownError(error, loggingService);  
  }  
  
  // Логируем в консоль в development  
  if (!environment.production) {  
    console.error('❌ Global Error:', error);  
  }  
  
  // Отправляем в Sentry  
  Sentry.captureException(error);  
  
  // Метрики  
  this.trackErrorMetrics(error);  
}  
  
private handleHttpError(  
  error: HttpResponse,  
  router: Router,  
  notificationService: NotificationService  
): void {  
  const status = error.status;  
  
  if (status === 401) {  
    // Неавторизован - редирект на логин
```

```

this.ngZone.run(() => {
  router.navigate(['/login'], {
    queryParams: { returnUrl: router.url }
  });
});
} else if (status === 403) {
  // Доступ запрещен
  notificationService.showWarning('У вас нет доступа к этому ресурсу');
} else if (status === 404) {
  // Не найдено
  notificationService.showInfo('Запрашиваемый ресурс не найден');
} else if (status >= 500) {
  // Серверная ошибка
  notificationService.showError('Сервер временно недоступен');
}
}

```

```

private handleError(
  error: TypeError,
  loggingService: LoggingService
): void {
  loggingService.logError({
    type: 'TypeError',
    message: error.message,
    stack: error.stack,
    context: {
      component: this.getComponentContext(error)
    }
  });
}

```

```

private handleGenericError(
  error: Error,
  loggingService: LoggingService,
  notificationService: NotificationService
): void {
  loggingService.logError({
    type: error.name,
    message: error.message,
    stack: error.stack
  });
}

```

```

// Показываем пользователю в production
if (environment.production) {

```

```

notificationService.showError(
  'Произошла ошибка. Мы уже работаем над её исправлением.'
);
}
}

```

```

private handleUnknownError(
  error: any,
  loggingService: LoggingService
): void {
  loggingService.logError({
    type: 'UnknownError',
    value: error,
    timestamp: new Date().toISOString()
  });
}

```

```

private getComponentContext(error: Error): string {
  // Парсим стек, чтобы найти компонент
  const stack = error.stack || '';
  const match = stack.match(/at ([^ ]+) \(/);
  return match ? match[1] : 'unknown';
}

```

```

private trackErrorMetrics(error: any): void {
  // Отправка метрик в Google Analytics
  gtag('event', 'exception', {
    description: error.message,
    fatal: true
  });

  // Кастомные метрики
  if (window.performance?.memory) {
    console.debug('Memory:', window.performance.memory);
  }
}

```

```

// Регистрация в модуле
@NgModule({
  providers: [
    {
      provide: ErrorHandler,
      useClass: GlobalErrorHandler
    }
  ]
})

```

```
    }  
  ]  
})  
export class AppModule {}
```

Часть 2: Глобальная обработка ошибок в Node.js

2.1. Uncaught Exceptions

```
javascript  
  
// =====  
// process.on('uncaughtException')  
// =====  
  
process.on('uncaughtException', (error, origin) => {  
  console.error('❌ Uncaught Exception:');  
  console.error('Error:', error);  
  console.error('Origin:', origin);  
  
  // Логирование  
  logger.error('Uncaught Exception', {  
    error: {  
      name: error.name,  
      message: error.message,  
      stack: error.stack  
    },  
    origin,  
    pid: process.pid,  
    memory: process.memoryUsage(),  
    uptime: process.uptime()  
  });  
  
  // Отправка в Sentry  
  Sentry.captureException(error, {  
    contexts: {  
      node: {  
        origin,  
        pid: process.pid,  
        version: process.version  
      }  
    }  
  })  
});
```

```
});

// Graceful shutdown
gracefulShutdown(error)
  .then(() => {
    process.exit(1);
  })
  .catch((shutdownError) => {
    console.error('Error during graceful shutdown:', shutdownError);
    process.exit(1);
  });
});
```

```
// Graceful Shutdown
async function gracefulShutdown(error) {
  console.log('🔴 Starting graceful shutdown...');

  // Создаем промис с таймаутом
  const shutdownTimeout = new Promise( (_, reject) => {
    setTimeout(() => reject(new Error('Shutdown timeout')), 30000);
  });
```

```
  try {
    await Promise.race([
      performShutdown(),
      shutdownTimeout
    ]);

    console.log('🟢 Graceful shutdown completed');
  } catch (error) {
    console.error('✖ Graceful shutdown failed:', error);
    throw error;
  }
}
```

```
async function performShutdown() {
  // 1. Закрываем HTTP сервер
  if (httpServer) {
    await new Promise((resolve) => {
      httpServer.close(resolve);
    });
    console.log('✓ HTTP server closed');
  }
}
```



```

// 2. Закрываем соединения с базами данных
if (mongoose.connection.readyState === 1) {
  await mongoose.connection.close();
  console.log('✓ MongoDB connection closed');
}

if (redisClient) {
  await redisClient.quit();
  console.log('✓ Redis connection closed');
}

// 3. Завершаем активные запросы
if (activeRequests.size > 0) {
  console.log(`Waiting for ${activeRequests.size} active requests...`);
  await Promise.all(Array.from(activeRequests.values()));
}

// 4. Закрываем логгер
await logger.end();
console.log('✓ Logger closed');

// 5. Отправляем последние метрики
await metrics.flush();
console.log('✓ Metrics flushed');
}

```

2.2. Unhandled Rejections

```

javascript

// =====
// process.on('unhandledRejection')
// =====

process.on('unhandledRejection', (reason, promise) => {
  console.error('❌ Unhandled Rejection:');
  console.error('Reason:', reason);
  console.error('Promise:', promise);

  // Нормализуем ошибку
  const error = reason instanceof Error
    ? reason
    : new Error(String(reason));

```

```

// Логирование
logger.error('Unhandled Rejection', {
  error: {
    name: error.name,
    message: error.message,
    stack: error.stack
  },
  reason: String(reason),
  promise: String(promise),
  pid: process.pid
});

// Отправка в Sentry
Sentry.captureException(error, {
  contexts: {
    promise: {
      reason: String(reason)
    }
  }
});

// В Node.js рекомендуется завершать процесс
// process.exit(1);
});

// Продвинутый обработчик с трекингом
class UnhandledRejectionTracker {
  constructor(options = {}) {
    this.rejections = new Map();
    this.warnThreshold = options.warnThreshold || 1000;
    this.criticalThreshold = options.criticalThreshold || 5000;

    this.init();
  }

  init() {
    process.on('unhandledRejection', this.track.bind(this));
    process.on('rejectionHandled', this.markHandled.bind(this));

    // Периодическая проверка
    setInterval(() => this.checkStaleRejections(), 60000);
  }

  track(reason, promise) {

```

```

const id = Symbol('rejection');
const timestamp = Date.now();
const error = this.normalizeError(reason);

this.rejections.set(id, {
  id,
  error,
  promise,
  timestamp,
  stack: error.stack,
  context: this.getContext()
});

logger.warn('Unhandled Rejection Tracked', {
  id: id.toString(),
  error: error.message,
  age: 0
});

// Отправляем метрики
metrics.increment('process.unhandledRejection', {
  errorType: error.name
});

return id;
}

markHandled(promise) {
  for (const [id, rejection] of this.rejections) {
    if (rejection.promise === promise) {
      const age = Date.now() - rejection.timestamp;

      this.rejections.delete(id);

      logger.info('Rejection Handled After Delay', {
        id: id.toString(),
        error: rejection.error.message,
        age: `${age}ms`
      });

      metrics.timing('process.rejection.handled', age);
      break;
    }
  }
}

```

```

}

checkStaleRejections() {
  const now = Date.now();

  for (const [id, rejection] of this.rejections) {
    const age = now - rejection.timestamp;

    if (age > this.criticalThreshold) {
      logger.error('Critical: Long-lived unhandled rejection', {
        id: id.toString(),
        error: rejection.error.message,
        age: `${age}ms`,
        context: rejection.context
      });

      // Отправляем алерт
      this.sendAlert('critical', rejection);
    } else if (age > this.warnThreshold) {
      logger.warn('Warning: Unhandled rejection still pending', {
        id: id.toString(),
        error: rejection.error.message,
        age: `${age}ms`
      });
    }
  }
}

// Метрики
metrics.gauge('process.unhandledRejections.count', this.rejections.size);
}

normalizeError(reason) {
  if (reason instanceof Error) {
    return reason;
  }

  if (typeof reason === 'string') {
    return new Error(reason);
  }

  if (reason && typeof reason === 'object') {
    const error = new Error(reason.message || 'Unknown rejection');
    error.original = reason;
  }
}

```

```

    return error;
}

return new Error(String(reason));
}

getContext() {
    return {
        pid: process.pid,
        memory: process.memoryUsage(),
        uptime: process.uptime(),
        versions: process.versions
    };
}

async sendAlert(severity, rejection) {
    if (!process.env.SLACK_WEBHOOK_URL) return;

    const message = {
        text: `❌ *${severity.toUpperCase()}: Unhandled Rejection*`,
        attachments: [{
            color: severity === 'critical' ? 'danger' : 'warning',
            fields: [
                {
                    title: 'Error',
                    value: rejection.error.message,
                    short: false
                },
                {
                    title: 'Age',
                    value: `${(Date.now() - rejection.timestamp) / 1000}s`,
                    short: true
                },
                {
                    title: 'PID',
                    value: String(process.pid),
                    short: true
                }
            ],
            footer: rejection.stack.split('\n')[1] || 'Unknown location'
        }]
    };

    try {

```

```

    await fetch(process.env.SLACK_WEBHOOK_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(message)
    });
  } catch (error) {
    logger.error('Failed to send Slack alert', error);
  }
}
}

const rejectionTracker = new UnhandledRejectionTracker({
  warnThreshold: 5000,      // 5 секунд
  criticalThreshold: 30000 // 30 секунд
});

```

2.3. Express Error Handling Middleware

```

javascript

// =====
// Express Global Error Handler
// =====

class ExpressErrorHandler {
  constructor(options = {}) {
    this.options = {
      logErrors: true,
      exposeStackTraces: process.env.NODE_ENV === 'development',
      includeRequestContext: true,
      ...options
    };
  }
}

// Middleware для обработки ошибок
handle() {
  return (err, req, res, next) => {
    // Пропускаем, если ответ уже отправлен
    if (res.headersSent) {
      return next(err);
    }

    // Нормализуем ошибку
    const error = this.normalizeError(err);
  }
}

```

```

// Обогащаем контекстом запроса
const enrichedError = this.enrichError(error, req);

// Логируем
if (this.options.logErrors) {
  this.logError(enrichedError, req);
}

// Отправляем в Sentry
this.captureError(enrichedError, req);

// Формируем ответ
const response = this.createErrorResponse(enrichedError, req);

res.status(response.statusCode).json(response.body);
};
}

normalizeError(err) {
  // Если это уже наша ошибка
  if (err instanceof AppError) {
    return err;
  }

  // HTTP ошибки
  if (err.status || err.statusCode) {
    return new HttpError(err.message, err.status || err.statusCode, {
      cause: err
    });
  }

  // Ошибки валидации
  if (err.name === 'ValidationError') {
    return new ValidationError(err.message, {
      fields: err.errors
    });
  }

  // Ошибки MongoDB
  if (err.name === 'MongoError' || err.name === 'MongoServerError') {
    if (err.code === 11000) {
      return new DuplicateResourceError(
        this.extractResourceFromMongo(err),

```

```

        this.extractCriteriaFromMongo(err),
        { cause: err }
    );
}
}

// Ошибки JWT
if (err.name === 'JsonWebTokenError') {
    return new AuthenticationError('Invalid token', { cause: err });
}

if (err.name === 'TokenExpiredError') {
    return new AuthenticationError('Token expired', { cause: err });
}

// Ошибки сети
if (err.code === 'ECONNREFUSED' || err.code === 'ETIMEDOUT') {
    return new NetworkError(err.address, { cause: err });
}

// Ошибки файловой системы
if (err.code === 'ENOENT') {
    return new NotFoundError('File', err.path, { cause: err });
}

// Неизвестная ошибка
return new InternalServerError('Internal server error', { cause: err });
}

enrichError(error, req) {
    if (!this.options.includeRequestContext) {
        return error;
    }

    error.requestContext = {
        id: req.id,
        method: req.method,
        url: req.url,
        path: req.path,
        params: req.params,
        query: req.query,
        body: this.sanitizeBody(req.body),
        headers: this.sanitizeHeaders(req.headers),
        ip: req.ip,
    };
}

```



```

    ips: req.ips,
    userAgent: req.get('user-agent'),
    referer: req.get('referrer'),
    timestamp: new Date().toISOString()
  };

  if (req.user) {
    error.userContext = {
      id: req.user.id,
      email: req.user.email,
      role: req.user.role
    };
  }

  return error;
}

sanitizeBody(body) {
  if (!body) return body;

  const sanitized = { ...body };
  const sensitiveFields = ['password', 'token', 'secret', 'creditCard'];

  sensitiveFields.forEach(field => {
    if (sanitized[field]) {
      sanitized[field] = '[FILTERED]';
    }
  });

  return sanitized;
}

sanitizeHeaders(headers) {
  const sanitized = { ...headers };
  const sensitiveHeaders = ['authorization', 'cookie', 'x-api-key'];

  sensitiveHeaders.forEach(header => {
    if (sanitized[header]) {
      sanitized[header] = '[FILTERED]';
    }
  });

  return sanitized;
}

```

```

logError(error, req) {
  const logEntry = {
    timestamp: new Date().toISOString(),
    level: error.severity || 'error',
    type: error.name,
    message: error.message,
    code: error.code,
    statusCode: error.statusCode || 500,
    requestId: req.id,
    userId: req.user?.id,
    path: req.path,
    method: req.method,
    ip: req.ip
  };

  if (error.stack && this.options.exposeStackTraces) {
    logEntry.stack = error.stack;
  }

  if (error.context) {
    logEntry.context = error.context;
  }

  if (error.cause) {
    logEntry.cause = {
      name: error.cause.name,
      message: error.cause.message,
      stack: error.cause.stack
    };
  }

  if (error.severity === 'critical') {
    logger.critical(logEntry);
  } else if (error.severity === 'error') {
    logger.error(logEntry);
  } else if (error.severity === 'warning') {
    logger.warn(logEntry);
  } else {
    logger.info(logEntry);
  }
}

captureError(error, req) {

```

```

Sentry.withScope(scope => {
  scope.setTag('request_id', req.id);
  scope.setTag('user_id', req.user?.id);
  scope.setTag('path', req.path);
  scope.setTag('method', req.method);

  scope.setContext('request', {
    url: req.url,
    headers: this.sanitizeHeaders(req.headers),
    query: req.query,
    params: req.params
  });

  if (req.user) {
    scope.setUser({
      id: req.user.id,
      email: req.user.email
    });
  }

  Sentry.captureException(error);
});
}

createErrorResponse(error, req) {
  const statusCode = error.statusCode || error.status || 500;
  const isDevelopment = process.env.NODE_ENV === 'development';

  const response = {
    success: false,
    error: {
      code: error.code || 'ERR_UNKNOWN',
      message: error.message || 'Internal server error'
    }
  };

  // Добавляем детали для определенных типов ошибок
  if (error.fields) {
    response.error.fields = error.fields;
  }

  if (error.resource) {
    response.error.resource = error.resource;
    response.error.resourceId = error.resourceId;
  }
}

```

```

}

// Добавляем стек в development
if (isDevelopment && error.stack) {
  response.error.stack = error.stack.split('\n');
  response.error.context = error.context;
  response.error.cause = error.cause?.message;
}

// Учитываем Accept-Language
const locale = req.acceptsLanguages(['ru', 'en']) || 'en';
if (error.translate) {
  response.error.message = error.translate(locale);
}

return {
  statusCode,
  body: response
};
}

// Middleware для обработки 404
notFound() {
  return (req, res, next) => {
    const error = new NotFoundError('Route', `${req.method} ${req.path}`);
    next(error);
  };
}

// Middleware для async обработчиков
wrapAsync(fn) {
  return (req, res, next) => {
    fn(req, res, next).catch(next);
  };
}

const expressErrorHandler = new ExpressErrorHandler({
  logErrors: true,
  exposeStackTraces: process.env.NODE_ENV === 'development',
  includeRequestContext: true
});

// Использование в Express

```

```

app.use(expressErrorHandler.notFound());
app.use(expressErrorHandler.handle());

// Для async функций
app.get('/users/:id', expressErrorHandler.wrapAsync(async (req, res) => {
  const user = await userService.findById(req.params.id);
  res.json(user);
}));

```

Часть 3: Системы мониторинга и отчетности

3.1. Sentry Integration

```

javascript

// =====
// Sentry Global Error Tracking
// =====

import * as Sentry from '@sentry/node';
import { ProfilingIntegration } from '@sentry/profiling-node';

class SentryService {
  constructor(options = {}) {
    this.initialized = false;
    this.options = {
      dsn: process.env.SENTRY_DSN,
      environment: process.env.NODE_ENV || 'development',
      release: process.env.npm_package_version,
      tracesSampleRate: 0.2,
      profilesSampleRate: 0.2,
      ...options
    };
  }

  init() {
    if (!this.options.dsn) {
      console.warn('Sentry DSN not provided, skipping initialization');
      return;
    }
  }

  Sentry.init({

```

```

dsn: this.options.dsn,
environment: this.options.environment,
release: this.options.release,
tracesSampleRate: this.options.tracesSampleRate,
profilesSampleRate: this.options.profilesSampleRate,

// Интеграции
integrations: [
  new ProfilingIntegration(),
  new Sentry.Integrations.Http({ tracing: true }),
  new Sentry.Integrations.Express({ app }),
  new Sentry.Integrations.GraphQL(),
  new Sentry.Integrations.Mongo(),
  new Sentry.Integrations.Postgres()
],

// Настройки
maxBreadcrumbs: 50,
debug: this.options.environment === 'development',
attachStacktrace: true,

// Фильтрация ошибок
beforeSend: this.beforeSend.bind(this),
beforeBreadcrumb: this.beforeBreadcrumb.bind(this)
});

this.initialized = true;
console.log('🟢 Sentry initialized');
}

beforeSend(event, hint) {
  const error = hint?.originalException;

  // Фильтруем определенные ошибки
  if (error instanceof ValidationError) {
    // Не отправляем ошибки валидации в Sentry
    return null;
  }

  if (error?.message?.includes('ResizeObserver')) {
    return null;
  }

  // Удаляем чувствительные данные

```

```

if (event.request?.headers) {
  delete event.request.headers.authorization;
  delete event.request.headers.cookie;
}

if (event.user) {
  // Хэшируем email
  if (event.user.email) {
    event.user.email = this.hashEmail(event.user.email);
  }
}

// Добавляем кастомные теги
event.tags = {
  ...event.tags,
  app_version: this.options.release,
  node_version: process.version
};

return event;
}

beforeBreadcrumb(breadcrumb, hint) {
  // Фильтруем breadcrumbs для консольных логов
  if (breadcrumb.category === 'console' && breadcrumb.level === 'debug') {
    return null;
  }

  // Анонимизируем URL
  if (breadcrumb.data?.url) {
    breadcrumb.data.url = this.anonymizeUrl(breadcrumb.data.url);
  }

  return breadcrumb;
}

captureException(error, context = {}) {
  if (!this.initialized) return;

  Sentry.withScope(scope => {
    // Добавляем теги
    Object.entries(context.tags || {}).forEach(([key, value]) => {
      scope.setTag(key, value);
    });
  });
}

```

```

// Добавляем контекст
Object.entries(context.contexts || {}).forEach(([key, value]) => {
  scope.setContext(key, value);
});

// Устанавливаем пользователя
if (context.user) {
  scope.setUser({
    id: context.user.id,
    email: this.hashEmail(context.user.email),
    ip_address: null // Не сохраняем IP
  });
}

// Уровень серьезности
if (context.level) {
  scope.setLevel(context.level);
}

// Fingerprint для группировки
if (context.fingerprint) {
  scope.setFingerprint(context.fingerprint);
}

Sentry.captureException(error);
});
}

captureMessage(message, level = 'info', context = {}) {
  if (!this.initialized) return;

  Sentry.withScope(scope => {
    scope.setLevel(level);

    Object.entries(context.tags || {}).forEach(([key, value]) => {
      scope.setTag(key, value);
    });

    Sentry.captureMessage(message);
  });
}

hashEmail(email) {

```



```

    if (!email) return null;
    return require('crypto')
      .createHash('sha256')
      .update(email.toLowerCase())
      .digest('hex')
      .substring(0, 16);
  }

  anonymizeUrl(url) {
    try {
      const parsed = new URL(url);
      return `${parsed.protocol}://${parsed.hostname}${parsed.pathname}`;
    } catch {
      return url;
    }
  }

  // Express middleware
  requestHandler() {
    return Sentry.Handlers.requestHandler();
  }

  tracingHandler() {
    return Sentry.Handlers.tracingHandler();
  }

  errorHandler() {
    return Sentry.Handlers.errorHandler();
  }
}

const sentry = new SentryService({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  release: process.env.npm_package_version,
  tracesSampleRate: 0.1
});

sentry.init();

// Использование в Express
app.use(sentry.requestHandler());
app.use(sentry.tracingHandler());

```

```
// Позже в коде
app.use(sentry.errorHandler());
```

3.2. Datadog APM Integration

```
javascript

// =====
// Datadog APM Integration
// =====

const tracer = require('dd-trace').init({
  service: 'my-app',
  env: process.env.NODE_ENV,
  version: process.env.npm_package_version,
  logInjection: true,
  runtimeMetrics: true,
  profiling: true,
  appsec: true,

  // Трекинг ошибок
  enableErrorTracking: true,

  // Интеграции
  plugins: {
    express: true,
    http: true,
    mongodb: true,
    redis: true,
    graphql: true
  }
});

class DatadogService {
  constructor() {
    this.tracer = tracer;
  }

  // Кастомный спан для отслеживания операций
  traceOperation(name, options = {}) {
    const span = tracer.startSpan(name, {
      childOf: options.parent,
      tags: {
        'resource.name': options.resource,

```

```

    'span.kind': options.kind || 'server',
    ...options.tags
  }
});

return {
  span,
  setTag: (key, value) => span.setTag(key, value),
  setError: (error) => span.setTag('error', error),
  finish: () => span.finish()
};
}

// Трекинг ошибок
trackError(error, context = {}) {
  const span = tracer.scope().active();

  if (span) {
    span.setTag('error', error);

    // Добавляем кастомные теги
    if (error.code) {
      span.setTag('error.code', error.code);
    }

    if (error.statusCode) {
      span.setTag('http.status_code', error.statusCode);
    }

    if (error.severity) {
      span.setTag('error.severity', error.severity);
    }
  }
}

// Отправляем метрику
tracer.dogstatsd.increment('errors.total', 1, {
  type: error.name,
  service: process.env.npm_package_name
});
}

// Мониторинг производительности
monitorPerformance() {
  // Метрики памяти

```

```

setInterval(() => {
  const mem = process.memoryUsage();
  tracer.dogstatsd.gauge('memory.heap_total', mem.heapTotal);
  tracer.dogstatsd.gauge('memory.heap_used', mem.heapUsed);
  tracer.dogstatsd.gauge('memory.rss', mem.rss);
  tracer.dogstatsd.gauge('memory.external', mem.external);
}, 10000);

// Mempuku event Loop
setInterval(() => {
  const start = Date.now();
  setImmediate(() => {
    const delay = Date.now() - start;
    tracer.dogstatsd.gauge('event_loop.delay', delay);
  });
}, 1000);

// Mempuku GC
if (global.gc) {
  setInterval(() => {
    const before = process.memoryUsage();
    global.gc();
    const after = process.memoryUsage();

    tracer.dogstatsd.gauge('gc.freed', before.heapUsed - after.heapUsed);
  }, 60000);
}
}

const datadog = new DatadogService();

// Express middleware
app.use((req, res, next) => {
  const span = tracer.startSpan('express.request', {
    tags: {
      'http.method': req.method,
      'http.url': req.url,
      'span.type': 'web'
    }
  });
});

tracer.scope().activate(span, () => {
  res.on('finish', () => {

```

```

span.setTag('http.status_code', res.statusCode);

if (res.statusCode >= 500) {
  span.setTag('error', true);
}

span.finish();
});

next();
});
});

```

3.3. Логирование ошибок

```

javascript

// =====
// Winston Logger with Error Handling
// =====

const winston = require('winston');
const { ElasticsearchTransport } = require('winston-elasticsearch');

class ErrorLogger {
  constructor(options = {}) {
    this.logger = this.createLogger(options);
    this.errorCount = 0;
    this.lastReset = Date.now();
  }

  createLogger(options) {
    const transports = [
      // Консольный транспорт
      new winston.transports.Console({
        format: winston.format.combine(
          winston.format.colorize(),
          winston.format.timestamp(),
          winston.format.printf(({ timestamp, level, message, ...meta }) => {
            return `${timestamp} [${level}]: ${message} ${
              Object.keys(meta).length ? JSON.stringify(meta, null, 2) : ''
            }`;
          })
        )
      })
    ]
  }
}

```

```
}},
```

```
// Файловый транспорт для ошибок
```

```
new winston.transports.File({  
  filename: 'logs/error.log',  
  level: 'error',  
  maxsize: 10485760, // 10MB  
  maxFiles: 5,  
  tailable: true,  
  format: winston.format.combine(  
    winston.format.timestamp(),  
    winston.format.json()  
  )  
}),
```

```
// Файловый транспорт для всех логов
```

```
new winston.transports.File({  
  filename: 'logs/combined.log',  
  maxsize: 10485760,  
  maxFiles: 5,  
  format: winston.format.combine(  
    winston.format.timestamp(),  
    winston.format.json()  
  )  
})  
];
```

```
// Elasticsearch транспорт для production
```

```
if (options.elasticsearch && process.env.NODE_ENV === 'production') {  
  transports.push(  
    new ElasticsearchTransport({  
      level: 'error',  
      clientOpts: {  
        node: options.elasticsearch.node,  
        auth: {  
          username: options.elasticsearch.username,  
          password: options.elasticsearch.password  
        }  
      },  
      index: 'logs-errors',  
      format: winston.format.combine(  
        winston.format.timestamp(),  
        winston.format.json()  
      )  
    })  
  );  
}
```

```

    })
  );
}

return winston.createLogger({
  level: options.level || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.splat(),
    winston.format.json()
  ),
  defaultMeta: {
    service: options.service || 'unknown',
    environment: process.env.NODE_ENV
  },
  transports,
  exceptionHandlers: [
    new winston.transports.File({ filename: 'logs/exceptions.log' })
  ],
  rejectionHandlers: [
    new winston.transports.File({ filename: 'logs/rejections.log' })
  ]
});
}

```

```

error(error, meta = {}) {
  this.errorCount++;

  // Проверяем rate limit
  if (this.isRateLimited()) {
    if (this.errorCount === this.options.rateLimit + 1) {
      this.logger.warn('Error rate limit exceeded', {
        count: this.errorCount,
        window: this.options.rateWindow || 60000
      });
    }
    return;
  }
}

```

```

const logEntry = {
  message: error.message || String(error),
  name: error.name,
  stack: error.stack,

```

```
    code: error.code,
    statusCode: error.statusCode,
    severity: error.severity,
    ...meta
  };

  if (error.context) {
    logEntry.context = error.context;
  }

  if (error.cause) {
    logEntry.cause = {
      name: error.cause.name,
      message: error.cause.message,
      stack: error.cause.stack
    };
  }

  if (error.requestContext) {
    logEntry.request = error.requestContext;
  }

  if (error.userContext) {
    logEntry.user = {
      id: error.userContext.id,
      role: error.userContext.role
    };
  }

  this.logger.error(logEntry);
}

warn(message, meta = {}) {
  this.logger.warn(message, meta);
}

info(message, meta = {}) {
  this.logger.info(message, meta);
}

debug(message, meta = {}) {
  this.logger.debug(message, meta);
}
```



```

isRateLimited() {
  const now = Date.now();
  const window = this.options.rateWindow || 60000; // 1 минута

  if (now - this.lastReset > window) {
    this.errorCount = 0;
    this.lastReset = now;
  }

  return this.errorCount > (this.options.rateLimit || 100);
}

async query(options = {}) {
  const queryOptions = {
    from: options.from || new Date(Date.now() - 24 * 60 * 60 * 1000),
    until: options.until || new Date(),
    limit: options.limit || 100,
    start: options.start || 0,
    order: options.order || 'desc',
    fields: options.fields
  };

  return new Promise((resolve, reject) => {
    this.logger.query(queryOptions, (err, results) => {
      if (err) {
        reject(err);
      } else {
        resolve(results);
      }
    });
  });
}

async stream(options = {}) {
  const stream = this.logger.stream({
    start: options.start || -10,
    limit: options.limit || 10
  });

  return new Promise((resolve) => {
    const entries = [];

    stream.on('log', (log) => {
      entries.push(log);
    });
  });
}

```

```

    });

    stream.on('end', () => {
        resolve(entries);
    });
    });
}
}

const logger = new ErrorLogger({
    service: 'my-api-service',
    level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
    rateLimit: 1000,
    rateWindow: 60000,
    elasticsearch: process.env.ELASTICSEARCH_URL ? {
        node: process.env.ELASTICSEARCH_URL,
        username: process.env.ELASTICSEARCH_USERNAME,
        password: process.env.ELASTICSEARCH_PASSWORD
    } : null
});

// Глобальные обработчики
process.on('uncaughtException', (error) => {
    logger.error(error, { type: 'uncaughtException' });
    process.exit(1);
});

process.on('unhandledRejection', (reason) => {
    const error = reason instanceof Error ? reason : new Error(String(reason));
    logger.error(error, { type: 'unhandledRejection' });
});

module.exports = logger;

```

Часть 4: Стратегии восстановления

4.1. Graceful Degradation

```

javascript

// =====
// Graceful Degradation Strategies

```

```
// =====
```

```
class DegradationStrategy {
  constructor(options = {}) {
    this.features = new Map();
    this.listeners = new Set();
    this.fallbackServices = new Map();

    this.options = {
      autoRestore: options.autoRestore || true,
      restoreDelay: options.restoreDelay || 60000,
      metricsEnabled: options.metricsEnabled || true,
      ...options
    };
  }
}
```

```
// Регистрация функциональности
```

```
registerFeature(name, options = {}) {
  this.features.set(name, {
    name,
    status: 'healthy',
    fallback: options.fallback,
    critical: options.critical || false,
    dependencies: options.dependencies || [],
    degradationHandler: options.degradationHandler,
    restoreHandler: options.restoreHandler,
    errors: 0,
    lastError: null,
    degradedAt: null
  });

  console.log(`  Registered feature: ${name}`);
}
```

```
// Обработка ошибки в функциональности
```

```
handleError(name, error) {
  const feature = this.features.get(name);

  if (!feature) {
    console.warn(`Unknown feature: ${name}`);
    return;
  }

  feature.errors++;
}
```

```

feature.lastError = error;

console.warn(`⚠ Error in feature ${name}: ${error.message}`);

// Пороговая логика
if (feature.status === 'healthy' && this.shouldDegrade(feature)) {
  this.degradeFeature(name);
}

// Отправка метрик
if (this.options.metricsEnabled) {
  metrics.increment('feature.errors', { feature: name });
}

// Уведомление слушателей
this.notifyListeners('error', { feature, error });
}

shouldDegrade(feature) {
  // Дegrадируем после 5 ошибок за последнюю минуту
  const errorsInLastMinute = feature.errors;
  return errorsInLastMinute >= 5;
}

// Дegrадация функциональности
degradeFeature(name) {
  const feature = this.features.get(name);

  if (feature.status === 'degraded') {
    return;
  }

  feature.status = 'degraded';
  feature.degradedAt = new Date();

  console.warn(`⚠ Degrading feature: ${name}`);

  // Вызов обработчика деградации
  if (feature.degradationHandler) {
    try {
      feature.degradationHandler(feature);
    } catch (error) {
      console.error(`Error in degradation handler for ${name}:`, error);
    }
  }
}

```

```
}
```

```
// Автоматическое восстановление
```

```
if (this.options.autoRestore) {  
  setTimeout(() => {  
    this.restoreFeature(name);  
  }, this.options.restoreDelay);  
}
```

```
// Каскадная деградация зависимостей
```

```
this.degradeDependencies(name);  
  
this.notifyListeners('degraded', { feature });  
}
```

```
// Восстановление функциональности
```

```
restoreFeature(name) {  
  const feature = this.features.get(name);  
  
  if (feature.status !== 'degraded') {  
    return;  
  }  
  
  console.log(`🔄 Attempting to restore feature: ${name}`);
```

```
// Проверка здоровья
```

```
this.checkHealth(name)  
  .then((isHealthy) => {  
    if (isHealthy) {  
      feature.status = 'healthy';  
      feature.errors = 0;  
      feature.degradedAt = null;  
  
      console.log(`✅ Restored feature: ${name}`);  
  
      if (feature.restoreHandler) {  
        feature.restoreHandler(feature);  
      }  
  
      this.restoreDependencies(name);  
      this.notifyListeners('restored', { feature });  
    } else {  
      // Повторная попытка позже  
      setTimeout(() => {
```

```

        this.restoreFeature(name);
    }, this.options.restoreDelay / 2);
}
})
.catch((error) => {
    console.error(`Health check failed for ${name}:`, error);
});
}

```

```

async checkHealth(name) {
    const feature = this.features.get(name);

    if (feature.degradationHandler) {
        try {
            const result = await feature.degradationHandler.check?();
            return result === true;
        } catch {
            return false;
        }
    }
}

```

```

// Базовая проверка
return feature.errors === 0;
}

```

```

degradeDependencies(name) {
    const feature = this.features.get(name);

    feature.dependencies.forEach(depName => {
        const dep = this.features.get(depName);

        if (dep && dep.status === 'healthy' && dep.critical) {
            this.degradeFeature(depName);
        }
    });
}

```

```

restoreDependencies(name) {
    const feature = this.features.get(name);

    feature.dependencies.forEach(depName => {
        const dep = this.features.get(depName);

        if (dep && dep.status === 'degraded' && dep.critical) {

```

```

        this.restoreFeature(depName);
    }
});
}

// Регистрация fallback сервиса
registerFallback(serviceName, fallbackFn) {
    this.fallbackServices.set(serviceName, fallbackFn);
    console.log(`    Registered fallback for: ${serviceName}`);
}

// Выполнение с fallback
async withFallback(serviceName, primaryFn, fallbackFn = null) {
    const feature = this.features.get(serviceName);

    // Если сервис деградирован, используем fallback
    if (feature?.status === 'degraded') {
        return this.executeFallback(serviceName, fallbackFn);
    }

    try {
        const result = await primaryFn();

        // Успешно - сбрасываем счетчик ошибок
        if (feature) {
            feature.errors = 0;
        }

        return result;
    } catch (error) {
        this.handleError(serviceName, error);

        // Используем fallback
        return this.executeFallback(serviceName, fallbackFn);
    }
}

async executeFallback(serviceName, fallbackFn = null) {
    const fallback = fallbackFn || this.fallbackServices.get(serviceName);

    if (fallback) {
        console.log(`    Using fallback for: ${serviceName}`);

        try {

```

```
    return await fallback();
  } catch (fallbackError) {
    console.error(`Fallback failed for ${serviceName}`, fallbackError);
    throw fallbackError;
  }
}
```

```
throw new Error(`No fallback available for ${serviceName}`);
}
```

// Добавление слушателя событий

```
addListener(callback) {
  this.listeners.add(callback);
  return () => this.listeners.delete(callback);
}
```

```
notifyListeners(event, data) {
  this.listeners.forEach(callback => {
    try {
      callback(event, data);
    } catch (error) {
      console.error('Error in listener:', error);
    }
  });
}
```

// Получение статуса всех функций

```
getStatus() {
  const status = {
    healthy: [],
    degraded: [],
    critical: []
  };

  this.features.forEach(feature => {
    status[feature.status].push(feature.name);

    if (feature.critical && feature.status === 'degraded') {
      status.critical.push(feature.name);
    }
  });

  return status;
}
```



```
}
```

```
const degradationStrategy = new DegradationStrategy({  
  autoRestore: true,  
  restoreDelay: 30000,  
  metricsEnabled: true  
});
```

```
// Регистрация функций
```

```
degradationStrategy.registerFeature('payment-processing', {  
  critical: true,  
  fallback: () => ({  
    success: false,  
    message: 'Payment service temporarily unavailable'  
  }),  
  degradationHandler: (feature) => {  
    console.log('Payment processing degraded, switching to manual approval');  
    // Переключаем на ручную обработку платежей  
  }  
});
```

```
degradationStrategy.registerFeature('recommendations', {  
  critical: false,  
  dependencies: ['analytics'],  
  fallback: () => {  
    // Возвращаем популярные товары вместо персонализированных  
    return getPopularProducts();  
  }  
});
```

```
degradationStrategy.registerFeature('analytics', {  
  critical: false  
});
```

```
// Использование в сервисе
```

```
class PaymentService {  
  async processPayment(order) {  
    return degradationStrategy.withFallback(  
      'payment-processing',  
      async () => {  
        const result = await paymentGateway.charge(order);  
        return { success: true, result };  
      },  
      async () => {
```

```

    // Fallback - сохраняем для ручной обработки
    await manualPaymentQueue.add(order);
    return {
      success: false,
      message: 'Payment will be processed manually',
      reference: order.id
    };
  }
);
}
}

```

4.2. Circuit Breaker Pattern

```

javascript

// =====
// Circuit Breaker Pattern
// =====

class CircuitBreaker {
  constructor(options = {}) {
    this.failureThreshold = options.failureThreshold || 5;
    this.successThreshold = options.successThreshold || 2;
    this.timeout = options.timeout || 30000;
    this.resetTimeout = options.resetTimeout || 60000;

    this.state = 'CLOSED'; // CLOSED, OPEN, HALF_OPEN
    this.failureCount = 0;
    this.successCount = 0;
    this.lastFailureTime = null;
    this.nextAttemptTime = null;

    this.metrics = {
      totalCalls: 0,
      successfulCalls: 0,
      failedCalls: 0,
      rejectedCalls: 0,
      timeouts: 0
    };
  }

  async execute(fn, fallback = null) {
    this.metrics.totalCalls++;
  }
}

```

```

if (!this.canExecute()) {
  this.metrics.rejectedCalls++;
  return this.handleRejection(fallback);
}

try {
  const timeoutPromise = this.createTimeout();
  const result = await Promise.race([fn(), timeoutPromise]);

  this.onSuccess();
  return result;
} catch (error) {
  this.onFailure(error);

  if (fallback) {
    return fallback(error);
  }

  throw error;
}

canExecute() {
  if (this.state === 'CLOSED') {
    return true;
  }

  if (this.state === 'OPEN') {
    if (Date.now() >= this.nextAttemptTime) {
      this.state = 'HALF_OPEN';
      this.successCount = 0;
      console.log('🔒 Circuit breaker: HALF_OPEN');
      return true;
    }
    return false;
  }

  return this.state === 'HALF_OPEN';
}

onSuccess() {
  this.metrics.successfulCalls++;
  this.failureCount = 0;

```

```

    this.lastFailureTime = null;

    if (this.state === 'HALF_OPEN') {
        this.successCount++;

        if (this.successCount >= this.successThreshold) {
            this.state = 'CLOSED';
            this.successCount = 0;
            console.log('🔌 Circuit breaker: CLOSED');
        }
    }
}

onFailure(error) {
    this.metrics.failedCalls++;
    this.failureCount++;
    this.lastFailureTime = Date.now();

    console.warn(`⚠️ Circuit breaker failure (${this.failureCount}/${this.failureThreshold}):`, error.message);

    if (this.state === 'HALF_OPEN' ||
        (this.state === 'CLOSED' && this.failureCount >= this.failureThreshold)) {
        this.trip();
    }
}

trip() {
    this.state = 'OPEN';
    this.nextAttemptTime = Date.now() + this.resetTimeout;

    console.error(`🛑 Circuit breaker: OPEN (reset in ${this.resetTimeout}ms)`);

    // Отправка алерта
    this.sendAlert();
}

createTimeout() {
    return new Promise((_, reject) => {
        setTimeout(() => {
            this.metrics.timeouts++;
            reject(new Error(`Circuit breaker timeout after ${this.timeout}ms`));
        }, this.timeout);
    });
}

```

```

}

handleRejection(fallback) {
  console.warn('🟡 Circuit breaker: Rejected execution');

  if (fallback) {
    return fallback(new Error('Circuit breaker is OPEN'));
  }

  throw new Error('Circuit breaker is OPEN');
}

async sendAlert() {
  if (!process.env.SLACK_WEBHOOK_URL) return;

  const message = {
    text: '🔴 *Circuit Breaker Opened*',
    attachments: [{
      color: 'danger',
      fields: [
        {
          title: 'Service',
          value: this.service || 'unknown',
          short: true
        },
        {
          title: 'Failures',
          value: String(this.failureCount),
          short: true
        },
        {
          title: 'Reset Time',
          value: `${this.resetTimeout / 1000}s`,
          short: true
        },
        {
          title: 'Metrics',
          value: `Total: ${this.metrics.totalCalls}, ` +
            `Success: ${this.metrics.successfulCalls}, ` +
            `Fail: ${this.metrics.failedCalls}`,
          short: false
        }
      ]
    }
  ]
}

```

```

};

try {
  await fetch(process.env.SLACK_WEBHOOK_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(message)
  });
} catch (error) {
  console.error('Failed to send Slack alert:', error);
}
}

getStatus() {
  return {
    state: this.state,
    failureCount: this.failureCount,
    successCount: this.successCount,
    lastFailureTime: this.lastFailureTime,
    nextAttemptTime: this.nextAttemptTime,
    metrics: this.metrics
  };
}

}

// Service c Circuit Breaker
class ResilientService {
  constructor(options = {}) {
    this.circuitBreaker = new CircuitBreaker({
      failureThreshold: options.failureThreshold || 5,
      successThreshold: options.successThreshold || 2,
      timeout: options.timeout || 5000,
      resetTimeout: options.resetTimeout || 60000
    });

    this.serviceName = options.serviceName;
  }

  async call(method, ...args) {
    return this.circuitBreaker.execute(
      () => this[method](...args),
      (error) => this.fallback(method, error, args)
    );
  }
}

```

```

async fallback(method, error, args) {
  console.warn(`Fallback for ${method}:`, error.message);

  return {
    success: false,
    error: error.message,
    fallback: true,
    timestamp: new Date().toISOString()
  };
}
}

// API клиент с Circuit Breaker
class APIClient extends ResilientService {
  constructor(baseUrl, options = {}) {
    super({
      serviceName: 'api-client',
      failureThreshold: 3,
      timeout: 10000,
      ...options
    });

    this.baseUrl = baseUrl;
  }

  async get(endpoint) {
    const response = await fetch(`${this.baseUrl}${endpoint}`);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }

    return response.json();
  }

  async post(endpoint, data) {
    const response = await fetch(`${this.baseUrl}${endpoint}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });

    if (!response.ok) {

```

```

    throw new Error(`HTTP ${response.status}: ${response.statusText}`);
  }

  return response.json();
}
}

// Использование
const apiClient = new APIClient('https://api.example.com');

app.get('/users/:id', async (req, res) => {
  try {
    const user = await apiClient.call('get', `/users/${req.params.id}`);
    res.json(user);
  } catch (error) {
    res.status(503).json({
      error: 'Service temporarily unavailable',
      fallback: error.fallback
    });
  }
});

// Мониторинг состояния
app.get('/status/circuit-breaker', (req, res) => {
  res.json({
    api: apiClient.circuitBreaker.getStatus()
  });
});

```

Часть 5: Интеграция и конфигурация

5.1. Полная система глобальной обработки ошибок

```

javascript

// =====
// Complete Global Error Handling System
// =====

class GlobalErrorSystem {
  constructor(options = {}) {
    this.options = {

```



```

environment: process.env.NODE_ENV || 'development',
serviceName: process.env.npm_package_name || 'unknown',
version: process.env.npm_package_version || '1.0.0',
sentry: {
  enabled: true,
  dsn: process.env.SENTRY_DSN
},
datadog: {
  enabled: true,
  apiKey: process.env.DATADOG_API_KEY
},
logging: {
  enabled: true,
  level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
  elasticsearch: process.env.ELASTICSEARCH_URL
},
recovery: {
  autoRestore: true,
  restoreDelay: 30000,
  circuitBreakerThreshold: 5
},
...options
};

this.components = new Map();
this.isInitialized = false;
}

async initialize() {
  console.log('🔧 Initializing Global Error System...');

  // 1. Инициализация логгера
  if (this.options.logging.enabled) {
    this.components.set('logger', new ErrorLogger({
      service: this.options.serviceName,
      level: this.options.logging.level,
      elasticsearch: this.options.logging.elasticsearch
    }));
    console.log('✓ Logger initialized');
  }

  // 2. Инициализация Sentry
  if (this.options.sentry.enabled && this.options.sentry.dsn) {
    this.components.set('sentry', new SentryService({

```

```

    dsn: this.options.sentry.dsn,
    environment: this.options.environment,
    release: this.options.version
  ));
  this.components.get('sentry').init();
  console.log('✓ Sentry initialized');
}

// 3. Инициализация Datadog
if (this.options.datadog.enabled && this.options.datadog.apiKey) {
  this.components.set('datadog', new DatadogService());
  this.components.get('datadog').monitorPerformance();
  console.log('✓ Datadog initialized');
}

// 4. Инициализация стратегии восстановления
this.components.set('degradation', new DegradationStrategy({
  autoRestore: this.options.recovery.autoRestore,
  restoreDelay: this.options.recovery.restoreDelay
}));
console.log('✓ Degradation strategy initialized');

// 5. Установка глобальных обработчиков
this.setupGlobalHandlers();

this.isInitialized = true;
console.log('☐ Global Error System initialized');

return this;
}

setupGlobalHandlers() {
  // Browser handlers
  if (typeof window !== 'undefined') {
    this.setupBrowserHandlers();
  }

  // Node.js handlers
  if (typeof process !== 'undefined') {
    this.setupNodeHandlers();
  }
}

setupBrowserHandlers() {

```

```
window.onerror = (message, source, lineno, colno, error) => {
  this.handleError(error || new Error(message), {
    type: 'window.onerror',
    source,
    lineno,
    colno
  });
  return true;
};
```

```
window.addEventListener('unhandledrejection', (event) => {
  const error = event.reason instanceof Error
    ? event.reason
    : new Error(String(event.reason));

  this.handleError(error, { type: 'unhandledrejection' });
  event.preventDefault();
});
}
```

```
setupNodeHandlers() {
  process.on('uncaughtException', (error, origin) => {
    this.handleError(error, {
      type: 'uncaughtException',
      origin
    });
  });
```

```
  if (this.options.environment === 'production') {
    // Graceful shutdown
    this.gracefulShutdown(error).then(() => {
      process.exit(1);
    });
  }
});
```

```
process.on('unhandledRejection', (reason, promise) => {
  const error = reason instanceof Error
    ? reason
    : new Error(String(reason));

  this.handleError(error, {
    type: 'unhandledRejection',
    promise
  });
});
```

```

});

process.on('warning', (warning) => {
  this.handleWarning(warning);
});

process.on('exit', (code) => {
  this.handleExit(code);
});
}

handleError(error, context = {}) {
  const errorId = this.generateErrorId();
  const timestamp = new Date().toISOString();

  console.error(`❌ [${errorId}]`, error.message);

  // Обогащаем ошибку контекстом
  error.errorId = errorId;
  error.timestamp = timestamp;
  error.context = { ...error.context, ...context };

  // Логирование
  if (this.components.has('logger')) {
    this.components.get('logger').error(error, {
      errorId,
      timestamp,
      ...context
    });
  }

  // Sentry
  if (this.components.has('sentry')) {
    this.components.get('sentry').captureException(error, {
      tags: context,
      fingerprint: [error.name, context.type].filter(Boolean)
    });
  }

  // Datadog
  if (this.components.has('datadog')) {
    this.components.get('datadog').trackError(error, context);
  }
}

```

```

// Деградация для критических ошибок
if (error.critical || context.critical) {
  this.components.get('degradation')?.degradeFeature(context.service);
}

// Метрики
this.trackErrorMetrics(error, context);
}

handleWarning(warning) {
  console.warn('⚠️', warning.message);

  if (this.components.has('logger')) {
    this.components.get('logger').warn(warning.message, {
      name: warning.name,
      stack: warning.stack
    });
  }
}

handleExit(code) {
  const level = code === 0 ? 'info' : 'error';

  console.log(`Process exit with code ${code}`);

  if (this.components.has('logger')) {
    this.components.get('logger')[level]('Process exit', { code });
  }

  // Отправка последних метрик
  if (this.components.has('datadog')) {
    this.components.get('datadog').flushMetrics();
  }
}

async gracefulShutdown(error) {
  console.log('🛑 Starting graceful shutdown...');

  const shutdownTimeout = new Promise((_, reject) => {
    setTimeout(() => reject(new Error('Shutdown timeout')), 30000);
  });

  try {
    await Promise.race([

```

```

    this.performShutdown(),
    shutdownTimeout
  ]);

  console.log('✅ Graceful shutdown completed');
} catch (error) {
  console.error('❌ Graceful shutdown failed:', error);
}
}

async performShutdown() {
  // Закрываем HTTP сервер
  if (global.httpServer) {
    await new Promise((resolve) => {
      global.httpServer.close(resolve);
    });
    console.log('✓ HTTP server closed');
  }

  // Закрываем соединения с БД
  // ...

  // Ожидаем завершения активных запросов
  // ...

  // Флашим логи
  if (this.components.has('logger')) {
    await this.components.get('logger').end();
    console.log('✓ Logger closed');
  }

  // Флашим метрики
  if (this.components.has('datadog')) {
    await this.components.get('datadog').flushMetrics();
    console.log('✓ Metrics flushed');
  }
}

generateErrorId() {
  return `${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
}

trackErrorMetrics(error, context) {
  // Отправка метрик в Google Analytics

```

```

if (typeof gtag !== 'undefined') {
  gtag('event', 'exception', {
    description: error.message,
    fatal: context.critical || false
  });
}

// Кастомные метрики
if (typeof metrics !== 'undefined') {
  metrics.increment('errors.total', {
    type: error.name,
    service: this.options.serviceName,
    environment: this.options.environment
  });
}

// Express middleware
expressMiddleware() {
  return (req, res, next) => {
    // Добавляем requestId
    req.id = this.generateErrorId();

    // Добавляем обработчики в res.locals
    res.locals.errorSystem = this;
    res.locals.errorId = req.id;

    // Перехватываем отправку ответов для логирования
    const originalJson = res.json;
    res.json = function(data) {
      if (res.statusCode >= 400) {
        req.errorResponse = data;
      }
      return originalJson.call(this, data);
    };

    next();
  };
}

// Express error handler
errorHandler() {
  return (err, req, res, next) => {
    const errorId = req.id || this.generateErrorId();
  };
}

```

```

this.handleError(err, {
  type: 'express',
  requestId: errorId,
  path: req.path,
  method: req.method,
  userId: req.user?.id,
  ip: req.ip
});

// Не показываем детали в production
if (this.options.environment === 'production') {
  res.status(err.statusCode || 500).json({
    success: false,
    error: {
      code: err.code || 'ERR_INTERNAL',
      message: 'Internal server error',
      requestId: errorId
    }
  });
} else {
  res.status(err.statusCode || 500).json({
    success: false,
    error: {
      code: err.code || 'ERR_INTERNAL',
      message: err.message,
      stack: err.stack,
      requestId: errorId
    }
  });
}
};
}
}

// Создание и экспорт глобального экземпляра
const globalErrorSystem = new GlobalErrorSystem({
  environment: process.env.NODE_ENV,
  serviceName: process.env.npm_package_name,
  version: process.env.npm_package_version,
  sentry: {
    enabled: true,
    dsn: process.env.SENTRY_DSN
  },

```



```

datadog: {
  enabled: !!process.env.DATADOG_API_KEY,
  apiKey: process.env.DATADOG_API_KEY
},
logging: {
  enabled: true,
  level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
  elasticsearch: process.env.ELASTICSEARCH_URL
},
recovery: {
  autoRestore: true,
  restoreDelay: 30000,
  circuitBreakerThreshold: 5
}
});

// Инициализация при старте приложения
if (require.main === module) {
  globalErrorSystem.initialize().catch(console.error);
}

module.exports = globalErrorSystem;

```

Часть 6: Тестирование глобальной обработки ошибок

6.1. Unit тесты

```

javascript

// =====
// Testing Global Error Handling
// =====

import { expect } from 'chai';
import sinon from 'sinon';
import { GlobalErrorSystem } from './error-system';

describe('Global Error System', () => {
  let errorSystem;
  let loggerStub;
  let sentryStub;

```

```

beforeEach(() => {
  loggerStub = {
    error: sinon.stub(),
    warn: sinon.stub(),
    info: sinon.stub()
  };

  sentryStub = {
    captureException: sinon.stub()
  };

  errorSystem = new GlobalErrorSystem({
    environment: 'test',
    serviceName: 'test-service',
    logging: { enabled: false }, // Отключаем для тестов
    sentry: { enabled: false }
  });

  // Мокаем компоненты
  errorSystem.components.set('logger', loggerStub);
  errorSystem.components.set('sentry', sentryStub);
});

afterEach(() => {
  sinon.restore();
});

describe('handleError', () => {
  it('should generate error ID and timestamp', () => {
    const error = new Error('Test error');

    errorSystem.handleError(error);

    expect(error.errorId).to.be.a('string');
    expect(error.timestamp).to.be.a('string');
  });

  it('should log error to logger', () => {
    const error = new Error('Test error');

    errorSystem.handleError(error, { test: true });

    expect(loggerStub.error.calledOnce).to.be.true;
    expect(loggerStub.error.firstCall.args[0]).to.equal(error);
  });
});

```

```

    expect(loggerStub.error.firstCall.args[1]).to.include({ test: true });
  });

  it('should send error to Sentry', () => {
    const error = new Error('Test error');

    errorSystem.handleError(error, { critical: true });

    expect(sentryStub.captureException.calledOnce).to.be.true;
    expect(sentryStub.captureException.firstCall.args[0]).to.equal(error);
  });
});

describe('gracefulShutdown', () => {
  it('should close HTTP server', async () => {
    const closeStub = sinon.stub().yields();
    global.httpServer = { close: closeStub };

    await errorSystem.gracefulShutdown(new Error('Test'));

    expect(closeStub.calledOnce).to.be.true;
  });

  it('should handle timeout', async () => {
    const closeStub = sinon.stub().yields();
    global.httpServer = { close: closeStub };

    // Замедляем shutdown
    errorSystem.performShutdown = () => new Promise(resolve => setTimeout(resolve,
31000));

    try {
      await errorSystem.gracefulShutdown(new Error('Test'));
      expect.fail('Should throw timeout error');
    } catch (error) {
      expect(error.message).to.include('Shutdown timeout');
    }
  });
});

describe('Express middleware', () => {
  it('should add request ID', () => {
    const req = {};
    const res = {};

```

```

const next = sinon.stub();

errorSystem.expressMiddleware()(req, res, next);

expect(req.id).to.be.a('string');
expect(next.calledOnce).to.be.true;
});

it('should handle errors with proper response', () => {
  const req = { id: 'test-id' };
  const res = {
    status: sinon.stub().returnsThis(),
    json: sinon.stub()
  };
  const next = sinon.stub();
  const error = new Error('Test error');
  error.statusCode = 400;

  errorSystem.errorHandler()(error, req, res, next);

  expect(res.status.calledWith(400)).to.be.true;
  expect(res.json.calledWith({
    success: false,
    error: {
      code: 'ERR_INTERNAL',
      message: 'Test error',
      stack: error.stack,
      requestId: 'test-id'
    }
  })).to.be.true;
});
});
});

describe('Circuit Breaker', () => {
  let circuitBreaker;

  beforeEach(() => {
    circuitBreaker = new CircuitBreaker({
      failureThreshold: 3,
      successThreshold: 2,
      timeout: 100,
      resetTimeout: 1000
    });
  });
});

```

```

});

it('should start in CLOSED state', () => {
  expect(circuitBreaker.state).to.equal('CLOSED');
});

it('should open after failure threshold', async () => {
  const failingFn = async () => {
    throw new Error('Test error');
  };

  for (let i = 0; i < 3; i++) {
    try {
      await circuitBreaker.execute(failingFn);
    } catch (error) {
      // Ожидаем ошибку
    }
  }

  expect(circuitBreaker.state).to.equal('OPEN');
  expect(circuitBreaker.failureCount).to.equal(3);
});

it('should reject executions when OPEN', async () => {
  // Принудительно открываем
  circuitBreaker.state = 'OPEN';
  circuitBreaker.nextAttemptTime = Date.now() + 1000;

  const fn = async () => 'success';

  try {
    await circuitBreaker.execute(fn);
    expect.fail('Should throw error');
  } catch (error) {
    expect(error.message).to.include('Circuit breaker is OPEN');
  }
});

it('should transition to HALF_OPEN after reset timeout', async () => {
  circuitBreaker.state = 'OPEN';
  circuitBreaker.nextAttemptTime = Date.now() - 1000; // Уже прошло

  expect(circuitBreaker.canExecute()).to.be.true;
  expect(circuitBreaker.state).to.equal('HALF_OPEN');
});

```

```

});

it('should close after success threshold in HALF_OPEN', async () => {
  circuitBreaker.state = 'HALF_OPEN';

  const successFn = async () => 'success';

  for (let i = 0; i < 2; i++) {
    await circuitBreaker.execute(successFn);
  }

  expect(circuitBreaker.state).to.equal('CLOSED');
});
});

```

6.2. Интеграционные тесты

```

javascript

describe('Global Error System Integration', () => {
  let app;
  let request;
  let errorSystem;

  before(async () => {
    errorSystem = new GlobalErrorSystem({
      environment: 'test',
      serviceName: 'test-service',
      logging: { enabled: false },
      sentry: { enabled: false },
      datadog: { enabled: false }
    });

    await errorSystem.initialize();

    app = express();
    app.use(errorSystem.expressMiddleware());
    app.get('/error', () => {
      throw new Error('Test error');
    });
    app.get('/async-error', async () => {
      await Promise.reject(new Error('Async error'));
    });
    app.use(errorSystem.errorHandler());
  });
});

```

```
request = supertest(app);
});

it('should handle synchronous errors', async () => {
  const response = await request
    .get('/error')
    .expect(500);

  expect(response.body).to.have.property('success', false);
  expect(response.body.error).to.have.property('requestId');
  expect(response.body.error.message).to.equal('Test error');
});

it('should handle asynchronous errors', async () => {
  const response = await request
    .get('/async-error')
    .expect(500);

  expect(response.body.success).to.be.false;
  expect(response.body.error.message).to.include('Async error');
});

it('should handle 404 errors', async () => {
  const response = await request
    .get('/not-found')
    .expect(404);

  expect(response.body.success).to.be.false;
  expect(response.body.error.code).to.equal('ERR_NOT_FOUND');
});
});
```

Заключение

Ключевые принципы глобальной обработки ошибок

1. **Централизация** — Единое место для обработки всех ошибок
2. **Контекст** — Обогащение ошибок информацией о запросе и окружении
3. **Мониторинг** — Автоматический сбор и анализ ошибок
4. **Восстановление** — Graceful degradation и Circuit Breaker
5. **Безопасность** — Фильтрация чувствительных данных

6. **UX** — Понятные сообщения для пользователей

7. **Тестирование** — Покрытие сценариев с ошибками

8. **Производительность** — Минимальное влияние на производительность

Чек-лист для production

javascript

```
const productionChecklist = {  
  // 📌 Глобальные обработчики  
  uncaughtException: 'process.on("uncaughtException")',  
  unhandledRejection: 'process.on("unhandledRejection")',  
  windowError: 'window.onerror',  
  
  // 📌 Мониторинг  
  sentry: 'Sentry.io integration',  
  logging: 'Structured logging with Winston',  
  metrics: 'Datadog/Prometheus metrics',  
  
  // 📌 Восстановление  
  gracefulShutdown: 'Graceful shutdown on errors',  
  circuitBreaker: 'Circuit breaker for external services',  
  fallbacks: 'Fallback strategies for degraded services',  
  
  // 📌 Безопасность  
  sanitization: 'Filter sensitive data from logs',  
  stackTraces: 'Hide stack traces in production',  
  
  // 📌 Тестирование  
  errorTests: 'Tests for error scenarios',  
  chaosTesting: 'Chaos engineering experiments'  
};
```

Глобальная обработка ошибок — это не просто техническое решение, а фундаментальная часть архитектуры приложения. Хорошо спроектированная система обработки ошибок:

- Защищает пользователей от неожиданного поведения
- Помогает разработчикам быстро находить и исправлять проблемы
- Обеспечивает стабильность и надежность приложения
- Дает метрики для принятия решений о развитии продукта

Инвестируйте время в проектирование глобальной обработки ошибок на ранних этапах разработки. Это окупится многократно в процессе эксплуатации приложения.

Глава 14: Метaproграммирование и рефлексия

14.1. Объект Reflect и его методы

Эпиграф: «Программа, которая пишет программы, или программа, которая знает о себе всё».

Введение: Что такое метaproграммирование?

Представьте себе хирурга, который делает операцию самому себе. Или художника, рисующего картину о том, как он рисует картину. Это и есть мета — взгляд на систему со стороны, когда код начинает управлять кодом же.

Метaproграммирование (Meta-programming) — это техника (или возможность) писать программы, которые могут обрабатывать другие программы (или самих себя) как данные.

В JavaScript долгое время метaproграммирование было ограничено. Мы могли:

1. Проверять типы (`typeof`, `instanceof`).
2. Перебирать свойства (`for...in`).
3. Использовать `eval` (очень опасная техника, ныне почти не используется).

До появления ES6 (ECMAScript 2015) рефлексия (способность кода исследовать самого себя) была "слепой" — мы могли видеть свойства объекта, но не могли перехватывать и изменять фундаментальные операции языка (например, то, как именно происходит чтение свойства).

ES6 совершил революцию, добавив два ключевых игрока:

1. `Proxy` — ловушка. Объект, который может перехватывать и перенаправлять операции над другим объектом.
2. `Reflect` — зеркало. Единый интерфейс (API) для выполнения стандартных операций языка.

Аналогия для запоминания:

- **Proxy** — это охрана в банке. Она перехватывает посетителя (`[[Get]]`), проверяет у него пропуск и решает: пустить внутрь или отправить восвояси.
 - **Reflect** — это сам банковский служащий. Он просто выполняет операцию: открывает ячейку, кладет деньги, закрывает ячейку. Он не задает вопросов.
-

14.1. Объект Reflect и его методы

Проблема, которую решает Reflect

До ES6 у JavaScript не было единого стандарта для выполнения внутренних операций.

Пример 1: Сложение как функция, а не оператор

В языке есть оператор `+`. Вы не можете вызвать его как функцию. Не существует `Math.add(2, 3)`. Reflect решил эту проблему для поля объектов.

Пример 2: Неоднозначность `in` и `delete`

Оператор `in` проверяет наличие свойства. Оператор `delete` удаляет свойство.

```
javascript
const obj = { a: 1 };
console.log('a' in obj); // true (окей)
delete obj.a; // Окей
```

Какая проблема?

1. Операторы нельзя передавать как колбэки (функции высшего порядка).
2. Операторы нельзя использовать в функциональном программировании так же удобно, как функции.
3. При работе с **Proxy** нам нужен был стандартный способ вызвать "оригинальное" поведение объекта изнутри ловушки.

Решение ES6:

Сделать все методы, которые есть в прототипе объекта (и операторы), доступными в виде статических методов единого объекта — **Reflect**.

14.1.1. Философия Reflect: «Все операции — функции»

Reflect — это не конструктор (вызвать `new Reflect()` нельзя). Это просто глобальный объект, хранящий набор статических методов.

Главное правило:

Любая операция, которая есть в языке для работы с объектами, дублирована методом Reflect, и этот метод возвращает более предсказуемый результат.

Сравнение «Оператор vs Функция»:

Оператор (старый способ)	Метод Reflect (новый, функциональный способ)	Особенность Reflect
<code>obj.key</code> или <code>obj[key]</code>	<code>Reflect.get(obj, key)</code>	Возвращает значение
<code>obj.key = value</code>	<code>Reflect.set(obj, key, value)</code>	Возвращает <code>boolean</code> (успех/неудача)
<code>delete obj.key</code>	<code>Reflect.deleteProperty(obj, key)</code>	Возвращает <code>boolean</code>
<code>'key' in obj</code>	<code>Reflect.has(obj, key)</code>	Возвращает <code>boolean</code>
<code>Object.keys(obj)</code>	<code>Reflect.ownKeys(obj)</code>	Возвращает массив всех ключей, включая <code>Symbol</code>

14.1.2. Детальный разбор методов Reflect

Всего у Reflect **13 методов**. Они зеркально отражают одноименные методы объекта `Proxy`. Рассмотрим их по группам.

Группа 1: Базовые операции над свойствами (Getter/Setter)

1. `Reflect.get(target, propertyKey[, receiver])`
Аналог чтения свойства: `target[propertyKey]`.

```
javascript
const person = { name: 'Alice', age: 25 };

// Старый способ
```

```

console.log(person.name); // Alice

// Reflect способ
console.log(Reflect.get(person, 'name')); // Alice

// Сложный случай: геттеры и receiver
const alice = {
  name: 'Alice',
  get greet() {
    return `Hello, I'm ${this.name}`;
  }
};

const bob = { name: 'Bob' };

// Передаем bob в качестве receiver (контекста this)
console.log(Reflect.get(alice, 'greet', bob)); // "Hello, I'm Bob"

```

Важно: Параметр `receiver` позволяет подменить `this` внутри геттера. Это критически важно для `Proxy`.

2. `Reflect.set(target, propertyKey, value[, receiver])`

Аналог присваивания: `target[propertyKey] = value`.

Главное отличие: возвращает `true`, если запись прошла успешно, и `false`, если нет.

```

javascript

const obj = {};

// Object.defineProperty может запретить запись
Object.defineProperty(obj, 'readOnly', {
  value: 42,
  writable: false
});

// Старый способ: молча провалится или выбросит ошибку в strict mode
obj.readOnly = 100; // В strict mode: TypeError

// Reflect.set: всегда возвращает флаг успеха
if (Reflect.set(obj, 'readOnly', 100)) {
  console.log('Успешно записано');
} else {
  console.log('Не удалось записать свойство'); // Выполнится это
}

```

```
console.log(obj.readOnly); // 42
```

3. Reflect.has(target, propertyKey)

Аналог оператора `in`.

javascript

```
const obj = { a: 1, [Symbol('id')]: 2 };
```

// Старый способ

```
console.log('a' in obj); // true
```

// Новый способ

```
console.log(Reflect.has(obj, 'a')); // true
```

// Преимущество: можно использовать в функциональном стиле

```
const properties = ['a', 'b', 'c'];
```

```
properties.filter(prop => Reflect.has(obj, prop)); // ['a']
```

4. Reflect.deleteProperty(target, propertyKey)

Аналог оператора `delete`.

Возвращает `true`, если удаление прошло успешно или свойства не существовало.

javascript

```
const obj = { x: 10, y: 20 };
```

```
Object.freeze(obj); // Замораживаем объект
```

// delete obj.x; // В strict mode: TypeError

```
if (Reflect.deleteProperty(obj, 'x')) {
```

```
  console.log('Удалено');
```

```
} else {
```

```
  console.log('Не удалено (объект заморожен)'); // Выполнится это
```

```
}
```

```
console.log(obj.x); // 10
```

Группа 2: Создание объектов и наследование

5. Reflect.construct(target, argumentsList[, newTarget])

Аналог оператора `new`. Это самый элегантный способ заменить `Object.create()` в некоторых задачах.

javascript

```

class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return `${this.name} makes a sound`;
  }
}

// Старый способ
const dog1 = new Animal('Rex');

// Новый способ (Reflect)
const dog2 = Reflect.construct(Animal, ['Rex']);

// Продвинутое использование: изменение прототипа на лету
function Dog(name) {
  this.name = name;
}
Dog.prototype.bark = function() {
  return `${this.name} says Woof!`;
};

// Создаем экземпляр Animal, но с прототипом Dog
const weirdDog = Reflect.construct(Animal, ['Rex'], Dog);
console.log(weirdDog.name); // Rex (свойство от Animal)
console.log(weirdDog.bark()); // Rex says Woof! (метод от Dog)
console.log(weirdDog.speak); // undefined (прототип не Animal)

```

Когда использовать: Паттерн «фабрика классов» или миксины.

6. Reflect.getPrototypeOf(target)

Аналог Object.getPrototypeOf().

```

javascript

const arr = [];
console.log(Reflect.getPrototypeOf(arr) === Array.prototype); // true

```

Отличие от Object.getPrototypeOf: Если передан не объект (примитив), Object попытается привести к объекту, Reflect выбросит TypeError.

```

javascript

Object.getPrototypeOf(1); // Number.prototype (приведение типов)

```

```
// Reflect.getPrototypeOf(1); // TypeError!
```

7. Reflect.setPrototypeOf(target, prototype)

Аналог `Object.setPrototypeOf()`.

Возвращает `boolean` (успех/неудача), в отличие от старого метода, который кидает ошибку.

javascript

```
const obj = {};  
Object.freeze(obj);
```

```
// Object.setPrototypeOf(obj, null); // TypeError: cannot set prototype of frozen object
```

```
if (Reflect.setPrototypeOf(obj, null)) {  
  console.log('Прототип изменен');  
} else {  
  console.log('Не удалось изменить прототип'); // Выполнится это  
}
```

Группа 3: Дескрипторы и атрибуты свойств

8. Reflect.defineProperty(target, propertyKey, attributes)

Аналог `Object.defineProperty()`.

Отличие: возвращает `boolean`, а не целевой объект.

javascript

```
const obj = {};
```

```
// Object.defineProperty возвращает obj (для чейнинга)
```

```
const result1 = Object.defineProperty(obj, 'a', { value: 1 });  
console.log(result1 === obj); // true
```

```
// Reflect.defineProperty возвращает boolean
```

```
const success = Reflect.defineProperty(obj, 'b', {  
  value: 2,  
  writable: false  
});  
console.log(success); // true  
console.log(obj.b); // 2
```

9. Reflect.getOwnPropertyDescriptor(target, propertyKey)

Аналог Object.getOwnPropertyDescriptor().

Идентичен по поведению, но использует сигнатуру, удобную для прокси.

Группа 4: Расширенные операции (Ключи)

10. Reflect.ownKeys(target)

Это жемчужина коллекции.

Возвращает массив **всех** собственных ключей объекта (строковые + символы + неперечисляемые).

```
javascript

const sym = Symbol('secret');
const obj = {
  a: 1,
  [sym]: 2,
};

// Делаем свойство неперечисляемым
Object.defineProperty(obj, 'b', {
  value: 3,
  enumerable: false
});

// Object.keys() – только перечисляемые строковые
console.log(Object.keys(obj)); // ['a']

// Object.getOwnPropertyNames() – все строковые (включая неперечисляемые)
console.log(Object.getOwnPropertyNames(obj)); // ['a', 'b']

// Object.getOwnPropertySymbols() – только символы
console.log(Object.getOwnPropertySymbols(obj)); // [Symbol(secret)]

// REFLECT.OWNKEYS() – ВСЁ В ОДНОМ!
console.log(Reflect.ownKeys(obj)); // ['a', 'b', Symbol(secret)]
```

14.1.3. Таблица сравнения: Reflect vs Object

Задача	Object	Reflect	Почему Reflect лучше?
Получить ключи	<code>keys()</code> , <code>getOwnPropertyNames()</code> , <code>ownKeys()</code> <code>getOwnPropertySymbols()</code>		1 метод вместо 3
Определить свойство	<code>defineProperty()</code> (возвращает <code>obj</code>)	<code>defineProperty()</code> (возвращает <code>boolean</code>)	Понятный флаг успеха
Удалить свойство	<code>delete</code> оператор	<code>deleteProperty()</code>	Функция, <code>boolean</code>
Проверить наличие	<code>in</code> оператор	<code>has()</code>	Функция
Вызвать конструктор	<code>new</code> оператор	<code>construct()</code>	Можно подменить <code>new.target</code>
Установить прототип	<code>setPrototypeOf()</code> (кидает ошибку)	<code>setPrototypeOf()</code> (возвращает <code>boolean</code>)	Не обрушивает программу
Работа с геттерами	Косвенно через <code>obj.prop</code>	<code>get()</code> с <code>receiver</code>	Контроль контекста <code>this</code>

14.1.4. Самый важный сценарий: работа в паре с Proxy

`Reflect` бесполезен на 50%, если использовать его в одиночку. Его главная сила раскрывается внутри ловушек `Proxy`.

Пример: Логирование доступа к свойствам без нарушения логики

```
javascript

const user = {
  name: 'John',
  age: 30,
  get birthYear() {
    return new Date().getFullYear() - this.age;
  }
};

const handler = {
```

```

get(target, prop, receiver) {
  console.log(`[LOG] Чтение свойства "${prop}"`);

  // ПЛОХО: так делать нельзя! Это ломает геттеры и контекст this
  // return target[prop];

  // ХОРОШО: используем Reflect
  return Reflect.get(target, prop, receiver);
},

set(target, prop, value, receiver) {
  console.log(`[LOG] Запись свойства "${prop}" = ${value}`);
  // Обязательно возвращаем boolean
  return Reflect.set(target, prop, value, receiver);
}
};

const proxyUser = new Proxy(user, handler);

console.log(proxyUser.birthYear);
// [LOG] Чтение свойства "birthYear"
// [LOG] Чтение свойства "age"
// 2025 (или текущий год)

proxyUser.age = 35;
// [LOG] Запись свойства "age" = 35

```

Почему обязательно нужен `Reflect.get(target, prop, receiver)`, а не `target[prop]`?

В примере выше `birthYear` — это **геттер**. Внутри него используется `this.age`.

1. Если мы вызовем `target[prop]`, контекстом `this` будет оригинальный объект `user`.
2. Если мы вызовем `Reflect.get` и передадим туда `receiver` (это наш `proxyUser`), контекстом `this` станет прокси.

Если мы не используем `Reflect`, геттер не увидит изменения, сделанные через прокси!

14.1.5. Антипаттерны и критика

1. Не используйте Reflect, если не работаете с Proxy

Это добавляет шум в код без реальной пользы.

```
javascript

// ПЛОХО (избыточно)
const name = Reflect.get(obj, 'name');

// ХОРОШО (просто и читаемо)
const name = obj.name;
```

2. Не забывайте про возвращаемое значение set

Ловушка `set` в Proxy **ОБЯЗАНА** вернуть `true`, если запись прошла успешно. Если вернуть `false` или `undefined`, в `strict mode` вылетит `TypeError`.

```
javascript

const handler = {
  set(target, prop, value) {
    // Забыли return
    Reflect.set(target, prop, value);
    // return undefined; // ОШИБКА!
  }
};
```

3. Reflect не работает с примитивами

В отличие от `Object.*`, `Reflect.*` строго типизирован.

14.1.6. Практические кейсы использования

Кейс 1: Валидация данных через Proxy + Reflect

```
javascript

function createValidatedUser(userData) {
  return new Proxy(userData, {
    set(target, prop, value) {
      if (prop === 'age') {
        if (typeof value !== 'number' || value < 0 || value > 150) {
          throw new Error('Возраст должен быть числом от 0 до 150');
        }
      }
    }
  });
}
```

```

    }
    if (prop === 'email') {
      if (!value.includes('@')) {
        throw new Error('Некорректный email');
      }
    }
    return Reflect.set(target, prop, value);
  }
});
}

```

```

const user = createValidatedUser({ name: 'John' });
user.age = 25; // OK
// user.age = -5; // Error: Возраст должен быть...

```

Кейс 2: Дефолтные значения (Default values)

javascript

```

function withDefaults(target, defaults) {
  return new Proxy(target, {
    get(target, prop, receiver) {
      const value = Reflect.get(target, prop, receiver);
      if (value === undefined) {
        return Reflect.get(defaults, prop, defaults);
      }
      return value;
    }
  });
}

```

```

const config = withDefaults({ theme: 'dark' }, { port: 3000, debug: false });
console.log(config.port); // 3000 (взято из defaults)
console.log(config.theme); // dark (взято из target)

```

Кейс 3: Приватные свойства (эмуляция)

javascript

```

function privateFields(obj, privateKeys) {
  const privateSet = new Set(privateKeys);
  return new Proxy(obj, {
    get(target, prop, receiver) {
      if (privateSet.has(prop)) {
        throw new Error(`Свойство "${String(prop)}" является приватным`);
      }
    }
  });
}

```

```
    return Reflect.get(target, prop, receiver);
  },
  ownKeys(target) {
    return Reflect.ownKeys(target).filter(key => !privateSet.has(key));
  }
});
}

const bankAccount = privateFields({
  owner: 'John',
  balance: 1000,
  _pin: 1234
}, ['_pin']);

console.log(bankAccount.owner); // John
// console.log(bankAccount._pin); // Error! Приватное свойство
console.log(Object.keys(bankAccount)); // ['owner', 'balance'] (_pin скрыт)
```

Резюме главы 14.1

1. `Reflect` — это не магия, это просто **стандартная библиотека функций** для работы с объектами.
2. Он делает три вещи:
 - ➔ Превращает операторы (`in`, `delete`, `new`) в функции.
 - ➔ Унифицирует получение ключей (`Reflect.ownKeys`).
 - ➔ Возвращает `boolean` вместо выбрасывания ошибок.
3. **Золотое правило:** Внутри ловушек `Proxy` **всегда** используйте `Reflect` для вызова оригинального поведения объекта.
4. `Reflect` и `Proxy` — это неразрывная пара. Одно без другого используется крайне редко.

Проверь себя:

В чем разница между `Object.keys()` и `Reflect.ownKeys()`?

Почему в `Proxy` для геттера нужно использовать `Reflect.get` с `receiver`?

14.2. Прокси-объекты и перехватчики

Эпиграф: «Перехват — это не слежка, это воспитание. Вы не наказываете объект, вы учите его вести себя иначе».

Введение: Что такое Proxy?

Представьте, что у вас есть знаменитость (объект). Сама знаменитость не берет трубку, не отвечает на звонки и не открывает дверь. У неё есть **агент (Proxy)**. Все звонки, контракты и предложения проходят через агента. Агент может:

1. **Отклонить** предложение (запретить операцию).
2. **Изменить** условия контракта (модифицировать данные).
3. **Записать** разговор (логирование).
4. **Притвориться** знаменитостью (виртуализация).

Proxy (Прокси) — это объект-обёртка, которая перехватывает фундаментальные операции (чтение свойств, запись, вызов функций, удаление и т.д.) над другим объектом (целевым объектом) и позволяет выполнить пользовательский код (ловушку, trap) до или вместо стандартного поведения.

Важнейшая терминология:

Термин	Оригинал	Описание
Целевой объект	target	Оригинальный объект, который мы оборачиваем. Знаменитость.
Обработчик	handler	Объект, содержащий ловушки (traps). Телефонная книга агента.
Ловушка	trap	Метод в обработчике, перехватывающий конкретную операцию. Скрипт агента: «Если звонят с таким-то предложением — делай так».
Прокси	proxy	Обёрнутый объект. Сам агент, с которым общается внешний мир.

Синтаксис:

javascript

```
const proxy = new Proxy(target, handler);
```

14.2.1. Исторический контекст: Почему Proxy — это революция?

До ES6 у разработчиков не было способа перехватывать саму механику языка.

Эпоха 1: Сеттеры и геттеры (ES5, 2009)

javascript

```
const obj = {  
  _age: 25,  
  get age() { return this._age; },  
  set age(value) {  
    if (value < 0) throw new Error('Отрицательный возраст');  
    this._age = value;  
  }  
};
```

Проблема: Нужно заранее знать, какие свойства мы хотим контролировать. Нельзя перехватить обращение к любому свойству.

Эпоха 2: Object.defineProperty (Vue 2)

Vue 2 использовал `Object.defineProperty` для реактивности. Но он не мог обнаружить добавление новых свойств (приходилось использовать `Vue.set`).

Эпоха 3: Proxy (ES6, 2015) — Тотальный контроль

Proxy позволяет перехватывать **13 фундаментальных операций языка**. Это не просто методы объекта — это уровень самого движка JavaScript.

Цитата: «Proxy — это ближе всего к тому, чтобы внедрить собственный код в чрево компилятора».

14.2.2. Полный справочник: 13 ловушек Proxy

Существует ровно 13 ловушек. Они делятся на 4 категории.

Категория 1: Получение и установка (Getters/Setters)

Ловушка	Перехватывает	Стандартное поведение (Reflect)
<code>get(target, prop, receiver)</code>	Чтение свойства: <code>proxy[prop]</code> , <code>proxy.prototype</code>	<code>Reflect.get(target, prop, receiver)</code>
<code>set(target, prop, value, receiver)</code>	Запись свойства: <code>proxy[prop] = value</code>	<code>Reflect.set(target, prop, value, receiver)</code>

Категория 2: Проверка наличия и удаление

Ловушка	Перехватывает	Стандартное поведение
<code>has(target, prop)</code>	Оператор <code>in</code> : <code>prop in proxy</code>	<code>Reflect.has(target, prop)</code>
<code>deleteProperty(target, prop)</code>	Оператор <code>delete</code> : <code>delete proxy[prop]</code>	<code>Reflect.deleteProperty(target, prop)</code>

Категория 3: Создание и наследование

Ловушка	Перехватывает	Стандартное поведение
<code>construct(target, argList, newTarget)</code>	Оператор <code>new</code> : <code>new proxy(...args)</code>	<code>Reflect.construct(target, argList, newTarget)</code>
<code>getPrototypeOf(target)</code>	<code>Object.getPrototypeOf(proxy)</code> , <code>proxy.__proto__</code>	<code>Reflect.getPrototypeOf(target)</code>
<code>setPrototypeOf(target, proto)</code>	<code>Object.setPrototypeOf(proxy, proto)</code>	<code>Reflect.setPrototypeOf(target, proto)</code>

Категория 4: Перечисление и ключи

Ловушка	Перехватывает	Стандартное поведение
<code>ownKeys(target)</code>	<code>Object.keys()</code> , <code>Object.getOwnPropertyNames()</code> , <code>Reflect.ownKeys()</code> , <code>Object.getOwnPropertySymbols()</code>	<code>Reflect.ownKeys(target)</code>

Ловушка	Перехватывает	Стандартное поведение
<code>getOwnPropertyDescriptor(target, prop)</code>	<code>Object.getPrototypeOf(target, prop)</code>	<code>Reflect.getPrototypeOf(target, prop)</code>
<code>defineProperty(target, prop, desc)</code>	<code>Object.defineProperty(target, prop, desc)</code>	<code>Reflect.defineProperty(target, prop, desc)</code>

Категория 5: Функциональные

Ловушка	Перехватывает	Стандартное поведение
<code>apply(target, thisArg, argList)</code>	Вызов функции: <code>proxy(...args), proxy.call(), proxy.apply()</code>	<code>Reflect.apply(target, thisArg, argList)</code>

Категория 6: Расширяемость

Ловушка	Перехватывает	Стандартное поведение
<code>isExtensible(target)</code>	<code>Object.isExtensible(target)</code>	<code>Reflect.isExtensible(target)</code>
<code>preventExtensions(target)</code>	<code>Object.preventExtensions(target)</code>	<code>Reflect.preventExtensions(target)</code>

14.2.3. Детальный разбор каждой ловушки с примерами

Ловушка 1: `get` — Чтение свойств

Это самая используемая ловушка.

Задача: Создать объект, который при обращении к несуществующему свойству не возвращает `undefined`, а генерирует ошибку.

```
javascript
const strictObject = (obj) => {
  return new Proxy(obj, {
    get(target, prop, receiver) {
      // Проверяем, есть ли свойство в самом объекте ИЛИ в прототипе
      if (!Reflect.has(target, prop)) {
        throw new ReferenceError(`Свойство "${String(prop)}" не определено`);
      }
    }
  });
}
```

```

    return Reflect.get(target, prop, receiver);
  }
});
};

const config = strictObject({ port: 3000 });
console.log(config.port); // 3000
// console.log(config.host); // ReferenceError: Свойство "host" не определено

```

Продвинутый пример: Ленивая загрузка (Lazy Initialization)

Представьте, что у вас есть тяжёлое свойство (например, огромная конфигурация или подключение к БД). Вы не хотите вычислять его, пока к нему не обратятся.

```

javascript

function lazyLoad(obj) {
  return new Proxy(obj, {
    get(target, prop, receiver) {
      // Если свойство – функция, и оно ещё не определено
      if (prop in target && typeof target[prop] === 'function') {
        console.log(`[Lazy] Инициализация модуля "${String(prop)}"...`);
        const result = target[prop](); // Вызываем фабрику
        target[prop] = result; // Заменяем функцию на результат
        return result;
      }
      return Reflect.get(target, prop, receiver);
    }
  });
}

const heavyModules = lazyLoad({
  database: () => {
    console.log('   □ Устанавливаем соединение с БД...');
    return { connected: true, query: (sql) => `Результат: ${sql}` };
  },
  logger: () => {
    console.log('   □ Настраиваем логгер...');
    return { log: (msg) => `[LOG]: ${msg}` };
  }
});

// Ничего не произошло – модули ещё не загружены!
console.log('Старт приложения');
// Только сейчас, при первом обращении:

```

```
console.log(heavyModules.database.query('SELECT * FROM users'));  
//   ▯ Устанавливаем соединение с БД...  
// Результат: SELECT * FROM users  
  
console.log(heavyModules.logger.log('Hello'));  
//   ▯ Настраиваем логгер...  
// [LOG]: Hello
```

Ловушка 2: `set` — Запись свойств

Задача: Валидация типов на лету.

```
javascript  
  
function typedObject(schema) {  
  return new Proxy(schema, {  
    set(target, prop, value) {  
      // Получаем ожидаемый тип из схемы (хранится в отдельном скрытом свойстве)  
      const expectedType = target[`_type_${String(prop)}`];  
  
      if (expectedType && typeof value !== expectedType) {  
        throw new TypeError(  
          `Свойство "${String(prop)}" должно быть типа ${expectedType}, получен ${typeof  
value}`  
        );  
      }  
  
      return Reflect.set(target, prop, value);  
    },  
  
    defineProperty(target, prop, descriptor) {  
      // Позволяем определять типы через специальный синтаксис  
      if (descriptor.type) {  
        target[`_type_${String(prop)}`] = descriptor.type;  
        delete descriptor.type;  
      }  
      return Reflect.defineProperty(target, prop, descriptor);  
    }  
  });  
}  
  
const user = typedObject({ name: 'John' });  
// Определяем "схему" через defineProperty (нестандартное расширение)
```

```
Reflect.defineProperty(user, 'age', {  
  value: 30,  
  type: 'number',  
  writable: true  
});
```

```
user.age = 35; // OK
```

```
// user.age = 'тридцать пять'; // TypeError: Свойство "age" должно быть типа number
```

Ловушка 3: `has` — Оператор `in`

Позволяет скрывать свойства от проверки существования.

Задача: Создать "магический" объект, который ведёт себя как массив, но имеет виртуальные свойства.

```
javascript  
  
class Range {  
  constructor(start, end) {  
    this.start = start;  
    this.end = end;  
  
    return new Proxy(this, {  
      has(target, prop) {  
        // Если спрашивают число в строковом виде ("5")  
        const index = Number(prop);  
        if (!isNaN(index)) {  
          return index >= target.start && index < target.end;  
        }  
        // Иначе стандартная проверка  
        return Reflect.has(target, prop);  
      },  
  
      get(target, prop, receiver) {  
        const index = Number(prop);  
        if (!isNaN(index)) {  
          if (index >= target.start && index < target.end) {  
            return `Элемент ${index}`;  
          }  
          return undefined;  
        }  
      }  
    });  
  }  
}
```

```
    return Reflect.get(target, prop, receiver);
  }
});
}
}

const numbers = new Range(1, 5);
console.log(3 in numbers); // true (элемент 3 существует виртуально)
console.log(10 in numbers); // false
console.log(numbers[2]); // "Элемент 2"
console.log(numbers.start); // 1 (реальное свойство)
```

Ловушка 4: `deleteProperty` — Удаление

Контролируем, что можно удалять, а что нельзя.

Задача: Защищённые свойства (неудаляемые и нестираемые).

```
javascript

function protectProperties(obj, protectedKeys = []) {
  const protectedSet = new Set(protectedKeys);

  return new Proxy(obj, {
    deleteProperty(target, prop) {
      if (protectedSet.has(prop)) {
        throw new Error(`Свойство "${String(prop)}" защищено от удаления`);
      }
      return Reflect.deleteProperty(target, prop);
    },

    set(target, prop, value) {
      if (protectedSet.has(prop) && target[prop] !== undefined) {
        throw new Error(`Свойство "${String(prop)}" защищено от изменения`);
      }
      return Reflect.set(target, prop, value);
    }
  });
}

const app = protectProperties({
  VERSION: '1.0.0',
```

```
apiKey: 'abc123',
debug: false
}, ['VERSION', 'apiKey']);

app.debug = true; // OK
// delete app.VERSION; // Error: Свойство "VERSION" защищено от удаления
// app.apiKey = 'new-key'; // Error: Свойство "apiKey" защищено от изменения
```

Ловушка 5: `apply` — Перехват вызова функции

Позволяет оборачивать функции и перехватывать их выполнение.

Задача: Создать функцию-счётчик вызовов.

```
javascript

function countCalls(fn) {
  let counter = 0;

  return new Proxy(fn, {
    apply(target, thisArg, argList) {
      counter++;
      console.log(`📞 Вызов #${counter} с аргументами:`, argList);

      // Измеряем время выполнения
      console.time(`fn-${counter}`);
      const result = Reflect.apply(target, thisArg, argList);
      console.timeEnd(`fn-${counter}`);

      return result;
    },

    // Добавляем метод для получения статистики
    get(target, prop) {
      if (prop === 'getCount') return () => counter;
      return Reflect.get(target, prop);
    }
  });
}

const sqrt = countCalls(Math.sqrt);
console.log(sqrt(4)); // 📞 Вызов #1 с аргументами: [4] fn-1: 0.02ms → 2
```

```
console.log(sqrt(16)); // 🚗 Вызов #2 с аргументами: [16] fn-2: 0.01ms → 4
console.log(`Всего вызовов: ${sqrt.getCount()}`); // Всего вызовов: 2
```

Ловушка 6: `construct` — Перехват оператора `new`

Позволяет перехватывать создание экземпляров класса.

Задача: Синглтон (Singleton) через Proxy.

```
javascript

function singleton(Class) {
  let instance = null;

  return new Proxy(Class, {
    construct(target, argList, newTarget) {
      if (!instance) {
        console.log('🆕 Создаём новый экземпляр');
        instance = Reflect.construct(target, argList, newTarget);
      } else {
        console.log('🔄 Возвращаем существующий экземпляр');
      }
      return instance;
    }
  });
}

class Database {
  constructor(name) {
    this.name = name;
    console.log(`🔧 Конструктор БД: ${name}`);
  }

  query(sql) {
    return `[${this.name}] ${sql}`;
  }
}

const SingletonDB = singleton(Database);

const db1 = new SingletonDB('MainDB');
// 🔧 Конструктор БД: MainDB
```

```
//  Создаём новый экземпляр
```

```
const db2 = new SingletonDB('SecondaryDB');
```

```
//  Возвращаем существующий экземпляр
```

```
console.log(db1 === db2); // true
```

```
console.log(db1.query('SELECT *')); // [MainDB] SELECT *
```

Ловушка 7: `ownKeys` и `getOwnPropertyDescriptor` — Контроль видимости

Позволяет фильтровать, какие свойства показывать в `Object.keys()`, `for...in` и т.д.

Задача: Создать объект, который скрывает приватные поля (с `_` в начале) от итерации.

javascript

```
function hidePrivate(obj) {
  return new Proxy(obj, {
    ownKeys(target) {
      // Reflect.ownKeys возвращает ВСЕ ключи, включая символы и неперечисляемые
      const allKeys = Reflect.ownKeys(target);
      // Фильтруем те, что начинаются с '_'
      return allKeys.filter(key =>
        typeof key !== 'string' || !key.startsWith('_')
      );
    },

    getOwnPropertyDescriptor(target, prop) {
      const descriptor = Reflect.getOwnPropertyDescriptor(target, prop);
      if (descriptor && typeof prop === 'string' && prop.startsWith('_')) {
        // Делаем приватные свойства неперечисляемыми
        descriptor.enumerable = false;
      }
      return descriptor;
    }
  });
}

const user = hidePrivate({
  name: 'Alice',
```



```

age: 30,
_password: 'secret123',
_token: 'xyz789',
[Symbol('internal')]: 'hidden'
});

console.log(Object.keys(user)); // ['name', 'age']
console.log(Object.getOwnPropertyNames(user)); // ['name', 'age']
console.log(Reflect.ownKeys(user));
// ['name', 'age', Symbol(internal)] — символы остаются, _поля скрыты

```

14.2.4. Инварианты Proxy: Неприкосновенный запас

Proxy — мощный инструмент, но у него есть **непреодолимые ограничения (инварианты)**. Разработчики ECMAScript специально добавили их, чтобы нельзя было сломать фундаментальные контракты языка.

Инвариант 1: Нельзя врать о типе объекта

Если целевой объект не является функцией, прокси нельзя вызвать как функцию.

```

javascript

const obj = {};
const proxy = new Proxy(obj, {
  apply: () => console.log('fake')
});
// proxy(); // TypeError: proxy is not a function

```

Инвариант 2: Нельзя врать о расширяемости

Если `Object.isExtensible(target) === false`, то ловушка `isExtensible` обязана вернуть `false`, а `preventExtensions` — `true`.

```

javascript

const frozen = Object.freeze({ a: 1 });
const proxy = new Proxy(frozen, {
  isExtensible(target) {
    return true; // ЛОЖЬ!
  }
});
// Object.isExtensible(proxy); // TypeError: 'isExtensible' on proxy: trap result does
// not reflect extensibility of proxy target (which is false)

```

Инвариант 3: Нельзя врать о дескрипторах

Если свойство существует в целевом объекте как **неконфигурируемое** и **неперезаписываемое**, то прокси не может изменить его дескриптор.

```
javascript

const math = Object.defineProperty({}, 'PI', {
  value: 3.14159,
  writable: false,
  configurable: false
});

const proxy = new Proxy(math, {
  getOwnPropertyDescriptor(target, prop) {
    return {
      value: 3.14, // ДРУГОЕ ЗНАЧЕНИЕ!
      writable: true,
      configurable: true
    };
  }
});

// Object.getOwnPropertyDescriptor(proxy, 'PI'); // TypeError!
```

Почему это важно?

Инварианты гарантируют, что даже с Proxy движок JavaScript остается предсказуемым и оптимизируемым. Вы не можете сломать `Math.PI` через прокси.

14.2.5. Производительность и компромиссы

Proxy — это **не бесплатная абстракция**. Каждый перехват добавляет накладные расходы.

Таблица производительности (усреднённые данные):

Операция	Без Proxy	С Proxy	Замедление
Чтение свойства	~0.01мкс	~0.5-5мкс	50-500x
Запись свойства	~0.01мкс	~0.5-5мкс	50-500x

Операция	Без Proxy	С Proxy	Замедление
Вызов функции	~0.1мкс	~1-10мкс	10-100х
Перебор ключей	~1мкс	~10-50мкс	10-50х

Когда НЕ использовать Proxy:

1. **Горячие циклы (hot paths)** — функции, вызываемые миллионы раз.
2. **Графические анимации** (60fps, критично к времени).
3. **Встроенные объекты** (Array, Map, Date) — некоторые внутренние слоты недоступны для прокси (существуют ограничения).

Когда использовать Proxy:

1. **Реактивность** (Vue 3, MobX) — изменения происходят по действиям пользователя, а не в цикле.
2. **Валидация** — вызывается редко.
3. **Логирование и отладка** — можно отключать в production.
4. **API-обёртки** — запросы к серверу (миллисекунды против наносекунд).

14.2.6. Реальные примеры из практики

Пример 1: Реактивность как в Vue 3

```

javascript

function reactive(target) {
  const subscribers = new Map(); // свойство -> массив колбэков

  return new Proxy(target, {
    get(target, prop, receiver) {
      // Отслеживание зависимости (effect)
      const currentEffect = activeEffect; // глобальная переменная
      if (currentEffect) {
        if (!subscribers.has(prop)) {
          subscribers.set(prop, new Set());
        }
        subscribers.get(prop).add(currentEffect);
      }

      return Reflect.get(target, prop, receiver);
    },
  });
}

```

```

set(target, prop, value, receiver) {
  const oldValue = target[prop];
  const result = Reflect.set(target, prop, value, receiver);

  // Оповещаем подписчиков
  if (subscribers.has(prop)) {
    subscribers.get(prop).forEach(effect => {
      effect(value, oldValue);
    });
  }

  return result;
}
});
}

let activeEffect = null;
function watchEffect(fn) {
  activeEffect = fn;
  fn(); // Вызовет сбор зависимостей
  activeEffect = null;
}

```

```

// Использование
const state = reactive({ count: 0, name: 'Alice' });

watchEffect(() => {
  console.log(`Эффект: count = ${state.count}`);
});
// Эффект: count = 0

state.count++; // Эффект: count = 1

```

Пример 2: Отмена (Revocable Proxy)

Иногда нужно полностью отключить прокси. Для этого существует `Proxy.revocable()`.

```

javascript

const { proxy, revoke } = Proxy.revocable(
  { secret: 'classified' },
  {

```

```
get(target, prop) {  
  if (prop === 'secret') {  
    return '***REDACTED***';  
  }  
  return target[prop];  
}  
}  
);
```

```
console.log(proxy.secret); // ***REDACTED***
```

```
// Через некоторое время — отзываем доступ  
revoke();
```

```
// console.log(proxy.secret); // TypeError: Cannot perform 'get' on a proxy that has been  
revoked
```

Применение: Платежные шлюзы, модули безопасности, токены авторизации.

14.2.7. Итог: Ментальная модель Proxy

1. **Proxy — это прослойка.** Он не изменяет оригинальный объект, он стоит перед ним.
 2. **13 ловушек = 13 возможностей изменить грамматику JS.** Вы можете сделать так, чтобы `obj.x` не читал `x`, а делал запрос к серверу.
 3. **Всегда используйте Reflect.** Без него вы сломаете `this`, геттеры и инварианты.
 4. **Proxy не серебряная пуля.** Это тяжелая артиллерия для инфраструктурного кода (фреймворки, библиотеки), а не для бизнес-логики в каждом контроллере.
-

14.2.8. Контрольные вопросы для самопроверки

1. **Можно ли с помощью Proxy сделать число итерируемым?**

Да. Можно обернуть число в Proxy и перехватить `Symbol.iterator` в ловушке `get`.

2. **Что произойдет, если в ловушке `set` вернуть `false`?**

В нестрогом режиме (`non-strict`) операция молча проигнорируется. В строгом режиме (`strict mode`) будет выброшена ошибка `TypeError`.

3. **В чем разница между Proxy и `Object.defineProperty`?**

`defineProperty` работает с конкретными, заранее известными

свойствами. Proxy работает с любым обращением к объекту и не требует предварительного знания структуры.

4. Почему Map и Set плохо проксируются?

У них есть внутренние слоты (`[[MapData]]`), которые недоступны через обычные свойства. Прокси не может их перехватить стандартными ловушками.

Резюме главы 14.2:

Proxy — это возможность писать код, который думает о коде. Это вход в мир метапрограммирования. Если вы пишете библиотеку, которая должна быть «магической» (Vue, MobX, Immer), вам нужен Proxy. Если вы пишете CRUD-приложение — скорее всего, вы обойдетесь без него.

14.3. Символы (Symbol) и их применение

Эпиграф: «Имя, которое нельзя произнести дважды. Секретная метка, видимая лишь посвящённым».

Введение: Проблема «Столкновения имён»

Представьте, что вы работаете в огромной корпорации, где одновременно 5000 разработчиков пишут код для одного гигантского приложения. Вы создаёте библиотеку для работы с пользователями и добавляете объекту `user` свойство `id`.

Через месяц другая команда подключает вашу библиотеку и тоже решает добавить свойство `user.id` — для своего модуля аналитики.

Результат: Коллапс. Ваше свойство перезаписано. Данные потеряны.

Классические решения до ES6:

1. Именованное по вендорному префиксу (как в CSS):

```
javascript  
  
user._libCompanyId = 123;  
user._analyticsTrackId = 456;
```

Недостаток: Ничто не мешает другому разработчику тоже начать с `_libCompany`. Это джунгли.

2. Хранение данных отдельно (WeakMap):

```
javascript  
  
const libData = new WeakMap();  
libData.set(user, { id: 123 });
```

Недостаток: Неудобно. Нет прямого доступа через `user.id`.

Решение ES6: Символ (Symbol)

Символ — это **примитивный** тип данных (седьмой примитив в JavaScript наряду с `string`, `number`, `boolean`, `null`, `undefined` и `bigint`). Гарантированно уникальный идентификатор.

```
javascript

const id1 = Symbol('id');
const id2 = Symbol('id');

console.log(id1 === id2); // false! Всегда false
```

14.3.1. Природа Символа: Уникальность и Скрытность

Символ — это не строка. Это атомарная, уникальная сущность.

```
javascript

const sym = Symbol('описание');
console.log(typeof sym); // "symbol"
console.log(sym.toString()); // "Symbol(описание)"
console.log(String(sym)); // "Symbol(описание)"
console.log(sym.description); // "описание" (ES2019)
```

Три главных свойства Символа:

1. **Абсолютная уникальность.** Даже одинаковые описания создают разные СИМВОЛЫ.
2. **Неявное преобразование запрещено.** Нельзя сложить символ со строкой.

```
javascript

const sym = Symbol('test');
// console.log(sym + 'string'); // TypeError: Cannot convert a Symbol value to a string
// Нужно явно: String(sym) или sym.toString()
```

3. **Игнорирование стандартными методами**

перебора. `Object.keys()`, `for...in`, `Object.values()` не видят символы. Только `Object.getOwnPropertySymbols()` и `Reflect.ownKeys()`.

14.3.2. Создание и идентификация символов

Локальные символы (обычные)

Каждый вызов `Symbol()` создаёт новый, уникальный символ.

```
javascript

const localA = Symbol('local');
const localB = Symbol('local');
console.log(localA === localB); // false
```

Глобальный реестр символов (`Symbol.for`)

Иногда нужно, чтобы символы с одинаковым описанием были одним и тем же символом — например, для кросс-библиотечного взаимодействия.

```
javascript

const globalA = Symbol.for('app.unique.id');
const globalB = Symbol.for('app.unique.id');
console.log(globalA === globalB); // true! Это один и тот же символ

// Получить ключ по символу
const key = Symbol.keyFor(globalA); // "app.unique.id"
```

Разница `Symbol()` vs `Symbol.for()`:

	<code>Symbol(key)</code>	<code>Symbol.for(key)</code>
Уникальность	Всегда новый	Один на всю программу
Хранение	В памяти, GC	В глобальном реестре
Доступ	Только по ссылке	По ключу из любого места
<code>keyFor()</code>	<code>undefined</code>	Ключ

14.3.3. Символы как ключи объектов: Иммунное поле

Символы можно использовать как имена свойств объектов. И это их главное применение.

```

javascript

const _id = Symbol('id');
const _token = Symbol('token');

class User {
  constructor(name, id, token) {
    this.name = name;
    this[_id] = id;          // Приватное поле (эмуляция)
    this[_token] = token;    // Приватное поле
  }

  getId() {
    return this[_id]; // Внутри класса доступ есть
  }
}

const user = new User('Alice', 123, 'secret');
console.log(user.name); // "Alice"
console.log(user._id); // undefined! Обычный доступ не работает
console.log(user.getId()); // 123

// Но! Полной приватности нет:
console.log(Object.getOwnPropertySymbols(user)); // [ Symbol(id), Symbol(token) ]
console.log(user[Object.getOwnPropertySymbols(user)[0]]); // 123 (доступно)

```

Важно: Символы не делают свойства приватными в смысле инкапсуляции (их всё ещё можно найти через рефлексию). Они делают их скрытыми от случайного вмешательства.

14.3.4. Well-Known Symbols: Внутренняя магия JavaScript

Самая мощная функция символов — **Well-Known Symbols**. Это встроенные символы, которые являются константами в объекте `Symbol`. Они позволяют переопределять поведение объектов на уровне языка.

Список Well-Known Symbols (ключевые):

Символ	Целевой объект	Перехватывает
<code>Symbol.iterator</code>	Итераторы	<code>for...of</code> , <code>spread</code> , <code>Array.from</code>

Символ	Целевой объект	Перехватывает
<code>Symbol.asyncIterator</code>	Асинхронные итераторы	<code>for await...of</code>
<code>Symbol.hasInstance</code>	Конструкторы	<code>instanceof</code>
<code>Symbol.toStringTag</code>	Все объекты	<code>Object.prototype.toString.call()</code>
<code>Symbol.toPrimitive</code>	Все объекты	Преобразование в примитив
<code>Symbol.isConcatSpreadable</code>	Массивы	<code>Array.prototype.concat()</code>
<code>Symbol.species</code>	Классы	Создание производных объектов
<code>Symbol.match</code> / <code>Symbol.replace</code> / <code>Symbol.search</code> / <code>Symbol.split</code>	RegExp, строки	Работа регулярных выражений
<code>Symbol.unscopables</code>	Объекты	Директива <code>with</code> (устаревшее)

14.3.4.1. `Symbol.iterator`: Сделать объект итерируемым

Это самый известный и часто используемый символ.

```

javascript

class Fibonacci {
  constructor(limit = 10) {
    this.limit = limit;
  }

  // Магия: делаем объект итерируемым
  [Symbol.iterator]() {
    let prev = 0, curr = 1, count = 0, limit = this.limit;

    return {
      next() {
        if (count++ >= limit) {
          return { done: true, value: undefined };
        }
        [prev, curr] = [curr, prev + curr];
        return { done: false, value: prev };
      }
    };
  }
}

```

```
const fib = new Fibonacci(8);
console.log([...fib]); // [1, 1, 2, 3, 5, 8, 13, 21]

// Теперь можно использовать в for...of!
for (const num of fib) {
  console.log(num); // 1,1,2,3,5,8,13,21
}
```

Глубокое понимание: Когда интерпретатор видит `for...of`, он:

1. Ищет у объекта метод `Symbol.iterator`.
 2. Вызывает его, получает объект-итератор (с методом `next`).
 3. Вызывает `next()` на каждой итерации.
-

14.3.4.2. `Symbol.asyncIterator`: Асинхронные потоки

Позволяет работать с асинхронными генераторами данных (постраничная загрузка, веб-сокеты, стримы).

```
javascript

class AsyncPaginator {
  constructor(url, pages = 3) {
    this.url = url;
    this.pages = pages;
  }

  async *[Symbol.asyncIterator]() {
    for (let page = 1; page <= this.pages; page++) {
      // Имитация асинхронного запроса
      const data = await this.fetchPage(page);
      yield data;
    }
  }

  async fetchPage(page) {
    await new Promise(resolve => setTimeout(resolve, 500)); // задержка сети
    return { page, items: [`item${page}-1`, `item${page}-2`] };
  }
}

(async () => {
```

```
const paginator = new AsyncPaginator('/api/users');

for await (const pageData of paginator) {
  console.log(pageData);
  // { page: 1, items: [...] } (через 500ms)
  // { page: 2, items: [...] } (через 1000ms)
  // { page: 3, items: [...] } (через 1500ms)
}
})();
```

14.3.4.3. `Symbol.hasInstance`: Контроль `instanceof`

Позволяет переопределить поведение оператора `instanceof`.

```
javascript

class Range {
  constructor(start, end) {
    this.start = start;
    this.end = end;
  }

  static [Symbol.hasInstance](instance) {
    return Number.isInteger(instance) &&
      instance >= this.prototype.constructor.prototype.start &&
      instance <= this.prototype.constructor.prototype.end;
  }
}

const range = new Range(1, 10);
console.log(5 instanceof range.constructor); // true
console.log(15 instanceof range.constructor); // false
```

Реальный кейс: Фреймворки могут использовать это для создания псевдоклассов, не являющихся настоящими конструкторами.

14.3.4.4. `Symbol.toStringTag`: Кастомизация `toString()`

Когда вы вызываете `Object.prototype.toString.call(obj)`, вы получаете `"[object Object]"`. Символ `toStringTag` позволяет это изменить.

javascript

```
class Vector2D {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  get [Symbol.toStringTag]() {
    return 'Vector2D';
  }
}

const v = new Vector2D(3, 4);
console.log(Object.prototype.toString.call(v)); // "[object Vector2D]"
// Вместо стандартного "[object Object]"

// Даже встроенные объекты используют это:
console.log(Object.prototype.toString.call(Promise.resolve())); // "[object Promise]"
```

Зачем это нужно? Сериализация, логи, отладка. Консоль браузера использует этот механизм.

14.3.4.5. `Symbol.toPrimitive`: Тотальный контроль преобразования

Объект пытается превратиться в число, строку или boolean. Вы можете перехватить этот процесс.

javascript

```
class Temperature {
  constructor(celsius) {
    this.celsius = celsius;
  }

  [Symbol.toPrimitive](hint) {
    // hint: "string", "number", "default"
    switch (hint) {
      case 'string':
        return `${this.celsius}°C`;
      case 'number':
        return this.celsius;
      case 'default':
```

```
    return this.celsius + ' degrees';
  }
}
```

```
const temp = new Temperature(25);
```

```
console.log(String(temp));    // "25°C" (hint = string)
console.log(+temp);           // 25 (hint = number)
console.log(temp + ' is warm'); // "25 degrees is warm" (hint = default)
```

Сравнение с `toString()` и `valueOf()`:

До ES6 были отдельно `toString()` (для строк) и `valueOf()` (для чисел). `Symbol.toPrimitive` объединяет их и даёт точный контроль.

14.3.4.6. `Symbol.species`: Кто твой родитель?

Определяет, какой конструктор использовать при создании производных объектов (например, `map()` возвращает новый массив — но какого именно класса?).

javascript

```
class MyArray extends Array {
  // При вызове map, filter и т.д. будет создаваться MyArray, а не Array
  static get [Symbol.species]() {
    return Array; // А вот и нет! Будем создавать обычный Array
  }
}
```

```
const myArr = new MyArray(1, 2, 3);
const mapped = myArr.map(x => x * 2);
```

```
console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array);   // true
```

Зачем ломать наследование? Производительность. Методы обычного `Array` оптимизированы лучше, чем кастомных наследников.

14.3.5. Практические паттерны с Symbol

Паттерн 1: Предотвращение конфликтов имён (Анклавы)

```
javascript

// Библиотека А: Работа с пользователями
const UserLib = (() => {
  const _id = Symbol('id');
  const _email = Symbol('email');

  class User {
    constructor(id, email) {
      this[_id] = id;
      this[_email] = email;
    }

    getProfile() {
      return {
        id: this[_id],
        email: this[_email]
      };
    }
  }

  return { User };
})();

// Библиотека В: Аналитика
const AnalyticsLib = (() => {
  const _id = Symbol('id'); // Совершенно другой символ!
  const _visitCount = Symbol('visitCount');

  function track(user) {
    user[_visitCount] = (user[_visitCount] || 0) + 1;
    user[_id] = `analytics-${Date.now()}`; // Не конфликтует с UserLib._id!
  }

  return { track };
})();

const user = new UserLib.User(123, 'alice@example.com');
AnalyticsLib.track(user);
AnalyticsLib.track(user);
```



```
console.log(user.getProfile()); // { id: 123, email: 'alice@example.com' }  
// Свойства аналитики не видны, не мешают, не перезаписывают данные!
```

Паттерн 2: Мета-программирование с сокрытием API

Создадим API, где публичные методы — обычные строки, а внутренние, служебные — символы.

```
javascript  
  
const _internals = Symbol('internals');  
  
class SafeDatabase {  
  constructor(connectionString) {  
    // Все внутренние данные и методы — в одном символе  
    this[_internals] = {  
      connection: null,  
      retryCount: 0,  
      maxRetries: 5,  
      connect() {  
        console.log(`Подключение к ${connectionString}...`);  
        this.connection = { connected: true };  
      },  
      logError(err) {  
        console.error(`[DB ERROR] ${err}`);  
      }  
    };  
  }  
  
  // Публичный API — чистые строки  
  async query(sql) {  
    if (!this[_internals].connection) {  
      await this[_internals].connect();  
    }  
    return `Результат: ${sql}`;  
  }  
  
  // Доступ к "приватным" методам через геттер  
  get internals() {  
    return this[_internals];  
  }  
}
```

```
const db = new SafeDatabase('postgresql://localhost/mydb');
console.log(db.query('SELECT *')); // Результат: SELECT *
console.log(db.internals.maxRetries); // 5
console.log(Object.keys(db)); // ['internals'] – символа _internals нет!
```

Паттерн 3: Создание перечислений (Enums)

В JavaScript нет настоящих enum. Символы решают проблему уникальности значений.

```
javascript

const Colors = {
  RED: Symbol('red'),
  GREEN: Symbol('green'),
  BLUE: Symbol('blue'),
  YELLOW: Symbol('yellow')
};

class Car {
  constructor(color) {
    if (!Object.values(Colors).includes(color)) {
      throw new Error('Недопустимый цвет');
    }
    this.color = color;
  }

  getColorName() {
    // Обратное преобразование: найти ключ по символу
    return Object.keys(Colors).find(key => Colors[key] === this.color);
  }
}

const car1 = new Car(Colors.RED);
console.log(car1.getColorName()); // "RED"

// const car2 = new Car('red'); // Ошибка! 'red' !== Symbol(red)
// const car3 = new Car(Symbol('red')); // Ошибка! Это другой символ
```

Преимущество над строковыми enum: Никто не может передать `'red'` по ошибке. Только точная ссылка на символ.

14.3.6. Производительность и ограничения

Символы — это примитивы

Они не занимают много памяти. Операции с символами как ключами объектов сравнимы по скорости с операциями со строками.

Символы не сериализуются

```
javascript

const sym = Symbol('secret');
const obj = { [sym]: 123 };

const json = JSON.stringify(obj);
console.log(json); // "{}" — символы игнорируются!

// Решение: хранить символы отдельно или использовать реестр
const globalSym = Symbol.for('persistent');
obj[globalSym] = 456;
// JSON.stringify всё равно игнорирует
```

Если нужно передавать символы между системами:

Используйте глобальный реестр и передавайте ключ (`Symbol.keyFor`).

14.3.7. Таблица: Символы и их влияние на итерацию

Метод	Видит строки	Видит символы	Видит неперечисляемые
<code>Object.keys()</code>	☐	×	×
<code>Object.getOwnPropertyNames()</code>	☐	×	☐
<code>Object.getOwnPropertySymbols()</code>	×	☐	☐
<code>Reflect.ownKeys()</code>	☐	☐	☐
<code>for...in</code>	☐	×	×
<code>JSON.stringify()</code>	☐	×	×

14.3.8. Итоги: Когда и зачем использовать Symbol

Используйте Symbol, когда:

1. **Нужны уникальные ключи свойств**, защищённые от случайных пересечений.
2. **Вы создаёте библиотеку/фреймворк** и не хотите конфликтовать с пользовательским кодом.
3. **Вы реализуете итераторы**, асинхронные итераторы или кастомное поведение (`toStringTag`, `toPrimitive`).
4. **Вы создаёте enum-подобные константы**, где значения должны быть абсолютно уникальны.
5. **Вы пишете метапрограммирование** с Proxy и Reflect, используя символы для скрытых слотов.

НЕ используйте Symbol, когда:

1. **Нужна настоящая приватность** (используйте `#private` поля ES2022).
 2. **Данные нужно отправлять в JSON** (символы потеряются).
 3. **Вы пишете простой CRUD** без глубокой библиотечной архитектуры.
-

14.3.9. Контрольные вопросы

1. **Можно ли получить символ из глобального реестра, зная только описание?**
Да. `Symbol.for('key')` вернёт существующий или создаст новый.
 2. **В чём разница между `Symbol()` и `Symbol.for()`?**
`Symbol()` — каждый раз новый уникальный символ. `Symbol.for()` — единый на весь реестр.
 3. **Почему `Object.keys()` игнорирует символы?**
По дизайну языка. Символы предназначены для мета-уровня, они не должны случайно всплывать в обычных операциях перебора.
 4. **Как проверить, является ли значение символом?**
`typeof value === 'symbol'`.
 5. **Можно ли использовать символ как прототип?**
Нет. Символ — примитив. `Object.setPrototypeOf(obj, sym)` вызовет ошибку.
-

Резюме главы 14.3:

Символы — это тихая революция в JavaScript. Они не кричат о себе, не видны в консоли, не попадают в JSON, не участвуют в `for...in`. Но именно они

позволяют библиотекам сосуществовать, не мешая друг другу, и дают разработчикам ключи к внутренним механизмам языка.

Символ — это секретный пароль объекта. И теперь вы знаете, как его использовать.

14.4. Дескрипторы свойств: `defineProperty`, `getOwnPropertyDescriptor`

Эпиграф: «Свойство — это не просто значение. Это личность со своим характером, правами и историей».

Введение: Анатомия свойства

В JavaScript у каждого свойства объекта есть **характер**. Когда мы пишем:

```
javascript  
  
const user = {  
  name: 'Alice'  
};
```

Мы видим только верхушку айсберга. Под водой скрывается **дескриптор свойства** — объект, который описывает:

- **[[Value]]** — значение (собственно, 'Alice').
- **[[Writable]]** — можно ли перезаписывать?
- **[[Enumerable]]** — будет ли видно в `for...in` и `Object.keys()`?
- **[[Configurable]]** — можно ли менять дескриптор и удалять свойство?

Исторический контекст:

До ES5 (2009) у разработчика не было инструментов, чтобы управлять этими атрибутами. Всё, что создавалось простым присваиванием, имело жёстко заданные флаги: `writable: true`, `enumerable: true`, `configurable: true`. Нельзя было сделать «настоящую константу» или «скрытое свойство».

ES5 подарил нам два ключевых метода:

1. `Object.defineProperty()` — создать свойство с полным контролем.
 2. `Object.getOwnPropertyDescriptor()` — посмотреть, как настроено свойство.
-

14.4.1. Дескрипторы: Две категории

Свойства в JavaScript бывают двух типов, и у них разные дескрипторы.

1. Свойства-данные (Data Descriptors)

Хранят значение. Имеют флаги: `value`, `writable`, `enumerable`, `configurable`.

2. Свойства-аксессоры (Accessor Descriptors)

Содержат функции геттера и сеттера. Имеют флаги: `get`, `set`, `enumerable`, `configurable`.

Важно: Дескриптор не может быть одновременно и данными, и аксессором. Нельзя указать и `value`, и `get` в одном дескрипторе — это вызовет ошибку.

14.4.2. Анатомия: Три флага и одно значение

Рассмотрим каждый флаг подробно.

1. `writable` — Разрешение на запись

```
javascript

const book = {};

Object.defineProperty(book, 'title', {
  value: 'Война и мир',
  writable: false,      // Нельзя изменить!
  enumerable: true,
  configurable: true
});

console.log(book.title); // 'Война и мир'
book.title = 'Анна Каренина'; // В strict mode: TypeError
console.log(book.title); // 'Война и мир' (значение не изменилось)
```

Ментальная модель:

`writable: false` — это не `const`. `const` запрещает присваивание самой переменной. А `writable: false` запрещает изменение свойства объекта. Разные уровни защиты.

2. enumerable — Видимость для итерации

```
javascript

const car = {
  brand: 'Toyota',
  model: 'Camry'
};

// Добавляем "скрытое" свойство
Object.defineProperty(car, 'vin', {
  value: '1HGBH41JXMN109186',
  enumerable: false,    // Не будет видно в Object.keys() и for...in
  writable: true,
  configurable: true
});

console.log(Object.keys(car)); // ['brand', 'model'] — vin не виден!
console.log(car.vin); // '1HGBH41JXMN109186' (но прямой доступ работает)

// for...in тоже игнорирует:
for (let key in car) {
  console.log(key); // 'brand', 'model'
}
```

Зачем скрывать?

Внутренние идентификаторы, служебные данные, кэши. Они нужны объекту, но не должны мешаться в переборах.

3. configurable — «Смертельный номер»

Это самый строгий флаг. Если `configurable: false`:

1. Нельзя удалить свойство (`delete`).
2. Нельзя изменить его дескриптор (нельзя переключить с `data` на `accessor` и наоборот).
3. Нельзя менять `enumerable`.
4. Нельзя менять `configurable` (самого себя).
5. **Можно** менять `writable` с `true` на `false` (но не обратно!).

```
javascript

const config = { apiUrl: 'https://api.example.com' };

Object.defineProperty(config, 'apiUrl', {
```



```
value: 'https://api.example.com',
configurable: false, // Запечатываем
writable: true,
enumerable: true
});

// delete config.apiUrl; // Ошибка! Нельзя удалить
// Object.defineProperty(config, 'apiUrl', { enumerable: false }); // Ошибка!

// Но writable можно выключить (это единственное исключение)
Object.defineProperty(config, 'apiUrl', { writable: false }); // OK
config.apiUrl = 'new-url'; // Теперь не работает (writable: false)
```

Аналогия:

`configurable: false` — это как замуровать дверь. Вы можете пользоваться комнатой, но не можете ни снести стену, ни переделать дверь в окно. Единственное, что можно — повесить табличку «Вход воспрещён» (`writable: false`).

14.4.3. Object.defineProperty: Режим бога

Синтаксис:

javascript

```
Object.defineProperty(obj, prop, descriptor);
```

Возвращает **тот же объект** (для чейнинга).

Множественное определение:

javascript

```
Object.defineProperties(obj, {
  prop1: descriptor1,
  prop2: descriptor2
});
```

Классический пример: Константы в объекте

javascript

```
const MATH_CONSTANTS = {};
```

```
Object.defineProperty(MATH_CONSTANTS, {
  PI: {
    value: 3.141592653589793,
    writable: false,
    enumerable: true,
    configurable: false
  },
  E: {
    value: 2.718281828459045,
    writable: false,
    enumerable: true,
    configurable: false
  },
  // Скрытое служебное свойство
  _version: {
    value: '1.0.0',
    writable: true,
    enumerable: false,
    configurable: false
  }
});

MATH_CONSTANTS.PI = 3.14; // Игнорируется (или ошибка в strict mode)
console.log(MATH_CONSTANTS.PI); // 3.141592653589793
console.log(Object.keys(MATH_CONSTANTS)); // ['PI', 'E'] – _version скрыт
```

14.4.4. Object.getOwnPropertyDescriptor: Рентгеновское зрение

Позволяет заглянуть внутрь и посмотреть, как настроено свойство.

```
javascript

const obj = { a: 1 };

// Простое присваивание
const descriptorA = Object.getOwnPropertyDescriptor(obj, 'a');
console.log(descriptorA);
// {
//   value: 1,
//   writable: true,
```

```
// enumerable: true,
// configurable: true
// }

// Аксессор
Object.defineProperty(obj, 'b', {
  get() { return this._b * 2; },
  set(val) { this._b = val; },
  enumerable: false
});

const descriptorB = Object.getOwnPropertyDescriptor(obj, 'b');
console.log(descriptorB);
// {
//   get: [Function: get],
//   set: [Function: set],
//   enumerable: false,
//   configurable: false (по умолчанию)
// }
```

Множественное получение:

```
javascript

Object.getOwnPropertyDescriptors(obj);
// Возвращает объект, где ключи — имена свойств, значения — их дескрипторы
```

Это важно для копирования объектов с сохранением флагов:

```
javascript

const clone = Object.defineProperties({}, Object.getOwnPropertyDescriptors(original));
```

14.4.5. Свойства-аксессоры: Геттеры и сеттеры

Дескрипторы позволяют создавать **виртуальные свойства** — такие, которые не хранят данные, а вычисляют их на лету.

```
javascript

class Circle {
  constructor(radius) {
    this._radius = radius;
  }
}
```

```

// Виртуальное свойство
get area() {
    return Math.PI * this._radius ** 2;
}

set diameter(value) {
    this._radius = value / 2;
}

get diameter() {
    return this._radius * 2;
}
}

const c = new Circle(5);
console.log(c.area); // 78.54 (вычисляется, не хранится)
c.diameter = 20;
console.log(c._radius); // 10
console.log(c.area); // 314.159...

```

Через `defineProperty` (динамическое создание):

```

javascript

const user = { firstName: 'John', lastName: 'Doe' };

Object.defineProperty(user, 'fullName', {
    get() {
        return `${this.firstName} ${this.lastName}`;
    },
    set(value) {
        [this.firstName, this.lastName] = value.split(' ');
    },
    enumerable: true,
    configurable: true
});

console.log(user.fullName); // "John Doe"
user.fullName = 'Jane Smith';
console.log(user.firstName); // "Jane"
console.log(user.lastName); // "Smith"

```

Критическое отличие аксессоров от данных:

- В дескрипторе аксессора **нет** полей `value` и `writable`.

- В дескрипторе данных **нет** полей `get` и `set`.
 - Попытка смешать их вызовет `TypeError`.
-

14.4.6. Значения по умолчанию

Если вы создаёте свойство через `defineProperty` и **не указываете** какие-то флаги, они принимают значения `false` (кроме `configurable` в некоторых контекстах, но правило такое: не сказал — значит `false`).

```
javascript

const obj = {};

Object.defineProperty(obj, 'a', { value: 1 });
// Не указаны: writable, enumerable, configurable

const desc = Object.getOwnPropertyDescriptor(obj, 'a');
console.log(desc);
// {
//   value: 1,
//   writable: false,      // !
//   enumerable: false,    // !
//   configurable: false   // !
// }
```

ЭТО ВАЖНО!

Простое присваивание `obj.a = 1` даёт `{ writable: true, enumerable: true, configurable: true }`.

`defineProperty` без флагов даёт **всё false**. Разные контексты, разное поведение.

14.4.7. Практические паттерны и антипаттерны

Паттерн 1: Неизменяемые конфигурации (Immutable Config)

```
javascript

const APP_CONFIG = {};

function defineConstant(obj, key, value) {
  Object.defineProperty(obj, key, {
```

```

    value,
    writable: false,
    enumerable: true,
    configurable: false
  });
}

defineConstant(APP_CONFIG, 'API_VERSION', 'v2');
defineConstant(APP_CONFIG, 'MAX_RETRIES', 3);
defineConstant(APP_CONFIG, 'TIMEOUT_MS', 5000);

// Никто не сможет изменить или удалить эти настройки
APP_CONFIG.MAX_RETRIES = 10; // Игнорируется
delete APP_CONFIG.API_VERSION; // Игнорируется (или ошибка в strict mode)

```

Паттерн 2: Ленивые вычисления с кэшированием (Memoization)

Создаём свойство, которое при первом чтении вычисляет значение, а затем заменяет себя на свойство-данные с этим значением.

```

javascript

const heavyObject = {
  expensiveComputation: () => {
    console.log('☐ Выполняем сложные вычисления...');
    return 42;
  }
};

Object.defineProperty(heavyObject, 'result', {
  get() {
    console.log('☐ Первое обращение: вычисляем');
    const value = this.expensiveComputation();

    // Переопределяем свойство!
    Object.defineProperty(this, 'result', {
      value,
      writable: false,
      enumerable: true,
      configurable: false
    });

    return value;
  },

```

```
configurable: true, // Нужно для переопределения
enumerable: true
});
```

```
console.log(heavyObject.result); //  Первое обращение: вычисляем → 42
console.log(heavyObject.result); // 42 (сразу, без вычислений!)
```

Паттерн 3: Скрытые приватные поля (до ES2022)

До появления настоящих приватных полей `#field` разработчики использовали `enumerable: false` для сокрытия.

```
javascript

function createBankAccount(owner, balance) {
  const obj = {
    owner,
    getBalance() {
      return this._balance;
    }
  };

  Object.defineProperty(obj, '_balance', {
    value: balance,
    writable: true,
    enumerable: false, // Не видно в keys/for-in
    configurable: false
  });

  Object.defineProperty(obj, 'deposit', {
    value: function(amount) {
      if (amount <= 0) throw new Error('Сумма должна быть положительной');
      this._balance += amount;
    },
    writable: false,
    enumerable: true
  });

  return obj;
}

const acc = createBankAccount('Alice', 1000);
console.log(Object.keys(acc)); // ['owner', 'getBalance', 'deposit'] — _balance скрыт!
console.log(acc._balance); // 1000 (но только если явно обратиться)
```

Антипаттерн: Слишком много `defineProperty` в хот-патах

```
javascript

// ПЛОХО: В цикле каждое присваивание через defineProperty
for (let i = 0; i < 10000; i++) {
  Object.defineProperty(obj, `prop${i}`, {
    value: i,
    writable: true,
    enumerable: true,
    configurable: true
  });
}

// ХОРОШО: Сначала создать объект, потом определить структуру
const props = {};
for (let i = 0; i < 10000; i++) {
  props[`prop${i}`] = { value: i, writable: true, enumerable: true, configurable: true };
}
Object.defineProperties(obj, props);
```

14.4.8. Дескрипторы и наследование

Дескрипторы **не наследуются**. Они применяются к собственным свойствам объекта.

```
javascript

const parent = {};
Object.defineProperty(parent, 'computed', {
  get() { return this._value * 2; },
  enumerable: true,
  configurable: true
});

const child = Object.create(parent);
child._value = 10;

console.log(child.computed); // 20 (работает через прототип)
console.log(child.hasOwnProperty('computed')); // false — свойство не собственное!

const descriptor = Object.getOwnPropertyDescriptor(child, 'computed'); // undefined
```


Важно: Методы `getOwnPropertyDescriptor` и `defineProperty` работают **только с собственными свойствами**. Через прототип они не видят.

14.4.9. Reflect.defineProperty и Reflect.getOwnPropertyDescriptor

В ES6 появились **дублёры** этих методов в объекте `Reflect`. Чем они отличаются?

Операция	Object	Reflect	Особенность
Определение	<code>defineProperty</code> (Возвращает <code>obj</code>)	<code>defineProperty</code> (Возвращает <code>boolean</code>)	Reflect даёт флаг успеха
Получение	<code>getOwnPropertyDescriptor</code> (Возвращает <code>desc</code> или <code>undefined</code>)	<code>getOwnPropertyDescriptor</code> (аналогично)	Идентичны

Зачем нужно возвращать boolean?

Для работы в ловушках Proxy. Ловушка `defineProperty` **обязана** вернуть `true`, если определение прошло успешно.

```
javascript
const handler = {
  defineProperty(target, prop, descriptor) {
    // Дополнительная логика...
    return Reflect.defineProperty(target, prop, descriptor); // boolean!
  }
};
```

14.4.10. Производительность и оптимизация движка

Важное предупреждение:

Использование `defineProperty` с немедленным указанием всех флагов **может ухудшить производительность**, если применять его после создания объекта.

Как работают движки (V8):

1. Объекты имеют «скрытые классы» (Hidden Classes).
2. При добавлении свойства через присваивание (`obj.x = 1`) движок переходит в предсказуемое состояние.
3. При использовании `defineProperty` с нестандартными флагами (`writable: false`, `enumerable: false`) объект выпадает в «медленный режим» (dictionary mode).

Совет:

Если вам нужны свойства с флагами, определяйте их **при создании объекта**, а не добавляйте потом.

```
javascript

// БЫСТРО (объект в быстром режиме)
const fast = { a: 1, b: 2 };

// МЕДЛЕННО (словарный режим)
const slow = {};
Object.defineProperty(slow, 'a', { value: 1, writable: true, enumerable: true, configurable: true });
Object.defineProperty(slow, 'b', { value: 2, writable: true, enumerable: true, configurable: true });
```

14.4.11. Итоговая таблица: Что можно и что нельзя

Действие	writable: false	configurable: false	Оба false
Читать	☐	☐	☐
Писать	×	☐ (если writable true)	×
Удалить	☐ (если configurable true)	×	×
Изменить writable	×	☐ (с true на false)	×
Изменить enumerable	☐ (если configurable true)	×	×
Переопределить дескриптор	☐ (если configurable true)	×	×

14.4.12. Контрольные вопросы

1. Можно ли сделать свойство, которое нельзя читать?

Нет. Дескрипторы управляют записью, удалением, перечислением, но не чтением. Любое свойство можно прочитать, если знаешь его имя.

2. В чём разница между `Object.freeze()` и `writable: false, configurable: false`?

`Object.freeze()` делает это для **всех** свойств сразу и запрещает добавление новых.

3. Что произойдёт, если определить свойство и с `value`, и с `get`?

`TypeError`. Дескриптор должен быть либо `data`, либо `accessor`.

4. Как скопировать объект со всеми дескрипторами?

`Object.defineProperties({}, Object.getOwnPropertyDescriptors(obj))`

5. Почему `defineProperty` без флагов выставляет их в `false`, а присваивание — в `true`?

Присваивание — это операция обновления существующей структуры. `defineProperty` — это акт творения. Создатель должен явно указать права.

Резюме главы 14.4

Дескрипторы свойств — это система безопасности и мета-информации JavaScript. Они позволяют:

1. **Защищать** данные от изменений (`writable: false`).
2. **Скрывать** служебные поля (`enumerable: false`).
3. **Запечатывать** структуру объекта (`configurable: false`).
4. **Создавать** виртуальные, вычисляемые свойства (геттеры/сеттеры).

Без дескрипторов JavaScript был бы языком, где всё всегда можно перезаписать, удалить и изменить. Дескрипторы дают нам контроль и дисциплину.

14.5. Заморозка объектов: seal, freeze, preventExtensions

Эпиграф: «Свобода — это ответственность. Иногда объект должен быть лишён свободы, чтобы система не рухнула».

Введение: Три уровня несвободы

Представьте, что вы — архитектор программной системы. Вы создали объект-конфигурацию, который должен быть **неизменным**. Или объект пользователя, структура которого должна быть **фиксированной**, но значения могут меняться. Или словарь, в который **нельзя добавлять** новые записи, но можно удалять существующие.

JavaScript предоставляет **три уровня защиты**, три метода, которые постепенно лишают объект свободы:

Уровень	Метод	Добавлять?	Удалять?	Изменять?
1. Нельзя расширять	<code>preventExtensions</code>	×	□	□
2. Запечатан	<code>seal</code>	×	×	□
3. Заморожен	<code>freeze</code>	×	×	×

Аналогия из жизни:

- `preventExtensions` — вы закрыли границу. Новые люди (свойства) не могут въехать в страну. Но живущие внутри могут уезжать (удаляться) и менять имена (изменять значения).
 - `seal` — вы не только закрыли границу, но и запретили эмиграцию. Все существующие граждане заперты внутри. Но они могут менять работу (значения).
 - `freeze` — абсолютный тоталитарный режим. Никто не въезжает, никто не выезжает, никто не меняет профессию. Всё застыло.
-

14.5.1. Уровень 1: `Object.preventExtensions()` — Закрытые границы

Что делает:

Запрещает добавление **новых** свойств в объект. Удалять и изменять существующие — можно.

Синтаксис:

```
javascript

const obj = { a: 1, b: 2 };
Object.preventExtensions(obj);

obj.c = 3; // Молча игнорируется (в strict mode — TypeError)
console.log(obj.c); // undefined

obj.a = 100; //  Работает!
delete obj.b; //  Работает!
```

Как проверить:

```
javascript

console.log(Object.isExtensible(obj)); // false
```

Где используется:

- Фиксация структуры объекта после инициализации.
- Предотвращение опечаток в ключах.
- Оптимизация производительности (движок знает точный набор свойств).

Важнейший нюанс:

`preventExtensions` не работает рекурсивно. Вложенные объекты остаются расширяемыми!

```
javascript

const config = {
  database: {
    host: 'localhost',
    port: 5432
  }
}
```

```
};
```

```
Object.preventExtensions(config);  
config.database.user = 'admin'; //  РАБОТАЕТ! Внутренний объект не заморожен  
console.log(config.database.user); // 'admin'
```

Как исправить: Нужно рекурсивно замораживать (об этом позже).

14.5.2. Уровень 2: `Object.seal()` — Запечатано

Что делает:

1. Вызывает `preventExtensions()` — запрещает добавление новых свойств.
2. Устанавливает **всем существующим свойствам** флаг `configurable: false`.

Следствия:

- Нельзя добавлять новые свойства.
- Нельзя удалять существующие свойства (`delete` не работает).
- **Можно** изменять значения существующих свойств (если `writable: true`).
- Нельзя менять дескрипторы (превратить свойство-данные в аксессор и наоборот).

Синтаксис:

```
javascript  
  
const user = {  
  name: 'Alice',  
  age: 30  
};  
  
Object.seal(user);  
  
// Проверка  
console.log(Object.isSealed(user)); // true  
console.log(Object.isExtensible(user)); // false  
  
// ✕ Нельзя добавить  
user.email = 'alice@example.com';
```

```
console.log(user.email); // undefined
```

```
// ✗ Нельзя удалить
```

```
delete user.age;
```

```
console.log(user.age); // 30
```

```
// ✔ Можно изменить (если writable true)
```

```
user.name = 'Bob';
```

```
console.log(user.name); // 'Bob'
```

```
// ✗ Нельзя переопределить дескриптор
```

```
Object.defineProperty(user, 'name', { enumerable: false }); // TypeError!
```

Где используется:

- Объекты, структура которых должна быть фиксирована (например, DTO — Data Transfer Objects).
 - Словари, куда нельзя добавлять новые ключи, но можно обновлять значения.
-

14.5.3. Уровень 3: `Object.freeze()` — Абсолютный ноль

Что делает:

1. Вызывает `seal()` — запрещает добавление/удаление, делает `configurable: false`.
2. Устанавливает **всем существующим свойствам-данным** флаг `writable: false`.

Следствия:

- Нельзя добавлять, удалять, изменять свойства.
- Объект становится **полностью неизменяемым** (immutable) на поверхности.
- **Важно:** Глубокой заморозки нет! Вложенные объекты остаются изменяемыми.

Синтаксис:

javascript

```
const settings = {  
  theme: 'dark',  
  notifications: {
```

```
    email: true,
    push: false
  }
};

Object.freeze(settings);

// Проверка
console.log(Object.isFrozen(settings)); // true

// ✗ Нельзя добавить
settings.version = '1.0';
console.log(settings.version); // undefined

// ✗ Нельзя удалить
delete settings.theme;
console.log(settings.theme); // 'dark'

// ✗ Нельзя изменить
settings.theme = 'light';
console.log(settings.theme); // 'dark' (молча игнорируется)

// ⚠️ НО! Вложенный объект НЕ ЗАМОРОЖЕН!
settings.notifications.email = false; // 🟢 РАБОТАЕТ!
console.log(settings.notifications.email); // false
```

Это критическое ограничение!

`Object.freeze()` — **неглубокий (shallow)**. Он замораживает только сами свойства объекта. Если значением свойства является объект — этот объект остаётся полностью изменяемым.

14.5.4. Глубокая заморозка (Deep Freeze)

Если вам нужна **полная неизменяемость**, нужно реализовать рекурсивную заморозку.

```
javascript

function deepFreeze(obj) {
```



```

// 1. Получаем все собственные ключи (включая символы)
const propNames = Reflect.ownKeys(obj);

// 2. Замораживаем каждое свойство-объект рекурсивно
for (const name of propNames) {
  const value = obj[name];

  // Если значение – объект или функция, и оно не заморожено
  if (value && typeof value === 'object' && !Object.isFrozen(value)) {
    deepFreeze(value);
  }
}

// 3. Замораживаем сам объект
return Object.freeze(obj);
}

// Использование
const config = {
  api: {
    url: 'https://api.example.com',
    timeout: 5000,
    headers: {
      'Content-Type': 'application/json'
    }
  },
  features: ['auth', 'analytics']
};

const frozenConfig = deepFreeze(config);

// Теперь ВСЁ заморожено!
frozenConfig.api.url = 'https://evil.com'; // × Не работает
frozenConfig.features.push('new'); // × Не работает (массив тоже заморожен)
frozenConfig.api.headers.Authorization = 'Bearer token'; // × Не работает

console.log(frozenConfig.features); // ['auth', 'analytics'] – неизменны

```

Предупреждение:

`deepFreeze` может быть **дорогой операцией** для больших объектов с циклическими ссылками. Нужно быть осторожным и добавлять проверку на циклы.

14.5.5. Сравнительная таблица: Что можно и что нельзя

Операция	Обычный объект	preventExtensions	seal	freeze
Добавить свойство	□	×	×	×
Удалить свойство	□	□	×	×
Изменить значение (writable: true)	□	□	□	×
Изменить дескриптор	□	□	×	×
Переопределить геттер/сеттер	□	□	×	×
Object.isExtensible()	true	false	false	false
Object.isSealed()	false	false	true	true
Object.isFrozen()	false	false	false	true

14.5.6. Внутреннее устройство: Как это работает на уровне движка?

Когда вы вызываете `preventExtensions`, `seal` или `freeze`, движок JavaScript (V8, SpiderMonkey) переводит объект в **режим медленного пути (slow path / dictionary mode)**.

Быстрый режим (Fast mode):

- Свойства хранятся в массиве с фиксированным смещением.
- Доступ к свойству — это одна-две инструкции процессора.
- Объект имеет «скрытый класс» (Hidden Class).

Медленный режим (Slow mode / Dictionary mode):

- Свойства хранятся в хеш-таблице (как у Map).
- Доступ медленнее (поиск по ключу).
- Объект теряет скрытый класс.

Почему?

Потому что после заморозки движок не ожидает изменений структуры. Но он не может гарантировать, что свойства останутся на тех же смещениях. Для безопасности он переводит объект в словарный режим.

Вывод:

Не замораживайте объекты, которые находятся в критических по производительности циклах. Замораживайте конфигурации при инициализации, но не в рантайме.

14.5.7. Практические паттерны

Паттерн 1: Неизменяемые конфигурации (Immutable Config)

javascript

```
const AppConfig = (function() {
  const config = {
    env: process.env.NODE_ENV || 'development',
    apiUrl: 'https://api.example.com',
    version: '1.0.0',
    features: {
      analytics: true,
      caching: true,
      debug: false
    }
  };

  // Глубокая заморозка при создании
  return deepFreeze(config);
})();

// Никто и никогда не сможет изменить конфигурацию
// AppConfig.env = 'production'; // × Ошибка (в strict mode)
```

Паттерн 2: Защита от опечаток (Typos Protection)

javascript

```
function createTypedObject(shape) {
  const instance = { ...shape };

  return new Proxy(instance, {
    set(target, prop, value) {
```

```

    if (!(prop in target)) {
        throw new Error(`Свойство "${String(prop)}" не определено в схеме объекта`);
    }
    return Reflect.set(target, prop, value);
},

deleteProperty(target, prop) {
    throw new Error(`Нельзя удалять свойства из типизированного объекта`);
}
});
}

const user = createTypedObject({ name: 'Alice', age: 30 });
user.name = 'Bob'; //  ОК
// user.email = 'bob@example.com'; // ✗ Ошибка: свойство не определено
// delete user.age; // ✗ Ошибка: нельзя удалять

```

Паттерн 3: Заморозка после инициализации (Freeze after build)

```

javascript

class ConfigurationBuilder {
    constructor() {
        this._config = {};
        this._frozen = false;
    }

    set(key, value) {
        if (this._frozen) {
            throw new Error('Нельзя изменить замороженную конфигурацию');
        }
        this._config[key] = value;
        return this; // для чейнинга
    }

    build() {
        this._frozen = true;
        return Object.freeze({ ...this._config }); // Возвращаем копию
    }
}

const config = new ConfigurationBuilder()
    .set('port', 3000)
    .set('host', 'localhost')

```

```
.set('debug', true)
.build();
```

```
// config.port = 4000; // ✕ Ошибка (заморожено)
```

14.5.8. Object vs Reflect: Двойники

Как и с дескрипторами, у этих методов есть двойники в `Reflect`:

Операция	Object	Reflect	Особенность
Проверка расширяемости	<code>isExtensible()</code>	<code>isExtensible()</code>	Идентичны
Запрет расширения	<code>preventExtensions()</code>	<code>preventExtensions()</code>	Reflect возвращает boolean
Проверка запечатанности	<code>isSealed()</code>	✕	Нет в Reflect
Проверка заморозки	<code>isFrozen()</code>	✕	Нет в Reflect

Пример с Reflect:

```
javascript
```

```
const obj = {};  
if (Reflect.preventExtensions(obj)) {  
  console.log('Объект успешно закрыт для расширения');  
} else {  
  console.log('Не удалось закрыть объект');  
}
```

Почему нет `seal` и `freeze` в Reflect?

Потому что `seal` и `freeze` — это составные операции. Они не являются фундаментальными примитивами языка в том смысле, в котором являются `preventExtensions` и `defineProperty`. Они надстройка.

14.5.9. Взаимодействие с Proxy

Это важно! Когда вы работаете с Proxy, ловушки `preventExtensions`, `isExtensible` **должны** соблюдать инварианты.

```
javascript

const target = {};
const proxy = new Proxy(target, {
  preventExtensions(target) {
    // Обязательно вызвать Reflect.preventExtensions на target!
    // Иначе инвариант нарушен
    return Reflect.preventExtensions(target);
  },

  isExtensible(target) {
    // Должно совпадать с реальным состоянием target
    return Reflect.isExtensible(target);
  }
});

Object.preventExtensions(proxy);
console.log(Object.isExtensible(proxy)); // false
```

Инвариант: Если `target` нерасширяем, `proxy.isExtensible()` **обязан** вернуть `false`. Иначе `TypeError`.

14.5.10. Производительность и оптимизация

Экспериментальные данные (V8):

Режим	Добавление свойства	Чтение свойства	Удаление свойства
Обычный объект	~2x (быстро)	1x (fast)	~2x
<code>preventExtensions</code>	× (невозможно)	1x	~2x
<code>seal</code>	×	1.5-2x	×
<code>freeze</code>	×	1.5-2x	×

Почему `seal` и `freeze` читаются медленнее?

Потому что объект переведён в словарный режим. Движок не может использовать inline-кэширование (IC), так как не уверен в структуре объекта.

Совет:

Используйте `Object.defineProperty` с явными флагами **при создании объекта**, чтобы остаться в быстром режиме, если производительность критична.

```
javascript
```

```
// БЫСТРО: структура фиксирована с рождения
```

```
const fastConfig = Object.defineProperties({}, {  
  host: { value: 'localhost', writable: false, enumerable: true, configurable: false },  
  port: { value: 3000, writable: false, enumerable: true, configurable: false }  
});
```

```
// МЕДЛЕННО: создали, потом заморозили
```

```
const slowConfig = { host: 'localhost', port: 3000 };  
Object.freeze(slowConfig); // Переход в медленный режим
```

14.5.11. Исключения и особенности

1. Значения-примитивы

Методы `preventExtensions`, `seal`, `freeze` работают **только с объектами**.

Передача примитива:

- В нестрогом режиме: примитив будет преобразован в объект-обёртку, заморожена обёртка.
- В строгом режиме: `TypeError`.

```
javascript
```

```
'use strict';
```

```
// Object.freeze(42); // TypeError: Cannot freeze
```

2. Массивы

Массивы — тоже объекты. Их можно заморозить.

javascript

```
const arr = [1, 2, 3];
Object.freeze(arr);

arr[0] = 100; // × Не работает (в strict mode TypeError)
arr.push(4); // × Не работает
console.log(arr.length); // 3
```

Зачем? Для создания абсолютно неизменяемых списков.

3. Функции

Функции — тоже объекты. Их свойства можно заморозить.

javascript

```
function greet() {
  return 'Hello';
}

greet.version = '1.0';
Object.freeze(greet);

greet.version = '2.0'; // × Не работает
greet.newProp = 'test'; // × Не работает
greet(); // ✔ Сама функция продолжает работать
```

14.5.12. Чек-лист: Какой метод выбрать?

Ситуация	Рекомендация
Хочу запретить добавлять новые свойства, но удалять и менять — можно	<code>preventExtensions</code>
Хочу фиксированную структуру, но значения могут меняться	<code>seal</code>
Хочу полностью неизменяемый объект (поверхностно)	<code>freeze</code>

Ситуация	Рекомендация
Хочу полностью неизменяемый объект (глубоко)	<code>deepFreeze</code>
Хочу запретить удаление свойств, но добавление разрешить	× Нет прямого метода (нужен Proxy)
Хочу запретить изменение значений, но добавление разрешить	× Нет прямого метода (нужен Proxy)

14.5.13. Контрольные вопросы

1. В чём разница между `seal` и `freeze`?

`seal` запрещает добавлять/удалять, но разрешает изменять. `freeze` запрещает всё, включая изменение значений.

2. Можно ли разморозить объект?

Нет. Это необратимая операция. Нельзя вернуть `writable: true` или `configurable: true` после `freeze`.

3. Почему `freeze` не замораживает вложенные объекты?

Потому что это дорого и может привести к циклическим ссылкам. Разработчик должен сам контролировать глубину.

4. Как проверить, заморожен ли объект?

`Object.isFrozen(obj)`. Но помните — для вложенных объектов это не сработает.

5. Что быстрее: `Object.freeze` большой структуры или `Object.defineProperty` для каждого поля?

`Object.freeze` обычно быстрее, потому что это одна операция, а не N. Но оба варианта медленнее, чем создание объекта с правильными флагами сразу.

Резюме главы 14.5

Три метода — три степени защиты:

1. `preventExtensions` — «Не входить». Границы на замке, внутри — полная свобода.

2. `seal` — «Не входить, не выходить». Структура фиксирована, содержимое изменяемо.

3. `freeze` — «Всё застыло». Абсолютная неизменяемость (поверхностная).

Эти методы — фундамент для написания безопасного, предсказуемого кода. Они позволяют создавать контракты: «Этот объект больше никогда не изменится». А `deepFreeze` даёт возможность распространить этот контракт на всю глубину структуры данных.

Глава 15: Память и производительность

15.1. Модель памяти в JavaScript

Эпиграф: «Память — это кладбище переменных. Хороший сборщик мусора — это садовник, который знает, какие цветы уже завяли».

15.1. Модель памяти в JavaScript

Введение: Иллюзия безграничности

JavaScript-разработчики часто живут в счастливом неведении. Мы создаём объекты, строки, массивы — и никогда не думаем о том, где они живут и как умирают. Нам кажется, что память бесконечна.

Это опасная иллюзия.

Браузер выделяет вашей вкладке строго ограниченный ресурс:

- **64–512 МБ** для вкладки Chrome (зависит от устройства и ОС).
- Node.js по умолчанию имеет лимит **~1.4–1.5 ГБ** для 64-битной системы.
- Мобильные браузеры ещё скупее — **до 1 ГБ**, но часто **200–500 МБ**.

Когда приложение переступает эту границу — браузер убивает вкладку. Пользователь видит: «Aw, snap!» в Chrome, «Out of memory» в других браузерах. Конец игры.

Понимание модели памяти — это навык выживания.

15.1.1. Две жизни данных: Стек и Куча

В JavaScript память разделена на два основных региона. Это не просто академическое знание — это ключ к пониманию того, почему одни операции быстрые, а другие медленные; почему одни данные копируются, а другие ссылаются.

Характеристика Стек (Stack)		Куча (Heap)
Типы данных	Примитивы (number, string, boolean, null, undefined, symbol, bigint)	Объекты, массивы, функции, замыкания
Размер	Маленький (КБ – МБ)	Огромный (МБ – ГБ)
Скорость	Наносекунды (L1 кэш процессора)	Микросекунды (произвольный доступ)
Структура	LIFO (Last In, First Out) — как стопка тарелок	Беспорядочная, как склад
Управление	Автоматическое (при входе/выходе из функции)	Сборщик мусора (GC)
Время жизни	Время выполнения функции	Пока есть ссылки

Аналогия для запоминания:

Представьте, что вы работаете в офисе.

🔵 **Стек** — это ваш рабочий стол. На нём лежат документы, с которыми вы работаете прямо сейчас. Вы открыли отчёт — положили на стол. Закрыли отчёт — убрали в ящик. На столе всегда порядок и чистота. Но на столе не может лежать 10 000 документов — просто не хватит места.

🔵 **Куча** — это архивный шкаф. Там лежат все остальные документы. Вы можете достать любой в любой момент, но это дольше (нужно встать, подойти, открыть ящик). Документы могут лежать годами. Если вы забудете, где они лежат, или потеряете ключ — они так и останутся в шкафу навсегда, занимая место (утечка памяти).

15.1.2. Анатомия фрейма стека

Каждый раз, когда JavaScript вызывает функцию, в стеке создаётся **фрейм (кадр)**. Это как записная книжка для одной функции.

Что лежит во фрейме:

1. **Локальные переменные** (примитивы и ссылки на объекты в куче).
2. **Параметры функции.**
3. **Возвращаемый адрес** (куда идти после завершения).
4. **Лексическое окружение** (для замыканий, но это сложнее).

Визуализация:

text

СТЕК ВЫЗОВОВ (CALL STACK) — растёт сверху вниз

Фрейм: global () ├ a: 42 (примитив в стеке) └ user: • (ссылка на объект в куче)
Фрейм: processData(id) ├ id: 123 (примитив) ├ cache: • (ссылка) └ temp: "pending" (строка-примитив)
Фрейм: validate(data) ├ data: • (ссылка на тот же объект) └ isValid: true (примитив)

КУЧА (HEAP) — хаотичное хранилище

Объект User #401 ├ name: "Alice" (строка в куче) ├ age: 30 (число в стеке? НЕТ!) └ [экземпляр класса User]
Массив #789 ├ [0]: 1, [1]: 2, [2]: 3 └ length: 3
Строка #999: "pending"

Критическое понимание:

Число 30 внутри объекта User НЕ лежит в стеке. Оно лежит в куче, внутри объекта. В стеке лежит только ссылка на объект. Сам объект — в куче.

15.1.3. Примитивы: Миф о стеке и интернирование

Распространённое заблуждение: «Все примитивы всегда живут в стеке».

Правда: Это зависит от контекста.

1. **Локальные переменные-примитивы** — ДА, в стеке.

```
javascript

function example() {
  let x = 10;      // x в стеке, значение 10 в стеке
  let y = 20;      // y в стеке, значение 20 в стеке
  return x + y;
}
```

2. **Примитивы как свойства объектов** — НЕТ, в куче.

```
javascript

const obj = {
  age: 30,          // age — свойство, 30 хранится в КУЧЕ (внутри obj)
  name: 'Alice'     // строка — тоже в куче
};
```

3. **Строки** — всегда в куче (кроме очень маленьких оптимизаций в некоторых движках). Строки — это примитивы, но они **переменной длины** и не могут лежать в стеке с фиксированным размером фрейма.

Интернирование строк (String Interning):

Движки JavaScript (V8, JavaScriptCore) оптимизируют память: если две строки выглядят одинаково, они могут указывать на одну и ту же область памяти.

```
javascript

const str1 = 'Hello, World!';
const str2 = 'Hello, World!';

// В некоторых случаях (но не всегда!) str1 === str2 по ссылке
```

// Это зависит от того, созданы ли строки во время компиляции (литералы)

Важно для разработчика:

- Конкатенация строк (+) создаёт **новую строку** в куче.
 - Работа со строками в циклах может создавать огромное количество мусора.
 - Используйте массивы и `join()` для сборки больших строк.
-

15.1.4. Ссылочная модель: Всё есть указатель

JavaScript скрывает от разработчика указатели, но они есть. Переменная, которая «содержит» объект, на самом деле содержит **ссылку (указатель) на область памяти в куче**.

```
javascript

let a = { value: 10 };
let b = a;      // Копируется ССЫЛКА, не объект!

b.value = 20;
console.log(a.value); // 20 – a и b указывают на один объект
```

Визуализация памяти:

text

СТЕК:

a: 0x7ffeefbfff5a8 (указатель на кучу)

b: 0x7ffeefbfff5a8 (ТОТ ЖЕ указатель!)

КУЧА:

0x7ffeefbfff5a8: { value: 20 } // Один объект, две ссылки

Сравнение с примитивами:

```
javascript

let x = 10;
let y = x;      // Копируется ЗНАЧЕНИЕ!

y = 20;
console.log(x); // 10 – x не изменился
```

Это фундаментальное различие определяет ВСЁ поведение памяти в JS.

15.1.5. Время жизни переменных: Лексическая область видимости

Переменные в стеке живут ровно столько, сколько выполняется функция.

```
javascript

function outer() {
  let bigData = new Array(1000000).fill('x'); // bigData – ссылка в стеке
  // Массив на 1 млн элементов – В КУЧЕ!

  function inner() {
    console.log(bigData.length); // inner имеет доступ к bigData (замыкание)
  }

  return inner; // Возвращаем функцию
}

const closure = outer();
// Функция outer ЗАВЕРШИЛАСЬ. Её фрейм в стеке УНИЧТОЖЕН.
// НО! bigData живёт в куче, потому что inner хранит ссылку на него!
```

Правило:

Объект в куче жив, пока на него есть хотя бы одна **достижимая ссылка** из стека (или из другого живого объекта в куче).

Это и есть основа сборки мусора.

15.1.6. Внутреннее устройство объектов в V8 (Hidden Classes и Property Arrays)

Для реального понимания памяти нужно заглянуть под капот V8 (движок Chrome, Node.js).

Наивное представление:

Объект — это хеш-таблица (словарь). Ключ → значение.

Реальность:

V8 использует сложную двухуровневую схему для оптимизации доступа к свойствам.

Уровень 1: Скрытые классы (Hidden Classes / Maps)

V8 создаёт «скрытый класс» для каждого объекта. Это описание структуры объекта: какие свойства есть и где они хранятся.

```
javascript

function Point(x, y) {
  this.x = x;
  this.y = y;
}

const p1 = new Point(1, 2);
const p2 = new Point(3, 4);
```

V8 создаёт:

1. **Hidden Class C0** (пустой объект).
2. Добавляем `x` → **Hidden Class C1** (свойство `x` по смещению 0).
3. Добавляем `y` → **Hidden Class C2** (свойство `x` по смещению 0, `y` по смещению 8).

`p1` и `p2` ссылаются на один и тот же **Hidden Class C2**. Это позволяет движку генерировать очень быстрый машинный код для доступа к свойствам — как к полям C-структуры.

Что ломает оптимизацию:

```
javascript

p1.z = 5; // p1 переходит на новый Hidden Class C3
p2.z = 6; // p2 переходит на Hidden Class C3 (но это разные переходы?)
// В итоге p1 и p2 могут иметь разные скрытые классы → деоптимизация
```

Уровень 2: Режимы хранения свойств

V8 хранит свойства в двух местах:

1. **In-object properties** — прямо в объекте. Ограниченное количество (обычно до 1022 в зависимости от размера объекта). Самый быстрый доступ.
2. **Property array** — отдельный массив для свойств, если их много.

3. **Dictionary mode (медленный режим)** — если структура объекта постоянно меняется, V8 сдаётся и переводит объект в хеш-таблицу.

Визуализация объекта в V8:

```
text

JSObject {
  *map: HiddenClass C2 (указатель)
  properties: [0x1234] (указатель на массив свойств или null)
  elements: [0x5678] (указатель на массив индексированных элементов)

  // In-object properties:
  [0]: x = 10
  [1]: y = 20
}
```

15.1.7. Массивы: Миф о непрерывности

```
javascript

const arr = [1, 2, 3];
```

Наивное представление:

Непрерывный блок памяти, где лежат числа 1, 2, 3.

Реальность в V8:

Есть ТРИ типа массивов, и они потребляют память по-разному.

Тип 1: Packed (плотные)

```
javascript

const arr1 = [1, 2, 3]; // PACKED_SMI_ELEMENTS (Small Integers)
const arr2 = [1, 2, 3.5]; // PACKED_DOUBLE_ELEMENTS
const arr3 = [1, 'two', 3]; // PACKED_ELEMENTS (общий)
```

Самый быстрый и экономный режим. Элементы хранятся непрерывно.

Тип 2: Holey (дырявые)

```
javascript

const arr = [1, , 3]; // HOLES_ELEMENTS
```

```
console.log(arr[1]); // undefined
```

Есть пропуски. Движок тратит дополнительное время на проверку «это пусто или undefined?».

Тип 3: Dictionary (словарь)

```
javascript  
  
const arr = [];  
arr[0] = 1;  
arr[999999] = 2; // Огромный разреженный массив
```

V8 понимает, что хранить 1 миллион пустых ячеек глупо, и переводит массив в словарь. Медленно, но экономно по памяти.

Ключевой вывод для разработчика:

- Избегайте разреженных массивов (с пропусками).
 - Не меняйте типы элементов массива на лету.
 - Используйте `Set` и `Map` для больших коллекций с частым удалением/добавлением.
-

15.1.8. Замыкания: Невидимые потребители памяти

Замыкания — одна из главных причин утечек памяти, которую новички не замечают.

```
javascript  
  
function createHeavyClosure() {  
  const hugeArray = new Array(1000000).fill('x'); // 1 МБ в куче  
  
  return function() {  
    // Эта функция даже не использует hugeArray!  
    console.log('Hello');  
  };  
}  
  
const closure = createHeavyClosure();  
// closure хранит ссылку на hugeArray? ДА!  
// Движок не может анализировать, используете вы переменную или нет.  
// ВСЯ переменная hugeArray сохраняется в замыкании.
```

Как это выглядит в памяти:

text

КУЧА:

```
0x1000: [hugeArray: 1 МБ данных] ←
      |
0x2000: createHeavyClosure scope |
      |— hugeArray: 0x1000 (ссылка)—|
      |— [[function]]: 0x3000
0x3000: closure function
      |— [[scope]]: 0x2000 (ссылка на контекст)
```

Решение:

Если переменная не используется в замыкании, объявляйте её внутри блока или присваивайте `null`.

javascript

```
function betterClosure() {
  { // Блок для изоляции
    const hugeArray = new Array(1000000).fill('x');
    // Используем hugeArray здесь
  } // hugeArray выходит из области видимости, готов к GC

  return function() {
    console.log('Hello');
  };
}
```

15.1.9. DOM и память: Двойная жизнь элементов

Когда вы работаете с DOM, память управляется **двумя менеджерами**: JavaScript GC и браузерным рендерером.

Проблема: Detached DOM nodes (отсоединённые узлы)

javascript

```
function createDetachedNode() {
  const div = document.createElement('div');
  div.textContent = 'Hello';
  return div;
}
```

```
}
```

```
const node = createDetachedNode();  
// node существует в памяти JavaScript  
// НО! Он не прикреплен к документу. Браузер не показывает его.  
// Память под этот div всё равно занята!  
  
document.body.appendChild(node); // Теперь прикреплен  
document.body.removeChild(node); // Откреплен, но node всё ещё в переменной!  
// Утечка: элемент в памяти, но не в DOM
```

Правило:

Если вы удалили элемент из DOM, **обнулите** все ссылки на него в JavaScript.

```
javascript  
  
// Плохо  
const elements = [];  
elements.push(document.getElementById('temp'));  
  
// Хорошо  
const tempElement = document.getElementById('temp');  
// Использовали...  
// Закончили  
tempElement.remove();  
// elements больше не содержит ссылку
```

15.1.10. Сборка мусора: Алгоритмы и поколения

Сборщик мусора в V8 — это сложнейшая инженерная система. Она основана на **гипотезе поколений**:

| «Большинство объектов умирают молодыми».

Поколение 0: Молодое (Nursery / Young Generation)

- Живёт в отдельной, небольшой области памяти.
- Объекты создаются здесь.
- Сборка **Scavenger** (очень частая, ~1-5% времени CPU).
- Быстрая, перемещает живые объекты в следующее поколение.
- Очистка происходит **параллельно** основному потоку.

Алгоритм: Cheney's algorithm (Semispace)

text

До сборки:	После сборки:
[FROM] [TO]	[FROM] [TO]
[ABC·DEF] [·...] → [·...] [ABCDEF]	
↓	↑

Живые объекты A,B,C,D,E,F перемещены в TO, FROM очищен, пространства меняются ролями.

Поколение 1: Старое (Old Generation)

- Объекты, пережившие несколько сборок в молодом поколении.
- Сборка **Mark-Sweep-Compact** (редкая, ~100-1000 мс).
- Трёхфазная:
 1. **Mark** — обход графа объектов от корней, пометка живых.
 2. **Sweep** — проход по куче, добавление мёртвых объектов в список свободной памяти.
 3. **Compact** (опционально) — перемещение живых объектов вместе для устранения фрагментации.

Корни (Roots) — точки старта обхода GC:

1. Глобальный объект (window/global).
 2. Локальные переменные в стеке.
 3. Ссылки из других потоков (Web Workers).
 4. Обработчики событий (Event Listeners).
 5. Объекты в кэшах (WeakMap/WeakSet не считаются корнями!).
-

15.1.11. WeakMap и WeakSet: Слабые ссылки

Обычные ссылки (Map, Array, Object) — **сильные**. Они удерживают объект в памяти, даже если он больше нигде не нужен.

```
javascript

let user = { name: 'Alice' };
const map = new Map();
map.set(user, 'metadata');

user = null; // Оригинальная ссылка удалена
// НО! map хранит сильную ссылку на user!
// Объект { name: 'Alice' } НЕ БУДЕТ собран GC!
```

Решение: WeakMap и WeakSet

javascript

```
let user = { name: 'Alice' };
const wm = new WeakMap();
wm.set(user, 'metadata');

user = null;
// Теперь на объект НЕТ сильных ссылок!
// GC может собрать его при следующем цикле.
// wm.get(user) больше не вернёт значение (user === null).
```

Ограничения WeakMap:

- Ключи — только объекты (не примитивы).
- Нельзя итерировать (нет `keys()`, `values()`, `entries()`, `size`).
- Нет метода `clear()`.

Где используется:

- Хранение метаданных для DOM-элементов.
- Кэши без утечек памяти.
- Приватные поля (до ES2022).

javascript

```
// Реальный кейс: кэш данных с автоматической очисткой
const cache = new WeakMap();

function processData(obj) {
  if (!cache.has(obj)) {
    cache.set(obj, heavyComputation(obj));
  }
  return cache.get(obj);
}

// Когда obj исчезнет — запись в кэше исчезнет автоматически!
```

15.1.12. Измерение памяти в браузере и Node.js

В браузере:

javascript

```
// Грубая оценка
console.log(performance.memory);
// {
//   usedJSHeapSize: 24300000,    // 24 МБ используется
```

```
// totalJSHeapSize: 35600000, // 35 МБ выделено
// jsHeapSizeLimit: 2190000000 // 2.1 ГБ лимит
// }

// Более точный инструмент – Chrome DevTools > Memory
// 1. Heap Snapshot – снимок кучи
// 2. Allocation instrumentation – запись выделений памяти
// 3. Allocation sampling – профилирование с выборкой
```

B Node.js:

```
javascript

const v8 = require('v8');
const process = require('process');

// Статистика памяти V8
console.log(v8.getHeapStatistics());
// {
//   total_heap_size: 7315456,
//   total_heap_size_executable: 524288,
//   total_physical_size: 7315456,
//   total_available_size: 2193572760,
//   used_heap_size: 5271096,
//   heap_size_limit: 2193750016
// }

// Память процесса
console.log(process.memoryUsage());
// {
//   rss: 20951040, // Resident Set Size – память в RAM
//   heapTotal: 10063872, // Всего выделено V8
//   heapUsed: 5395096, // Используется V8
//   external: 8762, // Память вне V8 (libuv, native addons)
//   arrayBuffers: 7894 // ArrayBuffer и SharedArrayBuffer
// }

// Для мониторинга утечек
let lastHeap = v8.getHeapStatistics().used_heap_size;
setInterval(() => {
  const currentHeap = v8.getHeapStatistics().used_heap_size;
  const diff = currentHeap - lastHeap;
  console.log(`Heap delta: ${diff > 0 ? '+' : ''}${diff} bytes`);
  lastHeap = currentHeap;
}, 5000);
```

15.1.13. Итог: Ментальная модель памяти

Для повседневной разработки нужно помнить 7 правил:

1. **Примитивы** — копируются по значению. Объекты — по ссылке.
2. **Стек** — маленький и быстрый. Не храните там огромные массивы данных (вы и не сможете, только ссылки).
3. **Куча** — большой и медленный. Здесь живут все данные.
4. **Ссылка ≠ владение**. Объект жив, пока есть хотя бы одна сильная ссылка из стека или от живого объекта в куче.
5. **Замыкания хранят ВСЁ** лексическое окружение, даже неиспользуемые переменные.
6. **DOM-элементы** — двусторонняя память. Удаляйте и ссылки, и из DOM.
7. **WeakMap/WeakSet** — единственный способ не создавать лишних сильных ссылок.

Дальнейшее чтение:

- V8: <https://v8.dev/blog>
- Chrome DevTools: <https://developer.chrome.com/docs/devtools/memory-problems/>
- Node.js: <https://nodejs.org/api/v8.html>

15.2. Сборка мусора и алгоритмы управления памятью

Эпиграф: «Сборщик мусора — это садовник, который поливает живые цветы и вырывает сорняки, не спрашивая разрешения у сада».

Введение: Невидимая рука рынка памяти

В 15.1 мы изучили **архитектуру** памяти: стек, кучу, поколения. Теперь мы заглянем в **динамику** — как эта память живёт, умирает и возрождается.

Главная идея, которую нужно усвоить перед чтением этой главы:

Сборка мусора — это не событие, а процесс. И этот процесс — сложнейшая инженерная симфония, где дирижёр (V8) управляет оркестром потоков, не останавливая музыку (ваш код).

Исторический экскурс (важно для понимания):

В 2003 году Эрик Липперт, архитектор JScript (движок Internet Explorer), писал: «JScript GC был спроектирован для простых веб-страничек. Если вам нужен контроль над памятью — вы используете не тот инструмент» .

Это было правдой **вчера**.

Сегодня, в эпоху Google Docs, Figma и 3D-игр в браузере, V8 (и другие современные движки) превратили сборщик мусора в инженерное чудо. Они используют **параллельные, инкрементальные и конкурентные** алгоритмы, которые позволяют собирать гигабайты мусора, не заставляя пользователя ждать .

15.2.1. Базовые алгоритмы: Фундамент

Прежде чем погружаться в хитросплетения V8, нужно понять три базовых алгоритма, на которых всё строится.

15.2.1.1. Mark-and-Sweep (Пометить и Подмести)

Это **дедушка** всех алгоритмов сборки мусора. Он был в JScript в 1995 году, он есть в V8 сегодня, и он никуда не уйдёт.

Принцип работы:

1. **Mark (Пометка)**: Начиная с корней (стек, глобальные объекты), GC обходит граф объектов и **помечает** каждый достижимый объект.
2. **Sweep (Подметание)**: Проходит по всей куче. Если объект **не помечен** → он мусор, его память освобождается и возвращается в систему.

Визуализация:

text

ДО MARKSWEEP: [A] ← [B] ← [C] [D] [E] [F]

 ↑ ↑

 ROOT (мусор)

ШАГ 1 (MARK): [✓A] ← [✓B] ← [✓C] [D] [E] [F]

 ↑

 ROOT

ШАГ 2 (SWEEP): [✓A] ← [✓B] ← [✓C] ■■■ ■■■ ■■■

 (память освобождена)

Достоинства:

- Решает проблему циклических ссылок (в отличие от ранних Reference Counting).
- Не требует перемещения объектов.

Недостатки:

- **Stop-The-World**: Пока GC помечает, программа не работает.
- **Фрагментация**: Освобождённые куски памяти разбросаны по куче.
- **Производительность**: При большом количестве живых объектов (например, 100+ МБ) обход графа может занимать **сотни миллисекунд**.

Историческая справка: В JScript (IE) порог срабатывания GC был наивен: 0x100 переменных, 0x1000 слотов массивов или 0x10000 байт строк. В 2005 году это вызывало экспоненциальное замедление на сложных AJAX-приложениях .

15.2.1.2. Mark-Compact (Пометить и Уплотнить)

Mark-Sweep оставляет после себя **фрагментацию**. Представьте: вы освободили 3 ячейки по 10 байт, а нужно выделить массив на 30 байт. Даже если суммарно памяти достаточно, непрерывного куска нет. Приходится снова запускать GC.

Mark-Contrast добавляет третий шаг:

3. **Compact (Уплотнение):** Все живые объекты **сдвигаются** в одну сторону, образуя непрерывный блок .

Визуалізація:

text

ДО COMPACT: [✓A][][][✓B][][][✓C][][]

↑ ↑ ↑

106 156 126 (непрерывного блока на 376 нет)

ПОСЛЕ КОМПАКТ: [✓A][✓B][✓C] [REDACTED]

↑ ↑ ↑

106 156 126 (освободился большой блок)

Цена уплотнения: Копирование объектов в памяти — **очень дорогая** операция. Поэтому V8 не уплотняет всю кучу сразу, а выбирает только сильно фрагментированные страницы .

15.2.1.3. Reference Counting (Подсчёт ссылок) — Историческая аномалия

Этот алгоритм **НЕ ИСПОЛЬЗУЕТСЯ** в современных движках для управления JS-объектами, но он жил в ранних версиях Netscape и до сих пор встречается в некоторых средах (например, COM-объекты в старом IE) .

Принцип: У каждого объекта есть счётчик ссылок. Каждый раз, когда новая переменная начинает ссылаться на объект, счётчик ++. Когда ссылка исчезает, счётчик --. Если счётчик = 0 → объект можно удалить.

Почему он умер:

```
javascript

function leak() {
  const a = {};
  const b = {};
  a.ref = b;
  b.ref = a; // Циклическая ссылка!
  return 'done';
}

// a и b покидают область видимости, но их счётчики ссылок = 1 (друг на друга)
// Они остаются в памяти НАВСЕГДА!
```

Современные движки используют **Mark-Sweep**, который умеет находить «острова недостижимости» .

15.2.2. Генерационная гипотеза: Почему 90% объектов — одноразовые

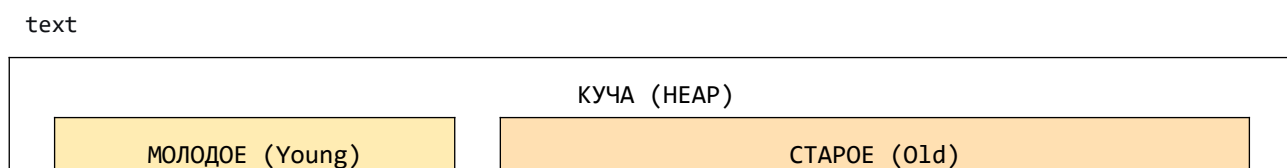
В 1984 году Дэвид Унгер сформулировал **генерационную гипотезу**:

«Большинство объектов умирают молодыми».

В V8 это звучит так: **до 95% новосозданных объектов не переживают даже одну сборку мусора** .

Следствие: Нет смысла сканировать всю кучу целиком каждый раз. Надо разделить объекты по возрасту и обрабатывать их по-разному.

Модель поколений в V8:



Nursery	Intermediate	OldPtr	OldData	Large
(Новые)	(Выжившие)	(указ-ли)	(данные)	(объекты)

Правила повышения в возрасте:

1. Объект создаётся в **Nursery** (ясла).
2. Пережил 1-й GC → перемещается в **Intermediate** (подросток).
3. Пережил 2-й GC → перемещается в **Old Generation** (старейшина) .

15.2.3. Minor GC (Scavenger): Алгоритм для молодых

Scavenger (или Scavenge) — это алгоритм, реализующий сборку **только молодого поколения**. Он основан на классическом **Cheney's Semispace Copy** алгоритме, но с современными параллельными оптимизациями .

15.2.3.1. Схема работы (Пошагово)

Шаг 1: Подготовка пространств

Молодое поколение разделено на два **равных** полу-пространства (semi-space):

- **From-Space** — здесь живут объекты (активная зона).
- **To-Space** — пустое, зарезервированное пространство.

Шаг 2: Маркировка (с ограничениями)

Scavenger не обходит всю кучу! Он использует два источника правды о живых объектах в молодом поколении:

1. **Корни (Roots)**: Стек, глобальные объекты, регистры.
2. **Старое → Молодое (Old-to-New)**: Список ссылок из старого поколения на молодое, который поддерживается **write barrier** .

Как работает Write Barrier:

```
cpp
```

```
// Псевдокод: при каждой операции присваивания
```

```
function writeBarrier(obj, field, value) {
    if (obj->isOld() && value->isYoung()) {
        oldToNewSet.add({obj, field, value}); // Запомнили перекрёстную ссылку!
```

```
}  
obj[field] = value;  
}
```

Зачем это нужно?

Без этого списка пришлось бы сканировать **всё старое поколение** (сотни МБ), чтобы найти ссылки на молодые объекты. Список old-to-new делает Minor GC **быстрым и независимым** от размера старой кучи.

Шаг 3: Эвакуация (Перемещение)

- Обнаруженные живые объекты **копируются** из From-Space в To-Space (или сразу в Old, если это их 2-й цикл).
- В исходном месте объекта остаётся **forwarding pointer** (адрес новой копии) .

Шаг 4: Обновление указателей

- Все, кто ссылался на старый адрес объекта, должны теперь ссылаться на новый.
- Здесь используются **store buffers** и **remembered sets** для эффективного обновления без полного сканирования памяти .

Шаг 5: Смена ролей

From-Space объявляется пустым, To-Space становится новым From-Space. Выделение памяти продолжается.

Визуализация цикла Scavenger:

text

ЦИКЛ 1:

[FROM] Активные объекты: A, B, C (D, E, F – мусор)

[TO] Пусто

ШАГ MARK&EVACUATE:

A, B, C (живые) → копируем в [TO]

Оригиналы A,B,C в [FROM] помечаем forwarding-адресами.

ШАГ SWITCH:

[FROM] становится [TO] (пустым)

[TO] становится [FROM] (содержит A,B,C)

ЦИКЛ 2:

Новые объекты G,H создаются в [FROM] (рядом с A,B,C)

Сложность алгоритма:

Scavenger тратит время **пропорционально количеству ЖИВЫХ объектов** в молодом поколении. Поскольку живых там мало (генерационная гипотеза!), GC очень быстрый — **1-5 мс** даже на мобильных устройствах.

15.2.3.2. Параллельный Scavenger (Современный V8)

До 2015 года Scavenger был **однопоточным**. V8 просто останавливал мир и копировал объекты в главном потоке.

Сейчас (Orinoco project) Scavenger использует **параллельный** режим :

Как это работает:

1. Главный поток и N вспомогательных потоков получают порции корней.
2. Каждый поток, следуя по ссылкам, находит живые объекты.
3. При попытке перемещения объекта возникает гонка (два потока нашли один объект).
4. Потоки используют **атомарные операции** (compare-and-swap) для захвата права на перемещение.
5. Победитель копирует объект, проигравший просто обновляет указатель по forwarding-адресу.
6. Для быстрого выделения памяти в To-Space каждый поток имеет **локальный буфер аллокации** .

Почему не конкурентный?

Потому что Scavenger **перемещает** объекты. Нельзя перемещать объекты, пока JavaScript работает — это сломает все указатели. Поэтому Minor GC всегда **Stop-The-World**, но очень короткий и параллельный .

15.2.4. Major GC (Mark-Compact): Алгоритм для стариков

Старое поколение может занимать **сотни мегабайт или гигабайты**. Останавливать мир на секунду для его полной очистки — неприемлемо для YouTube, Google Docs или игр.

Историческая справка:

В 2003 году сборка мусора на большой куче в JScript была экспоненциально медленной. Разработчики жаловались на «стену», о которую разбиваются сложные веб-приложения .

Решение V8:

Major GC в V8 превратился из простого Mark-Sweep-Compact в гибридный, **конкурентный + инкрементальный + параллельный** механизм .

15.2.4.1. Трёхфазная структура Major GC

Фаза 1: Маркировка (Marking)

Современная маркировка в V8 — это **конкурентный** процесс.

1. **Начало:** V8 определяет, что пора собирать мусор (обычно при достижении лимита кучи).
2. **Запуск помощников:** Запускаются конкурентные задачи на фоновых потоках.
3. **Конкурентная маркировка:** Фоновые потоки обходят граф объектов, пометая живые. **JavaScript продолжает работать параллельно!**
4. **Барьеры (Write Barriers):** Пока JS работает, он может менять ссылки. Если JS меняет ссылку с белого объекта на чёрный (или наоборот), **write barrier** вмешивается и корректирует цвет объекта, чтобы не потерять данные .

Трёхцветная абстракция (для понимания барьеров):

- **Белый:** Не посещён, вероятно, мусор.
- **Серый:** Посещён, но его ссылки ещё не обработаны.
- **Чёрный:** Посещён, все ссылки обработаны.

Проблема конкурентной маркировки:

text

Исходно: A (чёрный) → B (белый)

Мутация: A (чёрный) → null

 C (серый) → B (белый)

Без барьера: B останется белым, не будет помечен от C, будет ошибочно удалён!

Решение V8:

Используется **сатирический барьер** (snapshot-at-the-beginning) или **инкрементальный барьер обновления** (DI, Dijkstra-style). В современных версиях — комбинация подходов, гарантирующая, что ни один живой объект не будет пропущен .

Фаза 2: Подметание (Sweeping)

После завершения маркировки мы точно знаем: чёрные и серые = живые, белые = мёртвые.

Sweeping может быть:

- **Параллельным:** Потоки проходят по страницам памяти, находят непрерывные блоки мёртвых объектов и добавляют их в **free-list** (список свободных блоков).
- **Ленивым:** Страница помечается как «нуждается в подметании», но реальная очистка происходит только когда нужно выделить память на этой странице .

Free-list — это не просто список:

V8 поддерживает **раздельные списки** для разных размеров блоков (как кассы в супермаркете: для 8 байт, 16 байт, 32 байта...). Это позволяет быстро найти подходящий кусок памяти при аллокации .

Фаза 3: Уплотнение (Compaction)

Уплотнение — самая дорогая, но иногда необходимая операция. V8 не уплотняет всю старую кучу сразу, а выбирает **эвакуируемые страницы** на основе порога фрагментации .

Эвакуация страницы:

1. Все живые объекты на выбранной странице **копируются** на другие страницы (используя их free-list'ы).
2. Вся страница объявляется свободной.
3. Указатели на перемещённые объекты обновляются (с помощью remembered sets).

Цена: Копирование больших объёмов памяти. Но поскольку уплотнение происходит редко и только на сильно фрагментированных страницах, средняя цена приемлема.

15.2.5. Три кита Orinoco: Параллельный, Инкрементальный, Конкурентный

Проект **Orinoco** (назван в честь реки в Южной Америке) — это не единовременное изменение, а **эволюция** V8 GC с ~2016 года по настоящее время. Его цель: **убрать паузы Stop-The-World** .

Техника	Stop-The-World?	Как работает	Применение в V8
Параллельный	☐ (короткая пауза)	Несколько потоков делят работу	Minor GC, фаза Sweeping, фаза Compaction
Инкрементальный	▲ (много микро-пауз)	GC выполняется кусками, между ними — JS	Резервный режим для Major GC
Конкурентный	× (JS не останавливается)	Фоновые потоки, барьеры синхронизации	Маркировка в Major GC

Иллюстрация разницы :

text

STOP-THE-WORLD (старый стиль):



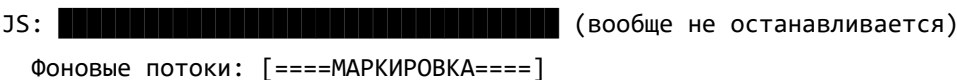
ПАРАЛЛЕЛЬНЫЙ (Minor GC):



ИНКРЕМЕНТАЛЬНЫЙ (резерв):



КОНКУРЕНТНЫЙ (Major GC Marking):



15.2.6. Триггеры: Когда просыпается сборщик?

Minor GC (Scavenger):

- Молодое поколение заполнено (размер Nursery + Intermediate достиг лимита).
- Запускается **очень часто** (десятки раз в секунду).
- Продолжительность: **< 5 мс** в норме.

Major GC (Mark-Compact):

- **Порог по памяти:** Использование старой кучи превысило динамически вычисляемый лимит.
- **Фрагментация:** Не удалось выделить объект большого размера, хотя суммарно памяти достаточно.
- **Вызов API:** `global.gc()` в Node.js (только с флагом `--expose-gc`).
- Продолжительность: может достигать **100-300 мс** на очень больших кучах, но в современном V8 — **20-50 мс** благодаря конкурентности.

Интересный факт из 2003 года:

JScript запускал GC **в начале каждого оператора**, если превышен порог по строкам/переменным/слотам массивов. Представьте производительность!

15.2.7. Утечки памяти: Когда GC не помогает

Сборщик мусора — не волшебник. Он собирает только **недостижимые** объекты. Если объект **достижим**, но **не нужен** — это утечка, и GC её не исправит.

Классические паттерны утечек (и их механика):

1. Замыкания с захватом контекста

```
javascript

function leakyModule() {
  const hugeData = new Array(1000000).fill('x');

  return function doSmallThing() {
    // Эта функция даже не использует hugeData!
```

```
    console.log('Hello');  
  };  
}
```

```
const fn = leakyModule();  
// hugeData остаётся в памяти, потому что движок НЕ анализирует,  
// какие переменные из захваченного контекста реально используются.  
// ВСЁ лексическое окружение сохраняется.
```

Решение: Использовать блочную область видимости для изоляции больших данных.

2. Отсоединённые DOM-элементы (Detached Nodes)

```
javascript  
  
const detachedElements = [];  
  
function createAndDetach() {  
  const div = document.createElement('div');  
  div.textContent = 'Leak';  
  // Не прикрепляем к DOM!  
  detachedElements.push(div); // Есть ссылка в массиве!  
}
```

Механика: Элемент не в документе, но JS на него ссылается → GC не трогает.

3. Слушатели событий без удаления

```
javascript  
  
class Component {  
  mount() {  
    window.addEventListener('resize', this.handleResize);  
    // Забыли removeEventListener  
  }  
}
```

Механика: `window` держит ссылку на метод компонента. Компонент недостижим? Нет, `window` до него достаёт.

4. Сборщики статистики и кэши без политик

```
javascript
```

```
const cache = new Map();
```

```
function process(obj) {  
  if (!cache.has(obj)) {  
    cache.set(obj, expensiveComputation(obj));  
  }  
  return cache.get(obj);  
}
```

// Объекты, переданные в process, навсегда остаются в cache (сильная ссылка)!

Решение: `WeakMap` — ссылка на ключ не учитывается при сборке мусора .

15.2.8. Диагностика: Инструменты памяти

Браузер (Chrome DevTools):

1. Heap Snapshot (Снимок кучи):

- Показывает **все** живые объекты.
- Позволяет искать утечки по `retainers` (кто держит ссылку).
- Фильтр: `detached` — отсоединённые DOM-элементы.

2. Allocation Timeline (Запись аллокаций):

- Записывает создание каждого объекта.
- Позволяет увидеть, какие функции генерируют мусор.
- Синие столбцы — аллокации, красные — сборки GC.

3. Performance GC meter:

- Включается в настройках DevTools.
- Показывает частоту и длительность GC в реальном времени.

Node.js:

```
javascript  
  
const v8 = require('v8');  
const heap = v8.getHeapStatistics();  
console.log({  
  used: (heap.used_heap_size / 1024 / 1024).toFixed(2) + ' MB',  
  limit: (heap.heap_size_limit / 1024 / 1024).toFixed(2) + ' MB',  
});
```

```
});
```

```
// Флаги запуска для диагностики:
```

```
// node --trace-gc app.js          # Логировать каждый GC
```

```
// node --max-old-space-size=256 # Ограничить память
```

```
// node --inspect                  # Подключить Chrome DevTools
```

15.2.9. Итоговая таблица: Сравнение алгоритмов V8

Параметр	Scavenger (Minor GC)	Mark-Compact (Major GC)
Где работает	Молодое поколение	Старое поколение + всё остальное
Алгоритм	Копирующий (Cheney)	Mark-Sweep + Mark-Compact
Перемещает объекты?	□ Всегда	□ Только выбранные страницы
Stop-The-World?	□ (короткая пауза)	▲ Маркировка — конкурентно, остальное — параллельно
Скорость	1-5 мс	20-100 мс
Частота	Очень часто	Редко
Барьеры	Write barrier для old->new	Write barrier для конкурентной маркировки

Резюме главы 15.2

1. **Сборка мусора в JavaScript — это не одна технология, а симбиоз алгоритмов**, разделённых по поколениям.
2. **Scavenger** — быстрый, параллельный, копирующий. Идеален для 95% объектов, которые умирают молодыми.
3. **Mark-Compact** — тяжёлая артиллерия. Использует **конкурентную маркировку**, чтобы JS не останавливался, и **параллельное уплотнение** для борьбы с фрагментацией.
4. **Генерационная гипотеза** — ключ к производительности. V8 тратит ресурсы там, где есть живые объекты, а не там, где пустота.

5. **Утечки памяти** — это всегда ошибка разработчика, а не GC. Слабые ссылки (`WeakMap`, `WeakSet`, `WeakRef`) — единственный способ сказать GC: «Мне это интересно, но не держи это в памяти ради меня».

15.3. Утечки памяти и их диагностика

Эпиграф: «Память — это кладбище забытых ссылок. Каждая утечка — это надгробие, которое вы не оплакали».

Введение: Тихий убийца

Представьте, что ваше приложение — это аквариум. В нём плавают рыбки (объекты). Сборщик мусора — это фильтр, который убирает отходы. Всё прекрасно.

Но однажды вы забыли закрыть крышку. В аквариум падает монетка (утечка). Потом ещё одна. Потом ещё. Рыбкам всё ещё комфортно. Фильтр работает. Но воды становится меньше, монетки занимают место.

Через месяц вы приходите и видите: рыбки плавают в лужице, задыхаются, а половина аквариума завалена металлоломом.

Это и есть утечка памяти. Она не убивает мгновенно. Она не вызывает ошибок сразу. Она просто постепенно, день за днём, отнимает ресурсы у вашего приложения, пока однажды вкладка не закрывается с сообщением «Out of Memory».

В этой главе мы научимся:

1. Видеть монетки в аквариуме (обнаруживать утечки).
 2. Понимать, откуда они падают (диагностировать причины).
 3. Чинить крышку (исправлять утечки).
-

15.3.1. Анатомия утечки: Что значит «утечка» в языке с GC?

В языках с ручным управлением памятью (C, C++) утечка — это **выделенная, но не освобождённая память**. Вы забыли вызвать `free()` — память потеряна навсегда.

В JavaScript (и Java, C#, Go) утечка — это иное понятие.

Утечка памяти в JS: Объект, который **достижим** из корней (стек, глобальные переменные), но **никогда больше не будет использован** программой.

Сборщик мусора видит: «Объект достижим → значит, он нужен». И не трогает его. А программа больше никогда к нему не обращается.

Метафора: Это как заброшенный склад, на который ведёт единственная дорога. Дорога есть (ссылка), но никто по ней не ходит. Склад стоит, занимает место, но не сносится, потому что формально у него есть владелец.

15.3.2. Классификация утечек: 5 кругов ада JavaScript

Круг 1: Глобальные переменные-аккумуляторы

Проблема: Необъявленная переменная (`x = 5`) или намеренное сохранение в `window`.

```
javascript

function accumulate(items) {
  // Забыли var/let/const!
  allItems = allItems || [];
  allItems.push(...items); // allItems = window.allItems
}

// Каждый вызов добавляет данные в window.allItems
accumulate([1, 2, 3]);
accumulate(new Array(1000000).fill('x'));
// Эти данные никогда не будут собраны GC
```

Диагностика:

- 🔵 В снимке кучи ищите свойства на `window`, которые раздуваются.
- 🔵 Используйте `'use strict'` — он превратит `allItems = ...` в ошибку.

Лечение:

```
javascript
```

```
// 1. Всегда объявляйте переменные
let allItems = [];

// 2. Если нужен кэш — используйте управляемый размер
const MAX_CACHE_SIZE = 100;
const cache = new Map();
cache.set(key, value);
if (cache.size > MAX_CACHE_SIZE) {
  const firstKey = cache.keys().next().value;
  cache.delete(firstKey);
}
```

Круг 2: Забытые таймеры и обработчики

Проблема: `setInterval` продолжает работать, а ссылки на его колбэк удерживают целый контекст.

```
javascript

function startUpdates() {
  const hugeData = new Array(1000000).fill('important');

  setInterval(() => {
    // Этот колбэк использует hugeData
    console.log(hugeData.length);
  }, 1000);
}

startUpdates();
// hugeData никогда не упрёт, потому что setInterval держит ссылку!
// Даже если функция startUpdates давно завершилась.
```

Диагностика:

- 🔵 Ищите активные `Timer` объекты в снимке кучи.
- 🔵 Смотрите на `setInterval` в Timeline — если они не очищаются при уходе со страницы — утечка.

Лечение:

```
javascript

class Component {
  constructor() {
    this.intervalId = null;
  }
}
```

```

    this.handleInterval = this.handleInterval.bind(this);
  }

  start() {
    // Сохраняем ID
    this.intervalId = setInterval(this.handleInterval, 1000);
  }

  stop() {
    // И очищаем при завершении
    if (this.intervalId) {
      clearInterval(this.intervalId);
      this.intervalId = null;
    }
  }

  handleInterval() {
    // Работаем
  }
}

// В React:
useEffect(() => {
  const timer = setInterval(() => {}, 1000);
  return () => clearInterval(timer); // Очистка при размонтировании
}, []);

```

Круг 3: Подписки без отписки (Event Listeners)

Проблема: Добавили обработчик на `window` или `EventEmitter`, но забыли `removeEventListener/off`.

```

javascript

class Overlay {
  constructor() {
    this.handleResize = () => {
      this.reposition();
    };

    window.addEventListener('resize', this.handleResize);
    // HET removeEventListener!
  }
}

```

```

reposition() {
  // ...
}
}

// Создаём оверлей
const modal = new Overlay();
// Закрываем оверлей и забываем
modal = null;
// Обработчик на window ВСЁ ЕЩЁ ВИСИТ! И ссылается на старый this!

```

Диагностика:

- DevTools > Elements > Event Listeners — видите лишние обработчики?
- Heap Snapshot — ищите `HTMLDivElement` и смотрите на retainers: часто висит на глобальном событии.

Лечение:

```

javascript

class SafeOverlay {
  constructor() {
    this.handleResize = this.handleResize.bind(this);
    this.init();
  }

  init() {
    window.addEventListener('resize', this.handleResize);
  }

  destroy() {
    // Симметричное удаление
    window.removeEventListener('resize', this.handleResize);
  }

  handleResize() {
    // ...
  }
}

// Использование
const overlay = new SafeOverlay();
// Когда нужно удалить:

```

```

overlay.destroy();

// React-паттерн:
useEffect(() => {
  const handler = () => {};
  window.addEventListener('resize', handler);
  return () => window.removeEventListener('resize', handler); // cleanup
}, []);

```

Круг 4: Замыкания и неожиданный контекст

Проблема: Замыкания захватывают **всё лексическое окружение**, а не только используемые переменные.

```

javascript

function createLeakyFunction() {
  const hugeArray = new Array(1000000).fill('leak');
  const someValue = 42;

  // Эта функция использует только someValue
  return function() {
    return someValue * 2;
  };
}

const fn = createLeakyFunction();
// hugeArray НЕ ИСПОЛЬЗУЕТСЯ, но живёт в памяти!
// Движок не анализирует, какие переменные реально нужны.

```

Визуализация замыкания в памяти:

```

text

Контекст выполнения createLeakyFunction (после завершения)
├─ hugeArray: ● (ссылка на массив 1МБ в куче) ← НЕ ИСПОЛЬЗУЕТСЯ!
├─ someValue: 42
└─
   ↑
   └─ [[Scope]] возвращённой функции — хранит ВСЁ контекст!

```

Диагностика:

- Heap Snapshot — найдите Closure объекты.
- Посмотрите на их размер — часто видно, что захвачен огромный массив.

🔵 Ищите паттерн: функция возвращается из функции, внутри которой были большие данные.

Лечение:

javascript

// СПОСОБ 1: Изолировать большие данные в блоке

```
function createFixedFunction() {  
  let someValue;  
  { // Новый блок области видимости  
    const hugeArray = new Array(1000000).fill('leak');  
    // Используем hugeArray здесь...  
    someValue = hugeArray.length; // Извлекаем только нужное  
  } // hugeArray выходит из области видимости → GC может собрать  
  
  return function() {  
    return someValue * 2;  
  };  
}
```

// СПОСОБ 2: Обнуление ссылок вручную

```
function createManualCleanFunction() {  
  const hugeArray = new Array(1000000).fill('leak');  
  const someValue = 42;  
  
  const result = function() {  
    return someValue * 2;  
  };  
};
```

// Явно удаляем ссылку на большие данные

// (работает, т.к. они больше не нужны)

```
hugeArray = null;  
  
return result;  
}
```

// СПОСОБ 3: WeakMap для опциональных данных (редко, но элегантно)

```
const optionalData = new WeakMap();  
  
function process() {  
  const data = { /* большие данные */ };  
  const fn = () => {  
    // Если понадобятся данные — достанем из WeakMap  
    const needed = optionalData.get(fn);
```

```
// ...
};
optionalData.set(fn, data);
return fn;
}
// Когда fn умрёт — data умрёт автоматически!
```

Круг 5: Detached DOM Nodes — призраки в машине

Это король утечек в браузере. Элемент удалён из DOM, но ссылка на него осталась в JavaScript. Браузер не может освободить память, потому что JS на него ссылается.

```
javascript

function createDetachedLeak() {
  const parent = document.createElement('div');
  const child = document.createElement('span');
  child.textContent = 'Hello';
  parent.appendChild(child);

  // Глобальное хранилище!
  window.cache = window.cache || [];
  window.cache.push(child); // Сохраняем ТОЛЬКО child

  // parent никто не хранит, parent умрёт.
  // child живёт в cache, но parent был удалён!
  // child теперь отсоединён (detached) — у него есть родитель в памяти,
  // но этот родитель не привязан к document!

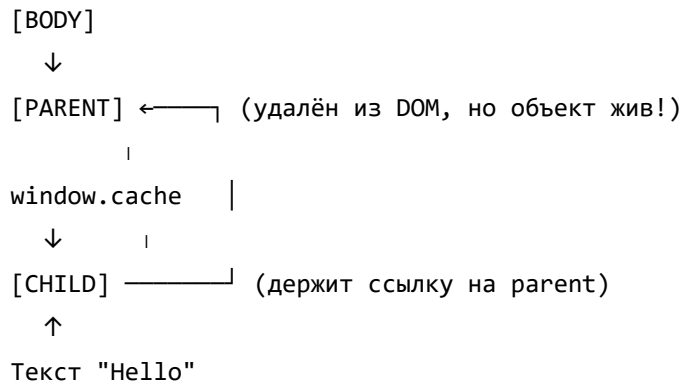
  return document.body.appendChild(parent);
}

// Позже:
document.body.removeChild(parent); // Удалили из DOM
// parent и child ВСЁ ЕЩЁ В ПАМЯТИ! window.cache[0] держит child,
// child держит parent через parentElement.
```

Визуализация:

```
text

Документ (DOM дерево)
↓
```

Диагностика:

1. Chrome DevTools → Memory → Heap Snapshot.
2. В фильтре введите `Detached`.
3. DevTools покажет все отсоединённые DOM-узлы.
4. Раскройте узел → Retainers — кто держит ссылку?

Лечение:

javascript

// ПРАВИЛО: Если удаляете из DOM — обнулите ссылки

```
class SafeComponent {
  constructor() {
    this.node = document.createElement('div');
    this.child = document.createElement('span');
    this.node.appendChild(this.child);
  }

  mount(parent) {
    parent.appendChild(this.node);
  }

  unmount() {
    // 1. Удаляем из DOM
    if (this.node.parentNode) {
      this.node.parentNode.removeChild(this.node);
    }

    // 2. Очищаем ссылки на дочерние элементы
    this.child = null;

    // 3. Очищаем ссылку на родительский узел
    this.node = null;
  }
}
```

```
}
```

// В React: React сам это делает. Но если вы работаете с прямым DOM:

```
const div = document.getElementById('temp');
```

```
div.remove();
```

// div = null; // ЗАБЫЛИ? → Утечка, если где-то ещё есть ссылка!

15.3.3. Инструментарий детектива: Как искать утечки

Метод 1: Performance Timeline (Быстрый тест)

Цель: Увидеть, растёт ли память при повторяющихся действиях.

Протокол:

1. Откройте вкладку Performance в DevTools.
2. Включите **Memory** чекбокс (показывает график памяти).
3. Нажмите **Record** (●).
4. Выполните действие, которое подозреваете в утечке (открыть/закрыть модальку, загрузить данные).
5. Нажмите **Stop**.
6. Смотрите на график **JS Heap**.

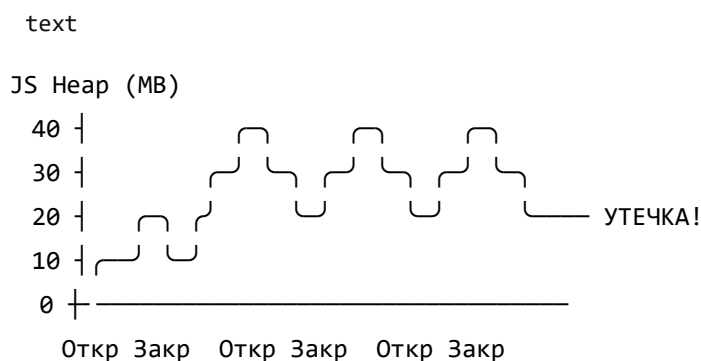
Интерпретация:

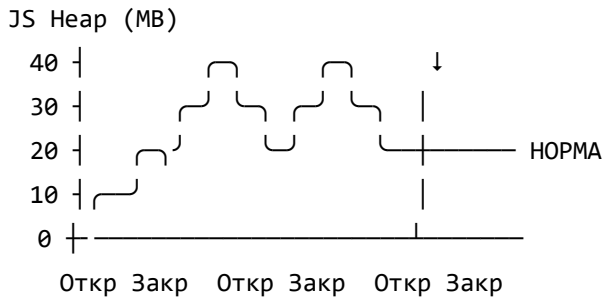
□ **Норма:** График идёт вверх (аллокации), потом резко падает (GC собрал мусор). Линия возвращается к исходному уровню.

▲ **Подозрение:** Каждый цикл добавляет память, и она не возвращается. График — лестница вверх.

□ **Утечка:** Память неуклонно растёт, GC не помогает.

Скриншот (текстовая имитация):





Метод 2: Три снимка (Heap Snapshots)

Это золотой стандарт диагностики.

Протокол:

1. Memory → Heap Snapshot (Сохранить как `snapshot-1.heapsnapshot`).
2. Выполните действие, создающее объекты (откройте модальку).
3. Сделайте второй снимок (`snapshot-2`).
4. Закройте модальку/удалите объекты.
5. **Принудительно запустите GC** (кнопка 🗑️ внизу Memory панели).
6. Сделайте третий снимок (`snapshot-3`).

Анализ:

- Сравните `snapshot-2` и `snapshot-3`.
- Выберите режим **Comparison** (Сравнение) вместо Summary.
- Сортируйте по **# Delta** (разница в количестве объектов).
- Смотрите на **Size Delta** (разница в байтах).

Что искать:

1. **Detached DOM-узлы** (фильтр: `Detached`).
2. **Объекты ваших классов** (фильтр: `Component`, `ViewModel`).
3. **Замыкания** (фильтр: `Closure`) — они не должны накапливаться.
4. **Массивы и строки большого размера**, которые не очищаются.

Пример:

text

Constructor	# Delta	Size Delta
Detached HTMLLIElement	+124	+156,456 🟩 (Вот утечка!)
MyComponent	+12	+34,896 🟩 (12 компонентов

		не удалились)
Closure	+45	+12,890  (замыкания висят)
(String)	+2,345	+567,890 (возможно, кэш строк)

Метод 3: Allocation Profiler (Трассировка аллокаций)

Помогает ответить на вопрос: **КТО создаёт эти объекты?**

Протокол:

1. Memory → **Allocation instrumentation on timeline**.
2. Нажмите Start.
3. Выполните действие.
4. Остановите запись.

Вы увидите: Временную шкалу с синими полосками. Чем выше полоска — тем больше памяти выделено в этот момент.

Интерактив:

- Выделите участок с высокой аллокацией.
- Внизу появится **список объектов**, созданных в этот момент.
- Раскройте объект → **Retainers** → вы увидите **стеки вызовов** (какая функция создала этот объект).

Пример:

text

Объект: Array (size 1,000,000)

└─ Стек:

```
at createReport (report.js:42)
at processData (data.js:156)
at onClick (ui.js:23) // Ага, при клике!
```

Метод 4: Node.js — Встроенный профайлер

Для серверного JavaScript.

```
javascript

// 1. Включите флаги при запуске
// node --inspect --max-old-space-size=256 server.js

// 2. Используйте v8.writeHeapSnapshot() (Node.js v11.13+)
const v8 = require('v8');

setInterval(() => {
  if (process.memoryUsage().heapUsed > 200 * 1024 * 1024) {
    const timestamp = Date.now();
    const filename = `heap-${timestamp}.heapsnapshot`;
    v8.writeHeapSnapshot(filename);
    console.log(`📸 Снимок кучи сохранён: ${filename}`);
  }
}, 60000); // Проверяем каждую минуту
```

Анализ: Загрузите `.heapsnapshot` в Chrome DevTools (Profiles → Load).

15.3.4. Тяжёлая артиллерия: WeakRef и FinalizationRegistry

В ES2021 появились инструменты для **финального контроля** над памятью. Они не для ежедневного использования, но для сложных кэшей — спасение.

WeakRef — слабая ссылка

Обычная ссылка (`obj`) — сильная. GC не тронет объект, пока есть хоть одна сильная ссылка.

WeakRef не защищает объект от сборки.

```
javascript

const cache = new Map();

function fetchData(url) {
```

```

if (cache.has(url)) {
  // Обычный кэш — вечный!
  return cache.get(url);
}
const data = heavyLoad(url);
cache.set(url, data);
return data;
}

```

C WeakRef:

```

javascript

const cache = new Map();

function fetchData(url) {
  if (cache.has(url)) {
    const weakRef = cache.get(url);
    const data = weakRef.deref(); // Пытаемся получить объект
    if (data) return data;
    // Объект уже собрал GC! Удаляем запись из кэша.
    cache.delete(url);
  }

  const data = heavyLoad(url);
  // Сохраняем СЛАБУЮ ссылку
  cache.set(url, new WeakRef(data));
  return data;
}

```

Важно: `deref()` может вернуть `undefined`, если GC уже собрал объект.

FinalizationRegistry — регистрация смерти

Позволяет выполнить код, когда объект **умер**.

```

javascript

const registry = new FinalizationRegistry((heldValue) => {
  // Этот колбэк вызовется ПОСЛЕ того, как объект собран GC!
  console.log(`Объект ${heldValue} был уничтожен`);
  // Здесь можно закрыть файл, освободить соединение, записать лог.
});

function createResource() {

```

```
const resource = { name: 'database connection' };
registry.register(resource, resource.name); // Второй аргумент — heldValue
return resource;
}
```

```
let res = createResource();
res = null; // Теперь объект готов к GC
// ... когда GC случится, в консоли будет:
// "Объект database connection был уничтожен"
```

Предупреждение: Не полагайтесь на точное время вызова. GC может не запуститься минутами. `FinalizationRegistry` — для дебага и очистки внешних ресурсов, не для бизнес-логики.

15.3.5. Чек-лист аудита памяти

Перед релизом:

- 🔵 **Глобальные переменные:** Нет ли случайного `window.myVar = ...?`
- 🔵 **setInterval/Timeout:** Все ли интервалы очищаются при `unmount/destroy`?
- 🔵 **Event Listeners:** Все ли подписки на глобальные объекты (`window, document, EventEmitter`) имеют симметричный `remove`?
- 🔵 **DOM-ссылки:** Есть ли переменные, хранящие удалённые элементы? (Поиск по `Detached` в снимках).
- 🔵 **Кэши:** Имеют ли ограничение по размеру? Используют ли `WeakMap` для опциональных данных?
- 🔵 **Замыкания:** Нет ли функций, возвращаемых из функций, внутри которых были огромные массивы?
- 🔵 **Сторонние библиотеки:** Проверьте их утечки (Charts, Video плееры, Карты).

Автоматизация в Node.js:

```
javascript

const v8 = require('v8');
const mem = process.memoryUsage();

// Критические пороги (настройте под проект)
if (mem.heapUsed > 500 * 1024 * 1024) { // 500 МБ
  console.error('❌ Память > 500 МБ! Сохраняем снимок...');
  v8.writeHeapSnapshot('leak-autosave.heapsnapshot');
}
```

В браузере:

```
javascript

// Performance Observer API
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (entry.entryType === 'memory' && entry.totalJSHeapSize > 500 * 1024 * 1024) {
      console.warn('▲ Потребление памяти > 500 МБ');
      // Можно отправить метрику в аналитику
    }
  }
});
observer.observe({ entryTypes: ['memory'] });
```

15.3.6. Реальный кейс: Погоня за призраком

Симптом: Внутренняя CRM-система на React тормозит после часа работы. Перезагрузка помогает, но пользователи злятся.

Шаг 1: Performance Timeline

- Записываем 5 циклов: открыть карточку клиента → закрыть.
- График памяти: **лестница вверх**. После 5 цикла GC не возвращает память к исходному уровню (+50 МБ за 5 циклов).
- Вердикт:** Есть утечка.

Шаг 2: Три снимка

- Snapshot 1: Только что загрузили страницу.
- Snapshot 2: Открыли карточку №1.
- Snapshot 3: Закрыли карточку, запустили GC.

Результат сравнения:

```
text

Constructor      | Delta | Size
-----|-----|-----
Detached HTMLDivElement | +45 | +1.2 МБ
Closure (CustomerCard) | +45 | +0.5 МБ
```

Найдено: 45 отсоединённых `div` и 45 замыканий компонента `CustomerCard`.

Шаг 3: Allocation Timeline

- Ставим запись.
- Открываем карточку.
- Смотрим, кто создаёт `CustomerCard` → `CustomerCardContainer`.
- Смотрим, кто держит ссылки после закрытия.

Retainers для Closure:

text

Closure (CustomerCard)

└─ Global / window.__reactEventHandlers

Ага! React использует глобальный кэш для обработчиков событий. Но почему он не очищается?

Шаг 4: Код

jsx

```
class CustomerCard extends React.Component {
  componentDidMount() {
    // ПРОБЛЕМА!
    window.addEventListener('resize', this.handleResize);
    // НЕТ componentWillUnmount!
  }

  handleResize = () => {
    this.updatePosition();
  };

  // Забыли!
  // componentWillUnmount() {
  //   window.removeEventListener('resize', this.handleResize);
  // }
}
```

Решение: Добавить `componentWillUnmount` или, для функциональных КОМПОНЕНТОВ:

jsx

```
useEffect(() => {
  const handler = () => { updatePosition(); };
  window.addEventListener('resize', handler);
  return () => window.removeEventListener('resize', handler);
});
```

```
}, []);
```

Результат после фикса:

- Повторный тест: память стабильна, GC возвращает к исходному уровню.
 - Утечка устранена.
-

Резюме главы 15.3

1. **Утечка в JavaScript — это достижимый, но ненужный объект.** GC не может его собрать, потому что видит ссылку.

2. **5 главных кругов ада:**

- ➔ Глобальные переменные
- ➔ Забытые таймеры
- ➔ Подписки без отписки
- ➔ Замыкания с захваченным контекстом
- ➔ Отсоединённые DOM-узлы

3. **Диагностика — это научный метод:**

- ➔ Performance → быстрый тест на лестницу.
- ➔ Heap Snapshots → сравнение «до/после» с ручным GC.
- ➔ Allocation Timeline → кто преступник?
- ➔ Retainers → мотив и оружие.

4. **Лечение:**

- ➔ Симметрия: `addEventListener` → `removeEventListener`, `setInterval` → `clearInterval`.
- ➔ Обнуление ссылок после удаления из DOM.
- ➔ Блочная область видимости для замыканий.
- ➔ `WeakMap/WeakRef` для кэшей.
- ➔ React: `useEffect` `cleanup`.

5. **Профилактика:**

- ➔ Линтеры: `eslint-plugin-react-hooks`, `no-unused-vars`.
- ➔ Мониторинг: `v8.writeHeapSnapshot()` при превышении порога.
- ➔ Культура кода: всегда думайте, **кто будет держать ссылку** на этот объект.

15.4. Профилирование производительности

Эпиграф: «Не оптимизируй то, что не профилировал. Измерь — или умри под грузом собственных догадок».

Введение: Искусство измерения

Представьте, что вы — капитан корабля. Ваш JavaScript-движок — это огромный турбинный двигатель. Он гудит, выпускает пар, вращает винты. Но вдруг корабль начинает терять скорость.

Что вы сделаете?

× «Наверное, надо заменить все `let` на ``const`» — это замена лампочки в каюте, когда заглох двигатель.

× «Я слышал, `for` быстрее ``forEach`» — это покраска корпуса, когда сломан винт.

□ **Правильно:** Вы спуститесь в машинное отделение с приборами, измерите давление в каждом цилиндре, температуру выхлопа, обороты турбины. **Профилирование** — это и есть ваши приборы.

Эта глава — навигационная карта машинного отделения V8.

Мы научимся:

1. **Измерять** — получать точные цифры, а не ощущения.
 2. **Находить** — локализовать узкое место с точностью до строки.
 3. **Понимать** — почему этот код медленный (деоптимизация, скрытые классы, GC).
 4. **Исправлять** — принимать обоснованные решения.
-

15.4.1. Фундаментальный принцип: Измеряй, а не гадай

Первый закон профилирования:

Человеческий мозг ужасен в угадывании узких мест. Мы запоминаем медленный код, который написали вчера, и не замечаем медленный код, написанный год назад.

Второй закон профилирования:

90% времени программа проводит в 10% кода. Ваша задача — найти эти 10%, а не переписывать всё.

Третий закон профилирования:

Микрооптимизации ($++i$ vs $i++$) имеют значение только внутри этих 10%. Снаружи они — потеря времени и читаемости.

Эксперимент-опровержение:

```
javascript
```

```
// Версия А: классический for
```

```
console.time('for');
let sum = 0;
for (let i = 0; i < 1000000; i++) {
  sum += i;
}
console.timeEnd('for');
```

```
// Версия Б: while
```

```
console.time('while');
sum = 0;
let j = 0;
while (j < 1000000) {
  sum += j;
  j++;
}
console.timeEnd('while');
```

```
// Версия В: forEach на виртуальном массиве
```

```
console.time('forEach');
sum = 0;
Array.from({ length: 1000000 }, (_, i) => sum += i);
console.timeEnd('forEach');
```

```
// Результаты (V8):  
// for: 3.2ms  
// while: 3.1ms (незначимо)  
// forEach: 45ms (ПОД 1 МЛН ЭЛЕМЕНТОВ ЭТО УБИЙСТВЕННО!)
```

Вывод: Проблема не в выборе `for` vs `while`, а в создании огромного массива в памяти. Профилирование показывает где, а не как именно писать цикл.

15.4.2. Инструмент 1: `performance.now()` и User-Timing API

Самый простой и точный способ измерить участок кода.

```
javascript  
  
// performance.now() – высокоточный таймер (до 5 мкс в современных браузерах)  
const start = performance.now();  
// ... измеряемый код ...  
const end = performance.now();  
console.log(`Операция заняла ${end - start} мс`);
```

User-Timing API (маркеры и меры):

```
javascript  
  
// Ставим метки  
performance.mark('start-fetch');  
await fetch('/api/data');  
performance.mark('end-fetch');  
  
// Измеряем расстояние между метками  
performance.measure('fetch-duration', 'start-fetch', 'end-fetch');  
  
// Получаем результаты  
const measures = performance.getEntriesByName('fetch-duration');  
console.log(measures[0].duration); // 342.12 мс  
  
// Очистка (чтобы не засорять память)  
performance.clearMarks('start-fetch');  
performance.clearMarks('end-fetch');  
performance.clearMeasures('fetch-duration');
```

Практический пример: измеряем скорость рендера React

jsx

```
function ProfilePage() {
  useEffect(() => {
    performance.mark('render-start');

    return () => {
      performance.mark('render-end');
      performance.measure('render-duration', 'render-start', 'render-end');
      const entry = performance.getEntriesByName('render-duration')[0];

      // Отправляем в аналитику, если долго
      if (entry.duration > 16) { // 60fps = 16.6ms
        reportMetric('slow-render', {
          duration: entry.duration,
          component: 'ProfilePage'
        });
      }

      performance.clearMarks();
      performance.clearMeasures();
    };
  }, []);

  return <div>{/* ... */</div>;
}
```

Ограничение: `performance.now()` измеряет синхронный код. Для асинхронных операций используйте маркеры.

15.4.3. Инструмент 2: Chrome DevTools Performance Panel

Это **швейцарский нож** профилирования. Не просто график — это рентген вашего приложения.

15.4.3.1. Запись профиля

Протокол:

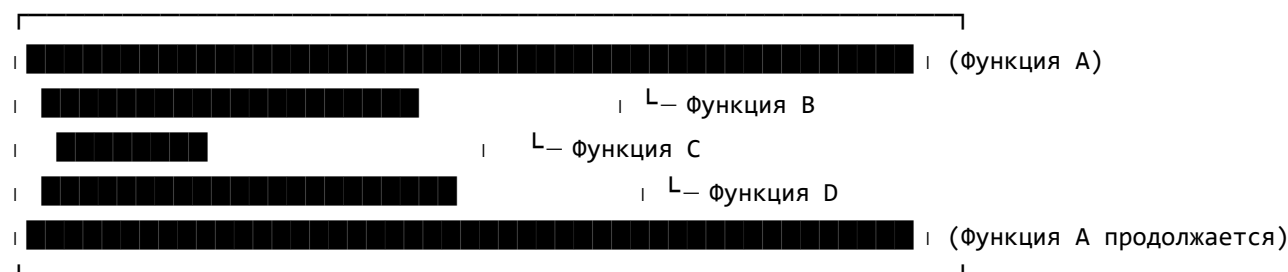
1. DevTools → Performance (⌘+E / Ctrl+E).
2. Настройки (шестерёнка):

-  **Enable advanced paint instrumentation** — детали по отрисовке.
 -  **Network** — запросы на таймлайне.
 -  **Memory** — график памяти.
3. Кнопка **Record** (●).
 4. Выполните действие.
 5. **Stop**.

15.4.3.2. Чтение flame chart (огненная диаграмма)

Это самый важный навык. Flame chart показывает, **кто вызвал кого и сколько времени занял.**

text

[illegible]

Интерпретация:

- **Ширина** блока = время выполнения.
- **Вложенность** = стек вызовов.
- **Цвета:**
 - Жёлтый — JavaScript.
 - Фиолетовый — рендеринг/стили.
 - Зелёный — отрисовка (Paint).
 - Синий — сетевые запросы.
 - Серый — системные вызовы, GC.

Что искать:

1. **Длинные жёлтые блоки (> 50 мс)** — медленный JS.
2. **Частые фиолетовые блоки** — излишний Reflow/Layout.
3. **Зелёные блоки на каждом кадре** — тяжёлая отрисовка.
4. **Красные треугольники в углу** — предупреждения (например, forced reflow).

15.4.3.3. Bottom-Up / Call Tree

Bottom-Up — агрегированный вид. Показывает **какие функции суммарно заняли больше всего времени**, независимо от того, кто их вызвал.

Как читать:

Функция	Само время	Общее время	Вызовы
processArray	320ms	320ms	156
JSON.parse	210ms	210ms	23
updateUI	45ms	980ms	12

Само время (Self Time) — время, проведённое **внутри** функции, без учёта вызванных ею функций.

Общее время (Total Time) — включая всё, что вызвала функция.

Инсайт: Если у `updateUI` само время 45ms, а общее 980ms — значит, она вызвала кучу других медленных функций.

15.4.4. Инструмент 3: V8 Deoptimization Tracing

Это высший пилотаж. Иногда код медленный не потому, что в нём много операций, а потому, что V8 **перестал его оптимизировать**.

Проблема: V8 компилирует «горячий» код в машинный. Но если он видит нестабильные типы (сегодня `number`, завтра `string`), он **деоптимизирует** функцию обратно в интерпретатор.

Как диагностировать:

```
bash
```

```
node --trace-opt --trace-deopt app.js
```

Пример вывода:

```
text
```



```
[deoptimizing (DEOPT eager): begin 0x3f9eb8fde9e9 function sum]
kind: wrong call target
type: double expected, got oddball (undefined)
deoptimizing from bytecode
...
```

Перевод: V8 ожидал число (`double`), получил `undefined`. Оптимизация отменена.

В браузере: Chrome DevTools → Performance → Включить `Runtime Call Stats`.

Что с этим делать:

1. **Стабилизируйте типы.** Не меняйте тип переменной.
2. **Не добавляйте свойства в объекты после создания** (используйте `Object.defineProperty` сразу все).
3. **Избегайте `delete`** — он переводит объект в словарный режим.

javascript

// ПЛОХО: деоптимизация

```
function add(x, y) {
  return x + y;
}

add(1, 2);      // оптимизировано под int
add(1.5, 2.3); // double — ещё ок
add('1', '2'); // !!! строка — деоптимизация!
```

// ХОРОШО: разделить функции

```
function addNumbers(x, y) { return x + y; }
function addStrings(x, y) { return x + y; }
```

15.4.5. Инструмент 4: Memory Profiler (уже знаком)

Из раздела 15.3 мы знаем, что память и производительность связаны. GC — это паузы.

Как измерить влияние GC:

1. Performance Panel → Включите **Memory**.
2. Смотрите на **график JS Heap**.
3. Где резкие падения — там работал GC.

4. Если падения частые и глубокие — аллокаций слишком много.

Диагноз: «Приложение создаёт тонны мусора».

Лечение:

- Избегайте создания объектов в циклах.
- Используйте пулы объектов (object pools).
- Переиспользуйте структуры данных.

javascript

// ПЛОХО: 1000 объектов за кадр

```
function renderBad(items) {  
  return items.map(item => ({  
    id: item.id,  
    name: item.name,  
    timestamp: Date.now()  
  }));  
}
```

// ЛУЧШЕ: мутировать существующие объекты

```
function renderBetter(items, cache) {  
  items.forEach((item, i) => {  
    cache[i] = cache[i] || {};  
    cache[i].id = item.id;  
    cache[i].name = item.name;  
    cache[i].timestamp = Date.now();  
  });  
  return cache.slice(0, items.length);  
}
```

15.4.6. Инструмент 5: React DevTools Profiler

Для React-приложений есть отдельный, очень мощный инструмент.

Установка: Расширение React DevTools → Вкладка **Profiler**.

Что показывает:

- **Flamegraph** компонентов.
- **Время рендера** каждого компонента.

- Почему рендер произошёл (props changed, state changed, hooks).
- Пропущенные оптимизации (компонент рендерится, хотя props не менялись).

Ключевой приём:

1. Запись взаимодействия.
2. Поиск компонентов с **серым подсветом** — они рендерились без необходимости.
3. Обернуть в `React.memo()` или использовать `useMemo/useCallback`.
jsx

// До: рендерится каждый раз при любом изменении

```
const ExpensiveList = ({ items }) => {  
  return items.map(/* ... */);  
};
```

// После: рендерится только если items изменились

```
const ExpensiveList = React.memo(({ items }) => {  
  return items.map(/* ... */);  
});
```

15.4.7. Методология: Поиск узкого места (пошагово)

Шаг 1: Гипотеза и скрининг

- Воспроизведите медленное действие.
- Performance Panel → быстрая запись (3-5 секунд).
- Смотрите на общую картину: это JS, рендеринг, сеть или GC?

Шаг 2: Локализация (Bottom-Up)

- Переключитесь на вкладку **Bottom-Up**.
- Сортируйте по **Self Time**.
- Первые 3-5 функций — ваши кандидаты.

Шаг 3: Детальный разбор

- Кликните на функцию → **Source**.
- DevTools покажет точную строку в исходном коде.
- Оцените сложность: $O(n^2)$? Забытый кэш? Неправильная структура данных?

Шаг 4: Эксперимент

- Измерьте `performance.now()` до/после гипотетического исправления.
- **Цифры не врут.** Если ускорение $< 5\%$ и не в горячем пути — не стоит того.

Шаг 5: Проверка регрессии

- После исправления — повторите запись.
 - Убедитесь, что не стало хуже в другом месте.
-

15.4.8. Типичные проблемы производительности и их решения

Проблема 1: Forced Reflow (принудительная перекomпоновка)

Симптомы: В flame chart много фиолетовых блоков Layout, возникающих сразу после JS.

Причина: Вы читаете геометрию (`offsetWidth`, `scrollTop`) после изменения стилей.

javascript

// ПЛОХО: 3 reflow за 1 операцию

```
element.style.width = '100px';
console.log(element.offsetWidth); // ← Reflow #1
element.style.height = '200px';
console.log(element.offsetHeight); // ← Reflow #2
element.style.margin = '10px';
console.log(element.offsetTop); // ← Reflow #3
```

// ХОРОШО: 1 reflow (в конце)

```
element.style.width = '100px';
element.style.height = '200px';
element.style.margin = '10px';
// Принудительно читаем после всех изменений
console.log(element.offsetWidth); // ← Reflow #1 (один)
```

Фикс: Используйте `transform` для анимаций (не вызывает Reflow). Читайте геометрию после пакетной записи.

Проблема 2: Медленный `JSON.parse` на больших данных

Симптом: Длинные жёлтые блоки в функции `JSON.parse`.

Причина: Вы парсите 50 МБ JSON при каждом ответе сервера.

Фикс:

- Сжимайте данные на сервере (gzip + JSON).
- Используйте `stream` API (в Node.js) или `JSON.parse` с `reviver`, если нужно только подмножество.
- Рассмотрите бинарные форматы (MessagePack, Protocol Buffers).

javascript

// В браузере: используем streams

```
const response = await fetch('/big-data.json');
const reader = response.body.pipeThrough(new TextDecoderStream()).getReader();
// Парсим чанками (сложно, но возможно)
```

// В Node.js: streams + JSON.parse построчно

```
const fs = require('fs');
const readline = require('readline');
const rl = readline.createInterface({ input: fs.createReadStream('big.json') });
for await (const line of rl) {
  const obj = JSON.parse(line); // Парсим по одной строке
}
```

Проблема 3: Утечка памяти из-за замыканий (повтор из 15.3)

Симптом: График памяти — лестница вверх. GC работает часто и долго.

Диагностика: Heap Snapshot → ищите Closure → смотрите размер.

Фикс: Блочная область видимости.

Проблема 4: Излишний рендер в React/Vue

Симптом: React Profiler показывает серые компоненты.

Фикс: `React.memo`, `useMemo`, `useCallback`, `PureComponent`.

jsx

// До: каждый рендер родителя вызывает рендер дочернего

```
const Child = ({ name }) => <div>{name}</div>;
```

// После: рендер только при изменении props

```
const Child = React.memo(({ name }) => <div>{name}</div>);
```

Важно: `React.memo` — это тоже код. Не оборачивайте всё подряд, только тяжёлые компоненты и компоненты в циклах.

Проблема 5: Медленные регулярные выражения

Симптом: Функция с регуляркой висит в Bottom-Up, и у неё большое Self Time.

Причина: Catastrophic Backtracking.

javascript

*// ОПАСНО: (a+)*b — при строке "aaaaaaaaaaaaaas" будет экспонента!*

```
const regex = /^(a+)*b$/;
```

// БЕЗОПАСНО: переписать без вложенных квантификаторов

```
const safeRegex = /^(a+)*b$/; // Замена на /^a+b$/
```

Фикс:

- Избегайте вложенных квантификаторов `((a+)*, (a*)*)`.
 - Тестируйте регулярки на инструментах типа regex101.com.
 - Для сложных разборов используйте парсеры, а не регулярки.
-

15.4.9. Профилирование Node.js (сервер)

На сервере другие приоритеты: пропускная способность, задержка ответа, использование CPU.

Инструмент 1: встроенный профайлер V8

bash

Запись профиля в файл

```
node --prof app.js
```

```
# Чтение профиля (сгенерирует isolate-*.Log)
node --prof-process isolate-0x*.log > profile.txt
```

Инструмент 2: Clinic.js (сторонний, но золотой стандарт)

```
bash

npm install -g clinic
clinic doctor -- node app.js
# Открывает браузер с интерактивным отчётом
```

Что показывает Clinic:

- 🔵 **Bubbleprof** — асинхронные операции (где узкие места ввода-вывода).
- 🔵 **Flame** — CPU профиль (классический флеймчарт).
- 🔵 **Hear profiler** — утечки памяти.

Инструмент 3: 0x (ещё один флеймчарт)

```
bash

npm install -g 0x
0x app.js
# Генерирует flamegraph.html
```

Критически важно для сервера:

- 🔵 **Не блокируйте Event Loop** — `fs.readFileSync` в обработке запроса — смерть.
- 🔵 **Измеряйте TTFB (Time To First Byte)** — время от запроса до первого байта ответа.
- 🔵 **Мониторьте Event Loop Lag** — если задержка > 100 мс, сервер задыхается.

```
javascript

// Мониторинг Event Loop в Node.js
let lastCheck = Date.now();
setInterval(() => {
  const now = Date.now();
  const lag = now - lastCheck - 1000; // Ожидали 1000 мс, а прошло больше?
  if (lag > 50) {
    console.error(`🚩 Event Loop Lag: ${lag}ms`);
    // Отправить в метрики
  }
  lastCheck = now;
}, 1000);
```

15.4.10. Таблица: Инструменты и их применение

Инструмент	Платформа	Что измеряет	Когда использовать
<code>performance.now()</code>	Браузер/Node	Конкретный участок кода	Микро-бенчмарки, проверка гипотез
Chrome DevTools Perf	Браузер	Всё (JS, render, paint, network)	Повседневная отладка
React DevTools	Браузер	Рендер компонентов	Оптимизация React
<code>node --prof</code>	Node.js	CPU профиль	Серверные узкие места
Clinic.js	Node.js	I/O, CPU, память	Комплексный анализ сервера
V8 Deopt Trace	Node.js/ Chrome	Деоптимизация	Понимание "почему V8 медленно"
Memory Snapshot	Браузер/Node	Утечки памяти	Раз в спринт

Резюме главы 15.4

1. **Не думайте — измеряйте.** Ваши догадки о том, что медленно, с вероятностью 80% неверны.
2. **Flame Chart — ваш главный друг.** Научитесь читать ширину блоков и стек вызовов.
3. **Bottom-Up покажет убийц.** Функции, которые суммарно едят больше всего времени.
4. **React DevTools обязателен для React-проектов.** Без него вы оптимизируете вслепую.
5. **V8 — не чёрная магия.** Он любит стабильные типы и предсказуемую структуру объектов.
6. **Node.js — про I/O.** Если сервер медленный, скорее всего, проблема в базе данных, API или блокировке Event Loop.
7. **Профилирование — это навык.** Чем чаще вы смотрите на Performance Panel, тем быстрее находите проблемы.

15.5. Оптимизация критического пути рендеринга

Эпиграф: «Пользователь не видит ваш код. Он видит пиксели. Чем быстрее пиксели появляются на экране, тем счастливее пользователь».

Введение: Миллисекунды решают судьбу

Представьте два интернет-магазина. Они продают один и тот же товар по одной цене. У них одинаковый ассортимент и одинаковые отзывы. Но один загружается за **0.9 секунды**, а другой — за **2.5 секунды**.

Кто победит?

Исследования Amazon, Google и Akamai сходятся в одном: **каждые 100 миллисекунд задержки снижают конверсию на 1–7%**. Задержка в 1 секунду — это миллиарды долларов потерь для крупных ритейлеров.

Парадокс фронтенд-разработчика:

Мы пишем красивый, модульный, хорошо структурированный код. Но пользователь не видит ни вашу архитектуру, ни ваши чистые функции. Он видит **белый экран**.

Критический путь рендеринга (Critical Rendering Path, CRP) — это **путь от байтов HTML к пикселям на экране**. Оптимизация CRP — это не полировка. Это **хирургия выживания**.

В этой главе мы:

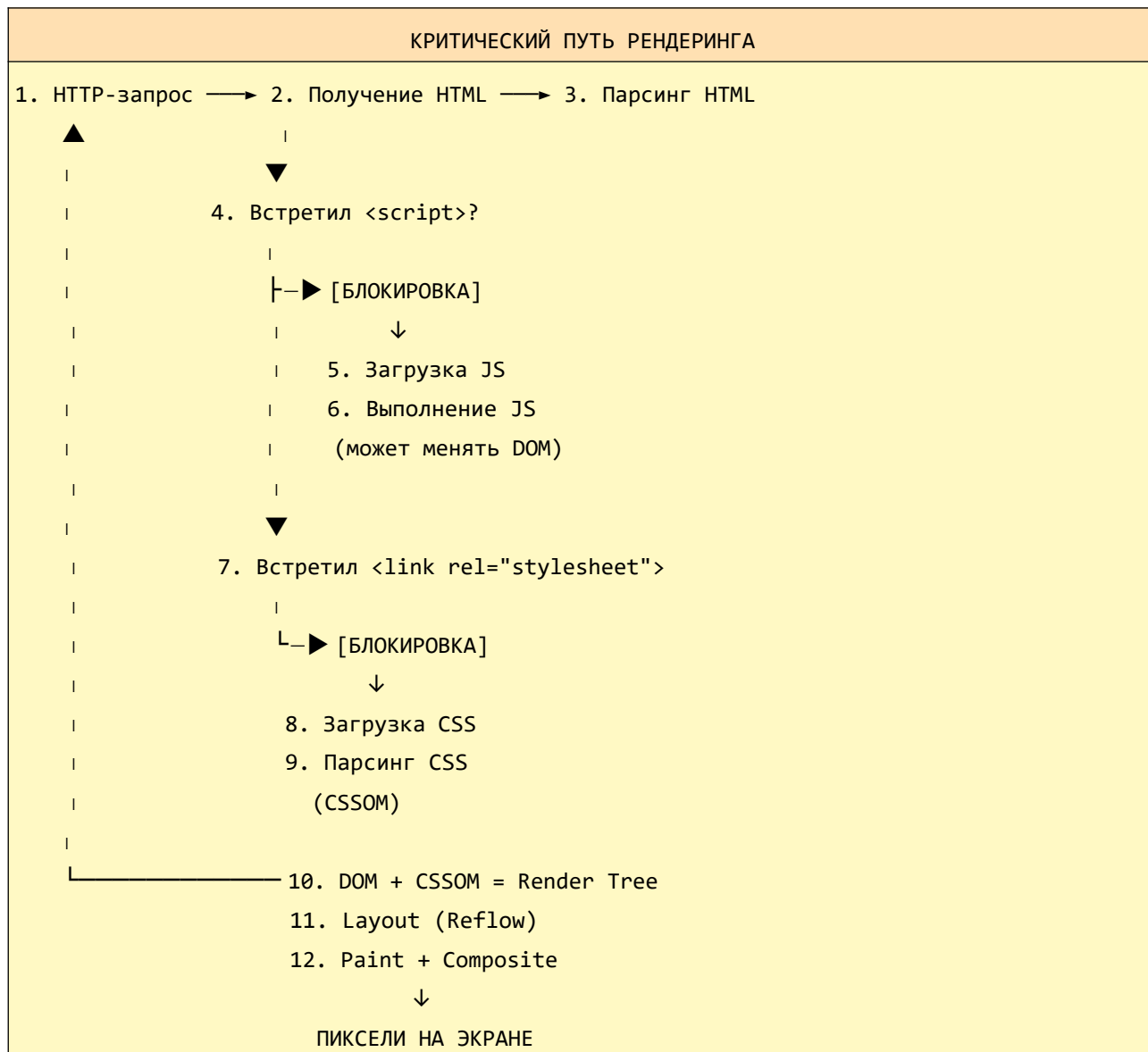
1. **Анатомируем** каждый шаг рендеринга (от запроса до пикселя).
 2. **Научимся видеть** невидимое — почему браузер стоит на месте.
 3. **Освоим 7 техник** ускорения, которые работают в реальных проектах.
 4. **Разберём Production-кейсы** с цифрами и графиками.
-

15.5.1. Анатомия CRP: 12 шагов к пикселю

Многие разработчики думают, что браузер «просто показывает страницу». На самом деле это **конвейер из 12+ этапов**, каждый из которых может стать пробкой .

Полная последовательность (Chromium):

text



Критические инсайты:

1. **HTML — первый и главный ресурс.** Без него ничего не начнётся.
2. **CSS блокирует рендеринг полностью.** Без CSSOM браузер **отказывается рисовать** что-либо, кроме белого экрана. Почему? Потому что если нарисовать контент без стилей, а потом применить CSS, всё дёрнется. Браузер ждёт .

3. **Синхронный JS блокирует парсинг HTML.** Браузер встречает `<script src="...">` → **СТОП**. Скачиваем? Ждём. Выполняем? Ждём. Только потом продолжаем читать HTML. Это **главный убийца производительности**.
4. **Картинки и шрифты НЕ блокируют рендеринг.** Они могут появиться позже, это нормально.

Запомните эту таблицу. Это база:

Ресурс	Блокирует DOM	Блокирует Рендеринг	Можно отложить?
HTML	Н/Д (это источник)	—	—
CSS (внешний)	✗	☐ (до CSSOM)	☐ (для не-критичного)
JS (синхронный)	☐ (до выполнения)	☐ (косвенно)	☐ (async/defer)
JS (async/defer)	✗	✗	☐
Шрифты	✗	⚡✗ (FOUC возможен)	☐
Изображения	✗	✗	☐ (lazy loading)

15.5.2. Визуализация невидимого: Как увидеть CRP в DevTools

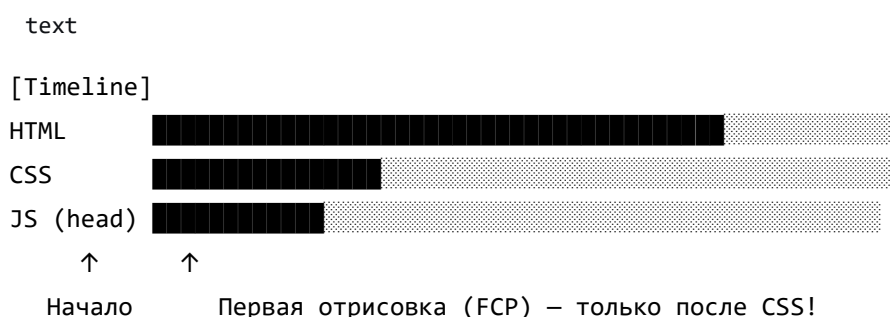
Прежде чем оптимизировать, научитесь **видеть проблему**.

Шаг 1. Запись профиля загрузки

1. Откройте Chrome DevTools → **Performance**.
2. Включите: ☐ **Screenshots**, ☐ **Disable network cache** (имитация первого захода).
3. Нажмите **Record and reload**.

Шаг 2. Чтение «водопада» (Network в Performance)

Смотрите не просто на загрузку ресурсов, а на **цепочки блокировок**.



парсинга

Ключевой паттерн (красный флаг):

Если **First Contentful Paint (FCP)** происходит **строго после** загрузки CSS и синхронного JS — ваша страница платит «налог на блокировку».

Шаг 3. Coverage — сколько кода реально нужно для первой отрисовки?

DevTools → Coverage (Ctrl+Shift+P → «Show Coverage»).

Показывает, какой процент CSS/JS реально использован **на текущей странице**.

Типичная картина для неоптимизированного проекта:

- CSS: использовано **15–25%** кода, остальное — правила для модальных, тултипов, футера.
- JS: использовано **10–20%** кода, остальное — скрипты для взаимодействий, которые не происходят в первую секунду.

Вывод: Вы заставляете пользователя платить за ресурсы, которые ему прямо сейчас не нужны. Это **главная утечка времени в CRP**.

15.5.3. Стратегия 1: Инлайнинг критического CSS (Critical CSS)

Проблема: Внешний CSS-файл — это дополнительный сетевой запрос. Даже если он очень маленький, это RTT (Round Trip Time). На мобильном 3G это **200–500 мс пустого экрана**.

Решение: CSS, необходимый для **отображения первого экрана**, встраивается прямо в `<head>` через `<style>`.

Как это выглядит на практике:

До:

```
html
<head>
  <link rel="stylesheet" href="styles.css"> <!-- 150 КБ, блокирует рендер -->
</head>
```

После:

```

html

<head>
<style>
  /* КРИТИЧЕСКИЙ CSS — только то, что видно в первый момент */
  .header { display: flex; ... }
  .product-card { ... }
  .buy-button { ... }
  /* Никаких стилей для футера, модалок, выпадашек! */
</style>

<!-- Некритический CSS загружается позже и не блокирует -->
<link rel="preload" href="non-critical.css" as="style"
      onload="this.rel='stylesheet'">
<noscript><link rel="stylesheet" href="non-critical.css"></noscript>
</head>

```

Почему это работает:

- Нет сетевого запроса → нет ожидания RTT.
- Браузер строит CSSOM мгновенно из инлайн-стилей.
- Первая отрисовка происходит на **сотни миллисекунд быстрее**.

Цифры: В кейсе Huawei с 电商-страницей инлайнинг критического CSS сократил FCP с **1.8с до 1.1с**.

Как автоматизировать (не вручную же писать):

1. **Critical CLI** (устареваает, но концепция жива).
 2. **Webpack/CRA** — `critters-webpack-plugin`.
 3. **Next.js** — встроенный CSS-in-JS с критическим CSS по умолчанию.
 4. **Chrome Puppeteer** — скриншот выше сгиба → анализ используемых стилей.
-

15.5.4. Стратегия 2: Укрощение JavaScript — `async`, `defer`, и `scheduler.yield`

JavaScript — самый опасный ресурс. Он может:

1. Блокировать парсинг HTML (`<script>` без атрибутов в `<head>`).
2. Блокировать рендеринг (если выполняется до завершения CSSOM).
3. Занимать основной поток на долгие миллисекунды, вызывая TTI (Time to Interactive) > 5с.

15.5.4.1. Раз и навсегда: `async` vs `defer`

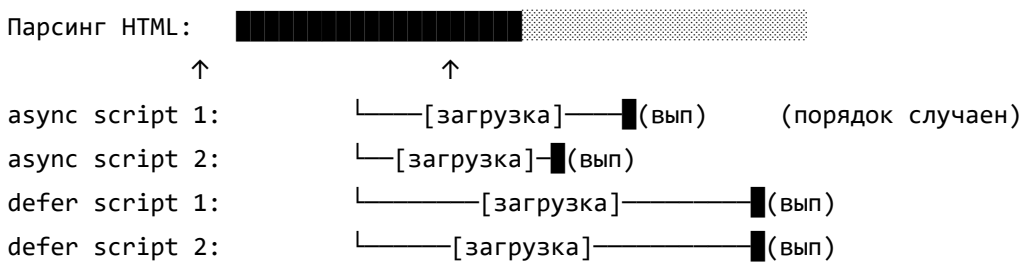
Запомните эту таблицу. Это вопрос на каждом собеседовании и причина 50% тормозов в продакшене.

Атрибут	async	defer
Порядок выполнения	Не гарантируется (кто быстрее загрузился)	Гарантирован (порядок в документе)
Блокировка парсинга HTML	× Нет (загружается параллельно, выполняется когда готов)	× Нет (загружается параллельно, выполняется после всего парсинга)
Время выполнения	Как только загрузился (может быть ДО DOMContentLoaded)	После полного парсинга HTML, перед DOMContentLoaded
Когда использовать	Счётчики, аналитика, независимые виджеты	Библиотеки, код приложения, зависимые скрипты
Для скриптов, изменяющих DOM	 Опасно (непредсказуемо)	 Безопасно

Визуалізація:

text

Парсинг HTML:



Правило большого пальца:

1. Всегда используйте `defer`, если скрипт нужен для работы страницы.
2. Используйте `async` только для абсолютно независимых скриптов (метрика, чат, реклама).
3. **Никогда** не используйте синхронные скрипты в `<head>`.

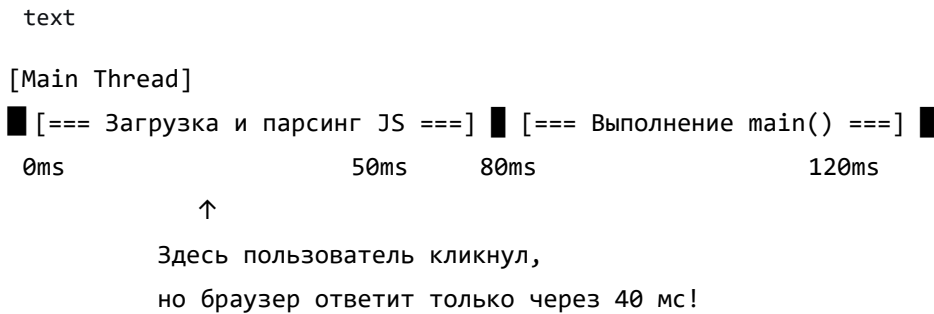
15.5.4.2. Длинные задачи (Long Tasks) и scheduler.yield()

Даже если скрипты загружены правильно, они могут выполняться слишком долго.

Что такое Long Task?

Любая задача на основном потоке, которая длится **более 50 мс**. После 50 мс браузер не может ответить на клик, нажатие клавиши, скролл. Пользователь чувствует «залипание» .

Как выглядит длинная задача в Performance:



Старый способ: `setTimeout(..., 0)`

```
javascript

function processLargeData() {
  for (let item of hugeArray) { // 200 мс! Ужас!
    // ...
  }
}

// «Оптимизация» (спорная)
function processLargeData() {
  let i = 0;
  function chunk() {
    while (i < hugeArray.length && performance.now() - start < 50) {
      // обработать один элемент
      i++;
    }
    if (i < hugeArray.length) {
      setTimeout(chunk, 0); // уступить поток
    }
  }
  chunk();
}
```

Недостатки: Эргономика ужасная, после 5 вложенных `setTimeout` браузер добавляет задержку, приоритет низкий .

Современный способ: `scheduler.yield()` (2024+)

javascript

```
async function processLargeData() {  
  for (const item of hugeArray) {  
    // Обработка элемента  
    processItem(item);  
  
    // Каждые N итераций уступаем поток  
    if (i % 10 === 0) {  
      await scheduler.yield(); // ← ВОТ ОН!  
    }  
  }  
}
```

Что делает scheduler.yield()?

1. Приостанавливает текущую задачу.
2. Возвращает управление браузеру.
3. Браузер может обработать накопившиеся события (клики, рендер).
4. Продолжение функции ставится в **приоритетную** очередь (в отличие от `setTimeout`).

Полифилл для старых браузеров:

javascript

```
function yieldToMain() {  
  if (globalThis.scheduler?.yield) {  
    return scheduler.yield();  
  }  
  return new Promise(resolve => setTimeout(resolve, 0));  
}
```

Когда использовать:

- Обработка больших массивов на клиенте.
- Фильтрация/сортировка таблиц.
- Рендеринг сложных списков.
- Разбор бинарных данных (WASM).

15.5.5. Стратегия 3: Предзагрузка критических ресурсов (Preload, Preconnect)

Стандартные сценарии:

- Браузер узнаёт о CSS, только когда доходит до `<link>` в HTML.
- Браузер узнаёт о шрифте, только когда CSS говорит `@font-face`.

Это поздно!

Решение: Resource Hints .

`<link rel="preload">` — команда: «скачай немедленно, не жди»

```
html

<!-- Шрифт для заголовков – критический ресурс -->
<link rel="preload" href="/fonts/Inter-Bold.woff2" as="font" crossorigin>

<!-- Критический JS (если вы всё же его используете синхронно) -->
<link rel="preload" href="critical.js" as="script">

<!-- Герой-изображение выше сгиба -->
<link rel="preload" href="hero.jpg" as="image">
```

Эффект: Браузер начинает загрузку ресурса **сразу после парсинга этого тега**, не дожидаясь, пока CSS/HTML его запросит.

`<link rel="preconnect">` — команда: «соединись заранее»

```
html

<!-- API, к которому вы обратитесь через 2 секунды -->
<link rel="preconnect" href="https://api.example.com">
```

Что происходит: DNS-запрос + TCP-handshake + TLS-negotiation выполняются ДО того, как fetch/XMLHttpRequest их вызовет.

`<link rel="dns-prefetch">` — устаревшая, но лёгкая версия `preconnect`

Только DNS, без TCP/TLS.

Критический паттерн для шрифтов:

Шрифты — частая причина **Flash Of Invisible Text (FOIT)**. Текст не виден, пока шрифт не загрузится.

```
html

<!-- В head -->
```

```
<link rel="preload" href="/font.woff2" as="font" type="font/woff2" crossorigin>
```

```
<!-- B CSS -->
```

```
@font-face {  
  font-family: 'Inter';  
  src: url('/font.woff2') format('woff2');  
  font-display: swap; /* ← Показывает системный шрифт сразу, потом заменяет */  
}
```

15.5.6. Стратегия 4: Минимизация Layout Shift (CLS)

CRP — это не только скорость первого пикселя, но и **стабильность**.

Cumulative Layout Shift (CLS) — метрика, измеряющая, насколько «прыгает» страница.

Паттерн-убийца:

1. HTML отрендерил контент.
2. Пришёл JS, добавил что-то сверху.
3. Всё поехало вниз. Пользователь уже собирался кликнуть — кликнул не туда.

Как оптимизировать CLS в контексте CRP:

1. Всегда задавайте размеры для медиа

html

```
<!-- ПЛОХО — браузер не знает высоту, отрисует 0px, потом дёрнется -->
```

```

```

```
<!-- ХОРОШО — браузер резервирует место -->
```

```

```

2. Резервируйте место для динамического контента

css

```
.ad-container {  
  min-height: 250px; /* Даже если реклама не загрузилась, место есть */  
  width: 100%;  
}
```

3. Избегайте вставки контента над существующим

Если вам нужно показать баннер, **резервируйте место сразу в HTML**, не вставляйте его через JS после загрузки.

15.5.7. Стратегия 5: Оптимизация рендеринга в Single Page Applications (SPA)

SPA (React, Vue, Angular) имеют **особый проклятый CRP**.

Классический CSR (Client-Side Rendering):

```
html
<body>
  <div id="root"></div>
  <script defer src="react-app.js"></script> <!-- 300 КБ -->
</body>
```

Проблема:

1. Браузер показывает **пустой белый экран**.
2. Загружается React-бандл.
3. React инициализируется.
4. React рендерит компоненты.
5. **Только теперь пользователь видит интерфейс.**

FCP = загрузка JS + инициализация + первый рендер. На медленных устройствах — **3-10 секунд** белого экрана.

Решения (по возрастанию сложности):

А. Серверный рендеринг (SSR) — Next.js, Nuxt, Angular Universal

- Сервер генерирует HTML с контентом.
- Пользователь **сразу видит страницу**.
- React загружается параллельно и «гидрирует» (делает интерфейс живым).

FCP улучшается с 3-5с до 0.8-1.5с.

В. Генерация статических сайтов (SSG) — Next.js (export), Astro, Gatsby

- HTML генерируется на этапе сборки.
- Отдаётся с CDN как статика.
- **Максимальная скорость**, идеально для контентных сайтов.

С. Частичная гидратация (Islands Architecture) — Astro, Qwik, React Server Components

- Не весь React-код отправляется в браузер.
- Только интерактивные «островки» получают JS.

Пример (Astro):

```
astro
---
// Это выполняется на сервере/сборке
const product = await fetchProduct();
---
<div class="product">
  <h1>{product.name}</h1>
  <p>{product.price}</p>
  <!-- Этот компонент статический, JS не нужен -->

  <!-- А этот — интерактивный, JS подгрузится только для него -->
  <AddToCart client:load />
</div>
```

Результат: Минимальный CRP. Браузер рендерит статический HTML мгновенно, JS загружается асинхронно для отдельных компонентов.

15.5.8. Стратегия 6: Code Splitting и Tree Shaking

Даже если вы используете SSR, JS всё равно загружается для гидратации. Его должно быть **мало**.

Code Splitting — не загружай всё сразу

```
javascript

// ПЛОХО — весь приложение в одном бандле
import { Chart } from 'chart.js';
```

// ХОРОШО — только когда пользователь открыл аналитику

```
const Chart = await import('chart.js');
```

В Webpack/Vite: `import()` автоматически создаёт отдельный чанк.

Критический CRP-паттерн:

jsx

// Lazy Load компонента, который ниже сгиба

```
const HeavyFooter = React.lazy(() => import('./Footer'));
```

```
function Page() {  
  return (  
    <>  
      <Header /> { /* Критический */}  
      <Content /> { /* Критический */}  
      <Suspense fallback={<div>Загрузка...</div>}>  
        <HeavyFooter /> { /* Некритический */}  
      </Suspense>  
    </>  
  );  
}
```

Tree Shaking — выкинь мёртвый код

- Используйте ES6-модули (`import/export`), а не CommonJS.
- Настройте `sideEffects: false` в `package.json`.
- Не импортируйте целиком библиотеки.

javascript

// ПЛОХО — 100 КБ мёртвого кода

```
import _ from 'lodash';
```

// ХОРОШО — 5 КБ только нужной функции

```
import debounce from 'lodash/debounce';
```

15.5.9. Стратегия 7: Профилирование и автоматизация

Оптимизация CRP — это не «сделал и забыл». Это ежедневная дисциплина.

Инструменты производственного контроля:

Lighthouse CI

```
bash
```

```
npm install -g @lhci/cli  
lhci collect --url=https://yoursite.com  
lhci assert --preset="perf" --thresholds.lcp=2500
```

Автоматически проваливает PR, если LCP > 2.5с .

Performance Observer API (мониторинг в рантайме)

```
javascript
```

```
// Отслеживаем Long Tasks в реальном времени  
const observer = new PerformanceObserver((list) => {  
  for (const entry of list.getEntries()) {  
    if (entry.duration > 50) {  
      console.warn(`Long Task: ${entry.duration}ms`, entry);  
      // Отправить в Sentry/LogRocket  
    }  
  }  
});  
observer.observe({ entryTypes: ['longtask'] });  
  
// Отслеживаем LCP  
new PerformanceObserver((list) => {  
  const entries = list.getEntries();  
  const lastEntry = entries[entries.length - 1];  
  console.log('LCP:', lastEntry.startTime);  
}).observe({ entryTypes: ['largest-contentful-paint'] });
```

Web Vitals JavaScript-библиотека

```
javascript
```

```
import { onLCP, onFID, onCLS } from 'web-vitals';  
  
onLCP(metric => sendToAnalytics(metric));  
onFID(metric => sendToAnalytics(metric));  
onCLS(metric => sendToAnalytics(metric));
```

15.5.10. Чек-лист: 10 вопросов перед релизом

1. CSS:

- ➔ Критический CSS инлайнирован в `<head>`?
- ➔ Некритический CSS загружается с `preload` + `onload`?
- ➔ Удалён неиспользуемый CSS? (проверка Coverage)

2. JavaScript:

- ➔ Синхронные скрипты в `<head>` отсутствуют?
- ➔ Скрипты имеют `defer` (или `async` для независимых)?
- ➔ Длинные задачи (`>50ms`) разбиты через `scheduler.yield()`?
- ➔ Реализован code splitting для маршрутов/компонентов?

3. Ресурсы:

- ➔ Критические ресурсы (шрифты, герой-изображения) предзагружены (`preload`)?
- ➔ Для сторонних доменов настроен `preconnect`?
- ➔ Изображения выше сгиба имеют явные `width/height`?

4. Архитектура:

- ➔ Для контентного сайта — SSG или SSR?
- ➔ Для SPA — ленивая загрузка модулей?
- ➔ Используются ли современные форматы изображений (WebP/AVIF)?

5. Мониторинг:

- ➔ Настроен Lighthouse CI в CI/CD?
 - ➔ Собираются Web Vitals в production?
-

Резюме главы 15.5

Критический путь рендеринга — это **квинтэссенция производительности фронтенда**.

1. **Браузер — консерватор.** Он не рисует ничего без CSS. Он останавливается перед синхронным JS.
2. **Инлайнинг — самый быстрый способ доставки.** Нет запроса — нет задержки.
3. **Defer — лучший друг.** Почти все скрипты должны быть `defer`.
4. **`scheduler.yield()` — будущее отзывчивости.** 50ms — граница между «быстро» и «тормозит».
5. **Архитектура имеет значение.** CSR — прошлый век. SSG/SSR/Islands — настоящий и будущий.

Главная мысль:

Пользователь пришёл к вам за контентом, а не за бандлом. Чем быстрее вы покажете контент и чем меньше будете мешать ему взаимодействовать, тем выше конверсия, счастье и прибыль.

15.6. Ленивая загрузка и код-сплиттинг

Эпиграф: «Не грузи то, что пользователь никогда не увидит. Не выполняй то, что пользователь никогда не нажмёт. Не заставляй платить за то, что не принесёт пользы прямо сейчас».

Введение: Экономика внимания и байтов

В предыдущем разделе мы оптимизировали **путь к первому пикселю**. Мы инлайнили CSS, предзагружали шрифты, выгоняли синхронный JS из `<head>`.

Но давайте посмотрим правде в глаза: **даже идеальный CRP не спасёт, если ваше приложение весит 10 МБ**.

Статистика, от которой невозможно убежать:

- Медианный размер десктопной страницы в 2024 году — **2.3 МБ** .
- Медианный размер мобильной страницы — **1.8 МБ** .
- 30% этого веса — **JavaScript** .
- 25% — **изображения** .
- 20% — **шрифты и CSS** .

Но главная проблема даже не в весе, а в том, что:

70% кода JavaScript на типовой странице никогда не выполняется.
50% CSS-правил никогда не применяются.
40% изображений находятся ниже сгиба и не видны в первые 10 секунд.

Ленивая загрузка (Lazy Loading) и код-сплиттинг (Code Splitting) — это ответ на вопрос: «Зачем мы заставили пользователя скачать модальку регистрации, если он пришёл просто почитать статью?».

15.6.1. Фундаментальные принципы

Принцип 1: Разделяй и властвуй

Не отправляйте один гигантский бандл. Разрежьте приложение на **чанки (chunks)** — логические куски.

Принцип 2: Откладывай до востребования

Не загружайте чанк, пока он не понадобится. Пользователь нажал «Открыть корзину» → загрузи код корзины.

Принцип 3: Предзагружай с умом

Если вы **уверены**, что пользователь вот-вот нажмёт (например, навёл курсор на кнопку), начните загрузку заранее.

Принцип 4: Кэшируй без потерь

Разделение на чанки позволяет кэшировать редко меняющиеся части (библиотеки, фреймворк) отдельно от часто меняющегося кода приложения.

15.6.2. Три уровня код-сплиттинга

Код-сплиттинг работает на **трёх уровнях абстракции**. В хорошем проекте используются все три.

Уровень 1: Вендорные чанки (Vendor Splitting)

Проблема:

Вы используете React, Lodash, Moment.js. Их код **почти никогда не меняется**. Но каждый раз, когда вы меняете строчку в своём компоненте, пользователь скачивает **весь бандл заново**, включая эти библиотеки.

Решение:

Отделите `node_modules` от вашего кода.

Webpack (конфигурация):

```
javascript

module.exports = {
  optimization: {
    splitChunks: {
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: 'vendors',
          chunks: 'all',
          priority: 10,
          enforce: true
        },
        // Отдельно — огромные библиотеки, которые меняются редко
        react: {
          test: /[\\/]node_modules[\\/](react|react-dom)[\\/]/,
          name: 'react',
          chunks: 'all',
          priority: 20
        }
      }
    }
  }
};
```

Результат:

```
text

main.a1b2c3.js    (ваш код) → 50 КБ, меняется часто
vendors.d4e5f6.js (все зависимости) → 300 КБ, меняется редко
react.7g8h9i.js   (React) → 130 КБ, почти никогда не меняется
```

Эффект: Пользователь скачивает React один раз при первом посещении. При следующих обновлениях вашего кода — только `main.*.js`. Экономия **сотен килобайт** при каждом деплое.

Уровень 2: Сплиттинг по маршрутам (Route-based Splitting)

Проблема:

У вас 50 страниц. Пользователь заходит на `/login`, а вы отправляете ему код страницы `/admin-panel`, `/settings`, `/profile` и ещё 47 страниц, которые он **никогда не откроет**.

Решение:

Загружайте код только для текущего маршрута.

React + React Router:

```
jsx

// ДО – всё сразу
import HomePage from './pages/HomePage';
import AboutPage from './pages/AboutPage';
import AdminPage from './pages/AdminPage';

// ПОСЛЕ – ленивая загрузка через React.Lazy
const HomePage = React.lazy(() => import('./pages/HomePage'));
const AboutPage = React.lazy(() => import('./pages/AboutPage'));
const AdminPage = React.lazy(() => import('./pages/AdminPage'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
        <Route path="/admin" element={<AdminPage />} />
      </Routes>
    </Suspense>
  );
}
```

Vue + Vue Router:

```
javascript

const routes = [
  {
    path: '/',
    component: () => import('./pages/HomePage.vue')
```

```

},
{
  path: '/about',
  component: () => import('./pages/AboutPage.vue')
}
];

```

Эффект: Начальный бандл уменьшается с 500 КБ до 80 КБ. Пользователь платит только за то, что видит.

Уровень 3: Сплиттинг внутри страницы (Component-level Splitting)

Проблема:

Даже на одной странице есть компоненты, которые не нужны для первого рендера. Модальное окно «Войти через Google», график аналитики, форма подписки в футере — они появляются только через 30 секунд или по клику.

Решение:

Ленивая загрузка прямо в JSX.

React:

```

jsx

const LoginModal = React.lazy(() => import('./LoginModal'));
const Chart = React.lazy(() => import('./Chart'));
const Footer = React.lazy(() => import('./Footer'));

function HomePage() {
  const [isLoginOpen, setIsLoginOpen] = useState(false);

  return (
    <>
      <Header />
      <Hero />

      <button onClick={() => setIsLoginOpen(true)}>Войти</button>

      {isLoginOpen && (
        <Suspense fallback=<div>Загрузка формы...</div>>
        <LoginModal onClose={() => setIsLoginOpen(false)} />
        </Suspense>
      )}
    </>
  )
}

```

```

    { /* Footer загружается только когда появился в viewport */ }
    <LazyLoad threshold={0.1}>
      <Suspense fallback={<div>Загрузка...</div>}>
        <Footer />
      </Suspense>
    </LazyLoad>
  </>
);
}

```

Vue 3:

```

vue

<template>
  <button @click="showModal = true">Войти</button>
  <teleport to="body">
    <Suspense>
      <LoginModal v-if="showModal" @close="showModal = false" />
    </Suspense>
  </teleport>
</template>

<script setup>
import { defineAsyncComponent, ref } from 'vue';
const LoginModal = defineAsyncComponent(() => import('./LoginModal.vue'));
const showModal = ref(false);
</script>

```

Эффект:

- 🔵 Пользователь, который никогда не нажимает «Войти», **не скачивает** код модальной (25 КБ).
 - 🔵 Пользователь, который не доскроллил до футера, **не скачивает** код подписки (15 КБ).
-

15.6.3. Инструмент: `import()` — динамический импорт

В основе всего код-сплиттинга лежит **динамический импорт** — функция, возвращающая Promise с модулем.

javascript

// Статический импорт (загружается сразу)

```
import { format } from 'date-fns';
```

// Динамический импорт (загружается при вызове)

```
const { format } = await import('date-fns');
```

Как это работает в браузере:

1. Вы вызываете `import('./path.js')`.
2. Браузер создаёт **новый сетевой запрос** за этим файлом.
3. Парсит, компилирует, выполняет.
4. Возвращает объект модуля.

Это и есть код-сплиттинг на минимальном уровне.

Важно: `import()` — не магия сборщика. Это **нативная функция браузера** (ES2020). Webpack/Rollup/Vite перехватывают её, чтобы создать отдельные файлы, но в рантайме это настоящий динамический импорт.

15.6.4. Ленивая загрузка изображений (Native Lazy Loading)

До 2019 года ленивая загрузка изображений была **танцами с бубном**: вычисление `getBoundingClientRect()`, подписка на `scroll`, throttling, флаги загрузки.

Сейчас есть `loading="lazy"`.

html

<!-- Изображение загрузится, только когда приблизится к viewport -->

```

```

<!-- Критическое изображение — загружается немедленно -->

```

```

<!-- iframe тоже можно грузить лениво -->

```
<iframe src="map.html" loading="lazy"></iframe>
```

Как это работает (технически):

- Браузер не скачивает ресурс при парсинге HTML.
- Он отслеживает позицию элемента относительно окна.

🔵 Когда элемент подходит на расстояние `threshold` (зависит от браузера, обычно 125-300px), начинается загрузка.

Цифры:

На типовой странице с 20 картинками ниже сгиба `loading="lazy"` сокращает **Initial Paint** на **15-25%** и экономит **2-5 МБ трафика** для мобильных пользователей.

Но есть нюанс:

`loading="lazy"` **не работает** для изображений выше сгиба (above the fold). Если вы поставите его на hero-изображение, браузер загрузит его позже, и вы получите **CLS и LCP > 4 секунды**. Только для ниже сгиба!

15.6.5. Предзагрузка с умом (Preloading, Prefetching)

Ленивая загрузка — это отлично. Но что, если мы **уверены**, что пользователь вот-вот перейдёт на следующую страницу?

Инструменты предварительной загрузки:

`<link rel="prefetch">` — загрузи, но не выполняй

Браузер скачивает ресурс с **низким приоритетом** и кладёт в кэш. Используется для **следующей навигации**.

```
html
```

```
<!-- Пользователь на главной, вероятно, пойдёт в каталог -->
```

```
<link rel="prefetch" href="/catalog.bundle.js" as="script">
```

`<link rel="preload">` — загрузи с высоким приоритетом

Мы уже использовали это для CRP. То же самое для чанков.

```
html
```

```
<!-- Мы знаем, что на этой странице точно будет график -->
```

```
<link rel="preload" href="/chart.bundle.js" as="script">
```


`<link rel="modulepreload">` — специально для ES-модулей

Как preload, но для модулей. Браузер не только скачивает, но и парсит/компилирует модуль.

html

```
<link rel="modulepreload" href="/heavy-component.js">
```

Прогрессивная предзагрузка (Intersection Observer + Prefetch)

javascript

// Когда пользователь навёл на ссылку — начинаем грузить

```
document.querySelectorAll('nav a').forEach(link => {
  link.addEventListener('mouseenter', () => {
    const href = link.getAttribute('href');
    if (href && href.startsWith('/products')) {
      const prefetchLink = document.createElement('link');
      prefetchLink.rel = 'prefetch';
      prefetchLink.href = href + '.bundle.js';
      prefetchLink.as = 'script';
      document.head.appendChild(prefetchLink);
    }
  }, { once: true }); // Только один раз
});
```

Эффект: Когда пользователь **действительно кликнет**, ресурс уже будет в кэше. Переход — мгновенный.

15.6.6. Реальный кейс: Модальное окно авторизации

Контекст:

- 🌐 Интернет-магазин.
- 🌐 Кнопка «Войти» в правом верхнем углу.
- 🌐 Модалка авторизации весит 45 КБ (JS + JSX + стили).

Проблема:

На главную страницу заходят 100 000 пользователей в день. Только 10% из них нажимают «Войти». Остальные 90% **бесплатно** скачивают 45 КБ кода, который им не нужен.

Решение (пошагово):

Шаг 1. Динамический импорт компонента

```
jsx

const AuthModal = React.lazy(() => import('./AuthModal'));

function Header() {
  const [isAuthOpen, setIsAuthOpen] = useState(false);

  return (
    <header>
      <button onClick={() => setIsAuthOpen(true)}>Войти</button>
      {isAuthOpen && (
        <Suspense fallback=<div>Загрузка...</div>>
        <AuthModal onClose={() => setIsAuthOpen(false)} />
      )}
    </header>
  );
}
```

Эффект: 90% пользователей не качают 45 КБ. Экономия: **45 КБ × 90 000 = 4 ГБ трафика в день.**

Шаг 2. Предзагрузка по наведению

```
jsx

const prefetchAuthModal = () => {
  const link = document.createElement('link');
  link.rel = 'modulepreload'; // или preload
  link.href = '/auth-modal.bundle.js';
  document.head.appendChild(link);
};

function Header() {
  return (
    <button
      onClick={() => setIsAuthOpen(true)}
      onMouseEnter={prefetchAuthModal}
    >
      Войти
    </button>
  );
}
```

```
);  
}
```

Эффект: 10% пользователей, которые **действительно хотят войти**, видят модальку **мгновенно** (ресурс уже в кэше). UX улучшается, конверсия в регистрацию растёт.

Шаг 3. Кэширование в Service Worker

```
javascript  
  
// sw.js  
self.addEventListener('fetch', event => {  
  if (event.request.url.includes('auth-modal')) {  
    event.respondWith(  
      caches.match(event.request).then(response => {  
        return response || fetch(event.request).then(response => {  
          const responseClone = response.clone();  
          caches.open('modal-v1').then(cache => {  
            cache.put(event.request, responseClone);  
          });  
          return response;  
        });  
      });  
    );  
  }  
});
```

Эффект: После первого открытия модалька загружается **из кэша** (0-10 мс).

15.6.7. Продвинутая техника: Виртуальные списки (Windowing)

Проблема:

У вас таблица на 10 000 строк. React создаёт 10 000 DOM-элементов. Браузер виснет на 2 секунды. Скролл тормозит. Память — 200+ МБ.

Решение:

Рендерить только те строки, которые видны на экране (плюс небольшой буфер). Это называется **виртуализация (windowing)**.

react-window (библиотека от автора React):

```

jsx

import { FixedSizeList as List } from 'react-window';

const Row = ({ index, style }) => (
  <div style={style}>
    Строка #{index + 1}
  </div>
);

const BigList = () => (
  <List
    height={600}           // высота контейнера
    itemCount={10000}      // всего строк
    itemSize={35}          // высота строки
    width={'100%'}
  >
    {Row}
  </List>
);

```

Что происходит под капотом:

- Создаётся **ровно столько DOM-элементов, сколько помещается на экране** (обычно 10-20 штук).
- При скролле **переиспользуются** существующие элементы — просто меняется их позиция и содержимое.
- Память: **2 МБ** вместо 200 МБ.
- Скролл: **60 fps** вместо 5 fps.

Аналоги для других фреймворков:

- Vue: `vue-virtual-scroller`
 - Angular: `cdk-virtual-scroll-viewport`
 - Svelte: `svelte-virtual-list`
-

15.6.8. Ленивая загрузка CSS (Critical CSS + Non-critical CSS)

Проблема:

У вас 150 КБ CSS. 80% из них — стили для модальных, футера, тултипов, которых нет на первом экране. Но браузер **ждёт весь CSS**, прежде чем нарисовать **хоть что-то**.

Решение 1: Инлайним критический CSS, остальное — грузим лениво.

```
html

<head>
  <!-- Критический CSS прямо здесь -->
  <style>
    .header { ... }
    .hero { ... }
    .button-primary { ... }
    /* Всё, что нужно для первого экрана */
  </style>

  <!-- Некритический CSS грузится без блокировки -->
  <link rel="preload" href="non-critical.css" as="style"
        onload="this.rel='stylesheet'">
  <noscript>
    <link rel="stylesheet" href="non-critical.css">
  </noscript>
</head>
```

Решение 2: Динамическая загрузка по требованию

```
javascript

// Грузим CSS только когда открывается модалка
async function loadModalStyles() {
  const link = document.createElement('link');
  link.rel = 'stylesheet';
  link.href = 'modal.css';
  document.head.appendChild(link);
  // Можно дождаться загрузки
  return new Promise(resolve => {
    link.onload = resolve;
  });
}

// В компоненте модалки
const Modal = () => {
  useEffect(() => {
    loadModalStyles();
  }, []);

  return <div className="modal">...</div>;
};
```

Эффект:

- **FCP:** с 1.8с до 1.0с (инлайновые стили).
 - **TBT (Total Blocking Time):** уменьшается, так как браузер не парсит 100 КБ мёртвого CSS в критический момент.
-

15.6.9. Ленивая загрузка шрифтов (Font Loading Optimization)

Проблема:

Браузер не знает о шрифтах, пока не загрузит и не распарсит CSS с `@font-face`. Затем он скачивает шрифт. Всё это время **текст не виден** (FOIT — Flash of Invisible Text).

Решение 1: Preload + font-display: swap

```
html

<!-- Начинаем загружать шрифт максимально рано -->
<link rel="preload" href="/fonts/Inter-Bold.woff2" as="font" type="font/woff2"
crossorigin>

<!-- В CSS -->
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter-Bold.woff2') format('woff2');
  font-display: swap; /* ← Волшебная строка */
}
```

font-display: swap: Браузер показывает текст системным шрифтом сразу, а когда загрузится кастомный шрифт — **заменяет**. Пользователь видит контент через 10 мс, а не через 500 мс.

Решение 2: Загрузка только используемых символов (Font Subsetting)

```
javascript

// Загружаем полный шрифт (150 КБ) — плохо
import '@fontsource/inter';

// Загружаем только кириллицу и латиницу (30 КБ) — хорошо
import '@fontsource/inter/cyrillic.css';
import '@fontsource/inter/latin.css';
```

Решение 3: Динамическая загрузка вариативных шрифтов

```
javascript

// Грузим "жирность" 400 и 700 только когда нужен bold
const loadBoldFont = () => {
  const link = document.createElement('link');
  link.href = '/fonts/inter-variable.woff2?weight=700';
  link.rel = 'stylesheet';
  document.head.appendChild(link);
};
```

15.6.10. Анализ бандла: Кто съел мою память?

Прежде чем резать код, нужно понять, **что именно резать**.

Инструмент 1: Webpack Bundle Analyzer

```
bash

npm install --save-dev webpack-bundle-analyzer
javascript

// next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
});
module.exports = withBundleAnalyzer({});

// Запуск:
ANALYZE=true npm run build
```

Результат: Интерактивная диаграмма, где каждый модуль — прямоугольник. Чем больше площадь — тем больше места в бандле.

Инструмент 2: Vite Bundle Visualizer

```
bash

npm install --save-dev rollup-plugin-visualizer
```

Инструмент 3: Chrome DevTools → Coverage

Загрузите страницу, откройте Coverage (Ctrl+Shift+P → «Show Coverage»). Запишите использование кода.

Красный цвет: код загружен, но не использован.

Зелёный цвет: код использован.

Цель: Минимизировать красный в критическом пути.

15.6.11. Антипаттерны: Чего НЕ надо делать

1. Ленивая загрузка всего подряд

jsx

// ПЛОХО: Header — критический, но ленивый

```
const Header = React.lazy(() => import('./Header'));
```

Почему: Браузер видит Suspense, рисует fallback, потом ждёт Header, потом рендерит. FCP — катастрофа. **Критические компоненты грузим синхронно.**

2. Слишком маленькие чанки

javascript

// ПЛОХО: 50 чанков по 1 КБ

```
import('./utils/get.js');
import('./utils/set.js');
import('./utils/delete.js');
```

Почему: Каждый чанк — это отдельный HTTP-запрос. На HTTP/1 это убийственно. На HTTP/2 лучше, но всё равно накладные расходы. **Золотая середина: 20-50 КБ на чанк.**

3. Ленивая загрузка выше сгиба

html

<!-- ПЛОХО: Герой-изображение с loading="lazy" -->

```

```

Почему: Браузер загрузит его слишком поздно. LCP будет ужасным.

4. Предзагрузка всего подряд

html


```
<!-- ПЛОХО: 50 preload-тегов в head -->
<link rel="preload" href="chunk1.js" as="script">
<link rel="preload" href="chunk2.js" as="script">
<!-- ... ещё 48 ... -->
```

Почему: Браузер начнёт качать 50 файлов одновременно. Конкуренция за полосу пропускания. Критические ресурсы будут ждать. **Preload только для 2-3 самых важных ресурсов.**

15.6.12. Чек-лист: Ленивая загрузка и код-сплиттинг

Перед релизом проверьте:

- **Vendor splitting:** `node_modules` вынесены в отдельные чанки?
 - **Route splitting:** Каждый маршрут — отдельный чанк (для SPA)?
 - **Component splitting:** Модалки, попапы, сложные виджеты ниже сгиба — динамический импорт?
 - **Images:** Все изображения ниже сгиба имеют `loading="lazy"`?
 - **Images:** Все изображения имеют `width/height` (резервирование места)?
 - **Fonts:** Настроен `font-display: swap`?
 - **Fonts:** Критические шрифты предзагружены (`preload`)?
 - **CSS:** Критический CSS инлайнирован, некритический — загружается лениво?
 - **Prefetch:** Настроена предзагрузка для вероятных переходов?
 - **Analysis:** Вы запускали Bundle Analyzer и знаете топ-5 «тяжёлых» модулей?
-

Резюме главы 15.6

1. **Код-сплиттинг — это не опция, а необходимость.** Любое приложение сложнее «Hello, World» должно резать бандл.
2. **Три уровня:** вендоры, маршруты, компоненты. Чем мельче зерно, тем точнее оптимизация, но выше сложность.
3. `import()` — **швейцарский нож.** Научитесь им пользоваться. Это основа всего.
4. **Ленивая загрузка — не только для JS.** Изображения, CSS, шрифты, `iframe` — всё это можно и нужно грузить отложено.
5. **Предзагрузка — это инь и янь ленивой загрузки.** Лениво — для того, что может не понадобиться. Предзагрузка — для того, что точно понадобится, но чуть позже.

6. **Измеряйте.** Без Bundle Analyzer вы слепы. Без Coverage вы не знаете, какой код реально работает.

Главная мысль:

Пользователь — это не бездонный контейнер для ваших байтов. У него есть лимит по трафику, по времени, по терпению. Уважайте его. Отдавайте только то, что нужно прямо сейчас. Остальное — по запросу.

КОНЕЦ 1-ОЙ ЧАСТИ УЧЕБНИКА
ОКОНЧАНИЕ — ВО 2-ОЙ ЧАСТИ УЧЕБНИКА